Domain Theory in Lean 4

Lars Warren Ericson[†], **Dana S. Scott**, **Vijay D'Silva**[‡], **Brian Milnes**[§]

October 2026
CMU-CS-26-140

School of Computer Science
Carnegie Mellon University
Pittsburgh, PA 15213

[†] Catskills Research Company, [‡] Google Research, [§] Unaffiliated

Keywords: Lean 4; formal verification; domain theory; continuous lattices; neighbourhood systems; information systems; Scott topology

Abstract

This document is a machine-checked Lean 4 formalization of Dana Scott's papers on domain theory: continuous lattices (1972), datatypes as lattices (1976), neighbourhood systems (1980), and information systems (1982), together with bridge theorems relating the lattice, neighbourhood, and information systems presentations.

Contents

1 Continuous Lattices (1972) 14

1.1 Introduction . . . 15
1.2 Injective spaces . . . 16
1.2.1 Definition 1.1: Injective Space . . . 17
1.2.2 Proposition 1.2 . . . 19
1.2.3 Proposition 1.3 . . . 19
1.2.4 Powers of $\mathbb{O}$ are injective . . . 20
1.2.5 Proposition 1.4 . . . 20
1.2.6 Proposition 1.5 . . . 21
1.2.7 RetractSubspace . . . 22
1.2.8 Corollary 1.6 . . . 22
1.2.9 Corollary 1.7 . . . 23
1.2.10 Absolute retracts . . . 24
1.3 Continuous lattices . . . 24
1.3.1 Specialization order . . . 26
1.3.2 Way-below relation . . . 33
1.3.3 Proposition 2.2 . . . 37
1.3.4 Definition 2.3 . . . 39
1.3.5 Proposition 2.4 . . . 40
1.3.6 Limits and convergence . . . 41
1.3.7 Scott maps . . . 42
1.3.8 Proposition 2.8 . . . 51
1.3.9 Proposition 2.9 . . . 53
1.3.10 Proposition 2.10 . . . 53
1.3.11 Proposition 2.11 . . . 53
1.3.12 Theorem 2.12 . . . 53
1.3.13 Proof of 2.9 . . . 53
1.3.14 Proof of 2.10 . . . 56
1.3.15 Proof of 2.11 . . . 60
1.3.16 Proof of 2.12 . . . 71
1.3.17 After Theorem 2.12 . . . 86
1.4 Function spaces . . . 87
1.4.1 Definition 3.1: Function space . . . 87
1.4.2 Proposition 3.2 . . . 96
1.4.3 Theorem 3.3 . . . 97
1.4.4 Corollary 3.4 . . . 152
1.4.5 Proposition 3.5 . . . 155
1.4.6 Retractions and extensions . . . 162
1.4.7 Definition 3.6 . . . 163
1.4.8 Proposition 3.7 . . . 165
1.4.9 Proposition 3.8 . . . 182
1.4.10 Lemma 3.9 . . . 192
1.4.11 Proposition 3.10 . . . 196

1.4.12 Definition 3.11: Projection space . . . 201
1.4.13 Proposition 3.12 . . . 201
1.4.14 Proposition 3.13 . . . 203
1.4.15 Proposition 3.14 . . . 206
1.5 Inverse limits . . . 213
1.5.1 The inverse system . . . 215
1.5.2 projLE . . . 216
1.5.3 projection . . . 218
1.5.4 Projections from the limit . . . 219
1.5.5 Embeddings along the tower . . . 220
1.5.6 Coordinates of an embedding . . . 221
1.5.7 Embeddings into the limit . . . 223
1.5.8 The mediating map . . . 226
1.5.9 inverseLimitLE . . . 228
1.5.10 inverseLimitLT . . . 229
1.5.11 The limit lattice . . . 230
1.5.12 inverseLimitTopology . . . 234
1.5.13 Proposition 4.1 . . . 238
1.5.14 Proposition 4.2 . . . 239
1.5.15 Corollary 4.3 . . . 240
1.5.16 The function space tower . . . 242
1.5.17 The functorial form . . . 272
1.5.18 Application . . . 273
1.5.19 Two projections . . . 273
1.5.20 Embedding every space . . . 273
1.5.21 Other functors . . . 274
1.5.22 Subspaces of a limit . . . 274
1.6 Correction . . . 275

2 Datatypes as Lattices (1976) 277
2.1 Introduction . . . 277
2.1.1 Historical note . . . 279
2.2 Continuous functions . . . 279
2.2.1 Introduction . . . 280
2.2.2 e . . . 282
2.2.3 pair . . . 284
2.2.4 Continuous . . . 287
2.2.5 Graph . . . 290
2.2.6 FixedPoint . . . 292
2.2.7 Embedding . . . 297
2.3 Computability and definability . . . 306
2.3.1 Historical note . . . 306
2.3.2 Meaning . . . 306
2.3.3 LAMBDA . . . 310
2.3.4 Theorem 2.1 . . . 321
2.3.5 Theorem 2.2 . . . 322

2.3.6 Theorem 2.3 . . . 325
2.3.7 Theorem 2.4 . . . 326
2.3.8 The combinator Y . . . 329
2.3.9 Theorem 2.5 . . . 332
2.3.10 The graph model . . . 338
2.3.11 Elementary definitions . . . 340
2.3.12 Sequences . . . 343
2.3.13 Distributive functions . . . 352
2.3.14 Primitive recursion . . . 359
2.3.15 Definition (computable) . . . 362
2.3.16 Computable functions . . . 362
2.3.17 Definability . . . 363
2.3.18 A law of Y . . . 375
2.4 Retracts and data types . . . 379
2.5 Closure operations and algebraic lattices . . . 387
2.6 Subsets and their classification . . . 395
2.7 Total functions and functionality . . . 634
2.8 Enumeration and degrees . . . 809

3 Neighborhood Systems (1980) 951
3.1 Introduction . . . 952
3.2 Domains given by neighbourhoods . . . 953
3.2.1 Introduction . . . 954
3.2.2 Definition 1.1 — neighbourhood system . . . 955
3.2.3 Factoid 1.1 — empty intersection is delta; recursive finite intersection . . . 956
3.2.4 Example 1.2 — two-token domain . . . 958
3.2.5 Example 1.3 — three-step chain . . . 959
3.2.6 Example 1.4 — finite binary tree . . . 960
3.2.7 Factoid 1.4a — nested or disjoint . . . 962
3.2.8 Example 1.5 — all nonempty subsets . . . 963
3.2.9 Factoid 1.5 — consistent iff nonempty; equivalent sequences same limit . . . 964
3.2.10 Definition 1.6 — ideal elements . . . 966
3.2.11 Definition 1.7 — principal filter . . . 967
3.2.12 Factoid 1.7 — principals are one-one; inclusion-reversing principals; union of principal filters . . . 968
3.2.13 Definition 1.8 — approximation and totality . . . 969
3.2.14 Factoid 1.8 — bottom element; bottom is least; finite filters are principal; prefix order; union of finite approximations . . . 970
3.2.15 Examples 1.2–1.5 (Revisited) — finite examples revisited . . . 972
3.2.16 Definition 1.9 — isomorphic domains . . . 973
3.2.17 Theorem 1.10 — basic open neighbourhoods . . . 974
3.2.18 Theorem 1.11 — intersections and directed unions . . . 976
3.2.19 Exercise 1.12 — final segments of integers . . . 978
3.2.20 Exercise 1.13 — infinite binary tree . . . 979
3.2.21 Exercise 1.14 — finite nonempty subsets . . . 980
3.2.22 Exercise 1.15 — finite elements without chains . . . 980

3.2.23 Exercise 1.16 — cofinite integer subsets . . . 981
3.2.24 Exercise 1.17 — rational open intervals . . . 982
3.2.25 Exercise 1.18 — consistency of neighbourhood families . . . 983
3.2.26 Exercise 1.19 — positive neighbourhood systems . . . 984
3.2.27 Exercise 1.20 — upward closed neighbourhoods . . . 985
3.2.28 Exercise 1.21 — complete neighbourhood systems . . . 986
3.2.29 Exercise 1.22 — topology of a domain . . . 987
3.2.30 Exercise 1.23 — decidable total elements . . . 988
3.2.31 Exercise 1.24 — extending partial elements . . . 989
3.2.32 Exercise 1.25 — final segments of ordinals . . . 989
3.2.33 Exercise 1.26 — ideals of a ring . . . 990
3.2.34 Exercise 1.27 — suprema of bounded sets . . . 991
3.3 Mappings between domains . . . 992
3.3.1 Introduction . . . 992
3.3.2 Definition 2.1 — approximable mapping . . . 993
3.3.3 Proposition 2.2 — elementwise action . . . 994
3.3.4 Example 2.3 — parity of leading zeros . . . 995
3.3.5 Example 2.4 — erase a run of ones . . . 997
3.3.6 Theorem 2.5 — category of approximable maps . . . 998
3.3.7 Proposition 2.6 — identity and composition . . . 999
3.3.8 Theorem 2.7 — isomorphisms are approximable . . . 1001
3.3.9 Lecture II exercises . . . 1002
3.3.10 Exercise 2.8 — determined by finite elements . . . 1002
3.3.11 Exercise 2.9 — directed union formula . . . 1003
3.3.12 Exercise 2.10 — intersection of two maps . . . 1004
3.3.13 Exercise 2.11 — directed unions of elements . . . 1005
3.3.14 Exercise 2.12 — directed unions of maps . . . 1006
3.3.15 Exercise 2.13 — approximable means continuous . . . 1007
3.3.16 Exercise 2.14 — isomorphisms of neighbourhoods . . . 1008
3.3.17 Exercise 2.15 — opens classified by maps . . . 1010
3.3.18 Exercise 2.16 — prefixing is approximable . . . 1011
3.3.19 Exercise 2.17 — equations for erase-ones . . . 1011
3.3.20 Exercise 2.17 — uniqueness of the erase-ones map . . . 1012
3.3.21 Exercise 2.18 — doubling binary digits . . . 1013
3.3.22 Exercise 2.19 — maps of two arguments . . . 1014
3.3.23 Exercise 2.20 — powerset of the integers . . . 1015
3.3.24 Exercise 2.21 — finite and infinite sequences . . . 1016
3.3.25 Exercise 2.22 — domains from closure properties . . . 1017
3.4 Domain constructs . . . 1019
3.4.1 Introduction . . . 1019
3.4.2 Definition 3.1 — product of systems . . . 1019
3.4.3 Proposition 3.2 — elements of a product . . . 1020
3.4.4 Definition 3.3 — projection mappings . . . 1022
3.4.5 Proposition 3.4 — pairing and projections . . . 1023
3.4.6 Theorem 3.5 — separate approximability . . . 1025

3.4.7 Lemma 3.6 — constant functions . . . 1026
3.4.8 Proposition 3.7 — closure under substitution . . . 1027
3.4.9 Definition 3.8 — function space . . . 1029
3.4.10 Proposition 3.9 — consistent step neighbourhoods . . . 1030
3.4.11 Theorem 3.10 — elements are mappings . . . 1032
3.4.12 Theorem 3.11 — evaluation combinator . . . 1033
3.4.13 Theorem 3.12 — currying . . . 1035
3.4.14 Theorem 3.13 — pointwise order of maps . . . 1037
3.4.15 Exercise 3.14 — tagged product of tokens . . . 1038
3.4.16 Exercise 3.15 — product isomorphisms . . . 1039
3.4.17 Exercise 3.16 — infinite sequences of elements . . . 1040
3.4.18 Exercise 3.17 — embedding binary sequences . . . 1041
3.4.19 Exercise 3.18 — separated sum . . . 1042
3.4.20 Exercise 3.19 — induced maps on constructs . . . 1043
3.4.21 Exercise 3.20 — sum and product functors . . . 1045
3.4.22 Exercise 3.21 — identity of step neighbourhoods . . . 1046
3.4.23 Exercise 3.22 — composition combinator . . . 1047
3.4.24 Exercise 3.23 — cartesian closed category . . . 1048
3.4.25 Exercise 3.24 — function-space isomorphisms . . . 1049
3.4.26 Exercise 3.25 — opens form a domain . . . 1050
3.4.27 Exercise 3.26 — conditional combinator . . . 1051
3.4.28 Exercise 3.27 — function space via closure . . . 1052
3.4.29 Exercise 3.28 — minimal step function . . . 1053
3.5 Fixed points and recursion . . . 1054
3.5.1 Introduction . . . 1054
3.5.2 Theorem 4.1 — least fixed point . . . 1055
3.5.3 Theorem 4.2 — fix combinator . . . 1056
3.5.4 Example 4.3 — partial functions on integers . . . 1058
3.5.5 Example 4.4 — sequence domain . . . 1062
3.5.6 Definition 4.5 — Peano axioms . . . 1065
3.5.7 Theorem 4.6 — Peano models isomorphic . . . 1066
3.5.8 Lecture IV exercises . . . 1068
3.5.9 Exercise 4.7 — fixed point above a seed . . . 1068
3.5.10 Exercise 4.8 — fixed-point induction . . . 1069
3.5.11 Exercise 4.9 — fix as a fixed point . . . 1070
3.5.12 Exercise 4.10 — domain above an element . . . 1071
3.5.13 Exercise 4.11 — characterizing the fix combinator . . . 1072
3.5.14 Exercise 4.12 — no greatest fixed point . . . 1073
3.5.15 Exercise 4.13 — avoiding the natural numbers . . . 1073
3.5.16 Exercise 4.14 — greatest powerset fixed point . . . 1075
3.5.17 Exercise 4.15 — maximal fixed points . . . 1075
3.5.18 Exercise 4.16 — optimal fixed point . . . 1076
3.5.19 Exercise 4.17 — generated submonoid . . . 1077
3.5.20 Exercise 4.18 — checking the number domain . . . 1078
3.5.21 Exercise 4.19 — Peano axioms for sequences . . . 1079

3.5.22 Exercise 4.20 — fix of a composite . . . 1080
3.5.23 Exercise 4.21 — order as a fixed point . . . 1081
3.5.24 Exercise 4.22 — inductive subset of a model . . . 1082
3.5.25 Exercise 4.23 — commuting with the iterates . . . 1083
3.5.26 Exercise 4.24 — Schroeder-Bernstein theorem . . . 1083
3.5.27 Exercise 4.25 — unary sequence domain . . . 1084
3.6 Typed λ-calculus . . . 1085
3.6.1 Introduction . . . 1085
3.6.2 Theorem 5.1 — terms denote approximable maps . . . 1089
3.6.3 Theorem 5.2 — beta conversion . . . 1091
3.6.4 Proposition 5.3 — simultaneous fixed points . . . 1092
3.6.5 Proposition 5.4 — fixed point under a parameter . . . 1094
3.6.6 Table 5.5 — table of combinators . . . 1095
3.6.7 Theorem 5.6 — partial recursive functions . . . 1096
3.6.8 Exercise 5.7 — currying the lambda notation . . . 1099
3.6.9 Exercise 5.8 — combinatory completeness . . . 1100
3.6.10 Exercise 5.9 — common fixed points . . . 1100
3.6.11 Exercise 5.10 — smash product . . . 1101
3.6.12 Exercise 5.11 — stack combinators . . . 1102
3.6.13 Exercise 5.12 — while combinator . . . 1103
3.6.14 Exercise 5.13 — pairing function on numbers . . . 1104
3.6.15 Exercise 5.13 — powerset isomorphisms . . . 1105
3.6.16 Exercise 5.14 — functions as graphs . . . 1105
3.6.17 Exercise 5.15 — star of a word . . . 1106
3.6.18 Exercise 5.16 — negation of sequences . . . 1107
3.7 Domain equations . . . 1108
3.7.1 Introduction . . . 1109
3.7.2 Example 6.1 — indefinite product . . . 1110
3.7.3 Example 6.2 — sequence domain equations . . . 1115
3.7.4 Definition 6.3 — functor . . . 1118
3.7.5 Definition 6.4 — algebra for a functor . . . 1120
3.7.6 Definition 6.5 — initial algebra . . . 1121
3.7.7 Proposition 6.6 — uniqueness of initial algebras . . . 1122
3.7.8 Proposition 6.7 — functor preserves initiality . . . 1123
3.7.9 Definition 6.8 — continuity on maps . . . 1124
3.7.10 Theorem 6.9 — homomorphism from initial algebra . . . 1125
3.7.11 Definition 6.10 — subdomain . . . 1127
3.7.12 Proposition 6.11 — lattice of subsystems . . . 1128
3.7.13 Proposition 6.12 — projection pair . . . 1129
3.7.14 Definition 6.13 — continuity on domains . . . 1130
3.7.15 Theorem 6.14 — existence of initial algebras . . . 1132
3.7.16 Lemma 6.15 — subdomains from projection pairs . . . 1135
3.7.17 Theorem 6.16 — initial algebra is minimal . . . 1137
3.7.18 Lecture VI exercises . . . 1139
3.7.19 Exercise 6.17 — algebras for sequence domains . . . 1139

3.7.20 Exercise 6.18 — infinite product as initial . . . 1140
3.7.21 Exercise 6.19 — strict sum and product . . . 1141
3.7.22 Exercise 6.20 — tokens of a functor . . . 1143
3.7.23 Exercise 6.21 — coalesced sum and smash . . . 1144
3.7.24 Exercise 6.22 — equations for the integers . . . 1145
3.7.25 Exercise 6.23 — syntactic expression domain . . . 1146
3.7.26 Exercise 6.24 — recursive domain equations . . . 1147
3.7.27 Exercise 6.25 — projection pair laws . . . 1148
3.7.28 Exercise 6.26 — lifting a domain . . . 1149
3.7.29 Exercise 6.27 — which embeddings hold . . . 1150
3.7.30 Exercise 6.28 — finite embeddings cancel . . . 1151
3.7.31 Exercise 6.29 — infinitary sum and product . . . 1152
3.8 Computability in effectively given domains . . . 1153
3.8.1 Introduction . . . 1154
3.8.2 Definition 7.1 — computable presentation . . . 1154
3.8.3 Definition 7.2 — computable mapping . . . 1156
3.8.4 Proposition 7.3 — computable maps form a category . . . 1157
3.8.5 Theorem 7.4 — sum and product effective . . . 1159
3.8.6 Theorem 7.5 — function space effective . . . 1161
3.8.7 Theorem 7.6 — fix is computable . . . 1163
3.8.8 Proposition 7.7 — lists effectively given . . . 1165
3.8.9 Example 7.8 — powerset effectively given . . . 1167
3.8.10 Definition 7.9 — power domain . . . 1168
3.8.11 Proposition 7.10 — power domain effective . . . 1170
3.8.12 Definition 7.11 — finite sets of elements . . . 1171
3.8.13 Proposition 7.12 — finite sets are approximable . . . 1173
3.8.14 Lecture VII exercises . . . 1174
3.8.15 Exercise 7.13 — axiomatic neighbourhood structure . . . 1174
3.8.16 Exercise 7.14 — primitive recursive witnesses . . . 1176
3.8.17 Exercise 7.15 — further constructs effective . . . 1177
3.8.18 Exercise 7.16 — curry as an enumeration . . . 1178
3.8.19 Exercise 7.17 — recursion over lists . . . 1178
3.8.20 Exercise 7.18 — effective isomorphism . . . 1180
3.8.21 Exercise 7.19 — power domain functor . . . 1181
3.8.22 Exercise 7.20 — union on power domains . . . 1182
3.8.23 Exercise 7.21 — distributing the power domain . . . 1183
3.8.24 Exercise 7.22 — domain of formal languages . . . 1184
3.8.25 Exercise 7.23 — graph and fun computable . . . 1185
3.8.26 Exercise 7.24 — LUCID history domain . . . 1186
3.9 Retracts of the universal domain . . . 1187
3.9.1 Introduction . . . 1188
3.9.2 Definition 8.1 — retraction . . . 1189
3.9.3 Proposition 8.2 — retractions from subdomains . . . 1189
3.9.4 Definition 8.3 — projections . . . 1191
3.9.5 Examples 8.4 — check, fade, and smash . . . 1192

3.9.6 Theorem 8.5 — finitary projection test . . . 1196
3.9.7 Theorem 8.6 — sub combinator . . . 1197
3.9.8 Definition 8.7 — interval presentation of U . . . 1199
3.9.9 Theorem 8.8 — universality of U . . . 1203
3.9.10 Definition 8.9 — constructs on projections . . . 1207
3.9.11 Proposition 8.10 — constructs preserve projections . . . 1212
3.9.12 Lecture VIII exercises . . . 1214
3.9.13 Exercise 8.11 — retraction on intervals . . . 1215
3.9.14 Exercise 8.12 — integer presentation of U . . . 1216
3.9.15 Exercise 8.13 — Boolean-algebra filters . . . 1229
3.9.16 Exercise 8.14 — closure operators . . . 1231
3.9.17 Exercise 8.15 — domain of subsystems . . . 1233
3.9.18 Exercise 8.16 — order of projections . . . 1238
3.9.19 Exercise 8.17 — explicit embeddings into U . . . 1240
3.9.20 Exercise 8.18 — missing proofs of 8.10 . . . 1243
3.9.21 Exercise 8.19 — closure under sum and product . . . 1246
3.9.22 Exercise 8.20 — self-embeddings of a domain . . . 1248
3.9.23 Exercise 8.21 — lists of a projection . . . 1250
3.9.24 Exercise 8.22 — comparing B and C . . . 1253
3.9.25 Exercise 8.23 — initial algebra inside U . . . 1255
3.9.26 Exercise 8.24 — coupled domain equations . . . 1257
3.9.27 Exercise 8.25 — reflexive function space . . . 1259
3.9.28 Exercise 8.26 — untyped calculus in U . . . 1307
3.9.29 Exercise 8.27 — infinite cartesian products . . . 1309

4 Information Systems (1982) 1314
4.1 Introduction . . . 1314
4.2 Information systems . . . 1316
4.3 The elements of a system . . . 1326
4.4 Domains as lattices and as topological spaces . . . 1356
4.5 Approximable mappings between domains . . . 1420
4.6 Products and sums of domains . . . 1427
4.7 The function space as a domain . . . 1454
4.8 Some domain equations . . . 1495
4.8.1 Factoid 8.1 . . . 1496
4.8.2 Factoid 8.2 . . . 1539
4.8.3 Factoid 8.3 . . . 1673
4.8.4 Factoid 8.4 . . . 1708

5 Bridge Theorems 1732
5.1 Introduction . . . 1732
5.2 Catalog of bridge theorems . . . 1733
5.3 Proof notes . . . 1735
5.3.1 Continuous lattices → neighbourhood systems . . . 1735
5.3.2 Neighbourhood systems → information systems . . . 1736
5.3.3 Information systems → neighbourhood systems . . . 1736

5.3.4 Information systems → ideal completion . . . 1736
5.3.5 Algebraic complete lattices → continuous lattices . . . 1736
5.3.6 Three-presentation equivalence (`presentation_domains_equiv`) . . . 1737
5.3.7 Constructions (`infoSys_constructions_equiv`) . . . 1737
5.4 Constructivity summary . . . 1837
5.5 Worked example — S-expressions / trees . . . 1837
5.5.1 Overview . . . 1838
5.5.2 Information system (1982) . . . 1838
5.5.3 Neighbourhood system (1980) . . . 1839
5.5.4 Continuous lattice (1972) . . . 1839
5.5.5 Equivalence for this instance . . . 1839
5.5.6 Domain equation and morphisms . . . 1839
5.6 ContinuousLatticeToNeighborhood . . . 2127
5.7 InfoSysConstructions . . . 2127
5.8 InfoSysToIdealCompletion . . . 2127
5.9 InfoSysToNeighborhood . . . 2128
5.10 NeighborhoodToInfoSys . . . 2128
5.11 PresentationDomains . . . 2128

6 Acknowledgments 2130
6.1 AI-assisted development . . . 2130
6.2 Artifact availability and reproducibility . . . 2130

7 References 2131

List of Figures

1 Abstract. . . . 13
2 Continuous Lattices (1972). . . . 15
3 Injective spaces. . . . 17
4 Continuous lattices. . . . 25
5 Specialization order. . . . 26
6 Scott maps. . . . 42
7 Function spaces. . . . 87
8 Inverse limits. . . . 214
9 The inverse system. . . . 215
10 Embeddings into the limit. . . . 223
11 The limit lattice. . . . 230
12 The function space tower. . . . 242
13 The function space tower. . . . 245
14 The function space tower. . . . 251
15 Datatypes as Lattices (1976). . . . 277
16 Continuous functions. . . . 280
17 Continuous. . . . 288
18 Graph. . . . 291
19 FixedPoint. . . . 293
20 Embedding. . . . 298
21 Historical note. . . . 306
22 Retracts and data types. . . . 379
23 Closure operations and algebraic lattices. . . . 387
24 Subsets and their classification. . . . 396
25 Total functions and functionality. . . . 635
26 Enumeration and degrees. . . . 809
27 Neighborhood Systems (1980). . . . 951
28 Domains given by neighbourhoods. . . . 954
29 Mappings between domains. . . . 992
30 Domain constructs. . . . 1019
31 Fixed points and recursion. . . . 1054
32 Typed λ-calculus. . . . 1085
33 Domain equations. . . . 1109
34 Computability in effectively given domains. . . . 1153
35 Retracts of the universal domain. . . . 1188
36 Information Systems (1982). . . . 1314
37 Information systems. . . . 1317
38 The elements of a system. . . . 1326
39 Domains as lattices and as topological spaces. . . . 1356
40 Approximable mappings between domains. . . . 1420
41 Products and sums of domains. . . . 1427
42 The function space as a domain. . . . 1455
43 Some domain equations. . . . 1496

44 Bridge Theorems. . . . 1732
45 Introduction. . . . 1733
46 Lean module map. . . . 1735
47 Worked example — S-expressions / trees. . . . 1838
48 ContinuousLatticeToNeighborhood. . . . 2127
49 InfoSysConstructions. . . . 2127
50 InfoSysToIdealCompletion. . . . 2128
51 InfoSysToNeighborhood. . . . 2128
52 NeighborhoodToInfoSys. . . . 2128
53 PresentationDomains. . . . 2129

Dependency diagram. The four papers stand on their own. The bridge theorems import continuous lattices, neighborhood systems, and information systems. Each box is one chapter. The arrowhead is on the chapter that imports. The number is how many direct imports. Datatypes as lattices and the references have no arrow.

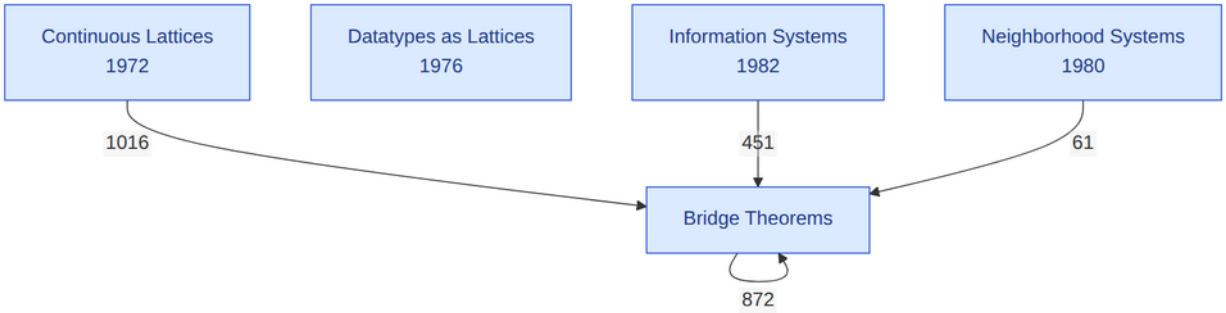


Figure 1: Abstract.

1 Continuous Lattices (1972)

Starting from the topological point of view a certain wide class of T_0-spaces is introduced having a very strong extension property for continuous functions with values in these spaces. It is then shown that all such spaces are complete lattices whose lattice structure determines the topology - these are the *continuous lattices* - and every such lattice has the extension property. With this foundation the lattices are studied in detail with respect to projections, subspaces, embeddings, and constructions such as products, sums, function spaces, and inverse limits. The main result of the paper is a proof that every topological space can be embedded in a continuous lattice which is homeomorphic (and isomorphic) to its own function space. The function algebra of such spaces provides mathematical models for the Church-Curry λ-calculus.

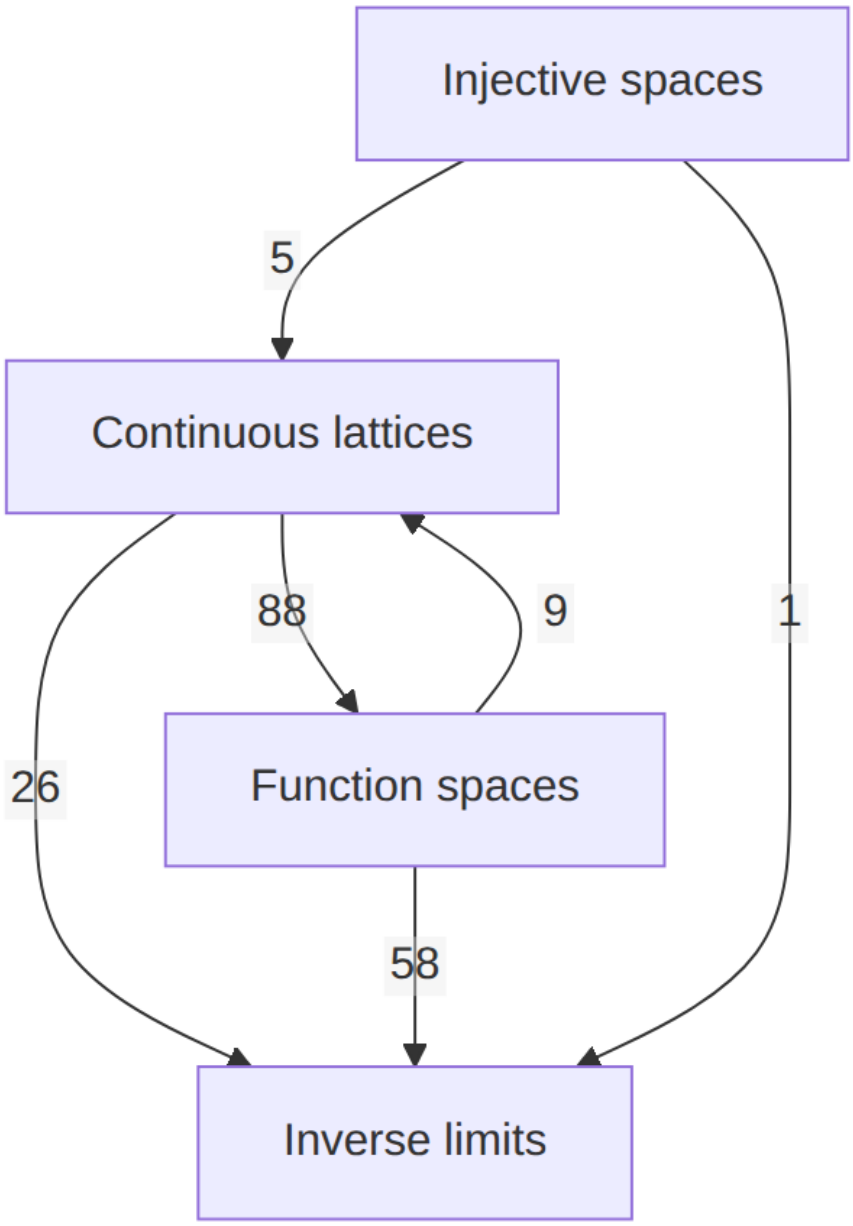


Figure 2: Continuous Lattices (1972).

1.1 Introduction

Through a roundabout chain of mathematical events I have become interested in T_0-spaces, those topological spaces satisfying the weakest separation axiom to the effect that two distinct points cannot share the same system of open neighborhoods. These spaces seem to have been originally suggested by Kolmogoroff and were introduced first in Alexandroff and Hopf (1935). Subsequent topology textbooks have dutifully recorded the definition but without much enthusiasm: mainly the idea is introduced to provide exercises.

In the book Čech (1966) for example, T_0-spaces are called *feebly semi-separated spaces*, which surely is a term expressing mild contempt. Some interest has been shown in finite T_0-spaces (finite T_1-spaces are necessarily discrete), but generally topology seems to go better under at least the Hausdorff separation axiom. The reason for this is no doubt the strong motivation we get from geometry, where points *are* points and where distinct points *can* be separated.

What I hope to show in this paper is that from a less geometric point of view T_0-spaces can be not only interesting but also natural. The interest for me lies in the construction of *function spaces*, and the main result is the production of a large number of T_0-spaces D such that D and $[D \to D]$ are *homeomorphic*. Here $[D \to D]$ is the space of all continuous functions from D into D with the topology of pointwise convergence (the product topology). It will be shown that every space can be embedded in such a space D, and that D can be chosen to have quite strong extension properties for D-valued continuous functions. These properties make D most convenient for applications to logic and recursive function theory, which was the author's original motivation. Some of the facts about these spaces seem to be most easily proved with the aid of some lattice theory, a circumstance that throws new light on the connections between topology and lattices. In fact, the required spaces are at the same time complete lattices whose topology is determined by the lattice structure in a special way, whence my title.

1.2 Injective spaces

All spaces are T_0-spaces, and we begin by defining a class of spaces to be called *injective*.

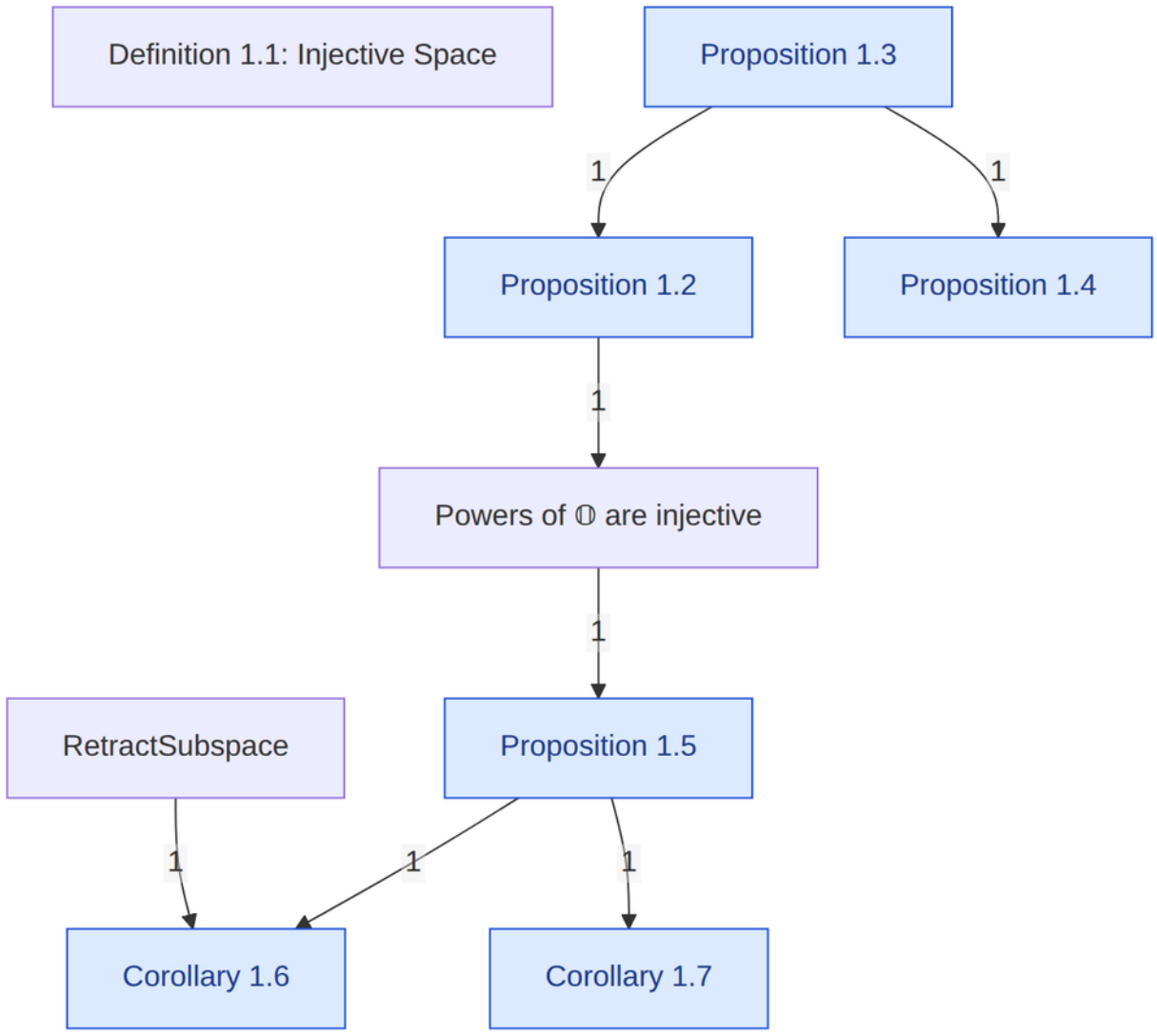


Figure 3: Injective spaces.

1.2.1 Definition 1.1: Injective Space

1.1 Definition. A T_0-space D is *injective* iff for arbitrary spaces X and Y if $X \subseteq Y$ as a subspace, then every continuous function $f : X \to D$ can be extended to a continuous function $\bar{f} : Y \to D$. As a diagram we have:

$$X \subseteq Y$$
$$X \xrightarrow{f} D$$
$$Y \xrightarrow{\bar{f}} D$$

Some people will object to this terminology because I use the subspace relationship rather than a monomorphism in the category of T_0-spaces and continuous maps. However, only the trivial 1-point space

is injective in the sense of monomorphisms in that category, and so the notion is uninteresting. If the reader prefers another terminology, I do not mind. As we shall see these spaces have very strong retraction properties.

A slightly less trivial example of an injective space is the 2-point space $\mathbb{O}$ with "points" $\bot$ and $\top$ where $\top$ is open but $\bot$ is not. (This space is sometimes called the *Sierpinski Space*.)

InjectiveSpace.lean

```
module

public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Maps.Basic
public import Mathlib.Topology.Constructions
public import Mathlib.Order.CompleteLattice.Basic

@[expose] public section

open Topology TopologicalSpace

universe u v w

class InjectiveSpace (D : Type v) [TopologicalSpace D] : Prop where
  extension :
    ∀ {X Y : Type u} [TopologicalSpace X] [TopologicalSpace Y] (e : X → Y),
      IsEmbedding e → ∀ f : C(X, D), ∃ g : C(Y, D), ∀ x, g (e x) = f x

abbrev Sierpinski := Prop

namespace InjectiveSpace

theorem sierpinski : InjectiveSpace.{u} Sierpinski := by
  constructor
  intro X Y _ _ e he f
  have hfopen : IsOpen {x | f x} := continuous_Prop.1 f.continuous
  rw [he.isInducing.isOpen_iff] at hfopen
  obtain ⟨V, hVopen, hVeq⟩ := hfopen
  refine ⟨⟨fun y => y ∈ V, continuous_Prop.2 hVopen⟩, fun x => ?_⟩
  have : (e x ∈ V) = (x ∈ {x | f x}) := by rw [← hVeq]; rfl
  simpa using this

theorem pi {ι : Type w} (D : ι → Type v) [∀ i, TopologicalSpace (D i)]
    (h : ∀ i, InjectiveSpace.{u} (D i)) : InjectiveSpace.{u} (∀ i, D i) := by
  constructor
  intro X Y _ _ e he f
  choose g hg using fun i =>
    (h i).extension e he ⟨fun x => f x i, (continuous_apply i).comp f.continuous⟩
  refine ⟨⟨fun y i => g i y, continuous_pi fun i => (g i).continuous⟩, fun x => ?_⟩
  funext i
  exact hg i x

theorem of_retract {D : Type v} {D' : Type v}
    [TopologicalSpace D] [TopologicalSpace D'] (hD : InjectiveSpace.{u} D)
    (s : C(D', D)) (r : C(D, D')) (hrs : ∀ d, r (s d) = d) :
```

```
    InjectiveSpace.{u} D' := by
  constructor
  intro X Y _ _ e he f
  obtain ⟨g, hg⟩ := hD.extension e he (s.comp f)
  refine ⟨r.comp g, fun x => ?_⟩
  show r (g (e x)) = f x
  rw [hg x]
  exact hrs (f x)

end InjectiveSpace
```

1.2.2 Proposition 1.2

1.2 Proposition. The space $\mathbb{O}$ is injective.

Proof: As is obvious, the continuous maps $f : X \to \mathbb{O}$ are in a one-one correspondence with the open subsets of X (consider $f^{-1}(\top)$). If $X \subseteq Y$ as a subspace, then an open subset of X is the restriction of some open subset of Y. Thus any $f : X \to \mathbb{O}$ can be extended to $\bar{f} : Y \to \mathbb{O}$. □

proposition_1_2.lean

```
module

public import DomainTheory.ContinuousLattice.Injective.InjectiveSpace

@[expose] public section

theorem proposition_1_2 : InjectiveSpace.{u} Sierpinski :=
  InjectiveSpace.sierpinski
```

1.2.3 Proposition 1.3

1.3 Proposition. The Cartesian product of any number of injective spaces is injective under the product topology.

Proof: The argument is standard. A map into the product can be projected onto each of the factors. Each of these projections can be extended. Then the separate maps can be put together again to make the required extended map into the product. □

We now have a large number of injective spaces, and further examples could be found using the next fact.

proposition_1_3.lean

```
module

public import DomainTheory.ContinuousLattice.Injective.InjectiveSpace
public import Mathlib.Topology.Constructions

@[expose] public section

open Topology TopologicalSpace

universe v w
```

```
theorem proposition_1_3 {ι : Type w} (D : ι → Type v) [∀ i, TopologicalSpace (D i)]
    (h : ∀ i, InjectiveSpace.{u} (D i)) : InjectiveSpace.{u} (∀ i, D i) :=
  InjectiveSpace.pi D h
```

1.2.4 Powers of $\mathbb{O}$ are injective

Scott's proof of **Proposition 1.5** uses the space `D = 𝕆^[U+1D512]`, a Cartesian power of the two-point Sierpiński space $\mathbb{O}$. That power is injective because:

- **Proposition 1.2** — the single space $\mathbb{O}$ (`Prop` with the Sierpiński topology) is injective;
- **Proposition 1.3** — a product of injective spaces is injective under the product topology.

For any index type ι, the Lean power is $\iota \to$ `Prop` with the product topology. The lemma `isInjectiveSpace_sierpinski_power` is exactly "apply 1.3 to $\mathtt{i} \mapsto$ `Prop` using 1.2 at each index." In Proposition 1.5 we take $\iota =$ `Opens X`.

`isInjectiveSpace_sierpinski_power.lean`

```
module

public import DomainTheory.ContinuousLattice.Injective.InjectiveSpace
public import Mathlib.Topology.Constructions
import DomainTheory.ContinuousLattice.Injective.proposition_1_2
import DomainTheory.ContinuousLattice.Injective.proposition_1_3

@[expose] public section

open Topology TopologicalSpace

universe w

theorem isInjectiveSpace_sierpinski_power (ι : Type w) :
    InjectiveSpace.{u} (ι → Prop) :=
  proposition_1_3 (fun (_ : ι) => Prop) (fun _ => proposition_1_2)
```

1.2.5 Proposition 1.4

1.4 Proposition. A retract of an injective space is injective.

Proof: Let D be injective. By a retract of D we understand a subspace $D' \subseteq D$ for which there exists a retraction map $j : D \to D'$ such that

$$D' = \{x \in D : j(x) = x\}.$$

Then if $f : X \to D'$ and $X \subseteq Y$, we have $f : X \to D$ as a continuous map also. Taking $\bar{f} : Y \to D$, we have only to form

$$j \circ \bar{f} : Y \to D'$$

to show that D' is injective. □

The relationship between arbitrary T_0-spaces and the injective spaces is given by the embedding theorem.

proposition_1_4.lean

```
module

public import DomainTheory.ContinuousLattice.Injective.InjectiveSpace
public import Mathlib.Topology.Defs.Basic

@[expose] public section

open Topology TopologicalSpace

universe v

theorem proposition_1_4 {D : Type v} {D' : Type v} [TopologicalSpace D] [
    TopologicalSpace D']
    (hD : InjectiveSpace.{u} D) (s : C(D', D)) (r : C(D, D')) (hrs : ∀ d, r (s d) = d)
    :
    InjectiveSpace.{u} D' :=
  InjectiveSpace.of_retract hD s r hrs
```

1.2.6 Proposition 1.5

1.5 Proposition. Every T_0-space can be embedded in an injective space; in fact, in a Cartesian power of the 2-element space $\mathbb{O}$.

Proof: The proof is well known (cf. Čech (1966), Theorem 26B.9, p. 484.) But we give the argument for completeness sake. Let X be the given space, and let $\mathfrak{O}$ be the class of open subsets of X. Let

$$D = \mathbb{O}^{\mathfrak{O}}$$

be the Cartesian power of $\mathbb{O}$. Then D is injective by 1.3. Define the map $e : X \to D$ by:

$$e(x)(U) = \begin{cases} \top & \text{if } x \in U, \\ \bot & \text{if } x \notin U, \end{cases}$$

for $x \in X$ and $U \in \mathfrak{O}$. This map e is continuous in view of the topology given to $\mathbb{O}$ and to D. The map e is one-one, because X is T_0. Finally, if $U \subseteq X$ is open, then

$$e(U) = e(x) : x \in U \quad = e(x) : e(x)(U) = \top = e(X) \cap t \in D : t(U) \in \top,$$

which shows that the image $e(U)$ is open in the subspace $e(X) \subseteq D$. Therefore $e : X \to D$ is an embedding of X as a subspace in D. □

proposition_1_5.lean

```
module

public import Mathlib.Topology.ContinuousMap.T0Sierpinski
public import DomainTheory.ContinuousLattice.Injective.InjectiveSpace
import DomainTheory.ContinuousLattice.Injective.isInjectiveSpace_sierpinski_power
```

```
@[expose] public section

open Topology TopologicalSpace

universe u

/-- **Scott 1972, Proposition 1.5.** Every `T₀`-space embeds into a power of 𝕆, and
    that
power is injective. -/
theorem proposition_1_5 (X : Type u) [TopologicalSpace X] [T0Space X] :
    InjectiveSpace.{u,u} (Opens X → Prop) ∧
      IsEmbedding (productOfMemOpens X) := by
  refine ⟨?_, productOfMemOpens_isEmbedding X⟩
  exact isInjectiveSpace_sierpinski_power (Opens X)
```

1.2.7 RetractSubspace

Corollary 1.6 characterizes injective spaces as retracts of powers of $\mathbb{O}$. The Lean development packages "`D` is a subspace of a larger space and the inclusion splits by a continuous retraction" as `RetractSubspace D' D`: a continuous section `D' → D` that is an embedding, together with a retraction `D → D'` fixing the subspace pointwise.

Corollary 1.6 then states that `D` is injective if and only if it is such a retract of some power `ι → Prop`.

RetractSubspace.lean

```
module

public import Mathlib.Topology.ContinuousMap.Defs
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Order.CompleteLattice.Basic

@[expose] public section

universe u

open Topology TopologicalSpace

/-- Scott's notion of retract: a subspace with a continuous retraction. -/
structure RetractSubspace (D' D : Type u) [TopologicalSpace D'] [TopologicalSpace D]
    where
  section' : C(D', D)
  isEmbedding_section : IsEmbedding section'
  retraction : C(D, D')
  retraction_section : ∀ d, retraction (section' d) = d
```

1.2.8 Corollary 1.6

1.6 Corollary. The injective spaces are exactly the retracts of the Cartesian powers of $\mathbb{O}$.

Proof: Such a retract is injective by 1.4. If D is injective, then it is (homeomorphic to) a subspace of a power of $\mathbb{O}$. But since D is injective the identity function on the subspace to itself can be extended to the whole of the power of $\mathbb{O}$ providing the required retraction. □

corollary_1_6.lean

```
module

public import DomainTheory.ContinuousLattice.Injective.RetractSubspace
public import DomainTheory.ContinuousLattice.Injective.InjectiveSpace
public import Mathlib.Topology.Separation.Basic
import DomainTheory.ContinuousLattice.Injective.isInjectiveSpace_sierpinski_power
import DomainTheory.ContinuousLattice.Injective.proposition_1_5

@[expose] public section

open Topology TopologicalSpace

universe u

theorem corollary_1_6 (D : Type u) [TopologicalSpace D] [T0Space D] :
    InjectiveSpace.{u,u} D ↔ ∃ (ι : Type u), Nonempty (RetractSubspace D (ι → Prop))
    := by
  constructor
  · intro hD
    obtain ⟨r, hr⟩ := hD.extension (productOfMemOpens D) (proposition_1_5 D).2 (
    ContinuousMap.id D)
    refine ⟨Opens D, ⟨⟨productOfMemOpens D, (proposition_1_5 D).2, r, hr⟩⟩⟩
  · rintro ⟨ι, ⟨R⟩⟩
    exact InjectiveSpace.of_retract (isInjectiveSpace_sierpinski_power ι) R.section' R.
    retraction
      R.retraction_section
```

1.2.9 Corollary 1.7

1.7 Corollary. A space is injective iff it is a retract of every space of which it is a subspace.

Proof: As in the proof of 1.6, this property is obvious for injective spaces. But in view of 1.5 every such space is a retract of a power of $\mathbb{O}$ and hence is injective. □

corollary_1_7.lean

```
module

public import DomainTheory.ContinuousLattice.Injective.InjectiveSpace
public import Mathlib.Topology.Separation.Basic
import DomainTheory.ContinuousLattice.Injective.isInjectiveSpace_sierpinski_power
import DomainTheory.ContinuousLattice.Injective.proposition_1_5

@[expose] public section

open Topology TopologicalSpace

universe u

theorem corollary_1_7 (D : Type u) [TopologicalSpace D] [T0Space D] :
    InjectiveSpace.{u,u} D ↔
      ∀ {Y : Type u} [TopologicalSpace Y] (e : D → Y), IsEmbedding e →
        ∃ r : C(Y, D), ∀ d, r (e d) = d := by
  constructor
```

```
· intro hD Y _ e he
  obtain ⟨r, hr⟩ := hD.extension e he (ContinuousMap.id D)
  exact ⟨r, hr⟩
· intro hD
  obtain ⟨r, hr⟩ := hD (productOfMemOpens D) (proposition_1_5 D).2
  exact InjectiveSpace.of_retract (isInjectiveSpace_sierpinski_power (Opens D))
    (productOfMemOpens D) r hr
```

1.2.10 Absolute retracts

As a result of these very elementary considerations, the injective spaces could be called *absolute retracts*, if one remembers to modify the standard definitions by using *arbitrary* subspaces rather than just *closed* subspaces. Note too that it is easy to show that the only continuous maps $e : X \to Y$ for which the extension property

$$X \xrightarrow{e} Y$$

$$X \xrightarrow{f} \mathbb{O}$$

$$Y \xrightarrow{\bar{f}} \mathbb{O}$$

could hold for *all* continuous $f : X \to \mathbb{O}$ are embeddings as subspaces.

Thus it would seem that we have a reasonably good initial grasp of the notion of injective spaces, but further constructions are considerably facilitated by the introduction of the lattice structure.

1.3 Continuous lattices

Every T_0-space becomes a partially ordered set under the definition:

$$x \sqsubseteq y \text{ iff whenever } x \in U \text{ and } U \text{ is open, then } y \in U.$$

Indeed, though this relation is reflexive and transitive, the condition that it be *antisymmetric* is exactly equivalent to the T_0-axiom.

In the converse direction, every partially ordered set $\langle X, \sqsubseteq \rangle$ can be so obtained, for we have only to define $U \subseteq X$ as being open if it satisfies the condition:

(i) whenever $x \in U$ and $x \sqsubseteq y$, then $y \in U$.

The axioms for partial order make X a T_0-space, because for any $y \in X$ the set

$$\{x \in X : x \not\sqsubseteq y\}$$

is open. This connection is not very interesting, however.

What *is* interesting in topological spaces is *convergence* and the properties of *limit points*. We shall discuss limits in terms of *nets*, in particular in terms of *monotone nets*. A monotone net in a T_0-space X is a function

$$x : I \to X$$

where $\langle I, \leq \rangle$ is a directed set and where $i \leq j$ implies $x_i \sqsubseteq x_j$ for all $i, j \in I$. In a T_1-space a monotone net is constant (hence, uninteresting) because the $\sqsubseteq$-relation is the identity. As usual (cf. Kelley (1955), p.66) we say that a net x *converges* to an element y iff whenever U is open and $y \in U$, then for some $i \in I$ we have $x_j \in U$ for all $j \geq i$. Note that a monotone net x converges to each of its terms x_i. Suppose that a monotone net x converges to an element y which is an upper bound to all the terms of x. Then y must be the *least upper bound*, which we write as:

$$y = \bigsqcup \{x_i : i \in I\}.$$

To see this, assume that z is any other upper bound with $x_i \sqsubseteq z$ for all $i \in I$. If U is open and $y \in U$, then $x_i \in U$ for some $i \in I$. But then $z \in U$, and so $y \sqsubseteq z$ follows.

We shall find that most of the facts about the topology of the spaces we are concerned with here can be expressed in terms of least upper bounds (lubs). It is not always the case, however, that lubs are limits. Thus, for a partially ordered set X, we impose a further restriction on its topology beyond condition (i) for saying when a subset U is open:

(ii) whenever $S \subseteq X$ is *directed*, $\bigsqcup S$ exists, and $\bigsqcup S \in U$, then $S \cap U \neq \emptyset$.

By a directed subset of X we of course mean that it is directed in the sense of the partial ordering $\sqsubseteq$. Note that in this paper directed sets are always *non-empty*. The sets satisfying (i) and (ii) form the *induced topology* on a partially ordered set X, which is still a T_0-space because the sets $\{x \in X : x \not\sqsubseteq y\}$ remain open even in the sense of (ii). Obviously a directed set $S \subseteq X$ can be regarded as a net, and now in view of (ii) it follows that S converges to $\bigsqcup S$ — if this lub exists. We can summarize this discussion as follows.

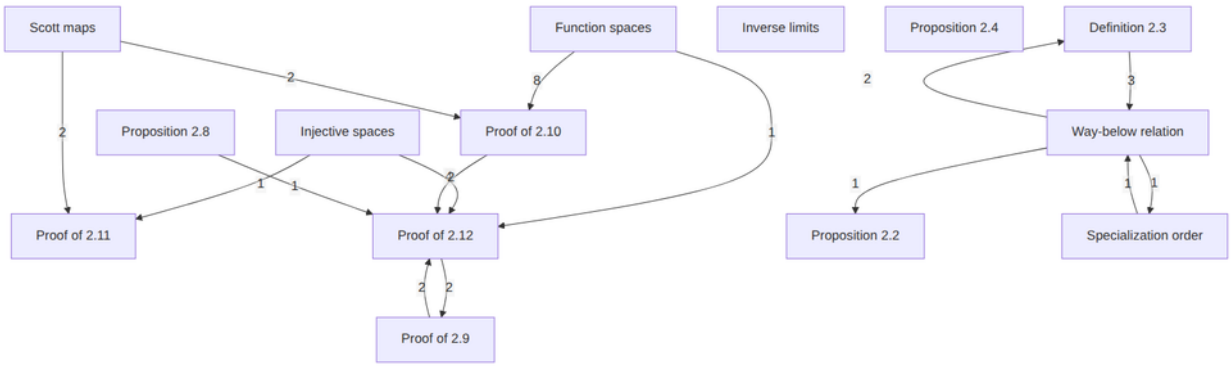


Figure 4: Continuous lattices.

1.3.1 Specialization order

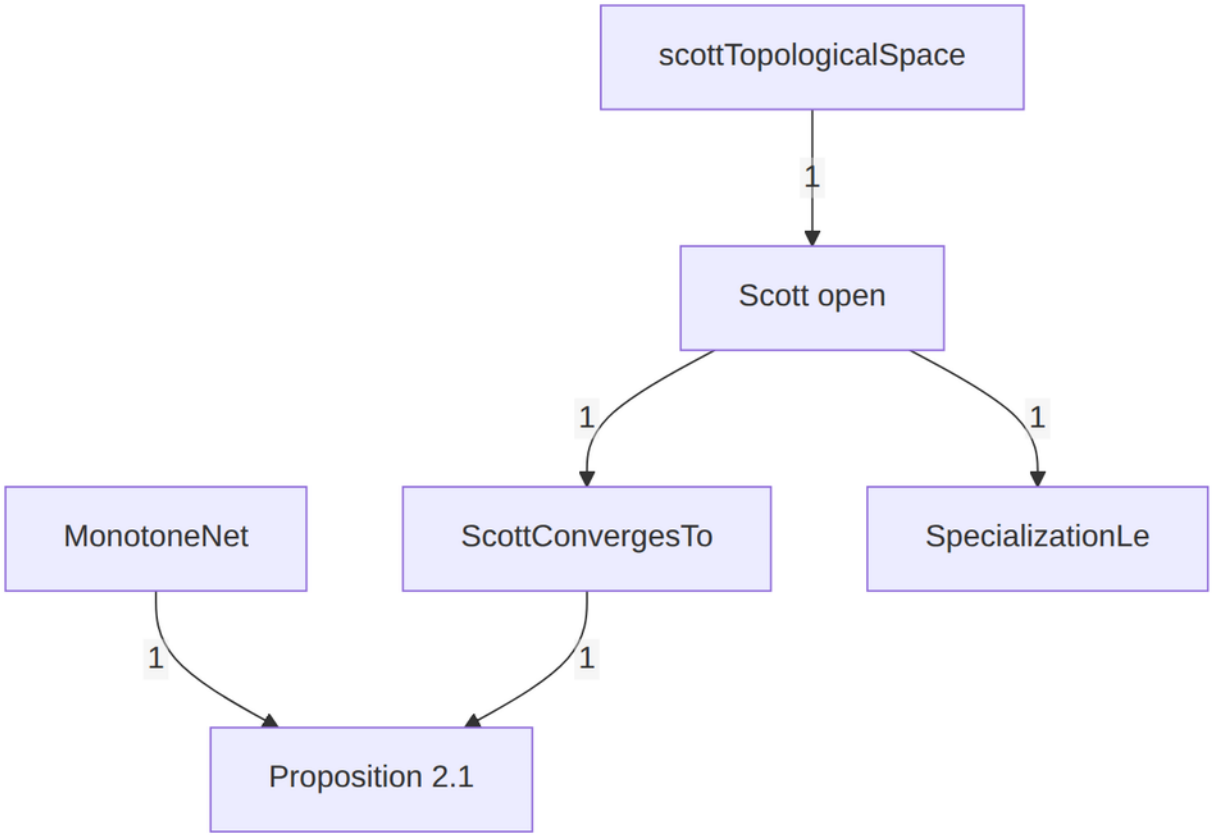


Figure 5: Specialization order.

scottTopologicalSpace `scottTopologicalSpace.lean`

```
module

public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Data.Set.Basic
public import Mathlib.Order.CompleteLattice.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

universe u

variable {X : Type*} {D : Type u} [TopologicalSpace X] [CompleteLattice D]
```

```
@[reducible] noncomputable def scottTopologicalSpace : TopologicalSpace D :=
  Topology.scott D (@Set.univ (Set D))

namespace scottTopologicalSpace

/-- The Scott topology of a partial order is `T₀`. -/
theorem t0Space : @T0Space D scottTopologicalSpace := by
  let : TopologicalSpace D := scottTopologicalSpace
  have : IsScott D (@Set.univ (Set D)) := ⟨rfl⟩
  exact inferInstanceAs (T0Space D)

end scottTopologicalSpace

end DomainTheory.ContinuousLattice
```

Scott open ScottOpen.lean

```
module

public import DomainTheory.ContinuousLattice.Specialization.scottTopologicalSpace
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

universe u

variable {X : Type*} {D : Type u} [TopologicalSpace X] [CompleteLattice D]

def ScottOpen (U : Set D) : Prop :=
  open CompleteLattice in
  letI : LE D := (toCompleteSemilatticeInf (α := D)).toPartialOrder.toPreorder.toLE
  letI : SupSet D := (toCompleteSemilatticeSup (α := D)).toSupSet
  IsUpperSet U ∧
    ∀ {{S : Set D}}, S.Nonempty → DirectedOn (· ≤ ·) S → sSup S ∈ U → (S ∩ U).
    Nonempty

namespace ScottOpen

theorem isUpperSet {U : Set D} (h : ScottOpen U) : IsUpperSet U := h.1
```

```
theorem inter {U V : Set D} (hU : ScottOpen U) (hV : ScottOpen V) : ScottOpen (U ∩ V)
    := by
  refine ⟨hU.1.inter hV.1, fun S hS hSdir hmem => ?_⟩
  obtain ⟨s₁, hs₁S, hs₁U⟩ := hU.2 hS hSdir hmem.1
  obtain ⟨s₂, hs₂S, hs₂V⟩ := hV.2 hS hSdir hmem.2
  obtain ⟨s₃, hs₃S, h₁, h₂⟩ := hSdir s₁ hs₁S s₂ hs₂S
  exact ⟨s₃, hs₃S, hU.1 h₁ hs₁U, hV.1 h₂ hs₂V⟩

theorem univ : ScottOpen (Set.univ : Set D) := by
  refine ⟨isUpperSet_univ, fun S hS _ _ => ?_⟩
  obtain ⟨s, hs⟩ := hS
  exact ⟨s, hs, Set.mem_univ s⟩

theorem sUnion {C : Set (Set D)} (hC : ∀ U ∈ C, ScottOpen U) : ScottOpen (⋃₀ C) := by
  refine ⟨isUpperSet_sUnion fun U hU => (hC U hU).1, fun S hS hSdir hmem => ?_⟩
  obtain ⟨U, hUC, hUmem⟩ := hmem
  obtain ⟨s, hsS, hsU⟩ := (hC U hUC).2 hS hSdir hUmem
  exact ⟨s, hsS, U, hUC, hsU⟩

theorem iff_dirSupInacc {U : Set D} : ScottOpen U ↔ IsUpperSet U ∧ DirSupInacc U := by
  constructor
  · intro ⟨hU, hU'⟩
    refine ⟨hU, fun d hd₁ hd₂ a ha hmem => ?_⟩
    rw [← IsLUB.sSup_eq ha] at hmem
    obtain ⟨s, hs, hsU⟩ := hU' hd₁ hd₂ hmem
    exact ⟨s, hs, hsU⟩
  · intro ⟨hU, hU'⟩
    refine ⟨hU, fun d hd₁ hd₂ hmem => ?_⟩
    obtain ⟨s, hs, hsU⟩ := hU' hd₁ hd₂ (isLUB_sSup d) hmem
    exact ⟨s, hs, hsU⟩

theorem isOpen_iff {U : Set D} : @IsOpen D scottTopologicalSpace U ↔ ScottOpen U := by
  rw [iff_dirSupInacc, scottTopologicalSpace]
  have hinst : @IsScott D (@Set.univ (Set D)) _ (Topology.scott D (@Set.univ (Set D)))
     :=
    @IsScott.mk D (@Set.univ (Set D)) _ (Topology.scott D (@Set.univ (Set D))) rfl
  simpa [dirSupInaccOn_univ] using
    @IsScott.isOpen_iff_isUpperSet_and_dirSupInaccOn D (@Set.univ (Set D)) _
      (Topology.scott D (@Set.univ (Set D))) U hinst

variable {ι : Type u} [Preorder ι] [IsDirected ι (· ≤ ·)]

variable {x : ι → D} {L y : D}

theorem not_le (L : D) : ScottOpen {z : D | ¬ z ≤ L} := by
  refine ⟨fun a b hab ha hb => ha (le_trans hab hb), fun S hSne hSdir hmem => ?_⟩
  by_contra hcon
  refine hmem (sSup_le fun s hs => ?_)
  by_contra hsL
  exact hcon ⟨s, hs, hsL⟩

end ScottOpen

end DomainTheory.ContinuousLattice
```

scottOpen_wayBelow.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow

@[expose] public section

namespace DomainTheory.ContinuousLattice

universe u

variable {D : Type u} [CompleteLattice D]

/-- Auxiliary: the up-set `{z | x ≪ z}` is itself Scott-open. (This is *not* Scott's
Proposition 2.2(vi) -- see `WayBelow.self_iff_scottOpen_Ici` for that -- but it is a
    standard
and useful fact, the openness of the sets `⇑x`.) -/
theorem scottOpen_wayBelow (x : D) : ScottOpen {z | x ≪ z} := by
  refine ⟨fun a b hab ha => ha.trans_le hab, fun S hS hSdir hmem => ?_⟩
  obtain ⟨U, hU, hsupU, hsub⟩ := hmem
  obtain ⟨s, hsS, hsU⟩ := hU.2 hS hSdir hsupU
  exact ⟨s, hsS, ⟨U, hU, hsU, hsub⟩⟩

end DomainTheory.ContinuousLattice
```

ScottConvergesTo ScottConvergesTo.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ScottOpen
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

universe u

variable {X : Type*} {D : Type u} [TopologicalSpace X] [CompleteLattice D]

variable {ι : Type u} [Preorder ι] [IsDirected ι (· ≤ ·)]

def ScottConvergesTo (x : ι → D) (y : D) : Prop :=
  ∀ U, ScottOpen U → y ∈ U → ∃ i, ∀ j ≥ i, x j ∈ U
```

```
end DomainTheory.ContinuousLattice
```

SpecializationLe SpecializationLe.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ScottOpen
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Topology.Defs.Filter
public import Mathlib.Topology.Continuous
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

universe u

variable {X : Type*} {D : Type u} [TopologicalSpace X] [CompleteLattice D]

/-- **Scott 1972, §2.** The *specialization order*: `x ⊑ y` when `x ∈ U` open implies `
    y ∈ U`. -/
def SpecializationLe (x y : X) : Prop :=
  ∀ U, IsOpen U → x ∈ U → y ∈ U

namespace SpecializationLe

/-- Scott's specialization order is the reverse of Mathlib's `Specializes` convention.
    -/
theorem iff_specializes {x y : X} :
    SpecializationLe x y ↔ y ⤳ x :=
  specializes_iff_forall_open.symm

variable {ι : Type u} [Preorder ι] [IsDirected ι (· ≤ ·)]

variable {x : ι → D} {L y : D}

/-- The specialization order of the Scott topology is the original lattice order. -/
theorem scott_iff {x y : D} :
    @SpecializationLe D scottTopologicalSpace x y ↔ x ≤ y := by
  constructor
  · intro hxy
    by_contra h
```

```
    exact hxy {z | ¬ z ≤ y} (ScottOpen.isOpen_iff.mpr (ScottOpen.not_le y)) h (by simp
    )
  · intro hxy U hU hxU
    exact (ScottOpen.isOpen_iff.mp hU).1 hxy hxU

/-- Continuous maps preserve Scott's specialization order. -/
theorem map {Y : Type*} [TopologicalSpace Y] {x y : X}
    (hxy : SpecializationLe x y) {f : X → Y} (hf : Continuous f) :
    SpecializationLe (f x) (f y) := by
  intro U hU hxU
  exact hxy (f ^-1' U) (hU.preimage hf) hxU

end SpecializationLe

end DomainTheory.ContinuousLattice
```

MonotoneNet MonotoneNet.lean

```
module
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

universe u

variable {X : Type*} {D : Type u} [TopologicalSpace X] [CompleteLattice D]

variable {ι : Type u} [Preorder ι] [IsDirected ι (· ≤ ·)]

class MonotoneNet (x : ι → D) : Prop where
  out : Monotone x

end DomainTheory.ContinuousLattice
```

Proposition 2.1 **2.1 Proposition.** *In a partially ordered set X with the induced topology, a monotone net $x : I \to X$ with a least upper bound converges to an element $y \in X$ iff*

$$y \sqsubseteq \bigsqcup \{x_i : i \in I\}.$$

□

proposition_2_1.lean

```
module

public import DomainTheory.ContinuousLattice.Specialization.MonotoneNet
public import DomainTheory.ContinuousLattice.Specialization.ScottConvergesTo
public import DomainTheory.ContinuousLattice.WayBelow.ScottOpen
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

universe u

variable {X : Type*} {D : Type u} [TopologicalSpace X] [CompleteLattice D]

variable {ι : Type u} [Preorder ι] [IsDirected ι (· ≤ ·)]

variable {x : ι → D} {L y : D}

namespace proposition_2_1

omit [IsDirected ι (· ≤ ·)] in
theorem le_of_converges (hL : IsLUB (range x) L) (hconv : ScottConvergesTo x y) : y ≤
    L := by
  by_contra hyL
  obtain ⟨i, hi⟩ := hconv {z : D | ¬ z ≤ L} (ScottOpen.not_le L) hyL
  exact hi i le_rfl (hL.1 ⟨i, rfl⟩)

theorem of_le [Nonempty ι] (hx : MonotoneNet x) (hL : IsLUB (range x) L) (hyL : y ≤ L)
    :
    ScottConvergesTo x y := by
  intro U hU hyU
  have hdir : DirectedOn (· ≤ ·) (range x) := by
    rintro _ ⟨i, rfl⟩ _ ⟨j, rfl⟩
    obtain ⟨k, hik, hjk⟩ := IsDirected.directed (r := (· ≤ ·)) i j
    exact ⟨x k, ⟨k, rfl⟩, hx.out hik, hx.out hjk⟩
  have hLU : sSup (range x) ∈ U := by
    rw [IsLUB.sSup_eq hL]
    exact hU.1 hyL hyU
  obtain ⟨s, hsS, hsU⟩ := hU.2 (Set.range_nonempty x) hdir hLU
  obtain ⟨i₀, rfl⟩ := hsS
  refine ⟨i₀, fun j hj => hU.1 (hx.out hj) hsU⟩
```

```
end proposition_2_1

theorem proposition_2_1 [Nonempty ι] (hx : MonotoneNet x) (hL : IsLUB (range x) L) :
    ScottConvergesTo x y ↔ y ≤ L :=
  ⟨fun hconv => proposition_2_1.le_of_converges hL hconv, proposition_2_1.of_le hx hL⟩

end DomainTheory.ContinuousLattice
```

1.3.2 Way-below relation

Our main interest will lie with those partially ordered sets in which every subset has a lub: namely, *complete lattices*. If D is such a space we write $\bot = \bigsqcup \emptyset$ and $\top = \bigsqcup D$ for the smallest and largest elements (read: bottom and top). As is well known, greatest lower bounds must exist, for:

$$\sqcap S = \bigsqcup \{x \in D : x \sqsubseteq y\} \text{ for all } y \in S$$

gives the definition.

Given a complete lattice D we define

$$x \ll y \text{ iff } y \in \text{Int}\{z \in D : x \sqsubseteq z\},$$

where the interior is taken in the sense of the induced topology. The relation $x \ll y$ behaves somewhat like a strict ordering relation; at least its meaning is clearly that y should be definitely larger than x in the partial ordering. Such a relation has many pleasant properties.

The primary purpose of introducing it is to provide a simple definition for the kind of spaces that are most useful to us. We first mention the most elementary features of this relation.

WayBelow.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ScottOpen
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

universe u

variable {D : Type u} [CompleteLattice D]

def WayBelow (x y : D) : Prop :=
  letI : LE D :=
    (CompleteLattice.toCompleteSemilatticeInf (α := D)).toPartialOrder.toPreorder.toLE
  letI : Preorder D :=
    (CompleteLattice.toCompleteSemilatticeInf (α := D)).toPartialOrder.toPreorder
  ∃ U : Set D, ScottOpen U ∧ y ∈ U ∧ U ⊆ Set.Ici x

@[inherit_doc] scoped infix:50 " ≪ " => WayBelow

/-- **Scott 1972, Proposition 2.2(i).** `⊥ ≪ x` for every `x`. -/
theorem bot_wayBelow (x : D) : (⊥ : D) ≪ x :=
```

```
  ⟨Set.univ, ScottOpen.univ, Set.mem_univ x, fun _ _ => bot_le⟩

/-- **Scott 1972, Proposition 2.2(ii).** `x ≪ z` and `y ≪ z` imply `x ⊔ y ≪ z`. -/
theorem WayBelow.sup {x y z : D} (hx : x ≪ z) (hy : y ≪ z) : x ⊔ y ≪ z := by
  obtain ⟨U, hU, hzU, hUsub⟩ := hx
  obtain ⟨V, hV, hzV, hVsub⟩ := hy
  refine ⟨U ∩ V, hU.inter hV, ⟨hzU, hzV⟩, fun w hw => ?_⟩
  exact Set.mem_Ici.2 (sup_le (hUsub hw.1) (hVsub hw.2))

/-- **Scott 1972, Proposition 2.2(iii).** `x ≪ y` and `y ≤ z` imply `x ≪ z` (monotone
    on the
right). -/
theorem WayBelow.trans_le {x y z : D} (h : x ≪ y) (hyz : y ≤ z) : x ≪ z := by
  obtain ⟨U, hU, hyU, hsub⟩ := h
  exact ⟨U, hU, hU.1 hyz hyU, hsub⟩

/-- **Scott 1972, Proposition 2.2(iv).** `x ≤ y` and `y ≪ z` imply `x ≪ z` (monotone
   on the
left). -/
theorem WayBelow.le_trans {x y z : D} (hxy : x ≤ y) (h : y ≪ z) : x ≪ z := by
  obtain ⟨U, hU, hzU, hsub⟩ := h
  refine ⟨U, hU, hzU, fun w hw => ?_⟩
  have hyw : y ≤ w := Set.mem_Ici.1 (hsub hw)
  exact Set.mem_Ici.2 (hxy.trans hyw)

/-- **Scott 1972, Proposition 2.2(v).** `x ≪ y` implies `x ≤ y`. -/
theorem WayBelow.le {x y : D} (h : x ≪ y) : x ≤ y := by
  obtain ⟨U, _, hyU, hsub⟩ := h
  exact hsub hyU

theorem WayBelow.self_iff_scottOpen_Ici {x : D} : x ≪ x ↔ ScottOpen (Set.Ici x) := by
  constructor
  · rintro ⟨U, hU, hxU, hsub⟩
    -- `Ici x = U`: `Ici x ⊆ U` since `U` is upper and `x ∈ U`; `U ⊆ Ici x` is `hsub`.
    have hIci : Set.Ici x = U :=
      le_antisymm (fun w hw => hU.1 (Set.mem_Ici.1 hw) hxU) hsub
    rw [hIci]; exact hU
  · intro hopen
    exact ⟨Set.Ici x, hopen, Set.self_mem_Ici, le_refl _⟩

theorem WayBelow.sSup_iff {x : D} {S : Set D} (hS : S.Nonempty)
    (hSdir : DirectedOn (· ≤ ·) S) : x ≪ sSup S ↔ ∃ y ∈ S, x ≪ y := by
  constructor
  · rintro ⟨U, hU, hsupU, hsub⟩
    obtain ⟨s, hsS, hsU⟩ := hU.2 hS hSdir hsupU
    exact ⟨s, hsS, U, hU, hsU, hsub⟩
  · rintro ⟨y, hyS, hxy⟩
    obtain ⟨U, hU, hyU, hsub⟩ := hxy
    exact ⟨U, hU, hU.1 (le_sSup hyS) hyU, hsub⟩

end DomainTheory.ContinuousLattice
```

directedOn_wayBelow.lean

```
module
```

```
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow

@[expose] public section

namespace DomainTheory.ContinuousLattice

universe u

variable {D : Type u} [CompleteLattice D]

/-- The set `{x | x ≪ y}` of elements way below `y` is directed: it is closed under
    binary
joins by Proposition 2.2(ii) (`WayBelow.sup`). This holds in *any* complete lattice. -/
theorem directedOn_wayBelow (y : D) : DirectedOn (· ≤ ·) {x | x ≪ y} :=
  fun a ha b hb => ⟨a ⊔ b, ha.sup hb, le_sup_left, le_sup_right⟩

end DomainTheory.ContinuousLattice
```

exists_wayBelow_Ici_subset.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
import DomainTheory.ContinuousLattice.WayBelow.directedOn_wayBelow
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

universe u

variable {D : Type u} [CompleteLattice D]

/-- A strengthening of `exists_wayBelow_subset`: the witness `a ≪ z` can be taken with
     the whole
principal up-set `Set.Ici a` (not merely `⇑a`) inside `U`. The element `a` produced
    lies in the
open `U`, which is upper, so `↑a ⊆ U`. -/
theorem exists_wayBelow_Ici_subset (hD : ContinuousLattice D) {U : Set D} (hU :
    ScottOpen U)
    {z : D} (hz : z ∈ U) : ∃ a, a ≪ z ∧ Set.Ici a ⊆ U := by
  have hne : {a | a ≪ z}.Nonempty := ⟨⊥, bot_wayBelow z⟩
  have hsup : sSup {a | a ≪ z} ∈ U := by rw [hD.sSup_wayBelow z]; exact hz
  obtain ⟨a, haz, haU⟩ := hU.2 hne (directedOn_wayBelow z) hsup
  exact ⟨a, haz, fun w hw => hU.1 (Set.mem_Ici.1 hw) haU⟩

end DomainTheory.ContinuousLattice
```

exists_wayBelow_subset.lean

```
module
```

```
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
import DomainTheory.ContinuousLattice.WayBelow.directedOn_wayBelow

@[expose] public section

namespace DomainTheory.ContinuousLattice

universe u

variable {D : Type u} [CompleteLattice D]

/-- In a continuous lattice the sets `⇑a = {z | a ≪ z}` form a basis of the Scott
    topology:
every Scott-open `U` containing `z` contains some basic neighbourhood `⇑a` of `z`.
    Indeed
`z = ⊔{a | a ≪ z}` is a directed supremum lying in the open set `U`, so some `a ≪ z`
    already
lies in `U`, and then `⇑a ⊆ ↑a ⊆ U`. -/
theorem exists_wayBelow_subset (hD : ContinuousLattice D) {U : Set D} (hU : ScottOpen
    U)
    {z : D} (hz : z ∈ U) : ∃ a, a ≪ z ∧ {w | a ≪ w} ⊆ U := by
  have hne : {a | a ≪ z}.Nonempty := ⟨⊥, bot_wayBelow z⟩
  have hsup : sSup {a | a ≪ z} ∈ U := by rw [hD.sSup_wayBelow z]; exact hz
  obtain ⟨a, haz, haU⟩ := hU.2 hne (directedOn_wayBelow z) hsup
  exact ⟨a, haz, fun w hw => hU.1 hw.le haU⟩

end DomainTheory.ContinuousLattice
```

isContinuousLattice_iff_isLUB_sInf_nhds.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
import DomainTheory.ContinuousLattice.WayBelow.sInf_wayBelow
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

universe u

variable {D : Type u} [CompleteLattice D]

/-- **Scott 1972, Proposition 2.4.** A complete lattice is continuous iff every
    element is the
supremum of the infima of its open neighbourhoods: `y = ⊔ {⊓U : y ∈ U open}`. This is
    Scott's
alternate form of Definition 2.3. -/
theorem isContinuousLattice_iff_isLUB_sInf_nhds :
    ContinuousLattice D ↔
      ∀ y : D, IsLUB {a : D | ∃ U, ScottOpen U ∧ y ∈ U ∧ a = sInf U} y := by
  constructor
  · intro h y
```

```
    refine ⟨?_, ?_⟩
    · rintro a ⟨U, hU, hyU, rfl⟩
      exact (sInf_wayBelow hU hyU).le
    · intro b hb
      refine (h.out y).2 ?_
      intro x hx
      obtain ⟨U, hU, hyU, hsub⟩ := hx
      have hxle : x ≤ sInf U := le_sInf fun _ hz => Set.mem_Ici.1 (hsub hz)
      exact hxle.trans (hb ⟨U, hU, hyU, rfl⟩)
  · intro h
    refine { out := ?_ }
    intro y
    refine ⟨fun x hx => hx.le, ?_⟩
    intro b hb
    refine (h y).2 ?_
    rintro a ⟨U, hU, hyU, rfl⟩
    exact hb (sInf_wayBelow hU hyU)

end DomainTheory.ContinuousLattice
```

sInf_wayBelow.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

universe u

variable {D : Type u} [CompleteLattice D]

/-- The infimum of a Scott-open neighbourhood of `y` is way below `y`: the open set is
    itself
the required witness. Scott uses this in moving between Definition 2.3 and Proposition
     2.4. -/
theorem sInf_wayBelow {U : Set D} (hU : ScottOpen U) {y : D} (hy : y ∈ U) :
    sInf U ≪ y :=
  ⟨U, hU, hy, fun _ hz => Set.mem_Ici.2 (sInf_le hz)⟩

end DomainTheory.ContinuousLattice
```

1.3.3 Proposition 2.2

2.2 Proposition. In a complete lattice D we have:

(i) $\bot \ll x$;

(ii) $x \ll z$ and $y \ll z$ imply $x \sqcup y \ll z$;

(iii) $x \ll y \sqsubseteq z$ implies $x \ll z$;

(iv) $x \sqsubseteq y \ll z$ implies $x \ll z$;

(v) $x \ll y$ implies $x \sqsubseteq y$;

(vi) $x \ll x$ iff $\{z \in D : x \sqsubseteq z\}$ is open;

(vii) if $S \subseteq D$ is directed, then $x \ll \bigsqcup S$ iff $x \ll y$ for some $y \in S$. $\square$

The proofs of these statements can be safely left to the reader.

proposition_2_2.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow

@[expose] public section

namespace DomainTheory.ContinuousLattice

universe u

variable {D : Type u} [CompleteLattice D]

/-- **Scott 1972, Proposition 2.2.** In a complete lattice:

(i) `⊥ ≪ x`;

(ii) `x ≪ z` and `y ≪ z` imply `x ⊔ y ≪ z`;

(iii) `x ≪ y` and `y ≤ z` imply `x ≪ z`;

(iv) `x ≤ y` and `y ≪ z` imply `x ≪ z`;

(v) `x ≪ y` implies `x ≤ y`;

(vi) `x ≪ x` iff `Set.Ici x` is Scott-open;

(vii) for a nonempty directed `S`, `x ≪ sSup S` iff `x ≪ y` for some `y ∈ S`.

Proved from `bot_wayBelow`, `WayBelow.sup`, `WayBelow.trans_le`, `WayBelow.le_trans`,
`WayBelow.le`, `WayBelow.self_iff_scottOpen_Ici`, and `WayBelow.sSup_iff`. -/
theorem proposition_2_2 :
    (∀ x : D, (⊥ : D) ≪ x) ∧
    (∀ x y z : D, x ≪ z → y ≪ z → x ⊔ y ≪ z) ∧
    (∀ x y z : D, x ≪ y → y ≤ z → x ≪ z) ∧
    (∀ x y z : D, x ≤ y → y ≪ z → x ≪ z) ∧
    (∀ x y : D, x ≪ y → x ≤ y) ∧
    (∀ x : D, x ≪ x ↔ ScottOpen (Set.Ici x)) ∧
    (∀ (x : D) (S : Set D), S.Nonempty → DirectedOn (· ≤ ·) S →
      (x ≪ sSup S ↔ ∃ y ∈ S, x ≪ y)) :=
  ⟨bot_wayBelow,
    fun _ _ _ hx hy => WayBelow.sup hx hy,
    fun _ _ _ h hyz => WayBelow.trans_le h hyz,
    fun _ _ _ hxy h => WayBelow.le_trans hxy h,
    fun _ _ h => WayBelow.le h,
```

```
    fun _ => WayBelow.self_iff_scottOpen_Ici,
    fun _ _ hS hdir => WayBelow.sSup_iff hS hdir⟩

end DomainTheory.ContinuousLattice
```

1.3.4 Definition 2.3

2.3 Definition. A *continuous lattice* is a complete lattice D in which for every $y \in D$ we have:

$$y = \bigsqcup \{x \in D : x \ll y\}.$$

As an alternate definition we find:

ContinuousLattice.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
import DomainTheory.ContinuousLattice.WayBelow.directedOn_wayBelow

@[expose] public section

namespace DomainTheory

open scoped DomainTheory.ContinuousLattice

universe u

variable {D : Type u} [CompleteLattice D]

class ContinuousLattice (D : Type u) [CompleteLattice D] : Prop where
  out : letI : LE D :=
    (CompleteLattice.toCompleteSemilatticeInf (α := D)).toPartialOrder.toPreorder.toLE
    ∀ y : D, IsLUB {x | x ≪ y} y
  sSup_wayBelow : ∀ y : D, sSup {x | x ≪ y} = y := fun y => (out y).sSup_eq

open ContinuousLattice

theorem ContinuousLattice.WayBelow.interpolate (hD : ContinuousLattice D) {a c : D} (
    hac : a ≪ c) :
    ∃ b, a ≪ b ∧ b ≪ c := by
  set M : Set D := {m | ∃ x, m ≪ x ∧ x ≪ c} with hM
  have hMdir : DirectedOn (· ≤ ·) M := by
    rintro m₁ ⟨x₁, hm₁x₁, hx₁c⟩ m₂ ⟨x₂, hm₂x₂, hx₂c⟩
    obtain ⟨x₃, hx₃c, hx₁₃, hx₂₃⟩ := directedOn_wayBelow c x₁ hx₁c x₂ hx₂c
    obtain ⟨m₃, hm₃x₃, hm₁m₃, hm₂m₃⟩ :=
      directedOn_wayBelow x₃ m₁ (hm₁x₁.trans_le hx₁₃) m₂ (hm₂x₂.trans_le hx₂₃)
    exact ⟨m₃, ⟨x₃, hm₃x₃, hx₃c⟩, hm₁m₃, hm₂m₃⟩
  have hMne : M.Nonempty := ⟨⊥, a, bot_wayBelow a, hac⟩
  have hsupM : sSup M = c := by
    refine le_antisymm (sSup_le ?_) ?_
    · rintro m ⟨x, hmx, hxc⟩
      exact hmx.le.trans hxc.le
    · rw [← hD.sSup_wayBelow c]
```

```
      refine sSup_le fun x hxc => ?_
      rw [← hD.sSup_wayBelow x]
      exact sSup_le fun m hmx => le_sSup ⟨x, hmx, hxc⟩
  rw [← hsupM] at hac
  obtain ⟨m, ⟨x, hmx, hxc⟩, ham⟩ := (WayBelow.sSup_iff hMne hMdir).1 hac
  exact ⟨x, ham.trans_le hmx.le, hxc⟩

end DomainTheory
```

1.3.5 Proposition 2.4

2.4 Proposition. A complete lattice D is continuous iff for every $y \in D$ we have:

$$y = \bigsqcup \{ \sqcap U : y \in U \},$$

where U ranges over the open subsets of D.

Proof: Suppose D is continuous. If $y \in D$ and $x \ll y$, then let $U = \{\text{Int} z : x \sqsubseteq z\}$, an open set. Now $y \in U$ by definition, and $\{U \subseteq z : x \sqsubseteq z\}$. Thus,

$$x \sqsubseteq \sqcap U \sqsubseteq y.$$

It easily follows by lattice theory that the equation of 2.3 implies that of 2.4.

In the converse direction we have only to note that if U is open and $y \in U$, then $\sqcap U \ll y$. The implication from 2.4 to 2.3 results at once. □

proposition_2_4.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.WayBelow.
    isContinuousLattice_iff_isLUB_sInf_nhds

@[expose] public section

namespace DomainTheory.ContinuousLattice

universe u

variable {D : Type u} [CompleteLattice D]

/-- **Scott 1972, Proposition 2.4.** A complete lattice is continuous iff every
    element is the
supremum of the infima of its open neighbourhoods. Synonym of
`isContinuousLattice_iff_isLUB_sInf_nhds`. -/
theorem proposition_2_4 :
    ContinuousLattice D ↔
      ∀ y : D, IsLUB {a : D | ∃ U, ScottOpen U ∧ y ∈ U ∧ a = sInf U} y :=
  isContinuousLattice_iff_isLUB_sInf_nhds

end DomainTheory.ContinuousLattice
```

1.3.6 Limits and convergence

What is the idea of this definition? A continuous lattice is more special than a complete lattice: not only are lubs to be limits but every element must be a limit from below. This rather rough remark can be made more precise. In any complete lattice D define the *principal limit* of a net $x : I \to D$ by the formula:

$$\lim\langle x_i : i \in I\rangle = \bigsqcup\{\sqcap x_j : j \geq i : i \in I\}.$$

Then specify that x converges to $y \in D$ iff

$$y \sqsubseteq \lim\langle\{x_i : i \in I\rangle\}.$$

Having a notion of convergence, we can then say that $U \subseteq D$ is open iff every net converging to an element of U is eventually in U. This gives nothing more than what we have called the induced topology above, as is easily checked. But now being in possession of a topology, we can redefine convergence in the usual way. Question: when do the two notions of convergence agree? Answer: if and only if D is a continuous lattice.

For obviously by construction the limit definition of convergence implies the topological. Now if D is a continuous lattice and x converges to y topologically, consider an open $U \subseteq D$ with $y \in U$. For some $i \in I$ we shall have $x_j \in U$ for all $j \geq i$. Therefore

$$\sqcap U \sqsubseteq \{\sqcap x_j : j \geq i \sqsubseteq \lim\langle x_i : i \in I\rangle\}.$$

From the formula of 2.4 it at once follows that $y \sqsubseteq \lim\langle\{x_i : i \in I\rangle\}$. Thus, in continuous lattices, we have shown that the two notions of convergence are the same.

Finally, suppose that the two notions coincide for a complete lattice D. Define a directed set $I = (U, z) : y, z \in U$, where z ranges over D and U over open subsets of D. This set is directed by the relation: $(U, z) \leq (V, w)$ iff $U \supseteq V$. Let $x : I \to D$ be given by: $x(U, z) = z$. Then x is a net converging to y topologically. But

$$\lim\langle x_i : i \in I\rangle = \bigsqcup\{\sqcap U : y \in U\}.$$

In this way we see that the assumption about the two styles of convergence implies that D is a continuous lattice in view of 2.4.

In T_0-spaces continuous functions are always monotonic (i.e. $\sqsubseteq$-preserving). For continuous lattices, by virtue of the remarks we have just made about limits, we can define the continuity of $f : D \to D'$ to mean that

$$f(\lim\langle\{x_i : i \in I\rangle) \sqsubseteq \overset{\prime}{\lim}\langle f(x_i) : i \in I\rangle\}$$

for all nets $x : I \to D$. This is all very fine, but general limits are messy to work with; we shall find it easier to state results in terms of lubs as in 2.5–2.7 below.

Before going any deeper, however, we should clear up another point about topologies. Suppose that D is any T_0-space which becomes a complete lattice under its induced partial ordering. Then it is evident from our definitions that every set open in the given topology is also open in the topology induced from the lattice structure. Question: when do the two topologies agree? Answer: a sufficient condition is that the equation:

$$y = \bigsqcup\{\sqcap U : y \in U\}$$

hold for all $y \in D$, where U ranges over the given open sets. Because in that case if V is open in the lattice sense and $y \in V$, then $\sqcap U \in V$ for some set U, open in the given sense, where $y \in U$. But $U \subseteq V$ follows, and so V is a union of given open sets and is itself open in the given topology. Of course this equation implies that D is a continuous lattice by virtue of 2.4. Notice that by the same token the sets of the form $\{y \in D : x \ll y\}$ will form a basis for the open sets of a continuous lattice.

hold for all $y \in D$, where U ranges over the given open sets. Because in that case if V is open in the lattice sense and $y \in V$, then $\sqcap U \in V$ for some set U, open in the given sense, where $y \in U$. But $U \subseteq V$ follows, and so V is a union of given open sets and is itself open in the given topology. Of course this equation implies that D is a continuous lattice by virtue of 2.4. Notice that by the same token the sets of the form $\{y \in D : x \ll y\}$ will form a basis for the open sets of a continuous lattice.

1.3.7 Scott maps

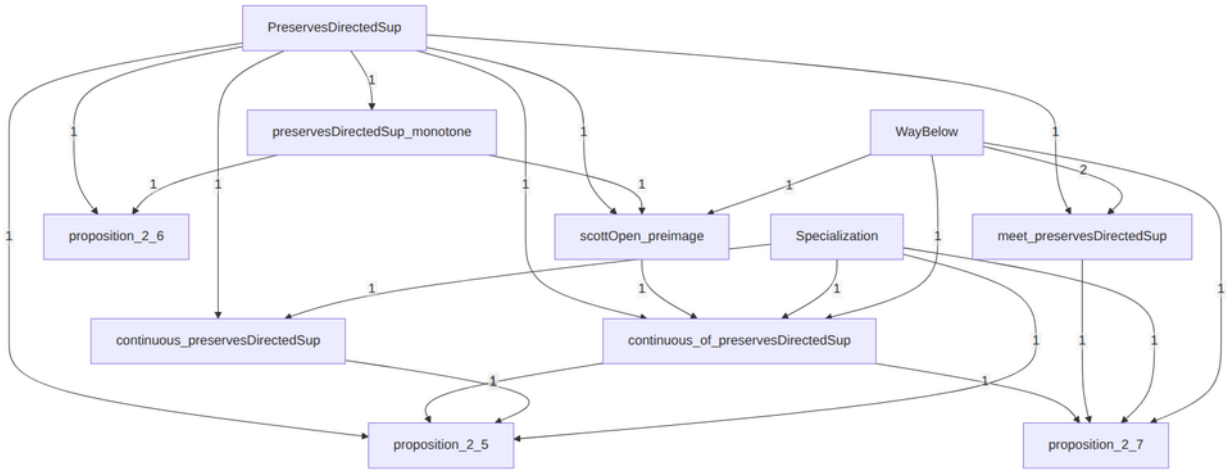


Figure 6: Scott maps.

Proposition 2.5 **2.5 Proposition.** *If D and D' are complete lattices with their induced topologies, then a function $f : D \to D'$ is continuous iff for all directed subsets $S \subseteq D$:*

$$f(\bigsqcup S) = \bigsqcup \{f(x) : x \in S\}.$$

Proof: If $f : D \to D'$ is continuous, the equation follows from the definition of continuous function and the fact that lubs are limits. Assume then that the equation holds for all directed sets S. Let $U' \subseteq D'$ be open in D' and let

$$U = \{x \in D : f(x) \in U'\}.$$

We must show that U is open in D. Note first that if $x \sqsubseteq y$, then $S = x, y$ is directed; hence,

$$f(x \sqcup y) = f(y) = f(x) \sqcup f(y),$$

so $f(x) \sqsubseteq f(y)$. Thus f is monotonic and so U satisfies condition (i). That U satisfies condition (ii) follows at once from the above equation. □

PreservesDirectedSup.lean

```
module
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology

variable {D D' D'' : Type*} [CompleteLattice D] [CompleteLattice D'] [CompleteLattice
    D'']

def PreservesDirectedSup (f : D → D') : Prop :=
  ∀ {{S : Set D}}, S.Nonempty → DirectedOn (· ≤ ·) S → f (sSup S) = sSup (f '' S)

end DomainTheory.ContinuousLattice
```

continuous_of_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.Specialization.scottTopologicalSpace
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.scottOpen_preimage
public import DomainTheory.ContinuousLattice.WayBelow.ScottOpen
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology

variable {D D' D'' : Type*} [CompleteLattice D] [CompleteLattice D'] [CompleteLattice
    D'']

theorem continuous_of_preservesDirectedSup {f : D → D'} (hf : PreservesDirectedSup f)
    :
    @Continuous D D' scottTopologicalSpace scottTopologicalSpace f := by
  rw [continuous_def]
  intro U hU
  rw [ScottOpen.isOpen_iff] at hU ⊢
  exact scottOpen_preimage hf hU
```

```
end DomainTheory.ContinuousLattice
```

continuous_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.Specialization.scottTopologicalSpace
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology

variable {D D' D'' : Type*} [CompleteLattice D] [CompleteLattice D'] [CompleteLattice
    D'']

theorem continuous_preservesDirectedSup {f : D → D'}
    (hf : @Continuous D D' scottTopologicalSpace scottTopologicalSpace f) :
    PreservesDirectedSup f := by
  have hD : @IsScott D univ _ (Topology.scott D univ) :=
    @IsScott.mk D univ _ (Topology.scott D univ) rfl
  have hD' : @IsScott D' univ _ (Topology.scott D' univ) :=
    @IsScott.mk D' univ _ (Topology.scott D' univ) rfl
  have hsc : ScottContinuous f :=
    scottContinuousOn_univ.1 <|
      (@Topology.IsScott.scottContinuousOn_iff_continuous D D' _
        (Topology.scott D univ) _ (Topology.scott D' univ) hD' f univ hD
        (fun _ _ _ => trivial)).2 hf
  intro S hS hSdir
  exact (hsc hS hSdir (isLUB_sSup S)).sSup_eq.symm

end DomainTheory.ContinuousLattice
```

preservesDirectedSup_monotone.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology

variable {D D' D'' : Type*} [CompleteLattice D] [CompleteLattice D'] [CompleteLattice
    D'']
```

```
theorem preservesDirectedSup_monotone {f : D → D'} (hf : PreservesDirectedSup f) :
    Monotone f := by
  intro x y hxy
  have hdir : DirectedOn (· ≤ ·) ({x, y} : Set D) := directedOn_pair hxy
  have hS : ({x, y} : Set D).Nonempty := ⟨x, Set.mem_insert _ _⟩
  have hsup : sSup ({x, y} : Set D) = y := by
    calc sSup ({x, y} : Set D) = x ⊔ y := sSup_pair
      _ = y := by apply le_antisymm; exact sup_le hxy le_rfl; exact le_sup_right
  have heq := hf hS hdir
  rw [hsup, Set.image_pair] at heq
  exact le_trans (le_sSup (Set.mem_insert _ _)) heq.symm.le

end DomainTheory.ContinuousLattice
```

proposition_2_5.lean

```
module

public import DomainTheory.ContinuousLattice.Specialization.scottTopologicalSpace
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.continuous_preservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology

variable {D D' D'' : Type*} [CompleteLattice D] [CompleteLattice D'] [CompleteLattice
    D'']

/-- **Scott 1972, Proposition 2.5.** Scott continuity ↔ preservation of directed
    suprema. -/
theorem proposition_2_5 (f : D → D') :
    (@Continuous D D' scottTopologicalSpace scottTopologicalSpace f) ↔
      PreservesDirectedSup f :=
  ⟨continuous_preservesDirectedSup, fun hf => continuous_of_preservesDirectedSup hf⟩

end DomainTheory.ContinuousLattice
```

scottOpen_preimage.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ScottOpen
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.preservesDirectedSup_monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section
```

```
namespace DomainTheory.ContinuousLattice

open Set Topology

variable {D D' D'' : Type*} [CompleteLattice D] [CompleteLattice D'] [CompleteLattice
    D'']

theorem scottOpen_preimage {f : D → D'} (hf : PreservesDirectedSup f) {U : Set D'}
    (hU : ScottOpen U) : ScottOpen (f ^-1' U) := by
  have hmono := preservesDirectedSup_monotone hf
  refine ⟨fun a b hab ha => hU.1 (hmono hab) ha, fun S hS hSdir hmem => ?_⟩
  rw [Set.mem_preimage] at hmem
  have hmem' : sSup (f '' S) ∈ U := by rw [← hf hS hSdir]; exact hmem
  obtain ⟨s, hsS, hsU⟩ := hU.2 (Set.image_nonempty.2 hS)
    (fun s hs t ht => by
      obtain ⟨a, haS, rfl⟩ := hs
      obtain ⟨b, hbS, rfl⟩ := ht
      obtain ⟨c, hcS, hac, hbc⟩ := hSdir a haS b hbS
      exact ⟨f c, Set.mem_image_of_mem f hcS, hmono hac, hmono hbc⟩) hmem'
  obtain ⟨a, haS, rfl⟩ := hsS
  exact ⟨a, haS, Set.mem_preimage.2 hsU⟩

end DomainTheory.ContinuousLattice
```

Proposition 2.6 **2.6 Proposition.** *With functions from complete lattices to complete lattices, a function of several variables is continuous in the variables jointly iff it is continuous in the variables separately.*

Proof: It will be sufficient to discuss functions of two variables. The product $D \times D'$ of two complete lattices is a complete lattice, and it is easy to check that the induced topology is the product topology. Since projection is continuous, joint continuity implies separate continuity. To check the converse suppose that

$$f : D \times D' \to D''$$

is a map where the separate continuity holds as follows:

$$f(\bigsqcup S, y) = \bigsqcup f(x, y) : x \in S$$

and

$$f(x, \bigsqcup S') = \bigsqcup \{f(x, y) : y \in S'\}$$

where $S \subseteq D$ and $S' \subseteq D'$ are directed and $x \in D$ and $y \in D'$. Let now

$$S^* \subseteq D \times D'$$

be directed in the product. The projection of S^* to D and of S^* to D' produces directed subsets $S \subseteq D$ and $S' \subseteq D'$. Note that

$$\bigsqcup S^* = (\bigsqcup S, \bigsqcup S').$$

Thus by assumption

$$f(\bigsqcup S^*) = \bigsqcup\{f(x,y) : x \in S, y \in S'\}.$$

But since S^* is directed, $x \in S$ and $y \in S'$ implies $x \sqsubseteq u$ and $y \sqsubseteq v$ for $(u,v) \in S^*$. Thus by monotonicity of f we can show

$$f(\bigsqcup S^*) = \bigsqcup\{f(u,v) : (u,v) \in S^*\}$$

and that gives the joint continuity. □

One of the justifications (by euphony at least) of the term *continuous lattice* is the fact that such spaces allow for so many continuous functions. One indication of this is the result:

proposition_2_6.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.preservesDirectedSup_monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology

variable {D D' D'' : Type*} [CompleteLattice D] [CompleteLattice D'] [CompleteLattice
    D'']

/-- **Scott 1972, Proposition 2.6.** A function of several variables between complete
    lattices is
(Scott-)continuous jointly iff it is continuous in each variable separately.
    Continuity is phrased
as preservation of directed suprema (Proposition 2.5); it suffices to treat two
    variables, the
product `D × D'` carrying the componentwise complete-lattice structure (whose induced
    topology is
the product topology). -/
theorem proposition_2_6 (f : D × D' → D'') :
    PreservesDirectedSup f ↔
      (∀ y, PreservesDirectedSup fun x => f (x, y)) ∧
        (∀ x, PreservesDirectedSup fun y => f (x, y)) := by
  constructor
  · -- joint continuity ⟹ separate continuity (precompose with the continuous slice
    maps)
    intro hf
    refine ⟨fun y S hS hSdir => ?_, fun x S hS hSdir => ?_⟩
    · set T : Set (D × D') := (fun x => (x, y)) '' S with hT
      have hTne : T.Nonempty := hS.image _
      have hTdir : DirectedOn (· ≤ ·) T := by
        rintro _ ⟨a, ha, rfl⟩ _ ⟨b, hb, rfl⟩
        obtain ⟨c, hc, hac, hbc⟩ := hSdir a ha b hb
        exact ⟨(c, y), Set.mem_image_of_mem _ hc, ⟨hac, le_rfl⟩, ⟨hbc, le_rfl⟩⟩
      have hfst : Prod.fst '' T = S := by rw [hT, Set.image_image]; simp
```

```
    have hsnd : Prod.snd '' T = {y} := by rw [hT, Set.image_image]; exact hS.
  image_const y
    have hsupT : sSup T = (sSup S, y) := by
      have e1 : (sSup T).1 = sSup S := by rw [Prod.fst_sSup, hfst]
      have e2 : (sSup T).2 = y := by rw [Prod.snd_sSup, hsnd, sSup_singleton]
      exact Prod.ext_iff.mpr ⟨e1, e2⟩
    have h := hf hTne hTdir
    rw [hsupT, hT, Set.image_image] at h
    simpa using h
  · set T : Set (D × D') := (fun y => (x, y)) '' S with hT
    have hTne : T.Nonempty := hS.image _
    have hTdir : DirectedOn (· ≤ ·) T := by
      rintro _ ⟨a, ha, rfl⟩ _ ⟨b, hb, rfl⟩
      obtain ⟨c, hc, hac, hbc⟩ := hSdir a ha b hb
      exact ⟨(x, c), Set.mem_image_of_mem _ hc, ⟨le_rfl, hac⟩, ⟨le_rfl, hbc⟩⟩
    have hfst : Prod.fst '' T = {x} := by rw [hT, Set.image_image]; exact hS.
  image_const x
    have hsnd : Prod.snd '' T = S := by rw [hT, Set.image_image]; simp
    have hsupT : sSup T = (x, sSup S) := by
      have e1 : (sSup T).1 = x := by rw [Prod.fst_sSup, hfst, sSup_singleton]
      have e2 : (sSup T).2 = sSup S := by rw [Prod.snd_sSup, hsnd]
      exact Prod.ext_iff.mpr ⟨e1, e2⟩
    have h := hf hTne hTdir
    rw [hsupT, hT, Set.image_image] at h
    simpa using h
· -- separate continuity ⟹ joint continuity (Scott 1972's directedness argument)
  rintro ⟨h1, h2⟩ Sstar hne hdir
  have hmono1 : ∀ y, Monotone fun x => f (x, y) := fun y =>
  preservesDirectedSup_monotone (h1 y)
  have hmono2 : ∀ x, Monotone fun y => f (x, y) := fun x =>
  preservesDirectedSup_monotone (h2 x)
  have hmono : Monotone f := by
    rintro ⟨a, b⟩ ⟨c, d⟩ hpq
    exact le_trans (hmono1 b hpq.1) (hmono2 c hpq.2)
  have hSne : (Prod.fst '' Sstar).Nonempty := hne.image _
  have hS'ne : (Prod.snd '' Sstar).Nonempty := hne.image _
  have hSdir : DirectedOn (· ≤ ·) (Prod.fst '' Sstar) := hdir.fst
  have hS'dir : DirectedOn (· ≤ ·) (Prod.snd '' Sstar) := hdir.snd
  have hsupStar : sSup Sstar = (sSup (Prod.fst '' Sstar), sSup (Prod.snd '' Sstar))
  :=
    Prod.ext_iff.mpr ⟨Prod.fst_sSup _, Prod.snd_sSup _⟩
  apply le_antisymm
  · rw [hsupStar]
    have e1 : f (sSup (Prod.fst '' Sstar), sSup (Prod.snd '' Sstar))
        = sSup ((fun x => f (x, sSup (Prod.snd '' Sstar))) '' (Prod.fst '' Sstar))
  :=
      h1 (sSup (Prod.snd '' Sstar)) hSne hSdir
    rw [e1]
    apply sSup_le
    rintro _ ⟨x, hx, rfl⟩
    show f (x, sSup (Prod.snd '' Sstar)) ≤ sSup (f '' Sstar)
    have e2 : f (x, sSup (Prod.snd '' Sstar))
        = sSup ((fun y => f (x, y)) '' (Prod.snd '' Sstar)) :=
      h2 x hS'ne hS'dir
    rw [e2]
    apply sSup_le
```

```
      rintro _ ⟨y, hy, rfl⟩
      show f (x, y) ≤ sSup (f '' Sstar)
      obtain ⟨p, hpS, hpfst⟩ := hx
      obtain ⟨q, hqS, hqsnd⟩ := hy
      obtain ⟨r, hrS, hpr, hqr⟩ := hdir p hpS q hqS
      have hxr : x ≤ r.1 := hpfst ▸ hpr.1
      have hyr : y ≤ r.2 := hqsnd ▸ hqr.2
      calc f (x, y) ≤ f (r.1, r.2) := hmono ⟨hxr, hyr⟩
        _ = f r := by rw [Prod.mk.eta]
        _ ≤ sSup (f '' Sstar) := le_sSup (Set.mem_image_of_mem f hrS)
    · apply sSup_le
      rintro _ ⟨p, hp, rfl⟩
      exact hmono (le_sSup hp)

end DomainTheory.ContinuousLattice
```

Proposition 2.7 **2.7 Proposition.** *In a continuous lattice D the finitary lattice operations $\sqcup$ and $\sqcap$ are continuous.*

Proof: It is trivial to show that $\sqcup$ is continuous in every complete lattice; this is not so for $\sqcap$. In view of 2.6 we need only show

$$x \sqcap \bigsqcup S = \bigsqcup \{x \sqcap y : y \in S\}$$

for every directed $S \subseteq D$. In fact it is enough to show

$$x \sqcap \bigsqcup S \sqsubseteq \bigsqcup \{x \sqcap y : y \in S\}$$

because the opposite inequality is valid in all complete lattices. In view of the fact that D is continuous, it is enough to show that

$$t \ll x \sqcap \bigsqcup S \text{ implies } t \sqsubseteq \bigsqcup \{x \sqcap y : y \in S\}.$$

So assume $t \ll x \sqcap \bigsqcup S$. Then $t \sqsubseteq x \sqcap \bigsqcup S \sqsubseteq x$. Also $t \ll \bigsqcup S$ because $x \sqcap \bigsqcup S \sqsubseteq \bigsqcup S$. Thus $t \ll y$ for some $y \in S$ since the set

$$\{z \in D : t \ll z\}$$

is open. But then $t \sqsubseteq y$, and so $t \sqsubseteq x \sqcap y$, and the result follows. □

It is now time to provide some examples of continuous lattices.

meet_preservesDirectedSup.lean

```
module
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section
```

```
namespace DomainTheory.ContinuousLattice

open Set Topology

variable {D D' D'' : Type*} [CompleteLattice D] [CompleteLattice D'] [CompleteLattice
    D'']

theorem meet_preservesDirectedSup (x : D) (hD : ContinuousLattice D) :
    PreservesDirectedSup (fun z => x ⊓ z) := by
  intro S hS hSdir
  apply le_antisymm
  · have hle : sSup {t | t ≪ x ⊓ sSup S} ≤ sSup (Set.image (fun z => x ⊓ z) S) := by
      apply sSup_le
      intro t ht
      obtain ⟨w, hwS, ht_w⟩ := (WayBelow.sSup_iff hS hSdir).1 (ht.trans_le inf_le_right
    )
      have hmem : x ⊓ w ∈ Set.image (fun z : D => x ⊓ z) S :=
        Set.mem_image_of_mem (fun z => x ⊓ z) hwS
      have hbound : x ⊓ w ≤ sSup (Set.image (fun z => x ⊓ z) S) := le_sSup hmem
      exact (le_inf (ht.le.trans inf_le_left) ht_w.le).trans hbound
    dsimp only
    rw [← hD.sSup_wayBelow (x ⊓ sSup S)]
    exact hle
  · apply sSup_le
    intro z hz
    obtain ⟨w, hwS, rfl⟩ := hz
    exact inf_le_inf le_rfl (le_sSup hwS)

end DomainTheory.ContinuousLattice
```

proposition_2_7.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.Specialization.scottTopologicalSpace
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.meet_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology

variable {D D' D'' : Type*} [CompleteLattice D] [CompleteLattice D'] [CompleteLattice
    D'']

namespace proposition_2_7

theorem sup :
    @Continuous (D × D) D scottTopologicalSpace scottTopologicalSpace (fun p : D × D
```

```
    => p.1 ⊔ p.2) :=
  continuous_of_preservesDirectedSup <| by
    intro S hS hSdir
    simpa using (ScottContinuous.sup₂ (β := D) (d := S) hS hSdir (isLUB_sSup S)).
    sSup_eq.symm

theorem inf_left (hD : ContinuousLattice D) (y : D) :
    @Continuous D D scottTopologicalSpace scottTopologicalSpace (fun x => x ⊓ y) :=
  continuous_of_preservesDirectedSup <| by
    intro S' hS' hSdir'
    rw [show (fun x => x ⊓ y) = fun z => y ⊓ z from funext fun x => inf_comm x y]
    have h := meet_preservesDirectedSup y hD
    exact h (S := S') hS' hSdir'

theorem inf_right (hD : ContinuousLattice D) (x : D) :
    @Continuous D D scottTopologicalSpace scottTopologicalSpace (fun y => x ⊓ y) :=
  continuous_of_preservesDirectedSup (meet_preservesDirectedSup x hD)

end proposition_2_7

end DomainTheory.ContinuousLattice
```

1.3.8 Proposition 2.8

2.8 Proposition. A finite lattice is a continuous lattice. □

directedOn_finite_sSup_mem.lean

```
module
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Order.Preorder.Finite
public import Mathlib.Basic.Finite.Defs
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Topology.ContinuousMap.T0Sierpinski
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Sets.Opens

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

variable {D : Type*} [CompleteLattice D]

/-- A non-empty **finite** directed set attains its supremum: `⊔S ∈ S`. A maximal
    element of
`S` (which exists by finiteness) is, by directedness, the greatest element, hence the
```

```
supremum. This is the order-theoretic heart of "finite ⟹ continuous" (Scott 1972,
    2.8). -/
theorem directedOn_finite_sSup_mem {S : Set D} (hSfin : S.Finite) (hSne : S.Nonempty)
    (hSdir : DirectedOn (· ≤ ·) S) : sSup S ∈ S := by
  obtain ⟨m, hm⟩ := hSfin.exists_maximal hSne
  have hub : m ∈ upperBounds S := by
    intro s hs
    obtain ⟨c, hcS, hmc, hsc⟩ := hSdir m hm.1 s hs
    exact hsc.trans (hm.2 hcS hmc)
  have hlub : IsLUB S m := ⟨hub, fun b hb => hb hm.1⟩
  rw [hlub.sSup_eq]
  exact hm.1

end DomainTheory.ContinuousLattice
```

proposition_2_8.lean

```
module
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice

public import DomainTheory.ContinuousLattice.Constructions.directedOn_finite_sSup_mem
public import Mathlib.Data.Fintype.EquivFin
public import Mathlib.Basic.Finite.Defs
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

variable {D : Type*} [CompleteLattice D]

/-- **Scott 1972, Proposition 2.8.** A finite lattice is a continuous lattice. In a
    finite
lattice every principal up-set `Set.Ici y` is Scott-open (a non-empty directed set is
    finite,
so it attains its supremum), hence `y ≪ y`; therefore `y` is the supremum of `{x | x ≪
    y}`. -/
theorem proposition_2_8 [Finite D] : ContinuousLattice D := by
  refine { out := ?_ }
  intro y
  have hopen : ScottOpen (Set.Ici y) := by
    refine ⟨isUpperSet_Ici y, fun S hSne hSdir hmem => ?_⟩
    exact ⟨sSup S, directedOn_finite_sSup_mem (Set.toFinite S) hSne hSdir, hmem⟩
  have hyy : y ≪ y := WayBelow.self_iff_scottOpen_Ici.mpr hopen
  exact ⟨fun x hx => hx.le, fun b hb => hb hyy⟩

end DomainTheory.ContinuousLattice
```

1.3.9 Proposition 2.9

2.9 Proposition. The Cartesian product of any number of continuous lattices is a continuous lattice with the induced topology agreeing with the product topology.

1.3.10 Proposition 2.10

2.10 Proposition. A retract of a continuous lattice is a continuous lattice with the subspace topology agreeing with the induced topology.

It would seem that the continuous lattices are starting to sound suspiciously like the injective spaces. Indeed, if we can prove the following, the circle will be complete.

1.3.11 Proposition 2.11

2.11 Proposition. Every continuous lattice is an injective space under its induced topology.

1.3.12 Theorem 2.12

2.12 Theorem. The injective spaces are exactly the continuous lattices.

This theorem is an immediate consequence of the preceding results: an injective space is a retract of a power of $\mathbb{O}$. But $\mathbb{O}$ is a finite lattice $(\bot \sqsubseteq \top)$, and so the given space is a continuous lattice under its induced topology. On the other hand a continuous lattice is injective. It remains then to prove 2.9–2.11.

1.3.13 Proof of 2.9

Proof of 2.9

Let D_i for $i \in I$ be a system of continuous lattices. The product

$$D^* = \prod_{i \in I} D_i$$

is a complete lattice in the usual way and has its induced topology. Suppose $y \in D^*$ and let $i \in I$. Then $y_i \in D_i$. Since D_i is a continuous lattice

$$y_i = \bigsqcup \{x \in D_i : x \ll y_i\}.$$

For $x \in D_i$, let $[x]^i \in D^*$ be defined by:

$$[x]^i_j = \begin{cases} x & \text{if } i = j, \\ \bot & \text{if } i \neq j. \end{cases}$$

Note that since D_i is continuous we have:

$$[y_i]^i = \bigsqcup \{[x]^i : x \ll y_i\},$$

and

$$y = \bigsqcup \{[y_i]^i : i \in I\}.$$

It follows that

$$y = \bigsqcup \bigcap \{z : z_i \in U \text{ for all } i \in I,\ y_i \in U\},$$

where i ranges over I and U over the open subsets of D_i, because

$$[x]^i \sqsubseteq \bigcap \{z : z_i \in U\}, \quad \text{where } U = \{u \in D_i : x \ll u\}.$$

But the sets $\{z : z_i \in U\}$ are open in the product sense, and so

$$y = \bigsqcup \bigcap \{U : y \in U\},$$

where U ranges now over the open subsets of the product topology on D^*. By the remark following 2.4 we conclude that D^* is continuous with the lattice-induced topology being the product topology. □

proposition_2_9.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.Specialization.scottTopologicalSpace
public import DomainTheory.ContinuousLattice.Constructions.Basic
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import DomainTheory.ContinuousLattice.WayBelow.ScottOpen
public import DomainTheory.ContinuousLattice.WayBelow.exists_wayBelow_Ici_subset
public import DomainTheory.ContinuousLattice.Constructions.isContinuousLattice_prop
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

variable {D : Type*} [CompleteLattice D]

theorem proposition_2_9 {ι : Type*} (E : ι → Type*) [∀ i, CompleteLattice (E i)]
    (hE : ∀ i, ContinuousLattice (E i)) :
    ContinuousLattice (∀ i, E i) ∧
      (scottTopologicalSpace : TopologicalSpace (∀ i, E i)) =
        @Pi.topologicalSpace ι E (fun _ => scottTopologicalSpace) := by
  classical
  have hprod : ContinuousLattice (∀ i, E i) := by
    refine { out := ?_ }
    intro y
    refine ⟨fun x hx => hx.le, fun b hb => ?_⟩
    rw [Pi.le_def]
    intro i
    rw [← (hE i).sSup_wayBelow (y i)]
    apply sSup_le
    intro a ha
    set e : (∀ j, E j) := Function.update (⊥ : ∀ j, E j) i a with he
    have hei : e i = a := by rw [he]; exact Function.update_self i a _
    have hcyl : e ≪ y := by
```

```
    obtain ⟨U, hU, hyiU, hUsub⟩ := ha
    refine ⟨{z : ∀ j, E j | z i ∈ U}, ?_, hyiU, ?_⟩
    · refine ⟨fun z w hzw hz => hU.1 (hzw i) hz, fun S hSne hSdir hmem => ?_⟩
      rw [Set.mem_ofPred_eq, sSup_apply_eq_sSup_image] at hmem
      have hdir' : DirectedOn (· ≤ ·) (Function.eval i '' S) := by
        rintro _ ⟨f, hf, rfl⟩ _ ⟨g, hg, rfl⟩
        obtain ⟨h, hhS, hfh, hgh⟩ := hSdir f hf g hg
        exact ⟨Function.eval i h, ⟨h, hhS, rfl⟩, hfh i, hgh i⟩
      obtain ⟨t, htimg, htU⟩ := hU.2 (hSne.image _) hdir' hmem
      obtain ⟨f, hfS, rfl⟩ := htimg
      exact ⟨f, hfS, htU⟩
    · intro z hz
      rw [Set.mem_Ici, Pi.le_def]
      intro j
      rcases eq_or_ne j i with rfl | hji
      · rw [hei]; exact Set.mem_Ici.1 (hUsub hz)
      · rw [he, Function.update_of_ne hji]; exact bot_le
  have hle : e i ≤ b i := (hb hcyl) i
  rw [hei] at hle
  exact hle
refine ⟨hprod, ?_⟩
apply le_antisymm
· refine le_iInf fun i => ?_
  rw [← continuous_iff_le_induced]
  exact continuous_of_preservesDirectedSup (fun _ _ _ => sSup_apply_eq_sSup_image)
· intro U hU
  rw [ScottOpen.isOpen_iff] at hU
  rw [@isOpen_iff_forall_mem_open _ (@Pi.topologicalSpace ι E (fun _ =>
  scottTopologicalSpace))]
  intro z hz
  obtain ⟨a, haz, haIci⟩ := exists_wayBelow_Ici_subset hprod hU hz
  obtain ⟨F, hF⟩ := wayBelow_finite_support haz
  have hproj : ∀ i, a i ≪ z i := fun i => wayBelow_proj haz i
  choose V hVopen hzV hVsub using hproj
  refine ⟨⋂ i ∈ F, (fun w : ∀ j, E j => w i) ^-1' V i, ?_, ?_, ?_⟩
  · intro w hw
    rw [Set.mem_iInter₂] at hw
    refine haIci (Set.mem_Ici.2 fun j => ?_)
    by_cases hjF : j ∈ F
    · exact Set.mem_Ici.1 (hVsub j (hw j hjF))
    · rw [hF j hjF]; exact bot_le
  · refine @isOpen_biInter_finset (∀ j, E j) ι
      (@Pi.topologicalSpace ι E (fun _ => scottTopologicalSpace)) F
      (fun i => (fun w : ∀ j, E j => w i) ^-1' V i) (fun i _ => ?_)
    have hVi : @IsOpen (E i) scottTopologicalSpace (V i) := ScottOpen.isOpen_iff.mpr
   (hVopen i)
    have hind : @IsOpen _
        (TopologicalSpace.induced (fun w : ∀ j, E j => w i) scottTopologicalSpace)
        ((fun w : ∀ j, E j => w i) ^-1' V i) :=
      (isOpen_induced_iff (t := scottTopologicalSpace)).mpr ⟨V i, hVi, rfl⟩
    exact iInf_le
      (fun i => TopologicalSpace.induced (fun w : ∀ j, E j => w i)
  scottTopologicalSpace) i _ hind
  · rw [Set.mem_iInter₂]
    exact fun i _ => hzV i
```

```
namespace proposition_2_9

theorem sierpinskiPower (ι : Type*) : ContinuousLattice (ι → Prop) :=
  (proposition_2_9 (fun _ => Prop) (fun _ => isContinuousLattice_prop)).1

end proposition_2_9

end DomainTheory.ContinuousLattice
```

scottTopology.lean

```
module

public import DomainTheory.ContinuousLattice.Specialization.scottTopologicalSpace
public import DomainTheory.ContinuousLattice.Constructions.scottTopology_prop
public import DomainTheory.ContinuousLattice.Constructions.proposition_2_9
public import DomainTheory.ContinuousLattice.Constructions.isContinuousLattice_prop
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

variable {D : Type*} [CompleteLattice D]

namespace scottTopology

theorem sierpinskiPower (ι : Type*) :
    (scottTopologicalSpace : TopologicalSpace (ι → Prop)) =
      (inferInstance : TopologicalSpace (ι → Prop)) := by
  rw [(proposition_2_9 (fun _ => Prop) (fun _ => isContinuousLattice_prop)).2]
  show (@Pi.topologicalSpace ι (fun _ => Prop) (fun _ => scottTopologicalSpace)) =
    @Pi.topologicalSpace ι (fun _ => Prop) (fun _ => sierpinskiSpace)
  congr 1
  funext _
  exact scottTopology_prop

end scottTopology

end DomainTheory.ContinuousLattice
```

1.3.14 Proof of 2.10

Proof of 2.10

Let D' be a continuous lattice and let $D \subseteq D'$ be a subspace which is a retract. We have for a suitable $j : D' \to D$,

$$D = \{x \in D' : j(x) = x\},$$

where of course j is continuous.

First a note of warning: though D is a subspace it is not a sublattice; that is, the partial ordering on D is the restriction of that of D', but the lubs of D are not those of D'. We shall have to distinguish operations by adding a prime ($'$) for those of D'.

Suppose $x, y \in D$. Let $z' = x \sqcup y \in D'$ (join taken in D') and define $z = j(z') \in D$. Now $x \sqsubseteq z'$ and $y \sqsubseteq z'$, and j is monotonic, so $x \sqsubseteq z$ and $y \sqsubseteq z$. Suppose $x \sqsubseteq w$ and $y \sqsubseteq w$ with $w \in D$. Then in D' we have $x \sqcup y \sqsubseteq w$; so $z \sqsubseteq w$ also. Hence we have shown that $z = x \sqcup y$ in D.

To show that D has a least element $\bot$ (which may be larger than the $\bot' \in D'$), we need a well-known lemma about monotonic functions: Every monotonic function on a complete lattice into itself has a least fixed point. (Cf. Birkhoff (1970), p. 115.) In our case j is monotonic and

$$\bot = \sqcap \{x \in D' : j(x) \sqsubseteq x\}$$

is the desired element in D.

Thus D is at least a semilattice with $\bot$ and $\sqcup$. To show that D is a lattice we need to show that every directed $S \subseteq D$ has a lub in D. Now we know ($S' = S$, join taken in the ambient lattice D'):

$$\bigsqcup S' \in D',$$

and this is a limit of a monotone net. So by 2.1, and the continuity of j:

$$j(\bigsqcup S') = \bigsqcup \{j(x) : x \in S\} \quad = \bigsqcup S$$

in D. In this way we now know that D is a complete lattice. We must still show that D is continuous.

Suppose $y \in D$. In D' we can write:

$$y = \bigsqcup \{x \in D' : x \ll y'\}$$

and this is the limit of a monotone net. Thus

$$j(y) = y = \bigsqcup j(x) : x \ll y, x \in D',$$

where the lub is taken in D. Note that the sets

$$U = \{z \in D : x \ll z\}$$

are open in D for each $x \in D'$. Note too that if $z \in U$, then $x \sqsubseteq z$ and so $j(x) \sqsubseteq j(z) = z$. This means that

$$j(x) \sqsubseteq \sqcap U$$

in D. We can then write in D:

$$y = \bigsqcup \{\sqcap U : y \in U\}$$

where U ranges over the open subsets of D, and so the lattice is continuous by 2.4. Inasmuch as the open sets U just used were open in the subspace topology, it follows by the remark after 2.4 that the subspace and the lattice-induced topologies coincide. □

`incl_preservesDirectedSup.lean`

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeRetraction
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace IsContinuousLatticeRetraction

/-- The inclusion of a retraction preserves directed suprema (it is a Scott map). -/
theorem incl_preservesDirectedSup (R : IsContinuousLatticeRetraction D D') :
    PreservesDirectedSup (R.incl : D → D') :=
  fun _ hS hSdir => R.incl.preservesDirectedSup_coe _ hS hSdir

end IsContinuousLatticeRetraction
end DomainTheory.ContinuousLattice
```

proposition_2_10.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.WayBelow.ScottOpen
public import DomainTheory.ContinuousLattice.WayBelow.scottOpen_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_image_retr_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    retr_wayBelow_of_wayBelow_incl
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    image_retr_wayBelow_directed
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    image_retr_wayBelow_nonempty
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_preservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeRetraction
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import DomainTheory.ContinuousLattice.ScottMaps.scottOpen_preimage
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section
```

```
namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace proposition_2_10

theorem a (R : IsContinuousLatticeRetraction D D') (hD' : ContinuousLattice D') :
    ContinuousLattice D := by
  refine { out := ?_ }
  intro d
  refine ⟨fun x hx => hx.le, fun b hb => ?_⟩
  rw [← R.sSup_image_retr_wayBelow hD' d]
  exact sSup_le fun _ ⟨x', hx', hxeq⟩ => hxeq ▸ hb (R.retr_wayBelow_of_wayBelow_incl hx
    ')

theorem b (R : IsContinuousLatticeRetraction D D') (hD' : ContinuousLattice D') :
    (scottTopologicalSpace : TopologicalSpace D) =
      TopologicalSpace.induced (R.incl : D → D') scottTopologicalSpace := by
  apply le_antisymm
  · intro U hU
    rw [isOpen_induced_iff (t := scottTopologicalSpace)] at hU
    obtain ⟨V', hV'open, rfl⟩ := hU
    rw [ScottOpen.isOpen_iff] at hV'open ⊢
    exact scottOpen_preimage R.incl_preservesDirectedSup hV'open
  · intro U hU
    rw [ScottOpen.isOpen_iff] at hU
    rw [@isOpen_iff_forall_mem_open _
      (TopologicalSpace.induced (R.incl : D → D') scottTopologicalSpace)]
    intro d hd
    have hmem : sSup (Set.image (R.retr : D' → D) {x' | x' ≪ R.incl d}) ∈ U := by
      rw [R.sSup_image_retr_wayBelow hD' d]; exact hd
    obtain ⟨p, hp, hpU⟩ :=
      hU.2 (R.image_retr_wayBelow_nonempty d) (R.image_retr_wayBelow_directed d) hmem
    obtain ⟨x', hx', rfl⟩ := hp
    refine ⟨(R.incl : D → D') ^-1' {z' | x' ≪ z'}, fun z hz => ?_, ?_, hx'⟩
    · have hjle := R.retr.monotone (show x' ≤ R.incl z from (hz : x' ≪ R.incl z).le)
      rw [R.retr_incl z] at hjle
      exact hU.1 hjle hpU
    · rw [isOpen_induced_iff (t := scottTopologicalSpace)]
      exact ⟨{z' | x' ≪ z'}, ScottOpen.isOpen_iff.mpr (scottOpen_wayBelow x'), rfl⟩

end proposition_2_10

theorem proposition_2_10 (R : IsContinuousLatticeRetraction D D') (hD' :
    ContinuousLattice D') :
    ContinuousLattice D ∧
      (scottTopologicalSpace : TopologicalSpace D) =
        TopologicalSpace.induced (R.incl : D → D') scottTopologicalSpace :=
  ⟨proposition_2_10.a R hD', proposition_2_10.b R hD'⟩
```

```
end DomainTheory.ContinuousLattice
```

1.3.15 Proof of 2.11

Proof of 2.11: Let D be a continuous lattice with its induced topology, and let $X \subseteq Y$ be two T_0-spaces in the subspace relation. Suppose

$$f : X \to D$$

is continuous. Define

$$\bar{f} : Y \to D$$

by the formula:

$$\bar{f}(y) = \bigsqcup \{ \sqcap \{ f(x) : x \in X \cap U \} : y \in U \},$$

where U ranges over the open subsets of Y. We need to show that $\bar{f}$ extends f and that it is continuous.

First, the continuity: Suppose that $d \in D$ and $d \ll \bar{f}(y)$; that is, $\bar{f}(y)$ belongs to a typical basic open subset of D. Since D is continuous, we can also find

$$d \ll d' \ll \bar{f}(y)$$

with $d' \in D$. From the definition of $\bar{f}$ it follows that

$$d' \sqsubseteq \sqcap \{ f(x) : x \in X \cap U \}$$

for some open $U \subseteq Y$ with $y \in U$. Thus

$$d' \sqsubseteq \bar{f}(y')$$

for all $y' \in U$ by virtue of the definition of $\bar{f}$. Since $d \ll d'$, we have also

$$d \ll \bar{f}(y')$$

for all $y' \in U$; in other words, the inverse image of the open subset of D determined by d under $\bar{f}$ is indeed open in Y, and $\bar{f}$ is thus continuous.

Next, the extension property: Note that the relationship

$$\bar{f}(x') \sqsubseteq f(x')$$

for all $x' \in X$ comes directly out of the definition of $\bar{f}$. For the converse, suppose $d \ll f(x')$ where $d \in D$. By assumption $f : X \to D$ is continuous, so

$$d \ll f(x'')$$

for all $x'' \in X \cap U$ where U is a suitable open subset of Y with $x' \in U$. In particular we have:

$$d \sqsubseteq \sqcap \{ f(x'') : x'' \in X \cap U \},$$

and so $d \sqsubseteq \bar{f}(x')$. Since $d \ll f(x')$ always implies $d \sqsubseteq \bar{f}(x')$, we see that $f(x') \sqsubseteq \bar{f}(x')$ follows by the continuity of D, and the proof is complete. □

Basic.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.scottOpen_preimage
public import Mathlib.Data.Finset.Lattice.Basic
public import Mathlib.Data.Finset.Insert
public import Mathlib.Data.Finset.Defs
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

variable {D : Type*} [CompleteLattice D]

/-- Plugging a value into a fixed coordinate, `v ↦ Function.update z i v`, preserves
    directed
suprema: away from `i` the result is the constant `z j`, and at `i` it is the identity.
     -/
theorem update_preservesDirectedSup {ι : Type*} [DecidableEq ι] {E : ι → Type*}
    [∀ i, CompleteLattice (E i)] (z : ∀ i, E i) (i : ι) :
    PreservesDirectedSup (fun v : E i => Function.update z i v) := by
  intro T hTne _
  show Function.update z i (sSup T) = sSup ((fun v : E i => Function.update z i v) ''
    T)
  funext j
  rw [sSup_apply_eq_sSup_image, Set.image_image]
  rcases eq_or_ne j i with hji | hji
  · rw [hji, Function.update_self]
    have h : (fun v : E i => Function.eval i (Function.update z i v)) = id := by
      funext v; simp [Function.eval, Function.update_self]
    rw [h, Set.image_id]
  · rw [Function.update_of_ne hji]
    have h : (fun v : E i => Function.eval j (Function.update z i v)) = fun _ => z j
    := by
      funext v; simp [Function.eval, Function.update_of_ne hji]
    rw [h, hTne.image_const, sSup_singleton]

/-- A way-below relation in a product projects to each coordinate: if `a ≪ z` in `∏ᵢ E
    ᵢ` then
`aᵢ ≪ zᵢ`. The witnessing Scott-open neighbourhood of `zᵢ` is the preimage of a product
     Scott-open
witness under `v ↦ Function.update z i v`, which is Scott-open by `
    update_preservesDirectedSup`. -/
theorem wayBelow_proj {ι : Type*} {E : ι → Type*} [∀ i, CompleteLattice (E i)]
    {a z : ∀ i, E i} (h : a ≪ z) (i : ι) : a i ≪ z i := by
  classical
  obtain ⟨W, hW, hzW, hWsub⟩ := h
```

```
  refine ⟨(fun v : E i => Function.update z i v) ^-1' W, ?_, ?_, ?_⟩
  · exact scottOpen_preimage (update_preservesDirectedSup z i) hW
  · show Function.update z i (z i) ∈ W
    rw [Function.update_eq_self]; exact hzW
  · intro v hv
    have hav : a ≤ Function.update z i v := Set.mem_Ici.1 (hWsub hv)
    have := hav i
    rwa [Function.update_self] at this

/-- A way-below element of a product has finite support: if `a ≪ z` in `∏ᵢ Eᵢ` then `aⱼ
    = ⊥` for
all but finitely many `j`. The finite truncations `Z F = (fun j => if j ∈ F then z j
   else ⊥)` form a
directed family with supremum `z`; since `a ≪ z = ⊔_F Z F`, already `a ≤ Z F` for some
    finite `F`,
forcing `aⱼ = ⊥` off `F`. -/
theorem wayBelow_finite_support {ι : Type*} {E : ι → Type*} [∀ i, CompleteLattice (E i)
   ]
    {a z : ∀ i, E i} (h : a ≪ z) : ∃ F : Finset ι, ∀ j ∈ F, a j = ⊥ := by
  classical
  set Z : Finset ι → (∀ j, E j) := fun F j => if j ∈ F then z j else ⊥ with hZ
  set 𝒵 : Set (∀ j, E j) := Set.range Z with h𝒵
  have hmono : Monotone Z := by
    intro F G hFG
    rw [Pi.le_def]; intro j
    simp only [hZ]
    by_cases hjF : j ∈ F
    · rw [ite_eq_left hjF, ite_eq_left (hFG hjF)]
    · rw [ite_eq_right hjF]; exact bot_le
  have h𝒵ne : 𝒵.Nonempty := ⟨Z ∅, ∅, rfl⟩
  have h𝒵dir : DirectedOn (· ≤ ·) 𝒵 := by
    rintro _ ⟨F, rfl⟩ _ ⟨G, rfl⟩
    exact ⟨Z (F ∪ G), ⟨F ∪ G, rfl⟩, hmono Finset.subset_union_left,
      hmono Finset.subset_union_right⟩
  have hsup : sSup 𝒵 = z := by
    apply le_antisymm
    · apply sSup_le
      rintro _ ⟨F, rfl⟩
      rw [Pi.le_def]; intro j
      simp only [hZ]
      by_cases hjF : j ∈ F
      · rw [ite_eq_left hjF]
      · rw [ite_eq_right hjF]; exact bot_le
    · rw [Pi.le_def]; intro j
      rw [sSup_apply_eq_sSup_image]
      refine le_sSup ⟨Z {j}, ⟨{j}, rfl⟩, ?_⟩
      simp [hZ, Function.eval]
  rw [← hsup] at h
  obtain ⟨d, hd𝒵, had⟩ := (WayBelow.sSup_iff h𝒵ne h𝒵dir).1 h
  obtain ⟨F, rfl⟩ := hd𝒵
  refine ⟨F, fun j hjF => ?_⟩
  have hj := had.le j
  simp only [hZ, ite_eq_right hjF] at hj
  exact le_bot_iff.1 hj

end DomainTheory.ContinuousLattice
```

continuous_eq_sSup_openInfs.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.Specialization.scottTopologicalSpace
public import DomainTheory.ContinuousLattice.WayBelow.ScottOpen
public import DomainTheory.ContinuousLattice.WayBelow.scottOpen_wayBelow
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

variable {D : Type*} [CompleteLattice D]

section InjectiveExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] {E : Type*} [
    CompleteLattice E]

/-- For a continuous `f' : Y → E` into a continuous lattice `E` (with its Scott
    topology), the
value `f' y` is reconstructed as the supremum over open neighbourhoods `U ∋ y` of the
    meets
`⊓ f''(U)`. This is the order-theoretic content behind the maximality clause of
    Proposition 3.8:
the `≤` direction interpolates from below using `f' y = ⊔ {a | a ≪ f' y}` and the
    openness of each
`f'^-1(⇑a)`; the `≥` direction is immediate since `f' y ∈ f''(U)` whenever `y ∈ U`. -/
theorem continuous_eq_sSup_openInfs (hE : ContinuousLattice E) {f' : Y → E}
    (hf' : @Continuous Y E _ scottTopologicalSpace f') (y : Y) :
    f' y = sSup {d | ∃ U : Set Y, IsOpen U ∧ y ∈ U ∧ d = sInf (f' '' U)} := by
  apply le_antisymm
  · rw [← hE.sSup_wayBelow (f' y)]
    refine sSup_le fun a ha => ?_
    have hWopen : @IsOpen E scottTopologicalSpace {z : E | a ≪ z} :=
      ScottOpen.isOpen_iff.mpr (scottOpen_wayBelow a)
    have hpre : IsOpen (f' ^-1' {z : E | a ≪ z}) := continuous_def.mp hf' _ hWopen
    have hyU : y ∈ f' ^-1' {z : E | a ≪ z} := ha
    refine le_trans ?_ (le_sSup ⟨f' ^-1' {z : E | a ≪ z}, hpre, hyU, rfl⟩)
    refine le_sInf ?_
    rintro w ⟨z, hzU, rfl⟩
    exact (hzU : a ≪ f' z).le
  · refine sSup_le ?_
    rintro d ⟨U, _hUo, hyU, rfl⟩
    exact sInf_le ⟨y, hyU, rfl⟩

end InjectiveExtension
end DomainTheory.ContinuousLattice
```

scottExtend.lean

```
module
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

variable {D : Type*} [CompleteLattice D]

section InjectiveExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] {E : Type*} [
    CompleteLattice E]

/-- Scott's canonical extension of `f : X → E` along `e : X → Y` (Scott 1972, proof of
     2.11):
`y ↦ ⊔ { ⊓ f''(e^-1V) : V an open neighbourhood of y }`. No topology on `E` is needed
    to *state*
the operator -- it is purely order-theoretic. -/
def scottExtend (e : X → Y) (f : X → E) (y : Y) : E :=
  sSup {d | ∃ V : Set Y, IsOpen V ∧ y ∈ V ∧ d = sInf (f '' (e ^-1' V))}

end InjectiveExtension

end DomainTheory.ContinuousLattice
```

scottExtend_aux_directed.lean

```
module
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

variable {D : Type*} [CompleteLattice D]

section InjectiveExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] {E : Type*} [
    CompleteLattice E]

omit [TopologicalSpace X] in
/-- The defining family of `scottExtend` is directed: open neighbourhoods are closed
    under
```

```
intersection, and `⊓ f''(e^-1 ·)` is monotone in the neighbourhood (smaller set, larger
    inf). -/
theorem scottExtend_aux_directed (e : X → Y) (f : X → E) (y : Y) :
    DirectedOn (· ≤ ·) {d | ∃ V : Set Y, IsOpen V ∧ y ∈ V ∧ d = sInf (f '' (e ^-1' V))}
    := by
  rintro _ ⟨V₁, hV₁o, hyV₁, rfl⟩ _ ⟨V₂, hV₂o, hyV₂, rfl⟩
  refine ⟨sInf (f '' (e ^-1' (V₁ ∩ V₂))), ⟨V₁ ∩ V₂, hV₁o.inter hV₂o, ⟨hyV₁, hyV₂⟩, rfl⟩, ?
    _, ?_⟩
  · exact sInf_le_sInf (Set.image_mono (Set.preimage_mono Set.inter_subset_left))
  · exact sInf_le_sInf (Set.image_mono (Set.preimage_mono Set.inter_subset_right))

end InjectiveExtension
end DomainTheory.ContinuousLattice
```

scottExtend_aux_nonempty.lean

```
module
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

variable {D : Type*} [CompleteLattice D]

section InjectiveExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] {E : Type*} [
    CompleteLattice E]

omit [TopologicalSpace X] in
theorem scottExtend_aux_nonempty (e : X → Y) (f : X → E) (y : Y) :
    {d | ∃ V : Set Y, IsOpen V ∧ y ∈ V ∧ d = sInf (f '' (e ^-1' V))}.Nonempty :=
  ⟨_, Set.univ, isOpen_univ, Set.mem_univ y, rfl⟩

end InjectiveExtension
end DomainTheory.ContinuousLattice
```

scottExtend_continuous.lean

```
module
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice

public import DomainTheory.ContinuousLattice.Specialization.scottTopologicalSpace
public import DomainTheory.ContinuousLattice.Constructions.scottExtend
public import DomainTheory.ContinuousLattice.Constructions.scottExtend_aux_directed
public import DomainTheory.ContinuousLattice.Constructions.scottExtend_aux_nonempty
public import DomainTheory.ContinuousLattice.WayBelow.ScottOpen
public import DomainTheory.ContinuousLattice.WayBelow.exists_wayBelow_subset
public import Mathlib.Topology.Defs.Basic
```

```
@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

variable {D : Type*} [CompleteLattice D]

section InjectiveExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] {E : Type*} [
    CompleteLattice E]

omit [TopologicalSpace X] in
/-- The extension is Scott-continuous. For a Scott-open `U` and a point `y₀` with `g y₀
    ∈ U`, the
basis lemma gives `a ≪ g y₀` with `⇑a ⊆ U`; since `g y₀` is a directed supremum, `a ≪
    ⊓ f''(e^-1V)`
for some open `V ∋ y₀`, and that value is `≤ g y''` for every `y' ∈ V`, so `V ⊆ g^-1U`.
    -/
theorem scottExtend_continuous (hE : ContinuousLattice E) (e : X → Y) (f : X → E) :
    @Continuous Y E _ scottTopologicalSpace (scottExtend e f) := by
  let : TopologicalSpace E := scottTopologicalSpace
  rw [continuous_def]
  intro U hU
  rw [ScottOpen.isOpen_iff] at hU
  rw [isOpen_iff_forall_mem_open]
  intro y₀ hy₀
  have hgy₀U : scottExtend e f y₀ ∈ U := hy₀
  obtain ⟨a, haz, hasub⟩ := exists_wayBelow_subset hE hU hgy₀U
  obtain ⟨d, hd, had⟩ := (WayBelow.sSup_iff (scottExtend_aux_nonempty e f y₀)
    (scottExtend_aux_directed e f y₀)).1 haz
  obtain ⟨V, hVo, hy₀V, rfl⟩ := hd
  refine ⟨V, ?_, hVo, hy₀V⟩
  intro y' hy'V
  show scottExtend e f y' ∈ U
  refine hasub ?_
  show a ≪ scottExtend e f y'
  exact had.trans_le (le_sSup ⟨V, hVo, hy'V, rfl⟩)

end InjectiveExtension
end DomainTheory.ContinuousLattice
```

scottExtend_eq_of_continuous.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.Specialization.scottTopologicalSpace
public import DomainTheory.ContinuousLattice.Constructions.scottExtend
public import DomainTheory.ContinuousLattice.WayBelow.ScottOpen
public import DomainTheory.ContinuousLattice.WayBelow.scottOpen_wayBelow
public import Mathlib.Topology.Defs.Basic
```

```
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

variable {D : Type*} [CompleteLattice D]

section InjectiveExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] {E : Type*} [
    CompleteLattice E]

/-- The extension agrees with `f` on the (embedded) subspace. The `≤` direction is
    immediate
(`f x₀` is one of the values being met); the `≥` direction uses continuity of `E`: for
    each
`a ≪ f x₀` the Scott-open `⇑a` pulls back along the continuous `f` and, by the
    embedding, to an
open `V ⊆ Y` on whose `e`-preimage `f ≥ a`, so `a ≤ ⊓ f''(e^-1V) ≤ g(e x₀)`. -/
theorem scottExtend_eq_of_continuous (hE : ContinuousLattice E) (e : X → Y)
    (he : IsEmbedding e) (f : X → E) (hf : @Continuous X E _ scottTopologicalSpace f)
    (x₀ : X) :
    scottExtend e f (e x₀) = f x₀ := by
  apply le_antisymm
  · refine sSup_le ?_
    rintro d ⟨V, hVo, hex₀V, rfl⟩
    exact sInf_le ⟨x₀, hex₀V, rfl⟩
  · rw [← hE.sSup_wayBelow (f x₀)]
    refine sSup_le fun a ha => ?_
    have hWopen : @IsOpen E scottTopologicalSpace {z : E | a ≪ z} :=
      ScottOpen.isOpen_iff.mpr (scottOpen_wayBelow a)
    have hpre : IsOpen (f ^-1' {z : E | a ≪ z}) := continuous_def.mp hf _ hWopen
    rw [he.isInducing.isOpen_iff] at hpre
    obtain ⟨V, hVo, hVeq⟩ := hpre
    have hx₀V : x₀ ∈ e ^-1' V := by rw [hVeq]; exact ha
    refine le_trans ?_ (le_sSup ⟨V, hVo, hx₀V, rfl⟩)
    refine le_sInf ?_
    rintro w ⟨x, hxV, rfl⟩
    have hxW : x ∈ f ^-1' {z : E | a ≪ z} := by rw [← hVeq]; exact hxV
    exact (hxW : a ≪ f x).le

end InjectiveExtension
end DomainTheory.ContinuousLattice
```

scottExtend_maximal.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.Specialization.scottTopologicalSpace
public import DomainTheory.ContinuousLattice.Constructions.scottExtend
public import DomainTheory.ContinuousLattice.Constructions.continuous_eq_sSup_openInfs
```

```
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

variable {D : Type*} [CompleteLattice D]

section InjectiveExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] {E : Type*} [
    CompleteLattice E]

omit [TopologicalSpace X] in
/-- **Maximality clause of Scott 1972, Proposition 3.8.** Any continuous solution `f'` of
`f' ∘ e = f` lies below `scottExtend e f`. Following Scott: expand `f'` via
`continuous_eq_sSup_openInfs`, restrict each meet from `U` to the embedded subspace `e(X) ∩ U`
(only enlarging the meet), and recognize the result as a defining term of `scottExtend`. -/
theorem scottExtend_maximal (hE : ContinuousLattice E) (e : X → Y) {f : X → E} {f' :
    Y → E}
    (hf' : @Continuous Y E _ scottTopologicalSpace f') (h_ext : ∀ x, f' (e x) = f x) (y
     : Y) :
    f' y ≤ scottExtend e f y := by
  rw [continuous_eq_sSup_openInfs hE hf' y]
  refine sSup_le ?_
  rintro d ⟨U, hUo, hyU, rfl⟩
  refine le_trans ?_ (le_sSup ⟨U, hUo, hyU, rfl⟩)
  refine le_sInf ?_
  rintro w ⟨x, hxU, rfl⟩
  rw [← h_ext x]
  exact sInf_le ⟨e x, hxU, rfl⟩

end InjectiveExtension
end DomainTheory.ContinuousLattice
```

scottExtend_maximal_le.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.Specialization.scottTopologicalSpace
public import DomainTheory.ContinuousLattice.Constructions.scottExtend
public import DomainTheory.ContinuousLattice.Constructions.continuous_eq_sSup_openInfs
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice
```

```
open Topology Set

variable {D : Type*} [CompleteLattice D]

section InjectiveExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] {E : Type*} [
    CompleteLattice E]

omit [TopologicalSpace X] in
/-- **Scott 1972, remark following 3.8.** `scottExtend e g` is also the maximal *sub*-
    solution: any
continuous `f'` with `f' ∘ e ⊑ g` satisfies `f' ⊑ scottExtend e g`. Same proof as
`scottExtend_maximal`, replacing the final equality `f' (e x) = f x` by the inequality
`f' (e x) ≤ g x`. -/
theorem scottExtend_maximal_le (hE : ContinuousLattice E) (e : X → Y) {g : X → E} {f'
     : Y → E}
    (hf' : @Continuous Y E _ scottTopologicalSpace f') (h_le : ∀ x, f' (e x) ≤ g x) (y
     : Y) :
    f' y ≤ scottExtend e g y := by
  rw [continuous_eq_sSup_openInfs hE hf' y]
  refine sSup_le ?_
  rintro d ⟨U, hUo, hyU, rfl⟩
  refine le_trans ?_ (le_sSup ⟨U, hUo, hyU, rfl⟩)
  refine le_sInf ?_
  rintro w ⟨x, hxU, rfl⟩
  exact le_trans (sInf_le ⟨e x, hxU, rfl⟩) (h_le x)

end InjectiveExtension
end DomainTheory.ContinuousLattice
```

scottTopology_prop.lean

```
module

public import DomainTheory.ContinuousLattice.Specialization.scottTopologicalSpace
public import DomainTheory.ContinuousLattice.WayBelow.ScottOpen
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

variable {D : Type*} [CompleteLattice D]

/-- The Scott topology on Scott's two-point space `𝕆 = Prop` is exactly the Sierpinski
     topology
(`generateFrom {{True}}`). The Scott-open sets of `Prop` are the upper sets `∅`, `{True
    }`, `univ`,
which are precisely the Sierpinski-open sets. This is the topological identification
    underlying
```

```
Theorem 2.12: the building block `𝕆` carries its Scott topology. -/
theorem scottTopology_prop :
    (scottTopologicalSpace : TopologicalSpace Prop) = sierpinskiSpace := by
  apply le_antisymm
  · -- `scott ≤ sierpinski`: the single Sierpinski sub-basic open `{True}` is Scott-
    open.
    show (scottTopologicalSpace : TopologicalSpace Prop) ≤ TopologicalSpace.
    generateFrom {{True}}
    apply le_generateFrom
    intro s hs
    rw [Set.mem_singleton_iff] at hs
    subst hs
    rw [ScottOpen.isOpen_iff]
    refine ⟨?_, ?_⟩
    · intro a b hab ha
      rw [Set.mem_singleton_iff] at ha ⊢
      exact le_antisymm le_top (ha ▸ hab)
    · intro S _ _ hmem
      rw [Set.mem_singleton_iff] at hmem
      have hex : ∃ p ∈ S, p := by rw [← sSup_Prop_eq, hmem]; trivial
      obtain ⟨p, hpS, hp⟩ := hex
      exact ⟨p, hpS, by rw [Set.mem_singleton_iff]; exact eq_true hp⟩
  · -- `sierpinski ≤ scott`: every Scott-open upper set of `Prop` is `∅`, `{True}`, or `
    univ`.
    rw [TopologicalSpace.le_def]
    intro U hU
    rw [ScottOpen.isOpen_iff] at hU
    by_cases hT : True ∈ U
    · by_cases hF : False ∈ U
      · have hUuniv : U = Set.univ := by
          ext p
          simp only [Set.mem_univ, iff_true]
          by_cases hp : p
          · rwa [eq_true hp]
          · rwa [eq_false hp]
        rw [hUuniv]; exact isOpen_univ
      · have hUtrue : U = {True} := by
          ext p
          rw [Set.mem_singleton_iff]
          constructor
          · intro hpU
            by_cases hp : p
            · exact eq_true hp
            · exact absurd (eq_false hp ▸ hpU) hF
          · intro hp; rw [hp]; exact hT
        rw [hUtrue]; exact isOpen_singleton_true
    · have hUempty : U = ∅ := by
        ext p
        simp only [Set.mem_empty_iff_false, iff_false]
        intro hpU
        exact hT (hU.1 le_top hpU)
      rw [hUempty]; exact isOpen_empty

end DomainTheory.ContinuousLattice
```

proposition_2_11.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.Specialization.scottTopologicalSpace
public import DomainTheory.ContinuousLattice.Injective.InjectiveSpace
public import DomainTheory.ContinuousLattice.Constructions.scottExtend_continuous
public import DomainTheory.ContinuousLattice.Constructions.
    scottExtend_eq_of_continuous
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

variable {D : Type*} [CompleteLattice D]

/-- **Scott 1972, Proposition 2.11.** Every continuous lattice is an injective space
    under its
induced (Scott) topology. The witness is `scottExtend`, which extends any continuous `f
    ` along
any embedding `e` (`scottExtend_eq_of_continuous`) and is itself continuous
(`scottExtend_continuous`). -/
theorem proposition_2_11 {E : Type*} [CompleteLattice E] (hE : ContinuousLattice E) :
    @InjectiveSpace E scottTopologicalSpace := by
  let : TopologicalSpace E := scottTopologicalSpace
  constructor
  intro X Y _ _ e he f
  exact ⟨⟨scottExtend e f, scottExtend_continuous hE e f⟩,
    fun x => scottExtend_eq_of_continuous hE e he f f.continuous x⟩

end DomainTheory.ContinuousLattice
```

1.3.16 Proof of 2.12

isContinuousLattice_prop.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import Mathlib.Order.CompleteLattice.Basic
public import DomainTheory.ContinuousLattice.Constructions.proposition_2_8
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set
```

```
variable {D : Type*} [CompleteLattice D]

/-- The Sierpinski space `Prop` (Scott's `𝕆`) is a continuous lattice: it is a finite
    complete
lattice, so Proposition 2.8 applies. -/
theorem isContinuousLattice_prop : ContinuousLattice Prop :=
  proposition_2_8

end DomainTheory.ContinuousLattice
```

IdemFix.lean

```
module

public import Mathlib.Order.CompleteLattice.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

section FixedPoints

variable {L : Type*} [CompleteLattice L]

/-- The fixed-point set `{x | e x = x}` of an endomap `e`, as a subtype. -/
abbrev IdemFix (e : L → L) : Type _ := {x : L // e x = x}

end FixedPoints

end DomainTheory.ContinuousLattice
```

IsContinuousLatticeu2eorderIso.lean

```
module
public import DomainTheory.ContinuousLattice.ContinuousLattices.orderIso_wayBelow
public import Mathlib.Order.BourbakiWitt
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

universe u

section Theorem212Transfer

variable {A B : Type u} [latticeA : CompleteLattice A] [latticeB : CompleteLattice B]

/-- Continuous-lattice structure transfers across an order isomorphism. -/
```

```
theorem orderIso (e : A ≃o B) (hB : ContinuousLattice B) :
    ContinuousLattice A := by
  refine { out := ?_ }
  intro y
  refine ⟨fun x hx => hx.le, fun b hb => ?_⟩
  apply e.le_iff_le.mp
  exact (hB.out (e y)).2 fun z hz => by
    have hz' : e.symm z ≪ y := by
      simpa using orderIso_wayBelow e.symm hz
    have := hb hz'
    simpa using e.monotone this

end Theorem212Transfer
end DomainTheory.ContinuousLattice
```

coe_sSup.lean

```
module

public import DomainTheory.ContinuousLattice.ContinuousLattices.completeLattice
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

section FixedPoints

variable {L : Type*} [CompleteLattice L]

namespace IdemFix

variable {e : L → L}

/-- The ambient supremum corrected by `e` is the supremum in `IdemFix e`. -/
theorem coe_sSup (hidem : ∀ x, e (e x) = e x) (hmono : Monotone e) (S : Set (IdemFix e)
    ) :
    letI := completeLattice hidem hmono
    ((sSup S : IdemFix e) : L) = e (sSup (Subtype.val '' S)) := rfl

end IdemFix

end FixedPoints

end DomainTheory.ContinuousLattice
```

completeLattice.lean

```
module

public import DomainTheory.ContinuousLattice.ContinuousLattices.isLUB_sSup
public import Mathlib.Topology.Defs.Basic
```

```
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

section FixedPoints

variable {L : Type*} [CompleteLattice L]

namespace IdemFix

variable {e : L → L}

@[reducible] noncomputable def completeLattice (hidem : ∀ x, e (e x) = e x) (hmono :
    Monotone e) :
    CompleteLattice (IdemFix e) :=
  letI := supSet hidem
  completeLatticeOfSup (IdemFix e) (isLUB_sSup hidem hmono)

end IdemFix

end FixedPoints

end DomainTheory.ContinuousLattice
```

idemFixRetraction.lean

```
module

public import DomainTheory.ContinuousLattice.ContinuousLattices.completeLattice
public import DomainTheory.ContinuousLattice.ContinuousLattices.
    idemFix_retr_preservesDirectedSup
public import DomainTheory.ContinuousLattice.ContinuousLattices.
    idemFix_incl_preservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeRetraction
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.preservesDirectedSup_monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

section FixedPoints

variable {L : Type*} [CompleteLattice L]
```

```
variable {e : L → L}

/-- The fixed-point set of a Scott-continuous idempotent `e` on `L`, with the
ambient-supremum-corrected complete-lattice structure, is a *retract of `L`*: the
    inclusion and the
corestriction of `e` are Scott-continuous and compose to the identity. -/
noncomputable def idemFixRetraction (hidem : ∀ x, e (e x) = e x) (hsc :
    PreservesDirectedSup e) :
    @IsContinuousLatticeRetraction (IdemFix e) L
      (IdemFix.completeLattice hidem (preservesDirectedSup_monotone hsc)) _ :=
  letI := IdemFix.completeLattice hidem (preservesDirectedSup_monotone hsc)
  { incl := ⟨Subtype.val,
      continuous_of_preservesDirectedSup (idemFix_incl_preservesDirectedSup hidem hsc)⟩
    retr := ⟨fun x => ⟨e x, hidem x⟩,
      continuous_of_preservesDirectedSup (idemFix_retr_preservesDirectedSup hidem hsc)⟩
    retr_incl := fun k => Subtype.ext k.2 }

end FixedPoints

end DomainTheory.ContinuousLattice
```

idemFix_incl_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.ContinuousLattices.completeLattice
public import DomainTheory.ContinuousLattice.ScottMaps.preservesDirectedSup_monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

section FixedPoints

variable {L : Type*} [CompleteLattice L]

variable {e : L → L}

theorem idemFix_incl_preservesDirectedSup (hidem : ∀ x, e (e x) = e x)
    (hsc : PreservesDirectedSup e) :
    letI := IdemFix.completeLattice hidem (preservesDirectedSup_monotone hsc)
    PreservesDirectedSup (Subtype.val : IdemFix e → L) := by
  let := IdemFix.completeLattice hidem (preservesDirectedSup_monotone hsc)
  intro S hS hSdir
  show e (sSup (Subtype.val '' S)) = sSup (Subtype.val '' S)
  have hdir' : DirectedOn (· ≤ ·) (Subtype.val '' S) := by
    rintro _ ⟨a, ha, rfl⟩ _ ⟨b, hb, rfl⟩
    obtain ⟨c, hc, hac, hbc⟩ := hSdir a ha b hb
    exact ⟨c.val, Set.mem_image_of_mem _ hc, hac, hbc⟩
  have hne' : (Subtype.val '' S).Nonempty := hS.image _
  rw [hsc hne' hdir']
```

```
  congr 1
  rw [← Set.image_comp]
  exact Set.image_congr (fun s _ => s.2)

end FixedPoints

end DomainTheory.ContinuousLattice
```

idemFix_isContinuousLattice.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.ContinuousLattices.completeLattice
public import DomainTheory.ContinuousLattice.ContinuousLattices.idemFixRetraction
public import DomainTheory.ContinuousLattice.FunctionSpaces.proposition_2_10
public import DomainTheory.ContinuousLattice.ScottMaps.preservesDirectedSup_monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

section FixedPoints

variable {L : Type*} [CompleteLattice L]

variable {e : L → L}

/-- **Fixed points of a Scott-continuous idempotent form a continuous lattice.**
    Combine the
retraction `idemFixRetraction` with Proposition 2.10(a). -/
theorem idemFix_isContinuousLattice (hidem : ∀ x, e (e x) = e x) (hsc :
    PreservesDirectedSup e)
    (hL : ContinuousLattice L) :
    @ContinuousLattice (IdemFix e)
      (IdemFix.completeLattice hidem (preservesDirectedSup_monotone hsc)) := by
  let := IdemFix.completeLattice hidem (preservesDirectedSup_monotone hsc)
  exact proposition_2_10.a (idemFixRetraction hidem hsc) hL

end FixedPoints
end DomainTheory.ContinuousLattice
```

idemFix_retr_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.ContinuousLattices.completeLattice
public import DomainTheory.ContinuousLattice.ContinuousLattices.coe_sSup
public import DomainTheory.ContinuousLattice.ScottMaps.preservesDirectedSup_monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic
```

```
@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

section FixedPoints

variable {L : Type*} [CompleteLattice L]

variable {e : L → L}

theorem idemFix_retr_preservesDirectedSup (hidem : ∀ x, e (e x) = e x)
    (hsc : PreservesDirectedSup e) :
    letI := IdemFix.completeLattice hidem (preservesDirectedSup_monotone hsc)
    PreservesDirectedSup (fun x : L => (⟨e x, hidem x⟩ : IdemFix e)) := by
  let := IdemFix.completeLattice hidem (preservesDirectedSup_monotone hsc)
  have hmono := preservesDirectedSup_monotone hsc
  intro T hT hTdir
  apply Subtype.ext
  rw [IdemFix.coe_sSup hidem hmono]
  show e (sSup T) = e (sSup (Subtype.val '' ((fun x : L => (⟨e x, hidem x⟩ : IdemFix e)
    ) '' T)))
  have himg : Subtype.val '' ((fun x : L => (⟨e x, hidem x⟩ : IdemFix e)) '' T) = e ''
    T := by
    rw [← Set.image_comp]; rfl
  rw [himg]
  have hTdir' : DirectedOn (· ≤ ·) (e '' T) := by
    rintro _ ⟨a, ha, rfl⟩ _ ⟨b, hb, rfl⟩
    obtain ⟨c, hc, hac, hbc⟩ := hTdir a ha b hb
    exact ⟨e c, Set.mem_image_of_mem _ hc, hmono hac, hmono hbc⟩
  have hTne' : (e '' T).Nonempty := hT.image _
  rw [hsc hT hTdir, hsc hTne' hTdir']
  congr 1
  rw [← Set.image_comp]
  exact Set.image_congr (fun x _ => (hidem x).symm)

end FixedPoints

end DomainTheory.ContinuousLattice
```

idemFix_scottTopology.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.ContinuousLattices.completeLattice
public import DomainTheory.ContinuousLattice.ContinuousLattices.idemFixRetraction
public import DomainTheory.ContinuousLattice.FunctionSpaces.proposition_2_10
public import DomainTheory.ContinuousLattice.ScottMaps.preservesDirectedSup_monotone
public import DomainTheory.ContinuousLattice.Specialization.scottTopologicalSpace
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic
```

```
@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

section FixedPoints

variable {L : Type*} [CompleteLattice L]

variable {e : L → L}

/-- **Scott topology of the fixed-point lattice = subspace topology.** Proposition
    2.10(b) applied
to `idemFixRetraction`: the Scott topology of `IdemFix e` is the topology induced by
    the inclusion
from the Scott topology of `L`. -/
theorem idemFix_scottTopology (hidem : ∀ x, e (e x) = e x) (hsc : PreservesDirectedSup
    e)
    (hL : ContinuousLattice L) :
    @scottTopologicalSpace (IdemFix e)
        (IdemFix.completeLattice hidem (preservesDirectedSup_monotone hsc)) =
      TopologicalSpace.induced (Subtype.val : IdemFix e → L) scottTopologicalSpace :=
    by
  let := IdemFix.completeLattice hidem (preservesDirectedSup_monotone hsc)
  exact proposition_2_10.b (idemFixRetraction hidem hsc) hL

end FixedPoints
end DomainTheory.ContinuousLattice
```

isLUB_sSup.lean

```
module

public import DomainTheory.ContinuousLattice.ContinuousLattices.supSet
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

section FixedPoints

variable {L : Type*} [CompleteLattice L]

namespace IdemFix

variable {e : L → L}

/-- With the ambient-then-`e` supremum, every set has a least upper bound in `IdemFix e
    `. This
needs only that `e` is monotone (which follows from Scott continuity). -/
```

```
theorem isLUB_sSup (hidem : ∀ x, e (e x) = e x) (hmono : Monotone e) :
    letI := supSet hidem
    ∀ S : Set (IdemFix e), IsLUB S (sSup S) := by
  let := supSet hidem
  intro S
  constructor
  · intro k hk
    rw [Subtype.coe_le_coe.symm, Subtype.coe_mk]
    show (k : L) ≤ e (sSup (Subtype.val '' S))
    calc (k : L) = e k := k.2.symm
      _ ≤ e (sSup (Subtype.val '' S)) :=
          hmono (le_sSup (Set.mem_image_of_mem _ hk))
  · intro w hw
    rw [Subtype.coe_le_coe.symm, Subtype.coe_mk]
    show e (sSup (Subtype.val '' S)) ≤ (w : L)
    calc e (sSup (Subtype.val '' S)) ≤ e (w : L) := by
          refine hmono (sSup_le ?_)
          rintro _ ⟨k, hk, rfl⟩
          exact hw hk
      _ = (w : L) := w.2

end IdemFix

end FixedPoints

end DomainTheory.ContinuousLattice
```

orderIsoScottHomeomorph.lean

```
module
public import DomainTheory.ContinuousLattice.ContinuousLattices.
    scottOpen_preimage_orderIso
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Algebra.Group.End
public import DomainTheory.ContinuousLattice.WayBelow.ScottOpen
public import DomainTheory.ContinuousLattice.Specialization.scottTopologicalSpace
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

universe u

section Theorem212Transfer

variable {A B : Type u} [latticeA : CompleteLattice A] [latticeB : CompleteLattice B]

/-- An order isomorphism is a homeomorphism between the corresponding Scott topologies
    . -/
noncomputable def orderIsoScottHomeomorph (e : A ≃o B) :
    @Homeomorph A B (@scottTopologicalSpace A latticeA)
```

```
      (@scottTopologicalSpace B latticeB) := by
  letI : TopologicalSpace A := @scottTopologicalSpace A latticeA
  letI : TopologicalSpace B := @scottTopologicalSpace B latticeB
  letI : Preorder A := latticeA.toPreorder
  letI : Preorder B := latticeB.toPreorder
  exact {
  toEquiv := e.toEquiv
  continuous_toFun := by
    rw [continuous_def]
    intro U hU
    exact ScottOpen.isOpen_iff.mpr
      (scottOpen_preimage_orderIso e (ScottOpen.isOpen_iff.mp hU))
  continuous_invFun := by
    rw [continuous_def]
    intro U hU
    exact ScottOpen.isOpen_iff.mpr
      (scottOpen_preimage_orderIso e.symm (ScottOpen.isOpen_iff.mp hU))
  }

end Theorem212Transfer

end DomainTheory.ContinuousLattice
```

orderIso_wayBelow.lean

```
module
public import DomainTheory.ContinuousLattice.ContinuousLattices.
    scottOpen_preimage_orderIso
public import Mathlib.Order.BourbakiWitt
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

universe u

section Theorem212Transfer

variable {A B : Type u} [latticeA : CompleteLattice A] [latticeB : CompleteLattice B]

/-- Order isomorphisms preserve Scott's topological way-below relation. -/
theorem orderIso_wayBelow (e : A ≃o B) {x y : A} (h : x ≪ y) :
    e x ≪ e y := by
  obtain ⟨U, hU, hyU, hUsub⟩ := h
  refine ⟨e.symm ^-1' U, scottOpen_preimage_orderIso e.symm hU, ?_, ?_⟩
  · simpa using hyU
  · intro z hz
    have hxz : x ≤ e.symm z := Set.mem_Ici.1 (hUsub hz)
    exact Set.mem_Ici.2 (by simpa using (e.le_iff_le.mpr hxz))
```

```
end Theorem212Transfer
end DomainTheory.ContinuousLattice
```

scottOpen_preimage_orderIso.lean

```
module
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Data.Set.Basic
public import Mathlib.Topology.Defs.Basic
public import DomainTheory.ContinuousLattice.WayBelow.ScottOpen

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

universe u

section Theorem212Transfer

variable {A B : Type u} [latticeA : CompleteLattice A] [latticeB : CompleteLattice B]

/-- An order isomorphism pulls Scott-open sets back to Scott-open sets. -/
theorem scottOpen_preimage_orderIso (e : A ≃o B) {U : Set B} (hU : ScottOpen U) :
    ScottOpen (e ^-1' U) := by
  refine ⟨fun x y hxy hx => hU.1 (e.monotone hxy) hx, ?_⟩
  intro S hSne hSdir hsup
  have hImgNe : (e '' S).Nonempty := hSne.image e
  have hImgDir : DirectedOn (· ≤ ·) (e '' S) := by
    rintro _ ⟨x, hx, rfl⟩ _ ⟨y, hy, rfl⟩
    obtain ⟨z, hz, hxz, hyz⟩ := hSdir x hx y hy
    exact ⟨e z, Set.mem_image_of_mem e hz, e.monotone hxz, e.monotone hyz⟩
  have hmap : e (sSup S) = sSup (e '' S) := by
    rw [e.map_sSup, sSup_image]
  obtain ⟨_, ⟨s, hs, rfl⟩, hes⟩ := hU.2 hImgNe hImgDir (hmap ▸ hsup)
  exact ⟨s, hs, hes⟩

end Theorem212Transfer

end DomainTheory.ContinuousLattice
```

supSet.lean

```
module

public import DomainTheory.ContinuousLattice.ContinuousLattices.IdemFix
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section
```

```
namespace DomainTheory.ContinuousLattice

open Topology Set

section FixedPoints

variable {L : Type*} [CompleteLattice L]

namespace IdemFix

variable {e : L → L}

@[reducible] noncomputable def supSet (hidem : ∀ x, e (e x) = e x) : SupSet (IdemFix e)
    :=
  ⟨fun S => ⟨e (sSup (Subtype.val '' S)), hidem _⟩⟩

end IdemFix

end FixedPoints

end DomainTheory.ContinuousLattice
```

theorem_2_12.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.Injective.InjectiveSpace
public import DomainTheory.ContinuousLattice.Specialization.scottTopologicalSpace
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

public import DomainTheory.ContinuousLattice.ContinuousLattices.theorem_2_12_more

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

universe u

/-- **Scott 1972, Theorem 2.12.** A `T₀`-space is injective iff it is (homeomorphic to)
    a
continuous lattice under its Scott topology. The forward direction is Proposition
   2.11; the
backward direction is `theorem_2_12.backward`. -/
theorem theorem_2_12 {D : Type u} [TopologicalSpace D] [T0Space D] :
    InjectiveSpace.{u, u} D ↔
      ∃ (E : Type u) (inst : CompleteLattice E),
        @ContinuousLattice E inst ∧
          Nonempty (@Homeomorph D E _ (@scottTopologicalSpace E inst)) := by
  refine ⟨theorem_2_12.backward, ?_⟩
  rintro ⟨E, inst, hE, ⟨h⟩⟩
  let τE : TopologicalSpace E := @scottTopologicalSpace E inst
```

```
  exact @InjectiveSpace.of_retract E D τE _ (theorem_2_12.forward hE)
    ⟨⇑h, h.continuous⟩ ⟨⇑h.symm, h.symm.continuous⟩ (fun d => h.left_inv d)

end DomainTheory.ContinuousLattice
```

theorem_2_12_more.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.ContinuousLattices.idemFix_scottTopology
public import DomainTheory.ContinuousLattice.ContinuousLattices.
    idemFix_isContinuousLattice
public import DomainTheory.ContinuousLattice.Constructions.proposition_2_9
public import DomainTheory.ContinuousLattice.Constructions.scottTopology
public import DomainTheory.ContinuousLattice.Injective.corollary_1_6
public import DomainTheory.ContinuousLattice.Injective.InjectiveSpace
public import DomainTheory.ContinuousLattice.ScottMaps.continuous_preservesDirectedSup
public import DomainTheory.ContinuousLattice.Specialization.scottTopologicalSpace
public import Mathlib.Data.Set.Basic
public import Mathlib.Topology.Defs.Basic
public import DomainTheory.ContinuousLattice.ContinuousLattices.
    IsContinuousLatticeu2eorderIso
public import DomainTheory.ContinuousLattice.ContinuousLattices.
    orderIsoScottHomeomorph
public import DomainTheory.ContinuousLattice.Specialization.SpecializationLe
public import DomainTheory.ContinuousLattice.Constructions.proposition_2_11
public import DomainTheory.ContinuousLattice.Injective.proposition_1_2
public import Mathlib.Topology.Order

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

universe u

/-- **Scott 1972, Theorem 2.12 (backward direction).** Every injective `T₀`-space is
    homeomorphic
to a continuous lattice equipped with its Scott topology.

The injective space `D` is a retract of a power `𝕆^I = (I → Prop)` of the Sierpinski
    space
(Corollary 1.6). Writing `s : D → 𝕆^I` for the embedding and `r : 𝕆^I → D` for the
    retraction,
`e := s ∘ r` is a Scott-continuous idempotent on `𝕆^I` (its topology is its Scott
    topology by
`scottTopology.sierpinskiPower`). The fixed-point set `IdemFix e` is therefore a
    continuous lattice
(`idemFix_isContinuousLattice`, via Proposition 2.10), and `d ↦ s d` is a
    homeomorphism `D ≃ₜ
IdemFix e` with inverse the retraction. -/
theorem theorem_2_12.backward {D : Type u} [TopologicalSpace D] [T0Space D]
    (hD : InjectiveSpace.{u, u} D) :
```

```
    ∃ (E : Type u) (inst : CompleteLattice E),
      @ContinuousLattice E inst ∧
        Nonempty (@Homeomorph D E _ (@scottTopologicalSpace E inst)) := by
  obtain ⟨ι, ⟨R⟩⟩ := (corollary_1_6 D).1 hD
  -- `e = s ∘ r` on the Sierpinski power `L = ι → Prop`.
  have hidem : ∀ x, R.section' (R.retraction (R.section' (R.retraction x)))
      = R.section' (R.retraction x) :=
    fun x => congrArg R.section' (R.retraction_section (R.retraction x))
  have hL : ContinuousLattice (ι → Prop) := proposition_2_9.sierpinskiPower ι
  -- `e` is Scott-continuous: it is product-continuous (`s ∘ r`), and the Scott
    topology of the
  -- Sierpinski power is its product topology.
  have hcont_e : @Continuous (ι → Prop) (ι → Prop) scottTopologicalSpace
    scottTopologicalSpace
      (fun x => R.section' (R.retraction x)) := by
    rw [scottTopology.sierpinskiPower ι]
    exact R.section'.continuous.comp R.retraction.continuous
  have hsc : PreservesDirectedSup (fun x => R.section' (R.retraction x)) :=
    continuous_preservesDirectedSup hcont_e
  refine ⟨IdemFix (fun x => R.section' (R.retraction x)),
    IdemFix.completeLattice hidem (preservesDirectedSup_monotone hsc),
    idemFix_isContinuousLattice hidem hsc hL, ?_⟩
  -- The Scott topology of `IdemFix e` equals the subspace topology it inherits as a
    subtype of the
  -- Sierpinski power: both are `induced Subtype.val` of the (Scott = product) topology
     of `𝕆^I`.
  have heq : (@scottTopologicalSpace (IdemFix (fun x => R.section' (R.retraction x)))
      (IdemFix.completeLattice hidem (preservesDirectedSup_monotone hsc)))
      = instTopologicalSpaceSubtype := by
    rw [idemFix_scottTopology hidem hsc hL, scottTopology.sierpinskiPower ι]
    rfl
  rw [heq]
  -- the homeomorphism `D ≃ₜ IdemFix e`, with the subspace topology (where `Continuous.
    comp` works)
  have hfix : ∀ d, (fun x => R.section' (R.retraction x)) (R.section' d) = R.section' d
     :=
    fun d => congrArg R.section' (R.retraction_section d)
  exact ⟨{
    toFun := fun d => ⟨R.section' d, hfix d⟩
    invFun := fun k => R.retraction k.1
    left_inv := fun d => R.retraction_section d
    right_inv := fun k => Subtype.ext k.2
    continuous_toFun := by exact R.section'.continuous.subtype_mk hfix
    continuous_invFun := by exact R.retraction.continuous.comp continuous_subtype_val
    }⟩

section Theorem212Transfer

variable {A B : Type u} [latticeA : CompleteLattice A] [latticeB : CompleteLattice B]

/-- **Theorem 2.12, source-faithful exact-order form.** If an injective `T₀` topology
    on an
already specified complete lattice has that lattice order as its specialization order,
     then the
specified lattice is continuous and the given topology is exactly its Scott topology.
    -/
```

```
theorem theorem_2_12.backward_exact [τA : TopologicalSpace A] [T0Space A]
    (hA : InjectiveSpace.{u, u} A)
    (hspec : ∀ x y : A, SpecializationLe x y ↔ @LE.le A latticeA.toLE x y) :
    ContinuousLattice A ∧
      (@scottTopologicalSpace A latticeA) = τA := by
  obtain ⟨E, instE, hE, ⟨h⟩⟩ := theorem_2_12.backward hA
  let : CompleteLattice E := instE
  let : TopologicalSpace E := @scottTopologicalSpace E instE
  let : Preorder A := latticeA.toPreorder
  let : Preorder E := instE.toPreorder
  let e : A ≃o E := h.toEquiv.toOrderIso
    (fun x y hxy => by
      exact SpecializationLe.scott_iff.mp
        (SpecializationLe.map ((hspec x y).mpr hxy) h.continuous))
    (fun x y hxy => by
      exact (hspec (h.symm x) (h.symm y)).mp
        (SpecializationLe.map (SpecializationLe.scott_iff.mpr hxy) h.symm.continuous))
  have hcont : ContinuousLattice A := ContinuousLattice.orderIso e hE
  refine ⟨hcont, ?_⟩
  have heqOrder :
      (@scottTopologicalSpace A latticeA) =
        TopologicalSpace.induced (e : A → E) (@scottTopologicalSpace E instE) := by
    let : TopologicalSpace A := @scottTopologicalSpace A latticeA
    exact (@orderIsoScottHomeomorph A E latticeA instE e).isInducing.eq_induced
  have heqHome :
      τA = TopologicalSpace.induced (h : A → E) (@scottTopologicalSpace E instE) :=
    (@Homeomorph.isInducing A E τA (@scottTopologicalSpace E instE) h).eq_induced
  exact heqOrder.trans heqHome.symm

end Theorem212Transfer

variable {D : Type*} [CompleteLattice D]

/-- **Scott 1972, Theorem 2.12 (forward direction).** Every continuous lattice is an
    injective
space under its Scott topology. This is the substantial half of the equivalence "
    injective
spaces = continuous lattices", and is exactly Proposition 2.11. -/
theorem theorem_2_12.forward {E : Type*} [CompleteLattice E] (hE : ContinuousLattice E
    ) :
    @InjectiveSpace E scottTopologicalSpace :=
  proposition_2_11 hE

variable {D : Type*} [CompleteLattice D]

/-- **Scott 1972, Theorem 2.12 (injectivity half, unconditional).** `Prop` is injective
(Sierpinski); the continuous-lattice half is now `isContinuousLattice_prop`. -/
theorem theorem_2_12.injective_half : InjectiveSpace Prop :=
  proposition_1_2

variable {D : Type*} [CompleteLattice D]

/-- **Scott 1972, Theorem 2.12 (backward, Sierpinski base case).** `Prop` is a
    continuous
lattice (`isContinuousLattice_prop`), so its injectivity (Proposition 1.2) is an
    instance of the
```

```
equivalence. -/
theorem theorem_2_12.sierpinski_backward (_h : ContinuousLattice Prop) :
    InjectiveSpace Prop :=
  proposition_1_2

end DomainTheory.ContinuousLattice
```

1.3.17 After Theorem 2.12

The lattice approach to injective spaces gives a completely "internal" characterization of them: in the first place the lattices are complete. Next we can define lattice theoretically:

$$x \ll y \quad \text{iff} \quad \text{whenever } y \sqsubseteq \bigsqcup Z \text{ and } Z \subseteq D \text{ is directed,}$$

then $x \sqsubseteq z$ for some $z \in Z$.

Finally we assume that for all $y \in D$:

$$y = \bigsqcup \{x \in D : x \ll y\}.$$

That makes D a continuous lattice with the sets $\{y \in D : x \ll y\}$ as a basis for the topology. Such T_0-spaces are injective and every injective space can be obtained in this way with the lattice structure being uniquely determined by the topology. Furthermore, as we have seen, the injective property can be exhibited, as in the proof of 2.11, by an explicit formula for extending functions.

The retract approach to injective spaces should also be considered. The Cartesian powers $\mathbb{O}^I$ are very simple spaces; indeed, as lattices these are just the Boolean algebras of *all subsets* of I (that is, isomorphic thereto). The topology has as a basis the

classes of sets containing given finite sets (the *weak topology*, cf. Nerode (1959)). A continuous function

$$j : \mathcal{P}I \to \mathcal{P}I$$

is one of "finite character" so that

$$j(X) = \bigcup j(F) : F \subseteq X$$

where $X \subseteq I$ and F ranges over *finite sets*. Such a function j is a *retraction* iff it is an *idempotent*:

$$j \circ j = j,$$

which means that the range of j is the set of fixed points of j. As we have seen

$$D = \{X \in \mathcal{P}I : j(X) = X\}$$

is a continuous lattice (under $\subseteq$ in this case), and *every* continuous lattice is isomorphic to one obtained in this way. This provides a representation theorem of sorts for continuous lattices, but it does not seem to be of too much help in proving theorems.

The reader should not forget that any space (any *given* number of spaces X, Y, ...) can be found as a *subspace* of a continuous lattice D. Since D is injective any continuous function $f : X \to Y$ can be extended to a continuous function $\bar{f} : D \to D$. Thus the continuous functions from D into D are a rich totality including *all* the structure of continuous functions on *all* the subspaces. And this remark brings us to the study of function spaces.

1.4 Function spaces

We recall the standard definition and introduce our notation for function spaces.

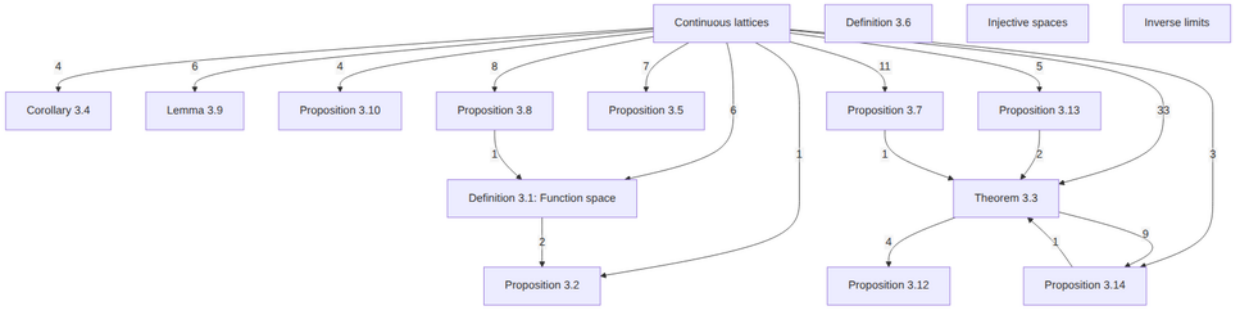


Figure 7: Function spaces.

1.4.1 Definition 3.1: Function space

3.1 Definition. For T_0-spaces X and Y we let $[X \to Y]$ be the *space of all continuous functions* $f : X \to Y$ endowed with the product topology, sometimes called the topology of pointwise convergence. This topology has as a subbase sets of the form:

$$\{f : f(x) \in U\}$$

where $x \in X$ and $U \subseteq Y$ is open.

The pointwise aspect of the topology is immediately apparent in the partial ordering.

ScottMap.lean

```
module

public import DomainTheory.ContinuousLattice.Specialization.scottTopologicalSpace
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

universe u v

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- Continuous maps between complete lattices with Scott's induced topologies. -/
```

```
def ScottMap (D : Type u) (D' : Type v) [CompleteLattice D] [CompleteLattice D'] :
    Type _ :=
  { f : D → D' // @Continuous D D' scottTopologicalSpace scottTopologicalSpace f }

namespace ScottMap

instance : CoeFun (ScottMap D D') (fun _ => D → D') where
  coe f := f.1

@[ext]
theorem ext {f g : ScottMap D D'} (h : ∀ x, f x = g x) : f = g :=
  Subtype.ext (funext h)

end ScottMap

end DomainTheory.ContinuousLattice
```

const.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Order.BourbakiWitt
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

def const (c : D') : ScottMap D D' :=
  ⟨fun _ => c, continuous_of_preservesDirectedSup fun S hS _ => by
    obtain ⟨x, hx⟩ := hS
    apply le_antisymm
    · exact le_sSup ⟨x, hx, rfl⟩
    · apply sSup_le
      intro b hb
      obtain ⟨y, hy, rfl⟩ := hb
      exact le_rfl⟩
```

```
end ScottMap

end DomainTheory.ContinuousLattice
```

const_apply.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.const
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

@[simp] theorem const_apply (c : D') (x : D) : (ScottMap.const c : D → D') x = c :=
    rfl

end ScottMap

end DomainTheory.ContinuousLattice
```

scottFunctionSpecializationLe.lean

```
module

public import DomainTheory.ContinuousLattice.Specialization.SpecializationLe
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    specializationLe_generateOpen
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottFunctionSubbasis_isOpen
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottFunctionTopology
public import Mathlib.Data.Set.Basic
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace
```

```
variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem scottFunctionSpecializationLe {f g : C(X, Y)} :
    letI := scottFunctionTopology X Y
    SpecializationLe f g ↔ ∀ x, SpecializationLe (f x) (g x) := by
  let : TopologicalSpace (C(X, Y)) := scottFunctionTopology X Y
  constructor
  · intro hfg x U hU hfU
    exact hfg {h | h x ∈ U} (scottFunctionSubbasis_isOpen (X := X) (Y := Y) hU) hfU
  · intro hfg U hU hfU
    exact specializationLe_generateOpen f g hfg hU hfU

end DomainTheory.ContinuousLattice
```

scottFunctionSubbasis_isOpen.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottFunctionTopology
public import Mathlib.Data.Set.Basic
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem scottFunctionSubbasis_isOpen {x : X} {U : Set Y} (hU : IsOpen U) :
    @IsOpen (C(X, Y)) (scottFunctionTopology X Y) {f : C(X, Y) | f x ∈ U} :=
  isOpen_generateFrom_of_mem (s := {f : C(X, Y) | f x ∈ U}) ⟨x, U, hU, rfl⟩

end DomainTheory.ContinuousLattice
```

scottFunctionSubbasis_isOpen_scott.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.Basic
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottFunctionTopology
public import Mathlib.Data.Set.Basic
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice
```

```
open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem scottFunctionSubbasis_isOpen_scott {x : D} {U : Set D'} (hU : @IsOpen D'
    scottTopologicalSpace U) :
    @IsOpen (ScottC D D') (@scottFunctionTopology D D' scottTopologicalSpace
    scottTopologicalSpace)
      {f : ScottC D D' | f x ∈ U} :=
  isOpen_generateFrom_of_mem (s := {f : ScottC D D' | f x ∈ U}) ⟨x, U, hU, rfl⟩

end DomainTheory.ContinuousLattice
```

scottFunctionTopology.lean

```
module

public import Mathlib.Topology.Order
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottFunctionEvalSubbasis
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

@[reducible] noncomputable def scottFunctionTopology (X Y : Type*) [TopologicalSpace X
    ]
    [TopologicalSpace Y] : TopologicalSpace (C(X, Y)) :=
  generateFrom (scottFunctionEvalSubbasis X Y)

end DomainTheory.ContinuousLattice
```

scottFunctionTopology_eq_induced_pi.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottFunctionSubbasis_isOpen
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    continuousMapInducedPiTopology
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottFunctionTopology
public import Mathlib.Data.Set.Basic
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Definition 3.1.** Scott's topology generated by evaluation cylinders
     is exactly
the topology induced on continuous maps from Mathlib's Pi topology on all underlying
    functions. -/
theorem scottFunctionTopology_eq_induced_pi (X Y : Type*) [TopologicalSpace X]
    [TopologicalSpace Y] :
    scottFunctionTopology X Y = continuousMapInducedPiTopology X Y := by
  apply le_antisymm
  · let : TopologicalSpace C(X, Y) := scottFunctionTopology X Y
    change scottFunctionTopology X Y ≤ continuousMapInducedPiTopology X Y
    rw [← continuous_iff_le_induced]
    exact continuous_pi fun x =>
      continuous_def.2 fun U hU =>
        scottFunctionSubbasis_isOpen (X := X) (Y := Y) (x := x) hU
  · apply le_generateFrom_iff_subset_isOpen.2
    rintro W ⟨x, U, hU, rfl⟩
    change @IsOpen C(X, Y)
      (TopologicalSpace.induced (fun f : C(X, Y) => (f : X → Y))
        (@Pi.topologicalSpace X (fun _ => Y) (fun _ => inferInstance)))
      {f : C(X, Y) | f x ∈ U}
    rw [isOpen_induced_iff]
    exact ⟨{g : X → Y | g x ∈ U}, (@continuous_apply X (fun _ => Y)
      (fun _ => inferInstance) x).isOpen_preimage U hU, rfl⟩

end DomainTheory.ContinuousLattice
```

scottMapInducedPiTopology.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace
```

```
universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

@[reducible] noncomputable def scottMapInducedPiTopology (D : Type u) (D' : Type v)
    [CompleteLattice D] [CompleteLattice D'] : TopologicalSpace (ScottMap D D') :=
  TopologicalSpace.induced (fun f : ScottMap D D' => (f : D → D'))
    (@Pi.topologicalSpace D (fun _ => D') (fun _ => scottTopologicalSpace))

end DomainTheory.ContinuousLattice
```

scottMapPointwiseSubbasis.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- Subbasic sets of the pointwise (product) topology on `[D → D']`: `{f | f x ∈ U}`
    for `U`
Scott-open in `D''`. -/
def scottMapPointwiseSubbasis (D D' : Type*) [CompleteLattice D] [CompleteLattice D']
    :
    Set (Set (ScottMap D D')) :=
  { W | ∃ (x : D) (U : Set D'),
      @IsOpen D' scottTopologicalSpace U ∧ W = {f : ScottMap D D' | (f : D → D') x ∈
    U} }

end DomainTheory.ContinuousLattice
```

scottMapPointwiseTopology.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.scottMapPointwiseSubbasis
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

@[reducible] noncomputable def scottMapPointwiseTopology (D D' : Type*)
    [CompleteLattice D] [CompleteLattice D'] : TopologicalSpace (ScottMap D D') :=
  generateFrom (scottMapPointwiseSubbasis D D')

end DomainTheory.ContinuousLattice
```

scottMapPointwiseTopology_eq_induced_pi.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.pointwiseSubbasic_isOpen
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottMapInducedPiTopology
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottMapPointwiseTopology
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- Scott's manually generated pointwise topology on `ScottMap` is the subspace
    topology inherited
from the literal Pi product of the codomain Scott topology. -/
theorem scottMapPointwiseTopology_eq_induced_pi :
    scottMapPointwiseTopology D D' = scottMapInducedPiTopology D D' := by
  apply le_antisymm
  · let : TopologicalSpace D' := scottTopologicalSpace
    let : TopologicalSpace (ScottMap D D') := scottMapPointwiseTopology D D'
```

```
    change scottMapPointwiseTopology D D' ≤ scottMapInducedPiTopology D D'
    rw [← continuous_iff_le_induced]
    exact continuous_pi fun x =>
      continuous_def.2 fun U hU => pointwiseSubbasic_isOpen (D := D) (D' := D') x hU
  · apply le_generateFrom_iff_subset_isOpen.2
    rintro W ⟨x, U, hU, rfl⟩
    let : TopologicalSpace D' := scottTopologicalSpace
    change @IsOpen (ScottMap D D')
      (TopologicalSpace.induced (fun f : ScottMap D D' => (f : D → D'))
        (@Pi.topologicalSpace D (fun _ => D') (fun _ => scottTopologicalSpace)))
      {f : ScottMap D D' | (f : D → D') x ∈ U}
    rw [isOpen_induced_iff]
    exact ⟨{g : D → D' | g x ∈ U}, (@continuous_apply D (fun _ => D')
      (fun _ => scottTopologicalSpace) x).isOpen_preimage U hU, rfl⟩

end DomainTheory.ContinuousLattice
```

scottMap_eq_sSup_openInfs.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottSubspaceExtendInf_le_of_mem
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.ScottMaps.proposition_2_5
public import DomainTheory.ContinuousLattice.ScottMaps.scottOpen_preimage
public import DomainTheory.ContinuousLattice.WayBelow.ScottOpen
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace Y] in
theorem scottMap_eq_sSup_openInfs (hD : ContinuousLattice D) (f' : ScottMap Y D) (y :
    Y) :
    (f' : Y → D) y =
      sSup { d | ∃ U, @IsOpen Y scottTopologicalSpace U ∧ y ∈ U ∧
        d = sInf (Set.image (f' : Y → D) U) } := by
  apply le_antisymm
  · rw [← hD.sSup_wayBelow (f' y)]
```

```
    apply sSup_le
    intro a ha
    obtain ⟨V, hV, hf'V, hVsub⟩ := ha
    have hVopen : @IsOpen D scottTopologicalSpace V := ScottOpen.isOpen_iff.mpr hV
    have hyU : y ∈ (f' : Y → D) ^-1' V := hf'V
    have hUScott : ScottOpen ((f' : Y → D) ^-1' V) :=
      scottOpen_preimage ((proposition_2_5 (Subtype.val f')).mp f'.property) hV
    have hUopen : @IsOpen Y scottTopologicalSpace ((f' : Y → D) ^-1' V) :=
      ScottOpen.isOpen_iff.mpr hUScott
    have ha' : a ≤ sInf (Set.image (f' : Y → D) ((f' : Y → D) ^-1' V)) := by
      apply le_sInf
      intro d hd
      obtain ⟨z, hzV, rfl⟩ := hd
      exact Set.mem_Ici.1 (hVsub hzV)
    have hmem : sInf (Set.image (f' : Y → D) ((f' : Y → D) ^-1' V)) ∈
        { d | ∃ U, @IsOpen Y scottTopologicalSpace U ∧ y ∈ U ∧
          d = sInf (Set.image (f' : Y → D) U) }  :=
      ⟨(f' : Y → D) ^-1' V, hUopen, hyU, rfl⟩
    exact le_trans ha' (le_sSup hmem)
  · apply sSup_le
    intro d hd
    obtain ⟨U, _hUopen, hyU, rfl⟩ := hd
    exact scottSubspaceExtendInf_le_of_mem (f' := f') hyU

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

1.4.2 Proposition 3.2

3.2 Proposition. *The induced partial ordering on* $[X \to Y]$ *is such that:*

$$f \sqsubseteq g \quad \text{iff} \quad f(x) \sqsubseteq g(x) \text{ for all } x \in X,$$

where $f, g \in [X \to Y]$. □

where $f, g \in [X \to Y]$. □

The first question, of course, is what kind of a partial ordering this is.

proposition_3_2.lean

```
module

public import DomainTheory.ContinuousLattice.Specialization.SpecializationLe
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottFunctionSpecializationLe
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottFunctionTopology
public import Mathlib.Data.Set.Basic
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace
```

```
variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Proposition 3.2.** The specialization order on `[X → Y]` is
    pointwise. -/
theorem proposition_3_2 {f g : C(X, Y)} :
    letI := scottFunctionTopology X Y
    (SpecializationLe f g ↔ ∀ x, SpecializationLe (f x) (g x)) :=
  scottFunctionSpecializationLe

end DomainTheory.ContinuousLattice
```

1.4.3 Theorem 3.3

3.3 Theorem. If D and D' are continuous lattices, then so is $[D \to D']$ under the induced partial ordering with the lattice topology agreeing with the product topology.

Proof: The argument is "pointwise." Thus, the constant function with value $\perp \in D'$ is obviously continuous and is the $\perp$ of $[D \to D']$ by 3.2. Since by 2.7 the lattice operation $\sqcup$ on D' is continuous, then if $f, g \in [D \to D']$ the composition $f \sqcup g$, defined by

$$(f \sqcup g)(x) = f(x) \sqcup g(x)$$

for all $x \in D$, is also continuous and represents the lub of f, g in $[D \to D']$. (The same arguments apply to $\top$ and $\sqcap$, so $[D \to D']$ is at least a lattice.) To show that $[D \to D']$ is complete it is sufficient now to show that lubs of directed subsets exist. So let $\mathcal{F} \subseteq [D \to D']$ be directed. Define a function from D into D' by the equation:

$$(\bigsqcup \mathcal{F})(x) = \bigsqcup \{f(x) : f \in \mathcal{F}\},$$

for all $x \in D$. If we can show that $\bigsqcup \mathcal{F}$ is continuous, then being in $[D \to D']$ it has to be the lub. Consider $U \subseteq D'$, an open subset. Taking the inverse image and remembering that $\mathcal{F}$ is directed, we find:

$$\{x : (\bigsqcup \mathcal{F})(x) \in U = \bigcup x : f(x) \in U : f \in \mathcal{F}\}.$$

This is an open set, and so $\bigsqcup \mathcal{F}$ is indeed continuous. (Warning: the infinite $\sqcap \mathcal{F}$ are not in general computed pointwise; however, it is easy to extend the above argument to show that arbitrary $\bigsqcup \mathcal{F}$ are.)

To show that $[D \to D']$ is continuous, we establish first that for $f \in [D \to D']$

$$f = \bigsqcup \overrightarrow{e}[e, \{e'\} : e' \ll f(e)\},$$

where e ranges over D and e' over D', and where the function $\overrightarrow{e}[e, e']$ is defined by:

$$\overrightarrow{e}[e, e'](x) = \begin{cases} e' & \text{if } e \ll x, \\ \perp & \text{if not,} \end{cases}$$

for all $x \in D$. Call the function on the right f'. Calculate:

$$f'(x) = \bigsqcup \overrightarrow{e}[e, e'](x) : e' \ll f(e)$$

$$= \bigsqcup \{e' : \exists e \ll x \; [e' \ll f(e)]\}$$

$$= \bigsqcup \{e' : e' \ll f(x) = f(x)\}.$$

With the equation for f proved, note next that for all $g \in [D \to D']$, $e' \ll g(e)$ implies $\overrightarrow{e}[e, e'] \sqsubseteq g$ by an easy pointwise argument. If we let $V = \{g : e' \ll g(e)\}$, we see then that V is open in the product topology and that $\overrightarrow{e}[e, e'] \sqsubseteq \sqcap V$. We may then conclude that $f = \bigsqcup \{\sqcap V : f \in V\}$, which proves both that $[D \to D']$ is a continuous lattice and that the two topologies agree by the remark following 2.4. □

The above theorem might possibly be generalized to $[X \to D]$ where X is merely a T_0-space, but I was unable to see the argument. In any case we are mostly interested in the continuous lattices. Note as a consequence of our proof:

Basic.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.fixMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.Basic.isLUB_sSup
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace
variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- Scott's notation `[X → Y]`: continuous maps with the pointwise (product) topology.
    -/
abbrev ScottFunctionSpace (X Y : Type*) [TopologicalSpace X] [TopologicalSpace Y] :=
  C(X, Y)

/-- Continuous maps `D → D'` with Scott topologies on domain and codomain. -/
abbrev ScottC (D D' : Type*) [CompleteLattice D] [CompleteLattice D'] :=
  @ContinuousMap D D' scottTopologicalSpace scottTopologicalSpace

namespace ScottMap
theorem continuous (f : ScottMap D D') :
    @Continuous D D' scottTopologicalSpace scottTopologicalSpace f :=
  f.2

end ScottMap
namespace IsContinuousLatticeRetraction
```

```
end IsContinuousLatticeRetraction

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

theorem directedOn_wayBelow (a : D) : DirectedOn (· ≤ ·) { z | z ≪ a } := by
  intro p hp q hq
  exact ⟨p ⊔ q, WayBelow.sup hp hq, le_sup_left, le_sup_right⟩

end IsContinuousLatticeProjection

variable {i : D → D'}

end Proposition310

namespace Proposition313

end Proposition313
namespace Proposition314
@[simp] theorem fixMap_apply (f : ScottMap D D) : (fixMap f : D) = fix f := rfl

end Proposition314

end DomainTheory.ContinuousLattice
```

isLUB_sSup.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.instSupSet
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

namespace Projections
```

```
theorem isLUB_sSup (T : Set (Projections D)) : IsLUB T (sSup T) := by
  constructor
  · intro p hp
    apply Subtype.coe_le_coe.mp
    show (p : ScottMap D D) ≤ ((sSup T : Projections D) : ScottMap D D)
    exact le_sSup (Set.mem_image_of_mem _ hp)
  · intro q hq
    apply Subtype.coe_le_coe.mp
    show ((sSup T : Projections D) : ScottMap D D) ≤ (q : ScottMap D D)
    refine sSup_le ?_
    rintro _ ⟨p, hp, rfl⟩
    exact Subtype.coe_le_coe.mpr (hq hp)

end Projections

end Projections

end DomainTheory.ContinuousLattice
```

BasicInst28.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.le_refl'
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_trans'
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_antisymm'
public import DomainTheory.ContinuousLattice.FunctionSpaces.lt_iff_le_not_ge'
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

instance instPartialOrder {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] : PartialOrder (ScottMap D D') where
  le := ScottMap.le
  lt := ScottMap.lt
  le_refl := le_refl'
  le_trans := le_trans'
```

```
  lt_iff_le_not_ge := lt_iff_le_not_ge'
  le_antisymm := le_antisymm'

end ScottMap

end DomainTheory.ContinuousLattice
```

BasicInst30.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.sSupMaps
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

noncomputable instance instSupSet {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] : SupSet (ScottMap D D') := ⟨sSupMaps⟩

end ScottMap

end DomainTheory.ContinuousLattice
```

BasicInst33.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.isLUB_sSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace
```

```
universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

noncomputable instance instCompleteLattice {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] : CompleteLattice (ScottMap D D') :=
  completeLatticeOfSup (ScottMap D D') isLUB_sSup

end ScottMap

end DomainTheory.ContinuousLattice
```

Projections.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.IsProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

/-- **Scott 1972, Definition 3.11.** The space `J_D` of projections of `D`, as a
    subtype of
`[D → D]`. -/
abbrev Projections (D : Type*) [CompleteLattice D] : Type _ := {j : ScottMap D D //
    IsProjection j}

end Projections

end DomainTheory.ContinuousLattice
```

ScottMapProjection.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMapRetraction
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Proposition 3.7 (projection half).** Under the same hypotheses with
projections, the induced pair on function spaces is also a projection. -/
structure ScottMapProjection (D₀ D₀' D₁ D₁' : Type*)
    [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [CompleteLattice D₁
    '] extends
    ScottMapRetraction D₀ D₀' D₁ D₁' where
  incl_retr_le : ∀ f' x, (incl (retr f')) x ≤ f' x

namespace ScottMapProjection

def functionSpace (P₀ : IsContinuousLatticeProjection D₀ D₀') (P₁ :
    IsContinuousLatticeProjection D₁ D₁') :
    ScottMapProjection D₀ D₀' D₁ D₁' where
  toScottMapRetraction :=
    ScottMapRetraction.functionSpace P₀.toIsContinuousLatticeRetraction P₁.
    toIsContinuousLatticeRetraction
  incl_retr_le f' y := by
    dsimp only [ScottMapRetraction.functionSpace, ScottMap.comp, Function.comp_def]
    exact le_trans (P₁.incl_retr_le (f' (P₀.incl (P₀.retr y))))
      (ScottMap.monotone f' (P₀.incl_retr_le y))

end ScottMapProjection

end DomainTheory.ContinuousLattice
```

ScottMapRetraction.lean

```
module
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeRetraction
public import DomainTheory.ContinuousLattice.FunctionSpaces.comp
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Proposition 3.7 (retraction half).** If `D_n` is a retract of `D'_n`
(`n = 0,1`), then `[D₀ → D₁]` is a retract of `[D'₀ → D'₁]` via
`f ↦ i₁ ∘ f ∘ j₀` and `f' ↦ j₁ ∘ f' ∘ i₀`. -/
structure ScottMapRetraction (D₀ D₀' D₁ D₁' : Type*)
    [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [CompleteLattice D₁
    '] where
  incl : ScottMap D₀ D₁ → ScottMap D₀' D₁'
  retr : ScottMap D₀' D₁' → ScottMap D₀ D₁
  retr_incl : ∀ f, retr (incl f) = f

namespace ScottMapRetraction

def functionSpace (R₀ : IsContinuousLatticeRetraction D₀ D₀') (R₁ :
    IsContinuousLatticeRetraction D₁ D₁') :
    ScottMapRetraction D₀ D₀' D₁ D₁' where
  incl f := ScottMap.comp R₁.incl (ScottMap.comp f R₀.retr)
  retr f' := ScottMap.comp R₁.retr (ScottMap.comp f' R₀.incl)
  retr_incl f := ScottMap.ext fun x => by simp [ScottMap.comp, R₀.retr_incl, R₁.
    retr_incl]

end ScottMapRetraction

end DomainTheory.ContinuousLattice
```

ambientSSup.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeRetraction
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section
```

```
namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace IsContinuousLatticeRetraction

/-- **Scott 1972, Prop 2.10 / March 1972 correction (p. 135).** For `S ⊆ D` a subspace
     of
ambient `D''`, write `⊔S'` for the supremum of `S` computed in `D''` (Scott's prime is on
    the
index, not the join). This is `sSup (i '' S)` in Lean. -/
noncomputable def ambientSSup (R : IsContinuousLatticeRetraction D D') (S : Set D) : D
    ' :=
  sSup (Set.image (R.incl : D → D') S)

end IsContinuousLatticeRetraction

end DomainTheory.ContinuousLattice
```

bot_apply.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

/-- The bottom Scott map is the constant `⊥`. -/
```

```
theorem bot_apply (x : D) : ((⊥ : ScottMap D D') : D → D') x = ⊥ := by
  have h : (⊥ : ScottMap D D') = sSup (∅ : Set (ScottMap D D')) := rfl
  rw [h, sSup_apply, Set.image_empty, sSup_empty]

end ScottMap

end DomainTheory.ContinuousLattice
```

comp.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Order.BourbakiWitt
public import DomainTheory.ContinuousLattice.ScottMaps.proposition_2_5
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.preservesDirectedSup_monotone
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

variable {D D' D'' : Type*} [CompleteLattice D] [CompleteLattice D'] [CompleteLattice
    D'']

namespace ScottMap

theorem preservesDirectedSup_comp {f : D' → D''} {g : D → D'} (hf :
    PreservesDirectedSup f)
    (hg : PreservesDirectedSup g) : PreservesDirectedSup (f ∘ g) := by
  intro S hS hSdir
  have hmono : Monotone g := preservesDirectedSup_monotone hg
  have hdir' : DirectedOn (· ≤ ·) (g '' S) := by
    intro p hp q hq
    obtain ⟨a, ha, rfl⟩ := hp
    obtain ⟨b, hb, rfl⟩ := hq
    obtain ⟨c, hc, hac, hbc⟩ := hSdir a ha b hb
    exact ⟨g c, Set.mem_image_of_mem g hc, hmono hac, hmono hbc⟩
  have hne : (g '' S).Nonempty := Set.image_nonempty.2 hS
  rw [Function.comp_apply, hg hS hSdir, hf hne hdir']
  congr 1
  exact (Set.image_comp f g S).symm

def comp (f : ScottMap D' D'') (g : ScottMap D D') : ScottMap D D'' :=
  ⟨f ∘ g, continuous_of_preservesDirectedSup (preservesDirectedSup_comp
    ((proposition_2_5 (Subtype.val f)).mp f.property)
    ((proposition_2_5 (Subtype.val g)).mp g.property))⟩

end ScottMap
```

```
end DomainTheory.ContinuousLattice
```

comp_apply.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.comp
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

@[simp] theorem comp_apply (f : ScottMap D' D'') (g : ScottMap D D') (x : D) :
    (f.comp g : D → D'') x = f (g x) := rfl

end ScottMap

end DomainTheory.ContinuousLattice
```

continuousMapInducedPiTopology.lean

```
module
public import Mathlib.Topology.Constructions
public import Mathlib.Data.Set.Basic
public import Mathlib.Order.CompleteLattice.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']
```

```
@[reducible] noncomputable def continuousMapInducedPiTopology (X Y : Type*)
    [TopologicalSpace X] [TopologicalSpace Y] : TopologicalSpace C(X, Y) :=
  TopologicalSpace.induced (fun f : C(X, Y) => (f : X → Y))
    (@Pi.topologicalSpace X (fun _ => Y) (fun _ => inferInstance))

end DomainTheory.ContinuousLattice
```

directedOn_finsetSupOf.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.finsetSupOf
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

theorem directedOn_finsetSupOf (S : Set D) :
    DirectedOn (· ≤ ·) (finsetSupOf S) := by
  classical
  intro a ha b hb
  obtain ⟨T₁, hT₁, rfl⟩ := ha
  obtain ⟨T₂, hT₂, rfl⟩ := hb
  have hunion : (T₁ ∪ T₂).sup id = T₁.sup id ⊔ T₂.sup id := by
    simpa using Finset.sup_union (s₁ := T₁) (s₂ := T₂) (f := id)
  refine ⟨T₁.sup id ⊔ T₂.sup id, ⟨T₁ ∪ T₂, ?_, hunion.symm⟩, le_sup_left, le_sup_right⟩
  intro x hx
  simp only [Finset.mem_union] at hx
  exact hx.elim (hT₁ x) (hT₂ x)

end IsContinuousLatticeProjection

end Proposition310

end DomainTheory.ContinuousLattice
```

finsetSupOf.lean

```
module

public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

/-- Finite sub-lubs used in Scott's proof of Proposition 3.10(i). -/
def finsetSupOf (S : Set D) : Set D :=
  { z | ∃ T : Finset D, (∀ x ∈ T, x ∈ S) ∧ z = T.sup id }

end IsContinuousLatticeProjection

end Proposition310

end DomainTheory.ContinuousLattice
```

idMap.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
```

```
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

/-- The identity Scott map. -/
def idMap : ScottMap D D :=
  ⟨fun x => x, continuous_of_preservesDirectedSup fun S _ _ => by
    show sSup S = sSup (Set.image (fun x => x) S)
    rw [Set.image_id']⟩

end ScottMap

end DomainTheory.ContinuousLattice
```

idMap_apply.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.idMap
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap
```

```
@[simp] theorem idMap_apply (x : D) : (idMap : ScottMap D D) x = x := rfl

end ScottMap

end DomainTheory.ContinuousLattice
```

image_retr_wayBelow_directed.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeRetraction
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Order.BourbakiWitt
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace IsContinuousLatticeRetraction

theorem image_retr_wayBelow_directed (R : IsContinuousLatticeRetraction D D') (d : D)
    :
    DirectedOn (· ≤ ·) (Set.image (R.retr : D' → D) {x' | x' ≪ R.incl d}) := by
  rintro _ ⟨a, ha, rfl⟩ _ ⟨b, hb, rfl⟩
  exact ⟨R.retr (a ⊔ b), ⟨a ⊔ b, WayBelow.sup ha hb, rfl⟩,
    R.retr.monotone le_sup_left, R.retr.monotone le_sup_right⟩

end IsContinuousLatticeRetraction
end DomainTheory.ContinuousLattice
```

image_retr_wayBelow_nonempty.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeRetraction
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace
```

```
variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace IsContinuousLatticeRetraction

theorem image_retr_wayBelow_nonempty (R : IsContinuousLatticeRetraction D D') (d : D)
    :
    (Set.image (R.retr : D' → D) {x' | x' ≪ R.incl d}).Nonempty :=
  ⟨R.retr ⊥, ⊥, bot_wayBelow _, rfl⟩

end IsContinuousLatticeRetraction
end DomainTheory.ContinuousLattice
```

instCompleteLattice.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.Basic.isLUB_sSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

namespace Projections

/-- **Scott 1972, Proposition 3.12.** `J_D` is a complete lattice (a `⊔`-closed
    subspace of
`[D → D]`). Suprema are inherited from `[D → D]`; infima are derived by `
    completeLatticeOfSup`. -/
noncomputable instance instCompleteLattice : CompleteLattice (Projections D) :=
  completeLatticeOfSup (Projections D) isLUB_sSup

end Projections

end Projections
```

```
end DomainTheory.ContinuousLattice
```

instSupSet.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst30
public import DomainTheory.ContinuousLattice.FunctionSpaces.Projections
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_sSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

namespace Projections

noncomputable instance instSupSet : SupSet (Projections D) :=
  ⟨fun T => ⟨sSup (Set.image Subtype.val T), isProjection_sSup (by rintro j ⟨p, _, rfl⟩;
     exact p.2)⟩⟩

end Projections

end Projections

end DomainTheory.ContinuousLattice
```

isLUB_sSup.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst28
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst30
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice
```

```
open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

theorem isLUB_sSup {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (F : Set (ScottMap D D')) :
    IsLUB F (sSup F) := by
  constructor
  · intro f hf
    rw [le_def]
    intro x
    rw [sSup_apply]
    exact le_sSup (Set.mem_image_of_mem _ hf)
  · intro g hg
    rw [le_def]
    intro x
    rw [sSup_apply]
    refine sSup_le ?_
    rintro _ ⟨f, hfF, rfl⟩
    exact (hg hfF) x

end ScottMap

end DomainTheory.ContinuousLattice
```

isProjection_bot.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.IsProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_iff
public import DomainTheory.ContinuousLattice.FunctionSpaces.bot_apply
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
```

```
variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

/-- `⊥` (the constant `⊥` map) is a projection -- Scott's `⊔∅ ∈ J_D`. -/
theorem isProjection_bot : IsProjection (⊥ : ScottMap D D) := by
  rw [isProjection_iff]
  refine ⟨fun x => ?_, fun x => ?_⟩
  · simp only [ScottMap.bot_apply]
  · rw [ScottMap.bot_apply]; exact bot_le

end Projections

end DomainTheory.ContinuousLattice
```

isProjection_directedSup.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst30
public import DomainTheory.ContinuousLattice.FunctionSpaces.IsProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_iff
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

/-- `J_D` is closed under *directed* joins (Scott 1972, 3.12). Continuity of each
    member lets the
inner `(⊔S)`-application distribute over the directed family `{ j x : j ∈ S }`, and
    directedness
```

```
plus idempotency collapse the double family `{ k (j x) }` to `(⨆S) x`. -/
theorem isProjection_directedSup {S : Set (ScottMap D D)} (hSne : S.Nonempty)
    (hSdir : DirectedOn (· ≤ ·) S) (hS : ∀ j ∈ S, IsProjection j) :
    IsProjection (sSup S) := by
  have hidem : ∀ j ∈ S, ∀ x, (j : D → D) (j x) = j x :=
    fun j hj => (isProjection_iff.mp (hS j hj)).1
  have hle : ∀ j ∈ S, ∀ x, (j : D → D) x ≤ x := fun j hj => (isProjection_iff.mp (hS
    j hj)).2
  rw [isProjection_iff]
  refine ⟨fun x => ?_, fun x => ?_⟩
  · set A := Set.image (fun j : ScottMap D D => (j : D → D) x) S with hA
    have hAne : A.Nonempty := hSne.image _
    have hAdir : DirectedOn (· ≤ ·) A := by
      rintro _ ⟨j, hj, rfl⟩ _ ⟨k, hk, rfl⟩
      obtain ⟨m, hm, hjm, hkm⟩ := hSdir j hj k hk
      exact ⟨(m : D → D) x, Set.mem_image_of_mem _ hm, hjm x, hkm x⟩
    have hsSx : ((sSup S : ScottMap D D) : D → D) x = sSup A := ScottMap.sSup_apply S
    x
    apply le_antisymm
    · rw [hsSx, ScottMap.sSup_apply]
      refine sSup_le ?_
      rintro _ ⟨k, hk, rfl⟩
      show (k : D → D) (sSup A) ≤ sSup A
      rw [k.preservesDirectedSup_coe A hAne hAdir]
      refine sSup_le ?_
      rintro _ ⟨_, ⟨j, hj, rfl⟩, rfl⟩
      obtain ⟨m, hm, hjm, hkm⟩ := hSdir j hj k hk
      calc (k : D → D) (j x) ≤ (m : D → D) (j x) := hkm (j x)
        _ ≤ (m : D → D) (m x) := m.monotone (hjm x)
        _ = (m : D → D) x := hidem m hm x
        _ ≤ sSup A := le_sSup (Set.mem_image_of_mem _ hm)
    · rw [hsSx, ScottMap.sSup_apply]
      refine sSup_le ?_
      rintro _ ⟨j, hj, rfl⟩
      have hjxA : (j : D → D) x ≤ sSup A := le_sSup (Set.mem_image_of_mem _ hj)
      calc (j : D → D) x = (j : D → D) (j x) := (hidem j hj x).symm
        _ ≤ (j : D → D) (sSup A) := j.monotone hjxA
        _ ≤ sSup (Set.image (fun f : ScottMap D D => (f : D → D) (sSup A)) S) :=
              le_sSup (Set.mem_image_of_mem _ hj)
  · rw [ScottMap.sSup_apply]
    refine sSup_le ?_
    rintro _ ⟨j, hj, rfl⟩
    exact hle j hj x

end Projections

end DomainTheory.ContinuousLattice
```

isProjection_finsetSup.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.IsProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_sup
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_bot
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

/-- `J_D` is closed under finite joins. -/
theorem isProjection_finsetSup {T : Finset (ScottMap D D)} (hT : ∀ j ∈ T, IsProjection
     j) :
    IsProjection (T.sup id) :=
  Finset.sup_induction isProjection_bot (fun _ ha _ hb => isProjection_sup ha hb) hT

end Projections

end DomainTheory.ContinuousLattice
```

isProjection_iff.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.IsProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.comp_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.idMap_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
```

```
    CompleteLattice D₁']

section Projections

/-- Pointwise characterization of projections: idempotent and deflationary. -/
theorem isProjection_iff {j : ScottMap D D} :
    IsProjection j ↔ (∀ x, (j : D → D) (j x) = j x) ∧ (∀ x, (j : D → D) x ≤ x) := by
  constructor
  · rintro ⟨hidem, hle⟩
    refine ⟨fun x => ?_, fun x => ?_⟩
    · have h : ((j.comp j : ScottMap D D) : D → D) x = (j : D → D) x := by rw [hidem]
      rwa [ScottMap.comp_apply] at h
    · have h := (ScottMap.le_def.mp hle) x
      rwa [ScottMap.idMap_apply] at h
  · rintro ⟨hidem, hle⟩
    exact ⟨ScottMap.ext fun x => by rw [ScottMap.comp_apply]; exact hidem x,
      ScottMap.le_def.mpr fun x => by rw [ScottMap.idMap_apply]; exact hle x⟩

end Projections

end DomainTheory.ContinuousLattice
```

isProjection_sSup.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst30
public import DomainTheory.ContinuousLattice.FunctionSpaces.IsProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_directedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_finsetSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.directedOn_finsetSupOf
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_finsetSupOf
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

/-- **Scott 1972, Proposition 3.12 (`⊔`-closure).** `J_D` is closed under arbitrary
    suprema in
`[D → D]`: every supremum is the directed supremum of finite sub-joins (`finsetSupOf`)
    , each a
```

```
projection by `isProjection_finsetSup`. -/
theorem isProjection_sSup {T : Set (ScottMap D D)} (hT : ∀ j ∈ T, IsProjection j) :
    IsProjection (sSup T) := by
  rw [IsContinuousLatticeProjection.sSup_finsetSupOf T]
  refine isProjection_directedSup ⟨⊥, ∅, by simp, by simp⟩
    (IsContinuousLatticeProjection.directedOn_finsetSupOf T) ?_
  rintro z ⟨F, hF, rfl⟩
  exact isProjection_finsetSup fun j hj => hT j (hF j hj)

end Projections

end DomainTheory.ContinuousLattice
```

isProjection_sup.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.IsProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.sup_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_iff
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

/-- `J_D` is closed under binary joins (Scott 1972, 3.12). The key step: since `j x ⊔ k
    x ⊑ x`,
monotonicity and idempotency force `j (j x ⊔ k x) = j x` and `k (j x ⊔ k x) = k x`. -/
theorem isProjection_sup {j k : ScottMap D D} (hj : IsProjection j) (hk : IsProjection
    k) :
    IsProjection (j ⊔ k) := by
  rw [isProjection_iff] at hj hk ⊢
  obtain ⟨hjidem, hjle⟩ := hj
  obtain ⟨hkidem, hkle⟩ := hk
  refine ⟨fun x => ?_, fun x => ?_⟩
  · have hjkle : (j : D → D) x ⊔ k x ≤ x := sup_le (hjle x) (hkle x)
    have hj_eq : (j : D → D) (j x ⊔ k x) = j x :=
      le_antisymm (j.monotone hjkle) (le_of_eq_of_le (hjidem x).symm (j.monotone
```

```
    le_sup_left))
    have hk_eq : (k : D → D) (j x ⊔ k x) = k x :=
      le_antisymm (k.monotone hjkle) (le_of_eq_of_le (hkidem x).symm (k.monotone
    le_sup_right))
    rw [ScottMap.sup_apply, ScottMap.sup_apply, hj_eq, hk_eq]
  · rw [ScottMap.sup_apply]; exact sup_le (hjle x) (hkle x)

end Projections

end DomainTheory.ContinuousLattice
```

le.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

def le {D : Type u} {D' : Type v} [CompleteLattice D] [CompleteLattice D']
    (f g : ScottMap D D') : Prop :=
  letI : LE D' :=
    (ChainCompletePartialOrder.instOfCompleteLattice
      (α := D')).toPartialOrder.toPreorder.toLE
  ∀ x, f x ≤ g x

end ScottMap

end DomainTheory.ContinuousLattice
```

le_antisymm'.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.le
```

```
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

/-- Antisymmetry of the pointwise order. -/
theorem le_antisymm' {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (f g : ScottMap D D')
    (hfg : ScottMap.le f g)
    (hgf : ScottMap.le g f) : f = g :=
  ScottMap.ext fun x => le_antisymm (hfg x) (hgf x)

end ScottMap

end DomainTheory.ContinuousLattice
```

le_def.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst28
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']
```

```
namespace ScottMap

theorem le_def {f g : ScottMap D D'} : f ≤ g ↔ ∀ x, (f : D → D') x ≤ g x := Iff.rfl

end ScottMap

end DomainTheory.ContinuousLattice
```

le_of_incl_le.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_sup_of_preservesSSup
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- From (i)+(ii): `i` is order-reflecting (`x ⊑ y ↔ i x ⊑ i y`), since `x ⊑ y ↔ x ⊔
    y = y`. -/
theorem le_of_incl_le (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S))
    (hinj : Function.Injective i) {x y : D} (h : i x ≤ i y) : x ≤ y := by
  have h1 : i (x ⊔ y) = i y := by rw [incl_sup_of_preservesSSup hi, sup_eq_right.mpr h
    ]
  exact sup_eq_right.mp (hinj h1)

end Proposition310

end DomainTheory.ContinuousLattice
```

le_refl'.lean

```
module
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaces.le
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

/-- Reflexivity of the pointwise order. -/
theorem le_refl' {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (f : ScottMap D D') :
    ScottMap.le f f :=
  fun _ => le_refl _

end ScottMap

end DomainTheory.ContinuousLattice
```

le_sSup_finsetSupOf.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.finsetSupOf
public import DomainTheory.ContinuousLattice.FunctionSpaces.mem_finsetSupOf_of_mem
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
```

```
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

theorem le_sSup_finsetSupOf (S : Set D) : sSup S ≤ sSup (finsetSupOf S) :=
  sSup_le fun _ hx => le_sSup (mem_finsetSupOf_of_mem hx)

end IsContinuousLatticeProjection

end Proposition310

end DomainTheory.ContinuousLattice
```

le_trans'.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.le
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

/-- Transitivity of the pointwise order. -/
theorem le_trans' {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (f g h : ScottMap D D')
    (hfg : ScottMap.le f g)
    (hgh : ScottMap.le g h) : ScottMap.le f h :=
  fun x => le_trans (hfg x) (hgh x)

end ScottMap

end DomainTheory.ContinuousLattice
```

lt.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.le
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

/-- Strict pointwise order induced by `ScottMap.le`. -/
def lt {D : Type u} {D' : Type v} [CompleteLattice D] [CompleteLattice D']
    (f g : ScottMap D D') : Prop :=
  ScottMap.le f g ∧ ¬ ScottMap.le g f

end ScottMap

end DomainTheory.ContinuousLattice
```

lt_iff_le_not_ge'.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.lt
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]
```

```
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

/-- The chosen strict pointwise order is `≤ ∧ ¬ ≥`. -/
theorem lt_iff_le_not_ge' {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (f g : ScottMap D D') :
    ScottMap.lt f g ↔
      ScottMap.le f g ∧ ¬ ScottMap.le g f :=
  Iff.rfl

end ScottMap

end DomainTheory.ContinuousLattice
```

mem_finsetSupOf_of_mem.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.finsetSupOf
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

theorem mem_finsetSupOf_of_mem {S : Set D} {x : D} (hx : x ∈ S) :
    x ∈ finsetSupOf S :=
  ⟨{x}, by simp [hx], by simp⟩

end IsContinuousLatticeProjection

end Proposition310

end DomainTheory.ContinuousLattice
```

monotone.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Order.BourbakiWitt
public import DomainTheory.ContinuousLattice.ScottMaps.proposition_2_5
public import DomainTheory.ContinuousLattice.ScottMaps.preservesDirectedSup_monotone

@[expose] public section

namespace DomainTheory.ContinuousLattice

variable {D D' : Type*} [CompleteLattice D] [CompleteLattice D']

namespace ScottMap

theorem monotone (f : ScottMap D D') : Monotone (f : D → D') :=
  preservesDirectedSup_monotone ((proposition_2_5 (Subtype.val f)).mp f.property)

end ScottMap

end DomainTheory.ContinuousLattice
```

pointwiseSubbasic_isOpen.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottMapPointwiseTopology
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem pointwiseSubbasic_isOpen (x : D) {U : Set D'} (hU : @IsOpen D'
    scottTopologicalSpace U) :
    @IsOpen (ScottMap D D') (scottMapPointwiseTopology D D')
      {f : ScottMap D D' | (f : D → D') x ∈ U} :=
  isOpen_generateFrom_of_mem ⟨x, U, hU, rfl⟩

end DomainTheory.ContinuousLattice
```

pointwiseSubbasic_scottOpen.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ScottOpen
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- Each pointwise-subbasic set `{f | f x ∈ U}` (`U` Scott-open) is Scott-open in the
    lattice
`[D → D']`, because suprema there are pointwise. This is the easy inclusion pointwise
    ⊆ Scott. -/
theorem pointwiseSubbasic_scottOpen (x : D) {U : Set D'} (hU : ScottOpen U) :
    ScottOpen {f : ScottMap D D' | (f : D → D') x ∈ U} := by
  refine ⟨fun f f' hff' hf => hU.1 (hff' x) hf, fun F hFne hFdir hmem => ?_⟩
  simp only [Set.mem_ofPred_eq, ScottMap.sSup_apply] at hmem
  have hne : (Set.image (fun g : ScottMap D D' => (g : D → D') x) F).Nonempty := hFne.
    image _
  have hdir : DirectedOn (· ≤ ·) (Set.image (fun g : ScottMap D D' => (g : D → D') x)
    F) := by
    rintro _ ⟨a, ha, rfl⟩ _ ⟨b, hb, rfl⟩
    obtain ⟨c, hc, hac, hbc⟩ := hFdir a ha b hb
    exact ⟨(c : D → D') x, Set.mem_image_of_mem _ hc, hac x, hbc x⟩
  obtain ⟨_, ⟨g, hgF, rfl⟩, hg⟩ := hU.2 hne hdir hmem
  exact ⟨g, hgF, hg⟩

end DomainTheory.ContinuousLattice
```

pointwise_sSup_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
```

```
variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

theorem pointwise_sSup_preservesDirectedSup (F : Set (ScottMap D D')) :
    PreservesDirectedSup (fun x => sSup (Set.image (fun f : ScottMap D D' => (f : D →
    D') x) F)) := by
  intro S hS hSdir
  set H := fun x => sSup (Set.image (fun f : ScottMap D D' => (f : D → D') x) F)
  show H (sSup S) = sSup (Set.image H S)
  apply le_antisymm
  · apply sSup_le
    intro b hb
    obtain ⟨f, hfF, rfl⟩ := hb
    change f (sSup S) ≤ sSup (Set.image H S)
    rw [f.preservesDirectedSup_coe S hS hSdir]
    apply sSup_le
    intro c hc
    obtain ⟨s, hsS, rfl⟩ := hc
    exact le_trans (le_sSup (Set.mem_image_of_mem (fun g : ScottMap D D' => (g : D → D
    ') s) hfF))
      (le_sSup (Set.mem_image_of_mem H hsS))
  · apply sSup_le
    intro b hb
    obtain ⟨s, hsS, rfl⟩ := hb
    apply sSup_le
    intro c hc
    obtain ⟨f, hfF, rfl⟩ := hc
    exact le_trans (ScottMap.monotone f (le_sSup hsS))
      (le_sSup (Set.mem_image_of_mem (fun g : ScottMap D D' => (g : D → D') (sSup S))
    hfF))

end ScottMap

end DomainTheory.ContinuousLattice
```

preservesDirectedSup_coe.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Order.BourbakiWitt
public import DomainTheory.ContinuousLattice.ScottMaps.proposition_2_5
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice
```

```
open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

theorem preservesDirectedSup_coe (f : ScottMap D D') (S : Set D) (hS : S.Nonempty)
    (hSdir : DirectedOn (· ≤ ·) S) :
    (f : D → D') (sSup S) = sSup (Set.image (f : D → D') S) := by
  have h := (proposition_2_5 (Subtype.val f)).mp f.property
  exact h hS hSdir

end ScottMap

end DomainTheory.ContinuousLattice
```

projection.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.con_
public import DomainTheory.ContinuousLattice.FunctionSpaces.min
public import DomainTheory.ContinuousLattice.FunctionSpaces.const_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition313
```

```
/-- **Scott 1972, Proposition 3.13.** `(con, min)` makes `D` a projection of `[D → D]`:
`min ∘ con = id` (a constant's least value is its value) and `con ∘ min ⊑ id` (the
    constant `f(⊥)`
is `≤ f` pointwise, since `f(⊥) ⊑ f(y)` by monotonicity). -/
noncomputable def projection : IsContinuousLatticeProjection D (ScottMap D D) where
  incl := con
  retr := min
  retr_incl := fun _ => rfl
  incl_retr_le := fun f => by
    rw [ScottMap.le_def]
    intro y
    show ((ScottMap.const ((f : D → D) ⊥) : ScottMap D D) : D → D) y ≤ (f : D → D) y
    rw [ScottMap.const_apply]
    exact f.monotone bot_le

end Proposition313

end DomainTheory.ContinuousLattice
```

sInf_image_le_sInf_image_of_subset.lean

```
module
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

theorem sInf_image_le_sInf_image_of_subset {α : Type*} {f : α → D} {S T : Set α}
    (hST : S ⊆ T) : sInf (Set.image f T) ≤ sInf (Set.image f S) :=
```

```
  sInf_le_sInf fun _ hd => by
    obtain ⟨s, hs, rfl⟩ := hd
    exact ⟨s, hST hs, rfl⟩

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

sSupMaps.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.sSupMaps_continuous
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

noncomputable def sSupMaps {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (F : Set (ScottMap D D')) : ScottMap D D'
     :=
  ⟨fun x => sSup (Set.image (fun f : ScottMap D D' => (f : D → D') x) F),
    sSupMaps_continuous F⟩

end ScottMap

end DomainTheory.ContinuousLattice
```

sSupMaps_apply.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSupMaps
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice
```

```
open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

theorem sSupMaps_apply (F : Set (ScottMap D D')) (x : D) :
    (sSupMaps F : D → D') x =
      sSup (Set.image (fun f : ScottMap D D' => (f : D → D') x) F) :=
  rfl

end ScottMap
end DomainTheory.ContinuousLattice
```

sSupMaps_continuous.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    pointwise_sSup_preservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

/-- The pointwise supremum function is Scott-continuous. Kept as a named
proof boundary so the Challenge fixes the function while leaving this proof open. -/
theorem sSupMaps_continuous {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (F : Set (ScottMap D D')) :
    @Continuous D D' scottTopologicalSpace scottTopologicalSpace
      (fun x => sSup (Set.image (fun f : ScottMap D D' => (f : D → D') x) F)) :=
```

```
  continuous_of_preservesDirectedSup (pointwise_sSup_preservesDirectedSup F)

end ScottMap

end DomainTheory.ContinuousLattice
```

sSup_apply.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst30
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

theorem sSup_apply (F : Set (ScottMap D D')) (x : D) :
    ((sSup F : ScottMap D D') : D → D') x =
      sSup (Set.image (fun f : ScottMap D D' => (f : D → D') x) F) :=
  rfl

end ScottMap

end DomainTheory.ContinuousLattice
```

sSup_finsetSupOf.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.finsetSupOf
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_le_finsetSupOf
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_sSup_finsetSupOf
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace
```

```
variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

theorem sSup_finsetSupOf (S : Set D) : sSup S = sSup (finsetSupOf S) :=
  le_antisymm (le_sSup_finsetSupOf S) (sSup_le_finsetSupOf S)

end IsContinuousLatticeProjection

end Proposition310

end DomainTheory.ContinuousLattice
```

sSup_image_prod_mk_left.lean

```
module
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
```

```
    CompleteLattice D₁']

theorem sSup_image_prod_mk_left (x : D) (S : Set D') (hS : S.Nonempty) :
    sSup (Set.image (fun y => (x, y)) S) = (x, sSup S) := by
  have himage : Set.image (fun y => (x, y)) S = ({x} : Set D) ×ˢ S := by
    ext ⟨a, b⟩
    simp only [Set.mem_image, Set.mem_prod, Set.mem_singleton_iff]
    constructor
    · rintro ⟨y, hy, h⟩
      obtain ⟨ha, hb⟩ := Prod.ext_iff.mp h
      subst hb
      exact ⟨ha.symm, hy⟩
    · rintro ⟨ha, hb⟩
      refine ⟨b, hb, Prod.ext_iff.mpr ⟨ha.symm, rfl⟩⟩
  have hx : sSup ({x} : Set D) = x := by
    apply le_antisymm
    · exact sSup_le fun z hz => by rw [Set.mem_singleton_iff] at hz; rw [hz]
    · exact le_sSup (Set.mem_singleton x)
  rw [himage, sSup_prod (hs := ⟨x, Set.mem_singleton x⟩) (ht := hS), hx]

end DomainTheory.ContinuousLattice
```

sSup_image_prod_mk_right.lean

```
module
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']
```

```
theorem sSup_image_prod_mk_right (y : D') (S : Set D) (hS : S.Nonempty) :
    sSup (Set.image (fun x => (x, y)) S) = (sSup S, y) := by
  have himage : Set.image (fun x => (x, y)) S = S ×ˢ ({y} : Set D') := by
    ext ⟨a, b⟩
    simp only [Set.mem_image, Set.mem_prod, Set.mem_singleton_iff]
    constructor
    · rintro ⟨x, hx, h⟩
      obtain ⟨ha, hb⟩ := Prod.ext_iff.mp h
      subst ha
      exact ⟨hx, hb.symm⟩
    · rintro ⟨ha, hb⟩
      exact ⟨a, ha, Prod.ext_iff.mpr ⟨rfl, hb.symm⟩⟩
  have hy : sSup ({y} : Set D') = y := by
    apply le_antisymm
    · exact sSup_le fun z hz => by rw [Set.mem_singleton_iff] at hz; rw [hz]
    · exact le_sSup (Set.mem_singleton y)
  rw [himage, sSup_prod (hs := hS) (ht := ⟨y, Set.mem_singleton y⟩), hy]

end DomainTheory.ContinuousLattice
```

sSup_image_retr_wayBelow.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeRetraction
public import DomainTheory.ContinuousLattice.WayBelow.directedOn_wayBelow
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace IsContinuousLatticeRetraction

/-- For `d` in the retract, the ambient way-below set of `i(d)` pushed back by `j` is a
     directed
family whose supremum (computed in `D`) is `d`. This is the identity `j(⨆S′) = ⨆S`
    applied to
`S = {j(x') : x' ≪ i(d)}`, combined with continuity of `D'`. -/
theorem sSup_image_retr_wayBelow (R : IsContinuousLatticeRetraction D D')
    (hD' : ContinuousLattice D') (d : D) :
    sSup (Set.image (R.retr : D' → D) {x' | x' ≪ R.incl d}) = d := by
```

```
  have hne : ({x' : D' | x' ≪ R.incl d}).Nonempty := ⟨⊥, bot_wayBelow _⟩
  have hdir : DirectedOn (· ≤ ·) {x' : D' | x' ≪ R.incl d} := directedOn_wayBelow _
  rw [← R.retr.preservesDirectedSup_coe _ hne hdir, hD'.sSup_wayBelow (R.incl d), R.
    retr_incl d]

end IsContinuousLatticeRetraction
end DomainTheory.ContinuousLattice
```

sSup_le_finsetSupOf.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.finsetSupOf
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

theorem sSup_le_finsetSupOf (S : Set D) : sSup (finsetSupOf S) ≤ sSup S :=
  sSup_le fun z hz => by
    obtain ⟨T, hTS, rfl⟩ := hz
    exact Finset.sup_le fun t ht => le_sSup (hTS t ht)

end IsContinuousLatticeProjection

end Proposition310

end DomainTheory.ContinuousLattice
```

specializationLe_generateOpen.lean

```
module

public import DomainTheory.ContinuousLattice.Specialization.SpecializationLe
public import Mathlib.Topology.Order
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottFunctionEvalSubbasis
```

```
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem specializationLe_generateOpen (f g : C(X, Y)) (hfg : ∀ x, SpecializationLe (f x
    ) (g x))
    {V : Set (C(X, Y))} (hV : GenerateOpen (scottFunctionEvalSubbasis X Y) V) (hf : f
    ∈ V) :
    g ∈ V :=
  match hV with
  | .basic W hW => by
    obtain ⟨x, U, hU, rfl⟩ := hW
    exact hfg x U hU hf
  | .univ => hf
  | .inter s t hs ht => by
    rcases hf with ⟨hfs, hft⟩
    exact Set.mem_inter (specializationLe_generateOpen f g hfg hs hfs)
      (specializationLe_generateOpen f g hfg ht hft)
  | .sUnion S hS => by
    obtain ⟨W, hWS, hfW⟩ := Set.mem_sUnion.mp hf
    exact Set.mem_sUnion.mpr ⟨W, hWS, specializationLe_generateOpen f g hfg (hS W hWS)
    hfW⟩

end DomainTheory.ContinuousLattice
```

stepFun.lean

```
module
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section
```

```
namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- The (value of the) step function `e[e, e']` at `x`: `e'` if `e ≪ x`, else `⊥`. -/
noncomputable def stepFun (e : D) (e' : D') (x : D) : D' := ⋃ _ : e ≪ x, e'

end DomainTheory.ContinuousLattice
```

stepFun_of_not_wayBelow.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem stepFun_of_not_wayBelow {e : D} {e' : D'} {x : D} (h : ¬ e ≪ x) :
    stepFun e e' x = ⊥ :=
  iSup_neg h

end DomainTheory.ContinuousLattice
```

stepFun_of_wayBelow.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic
```

```
@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem stepFun_of_wayBelow {e : D} {e' : D'} {x : D} (h : e ≪ x) : stepFun e e' x = e
    ' :=
  iSup_pos h

end DomainTheory.ContinuousLattice
```

stepFun_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun_of_not_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun_of_wayBelow
public import Mathlib.Order.BourbakiWitt
public import DomainTheory.ContinuousLattice.WayBelow.scottOpen_wayBelow
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- Step functions are Scott-continuous: `{x | e ≪ x}` is Scott-open, so the
    conditional commutes
with directed suprema. -/
theorem stepFun_preservesDirectedSup (e : D) (e' : D') :
    PreservesDirectedSup (stepFun e e') := by
```

```
  intro S hS hSdir
  show stepFun e e' (sSup S) = sSup (Set.image (stepFun e e') S)
  by_cases h : e ≪ sSup S
  · rw [stepFun_of_wayBelow h]
    obtain ⟨s, hsS, hes⟩ := (scottOpen_wayBelow e).2 hS hSdir h
    apply le_antisymm
    · refine le_sSup_of_le (Set.mem_image_of_mem _ hsS) ?_
      rw [stepFun_of_wayBelow hes]
    · refine sSup_le ?_
      rintro _ ⟨x, _, rfl⟩
      exact iSup_le fun _ => le_rfl
  · rw [stepFun_of_not_wayBelow h]
    refine (sSup_eq_bot.2 ?_).symm
    rintro _ ⟨x, hxS, rfl⟩
    refine stepFun_of_not_wayBelow (fun hex => ?_)
    exact h (hex.trans_le (le_sSup hxS))

end DomainTheory.ContinuousLattice
```

stepMap.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    stepFun_preservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- The step function `e[e, e']` as a Scott map. -/
noncomputable def stepMap (e : D) (e' : D') : ScottMap D D' :=
  ⟨stepFun e e', continuous_of_preservesDirectedSup (stepFun_preservesDirectedSup e e')
    ⟩

end DomainTheory.ContinuousLattice
```

stepMap_apply_of_wayBelow.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun_of_wayBelow
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem stepMap_apply_of_wayBelow {e : D} {e' : D'} {x : D} (h : e ≪ x) :
    (stepMap e e' : D → D') x = e' :=
  stepFun_of_wayBelow h

end DomainTheory.ContinuousLattice
```

stepMap_le_of_wayBelow.lean

```
module
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_apply_of_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun_of_not_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst28
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- If `e' ≪ g e` then the step function `e[e, e']` lies below `g` (a pointwise
    argument). -/
theorem stepMap_le_of_wayBelow {e : D} {e' : D'} {g : ScottMap D D'}
    (h : e' ≪ (g : D → D') e) : stepMap e e' ≤ g := by
  rw [ScottMap.le_def]
  intro x
```

```
  by_cases hex : e ≪ x
  · rw [stepMap_apply_of_wayBelow hex]
    exact h.le.trans (g.monotone hex.le)
  · show stepFun e e' x ≤ (g : D → D') x
    rw [stepFun_of_not_wayBelow hex]
    exact bot_le

end DomainTheory.ContinuousLattice
```

stepMap_pointwise_sSup.lean

```
module
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice

public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.WayBelow.directedOn_wayBelow
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Pointwise reconstruction.** For a Scott map `f`, the supremum of the values of
    the step
functions below `f` recovers `f x`: `f x = ⊔ {e' | ∃ e ≪ x, e' ≪ f e}`. Uses
    continuity of `D`
(`x = ⊔{e | e ≪ x}`), Scott-continuity of `f`, and continuity of `D''`. -/
theorem stepMap_pointwise_sSup (hD : ContinuousLattice D) (hD' : ContinuousLattice D')
    (f : ScottMap D D') (x : D) :
    sSup {e' : D' | ∃ e, e ≪ x ∧ e' ≪ (f : D → D') e} = (f : D → D') x := by
  apply le_antisymm
  · refine sSup_le ?_
    rintro e' ⟨e, hex, he'⟩
    exact he'.le.trans (f.monotone hex.le)
  · rw [← hD'.sSup_wayBelow ((f : D → D') x)]
    refine sSup_le ?_
    intro w hw
    have hfx : (f : D → D') x = sSup (Set.image (f : D → D') {e | e ≪ x}) := by
      rw [← f.preservesDirectedSup_coe {e | e ≪ x} ⟨⊥, bot_wayBelow x⟩ (
    directedOn_wayBelow x),
        hD.sSup_wayBelow x]
    rw [hfx] at hw
    have hne : (Set.image (f : D → D') {e | e ≪ x}).Nonempty := ⟨f ⊥, ⊥, bot_wayBelow
```

```
     x, rfl⟩
    have hdir : DirectedOn (· ≤ ·) (Set.image (f : D → D') {e | e ≪ x}) := by
      rintro _ ⟨a, ha, rfl⟩ _ ⟨b, hb, rfl⟩
      obtain ⟨c, hc, hac, hbc⟩ := directedOn_wayBelow x a ha b hb
      exact ⟨f c, Set.mem_image_of_mem _ hc, f.monotone hac, f.monotone hbc⟩
    obtain ⟨_, ⟨e, hex, rfl⟩, hwe⟩ := (WayBelow.sSup_iff hne hdir).1 hw
    exact le_sSup ⟨e, hex, hwe⟩

end DomainTheory.ContinuousLattice
```

stepMap_wayBelow.lean

```
module
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_le_of_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **The step function is way below `f`.** If `e' ≪ f e`, then `e[e, e'] ≪ f` in `[D
    → D']`,
witnessed by the Scott-open set `{g | e' ≪ g e}` (open because suprema in `[D → D']`
    are pointwise
and `{· ≪ ·}` is inaccessible by directed suprema). -/
theorem stepMap_wayBelow {e : D} {e' : D'} {f : ScottMap D D'}
    (he' : e' ≪ (f : D → D') e) : stepMap e e' ≪ f := by
  refine ⟨{g : ScottMap D D' | e' ≪ (g : D → D') e}, ⟨?_, ?_⟩, he', ?_⟩
  · intro g g' hgg' hg
    exact hg.trans_le (hgg' e)
  · intro F hFne hFdir hmem
    simp only [Set.mem_ofPred_eq, ScottMap.sSup_apply] at hmem
    have hne : (Set.image (fun g : ScottMap D D' => (g : D → D') e) F).Nonempty :=
    hFne.image _
    have hdir : DirectedOn (· ≤ ·) (Set.image (fun g : ScottMap D D' => (g : D → D') e
    ) F) := by
      rintro _ ⟨a, ha, rfl⟩ _ ⟨b, hb, rfl⟩
      obtain ⟨c, hc, hac, hbc⟩ := hFdir a ha b hb
      exact ⟨(c : D → D') e, Set.mem_image_of_mem _ hc, hac e, hbc e⟩
    obtain ⟨_, ⟨g, hgF, rfl⟩, hg⟩ := (WayBelow.sSup_iff hne hdir).1 hmem
    exact ⟨g, hgF, hg⟩
  · intro g hg
```

```
    exact Set.mem_Ici.2 (stepMap_le_of_wayBelow hg)

end DomainTheory.ContinuousLattice
```

sup_apply.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

/-- The (completeLatticeOfSup-derived) binary join of Scott maps is computed pointwise
    . -/
theorem sup_apply (f g : ScottMap D D') (x : D) :
    ((f ⊔ g : ScottMap D D') : D → D') x = f x ⊔ g x := by
  have h : (f ⊔ g : ScottMap D D') = sSup ({f, g} : Set (ScottMap D D')) := rfl
  rw [h, sSup_apply, Set.image_pair]
  exact sSup_pair

end ScottMap

end DomainTheory.ContinuousLattice
```

theorem_3_3.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottMapPointwiseTopology
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import Mathlib.Topology.Defs.Basic

public import DomainTheory.ContinuousLattice.FunctionSpaces.theorem_3_3_more

@[expose] public section

namespace DomainTheory.ContinuousLattice
```

```
open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Theorem 3.3 (full statement).** For continuous lattices `D`, `D'`,
    the function
space `[D → D']` is a continuous lattice (`theorem_3_3.isContinuousLattice`) whose
    Scott topology
agrees with the topology of pointwise convergence (`theorem_3_3.topology`). -/
theorem theorem_3_3 (hD : ContinuousLattice D) (hD' : ContinuousLattice D') :
    ContinuousLattice (ScottMap D D') ∧
      (scottTopologicalSpace : TopologicalSpace (ScottMap D D')) =
    scottMapPointwiseTopology D D' :=
  ⟨theorem_3_3.isContinuousLattice hD hD', theorem_3_3.topology hD hD'⟩

end DomainTheory.ContinuousLattice
```

theorem_3_3_more.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_pointwise_sSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_apply_of_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    wayBelow_le_finset_sup_step
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    pointwiseSubbasic_scottOpen
public import DomainTheory.ContinuousLattice.FunctionSpaces.pointwiseSubbasic_isOpen
public import DomainTheory.ContinuousLattice.FunctionSpaces.wayBelow_finset_sup
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottMapPointwiseTopology
public import DomainTheory.ContinuousLattice.WayBelow.ScottOpen
public import DomainTheory.ContinuousLattice.WayBelow.exists_wayBelow_subset
public import DomainTheory.ContinuousLattice.WayBelow.scottOpen_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottMapPointwiseTopology_eq_induced_pi
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottMapInducedPiTopology

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace
```

```
variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Theorem 3.3 (order content).** If `D` and `D'` are continuous
    lattices, then so
is `[D → D']` under the pointwise order. Every `f` is the supremum of the step
    functions way below
it: `f = ⊔ {e[e,e'] | e' ≪ f e}`, and each such step function is way below `f`. -/
theorem theorem_3_3.isContinuousLattice (hD : ContinuousLattice D)
    (hD' : ContinuousLattice D') : ContinuousLattice (ScottMap D D') := by
  refine { out := ?_ }
  intro f
  refine ⟨fun h hh => hh.le, fun g hg => ?_⟩
  rw [ScottMap.le_def]
  intro x
  rw [← stepMap_pointwise_sSup hD hD' f x]
  refine sSup_le ?_
  rintro e' ⟨e, hex, he'⟩
  have hle : stepMap e e' ≤ g := hg (stepMap_wayBelow he')
  have hx := (ScottMap.le_def.1 hle) x
  rwa [stepMap_apply_of_wayBelow hex] at hx

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Theorem 3.3(b).** The Scott (lattice) topology on `[D → D']` agrees
    with the
topology of pointwise convergence. -/
theorem theorem_3_3.topology (hD : ContinuousLattice D) (hD' : ContinuousLattice D') :
    (scottTopologicalSpace : TopologicalSpace (ScottMap D D')) =
    scottMapPointwiseTopology D D' := by
  have hL : ContinuousLattice (ScottMap D D') := theorem_3_3.isContinuousLattice hD hD
    '
  apply le_antisymm
  · -- pointwise ⊆ Scott: each subbasic set is Scott-open in the lattice
    apply le_generateFrom_iff_subset_isOpen.2
    rintro W ⟨x, U, hUopen, rfl⟩
    exact ScottOpen.isOpen_iff.mpr (pointwiseSubbasic_scottOpen x (ScottOpen.
    isOpen_iff.mp hUopen))
  · -- Scott ⊆ pointwise: each Scott-open lattice set is pointwise-open, via the `⇑φ`
    basis
    intro U hU
    rw [ScottOpen.isOpen_iff] at hU
    rw [@isOpen_iff_forall_mem_open (ScottMap D D') (scottMapPointwiseTopology D D')]
    intro g hgU
    obtain ⟨φ, hφg, hφsub⟩ := exists_wayBelow_subset hL hU hgU
    obtain ⟨F, hF, hφF⟩ := wayBelow_le_finset_sup_step hD hD' hφg
    refine ⟨⋂ p ∈ F, {h : ScottMap D D' | (p.2 : D') ≪ (h : D → D') p.1}, ?_, ?_, ?_⟩
    · intro h hh
      refine hφsub (WayBelow.le_trans hφF (wayBelow_finset_sup fun p hp => ?_))
```

```
      exact stepMap_wayBelow (Set.mem_iInter₂.1 hh p hp)
    · exact @isOpen_biInter_finset (ScottMap D D') (D × D') (scottMapPointwiseTopology
    D D') F _
        (fun p _ => pointwiseSubbasic_isOpen p.1 (ScottOpen.isOpen_iff.mpr (
    scottOpen_wayBelow p.2)))
    · exact Set.mem_iInter₂.2 fun p hp => hF p hp

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- The literal product/subspace form of Theorem 3.3: the Scott lattice topology on `[D
     → D']`
is induced from Mathlib's Pi topology on the underlying functions. -/
theorem theorem_3_3.topology_induced_pi (hD : ContinuousLattice D)
    (hD' : ContinuousLattice D') :
    (scottTopologicalSpace : TopologicalSpace (ScottMap D D')) =
      scottMapInducedPiTopology D D' := by
  rw [theorem_3_3.topology hD hD', scottMapPointwiseTopology_eq_induced_pi]

end DomainTheory.ContinuousLattice
```

toOrderHom.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- The monotone map underlying a Scott map, suitable for `OrderHom.lfp`. -/
def toOrderHom (f : ScottMap D D) : D →o D := ⟨(f : D → D), f.monotone⟩

end Proposition314

end DomainTheory.ContinuousLattice
```

wayBelow_finset_sup.lean

```
module

public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- A finite sup of elements way below `g` is way below `g`. -/
theorem wayBelow_finset_sup {ι L : Type*} [CompleteLattice L] {s : Finset ι} {f : ι → L
    } {g : L}
    (h : ∀ i ∈ s, f i ≪ g) : s.sup f ≪ g :=
  Finset.sup_induction (p := fun a => a ≪ g) (bot_wayBelow g) (fun _ ha _ hb => ha.sup
    hb) h

end DomainTheory.ContinuousLattice
```

wayBelow_le_finset_sup_step.lean

```
module
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice

public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_pointwise_sSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_le_of_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_apply_of_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic
```

```
@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Step-function decomposition of `≪`.** If `φ ≪ g` in `[D → D']`, then `φ` lies
    below a finite
join of step functions `e[eᵢ,eᵢ']` with `eᵢ' ≪ g eᵢ`. The finite joins of such step
    functions form
a directed family with supremum `g`, so `WayBelow.sSup_iff` produces one above `φ`. -/
theorem wayBelow_le_finset_sup_step (hD : ContinuousLattice D) (hD' :
    ContinuousLattice D')
    {φ g : ScottMap D D'} (h : φ ≪ g) :
    ∃ F : Finset (D × D'), (∀ p ∈ F, (p.2 : D') ≪ (g : D → D') p.1) ∧
      φ ≤ F.sup (fun p => stepMap p.1 p.2) := by
  classical
  set Sg : Set (ScottMap D D') :=
    (fun F : Finset (D × D') => F.sup (fun p => stepMap p.1 p.2)) ''
      {F | ∀ p ∈ F, (p.2 : D') ≪ (g : D → D') p.1} with hSg
  have hSgne : Sg.Nonempty := by
    refine ⟨_, ∅, ?_, rfl⟩
    intro p hp
    exact absurd hp (Finset.notMem_empty p)
  have hSgdir : DirectedOn (· ≤ ·) Sg := by
    rintro _ ⟨F₁, hF₁, rfl⟩ _ ⟨F₂, hF₂, rfl⟩
    refine ⟨(F₁ ∪ F₂).sup (fun p => stepMap p.1 p.2), ⟨F₁ ∪ F₂, fun p hp => ?_, rfl⟩,
      Finset.sup_mono Finset.subset_union_left, Finset.sup_mono Finset.
    subset_union_right⟩
    rcases Finset.mem_union.1 hp with hp | hp
    · exact hF₁ p hp
    · exact hF₂ p hp
  have hSgsup : sSup Sg = g := by
    apply le_antisymm
    · refine sSup_le ?_
      rintro _ ⟨F, hF, rfl⟩
      exact Finset.sup_le fun p hp => stepMap_le_of_wayBelow (hF p hp)
    · rw [ScottMap.le_def]
      intro x
      rw [← stepMap_pointwise_sSup hD hD' g x]
      refine sSup_le ?_
      rintro e' ⟨e, hex, he'⟩
      have hmemSg : stepMap e e' ∈ Sg := by
        refine ⟨{(e, e')}, fun p hp => ?_, ?_⟩
        · rw [Finset.mem_singleton] at hp; subst hp; exact he'
        · show ({(e, e')} : Finset (D × D')).sup (fun p => stepMap p.1 p.2) = stepMap e
     e'
          rw [Finset.sup_singleton]
      have hx : (stepMap e e' : D → D') x ≤ ((sSup Sg : ScottMap D D') : D → D') x :=
     by
```

```
        rw [ScottMap.sSup_apply]
        exact le_sSup ⟨stepMap e e', hmemSg, rfl⟩
      rwa [stepMap_apply_of_wayBelow hex] at hx
  rw [← hSgsup] at h
  obtain ⟨_, ⟨F, hF, rfl⟩, hφs⟩ := (WayBelow.sSup_iff hSgne hSgdir).1 h
  exact ⟨F, hF, hφs.le⟩

end DomainTheory.ContinuousLattice
```

1.4.4 Corollary 3.4

3.4 Corollary. For continuous lattices D and D', the evaluation map:

$$\mathrm{eval} : [D \to D'] \times D \to D'$$

is continuous.

Proof: Here $\mathrm{eval}(f, x) = f(x)$. With f fixed, this is obviously continuous. With x fixed, we proved the continuity above with our calculation of $\bigsqcup \mathcal{F}$ in view of 2.5. Hence applying 5.3 and 2.6, we conclude that eval is jointly continuous. □

This result gives only one example of the masses of continuous functions that are available on continuous lattices. As another fundamental example we have:

corollary_3_4.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.Basic
public import DomainTheory.ContinuousLattice.FunctionSpaces.corollary_3_4_eval_on_C
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottFunctionTopology
public import Mathlib.Data.Set.Basic
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Corollary 3.4 (fixed `x`).** Evaluation at fixed `x` is continuous on
     `[D → D']`
(with Scott topologies on `D` and `D'`); this is one half of the separate-continuity
    input to
joint continuity. -/
theorem corollary_3_4 (x : D) :
    @Continuous (ScottC D D') D'
      (@scottFunctionTopology D D' scottTopologicalSpace scottTopologicalSpace)
      scottTopologicalSpace (fun f : ScottC D D' => f x) :=
```

```
  corollary_3_4_eval_on_C (D := D) (D' := D') x

end DomainTheory.ContinuousLattice
```

corollary_3_4_eval_on_C.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.Basic
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottFunctionSubbasis_isOpen_scott
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottFunctionTopology
public import Mathlib.Data.Set.Basic
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem corollary_3_4_eval_on_C (x : D) :
    @Continuous (ScottC D D') D'
      (@scottFunctionTopology D D' scottTopologicalSpace scottTopologicalSpace)
      scottTopologicalSpace (fun f : ScottC D D' => f x) :=
  continuous_def.2 fun _U hU => scottFunctionSubbasis_isOpen_scott (D := D) (D' := D')
    (x := x) hU

end DomainTheory.ContinuousLattice
```

corollary_3_4_jointly_continuous.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    corollary_3_4_preservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
```

```
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Corollary 3.4.** The evaluation map `eval : [D → D'] × D → D'`, `(f
    , x) ↦ f x`,
is (jointly) Scott-continuous. Via Theorem 3.3(b) and Proposition 2.9(b) the Scott
    topology of the
product lattice is the product of the pointwise topology on `[D → D']` and the Scott
    topology on
`D`, so this is exactly joint continuity for Scott's product topology. -/
theorem corollary_3_4_jointly_continuous :
    @Continuous (ScottMap D D' × D) D' scottTopologicalSpace scottTopologicalSpace
      (fun p : ScottMap D D' × D => (p.1 : D → D') p.2) :=
  continuous_of_preservesDirectedSup corollary_3_4_preservesDirectedSup

end DomainTheory.ContinuousLattice
```

corollary_3_4_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.Basic
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.ScottMaps.proposition_2_6
public import DomainTheory.ContinuousLattice.ScottMaps.proposition_2_5
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Corollary 3.4 (joint continuity, core).** Evaluation `[D → D'] × D
    → D'`,
`(f, x) ↦ f x`, preserves directed suprema. By Proposition 2.6 it suffices to check
    separate
Scott-continuity: in `x` (with `f` fixed) it is `f`'s own continuity, and in `f` (with `
    x` fixed)
it is the pointwise formula for suprema in `[D → D']` (`ScottMap.sSup_apply`). -/
theorem corollary_3_4_preservesDirectedSup :
    PreservesDirectedSup (fun p : ScottMap D D' × D => (p.1 : D → D') p.2) := by
  rw [proposition_2_6]
  constructor
  · intro y S hS hSdir
```

```
    show ((sSup S : ScottMap D D') : D → D') y
        = sSup ((fun x : ScottMap D D' => (x : D → D') y) '' S)
    rw [ScottMap.sSup_apply]
  · intro x
    exact (proposition_2_5 _).mp x.continuous

end DomainTheory.ContinuousLattice
```

scottFunctionEvalSubbasis.lean

```
module
public import Mathlib.Topology.ContinuousMap.Defs
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- Subbasic sets for Scott's function-space topology (pointwise convergence). -/
def scottFunctionEvalSubbasis (X Y : Type*) [TopologicalSpace X] [TopologicalSpace Y]
    :
    Set (Set (C(X, Y))) :=
  { W | ∃ (x : X) (U : Set Y), IsOpen U ∧ W = {f : C(X, Y) | f x ∈ U} }

end DomainTheory.ContinuousLattice
```

1.4.5 Proposition 3.5

3.5 Proposition. For continuous lattices D, D', and D'', the map of functional abstraction:

$$\text{lambda} : [[D \times D'] \to D''] \to [D \to [D' \to D'']]$$

is continuous.

Proof: Here lambda is defined by:

$$\mathrm{lambda}(f)(x)(y) = f(x,y)$$

where $f \in [[D \times D'] \to D'']$ and $x \in D$ and $y \in D'$. What is particularly interesting here is that by virtue of 3.3 we are making use of $[D \to [D' \to D'']]$ as a continuous lattice. The principle being stated here can be formulated more broadly in this way:

> If an expression $\mathcal{E}(x,y,z,\dots)$ is continuous in all its variables $x,y,z,\dots$ with values in D' as x ranges in D, then the expression
>
> $$\lambda x : D.\mathcal{E}(x,y,z,\dots)$$
>
> with values in $[D \to D']$ is continuous in the remaining variables $y,z,\dots$.

The λ-notation is a notation for functions, where in the above the variable after the λ is the *argument* and the expression after the . is the *value* (as a function of the argument). Thus we could write:

$$\mathrm{lambda} = \lambda f : [[D \times D'] \to D''].\lambda x : D.\lambda y : D'.f(x,y),$$

and, because $f(x,y)$ is continuous in f, x, and y, our conclusion follows. But often it is more readable not to write equations between functions but rather equations between values for definitional purposes.

The proof of the principle is easy. For let the variable y, say range over D'' and let $S \subseteq D''$ be a directed subset. Then

$$\lambda x : D.\mathcal{E}(x,\bigsqcup S,z,\dots) = \lambda x : D.\bigsqcup \mathcal{E}(x,y,z,\dots) : y \in S \quad = \bigsqcup \lambda x : D.\mathcal{E}(x,y,z,\dots) : y \in S,$$

because the lubs of functions are computed pointwise. □

We need not enumerate the many corollaries that follow easily now from this result. We mention, however, that composition $f \circ g$ of functions (on continuous lattices) is continuous in the two function variables, where we write

$$(f \circ g)(x) = f(g(x)).$$

curry_left_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_image_prod_mk_right
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice
```

```
open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- Currying in the `x`-variable: `x ↦ f (x, y)` is Scott-continuous (used for
    continuity of
`lambda f` as a map `D → [D' → D'']`). -/
theorem curry_left_preservesDirectedSup (f : ScottMap (D × D') D'') (y : D') :
    PreservesDirectedSup (fun x => f (x, y)) := by
  intro S hS hSdir
  have hS' : DirectedOn (· ≤ ·) (Set.image (fun x => (x, y)) S) := by
    intro p hp q hq
    obtain ⟨a, ha, rfl⟩ := hp
    obtain ⟨b, hb, rfl⟩ := hq
    obtain ⟨c, hc, hac, hbc⟩ := hSdir a ha b hb
    exact ⟨(c, y), Set.mem_image_of_mem _ hc, And.intro (And.intro hac le_rfl) (And.
    intro hbc le_rfl)⟩
  have hS'ne : (Set.image (fun x => (x, y)) S).Nonempty := by
    obtain ⟨s, hs⟩ := hS
    exact ⟨(s, y), s, hs, rfl⟩
  rw [show (fun x => f (x, y)) (sSup S) = f (sSup S, y) from rfl,
    ← sSup_image_prod_mk_right y S hS,
    f.preservesDirectedSup_coe (Set.image (fun x => (x, y)) S) hS'ne hS']
  congr 1
  simp [Set.image_image]

end DomainTheory.ContinuousLattice
```

curry_right_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_image_prod_mk_left
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
```

```
variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem curry_right_preservesDirectedSup (f : ScottMap (D × D') D'') (x : D) :
    PreservesDirectedSup (fun y => f (x, y)) := by
  intro S hS hSdir
  have hS' : DirectedOn (· ≤ ·) (Set.image (fun y => (x, y)) S) := by
    intro p hp q hq
    obtain ⟨a, ha, rfl⟩ := hp
    obtain ⟨b, hb, rfl⟩ := hq
    obtain ⟨c, hc, hac, hbc⟩ := hSdir a ha b hb
    exact ⟨(x, c), Set.mem_image_of_mem _ hc, And.intro (And.intro le_rfl hac) (And.
    intro le_rfl hbc)⟩
  have hS'ne : (Set.image (fun y => (x, y)) S).Nonempty := by
    obtain ⟨s, hs⟩ := hS
    exact ⟨(x, s), s, hs, rfl⟩
  rw [show (fun y => f (x, y)) (sSup S) = f (x, sSup S) from rfl,
    ← sSup_image_prod_mk_left x S hS,
    f.preservesDirectedSup_coe (Set.image (fun y => (x, y)) S) hS'ne hS']
  congr 1
  simp [Set.image_image]

end DomainTheory.ContinuousLattice
```

lambda_outer_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottLambdaAt
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    curry_left_preservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']
```

```
/-- The outer half of currying: `x ↦ (y ↦ f (x, y))` preserves directed suprema (so `
    lambda f` is a
Scott map `D → [D' → D'']`). Equality in `[D' → D'']` is pointwise, reducing to `
    curry_left`. -/
theorem lambda_outer_preservesDirectedSup (f : ScottMap (D × D') D'') :
    PreservesDirectedSup (fun x => scottLambdaAt f x) := by
  intro S hS hSdir
  apply ScottMap.ext
  intro y
  show f (sSup S, y)
      = ((sSup (Set.image (fun x => scottLambdaAt f x) S) : ScottMap D' D'') : D' → D
    '') y
  rw [ScottMap.sSup_apply, Set.image_image]
  exact curry_left_preservesDirectedSup f y hS hSdir

end DomainTheory.ContinuousLattice
```

proposition_3_5.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    proposition_3_5_preservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottLambda
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Proposition 3.5.** Functional abstraction `lambda` is Scott-
    continuous. -/
theorem proposition_3_5 :
    @Continuous (ScottMap (D × D') D'') (ScottMap D (ScottMap D' D''))
      scottTopologicalSpace scottTopologicalSpace scottLambda :=
  continuous_of_preservesDirectedSup proposition_3_5_preservesDirectedSup

end DomainTheory.ContinuousLattice
```

proposition_3_5_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottLambda_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottLambda
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- `lambda` preserves directed suprema: both sides evaluate, pointwise at `(x, y)`, to
`⨆ {f (x, y) | f ∈ 𝓕}`, since suprema in every function lattice involved are pointwise
    . -/
theorem proposition_3_5_preservesDirectedSup :
    PreservesDirectedSup
      (scottLambda : ScottMap (D × D') D'' → ScottMap D (ScottMap D' D'')) := by
  intro 𝓕 h𝓕 h𝓕dir
  apply ScottMap.ext
  intro x
  apply ScottMap.ext
  intro y
  rw [scottLambda_apply, ScottMap.sSup_apply, ScottMap.sSup_apply, ScottMap.sSup_apply
    ,
    Set.image_image, Set.image_image]
  congr 1

end DomainTheory.ContinuousLattice
```

scottLambda.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottLambdaAt
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    lambda_outer_preservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace
```

```
variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Proposition 3.5.** Functional abstraction
`lambda : [[D × D'] → D''] → [D → [D' → D'']]`, `lambda f x y = f (x, y)`. By
    Theorem 3.3,
`[D → [D' → D'']]` is itself a continuous lattice, and `lambda f` is a Scott map. -/
noncomputable def scottLambda (f : ScottMap (D × D') D'') : ScottMap D (ScottMap D' D
    '') :=
  ⟨fun x => scottLambdaAt f x,
    continuous_of_preservesDirectedSup (lambda_outer_preservesDirectedSup f)⟩

end DomainTheory.ContinuousLattice
```

scottLambdaAt.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    curry_right_preservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Proposition 3.5 (right).** Currying in the `y`-variable is Scott-
    continuous. -/
noncomputable def scottLambdaAt (f : ScottMap (D × D') D'') (x : D) : ScottMap D' D''
    :=
  ⟨fun y => f (x, y), continuous_of_preservesDirectedSup (
    curry_right_preservesDirectedSup f x)⟩

end DomainTheory.ContinuousLattice
```

scottLambda_apply.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.scottLambda
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

@[simp] theorem scottLambda_apply (f : ScottMap (D × D') D'') (x : D) (y : D') :
    ((scottLambda f x : ScottMap D' D'') : D' → D'') y = (f : (D × D') → D'') (x, y)
    :=
  rfl

end DomainTheory.ContinuousLattice
```

1.4.6 Retractions and extensions

What will be useful will be to return at this point to a discussion of the injective properties of continuous lattices. If one continuous lattice is a subspace of another it is of course a retract. This relationship between spaces can be given by a pair of continuous maps

$$i : D \to D' \quad \text{and} \quad j : D' \to D,$$

where

$$j \circ i = \mathrm{id}_D = \lambda x : D.x.$$

The composition $i \circ j : D' \to i(D)$ is the retraction onto the subspace corresponding to D under i. Now if we have a diagram:

$$D \underset{j}{\overset{i}{\rightleftarrows}} D' f \searrow \qquad \swarrow \tilde{f} = f \circ j D''$$

the given continuous f is at once extendable from D to D' by the obvious definition of $\tilde{f}$. This $\tilde{f}$ is not the $\bar{f}$ used in the proof of 2.11, and it will be well to sort out the connections. On one side note that if f' is any function which extends f, then we have $f = f' \circ i$. But this implies

$$\tilde{f} = f \circ j = f' \circ i \circ j,$$

which shows that $\tilde{f}$ is a "degraded" version of f'. There is one situation where this type of degrading is especially nice.

1.4.7 Definition 3.6

3.6 Definition. A continuous lattice D is said to be a *projection* of a continuous lattice D' iff there are a pair of continuous maps

$$i : D \to D' \quad \text{and} \quad j : D' \to D$$

such that not only

$$j \circ i = \mathrm{id}_D,$$

but also

$$i \circ j \sqsubseteq \mathrm{id}_{D'}.$$

Thus, in case our retraction is a projection, we have $\tilde{f} \sqsubseteq f'$, which means that $\tilde{f}$ is the *minimal* extension of $f \in [D \to D'']$ to a function in $[D' \to D'']$. We will discuss the nature of $\tilde{f}$ in a moment. But before we do we pause to remark that the correspondence $f \rightsquigarrow \tilde{f}$ is *continuous*, and this fact is easily extended.

IsContinuousLatticeProjection.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeRetraction
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

universe u v

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Definition 3.6.** A *projection* of continuous lattices: a retract
    with
`i ∘ j ⊑ id`. -/
structure IsContinuousLatticeProjection (D : Type u) (D' : Type v)
    [CompleteLattice D] [CompleteLattice D']
```

```
    extends IsContinuousLatticeRetraction D D' where
  incl_retr_le : ∀ d,
    @LE.le D'
      (ChainCompletePartialOrder.instOfCompleteLattice (α := D')).toPartialOrder.
    toPreorder.toLE
      (incl (retr d)) d

end DomainTheory.ContinuousLattice
```

IsContinuousLatticeRetraction.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

universe u v

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Definition 3.6.** A *retraction* of continuous lattices. -/
structure IsContinuousLatticeRetraction (D : Type u) (D' : Type v)
    [CompleteLattice D] [CompleteLattice D'] where
  incl : ScottMap D D'
  retr : ScottMap D' D
  retr_incl : ∀ d, retr (incl d) = d

end DomainTheory.ContinuousLattice
```

IsProjection.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.comp
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst28
public import DomainTheory.ContinuousLattice.FunctionSpaces.idMap
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section
```

```
namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

/-- **Scott 1972, Definition 3.11.** `J_D = { j ∈ [D → D] : j = j ∘ j ⊑ id }`: the
    Scott-continuous
projections of `D` (idempotent self-maps below the identity), as a predicate on `[D →
    D]`. -/
def IsProjection (j : ScottMap D D) : Prop :=
  j.comp j = j ∧ j ≤ ScottMap.idMap

end Projections

end DomainTheory.ContinuousLattice
```

1.4.8 Proposition 3.7

3.7 Proposition. Suppose the two pairs of maps

$$i_n : D_n \to D'_n \quad \text{and} \quad j_n : D'_n \to D_n$$

for $n = 0, 1$ make D_n a retraction (projection) of D'_n. Then $[D_0 \to D_1]$ is also a retraction (projection) of $[D'_0 \to D'_1]$ by means of the pair of maps:

$$\vec{i}(f) = i_1 \circ f \circ j_0 \text{ , and}$$

$$\vec{j}(f') = j_1 \circ f' \circ i_0 \text{ ,}$$

where $f \in [D_0 \to D_1]$ and $f' \in [D'_0 \to D'_1]$. □

Returning now to $\bar{f}$ we can prove:

`conversеProjection.lean`

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_converseRetr_le
public import DomainTheory.ContinuousLattice.FunctionSpaces.converseRetr_incl
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    converseRetr_preservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- The projection assembled from a map `i` satisfying 3.10(i)-(iii). -/
noncomputable def converseProjection
    (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S)) (hinj : Function.Injective i)
    (hwb : ∀ {x y : D}, x ≪ y → i x ≪ i y) (hD : ContinuousLattice D) :
    IsContinuousLatticeProjection D D' where
  incl := ⟨i, continuous_of_preservesDirectedSup (fun _ _ _ => hi _)⟩
  retr := ⟨converseRetr i, continuous_of_preservesDirectedSup
    (converseRetr_preservesDirectedSup hi hwb hD)⟩
  retr_incl := fun d => converseRetr_incl hi hinj d
  incl_retr_le := fun x' => incl_converseRetr_le hi x'

end Proposition310

end DomainTheory.ContinuousLattice
```

converseRetr.lean

```
module
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
```

```
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- **Scott 1972, Proposition 3.10(iv).** Scott's candidate retraction `j(x') = ⊔ { x
    | i(x) ⊑ x' }`. -/
noncomputable def converseRetr (i : D → D') (x' : D') : D :=
  sSup {x | i x ≤ x'}

end Proposition310

end DomainTheory.ContinuousLattice
```

converseRetr_incl.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.converseRetr
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_of_incl_le
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_mono_of_preservesSSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]
```

```
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- `j ∘ i = id` (uses (i)+(ii)): `{x | i x ⊑ i y} = Iic y`, whose join is `y`. -/
theorem converseRetr_incl (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S))
    (hinj : Function.Injective i) (y : D) : converseRetr i (i y) = y := by
  have hset : {x | i x ≤ i y} = Set.Iic y := by
    ext x
    simp only [Set.mem_ofPred_eq, Set.mem_Iic]
    exact ⟨le_of_incl_le hi hinj, fun hx => incl_mono_of_preservesSSup hi hx⟩
  show sSup {x | i x ≤ i y} = y
  rw [hset]
  exact le_antisymm (sSup_le fun _ hx => hx) (le_sSup le_rfl)

end Proposition310

end DomainTheory.ContinuousLattice
```

converseRetr_mono.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.converseRetr
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}
```

```
theorem converseRetr_mono (i : D → D') : Monotone (converseRetr i) := by
  intro a b hab
  exact sSup_le_sSup fun _ hx => le_trans hx hab

end Proposition310

end DomainTheory.ContinuousLattice
```

converseRetr_preservesDirectedSup.lean

```
module
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.converseRetr
public import DomainTheory.ContinuousLattice.FunctionSpaces.converseRetr_mono
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- `j` is Scott-continuous (uses (i)+(iii) and continuity of `D`). For directed `S'`:
    monotonicity
gives `⊔ j''S' ⊑ j(⊔S')`; conversely if `i x ⊑ ⊔S'` then for every `z ≪ x` we have `i
    z ≪ i x ⊑ ⊔S'`,
so `i z ⊑ x'` for some `x' ∈ S'` (directedness), whence `z ⊑ j(x') ⊑ ⊔ j''S'`;
    continuity of `D`
then gives `x ⊑ ⊔ j''S'`. -/
theorem converseRetr_preservesDirectedSup
    (_hi : ∀ S : Set D, i (sSup S) = sSup (i '' S))
    (hwb : ∀ {x y : D}, x ≪ y → i x ≪ i y) (hD : ContinuousLattice D) :
    PreservesDirectedSup (converseRetr i) := by
  intro S' hS'ne hS'dir
  apply le_antisymm
```

```
  · show sSup {x | i x ≤ sSup S'} ≤ sSup (converseRetr i '' S')
    apply sSup_le
    intro x hx
    have hx' : i x ≤ sSup S' := hx
    rw [← hD.sSup_wayBelow x]
    apply sSup_le
    intro z hz
    have hiz : i z ≪ sSup S' := (hwb hz).trans_le hx'
    obtain ⟨x', hx'S', hzx'⟩ := (WayBelow.sSup_iff hS'ne hS'dir).1 hiz
    have hz_mem : z ∈ {x | i x ≤ x'} := hzx'.le
    exact le_trans (le_sSup hz_mem) (le_sSup ⟨x', hx'S', rfl⟩)
  · apply sSup_le
    rintro _ ⟨x', hx'S', rfl⟩
    exact converseRetr_mono i (le_sSup hx'S')

end Proposition310

end DomainTheory.ContinuousLattice
```

incl_bot.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.retr_bot
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

/-- **Scott 1972, Proposition 3.10(i), empty case.** `i(⊥) = ⊥`. -/
theorem incl_bot (P : IsContinuousLatticeProjection D D') : P.incl (⊥ : D) = ⊥ :=
  le_antisymm (by simpa [retr_bot P] using P.incl_retr_le (⊥ : D')) bot_le

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

incl_converseRetr_le.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.converseRetr
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- `i ∘ j ⊑ id` (uses only (i)). -/
theorem incl_converseRetr_le (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S)) (x' : D') :
    i (converseRetr i x') ≤ x' := by
  show i (sSup {x | i x ≤ x'}) ≤ x'
  rw [hi]
  exact sSup_le (by rintro _ ⟨x, hx, rfl⟩; exact hx)

end Proposition310

end DomainTheory.ContinuousLattice
```

incl_finset_sup.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_sup
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_bot
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
```

```
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

theorem incl_finset_sup (P : IsContinuousLatticeProjection D D') (T : Finset D) :
    P.incl (T.sup (fun x => x)) = T.sup (fun x => P.incl x) := by
  classical
  refine Finset.induction_on T ?base ?step
  · simp only [sup_empty]
    exact incl_bot P
  · intro a s _ ih
    simp only [sup_insert]
    rw [incl_sup P, ih]

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

incl_injective.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

/-- **Scott 1972, Proposition 3.10(ii).** The inclusion map of a projection is
    injective. -/
theorem incl_injective (P : IsContinuousLatticeProjection D D') {x y : D} (h : P.incl
    x = P.incl y) : x = y := by
```

```
  rw [← P.retr_incl x, ← P.retr_incl y, h]

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

incl_mono_of_preservesSSup.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_sup_of_preservesSSup
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- From (i): `i` is monotone. -/
theorem incl_mono_of_preservesSSup (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S)) :
    Monotone i := by
  intro x y hxy
  have h := incl_sup_of_preservesSSup hi x y
  rw [sup_eq_right.mpr hxy] at h
  rw [h]; exact le_sup_left

end Proposition310

end DomainTheory.ContinuousLattice
```

incl_sSup.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_finset_sup
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_bot
public import DomainTheory.ContinuousLattice.FunctionSpaces.directedOn_finsetSupOf
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_finsetSupOf
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaces.mem_finsetSupOf_of_mem
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

/-- **Scott 1972, Proposition 3.10(i).** Projections preserve arbitrary suprema. -/
theorem incl_sSup (P : IsContinuousLatticeProjection D D') (S : Set D) :
    P.incl (sSup S) = sSup (Set.image (P.incl : D → D') S) := by
  rcases S.eq_empty_or_nonempty with hS | hS
  · subst hS
    simp only [sSup_empty, Set.image_empty, incl_bot P]
  · rw [sSup_finsetSupOf S]
    have hdir := directedOn_finsetSupOf S
    have hne : (finsetSupOf S).Nonempty := by
      obtain ⟨x, hx⟩ := hS
      exact ⟨x, mem_finsetSupOf_of_mem hx⟩
    rw [P.incl.preservesDirectedSup_coe (finsetSupOf S) hne hdir]
    apply le_antisymm
    · apply sSup_le
      intro b hb
      obtain ⟨z, hz, rfl⟩ := hb
      obtain ⟨T, hT, rfl⟩ := hz
      have h := incl_finset_sup P T
      conv_lhs => rw [show T.sup id = T.sup (fun x => x) from rfl]
      rw [h]
      apply Finset.sup_le
      intro t ht
      exact le_sSup (Set.mem_image_of_mem (P.incl : D → D') (hT t ht))
    · apply sSup_le
      intro b hb
      obtain ⟨x, hxS, rfl⟩ := (Set.mem_image (P.incl : D → D') S b).1 hb
      exact le_sSup (Set.mem_image_of_mem (P.incl : D → D') (mem_finsetSupOf_of_mem
    hxS))

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

incl_sup.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

/-- **Scott 1972, Proposition 3.10(i), binary case.** `i(x ⊔ y) = i(x) ⊔ i(y)`. -/
theorem incl_sup (P : IsContinuousLatticeProjection D D') (x y : D) :
    P.incl (x ⊔ y) = P.incl x ⊔ P.incl y := by
  apply le_antisymm
  · have hx : x ≤ P.retr (P.incl x ⊔ P.incl y) :=
      le_trans (le_of_eq (P.retr_incl x).symm) (P.retr.monotone le_sup_left)
    have hy : y ≤ P.retr (P.incl x ⊔ P.incl y) :=
      le_trans (le_of_eq (P.retr_incl y).symm) (P.retr.monotone le_sup_right)
    exact le_trans (P.incl.monotone (sup_le hx hy)) (P.incl_retr_le (P.incl x ⊔ P.incl
     y))
  · exact sup_le (P.incl.monotone le_sup_left) (P.incl.monotone le_sup_right)

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

incl_sup_of_preservesSSup.lean

```
module
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
```

```
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- From (i): `i` preserves binary joins (Scott checks (i) on two-element sets). -/
theorem incl_sup_of_preservesSSup (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S)) (x y :
     D) :
    i (x ⊔ y) = i x ⊔ i y := by
  calc i (x ⊔ y) = i (sSup {x, y}) := by rw [sSup_pair]
    _ = sSup (i '' {x, y}) := hi _
    _ = sSup {i x, i y} := by rw [Set.image_pair]
    _ = i x ⊔ i y := sSup_pair

end Proposition310

end DomainTheory.ContinuousLattice
```

incl_wayBelow.lean

```
module
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice

public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice
```

```
open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

/-- **Scott 1972, Proposition 3.10(iii).** Projections preserve the way-below relation
    . -/
theorem incl_wayBelow (P : IsContinuousLatticeProjection D D') (hD' :
    ContinuousLattice D')
    {x y : D} (h : x ≪ y) : @WayBelow D' _ (P.incl x) (P.incl y) := by
  set W : Set D' := { z' | @WayBelow D' _ z' (P.incl y) }
  have hWne : W.Nonempty := ⟨(⊥ : D'), @bot_wayBelow D' _ (P.incl y)⟩
  have hWdir : DirectedOn (· ≤ ·) W := by
    intro p hp q hq
    exact ⟨p ⊔ q, @WayBelow.sup D' _ p q (P.incl y) hp hq, le_sup_left, le_sup_right⟩
  have hy : y = sSup (Set.image (P.retr : D' → D) W) := by
    rw [← P.retr.preservesDirectedSup_coe W hWne hWdir, hD'.sSup_wayBelow (P.incl y),
      P.retr_incl y]
  have hImgne : (Set.image (P.retr : D' → D) W).Nonempty := by
    obtain ⟨z', hz'⟩ := hWne
    exact ⟨P.retr z', z', hz', rfl⟩
  have hImgdir : DirectedOn (· ≤ ·) (Set.image (P.retr : D' → D) W) := by
    intro p hp q hq
    obtain ⟨a, ha, rfl⟩ := hp
    obtain ⟨b, hb, rfl⟩ := hq
    refine ⟨P.retr (a ⊔ b), ⟨a ⊔ b, @WayBelow.sup D' _ a b (P.incl y) ha hb, rfl⟩, ?_,
    ?_⟩
    · exact P.retr.monotone le_sup_left
    · exact P.retr.monotone le_sup_right
  have hx : x ≪ sSup (Set.image (P.retr : D' → D) W) := by rwa [← hy]
  obtain ⟨prz, hprz, hxpr⟩ := (WayBelow.sSup_iff hImgne hImgdir).1 hx
  obtain ⟨z', hz', rfl⟩ := hprz
  exact @WayBelow.le_trans D' _ (P.incl x) z' (P.incl y)
    (le_trans (P.incl.monotone hxpr.le) (P.incl_retr_le z')) hz'

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

proposition_3_7_projection.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMapProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic
```

```
@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Proposition 3.7 (projection).** Function spaces inherit projections.
    -/
def proposition_3_7_projection (P₀ : IsContinuousLatticeProjection D₀ D₀')
    (P₁ : IsContinuousLatticeProjection D₁ D₁') :
    ScottMapProjection D₀ D₀' D₁ D₁' :=
  ScottMapProjection.functionSpace P₀ P₁

end DomainTheory.ContinuousLattice
```

proposition_3_7_retraction.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMapRetraction
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Proposition 3.7 (retraction).** Function spaces inherit retractions.
    -/
def proposition_3_7_retraction (R₀ : IsContinuousLatticeRetraction D₀ D₀')
    (R₁ : IsContinuousLatticeRetraction D₁ D₁') :
    ScottMapRetraction D₀ D₀' D₁ D₁' :=
  ScottMapRetraction.functionSpace R₀ R₁
```

```
end DomainTheory.ContinuousLattice
```

retr_ambientSSup_eq_sSup.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.ambientSSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace IsContinuousLatticeRetraction

/-- **Scott 1972, Prop 2.10 / March 1972 correction (p. 135).** For directed `S ⊆ D`,
`j(⊔S') = ⊔S`: the retraction sends the ambient join back to the subspace join. -/
theorem retr_ambientSSup_eq_sSup (R : IsContinuousLatticeRetraction D D') (S : Set D)
    (hS : S.Nonempty) (hdir : DirectedOn (· ≤ ·) S) :
    R.retr (ambientSSup R S) = sSup S := by
  unfold ambientSSup
  have hdir' : DirectedOn (· ≤ ·) (Set.image (R.incl : D → D') S) := by
    intro p hp q hq
    obtain ⟨a, ha, rfl⟩ := hp
    obtain ⟨b, hb, rfl⟩ := hq
    obtain ⟨c, hc, hac, hbc⟩ := hdir a ha b hb
    exact ⟨R.incl c, Set.mem_image_of_mem _ hc, R.incl.monotone hac, R.incl.monotone
    hbc⟩
  have hne' : (Set.image (R.incl : D → D') S).Nonempty := Set.image_nonempty.2 hS
  rw [R.retr.preservesDirectedSup_coe (Set.image (R.incl : D → D') S) hne' hdir']
  rw [Set.image_image, Set.image_congr fun a _ => R.retr_incl a]
  simp

end IsContinuousLatticeRetraction
end DomainTheory.ContinuousLattice
```

retr_bot.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic
```

```
@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

theorem retr_bot (P : IsContinuousLatticeProjection D D') : (P.retr : D' → D) ⊥ = ⊥
    := by
  have hle : (⊥ : D') ≤ P.incl (⊥ : D) := bot_le
  have heq : (P.retr : D' → D) (P.incl (⊥ : D)) = (⊥ : D) := P.retr_incl (d := (⊥ : D
    ))
  exact le_antisymm (le_trans (P.retr.monotone hle) (le_of_eq heq)) bot_le

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

retr_eq_sSup.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset
```

```
namespace IsContinuousLatticeProjection

/-- **Scott 1972, Proposition 3.10(iv), uniqueness.** The retraction of any projection
    is forced to
be Scott's formula `j(x') = ⊔ { x | i(x) ⊑ x' }`. `≤`: `j(x')` itself satisfies `i(j x
    ') ⊑ x'`
(by `i ∘ j ⊑ id`), so it is a member of the set; `≥`: each member `x` (with `i x ⊑ x'`)
    satisfies
`x = j(i x) ⊑ j(x')` by `j ∘ i = id` and monotonicity. -/
theorem retr_eq_sSup (P : IsContinuousLatticeProjection D D') (x' : D') :
    (P.retr : D' → D) x' = sSup {x | (P.incl : D → D') x ≤ x'} := by
  apply le_antisymm
  · exact le_sSup (show (P.incl : D → D') (P.retr x') ≤ x' from P.incl_retr_le x')
  · refine sSup_le fun x hx => ?_
    calc x = (P.retr : D' → D) (P.incl x) := (P.retr_incl x).symm
      _ ≤ (P.retr : D' → D) x' := P.retr.monotone hx

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

retr_wayBelow_of_wayBelow_incl.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_preservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeRetraction
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import DomainTheory.ContinuousLattice.ScottMaps.scottOpen_preimage
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace IsContinuousLatticeRetraction

/-- **Heart of Scott's proof of 2.10.** If `x' ≪ i(d)` in the ambient continuous
    lattice `D'`,
then its image `j(x')` is way below `d` in the retract `D`. The Scott-open witness in `
    D` is the
preimage `i^-1V'` of an ambient Scott-open witness `V'` (Scott-open because `i`
    preserves directed
suprema); for `z ∈ i^-1V'` we have `x' ⊑ i(z)`, hence `j(x') ⊑ j(i(z)) = z`. No
    projection
```

```
hypothesis is needed -- `j ∘ i = id` and monotonicity suffice. -/
theorem retr_wayBelow_of_wayBelow_incl (R : IsContinuousLatticeRetraction D D')
    {d : D} {x' : D'} (hx' : x' ≪ R.incl d) : R.retr x' ≪ d := by
  obtain ⟨U', hU'open, hd_in, hU'sub⟩ := hx'
  refine ⟨(R.incl : D → D') ^-1' U', scottOpen_preimage R.incl_preservesDirectedSup hU
    'open,
    hd_in, fun z hz => ?_⟩
  have hle : x' ≤ R.incl z := Set.mem_Ici.1 (hU'sub hz)
  have hjle := R.retr.monotone hle
  rw [R.retr_incl z] at hjle
  exact Set.mem_Ici.2 hjle

end IsContinuousLatticeRetraction
end DomainTheory.ContinuousLattice
```

1.4.9 Proposition 3.8

3.8 Proposition. If D is a continuous lattice and $e : X \to Y$ a subspace embedding, then for each $f : X \to D$, the function $\bar{f} : Y \to D$ given by the formula

$$\bar{f}(y) = \bigsqcap\{\sqcap f(x) : e(x) \in U : y \in U\},$$

where U ranges over open subsets of Y and x over X, is the maximal extension of f to a function in the partially ordered set $[Y \to D]$.

Proof: We are saying that $\bar{f}$ is the maximal solution to the equation

$$f = \bar{f} \circ e.$$

We already know it is a solution, so let f' be any other. We have

$$f'(y) = \bigsqcap \sqcap f'(z) : z \in U : y \in U \quad \sqsubseteq \bigsqcap \sqcap f'(z) : z \in e(X) \cap U : y \in U = \bigsqcap \sqcap f'(e(x)) : e(x) \in U : y \in U \quad = \bigsqcap\{\sqcap$$

which establishes that $f' \sqsubseteq \bar{f}$. □

By the same argument we could show that $\bar{f}$ is the maximal solution of $\bar{f} \circ e \sqsubseteq f$. An interesting question is whether the correspondence $f \rightsquigarrow \bar{f}$ is continuous. I very much doubt it, but at this moment a counterexample escapes me. It is clear that the correspondence is monotonic, for if $f \sqsubseteq g$, then the formula of 3.8 shows that $\bar{f} \sqsubseteq \bar{g}$. This gives us a neat argument for the previous remark: if $g \circ e \sqsubseteq f$, then

$$\overline{g \circ e} \sqsubseteq \bar{f}.$$

But $g \circ e = \overline{g \circ e} \circ e$, so by 3.8, $g \sqsubseteq \overline{g \circ e}$, and $\bar{f}$ is thus maximal.

In the case that the range spaces are being extended, the following lemma relating the extensions will be very useful when we consider inverse limits.

proposition_3_8.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.Specialization.scottTopologicalSpace
```

```
public import DomainTheory.ContinuousLattice.Constructions.scottExtend_maximal
public import DomainTheory.ContinuousLattice.Constructions.scottExtend_continuous
public import DomainTheory.ContinuousLattice.Constructions.
    scottExtend_eq_of_continuous
public import DomainTheory.ContinuousLattice.Constructions.scottExtend
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

variable {D : Type*} [CompleteLattice D]

section InjectiveExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] {E : Type*} [
    CompleteLattice E]

/-- **Scott 1972, Proposition 3.8.** If `E` is a continuous lattice and `e : X → Y` a
    subspace
embedding, then for each continuous `f : X → E` the explicit formula `scottExtend e f`
     is the
*maximal extension* of `f` to `[Y → E]`: it is Scott-continuous (`
    scottExtend_continuous`), it
restricts to `f` along `e` (`scottExtend_eq_of_continuous`), and it dominates every
    continuous
solution of `f' ∘ e = f` (`scottExtend_maximal`). -/
theorem proposition_3_8 (hE : ContinuousLattice E) (e : X → Y) (he : IsEmbedding e)
    (f : X → E) (hf : @Continuous X E _ scottTopologicalSpace f) :
    @Continuous Y E _ scottTopologicalSpace (scottExtend e f)
      ∧ (∀ x, scottExtend e f (e x) = f x)
      ∧ (∀ f' : Y → E, @Continuous Y E _ scottTopologicalSpace f' → (∀ x, f' (e x) =
    f x) →
          ∀ y, f' y ≤ scottExtend e f y) :=
  ⟨scottExtend_continuous hE e f,
   fun x => scottExtend_eq_of_continuous hE e he f hf x,
   fun _f' hf' h_ext y => scottExtend_maximal hE e hf' h_ext y⟩

end InjectiveExtension
end DomainTheory.ContinuousLattice
```

scottSubspaceExtend.lean

```
module

public import DomainTheory.ContinuousLattice.Specialization.scottTopologicalSpace
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section
```

```
namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

/-- **Scott 1972, Proposition 3.8.** Scott's maximal subspace extension `f : Y → D`.
    -/
noncomputable def scottSubspaceExtend (e : X → Y) (f : X → D) (y : Y) : D :=
  sSup { d | ∃ U, @IsOpen Y scottTopologicalSpace U ∧ y ∈ U ∧ d =
    scottSubspaceExtendInf e f U }

end SubspaceExtension

end DomainTheory.ContinuousLattice
```

scottSubspaceExtendInf.lean

```
module
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
```

```
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

/-- **Scott 1972, Proposition 3.8.** The infimum term `⊓ { f(x) : e(x) ∈ U }` for open
    `U ⊆ Y`. -/
noncomputable def scottSubspaceExtendInf (e : X → Y) (f : X → D) (U : Set Y) : D :=
  sInf (f '' (e ^-1' U))

end SubspaceExtension

end DomainTheory.ContinuousLattice
```

scottSubspaceExtendInf_eq_of_comp.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] in
theorem scottSubspaceExtendInf_eq_of_comp {e : X → Y} {f : X → D} {U : Set Y}
    (f' : ScottMap Y D) (h : ∀ x, f' (e x) = f x) :
    scottSubspaceExtendInf e f U = sInf (Set.image (f' : Y → D) (Set.image e Set.univ
    ∩ U)) := by
  have hset : f '' (e ^-1' U) = Set.image (f' : Y → D) (Set.image e Set.univ ∩ U) :=
    by
    ext d
    simp only [Set.mem_image, Set.mem_preimage, Set.mem_inter_iff, Set.mem_univ,
    true_and]
    constructor
    · rintro ⟨x, hxU, heq⟩
      refine ⟨e x, ⟨?_, hxU⟩, ?_⟩
      · exact ⟨x, rfl⟩
```

```
      · rw [h x, heq]
    · rintro ⟨y, ⟨hex, hyU⟩, heq⟩
      obtain ⟨x, _, rfl⟩ := hex
      exact ⟨x, hyU, (h x).symm.trans heq⟩
  rw [scottSubspaceExtendInf, hset]

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

scottSubspaceExtendInf_eq_of_ext.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y] in
theorem scottSubspaceExtendInf_eq_of_ext {e : X → Y} {f g' : X → D} (h : ∀ x, f x = g
    ' x) (U : Set Y) :
    scottSubspaceExtendInf e f U = scottSubspaceExtendInf e g' U := by
  unfold scottSubspaceExtendInf
  congr 1
  ext d
  simp [Set.mem_image, Set.mem_preimage, h]

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

scottSubspaceExtendInf_le_of_mem.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice
```

```
open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace Y] in
theorem scottSubspaceExtendInf_le_of_mem {f' : ScottMap Y D} {U : Set Y} {y : Y}
    (hyU : y ∈ U) : sInf (Set.image (f' : Y → D) U) ≤ f' y :=
  sInf_le (Set.mem_image_of_mem _ hyU)

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

scottSubspaceExtendInf_mono.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y] in
theorem scottSubspaceExtendInf_mono {e : X → Y} {f g₀ : X → D} (hfg : ∀ x, f x ≤ g₀ x
    ) (U : Set Y) :
    scottSubspaceExtendInf e f U ≤ scottSubspaceExtendInf e g₀ U := by
  apply le_sInf
  intro d hd
  obtain ⟨x, hxU, rfl⟩ := hd
  exact le_trans (sInf_le ⟨x, hxU, rfl⟩) (hfg x)

end SubspaceExtension
```

```
end DomainTheory.ContinuousLattice
```

scottSubspaceExtendInf_mono_incl.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y] in
theorem scottSubspaceExtendInf_mono_incl {e : X → Y} {f : X → D} {g : X → D'}
    (P : IsContinuousLatticeProjection D D') (hfg : ∀ x, f x = P.retr (g x)) (U : Set Y
    ) :
    scottSubspaceExtendInf e (fun x => P.incl (f x)) U ≤ scottSubspaceExtendInf e g U
    := by
  apply le_sInf
  intro d hd
  obtain ⟨x, hxU, rfl⟩ := hd
  have hx : x ∈ e ^-1' U := Set.mem_preimage.mpr hxU
  have hmem : P.incl (f x) ∈ (fun x => P.incl (f x)) '' (e ^-1' U) :=
    Set.mem_image_of_mem _ hx
  have hincl : P.incl (f x) ≤ g x := by rw [hfg x]; exact P.incl_retr_le (g x)
  exact le_trans (sInf_le hmem) hincl

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

scottSubspaceExtendInf_mono_incl_apply.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic
```

```
@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y] in
theorem scottSubspaceExtendInf_mono_incl_apply {e : X → Y} {f : X → D} (U : Set Y)
    (P : IsContinuousLatticeProjection D D') :
    P.incl (scottSubspaceExtendInf e f U) ≤ scottSubspaceExtendInf e (fun x => P.incl
    (f x)) U := by
  apply le_sInf
  intro d hd
  obtain ⟨x, hxU, rfl⟩ := hd
  exact P.incl.monotone (sInf_le (Set.mem_image_of_mem f (Set.mem_preimage.mpr hxU)))

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

scottSubspaceExtendInf_mono_retr.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]
```

```
omit [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y] in
theorem scottSubspaceExtendInf_mono_retr {e : X → Y} {g : X → D'}
    (P : IsContinuousLatticeProjection D D') (U : Set Y) :
    P.retr (scottSubspaceExtendInf e g U) ≤ scottSubspaceExtendInf e (fun x => P.retr
    (g x)) U := by
  apply le_sInf
  intro d hd
  obtain ⟨x, hxU, rfl⟩ := hd
  have hx : x ∈ e ^-1' U := Set.mem_preimage.mpr hxU
  exact (P.retr.monotone (sInf_le (Set.mem_image_of_mem g hx))).trans le_rfl

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

scottSubspaceExtend_eq_of_ext.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottSubspaceExtendInf_eq_of_ext
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtend
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] in
theorem scottSubspaceExtend_eq_of_ext {e : X → Y} {f g' : X → D} (h : ∀ x, f x = g' x
    ) (y : Y) :
    scottSubspaceExtend e f y = scottSubspaceExtend e g' y := by
  unfold scottSubspaceExtend
  congr 1
  ext d
  simp [Set.mem_ofPred_eq, scottSubspaceExtendInf_eq_of_ext h]

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

scottSubspaceExtend_maximal.lean

```
module
```

```
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottSubspaceExtendInf_eq_of_comp
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottMap_eq_sSup_openInfs
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    sInf_image_le_sInf_image_of_subset
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtend
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

/-- **Scott 1972, Proposition 3.8 (subspace variant).** `f` (with the Scott topology on
     `Y`) is
the maximal extension along a subspace embedding. The faithful statement (arbitrary
    topology on
`Y`) is `proposition_3_8` in `Constructions.lean`. -/
theorem scottSubspaceExtend_maximal (hD : ContinuousLattice D) (e : X → Y) (_he :
    IsEmbedding e)
    (f : X → D) (f' : ScottMap Y D) (h_ext : ∀ x, f' (e x) = f x) (y : Y) :
    (f' : Y → D) y ≤ scottSubspaceExtend e f y := by
  have hEq := scottMap_eq_sSup_openInfs hD f' y
  have hmain :
      sSup { d | ∃ U, @IsOpen Y scottTopologicalSpace U ∧ y ∈ U ∧
        d = sInf (Set.image (f' : Y → D) U) } ≤ scottSubspaceExtend e f y := by
    unfold scottSubspaceExtend
    apply sSup_le
    intro d hd
    obtain ⟨U, hUopen, hyU, rfl⟩ := hd
    have hmem : scottSubspaceExtendInf e f U ∈
        { d | ∃ U, @IsOpen Y scottTopologicalSpace U ∧ y ∈ U ∧ d =
    scottSubspaceExtendInf e f U } :=
      ⟨U, hUopen, hyU, rfl⟩
    refine le_trans ?_ (le_sSup hmem)
    have hrestrict :
        sInf (Set.image (f' : Y → D) U) ≤
          sInf (Set.image (f' : Y → D) (Set.image e Set.univ ∩ U)) :=
      sInf_image_le_sInf_image_of_subset
        (show Set.image e Set.univ ∩ U ⊆ U from inter_subset_right)
    refine le_trans hrestrict (le_of_eq (scottSubspaceExtendInf_eq_of_comp (f' := f')
    h_ext).symm)
```

```
  exact le_trans (le_of_eq hEq) hmain

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

scottSubspaceExtend_mono.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottSubspaceExtendInf_mono
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtend
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] in
theorem scottSubspaceExtend_mono {e : X → Y} {f g₀ : X → D} (hfg : ∀ x, f x ≤ g₀ x) (
    y : Y) :
    scottSubspaceExtend e f y ≤ scottSubspaceExtend e g₀ y := by
  apply sSup_le
  intro d hd
  obtain ⟨U, hUopen, hyU, rfl⟩ := hd
  have hmem : scottSubspaceExtendInf e g₀ U ∈
      { d | ∃ U, @IsOpen Y scottTopologicalSpace U ∧ y ∈ U ∧ d =
    scottSubspaceExtendInf e g₀ U } :=
    ⟨U, hUopen, hyU, rfl⟩
  exact le_trans (scottSubspaceExtendInf_mono hfg U) (le_sSup hmem)

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

1.4.10 Lemma 3.9

3.9 Lemma. Consider the diagram:

$$X \xrightarrow{e} Y \quad f\big\downarrow \qquad g \searrow \quad \bar{f} \swarrow \qquad \downarrow \bar{g}D \xrightarrow[j]{i} D'$$

where the upper row is a subspace embedding and the lower is a projection. If the given functions f and g are extended to $\bar{f}$ and $\bar{g}$ as in 3.8, and if $f = j \circ g$, then $\bar{f} = j \circ \bar{g}$ also.

Proof: $\bar{f}$ and $\bar{g}$ are maximal solutions of $f = \bar{f} \circ e$ and $g = \bar{g} \circ e$. Therefore since

$$f = j \circ g = j \circ \bar{g} \circ e,$$

we see that

$$j \circ \bar{g} \sqsubseteq \bar{f}.$$

Note also that

$$i \circ \bar{f} \circ e = i \circ f = i \circ j \circ g \sqsubseteq g,$$

and so by the remark following 3.8, we have

$$i \circ \bar{f} \sqsubseteq \bar{g}.$$

Therefore

$$\bar{f} = j \circ i \circ \bar{f} \sqsubseteq j \circ \bar{g},$$

which proves the equality. □

Whether this lemma is true for retractions in any form, I do not know. My proof seems to require the stronger projection relationship. I suspect there may be difficulties. In general projections are better behaved than retractions. By the way the word projection seems to be properly used in 3.6, for the projection $j : D \times D' \to D$ of the Cartesian product of two continuous lattices onto the first factor *is* a projection with partial inverse $i : D \to D \times D'$ defined by

$$i(x) = (x, \bot)$$

for $x \in D$.

lemma_3_9.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.Basic
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.Constructions.scottExtend_maximal_le
public import DomainTheory.ContinuousLattice.Constructions.scottExtend_maximal
public import DomainTheory.ContinuousLattice.Constructions.scottExtend_continuous
public import DomainTheory.ContinuousLattice.Constructions.
    scottExtend_eq_of_continuous
public import DomainTheory.ContinuousLattice.Constructions.scottExtend
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice
```

```
open Topology Set

section Lemma39

variable {X Y D D' : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  [CompleteLattice D] [CompleteLattice D']

/-- **Scott 1972, Lemma 3.9.** With `e : X → Y` a subspace embedding and `i, j : D ⇄
    D'` a
projection on the range, if continuous `f : X → D` and `g : X → D'` satisfy `f = j ∘ g
    `, then the
maximal extensions (Proposition 3.8) satisfy `f = j ∘ g`.

The proof is Scott's, recast through the faithful `scottExtend`:
* `j ∘ g ⊑ f`: `j ∘ g` is a continuous solution of `(j ∘ g) ∘ e = j ∘ g = f`, so
    maximality of
  `f` (`scottExtend_maximal`) gives the bound;
* `i ∘ f ⊑ g`: `(i ∘ f) ∘ e = i ∘ f = i ∘ j ∘ g ⊑ g` (since `i ∘ j ⊑ id`), so the *sub
    *-solution
  maximality of `g` (`scottExtend_maximal_le`, the remark after 3.8) gives the bound;
* combining: `f = j ∘ i ∘ f ⊑ j ∘ g ⊑ f` (using `j ∘ i = id` and monotonicity of `j`).
    -/
theorem lemma_3_9 (hD : ContinuousLattice D) (hD' : ContinuousLattice D')
    (P : IsContinuousLatticeProjection D D') (e : X → Y) (he : IsEmbedding e)
    {f : X → D} {g : X → D'} (hf : @Continuous X D _ scottTopologicalSpace f)
    (hg : @Continuous X D' _ scottTopologicalSpace g)
    (hfg : ∀ x, f x = (P.retr : D' → D) (g x)) (y : Y) :
    scottExtend e f y = (P.retr : D' → D) (scottExtend e g y) := by
  have hfbar : @Continuous Y D _ scottTopologicalSpace (scottExtend e f) :=
    scottExtend_continuous hD e f
  have hgbar : @Continuous Y D' _ scottTopologicalSpace (scottExtend e g) :=
    scottExtend_continuous hD' e g
  -- `j ∘ g` and `i ∘ f` are continuous (Scott topology is not an instance; register it
     locally so
  -- `Continuous.comp` can fire, scoped to avoid the lattice `≤` clashing with
    specialization order).
  have hjg : @Continuous Y D _ scottTopologicalSpace (fun z => (P.retr : D' → D) (
    scottExtend e g z)) := by
    let : TopologicalSpace D := scottTopologicalSpace
    let : TopologicalSpace D' := scottTopologicalSpace
    exact P.retr.continuous.comp hgbar
  have hif : @Continuous Y D' _ scottTopologicalSpace (fun z => (P.incl : D → D') (
    scottExtend e f z)) := by
    let : TopologicalSpace D := scottTopologicalSpace
    let : TopologicalSpace D' := scottTopologicalSpace
    exact P.incl.continuous.comp hfbar
  -- Step A: `j ∘ g ⊑ f`.
  have hA : ∀ z, (P.retr : D' → D) (scottExtend e g z) ≤ scottExtend e f z := by
    intro z
    refine scottExtend_maximal hD e hjg ?_ z
    intro x
    show (P.retr : D' → D) (scottExtend e g (e x)) = f x
    rw [scottExtend_eq_of_continuous hD' e he g hg x, ← hfg x]
  -- Step B: `i ∘ f ⊑ g`.
  have hB : ∀ z, (P.incl : D → D') (scottExtend e f z) ≤ scottExtend e g z := by
```

```
    intro z
    refine scottExtend_maximal_le hD' e hif ?_ z
    intro x
    show (P.incl : D → D') (scottExtend e f (e x)) ≤ g x
    rw [scottExtend_eq_of_continuous hD e he f hf x, hfg x]
    exact P.incl_retr_le (g x)
  refine le_antisymm ?_ (hA y)
  calc scottExtend e f y
      = (P.retr : D' → D) ((P.incl : D → D') (scottExtend e f y)) :=
        (P.retr_incl (scottExtend e f y)).symm
    _ ≤ (P.retr : D' → D) (scottExtend e g y) := P.retr.monotone (hB y)

end Lemma39
end DomainTheory.ContinuousLattice
```

lemma_3_9_incl_inf.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottSubspaceExtendInf_mono_incl_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottSubspaceExtendInf_mono_incl
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y] in
/-- **Scott 1972, Lemma 3.9 (inclusion at infimum level).** Used in the inverse-limit
argument (Theorem 4.4). -/
theorem lemma_3_9_incl_inf {e : X → Y} (P : IsContinuousLatticeProjection D D')
    {f : X → D} {g : X → D'} (hfg : ∀ x, f x = P.retr (g x)) (U : Set Y) :
    P.incl (scottSubspaceExtendInf e f U) ≤ scottSubspaceExtendInf e g U :=
  le_trans (scottSubspaceExtendInf_mono_incl_apply (e := e) (f := f) U P)
    (scottSubspaceExtendInf_mono_incl (e := e) (f := f) (g := g) P hfg U)

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

lemma_3_9_retr_inf.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottSubspaceExtendInf_mono_retr
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y] in
/-- **Scott 1972, Lemma 3.9 (retraction at infimum level).** The global equality
`f = j ∘ g` assembles these infimum bounds via Proposition 3.10(i). -/
theorem lemma_3_9_retr_inf {e : X → Y} (P : IsContinuousLatticeProjection D D') {g : X
     → D'}
    (U : Set Y) :
    P.retr (scottSubspaceExtendInf e g U) ≤ scottSubspaceExtendInf e (fun x => P.retr
    (g x)) U :=
  scottSubspaceExtendInf_mono_retr (e := e) (g := g) P U

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

1.4.11 Proposition 3.10

3.10 Proposition. If the continuous lattice D is a projection of the continuous lattice D' via the pair of maps i, j; then for all $S \subseteq D$ and all $x, y \in D$ we have:

(i) $i(\bigsqcup S) = \bigsqcup\{i(x) : x \in S\}$,

(ii) $i(x) = i(y)$ implies $x = y$,

(iii) $x \ll y$ implies $i(x) \ll i(y)$.

Conversely, if a map $i : D \to D'$ satisfies (i)–(iii), then there exists a continuous $j : D' \to D$ making D a projection of D', and in fact j is uniquely determined by:

(iv) $j(x') = \bigsqcup\{x \in D : i(x) \sqsubseteq x'\}$

for all $x' \in D'$.

Proof: Equation (i) holds for directed $S \subseteq D$ because i is continuous. To have it hold for arbitrary S it is only necessary to check it for finite sets, because every lub is the directed lub of finite sublubs. (The last word of that sentence is an unfortunate accident.) Further, to check the equation for finite sets it is enough to check it for the empty set and for two element sets. Thus, $i(\bot) = \bot$, because $j(i(\bot)) = \bot$ and since $\bot \sqsubseteq i(\bot)$,

$$j(\bot) \sqsubseteq j(i(\bot)) = \bot,$$

so $j(\bot) = \bot$. Whence $i(\bot) = i(j(\bot)) \sqsubseteq \bot$. Next $i(x \sqcup y) = i(x) \sqcup i(y)$, because first

$$i(x) \sqcup i(y) \sqsubseteq i(x \sqcup y)$$

by monotonicity. Then note that

$$i(x) \sqsubseteq i(x) \sqcup i(y)$$

and so

$$x = j(i(x)) \sqsubseteq j(i(x) \sqcup i(y)).$$

Similarly

$$y \sqsubseteq j(i(x) \sqcup i(y)),$$

whence

$$x \sqcup y \sqsubseteq j(i(x) \sqcup i(y)).$$

But then

$$i(x \sqcup y) \sqsubseteq i(j(i(x) \sqcup i(y))) \sqsubseteq i(x) \sqcup i(y),$$

which completes the argument for equation (i).

Condition (ii) is obvious. For (iii) we argue as follows. Assume $x \ll y$. Since D' is continuous we can write:

$$i(y) = \bigsqcup\{z' \in D' : z' \ll i(y)\},$$

and conclude by the continuity of j that:

$$y = j(i(y)) = \bigsqcup j(z') : z' \ll i(y).$$

But $x \ll y$, so $x \ll j(z')$ for some $z' \ll i(y)$. Now $x \sqsubseteq j(z')$ follows; therefore $i(x) \sqsubseteq i(j(z')) \sqsubseteq z'$. Thus $i(x) \ll i(y)$.

Turning now to the converse, assume of the map i that it satisfies (i)–(iii). Compute:

$$i(j(x')) = \bigsqcup i(x) : i(x) \sqsubseteq x' \sqsubseteq x'.$$

This is correct because i is continuous and the set $\{x : i(x) \sqsubseteq x'\}$ is directed in view of condition (i). Thus $i \circ j \sqsubseteq \mathrm{id}_{D'}$. Note that by virtue of (i) and (ii) it is the case that

$$i(x) \sqsubseteq i(y) \text{ implies } x \sqsubseteq y.$$

(The reason is that $x \sqsubseteq y$ is equivalent to $x \sqcup y = y$.) This remark allows us to compute:

$$\begin{aligned} j(i(y)) &= \bigsqcup\{x : i(x) \sqsubseteq i(y)\} \\ &= \bigsqcup\{x : x \sqsubseteq y = y\}. \end{aligned}$$

Hence, $j \circ i = \mathrm{id}_D$. It remains to show that j is continuous. Suppose $S' \subseteq D'$ is directed. Since j is by definition monotonic, it is sufficient to prove that

$$j(\bigsqcup S') \sqsubseteq \bigsqcup j(x') : x' \in S'.$$

Now

$$j(\bigsqcup S') = \bigsqcup\{x : i(x) \sqsubseteq \bigsqcup S'\},$$

so suppose $i(x) \sqsubseteq \bigsqcup S'$. Let $z \ll x$; whence $i(z) \ll i(x)$. Thus $i(z) \ll x'$ for some $x' \in S'$, and therefore $i(z) \sqsubseteq x'$. We obtain then $z \sqsubseteq j(x')$, which means that

$$z \sqsubseteq \bigsqcup j(x') : x' \in S'$$

holds for all $z \ll x$. By the continuity of D we conclude

$$x \sqsubseteq \bigsqcup j(x') : x' \in S'$$

holds for all x with $i(x) \sqsubseteq \bigsqcup S'$. The desired result follows. □

As a corollary of 3.10 we can easily see which *subspaces* of a continuous lattice D' are projections of it. Such a subspace $D \subseteq D'$ must first be closed under $\bigsqcup$. That is, if $S \subseteq D$, then $\bigsqcup S \in D$ for

all S, where the lub is taken in the sense of D'. The identity map on D will then satisfy (i) and (ii). But this is not enough, since we would not know that D is a continuous lattice, nor whether (iii) holds. The following additional condition would be sufficient, if assumed for all $y \in D$:

$$y = \bigsqcup\{x \in D : x \ll y\},$$

where $\ll$ is taken in the sense of D'. This implies that

$$y = \bigsqcup\{\sqcap(D \cap U) : y \in U\}$$

where U ranges over the open subsets of D' and where the $\sqcap$ is taken in the sense of D. This condition makes the subspace topology the same as the lattice topology on D and besides makes D continuous, which is just what we need. (Another way to put it is that whenever $z \ll y$, where $y \in D$ but $z \in D'$, then $z \sqsubseteq x \ll y$, for some $x \in D$.)

It seems a bit troublesome to characterize in a simple way just which maps $j : D' \to D$ are projections. (Other than saying outright that the map $i : D \to D'$ such that for all $x \in D$:

$$i(x) = \sqcap\{x' \in D' : x \sqsubseteq j(x')\}$$

is the continuous partial inverse to j.) But we can say very easily which continuous maps $j : D' \to D'$ are projections onto *subspaces*; namely, we must have

$$j = j \circ j \sqsubseteq \mathrm{id}.$$

The subspace in question then is:

$$D = \{x \in D' : j(x) = x\}.$$

This non-empty subspace is the exact range of j and is closed under $\bigsqcup$. Let $y \in D$. Then if $x' \ll y$ in D', we find $j(x') \sqsubseteq x' \ll y$. Thus since

$$y = \bigsqcup\{x' \in D' : x' \ll y\},$$

we see that

$$y = j(y) = \bigsqcup\{j(x') : x' \ll y\}.$$

But each $j(x') \in D$, so $y = \bigsqcup\{x \in D : x \ll y\}$, as desired.

The foregoing discussion suggests looking more closely at the space of all projections of a continuous lattice since they are so easily characterized.

proposition_3_10_converse.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.converseProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- **Scott 1972, Proposition 3.10 (converse).** If `i : D → D'` (between continuous
    lattices)
```

```
satisfies (i) preservation of arbitrary suprema, (ii) injectivity, and (iii)
    preservation of `≪`,
then there is a continuous `j` making `D` a projection of `D'` via `i`, with `j` given
    by Scott's
formula (iv) `j(x') = ⊔ { x | i(x) ⊑ x' }`. -/
theorem proposition_3_10_converse
    (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S)) (hinj : Function.Injective i)
    (hwb : ∀ {x y : D}, x ≪ y → i x ≪ i y) (hD : ContinuousLattice D) :
    ∃ P : IsContinuousLatticeProjection D D',
      (∀ d, (P.incl : D → D') d = i d)
        ∧ (∀ x', (P.retr : D' → D) x' = sSup {x | i x ≤ x'}) :=
  ⟨converseProjection hi hinj hwb hD, fun _ => rfl, fun _ => rfl⟩

end Proposition310
end DomainTheory.ContinuousLattice
```

proposition_3_10_forward.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_sSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_injective
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

/-- **Scott 1972, Proposition 3.10(i)-(iii), bundled.** The inclusion `i` of a
    projection preserves
arbitrary suprema, is injective, and preserves the way-below relation. -/
theorem proposition_3_10_forward (P : IsContinuousLatticeProjection D D')
    (hD' : ContinuousLattice D') :
    (∀ S : Set D, (P.incl : D → D') (sSup S) = sSup (Set.image (P.incl : D → D') S))
      ∧ Function.Injective (P.incl : D → D')
```

```
      ∧ (∀ x y : D, x ≪ y → (P.incl : D → D') x ≪ (P.incl : D → D') y) :=
  ⟨fun S => P.incl_sSup S, fun _ _ h => P.incl_injective h,
    fun _ _ h => P.incl_wayBelow hD' h⟩

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

1.4.12 Definition 3.11: Projection space

3.11 Definition. Given a continuous lattice D, we let the space of projections be denoted by:

$$J_D = j \in [D \to \{D] : j = j \circ j \sqsubseteq \mathrm{id}\}.$$

1.4.13 Proposition 3.12

3.12 Proposition. For a continuous lattice D the space J_D of projections forms a complete lattice as a $\bigsqcup$-closed subspace of $[D \to D]$.

Proof: The constant function $\bot \in J_D$ obviously, so J_D contains $\bigsqcup \emptyset$. Suppose $j, k \in J_D$. We wish to show that $j \sqcup k \in J_D$. Compute:

$$(j \sqcup k)((j \sqcup k)(x)) = j(j(x) \sqcup k(x)) \sqcup k(j(x) \sqcup k(x)).$$

But note:

$$j(x) \sqsubseteq j(j(x)) \sqsubseteq j(j(x) \sqcup k(x)) \sqsubseteq j(x),$$

because $j(x) \sqcup k(x) \sqsubseteq x$. Similarly for $k(x)$. Therefore, we find that $(j \sqcup k) \circ (j \sqcup k) = j \sqcup k \sqsubseteq \mathrm{id}$.

Suppose finally that $S \subseteq J_D$ is directed. We wish to show that $\bigsqcup S \in J_D$. Clearly $\bigsqcup S \sqsubseteq \mathrm{id}$, so compute by continuity of $\circ$:

$$\bigsqcup S \circ \bigsqcup S = \bigsqcup \{j \circ j : j \in S\} = \bigsqcup \{j : j \in S\} = \bigsqcup S.$$

It follows that J_D is $\bigsqcup$-closed and hence is a complete lattice. $\square$

The significance of the above result becomes clearer if we consider the connection between projections and subspaces. Let us write:

$$D(j) = \{x \in D : j(x) = x\}.$$

For $j \in J_D$, each $D(j)$ is a projection of D onto a subspace. We show first that

$$j \sqsubseteq k \quad \text{iff} \quad D(j) \subseteq D(k).$$

Because if $j \sqsubseteq k$, then $j \sqsubseteq j \circ j \sqsubseteq k \circ j \sqsubseteq \mathrm{id} \circ j = j$. Therefore if $j(x) = x$, then $k(x) = k(j(x)) = j(x) = x$, which means that $D(j) \subseteq D(k)$. On the other hand, if $D(j) \subseteq D(k)$, then since $j(D) \subseteq D(j)$, we see that $k \circ j = j$, and so $j \sqsubseteq k \circ \mathrm{id} = k$. Hence J_D is isomorphic to the partially ordered set of subspaces of D that are projections. We thus conclude that these subspaces form a lattice. In fact, it is easy to show that

$$D(j \sqcup k) = x \sqcup \{y : x \in D(j)\}, y \in D(k).$$

Similarly, if S is a directed set of J_D, then $D(\bigsqcup S)$ is the $\bigsqcup$-closure in D of the subset:

$$\{\bigsqcup D(j) : j \in S.\}$$

These are not very deep facts, but their proofs were very much facilitated by the introduction of J_D and the utilization of the lattice structure of $[D \to D]$. Along the same line we can define for $j, k \in J_D$ a function $(j \to k) \in [D \to D] \to [D \to D]$ by the equation

$$(j \to k)(f) = k \circ f \circ j.$$

It is very easy to show that $(j \to k) \in J_{[D \to D]}$, that $(j \to k)$ is continuous in j and k, and that $[D \to D](j \to k)$ is isomorphic to $[D(j) \to D(k)]$. There are many other interesting operations on projections corresponding to other constructs besides these. And, just as with $(j \to k)$, the operations are continuous. This makes it possible to prove existence theorems about subspaces by using results like the fixed-point theorem for continuous functions. It would be even nicer if J_D turns out to be a continuous lattice itself, but as far as I can tell this is not likely to be the case.

Before we turn to the iterated function-space construction by inverse limits, there are a couple of other connections between spaces and function spaces that are useful to know.

proposition_3_12.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.instCompleteLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.Projections
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_sSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst30
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

/-- **Scott 1972, Proposition 3.12.** For a (complete, in particular continuous)
    lattice `D`, the
projections `J_D` form a complete lattice as a `⊔`-closed subspace of `[D → D]`: the `
    ⊔`-closure is
`isProjection_sSup`, and the complete-lattice structure is `Projections.
    instCompleteLattice`. -/
theorem proposition_3_12 :
    (∀ T : Set (ScottMap D D), (∀ j ∈ T, IsProjection j) → IsProjection (sSup T))
```

```
    ∧ Nonempty (CompleteLattice (Projections D)) :=
  ⟨fun _ h => isProjection_sSup h, ⟨Projections.instCompleteLattice⟩⟩

end Projections
end DomainTheory.ContinuousLattice
```

1.4.14 Proposition 3.13

3.13 Proposition. Every continuous lattice D is a projection of its function space $[D \to D]$.

Proof: Consider the following pair of mappings $\mathrm{con} : D \to [D \to D]$ and $\mathrm{min} : [D \to D] \to D$ where

$$\mathrm{con}(x)(y) = x$$

and

$$\mathrm{min}(f) = f(\bot)$$

for all $x, y \in D$ and $f \in [D \to D]$. They are obviously continuous. The map con matches every element of D with the corresponding *constant* function in $[D \to D]$. The map min associates to every function in $[D \to D]$ its *minimum* value in the partial ordering. The proof that this pair forms a projection is trivial. □

The pair $\mathrm{con}, \mathrm{min}$ are not the only pair for making D a projection of $[D \to D]$. The following pair of maps were suggested by David Park:

$$\lambda x.\vec{e}[t, x] \quad \text{and} \quad \lambda f.f(t),$$

where x ranges over D, and f over $[D \to D]$, and where t is a fixed *isolated* element of D (that is, $t \ll t$ holds). The pair con and min will result if we set $t = \bot$. (Note that the expression $\vec{e}[t, x]$, though continuous in x, is not continuous—or even monotonic—in the variable t.) A lattice may very well possess a large number of isolated

elements, whence a large number of projections. And furthermore this is the only way the function $j = \lambda f.f(t)$ can be a projection. For assume the existence of an inverse $i : D \to [D \to D]$ satisfying the proper conditions. Then it would be the case that

$$i(x)(t) = x$$

and

$$i(f(t))(y) \sqsubseteq f(y)$$

for all $x, y \in D$ and all $f \in [D \to D]$. We can prove for all $u \in D$, if $t \not\sqsubseteq u$, then

$$i(x)(u) = \bot$$

by substituting $\vec{e}[v, x]$ for f in the second equation above, where v is chosen so that $v \ll t$ but not $v \ll u$. But then note that

$$i(x)(t) = \bigsqcup\{i(x)(u) : u \ll t\}.$$

If not $t \ll t$, then $u \ll t$ implies $t \not\sqsubseteq u$, which leads to absurdity. Hence t must be isolated, and, as we noted earlier, the function i is uniquely determined as being the one we already knew. Aside from these pairs of projections one could obtain others by combinations with automorphisms. I was unable to determine whether there are further pairs of an essentially different nature.

The topic of projections in these spaces is rather interesting since one has in a way more freedom in T_0-spaces (particularly in injective spaces) than in ordinary spaces for defining functions. As another example, consider the Cartesian square $D \times D$. Aside from the two obvious projections onto D, there is also the "diagonal" system given by the pair:

$$\lambda x.(x, x) \qquad \text{and} \qquad \lambda(x, y).x \sqcap y$$

We shall note in the next section how the choice of an initial projection effects the construction of an inverse limit.

The projections are not the only useful functions in $[D \to D] \to D$. As a final example of what can be done in function spaces we mention the fixed-point operator.

con_.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.const
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.const_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition313

/-- **Scott 1972, Proposition 3.13.** `con : D → [D → D]` sends `x` to the constant
    function `x`. -/
noncomputable def con : ScottMap D (ScottMap D D) :=
  ⟨fun x => ScottMap.const x, continuous_of_preservesDirectedSup (by
    intro S _ _
    apply ScottMap.ext
    intro y
```

```
    rw [ScottMap.sSup_apply, Set.image_image]
    simp only [ScottMap.const_apply, Set.image_id'])⟩

end Proposition313

end DomainTheory.ContinuousLattice
```

min.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition313

/-- **Scott 1972, Proposition 3.13.** `min : [D → D] → D` sends `f` to its least value
     `f(⊥)`. -/
noncomputable def min : ScottMap (ScottMap D D) D :=
  ⟨fun f => (f : D → D) ⊥, continuous_of_preservesDirectedSup (by
    intro F _ _
    show ((sSup F : ScottMap D D) : D → D) ⊥
      = sSup (Set.image (fun f : ScottMap D D => (f : D → D) ⊥) F)
    rw [ScottMap.sSup_apply])⟩

end Proposition313

end DomainTheory.ContinuousLattice
```

proposition_3_13.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.projection
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Proposition 3.13.** Every continuous lattice `D` is a projection of
    its function
space `[D → D]`, via `con`/`min` (`Proposition313.projection`). -/
theorem proposition_3_13 (_hD : ContinuousLattice D) :
    Nonempty (IsContinuousLatticeProjection D (ScottMap D D)) :=
  ⟨Proposition313.projection⟩

end DomainTheory.ContinuousLattice
```

1.4.15 Proposition 3.14

3.14 Proposition. *For a continuous lattice D there is a uniquely determined continuous mapping*

$$\mathrm{fix} : [D \to D] \to D$$

where for all $f \in [D \to D]$ and $x \in D$

$$f(\mathrm{fix}(f)) = \mathrm{fix}(f)$$

and whenever $f(x) \sqsubseteq x$, then $\mathrm{fix}(f) \sqsubseteq x$.
and whenever $f(x) \sqsubseteq x$, then

$$\mathrm{fix}(f) \sqsubseteq x.$$

Proof: The proof of the existence of minimal fixed points in complete lattices is well known, as was mentioned in the proof of 2.10.

To establish the continuity, it is sufficient to remark that since all functions $f \in [D \to D]$ are continuous, we have

$$\mathrm{fix}(f) = \bigsqcup_{n=0}^{\infty} f^n(\bot),$$

where $f^n(x) = f(f(\ldots f(x)\ldots))$ (n times). Thus fix is the point-wise lub of continuous functions on $[D \to D]$ and is thus itself continuous. □

fix.lean

```
module

public import Mathlib.Order.FixedPoints
public import DomainTheory.ContinuousLattice.FunctionSpaces.toOrderHom
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- **Scott 1972, Proposition 3.14.** `fix f` is the least (pre-)fixed point of `f`,
    supplied by
mathlib's `OrderHom.lfp`. -/
noncomputable def fix (f : ScottMap D D) : D := (toOrderHom f).lfp

end Proposition314

end DomainTheory.ContinuousLattice
```

fixMap.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_preservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]
```

```
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- **Scott 1972, Proposition 3.14.** The fixed-point operator as a Scott-continuous
    map. -/
noncomputable def fixMap : ScottMap (ScottMap D D) D :=
  ⟨fix, continuous_of_preservesDirectedSup fix_preservesDirectedSup⟩

end Proposition314

end DomainTheory.ContinuousLattice
```

fix_eq.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.fix
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- `fix f` is a fixed point of `f`. -/
theorem fix_eq (f : ScottMap D D) : (f : D → D) (fix f) = fix f :=
  (toOrderHom f).map_lfp

end Proposition314

end DomainTheory.ContinuousLattice
```

fix_le.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.fix
```

```
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- `fix f` is below every pre-fixed point: `f x ⊑ x ⟹ fix f ⊑ x`. -/
theorem fix_le {f : ScottMap D D} {x : D} (h : (f : D → D) x ≤ x) : fix f ≤ x :=
  (toOrderHom f).lfp_le h

end Proposition314

end DomainTheory.ContinuousLattice
```

fix_mono.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst28
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_le
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_eq
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']
```

```
namespace Proposition314

/-- `fix` is monotone in `f`: if `f ⊑ f'` then `fix f ⊑ fix f'`, since `f (fix f') ⊑ f'
     (fix f') =
fix f'` makes `fix f'` a pre-fixed point of `f`. -/
theorem fix_mono {f f' : ScottMap D D} (hff' : f ≤ f') : fix f ≤ fix f' :=
  fix_le (le_trans (ScottMap.le_def.mp hff' (fix f')) (le_of_eq (fix_eq f')))

end Proposition314

end DomainTheory.ContinuousLattice
```

fix_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_mono
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_le
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_eq
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- **Scott 1972, Proposition 3.14 (continuity).** `fix` preserves directed suprema.
    Direct
lattice argument (no Kleene iteration): write `g = ⊔S` and `a = ⊔{fix f : f ∈ S}`. The
    reverse
bound `a ⊑ fix g` is `fix`-monotonicity. For `fix g ⊑ a` it suffices (by `fix_le`) that
     `a` is a
pre-fixed point of `g`. Now `g a = ⊔_{f∈S} f a` (pointwise sup), and each `f a = ⊔_{f'∈
    S} f (fix f')`
(continuity of `f` on the directed family `{fix f'}`); for any `f, f' ∈ S` pick `h ∈ S`
     above both,
```

```
so `f (fix f') ⊑ h (fix f') ⊑ h (fix h) = fix h ⊑ a`. Hence `g a ⊑ a`. -/
theorem fix_preservesDirectedSup : PreservesDirectedSup (fix : ScottMap D D → D) := by
  intro S hSne hSdir
  set T : Set D := Set.image fix S with hTdef
  have hTne : T.Nonempty := hSne.image fix
  have hTdir : DirectedOn (· ≤ ·) T := by
    rintro _ ⟨f, hf, rfl⟩ _ ⟨f', hf', rfl⟩
    obtain ⟨h, hh, hfh, hf'h⟩ := hSdir f hf f' hf'
    exact ⟨fix h, Set.mem_image_of_mem fix hh, fix_mono hfh, fix_mono hf'h⟩
  show fix (sSup S) = sSup T
  apply le_antisymm
  · apply fix_le
    show ((sSup S : ScottMap D D) : D → D) (sSup T) ≤ sSup T
    rw [ScottMap.sSup_apply]
    apply sSup_le
    rintro _ ⟨f, hf, rfl⟩
    show (f : D → D) (sSup T) ≤ sSup T
    rw [f.preservesDirectedSup_coe T hTne hTdir]
    apply sSup_le
    rintro _ ⟨_, ⟨f', hf', rfl⟩, rfl⟩
    obtain ⟨h, hh, hfh, hf'h⟩ := hSdir f hf f' hf'
    calc (f : D → D) (fix f')
        ≤ (h : D → D) (fix f') := ScottMap.le_def.mp hfh (fix f')
      _ ≤ (h : D → D) (fix h) := h.monotone (fix_mono hf'h)
      _ = fix h := fix_eq h
      _ ≤ sSup T := le_sSup (Set.mem_image_of_mem fix hh)
  · apply sSup_le
    rintro _ ⟨f, hf, rfl⟩
    exact fix_mono (le_sSup hf)

end Proposition314

end DomainTheory.ContinuousLattice
```

fix_unique.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_le
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_eq
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']
```

```
namespace Proposition314

/-- Uniqueness: any value that is a fixed point of `f` and below every pre-fixed point
    equals
`fix f` (the least fixed point is unique). -/
theorem fix_unique {f : ScottMap D D} {a : D} (hfix : (f : D → D) a = a)
    (hleast : ∀ x, (f : D → D) x ≤ x → a ≤ x) : a = fix f :=
  le_antisymm (hleast (fix f) (le_of_eq (fix_eq f))) (fix_le hfix.le)

end Proposition314
end DomainTheory.ContinuousLattice
```

proposition_3_14.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_unique
public import DomainTheory.ContinuousLattice.FunctionSpaces.fixMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_le
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_eq
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Proposition 3.14.** For a continuous lattice `D` there is a uniquely
    determined
continuous mapping `fix : [D → D] → D` such that `f (fix f) = fix f` for all `f`, and
    `fix f ⊑ x`
whenever `f x ⊑ x`. Existence and continuity are `Proposition314.fixMap`; the defining
    equations are
`fix_eq`/`fix_le`; uniqueness (any operator with these two properties agrees with `fix
    `) is
`fix_unique`. -/
theorem proposition_3_14 (_hD : ContinuousLattice D) :
    ∃ Fix : ScottMap (ScottMap D D) D,
      (∀ f : ScottMap D D, (f : D → D) (Fix f) = Fix f)
        ∧ (∀ (f : ScottMap D D) (x : D), (f : D → D) x ≤ x → (Fix f : D) ≤ x)
        ∧ (∀ g : ScottMap D D → D,
            (∀ f : ScottMap D D, (f : D → D) (g f) = g f) →
            (∀ (f : ScottMap D D) (x : D), (f : D → D) x ≤ x → g f ≤ x) →
            ∀ f : ScottMap D D, g f = Fix f) :=
  ⟨Proposition314.fixMap, fun f => Proposition314.fix_eq f,
```

```
    fun _ _ h => Proposition314.fix_le h,
    fun _ hfix hleast f => Proposition314.fix_unique (hfix f) (fun x hx => hleast f x
    hx)⟩

end DomainTheory.ContinuousLattice
```

1.5 Inverse limits

By an *inverse system of spaces* we understand as usual a sequence

$$\langle X_n, j_n \rangle_{n=0}^{\infty}$$

of T_0-spaces and continuous maps $j_n : X_{n+1} \to X_n$. The space X_∞, called the *inverse limit* of the sequence, is constructed in the familiar way as that subspace of the product space consisting of exactly those infinite sequences

$$x = \langle x_n \rangle_{n=0}^{\infty},$$

where for each n we have $x_n \in X_n$, and

$$j_n(x_{n+1}) = x_n.$$

The space X_∞ is given the product topology, and the maps $j_{\infty\{n} : X_\infty \to X_n\}$ such that

$$j_{\infty n}(x) = x_n$$

are of course continous and satisfy the recursion equation:

$$j_{\infty n} = j_n \circ j_{\infty(n+1)}.$$

Besides this we have the expected extension property for any system of continuous maps

$$f_n : Y \to X_n$$

where for each n

$$f_n = j_n \circ f_{n+1}.$$

Because, we can define

$$\{f_\infty : Y \to X_\infty\}$$

by the equation

$$f_\infty(y) = \langle f_n(y) \rangle_{n=0}^{\infty}$$

for all $y \in Y$; whence

$$f_n = j_{\infty n} \circ f_\infty$$

holds. So much for a review of inverse limits. In this paper our interest will center on rather special inverse systems and their limits.

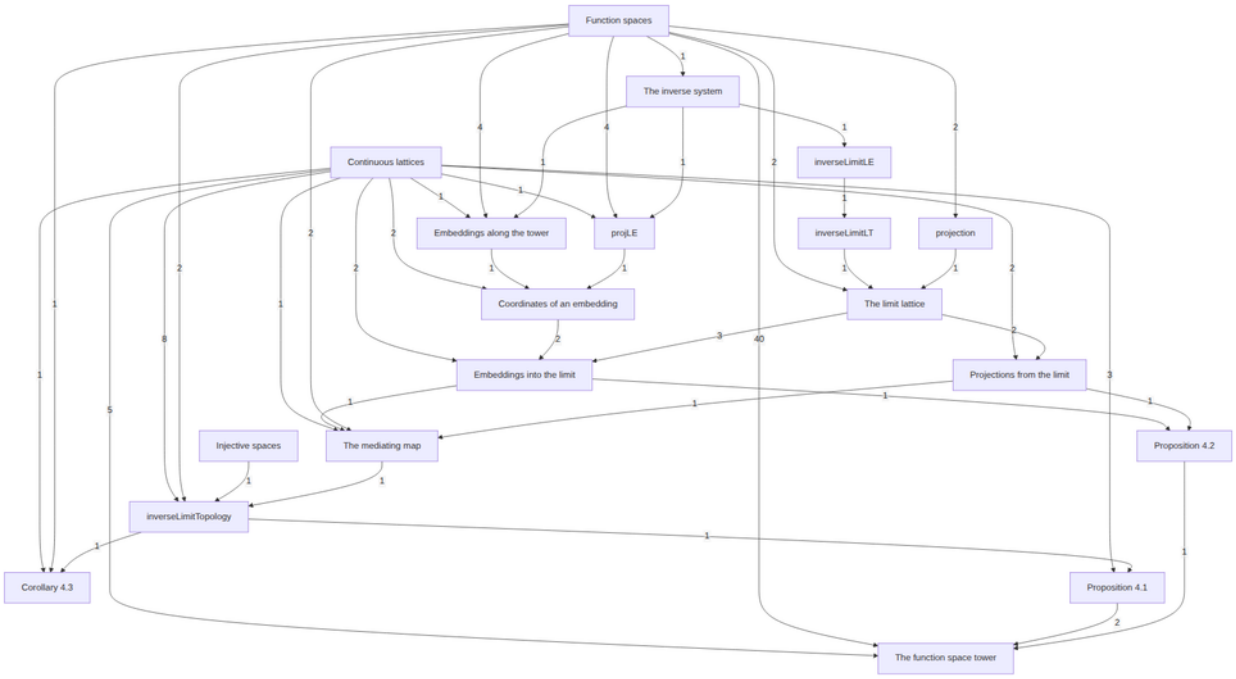


Figure 8: Inverse limits.

1.5.1 The inverse system

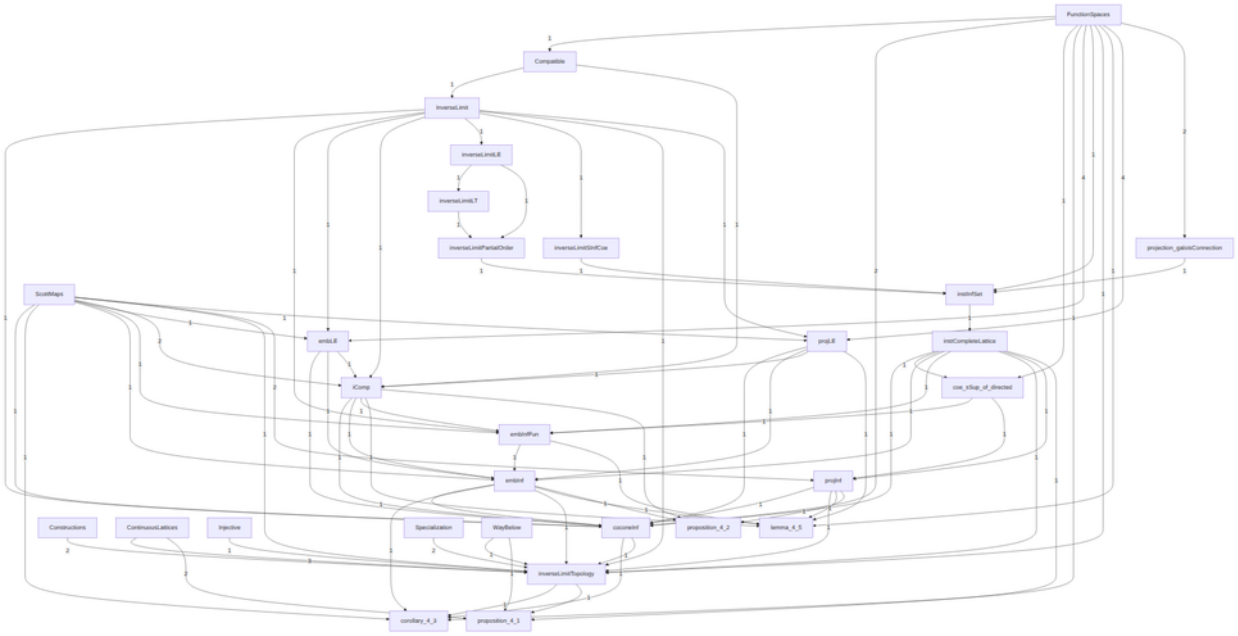


Figure 9: The inverse system.

Compatible `Compatible.lean`

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- Compatibility of a sequence: `j_n(x_{n+1}) = x_n` for all `n`. -/
def Compatible (x : ∀ n, D n) : Prop := ∀ n, (P n).retr (x (n + 1)) = x n

end InverseLimit

end DomainTheory.ContinuousLattice
```

InverseLimit InverseLimit.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.Compatible
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- **Scott 1972, §4.** The inverse limit `D_∞` as the subspace of compatible
    sequences. -/
abbrev InverseLimit : Type u := {x : ∀ n, D n // Compatible D P x}

end InverseLimit

end DomainTheory.ContinuousLattice
```

1.5.2 projLE

projLE.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.comp
public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]
```

```
variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- Descend the tower of projections: for `m ≤ n`, `projLE h = j_m ∘ ... ∘ j_{n-1} :
    D_n → D_m`. -/
def projLE {m n : ℕ} (h : m ≤ n) (x : D n) : D m :=
  Nat.leRecOn (C := fun k => D k → D m) h
    (fun {k} (g : D k → D m) (w : D (k + 1)) => g ((P k).retr w)) id x

namespace projLE

theorem self {m : ℕ} (h : m ≤ m) (x : D m) : projLE D P h x = x := by
  simp only [projLE, Nat.leRecOn_self, id_eq]

theorem succ {m n : ℕ} (h1 : m ≤ n) (h2 : m ≤ n + 1) (z : D (n + 1)) :
    projLE D P h2 z = projLE D P h1 ((P n).retr z) := by
  simp only [projLE]
  rw [Nat.leRecOn_succ (C := fun k => D k → D m) h1]

theorem mono {m n : ℕ} (h : m ≤ n) {x y : D n} (hxy : x ≤ y) :
    projLE D P h x ≤ projLE D P h y := by
  induction n, h using Nat.le_induction with
  | base =>
      rw [self, self]
      exact hxy
  | succ k hk ih =>
      rw [succ D P hk (Nat.le_succ_of_le hk), succ D P hk (Nat.le_succ_of_le hk)]
      exact ih ((P k).retr.monotone hxy)

/-- For a compatible sequence `y`, descending from level `n` to `m ≤ n` recovers `y_m`.
    -/
theorem compatible {m : ℕ} (y : InverseLimit D P) :
    ∀ {n : ℕ} (h : m ≤ n), projLE D P h (y.1 n) = y.1 m := by
  intro n h
  induction n, h using Nat.le_induction with
  | base => rw [self]
  | succ k hk ih => rw [succ D P hk (Nat.le_succ_of_le hk), y.2 k, ih]

/-- Peeling the last projection: `(P m).retr ∘ projLE (m+1 ≤ n) = projLE (m ≤ n)`. -/
theorem retr {m : ℕ} : ∀ {n : ℕ} (h : m + 1 ≤ n) (x : D n),
    (P m).retr (projLE D P h x) = projLE D P (Nat.le_of_succ_le h) x := by
  intro n h
  induction n, h using Nat.le_induction with
  | base =>
      intro x
      rw [succ D P (le_refl m) (Nat.le_of_succ_le (le_refl (m + 1))) x]
      simp only [self]
  | succ k hk ih =>
      intro x
      rw [succ D P hk (Nat.le_succ_of_le hk) x, ih ((P k).retr x),
        succ D P (Nat.le_of_succ_le hk) (Nat.le_of_succ_le (Nat.le_succ_of_le hk)) x]

/-- The tower of projections is Scott-continuous (a composite of the `j_k`). -/
theorem preservesDirectedSup {m n : ℕ} (h : m ≤ n) :
    PreservesDirectedSup (projLE D P h) := by
  induction n, h using Nat.le_induction with
```

```
  | base =>
      have hid : projLE D P (le_refl m) = id := funext (fun x => self D P (le_refl m)
    x)
      rw [hid]; intro S _ _; simp
  | succ k hk ih =>
      have hfun : projLE D P (Nat.le_succ_of_le hk)
          = projLE D P hk ∘ (fun z : D (k + 1) => (P k).retr z) :=
        funext (fun x => succ D P hk (Nat.le_succ_of_le hk) x)
      rw [hfun]
      exact ScottMap.preservesDirectedSup_comp ih
        (fun S hS hd => (P k).retr.preservesDirectedSup_coe S hS hd)

end projLE

end InverseLimit

end DomainTheory.ContinuousLattice
```

1.5.3 projection

projection_galoisConnection.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- The embedding-projection pair of a projection is a Galois connection `iₙ ⊣ jₙ`:
    from
`jₙ ∘ iₙ = id` and `iₙ ∘ jₙ ⊑ id` we get `iₙ x ⊑ y ↔ x ⊑ jₙ y`. In particular `jₙ` (the
    upper
adjoint) preserves arbitrary infima. -/
theorem projection_galoisConnection (n : ℕ) :
    GaloisConnection ((P n).incl : D n → D (n + 1)) ((P n).retr : D (n + 1) → D n) :=
     by
  intro x y
  constructor
  · intro h
    have h' := (P n).retr.monotone h
```

```
    rwa [(P n).retr_incl] at h'
  · intro h
    exact le_trans ((P n).incl.monotone h) ((P n).incl_retr_le y)

end InverseLimit

end DomainTheory.ContinuousLattice
```

1.5.4 Projections from the limit

projInf.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.instCompleteLattice
public import DomainTheory.ContinuousLattice.InverseLimits.coe_sSup_of_directed
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- The projection `j_{∞n} : D_∞ → Dₙ` as a bare function. -/
def projInfFun (n : ℕ) (y : InverseLimit D P) : D n := y.1 n

namespace projInfFun

theorem preservesDirectedSup (n : ℕ) : PreservesDirectedSup (projInfFun D P n) := by
  intro S hS hSdir
  have hL : (sSup S : InverseLimit D P).1 n = sSup ((fun y : InverseLimit D P => y.1 n
    ) '' S) := by
    rw [coe_sSup_of_directed D P S hS hSdir, sSup_apply_eq_sSup_image]
    congr 1
    rw [Set.image_image]
  exact hL

end projInfFun

/-- The projection `j_{∞n} : D_∞ → Dₙ`, Scott-continuous. -/
noncomputable def projInf (n : ℕ) : ScottMap (InverseLimit D P) (D n) :=
```

```
  ⟨projInfFun D P n, continuous_of_preservesDirectedSup (projInfFun.
    preservesDirectedSup D P n)⟩

end InverseLimit

end DomainTheory.ContinuousLattice
```

1.5.5 Embeddings along the tower

embLE.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.comp
public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- Climb the tower of embeddings: for `n ≤ m`, `embLE h = i_{m-1} ∘ ... ∘ iₙ : Dₙ →
    D_m`. -/
def embLE {n m : ℕ} (h : n ≤ m) (x : D n) : D m :=
  Nat.leRecOn h (fun {k} (y : D k) => (P k).incl y) x

theorem embLE_self {n : ℕ} (h : n ≤ n) (x : D n) : embLE D P h x = x := by
  simp only [embLE, Nat.leRecOn_self]

theorem embLE_succ {n m : ℕ} (h1 : n ≤ m) (h2 : n ≤ m + 1) (x : D n) :
    embLE D P h2 x = (P m).incl (embLE D P h1 x) := by
  simp only [embLE]
  rw [Nat.leRecOn_succ h1]

theorem embLE_succ_left {n m : ℕ} (h1 : n ≤ m) (h2 : n + 1 ≤ m) (x : D n) :
    embLE D P h2 ((P n).incl x) = embLE D P h1 x := by
  simp only [embLE]
  exact Nat.leRecOn_succ_left x h1 h2
```

```
/-- For a compatible sequence `y`, climbing `y_n` up to level `m ≥ n` stays below `y_m`.
    -/
theorem embLE_le {n : ℕ} (y : InverseLimit D P) {m : ℕ} (h : n ≤ m) :
    embLE D P h (y.1 n) ≤ y.1 m := by
  induction m, h using Nat.le_induction with
  | base => exact le_of_eq (embLE_self D P _ _)
  | succ k hk ih =>
      rw [embLE_succ D P hk (Nat.le_succ_of_le hk)]
      calc (P k).incl (embLE D P hk (y.1 n))
          ≤ (P k).incl (y.1 k) := (P k).incl.monotone ih
        _ = (P k).incl ((P k).retr (y.1 (k + 1))) := by rw [y.2 k]
        _ ≤ y.1 (k + 1) := (P k).incl_retr_le _

theorem embLE_mono {n m : ℕ} (h : n ≤ m) {x y : D n} (hxy : x ≤ y) :
    embLE D P h x ≤ embLE D P h y := by
  induction m, h using Nat.le_induction with
  | base =>
      rw [embLE_self, embLE_self]
      exact hxy
  | succ k hk ih =>
      rw [embLE_succ D P hk (Nat.le_succ_of_le hk), embLE_succ D P hk (Nat.
    le_succ_of_le hk)]
      exact (P k).incl.monotone ih

/-- The tower of embeddings is Scott-continuous (a composite of the `i_k`). -/
theorem embLE_preservesDirectedSup {n m : ℕ} (h : n ≤ m) :
    PreservesDirectedSup (embLE D P h) := by
  induction m, h using Nat.le_induction with
  | base =>
      have hid : embLE D P (le_refl n) = id := funext (fun x => embLE_self D P (
    le_refl n) x)
      rw [hid]; intro S _ _; simp
  | succ k hk ih =>
      have hfun : embLE D P (Nat.le_succ_of_le hk)
          = (fun y : D k => (P k).incl y) ∘ embLE D P hk :=
        funext (fun x => embLE_succ D P hk (Nat.le_succ_of_le hk) x)
      rw [hfun]
      exact ScottMap.preservesDirectedSup_comp
        (fun S hS hd => (P k).incl.preservesDirectedSup_coe S hS hd) ih

end InverseLimit

end DomainTheory.ContinuousLattice
```

1.5.6 Coordinates of an embedding

iComp.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.embLE
public import DomainTheory.ContinuousLattice.InverseLimits.projLE
public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import DomainTheory.ContinuousLattice.InverseLimits.Compatible
```

```
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.preservesDirectedSup_monotone
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- Scott's embedding component `i_{n∞}(x)_m`: climb for `m ≥ n`, descend for `m < n`.
    -/
def iComp (n : ℕ) (x : D n) (m : ℕ) : D m :=
  if h : n ≤ m then embLE D P h x else projLE D P (le_of_lt (not_le.mp h)) x

theorem iComp_of_le {n m : ℕ} (h : n ≤ m) (x : D n) : iComp D P n x m = embLE D P h x
    :=
  dite_eq_left h

theorem iComp_of_ge {n m : ℕ} (h : ¬ n ≤ m) (x : D n) :
    iComp D P n x m = projLE D P (le_of_lt (not_le.mp h)) x :=
  dite_eq_right h

theorem iComp_self (n : ℕ) (x : D n) : iComp D P n x n = x := by
  rw [iComp_of_le D P (le_refl n), embLE_self]

/-- `i_{n∞}(y_n) ⊑ y` coordinatewise (the heart of `incl_retr_le` for `j_{∞n}`). -/
theorem iComp_incl_le (n : ℕ) (y : InverseLimit D P) (m : ℕ) :
    iComp D P n (y.1 n) m ≤ y.1 m := by
  by_cases h : n ≤ m
  · rw [iComp_of_le D P h]; exact embLE_le D P y h
  · rw [iComp_of_ge D P h]
    exact le_of_eq (projLE.compatible D P y (le_of_lt (not_le.mp h)))

theorem iComp_preservesDirectedSup (n m : ℕ) :
    PreservesDirectedSup (fun x : D n => iComp D P n x m) := by
  by_cases h : n ≤ m
  · have hfun : (fun x : D n => iComp D P n x m) = embLE D P h :=
      funext (fun x => iComp_of_le D P h x)
    rw [hfun]; exact embLE_preservesDirectedSup D P h
  · have hfun : (fun x : D n => iComp D P n x m) = projLE D P (le_of_lt (not_le.mp h))
    :=
      funext (fun x => iComp_of_ge D P h x)
    rw [hfun]; exact projLE.preservesDirectedSup D P _

theorem iComp_monotone (n m : ℕ) : Monotone (fun x : D n => iComp D P n x m) :=
  preservesDirectedSup_monotone (iComp_preservesDirectedSup D P n m)
```

```
/-- The sequence `i_{n∞}(x)` is compatible, hence a genuine point of `D_∞`. -/
theorem iComp_compatible (n : ℕ) (x : D n) : Compatible D P (iComp D P n x) := by
  intro m
  by_cases h1 : n ≤ m
  · rw [iComp_of_le D P h1, iComp_of_le D P (Nat.le_succ_of_le h1),
      embLE_succ D P h1 (Nat.le_succ_of_le h1), (P m).retr_incl]
  · by_cases h2 : n ≤ m + 1
    · have hn : n = m + 1 := le_antisymm h2 (not_le.mp h1)
      subst hn
      rw [iComp_of_le D P (le_refl (m + 1)), embLE_self, iComp_of_ge D P h1,
        projLE.succ D P (le_refl m) (le_of_lt (not_le.mp h1)) x, projLE.self]
    · rw [iComp_of_ge D P h2, iComp_of_ge D P h1,
        projLE.retr D P (le_of_lt (not_le.mp h2)) x]

end InverseLimit

end DomainTheory.ContinuousLattice
```

1.5.7 Embeddings into the limit

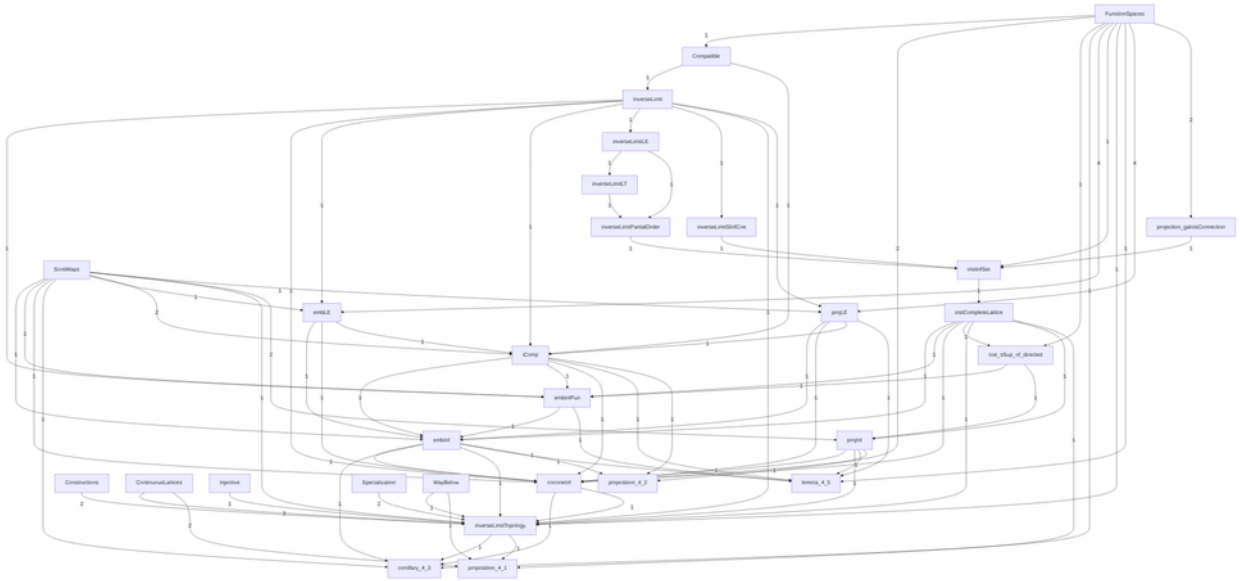


Figure 10: Embeddings into the limit.

embInfFun `embInfFun.lean`

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import DomainTheory.ContinuousLattice.InverseLimits.iComp
public import DomainTheory.ContinuousLattice.InverseLimits.instCompleteLattice
public import DomainTheory.ContinuousLattice.InverseLimits.coe_sSup_of_directed
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
```

```
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- The embedding `i_{n∞} : Dₙ → D_∞` as a bare function into the inverse limit. -/
def embInfFun (n : ℕ) (x : D n) : InverseLimit D P := ⟨iComp D P n x, iComp_compatible
    D P n x⟩

@[simp] theorem embInfFun_coe (n : ℕ) (x : D n) : (embInfFun D P n x).1 = iComp D P n
    x := rfl

theorem embInf_monotone (n : ℕ) : Monotone (embInfFun D P n) := by
  intro x x' hxx
  rw [← Subtype.coe_le_coe]
  intro m
  exact iComp_monotone D P n m hxx

theorem embInf_preservesDirectedSup (n : ℕ) : PreservesDirectedSup (embInfFun D P n)
    := by
  intro S hS hSdir
  have hTne : (embInfFun D P n '' S).Nonempty := hS.image _
  have hTdir : DirectedOn (· ≤ ·) (embInfFun D P n '' S) := by
    rintro _ ⟨a, ha, rfl⟩ _ ⟨b, hb, rfl⟩
    obtain ⟨c, hc, hac, hbc⟩ := hSdir a ha b hb
    exact ⟨embInfFun D P n c, Set.mem_image_of_mem _ hc,
      embInf_monotone D P n hac, embInf_monotone D P n hbc⟩
  apply Subtype.ext
  rw [coe_sSup_of_directed D P (embInfFun D P n '' S) hTne hTdir]
  funext m
  have heq : Function.eval m '' (Subtype.val '' (embInfFun D P n '' S))
      = (fun x => iComp D P n x m) '' S := by
    simp only [Set.image_image, embInfFun_coe, Function.eval]
  rw [sSup_apply_eq_sSup_image, heq]
  exact iComp_preservesDirectedSup D P n m hS hSdir

end InverseLimit

end DomainTheory.ContinuousLattice
```

embInf embInf.lean

```
module
```

```
public import DomainTheory.ContinuousLattice.InverseLimits.instCompleteLattice
public import DomainTheory.ContinuousLattice.InverseLimits.embInfFun
public import DomainTheory.ContinuousLattice.InverseLimits.embLE
public import DomainTheory.ContinuousLattice.InverseLimits.iComp
public import DomainTheory.ContinuousLattice.InverseLimits.projLE
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- The embedding `i_{n∞} : D_n → D_∞`, Scott-continuous. -/
noncomputable def embInf (n : ℕ) : ScottMap (D n) (InverseLimit D P) :=
  ⟨embInfFun D P n, continuous_of_preservesDirectedSup (embInf_preservesDirectedSup D P
     n)⟩

/-- **Scott 1972, §4 (recursion equation).** `i_{n∞} = i_{(n+1)∞} ∘ i_n`. -/
theorem embInf_succ (n : ℕ) (x : D n) :
    embInf D P (n + 1) ((P n).incl x) = embInf D P n x := by
  apply Subtype.ext
  funext m
  show iComp D P (n + 1) ((P n).incl x) m = iComp D P n x m
  by_cases h1 : n ≤ m
  · by_cases h2 : n + 1 ≤ m
    · rw [iComp_of_le D P h2, iComp_of_le D P h1, embLE_succ_left D P h1 h2]
    · have hmn : m = n := le_antisymm (Nat.lt_succ_iff.mp (not_le.mp h2)) h1
      subst hmn
      rw [iComp_self, iComp_of_ge D P h2,
        projLE.succ D P (le_refl m) (le_of_lt (not_le.mp h2)) ((P m).incl x),
        projLE.self, (P m).retr_incl]
  · have h2 : ¬ n + 1 ≤ m := fun h => h1 (le_trans (Nat.le_succ n) h)
    rw [iComp_of_ge D P h2, iComp_of_ge D P h1,
      projLE.succ D P (le_of_lt (not_le.mp h1)) (le_of_lt (not_le.mp h2)) ((P n).incl
    x),
      (P n).retr_incl]

/-- The family `n ↦ i_{n∞}(x_n)` is monotone (the lub defining `x` is monotone). -/
theorem embInf_le_succ (x : InverseLimit D P) (n : ℕ) :
    embInf D P n (x.1 n) ≤ embInf D P (n + 1) (x.1 (n + 1)) := by
  rw [← embInf_succ D P n (x.1 n)]
  exact embInf_monotone D P (n + 1)
    (by calc (P n).incl (x.1 n) = (P n).incl ((P n).retr (x.1 (n + 1))) := by rw [x.2
    n]
```

```
          _ ≤ x.1 (n + 1) := (P n).incl_retr_le _)

theorem embInf_family_directed (x : InverseLimit D P) :
    DirectedOn (· ≤ ·) (Set.range (fun n => embInf D P n (x.1 n))) :=
  directedOn_range.2 (monotone_nat_of_le_succ (embInf_le_succ D P x)).directed_le

end InverseLimit

end DomainTheory.ContinuousLattice
```

1.5.8 The mediating map

coconeInf.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import DomainTheory.ContinuousLattice.InverseLimits.embLE
public import DomainTheory.ContinuousLattice.InverseLimits.projLE
public import DomainTheory.ContinuousLattice.InverseLimits.embInf
public import DomainTheory.ContinuousLattice.InverseLimits.iComp
public import DomainTheory.ContinuousLattice.InverseLimits.instCompleteLattice
public import DomainTheory.ContinuousLattice.InverseLimits.projInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

variable {D' : Type*} [CompleteLattice D']

/-- The mediating map of Corollary 4.3: `f∞(x) = ⋃ₙ fₙ(xₙ)`. -/
def coconeInf (f : ∀ n, ScottMap (D n) D') (x : InverseLimit D P) : D' :=
  ⋃ n, f n (x.1 n)

theorem coconeInf_apply (f : ∀ n, ScottMap (D n) D') (x : InverseLimit D P) :
    coconeInf D P f x = ⋃ n, f n (x.1 n) := rfl

/-- Climbing then applying `f` is constant: `f_m(i_{m-1}...iₙ x) = fₙ(x)`. -/
theorem coconeInf_climb (f : ∀ n, ScottMap (D n) D')
    (hf : ∀ n x, f n x = f (n + 1) ((P n).incl x)) {n : ℕ} (x : D n) :
```

```
    ∀ {m : ℕ} (h : n ≤ m), f m (embLE D P h x) = f n x := by
  intro m h
  induction m, h using Nat.le_induction with
  | base => rw [embLE_self]
  | succ k hk ih => rw [embLE_succ D P hk (Nat.le_succ_of_le hk), ← hf k, ih]

/-- Descending then applying `f` only decreases: `f_m(j_m...j_{n-1} x) ⊑ f_n(x)`. -/
theorem coconeInf_descend (f : ∀ n, ScottMap (D n) D')
    (hf : ∀ n x, f n x = f (n + 1) ((P n).incl x)) {m : ℕ} :
    ∀ {n : ℕ} (h : m ≤ n) (x : D n), f m (projLE D P h x) ≤ f n x := by
  intro n h
  induction n, h using Nat.le_induction with
  | base => intro x; exact le_of_eq (congrArg (f m) (projLE.self D P (le_refl m) x))
  | succ k hk ih =>
      intro x
      rw [projLE.succ D P hk (Nat.le_succ_of_le hk)]
      calc f m (projLE D P hk ((P k).retr x)) ≤ f k ((P k).retr x) := ih ((P k).retr x
    )
        _ = f (k + 1) ((P k).incl ((P k).retr x)) := hf k _
        _ ≤ f (k + 1) x := (f (k + 1)).monotone ((P k).incl_retr_le x)

/-- **Scott 1972, Corollary 4.3 (factorization).** `f_n = f∞ ∘ i_{n∞}`. -/
theorem coconeInf_comp_embInf (f : ∀ n, ScottMap (D n) D')
    (hf : ∀ n x, f n x = f (n + 1) ((P n).incl x)) (n : ℕ) (x : D n) :
    coconeInf D P f (embInf D P n x) = f n x := by
  apply le_antisymm
  · show (⋃ m, f m ((embInf D P n x).1 m)) ≤ f n x
    refine iSup_le fun m => ?_
    show f m (iComp D P n x m) ≤ f n x
    by_cases h : n ≤ m
    · rw [iComp_of_le D P h]; exact le_of_eq (coconeInf_climb D P f hf x h)
    · rw [iComp_of_ge D P h]; exact coconeInf_descend D P f hf (le_of_lt (not_le.mp h))
     x
  · refine le_trans (le_of_eq ?_) (le_iSup (fun m => f m ((embInf D P n x).1 m)) n)
    rw [show (embInf D P n x).1 n = x from iComp_self D P n x]

/-- `f∞` is Scott-continuous: `f∞(⊔S) = ⊔ f∞(S)` for directed `S` (each `f_n` is
    continuous and a
double `⋃` over `ℕ × S` commutes). -/
theorem coconeInf_preservesDirectedSup (f : ∀ n, ScottMap (D n) D') :
    PreservesDirectedSup (coconeInf D P f) := by
  intro S hS hSdir
  have hev : ∀ n, DirectedOn (· ≤ ·) ((fun s : InverseLimit D P => s.1 n) '' S) := by
    intro n
    rintro _ ⟨a, ha, rfl⟩ _ ⟨b, hb, rfl⟩
    obtain ⟨c, hc, hac, hbc⟩ := hSdir a ha b hb
    exact ⟨c.1 n, ⟨c, hc, rfl⟩, hac n, hbc n⟩
  have key : ∀ n, f n ((sSup S).1 n) = sSup ((fun s : InverseLimit D P => f n (s.1 n))
    '' S) := by
    intro n
    have h1 : (sSup S).1 n = sSup ((fun s : InverseLimit D P => s.1 n) '' S) :=
      projInfFun.preservesDirectedSup D P n hS hSdir
    rw [h1, (f n).preservesDirectedSup_coe _ (hS.image _) (hev n), Set.image_image]
  show (⋃ n, f n ((sSup S).1 n)) = sSup (coconeInf D P f '' S)
  simp_rw [key, sSup_image', coconeInf_apply]
  rw [iSup_comm]
```

```
end InverseLimit

end DomainTheory.ContinuousLattice
```

1.5.9 inverseLimitLE

inverseLimitLE.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- Coordinatewise order on compatible sequences. Kept reducible so existing
subtype-order lemmas see the underlying pointwise relation definitionally. -/
abbrev inverseLimitLE (x y : InverseLimit D P) : Prop :=
  ∀ n,
    @LE.le (D n)
      (ChainCompletePartialOrder.instOfCompleteLattice
        (α := D n)).toPartialOrder.toPreorder.toLE
      (x.1 n) (y.1 n)

namespace inverseLimitLE

/-- Reflexivity of the coordinatewise inverse-limit order. -/
theorem refl (x : InverseLimit D P) :
    inverseLimitLE D P x x :=
  fun n => le_refl (x.1 n)

/-- Transitivity of the coordinatewise inverse-limit order. -/
theorem trans (x y z : InverseLimit D P)
    (hxy : inverseLimitLE D P x y) (hyz : inverseLimitLE D P y z) :
    inverseLimitLE D P x z :=
  fun n => le_trans (hxy n) (hyz n)

/-- Antisymmetry of the coordinatewise inverse-limit order. -/
theorem antisymm (x y : InverseLimit D P)
    (hxy : inverseLimitLE D P x y) (hyx : inverseLimitLE D P y x) :
    x = y :=
```

```
  Subtype.ext (funext fun n => le_antisymm (hxy n) (hyx n))

end inverseLimitLE

end InverseLimit

end DomainTheory.ContinuousLattice
```

1.5.10 inverseLimitLT

inverseLimitLT.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.inverseLimitLE
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- Strict coordinatewise order induced by `inverseLimitLE`. -/
abbrev inverseLimitLT (x y : InverseLimit D P) : Prop :=
  inverseLimitLE D P x y ∧ ¬ inverseLimitLE D P y x

namespace inverseLimitLT

/-- The chosen strict inverse-limit order is `≤ ∧ ¬ ≥`. -/
theorem iff_LE_not_GE (x y : InverseLimit D P) :
    inverseLimitLT D P x y ↔
      inverseLimitLE D P x y ∧ ¬ inverseLimitLE D P y x :=
  Iff.rfl

end inverseLimitLT

end InverseLimit

end DomainTheory.ContinuousLattice
```

1.5.11 The limit lattice

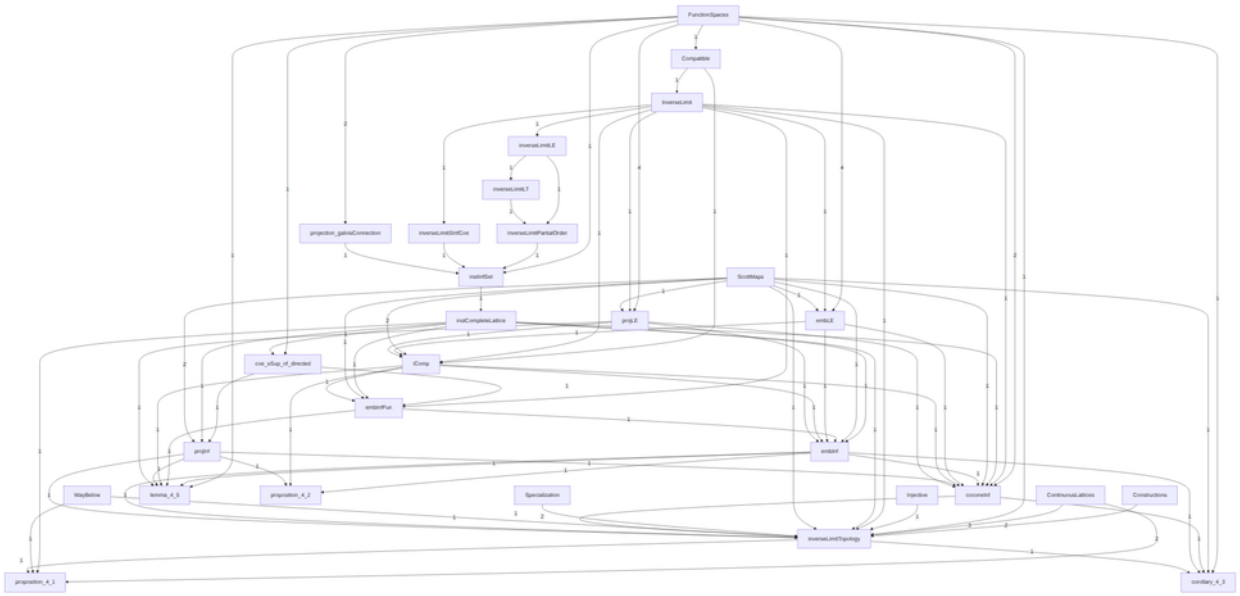


Figure 11: The limit lattice.

inverseLimitPartialOrder `inverseLimitPartialOrder.lean`

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.inverseLimitLE
public import DomainTheory.ContinuousLattice.InverseLimits.inverseLimitLT
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- The inverse limit carries the coordinatewise order inherited from its
ambient product. This declaration makes that order explicit instead of
leaving typeclass search to choose an equivalent route. -/
instance inverseLimitPartialOrder : PartialOrder (InverseLimit D P) where
  le := inverseLimitLE D P
  lt := inverseLimitLT D P
  le_refl := inverseLimitLE.refl D P
```

```
  le_trans := inverseLimitLE.trans D P
  lt_iff_le_not_ge := inverseLimitLT.iff_LE_not_GE D P
  le_antisymm := inverseLimitLE.antisymm D P

end InverseLimit

end DomainTheory.ContinuousLattice
```

inverseLimitSInfCoe inverseLimitSInfCoe.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- The coordinatewise infimum of a family of compatible sequences, before
packaging the compatibility proof. -/
noncomputable def inverseLimitSInfCoe (S : Set (InverseLimit D P)) : ∀ n, D n :=
  fun n => sInf ((fun x : InverseLimit D P => x.1 n) '' S)

end InverseLimit

end DomainTheory.ContinuousLattice
```

InfSet instInfSet.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.inverseLimitPartialOrder
public import DomainTheory.ContinuousLattice.InverseLimits.inverseLimitSInfCoe
public import DomainTheory.ContinuousLattice.InverseLimits.projection_galoisConnection
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice
```

```
open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- `jₙ` preserves arbitrary infima (it is the upper adjoint of `iₙ`). -/
theorem retr_sInf (n : ℕ) (A : Set (D (n + 1))) :
    (P n).retr (sInf A) = sInf ((P n).retr '' A) := by
  rw [(projection_galoisConnection D P n).u_sInf, sInf_image]

/-- The pointwise infimum of compatible sequences is compatible. -/
theorem compatible_sInf (S : Set (InverseLimit D P)) :
    Compatible D P (inverseLimitSInfCoe D P S) := by
  intro n
  rw [inverseLimitSInfCoe, inverseLimitSInfCoe, retr_sInf]
  congr 1
  rw [Set.image_image]
  exact Set.image_congr (by rintro x hx; exact x.2 n)

noncomputable instance instInfSet : InfSet (InverseLimit D P) :=
  ⟨fun S => ⟨inverseLimitSInfCoe D P S, compatible_sInf D P S⟩⟩

theorem coe_sInf (S : Set (InverseLimit D P)) :
    ((sInf S : InverseLimit D P) : ∀ n, D n) = inverseLimitSInfCoe D P S := rfl

theorem isGLB_sInf' (S : Set (InverseLimit D P)) :
    @IsGLB (InverseLimit D P)
      (inverseLimitPartialOrder D P).toPreorder.toLE S (sInf S) := by
  constructor
  · intro x hx n
    exact sInf_le (Set.mem_image_of_mem _ hx)
  · intro b hb n
    refine le_sInf ?_
    rintro _ ⟨x, hx, rfl⟩
    exact hb hx n

end InverseLimit

end DomainTheory.ContinuousLattice
```

instCompleteLattice instCompleteLattice.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.instInfSet
public import Mathlib.Data.Set.Basic

@[expose] public section
```

```
namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

noncomputable instance instCompleteLattice : CompleteLattice (InverseLimit D P) :=
  @completeLatticeOfInf (InverseLimit D P) (inverseLimitPartialOrder D P)
    inferInstance (isGLB_sInf' D P)

end InverseLimit

end DomainTheory.ContinuousLattice
```

coe_sSup_of_directed coe_sSup_of_directed.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.instCompleteLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- For a directed, nonempty family, the inverse-limit supremum is computed pointwise
    (each `j_n`
is Scott-continuous, so the pointwise sup of compatible sequences is compatible and is
     the least
upper bound in `D_∞`). -/
theorem coe_sSup_of_directed (S : Set (InverseLimit D P)) (hSne : S.Nonempty)
    (hSdir : DirectedOn (· ≤ ·) S) :
    ((sSup S : InverseLimit D P) : ∀ n, D n) = sSup (Subtype.val '' S) := by
  have hcompat : Compatible D P (sSup (Subtype.val '' S)) := by
    intro n
    rw [sSup_apply_eq_sSup_image, sSup_apply_eq_sSup_image]
```

```
    set A : Set (D (n + 1)) := (fun f : ∀ m, D m => f (n + 1)) '' (Subtype.val '' S)
    with hA
    have hAne : A.Nonempty := (hSne.image _).image _
    have hAdir : DirectedOn (· ≤ ·) A := by
      rintro _ ⟨_, ⟨x, hxS, rfl⟩, rfl⟩ _ ⟨_, ⟨y, hyS, rfl⟩, rfl⟩
      obtain ⟨z, hzS, hxz, hyz⟩ := hSdir x hxS y hyS
      exact ⟨z.1 (n + 1), ⟨z.1, ⟨z, hzS, rfl⟩, rfl⟩, hxz (n + 1), hyz (n + 1)⟩
    rw [(P n).retr.preservesDirectedSup_coe A hAne hAdir]
    congr 1
    rw [hA, Set.image_image]
    exact Set.image_congr (by rintro _ ⟨x, _, rfl⟩; exact x.2 n)
  set p : InverseLimit D P := ⟨sSup (Subtype.val '' S), hcompat⟩ with hp
  have hlub : IsLUB S p := by
    constructor
    · intro x hx
      refine Subtype.coe_le_coe.mp ?_
      exact le_sSup (Set.mem_image_of_mem _ hx)
    · intro b hb
      refine Subtype.coe_le_coe.mp ?_
      refine sSup_le ?_
      rintro _ ⟨x, hx, rfl⟩
      exact Subtype.coe_le_coe.mpr (hb hx)
  rw [(isLUB_sSup S).unique hlub]

end InverseLimit

end DomainTheory.ContinuousLattice
```

1.5.12 inverseLimitTopology

inverseLimitTopology.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import DomainTheory.ContinuousLattice.InverseLimits.instCompleteLattice
public import DomainTheory.ContinuousLattice.InverseLimits.projInf
public import DomainTheory.ContinuousLattice.InverseLimits.embInf
public import DomainTheory.ContinuousLattice.InverseLimits.coconeInf
public import DomainTheory.ContinuousLattice.Specialization.SpecializationLe
public import DomainTheory.ContinuousLattice.Specialization.scottTopologicalSpace
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.ContinuousLattices.lemma_3_9
public import DomainTheory.ContinuousLattice.ContinuousLattices.theorem_2_12
public import DomainTheory.ContinuousLattice.ContinuousLattices.theorem_2_12_more
public import DomainTheory.ContinuousLattice.Constructions.scottExtend_continuous
public import DomainTheory.ContinuousLattice.Constructions.
    scottExtend_eq_of_continuous
public import DomainTheory.ContinuousLattice.Injective.InjectiveSpace
public import DomainTheory.ContinuousLattice.FunctionSpaces.Basic
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Data.Set.Basic
```

```
@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set
universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

@[reducible] noncomputable def inverseLimitTopology : TopologicalSpace (InverseLimit D
    P) :=
  TopologicalSpace.induced (Subtype.val : InverseLimit D P → ∀ n, D n)
    (@Pi.topologicalSpace ℕ D (fun _ => scottTopologicalSpace))

namespace inverseLimit
/-- Every coordinate projection from the explicit product/subspace inverse-limit
    topology is
continuous. -/
theorem coordinate_continuous (n : ℕ) :
    @Continuous (InverseLimit D P) (D n) (inverseLimitTopology D P)
      scottTopologicalSpace (fun x => x.1 n) := by
  let : ∀ n, TopologicalSpace (D n) := fun _ => scottTopologicalSpace
  rw [continuous_def]
  intro U hU
  rw [isOpen_induced_iff]
  exact ⟨{x : ∀ n, D n | x n ∈ U},
    (@continuous_apply ℕ D (fun _ => scottTopologicalSpace) n).isOpen_preimage U hU,
    rfl⟩

/-- The explicit inverse-limit topology is `T₀`, since the inclusion into the product
    of the
stage Scott spaces is an embedding. -/
theorem t0Space :
    @T0Space (InverseLimit D P) (inverseLimitTopology D P) := by
  let : ∀ n, TopologicalSpace (D n) := fun _ => scottTopologicalSpace
  let : ∀ n, T0Space (D n) := fun _ => scottTopologicalSpace.t0Space
  let : TopologicalSpace (InverseLimit D P) := inverseLimitTopology D P
  have hval : Topology.IsEmbedding (Subtype.val : InverseLimit D P → ∀ n, D n) :=
    ⟨⟨rfl⟩, Subtype.val_injective⟩
  exact hval.t0Space

/-- The specialization order of the explicit inverse-limit topology is the
    componentwise order. -/
theorem specializationLe_iff (x y : InverseLimit D P) :
    @SpecializationLe (InverseLimit D P) (inverseLimitTopology D P) x y ↔ x ≤ y := by
  let stageOrder : ∀ n, Preorder (D n) := fun _ => inferInstance
  let limitOrder : Preorder (InverseLimit D P) := inferInstance
  let : ∀ n, TopologicalSpace (D n) := fun _ => scottTopologicalSpace
  let : TopologicalSpace (InverseLimit D P) := inverseLimitTopology D P
  let : ∀ n, Preorder (D n) := stageOrder
  let : Preorder (InverseLimit D P) := limitOrder
  have hval : Topology.IsEmbedding (Subtype.val : InverseLimit D P → ∀ n, D n) :=
```

```
    ⟨⟨rfl⟩, Subtype.val_injective⟩
  rw [SpecializationLe.iff_specializes, ← hval.isInducing.specializes_iff,
    specializes_pi]
  constructor
  · intro h n
    rw [← SpecializationLe.scott_iff, SpecializationLe.iff_specializes]
    exact h n
  · intro h n
    rw [← SpecializationLe.iff_specializes, SpecializationLe.scott_iff]
    exact h n

/-- **Scott 1972, Proposition 4.1 (injectivity step).** Coordinatewise maximal
    extensions
assemble to an extension into the inverse limit. Lemma 3.9 is exactly what supplies
    compatibility
of adjacent coordinates. -/
theorem isInjectiveSpace (hD : ∀ n, ContinuousLattice (D n)) :
    @InjectiveSpace.{u, u} (InverseLimit D P) (inverseLimitTopology D P) := by
  let stageOrder : ∀ n, Preorder (D n) := fun _ => inferInstance
  let limitOrder : Preorder (InverseLimit D P) := inferInstance
  let : ∀ n, TopologicalSpace (D n) := fun _ => scottTopologicalSpace
  let : TopologicalSpace (InverseLimit D P) := inverseLimitTopology D P
  let : ∀ n, Preorder (D n) := stageOrder
  let : Preorder (InverseLimit D P) := limitOrder
  constructor
  intro X Y _ _ e he f
  let fcoord (n : ℕ) (x : X) : D n := (f x).1 n
  have hfcoord (n : ℕ) : @Continuous X (D n) _ scottTopologicalSpace (fcoord n) :=
    (coordinate_continuous D P n).comp f.continuous
  let fbar (n : ℕ) : Y → D n := scottExtend e (fcoord n)
  have hfbar_cont (n : ℕ) :
      @Continuous Y (D n) _ scottTopologicalSpace (fbar n) :=
    scottExtend_continuous (hD n) e (fcoord n)
  have hfbar_compat (y : Y) (n : ℕ) :
      (P n).retr (fbar (n + 1) y) = fbar n y := by
    symm
    exact lemma_3_9 (hD n) (hD (n + 1)) (P n) e he
      (hfcoord n) (hfcoord (n + 1)) (fun x => (f x).2 n |>.symm) y
  let fbarInf (y : Y) : InverseLimit D P :=
    ⟨fun n => fbar n y, hfbar_compat y⟩
  have hfbarInf_cont :
      @Continuous Y (InverseLimit D P) _ (inverseLimitTopology D P) fbarInf := by
    change @Continuous Y (InverseLimit D P) _
      (TopologicalSpace.induced (Subtype.val : InverseLimit D P → ∀ n, D n)
        (@Pi.topologicalSpace ℕ D (fun _ => scottTopologicalSpace))) fbarInf
    rw [continuous_induced_rng]
    exact continuous_pi hfbar_cont
  refine ⟨⟨fbarInf, hfbarInf_cont⟩, fun x => ?_⟩
  apply Subtype.ext
  funext n
  exact scottExtend_eq_of_continuous (hD n) e he (fcoord n) (hfcoord n) x

/-- The Scott lattice topology constructed on `D∞` agrees with Scott's literal inverse
    -limit
topology: the subspace topology inherited from the Pi product of the stage Scott
    topologies. -/
```

```
theorem scottTopology_eq_induced_pi (hD : ∀ n, ContinuousLattice (D n)) :
    (scottTopologicalSpace : TopologicalSpace (InverseLimit D P)) =
      inverseLimitTopology D P := by
  let : TopologicalSpace (InverseLimit D P) := inverseLimitTopology D P
  let : T0Space (InverseLimit D P) := t0Space D P
  exact (theorem_2_12.backward_exact (isInjectiveSpace D P hD)
    (specializationLe_iff D P)).2

/-- **Scott 1972, §4.** Each `x ∈ D_∞` is the (monotone) lub of its projections:
`x = ⋃ₙ i_{n∞}(xₙ)`. -/
theorem eq_iSup (x : InverseLimit D P) :
    x = ⋃ n, embInf D P n (x.1 n) := by
  have hdir := embInf_family_directed D P x
  have hne : (Set.range (fun n => embInf D P n (x.1 n))).Nonempty := Set.
    range_nonempty _
  refine le_antisymm ?_ ?_
  · rw [← Subtype.coe_le_coe]
    have hcoe : ((⋃ n, embInf D P n (x.1 n) : InverseLimit D P)).1
        = sSup (Subtype.val '' Set.range (fun n => embInf D P n (x.1 n))) := by
      rw [← sSup_range]
      exact coe_sSup_of_directed D P _ hne hdir
    rw [hcoe]
    intro m
    have hmem : (embInf D P m (x.1 m)).1
        ∈ Subtype.val '' Set.range (fun n => embInf D P n (x.1 n)) :=
      ⟨embInf D P m (x.1 m), ⟨m, rfl⟩, rfl⟩
    have hle := (le_sSup hmem) m
    rwa [show (embInf D P m (x.1 m)).1 m = x.1 m from iComp_self D P m (x.1 m)] at hle
  · exact iSup_le fun n => by
      rw [← Subtype.coe_le_coe]; intro m; exact iComp_incl_le D P n x m

end inverseLimit
namespace inverseLimitTopology
/-- Proposition 4.2's projection `j_{∞n}` is a continuous coordinate map for the
    explicit
product/subspace inverse-limit topology. -/
theorem projInf_continuous (n : ℕ) :
    @Continuous (InverseLimit D P) (D n) (inverseLimitTopology D P)
      scottTopologicalSpace (projInf D P n) :=
  inverseLimit.coordinate_continuous D P n

/-- Proposition 4.2's embedding `i_{n∞}` is also continuous for Scott's literal
    inverse-limit
topology, namely the subspace topology inherited from the product `∏ₘ Dₘ`. -/
theorem embInf_continuous (hD : ∀ n, ContinuousLattice (D n)) (n : ℕ) :
    @Continuous (D n) (InverseLimit D P) scottTopologicalSpace
      (inverseLimitTopology D P) (embInf D P n) := by
  rw [← inverseLimit.scottTopology_eq_induced_pi D P hD]
  exact (embInf D P n).continuous

/-- Corollary 4.3's formula `f∞(x) = ⋃ₙ fₙ(xₙ)` is continuous not only for the
    reconstructed Scott
topology on `D∞`, but explicitly for Scott's product/subspace inverse-limit topology.
    -/
theorem coconeInf_continuous (hD : ∀ n, ContinuousLattice (D n))
    {D' : Type*} [CompleteLattice D']
```

```
    (f : ∀ n, ScottMap (D n) D') :
    @Continuous (InverseLimit D P) D' (inverseLimitTopology D P)
      scottTopologicalSpace (coconeInf D P f) := by
  rw [← inverseLimit.scottTopology_eq_induced_pi D P hD]
  exact continuous_of_preservesDirectedSup (coconeInf_preservesDirectedSup D P f)

end inverseLimitTopology

end InverseLimit

end DomainTheory.ContinuousLattice
```

1.5.13 Proposition 4.1

4.1 Proposition. Let $\langle D_n, j_n \rangle_{n=0}^{\infty}$ be an inverse system of continuous lattices where each $j_n : D_{n+1} \to D_n$ is a projection. Then the inverse limit space D_∞ is also a continuous lattice.

Proof: We need only show that D_∞ as a T_0-space is injective. So suppose $\{f_\infty : X \to D_\infty\}$ is given and $X \subseteq Y$. Define $f_n : X \to D_n$ by $f_n = j_{\infty n} \circ f_\infty$. Let $\bar{f} * n : Y \to D_n$ be the maximal extension of f_n according to 3.8. Now we can see the point of Lemma 3.9: by this construction we guarantee that $\bar{f}n = j_n \circ \bar{f}n + 1$. Hence the required $\bar{f} * \infty : Y \to D_\infty$ exists. □

I do not know at the time of writing whether this theorem on inverse limits of continuous lattices extends to sequences where, say, the j_n are only retractions. Fortunately, sufficiently many projections exist to make this construction useful. Note that by reference to the product space construction of D_∞, its lattice ordering is given simply by the relation:

$$x \sqsubseteq y \quad \text{iff} \quad x_n \sqsubseteq y_n \text{ for all } n.$$

proposition_4_1.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.InverseLimits.instCompleteLattice
public import DomainTheory.ContinuousLattice.ContinuousLattices.theorem_2_12
public import DomainTheory.ContinuousLattice.ContinuousLattices.theorem_2_12_more
public import DomainTheory.ContinuousLattice.InverseLimits.inverseLimitTopology
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]
variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))
```

```
/-- **Scott 1972, Proposition 4.1.** The inverse limit is injective by the
    coordinatewise
Proposition 3.8/Lemma 3.9 construction; Theorem 2.12 then gives the continuous-lattice
     structure
on the existing componentwise complete lattice. -/
theorem proposition_4_1 (hD : ∀ n, ContinuousLattice (D n)) :
    ContinuousLattice (InverseLimit D P) := by
  let : TopologicalSpace (InverseLimit D P) := inverseLimitTopology D P
  let : T0Space (InverseLimit D P) := inverseLimit.t0Space D P
  exact (theorem_2_12.backward_exact (inverseLimit.isInjectiveSpace D P hD)
    (inverseLimit.specializationLe_iff D P)).1

end InverseLimit
end DomainTheory.ContinuousLattice
```

1.5.14 Proposition 4.2

4.2 Proposition. Let $\langle D_n, j_n\rangle_{n=0}^{\infty}$ and D_∞ be as in 4.1. Then the maps $j_{\infty\{n} : D_\infty \to D_n\}$ are projections.

Proof: The projections $j_n : D_{n+1} \to D_n$, as we know, have their uniquely determined inverses $i_n : D_n \to D_{n+1}$. We can define $\{i_{n\infty} : D_n \to D_\infty\}$ by the equation:

$$i_{n\infty}(x) = \langle y_m\rangle_{m=0}^{\infty}$$

where

$$y_m = \begin{cases} j_m(y_{m+1}) & \text{if } m < n, \\ x & \text{if } m = n, \\ i_m(y_{m-1}) & \text{if } m > n. \end{cases}$$

The proof that $i_{n\infty}$ and $j_{\infty n}$ form a projection is now an easy computation. □

One should note also the recursion equation:

$$i_{n\infty} = i_{(n+1)\infty} \circ i_n.$$

These maps also make it possible to state this useful equation:

$$x = \bigsqcup_{n=0}^{\infty} i_{n\infty}(x_n),$$

where $x \in D_\infty$. It is easy to check that this is a monotone lub, and so we can say each $x \in D_\infty$ is the limit of its projections x_n. In fact, from what we know about projections, x_n is the best possible approximation to x in the space D_n.

proposition_4_2.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.iComp
public import DomainTheory.ContinuousLattice.InverseLimits.embInf
public import DomainTheory.ContinuousLattice.InverseLimits.projInf
public import Mathlib.Data.Set.Basic
```

```
@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- **Scott 1972, Proposition 4.2.** Each `j_{∞n} : D_∞ → Dₙ` is a projection of
    continuous
lattices, with embedding `i_{n∞} = embInf n`. -/
noncomputable def proposition_4_2 (n : ℕ) :
    IsContinuousLatticeProjection (D n) (InverseLimit D P) where
  incl := embInf D P n
  retr := projInf D P n
  retr_incl x := iComp_self D P n x
  incl_retr_le y := by
    rw [← Subtype.coe_le_coe]
    intro m
    exact iComp_incl_le D P n y m

end InverseLimit

end DomainTheory.ContinuousLattice
```

1.5.15 Corollary 4.3

4.3 Corollary. Let the spaces be as in 4.1 and 4.2. Let D' be any complete lattice and suppose continuous functions $f_n : D_n \to D'$ are given so that $f_n = f_{n+1} \circ i_n$. Then we can define $\{f_\infty : D_\infty \to D'\}$ by the equation:

$$f_\infty(x) = \bigsqcup_{n=0}^{\infty} f_n(x_n)$$

for $x \in D_\infty$, and we have $f_n = f_\infty \circ i_{n\infty}$. $\square$

The import of this last result is that within the category of complete lattices, the space D_∞ is not only the inverse limit of the D_n, but it is also the *direct* limit. (One system of spaces here uses the j_n as connecting maps, the other the i_n.) This is the algebraic result that lies at the heart of our main result about inverse limits of function spaces.

Turning to function spaces, let $D = D_0$ be a given continuous lattice. As we have seen in 3.13, there are many ways of making D_0 a projection of $D_1 = [D_0 \to D_0]$. Choose one such given by a pair i_0, j_0. Define recursively:

$$D_{n+1} = [D_n \to D_n]$$

and introduce the pairs i_{n+1}, j_{n+1} making D_{n+1} a projection of D_{n+2} by the method of 3.7. Specifically we shall have for $x \in D_{n+1}$ and $x' \in D_{n+2}$:

$$i_{n+1}(x) = i_n \circ x \circ j_n,$$

$$j_{n+1}(x') = j_n \circ x' \circ i_n.$$

Since these spaces are more than continuous lattices being function spaces, it is interesting to note that the maps i_n preserve function value as an algebraic operation as follows:

$$i_n(f(x)) = i_{n+1}(f)(i_n(x)),$$

where $x \in D_n$ and $f \in D_{n+1}$. Thus in passing to the limit space D_∞ something of functional application must also be preserved. The precise result shows that indeed D_∞ becomes its own function space.

corollary_4_3.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.embInf
public import DomainTheory.ContinuousLattice.InverseLimits.coconeInf
public import DomainTheory.ContinuousLattice.InverseLimits.inverseLimitTopology
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]
variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

variable {D' : Type*} [CompleteLattice D']

/-- **Scott 1972, Corollary 4.3.** Within complete lattices, `D_∞` is also the *direct
    * limit of
the `Dₙ` along the embeddings `iₙ`: for every compatible cocone `fₙ : Dₙ → D'` (
    continuous, with
`fₙ = f_{n+1} ∘ iₙ`) there is a **unique** continuous `f∞ : D_∞ → D'` with `fₙ = f∞ ∘
     i_{n∞}`,
namely `f∞(x) = ⋃ₙ fₙ(xₙ)`. -/
theorem corollary_4_3 (f : ∀ n, ScottMap (D n) D')
    (hf : ∀ n x, f n x = f (n + 1) ((P n).incl x)) :
    ∃! g : ScottMap (InverseLimit D P) D', ∀ n x, f n x = g (embInf D P n x) := by
  refine ⟨⟨coconeInf D P f,
    continuous_of_preservesDirectedSup (coconeInf_preservesDirectedSup D P f)⟩, ?_, ?_⟩
```

```
  · exact fun n x => (coconeInf_comp_embInf D P f hf n x).symm
  · intro g hg
    ext x
    -- `g x = f∞ x`, since `x = ⋃ₙ i_{n∞}(xₙ)` and `g` is continuous on this directed
    family
    calc g x = g (⋃ n, embInf D P n (x.1 n)) := by rw [← inverseLimit.eq_iSup D P x]
      _ = g (sSup (Set.range fun n => embInf D P n (x.1 n))) := by rw [sSup_range]
      _ = sSup (g '' Set.range fun n => embInf D P n (x.1 n)) :=
        g.preservesDirectedSup_coe _ (Set.range_nonempty _) (embInf_family_directed
    D P x)
      _ = ⋃ n, g (embInf D P n (x.1 n)) := by rw [← Set.range_comp, sSup_range]; rfl
      _ = ⋃ n, f n (x.1 n) := by simp_rw [← hg]

end InverseLimit
end DomainTheory.ContinuousLattice
```

1.5.16 The function space tower

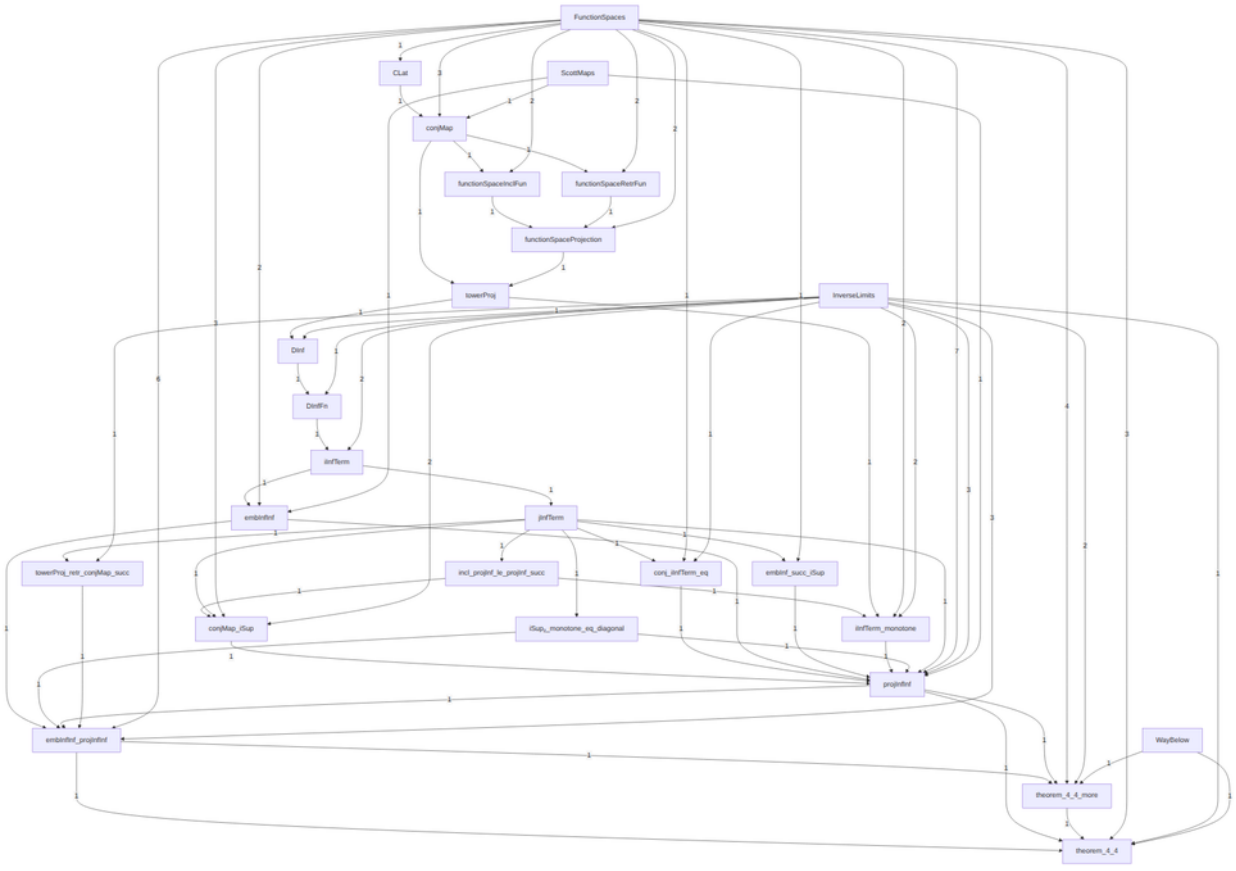


Figure 12: The function space tower.

The tower of lattices `CLat.lean`

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33

@[expose] public section

universe u

namespace DomainTheory.ContinuousLattice

/-- A complete lattice bundled with its instance, used to define the function-space
    tower by
recursion on `ℕ` (the type at level `n+1` depends on the lattice structure at level `n
    `). -/
structure CLat : Type (u + 1) where
  carrier : Type u
  [str : CompleteLattice carrier]

attribute [instance] CLat.str

/-- The tower `D₀, [D₀→D₀], [[D₀→D₀]→[D₀→D₀]], ...` as bundled complete lattices. -/
noncomputable def towerCLat (D₀ : CLat.{u}) : ℕ → CLat.{u}
  | 0 => D₀
  | (n + 1) => ⟨ScottMap (towerCLat D₀ n).carrier (towerCLat D₀ n).carrier⟩

/-- The carrier `Dₙ` of the function-space tower. -/
def towerType (D₀ : CLat.{u}) (n : ℕ) : Type u := (towerCLat D₀ n).carrier

noncomputable instance towerCompleteLattice (D₀ : CLat.{u}) (n : ℕ) :
    CompleteLattice (towerType D₀ n) := (towerCLat D₀ n).str

/-- `D_{n+1}` is *definitionally* the function space `[Dₙ → Dₙ]`. -/
theorem towerType_succ (D₀ : CLat.{u}) (n : ℕ) :
    towerType D₀ (n + 1) = ScottMap (towerType D₀ n) (towerType D₀ n) := rfl

@[simp] theorem towerType_zero (D₀ : CLat.{u}) : towerType D₀ 0 = D₀.carrier := rfl

/-- View an element of `D_{n+1}` as the Scott map `[Dₙ → Dₙ]` it definitionally is. -/
def towerToMap {D₀ : CLat.{u}} {n : ℕ} (f : towerType D₀ (n + 1)) :
    ScottMap (towerType D₀ n) (towerType D₀ n) := f

/-- Apply an element of `D_{n+1}` as a function `Dₙ → Dₙ` (definitional via `
    towerType_succ`). -/
instance towerCoeFun {D₀ : CLat.{u}} {n : ℕ} :
    CoeFun (towerType D₀ (n + 1)) (fun _ => towerType D₀ n → towerType D₀ n) where
  coe f := towerToMap f

@[simp] theorem towerToMap_coe {D₀ : CLat.{u}} {n : ℕ} (f : towerType D₀ (n + 1))
    (x : towerType D₀ n) : towerToMap f x = f x := rfl

end DomainTheory.ContinuousLattice
```

Conjugation conjMap.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaceTower.CLat
public import DomainTheory.ContinuousLattice.FunctionSpaces.comp
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

universe u
attribute [instance] CLat.str

section Conj

open Set

variable {Y Z W : Type u} [CompleteLattice Y] [CompleteLattice Z] [CompleteLattice W]

/-- Conjugation `f ↦ post ∘ f ∘ pre` as a bare function `[Y → Y] → [W → Z]`. -/
def conjMapFun (post : ScottMap Y Z) (pre : ScottMap W Y) (f : ScottMap Y Y) :
    ScottMap W Z :=
  post.comp (f.comp pre)

namespace conjMap

theorem fun_apply (post : ScottMap Y Z) (pre : ScottMap W Y) (f : ScottMap Y Y) (x : W
    ) :
    conjMapFun post pre f x = post (f (pre x)) := rfl

theorem preservesDirectedSup (post : ScottMap Y Z) (pre : ScottMap W Y) :
    PreservesDirectedSup (conjMapFun post pre) := by
  intro F hF hFdir
  apply ScottMap.ext
  intro x
  have hdir : DirectedOn (· ≤ ·) ((fun f : ScottMap Y Y => f (pre x)) '' F) := by
    rintro _ ⟨a, ha, rfl⟩ _ ⟨b, hb, rfl⟩
    obtain ⟨c, hc, hac, hbc⟩ := hFdir a ha b hb
    exact ⟨c (pre x), ⟨c, hc, rfl⟩, hac (pre x), hbc (pre x)⟩
  show post ((sSup F : ScottMap Y Y) (pre x)) = (sSup (conjMapFun post pre '' F) :
    ScottMap W Z) x
  rw [ScottMap.sSup_apply F (pre x), post.preservesDirectedSup_coe _ (hF.image _) hdir
    ,
    ScottMap.sSup_apply, Set.image_image, Set.image_image]
  rfl

end conjMap

/-- Conjugation `f ↦ post ∘ f ∘ pre` as a Scott map `[Y → Y] → [W → Z]`. -/
noncomputable def conjMap (post : ScottMap Y Z) (pre : ScottMap W Y) :
    ScottMap (ScottMap Y Y) (ScottMap W Z) :=
```

```
  ⟨conjMapFun post pre, continuous_of_preservesDirectedSup (conjMap.
    preservesDirectedSup post pre)⟩

namespace conjMap

@[simp] theorem apply (post : ScottMap Y Z) (pre : ScottMap W Y) (f : ScottMap Y Y) (x
     : W) :
    conjMap post pre f x = post (f (pre x)) := rfl

end conjMap

end Conj

end DomainTheory.ContinuousLattice
```

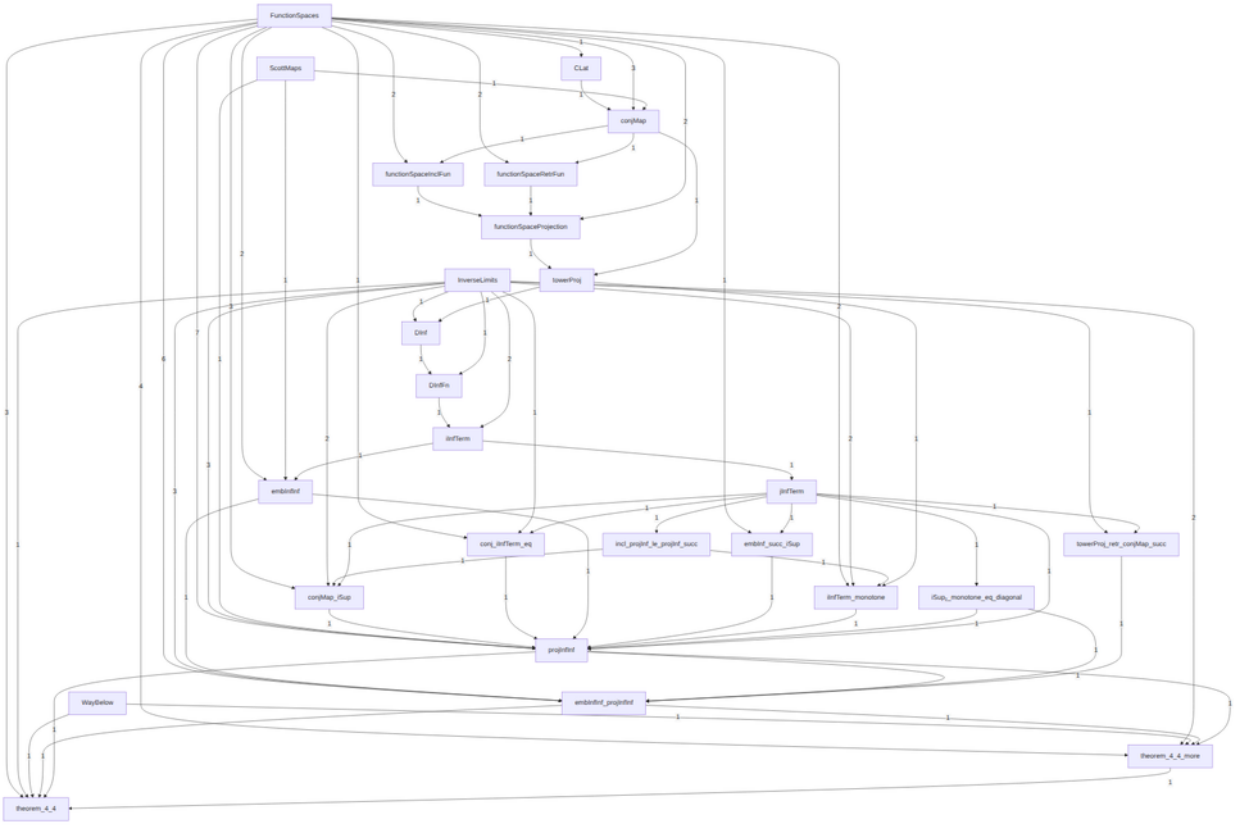


Figure 13: The function space tower.

The projection on a function space

functionSpaceInclFun `functionSpaceInclFun.lean`

```
module
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaceTower.conjMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.Basic
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection

@[expose] public section

namespace DomainTheory.ContinuousLattice

universe u
attribute [instance] CLat.str

namespace functionSpaceInclFun

/-- Continuity of the inner map `i ∘ f ∘ j`. -/
theorem continuous {D D' : Type u}
    [CompleteLattice D] [CompleteLattice D']
    (P : IsContinuousLatticeProjection D D') (f : ScottMap D D) :
    @Continuous D' D' scottTopologicalSpace scottTopologicalSpace
      (fun x => P.incl (f (P.retr x))) :=
  @Continuous.congr D' D' scottTopologicalSpace scottTopologicalSpace
    (P.incl.comp (f.comp P.retr)).1 (fun x => P.incl (f (P.retr x)))
    (P.incl.comp (f.comp P.retr)).continuous (fun _ => rfl)

end functionSpaceInclFun

/-- The inclusion part of `[P → P]`, with its underlying function explicit. -/
def functionSpaceInclFun {D D' : Type u} [CompleteLattice D] [CompleteLattice D']
    (P : IsContinuousLatticeProjection D D') (f : ScottMap D D) : ScottMap D' D' :=
  ⟨fun x => P.incl (f (P.retr x)), functionSpaceInclFun.continuous P f⟩

namespace functionSpaceInclFun

/-- Continuity, for the lattice Scott topologies on the function spaces, of
`f ↦ i ∘ f ∘ j`. -/
theorem outer_continuous {D D' : Type u}
    [CompleteLattice D] [CompleteLattice D']
    (P : IsContinuousLatticeProjection D D') :
    @Continuous (ScottMap D D) (ScottMap D' D')
      scottTopologicalSpace scottTopologicalSpace (functionSpaceInclFun P) := by
  exact @Continuous.congr (ScottMap D D) (ScottMap D' D')
    scottTopologicalSpace scottTopologicalSpace (conjMap P.incl P.retr).1
    (functionSpaceInclFun P) (conjMap P.incl P.retr).continuous
    (fun f => ScottMap.ext fun _ => rfl)

end functionSpaceInclFun

end DomainTheory.ContinuousLattice
```

functionSpaceRetrFun functionSpaceRetrFun.lean

```
module
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaceTower.conjMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.Basic
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection

@[expose] public section

namespace DomainTheory.ContinuousLattice

universe u
attribute [instance] CLat.str

namespace functionSpaceRetrFun

/-- Continuity of the inner map `j ∘ g ∘ i`. -/
theorem continuous {D D' : Type u}
    [CompleteLattice D] [CompleteLattice D']
    (P : IsContinuousLatticeProjection D D') (g : ScottMap D' D') :
    @Continuous D D scottTopologicalSpace scottTopologicalSpace
      (fun x => P.retr (g (P.incl x))) :=
  @Continuous.congr D D scottTopologicalSpace scottTopologicalSpace
    (P.retr.comp (g.comp P.incl)).1 (fun x => P.retr (g (P.incl x)))
    (P.retr.comp (g.comp P.incl)).continuous (fun _ => rfl)

end functionSpaceRetrFun

/-- The retraction part of `[P → P]`, with its underlying function explicit. -/
def functionSpaceRetrFun {D D' : Type u} [CompleteLattice D] [CompleteLattice D']
    (P : IsContinuousLatticeProjection D D') (g : ScottMap D' D') : ScottMap D D :=
  ⟨fun x => P.retr (g (P.incl x)), functionSpaceRetrFun.continuous P g⟩

namespace functionSpaceRetrFun

/-- Continuity, for the lattice Scott topologies on the function spaces, of
`g ↦ j ∘ g ∘ i`. -/
theorem outer_continuous {D D' : Type u}
    [CompleteLattice D] [CompleteLattice D']
    (P : IsContinuousLatticeProjection D D') :
    @Continuous (ScottMap D' D') (ScottMap D D)
      scottTopologicalSpace scottTopologicalSpace (functionSpaceRetrFun P) := by
  exact @Continuous.congr (ScottMap D' D') (ScottMap D D)
    scottTopologicalSpace scottTopologicalSpace (conjMap P.retr P.incl).1
    (functionSpaceRetrFun P) (conjMap P.retr P.incl).continuous
    (fun g => ScottMap.ext fun _ => rfl)

end functionSpaceRetrFun

end DomainTheory.ContinuousLattice
```

functionSpaceProjection functionSpaceProjection.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaceTower.functionSpaceInclFun
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaceTower.functionSpaceRetrFun
public import DomainTheory.ContinuousLattice.FunctionSpaces.proposition_3_7_projection
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def

@[expose] public section

namespace DomainTheory.ContinuousLattice

universe u

attribute [instance] CLat.str

namespace functionSpaceProjection

/-- Projection inequality for the diagonal Proposition 3.7 construction. -/
theorem inclRetrLe {D D' : Type u} [CompleteLattice D] [CompleteLattice D']
    (P : IsContinuousLatticeProjection D D') :
    ∀ g, functionSpaceInclFun P (functionSpaceRetrFun P g) ≤ g := by
  intro g
  exact ScottMap.le_def.mpr ((proposition_3_7_projection P P).incl_retr_le g)

/-- Retraction law for the diagonal Proposition 3.7 construction. -/
theorem retrIncl {D D' : Type u} [CompleteLattice D] [CompleteLattice D']
    (P : IsContinuousLatticeProjection D D') :
    ∀ f, functionSpaceRetrFun P (functionSpaceInclFun P f) = f :=
  (proposition_3_7_projection P P).retr_incl

end functionSpaceProjection

/-- Scott's Proposition 3.7 construction specialized diagonally. -/
noncomputable def functionSpaceProjection
    {D D' : Type u} [CompleteLattice D] [CompleteLattice D']
    (P : IsContinuousLatticeProjection D D') :
    IsContinuousLatticeProjection (ScottMap D D) (ScottMap D' D') where
  incl := ⟨functionSpaceInclFun P, functionSpaceInclFun.outer_continuous P⟩
  retr := ⟨functionSpaceRetrFun P, functionSpaceRetrFun.outer_continuous P⟩
  retr_incl := functionSpaceProjection.retrIncl P
  incl_retr_le := functionSpaceProjection.inclRetrLe P

end DomainTheory.ContinuousLattice
```

Projections of the tower towerProj.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaceTower.
    functionSpaceProjection
public import DomainTheory.ContinuousLattice.FunctionSpaceTower.conjMap

@[expose] public section

namespace DomainTheory.ContinuousLattice

attribute [instance] CLat.str
```

```
/-- The projection tower from Scott's recursion immediately before Theorem 4.4:
`j_{n+1} = [j_n → j_n]`, anchored at `j₀ : [D₀ → D₀] → D₀`.

At each successor, the maps and projection laws are the diagonal specialization
`proposition_3_7_projection P P`.  `conjMap` supplies the outer Scott-continuity needed
     to bundle
those canonical maps as an `IsContinuousLatticeProjection`. -/
noncomputable def towerProj (D₀ : CLat.{u})
    (j₀ : IsContinuousLatticeProjection D₀.carrier (ScottMap D₀.carrier D₀.carrier)) :
    ∀ n, IsContinuousLatticeProjection (towerType D₀ n) (towerType D₀ (n + 1))
  | 0 => j₀
  | (n + 1) => functionSpaceProjection (towerProj D₀ j₀ n)

namespace towerProj

theorem succ (D₀ : CLat.{u})
    (j₀ : IsContinuousLatticeProjection D₀.carrier (ScottMap D₀.carrier D₀.carrier)) (n
     : ℕ) :
    let P := towerProj D₀ j₀ n
    (towerProj D₀ j₀ (n + 1)).incl = conjMap P.incl P.retr ∧
      (towerProj D₀ j₀ (n + 1)).retr = conjMap P.retr P.incl := by
  exact ⟨rfl, rfl⟩

section Tower

variable (D₀ : CLat.{u})
  (j₀ : IsContinuousLatticeProjection D₀.carrier (ScottMap D₀.carrier D₀.carrier))

/-- **Scott 1972, recursion before Theorem 4.4 (from Proposition 3.7).**
`i_{n+1}(x) = iₙ ∘ x ∘ jₙ`. -/
theorem succ_incl_apply (n : ℕ) (x : towerType D₀ (n + 1)) (y : towerType D₀ (n + 1)) :
    ((towerProj D₀ j₀ (n + 1)).incl x) y
      = (towerProj D₀ j₀ n).incl (x ((towerProj D₀ j₀ n).retr y)) := rfl

/-- **Scott 1972, recursion before Theorem 4.4 (from Proposition 3.7).**
`j_{n+1}(x') = jₙ ∘ x' ∘ iₙ`. -/
theorem succ_retr_apply (n : ℕ) (x' : towerType D₀ (n + 2)) (y : towerType D₀ n) :
    ((towerProj D₀ j₀ (n + 1)).retr x') y
      = (towerProj D₀ j₀ n).retr (x' ((towerProj D₀ j₀ n).incl y)) := rfl

/-- **Scott 1972, §4 (application is preserved).** `iₙ(f(x)) = i_{n+1}(f)(iₙ(x))` for `
    f ∈ D_{n+1}`,
`x ∈ Dₙ`. The embeddings turn functional application into application one level up. -/
theorem incl_apply (n : ℕ) (f : towerType D₀ (n + 1)) (x : towerType D₀ n) :
    (towerProj D₀ j₀ n).incl (f x)
      = ((towerProj D₀ j₀ (n + 1)).incl f) ((towerProj D₀ j₀ n).incl x) := by
  rw [succ_incl_apply, (towerProj D₀ j₀ n).retr_incl]

end Tower

end towerProj

end DomainTheory.ContinuousLattice
```

DInf DInf.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import DomainTheory.ContinuousLattice.FunctionSpaceTower.towerProj

@[expose] public section

namespace DomainTheory.ContinuousLattice

universe u

attribute [instance] CLat.str

section LimitMaps

variable (D₀ : CLat.{u})
  (j₀ : IsContinuousLatticeProjection D₀.carrier (ScottMap D₀.carrier D₀.carrier))

/-- The inverse limit `D_∞` of the function-space tower `⟨Dₙ, jₙ⟩`. -/
abbrev DInf : Type u := InverseLimit (towerType D₀) (towerProj D₀ j₀)

end LimitMaps

end DomainTheory.ContinuousLattice
```

DInfFn DInfFn.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.instCompleteLattice
public import DomainTheory.ContinuousLattice.FunctionSpaceTower.DInf

@[expose] public section

namespace DomainTheory.ContinuousLattice

universe u

attribute [instance] CLat.str

section LimitMaps

variable (D₀ : CLat.{u})
  (j₀ : IsContinuousLatticeProjection D₀.carrier (ScottMap D₀.carrier D₀.carrier))

/-- The function space `[D_∞ → D_∞]`. -/
abbrev DInfFn : Type u := ScottMap (DInf D₀ j₀) (DInf D₀ j₀)

end LimitMaps

end DomainTheory.ContinuousLattice
```

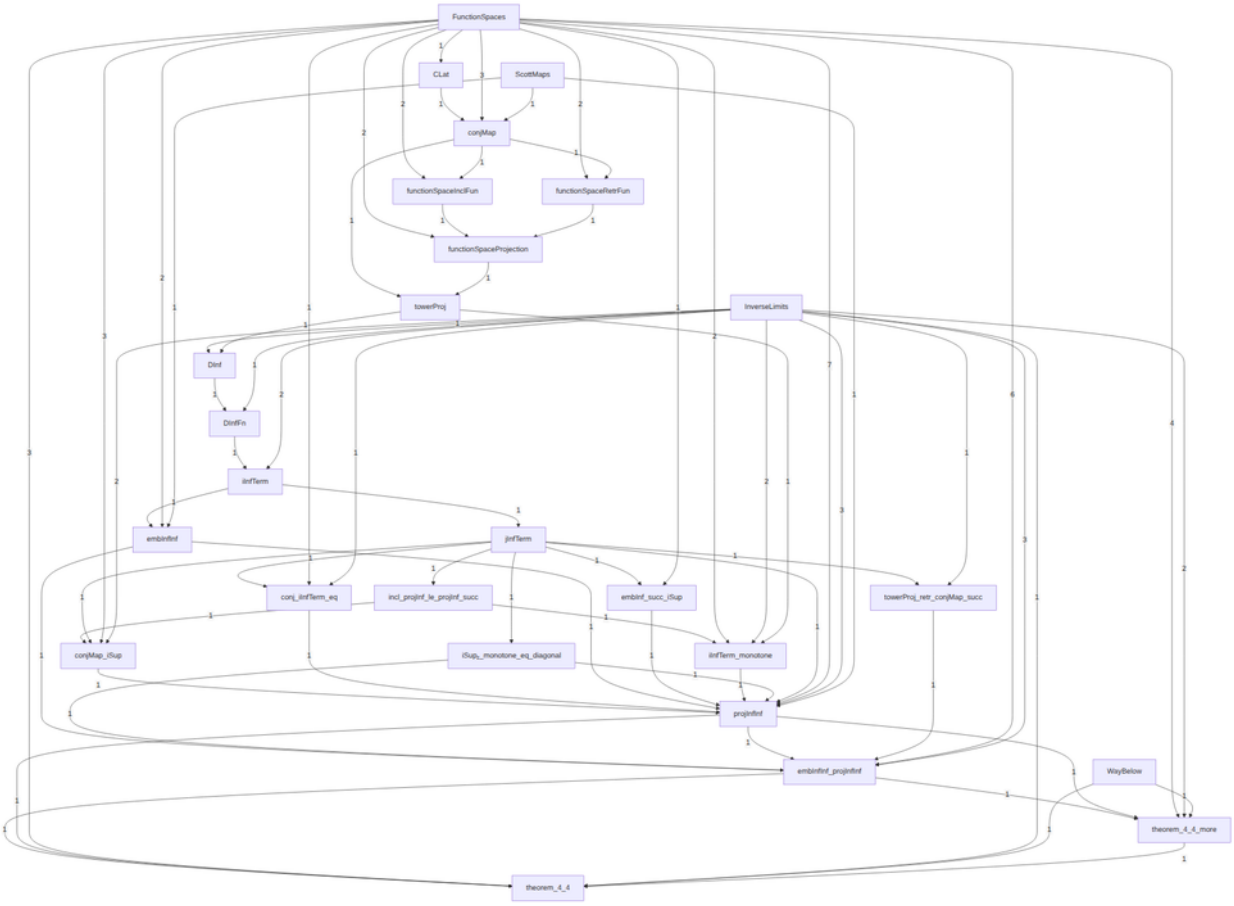


Figure 14: The function space tower.

Proof that the limit and its function space are inverse

iInfTerm `iInfTerm.lean`

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.embInf
public import DomainTheory.ContinuousLattice.InverseLimits.projInf
public import DomainTheory.ContinuousLattice.FunctionSpaceTower.DInfFn

@[expose] public section

namespace DomainTheory.ContinuousLattice

attribute [instance] CLat.str

section LimitMaps

variable (D₀ : CLat.{u})
  (j₀ : IsContinuousLatticeProjection D₀.carrier (ScottMap D₀.carrier D₀.carrier))
```

```
/-- The `n`-th summand of `i_∞`: the Scott map `x ↦ i_{n∞} ∘ x_{n+1} ∘ j_{∞n}`, where
     `x_{n+1}` is
the `(n+1)`-st component of `x ∈ D_∞`. As a map `D_∞ → [D_∞ → D_∞]` it is the
    composite of the
component projection `j_{∞(n+1)}` with conjugation by `(i_{n∞}, j_{∞n})`. -/
noncomputable def iInfTerm (n : ℕ) : ScottMap (DInf D₀ j₀) (DInfFn D₀ j₀) :=
  (conjMap (embInf (towerType D₀) (towerProj D₀ j₀) n)
           (projInf (towerType D₀) (towerProj D₀ j₀) n)).comp
    (projInf (towerType D₀) (towerProj D₀ j₀) (n + 1))

namespace iInfTerm

@[simp] theorem apply (n : ℕ) (x z : DInf D₀ j₀) :
    (iInfTerm D₀ j₀ n x) z
      = embInf (towerType D₀) (towerProj D₀ j₀) n
          ((x.1 (n + 1)) ((projInf (towerType D₀) (towerProj D₀ j₀) n) z)) := rfl

end iInfTerm

end LimitMaps

end DomainTheory.ContinuousLattice
```

embInfInf embInfInf.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaceTower.iInfTerm
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.Basic
public import DomainTheory.ContinuousLattice.ScottMaps.proposition_2_5
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

attribute [instance] CLat.str

section LimitMaps

variable (D₀ : CLat.{u})
  (j₀ : IsContinuousLatticeProjection D₀.carrier (ScottMap D₀.carrier D₀.carrier))

/-- **Scott 1972, §4 (Theorem 4.4).** The embedding `i_∞ : D_∞ → [D_∞ → D_∞]`,
`i_∞(x) = ⋃ₙ i_{n∞} ∘ x_{n+1} ∘ j_{∞n}`. It is Scott-continuous because it is a
    supremum of the
Scott maps `iInfTerm n` (suprema in `[D_∞ → [D_∞ → D_∞]]` are computed pointwise).
    -/
noncomputable def embInfInf : ScottMap (DInf D₀ j₀) (DInfFn D₀ j₀) :=
  ⋃ n, iInfTerm D₀ j₀ n

namespace embInfInf
```

```
/-- `i_∞` evaluated at `x` is the pointwise supremum of the summands `iInfTerm n x`. -/
theorem apply (x : DInf D₀ j₀) :
    embInfInf D₀ j₀ x = ⋃ n, iInfTerm D₀ j₀ n x := by
  show (sSup (Set.range (iInfTerm D₀ j₀)) : ScottMap _ _) x = _
  rw [ScottMap.sSup_apply, ← Set.range_comp, sSup_range]
  rfl

/-- `i_∞` is Scott-continuous (it is a bundled `ScottMap`). -/
theorem preservesDirectedSup :
    PreservesDirectedSup (embInfInf D₀ j₀ : DInf D₀ j₀ → DInfFn D₀ j₀) :=
  (proposition_2_5 _).mp (embInfInf D₀ j₀).continuous

end embInfInf

end LimitMaps

end DomainTheory.ContinuousLattice
```

jInfTerm jInfTerm.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaceTower.iInfTerm

@[expose] public section

namespace DomainTheory.ContinuousLattice

attribute [instance] CLat.str

section LimitMaps

variable (D₀ : CLat.{u})
  (j₀ : IsContinuousLatticeProjection D₀.carrier (ScottMap D₀.carrier D₀.carrier))

/-- The `n`-th summand of `j_∞`: the Scott map `f ↦ i_{(n+1)∞}(j_{∞n} ∘ f ∘ i_{n∞})
    `. As a map
`[D_∞ → D_∞] → D_∞` it is conjugation by `(j_{∞n}, i_{n∞})` (landing in `D_{n+1}`)
     followed by the
embedding `i_{(n+1)∞}`. -/
noncomputable def jInfTerm (n : ℕ) : ScottMap (DInfFn D₀ j₀) (DInf D₀ j₀) :=
  (embInf (towerType D₀) (towerProj D₀ j₀) (n + 1)).comp
    (conjMap (projInf (towerType D₀) (towerProj D₀ j₀) n)
             (embInf (towerType D₀) (towerProj D₀ j₀) n))

namespace jInfTerm

@[simp] theorem apply (n : ℕ) (f : DInfFn D₀ j₀) :
    jInfTerm D₀ j₀ n f
      = embInf (towerType D₀) (towerProj D₀ j₀) (n + 1)
          (conjMap (projInf (towerType D₀) (towerProj D₀ j₀) n)
                   (embInf (towerType D₀) (towerProj D₀ j₀) n) f) := rfl
```

```
end jInfTerm

end LimitMaps

end DomainTheory.ContinuousLattice
```

incl_projInf incl_projInf_le_projInf_succ.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaceTower.jInfTerm

@[expose] public section

namespace DomainTheory.ContinuousLattice

attribute [instance] CLat.str

section LimitMaps

variable (D₀ : CLat.{u})
  (j₀ : IsContinuousLatticeProjection D₀.carrier (ScottMap D₀.carrier D₀.carrier))

section Thm44b

variable (D₀ : CLat.{u})
  (j₀ : IsContinuousLatticeProjection D₀.carrier (ScottMap D₀.carrier D₀.carrier))

/-- `i_{n∞}(yₙ) ⊑ y_{n+1}`: climbing one level and embedding stays below the next
    component. -/
theorem incl_projInf_le_projInf_succ (n : ℕ) (w : DInf D₀ j₀) :
    (towerProj D₀ j₀ n).incl (projInf (towerType D₀) (towerProj D₀ j₀) n w)
      ≤ projInf (towerType D₀) (towerProj D₀ j₀) (n + 1) w := by
  have h := (towerProj D₀ j₀ n).incl_retr_le
    (projInf (towerType D₀) (towerProj D₀ j₀) (n + 1) w)
  rwa [show (towerProj D₀ j₀ n).retr (projInf (towerType D₀) (towerProj D₀ j₀) (n + 1) w
    )
      = projInf (towerType D₀) (towerProj D₀ j₀) n w from w.2 n] at h

end Thm44b

end LimitMaps

end DomainTheory.ContinuousLattice
```

iInfTerm_monotone iInfTerm_monotone.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaceTower.towerProj
public import DomainTheory.ContinuousLattice.FunctionSpaceTower.
    incl_projInf_le_projInf_succ
```

```
public import DomainTheory.ContinuousLattice.InverseLimits.embInf
public import DomainTheory.ContinuousLattice.InverseLimits.embInfFun
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone

@[expose] public section

namespace DomainTheory.ContinuousLattice

attribute [instance] CLat.str

section LimitMaps

variable (D₀ : CLat.{u})
  (j₀ : IsContinuousLatticeProjection D₀.carrier (ScottMap D₀.carrier D₀.carrier))

section Thm44b

variable (D₀ : CLat.{u})
  (j₀ : IsContinuousLatticeProjection D₀.carrier (ScottMap D₀.carrier D₀.carrier))

/-- `i_∞` is monotone in its summand index. -/
theorem iInfTerm_le_succ (x : DInf D₀ j₀) (m : ℕ) :
    iInfTerm D₀ j₀ m x ≤ iInfTerm D₀ j₀ (m + 1) x := by
  rw [ScottMap.le_def]
  intro z
  rw [iInfTerm.apply, iInfTerm.apply,
    ← embInf_succ (towerType D₀) (towerProj D₀ j₀) m
      (x.1 (m + 1) (projInf (towerType D₀) (towerProj D₀ j₀) m z))]
  refine embInf_monotone (towerType D₀) (towerProj D₀ j₀) (m + 1) ?_
  have hab : (towerProj D₀ j₀ m).incl (projInf (towerType D₀) (towerProj D₀ j₀) m z)
      ≤ projInf (towerType D₀) (towerProj D₀ j₀) (m + 1) z :=
    incl_projInf_le_projInf_succ D₀ j₀ m z
  calc (towerProj D₀ j₀ m).incl
          (x.1 (m + 1) (projInf (towerType D₀) (towerProj D₀ j₀) m z))
      = (towerProj D₀ j₀ m).incl
          (((towerProj D₀ j₀ (m + 1)).retr (x.1 (m + 2)))
            (projInf (towerType D₀) (towerProj D₀ j₀) m z)) := by
        rw [show ((towerProj D₀ j₀ (m + 1)).retr (x.1 (m + 2))) = x.1 (m + 1) from x.2
    (m + 1)]
    _ = (towerProj D₀ j₀ m).incl
          ((towerProj D₀ j₀ m).retr
            (x.1 (m + 2) ((towerProj D₀ j₀ m).incl
              (projInf (towerType D₀) (towerProj D₀ j₀) m z)))) := by
        rw [towerProj.succ_retr_apply]
    _ ≤ x.1 (m + 2) ((towerProj D₀ j₀ m).incl
          (projInf (towerType D₀) (towerProj D₀ j₀) m z)) :=
        (towerProj D₀ j₀ m).incl_retr_le _
    _ ≤ x.1 (m + 2) (projInf (towerType D₀) (towerProj D₀ j₀) (m + 1) z) :=
        ScottMap.monotone (x.1 (m + 2)) hab

theorem iInfTerm_monotone (x : DInf D₀ j₀) : Monotone (fun m => iInfTerm D₀ j₀ m x) :=
  monotone_nat_of_le_succ (iInfTerm_le_succ D₀ j₀ x)

end Thm44b
```

```
end LimitMaps

end DomainTheory.ContinuousLattice
```

embInf_succ_iSup embInf_succ_iSup.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaceTower.jInfTerm
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

attribute [instance] CLat.str

section LimitMaps

variable (D₀ : CLat.{u})
  (j₀ : IsContinuousLatticeProjection D₀.carrier (ScottMap D₀.carrier D₀.carrier))

section Thm44b

variable (D₀ : CLat.{u})
  (j₀ : IsContinuousLatticeProjection D₀.carrier (ScottMap D₀.carrier D₀.carrier))

/-- The inverse-limit embedding commutes with a supremum in `D_{n+1}` (monotone ⇒
    directed). -/
theorem embInf_succ_iSup (n : ℕ) (f : ℕ → towerType D₀ (n + 1)) (hf : Monotone f) :
    embInf (towerType D₀) (towerProj D₀ j₀) (n + 1) (⋃ m, f m) =
      ⋃ m, embInf (towerType D₀) (towerProj D₀ j₀) (n + 1) (f m) := by
  have hdir : DirectedOn (· ≤ ·) (Set.range f) :=
    directedOn_range.2 fun a b => ⟨max a b, hf (le_max_left a b), hf (le_max_right a b)
    ⟩
  have hne := Set.range_nonempty f
  rw [show (⋃ m, f m) = sSup (Set.range f) from sSup_range.symm,
    (embInf (towerType D₀) (towerProj D₀ j₀) (n + 1)).preservesDirectedSup_coe
      (Set.range f) hne hdir, ← Set.range_comp]
  rfl

end Thm44b

end LimitMaps

end DomainTheory.ContinuousLattice
```

iSup2 iSup2_monotone_eq_diagonal.lean

```
module
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaceTower.jInfTerm

@[expose] public section

namespace DomainTheory.ContinuousLattice

attribute [instance] CLat.str

section LimitMaps

variable (D₀ : CLat.{u})
  (j₀ : IsContinuousLatticeProjection D₀.carrier (ScottMap D₀.carrier D₀.carrier))

section Thm44b

variable (D₀ : CLat.{u})
  (j₀ : IsContinuousLatticeProjection D₀.carrier (ScottMap D₀.carrier D₀.carrier))

/-- A monotone double `iSup` over `ℕ × ℕ` equals the diagonal `iSup`. -/
theorem iSup₂_monotone_eq_diagonal {α : Type*} [CompleteLattice α] (f : ℕ → ℕ → α)
    (hfm : ∀ n, Monotone (f n)) (hfn : ∀ m, Monotone (fun n => f n m)) :
    ⨆ n, ⨆ m, f n m = ⨆ n, f n n := by
  apply le_antisymm
  · refine iSup_le fun n => iSup_le fun m => ?_
    have hk : n ≤ n ⊔ m := le_sup_left
    have hk' : m ≤ n ⊔ m := le_sup_right
    calc f n m ≤ f (n ⊔ m) m := hfn m hk
      _ ≤ f (n ⊔ m) (n ⊔ m) := hfm (n ⊔ m) hk'
      _ ≤ ⨆ n', f n' n' := le_iSup (fun n' => f n' n') (n ⊔ m)
  · refine iSup_le fun n => le_trans (le_iSup (f n) n) (le_iSup (fun n' => ⨆ m, f n' m)
    n)

end Thm44b

end LimitMaps

end DomainTheory.ContinuousLattice
```

conjMap conjMap_iSup.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaceTower.jInfTerm
public import DomainTheory.ContinuousLattice.FunctionSpaceTower.
   incl_projInf_le_projInf_succ
public import DomainTheory.ContinuousLattice.InverseLimits.embInf
public import DomainTheory.ContinuousLattice.InverseLimits.embInfFun
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Data.Set.Basic

@[expose] public section
```

```
namespace DomainTheory.ContinuousLattice

attribute [instance] CLat.str

section LimitMaps

variable (D₀ : CLat.{u})
  (j₀ : IsContinuousLatticeProjection D₀.carrier (ScottMap D₀.carrier D₀.carrier))

/-- Conjugation commutes with a supremum indexed by `ℕ` (the range is directed when `f`
    is monotone).
Since `conjMap post pre` is itself a Scott map, this is just preservation of directed
    suprema. -/
theorem conjMap_iSup (n : ℕ) (post : ScottMap (DInf D₀ j₀) (towerType D₀ n))
    (pre : ScottMap (towerType D₀ n) (DInf D₀ j₀))
    (f : ℕ → ScottMap (DInf D₀ j₀) (DInf D₀ j₀)) (hf : Monotone f) :
    conjMap post pre (⋃ m, f m) = ⋃ m, conjMap post pre (f m) := by
  have hdir : DirectedOn (· ≤ ·) (Set.range f) :=
    directedOn_range.2 fun a b => ⟨max a b, hf (le_max_left a b), hf (le_max_right a b)
    ⟩
  have hne := Set.range_nonempty f
  rw [show (⋃ m, f m) = sSup (Set.range f) from sSup_range.symm,
    (conjMap post pre).preservesDirectedSup_coe (Set.range f) hne hdir, ← Set.
    range_comp]
  rfl

/-- **Conjugation climbs the tower (one step).** Lifting `conjMap (j_{∞n}, i_{n∞}) f`
    along `i_{n+1}`
stays below `conjMap (j_{∞(n+1)}, i_{(n+1)∞}) f`. Both sides live in `D_{n+2}`; the
    inequality is the
algebraic content that makes the double sup defining `j_∞ ∘ i_∞` monotone in the
    level index `n`. -/
theorem conjMap_incl_le_conjMap_succ (n : ℕ) (f : DInfFn D₀ j₀) :
    (towerProj D₀ j₀ (n + 1)).incl
        (conjMap (projInf (towerType D₀) (towerProj D₀ j₀) n)
                 (embInf (towerType D₀) (towerProj D₀ j₀) n) f)
      ≤ conjMap (projInf (towerType D₀) (towerProj D₀ j₀) (n + 1))
                (embInf (towerType D₀) (towerProj D₀ j₀) (n + 1)) f := by
  refine ScottMap.le_def.mpr fun y => ?_
  calc (towerProj D₀ j₀ n).incl
          (projInf (towerType D₀) (towerProj D₀ j₀) n
            (f (embInf (towerType D₀) (towerProj D₀ j₀) n ((towerProj D₀ j₀ n).retr y)))
    )
      ≤ (towerProj D₀ j₀ n).incl
          (projInf (towerType D₀) (towerProj D₀ j₀) n
            (f (embInf (towerType D₀) (towerProj D₀ j₀) (n + 1) y))) := by
        refine (towerProj D₀ j₀ n).incl.monotone
          ((projInf (towerType D₀) (towerProj D₀ j₀) n).monotone (f.monotone ?_))
        rw [← embInf_succ (towerType D₀) (towerProj D₀ j₀) n ((towerProj D₀ j₀ n).retr
    y)]
        exact embInf_monotone (towerType D₀) (towerProj D₀ j₀) (n + 1)
          ((towerProj D₀ j₀ n).incl_retr_le y)
    _ ≤ projInf (towerType D₀) (towerProj D₀ j₀) (n + 1)
          (f (embInf (towerType D₀) (towerProj D₀ j₀) (n + 1) y)) :=
        incl_projInf_le_projInf_succ D₀ j₀ n
          (f (embInf (towerType D₀) (towerProj D₀ j₀) (n + 1) y))
```

```
end LimitMaps

end DomainTheory.ContinuousLattice
```

conj conj_iInfTerm_eq.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaceTower.jInfTerm
public import DomainTheory.ContinuousLattice.InverseLimits.proposition_4_2
public import DomainTheory.ContinuousLattice.FunctionSpaces.proposition_3_7_projection

@[expose] public section

namespace DomainTheory.ContinuousLattice

attribute [instance] CLat.str

section LimitMaps

variable (D₀ : CLat.{u})
  (j₀ : IsContinuousLatticeProjection D₀.carrier (ScottMap D₀.carrier D₀.carrier))

section Thm44b

variable (D₀ : CLat.{u})
  (j₀ : IsContinuousLatticeProjection D₀.carrier (ScottMap D₀.carrier D₀.carrier))

/-- **Diagonal simplification.** `conjMap (j_{∞n}, i_{n∞})` applied to the `n`-th
    summand of `i_∞`
recovers the component `x_{n+1}`. This is Proposition 3.7's equation
`j_{[·]} ∘ i_{[·]} = id`, diagonally specialized to the projection from Proposition 4.2.
    -/
theorem conj_iInfTerm_eq (n : ℕ) (x : DInf D₀ j₀) :
    conjMap (projInf (towerType D₀) (towerProj D₀ j₀) n)
            (embInf (towerType D₀) (towerProj D₀ j₀) n)
            (iInfTerm D₀ j₀ n x) = x.1 (n + 1) :=
  (proposition_3_7_projection
      (proposition_4_2 (towerType D₀) (towerProj D₀ j₀) n)
      (proposition_4_2 (towerType D₀) (towerProj D₀ j₀) n)).retr_incl
    (projInf (towerType D₀) (towerProj D₀ j₀) (n + 1) x)

end Thm44b

end LimitMaps

end DomainTheory.ContinuousLattice
```

towerProj towerProj_retr_conjMap_succ.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaceTower.jInfTerm
public import DomainTheory.ContinuousLattice.InverseLimits.embInf

@[expose] public section

namespace DomainTheory.ContinuousLattice

attribute [instance] CLat.str

section LimitMaps

variable (D₀ : CLat.{u})
  (j₀ : IsContinuousLatticeProjection D₀.carrier (ScottMap D₀.carrier D₀.carrier))

section Thm44b

variable (D₀ : CLat.{u})
  (j₀ : IsContinuousLatticeProjection D₀.carrier (ScottMap D₀.carrier D₀.carrier))

/-- **Step 1 (Lemma-4.5 recursion).** `j_{n+1}(j_{∞(n+1)} ∘ f ∘ i_{(n+1)∞}) = j_{∞n}
    ∘ f ∘ i_{n∞}`.
This is the equality counterpart of `conjMap_incl_le_conjMap_succ`; it is the
    hypothesis Lemma 4.5
needs to recognize `j_∞(f)` from its restrictions. -/
theorem towerProj_retr_conjMap_succ (n : ℕ) (f : DInfFn D₀ j₀) :
    (towerProj D₀ j₀ (n + 1)).retr
        (conjMap (projInf (towerType D₀) (towerProj D₀ j₀) (n + 1))
                 (embInf (towerType D₀) (towerProj D₀ j₀) (n + 1)) f)
      = conjMap (projInf (towerType D₀) (towerProj D₀ j₀) n)
                (embInf (towerType D₀) (towerProj D₀ j₀) n) f := by
  apply ScottMap.ext
  intro y
  show (towerProj D₀ j₀ n).retr
      (projInf (towerType D₀) (towerProj D₀ j₀) (n + 1)
        (f (embInf (towerType D₀) (towerProj D₀ j₀) (n + 1)
          ((towerProj D₀ j₀ n).incl y))))
    = projInf (towerType D₀) (towerProj D₀ j₀) n
        (f (embInf (towerType D₀) (towerProj D₀ j₀) n y))
  rw [embInf_succ (towerType D₀) (towerProj D₀ j₀) n y]
  exact (f (embInf (towerType D₀) (towerProj D₀ j₀) n y)).2 n

end Thm44b

end LimitMaps

end DomainTheory.ContinuousLattice
```

projInfInf projInfInf.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaceTower.jInfTerm
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaceTower.embInfInf
public import DomainTheory.ContinuousLattice.FunctionSpaceTower.conjMap_iSup
public import DomainTheory.ContinuousLattice.FunctionSpaceTower.embInf_succ_iSup
public import DomainTheory.ContinuousLattice.FunctionSpaceTower.conj_iInfTerm_eq
public import DomainTheory.ContinuousLattice.FunctionSpaceTower.iInfTerm_monotone
public import DomainTheory.ContinuousLattice.FunctionSpaceTower.iSup₂
    _monotone_eq_diagonal
public import DomainTheory.ContinuousLattice.InverseLimits.inverseLimitTopology
public import DomainTheory.ContinuousLattice.InverseLimits.embInf
public import DomainTheory.ContinuousLattice.InverseLimits.embInfFun
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.Basic
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import DomainTheory.ContinuousLattice.FunctionSpaces.comp_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.idMap_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.idMap
public import DomainTheory.ContinuousLattice.ScottMaps.proposition_2_5
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

attribute [instance] CLat.str

section LimitMaps

variable (D₀ : CLat.{u})
  (j₀ : IsContinuousLatticeProjection D₀.carrier (ScottMap D₀.carrier D₀.carrier))

/-- **Scott 1972, §4 (Theorem 4.4).** The projection `j_∞ : [D_∞ → D_∞] → D_∞`,
`j_∞(f) = ⋃ₙ i_{(n+1)∞}(j_{∞n} ∘ f ∘ i_{n∞})`. Scott-continuous as a supremum of the
    Scott maps
`jInfTerm n`. -/
noncomputable def projInfInf : ScottMap (DInfFn D₀ j₀) (DInf D₀ j₀) :=
  ⋃ n, jInfTerm D₀ j₀ n

namespace projInfInf

/-- `j_∞` evaluated at `f` is the supremum of the summands `jInfTerm n f`. -/
theorem apply (f : DInfFn D₀ j₀) :
    projInfInf D₀ j₀ f = ⋃ n, jInfTerm D₀ j₀ n f := by
  show (sSup (Set.range (jInfTerm D₀ j₀)) : ScottMap _ _) f = _
  rw [ScottMap.sSup_apply, ← Set.range_comp, sSup_range]
  rfl

/-- `j_∞` is Scott-continuous (it is a bundled `ScottMap`). -/
theorem preservesDirectedSup :
    PreservesDirectedSup (projInfInf D₀ j₀ : DInfFn D₀ j₀ → DInf D₀ j₀) :=
  (proposition_2_5 _).mp (projInfInf D₀ j₀).continuous

/-- `j_∞(i_∞(x)) = ⋃ₙ i_{(n+1)∞}(x_{n+1})`. -/
theorem embInfInf_eq (x : DInf D₀ j₀) :
    projInfInf D₀ j₀ (embInfInf D₀ j₀ x) = ⋃ n, embInf (towerType D₀) (towerProj D₀ j₀)
    (n + 1) (x.1 (n + 1)) := by
```

```
rw [apply]
set g := fun n m =>
  embInf (towerType D₀) (towerProj D₀ j₀) (n + 1)
    (conjMap (projInf (towerType D₀) (towerProj D₀ j₀) n)
             (embInf (towerType D₀) (towerProj D₀ j₀) n)
             (iInfTerm D₀ j₀ m x)) with hg
have hmono (n : ℕ) : Monotone (fun m =>
    conjMap (projInf (towerType D₀) (towerProj D₀ j₀) n)
            (embInf (towerType D₀) (towerProj D₀ j₀) n)
            (iInfTerm D₀ j₀ m x)) := fun a b hab =>
  (conjMap (projInf (towerType D₀) (towerProj D₀ j₀) n)
           (embInf (towerType D₀) (towerProj D₀ j₀) n)).monotone
    (ScottMap.le_def.mpr (iInfTerm_monotone D₀ j₀ x hab))
have hinner (n : ℕ) : jInfTerm D₀ j₀ n (embInfInf D₀ j₀ x) = ⋃ m, g n m := by
  calc jInfTerm D₀ j₀ n (embInfInf D₀ j₀ x)
      = embInf (towerType D₀) (towerProj D₀ j₀) (n + 1)
          (⋃ m, conjMap (projInf (towerType D₀) (towerProj D₀ j₀) n)
                        (embInf (towerType D₀) (towerProj D₀ j₀) n)
                        (iInfTerm D₀ j₀ m x)) := by
        rw [jInfTerm.apply, embInfInf.apply,
          conjMap_iSup D₀ j₀ n (projInf (towerType D₀) (towerProj D₀ j₀) n)
            (embInf (towerType D₀) (towerProj D₀ j₀) n)
            (fun m => iInfTerm D₀ j₀ m x) (iInfTerm_monotone D₀ j₀ x)]
        rfl
    _ = ⋃ m, embInf (towerType D₀) (towerProj D₀ j₀) (n + 1)
            (conjMap (projInf (towerType D₀) (towerProj D₀ j₀) n)
                     (embInf (towerType D₀) (towerProj D₀ j₀) n)
                     (iInfTerm D₀ j₀ m x)) :=
        embInf_succ_iSup D₀ j₀ n _ (hmono n)
    _ = ⋃ m, g n m := by simp only [hg]
have g_mono_m (n : ℕ) : Monotone (g n) := by
  intro a b hab
  rw [hg]
  exact (embInf (towerType D₀) (towerProj D₀ j₀) (n + 1)).monotone
    ((conjMap (projInf (towerType D₀) (towerProj D₀ j₀) n)
              (embInf (towerType D₀) (towerProj D₀ j₀) n)).monotone
      (ScottMap.le_def.mpr (iInfTerm_monotone D₀ j₀ x hab)))
have g_mono_n_succ (m n : ℕ) : g n m ≤ g (n + 1) m := by
  rw [hg]
  dsimp only
  rw [← embInf_succ (towerType D₀) (towerProj D₀ j₀) (n + 1)
      (conjMap (projInf (towerType D₀) (towerProj D₀ j₀) n)
               (embInf (towerType D₀) (towerProj D₀ j₀) n)
               (iInfTerm D₀ j₀ m x))]
  exact embInf_monotone (towerType D₀) (towerProj D₀ j₀) (n + 2)
    (conjMap_incl_le_conjMap_succ D₀ j₀ n (iInfTerm D₀ j₀ m x))
have g_mono_n (m : ℕ) : Monotone (fun n => g n m) :=
  monotone_nat_of_le_succ (g_mono_n_succ m)
have hin : (⋃ n, jInfTerm D₀ j₀ n (embInfInf D₀ j₀ x)) = ⋃ n, ⋃ m, g n m := by
  congr 1
  funext n
  exact hinner n
rw [hin, iSup₂_monotone_eq_diagonal g g_mono_m g_mono_n]
congr 1
funext n
rw [hg]
```

```
  dsimp only
  rw [conj_iInfTerm_eq D₀ j₀ n x]

/-- **Scott 1972, §4 (Theorem 4.4, first half).** `j_∞ ∘ i_∞ = id` on `D_∞`. -/
theorem comp_embInfInf :
    (projInfInf D₀ j₀).comp (embInfInf D₀ j₀) = ScottMap.idMap := by
  apply ScottMap.ext
  intro x
  have hmono : Monotone (fun k => embInf (towerType D₀) (towerProj D₀ j₀) k (x.1 k)) :=
    monotone_nat_of_le_succ (embInf_le_succ (towerType D₀) (towerProj D₀ j₀) x)
  rw [ScottMap.comp_apply, ScottMap.idMap_apply, embInfInf_eq D₀ j₀ x,
    Monotone.iSup_nat_add hmono 1]
  exact (inverseLimit.eq_iSup (towerType D₀) (towerProj D₀ j₀) x).symm

theorem embInfInf (x : DInf D₀ j₀) :
    projInfInf D₀ j₀ (embInfInf D₀ j₀ x) = x := by
  have h := congrArg (fun g => g x) (comp_embInfInf D₀ j₀)
  simpa [ScottMap.comp_apply, ScottMap.idMap_apply] using h

end projInfInf

end LimitMaps

end DomainTheory.ContinuousLattice
```

embInfInf embInfInf_projInfInf.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaceTower.embInfInf
public import DomainTheory.ContinuousLattice.FunctionSpaceTower.projInfInf
public import DomainTheory.ContinuousLattice.FunctionSpaceTower.iSup₂
    _monotone_eq_diagonal
public import DomainTheory.ContinuousLattice.FunctionSpaceTower.
    towerProj_retr_conjMap_succ
public import DomainTheory.ContinuousLattice.InverseLimits.lemma_4_5
public import DomainTheory.ContinuousLattice.InverseLimits.inverseLimitTopology
public import DomainTheory.ContinuousLattice.InverseLimits.embInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.comp_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.idMap_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.idMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

attribute [instance] CLat.str

section LimitMaps
```

```
variable (D₀ : CLat.{u})
  (j₀ : IsContinuousLatticeProjection D₀.carrier (ScottMap D₀.carrier D₀.carrier))

/-- **Scott 1972, §4 (Theorem 4.4, second half).** `i_∞ ∘ j_∞ = id` on `[D_∞ → D_∞
    ]`. -/
theorem embInfInf_comp_projInfInf :
    (embInfInf D₀ j₀).comp (projInfInf D₀ j₀) = ScottMap.idMap := by
  apply ScottMap.ext
  intro f
  rw [ScottMap.comp_apply, ScottMap.idMap_apply]
  apply ScottMap.ext
  intro z
  -- `r_n = i_{n∞} ∘ j_{∞n}`, the Proposition-4.2 approximation of the identity on `D_
    ∞`.
  set r : ℕ → ScottMap (DInf D₀ j₀) (DInf D₀ j₀) :=
    fun n => (embInf (towerType D₀) (towerProj D₀ j₀) n).comp
               (projInf (towerType D₀) (towerProj D₀ j₀) n) with hr
  have hrw : ∀ n (w : DInf D₀ j₀), r n w
      = embInf (towerType D₀) (towerProj D₀ j₀) n
          (projInf (towerType D₀) (towerProj D₀ j₀) n w) := by
    intro n w
    simp only [hr, ScottMap.comp_apply]
  have hr_mono : ∀ (w : DInf D₀ j₀), Monotone (fun m => r m w) := by
    intro w
    refine monotone_nat_of_le_succ (fun m => ?_)
    show r m w ≤ r (m + 1) w
    rw [hrw, hrw]
    exact embInf_le_succ (towerType D₀) (towerProj D₀ j₀) w m
  -- The functional equation `id = ⋃_n r_n` (remark following Proposition 4.2), pointwise
    .
  have hA : ∀ (w : DInf D₀ j₀), w = ⋃ m, r m w := by
    intro w
    have h1 : (⋃ m, r m w)
        = ⋃ m, embInf (towerType D₀) (towerProj D₀ j₀) m
                 (projInf (towerType D₀) (towerProj D₀ j₀) m w) :=
      iSup_congr (fun m => hrw m w)
    rw [h1]
    exact inverseLimit.eq_iSup (towerType D₀) (towerProj D₀ j₀) w
  -- Evaluating a supremum of Scott maps `D_∞ → D_∞` at a point is pointwise.
  have hsup_apply : ∀ (g : ℕ → ScottMap (DInf D₀ j₀) (DInf D₀ j₀)) (w : DInf D₀ j₀),
      (⋃ n, g n) w = ⋃ n, g n w := by
    intro g w
    rw [show (⋃ n, g n) = sSup (Set.range g) from sSup_range.symm,
      ScottMap.sSup_apply, ← Set.range_comp, sSup_range]
    rfl
  -- A Scott map commutes with a monotone `ℕ`-indexed supremum.
  have hcont : ∀ (g : ScottMap (DInf D₀ j₀) (DInf D₀ j₀)) (a : ℕ → DInf D₀ j₀),
      Monotone a → g (⋃ m, a m) = ⋃ m, g (a m) := by
    intro g a ha
    have hdir : DirectedOn (· ≤ ·) (Set.range a) :=
      directedOn_range.2 fun i j => ⟨max i j, ha (le_max_left i j), ha (le_max_right i
    j)⟩
    rw [show (⋃ m, a m) = sSup (Set.range a) from sSup_range.symm,
      g.preservesDirectedSup_coe (Set.range a) (Set.range_nonempty a) hdir,
      ← Set.range_comp, sSup_range]
    rfl
```

```
  -- `j_∞(f) = ⋃ₖ i_{(k+1)∞}(j_{∞k} ∘ f ∘ i_{k∞})`.
  have hpi : projInfInf D₀ j₀ f
      = ⋃ k, embInf (towerType D₀) (towerProj D₀ j₀) (k + 1)
          (conjMap (projInf (towerType D₀) (towerProj D₀ j₀) k)
                   (embInf (towerType D₀) (towerProj D₀ j₀) k) f) := by
    rw [projInfInf.apply]
    exact iSup_congr (fun n => jInfTerm.apply D₀ j₀ n f)
  -- Lemma 4.5: the `(n+1)`-st component of `j_∞(f)` is the restriction `j_{∞n} ∘ f ∘
    i_{n∞}`.
  have hcoord : ∀ n, (projInfInf D₀ j₀ f).1 (n + 1)
      = conjMap (projInf (towerType D₀) (towerProj D₀ j₀) n)
                (embInf (towerType D₀) (towerProj D₀ j₀) n) f := by
    intro n
    rw [hpi]
    exact lemma_4_5 (towerType D₀) (towerProj D₀ j₀)
      (fun k => conjMap (projInf (towerType D₀) (towerProj D₀ j₀) k)
                        (embInf (towerType D₀) (towerProj D₀ j₀) k) f)
      (fun m => towerProj_retr_conjMap_succ D₀ j₀ m f) n
  -- Evaluate `i_∞(j_∞(f))` pointwise as a sup of summands.
  have hev : embInfInf D₀ j₀ (projInfInf D₀ j₀ f) z
      = ⋃ n, (iInfTerm D₀ j₀ n (projInfInf D₀ j₀ f)) z := by
    rw [embInfInf.apply]
    exact hsup_apply (fun n => iInfTerm D₀ j₀ n (projInfInf D₀ j₀ f)) z
  -- Each summand is `rₙ (f (rₙ z))` (using `hcoord` and `conjMap`).
  have hterm : ∀ n, (iInfTerm D₀ j₀ n (projInfInf D₀ j₀ f)) z = r n (f (r n z)) := by
    intro n
    rw [hrw, hrw, iInfTerm.apply, hcoord n]
    rfl
  have hmono_frz : Monotone (fun m => f (r m z)) :=
    fun a b hab => f.monotone (hr_mono z hab)
  have hfm : ∀ n, Monotone (fun m => r n (f (r m z))) :=
    fun n _ _ hab => (r n).monotone (f.monotone (hr_mono z hab))
  have hfn : ∀ m, Monotone (fun n => r n (f (r m z))) :=
    fun m => hr_mono (f (r m z))
  -- The analytic step (Scott ~1326-1334): confine the lub via continuity of `f`, then
    collapse the
  -- monotone double sup to its diagonal.
  have hfz : f z = ⋃ n, r n (f (r n z)) :=
    calc f z = ⋃ k, r k (f z) := hA (f z)
      _ = ⋃ k, r k (f (⋃ m, r m z)) := by
            refine iSup_congr (fun k => ?_)
            rw [← hA z]
      _ = ⋃ k, r k (⋃ m, f (r m z)) := by
            refine iSup_congr (fun k => ?_)
            rw [hcont f (fun m => r m z) (hr_mono z)]
      _ = ⋃ k, ⋃ m, r k (f (r m z)) :=
            iSup_congr (fun k => hcont (r k) (fun m => f (r m z)) hmono_frz)
      _ = ⋃ n, r n (f (r n z)) :=
            iSup₂_monotone_eq_diagonal (fun n m => r n (f (r m z))) hfm hfn
  calc embInfInf D₀ j₀ (projInfInf D₀ j₀ f) z
      = ⋃ n, (iInfTerm D₀ j₀ n (projInfInf D₀ j₀ f)) z := hev
    _ = ⋃ n, r n (f (r n z)) := iSup_congr hterm
    _ = f z := hfz.symm

theorem embInfInf_projInfInf (f : DInfFn D₀ j₀) :
    embInfInf D₀ j₀ (projInfInf D₀ j₀ f) = f := by
```

```
  have h := congrArg (fun g => g f) (embInfInf_comp_projInfInf D₀ j₀)
  simpa [ScottMap.comp_apply, ScottMap.idMap_apply] using h

theorem embInfInf_le_iff (x y : DInf D₀ j₀) :
    embInfInf D₀ j₀ x ≤ embInfInf D₀ j₀ y ↔ x ≤ y := by
  constructor
  · intro h
    have := (projInfInf D₀ j₀).monotone h
    rwa [projInfInf.embInfInf, projInfInf.embInfInf] at this
  · intro h; exact (embInfInf D₀ j₀).monotone h

end LimitMaps

end DomainTheory.ContinuousLattice
```

Theorem 4.4 4.4 Theorem. *The inverse limit D_∞ of the recursively defined sequence $\langle D_n, j_n\rangle_{n=0}^{\infty}$ of function spaces is not only a continuous lattice, but it is also homeomorphic to its own function space $[D_\infty \to D_\infty]$.*

Proof: We can write down directly the pair of maps i_∞, j_∞ that provide the homeomorphism:

$$i_\infty(x) = \bigsqcup_{n=0}^{\infty} (i_{n\infty} \circ x_{n+1} \circ j_{\infty n}),$$

$$j_\infty(f) = \bigsqcup_{n=0}^{\infty} i_{(n+1)\infty}(j_{\infty n} \circ f \circ i_{n\infty}).$$

Note that these formulae are simply generalizations at the limit for the formulae we used to define i_n, j_n in the first place. Thus it is not surprising that they would provide a projection of $[D_\infty \to D_\infty]$ upon D_∞. Indeed we can compute out $j_\infty(i_\infty(x))$, noting that all the lubs are monotone and that a double monotone limit can always be replaced by a single one in view of the continuity of the operations involved, obtaining

$$j_\infty(i_\infty(x)) = \bigsqcup_{n=0}^{\infty} i_{(n+1)\infty}(j_{\infty n} \circ i_{n\infty} \circ x_{n+1} \circ j_{\infty n} \circ i_{n\infty}) \quad = \bigsqcup_{n=0}^{\infty} i_{(n+1)\infty}(x_{n+1}) = x.$$

In the converse order the calculation is only a bit more complicated. The idea is that since all the functions f are continuous and since the elements x are the limits of their approximations, then each f is actually completely determined by its

In the converse order the calculation is only a bit more complicated. The idea is that since all the functions f are continuous and since the elements x are the limits of their approximations, then each f is actually completely determined by its sequence of restrictions $j_{\infty n} \circ f \circ i_{n\infty} \in D_{n+1}$. This simple idea can be made more precise with the aid of a lemma about D_∞, which allows us in certain cases to recognize projections from limits.

theorem_4_4.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaceTower.projInfInf
public import DomainTheory.ContinuousLattice.FunctionSpaceTower.embInfInf_projInfInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import DomainTheory.ContinuousLattice.InverseLimits.proposition_4_1
public import DomainTheory.ContinuousLattice.FunctionSpaces.theorem_3_3
public import DomainTheory.ContinuousLattice.FunctionSpaces.theorem_3_3_more
public import DomainTheory.ContinuousLattice.FunctionSpaceTower.theorem_4_4_more

@[expose] public section

namespace DomainTheory.ContinuousLattice

attribute [instance] CLat.str

section LimitMaps

variable (D₀ : CLat.{u})
  (j₀ : IsContinuousLatticeProjection D₀.carrier (ScottMap D₀.carrier D₀.carrier))

section Thm44d

variable (D₀ : CLat.{u})
  (j₀ : IsContinuousLatticeProjection D₀.carrier (ScottMap D₀.carrier D₀.carrier))

/-- The inverse pair Scott writes down in the proof of Theorem 4.4:
`i_∞ = embInfInf` and `j_∞ = projInfInf` satisfy `j_∞ ∘ i_∞ = id` and `i_∞ ∘ j_∞ =
    id`. -/
theorem theorem_4_4_inverses :
    (projInfInf D₀ j₀).comp (embInfInf D₀ j₀) = ScottMap.idMap ∧
    (embInfInf D₀ j₀).comp (projInfInf D₀ j₀) = ScottMap.idMap :=
  ⟨projInfInf.comp_embInfInf D₀ j₀, embInfInf_comp_projInfInf D₀ j₀⟩

/-- Lattice isomorphism along the same pair `i_∞`, `j_∞`. Scott's abstract also says
    the spaces
are isomorphic; this is not the statement of Theorem 4.4 (that statement is the
    homeomorphism). -/
noncomputable def theorem_4_4_orderIso : OrderIso (DInf D₀ j₀) (DInfFn D₀ j₀) :=
  (Equiv.mk (embInfInf D₀ j₀) (projInfInf D₀ j₀)
      (projInfInf.embInfInf D₀ j₀) (embInfInf_projInfInf D₀ j₀)).toOrderIso
    (embInfInf D₀ j₀).monotone (projInfInf D₀ j₀).monotone

/-- **Scott 1972, Theorem 4.4** (`sources/ScottContinLatt1972.md`):
*The inverse limit $D_\infty$ of the recursively defined sequence
$\langle D_n, j_n \rangle_{n=0}^{\infty}$ of function spaces is not only a
continuous lattice, but it is also homeomorphic to its own function space
$[D_\infty \to D_\infty]$.*  Here ?inverse limit? carries Scott's subspace
topology from the Pi product, while the function space carries Definition 3.1's
pointwise/Pi-induced topology. -/
theorem theorem_4_4 (h₀ : ContinuousLattice D₀.carrier) :
    @ContinuousLattice (DInf D₀ j₀) inferInstance ∧
      Nonempty
        (@Homeomorph (DInf D₀ j₀) (DInfFn D₀ j₀)
          (inverseLimitTopology (towerType D₀) (towerProj D₀ j₀))
          (scottMapInducedPiTopology (DInf D₀ j₀) (DInf D₀ j₀))) := by
  have hTower : ∀ n, ContinuousLattice (towerType D₀ n) := by
    intro n
```

```
    induction n with
    | zero => exact h₀
    | succ n ih => exact theorem_3_3.isContinuousLattice ih ih
  refine ⟨proposition_4_1 (towerType D₀) (towerProj D₀ j₀) hTower, ?_⟩
  exact ⟨theorem_4_4.sourceTopology_homeomorph D₀ j₀ hTower⟩

end Thm44d
end LimitMaps

end DomainTheory.ContinuousLattice
```

Lemma 4.5 **4.5 Lemma.** Suppose for each n we have $u_{(n+1)} \in D_{n+1}$ and we let:

$$u = \bigsqcup_{n=0}^{\infty} i_{(n+1)\infty}(u_{(n+1)}).$$

Then if

$$j_{n+1}(u_{(n+2)}) = u_{(n+1)}$$

for each n, we can conclude that:

$$j_{\infty(n+1)}(u) = u_{(n+1)}.$$

Proof: If the sequence $u_{(n+1)}$ satisfies the recursion, then the limit defining u is monotonic. Therefore by continuity of projection it suffices to prove that

$$j_{\infty(n+1)}(i_{(m+1)\infty}(u_{(m+1)})) = u_{(n+1)}$$

for all $m \geqslant n$. This is obvious for $m = n$, and it can be readily proved by induction for larger m using the various recursion equations. (Properly speaking the induction is done on the quantity $(m - n)$ using both n and m as variables.) □

lemma_4_5.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.embInf
public import DomainTheory.ContinuousLattice.InverseLimits.embInfFun
public import DomainTheory.ContinuousLattice.InverseLimits.iComp
public import DomainTheory.ContinuousLattice.InverseLimits.projInf
public import DomainTheory.ContinuousLattice.InverseLimits.projLE
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u
```

```
section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]
variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- **Scott 1972, Lemma 4.5.** A criterion for recognizing projections out of the
    limit: if a
sequence `u : ∀ n, D_{n+1}` satisfies the (shifted) recursion `j_{n+1}(u_{n+2}) = u_{n
    +1}`, then the
monotone limit `u_∞ = ⋃ₙ i_{(n+1)∞}(uₙ)` has `j_{∞(n+1)}(u_∞) = uₙ`.

The proof follows Scott's argument at source lines 1316–1320.  First the defining
    family is
monotone.  Continuity of `j_{∞(n+1)}` therefore moves the projection through its
    directed
supremum.  Finally, for every `m ≥ n`, the identity
`j_{∞(n+1)}(i_{(m+1)∞}(u_m)) = u_n` is proved by induction on `m - n`, peeling one
    projection with
`projLE.succ` and then applying the recursion hypothesis `j_{m+1}(u_{m+1}) = u_m`. -/
theorem lemma_4_5 (u : ∀ n, D (n + 1))
    (hu : ∀ n, (P (n + 1)).retr (u (n + 1)) = u n) (n : ℕ) :
    (⋃ k, embInf D P (k + 1) (u k) : InverseLimit D P).1 (n + 1) = u n := by
  let a : ℕ → InverseLimit D P := fun k => embInf D P (k + 1) (u k)
  have ha_succ (k : ℕ) : a k ≤ a (k + 1) := by
    rw [show a k = embInf D P (k + 1) (u k) from rfl,
      show a (k + 1) = embInf D P (k + 2) (u (k + 1)) from rfl,
      ← embInf_succ D P (k + 1) (u k)]
    exact embInf_monotone D P (k + 2)
      (by calc (P (k + 1)).incl (u k)
            = (P (k + 1)).incl ((P (k + 1)).retr (u (k + 1))) := by rw [hu k]
          _ ≤ u (k + 1) := (P (k + 1)).incl_retr_le _)
  have ha : Monotone a := monotone_nat_of_le_succ ha_succ
  have hprojLE : ∀ d n : ℕ,
      projLE D P (show n + 1 ≤ n + d + 1 by omega) (u (n + d)) = u n := by
    intro d
    induction d with
    | zero =>
        intro n
        simpa only [Nat.add_zero] using
          (projLE.self D P (le_refl (n + 1)) (u n))
    | succ d ih =>
        intro n
        simp only [Nat.add_succ]
        rw [projLE.succ D P
          (show n + 1 ≤ n + d + 1 by omega)
          (show n + 1 ≤ (n + d + 1) + 1 by omega)]
        rw [hu (n + d)]
        exact ih n
  have hprojection : ∀ d n : ℕ,
      (embInf D P (n + d + 1) (u (n + d))).1 (n + 1) = u n := by
    intro d n
    cases d with
    | zero =>
        change iComp D P (n + 1) (u n) (n + 1) = u n
        exact iComp_self D P (n + 1) (u n)
```

```
    | succ d =>
        change iComp D P (n + (d + 1) + 1) (u (n + (d + 1))) (n + 1) = u n
        rw [iComp_of_ge D P (by omega)]
        exact hprojLE (d + 1) n
  have hdir : DirectedOn (· ≤ ·) (Set.range a) :=
    directedOn_range.2 ha.directed_le
  have hcont := (projInf D P (n + 1)).preservesDirectedSup_coe
    (Set.range a) (Set.range_nonempty a) hdir
  rw [show (⋃ k, embInf D P (k + 1) (u k) : InverseLimit D P)
      = sSup (Set.range a) from sSup_range.symm]
  change (projInf D P (n + 1)) (sSup (Set.range a)) = u n
  rw [hcont, ← Set.range_comp, sSup_range]
  apply le_antisymm
  · exact iSup_le fun m => by
      by_cases hmn : n ≤ m
      · obtain ⟨d, rfl⟩ := Nat.exists_eq_add_of_le hmn
        exact le_of_eq (hprojection d n)
      · exact le_trans ((ha (Nat.le_of_not_ge hmn)) (n + 1))
          (le_of_eq (hprojection 0 n))
  · exact le_trans (le_of_eq (hprojection 0 n).symm) (le_iSup (fun m => (a m).1 (n + 1)
    ) n)

end InverseLimit
end DomainTheory.ContinuousLattice
```

Proof of 4.4 concluded **Proof of 4.4 concluded:** The lemma applies at once to our calculation, for we find:

$$i_\infty(j_\infty(f)) = \bigsqcup_{n=0}^{\infty}(i_{n\infty}\circ j_{\infty n}\circ f\circ i_{n\infty}\circ j_{\infty n}) \quad = \bigsqcup_{n=0}^{\infty}(i_{n\infty}\circ j_{\infty n})\circ f\circ \bigsqcup_{n=0}^{\infty}(i_{n\infty}\circ j_{\infty n}).$$

Here we have just applied the continuity of f to be able to confine the lub on the right. But now by the remark following 4.2, we note the functional equation:

$$\mathrm{id} = \bigsqcup_{n=0}^{\infty}(i_{n\infty}\circ j_{\infty n}),$$

and the proof that i_∞ and j_∞ are inverse to one another is complete. □

theorem_4_4_more.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaceTower.projInfInf
public import DomainTheory.ContinuousLattice.FunctionSpaceTower.embInfInf_projInfInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.Basic
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.InverseLimits.inverseLimitTopology
public import DomainTheory.ContinuousLattice.InverseLimits.proposition_4_1
public import DomainTheory.ContinuousLattice.FunctionSpaces.theorem_3_3
public import DomainTheory.ContinuousLattice.FunctionSpaces.theorem_3_3_more
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottMapInducedPiTopology
```

```
@[expose] public section

namespace DomainTheory.ContinuousLattice

attribute [instance] CLat.str

section LimitMaps

variable (D₀ : CLat.{u})
  (j₀ : IsContinuousLatticeProjection D₀.carrier (ScottMap D₀.carrier D₀.carrier))

section Thm44d

variable (D₀ : CLat.{u})
  (j₀ : IsContinuousLatticeProjection D₀.carrier (ScottMap D₀.carrier D₀.carrier))

/-- The pair $i_\infty, j_\infty$ from the proof of Theorem 4.4, first packaged using
    the
Scott lattice topologies on both sides.  Scott writes
`i∞(x) = ⋃ₙ (i_{n∞} ∘ x_{n+1} ∘ j_{∞n})` and
`j∞(f) = ⋃ₙ i_{(n+1)∞}(j_{∞n} ∘ f ∘ i_{n∞})`, and says
(`sources/ScottContinLatt1972.md`): ?We can write down directly the pair of maps
`i∞, j∞` that provide the homeomorphism.?

This helper is deliberately distinct from the source-topology homeomorphism in the
    numbered
theorem below. -/
noncomputable def theorem_4_4.scottTopology_homeomorph :
    @Homeomorph (DInf D₀ j₀) (DInfFn D₀ j₀) scottTopologicalSpace scottTopologicalSpace
     :=
  @Homeomorph.mk (DInf D₀ j₀) (DInfFn D₀ j₀) scottTopologicalSpace
    scottTopologicalSpace
    (Equiv.mk (embInfInf D₀ j₀) (projInfInf D₀ j₀)
      (projInfInf.embInfInf D₀ j₀) (embInfInf_projInfInf D₀ j₀))
    (embInfInf D₀ j₀).continuous (projInfInf D₀ j₀).continuous

end Thm44d

end LimitMaps

attribute [instance] CLat.str

section LimitMaps

variable (D₀ : CLat.{u})
  (j₀ : IsContinuousLatticeProjection D₀.carrier (ScottMap D₀.carrier D₀.carrier))

section Thm44d

variable (D₀ : CLat.{u})
  (j₀ : IsContinuousLatticeProjection D₀.carrier (ScottMap D₀.carrier D₀.carrier))

/-- Scott's Theorem 4.4 homeomorphism with the literal topologies used in the source:
the inverse-limit subspace topology from the Pi product on `D∞`, and Definition 3.1's
pointwise (equivalently Pi-induced) topology on `[D∞ → D∞]`.
```

```
The topology changes are the mathematical identifications supplied by Proposition 4.1
    and
Theorem 3.3; the underlying maps remain Scott's displayed `i∞` and `j∞` formulas above
    . -/
noncomputable def theorem_4_4.sourceTopology_homeomorph
    (hTower : ∀ n, ContinuousLattice (towerType D₀ n)) :
    @Homeomorph (DInf D₀ j₀) (DInfFn D₀ j₀)
      (inverseLimitTopology (towerType D₀) (towerProj D₀ j₀))
      (scottMapInducedPiTopology (DInf D₀ j₀) (DInf D₀ j₀)) := by
  have hDInf : ContinuousLattice (DInf D₀ j₀) :=
    proposition_4_1 (towerType D₀) (towerProj D₀ j₀) hTower
  rw [← inverseLimit.scottTopology_eq_induced_pi
        (towerType D₀) (towerProj D₀ j₀) hTower,
      ← theorem_3_3.topology_induced_pi hDInf hDInf]
  exact theorem_4_4.scottTopology_homeomorph D₀ j₀

end Thm44d

end LimitMaps

end DomainTheory.ContinuousLattice
```

1.5.17 The functorial form

We can explain the idea of this proof in other terms using a suggestion made to me by F. W. Lawvere. Consider the category of

We can explain the idea of this proof in other terms using a suggestion made to me by F. W. Lawvere. Consider the category of continuous lattices and projections. In that category our D_∞ is, as we have remarked, both a *direct* and an *inverse* limit. Note too that with regard to projections $[D \to D']$ is a *functor*, for we can also define $[j \to j']$ when the maps are projections. In this language our particular inverse system is defined by the recursion:

$$D_{n+1} = [D_n \to D_n] \quad \text{and} \quad j_{n+1} = [j_n \to j_n],$$

where D_0 and j_0 are given in advance. Now the function space construction is continuous in its two arguments, turning an inverse limit on the right into an inverse limit and a direct limit on the *left* also into an *inverse* limit. A repeated limit can be made into a simple limit, so we can write:

$$D_\infty = \varprojlim \langle D_n, j_n \rangle_{n=0}^{\infty} \quad = \varinjlim \langle D_n, i_n \rangle_{n=0}^{\infty}$$

and

$$[D_\infty \to D_\infty] = \varprojlim \langle [D_n \to D_n], [j_n \to j_n] \rangle_{n=0}^{\infty} \quad = \varprojlim \langle D_{n+1}, j_{n+1} \rangle_{n=0}^{\infty} = D_\infty \quad \textit{(up to isomorphism).}$$

A full checking of the details involved would not make the argument appreciably simpler over the more "element-by-element" argument I have presented. In fact, the proofs are actually the same. But thinking of the result in terms of properties of functors does seem to isolate very well the essential idea and to show how simple it is.

One must only add here a note of caution: the proper choice of category must be done with care. Thus it seems to me that the use of projections rather than arbitrary continuous maps is necessary.

Inasmuch as I have not checked all details in this form, I hope what I say is correct.

1.5.18 Application

Since we have shown $[D_\infty \to D_\infty]$ to be homeomorphic to D_∞, we can begin to regard them as the same. In particular there ought to be some function space structure to transfer to D_∞. This can be done by defining functional application for any elements $x, y \in D_\infty$ by the equation:

$$x(y) = \bigsqcup_{n=0}^{\infty} i_{n\infty}(x_{n+1}(y_n)).$$

Similarly we can define λ-abstraction on continuous expressions:

$$\lambda x.[\ldots x \ldots] = j_\infty(\lambda\{x : D_\infty\}.[\ldots x \ldots]),$$

and in this way D_∞ becomes a model for the λ-calculus of Church and Curry. The model-theoretic and proof-theoretic aspects of this result will be explained in another paper (Scott (1972)).

1.5.19 Two projections

Suppose we were to start with the least, non-trivial lattice $0 = \top, \bot$ for D_0. Now $D_1 = [0 \to 0]$ has exactly three elements and there are just two ways of defining a projection $j_0 : D_1 \to D_0$. They are illustrated in the figure:

Left projection

```
⊤  ←−− j₀ −−  [⊤,⊤]
⊥  ←−− j₀ −−  [⊥,⊤]
⊥  ←−− j₀ −−  [⊥,⊥]
```

Right projection

```
⊤  ←−− j₀ −−  [⊤,⊤]
⊤  ←−− j₀ −−  [⊥,⊤]
⊥  ←−− j₀ −−  [⊥,⊥]
```

Hence our construction gives two limit spaces D_∞ and $D' * \infty$. Are they the same? No, they are not. It can be shown, for example, that the $\top$ of $D * \infty$ is *isolated* (that is, $\top \ll \top$), while the same is not true of $D' * \infty$. More interestingly, David Park has proved that the fixed-point operator fix mentioned in 3.14 has algebraic properties in $D * \infty$ quite different from those in $D' * \infty$. By algebraic here, we of course have reference to the functional algebra embodied in the application operation $x(y)$ defined on these limit spaces. Note, by the way, that in view of our isomorphism result we can regard fix (or any other similar continuous function for that matter) as an element of $D * \infty$. This makes the "algebra" of D_∞ quite a rich field for study.

1.5.20 Embedding every space

The reader will have surely remarked that by virtue of 1.5, every T_0-space X whatsoever can be embedded as a subspace in a D_∞. Besides this all the continuous functions on X (oh, into D_∞, say) can be extended to D_∞; whence they can be regarded as *elements* of D_∞. Thus we have been able to embed not only the

topology of X into D_∞ but also all of the continuous function theory over X. So far this is only a "logical" construction. For more interesting "mathematical" results we shall have to investigate whether any useful theorems about the usual function spaces $[X \to X]$ can be obtained with the aid of D_∞. This method can easily be employed for real- or complex-valued continuous functions, though it seems more oriented toward pointwise convergence than anything else. Still, there seems to be a chance it might be useful—especially if one wished to consider continuous *operators* on function spaces.

1.5.21 Other functors

The idea of forming the limit space can also be applied to *other* functors besides $[D \to D]$. Thus instead of solving the "equation"

$$D = [D \to D]$$

as we have done with the D_∞ construction, we could also solve:

$$V = T + [V \times V] + [V + V] + [V \to V],$$

for example. Here $T = \bot, 0, 1, \top$ is the four-element lattice with 0 and 1 as incomparable elements. By $[V \times V]$ and $[V \to V]$ we understand the usual Cartesian product and function space construction. The $+$ operator, on the other hand works only in the category of lattices with $\top$ as an isolated element. It is defined so as to make:

$$\begin{array}{ccc} D + D' & \longleftarrow & D' \\ \uparrow & & \uparrow \\ D & \longleftarrow & 0 \end{array}$$

a push-out diagram, where the maps from 0 are meant to match $\bot$ with $\bot$ and $\top$ with $\top$. The point of requiring $\top$ to be isolated is that both D and D' become projections of $D + D'$. This construction, though not quite a disjoint union, has many properties in common with that operation on spaces. In particular, if we consider the category with projections as maps, the construction

$$\mathbb{F}(D) = T + [D \times D] + [D + D] + [D \to D]$$

is a *functor*. Furthermore, we can project $\mathbb{F}(T)$ onto T in an obvious way, thereby setting things up for an inverse limit construction:

$$V = \varprojlim \langle \mathbb{F}^n(T), j_n \rangle_{n=0}^{\infty}.$$

The resulting continuous lattice satisfies the desired equation up to isomorphism.

1.5.22 Subspaces of a limit

The space V constructed in the way just indicated is very rich in subspaces. To see this, consider the space $\mathcal{J}_V$ of proper projections j where $j(\top) = \top$. As in 3.12 this is a complete lattice. Now that $[V \times V]$ and $[V + V]$ and $[V \to V]$ are regarded as *subspaces* of this "universe" V itself, we can easily define *continuous operations*

$$(j \times k)\ ,\ (j + k)\ ,\ \text{and}\ (j \to k)$$

on the projections obtaining again elements of $\mathcal{J}_V$. The projections so obtained correspond to the indicated constructions of subspaces, of course. (Indeed, if we had the time and space, we could show that $\mathcal{J}_V$ becomes a very interesting category). There will be a particular

projection t corresponding to T, and reason for doing all this is to show that the existence of subspaces of V can now be established by solving equations in $\mathcal{J}_V$. For example, by the fixed-point construction we could find a $j \in \mathcal{J}_V$ such that

$$j = t + (t \times j) + ((j \times j) \to j).$$

The range of j would then be a subspace $W \subseteq V$ such that W solves the equation:

$$W = T + [T \times W] + [[W \times W] \to W].$$

And these are only a few examples: simultaneous equations are possible, and many other operators are waiting for discovery and application.

1.6 Correction

Correction (Added March, 1972). Robin Milner has pointed out to me that there is an error in the remark in the paragraph immediately preceding Proposition 2.5. I was mistaken in saying that if D is a T_0-space which becomes a complete lattice under its induced partial ordering, then every set open in the given topology is also open in the induced topology. There are many counterexamples to this statement. Let D be any complete lattice. There are two extreme T_0-topologies which will induce the given partial ordering. The smallest such topology has as a sub-base for its open sets those sets of the form:

$$\{x \in D : x \not\sqsubseteq y\}.$$

These sets are easily proved to be open in any T_0-topology which induces the partial ordering. At the other extreme consider sets of the form:

$$\{x \in D : y \sqsubseteq x.\}$$

Such sets will give a base for a T_0-topology that is the maximal topology inducing the given partial ordering. Clearly they need not be open in the induced lattice topology; in particular, they may well fail to satisfy conditions (ii) on open sets. To make the remark in question correct, we must thus suppose that the given T_0-topology is *contained within* the induced lattice topology. The equation given in the paragraph indicated will then be a sufficient condition for the two topologies to be identical.

The remark was employed in the proof of three different propositions: 2.9, 2.10, and 3.3. In the case of 2.9 one must verify that the product topology is contained within the lattice topology. This need only be done for the basis for the product topology, and for such basic open sets the result needed is obvious. In the case of proposition 2.10 the question concerns a relationship between the topologies of a space and a subspace; the spaces in question are also lattices. Note in passing that a lub in the subspace is generally *larger* in the partial ordering than the corresponding lub relative to the whole space. This puts the inequalities in the wrong direction, and so it is not immediate that a relativized open set for the subspace is open in the lattice topology of the subspace. However, in this case we can appeal to the continuous retraction. Recall that the relativized open sets of the kind that we used in the proof of 2.10 are of the form:

$$U = \{z \in D : x < z\}\ .$$

Suppose then that S is a directed set, and that using the lub in the sense of D we have

$$\bigsqcup S \in U \ .$$

Referring back to the proof of 2.10 we know that ($S' = S \subseteq D$, but $\bigsqcup S'$ is the join computed in the ambient lattice D')

$$j(\bigsqcup S') = \bigsqcup S \ ,$$

which means that

$$x < j(\bigsqcup S') \ .$$

From this it follows that

$$x < j(z), \text{ some } z \in S \ .$$

Now $j(z) = z$, and we have what we need. This argument suffices only for a special type of open sets; but these open sets form a base for the topology, and so the argument is quite general.

Turning now to the proof of theorem 2.3 we note that the topology on the function space is simply the *relativized* product topology. There is no difficulty with lubs in this case, because, as we showed in the proof of that theorem, all lubs are calculated pointwise. Thus, it is easy to verify now that the sets open in the product topology are also open in the lattice topology.

2 Datatypes as Lattices (1976)

Dependencies. Each box is one section. The arrowhead is on the section that imports. The number is how many direct imports. An arrow already implied by a longer path is left out.

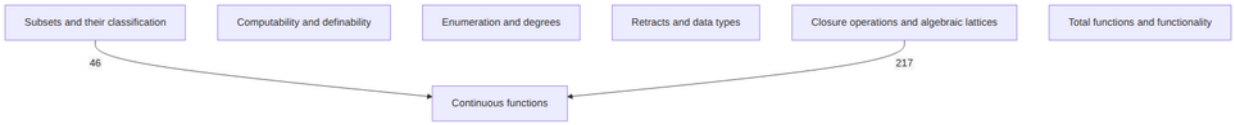


Figure 15: Datatypes as Lattices (1976).

2.1 Introduction

To the Memory of Christopher Strachey, 1916–1975

The meaning of many kinds of expressions in programming languages can be taken as elements of certain spaces of "partial" objects. In this report these spaces are modeled in one universal domain $\mathbf{P}\omega$, the set of all subsets of the integers. This domain renders the connection of this semantic theory with the ordinary theory of number theoretic (especially general recursive) functions clear and straightforward.

Investigations begun in 1969 with Christopher Strachey led to the idea that the denotations of many kinds of expressions in programming languages could be taken as elements of certain kinds of spaces of "partial" objects. As these spaces could be treated as function spaces, their structure at first seemed excessively complicated—even impossible. But then the author discovered that there were many more spaces than we had first imagined—even wanted. They could be presented as lattices (or as some prefer, semilattices), and the main technique was to employ topological ideas, in particular the notion of a continuous function. This approach and its applications have been presented in a number of publications, but that part of the foundation concerned with *computability* (in the sense of recursion theory) was never before adequately exposed. The purpose of the present paper is to provide such a foundation and to simplify the whole presentation by a de-emphasis of abstract ideas.

The main innovation in this report is to model everything within one "universal" domain $\mathbf{P}\omega = \{x \mid x \subseteq \omega\}$, the domain of all subsets of the set ω of nonnegative integers. The advantages are many: by the most elementary considerations $\mathbf{P}\omega$ is recognized to be a lattice and a topological space. In fact, $\mathbf{P}\omega$ is a *continuous lattice*, even an *algebraic lattice*, but in the beginning we do not even need to define such an "advanced" concept; we can save these ideas for an *analysis* of what has been done here in a more direct way. Next by taking the set of integers as the basis of the construction, the connection with the ordinary theory of number-theoretic (especially, general recursive) functions can be made clear and straightforward.

The model $\mathbf{P}\omega$ can be intuitively viewed as the domain of *multiple-valued* integers; what is new in the presentation is that functions are not only multiple-valued but also "multiple-argumented". This remark is given a precise sense in § 2 below, but the upshot of the idea is that multiple-valued integers are regarded as objects in themselves—possibly infinite—and as something more than just the collection of single integers contained in them. This combination of the finite with the infinite into a single domain, together with the idea that a continuous function can be reduced to its *graph* (in the end, a set of integers), makes it possible to view an $x \in \mathbf{P}\omega$ at one time as a value, at another as an argument, then as an integer, then as a function, and still later as a functional (or combinator). The "paradox" of self-application (as in $x(x)$) is solved by allowing the *same* x to be used in two *different* ways. This is done in ordinary recursion theory via Gödel numbers (as in $\{e\}(e)$), but the advantage of the present theory is that not only is the function concept the

extensional one, but it includes *arbitrary* continuous functions and not just the computable ones.

Section 1 introduces the elementary ideas on the topology of $\mathbf{P}\omega$ and the continuous functions including the fixed-point theorem. Section 2 has to do with computability and definability. The language LAMBDA is introduced as an extension of the pure λ-calculus by four arithmetical combinators; in fact, it is indicated in § 3 how the whole system could be based on *one* combinator. What is shown is that computability in $\mathbf{P}\omega$ according to the natural definition (which assumes that we already know what a recursively enumerable set of integers is) is equivalent to LAMBDA-definability. The main tool is, not surprisingly, the first recursion theorem formulated with the aid of the so-called *paradoxical combinator* $\mathbf{Y}$. The plan is hardly original, but the point is to work out what it all means in the model.

Along the way we have to show how to give every λ-term a denotation in $\mathbf{P}\omega$; the resulting principles of λ-calculus that are thereby verified are summarized in Table 1. Of these the first three, (α), (β), and (ξ), are indeed valid in the model; however, rule (η), which is a stronger version of extensionality, *fails* in the $\mathbf{P}\omega$ model. This should not be regarded as a disadvantage since the import of (η) is to suppose every object is a function. A quick construction of these special models is indicated at the end of § 5. Since $\mathbf{P}\omega$ is partially ordered by $\subseteq$, there are also laws involving this relation. Law (ξ^*) is an improvement of (ξ); while (μ) is a form of monotonicity for application.

TABLE 1

Some laws of λ-calculus

(α)	$\lambda x.\tau = \lambda y.\tau[y/x]$
(β)	$(\lambda x.\tau)(y) = \tau[y/x]$
(ξ)	$\lambda x.\tau = \lambda x.\sigma \quad \text{if} \quad \forall x.\, \tau = \sigma$
(η)	$y = \lambda x.y(x)$
(ξ^*)	$\lambda x.\tau \subseteq \lambda x.\sigma \quad \text{if} \quad \forall x.\, \tau \subseteq \sigma$
(μ)	$x \subseteq y$ and $u \subseteq v$ imply $u(x) \subseteq v(y)$

Section 3 has to do with enumeration and degrees. Gödel numbers for LAMBDA are defined in a very easy way which takes advantage of the notation of combinators. This leads to the second recursion theorem, and results on incompleteness and undecidability follow along standard lines. *Relative recursiveness* is also very easy to define in the system, and we make the tie-in with *enumeration degrees* which correspond to finitely generated combinatory subalgebras of $\mathbf{P}\omega$. Finally a theorem of Myhill and Shepherdson is interpreted as a most satisfactory completeness property for definability in the system.

Sections 4 and 5 show how a calculus of *retracts* leads to quite simple definitions of a host of useful domains (as lattices). Section 6 investigates the classification of other subsets (*nonlattices*) of $\mathbf{P}\omega$; while § 7 contrasts partial (multiple-valued) functions with total functions, and interprets various theories of functionality. Connections with category theory are mentioned.

What is demonstrated in this work is how the language LAMBDA, together with its interpretation in $\mathbf{P}\omega$, is an extremely convenient vehicle for *definitions* of computable functions on complex structures (all taken as subdomains of $\mathbf{P}\omega$). It is a "high-level" programming language for recursion theory. It is *applied* combinatory logic, which in usefulness goes far beyond anything envisioned in the standard literature. What has been shown is how many interesting predicates can be *expressed* as equations between continuous functions. What is needed next is a development of the *proof theory* of the system along the lines of the work of Milner, which incorporates the author's extension of McCarthy's rule of recursion induction to this high-level language. Then we will have a flexible and practical "mathematical" theory of computation.

2.1.1 Historical note

The paper comes out of work with Christopher Strachey begun in 1969. He treated the λ-calculus as ordinary algebra. Scott's first answer was a typed theory of function spaces, later developed as LCF by Robin Milner and his collaborators. Those function spaces, which he had met in the recursion theory of Nerode and Platek, led to the inverse-limit construction of function spaces of infinite type: models of the type-free λ-calculus, written up in Scott (1971) and Scott (1973).

Plotkin (1972) had proposed a set-theoretic model. This paper is the form of that idea in which the connection with ordinary recursion theory is immediate, because the elements are sets of integers. Scott presented it at Oberwolfach at Easter 1973, and in a shorter version at the Third Scandinavian Logic Symposium (Scott (1975)). Lectures at Kiel in July 1974 covered most of the paper; the text, without the acknowledgments or the references, appeared in Springer Lecture Notes in Mathematics, volume 499. Once the definitions had settled, the model was Plotkin's, and it was already present in the enumeration operators of Myhill and Shepherdson (1955) and Friedberg and Rogers (1959).

At the end of §2 of *Continuous lattices* (Scott (1972)) he had written that representing a continuous-function space as a computable retract "does not seem to be of too much help in proving theorems." That representation is the construction of this paper. Attributions for the separate sections are recorded on those sections.

2.2 Continuous functions

The constructions below are the enumeration of finite subsets, the pairing of natural numbers, the positive topology, the graph of a continuous function, the least fixed point, and the embedding of a continuous function.

Dependency diagram. Each box is one construction. The arrowhead is on the note that imports. The number is how many direct imports. An arrow already implied by a longer path is left out.

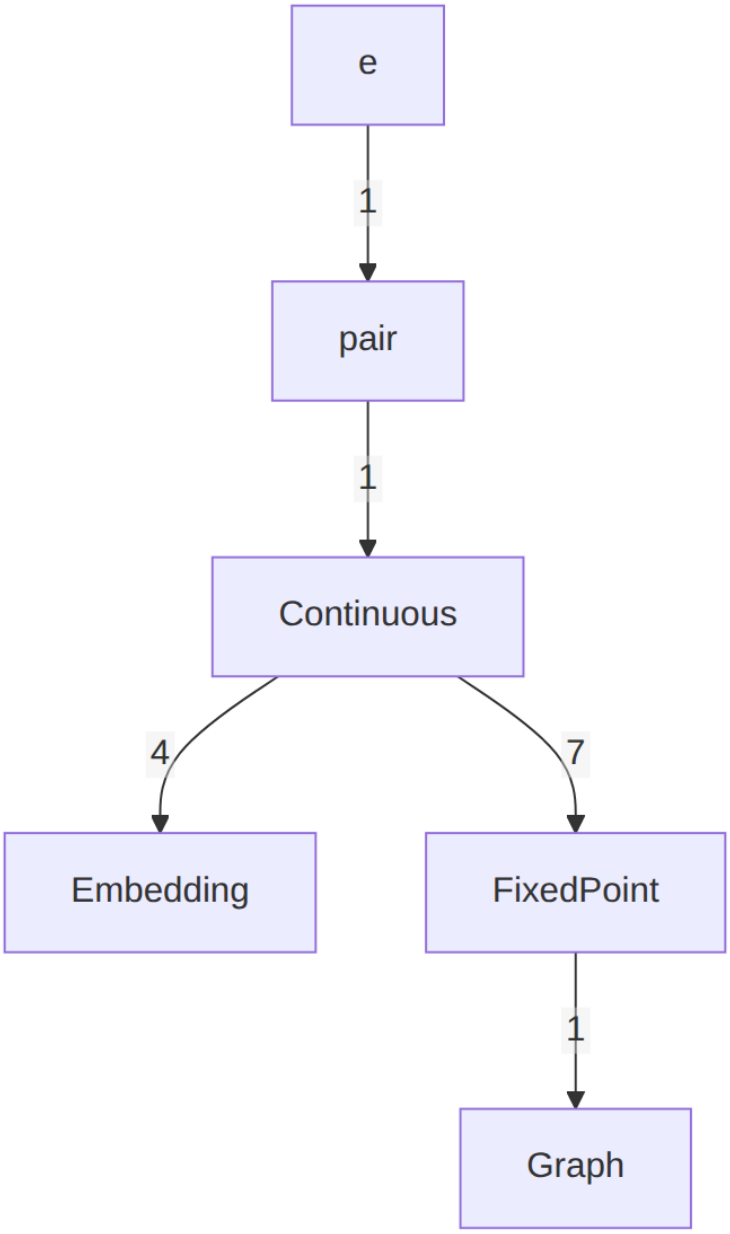


Figure 16: Continuous functions.

2.2.1 Introduction

The domain $\mathbf{P}\omega$ of all subsets of the set ω of nonnegative integers is a complete lattice under the partial ordering $\subseteq$ of set inclusion, as is well known. We use the usual symbols $\cup$, $\cap$, $\bigcup$, $\bigcap$ for the finite and infinite lattice operations of union and intersection. $\mathbf{P}\omega$ is of course also a Boolean algebra; and for complements we write

$$\sim x = \{n \mid n \notin x\}$$

where it is understood that such variables as i, j, k, l, m, n range over *integers in* ω, while u, v, w, x, y, z range over *subsets of* ω.

The domain $\mathbf{P}\omega$ can also be made into a topological space—in many ways. A common method is to match each $x \subseteq \omega$ with the corresponding characteristic function in $\{0,1\}^\omega$, and to take the induced product topology. In this way $\mathbf{P}\omega$ is a totally disconnected compact Hausdorff space homeomorphic to the Cantor "middle-third" set. This is *not* the topology we want; it is a *positive-and-negative* topology which makes the function $\sim x$ continuous. We want a weaker topology: the topology of positive "information", which has the advantage that all continuous functions possess fixed points. (The equation $x = \sim x$ is impossible.) The topology that we *do* want is exactly that appropriate to considering $\mathbf{P}\omega$ to be a continuous lattice. But all this terminology of abstract mathematics is quite unnecessary, since the required definitions can be given in very elementary terms.

To make the topology "visible", we introduce a standard enumeration $\{e_n \mid n \in \omega\}$ of all *finite* subsets of ω. Specifically we set

$$e_n = \{k_0, k_1, \cdots, k_{m-1}\},$$

provided that $k_0 < k_1 < \cdots < k_{m-1}$ and $n = \sum_{i<m} 2^{k_i}$. Thus n is the code number for e_n, and the elements of e_n are the exponents in the binary expansion of the integer n. This is a one-to-one enumeration of finite subsets, where $k \in e_n$ always implies $k < n$, the function $\max(e_n)$ is (primitive) recursive in n, and the relations $k \in e_n$, $e_n \subseteq e_m$, $e_n = e_m \cup e_k$ are all (primitive) recursive in k, m, n.

Topologically speaking the finite sets e_n are *dense* in the space $\mathbf{P}\omega$, for each $x \in \mathbf{P}\omega$ is the "limit" of its finite subsets in the sense that

$$x = \bigcup \{e_n \mid e_n \subseteq x\}.$$

To make this precise we need a rigorous definition of *open subset* of $\mathbf{P}\omega$.

Motivation Give $\{\bot, \top\}$ the weak T_0-topology with open sets $\varnothing$, $\{\top\}$, and $\{\bot, \top\}$. That is the Sierpinski space, and the product $\{\bot, \top\}^\omega$ in the product topology is $\mathbf{P}\omega$. The finite sets e_n are the basic open sets of the product. Theorem 1.1 is that observation written out in elementary terms.

On a subspace $X \subseteq \mathbf{P}\omega$ a finite set e_n need not lie in X, so continuity of $f : X \to \mathbf{P}\omega$ cannot be written by applying f to e_n. Theorem 1.5 is the substitute. Such an f is continuous exactly when it is the restriction of its extension $\bar{f}$, and for $x \in X$ that reads

$$m \in f(x) \quad \text{iff} \quad \exists e_n \subseteq x \; \forall x' \in X \; [e_n \subseteq x' \text{ implies } m \in f(x')].$$

The extension $\bar{f} : \mathbf{P}\omega \to \mathbf{P}\omega$ is continuous for an arbitrary f. It extends f precisely when f is continuous.

The fixed-point formula uses the same fact in the form Scott recorded for an increasing sequence: if $x_n \subseteq x_{n+1}$ and f is continuous, then f preserves the union of the x_n. Every fixed point lies above every iterate of $\varnothing$, so the union is the least one.

Historical note The positive topology came to Scott's attention through Nerode (1959). Continuous functionals had been studied by Kleene, Platek, and others, and Platek (1964) is where monotonicity is stressed. Graphs and application are used in the same way by Plotkin (1972) and by Rogers (1967, p. 147). The fixed-point theorem is Tarski's (1955). The extension theorem is Scott's, in the spirit of point-set topology, and it is the characterization of continuous lattices in Scott (1972). The embedding theorem is the standard one; Čech (1966) is a reference.

2.2.2 e

The powerset of the natural numbers, and the enumeration of its finite subsets by the binary code `e`. The declarations named `e_` are the facts about that code. The others are the domain, the membership tests, and the coding of finite lists and finite sets that those facts use.

`e.lean`

```
module

public import Mathlib.Data.Set.Finite.Basic
public import Mathlib.Algebra.GroupWithZero.Nat
public import Mathlib.Order.SetNotation

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, §1.** `Pω = {x | x ⊆ ω}`. -/
abbrev Pomega := Set ℕ

/-- **Scott 1976, after (2.2).** `⊥ = ∅`. -/
def botElem : Pomega := ∅

/-- **Scott 1976, after (2.2).** `⊤ = ω`. -/
def topElem : Pomega := Set.univ

@[simp] theorem mem_botElem (n : ℕ) : n ∈ botElem ↔ False := by simp [botElem]

@[simp] theorem mem_topElem (n : ℕ) : n ∈ topElem := Set.mem_univ n

/-- **Scott 1976, §1.** Finite set coded by the binary expansion of `n`. -/
def e (n : ℕ) : Pomega := {k | n.testBit k}

@[simp] theorem mem_e {n k : ℕ} : k ∈ e n ↔ n.testBit k := Iff.rfl

theorem le_two_pow (k : ℕ) : k ≤ 2 ^ k := by
  induction k with
  | zero => decide
  | succ k ih =>
    calc
      k + 1 ≤ 2 ^ k + 1 := Nat.succ_le_succ ih
      _ ≤ 2 ^ k + 2 ^ k := Nat.add_le_add_left Nat.one_le_two_pow _
      _ = 2 ^ k * 2 := (Nat.mul_two (2 ^ k)).symm
      _ = 2 ^ (k + 1) := (Nat.pow_succ 2 k).symm

theorem two_pow_le_of_testBit {n k : ℕ} (h : n.testBit k) : 2 ^ k ≤ n := by
```

```
  have hmod : (n >>> k) % 2 = 1 := by simpa [Nat.testBit] using h
  have hpos : 0 < n >>> k := Nat.pos_of_ne_zero fun hz => by simp [hz] at hmod
  have hdiv : 1 ≤ n / 2 ^ k := by
    rwa [Nat.shiftRight_eq_div_pow] at hpos
  simpa using (Nat.le_div_iff_mul_le (Nat.two_pow_pos k)).mp hdiv

theorem mem_e_le {n k : ℕ} (h : k ∈ e n) : k ≤ n :=
  (le_two_pow k).trans (two_pow_le_of_testBit h)

instance (n k : ℕ) : Decidable (k ∈ e n) :=
  inferInstanceAs (Decidable (n.testBit k))

instance (n m : ℕ) : Decidable (e n ⊆ e m) :=
  decidable_of_iff (∀ k ≤ n, k ∈ e n → k ∈ e m) ⟨
    fun h k hk => h k (mem_e_le hk) hk,
    fun h _k _ hk => h hk⟩

/-- Bit-mask code of a finite list of naturals. -/
def codeList : List ℕ → ℕ
  | [] => 0
  | k :: ks => 2 ^ k ||| codeList ks

noncomputable def codeFinset (s : Finset ℕ) : ℕ := codeList s.val.toList

/-- Finite pieces of `x` used in directed unions. -/
def eBelow (x : Pomega) (n : ℕ) : Pomega :=
  {k | e n ⊆ x ∧ k ∈ e n}

theorem e_zero : e 0 = ∅ := by
  ext k
  simp [e]

theorem e_pow2 (k : ℕ) : e (2 ^ k) = {k} := by
  ext i
  simp [e, Nat.testBit_two_pow]

theorem e_finite (n : ℕ) : (e n).Finite :=
  (Finset.range (n + 1)).finite_toSet.subset fun _k hk =>
    Finset.mem_range.mpr (Nat.lt_succ_of_le (mem_e_le hk))

theorem e_or (n m : ℕ) : e (n ||| m) = e n ∪ e m := by
  ext k
  simp [e, Nat.testBit_or]

theorem e_or_of_subset {n m : ℕ} {x : Pomega} (hn : e n ⊆ x) (hm : e m ⊆ x) :
    e (n ||| m) ⊆ x := by
  rw [e_or]
  exact Set.union_subset hn hm

theorem lt_two_pow' (k : ℕ) : k < 2 ^ k := by
  induction k with
  | zero => decide
  | succ k ih =>
    calc
      k + 1 ≤ 2 ^ k := Nat.succ_le_of_lt ih
      _ < 2 ^ k + 2 ^ k := Nat.lt_add_of_pos_right (Nat.two_pow_pos k)
```

```
      _ = 2 ^ (k + 1) := by rw [← Nat.two_mul, Nat.mul_comm, Nat.pow_succ]

/-- **Scott 1976, §1.** `k ∈ e_n` implies `k < n`. -/
theorem mem_e_lt {n k : ℕ} (h : k ∈ e n) : k < n := by
  have hle := mem_e_le h
  rcases eq_or_lt_of_le hle with rfl | hlt
  · exact (lt_irrefl k ((lt_two_pow' k).trans_le (two_pow_le_of_testBit h))).elim
  · exact hlt

theorem singleton_eq_e (k : ℕ) : ({k} : Pomega) = e (2 ^ k) := (e_pow2 k).symm

theorem exists_e_mem {x : Pomega} {k : ℕ} (hk : k ∈ x) :
    ∃ n, e n ⊆ x ∧ k ∈ e n :=
  ⟨2 ^ k, by simp [e_pow2, Set.singleton_subset_iff, hk], by simp [e_pow2]⟩

theorem union_e_eq (x : Pomega) : (⋃ n, eBelow x n) = x := by
  ext k
  constructor
  · intro hk
    obtain ⟨n, hn, hke⟩ := Set.mem_iUnion.mp hk
    exact hn hke
  · intro hk
    refine Set.mem_iUnion.mpr ⟨2 ^ k, ?_⟩
    simp [eBelow, e_pow2, Set.singleton_subset_iff, hk]

theorem e_codeList (ks : List ℕ) : e (codeList ks) = {k | k ∈ ks} := by
  induction ks with
  | nil => simp [codeList, e_zero]
  | cons k ks ih =>
    ext i
    simp [codeList, e_or, e_pow2, ih]

theorem e_codeFinset (s : Finset ℕ) : e (codeFinset s) = (s : Set ℕ) := by
  ext k
  simp [codeFinset, e_codeList]

theorem exists_code (s : Finset ℕ) : ∃ n, e n = (s : Set ℕ) :=
  ⟨codeFinset s, e_codeFinset s⟩

theorem exists_code_of_finite {s : Pomega} (hs : s.Finite) :
    ∃ n, e n = s := by
  obtain ⟨t, ht⟩ := hs.exists_finset_coe
  exact ⟨codeFinset t, ht ▸ e_codeFinset t⟩

end DomainTheory.DataTypesAsLattices
```

2.2.3 pair

The Cantor pairing of two natural numbers into one, its inverse, and the triangular-number facts that show the pairing is injective. This imports the enumeration.

pair.lean

```
module
```

```
public import Mathlib.Data.Nat.Bitwise
public import Mathlib.Data.Set.Finite.Basic
public import Mathlib.Data.Set.Lattice.Bounded
public import Mathlib.Data.Set.Lattice.Disjoint
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, §1.** Diagonal pairing `(n, m) = ?(n+m)(n+m+1) + m`. -/
def pair (n m : ℕ) : ℕ := (n + m) * (n + m + 1) / 2 + m

def triangle (w : ℕ) : ℕ := w * (w + 1) / 2

theorem pair_eq_triangle (n m : ℕ) : pair n m = triangle (n + m) + m := rfl

theorem triangle_succ (w : ℕ) : triangle (w + 1) = triangle w + (w + 1) := by
  have hex : (w + 1) * (w + 2) = w * (w + 1) + 2 * (w + 1) := by
    calc
      (w + 1) * (w + 2) = (w + 1) * w + (w + 1) * 2 := Nat.mul_add _ _ _
      _ = w * (w + 1) + 2 * (w + 1) := by
        rw [Nat.mul_comm (w + 1) w, Nat.mul_comm (w + 1) 2]
  simp only [triangle]
  rw [show (w + 1) * ((w + 1) + 1) = (w + 1) * (w + 2) from rfl, hex]
  exact Nat.add_mul_div_left _ _ (by decide : 0 < 2)

theorem exists_triangle_bucket (k : ℕ) :
    ∃ w, triangle w ≤ k ∧ k < triangle (w + 1) := by
  let P := fun w : ℕ => k < triangle (w + 1)
  have hP : ∃ w, P w := ⟨k, by
    change k < triangle (k + 1)
    rw [triangle_succ]
    exact lt_of_lt_of_le (Nat.lt_succ_self k) (Nat.le_add_left (k + 1) _)⟩
  let w := Nat.find hP
  refine ⟨w, ?_, Nat.find_spec hP⟩
  cases hw : w with
  | zero => exact Nat.zero_le k
  | succ w' =>
    have hmin : ¬ P w' :=
      Nat.find_min hP (Nat.lt_of_succ_le (by
        have : w = Nat.find hP := rfl
        rw [← this, hw]))
    exact Nat.le_of_not_gt hmin

theorem exists_pair (k : ℕ) : ∃ n m, pair n m = k := by
  obtain ⟨w, hle, hlt⟩ := exists_triangle_bucket k
  refine ⟨w - (k - triangle w), k - triangle w, ?_⟩
  have hlt' : k < triangle w + w + 1 := by
    have : k < triangle w + (w + 1) := by rwa [triangle_succ] at hlt
    exact Nat.add_assoc (triangle w) w 1 ▸ this
  have hk : k ≤ triangle w + w := Nat.lt_succ_iff.mp hlt'
  have hm : k - triangle w ≤ w :=
    Nat.le_of_add_le_add_left ((Nat.add_sub_of_le hle).symm ▸ hk)
  have : w - (k - triangle w) + (k - triangle w) = w := Nat.sub_add_cancel hm
  rw [pair_eq_triangle, this, Nat.add_sub_of_le hle]
```

```
/-- Inverse of Cantor pairing. -/
noncomputable def unpair (k : ℕ) : ℕ × ℕ :=
  ((exists_pair k).choose, (exists_pair k).choose_spec.choose)

theorem pair_le_left (n m : ℕ) : n ≤ pair n m := by
  have htri : n ≤ triangle (n + m) := by
    cases n with
    | zero => exact Nat.zero_le _
    | succ n =>
      have hmul : n.succ * 2 ≤ (n.succ + m) * (n.succ + m + 1) :=
        Nat.mul_le_mul
          (Nat.le_add_right n.succ m)
          (Nat.succ_le_succ
            (Nat.le_trans (Nat.succ_le_succ (Nat.zero_le n)) (Nat.le_add_right n.succ
   m)))
      exact (Nat.le_div_iff_mul_le (by decide : 0 < 2)).mpr hmul
  exact htri.trans (Nat.le_add_right _ _)

theorem pair_lt_left (n m : ℕ) : n < pair n m + 1 :=
  Nat.lt_succ_of_le (pair_le_left n m)

theorem pair_le_right (n m : ℕ) : m ≤ pair n m :=
  Nat.le_add_left _ _

theorem pair_lt_right (n m : ℕ) : m < pair n m + 1 :=
  Nat.lt_succ_of_le (pair_le_right n m)

theorem pair_unpair (k : ℕ) : pair (unpair k).1 (unpair k).2 = k :=
  (exists_pair k).choose_spec.choose_spec

theorem triangle_le_of_le {a b : ℕ} (h : a ≤ b) : triangle a ≤ triangle b := by
  induction h with
  | refl => exact le_rfl
  | step h ih =>
    rw [triangle_succ]
    exact le_trans ih (Nat.le_add_right _ _)

theorem triangle_le_pair (n m : ℕ) :
    triangle (n + m) ≤ pair n m ∧ pair n m < triangle (n + m + 1) := by
  constructor
  · exact Nat.le_add_right _ _
  · rw [triangle_succ]
    exact Nat.add_lt_add_left (Nat.lt_succ_of_le (Nat.le_add_left m n)) _

theorem triangle_lt_of_lt {a b : ℕ} (h : a < b) : triangle a < triangle b := by
  have : a + 1 ≤ b := Nat.succ_le_of_lt h
  calc
    triangle a < triangle a + (a + 1) := Nat.lt_add_of_pos_right (Nat.succ_pos _)
    _ = triangle (a + 1) := (triangle_succ a).symm
    _ ≤ triangle b := triangle_le_of_le this

theorem pair_inj {n₁ m₁ n₂ m₂ : ℕ} (h : pair n₁ m₁ = pair n₂ m₂) :
    n₁ = n₂ ∧ m₁ = m₂ := by
  have h1 := triangle_le_pair n₁ m₁
  have h2 := triangle_le_pair n₂ m₂
```

```
  have hw : n₁ + m₁ = n₂ + m₂ := by
    rcases lt_trichotomy (n₁ + m₁) (n₂ + m₂) with hlt | heq | hgt
    · have hle : triangle (n₁ + m₁ + 1) ≤ triangle (n₂ + m₂) :=
        triangle_le_of_le (Nat.succ_le_of_lt hlt)
      have : pair n₁ m₁ < pair n₁ m₁ :=
        calc
          pair n₁ m₁ < triangle (n₁ + m₁ + 1) := h1.2
          _ ≤ triangle (n₂ + m₂) := hle
          _ ≤ pair n₂ m₂ := h2.1
          _ = pair n₁ m₁ := h.symm
      exact (lt_irrefl _ this).elim
    · exact heq
    · have hle : triangle (n₂ + m₂ + 1) ≤ triangle (n₁ + m₁) :=
        triangle_le_of_le (Nat.succ_le_of_lt hgt)
      have : pair n₂ m₂ < pair n₂ m₂ :=
        calc
          pair n₂ m₂ < triangle (n₂ + m₂ + 1) := h2.2
          _ ≤ triangle (n₁ + m₁) := hle
          _ ≤ pair n₁ m₁ := h1.1
          _ = pair n₂ m₂ := h
      exact (lt_irrefl _ this).elim
  have hm : m₁ = m₂ := by
    have : triangle (n₁ + m₁) + m₁ = triangle (n₂ + m₂) + m₂ := h
    rw [hw] at this
    exact Nat.add_left_cancel this
  have hn : n₁ = n₂ := Nat.add_right_cancel (hw.trans (by rw [hm]))
  exact ⟨hn, hm⟩

theorem unpair_pair (n m : ℕ) : unpair (pair n m) = (n, m) := by
  have h := pair_unpair (pair n m)
  exact Prod.ext (pair_inj h).1 (pair_inj h).2

end DomainTheory.DataTypesAsLattices
```

2.2.4 Continuous

Dependencies. Inside Continuous, each box is one Lean file. A box named as another folder in this chapter is that whole folder. A box named as another chapter is that whole chapter. The arrowhead points at the file or folder that imports, and the number is how many direct imports.

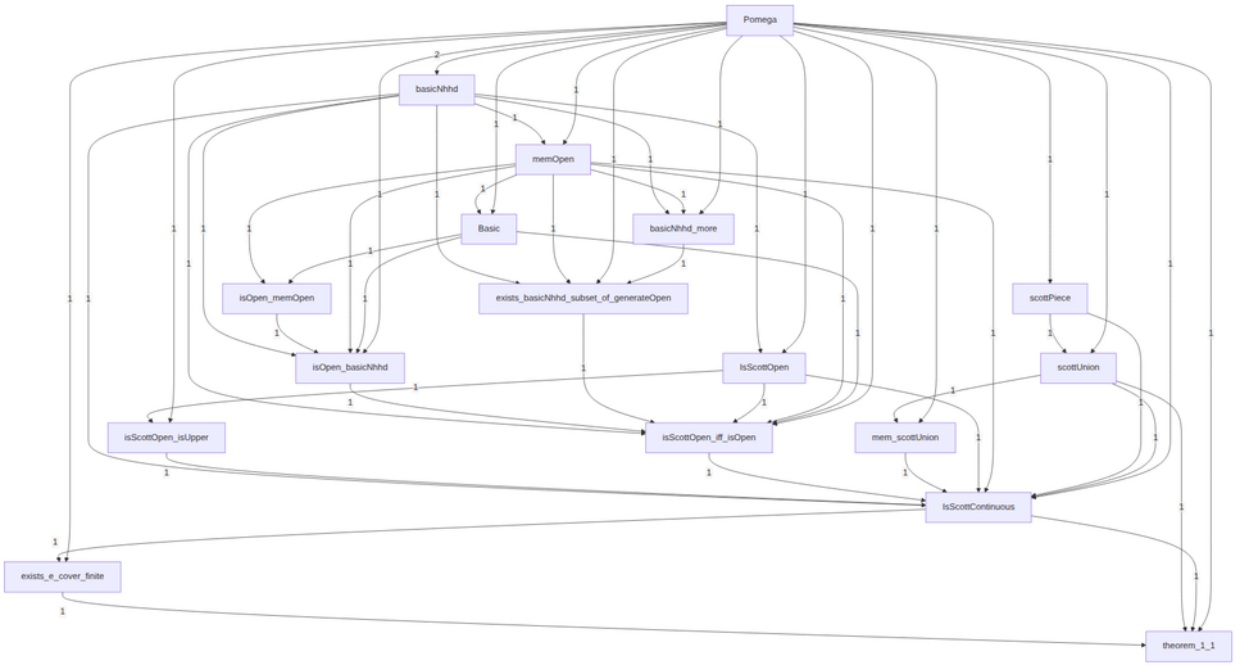


Figure 17: Continuous.

basicNhhd.lean

```
module

public import Mathlib.Topology.Bases
public import Mathlib.Topology.Order
public import Mathlib.Data.Nat.Bitwise
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Pomega.pair

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace

/-- **Scott 1976, §1, Definition.** A basic neighbourhood is
`{x ∈ Pω | e n ⊆ x}` for some finite code `n`. -/
def basicNhhd (n : ℕ) : Set Pomega := {x | e n ⊆ x}

namespace basicNhhd

theorem inter (n m : ℕ) :
    basicNhhd n ∩ basicNhhd m = basicNhhd (n ||| m) := by
  ext x
  simp [basicNhhd, e_or, Set.union_subset_iff]

theorem zero : basicNhhd 0 = Set.univ := by
  ext x
```

```
  simp [basicNhhd, e_zero]

theorem mono {n m : ℕ} (h : e n ⊆ e m) : basicNhhd m ⊆ basicNhhd n := by
  intro x hx
  exact h.trans hx

end basicNhhd

end DomainTheory.DataTypesAsLattices
```

Definition (basis) **DEFINITION.** A *basis* for the neighborhoods of $\mathbf{P}\omega$ consists of those sets of the form:

$$\{x \in \mathbf{P}\omega \mid e_n \subseteq x\},$$

for a given e_n. An arbitrary *open subset* is then a union of basic neighborhoods.

It is easy to prove that an open subset $U \subseteq \mathbf{P}\omega$ is just a set of "finite character"; that is, a set such that for all $x \in \mathbf{P}\omega$ we have $x \in U$ if and only if some finite subset of x also belongs to U. An alternate approach would define directly what we mean by a continuous function using the idea that such functions must preserve limits.

Definition (continuous) **DEFINITION.** A function $f : \mathbf{P}\omega \to \mathbf{P}\omega$ is *continuous* iff for all $x \in \mathbf{P}\omega$ we have:

$$f(x) = \bigcup\{f(e_n) \mid e_n \subseteq x\}.$$

Again it is an easy exercise to prove that a function is continuous in the sense of this definition iff it is continuous in the usual topological sense (namely: inverse images of open sets are open). For giving proofs it is even more convenient to have the usual ε-δ formulation of continuity.

Theorem 1.1 **THEOREM 1.1** (The characterization theorem). A function $f : \mathbf{P}\omega \to \mathbf{P}\omega$ is *continuous* iff for all $x \in \mathbf{P}\omega$ and all e_m we have:

$$e_m \subseteq f(x) \quad \text{iff} \quad \exists e_n \subseteq x.\ e_m \subseteq f(e_n).$$

Note that open sets and continuous functions have a *monotonicity property*:
whenever $x \subseteq y$ *and* $x \in U$, *then* $y \in U$; *and*
whenever $x \subseteq y$, *then* $f(x) \subseteq f(y)$.

This gives a precise expression to the "positive" character of our topology. However, note too that openness and continuity mean rather more than just monotonicity. In particular, a continuous function is completely determined by the pairs of integers n, m such that $m \in f(e_n)$, as can be seen from the definition. (Hence, there are only a continuum number of continuous functions, but more than a continuum number of monotonic functions.) This brings us to the definition of the *graph* of a continuous function.

To formulate the definition, we introduce a standard enumeration (n, m) of all pairs of integers. Specifically we set

$$(n, m) = \tfrac{1}{2}(n + m)(n + m + 1) + m.$$

This is the enumeration along the "little diagonals" going from left to right, and it produces the ordering:

$$(0,0),(1,0),(0,1),(2,0),(1,1),(0,2),(3,0),(2,1),\ldots.$$

Note that $n \leq (n,m)$ and $m \leq (n,m)$ with equality possible only in the cases of $(0,0)$ and $(1,0)$. This is a one-to-one enumeration, and the inverse functions are (primitive) recursive—but we do not require at the present any notation for them.

scottPiece.lean

```
module

public import Mathlib.Topology.Bases
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import Mathlib.Tactic.Continuity.Init

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace

/-- Directed union of `f` on finite subsets of `x`. -/
def scottPiece (f : Pomega → Pomega) (x : Pomega) (n : ℕ) : Pomega :=
  {k | e n ⊆ x ∧ k ∈ f (e n)}

end DomainTheory.DataTypesAsLattices
```

scottUnion.lean

```
module

public import Mathlib.Topology.Bases
public import DomainTheory.DataTypesAsLattices.Continuous.scottPiece
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace

def scottUnion (f : Pomega → Pomega) (x : Pomega) : Pomega :=
  ⋃ n, scottPiece f x n

end DomainTheory.DataTypesAsLattices
```

2.2.5 Graph

Dependencies. Inside Graph, each box is one subfolder, or one Lean file sitting directly in the folder. A box named as another folder in this chapter is that whole folder. A box named as another chapter is that whole chapter. The arrowhead points at the file or folder that imports, and the number is how many direct imports.

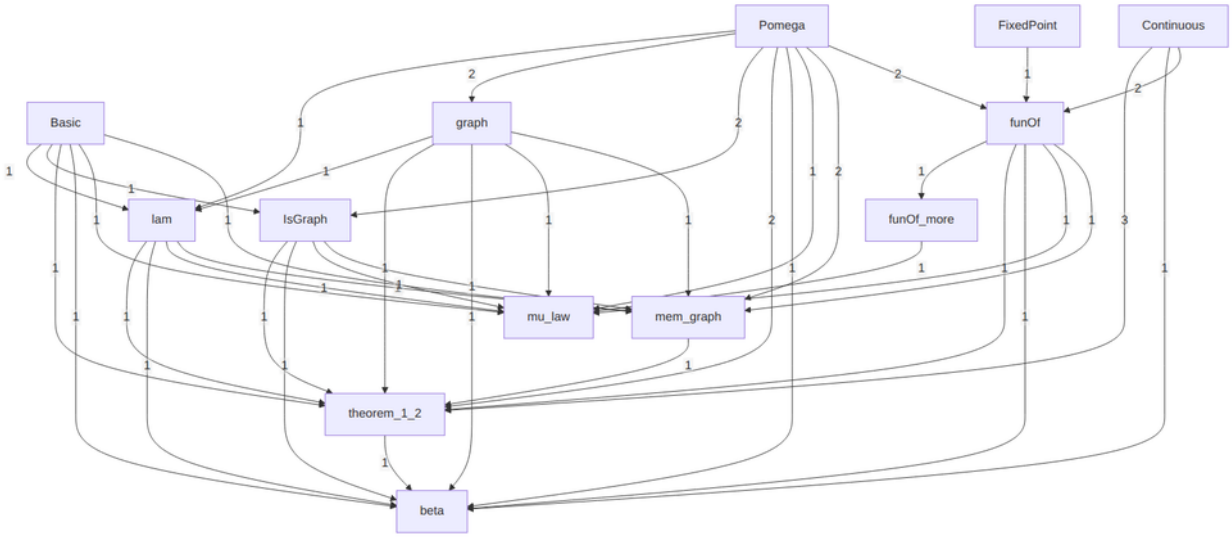


Figure 18: Graph.

IsGraph.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Graph.Basic.notation3

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, Theorem 1.2 (iii).** Equality in (ii) iff `u` is downward
closed in the finite-set argument: `(k, m) ∈ u` and `e k ⊆ e n` imply `(n, m) ∈ u`. -/
class IsGraph (u : Pomega) : Prop where
  out : ∀ {{k m n}}, pair k m ∈ u → e k ⊆ e n → pair n m ∈ u

end DomainTheory.DataTypesAsLattices
```

Definition (graph) **DEFINITION.** The graph of a continuous function $f : \mathbf{P}\omega \to \mathbf{P}\omega$ is defined by the equation:

$$\mathbf{graph}(f) = \{(n, m) \mid m \in f(e_n)\};$$

while the function determined by any set $u \subseteq \omega$ is defined by the equation:

$$\mathbf{fun}(u)(x) = \{m \mid \exists e_n \subseteq x.\ (n, m) \in u\}.$$

Theorem 1.2 **THEOREM 1.2** (The graph theorem). *Every continuous function is uniquely determined by its graph in the sense that:*

(i) $\mathbf{fun}(\mathbf{graph}(f)) = f$.

Conversely, every set of integers determines a continuous function and we have:

(ii) $u \subseteq \mathbf{graph}(\mathbf{fun}(u))$,

where equality holds just in case u satisfies:

(iii) *whenever $(k, m) \in u$ and $e_k \subseteq e_n$, then $(n, m) \in u$.*

Besides functions of one variable we need to consider also functions of several variables. The official definition for one variable given above can be extended simply by saying $f(x, y, \dots)$ is continuous iff it is continuous in each of $x, y, \dots$. Those familiar with the product topology can prove that for our special positive topology on $\mathbf{P}\omega$ this is equivalent to being continuous on the product space (continuous in the variables jointly). Those interested only in elementary proofs can calculate out directly from the definition (with the aid of 1.1) that continuity behaves under combinations by substitution [as in: $f(g(x, y), h(y, x, y))$].

2.2.6 FixedPoint

Dependencies. Inside FixedPoint, each box is one Lean file. A box named as another folder in this chapter is that whole folder. A box named as another chapter is that whole chapter. The arrowhead points at the file or folder that imports, and the number is how many direct imports.

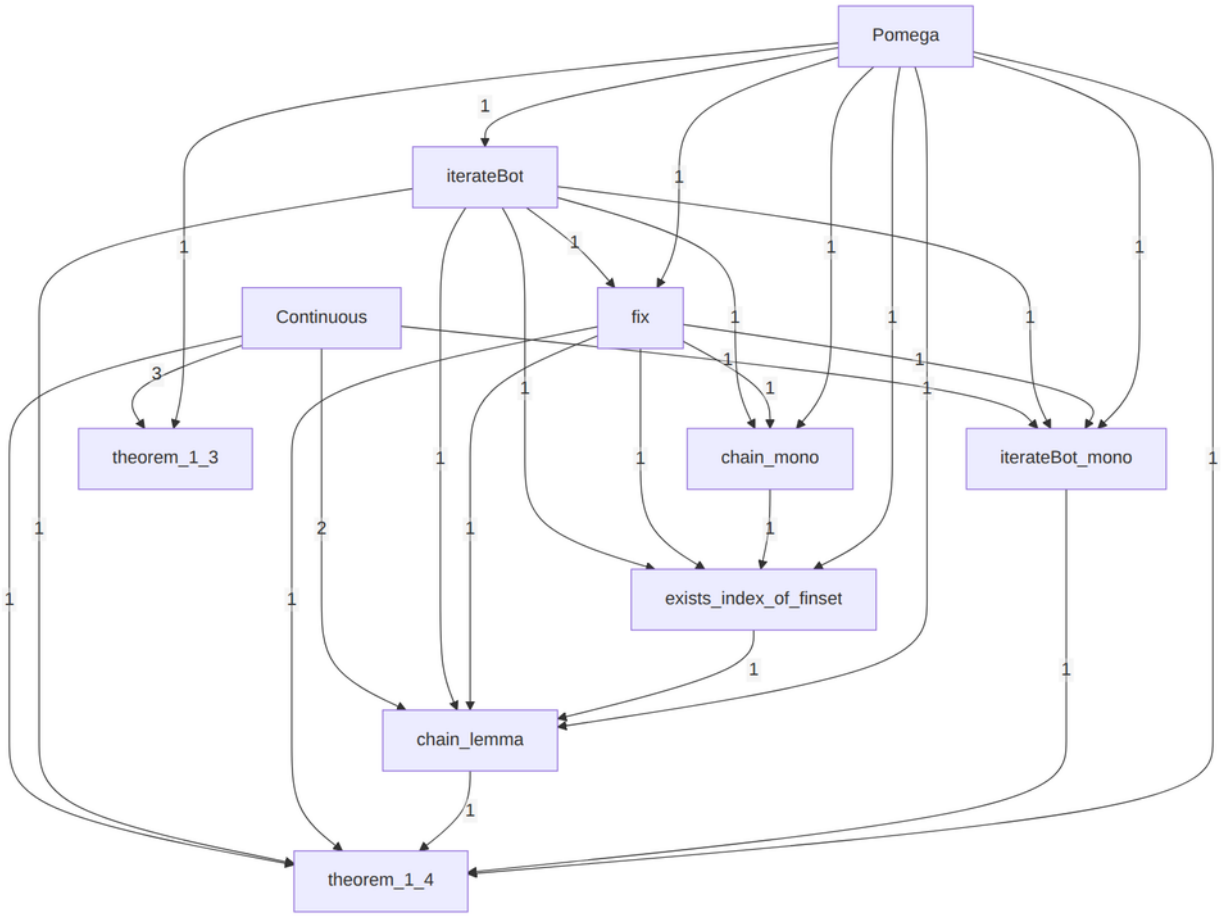


Figure 19: FixedPoint.

iterateBot.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- Iterate `f` starting from `⊥`. -/
def iterateBot (f : Pomega → Pomega) : ℕ → Pomega
  | 0 => botElem
  | n + 1 => f (iterateBot f n)

end DomainTheory.DataTypesAsLattices
```

fix.lean

```
module
```

```
public import DomainTheory.DataTypesAsLattices.FixedPoint.iterateBot
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, Theorem 1.4.** Least fixed point `fix(f) = ⋃_n f?(∅)`. -/
def fix (f : Pomega → Pomega) : Pomega :=
  ⋃ n, iterateBot f n

end DomainTheory.DataTypesAsLattices
```

chain_mono.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.FixedPoint.iterateBot
public import DomainTheory.DataTypesAsLattices.FixedPoint.fix

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem chain_mono (xs : ℕ → Pomega) (hmono : ∀ n, xs n ⊆ xs (n + 1)) :
    ∀ a b, a ≤ b → xs a ⊆ xs b := by
  intro a b hab
  obtain ⟨d, hd⟩ := Nat.exists_eq_add_of_le hab
  subst hd
  clear hab
  induction d with
  | zero => simp
  | succ d ih => exact subset_trans ih (hmono _)

end DomainTheory.DataTypesAsLattices
```

Theorem 1.3 **THEOREM 1.3** (The substitution theorem). *Continuous functions of several variables on* $\mathbf{P}\omega$ *are closed under substitution.*

theorem_1_3.lean

```
module

public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Continuous.theorem_1_1
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Continuous.scottUnion

@[expose] public section

namespace DomainTheory.DataTypesAsLattices
```

```
/-- **Scott 1976, Theorem 1.3, composition.** -/
theorem theorem_1_3.comp {f g : Pomega → Pomega}
    (hf : IsScottContinuous f) (hg : IsScottContinuous g) :
    IsScottContinuous (f ∘ g) := by
  constructor
  intro x
  ext k
  constructor
  · intro hk
    have hk' : k ∈ f (g x) := hk
    have := (theorem_1_1 f).mp hf (g x) (2 ^ k)
    have hfin : e (2 ^ k) ⊆ f (g x) := by
      simpa [e_pow2, Set.singleton_subset_iff] using hk'
    obtain ⟨m, hm, hkm⟩ := this.mp hfin
    have := (theorem_1_1 g).mp hg x m
    obtain ⟨n, hn, hmn⟩ := this.mp hm
    refine mem_scottUnion.mpr ⟨n, hn, ?_⟩
    have : e (2 ^ k) ⊆ f (g (e n)) :=
      subset_trans hkm (IsScottContinuous.monotone hf hmn)
    simpa [e_pow2, Set.singleton_subset_iff] using this
  · intro hk
    obtain ⟨n, hn, hkn⟩ := mem_scottUnion.mp hk
    exact IsScottContinuous.monotone hf (IsScottContinuous.monotone hg hn) hkn

/-- **Scott 1976, Theorem 1.3 (The substitution theorem), binary case.**
If `f` is continuous in each argument separately, then `x ↦ f(x,x)` is
continuous. -/
theorem theorem_1_3.diag {f : Pomega → Pomega → Pomega}
    (hfx : ∀ y, IsScottContinuous (fun x => f x y))
    (hfy : ∀ x, IsScottContinuous (fun y => f x y))
    {x : Pomega} :
    f x x = ⋃ n, {k | e n ⊆ x ∧ k ∈ f (e n) (e n)} := by
  apply subset_antisymm
  · intro k hk
    have : k ∈ scottUnion (fun z => f z x) x := by
      rw [← (hfx x).out x]; exact hk
    obtain ⟨n, hn, hkn⟩ := mem_scottUnion.mp this
    have : k ∈ scottUnion (fun y => f (e n) y) x := by
      rw [← (hfy (e n)).out x]; exact hkn
    obtain ⟨m, hm, hkm⟩ := mem_scottUnion.mp this
    refine Set.mem_iUnion.mpr ⟨n ||| m, ?_⟩
    refine ⟨e_or_of_subset hn hm, ?_⟩
    have hle : e n ⊆ e (n ||| m) := by rw [e_or]; exact Set.subset_union_left
    have hre : e m ⊆ e (n ||| m) := by rw [e_or]; exact Set.subset_union_right
    exact IsScottContinuous.monotone (hfy (e (n ||| m))) hre
      (IsScottContinuous.monotone (hfx (e m)) hle hkm)
  · intro k hk
    obtain ⟨n, hn, hkn⟩ := Set.mem_iUnion.mp hk
    exact IsScottContinuous.monotone (hfy x) hn
      (IsScottContinuous.monotone (hfx (e n)) hn hkn)

/-- **Scott 1976, Theorem 1.3, binary substitution.** If `f` is continuous
in each argument and `g, h` are continuous, then `x ↦ f(g x, h x)` is
continuous. -/
theorem theorem_1_3.tuple {f : Pomega → Pomega → Pomega}
    (hfx : ∀ y, IsScottContinuous (fun x => f x y))
```

```
    (hfy : ∀ x, IsScottContinuous (fun y => f x y))
    {g h : Pomega → Pomega} (hg : IsScottContinuous g) (hh : IsScottContinuous h) :
    IsScottContinuous (fun x => f (g x) (h x)) := by
  constructor
  intro x
  have H := theorem_1_3.diag (x := x)
    (f := fun u v => f (g u) (h v))
    (fun v => theorem_1_3.comp (hfx (h v)) hg)
    (fun u => theorem_1_3.comp (hfy (g u)) hh)
  ext k
  constructor
  · intro hk
    have : k ∈ ⋃ n, {k | e n ⊆ x ∧ k ∈ f (g (e n)) (h (e n))} := by rwa [← H]
    obtain ⟨n, hn, hkn⟩ := Set.mem_iUnion.mp this
    exact mem_scottUnion.mpr ⟨n, hn, hkn⟩
  · intro hk
    obtain ⟨n, hn, hkn⟩ := mem_scottUnion.mp hk
    have : k ∈ ⋃ n, {k | e n ⊆ x ∧ k ∈ f (g (e n)) (h (e n))} :=
      Set.mem_iUnion.mpr ⟨n, hn, hkn⟩
    rwa [← H] at this

/-- **Scott 1976, Theorem 1.3, ternary substitution.** -/
theorem theorem_1_3.nary {f : Pomega → Pomega → Pomega → Pomega}
    (hf0 : ∀ y z, IsScottContinuous (fun x => f x y z))
    (hf1 : ∀ x z, IsScottContinuous (fun y => f x y z))
    (hf2 : ∀ x y, IsScottContinuous (fun z => f x y z))
    {g h i : Pomega → Pomega}
    (hg : IsScottContinuous g) (hh : IsScottContinuous h) (hi : IsScottContinuous i) :
    IsScottContinuous (fun x => f (g x) (h x) (i x)) := by
  constructor
  intro x
  have H := theorem_1_3.diag (x := x)
    (f := fun u v => f (g u) (h u) (i v))
    (fun v => theorem_1_3.tuple (fun y => hf0 y (i v)) (fun x => hf1 x (i v)) hg hh)
    (fun u => theorem_1_3.comp (hf2 (g u) (h u)) hi)
  ext k
  constructor
  · intro hk
    have : k ∈ ⋃ n, {k | e n ⊆ x ∧ k ∈ f (g (e n)) (h (e n)) (i (e n))} := by
      rwa [← H]
    obtain ⟨n, hn, hkn⟩ := Set.mem_iUnion.mp this
    exact mem_scottUnion.mpr ⟨n, hn, hkn⟩
  · intro hk
    obtain ⟨n, hn, hkn⟩ := mem_scottUnion.mp hk
    have : k ∈ ⋃ n, {k | e n ⊆ x ∧ k ∈ f (g (e n)) (h (e n)) (i (e n))} :=
      Set.mem_iUnion.mpr ⟨n, hn, hkn⟩
    rwa [← H] at this

/-- **Scott 1976, Theorem 1.3 (The substitution theorem).** Continuous
functions of several variables on `Pω` are closed under substitution. -/
theorem theorem_1_3 {f g : Pomega → Pomega}
    (hf : IsScottContinuous f) (hg : IsScottContinuous g) :
    IsScottContinuous (fun x => f (g x)) :=
  theorem_1_3.comp hf hg

end DomainTheory.DataTypesAsLattices
```

Theorem 1.4 The other general fact about continuous functions that we shall use constantly concerns fixed points, whose existence can be proved using a well-known method.

THEOREM 1.4 (The fixed-point theorem). *Every continuous function $f : \mathbf{P}\omega \to \mathbf{P}\omega$ has a least fixed point given by the formula:*

$$\mathbf{fix}(f) = \bigcup\{f^n(\varnothing) \mid n \in \omega\},$$

where $\varnothing$ is the empty set and f^n is the n-fold composition of f with itself.

Actually **fix** is a *functional* with continuity properties of its own. We shall not give the required definitions here because they can be more easily derived from the construction of the model given in the next section.

For those familiar with the abstract theory of topological spaces we give in conclusion two general facts about continuous functions with values in $\mathbf{P}\omega$ which indicate the scope and generality of our method.

exists_index_of_finset.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.FixedPoint.iterateBot
public import DomainTheory.DataTypesAsLattices.FixedPoint.fix
public import DomainTheory.DataTypesAsLattices.FixedPoint.chain_mono

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem exists_index_of_finset (xs : ℕ → Pomega)
    (hmono : ∀ n, xs n ⊆ xs (n + 1))
    (s : Finset ℕ) (h : ∀ k ∈ s, ∃ n, k ∈ xs n) :
    ∃ N, (s : Set ℕ) ⊆ xs N := by
  induction s using Finset.induction with
  | empty => exact ⟨0, by simp⟩
  | insert i t hit ih =>
    obtain ⟨N, hN⟩ := ih fun j hj => h j (Finset.mem_insert_of_mem hj)
    obtain ⟨Ni, hNi⟩ := h i (Finset.mem_insert_self _ _)
    refine ⟨max N Ni, ?_⟩
    intro j hj
    rw [Finset.coe_insert, Set.mem_insert_iff] at hj
    rcases hj with rfl | hj
    · exact chain_mono xs hmono Ni (max N Ni) (le_max_right _ _) hNi
    · exact chain_mono xs hmono N (max N Ni) (le_max_left _ _) (hN hj)

end DomainTheory.DataTypesAsLattices
```

2.2.7 Embedding

Dependencies. Inside Embedding, each box is one Lean file. A box named as another folder in this chapter is that whole folder. A box named as another chapter is that whole chapter. The arrowhead points at the file or folder that imports, and the number is how many direct imports.

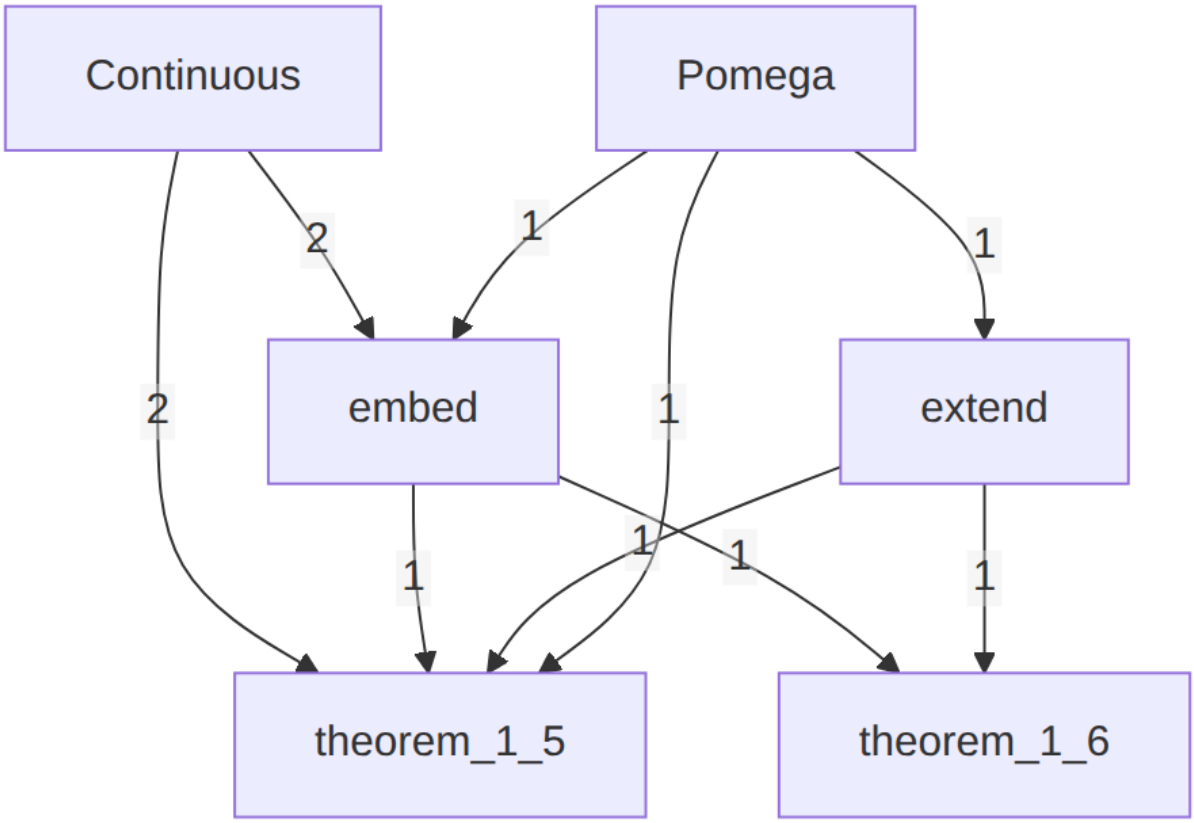


Figure 20: Embedding.

extend.lean

```
module

public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import Mathlib.Data.Set.Lattice.Indexed
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace Topology Function

/-- **Scott 1976, Theorem 1.5.** Maximal continuous extension of
`f : X → Pω` along a subspace inclusion `X ⊆ Y`:
`f(y) = ⋃ { ⋂ {f(x) | x ∈ X ∩ U} | y ∈ U, U open in Y }`. -/
def extend {Y : Type*} [TopologicalSpace Y] {X : Set Y}
    (f : X → Pomega) (y : Y) : Pomega :=
  ⋃₀ { t | ∃ U : Set Y, IsOpen U ∧ y ∈ U ∧ t = ⋂₀ (f '' {x : X | ↑x ∈ U}) }
```

```
namespace extend

theorem mem {Y : Type*} [TopologicalSpace Y] {X : Set Y}
    (f : X → Pomega) {y : Y} {k : ℕ} :
    k ∈ extend (Y := Y) (X := X) f y ↔
      ∃ U : Set Y, IsOpen U ∧ y ∈ U ∧ ∀ x : X, ↑x ∈ U → k ∈ f x := by
  constructor
  · intro hk
    rcases Set.mem_sUnion.mp hk with ⟨t, ⟨U, hU, hy, rfl⟩, hkt⟩
    refine ⟨U, hU, hy, ?_⟩
    intro x hx
    exact Set.mem_sInter.mp hkt (f x) ⟨x, hx, rfl⟩
  · intro ⟨U, hU, hy, hall⟩
    refine Set.mem_sUnion.mpr ⟨⋂₀ (f '' {x : X | ↑x ∈ U}), ⟨U, hU, hy, rfl⟩, ?_⟩
    intro t ht
    rcases ht with ⟨x, hx, rfl⟩
    exact hall x hx

theorem isOpen_mem {Y : Type*} [TopologicalSpace Y] {X : Set Y}
    (f : X → Pomega) (k : ℕ) :
    IsOpen {y : Y | k ∈ extend (Y := Y) (X := X) f y} := by
  have : {y : Y | k ∈ extend (Y := Y) (X := X) f y} =
      ⋃ U ∈ {U : Set Y | IsOpen U ∧ ∀ x : X, ↑x ∈ U → k ∈ f x}, U := by
    ext y
    constructor
    · intro hy
      obtain ⟨U, hU, hyU, hall⟩ := (mem f).mp hy
      exact Set.mem_iUnion₂.mpr ⟨U, ⟨hU, hall⟩, hyU⟩
    · intro hy
      obtain ⟨U, hU, hyU⟩ := Set.mem_iUnion₂.mp hy
      exact (mem f).mpr ⟨U, hU.1, hyU, hU.2⟩
  rw [this]
  exact isOpen_biUnion fun _ hU => hU.1

end extend

end DomainTheory.DataTypesAsLattices
```

memOpen.lean

```
module

public import Mathlib.Topology.Bases
public import DomainTheory.DataTypesAsLattices.Continuous.basicNhhd
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace

/-- Subbasic open `{x | k ∈ x}` of the positive topology. -/
def memOpen (k : ℕ) : Set Pomega := {x | k ∈ x}
```

```
namespace memOpen

theorem eq_basicNhhd (k : ℕ) : memOpen k = basicNhhd (2 ^ k) := by
  ext x
  simp [memOpen, basicNhhd, e_pow2, Set.singleton_subset_iff]

end memOpen

end DomainTheory.DataTypesAsLattices
```

Basic.lean

```
module

public import Mathlib.Topology.Order
public import DomainTheory.DataTypesAsLattices.Continuous.memOpen
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace

/-- **Scott 1976, §1.** The positive (?information?) topology on `Pω`,
generated by the sets `{x | k ∈ x}`. -/
instance : TopologicalSpace Pomega :=
  .generateFrom (Set.range memOpen)
end DomainTheory.DataTypesAsLattices
```

embed.lean

```
module

public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.memOpen
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Continuous.Basic
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace Topology Function

/-- **Scott 1976, Theorem 1.6.** The embedding `?(x) = {n | x ∈ U n}`. -/
def embed {X : Type*} (U : ℕ → Set X) (x : X) : Pomega :=
  {n | x ∈ U n}

namespace embed

theorem continuous {X : Type*} [TopologicalSpace X]
```

```
    (U : ℕ → Set X) (hbasis : IsTopologicalBasis (Set.range U)) :
    Continuous (embed U) :=
  continuous_generateFrom_iff.2 fun t ⟨n, hn⟩ => by
    subst hn
    change IsOpen (U n)
    exact hbasis.isOpen ⟨n, rfl⟩

theorem inducing {X : Type*} [TopologicalSpace X]
    (U : ℕ → Set X) (hbasis : IsTopologicalBasis (Set.range U)) :
    IsInducing (embed U) := by
  refine ⟨?_⟩
  rw [hbasis.eq_generateFrom]
  change generateFrom (Set.range U) =
    (generateFrom (Set.range memOpen)).induced (embed U)
  rw [induced_generateFrom_eq]
  congr 1
  ext V
  constructor
  · rintro ⟨n, rfl⟩
    exact ⟨memOpen n, ⟨n, rfl⟩, rfl⟩
  · rintro ⟨_, ⟨n, rfl⟩, rfl⟩
    exact ⟨n, rfl⟩

theorem injective {X : Type*} [TopologicalSpace X] [T0Space X]
    (U : ℕ → Set X) (hbasis : IsTopologicalBasis (Set.range U)) :
    Injective (embed U) := by
  intro x y hxy
  have hmem : ∀ n, x ∈ U n ↔ y ∈ U n := fun n =>
    show n ∈ embed U x ↔ n ∈ embed U y by rw [hxy]
  refine Inseparable.eq ?_
  refine inseparable_iff_forall_isOpen.mpr ?_
  intro V hV
  constructor
  · intro hx
    obtain ⟨t, ⟨n, rfl⟩, hxt, htV⟩ :=
      hbasis.mem_nhds_iff.mp (hV.mem_nhds hx)
    exact htV ((hmem n).mp hxt)
  · intro hy
    obtain ⟨t, ⟨n, rfl⟩, hyt, htV⟩ :=
      hbasis.mem_nhds_iff.mp (hV.mem_nhds hy)
    exact htV ((hmem n).mpr hyt)

end embed

end DomainTheory.DataTypesAsLattices
```

basicNhhd_more.lean

```
module

public import Mathlib.Topology.Bases
public import Mathlib.Topology.Order
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Continuous.basicNhhd
public import DomainTheory.DataTypesAsLattices.Continuous.memOpen
```

```
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace

namespace basicNhhd

/-- A basic neighbourhood is a finite intersection of subbasic opens. -/
theorem eq_iInter (n : ℕ) :
    basicNhhd n = ⋂ k ∈ e n, memOpen k := by
  ext x
  simp [basicNhhd, memOpen, Set.subset_def]

end basicNhhd

end DomainTheory.DataTypesAsLattices
```

exists_basicNhhd_subset_of_generateOpen.lean

```
module

public import Mathlib.Topology.Order
public import DomainTheory.DataTypesAsLattices.Continuous.memOpen
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Continuous.basicNhhd
public import DomainTheory.DataTypesAsLattices.Continuous.basicNhhd_more
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace

/-- Every open neighbourhood of `x` contains a basic neighbourhood of `x`. -/
theorem exists_basicNhhd_subset_of_generateOpen {U : Set Pomega}
    (hU : GenerateOpen (Set.range memOpen) U) {x : Pomega} (hx : x ∈ U) :
    ∃ n, e n ⊆ x ∧ basicNhhd n ⊆ U := by
  induction hU generalizing x with
  | basic s hs =>
    obtain ⟨k, rfl⟩ := hs
    refine ⟨2 ^ k, ?_, ?_⟩
    · simpa [memOpen, e_pow2, Set.singleton_subset_iff] using hx
    · rw [← memOpen.eq_basicNhhd]
  | univ =>
    exact ⟨0, by simp [e_zero], by simp [basicNhhd.zero]⟩
  | inter _s _t _hs _ht ihs iht =>
    obtain ⟨n, hn, hnU⟩ := ihs hx.1
    obtain ⟨m, hm, hmU⟩ := iht hx.2
    refine ⟨n ||| m, e_or_of_subset hn hm, ?_⟩
    rw [← basicNhhd.inter]
    exact Set.inter_subset_inter hnU hmU
  | sUnion S _hS ih =>
    obtain ⟨V, hV, hxV⟩ := hx
    obtain ⟨n, hn, hnV⟩ := ih V hV hxV
```

```
    exact ⟨n, hn, hnV.trans (Set.subset_sUnion_of_mem hV)⟩

end DomainTheory.DataTypesAsLattices
```

Theorem 1.5 **THEOREM 1.5** (The extension theorem). *Let X and Y be arbitrary topological spaces where $X \subseteq Y$ as a subspace. Then every continuous function $f : X \to \mathbf{P}\omega$ can be extended to a continuous function $\bar{f} : Y \to \mathbf{P}\omega$ defined by the equation:*

$$\bar{f}(y) = \bigcup\{\bigcap\{f(x) \mid x \in X \cap U\} \mid y \in U\},$$

where $y \in Y$, and U ranges over the open subsets of Y.

isOpen_memOpen.lean

```
module

public import Mathlib.Topology.Bases
public import DomainTheory.DataTypesAsLattices.Continuous.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.memOpen

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace

theorem isOpen_memOpen (k : ℕ) : IsOpen (memOpen k) :=
  TopologicalSpace.isOpen_generateFrom_of_mem ⟨k, rfl⟩

end DomainTheory.DataTypesAsLattices
```

theorem_1_5.lean

```
module

public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.isOpen_memOpen
public import DomainTheory.DataTypesAsLattices.Continuous.memOpen
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Embedding.extend
public import DomainTheory.DataTypesAsLattices.Embedding.embed

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace Topology Function

/-- **Scott 1976, Theorem 1.5 (The extension theorem), continuity.** -/
theorem theorem_1_5 {Y : Type*} [TopologicalSpace Y] {X : Set Y}
```

```
    (f : X → Pomega) : Continuous (extend (Y := Y) (X := X) f) := by
  refine continuous_generateFrom_iff.2 ?_
  rintro _ ⟨k, rfl⟩
  exact extend.isOpen_mem f k

/-- **Scott 1976, Theorem 1.5**, extension property when `f` is continuous. -/
theorem theorem_1_5.extension {Y : Type*} [TopologicalSpace Y] {X : Set Y}
    (f : X → Pomega) (hf : Continuous f) (x : X) :
    extend (Y := Y) (X := X) f ↑x = f x := by
  apply subset_antisymm
  · intro k hk
    obtain ⟨U, _, hxU, hall⟩ := (extend.mem f).mp hk
    exact hall x hxU
  · intro k hk
    have hpre : IsOpen (f ^-1' memOpen k) :=
      IsOpen.preimage hf (isOpen_memOpen k)
    obtain ⟨U, hU, hrel⟩ := isOpen_induced_iff.mp hpre
    refine (extend.mem f).mpr ⟨U, hU, ?_, ?_⟩
    · have : x ∈ Subtype.val ^-1' U := by
        rwa [hrel]
      exact this
    · intro z hz
      have : z ∈ f ^-1' memOpen k := by
        rwa [← hrel]
      exact this

end DomainTheory.DataTypesAsLattices
```

isOpen_basicNhhd.lean

```
module

public import Mathlib.Topology.Bases
public import DomainTheory.DataTypesAsLattices.Continuous.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.isOpen_memOpen
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Continuous.memOpen
public import DomainTheory.DataTypesAsLattices.Continuous.basicNhhd

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace

theorem isOpen_basicNhhd (n : ℕ) : IsOpen (basicNhhd n) := by
  obtain ⟨s, hs⟩ := (e_finite n).exists_finset_coe
  have h : basicNhhd n = ⋂ k ∈ s, memOpen k := by
    ext x
    constructor
    · intro hx
      refine Set.mem_iInter₂.mpr ?_
      intro k hk
      have : k ∈ e n := by
        have : k ∈ (s : Set ℕ) := hk
```

```
        rwa [hs] at this
      exact hx this
    · intro hx k hk
      have : k ∈ s := by
        have : k ∈ e n := hk
        rwa [← hs] at this
      exact Set.mem_iInter₂.mp hx k this
  rw [h]
  exact isOpen_biInter_finset fun _ _ => isOpen_memOpen _

end DomainTheory.DataTypesAsLattices
```

Theorem 1.6 **THEOREM 1.6** (The embedding theorem). *Every T_0-space X with a countable basis $\{U_n \mid n \in \omega\}$ for its topology can be embedded in $\mathbf{P}\omega$ by the continuous function $\varepsilon : X \to \mathbf{P}\omega$ defined by the equation:*

$$\varepsilon(x) = \{n \mid x \in U_n\}.$$

Technically the T_0-hypothesis is what is needed to show that ε is one-to-one. The upshot of these two theorems is that in looking for (reasonable) topological structures we can confine attention to the subspaces of $\mathbf{P}\omega$ and to continuous functions defined on *all* of $\mathbf{P}\omega$. Thus the emphasis on a single space is justified structurally. What we shall see in the remainder of this work is that the use of a single space is also justified practically because the required subspaces and functions can be *defined* in very simple ways by a natural method of equations.

theorem_1_6.lean

```
module

public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Embedding.extend
public import DomainTheory.DataTypesAsLattices.Embedding.embed

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace Topology Function

/-- **Scott 1976, Theorem 1.6 (The embedding theorem).**
Every T₀ space with a countable basis embeds in `Pω`. -/
theorem theorem_1_6 {X : Type*} [TopologicalSpace X] [T0Space X]
    (U : ℕ → Set X) (hbasis : IsTopologicalBasis (Set.range U)) :
    IsEmbedding (embed U) where
  toIsInducing := embed.inducing U hbasis
  injective := embed.injective U hbasis

theorem theorem_1_6.injective_continuous {X : Type*} [TopologicalSpace X]
    [T0Space X] (U : ℕ → Set X) (hbasis : IsTopologicalBasis (Set.range U)) :
    Injective (embed U) ∧ Continuous (embed U) :=
```

```
  ⟨embed.injective U hbasis, embed.continuous U hbasis⟩

end DomainTheory.DataTypesAsLattices
```

2.3 Computability and definability

2.3.1 Historical note

The language LAMBDA is Scott's. Plotkin and Rogers already used graphs, application, and conversion, and the combinators they had would have defined abstraction if anyone had asked; the abstraction operation is what this section supplies. The reduction theorem and the combinator theorem are the classical ones from combinatory logic. The first recursion theorem is classical as well. The argument that Curry's combinator $\mathbf{Y}$ is the least fixed point in this model is David Park's, from an unpublished note of 1970, and it is recorded with Theorem 2.5. The equation $\mathbf{Y}(u) = u(\mathbf{Y}(u))$ is the usual calculation from (2.8). Leastness uses the pairing (n, m) and the finite sets e_l, and does not appeal to Theorem 1.4. The definition of computability and Theorem 2.6 identify the LAMBDA-definable functions with the enumeration operators.

Dependency diagram. Each box is one note that contains Lean. The arrowhead is on the note that imports. The number is how many direct imports. An arrow already implied by a longer path is left out.

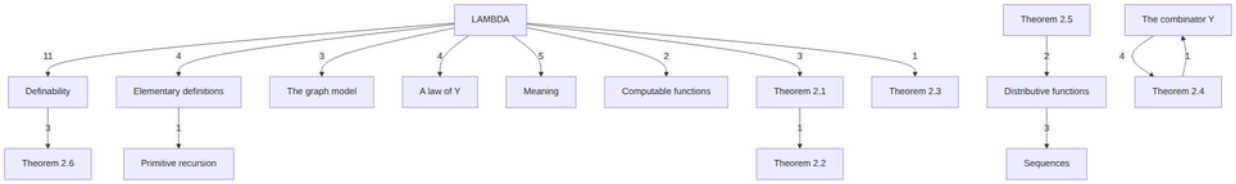


Figure 21: Historical note.

2.3.2 Meaning

The purpose of the first section was to introduce in a simple-minded way the basic notions about the topology of $\mathbf{P}\omega$ and its continuous functions. In this section we wish to present the details of a powerful language for defining *particular* functions—especially computable functions—and initiate the study of the *use* of these functions. This study is then extended in different ways in the following sections.

Before looking at the language, a short discussion of the "meaning" of the elements $x \in \mathbf{P}\omega$ will be helpful from the point of view of motivation. Now in itself $x \in \mathbf{P}\omega$ is a set, but this does not reveal its *meaning*. Actually x has no "fixed" meaning, because it can be *used* in strikingly different ways; we look for meaning here solely in terms of use. Nevertheless it is possible to give some coherent guidelines.

In the first place it is convenient to let the singleton subsets $\{n\} \in \mathbf{P}\omega$ stand for the corresponding integers. In fact, we shall enforce by convention the equation $n = \{n\}$ as a way of simplifying notation. In this way, $\omega \subseteq \mathbf{P}\omega$ as a subset. (Note that our convention conflicts with such set-theoretical equations as $5 = \{0, 1, 2, 3, 4\}$. What we have done is to abandon the usual set-theoretical conventions in favor of a slight redefinition of *set of integers* which produces a more helpful convention for present purposes.) So, if we choose, a singleton "means" a single integer. The next question is what a "large" set $x \in \mathbf{P}\omega$ could mean. Here is an answer: if we write:

$$x = \{0, 5, 17\} = 0 \cup 5 \cup 17,$$

we are thinking of x as a *multiple integer*. This is especially useful in the case of multiple-valued function where we can write:

$$f(a) = 0 \cup 5 \cup 17.$$

Then "$m \in f(a)$" can be interpreted as "m is one value of f at a." Now $a \in \mathbf{P}\omega$, too, and so it is a multiple integer also. This brings us to an important point.

A multiple integer is (often) *more* than just the (random) collection of its elements. From the definition of continuity, $m \in f(a)$ is equivalent to $m \in f(e_n)$ with $e_n \subseteq a$. We may not be able to reduce this to $m \in f(\{n\})$ with $n \in a$ without additional assumptions on f. Indeed we shall take advantage of the feature of continuous functions whereby the elements of an argument a can join in *cooperation* in determining $f(a)$. Needless to say, continuity implies that the cooperation cannot extend beyond *finite* configurations, and so we can say that a is the union (or limit) of its finite subsets. However, finitary cooperation will be found to be quite a powerful notion.

Where does this interpretation leave the empty set $\varnothing$? When we write "$f(a) = \varnothing$" we can read this as "f has no value at a", or "f is undefined at a". In this case $f(a)$ exists (as a set), but it is not "defined" as an integer. Single- (or singleton-) valued functions are "well-defined", but multiple-valued functions are rather "over-defined".

How does this interpretation fit in with monotonicity? In case $a \subseteq b$ and $m \in f(a)$, then we must have $m \in f(b)$. We can read "$a \subseteq b$" as "b is an *improvement of* a" or "b is *better-defined than* a." The point of monotonicity is that the better we define an argument, the better we define a value. "Better" does not imply "well" (that is, singleton-valuedness), and overdefinedness may well creep in. This is not the fault of the function; it is our fault for not choosing a different function.

As a subspace $\omega \subseteq \mathbf{P}\omega$ is discrete. This implies that arbitrary functions $p : \omega \to \omega$ are continuous. Note that $p : \omega \to \mathbf{P}\omega$ as well, because $\omega \subseteq \mathbf{P}\omega$. By Theorem 1.5 we can extend p continuously to $\bar{p} : \mathbf{P}\omega \to \mathbf{P}\omega$. The formula given produces this function:

$$\bar{p}(x) = \begin{cases} \bigcap\{p(n) \mid n \in \omega\} & \text{if } x = \varnothing; \\ p(n) & \text{if } x = n \in \omega; \\ \omega & \text{otherwise.} \end{cases} \tag{2.1}$$

This is a rather abrupt extension of p (the *maximal extension*); a more gradual, continuous extension (the *minimal extension*) is determined by this equation:

$$\hat{p}(x) = \bigcup\{p(n) \mid n \in x\}. \tag{2.2}$$

The same formulae work for *all* multiple-valued functions $p : \omega \to \mathbf{P}\omega$. Functions like $f = \hat{p}$ are exactly characterized as being those continuous functions $f : \mathbf{P}\omega \to \mathbf{P}\omega$ which in addition are *distributive* in the sense of these equations:

$$f(x \cup y) = f(x) \cup f(y) \quad \text{and} \quad f(\varnothing) = \varnothing.$$

The sets $\varnothing$ and ω play special roles. When we consider them as elements of $\mathbf{P}\omega$ we shall employ the notation:

$$\bot = \varnothing \quad \text{and} \quad \top = \omega$$

The element $\bot$ is the most "undefined" integer, and $\top$ is the most "overdefined". All others are in between.

One last general point on meaning: suppose $x \in \mathbf{P}\omega$ and $k \in x$. Then $k = (n, m)$ for suitable (uniquely determined) integers n and m. That is to say, every element of x can be regarded as an ordered pair; thus, x can be used as a relation. Such an operation as

$$(2.3) \quad x; y = \{(n, l) \mid \exists m.\, (n, m) \in x,\, (m, l) \in y\}$$

is then a continuous function of two variables that treats both x and y as relations. On the other hand we could define a quite different continuous function such as

$$(2.4) \quad x + y = \{n + m \mid n \in x,\, m \in y\}$$

which treats both x and y arithmetically. The only reason we shall probably never write $(x + y); x$ again is that the values of this perfectly well-defined continuous function are, for the most part, quite uninteresting. There is, however, no theoretical reason why we cannot use the same set with several different "meanings" in the same formula. Of course if we do so, it is to be expected that we will show the point of doing this in the special case. We turn now to the definition of the general language for defining all such functions.

eq_2_1_empty.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.maxExtend
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.1).** Maximal extension by cases. -/
theorem eq_2_1_empty (p : ℕ → Pomega) :
    maxExtend p botElem = ⋂ n, p n := by
  ext k
  constructor
  · intro hk
    rcases hk with ⟨_, hall⟩ | h | h
    · exact Set.mem_iInter.mpr hall
    · rcases h with ⟨n, hn, _⟩
      have : n ∈ (∅ : Pomega) := by
        have : (∅ : Pomega) = ofNat n := hn
        rw [this]; simp [ofNat]
      exact this.elim
    · exact (h.1 rfl).elim
  · intro hk
    exact Or.inl ⟨rfl, fun n => Set.mem_iInter.mp hk n⟩

end DomainTheory.DataTypesAsLattices
```

minExtend.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import Mathlib.Data.Set.Lattice.Indexed
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.2).** Minimal (distributive) extension `p(x) = ⋃ {p(n) | n ∈ x}`.
    -/
def minExtend (p : ℕ → Pomega) (x : Pomega) : Pomega :=
  ⋃ n ∈ x, p n

namespace minExtend

theorem bot (p : ℕ → Pomega) : minExtend p botElem = botElem := by
  simp [minExtend, botElem]

theorem isScottContinuous (p : ℕ → Pomega) :
    IsScottContinuous (minExtend p) := by
  constructor
  intro x
  ext k
  constructor
  · intro hk
    obtain ⟨n, hn, hkn⟩ := Set.mem_iUnion₂.mp hk
    exact mem_scottUnion.mpr ⟨2 ^ n, by simp [e_pow2, Set.singleton_subset_iff, hn],
      by simp [minExtend, e_pow2, hkn]⟩
  · intro hk
    obtain ⟨m, hm, hkm⟩ := mem_scottUnion.mp hk
    obtain ⟨n, hn, hkn⟩ := Set.mem_iUnion₂.mp hkm
    exact Set.mem_iUnion₂.mpr ⟨n, hm hn, hkn⟩

theorem union (p : ℕ → Pomega) (x y : Pomega) :
    minExtend p (x ∪ y) = minExtend p x ∪ minExtend p y := by
  ext k
  simp [minExtend, or_and_right, exists_or]

end minExtend

end DomainTheory.DataTypesAsLattices
```

maxExtend.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices
```

```
/-- **Scott 1976, (2.1).** Maximal extension, written without a decidability side
    condition. -/
def maxExtend (p : ℕ → Pomega) (x : Pomega) : Pomega :=
  {k | (x = ∅ ∧ ∀ n, k ∈ p n) ∨
    (∃ n, x = ofNat n ∧ k ∈ p n) ∨
    (x = ∅ ∧ ∀ n, x = ofNat n)}

end DomainTheory.DataTypesAsLattices
```

2.3.3 LAMBDA

The syntax and semantics of the language LAMBDA are set out in Table 2. The syntax is indicated on the left, and the meanings of the combinations are shown on the right as subsets of $\mathbf{P}\omega$. This is the basic language and could have been given (less understandably) in terms of combinators (see Theorem 2.4). It is, however, a very primitive language, and we shall require many definitions before we can see why such functions as in (2.3) and (2.4) are themselves definable.

TABLE 2

The language LAMBDA

$$0 = \{0\}$$

$$x + 1 = \{n + 1 \mid n \in x\}$$

$$x - 1 = \{n \mid n + 1 \in x\}$$

$$z \supset x, y = \{n \in x \mid 0 \in z\} \cup \{m \in y \mid \exists k.\, k + 1 \in z\}$$

$$u(x) = \{m \mid \exists e_n \subseteq x.\, (n, m) \in u\}$$

$$\lambda x.\, \tau = \{(n, m) \mid m \in \tau[e_n/x]\}$$

The definition has been left somewhat informal in hopes that it will be more understandable. In the above, τ is any term of the language. LAMBDA is type-free and allows any combination to be made by substitution into the given functions. There is one primitive constant (0); there are two unary functions $(x + 1,\ x - 1)$; there is one binary function $(u(x))$ and one ternary function $(z \supset x, y)$; finally there is one variable binding operator $(\lambda x.\, \tau)$. The first three equations have obvious sense. In the fourth, $z \supset x, y$ is McCarthy's conditional expression (a test for zero). Next $u(x)$ defines *application* (u is treated as a graph and x as a set), and $\lambda x.\, \tau$ is *functional abstraction* (compare the definition of **fun**). In defining $\lambda x.\, \tau$, we use $\tau[e_n/x]$ as a shorthand for evaluating the term τ when the variable x is given the value e_n.

Note that the functions are all multiple-valued. Thus we have such a result as:

$$(2.5) \quad (6 \cup 10) + 1 = 7 \cup 11.$$

The partial character of subtraction has expression as:

$$(2.6)\quad 0 - 1 = \bot.$$

We shall see how to define $+$ and $-$ in LAMBDA later. The conditional could also have been defined by cases:

$$(2.7)\quad z \supset x, y = \begin{cases} \bot & \text{if } z = \bot; \\ x & \text{if } z = 0; \\ y & \text{if } 0 \notin z \neq \bot; \\ x \cup y & \text{if } 0 \in z \neq 0. \end{cases}$$

We say that a LAMBDA-term τ defines a function of its free variables (at least).

pair_zero_zero.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.pair

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem pair_zero_zero : pair 0 0 = 0 := by simp [pair]
end DomainTheory.DataTypesAsLattices
```

Comb.lean

```
module

public import Mathlib.Data.Nat.Basic

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- Combinatory expressions generated from the six constants and variables. -/
inductive Comb where
  | var : ℕ → Comb
  | zero | suc | pred | cond | K | S
  | app : Comb → Comb → Comb

end DomainTheory.DataTypesAsLattices
```

triangle_eq_zero.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.pair

@[expose] public section
```

```
namespace DomainTheory.DataTypesAsLattices

theorem triangle_eq_zero {w : ℕ} (h : triangle w = 0) : w = 0 := by
  match w with
  | 0 => rfl
  | n + 1 =>
    have h1 : 1 ≤ n + 1 := Nat.succ_le_succ (Nat.zero_le _)
    have h2 : 2 ≤ (n + 1) * (n + 2) :=
      calc 2 ≤ 1 * 2 := by decide
           _ ≤ (n + 1) * (n + 2) := Nat.mul_le_mul h1 (Nat.succ_le_succ h1)
    have : 0 < (n + 1) * (n + 2) / 2 := Nat.div_pos h2 (by decide)
    exact (Nat.ne_of_gt this h).elim

end DomainTheory.DataTypesAsLattices
```

pair_one_zero.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.pair

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem pair_one_zero : pair 1 0 = 1 := by simp [pair]

end DomainTheory.DataTypesAsLattices
```

exists_least_not_mem.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

lemma exists_least_not_mem {a : Pomega} (hex : ∃ k, k ∈ a) :
    ∃ k, k ∈ a ∧ ∀ i < k, i ∈ a := by
  have : ∀ n, n ∈ a → ∃ k ≤ n, k ∈ a ∧ ∀ i < k, i ∈ a := by
    intro n
    induction n using Nat.strong_induction_on with
    | h n ih =>
      intro hn
      rcases Classical.em (∀ i < n, i ∈ a) with hmin | hmin
      · exact ⟨n, le_rfl, hn, hmin⟩
      · obtain ⟨i, hi⟩ := not_forall.mp hmin
        have hi' : i < n := (Classical.not_imp.mp hi).1
        have hia : i ∈ a := (Classical.not_imp.mp hi).2
        obtain ⟨k, hki, hk, hall⟩ := ih i hi' hia
        exact ⟨k, hki.trans hi'.le, hk, hall⟩
  obtain ⟨n, hn⟩ := hex
  obtain ⟨k, _, hk, hall⟩ := this n hn
```

```
  exact ⟨k, hk, hall⟩

end DomainTheory.DataTypesAsLattices
```

ofNat.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- Scott's integer convention: `n = {n}` as an element of `Pω`. -/
def ofNat (n : ℕ) : Pomega := {n}

end DomainTheory.DataTypesAsLattices
```

succSet.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, Table 2.** Successor `x + 1 = {n + 1 | n ∈ x}`. -/
def succSet (x : Pomega) : Pomega :=
  {k | ∃ n ∈ x, k = n + 1}

end DomainTheory.DataTypesAsLattices
```

e_one.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem e_one : e 1 = ({0} : Pomega) := by
  simpa using e_pow2 0

end DomainTheory.DataTypesAsLattices
```

envSet.lean

```
module
```

```
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices
def envSet (? : ℕ → Pomega) (i : ℕ) (v : Pomega) : ℕ → Pomega :=
  Function.update ? i v

namespace envSet

theorem comm {? : ℕ → Pomega} {i j : ℕ} (hij : i = j) (v w : Pomega) :
    envSet (envSet ? i v) j w = envSet (envSet ? j w) i v := by
  funext k
  unfold envSet
  by_cases hj : k = j
  · subst hj
    simp [Function.update, hij.symm]
  · by_cases hi : k = i
    · subst hi
      simp [Function.update, hij]
    · simp [Function.update, hi, hj]

end envSet

end DomainTheory.DataTypesAsLattices
```

predSet.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, Table 2.** Predecessor `x ? 1 = {n | n + 1 ∈ x}`. -/
def predSet (x : Pomega) : Pomega :=
  {k | k + 1 ∈ x}

end DomainTheory.DataTypesAsLattices
```

pair_eq_right.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Lambda.triangle_eq_zero

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem pair_eq_right {n m : ℕ} (h : pair n m = m) : n = 0 ∧ m = 0 := by
```

```
  have : triangle (n + m) = 0 := by
    have h' : triangle (n + m) + m = m := by simpa [pair_eq_triangle] using h
    omega
  have : n + m = 0 := triangle_eq_zero this
  omega

end DomainTheory.DataTypesAsLattices
```

zeroC.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def zeroC : Pomega := ofNat 0

end DomainTheory.DataTypesAsLattices
```

exists_least_mem.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

lemma exists_least_mem {a : Pomega} (hex : ∃ k, k ∈ a) :
    ∃ k, k ∈ a ∧ ∀ i < k, i ∈ a := by
  have : ∀ n, n ∈ a → ∃ k ≤ n, k ∈ a ∧ ∀ i < k, i ∈ a := by
    intro n
    induction n using Nat.strong_induction_on with
    | h n ih =>
      intro hn
      rcases Classical.em (∀ i < n, i ∈ a) with hmin | hmin
      · exact ⟨n, le_rfl, hn, hmin⟩
      · obtain ⟨i, hi⟩ := not_forall.mp hmin
        have hi' : i < n := (Classical.not_imp.mp hi).1
        have hia : i ∈ a := not_not.mp (Classical.not_imp.mp hi).2
        obtain ⟨k, hki, hk, hall⟩ := ih i hi' hia
        exact ⟨k, hki.trans hi'.le, hk, hall⟩
  obtain ⟨n, hn⟩ := hex
  obtain ⟨k, _, hk, hall⟩ := this n hn
  exact ⟨k, hk, hall⟩

end DomainTheory.DataTypesAsLattices
```

mem_ofNat.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

@[simp] theorem mem_ofNat {n k : ℕ} : k ∈ ofNat n ↔ k = n := by simp [ofNat]

end DomainTheory.DataTypesAsLattices
```

eq_2_1_singleton.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.maxExtend
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem eq_2_1_singleton (p : ℕ → Pomega) (n : ℕ) :
    maxExtend p (ofNat n) = p n := by
  ext k
  constructor
  · intro hk
    rcases hk with ⟨he, _⟩ | ⟨m, hm, hkm⟩ | ⟨_, hall⟩
    · have hne : ofNat n = ∅ := Set.Nonempty.ne_empty ⟨n, (rfl : n ∈ ofNat n)⟩
      exact (hne he).elim
    · have : n = m := by
        have hn : n ∈ ofNat m := hm ▸ (rfl : n ∈ ofNat n)
        simpa [ofNat] using hn
      subst this; exact hkm
    · exact (hall n rfl).elim
  · intro hk
    exact Or.inr (Or.inl ⟨n, rfl, hk⟩)

end DomainTheory.DataTypesAsLattices
```

eq_2_1_else.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.maxExtend
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices
```

```
theorem eq_2_1_else (p : ℕ → Pomega) {x : Pomega}
    (hne : x = botElem) (hsing : ∀ n, x = ofNat n) :
    maxExtend p x = topElem := by
  ext k
  constructor
  · intro; trivial
  · intro
    exact Or.inr (Or.inr ⟨hne, hsing⟩)

end DomainTheory.DataTypesAsLattices
```

eq_2_5.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.succSet

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.5).** `(6 ∪ 10) + 1 = 7 ∪ 11`. -/
theorem eq_2_5 : succSet (ofNat 6 ∪ ofNat 10) = ofNat 7 ∪ ofNat 11 := by
  ext k
  constructor
  · intro ⟨n, hn, hk⟩
    rcases hn with h6 | h10
    · simp [ofNat] at h6; subst h6; simp [ofNat, hk]
    · simp [ofNat] at h10; subst h10; simp [ofNat, hk]
  · intro hk
    rcases hk with h7 | h11
    · simp [ofNat] at h7; exact ⟨6, Or.inl (by simp [ofNat]), by simp [h7]⟩
    · simp [ofNat] at h11; exact ⟨10, Or.inr (by simp [ofNat]), by simp [h11]⟩

end DomainTheory.DataTypesAsLattices
```

eq_2_6.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.predSet

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.6).** `0 ? 1 = ⊥`. -/
theorem eq_2_6 : predSet (ofNat 0) = botElem := by
  ext k
```

```
  simp [predSet, ofNat, botElem]

end DomainTheory.DataTypesAsLattices
```

mem_scottUnion.lean

```
module

public import Mathlib.Topology.Bases
public import DomainTheory.DataTypesAsLattices.Continuous.scottUnion
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace

theorem mem_scottUnion {f : Pomega → Pomega} {x : Pomega} {k : ℕ} :
    k ∈ scottUnion f x ↔ ∃ n, e n ⊆ x ∧ k ∈ f (e n) := by
  constructor
  · intro hk
    obtain ⟨n, hn, hfk⟩ := Set.mem_iUnion.mp hk
    exact ⟨n, hn, hfk⟩
  · intro ⟨n, hn, hk⟩
    exact Set.mem_iUnion.mpr ⟨n, hn, hk⟩

end DomainTheory.DataTypesAsLattices
```

graph.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Pomega.e
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, §1, Definition.**
`graph(f) = {(n, m) | m ∈ f(e n)}`. -/
def graph (f : Pomega → Pomega) : Pomega :=
  {p | ∃ n m, p = pair n m ∧ m ∈ f (e n)}

end DomainTheory.DataTypesAsLattices
```

sucC.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.succSet
```

```
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def sucC : Pomega := graph succSet

end DomainTheory.DataTypesAsLattices
```

Kcomb.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Pomega.e
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def Kcomb : Pomega := graph (fun x => graph (fun _ => x))

end DomainTheory.DataTypesAsLattices
```

predC.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.predSet

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def predC : Pomega := graph predSet

end DomainTheory.DataTypesAsLattices
```

lam.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.Basic.notation3

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- Abstraction: `λ x. τ` as a graph (Table 2). -/
```

```
def lam (τ : Pomega → Pomega) : Pomega := graph τ

namespace lam

theorem eq_graph (τ : Pomega → Pomega) : lam τ = graph τ := rfl

end lam

end DomainTheory.DataTypesAsLattices
```

Term.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.lam
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, Table 2.** Syntax of LAMBDA. -/
inductive Term where
  | var : ℕ → Term
  | zero : Term
  | succ : Term → Term
  | pred : Term → Term
  | cond : Term → Term → Term → Term
  | app : Term → Term → Term
  | lam : ℕ → Term → Term

end DomainTheory.DataTypesAsLattices
```

Avoids.lean

```
module

public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Lambda.Term
public import DomainTheory.DataTypesAsLattices.Graph.lam

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable

open Nat.Partrec (Code)

open Nat.Partrec.Code

/-- `t` does not depend on environment slot `i`. -/
def Term.Avoids (i : ℕ) : Term → Prop
  | .var j => j = i
```

```
  | .zero => True
  | .succ t => t.Avoids i
  | .pred t => t.Avoids i
  | .cond z x y => z.Avoids i ∧ x.Avoids i ∧ y.Avoids i
  | .app u x => u.Avoids i ∧ x.Avoids i
  | .lam j body => j = i ∨ body.Avoids i

end DomainTheory.DataTypesAsLattices
```

rename.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.lam
public import DomainTheory.DataTypesAsLattices.Lambda.Term
public import DomainTheory.DataTypesAsLattices.Enumeration.Avoids

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- Rename free `i` to a completely fresh `j`. -/
def rename (i j : ℕ) : Term → Term
  | .var k => if k = i then .var j else .var k
  | .zero => .zero
  | .succ t => .succ (rename i j t)
  | .pred t => .pred (rename i j t)
  | .cond z x y => .cond (rename i j z) (rename i j x) (rename i j y)
  | .app u x => .app (rename i j u) (rename i j x)
  | .lam k body => if k = i then .lam k body else .lam k (rename i j body)

end DomainTheory.DataTypesAsLattices
```

2.3.4 Theorem 2.1

THEOREM 2.1 (The continuity theorem). *All LAMBDA definable functions are continuous.*

theorem_2_1.lean

```
module

public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.Term
public import DomainTheory.DataTypesAsLattices.Lambda.envSet
public import DomainTheory.DataTypesAsLattices.Lambda.interp
public import DomainTheory.DataTypesAsLattices.Enumeration.Avoids
@[expose] public section

namespace DomainTheory.DataTypesAsLattices
```

```
/-- **Scott 1976, Theorem 2.1 (The continuity theorem).** -/
theorem theorem_2_1 (t : Term) (? : ℕ → Pomega) (i : ℕ) :
    IsScottContinuous (fun v => interp t (envSet ? i v)) :=
  interp.var_isScottContinuous t ? i

end DomainTheory.DataTypesAsLattices
```

2.3.5 Theorem 2.2

Once that is proved, we can use Theorem 1.2 to establish:

THEOREM 2.2 (The conversion theorem). *The three basic principles* (α), (β), (ξ) *of* λ*-conversion are all valid in the model.*

primRecStepTerm.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.lam
public import DomainTheory.DataTypesAsLattices.Lambda.Term
public import DomainTheory.DataTypesAsLattices.Enumeration.Avoids
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.envSet
public import DomainTheory.DataTypesAsLattices.Lambda.interp
public import DomainTheory.DataTypesAsLattices.Lambda.theorem_2_2
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.predSet

@[expose] public section

namespace DomainTheory.DataTypesAsLattices
/-- Combinatory primitive-recursion step `λa λf λu λn. n ? a, f(n?1)(u(n?1))`. -/
def primRecStepTerm : Term :=
  .lam 0 (.lam 1 (.lam 2 (.lam 3
    (.cond (.var 3) (.var 0)
      (.app (.app (.var 1) (.pred (.var 3)))
        (.app (.var 2) (.pred (.var 3)))))))))

namespace primRecStepTerm

theorem interp (a f u n : Pomega) :
    funOf (funOf (funOf (funOf (interp primRecStepTerm (fun _ => botElem)) a) f) u) n
    =
      condSet n a (funOf (funOf f (predSet n)) (funOf u (predSet n))) := by
  change funOf (funOf (funOf (funOf (DomainTheory.DataTypesAsLattices.interp (Term.lam
     0 _) (fun _ => botElem)) a) f) u) n = _
  rw [theorem_2_2.beta]
  rw [theorem_2_2.beta]
  rw [theorem_2_2.beta]
  rw [theorem_2_2.beta]
  simp [_root_.DomainTheory.DataTypesAsLattices.interp, envSet, Function.update]
```

```
end primRecStepTerm

end DomainTheory.DataTypesAsLattices
```

primRecStep.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.predSet
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.envSet
public import DomainTheory.DataTypesAsLattices.Lambda.interp
public import DomainTheory.DataTypesAsLattices.Lambda.Term
public import DomainTheory.DataTypesAsLattices.Lambda.primRecStepTerm
public import DomainTheory.DataTypesAsLattices.Lambda.id_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Graph.lam
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Lambda.theorem_2_2
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
public import DomainTheory.DataTypesAsLattices.Lambda.predSet_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def primRecStep (a f u : Pomega) : Pomega :=
  graph (fun n => condSet n a (funOf (funOf f (predSet n)) (funOf u (predSet n))))

namespace primRecStep

theorem body_isScottContinuous_n (a f u : Pomega) :
    IsScottContinuous (fun n =>
      condSet n a (funOf (funOf f (predSet n)) (funOf u (predSet n)))) :=
  theorem_1_3.tuple
    (fun y => condSet.isScottContinuous_left a y)
    (fun n => condSet.isScottContinuous_right n a)
    id_isScottContinuous
    (theorem_1_3.tuple
      (fun y => funOf.isScottContinuous_left y)
      (fun x => funOf.isScottContinuous x)
      (theorem_1_3 (funOf.isScottContinuous f) predSet.isScottContinuous)
      (theorem_1_3 (funOf.isScottContinuous u) predSet.isScottContinuous))

theorem graph_of (a f : Pomega) :
    graph (primRecStep a f) =
```

```
      funOf (funOf (interp primRecStepTerm (fun _ => botElem)) a) f := by
  change graph (primRecStep a f) =
    funOf (funOf (DomainTheory.DataTypesAsLattices.interp (Term.lam 0 (.lam 1 (.lam 2
    (.lam 3
      (.cond (.var 3) (.var 0)
        (.app (.app (.var 1) (.pred (.var 3)))
          (.app (.var 2) (.pred (.var 3)))))))))) (fun _ => botElem)) a) f
  rw [theorem_2_2.beta, theorem_2_2.beta]
  apply congrArg graph
  funext u
  apply congrArg graph
  funext n
  simp [DomainTheory.DataTypesAsLattices.interp, envSet, Function.update]

theorem isScottContinuous (a f : Pomega) :
    IsScottContinuous (primRecStep a f) :=
  graph_const_isScottContinuous
    (fun u n => condSet n a (funOf (funOf f (predSet n)) (funOf u (predSet n))))
    (fun n => theorem_1_3 (condSet.isScottContinuous_right n a)
      (theorem_1_3 (funOf.isScottContinuous (funOf f (predSet n)))
        (funOf.isScottContinuous_left (predSet n))))

theorem app (a f u n : Pomega) :
    funOf (primRecStep a f u) n =
      condSet n a (funOf (funOf f (predSet n)) (funOf u (predSet n))) :=
  beta (primRecStep.body_isScottContinuous_n a f u) n

end primRecStep

end DomainTheory.DataTypesAsLattices
```

theorem_2_2.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.lam
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.Term
public import DomainTheory.DataTypesAsLattices.Lambda.envSet
public import DomainTheory.DataTypesAsLattices.Lambda.interp
public import DomainTheory.DataTypesAsLattices.Lambda.rename

public import DomainTheory.DataTypesAsLattices.Enumeration.Avoids
public import DomainTheory.DataTypesAsLattices.Lambda.theorem_2_1

@[expose] public section

namespace DomainTheory.DataTypesAsLattices
/-- **Scott 1976, Table 1 (α).** Bound variables may be renamed when the
bodies agree after the rename (Scott: "`x` is a bound variable"). -/
theorem theorem_2_2.alpha (i j : ℕ) (body : Term) (? : ℕ → Pomega)
    (h : ∀ v, interp body (envSet ? i v) =
```

```
            interp (rename i j body) (envSet ? j v)) :
    interp (.lam i body) ? = interp (.lam j (rename i j body)) ? := by
  unfold interp
  exact congrArg graph (funext h)

/-- **Scott 1976, Theorem 2.2 (?).** -/
theorem theorem_2_2.xi {i : ℕ} {τ σ : Term} {? : ℕ → Pomega}
    (h : ∀ v, interp τ (envSet ? i v) = interp σ (envSet ? i v)) :
    interp (.lam i τ) ? = interp (.lam i σ) ? := by
  simp [interp, h]

/-- **Scott 1976, Theorem 2.2 (β).** -/
theorem theorem_2_2.beta (i : ℕ) (body : Term) (? : ℕ → Pomega) (y : Pomega) :
    funOf (interp (.lam i body) ?) y = interp body (envSet ? i y) :=
  DomainTheory.DataTypesAsLattices.beta (theorem_2_1 body ? i) y

/-- **Scott 1976, Theorem 2.2 (The conversion theorem).**
The three basic principles (α), (β), (?) are valid in the model. -/
theorem theorem_2_2 (i : ℕ) (body : Term) (? : ℕ → Pomega) (y : Pomega) :
    (∀ j, (∀ v, interp body (envSet ? i v) =
            interp (rename i j body) (envSet ? j v)) →
        interp (.lam i body) ? = interp (.lam j (rename i j body)) ?) ∧
      funOf (interp (.lam i body) ?) y = interp body (envSet ? i y) ∧
        ∀ σ, (∀ v, interp body (envSet ? i v) = interp σ (envSet ? i v)) →
          interp (.lam i body) ? = interp (.lam i σ) ? :=
  ⟨fun j h => theorem_2_2.alpha i j body ? h,
    theorem_2_2.beta i body ? y,
    fun _σ h => theorem_2_2.xi h⟩

end DomainTheory.DataTypesAsLattices
```

2.3.6 Theorem 2.3

By "model" here we of course understand the interpretation of the language where the semantics gives terms denotations in $\mathbf{P}\omega$ according to the stated definition. Through this interpretation, more properly speaking, $\mathbf{P}\omega$ becomes a model for the axioms (α), (β), (ξ). Two well-known results of the calculus of λ-conversion allow the reduction of functions of several variables to those of *one*, and the reduction of all the primitives to *combinators* (constants)—all this with the aid of the binary operation of application.

THEOREM 2.3 (The reduction theorem). *Any continuous function of k-variables can be written as*

$$f(x_0, x_1, \cdots, x_{k-1}) = u(x_0)(x_1)\cdots(x_{k-1}),$$

where u is a suitably chosen element of $\mathbf{P}\omega$.

theorem_2_3.lean

```
module

public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Graph.graph
```

```
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.theorem_1_2
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.ScottContinuousFin
public import DomainTheory.DataTypesAsLattices.Lambda.curryFin
public import DomainTheory.DataTypesAsLattices.Lambda.nestApplyFin
public import DomainTheory.DataTypesAsLattices.Lambda.curry2
public import DomainTheory.DataTypesAsLattices.Lambda.curry3
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, Theorem 2.3 (The reduction theorem).**
Any continuous function of `k` variables is represented by an element
`u ∈ Pω` via iterated application. -/
theorem theorem_2_3 {n : ℕ} {f : (Fin n → Pomega) → Pomega}
    (hf : ScottContinuousFin f) :
    ∃ u, ∀ xs, nestApplyFin u xs = f xs :=
  ⟨curryFin f, curryFin.app hf⟩

/-- **Scott 1976, Theorem 2.3, unary case.** -/
theorem theorem_2_3.unary {f : Pomega → Pomega} (hf : IsScottContinuous f) :
    ∃ u, ∀ x, funOf u x = f x :=
  ⟨graph f, fun x => by rw [theorem_1_2.i hf]⟩

/-- **Scott 1976, Theorem 2.3 (The reduction theorem), binary case.** -/
theorem theorem_2_3.binary {f : Pomega → Pomega → Pomega}
    (hfx : ∀ y, IsScottContinuous (fun x => f x y))
    (hfy : ∀ x, IsScottContinuous (fun y => f x y)) :
    ∃ u, ∀ x y, funOf (funOf u x) y = f x y :=
  ⟨curry2 f hfx hfy, curry2.app f hfx hfy⟩

/-- **Scott 1976, Theorem 2.3, ternary case** (the usual reduction of `k`). -/
theorem theorem_2_3.ternary {f : Pomega → Pomega → Pomega → Pomega}
    (hf0 : ∀ y z, IsScottContinuous (fun x => f x y z))
    (hf1 : ∀ x z, IsScottContinuous (fun y => f x y z))
    (hf2 : ∀ x y, IsScottContinuous (fun z => f x y z)) :
    ∃ u, ∀ x y z, funOf (funOf (funOf u x) y) z = f x y z :=
  ⟨curry3 f hf0 hf1 hf2, curry3.app f hf0 hf1 hf2⟩

end DomainTheory.DataTypesAsLattices
```

2.3.7 Theorem 2.4

THEOREM 2.4 (The combinator theorem). *The LAMBDA-definable functions* can be generated (*from variables*) by iterated application with the aid of these six constants:

$$\mathbf{0} = 0$$

$$\mathbf{suc} = \lambda x.\, x + 1$$

$$\mathbf{pred} = \lambda x.\, x - 1$$

$$\mathbf{cond} = \lambda x \lambda y \lambda z.\, z \supset x, y$$

$$\mathbf{K} = \lambda x \lambda y.\, x$$

$$\mathbf{S} = \lambda u \lambda v \lambda x.\, u(x)(v(x))$$

But the result that makes all this model building and combinatory logic mathematically interesting concerns the so-called *paradoxical combinator* defined by the equation:

$$(2.8) \quad \mathbf{Y} = \lambda u.\, (\lambda x.\, u(x(x)))(\lambda x.\, u(x(x))).$$

theorem_2_4.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.IsCombinatory
public import DomainTheory.DataTypesAsLattices.Lambda.zeroC
public import DomainTheory.DataTypesAsLattices.Lambda.sucC
public import DomainTheory.DataTypesAsLattices.Lambda.Kcomb
public import DomainTheory.DataTypesAsLattices.Lambda.predC
public import DomainTheory.DataTypesAsLattices.Lambda.condC
public import DomainTheory.DataTypesAsLattices.Lambda.Scomb
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.Term
public import DomainTheory.DataTypesAsLattices.Lambda.interp
public import DomainTheory.DataTypesAsLattices.Lambda.ofComb
public import DomainTheory.DataTypesAsLattices.Lambda.erase
public import DomainTheory.DataTypesAsLattices.Lambda.eq_2_6
public import DomainTheory.DataTypesAsLattices.Enumeration.Avoids
public import DomainTheory.DataTypesAsLattices.Lambda.predC_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, Theorem 2.4 (The combinator theorem).** -/
theorem theorem_2_4 : IsCombinatory zeroC ∧ IsCombinatory sucC ∧ IsCombinatory predC ∧
    IsCombinatory condC ∧ IsCombinatory Kcomb ∧ IsCombinatory Scomb :=
  ⟨.zero, .suc, .pred, .cond, .K, .S⟩

/-- **Scott 1976, Theorem 2.4 (The combinator theorem).** Every LAMBDA
term equals an applicative combination of the six constants and its
free variables. -/
theorem theorem_2_4.complete (t : Term) (? : ℕ → Pomega) :
    interp t ? = ofComb (erase t) ? :=
  (erase.correct t ?).symm

theorem theorem_2_4.closed (t : Term) :
```

```
    IsCombinatory (interp t (fun _ => botElem)) := by
  rw [theorem_2_4.complete]
  refine ofComb.combinatory _ _ fun _ => ?_
  have hbot : funOf predC zeroC = botElem := by
    rw [predC.app]; exact eq_2_6
  exact hbot ▶ IsCombinatory.app IsCombinatory.pred IsCombinatory.zero

end DomainTheory.DataTypesAsLattices
```

Ycomb_more.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.Ycomb
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.omegaComb
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Lambda.IsCombinatory
public import DomainTheory.DataTypesAsLattices.Lambda.interp
public import DomainTheory.DataTypesAsLattices.Lambda.Yterm
public import DomainTheory.DataTypesAsLattices.Lambda.diagApp_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.theorem_2_4

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace Ycomb

theorem app (u : Pomega) :
    funOf Ycomb u = funOf (omegaComb u) (omegaComb u) :=
  beta (theorem_1_3 diagApp_isScottContinuous omegaComb.isScottContinuous) u

theorem combinatory : IsCombinatory Ycomb := by
  simpa [Yterm.interp] using theorem_2_4.closed Yterm

/-- **Scott 1976, (2.8) unfolding.** `Y(u) = u(Y(u))`. -/
theorem unfold (u : Pomega) :
    funOf Ycomb u = funOf u (funOf Ycomb u) := by
  calc
    funOf Ycomb u = funOf (omegaComb u) (omegaComb u) := Ycomb.app u
    _ = funOf u (funOf (omegaComb u) (omegaComb u)) :=
      omegaComb.app u (omegaComb u)
    _ = funOf u (funOf Ycomb u) := by rw [Ycomb.app]

end Ycomb

end DomainTheory.DataTypesAsLattices
```

2.3.8 The combinator Y

But the result that makes all this model building and combinatory logic mathematically interesting concerns the so-called *paradoxical combinator* defined by the equation:

$$\mathbf{Y} = \lambda u.\,(\lambda x.\,u(x(x)))(\lambda x.\,u(x(x))). \tag{2.8}$$

Yterm.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.lam
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.Term
public import DomainTheory.DataTypesAsLattices.Lambda.interp
public import DomainTheory.DataTypesAsLattices.Lambda.envSet
public import DomainTheory.DataTypesAsLattices.Lambda.omegaComb
public import DomainTheory.DataTypesAsLattices.Lambda.Ycomb

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- Closed term for `Y`. -/
def Yterm : Term :=
  .lam 0 (.app
    (.lam 1 (.app (.var 0) (.app (.var 1) (.var 1))))
    (.lam 1 (.app (.var 0) (.app (.var 1) (.var 1)))))

namespace Yterm

theorem interp :
    _root_.DomainTheory.DataTypesAsLattices.interp Yterm (fun _ => botElem) =
      Ycomb := by
  simp [Yterm, _root_.DomainTheory.DataTypesAsLattices.interp, Ycomb, omegaComb,
    envSet, Function.update]

end Yterm

end DomainTheory.DataTypesAsLattices
```

diagApp_isScottContinuous.lean

```
module

public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Continuous.mem_scottUnion
```

```
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem diagApp_isScottContinuous : IsScottContinuous (fun x => funOf x x) := by
  constructor
  intro x
  have h := theorem_1_3.diag (x := x)
    (f := fun u y => funOf u y)
    (fun y => funOf.isScottContinuous_left y)
    (fun u => funOf.isScottContinuous u)
  ext k
  constructor
  · intro hk
    have : k ∈ ⋃ n, {k | e n ⊆ x ∧ k ∈ funOf (e n) (e n)} := by rwa [← h]
    obtain ⟨n, hn, hkn⟩ := Set.mem_iUnion.mp this
    exact mem_scottUnion.mpr ⟨n, hn, hkn⟩
  · intro hk
    obtain ⟨n, hn, hkn⟩ := mem_scottUnion.mp hk
    have : k ∈ ⋃ n, {k | e n ⊆ x ∧ k ∈ funOf (e n) (e n)} :=
      Set.mem_iUnion.mpr ⟨n, hn, hkn⟩
    rwa [← h] at this

end DomainTheory.DataTypesAsLattices
```

omegaComb.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Lambda.diagApp_isScottContinuous
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices
/-- **Scott 1976, (2.8).** `ω(u) = λ x. u(x(x))`. -/
def omegaComb (u : Pomega) : Pomega :=
  graph (fun x => funOf u (funOf x x))

namespace omegaComb
```

```
theorem app (u x : Pomega) :
    funOf (omegaComb u) x = funOf u (funOf x x) :=
  beta (theorem_1_3 (funOf.isScottContinuous u) diagApp_isScottContinuous) x

theorem isScottContinuous : IsScottContinuous omegaComb :=
  graph_const_isScottContinuous
    (fun u x => funOf u (funOf x x))
    (fun x => funOf.isScottContinuous_left (funOf x x))

end omegaComb

end DomainTheory.DataTypesAsLattices
```

Ycomb.lean

```
module
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.Term
public import DomainTheory.DataTypesAsLattices.Lambda.IsCombinatory
public import DomainTheory.DataTypesAsLattices.Lambda.Comb
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.omegaComb

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.8).** `Y = λ u. ω(u)(ω(u))`. -/
def Ycomb : Pomega :=
  graph (fun u => funOf (omegaComb u) (omegaComb u))

end DomainTheory.DataTypesAsLattices
```

omega_first_recursion.lean

```
module

public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.FixedPoint.fix
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_4
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.theorem_1_2
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.omegaComb
public import DomainTheory.DataTypesAsLattices.Graph.Basic.notation3
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
```

```
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- Scott's core induction proving that `ω(graph f)(ω(graph f))` is the
least fixed point of `f`. -/
theorem omega_first_recursion {f : Pomega → Pomega} (hf : IsScottContinuous f) :
    funOf (omegaComb (graph f)) (omegaComb (graph f)) = fix f := by
  let d := omegaComb (graph f)
  have hd : ∀ x, funOf d x = f (funOf x x) := by
    intro x
    simp [d, omegaComb.app, theorem_1_2.i hf]
  have h? : funOf d d = f (funOf d d) := hd d
  refine subset_antisymm ?_ ((theorem_1_4 hf).2 _ h?.symm)
  intro m hm
  -- Scott's argument: reduce to `e_l ⊆ d → e_l ? e_l ⊆ a` with `a = fix f`.
  let a := fix f
  have ha : f a = a := (theorem_1_4 hf).1
  have key : ∀ l, e l ⊆ d → funOf (e l) (e l) ⊆ a := by
    intro l
    induction l using Nat.strongRecOn with
    | ind l ih =>
      intro hld k hk
      obtain ⟨n, hn, hp⟩ := hk
      have hnlt : n < l := (pair_le_left n k).trans_lt (mem_e_lt hp)
      have hnd : e n ⊆ d := subset_trans hn hld
      have hna : funOf (e n) (e n) ⊆ a := ih n hnlt hnd
      -- `(n,k) ∈ e_l ⊆ d = λx. f(x x)`, so `k ∈ f(e n ? e n)`
      have : k ∈ funOf d (e n) := ⟨n, subset_rfl, hld hp⟩
      have : k ∈ f (funOf (e n) (e n)) := by
        rwa [hd] at this
      have : k ∈ f a := IsScottContinuous.monotone hf hna this
      rwa [ha] at this
  rw [funOf.diag_iUnion] at hm
  obtain ⟨l, hl, hml⟩ := Set.mem_iUnion.mp hm
  exact key l hl hml

end DomainTheory.DataTypesAsLattices
```

2.3.9 Theorem 2.5

THEOREM 2.5 (The first recursion theorem). *If u is the graph of a continuous function f, then $\mathbf{Y}(u) = \mathbf{fix}(f)$, the least fixed point of f.*

There are two points to note here: the fixed point is LAMBDA-definable if f is; and $\mathbf{Y}$ defines a continuous operator. The word "recursion" is attached to the theorem because fixed points are employed to solve recursion equations. It would not be correct to call the fixed-point theorem (Theorem 1.4) the recursion theorem since it only shows that fixed points exist and not how they are definable in a language. The *second* recursion theorem (in Kleene's terminology) is related, but it involves Gödel numbers as introduced in § 3.

From this point on we see no need to distinguish continuous functions from elements of $\mathbf{P}\omega$; a continuous function will be *identified* with its graph. Note that u is a graph iff $u = \lambda x.\, u(x)$, which is equivalent to Theorem 1.2 (iii). For this reason (functions are graphs) we propose the name *Graph Model* for this model of the λ-calculus. (There is more to LAMBDA than just λ, however.)

The identification of functions with graphs entails that the function space of all continuous functions from $\mathbf{P}\omega$ into $\mathbf{P}\omega$ is to be identified (one-to-one) with the subspace

$$\mathbf{FUN} = \{u \mid u = \lambda x.\, u(x)\} \subseteq \mathbf{P}\omega.$$

The identification is *topological* in that the subspace topology agree with the product topology on the function space. This is the topology of pointwise convergence and is closely connected with the lattice structure on the function space which is also defined pointwise (that is, argumentwise). In the notation of λ-abstraction we can express this as the extension of the axiom of extensionality called (ξ^*) in Table 1 of the Introduction. The laws in Table 1 are not the only ones valid in the model, however. We may also note such argumentwise distribution laws as:

$$(2.9) \quad (f \cup g)(x) = f(x) \cup g(x);$$

$$(2.10) \quad (\lambda x.\, \tau) \cup (\lambda x.\, \sigma) = \lambda x.\, (\tau \cup \sigma);$$

$$(2.11) \quad (f \cap g)(x) = f(x) \cap g(x);$$

$$(2.12) \quad (\lambda x.\, \tau) \cap (\lambda x.\, \sigma) = \lambda x.\, (\tau \cap \sigma).$$

In the above f and g must be graphs. It is also true that if $\mathcal{F} \subseteq \mathbf{P}\omega$, then

$$(2.13) \quad \bigcup\{f \mid f \in \mathcal{F}\}(x) = \bigcup\{f(x) \mid f \in \mathcal{F}\},$$

but the same does not hold for $\cap$.

We state now a sequence of minor results which show why some simple functions and constants are LAMBDA-definable.

$$(2.14) \quad \bot = (\lambda x.\, x(x))(\lambda x.\, x(x));$$

$$(2.15) \quad x \cup y = (\lambda z.\, 0) \supset x, y; \quad \text{(Hint: } 0, 1 \in \lambda z.\, 0)$$

$$(2.16) \quad \top = \mathbf{Y}(\lambda x.\, 0 \cup (x + 1));$$

$$(2.17) \quad x \cap y = \mathbf{Y}(\lambda f \lambda x \lambda y.\, x \supset (y \supset 0, \bot), f(x - 1)(y - 1) + 1)(x)(y).$$

The elements $\bot$ and $\top$ are graphs, by the way, and we can characterize them as the only fixed points of the combinator $\mathbf{K}$:

$$(2.18) \quad a = \lambda x.\, a \quad \text{iff} \quad a = \bot \quad \text{or} \quad a = \top.$$

Next we use the notation $\langle x_0, x_1, \ldots, x_{n-1}\rangle$ for the function $\hat{p}$ where $p : \omega \to \mathbf{P}\omega$ is defined by:

$$p(i) = \begin{cases} x_i & \text{if } i < n; \\ \bot & \text{if } i \geq n. \end{cases}$$

$$(2.19)\quad \langle\rangle = \bot$$

$$(2.20)\quad \langle x\rangle = \lambda z.\, z \supset x, \bot$$

$$(2.21)\quad \langle x, y\rangle = \lambda z.\, z \supset x, (z - 1 \supset y, \bot)$$

$$(2.22)\quad \langle x_0, x_1, \ldots, x_n\rangle = \lambda z.\, z \supset x_0, \langle x_1, \ldots, x_n\rangle(z - 1).$$

Obviously we should formalize the subscript notation so that $u_x = u(x)$; then we find:

$$(2.23)\quad \langle x_0, x_1, \ldots, x_n\rangle_i = \begin{cases} x_i & \text{if } i < n; \\ \bot & \text{if } i \geq n. \end{cases}$$

This gives us the method of LAMBDA-defining *finite sequences* (in a quite natural way), and the next step is to consider *infinite sequences*. But these are just the functions $\hat{p}$ where $p : \omega \to \mathbf{P}\omega$ is arbitrary. What we need then is a condition expressible in the language equivalent to saying $u = \hat{p}$ for some p. This is the same as

$$u = \lambda x. \bigcup\{u_i \mid i \in x\}.$$

but the $\cup$- and set-notation is not part of LAMBDA. We are forced into a recursive definition:

$$(2.24)\quad \$ = \mathbf{Y}(\lambda s \lambda u \lambda z.\, z \supset u_0,\ s(\lambda t.\, u_{t+1})(z - 1)).$$

This equation generalizes (2.22) and we have:

$$(2.25)\quad \$(u) = \lambda x. \bigcup\{u_i \mid i \in x\}.$$

Thus the combinator \$"revalues" an element as a distributive function. This suggests introducing theλ-notation for such functions by the equation:

$$(2.26)\quad \lambda n \in \omega.\, \tau = \$(\lambda z.\, \tau[z/n]).$$

With all these conventions LAMBDA-notation becomes very much like ordinary mathematical notation without too much strain.

Suppose that f is any continuous function and $a \in \mathbf{P}\omega$. We can define $p : \omega \to \mathbf{P}\omega$ in the ordinary way by primitive recursion where:

$$p(0) = a;$$

$$p(n + 1) = f(n)(p(n)).$$

The question is: can we give a LAMBDA-definition for $\hat{p}$ (in terms of f and a as constants, say)? The answer is clear, for we can prove:

$$(2.27)\quad \hat{p} = \mathbf{Y}(\lambda u \lambda n \in \omega.\, n \supset a,\ f(n - 1)(u(n - 1))).$$

This already shows that a large part of the definitions of recursion theory can be given in this special language. Of course, simultaneous (primitive) recursions can be transcribed into LAMBDA with the aid of the ordered tuples of (2.22), (2.23) above. But we can go further and connect with partial (and general) recursive functions. We state first a definition.

theorem_2_5.lean

```
module

public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.FixedPoint.fix
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.Ycomb
public import DomainTheory.DataTypesAsLattices.Lambda.omega_first_recursion
public import DomainTheory.DataTypesAsLattices.Lambda.Ycomb_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, Theorem 2.5 (The first recursion theorem).**
If `u` is the graph of a continuous `f`, then `Y(u) = fix(f)`. -/
theorem theorem_2_5 {f : Pomega → Pomega} (hf : IsScottContinuous f) :
    funOf Ycomb (graph f) = fix f := by
  rw [Ycomb.app]
  exact omega_first_recursion hf

end DomainTheory.DataTypesAsLattices
```

dollarC.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.seqFun
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Lambda.IsCombinatory
public import DomainTheory.DataTypesAsLattices.Lambda.interp
public import DomainTheory.DataTypesAsLattices.Lambda.Ycomb
public import DomainTheory.DataTypesAsLattices.Lambda.dollarStep
public import DomainTheory.DataTypesAsLattices.Lambda.dollarStepTerm
public import DomainTheory.DataTypesAsLattices.Lambda.termDollarStepBody
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.termDollarStep
public import DomainTheory.DataTypesAsLattices.Lambda.theorem_2_5
public import DomainTheory.DataTypesAsLattices.Lambda.theorem_2_4
public import DomainTheory.DataTypesAsLattices.FixedPoint.fix
```

```
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_4
public import DomainTheory.DataTypesAsLattices.Lambda.Ycomb_more
public import DomainTheory.DataTypesAsLattices.Lambda.seqFun_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices
def dollarC : Pomega :=
  graph (fun u => graph (fun x => seqFun u x))

namespace dollarC
theorem app (u : Pomega) :
    funOf dollarC u = graph (fun x => seqFun u x) :=
  beta (graph_const_isScottContinuous (fun u x => seqFun u x)
    (fun x => seqFun.left_isScottContinuous x)) u

theorem eq_fix_termDollarStep : dollarC = fix termDollarStep := by
  have hfix : termDollarStep (fix termDollarStep) = fix termDollarStep :=
    (theorem_1_4 termDollarStep.isScottContinuous).1
  have hbody : ∀ u z, termDollarStepBody (fix termDollarStep) u z = seqFun u z := by
    intro u z
    calc termDollarStepBody (fix termDollarStep) u z
        = funOf (funOf (termDollarStep (fix termDollarStep)) u) z :=
          (termDollarStep.app2 _ _ _).symm
      _ = funOf (funOf (fix termDollarStep) u) z := by rw [hfix]
      _ = seqFun u z := termDollarStep.fix_app u z
  have : termDollarStep (fix termDollarStep) =
      graph (fun u => graph (fun z => seqFun u z)) := by
    apply congrArg graph
    funext u
    apply congrArg graph
    funext z
    exact hbody u z
  exact this.symm.trans hfix

/-- **Scott 1976, (2.24).** `$ is combinatory: $ = Y(λs λu λz. z ? u₀, s(λt. u_{t+1})(z
    ?1))`. -/
theorem combinatory : IsCombinatory dollarC := by
  have hY : funOf Ycomb (interp dollarStepTerm (fun _ => botElem)) = dollarC := by
    rw [dollarStepTerm.interp, theorem_2_5 termDollarStep.isScottContinuous,
      dollarC.eq_fix_termDollarStep]
  exact hY ▸ IsCombinatory.app Ycomb.combinatory (theorem_2_4.closed dollarStepTerm)

end dollarC

end DomainTheory.DataTypesAsLattices
```

eq_2_24.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.Ycomb
public import DomainTheory.DataTypesAsLattices.Lambda.seqFun
```

```
public import DomainTheory.DataTypesAsLattices.Lambda.dollarC
public import DomainTheory.DataTypesAsLattices.Lambda.seqShift
public import DomainTheory.DataTypesAsLattices.Lambda.dollarStepBody
public import DomainTheory.DataTypesAsLattices.Lambda.dollarStep
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import Mathlib.Data.Set.Lattice.Indexed
public import DomainTheory.DataTypesAsLattices.FixedPoint.fix
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.succSet
public import DomainTheory.DataTypesAsLattices.Lambda.predSet
public import DomainTheory.DataTypesAsLattices.Lambda.theorem_2_5
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_4
public import DomainTheory.DataTypesAsLattices.Lambda.seqShift_more
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.24).** `seq(u)(x) = x ? u_0, seq(λt. u_{t+1})(x?1)`. -/
theorem eq_2_24 (u x : Pomega) :
    seqFun u x =
      condSet x (funOf u (ofNat 0))
        (seqFun (seqShift u) (predSet x)) := by
  ext k
  constructor
  · intro hk
    obtain ⟨i, hi, hki⟩ := Set.mem_iUnion₂.mp hk
    cases i with
    | zero => exact Or.inl ⟨hki, hi⟩
    | succ i =>
      refine Or.inr ⟨?_, i, hi⟩
      refine Set.mem_iUnion₂.mpr ⟨i, hi, ?_⟩
      have : funOf (seqShift u) (ofNat i) = funOf u (ofNat (i + 1)) := by
        rw [seqShift.app]
        ext m
        constructor
        · intro hm
          obtain ⟨j, hj, hmj⟩ := Set.mem_iUnion₂.mp hm
          simp [succSet, ofNat] at hj; subst hj; exact hmj
        · intro hm
          exact Set.mem_iUnion₂.mpr ⟨i + 1, ⟨i, rfl, rfl⟩, hm⟩
      simpa [this] using hki
  · intro hk
    rcases hk with ⟨hk0, h0x⟩ | ⟨hpos, t, ht⟩
    · exact Set.mem_iUnion₂.mpr ⟨0, h0x, hk0⟩
    · obtain ⟨i, hi, hki⟩ := Set.mem_iUnion₂.mp hpos
      have : funOf (seqShift u) (ofNat i) = funOf u (ofNat (i + 1)) := by
        rw [seqShift.app]
        ext m
        constructor
        · intro hm
          obtain ⟨j, hj, hmj⟩ := Set.mem_iUnion₂.mp hm
          simp [succSet, ofNat] at hj; subst hj; exact hmj
        · intro hm
          exact Set.mem_iUnion₂.mpr ⟨i + 1, ⟨i, rfl, rfl⟩, hm⟩
```

```
      refine Set.mem_iUnion₂.mpr ⟨i + 1, hi, ?_⟩
      simpa [this] using hki

namespace eq_2_24

/-- **Scott 1976, (2.24).** `$ = Y(λs λu λz. z ? u₀, s(λt. u_{t+1})(z?1))`. -/
theorem Y : dollarC = fix dollarStep := by
  have hfix : dollarStep (fix dollarStep) = fix dollarStep :=
    (theorem_1_4 dollarStep.isScottContinuous).1
  have hbody : ∀ u z, dollarStepBody (fix dollarStep) u z = seqFun u z := by
    intro u z
    calc dollarStepBody (fix dollarStep) u z
        = funOf (funOf (dollarStep (fix dollarStep)) u) z :=
          (dollarStep.app2 _ _ _).symm
      _ = funOf (funOf (fix dollarStep) u) z := by rw [hfix]
      _ = seqFun u z := dollarStep.fix_app u z
  have : dollarStep (fix dollarStep) =
      graph (fun u => graph (fun z => seqFun u z)) := by
    apply congrArg graph
    funext u
    apply congrArg graph
    funext z
    exact hbody u z
  exact this.symm.trans hfix

/-- **Scott 1976, (2.24).** `$ = Y(step)` as a combinator application. -/
theorem Ycomb : funOf Ycomb (graph dollarStep) = dollarC := by
  rw [theorem_2_5 dollarStep.isScottContinuous, eq_2_24.Y]

end eq_2_24

end DomainTheory.DataTypesAsLattices
```

2.3.10 The graph model

From this point on we see no need to distinguish continuous functions from elements of $\mathbf{P}\omega$; a continuous function will be *identified* with its graph. Note that u is a graph iff $u = \lambda x.\, u(x)$, which is equivalent to Theorem 1.2 (iii). For this reason (functions are graphs) we propose the name *Graph Model* for this model of the λ-calculus. (There is more to LAMBDA than just λ, however.)

The identification of functions with graphs entails that the function space of all continuous functions from $\mathbf{P}\omega$ into $\mathbf{P}\omega$ is to be identified (one-to-one) with the subspace

$$\mathbf{FUN} = \{u \mid u = \lambda x.\, u(x)\} \subseteq \mathbf{P}\omega.$$

The identification is *topological* in that the subspace topology agree with the product topology on the function space. This is the topology of pointwise convergence and is closely connected with the lattice structure on the function space which is also defined pointwise (that is, argumentwise). In the notation of λ-abstraction we can express this as the extension of the axiom of extensionality called (ξ^*) in Table 1 of the Introduction. The laws in Table 1 are not the only ones valid in the model, however. We may also note such argumentwise distribution laws as:

$$(2.9) \quad (f \cup g)(x) = f(x) \cup g(x);$$

$$(2.10)\quad (\lambda x.\,\tau) \cup (\lambda x.\,\sigma) = \lambda x.\,(\tau \cup \sigma);$$

$$(2.11)\quad (f \cap g)(x) = f(x) \cap g(x);$$

$$(2.12)\quad (\lambda x.\,\tau) \cap (\lambda x.\,\sigma) = \lambda x.\,(\tau \cap \sigma).$$

In the above f and g must be graphs. It is also true that if $\mathcal{F} \subseteq \mathbf{P}\omega$, then

$$(2.13)\quad \bigcup\{f \mid f \in \mathcal{F}\}(x) = \bigcup\{f(x) \mid f \in \mathcal{F}\},$$

but the same does not hold for $\cap$.

eq_2_12.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.12).** Abstraction distributes over intersection. -/
theorem eq_2_12 (τ σ : Pomega → Pomega) :
    graph τ ∩ graph σ = graph (fun x => τ x ∩ σ x) := by
  ext p
  constructor
  · intro ⟨hτ, hσ⟩
    rcases hτ with ⟨n, m, hpτ, hmτ⟩
    rcases hσ with ⟨n', m', hpσ, hmσ⟩
    have hp' : pair n m = pair n' m' := hpτ.symm.trans hpσ
    obtain ⟨rfl, rfl⟩ := pair_inj hp'
    exact ⟨n, m, hpτ, hmτ, hmσ⟩
  · intro h
    rcases h with ⟨n, m, hp, hmτ, hmσ⟩
    exact ⟨⟨n, m, hp, hmτ⟩, ⟨n, m, hp, hmσ⟩⟩

end DomainTheory.DataTypesAsLattices
```

eq_2_10.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices
```

```
/-- **Scott 1976, (2.10).** Abstraction distributes over union. -/
theorem eq_2_10 (τ σ : Pomega → Pomega) :
    graph τ ∪ graph σ = graph (fun x => τ x ∪ σ x) := by
  ext p
  constructor
  · intro hp
    rcases hp with h | h
    · rcases h with ⟨n, m, hp, hm⟩
      exact ⟨n, m, hp, Or.inl hm⟩
    · rcases h with ⟨n, m, hp, hm⟩
      exact ⟨n, m, hp, Or.inr hm⟩
  · intro h
    rcases h with ⟨n, m, hp, hm⟩
    rcases hm with hm | hm
    · exact Or.inl ⟨n, m, hp, hm⟩
    · exact Or.inr ⟨n, m, hp, hm⟩

end DomainTheory.DataTypesAsLattices
```

xi_star.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, Table 1 (?*).** Abstraction is monotone. -/
theorem xi_star {τ σ : Pomega → Pomega} (h : ∀ x, τ x ⊆ σ x) :
    graph τ ⊆ graph σ := by
  intro p hp
  rcases hp with ⟨n, m, rfl, hm⟩
  exact ⟨n, m, rfl, h _ hm⟩

end DomainTheory.DataTypesAsLattices
```

2.3.11 Elementary definitions

We state now a sequence of minor results which show why some simple functions and constants are LAMBDA-definable.

$$(2.14) \quad \bot = (\lambda x.\, x(x))(\lambda x.\, x(x));$$

$$(2.15) \quad x \cup y = (\lambda z.\, 0) \supset x, y; \quad \text{(Hint: } 0, 1 \in \lambda z.\, 0)$$

$$(2.16) \quad \top = \mathbf{Y}(\lambda x.\, 0 \cup (x+1));$$

$$(2.17)\quad x \cap y = \mathbf{Y}(\lambda f \lambda x \lambda y.\, x \supset (y \supset 0, \bot), f(x-1)(y-1)+1)(x)(y).$$

The elements $\bot$ and $\top$ are graphs, by the way, and we can characterize them as the only fixed points of the combinator $\mathbf{K}$:

$$(2.18)\quad a = \lambda x.\, a \quad \text{iff} \quad a = \bot \quad \text{or} \quad a = \top.$$

eq_2_18.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.pair_eq_right
public import DomainTheory.DataTypesAsLattices.Lambda.exists_least_not_mem
public import DomainTheory.DataTypesAsLattices.Lambda.exists_least_mem

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace eq_2_18

/-- **Scott 1976, (2.18).** Constants as graphs of constant maps. -/
theorem bot : botElem = graph (fun _ => botElem) := by
  ext p
  constructor
  · intro hp; exact hp.elim
  · intro hp
    rcases hp with ⟨_, _, _, hm⟩
    simp [botElem] at hm

theorem top : topElem = graph (fun _ => topElem) := by
  ext p
  constructor
  · intro _
    obtain ⟨n, m, rfl⟩ := exists_pair p
    exact ⟨n, m, rfl, Set.mem_univ m⟩
  · intro; trivial

end eq_2_18

/-- **Scott 1976, (2.18).** `a = λx. a` iff `a = ⊥` or `a = ⊤`. -/
theorem eq_2_18 (a : Pomega) :
    a = graph (fun _ => a) ↔ a = botElem ∨ a = topElem := by
  constructor
  · intro ha
    by_cases htop : a = topElem
    · exact Or.inr htop
    · have hex : ∃ k, k ∈ a := by
        by_contra h
        apply htop
        ext k
```

```
        constructor
        · intro; trivial
        · intro
          exact not_not.mp (not_exists.mp h k)
      obtain ⟨k, hk, hmin⟩ := exists_least_not_mem hex
      obtain ⟨n, m, hnm⟩ := exists_pair k
      have hm : m ∈ a := by
        intro hma
        have : pair n m ∈ a :=
          ha.symm ▸ (⟨n, m, rfl, hma⟩ : pair n m ∈ graph (fun _ => a))
        exact hk (by rwa [hnm] at this)
      have : m = k :=
        le_antisymm (by rw [← hnm]; exact pair_le_right n m)
          (le_of_not_gt fun hlt => hm (hmin m hlt))
      have hnmk : pair n k = k := by simpa [this] using hnm
      have ⟨_, hk0⟩ := pair_eq_right hnmk
      have h0 : 0 ∈ a := by simpa [hk0] using hk
      by_cases hbot : a = botElem
      · exact Or.inl hbot
      · obtain ⟨l, hl, hlmin⟩ := exists_least_mem (Set.nonempty_iff_ne_empty.mpr hbot)
        obtain ⟨i, j, hij⟩ := exists_pair l
        have hj : j ∈ a := by
          have : pair i j ∈ graph (fun _ => a) :=
            ha ▸ (by rwa [hij] : pair i j ∈ a)
          rcases this with ⟨i', j', hp, hj'⟩
          obtain ⟨rfl, rfl⟩ := pair_inj hp
          exact hj'
        have : j = l :=
          le_antisymm (by rw [← hij]; exact pair_le_right i j)
            (le_of_not_gt fun hlt => hlmin j hlt hj)
        have hil : pair i l = l := by simpa [this] using hij
        have ⟨_, hl0⟩ := pair_eq_right hil
        exact (h0 (by simpa [hl0] using hl)).elim
  · intro h
    rcases h with rfl | rfl
    · exact eq_2_18.bot
    · exact eq_2_18.top

end DomainTheory.DataTypesAsLattices
```

pack4.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Pomega.pair

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable
```

```
open Nat.Partrec (Code)

open Nat.Partrec.Code

def pack4 (n0 n1 n2 m : ℕ) : ℕ :=
  Nat.pair n0 (Nat.pair n1 (Nat.pair n2 m))

namespace pack4

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

def proj1 (t : ℕ) : ℕ := (Nat.unpair t).1

def proj2 (t : ℕ) : ℕ := (Nat.unpair (Nat.unpair t).2).1

def proj3 (t : ℕ) : ℕ := (Nat.unpair (Nat.unpair (Nat.unpair t).2).2).1

def proj4 (t : ℕ) : ℕ := (Nat.unpair (Nat.unpair (Nat.unpair t).2).2).2

end pack4

end DomainTheory.DataTypesAsLattices
```

2.3.12 Sequences

Next we use the notation $\langle x_0, x_1, \ldots, x_{n-1}\rangle$ for the function $\hat{p}$ where $p : \omega \to \mathbf{P}\omega$ is defined by:

$$p(i) = \begin{cases} x_i & \text{if } i < n; \\ \bot & \text{if } i \geq n. \end{cases}$$

$$\langle\rangle = \bot \tag{2.19}$$

$$\langle x\rangle = \lambda z.\, z \supset x, \bot \tag{2.20}$$

$$\langle x, y\rangle = \lambda z.\, z \supset x, (z - 1 \supset y, \bot) \tag{2.21}$$

$$\langle x_0, x_1, \ldots, x_n\rangle = \lambda z.\, z \supset x_0, \langle x_1, \ldots, x_n\rangle(z - 1). \tag{2.22}$$

Obviously we should formalize the subscript notation so that $u_x = u(x)$; then we find:

$$\langle x_0, x_1, \ldots, x_n\rangle_i = \begin{cases} x_i & \text{if } i < n; \\ \bot & \text{if } i \geq n. \end{cases} \tag{2.23}$$

This gives us the method of LAMBDA-defining *finite sequences* (in a quite natural way), and the next step is to consider *infinite sequences*. But these are just the functions $\hat{p}$ where $p : \omega \to \mathbf{P}\omega$ is arbitrary. What we need then is a condition expressible in the language equivalent to saying $u = \hat{p}$ for some p. This is the same as

$$u = \lambda x. \bigcup \{u_i \mid i \in x\}.$$

mem_seqFunLt.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.seqFunLt
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import Mathlib.Data.Set.Lattice.Indexed
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem mem_seqFunLt {u x : Pomega} {n k : ℕ} :
    k ∈ seqFunLt u x n ↔ ∃ i ∈ x, i < n ∧ k ∈ funOf u (ofNat i) := by
  constructor
  · intro hk
    obtain ⟨i, hi, hrest⟩ := Set.mem_iUnion₂.mp hk
    obtain ⟨hlt, hki⟩ := Set.mem_iUnion.mp hrest
    exact ⟨i, hi, hlt, hki⟩
  · intro ⟨i, hi, hlt, hki⟩
    exact Set.mem_iUnion₂.mpr ⟨i, hi, Set.mem_iUnion.mpr ⟨hlt, hki⟩⟩

end DomainTheory.DataTypesAsLattices
```

seq2.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.predSet
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Continuous.mem_scottUnion
public import DomainTheory.DataTypesAsLattices.Continuous.scottUnion
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import Mathlib.Data.Set.Lattice.Indexed
public import DomainTheory.DataTypesAsLattices.Lambda.predSet_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def seq2 (x y : Pomega) : Pomega :=
  graph (fun z => condSet z x (condSet (predSet z) y botElem))
```

```
namespace seq2

theorem body_isScottContinuous (x y : Pomega) :
    IsScottContinuous (fun z => condSet z x (condSet (predSet z) y botElem)) := by
  constructor
  intro z
  have H := theorem_1_3.diag (x := z)
    (f := fun z w => condSet z x (condSet (predSet w) y botElem))
    (fun _w => condSet.isScottContinuous_left x _)
    (fun _z =>
      theorem_1_3 (condSet.isScottContinuous_right _z x)
        (theorem_1_3 (condSet.isScottContinuous_left y botElem)
          predSet.isScottContinuous))
  ext k
  constructor
  · intro hk
    have : k ∈ ⋃ n, {k | e n ⊆ z ∧
        k ∈ condSet (e n) x (condSet (predSet (e n)) y botElem)} := by
      rwa [← H]
    obtain ⟨n, hn, hkn⟩ := Set.mem_iUnion.mp this
    exact mem_scottUnion.mpr ⟨n, hn, hkn⟩
  · intro hk
    obtain ⟨n, hn, hkn⟩ := mem_scottUnion.mp hk
    have : k ∈ ⋃ n, {k | e n ⊆ z ∧
        k ∈ condSet (e n) x (condSet (predSet (e n)) y botElem)} :=
      Set.mem_iUnion.mpr ⟨n, hn, hkn⟩
    rwa [← H] at this

theorem app (x y z : Pomega) :
    funOf (seq2 x y) z = condSet z x (condSet (predSet z) y botElem) :=
  beta (seq2.body_isScottContinuous x y) z

theorem one (x y : Pomega) : funOf (seq2 x y) (ofNat 1) = y := by
  rw [app]
  ext n
  simp [condSet, predSet, ofNat, botElem]

theorem zero (x y : Pomega) : funOf (seq2 x y) (ofNat 0) = x := by
  rw [app]
  ext n
  simp [condSet, predSet, ofNat, botElem]

theorem tag_ne (x y : Pomega) :
    seq2 (ofNat 0) x = seq2 (ofNat 1) y := by
  intro h
  have hfun := congrArg (fun w => funOf w (ofNat 0)) h
  have hx : (0 : ℕ) ∈ funOf (seq2 (ofNat 0) x) (ofNat 0) :=
    ⟨1, fun k hk =>
      match k, hk with
      | 0, _ => rfl
      | k + 1, hk =>
        False.elim (Bool.false_ne_true
          (((Nat.testBit_succ 1 k).trans (Nat.zero_testBit k)).symm.trans hk)),
      ⟨1, 0, rfl, Or.inl ⟨rfl, rfl⟩⟩⟩
  have hy : (0 : ℕ) ∈ funOf (seq2 (ofNat 1) y) (ofNat 0) := by
```

```
    intro ⟨k, hk, hp⟩
    rcases hp with ⟨k', m, heq, hm⟩
    obtain ⟨rfl, rfl⟩ := pair_inj heq
    rcases hm with ⟨h1, _⟩ | ⟨_, ⟨t, ht⟩⟩
    · exact Nat.zero_ne_one h1
    · exact Nat.succ_ne_zero t (hk ht)
  exact hy (hfun ▸ hx)

end seq2

end DomainTheory.DataTypesAsLattices
```

eq_2_23_ge.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.seq2
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.predSet
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem eq_2_23_ge (x y : Pomega) (n : ℕ) :
    funOf (seq2 x y) (ofNat (n + 2)) = botElem := by
  rw [seq2.app]
  ext k
  simp [condSet, predSet, ofNat, botElem]

end DomainTheory.DataTypesAsLattices
```

eq_2_23_one.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.seq2
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem eq_2_23_one (x y : Pomega) : funOf (seq2 x y) (ofNat 1) = y :=
  seq2.one x y

end DomainTheory.DataTypesAsLattices
```

eq_2_23_zero.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.seq2
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.23).** Projection of a binary sequence. -/
theorem eq_2_23_zero (x y : Pomega) : funOf (seq2 x y) (ofNat 0) = x :=
  seq2.zero x y

end DomainTheory.DataTypesAsLattices
```

seqCons.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.predSet
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.id_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
public import DomainTheory.DataTypesAsLattices.Lambda.predSet_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices
/-- **Scott 1976, (2.22).** Longer sequences by cons. -/
def seqCons (x xs : Pomega) : Pomega :=
  graph (fun z => condSet z x (funOf xs (predSet z)))

namespace seqCons

theorem body (x xs : Pomega) :
    IsScottContinuous (fun z => condSet z x (funOf xs (predSet z))) :=
  theorem_1_3.tuple
    (fun _ => condSet.isScottContinuous_left x _)
    (fun z => condSet.isScottContinuous_right z x)
    id_isScottContinuous
    (theorem_1_3 (funOf.isScottContinuous xs) predSet.isScottContinuous)

end seqCons
```

```
end DomainTheory.DataTypesAsLattices
```

eq_2_22.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.seqCons
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.predSet
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem eq_2_22 (x xs z : Pomega) :
    funOf (seqCons x xs) z = condSet z x (funOf xs (predSet z)) :=
  beta (seqCons.body x xs) z

end DomainTheory.DataTypesAsLattices
```

seqFunLt_more.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.seqFunLt
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.seqShift
public import DomainTheory.DataTypesAsLattices.Lambda.termShift
public import DomainTheory.DataTypesAsLattices.Lambda.mem_seqFunLt
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.predSet
public import DomainTheory.DataTypesAsLattices.Lambda.seqShift_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace seqFunLt

theorem succ (u z : Pomega) (n : ℕ) :
    condSet z (funOf u (ofNat 0)) (seqFunLt (seqShift u) (predSet z) n) =
      seqFunLt u z (n + 1) := by
  ext k
  constructor
  · intro hk
    rcases hk with ⟨hk0, h0z⟩ | ⟨hpos, t, ht⟩
    · exact (mem_seqFunLt).mpr ⟨0, h0z, Nat.succ_pos n, hk0⟩
    · obtain ⟨i, hi, hlt, hki⟩ := (mem_seqFunLt).mp hpos
      refine (mem_seqFunLt).mpr ⟨i + 1, hi, Nat.succ_lt_succ hlt, ?_⟩
      simpa [seqShift.ofNat] using hki
```

```
  · intro hk
    obtain ⟨i, hi, hlt, hki⟩ := (mem_seqFunLt).mp hk
    cases i with
    | zero => exact Or.inl ⟨hki, hi⟩
    | succ i =>
      refine Or.inr ⟨?_, i, hi⟩
      refine (mem_seqFunLt).mpr ⟨i, hi, Nat.lt_of_succ_lt_succ hlt, ?_⟩
      simpa [seqShift.ofNat] using hki

theorem term (u z : Pomega) (n : ℕ) :
    condSet z (funOf u (ofNat 0)) (seqFunLt (termShift u) (predSet z) n) =
      seqFunLt u z (n + 1) := by
  ext k
  constructor
  · intro hk
    rcases hk with ⟨hk0, h0z⟩ | ⟨hpos, t, ht⟩
    · exact (mem_seqFunLt).mpr ⟨0, h0z, Nat.succ_pos n, hk0⟩
    · obtain ⟨i, hi, hlt, hki⟩ := (mem_seqFunLt).mp hpos
      refine (mem_seqFunLt).mpr ⟨i + 1, hi, Nat.succ_lt_succ hlt, ?_⟩
      simpa [termShift.ofNat] using hki
  · intro hk
    obtain ⟨i, hi, hlt, hki⟩ := (mem_seqFunLt).mp hk
    cases i with
    | zero => exact Or.inl ⟨hki, hi⟩
    | succ i =>
      refine Or.inr ⟨?_, i, hi⟩
      refine (mem_seqFunLt).mpr ⟨i, hi, Nat.lt_of_succ_lt_succ hlt, ?_⟩
      simpa [termShift.ofNat] using hki

theorem zero (u z : Pomega) : seqFunLt u z 0 = botElem := by
  ext k
  simp [mem_seqFunLt, botElem]

end seqFunLt

end DomainTheory.DataTypesAsLattices
```

seqFun_more.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.seqFun
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.seqFunLt
public import DomainTheory.DataTypesAsLattices.Lambda.mem_seqFunLt
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.mem_scottUnion
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Continuous.scottUnion
```

```
public import Mathlib.Data.Set.Lattice.Indexed

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace seqFun

theorem eq_iUnion_lt (u z : Pomega) :
    seqFun u z = ⋃ n, seqFunLt u z n := by
  ext k
  constructor
  · intro hk
    obtain ⟨i, hi, hki⟩ := Set.mem_iUnion₂.mp hk
    exact Set.mem_iUnion.mpr ⟨i + 1, (mem_seqFunLt).mpr ⟨i, hi, Nat.lt_succ_self i, hki
    ⟩⟩
  · intro hk
    obtain ⟨n, hn⟩ := Set.mem_iUnion.mp hk
    obtain ⟨i, hi, _, hki⟩ := (mem_seqFunLt).mp hn
    exact Set.mem_iUnion₂.mpr ⟨i, hi, hki⟩

theorem left_isScottContinuous (x : Pomega) :
    IsScottContinuous (fun u => seqFun u x) := by
  constructor
  intro u
  ext k
  constructor
  · intro hk
    obtain ⟨i, hi, hki⟩ := Set.mem_iUnion₂.mp hk
    have : k ∈ scottUnion (fun v => funOf v (ofNat i)) u := by
      rw [← (funOf.isScottContinuous_left (ofNat i)).out u]; exact hki
    obtain ⟨n, hn, hkn⟩ := mem_scottUnion.mp this
    exact mem_scottUnion.mpr ⟨n, hn, Set.mem_iUnion₂.mpr ⟨i, hi, hkn⟩⟩
  · intro hk
    obtain ⟨n, hn, hkn⟩ := mem_scottUnion.mp hk
    obtain ⟨i, hi, hki⟩ := Set.mem_iUnion₂.mp hkn
    exact Set.mem_iUnion₂.mpr ⟨i, hi,
      IsScottContinuous.monotone (funOf.isScottContinuous_left (ofNat i)) hn hki⟩

theorem right_isScottContinuous (u : Pomega) :
    IsScottContinuous (fun x => seqFun u x) := by
  constructor
  intro x
  ext k
  constructor
  · intro hk
    obtain ⟨i, hi, hki⟩ := Set.mem_iUnion₂.mp hk
    exact mem_scottUnion.mpr ⟨2 ^ i, by simp [e_pow2, Set.singleton_subset_iff, hi],
      Set.mem_iUnion₂.mpr ⟨i, by simp [e_pow2], hki⟩⟩
  · intro hk
    obtain ⟨n, hn, hkn⟩ := mem_scottUnion.mp hk
    obtain ⟨i, hi, hki⟩ := Set.mem_iUnion₂.mp hkn
    exact Set.mem_iUnion₂.mpr ⟨i, hn hi, hki⟩

end seqFun
```

```
end DomainTheory.DataTypesAsLattices
```

seqShift_more.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.seqShift
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.seqFun
public import DomainTheory.DataTypesAsLattices.Lambda.succSet
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import Mathlib.Data.Set.Lattice.Indexed
public import DomainTheory.DataTypesAsLattices.Lambda.seqFun_more
public import DomainTheory.DataTypesAsLattices.Lambda.succSet_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace seqShift

theorem isScottContinuous (u : Pomega) :
    IsScottContinuous (fun z => seqFun u (succSet z)) :=
  theorem_1_3 (seqFun.right_isScottContinuous u) succSet.isScottContinuous

theorem app (u z : Pomega) :
    funOf (seqShift u) z = seqFun u (succSet z) :=
  beta (seqShift.isScottContinuous u) z

theorem left_isScottContinuous : IsScottContinuous seqShift :=
  graph_const_isScottContinuous (fun u z => seqFun u (succSet z))
    (fun z => seqFun.left_isScottContinuous (succSet z))

theorem ofNat (u : Pomega) (i : ℕ) :
    funOf (seqShift u) (ofNat i) = funOf u (ofNat (i + 1)) := by
  rw [seqShift.app]
  ext m
  constructor
  · intro hm
    obtain ⟨j, hj, hmj⟩ := Set.mem_iUnion₂.mp hm
    simp [succSet] at hj; subst hj; exact hmj
  · intro hm
    exact Set.mem_iUnion₂.mpr ⟨i + 1, ⟨i, rfl, rfl⟩, hm⟩

end seqShift

end DomainTheory.DataTypesAsLattices
```

seqFunLt.lean

```
module
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.Term
public import DomainTheory.DataTypesAsLattices.Lambda.IsCombinatory
public import DomainTheory.DataTypesAsLattices.Lambda.Comb
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices
/-- Finite stage of the `$` iteration: `⋃ { uᵢ | i ∈ x, i < n }`. -/
def seqFunLt (u x : Pomega) (n : ℕ) : Pomega :=
  ⋃ i ∈ x, ⋃ _ : i < n, funOf u (ofNat i)

end DomainTheory.DataTypesAsLattices
```

seqShift.lean

```
module
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.Term
public import DomainTheory.DataTypesAsLattices.Lambda.IsCombinatory
public import DomainTheory.DataTypesAsLattices.Lambda.Comb
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.seqFun

@[expose] public section

namespace DomainTheory.DataTypesAsLattices
/-- Shift of a sequence: `(λt. u_{t+1})`. -/
def seqShift (u : Pomega) : Pomega :=
  graph (fun z => seqFun u (succSet z))

end DomainTheory.DataTypesAsLattices
```

2.3.13 Distributive functions

The $\cup$- and set-notation is not part of LAMBDA. We are forced into a recursive definition:

$$(2.24) \quad \$ = \mathbf{Y}(\lambda s \lambda u \lambda z.\, z \supset u_0,\, s(\lambda t.\, u_{t+1})(z-1)).$$

This equation generalizes (2.22) and we have:

$$(2.25) \quad \$(u) = \lambda x.\, \bigcup \{u_i \mid i \in x\}.$$

Thus the combinator \$"revalues" an element as a distributive function. This suggests introducing theλ-notation for such functions by the equation:

$$(2.26) \quad \lambda n \in \omega.\, \tau = \$(\lambda z.\, \tau[z/n]).$$

With all these conventions LAMBDA-notation becomes very much like ordinary mathematical notation without too much strain.

dollarStepTerm.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.lam
public import DomainTheory.DataTypesAsLattices.Lambda.Term
public import DomainTheory.DataTypesAsLattices.Enumeration.Avoids
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.envSet
public import DomainTheory.DataTypesAsLattices.Lambda.interp
public import DomainTheory.DataTypesAsLattices.Lambda.termShift
public import DomainTheory.DataTypesAsLattices.Lambda.termDollarStepBody
public import DomainTheory.DataTypesAsLattices.Lambda.termDollarStep
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.succSet

@[expose] public section

namespace DomainTheory.DataTypesAsLattices
/-- Paper combinator for `$`: `λs λu λz. z ? u(0), s(λt. u(t+1))(z?1)`. -/
def dollarStepTerm : Term :=
  .lam 0 (.lam 1 (.lam 2
    (.cond (.var 2)
      (.app (.var 1) .zero)
      (.app (.app (.var 0) (.lam 3 (.app (.var 1) (.succ (.var 3)))))
        (.pred (.var 2))))))

namespace dollarStepTerm

theorem interp :
    interp dollarStepTerm (fun _ => botElem) = graph termDollarStep := by
  unfold dollarStepTerm termDollarStep termDollarStepBody termShift DomainTheory.
    DataTypesAsLattices.interp
  apply congrArg graph
  funext s
  simp only[envSet]
  apply congrArg graph
  funext u
  simp only[envSet]
  apply congrArg graph
  funext z
  simp[envSet, condSet, succSet]
  rfl

end dollarStepTerm

end DomainTheory.DataTypesAsLattices
```

dollarStepBody.lean

```
module
```

```
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.seqShift
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.predSet
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.id_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
public import DomainTheory.DataTypesAsLattices.Lambda.predSet_more
public import DomainTheory.DataTypesAsLattices.Lambda.seqShift_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices
/-- **Scott 1976, (2.24).** The step of `$ = Y(λs λu λz. z ? u₀, s(λt. u_{t+1})(z?1))`.
    -/
def dollarStepBody (s u z : Pomega) : Pomega :=
  condSet z (funOf u (ofNat 0))
    (funOf (funOf s (seqShift u)) (predSet z))

namespace dollarStepBody

theorem isScottContinuous_s (u z : Pomega) :
    IsScottContinuous (fun s => dollarStepBody s u z) := by
  have hinner : IsScottContinuous
      (fun s => funOf (funOf s (seqShift u)) (predSet z)) :=
    theorem_1_3 (f := fun w => funOf w (predSet z))
      (g := fun s => funOf s (seqShift u))
      (funOf.isScottContinuous_left (predSet z))
      (funOf.isScottContinuous_left (seqShift u))
  exact theorem_1_3 (condSet.isScottContinuous_right z (funOf u (ofNat 0))) hinner

theorem isScottContinuous_u (s z : Pomega) :
    IsScottContinuous (fun u => dollarStepBody s u z) := by
  have helse : IsScottContinuous
      (fun u => funOf (funOf s (seqShift u)) (predSet z)) :=
    theorem_1_3 (f := fun w => funOf w (predSet z))
      (g := fun u => funOf s (seqShift u))
      (funOf.isScottContinuous_left (predSet z))
      (theorem_1_3 (funOf.isScottContinuous s) seqShift.left_isScottContinuous)
  exact theorem_1_3.tuple
    (fun y => condSet.isScottContinuous_mid z y)
    (fun x => condSet.isScottContinuous_right z x)
    (funOf.isScottContinuous_left (ofNat 0)) helse

theorem isScottContinuous_z (s u : Pomega) :
    IsScottContinuous (fun z => dollarStepBody s u z) :=
  theorem_1_3.tuple
    (fun y => condSet.isScottContinuous_left (funOf u (ofNat 0)) y)
    (fun z => condSet.isScottContinuous_right z (funOf u (ofNat 0)))
    id_isScottContinuous
    (theorem_1_3 (funOf.isScottContinuous (funOf s (seqShift u)))
      predSet.isScottContinuous)
```

```
end dollarStepBody

end DomainTheory.DataTypesAsLattices
```

dollarStep.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.dollarStepBody
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Lambda.seqFun
public import DomainTheory.DataTypesAsLattices.Lambda.seqFunLt
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import DomainTheory.DataTypesAsLattices.FixedPoint.iterateBot
public import DomainTheory.DataTypesAsLattices.FixedPoint.fix
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
public import DomainTheory.DataTypesAsLattices.Lambda.seqFunLt_more
public import DomainTheory.DataTypesAsLattices.Lambda.seqFun_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def dollarStep (s : Pomega) : Pomega :=
  graph (fun u => graph (fun z => dollarStepBody s u z))

namespace dollarStep

theorem isScottContinuous : IsScottContinuous dollarStep :=
  graph_const_isScottContinuous
    (fun s u => graph (fun z => dollarStepBody s u z))
    (fun u => graph_const_isScottContinuous
      (fun s z => dollarStepBody s u z)
      (fun z => dollarStepBody.isScottContinuous_s u z))

theorem app (s u : Pomega) :
    funOf (dollarStep s) u = graph (fun z => dollarStepBody s u z) :=
  beta (graph_const_isScottContinuous (fun u z => dollarStepBody s u z)
    (fun z => dollarStepBody.isScottContinuous_u s z)) u

theorem app2 (s u z : Pomega) :
    funOf (funOf (dollarStep s) u) z = dollarStepBody s u z := by
  rw [dollarStep.app]
  exact beta (dollarStepBody.isScottContinuous_z s u) z

theorem iterate (n : ℕ) (u z : Pomega) :
```

```
    funOf (funOf (iterateBot dollarStep n) u) z = seqFunLt u z n := by
  induction n generalizing u z with
  | zero =>
    simp [iterateBot, funOf.bot, seqFunLt.zero]
  | succ n ih =>
    rw [iterateBot, dollarStep.app2, dollarStepBody, ih, seqFunLt.succ]

theorem fix_app (u z : Pomega) :
    funOf (funOf (fix dollarStep) u) z = seqFun u z := by
  have : funOf (funOf (fix dollarStep) u) z =
      ⋃ n, funOf (funOf (iterateBot dollarStep n) u) z := by
    simp [fix, funOf.iUnion]
  rw [this, seqFun.eq_iUnion_lt]
  ext k
  constructor
  · intro hk
    obtain ⟨n, hn⟩ := Set.mem_iUnion.mp hk
    exact Set.mem_iUnion.mpr ⟨n, by simpa [dollarStep.iterate] using hn⟩
  · intro hk
    obtain ⟨n, hn⟩ := Set.mem_iUnion.mp hk
    exact Set.mem_iUnion.mpr ⟨n, by simpa [dollarStep.iterate] using hn⟩

end dollarStep

end DomainTheory.DataTypesAsLattices
```

eq_2_25.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.seqFun
public import DomainTheory.DataTypesAsLattices.Lambda.dollarC
public import DomainTheory.DataTypesAsLattices.Lambda.seqFun_more
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.25).** `$ (u) = λx. ⋃ {u_i | i ∈ x}`. -/
theorem eq_2_25 (u x : Pomega) :
    funOf (funOf dollarC u) x = seqFun u x := by
  rw [dollarC.app]
  exact beta (seqFun.right_isScottContinuous u) x

end DomainTheory.DataTypesAsLattices
```

termDollarStepBody.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
```

```
public import DomainTheory.DataTypesAsLattices.Lambda.termShift
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.predSet
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.dollarStepBody
public import DomainTheory.DataTypesAsLattices.Lambda.id_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
public import DomainTheory.DataTypesAsLattices.Lambda.predSet_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices
def termDollarStepBody (s u z : Pomega) : Pomega :=
  condSet z (funOf u (ofNat 0))
    (funOf (funOf s (termShift u)) (predSet z))

namespace termDollarStepBody

theorem isScottContinuous_u (s z : Pomega) :
    IsScottContinuous (fun u => termDollarStepBody s u z) := by
  have helse : IsScottContinuous
      (fun u => funOf (funOf s (termShift u)) (predSet z)) :=
    theorem_1_3 (f := fun w => funOf w (predSet z))
      (g := fun u => funOf s (termShift u))
      (funOf.isScottContinuous_left (predSet z))
      (theorem_1_3 (funOf.isScottContinuous s) termShift.left_isScottContinuous)
  exact theorem_1_3.tuple
    (fun y => condSet.isScottContinuous_mid z y)
    (fun x => condSet.isScottContinuous_right z x)
    (funOf.isScottContinuous_left (ofNat 0)) helse

theorem isScottContinuous_z (s u : Pomega) :
    IsScottContinuous (fun z => termDollarStepBody s u z) :=
  theorem_1_3.tuple
    (fun y => condSet.isScottContinuous_left (funOf u (ofNat 0)) y)
    (fun z => condSet.isScottContinuous_right z (funOf u (ofNat 0)))
    id_isScottContinuous
    (theorem_1_3 (funOf.isScottContinuous (funOf s (termShift u)))
      predSet.isScottContinuous)

theorem isScottContinuous_s (u z : Pomega) :
    IsScottContinuous (fun s => termDollarStepBody s u z) := by
  have hinner : IsScottContinuous
      (fun s => funOf (funOf s (termShift u)) (predSet z)) :=
    theorem_1_3 (f := fun w => funOf w (predSet z))
      (g := fun s => funOf s (termShift u))
      (funOf.isScottContinuous_left (predSet z))
      (funOf.isScottContinuous_left (termShift u))
  exact theorem_1_3 (condSet.isScottContinuous_right z (funOf u (ofNat 0))) hinner
```

```
end termDollarStepBody

end DomainTheory.DataTypesAsLattices
```

seqFun.lean

```
module
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.Term
public import DomainTheory.DataTypesAsLattices.Lambda.IsCombinatory
public import DomainTheory.DataTypesAsLattices.Lambda.Comb
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.24)-(2.25).** Revaluation as a distributive sequence. -/
def seqFun (u x : Pomega) : Pomega :=
  ⋃ i ∈ x, funOf u (ofNat i)

end DomainTheory.DataTypesAsLattices
```

termDollarStep.lean

```
module

public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.FixedPoint.iterateBot
public import DomainTheory.DataTypesAsLattices.FixedPoint.fix
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.termDollarStepBody
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.seqFunLt
public import DomainTheory.DataTypesAsLattices.Lambda.seqFun
public import DomainTheory.DataTypesAsLattices.Lambda.dollarStep
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
public import DomainTheory.DataTypesAsLattices.Lambda.seqFunLt_more
public import DomainTheory.DataTypesAsLattices.Lambda.seqFun_more
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def termDollarStep (s : Pomega) : Pomega :=
  graph (fun u => graph (fun z => termDollarStepBody s u z))

namespace termDollarStep
```

```
theorem isScottContinuous : IsScottContinuous termDollarStep :=
  graph_const_isScottContinuous
    (fun s u => graph (fun z => termDollarStepBody s u z))
    (fun u => graph_const_isScottContinuous
      (fun s z => termDollarStepBody s u z)
      (fun z => termDollarStepBody.isScottContinuous_s u z))

theorem app (s u : Pomega) :
    funOf (termDollarStep s) u = graph (fun z => termDollarStepBody s u z) :=
  beta (graph_const_isScottContinuous (fun u z => termDollarStepBody s u z)
    (fun z => termDollarStepBody.isScottContinuous_u s z)) u

theorem app2 (s u z : Pomega) :
    funOf (funOf (termDollarStep s) u) z = termDollarStepBody s u z := by
  rw [app]
  exact beta (termDollarStepBody.isScottContinuous_z s u) z

theorem iterate (n : ℕ) (u z : Pomega) :
    funOf (funOf (iterateBot termDollarStep n) u) z = seqFunLt u z n := by
  induction n generalizing u z with
  | zero =>
    simp [iterateBot, funOf.bot, seqFunLt.zero]
  | succ n ih =>
    rw [iterateBot, app2, termDollarStepBody, ih, seqFunLt.term]

theorem fix_app (u z : Pomega) :
    funOf (funOf (fix termDollarStep) u) z = seqFun u z := by
  have : funOf (funOf (fix termDollarStep) u) z =
      ⋃ n, funOf (funOf (iterateBot termDollarStep n) u) z := by
    simp [fix, funOf.iUnion]
  rw [this, seqFun.eq_iUnion_lt]
  ext k
  constructor
  · intro hk
    obtain ⟨n, hn⟩ := Set.mem_iUnion.mp hk
    exact Set.mem_iUnion.mpr ⟨n, by simpa [iterate] using hn⟩
  · intro hk
    obtain ⟨n, hn⟩ := Set.mem_iUnion.mp hk
    exact Set.mem_iUnion.mpr ⟨n, by simpa [iterate] using hn⟩

end termDollarStep

end DomainTheory.DataTypesAsLattices
```

2.3.14 Primitive recursion

Suppose that f is any continuous function and $a \in \mathbf{P}\omega$. We can define $p : \omega \to \mathbf{P}\omega$ in the ordinary way by primitive recursion where:

$$p(0) = a;$$

$$p(n+1) = f(n)(p(n)).$$

The question is: can we give a LAMBDA-definition for $\hat{p}$ (in terms of f and a as constants, say)? The answer is clear, for we can prove:

$$(2.27) \quad \hat{p} = \mathbf{Y}(\lambda u \lambda n \in \omega.\, n \supset a,\ f(n-1)(u(n-1))).$$

This already shows that a large part of the definitions of recursion theory can be given in this special language. Of course, simultaneous (primitive) recursions can be transcribed into LAMBDA with the aid of the ordered tuples of (2.22), (2.23) above. But we can go further and connect with partial (and general) recursive functions. We state first a definition.

primRecOpTerm.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Lambda.Term
public import DomainTheory.DataTypesAsLattices.Graph.lam
public import DomainTheory.DataTypesAsLattices.Enumeration.Avoids
@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable

open Nat.Partrec (Code)

open Nat.Partrec.Code

def primRecOpTerm : Term :=
  .lam 0 (.lam 1
    (.app (.var 2)
      (.app (.var 3) (.app (.app (.var 4) (.var 0)) (.var 1)))))

end DomainTheory.DataTypesAsLattices
```

sqrtIter.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Computability.pack4

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable

open Nat.Partrec (Code)
```

```
open Nat.Partrec.Code

/-- The bounded iteration used to compute `Nat.sqrt n`. -/
def sqrtIter (n : ℕ) : ℕ → ℕ
  | 0 => 0
  | k + 1 =>
      let a := sqrtIter n k
      if (a + 1) * (a + 1) ≤ n then a + 1 else a

namespace sqrtIter

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem eq_min (n k : ℕ) : sqrtIter n k = min k (Nat.sqrt n) := by
  induction k with
  | zero => simp [sqrtIter]
  | succ k ih =>
    rw [sqrtIter, ih]
    by_cases h : k < Nat.sqrt n
    · have hmin : min k (Nat.sqrt n) = k := min_eq_left h.le
      have hs : (k + 1) * (k + 1) ≤ n := Nat.le_sqrt.mp h
      simp [hmin, hs, min_eq_left (Nat.succ_le_iff.mpr h)]
    · have hk : Nat.sqrt n ≤ k := Nat.le_of_not_gt h
      have hmin : min k (Nat.sqrt n) = Nat.sqrt n := min_eq_right hk
      have hs : ¬(Nat.sqrt n + 1) * (Nat.sqrt n + 1) ≤ n :=
        Nat.not_le.mpr (Nat.lt_succ_sqrt n)
      simp [hmin, hs, min_eq_right (hk.trans (Nat.le_succ k))]

theorem self (n : ℕ) : sqrtIter n n = Nat.sqrt n := by
  rw [sqrtIter.eq_min, min_eq_right (Nat.sqrt_le_self n)]

end sqrtIter

end DomainTheory.DataTypesAsLattices
```

sqrtStepTerm.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Lambda.Term
public import DomainTheory.DataTypesAsLattices.Graph.lam
public import DomainTheory.DataTypesAsLattices.Enumeration.Avoids
@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable

open Nat.Partrec (Code)
```

```
open Nat.Partrec.Code

/-- `λn _ a. if (a+1)^2 ≤ n then a+1 else a`. -/
def sqrtStepTerm : Term :=
  .lam 0 (.lam 1 (.lam 2
    (.cond
      (.app (.app (.var 3)
        (.app (.app (.var 4) (.succ (.var 2))) (.succ (.var 2))))
        (.var 0))
      (.var 2)
      (.succ (.var 2)))))

end DomainTheory.DataTypesAsLattices
```

2.3.15 Definition (computable)

DEFINITION. A continuous function f of k-variables is *computable* iff the relationship

$$m \in f(e_{n_0})(e_{n_1})\cdots(e_{n_{k-1}})$$

is recursively enumerable (r.e.) in the integer variables $m, n_0, n_1, \ldots, n_{k-1}$.

If q is a partial recursive function in the usual sense, then we can regard it as a mapping $q : \omega \to \omega \cup \{\bot\}$, where $q(n) = \bot$ means that q is undefined at n. Saying that q is partial recursive is just to say that $m \in q(n)$ is r.e. as a relation in n and m. It is easy to see that this is in turn equivalent to the recursive enumerability of the relationship $m \in \hat{q}(e_n)$; and so our definition is formally a generalization of the usual one. But it is also intuitively reasonable. To "compute" $y = f(x)$, in the one-variable case, we proceed by enumeration. First we begin the enumeration of all finite subsets $e_n \subseteq x$. For each of these f starts up an enumeration of the set $f(e_n)$; so we sit back and observe which $m \in f(e_n)$ by enumeration. The totality of all such m for all $e_n \subseteq x$ forms in the end the set y.

2.3.16 Computable functions

LambdaDefinable.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Lambda.Term
public import DomainTheory.DataTypesAsLattices.Lambda.interp
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Enumeration.Avoids
@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

/-- Closed LAMBDA-definable elements (empty environment). -/
class LambdaDefinable (u : Pomega) : Prop where
  out : ∃ t : Term, interp t (fun _ => botElem) = u

end DomainTheory.DataTypesAsLattices
```

(Already included: `1976_Definition_computable.md`.)

2.3.17 Definability

As a hint for the proof we may note that the method of (2.27) shows that all primitive recursive functions p have the corresponding $\hat{p}$ LAMBDA-definable. Next we remark that a nonempty r.e. set is the range of a primitive recursive function; but the range of p is $\hat{p}(\top)$, which is clearly LAMBDA-definable. That any LAMBDA-definable set (graph) is r.e. is obvious from the definition of the language itself.

`unpairSqrtTerm.lean`

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Lambda.Term
public import DomainTheory.DataTypesAsLattices.Enumeration.Avoids
@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable

open Nat.Partrec (Code)

open Nat.Partrec.Code

def unpairSqrtTerm : Term := .app (.var 1) (.var 0)

end DomainTheory.DataTypesAsLattices
```

`unpairSquareTerm.lean`

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Lambda.Term
public import DomainTheory.DataTypesAsLattices.Computability.unpairSqrtTerm
public import DomainTheory.DataTypesAsLattices.Enumeration.Avoids

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable

open Nat.Partrec (Code)

open Nat.Partrec.Code

def unpairSquareTerm : Term :=
```

```
  .app (.app (.var 3) unpairSqrtTerm) unpairSqrtTerm

end DomainTheory.DataTypesAsLattices
```

unpairDeltaTerm.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Lambda.Term
public import DomainTheory.DataTypesAsLattices.Computability.unpairSquareTerm
public import DomainTheory.DataTypesAsLattices.Enumeration.Avoids

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable

open Nat.Partrec (Code)

open Nat.Partrec.Code

def unpairDeltaTerm : Term :=
  .app (.app (.var 2) (.var 0)) unpairSquareTerm

end DomainTheory.DataTypesAsLattices
```

unpairTestTerm.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Lambda.Term
public import DomainTheory.DataTypesAsLattices.Computability.unpairSqrtTerm
public import DomainTheory.DataTypesAsLattices.Computability.unpairDeltaTerm
public import DomainTheory.DataTypesAsLattices.Enumeration.Avoids

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable

open Nat.Partrec (Code)

open Nat.Partrec.Code

def unpairTestTerm : Term :=
  .app (.app (.var 4) (.succ unpairDeltaTerm)) unpairSqrtTerm
```

```
end DomainTheory.DataTypesAsLattices
```

unpairSndTerm.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Lambda.Term
public import DomainTheory.DataTypesAsLattices.Graph.lam
public import DomainTheory.DataTypesAsLattices.Computability.unpairSqrtTerm
public import DomainTheory.DataTypesAsLattices.Computability.unpairDeltaTerm
public import DomainTheory.DataTypesAsLattices.Computability.unpairTestTerm
public import DomainTheory.DataTypesAsLattices.Enumeration.Avoids

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable

open Nat.Partrec (Code)

open Nat.Partrec.Code

def unpairSndTerm : Term :=
  .lam 0 (.cond unpairTestTerm
    (.app (.app (.var 2) unpairDeltaTerm) unpairSqrtTerm)
    unpairSqrtTerm)

end DomainTheory.DataTypesAsLattices
```

Theorem 2.6 THEOREM 2.6 (The definability theorem). *For a k-ary continuous function f, the following are equivalent:*

(i) f is computable;

(ii) $\lambda x_0 \lambda x_1 \cdots \lambda x_{k-1}.\, f(x_0)(x_1)\cdots(x_{k-1})$ as a set is r.e.;

(iii) $\lambda x_0 \lambda x_1 \cdots \lambda x_{k-1}.\, f(x_0)(x_1)\cdots(x_{k-1})$ is LAMBDA-definable.

As a hint for the proof we may note that the method of (2.27) shows that all primitive recursive functions p have the corresponding $\hat{p}$ LAMBDA-definable. Next we remark that a nonempty r.e. set is the range of a primitive recursive function; but the range of p is $\hat{p}(\top)$, which is clearly LAMBDA-definable. That any LAMBDA-definable set (graph) is r.e. is obvious from the definition of the language itself.

We may draw some interesting conclusions from the definability theorem. In the first place, we see that the countable collection $\mathbf{RE} \subseteq \mathbf{P}\omega$ of r.e. sets is closed under application and LAMBDA-definability. Indeed it forms a model for the λ-calculus (axioms (α), (β), and (ξ^*) at least) and it also contains the arithmetical combinators. (Clearly there will be many intermediate submodels.) In the second place, we can see now how very easy it is to interpret λ-calculus in ordinary arithmetical recursion theory by means of quite elementary operations on r.e. sets. Thus the equivalence of λ-definability with partial recursiveness seems not to be all that good a piece of evidence for Church's Thesis. In his 1936 paper (a footnote on p. 346) Church says about λ-definability:

> The fact, however, that two such widely different and (in the opinion of the author) equally natural definitions of effective calculability turn out to be equivalent adds to the strength of the reasons adduced below for believing that they constitute as general a characterization of this notion as is consistent with the usual intuitive understanding of it.

The point never struck the present author as an especially telling one, and the reduction of λ-calculus to r.e. theory shows that the divergence between the theories is not at all wide. Of course it is a pleasant surprise to see how many complicated things can be defined in pure λ-calculus (without arithmetical combinators), but this fact cuts the wrong way as evidence for the thesis (we want stronger theories, not weaker ones). Post systems (or even first-order theories) are much better to mention in this connection, since they are obviously more inclusive in giving enumerations than Turing machines or Herbrand–Gödel recursion equations. But the equivalence proofs are all so easy! What one would like to see is a "natural" definition where the equivalence with r.e. is not just a mechanical exercise involving a few tricks of coding.

In the course of the development in this section we have stated many equations which are not found in Table 1, and which involve new combinators. In conclusion we would like to mention an equation about $\mathbf{Y}$ which holds in the model, which can be stated in pure λ-calculus, and which cannot be proved by ordinary reduction (though we shall not try to justify this last statement here). In order to shorten the calculations, we note from definition (2.8) that $\mathbf{Y}(u) = \mathbf{Y}(\lambda y.\, u(y))$; so by Theorem 2.5 this also equals $u(\mathbf{Y}(u))$.

$$(2.28) \quad \mathbf{Y}(\lambda f \lambda x.\, g(x)(f(x))) = \lambda x.\, \mathbf{Y}(g(x)).$$

Call the left side f' and the right f''. Now $f'' = \lambda x.\, g(x)(\mathbf{Y}(g(x))) = \lambda x.\, g(x)(f''(x))$, thus $f' \subseteq f''$, because f' is a least fixed point. On the other hand $f' = \lambda x.\, g(x)(f'(x))$, so $f'(x) = g(x)(f'(x))$. Thus $f''(x) \subseteq f'(x)$, because $f''(x)$ is a least fixed point. As this holds for all x, we see that $f'' \subseteq f'$; and so they are equal. There must be many other such equations.

Motivation The three conditions meet as follows. Passing from a computable f to its iterated graph replaces $m \in f(e_{n_0})(e_{n_1}) \cdots (e_{n_{k-1}})$ by membership of the iterated pair $(n_0, (n_1, (\cdots, (n_{k-1}, m) \cdots)))$ in that graph, so the function is computable exactly when the graph is r.e. The hint above shows that an r.e. graph is LAMBDA-definable. In the other direction each of the six combinators of Theorem 2.4 is r.e., and application preserves r.e. sets: $m \in u(x)$ is an existential quantification over a finite $e_n \subseteq x$ together with a bounded universal quantification, and membership in u and in x occurs positively.

IsRE.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Pomega.e
@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

/-- Recursively enumerable subsets of `ω`, as elements of `Pω`. -/
class IsRE (u : Pomega) : Prop where
  out : REPred fun n : ℕ => n ∈ u

end DomainTheory.DataTypesAsLattices
```

REPredu2eempty.lean

```
module

public import Mathlib.Computability.RE

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

theorem REPred.empty : REPred fun _ : ℕ => False := by
  unfold REPred
  refine (Partrec.none (α := ℕ) (σ := Unit)).of_eq fun _ => ?_
  ext
  simp [Part.assert]

end DomainTheory.DataTypesAsLattices
```

REPredu2eor.lean

```
module

public import Mathlib.Computability.RE

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

theorem REPred.or {α} [Primcodable α] {p q : α → Prop}
    (hp : REPred p) (hq : REPred q) :
    REPred fun a => p a ∨ q a := by
  obtain ⟨k, hk, H⟩ := Partrec.merge' hp hq
  have hg : Computable₂ fun (_ : α) (_ : Unit) => () :=
    (Computable.const (α := α × Unit) ()).to₂
  refine Partrec.of_eq (hk.map hg) fun a => ?_
  ext
  constructor
  · intro hu
    rw [Part.mem_map_iff] at hu
    obtain ⟨x, hx, hxu⟩ := hu
    have hx' := (H a).1 x hx
    have hpq : p a ∨ q a := by
      cases hx' with
      | inl h =>
        exact Or.inl (Part.mem_assert_iff.1 h).1
      | inr h =>
        exact Or.inr (Part.mem_assert_iff.1 h).1
    exact Part.mem_assert_iff.2 ⟨hpq, Part.mem_some_iff.2 hxu.symm⟩
  · intro hu
    have hpq : p a ∨ q a := (Part.mem_assert_iff.1 hu).1
    have hd : (k a).Dom := (H a).2.mpr <|
      hpq.imp (fun h => ⟨h, trivial⟩) (fun h => ⟨h, trivial⟩)
```

```
    rw [Part.mem_map_iff]
    exact ⟨(), ⟨hd, rfl⟩, (Part.mem_some_iff.1 (Part.mem_assert_iff.1 hu).2).symm⟩

end DomainTheory.DataTypesAsLattices
```

REPredu2etrue.lean

```
module

public import Mathlib.Computability.RE

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

theorem REPred.true {α} [Primcodable α] : REPred fun _ : α => True := by
  unfold REPred
  refine (Decidable.Partrec.const' (Part.some ())).of_eq fun _ => ?_
  ext
  simp [Part.assert]

end DomainTheory.DataTypesAsLattices
```

enumFun.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Pomega.pair
@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable

open Nat.Partrec (Code)

open Nat.Partrec.Code

/-- Stage-bounded enumerator associated to a partial-recursive code. -/
def enumFun (c : Code) (t : ℕ) : ℕ :=
  bif (evaln t.unpair.2 c t.unpair.1).isSome
    then t.unpair.1 + 1
    else 0

end DomainTheory.DataTypesAsLattices
```

mem_graph_const_iff.lean

```
module
```

```
public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem mem_graph_const_iff (n m : ℕ) :
    m ∈ graph (fun _ => e n) ↔
      ∃ t, m = pair (Nat.unpair t).1 (Nat.unpair t).2 ∧
        n.testBit (Nat.unpair t).2 = true := by
  constructor
  · intro ⟨a, b, hm, hb⟩
    exact ⟨Nat.pair a b, by simp [Nat.unpair_pair, hm],
      by simp [Nat.unpair_pair, mem_e.mp hb]⟩
  · intro ⟨t, ht, hb⟩
    exact ⟨(Nat.unpair t).1, (Nat.unpair t).2, ht, mem_e.mpr hb⟩

end DomainTheory.DataTypesAsLattices
```

unpairFstTerm.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Lambda.Term
public import DomainTheory.DataTypesAsLattices.Graph.lam
public import DomainTheory.DataTypesAsLattices.Computability.unpairSqrtTerm
public import DomainTheory.DataTypesAsLattices.Computability.unpairDeltaTerm
public import DomainTheory.DataTypesAsLattices.Computability.unpairTestTerm
public import DomainTheory.DataTypesAsLattices.Enumeration.Avoids

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable

open Nat.Partrec (Code)

open Nat.Partrec.Code

def unpairFstTerm : Term :=
  .lam 0 (.cond unpairTestTerm unpairSqrtTerm unpairDeltaTerm)
```

```
end DomainTheory.DataTypesAsLattices
```

applyNat.lean

```
module

public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Pomega.pair

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (3.4).** `apply(n)(m) = (n, m) + 1`. -/
def applyNat (n m : ℕ) : ℕ := pair n m + 1

end DomainTheory.DataTypesAsLattices
```

num.lean

```
module

public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Enumeration.applyNat

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (3.11).** `num(0) = 1`, `num(n+1) = apply(12, num n)`. -/
def num : ℕ → ℕ
  | 0 => 1
  | n + 1 => applyNat 12 (num n)

end DomainTheory.DataTypesAsLattices
```

eSubset_iff_stage.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices
```

```
open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem eSubset_iff_stage {x : Pomega} {c : Code}
    (hc : Code.eval c = fun k => Part.assert (k ∈ x) fun _ => Part.some (0 : ℕ))
    (n : ℕ) :
    e n ⊆ x ↔ ∃ s, ∀ k < n, n.testBit k = false ∨ (evaln s c k).isSome = true := by
  constructor
  · intro hsub
    let stage : ℕ → ℕ := fun k =>
      if h : n.testBit k = true then
        ((evaln_complete (x := 0)).1 (by
          rw [hc]
          exact Part.mem_assert_iff.mpr ⟨hsub (mem_e.mpr h), Part.mem_some_iff.mpr rfl⟩
    )).choose
      else 0
    refine ⟨(Finset.range n).sup stage, fun k hlt => ?_⟩
    by_cases hb : n.testBit k = true
    · refine Or.inr ?_
      have hle : stage k ≤ (Finset.range n).sup stage :=
        Finset.le_sup (Finset.mem_range.mpr hlt)
      have h0 : 0 ∈ evaln (stage k) c k := by
        simp only [stage, hb, ↓reduceDIte]
        exact ((evaln_complete (x := 0)).1 (by
          rw [hc]
          exact Part.mem_assert_iff.mpr ⟨hsub (mem_e.mpr hb), Part.mem_some_iff.mpr rfl
    ⟩)).choose_spec
      exact (Option.isSome_of_mem (evaln_mono hle h0))
    · exact Or.inl (Bool.eq_false_iff.mpr hb)
  · intro ⟨s, hs⟩ k hk
    have hlt : k < n := mem_e_lt hk
    have hsome : (evaln s c k).isSome = true :=
      (hs k hlt).resolve_left fun hfalse => Bool.eq_false_iff.mp hfalse (mem_e.mp hk)
    obtain ⟨val, hval⟩ := Option.isSome_iff_exists.mp hsome
    have : val ∈ Code.eval c k :=
      evaln_sound (by simpa [Option.mem_def] using hval)
    have : val ∈ (Part.assert (k ∈ x) fun _ => Part.some (0 : ℕ)) := by
      simpa [hc] using this
    exact (Part.mem_assert_iff.mp this).1

end DomainTheory.DataTypesAsLattices
```

precTerm.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Lambda.Term
public import DomainTheory.DataTypesAsLattices.Graph.lam
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Enumeration.Avoids
@[expose] public section

set_option maxHeartbeats 2000000
```

```
namespace DomainTheory.DataTypesAsLattices

open Encodable

open Nat.Partrec (Code)

open Nat.Partrec.Code

/-- On packed input `(z,n)`, recurse from `f z` with
step `λy ih. g (pair z (pair y ih))`. -/
def precTerm : Term :=
  .lam 0
    (.app
      (.app
        (.app (.var 3) (.app (.var 4) (.app (.var 6) (.var 0))))
        (.lam 1 (.lam 2
          (.app (.var 5)
            (.app (.app (.var 8) (.app (.var 6) (.var 0)))
              (.app (.app (.var 8) (.var 1)) (.var 2)))))))
      (.app (.var 7) (.var 0)))

end DomainTheory.DataTypesAsLattices
```

eSubset_funOf_e_iff.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Pomega.e

public import DomainTheory.DataTypesAsLattices.Graph.Basic.notation3
@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem eSubset_funOf_e_iff (k a b : ℕ) :
    e k ⊆ e a ? e b ↔
      ∀ i < k, k.testBit i = false ∨ i ∈ e a ? e b := by
  constructor
  · intro h i hi
    by_cases hb : k.testBit i = true
    · exact Or.inr (h (mem_e.mpr hb))
    · exact Or.inl (Bool.eq_false_iff.mpr hb)
  · intro h i hi
    have hlt : i < k := mem_e_lt hi
    exact (h i hlt).resolve_left fun hf =>
      Bool.eq_false_iff.mp hf (mem_e.mp hi)
```

```
end DomainTheory.DataTypesAsLattices
```

pairTerm.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Lambda.Term
public import DomainTheory.DataTypesAsLattices.Graph.lam
public import DomainTheory.DataTypesAsLattices.Enumeration.Avoids
@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable

open Nat.Partrec (Code)

open Nat.Partrec.Code

/-- `λa λb. (b-a) ? (a*a+a+b), (b*b+a)` with free `add, mul, sub`. -/
def pairTerm : Term :=
  .lam 3 (.lam 4
    (.cond (.app (.app (.var 2) (.var 4)) (.var 3))
      (.app (.app (.var 0)
          (.app (.app (.var 0)
              (.app (.app (.var 1) (.var 3)) (.var 3)))
            (.var 3)))
        (.var 4))
      (.app (.app (.var 0)
          (.app (.app (.var 1) (.var 4)) (.var 4)))
        (.var 3))))

end DomainTheory.DataTypesAsLattices
```

eSubset_e_iff.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem eSubset_e_iff (n b : ℕ) :
```

```
    e n ⊆ e b ↔ ∀ k < n, n.testBit k = false ∨ b.testBit k = true := by
  constructor
  · intro h k hk
    by_cases hb : n.testBit k = true
    · exact Or.inr (mem_e.mp (h (mem_e.mpr hb)))
    · exact Or.inl (Bool.eq_false_iff.mpr hb)
  · intro h k hk
    have hlt : k < n := mem_e_lt hk
    exact mem_e.mpr ((h k hlt).resolve_left fun hf =>
      Bool.eq_false_iff.mp hf (mem_e.mp hk))

end DomainTheory.DataTypesAsLattices
```

lt_two_pow_of_high_bits.lean

```
module

public import Mathlib.Computability.RE
public import Mathlib.Data.Nat.Bitwise

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem lt_two_pow_of_high_bits {n k : ℕ}
    (h : ∀ j, k ≤ j → n.testBit j = false) : n < 2 ^ k := by
  induction n using Nat.binaryRec generalizing k with
  | zero => simp
  | bit b n ih =>
    cases k with
    | zero =>
      have hb : b = false := by
        simpa [Nat.testBit_bit_zero] using h 0 le_rfl
      have hn0 : n = 0 := Nat.zero_of_testBit_eq_false fun j => by
        simpa [Nat.testBit_bit_succ] using h (j + 1) (Nat.zero_le (j + 1))
      simp [hb, hn0]
    | succ k =>
      have : n < 2 ^ k := ih fun j hj => by
        simpa [Nat.testBit_bit_succ] using h (j + 1) (Nat.succ_le_succ hj)
      exact Nat.bit_lt_two_pow_succ_iff.mpr this

end DomainTheory.DataTypesAsLattices
```

bits_subset_lt_two_pow.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Pomega.e
```

```
public import DomainTheory.DataTypesAsLattices.Computability.lt_two_pow_of_high_bits

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem bits_subset_lt_two_pow {n b : ℕ} (h : e n ⊆ e b) : n < 2 ^ (b + 1) :=
  lt_two_pow_of_high_bits fun _j hj =>
    Bool.eq_false_iff.2 fun hbit =>
      Nat.not_le.mpr hj (mem_e_le (h (mem_e.mpr hbit)))

end DomainTheory.DataTypesAsLattices
```

(Already included: `1976_Theorem_2_6.md`.)

2.3.18 A law of Y

In the course of the development in this section we have stated many equations which are not found in Table 1, and which involve new combinators. In conclusion we would like to mention an equation about $\mathbf{Y}$ which holds in the model, which can be stated in pure λ-calculus, and which cannot be proved by ordinary reduction (though we shall not try to justify this last statement here). In order to shorten the calculations, we note from definition (2.8) that $\mathbf{Y}(u) = \mathbf{Y}(\lambda y.\, u(y))$; so by Theorem 2.5 this also equals $u(\mathbf{Y}(u))$.

$$(2.28) \quad \mathbf{Y}(\lambda f \lambda x.\, g(x)(f(x))) = \lambda x.\, \mathbf{Y}(g(x)).$$

Call the left side f' and the right f''. Now $f'' = \lambda x.\, g(x)(\mathbf{Y}(g(x))) = \lambda x.\, g(x)(f''(x))$, thus $f' \subseteq f''$, because f' is a least fixed point. On the other hand $f' = \lambda x.\, g(x)(f'(x))$, so $f'(x) = g(x)(f'(x))$. Thus $f''(x) \subseteq f'(x)$, because $f''(x)$ is a least fixed point. As this holds for all x, we see that $f'' \subseteq f'$; and so they are equal. There must be many other such equations.

`eq_2_28.lean`

```
module

public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.FixedPoint.fix
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_4
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.Ymap
public import DomainTheory.DataTypesAsLattices.Lambda.Fofg
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.FixedPoint.iterateBot
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.lam
```

```
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.28).** Least-fixed-point unfolding `Y(g(x)) = g(x)(Y(g(x)))`. -/
theorem eq_2_28 {g : Pomega → Pomega → Pomega}
    (hgy : ∀ x, IsScottContinuous (fun y => g x y)) (x : Pomega) :
    fix (g x) = g x (fix (g x)) :=
  (theorem_1_4 (hgy x)).1.symm

namespace eq_2_28

/-- **Scott 1976, (2.28).** `Y(λf λx. g(x)(f(x))) = λx. Y(g(x))`. -/
theorem commute {g : Pomega → Pomega → Pomega}
    (hgx : ∀ y, IsScottContinuous (fun x => g x y))
    (hgy : ∀ x, IsScottContinuous (fun y => g x y)) :
    fix (Fofg g) = Ymap g := by
  have hF : IsScottContinuous (Fofg g) :=
    graph_const_isScottContinuous (fun f x => g x (funOf f x))
      (fun x => theorem_1_3 (hgy x) (funOf.isScottContinuous_left x))
  have hfix : Fofg g (Ymap g) = Ymap g := by
    apply congrArg graph
    funext x
    rw [Ymap.app hgx hgy]
    exact (theorem_1_4 (hgy x)).1
  have hle : fix (Fofg g) ⊆ Ymap g := (theorem_1_4 hF).2 (Ymap g) hfix
  refine subset_antisymm hle ?_
  have hiter : ∀ n x, funOf (iterateBot (Fofg g) n) x = iterateBot (g x) n := by
    intro n
    induction n with
    | zero =>
      intro x
      ext m
      simp [iterateBot, funOf, botElem]
    | succ n ih =>
      intro x
      have hcont : IsScottContinuous
          (fun z => g z (funOf (iterateBot (Fofg g) n) z)) :=
        theorem_1_3.tuple hgx hgy id_isScottContinuous
          (funOf.isScottContinuous (iterateBot (Fofg g) n))
      change funOf (lam (fun z => g z (funOf (iterateBot (Fofg g) n) z))) x =
        g x (iterateBot (g x) n)
      rw [beta hcont, ih]
  intro p hp
  rcases hp with ⟨n, m, rfl, hm⟩
  obtain ⟨k, hkm⟩ := Set.mem_iUnion.mp hm
  cases k with
  | zero =>
    simp [iterateBot, botElem] at hkm
  | succ k =>
    have : pair n m ∈ iterateBot (Fofg g) (k + 1) :=
```

```
      ⟨n, m, rfl, by
        change m ∈ g (e n) (funOf (iterateBot (Fofg g) k) (e n))
        rw [hiter]
        exact hkm⟩
    exact Set.mem_iUnion.mpr ⟨k + 1, this⟩

end eq_2_28

end DomainTheory.DataTypesAsLattices
```

iterateBot_param_isScottContinuous.lean

```
module

public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.FixedPoint.iterateBot
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.id_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.const_isScottContinuous

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem iterateBot_param_isScottContinuous
    {g : Pomega → Pomega → Pomega}
    (hgx : ∀ y, IsScottContinuous (fun x => g x y))
    (hgy : ∀ x, IsScottContinuous (fun y => g x y)) :
    ∀ n, IsScottContinuous (fun x => iterateBot (g x) n)
  | 0 => const_isScottContinuous botElem
  | n + 1 =>
    theorem_1_3.tuple (hgx) (hgy) id_isScottContinuous
      (iterateBot_param_isScottContinuous hgx hgy n)

end DomainTheory.DataTypesAsLattices
```

fix_param_isScottContinuous.lean

```
module

public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.FixedPoint.iterateBot
public import DomainTheory.DataTypesAsLattices.FixedPoint.fix
public import DomainTheory.DataTypesAsLattices.Continuous.mem_scottUnion
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Continuous.scottUnion
public import DomainTheory.DataTypesAsLattices.Pomega.e
```

```
public import DomainTheory.DataTypesAsLattices.Lambda.
    iterateBot_param_isScottContinuous

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem fix_param_isScottContinuous
    {g : Pomega → Pomega → Pomega}
    (hgx : ∀ y, IsScottContinuous (fun x => g x y))
    (hgy : ∀ x, IsScottContinuous (fun y => g x y)) :
    IsScottContinuous (fun x => fix (g x)) := by
  constructor
  intro x
  ext k
  constructor
  · intro hk
    obtain ⟨n, hkn⟩ := Set.mem_iUnion.mp hk
    have := (iterateBot_param_isScottContinuous hgx hgy n).out x
    have : k ∈ scottUnion (fun z => iterateBot (g z) n) x := by
      rwa [← this]
    obtain ⟨m, hm, hkm⟩ := mem_scottUnion.mp this
    exact mem_scottUnion.mpr ⟨m, hm, Set.mem_iUnion.mpr ⟨n, hkm⟩⟩
  · intro hk
    obtain ⟨m, hm, hkm⟩ := mem_scottUnion.mp hk
    obtain ⟨n, hkn⟩ := Set.mem_iUnion.mp hkm
    exact Set.mem_iUnion.mpr ⟨n,
      IsScottContinuous.monotone (iterateBot_param_isScottContinuous hgx hgy n) hm hkn⟩

end DomainTheory.DataTypesAsLattices
```

Ymap.lean

```
module

public import DomainTheory.DataTypesAsLattices.FixedPoint.fix
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.fix_param_isScottContinuous

@[expose] public section

namespace DomainTheory.DataTypesAsLattices
def Ymap (g : Pomega → Pomega → Pomega) : Pomega :=
  graph (fun x => fix (g x))

namespace Ymap
```

```
theorem app {g : Pomega → Pomega → Pomega}
    (hgx : ∀ y, IsScottContinuous (fun x => g x y))
    (hgy : ∀ x, IsScottContinuous (fun y => g x y)) (x : Pomega) :
    funOf (Ymap g) x = fix (g x) :=
  beta (fix_param_isScottContinuous hgx hgy) x

end Ymap

end DomainTheory.DataTypesAsLattices
```

2.4 Retracts and data types

Historical note. A retract is an ordinary notion of topology. The use of computable retracts as data types, the operations on those retracts, and the recursive definitions by fixed-point equations are Scott's. A continuous lattice, in the sense of Scott (1972), has continuous meet and join, and as a space it is a T_0-space satisfying the extension theorem. The connection between lattices and fixed points was already known. The set of all retracts is a complete lattice by Theorem 4.1, and it is not itself a continuous lattice. Ershov solved that question after the body of the paper was written; Hosono and Sato (1975) found essentially the same argument independently. The argument is given with the ordering of retracts. The correspondence was Scott's introduction to Ershov's numeration theory. Applications to the semantics of flow diagrams are in Scott (1971), and were taken up by Goguen et al. (1975) and by Reynolds (1974).

Dependencies. Inside Retracts, each box is one Lean file. A box named as another folder in this chapter is that whole folder. A box named as another chapter is that whole chapter. The arrowhead points at the file or folder that imports, and the number is how many direct imports.

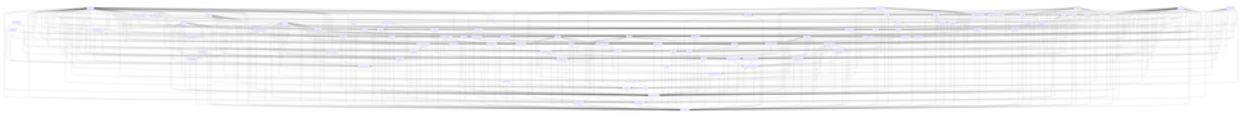


Figure 22: Retracts and data types.

Data can be structured in many ways: ordered tuples, lists, arrays, trees, streams, and even operations and functions. The last point becomes clear if one thinks of *parameters*. We would normally hardly consider the pairing function $\lambda x \lambda y.\, \langle x, y\rangle$ as being in itself a piece of data. But if we treat the first variable as a *parameter*, then it can be specialized to a *fixed value*, say the element a, producing the function $\lambda y.\, \langle a, y\rangle$. This function is more likely to be the output of some process and *in itself* can be considered as a datum. It is rather like one *whole row* of a matrix. If we were to regard a two-argument function f as being a matrix, then its ath row would be exactly $\lambda y.\, f(a)(y)$. If s were a selection function, then, for example, $\lambda y.\, f(s(y))(y)$ would represent the selection of one element out of each column of the matrix. This selection could be taken as a specialization of parameters in the operator $\lambda u \lambda v \lambda y.\, u(v(y))(y)$. We have not been very definite here about the exact nature of the fixed a, f, or s, or the range of the variable y or the range of values of the function f. The point is only to recall a few elements of structure and to suggest an abstract view of data going beyond the usual iterated arrays and trees.

What then is a *data type*? Answer: a type of data. That is to say, a collection of data that have been grouped together for reasons of similarity of structure or perhaps mere convenience. Thus the collection may very well be a mixed bag, but more often than not canons of taste or demands of simplicity dictate an

adherence to regularity. The grouping may be formed to eliminate irrelevant objects and focus the attention in other ways. It is frequently a matter of good organization that aids the understanding of complex definitions. In programming languages, one of the major reasons for making an explicit declaration of a data type (that is, the restriction of certain variables to certain *"modes"*) is that the computed objects of that type can enjoy a special *representation* in the machine that allows the manipulation of these objects via the chosen representation to be reasonably efficient. This is a very critical matter for good language design and good compiler writing. In this report, however, we cannot discuss the problems of representation, important as they may be. Our objective here is conceptual organization, and we wish to show how such ideas, in the language for computable functions used here, can find the proper expression.

Which are the data types that can be defined in LAMBDA? No final answer can be given since the number is infinite and inexhaustible. From one point of view, however, there is only one: $\mathbf{P}\omega$ itself. It is the universal type and all other types are subtypes of it; so $\mathbf{P}\omega$ plays a primary role in this exposition. But in a way it is too big, or at least too complex, since each of its elements can be used in so many different ways. When we specify a subtype the intention is to restrict attention to a special use. But even the various subtypes overlap, and so the same elements still get different uses. Style in writing definitions will usually make the differentiation clear though. The main innovation to be described in this section is the use of LAMBDA expressions to define types *as well as* elements. Certain expressions define *retracts* (or better: retraction mappings), and it is the *ranges* (or as we shall see: *sets of finite points*) of such retracts that form the groupings into types. Thus LAMBDA provides a calculus of type definitions including *recursive* type definitions. Examples will be explained both here and in the following sections. Note that types as retracts turn out to be types as *lattices*, that is, types of partial and many-valued objects. The problem of cutting these lattice types down to the perfect or complete objects is discussed in § 6. Another view of types and functionality of mappings is presented in § 7.

The notion of a retract comes from (analytic) topology, but it seems almost an accident that the idea can be applied in the present context. The word is employed not because there is some deep tie-up with topology but because it is short and rather descriptive. Three easy examples will motivate the general plan:

$$(4.1) \quad \mathbf{fun} = \lambda u \lambda x.\, u(x);$$

$$(4.2) \quad \mathbf{pair} = \lambda u.\, \langle u_0, u_1 \rangle;$$

$$(4.3) \quad \mathbf{bool} = \lambda u.\, u \sqsupset 0, 1.$$

Here $\sqsupset$ is the *doubly strict conditional* defined by

$$(4.4) \quad z \sqsupset x, y = z \supset (z \supset x, \top), (z \supset \top, y),$$

which has the property that if z is both zero and positive, then it takes the value $\top$ instead of the value $x \cup y$.

- Definition (retract)
- Definition (membership)
- Theorem 4.1
- Definition (ordering)

- Theorem 4.2
- Theorem 4.3
- Theorem 4.4
- Theorem 4.5
- Theorem 4.6

graph_ext.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Pomega.e
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem graph_ext {f g : Pomega → Pomega} (h : ∀ x, f x = g x) :
    graph f = graph g := by
  simp [graph, h]

end DomainTheory.DataTypesAsLattices
```

DirectedSet.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- A nonempty directed subset of a preorder. -/
class DirectedSet {α : Type u} [Preorder α] (s : Set α) : Prop where
  out : s.Nonempty ∧
    ∀ {{x}}, x ∈ s → ∀ {{y}}, y ∈ s →
      ∃ z ∈ s, x ≤ z ∧ y ≤ z

end DomainTheory.DataTypesAsLattices
```

predSet_ofNat_zero.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.predSet
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section
```

```
namespace DomainTheory.DataTypesAsLattices

theorem predSet_ofNat_zero : predSet (ofNat 0) = botElem := by
  ext k
  simp [predSet, ofNat, botElem]

end DomainTheory.DataTypesAsLattices
```

completeLatticeLE.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- Pin `≤` / `sSup` to the `CompleteLattice` instance path so Challenge and
Solution elaborate `CountablyAlgebraic.mk` identically. -/
abbrev completeLatticeLE {α : Type u} [CompleteLattice α] : LE α :=
  @Preorder.toLE α
    (@PartialOrder.toPreorder α
      (@CompleteSemilatticeInf.toPartialOrder α
        (@CompleteLattice.toCompleteSemilatticeInf α ?_?)))

end DomainTheory.DataTypesAsLattices
```

completeLatticePreorder.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

abbrev completeLatticePreorder {α : Type u} [CompleteLattice α] : Preorder α :=
  @PartialOrder.toPreorder α
    (@CompleteSemilatticeInf.toPartialOrder α
      (@CompleteLattice.toCompleteSemilatticeInf α ?_?))

end DomainTheory.DataTypesAsLattices
```

completeLatticeSupSet.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Data.Finset.Basic
```

```
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

abbrev completeLatticeSupSet {α : Type u} [CompleteLattice α] : SupSet α :=
  @CompleteSemilatticeSup.toSupSet α
    (@CompleteLattice.toCompleteSemilatticeSup α ?_?)

end DomainTheory.DataTypesAsLattices
```

WayBelow.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Data.Finset.Basic
public import DomainTheory.DataTypesAsLattices.FixedLattices.DirectedSet
public import DomainTheory.DataTypesAsLattices.FixedLattices.completeLatticeLE
public import DomainTheory.DataTypesAsLattices.FixedLattices.completeLatticeSupSet
public import DomainTheory.DataTypesAsLattices.FixedLattices.completeLatticePreorder

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- The way-below relation, phrased using directed suprema. -/
def WayBelow {α : Type u} [CompleteLattice α] (x y : α) : Prop :=
  ∀ s : Set α, @DirectedSet α completeLatticePreorder s →
    @LE.le α completeLatticeLE y (@sSup α completeLatticeSupSet s) →
    ∃ z ∈ s, @LE.le α completeLatticeLE x z

end DomainTheory.DataTypesAsLattices
```

Fixpoints.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- The range / fixed-point set of a map. -/
def Fixpoints (f : Pomega → Pomega) : Set Pomega :=
  {x | f x = x}

end DomainTheory.DataTypesAsLattices
```

graph_const_bot.lean

```
module
```

```
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem graph_const_bot : graph (fun _ => botElem) = botElem := by
  ext p
  simp [graph, botElem]

end DomainTheory.DataTypesAsLattices
```

predSet_eq_ofNat_of_no_zero.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.predSet
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem predSet_eq_ofNat_of_no_zero {u : Pomega} {k : ℕ}
    (h0 : 0 ∈ u) (hk : predSet u = ofNat k) :
    u = ofNat (k + 1) := by
  ext m
  constructor
  · intro hm
    have hm0 : m = 0 := fun h => h0 (by simpa [h] using hm)
    obtain ⟨t, rfl⟩ := Nat.exists_eq_succ_of_ne_zero hm0
    have : t ∈ predSet u := hm
    have : t = k := by
      have : t ∈ ofNat k := by rwa [hk] at this
      simpa [ofNat] using this
    simp [ofNat, this]
  · intro hm
    simp [ofNat] at hm
    subst hm
    have : k ∈ predSet u := by
      rw [hk]; simp [ofNat]
    exact this

end DomainTheory.DataTypesAsLattices
```

vals.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.Fixpoints

@[expose] public section
```

```
namespace DomainTheory.DataTypesAsLattices

/-- Image of a set of fixed-point subtypes. -/
def Fixpoints.vals {f : Pomega → Pomega} (A : Set (Fixpoints f)) : Set Pomega :=
  Subtype.val '' A

namespace Fixpoints.vals

theorem subset {f : Pomega → Pomega} (A : Set (Fixpoints f)) :
    Fixpoints.vals A ⊆ Fixpoints f := by
  intro x hx
  obtain ⟨y, _, rfl⟩ := hx
  exact y.property

end Fixpoints.vals

end DomainTheory.DataTypesAsLattices
```

DirectedSubset.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.Fixpoints
public import DomainTheory.DataTypesAsLattices.Retracts.vals

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- Directedness of fixed points under subset, without a lattice instance. -/
class DirectedSubset {f : Pomega → Pomega} (s : Set (Fixpoints f)) : Prop where
  out : s.Nonempty ∧
    ∀ {{x}}, x ∈ s → ∀ {{y}}, y ∈ s →
      ∃ z ∈ s, (x : Pomega) ⊆ z ∧ (y : Pomega) ⊆ z

end DomainTheory.DataTypesAsLattices
```

directed_finite_cover.lean

```
module

public import DomainTheory.DataTypesAsLattices.FixedLattices.DirectedSet
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.Fixpoints
public import DomainTheory.DataTypesAsLattices.Retracts.vals

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem directed_finite_cover {A : Set (Fixpoints f)}
    (hA : DirectedSet A) {t : Pomega} (ht : t.Finite)
    (hsub : t ⊆ ⋃₀ (Fixpoints.vals A)) :
```

```
    ∃ z ∈ A, t ⊆ z := by
  classical
  have aux : ∀ s : Finset ℕ,
      (s : Set ℕ) ⊆ ⋃₀ (Fixpoints.vals A) →
        ∃ z ∈ A, (s : Set ℕ) ⊆ z := by
    intro s
    induction s using Finset.induction with
    | empty =>
        intro _
        obtain ⟨z, hz⟩ := hA.out.1
        exact ⟨z, hz, by simp⟩
    | insert k s hks ih =>
        intro hs
        obtain ⟨u, hu, hsu⟩ :=
          ih (fun m hm => hs (by simp [hm]))
        obtain ⟨v, hv, hkv⟩ :=
          Set.mem_sUnion.mp
            (hs (show k ∈ (insert k s : Finset ℕ) by simp))
        obtain ⟨v', hv'A, rfl⟩ := hv
        obtain ⟨z, hz, huz, hvz⟩ := hA.out.2 hu hv'A
        refine ⟨z, hz, ?_⟩
        intro m hm
        simp only [Finset.coe_insert, Set.mem_insert_iff] at hm
        rcases hm with rfl | hm
        · exact hvz hkv
        · exact huz (hsu hm)
  obtain ⟨z, hzA, hz⟩ := aux ht.toFinset (by simpa using hsub)
  exact ⟨z, hzA, fun m hm => hz (by simpa using hm)⟩

end DomainTheory.DataTypesAsLattices
```

IsCompactSubset.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.Fixpoints
public import DomainTheory.DataTypesAsLattices.Retracts.DirectedSubset
public import DomainTheory.DataTypesAsLattices.Retracts.vals

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- Compactness of a fixed point under subset, without a lattice instance. -/
def IsCompactSubset {f : Pomega → Pomega} (x : Fixpoints f) : Prop :=
  ∀ s : Set (Fixpoints f),
    DirectedSubset s →
      (x : Pomega) ⊆ ⋃₀ (Subtype.val '' s) →
      ∃ z ∈ s, (x : Pomega) ⊆ z

end DomainTheory.DataTypesAsLattices
```

openR.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (4.6).** The open-set retract. -/
def openR : Pomega :=
  graph (fun u => {m | ∃ n, e n ⊆ e m ∧ n ∈ u})

end DomainTheory.DataTypesAsLattices
```

2.5 Closure operations and algebraic lattices

Historical note. Algebraic lattices and closure operations predate the paper: Grätzer (1968) for the lattices, Tarski (1930) for the closures. Per Martin-Löf and Peter Hancock suggested that the continuous closures form a universe, and the construction of $\mathbf{V}$ is essentially theirs. The limit theorem is Scott's.

Dependencies. Inside Closures, each box is one Lean file. A box named as another folder in this chapter is that whole folder. A box named as another chapter is that whole chapter. The arrowhead points at the file or folder that imports, and the number is how many direct imports.



Figure 23: Closure operations and algebraic lattices.

Given any family of (finitary) operations on a set (say, ω) there is a closure operation defined on the subsets of that set obtained by forming the *least subset* including the given elements and *closed* under the operations. Examples are very familiar from algebra: the subgroup generated by a set of elements, the subspace spanned by a set of vectors, the convex hull of a set of geometric points. We simplify matters here by restricting attention to closures operating on sets in $\mathbf{P}\omega$, but the idea is quite general. The main point about these "algebraic" closure operations—as distinguished from topological closure operations—is that they are *continuous*. Thus, in the case of subgroups, if an element belongs to the subgroup generated by some elements, then it also belongs to the subgroup generated by *finitely* many of them. In the context of $\mathbf{P}\omega$ we can state the characteristic condition very simply.

- Definition (closure)
- Theorem 5.1
- Theorem 5.2
- Theorem 5.3
- Theorem 5.4

- Theorem 5.5
- Theorem 5.6

Isolated.lean

```
module

public import Mathlib.Algebra.Ring.Parity
public import DomainTheory.DataTypesAsLattices.Pomega.e
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- Isolated (compact) elements of `Pω` are the finite sets `e n`. -/
class Isolated (x : Pomega) : Prop where
  out : ∃ n, x = e n

end DomainTheory.DataTypesAsLattices
```

directed_finite_cover_subset.lean

```
module

public import Mathlib.Algebra.Ring.Parity
public import DomainTheory.DataTypesAsLattices.Retracts.Fixpoints
public import DomainTheory.DataTypesAsLattices.Retracts.DirectedSubset
public import DomainTheory.DataTypesAsLattices.Retracts.vals
public import DomainTheory.DataTypesAsLattices.Retracts.directed_finite_cover
public import DomainTheory.DataTypesAsLattices.FixedLattices.DirectedSet
public import DomainTheory.DataTypesAsLattices.Pomega.e
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem directed_finite_cover_subset {f : Pomega → Pomega}
    {A : Set (Fixpoints f)} (hA : DirectedSubset A) {t : Pomega}
    (ht : t.Finite) (hsub : t ⊆ ⋃₀ (Subtype.val '' A)) :
    ∃ z ∈ A, t ⊆ z := by
  have hA' : DirectedSet A := by
    refine ⟨⟨hA.out.1, ?_⟩⟩
    intro x hx y hy
    obtain ⟨z, hz, hxz, hyz⟩ := hA.out.2 hx hy
    exact ⟨z, hz, hxz, hyz⟩
  simpa [Fixpoints.vals] using directed_finite_cover hA' ht (by
    simpa [Fixpoints.vals] using hsub)

end DomainTheory.DataTypesAsLattices
```

squarePair.lean

```
module
```

```
public import Mathlib.Algebra.Ring.Parity
public import DomainTheory.DataTypesAsLattices.Pomega.e
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (5.5)-(5.7).** Disjoint pairing on sets. -/
def squarePair (x y : Pomega) : Pomega :=
  {k | (∃ n ∈ x, k = 2 * n) ∨ (∃ m ∈ y, k = 2 * m + 1)}

end DomainTheory.DataTypesAsLattices
```

IsCompact.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Data.Finset.Basic
public import DomainTheory.DataTypesAsLattices.FixedLattices.WayBelow

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- Compactness internal to a complete lattice. -/
def IsCompact {α : Type u} [CompleteLattice α] (x : α) : Prop :=
  WayBelow x x

end DomainTheory.DataTypesAsLattices
```

CountablyAlgebraic.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Data.Finset.Basic
public import DomainTheory.DataTypesAsLattices.FixedLattices.DirectedSet
public import DomainTheory.DataTypesAsLattices.FixedLattices.WayBelow
public import DomainTheory.DataTypesAsLattices.FixedLattices.IsCompact
public import DomainTheory.DataTypesAsLattices.FixedLattices.completeLatticeLE
public import DomainTheory.DataTypesAsLattices.FixedLattices.completeLatticeSupSet
public import DomainTheory.DataTypesAsLattices.FixedLattices.completeLatticePreorder

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- A concrete countable algebraic lattice: `compact` enumerates exactly the
compact elements and `density` says that they generate every point. -/
structure CountablyAlgebraic (α : Type u) [CompleteLattice α] where
  compact : ℕ → α
  compact_isCompact : ∀ n, IsCompact (compact n)
  compact_complete : ∀ x, IsCompact x → ∃ n, x = compact n
  density : ∀ x, x = @sSup α completeLatticeSupSet
    {k : α | IsCompact k ∧ @LE.le α completeLatticeLE k x}
```

```
end DomainTheory.DataTypesAsLattices
```

finiteBasisSups.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Data.Finset.Basic
public import DomainTheory.DataTypesAsLattices.FixedLattices.CountablyAlgebraic

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- Suprema generated by finite parts of a set of basis indices. -/
def finiteBasisSups {α : Type u} [CompleteLattice α]
    (d : ℕ → α) (x : Set ℕ) : Set α :=
  {y | ∃ t : Finset ℕ, (t : Set ℕ) ⊆ x ∧
    y = sSup (d '' (t : Set ℕ))}

end DomainTheory.DataTypesAsLattices
```

ContinuousBasis.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Data.Finset.Basic
public import DomainTheory.DataTypesAsLattices.FixedLattices.CountablyAlgebraic
public import DomainTheory.DataTypesAsLattices.FixedLattices.completeLatticeSupSet
public import DomainTheory.DataTypesAsLattices.FixedLattices.completeLatticePreorder
public import DomainTheory.DataTypesAsLattices.FixedLattices.WayBelow

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- A basis witnesses the continuous-lattice property by way-below elements. -/
class ContinuousBasis {α : Type u} [CompleteLattice α]
    (basis : α → Set α) : Prop where
  out : ∀ x, @DirectedSet α completeLatticePreorder (basis x) ∧
    (∀ y ∈ basis x, WayBelow y x) ∧
    x = @sSup α completeLatticeSupSet (basis x)

end DomainTheory.DataTypesAsLattices
```

finiteBasisSups_directed.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Data.Finset.Basic
public import DomainTheory.DataTypesAsLattices.FixedLattices.CountablyAlgebraic
```

```
public import DomainTheory.DataTypesAsLattices.FixedLattices.ContinuousBasis
public import DomainTheory.DataTypesAsLattices.FixedLattices.finiteBasisSups

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem finiteBasisSups_directed {α : Type u} [CompleteLattice α]
    (d : ℕ → α) (x : Set ℕ) :
    DirectedSet (finiteBasisSups d x) := by
  constructor
  constructor
  · exact ⟨sSup (d '' ((∅ : Finset ℕ) : Set ℕ)), ∅, by simp, rfl⟩
  · rintro y ⟨t, htx, rfl⟩ z ⟨u, hux, rfl⟩
    refine ⟨sSup (d '' ((t ∪ u : Finset ℕ) : Set ℕ)),
      ⟨t ∪ u, ?_, rfl⟩, ?_, ?_⟩
    · intro n hn
      simp only [Finset.coe_union, Set.mem_union] at hn
      exact hn.elim (fun h => htx h) (fun h => hux h)
    · apply sSup_le
      intro q hq
      apply le_sSup
      exact Set.image_mono (by
        intro n hn
        exact by simp [hn]) hq
    · apply sSup_le
      intro q hq
      apply le_sSup
      exact Set.image_mono (by
        intro n hn
        exact by simp [hn]) hq

end DomainTheory.DataTypesAsLattices
```

sSup_finiteBasisSups.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Data.Finset.Basic
public import DomainTheory.DataTypesAsLattices.FixedLattices.CountablyAlgebraic
public import DomainTheory.DataTypesAsLattices.FixedLattices.ContinuousBasis
public import DomainTheory.DataTypesAsLattices.FixedLattices.finiteBasisSups

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem sSup_finiteBasisSups {α : Type u} [CompleteLattice α]
    (d : ℕ → α) (x : Set ℕ) :
    sSup (finiteBasisSups d x) = sSup (d '' x) := by
  apply le_antisymm
  · apply sSup_le
    rintro y ⟨t, htx, rfl⟩
    apply sSup_le
```

```
    intro q hq
    exact le_sSup (Set.image_mono htx hq)
  · apply sSup_le
    rintro _ ⟨n, hn, rfl⟩
    calc
      d n ≤ sSup (d '' (({n} : Finset ℕ) : Set ℕ)) :=
        le_sSup ⟨n, by simp, rfl⟩
      _ ≤ sSup (finiteBasisSups d x) :=
        le_sSup ⟨{n}, by simpa, rfl⟩

end DomainTheory.DataTypesAsLattices
```

CountablyAlgebraicu2ecompact_finite_support.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Data.Finset.Basic
public import DomainTheory.DataTypesAsLattices.FixedLattices.CountablyAlgebraic
public import DomainTheory.DataTypesAsLattices.FixedLattices.ContinuousBasis
public import DomainTheory.DataTypesAsLattices.FixedLattices.finiteBasisSups
public import DomainTheory.DataTypesAsLattices.FixedLattices.finiteBasisSups_directed
public import DomainTheory.DataTypesAsLattices.FixedLattices.sSup_finiteBasisSups

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem CountablyAlgebraic.compact_finite_support
    {α : Type u} [CompleteLattice α] (D : CountablyAlgebraic α)
    {m : ℕ} {x : Set ℕ}
    (h : D.compact m ≤ sSup (D.compact '' x)) :
    ∃ t : Finset ℕ, (t : Set ℕ) ⊆ x ∧
      D.compact m ≤ sSup (D.compact '' (t : Set ℕ)) := by
  have h' : D.compact m ≤ sSup (finiteBasisSups D.compact x) := by
    rwa [sSup_finiteBasisSups]
  obtain ⟨y, ⟨t, htx, rfl⟩, hmy⟩ :=
    D.compact_isCompact m (finiteBasisSups D.compact x)
      (finiteBasisSups_directed D.compact x) h'
  exact ⟨t, htx, hmy⟩

end DomainTheory.DataTypesAsLattices
```

representationEncode.lean

```
module

public import Mathlib.Algebra.Ring.Parity
public import DomainTheory.DataTypesAsLattices.FixedLattices.CountablyAlgebraic
public import DomainTheory.DataTypesAsLattices.FixedLattices.
    CountablyAlgebraicu2ecompact_finite_support
public import DomainTheory.DataTypesAsLattices.Pomega.e
@[expose] public section

namespace DomainTheory.DataTypesAsLattices
```

```
def representationEncode {α : Type u} [CompleteLattice α]
    (D : CountablyAlgebraic α) (x : α) : Pomega :=
  {n | D.compact n ≤ x}

end DomainTheory.DataTypesAsLattices
```

representationDecode.lean

```
module

public import Mathlib.Algebra.Ring.Parity
public import DomainTheory.DataTypesAsLattices.FixedLattices.CountablyAlgebraic
public import DomainTheory.DataTypesAsLattices.FixedLattices.
    CountablyAlgebraicu2ecompact_finite_support
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.FixedLattices.IsCompact
public import DomainTheory.DataTypesAsLattices.Closures.representationEncode

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def representationDecode {α : Type u} [CompleteLattice α]
    (D : CountablyAlgebraic α) (x : Pomega) : α :=
  sSup (D.compact '' x)

namespace representationDecode

theorem encode
    {α : Type u} [CompleteLattice α] (D : CountablyAlgebraic α) (x : α) :
    representationDecode D (representationEncode D x) = x := by
  apply le_antisymm
  · apply sSup_le
    rintro _ ⟨n, hn, rfl⟩
    exact hn
  · calc
      x = sSup {k : α | IsCompact k ∧ k ≤ x} := D.density x
      _ ≤ representationDecode D (representationEncode D x) := by
        apply sSup_le
        rintro k ⟨hkc, hkx⟩
        obtain ⟨n, rfl⟩ := D.compact_complete k hkc
        exact le_sSup ⟨n, hkx, rfl⟩

end representationDecode

end DomainTheory.DataTypesAsLattices
```

exists_mem_of_ne_bot.lean

```
module

public import Mathlib.Algebra.Ring.Parity
public import DomainTheory.DataTypesAsLattices.Pomega.e
@[expose] public section
```

```
namespace DomainTheory.DataTypesAsLattices

theorem exists_mem_of_ne_bot {x : Pomega} (h : x = botElem) : ∃ m, m ∈ x := by
  by_contra hc
  refine h ?_
  ext n
  simp only [mem_botElem, iff_false]
  exact fun hn => hc ⟨n, hn⟩

end DomainTheory.DataTypesAsLattices
```

predSet_succSet.lean

```
module

public import Mathlib.Algebra.Ring.Parity
public import DomainTheory.DataTypesAsLattices.Lambda.succSet
public import DomainTheory.DataTypesAsLattices.Lambda.predSet
public import DomainTheory.DataTypesAsLattices.Pomega.e
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem predSet_succSet (x : Pomega) : predSet (succSet x) = x := by
  ext k
  constructor
  · intro ⟨n, hn, heq⟩
    exact (Nat.succ_injective heq) ▸ hn
  · intro hk
    exact ⟨k, hk, rfl⟩

end DomainTheory.DataTypesAsLattices
```

predSet_union.lean

```
module

public import Mathlib.Algebra.Ring.Parity
public import DomainTheory.DataTypesAsLattices.Lambda.predSet
public import DomainTheory.DataTypesAsLattices.Pomega.e
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem predSet_union (x y : Pomega) :
    predSet (x ∪ y) = predSet x ∪ predSet y := by
  ext k; simp [predSet]

end DomainTheory.DataTypesAsLattices
```

topElem_ne_botElem.lean

```
module
```

```
public import Mathlib.Algebra.Ring.Parity
public import DomainTheory.DataTypesAsLattices.Pomega.e
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem topElem_ne_botElem : (topElem : Pomega) = botElem := by
  intro h
  have h0 : (0 : ℕ) ∈ (botElem : Pomega) := h ▸ Set.mem_univ 0
  simp [botElem] at h0

end DomainTheory.DataTypesAsLattices
```

union_botElem.lean

```
module

public import Mathlib.Algebra.Ring.Parity
public import DomainTheory.DataTypesAsLattices.Pomega.e
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem union_botElem (x : Pomega) : x ∪ botElem = x := Set.union_empty x

end DomainTheory.DataTypesAsLattices
```

botElem_union.lean

```
module

public import Mathlib.Algebra.Ring.Parity
public import DomainTheory.DataTypesAsLattices.Pomega.e
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem botElem_union (x : Pomega) : botElem ∪ x = x := Set.empty_union x

end DomainTheory.DataTypesAsLattices
```

2.6 Subsets and their classification

Historical note. The classification of these subsets by notions from descriptive set theory, and the connection with Wadge reducibility, are in the work of Adrian Tang. The normal forms for the classes, such as Table 3, are Scott's.

Dependencies. Inside Classification, each box is one Lean file. A box named as another folder in this chapter is that whole folder. A box named as another chapter is that whole chapter. The arrowhead points at the file or folder that imports, and the number is how many direct imports.



Figure 24: Subsets and their classification.

Retracts produce very special subsets of $\mathbf{P}\omega$: a retract always has a nonempty range which forms a lattice under $\sqsubseteq$. For example, the range of **int** is $\{\bot, \top\} \cup \omega$. We often wish to eliminate $\bot$ and $\top$; and with a retract like **tree** the situation is more complex, since combinations like $\langle 1, \langle\langle 1, \langle \bot, \langle 0\rangle\rangle\rangle, \top\rangle\rangle$ might require elimination. In these two cases the method is simple.

Consider these two functions:

$$\mathbf{mid} = \lambda x : \mathbf{int}.\, x \supset 0, 0 \tag{6.1}$$

$$\mathbf{perf} = \lambda u : \mathbf{tree}.\, \mathbf{which}(u) \supset 0,\ \Delta(\mathbf{perf}(\mathbf{fst}(\mathbf{out}(u))))(\mathbf{perf}(\mathbf{snd}(\mathbf{out}(u)))) \tag{6.2}$$

where Δ is a special combinator:

$$\Delta = \lambda x \lambda y.\, (x \supset (y \supset 0, \top), \top) \cup (y \supset (x \supset 0, \top), \top). \tag{6.3}$$

We find that $\omega = \{x : \mathbf{int} \mid \mathbf{mid}(x) = 0\}$. In the case of trees, note first this behavior of Δ:

Δ	$\bot$	0	$\top$
$\bot$	$\bot$	$\bot$	$\top$
0	$\bot$	0	$\top$
$\top$	$\top$	$\top$	$\top$

The question is: what subset is $\{u : \mathbf{tree} \mid \mathbf{perf}(u) = 0\}$?

Now **perf** is defined recursively. We can see that

$$\mathbf{perf}(\bot) = \bot, \quad \mathbf{perf}(\top) = \top, \quad \mathbf{perf}(\langle 0\rangle) = 0,$$

and

$$\mathbf{perf}(\langle 1, \langle x, y\rangle\rangle) = \Delta(\mathbf{perf}(x))(\mathbf{perf}(y))$$

when $x, y : \mathbf{tree}$. Every tree, aside from $\top$ or $\bot$, is either *atomic* or a pair of trees. The atomic tree is "perfect" (that is, $\mathbf{perf}(\langle 0\rangle) = 0$). A finite tree which does not contain $\bot$ or $\top$ is perfect—as we can see inductively using the table above for Δ. An infinite tree is never perfect: either some branch ends in $\top$ and **perf** maps it to $\top$, or $\top$ is never reached and **perf** maps it to $\bot$. Thus the subset in question is then seen to be the set of *finite* trees generated from the atom by pairing. This is clearly a desirable subset, and it is sorted out by a function with a simple recursive definition. The general question is: what subsets can be characterized by equations? The answer can be given by reference to the topology of $\mathbf{P}\omega$.

- Definition (classes)

- Theorem 6.1
- Theorem 6.2
- Theorem 6.3
- Theorem 6.4
- Theorem 6.5
- Theorem 6.6
- Theorem 6.7

IsScottOpen.lean

```
module

public import Mathlib.Topology.Bases
public import DomainTheory.DataTypesAsLattices.Continuous.basicNhhd
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace

/-- **Scott 1976, §1, Definition.** An open set is a union of basic neighbourhoods. -/
def IsScottOpen (U : Set Pomega) : Prop :=
  ∃ B : Set ℕ, U = ⋃ n ∈ B, basicNhhd n

end DomainTheory.DataTypesAsLattices
```

IsBooleanOpen.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottOpen
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import Mathlib.Data.Set.Finite.Powerset

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace

inductive IsBooleanOpen : Set Pomega → Prop
```

```
  | ofOpen {U} : IsScottOpen U → IsBooleanOpen U
  | union {U V} : IsBooleanOpen U → IsBooleanOpen V → IsBooleanOpen (U ∪ V)
  | inter {U V} : IsBooleanOpen U → IsBooleanOpen V → IsBooleanOpen (U ∩ V)
  | compl {U} : IsBooleanOpen U → IsBooleanOpen U?

end DomainTheory.DataTypesAsLattices
```

isScottOpen_iff_isOpen.lean

```
module

public import Mathlib.Topology.Bases
public import DomainTheory.DataTypesAsLattices.Continuous.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.isOpen_basicNhhd
public import DomainTheory.DataTypesAsLattices.Continuous.
    exists_basicNhhd_subset_of_generateOpen
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottOpen
public import Mathlib.Data.Set.Lattice.Indexed
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Continuous.memOpen
public import DomainTheory.DataTypesAsLattices.Continuous.basicNhhd

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace

/-- **Scott 1976, §1.** Scott-open sets are exactly the opens of the
positive information topology. -/
theorem isScottOpen_iff_isOpen (U : Set Pomega) : IsScottOpen U ↔ IsOpen U := by
  constructor
  · intro ⟨B, hU⟩
    rw [hU]
    exact isOpen_biUnion fun n _ => isOpen_basicNhhd n
  · intro hU
    refine ⟨{n | basicNhhd n ⊆ U}, ?_⟩
    ext x
    constructor
    · intro hx
      have hgen : GenerateOpen (Set.range memOpen) U := hU
      obtain ⟨n, hn, hsub⟩ := exists_basicNhhd_subset_of_generateOpen hgen hx
      exact Set.mem_iUnion₂.mpr ⟨n, hsub, hn⟩
    · intro hx
      obtain ⟨n, hn, hxn⟩ := Set.mem_iUnion₂.mp hx
      exact hn hxn

end DomainTheory.DataTypesAsLattices
```

isScottOpen_isUpper.lean

```
module

public import Mathlib.Topology.Bases
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottOpen
```

```
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Data.Set.Lattice.Indexed
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace

/-- Open sets are upper sets: if `x ∈ U` and `x ⊆ y` then `y ∈ U`. -/
theorem isScottOpen_isUpper {U : Set Pomega} (hU : IsScottOpen U)
    {x y : Pomega} (hx : x ∈ U) (hxy : x ⊆ y) : y ∈ U := by
  obtain ⟨B, rfl⟩ := hU
  obtain ⟨n, hn, hxn⟩ := Set.mem_iUnion₂.mp hx
  exact Set.mem_iUnion₂.mpr ⟨n, hn, subset_trans hxn hxy⟩

end DomainTheory.DataTypesAsLattices
```

IsScottContinuous.lean

```
module

public import Mathlib.Topology.Bases
public import DomainTheory.DataTypesAsLattices.Continuous.scottUnion
public import DomainTheory.DataTypesAsLattices.Continuous.mem_scottUnion
public import DomainTheory.DataTypesAsLattices.Continuous.isScottOpen_isUpper
public import DomainTheory.DataTypesAsLattices.Continuous.isScottOpen_iff_isOpen
public import Mathlib.Data.Set.Lattice.Indexed
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottOpen
public import DomainTheory.DataTypesAsLattices.Continuous.scottPiece
public import DomainTheory.DataTypesAsLattices.Continuous.memOpen
public import DomainTheory.DataTypesAsLattices.Continuous.basicNhhd

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace

/-- **Scott 1976, §1, Definition.** `f : Pω → Pω` is continuous iff
`f(x) = ⋃ {f(e n) | e n ⊆ x}`. -/
class IsScottContinuous (f : Pomega → Pomega) : Prop where
  out : ∀ x, f x = scottUnion f x

namespace IsScottContinuous

theorem monotone {f : Pomega → Pomega}
    (hf : IsScottContinuous f) {x y : Pomega} (h : x ⊆ y) : f x ⊆ f y := by
  intro m hm
  rw [hf.out x, mem_scottUnion] at hm
  rw [hf.out y, mem_scottUnion]
  obtain ⟨n, hn, hmn⟩ := hm
  exact ⟨n, subset_trans hn h, hmn⟩
```

```
/-- Continuous functions are determined by their values on finite sets. -/
theorem determined {f g : Pomega → Pomega}
    (hf : IsScottContinuous f) (hg : IsScottContinuous g)
    (hfin : ∀ n, f (e n) = g (e n)) : f = g := by
  funext x
  rw [hf.out x, hg.out x]
  simp [scottUnion, scottPiece, hfin]

/-- Scott continuity coincides with topological continuity `Pω → Pω`. -/
theorem iff_continuous (f : Pomega → Pomega) :
    IsScottContinuous f ↔ Continuous f := by
  constructor
  · intro hf
    refine continuous_generateFrom_iff.2 ?_
    rintro _ ⟨k, rfl⟩
    rw [← isScottOpen_iff_isOpen]
    refine ⟨{n | k ∈ f (e n)}, ?_⟩
    ext x
    constructor
    · intro hx
      have : k ∈ scottUnion f x := by rwa [← hf.out x]
      obtain ⟨n, hn, hkn⟩ := mem_scottUnion.mp this
      exact Set.mem_iUnion₂.mpr ⟨n, hkn, hn⟩
    · intro hx
      obtain ⟨n, hkn, hn⟩ := Set.mem_iUnion₂.mp hx
      exact monotone hf hn hkn
  · intro hfcont
    constructor
    intro x
    ext k
    have hopen : IsScottOpen (f ^-1' memOpen k) :=
      (isScottOpen_iff_isOpen _).mpr (IsOpen.preimage hfcont (isOpen_memOpen k))
    constructor
    · intro hk
      obtain ⟨B, hB⟩ := hopen
      have hx : x ∈ ⋃ n ∈ B, basicNhhd n := by
        have : x ∈ f ^-1' memOpen k := hk
        rwa [hB] at this
      obtain ⟨n, hn, hxn⟩ := Set.mem_iUnion₂.mp hx
      have hen : k ∈ f (e n) := by
        have : e n ∈ f ^-1' memOpen k := by
          rw [hB]
          exact Set.mem_iUnion₂.mpr ⟨n, hn, (subset_rfl : e n ⊆ e n)⟩
        exact this
      exact mem_scottUnion.mpr ⟨n, hxn, hen⟩
    · intro hk
      obtain ⟨n, hn, hkn⟩ := mem_scottUnion.mp hk
      have : e n ∈ f ^-1' memOpen k := hkn
      exact isScottOpen_isUpper hopen this hn

end IsScottContinuous

end DomainTheory.DataTypesAsLattices
```

ScottContinuousFin.lean

```
module

public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- The recursive content of finite Scott continuity, one argument at a time. -/
def ScottContinuousFin.Body : ∀ {n : ℕ}, ((Fin n → Pomega) → Pomega) → Prop
  | 0, _ => True
  | n + 1, f =>
      (∀ xs : Fin n → Pomega, IsScottContinuous (fun x => f (Fin.cons x xs))) ∧
      (∀ x : Pomega, ScottContinuousFin.Body (fun xs : Fin n → Pomega => f (Fin.cons x
    xs)))

/-- Continuity of an n-ary map, taken one argument at a time from the left. -/
class ScottContinuousFin {n : ℕ} (f : (Fin n → Pomega) → Pomega) : Prop where
  out : ScottContinuousFin.Body f

end DomainTheory.DataTypesAsLattices
```

succSet_more.lean

```
module

public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.mem_scottUnion
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.succSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace succSet

theorem isScottContinuous : IsScottContinuous succSet := by
  constructor
  intro x
  ext k
  constructor
```

```
  · intro ⟨n, hn, hk⟩
    refine mem_scottUnion.mpr ⟨2 ^ n, by simp [e_pow2, Set.singleton_subset_iff, hn], ?
    _⟩
    exact ⟨n, by simp [e_pow2], hk⟩
  · intro hk
    obtain ⟨m, hm, n, hn, hk⟩ := mem_scottUnion.mp hk
    exact ⟨n, hm hn, hk⟩

theorem ofNat (j : ℕ) : succSet (ofNat j) = ofNat (j + 1) := by
  ext k
  simp [succSet]

end succSet

end DomainTheory.DataTypesAsLattices
```

condSet.lean

```
module

public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.mem_scottUnion
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import Mathlib.Data.Nat.Bitwise

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, Table 2.** McCarthy conditional. -/
def condSet (z x y : Pomega) : Pomega :=
  {n | n ∈ x ∧ 0 ∈ z} ∪ {m | m ∈ y ∧ ∃ k, k + 1 ∈ z}

namespace condSet

theorem bot (x y : Pomega) : condSet botElem x y = botElem := by
  ext n
  simp [condSet, botElem]

theorem ofNat_zero (x y : Pomega) : condSet (ofNat 0) x y = x := by
  ext n
  simp [condSet, ofNat]

theorem ofNat_one (x y : Pomega) : condSet (ofNat 1) x y = y := by
  ext n
  simp [condSet, ofNat]

theorem ofNat_succ (n : ℕ) (x y : Pomega) :
    condSet (ofNat (n + 1)) x y = y := by
  ext k
```

```
  simp [condSet, ofNat]

theorem same_bot (x : Pomega) : condSet botElem x x = botElem :=
  bot x x

theorem same_of_mem {z x : Pomega} {k : ℕ} (hk : k ∈ z) :
    condSet z x x = x := by
  ext n
  constructor
  · intro hn
    rcases hn with ⟨hnx, _⟩ | ⟨hnx, _⟩
    · exact hnx
    · exact hnx
  · intro hnx
    cases k with
    | zero => exact Or.inl ⟨hnx, hk⟩
    | succ t => exact Or.inr ⟨hnx, t, hk⟩

theorem same (z x : Pomega) (hz : z = botElem) : condSet z x x = x := by
  obtain ⟨k, hk⟩ : ∃ k, k ∈ z := by
    by_contra h
    push Not at h
    exact hz (Set.ext fun n => by simp [botElem, h n])
  exact same_of_mem hk

theorem isScottContinuous_left (x y : Pomega) :
    IsScottContinuous (fun z => condSet z x y) := by
  constructor
  intro z
  ext k
  constructor
  · intro hk
    rcases hk with ⟨hkx, h0⟩ | ⟨hky, ⟨t, ht⟩⟩
    · have e1 : e 1 = ({0} : Pomega) := by simpa using e_pow2 0
      refine mem_scottUnion.mpr ⟨1, ?_, Or.inl ⟨hkx, ?_⟩⟩
      · simpa [e1, Set.singleton_subset_iff] using h0
      · simp [e1]
    · refine mem_scottUnion.mpr ⟨2 ^ (t + 1), ?_, Or.inr ⟨hky, t, ?_⟩⟩
      · simp [e_pow2, Set.singleton_subset_iff, ht]
      · simp [e_pow2]
  · intro hk
    obtain ⟨m, hm, hkm⟩ := mem_scottUnion.mp hk
    rcases hkm with ⟨hkx, h0⟩ | ⟨hky, ⟨t, ht⟩⟩
    · exact Or.inl ⟨hkx, hm h0⟩
    · exact Or.inr ⟨hky, t, hm ht⟩

theorem isScottContinuous_mid (z y : Pomega) :
    IsScottContinuous (fun x => condSet z x y) := by
  constructor
  intro x
  ext k
  constructor
  · intro hk
    rcases hk with ⟨hkx, h0⟩ | hky
    · refine mem_scottUnion.mpr ⟨2 ^ k, by simp [e_pow2, Set.singleton_subset_iff, hkx
    ],
```

```
        Or.inl ⟨by simp [e_pow2], h0⟩
    · refine mem_scottUnion.mpr ⟨0, by simp [e_zero], Or.inr hky⟩
  · intro hk
    obtain ⟨m, hm, hkm⟩ := mem_scottUnion.mp hk
    rcases hkm with ⟨hkx, h0⟩ | hky
    · exact Or.inl ⟨hm hkx, h0⟩
    · exact Or.inr hky

theorem isScottContinuous_right (z x : Pomega) :
    IsScottContinuous (fun y => condSet z x y) := by
  constructor
  intro y
  ext k
  constructor
  · intro hk
    rcases hk with hzx | ⟨hky, ht⟩
    · refine mem_scottUnion.mpr ⟨0, by simp [e_zero], Or.inl hzx⟩
    · refine mem_scottUnion.mpr ⟨2 ^ k, by simp [e_pow2, Set.singleton_subset_iff, hky
    ],
        Or.inr ⟨by simp [e_pow2], ht⟩⟩
  · intro hk
    obtain ⟨m, hm, hkm⟩ := mem_scottUnion.mp hk
    rcases hkm with hzx | ⟨hky, ht⟩
    · exact Or.inl hzx
    · exact Or.inr ⟨hm hky, ht⟩

private theorem exists_pos_bit {n : ℕ} (hn : 2 ≤ n) :
    ∃ k, n.testBit (k + 1) = true := by
  by_contra hex
  push Not at hex
  have hfalse : ∀ k, n.testBit (k + 1) = false := fun k =>
    Bool.eq_false_iff.2 (hex k)
  have : n = 0 ∨ n = 1 := by
    cases h0 : n.testBit 0 with
    | false =>
      exact Or.inl <| Nat.zero_of_testBit_eq_false fun i => by
        cases i with
        | zero => exact h0
        | succ k => exact hfalse k
    | true =>
      exact Or.inr <| Nat.eq_of_testBit_eq fun i => by
        cases i with
        | zero => simp [h0]
        | succ k =>
          rw [hfalse k]
          exact (Nat.testBit_eq_false_of_lt
            (Nat.one_lt_two_pow (Nat.succ_ne_zero k))).symm
  omega

theorem e_iff (n0 n1 n2 m : ℕ) :
    m ∈ condSet (e n2) (e n0) (e n1) ↔
      (n0.testBit m = true ∧ n2.testBit 0 = true) ∨
        (n1.testBit m = true ∧ 2 ≤ n2) := by
  constructor
  · intro h
    rcases h with ⟨hm, h0⟩ | ⟨hm, ⟨k, hk⟩⟩
```

```
    · exact Or.inl ⟨mem_e.mp hm, mem_e.mp h0⟩
    · exact Or.inr ⟨mem_e.mp hm,
        (Nat.pow_le_pow_right (by decide : 0 < (2 : ℕ))
          (Nat.le_add_left 1 k) : (2 : ℕ) ^ 1 ≤ 2 ^ (k + 1)).trans
          (two_pow_le_of_testBit (mem_e.mp hk))⟩
  · intro h
    rcases h with ⟨hm, h0⟩ | ⟨hm, hn2⟩
    · exact Or.inl ⟨mem_e.mpr hm, mem_e.mpr h0⟩
    · obtain ⟨k, hk⟩ := exists_pos_bit hn2
      exact Or.inr ⟨mem_e.mpr hm, k, mem_e.mpr hk⟩

end condSet

end DomainTheory.DataTypesAsLattices
```

predSet_more.lean

```
module

public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.mem_scottUnion
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.predSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace predSet

theorem isScottContinuous : IsScottContinuous predSet := by
  constructor
  intro x
  ext k
  constructor
  · intro hk
    refine mem_scottUnion.mpr ⟨2 ^ (k + 1), ?_, ?_⟩
    · simpa [e_pow2, Set.singleton_subset_iff, predSet] using hk
    · simp [predSet, e_pow2]
  · intro hk
    obtain ⟨m, hm, hkm⟩ := mem_scottUnion.mp hk
    exact hm hkm

theorem ofNat_succ (j : ℕ) : predSet (ofNat (j + 1)) = ofNat j := by
  ext k
  simp [predSet, ofNat]

end predSet

end DomainTheory.DataTypesAsLattices
```

eq_2_7_succ.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem eq_2_7_succ (x y : Pomega) (k : ℕ) :
    condSet (ofNat (k + 1)) x y = y := by
  ext n
  simp [condSet, ofNat]

end DomainTheory.DataTypesAsLattices
```

eq_2_7_zero.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem eq_2_7_zero (x y : Pomega) : condSet (ofNat 0) x y = x := by
  ext n
  simp [condSet, ofNat]

end DomainTheory.DataTypesAsLattices
```

eq_2_7_bot.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.condSet

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.7).** Conditional cases. -/
theorem eq_2_7_bot (x y : Pomega) : condSet botElem x y = botElem := by
  ext n
  simp [condSet, botElem]
```

```
end DomainTheory.DataTypesAsLattices
```

eq_2_7_mix.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem eq_2_7_mix (x y z : Pomega) (h0 : 0 ∈ z) (hne : z = ofNat 0) :
    condSet z x y = x ∪ y := by
  have hpos : ∃ t, t + 1 ∈ z := by
    by_contra h
    apply hne
    ext k
    constructor
    · intro hk
      have : k = 0 := by
        by_contra hk0
        exact h ⟨k - 1, by
          have : k = (k - 1) + 1 := (Nat.succ_pred_eq_of_ne_zero hk0).symm
          rwa [← this]⟩
      simp [this, ofNat]
    · intro hk
      simp [ofNat] at hk
      subst hk
      exact h0
  ext n
  constructor
  · intro hn
    rcases hn with ⟨hnx, _⟩ | ⟨hny, _⟩
    · exact Or.inl hnx
    · exact Or.inr hny
  · intro hn
    rcases hn with hnx | hny
    · exact Or.inl ⟨hnx, h0⟩
    · exact Or.inr ⟨hny, hpos⟩

end DomainTheory.DataTypesAsLattices
```

eq_2_7_pos.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.condSet

@[expose] public section
```

```
namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.7), remaining cases.** -/
theorem eq_2_7_pos (x y z : Pomega) (h0 : 0 ∈ z) (hne : z = botElem) :
    condSet z x y = y := by
  have hpos : ∃ t, t + 1 ∈ z := by
    obtain ⟨k, hk⟩ := Set.nonempty_iff_ne_empty.mpr hne
    have : k = 0 := fun h => h0 (h ▸ hk)
    exact ⟨k - 1, by
      have : k = (k - 1) + 1 := (Nat.succ_pred_eq_of_ne_zero this).symm
      rwa [← this]⟩
  ext n
  constructor
  · intro hn
    rcases hn with ⟨_, hz⟩ | ⟨hny, _⟩
    · exact (h0 hz).elim
    · exact hny
  · intro hny
    exact Or.inr ⟨hny, hpos⟩

end DomainTheory.DataTypesAsLattices
```

condC.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.condSet

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def condC : Pomega :=
  graph (fun x => graph (fun y => graph (fun z => condSet z x y)))

end DomainTheory.DataTypesAsLattices
```

graph_const_isScottContinuous.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Continuous.mem_scottUnion
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Continuous.scottUnion
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- `graph (fun y => g x y)` is continuous in `x` when `g` is continuous in `x`. -/
```

```
theorem graph_const_isScottContinuous
    (g : Pomega → Pomega → Pomega)
    (hg : ∀ y, IsScottContinuous (fun x => g x y)) :
    IsScottContinuous (fun x => graph (fun y => g x y)) := by
  constructor
  intro x
  ext k
  constructor
  · intro hk
    rcases hk with ⟨n, m, rfl, hm⟩
    have : m ∈ scottUnion (fun z => g z (e n)) x := by
      rw [← (hg (e n)).out x]; exact hm
    obtain ⟨p, hp, hmp⟩ := mem_scottUnion.mp this
    exact mem_scottUnion.mpr ⟨p, hp, n, m, rfl, hmp⟩
  · intro hk
    obtain ⟨p, hp, n, m, rfl, hm⟩ := mem_scottUnion.mp hk
    exact ⟨n, m, rfl, IsScottContinuous.monotone (hg (e n)) hp hm⟩

end DomainTheory.DataTypesAsLattices
```

const_isScottContinuous.lean

```
module

public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.mem_scottUnion
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem const_isScottContinuous (c : Pomega) :
    IsScottContinuous (fun _ : Pomega => c) := by
  constructor
  intro x
  ext k
  constructor
  · intro hk
    exact mem_scottUnion.mpr ⟨0, by simp [e_zero], hk⟩
  · intro hk
    obtain ⟨_, _, hkn⟩ := mem_scottUnion.mp hk
    exact hkn

end DomainTheory.DataTypesAsLattices
```

id_isScottContinuous.lean

```
module

public import Mathlib.Topology.Bases
```

```
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.mem_scottUnion
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem id_isScottContinuous : IsScottContinuous (fun x : Pomega => x) := by
  constructor
  intro x
  ext k
  constructor
  · intro hk
    exact mem_scottUnion.mpr ⟨2 ^ k, by simp [e_pow2, Set.singleton_subset_iff, hk],
      by simp [e_pow2]⟩
  · intro hk
    obtain ⟨n, hn, hkn⟩ := mem_scottUnion.mp hk
    exact hn hkn

end DomainTheory.DataTypesAsLattices
```

eq_2_15.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.pair_zero_zero
public import DomainTheory.DataTypesAsLattices.Lambda.pair_one_zero
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.15).** `x ∪ y = (λz. 0) ? x, y`. -/
theorem eq_2_15 (x y : Pomega) :
    condSet (graph (fun _ => ofNat 0)) x y = x ∪ y := by
  have h0 : 0 ∈ graph (fun _ => ofNat 0) :=
    ⟨0, 0, pair_zero_zero.symm, by simp [ofNat]⟩
  have h1 : 1 ∈ graph (fun _ => ofNat 0) :=
    ⟨1, 0, pair_one_zero.symm, by simp [ofNat]⟩
  ext n
  constructor
  · intro hn
    rcases hn with ⟨hnx, _⟩ | ⟨hny, _⟩
    · exact Or.inl hnx
    · exact Or.inr hny
  · intro hn
```

```
    rcases hn with hnx | hny
    · exact Or.inl ⟨hnx, h0⟩
    · exact Or.inr ⟨hny, 0, h1⟩

end DomainTheory.DataTypesAsLattices
```

topRec.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.succSet
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.16).** `⊤ = Y(λx. 0 ∪ (x + 1))`. -/
def topRec (x : Pomega) : Pomega := ofNat 0 ∪ succSet x

namespace topRec

theorem isScottContinuous : IsScottContinuous topRec := by
  constructor
  intro x
  ext k
  constructor
  · intro hk
    rcases hk with hk | ⟨n, hn, rfl⟩
    · exact mem_scottUnion.mpr ⟨0, by simp [e_zero], Or.inl hk⟩
    · exact mem_scottUnion.mpr ⟨2 ^ n, by simp [e_pow2, Set.singleton_subset_iff, hn],
        Or.inr ⟨n, by simp [e_pow2], rfl⟩⟩
  · intro hk
    obtain ⟨m, hm, hkm⟩ := mem_scottUnion.mp hk
    rcases hkm with hk | ⟨n, hn, rfl⟩
    · exact Or.inl hk
    · exact Or.inr ⟨n, hm hn, rfl⟩

end topRec

end DomainTheory.DataTypesAsLattices
```

eq_2_16.lean

```
module

public import DomainTheory.DataTypesAsLattices.FixedPoint.iterateBot
public import DomainTheory.DataTypesAsLattices.FixedPoint.fix
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.topRec
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
```

```
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem eq_2_16 : fix topRec = topElem := by
  apply subset_antisymm
  · intro k _; trivial
  · intro k _
    have hiter : ∀ n, n ∈ iterateBot topRec (n + 1) := by
      intro n
      induction n with
      | zero => simp [iterateBot, topRec, ofNat]
      | succ n ih =>
        refine Or.inr ⟨n, ?_, rfl⟩
        simpa [iterateBot] using ih
    exact Set.mem_iUnion.mpr ⟨k + 1, hiter k⟩

end DomainTheory.DataTypesAsLattices
```

sUnion_Fixpoints_le_image.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.Fixpoints
public import DomainTheory.DataTypesAsLattices.Retracts.vals

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem sUnion_Fixpoints_le_image {f : Pomega → Pomega} (hf : IsScottContinuous f)
    {A : Set Pomega} (hA : A ⊆ Fixpoints f) :
    ⋃₀ A ⊆ f (⋃₀ A) := by
  intro k hk
  obtain ⟨x, hxA, hkx⟩ := Set.mem_sUnion.mp hk
  have hx : f x = x := hA hxA
  have : k ∈ f x := by rwa [hx]
  exact IsScottContinuous.monotone hf (Set.subset_sUnion_of_mem hxA) this

end DomainTheory.DataTypesAsLattices
```

gfpBelow.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
```

```
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.Fixpoints
public import DomainTheory.DataTypesAsLattices.Retracts.vals

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- Greatest postfixed point of `z ↦ f z ∩ b` (Knaster-Tarski inf below `b`). -/
def gfpBelow (f : Pomega → Pomega) (b : Pomega) : Pomega :=
  ⋃₀ {x | x ⊆ f x ∩ b}

namespace gfpBelow

theorem subset (f : Pomega → Pomega) (b : Pomega) :
    gfpBelow f b ⊆ b := by
  intro k hk
  obtain ⟨x, hx, hkx⟩ := Set.mem_sUnion.mp hk
  exact (hx hkx).2

theorem postfixed {f : Pomega → Pomega} (hf : IsScottContinuous f)
    (b : Pomega) : gfpBelow f b ⊆ f (gfpBelow f b) ∩ b := by
  intro k hk
  obtain ⟨x, hx, hkx⟩ := Set.mem_sUnion.mp hk
  have hxz : x ⊆ gfpBelow f b := fun _ hm => Set.mem_sUnion.mpr ⟨x, hx, hm⟩
  exact ⟨IsScottContinuous.monotone hf hxz (hx hkx).1, (hx hkx).2⟩

theorem le {f : Pomega → Pomega} (_hf : IsScottContinuous f) (b : Pomega)
    (_hb : f b ⊆ b) : gfpBelow f b ⊆ b :=
  subset f b

theorem fixed_inter {f : Pomega → Pomega} (hf : IsScottContinuous f)
    (b : Pomega) : f (gfpBelow f b) ∩ b = gfpBelow f b := by
  apply subset_antisymm
  · intro k hk
    refine Set.mem_sUnion.mpr ⟨f (gfpBelow f b) ∩ b, ?_, hk⟩
    have hle : gfpBelow f b ⊆ f (gfpBelow f b) ∩ b := postfixed hf b
    have : f (gfpBelow f b) ⊆ f (f (gfpBelow f b) ∩ b) :=
      IsScottContinuous.monotone hf hle
    intro m hm
    exact ⟨this hm.1, hm.2⟩
  · exact postfixed hf b

theorem isGLB {f : Pomega → Pomega} (_hf : IsScottContinuous f)
    {A : Set Pomega} (_hA : A ⊆ Fixpoints f) {z : Pomega}
    (hz : f z = z) (hlb : ∀ x ∈ A, z ⊆ x) :
    z ⊆ gfpBelow f (⋂₀ A) := by
  have hzI : z ⊆ ⋂₀ A := fun k hk y hy => hlb y hy hk
  have hpost : z ⊆ f z ∩ ⋂₀ A := fun k hk => ⟨by rwa [hz], hzI hk⟩
  exact fun k hk => Set.mem_sUnion.mpr ⟨z, hpost, hk⟩

end gfpBelow
```

```
end DomainTheory.DataTypesAsLattices
```

iterateFrom.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.FixedPoint.iterateBot

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- Directed iterates of `f` starting from `x` (Scott's sup construction). -/
def iterateFrom (f : Pomega → Pomega) (x : Pomega) : ℕ → Pomega
  | 0 => x
  | n + 1 => f (iterateFrom f x n)

namespace iterateFrom

theorem bot (f : Pomega → Pomega) (n : ℕ) :
    iterateFrom f botElem n = iterateBot f n := by
  induction n with
  | zero => rfl
  | succ n ih => simp [iterateFrom, iterateBot, ih]

theorem mono {f : Pomega → Pomega} (hf : IsScottContinuous f)
    {x : Pomega} (hx : x ⊆ f x) :
    ∀ n, iterateFrom f x n ⊆ iterateFrom f x (n + 1)
  | 0 => hx
  | n + 1 => IsScottContinuous.monotone hf (mono hf hx n)

end iterateFrom

end DomainTheory.DataTypesAsLattices
```

IsComputable.lean

```
module

public import Mathlib.Computability.RE
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
```

```
public import DomainTheory.DataTypesAsLattices.Computability.IsRE

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, §2, Definition.** A continuous `k`-ary map is computable
when membership in its values on finite arguments is r.e. Unary case: -/
def IsComputable (f : Pomega → Pomega) : Prop :=
  IsScottContinuous f ∧ IsRE (graph f)

end DomainTheory.DataTypesAsLattices
```

exists_e_cover_finite.lean

```
module

public import Mathlib.Topology.Bases
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace

/-- If each element of a finite set is realized at some `e n ⊆ x`, the
bitwise-OR of those codes realizes the whole finite set (using monotonicity). -/
theorem exists_e_cover_finite {f : Pomega → Pomega} (hf : IsScottContinuous f)
    {x : Pomega} (s : Finset ℕ)
    (h : ∀ k ∈ s, ∃ n, e n ⊆ x ∧ k ∈ f (e n)) :
    ∃ N, e N ⊆ x ∧ (s : Set ℕ) ⊆ f (e N) := by
  induction s using Finset.induction with
  | empty => exact ⟨0, by simp [e_zero], by simp⟩
  | insert k s hks ih =>
    obtain ⟨N, hN, hs⟩ := ih fun j hj => h j (Finset.mem_insert_of_mem hj)
    obtain ⟨n, hn, hk⟩ := h k (Finset.mem_insert_self _ _)
    refine ⟨n ||| N, e_or_of_subset hn hN, ?_⟩
    intro j hj
    rw [Finset.coe_insert, Set.mem_insert_iff] at hj
    rcases hj with rfl | hj
    · have : e n ⊆ e (n ||| N) := by rw [e_or]; exact Set.subset_union_left
      exact IsScottContinuous.monotone hf this hk
    · have : e N ⊆ e (n ||| N) := by rw [e_or]; exact Set.subset_union_right
      exact IsScottContinuous.monotone hf this (hs hj)

end DomainTheory.DataTypesAsLattices
```

theorem_1_1.lean

```
module
```

```
public import Mathlib.Topology.Bases
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Continuous.exists_e_cover_finite
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Continuous.scottUnion

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace

/-- **Scott 1976, Theorem 1.1 (The characterization theorem).**
`f` is continuous iff `e m ⊆ f(x)` just when some finite `e n ⊆ x` already
satisfies `e m ⊆ f(e n)`. -/
theorem theorem_1_1 (f : Pomega → Pomega) :
    IsScottContinuous f ↔
      ∀ x m, e m ⊆ f x ↔ ∃ n, e n ⊆ x ∧ e m ⊆ f (e n) := by
  constructor
  · intro hf x m
    constructor
    · intro hmf
      have : ∀ k ∈ e m, ∃ n, e n ⊆ x ∧ k ∈ f (e n) := by
        intro k hk
        have : k ∈ scottUnion f x := by
          rw [← hf.out x]; exact hmf hk
        exact mem_scottUnion.mp this
      obtain ⟨s, hs⟩ := (e_finite m).exists_finset_coe
      obtain ⟨N, hN, hsf⟩ := exists_e_cover_finite hf s (fun k hk =>
        this k (by
          have : k ∈ (s : Set ℕ) := hk
          rwa [hs] at this))
      exact ⟨N, hN, by simpa [hs] using hsf⟩
    · intro ⟨n, hn, hmf⟩
      exact subset_trans hmf (IsScottContinuous.monotone hf hn)
  · intro h
    constructor
    intro x
    ext k
    constructor
    · intro hk
      have : e (2 ^ k) ⊆ f x := by
        simp [e_pow2, Set.singleton_subset_iff, hk]
      obtain ⟨n, hn, hkn⟩ := (h x (2 ^ k)).mp this
      refine mem_scottUnion.mpr ⟨n, hn, ?_⟩
      simpa [e_pow2, Set.singleton_subset_iff] using hkn
    · intro hk
      obtain ⟨n, hn, hkn⟩ := mem_scottUnion.mp hk
      have : e (2 ^ k) ⊆ f (e n) := by
        simp [e_pow2, Set.singleton_subset_iff, hkn]
      have : e (2 ^ k) ⊆ f x := (h x (2 ^ k)).mpr ⟨n, hn, this⟩
      simpa [e_pow2, Set.singleton_subset_iff] using this

end DomainTheory.DataTypesAsLattices
```

chain_lemma.lean

```
module

public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Continuous.theorem_1_1
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Order
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.FixedPoint.iterateBot
public import DomainTheory.DataTypesAsLattices.FixedPoint.fix
public import DomainTheory.DataTypesAsLattices.FixedPoint.exists_index_of_finset

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, Appendix lemma.** Continuous maps preserve ascending
countable unions. -/
theorem chain_lemma {f : Pomega → Pomega} (hf : IsScottContinuous f)
    (xs : ℕ → Pomega) (hmono : ∀ n, xs n ⊆ xs (n + 1)) :
    f (⋃ n, xs n) = ⋃ n, f (xs n) := by
  apply subset_antisymm
  · intro k hk
    have hchar := (theorem_1_1 f).mp hf (⋃ n, xs n) (2 ^ k)
    have hfin : e (2 ^ k) ⊆ f (⋃ n, xs n) := by
      simp [e_pow2, Set.singleton_subset_iff, hk]
    obtain ⟨m, hm, hkm⟩ := hchar.mp hfin
    have hwit : ∀ i ∈ e m, ∃ n, i ∈ xs n := by
      intro i hi
      obtain ⟨n, hix⟩ := Set.mem_iUnion.mp (hm hi)
      exact ⟨n, hix⟩
    obtain ⟨s, hs⟩ := (e_finite m).exists_finset_coe
    obtain ⟨N, hN⟩ := exists_index_of_finset xs hmono s (fun i hi =>
      hwit i (by
        have : i ∈ (s : Set ℕ) := hi
        rwa [hs] at this))
    have hem : e m ⊆ xs N := by rwa [← hs]
    refine Set.mem_iUnion.mpr ⟨N, ?_⟩
    have : e (2 ^ k) ⊆ f (xs N) :=
      subset_trans hkm (IsScottContinuous.monotone hf hem)
    simpa [e_pow2, Set.singleton_subset_iff] using this
  · intro k hk
    obtain ⟨n, hkn⟩ := Set.mem_iUnion.mp hk
    exact IsScottContinuous.monotone hf (Set.subset_iUnion xs n) hkn

end DomainTheory.DataTypesAsLattices
```

lfpAbove.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
```

```
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Embedding.extend
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.iterateFrom
public import DomainTheory.DataTypesAsLattices.Retracts.Fixpoints
public import DomainTheory.DataTypesAsLattices.Retracts.sUnion_Fixpoints_le_image
public import DomainTheory.DataTypesAsLattices.FixedPoint.chain_lemma
public import DomainTheory.DataTypesAsLattices.FixedPoint.fix
public import DomainTheory.DataTypesAsLattices.Retracts.vals

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- Least upper bound of fixed points above `x`: `⋃_n f?(x)`. -/
def lfpAbove (f : Pomega → Pomega) (x : Pomega) : Pomega :=
  ⋃ n, iterateFrom f x n

namespace lfpAbove

theorem bot (f : Pomega → Pomega) : lfpAbove f botElem = fix f := by
  simp [lfpAbove, fix, iterateFrom.bot]

theorem ub {f : Pomega → Pomega} {A : Set Pomega} :
    ∀ x ∈ A, x ⊆ lfpAbove f (⋃₀ A) := by
  intro x hx k hk
  exact Set.mem_iUnion.mpr ⟨0, Set.subset_sUnion_of_mem hx hk⟩

theorem fixed {f : Pomega → Pomega} (hf : IsScottContinuous f)
    {x : Pomega} (hx : x ⊆ f x) : f (lfpAbove f x) = lfpAbove f x := by
  have hchain := chain_lemma hf (iterateFrom f x) (iterateFrom.mono hf hx)
  unfold lfpAbove
  rw [hchain]
  ext k
  constructor
  · intro hk
    obtain ⟨n, hkn⟩ := Set.mem_iUnion.mp hk
    exact Set.mem_iUnion.mpr ⟨n + 1, hkn⟩
  · intro hk
    obtain ⟨n, hkn⟩ := Set.mem_iUnion.mp hk
    cases n with
    | zero => exact Set.mem_iUnion.mpr ⟨0, hx hkn⟩
    | succ n => exact Set.mem_iUnion.mpr ⟨n, hkn⟩

theorem le_of_ub {f : Pomega → Pomega} (hf : IsScottContinuous f)
    {A : Set Pomega} {z : Pomega} (hz : f z = z) (hub : ∀ x ∈ A, x ⊆ z) :
    lfpAbove f (⋃₀ A) ⊆ z := by
  have h0 : ⋃₀ A ⊆ z := by
    intro k hk
    obtain ⟨x, hxA, hkx⟩ := Set.mem_sUnion.mp hk
    exact hub x hxA hkx
  have hiter : ∀ n, iterateFrom f (⋃₀ A) n ⊆ z := by
    intro n
    induction n with
    | zero => simpa [iterateFrom] using h0
```

```
    | succ n ih => simpa [iterateFrom, hz] using IsScottContinuous.monotone hf ih
  intro k hk
  obtain ⟨n, hkn⟩ := Set.mem_iUnion.mp hk
  exact hiter n hkn

theorem mem {f : Pomega → Pomega} (hf : IsScottContinuous f)
    {A : Set Pomega} (hA : A ⊆ Fixpoints f) :
    lfpAbove f (⋃₀ A) ∈ Fixpoints f :=
  fixed hf (sUnion_Fixpoints_le_image hf hA)

end lfpAbove

end DomainTheory.DataTypesAsLattices
```

iterateBot_mono.lean

```
module

public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Order
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.FixedPoint.iterateBot
public import DomainTheory.DataTypesAsLattices.FixedPoint.fix

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem iterateBot_mono {f : Pomega → Pomega} (hf : IsScottContinuous f) :
    ∀ n, iterateBot f n ⊆ iterateBot f (n + 1)
  | 0 => by
    simp [iterateBot, botElem]
  | n + 1 =>
    IsScottContinuous.monotone hf (iterateBot_mono hf n)

end DomainTheory.DataTypesAsLattices
```

theorem_1_4.lean

```
module

public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Order
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.FixedPoint.iterateBot
public import DomainTheory.DataTypesAsLattices.FixedPoint.fix
public import DomainTheory.DataTypesAsLattices.FixedPoint.chain_lemma
public import DomainTheory.DataTypesAsLattices.FixedPoint.iterateBot_mono

@[expose] public section

namespace DomainTheory.DataTypesAsLattices
```

```
/-- **Scott 1976, Theorem 1.4 (The fixed-point theorem).** -/
theorem theorem_1_4 {f : Pomega → Pomega} (hf : IsScottContinuous f) :
    f (fix f) = fix f ∧ ∀ x, f x = x → fix f ⊆ x := by
  constructor
  · -- f(fix f) = ⋃ f(f?(∅)) = ⋃ f?^{+1}(∅) = fix f
    have hchain := chain_lemma hf (iterateBot f) (iterateBot_mono hf)
    unfold fix
    rw [hchain]
    ext k
    constructor
    · intro hk
      obtain ⟨n, hkn⟩ := Set.mem_iUnion.mp hk
      exact Set.mem_iUnion.mpr ⟨n + 1, hkn⟩
    · intro hk
      obtain ⟨n, hkn⟩ := Set.mem_iUnion.mp hk
      cases n with
      | zero => simp [iterateBot, botElem] at hkn
      | succ n => exact Set.mem_iUnion.mpr ⟨n, hkn⟩
  · intro x hx k hk
    obtain ⟨n, hkn⟩ := Set.mem_iUnion.mp hk
    have hsub : ∀ n, iterateBot f n ⊆ x := by
      intro n
      induction n with
      | zero => simp [iterateBot, botElem]
      | succ n ih =>
        simpa [iterateBot, hx] using IsScottContinuous.monotone hf ih
    exact hsub n hkn

end DomainTheory.DataTypesAsLattices
```

funOf.lean

```
module
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Continuous.mem_scottUnion
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import Mathlib.Topology.Bases

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, §1, Definition / Table 2.**
`fun(u)(x) = {m | ∃ e n ⊆ x. (n, m) ∈ u}`. -/
def funOf (u : Pomega) (x : Pomega) : Pomega :=
  {m | ∃ n, e n ⊆ x ∧ pair n m ∈ u}

namespace funOf

theorem monotone_left {u v : Pomega} (h : u ⊆ v) (x : Pomega) :
    funOf u x ⊆ funOf v x := by
  intro m hm
  obtain ⟨n, hn, hp⟩ := hm
```

```
  exact ⟨n, hn, h hp⟩

theorem monotone_right (u : Pomega) {x y : Pomega} (h : x ⊆ y) :
    funOf u x ⊆ funOf u y := by
  intro m hm
  obtain ⟨n, hn, hp⟩ := hm
  exact ⟨n, subset_trans hn h, hp⟩

theorem bot (x : Pomega) : funOf botElem x = botElem := by
  ext m
  simp [funOf, botElem]

theorem iUnion (xs : ℕ → Pomega) (x : Pomega) :
    funOf (⋃ n, xs n) x = ⋃ n, funOf (xs n) x := by
  ext m
  constructor
  · intro ⟨n, hn, hk⟩
    obtain ⟨k, hmem⟩ := Set.mem_iUnion.mp hk
    exact Set.mem_iUnion.mpr ⟨k, ⟨n, hn, hmem⟩⟩
  · intro hm
    obtain ⟨k, ⟨n, hn, hmem⟩⟩ := Set.mem_iUnion.mp hm
    exact ⟨n, hn, Set.mem_iUnion.mpr ⟨k, hmem⟩⟩

theorem isScottContinuous (u : Pomega) : IsScottContinuous (funOf u) := by
  constructor
  intro x
  ext m
  constructor
  · intro ⟨n, hn, hp⟩
    exact mem_scottUnion.mpr ⟨n, hn, ⟨n, subset_rfl, hp⟩⟩
  · intro hm
    obtain ⟨n, hn, ⟨k, hk, hp⟩⟩ := mem_scottUnion.mp hm
    exact ⟨k, subset_trans hk hn, hp⟩

theorem isScottContinuous_left (y : Pomega) :
    IsScottContinuous (fun u => funOf u y) := by
  constructor
  intro u
  ext m
  constructor
  · intro ⟨n, hn, hp⟩
    refine mem_scottUnion.mpr ⟨2 ^ pair n m, ?_, ⟨n, hn, ?_⟩⟩
    · simp [e_pow2, Set.singleton_subset_iff, hp]
    · simp [e_pow2]
  · intro hm
    obtain ⟨k, hk, n, hn, hp⟩ := mem_scottUnion.mp hm
    exact ⟨n, hn, hk hp⟩

/-- Appendix calculation: `d(d) = ⋃ { e_l(e_l) | e_l ⊆ d }`. -/
theorem diag_iUnion (d : Pomega) :
    funOf d d = ⋃ l, {m | e l ⊆ d ∧ m ∈ funOf (e l) (e l)} := by
  have h := theorem_1_3.diag (x := d)
    (f := fun u y => funOf u y)
    (fun y => isScottContinuous_left y)
    (fun u => isScottContinuous u)
  exact h
```

```
end funOf

end DomainTheory.DataTypesAsLattices
```

nestApplyFin.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Graph.graph

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- Iterated application of a nested-graph representative. -/
def nestApplyFin (u : Pomega) : ∀ {n : ℕ}, (Fin n → Pomega) → Pomega
  | 0, _ => u
  | _n + 1, xs => nestApplyFin (funOf u (xs 0)) (Fin.tail xs)

end DomainTheory.DataTypesAsLattices
```

interp.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.lam
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.Term
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.envSet
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.succSet
public import DomainTheory.DataTypesAsLattices.Lambda.predSet
public import DomainTheory.DataTypesAsLattices.Enumeration.Avoids
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Continuous.mem_scottUnion
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Continuous.exists_e_cover_finite
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Continuous.scottUnion
public import DomainTheory.DataTypesAsLattices.Lambda.id_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.const_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.predSet_more
public import DomainTheory.DataTypesAsLattices.Lambda.succSet_more
```

```
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, Table 2.** Semantics of a LAMBDA term. -/
def interp : Term → (ℕ → Pomega) → Pomega
  | .var i, ? => ? i
  | .zero, _ => ofNat 0
  | .succ t, ? => succSet (interp t ?)
  | .pred t, ? => predSet (interp t ?)
  | .cond z x y, ? => condSet (interp z ?) (interp x ?) (interp y ?)
  | .app u x, ? => funOf (interp u ?) (interp x ?)
  | .lam i body, ? => graph (fun v => interp body (envSet ? i v))

namespace interp

/-- Appendix lemma for Theorem 2.1: abstraction preserves continuity in the other
    variables. -/
theorem var_isScottContinuous (t : Term) (? : ℕ → Pomega) (i : ℕ) :
    IsScottContinuous (fun v => interp t (envSet ? i v)) := by
  induction t generalizing ? i with
  | var j =>
    constructor
    intro x
    by_cases hj : j = i
    · subst hj
      simpa [interp, envSet, Function.update] using id_isScottContinuous.out x
    · simpa [interp, envSet, Function.update, hj] using (const_isScottContinuous (? j))
    .out x
  | zero =>
    constructor
    intro x
    simpa [interp] using (const_isScottContinuous (ofNat 0)).out x
  | succ t ih =>
    exact theorem_1_3 succSet.isScottContinuous (ih ? i)
  | pred t ih =>
    exact theorem_1_3 predSet.isScottContinuous (ih ? i)
  | cond z x y iz ix iy =>
    constructor
    intro w
    ext n
    simp only [interp]
    constructor
    · intro hn
      rcases hn with ⟨hnx, h0⟩ | ⟨hny, t, ht⟩
      · have : n ∈ scottUnion (fun v => interp x (envSet ? i v)) w := by
          rw [← (ix ? i).out w]; exact hnx
        obtain ⟨m, hm, hnm⟩ := mem_scottUnion.mp this
        have : 0 ∈ scottUnion (fun v => interp z (envSet ? i v)) w := by
          rw [← (iz ? i).out w]; exact h0
        obtain ⟨m', hm', h0m⟩ := mem_scottUnion.mp this
        refine mem_scottUnion.mpr ⟨m ||| m', e_or_of_subset hm hm', Or.inl ⟨?_, ?_⟩⟩
        · exact IsScottContinuous.monotone (ix ? i)
            (by rw [e_or]; exact Set.subset_union_left) hnm
        · exact IsScottContinuous.monotone (iz ? i)
```

```
          (by rw [e_or]; exact Set.subset_union_right) h0m
    · have : n ∈ scottUnion (fun v => interp y (envSet ? i v)) w := by
        rw [← (iy ? i).out w]; exact hny
      obtain ⟨m, hm, hnm⟩ := mem_scottUnion.mp this
      have : t + 1 ∈ scottUnion (fun v => interp z (envSet ? i v)) w := by
        rw [← (iz ? i).out w]; exact ht
      obtain ⟨m', hm', htm⟩ := mem_scottUnion.mp this
      refine mem_scottUnion.mpr ⟨m ||| m', e_or_of_subset hm hm', Or.inr ⟨?_, t, ?_⟩⟩
      · exact IsScottContinuous.monotone (iy ? i)
          (by rw [e_or]; exact Set.subset_union_left) hnm
      · exact IsScottContinuous.monotone (iz ? i)
          (by rw [e_or]; exact Set.subset_union_right) htm
  · intro hn
    obtain ⟨m, hm, hnm⟩ := mem_scottUnion.mp hn
    rcases hnm with ⟨hnx, h0⟩ | ⟨hny, t, ht⟩
    · exact Or.inl ⟨IsScottContinuous.monotone (ix ? i) hm hnx,
        IsScottContinuous.monotone (iz ? i) hm h0⟩
    · exact Or.inr ⟨IsScottContinuous.monotone (iy ? i) hm hny, t,
        IsScottContinuous.monotone (iz ? i) hm ht⟩
| app u x iu ix =>
  constructor
  intro w
  ext n
  simp only [interp]
  constructor
  · intro ⟨p, hp, hpair⟩
    have : pair p n ∈ scottUnion (fun v => interp u (envSet ? i v)) w := by
      rw [← (iu ? i).out w]; exact hpair
    obtain ⟨m, hm, hpm⟩ := mem_scottUnion.mp this
    obtain ⟨s, hs⟩ := (e_finite p).exists_finset_coe
    have hall : ∀ t ∈ s, ∃ q, e q ⊆ w ∧ t ∈ interp x (envSet ? i (e q)) := by
      intro t ht
      have ht' : t ∈ e p := by
        have : t ∈ (s : Set ℕ) := ht
        rwa [hs] at this
      have : t ∈ interp x (envSet ? i w) := hp ht'
      have : t ∈ scottUnion (fun v => interp x (envSet ? i v)) w := by
        rwa [← (ix ? i).out w]
      exact mem_scottUnion.mp this
    obtain ⟨N, hN, hsN⟩ := exists_e_cover_finite (ix ? i) s hall
    refine mem_scottUnion.mpr ⟨m ||| N, e_or_of_subset hm hN, p, ?_, ?_⟩
    · have : e p ⊆ interp x (envSet ? i (e (m ||| N))) := by
        rw [← hs]
        intro t ht
        exact IsScottContinuous.monotone (ix ? i)
          (by rw [e_or]; exact Set.subset_union_right) (hsN ht)
      exact this
    · exact IsScottContinuous.monotone (iu ? i)
        (by rw [e_or]; exact Set.subset_union_left) hpm
  · intro hn
    obtain ⟨m, hm, p, hp, hpair⟩ := mem_scottUnion.mp hn
    exact ⟨p, subset_trans hp (IsScottContinuous.monotone (ix ? i) hm),
      IsScottContinuous.monotone (iu ? i) hm hpair⟩
| lam j body ih =>
  by_cases hj : j = i
  · subst hj
```

```
      constructor
      intro w
      simpa [interp, envSet, Function.update] using
        (const_isScottContinuous (graph (fun v => interp body (envSet ? j v)))).out w
    · have hmap : IsScottContinuous (fun w =>
          graph (fun v => interp body (envSet (envSet ? i w) j v))) := by
        refine graph_const_isScottContinuous
          (fun w v => interp body (envSet (envSet ? i w) j v)) ?_
        intro v
        have : (fun w => interp body (envSet (envSet ? i w) j v)) =
            (fun w => interp body (envSet (envSet ? j v) i w)) := by
          funext w; rw [envSet.comm hj]
        rw [this]
        exact ih (envSet ? j v) i
      constructor
      intro w
      simpa [interp] using hmap.out w

end interp

end DomainTheory.DataTypesAsLattices
```

Fofg.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def Fofg (g : Pomega → Pomega → Pomega) (f : Pomega) : Pomega :=
  graph (fun x => g x (funOf f x))

end DomainTheory.DataTypesAsLattices
```

Scomb.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def Scomb : Pomega :=
  graph (fun u => graph (fun v => graph (fun x => funOf (funOf u x) (funOf v x))))
```

```
end DomainTheory.DataTypesAsLattices
```

IsCombinatory.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.zeroC
public import DomainTheory.DataTypesAsLattices.Lambda.sucC
public import DomainTheory.DataTypesAsLattices.Lambda.predC
public import DomainTheory.DataTypesAsLattices.Lambda.condC
public import DomainTheory.DataTypesAsLattices.Lambda.Kcomb
public import DomainTheory.DataTypesAsLattices.Lambda.Scomb

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- Combinatory closure of the six constants of Theorem 2.4. -/
inductive IsCombinatory : Pomega → Prop
  | zero : IsCombinatory zeroC
  | suc : IsCombinatory sucC
  | pred : IsCombinatory predC
  | cond : IsCombinatory condC
  | K : IsCombinatory Kcomb
  | S : IsCombinatory Scomb
  | app {u v} : IsCombinatory u → IsCombinatory v → IsCombinatory (funOf u v)

end DomainTheory.DataTypesAsLattices
```

dcondSet.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.Term
public import DomainTheory.DataTypesAsLattices.Lambda.IsCombinatory
public import DomainTheory.DataTypesAsLattices.Lambda.Comb
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (4.4).** Doubly strict conditional
`z ? x, y = z ? (z ? x, ⊤), (z ? ⊤, y)`. -/
def dcondSet (z x y : Pomega) : Pomega :=
  condSet z (condSet z x topElem) (condSet z topElem y)

end DomainTheory.DataTypesAsLattices
```

dcondSet_more.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.dcondSet
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.id_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.predSet
public import DomainTheory.DataTypesAsLattices.Lambda.predSet_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace dcondSet

theorem isScottContinuous_left (x y : Pomega) :
    IsScottContinuous (fun z => dcondSet z x y) :=
  theorem_1_3.nary
    (fun y z => condSet.isScottContinuous_left y z)
    (fun x z => condSet.isScottContinuous_mid x z)
    (fun x y => condSet.isScottContinuous_right x y)
    id_isScottContinuous
    (condSet.isScottContinuous_left x topElem)
    (condSet.isScottContinuous_left topElem y)

theorem bot (x y : Pomega) : dcondSet botElem x y = botElem := by
  ext n
  simp [dcondSet, condSet, botElem]

theorem cases (z : Pomega) :
    (0 ∈ z ∧ ¬ ∃ k, k + 1 ∈ z) ∨
      (0 ∈ z ∧ ∃ k, k + 1 ∈ z) ∨
        (0 ∈ z ∧ ∃ k, k + 1 ∈ z) ∨
          (0 ∈ z ∧ ¬ ∃ k, k + 1 ∈ z) := by
  by_cases h0 : 0 ∈ z
  · by_cases hp : ∃ k, k + 1 ∈ z
    · exact Or.inr (Or.inr (Or.inl ⟨h0, hp⟩))
    · exact Or.inl ⟨h0, hp⟩
  · by_cases hp : ∃ k, k + 1 ∈ z
    · exact Or.inr (Or.inl ⟨h0, hp⟩)
    · exact Or.inr (Or.inr (Or.inr ⟨h0, hp⟩))

theorem isScottContinuous_mid (z y : Pomega) :
    IsScottContinuous (fun x => dcondSet z x y) :=
  theorem_1_3 (f := fun w => condSet z w (condSet z topElem y))
    (g := fun x => condSet z x topElem)
    (condSet.isScottContinuous_mid z (condSet z topElem y))
    (condSet.isScottContinuous_mid z topElem)
```

```
theorem isScottContinuous_right (z x : Pomega) :
    IsScottContinuous (fun y => dcondSet z x y) :=
  theorem_1_3 (f := fun w => condSet z (condSet z x topElem) w)
    (g := fun y => condSet z topElem y)
    (condSet.isScottContinuous_right z (condSet z x topElem))
    (condSet.isScottContinuous_right z topElem)

theorem of_empty {z x y : Pomega}
    (h0 : 0 ∈ z) (hp : ¬ ∃ k, k + 1 ∈ z) :
    dcondSet z x y = botElem := by
  ext n
  constructor
  · intro hn
    rcases hn with ⟨_, hz⟩ | ⟨_, ht⟩
    · exact (h0 hz).elim
    · exact (hp ht).elim
  · intro hn
    simp [botElem] at hn

theorem of_mixed {z x y : Pomega}
    (h0 : 0 ∈ z) (hp : ∃ k, k + 1 ∈ z) :
    dcondSet z x y = topElem := by
  have hL : condSet z x topElem = topElem := by
    ext n
    constructor
    · intro _; exact Set.mem_univ n
    · intro _; exact Or.inr ⟨Set.mem_univ n, hp⟩
  have hR : condSet z topElem y = topElem := by
    ext n
    constructor
    · intro _; exact Set.mem_univ n
    · intro _; exact Or.inl ⟨Set.mem_univ n, h0⟩
  rw [dcondSet, hL, hR]
  ext n
  constructor
  · intro _; exact Set.mem_univ n
  · intro _; exact Or.inl ⟨Set.mem_univ n, h0⟩

theorem of_pos_not_zero {z x y : Pomega}
    (h0 : 0 ∈ z) (hp : ∃ k, k + 1 ∈ z) :
    dcondSet z x y = y := by
  have hL : condSet z x topElem = topElem := by
    ext n
    constructor
    · intro _; exact Set.mem_univ n
    · intro _; exact Or.inr ⟨Set.mem_univ n, hp⟩
  have hR : condSet z topElem y = y := by
    ext n
    constructor
    · intro hn
      rcases hn with ⟨_, hz⟩ | ⟨hy, _⟩
      · exact (h0 hz).elim
      · exact hy
    · intro hy
      exact Or.inr ⟨hy, hp⟩
  rw [dcondSet, hL, hR]
```

```
  ext n
  constructor
  · intro hn
    rcases hn with ⟨_, hz⟩ | ⟨hy, _⟩
    · exact (h0 hz).elim
    · exact hy
  · intro hy
    exact Or.inr ⟨hy, hp⟩

theorem of_zero_not_pos {z x y : Pomega}
    (h0 : 0 ∈ z) (hp : ¬ ∃ k, k + 1 ∈ z) :
    dcondSet z x y = x := by
  have hL : condSet z x topElem = x := by
    ext n
    constructor
    · intro hn
      rcases hn with ⟨hx, _⟩ | ⟨_, ht⟩
      · exact hx
      · exact (hp ht).elim
    · intro hx
      exact Or.inl ⟨hx, h0⟩
  have hR : condSet z topElem y = topElem := by
    ext n
    constructor
    · intro _; exact Set.mem_univ n
    · intro _; exact Or.inl ⟨Set.mem_univ n, h0⟩
  rw [dcondSet, hL, hR]
  ext n
  constructor
  · intro hn
    rcases hn with ⟨hx, _⟩ | ⟨_, ht⟩
    · exact hx
    · exact (hp ht).elim
  · intro hx
    exact Or.inl ⟨hx, h0⟩

theorem ofNat_one (x y : Pomega) : dcondSet (ofNat 1) x y = y := by
  ext n
  simp [dcondSet, condSet, ofNat, topElem]

theorem ofNat_succ (n : ℕ) (x y : Pomega) :
    dcondSet (ofNat (n + 1)) x y = y :=
  dcondSet.of_pos_not_zero (by simp [ofNat]) ⟨n, by simp [ofNat]⟩

theorem ofNat_zero (x y : Pomega) : dcondSet (ofNat 0) x y = x := by
  ext n
  simp [dcondSet, condSet, ofNat, topElem]

theorem top (x y : Pomega) : dcondSet topElem x y = topElem := by
  ext n
  simp [dcondSet, condSet, topElem]

end dcondSet

end DomainTheory.DataTypesAsLattices
```

ofComb.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.Comb
public import DomainTheory.DataTypesAsLattices.Lambda.zeroC
public import DomainTheory.DataTypesAsLattices.Lambda.sucC
public import DomainTheory.DataTypesAsLattices.Lambda.Kcomb
public import DomainTheory.DataTypesAsLattices.Lambda.predC
public import DomainTheory.DataTypesAsLattices.Lambda.condC
public import DomainTheory.DataTypesAsLattices.Lambda.Scomb
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.IsCombinatory
public import DomainTheory.DataTypesAsLattices.Lambda.envSet
public import DomainTheory.DataTypesAsLattices.Lambda.id_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.const_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices
def ofComb : Comb → (ℕ → Pomega) → Pomega
  | .var i, ? => ? i
  | .zero, _ => zeroC
  | .suc, _ => sucC
  | .pred, _ => predC
  | .cond, _ => condC
  | .K, _ => Kcomb
  | .S, _ => Scomb
  | .app u v, ? => funOf (ofComb u ?) (ofComb v ?)

namespace ofComb

theorem update_isScottContinuous (c : Comb) (? : ℕ → Pomega) (i : ℕ) :
    IsScottContinuous (fun v => ofComb c (envSet ? i v)) := by
  induction c generalizing ? i with
  | var j =>
    constructor
    intro x
    by_cases hj : j = i
    · subst hj; simpa [ofComb, envSet, Function.update] using id_isScottContinuous.out
    x
    · simpa [ofComb, envSet, Function.update, hj] using (const_isScottContinuous (? j))
    .out x
  | zero => constructor; intro x; simpa [ofComb] using (const_isScottContinuous zeroC)
    .out x
  | suc => constructor; intro x; simpa [ofComb] using (const_isScottContinuous sucC).
    out x
  | pred => constructor; intro x; simpa [ofComb] using (const_isScottContinuous predC)
```

```
    .out x
  | cond => constructor; intro x; simpa [ofComb] using (const_isScottContinuous condC)
    .out x
  | K => constructor; intro x; simpa [ofComb] using (const_isScottContinuous Kcomb).
    out x
  | S => constructor; intro x; simpa [ofComb] using (const_isScottContinuous Scomb).
    out x
  | app u v iu iv =>
    exact theorem_1_3.tuple
      (fun y => funOf.isScottContinuous_left y)
      (fun t => funOf.isScottContinuous t)
      (iu ? i) (iv ? i)

theorem combinatory (c : Comb) (? : ℕ → Pomega)
    (h? : ∀ i, IsCombinatory (? i)) :
    IsCombinatory (ofComb c ?) := by
  induction c with
  | var i => exact h? i
  | zero => exact .zero
  | suc => exact .suc
  | pred => exact .pred
  | cond => exact .cond
  | K => exact .K
  | S => exact .S
  | app _ _ iu iv => exact .app iu iv

end ofComb

end DomainTheory.DataTypesAsLattices
```

eq_2_9.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.9).** Application distributes over unions. -/
theorem eq_2_9 (f g x : Pomega) :
    funOf (f ∪ g) x = funOf f x ∪ funOf g x := by
  ext m
  constructor
  · intro ⟨n, hn, hp⟩
    rcases hp with hp | hp
    · exact Or.inl ⟨n, hn, hp⟩
    · exact Or.inr ⟨n, hn, hp⟩
  · intro h
    rcases h with ⟨n, hn, hp⟩ | ⟨n, hn, hp⟩
    · exact ⟨n, hn, Or.inl hp⟩
    · exact ⟨n, hn, Or.inr hp⟩
```

```
end DomainTheory.DataTypesAsLattices
```

FUN.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, §2.** The subspace `FUN` of graphs. -/
def FUN : Set Pomega := {u | u = graph (funOf u)}

end DomainTheory.DataTypesAsLattices
```

eq_2_13.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import Mathlib.Data.Set.Lattice.Indexed

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.13).** Application distributes over arbitrary unions. -/
theorem eq_2_13 (F : Set Pomega) (x : Pomega) :
    funOf (⋃₀ F) x = ⋃ u ∈ F, funOf u x := by
  ext m
  constructor
  · intro ⟨n, hn, hp⟩
    obtain ⟨u, hu, hpu⟩ := Set.mem_sUnion.mp hp
    exact Set.mem_iUnion₂.mpr ⟨u, hu, n, hn, hpu⟩
  · intro hm
    obtain ⟨u, hu, n, hn, hp⟩ := Set.mem_iUnion₂.mp hm
    exact ⟨n, hn, ⟨u, hu, hp⟩⟩

end DomainTheory.DataTypesAsLattices
```

eq_2_11.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.IsGraph
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Pomega.pair
```

```
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.11).** One inclusion; equality needs both arguments to be graphs.
    -/
theorem eq_2_11 (f g x : Pomega) :
    funOf (f ∩ g) x ⊆ funOf f x ∩ funOf g x := by
  intro m ⟨n, hn, hp, hq⟩
  exact ⟨⟨n, hn, hp⟩, ⟨n, hn, hq⟩⟩

namespace eq_2_11

/-- **Scott 1976, (2.11).** Equality when both arguments are graphs. -/
theorem graphs {f g : Pomega} (hf : IsGraph f) (hg : IsGraph g)
    (x : Pomega) : funOf (f ∩ g) x = funOf f x ∩ funOf g x := by
  apply subset_antisymm (_root_.DomainTheory.DataTypesAsLattices.eq_2_11 f g x)
  intro m ⟨hf', hg'⟩
  obtain ⟨k, hk, hfk⟩ := hf'
  obtain ⟨l, hl, hgl⟩ := hg'
  refine ⟨k ||| l, e_or_of_subset hk hl, ⟨?_, ?_⟩⟩
  · exact hf.out hfk (by rw [e_or]; exact Set.subset_union_left)
  · exact hg.out hgl (by rw [e_or]; exact Set.subset_union_right)

end eq_2_11

end DomainTheory.DataTypesAsLattices
```

topRecTerm.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Lambda.Term
public import DomainTheory.DataTypesAsLattices.Graph.lam
public import DomainTheory.DataTypesAsLattices.Computability.pack4
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Lambda.envSet
public import DomainTheory.DataTypesAsLattices.Lambda.topRec
public import DomainTheory.DataTypesAsLattices.Lambda.interp
public import DomainTheory.DataTypesAsLattices.Enumeration.Avoids
public import DomainTheory.DataTypesAsLattices.Lambda.eq_2_15

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable

open Nat.Partrec (Code)
```

```
open Nat.Partrec.Code

/-- **Scott 1976, (2.16).** Combinator for `λx. 0 ∪ (x+1)`. -/
def topRecTerm : Term :=
  .lam 0 (.cond (.lam 1 .zero) .zero (.succ (.var 0)))

namespace topRecTerm

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem interp :
    interp topRecTerm (fun _ => botElem) = graph topRec := by
  apply congrArg graph
  funext v
  simp[topRec, _root_.DomainTheory.DataTypesAsLattices.interp, envSet, Function.update
    , _root_.DomainTheory.DataTypesAsLattices.eq_2_15]

end topRecTerm

end DomainTheory.DataTypesAsLattices
```

interStep.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.succSet
public import DomainTheory.DataTypesAsLattices.Lambda.predSet
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
public import DomainTheory.DataTypesAsLattices.Lambda.succSet_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices
/-- Least operator solving the (2.17) recursion. -/
def interStep (f : Pomega) : Pomega :=
  graph (fun x => graph (fun y =>
    condSet x (condSet y (ofNat 0) botElem)
      (succSet (funOf (funOf f (predSet x)) (predSet y)))))

namespace interStep
```

```
theorem isScottContinuous :
    IsScottContinuous interStep := by
  refine graph_const_isScottContinuous
    (fun f x => graph (fun y =>
      condSet x (condSet y (ofNat 0) botElem)
        (succSet (funOf (funOf f (predSet x)) (predSet y))))) ?_
  intro x
  refine graph_const_isScottContinuous
    (fun f y =>
      condSet x (condSet y (ofNat 0) botElem)
        (succSet (funOf (funOf f (predSet x)) (predSet y)))) ?_
  intro y
  refine theorem_1_3
    (condSet.isScottContinuous_right x (condSet y (ofNat 0) botElem)) ?_
  refine theorem_1_3 succSet.isScottContinuous ?_
  exact theorem_1_3
    (f := fun t => funOf t (predSet y))
    (g := fun f => funOf f (predSet x))
    (funOf.isScottContinuous_left (predSet y))
    (funOf.isScottContinuous_left (predSet x))

end interStep

end DomainTheory.DataTypesAsLattices
```

combinatory_isLambdaDefinable.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Lambda.IsCombinatory
public import DomainTheory.DataTypesAsLattices.Lambda.envSet
public import DomainTheory.DataTypesAsLattices.Lambda.interp
public import DomainTheory.DataTypesAsLattices.Lambda.sucC
public import DomainTheory.DataTypesAsLattices.Lambda.predC
public import DomainTheory.DataTypesAsLattices.Lambda.condC
public import DomainTheory.DataTypesAsLattices.Lambda.Kcomb
public import DomainTheory.DataTypesAsLattices.Lambda.Scomb
public import DomainTheory.DataTypesAsLattices.Graph.lam
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Computability.LambdaDefinable

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

/-- Combinatory elements are LAMBDA-definable (converse of Theorem 2.4). -/
theorem combinatory_isLambdaDefinable {u : Pomega} (hu : IsCombinatory u) :
    LambdaDefinable u := by
  induction hu with
  | zero => exact ⟨⟨.zero, rfl⟩⟩
  | suc =>
```

```
    refine ⟨⟨.lam 0 (.succ (.var 0)), ?_⟩⟩
    simp [interp, sucC, envSet, Function.update]
  | pred =>
    refine ⟨⟨.lam 0 (.pred (.var 0)), ?_⟩⟩
    simp [interp, predC, envSet, Function.update]
  | cond =>
    refine ⟨⟨.lam 0 (.lam 1 (.lam 2
        (.cond (.var 2) (.var 0) (.var 1)))), ?_⟩⟩
    simp [interp, condC, envSet, Function.update]
  | K =>
    refine ⟨⟨.lam 0 (.lam 1 (.var 0)), ?_⟩⟩
    simp [interp, Kcomb, envSet, Function.update]
  | S =>
    refine ⟨⟨.lam 0 (.lam 1 (.lam 2
        (.app (.app (.var 0) (.var 2)) (.app (.var 1) (.var 2))))), ?_⟩⟩
    simp [interp, Scomb, envSet, Function.update]
  | app _ _ iu iv =>
    obtain ⟨t, ht⟩ := iu.out
    obtain ⟨s, hs⟩ := iv.out
    exact ⟨⟨.app t s, by simp [interp, ht, hs]⟩⟩

end DomainTheory.DataTypesAsLattices
```

Realizes.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable

open Nat.Partrec (Code)

open Nat.Partrec.Code

/-- A combinator realizes a number-theoretic function on numerals. -/
def Realizes (u : Pomega) (p : ℕ → ℕ) : Prop :=
  ∀ n, funOf u (ofNat n) = ofNat (p n)

end DomainTheory.DataTypesAsLattices
```

typed.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
```

```
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, §4, Definition.** `u : a` means `u = a(u)`. -/
def typed (u a : Pomega) : Prop :=
  u = funOf a u

end DomainTheory.DataTypesAsLattices
```

dcondSet_bool_idem.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.dcondSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.dcondSet_more
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem dcondSet_bool_idem (u : Pomega) :
    dcondSet (dcondSet u (ofNat 0) (ofNat 1)) (ofNat 0) (ofNat 1) =
      dcondSet u (ofNat 0) (ofNat 1) := by
  rcases dcondSet.cases u with h | h | h | h
  · rw [dcondSet.of_zero_not_pos h.1 h.2, dcondSet.ofNat_zero]
  · rw [dcondSet.of_pos_not_zero h.1 h.2, dcondSet.ofNat_one]
  · rw [dcondSet.of_mixed h.1 h.2, dcondSet.top]
  · rw [dcondSet.of_empty h.1 h.2, dcondSet.bot]

end DomainTheory.DataTypesAsLattices
```

addStepC.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Lambda.sucC
public import DomainTheory.DataTypesAsLattices.Lambda.Kcomb
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Computability.pack4
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.IsCombinatory
@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable

open Nat.Partrec (Code)
```

```
open Nat.Partrec.Code

def addStepC : Pomega := funOf Kcomb sucC

namespace addStepC

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem combinatory : IsCombinatory addStepC :=
  .app .K .suc

end addStepC

end DomainTheory.DataTypesAsLattices
```

notation3.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.funOf

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

infixl:70 " ? " => funOf

end DomainTheory.DataTypesAsLattices
```

funOf_more.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.funOf

@[expose] public section
```

IsStrict.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- Strictness: `a(⊥) = ⊥`, equivalently `⊥ ⊑ a` for retracts. -/
def IsStrict (a : Pomega) : Prop :=
  funOf a botElem = botElem
```

```
end DomainTheory.DataTypesAsLattices
```

dcondSet_same_ofNat.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.dcondSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.predSet
public import DomainTheory.DataTypesAsLattices.Lambda.dcondSet_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem dcondSet_same_ofNat (n : ℕ) (x : Pomega) :
    dcondSet (ofNat n) x x = x :=
  match n with
  | 0 => dcondSet.ofNat_zero x x
  | n + 1 => dcondSet.ofNat_succ n x x

end DomainTheory.DataTypesAsLattices
```

funOf_botElem.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem funOf_botElem (x : Pomega) : funOf botElem x = botElem := by
  ext m
  simp [funOf, botElem]

end DomainTheory.DataTypesAsLattices
```

theorem_4_1.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.Fixpoints
public import DomainTheory.DataTypesAsLattices.Retracts.gfpBelow

@[expose] public section
```

```
namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, Theorem 4.1 (The lattice theorem), complete-lattice half.**
The inf `gfpBelow f (⋂ A)` is a fixed point of `f` itself. -/
theorem theorem_4_1.complete (f : Pomega → Pomega) (hf : IsScottContinuous f)
    (A : Set Pomega) (hA : A ⊆ Fixpoints f) :
    gfpBelow f (⋂₀ A) ∈ Fixpoints f := by
  let z := gfpBelow f (⋂₀ A)
  have hinter : f z ∩ ⋂₀ A = z := gfpBelow.fixed_inter hf (⋂₀ A)
  have hzle : z ⊆ ⋂₀ A := gfpBelow.subset f (⋂₀ A)
  have hfz : f z ⊆ ⋂₀ A := by
    intro k hk y hy
    have hzy : z ⊆ y := subset_trans hzle (Set.sInter_subset_of_mem hy)
    have : k ∈ f y := IsScottContinuous.monotone hf hzy hk
    have hyf : f y = y := hA hy
    rwa [hyf] at this
  have : f z ∩ ⋂₀ A = f z := Set.inter_eq_left.mpr hfz
  change f z = z
  rwa [this] at hinter

end DomainTheory.DataTypesAsLattices
```

fixpointsCompleteLattice.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Embedding.extend
public import DomainTheory.DataTypesAsLattices.FixedPoint.fix
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_4
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.Fixpoints
public import DomainTheory.DataTypesAsLattices.Retracts.lfpAbove
public import DomainTheory.DataTypesAsLattices.Retracts.gfpBelow
public import DomainTheory.DataTypesAsLattices.Retracts.theorem_4_1
public import DomainTheory.DataTypesAsLattices.Retracts.vals
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

@[instance_reducible]
def fixpointsCompleteLattice {f : Pomega → Pomega} (hf : IsScottContinuous f) :
    CompleteLattice (Fixpoints f) :=
  letI : SupSet (Fixpoints f) :=
    ⟨fun A => ⟨lfpAbove f (⋃₀ (Fixpoints.vals A)),
      lfpAbove.mem hf (Fixpoints.vals.subset A)⟩⟩
  { completeLatticeOfSup (Fixpoints f) (fun A => by
      refine ⟨?ub, ?least⟩
      · intro x hx
```

```
        change (x : Pomega) ⊆ lfpAbove f (⋃₀ (Fixpoints.vals A))
        exact lfpAbove.ub x.val ⟨x, hx, rfl⟩
      · intro z hz
        change lfpAbove f (⋃₀ (Fixpoints.vals A)) ⊆ z
        exact lfpAbove.le_of_ub hf z.property (fun y hy => by
          obtain ⟨w, hwA, rfl⟩ := hy
          exact hz hwA)) with
    sInf := fun A => ⟨gfpBelow f (⋂₀ (Fixpoints.vals A)),
      theorem_4_1.complete f hf (Fixpoints.vals A) (Fixpoints.vals.subset A)⟩
    isGLB_sInf := fun A => by
      refine ⟨?lb, ?greatest⟩
      · intro x hx
        change gfpBelow f (⋂₀ (Fixpoints.vals A)) ⊆ x
        exact subset_trans (gfpBelow.subset f _)
          (Set.sInter_subset_of_mem ⟨x, hx, rfl⟩)
      · intro z hz
        change (z : Pomega) ⊆ gfpBelow f (⋂₀ (Fixpoints.vals A))
        exact gfpBelow.isGLB hf (Fixpoints.vals.subset A) z.property (fun y hy => by
          obtain ⟨w, hwA, rfl⟩ := hy
          exact hz hwA)
    bot := ⟨fix f, (theorem_1_4 hf).1⟩
    bot_le := fun x => (theorem_1_4 hf).2 x.val x.property }

end DomainTheory.DataTypesAsLattices
```

BasicInst36.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.Fixpoints
public import DomainTheory.DataTypesAsLattices.Retracts.fixpointsCompleteLattice
public import DomainTheory.DataTypesAsLattices.Retracts.vals
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

noncomputable instance fixpointsFunOfCompleteLattice (a : Pomega) :
    CompleteLattice (Fixpoints (funOf a)) :=
  fixpointsCompleteLattice (funOf.isScottContinuous a)

end DomainTheory.DataTypesAsLattices
```

Realizes2.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
@[expose] public section
```

```
set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable

open Nat.Partrec (Code)

open Nat.Partrec.Code

def Realizes2 (u : Pomega) (p : ℕ → ℕ → ℕ) : Prop :=
  ∀ n m, funOf (funOf u (ofNat n)) (ofNat m) = ofNat (p n m)

end DomainTheory.DataTypesAsLattices
```

mem_Scomb_iff.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Lambda.Scomb
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Pomega.e

public import DomainTheory.DataTypesAsLattices.Graph.Basic.notation3
@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem mem_Scomb_iff (p : ℕ) :
    p ∈ Scomb ↔
      ∃ n0 n1 n2 m, p = pair n0 (pair n1 (pair n2 m)) ∧
        m ∈ (e n0 ? e n2) ? (e n1 ? e n2) := by
  constructor
  · intro ⟨n0, p1, hp, hp1⟩
    obtain ⟨n1, p2, hp1', hp2⟩ := hp1
    obtain ⟨n2, m, hp2', hm⟩ := hp2
    exact ⟨n0, n1, n2, m, by simp [hp, hp1', hp2'], hm⟩
  · intro ⟨n0, n1, n2, m, hp, hm⟩
    exact ⟨n0, pair n1 (pair n2 m), hp,
      ⟨n1, pair n2 m, rfl, ⟨n2, m, rfl, hm⟩⟩⟩

end DomainTheory.DataTypesAsLattices
```

mem_condC_iff.lean

```
module
```

```
public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Lambda.condC
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Lambda.condSet

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem mem_condC_iff (p : ℕ) :
    p ∈ condC ↔
      ∃ n0 n1 n2 m, p = pair n0 (pair n1 (pair n2 m)) ∧
        ((n0.testBit m = true ∧ n2.testBit 0 = true) ∨
          (n1.testBit m = true ∧ 2 ≤ n2)) := by
  constructor
  · intro ⟨n0, p1, hp, hp1⟩
    obtain ⟨n1, p2, hp1', hp2⟩ := hp1
    obtain ⟨n2, m, hp2', hm⟩ := hp2
    exact ⟨n0, n1, n2, m, by simp [hp, hp1', hp2'],
      (condSet.e_iff n0 n1 n2 m).mp hm⟩
  · intro ⟨n0, n1, n2, m, hp, hm⟩
    exact ⟨n0, pair n1 (pair n2 m), hp,
      ⟨n1, pair n2 m, rfl,
        ⟨n2, m, rfl, (condSet.e_iff n0 n1 n2 m).mpr hm⟩⟩⟩

end DomainTheory.DataTypesAsLattices
```

mem_graph.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.IsGraph
public import DomainTheory.DataTypesAsLattices.Graph.lam
public import DomainTheory.DataTypesAsLattices.Graph.Basic.notation3

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem mem_graph {f : Pomega → Pomega} {p : ℕ} :
    p ∈ graph f ↔ ∃ n m, p = pair n m ∧ m ∈ f (e n) := Iff.rfl

end DomainTheory.DataTypesAsLattices
```

theorem_1_2.lean

```
module

public import DomainTheory.DataTypesAsLattices.Continuous.mem_scottUnion
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Order
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Continuous.scottUnion
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.IsGraph
public import DomainTheory.DataTypesAsLattices.Graph.lam
public import DomainTheory.DataTypesAsLattices.Graph.Basic.notation3
public import DomainTheory.DataTypesAsLattices.Graph.mem_graph

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, Theorem 1.2 (i).** `fun(graph(f)) = f` when `f` is continuous. -/
theorem theorem_1_2.i {f : Pomega → Pomega} (hf : IsScottContinuous f) :
    funOf (graph f) = f := by
  funext x
  ext m
  constructor
  · intro ⟨n, hn, hp⟩
    obtain ⟨n', m', heq, hm'⟩ := hp
    obtain ⟨rfl, rfl⟩ := pair_inj heq
    exact IsScottContinuous.monotone hf hn hm'
  · intro hm
    have : m ∈ scottUnion f x := by
      rw [← hf.out x]; exact hm
    obtain ⟨n, hn, hmn⟩ := mem_scottUnion.mp this
    exact ⟨n, hn, ⟨n, m, rfl, hmn⟩⟩

/-- **Scott 1976, Theorem 1.2 (ii).** `u ⊆ graph(fun(u))`. -/
theorem theorem_1_2.ii (u : Pomega) : u ⊆ graph (funOf u) := by
  intro p hp
  obtain ⟨n, m, rfl⟩ := exists_pair p
  exact ⟨n, m, rfl, ⟨n, subset_rfl, hp⟩⟩

theorem theorem_1_2.iii (u : Pomega) :
    graph (funOf u) = u ↔ IsGraph u := by
  constructor
  · intro heqGraph
    constructor
    intro k m n hkm hkn
    have hmem : pair k m ∈ graph (funOf u) := heqGraph.symm ▶ hkm
    rcases mem_graph.mp hmem with ⟨k', m', hp, hmfun⟩
    have hk : k = k' := (pair_inj hp).1
    have hm : m = m' := (pair_inj hp).2
    have hmfun' : m ∈ funOf u (e k) := by
      rw [hk, hm]; exact hmfun
```

```
    rcases hmfun' with ⟨j, hj, hpj⟩
    have hgraph : pair n m ∈ graph (funOf u) :=
      mem_graph.mpr ⟨n, m, rfl, ⟨j, subset_trans hj hkn, hpj⟩⟩
    exact heqGraph ▸ hgraph
  · intro hu
    apply subset_antisymm
    · intro p hp
      rcases hp with ⟨n, m, rfl, ⟨k, hk, hkm⟩⟩
      exact hu.out hkm hk
    · exact theorem_1_2.ii u

/-- **Scott 1976, Theorem 1.2 (The graph theorem).** -/
theorem theorem_1_2 {f : Pomega → Pomega} (hf : IsScottContinuous f) (u : Pomega) :
    funOf (graph f) = f ∧ u ⊆ graph (funOf u) ∧
      (graph (funOf u) = u ↔ IsGraph u) :=
  ⟨theorem_1_2.i hf, theorem_1_2.ii u, theorem_1_2.iii u⟩

end DomainTheory.DataTypesAsLattices
```

beta.lean

```
module

public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Order
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.IsGraph
public import DomainTheory.DataTypesAsLattices.Graph.lam
public import DomainTheory.DataTypesAsLattices.Graph.Basic.notation3
public import DomainTheory.DataTypesAsLattices.Graph.theorem_1_2

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (β) for continuous `τ`.** `(λ x. τ)(y) = τ[y/x]`. -/
theorem beta {τ : Pomega → Pomega} (hτ : IsScottContinuous τ) (y : Pomega) :
    funOf (lam τ) y = τ y := by
  simp [lam, theorem_1_2.i hτ]

end DomainTheory.DataTypesAsLattices
```

curry2.lean

```
module

public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Graph.graph
```

```
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous

@[expose] public section

namespace DomainTheory.DataTypesAsLattices
def curry2 (f : Pomega → Pomega → Pomega)
    (_hfx : ∀ y, IsScottContinuous (fun x => f x y))
    (_hfy : ∀ x, IsScottContinuous (fun y => f x y)) : Pomega :=
  graph (fun x => graph (fun y => f x y))

namespace curry2

theorem app (f : Pomega → Pomega → Pomega)
    (hfx : ∀ y, IsScottContinuous (fun x => f x y))
    (hfy : ∀ x, IsScottContinuous (fun y => f x y)) (x y : Pomega) :
    funOf (funOf (curry2 f hfx hfy) x) y = f x y := by
  have h1 : funOf (curry2 f hfx hfy) x = graph (fun y => f x y) :=
    beta (graph_const_isScottContinuous f hfx) x
  rw [h1]
  exact beta (hfy x) y

end curry2

end DomainTheory.DataTypesAsLattices
```

Scomb_more.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.Scomb
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace Scomb

theorem inner_cont (u v : Pomega) :
    IsScottContinuous (fun x => funOf (funOf u x) (funOf v x)) :=
```

```
  theorem_1_3.tuple
    (fun y => funOf.isScottContinuous_left y)
    (fun t => funOf.isScottContinuous t)
    (funOf.isScottContinuous u)
    (funOf.isScottContinuous v)

theorem map_cont :
    IsScottContinuous (fun u => graph (fun v =>
      graph (fun x => funOf (funOf u x) (funOf v x)))) :=
  graph_const_isScottContinuous
    (fun u v => graph (fun x => funOf (funOf u x) (funOf v x)))
    (fun v => graph_const_isScottContinuous
      (fun u x => funOf (funOf u x) (funOf v x))
      (fun x => theorem_1_3
        (f := fun t => funOf t (funOf v x))
        (g := fun u => funOf u x)
        (funOf.isScottContinuous_left (funOf v x))
        (funOf.isScottContinuous_left x)))

theorem mid_cont (u : Pomega) :
    IsScottContinuous (fun v =>
      graph (fun x => funOf (funOf u x) (funOf v x))) :=
  graph_const_isScottContinuous
    (fun v x => funOf (funOf u x) (funOf v x))
    (fun x => theorem_1_3 (funOf.isScottContinuous (funOf u x))
      (funOf.isScottContinuous_left x))

theorem beta (f : Pomega) :
    funOf Scomb f =
      graph (fun v => graph (fun x => funOf (funOf f x) (funOf v x))) :=
  _root_.DomainTheory.DataTypesAsLattices.beta Scomb.map_cont f

theorem beta2 (f g : Pomega) :
    funOf (funOf Scomb f) g =
      graph (fun x => funOf (funOf f x) (funOf g x)) := by
  rw [Scomb.beta]
  exact _root_.DomainTheory.DataTypesAsLattices.beta (Scomb.mid_cont f) g

theorem beta3 (f g x : Pomega) :
    funOf (funOf (funOf Scomb f) g) x = funOf (funOf f x) (funOf g x) := by
  rw [Scomb.beta2]
  exact _root_.DomainTheory.DataTypesAsLattices.beta (Scomb.inner_cont f g) x

end Scomb

end DomainTheory.DataTypesAsLattices
```

curryFin.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
```

```
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.ScottContinuousFin
public import DomainTheory.DataTypesAsLattices.Lambda.nestApplyFin
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.beta

@[expose] public section

namespace DomainTheory.DataTypesAsLattices
/-- Nested-graph representative of an n-ary continuous map. -/
def curryFin : ∀ {n : ℕ}, ((Fin n → Pomega) → Pomega) → Pomega
  | 0, f => f Fin.elim0
  | _n + 1, f => graph (fun x => curryFin (fun xs => f (Fin.cons x xs)))

namespace curryFin

theorem isScottContinuous {n : ℕ} :
    ∀ {f : (Fin (n + 1) → Pomega) → Pomega},
      ScottContinuousFin f →
      IsScottContinuous (fun x => curryFin (fun xs : Fin n → Pomega => f (Fin.cons x
    xs))) := by
  induction n with
  | zero =>
    intro f hf
    exact hf.out.1 Fin.elim0
  | succ n ih =>
    intro f hf
    refine graph_const_isScottContinuous
      (fun x y => curryFin (fun zs : Fin n → Pomega =>
        f (Fin.cons x (Fin.cons y zs)))) (fun y =>
      ih (f := fun xs : Fin (n + 1) → Pomega =>
          f (Fin.cons (xs 0) (Fin.cons y (Fin.tail xs)))) ⟨⟨?_, ?_⟩⟩)
    · intro zs
      simpa [Fin.cons_zero, Fin.cons_succ] using hf.out.1 (Fin.cons y zs)
    · intro x
      simpa [Fin.cons_zero, Fin.cons_succ] using (hf.out.2 x).2 y

theorem app {n : ℕ} {f : (Fin n → Pomega) → Pomega}
    (hf : ScottContinuousFin f) (xs : Fin n → Pomega) :
    nestApplyFin (curryFin f) xs = f xs := by
  induction n with
  | zero =>
    change curryFin f = f xs
    exact congrArg f (funext fun i => nomatch i)
  | succ n ih =>
    have hβ : funOf (curryFin f) (xs 0) =
        curryFin (fun ys => f (Fin.cons (xs 0) ys)) :=
      beta (curryFin.isScottContinuous hf) (xs 0)
    change nestApplyFin (funOf (curryFin f) (xs 0)) (Fin.tail xs) = f xs
    rw [hβ, ih ⟨hf.out.2 (xs 0)⟩ (Fin.tail xs)]
    exact congrArg f (Fin.cons_self_tail xs)

end curryFin
```

```
end DomainTheory.DataTypesAsLattices
```

termShift.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.Term
public import DomainTheory.DataTypesAsLattices.Lambda.seqShift
public import DomainTheory.DataTypesAsLattices.Lambda.succSet
public import DomainTheory.DataTypesAsLattices.Enumeration.Avoids
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
public import DomainTheory.DataTypesAsLattices.Lambda.succSet_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- Term-level shift `λt. u(t+1)`, which agrees with `seqShift` on numerals. -/
def termShift (u : Pomega) : Pomega :=
  graph (fun t => funOf u (succSet t))

namespace termShift

theorem isScottContinuous (u : Pomega) :
    IsScottContinuous (fun t => funOf u (succSet t)) :=
  theorem_1_3 (funOf.isScottContinuous u) succSet.isScottContinuous

theorem app (u t : Pomega) :
    funOf (termShift u) t = funOf u (succSet t) :=
  _root_.DomainTheory.DataTypesAsLattices.beta (termShift.isScottContinuous u) t

theorem left_isScottContinuous : IsScottContinuous termShift :=
  graph_const_isScottContinuous (fun u t => funOf u (succSet t))
    (fun t => funOf.isScottContinuous_left (succSet t))

theorem ofNat (u : Pomega) (i : ℕ) :
    funOf (termShift u) (ofNat i) = funOf u (ofNat (i + 1)) := by
  rw [termShift.app, succSet.ofNat]

end termShift

end DomainTheory.DataTypesAsLattices
```

condC_more.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.condC
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace condC

theorem map_cont :
    IsScottContinuous (fun x => graph (fun y => graph (fun z => condSet z x y))) :=
  graph_const_isScottContinuous
    (fun x y => graph (fun z => condSet z x y))
    (fun y => graph_const_isScottContinuous (fun x z => condSet z x y)
      (fun z => condSet.isScottContinuous_mid z y))

theorem beta (x : Pomega) :
    funOf condC x = graph (fun y => graph (fun z => condSet z x y)) :=
  _root_.DomainTheory.DataTypesAsLattices.beta condC.map_cont x

theorem beta2 (x y : Pomega) :
    funOf (funOf condC x) y = graph (fun z => condSet z x y) := by
  rw [condC.beta]
  exact _root_.DomainTheory.DataTypesAsLattices.beta (graph_const_isScottContinuous (
    fun y z => condSet z x y)
    (fun z => condSet.isScottContinuous_right z x)) y

theorem beta3 (x y z : Pomega) :
    funOf (funOf (funOf condC x) y) z = condSet z x y := by
  rw [condC.beta2]
  exact _root_.DomainTheory.DataTypesAsLattices.beta (condSet.isScottContinuous_left x
    y) z

end condC

end DomainTheory.DataTypesAsLattices
```

curry3.lean

```
module
```

```
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous

@[expose] public section

namespace DomainTheory.DataTypesAsLattices
def curry3 (f : Pomega → Pomega → Pomega → Pomega)
    (_hf0 : ∀ y z, IsScottContinuous (fun x => f x y z))
    (_hf1 : ∀ x z, IsScottContinuous (fun y => f x y z))
    (_hf2 : ∀ x y, IsScottContinuous (fun z => f x y z)) : Pomega :=
  graph (fun x => graph (fun y => graph (fun z => f x y z)))

namespace curry3

theorem app (f : Pomega → Pomega → Pomega → Pomega)
    (hf0 : ∀ y z, IsScottContinuous (fun x => f x y z))
    (hf1 : ∀ x z, IsScottContinuous (fun y => f x y z))
    (hf2 : ∀ x y, IsScottContinuous (fun z => f x y z)) (x y z : Pomega) :
    funOf (funOf (funOf (curry3 f hf0 hf1 hf2) x) y) z = f x y z := by
  have h1 : funOf (curry3 f hf0 hf1 hf2) x =
      graph (fun y => graph (fun z => f x y z)) :=
    beta (graph_const_isScottContinuous
      (fun x y => graph (fun z => f x y z))
      (fun y => graph_const_isScottContinuous (fun x z => f x y z)
        (fun z => hf0 y z))) x
  have h2 : funOf (funOf (curry3 f hf0 hf1 hf2) x) y =
      graph (fun z => f x y z) := by
    rw [h1]
    exact beta (graph_const_isScottContinuous (fun y z => f x y z)
      (fun z => hf1 x z)) y
  rw [h2]
  exact beta (hf2 x y) z

end curry3

end DomainTheory.DataTypesAsLattices
```

sucC_more.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.sucC
public import DomainTheory.DataTypesAsLattices.Lambda.succSet_more
```

```
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace sucC

theorem app (x : Pomega) : funOf sucC x = succSet x :=
  beta succSet.isScottContinuous x

end sucC

end DomainTheory.DataTypesAsLattices
```

predC_more.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.predC
public import DomainTheory.DataTypesAsLattices.Lambda.predSet_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace predC

theorem app (x : Pomega) : funOf predC x = predSet x :=
  beta predSet.isScottContinuous x

end predC

end DomainTheory.DataTypesAsLattices
```

Kcomb_more.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.Kcomb
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.id_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.const_isScottContinuous

@[expose] public section

namespace DomainTheory.DataTypesAsLattices
```

```
namespace Kcomb

theorem beta (x : Pomega) :
    funOf Kcomb x = graph (fun _ => x) :=
  _root_.DomainTheory.DataTypesAsLattices.beta
    (graph_const_isScottContinuous (fun x _ => x)
      (fun _ => id_isScottContinuous)) x

theorem beta2 (x y : Pomega) : funOf (funOf Kcomb x) y = x := by
  rw [Kcomb.beta]
  exact _root_.DomainTheory.DataTypesAsLattices.beta (const_isScottContinuous x) y

end Kcomb

end DomainTheory.DataTypesAsLattices
```

SKK_eq_I.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Lambda.Kcomb
public import DomainTheory.DataTypesAsLattices.Lambda.Scomb
public import DomainTheory.DataTypesAsLattices.Lambda.Kcomb_more
public import DomainTheory.DataTypesAsLattices.Lambda.Scomb_more
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem SKK_eq_I : funOf (funOf Scomb Kcomb) Kcomb = graph (fun x => x) := by
  rw [Scomb.beta2]
  apply congrArg graph
  funext x
  simp [Kcomb.beta2]

end DomainTheory.DataTypesAsLattices
```

abs.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.Comb
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.theorem_1_2
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.envSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofComb
public import DomainTheory.DataTypesAsLattices.Lambda.Kcomb
public import DomainTheory.DataTypesAsLattices.Lambda.Scomb
public import DomainTheory.DataTypesAsLattices.Lambda.SKK_eq_I
public import DomainTheory.DataTypesAsLattices.Graph.beta
```

```
public import DomainTheory.DataTypesAsLattices.Lambda.Kcomb_more
public import DomainTheory.DataTypesAsLattices.Lambda.Scomb_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def abs (i : ℕ) : Comb → Comb
  | .var j => if j = i then .app (.app .S .K) .K else .app .K (.var j)
  | .zero => .app .K .zero
  | .suc => .app .K .suc
  | .pred => .app .K .pred
  | .cond => .app .K .cond
  | .K => .app .K .K
  | .S => .app .K .S
  | .app u v => .app (.app .S (abs i u)) (abs i v)

namespace abs

theorem correct (i : ℕ) (c : Comb) (? : ℕ → Pomega) :
    ofComb (abs i c) ? = graph (fun v => ofComb c (envSet ? i v)) := by
  induction c generalizing ? with
  | var j =>
    simp only [abs, ofComb]
    by_cases hj : j = i
    · rw [ite_eq_left hj]
      simp only [ofComb]
      convert SKK_eq_I
      funext v
      simp [envSet, Function.update, hj]
    · rw [ite_eq_right hj]
      simp [ofComb, Kcomb.beta, envSet, Function.update, hj]
  | zero => simp [abs, ofComb, Kcomb.beta]
  | suc => simp [abs, ofComb, Kcomb.beta]
  | pred => simp [abs, ofComb, Kcomb.beta]
  | cond => simp [abs, ofComb, Kcomb.beta]
  | K => simp [abs, ofComb, Kcomb.beta]
  | S => simp [abs, ofComb, Kcomb.beta]
  | app u v iu iv =>
    simp only [abs, ofComb]
    rw [Scomb.beta2, iu, iv]
    congr 1
    funext x
    rw [theorem_1_2.i (ofComb.update_isScottContinuous u ? i)]
    rw [theorem_1_2.i (ofComb.update_isScottContinuous v ? i)]

end abs

end DomainTheory.DataTypesAsLattices
```

erase.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.lam
```

```
public import DomainTheory.DataTypesAsLattices.Lambda.Term
public import DomainTheory.DataTypesAsLattices.Lambda.Comb
public import DomainTheory.DataTypesAsLattices.Lambda.abs
public import DomainTheory.DataTypesAsLattices.Enumeration.Avoids
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.envSet
public import DomainTheory.DataTypesAsLattices.Lambda.interp
public import DomainTheory.DataTypesAsLattices.Lambda.ofComb
public import DomainTheory.DataTypesAsLattices.Lambda.sucC
public import DomainTheory.DataTypesAsLattices.Lambda.predC
public import DomainTheory.DataTypesAsLattices.Lambda.condC
public import DomainTheory.DataTypesAsLattices.Lambda.zeroC
public import DomainTheory.DataTypesAsLattices.Lambda.condC_more
public import DomainTheory.DataTypesAsLattices.Lambda.predC_more
public import DomainTheory.DataTypesAsLattices.Lambda.sucC_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def erase : Term → Comb
  | .var i => .var i
  | .zero => .zero
  | .succ t => .app .suc (erase t)
  | .pred t => .app .pred (erase t)
  | .cond z x y => .app (.app (.app .cond (erase x)) (erase y)) (erase z)
  | .app u x => .app (erase u) (erase x)
  | .lam i body => abs i (erase body)

namespace erase

theorem correct (t : Term) (? : ℕ → Pomega) :
    ofComb (erase t) ? = interp t ? := by
  induction t generalizing ? with
  | var i => simp [erase, ofComb, interp]
  | zero => simp [erase, ofComb, interp, zeroC]
  | succ t ih => simp [erase, ofComb, interp, sucC.app, ih]
  | pred t ih => simp [erase, ofComb, interp, predC.app, ih]
  | cond z x y iz ix iy =>
    simp [erase, ofComb, interp, condC.beta3, iz, ix, iy]
  | app u x iu ix => simp [erase, ofComb, interp, iu, ix]
  | lam i body ih =>
    simp only [erase, interp]
    rw [abs.correct]
    congr 1
    funext v
    exact ih (envSet ? i v)

end erase

end DomainTheory.DataTypesAsLattices
```

ofNat_more.lean

```
module
```

```
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.IsCombinatory
public import DomainTheory.DataTypesAsLattices.Lambda.zeroC
public import DomainTheory.DataTypesAsLattices.Lambda.sucC_more
public import DomainTheory.DataTypesAsLattices.Lambda.succSet_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace ofNat

theorem combinatory : ∀ n, IsCombinatory (ofNat n)
  | 0 => by
    simpa [zeroC] using IsCombinatory.zero
  | n + 1 => by
    have h := IsCombinatory.app IsCombinatory.suc (combinatory n)
    simpa [sucC.app, succSet.ofNat] using h

end ofNat

end DomainTheory.DataTypesAsLattices
```

union_via_cond.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.eq_2_15
public import DomainTheory.DataTypesAsLattices.Lambda.Kcomb
public import DomainTheory.DataTypesAsLattices.Lambda.condC
public import DomainTheory.DataTypesAsLattices.Lambda.zeroC
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Lambda.Kcomb_more
public import DomainTheory.DataTypesAsLattices.Lambda.condC_more
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.15)** as a combinator: `cond(x)(y)(K(0)) = x ∪ y`. -/
theorem union_via_cond (x y : Pomega) :
    funOf (funOf (funOf condC x) y) (funOf Kcomb zeroC) = x ∪ y := by
  rw [condC.beta3, Kcomb.beta]
  exact eq_2_15 x y

end DomainTheory.DataTypesAsLattices
```

interC.lean

```
module

public import DomainTheory.DataTypesAsLattices.FixedPoint.fix
```

```
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.interStep
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_4
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.id_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.succSet
public import DomainTheory.DataTypesAsLattices.Lambda.predSet
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
public import DomainTheory.DataTypesAsLattices.Lambda.predSet_more
public import DomainTheory.DataTypesAsLattices.Lambda.succSet_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def interC : Pomega := fix interStep

namespace interC

theorem else_isScottContinuous (y : Pomega) :
    IsScottContinuous (fun x =>
      succSet (funOf (funOf interC (predSet x)) (predSet y))) := by
  refine theorem_1_3 (f := succSet) (g := fun x =>
      funOf (funOf interC (predSet x)) (predSet y))
    succSet.isScottContinuous ?_
  refine theorem_1_3
    (f := fun t => funOf t (predSet y))
    (g := fun x => funOf interC (predSet x))
    (funOf.isScottContinuous_left (predSet y)) ?_
  exact theorem_1_3 (funOf.isScottContinuous interC) predSet.isScottContinuous

theorem then_else_isScottContinuous (x : Pomega) :
    IsScottContinuous (fun y =>
      condSet x (condSet y (ofNat 0) botElem)
        (succSet (funOf (funOf interC (predSet x)) (predSet y)))) :=
  theorem_1_3.tuple
    (fun els => condSet.isScottContinuous_mid x els)
    (fun th => condSet.isScottContinuous_right x th)
    (condSet.isScottContinuous_left (ofNat 0) botElem)
    (theorem_1_3 succSet.isScottContinuous
      (theorem_1_3 (funOf.isScottContinuous (funOf interC (predSet x)))
        predSet.isScottContinuous))

theorem body_isScottContinuous :
    IsScottContinuous (fun x =>
```

```
      graph (fun y =>
        condSet x (condSet y (ofNat 0) botElem)
          (succSet (funOf (funOf interC (predSet x)) (predSet y))))) :=
  graph_const_isScottContinuous
    (fun x y =>
      condSet x (condSet y (ofNat 0) botElem)
        (succSet (funOf (funOf interC (predSet x)) (predSet y))))
    (fun y =>
      theorem_1_3.tuple
        (fun els => condSet.isScottContinuous_left
          (condSet y (ofNat 0) botElem) els)
        (fun tes => condSet.isScottContinuous_right tes
          (condSet y (ofNat 0) botElem))
        id_isScottContinuous
        (interC.else_isScottContinuous y))

theorem unfold : interC = interStep interC :=
  (theorem_1_4 interStep.isScottContinuous).1.symm

theorem app_graph (x : Pomega) :
    funOf interC x =
      graph (fun y =>
        condSet x (condSet y (ofNat 0) botElem)
          (succSet (funOf (funOf interC (predSet x)) (predSet y)))) := by
  nth_rw 1 [interC.unfold]
  exact _root_.DomainTheory.DataTypesAsLattices.beta interC.body_isScottContinuous x

theorem app (x y : Pomega) :
    funOf (funOf interC x) y =
      condSet x (condSet y (ofNat 0) botElem)
        (succSet (funOf (funOf interC (predSet x)) (predSet y))) := by
  rw [interC.app_graph]
  exact _root_.DomainTheory.DataTypesAsLattices.beta (interC.
    then_else_isScottContinuous x) y

end interC

end DomainTheory.DataTypesAsLattices
```

eq_2_17.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.interC
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.succSet
public import DomainTheory.DataTypesAsLattices.Lambda.predSet
@[expose] public section

namespace DomainTheory.DataTypesAsLattices
```

```
/-- **Scott 1976, (2.17).** `x ∩ y = Y(λf λx λy. x ? (y ? 0, ⊥), f(x?1)(y?1)+1)(x)(y)
    `. -/
theorem eq_2_17 (x y : Pomega) :
    funOf (funOf interC x) y = x ∩ y := by
  have hmem : ∀ n x y, n ∈ funOf (funOf interC x) y ↔ n ∈ x ∩ y := by
    intro n
    induction n using Nat.strong_induction_on with
    | h n ih =>
      intro x y
      rw [interC.app]
      constructor
      · intro hn
        rcases hn with ⟨hL, h0x⟩ | ⟨⟨k, hk, rfl⟩, _t, _ht⟩
        · rcases hL with ⟨hk0, h0y⟩ | ⟨hbot, _⟩
          · simp [ofNat] at hk0; subst hk0
            exact ⟨h0x, h0y⟩
          · simp [botElem] at hbot
        · have hk' : k ∈ predSet x ∩ predSet y :=
            (ih k (Nat.lt_succ_self k) (predSet x) (predSet y)).mp hk
          exact ⟨hk'.1, hk'.2⟩
      · intro ⟨hnx, hny⟩
        cases n with
        | zero => exact Or.inl ⟨Or.inl ⟨rfl, hny⟩, hnx⟩
        | succ k =>
          refine Or.inr ⟨⟨k, ?_, rfl⟩, k, hnx⟩
          exact (ih k (Nat.lt_succ_self k) (predSet x) (predSet y)).mpr ⟨hnx, hny⟩
  ext n
  exact hmem n x y

namespace eq_2_17

/-- **Scott 1976, (2.17), fundamental equation.** -/
theorem step (x y : Pomega) :
    x ∩ y =
      condSet x (condSet y (ofNat 0) botElem)
        (succSet (predSet x ∩ predSet y)) := by
  ext n
  constructor
  · intro ⟨hnx, hny⟩
    cases n with
    | zero =>
      exact Or.inl ⟨Or.inl ⟨rfl, hny⟩, hnx⟩
    | succ n =>
      refine Or.inr ⟨⟨n, ⟨hnx, hny⟩, rfl⟩, n, hnx⟩
  · intro hn
    rcases hn with ⟨hL, h0x⟩ | ⟨⟨k, hk, rfl⟩, _t, _ht⟩
    · rcases hL with ⟨hk0, h0y⟩ | ⟨hbot, _⟩
      · simp [ofNat] at hk0; subst hk0
        exact ⟨h0x, h0y⟩
      · simp [botElem] at hbot
    · exact ⟨hk.1, hk.2⟩

end eq_2_17

end DomainTheory.DataTypesAsLattices
```

eq_2_14.lean

```
module

public import DomainTheory.DataTypesAsLattices.FixedPoint.iterateBot
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.theorem_1_2
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.omegaComb
public import DomainTheory.DataTypesAsLattices.Lambda.id_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.omega_first_recursion

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.14).** `⊥ = (λx. x(x))(λx. x(x))`. -/
public theorem eq_2_14 :
    funOf (graph (fun x => funOf x x)) (graph (fun x => funOf x x)) = botElem := by
  have hω : omegaComb (graph fun x => x) = graph (fun x => funOf x x) := by
    ext p
    constructor
    · intro hp
      rcases hp with ⟨n, m, rfl, hm⟩
      rw [theorem_1_2.i id_isScottContinuous] at hm
      exact ⟨n, m, rfl, hm⟩
    · intro hp
      rcases hp with ⟨n, m, rfl, hm⟩
      refine ⟨n, m, rfl, ?_⟩
      rw [theorem_1_2.i id_isScottContinuous]
      exact hm
  rw [← hω, omega_first_recursion id_isScottContinuous]
  apply subset_antisymm
  · intro k hk
    obtain ⟨n, hkn⟩ := Set.mem_iUnion.mp hk
    have : ∀ n, iterateBot (fun x => x) n = botElem := by
      intro n
      induction n with
      | zero => rfl
      | succ n ih => simp [iterateBot, ih]
    simp [this n] at hkn
  · intro k hk
    exact hk.elim

end DomainTheory.DataTypesAsLattices
```

primRecVal.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.FixedPoint.fix
```

```
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_4
public import DomainTheory.DataTypesAsLattices.Lambda.primRecStep
public import DomainTheory.DataTypesAsLattices.Lambda.eq_2_7_zero
public import DomainTheory.DataTypesAsLattices.Lambda.eq_2_7_succ
public import DomainTheory.DataTypesAsLattices.Lambda.predSet
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.predSet_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.27).** Primitive recursion as a distributive function. -/
def primRecVal (a f : Pomega) : ℕ → Pomega
  | 0 => a
  | n + 1 => funOf (funOf f (ofNat n)) (primRecVal a f n)

namespace primRecVal

/-- **Scott 1976, (2.27).** The fixed point of the primrec step agrees
with integer recursion. -/
theorem fix (a f : Pomega) : ∀ n,
    funOf (DomainTheory.DataTypesAsLattices.fix (primRecStep a f)) (ofNat n) =
      primRecVal a f n
  | 0 => by
    have hfix := (theorem_1_4 (primRecStep.isScottContinuous a f)).1
    calc funOf (DomainTheory.DataTypesAsLattices.fix (primRecStep a f)) (ofNat 0)
        = funOf (primRecStep a f
            (DomainTheory.DataTypesAsLattices.fix (primRecStep a f))) (ofNat 0) := by
          rw [hfix]
      _ = condSet (ofNat 0) a
            (funOf (funOf f (predSet (ofNat 0)))
              (funOf (DomainTheory.DataTypesAsLattices.fix (primRecStep a f))
                (predSet (ofNat 0)))) :=
          primRecStep.app a f _ _
      _ = a := eq_2_7_zero _ _
  | n + 1 => by
    have hfix := (theorem_1_4 (primRecStep.isScottContinuous a f)).1
    calc funOf (DomainTheory.DataTypesAsLattices.fix (primRecStep a f)) (ofNat (n + 1)
    )
        = funOf (primRecStep a f
            (DomainTheory.DataTypesAsLattices.fix (primRecStep a f))) (ofNat (n + 1))
    := by
          rw [hfix]
      _ = condSet (ofNat (n + 1)) a
            (funOf (funOf f (predSet (ofNat (n + 1))))
              (funOf (DomainTheory.DataTypesAsLattices.fix (primRecStep a f))
                (predSet (ofNat (n + 1))))) :=
          primRecStep.app a f _ _
      _ = funOf (funOf f (ofNat n))
            (funOf (DomainTheory.DataTypesAsLattices.fix (primRecStep a f)) (ofNat n))
     := by
          rw [eq_2_7_succ, predSet.ofNat_succ]
      _ = funOf (funOf f (ofNat n)) (primRecVal a f n) := by
          rw [?fix? a f n]
```

```
end primRecVal

end DomainTheory.DataTypesAsLattices
```

primRecHat.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.primRecVal
public import Mathlib.Data.Set.Lattice.Indexed
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def primRecHat (a f x : Pomega) : Pomega :=
  ⋃ n ∈ x, primRecVal a f n

namespace primRecHat

theorem zero (a f : Pomega) : primRecHat a f (ofNat 0) = a := by
  ext k
  constructor
  · intro hk
    obtain ⟨n, hn, hkn⟩ := Set.mem_iUnion₂.mp hk
    subst hn; simpa [primRecVal] using hkn
  · intro hk
    exact Set.mem_iUnion₂.mpr ⟨0, rfl, hk⟩

/-- **Scott 1976, (2.27).** The distributive extension agrees with the
integer recursion on singletons. -/
theorem ofNat (a f : Pomega) : ∀ n,
    primRecHat a f (ofNat n) = primRecVal a f n
  | 0 => primRecHat.zero a f
  | n + 1 => by
    ext k
    constructor
    · intro hk
      obtain ⟨m, hm, hkm⟩ := Set.mem_iUnion₂.mp hk
      subst hm; simpa [primRecVal] using hkm
    · intro hk
      exact Set.mem_iUnion₂.mpr ⟨n + 1, rfl, hk⟩

end primRecHat

end DomainTheory.DataTypesAsLattices
```

primRecVal_add.lean

```
module

public import Mathlib.Computability.RE
```

```
public import DomainTheory.DataTypesAsLattices.Lambda.primRecVal
public import DomainTheory.DataTypesAsLattices.Lambda.sucC
public import DomainTheory.DataTypesAsLattices.Lambda.Kcomb
public import DomainTheory.DataTypesAsLattices.Lambda.succSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Computability.pack4
public import DomainTheory.DataTypesAsLattices.Computability.addStepC
public import DomainTheory.DataTypesAsLattices.Lambda.Kcomb_more
public import DomainTheory.DataTypesAsLattices.Lambda.sucC_more
public import DomainTheory.DataTypesAsLattices.Lambda.succSet_more

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem primRecVal_add (k : ℕ) : ∀ n,
    primRecVal (ofNat k) addStepC n = ofNat (k + n)
  | 0 => rfl
  | n + 1 => by
    calc primRecVal (ofNat k) addStepC (n + 1)
        = funOf (funOf addStepC (ofNat n))
            (primRecVal (ofNat k) addStepC n) := rfl
      _ = funOf sucC (primRecVal (ofNat k) addStepC n) := by
          simp [addStepC, Kcomb.beta2]
      _ = funOf sucC (ofNat (k + n)) := by rw [primRecVal_add k n]
      _ = ofNat (k + n + 1) := by rw [sucC.app, succSet.ofNat]

end DomainTheory.DataTypesAsLattices
```

eq_2_27.lean

```
module

public import DomainTheory.DataTypesAsLattices.FixedPoint.fix
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.dollarC
public import DomainTheory.DataTypesAsLattices.Lambda.primRecVal
public import DomainTheory.DataTypesAsLattices.Lambda.primRecHat
public import DomainTheory.DataTypesAsLattices.Lambda.primRecStep
public import DomainTheory.DataTypesAsLattices.Lambda.eq_2_25
public import Mathlib.Data.Set.Lattice.Indexed
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.27).** `p = Y(λu λn∈ω. n ? a, f(n?1)(u(n?1)))`. -/
theorem eq_2_27 (a f x : Pomega) :
```

```
    primRecHat a f x = funOf (funOf dollarC (fix (primRecStep a f))) x := by
  rw [eq_2_25]
  ext k
  constructor
  · intro hk
    obtain ⟨n, hn, hkn⟩ := Set.mem_iUnion₂.mp hk
    refine Set.mem_iUnion₂.mpr ⟨n, hn, ?_⟩
    simpa [primRecVal.fix] using hkn
  · intro hk
    obtain ⟨n, hn, hkn⟩ := Set.mem_iUnion₂.mp hk
    refine Set.mem_iUnion₂.mpr ⟨n, hn, ?_⟩
    simpa [primRecVal.fix] using hkn

end DomainTheory.DataTypesAsLattices
```

primRecComb.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Lambda.interp
public import DomainTheory.DataTypesAsLattices.Lambda.Ycomb
public import DomainTheory.DataTypesAsLattices.Lambda.dollarC
public import DomainTheory.DataTypesAsLattices.Lambda.primRecStepTerm
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Computability.pack4
public import DomainTheory.DataTypesAsLattices.Lambda.primRecHat
public import DomainTheory.DataTypesAsLattices.Lambda.primRecVal
public import DomainTheory.DataTypesAsLattices.Lambda.primRecStep
public import DomainTheory.DataTypesAsLattices.Lambda.Ycomb_more
public import DomainTheory.DataTypesAsLattices.Lambda.eq_2_27
@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable

open Nat.Partrec (Code)

open Nat.Partrec.Code

/-- Scott (2.27) packaged as a combinator. -/
def primRecComb (a f : Pomega) : Pomega :=
  funOf dollarC
    (funOf Ycomb
      (funOf (funOf (interp primRecStepTerm (fun _ => botElem)) a) f))

namespace primRecComb

open Encodable
open Nat.Partrec (Code)
```

```
open Nat.Partrec.Code

theorem combinatory {a f : Pomega}
    (ha : IsCombinatory a) (hf : IsCombinatory f) :
    IsCombinatory (primRecComb a f) :=
  .app dollarC.combinatory <|
    .app Ycomb.combinatory <|
      .app (.app (theorem_2_4.closed primRecStepTerm) ha) hf

theorem ofNat (a f : Pomega) (n : ℕ) :
    funOf (primRecComb a f) (_root_.DomainTheory.DataTypesAsLattices.ofNat n) =
      primRecVal a f n := by
  have hgraph := primRecStep.graph_of a f
  have hfix := theorem_2_5 (primRecStep.isScottContinuous a f)
  simp only [primRecComb]
  rw [← hgraph, hfix, ← _root_.DomainTheory.DataTypesAsLattices.eq_2_27,
    primRecHat.ofNat]

end primRecComb

end DomainTheory.DataTypesAsLattices
```

sndC.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def sndC : Pomega := graph (fun u => funOf u (ofNat 1))

namespace sndC

theorem app (u : Pomega) : funOf sndC u = funOf u (ofNat 1) :=
  beta (funOf.isScottContinuous_left (ofNat 1)) u

end sndC

end DomainTheory.DataTypesAsLattices
```

intF.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import DomainTheory.DataTypesAsLattices.Lambda.dcondSet
public import DomainTheory.DataTypesAsLattices.Lambda.predSet
```

```
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.id_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.const_isScottContinuous
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
public import DomainTheory.DataTypesAsLattices.Lambda.dcondSet_more
public import DomainTheory.DataTypesAsLattices.Lambda.predSet_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (4.7).** The integer functional
`int(u) = u ? 0, (int(u?1) ? u, u)`. -/
def intF (f : Pomega) : Pomega :=
  graph (fun u =>
    dcondSet u (ofNat 0) (dcondSet (funOf f (predSet u)) u u))

namespace intF

theorem map_isScottContinuous (f : Pomega) :
    IsScottContinuous (fun u =>
      dcondSet u (ofNat 0) (dcondSet (funOf f (predSet u)) u u)) :=
  theorem_1_3.nary
    (fun y z => dcondSet.isScottContinuous_left y z)
    (fun x z => dcondSet.isScottContinuous_mid x z)
    (fun x y => dcondSet.isScottContinuous_right x y)
    id_isScottContinuous
    (const_isScottContinuous (ofNat 0))
    (theorem_1_3.nary
      (fun y z => dcondSet.isScottContinuous_left y z)
      (fun x z => dcondSet.isScottContinuous_mid x z)
      (fun x y => dcondSet.isScottContinuous_right x y)
      (theorem_1_3 (funOf.isScottContinuous f) predSet.isScottContinuous)
      id_isScottContinuous
      id_isScottContinuous)

theorem isScottContinuous : IsScottContinuous intF :=
  graph_const_isScottContinuous
    (fun f u => dcondSet u (ofNat 0) (dcondSet (funOf f (predSet u)) u u))
    (fun u =>
      theorem_1_3
        (f := fun z => dcondSet u (ofNat 0) z)
        (g := fun f => dcondSet (funOf f (predSet u)) u u)
        (dcondSet.isScottContinuous_right u (ofNat 0))
        (theorem_1_3
```

```
          (f := fun z => dcondSet z u u)
          (g := fun f => funOf f (predSet u))
          (dcondSet.isScottContinuous_left u u)
          (funOf.isScottContinuous_left (predSet u))))

theorem app (f u : Pomega) :
    funOf (intF f) u =
      dcondSet u (ofNat 0) (dcondSet (funOf f (predSet u)) u u) :=
  beta (map_isScottContinuous f) u

end intF

end DomainTheory.DataTypesAsLattices
```

pairSeq.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.seq2
public import DomainTheory.DataTypesAsLattices.Pomega.e
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.21) / (4.2).** Pairing as a distributive sequence. -/
def pairSeq (x y : Pomega) : Pomega := seq2 x y

end DomainTheory.DataTypesAsLattices
```

comp.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (4.5).** Composition `u ∘ v = λx. u(v(x))`. -/
def comp (u v : Pomega) : Pomega :=
  graph (fun x => funOf u (funOf v x))

namespace comp

theorem app (u v x : Pomega) :
    funOf (comp u v) x = funOf u (funOf v x) :=
  beta (theorem_1_3 (funOf.isScottContinuous u) (funOf.isScottContinuous v)) x
```

```
theorem isScottContinuous_right (u : Pomega) :
    IsScottContinuous (fun v => comp u v) :=
  graph_const_isScottContinuous (fun v x => funOf u (funOf v x))
    (fun x => theorem_1_3 (funOf.isScottContinuous u) (funOf.isScottContinuous_left x)
    )

theorem right_const (a : Pomega) :
    IsScottContinuous (fun u => comp u a) :=
  graph_const_isScottContinuous (fun u x => funOf u (funOf a x))
    (fun x => funOf.isScottContinuous_left (funOf a x))

end comp

end DomainTheory.DataTypesAsLattices
```

arrowR.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.comp
public import DomainTheory.DataTypesAsLattices.Retracts.IsStrict
public import DomainTheory.DataTypesAsLattices.Retracts.typed
public import DomainTheory.DataTypesAsLattices.Retracts.funOf_botElem
public import DomainTheory.DataTypesAsLattices.Retracts.graph_const_bot
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def arrowR (a b : Pomega) : Pomega :=
  graph (fun u => comp b (comp u a))

namespace arrowR

theorem map_isScottContinuous (a b : Pomega) :
    IsScottContinuous (fun u => comp b (comp u a)) :=
  theorem_1_3 (comp.isScottContinuous_right b) (comp.right_const a)

theorem app (a b u : Pomega) :
    funOf (arrowR a b) u = comp b (comp u a) :=
  beta (map_isScottContinuous a b) u

theorem isStrict {a b : Pomega} (hb : IsStrict b) :
    IsStrict (arrowR a b) := by
  change funOf (arrowR a b) botElem = botElem
  rw [app]
```

```
  have hbot : comp botElem a = botElem := by
    unfold comp
    simp [funOf_botElem, graph_const_bot]
  rw [hbot]
  unfold comp
  simp only [funOf_botElem]
  exact (congrArg graph (funext fun _ => hb)).trans graph_const_bot

theorem typed_of {a b u : Pomega} (hu : typed u (arrowR a b)) :
    u = comp b (comp u a) := by
  simpa [typed, app] using hu

end arrowR

end DomainTheory.DataTypesAsLattices
```

eq_4_11.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.comp
public import DomainTheory.DataTypesAsLattices.Retracts.arrowR
public import DomainTheory.DataTypesAsLattices.Retracts.graph_ext

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (4.11).** Action of composed function-space retracts. -/
theorem eq_4_11 (a b a' b' u : Pomega) :
    funOf (arrowR a b) (funOf (arrowR a' b') u) =
      comp b (comp (comp b' (comp u a')) a) := by
  rw [arrowR.app, arrowR.app]

namespace eq_4_11

/-- **Scott 1976, (4.11).** Composition of function-space retracts. -/
theorem comp (a b a' b' : Pomega) :
    DomainTheory.DataTypesAsLattices.comp (arrowR a b) (arrowR a' b') =
      arrowR (DomainTheory.DataTypesAsLattices.comp a' a)
        (DomainTheory.DataTypesAsLattices.comp b b') := by
  apply graph_ext
  intro u
  calc
    funOf (arrowR a b) (funOf (arrowR a' b') u)
        = DomainTheory.DataTypesAsLattices.comp b
            (DomainTheory.DataTypesAsLattices.comp (funOf (arrowR a' b') u) a) :=
          arrowR.app _ _ _
    _ = DomainTheory.DataTypesAsLattices.comp b
            (DomainTheory.DataTypesAsLattices.comp
              (DomainTheory.DataTypesAsLattices.comp b'
                (DomainTheory.DataTypesAsLattices.comp u a')) a) := by
          rw [arrowR.app]
```

```
    _ = DomainTheory.DataTypesAsLattices.comp
            (DomainTheory.DataTypesAsLattices.comp b b')
            (DomainTheory.DataTypesAsLattices.comp u
              (DomainTheory.DataTypesAsLattices.comp a' a)) := by
          apply graph_ext
          intro x
          simp only [DomainTheory.DataTypesAsLattices.comp.app]

end eq_4_11

end DomainTheory.DataTypesAsLattices
```

ComputableFin.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Lambda.ScottContinuousFin
public import DomainTheory.DataTypesAsLattices.Lambda.curryFin
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Computability.IsRE
public import DomainTheory.DataTypesAsLattices.Graph.graph

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable

open Nat.Partrec (Code)

open Nat.Partrec.Code

/-- **Scott 1976, §2, Definition.** A `k`-ary continuous map is computable
when its nested graph (the pairing encoding of
`m ∈ f(e n₀)?(e n_{k-1})`) is r.e. -/
class ComputableFin {n : ℕ} (f : (Fin n → Pomega) → Pomega) : Prop where
  out : ScottContinuousFin f ∧ IsRE (curryFin f)

end DomainTheory.DataTypesAsLattices
```

SK_comp.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Lambda.Kcomb
public import DomainTheory.DataTypesAsLattices.Lambda.Scomb
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Computability.pack4

public import DomainTheory.DataTypesAsLattices.Graph.Basic.notation3
public import DomainTheory.DataTypesAsLattices.Lambda.Kcomb_more
```

```
public import DomainTheory.DataTypesAsLattices.Lambda.Scomb_more
@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem SK_comp (f g x : Pomega) :
    (Scomb ? (Kcomb ? f) ? g) ? x = f ? (g ? x) := by
  rw [Scomb.beta3, Kcomb.beta2]

end DomainTheory.DataTypesAsLattices
```

primRecZeroC.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Lambda.interp
public import DomainTheory.DataTypesAsLattices.Lambda.Ycomb
public import DomainTheory.DataTypesAsLattices.Lambda.dollarC
public import DomainTheory.DataTypesAsLattices.Lambda.primRecStepTerm
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.Kcomb
public import DomainTheory.DataTypesAsLattices.Lambda.Scomb
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Graph.Basic.notation3
public import DomainTheory.DataTypesAsLattices.Computability.pack4
public import DomainTheory.DataTypesAsLattices.Computability.primRecComb
public import DomainTheory.DataTypesAsLattices.Lambda.Ycomb_more
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat_more
public import DomainTheory.DataTypesAsLattices.Computability.SK_comp
@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable

open Nat.Partrec (Code)

open Nat.Partrec.Code

/-- Primitive recursion with zero initial value, viewed as an operator on its step. -/
def primRecZeroC : Pomega :=
  Scomb ? (Kcomb ? dollarC) ?
    (Scomb ? (Kcomb ? Ycomb) ?
      (interp primRecStepTerm (fun _ => botElem) ? ofNat 0))

namespace primRecZeroC
```

```
open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem app (f : Pomega) :
    primRecZeroC ? f = primRecComb (ofNat 0) f := by
  rw [primRecZeroC, SK_comp, SK_comp]
  rfl

theorem combinatory : IsCombinatory primRecZeroC :=
  .app (.app .S (.app .K dollarC.combinatory))
    (.app (.app .S (.app .K Ycomb.combinatory))
      (.app (theorem_2_4.closed primRecStepTerm) (ofNat.combinatory 0)))

end primRecZeroC

end DomainTheory.DataTypesAsLattices
```

addC.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Lambda.interp
public import DomainTheory.DataTypesAsLattices.Lambda.Ycomb
public import DomainTheory.DataTypesAsLattices.Lambda.dollarC
public import DomainTheory.DataTypesAsLattices.Lambda.primRecStepTerm
public import DomainTheory.DataTypesAsLattices.Lambda.Kcomb
public import DomainTheory.DataTypesAsLattices.Lambda.Scomb
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Computability.addStepC
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Computability.pack4
public import DomainTheory.DataTypesAsLattices.Computability.Realizes2
public import DomainTheory.DataTypesAsLattices.Computability.primRecComb
public import DomainTheory.DataTypesAsLattices.Lambda.Kcomb_more
public import DomainTheory.DataTypesAsLattices.Lambda.Scomb_more
public import DomainTheory.DataTypesAsLattices.Lambda.Ycomb_more
public import DomainTheory.DataTypesAsLattices.Computability.primRecVal_add

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable

open Nat.Partrec (Code)

open Nat.Partrec.Code

/-- `λx. primRecComb x (K suc)`. -/
```

```
def addC : Pomega :=
  funOf (funOf Scomb (funOf Kcomb dollarC))
    (funOf (funOf Scomb (funOf Kcomb Ycomb))
      (funOf (funOf Scomb (interp primRecStepTerm (fun _ => botElem)))
        (funOf Kcomb addStepC)))

namespace addC

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem app (x : Pomega) : funOf addC x = primRecComb x addStepC := by
  simp [addC, primRecComb, Scomb.beta3, Kcomb.beta2]

theorem combinatory : IsCombinatory addC :=
  .app (.app .S (.app .K dollarC.combinatory))
    (.app (.app .S (.app .K Ycomb.combinatory))
      (.app (.app .S (theorem_2_4.closed primRecStepTerm))
        (.app .K addStepC.combinatory)))

theorem realizes2 : Realizes2 addC fun n m => n + m := fun n m => by
  simp [addC.app, primRecComb.ofNat, primRecVal_add]

end addC

end DomainTheory.DataTypesAsLattices
```

primRecVal_mul.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Lambda.primRecVal
public import DomainTheory.DataTypesAsLattices.Lambda.Kcomb
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Computability.pack4
public import DomainTheory.DataTypesAsLattices.Computability.addC
public import DomainTheory.DataTypesAsLattices.Lambda.Kcomb_more
@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem primRecVal_mul (k : ℕ) : ∀ n,
    primRecVal (ofNat 0) (funOf Kcomb (funOf addC (ofNat k))) n = ofNat (k * n)
  | 0 => by simp [primRecVal]
  | n + 1 => by
    calc primRecVal (ofNat 0) (funOf Kcomb (funOf addC (ofNat k))) (n + 1)
```

```
        = funOf (funOf (funOf Kcomb (funOf addC (ofNat k))) (ofNat n))
            (primRecVal (ofNat 0) (funOf Kcomb (funOf addC (ofNat k))) n) :=
          rfl
      _ = funOf (funOf addC (ofNat k))
            (ofNat (k * n)) := by
          rw [Kcomb.beta2, primRecVal_mul k n]
      _ = ofNat (k + k * n) := addC.realizes2 k (k * n)
      _ = ofNat (k * (n + 1)) := by rw [Nat.mul_succ, Nat.add_comm]

end DomainTheory.DataTypesAsLattices
```

botElem_combinatory.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Lambda.IsCombinatory
public import DomainTheory.DataTypesAsLattices.Lambda.eq_2_6
public import DomainTheory.DataTypesAsLattices.Lambda.predC
public import DomainTheory.DataTypesAsLattices.Lambda.zeroC
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Computability.pack4

public import DomainTheory.DataTypesAsLattices.Graph.Basic.notation3
public import DomainTheory.DataTypesAsLattices.Lambda.predC_more
@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem botElem_combinatory : IsCombinatory botElem := by
  have h : predC ? zeroC = botElem := by
    rw [predC.app]
    simpa [zeroC] using eq_2_6
  exact h ▶ .app .pred .zero

end DomainTheory.DataTypesAsLattices
```

primRecOpEnv.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Lambda.interp
public import DomainTheory.DataTypesAsLattices.Lambda.Ycomb
public import DomainTheory.DataTypesAsLattices.Lambda.dollarC
public import DomainTheory.DataTypesAsLattices.Lambda.primRecStepTerm
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Computability.pack4
public import DomainTheory.DataTypesAsLattices.Computability.primRecComb
```

```
public import DomainTheory.DataTypesAsLattices.Lambda.Ycomb_more
public import DomainTheory.DataTypesAsLattices.Computability.botElem_combinatory
@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable

open Nat.Partrec (Code)

open Nat.Partrec.Code

/-- Curried operator `λa f. primRecComb a f`. -/
def primRecOpEnv : ℕ → Pomega
  | 2 => dollarC
  | 3 => Ycomb
  | 4 => interp primRecStepTerm (fun _ => botElem)
  | _ => botElem

namespace primRecOpEnv

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem combinatory : ∀ i, IsCombinatory (primRecOpEnv i)
  | 2 => dollarC.combinatory
  | 3 => Ycomb.combinatory
  | 4 => theorem_2_4.closed primRecStepTerm
  | 0 | 1 => botElem_combinatory
  | _ + 5 => botElem_combinatory

end primRecOpEnv

end DomainTheory.DataTypesAsLattices
```

primRecOpC.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Lambda.interp
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Computability.primRecOpEnv
public import DomainTheory.DataTypesAsLattices.Computability.primRecOpTerm
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Graph.Basic.notation3
public import DomainTheory.DataTypesAsLattices.Computability.pack4
public import DomainTheory.DataTypesAsLattices.Computability.primRecComb
public import DomainTheory.DataTypesAsLattices.Lambda.envSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofComb

@[expose] public section
```

```
set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable

open Nat.Partrec (Code)

open Nat.Partrec.Code

def primRecOpC : Pomega := interp primRecOpTerm primRecOpEnv

namespace primRecOpC

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem app (a f : Pomega) :
    (primRecOpC ? a) ? f = primRecComb a f := by
  rw [primRecOpC, primRecOpTerm, theorem_2_2.beta, theorem_2_2.beta]
  simp [interp, envSet, Function.update, primRecOpEnv, primRecComb]

theorem combinatory : IsCombinatory primRecOpC := by
  rw [primRecOpC, theorem_2_4.complete]
  exact ofComb.combinatory _ _ primRecOpEnv.combinatory

end primRecOpC

end DomainTheory.DataTypesAsLattices
```

safePredC.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Lambda.interp
public import DomainTheory.DataTypesAsLattices.Graph.lam
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Computability.pack4
public import DomainTheory.DataTypesAsLattices.Lambda.envSet
public import DomainTheory.DataTypesAsLattices.Lambda.predSet
public import DomainTheory.DataTypesAsLattices.Lambda.theorem_2_2
public import DomainTheory.DataTypesAsLattices.Lambda.theorem_2_4
public import DomainTheory.DataTypesAsLattices.Lambda.eq_2_7_succ
public import DomainTheory.DataTypesAsLattices.Lambda.eq_2_7_zero
public import DomainTheory.DataTypesAsLattices.Lambda.eq_2_7_bot
public import DomainTheory.DataTypesAsLattices.Lambda.predSet_more
@[expose] public section

set_option maxHeartbeats 2000000
```

```
namespace DomainTheory.DataTypesAsLattices

open Encodable

open Nat.Partrec (Code)

open Nat.Partrec.Code

/-- `λx. x ? 0, x?1`, so numerals stay numerals. -/
def safePredC : Pomega :=
  interp (.lam 0 (.cond (.var 0) .zero (.pred (.var 0)))) (fun _ => botElem)

namespace safePredC

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem combinatory : IsCombinatory safePredC :=
  theorem_2_4.closed _

theorem succ (n : ℕ) : funOf safePredC (ofNat (n + 1)) = ofNat n := by
  simp only [safePredC]
  rw [theorem_2_2.beta]
  simp [interp, envSet, Function.update, eq_2_7_succ, predSet.ofNat_succ]

theorem zero : funOf safePredC (ofNat 0) = ofNat 0 := by
  simp only [safePredC]
  rw [theorem_2_2.beta]
  simp [interp, envSet, Function.update, eq_2_7_zero]

end safePredC

end DomainTheory.DataTypesAsLattices
```

subStepC.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Lambda.Kcomb
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Computability.safePredC
public import DomainTheory.DataTypesAsLattices.Computability.pack4

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable
```

```
open Nat.Partrec (Code)

open Nat.Partrec.Code

def subStepC : Pomega := funOf Kcomb safePredC

namespace subStepC

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem combinatory : IsCombinatory subStepC :=
  .app .K safePredC.combinatory

end subStepC

end DomainTheory.DataTypesAsLattices
```

primRecVal_sub.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Lambda.primRecVal
public import DomainTheory.DataTypesAsLattices.Lambda.Kcomb
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Computability.pack4
public import DomainTheory.DataTypesAsLattices.Computability.safePredC
public import DomainTheory.DataTypesAsLattices.Computability.subStepC
public import DomainTheory.DataTypesAsLattices.Lambda.Kcomb_more
@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem primRecVal_sub (k : ℕ) : ∀ n,
    primRecVal (ofNat k) subStepC n = ofNat (k - n)
  | 0 => by simp [primRecVal]
  | n + 1 => by
    calc primRecVal (ofNat k) subStepC (n + 1)
        = funOf (funOf subStepC (ofNat n))
            (primRecVal (ofNat k) subStepC n) := rfl
      _ = funOf safePredC (primRecVal (ofNat k) subStepC n) := by
          simp [subStepC, Kcomb.beta2]
      _ = funOf safePredC (ofNat (k - n)) := by rw [primRecVal_sub k n]
      _ = ofNat (k - (n + 1)) := by
          cases h : k - n with
          | zero =>
```

```
            have : k - (n + 1) = 0 := by omega
            simp [safePredC.zero, this]
          | succ k' =>
            have : k - (n + 1) = k' := by omega
            simp [safePredC.succ, this]

end DomainTheory.DataTypesAsLattices
```

unionTerm.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.interp
public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Lambda.Term
public import DomainTheory.DataTypesAsLattices.Graph.lam
public import DomainTheory.DataTypesAsLattices.Lambda.Kcomb
public import DomainTheory.DataTypesAsLattices.Lambda.condC
public import DomainTheory.DataTypesAsLattices.Lambda.union_via_cond
public import DomainTheory.DataTypesAsLattices.Lambda.envSet
public import DomainTheory.DataTypesAsLattices.Lambda.zeroC
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.Kcomb_more
public import DomainTheory.DataTypesAsLattices.Lambda.condC_more

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

/-- A closed term for binary union.  Keeping this term explicit avoids making
`fin` choose a fresh code separately for every finite set. -/
def unionTerm : Term :=
  .lam 0 (.lam 1 (.cond (.lam 2 .zero) (.var 0) (.var 1)))

namespace unionTerm

theorem interp :
    interp unionTerm (fun _ => botElem) =
      graph (fun x => graph (fun y => x ∪ y)) := by
  unfold unionTerm
  simp only[_root_.DomainTheory.DataTypesAsLattices.interp, envSet, Function.
    update_self]
  apply congrArg graph
  funext x
  apply congrArg graph
  funext y
  change condSet (graph fun _ => zeroC) x y = x ∪ y
  simpa [condC.beta3, Kcomb.beta] using union_via_cond x y
```

```
end unionTerm

end DomainTheory.DataTypesAsLattices
```

unionOperator_combinatory.lean

```
module

public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Lambda.IsCombinatory
public import DomainTheory.DataTypesAsLattices.Lambda.theorem_2_4
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Enumeration.unionTerm
@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

theorem unionOperator_combinatory :
    IsCombinatory (graph (fun x => graph (fun y => x ∪ y))) := by
  rw [← unionTerm.interp]
  exact theorem_2_4.closed unionTerm

end DomainTheory.DataTypesAsLattices
```

Retract.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.comp

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, §4, Definition.** `a` is a retract iff `a = a ∘ a`. -/
class Retract (a : Pomega) : Prop where
  out : a = comp a a

end DomainTheory.DataTypesAsLattices
```

funRetract.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Retracts.graph_ext
```

```
public import DomainTheory.DataTypesAsLattices.Retracts.Retract
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (4.1).** The function-space retract `fun = λu λx. u(x)`. -/
def funRetract : Pomega :=
  graph (fun u => graph (fun x => funOf u x))

namespace funRetract

theorem app (u : Pomega) :
    funOf funRetract u = graph (fun x => funOf u x) :=
  beta (graph_const_isScottContinuous (fun u x => funOf u x)
    (fun x => funOf.isScottContinuous_left x)) u

theorem isRetract : Retract funRetract := by
  constructor
  apply graph_ext
  intro u
  rw [funRetract.app, funRetract.app]
  apply graph_ext
  intro x
  exact (beta (funOf.isScottContinuous u) x).symm

end funRetract

end DomainTheory.DataTypesAsLattices
```

intR.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.dcondSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.predSet
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.intF
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_4
public import DomainTheory.DataTypesAsLattices.FixedPoint.fix
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Retracts.dcondSet_same_ofNat
public import DomainTheory.DataTypesAsLattices.Retracts.predSet_eq_ofNat_of_no_zero
public import DomainTheory.DataTypesAsLattices.Retracts.graph_ext
public import DomainTheory.DataTypesAsLattices.Retracts.Retract
public import DomainTheory.DataTypesAsLattices.Lambda.dcondSet_more
public import DomainTheory.DataTypesAsLattices.Lambda.predSet_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices
```

```
/-- **Scott 1976, (4.7).** Integer retract as the least fixed point. -/
def intR : Pomega := fix intF

namespace intR

theorem eq_intF : intF intR = intR :=
  (theorem_1_4 intF.isScottContinuous).1

theorem app (u : Pomega) :
    funOf intR u =
      dcondSet u (ofNat 0) (dcondSet (funOf intR (predSet u)) u u) := by
  have h := congrArg (fun a => funOf a u) eq_intF
  simpa [intF.app] using h.symm

theorem bot : funOf intR botElem = botElem := by
  rw [app, dcondSet.bot]

theorem app_ofNat : ∀ n, funOf intR (ofNat n) = ofNat n
  | 0 => by
    rw [app, dcondSet.ofNat_zero]
  | n + 1 => by
    rw [app, predSet.ofNat_succ, app_ofNat n, dcondSet_same_ofNat,
      dcondSet.ofNat_succ]

theorem top_of_nonsingleton :
    ∀ n u, n ∈ u → (∀ k ∈ u, n ≤ k) → u = botElem → (∀ k, u = ofNat k) →
      funOf intR u = topElem := by
  intro n
  induction n using Nat.strongRecOn with
  | ind n ih =>
    intro u hn hmin hne hns
    rw [app]
    by_cases h0 : 0 ∈ u
    · have hp : ∃ t, t + 1 ∈ u := by
        by_contra hp
        apply hns 0
        ext m
        constructor
        · intro hm
          have : m = 0 := by
            by_contra hm0
            obtain ⟨t, rfl⟩ := Nat.exists_eq_succ_of_ne_zero hm0
            exact hp ⟨t, hm⟩
          simp [ofNat, this]
        · intro hm
          simpa [ofNat] using hm.symm ▸ h0
      rw [dcondSet.of_mixed h0 hp]
    · have hn0 : n = 0 := fun h => h0 (by simpa [h] using hn)
      obtain ⟨n', rfl⟩ := Nat.exists_eq_succ_of_ne_zero hn0
      have hn' : n' ∈ predSet u := hn
      have hmin' : ∀ k ∈ predSet u, n' ≤ k := by
        intro k hk
        have : n' + 1 ≤ k + 1 := hmin (k + 1) hk
        omega
      have hne' : predSet u = botElem := by
        intro hempty
```

```
        have : n' ∈ predSet u := hn'
        rw [hempty] at this
        simp [botElem] at this
      have hns' : ∀ k, predSet u = ofNat k := by
        intro k hk
        exact hns (k + 1) (predSet_eq_ofNat_of_no_zero h0 hk)
      have hpred : funOf intR (predSet u) = topElem :=
        ih n' (Nat.lt_succ_self n') (predSet u) hn' hmin' hne' hns'
      rw [hpred, dcondSet.top, dcondSet.of_pos_not_zero h0 ⟨n', hn⟩]

theorem top : funOf intR topElem = topElem :=
  top_of_nonsingleton 0 topElem (Set.mem_univ 0)
    (fun _ _ => Nat.zero_le _)
    (by
      intro h
      have : (0 : ℕ) ∈ (botElem : Pomega) := by
        rw [← h]
        exact Set.mem_univ 0
      simp [botElem] at this)
    (fun n h => by
      have h0 : (0 : ℕ) ∈ ofNat n := by
        have : (0 : ℕ) ∈ topElem := Set.mem_univ 0
        rwa [h] at this
      have h1 : (1 : ℕ) ∈ ofNat n := by
        have : (1 : ℕ) ∈ topElem := Set.mem_univ 1
        rwa [h] at this
      simp [ofNat] at h0 h1
      omega)

theorem idem (x : Pomega) : funOf intR (funOf intR x) = funOf intR x := by
  by_cases hbot : x = botElem
  · subst hbot
    rw [bot, bot]
  · by_cases hsing : ∃ n, x = ofNat n
    · obtain ⟨n, rfl⟩ := hsing
      rw [app_ofNat, app_ofNat]
    · have hx : funOf intR x = topElem := by
        have hne : x.Nonempty := by
          rw [Set.nonempty_iff_ne_empty]
          simpa [botElem] using hbot
        exact top_of_nonsingleton (sInf x) x (Nat.sInf_mem hne)
          (fun k hk => Nat.sInf_le hk) hbot (fun n hn => hsing ⟨n, hn⟩)
      rw [hx, top]

theorem isGraph : intR = graph (funOf intR) := by
  refine eq_intF.symm.trans ?_
  apply graph_ext
  intro u
  exact (app u).symm

theorem isRetract : Retract intR := by
  constructor
  change intR = graph (fun x => funOf intR (funOf intR x))
  refine isGraph.trans ?_
  apply graph_ext
  intro x
```

```
  exact (idem x).symm

end intR

end DomainTheory.DataTypesAsLattices
```

retract_interpolation.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Continuous.mem_scottUnion
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Continuous.scottUnion
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.Retract
public import DomainTheory.DataTypesAsLattices.Retracts.typed
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- Continuous-lattice interpolation: `x = ⋃ { a(e n) | e n ⊆ x }`. -/
theorem retract_interpolation {a x : Pomega} (_ha : Retract a) (hx : typed x a) :
    x = ⋃ n, {k | e n ⊆ x ∧ k ∈ funOf a (e n)} := by
  have hx' : funOf a x = x := hx.symm
  have hsc := (funOf.isScottContinuous a).out x
  apply subset_antisymm
  · intro k hk
    have : k ∈ funOf a x := by rwa [hx']
    have : k ∈ scottUnion (funOf a) x := by rwa [← hsc]
    obtain ⟨n, hn, hkn⟩ := mem_scottUnion.mp this
    exact Set.mem_iUnion.mpr ⟨n, hn, hkn⟩
  · intro k hk
    obtain ⟨n, hn, hkn⟩ := Set.mem_iUnion.mp hk
    have : k ∈ funOf a x :=
      IsScottContinuous.monotone (funOf.isScottContinuous a) hn hkn
    rwa [hx'] at this

end DomainTheory.DataTypesAsLattices
```

boolR.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.dcondSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
```

```
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Retracts.graph_ext
public import DomainTheory.DataTypesAsLattices.Retracts.dcondSet_bool_idem
public import DomainTheory.DataTypesAsLattices.Retracts.Retract
public import DomainTheory.DataTypesAsLattices.Lambda.dcondSet_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (4.3).** Boolean retract via the doubly strict conditional. -/
def boolR : Pomega :=
  graph (fun u => dcondSet u (ofNat 0) (ofNat 1))

namespace boolR

theorem app (u : Pomega) :
    funOf boolR u = dcondSet u (ofNat 0) (ofNat 1) :=
  beta (dcondSet.isScottContinuous_left (ofNat 0) (ofNat 1)) u

theorem isRetract : Retract boolR := by
  constructor
  apply graph_ext
  intro u
  rw [app, app, dcondSet_bool_idem]

end boolR

end DomainTheory.DataTypesAsLattices
```

Icomb.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.id_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Retracts.Retract

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- The identity combinator `I = λu. u`, the largest retract. -/
def Icomb : Pomega := graph (fun x => x)

namespace Icomb

theorem app (x : Pomega) : funOf Icomb x = x :=
  beta id_isScottContinuous x

theorem isRetract : Retract Icomb := by
```

```
  constructor
  change Icomb = graph (fun x => funOf Icomb (funOf Icomb x))
  have : (fun x => funOf Icomb (funOf Icomb x)) = fun x => x := by
    funext x
    simp [app]
  rw [this]
  rfl

end Icomb

end DomainTheory.DataTypesAsLattices
```

retractLe.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.comp
public import DomainTheory.DataTypesAsLattices.Retracts.Retract

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, §4, Definition.** `a ⊑ b` iff `a = a ∘ b = b ∘ a`. -/
def retractLe (a b : Pomega) : Prop :=
  a = comp a b ∧ a = comp b a

namespace retractLe

theorem refl {a : Pomega} (ha : Retract a) : retractLe a a :=
  ⟨ha.out, ha.out⟩

theorem trans {a b c : Pomega}
    (hab : retractLe a b) (hbc : retractLe b c) :
    retractLe a c := by
  rcases hab with ⟨hab₁, hab₂⟩
  rcases hbc with ⟨hbc₁, hbc₂⟩
  constructor
  · apply Eq.trans hab₁
    apply Eq.symm
    calc
      comp a c = graph (fun x => funOf a (funOf c x)) := rfl
      _ = graph (fun x => funOf (comp a b) (funOf c x)) := by simp [← hab₁]
      _ = graph (fun x => funOf a (funOf b (funOf c x))) := by simp [comp.app]
      _ = graph (fun x => funOf a (funOf (comp b c) x)) := by simp [comp.app]
      _ = graph (fun x => funOf a (funOf b x)) := by simp [← hbc₁]
      _ = comp a b := rfl
  · apply Eq.trans hab₂
    apply Eq.symm
    calc
      comp c a = graph (fun x => funOf c (funOf a x)) := rfl
      _ = graph (fun x => funOf c (funOf (comp b a) x)) := by simp [← hab₂]
```

```
      _ = graph (fun x => funOf c (funOf b (funOf a x))) := by simp [comp.app]
      _ = graph (fun x => funOf (comp c b) (funOf a x)) := by simp [comp.app]
      _ = graph (fun x => funOf b (funOf a x)) := by simp [← hbc₂]
      _ = comp b a := rfl

end retractLe

end DomainTheory.DataTypesAsLattices
```

Closure.lean

```
module

public import Mathlib.Algebra.Ring.Parity
public import DomainTheory.DataTypesAsLattices.Retracts.Retract
public import DomainTheory.DataTypesAsLattices.Retracts.Icomb
public import DomainTheory.DataTypesAsLattices.Pomega.e
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, §5, Definition.** `a` is a closure operation iff `I ⊆ a = a ∘ a`. -/
class Closure (a : Pomega) : Prop where
  out : Icomb ⊆ a ∧ Retract a

end DomainTheory.DataTypesAsLattices
```

Icomb_isClosure.lean

```
module

public import Mathlib.Algebra.Ring.Parity
public import DomainTheory.DataTypesAsLattices.Retracts.Icomb
public import DomainTheory.DataTypesAsLattices.Closures.Closure
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem Icomb_isClosure : Closure Icomb :=
  ⟨⟨subset_rfl, Icomb.isRetract⟩⟩

end DomainTheory.DataTypesAsLattices
```

retract_app.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.comp
public import DomainTheory.DataTypesAsLattices.Retracts.Retract
```

```
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem retract_app {a : Pomega} (ha : Retract a) (x : Pomega) :
    funOf a (funOf a x) = funOf a x := by
  have := congrArg (fun u => funOf u x) ha.out
  simpa [comp.app] using this.symm

end DomainTheory.DataTypesAsLattices
```

REPredu2eand.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Retracts.comp

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

theorem REPred.and {α} [Primcodable α] {p q : α → Prop}
    (hp : REPred p) (hq : REPred q) :
    REPred fun a => p a ∧ q a := by
  refine (hp.bind (hq.comp Computable.fst).to₂).of_eq fun a => ?_
  ext
  simp [Part.assert]

end DomainTheory.DataTypesAsLattices
```

REPredu2ecomp.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Retracts.comp

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

theorem REPred.comp {α β} [Primcodable α] [Primcodable β] {p : β → Prop}
    (hp : REPred p) {f : α → β} (hf : Computable f) :
    REPred fun a => p (f a) :=
  Partrec.comp hp hf

end DomainTheory.DataTypesAsLattices
```

REPredu2eexists.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.comp

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

/-- Projection of an r.e. binary predicate, by dovetailing `Code.evaln`. -/
theorem REPred.exists {p : ℕ → ℕ → Prop}
    (hp : REPred fun q : ℕ × ℕ => p q.1 q.2) :
    REPred fun n => ∃ m, p n m := by
  have hnat : Nat.Partrec fun k : ℕ =>
      Part.bind (decode₂ (ℕ × ℕ) k) fun q : ℕ × ℕ =>
        (Part.assert (p q.1 q.2) fun _ => Part.some ()).map encode :=
    (Partrec.bind_decode₂_iff
      (f := fun q : ℕ × ℕ => Part.assert (p q.1 q.2) fun _ => Part.some ())).mp hp
  obtain ⟨c, hc⟩ := Code.exists_code.mp hnat
  have hfn : Computable₂ fun n k : ℕ =>
      (evaln k.unpair.2 c (encode (n, k.unpair.1))).map fun _ : ℕ => () := by
    refine (Primrec.option_map ?_ (Primrec.const ()).to₂).to_comp
    exact primrec_evaln.comp <|
      Primrec.pair
        (Primrec.pair (Primrec.snd.comp (Primrec.unpair.comp Primrec.snd))
          (Primrec.const c))
        (Primrec.encode.comp
          (Primrec.pair Primrec.fst
            (Primrec.fst.comp (Primrec.unpair.comp Primrec.snd))))
  have hr : REPred fun n =>
      (Nat.rfindOpt fun k =>
        (evaln k.unpair.2 c (encode (n, k.unpair.1))).map fun _ : ℕ => ()).Dom :=
    Partrec.dom_re (Partrec.rfindOpt hfn)
  refine _root_.REPred.of_eq hr fun n => ?_
  constructor
  · intro hdom
    rw [Nat.rfindOpt_dom] at hdom
    obtain ⟨k, u, hu⟩ := hdom
    cases hopt : evaln k.unpair.2 c (encode (n, k.unpair.1)) with
    | none =>
      rw [hopt] at hu
      cases hu
    | some val =>
      have hin : val ∈ evaln k.unpair.2 c (encode (n, k.unpair.1)) := by
        rw [hopt]; rfl
      have hsound : val ∈ Code.eval c (encode (n, k.unpair.1)) :=
        evaln_sound hin
      rw [hc] at hsound
```

```
      rw [Part.mem_bind_iff] at hsound
      obtain ⟨q, hq, hv⟩ := hsound
      have hqe : decode₂ (ℕ × ℕ) (encode (n, k.unpair.1)) = some q := by
        simpa using hq
      have hq' : q = (n, k.unpair.1) :=
        Option.some_inj.mp (hqe.symm.trans (decode₂_encode (n, k.unpair.1)))
      subst hq'
      rw [Part.mem_map_iff] at hv
      obtain ⟨u, hu, _⟩ := hv
      exact ⟨k.unpair.1, (Part.mem_assert_iff.mp hu).1⟩
  · intro ⟨m, hm⟩
    have hmem : encode () ∈ Code.eval c (encode (n, m)) := by
      rw [hc, Part.mem_bind_iff]
      refine ⟨(n, m), ?_, ?_⟩
      · rw [decode₂_encode]
        exact Part.mem_some_iff.mpr rfl
      · rw [Part.mem_map_iff]
        exact ⟨(), Part.mem_assert_iff.mpr ⟨hm, Part.mem_some_iff.mpr rfl⟩, rfl⟩
    obtain ⟨s, hs⟩ := evaln_complete.mp hmem
    have hs' : evaln s c (Nat.pair n m) = some (encode ()) := hs
    refine (Nat.rfindOpt_dom (f := fun k =>
      (evaln k.unpair.2 c (encode (n, k.unpair.1))).map fun _ => ())).mpr
      ⟨Nat.pair m s, (), ?_⟩
    simp [Nat.unpair_pair, hs']

end DomainTheory.DataTypesAsLattices
```

mulC.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Lambda.interp
public import DomainTheory.DataTypesAsLattices.Lambda.Ycomb
public import DomainTheory.DataTypesAsLattices.Lambda.dollarC
public import DomainTheory.DataTypesAsLattices.Lambda.primRecStepTerm
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.Kcomb
public import DomainTheory.DataTypesAsLattices.Lambda.Scomb
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Computability.addC
public import DomainTheory.DataTypesAsLattices.Graph.Basic.notation3
public import DomainTheory.DataTypesAsLattices.Computability.pack4
public import DomainTheory.DataTypesAsLattices.Computability.Realizes2
public import DomainTheory.DataTypesAsLattices.Computability.primRecComb
public import DomainTheory.DataTypesAsLattices.Lambda.Ycomb_more
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat_more
public import DomainTheory.DataTypesAsLattices.Computability.SK_comp
public import DomainTheory.DataTypesAsLattices.Computability.primRecVal_mul

@[expose] public section

set_option maxHeartbeats 2000000
```

```
namespace DomainTheory.DataTypesAsLattices

open Encodable

open Nat.Partrec (Code)

open Nat.Partrec.Code

/-- `λx. primRecComb 0 (K (addC x))`. -/
def mulC : Pomega :=
  funOf (funOf Scomb (funOf Kcomb dollarC))
    (funOf (funOf Scomb (funOf Kcomb Ycomb))
      (funOf (funOf Scomb
          (funOf Kcomb
            (funOf (interp primRecStepTerm (fun _ => botElem)) (ofNat 0))))
        (funOf (funOf Scomb (funOf Kcomb Kcomb)) addC)))

namespace mulC

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem app (x : Pomega) :
    mulC ? x = primRecComb (ofNat 0) (Kcomb ? (addC ? x)) := by
  rw [mulC, primRecComb, SK_comp, SK_comp, SK_comp, SK_comp]

theorem combinatory : IsCombinatory mulC :=
  .app (.app .S (.app .K dollarC.combinatory))
    (.app (.app .S (.app .K Ycomb.combinatory))
      (.app (.app .S
          (.app .K (.app (theorem_2_4.closed primRecStepTerm)
            (ofNat.combinatory 0))))
        (.app (.app .S (.app .K .K)) addC.combinatory)))

theorem realizes2 : Realizes2 mulC fun n m => n * m := fun n m => by
  simp [mulC.app, primRecComb.ofNat, primRecVal_mul]

end mulC

end DomainTheory.DataTypesAsLattices
```

primrecPred_bool.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Retracts.comp

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices
```

```
open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem primrecPred_bool {α} [Primcodable α] {f : α → Bool} (hf : Primrec f) :
    PrimrecPred fun a => f a = true :=
  Primrec.eq.comp hf (Primrec.const true)

end DomainTheory.DataTypesAsLattices
```

primrec_enumFun.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Computability.pack4
public import DomainTheory.DataTypesAsLattices.Computability.enumFun
public import DomainTheory.DataTypesAsLattices.Retracts.comp

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem primrec_enumFun (c : Code) : Nat.Primrec (enumFun c) := by
  have heval : Primrec fun t : ℕ => evaln t.unpair.2 c t.unpair.1 :=
    primrec_evaln.comp <|
      Primrec.pair
        (Primrec.pair (Primrec.snd.comp Primrec.unpair) (Primrec.const c))
        (Primrec.fst.comp Primrec.unpair)
  have h : Primrec fun t : ℕ =>
      bif (evaln t.unpair.2 c t.unpair.1).isSome
        then t.unpair.1 + 1 else 0 :=
    (Primrec.cond (Primrec.option_isSome.comp heval)
      (Primrec.succ.comp (Primrec.fst.comp Primrec.unpair))
      (Primrec.const 0)).of_eq fun t => by simp [Nat.succ_eq_add_one]
  exact Primrec.nat_iff.mp (h.of_eq fun _ => rfl)

end DomainTheory.DataTypesAsLattices
```

primrec_pack4_proj1.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Computability.pack4
public import DomainTheory.DataTypesAsLattices.Retracts.comp
```

```
@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem primrec_pack4_proj1 : Primrec pack4.proj1 :=
  Primrec.fst.comp Primrec.unpair

end DomainTheory.DataTypesAsLattices
```

primrec_pack4_proj2.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Computability.pack4
public import DomainTheory.DataTypesAsLattices.Retracts.comp

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem primrec_pack4_proj2 : Primrec pack4.proj2 :=
  Primrec.fst.comp (Primrec.unpair.comp (Primrec.snd.comp Primrec.unpair))

end DomainTheory.DataTypesAsLattices
```

primrec_pack4_proj3.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Computability.pack4
public import DomainTheory.DataTypesAsLattices.Retracts.comp

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable
```

```
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem primrec_pack4_proj3 : Primrec pack4.proj3 :=
  Primrec.fst.comp (Primrec.unpair.comp
    (Primrec.snd.comp (Primrec.unpair.comp (Primrec.snd.comp Primrec.unpair))))

end DomainTheory.DataTypesAsLattices
```

primrec_pack4_proj4.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Computability.pack4
public import DomainTheory.DataTypesAsLattices.Retracts.comp

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem primrec_pack4_proj4 : Primrec pack4.proj4 :=
  Primrec.snd.comp (Primrec.unpair.comp
    (Primrec.snd.comp (Primrec.unpair.comp (Primrec.snd.comp Primrec.unpair))))

end DomainTheory.DataTypesAsLattices
```

primrec_pair.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Retracts.comp

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

theorem primrec_pair : Primrec₂ pair :=
  (Primrec.nat_add.comp
    (Primrec.nat_div.comp
      (Primrec.nat_mul.comp
        (Primrec.nat_add.comp Primrec.fst Primrec.snd)
        (Primrec.succ.comp (Primrec.nat_add.comp Primrec.fst Primrec.snd)))
      (Primrec.const 2))
```

```
    Primrec.snd).of_eq fun _ => rfl

end DomainTheory.DataTypesAsLattices
```

IsREu2epair_mem.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Computability.IsRE
public import DomainTheory.DataTypesAsLattices.Computability.primrec_pair
public import DomainTheory.DataTypesAsLattices.Retracts.comp

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem IsRE.pair_mem {u : Pomega} (hu : IsRE u) :
    REPred fun q : ℕ × ℕ => pair q.1 q.2 ∈ u :=
  hu.out.comp primrec_pair.to_comp

end DomainTheory.DataTypesAsLattices
```

primrecPred_Scomb_eq.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Computability.pack4
public import DomainTheory.DataTypesAsLattices.Computability.primrec_pair
public import DomainTheory.DataTypesAsLattices.Computability.primrec_pack4_proj1
public import DomainTheory.DataTypesAsLattices.Computability.primrec_pack4_proj2
public import DomainTheory.DataTypesAsLattices.Computability.primrec_pack4_proj3
public import DomainTheory.DataTypesAsLattices.Computability.primrec_pack4_proj4
public import DomainTheory.DataTypesAsLattices.Retracts.comp

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code
```

```
theorem primrecPred_Scomb_eq :
    PrimrecPred fun q : ℕ × ℕ =>
      q.1 = pair (pack4.proj1 q.2)
        (pair (pack4.proj2 q.2)
          (pair (pack4.proj3 q.2) (pack4.proj4 q.2))) :=
  Primrec.eq.comp Primrec.fst
    (primrec_pair.comp (primrec_pack4_proj1.comp Primrec.snd)
      (primrec_pair.comp (primrec_pack4_proj2.comp Primrec.snd)
        (primrec_pair.comp (primrec_pack4_proj3.comp Primrec.snd)
          (primrec_pack4_proj4.comp Primrec.snd))))

end DomainTheory.DataTypesAsLattices
```

primrecPred_eq_scott_pair.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Computability.primrec_pair
public import DomainTheory.DataTypesAsLattices.Retracts.comp

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem primrecPred_eq_scott_pair :
    PrimrecPred fun t : ℕ × ℕ × ℕ => t.1 = pair t.2.1 t.2.2 :=
  Primrec.eq.comp Primrec.fst
    (primrec_pair.comp (Primrec.fst.comp Primrec.snd) (Primrec.snd.comp Primrec.snd))

end DomainTheory.DataTypesAsLattices
```

IsREu2egraph_of_re.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Computability.IsRE
public import DomainTheory.DataTypesAsLattices.Computability.primrecPred_eq_scott_pair
public import DomainTheory.DataTypesAsLattices.Retracts.comp
public import DomainTheory.DataTypesAsLattices.Computability.REPredu2eexists
public import DomainTheory.DataTypesAsLattices.Computability.REPredu2eand

@[expose] public section
```

```
set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem IsRE.graph_of_re {f : Pomega → Pomega}
    (h : REPred fun q : ℕ × ℕ => q.2 ∈ f (e q.1)) :
    IsRE (graph f) := by
  have heq : PrimrecPred fun q : ℕ × ℕ =>
      q.1 = pair (Nat.unpair q.2).1 (Nat.unpair q.2).2 :=
    primrecPred_eq_scott_pair.comp
      (Primrec.pair Primrec.fst (Primrec.unpair.comp Primrec.snd))
  have hmem : REPred fun q : ℕ × ℕ =>
      (Nat.unpair q.2).2 ∈ f (e (Nat.unpair q.2).1) :=
    h.comp (Primrec.pair
      (Primrec.fst.comp (Primrec.unpair.comp Primrec.snd))
      (Primrec.snd.comp (Primrec.unpair.comp Primrec.snd))).to_comp
  have hex : REPred fun p =>
      ∃ t, p = pair (Nat.unpair t).1 (Nat.unpair t).2 ∧
        (Nat.unpair t).2 ∈ f (e (Nat.unpair t).1) :=
    REPred.exists (REPred.and heq.computablePred.to_re hmem)
  constructor
  refine hex.of_eq fun p => ?_
  constructor
  · intro ⟨t, ht, hm⟩
    exact ⟨(Nat.unpair t).1, (Nat.unpair t).2, ht, hm⟩
  · intro ⟨n, m, hm, hmem'⟩
    refine ⟨Nat.pair n m, ?_, ?_⟩
    · simpa [Nat.unpair_pair] using hm
    · simpa [Nat.unpair_pair] using hmem'

end DomainTheory.DataTypesAsLattices
```

primrec_twoPow.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Retracts.comp

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

theorem primrec_twoPow : Primrec fun k : ℕ => 2 ^ k :=
  (Primrec.nat_rec' (f := id) (g := fun _ => (1 : ℕ))
    (h := fun (_ : ℕ) (p : ℕ × ℕ) => p.2 * 2)
    Primrec.id (Primrec.const 1)
    (show Primrec fun q : ℕ × ℕ × ℕ => q.2.2 * 2 from
      Primrec.nat_mul.comp (Primrec.snd.comp Primrec.snd) (Primrec.const 2))).of_eq
```

```
    fun k => by
      induction k with
      | zero => simp
      | succ k ih =>
        change (Nat.rec 1 (fun n IH => (n, IH).2 * 2) k) * 2 = 2 ^ (k + 1)
        have ih' : Nat.rec 1 (fun n IH => (n, IH).2 * 2) k = 2 ^ k := by
          simpa [id] using ih
        rw [ih', Nat.pow_succ, Nat.mul_comm]

end DomainTheory.DataTypesAsLattices
```

primrec_testBit.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Computability.primrec_twoPow
public import DomainTheory.DataTypesAsLattices.Retracts.comp

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

theorem primrec_testBit : Primrec₂ fun n k : ℕ => n.testBit k :=
  (Primrec.eq.decide.comp
    (Primrec.nat_mod.comp
      (Primrec.nat_div.comp Primrec.fst (primrec_twoPow.comp Primrec.snd))
      (Primrec.const 2))
    (Primrec.const 1)).of_eq fun _p =>
      Nat.testBit_eq_decide_div_mod_eq.symm

end DomainTheory.DataTypesAsLattices
```

IsREu2eKcomb.lean

```
module
public import DomainTheory.DataTypesAsLattices.Computability.REPredu2eempty
public import DomainTheory.DataTypesAsLattices.Computability.REPredu2etrue
public import DomainTheory.DataTypesAsLattices.Computability.REPredu2ecomp
public import DomainTheory.DataTypesAsLattices.Computability.REPredu2eexists
public import DomainTheory.DataTypesAsLattices.Computability.REPredu2eor
public import DomainTheory.DataTypesAsLattices.Computability.REPredu2eand

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Lambda.Kcomb
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Computability.IsRE
public import DomainTheory.DataTypesAsLattices.Computability.primrec_testBit
public import DomainTheory.DataTypesAsLattices.Computability.primrec_pair
public import DomainTheory.DataTypesAsLattices.Computability.primrecPred_bool
public import DomainTheory.DataTypesAsLattices.Computability.mem_graph_const_iff
public import DomainTheory.DataTypesAsLattices.Retracts.comp
```

```
public import DomainTheory.DataTypesAsLattices.Computability.IsREu2egraph_of_re

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem IsRE.Kcomb : IsRE Kcomb :=
  IsRE.graph_of_re (f := fun x => graph (fun _ => x)) <| by
    have hP : PrimrecPred fun w : ℕ × ℕ =>
        (Nat.unpair w.1).2 = pair (Nat.unpair w.2).1 (Nat.unpair w.2).2 ∧
          (Nat.unpair w.1).1.testBit (Nat.unpair w.2).2 = true :=
      (Primrec.eq.comp
          (Primrec.snd.comp (Primrec.unpair.comp Primrec.fst))
          (primrec_pair.comp
            (Primrec.fst.comp (Primrec.unpair.comp Primrec.snd))
            (Primrec.snd.comp (Primrec.unpair.comp Primrec.snd)))).and
        (primrecPred_bool (Primrec₂.comp primrec_testBit
          (Primrec.fst.comp (Primrec.unpair.comp Primrec.fst))
          (Primrec.snd.comp (Primrec.unpair.comp Primrec.snd))))
    have hex : REPred fun w : ℕ =>
        ∃ t, (Nat.unpair w).2 = pair (Nat.unpair t).1 (Nat.unpair t).2 ∧
          (Nat.unpair w).1.testBit (Nat.unpair t).2 = true :=
      REPred.exists hP.computablePred.to_re
    exact _root_.REPred.of_eq
      (hex.comp (Primrec₂.natPair.comp Primrec.fst Primrec.snd).to_comp)
      fun q => by
        simp only [Nat.unpair_pair]
        exact (mem_graph_const_iff q.1 q.2).symm

end DomainTheory.DataTypesAsLattices
```

IsREu2econdC.lean

```
module
public import DomainTheory.DataTypesAsLattices.Computability.REPredu2eempty
public import DomainTheory.DataTypesAsLattices.Computability.REPredu2etrue
public import DomainTheory.DataTypesAsLattices.Computability.REPredu2ecomp
public import DomainTheory.DataTypesAsLattices.Computability.REPredu2eor
public import DomainTheory.DataTypesAsLattices.Computability.REPredu2eand

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Lambda.condC
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Computability.IsRE
public import DomainTheory.DataTypesAsLattices.Computability.primrec_testBit
public import DomainTheory.DataTypesAsLattices.Computability.primrec_pair
public import DomainTheory.DataTypesAsLattices.Computability.primrecPred_bool
public import DomainTheory.DataTypesAsLattices.Computability.mem_condC_iff
public import DomainTheory.DataTypesAsLattices.Retracts.comp
```

```
public import DomainTheory.DataTypesAsLattices.Computability.REPredu2eexists

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem IsRE.condC : IsRE condC := by
  have heq : PrimrecPred fun q : ℕ × ℕ =>
      q.1 = pair (Nat.unpair q.2).1
        (pair (Nat.unpair (Nat.unpair q.2).2).1
          (pair (Nat.unpair (Nat.unpair (Nat.unpair q.2).2).2).1
            (Nat.unpair (Nat.unpair (Nat.unpair q.2).2).2).2)) :=
    Primrec.eq.comp Primrec.fst
      (primrec_pair.comp
        (Primrec.fst.comp (Primrec.unpair.comp Primrec.snd))
        (primrec_pair.comp
          (Primrec.fst.comp (Primrec.unpair.comp
            (Primrec.snd.comp (Primrec.unpair.comp Primrec.snd))))
          (primrec_pair.comp
            (Primrec.fst.comp (Primrec.unpair.comp
              (Primrec.snd.comp (Primrec.unpair.comp
                (Primrec.snd.comp (Primrec.unpair.comp Primrec.snd))))))
            (Primrec.snd.comp (Primrec.unpair.comp
              (Primrec.snd.comp (Primrec.unpair.comp
                (Primrec.snd.comp (Primrec.unpair.comp Primrec.snd)))))))))
  have hbit0 : PrimrecPred fun q : ℕ × ℕ =>
      (Nat.unpair q.2).1.testBit
          (Nat.unpair (Nat.unpair (Nat.unpair q.2).2).2).2 = true ∧
        (Nat.unpair (Nat.unpair (Nat.unpair q.2).2).2).1.testBit 0 = true :=
    (primrecPred_bool (Primrec₂.comp primrec_testBit
        (Primrec.fst.comp (Primrec.unpair.comp Primrec.snd))
        (Primrec.snd.comp (Primrec.unpair.comp
          (Primrec.snd.comp (Primrec.unpair.comp
            (Primrec.snd.comp (Primrec.unpair.comp Primrec.snd)))))))).and
      (primrecPred_bool (Primrec₂.comp primrec_testBit
        (Primrec.fst.comp (Primrec.unpair.comp
          (Primrec.snd.comp (Primrec.unpair.comp
            (Primrec.snd.comp (Primrec.unpair.comp Primrec.snd))))))
        (Primrec.const 0)))
  have hbit1 : PrimrecPred fun q : ℕ × ℕ =>
      (Nat.unpair (Nat.unpair q.2).2).1.testBit
          (Nat.unpair (Nat.unpair (Nat.unpair q.2).2).2).2 = true ∧
        2 ≤ (Nat.unpair (Nat.unpair (Nat.unpair q.2).2).2).1 :=
    (primrecPred_bool (Primrec₂.comp primrec_testBit
        (Primrec.fst.comp (Primrec.unpair.comp
          (Primrec.snd.comp (Primrec.unpair.comp Primrec.snd))))
        (Primrec.snd.comp (Primrec.unpair.comp
          (Primrec.snd.comp (Primrec.unpair.comp
            (Primrec.snd.comp (Primrec.unpair.comp Primrec.snd)))))))).and
      (Primrec.nat_le.comp (Primrec.const 2)
```

```
        (Primrec.fst.comp (Primrec.unpair.comp
          (Primrec.snd.comp (Primrec.unpair.comp
            (Primrec.snd.comp (Primrec.unpair.comp Primrec.snd)))))))
  have hinner : PrimrecPred fun q : ℕ × ℕ =>
      q.1 = pair (Nat.unpair q.2).1
          (pair (Nat.unpair (Nat.unpair q.2).2).1
            (pair (Nat.unpair (Nat.unpair (Nat.unpair q.2).2).2).1
              (Nat.unpair (Nat.unpair (Nat.unpair q.2).2).2).2)) ∧
        (((Nat.unpair q.2).1.testBit
              (Nat.unpair (Nat.unpair (Nat.unpair q.2).2).2).2 = true ∧
            (Nat.unpair (Nat.unpair (Nat.unpair q.2).2).2).1.testBit 0 =
              true) ∨
          ((Nat.unpair (Nat.unpair q.2).2).1.testBit
              (Nat.unpair (Nat.unpair (Nat.unpair q.2).2).2).2 = true ∧
            2 ≤ (Nat.unpair (Nat.unpair (Nat.unpair q.2).2).2).1)) :=
    heq.and (hbit0.or hbit1)
  have hex : REPred fun p =>
      ∃ t, p = pair (Nat.unpair t).1
          (pair (Nat.unpair (Nat.unpair t).2).1
            (pair (Nat.unpair (Nat.unpair (Nat.unpair t).2).2).1
              (Nat.unpair (Nat.unpair (Nat.unpair t).2).2).2)) ∧
        (((Nat.unpair t).1.testBit
              (Nat.unpair (Nat.unpair (Nat.unpair t).2).2).2 = true ∧
            (Nat.unpair (Nat.unpair (Nat.unpair t).2).2).1.testBit 0 =
              true) ∨
          ((Nat.unpair (Nat.unpair t).2).1.testBit
              (Nat.unpair (Nat.unpair (Nat.unpair t).2).2).2 = true ∧
            2 ≤ (Nat.unpair (Nat.unpair (Nat.unpair t).2).2).1)) :=
    REPred.exists hinner.computablePred.to_re
  constructor
  refine hex.of_eq fun p => ?_
  constructor
  · intro ⟨t, ht, hm⟩
    exact (mem_condC_iff p).mpr
      ⟨(Nat.unpair t).1, (Nat.unpair (Nat.unpair t).2).1,
        (Nat.unpair (Nat.unpair (Nat.unpair t).2).2).1,
        (Nat.unpair (Nat.unpair (Nat.unpair t).2).2).2, ht, hm⟩
  · intro hp
    obtain ⟨n0, n1, n2, m, ht, hm⟩ := (mem_condC_iff p).mp hp
    refine ⟨Nat.pair n0 (Nat.pair n1 (Nat.pair n2 m)), ?_, ?_⟩
    · simpa [Nat.unpair_pair] using ht
    · simpa [Nat.unpair_pair] using hm

end DomainTheory.DataTypesAsLattices
```

subC.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Lambda.interp
public import DomainTheory.DataTypesAsLattices.Lambda.Ycomb
public import DomainTheory.DataTypesAsLattices.Lambda.dollarC
public import DomainTheory.DataTypesAsLattices.Lambda.primRecStepTerm
public import DomainTheory.DataTypesAsLattices.Lambda.Kcomb
```

```
public import DomainTheory.DataTypesAsLattices.Lambda.Scomb
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Computability.subStepC
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Computability.pack4
public import DomainTheory.DataTypesAsLattices.Computability.Realizes2
public import DomainTheory.DataTypesAsLattices.Computability.primRecComb
public import DomainTheory.DataTypesAsLattices.Lambda.Kcomb_more
public import DomainTheory.DataTypesAsLattices.Lambda.Scomb_more
public import DomainTheory.DataTypesAsLattices.Lambda.Ycomb_more
public import DomainTheory.DataTypesAsLattices.Computability.primRecVal_sub

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable

open Nat.Partrec (Code)

open Nat.Partrec.Code

def subC : Pomega :=
  funOf (funOf Scomb (funOf Kcomb dollarC))
    (funOf (funOf Scomb (funOf Kcomb Ycomb))
      (funOf (funOf Scomb (interp primRecStepTerm (fun _ => botElem)))
        (funOf Kcomb subStepC)))

namespace subC

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem app (x : Pomega) : funOf subC x = primRecComb x subStepC := by
  simp [subC, primRecComb, Scomb.beta3, Kcomb.beta2]

theorem combinatory : IsCombinatory subC :=
  .app (.app .S (.app .K dollarC.combinatory))
    (.app (.app .S (.app .K Ycomb.combinatory))
      (.app (.app .S (theorem_2_4.closed primRecStepTerm))
        (.app .K subStepC.combinatory)))

theorem realizes2 : Realizes2 subC fun n m => n - m := fun n m => by
  simp [subC.app, primRecComb.ofNat, primRecVal_sub]

end subC

end DomainTheory.DataTypesAsLattices
```

topElem_combinatory.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Lambda.IsCombinatory
public import DomainTheory.DataTypesAsLattices.Lambda.interp
public import DomainTheory.DataTypesAsLattices.Lambda.Ycomb
public import DomainTheory.DataTypesAsLattices.Lambda.theorem_2_4
public import DomainTheory.DataTypesAsLattices.Lambda.theorem_2_5
public import DomainTheory.DataTypesAsLattices.Lambda.topRec
public import DomainTheory.DataTypesAsLattices.Lambda.eq_2_16
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Computability.pack4
public import DomainTheory.DataTypesAsLattices.Computability.topRecTerm
public import DomainTheory.DataTypesAsLattices.Lambda.Ycomb_more
@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem topElem_combinatory : IsCombinatory topElem := by
  have h : funOf Ycomb (interp topRecTerm (fun _ => botElem)) = topElem := by
    rw [topRecTerm.interp, theorem_2_5 topRec.isScottContinuous, eq_2_16]
  exact h ▸ IsCombinatory.app Ycomb.combinatory (theorem_2_4.closed topRecTerm)

end DomainTheory.DataTypesAsLattices
```

enumRangeC.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Lambda.dollarC
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Graph.Basic.notation3
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import Mathlib.Data.Set.Lattice.Indexed
public import DomainTheory.DataTypesAsLattices.Computability.Realizes
public import DomainTheory.DataTypesAsLattices.Computability.pack4
public import DomainTheory.DataTypesAsLattices.Lambda.eq_2_25
public import DomainTheory.DataTypesAsLattices.Computability.topElem_combinatory

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable
```

```
open Nat.Partrec (Code)

open Nat.Partrec.Code

def enumRangeC (r : Pomega) : Pomega :=
  (dollarC ? r) ? topElem

namespace enumRangeC

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem eq {r : Pomega} {p : ℕ → ℕ} (hr : Realizes r p) :
    enumRangeC r = ⋃ n, ofNat (p n) := by
  rw [enumRangeC, _root_.DomainTheory.DataTypesAsLattices.eq_2_25]
  ext k
  constructor
  · intro hk
    obtain ⟨n, _, hkn⟩ := Set.mem_iUnion₂.mp hk
    exact Set.mem_iUnion.mpr ⟨n, by simpa [hr n] using hkn⟩
  · intro hk
    obtain ⟨n, hkn⟩ := Set.mem_iUnion.mp hk
    exact Set.mem_iUnion₂.mpr ⟨n, Set.mem_univ n, by simpa [hr n] using hkn⟩

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem combinatory {r : Pomega} (hr : IsCombinatory r) :
    IsCombinatory (enumRangeC r) :=
  .app (.app dollarC.combinatory hr) topElem_combinatory

end enumRangeC

end DomainTheory.DataTypesAsLattices
```

bot_combinatory.lean

```
module

public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Lambda.IsCombinatory
public import DomainTheory.DataTypesAsLattices.Lambda.eq_2_6
public import DomainTheory.DataTypesAsLattices.Lambda.predC
public import DomainTheory.DataTypesAsLattices.Lambda.zeroC
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.predC_more

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices
```

```
theorem bot_combinatory : IsCombinatory botElem :=
  (show funOf predC zeroC = botElem by rw [predC.app]; exact eq_2_6) ▶
    IsCombinatory.app IsCombinatory.pred IsCombinatory.zero

end DomainTheory.DataTypesAsLattices
```

RetractInverseLimit.lean

```
module

public import DomainTheory.DataTypesAsLattices.FixedPoint.iterateBot
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.typed

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- The inverse limit of the ranges of `F?(⊥)`: points are typed at every
stage and adjacent coordinates satisfy Scott's projection equation. -/
def RetractInverseLimit (F : Pomega → Pomega) :=
  {v : ℕ → Pomega //
    (∀ n, typed (v n) (iterateBot F n)) ∧
      ∀ n, v n = funOf (iterateBot F n) (v (n + 1))}

end DomainTheory.DataTypesAsLattices
```

BasicInst55.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.RetractInverseLimit

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

instance (F : Pomega → Pomega) : PartialOrder (RetractInverseLimit F) where
  le v w := ∀ n, v.1 n ⊆ w.1 n
  le_refl _ _ := subset_rfl
  le_trans _ _ _ hvw hwz n := subset_trans (hvw n) (hwz n)
  le_antisymm v w hvw hwv := by
    apply Subtype.ext
    funext n
    exact Set.Subset.antisymm (hvw n) (hwv n)

end DomainTheory.DataTypesAsLattices
```

bot_retractLe.lean

```
module
```

```
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.comp
public import DomainTheory.DataTypesAsLattices.Retracts.retractLe
public import DomainTheory.DataTypesAsLattices.Retracts.funOf_botElem
public import DomainTheory.DataTypesAsLattices.Retracts.graph_const_bot
public import DomainTheory.DataTypesAsLattices.Retracts.IsStrict

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- The strict bottom retract is below every strict retract in Scott's
projection order. -/
theorem bot_retractLe {a : Pomega} (ha : IsStrict a) :
    retractLe botElem a := by
  constructor
  · unfold comp
    have h : (fun x => funOf botElem (funOf a x)) = fun _ => botElem := by
      funext x
      exact funOf_botElem (funOf a x)
    rw [h, graph_const_bot]
  · unfold comp
    have h : (fun x => funOf a (funOf botElem x)) = fun _ => botElem := by
      funext x
      rw [funOf_botElem, ha]
    rw [h, graph_const_bot]

end DomainTheory.DataTypesAsLattices
```

curryC.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.pairSeq

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (4.23).** Currying combinator. -/
def curryC : Pomega :=
  graph (fun u => graph (fun x => graph (fun y => funOf u (pairSeq x y))))

end DomainTheory.DataTypesAsLattices
```

diagC.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
```

```
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.pairSeq

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (4.16).** Diagonal combinator. -/
def diagC : Pomega := graph (fun u => pairSeq u u)

end DomainTheory.DataTypesAsLattices
```

directed_sUnion_fixed.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import DomainTheory.DataTypesAsLattices.FixedLattices.DirectedSet
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.mem_scottUnion
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Continuous.scottUnion
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.Fixpoints
public import DomainTheory.DataTypesAsLattices.Retracts.vals
public import DomainTheory.DataTypesAsLattices.Retracts.sUnion_Fixpoints_le_image
public import DomainTheory.DataTypesAsLattices.Retracts.directed_finite_cover

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem directed_sUnion_fixed {f : Pomega → Pomega} (hf : IsScottContinuous f)
    {A : Set (Fixpoints f)} (hA : DirectedSet A) :
    ⋃₀ (Fixpoints.vals A) ∈ Fixpoints f := by
  apply subset_antisymm
  · intro k hk
    have hk' : k ∈ scottUnion f (⋃₀ (Fixpoints.vals A)) := by
      rwa [← hf.out (⋃₀ (Fixpoints.vals A))]
    obtain ⟨n, hn, hkn⟩ := mem_scottUnion.mp hk'
    obtain ⟨z, hzA, hnz⟩ :=
      directed_finite_cover hA (e_finite n) hn
    have : k ∈ f z :=
      IsScottContinuous.monotone hf hnz hkn
    rw [z.property] at this
    exact Set.mem_sUnion.mpr ⟨z, ⟨z, hzA, rfl⟩, this⟩
  · exact sUnion_Fixpoints_le_image hf (Fixpoints.vals.subset A)

end DomainTheory.DataTypesAsLattices
```

sSup_fixed_eq_sUnion.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import DomainTheory.DataTypesAsLattices.FixedLattices.DirectedSet
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.Fixpoints
public import DomainTheory.DataTypesAsLattices.Retracts.vals
public import DomainTheory.DataTypesAsLattices.Retracts.fixpointsCompleteLattice
public import DomainTheory.DataTypesAsLattices.Retracts.directed_sUnion_fixed

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem sSup_fixed_eq_sUnion {f : Pomega → Pomega} (hf : IsScottContinuous f)
    {A : Set (Fixpoints f)} (hA : DirectedSet A) :
    ((@sSup (Fixpoints f) (fixpointsCompleteLattice hf).toSupSet A :
        Fixpoints f) : Pomega) =
      ⋃₀ (Fixpoints.vals A) := by
  let := fixpointsCompleteLattice hf
  let u : Fixpoints f :=
    ⟨⋃₀ (Fixpoints.vals A), directed_sUnion_fixed hf hA⟩
  apply subset_antisymm
  · have hle : sSup A ≤ u := by
      apply sSup_le
      intro x hx
      exact Set.subset_sUnion_of_mem ⟨x, hx, rfl⟩
    exact hle
  · intro k hk
    obtain ⟨x, hxA, hkx⟩ := Set.mem_sUnion.mp hk
    obtain ⟨y, hyA, rfl⟩ := hxA
    exact (le_sSup hyA) hkx

end DomainTheory.DataTypesAsLattices
```

retractBasis.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import DomainTheory.DataTypesAsLattices.FixedLattices.WayBelow
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.Fixpoints
public import DomainTheory.DataTypesAsLattices.Retracts.Retract
```

```
public import DomainTheory.DataTypesAsLattices.Retracts.retract_app
public import DomainTheory.DataTypesAsLattices.Retracts.retract_interpolation
public import DomainTheory.DataTypesAsLattices.Retracts.sSup_fixed_eq_sUnion
public import DomainTheory.DataTypesAsLattices.Retracts.directed_finite_cover
public import DomainTheory.DataTypesAsLattices.FixedLattices.DirectedSet
public import DomainTheory.DataTypesAsLattices.Retracts.DirectedSubset
public import DomainTheory.DataTypesAsLattices.Retracts.vals
public import DomainTheory.DataTypesAsLattices.Retracts.typed
public import DomainTheory.DataTypesAsLattices.Retracts.BasicInst36
public import DomainTheory.DataTypesAsLattices.Retracts.fixpointsCompleteLattice
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- The canonical finite-image basis of the fixed-point lattice of a retract. -/
def retractBasis (a : Pomega) (x : Fixpoints (funOf a)) :
    Set (Fixpoints (funOf a)) :=
  {y | ∃ n, e n ⊆ x ∧ (y : Pomega) = funOf a (e n)}

namespace retractBasis

theorem directed {a : Pomega} (ha : Retract a)
    (x : Fixpoints (funOf a)) :
    DirectedSet (retractBasis a x) := by
  constructor
  constructor
  · let y : Fixpoints (funOf a) :=
      ⟨funOf a (e 0), retract_app ha (e 0)⟩
    exact ⟨y, 0, by simp [e_zero], rfl⟩
  · intro y hy z hz
    obtain ⟨n, hnx, hyn⟩ := hy
    obtain ⟨m, hmx, hzm⟩ := hz
    let w : Fixpoints (funOf a) :=
      ⟨funOf a (e (n ||| m)), retract_app ha (e (n ||| m))⟩
    refine ⟨w, ⟨n ||| m, e_or_of_subset hnx hmx, rfl⟩, ?_, ?_⟩
    · change (y : Pomega) ⊆ (w : Pomega)
      rw [hyn]
      change funOf a (e n) ⊆ funOf a (e (n ||| m))
      rw [e_or]
      exact IsScottContinuous.monotone (funOf.isScottContinuous a)
        Set.subset_union_left
    · change (z : Pomega) ⊆ (w : Pomega)
      rw [hzm]
      change funOf a (e m) ⊆ funOf a (e (n ||| m))
      rw [e_or]
      exact IsScottContinuous.monotone (funOf.isScottContinuous a)
        Set.subset_union_right

theorem directedSubset {a : Pomega} (ha : Retract a)
    (x : Fixpoints (funOf a)) :
    DirectedSubset (retractBasis a x) := by
  have h := directed ha x
  refine ⟨⟨h.out.1, ?_⟩⟩
  intro y hy z hz
  obtain ⟨w, hw, hyw, hzw⟩ := h.out.2 hy hz
```

```
  exact ⟨w, hw, hyw, hzw⟩

theorem eq_sSup {a : Pomega} (ha : Retract a)
    (x : Fixpoints (funOf a)) :
    x = sSup (retractBasis a x) := by
  apply Subtype.ext
  rw [sSup_fixed_eq_sUnion (funOf.isScottContinuous a)
    (directed ha x)]
  have hx : typed (x : Pomega) a := x.property.symm
  rw [retract_interpolation ha hx]
  ext k
  constructor
  · intro hk
    obtain ⟨n, hnx, hkn⟩ := Set.mem_iUnion.mp hk
    let y : Fixpoints (funOf a) :=
      ⟨funOf a (e n), retract_app ha (e n)⟩
    exact Set.mem_sUnion.mpr
      ⟨y, ⟨y, ⟨n, hnx, rfl⟩, rfl⟩, hkn⟩
  · intro hk
    obtain ⟨_, ⟨y, ⟨n, hnx, hyn⟩, rfl⟩, hky⟩ :=
      Set.mem_sUnion.mp hk
    exact Set.mem_iUnion.mpr ⟨n, hnx, hyn ▸ hky⟩

theorem eq_sUnion {a : Pomega} (ha : Retract a)
    (x : Fixpoints (funOf a)) :
    (x : Pomega) = ⋃₀ (Subtype.val '' retractBasis a x) := by
  have h := eq_sSup ha x
  have hsu := sSup_fixed_eq_sUnion (funOf.isScottContinuous a)
    (directed ha x)
  have : (x : Pomega) = (sSup (retractBasis a x) : Fixpoints (funOf a)) :=
    congrArg Subtype.val h
  exact this.trans hsu

theorem wayBelow {a : Pomega} (_ha : Retract a)
    {x y : Fixpoints (funOf a)} (hy : y ∈ retractBasis a x) :
    WayBelow y x := by
  intro A hA hxA
  obtain ⟨n, hnx, hyn⟩ := hy
  have hsup :
      ((sSup A : Fixpoints (funOf a)) : Pomega) =
        ⋃₀ (Fixpoints.vals A) :=
    sSup_fixed_eq_sUnion (funOf.isScottContinuous a) hA
  have hfinite :
      e n ⊆ ⋃₀ (Fixpoints.vals A) := by
    rw [← hsup]
    exact hnx.trans hxA
  obtain ⟨z, hzA, hnz⟩ :=
    directed_finite_cover hA (e_finite n) hfinite
  refine ⟨z, hzA, ?_⟩
  change (y : Pomega) ⊆ z
  rw [hyn]
  have := IsScottContinuous.monotone (funOf.isScottContinuous a) hnz
  rwa [z.property] at this

end retractBasis
```

```
end DomainTheory.DataTypesAsLattices
```

evalC.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (4.22).** Evaluation combinator. -/
def evalC : Pomega :=
  graph (fun u => funOf (funOf u (ofNat 0)) (funOf u (ofNat 1)))

end DomainTheory.DataTypesAsLattices
```

inverseLimit_mono.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.FixedPoint.iterateBot
public import DomainTheory.DataTypesAsLattices.FixedPoint.iterateBot_mono
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.RetractInverseLimit
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- A coherent inverse-limit sequence is increasing in the ambient
powerset order. -/
theorem inverseLimit_mono {F : Pomega → Pomega}
    (hf : IsScottContinuous F) (v : RetractInverseLimit F) :
    ∀ n, v.1 n ⊆ v.1 (n + 1) := by
  intro n
  rw [v.property.2 n]
  have hcode : iterateBot F n ⊆ iterateBot F (n + 1) :=
    iterateBot_mono hf n
  have h := funOf.monotone_left hcode (v.1 (n + 1))
  rw [← v.property.1 (n + 1)] at h
  exact h
```

```
end DomainTheory.DataTypesAsLattices
```

inverseLimit_chain.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.FixedPoint.chain_mono
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.RetractInverseLimit
public import DomainTheory.DataTypesAsLattices.Retracts.inverseLimit_mono

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem inverseLimit_chain {F : Pomega → Pomega}
    (hf : IsScottContinuous F) (v : RetractInverseLimit F) :
    ∀ n m, n ≤ m → v.1 n ⊆ v.1 m :=
  chain_mono v.1 (inverseLimit_mono hf v)

end DomainTheory.DataTypesAsLattices
```

IsREu2egraph_of_primrecRel.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Computability.IsRE
public import DomainTheory.DataTypesAsLattices.Computability.primrecPred_eq_scott_pair
public import DomainTheory.DataTypesAsLattices.Retracts.comp
public import DomainTheory.DataTypesAsLattices.Computability.REPredu2eexists

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

/-- Encode a Scott-pair graph clause as a single `Nat.pair` search index. -/
theorem IsRE.graph_of_primrecRel {f : Pomega → Pomega}
    (hP : PrimrecRel fun n m : ℕ => m ∈ f (e n)) :
    IsRE (graph f) := by
```

```
  have hinner : PrimrecPred fun q : ℕ × ℕ =>
      q.1 = pair (Nat.unpair q.2).1 (Nat.unpair q.2).2 ∧
        (Nat.unpair q.2).2 ∈ f (e (Nat.unpair q.2).1) :=
    (primrecPred_eq_scott_pair.comp
      (Primrec.pair Primrec.fst (Primrec.unpair.comp Primrec.snd))).and
      (hP.comp
        (Primrec.fst.comp (Primrec.unpair.comp Primrec.snd))
        (Primrec.snd.comp (Primrec.unpair.comp Primrec.snd)))
  have hex : REPred fun p =>
      ∃ t, p = pair (Nat.unpair t).1 (Nat.unpair t).2 ∧
        (Nat.unpair t).2 ∈ f (e (Nat.unpair t).1) :=
    REPred.exists (p := fun p t =>
        p = pair (Nat.unpair t).1 (Nat.unpair t).2 ∧
          (Nat.unpair t).2 ∈ f (e (Nat.unpair t).1))
      hinner.computablePred.to_re
  constructor
  refine hex.of_eq fun p => ?_
  constructor
  · intro ⟨t, ht, hm⟩
    exact ⟨(Nat.unpair t).1, (Nat.unpair t).2, ht, hm⟩
  · intro ⟨n, m, hm, hmem⟩
    refine ⟨Nat.pair n m, ?_, ?_⟩
    · simpa [Nat.unpair_pair] using hm
    · simpa [Nat.unpair_pair] using hmem

end DomainTheory.DataTypesAsLattices
```

IsREu2epredC.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Lambda.predSet
public import DomainTheory.DataTypesAsLattices.Lambda.predC
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Computability.IsRE
public import DomainTheory.DataTypesAsLattices.Computability.primrec_testBit
public import DomainTheory.DataTypesAsLattices.Computability.primrecPred_bool
public import DomainTheory.DataTypesAsLattices.Retracts.comp
public import DomainTheory.DataTypesAsLattices.Computability.
    IsREu2egraph_of_primrecRel

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem IsRE.predC : IsRE predC := by
  have hp : PrimrecRel fun n m : ℕ => n.testBit (m + 1) = true :=
    primrecPred_bool (Primrec₂.comp primrec_testBit Primrec.fst (Primrec.succ.comp
```

```
    Primrec.snd))
  constructor
  refine (IsRE.graph_of_primrecRel (f := DataTypesAsLattices.predSet)
      (hp.of_eq fun n m => ?_)).out.of_eq ?_
  · constructor
    · intro h; exact mem_e.mpr h
    · intro h; exact mem_e.mp h
  · intro p; rfl

end DomainTheory.DataTypesAsLattices
```

IsREu2esucC.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Lambda.succSet
public import DomainTheory.DataTypesAsLattices.Lambda.sucC
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Computability.IsRE
public import DomainTheory.DataTypesAsLattices.Computability.primrec_testBit
public import DomainTheory.DataTypesAsLattices.Computability.primrecPred_bool
public import DomainTheory.DataTypesAsLattices.Retracts.comp
public import DomainTheory.DataTypesAsLattices.Computability.
    IsREu2egraph_of_primrecRel

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem IsRE.sucC : IsRE sucC := by
  have hp : PrimrecRel fun n m : ℕ => m = 0 ∧ n.testBit m.pred = true :=
    (PrimrecPred.not (Primrec.eq.comp Primrec.snd (Primrec.const 0))).and
      (primrecPred_bool
        (Primrec₂.comp primrec_testBit Primrec.fst (Primrec.pred.comp Primrec.snd)))
  constructor
  refine (IsRE.graph_of_primrecRel (f := DataTypesAsLattices.succSet)
      (hp.of_eq fun n m => ?_)).out.of_eq ?_
  · constructor
    · intro ⟨hm0, hbit⟩
      exact ⟨m.pred, mem_e.mpr hbit, (Nat.succ_pred_eq_of_ne_zero hm0).symm⟩
    · intro ⟨k, hk, hm⟩
      subst hm
      exact ⟨Nat.succ_ne_zero k, mem_e.mp hk⟩
  · intro p; rfl

end DomainTheory.DataTypesAsLattices
```

IsREu2enumeral.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Computability.IsRE
public import DomainTheory.DataTypesAsLattices.Retracts.comp

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

/-- Named `numeral` so it does not shadow `ofNat`. -/
theorem IsRE.numeral (k : ℕ) : IsRE (ofNat k) := by
  have : ComputablePred fun n : ℕ => n = k :=
    ⟨inferInstance, (Primrec.beq.comp Primrec.id (Primrec.const k)).to_comp⟩
  constructor
  exact REPred.of_eq this.to_re fun n => by simp [ofNat]

end DomainTheory.DataTypesAsLattices
```

IsREu2ezeroC.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Lambda.zeroC
public import DomainTheory.DataTypesAsLattices.Computability.IsRE
public import DomainTheory.DataTypesAsLattices.Computability.IsREu2enumeral

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

theorem IsRE.zeroC : IsRE zeroC :=
  IsRE.numeral 0

end DomainTheory.DataTypesAsLattices
```

composeC.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Lambda.Kcomb
public import DomainTheory.DataTypesAsLattices.Lambda.Scomb
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Computability.Realizes
public import DomainTheory.DataTypesAsLattices.Computability.pack4
public import DomainTheory.DataTypesAsLattices.Lambda.IsCombinatory
public import DomainTheory.DataTypesAsLattices.Lambda.Kcomb_more
```

```
public import DomainTheory.DataTypesAsLattices.Lambda.Scomb_more

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable

open Nat.Partrec (Code)

open Nat.Partrec.Code

def composeC (u v : Pomega) : Pomega :=
  funOf (funOf Scomb (funOf Kcomb u)) v

namespace composeC

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem combinatory {u v : Pomega}
    (hu : IsCombinatory u) (hv : IsCombinatory v) :
    IsCombinatory (composeC u v) :=
  .app (.app .S (.app .K hu)) hv

theorem realizes {u v : Pomega} {p q : ℕ → ℕ}
    (hu : Realizes u p) (hv : Realizes v q) :
    Realizes (composeC u v) (p ∘ q) := fun n => by
  rw [composeC, Scomb.beta3, Kcomb.beta2, hv n, hu (q n)]
  rfl

end composeC

end DomainTheory.DataTypesAsLattices
```

constComb.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.Kcomb
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Computability.Realizes
public import DomainTheory.DataTypesAsLattices.Computability.pack4
public import DomainTheory.DataTypesAsLattices.Lambda.Kcomb_more
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat_more

@[expose] public section

set_option maxHeartbeats 2000000
```

```
namespace DomainTheory.DataTypesAsLattices

open Encodable

open Nat.Partrec (Code)

open Nat.Partrec.Code

def constComb (k : ℕ) : Pomega := funOf Kcomb (ofNat k)

namespace constComb

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem combinatory (k : ℕ) : IsCombinatory (constComb k) :=
  .app .K (ofNat.combinatory k)

theorem realizes (k : ℕ) : Realizes (constComb k) fun _ => k := fun _ => by
  simp [constComb, Kcomb.beta2]

end constComb

end DomainTheory.DataTypesAsLattices
```

isZeroC.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.condC
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Computability.pack4
public import DomainTheory.DataTypesAsLattices.Lambda.eq_2_7_succ
public import DomainTheory.DataTypesAsLattices.Lambda.eq_2_7_zero
public import DomainTheory.DataTypesAsLattices.Lambda.condC_more
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat_more
@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable

open Nat.Partrec (Code)

open Nat.Partrec.Code

/-- `cond 1 0 n` is `1` on `0` and `0` on positive numerals. -/
def isZeroC : Pomega := funOf (funOf condC (ofNat 1)) (ofNat 0)
```

```
namespace isZeroC

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem combinatory : IsCombinatory isZeroC :=
  .app (.app .cond (ofNat.combinatory 1)) (ofNat.combinatory 0)

theorem succ (n : ℕ) : funOf isZeroC (ofNat (n + 1)) = ofNat 0 := by
  simp only [isZeroC]
  rw [condC.beta3]
  exact eq_2_7_succ (ofNat 1) (ofNat 0) n

theorem zero : funOf isZeroC (ofNat 0) = ofNat 1 := by
  simp only [isZeroC]
  rw [condC.beta3, eq_2_7_zero]

end isZeroC

end DomainTheory.DataTypesAsLattices
```

leC.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Lambda.Kcomb
public import DomainTheory.DataTypesAsLattices.Lambda.Scomb
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Computability.subC
public import DomainTheory.DataTypesAsLattices.Computability.isZeroC

public import DomainTheory.DataTypesAsLattices.Graph.Basic.notation3
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Computability.pack4
public import DomainTheory.DataTypesAsLattices.Lambda.Kcomb_more
public import DomainTheory.DataTypesAsLattices.Lambda.Scomb_more
@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable

open Nat.Partrec (Code)

open Nat.Partrec.Code

/-- `λn λm. isZero (n ? m)`. -/
def leC : Pomega :=
  Scomb ? (Kcomb ? (Scomb ? (Kcomb ? isZeroC))) ? subC
```

```
namespace leC

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem combinatory : IsCombinatory leC :=
  .app (.app .S (.app .K (.app .S (.app .K isZeroC.combinatory))))
    subC.combinatory

theorem realizes2 (n m : ℕ) :
    funOf (funOf leC (ofNat n)) (ofNat m) =
      ofNat (if n ≤ m then 1 else 0) := by
  unfold leC
  rw [Scomb.beta3, Kcomb.beta2, Scomb.beta3, Kcomb.beta2, subC.realizes2]
  by_cases h : n ≤ m
  · have : n - m = 0 := Nat.sub_eq_zero_of_le h
    simp [h, this, isZeroC.zero]
  · have : 0 < n - m := Nat.sub_pos_of_lt (Nat.not_le.mp h)
    obtain ⟨k, hk⟩ := Nat.exists_eq_succ_of_ne_zero this.ne'
    simp [h, hk, isZeroC.succ]

end leC

end DomainTheory.DataTypesAsLattices
```

sqrtEnv.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Computability.mulC
public import DomainTheory.DataTypesAsLattices.Computability.leC
public import DomainTheory.DataTypesAsLattices.Computability.pack4
public import DomainTheory.DataTypesAsLattices.Computability.botElem_combinatory

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable

open Nat.Partrec (Code)

open Nat.Partrec.Code

def sqrtEnv : ℕ → Pomega
  | 3 => leC
  | 4 => mulC
  | _ => botElem
```

```
namespace sqrtEnv

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem combinatory : ∀ i, IsCombinatory (sqrtEnv i)
  | 3 => leC.combinatory
  | 4 => mulC.combinatory
  | 0 | 1 | 2 => botElem_combinatory
  | _ + 5 => botElem_combinatory

end sqrtEnv

end DomainTheory.DataTypesAsLattices
```

sqrtStepC.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Lambda.interp
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Computability.sqrtEnv
public import DomainTheory.DataTypesAsLattices.Computability.sqrtStepTerm
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Graph.Basic.notation3
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Computability.pack4
public import DomainTheory.DataTypesAsLattices.Computability.mulC
public import DomainTheory.DataTypesAsLattices.Computability.leC
public import DomainTheory.DataTypesAsLattices.Lambda.envSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofComb
public import DomainTheory.DataTypesAsLattices.Lambda.succSet
public import DomainTheory.DataTypesAsLattices.Lambda.succSet_more

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable

open Nat.Partrec (Code)

open Nat.Partrec.Code

def sqrtStepC : Pomega := interp sqrtStepTerm sqrtEnv

namespace sqrtStepC

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code
```

```
theorem app (n i a : ℕ) :
    ((sqrtStepC ? ofNat n) ? ofNat i) ? ofNat a =
      ofNat (if (a + 1) * (a + 1) ≤ n then a + 1 else a) := by
  unfold sqrtStepC sqrtStepTerm
  rw [theorem_2_2.beta, theorem_2_2.beta, theorem_2_2.beta]
  simp only [interp, envSet, Function.update_self,
    Function.update_of_ne (by decide : (3 : ℕ) = 0),
    Function.update_of_ne (by decide : (4 : ℕ) = 0),
    Function.update_of_ne (by decide : (3 : ℕ) = 1),
    Function.update_of_ne (by decide : (4 : ℕ) = 1),
    Function.update_of_ne (by decide : (3 : ℕ) = 2),
    Function.update_of_ne (by decide : (4 : ℕ) = 2), sqrtEnv]
  simp [Function.update]
  rw [succSet.ofNat, mulC.realizes2, leC.realizes2]
  by_cases h : (a + 1) * (a + 1) ≤ n
  · simp [h, eq_2_7_succ]
  · simp [h, eq_2_7_zero]

theorem combinatory : IsCombinatory sqrtStepC := by
  rw [sqrtStepC, theorem_2_4.complete]
  exact ofComb.combinatory _ _ sqrtEnv.combinatory

end sqrtStepC

end DomainTheory.DataTypesAsLattices
```

primRecVal_sqrt.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Lambda.primRecVal
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Computability.pack4
public import DomainTheory.DataTypesAsLattices.Computability.sqrtIter
public import DomainTheory.DataTypesAsLattices.Computability.sqrtStepC
public import DomainTheory.DataTypesAsLattices.Graph.Basic.notation3
@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem primRecVal_sqrt (n : ℕ) : ∀ k,
    primRecVal (ofNat 0) (sqrtStepC ? ofNat n) k = ofNat (sqrtIter n k)
  | 0 => rfl
  | k + 1 => by
      rw [show primRecVal (ofNat 0) (sqrtStepC ? ofNat n) (k + 1) =
        ((sqrtStepC ? ofNat n) ? ofNat k) ?
          primRecVal (ofNat 0) (sqrtStepC ? ofNat n) k by rfl,
```

```
        primRecVal_sqrt n k, sqrtStepC.app]
      rfl

end DomainTheory.DataTypesAsLattices
```

primrecPred_common_stage.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Computability.primrec_testBit
public import DomainTheory.DataTypesAsLattices.Computability.primrecPred_bool
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.comp

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

/-- Common-stage certificate: a finite family of r.e. facts all halt at one `evaln`
    bound. -/
theorem primrecPred_common_stage (c : Code) :
    PrimrecPred fun p : ℕ × ℕ =>
      ∀ k < p.1, p.1.testBit k = false ∨ (evaln p.2 c k).isSome = true := by
  have heval : Primrec fun q : ℕ × ℕ => evaln q.2 c q.1 :=
    primrec_evaln.comp <|
      Primrec.pair (Primrec.pair Primrec.snd (Primrec.const c)) Primrec.fst
  have hR : PrimrecRel fun k q : ℕ =>
      q.unpair.1.testBit k = false ∨ (evaln q.unpair.2 c k).isSome = true :=
    ((primrecPred_bool
        (Primrec₂.comp primrec_testBit
          (Primrec.fst.comp (Primrec.unpair.comp Primrec.snd)) Primrec.fst)).not.or
      (primrecPred_bool (Primrec.option_isSome.comp
        (primrec_evaln.comp <|
          Primrec.pair
            (Primrec.pair
              (Primrec.snd.comp (Primrec.unpair.comp Primrec.snd))
              (Primrec.const c))
            Primrec.fst)))).of_eq fun p => by
      simp [Bool.not_eq_true]
  refine (hR.forall_lt.comp Primrec.fst
      (Primrec₂.natPair.comp Primrec.fst Primrec.snd)).of_eq fun p => ?_
  simp [Nat.unpair_pair]

end DomainTheory.DataTypesAsLattices
```

IsREu2eeSubset.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Computability.IsRE
public import DomainTheory.DataTypesAsLattices.Computability.primrecPred_common_stage
public import DomainTheory.DataTypesAsLattices.Computability.eSubset_iff_stage
public import DomainTheory.DataTypesAsLattices.Computability.REPredu2eexists

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

/-- Finite inclusion `e n ⊆ x` is r.e. in `n` when `x` is, via one common `evaln` stage
    . -/
theorem IsRE.eSubset {x : Pomega} (hx : IsRE x) :
    REPred fun n => e n ⊆ x := by
  have hf : Partrec fun k : ℕ =>
      (Part.assert (k ∈ x) fun _ => Part.some (0 : ℕ)) :=
    hx.out.map (Computable.const (0 : ℕ)).to₂
  obtain ⟨c, hc⟩ := Code.exists_code.1 (Partrec.nat_iff.1 hf)
  have hex : REPred fun n =>
      ∃ s, ∀ k < n, n.testBit k = false ∨ (evaln s c k).isSome = true :=
    REPred.exists (p := fun n s =>
        ∀ k < n, n.testBit k = false ∨ (evaln s c k).isSome = true)
      (primrecPred_common_stage c).computablePred.to_re
  exact hex.of_eq fun n => (eSubset_iff_stage hc n).symm

end DomainTheory.DataTypesAsLattices
```

IsREu2efunOf.lean

```
module
public import DomainTheory.DataTypesAsLattices.Computability.REPredu2eempty
public import DomainTheory.DataTypesAsLattices.Computability.REPredu2etrue
public import DomainTheory.DataTypesAsLattices.Computability.REPredu2ecomp
public import DomainTheory.DataTypesAsLattices.Computability.REPredu2eor

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Computability.IsRE

public import DomainTheory.DataTypesAsLattices.Graph.Basic.notation3
public import DomainTheory.DataTypesAsLattices.Retracts.comp
public import DomainTheory.DataTypesAsLattices.Computability.REPredu2eand
public import DomainTheory.DataTypesAsLattices.Computability.REPredu2eexists
public import DomainTheory.DataTypesAsLattices.Computability.IsREu2epair_mem
```

```
public import DomainTheory.DataTypesAsLattices.Computability.IsREu2eeSubset
@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

/-- **Scott 1976, Appendix.** Application of r.e. elements is r.e. -/
theorem IsRE.funOf {u x : Pomega} (hu : IsRE u) (hx : IsRE x) : IsRE (u ? x) := by
  have hand : REPred fun q : ℕ × ℕ => e q.1 ⊆ x ∧ pair q.1 q.2 ∈ u :=
    REPred.and ((IsRE.eSubset hx).comp Computable.fst) (IsRE.pair_mem hu)
  have hex : REPred fun m => ∃ n, e n ⊆ x ∧ pair n m ∈ u :=
    REPred.exists (p := fun m n => e n ⊆ x ∧ pair n m ∈ u)
      (hand.comp (Primrec.pair Primrec.snd Primrec.fst).to_comp)
  constructor
  exact hex.of_eq fun _ => Iff.rfl

end DomainTheory.DataTypesAsLattices
```

sqrtC.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Lambda.Kcomb
public import DomainTheory.DataTypesAsLattices.Lambda.Scomb
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Computability.primRecZeroC
public import DomainTheory.DataTypesAsLattices.Computability.sqrtStepC

public import DomainTheory.DataTypesAsLattices.Graph.Basic.notation3
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Computability.Realizes
public import DomainTheory.DataTypesAsLattices.Computability.pack4
public import DomainTheory.DataTypesAsLattices.Computability.sqrtIter
public import DomainTheory.DataTypesAsLattices.Computability.primRecComb
public import DomainTheory.DataTypesAsLattices.Lambda.Kcomb_more
public import DomainTheory.DataTypesAsLattices.Lambda.Scomb_more
public import DomainTheory.DataTypesAsLattices.Computability.primRecVal_sqrt
@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable

open Nat.Partrec (Code)

open Nat.Partrec.Code
```

```
/-- A combinator realizing integer square root. -/
def sqrtC : Pomega :=
  Scomb ?
    (Scomb ? (Kcomb ? primRecZeroC) ? sqrtStepC) ?
    (Scomb ? Kcomb ? Kcomb)

namespace sqrtC

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem combinatory : IsCombinatory sqrtC :=
  .app
    (.app .S
      (.app (.app .S (.app .K primRecZeroC.combinatory))
        sqrtStepC.combinatory))
    (.app (.app .S .K) .K)

theorem realizes : Realizes sqrtC Nat.sqrt := fun n => by
  rw [sqrtC, Scomb.beta3, Scomb.beta3, Scomb.beta3, Kcomb.beta2, Kcomb.beta2,
    primRecZeroC.app, primRecComb.ofNat,
    primRecVal_sqrt, sqrtIter.self]

end sqrtC

end DomainTheory.DataTypesAsLattices
```

unpairEnv.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Computability.subC
public import DomainTheory.DataTypesAsLattices.Computability.mulC
public import DomainTheory.DataTypesAsLattices.Computability.leC
public import DomainTheory.DataTypesAsLattices.Computability.sqrtC
public import DomainTheory.DataTypesAsLattices.Computability.pack4

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable

open Nat.Partrec (Code)

open Nat.Partrec.Code

def unpairEnv : ℕ → Pomega
  | 1 => sqrtC
  | 2 => subC
```

```
  | 3 => mulC
  | 4 => leC
  | _ => botElem

namespace unpairEnv

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem combinatory : ∀ i, IsCombinatory (unpairEnv i)
  | 1 => sqrtC.combinatory
  | 2 => subC.combinatory
  | 3 => mulC.combinatory
  | 4 => leC.combinatory
  | 0 => botElem_combinatory
  | _ + 5 => botElem_combinatory

end unpairEnv

end DomainTheory.DataTypesAsLattices
```

unpairFstC.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Lambda.interp
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Computability.unpairEnv
public import DomainTheory.DataTypesAsLattices.Computability.unpairFstTerm
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Computability.sqrtC
public import DomainTheory.DataTypesAsLattices.Computability.Realizes
public import DomainTheory.DataTypesAsLattices.Computability.unpairDeltaTerm
public import DomainTheory.DataTypesAsLattices.Computability.subC
public import DomainTheory.DataTypesAsLattices.Computability.pack4
public import DomainTheory.DataTypesAsLattices.Computability.unpairTestTerm
public import DomainTheory.DataTypesAsLattices.Computability.mulC
public import DomainTheory.DataTypesAsLattices.Computability.leC
public import DomainTheory.DataTypesAsLattices.Computability.unpairSqrtTerm
public import DomainTheory.DataTypesAsLattices.Computability.unpairSquareTerm
public import DomainTheory.DataTypesAsLattices.Lambda.envSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofComb
public import DomainTheory.DataTypesAsLattices.Lambda.succSet
public import DomainTheory.DataTypesAsLattices.Lambda.succSet_more

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices
```

```
open Encodable

open Nat.Partrec (Code)

open Nat.Partrec.Code

def unpairFstC : Pomega := interp unpairFstTerm unpairEnv

namespace unpairFstC

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem combinatory : IsCombinatory unpairFstC := by
  rw [unpairFstC, theorem_2_4.complete]
  exact ofComb.combinatory _ _ unpairEnv.combinatory

theorem realizes : Realizes unpairFstC (fun n => n.unpair.1) := fun n => by
  rw [unpairFstC, unpairFstTerm, theorem_2_2.beta]
  simp only [unpairTestTerm, unpairDeltaTerm, unpairSquareTerm, unpairSqrtTerm,
    interp, envSet, Function.update_self,
    Function.update_of_ne (by decide : (1 : ℕ) = 0),
    Function.update_of_ne (by decide : (2 : ℕ) = 0),
    Function.update_of_ne (by decide : (3 : ℕ) = 0),
    Function.update_of_ne (by decide : (4 : ℕ) = 0), unpairEnv]
  rw [sqrtC.realizes, mulC.realizes2, subC.realizes2, succSet.ofNat,
    leC.realizes2]
  unfold Nat.unpair
  by_cases h : n - Nat.sqrt n * Nat.sqrt n < Nat.sqrt n
  · have hle : n - Nat.sqrt n * Nat.sqrt n + 1 ≤ Nat.sqrt n :=
      Nat.succ_le_iff.mpr h
    simp [h, hle, eq_2_7_succ]
  · have hle : ¬n - Nat.sqrt n * Nat.sqrt n + 1 ≤ Nat.sqrt n := by omega
    simp [h, hle, eq_2_7_zero]

end unpairFstC

end DomainTheory.DataTypesAsLattices
```

unpairSndC.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Lambda.interp
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Computability.unpairEnv
public import DomainTheory.DataTypesAsLattices.Computability.unpairSndTerm
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Computability.sqrtC
public import DomainTheory.DataTypesAsLattices.Computability.Realizes
public import DomainTheory.DataTypesAsLattices.Computability.unpairDeltaTerm
```

```
public import DomainTheory.DataTypesAsLattices.Computability.subC
public import DomainTheory.DataTypesAsLattices.Computability.pack4
public import DomainTheory.DataTypesAsLattices.Computability.unpairTestTerm
public import DomainTheory.DataTypesAsLattices.Computability.mulC
public import DomainTheory.DataTypesAsLattices.Computability.leC
public import DomainTheory.DataTypesAsLattices.Computability.unpairSqrtTerm
public import DomainTheory.DataTypesAsLattices.Computability.unpairSquareTerm
public import DomainTheory.DataTypesAsLattices.Lambda.envSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofComb
public import DomainTheory.DataTypesAsLattices.Lambda.succSet
public import DomainTheory.DataTypesAsLattices.Lambda.succSet_more

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable

open Nat.Partrec (Code)

open Nat.Partrec.Code

def unpairSndC : Pomega := interp unpairSndTerm unpairEnv

namespace unpairSndC

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem combinatory : IsCombinatory unpairSndC := by
  rw [unpairSndC, theorem_2_4.complete]
  exact ofComb.combinatory _ _ unpairEnv.combinatory

theorem realizes : Realizes unpairSndC (fun n => n.unpair.2) := fun n => by
  rw [unpairSndC, unpairSndTerm, theorem_2_2.beta]
  simp only [unpairTestTerm, unpairDeltaTerm, unpairSquareTerm, unpairSqrtTerm,
    interp, envSet, Function.update_self,
    Function.update_of_ne (by decide : (1 : ℕ) = 0),
    Function.update_of_ne (by decide : (2 : ℕ) = 0),
    Function.update_of_ne (by decide : (3 : ℕ) = 0),
    Function.update_of_ne (by decide : (4 : ℕ) = 0), unpairEnv]
  rw [sqrtC.realizes, mulC.realizes2, subC.realizes2, succSet.ofNat,
    leC.realizes2, subC.realizes2]
  unfold Nat.unpair
  by_cases h : n - Nat.sqrt n * Nat.sqrt n < Nat.sqrt n
  · have hle : n - Nat.sqrt n * Nat.sqrt n + 1 ≤ Nat.sqrt n :=
      Nat.succ_le_iff.mpr h
    simp [h, hle, eq_2_7_succ]
  · have hle : ¬n - Nat.sqrt n * Nat.sqrt n + 1 ≤ Nat.sqrt n := by omega
    simp [h, hle, eq_2_7_zero]

end unpairSndC
```

```
end DomainTheory.DataTypesAsLattices
```

RE.lean

```
module

public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Lambda.IsCombinatory
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

/-- The r.e. / LAMBDA-definable closed elements. -/
def RE : Set Pomega := {u | IsCombinatory u}

end DomainTheory.DataTypesAsLattices
```

mem_funOf_e_iff.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Computability.bits_subset_lt_two_pow

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem mem_funOf_e_iff (a b m : ℕ) :
    m ∈ funOf (e a) (e b) ↔
      ∃ n < 2 ^ (b + 1), e n ⊆ e b ∧ a.testBit (pair n m) := by
  constructor
  · intro ⟨n, hn, hp⟩
    exact ⟨n, bits_subset_lt_two_pow hn, hn, mem_e.mp hp⟩
  · intro ⟨n, _, hn, hbit⟩
    exact ⟨n, hn, mem_e.mpr hbit⟩

end DomainTheory.DataTypesAsLattices
```

sucC_zero.lean

```
module

public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Lambda.sucC
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.zeroC
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Lambda.sucC_more

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, Theorem 3.1 (The generator theorem).**
All LAMBDA-definable elements are obtained from `G` by iterated application. -/
theorem sucC_zero : funOf sucC zeroC = ofNat 1 := by
  rw [sucC.app]
  ext k
  constructor
  · intro ⟨n, hn, hk⟩
    simp [zeroC, ofNat] at hn
    subst hn
    simpa [ofNat] using hk
  · intro hk
    simp [ofNat] at hk
    exact ⟨0, rfl, hk⟩

end DomainTheory.DataTypesAsLattices
```

arithEnv.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Computability.addC
public import DomainTheory.DataTypesAsLattices.Computability.subC
public import DomainTheory.DataTypesAsLattices.Computability.mulC
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Computability.pack4
public import DomainTheory.DataTypesAsLattices.Lambda.zeroC
public import DomainTheory.DataTypesAsLattices.Lambda.predC
public import DomainTheory.DataTypesAsLattices.Lambda.predC_more
public import DomainTheory.DataTypesAsLattices.Lambda.eq_2_6

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable
```

```
open Nat.Partrec (Code)

open Nat.Partrec.Code

def arithEnv : ℕ → Pomega
  | 0 => addC
  | 1 => mulC
  | 2 => subC
  | _ => botElem

namespace arithEnv

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem combinatory : ∀ i, IsCombinatory (arithEnv i)
  | 0 => addC.combinatory
  | 1 => mulC.combinatory
  | 2 => subC.combinatory
  | n + 3 => by
    have hbot : funOf predC zeroC = botElem := by
      rw [predC.app]; exact _root_.DomainTheory.DataTypesAsLattices.eq_2_6
    simpa [arithEnv, hbot] using
      IsCombinatory.app IsCombinatory.pred IsCombinatory.zero

end arithEnv

end DomainTheory.DataTypesAsLattices
```

pairComb.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Lambda.interp
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Computability.pairTerm
public import DomainTheory.DataTypesAsLattices.Computability.arithEnv
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Computability.addC
public import DomainTheory.DataTypesAsLattices.Computability.subC
public import DomainTheory.DataTypesAsLattices.Computability.pack4
public import DomainTheory.DataTypesAsLattices.Computability.mulC
public import DomainTheory.DataTypesAsLattices.Lambda.envSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofComb

@[expose] public section

set_option maxHeartbeats 2000000
```

```
namespace DomainTheory.DataTypesAsLattices

open Encodable

open Nat.Partrec (Code)

open Nat.Partrec.Code

def pairComb : Pomega := interp pairTerm arithEnv

namespace pairComb

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem app (n m : ℕ) :
    funOf (funOf pairComb (ofNat n)) (ofNat m) = ofNat (Nat.pair n m) := by
  unfold pairComb pairTerm
  rw [theorem_2_2.beta, theorem_2_2.beta]
  simp [interp, envSet, Function.update, arithEnv]
  rw [subC.realizes2, mulC.realizes2, mulC.realizes2, addC.realizes2,
    addC.realizes2, addC.realizes2]
  simp [Nat.pair]
  by_cases h : n < m
  · have : m - n = 0 := Nat.sub_ne_zero_of_lt h
    obtain ⟨k, hk⟩ := Nat.exists_eq_succ_of_ne_zero this
    rw [hk, eq_2_7_succ]
    simp [h]
  · have : m - n = 0 := Nat.sub_eq_zero_of_le (Nat.not_lt.mp h)
    rw [this, eq_2_7_zero]
    simp [h]

theorem combinatory : IsCombinatory pairComb := by
  rw [pairComb, theorem_2_4.complete]
  exact ofComb.combinatory _ _ arithEnv.combinatory

end pairComb

end DomainTheory.DataTypesAsLattices
```

pairRealizer.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Lambda.Kcomb
public import DomainTheory.DataTypesAsLattices.Lambda.Scomb
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Computability.pairComb

public import DomainTheory.DataTypesAsLattices.Graph.Basic.notation3
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Computability.Realizes
public import DomainTheory.DataTypesAsLattices.Computability.pack4
```

```
public import DomainTheory.DataTypesAsLattices.Lambda.Kcomb_more
public import DomainTheory.DataTypesAsLattices.Lambda.Scomb_more
@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable

open Nat.Partrec (Code)

open Nat.Partrec.Code

def pairRealizer (u v : Pomega) : Pomega :=
  Scomb ? (Scomb ? (Kcomb ? pairComb) ? u) ? v

namespace pairRealizer

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem combinatory {u v : Pomega}
    (hu : IsCombinatory u) (hv : IsCombinatory v) :
    IsCombinatory (pairRealizer u v) :=
  .app (.app .S (.app (.app .S (.app .K pairComb.combinatory)) hu)) hv

theorem realizes {u v : Pomega} {p q : ℕ → ℕ}
    (hu : Realizes u p) (hv : Realizes v q) :
    Realizes (pairRealizer u v) (fun n => Nat.pair (p n) (q n)) := fun n => by
  rw [pairRealizer, Scomb.beta3, Scomb.beta3, Kcomb.beta2,
    hu, hv, pairComb.app]

end pairRealizer

end DomainTheory.DataTypesAsLattices
```

precEnv.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Computability.pairComb
public import DomainTheory.DataTypesAsLattices.Computability.unpairFstC
public import DomainTheory.DataTypesAsLattices.Computability.unpairSndC
public import DomainTheory.DataTypesAsLattices.Computability.primRecOpC
public import DomainTheory.DataTypesAsLattices.Computability.pack4

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices
```

```
open Encodable

open Nat.Partrec (Code)

open Nat.Partrec.Code

/-- Environment for the primitive-recursion constructor of `Nat.Primrec`. -/
def precEnv (f g : Pomega) : ℕ → Pomega
  | 3 => primRecOpC
  | 4 => f
  | 5 => g
  | 6 => unpairFstC
  | 7 => unpairSndC
  | 8 => pairComb
  | _ => botElem

namespace precEnv

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem combinatory {f g : Pomega}
    (hf : IsCombinatory f) (hg : IsCombinatory g) :
    ∀ i, IsCombinatory (precEnv f g i)
  | 3 => primRecOpC.combinatory
  | 4 => hf
  | 5 => hg
  | 6 => unpairFstC.combinatory
  | 7 => unpairSndC.combinatory
  | 8 => pairComb.combinatory
  | 0 | 1 | 2 => botElem_combinatory
  | _ + 9 => botElem_combinatory

end precEnv

end DomainTheory.DataTypesAsLattices
```

precC.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Lambda.interp
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Computability.precEnv
public import DomainTheory.DataTypesAsLattices.Computability.precTerm
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Graph.lam
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Graph.Basic.notation3
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Computability.Realizes
public import DomainTheory.DataTypesAsLattices.Computability.pairComb
```

```
public import DomainTheory.DataTypesAsLattices.Computability.unpairSndC
public import DomainTheory.DataTypesAsLattices.Computability.unpairFstC
public import DomainTheory.DataTypesAsLattices.Computability.pack4
public import DomainTheory.DataTypesAsLattices.Computability.primRecOpC
public import DomainTheory.DataTypesAsLattices.Computability.primRecComb
public import DomainTheory.DataTypesAsLattices.Lambda.envSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofComb
public import DomainTheory.DataTypesAsLattices.Lambda.primRecVal

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable

open Nat.Partrec (Code)

open Nat.Partrec.Code

def precC (f g : Pomega) : Pomega := interp precTerm (precEnv f g)

namespace precC

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem combinatory {f g : Pomega}
    (hf : IsCombinatory f) (hg : IsCombinatory g) :
    IsCombinatory (precC f g) := by
  rw [precC, theorem_2_4.complete]
  exact ofComb.combinatory _ _ (precEnv.combinatory hf hg)

theorem realizes {f g : Pomega} {p q : ℕ → ℕ}
    (hf : Realizes f p) (hg : Realizes g q) :
    Realizes (precC f g)
      (Nat.unpaired fun z n => n.rec (p z) fun y ih => q (Nat.pair z (Nat.pair y ih)))
    := by
  intro w
  rw [precC, precTerm, theorem_2_2.beta]
  simp only [interp, envSet, Function.update_self,
    Function.update_of_ne (by decide : (3 : ℕ) = 0),
    Function.update_of_ne (by decide : (4 : ℕ) = 0),
    Function.update_of_ne (by decide : (6 : ℕ) = 0),
    Function.update_of_ne (by decide : (7 : ℕ) = 0),
    precEnv]
  simp [Function.update]
  rw [unpairFstC.realizes, unpairSndC.realizes, hf, primRecOpC.app,
    primRecComb.ofNat]
  let z := w.unpair.1
  let step : Pomega :=
    interp (.lam 1 (.lam 2
      (.app (.var 5)
        (.app (.app (.var 8) (ofNat z |> fun _ => .app (.var 6) (.var 0)))
```

```
          (.app (.app (.var 8) (.var 1)) (.var 2))))))
      (Function.update (precEnv f g) 0 (ofNat w))
  have hstep (y ih : ℕ) :
      (step ? ofNat y) ? ofNat ih =
        ofNat (q (Nat.pair z (Nat.pair y ih))) := by
    dsimp [step, z]
    rw [theorem_2_2.beta, theorem_2_2.beta]
    simp [interp, envSet, Function.update, precEnv]
    rw [unpairFstC.realizes, pairComb.app, pairComb.app, hg]
  have hrec : ∀ n,
      primRecVal (ofNat (p z)) step n =
        ofNat (n.rec (p z) fun y ih => q (Nat.pair z (Nat.pair y ih))) := by
    intro n
    induction n with
    | zero => rfl
    | succ n ih =>
      change (step ? ofNat n) ? primRecVal (ofNat (p z)) step n = _
      rw [ih, hstep]
  exact hrec w.unpair.2

end precC

end DomainTheory.DataTypesAsLattices
```

primrec_realized.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Lambda.IsCombinatory
public import DomainTheory.DataTypesAsLattices.Lambda.sucC
public import DomainTheory.DataTypesAsLattices.Lambda.succSet
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Computability.pack4
public import DomainTheory.DataTypesAsLattices.Computability.Realizes
public import DomainTheory.DataTypesAsLattices.Computability.composeC
public import DomainTheory.DataTypesAsLattices.Computability.constComb
public import DomainTheory.DataTypesAsLattices.Computability.unpairFstC
public import DomainTheory.DataTypesAsLattices.Computability.unpairSndC
public import DomainTheory.DataTypesAsLattices.Computability.pairRealizer
public import DomainTheory.DataTypesAsLattices.Computability.precC
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Retracts.comp
public import DomainTheory.DataTypesAsLattices.Lambda.sucC_more
public import DomainTheory.DataTypesAsLattices.Lambda.succSet_more
@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code
```

```
/-- Every unary primitive-recursive function has a combinatory realizer. -/
theorem primrec_realized {p : ℕ → ℕ} (hp : Nat.Primrec p) :
    ∃ u, IsCombinatory u ∧ Realizes u p := by
  induction hp with
  | zero =>
      exact ⟨constComb 0, constComb.combinatory 0, constComb.realizes 0⟩
  | succ =>
      exact ⟨sucC, .suc, fun n => by rw [sucC.app, succSet.ofNat]⟩
  | left =>
      exact ⟨unpairFstC, unpairFstC.combinatory, unpairFstC.realizes⟩
  | right =>
      exact ⟨unpairSndC, unpairSndC.combinatory, unpairSndC.realizes⟩
  | pair _ _ ihf ihg =>
      obtain ⟨u, hu, hru⟩ := ihf
      obtain ⟨v, hv, hrv⟩ := ihg
      exact ⟨pairRealizer u v, pairRealizer.combinatory hu hv,
        pairRealizer.realizes hru hrv⟩
  | comp _ _ ihf ihg =>
      obtain ⟨u, hu, hru⟩ := ihf
      obtain ⟨v, hv, hrv⟩ := ihg
      exact ⟨composeC u v, composeC.combinatory hu hv, composeC.realizes hru hrv⟩
  | prec _ _ ihf ihg =>
      obtain ⟨u, hu, hru⟩ := ihf
      obtain ⟨v, hv, hrv⟩ := ihg
      exact ⟨precC u v, precC.combinatory hu hv, precC.realizes hru hrv⟩

end DomainTheory.DataTypesAsLattices
```

IsREu2ecombinatory.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Lambda.IsCombinatory
public import DomainTheory.DataTypesAsLattices.Lambda.predC
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.predSet
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Computability.IsRE
public import DomainTheory.DataTypesAsLattices.Computability.pack4
public import DomainTheory.DataTypesAsLattices.Computability.enumFun
public import DomainTheory.DataTypesAsLattices.Computability.enumRangeC
public import DomainTheory.DataTypesAsLattices.Computability.primrec_realized
public import DomainTheory.DataTypesAsLattices.Computability.primrec_enumFun
public import DomainTheory.DataTypesAsLattices.Graph.Basic.notation3
@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code
```

```
/-- **Scott 1976, Appendix.** Every r.e. element belongs to the
combinatory closure. -/
theorem IsRE.combinatory {u : Pomega} (hu : IsRE u) : IsCombinatory u := by
  have hpart : Partrec fun k : ℕ =>
      Part.assert (k ∈ u) fun _ => Part.some (0 : ℕ) :=
    hu.out.map (Computable.const (0 : ℕ)).to₂
  obtain ⟨c, hc⟩ := Code.exists_code.1 (Partrec.nat_iff.1 hpart)
  obtain ⟨r, hr, hreal⟩ := primrec_realized (primrec_enumFun c)
  have hrange := enumRangeC.eq hreal
  have hpred : DataTypesAsLattices.predSet (enumRangeC r) = u := by
    ext n
    constructor
    · intro hn
      rw [hrange] at hn
      obtain ⟨t, ht⟩ := Set.mem_iUnion.mp hn
      have heq : enumFun c t = n + 1 := by simpa [ofNat] using ht.symm
      have hs : (evaln t.unpair.2 c t.unpair.1).isSome = true := by
        by_contra h
        have : (evaln t.unpair.2 c t.unpair.1).isSome = false :=
          Bool.eq_false_of_not_eq_true h
        simp [enumFun, this] at heq
      have hfirst : t.unpair.1 = n := by
        simp [enumFun, hs] at heq
        omega
      obtain ⟨v, hv⟩ := Option.isSome_iff_exists.mp hs
      have hmem : v ∈ Code.eval c t.unpair.1 :=
        evaln_sound (by simpa [Option.mem_def] using hv)
      rw [hc] at hmem
      simpa [hfirst] using (Part.mem_assert_iff.mp hmem).1
    · intro hn
      have hmem : 0 ∈ Code.eval c n := by
        rw [hc]
        exact Part.mem_assert_iff.mpr ⟨hn, Part.mem_some_iff.mpr rfl⟩
      obtain ⟨s, hs⟩ := evaln_complete.mp hmem
      rw [hrange]
      refine Set.mem_iUnion.mpr ⟨Nat.pair n s, ?_⟩
      have hsome : (evaln s c n).isSome = true := Option.isSome_of_mem hs
      simp [enumFun, Nat.unpair_pair, hsome, ofNat]
  have hcomb : IsCombinatory (DataTypesAsLattices.predC ? enumRangeC r) :=
    .app .pred (enumRangeC.combinatory hr)
  rw [DataTypesAsLattices.predC.app, hpred] at hcomb
  exact hcomb

end DomainTheory.DataTypesAsLattices
```

primrecRel_eSubset.lean

```
module
public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Computability.IsRE
public import DomainTheory.DataTypesAsLattices.Computability.IsComputable
public import DomainTheory.DataTypesAsLattices.Computability.LambdaDefinable
```

```
public import DomainTheory.DataTypesAsLattices.Computability.pack4
public import DomainTheory.DataTypesAsLattices.Computability.topRecTerm
public import DomainTheory.DataTypesAsLattices.Computability.Realizes
public import DomainTheory.DataTypesAsLattices.Computability.Realizes2
public import DomainTheory.DataTypesAsLattices.Computability.composeC
public import DomainTheory.DataTypesAsLattices.Computability.constComb
public import DomainTheory.DataTypesAsLattices.Computability.primRecComb
public import DomainTheory.DataTypesAsLattices.Computability.addStepC
public import DomainTheory.DataTypesAsLattices.Computability.addC
public import DomainTheory.DataTypesAsLattices.Computability.safePredC
public import DomainTheory.DataTypesAsLattices.Computability.subStepC
public import DomainTheory.DataTypesAsLattices.Computability.subC
public import DomainTheory.DataTypesAsLattices.Computability.mulC
public import DomainTheory.DataTypesAsLattices.Computability.isZeroC
public import DomainTheory.DataTypesAsLattices.Computability.leC
public import DomainTheory.DataTypesAsLattices.Computability.pairTerm
public import DomainTheory.DataTypesAsLattices.Computability.arithEnv
public import DomainTheory.DataTypesAsLattices.Computability.pairComb
public import DomainTheory.DataTypesAsLattices.Computability.primRecZeroC
public import DomainTheory.DataTypesAsLattices.Computability.sqrtIter
public import DomainTheory.DataTypesAsLattices.Computability.sqrtEnv
public import DomainTheory.DataTypesAsLattices.Computability.sqrtStepTerm
public import DomainTheory.DataTypesAsLattices.Computability.sqrtStepC
public import DomainTheory.DataTypesAsLattices.Computability.sqrtC
public import DomainTheory.DataTypesAsLattices.Computability.unpairEnv
public import DomainTheory.DataTypesAsLattices.Computability.unpairSqrtTerm
public import DomainTheory.DataTypesAsLattices.Computability.unpairSquareTerm
public import DomainTheory.DataTypesAsLattices.Computability.unpairDeltaTerm
public import DomainTheory.DataTypesAsLattices.Computability.unpairTestTerm
public import DomainTheory.DataTypesAsLattices.Computability.unpairFstTerm
public import DomainTheory.DataTypesAsLattices.Computability.unpairSndTerm
public import DomainTheory.DataTypesAsLattices.Computability.unpairFstC
public import DomainTheory.DataTypesAsLattices.Computability.unpairSndC
public import DomainTheory.DataTypesAsLattices.Computability.primRecOpEnv
public import DomainTheory.DataTypesAsLattices.Computability.primRecOpTerm
public import DomainTheory.DataTypesAsLattices.Computability.primRecOpC
public import DomainTheory.DataTypesAsLattices.Computability.pairRealizer
public import DomainTheory.DataTypesAsLattices.Computability.precEnv
public import DomainTheory.DataTypesAsLattices.Computability.precTerm
public import DomainTheory.DataTypesAsLattices.Computability.precC
public import DomainTheory.DataTypesAsLattices.Computability.enumFun
public import DomainTheory.DataTypesAsLattices.Computability.enumRangeC
public import DomainTheory.DataTypesAsLattices.Computability.ComputableFin
public import DomainTheory.DataTypesAsLattices.Computability.REPredu2ecomp
public import DomainTheory.DataTypesAsLattices.Computability.primrec_testBit
public import DomainTheory.DataTypesAsLattices.Computability.REPredu2eor
public import DomainTheory.DataTypesAsLattices.Computability.primrecPred_bool
public import DomainTheory.DataTypesAsLattices.Computability.eSubset_e_iff

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices
open Encodable
open Nat.Partrec (Code)
```

```
open Nat.Partrec.Code

theorem primrecRel_eSubset : PrimrecRel fun n b : ℕ => e n ⊆ e b := by
  have hR : PrimrecRel fun k q : ℕ =>
      q.unpair.1.testBit k = false ∨ q.unpair.2.testBit k = true :=
    ((primrecPred_bool
        (Primrec₂.comp primrec_testBit
          (Primrec.fst.comp (Primrec.unpair.comp Primrec.snd)) Primrec.fst)).not.or
      (primrecPred_bool
        (Primrec₂.comp primrec_testBit
          (Primrec.snd.comp (Primrec.unpair.comp Primrec.snd)) Primrec.fst))).of_eq
      fun _ => by simp [Bool.not_eq_true]
  refine (hR.forall_lt.comp Primrec.fst
      (Primrec₂.natPair.comp Primrec.fst Primrec.snd)).of_eq fun p => ?_
  simpa [Nat.unpair_pair] using (eSubset_e_iff p.1 p.2).symm

end DomainTheory.DataTypesAsLattices
```

primrecRel_mem_funOf_e.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Computability.primrec_twoPow
public import DomainTheory.DataTypesAsLattices.Computability.primrec_testBit
public import DomainTheory.DataTypesAsLattices.Computability.primrec_pair
public import DomainTheory.DataTypesAsLattices.Computability.primrecPred_bool
public import DomainTheory.DataTypesAsLattices.Computability.primrecRel_eSubset
public import DomainTheory.DataTypesAsLattices.Computability.mem_funOf_e_iff

public import DomainTheory.DataTypesAsLattices.Graph.Basic.notation3
public import DomainTheory.DataTypesAsLattices.Retracts.comp
@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem primrecRel_mem_funOf_e :
    PrimrecRel fun q m : ℕ =>
      m ∈ e (Nat.unpair q).1 ? e (Nat.unpair q).2 := by
  have hinner : PrimrecRel fun n packed : ℕ =>
      e n ⊆ e (Nat.unpair (Nat.unpair packed).1).2 ∧
        (Nat.unpair (Nat.unpair packed).1).1.testBit
          (pair n (Nat.unpair packed).2) :=
    ((primrecRel_eSubset.comp Primrec.fst
        (Primrec.snd.comp (Primrec.unpair.comp
          (Primrec.fst.comp (Primrec.unpair.comp Primrec.snd))))).and
      (primrecPred_bool (Primrec₂.comp primrec_testBit
```

```
        (Primrec.fst.comp (Primrec.unpair.comp
          (Primrec.fst.comp (Primrec.unpair.comp Primrec.snd))))
        (primrec_pair.comp Primrec.fst
          (Primrec.snd.comp (Primrec.unpair.comp Primrec.snd))))))
  refine (hinner.exists_lt.comp
      (primrec_twoPow.comp (Primrec.succ.comp
        (Primrec.snd.comp (Primrec.unpair.comp Primrec.fst))))
      (Primrec₂.natPair.comp Primrec.fst Primrec.snd)).of_eq
    fun q => ?_
  simp only [Nat.unpair_pair]
  exact (mem_funOf_e_iff q.1.unpair.1 q.1.unpair.2 q.2).symm

end DomainTheory.DataTypesAsLattices
```

primrecRel_Scomb_bit.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Computability.pack4
public import DomainTheory.DataTypesAsLattices.Computability.primrec_testBit
public import DomainTheory.DataTypesAsLattices.Computability.primrecPred_bool
public import DomainTheory.DataTypesAsLattices.Computability.primrecRel_mem_funOf_e
public import DomainTheory.DataTypesAsLattices.Computability.primrec_pack4_proj2
public import DomainTheory.DataTypesAsLattices.Computability.primrec_pack4_proj3

public import DomainTheory.DataTypesAsLattices.Graph.Basic.notation3
public import DomainTheory.DataTypesAsLattices.Retracts.comp
@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem primrecRel_Scomb_bit :
    PrimrecRel fun i kt : ℕ =>
      (Nat.unpair kt).1.testBit i = false ∨
        i ∈ e (pack4.proj2 (Nat.unpair kt).2) ?
          e (pack4.proj3 (Nat.unpair kt).2) :=
  ((primrecPred_bool (Primrec₂.comp primrec_testBit
      (Primrec.fst.comp (Primrec.unpair.comp Primrec.snd))
      Primrec.fst)).not.or
    (primrecRel_mem_funOf_e.comp
      (Primrec₂.natPair.comp
        (primrec_pack4_proj2.comp (Primrec.snd.comp (Primrec.unpair.comp Primrec.snd))
    )
        (primrec_pack4_proj3.comp (Primrec.snd.comp (Primrec.unpair.comp Primrec.snd))
    ))
      Primrec.fst)).of_eq fun p => by simp [Bool.not_eq_true]
```

```
end DomainTheory.DataTypesAsLattices
```

primrecPred_Scomb_subset.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Computability.pack4
public import DomainTheory.DataTypesAsLattices.Computability.primrecRel_Scomb_bit

public import DomainTheory.DataTypesAsLattices.Graph.Basic.notation3
public import DomainTheory.DataTypesAsLattices.Retracts.comp
@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem primrecPred_Scomb_subset :
    PrimrecPred fun p : ℕ × ℕ =>
      ∀ i < p.1, p.1.testBit i = false ∨
        i ∈ e (pack4.proj2 p.2) ? e (pack4.proj3 p.2) :=
  (primrecRel_Scomb_bit.forall_lt.comp Primrec.fst
    (Primrec₂.natPair.comp Primrec.fst Primrec.snd)).of_eq fun p => by
      simp [Nat.unpair_pair]

end DomainTheory.DataTypesAsLattices
```

primrecRel_Scomb_app.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Computability.pack4
public import DomainTheory.DataTypesAsLattices.Computability.primrec_pair
public import DomainTheory.DataTypesAsLattices.Computability.primrecRel_mem_funOf_e
public import DomainTheory.DataTypesAsLattices.Computability.eSubset_funOf_e_iff
public import DomainTheory.DataTypesAsLattices.Computability.primrec_pack4_proj1
public import DomainTheory.DataTypesAsLattices.Computability.primrec_pack4_proj3
public import DomainTheory.DataTypesAsLattices.Computability.primrec_pack4_proj4
public import DomainTheory.DataTypesAsLattices.Computability.primrecPred_Scomb_subset

public import DomainTheory.DataTypesAsLattices.Graph.Basic.notation3
public import DomainTheory.DataTypesAsLattices.Retracts.comp
@[expose] public section
```

```
set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem primrecRel_Scomb_app :
    PrimrecRel fun k t : ℕ =>
      e k ⊆ e (pack4.proj2 t) ? e (pack4.proj3 t) ∧
        pair k (pack4.proj4 t) ∈ e (pack4.proj1 t) ? e (pack4.proj3 t) := by
  have hmem : PrimrecPred fun p : ℕ × ℕ =>
      pair p.1 (pack4.proj4 p.2) ∈
        e (pack4.proj1 p.2) ? e (pack4.proj3 p.2) :=
    (primrecRel_mem_funOf_e.comp
      (Primrec₂.natPair.comp (primrec_pack4_proj1.comp Primrec.snd)
        (primrec_pack4_proj3.comp Primrec.snd))
      (primrec_pair.comp Primrec.fst (primrec_pack4_proj4.comp Primrec.snd))).of_eq
      fun p => by simp [Nat.unpair_pair]
  refine (primrecPred_Scomb_subset.and hmem).of_eq fun p => ?_
  constructor
  · intro ⟨hs, hm⟩
    exact ⟨(eSubset_funOf_e_iff _ _ _).mpr hs, hm⟩
  · intro ⟨hs, hm⟩
    exact ⟨(eSubset_funOf_e_iff _ _ _).mp hs, hm⟩

end DomainTheory.DataTypesAsLattices
```

rePred_Scomb_nested.lean

```
module

public import DomainTheory.DataTypesAsLattices.Computability.REPredu2eexists
public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Computability.pack4
public import DomainTheory.DataTypesAsLattices.Computability.primrecRel_Scomb_app

public import DomainTheory.DataTypesAsLattices.Graph.Basic.notation3
@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem rePred_Scomb_nested :
    REPred fun t : ℕ =>
      pack4.proj4 t ∈
        (e (pack4.proj1 t) ? e (pack4.proj3 t)) ?
```

```
          (e (pack4.proj2 t) ? e (pack4.proj3 t)) :=
  (REPred.exists (p := fun t k =>
      e k ⊆ e (pack4.proj2 t) ? e (pack4.proj3 t) ∧
        pair k (pack4.proj4 t) ∈ e (pack4.proj1 t) ? e (pack4.proj3 t))
    (primrecRel_Scomb_app.swap.computablePred.to_re)).of_eq fun _ => Iff.rfl

end DomainTheory.DataTypesAsLattices
```

IsREu2eScomb.lean

```
module
public import DomainTheory.DataTypesAsLattices.Computability.REPredu2eempty
public import DomainTheory.DataTypesAsLattices.Computability.REPredu2etrue
public import DomainTheory.DataTypesAsLattices.Computability.REPredu2ecomp
public import DomainTheory.DataTypesAsLattices.Computability.REPredu2eexists
public import DomainTheory.DataTypesAsLattices.Computability.REPredu2eor
public import DomainTheory.DataTypesAsLattices.Computability.REPredu2eand

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Lambda.Scomb
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Computability.IsRE
public import DomainTheory.DataTypesAsLattices.Computability.pack4
public import DomainTheory.DataTypesAsLattices.Computability.mem_Scomb_iff
public import DomainTheory.DataTypesAsLattices.Computability.rePred_Scomb_nested
public import DomainTheory.DataTypesAsLattices.Computability.primrecPred_Scomb_eq

public import DomainTheory.DataTypesAsLattices.Graph.Basic.notation3
public import DomainTheory.DataTypesAsLattices.Retracts.comp
@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem IsRE.Scomb : IsRE Scomb := by
  have hmem : REPred fun q : ℕ × ℕ =>
      pack4.proj4 q.2 ∈
        (e (pack4.proj1 q.2) ? e (pack4.proj3 q.2)) ?
          (e (pack4.proj2 q.2) ? e (pack4.proj3 q.2)) :=
    rePred_Scomb_nested.comp Primrec.snd.to_comp
  have hex : REPred fun p =>
      ∃ t, p = pair (pack4.proj1 t)
          (pair (pack4.proj2 t) (pair (pack4.proj3 t) (pack4.proj4 t))) ∧
        pack4.proj4 t ∈
          (e (pack4.proj1 t) ? e (pack4.proj3 t)) ?
            (e (pack4.proj2 t) ? e (pack4.proj3 t)) :=
    REPred.exists (REPred.and primrecPred_Scomb_eq.computablePred.to_re hmem)
  constructor
  refine hex.of_eq fun p => ?_
```

```
  constructor
  · intro ⟨t, ht, hm⟩
    exact (mem_Scomb_iff p).mpr
      ⟨pack4.proj1 t, pack4.proj2 t, pack4.proj3 t, pack4.proj4 t, ht, hm⟩
  · intro hp
    obtain ⟨n0, n1, n2, m, ht, hm⟩ := (mem_Scomb_iff p).mp hp
    refine ⟨pack4 n0 n1 n2 m, ?_, ?_⟩
    · simpa [pack4, pack4.proj1, pack4.proj2, pack4.proj3, pack4.proj4,
        Nat.unpair_pair] using ht
    · simpa [pack4, pack4.proj1, pack4.proj2, pack4.proj3, pack4.proj4,
        Nat.unpair_pair] using hm

end DomainTheory.DataTypesAsLattices
```

combinatory_isRE.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Lambda.IsCombinatory
public import DomainTheory.DataTypesAsLattices.Lambda.zeroC
public import DomainTheory.DataTypesAsLattices.Lambda.sucC
public import DomainTheory.DataTypesAsLattices.Lambda.predC
public import DomainTheory.DataTypesAsLattices.Lambda.condC
public import DomainTheory.DataTypesAsLattices.Lambda.Kcomb
public import DomainTheory.DataTypesAsLattices.Lambda.Scomb
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Computability.IsRE
public import DomainTheory.DataTypesAsLattices.Computability.IsREu2efunOf
public import DomainTheory.DataTypesAsLattices.Computability.IsREu2eScomb
public import DomainTheory.DataTypesAsLattices.Computability.IsREu2eKcomb
public import DomainTheory.DataTypesAsLattices.Computability.IsREu2econdC
public import DomainTheory.DataTypesAsLattices.Computability.IsREu2epredC
public import DomainTheory.DataTypesAsLattices.Computability.IsREu2esucC
public import DomainTheory.DataTypesAsLattices.Computability.IsREu2ezeroC
public import DomainTheory.DataTypesAsLattices.Computability.pack4

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

/-- Combinatory elements are r.e. (one direction of the appendix equivalence). -/
theorem combinatory_isRE {u : Pomega} (hu : IsCombinatory u) : IsRE u := by
  induction hu with
  | zero => exact IsRE.zeroC
  | suc => exact IsRE.sucC
  | pred => exact IsRE.predC
  | cond => exact IsRE.condC
```

```
  | K => exact IsRE.Kcomb
  | S => exact IsRE.Scomb
  | app _ _ iu iv => exact IsRE.funOf iu iv

end DomainTheory.DataTypesAsLattices
```

IsREu2eiff_combinatory.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Lambda.IsCombinatory
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Computability.IsRE
public import DomainTheory.DataTypesAsLattices.Computability.pack4
public import DomainTheory.DataTypesAsLattices.Computability.combinatory_isRE
public import DomainTheory.DataTypesAsLattices.Computability.IsREu2ecombinatory

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem IsRE.iff_combinatory {u : Pomega} : IsRE u ↔ IsCombinatory u :=
  ⟨IsRE.combinatory, combinatory_isRE⟩

end DomainTheory.DataTypesAsLattices
```

BooleanMemE.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import Mathlib.Data.Set.Finite.Powerset

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace

/-- Witness used in the inductive proof of Theorem 6.6. -/
def BooleanMemE (U : Set Pomega) : Prop :=
  ∃ (f : Pomega → Pomega) (N : ℕ) (E : Set Pomega),
```

```
    IsScottContinuous f ∧
      (∀ x, f x ⊆ {m | m < N}) ∧
        E.Finite ∧ (∀ e ∈ E, e ⊆ {m | m < N}) ∧
          U = {x | f x ∈ E}

end DomainTheory.DataTypesAsLattices
```

IsBdelta.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import Mathlib.Data.Set.Finite.Powerset
public import DomainTheory.DataTypesAsLattices.Classification.IsBooleanOpen

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace

def IsBdelta (U : Set Pomega) : Prop :=
  ∃ Bs : ℕ → Set Pomega, (∀ n, IsBooleanOpen (Bs n)) ∧ U = ⋂ n, Bs n

end DomainTheory.DataTypesAsLattices
```

IsFcapG.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottOpen
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import Mathlib.Data.Set.Finite.Powerset

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace

def IsFcapG (U : Set Pomega) : Prop :=
  ∃ C O, IsScottOpen C? ∧ IsScottOpen O ∧ U = C ∩ O

end DomainTheory.DataTypesAsLattices
```

ScottB.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import Mathlib.Data.Set.Finite.Powerset
public import DomainTheory.DataTypesAsLattices.Classification.IsBooleanOpen

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace

def ScottB : Set (Set Pomega) := {U | IsBooleanOpen U}

end DomainTheory.DataTypesAsLattices
```

ThreeValued.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import Mathlib.Data.Set.Finite.Powerset

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace

class ThreeValued (x : Pomega) : Prop where
  out : x = botElem ∨ x = ofNat 0 ∨ x = topElem

end DomainTheory.DataTypesAsLattices
```

charOpen.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
```

```
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import Mathlib.Data.Set.Finite.Powerset

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace

/-- Characteristic-style map used in Theorem 6.1: `0 ∈ f(x)` iff some
finite piece of `x` already lies in `U`. -/
def charOpen (U : Set Pomega) (x : Pomega) : Pomega :=
  {k | k = 0 ∧ ∃ n, e n ⊆ x ∧ e n ∈ U}

end DomainTheory.DataTypesAsLattices
```

charTop.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import Mathlib.Data.Set.Finite.Powerset

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace

def charTop (U : Set Pomega) (x : Pomega) : Pomega :=
  {_k | ∃ n, e n ⊆ x ∧ e n ∈ U}

end DomainTheory.DataTypesAsLattices
```

closedToZero.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import Mathlib.Data.Set.Finite.Powerset
public import DomainTheory.DataTypesAsLattices.Classification.charTop
```

```
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace

/-- Three-valued characteristic of a Scott-closed set: `0` on `C`, `⊤` off `C`. -/
def closedToZero (C : Set Pomega) (x : Pomega) : Pomega :=
  ofNat 0 ∪ charTop C? x

end DomainTheory.DataTypesAsLattices
```

evenPart.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import Mathlib.Data.Set.Finite.Powerset

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace

def evenPart (y : Pomega) : Pomega := {n | 2 * n ∈ y}

end DomainTheory.DataTypesAsLattices
```

oddEvenPack.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import Mathlib.Data.Set.Finite.Powerset

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace

def oddEvenPack (f g : Pomega → Pomega) (x : Pomega) : Pomega :=
```

```
  {k | (∃ n, k = 2 * n + 1 ∧ n ∈ f x) ∨ (∃ n, k = 2 * n ∧ n ∈ g x)}

end DomainTheory.DataTypesAsLattices
```

oddPart.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import Mathlib.Data.Set.Finite.Powerset

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace

def oddPart (y : Pomega) : Pomega := {n | 2 * n + 1 ∈ y}

end DomainTheory.DataTypesAsLattices
```

pairIndexed.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import Mathlib.Data.Set.Finite.Powerset

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace

def pairIndexed (fs : ℕ → Pomega → Pomega) (x : Pomega) : Pomega :=
  {k | ∃ n m, k = pair n m ∧ m ∈ fs n x}

end DomainTheory.DataTypesAsLattices
```

typicalBdelta.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import Mathlib.Data.Set.Finite.Powerset

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace

def typicalBdelta : Set Pomega :=
  {u | funOf u (ofNat 0) = funOf u (ofNat 1)}

end DomainTheory.DataTypesAsLattices
```

uvU.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import Mathlib.Data.Set.Finite.Powerset

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace

/-- Scott's table `u` on `{⊥, 0, ⊤}`, extended continuously to all of `Pω`. -/
def uvU (x y : Pomega) : Pomega :=
  {k | (k = 0 ∧ (x.Nonempty ∨ y.Nonempty)) ∨
       (k = 1 ∧ ((0 ∈ x ∧ ∃ n, n + 1 ∈ y) ∨ ((∃ n, n + 1 ∈ x) ∧ 0 ∈ y)))}

end DomainTheory.DataTypesAsLattices
```

uvV.lean

```
module
```

```
public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import Mathlib.Data.Set.Finite.Powerset

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace

/-- Scott's table `v` on `{⊥, 0, ⊤}`, extended continuously to all of `Pω`. -/
def uvV (x y : Pomega) : Pomega :=
  {k | k = 0 ∨
       (k = 1 ∧ ((∃ n, n + 1 ∈ x) ∨ (∃ n, n + 1 ∈ y))) ∨
       (0 ∈ x ∧ (∃ n, n + 1 ∈ x) ∧ 0 ∈ y ∧ (∃ n, n + 1 ∈ y))}

end DomainTheory.DataTypesAsLattices
```

zeroPrime.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import Mathlib.Data.Set.Finite.Powerset

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace

/-- Scott's auxiliary value `0'` used in the many-valued tables of Theorem 6.7.
    The vision transcription writes `0' = 0 ∪ ⊥`, which equals `0` and would
    collapse the tables; the tables need a point distinct from `⊥`, `0`, and
    `⊤`, so we take `0 ∪ 1`. -/
def zeroPrime : Pomega := ofNat 0 ∪ ofNat 1

end DomainTheory.DataTypesAsLattices
```

Basic.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
```

```
public import DomainTheory.DataTypesAsLattices.Lambda.seq2
public import DomainTheory.DataTypesAsLattices.Lambda.e_one
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Lambda.const_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.predSet
public import DomainTheory.DataTypesAsLattices.Continuous.isScottOpen_isUpper
public import DomainTheory.DataTypesAsLattices.Continuous.mem_scottUnion
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Continuous.isOpen_memOpen
public import DomainTheory.DataTypesAsLattices.Continuous.isScottOpen_iff_isOpen
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottOpen
public import DomainTheory.DataTypesAsLattices.Continuous.scottUnion
public import DomainTheory.DataTypesAsLattices.Continuous.memOpen
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import Mathlib.Data.Set.Finite.Powerset

public import DomainTheory.DataTypesAsLattices.Classification.IsBooleanOpen
public import DomainTheory.DataTypesAsLattices.Classification.charOpen
public import DomainTheory.DataTypesAsLattices.Classification.ScottB
public import DomainTheory.DataTypesAsLattices.Classification.IsFcapG
public import DomainTheory.DataTypesAsLattices.Classification.IsBdelta
public import DomainTheory.DataTypesAsLattices.Classification.charTop
public import DomainTheory.DataTypesAsLattices.Classification.pairIndexed
public import DomainTheory.DataTypesAsLattices.Classification.oddEvenPack
public import DomainTheory.DataTypesAsLattices.Classification.evenPart
public import DomainTheory.DataTypesAsLattices.Classification.oddPart
public import DomainTheory.DataTypesAsLattices.Classification.BooleanMemE
public import DomainTheory.DataTypesAsLattices.Classification.closedToZero
public import DomainTheory.DataTypesAsLattices.Classification.zeroPrime
public import DomainTheory.DataTypesAsLattices.Classification.uvU
public import DomainTheory.DataTypesAsLattices.Classification.uvV
public import DomainTheory.DataTypesAsLattices.Classification.ThreeValued
public import DomainTheory.DataTypesAsLattices.Classification.typicalBdelta
public import Mathlib.Data.Set.Lattice.Indexed
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace

theorem charOpen_isScottContinuous (U : Set Pomega) :
    IsScottContinuous (charOpen U) := by
  constructor
  intro x
  ext k
```

```
  constructor
  · intro ⟨hk0, n, hn, hne⟩
    exact mem_scottUnion.mpr ⟨n, hn, hk0, n, subset_rfl, hne⟩
  · intro hk
    obtain ⟨m, hm, hk0, n, hn, hne⟩ := mem_scottUnion.mp hk
    exact ⟨hk0, n, subset_trans hn hm, hne⟩

theorem isScottOpen_univ : IsScottOpen (Set.univ : Set Pomega) :=
  (isScottOpen_iff_isOpen _).mpr isOpen_univ

theorem isScottOpen_empty : IsScottOpen (∅ : Set Pomega) :=
  (isScottOpen_iff_isOpen _).mpr isOpen_empty

theorem isScottOpen_union {U V : Set Pomega} (hU : IsScottOpen U) (hV : IsScottOpen V)
    :
    IsScottOpen (U ∪ V) :=
  (isScottOpen_iff_isOpen _).mpr
    (((isScottOpen_iff_isOpen _).mp hU).union ((isScottOpen_iff_isOpen _).mp hV))

theorem isScottOpen_inter {U V : Set Pomega} (hU : IsScottOpen U) (hV : IsScottOpen V)
    :
    IsScottOpen (U ∩ V) :=
  (isScottOpen_iff_isOpen _).mpr
    (((isScottOpen_iff_isOpen _).mp hU).inter ((isScottOpen_iff_isOpen _).mp hV))

theorem isScottOpen_iUnion {ι : Type*} (U : ι → Set Pomega) (hU : ∀ i, IsScottOpen (U i
    )) :
    IsScottOpen (⋃ i, U i) :=
  (isScottOpen_iff_isOpen _).mpr (isOpen_iUnion fun i => (isScottOpen_iff_isOpen _).mp
    (hU i))

theorem mem_preimage_isScottOpen {f : Pomega → Pomega} (hf : IsScottContinuous f) (k :
    ℕ) :
    IsScottOpen {x | k ∈ f x} := by
  have : {x | k ∈ f x} = f ^-1' memOpen k := rfl
  rw [this, isScottOpen_iff_isOpen]
  exact IsOpen.preimage ((IsScottContinuous.iff_continuous f).mp hf) (isOpen_memOpen k
    )

theorem mem_isScottOpen_iff {U : Set Pomega} (hU : IsScottOpen U) {x : Pomega} :
    x ∈ U ↔ ∃ n, e n ⊆ x ∧ e n ∈ U := by
  constructor
  · intro hx
    obtain ⟨B, rfl⟩ := hU
    obtain ⟨n, hn, hxn⟩ := Set.mem_iUnion₂.mp hx
    exact ⟨n, hxn, Set.mem_iUnion₂.mpr ⟨n, hn, (subset_rfl : e n ⊆ e n)⟩⟩
  · intro ⟨n, hn, hne⟩
    exact isScottOpen_isUpper hU hne hn

theorem charTop_isScottContinuous (U : Set Pomega) :
    IsScottContinuous (charTop U) := by
  constructor
  intro x
  ext k
  constructor
  · intro ⟨n, hn, hne⟩
```

```
    exact mem_scottUnion.mpr ⟨n, hn, n, subset_rfl, hne⟩
  · intro hk
    obtain ⟨m, hm, n, hn, hne⟩ := mem_scottUnion.mp hk
    exact ⟨n, subset_trans hn hm, hne⟩

theorem charTop_eq_top {U : Set Pomega} (hU : IsScottOpen U) (x : Pomega) :
    charTop U x = topElem ↔ x ∈ U := by
  constructor
  · intro h
    have : 0 ∈ charTop U x := by rw [h]; exact Set.mem_univ 0
    obtain ⟨n, hn, hne⟩ := this
    exact isScottOpen_isUpper hU hne hn
  · intro hx
    ext k
    constructor
    · intro _; trivial
    · intro _
      exact (mem_isScottOpen_iff hU).mp hx

theorem charTop_eq_bot {U : Set Pomega} (hU : IsScottOpen U) (x : Pomega) :
    charTop U x = botElem ↔ x ∈ U := by
  constructor
  · intro h hx
    have : charTop U x = topElem := (charTop_eq_top hU x).mpr hx
    rw [h] at this
    have : (0 : ℕ) ∈ (botElem : Pomega) := by
      rw [this]; exact Set.mem_univ 0
    exact (mem_botElem 0).mp this
  · intro hx
    ext k
    constructor
    · intro ⟨n, hn, hne⟩
      exact hx (isScottOpen_isUpper hU hne hn)
    · intro hk
      exact ((mem_botElem k).mp hk).elim

theorem charTop_eq_bot_of_closed {C : Set Pomega} (hC : IsScottOpen C?) (x : Pomega) :
    charTop C? x = botElem ↔ x ∈ C := by
  rw [charTop_eq_bot hC]
  simp

theorem pairIndexed_isScottContinuous (fs : ℕ → Pomega → Pomega)
    (hfs : ∀ n, IsScottContinuous (fs n)) :
    IsScottContinuous (pairIndexed fs) := by
  constructor
  intro x
  ext k
  constructor
  · intro ⟨n, m, hk, hm⟩
    have : m ∈ scottUnion (fs n) x := by rw [← (hfs n).out x]; exact hm
    obtain ⟨p, hp, hmp⟩ := mem_scottUnion.mp this
    exact mem_scottUnion.mpr ⟨p, hp, n, m, hk, hmp⟩
  · intro hk
    obtain ⟨p, hp, n, m, hk, hm⟩ := mem_scottUnion.mp hk
    exact ⟨n, m, hk, IsScottContinuous.monotone (hfs n) hp hm⟩
```

```
theorem oddEvenPack_isScottContinuous {f g : Pomega → Pomega}
    (hf : IsScottContinuous f) (hg : IsScottContinuous g) :
    IsScottContinuous (oddEvenPack f g) := by
  constructor
  intro x
  ext k
  constructor
  · intro hk
    rcases hk with ⟨n, hk, hn⟩ | ⟨n, hk, hn⟩
    · have : n ∈ scottUnion f x := by rw [← hf.out x]; exact hn
      obtain ⟨p, hp, hnp⟩ := mem_scottUnion.mp this
      exact mem_scottUnion.mpr ⟨p, hp, Or.inl ⟨n, hk, hnp⟩⟩
    · have : n ∈ scottUnion g x := by rw [← hg.out x]; exact hn
      obtain ⟨p, hp, hnp⟩ := mem_scottUnion.mp this
      exact mem_scottUnion.mpr ⟨p, hp, Or.inr ⟨n, hk, hnp⟩⟩
  · intro hk
    obtain ⟨p, hp, h⟩ := mem_scottUnion.mp hk
    rcases h with ⟨n, hk, hn⟩ | ⟨n, hk, hn⟩
    · exact Or.inl ⟨n, hk, IsScottContinuous.monotone hf hp hn⟩
    · exact Or.inr ⟨n, hk, IsScottContinuous.monotone hg hp hn⟩

theorem evenPart_oddEven (f g : Pomega → Pomega) (x : Pomega) :
    evenPart (oddEvenPack f g x) = g x := by
  ext n
  constructor
  · intro hn
    rcases hn with ⟨k, hk, hkn⟩ | ⟨k, hk, hkn⟩
    · have : 2 * n = 2 * k + 1 := hk
      omega
    · have : k = n := by
        have : 2 * n = 2 * k := hk
        omega
      simpa [this] using hkn
  · intro hn
    exact Or.inr ⟨n, rfl, hn⟩

theorem oddPart_oddEven (f g : Pomega → Pomega) (x : Pomega) :
    oddPart (oddEvenPack f g x) = f x := by
  ext n
  constructor
  · intro hn
    rcases hn with ⟨k, hk, hkn⟩ | ⟨k, hk, hkn⟩
    · have : k = n := by
        have : 2 * n + 1 = 2 * k + 1 := hk
        omega
      simpa [this] using hkn
    · have : 2 * n + 1 = 2 * k := hk
      omega
  · intro hn
    exact Or.inl ⟨n, rfl, hn⟩

theorem oddEvenPack_subset {f g : Pomega → Pomega} {x : Pomega} {N M : ℕ}
    (hf : f x ⊆ {m | m < N}) (hg : g x ⊆ {m | m < M}) :
    oddEvenPack f g x ⊆ {k | k < 2 * Nat.max N M + 1} := by
  intro k hk
  rcases hk with ⟨n, rfl, hn⟩ | ⟨n, rfl, hn⟩
```

```
  · have hnN : n < N := hf hn
    have : n < Nat.max N M := lt_of_lt_of_le hnN (Nat.le_max_left _ _)
    exact Nat.succ_lt_succ (Nat.mul_lt_mul_of_pos_left this (by decide : 0 < 2))
  · have hnM : n < M := hg hn
    have hlt : n < Nat.max N M := lt_of_lt_of_le hnM (Nat.le_max_right _ _)
    have : 2 * n < 2 * Nat.max N M + 1 :=
      Nat.lt_succ_of_le (Nat.mul_le_mul_left 2 (Nat.le_of_lt hlt))
    exact this

theorem finite_Iio_nat (N : ℕ) : ({m : ℕ | m < N}).Finite :=
  Set.Finite.ofFinset (Finset.range N) fun x => by simp

theorem finite_subsets_lt (N : ℕ) :
    ({y : Pomega | y ⊆ {m | m < N}}).Finite :=
  (finite_Iio_nat N).finite_subsets

theorem isBooleanOpen_of_closed {U : Set Pomega} (hU : IsScottOpen U?) :
    IsBooleanOpen U := by
  have h := (IsBooleanOpen.ofOpen hU).compl
  rwa [compl_compl] at h

theorem isBooleanOpen_of_fcapg {U : Set Pomega} (hU : IsFcapG U) : IsBooleanOpen U :=
   by
  obtain ⟨C, O, hC, hO, rfl⟩ := hU
  exact (isBooleanOpen_of_closed hC).inter (IsBooleanOpen.ofOpen hO)

theorem isBooleanOpen_sUnion_finite {S : Set (Set Pomega)}
    (hS : S.Finite) (hU : ∀ U ∈ S, IsBooleanOpen U) :
    IsBooleanOpen (⋃₀ S) := by
  revert hU
  induction S, hS using Set.Finite.induction_on with
  | empty =>
    intro
    simp
    exact IsBooleanOpen.ofOpen isScottOpen_empty
  | insert ha hs ih =>
    intro hUV
    rename_i a s
    simp [Set.sUnion_insert]
    exact (hUV a (Set.mem_insert a s)).union
      (ih fun u hu => hUV u (Set.mem_insert_of_mem a hu))

theorem booleanMemE_ofOpen {U : Set Pomega} (hU : IsScottOpen U) :
    BooleanMemE U := by
  refine ⟨charOpen U, 1, {ofNat 0}, charOpen_isScottContinuous U, ?_, ?_, ?_, ?_⟩
  · intro x k hk
    rcases hk with ⟨rfl, _⟩
    exact Nat.zero_lt_one
  · exact Set.finite_singleton _
  · intro e he
    simp [ofNat] at he
    subst he
    intro k hk
    simp at hk
    subst hk
    exact Nat.zero_lt_one
```

```
  · ext x
    constructor
    · intro hx
      change charOpen U x = ofNat 0
      ext k
      constructor
      · intro ⟨hk0, n, hn, hne⟩
        simp [ofNat, hk0]
      · intro hk
        simp [ofNat] at hk
        subst hk
        obtain ⟨n, hn, hne⟩ := (mem_isScottOpen_iff hU).mp hx
        exact ⟨rfl, n, hn, hne⟩
    · intro hx
      have : 0 ∈ charOpen U x := by
        have : charOpen U x = ofNat 0 := hx
        rw [this]; simp [ofNat]
      obtain ⟨_, n, hn, hne⟩ := this
      exact isScottOpen_isUpper hU hne hn

theorem booleanMemE_union {U V : Set Pomega}
    (hU : BooleanMemE U) (hV : BooleanMemE V) : BooleanMemE (U ∪ V) := by
  obtain ⟨f, N, E, hf, hfb, hEf, hEsub, rfl⟩ := hU
  obtain ⟨g, M, F, hg, hgb, hFf, hFsub, rfl⟩ := hV
  let N' := 2 * max N M + 1
  let E' : Set Pomega :=
    {y | y ⊆ {m | m < N'} ∧ (evenPart y ∈ F ∨ oddPart y ∈ E)}
  refine ⟨oddEvenPack f g, N', E',
    oddEvenPack_isScottContinuous hf hg, ?_, ?_, ?_, ?_⟩
  · intro x
    exact oddEvenPack_subset (hfb x) (hgb x)
  · have hpow := finite_subsets_lt N'
    exact hpow.subset fun y hy => hy.1
  · intro y hy; exact hy.1
  · ext x
    constructor
    · intro hx
      refine ⟨oddEvenPack_subset (hfb x) (hgb x), ?_⟩
      rw [evenPart_oddEven, oddPart_oddEven]
      rcases hx with h | h
      · exact Or.inr h
      · exact Or.inl h
    · intro hx
      have hx' := hx.2
      rw [evenPart_oddEven, oddPart_oddEven] at hx'
      rcases hx' with h | h
      · exact Or.inr h
      · exact Or.inl h

theorem booleanMemE_inter {U V : Set Pomega}
    (hU : BooleanMemE U) (hV : BooleanMemE V) : BooleanMemE (U ∩ V) := by
  obtain ⟨f, N, E, hf, hfb, hEf, hEsub, rfl⟩ := hU
  obtain ⟨g, M, F, hg, hgb, hFf, hFsub, rfl⟩ := hV
  let N' := 2 * max N M + 1
  let E' : Set Pomega :=
    {y | y ⊆ {m | m < N'} ∧ evenPart y ∈ F ∧ oddPart y ∈ E}
```

```
  refine ⟨oddEvenPack f g, N', E',
    oddEvenPack_isScottContinuous hf hg, ?_, ?_, ?_, ?_⟩
  · intro x
    exact oddEvenPack_subset (hfb x) (hgb x)
  · have hpow := finite_subsets_lt N'
    exact hpow.subset fun y hy => hy.1
  · intro y hy; exact hy.1
  · ext x
    constructor
    · intro hx
      refine ⟨oddEvenPack_subset (hfb x) (hgb x), ?_⟩
      rw [evenPart_oddEven, oddPart_oddEven]
      exact ⟨hx.2, hx.1⟩
    · intro hx
      have : evenPart (oddEvenPack f g x) ∈ F ∧
          oddPart (oddEvenPack f g x) ∈ E := ⟨hx.2.1, hx.2.2⟩
      rw [evenPart_oddEven, oddPart_oddEven] at this
      exact ⟨this.2, this.1⟩

theorem booleanMemE_compl {U : Set Pomega} (hU : BooleanMemE U) :
    BooleanMemE U? := by
  obtain ⟨f, N, E, hf, hfb, hEf, hEsub, rfl⟩ := hU
  let E' : Set Pomega := {y | y ⊆ {m | m < N} ∧ y ∈ E}
  refine ⟨f, N, E', hf, hfb, ?_, fun y hy => hy.1, ?_⟩
  · exact (finite_subsets_lt N).subset fun y hy => hy.1
  · ext x
    constructor
    · intro hx
      exact ⟨hfb x, hx⟩
    · intro hx
      exact hx.2

/-- **Scott 1976, Theorem 6.6 (The ? theorem).** -/
theorem theorem_6_6 :
    ∀ U, U ∈ ScottB ↔
      ∃ f, IsScottContinuous f ∧ ∃ E : Set Pomega,
        E.Finite ∧ (∀ e ∈ E, e.Finite) ∧ U = {x | f x ∈ E} := by
  intro U
  constructor
  · intro hU
    have : BooleanMemE U := by
      induction hU with
      | ofOpen hU => exact booleanMemE_ofOpen hU
      | union _ _ ihU ihV => exact booleanMemE_union ihU ihV
      | inter _ _ ihU ihV => exact booleanMemE_inter ihU ihV
      | compl _ ih => exact booleanMemE_compl ih
    obtain ⟨f, N, E, hf, hfb, hEf, hEsub, rfl⟩ := this
    refine ⟨f, hf, E, hEf, fun e he => (finite_Iio_nat N).subset (hEsub e he), rfl⟩
  · rintro ⟨f, hf, E, hE, hEfin, rfl⟩
    have : {x | f x ∈ E} = ⋃₀ ((fun e => {x | f x = e}) '' E) := by
      ext x
      constructor
      · intro hx
        exact ⟨{y | f y = f x}, ⟨f x, hx, rfl⟩, rfl⟩
      · intro hx
        obtain ⟨_, ⟨e, he, rfl⟩, hx'⟩ := hx
```

```
      have hxeq : f x = e := hx'
      change f x ∈ E
      rwa [hxeq]
  rw [this]
  refine isBooleanOpen_sUnion_finite (hE.image _) fun W hW => ?_
  obtain ⟨e, heE, rfl⟩ := hW
  have hefin : e.Finite := hEfin e heE
  have hopen : IsScottOpen {x | e ⊆ f x} := by
    have : {x | e ⊆ f x} = ⋂ k ∈ hefin.toFinset, {x | k ∈ f x} := by
      ext x
      constructor
      · intro hx


        exact Set.mem_iInter₂.mpr fun k hk => hx (hefin.mem_toFinset.mp hk)
      · intro hx k hk
        exact Set.mem_iInter₂.mp hx k (hefin.mem_toFinset.mpr hk)
    rw [this, isScottOpen_iff_isOpen]
    exact isOpen_biInter_finset fun k _ =>
      (isScottOpen_iff_isOpen _).mp (mem_preimage_isScottOpen hf k)
  have hclosed : IsScottOpen {x | f x ⊆ e}? := by
    have : {x | f x ⊆ e}? = ⋃ k, {x | k ∈ e ∧ k ∈ f x} := by
      ext x
      constructor
      · intro hx
        obtain ⟨k, hk, hke⟩ := Set.not_subset.mp hx
        exact Set.mem_iUnion.mpr ⟨k, ⟨hke, hk⟩⟩
      · intro hx
        obtain ⟨k, hke, hk⟩ := Set.mem_iUnion.mp hx
        intro hsub
        exact hke (hsub hk)
    rw [this]
    exact isScottOpen_iUnion _ fun k => by
      by_cases hke : k ∈ e
      · have : {x | k ∈ e ∧ k ∈ f x} = (∅ : Set Pomega) := by
          ext x; simp [hke]
        rw [this]; exact isScottOpen_empty
      · have : {x | k ∈ e ∧ k ∈ f x} = {x | k ∈ f x} := by
          ext x; simp [hke]
        rw [this]; exact mem_preimage_isScottOpen hf k
  exact isBooleanOpen_of_fcapg ⟨{x | f x ⊆ e}, {x | e ⊆ f x}, hclosed, hopen, by
    ext x
    constructor
    · intro hx
      have : f x = e := by
        have : f x ∈ ({e} : Set Pomega) := hx
        simpa using this
      exact ⟨this.subset, this.symm.subset⟩
    · intro hx
      have : f x = e := subset_antisymm hx.1 hx.2
      simp [this]⟩

/-- **Scott 1976, Theorem 6.7, equalizer → `B_?`.**
`{x | f x = g x} = ⋂_n ({n ∈ f ∧ n ∈ g} ∪ {n ∈ f ∧ n ∈ g})`. -/
theorem theorem_6_7_of_equalizer :
    ∀ U, (∃ f g, IsScottContinuous f ∧ IsScottContinuous g ∧
      U = {x | f x = g x}) → IsBdelta U := by
```

```
  rintro U ⟨f, g, hf, hg, rfl⟩
  refine ⟨fun n => {x | n ∈ f x ∧ n ∈ g x} ∪ {x | n ∈ f x ∧ n ∈ g x}, ?_, ?_⟩
  · intro n
    refine (IsBooleanOpen.ofOpen
      (isScottOpen_inter (mem_preimage_isScottOpen hf n)
        (mem_preimage_isScottOpen hg n))).union
      (isBooleanOpen_of_closed ?_)
    have : {x | n ∈ f x ∧ n ∈ g x}? = {x | n ∈ f x} ∪ {x | n ∈ g x} := by
      ext x
      constructor
      · intro hx
        have : ¬ (n ∈ f x ∧ n ∈ g x) := hx
        by_cases hfx : n ∈ f x
        · exact Or.inl hfx
        · by_cases hgx : n ∈ g x
          · exact Or.inr hgx
          · exact (this ⟨hfx, hgx⟩).elim
      · intro hx h
        rcases hx with hf' | hg'
        · exact h.1 hf'
        · exact h.2 hg'
    rw [this]
    exact isScottOpen_union (mem_preimage_isScottOpen hf n)
      (mem_preimage_isScottOpen hg n)
  · ext x
    constructor
    · intro h
      refine Set.mem_iInter.mpr fun n => ?_
      have heq : f x = g x := h
      by_cases hn : n ∈ f x
      · exact Or.inl ⟨hn, by rwa [← heq]⟩
      · exact Or.inr ⟨hn, by simpa [heq] using hn⟩
    · intro hx
      ext n
      have hn := Set.mem_iInter.mp hx n
      constructor
      · intro hfx
        rcases hn with ⟨_, hng⟩ | ⟨hnf, _⟩
        · exact hng
        · exact (hnf hfx).elim
      · intro hgx
        rcases hn with ⟨hnf, _⟩ | ⟨_, hng⟩
        · exact hnf
        · exact (hng hgx).elim

/-- **Scott 1976, Theorem 6.7, converse packing.**
A countable intersection of equalizers is again an equalizer, via
`pairIndexed`. -/
theorem theorem_6_7_iInter_equalizer (f g : ℕ → Pomega → Pomega)
    (hf : ∀ n, IsScottContinuous (f n)) (hg : ∀ n, IsScottContinuous (g n)) :
    ∃ F G, IsScottContinuous F ∧ IsScottContinuous G ∧
      {x | ∀ n, f n x = g n x} = {x | F x = G x} :=
  ⟨pairIndexed f, pairIndexed g,
    pairIndexed_isScottContinuous f hf, pairIndexed_isScottContinuous g hg, by
    ext x
    constructor
```

```
    · intro h
      ext k
      constructor
      · intro ⟨n, m, hk, hm⟩
        exact ⟨n, m, hk, by rwa [← h n]⟩
      · intro ⟨n, m, hk, hm⟩
        exact ⟨n, m, hk, by rwa [h n]⟩
    · intro h n
      ext m
      constructor
      · intro hm
        have : pair n m ∈ pairIndexed g x := by
          have : pair n m ∈ pairIndexed f x := ⟨n, m, rfl, hm⟩
          rwa [h] at this
        obtain ⟨n', m', heq, hm'⟩ := this
        obtain ⟨rfl, rfl⟩ := pair_inj heq
        exact hm'
      · intro hm
        have : pair n m ∈ pairIndexed f x := by
          have : pair n m ∈ pairIndexed g x := ⟨n, m, rfl, hm⟩
          rwa [← h] at this
        obtain ⟨n', m', heq, hm'⟩ := this
        obtain ⟨rfl, rfl⟩ := pair_inj heq
        exact hm'⟩

/-- Every `B_?` written as a countable intersection of equalizers is itself
an equalizer of continuous maps. -/
theorem theorem_6_7_converse_of_equalizers (f g : ℕ → Pomega → Pomega)
    (hf : ∀ n, IsScottContinuous (f n)) (hg : ∀ n, IsScottContinuous (g n))
    {U : Set Pomega} (hU : U = {x | ∀ n, f n x = g n x}) :
    ∃ F G, IsScottContinuous F ∧ IsScottContinuous G ∧ U = {x | F x = G x} := by
  obtain ⟨F, G, hF, hG, h⟩ := theorem_6_7_iInter_equalizer f g hf hg
  exact ⟨F, G, hF, hG, hU.trans h⟩

/-- Union is Scott-continuous in the left argument. -/
theorem union_isScottContinuous_left (y : Pomega) :
    IsScottContinuous (fun x => (x ∪ y : Pomega)) := by
  constructor
  intro x
  ext k
  constructor
  · intro hk
    rcases hk with h | h
    · exact mem_scottUnion.mpr ⟨2 ^ k, by simp [e_pow2, Set.singleton_subset_iff, h],
        Or.inl (by simp [e_pow2])⟩
    · exact mem_scottUnion.mpr ⟨0, by simp [e_zero], Or.inr h⟩
  · intro hk
    obtain ⟨n, hn, hkn⟩ := mem_scottUnion.mp hk
    rcases hkn with h | h
    · exact Or.inl (hn h)
    · exact Or.inr h

/-- Union is Scott-continuous in the right argument. -/
theorem union_isScottContinuous_right (x : Pomega) :
    IsScottContinuous (fun y => (x ∪ y : Pomega)) := by
  constructor
```

```
  intro y
  ext k
  constructor
  · intro hk
    rcases hk with h | h
    · exact mem_scottUnion.mpr ⟨0, by simp [e_zero], Or.inl h⟩
    · exact mem_scottUnion.mpr ⟨2 ^ k, by simp [e_pow2, Set.singleton_subset_iff, h],
        Or.inr (by simp [e_pow2])⟩
  · intro hk
    obtain ⟨n, hn, hkn⟩ := mem_scottUnion.mp hk
    rcases hkn with h | h
    · exact Or.inl h
    · exact Or.inr (hn h)

theorem closedToZero_isScottContinuous (C : Set Pomega) :
    IsScottContinuous (closedToZero C) :=
  theorem_1_3.tuple (f := fun a b => (a ∪ b : Pomega))
    union_isScottContinuous_left union_isScottContinuous_right
    (const_isScottContinuous (ofNat 0)) (charTop_isScottContinuous C?)

theorem uvU_mono_left {x x' y : Pomega} (h : x ⊆ x') : uvU x y ⊆ uvU x' y := by
  intro k hk
  rcases hk with ⟨rfl, hne⟩ | ⟨rfl, h1⟩
  · exact Or.inl ⟨rfl, hne.imp_left fun ⟨n, hn⟩ => ⟨n, h hn⟩⟩
  · rcases h1 with h10 | ⟨⟨n, hn⟩, h0⟩
    · exact Or.inr ⟨rfl, Or.inl ⟨h h10.1, h10.2⟩⟩
    · exact Or.inr ⟨rfl, Or.inr ⟨⟨n, h hn⟩, h0⟩⟩

theorem uvU_mono_right {x y y' : Pomega} (h : y ⊆ y') : uvU x y ⊆ uvU x y' := by
  intro k hk
  rcases hk with ⟨rfl, hne⟩ | ⟨rfl, h1⟩
  · exact Or.inl ⟨rfl, hne.imp_right fun ⟨n, hn⟩ => ⟨n, h hn⟩⟩
  · rcases h1 with ⟨h0, ⟨n, hn⟩⟩ | h10
    · exact Or.inr ⟨rfl, Or.inl ⟨h0, ⟨n, h hn⟩⟩⟩
    · exact Or.inr ⟨rfl, Or.inr ⟨h10.1, h h10.2⟩⟩

theorem uvV_mono_left {x x' y : Pomega} (h : x ⊆ x') : uvV x y ⊆ uvV x' y := by
  intro k hk
  rcases hk with hk0 | ⟨hk1, hpos⟩ | ⟨h0, ⟨n, hn⟩, h0y, hpy⟩
  · exact Or.inl hk0
  · exact Or.inr (Or.inl ⟨hk1, hpos.imp_left fun ⟨n, hn⟩ => ⟨n, h hn⟩⟩)
  · exact Or.inr (Or.inr ⟨h h0, ⟨n, h hn⟩, h0y, hpy⟩)

theorem uvV_mono_right {x y y' : Pomega} (h : y ⊆ y') : uvV x y ⊆ uvV x y' := by
  intro k hk
  rcases hk with hk0 | ⟨hk1, hpos⟩ | ⟨h0, hpx, h0y, ⟨n, hn⟩⟩
  · exact Or.inl hk0
  · exact Or.inr (Or.inl ⟨hk1, hpos.imp_right fun ⟨n, hn⟩ => ⟨n, h hn⟩⟩)
  · exact Or.inr (Or.inr ⟨h0, hpx, h h0y, ⟨n, h hn⟩⟩)

theorem uvU_isScottContinuous_left (y : Pomega) :
    IsScottContinuous (fun x => uvU x y) := by
  constructor
  intro x
  ext k
  constructor
```

```
  · intro hk
    rcases hk with ⟨rfl, hne⟩ | ⟨rfl, h1⟩
    · rcases hne with ⟨n, hn⟩ | hy
      · exact mem_scottUnion.mpr ⟨2 ^ n, by simp [e_pow2, Set.singleton_subset_iff, hn
    ],
          Or.inl ⟨rfl, Or.inl ⟨n, by simp [e_pow2]⟩⟩⟩
      · exact mem_scottUnion.mpr ⟨0, by simp [e_zero], Or.inl ⟨rfl, Or.inr hy⟩⟩
    · rcases h1 with ⟨h0, hp⟩ | ⟨⟨n, hn⟩, h0⟩
      · exact mem_scottUnion.mpr ⟨1, by simp [e_one, h0],
          Or.inr ⟨rfl, Or.inl ⟨by simp [e_one], hp⟩⟩⟩
      · exact mem_scottUnion.mpr ⟨2 ^ (n + 1), by simp [e_pow2, Set.
    singleton_subset_iff, hn],
          Or.inr ⟨rfl, Or.inr ⟨⟨n, by simp [e_pow2]⟩, h0⟩⟩⟩
  · intro hk
    obtain ⟨m, hm, hkm⟩ := mem_scottUnion.mp hk
    exact uvU_mono_left hm hkm

theorem uvU_isScottContinuous_right (x : Pomega) :
    IsScottContinuous (fun y => uvU x y) := by
  constructor
  intro y
  ext k
  constructor
  · intro hk
    rcases hk with ⟨rfl, hne⟩ | ⟨rfl, h1⟩
    · rcases hne with hx | ⟨n, hn⟩
      · exact mem_scottUnion.mpr ⟨0, by simp [e_zero], Or.inl ⟨rfl, Or.inl hx⟩⟩
      · exact mem_scottUnion.mpr ⟨2 ^ n, by simp [e_pow2, Set.singleton_subset_iff, hn
    ],
          Or.inl ⟨rfl, Or.inr ⟨n, by simp [e_pow2]⟩⟩⟩
    · rcases h1 with ⟨h0, ⟨n, hn⟩⟩ | ⟨hp, h0⟩
      · exact mem_scottUnion.mpr ⟨2 ^ (n + 1), by simp [e_pow2, Set.
    singleton_subset_iff, hn],
          Or.inr ⟨rfl, Or.inl ⟨h0, ⟨n, by simp [e_pow2]⟩⟩⟩⟩
      · exact mem_scottUnion.mpr ⟨1, by simp [e_one, h0],
          Or.inr ⟨rfl, Or.inr ⟨hp, by simp [e_one]⟩⟩⟩
  · intro hk
    obtain ⟨m, hm, hkm⟩ := mem_scottUnion.mp hk
    exact uvU_mono_right hm hkm

theorem uvV_isScottContinuous_left (y : Pomega) :
    IsScottContinuous (fun x => uvV x y) := by
  constructor
  intro x
  ext k
  constructor
  · intro hk
    rcases hk with hk0 | ⟨rfl, hpos⟩ | ⟨h0, ⟨n, hn⟩, h0y, hpy⟩
    · exact mem_scottUnion.mpr ⟨0, by simp [e_zero], Or.inl hk0⟩
    · rcases hpos with ⟨n, hn⟩ | hy
      · exact mem_scottUnion.mpr ⟨2 ^ (n + 1), by simp [e_pow2, Set.
    singleton_subset_iff, hn],
          Or.inr (Or.inl ⟨rfl, Or.inl ⟨n, by simp [e_pow2]⟩⟩)⟩
      · exact mem_scottUnion.mpr ⟨0, by simp [e_zero],
          Or.inr (Or.inl ⟨rfl, Or.inr hy⟩)⟩
    · refine mem_scottUnion.mpr ⟨1 ||| 2 ^ (n + 1), e_or_of_subset
```

```
        (by simp [e_one, Set.singleton_subset_iff, h0])
        (by simp [e_pow2, Set.singleton_subset_iff, hn]), ?_⟩
      refine Or.inr (Or.inr ⟨?_, ⟨n, ?_⟩, h0y, hpy⟩)
      · simp [e_or, e_one]
      · simp [e_or, e_pow2]
  · intro hk
    obtain ⟨m, hm, hkm⟩ := mem_scottUnion.mp hk
    exact uvV_mono_left hm hkm

theorem uvV_isScottContinuous_right (x : Pomega) :
    IsScottContinuous (fun y => uvV x y) := by
  constructor
  intro y
  ext k
  constructor
  · intro hk
    rcases hk with hk0 | ⟨rfl, hpos⟩ | ⟨h0, hpx, h0y, ⟨n, hn⟩⟩
    · exact mem_scottUnion.mpr ⟨0, by simp [e_zero], Or.inl hk0⟩
    · rcases hpos with hx | ⟨n, hn⟩
      · exact mem_scottUnion.mpr ⟨0, by simp [e_zero],
          Or.inr (Or.inl ⟨rfl, Or.inl hx⟩)⟩
      · exact mem_scottUnion.mpr ⟨2 ^ (n + 1), by simp [e_pow2, Set.
    singleton_subset_iff, hn],
          Or.inr (Or.inl ⟨rfl, Or.inr ⟨n, by simp [e_pow2]⟩⟩)⟩
    · refine mem_scottUnion.mpr ⟨1 ||| 2 ^ (n + 1), e_or_of_subset
        (by simp [e_one, Set.singleton_subset_iff, h0y])
        (by simp [e_pow2, Set.singleton_subset_iff, hn]), ?_⟩
      refine Or.inr (Or.inr ⟨h0, hpx, ?_, ⟨n, ?_⟩⟩)
      · simp [e_or, e_one]
      · simp [e_or, e_pow2]
  · intro hk
    obtain ⟨m, hm, hkm⟩ := mem_scottUnion.mp hk
    exact uvV_mono_right hm hkm

theorem bot_ne_ofNat_zero : botElem = ofNat 0 := by
  intro h
  have : (0 : ℕ) ∈ botElem := by rw [h]; simp [ofNat]
  exact (mem_botElem 0).mp this

theorem top_ne_ofNat_zero : topElem = ofNat 0 := by
  intro h
  have : (1 : ℕ) ∈ ofNat 0 := by rw [← h]; exact Set.mem_univ 1
  simp [ofNat] at this

theorem zeroPrime_ne_ofNat_zero : zeroPrime = ofNat 0 := by
  intro h
  have : (1 : ℕ) ∈ ofNat 0 := by
    have : (1 : ℕ) ∈ zeroPrime := Or.inr rfl
    rwa [h] at this
  simp [ofNat] at this

theorem zeroPrime_ne_top : zeroPrime = topElem := by
  intro h
  have : (2 : ℕ) ∈ zeroPrime := by rw [h]; exact Set.mem_univ 2
  simp [zeroPrime, ofNat] at this
```

```
theorem uvU_bot_bot : uvU botElem botElem = botElem := by
  ext k
  simp [uvU, botElem]

theorem uvV_bot_bot : uvV botElem botElem = ofNat 0 := by
  ext k
  constructor
  · intro hk
    rcases hk with hk0 | ⟨hk1, hpos⟩ | htop
    · simpa [ofNat] using hk0
    · rcases hpos with ⟨n, hn⟩ | ⟨n, hn⟩ <;> exact ((mem_botElem (n + 1)).mp hn).elim
    · exact ((mem_botElem 0).mp htop.1).elim
  · intro hk
    simp [ofNat] at hk
    exact Or.inl hk

theorem ofNat_zero_nonempty : (ofNat 0).Nonempty :=
  ⟨0, rfl⟩

theorem ofNat_zero_not_pos : ¬ ∃ n, n + 1 ∈ ofNat 0 := by
  intro ⟨n, hn⟩
  simp [ofNat] at hn

theorem uvU_bot_zero : uvU botElem (ofNat 0) = ofNat 0 := by
  ext k
  constructor
  · intro hk
    rcases hk with ⟨hk0, _⟩ | ⟨hk1, h1⟩
    · simpa [ofNat] using hk0
    · rcases h1 with ⟨h0, _⟩ | ⟨⟨n, hn⟩, _⟩
      · exact ((mem_botElem 0).mp h0).elim
      · exact ((mem_botElem (n + 1)).mp hn).elim
  · intro hk
    simp [ofNat] at hk
    exact Or.inl ⟨hk, Or.inr ofNat_zero_nonempty⟩

theorem uvV_bot_zero : uvV botElem (ofNat 0) = ofNat 0 := by
  ext k
  constructor
  · intro hk
    rcases hk with hk0 | ⟨hk1, hpos⟩ | htop
    · simpa [ofNat] using hk0
    · rcases hpos with ⟨n, hn⟩ | hp
      · exact ((mem_botElem (n + 1)).mp hn).elim
      · exact (ofNat_zero_not_pos hp).elim
    · exact ((mem_botElem 0).mp htop.1).elim
  · intro hk
    simp [ofNat] at hk
    exact Or.inl hk

theorem uvU_zero_bot : uvU (ofNat 0) botElem = ofNat 0 := by
  ext k
  constructor
  · intro hk
    rcases hk with ⟨hk0, _⟩ | ⟨_, h1⟩
    · simpa [ofNat] using hk0
```

```
    · rcases h1 with ⟨_, ⟨n, hn⟩⟩ | ⟨⟨n, hn⟩, _⟩
      · exact ((mem_botElem (n + 1)).mp hn).elim
      · exact (ofNat_zero_not_pos ⟨n, hn⟩).elim
  · intro hk
    simp [ofNat] at hk
    exact Or.inl ⟨hk, Or.inl ofNat_zero_nonempty⟩

theorem uvV_zero_bot : uvV (ofNat 0) botElem = ofNat 0 := by
  ext k
  constructor
  · intro hk
    rcases hk with hk0 | ⟨_, hpos⟩ | htop
    · simpa [ofNat] using hk0
    · rcases hpos with hp | ⟨n, hn⟩
      · exact (ofNat_zero_not_pos hp).elim
      · exact ((mem_botElem (n + 1)).mp hn).elim
    · exact ((mem_botElem 0).mp htop.2.2.1).elim
  · intro hk
    simp [ofNat] at hk
    exact Or.inl hk

theorem uvU_zero_zero : uvU (ofNat 0) (ofNat 0) = ofNat 0 := by
  ext k
  constructor
  · intro hk
    rcases hk with ⟨hk0, _⟩ | ⟨_, h1⟩
    · simpa [ofNat] using hk0


    · rcases h1 with ⟨_, hp⟩ | ⟨hp, _⟩
      · exact (ofNat_zero_not_pos hp).elim
      · exact (ofNat_zero_not_pos hp).elim
  · intro hk
    simp [ofNat] at hk
    exact Or.inl ⟨hk, Or.inl ofNat_zero_nonempty⟩

theorem uvV_zero_zero : uvV (ofNat 0) (ofNat 0) = ofNat 0 := by
  ext k
  constructor
  · intro hk
    rcases hk with hk0 | ⟨_, hpos⟩ | htop
    · simpa [ofNat] using hk0
    · rcases hpos with hp | hp
      · exact (ofNat_zero_not_pos hp).elim
      · exact (ofNat_zero_not_pos hp).elim
    · exact (ofNat_zero_not_pos htop.2.1).elim
  · intro hk
    simp [ofNat] at hk
    exact Or.inl hk

theorem top_pos : ∃ n, n + 1 ∈ topElem :=
  ⟨0, Set.mem_univ 1⟩

theorem uvU_bot_top : uvU botElem topElem = ofNat 0 := by
  ext k
  constructor
  · intro hk
```

```
    rcases hk with ⟨hk0, _⟩ | ⟨_, h1⟩
    · simpa [ofNat] using hk0
    · rcases h1 with ⟨h0, _⟩ | ⟨⟨n, hn⟩, _⟩
      · exact ((mem_botElem 0).mp h0).elim
      · exact ((mem_botElem (n + 1)).mp hn).elim
  · intro hk
    simp [ofNat] at hk
    exact Or.inl ⟨hk, Or.inr ⟨0, Set.mem_univ 0⟩⟩

theorem uvV_bot_top : uvV botElem topElem = zeroPrime := by
  ext k
  constructor
  · intro hk
    rcases hk with hk0 | ⟨hk1, _⟩ | htop
    · exact Or.inl hk0
    · exact Or.inr hk1
    · exact ((mem_botElem 0).mp htop.1).elim
  · intro hk
    rcases hk with hk0 | hk1
    · exact Or.inl hk0
    · exact Or.inr (Or.inl ⟨hk1, Or.inr top_pos⟩)

theorem uvU_top_bot : uvU topElem botElem = ofNat 0 := by
  ext k
  constructor
  · intro hk
    rcases hk with ⟨hk0, _⟩ | ⟨_, h1⟩
    · simpa [ofNat] using hk0
    · rcases h1 with ⟨_, ⟨n, hn⟩⟩ | ⟨_, h0⟩
      · exact ((mem_botElem (n + 1)).mp hn).elim
      · exact ((mem_botElem 0).mp h0).elim
  · intro hk
    simp [ofNat] at hk
    exact Or.inl ⟨hk, Or.inl ⟨0, Set.mem_univ 0⟩⟩

theorem uvV_top_bot : uvV topElem botElem = zeroPrime := by
  ext k
  constructor
  · intro hk
    rcases hk with hk0 | ⟨hk1, _⟩ | htop
    · exact Or.inl hk0
    · exact Or.inr hk1
    · exact ((mem_botElem 0).mp htop.2.2.1).elim
  · intro hk
    rcases hk with hk0 | hk1
    · exact Or.inl hk0
    · exact Or.inr (Or.inl ⟨hk1, Or.inl top_pos⟩)

theorem uvU_zero_top : uvU (ofNat 0) topElem = zeroPrime := by
  ext k
  constructor
  · intro hk
    rcases hk with ⟨hk0, _⟩ | ⟨hk1, _⟩
    · exact Or.inl hk0
    · exact Or.inr hk1
  · intro hk
```

```
    rcases hk with hk0 | hk1
    · exact Or.inl ⟨hk0, Or.inr ⟨0, Set.mem_univ 0⟩⟩
    · exact Or.inr ⟨hk1, Or.inl ⟨rfl, top_pos⟩⟩

theorem uvV_zero_top : uvV (ofNat 0) topElem = zeroPrime := by
  ext k
  constructor
  · intro hk
    rcases hk with hk0 | ⟨hk1, _⟩ | htop
    · exact Or.inl hk0
    · exact Or.inr hk1
    · exact (ofNat_zero_not_pos htop.2.1).elim
  · intro hk
    rcases hk with hk0 | hk1
    · exact Or.inl hk0
    · exact Or.inr (Or.inl ⟨hk1, Or.inr top_pos⟩)

theorem uvU_top_zero : uvU topElem (ofNat 0) = zeroPrime := by
  ext k
  constructor
  · intro hk
    rcases hk with ⟨hk0, _⟩ | ⟨hk1, _⟩
    · exact Or.inl hk0
    · exact Or.inr hk1
  · intro hk
    rcases hk with hk0 | hk1
    · exact Or.inl ⟨hk0, Or.inl ⟨0, Set.mem_univ 0⟩⟩
    · exact Or.inr ⟨hk1, Or.inr ⟨top_pos, rfl⟩⟩

theorem uvV_top_zero : uvV topElem (ofNat 0) = zeroPrime := by
  ext k
  constructor
  · intro hk
    rcases hk with hk0 | ⟨hk1, _⟩ | htop
    · exact Or.inl hk0
    · exact Or.inr hk1
    · exact (ofNat_zero_not_pos htop.2.2.2).elim
  · intro hk
    rcases hk with hk0 | hk1
    · exact Or.inl hk0
    · exact Or.inr (Or.inl ⟨hk1, Or.inl top_pos⟩)

theorem uvU_top_top : uvU topElem topElem = zeroPrime := by
  ext k
  constructor
  · intro hk
    rcases hk with ⟨hk0, _⟩ | ⟨hk1, _⟩
    · exact Or.inl hk0
    · exact Or.inr hk1
  · intro hk
    rcases hk with hk0 | hk1
    · exact Or.inl ⟨hk0, Or.inl ⟨0, Set.mem_univ 0⟩⟩
    · exact Or.inr ⟨hk1, Or.inl ⟨Set.mem_univ 0, top_pos⟩⟩

theorem uvV_top_top : uvV topElem topElem = topElem := by
  ext k
```

```
  constructor
  · intro _; exact Set.mem_univ k
  · intro _
    exact Or.inr (Or.inr ⟨Set.mem_univ 0, top_pos, Set.mem_univ 0, top_pos⟩)

/-- On `{⊥, 0, ⊤}`, `u(x)(y) = v(x)(y)` iff `x = 0` or `y = 0`. -/
theorem uv_eq_iff {x y : Pomega} (hx : ThreeValued x) (hy : ThreeValued y) :
    uvU x y = uvV x y ↔ x = ofNat 0 ∨ y = ofNat 0 := by
  rcases hx.out with rfl | rfl | rfl <;> rcases hy.out with rfl | rfl | rfl
  · simp [uvU_bot_bot, uvV_bot_bot, bot_ne_ofNat_zero]
  · simp [uvU_bot_zero, uvV_bot_zero]
  · constructor
    · intro h
      rw [uvU_bot_top, uvV_bot_top] at h
      exact (zeroPrime_ne_ofNat_zero h.symm).elim
    · intro h
      rcases h with h | h
      · exact (bot_ne_ofNat_zero h).elim
      · exact (top_ne_ofNat_zero h).elim
  · simp [uvU_zero_bot, uvV_zero_bot]
  · simp [uvU_zero_zero, uvV_zero_zero]
  · simp [uvU_zero_top, uvV_zero_top]
  · constructor
    · intro h
      rw [uvU_top_bot, uvV_top_bot] at h
      exact (zeroPrime_ne_ofNat_zero h.symm).elim
    · intro h
      rcases h with h | h
      · exact (top_ne_ofNat_zero h).elim
      · exact (bot_ne_ofNat_zero h).elim
  · simp [uvU_top_zero, uvV_top_zero]
  · constructor
    · intro h
      rw [uvU_top_top, uvV_top_top] at h
      exact (zeroPrime_ne_top h).elim
    · intro h
      rcases h with h | h <;> exact (top_ne_ofNat_zero h).elim

theorem charOpen_three (U : Set Pomega) (x : Pomega) :
    ThreeValued (charOpen U x) := by
  constructor
  by_cases h : 0 ∈ charOpen U x
  · refine Or.inr (Or.inl ?_)
    ext k
    constructor
    · intro ⟨hk0, _⟩; simp [ofNat, hk0]
    · intro hk; simp [ofNat] at hk; subst hk; exact h
  · refine Or.inl ?_
    ext k
    constructor
    · intro hk
      obtain ⟨rfl, n, hn, hne⟩ := hk
      exact h ⟨rfl, n, hn, hne⟩
    · intro hk
      exact ((mem_botElem k).mp hk).elim
```

```
theorem closedToZero_three {C : Set Pomega} (hC : IsScottOpen C?) (x : Pomega) :
    ThreeValued (closedToZero C x) := by
  constructor
  by_cases hx : x ∈ C
  · refine Or.inr (Or.inl ?_)
    have : charTop C? x = botElem := (charTop_eq_bot_of_closed hC x).mpr hx
    ext k
    simp [closedToZero, this, ofNat, botElem]
  · refine Or.inr (Or.inr ?_)
    have : charTop C? x = topElem := (charTop_eq_top hC x).mpr hx
    ext k
    simp [closedToZero, this, ofNat, topElem]

theorem charOpen_eq_zero {U : Set Pomega} (hU : IsScottOpen U) (x : Pomega) :
    charOpen U x = ofNat 0 ↔ x ∈ U := by
  constructor
  · intro h
    have : 0 ∈ charOpen U x := by rw [h]; simp [ofNat]
    obtain ⟨_, n, hn, hne⟩ := this
    exact isScottOpen_isUpper hU hne hn
  · intro hx
    ext k
    constructor
    · intro ⟨hk0, _⟩; simp [ofNat, hk0]
    · intro hk
      simp [ofNat] at hk
      subst hk
      obtain ⟨n, hn, hne⟩ := (mem_isScottOpen_iff hU).mp hx
      exact ⟨rfl, n, hn, hne⟩

theorem closedToZero_eq_zero {C : Set Pomega} (hC : IsScottOpen C?) (x : Pomega) :
    closedToZero C x = ofNat 0 ↔ x ∈ C := by
  constructor
  · intro h
    by_contra hx
    have htop : charTop C? x = topElem := (charTop_eq_top hC x).mpr hx
    have : closedToZero C x = topElem := by
      ext k
      simp [closedToZero, htop]
    exact top_ne_ofNat_zero (this.symm.trans h)
  · intro hx
    have : charTop C? x = botElem := (charTop_eq_bot_of_closed hC x).mpr hx
    ext k
    simp [closedToZero, this, ofNat, botElem]

theorem isScottOpen_not_subset {f : Pomega → Pomega} (hf : IsScottContinuous f)
    (e : Pomega) : IsScottOpen {x | ¬ f x ⊆ e} := by
  have : {x | ¬ f x ⊆ e} = ⋃ k, {x | k ∈ e ∧ k ∈ f x} := by
    ext x
    constructor
    · intro hx
      obtain ⟨k, hk, hke⟩ := Set.not_subset.mp hx
      exact Set.mem_iUnion.mpr ⟨k, ⟨hke, hk⟩⟩
    · intro hx
      obtain ⟨k, hke, hk⟩ := Set.mem_iUnion.mp hx
      intro hsub
```

```
      exact hke (hsub hk)
  rw [this]
  exact isScottOpen_iUnion _ fun k => by
    by_cases hke : k ∈ e
    · have : {x | k ∈ e ∧ k ∈ f x} = (∅ : Set Pomega) := by
        ext x; simp [hke]
      rw [this]; exact isScottOpen_empty
    · have : {x | k ∈ e ∧ k ∈ f x} = {x | k ∈ f x} := by
        ext x; simp [hke]
      rw [this]; exact mem_preimage_isScottOpen hf k

theorem isScottOpen_subset_preimage {f : Pomega → Pomega} (hf : IsScottContinuous f)
    {e : Pomega} (he : e.Finite) : IsScottOpen {x | e ⊆ f x} := by
  have : {x | e ⊆ f x} = ⋂ k ∈ he.toFinset, {x | k ∈ f x} := by
    ext x
    constructor
    · intro hx
      exact Set.mem_iInter₂.mpr fun k hk => hx (he.mem_toFinset.mp hk)
    · intro hx k hk
      exact Set.mem_iInter₂.mp hx k (he.mem_toFinset.mpr hk)
  rw [this, isScottOpen_iff_isOpen]
  exact isOpen_biInter_finset fun k _ =>
    (isScottOpen_iff_isOpen _).mp (mem_preimage_isScottOpen hf k)

theorem ne_eq_open_union_closed {f : Pomega → Pomega} (hf : IsScottContinuous f)
    {e : Pomega} (he : e.Finite) :
    {x | f x = e} = {x | ¬ f x ⊆ e} ∪ {x | ¬ e ⊆ f x} ∧
      IsScottOpen {x | ¬ f x ⊆ e} ∧
      IsScottOpen {x | ¬ e ⊆ f x}? := by
  refine ⟨?_, isScottOpen_not_subset hf e, ?_⟩
  · ext x
    constructor
    · intro hne
      by_cases hsub : f x ⊆ e
      · refine Or.inr fun hsup => hne (subset_antisymm hsub hsup)
      · exact Or.inl hsub
    · intro h heq
      rcases h with h | h
      · exact h heq.subset
      · exact h heq.symm.subset
  · have : {x | ¬ e ⊆ f x}? = {x | e ⊆ f x} := by
      ext x; simp
    rw [this]
    exact isScottOpen_subset_preimage hf he

/-- Every Boolean set is a countable intersection of `(open ∪ closed)` sets,
    by writing the complement in the Theorem 6.6 form `{f ∈ E}` and taking
    De Morgan duals. Extra indices are padded by `univ ∪ ∅`. -/
theorem isBooleanOpen_as_iInter_open_union_closed {U : Set Pomega}
    (hU : IsBooleanOpen U) :
    ∃ (O C : ℕ → Set Pomega),
      (∀ n, IsScottOpen (O n)) ∧ (∀ n, IsScottOpen (C n)?) ∧
        U = ⋂ n, O n ∪ C n := by
  have hUc : U? ∈ ScottB := hU.compl
  obtain ⟨f, hf, E, hE, hEfin, hEeq⟩ := (theorem_6_6 U?).mp hUc
  have hrec : ∀ {S : Set Pomega} (hS : S.Finite),
```

```
    (∀ e ∈ S, e.Finite) →
    ∃ O C : ℕ → Set Pomega,
      (∀ n, IsScottOpen (O n)) ∧ (∀ n, IsScottOpen (C n)?) ∧
        {x | f x ∈ S} = ⋂ n, O n ∪ C n := by
  intro S hS
  induction S, hS using Set.Finite.induction_on with
  | empty =>
    intro
    refine ⟨fun _ => Set.univ, fun _ => ∅, fun _ => isScottOpen_univ,
      fun _ => by rw [Set.compl_empty]; exact isScottOpen_univ, ?_⟩
    ext x
    simp
  | insert ha hs ih =>
    intro hfin
    rename_i a s
    obtain ⟨Os, Cs, hOs, hCs, hseq⟩ :=
      ih fun e he => hfin e (Set.mem_insert_of_mem a he)
    have haFin : a.Finite := hfin a (Set.mem_insert a s)
    obtain ⟨hslice, hOa, hCa⟩ := ne_eq_open_union_closed hf haFin
    refine ⟨fun n => match n with
      | 0 => {x | ¬ f x ⊆ a}
      | _n + 1 => Os _n,
      fun n => match n with
      | 0 => {x | ¬ a ⊆ f x}
      | _n + 1 => Cs _n, ?_, ?_, ?_⟩
    · intro n
      cases n with
      | zero => exact hOa
      | succ n => exact hOs n
    · intro n
      cases n with
      | zero => exact hCa
      | succ n => exact hCs n
    · have hUeq : {x | f x ∈ insert a s} =
          ({x | f x = a} ∩ {x | f x ∈ s}) := by
        ext x
        simp [Set.mem_insert_iff]
      rw [hUeq, hslice, hseq]
      ext x
      constructor
      · intro ⟨ha', hs'⟩
        refine Set.mem_iInter.mpr fun n => ?_
        cases n with
        | zero => exact ha'
        | succ n => exact Set.mem_iInter.mp hs' n
      · intro hx
        exact ⟨Set.mem_iInter.mp hx 0,
          Set.mem_iInter.mpr fun n => Set.mem_iInter.mp hx (n + 1)⟩
obtain ⟨O, C, hO, hC, hOC⟩ := hrec hE hEfin
refine ⟨O, C, hO, hC, ?_⟩
have hUeq : U = {x | f x ∈ E} := by
  ext x
  constructor
  · intro hxU hfxE
    have hxUc : x ∈ U? := by
      rw [hEeq]
```

```
        exact hfxE
      exact hxUc hxU
    · intro hx
      by_contra hxU
      apply hx
      change x ∈ {x | f x ∈ E}
      rwa [← hEeq]
  rw [hUeq, hOC]

/-- **Scott 1976, Theorem 6.7, `B_?` → equalizer.**
Each Boolean slice is a countable intersection of `(open ∪ closed)` sets;
each such union is `{fₙ = 0} ∪ {gₙ = 0}` with three-valued continuous `fₙ, gₙ`;
Scott's tables pack that union as an equalizer; `pairIndexed` packs the
countable intersection. -/
theorem theorem_6_7_of_bdelta {U : Set Pomega} (hU : IsBdelta U) :
    ∃ f g, IsScottContinuous f ∧ IsScottContinuous g ∧ U = {x | f x = g x} := by
  obtain ⟨Bs, hB, rfl⟩ := hU
  have hslice : ∀ n, ∃ O C : ℕ → Set Pomega,
      (∀ k, IsScottOpen (O k)) ∧ (∀ k, IsScottOpen (C k)?) ∧
        Bs n = ⋂ k, O k ∪ C k :=
    fun n => isBooleanOpen_as_iInter_open_union_closed (hB n)
  choose Os Cs hOs hCs hEq using hslice
  let O (m : ℕ) : Set Pomega := Os (unpair m).1 (unpair m).2
  let C (m : ℕ) : Set Pomega := Cs (unpair m).1 (unpair m).2
  have hO : ∀ m, IsScottOpen (O m) := fun m => hOs (unpair m).1 (unpair m).2
  have hC : ∀ m, IsScottOpen (C m)? := fun m => hCs (unpair m).1 (unpair m).2
  have hUC : (⋂ n, Bs n) = ⋂ m, O m ∪ C m := by
    ext x
    constructor
    · intro hx
      refine Set.mem_iInter.mpr fun m => ?_
      have hx' := Set.mem_iInter.mp hx (unpair m).1
      rw [hEq] at hx'
      exact Set.mem_iInter.mp hx' (unpair m).2
    · intro hx
      refine Set.mem_iInter.mpr fun n => ?_


      rw [hEq]
      refine Set.mem_iInter.mpr fun k => ?_
      simpa [O, C, unpair_pair] using Set.mem_iInter.mp hx (pair n k)
  let f : ℕ → Pomega → Pomega := fun m x => charOpen (O m) x
  let g : ℕ → Pomega → Pomega := fun m x => closedToZero (C m) x
  have hf : ∀ m, IsScottContinuous (f m) :=
    fun m => charOpen_isScottContinuous (O m)
  have hg : ∀ m, IsScottContinuous (g m) :=
    fun m => closedToZero_isScottContinuous (C m)
  let F : ℕ → Pomega → Pomega := fun m x => uvU (f m x) (g m x)
  let G : ℕ → Pomega → Pomega := fun m x => uvV (f m x) (g m x)
  have hF : ∀ m, IsScottContinuous (F m) :=
    fun m => theorem_1_3.tuple (uvU_isScottContinuous_left) (
    uvU_isScottContinuous_right)
      (hf m) (hg m)
  have hG : ∀ m, IsScottContinuous (G m) :=
    fun m => theorem_1_3.tuple (uvV_isScottContinuous_left) (
    uvV_isScottContinuous_right)
      (hf m) (hg m)
```

```
  have hpack : (⋂ n, Bs n) = {x | ∀ m, F m x = G m x} := by
    rw [hUC]
    ext x
    constructor
    · intro hx m
      have hx' := Set.mem_iInter.mp hx m
      have hfg : f m x = ofNat 0 ∨ g m x = ofNat 0 := by
        rcases hx' with hOm | hCm
        · exact Or.inl ((charOpen_eq_zero (hO m) x).mpr hOm)
        · exact Or.inr ((closedToZero_eq_zero (hC m) x).mpr hCm)
      exact (uv_eq_iff (charOpen_three (O m) x) (closedToZero_three (hC m) x)).mpr hfg
    · intro hx
      refine Set.mem_iInter.mpr fun m => ?_
      have heq := (uv_eq_iff (charOpen_three (O m) x)
        (closedToZero_three (hC m) x)).mp (hx m)
      rcases heq with hf0 | hg0
      · exact Or.inl ((charOpen_eq_zero (hO m) x).mp hf0)
      · exact Or.inr ((closedToZero_eq_zero (hC m) x).mp hg0)
  exact theorem_6_7_converse_of_equalizers F G hF hG hpack

/-- **Scott 1976, Theorem 6.7 (The ?_? theorem).**
The `B_?` subsets of `Pω` are exactly the equalizers of pairs of continuous
maps. -/
theorem theorem_6_7 :
    ∀ U, IsBdelta U ↔
      ∃ f g, IsScottContinuous f ∧ IsScottContinuous g ∧
        U = {x | f x = g x} :=
  fun U => ⟨theorem_6_7_of_bdelta, theorem_6_7_of_equalizer U⟩

theorem seq2_isScottContinuous_left (y : Pomega) :
    IsScottContinuous (fun x => seq2 x y) :=
  graph_const_isScottContinuous
    (fun x z => condSet z x (condSet (predSet z) y botElem))
    (fun z => condSet.isScottContinuous_mid z _)

theorem seq2_isScottContinuous_right (x : Pomega) :
    IsScottContinuous (fun y => seq2 x y) :=
  graph_const_isScottContinuous
    (fun y z => condSet z x (condSet (predSet z) y botElem))
    (fun z =>
      theorem_1_3 (condSet.isScottContinuous_right z x)
        (condSet.isScottContinuous_mid (predSet z) botElem))

theorem seq2_pair_isScottContinuous {f g : Pomega → Pomega}
    (hf : IsScottContinuous f) (hg : IsScottContinuous g) :
    IsScottContinuous (fun x => seq2 (f x) (g x)) :=
  theorem_1_3.tuple seq2_isScottContinuous_left seq2_isScottContinuous_right hf hg

/-- **Scott 1976, Table 3 / Theorem 6.7.** `?_?` is the class of continuous
preimages of `{u | u₀ = u₁}`. -/
theorem typicalBdelta_universal (U : Set Pomega) :
    IsBdelta U ↔ ∃ f, IsScottContinuous f ∧ U = {x | f x ∈ typicalBdelta} := by
  constructor
  · intro hU
    obtain ⟨F, G, hF, hG, rfl⟩ := (theorem_6_7 U).mp hU
    refine ⟨fun x => seq2 (F x) (G x), seq2_pair_isScottContinuous hF hG, ?_⟩
```

```
    ext x
    constructor
    · intro h
      change funOf (seq2 (F x) (G x)) (ofNat 0) =
        funOf (seq2 (F x) (G x)) (ofNat 1)
      rw [seq2.zero, seq2.one]
      exact h
    · intro h
      have : funOf (seq2 (F x) (G x)) (ofNat 0) =
          funOf (seq2 (F x) (G x)) (ofNat 1) := h
      rw [seq2.zero, seq2.one] at this
      exact this
  · rintro ⟨f, hf, rfl⟩
    exact (theorem_6_7 _).mpr
      ⟨fun x => funOf (f x) (ofNat 0), fun x => funOf (f x) (ofNat 1),
        theorem_1_3 (f := fun u => funOf u (ofNat 0))
          (funOf.isScottContinuous_left (ofNat 0)) hf,
        theorem_1_3 (f := fun u => funOf u (ofNat 1))
          (funOf.isScottContinuous_left (ofNat 1)) hf, rfl⟩
end DomainTheory.DataTypesAsLattices
```

IsScottGdelta.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottOpen
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import Mathlib.Data.Set.Finite.Powerset

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace

def IsScottGdelta (U : Set Pomega) : Prop :=
  ∃ Us : ℕ → Set Pomega, (∀ n, IsScottOpen (Us n)) ∧ U = ⋂ n, Us n

end DomainTheory.DataTypesAsLattices
```

IsFcapGdelta.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottOpen
```

```
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import Mathlib.Data.Set.Finite.Powerset
public import DomainTheory.DataTypesAsLattices.Classification.IsScottGdelta

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace

def IsFcapGdelta (U : Set Pomega) : Prop :=
  ∃ C G, IsScottOpen C? ∧ IsScottGdelta G ∧ U = C ∩ G

end DomainTheory.DataTypesAsLattices
```

ScottF.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottOpen
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import Mathlib.Data.Set.Finite.Powerset

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace

def ScottF : Set (Set Pomega) := {U | IsScottOpen U?}

end DomainTheory.DataTypesAsLattices
```

fcapGdeltaPack.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import Mathlib.Data.Set.Finite.Powerset

@[expose] public section

namespace DomainTheory.DataTypesAsLattices
```

```
open TopologicalSpace

def fcapGdeltaPack (f g : Pomega → Pomega) (x : Pomega) : Pomega :=
  {k | (k = 0 ∧ 0 ∈ f x) ∨ ∃ n, k = n + 1 ∧ n ∈ g x}

end DomainTheory.DataTypesAsLattices
```

zero_not_mem_succSet_top.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Lambda.succSet
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import Mathlib.Data.Set.Finite.Powerset

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace

theorem zero_not_mem_succSet_top : 0 ∈ succSet topElem := by
  intro h
  obtain ⟨n, _, hn⟩ := h
  exact Nat.succ_ne_zero n hn.symm

end DomainTheory.DataTypesAsLattices
```

fcapGdeltaPack_eq.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Lambda.succSet
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import Mathlib.Data.Set.Finite.Powerset
public import DomainTheory.DataTypesAsLattices.Classification.fcapGdeltaPack
public import DomainTheory.DataTypesAsLattices.Classification.zero_not_mem_succSet_top

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace
```

```
theorem fcapGdeltaPack_eq {f g : Pomega → Pomega} {x : Pomega}
    (hfval : f x = botElem ∨ f x = topElem) :
    fcapGdeltaPack f g x = succSet topElem ↔ f x = botElem ∧ g x = topElem := by
  constructor
  · intro h
    constructor
    · have h0 : 0 ∈ fcapGdeltaPack f g x := by
        rw [h]; exact zero_not_mem_succSet_top
      have : 0 ∈ f x := fun hf0 => h0 (Or.inl ⟨rfl, hf0⟩)
      rcases hfval with hf | hf
      · exact hf
      · exact (this (by rw [hf]; exact Set.mem_univ 0)).elim
    · ext n
      constructor
      · intro _; trivial
      · intro _
        have : n + 1 ∈ fcapGdeltaPack f g x := by
          rw [h]; exact ⟨n, Set.mem_univ n, rfl⟩
        rcases this with ⟨hk0, _⟩ | ⟨m, hm, hmn⟩
        · exact (Nat.succ_ne_zero n (by simp [hk0])).elim
        · exact Nat.succ_injective hm ▶ hmn
  · intro ⟨hf, hg⟩
    ext k
    constructor
    · intro hk
      rcases hk with ⟨_, h0⟩ | ⟨n, hk, hn⟩
      · rw [hf] at h0; exact ((mem_botElem 0).mp h0).elim
      · exact ⟨n, Set.mem_univ n, hk⟩
    · intro hk
      obtain ⟨n, _, rfl⟩ := hk
      refine Or.inr ⟨n, rfl, ?_⟩
      rw [hg]; exact Set.mem_univ n

end DomainTheory.DataTypesAsLattices
```

fcapGdeltaPack_isScottContinuous.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.mem_scottUnion
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Continuous.scottUnion
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import Mathlib.Data.Set.Finite.Powerset
public import DomainTheory.DataTypesAsLattices.Classification.fcapGdeltaPack

@[expose] public section
```

```
namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace

theorem fcapGdeltaPack_isScottContinuous {f g : Pomega → Pomega}
    (hf : IsScottContinuous f) (hg : IsScottContinuous g) :
    IsScottContinuous (fcapGdeltaPack f g) := by
  constructor
  intro x
  ext k
  constructor
  · intro hk
    rcases hk with ⟨hk0, h0⟩ | ⟨n, hk, hn⟩
    · have : 0 ∈ scottUnion f x := by rw [← hf.out x]; exact h0
      obtain ⟨p, hp, hp0⟩ := mem_scottUnion.mp this
      exact mem_scottUnion.mpr ⟨p, hp, Or.inl ⟨hk0, hp0⟩⟩
    · have : n ∈ scottUnion g x := by rw [← hg.out x]; exact hn
      obtain ⟨p, hp, hnp⟩ := mem_scottUnion.mp this
      exact mem_scottUnion.mpr ⟨p, hp, Or.inr ⟨n, hk, hnp⟩⟩
  · intro hk
    obtain ⟨p, hp, h⟩ := mem_scottUnion.mp hk
    rcases h with ⟨hk0, h0⟩ | ⟨n, hk, hn⟩
    · exact Or.inl ⟨hk0, IsScottContinuous.monotone hf hp h0⟩
    · exact Or.inr ⟨n, hk, IsScottContinuous.monotone hg hp hn⟩

end DomainTheory.DataTypesAsLattices
```

oddEvenPack_eq_zero.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import Mathlib.Data.Set.Finite.Powerset
public import DomainTheory.DataTypesAsLattices.Classification.oddEvenPack

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace

theorem oddEvenPack_eq_zero {f g : Pomega → Pomega} (x : Pomega) :
    oddEvenPack f g x = ofNat 0 ↔ f x = botElem ∧ g x = ofNat 0 := by
  constructor
  · intro h
    constructor
    · ext n
      constructor
```

```
      · intro hn
        have : 2 * n + 1 ∈ oddEvenPack f g x := Or.inl ⟨n, rfl, hn⟩
        rw [h] at this
        simp [ofNat] at this
      · intro hn
        exact ((mem_botElem n).mp hn).elim
    · ext n
      constructor
      · intro hn
        have hmem : 2 * n ∈ oddEvenPack f g x := Or.inr ⟨n, rfl, hn⟩
        rw [h] at hmem
        have : 2 * n = 0 := by simpa [ofNat] using hmem
        have hn0 : n = 0 := by
          have := Nat.mul_eq_zero.mp this
          exact this.resolve_left (by decide)
        simpa [ofNat] using hn0
      · intro hn
        simp [ofNat] at hn
        subst hn
        have : (0 : ℕ) ∈ oddEvenPack f g x := by rw [h]; simp [ofNat]
        rcases this with ⟨n, hk, _⟩ | ⟨n, hk, hn⟩
        · omega
        · have : n = 0 := by omega
          simpa [this] using hn
  · intro ⟨hf, hg⟩
    ext k
    constructor
    · intro hk
      rcases hk with ⟨n, hk, hn⟩ | ⟨n, hk, hn⟩
      · rw [hf] at hn; exact ((mem_botElem n).mp hn).elim
      · rw [hg] at hn; simp [ofNat] at hn; simp [ofNat, hk, hn]
    · intro hk
      simp [ofNat] at hk
      subst hk
      refine Or.inr ⟨0, rfl, ?_⟩
      rw [hg]; simp [ofNat]

end DomainTheory.DataTypesAsLattices
```

pairIndexed_eq_top.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import Mathlib.Data.Set.Finite.Powerset
public import DomainTheory.DataTypesAsLattices.Classification.pairIndexed

@[expose] public section
```

```
namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace

theorem pairIndexed_eq_top (fs : ℕ → Pomega → Pomega) (x : Pomega) :
    pairIndexed fs x = topElem ↔ ∀ n, fs n x = topElem := by
  constructor
  · intro h n
    ext m
    constructor
    · intro _; trivial
    · intro _
      have : pair n m ∈ pairIndexed fs x := by rw [h]; exact Set.mem_univ _
      obtain ⟨n', m', heq, hm⟩ := this
      obtain ⟨rfl, rfl⟩ := pair_inj heq
      exact hm
  · intro h
    ext k
    constructor
    · intro _; trivial
    · intro _
      obtain ⟨n, m, hk⟩ := exists_pair k
      exact ⟨n, m, hk.symm, by rw [h n]; exact Set.mem_univ m⟩

end DomainTheory.DataTypesAsLattices
```

theorem_6_1.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.isScottOpen_isUpper
public import DomainTheory.DataTypesAsLattices.Continuous.mem_scottUnion
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottOpen
public import DomainTheory.DataTypesAsLattices.Continuous.scottUnion
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import Mathlib.Data.Set.Finite.Powerset
public import DomainTheory.DataTypesAsLattices.Classification.charOpen
public import DomainTheory.DataTypesAsLattices.Classification.Basic

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace

/-- **Scott 1976, Theorem 6.1 (The ? theorem).** -/
theorem theorem_6_1 :
```

```
    (∀ f : Pomega → Pomega, IsScottContinuous f →
        IsScottOpen {x | 0 ∈ f x}) ∧
      (∀ U, IsScottOpen U →
        ∃ f, IsScottContinuous f ∧ U = {x | 0 ∈ f x}) := by
  constructor
  · intro f hf
    refine ⟨{n | 0 ∈ f (e n)}, ?_⟩
    ext x
    constructor
    · intro hx
      have : 0 ∈ scottUnion f x := by rwa [← hf.out x]
      obtain ⟨n, hn, h0⟩ := mem_scottUnion.mp this
      exact Set.mem_iUnion₂.mpr ⟨n, h0, hn⟩
    · intro hx
      obtain ⟨n, h0, hn⟩ := Set.mem_iUnion₂.mp hx
      exact IsScottContinuous.monotone hf hn h0
  · intro U hU
    refine ⟨charOpen U, charOpen_isScottContinuous U, ?_⟩
    ext x
    constructor
    · intro hx
      obtain ⟨B, rfl⟩ := hU
      obtain ⟨n, hn, hxn⟩ := Set.mem_iUnion₂.mp hx
      refine ⟨rfl, n, hxn, ?_⟩
      exact Set.mem_iUnion₂.mpr ⟨n, hn, (subset_rfl : e n ⊆ e n)⟩
    · intro ⟨_, n, hn, hne⟩
      exact isScottOpen_isUpper hU hne hn

end DomainTheory.DataTypesAsLattices
```

theorem_6_2.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottOpen
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import Mathlib.Data.Set.Finite.Powerset
public import DomainTheory.DataTypesAsLattices.Classification.ScottF
public import DomainTheory.DataTypesAsLattices.Classification.charTop
public import DomainTheory.DataTypesAsLattices.Classification.Basic

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace

/-- **Scott 1976, Theorem 6.2 (The ? theorem).** -/
```

```
theorem theorem_6_2 :
    ∀ U, U ∈ ScottF ↔ ∃ f, IsScottContinuous f ∧ U = {x | f x = botElem} := by
  intro U
  constructor
  · intro hU
    refine ⟨charTop U?, charTop_isScottContinuous U?, ?_⟩
    ext x
    exact (charTop_eq_bot_of_closed hU x).symm
  · rintro ⟨f, hf, rfl⟩
    change IsScottOpen {x | f x = botElem}?
    have : {x | f x = botElem}? = ⋃ k, {x | k ∈ f x} := by
      ext x
      constructor
      · intro hx
        have hx' : f x = botElem := hx
        have : ∃ k, k ∈ f x := by
          by_contra hnone
          apply hx'
          ext n
          constructor
          · intro hn
            exact (hnone ⟨n, hn⟩).elim
          · intro hn
            exact ((mem_botElem n).mp hn).elim
        exact Set.mem_iUnion.mpr this
      · intro hx hbot
        obtain ⟨k, hk⟩ := Set.mem_iUnion.mp hx
        have hk' : k ∈ f x := hk
        have hbot' : f x = botElem := hbot
        rw [hbot'] at hk'
        exact ((mem_botElem k).mp hk').elim
    rw [this]
    exact isScottOpen_iUnion _ fun k => mem_preimage_isScottOpen hf k

end DomainTheory.DataTypesAsLattices
```

theorem_6_3.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import Mathlib.Data.Set.Finite.Powerset
public import DomainTheory.DataTypesAsLattices.Classification.pairIndexed_eq_top
public import DomainTheory.DataTypesAsLattices.Classification.IsScottGdelta
public import DomainTheory.DataTypesAsLattices.Classification.Basic
public import DomainTheory.DataTypesAsLattices.Classification.charTop
public import DomainTheory.DataTypesAsLattices.Classification.pairIndexed
```

```
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace

/-- **Scott 1976, Theorem 6.3 (The ?_? theorem).** -/
theorem theorem_6_3 :
    ∀ U, IsScottGdelta U ↔ ∃ f, IsScottContinuous f ∧ U = {x | f x = topElem} := by
  intro U
  constructor
  · rintro ⟨Us, hUs, rfl⟩
    let fs := fun n => charTop (Us n)
    refine ⟨pairIndexed fs, pairIndexed_isScottContinuous fs fun n =>
      charTop_isScottContinuous (Us n), ?_⟩
    ext x
    constructor
    · intro hx
      change pairIndexed fs x = topElem
      exact (pairIndexed_eq_top fs x).mpr fun n =>
        (charTop_eq_top (hUs n) x).mpr (Set.mem_iInter.mp hx n)
    · intro hf
      have hf' : pairIndexed fs x = topElem := hf
      exact Set.mem_iInter.mpr fun n =>
        (charTop_eq_top (hUs n) x).mp ((pairIndexed_eq_top fs x).mp hf' n)
  · rintro ⟨f, hf, rfl⟩
    refine ⟨fun n => {x | n ∈ f x}, fun n => mem_preimage_isScottOpen hf n, ?_⟩
    ext x
    constructor
    · intro hx
      refine Set.mem_iInter.mpr fun n => ?_
      have : f x = topElem := hx
      simp [this]
    · intro hx
      ext k
      constructor
      · intro _; trivial
      · intro _
        exact Set.mem_iInter.mp hx k

end DomainTheory.DataTypesAsLattices
```

theorem_6_4.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Continuous.isScottOpen_isUpper
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
```

```
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import Mathlib.Data.Set.Finite.Powerset
public import DomainTheory.DataTypesAsLattices.Classification.oddEvenPack_eq_zero
public import DomainTheory.DataTypesAsLattices.Classification.IsFcapG
public import DomainTheory.DataTypesAsLattices.Classification.Basic

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace

/-- **Scott 1976, Theorem 6.4 (The ? ∩ ? theorem).** -/
theorem theorem_6_4 :
    ∀ U, IsFcapG U ↔ ∃ f, IsScottContinuous f ∧ U = {x | f x = ofNat 0} := by
  intro U
  constructor
  · rintro ⟨C, O, hC, hO, rfl⟩
    refine ⟨oddEvenPack (charTop C?) (charOpen O),
      oddEvenPack_isScottContinuous (charTop_isScottContinuous C?)
        (charOpen_isScottContinuous O), ?_⟩
    ext x
    constructor
    · intro hx
      change oddEvenPack (charTop C?) (charOpen O) x = ofNat 0
      refine (oddEvenPack_eq_zero x).mpr ?_
      have hx' : x ∈ C ∧ x ∈ O := hx
      constructor
      · exact (charTop_eq_bot_of_closed hC x).mpr hx'.1
      · ext k
        constructor
        · intro ⟨hk0, n, hn, hne⟩
          simp [ofNat, hk0]
        · intro hk
          simp [ofNat] at hk
          subst hk
          obtain ⟨n, hn, hne⟩ := (mem_isScottOpen_iff hO).mp hx'.2
          exact ⟨rfl, n, hn, hne⟩
    · intro hf
      have hf' := (oddEvenPack_eq_zero (f := charTop C?) (g := charOpen O) x).mp hf
      constructor
      · exact (charTop_eq_bot_of_closed hC x).mp hf'.1
      · have : 0 ∈ charOpen O x := by rw [hf'.2]; simp [ofNat]
        obtain ⟨_, n, hn, hne⟩ := this
        exact isScottOpen_isUpper hO hne hn
  · rintro ⟨f, hf, rfl⟩
    refine ⟨{x | f x ⊆ ofNat 0}, {x | 0 ∈ f x}, ?_, mem_preimage_isScottOpen hf 0, ?_⟩
    · have : {x | f x ⊆ ofNat 0}? = ⋃ k : ℕ, {x | k = 0 ∧ k ∈ f x} := by
        ext x
        constructor
        · intro hx
          have : ¬ f x ⊆ ofNat 0 := hx
          obtain ⟨k, hk, hk0⟩ := Set.not_subset.mp this
          refine Set.mem_iUnion.mpr ⟨k, ?_⟩
          simp [ofNat] at hk0
          exact ⟨hk0, hk⟩
```

```
        · intro hx
          obtain ⟨k, hk0, hk⟩ := Set.mem_iUnion.mp hx
          intro hsub
          exact hk0 (hsub hk)
      rw [this]
      exact isScottOpen_iUnion _ fun k => by
        by_cases hk0 : k = 0
        · subst hk0
          have : {x | (0 : ℕ) = 0 ∧ 0 ∈ f x} = (∅ : Set Pomega) := by
            ext x; simp
          rw [this]; exact isScottOpen_empty
        · have : {x | k = 0 ∧ k ∈ f x} = {x | k ∈ f x} := by
            ext x; simp [hk0]
          rw [this]; exact mem_preimage_isScottOpen hf k
    · ext x
      constructor
      · intro hx
        change f x ⊆ ofNat 0 ∧ 0 ∈ f x
        have hx' : f x = ofNat 0 := hx
        constructor
        · simp [hx']
        · simp [hx', ofNat]
      · intro hx
        change f x = ofNat 0
        ext n
        simp [ofNat]
        constructor
        · intro hn; exact hx.1 hn
        · intro hn; simpa [hn] using hx.2

end DomainTheory.DataTypesAsLattices
```

theorem_6_5.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Lambda.succSet
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import Mathlib.Data.Set.Finite.Powerset
public import DomainTheory.DataTypesAsLattices.Classification.theorem_6_3
public import DomainTheory.DataTypesAsLattices.Classification.
    fcapGdeltaPack_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Classification.zero_not_mem_succSet_top
public import DomainTheory.DataTypesAsLattices.Classification.fcapGdeltaPack_eq
public import DomainTheory.DataTypesAsLattices.Classification.IsFcapGdelta
public import DomainTheory.DataTypesAsLattices.Classification.Basic
public import DomainTheory.DataTypesAsLattices.Classification.charTop
```

```
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace

/-- **Scott 1976, Theorem 6.5 (The ? ∩ ?_? theorem).** -/
theorem theorem_6_5 :
    ∀ U, IsFcapGdelta U ↔
      ∃ f, IsScottContinuous f ∧ U = {x | f x = succSet topElem} := by
  intro U
  constructor
  · rintro ⟨C, G, hC, hG, rfl⟩
    obtain ⟨fG, hfG, rflG⟩ := (theorem_6_3 G).mp hG
    refine ⟨fcapGdeltaPack (charTop C?) fG,
      fcapGdeltaPack_isScottContinuous (charTop_isScottContinuous C?) hfG, ?_⟩
    ext x
    have hfval : charTop C? x = botElem ∨ charTop C? x = topElem := by
      by_cases hx : x ∈ C
      · exact Or.inl ((charTop_eq_bot_of_closed hC x).mpr hx)
      · exact Or.inr ((charTop_eq_top hC x).mpr hx)
    constructor
    · intro ⟨hxC, hxG⟩
      have hf : charTop C? x = botElem := (charTop_eq_bot_of_closed hC x).mpr hxC
      have hg : fG x = topElem := by rw [rflG] at hxG; exact hxG
      exact (fcapGdeltaPack_eq hfval).mpr ⟨hf, hg⟩
    · intro hpack
      obtain ⟨hf, hg⟩ := (fcapGdeltaPack_eq hfval).mp hpack
      exact ⟨(charTop_eq_bot_of_closed hC x).mp hf, by rw [rflG]; exact hg⟩
  · rintro ⟨f, hf, rfl⟩
    refine ⟨{x | f x ⊆ succSet topElem}, {x | succSet topElem ⊆ f x}, ?_, ?_, ?_⟩
    · have : {x | f x ⊆ succSet topElem}? = {x | 0 ∈ f x} := by
        ext x
        constructor
        · intro hx
          obtain ⟨k, hk, hknotin⟩ := Set.not_subset.mp hx
          have hk0 : k = 0 := by
            by_contra hne
            exact hknotin ⟨k - 1, Set.mem_univ _, (Nat.succ_pred_eq_of_ne_zero hne).
    symm⟩
          simpa [hk0] using hk
        · intro h0 hsub
          exact zero_not_mem_succSet_top (hsub h0)
      rw [this]
      exact mem_preimage_isScottOpen hf 0
    · refine ⟨fun n => {x | n + 1 ∈ f x}, fun n => mem_preimage_isScottOpen hf (n + 1),
     ?_⟩
      ext x
      constructor
      · intro hx
        exact Set.mem_iInter.mpr fun n => hx ⟨n, Set.mem_univ n, rfl⟩
      · intro hx k hk
        obtain ⟨n, _, rfl⟩ := hk
        exact Set.mem_iInter.mp hx n
    · ext x
      constructor
```

```
      · intro hx
        change f x ⊆ succSet topElem ∧ succSet topElem ⊆ f x
        have hx' : f x = succSet topElem := hx
        constructor <;> simp [hx']
      · intro hx
        change f x = succSet topElem
        exact subset_antisymm hx.1 hx.2

end DomainTheory.DataTypesAsLattices
```

Vapply.lean

```
module

public import Mathlib.Algebra.Ring.Parity
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (5.14)-(5.15).** The universe combinator
`V(a)(x) = ⋂ { y | x ⊆ y ∧ a(y) ⊆ y }`. -/
def Vapply (a x : Pomega) : Pomega :=
  ⋂₀ {y | x ⊆ y ∧ funOf a y ⊆ y}

end DomainTheory.DataTypesAsLattices
```

eq_5_1.lean

```
module

public import Mathlib.Algebra.Ring.Parity
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.theorem_1_2
public import DomainTheory.DataTypesAsLattices.Pomega.e
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (5.1).** `u ⊆ λx. u(x)`. -/
theorem eq_5_1 (u : Pomega) : u ⊆ graph (fun x => funOf u x) :=
  theorem_1_2.ii u

end DomainTheory.DataTypesAsLattices
```

eq_5_16.lean

```
module

public import Mathlib.Algebra.Ring.Parity
public import DomainTheory.DataTypesAsLattices.Pomega.e
```

```
public import DomainTheory.DataTypesAsLattices.Closures.Vapply
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (5.16).** `x ⊆ V(a)(x)`. -/
theorem eq_5_16 (a x : Pomega) : x ⊆ Vapply a x := by
  intro k hk y hy
  exact hy.1 hk

end DomainTheory.DataTypesAsLattices
```

eq_5_17.lean

```
module

public import Mathlib.Algebra.Ring.Parity
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Closures.Vapply
public import DomainTheory.DataTypesAsLattices.Closures.eq_5_16

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (5.17).** `V(a)` is idempotent. -/
theorem eq_5_17 (a x : Pomega) : Vapply a (Vapply a x) = Vapply a x := by
  apply subset_antisymm
  · intro k hk y hy
    have : Vapply a x ⊆ y := by
      intro m hm
      exact hm y hy
    exact hk y ⟨this, hy.2⟩
  · exact eq_5_16 a (Vapply a x)

end DomainTheory.DataTypesAsLattices
```

eq_5_18.lean

```
module

public import Mathlib.Algebra.Ring.Parity
public import DomainTheory.DataTypesAsLattices.Retracts.Icomb
public import DomainTheory.DataTypesAsLattices.Retracts.retract_app
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Closures.Closure
public import DomainTheory.DataTypesAsLattices.Closures.Vapply
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (5.18).** If `a` is already a closure, then `V(a)(x) ⊆ a(x)`. -/
theorem eq_5_18 {a : Pomega} (ha : Closure a) (x : Pomega) :
```

```
    Vapply a x ⊆ funOf a x := by
  intro k hk
  have hx : x ⊆ funOf a x := by
    have := funOf.monotone_left ha.out.1 x
    rwa [Icomb.app] at this
  have haa : funOf a (funOf a x) ⊆ funOf a x := by
    simp [retract_app ha.out.2]
  exact hk (funOf a x) ⟨hx, haa⟩

namespace eq_5_18

/-- The missing inclusion of (5.18): if `x ⊆ y` and `a(y) ⊆ y` then `a(x) ⊆ y`. -/
theorem rev (a x : Pomega) :
    funOf a x ⊆ Vapply a x := by
  intro k hk y hy
  exact hy.2 (funOf.monotone_right a hy.1 hk)

/-- **Scott 1976, (5.18).** Closures are fixed by `V` on values. -/
theorem eq {a : Pomega} (ha : Closure a) (x : Pomega) :
    Vapply a x = funOf a x :=
  subset_antisymm (eq_5_18 ha x) (eq_5_18.rev a x)

end eq_5_18

end DomainTheory.DataTypesAsLattices
```

union_left_isScottContinuous.lean

```
module

public import Mathlib.Algebra.Ring.Parity
public import Mathlib.Order.CompleteLattice.Defs
public import DomainTheory.DataTypesAsLattices.Lambda.id_isScottContinuous
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.mem_scottUnion
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Continuous.scottUnion
public import DomainTheory.DataTypesAsLattices.Pomega.e
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem union_left_isScottContinuous (c : Pomega) :
    IsScottContinuous (fun x => x ∪ c) := by
  constructor
  intro x
  ext k
  constructor
  · intro hk
    rcases hk with hk | hk
    · have : k ∈ scottUnion (fun z => z) x := by
        rw [← id_isScottContinuous.out x]; exact hk
```

```
      obtain ⟨n, hn, hkn⟩ := mem_scottUnion.mp this
      exact mem_scottUnion.mpr ⟨n, hn, Or.inl hkn⟩
    · exact mem_scottUnion.mpr ⟨0, by simp [e_zero], Or.inr hk⟩
  · intro hk
    obtain ⟨n, hn, hkn⟩ := mem_scottUnion.mp hk
    rcases hkn with hkn | hkn
    · exact Or.inl (IsScottContinuous.monotone id_isScottContinuous hn hkn)
    · exact Or.inr hkn

end DomainTheory.DataTypesAsLattices
```

union_right_isScottContinuous.lean

```
module

public import Mathlib.Algebra.Ring.Parity
public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.mem_scottUnion
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Continuous.scottUnion
public import DomainTheory.DataTypesAsLattices.Pomega.e
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem union_right_isScottContinuous (x : Pomega) {f : Pomega → Pomega}
    (hf : IsScottContinuous f) :
    IsScottContinuous (fun y => x ∪ f y) := by
  constructor
  intro y
  ext k
  constructor
  · intro hk
    rcases hk with hk | hk
    · exact mem_scottUnion.mpr ⟨0, by simp [e_zero], Or.inl hk⟩
    · have : k ∈ scottUnion f y := by rwa [← hf.out y]
      obtain ⟨n, hn, hkn⟩ := mem_scottUnion.mp this
      exact mem_scottUnion.mpr ⟨n, hn, Or.inr hkn⟩
  · intro hk
    obtain ⟨n, hn, hkn⟩ := mem_scottUnion.mp hk
    rcases hkn with hkn | hkn
    · exact Or.inl hkn
    · exact Or.inr (IsScottContinuous.monotone hf hn hkn)

end DomainTheory.DataTypesAsLattices
```

Vstep.lean

```
module
```

```
public import Mathlib.Algebra.Ring.Parity
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Closures.union_right_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (5.14).** `V(a)(x) = Y(λy. x ∪ a(y))`. -/
def Vstep (a x : Pomega) (y : Pomega) : Pomega := x ∪ funOf a y

namespace Vstep

theorem isScottContinuous (a x : Pomega) :
    IsScottContinuous (Vstep a x) :=
  union_right_isScottContinuous x (funOf.isScottContinuous a)

end Vstep

end DomainTheory.DataTypesAsLattices
```

VapplyY.lean

```
module

public import Mathlib.Algebra.Ring.Parity
public import DomainTheory.DataTypesAsLattices.FixedPoint.fix
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Closures.Vstep

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def VapplyY (a x : Pomega) : Pomega := fix (Vstep a x)

end DomainTheory.DataTypesAsLattices
```

Vapply_more.lean

```
module

public import DomainTheory.DataTypesAsLattices.Closures.Vapply
public import Mathlib.Algebra.Ring.Parity
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
public import DomainTheory.DataTypesAsLattices.Pomega.e
```

```
public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.theorem_1_2
public import DomainTheory.DataTypesAsLattices.Closures.Vstep
public import DomainTheory.DataTypesAsLattices.Closures.VapplyY
public import DomainTheory.DataTypesAsLattices.Closures.eq_5_16
public import DomainTheory.DataTypesAsLattices.Closures.eq_5_17
public import DomainTheory.DataTypesAsLattices.Closures.Closure
public import DomainTheory.DataTypesAsLattices.Closures.union_right_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Closures.union_left_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.fix_param_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Retracts.graph_ext
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_4

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace Vapply

/-- `V(a)(x)` is a prefixpoint of `y ↦ x ∪ a(y)`. -/
theorem prefixpoint (a x : Pomega) :
    x ⊆ Vapply a x ∧ funOf a (Vapply a x) ⊆ Vapply a x := by
  refine ⟨_root_.DomainTheory.DataTypesAsLattices.eq_5_16 a x, ?_⟩
  intro k hk y hy
  have hV : Vapply a x ⊆ y := fun m hm => hm y hy
  exact hy.2 (funOf.monotone_right a hV hk)

/-- **Scott 1976, Theorem 5.6 / (5.15).** The universe applied to `x` is
closed under `a` and contains `x`. -/
theorem closed (a x : Pomega) :
    x ∪ funOf a (Vapply a x) ⊆ Vapply a x := by
  intro k hk
  rcases hk with hk | hk
  · exact _root_.DomainTheory.DataTypesAsLattices.eq_5_16 a x hk
  · exact (Vapply.prefixpoint a x).2 hk

theorem eq_Vstep (a x : Pomega) :
    Vapply a x = Vstep a x (Vapply a x) := by
  apply subset_antisymm
  · intro k hk
    have hset : x ∪ funOf a (Vapply a x) ∈
        {y | x ⊆ y ∧ funOf a y ⊆ y} :=
      ⟨Set.subset_union_left, fun m hm =>
        Set.subset_union_right
          (IsScottContinuous.monotone (funOf.isScottContinuous a)
            (Vapply.closed a x) hm)⟩
    exact hk (x ∪ funOf a (Vapply a x)) hset
  · exact Vapply.closed a x

/-- **Scott 1976, Theorem 5.5.** `V(a)(x) = a(x)` for every `x` implies
```

```
`a` is expansive and idempotent on values, hence a closure once `a` is
already a graph retract. -/
theorem eq_imp_expansive {a : Pomega}
    (h : ∀ x, Vapply a x = funOf a x) (x : Pomega) :
    x ⊆ funOf a x :=
  (h x) ▸ _root_.DomainTheory.DataTypesAsLattices.eq_5_16 a x

theorem eq_imp_idem {a : Pomega}
    (h : ∀ x, Vapply a x = funOf a x) (x : Pomega) :
    funOf a (funOf a x) = funOf a x := by
  have := _root_.DomainTheory.DataTypesAsLattices.eq_5_17 a x
  rwa [h (Vapply a x), h x] at this

/-- Library form of Scott (5.14): `Y(Vstep a x) = Vapply a x`. -/
theorem eq_Y (a x : Pomega) : VapplyY a x = Vapply a x := by
  have hfix : Vstep a x (VapplyY a x) = VapplyY a x :=
    (theorem_1_4 (Vstep.isScottContinuous a x)).1
  apply subset_antisymm
  · exact (theorem_1_4 (Vstep.isScottContinuous a x)).2 (Vapply a x)
      (Vapply.eq_Vstep a x).symm
  · intro k hk
    have hx : x ⊆ VapplyY a x := by
      intro m hm
      have : m ∈ Vstep a x (VapplyY a x) := Or.inl hm
      rwa [hfix] at this
    have ha : funOf a (VapplyY a x) ⊆ VapplyY a x := by
      intro m hm
      have : m ∈ Vstep a x (VapplyY a x) := Or.inr hm
      rwa [hfix] at this
    exact hk (VapplyY a x) ⟨hx, ha⟩

theorem left_isScottContinuous (x : Pomega) :
    IsScottContinuous (fun a => Vapply a x) := by
  have : (fun a => Vapply a x) = (fun a => VapplyY a x) := by
    funext a; exact (eq_Y a x).symm
  rw [this]
  exact fix_param_isScottContinuous
    (fun y => union_right_isScottContinuous x (funOf.isScottContinuous_left y))
    (fun a => Vstep.isScottContinuous a x)

theorem right_isScottContinuous (a : Pomega) :
    IsScottContinuous (fun x => Vapply a x) := by
  have : (fun x => Vapply a x) = (fun x => VapplyY a x) := by
    funext x; exact (eq_Y a x).symm
  rw [this]
  exact fix_param_isScottContinuous
    (fun y => union_left_isScottContinuous (funOf a y))
    (fun x => Vstep.isScottContinuous a x)

/-- **Scott 1976, (5.19).** `V(a)` is always a closure, so `V(V(a)) = V(a)`. -/
theorem graph_isClosure (a : Pomega) :
    Closure (graph (fun x => Vapply a x)) := by
  constructor
  constructor
  · intro p hp
    rcases hp with ⟨n, m, rfl, hm⟩
```

```
    exact ⟨n, m, rfl, _root_.DomainTheory.DataTypesAsLattices.eq_5_16 a (e n) hm⟩
  · constructor
    change graph (fun x => Vapply a x) =
        graph (fun x =>
          funOf (graph (fun z => Vapply a z))
            (funOf (graph (fun z => Vapply a z)) x))
    apply graph_ext
    intro x
    have happ : funOf (graph (fun z => Vapply a z)) =
        fun z => Vapply a z :=
      theorem_1_2.i (Vapply.right_isScottContinuous a)
    simpa [happ] using (_root_.DomainTheory.DataTypesAsLattices.eq_5_17 a x).symm

end Vapply

end DomainTheory.DataTypesAsLattices
```

Vcomb.lean

```
module

public import Mathlib.Algebra.Ring.Parity
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Closures.Vapply
public import DomainTheory.DataTypesAsLattices.Closures.Vapply_more
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (5.14).** The universe combinator. -/
def Vcomb : Pomega :=
  graph (fun a => graph (fun x => Vapply a x))

namespace Vcomb

theorem app (a : Pomega) :
    funOf Vcomb a = graph (fun x => Vapply a x) :=
  beta (graph_const_isScottContinuous (fun a x => Vapply a x)
    (fun x => Vapply.left_isScottContinuous x)) a

end Vcomb

end DomainTheory.DataTypesAsLattices
```

boxTag.lean

```
module

public import Mathlib.Algebra.Ring.Parity
```

```
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (5.13).** The occupancy tag `[u]ᵢ ? i, i`: the constant
`i` whenever the component is nonempty, and `⊥` when it is empty. -/
def boxTag (i : ℕ) (x : Pomega) : Pomega :=
  condSet x (ofNat i) (ofNat i)

namespace boxTag

theorem bot (i : ℕ) : boxTag i botElem = botElem := by
  ext n
  simp [boxTag, condSet, botElem]

theorem isScottContinuous (i : ℕ) : IsScottContinuous (boxTag i) := by
  have hfun : boxTag i = fun z => condSet z (ofNat i) (ofNat i) := rfl
  rw [hfun]
  exact condSet.isScottContinuous_left (ofNat i) (ofNat i)

theorem of_mem {x : Pomega} {m : ℕ} (hm : m ∈ x) (i : ℕ) :
    boxTag i x = ofNat i := by
  ext n
  simp only [boxTag, condSet, Set.mem_union, Set.mem_ofPred_eq]
  constructor
  · rintro (⟨hn, _⟩ | ⟨hn, _⟩) <;> exact hn
  · intro hn
    cases m with
    | zero => exact Or.inl ⟨hn, hm⟩
    | succ k => exact Or.inr ⟨hn, k, hm⟩

end boxTag

end DomainTheory.DataTypesAsLattices
```

closure_expansive.lean

```
module

public import Mathlib.Algebra.Ring.Parity
public import DomainTheory.DataTypesAsLattices.Retracts.Icomb
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Closures.Closure
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem closure_expansive {a : Pomega} (ha : Closure a) (x : Pomega) :
```

```
    x ⊆ funOf a x := by
  have := funOf.monotone_left ha.out.1 x
  rwa [Icomb.app] at this

end DomainTheory.DataTypesAsLattices
```

closureCompact.lean

```
module

public import Mathlib.Algebra.Ring.Parity
public import DomainTheory.DataTypesAsLattices.Retracts.comp
public import DomainTheory.DataTypesAsLattices.Retracts.Fixpoints
public import Mathlib.Order.CompleteLattice.Defs
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Graph.theorem_1_2
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Closures.Closure
public import DomainTheory.DataTypesAsLattices.FixedLattices.IsCompact
public import DomainTheory.DataTypesAsLattices.Retracts.retractBasis
public import DomainTheory.DataTypesAsLattices.Retracts.vals
public import DomainTheory.DataTypesAsLattices.Retracts.IsCompactSubset
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
public import DomainTheory.DataTypesAsLattices.Closures.closure_expansive
public import DomainTheory.DataTypesAsLattices.Closures.directed_finite_cover_subset

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- The fixed point generated by the finite set `e n`. -/
def closureCompact (a : Pomega) (ha : Closure a) (n : ℕ) :
    Fixpoints (funOf a) :=
  ⟨funOf a (e n), by
    have hcont : IsScottContinuous (fun y => funOf a (funOf a y)) :=
      theorem_1_3 (funOf.isScottContinuous a) (funOf.isScottContinuous a)
    have hbeta := (theorem_1_2 hcont (comp a a)).1
    have h := congrArg (fun u => funOf u (e n)) ha.out.2.out
    have hcomp : funOf (comp a a) (e n) = funOf a (funOf a (e n)) :=
      congrArg (fun f => f (e n)) hbeta
    exact hcomp.symm.trans h.symm⟩

namespace closureCompact

theorem isCompact {a : Pomega} (ha : Closure a) (n : ℕ) :
    IsCompact (closureCompact a ha n) := by
  apply retractBasis.wayBelow ha.out.2
  exact ⟨n, closure_expansive ha (e n), rfl⟩
```

```
theorem isCompactSubset {a : Pomega} (ha : Closure a)
    (n : ℕ) : IsCompactSubset (closureCompact a ha n) := by
  intro s hs hsub
  have hen : e n ⊆ ⋃₀ (Subtype.val '' s) :=
    (closure_expansive ha (e n)).trans hsub
  obtain ⟨z, hz, hnz⟩ := directed_finite_cover_subset hs (e_finite n) hen
  refine ⟨z, hz, ?_⟩
  have := IsScottContinuous.monotone (funOf.isScottContinuous a) hnz
  rwa [z.property] at this

end closureCompact

end DomainTheory.DataTypesAsLattices
```

compactFixedBelow.lean

```
module

public import Mathlib.Algebra.Ring.Parity
public import DomainTheory.DataTypesAsLattices.Retracts.Fixpoints
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- Compact fixed points below `x`: the isolated points `a(e n)` contained in `x`. -/
def compactFixedBelow (a : Pomega) (x : Fixpoints (funOf a)) :
    Set (Fixpoints (funOf a)) :=
  {k | (∃ n, (k : Pomega) = funOf a (e n)) ∧ (k : Pomega) ⊆ (x : Pomega)}

end DomainTheory.DataTypesAsLattices
```

representClosure.lean

```
module

public import Mathlib.Algebra.Ring.Parity
public import DomainTheory.DataTypesAsLattices.FixedLattices.CountablyAlgebraic
public import DomainTheory.DataTypesAsLattices.FixedLattices.
    CountablyAlgebraicu2ecompact_finite_support
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Continuous.mem_scottUnion
public import DomainTheory.DataTypesAsLattices.Continuous.scottUnion
public import Mathlib.Order.CompleteLattice.Defs

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, Theorem 5.2, construction.**
`representClosure D x` is the set of compact indices below the supremum
generated by the indices in `x`. -/
```

```
def representClosure {α : Type u} [CompleteLattice α]
    (D : CountablyAlgebraic α) (x : Pomega) : Pomega :=
  {m | D.compact m ≤ sSup (D.compact '' x)}

namespace representClosure

theorem mono {α : Type u} [CompleteLattice α]
    (D : CountablyAlgebraic α) {x y : Pomega} (hxy : x ⊆ y) :
    representClosure D x ⊆ representClosure D y := by
  intro m hm
  exact hm.trans (by
    apply sSup_le
    intro q hq
    exact le_sSup (Set.image_mono hxy hq))

theorem isScottContinuous
    {α : Type u} [CompleteLattice α] (D : CountablyAlgebraic α) :
    IsScottContinuous (representClosure D) := by
  constructor
  intro x
  ext m
  constructor
  · intro hm
    obtain ⟨t, htx, hmt⟩ := D.compact_finite_support hm
    obtain ⟨n, hn⟩ := exists_code t
    refine mem_scottUnion.mpr ⟨n, ?_, ?_⟩
    · rw [hn]
      exact htx
    · change D.compact m ≤ sSup (D.compact '' e n)
      rwa [hn]
  · intro hm
    obtain ⟨n, hnx, hmn⟩ := mem_scottUnion.mp hm
    exact representClosure.mono D hnx hmn

theorem expansive {α : Type u} [CompleteLattice α]
    (D : CountablyAlgebraic α) (x : Pomega) :
    x ⊆ representClosure D x := by
  intro n hn
  exact le_sSup ⟨n, hn, rfl⟩

theorem idem {α : Type u} [CompleteLattice α]
    (D : CountablyAlgebraic α) (x : Pomega) :
    representClosure D (representClosure D x) = representClosure D x := by
  apply subset_antisymm
  · intro m hm
    exact hm.trans (by
      apply sSup_le
      intro q hq
      obtain ⟨n, hn, rfl⟩ := hq
      exact hn)
  · exact representClosure.expansive D (representClosure D x)

end representClosure

end DomainTheory.DataTypesAsLattices
```

representClosureCode.lean

```
module

public import Mathlib.Algebra.Ring.Parity
public import DomainTheory.DataTypesAsLattices.FixedLattices.CountablyAlgebraic
public import DomainTheory.DataTypesAsLattices.FixedLattices.
    CountablyAlgebraicu2ecompact_finite_support
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Closures.representClosure
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Retracts.graph_ext
public import DomainTheory.DataTypesAsLattices.Closures.Closure
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- The `Pω` code of the representing closure operation. -/
def representClosureCode {α : Type u} [CompleteLattice α]
    (D : CountablyAlgebraic α) : Pomega :=
  graph (representClosure D)

namespace representClosureCode

theorem app {α : Type u} [CompleteLattice α]
    (D : CountablyAlgebraic α) (x : Pomega) :
    funOf (representClosureCode D) x = representClosure D x :=
  beta (representClosure.isScottContinuous D) x

theorem isClosure
    {α : Type u} [CompleteLattice α] (D : CountablyAlgebraic α) :
    Closure (representClosureCode D) := by
  constructor
  constructor
  · intro p hp
    rcases hp with ⟨n, m, rfl, hm⟩
    exact ⟨n, m, rfl, representClosure.expansive D (e n) hm⟩
  · constructor
    change representClosureCode D =
      graph (fun x => funOf (representClosureCode D)
        (funOf (representClosureCode D) x))
    apply graph_ext
    intro x
    rw [representClosureCode.app, representClosureCode.app,
      representClosure.idem]

end representClosureCode

end DomainTheory.DataTypesAsLattices
```

representationEncode_more.lean

```
module

public import DomainTheory.DataTypesAsLattices.Closures.representationEncode
public import Mathlib.Algebra.Ring.Parity
public import DomainTheory.DataTypesAsLattices.FixedLattices.CountablyAlgebraic
public import DomainTheory.DataTypesAsLattices.FixedLattices.
    CountablyAlgebraicu2ecompact_finite_support
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Closures.representClosureCode
public import DomainTheory.DataTypesAsLattices.Closures.representationDecode

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace representationEncode

theorem fixed
    {α : Type u} [CompleteLattice α] (D : CountablyAlgebraic α) (x : α) :
    funOf (representClosureCode D) (representationEncode D x) =
      representationEncode D x := by
  rw [representClosureCode.app]
  ext n
  change D.compact n ≤
      representationDecode D (representationEncode D x) ↔
    D.compact n ≤ x
  rw [representationDecode.encode]

end representationEncode

end DomainTheory.DataTypesAsLattices
```

representationOrderIso.lean

```
module

public import Mathlib.Algebra.Ring.Parity
public import DomainTheory.DataTypesAsLattices.Retracts.Fixpoints
public import DomainTheory.DataTypesAsLattices.FixedLattices.CountablyAlgebraic
public import DomainTheory.DataTypesAsLattices.FixedLattices.
    CountablyAlgebraicu2ecompact_finite_support
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Closures.representClosure
public import DomainTheory.DataTypesAsLattices.Closures.representClosureCode
public import DomainTheory.DataTypesAsLattices.Closures.representationEncode
public import DomainTheory.DataTypesAsLattices.Closures.representationDecode
public import DomainTheory.DataTypesAsLattices.Retracts.vals
public import DomainTheory.DataTypesAsLattices.Closures.representationEncode_more
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- The order isomorphism in Scott's representation theorem. -/
def representationOrderIso {α : Type u} [CompleteLattice α]
```

```
    (D : CountablyAlgebraic α) :
    α ≃o Fixpoints (funOf (representClosureCode D)) where
  toFun x := ⟨representationEncode D x, representationEncode.fixed D x⟩
  invFun y := representationDecode D y
  left_inv := representationDecode.encode D
  right_inv y := by
    apply Subtype.ext
    change representationEncode D (representationDecode D y) = y
    change representClosure D y = y
    rw [← representClosureCode.app, y.property]
  map_rel_iff' := by
    intro x y
    constructor
    · intro h
      rw [← representationDecode.encode D x,
        ← representationDecode.encode D y]
      apply sSup_le
      rintro _ ⟨n, hn, rfl⟩
      exact le_sSup ⟨n, h hn, rfl⟩
    · intro h n hn
      exact hn.trans h

end DomainTheory.DataTypesAsLattices
```

retractBasis_eq_compactFixedBelow.lean

```
module

public import Mathlib.Algebra.Ring.Parity
public import DomainTheory.DataTypesAsLattices.Retracts.Fixpoints
public import DomainTheory.DataTypesAsLattices.Retracts.retractBasis
public import Mathlib.Order.CompleteLattice.Defs
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Closures.Closure
public import DomainTheory.DataTypesAsLattices.Closures.closure_expansive
public import DomainTheory.DataTypesAsLattices.Closures.compactFixedBelow
public import DomainTheory.DataTypesAsLattices.Retracts.vals
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem retractBasis_eq_compactFixedBelow {a : Pomega} (ha : Closure a)
    (x : Fixpoints (funOf a)) :
    retractBasis a x = compactFixedBelow a x := by
  ext k
  constructor
  · rintro ⟨n, hnx, hkn⟩
```

```
    refine ⟨⟨n, hkn⟩, ?_⟩
    rw [hkn]
    have := IsScottContinuous.monotone (funOf.isScottContinuous a) hnx
    rwa [x.property] at this
  · rintro ⟨⟨n, hkn⟩, hkx⟩
    refine ⟨n, ?_, hkn⟩
    have hsub : funOf a (e n) ⊆ (x : Pomega) := by
      rwa [hkn] at hkx
    exact (closure_expansive ha (e n)).trans hsub

end DomainTheory.DataTypesAsLattices
```

fixed_eq_sSup_compacts.lean

```
module

public import Mathlib.Algebra.Ring.Parity
public import DomainTheory.DataTypesAsLattices.Retracts.Fixpoints
public import DomainTheory.DataTypesAsLattices.Retracts.retractBasis
public import DomainTheory.DataTypesAsLattices.FixedLattices.DirectedSet
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Closures.Closure
public import DomainTheory.DataTypesAsLattices.Closures.compactFixedBelow
public import DomainTheory.DataTypesAsLattices.Closures.
    retractBasis_eq_compactFixedBelow
public import DomainTheory.DataTypesAsLattices.Retracts.vals

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem fixed_eq_sSup_compacts {a : Pomega} (ha : Closure a)
    (x : Fixpoints (funOf a)) :
    DirectedSet (compactFixedBelow a x) ∧
      x = sSup (compactFixedBelow a x) := by
  rw [← retractBasis_eq_compactFixedBelow ha x]
  exact ⟨retractBasis.directed ha.out.2 x, retractBasis.eq_sSup ha.out.2 x⟩

end DomainTheory.DataTypesAsLattices
```

squareFst.lean

```
module

public import Mathlib.Algebra.Ring.Parity
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.mem_scottUnion
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
@[expose] public section
```

```
namespace DomainTheory.DataTypesAsLattices

def squareFst (u : Pomega) : Pomega :=
  {n | 2 * n ∈ u}

namespace squareFst

theorem bot : squareFst botElem = botElem := by
  ext n
  simp [squareFst, botElem]

theorem isScottContinuous : IsScottContinuous squareFst := by
  constructor
  intro x
  ext n
  constructor
  · intro hn
    refine mem_scottUnion.mpr ⟨2 ^ (2 * n), ?_, ?_⟩
    · simpa [e_pow2, Set.singleton_subset_iff, squareFst] using hn
    · simp [squareFst, e_pow2]
  · intro hn
    obtain ⟨m, hm, hnm⟩ := mem_scottUnion.mp hn
    exact hm hnm

theorem top : squareFst topElem = topElem := by
  ext n
  constructor
  · intro; exact Set.mem_univ n
  · intro; exact Set.mem_univ (2 * n)

end squareFst

end DomainTheory.DataTypesAsLattices
```

eq_5_9.lean

```
module

public import Mathlib.Algebra.Ring.Parity
public import DomainTheory.DataTypesAsLattices.Closures.squarePair
public import DomainTheory.DataTypesAsLattices.Closures.squareFst
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (5.9)-(5.10).** -/
theorem eq_5_9 (x y : Pomega) : squareFst (squarePair x y) = x := by
  ext n
  constructor
  · intro hn
    rcases hn with ⟨k, hk, h⟩ | ⟨m, _, h⟩
    · have : k = n := by omega
```

```
      simpa [this] using hk
    · omega
  · intro hn
    exact Or.inl ⟨n, hn, rfl⟩

end DomainTheory.DataTypesAsLattices
```

squareSnd.lean

```
module

public import Mathlib.Algebra.Ring.Parity
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.mem_scottUnion
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def squareSnd (u : Pomega) : Pomega :=
  {m | 2 * m + 1 ∈ u}

namespace squareSnd

theorem bot : squareSnd botElem = botElem := by
  ext m
  simp [squareSnd, botElem]

theorem isScottContinuous : IsScottContinuous squareSnd := by
  constructor
  intro x
  ext m
  constructor
  · intro hm
    refine mem_scottUnion.mpr ⟨2 ^ (2 * m + 1), ?_, ?_⟩
    · simpa [e_pow2, Set.singleton_subset_iff, squareSnd] using hm
    · simp [squareSnd, e_pow2]
  · intro hm
    obtain ⟨k, hk, hkm⟩ := mem_scottUnion.mp hm
    exact hk hkm

theorem top : squareSnd topElem = topElem := by
  ext m
  constructor
  · intro; exact Set.mem_univ m
  · intro; exact Set.mem_univ (2 * m + 1)

end squareSnd

end DomainTheory.DataTypesAsLattices
```

eq_5_10.lean

```
module

public import Mathlib.Algebra.Ring.Parity
public import DomainTheory.DataTypesAsLattices.Closures.squarePair
public import DomainTheory.DataTypesAsLattices.Closures.squareSnd
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (5.9)-(5.10).** -/
theorem eq_5_10 (x y : Pomega) : squareSnd (squarePair x y) = y := by
  ext m
  constructor
  · intro hm
    rcases hm with ⟨n, _, h⟩ | ⟨k, hk, h⟩
    · omega
    · have : k = m := by omega
      simpa [this] using hk
  · intro hm
    exact Or.inr ⟨m, hm, rfl⟩

end DomainTheory.DataTypesAsLattices
```

squarePair_more.lean

```
module

public import DomainTheory.DataTypesAsLattices.Closures.squarePair
public import Mathlib.Algebra.Ring.Parity
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.mem_scottUnion
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Closures.squareFst
public import DomainTheory.DataTypesAsLattices.Closures.squareSnd
public import DomainTheory.DataTypesAsLattices.Closures.eq_5_9
public import DomainTheory.DataTypesAsLattices.Closures.eq_5_10

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace squarePair

theorem bot_bot : squarePair botElem botElem = (botElem : Pomega) := by
  ext k
  simp [squarePair, botElem]
```

```
theorem isScottContinuous_left (y : Pomega) :
    IsScottContinuous (fun x => squarePair x y) := by
  constructor
  intro x
  ext k
  constructor
  · intro hk
    rcases hk with ⟨n, hn, hk⟩ | ⟨m, hm, hk⟩
    · exact mem_scottUnion.mpr ⟨2 ^ n, by simp [e_pow2, Set.singleton_subset_iff, hn],
        Or.inl ⟨n, by simp [e_pow2], hk⟩⟩
    · exact mem_scottUnion.mpr ⟨0, by simp [e_zero], Or.inr ⟨m, hm, hk⟩⟩
  · intro hk
    obtain ⟨p, hp, hkp⟩ := mem_scottUnion.mp hk
    rcases hkp with ⟨n, hn, hk⟩ | ⟨m, hm, hk⟩
    · exact Or.inl ⟨n, hp hn, hk⟩
    · exact Or.inr ⟨m, hm, hk⟩

theorem isScottContinuous_right (x : Pomega) :
    IsScottContinuous (fun y => squarePair x y) := by
  constructor
  intro y
  ext k
  constructor
  · intro hk
    rcases hk with ⟨n, hn, hk⟩ | ⟨m, hm, hk⟩
    · exact mem_scottUnion.mpr ⟨0, by simp [e_zero], Or.inl ⟨n, hn, hk⟩⟩
    · exact mem_scottUnion.mpr ⟨2 ^ m, by simp [e_pow2, Set.singleton_subset_iff, hm],
        Or.inr ⟨m, by simp [e_pow2], hk⟩⟩
  · intro hk
    obtain ⟨p, hp, hkp⟩ := mem_scottUnion.mp hk
    rcases hkp with ⟨n, hn, hk⟩ | ⟨m, hm, hk⟩
    · exact Or.inl ⟨n, hn, hk⟩
    · exact Or.inr ⟨m, hp hm, hk⟩

theorem mono {x x' y y' : Pomega} (hx : x ⊆ x') (hy : y ⊆ y') :
    squarePair x y ⊆ squarePair x' y' := by
  intro k hk
  rcases hk with ⟨n, hn, hk⟩ | ⟨m, hm, hk⟩
  · exact Or.inl ⟨n, hx hn, hk⟩
  · exact Or.inr ⟨m, hy hm, hk⟩

theorem inj {x x' y y' : Pomega}
    (h : squarePair x y = squarePair x' y') : x = x' ∧ y = y' :=
  ⟨(DomainTheory.DataTypesAsLattices.eq_5_9 x y).symm.trans
      ((congrArg squareFst h).trans (DomainTheory.DataTypesAsLattices.eq_5_9 x' y')),
    (DomainTheory.DataTypesAsLattices.eq_5_10 x y).symm.trans
      ((congrArg squareSnd h).trans (DomainTheory.DataTypesAsLattices.eq_5_10 x' y'))⟩

end squarePair

end DomainTheory.DataTypesAsLattices
```

boxTensor.lean

```
module
```

```
public import Mathlib.Algebra.Ring.Parity
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Closures.squarePair
public import DomainTheory.DataTypesAsLattices.Closures.squareFst
public import DomainTheory.DataTypesAsLattices.Closures.squareSnd
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Retracts.Retract
public import DomainTheory.DataTypesAsLattices.Retracts.retract_app
public import DomainTheory.DataTypesAsLattices.Closures.eq_5_9
public import DomainTheory.DataTypesAsLattices.Closures.eq_5_10
public import DomainTheory.DataTypesAsLattices.Retracts.graph_ext
public import DomainTheory.DataTypesAsLattices.Closures.squarePair_more
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (5.12).** Product of closures. -/
def boxTensor (a b : Pomega) : Pomega :=
  graph (fun u => squarePair (funOf a (squareFst u)) (funOf b (squareSnd u)))

namespace boxTensor

theorem app (a b u : Pomega) :
    funOf (boxTensor a b) u =
      squarePair (funOf a (squareFst u)) (funOf b (squareSnd u)) :=
  beta (theorem_1_3.tuple
    (f := fun x y => squarePair (funOf a x) (funOf b y))
    (fun y => theorem_1_3 (f := fun x => squarePair x (funOf b y)) (g := funOf a)
      (squarePair.isScottContinuous_left (funOf b y))
      (funOf.isScottContinuous a))
    (fun x => theorem_1_3 (f := fun y => squarePair (funOf a x) y) (g := funOf b)
      (squarePair.isScottContinuous_right (funOf a x))
      (funOf.isScottContinuous b))
    squareFst.isScottContinuous squareSnd.isScottContinuous) u

theorem isRetract {a b : Pomega} (ha : Retract a) (hb : Retract b) :
    Retract (boxTensor a b) := by
  constructor
  change boxTensor a b =
    graph (fun u => funOf (boxTensor a b) (funOf (boxTensor a b) u))
  apply graph_ext
  intro u
  rw [boxTensor.app, boxTensor.app, _root_.DomainTheory.DataTypesAsLattices.eq_5_9,
    _root_.DomainTheory.DataTypesAsLattices.eq_5_10, retract_app ha, retract_app hb]

end boxTensor

end DomainTheory.DataTypesAsLattices
```

arrowR_diag_isScottContinuous.lean

```
module

public import Mathlib.Algebra.Ring.Parity
public import DomainTheory.DataTypesAsLattices.Retracts.comp
public import DomainTheory.DataTypesAsLattices.Retracts.arrowR
public import Mathlib.Order.CompleteLattice.Defs
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Lambda.id_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem arrowR_diag_isScottContinuous :
    IsScottContinuous (fun a => arrowR a a) := by
  have hbody : ∀ u, IsScottContinuous (fun a =>
      graph (fun x => funOf a (funOf u (funOf a x)))) :=
    fun u =>
      graph_const_isScottContinuous
        (fun a x => funOf a (funOf u (funOf a x)))
        (fun x =>
          theorem_1_3.tuple
            (f := fun a b => funOf a (funOf u (funOf b x)))
            (fun b => funOf.isScottContinuous_left (funOf u (funOf b x)))
            (fun a =>
              theorem_1_3 (funOf.isScottContinuous a)
                (theorem_1_3 (funOf.isScottContinuous u)
                  (funOf.isScottContinuous_left x)))
            id_isScottContinuous id_isScottContinuous)
  have heq : (fun a => arrowR a a) = (fun a =>
      graph (fun u => graph (fun x => funOf a (funOf u (funOf a x))))) := by
    funext a
    apply congrArg graph
    funext u
    change graph (fun x => funOf a (funOf (comp u a) x)) =
      graph (fun x => funOf a (funOf u (funOf a x)))
    apply congrArg graph
    funext x
    rw [comp.app]
  rw [heq]
  exact graph_const_isScottContinuous
    (fun a u => graph (fun x => funOf a (funOf u (funOf a x))))
    hbody

end DomainTheory.DataTypesAsLattices
```

selfArrowStep.lean

```
module

public import Mathlib.Algebra.Ring.Parity
public import DomainTheory.DataTypesAsLattices.Retracts.arrowR
public import DomainTheory.DataTypesAsLattices.Retracts.Icomb
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.mem_scottUnion
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Continuous.scottUnion
public import DomainTheory.DataTypesAsLattices.Closures.arrowR_diag_isScottContinuous
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (5.25).** Step of `Y(λa. I ∪ (a o→ a))`. -/
def selfArrowStep (a : Pomega) : Pomega := Icomb ∪ arrowR a a

namespace selfArrowStep

theorem isScottContinuous :
    IsScottContinuous selfArrowStep := by
  constructor
  intro a
  have h := arrowR_diag_isScottContinuous.out a
  ext k
  constructor
  · intro hk
    rcases hk with hk | hk
    · exact mem_scottUnion.mpr ⟨0, by simp [e_zero], Or.inl hk⟩
    · have : k ∈ scottUnion (fun a => arrowR a a) a := by rwa [← h]
      obtain ⟨n, hn, hkn⟩ := mem_scottUnion.mp this
      exact mem_scottUnion.mpr ⟨n, hn, Or.inr hkn⟩
  · intro hk
    obtain ⟨n, hn, hkn⟩ := mem_scottUnion.mp hk
    rcases hkn with hkn | hkn
    · exact Or.inl hkn
    · exact Or.inr (IsScottContinuous.monotone arrowR_diag_isScottContinuous hn hkn)

end selfArrowStep

end DomainTheory.DataTypesAsLattices
```

dEq.lean

```
module

public import Mathlib.Algebra.Ring.Parity
public import DomainTheory.DataTypesAsLattices.FixedPoint.fix
public import DomainTheory.DataTypesAsLattices.Pomega.e
```

```
public import DomainTheory.DataTypesAsLattices.Closures.selfArrowStep

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (5.25).** Least solution of `d = I ∪ (d o→ d)`. -/
def dEq : Pomega := fix selfArrowStep

end DomainTheory.DataTypesAsLattices
```

compactFixedBelow_more.lean

```
module

public import Mathlib.Algebra.Ring.Parity
public import DomainTheory.DataTypesAsLattices.Retracts.Fixpoints
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.retractBasis
public import DomainTheory.DataTypesAsLattices.Retracts.vals
public import DomainTheory.DataTypesAsLattices.Closures.Closure
public import DomainTheory.DataTypesAsLattices.Closures.compactFixedBelow
public import DomainTheory.DataTypesAsLattices.Closures.
    retractBasis_eq_compactFixedBelow

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace compactFixedBelow

theorem directedSubset {a : Pomega} (ha : Closure a)
    (x : Fixpoints (funOf a)) :
    DirectedSubset (compactFixedBelow a x) := by
  rw [← retractBasis_eq_compactFixedBelow ha x]
  exact retractBasis.directedSubset ha.out.2 x

theorem sUnion {a : Pomega} (ha : Closure a)
    (x : Pomega) = ⋃₀ (Subtype.val '' compactFixedBelow a x) := by
  rw [← retractBasis_eq_compactFixedBelow ha x]
  exact retractBasis.eq_sUnion ha.out.2 x

end compactFixedBelow

end DomainTheory.DataTypesAsLattices
```

compact_fixed_iff.lean

```
module

public import Mathlib.Algebra.Ring.Parity
public import DomainTheory.DataTypesAsLattices.Retracts.Fixpoints
public import DomainTheory.DataTypesAsLattices.Retracts.retractBasis
```

```
public import Mathlib.Order.CompleteLattice.Defs
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.FixedLattices.IsCompact
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Closures.Closure
public import DomainTheory.DataTypesAsLattices.Closures.closureCompact
public import DomainTheory.DataTypesAsLattices.Closures.compactFixedBelow
public import DomainTheory.DataTypesAsLattices.Retracts.vals
public import DomainTheory.DataTypesAsLattices.Retracts.IsCompactSubset
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
public import DomainTheory.DataTypesAsLattices.Closures.compactFixedBelow_more
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem compact_fixed_iff {a : Pomega} (ha : Closure a)
    (x : Fixpoints (funOf a)) :
    IsCompact x ↔ ∃ n, x = closureCompact a ha n := by
  constructor
  · intro hx
    have hdir := retractBasis.directed ha.out.2 x
    obtain ⟨y, hy, hxy⟩ :=
      hx (retractBasis a x) hdir (le_of_eq (retractBasis.eq_sSup ha.out.2 x))
    obtain ⟨n, hnx, hyn⟩ := hy
    have hyc : y = closureCompact a ha n := by
      apply Subtype.ext
      exact hyn
    subst y
    refine ⟨n, le_antisymm hxy ?_⟩
    change funOf a (e n) ⊆ (x : Pomega)
    have := IsScottContinuous.monotone (funOf.isScottContinuous a) hnx
    rwa [x.property] at this
  · rintro ⟨n, rfl⟩
    exact closureCompact.isCompact ha n

namespace compact_fixed_iff

theorem subset {a : Pomega} (ha : Closure a)
    (x : Fixpoints (funOf a)) :
    IsCompactSubset x ↔ ∃ n, x = closureCompact a ha n := by
  constructor
  · intro hx
    obtain ⟨y, hy, hxy⟩ :=
      hx (compactFixedBelow a x) (compactFixedBelow.directedSubset ha x)
        (le_of_eq (compactFixedBelow.sUnion ha x))
    obtain ⟨⟨n, hn⟩, hyx⟩ := hy
    have hyx' : (y : Pomega) ⊆ x := hyx
    have : y = x := Subtype.ext (subset_antisymm hyx' hxy)
    refine ⟨n, this.symm.trans (Subtype.ext hn)⟩
  · rintro ⟨n, rfl⟩
    exact closureCompact.isCompactSubset ha n
```

```
end compact_fixed_iff

end DomainTheory.DataTypesAsLattices
```

closureRangeCountablyAlgebraic.lean

```
module

public import Mathlib.Algebra.Ring.Parity
public import DomainTheory.DataTypesAsLattices.Retracts.Fixpoints
public import Mathlib.Order.CompleteLattice.Defs
public import DomainTheory.DataTypesAsLattices.FixedLattices.CountablyAlgebraic
public import DomainTheory.DataTypesAsLattices.FixedLattices.
    CountablyAlgebraicu2ecompact_finite_support
public import DomainTheory.DataTypesAsLattices.FixedLattices.IsCompact
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Closures.Closure
public import DomainTheory.DataTypesAsLattices.Closures.closureCompact
public import DomainTheory.DataTypesAsLattices.Closures.compact_fixed_iff
public import DomainTheory.DataTypesAsLattices.Closures.compactFixedBelow
public import DomainTheory.DataTypesAsLattices.Closures.fixed_eq_sSup_compacts
public import DomainTheory.DataTypesAsLattices.Retracts.vals

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- Theorem 5.1 packaged as a concrete countably algebraic lattice. -/
def closureRangeCountablyAlgebraic (a : Pomega) (ha : Closure a) :
    CountablyAlgebraic (Fixpoints (funOf a)) where
  compact := closureCompact a ha
  compact_isCompact := closureCompact.isCompact ha
  compact_complete := fun x hx => (compact_fixed_iff ha x).mp hx
  density := fun x => by
    have hsets : {k | IsCompact k ∧ k ≤ x} = compactFixedBelow a x := by
      ext k
      constructor
      · intro ⟨hk, hkx⟩
        obtain ⟨n, hn⟩ := (compact_fixed_iff ha k).mp hk
        exact ⟨⟨n, congrArg Subtype.val hn⟩, hkx⟩
      · intro ⟨⟨n, hn⟩, hkx⟩
        exact ⟨(compact_fixed_iff ha k).mpr ⟨n, Subtype.ext hn⟩, hkx⟩
    rw [hsets]
    exact (fixed_eq_sSup_compacts ha x).2

end DomainTheory.DataTypesAsLattices
```

theorem_5_5.lean

```
module

public import Mathlib.Algebra.Ring.Parity
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Closures.Closure
public import DomainTheory.DataTypesAsLattices.Closures.Isolated
public import DomainTheory.DataTypesAsLattices.Closures.squarePair
public import DomainTheory.DataTypesAsLattices.Closures.squareFst
public import DomainTheory.DataTypesAsLattices.Closures.squareSnd
public import DomainTheory.DataTypesAsLattices.Closures.boxTensor
public import DomainTheory.DataTypesAsLattices.Closures.boxTag
public import DomainTheory.DataTypesAsLattices.Closures.Vapply
public import DomainTheory.DataTypesAsLattices.Closures.closureCompact
public import DomainTheory.DataTypesAsLattices.Closures.compactFixedBelow
public import DomainTheory.DataTypesAsLattices.Closures.Vstep
public import DomainTheory.DataTypesAsLattices.Closures.VapplyY
public import DomainTheory.DataTypesAsLattices.Closures.closureRangeCountablyAlgebraic
public import DomainTheory.DataTypesAsLattices.Closures.representClosure
public import DomainTheory.DataTypesAsLattices.Closures.representClosureCode
public import DomainTheory.DataTypesAsLattices.Closures.representationEncode
public import DomainTheory.DataTypesAsLattices.Closures.representationEncode_more
public import DomainTheory.DataTypesAsLattices.Closures.representationDecode
public import DomainTheory.DataTypesAsLattices.Closures.representationOrderIso
public import DomainTheory.DataTypesAsLattices.Closures.Vcomb
public import DomainTheory.DataTypesAsLattices.Closures.selfArrowStep
public import DomainTheory.DataTypesAsLattices.Closures.dEq
public import DomainTheory.DataTypesAsLattices.Closures.closure_expansive
public import DomainTheory.DataTypesAsLattices.Closures.compact_fixed_iff
public import DomainTheory.DataTypesAsLattices.Closures.
    retractBasis_eq_compactFixedBelow
public import DomainTheory.DataTypesAsLattices.Closures.fixed_eq_sSup_compacts
public import DomainTheory.DataTypesAsLattices.Closures.eq_5_17
public import DomainTheory.DataTypesAsLattices.Closures.eq_5_18

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, Theorem 5.5 / (5.18).** `V` fixes closures on values. -/
theorem theorem_5_5 {a : Pomega} (ha : Closure a) (x : Pomega) :
    Vapply a x = funOf a x ∧ Vapply a (Vapply a x) = Vapply a x :=
  ⟨eq_5_18.eq ha x, eq_5_17 a x⟩

end DomainTheory.DataTypesAsLattices
```

eq_5_11.lean

```
module

public import Mathlib.Algebra.Ring.Parity
public import DomainTheory.DataTypesAsLattices.Retracts.Icomb
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Closures.Closure
```

```
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (5.11).** Closures act as expansive maps. -/
theorem eq_5_11 {a b u x : Pomega} (ha : Closure a) (hb : Closure b) :
    funOf u x ⊆ funOf b (funOf u (funOf a x)) := by
  have hx : x ⊆ funOf a x := by
    have := funOf.monotone_left ha.out.1 x
    rwa [Icomb.app] at this
  have hy : funOf u (funOf a x) ⊆ funOf b (funOf u (funOf a x)) := by
    have := funOf.monotone_left hb.out.1 (funOf u (funOf a x))
    rwa [Icomb.app] at this
  exact subset_trans (funOf.monotone_right u hx) hy

end DomainTheory.DataTypesAsLattices
```

Icomb_subset_arrowR.lean

```
module

public import Mathlib.Algebra.Ring.Parity
public import DomainTheory.DataTypesAsLattices.Retracts.comp
public import DomainTheory.DataTypesAsLattices.Retracts.arrowR
public import DomainTheory.DataTypesAsLattices.Retracts.Icomb
public import DomainTheory.DataTypesAsLattices.Lambda.xi_star
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Closures.Closure
public import DomainTheory.DataTypesAsLattices.Closures.eq_5_1
public import DomainTheory.DataTypesAsLattices.Closures.eq_5_11

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, Theorem 5.3.** The function space of two closures is a
closure: it is a retract and contains `I`. -/
theorem Icomb_subset_arrowR {a b : Pomega} (ha : Closure a) (hb : Closure b) :
    Icomb ⊆ arrowR a b := by
  intro p hp
  rcases hp with ⟨n, m, rfl, hm⟩
  have h1 : e n ⊆ graph (fun x => funOf (e n) x) := eq_5_1 (e n)
  have h2 : graph (fun x => funOf (e n) x) ⊆
      graph (fun x => funOf b (funOf (e n) (funOf a x))) :=
    xi_star (fun x => eq_5_11 ha hb)
  have h3 :
      graph (fun x => funOf b (funOf (e n) (funOf a x))) =
        comp b (comp (e n) a) := by
    apply congrArg graph
    funext x
    simp [comp.app]
```

```
  have hsub : e n ⊆ comp b (comp (e n) a) := by
    rw [← h3]; exact h1.trans h2
  exact ⟨n, m, rfl, hsub hm⟩

end DomainTheory.DataTypesAsLattices
```

Icomb_subset_graph.lean

```
module

public import Mathlib.Algebra.Ring.Parity
public import DomainTheory.DataTypesAsLattices.Retracts.Icomb
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Pomega.e
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- Graph expansiveness `I ⊆ λx. f(x)` is value-expansiveness on finite sets. -/
theorem Icomb_subset_graph {f : Pomega → Pomega} :
    Icomb ⊆ graph f ↔ ∀ n, e n ⊆ f (e n) := by
  constructor
  · intro h n m hm
    have := h ⟨n, m, rfl, hm⟩
    rcases this with ⟨n', m', heq, hm'⟩
    obtain ⟨rfl, rfl⟩ := pair_inj heq
    exact hm'
  · intro h p hp
    rcases hp with ⟨n, m, rfl, hm⟩
    exact ⟨n, m, rfl, h n hm⟩

end DomainTheory.DataTypesAsLattices
```

eq_5_8.lean

```
module

public import Mathlib.Algebra.Ring.Parity
public import DomainTheory.DataTypesAsLattices.Closures.squarePair
public import DomainTheory.DataTypesAsLattices.Closures.squareFst
public import DomainTheory.DataTypesAsLattices.Closures.squareSnd
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (5.8).** -/
theorem eq_5_8 (u : Pomega) : squarePair (squareFst u) (squareSnd u) = u := by
  ext k
  constructor
  · intro hk
    rcases hk with ⟨n, hn, hk⟩ | ⟨m, hm, hk⟩
```

```
    · simpa [squareFst, hk] using hn
    · simpa [squareSnd, hk] using hm
  · intro hk
    rcases Nat.even_or_odd k with h | h
    · obtain ⟨n, rfl⟩ := h
      have heq : n + n = 2 * n := by omega
      have hn : 2 * n ∈ u := by rwa [← heq]
      exact Or.inl ⟨n, hn, heq⟩
    · obtain ⟨m, rfl⟩ := h
      exact Or.inr ⟨m, hk, rfl⟩

end DomainTheory.DataTypesAsLattices
```

union_const_left_isScottContinuous.lean

```
module

public import Mathlib.Algebra.Ring.Parity
public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Closures.union_left_isScottContinuous

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem union_const_left_isScottContinuous (c : Pomega) :
    IsScottContinuous (fun x => c ∪ x) := by
  have : (fun x : Pomega => c ∪ x) = (fun x => x ∪ c) := by
    funext x; exact Set.union_comm c x
  rw [this]
  exact union_left_isScottContinuous c

end DomainTheory.DataTypesAsLattices
```

boxShift.lean

```
module

public import Mathlib.Algebra.Ring.Parity
public import DomainTheory.DataTypesAsLattices.Retracts.Retract
public import DomainTheory.DataTypesAsLattices.Retracts.Icomb
public import DomainTheory.DataTypesAsLattices.Retracts.retract_app
public import DomainTheory.DataTypesAsLattices.Retracts.graph_ext
public import DomainTheory.DataTypesAsLattices.Retracts.predSet_ofNat_zero
public import Mathlib.Order.CompleteLattice.Defs
public import DomainTheory.DataTypesAsLattices.Lambda.succSet
public import DomainTheory.DataTypesAsLattices.Lambda.predSet
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Graph.beta
```

```
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Closures.Closure
public import DomainTheory.DataTypesAsLattices.Closures.
    union_const_left_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Closures.predSet_union
public import DomainTheory.DataTypesAsLattices.Closures.predSet_succSet
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
public import DomainTheory.DataTypesAsLattices.Lambda.predSet_more
public import DomainTheory.DataTypesAsLattices.Lambda.succSet_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (5.13).** Shift used in the sum of closures. -/
def boxShift (a : Pomega) : Pomega :=
  graph (fun x => ofNat 0 ∪ succSet (funOf a (predSet x)))

namespace boxShift

theorem map_isScottContinuous (a : Pomega) :
    IsScottContinuous (fun x => ofNat 0 ∪ succSet (funOf a (predSet x))) :=
  theorem_1_3 (union_const_left_isScottContinuous (ofNat 0))
    (theorem_1_3 succSet.isScottContinuous
      (theorem_1_3 (funOf.isScottContinuous a) predSet.isScottContinuous))

theorem app (a x : Pomega) :
    funOf (boxShift a) x = ofNat 0 ∪ succSet (funOf a (predSet x)) :=
  beta (map_isScottContinuous a) x

theorem expansive {a : Pomega} (ha : Closure a) (x : Pomega) :
    x ⊆ funOf (boxShift a) x := by
  rw [app]
  intro k hk
  cases k with
  | zero => exact Or.inl rfl
  | succ k =>
    refine Or.inr ⟨k, ?_, rfl⟩
    have hx : predSet x ⊆ funOf a (predSet x) := by
      have := funOf.monotone_left ha.out.1 (predSet x)
      rwa [Icomb.app] at this
    exact hx hk

theorem isRetract {a : Pomega} (ha : Retract a) :
    Retract (boxShift a) := by
  constructor
  change boxShift a =
    graph (fun x => funOf (boxShift a) (funOf (boxShift a) x))
```

```
  apply graph_ext
  intro x
  rw [app, app]
  have hpred : predSet (ofNat 0 ∪ succSet (funOf a (predSet x))) =
      funOf a (predSet x) := by
    rw [predSet_union, predSet_ofNat_zero, predSet_succSet]
    simp [botElem]
  rw [hpred, retract_app ha]

theorem zero_mem_app (a x : Pomega) : (0 : ℕ) ∈ funOf (boxShift a) x := by
  rw [app]
  exact Or.inl (by simp [ofNat])

theorem app_ne_bot (a x : Pomega) : funOf (boxShift a) x = botElem := by
  intro h
  have h0 := zero_mem_app a x
  rw [h] at h0
  simp [botElem] at h0

end boxShift

end DomainTheory.DataTypesAsLattices
```

boxPlus.lean

```
module

public import Mathlib.Algebra.Ring.Parity
public import DomainTheory.DataTypesAsLattices.Retracts.Retract
public import DomainTheory.DataTypesAsLattices.Retracts.typed
public import DomainTheory.DataTypesAsLattices.Retracts.retract_app
public import DomainTheory.DataTypesAsLattices.Retracts.graph_ext
public import Mathlib.Order.CompleteLattice.Defs
public import DomainTheory.DataTypesAsLattices.Lambda.dcondSet
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Closures.Closure
public import DomainTheory.DataTypesAsLattices.Closures.squarePair
public import DomainTheory.DataTypesAsLattices.Closures.squareFst
public import DomainTheory.DataTypesAsLattices.Closures.squareSnd
public import DomainTheory.DataTypesAsLattices.Closures.boxShift
public import DomainTheory.DataTypesAsLattices.Closures.boxTag
public import DomainTheory.DataTypesAsLattices.Closures.eq_5_8
public import DomainTheory.DataTypesAsLattices.Closures.eq_5_9
public import DomainTheory.DataTypesAsLattices.Closures.eq_5_10
public import DomainTheory.DataTypesAsLattices.Closures.union_left_isScottContinuous
```

```
public import DomainTheory.DataTypesAsLattices.Closures.
    union_const_left_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Closures.union_botElem
public import DomainTheory.DataTypesAsLattices.Closures.botElem_union
public import DomainTheory.DataTypesAsLattices.Closures.exists_mem_of_ne_bot
public import DomainTheory.DataTypesAsLattices.Closures.topElem_ne_botElem
public import DomainTheory.DataTypesAsLattices.Closures.squarePair_more
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
public import DomainTheory.DataTypesAsLattices.Lambda.dcondSet_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (5.13).** Sum of closures:
`a ? b = λu.([u]₀ ? 0, 0) ∪ ([u]₁ ? 1, 1) ? [a'([u]₀), ⊥], [⊥, b'([u]₁)]`.
The test is the union of the two occupancy tags, so an inhabited left
component dispatches to the left branch, an inhabited right component to
the right branch, both to `⊤` (via the doubly strict (4.4)) and neither
to `⊥`. -/
def boxPlus (a b : Pomega) : Pomega :=
  graph (fun u =>
    dcondSet (boxTag 0 (squareFst u) ∪ boxTag 1 (squareSnd u))
      (squarePair (funOf (boxShift a) (squareFst u)) botElem)
      (squarePair botElem (funOf (boxShift b) (squareSnd u))))

namespace boxPlus

/- `def`+`namespace boxPlus` shadows bare `eq_5_*`. -/
private abbrev squarePair_eta := _root_.DomainTheory.DataTypesAsLattices.eq_5_8
private abbrev squareFst_of_pair := _root_.DomainTheory.DataTypesAsLattices.eq_5_9
private abbrev squareSnd_of_pair := _root_.DomainTheory.DataTypesAsLattices.eq_5_10

theorem test_isScottContinuous :
    IsScottContinuous (fun u => boxTag 0 (squareFst u) ∪ boxTag 1 (squareSnd u)) :=
  theorem_1_3.tuple
    (fun y => union_left_isScottContinuous y)
    (fun x => union_const_left_isScottContinuous x)
    (theorem_1_3 (boxTag.isScottContinuous 0) squareFst.isScottContinuous)
    (theorem_1_3 (boxTag.isScottContinuous 1) squareSnd.isScottContinuous)

theorem map_isScottContinuous (a b : Pomega) :
    IsScottContinuous (fun u =>
      dcondSet (boxTag 0 (squareFst u) ∪ boxTag 1 (squareSnd u))
        (squarePair (funOf (boxShift a) (squareFst u)) botElem)
        (squarePair botElem (funOf (boxShift b) (squareSnd u)))) :=
  theorem_1_3.nary
    (fun y z => dcondSet.isScottContinuous_left y z)
    (fun x z => dcondSet.isScottContinuous_mid x z)
    (fun x y => dcondSet.isScottContinuous_right x y)
    test_isScottContinuous
    (theorem_1_3 (f := fun x => squarePair x botElem)
      (g := fun u => funOf (boxShift a) (squareFst u))
      (squarePair.isScottContinuous_left botElem)
      (theorem_1_3 (funOf.isScottContinuous (boxShift a)) squareFst.isScottContinuous)
    )
```

```
    (theorem_1_3 (f := fun y => squarePair botElem y)
      (g := fun u => funOf (boxShift b) (squareSnd u))
      (squarePair.isScottContinuous_right botElem)
      (theorem_1_3 (funOf.isScottContinuous (boxShift b)) squareSnd.isScottContinuous)
    )

theorem app (a b u : Pomega) :
    funOf (boxPlus a b) u =
      dcondSet (boxTag 0 (squareFst u) ∪ boxTag 1 (squareSnd u))
        (squarePair (funOf (boxShift a) (squareFst u)) botElem)
        (squarePair botElem (funOf (boxShift b) (squareSnd u))) :=
  beta (map_isScottContinuous a b) u

theorem app_empty {a b u : Pomega} (hx : squareFst u = botElem)
    (hy : squareSnd u = botElem) :
    funOf (boxPlus a b) u = botElem := by
  rw [app, hx, hy, boxTag.bot, boxTag.bot, Set.union_self, dcondSet.bot]

theorem app_left {a b u : Pomega} (hx : squareFst u = botElem)
    (hy : squareSnd u = botElem) :
    funOf (boxPlus a b) u =
      squarePair (funOf (boxShift a) (squareFst u)) botElem := by
  obtain ⟨m, hm⟩ := exists_mem_of_ne_bot hx
  rw [app, boxTag.of_mem hm, hy, boxTag.bot, union_botElem,
    dcondSet.ofNat_zero]

theorem app_right {a b u : Pomega} (hx : squareFst u = botElem)
    (hy : squareSnd u = botElem) :
    funOf (boxPlus a b) u =
      squarePair botElem (funOf (boxShift b) (squareSnd u)) := by
  obtain ⟨m, hm⟩ := exists_mem_of_ne_bot hy
  rw [app, boxTag.of_mem hm, hx, boxTag.bot, botElem_union,
    dcondSet.ofNat_one]

theorem app_mixed {a b u : Pomega} (hx : squareFst u = botElem)
    (hy : squareSnd u = botElem) :
    funOf (boxPlus a b) u = topElem := by
  obtain ⟨m, hm⟩ := exists_mem_of_ne_bot hx
  obtain ⟨k, hk⟩ := exists_mem_of_ne_bot hy
  rw [app, boxTag.of_mem hm, boxTag.of_mem hk]
  exact dcondSet.of_mixed (by simp [ofNat]) ⟨0, by simp [ofNat]⟩

theorem app_bot (a b : Pomega) :
    funOf (boxPlus a b) botElem = botElem :=
  app_empty squareFst.bot squareSnd.bot

theorem app_top (a b : Pomega) :
    funOf (boxPlus a b) topElem = topElem :=
  app_mixed (by rw [squareFst.top]; exact topElem_ne_botElem)
    (by rw [squareSnd.top]; exact topElem_ne_botElem)

theorem app_app {a b : Pomega} (ha : Retract a) (hb : Retract b)
    (u : Pomega) :
    funOf (boxPlus a b) (funOf (boxPlus a b) u) = funOf (boxPlus a b) u := by
  by_cases hx : squareFst u = botElem <;> by_cases hy : squareSnd u = botElem
  · rw [app_empty hx hy, app_bot]
```

```
  · rw [app_right hx hy,
      app_right (u := squarePair botElem (funOf (boxShift b) (squareSnd u)))
        (squareFst_of_pair _ _) (by rw [squareSnd_of_pair]; exact boxShift.app_ne_bot
    b _),
      squareSnd_of_pair, retract_app (boxShift.isRetract hb)]
  · rw [app_left hx hy,
      app_left (u := squarePair (funOf (boxShift a) (squareFst u)) botElem)
        (by rw [squareFst_of_pair]; exact boxShift.app_ne_bot a _) (squareSnd_of_pair
    _ _),
      squareFst_of_pair, retract_app (boxShift.isRetract ha)]
  · rw [app_mixed hx hy, app_top]

theorem isRetract {a b : Pomega} (ha : Retract a) (hb : Retract b) :
    Retract (boxPlus a b) := by
  constructor
  change boxPlus a b =
    graph (fun u => funOf (boxPlus a b) (funOf (boxPlus a b) u))
  apply graph_ext
  intro u
  rw [app_app ha hb u, app]

theorem expansive {a b : Pomega} (ha : Closure a) (hb : Closure b)
    (u : Pomega) : u ⊆ funOf (boxPlus a b) u := by
  have h8 : squarePair (squareFst u) (squareSnd u) = u := squarePair_eta u
  by_cases hx : squareFst u = botElem <;> by_cases hy : squareSnd u = botElem
  · rw [hx, hy, squarePair.bot_bot] at h8
    rw [app_empty hx hy, ← h8]
  · rw [app_right hx hy]
    have hsub : squarePair (squareFst u) (squareSnd u) ⊆
        squarePair botElem (funOf (boxShift b) (squareSnd u)) := by
      rw [hx]
      exact squarePair.mono subset_rfl (boxShift.expansive hb (squareSnd u))
    rwa [h8] at hsub
  · rw [app_left hx hy]
    have hsub : squarePair (squareFst u) (squareSnd u) ⊆
        squarePair (funOf (boxShift a) (squareFst u)) botElem := by
      rw [hy]
      exact squarePair.mono (boxShift.expansive ha (squareFst u)) subset_rfl
    rwa [h8] at hsub
  · rw [app_mixed hx hy]
    exact fun k _ => Set.mem_univ k

theorem typed_bot (a b : Pomega) :
    typed botElem (boxPlus a b) :=
  (app_bot a b).symm

theorem typed_top (a b : Pomega) :
    typed topElem (boxPlus a b) :=
  (app_top a b).symm

theorem typed_inl {a b x : Pomega} (hx : typed x (boxShift a)) :
    typed (squarePair x botElem) (boxPlus a b) := by
  have hx0 : x = botElem := by
    rw [hx]; exact boxShift.app_ne_bot a x
  rw [typed, app_left (by rw [squareFst_of_pair]; exact hx0) (squareSnd_of_pair _ _),
    squareFst_of_pair,
```

```
    ← hx]

theorem typed_inr {a b y : Pomega} (hy : typed y (boxShift b)) :
    typed (squarePair botElem y) (boxPlus a b) := by
  have hy0 : y = botElem := by
    rw [hy]; exact boxShift.app_ne_bot b y
  rw [typed, app_right (squareFst_of_pair _ _) (by rw [squareSnd_of_pair]; exact hy0),
     squareSnd_of_pair,
    ← hy]

theorem typed_iff {a b u : Pomega} (ha : Retract a) (hb : Retract b) :
    typed u (boxPlus a b) ↔
      u = botElem ∨ u = topElem ∨
        (∃ x, u = squarePair x botElem ∧ typed x (boxShift a)) ∨
          (∃ y, u = squarePair botElem y ∧ typed y (boxShift b)) := by
  constructor
  · intro hu
    have hu' : u = funOf (boxPlus a b) u := hu
    by_cases hx : squareFst u = botElem <;> by_cases hy : squareSnd u = botElem
    · exact Or.inl (hu'.trans (app_empty hx hy))
    · refine Or.inr (Or.inr (Or.inr ⟨funOf (boxShift b) (squareSnd u), ?_, ?_⟩))
      · exact hu'.trans (app_right hx hy)
      · exact (retract_app (boxShift.isRetract hb) (squareSnd u)).symm
    · refine Or.inr (Or.inr (Or.inl ⟨funOf (boxShift a) (squareFst u), ?_, ?_⟩))
      · exact hu'.trans (app_left hx hy)
      · exact (retract_app (boxShift.isRetract ha) (squareFst u)).symm
    · exact Or.inr (Or.inl (hu'.trans (app_mixed hx hy)))
  · intro h
    rcases h with hu | hu | ⟨x, hx, hxt⟩ | ⟨y, hy, hyt⟩
    · rw [hu]; exact typed_bot a b
    · rw [hu]; exact typed_top a b
    · rw [hx]; exact typed_inl hxt
    · rw [hy]; exact typed_inr hyt

end boxPlus

end DomainTheory.DataTypesAsLattices
```

Icomb_subset_boxPlus.lean

```
module

public import Mathlib.Algebra.Ring.Parity
public import DomainTheory.DataTypesAsLattices.Retracts.Icomb
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Closures.Closure
public import DomainTheory.DataTypesAsLattices.Closures.boxPlus
public import DomainTheory.DataTypesAsLattices.Closures.Icomb_subset_graph

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem Icomb_subset_boxPlus {a b : Pomega} (ha : Closure a) (hb : Closure b) :
```

```
    Icomb ⊆ boxPlus a b :=
  Icomb_subset_graph.mpr fun n => by
    rw [← boxPlus.app]
    exact boxPlus.expansive ha hb (e n)

end DomainTheory.DataTypesAsLattices
```

Icomb_subset_boxTensor.lean

```
module

public import Mathlib.Algebra.Ring.Parity
public import DomainTheory.DataTypesAsLattices.Retracts.Icomb
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Closures.Closure
public import DomainTheory.DataTypesAsLattices.Closures.squarePair
public import DomainTheory.DataTypesAsLattices.Closures.squareFst
public import DomainTheory.DataTypesAsLattices.Closures.squareSnd
public import DomainTheory.DataTypesAsLattices.Closures.boxTensor
public import DomainTheory.DataTypesAsLattices.Closures.eq_5_8
public import DomainTheory.DataTypesAsLattices.Closures.squarePair_more
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem Icomb_subset_boxTensor {a b : Pomega} (ha : Closure a) (hb : Closure b) :
    Icomb ⊆ boxTensor a b := by
  intro p hp
  rcases hp with ⟨n, m, rfl, hm⟩
  have hsub : squarePair (squareFst (e n)) (squareSnd (e n)) ⊆
      squarePair (funOf a (squareFst (e n))) (funOf b (squareSnd (e n))) := by
    refine squarePair.mono ?_ ?_
    · have := funOf.monotone_left ha.out.1 (squareFst (e n))
      rwa [Icomb.app] at this
    · have := funOf.monotone_left hb.out.1 (squareSnd (e n))
      rwa [Icomb.app] at this
  refine ⟨n, m, rfl, ?_⟩
  exact hsub (by simpa [eq_5_8] using hm)

end DomainTheory.DataTypesAsLattices
```

eq_5_12.lean

```
module

public import Mathlib.Algebra.Ring.Parity
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Closures.squarePair
public import DomainTheory.DataTypesAsLattices.Closures.squareFst
public import DomainTheory.DataTypesAsLattices.Closures.squareSnd
```

```
public import DomainTheory.DataTypesAsLattices.Closures.boxTensor
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (5.12).** Unfolding of the boxed product. -/
theorem eq_5_12 (a b : Pomega) :
    boxTensor a b = graph (fun u =>
      squarePair (funOf a (squareFst u)) (funOf b (squareSnd u))) :=
  rfl

end DomainTheory.DataTypesAsLattices
```

eq_5_13.lean

```
module

public import Mathlib.Algebra.Ring.Parity
public import DomainTheory.DataTypesAsLattices.Lambda.dcondSet
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Closures.squarePair
public import DomainTheory.DataTypesAsLattices.Closures.squareFst
public import DomainTheory.DataTypesAsLattices.Closures.squareSnd
public import DomainTheory.DataTypesAsLattices.Closures.boxShift
public import DomainTheory.DataTypesAsLattices.Closures.boxPlus
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (5.13).** Unfolding of the boxed sum. -/
theorem eq_5_13 (a b : Pomega) :
    boxPlus a b = graph (fun u =>
      dcondSet (condSet (squareFst u) (ofNat 0) (ofNat 0) ∪
          condSet (squareSnd u) (ofNat 1) (ofNat 1))
        (squarePair (funOf (boxShift a) (squareFst u)) botElem)
        (squarePair botElem (funOf (boxShift b) (squareSnd u)))) :=
  rfl

end DomainTheory.DataTypesAsLattices
```

eq_5_14.lean

```
module

public import Mathlib.Algebra.Ring.Parity
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Closures.Vapply
public import DomainTheory.DataTypesAsLattices.Closures.VapplyY
public import DomainTheory.DataTypesAsLattices.Closures.Vapply_more

@[expose] public section
```

```
namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (5.14) = (5.15).** -/
theorem eq_5_14 (a x : Pomega) : VapplyY a x = Vapply a x :=
  Vapply.eq_Y a x

end DomainTheory.DataTypesAsLattices
```

eq_5_2.lean

```
module

public import Mathlib.Algebra.Ring.Parity
public import DomainTheory.DataTypesAsLattices.Retracts.arrowR
public import DomainTheory.DataTypesAsLattices.Retracts.Icomb
public import DomainTheory.DataTypesAsLattices.Closures.Icomb_isClosure
public import DomainTheory.DataTypesAsLattices.Closures.Icomb_subset_arrowR

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (5.2).** `I ⊆ I o→ I`. -/
theorem eq_5_2 : Icomb ⊆ arrowR Icomb Icomb :=
  Icomb_subset_arrowR Icomb_isClosure Icomb_isClosure

end DomainTheory.DataTypesAsLattices
```

theorem_5_4.lean

```
module

public import Mathlib.Algebra.Ring.Parity
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Closures.Closure
public import DomainTheory.DataTypesAsLattices.Closures.boxTensor
public import DomainTheory.DataTypesAsLattices.Closures.Icomb_subset_boxTensor
public import DomainTheory.DataTypesAsLattices.Closures.boxPlus
public import DomainTheory.DataTypesAsLattices.Closures.Icomb_subset_boxPlus

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, Theorem 5.4 (product half).** The boxed product of two
closures is a closure. -/
theorem theorem_5_4 {a b : Pomega} (ha : Closure a) (hb : Closure b) :
    Closure (boxTensor a b) :=
  ⟨⟨Icomb_subset_boxTensor ha hb, boxTensor.isRetract ha.out.2 hb.out.2⟩⟩

namespace theorem_5_4

/-- **Scott 1976, Theorem 5.4 (sum half).** If `a` and `b` are closure
operations then so is `a ? b`. -/
```

```
theorem plus {a b : Pomega} (ha : Closure a) (hb : Closure b) :
    Closure (boxPlus a b) :=
  ⟨⟨Icomb_subset_boxPlus ha hb, boxPlus.isRetract ha.out.2 hb.out.2⟩⟩

end theorem_5_4

end DomainTheory.DataTypesAsLattices
```

eq_5_25.lean

```
module

public import Mathlib.Algebra.Ring.Parity
public import DomainTheory.DataTypesAsLattices.Retracts.arrowR
public import DomainTheory.DataTypesAsLattices.Retracts.Icomb
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_4
public import DomainTheory.DataTypesAsLattices.Closures.selfArrowStep
public import DomainTheory.DataTypesAsLattices.Closures.dEq
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (5.25).** `d = I ∪ (d o→ d)`. -/
theorem eq_5_25 : dEq = Icomb ∪ arrowR dEq dEq :=
  (theorem_1_4 selfArrowStep.isScottContinuous).1.symm

namespace eq_5_25

theorem arrow (hI : Icomb ⊆ arrowR dEq dEq) :
    dEq = arrowR dEq dEq := by
  apply subset_antisymm
  · nth_rw 1 [eq_5_25]
    exact Set.union_subset hI subset_rfl
  · nth_rw 3 [eq_5_25]
    exact Set.subset_union_right

end eq_5_25

end DomainTheory.DataTypesAsLattices
```

openR_map_isScottContinuous.lean

```
module

public import Mathlib.Algebra.Ring.Parity
public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.mem_scottUnion
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
@[expose] public section
```

```
namespace DomainTheory.DataTypesAsLattices

theorem openR_map_isScottContinuous :
    IsScottContinuous (fun u => {m | ∃ n, e n ⊆ e m ∧ n ∈ u}) := by
  constructor
  intro u
  ext m
  constructor
  · intro ⟨n, hn, hnu⟩
    exact mem_scottUnion.mpr ⟨2 ^ n, by simp [e_pow2, Set.singleton_subset_iff, hnu],
      ⟨n, hn, by simp [e_pow2]⟩⟩
  · intro hm
    obtain ⟨k, hk, n, hn, hnk⟩ := mem_scottUnion.mp hm
    exact ⟨n, hn, hk hnk⟩

end DomainTheory.DataTypesAsLattices
```

openR_app.lean

```
module

public import Mathlib.Algebra.Ring.Parity
public import DomainTheory.DataTypesAsLattices.Retracts.openR
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Closures.openR_map_isScottContinuous

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem openR_app (u : Pomega) :
    funOf openR u = {m | ∃ n, e n ⊆ e m ∧ n ∈ u} :=
  beta openR_map_isScottContinuous u

end DomainTheory.DataTypesAsLattices
```

openR_expansive.lean

```
module

public import Mathlib.Algebra.Ring.Parity
public import DomainTheory.DataTypesAsLattices.Retracts.openR
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Closures.openR_app

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem openR_expansive (u : Pomega) : u ⊆ funOf openR u := by
  intro k hk
  rw [openR_app]
```

```
  exact ⟨k, subset_rfl, hk⟩

end DomainTheory.DataTypesAsLattices
```

theorem_5_1.lean

```
module

public import Mathlib.Algebra.Ring.Parity
public import DomainTheory.DataTypesAsLattices.Retracts.Fixpoints
public import DomainTheory.DataTypesAsLattices.Retracts.DirectedSubset
public import DomainTheory.DataTypesAsLattices.Retracts.IsCompactSubset
public import DomainTheory.DataTypesAsLattices.Retracts.typed
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Closures.Closure
public import DomainTheory.DataTypesAsLattices.Closures.Isolated
public import DomainTheory.DataTypesAsLattices.Closures.closureCompact
public import DomainTheory.DataTypesAsLattices.Closures.compactFixedBelow
public import DomainTheory.DataTypesAsLattices.Closures.compact_fixed_iff
public import DomainTheory.DataTypesAsLattices.Retracts.vals
public import DomainTheory.DataTypesAsLattices.Closures.compactFixedBelow_more
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, Theorem 5.1 (The algebraic lattice theorem).**
The compact points of a closure range are exactly the `a(e n)`, and every
fixed point is their directed supremum. -/
theorem theorem_5_1 {a : Pomega} (ha : Closure a) :
    (∀ x : Fixpoints (funOf a),
      IsCompactSubset x ↔ ∃ n, x = closureCompact a ha n) ∧
    ∀ x : Fixpoints (funOf a),
      DirectedSubset (compactFixedBelow a x) ∧
        (x : Pomega) = ⋃₀ (Subtype.val '' compactFixedBelow a x) :=
  ⟨compact_fixed_iff.subset ha, fun x =>
    ⟨compactFixedBelow.directedSubset ha x, compactFixedBelow.sUnion ha x⟩⟩

/-- Isolated points of a closure are its values on finite sets, and
conversely every `a(e n)` is a typed point. -/
theorem theorem_5_1.isolated {a : Pomega} (_ha : Closure a) {x : Pomega}
    (hx : typed x a) :
    Isolated x → ∃ n, x = funOf a (e n) := by
  intro h
  obtain ⟨n, hn⟩ := h.out
  refine ⟨n, ?_⟩
  rw [hn] at hx ⊢
  exact hx

end DomainTheory.DataTypesAsLattices
```

theorem_5_2.lean

```
module
```

```
public import Mathlib.Algebra.Ring.Parity
public import DomainTheory.DataTypesAsLattices.Retracts.Fixpoints
public import Mathlib.Order.CompleteLattice.Defs
public import DomainTheory.DataTypesAsLattices.FixedLattices.CountablyAlgebraic
public import DomainTheory.DataTypesAsLattices.FixedLattices.
    CountablyAlgebraicu2ecompact_finite_support
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Closures.Closure
public import DomainTheory.DataTypesAsLattices.Closures.representClosure
public import DomainTheory.DataTypesAsLattices.Closures.representClosureCode
public import DomainTheory.DataTypesAsLattices.Closures.representationOrderIso
public import DomainTheory.DataTypesAsLattices.Retracts.vals
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, Theorem 5.2 (The representation theorem).**
The constructed code is a continuous closure operation and its fixed-point
range is order-isomorphic to the source lattice. -/
theorem theorem_5_2 {α : Type u} [CompleteLattice α]
    (D : CountablyAlgebraic α) :
    IsScottContinuous (representClosure D) ∧
      Closure (representClosureCode D) ∧
      ∃ φ : α → Fixpoints (funOf (representClosureCode D)),
        Function.Bijective φ ∧
          ∀ x y, x ≤ y ↔ (φ x : Pomega) ⊆ φ y :=
  ⟨representClosure.isScottContinuous D,
    representClosureCode.isClosure D,
    ⟨fun x => representationOrderIso D x,
      (representationOrderIso D).toEquiv.bijective,
      fun x y => by
        constructor
        · intro hxy
          exact (representationOrderIso D).map_rel_iff.mpr hxy
        · intro hsub
          exact (representationOrderIso D).map_rel_iff.mp hsub⟩⟩

end DomainTheory.DataTypesAsLattices
```

theorem_4_1_more.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import DomainTheory.DataTypesAsLattices.FixedLattices.DirectedSet
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
```

```
public import DomainTheory.DataTypesAsLattices.Embedding.extend
public import DomainTheory.DataTypesAsLattices.Embedding.embed
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.Retract
public import DomainTheory.DataTypesAsLattices.Retracts.Fixpoints
public import DomainTheory.DataTypesAsLattices.Retracts.DirectedSubset
public import DomainTheory.DataTypesAsLattices.Retracts.typed
public import DomainTheory.DataTypesAsLattices.Retracts.lfpAbove
public import DomainTheory.DataTypesAsLattices.Retracts.gfpBelow
public import DomainTheory.DataTypesAsLattices.Retracts.vals
public import DomainTheory.DataTypesAsLattices.Retracts.retractBasis
public import DomainTheory.DataTypesAsLattices.Retracts.directed_finite_cover
public import DomainTheory.DataTypesAsLattices.Retracts.theorem_4_1

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, Theorem 4.1 (The lattice theorem).**
Fixed points of a continuous function form a complete lattice under `⊆`;
those of a retract form a continuous lattice. -/
theorem theorem_4_1 {f : Pomega → Pomega} (hf : IsScottContinuous f) :
    (∀ A : Set Pomega, A ⊆ Fixpoints f →
      gfpBelow f (⋂₀ A) ∈ Fixpoints f ∧
        lfpAbove f (⋃₀ A) ∈ Fixpoints f) ∧
      ∀ {a} (_ha : Retract a),
        Fixpoints (funOf a) = {x | typed x a} ∧
          ∀ x : Fixpoints (funOf a),
            DirectedSubset (retractBasis a x) ∧
              (x : Pomega) = ⋃₀ (Subtype.val '' retractBasis a x) := by
  constructor
  · intro A hA
    exact ⟨theorem_4_1.complete f hf A hA, lfpAbove.mem hf hA⟩
  · intro a ha
    constructor
    · ext x
      simp [Fixpoints, typed, eq_comm]
    · intro x
      exact ⟨retractBasis.directedSubset ha x, retractBasis.eq_sUnion ha x⟩

/-- **Scott 1976, Theorem 4.1, continuous-lattice half.**
The finite-image points form a directed way-below basis for every point in
the range of a retract. Stated with subset directedness so the compared
type does not mention a `CompleteLattice` instance. -/
theorem theorem_4_1.continuous {a : Pomega} (ha : Retract a) :
    ∀ x : Fixpoints (funOf a),
      DirectedSubset (retractBasis a x) ∧
        (x : Pomega) = ⋃₀ (Subtype.val '' retractBasis a x) ∧
        ∀ y ∈ retractBasis a x,
          ∀ s : Set (Fixpoints (funOf a)),
            DirectedSubset s →
              (x : Pomega) ⊆ ⋃₀ (Subtype.val '' s) →
              ∃ z ∈ s, (y : Pomega) ⊆ z := by
  intro x
```

```
  refine ⟨retractBasis.directedSubset ha x, retractBasis.eq_sUnion ha x, ?_⟩
  intro y hy s hs hxs
  obtain ⟨n, hnx, hyn⟩ := hy
  have hen : e n ⊆ ⋃₀ (Subtype.val '' s) := hnx.trans hxs
  have hs' : DirectedSet s :=
    ⟨⟨hs.out.1, fun u hu v hv => by
      obtain ⟨w, hw, huw, hvw⟩ := hs.out.2 hu hv
      exact ⟨w, hw, huw, hvw⟩⟩⟩
  obtain ⟨z, hz, hnz⟩ :=
    directed_finite_cover hs' (e_finite n) (by simpa [Fixpoints.vals] using hen)
  refine ⟨z, hz, ?_⟩
  rw [hyn]
  have := IsScottContinuous.monotone (funOf.isScottContinuous a) hnz
  rwa [z.property] at this

end DomainTheory.DataTypesAsLattices
```

theorem_4_2.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.Retract
public import DomainTheory.DataTypesAsLattices.Retracts.retractLe

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, Theorem 4.2 (The partial ordering theorem).** -/
theorem theorem_4_2 {a b c : Pomega} :
    (Retract a → retractLe a a) ∧
      (retractLe a b → retractLe b a → a = b) ∧
      (retractLe a b → retractLe b c → retractLe a c) := by
  refine ⟨retractLe.refl, ?_, ?_⟩
  · intro hab hba
    exact hab.1.trans (hba.2.symm)
  · intro hab hbc
    exact retractLe.trans hab hbc

end DomainTheory.DataTypesAsLattices
```

2.7 Total functions and functionality

Historical note. Functionality had long been studied in combinatory logic. Hindley, Lercher, and Seldin (1972) is the introduction Scott used. This section is his reading of that theory in $\mathbf{P}\omega$. The connection with the functional interpretation of intuitionistic logic goes back to Curry and Kleene, and Theorem 7.3 is the standard result of that interpretation. Theorem 7.4, that the obvious typing characterizes the iterators, is due to Gordon Plotkin.

Dependencies. Inside Functionality, each box is one Lean file. A box named as another folder in this chapter is that whole folder. A box named as another chapter is that whole chapter. The arrowhead points at the file or folder that imports, and the number is how many direct imports.

Figure 25: Total functions and functionality.

There is an inevitable conflict between the concepts of *total* and *partial* functions: we desire the former, but it is the latter we usually get. Total functions are better because they are "well-defined" at all their arguments, but the rub is that there is no general way of deciding when a *definition* is going to be well-defined in all its uses. In analysis we have singularities, and in recursion theory we have endless, nonfinishing computations. In the present theory we have in effect evaded the question in two ways. First we have embraced the partial function as the norm. But secondly, and possibly confusingly, the multiple-valued functions are normal, total functions from $\mathbf{P}\omega$ into $\mathbf{P}\omega$. The point, of course, is that we are making a *model* of the partial functions in terms of ordinary mathematical functions. But note that the success of the *model* lies in *not* using arbitrary functions: it is only the continuous functions that correspond to the kind of partial functions we wanted to study. It would be a mistake to think of the variables in λ-calculus as ranging over arbitrary functions—and this mistake was made by both Church and Curry. The fixed-point operator $\mathbf{Y}$ shows that we must restrict attention to functions which do have fixed points. It is certainly the case that $\mathbf{P}\omega$ is not the only model for the λ-calculus, but it is a very satisfactory model and is rich enough to illustrate what can and what cannot be done with partial functions.

Whatever the pleasures of partial functions (and the multiple-valued ones, too), the desire for total functions remains. Take the integers. We are more interested in ω than $\omega \cup \{\bot, \top\}$. Since the multiple values $\bot$ and $\top$ are but two in number, it is easy to avoid them. The problem becomes tiresome in considering functions, however. The lattice represented by the retract $\mathbf{int}\circ \to \mathbf{int}$ is much too large, in that there are as many nontotal functions in this domain as total ones. The aim of the present section is to introduce an interpretation of a theory of functionality in the model $\mathbf{P}\omega$ that provides a convenient way of restricting attention to the functions (or other objects) that are total in the desired sense. The theory of functionality is rather like proposals of Curry, but not quite the same for important reasons as we will see.

In the theory of retracts of §§ 5 and 6, the plan of "restricting attention" was the very simple one of restricting to a *subset*. It was made notationally simple as the subsets in question could be parameterized by continuous functions. The retraction mappings stand in for their ranges. Even better, certain continuous functions act on these retractions as space-forming functors (such as $\oplus$ and $\circ \to$), which gives greater notational simplicity because one language is able to serve for several tasks. When we pass to the theory of total functions, this same kind of simplicity is no longer possible owing to an increase in quantifier complexity in the necessary definitions. (This remark is made definite below.) Another point where there is some loss of simplicity concerns the representation of entities in $\mathbf{P}\omega$: subsets will no longer be enough, since we will need *quotients* of subsets. This is not a very startling point. Many constructions are affected in a natural way via equivalence classes. An equivalence relation makes you blind to certain distinctions. It may be easier also to remain a bit blind than to search for the most beautiful representative of an equivalence class: there may be nothing to choose between several candidates, and it can cost too much effort to attempt a choice. Thus our first agreement is that for many purposes a *kind* of object can be taken as a set of equivalence classes for an equivalence relation on a subset of $\mathbf{P}\omega$.

Because $\mathbf{P}\omega$ is closed under the pairing function $\lambda x \lambda y.\, \langle x, y\rangle$, we shall construe relations on subsets of $\mathbf{P}\omega$ as subsets of $\mathbf{P}\omega$ all of whose elements are ordered pairs. That is, a relation A satisfies this inclusion:

$$(7.1) \quad A \subseteq \{\langle x, y\rangle \mid x, y \in \mathbf{P}\omega\}.$$

- Definition (equivalence)

- Theorem 7.1
- Theorem 7.2
- Theorem 7.3
- Theorem 7.4

sigmaJ.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Graph.theorem_1_2
public import DomainTheory.DataTypesAsLattices.Continuous.mem_scottUnion

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (7.17).** Independent successor functions. -/
def sigmaJ (j : ℕ) (x : Pomega) : Pomega :=
  {p | ∃ k, pair j (k + 1) = p ∧ pair j k ∈ x}

namespace sigmaJ

theorem isScottContinuous (j : ℕ) : IsScottContinuous (sigmaJ j) := by
  constructor
  intro x
  ext p
  constructor
  · intro ⟨k, heq, hp⟩
    refine mem_scottUnion.mpr ⟨2 ^ pair j k, ?_, k, heq, ?_⟩
    · simpa [e_pow2, Set.singleton_subset_iff] using hp
    · simp [e_pow2]
  · intro hp
    obtain ⟨n, hn, k, heq, hpk⟩ := mem_scottUnion.mp hp
    exact ⟨k, heq, hn hpk⟩

theorem app (j : ℕ) (x : Pomega) :
    funOf (graph (sigmaJ j)) x = sigmaJ j x := by
  simpa using congrArg (fun g => g x) (theorem_1_2.i (isScottContinuous j))

theorem bot (j : ℕ) : sigmaJ j botElem = botElem := by
  ext p
  simp [sigmaJ, botElem]
```

```
theorem of_no_tag {j : ℕ} {u : Pomega}
    (hu : ∀ k, pair j k ∈ u) :
    sigmaJ j u = botElem := by
  ext p
  constructor
  · intro ⟨k, _hp, hk⟩
    exact (hu k hk).elim
  · simp [botElem]

theorem pair_union (j j' m : ℕ) (hne : j = j') :
    sigmaJ j (({pair j m} : Pomega) ∪ {pair j' m}) =
      ({pair j (m + 1)} : Pomega) := by
  ext p
  constructor
  · intro ⟨k, hp, hk⟩
    rcases hk with hk | hk
    · have hkm : k = m := (pair_inj hk).2
      subst k
      simpa using hp.symm
    · have hj : j = j' := (pair_inj hk).1
      exact (hne hj).elim
  · intro hp
    exact ⟨m, hp.symm, Or.inl (by simp)⟩

theorem succ_ne (j j' m : ℕ) (hne : j = j') :
    sigmaJ j {pair j' m} = botElem := by
  ext p
  constructor
  · intro ⟨k, _hk, hp⟩
    have : pair j k = pair j' m := by
      simpa [Set.mem_singleton_iff] using hp
    exact (hne (pair_inj this).1).elim
  · intro hp
    simp [botElem] at hp

theorem succ_singleton (j m : ℕ) :
    sigmaJ j {pair j m} = ({pair j (m + 1)} : Pomega) := by
  ext p
  constructor
  · intro ⟨k, heq, hp⟩
    have : k = m := (pair_inj hp).2
    subst this
    simp [heq]
  · intro hp
    exact ⟨m, hp.symm, by simp⟩

theorem union_fresh {j m : ℕ} {u : Pomega}
    (hu : ∀ k, pair j k ∈ u) :
    sigmaJ j (u ∪ {pair j m}) = ({pair j (m + 1)} : Pomega) := by
  rw [show u ∪ {pair j m} = ({pair j m} : Pomega) ∪ u by
    exact Set.union_comm _ _]
  ext p
  constructor
  · intro ⟨k, hp, hk⟩
    rcases hk with hk | hk
```

```
    · have hkm : k = m := (pair_inj hk).2
      subst k
      simpa using hp.symm
    · exact (hu k hk).elim
  · intro hp
    exact ⟨m, hp.symm, Or.inl (by simp)⟩

end sigmaJ

end DomainTheory.DataTypesAsLattices
```

LambTerm.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.lam

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (4.43).** Modified LAMBDA terms for the `lamb` model. -/
inductive LambTerm where
  | var : ℕ → LambTerm
  | zero : LambTerm
  | succ : LambTerm → LambTerm
  | pred : LambTerm → LambTerm
  | condq : LambTerm → LambTerm → LambTerm → LambTerm → LambTerm
  | app : LambTerm → LambTerm → LambTerm
  | lam : ℕ → LambTerm → LambTerm

end DomainTheory.DataTypesAsLattices
```

RestrictedEquiv.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, §7, Definition.** A restricted equivalence is a
symmetric transitive relation, coded as a set of pairs. -/
structure RestrictedEquiv where
  rel : Pomega → Pomega → Prop
  symm : ∀ {x y}, rel x y → rel y x
  trans : ∀ {x y z}, rel x y → rel y z → rel x z

end DomainTheory.DataTypesAsLattices
```

continuous_on_diag.lean

```
module
```

```
public import Mathlib.Order.CompleteLattice.Defs
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.mem_scottUnion
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- Diagonal continuity, packaged from Theorem 1.3. -/
theorem continuous_on_diag {F : Pomega → Pomega → Pomega}
    (h1 : ∀ y, IsScottContinuous (fun x => F x y))
    (h2 : ∀ x, IsScottContinuous (fun y => F x y)) :
    IsScottContinuous (fun x => F x x) := by
  constructor
  intro x
  have H := theorem_1_3.diag (x := x) (f := F) h1 h2
  ext k
  constructor
  · intro hk
    have : k ∈ ⋃ n, {k | e n ⊆ x ∧ k ∈ F (e n) (e n)} := by rwa [← H]
    obtain ⟨n, hn, hkn⟩ := Set.mem_iUnion.mp this
    exact mem_scottUnion.mpr ⟨n, hn, hkn⟩
  · intro hk
    obtain ⟨n, hn, hkn⟩ := mem_scottUnion.mp hk
    have : k ∈ ⋃ n, {k | e n ⊆ x ∧ k ∈ F (e n) (e n)} :=
      Set.mem_iUnion.mpr ⟨n, hn, hkn⟩
    rwa [← H] at this

end DomainTheory.DataTypesAsLattices
```

continuous_tuple.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.continuous_on_diag

@[expose] public section

namespace DomainTheory.DataTypesAsLattices
```

```
theorem continuous_tuple {F : Pomega → Pomega → Pomega}
    (h1 : ∀ y, IsScottContinuous (fun x => F x y))
    (h2 : ∀ x, IsScottContinuous (fun y => F x y))
    {g h : Pomega → Pomega} (hg : IsScottContinuous g) (hh : IsScottContinuous h) :
    IsScottContinuous (fun x => F (g x) (h x)) :=
  continuous_on_diag
    (fun v => theorem_1_3 (h1 (h v)) hg)
    (fun u => theorem_1_3 (h2 (g u)) hh)

end DomainTheory.DataTypesAsLattices
```

arrowR_diag_isScottContinuous.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.arrowR
public import DomainTheory.DataTypesAsLattices.Retracts.comp
public import DomainTheory.DataTypesAsLattices.Retracts.graph_ext
public import Mathlib.Order.CompleteLattice.Defs
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Lambda.id_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Functionality.continuous_tuple
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem arrowR_diag_isScottContinuous :
    IsScottContinuous (fun z => arrowR z z) := by
  apply graph_const_isScottContinuous
  intro u
  have h :
      IsScottContinuous (fun z =>
        graph (fun x => funOf z (funOf u (funOf z x)))) :=
    graph_const_isScottContinuous
      (fun z x => funOf z (funOf u (funOf z x)))
      (fun x =>
        continuous_tuple
          (fun y => funOf.isScottContinuous_left (funOf u (funOf y x)))
          (fun z =>
            theorem_1_3 (funOf.isScottContinuous z)
              (theorem_1_3 (funOf.isScottContinuous u)
                (funOf.isScottContinuous_left x)))
          id_isScottContinuous
          id_isScottContinuous)
  convert h using 1
```

```
  funext z
  apply graph_ext
  intro x
  simp only [comp.app]

end DomainTheory.DataTypesAsLattices
```

continuous_nary.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.continuous_on_diag
public import DomainTheory.DataTypesAsLattices.Functionality.continuous_tuple

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem continuous_nary {F : Pomega → Pomega → Pomega → Pomega}
    (hf0 : ∀ y z, IsScottContinuous (fun x => F x y z))
    (hf1 : ∀ x z, IsScottContinuous (fun y => F x y z))
    (hf2 : ∀ x y, IsScottContinuous (fun z => F x y z))
    {g h i : Pomega → Pomega}
    (hg : IsScottContinuous g) (hh : IsScottContinuous h) (hi : IsScottContinuous i) :
    IsScottContinuous (fun x => F (g x) (h x) (i x)) :=
  continuous_on_diag
    (fun v => continuous_tuple (fun y => hf0 y (i v)) (fun x => hf1 x (i v)) hg hh)
    (fun u => theorem_1_3 (hf2 (g u) (h u)) hi)

end DomainTheory.DataTypesAsLattices
```

expPayload.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def expPayload (e : Pomega) : Pomega := funOf e (ofNat 1)

end DomainTheory.DataTypesAsLattices
```

expTag.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- Tag and payload of a seven-sum abstract-syntax code. -/
def expTag (e : Pomega) : Pomega := funOf e (ofNat 0)

end DomainTheory.DataTypesAsLattices
```

hApply.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.expPayload
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- Recursive call of a candidate `?` on a subexpression. -/
def hApply (H e t : Pomega) : Pomega := funOf (funOf H e) t

namespace hApply

theorem isScottContinuous_H (e t : Pomega) :
    IsScottContinuous (fun H => hApply H e t) :=
  theorem_1_3 (f := fun u => funOf u t) (g := fun H => funOf H e)
    (funOf.isScottContinuous_left t) (funOf.isScottContinuous_left e)

theorem isScottContinuous_e (H t : Pomega) :
    IsScottContinuous (fun e => hApply H e t) :=
  theorem_1_3 (f := fun u => funOf u t) (g := funOf H)
    (funOf.isScottContinuous_left t) (funOf.isScottContinuous H)

theorem payload_isScottContinuous_e (H t : Pomega) (i : ℕ) :
    IsScottContinuous (fun e =>
      hApply H (funOf (expPayload e) (ofNat i)) t) :=
```

```
  theorem_1_3 (f := fun e => hApply H e t)
    (g := fun e => funOf (expPayload e) (ofNat i))
    (isScottContinuous_e H t)
    (theorem_1_3 (f := fun u => funOf u (ofNat i)) (g := expPayload)
      (funOf.isScottContinuous_left (ofNat i))
      (funOf.isScottContinuous_left (ofNat 1)))

end hApply

end DomainTheory.DataTypesAsLattices
```

hVar.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Functionality.expPayload
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def hVar (e t : Pomega) : Pomega := funOf t (expPayload e)

namespace hVar

theorem isScottContinuous_t (e : Pomega) :
    IsScottContinuous (fun t => hVar e t) :=
  funOf.isScottContinuous_left (expPayload e)

theorem isScottContinuous_e (t : Pomega) :
    IsScottContinuous (fun e => hVar e t) :=
  theorem_1_3 (f := funOf t) (g := expPayload)
    (funOf.isScottContinuous t) (funOf.isScottContinuous_left (ofNat 1))

end hVar

end DomainTheory.DataTypesAsLattices
```

inrightC.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.pairSeq
```

```
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (4.27).** Right injection. -/
def inrightC : Pomega := graph (fun x => pairSeq (ofNat 1) x)

end DomainTheory.DataTypesAsLattices
```

mem.lean

```
module

public import DomainTheory.DataTypesAsLattices.Embedding.extend
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.RestrictedEquiv

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- `x : A` means `x A x`. -/
def RestrictedEquiv.mem (A : RestrictedEquiv) (x : Pomega) : Prop :=
  A.rel x x

end DomainTheory.DataTypesAsLattices
```

Z.lean

```
module

public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Functionality.RestrictedEquiv
public import DomainTheory.DataTypesAsLattices.Functionality.continuous_tuple
public import DomainTheory.DataTypesAsLattices.Functionality.mem
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Lambda.const_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.id_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Pomega.pair
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (7.15)-(7.16).** Iterators `Z_n`. -/
def Z : ℕ → Pomega → Pomega → Pomega
  | 0, _, x => x
```

```
  | n + 1, f, x => funOf f (Z n f x)

namespace Z

theorem rel (A : RestrictedEquiv) (n : ℕ) {f f' x y : Pomega}
    (hf : ∀ a b, A.rel a b → A.rel (funOf f a) (funOf f' b))
    (hxy : A.rel x y) :
    A.rel (Z n f x) (Z n f' y) := by
  induction n with
  | zero => simpa [Z] using hxy
  | succ n ih =>
    simpa [Z] using hf _ _ ih

theorem finite_bound (p q m : ℕ) :
    ∀ k ∈ Z m (e p) (e q), k ≤ max p q := by
  induction m with
  | zero =>
      intro k hk
      exact (mem_e_le hk).trans (Nat.le_max_right p q)
  | succ m ih =>
      intro k hk
      obtain ⟨r, _hr, hpair⟩ := hk
      exact (pair_le_right r k).trans
        ((mem_e_le hpair).trans (Nat.le_max_left p q))

theorem finite_no_tag (p q m k : ℕ) :
    pair (max p q + 1) k ∈ Z m (e p) (e q) := by
  intro hk
  have hle := Z.finite_bound p q m _ hk
  exact (Nat.not_succ_le_self (max p q))
    ((pair_le_left (max p q + 1) k).trans hle)

theorem graph_funOf (n : ℕ) (f x : Pomega) :
    Z n (graph (fun y => funOf f y)) x = Z n f x := by
  induction n with
  | zero => simp [Z]
  | succ n ih =>
    simp [Z, ih]
    have : funOf (graph (fun y => funOf f y)) (Z n f x) = funOf f (Z n f x) :=
      beta (funOf.isScottContinuous f) (Z n f x)
    exact this

theorem isScottContinuous_left (n : ℕ) (x : Pomega) :
    IsScottContinuous (fun f => Z n f x) := by
  induction n with
  | zero => simpa [Z] using const_isScottContinuous x
  | succ n ih =>
    simpa [Z] using
      continuous_tuple
        (fun y => funOf.isScottContinuous_left y)
        (fun u => funOf.isScottContinuous u)
        id_isScottContinuous ih

theorem isScottContinuous_right (n : ℕ) (f : Pomega) :
    IsScottContinuous (fun x => Z n f x) := by
  induction n with
```

```
  | zero => simpa [Z] using id_isScottContinuous
  | succ n ih =>
    simpa [Z] using theorem_1_3 (funOf.isScottContinuous f) ih

end Z

end DomainTheory.DataTypesAsLattices
```

eq_7_18_ne.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.Z
public import DomainTheory.DataTypesAsLattices.Functionality.sigmaJ

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (7.18).** Distinct tags: `σ_j^{m+1}({(j',0)}) = ⊥`. -/
theorem eq_7_18_ne (j j' m : ℕ) (hne : j = j') :
    Z (m + 1) (graph (sigmaJ j)) {pair j' 0} = botElem := by
  induction m with
  | zero =>
    calc
      Z 1 (graph (sigmaJ j)) {pair j' 0}
        = funOf (graph (sigmaJ j)) {pair j' 0} := rfl
      _ = sigmaJ j {pair j' 0} := sigmaJ.app _ _
      _ = botElem := sigmaJ.succ_ne j j' 0 hne
  | succ m ih =>
    calc
      Z (m + 1 + 1) (graph (sigmaJ j)) {pair j' 0}
        = funOf (graph (sigmaJ j))
            (Z (m + 1) (graph (sigmaJ j)) {pair j' 0}) := rfl
      _ = sigmaJ j (Z (m + 1) (graph (sigmaJ j)) {pair j' 0}) :=
          sigmaJ.app _ _
      _ = sigmaJ j botElem := by rw [ih]
      _ = botElem := sigmaJ.bot j

end DomainTheory.DataTypesAsLattices
```

eq_7_18_eq.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.Z
public import DomainTheory.DataTypesAsLattices.Functionality.sigmaJ
```

```
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (7.18).** Equal tags: `σ_j^m({(j,0)}) = {(j,m)}`. -/
theorem eq_7_18_eq (j m : ℕ) :
    Z m (graph (sigmaJ j)) {pair j 0} = ({pair j m} : Pomega) := by
  induction m with
  | zero => rfl
  | succ m ih =>
    calc
      Z (m + 1) (graph (sigmaJ j)) {pair j 0}
        = funOf (graph (sigmaJ j)) (Z m (graph (sigmaJ j)) {pair j 0}) := rfl
      _ = sigmaJ j (Z m (graph (sigmaJ j)) {pair j 0}) := sigmaJ.app _ _
      _ = sigmaJ j {pair j m} := by rw [ih]
      _ = {pair j (m + 1)} := sigmaJ.succ_singleton j m

end DomainTheory.DataTypesAsLattices
```

Zcomb.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.graph_ext
public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.Z
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Graph.beta

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (7.15)-(7.16).** Combinators `Z_n = λf λx. f?(x)`. -/
def Zcomb (n : ℕ) : Pomega :=
  graph (fun f => graph (fun x => Z n f x))

namespace Zcomb

theorem map_isScottContinuous (n : ℕ) :
    IsScottContinuous (fun f => graph (fun x => Z n f x)) :=
  graph_const_isScottContinuous (fun f x => Z n f x)
    (fun x => Z.isScottContinuous_left n x)

theorem app (n : ℕ) (f : Pomega) :
    funOf (Zcomb n) f = graph (fun x => Z n f x) :=
```

```
  beta (Zcomb.map_isScottContinuous n) f

theorem app2 (n : ℕ) (f x : Pomega) :
    funOf (funOf (Zcomb n) f) x = Z n f x := by
  rw [Zcomb.app]
  exact beta (Z.isScottContinuous_right n f) x

theorem isGraph (n : ℕ) :
    Zcomb n = graph (fun f => funOf (Zcomb n) f) := by
  apply graph_ext
  intro f
  rw [Zcomb.app]

theorem app_isGraph (n : ℕ) (f : Pomega) :
    funOf (Zcomb n) f = graph (fun x => funOf (funOf (Zcomb n) f) x) := by
  rw [Zcomb.app]
  apply graph_ext
  intro x
  exact (beta (Z.isScottContinuous_right n f) x).symm

end Zcomb

end DomainTheory.DataTypesAsLattices
```

arrowE.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Functionality.RestrictedEquiv
public import DomainTheory.DataTypesAsLattices.Functionality.mem

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (7.6).** Function space of restricted equivalences. -/
def arrowE (A B : RestrictedEquiv) : RestrictedEquiv where
  rel := fun f g =>
    f = graph (fun x => funOf f x) ∧
      g = graph (fun x => funOf g x) ∧
        ∀ x y, A.rel x y → B.rel (funOf f x) (funOf g y)
  symm := by
    intro f g ⟨hf, hg, h⟩
    exact ⟨hg, hf, fun x y hxy => B.symm (h y x (A.symm hxy))⟩
  trans := by
    intro f g k ⟨hf, hg, hfg⟩ ⟨hg', hk, hgk⟩
    exact ⟨hf, hk, fun x y hxy =>
      B.trans (hfg x y hxy) (hgk y y (A.trans (A.symm hxy) hxy))⟩

end DomainTheory.DataTypesAsLattices
```

normalized_finite_app.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem normalized_finite_app (p : ℕ) (u : Pomega) :
    funOf (graph (fun y => funOf (e p) y)) u = funOf (e p) u :=
  beta (funOf.isScottContinuous (e p)) u

end DomainTheory.DataTypesAsLattices
```

finite_sigma_plain_app.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.eq_2_9
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.Z
public import DomainTheory.DataTypesAsLattices.Functionality.sigmaJ
public import DomainTheory.DataTypesAsLattices.Functionality.normalized_finite_app

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem finite_sigma_plain_app (p q m : ℕ) :
    funOf
        (graph (fun y => funOf (e p) y) ∪
          graph (sigmaJ (max p q + 1)))
        (Z m (e p) (e q)) =
      Z (m + 1) (e p) (e q) := by
  rw [eq_2_9, normalized_finite_app, sigmaJ.app,
    sigmaJ.of_no_tag (fun k => Z.finite_no_tag p q m k)]
  simp [botElem, Z]

end DomainTheory.DataTypesAsLattices
```

outC.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.sndC
public import DomainTheory.DataTypesAsLattices.Pomega.e
```

```
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (4.31).** Untagged payload; same combinator as `snd`. -/
def outC : Pomega := sndC

end DomainTheory.DataTypesAsLattices
```

pairSeq_app_one.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.pairSeq
public import DomainTheory.DataTypesAsLattices.Lambda.seq2
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem pairSeq_app_one (x y : Pomega) : funOf (pairSeq x y) (ofNat 1) = y :=
  seq2.one x y

end DomainTheory.DataTypesAsLattices
```

pairSeq_app_zero.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.pairSeq
public import DomainTheory.DataTypesAsLattices.Lambda.seq2
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem pairSeq_app_zero (x y : Pomega) : funOf (pairSeq x y) (ofNat 0) = x :=
  seq2.zero x y

end DomainTheory.DataTypesAsLattices
```

pairSeq_inj.lean

```
module
```

```
public import DomainTheory.DataTypesAsLattices.Retracts.pairSeq
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.pairSeq_app_zero
public import DomainTheory.DataTypesAsLattices.Functionality.pairSeq_app_one

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem pairSeq_inj {x y x' y' : Pomega} (h : pairSeq x y = pairSeq x' y') :
    x = x' ∧ y = y' := by
  constructor
  · have := congrArg (fun w => funOf w (ofNat 0)) h
    rw [pairSeq_app_zero, pairSeq_app_zero] at this
    exact this
  · have := congrArg (fun w => funOf w (ofNat 1)) h
    rw [pairSeq_app_one, pairSeq_app_one] at this
    exact this

end DomainTheory.DataTypesAsLattices
```

pairElem.lean

```
module

public import DomainTheory.DataTypesAsLattices.Functionality.pairSeq_inj
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.seq2
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.pairSeq
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- Pairing of elements of `Pω` as a graph (Table 2 / (2.21)). -/
def pairElem (x y : Pomega) : Pomega := seq2 x y

namespace pairElem

theorem tag_ne (x y : Pomega) :
    pairElem (ofNat 0) x = pairElem (ofNat 1) y :=
  seq2.tag_ne x y

theorem eq_pairSeq (x y : Pomega) : pairElem x y = pairSeq x y := rfl

theorem fst (x y : Pomega) : funOf (pairElem x y) (ofNat 0) = x :=
  seq2.zero x y

theorem snd (x y : Pomega) : funOf (pairElem x y) (ofNat 1) = y :=
  seq2.one x y
```

```
theorem inj {x y x' y' : Pomega} (h : pairElem x y = pairElem x' y') :
    x = x' ∧ y = y' :=
  pairSeq_inj h

end pairElem

end DomainTheory.DataTypesAsLattices
```

pairSeq_isScottContinuous_right.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.pairSeq
public import Mathlib.Order.CompleteLattice.Defs
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Lambda.predSet
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem pairSeq_isScottContinuous_right (x : Pomega) :
    IsScottContinuous (fun y => pairSeq x y) :=
  graph_const_isScottContinuous
    (fun y z => condSet z x (condSet (predSet z) y botElem))
    (fun z =>
      theorem_1_3 (condSet.isScottContinuous_right z x)
        (condSet.isScottContinuous_mid (predSet z) botElem))

end DomainTheory.DataTypesAsLattices
```

inleftC.lean

```
module

public import DomainTheory.DataTypesAsLattices.Functionality.
    pairSeq_isScottContinuous_right
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.pairSeq

@[expose] public section
```

```
namespace DomainTheory.DataTypesAsLattices

def inleftC : Pomega := graph (fun x => pairSeq (ofNat 0) x)

namespace inleftC

theorem app (x : Pomega) :
    funOf inleftC x = pairSeq (ofNat 0) x :=
  beta (pairSeq_isScottContinuous_right (ofNat 0)) x

end inleftC

end DomainTheory.DataTypesAsLattices
```

hZero.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.inleftC

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def hZero : Pomega := funOf inleftC (ofNat 0)

end DomainTheory.DataTypesAsLattices
```

predSet_iterate_expTag_isScottContinuous.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import DomainTheory.DataTypesAsLattices.Lambda.predSet
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.expTag
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
public import DomainTheory.DataTypesAsLattices.Lambda.predSet_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices
```

```
theorem predSet_iterate_expTag_isScottContinuous (k : ℕ) :
    IsScottContinuous (fun e => predSet^[k] (expTag e)) := by
  induction k with
  | zero => exact funOf.isScottContinuous_left (ofNat 0)
  | succ k ih =>
    have h := theorem_1_3 (f := predSet)
      (g := fun e => predSet^[k] (expTag e))
      predSet.isScottContinuous ih
    convert h using 1
    funext e
    exact Function.iterate_succ_apply' predSet k (expTag e)

end DomainTheory.DataTypesAsLattices
```

predSet_iterate_ofNat.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.predSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.predSet_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem predSet_iterate_ofNat :
    ∀ k i, predSet^[k] (ofNat (k + i)) = ofNat i
  | 0, i => by
    simp [Function.iterate_zero]
  | k + 1, i => by
    rw [Function.iterate_succ_apply]
    have : k + 1 + i = k + i + 1 := by omega
    rw [this, predSet.ofNat_succ]
    exact predSet_iterate_ofNat k i

end DomainTheory.DataTypesAsLattices
```

predSet_iterate_self.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.predSet
public import DomainTheory.DataTypesAsLattices.Functionality.predSet_iterate_ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem predSet_iterate_self (k : ℕ) :
    predSet^[k] (ofNat k) = ofNat 0 := by
```

```
  simpa using predSet_iterate_ofNat k 0

end DomainTheory.DataTypesAsLattices
```

seq4_body_isScottContinuous.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import DomainTheory.DataTypesAsLattices.Lambda.predSet
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.id_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.const_isScottContinuous
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.continuous_nary
public import DomainTheory.DataTypesAsLattices.Lambda.predSet_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem seq4_body_isScottContinuous (a b c d : Pomega) :
    IsScottContinuous (fun z =>
      condSet z a
        (condSet (predSet z) b
          (condSet (predSet (predSet z)) c
            (condSet (predSet^[3] z) d botElem)))) :=
  continuous_nary
    (fun y t => condSet.isScottContinuous_left y t)
    (fun x t => condSet.isScottContinuous_mid x t)
    (fun x y => condSet.isScottContinuous_right x y)
    id_isScottContinuous (const_isScottContinuous a)
    (continuous_nary
      (fun y t => condSet.isScottContinuous_left y t)
      (fun x t => condSet.isScottContinuous_mid x t)
      (fun x y => condSet.isScottContinuous_right x y)
      predSet.isScottContinuous (const_isScottContinuous b)
      (continuous_nary
        (fun y t => condSet.isScottContinuous_left y t)
        (fun x t => condSet.isScottContinuous_mid x t)
        (fun x y => condSet.isScottContinuous_right x y)
        (theorem_1_3 (f := predSet) (g := predSet)
          predSet.isScottContinuous predSet.isScottContinuous)
        (const_isScottContinuous c)
        (continuous_nary
          (fun y t => condSet.isScottContinuous_left y t)
          (fun x t => condSet.isScottContinuous_mid x t)
          (fun x y => condSet.isScottContinuous_right x y)
```

```
          (theorem_1_3 (f := predSet)
            (g := fun z => predSet (predSet z))
            predSet.isScottContinuous
            (theorem_1_3 (f := predSet) (g := predSet)
              predSet.isScottContinuous predSet.isScottContinuous))
          (const_isScottContinuous d) (const_isScottContinuous botElem))))

end DomainTheory.DataTypesAsLattices
```

sumE.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Functionality.RestrictedEquiv
public import DomainTheory.DataTypesAsLattices.Functionality.pairElem

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (7.8).** Sum of restricted equivalences. -/
def sumE (A B : RestrictedEquiv) : RestrictedEquiv where
  rel := fun u v =>
    (u = pairElem (ofNat 0) (funOf u (ofNat 1)) ∧
        v = pairElem (ofNat 0) (funOf v (ofNat 1)) ∧
        A.rel (funOf u (ofNat 1)) (funOf v (ofNat 1))) ∨
      (u = pairElem (ofNat 1) (funOf u (ofNat 1)) ∧
        v = pairElem (ofNat 1) (funOf v (ofNat 1)) ∧
        B.rel (funOf u (ofNat 1)) (funOf v (ofNat 1)))
  symm := by
    intro u v h
    rcases h with ⟨hu, hv, hA⟩ | ⟨hu, hv, hB⟩
    · exact Or.inl ⟨hv, hu, A.symm hA⟩
    · exact Or.inr ⟨hv, hu, B.symm hB⟩
  trans := by
    intro u v w h1 h2
    rcases h1 with ⟨hu, hv, hA⟩ | ⟨hu, hv, hB⟩
    · rcases h2 with ⟨hv', hw, hA'⟩ | ⟨hv', hw, _⟩
      · exact Or.inl ⟨hu, hw, A.trans hA hA'⟩
      · exact (pairElem.tag_ne (funOf v (ofNat 1)) (funOf v (ofNat 1))
          (hv.symm.trans hv')).elim
    · rcases h2 with ⟨hv', hw, _⟩ | ⟨hv', hw, hB'⟩
      · exact (pairElem.tag_ne (funOf v (ofNat 1)) (funOf v (ofNat 1))
          (hv'.symm.trans hv)).elim
      · exact Or.inr ⟨hu, hw, B.trans hB hB'⟩

end DomainTheory.DataTypesAsLattices
```

sumMed.lean

```
module
```

```
public import DomainTheory.DataTypesAsLattices.Retracts.graph_ext
public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Embedding.extend
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.RestrictedEquiv
public import DomainTheory.DataTypesAsLattices.Functionality.pairElem
public import DomainTheory.DataTypesAsLattices.Functionality.arrowE
public import DomainTheory.DataTypesAsLattices.Functionality.sumE
public import DomainTheory.DataTypesAsLattices.Functionality.continuous_nary
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Functionality.mem
public import DomainTheory.DataTypesAsLattices.Lambda.interC
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- Coproduct mediator in the category of restricted equivalences. -/
def sumMed (f g : Pomega) : Pomega :=
  graph (fun u =>
    condSet (funOf u (ofNat 0))
      (funOf f (funOf u (ofNat 1)))
      (funOf g (funOf u (ofNat 1))))

namespace sumMed

theorem body_isScottContinuous (f g : Pomega) :
    IsScottContinuous (fun u =>
      condSet (funOf u (ofNat 0))
        (funOf f (funOf u (ofNat 1)))
        (funOf g (funOf u (ofNat 1)))) :=
  continuous_nary
    (fun y z => condSet.isScottContinuous_left y z)
    (fun x z => condSet.isScottContinuous_mid x z)
    (fun x y => condSet.isScottContinuous_right x y)
    (funOf.isScottContinuous_left (ofNat 0))
    (theorem_1_3 (funOf.isScottContinuous f)
      (funOf.isScottContinuous_left (ofNat 1)))
    (theorem_1_3 (funOf.isScottContinuous g)
      (funOf.isScottContinuous_left (ofNat 1)))

theorem app (f g u : Pomega) :
    funOf (sumMed f g) u =
      condSet (funOf u (ofNat 0))
        (funOf f (funOf u (ofNat 1)))
```

```
        (funOf g (funOf u (ofNat 1))) :=
  beta (sumMed.body_isScottContinuous f g) u

theorem app_inl (f g x : Pomega) :
    funOf (sumMed f g) (pairElem (ofNat 0) x) = funOf f x := by
  rw [sumMed.app, pairElem.fst, pairElem.snd, condSet.ofNat_zero]

theorem app_inr (f g x : Pomega) :
    funOf (sumMed f g) (pairElem (ofNat 1) x) = funOf g x := by
  rw [sumMed.app, pairElem.fst, pairElem.snd, condSet.ofNat_one]

theorem isGraph (f g : Pomega) :
    sumMed f g = graph (fun u => funOf (sumMed f g) u) := by
  apply graph_ext
  intro u
  exact (sumMed.app f g u).symm

theorem rel {A B C : RestrictedEquiv} {f g : Pomega}
    (hf : (arrowE A C).mem f) (hg : (arrowE B C).mem g) :
    (arrowE (sumE A B) C).mem (sumMed f g) := by
  refine ⟨sumMed.isGraph f g, sumMed.isGraph f g, ?_⟩
  intro u v huv
  rcases huv with ⟨hu, hv, hA⟩ | ⟨hu, hv, hB⟩
  · have hmu : funOf (sumMed f g) u = funOf f (funOf u (ofNat 1)) :=
      (congrArg (fun w => funOf (sumMed f g) w) hu).trans
        (sumMed.app_inl f g _)
    have hmv : funOf (sumMed f g) v = funOf f (funOf v (ofNat 1)) :=
      (congrArg (fun w => funOf (sumMed f g) w) hv).trans
        (sumMed.app_inl f g _)
    rw [hmu, hmv]
    exact hf.2.2 (funOf u (ofNat 1)) (funOf v (ofNat 1)) hA
  · have hmu : funOf (sumMed f g) u = funOf g (funOf u (ofNat 1)) :=
      (congrArg (fun w => funOf (sumMed f g) w) hu).trans
        (sumMed.app_inr f g _)
    have hmv : funOf (sumMed f g) v = funOf g (funOf v (ofNat 1)) :=
      (congrArg (fun w => funOf (sumMed f g) w) hv).trans
        (sumMed.app_inr f g _)
    rw [hmu, hmv]
    exact hg.2.2 (funOf u (ofNat 1)) (funOf v (ofNat 1)) hB

theorem rel_unique {A B C : RestrictedEquiv} {f g h : Pomega}
    (hh : (arrowE (sumE A B) C).mem h)
    (hinl : ∀ x y, A.rel x y →
      C.rel (funOf h (pairElem (ofNat 0) x)) (funOf f y))
    (hinr : ∀ x y, B.rel x y →
      C.rel (funOf h (pairElem (ofNat 1) x)) (funOf g y)) :
    (arrowE (sumE A B) C).rel h (sumMed f g) := by
  refine ⟨hh.1, sumMed.isGraph f g, ?_⟩
  intro u v huv
  rcases huv with ⟨hu, hv, hA⟩ | ⟨hu, hv, hB⟩
  · have hhu :
        funOf h u = funOf h (pairElem (ofNat 0) (funOf u (ofNat 1))) :=
      congrArg (fun w => funOf h w) hu
    have hmv : funOf (sumMed f g) v = funOf f (funOf v (ofNat 1)) :=
      (congrArg (fun w => funOf (sumMed f g) w) hv).trans
        (sumMed.app_inl f g _)
```

```
    rw [hhu, hmv]
    exact hinl (funOf u (ofNat 1)) (funOf v (ofNat 1)) hA
  · have hhu :
        funOf h u = funOf h (pairElem (ofNat 1) (funOf u (ofNat 1))) :=
      congrArg (fun w => funOf h w) hu
    have hmv : funOf (sumMed f g) v = funOf g (funOf v (ofNat 1)) :=
      (congrArg (fun w => funOf (sumMed f g) w) hv).trans
        (sumMed.app_inr f g _)
    rw [hhu, hmv]
    exact hinr (funOf u (ofNat 1)) (funOf v (ofNat 1)) hB

end sumMed

end DomainTheory.DataTypesAsLattices
```

typed_apply_retract.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.Retract
public import DomainTheory.DataTypesAsLattices.Retracts.typed
public import DomainTheory.DataTypesAsLattices.Retracts.retract_app
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem typed_apply_retract {a x : Pomega} (ha : Retract a) :
    typed (funOf a x) a :=
  (retract_app ha x).symm

end DomainTheory.DataTypesAsLattices
```

bot_isRetract.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.comp
public import DomainTheory.DataTypesAsLattices.Retracts.Retract

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem bot_isRetract : Retract botElem := by
  constructor
  unfold comp botElem
  ext p
  simp [graph, funOf]
```

```
end DomainTheory.DataTypesAsLattices
```

fstC.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (4.14)-(4.15).** Projections of a pair-as-function. -/
def fstC : Pomega := graph (fun u => funOf u (ofNat 0))

namespace fstC

theorem app (u : Pomega) : funOf fstC u = funOf u (ofNat 0) :=
  beta (funOf.isScottContinuous_left (ofNat 0)) u

end fstC

end DomainTheory.DataTypesAsLattices
```

whichC.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.fstC

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def whichC : Pomega := fstC

namespace whichC

theorem app (u : Pomega) : funOf whichC u = funOf u (ofNat 0) :=
  fstC.app u

end whichC

end DomainTheory.DataTypesAsLattices
```

hApp.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.const_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.whichC
public import DomainTheory.DataTypesAsLattices.Functionality.outC
public import DomainTheory.DataTypesAsLattices.Functionality.expPayload
public import DomainTheory.DataTypesAsLattices.Functionality.hApply
public import DomainTheory.DataTypesAsLattices.Functionality.continuous_tuple
public import DomainTheory.DataTypesAsLattices.Functionality.continuous_nary
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def hApp (H e t : Pomega) : Pomega :=
  condSet (funOf whichC (hApply H (funOf (expPayload e) (ofNat 0)) t)) botElem
    (funOf (funOf outC (hApply H (funOf (expPayload e) (ofNat 0)) t))
      (hApply H (funOf (expPayload e) (ofNat 1)) t))

namespace hApp

theorem isScottContinuous_H (e t : Pomega) :
    IsScottContinuous (fun H => hApp H e t) :=
  continuous_nary
    (fun y z => condSet.isScottContinuous_left y z)
    (fun x z => condSet.isScottContinuous_mid x z)
    (fun x y => condSet.isScottContinuous_right x y)
    (theorem_1_3 (funOf.isScottContinuous whichC)
      (hApply.isScottContinuous_H _ t))
    (const_isScottContinuous botElem)
    (continuous_tuple
      (fun y => funOf.isScottContinuous_left y)
      (fun u => funOf.isScottContinuous u)
      (theorem_1_3 (funOf.isScottContinuous outC)
        (hApply.isScottContinuous_H _ t))
      (hApply.isScottContinuous_H _ t))

theorem isScottContinuous_t (H e : Pomega) :
    IsScottContinuous (fun t => hApp H e t) :=
  continuous_nary
    (fun y z => condSet.isScottContinuous_left y z)
    (fun x z => condSet.isScottContinuous_mid x z)
```

```
    (fun x y => condSet.isScottContinuous_right x y)
    (theorem_1_3 (f := funOf whichC)
      (g := fun t => hApply H (funOf (expPayload e) (ofNat 0)) t)
      (funOf.isScottContinuous whichC)
      (funOf.isScottContinuous (funOf H (funOf (expPayload e) (ofNat 0)))))
    (const_isScottContinuous botElem)
    (continuous_tuple
      (fun y => funOf.isScottContinuous_left y)
      (fun u => funOf.isScottContinuous u)
      (theorem_1_3 (f := funOf outC)
        (g := fun t => hApply H (funOf (expPayload e) (ofNat 0)) t)
        (funOf.isScottContinuous outC)
        (funOf.isScottContinuous (funOf H (funOf (expPayload e) (ofNat 0)))))
      (funOf.isScottContinuous (funOf H (funOf (expPayload e) (ofNat 1)))))

theorem isScottContinuous_e (H t : Pomega) :
    IsScottContinuous (fun e => hApp H e t) :=
  continuous_nary
    (fun y z => condSet.isScottContinuous_left y z)
    (fun x z => condSet.isScottContinuous_mid x z)
    (fun x y => condSet.isScottContinuous_right x y)
    (theorem_1_3 (f := funOf whichC)
      (g := fun e => hApply H (funOf (expPayload e) (ofNat 0)) t)
      (funOf.isScottContinuous whichC)
      (hApply.payload_isScottContinuous_e H t 0))
    (const_isScottContinuous botElem)
    (continuous_tuple
      (fun y => funOf.isScottContinuous_left y)
      (fun u => funOf.isScottContinuous u)
      (theorem_1_3 (f := funOf outC)
        (g := fun e => hApply H (funOf (expPayload e) (ofNat 0)) t)
        (funOf.isScottContinuous outC)
        (hApply.payload_isScottContinuous_e H t 0))
      (hApply.payload_isScottContinuous_e H t 1))

end hApp

end DomainTheory.DataTypesAsLattices
```

hPred.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Functionality.continuous_nary
public import DomainTheory.DataTypesAsLattices.Functionality.expPayload
public import DomainTheory.DataTypesAsLattices.Functionality.hApply
public import DomainTheory.DataTypesAsLattices.Functionality.inleftC
public import DomainTheory.DataTypesAsLattices.Functionality.outC
```

```
public import DomainTheory.DataTypesAsLattices.Functionality.whichC
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.const_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.predSet
public import DomainTheory.DataTypesAsLattices.Lambda.predSet_more
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def hPred (H e t : Pomega) : Pomega :=
  condSet (funOf whichC (hApply H (expPayload e) t))
    (funOf inleftC (predSet (funOf outC (hApply H (expPayload e) t))))
    botElem

namespace hPred

theorem isScottContinuous_H (e t : Pomega) :
    IsScottContinuous (fun H => hPred H e t) :=
  continuous_nary
    (fun y z => condSet.isScottContinuous_left y z)
    (fun x z => condSet.isScottContinuous_mid x z)
    (fun x y => condSet.isScottContinuous_right x y)
    (theorem_1_3 (funOf.isScottContinuous whichC) (hApply.isScottContinuous_H _ t))
    (theorem_1_3 (funOf.isScottContinuous inleftC)
      (theorem_1_3 predSet.isScottContinuous
        (theorem_1_3 (funOf.isScottContinuous outC)
          (hApply.isScottContinuous_H _ t))))
    (const_isScottContinuous botElem)

theorem isScottContinuous_t (H e : Pomega) :
    IsScottContinuous (fun t => hPred H e t) :=
  continuous_nary
    (fun y z => condSet.isScottContinuous_left y z)
    (fun x z => condSet.isScottContinuous_mid x z)
    (fun x y => condSet.isScottContinuous_right x y)
    (theorem_1_3 (f := funOf whichC) (g := fun t => hApply H (expPayload e) t)
      (funOf.isScottContinuous whichC)
      (funOf.isScottContinuous (funOf H (expPayload e))))
    (theorem_1_3 (f := funOf inleftC)
      (g := fun t => predSet (funOf outC (hApply H (expPayload e) t)))
      (funOf.isScottContinuous inleftC)
      (theorem_1_3 (f := predSet)
        (g := fun t => funOf outC (hApply H (expPayload e) t))
        predSet.isScottContinuous
        (theorem_1_3 (f := funOf outC) (g := fun t => hApply H (expPayload e) t)
          (funOf.isScottContinuous outC)
          (funOf.isScottContinuous (funOf H (expPayload e))))))
    (const_isScottContinuous botElem)

theorem isScottContinuous_e (H t : Pomega) :
    IsScottContinuous (fun e => hPred H e t) :=
```

```
  let hrec :=
    theorem_1_3 (f := fun e => hApply H e t) (g := expPayload)
      (hApply.isScottContinuous_e H t)
      (funOf.isScottContinuous_left (ofNat 1))
  continuous_nary
    (fun y z => condSet.isScottContinuous_left y z)
    (fun x z => condSet.isScottContinuous_mid x z)
    (fun x y => condSet.isScottContinuous_right x y)
    (theorem_1_3 (f := funOf whichC) (g := fun e => hApply H (expPayload e) t)
      (funOf.isScottContinuous whichC) hrec)
    (theorem_1_3 (f := funOf inleftC)
      (g := fun e => predSet (funOf outC (hApply H (expPayload e) t)))
      (funOf.isScottContinuous inleftC)
      (theorem_1_3 (f := predSet)
        (g := fun e => funOf outC (hApply H (expPayload e) t))
        predSet.isScottContinuous
        (theorem_1_3 (f := funOf outC) (g := fun e => hApply H (expPayload e) t)
          (funOf.isScottContinuous outC) hrec)))
    (const_isScottContinuous botElem)

end hPred

end DomainTheory.DataTypesAsLattices
```

hSucc.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Functionality.continuous_nary
public import DomainTheory.DataTypesAsLattices.Functionality.expPayload
public import DomainTheory.DataTypesAsLattices.Functionality.hApply
public import DomainTheory.DataTypesAsLattices.Functionality.inleftC
public import DomainTheory.DataTypesAsLattices.Functionality.outC
public import DomainTheory.DataTypesAsLattices.Functionality.whichC
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.const_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.succSet
public import DomainTheory.DataTypesAsLattices.Lambda.succSet_more
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def hSucc (H e t : Pomega) : Pomega :=
```

```
  condSet (funOf whichC (hApply H (expPayload e) t))
    (funOf inleftC (succSet (funOf outC (hApply H (expPayload e) t))))
    botElem

namespace hSucc

theorem isScottContinuous_H (e t : Pomega) :
    IsScottContinuous (fun H => hSucc H e t) :=
  continuous_nary
    (fun y z => condSet.isScottContinuous_left y z)
    (fun x z => condSet.isScottContinuous_mid x z)
    (fun x y => condSet.isScottContinuous_right x y)
    (theorem_1_3 (funOf.isScottContinuous whichC) (hApply.isScottContinuous_H _ t))
    (theorem_1_3 (funOf.isScottContinuous inleftC)
      (theorem_1_3 succSet.isScottContinuous
        (theorem_1_3 (funOf.isScottContinuous outC)
          (hApply.isScottContinuous_H _ t))))
    (const_isScottContinuous botElem)

theorem isScottContinuous_t (H e : Pomega) :
    IsScottContinuous (fun t => hSucc H e t) :=
  continuous_nary
    (fun y z => condSet.isScottContinuous_left y z)
    (fun x z => condSet.isScottContinuous_mid x z)
    (fun x y => condSet.isScottContinuous_right x y)
    (theorem_1_3 (f := funOf whichC) (g := fun t => hApply H (expPayload e) t)
      (funOf.isScottContinuous whichC)
      (funOf.isScottContinuous (funOf H (expPayload e))))
    (theorem_1_3 (f := funOf inleftC)
      (g := fun t => succSet (funOf outC (hApply H (expPayload e) t)))
      (funOf.isScottContinuous inleftC)
      (theorem_1_3 (f := succSet)
        (g := fun t => funOf outC (hApply H (expPayload e) t))
        succSet.isScottContinuous
        (theorem_1_3 (f := funOf outC) (g := fun t => hApply H (expPayload e) t)
          (funOf.isScottContinuous outC)
          (funOf.isScottContinuous (funOf H (expPayload e))))))
    (const_isScottContinuous botElem)

theorem isScottContinuous_e (H t : Pomega) :
    IsScottContinuous (fun e => hSucc H e t) :=
  let hrec :=
    theorem_1_3 (f := fun e => hApply H e t) (g := expPayload)
      (hApply.isScottContinuous_e H t)
      (funOf.isScottContinuous_left (ofNat 1))
  continuous_nary
    (fun y z => condSet.isScottContinuous_left y z)
    (fun x z => condSet.isScottContinuous_mid x z)
    (fun x y => condSet.isScottContinuous_right x y)
    (theorem_1_3 (f := funOf whichC) (g := fun e => hApply H (expPayload e) t)
      (funOf.isScottContinuous whichC) hrec)
    (theorem_1_3 (f := funOf inleftC)
      (g := fun e => succSet (funOf outC (hApply H (expPayload e) t)))
      (funOf.isScottContinuous inleftC)
      (theorem_1_3 (f := succSet)
        (g := fun e => funOf outC (hApply H (expPayload e) t))
```

```
        succSet.isScottContinuous
        (theorem_1_3 (f := funOf outC) (g := fun e => hApply H (expPayload e) t)
          (funOf.isScottContinuous outC) hrec)))
    (const_isScottContinuous botElem)

end hSucc

end DomainTheory.DataTypesAsLattices
```

plusR.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.dcondSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.pairSeq

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (4.10), modified.** The displayed equation writes the
ordinary McCarthy conditional `?`. Lean uses the doubly strict
conditional `?` of (4.4), as in `bool` (4.3), which (4.8)-(4.10)
generalize. Mixed tags therefore map to `⊤` rather than the union of
the two injections; that is required for `?` to be a retract
(Theorem 4.5). This is not a symbol-for-symbol transcription of the
displayed (4.10). -/
def plusR (a b : Pomega) : Pomega :=
  graph (fun u =>
    dcondSet (funOf u (ofNat 0))
      (pairSeq (ofNat 0) (funOf a (funOf u (ofNat 1))))
      (pairSeq (ofNat 1) (funOf b (funOf u (ofNat 1)))))

end DomainTheory.DataTypesAsLattices
```

seq4.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.predSet
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- Four-place sequence used by the expanded product of (4.44). -/
def seq4 (a b c d : Pomega) : Pomega :=
  graph (fun z =>
```

```
    condSet z a
      (condSet (predSet z) b
        (condSet (predSet (predSet z)) c
          (condSet (predSet (predSet (predSet z))) d botElem))))

end DomainTheory.DataTypesAsLattices
```

seq4_app.lean

```
module

public import DomainTheory.DataTypesAsLattices.Functionality.predSet_iterate_self
public import DomainTheory.DataTypesAsLattices.Functionality.
    seq4_body_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.predSet
public import DomainTheory.DataTypesAsLattices.Lambda.predSet_more
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.seq4

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace seq4_app

theorem app (a b c d z : Pomega) :
    funOf (seq4 a b c d) z =
      condSet z a
        (condSet (predSet z) b
          (condSet (predSet (predSet z)) c
            (condSet (predSet^[3] z) d botElem))) :=
  beta (seq4_body_isScottContinuous a b c d) z

theorem zero (a b c d : Pomega) :
    funOf (seq4 a b c d) (ofNat 0) = a := by
  rw [seq4_app.app, condSet.ofNat_zero]

theorem one (a b c d : Pomega) :
    funOf (seq4 a b c d) (ofNat 1) = b := by
  rw [seq4_app.app, condSet.ofNat_one, predSet.ofNat_succ, condSet.ofNat_zero]

theorem two (a b c d : Pomega) :
    funOf (seq4 a b c d) (ofNat 2) = c := by
  rw [seq4_app.app, condSet.ofNat_succ, predSet.ofNat_succ, condSet.ofNat_one,
    predSet.ofNat_succ, condSet.ofNat_zero]

theorem three (a b c d : Pomega) :
    funOf (seq4 a b c d) (ofNat 3) = d := by
  rw [seq4_app.app, condSet.ofNat_succ, predSet.ofNat_succ, condSet.ofNat_succ,
    predSet.ofNat_succ, condSet.ofNat_one, predSet_iterate_self,
    condSet.ofNat_zero]
```

```
end seq4_app

end DomainTheory.DataTypesAsLattices
```

tagInj.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.pairSeq

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- Tag injection `⟨i, x⟩` used by the expanded n-ary sum of (4.44). -/
def tagInj (i : ℕ) (x : Pomega) : Pomega := pairSeq (ofNat i) x

end DomainTheory.DataTypesAsLattices
```

tagInj_isScottContinuous.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.tagInj
public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Functionality.
    pairSeq_isScottContinuous_right

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem tagInj_isScottContinuous (i : ℕ) :
    IsScottContinuous (tagInj i) :=
  pairSeq_isScottContinuous_right (ofNat i)

end DomainTheory.DataTypesAsLattices
```

encodeExp.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.pairSeq
public import DomainTheory.DataTypesAsLattices.Retracts.tagInj
```

```
public import DomainTheory.DataTypesAsLattices.Retracts.seq4
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Graph.lam
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.LambTerm

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (4.44).** Abstract-syntax encoding of a `LambTerm`. -/
def encodeExp : LambTerm → Pomega
  | .var n => tagInj 0 (ofNat n)
  | .zero => tagInj 1 botElem
  | .succ τ => tagInj 2 (encodeExp τ)
  | .pred τ => tagInj 3 (encodeExp τ)
  | .condq ? τ σ ? =>
      tagInj 4 (seq4 (encodeExp ?) (encodeExp τ) (encodeExp σ) (encodeExp ?))
  | .app τ σ => tagInj 5 (pairSeq (encodeExp τ) (encodeExp σ))
  | .lam n τ => tagInj 6 (pairSeq (ofNat n) (encodeExp τ))

end DomainTheory.DataTypesAsLattices
```

tagInj_one.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.tagInj
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.pairSeq_app_one

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem tagInj_one (i : ℕ) (x : Pomega) :
    funOf (tagInj i x) (ofNat 1) = x :=
  pairSeq_app_one (ofNat i) x

end DomainTheory.DataTypesAsLattices
```

tagInj_zero.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.tagInj
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.pairSeq_app_zero
```

```
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem tagInj_zero (i : ℕ) (x : Pomega) :
    funOf (tagInj i x) (ofNat 0) = ofNat i :=
  pairSeq_app_zero (ofNat i) x

end DomainTheory.DataTypesAsLattices
```

plus7Body.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.dcondSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.predSet
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.tagInj

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- Body of the seven-way tagged sum of (4.44). -/
def plus7Body (a0 a1 a2 a3 a4 a5 a6 u : Pomega) : Pomega :=
  dcondSet (funOf u (ofNat 0)) (tagInj 0 (funOf a0 (funOf u (ofNat 1))))
    (dcondSet (predSet (funOf u (ofNat 0))) (tagInj 1 (funOf a1 (funOf u (ofNat 1))))
      (dcondSet (predSet (predSet (funOf u (ofNat 0))))
        (tagInj 2 (funOf a2 (funOf u (ofNat 1))))
        (dcondSet (predSet^[3] (funOf u (ofNat 0)))
          (tagInj 3 (funOf a3 (funOf u (ofNat 1))))
          (dcondSet (predSet^[4] (funOf u (ofNat 0)))
            (tagInj 4 (funOf a4 (funOf u (ofNat 1))))
            (dcondSet (predSet^[5] (funOf u (ofNat 0)))
              (tagInj 5 (funOf a5 (funOf u (ofNat 1))))
              (dcondSet (predSet^[6] (funOf u (ofNat 0)))
                (tagInj 6 (funOf a6 (funOf u (ofNat 1))))
                topElem))))))

end DomainTheory.DataTypesAsLattices
```

plus7.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.plus7Body

@[expose] public section

namespace DomainTheory.DataTypesAsLattices
```

```
/-- Seven-way tagged sum with indices `0,...,6`, dispatching by iterated
predecessor on the tag so mixed tags collapse to `⊤` as in (4.4). -/
def plus7 (a0 a1 a2 a3 a4 a5 a6 : Pomega) : Pomega :=
  graph (plus7Body a0 a1 a2 a3 a4 a5 a6)

end DomainTheory.DataTypesAsLattices
```

tensor4.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.seq4

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, before (4.44).** Four-fold product by expanded indices,
not iterated binary `?`. -/
def tensor4 (a b c d : Pomega) : Pomega :=
  graph (fun u =>
    seq4 (funOf a (funOf u (ofNat 0)))
      (funOf b (funOf u (ofNat 1)))
      (funOf c (funOf u (ofNat 2)))
      (funOf d (funOf u (ofNat 3))))

end DomainTheory.DataTypesAsLattices
```

tensorR.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.pairSeq

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (4.9).** Product of retracts `a ? b = λu. ⟨a(u₀), b(u₁)⟩`. -/
def tensorR (a b : Pomega) : Pomega :=
  graph (fun u =>
    pairSeq (funOf a (funOf u (ofNat 0))) (funOf b (funOf u (ofNat 1))))

end DomainTheory.DataTypesAsLattices
```

treeF.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.tensorR
public import DomainTheory.DataTypesAsLattices.Retracts.plusR

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (4.38).** The tree functor. -/
def treeF (z : Pomega) : Pomega := plusR botElem (tensorR z z)

end DomainTheory.DataTypesAsLattices
```

treeR.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.Ycomb
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.treeF

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def treeR : Pomega := funOf Ycomb (graph treeF)

end DomainTheory.DataTypesAsLattices
```

updateEnv.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (4.41).** Environment update `t[x/n]`. -/
def updateEnv (t x : Pomega) (n : ℕ) : Pomega :=
  graph (fun m => if m = ofNat n then x else funOf t m)

end DomainTheory.DataTypesAsLattices
```

updateEnv_isScottContinuous_t.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.updateEnv
public import Mathlib.Order.CompleteLattice.Defs
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Lambda.const_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem updateEnv_isScottContinuous_t (x : Pomega) (n : ℕ) :
    IsScottContinuous (fun t => updateEnv t x n) :=
  graph_const_isScottContinuous
    (fun t m => if m = ofNat n then x else funOf t m)
    (fun m => by
      by_cases h : m = ofNat n
      · simpa [h] using const_isScottContinuous (c := x)
      · simpa [h] using funOf.isScottContinuous_left m)

end DomainTheory.DataTypesAsLattices
```

updateEnvUnion.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Continuous.mem_scottUnion
public import DomainTheory.DataTypesAsLattices.Continuous.scottUnion
public import DomainTheory.DataTypesAsLattices.Functionality.
    updateEnv_isScottContinuous_t
public import DomainTheory.DataTypesAsLattices.Lambda.minExtend
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.updateEnv

@[expose] public section

namespace DomainTheory.DataTypesAsLattices
```

```
def updateEnvUnion (t x idx : Pomega) : Pomega :=
  minExtend (fun n => updateEnv t x n) idx

namespace updateEnvUnion

theorem ofNat (t x : Pomega) (n : ℕ) :
    updateEnvUnion t x (DomainTheory.DataTypesAsLattices.ofNat n) = updateEnv t x n :=
     by
  ext k
  constructor
  · intro hk
    obtain ⟨m, hm, hkm⟩ := Set.mem_iUnion₂.mp hk
    have : m = n := by
      simpa [DomainTheory.DataTypesAsLattices.ofNat, Set.singleton_subset_iff] using
    hm
    subst m
    exact hkm
  · intro hk
    exact Set.mem_iUnion₂.mpr ⟨n, by simp [DomainTheory.DataTypesAsLattices.ofNat], hk⟩

theorem isScottContinuous_idx (t x : Pomega) :
    IsScottContinuous (updateEnvUnion t x) :=
  minExtend.isScottContinuous (fun n => updateEnv t x n)

theorem isScottContinuous_t (x idx : Pomega) :
    IsScottContinuous (fun t => updateEnvUnion t x idx) := by
  constructor
  intro t
  ext k
  constructor
  · intro hk
    obtain ⟨n, hn, hkn⟩ := Set.mem_iUnion₂.mp hk
    have : k ∈ scottUnion (fun t => updateEnv t x n) t := by
      rw [← (updateEnv_isScottContinuous_t x n).out t]; exact hkn
    obtain ⟨m, hm, hkm⟩ := mem_scottUnion.mp this
    exact mem_scottUnion.mpr ⟨m, hm, Set.mem_iUnion₂.mpr ⟨n, hn, hkm⟩⟩
  · intro hk
    obtain ⟨m, hm, hkm⟩ := mem_scottUnion.mp hk
    obtain ⟨n, hn, hkn⟩ := Set.mem_iUnion₂.mp hkm
    exact Set.mem_iUnion₂.mpr ⟨n, hn,
      IsScottContinuous.monotone (updateEnv_isScottContinuous_t x n) hm hkn⟩

end updateEnvUnion

end DomainTheory.DataTypesAsLattices
```

hLam.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
```

```
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.inrightC
public import DomainTheory.DataTypesAsLattices.Functionality.expPayload
public import DomainTheory.DataTypesAsLattices.Functionality.hApply
public import DomainTheory.DataTypesAsLattices.Functionality.updateEnvUnion
public import DomainTheory.DataTypesAsLattices.Functionality.continuous_tuple
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def hLam (H e t : Pomega) : Pomega :=
  funOf inrightC
    (graph (fun x =>
      hApply H (funOf (expPayload e) (ofNat 1))
        (updateEnvUnion t x (funOf (expPayload e) (ofNat 0)))))

namespace hLam

theorem isScottContinuous_H (e t : Pomega) :
    IsScottContinuous (fun H => hLam H e t) :=
  theorem_1_3 (funOf.isScottContinuous inrightC)
    (graph_const_isScottContinuous
      (fun H x =>
        hApply H (funOf (expPayload e) (ofNat 1))
          (updateEnvUnion t x (funOf (expPayload e) (ofNat 0))))
      (fun _ => hApply.isScottContinuous_H _ _))

theorem isScottContinuous_t (H e : Pomega) :
    IsScottContinuous (fun t => hLam H e t) :=
  theorem_1_3 (f := funOf inrightC)
    (g := fun t =>
      graph (fun x =>
        hApply H (funOf (expPayload e) (ofNat 1))
          (updateEnvUnion t x (funOf (expPayload e) (ofNat 0)))))
    (funOf.isScottContinuous inrightC)
    (graph_const_isScottContinuous
      (fun t x =>
        hApply H (funOf (expPayload e) (ofNat 1))
          (updateEnvUnion t x (funOf (expPayload e) (ofNat 0))))
      (fun x =>
        theorem_1_3
          (f := funOf (funOf H (funOf (expPayload e) (ofNat 1))))
          (g := fun t => updateEnvUnion t x (funOf (expPayload e) (ofNat 0)))
          (funOf.isScottContinuous (funOf H (funOf (expPayload e) (ofNat 1))))
          (updateEnvUnion.isScottContinuous_t x
            (funOf (expPayload e) (ofNat 0)))))

theorem isScottContinuous_e (H t : Pomega) :
```

```
    IsScottContinuous (fun e => hLam H e t) :=
  theorem_1_3 (f := funOf inrightC)
    (g := fun e =>
      graph (fun x =>
        hApply H (funOf (expPayload e) (ofNat 1))
          (updateEnvUnion t x (funOf (expPayload e) (ofNat 0)))))
    (funOf.isScottContinuous inrightC)
    (graph_const_isScottContinuous
      (fun e x =>
        hApply H (funOf (expPayload e) (ofNat 1))
          (updateEnvUnion t x (funOf (expPayload e) (ofNat 0))))
      (fun x =>
        continuous_tuple
          (fun y =>
            theorem_1_3 (f := fun u => funOf u y) (g := funOf H)
              (funOf.isScottContinuous_left y) (funOf.isScottContinuous H))
          (fun u => funOf.isScottContinuous (funOf H u))
          (theorem_1_3 (f := fun u => funOf u (ofNat 1)) (g := expPayload)
            (funOf.isScottContinuous_left (ofNat 1))
            (funOf.isScottContinuous_left (ofNat 1)))
          (theorem_1_3 (f := updateEnvUnion t x)
            (g := fun e => funOf (expPayload e) (ofNat 0))
            (updateEnvUnion.isScottContinuous_idx t x)
            (theorem_1_3 (f := fun u => funOf u (ofNat 0)) (g := expPayload)
              (funOf.isScottContinuous_left (ofNat 0))
              (funOf.isScottContinuous_left (ofNat 1))))))

end hLam

end DomainTheory.DataTypesAsLattices
```

funOf_e_union_fresh.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem funOf_e_union_fresh (p j m : ℕ) (u : Pomega) (hpj : p < j) :
    funOf (e p) (u ∪ {pair j m}) = funOf (e p) u := by
  apply subset_antisymm
  · intro k hk
    obtain ⟨r, hr, hprk⟩ := hk
    refine ⟨r, ?_, hprk⟩
    intro t ht
    rcases hr ht with htu | htag
    · exact htu
    · have htEq : t = pair j m := by simpa using htag
      have hjr : j ≤ r := by
```

```
        rw [htEq] at ht
        exact (pair_le_left j m).trans (mem_e_le ht)
      have hrp : r ≤ p := (pair_le_left r k).trans (mem_e_le hprk)
      exact (Nat.not_lt_of_ge (hjr.trans hrp) hpj).elim
  · exact funOf.monotone_right (e p) Set.subset_union_left

end DomainTheory.DataTypesAsLattices
```

normalized_finite_fresh_app.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.Z
public import DomainTheory.DataTypesAsLattices.Functionality.funOf_e_union_fresh
public import DomainTheory.DataTypesAsLattices.Functionality.normalized_finite_app

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem normalized_finite_fresh_app (p q m : ℕ) :
    funOf (graph (fun y => funOf (e p) y))
        (Z m (e p) (e q) ∪ {pair (max p q + 1) m}) =
      Z (m + 1) (e p) (e q) := by
  rw [normalized_finite_app,
    funOf_e_union_fresh p (max p q + 1) m _
      (Nat.lt_succ_of_le (Nat.le_max_left p q))]
  rfl

end DomainTheory.DataTypesAsLattices
```

finite_sigma_union_app.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.eq_2_9
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.Z
public import DomainTheory.DataTypesAsLattices.Functionality.sigmaJ
public import DomainTheory.DataTypesAsLattices.Functionality.
    normalized_finite_fresh_app

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem finite_sigma_union_app (p q m : ℕ) :
```

```
    funOf
        (graph (fun y => funOf (e p) y) ∪
          graph (sigmaJ (max p q + 1)))
        (Z m (e p) (e q) ∪ {pair (max p q + 1) m}) =
      Z (m + 1) (e p) (e q) ∪ {pair (max p q + 1) (m + 1)} := by
  rw [eq_2_9, normalized_finite_fresh_app, sigmaJ.app,
    sigmaJ.union_fresh (fun k => Z.finite_no_tag p q m k)]

end DomainTheory.DataTypesAsLattices
```

funOf_top.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem funOf_top (x : Pomega) : funOf topElem x = topElem := by
  ext m
  constructor
  · intro
    trivial
  · intro
    exact ⟨0, by simp [e_zero], trivial⟩

end DomainTheory.DataTypesAsLattices
```

plus7_comp_isScottContinuous.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.tagInj
public import DomainTheory.DataTypesAsLattices.Retracts.plus7Body
public import DomainTheory.DataTypesAsLattices.Retracts.plus7
public import Mathlib.Order.CompleteLattice.Defs
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Lambda.const_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.dcondSet
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.predSet
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.continuous_nary
public import DomainTheory.DataTypesAsLattices.Functionality.tagInj_isScottContinuous
```

```
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
public import DomainTheory.DataTypesAsLattices.Lambda.dcondSet_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem plus7_comp_isScottContinuous
    {f0 f1 f2 f3 f4 f5 f6 : Pomega → Pomega}
    (h0 : IsScottContinuous f0) (h1 : IsScottContinuous f1)
    (h2 : IsScottContinuous f2) (h3 : IsScottContinuous f3)
    (h4 : IsScottContinuous f4) (h5 : IsScottContinuous f5)
    (h6 : IsScottContinuous f6) :
    IsScottContinuous (fun z =>
      plus7 (f0 z) (f1 z) (f2 z) (f3 z) (f4 z) (f5 z) (f6 z)) :=
  graph_const_isScottContinuous
    (fun z u => plus7Body (f0 z) (f1 z) (f2 z) (f3 z) (f4 z) (f5 z) (f6 z) u)
    (fun u => by
      have htag (i : ℕ) {f : Pomega → Pomega} (hf : IsScottContinuous f) :
          IsScottContinuous (fun z =>
            tagInj i (funOf (f z) (funOf u (ofNat 1)))) :=
        theorem_1_3 (f := tagInj i)
          (g := fun z => funOf (f z) (funOf u (ofNat 1)))
          (tagInj_isScottContinuous i)
          (theorem_1_3 (f := fun w => funOf w (funOf u (ofNat 1))) (g := f)
            (funOf.isScottContinuous_left (funOf u (ofNat 1))) hf)
      exact continuous_nary
        (fun y t => dcondSet.isScottContinuous_left y t)
        (fun x t => dcondSet.isScottContinuous_mid x t)
        (fun x y => dcondSet.isScottContinuous_right x y)
        (const_isScottContinuous (funOf u (ofNat 0))) (htag 0 h0)
        (continuous_nary
          (fun y t => dcondSet.isScottContinuous_left y t)
          (fun x t => dcondSet.isScottContinuous_mid x t)
          (fun x y => dcondSet.isScottContinuous_right x y)
          (const_isScottContinuous (predSet (funOf u (ofNat 0)))) (htag 1 h1)
          (continuous_nary
            (fun y t => dcondSet.isScottContinuous_left y t)
            (fun x t => dcondSet.isScottContinuous_mid x t)
            (fun x y => dcondSet.isScottContinuous_right x y)
            (const_isScottContinuous (predSet (predSet (funOf u (ofNat 0)))))
            (htag 2 h2)
            (continuous_nary
              (fun y t => dcondSet.isScottContinuous_left y t)
              (fun x t => dcondSet.isScottContinuous_mid x t)
              (fun x y => dcondSet.isScottContinuous_right x y)
              (const_isScottContinuous (predSet^[3] (funOf u (ofNat 0))))
              (htag 3 h3)
              (continuous_nary
                (fun y t => dcondSet.isScottContinuous_left y t)
                (fun x t => dcondSet.isScottContinuous_mid x t)
                (fun x y => dcondSet.isScottContinuous_right x y)
                (const_isScottContinuous (predSet^[4] (funOf u (ofNat 0))))
                (htag 4 h4)
                (continuous_nary
                  (fun y t => dcondSet.isScottContinuous_left y t)
```

```
                  (fun x t => dcondSet.isScottContinuous_mid x t)
                  (fun x y => dcondSet.isScottContinuous_right x y)
                  (const_isScottContinuous (predSet^[5] (funOf u (ofNat 0))))
                  (htag 5 h5)
                  (continuous_nary
                    (fun y t => dcondSet.isScottContinuous_left y t)
                    (fun x t => dcondSet.isScottContinuous_mid x t)
                    (fun x y => dcondSet.isScottContinuous_right x y)
                    (const_isScottContinuous (predSet^[6] (funOf u (ofNat 0))))
                    (htag 6 h6) (const_isScottContinuous topElem))))))))

end DomainTheory.DataTypesAsLattices
```

plus7_map_isScottContinuous.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.tagInj
public import DomainTheory.DataTypesAsLattices.Retracts.plus7Body
public import Mathlib.Order.CompleteLattice.Defs
public import DomainTheory.DataTypesAsLattices.Lambda.predSet
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Lambda.const_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.dcondSet
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.continuous_nary
public import DomainTheory.DataTypesAsLattices.Functionality.tagInj_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
public import DomainTheory.DataTypesAsLattices.Lambda.dcondSet_more
public import DomainTheory.DataTypesAsLattices.Lambda.predSet_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem plus7_map_isScottContinuous (a0 a1 a2 a3 a4 a5 a6 : Pomega) :
    IsScottContinuous (plus7Body a0 a1 a2 a3 a4 a5 a6) := by
  have htag (i : ℕ) (a : Pomega) :
      IsScottContinuous (fun u => tagInj i (funOf a (funOf u (ofNat 1)))) :=
    theorem_1_3 (tagInj_isScottContinuous i)
      (theorem_1_3 (funOf.isScottContinuous a)
        (funOf.isScottContinuous_left (ofNat 1)))
  have hz : IsScottContinuous (fun u => funOf u (ofNat 0)) :=
    funOf.isScottContinuous_left (ofNat 0)
  have hpred : ∀ k, IsScottContinuous
      (fun u => predSet^[k] (funOf u (ofNat 0))) := by
    intro k
```

```
    induction k with
    | zero => exact hz
    | succ k ih =>
      have h := theorem_1_3 (f := predSet)
        (g := fun u => predSet^[k] (funOf u (ofNat 0)))
        predSet.isScottContinuous ih
      convert h using 1
      funext u
      exact Function.iterate_succ_apply' predSet k (funOf u (ofNat 0))
  refine continuous_nary
    (fun y z => dcondSet.isScottContinuous_left y z)
    (fun x z => dcondSet.isScottContinuous_mid x z)
    (fun x y => dcondSet.isScottContinuous_right x y)
    hz (htag 0 a0)
    (continuous_nary
      (fun y z => dcondSet.isScottContinuous_left y z)
      (fun x z => dcondSet.isScottContinuous_mid x z)
      (fun x y => dcondSet.isScottContinuous_right x y)
      (hpred 1) (htag 1 a1)
      (continuous_nary
        (fun y z => dcondSet.isScottContinuous_left y z)
        (fun x z => dcondSet.isScottContinuous_mid x z)
        (fun x y => dcondSet.isScottContinuous_right x y)
        (hpred 2) (htag 2 a2)
        (continuous_nary
          (fun y z => dcondSet.isScottContinuous_left y z)
          (fun x z => dcondSet.isScottContinuous_mid x z)
          (fun x y => dcondSet.isScottContinuous_right x y)
          (hpred 3) (htag 3 a3)
          (continuous_nary
            (fun y z => dcondSet.isScottContinuous_left y z)
            (fun x z => dcondSet.isScottContinuous_mid x z)
            (fun x y => dcondSet.isScottContinuous_right x y)
            (hpred 4) (htag 4 a4)
            (continuous_nary
              (fun y z => dcondSet.isScottContinuous_left y z)
              (fun x z => dcondSet.isScottContinuous_mid x z)
              (fun x y => dcondSet.isScottContinuous_right x y)
              (hpred 5) (htag 5 a5)
              (continuous_nary
                (fun y z => dcondSet.isScottContinuous_left y z)
                (fun x z => dcondSet.isScottContinuous_mid x z)
                (fun x y => dcondSet.isScottContinuous_right x y)
                (hpred 6) (htag 6 a6)
                (const_isScottContinuous topElem)))))))

end DomainTheory.DataTypesAsLattices
```

plus7_app.lean

```
module

public import DomainTheory.DataTypesAsLattices.Functionality.funOf_top
public import DomainTheory.DataTypesAsLattices.Functionality.
    plus7_map_isScottContinuous
```

```
public import DomainTheory.DataTypesAsLattices.Functionality.predSet_iterate_ofNat
public import DomainTheory.DataTypesAsLattices.Functionality.predSet_iterate_self
public import DomainTheory.DataTypesAsLattices.Functionality.tagInj_one
public import DomainTheory.DataTypesAsLattices.Functionality.tagInj_zero
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
public import DomainTheory.DataTypesAsLattices.Lambda.dcondSet
public import DomainTheory.DataTypesAsLattices.Lambda.dcondSet_more
public import DomainTheory.DataTypesAsLattices.Lambda.predSet
public import DomainTheory.DataTypesAsLattices.Lambda.predSet_more
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.plus7
public import DomainTheory.DataTypesAsLattices.Retracts.plus7Body
public import DomainTheory.DataTypesAsLattices.Retracts.tagInj

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace plus7_app

theorem app (a0 a1 a2 a3 a4 a5 a6 u : Pomega) :
    funOf (plus7 a0 a1 a2 a3 a4 a5 a6) u =
      plus7Body a0 a1 a2 a3 a4 a5 a6 u :=
  beta (plus7_map_isScottContinuous a0 a1 a2 a3 a4 a5 a6) u

theorem bot (a0 a1 a2 a3 a4 a5 a6 : Pomega) :
    funOf (plus7 a0 a1 a2 a3 a4 a5 a6) botElem = botElem := by
  rw [plus7_app.app, plus7Body, funOf.bot, dcondSet.bot]

theorem top (a0 a1 a2 a3 a4 a5 a6 : Pomega) :
    funOf (plus7 a0 a1 a2 a3 a4 a5 a6) topElem = topElem := by
  rw [plus7_app.app, plus7Body, funOf_top, dcondSet.top]

theorem tag0 (a0 a1 a2 a3 a4 a5 a6 x : Pomega) :
    funOf (plus7 a0 a1 a2 a3 a4 a5 a6) (tagInj 0 x) = tagInj 0 (funOf a0 x) := by
  rw [plus7_app.app, plus7Body, tagInj_zero, tagInj_one, dcondSet.ofNat_zero]

theorem tag1 (a0 a1 a2 a3 a4 a5 a6 x : Pomega) :
    funOf (plus7 a0 a1 a2 a3 a4 a5 a6) (tagInj 1 x) = tagInj 1 (funOf a1 x) := by
  rw [plus7_app.app, plus7Body, tagInj_zero, tagInj_one, predSet.ofNat_succ,
    dcondSet.ofNat_one, dcondSet.ofNat_zero]

theorem tag2 (a0 a1 a2 a3 a4 a5 a6 x : Pomega) :
    funOf (plus7 a0 a1 a2 a3 a4 a5 a6) (tagInj 2 x) = tagInj 2 (funOf a2 x) := by
  rw [plus7_app.app, plus7Body, tagInj_zero, tagInj_one, predSet.ofNat_succ,
    dcondSet.ofNat_succ, predSet.ofNat_succ, dcondSet.ofNat_one,
    dcondSet.ofNat_zero]

theorem tag3 (a0 a1 a2 a3 a4 a5 a6 x : Pomega) :
    funOf (plus7 a0 a1 a2 a3 a4 a5 a6) (tagInj 3 x) = tagInj 3 (funOf a3 x) := by
  rw [plus7_app.app, plus7Body, tagInj_zero, tagInj_one, dcondSet.ofNat_succ,
    predSet.ofNat_succ, dcondSet.ofNat_succ, predSet.ofNat_succ,
    dcondSet.ofNat_one, predSet_iterate_self, dcondSet.ofNat_zero]
```

```
theorem tag4 (a0 a1 a2 a3 a4 a5 a6 x : Pomega) :
    funOf (plus7 a0 a1 a2 a3 a4 a5 a6) (tagInj 4 x) = tagInj 4 (funOf a4 x) := by
  rw [plus7_app.app, plus7Body, tagInj_zero, tagInj_one, dcondSet.ofNat_succ,
    predSet.ofNat_succ, dcondSet.ofNat_succ, predSet.ofNat_succ,
    dcondSet.ofNat_succ, predSet_iterate_ofNat (k := 3) (i := 1),
    dcondSet.ofNat_one, predSet_iterate_self, dcondSet.ofNat_zero]

theorem tag5 (a0 a1 a2 a3 a4 a5 a6 x : Pomega) :
    funOf (plus7 a0 a1 a2 a3 a4 a5 a6) (tagInj 5 x) = tagInj 5 (funOf a5 x) := by
  rw [plus7_app.app, plus7Body, tagInj_zero, tagInj_one, dcondSet.ofNat_succ,
    predSet.ofNat_succ, dcondSet.ofNat_succ, predSet.ofNat_succ,
    dcondSet.ofNat_succ, predSet_iterate_ofNat (k := 3) (i := 2),
    dcondSet.ofNat_succ, predSet_iterate_ofNat (k := 4) (i := 1),
    dcondSet.ofNat_one, predSet_iterate_self, dcondSet.ofNat_zero]

theorem tag6 (a0 a1 a2 a3 a4 a5 a6 x : Pomega) :
    funOf (plus7 a0 a1 a2 a3 a4 a5 a6) (tagInj 6 x) = tagInj 6 (funOf a6 x) := by
  rw [plus7_app.app, plus7Body, tagInj_zero, tagInj_one, dcondSet.ofNat_succ,
    predSet.ofNat_succ, dcondSet.ofNat_succ, predSet.ofNat_succ,
    dcondSet.ofNat_succ, predSet_iterate_ofNat (k := 3) (i := 3),
    dcondSet.ofNat_succ, predSet_iterate_ofNat (k := 4) (i := 2),
    dcondSet.ofNat_succ, predSet_iterate_ofNat (k := 5) (i := 1),
    dcondSet.ofNat_one, predSet_iterate_self, dcondSet.ofNat_zero]

end plus7_app

end DomainTheory.DataTypesAsLattices
```

plus7_isRetract.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.Retract
public import DomainTheory.DataTypesAsLattices.Retracts.tagInj
public import DomainTheory.DataTypesAsLattices.Retracts.plus7Body
public import DomainTheory.DataTypesAsLattices.Retracts.plus7
public import DomainTheory.DataTypesAsLattices.Retracts.retract_app
public import DomainTheory.DataTypesAsLattices.Retracts.graph_ext
public import DomainTheory.DataTypesAsLattices.Lambda.dcondSet
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Lambda.predSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.funOf_top
public import DomainTheory.DataTypesAsLattices.Functionality.predSet_iterate_ofNat
public import DomainTheory.DataTypesAsLattices.Functionality.predSet_iterate_self
public import DomainTheory.DataTypesAsLattices.Functionality.plus7_app
public import DomainTheory.DataTypesAsLattices.Functionality.tagInj_zero
public import DomainTheory.DataTypesAsLattices.Functionality.tagInj_one
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
public import DomainTheory.DataTypesAsLattices.Lambda.dcondSet_more
public import DomainTheory.DataTypesAsLattices.Lambda.predSet_more
```

```
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem plus7_isRetract {a0 a1 a2 a3 a4 a5 a6 : Pomega}
    (h0 : Retract a0) (h1 : Retract a1) (h2 : Retract a2)
    (h3 : Retract a3) (h4 : Retract a4) (h5 : Retract a5)
    (h6 : Retract a6) :
    Retract (plus7 a0 a1 a2 a3 a4 a5 a6) := by
  constructor
  change plus7 a0 a1 a2 a3 a4 a5 a6 =
    graph (fun u => funOf (plus7 a0 a1 a2 a3 a4 a5 a6)
      (funOf (plus7 a0 a1 a2 a3 a4 a5 a6) u))
  apply graph_ext
  intro u
  rw [plus7_app.app, plus7_app.app]
  have hbody0 (x : Pomega) :
      plus7Body a0 a1 a2 a3 a4 a5 a6 (tagInj 0 x) = tagInj 0 (funOf a0 x) := by
    rw [plus7Body, tagInj_zero, tagInj_one, dcondSet.ofNat_zero]
  have hbody1 (x : Pomega) :
      plus7Body a0 a1 a2 a3 a4 a5 a6 (tagInj 1 x) = tagInj 1 (funOf a1 x) := by
    rw [plus7Body, tagInj_zero, tagInj_one, predSet.ofNat_succ,
      dcondSet.ofNat_one, dcondSet.ofNat_zero]
  have hbody2 (x : Pomega) :
      plus7Body a0 a1 a2 a3 a4 a5 a6 (tagInj 2 x) = tagInj 2 (funOf a2 x) := by
    rw [plus7Body, tagInj_zero, tagInj_one, predSet.ofNat_succ,
      dcondSet.ofNat_succ, predSet.ofNat_succ, dcondSet.ofNat_one,
      dcondSet.ofNat_zero]
  have hbody3 (x : Pomega) :
      plus7Body a0 a1 a2 a3 a4 a5 a6 (tagInj 3 x) = tagInj 3 (funOf a3 x) := by
    rw [plus7Body, tagInj_zero, tagInj_one, dcondSet.ofNat_succ,
      predSet.ofNat_succ, dcondSet.ofNat_succ, predSet.ofNat_succ,
      dcondSet.ofNat_one, predSet_iterate_self, dcondSet.ofNat_zero]
  have hbody4 (x : Pomega) :
      plus7Body a0 a1 a2 a3 a4 a5 a6 (tagInj 4 x) = tagInj 4 (funOf a4 x) := by
    rw [plus7Body, tagInj_zero, tagInj_one, dcondSet.ofNat_succ,
      predSet.ofNat_succ, dcondSet.ofNat_succ, predSet.ofNat_succ,
      dcondSet.ofNat_succ, predSet_iterate_ofNat (k := 3) (i := 1),
      dcondSet.ofNat_one, predSet_iterate_self, dcondSet.ofNat_zero]
  have hbody5 (x : Pomega) :
      plus7Body a0 a1 a2 a3 a4 a5 a6 (tagInj 5 x) = tagInj 5 (funOf a5 x) := by
    rw [plus7Body, tagInj_zero, tagInj_one, dcondSet.ofNat_succ,
      predSet.ofNat_succ, dcondSet.ofNat_succ, predSet.ofNat_succ,
      dcondSet.ofNat_succ, predSet_iterate_ofNat (k := 3) (i := 2),
      dcondSet.ofNat_succ, predSet_iterate_ofNat (k := 4) (i := 1),
      dcondSet.ofNat_one, predSet_iterate_self, dcondSet.ofNat_zero]
  have hbody6 (x : Pomega) :
      plus7Body a0 a1 a2 a3 a4 a5 a6 (tagInj 6 x) = tagInj 6 (funOf a6 x) := by
    rw [plus7Body, tagInj_zero, tagInj_one, dcondSet.ofNat_succ,
      predSet.ofNat_succ, dcondSet.ofNat_succ, predSet.ofNat_succ,
      dcondSet.ofNat_succ, predSet_iterate_ofNat (k := 3) (i := 3),
      dcondSet.ofNat_succ, predSet_iterate_ofNat (k := 4) (i := 2),
      dcondSet.ofNat_succ, predSet_iterate_ofNat (k := 5) (i := 1),
      dcondSet.ofNat_one, predSet_iterate_self, dcondSet.ofNat_zero]
  have hbodyTop : plus7Body a0 a1 a2 a3 a4 a5 a6 topElem = topElem := by
    rw [plus7Body, funOf_top, dcondSet.top]
```

```
  have hbodyBot : plus7Body a0 a1 a2 a3 a4 a5 a6 botElem = botElem := by
    rw [plus7Body, funOf.bot, dcondSet.bot]
  rcases dcondSet.cases (funOf u (ofNat 0)) with hz | hz | hz | hz
  · have hL : plus7Body a0 a1 a2 a3 a4 a5 a6 u =
        tagInj 0 (funOf a0 (funOf u (ofNat 1))) := by
      rw [plus7Body, dcondSet.of_zero_not_pos hz.1 hz.2]
    rw [hL, hbody0, retract_app h0]
  · rw [plus7Body, dcondSet.of_pos_not_zero hz.1 hz.2]
    rcases dcondSet.cases (predSet (funOf u (ofNat 0))) with hz1 | hz1 | hz1 | hz1
    · rw [dcondSet.of_zero_not_pos hz1.1 hz1.2, hbody1, retract_app h1]
    · rw [dcondSet.of_pos_not_zero hz1.1 hz1.2]
      rcases dcondSet.cases (predSet (predSet (funOf u (ofNat 0)))) with
        hz2 | hz2 | hz2 | hz2
      · rw [dcondSet.of_zero_not_pos hz2.1 hz2.2, hbody2, retract_app h2]
      · rw [dcondSet.of_pos_not_zero hz2.1 hz2.2]
        rcases dcondSet.cases (predSet^[3] (funOf u (ofNat 0))) with
          hz3 | hz3 | hz3 | hz3
        · rw [dcondSet.of_zero_not_pos hz3.1 hz3.2, hbody3, retract_app h3]
        · rw [dcondSet.of_pos_not_zero hz3.1 hz3.2]
          rcases dcondSet.cases (predSet^[4] (funOf u (ofNat 0))) with
            hz4 | hz4 | hz4 | hz4
          · rw [dcondSet.of_zero_not_pos hz4.1 hz4.2, hbody4, retract_app h4]
          · rw [dcondSet.of_pos_not_zero hz4.1 hz4.2]
            rcases dcondSet.cases (predSet^[5] (funOf u (ofNat 0))) with
              hz5 | hz5 | hz5 | hz5
            · rw [dcondSet.of_zero_not_pos hz5.1 hz5.2, hbody5, retract_app h5]
            · rw [dcondSet.of_pos_not_zero hz5.1 hz5.2]
              rcases dcondSet.cases (predSet^[6] (funOf u (ofNat 0))) with
                hz6 | hz6 | hz6 | hz6
              · rw [dcondSet.of_zero_not_pos hz6.1 hz6.2, hbody6, retract_app h6]
              · rw [dcondSet.of_pos_not_zero hz6.1 hz6.2, hbodyTop]
              · rw [dcondSet.of_mixed hz6.1 hz6.2, hbodyTop]
              · rw [dcondSet.of_empty hz6.1 hz6.2, hbodyBot]
            · rw [dcondSet.of_mixed hz5.1 hz5.2, hbodyTop]
            · rw [dcondSet.of_empty hz5.1 hz5.2, hbodyBot]
          · rw [dcondSet.of_mixed hz4.1 hz4.2, hbodyTop]
          · rw [dcondSet.of_empty hz4.1 hz4.2, hbodyBot]
        · rw [dcondSet.of_mixed hz3.1 hz3.2, hbodyTop]
        · rw [dcondSet.of_empty hz3.1 hz3.2, hbodyBot]
      · rw [dcondSet.of_mixed hz2.1 hz2.2, hbodyTop]
      · rw [dcondSet.of_empty hz2.1 hz2.2, hbodyBot]
    · rw [dcondSet.of_mixed hz1.1 hz1.2, hbodyTop]
    · rw [dcondSet.of_empty hz1.1 hz1.2, hbodyBot]
  · rw [plus7Body, dcondSet.of_mixed hz.1 hz.2, hbodyTop]
  · rw [plus7Body, dcondSet.of_empty hz.1 hz.2, hbodyBot]

end DomainTheory.DataTypesAsLattices
```

plusR_map_isScottContinuous.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.pairSeq
public import Mathlib.Order.CompleteLattice.Defs
```

```
public import DomainTheory.DataTypesAsLattices.Lambda.dcondSet
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.continuous_nary
public import DomainTheory.DataTypesAsLattices.Functionality.
    pairSeq_isScottContinuous_right
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
public import DomainTheory.DataTypesAsLattices.Lambda.dcondSet_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem plusR_map_isScottContinuous (a b : Pomega) :
    IsScottContinuous (fun u =>
      dcondSet (funOf u (ofNat 0))
        (pairSeq (ofNat 0) (funOf a (funOf u (ofNat 1))))
        (pairSeq (ofNat 1) (funOf b (funOf u (ofNat 1))))) :=
  continuous_nary
    (fun y z => dcondSet.isScottContinuous_left y z)
    (fun x z => dcondSet.isScottContinuous_mid x z)
    (fun x y => dcondSet.isScottContinuous_right x y)
    (funOf.isScottContinuous_left (ofNat 0))
    (theorem_1_3 (pairSeq_isScottContinuous_right (ofNat 0))
      (theorem_1_3 (funOf.isScottContinuous a)
        (funOf.isScottContinuous_left (ofNat 1))))
    (theorem_1_3 (pairSeq_isScottContinuous_right (ofNat 1))
      (theorem_1_3 (funOf.isScottContinuous b)
        (funOf.isScottContinuous_left (ofNat 1))))

end DomainTheory.DataTypesAsLattices
```

plusR_app.lean

```
module

public import DomainTheory.DataTypesAsLattices.Functionality.funOf_top
public import DomainTheory.DataTypesAsLattices.Functionality.
    plusR_map_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
public import DomainTheory.DataTypesAsLattices.Lambda.dcondSet
public import DomainTheory.DataTypesAsLattices.Lambda.dcondSet_more
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.pairSeq
public import DomainTheory.DataTypesAsLattices.Retracts.plusR
```

```
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace plusR_app

theorem app (a b u : Pomega) :
    funOf (plusR a b) u =
      dcondSet (funOf u (ofNat 0))
        (pairSeq (ofNat 0) (funOf a (funOf u (ofNat 1))))
        (pairSeq (ofNat 1) (funOf b (funOf u (ofNat 1)))) :=
  beta (plusR_map_isScottContinuous a b) u

theorem bot (a b : Pomega) :
    funOf (plusR a b) botElem = botElem := by
  rw [plusR_app.app, funOf.bot, dcondSet.bot]

theorem top (a b : Pomega) :
    funOf (plusR a b) topElem = topElem := by
  rw [plusR_app.app, funOf_top, dcondSet.top]

end plusR_app

end DomainTheory.DataTypesAsLattices
```

plusR_isRetract.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.Retract
public import DomainTheory.DataTypesAsLattices.Retracts.plusR
public import DomainTheory.DataTypesAsLattices.Retracts.retract_app
public import DomainTheory.DataTypesAsLattices.Retracts.graph_ext
public import DomainTheory.DataTypesAsLattices.Lambda.dcondSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.pairSeq_app_zero
public import DomainTheory.DataTypesAsLattices.Functionality.pairSeq_app_one
public import DomainTheory.DataTypesAsLattices.Functionality.plusR_app
public import DomainTheory.DataTypesAsLattices.Lambda.dcondSet_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, Theorem 4.5 (i).** `a ? b` is a retract when `a` and `b` are. -/
theorem plusR_isRetract {a b : Pomega} (ha : Retract a) (hb : Retract b) :
    Retract (plusR a b) := by
  constructor
  change plusR a b =
    graph (fun u => funOf (plusR a b) (funOf (plusR a b) u))
```

```
  apply graph_ext
  intro u
  rw [plusR_app.app a b u]
  rcases dcondSet.cases (funOf u (ofNat 0)) with h | h | h | h
  · rw [dcondSet.of_zero_not_pos h.1 h.2, plusR_app.app,
      pairSeq_app_zero, pairSeq_app_one, dcondSet.ofNat_zero, retract_app ha]
  · rw [dcondSet.of_pos_not_zero h.1 h.2, plusR_app.app,
      pairSeq_app_zero, pairSeq_app_one, dcondSet.ofNat_one, retract_app hb]
  · rw [dcondSet.of_mixed h.1 h.2, plusR_app.top]
  · rw [dcondSet.of_empty h.1 h.2, plusR_app.bot]

end DomainTheory.DataTypesAsLattices
```

plusR_right_isScottContinuous.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.pairSeq
public import DomainTheory.DataTypesAsLattices.Retracts.plusR
public import Mathlib.Order.CompleteLattice.Defs
public import DomainTheory.DataTypesAsLattices.Lambda.dcondSet
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.
    pairSeq_isScottContinuous_right
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
public import DomainTheory.DataTypesAsLattices.Lambda.dcondSet_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem plusR_right_isScottContinuous (a : Pomega) :
    IsScottContinuous (fun b => plusR a b) :=
  graph_const_isScottContinuous
    (fun b u =>
      dcondSet (funOf u (ofNat 0))
        (pairSeq (ofNat 0) (funOf a (funOf u (ofNat 1))))
        (pairSeq (ofNat 1) (funOf b (funOf u (ofNat 1)))))
    (fun u =>
      theorem_1_3
        (f := fun y =>
          dcondSet (funOf u (ofNat 0))
            (pairSeq (ofNat 0) (funOf a (funOf u (ofNat 1)))) y)
        (g := fun b => pairSeq (ofNat 1) (funOf b (funOf u (ofNat 1))))
        (dcondSet.isScottContinuous_right
```

```
          (funOf u (ofNat 0))
          (pairSeq (ofNat 0) (funOf a (funOf u (ofNat 1)))))
        (theorem_1_3 (pairSeq_isScottContinuous_right (ofNat 1))
          (funOf.isScottContinuous_left (funOf u (ofNat 1)))))

end DomainTheory.DataTypesAsLattices
```

tensorR_right_isScottContinuous.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.pairSeq
public import DomainTheory.DataTypesAsLattices.Retracts.tensorR
public import Mathlib.Order.CompleteLattice.Defs
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.
    pairSeq_isScottContinuous_right
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem tensorR_right_isScottContinuous (a : Pomega) :
    IsScottContinuous (fun b => tensorR a b) :=
  graph_const_isScottContinuous
    (fun b u =>
      pairSeq (funOf a (funOf u (ofNat 0))) (funOf b (funOf u (ofNat 1))))
    (fun u =>
      theorem_1_3
        (f := fun y => pairSeq (funOf a (funOf u (ofNat 0))) y)
        (g := fun b => funOf b (funOf u (ofNat 1)))
        (pairSeq_isScottContinuous_right (funOf a (funOf u (ofNat 0))))
        (funOf.isScottContinuous_left (funOf u (ofNat 1))))

end DomainTheory.DataTypesAsLattices
```

expF.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.tensorR
public import DomainTheory.DataTypesAsLattices.Retracts.intR
public import DomainTheory.DataTypesAsLattices.Retracts.tensor4
```

```
public import DomainTheory.DataTypesAsLattices.Retracts.plus7

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (4.44).** Abstract-syntax functor. -/
def expF (z : Pomega) : Pomega :=
  plus7 intR botElem z z (tensor4 z z z z) (tensorR z z) (tensorR intR z)

end DomainTheory.DataTypesAsLattices
```

expR.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.Ycomb
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.expF

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (4.44).** `exp` as the least fixed point of the seven-tag
syntax functor. -/
def expR : Pomega := funOf Ycomb (graph expF)

end DomainTheory.DataTypesAsLattices
```

iterates_areRetracts.lean

```
module

public import DomainTheory.DataTypesAsLattices.FixedPoint.iterateBot
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.Retract
public import DomainTheory.DataTypesAsLattices.Retracts.bot_isRetract

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem iterates_areRetracts {F : Pomega → Pomega}
    (hret : ∀ a, Retract a → Retract (F a)) :
    ∀ n, Retract (iterateBot F n)
  | 0 => by simpa [iterateBot] using bot_isRetract
  | n + 1 => by simpa [iterateBot] using hret _ (iterates_areRetracts hret n)

end DomainTheory.DataTypesAsLattices
```

iterateBot_isStrict.lean

```
module

public import DomainTheory.DataTypesAsLattices.FixedPoint.iterateBot
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.Retract
public import DomainTheory.DataTypesAsLattices.Retracts.funOf_botElem
public import DomainTheory.DataTypesAsLattices.Retracts.IsStrict
public import DomainTheory.DataTypesAsLattices.Retracts.iterates_areRetracts

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- Strictness of all Kleene approximants of a strictness-preserving
retract functor. -/
theorem iterateBot_isStrict {F : Pomega → Pomega}
    (hret : ∀ a, Retract a → Retract (F a))
    (hstrict : ∀ a, Retract a → IsStrict a → IsStrict (F a)) :
    ∀ n, IsStrict (iterateBot F n)
  | 0 => by simp [iterateBot, IsStrict, funOf_botElem]
  | n + 1 =>
      hstrict _ (iterates_areRetracts hret n)
        (iterateBot_isStrict hret hstrict n)

end DomainTheory.DataTypesAsLattices
```

iterateBot_retractLe.lean

```
module

public import DomainTheory.DataTypesAsLattices.FixedPoint.iterateBot
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.Retract
public import DomainTheory.DataTypesAsLattices.Retracts.retractLe
public import DomainTheory.DataTypesAsLattices.Retracts.IsStrict
public import DomainTheory.DataTypesAsLattices.Retracts.iterates_areRetracts
public import DomainTheory.DataTypesAsLattices.Retracts.bot_retractLe
public import DomainTheory.DataTypesAsLattices.Retracts.iterateBot_isStrict
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace iterateBot_retractLe

/-- Projection-order monotonicity gives adjacent projection equations for
the Kleene chain. -/
theorem succ {F : Pomega → Pomega}
    (hret : ∀ a, Retract a → Retract (F a))
    (hstrict : ∀ a, Retract a → IsStrict a → IsStrict (F a))
    (hmono : ∀ {a b}, Retract a → IsStrict a →
      Retract b → IsStrict b → retractLe a b → retractLe (F a) (F b)) :
    ∀ n, retractLe (iterateBot F n) (iterateBot F (n + 1))
  | 0 => by
      simpa [iterateBot] using
        (bot_retractLe (iterateBot_isStrict hret hstrict 1))
```

```
  | n + 1 => by
      simpa only [iterateBot] using
        hmono (iterates_areRetracts hret n) (iterateBot_isStrict hret hstrict n)
          (iterates_areRetracts hret (n + 1))
          (iterateBot_isStrict hret hstrict (n + 1))
          (?succ? hret hstrict hmono n)

end iterateBot_retractLe

/-- Projection equations between arbitrary comparable Kleene stages. -/
theorem iterateBot_retractLe {F : Pomega → Pomega}
    (hret : ∀ a, Retract a → Retract (F a))
    (hstrict : ∀ a, Retract a → IsStrict a → IsStrict (F a))
    (hmono : ∀ {a b}, Retract a → IsStrict a →
      Retract b → IsStrict b → retractLe a b → retractLe (F a) (F b))
    {n m : ℕ} (hnm : n ≤ m) :
    retractLe (iterateBot F n) (iterateBot F m) := by
  induction m, hnm using Nat.le_induction with
  | base =>
      exact retractLe.refl (iterates_areRetracts hret n)
  | succ m _ ih =>
      exact retractLe.trans ih (iterateBot_retractLe.succ hret hstrict hmono m)

end DomainTheory.DataTypesAsLattices
```

fixedToInverseLimit.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.FixedPoint.iterateBot
public import DomainTheory.DataTypesAsLattices.FixedPoint.fix
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.comp
public import DomainTheory.DataTypesAsLattices.Retracts.Retract
public import DomainTheory.DataTypesAsLattices.Retracts.Fixpoints
public import DomainTheory.DataTypesAsLattices.Retracts.retractLe
public import DomainTheory.DataTypesAsLattices.Retracts.retract_app
public import DomainTheory.DataTypesAsLattices.Retracts.IsStrict
public import DomainTheory.DataTypesAsLattices.Retracts.iterates_areRetracts
public import DomainTheory.DataTypesAsLattices.Retracts.iterateBot_retractLe
public import DomainTheory.DataTypesAsLattices.Retracts.RetractInverseLimit
public import DomainTheory.DataTypesAsLattices.Retracts.vals

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- A point of the fixed-point range determines its sequence of finite
```

```
projections. -/
def fixedToInverseLimit {F : Pomega → Pomega}
    (_hf : IsScottContinuous F)
    (hret : ∀ a, Retract a → Retract (F a))
    (hstrict : ∀ a, Retract a → IsStrict a → IsStrict (F a))
    (hmono : ∀ {a b}, Retract a → IsStrict a →
      Retract b → IsStrict b → retractLe a b → retractLe (F a) (F b))
    (u : Fixpoints (funOf (fix F))) : RetractInverseLimit F :=
  ⟨fun n => funOf (iterateBot F n) u.1,
    ⟨fun n => (retract_app (iterates_areRetracts hret n) u.1).symm,
      fun n => by
        have hle := iterateBot_retractLe.succ hret hstrict hmono n
        have happ := congrArg (fun a => funOf a u.1) hle.1
        simpa only [comp.app] using happ⟩⟩

end DomainTheory.DataTypesAsLattices
```

inverseLimit_project_earlier.lean

```
module

public import DomainTheory.DataTypesAsLattices.FixedPoint.iterateBot
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.comp
public import DomainTheory.DataTypesAsLattices.Retracts.Retract
public import DomainTheory.DataTypesAsLattices.Retracts.retractLe
public import DomainTheory.DataTypesAsLattices.Retracts.IsStrict
public import DomainTheory.DataTypesAsLattices.Retracts.iterateBot_retractLe
public import DomainTheory.DataTypesAsLattices.Retracts.RetractInverseLimit
public import DomainTheory.DataTypesAsLattices.Retracts.typed

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- Earlier typed coordinates are fixed by every later projection. -/
theorem inverseLimit_project_earlier {F : Pomega → Pomega}
    (hret : ∀ a, Retract a → Retract (F a))
    (hstrict : ∀ a, Retract a → IsStrict a → IsStrict (F a))
    (hmono : ∀ {a b}, Retract a → IsStrict a →
      Retract b → IsStrict b → retractLe a b → retractLe (F a) (F b))
    (v : RetractInverseLimit F) {m n : ℕ} (hmn : m ≤ n) :
    funOf (iterateBot F n) (v.1 m) = v.1 m := by
  have hle := iterateBot_retractLe hret hstrict hmono hmn
  have hm : funOf (iterateBot F m) (v.1 m) = v.1 m :=
    (v.property.1 m).symm
  have happ := congrArg (fun a => funOf a (v.1 m)) hle.2
  simpa only [comp.app, hm] using happ.symm

end DomainTheory.DataTypesAsLattices
```

inverseLimit_project_later.lean

```
module
```

```
public import DomainTheory.DataTypesAsLattices.FixedPoint.iterateBot
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.comp
public import DomainTheory.DataTypesAsLattices.Retracts.Retract
public import DomainTheory.DataTypesAsLattices.Retracts.retractLe
public import DomainTheory.DataTypesAsLattices.Retracts.IsStrict
public import DomainTheory.DataTypesAsLattices.Retracts.iterateBot_retractLe
public import DomainTheory.DataTypesAsLattices.Retracts.RetractInverseLimit

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- Repeated coherence: every later coordinate projects to an earlier one. -/
theorem inverseLimit_project_later {F : Pomega → Pomega}
    (hret : ∀ a, Retract a → Retract (F a))
    (hstrict : ∀ a, Retract a → IsStrict a → IsStrict (F a))
    (hmono : ∀ {a b}, Retract a → IsStrict a →
      Retract b → IsStrict b → retractLe a b → retractLe (F a) (F b))
    (v : RetractInverseLimit F) {n m : ℕ} (hnm : n ≤ m) :
    funOf (iterateBot F n) (v.1 m) = v.1 n := by
  induction m, hnm using Nat.le_induction with
  | base => exact (v.property.1 n).symm
  | succ m hnm ih =>
      have hle := iterateBot_retractLe hret hstrict hmono hnm
      have happ :=
        congrArg (fun a => funOf a (v.1 (m + 1))) hle.1
      rw [comp.app, ← v.property.2 m, ih] at happ
      exact happ

end DomainTheory.DataTypesAsLattices
```

inverseLimit_project_iUnion.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.FixedPoint.iterateBot
public import DomainTheory.DataTypesAsLattices.FixedPoint.chain_lemma
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.Retract
public import DomainTheory.DataTypesAsLattices.Retracts.retractLe
public import DomainTheory.DataTypesAsLattices.Retracts.IsStrict
public import DomainTheory.DataTypesAsLattices.Retracts.RetractInverseLimit
public import DomainTheory.DataTypesAsLattices.Retracts.inverseLimit_mono
public import DomainTheory.DataTypesAsLattices.Retracts.inverseLimit_chain
public import DomainTheory.DataTypesAsLattices.Retracts.inverseLimit_project_later
```

```
public import DomainTheory.DataTypesAsLattices.Retracts.inverseLimit_project_earlier
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- Every stage projection of the union of a coherent sequence is its
corresponding coordinate. -/
theorem inverseLimit_project_iUnion {F : Pomega → Pomega}
    (hf : IsScottContinuous F)
    (hret : ∀ a, Retract a → Retract (F a))
    (hstrict : ∀ a, Retract a → IsStrict a → IsStrict (F a))
    (hmono : ∀ {a b}, Retract a → IsStrict a →
      Retract b → IsStrict b → retractLe a b → retractLe (F a) (F b))
    (v : RetractInverseLimit F) (n : ℕ) :
    funOf (iterateBot F n) (⋃ m, v.1 m) = v.1 n := by
  rw [chain_lemma (funOf.isScottContinuous (iterateBot F n)) v.1
    (inverseLimit_mono hf v)]
  apply subset_antisymm
  · intro k hk
    obtain ⟨m, hkm⟩ := Set.mem_iUnion.mp hk
    by_cases hmn : m ≤ n
    · rw [inverseLimit_project_earlier hret hstrict hmono v hmn] at hkm
      exact inverseLimit_chain hf v m n hmn hkm
    · have hnm : n ≤ m := Nat.le_of_lt (Nat.lt_of_not_ge hmn)
      rwa [inverseLimit_project_later hret hstrict hmono v hnm] at hkm
  · intro k hk
    exact Set.mem_iUnion.mpr ⟨n, by
      rw [inverseLimit_project_later hret hstrict hmono v (le_refl n)]
      exact hk⟩

end DomainTheory.DataTypesAsLattices
```

inverseLimit_iUnion_typed.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.FixedPoint.fix
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.Retract
public import DomainTheory.DataTypesAsLattices.Retracts.typed
public import DomainTheory.DataTypesAsLattices.Retracts.retractLe
public import DomainTheory.DataTypesAsLattices.Retracts.IsStrict
public import DomainTheory.DataTypesAsLattices.Retracts.RetractInverseLimit
public import DomainTheory.DataTypesAsLattices.Retracts.inverseLimit_project_iUnion
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
```

```
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- The union of a coherent sequence lies in the range of the least fixed
point retract. -/
theorem inverseLimit_iUnion_typed {F : Pomega → Pomega}
    (hf : IsScottContinuous F)
    (hret : ∀ a, Retract a → Retract (F a))
    (hstrict : ∀ a, Retract a → IsStrict a → IsStrict (F a))
    (hmono : ∀ {a b}, Retract a → IsStrict a →
      Retract b → IsStrict b → retractLe a b → retractLe (F a) (F b))
    (v : RetractInverseLimit F) :
    typed (⋃ n, v.1 n) (fix F) := by
  change (⋃ n, v.1 n) = funOf (fix F) (⋃ n, v.1 n)
  rw [fix, funOf.iUnion]
  apply Eq.symm
  congr 1
  funext n
  exact inverseLimit_project_iUnion hf hret hstrict hmono v n

end DomainTheory.DataTypesAsLattices
```

inverseLimitToFixed.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.FixedPoint.fix
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.Retract
public import DomainTheory.DataTypesAsLattices.Retracts.Fixpoints
public import DomainTheory.DataTypesAsLattices.Retracts.retractLe
public import DomainTheory.DataTypesAsLattices.Retracts.IsStrict
public import DomainTheory.DataTypesAsLattices.Retracts.RetractInverseLimit
public import DomainTheory.DataTypesAsLattices.Retracts.inverseLimit_iUnion_typed
public import DomainTheory.DataTypesAsLattices.Retracts.vals

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- A coherent sequence determines a point of the least fixed-point range
by taking the union of its coordinates. -/
def inverseLimitToFixed {F : Pomega → Pomega}
    (hf : IsScottContinuous F)
    (hret : ∀ a, Retract a → Retract (F a))
    (hstrict : ∀ a, Retract a → IsStrict a → IsStrict (F a))
    (hmono : ∀ {a b}, Retract a → IsStrict a →
```

```
      Retract b → IsStrict b → retractLe a b → retractLe (F a) (F b))
    (v : RetractInverseLimit F) : Fixpoints (funOf (fix F)) :=
  ⟨⋃ n, v.1 n, (inverseLimit_iUnion_typed hf hret hstrict hmono v).symm⟩

end DomainTheory.DataTypesAsLattices
```

lambF.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.arrowR
public import DomainTheory.DataTypesAsLattices.Retracts.plusR
public import DomainTheory.DataTypesAsLattices.Retracts.intR

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (4.39).** `lamb = int ? (lamb o→ lamb)`. -/
def lambF (z : Pomega) : Pomega := plusR intR (arrowR z z)

end DomainTheory.DataTypesAsLattices
```

lambR.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.Ycomb
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.lambF

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def lambR : Pomega := funOf Ycomb (graph lambF)

end DomainTheory.DataTypesAsLattices
```

Hinterp.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Functionality.LambTerm
```

```
public import DomainTheory.DataTypesAsLattices.Functionality.continuous_nary
public import DomainTheory.DataTypesAsLattices.Functionality.continuous_tuple
public import DomainTheory.DataTypesAsLattices.Functionality.inleftC
public import DomainTheory.DataTypesAsLattices.Functionality.inrightC
public import DomainTheory.DataTypesAsLattices.Functionality.outC
public import DomainTheory.DataTypesAsLattices.Functionality.whichC
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.lam
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.const_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.predSet
public import DomainTheory.DataTypesAsLattices.Lambda.predSet_more
public import DomainTheory.DataTypesAsLattices.Lambda.succSet
public import DomainTheory.DataTypesAsLattices.Lambda.succSet_more
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.lambR
public import DomainTheory.DataTypesAsLattices.Retracts.updateEnv

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def Hinterp : LambTerm → Pomega → Pomega
  | .var n, t => funOf t (ofNat n)
  | .zero, _ => funOf inleftC (ofNat 0)
  | .succ τ, t =>
      condSet (funOf whichC (Hinterp τ t))
        (funOf inleftC (succSet (funOf outC (Hinterp τ t))))
        botElem
  | .pred τ, t =>
      condSet (funOf whichC (Hinterp τ t))
        (funOf inleftC (predSet (funOf outC (Hinterp τ t))))
        botElem
  | .condq ? τ σ ?, t =>
      funOf lambR (condSet (funOf whichC (Hinterp ? t))
        (condSet (funOf outC (Hinterp ? t)) (Hinterp τ t) (Hinterp σ t))
        (Hinterp ? t))
  | .app τ σ, t =>
      condSet (funOf whichC (Hinterp τ t)) botElem
        (funOf (funOf outC (Hinterp τ t)) (Hinterp σ t))
  | .lam n τ, t =>
      funOf inrightC (graph (fun x => Hinterp τ (updateEnv t x n)))

namespace Hinterp

theorem isScottContinuous (τ : LambTerm) :
    IsScottContinuous (Hinterp τ) := by
  induction τ with
  | var n =>
    exact funOf.isScottContinuous_left (ofNat n)
  | zero =>
    exact const_isScottContinuous _
```

```
| succ τ ih =>
  exact continuous_nary
    (fun y z => condSet.isScottContinuous_left y z)
    (fun x z => condSet.isScottContinuous_mid x z)
    (fun x y => condSet.isScottContinuous_right x y)
    (theorem_1_3 (funOf.isScottContinuous whichC) ih)
    (theorem_1_3 (funOf.isScottContinuous inleftC)
      (theorem_1_3 succSet.isScottContinuous
        (theorem_1_3 (funOf.isScottContinuous outC) ih)))
    (const_isScottContinuous botElem)
| pred τ ih =>
  exact continuous_nary
    (fun y z => condSet.isScottContinuous_left y z)
    (fun x z => condSet.isScottContinuous_mid x z)
    (fun x y => condSet.isScottContinuous_right x y)
    (theorem_1_3 (funOf.isScottContinuous whichC) ih)
    (theorem_1_3 (funOf.isScottContinuous inleftC)
      (theorem_1_3 predSet.isScottContinuous
        (theorem_1_3 (funOf.isScottContinuous outC) ih)))
    (const_isScottContinuous botElem)
| condq ? τ σ ? ih? ihτ ihσ ih? =>
  exact theorem_1_3 (f := funOf lambR)
    (g := fun t =>
      condSet (funOf whichC (Hinterp ? t))
        (condSet (funOf outC (Hinterp ? t)) (Hinterp τ t) (Hinterp σ t))
        (Hinterp ? t))
    (funOf.isScottContinuous lambR)
    (continuous_nary
      (fun y z => condSet.isScottContinuous_left y z)
      (fun x z => condSet.isScottContinuous_mid x z)
      (fun x y => condSet.isScottContinuous_right x y)
      (theorem_1_3 (funOf.isScottContinuous whichC) ih?)
      (continuous_nary
        (fun y z => condSet.isScottContinuous_left y z)
        (fun x z => condSet.isScottContinuous_mid x z)
        (fun x y => condSet.isScottContinuous_right x y)
        (theorem_1_3 (funOf.isScottContinuous outC) ih?) ihτ ihσ)
      ih?)
| app τ σ ihτ ihσ =>
  exact continuous_nary
    (fun y z => condSet.isScottContinuous_left y z)
    (fun x z => condSet.isScottContinuous_mid x z)
    (fun x y => condSet.isScottContinuous_right x y)
    (theorem_1_3 (funOf.isScottContinuous whichC) ihτ)
    (const_isScottContinuous botElem)
    (continuous_tuple
      (fun y => funOf.isScottContinuous_left y)
      (fun u => funOf.isScottContinuous u)
      (theorem_1_3 (funOf.isScottContinuous outC) ihτ) ihσ)
| lam n τ ih =>
  exact theorem_1_3 (f := funOf inrightC)
    (g := fun t => graph (fun x => Hinterp τ (updateEnv t x n)))
    (funOf.isScottContinuous inrightC)
    (graph_const_isScottContinuous
      (fun t x => Hinterp τ (updateEnv t x n))
      (fun x => theorem_1_3 ih
```

```
          (graph_const_isScottContinuous
            (fun t m => if m = ofNat n then x else funOf t m)
            (fun m => by
              by_cases h : m = ofNat n
              · simpa [h] using const_isScottContinuous (c := x)
              · simpa [h] using funOf.isScottContinuous_left m))))

end Hinterp

end DomainTheory.DataTypesAsLattices
```

hCond.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Functionality.continuous_nary
public import DomainTheory.DataTypesAsLattices.Functionality.expPayload
public import DomainTheory.DataTypesAsLattices.Functionality.hApply
public import DomainTheory.DataTypesAsLattices.Functionality.outC
public import DomainTheory.DataTypesAsLattices.Functionality.whichC
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.lambR

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def hCond (H e t : Pomega) : Pomega :=
  funOf lambR
    (condSet (funOf whichC (hApply H (funOf (expPayload e) (ofNat 0)) t))
      (condSet (funOf outC (hApply H (funOf (expPayload e) (ofNat 0)) t))
        (hApply H (funOf (expPayload e) (ofNat 1)) t)
        (hApply H (funOf (expPayload e) (ofNat 2)) t))
      (hApply H (funOf (expPayload e) (ofNat 3)) t))

namespace hCond

theorem isScottContinuous_H (e t : Pomega) :
    IsScottContinuous (fun H => hCond H e t) :=
  theorem_1_3 (funOf.isScottContinuous lambR)
    (continuous_nary
      (fun y z => condSet.isScottContinuous_left y z)
      (fun x z => condSet.isScottContinuous_mid x z)
      (fun x y => condSet.isScottContinuous_right x y)
```

```
      (theorem_1_3 (funOf.isScottContinuous whichC)
        (hApply.isScottContinuous_H _ t))
      (continuous_nary
        (fun y z => condSet.isScottContinuous_left y z)
        (fun x z => condSet.isScottContinuous_mid x z)
        (fun x y => condSet.isScottContinuous_right x y)
        (theorem_1_3 (funOf.isScottContinuous outC)
          (hApply.isScottContinuous_H _ t))
        (hApply.isScottContinuous_H _ t)
        (hApply.isScottContinuous_H _ t))
      (hApply.isScottContinuous_H _ t))

theorem isScottContinuous_t (H e : Pomega) :
    IsScottContinuous (fun t => hCond H e t) :=
  theorem_1_3 (f := funOf lambR)
    (g := fun t =>
      condSet (funOf whichC (hApply H (funOf (expPayload e) (ofNat 0)) t))
        (condSet (funOf outC (hApply H (funOf (expPayload e) (ofNat 0)) t))
          (hApply H (funOf (expPayload e) (ofNat 1)) t)
          (hApply H (funOf (expPayload e) (ofNat 2)) t))
        (hApply H (funOf (expPayload e) (ofNat 3)) t))
    (funOf.isScottContinuous lambR)
    (continuous_nary
      (fun y z => condSet.isScottContinuous_left y z)
      (fun x z => condSet.isScottContinuous_mid x z)
      (fun x y => condSet.isScottContinuous_right x y)
      (theorem_1_3 (f := funOf whichC)
        (g := fun t => hApply H (funOf (expPayload e) (ofNat 0)) t)
        (funOf.isScottContinuous whichC)
        (funOf.isScottContinuous (funOf H (funOf (expPayload e) (ofNat 0)))))
      (continuous_nary
        (fun y z => condSet.isScottContinuous_left y z)
        (fun x z => condSet.isScottContinuous_mid x z)
        (fun x y => condSet.isScottContinuous_right x y)
        (theorem_1_3 (f := funOf outC)
          (g := fun t => hApply H (funOf (expPayload e) (ofNat 0)) t)
          (funOf.isScottContinuous outC)
          (funOf.isScottContinuous (funOf H (funOf (expPayload e) (ofNat 0)))))
        (funOf.isScottContinuous (funOf H (funOf (expPayload e) (ofNat 1))))
        (funOf.isScottContinuous (funOf H (funOf (expPayload e) (ofNat 2)))))
      (funOf.isScottContinuous (funOf H (funOf (expPayload e) (ofNat 3)))))

theorem isScottContinuous_e (H t : Pomega) :
    IsScottContinuous (fun e => hCond H e t) :=
  theorem_1_3 (f := funOf lambR)
    (g := fun e =>
      condSet (funOf whichC (hApply H (funOf (expPayload e) (ofNat 0)) t))
        (condSet (funOf outC (hApply H (funOf (expPayload e) (ofNat 0)) t))
          (hApply H (funOf (expPayload e) (ofNat 1)) t)
          (hApply H (funOf (expPayload e) (ofNat 2)) t))
        (hApply H (funOf (expPayload e) (ofNat 3)) t))
    (funOf.isScottContinuous lambR)
    (continuous_nary
      (fun y z => condSet.isScottContinuous_left y z)
      (fun x z => condSet.isScottContinuous_mid x z)
      (fun x y => condSet.isScottContinuous_right x y)
```

```
      (theorem_1_3 (f := funOf whichC)
        (g := fun e => hApply H (funOf (expPayload e) (ofNat 0)) t)
        (funOf.isScottContinuous whichC)
        (hApply.payload_isScottContinuous_e H t 0))
      (continuous_nary
        (fun y z => condSet.isScottContinuous_left y z)
        (fun x z => condSet.isScottContinuous_mid x z)
        (fun x y => condSet.isScottContinuous_right x y)
        (theorem_1_3 (f := funOf outC)
          (g := fun e => hApply H (funOf (expPayload e) (ofNat 0)) t)
          (funOf.isScottContinuous outC)
          (hApply.payload_isScottContinuous_e H t 0))
        (hApply.payload_isScottContinuous_e H t 1)
        (hApply.payload_isScottContinuous_e H t 2))
      (hApply.payload_isScottContinuous_e H t 3))

end hCond

end DomainTheory.DataTypesAsLattices
```

Hstep.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Functionality.continuous_nary
public import DomainTheory.DataTypesAsLattices.Functionality.expPayload
public import DomainTheory.DataTypesAsLattices.Functionality.expTag
public import DomainTheory.DataTypesAsLattices.Functionality.hApp
public import DomainTheory.DataTypesAsLattices.Functionality.hCond
public import DomainTheory.DataTypesAsLattices.Functionality.hLam
public import DomainTheory.DataTypesAsLattices.Functionality.hPred
public import DomainTheory.DataTypesAsLattices.Functionality.hSucc
public import DomainTheory.DataTypesAsLattices.Functionality.hVar
public import DomainTheory.DataTypesAsLattices.Functionality.hZero
public import DomainTheory.DataTypesAsLattices.Functionality.
    predSet_iterate_expTag_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Functionality.predSet_iterate_ofNat
public import DomainTheory.DataTypesAsLattices.Functionality.predSet_iterate_self
public import DomainTheory.DataTypesAsLattices.Functionality.tagInj_one
public import DomainTheory.DataTypesAsLattices.Functionality.tagInj_zero
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
public import DomainTheory.DataTypesAsLattices.Lambda.const_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.dcondSet
public import DomainTheory.DataTypesAsLattices.Lambda.dcondSet_more
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.predSet
public import DomainTheory.DataTypesAsLattices.Lambda.predSet_more
public import DomainTheory.DataTypesAsLattices.Pomega.e
```

```
public import DomainTheory.DataTypesAsLattices.Retracts.tagInj
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- One continuous step of (4.43), dispatched on the (4.44) tags. -/
def Hstep (H e t : Pomega) : Pomega :=
  dcondSet (expTag e) (hVar e t)
    (dcondSet (predSet (expTag e)) hZero
      (dcondSet (predSet (predSet (expTag e))) (hSucc H e t)
        (dcondSet (predSet^[3] (expTag e)) (hPred H e t)
          (dcondSet (predSet^[4] (expTag e)) (hCond H e t)
            (dcondSet (predSet^[5] (expTag e)) (hApp H e t)
              (dcondSet (predSet^[6] (expTag e)) (hLam H e t)
                topElem))))))

namespace Hstep

theorem tag0 (H x t : Pomega) :
    Hstep H (tagInj 0 x) t = funOf t x := by
  simp only [Hstep, expTag, expPayload, hVar, tagInj_zero, tagInj_one,
    dcondSet.ofNat_zero]

theorem tag1 (H x t : Pomega) :
    Hstep H (tagInj 1 x) t = hZero := by
  simp only [Hstep, expTag, tagInj_zero, predSet.ofNat_succ,
    dcondSet.ofNat_one, dcondSet.ofNat_zero]

theorem tag2 (H x t : Pomega) :
    Hstep H (tagInj 2 x) t = hSucc H (tagInj 2 x) t := by
  simp only [Hstep, expTag, tagInj_zero, predSet.ofNat_succ,
    dcondSet.ofNat_succ, dcondSet.ofNat_zero]

theorem tag3 (H x t : Pomega) :
    Hstep H (tagInj 3 x) t = hPred H (tagInj 3 x) t := by
  simp only [Hstep, expTag, tagInj_zero, dcondSet.ofNat_succ,
    predSet.ofNat_succ, predSet_iterate_self,
    dcondSet.ofNat_zero]

theorem tag4 (H x t : Pomega) :
    Hstep H (tagInj 4 x) t = hCond H (tagInj 4 x) t := by
  simp only [Hstep, expTag, tagInj_zero, dcondSet.ofNat_succ,
    predSet.ofNat_succ, predSet_iterate_ofNat (k := 3) (i := 1),
    predSet_iterate_self, dcondSet.ofNat_zero]

theorem tag5 (H x t : Pomega) :
    Hstep H (tagInj 5 x) t = hApp H (tagInj 5 x) t := by
  simp only [Hstep, expTag, tagInj_zero, dcondSet.ofNat_succ,
    predSet.ofNat_succ, predSet_iterate_ofNat (k := 3) (i := 2),
    predSet_iterate_ofNat (k := 4) (i := 1),
    predSet_iterate_self, dcondSet.ofNat_zero]

theorem tag6 (H x t : Pomega) :
    Hstep H (tagInj 6 x) t = hLam H (tagInj 6 x) t := by
  simp only [Hstep, expTag, tagInj_zero, dcondSet.ofNat_succ,
    predSet.ofNat_succ, predSet_iterate_ofNat (k := 3) (i := 3),
```

```
    predSet_iterate_ofNat (k := 4) (i := 2),
    predSet_iterate_ofNat (k := 5) (i := 1),
    predSet_iterate_self, dcondSet.ofNat_zero]

theorem isScottContinuous_H (e t : Pomega) :
    IsScottContinuous (fun H => Hstep H e t) :=
  continuous_nary
    (fun y z => dcondSet.isScottContinuous_left y z)
    (fun x z => dcondSet.isScottContinuous_mid x z)
    (fun x y => dcondSet.isScottContinuous_right x y)
    (const_isScottContinuous (expTag e)) (const_isScottContinuous (hVar e t))
    (continuous_nary
      (fun y z => dcondSet.isScottContinuous_left y z)
      (fun x z => dcondSet.isScottContinuous_mid x z)
      (fun x y => dcondSet.isScottContinuous_right x y)
      (const_isScottContinuous (predSet (expTag e)))
      (const_isScottContinuous hZero)
      (continuous_nary
        (fun y z => dcondSet.isScottContinuous_left y z)
        (fun x z => dcondSet.isScottContinuous_mid x z)
        (fun x y => dcondSet.isScottContinuous_right x y)
        (const_isScottContinuous (predSet (predSet (expTag e))))
        (hSucc.isScottContinuous_H e t)
        (continuous_nary
          (fun y z => dcondSet.isScottContinuous_left y z)
          (fun x z => dcondSet.isScottContinuous_mid x z)
          (fun x y => dcondSet.isScottContinuous_right x y)
          (const_isScottContinuous (predSet^[3] (expTag e)))
          (hPred.isScottContinuous_H e t)
          (continuous_nary
            (fun y z => dcondSet.isScottContinuous_left y z)
            (fun x z => dcondSet.isScottContinuous_mid x z)
            (fun x y => dcondSet.isScottContinuous_right x y)
            (const_isScottContinuous (predSet^[4] (expTag e)))
            (hCond.isScottContinuous_H e t)
            (continuous_nary
              (fun y z => dcondSet.isScottContinuous_left y z)
              (fun x z => dcondSet.isScottContinuous_mid x z)
              (fun x y => dcondSet.isScottContinuous_right x y)
              (const_isScottContinuous (predSet^[5] (expTag e)))
              (hApp.isScottContinuous_H e t)
              (continuous_nary
                (fun y z => dcondSet.isScottContinuous_left y z)
                (fun x z => dcondSet.isScottContinuous_mid x z)
                (fun x y => dcondSet.isScottContinuous_right x y)
                (const_isScottContinuous (predSet^[6] (expTag e)))
                (hLam.isScottContinuous_H e t)
                (const_isScottContinuous topElem)))))))

theorem isScottContinuous_t (H e : Pomega) :
    IsScottContinuous (fun t => Hstep H e t) :=
  continuous_nary
    (fun y z => dcondSet.isScottContinuous_left y z)
    (fun x z => dcondSet.isScottContinuous_mid x z)
    (fun x y => dcondSet.isScottContinuous_right x y)
    (const_isScottContinuous (expTag e))
```

```
    (hVar.isScottContinuous_t e)
    (continuous_nary
      (fun y z => dcondSet.isScottContinuous_left y z)
      (fun x z => dcondSet.isScottContinuous_mid x z)
      (fun x y => dcondSet.isScottContinuous_right x y)
      (const_isScottContinuous (predSet (expTag e)))
      (const_isScottContinuous hZero)
      (continuous_nary
        (fun y z => dcondSet.isScottContinuous_left y z)
        (fun x z => dcondSet.isScottContinuous_mid x z)
        (fun x y => dcondSet.isScottContinuous_right x y)
        (const_isScottContinuous (predSet (predSet (expTag e))))
        (hSucc.isScottContinuous_t H e)
        (continuous_nary
          (fun y z => dcondSet.isScottContinuous_left y z)
          (fun x z => dcondSet.isScottContinuous_mid x z)
          (fun x y => dcondSet.isScottContinuous_right x y)
          (const_isScottContinuous (predSet^[3] (expTag e)))
          (hPred.isScottContinuous_t H e)
          (continuous_nary
            (fun y z => dcondSet.isScottContinuous_left y z)
            (fun x z => dcondSet.isScottContinuous_mid x z)
            (fun x y => dcondSet.isScottContinuous_right x y)
            (const_isScottContinuous (predSet^[4] (expTag e)))
            (hCond.isScottContinuous_t H e)
            (continuous_nary
              (fun y z => dcondSet.isScottContinuous_left y z)
              (fun x z => dcondSet.isScottContinuous_mid x z)
              (fun x y => dcondSet.isScottContinuous_right x y)
              (const_isScottContinuous (predSet^[5] (expTag e)))
              (hApp.isScottContinuous_t H e)
              (continuous_nary
                (fun y z => dcondSet.isScottContinuous_left y z)
                (fun x z => dcondSet.isScottContinuous_mid x z)
                (fun x y => dcondSet.isScottContinuous_right x y)
                (const_isScottContinuous (predSet^[6] (expTag e)))
                (hLam.isScottContinuous_t H e)
                (const_isScottContinuous topElem)))))))

theorem isScottContinuous_e (H t : Pomega) :
    IsScottContinuous (fun e => Hstep H e t) :=
  continuous_nary
    (fun y z => dcondSet.isScottContinuous_left y z)
    (fun x z => dcondSet.isScottContinuous_mid x z)
    (fun x y => dcondSet.isScottContinuous_right x y)
    (funOf.isScottContinuous_left (ofNat 0))
    (hVar.isScottContinuous_e t)
    (continuous_nary
      (fun y z => dcondSet.isScottContinuous_left y z)
      (fun x z => dcondSet.isScottContinuous_mid x z)
      (fun x y => dcondSet.isScottContinuous_right x y)
      (predSet_iterate_expTag_isScottContinuous 1)
      (const_isScottContinuous hZero)
      (continuous_nary
        (fun y z => dcondSet.isScottContinuous_left y z)
        (fun x z => dcondSet.isScottContinuous_mid x z)
```

```
        (fun x y => dcondSet.isScottContinuous_right x y)
        (predSet_iterate_expTag_isScottContinuous 2)
        (hSucc.isScottContinuous_e H t)
        (continuous_nary
          (fun y z => dcondSet.isScottContinuous_left y z)
          (fun x z => dcondSet.isScottContinuous_mid x z)
          (fun x y => dcondSet.isScottContinuous_right x y)
          (predSet_iterate_expTag_isScottContinuous 3)
          (hPred.isScottContinuous_e H t)
          (continuous_nary
            (fun y z => dcondSet.isScottContinuous_left y z)
            (fun x z => dcondSet.isScottContinuous_mid x z)
            (fun x y => dcondSet.isScottContinuous_right x y)
            (predSet_iterate_expTag_isScottContinuous 4)
            (hCond.isScottContinuous_e H t)
            (continuous_nary
              (fun y z => dcondSet.isScottContinuous_left y z)
              (fun x z => dcondSet.isScottContinuous_mid x z)
              (fun x y => dcondSet.isScottContinuous_right x y)
              (predSet_iterate_expTag_isScottContinuous 5)
              (hApp.isScottContinuous_e H t)
              (continuous_nary
                (fun y z => dcondSet.isScottContinuous_left y z)
                (fun x z => dcondSet.isScottContinuous_mid x z)
                (fun x y => dcondSet.isScottContinuous_right x y)
                (predSet_iterate_expTag_isScottContinuous 6)
                (hLam.isScottContinuous_e H t)
                (const_isScottContinuous topElem)))))))
end Hstep

end DomainTheory.DataTypesAsLattices
```

envR.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.lambR

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (4.40).** `env = λt. lamb ∘ seq(t)`. -/
def envR : Pomega :=
  graph (fun t => graph (fun n => funOf lambR (funOf t n)))

end DomainTheory.DataTypesAsLattices
```

HF.lean

```
module
```

```
public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Functionality.Hstep
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def HF (H : Pomega) : Pomega :=
  graph (fun e => graph (fun t => Hstep H e t))

namespace HF

theorem inner_isScottContinuous (H : Pomega) :
    IsScottContinuous (fun e => graph (fun t => Hstep H e t)) :=
  graph_const_isScottContinuous (fun e t => Hstep H e t)
    (fun t => Hstep.isScottContinuous_e H t)

theorem isScottContinuous : IsScottContinuous HF :=
  graph_const_isScottContinuous
    (fun H e => graph (fun t => Hstep H e t))
    (fun e => graph_const_isScottContinuous (fun H t => Hstep H e t)
      (fun t => Hstep.isScottContinuous_H e t))

theorem app (H e : Pomega) :
    funOf (HF H) e = graph (fun t => Hstep H e t) :=
  beta (HF.inner_isScottContinuous H) e

theorem app2 (H e t : Pomega) :
    funOf (funOf (HF H) e) t = Hstep H e t := by
  rw [HF.app]
  exact beta (Hstep.isScottContinuous_t H e) t

end HF

end DomainTheory.DataTypesAsLattices
```

Hfun.lean

```
module

public import DomainTheory.DataTypesAsLattices.FixedPoint.fix
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_4
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.lam
```

```
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.LambTerm
public import DomainTheory.DataTypesAsLattices.Functionality.HF
public import DomainTheory.DataTypesAsLattices.Functionality.Hinterp
public import DomainTheory.DataTypesAsLattices.Functionality.encodeExp
public import DomainTheory.DataTypesAsLattices.Functionality.expPayload
public import DomainTheory.DataTypesAsLattices.Functionality.hApply
public import DomainTheory.DataTypesAsLattices.Functionality.hSucc
public import DomainTheory.DataTypesAsLattices.Functionality.hPred
public import DomainTheory.DataTypesAsLattices.Functionality.hCond
public import DomainTheory.DataTypesAsLattices.Functionality.hApp
public import DomainTheory.DataTypesAsLattices.Functionality.hLam
public import DomainTheory.DataTypesAsLattices.Functionality.Hstep
public import DomainTheory.DataTypesAsLattices.Functionality.pairSeq_app_zero
public import DomainTheory.DataTypesAsLattices.Functionality.pairSeq_app_one
public import DomainTheory.DataTypesAsLattices.Functionality.tagInj_one
public import DomainTheory.DataTypesAsLattices.Functionality.seq4_app
public import DomainTheory.DataTypesAsLattices.Functionality.updateEnvUnion

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- Raw least fixed point of the (4.43) operator. -/
def Hfun : Pomega := fix HF

namespace Hfun

theorem fixed : HF Hfun = Hfun :=
  (theorem_1_4 HF.isScottContinuous).1

theorem app2 (e t : Pomega) :
    funOf (funOf Hfun e) t = Hstep Hfun e t := by
  have h := congrArg (fun H => funOf (funOf H e) t) fixed
  simpa [HF.app2] using h.symm

theorem interp : ∀ τ : LambTerm, ∀ t : Pomega,
    funOf (funOf Hfun (encodeExp τ)) t = Hinterp τ t := by
  intro τ
  induction τ with
  | var n =>
    intro t
    rw [app2, encodeExp, Hstep.tag0]
    rfl
  | zero =>
    intro t
    rw [app2, encodeExp, Hstep.tag1]
    rfl
  | succ τ ih =>
    intro t
    rw [app2, encodeExp, Hstep.tag2]
    simp only [hSucc, hApply, expPayload, tagInj_one]
    rw [ih]
    rfl
  | pred τ ih =>
    intro t
```

```
    rw [app2, encodeExp, Hstep.tag3]
    simp only [hPred, hApply, expPayload, tagInj_one]
    rw [ih]
    rfl
  | condq ? τ σ ? ih? ihτ ihσ ih? =>
    intro t
    rw [app2, encodeExp, Hstep.tag4]
    simp only [hCond, hApply, expPayload, tagInj_one, seq4_app.zero,
      seq4_app.one, seq4_app.two, seq4_app.three]
    rw [ih?, ihτ, ihσ, ih?]
    rfl
  | app τ σ ihτ ihσ =>
    intro t
    rw [app2, encodeExp, Hstep.tag5]
    simp only [hApp, hApply, expPayload, tagInj_one, pairSeq_app_zero,
      pairSeq_app_one]
    rw [ihτ, ihσ]
    rfl
  | lam n τ ih =>
    intro t
    rw [app2, encodeExp, Hstep.tag6]
    simp only [hLam, hApply, expPayload, tagInj_one, pairSeq_app_zero,
      pairSeq_app_one, updateEnvUnion.ofNat]
    congr 1
    apply congrArg graph
    funext x
    exact ih _

theorem decode (τ : LambTerm) :
    funOf Hfun (encodeExp τ) = graph (Hinterp τ) := by
  have hβ : funOf Hfun (encodeExp τ) = graph (fun t => Hstep Hfun (encodeExp τ) t) :=
    by
    have := congrArg (fun H => funOf H (encodeExp τ)) fixed
    simpa [HF.app] using this.symm
  refine hβ.trans ?_
  apply congrArg graph
  funext t
  simpa [app2] using interp τ t

end Hfun

end DomainTheory.DataTypesAsLattices
```

envR_map_isScottContinuous.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.lambR
public import Mathlib.Order.CompleteLattice.Defs
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
```

```
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem envR_map_isScottContinuous :
    IsScottContinuous (fun t => graph (fun n => funOf lambR (funOf t n))) :=
  graph_const_isScottContinuous
    (fun t n => funOf lambR (funOf t n))
    (fun n => theorem_1_3 (funOf.isScottContinuous lambR)
      (funOf.isScottContinuous_left n))

end DomainTheory.DataTypesAsLattices
```

envR_app.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.lambR
public import DomainTheory.DataTypesAsLattices.Retracts.envR
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.
    envR_map_isScottContinuous

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem envR_app (t : Pomega) :
    funOf envR t = graph (fun n => funOf lambR (funOf t n)) :=
  beta envR_map_isScottContinuous t

end DomainTheory.DataTypesAsLattices
```

lambF_isScottContinuous.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.arrowR
public import DomainTheory.DataTypesAsLattices.Retracts.plusR
public import DomainTheory.DataTypesAsLattices.Retracts.intR
public import DomainTheory.DataTypesAsLattices.Retracts.lambF
public import Mathlib.Order.CompleteLattice.Defs
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
```

```
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Functionality.
    plusR_right_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Functionality.
    arrowR_diag_isScottContinuous

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem lambF_isScottContinuous : IsScottContinuous lambF :=
  theorem_1_3 (f := fun b => plusR intR b) (g := fun z => arrowR z z)
    (plusR_right_isScottContinuous intR)
    arrowR_diag_isScottContinuous

end DomainTheory.DataTypesAsLattices
```

retract_isGraph.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.theorem_1_2
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.Retract
public import DomainTheory.DataTypesAsLattices.Retracts.retract_app

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem retract_isGraph {a : Pomega} (ha : Retract a) : graph (funOf a) = a := by
  apply subset_antisymm
  · intro p hp
    rcases hp with ⟨n, m, rfl, hm⟩
    have ha' : a = graph (fun x => funOf a (funOf a x)) := ha.out
    have : m ∈ funOf a (funOf a (e n)) := by
      simpa [retract_app ha] using hm
    rw [ha']
    exact ⟨n, m, rfl, this⟩
  · exact theorem_1_2.ii a

end DomainTheory.DataTypesAsLattices
```

iUnion_retract_isGraph.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.IsGraph
public import DomainTheory.DataTypesAsLattices.Graph.theorem_1_2
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.Retract
public import DomainTheory.DataTypesAsLattices.Retracts.retract_isGraph
```

```
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem iUnion_retract_isGraph {xs : ℕ → Pomega} (h : ∀ n, Retract (xs n)) :
    IsGraph (⋃ n, xs n) := by
  constructor
  intro k m n hkm hkn
  obtain ⟨i, hi⟩ := Set.mem_iUnion.mp hkm
  have hiG : IsGraph (xs i) := (theorem_1_2.iii (xs i)).mp (retract_isGraph (h i))
  exact Set.mem_iUnion.mpr ⟨i, hiG.out hi hkn⟩

end DomainTheory.DataTypesAsLattices
```

pairSeq_isScottContinuous_left.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.pairSeq
public import Mathlib.Order.CompleteLattice.Defs
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Lambda.predSet
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem pairSeq_isScottContinuous_left (y : Pomega) :
    IsScottContinuous (fun x => pairSeq x y) :=
  graph_const_isScottContinuous
    (fun x z => condSet z x (condSet (predSet z) y botElem))
    (fun z => condSet.isScottContinuous_mid z _)

end DomainTheory.DataTypesAsLattices
```

tensorR_diag_isScottContinuous.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.pairSeq
public import DomainTheory.DataTypesAsLattices.Retracts.tensorR
public import Mathlib.Order.CompleteLattice.Defs
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import Mathlib.Topology.Bases
```

```
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Functionality.continuous_tuple
public import DomainTheory.DataTypesAsLattices.Functionality.
    pairSeq_isScottContinuous_left
public import DomainTheory.DataTypesAsLattices.Functionality.
    pairSeq_isScottContinuous_right
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem tensorR_diag_isScottContinuous :
    IsScottContinuous (fun z => tensorR z z) :=
  graph_const_isScottContinuous
    (fun z u =>
      pairSeq (funOf z (funOf u (ofNat 0))) (funOf z (funOf u (ofNat 1))))
    (fun u =>
      continuous_tuple
        (fun y => pairSeq_isScottContinuous_left y)
        (fun x => pairSeq_isScottContinuous_right x)
        (funOf.isScottContinuous_left (funOf u (ofNat 0)))
        (funOf.isScottContinuous_left (funOf u (ofNat 1))))

end DomainTheory.DataTypesAsLattices
```

tensorR_map_isScottContinuous.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.pairSeq
public import Mathlib.Order.CompleteLattice.Defs
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.continuous_tuple
public import DomainTheory.DataTypesAsLattices.Functionality.
    pairSeq_isScottContinuous_left
public import DomainTheory.DataTypesAsLattices.Functionality.
    pairSeq_isScottContinuous_right
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more

@[expose] public section
```

```
namespace DomainTheory.DataTypesAsLattices

theorem tensorR_map_isScottContinuous (a b : Pomega) :
    IsScottContinuous (fun u =>
      pairSeq (funOf a (funOf u (ofNat 0))) (funOf b (funOf u (ofNat 1)))) :=
  continuous_tuple
    (fun y => pairSeq_isScottContinuous_left y)
    (fun x => pairSeq_isScottContinuous_right x)
    (theorem_1_3 (funOf.isScottContinuous a) (funOf.isScottContinuous_left (ofNat 0)))
    (theorem_1_3 (funOf.isScottContinuous b) (funOf.isScottContinuous_left (ofNat 1)))

end DomainTheory.DataTypesAsLattices
```

tensorR_app.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.pairSeq
public import DomainTheory.DataTypesAsLattices.Retracts.tensorR
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.
    tensorR_map_isScottContinuous

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem tensorR_app (a b u : Pomega) :
    funOf (tensorR a b) u =
      pairSeq (funOf a (funOf u (ofNat 0))) (funOf b (funOf u (ofNat 1))) :=
  beta (tensorR_map_isScottContinuous a b) u

end DomainTheory.DataTypesAsLattices
```

tensorR_isRetract.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.Retract
public import DomainTheory.DataTypesAsLattices.Retracts.tensorR
public import DomainTheory.DataTypesAsLattices.Retracts.retract_app
public import DomainTheory.DataTypesAsLattices.Retracts.graph_ext
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.pairSeq_app_zero
public import DomainTheory.DataTypesAsLattices.Functionality.pairSeq_app_one
public import DomainTheory.DataTypesAsLattices.Functionality.tensorR_app

@[expose] public section
```

```
namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, Theorem 4.4.** Products of retracts are retracts. -/
theorem tensorR_isRetract {a b : Pomega} (ha : Retract a) (hb : Retract b) :
    Retract (tensorR a b) := by
  constructor
  change tensorR a b =
    graph (fun u => funOf (tensorR a b) (funOf (tensorR a b) u))
  apply graph_ext
  intro u
  rw [tensorR_app, tensorR_app, pairSeq_app_zero, pairSeq_app_one,
    retract_app ha, retract_app hb]

end DomainTheory.DataTypesAsLattices
```

seq4_comp_isScottContinuous.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.seq4
public import Mathlib.Order.CompleteLattice.Defs
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.const_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Lambda.predSet
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.continuous_nary

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem seq4_comp_isScottContinuous
    {f g h i : Pomega → Pomega}
    (hf : IsScottContinuous f) (hg : IsScottContinuous g)
    (hh : IsScottContinuous h) (hi : IsScottContinuous i) :
    IsScottContinuous (fun u => seq4 (f u) (g u) (h u) (i u)) :=
  graph_const_isScottContinuous
    (fun u z =>
      condSet z (f u)
        (condSet (predSet z) (g u)
          (condSet (predSet (predSet z)) (h u)
            (condSet (predSet (predSet (predSet z))) (i u) botElem))))
    (fun z =>
      continuous_nary
        (fun y t => condSet.isScottContinuous_left y t)
        (fun x t => condSet.isScottContinuous_mid x t)
        (fun x y => condSet.isScottContinuous_right x y)
```

```
        (const_isScottContinuous z) hf
        (continuous_nary
          (fun y t => condSet.isScottContinuous_left y t)
          (fun x t => condSet.isScottContinuous_mid x t)
          (fun x y => condSet.isScottContinuous_right x y)
          (const_isScottContinuous (predSet z)) hg
          (continuous_nary
            (fun y t => condSet.isScottContinuous_left y t)
            (fun x t => condSet.isScottContinuous_mid x t)
            (fun x y => condSet.isScottContinuous_right x y)
            (const_isScottContinuous (predSet (predSet z))) hh
            (continuous_nary
              (fun y t => condSet.isScottContinuous_left y t)
              (fun x t => condSet.isScottContinuous_mid x t)
              (fun x y => condSet.isScottContinuous_right x y)
              (const_isScottContinuous (predSet (predSet (predSet z))))
              hi (const_isScottContinuous botElem)))))

end DomainTheory.DataTypesAsLattices
```

tensor4_diag_isScottContinuous.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.seq4
public import DomainTheory.DataTypesAsLattices.Retracts.tensor4
public import Mathlib.Order.CompleteLattice.Defs
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Functionality.
    seq4_comp_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem tensor4_diag_isScottContinuous :
    IsScottContinuous (fun z => tensor4 z z z z) :=
  graph_const_isScottContinuous
    (fun z u =>
      seq4 (funOf z (funOf u (ofNat 0)))
        (funOf z (funOf u (ofNat 1)))
        (funOf z (funOf u (ofNat 2)))
        (funOf z (funOf u (ofNat 3))))
    (fun u =>
      seq4_comp_isScottContinuous
        (funOf.isScottContinuous_left (funOf u (ofNat 0)))
```

```
        (funOf.isScottContinuous_left (funOf u (ofNat 1)))
        (funOf.isScottContinuous_left (funOf u (ofNat 2)))
        (funOf.isScottContinuous_left (funOf u (ofNat 3))))

end DomainTheory.DataTypesAsLattices
```

expF_isScottContinuous.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.intR
public import DomainTheory.DataTypesAsLattices.Retracts.expF
public import Mathlib.Order.CompleteLattice.Defs
public import DomainTheory.DataTypesAsLattices.Lambda.id_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.const_isScottContinuous
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.
    tensorR_diag_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Functionality.
    tensor4_diag_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Functionality.
    tensorR_right_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Functionality.
    plus7_comp_isScottContinuous

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem expF_isScottContinuous : IsScottContinuous expF :=
  plus7_comp_isScottContinuous
    (const_isScottContinuous intR)
    (const_isScottContinuous botElem)
    id_isScottContinuous
    id_isScottContinuous
    tensor4_diag_isScottContinuous
    tensorR_diag_isScottContinuous
    (tensorR_right_isScottContinuous intR)

end DomainTheory.DataTypesAsLattices
```

tensor4_map_isScottContinuous.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.seq4
public import Mathlib.Order.CompleteLattice.Defs
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
```

```
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.
    seq4_comp_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem tensor4_map_isScottContinuous (a b c d : Pomega) :
    IsScottContinuous (fun u =>
      seq4 (funOf a (funOf u (ofNat 0)))
        (funOf b (funOf u (ofNat 1)))
        (funOf c (funOf u (ofNat 2)))
        (funOf d (funOf u (ofNat 3)))) :=
  seq4_comp_isScottContinuous
    (theorem_1_3 (funOf.isScottContinuous a) (funOf.isScottContinuous_left (ofNat 0)))
    (theorem_1_3 (funOf.isScottContinuous b) (funOf.isScottContinuous_left (ofNat 1)))
    (theorem_1_3 (funOf.isScottContinuous c) (funOf.isScottContinuous_left (ofNat 2)))
    (theorem_1_3 (funOf.isScottContinuous d) (funOf.isScottContinuous_left (ofNat 3)))

end DomainTheory.DataTypesAsLattices
```

tensor4_app.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.seq4
public import DomainTheory.DataTypesAsLattices.Retracts.tensor4
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.
    tensor4_map_isScottContinuous

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem tensor4_app (a b c d u : Pomega) :
    funOf (tensor4 a b c d) u =
      seq4 (funOf a (funOf u (ofNat 0)))
        (funOf b (funOf u (ofNat 1)))
        (funOf c (funOf u (ofNat 2)))
        (funOf d (funOf u (ofNat 3))) :=
  beta (tensor4_map_isScottContinuous a b c d) u
```

```
end DomainTheory.DataTypesAsLattices
```

tensor4_isRetract.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.Retract
public import DomainTheory.DataTypesAsLattices.Retracts.tensor4
public import DomainTheory.DataTypesAsLattices.Retracts.retract_app
public import DomainTheory.DataTypesAsLattices.Retracts.graph_ext
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.tensor4_app
public import DomainTheory.DataTypesAsLattices.Functionality.seq4_app

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem tensor4_isRetract {a b c d : Pomega}
    (ha : Retract a) (hb : Retract b) (hc : Retract c) (hd : Retract d) :
    Retract (tensor4 a b c d) := by
  constructor
  change tensor4 a b c d =
    graph (fun u => funOf (tensor4 a b c d) (funOf (tensor4 a b c d) u))
  apply graph_ext
  intro u
  rw [tensor4_app, tensor4_app, seq4_app.zero, seq4_app.one, seq4_app.two,
    seq4_app.three, retract_app ha, retract_app hb, retract_app hc, retract_app hd]

end DomainTheory.DataTypesAsLattices
```

mu_law.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.IsGraph
public import DomainTheory.DataTypesAsLattices.Graph.lam
public import DomainTheory.DataTypesAsLattices.Graph.Basic.notation3
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (?).** Application is monotone in both arguments. -/
theorem mu_law {u v x y : Pomega} (hu : u ⊆ v) (hx : x ⊆ y) :
    funOf u x ⊆ funOf v y :=
  subset_trans (funOf.monotone_left hu x) (funOf.monotone_right v hx)
```

```
end DomainTheory.DataTypesAsLattices
```

funOf_fix_idem.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.FixedPoint.iterateBot
public import DomainTheory.DataTypesAsLattices.FixedPoint.fix
public import DomainTheory.DataTypesAsLattices.FixedPoint.chain_mono
public import DomainTheory.DataTypesAsLattices.FixedPoint.chain_lemma
public import DomainTheory.DataTypesAsLattices.FixedPoint.iterateBot_mono
public import DomainTheory.DataTypesAsLattices.Graph.mu_law
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.Retract
public import DomainTheory.DataTypesAsLattices.Retracts.retract_app
public import DomainTheory.DataTypesAsLattices.Retracts.iterates_areRetracts
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem funOf_fix_idem {F : Pomega → Pomega} (hf : IsScottContinuous F)
    (hret : ∀ a, Retract a → Retract (F a)) (x : Pomega) :
    funOf (fix F) (funOf (fix F) x) = funOf (fix F) x := by
  have hchain : ∀ n m, n ≤ m → iterateBot F n ⊆ iterateBot F m :=
    chain_mono (iterateBot F) (iterateBot_mono hf)
  have hidem : ∀ n, funOf (iterateBot F n) (funOf (iterateBot F n) x) =
      funOf (iterateBot F n) x :=
    fun n => retract_app (iterates_areRetracts hret n) x
  have hcl : ∀ n,
      funOf (iterateBot F n) (⋃ m, funOf (iterateBot F m) x) =
        ⋃ m, funOf (iterateBot F n) (funOf (iterateBot F m) x) :=
    fun n => chain_lemma (funOf.isScottContinuous (iterateBot F n))
      (fun m => funOf (iterateBot F m) x)
      (fun m => funOf.monotone_left (iterateBot_mono hf m) x)
  apply subset_antisymm
  · intro k hk
    have hk' : k ∈ funOf (⋃ n, iterateBot F n)
        (⋃ m, funOf (iterateBot F m) x) := by
      simpa [fix, funOf.iUnion] using hk
    obtain ⟨n, hn, hp⟩ := hk'
    obtain ⟨i, hi⟩ := Set.mem_iUnion.mp hp
    have : k ∈ funOf (iterateBot F i) (⋃ m, funOf (iterateBot F m) x) :=
      ⟨n, hn, hi⟩
    have : k ∈ ⋃ m, funOf (iterateBot F i) (funOf (iterateBot F m) x) := by
      rwa [hcl i] at this
    obtain ⟨j, hj⟩ := Set.mem_iUnion.mp this
```

```
    let N := max i j
    have hij : funOf (iterateBot F i) (funOf (iterateBot F j) x) ⊆
        funOf (iterateBot F N) (funOf (iterateBot F N) x) :=
      mu_law (hchain i N (le_max_left i j))
        (funOf.monotone_left (hchain j N (le_max_right i j)) x)
    have : k ∈ funOf (iterateBot F N) x := by
      have : k ∈ funOf (iterateBot F N) (funOf (iterateBot F N) x) := hij hj
      rwa [hidem N] at this
    simpa [fix, funOf.iUnion] using Set.mem_iUnion.mpr ⟨N, this⟩
  · intro k hk
    have : k ∈ ⋃ n, funOf (iterateBot F n) x := by
      simpa [fix, funOf.iUnion] using hk
    obtain ⟨n, hkn⟩ := Set.mem_iUnion.mp this
    have : k ∈ funOf (iterateBot F n) (funOf (iterateBot F n) x) := by
      rw [hidem n]
      exact hkn
    have : k ∈ funOf (iterateBot F n) (⋃ m, funOf (iterateBot F m) x) :=
      funOf.monotone_right _ (Set.subset_iUnion (fun m => funOf (iterateBot F m) x) n)
     this
    have : k ∈ funOf (⋃ i, iterateBot F i) (⋃ m, funOf (iterateBot F m) x) :=
      funOf.monotone_left (Set.subset_iUnion (iterateBot F) n) _ this
    simpa [fix, funOf.iUnion] using this

end DomainTheory.DataTypesAsLattices
```

theorem_4_6.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import DomainTheory.DataTypesAsLattices.Lambda.Ycomb
public import DomainTheory.DataTypesAsLattices.Lambda.theorem_2_5
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.FixedPoint.iterateBot
public import DomainTheory.DataTypesAsLattices.FixedPoint.fix
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.IsGraph
public import DomainTheory.DataTypesAsLattices.Graph.theorem_1_2
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.Retract
public import DomainTheory.DataTypesAsLattices.Retracts.Fixpoints
public import DomainTheory.DataTypesAsLattices.Retracts.retractLe
public import DomainTheory.DataTypesAsLattices.Retracts.IsStrict
public import DomainTheory.DataTypesAsLattices.Retracts.RetractInverseLimit
public import DomainTheory.DataTypesAsLattices.Retracts.fixedToInverseLimit
public import DomainTheory.DataTypesAsLattices.Retracts.inverseLimitToFixed
public import DomainTheory.DataTypesAsLattices.Retracts.iterates_areRetracts
public import DomainTheory.DataTypesAsLattices.Retracts.inverseLimit_project_iUnion
public import DomainTheory.DataTypesAsLattices.Retracts.graph_ext
public import DomainTheory.DataTypesAsLattices.Retracts.iUnion_retract_isGraph
```

```
public import DomainTheory.DataTypesAsLattices.Retracts.funOf_fix_idem
public import DomainTheory.DataTypesAsLattices.Retracts.vals
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
public import DomainTheory.DataTypesAsLattices.Retracts.BasicInst55

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, Theorem 4.6 (The limit theorem).**
Kleene iterates of a retract-preserving continuous `F` are retracts, and
`Y(F)` is a retract by the chain-union calculation. -/
theorem theorem_4_6 {F : Pomega → Pomega}
    (hf : IsScottContinuous F)
    (hret : ∀ a, Retract a → Retract (F a)) :
    (∀ n, Retract (iterateBot F n)) ∧
      Retract (funOf Ycomb (graph F)) := by
  refine ⟨iterates_areRetracts hret, ?_⟩
  have hY : funOf Ycomb (graph F) = fix F := theorem_2_5 hf
  rw [hY]
  constructor
  change fix F = graph (fun x => funOf (fix F) (funOf (fix F) x))
  have hG : IsGraph (fix F) :=
    iUnion_retract_isGraph (iterates_areRetracts hret)
  have hgr : graph (funOf (fix F)) = fix F := (theorem_1_2.iii (fix F)).mpr hG
  have hidem' : graph (fun x => funOf (fix F) (funOf (fix F) x)) =
      graph (funOf (fix F)) :=
    graph_ext (fun x => funOf_fix_idem hf hret x)
  exact (hidem'.trans hgr).symm

namespace theorem_4_6

/-- **Scott 1976, Theorem 4.6, inverse-limit half.** The maps
`u ↦ (F?(⊥)(u))ₙ` and `(vₙ) ↦ ⋃ₙ vₙ` are inverse order isomorphisms. -/
def inverseLimit {F : Pomega → Pomega}
    (hf : IsScottContinuous F)
    (hret : ∀ a, Retract a → Retract (F a))
    (hstrict : ∀ a, Retract a → IsStrict a → IsStrict (F a))
    (hmono : ∀ {a b}, Retract a → IsStrict a →
      Retract b → IsStrict b → retractLe a b → retractLe (F a) (F b)) :
    Fixpoints (funOf (fix F)) ≃o RetractInverseLimit F where
  toFun := fixedToInverseLimit hf hret hstrict hmono
  invFun := inverseLimitToFixed hf hret hstrict hmono
  left_inv u := by
    apply Subtype.ext
    change (⋃ n, funOf (iterateBot F n) u.1) = u.1
    rw [← funOf.iUnion, ← fix, u.property]
  right_inv v := by
    apply Subtype.ext
    funext n
    exact inverseLimit_project_iUnion hf hret hstrict hmono v n
  map_rel_iff' := by
    intro u w
    constructor
    · intro h
      change ∀ n, funOf (iterateBot F n) u.1 ⊆
```

```
        funOf (iterateBot F n) w.1 at h
      change u.1 ⊆ w.1
      have hu : u.1 = ⋃ n, funOf (iterateBot F n) u.1 := by
        calc
          u.1 = funOf (fix F) u.1 := u.property.symm
          _ = ⋃ n, funOf (iterateBot F n) u.1 := by
            change funOf (⋃ n, iterateBot F n) u.1 = _
            rw [funOf.iUnion]
      have hw : w.1 = ⋃ n, funOf (iterateBot F n) w.1 := by
        calc
          w.1 = funOf (fix F) w.1 := w.property.symm
          _ = ⋃ n, funOf (iterateBot F n) w.1 := by
            change funOf (⋃ n, iterateBot F n) w.1 = _
            rw [funOf.iUnion]
      rw [hu, hw]
      intro k hk
      obtain ⟨n, hkn⟩ := Set.mem_iUnion.mp hk
      exact Set.mem_iUnion.mpr ⟨n, h n hkn⟩
    · intro huw
      change u.1 ⊆ w.1 at huw
      change ∀ n, funOf (iterateBot F n) u.1 ⊆
        funOf (iterateBot F n) w.1
      intro n
      exact funOf.monotone_right (iterateBot F n) huw

/-- **Scott 1976, Theorem 4.6**, inverse-limit half packaged with the
retract half. Under the paper's extra strictness and `⊑`-monotonicity
hypotheses, the range of `Y(F)` is order-isomorphic to the inverse
limit of the ranges of `F?(⊥)`. Stated without `≃o` so the compared
type does not mention a `CompleteLattice` instance. -/
theorem limit {F : Pomega → Pomega}
    (hf : IsScottContinuous F)
    (hret : ∀ a, Retract a → Retract (F a))
    (hstrict : ∀ a, Retract a → IsStrict a → IsStrict (F a))
    (hmono : ∀ {a b}, Retract a → IsStrict a →
      Retract b → IsStrict b → retractLe a b → retractLe (F a) (F b)) :
    (∀ n, Retract (iterateBot F n)) ∧
      Retract (funOf Ycomb (graph F)) ∧
        ∃ φ : Fixpoints (funOf (fix F)) → RetractInverseLimit F,
          Function.Bijective φ ∧
            ∀ x y : Fixpoints (funOf (fix F)),
              (x : Pomega) ⊆ (y : Pomega) ↔
                ∀ n, (φ x).1 n ⊆ (φ y).1 n := by
  refine ⟨(theorem_4_6 hf hret).1, (theorem_4_6 hf hret).2, ?_⟩
  let iso := theorem_4_6.inverseLimit hf hret hstrict hmono
  refine ⟨iso, iso.bijective, fun x y => ⟨?_, ?_⟩⟩
  · intro hxy n
    exact funOf.monotone_right (iterateBot F n) hxy
  · intro h
    have hx : (x : Pomega) = ⋃ n, funOf (iterateBot F n) (x : Pomega) := by
      calc
        (x : Pomega) = funOf (fix F) (x : Pomega) := x.property.symm
        _ = ⋃ n, funOf (iterateBot F n) (x : Pomega) := by
          change funOf (⋃ n, iterateBot F n) (x : Pomega) = _
          rw [funOf.iUnion]
    have hy : (y : Pomega) = ⋃ n, funOf (iterateBot F n) (y : Pomega) := by
```

```
      calc
        (y : Pomega) = funOf (fix F) (y : Pomega) := y.property.symm
        _ = ⋃ n, funOf (iterateBot F n) (y : Pomega) := by
          change funOf (⋃ n, iterateBot F n) (y : Pomega) = _
          rw [funOf.iUnion]
    change ∀ n, funOf (iterateBot F n) (x : Pomega) ⊆
      funOf (iterateBot F n) (y : Pomega) at h
    rw [hx, hy]
    intro k hk
    obtain ⟨n, hn⟩ := Set.mem_iUnion.mp hk
    exact Set.mem_iUnion.mpr ⟨n, h n hn⟩

end theorem_4_6

end DomainTheory.DataTypesAsLattices
```

expR_isRetract.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.Retract
public import DomainTheory.DataTypesAsLattices.Retracts.intR
public import DomainTheory.DataTypesAsLattices.Retracts.expR
public import DomainTheory.DataTypesAsLattices.Retracts.bot_isRetract
public import DomainTheory.DataTypesAsLattices.Retracts.theorem_4_6
public import DomainTheory.DataTypesAsLattices.Functionality.tensorR_isRetract
public import DomainTheory.DataTypesAsLattices.Functionality.tensor4_isRetract
public import DomainTheory.DataTypesAsLattices.Functionality.plus7_isRetract
public import DomainTheory.DataTypesAsLattices.Functionality.expF_isScottContinuous

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, Theorem 4.6 applied to (4.44).** -/
theorem expR_isRetract : Retract expR :=
  (theorem_4_6 expF_isScottContinuous (fun _a ha =>
    plus7_isRetract intR.isRetract bot_isRetract ha ha
      (tensor4_isRetract ha ha ha ha) (tensorR_isRetract ha ha)
      (tensorR_isRetract intR.isRetract ha))).2

end DomainTheory.DataTypesAsLattices
```

theorem_4_3_more.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.termDollarStep
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.comp
public import DomainTheory.DataTypesAsLattices.Retracts.Retract
public import DomainTheory.DataTypesAsLattices.Retracts.Fixpoints
public import DomainTheory.DataTypesAsLattices.Retracts.DirectedSubset
```

```
public import DomainTheory.DataTypesAsLattices.Retracts.IsCompactSubset
public import DomainTheory.DataTypesAsLattices.Retracts.typed
public import DomainTheory.DataTypesAsLattices.Retracts.funRetract
public import DomainTheory.DataTypesAsLattices.Retracts.arrowR
public import DomainTheory.DataTypesAsLattices.Retracts.fstC
public import DomainTheory.DataTypesAsLattices.Retracts.sndC
public import DomainTheory.DataTypesAsLattices.Retracts.pairSeq
public import DomainTheory.DataTypesAsLattices.Retracts.tensorR
public import DomainTheory.DataTypesAsLattices.Retracts.plusR
public import DomainTheory.DataTypesAsLattices.Retracts.retractLe
public import DomainTheory.DataTypesAsLattices.Retracts.iterateFrom
public import DomainTheory.DataTypesAsLattices.Retracts.lfpAbove
public import DomainTheory.DataTypesAsLattices.Retracts.gfpBelow
public import DomainTheory.DataTypesAsLattices.Retracts.Icomb
public import DomainTheory.DataTypesAsLattices.Retracts.vals
public import DomainTheory.DataTypesAsLattices.Retracts.fixpointsCompleteLattice
public import DomainTheory.DataTypesAsLattices.Retracts.IsStrict
public import DomainTheory.DataTypesAsLattices.Retracts.diagC
public import DomainTheory.DataTypesAsLattices.Retracts.evalC
public import DomainTheory.DataTypesAsLattices.Retracts.curryC
public import DomainTheory.DataTypesAsLattices.Retracts.intF
public import DomainTheory.DataTypesAsLattices.Retracts.treeF
public import DomainTheory.DataTypesAsLattices.Retracts.RetractInverseLimit
public import DomainTheory.DataTypesAsLattices.Retracts.fixedToInverseLimit
public import DomainTheory.DataTypesAsLattices.Retracts.inverseLimitToFixed
public import DomainTheory.DataTypesAsLattices.Retracts.theorem_4_6
public import DomainTheory.DataTypesAsLattices.Retracts.treeR
public import DomainTheory.DataTypesAsLattices.Retracts.boolR
public import DomainTheory.DataTypesAsLattices.Retracts.openR
public import DomainTheory.DataTypesAsLattices.Retracts.lambF
public import DomainTheory.DataTypesAsLattices.Retracts.lambR
public import DomainTheory.DataTypesAsLattices.Retracts.envR
public import DomainTheory.DataTypesAsLattices.Retracts.updateEnv
public import DomainTheory.DataTypesAsLattices.Retracts.tagInj
public import DomainTheory.DataTypesAsLattices.Retracts.seq4
public import DomainTheory.DataTypesAsLattices.Retracts.tensor4
public import DomainTheory.DataTypesAsLattices.Retracts.plus7Body
public import DomainTheory.DataTypesAsLattices.Retracts.plus7
public import DomainTheory.DataTypesAsLattices.Retracts.expF
public import DomainTheory.DataTypesAsLattices.Retracts.expR
public import DomainTheory.DataTypesAsLattices.Retracts.sUnion_Fixpoints_le_image
public import DomainTheory.DataTypesAsLattices.Retracts.theorem_4_1
public import DomainTheory.DataTypesAsLattices.Retracts.BasicInst36
public import DomainTheory.DataTypesAsLattices.Retracts.retract_app
public import DomainTheory.DataTypesAsLattices.Retracts.funOf_botElem
public import DomainTheory.DataTypesAsLattices.Retracts.graph_const_bot
public import DomainTheory.DataTypesAsLattices.Retracts.bot_isRetract
public import DomainTheory.DataTypesAsLattices.Retracts.iterates_areRetracts
public import DomainTheory.DataTypesAsLattices.Retracts.bot_retractLe
public import DomainTheory.DataTypesAsLattices.Retracts.iterateBot_isStrict
public import DomainTheory.DataTypesAsLattices.Retracts.iterateBot_retractLe
public import DomainTheory.DataTypesAsLattices.Retracts.BasicInst55
public import DomainTheory.DataTypesAsLattices.Retracts.inverseLimit_mono
public import DomainTheory.DataTypesAsLattices.Retracts.inverseLimit_chain
public import DomainTheory.DataTypesAsLattices.Retracts.inverseLimit_project_later
public import DomainTheory.DataTypesAsLattices.Retracts.inverseLimit_project_earlier
```

```
public import DomainTheory.DataTypesAsLattices.Retracts.inverseLimit_project_iUnion
public import DomainTheory.DataTypesAsLattices.Retracts.inverseLimit_iUnion_typed
public import DomainTheory.DataTypesAsLattices.Retracts.graph_ext

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, Theorem 4.3 (i).** On elements, `a o→ b` acts as `b ∘ ? ∘ a`,
and that action is idempotent when `a, b` are retracts. -/
theorem theorem_4_3.i {a b u : Pomega} (ha : Retract a) (hb : Retract b) :
    funOf (arrowR a b) (funOf (arrowR a b) u) = funOf (arrowR a b) u := by
  rw [arrowR.app, arrowR.app]
  apply graph_ext
  intro x
  simp [comp.app, retract_app ha, retract_app hb]

/-- **Scott 1976, Theorem 4.3 (ii).** `u : a o→ b` means `u` is restricted
to `a` and takes values in `b`. -/
theorem theorem_4_3.ii {a b u x : Pomega}
    (hu : typed u (arrowR a b)) (hx : typed x a) :
    typed (funOf u x) b := by
  have hu' : u = comp b (comp u a) := by
    simpa [typed, arrowR.app] using hu
  have : funOf u x = funOf b (funOf u (funOf a x)) := by
    have h1 : funOf u x = funOf (comp b (comp u a)) x :=
      congrArg (fun w => funOf w x) hu'
    rw [h1, comp.app, comp.app]
  rwa [show funOf a x = x from hx.symm] at this

end DomainTheory.DataTypesAsLattices
```

typed_more.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.typed
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.arrowR
public import DomainTheory.DataTypesAsLattices.Retracts.comp
public import DomainTheory.DataTypesAsLattices.Retracts.Retract
public import DomainTheory.DataTypesAsLattices.Retracts.theorem_4_3_more
public import DomainTheory.DataTypesAsLattices.Retracts.retract_app
public import DomainTheory.DataTypesAsLattices.Retracts.graph_ext
public import DomainTheory.DataTypesAsLattices.Graph.graph

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace typed

theorem arrowR_range {a b u : Pomega}
    (hb : Retract b) (hu : typed u (arrowR a b)) :
```

```
    u = comp b u := by
  have hu' := arrowR.typed_of hu
  rw [hu']
  apply graph_ext
  intro x
  simp only [comp.app]
  rw [retract_app hb]

theorem arrowR_restrict {a b u : Pomega}
    (ha : Retract a) (hu : typed u (arrowR a b)) :
    u = comp u a := by
  have hu' := arrowR.typed_of hu
  have h : comp (comp b (comp u a)) a = comp b (comp u a) := by
    apply graph_ext
    intro x
    simp only [comp.app]
    rw [retract_app ha]
  have : comp u a = comp (comp b (comp u a)) a := by
    conv_lhs => rw [hu']
  exact ((this.trans h).trans hu'.symm).symm

theorem arrowR_iff {a b u : Pomega} (ha : Retract a) (_hb : Retract b) :
    typed u (arrowR a b) ↔
      u = comp u a ∧ ∀ x, typed x a → typed (funOf u x) b := by
  constructor
  · intro hu
    exact ⟨typed.arrowR_restrict ha hu, fun x hx => theorem_4_3.ii hu hx⟩
  · intro ⟨hrest, hval⟩
    change u = funOf (arrowR a b) u
    rw [arrowR.app]
    apply Eq.trans hrest
    apply graph_ext
    intro x
    have hx : typed (funOf a x) a := (retract_app ha x).symm
    have hbx : funOf u (funOf a x) = funOf b (funOf u (funOf a x)) :=
      hval (funOf a x) hx
    simp only [comp.app]
    exact hbx

theorem comp {a b c f f' : Pomega}
    (ha : Retract a) (hb : Retract b) (_hc : Retract c)
    (hf : typed f (arrowR a b)) (hf' : typed f' (arrowR b c)) :
    typed (DomainTheory.DataTypesAsLattices.comp f' f) (arrowR a c) := by
  have hf1 := arrowR.typed_of hf
  have hf2 := arrowR.typed_of hf'
  change DomainTheory.DataTypesAsLattices.comp f' f =
      funOf (arrowR a c) (DomainTheory.DataTypesAsLattices.comp f' f)
  rw [arrowR.app]
  apply graph_ext
  intro x
  have hfx : funOf f x = funOf b (funOf f (funOf a x)) := by
    simpa [DomainTheory.DataTypesAsLattices.comp.app] using
      congrArg (fun w => funOf w x) hf1
  have hrest := typed.arrowR_restrict ha hf
  have hfa : funOf f (funOf a x) = funOf f x := by
    simpa [DomainTheory.DataTypesAsLattices.comp.app] using
```

```
      (congrArg (fun w => funOf w x) hrest).symm
  have hf'fx : funOf f' (funOf f x) =
      funOf c (funOf f' (funOf b (funOf f x))) := by
    simpa [DomainTheory.DataTypesAsLattices.comp.app] using
      congrArg (fun w => funOf w (funOf f x)) hf2
  have hrange := typed.arrowR_range hb hf
  have hbval : funOf b (funOf f (funOf a x)) = funOf f (funOf a x) := by
    simpa [DomainTheory.DataTypesAsLattices.comp.app] using
      (congrArg (fun w => funOf w (funOf a x)) hrange).symm
  simp only [DomainTheory.DataTypesAsLattices.comp.app]
  rw [hf'fx, ← hfa, hbval, hfa]

end typed

end DomainTheory.DataTypesAsLattices
```

arrowR_more.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.arrowR
public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.comp
public import DomainTheory.DataTypesAsLattices.Retracts.IsStrict
public import DomainTheory.DataTypesAsLattices.Retracts.funOf_botElem
public import DomainTheory.DataTypesAsLattices.Retracts.graph_const_bot
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Retracts.retractLe
public import DomainTheory.DataTypesAsLattices.Retracts.typed
public import DomainTheory.DataTypesAsLattices.Retracts.eq_4_11
public import DomainTheory.DataTypesAsLattices.Retracts.Retract
public import DomainTheory.DataTypesAsLattices.Retracts.graph_ext
public import DomainTheory.DataTypesAsLattices.Retracts.theorem_4_3_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace arrowR

theorem isRetract {a b : Pomega} (ha : Retract a) (hb : Retract b) :
    Retract (arrowR a b) := by
  constructor
  change arrowR a b =
    graph (fun x => funOf (arrowR a b) (funOf (arrowR a b) x))
```

```
  apply graph_ext
  intro u
  rw [theorem_4_3.i ha hb, app]

theorem mono {a b a' b' : Pomega}
    (hab : _root_.DomainTheory.DataTypesAsLattices.retractLe a a')
    (hbb : _root_.DomainTheory.DataTypesAsLattices.retractLe b b') :
    _root_.DomainTheory.DataTypesAsLattices.retractLe (arrowR a b) (arrowR a' b') :=
    by
  constructor
  · have h : arrowR (comp a' a) (comp b b') = arrowR a b := by
      rw [← hab.2, ← hbb.1]
    exact (h.symm.trans (eq_4_11.comp a b a' b').symm)
  · have h : arrowR (comp a a') (comp b' b) = arrowR a b := by
      rw [← hab.1, ← hbb.2]
    exact (h.symm.trans (eq_4_11.comp a' b' a b).symm)

theorem functor {a b a' b' f f' : Pomega}
    (hf : typed f (arrowR a b)) (hf' : typed f' (arrowR a' b')) :
    typed (arrowR f f') (arrowR (arrowR b a') (arrowR a b')) := by
  change arrowR f f' =
    funOf (arrowR (arrowR b a') (arrowR a b')) (arrowR f f')
  rw [app]
  have inner := eq_4_11.comp f f' b a'
  have outer := eq_4_11.comp a b' (comp b f) (comp f' a')
  have hf1 := typed_of hf
  have hf2 := typed_of hf'
  apply Eq.symm
  calc
    comp (arrowR a b') (comp (arrowR f f') (arrowR b a'))
        = comp (arrowR a b') (arrowR (comp b f) (comp f' a')) := by rw [inner]
    _ = arrowR (comp (comp b f) a) (comp b' (comp f' a')) := outer
    _ = arrowR f f' := by
      apply graph_ext
      intro u
      apply graph_ext
      intro x
      have hfx : funOf f x = funOf b (funOf f (funOf a x)) := by
        simpa [comp.app] using congrArg (fun w => funOf w x) hf1
      have hf'y : ∀ y, funOf f' y = funOf b' (funOf f' (funOf a' y)) := by
        intro y
        simpa [comp.app] using congrArg (fun w => funOf w y) hf2
      simp only [comp.app]
      rw [hfx]
      exact (hf'y _).symm

end arrowR

end DomainTheory.DataTypesAsLattices
```

lambR_isRetract.lean

```
module
```

```
public import DomainTheory.DataTypesAsLattices.Retracts.Retract
public import DomainTheory.DataTypesAsLattices.Retracts.arrowR
public import DomainTheory.DataTypesAsLattices.Retracts.intR
public import DomainTheory.DataTypesAsLattices.Retracts.lambR
public import DomainTheory.DataTypesAsLattices.Retracts.theorem_4_6
public import DomainTheory.DataTypesAsLattices.Functionality.plusR_isRetract
public import DomainTheory.DataTypesAsLattices.Functionality.lambF_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Retracts.arrowR_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, Theorem 4.6 applied to (4.39).** -/
theorem lambR_isRetract : Retract lambR :=
  (theorem_4_6 lambF_isScottContinuous (fun _a ha =>
    plusR_isRetract intR.isRetract (arrowR.isRetract ha ha))).2

end DomainTheory.DataTypesAsLattices
```

envR_isRetract.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.Retract
public import DomainTheory.DataTypesAsLattices.Retracts.lambR
public import DomainTheory.DataTypesAsLattices.Retracts.envR
public import DomainTheory.DataTypesAsLattices.Retracts.retract_app
public import DomainTheory.DataTypesAsLattices.Retracts.graph_ext
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Functionality.lambR_isRetract
public import DomainTheory.DataTypesAsLattices.Functionality.envR_app
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem envR_isRetract : Retract envR := by
  constructor
  change envR = graph (fun t => funOf envR (funOf envR t))
  apply graph_ext
  intro t
  rw [envR_app, envR_app]
  apply graph_ext
  intro n
  have hβ : funOf (graph (fun m => funOf lambR (funOf t m))) n =
      funOf lambR (funOf t n) :=
    beta (theorem_1_3 (funOf.isScottContinuous lambR)
      (funOf.isScottContinuous t)) n
  rw [hβ, retract_app lambR_isRetract]
```

```
end DomainTheory.DataTypesAsLattices
```

Hcomb.lean

```
module

public import DomainTheory.DataTypesAsLattices.Functionality.Hfun
public import DomainTheory.DataTypesAsLattices.Functionality.envR_isRetract
public import DomainTheory.DataTypesAsLattices.Functionality.expR_isRetract
public import DomainTheory.DataTypesAsLattices.Functionality.lambR_isRetract
public import DomainTheory.DataTypesAsLattices.Functionality.typed_apply_retract
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.arrowR
public import DomainTheory.DataTypesAsLattices.Retracts.arrowR_more
public import DomainTheory.DataTypesAsLattices.Retracts.comp
public import DomainTheory.DataTypesAsLattices.Retracts.envR
public import DomainTheory.DataTypesAsLattices.Retracts.expR
public import DomainTheory.DataTypesAsLattices.Retracts.lambR
public import DomainTheory.DataTypesAsLattices.Retracts.typed

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def Hcomb : Pomega :=
  funOf (arrowR expR (arrowR envR lambR)) Hfun

namespace Hcomb

theorem app (e : Pomega) :
    funOf Hcomb e =
      funOf (arrowR envR lambR) (funOf Hfun (funOf expR e)) := by
  rw [Hcomb, arrowR.app]
  simp only [comp.app]

theorem typed :
    typed Hcomb (arrowR expR (arrowR envR lambR)) :=
  typed_apply_retract
    (arrowR.isRetract expR_isRetract
      (arrowR.isRetract envR_isRetract lambR_isRetract))

end Hcomb

end DomainTheory.DataTypesAsLattices
```

theorem_5_5_more.lean

```
module

public import Mathlib.Algebra.Ring.Parity
public import DomainTheory.DataTypesAsLattices.Retracts.graph_ext
public import DomainTheory.DataTypesAsLattices.Retracts.retract_isGraph
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Graph.graph
```

```
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Closures.Closure
public import DomainTheory.DataTypesAsLattices.Closures.Vapply
public import DomainTheory.DataTypesAsLattices.Closures.Vapply_more
public import DomainTheory.DataTypesAsLattices.Closures.Vcomb
public import DomainTheory.DataTypesAsLattices.Closures.eq_5_16
public import DomainTheory.DataTypesAsLattices.Closures.eq_5_17
public import DomainTheory.DataTypesAsLattices.Closures.eq_5_18
public import DomainTheory.DataTypesAsLattices.Closures.theorem_5_5

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, Theorem 5.5 (partial).** `V(a)` is always a closure of `x`,
and `V` fixes exactly the closure operations in the sense of (5.18) on values. -/
theorem theorem_5_5.values (a x : Pomega) :
    x ⊆ Vapply a x ∧ Vapply a (Vapply a x) = Vapply a x :=
  ⟨eq_5_16 a x, eq_5_17 a x⟩

/-- **Scott 1976, (5.18).** `a = V(a)` iff `a` is a closure. -/
theorem theorem_5_5.iff (a : Pomega) :
    Closure a ↔ funOf Vcomb a = a := by
  constructor
  · intro ha
    rw [Vcomb.app]
    have : graph (fun x => Vapply a x) = graph (funOf a) :=
      graph_ext fun x => eq_5_18.eq ha x
    exact this.trans (retract_isGraph ha.out.2)
  · intro h
    have ha : a = graph (fun x => Vapply a x) :=
      h.symm.trans (Vcomb.app a)
    have happ : ∀ x, funOf a x = Vapply a x := by
      intro x
      conv_lhs => rw [ha]
      exact beta (Vapply.right_isScottContinuous a) x
    constructor
    constructor
    · intro p hp
      rcases hp with ⟨n, m, rfl, hm⟩
      have : pair n m ∈ graph (fun x => Vapply a x) :=
        ⟨n, m, rfl, eq_5_16 a (e n) hm⟩
      rwa [← ha] at this
    · have : graph (fun x => Vapply a x) =
          graph (fun x => funOf a (funOf a x)) :=
        graph_ext fun x =>
          calc Vapply a x
              = Vapply a (Vapply a x) := (eq_5_17 a x).symm
            _ = funOf a (Vapply a x) := (happ (Vapply a x)).symm
            _ = funOf a (funOf a x) := by rw [← happ x]
      exact ⟨ha.trans this⟩

end DomainTheory.DataTypesAsLattices
```

eq_5_19.lean

```
module

public import Mathlib.Algebra.Ring.Parity
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Closures.Vapply
public import DomainTheory.DataTypesAsLattices.Closures.Vcomb
public import DomainTheory.DataTypesAsLattices.Closures.theorem_5_5_more
public import DomainTheory.DataTypesAsLattices.Closures.Vapply_more
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem eq_5_19 (a : Pomega) :
    funOf Vcomb (funOf Vcomb a) = funOf Vcomb a := by
  have hcl := Vapply.graph_isClosure a
  have hfix := (theorem_5_5.iff (graph (fun x => Vapply a x))).mp hcl
  have ha : funOf Vcomb a = graph (fun x => Vapply a x) := Vcomb.app a
  rw [ha]
  exact hfix

end DomainTheory.DataTypesAsLattices
```

Vcomb_more.lean

```
module

public import DomainTheory.DataTypesAsLattices.Closures.Vcomb
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Closures.eq_5_19
public import DomainTheory.DataTypesAsLattices.Retracts.Retract
public import DomainTheory.DataTypesAsLattices.Retracts.graph_ext

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace Vcomb

theorem isRetract : Retract Vcomb := by
  constructor
  change Vcomb = graph (fun a => funOf Vcomb (funOf Vcomb a))
  apply graph_ext
  intro a
  exact (Vcomb.app a).symm.trans
    (_root_.DomainTheory.DataTypesAsLattices.eq_5_19 a).symm

end Vcomb

end DomainTheory.DataTypesAsLattices
```

eq_5_21.lean

```
module

public import Mathlib.Algebra.Ring.Parity
public import DomainTheory.DataTypesAsLattices.Lambda.xi_star
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Closures.Vcomb
public import DomainTheory.DataTypesAsLattices.Closures.eq_5_1
public import DomainTheory.DataTypesAsLattices.Closures.eq_5_18
public import DomainTheory.DataTypesAsLattices.Closures.Vcomb_more
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (5.21).** `a ⊆ V(a)`. -/
theorem eq_5_21 (a : Pomega) : a ⊆ funOf Vcomb a := by
  rw [Vcomb.app]
  exact (eq_5_1 a).trans (xi_star fun x => eq_5_18.rev a x)

end DomainTheory.DataTypesAsLattices
```

Icomb_subset_Vcomb.lean

```
module

public import Mathlib.Algebra.Ring.Parity
public import DomainTheory.DataTypesAsLattices.Retracts.Icomb
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Closures.Vapply
public import DomainTheory.DataTypesAsLattices.Closures.Vcomb
public import DomainTheory.DataTypesAsLattices.Closures.eq_5_21
public import DomainTheory.DataTypesAsLattices.Closures.Vcomb_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem Icomb_subset_Vcomb : Icomb ⊆ Vcomb := by
  intro p hp
  rcases hp with ⟨n, m, rfl, hm⟩
  have : m ∈ funOf Vcomb (e n) := eq_5_21 (e n) hm
  have ha : funOf Vcomb (e n) = graph (fun x => Vapply (e n) x) :=
    Vcomb.app (e n)
  refine ⟨n, m, rfl, ?_⟩
  rwa [ha] at this

end DomainTheory.DataTypesAsLattices
```

theorem_5_5_universe.lean

```
module

public import Mathlib.Algebra.Ring.Parity
public import DomainTheory.DataTypesAsLattices.Closures.Closure
public import DomainTheory.DataTypesAsLattices.Closures.Vcomb
public import DomainTheory.DataTypesAsLattices.Closures.Vcomb_more
public import DomainTheory.DataTypesAsLattices.Closures.Icomb_subset_Vcomb
public import DomainTheory.DataTypesAsLattices.Closures.theorem_5_5_more
public import DomainTheory.DataTypesAsLattices.Retracts.typed
public import DomainTheory.DataTypesAsLattices.Graph.funOf

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, Theorem 5.5.** `V` itself is a closure, and its fixed
points are exactly the closure operations. -/
theorem theorem_5_5.universe :
    Closure Vcomb ∧ ∀ a, Closure a ↔ typed a Vcomb :=
  ⟨⟨Icomb_subset_Vcomb, Vcomb.isRetract⟩⟩,
    fun a => (theorem_5_5.iff a).trans ⟨Eq.symm, Eq.symm⟩

end DomainTheory.DataTypesAsLattices
```

eq_5_22.lean

```
module

public import Mathlib.Algebra.Ring.Parity
public import DomainTheory.DataTypesAsLattices.Retracts.typed
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Closures.boxTensor
public import DomainTheory.DataTypesAsLattices.Closures.Vcomb
public import DomainTheory.DataTypesAsLattices.Closures.theorem_5_4
public import DomainTheory.DataTypesAsLattices.Closures.theorem_5_5
public import DomainTheory.DataTypesAsLattices.Closures.theorem_5_5_universe
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (5.22).** Boxed product of closures is a closure, so
the map is typed on `V`. -/
theorem eq_5_22 {a b : Pomega} (ha : typed a Vcomb) (hb : typed b Vcomb) :
    typed (boxTensor a b) Vcomb :=
  (theorem_5_5.universe.2 (boxTensor a b)).mp
    (theorem_5_4 ((theorem_5_5.universe.2 a).mpr ha)
      ((theorem_5_5.universe.2 b).mpr hb))

end DomainTheory.DataTypesAsLattices
```

eq_5_23.lean

```
module

public import Mathlib.Algebra.Ring.Parity
```

```
public import DomainTheory.DataTypesAsLattices.Retracts.typed
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Closures.boxPlus
public import DomainTheory.DataTypesAsLattices.Closures.Vcomb
public import DomainTheory.DataTypesAsLattices.Closures.theorem_5_4
public import DomainTheory.DataTypesAsLattices.Closures.theorem_5_5
public import DomainTheory.DataTypesAsLattices.Closures.theorem_5_5_universe
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (5.23).** Boxed sum of closures is a closure, so the map
is typed on `V`. -/
theorem eq_5_23 {a b : Pomega} (ha : typed a Vcomb) (hb : typed b Vcomb) :
    typed (boxPlus a b) Vcomb :=
  (theorem_5_5.universe.2 (boxPlus a b)).mp
    (theorem_5_4.plus ((theorem_5_5.universe.2 a).mpr ha)
      ((theorem_5_5.universe.2 b).mpr hb))

end DomainTheory.DataTypesAsLattices
```

theorem_4_3.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.comp
public import DomainTheory.DataTypesAsLattices.Retracts.Retract
public import DomainTheory.DataTypesAsLattices.Retracts.Fixpoints
public import DomainTheory.DataTypesAsLattices.Retracts.DirectedSubset
public import DomainTheory.DataTypesAsLattices.Retracts.IsCompactSubset
public import DomainTheory.DataTypesAsLattices.Retracts.typed
public import DomainTheory.DataTypesAsLattices.Retracts.funRetract
public import DomainTheory.DataTypesAsLattices.Retracts.arrowR
public import DomainTheory.DataTypesAsLattices.Retracts.fstC
public import DomainTheory.DataTypesAsLattices.Retracts.sndC
public import DomainTheory.DataTypesAsLattices.Retracts.pairSeq
public import DomainTheory.DataTypesAsLattices.Retracts.tensorR
public import DomainTheory.DataTypesAsLattices.Retracts.plusR
public import DomainTheory.DataTypesAsLattices.Retracts.retractLe
public import DomainTheory.DataTypesAsLattices.Retracts.iterateFrom
public import DomainTheory.DataTypesAsLattices.Retracts.lfpAbove
public import DomainTheory.DataTypesAsLattices.Retracts.gfpBelow
public import DomainTheory.DataTypesAsLattices.Retracts.Icomb
public import DomainTheory.DataTypesAsLattices.Retracts.vals
public import DomainTheory.DataTypesAsLattices.Retracts.fixpointsCompleteLattice
public import DomainTheory.DataTypesAsLattices.Retracts.IsStrict
public import DomainTheory.DataTypesAsLattices.Retracts.diagC
public import DomainTheory.DataTypesAsLattices.Retracts.evalC
public import DomainTheory.DataTypesAsLattices.Retracts.curryC
public import DomainTheory.DataTypesAsLattices.Retracts.intF
public import DomainTheory.DataTypesAsLattices.Retracts.treeF
public import DomainTheory.DataTypesAsLattices.Retracts.RetractInverseLimit
public import DomainTheory.DataTypesAsLattices.Retracts.fixedToInverseLimit
```

```
public import DomainTheory.DataTypesAsLattices.Retracts.inverseLimitToFixed
public import DomainTheory.DataTypesAsLattices.Retracts.treeR
public import DomainTheory.DataTypesAsLattices.Retracts.boolR
public import DomainTheory.DataTypesAsLattices.Retracts.openR
public import DomainTheory.DataTypesAsLattices.Retracts.lambF
public import DomainTheory.DataTypesAsLattices.Retracts.lambR
public import DomainTheory.DataTypesAsLattices.Retracts.envR
public import DomainTheory.DataTypesAsLattices.Retracts.updateEnv
public import DomainTheory.DataTypesAsLattices.Retracts.tagInj
public import DomainTheory.DataTypesAsLattices.Retracts.seq4
public import DomainTheory.DataTypesAsLattices.Retracts.tensor4
public import DomainTheory.DataTypesAsLattices.Retracts.plus7Body
public import DomainTheory.DataTypesAsLattices.Retracts.plus7
public import DomainTheory.DataTypesAsLattices.Retracts.expF
public import DomainTheory.DataTypesAsLattices.Retracts.expR
public import DomainTheory.DataTypesAsLattices.Retracts.sUnion_Fixpoints_le_image
public import DomainTheory.DataTypesAsLattices.Retracts.BasicInst36
public import DomainTheory.DataTypesAsLattices.Retracts.retract_app
public import DomainTheory.DataTypesAsLattices.Retracts.funOf_botElem
public import DomainTheory.DataTypesAsLattices.Retracts.graph_const_bot
public import DomainTheory.DataTypesAsLattices.Retracts.bot_isRetract
public import DomainTheory.DataTypesAsLattices.Retracts.iterates_areRetracts
public import DomainTheory.DataTypesAsLattices.Retracts.bot_retractLe
public import DomainTheory.DataTypesAsLattices.Retracts.iterateBot_isStrict
public import DomainTheory.DataTypesAsLattices.Retracts.iterateBot_retractLe
public import DomainTheory.DataTypesAsLattices.Retracts.BasicInst55
public import DomainTheory.DataTypesAsLattices.Retracts.inverseLimit_mono
public import DomainTheory.DataTypesAsLattices.Retracts.inverseLimit_chain
public import DomainTheory.DataTypesAsLattices.Retracts.inverseLimit_project_later
public import DomainTheory.DataTypesAsLattices.Retracts.inverseLimit_project_earlier
public import DomainTheory.DataTypesAsLattices.Retracts.inverseLimit_project_iUnion
public import DomainTheory.DataTypesAsLattices.Retracts.inverseLimit_iUnion_typed
public import DomainTheory.DataTypesAsLattices.Retracts.typed_more
public import DomainTheory.DataTypesAsLattices.Retracts.arrowR_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, Theorem 4.3 (The function space theorem).** -/
theorem theorem_4_3 {a b a' b' c : Pomega}
    (ha : Retract a) (hb : Retract b) (_ha' : Retract a')
    (_hb' : Retract b') (hc : Retract c) :
    Retract (arrowR a b) ∧
      (IsStrict b → IsStrict (arrowR a b)) ∧
      (∀ u, typed u (arrowR a b) ↔
        u = comp u a ∧ ∀ x, typed x a → typed (funOf u x) b) ∧
      (retractLe a a' → retractLe b b' →
        retractLe (arrowR a b) (arrowR a' b')) ∧
      (∀ f f', typed f (arrowR a b) → typed f' (arrowR a' b') →
        typed (arrowR f f') (arrowR (arrowR b a') (arrowR a b'))) ∧
      (∀ f f', typed f (arrowR a b) → typed f' (arrowR b c) →
        typed (comp f' f) (arrowR a c)) :=
  ⟨arrowR.isRetract ha hb, arrowR.isStrict, fun _ => typed.arrowR_iff ha hb,
    arrowR.mono, fun _ _ => arrowR.functor,
    fun _ _ hf hf' => typed.comp ha hb hc hf hf'⟩
```

```
end DomainTheory.DataTypesAsLattices
```

theorem_5_3.lean

```
module

public import Mathlib.Algebra.Ring.Parity
public import DomainTheory.DataTypesAsLattices.Retracts.Retract
public import DomainTheory.DataTypesAsLattices.Retracts.arrowR
public import DomainTheory.DataTypesAsLattices.Retracts.graph_ext
public import DomainTheory.DataTypesAsLattices.Retracts.theorem_4_3
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Closures.Closure
public import DomainTheory.DataTypesAsLattices.Closures.Icomb_subset_arrowR
public import DomainTheory.DataTypesAsLattices.Retracts.arrowR_more
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, Theorem 5.3 (The function space theorem for algebraic
lattices).** The function-space retract of two closures is a retract. -/
theorem theorem_5_3 {a b : Pomega} (ha : Closure a) (hb : Closure b) :
    Retract (arrowR a b) := by
  constructor
  change arrowR a b = graph (fun x => funOf (arrowR a b) (funOf (arrowR a b) x))
  apply graph_ext
  intro u
  rw [theorem_4_3.i ha.out.2 hb.out.2, arrowR.app]

namespace theorem_5_3

theorem closure {a b : Pomega} (ha : Closure a) (hb : Closure b) :
    Closure (arrowR a b) :=
  ⟨⟨Icomb_subset_arrowR ha hb, theorem_5_3 ha hb⟩⟩

end theorem_5_3

end DomainTheory.DataTypesAsLattices
```

eq_5_24.lean

```
module

public import Mathlib.Algebra.Ring.Parity
public import DomainTheory.DataTypesAsLattices.Retracts.typed
public import DomainTheory.DataTypesAsLattices.Retracts.arrowR
public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Pomega.e
```

```
public import DomainTheory.DataTypesAsLattices.Closures.Vcomb
public import DomainTheory.DataTypesAsLattices.Closures.theorem_5_5
public import DomainTheory.DataTypesAsLattices.Closures.theorem_5_3
public import DomainTheory.DataTypesAsLattices.Closures.theorem_5_5_universe
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (5.24).** Function space of closures is a closure. -/
theorem eq_5_24 {a b : Pomega} (ha : typed a Vcomb) (hb : typed b Vcomb) :
    typed (arrowR a b) Vcomb :=
  (theorem_5_5.universe.2 (arrowR a b)).mp
    (theorem_5_3.closure ((theorem_5_5.universe.2 a).mpr ha)
      ((theorem_5_5.universe.2 b).mpr hb))

end DomainTheory.DataTypesAsLattices
```

theorem_5_6.lean

```
module

public import Mathlib.Algebra.Ring.Parity
public import DomainTheory.DataTypesAsLattices.Retracts.typed
public import DomainTheory.DataTypesAsLattices.Retracts.arrowR
public import DomainTheory.DataTypesAsLattices.Retracts.comp
public import DomainTheory.DataTypesAsLattices.Retracts.retract_app
public import DomainTheory.DataTypesAsLattices.Lambda.Ycomb
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Closures.Closure
public import DomainTheory.DataTypesAsLattices.Closures.Vcomb
public import DomainTheory.DataTypesAsLattices.Closures.theorem_5_5
public import DomainTheory.DataTypesAsLattices.Retracts.Icomb
public import DomainTheory.DataTypesAsLattices.Retracts.iterateFrom
public import DomainTheory.DataTypesAsLattices.Retracts.lfpAbove
public import DomainTheory.DataTypesAsLattices.Closures.theorem_5_3
public import DomainTheory.DataTypesAsLattices.Closures.Icomb_isClosure
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Retracts.graph_ext
public import DomainTheory.DataTypesAsLattices.Closures.Vcomb_more
public import DomainTheory.DataTypesAsLattices.Lambda.Ycomb_more
public import DomainTheory.DataTypesAsLattices.Retracts.arrowR_more
public import DomainTheory.DataTypesAsLattices.Closures.theorem_5_5_universe
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, Theorem 5.6 (The limit theorem for algebraic lattices).**
`(λf : Vo→V. Y(f)) : (Vo→V) o→ V`. -/
theorem theorem_5_6 {f : Pomega} (hf : typed f (arrowR Vcomb Vcomb)) :
    typed (funOf Ycomb f) Vcomb := by
  set y := funOf Ycomb f
  have hy : y = funOf f y := Ycomb.unfold f
  have hf' : ∀ x, funOf f x = funOf Vcomb (funOf f (funOf Vcomb x)) := by
    intro x
```

```
    simpa [comp.app] using congrArg (fun w => funOf w x) (arrowR.typed_of hf)
  have hyV : y = funOf Vcomb (funOf f (funOf Vcomb y)) :=
    hy.trans (hf' y)
  have h1 : funOf Vcomb y = funOf Vcomb (funOf f y) :=
    congrArg (funOf Vcomb) hy
  have h2 : funOf Vcomb (funOf f y) =
      funOf Vcomb (funOf Vcomb (funOf f (funOf Vcomb y))) :=
    congrArg (funOf Vcomb) (hf' y)
  have h3 : funOf Vcomb (funOf Vcomb (funOf f (funOf Vcomb y))) =
      funOf Vcomb (funOf f (funOf Vcomb y)) :=
    retract_app Vcomb.isRetract _
  exact (h1.trans (h2.trans (h3.trans hyV.symm))).symm

namespace theorem_5_6

/-- **Scott 1976, Theorem 5.6, combinator form.**
`(λf : Vo→V. Y(f)) : (Vo→V) o→ V`. -/
theorem combinator :
    typed (comp Ycomb (arrowR Vcomb Vcomb))
      (arrowR (arrowR Vcomb Vcomb) Vcomb) := by
  rw [typed, arrowR.app]
  apply graph_ext
  intro f
  have hV : Closure Vcomb := theorem_5_5.universe.1
  have harr : Retract (arrowR Vcomb Vcomb) := theorem_5_3 hV hV
  have hf' : typed (funOf (arrowR Vcomb Vcomb) f) (arrowR Vcomb Vcomb) :=
    (retract_app harr f).symm
  have hy := theorem_5_6 hf'
  simp only [comp.app]
  rw [retract_app harr]
  exact hy

/-- Kleene iterates of `I` under a closure-preserving operator stay closures. -/
theorem iterates {F : Pomega → Pomega}
    (_hF : IsScottContinuous F)
    (hcl : ∀ a, Closure a → Closure (F a)) :
    (∀ n, Closure (iterateFrom F Icomb n)) ∧
      Icomb ⊆ lfpAbove F Icomb := by
  have hiter : ∀ n, Closure (iterateFrom F Icomb n) := by
    intro n
    induction n with
    | zero => exact Icomb_isClosure
    | succ n ih => exact hcl _ ih
  exact ⟨hiter, Set.subset_iUnion (iterateFrom F Icomb) 0⟩

end theorem_5_6

end DomainTheory.DataTypesAsLattices
```

Ea.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.typed
public import DomainTheory.DataTypesAsLattices.Pomega.e
```

```
public import DomainTheory.DataTypesAsLattices.Functionality.RestrictedEquiv
public import DomainTheory.DataTypesAsLattices.Functionality.mem

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (7.4).** Identity relation on the range of a retract. -/
def Ea (a : Pomega) : RestrictedEquiv where
  rel := fun x y => typed x a ∧ typed y a ∧ x = y
  symm := by
    intro x y ⟨hx, hy, h⟩
    exact ⟨hy, hx, h.symm⟩
  trans := by
    intro x y z ⟨hx, hy, hxy⟩ ⟨_, hz, hyz⟩
    exact ⟨hx, hz, hxy.trans hyz⟩

end DomainTheory.DataTypesAsLattices
```

Htyped.lean

```
module

public import DomainTheory.DataTypesAsLattices.Functionality.Hinterp
public import DomainTheory.DataTypesAsLattices.Functionality.LambTerm
public import DomainTheory.DataTypesAsLattices.Functionality.envR_isRetract
public import DomainTheory.DataTypesAsLattices.Functionality.lambR_isRetract
public import DomainTheory.DataTypesAsLattices.Functionality.typed_apply_retract
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.arrowR
public import DomainTheory.DataTypesAsLattices.Retracts.arrowR_more
public import DomainTheory.DataTypesAsLattices.Retracts.comp
public import DomainTheory.DataTypesAsLattices.Retracts.envR
public import DomainTheory.DataTypesAsLattices.Retracts.lambR
public import DomainTheory.DataTypesAsLattices.Retracts.typed

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def Htyped (τ : LambTerm) : Pomega :=
  funOf (arrowR envR lambR) (graph (Hinterp τ))

namespace Htyped

theorem app (τ : LambTerm) (t : Pomega) :
    funOf (Htyped τ) t = funOf lambR (Hinterp τ (funOf envR t)) := by
  rw [Htyped, arrowR.app]
  simp only [comp.app]
  apply congrArg (funOf lambR)
  exact beta (Hinterp.isScottContinuous τ) (funOf envR t)
```

```
theorem typed (τ : LambTerm) :
    typed (Htyped τ) (arrowR envR lambR) :=
  typed_apply_retract
    (arrowR.isRetract envR_isRetract lambR_isRetract)

end Htyped

end DomainTheory.DataTypesAsLattices
```

Icomb_isGraph.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.Icomb
public import DomainTheory.DataTypesAsLattices.Retracts.graph_ext
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem Icomb_isGraph : Icomb = graph (fun x => funOf Icomb x) := by
  apply graph_ext
  intro x
  rw [Icomb.app]

end DomainTheory.DataTypesAsLattices
```

Kcomb_app.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.id_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Lambda.Kcomb
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem Kcomb_app (x : Pomega) :
    funOf Kcomb x = graph (fun _ => x) :=
  beta (graph_const_isScottContinuous (fun x _ => x)
    (fun _ => id_isScottContinuous)) x

end DomainTheory.DataTypesAsLattices
```

Kcomb_app2.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.const_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Lambda.Kcomb
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.Kcomb_app

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem Kcomb_app2 (x y : Pomega) : funOf (funOf Kcomb x) y = x := by
  rw [Kcomb_app]
  exact beta (const_isScottContinuous x) y

end DomainTheory.DataTypesAsLattices
```

Kcomb_app_isGraph.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.graph_ext
public import DomainTheory.DataTypesAsLattices.Lambda.const_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Lambda.Kcomb
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.Kcomb_app

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem Kcomb_app_isGraph (x : Pomega) :
    funOf Kcomb x = graph (fun y => funOf (funOf Kcomb x) y) := by
  rw [Kcomb_app]
  apply graph_ext
  intro y
  exact (beta (const_isScottContinuous x) y).symm

end DomainTheory.DataTypesAsLattices
```

Kcomb_isGraph.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.graph_ext
public import DomainTheory.DataTypesAsLattices.Lambda.Kcomb
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Functionality.Kcomb_app
```

```
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem Kcomb_isGraph : Kcomb = graph (fun x => funOf Kcomb x) := by
  apply graph_ext
  intro x
  rw [Kcomb_app]

end DomainTheory.DataTypesAsLattices
```

Nk.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Functionality.RestrictedEquiv
public import DomainTheory.DataTypesAsLattices.Functionality.mem

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (7.12).** Finite discrete spaces. -/
def Nk (k : ℕ) : RestrictedEquiv where
  rel := fun x y => (∃ n < k, x = ofNat n) ∧ x = y
  symm := by
    intro x y ⟨hn, h⟩
    exact ⟨by rw [← h]; exact hn, h.symm⟩
  trans := by
    intro x y z ⟨hn, hxy⟩ ⟨_, hyz⟩
    exact ⟨hn, hxy.trans hyz⟩

end DomainTheory.DataTypesAsLattices
```

Nrel.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Functionality.RestrictedEquiv
public import DomainTheory.DataTypesAsLattices.Functionality.mem

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (7.9).** The type of total integers. -/
def Nrel : RestrictedEquiv where
  rel := fun x y => (∃ n, x = ofNat n) ∧ x = y
  symm := by
    intro x y ⟨hn, h⟩
    exact ⟨by rw [← h]; exact hn, h.symm⟩
  trans := by
    intro x y z ⟨hn, hxy⟩ ⟨_, hyz⟩
```

```
    exact ⟨hn, hxy.trans hyz⟩

end DomainTheory.DataTypesAsLattices
```

Scomb_map_isScottContinuous.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem Scomb_map_isScottContinuous :
    IsScottContinuous (fun u => graph (fun v =>
      graph (fun x => funOf (funOf u x) (funOf v x)))) :=
  graph_const_isScottContinuous
    (fun u v => graph (fun x => funOf (funOf u x) (funOf v x)))
    (fun v =>
      graph_const_isScottContinuous
        (fun u x => funOf (funOf u x) (funOf v x))
        (fun x =>
          theorem_1_3
            (f := fun t => funOf t (funOf v x))
            (g := fun u => funOf u x)
            (funOf.isScottContinuous_left (funOf v x))
            (funOf.isScottContinuous_left x)))

end DomainTheory.DataTypesAsLattices
```

Scomb_app.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Lambda.Scomb
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.
    Scomb_map_isScottContinuous

@[expose] public section
```

```
namespace DomainTheory.DataTypesAsLattices

theorem Scomb_app (f : Pomega) :
    funOf Scomb f =
      graph (fun v => graph (fun x => funOf (funOf f x) (funOf v x))) :=
  beta Scomb_map_isScottContinuous f

end DomainTheory.DataTypesAsLattices
```

Scomb_mid_isScottContinuous.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem Scomb_mid_isScottContinuous (u : Pomega) :
    IsScottContinuous (fun v =>
      graph (fun x => funOf (funOf u x) (funOf v x))) :=
  graph_const_isScottContinuous
    (fun v x => funOf (funOf u x) (funOf v x))
    (fun x =>
      theorem_1_3 (funOf.isScottContinuous (funOf u x))
        (funOf.isScottContinuous_left x))

end DomainTheory.DataTypesAsLattices
```

Scomb_app2.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Lambda.Scomb
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.
    Scomb_mid_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Functionality.Scomb_app

@[expose] public section
```

```
namespace DomainTheory.DataTypesAsLattices

theorem Scomb_app2 (f g : Pomega) :
    funOf (funOf Scomb f) g =
      graph (fun x => funOf (funOf f x) (funOf g x)) := by
  rw [Scomb_app]
  exact beta (Scomb_mid_isScottContinuous f) g

end DomainTheory.DataTypesAsLattices
```

Scomb_inner_isScottContinuous.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.continuous_tuple
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem Scomb_inner_isScottContinuous (u v : Pomega) :
    IsScottContinuous (fun x => funOf (funOf u x) (funOf v x)) :=
  continuous_tuple
    (fun y => funOf.isScottContinuous_left y)
    (fun t => funOf.isScottContinuous t)
    (funOf.isScottContinuous u)
    (funOf.isScottContinuous v)

end DomainTheory.DataTypesAsLattices
```

Scomb_app2_isGraph.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.graph_ext
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Lambda.Scomb
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.
    Scomb_inner_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Functionality.Scomb_app2

@[expose] public section
```

```
namespace DomainTheory.DataTypesAsLattices

theorem Scomb_app2_isGraph (f g : Pomega) :
    funOf (funOf Scomb f) g =
      graph (fun x => funOf (funOf (funOf Scomb f) g) x) := by
  rw [Scomb_app2]
  apply graph_ext
  intro x
  exact (beta (Scomb_inner_isScottContinuous f g) x).symm

end DomainTheory.DataTypesAsLattices
```

Scomb_app3.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Lambda.Scomb
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.
    Scomb_inner_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Functionality.Scomb_app2

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem Scomb_app3 (f g x : Pomega) :
    funOf (funOf (funOf Scomb f) g) x = funOf (funOf f x) (funOf g x) := by
  rw [Scomb_app2]
  exact beta (Scomb_inner_isScottContinuous f g) x

end DomainTheory.DataTypesAsLattices
```

Scomb_app_isGraph.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.graph_ext
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Lambda.Scomb
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.
    Scomb_mid_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Functionality.Scomb_app

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem Scomb_app_isGraph (f : Pomega) :
```

```
    funOf Scomb f = graph (fun g => funOf (funOf Scomb f) g) := by
  rw [Scomb_app]
  apply graph_ext
  intro g
  exact (beta (Scomb_mid_isScottContinuous f) g).symm

end DomainTheory.DataTypesAsLattices
```

Scomb_isGraph.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.graph_ext
public import DomainTheory.DataTypesAsLattices.Lambda.Scomb
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Functionality.Scomb_app

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem Scomb_isGraph : Scomb = graph (fun u => funOf Scomb u) := by
  apply graph_ext
  intro u
  rw [Scomb_app]

end DomainTheory.DataTypesAsLattices
```

comp_isGraph.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.comp
public import DomainTheory.DataTypesAsLattices.Retracts.graph_ext
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem comp_isGraph (u v : Pomega) :
    comp u v = graph (fun x => funOf (comp u v) x) := by
  apply graph_ext
  intro x
  exact (comp.app u v x).symm

end DomainTheory.DataTypesAsLattices
```

curryC_isScottContinuous.lean

```
module
```

```
public import DomainTheory.DataTypesAsLattices.Retracts.pairSeq
public import Mathlib.Order.CompleteLattice.Defs
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem curryC_isScottContinuous :
    IsScottContinuous (fun u =>
      graph (fun x => graph (fun y => funOf u (pairSeq x y)))) :=
  graph_const_isScottContinuous
    (fun u x => graph (fun y => funOf u (pairSeq x y)))
    (fun x =>
      graph_const_isScottContinuous
        (fun u y => funOf u (pairSeq x y))
        (fun y => funOf.isScottContinuous_left (pairSeq x y)))

end DomainTheory.DataTypesAsLattices
```

curryC_app.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.pairSeq
public import DomainTheory.DataTypesAsLattices.Retracts.curryC
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.curryC_isScottContinuous

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem curryC_app (u : Pomega) :
    funOf curryC u = graph (fun x => graph (fun y => funOf u (pairSeq x y))) :=
  beta curryC_isScottContinuous u

end DomainTheory.DataTypesAsLattices
```

curry_mid_isScottContinuous.lean

```
module
```

```
public import DomainTheory.DataTypesAsLattices.Retracts.pairSeq
public import Mathlib.Order.CompleteLattice.Defs
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.
    pairSeq_isScottContinuous_left
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem curry_mid_isScottContinuous (u : Pomega) :
    IsScottContinuous (fun x => graph (fun y => funOf u (pairSeq x y))) :=
  graph_const_isScottContinuous
    (fun x y => funOf u (pairSeq x y))
    (fun y => theorem_1_3 (funOf.isScottContinuous u)
      (pairSeq_isScottContinuous_left y))

end DomainTheory.DataTypesAsLattices
```

diagC_app.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.pairSeq
public import DomainTheory.DataTypesAsLattices.Retracts.diagC
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.continuous_on_diag
public import DomainTheory.DataTypesAsLattices.Functionality.
    pairSeq_isScottContinuous_left
public import DomainTheory.DataTypesAsLattices.Functionality.
    pairSeq_isScottContinuous_right

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem diagC_app (u : Pomega) : funOf diagC u = pairSeq u u :=
  beta (continuous_on_diag
    pairSeq_isScottContinuous_left pairSeq_isScottContinuous_right) u

end DomainTheory.DataTypesAsLattices
```

emptyE.lean

```
module

public import DomainTheory.DataTypesAsLattices.Functionality.RestrictedEquiv
public import DomainTheory.DataTypesAsLattices.Functionality.mem

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- Empty restricted equivalence (never related). -/
def emptyE : RestrictedEquiv where
  rel := fun _ _ => False
  symm := fun h => False.elim h
  trans := fun h _ => False.elim h

end DomainTheory.DataTypesAsLattices
```

eq_4_44.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.expF
public import DomainTheory.DataTypesAsLattices.Retracts.expR
public import DomainTheory.DataTypesAsLattices.Lambda.theorem_2_5
public import DomainTheory.DataTypesAsLattices.FixedPoint.fix
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_4
public import DomainTheory.DataTypesAsLattices.Functionality.expF_isScottContinuous

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (4.44).** `exp` is the least fixed point of the seven-tag
syntax functor. -/
theorem eq_4_44 : expR = expF expR := by
  have hY : expR = fix expF := theorem_2_5 expF_isScottContinuous
  have hfix : expF expR = expR := by
    rw [hY]
    exact (theorem_1_4 expF_isScottContinuous).1
  exact hfix.symm

end DomainTheory.DataTypesAsLattices
```

plus7_typed_inl0.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.typed
public import DomainTheory.DataTypesAsLattices.Retracts.tagInj
public import DomainTheory.DataTypesAsLattices.Retracts.plus7
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.plus7_app

@[expose] public section
```

```
namespace DomainTheory.DataTypesAsLattices

theorem plus7_typed_inl0 {a0 a1 a2 a3 a4 a5 a6 x : Pomega} (hx : typed x a0) :
    typed (tagInj 0 x) (plus7 a0 a1 a2 a3 a4 a5 a6) := by
  rw [typed, plus7_app.tag0, ← hx]

end DomainTheory.DataTypesAsLattices
```

plus7_typed_inl1.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.typed
public import DomainTheory.DataTypesAsLattices.Retracts.tagInj
public import DomainTheory.DataTypesAsLattices.Retracts.plus7
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.plus7_app

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem plus7_typed_inl1 {a0 a1 a2 a3 a4 a5 a6 x : Pomega} (hx : typed x a1) :
    typed (tagInj 1 x) (plus7 a0 a1 a2 a3 a4 a5 a6) := by
  rw [typed, plus7_app.tag1, ← hx]

end DomainTheory.DataTypesAsLattices
```

plus7_typed_inl2.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.typed
public import DomainTheory.DataTypesAsLattices.Retracts.tagInj
public import DomainTheory.DataTypesAsLattices.Retracts.plus7
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.plus7_app

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem plus7_typed_inl2 {a0 a1 a2 a3 a4 a5 a6 x : Pomega} (hx : typed x a2) :
    typed (tagInj 2 x) (plus7 a0 a1 a2 a3 a4 a5 a6) := by
  rw [typed, plus7_app.tag2, ← hx]

end DomainTheory.DataTypesAsLattices
```

plus7_typed_inl3.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.typed
public import DomainTheory.DataTypesAsLattices.Retracts.tagInj
public import DomainTheory.DataTypesAsLattices.Retracts.plus7
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.plus7_app

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem plus7_typed_inl3 {a0 a1 a2 a3 a4 a5 a6 x : Pomega} (hx : typed x a3) :
    typed (tagInj 3 x) (plus7 a0 a1 a2 a3 a4 a5 a6) := by
  rw [typed, plus7_app.tag3, ← hx]

end DomainTheory.DataTypesAsLattices
```

plus7_typed_inl4.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.typed
public import DomainTheory.DataTypesAsLattices.Retracts.tagInj
public import DomainTheory.DataTypesAsLattices.Retracts.plus7
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.plus7_app

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem plus7_typed_inl4 {a0 a1 a2 a3 a4 a5 a6 x : Pomega} (hx : typed x a4) :
    typed (tagInj 4 x) (plus7 a0 a1 a2 a3 a4 a5 a6) := by
  rw [typed, plus7_app.tag4, ← hx]

end DomainTheory.DataTypesAsLattices
```

plus7_typed_inl5.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.typed
public import DomainTheory.DataTypesAsLattices.Retracts.tagInj
public import DomainTheory.DataTypesAsLattices.Retracts.plus7
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.plus7_app

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem plus7_typed_inl5 {a0 a1 a2 a3 a4 a5 a6 x : Pomega} (hx : typed x a5) :
```

```
    typed (tagInj 5 x) (plus7 a0 a1 a2 a3 a4 a5 a6) := by
  rw [typed, plus7_app.tag5, ← hx]

end DomainTheory.DataTypesAsLattices
```

plus7_typed_inl6.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.typed
public import DomainTheory.DataTypesAsLattices.Retracts.tagInj
public import DomainTheory.DataTypesAsLattices.Retracts.plus7
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.plus7_app

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem plus7_typed_inl6 {a0 a1 a2 a3 a4 a5 a6 x : Pomega} (hx : typed x a6) :
    typed (tagInj 6 x) (plus7 a0 a1 a2 a3 a4 a5 a6) := by
  rw [typed, plus7_app.tag6, ← hx]

end DomainTheory.DataTypesAsLattices
```

typed_bot_botElem.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.typed
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem typed_bot_botElem : typed botElem botElem := by
  change botElem = funOf botElem botElem
  rw [funOf.bot]

end DomainTheory.DataTypesAsLattices
```

typed_ofNat_intR.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.typed
public import DomainTheory.DataTypesAsLattices.Retracts.intR
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
```

```
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem typed_ofNat_intR (n : ℕ) : typed (ofNat n) intR :=
  (intR.app_ofNat n).symm

end DomainTheory.DataTypesAsLattices
```

typed_pairSeq_tensorR.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.typed
public import DomainTheory.DataTypesAsLattices.Retracts.pairSeq
public import DomainTheory.DataTypesAsLattices.Retracts.tensorR
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.pairSeq_app_zero
public import DomainTheory.DataTypesAsLattices.Functionality.pairSeq_app_one
public import DomainTheory.DataTypesAsLattices.Functionality.pairSeq_inj
public import DomainTheory.DataTypesAsLattices.Functionality.tensorR_app

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem typed_pairSeq_tensorR (a b x y : Pomega) :
    typed (pairSeq x y) (tensorR a b) ↔ typed x a ∧ typed y b := by
  constructor
  · intro h
    have h' : pairSeq x y =
        pairSeq (funOf a x) (funOf b y) := by
      rw [typed, tensorR_app, pairSeq_app_zero, pairSeq_app_one] at h
      exact h
    exact pairSeq_inj h'
  · intro ⟨hx, hy⟩
    rw [typed, tensorR_app, pairSeq_app_zero, pairSeq_app_one, ← hx, ← hy]

end DomainTheory.DataTypesAsLattices
```

encodeExp_typed.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.typed
public import DomainTheory.DataTypesAsLattices.Retracts.intR
public import DomainTheory.DataTypesAsLattices.Retracts.expR
public import DomainTheory.DataTypesAsLattices.Graph.lam
public import DomainTheory.DataTypesAsLattices.Functionality.LambTerm
public import DomainTheory.DataTypesAsLattices.Functionality.encodeExp
public import DomainTheory.DataTypesAsLattices.Functionality.typed_pairSeq_tensorR
public import DomainTheory.DataTypesAsLattices.Functionality.tensor4_app
```

```
public import DomainTheory.DataTypesAsLattices.Functionality.seq4_app
public import DomainTheory.DataTypesAsLattices.Functionality.eq_4_44
public import DomainTheory.DataTypesAsLattices.Functionality.plus7_typed_inl0
public import DomainTheory.DataTypesAsLattices.Functionality.plus7_typed_inl1
public import DomainTheory.DataTypesAsLattices.Functionality.plus7_typed_inl2
public import DomainTheory.DataTypesAsLattices.Functionality.plus7_typed_inl3
public import DomainTheory.DataTypesAsLattices.Functionality.plus7_typed_inl4
public import DomainTheory.DataTypesAsLattices.Functionality.plus7_typed_inl5
public import DomainTheory.DataTypesAsLattices.Functionality.plus7_typed_inl6
public import DomainTheory.DataTypesAsLattices.Functionality.typed_ofNat_intR
public import DomainTheory.DataTypesAsLattices.Functionality.typed_bot_botElem

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem encodeExp_typed : ∀ τ : LambTerm, typed (encodeExp τ) expR := by
  intro τ
  induction τ with
  | var n =>
    rw [encodeExp, eq_4_44]
    exact plus7_typed_inl0 (typed_ofNat_intR n)
  | zero =>
    rw [encodeExp, eq_4_44]
    exact plus7_typed_inl1 typed_bot_botElem
  | succ τ ih =>
    rw [encodeExp, eq_4_44]
    exact plus7_typed_inl2 ih
  | pred τ ih =>
    rw [encodeExp, eq_4_44]
    exact plus7_typed_inl3 ih
  | condq ? τ σ ? ih? ihτ ihσ ih? =>
    rw [encodeExp, eq_4_44]
    refine plus7_typed_inl4 ?_
    rw [typed, tensor4_app, seq4_app.zero, seq4_app.one, seq4_app.two,
      seq4_app.three, ← ih?, ← ihτ, ← ihσ, ← ih?]
  | app τ σ ihτ ihσ =>
    rw [encodeExp, eq_4_44]
    refine plus7_typed_inl5 ?_
    exact (typed_pairSeq_tensorR expR expR _ _).mpr ⟨ihτ, ihσ⟩
  | lam n τ ih =>
    rw [encodeExp, eq_4_44]
    refine plus7_typed_inl6 ?_
    exact (typed_pairSeq_tensorR intR expR _ _).mpr ⟨typed_ofNat_intR n, ih⟩

end DomainTheory.DataTypesAsLattices
```

eq_4_12.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.comp
public import DomainTheory.DataTypesAsLattices.Retracts.tensorR
public import DomainTheory.DataTypesAsLattices.Retracts.graph_ext
public import DomainTheory.DataTypesAsLattices.Pomega.e
```

```
public import DomainTheory.DataTypesAsLattices.Functionality.pairSeq_app_zero
public import DomainTheory.DataTypesAsLattices.Functionality.pairSeq_app_one
public import DomainTheory.DataTypesAsLattices.Functionality.tensorR_app

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (4.12).** `(a ? b) ∘ (a' ? b') = (a ∘ a') ? (b ∘ b')`. -/
theorem eq_4_12 (a b a' b' : Pomega) :
    comp (tensorR a b) (tensorR a' b') = tensorR (comp a a') (comp b b') := by
  apply graph_ext
  intro u
  rw [tensorR_app, tensorR_app, pairSeq_app_zero, pairSeq_app_one]
  simp only [comp.app]

end DomainTheory.DataTypesAsLattices
```

eq_4_13.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.comp
public import DomainTheory.DataTypesAsLattices.Retracts.plusR
public import DomainTheory.DataTypesAsLattices.Retracts.graph_ext
public import DomainTheory.DataTypesAsLattices.Lambda.dcondSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.pairSeq_app_zero
public import DomainTheory.DataTypesAsLattices.Functionality.pairSeq_app_one
public import DomainTheory.DataTypesAsLattices.Functionality.plusR_app
public import DomainTheory.DataTypesAsLattices.Lambda.dcondSet_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (4.13).** `(a ? b) ∘ (a' ? b') = (a ∘ a') ? (b ∘ b')`. -/
theorem eq_4_13 (a b a' b' : Pomega) :
    comp (plusR a b) (plusR a' b') = plusR (comp a a') (comp b b') := by
  apply graph_ext
  intro u
  rw [plusR_app.app a' b' u]
  rcases dcondSet.cases (funOf u (ofNat 0)) with h | h | h | h
  · rw [dcondSet.of_zero_not_pos h.1 h.2, plusR_app.app,
      pairSeq_app_zero, pairSeq_app_one, dcondSet.ofNat_zero,
      dcondSet.of_zero_not_pos h.1 h.2]
    simp only [comp.app]
  · rw [dcondSet.of_pos_not_zero h.1 h.2, plusR_app.app,
      pairSeq_app_zero, pairSeq_app_one, dcondSet.ofNat_one,
      dcondSet.of_pos_not_zero h.1 h.2]
    simp only [comp.app]
  · rw [dcondSet.of_mixed h.1 h.2, plusR_app.top, dcondSet.of_mixed h.1 h.2]
  · rw [dcondSet.of_empty h.1 h.2, plusR_app.bot, dcondSet.of_empty h.1 h.2]
```

```
end DomainTheory.DataTypesAsLattices
```

eq_4_17.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.comp
public import DomainTheory.DataTypesAsLattices.Retracts.Retract
public import DomainTheory.DataTypesAsLattices.Retracts.typed
public import DomainTheory.DataTypesAsLattices.Retracts.arrowR
public import DomainTheory.DataTypesAsLattices.Retracts.fstC
public import DomainTheory.DataTypesAsLattices.Retracts.tensorR
public import DomainTheory.DataTypesAsLattices.Retracts.retract_app
public import DomainTheory.DataTypesAsLattices.Retracts.graph_ext
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.pairSeq_app_zero
public import DomainTheory.DataTypesAsLattices.Functionality.tensorR_app
public import DomainTheory.DataTypesAsLattices.Functionality.tensorR_isRetract
public import DomainTheory.DataTypesAsLattices.Retracts.arrowR_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (4.17).** `fst ∘ (a ? b) : (a ? b) ⊸ a`. -/
theorem eq_4_17 {a b : Pomega} (ha : Retract a) (hb : Retract b) :
    typed (comp fstC (tensorR a b)) (arrowR (tensorR a b) a) := by
  change comp fstC (tensorR a b) =
    funOf (arrowR (tensorR a b) a) (comp fstC (tensorR a b))
  rw [arrowR.app]
  apply graph_ext
  intro u
  have hab := tensorR_isRetract ha hb
  simp only [comp.app, fstC.app, retract_app hab]
  rw [tensorR_app, pairSeq_app_zero, retract_app ha]

end DomainTheory.DataTypesAsLattices
```

eq_4_18.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.comp
public import DomainTheory.DataTypesAsLattices.Retracts.Retract
public import DomainTheory.DataTypesAsLattices.Retracts.typed
public import DomainTheory.DataTypesAsLattices.Retracts.arrowR
public import DomainTheory.DataTypesAsLattices.Retracts.sndC
public import DomainTheory.DataTypesAsLattices.Retracts.tensorR
public import DomainTheory.DataTypesAsLattices.Retracts.retract_app
public import DomainTheory.DataTypesAsLattices.Retracts.graph_ext
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.pairSeq_app_one
```

```
public import DomainTheory.DataTypesAsLattices.Functionality.tensorR_app
public import DomainTheory.DataTypesAsLattices.Functionality.tensorR_isRetract
public import DomainTheory.DataTypesAsLattices.Retracts.arrowR_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (4.18).** `snd ∘ (a ? b) : (a ? b) ∘→ b`. -/
theorem eq_4_18 {a b : Pomega} (ha : Retract a) (hb : Retract b) :
    typed (comp sndC (tensorR a b)) (arrowR (tensorR a b) b) := by
  change comp sndC (tensorR a b) =
    funOf (arrowR (tensorR a b) b) (comp sndC (tensorR a b))
  rw [arrowR.app]
  apply graph_ext
  intro u
  have hab := tensorR_isRetract ha hb
  simp only [comp.app, sndC.app, retract_app hab]
  rw [tensorR_app, pairSeq_app_one, retract_app hb]

end DomainTheory.DataTypesAsLattices
```

eq_4_19.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.comp
public import DomainTheory.DataTypesAsLattices.Retracts.Retract
public import DomainTheory.DataTypesAsLattices.Retracts.typed
public import DomainTheory.DataTypesAsLattices.Retracts.arrowR
public import DomainTheory.DataTypesAsLattices.Retracts.tensorR
public import DomainTheory.DataTypesAsLattices.Retracts.diagC
public import DomainTheory.DataTypesAsLattices.Retracts.retract_app
public import DomainTheory.DataTypesAsLattices.Retracts.graph_ext
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.pairSeq_app_zero
public import DomainTheory.DataTypesAsLattices.Functionality.pairSeq_app_one
public import DomainTheory.DataTypesAsLattices.Functionality.tensorR_app
public import DomainTheory.DataTypesAsLattices.Functionality.diagC_app
public import DomainTheory.DataTypesAsLattices.Retracts.arrowR_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (4.19).** `diag ∘ a : a ∘→ a ? a`. -/
theorem eq_4_19 {a : Pomega} (ha : Retract a) :
    typed (comp diagC a) (arrowR a (tensorR a a)) := by
  change comp diagC a = funOf (arrowR a (tensorR a a)) (comp diagC a)
  rw [arrowR.app]
  apply graph_ext
  intro u
  simp only [comp.app, diagC_app, retract_app ha]
  rw [tensorR_app, pairSeq_app_zero, pairSeq_app_one, retract_app ha]
```

```
end DomainTheory.DataTypesAsLattices
```

eq_4_20.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.comp
public import DomainTheory.DataTypesAsLattices.Retracts.fstC
public import DomainTheory.DataTypesAsLattices.Retracts.tensorR
public import DomainTheory.DataTypesAsLattices.Retracts.graph_ext
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.pairSeq_app_zero
public import DomainTheory.DataTypesAsLattices.Functionality.tensorR_app

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (4.20).** `fst ∘ (f ? f') = f ∘ fst`. -/
theorem eq_4_20 (f f' : Pomega) :
    comp fstC (tensorR f f') = comp f fstC := by
  simp only [comp]
  apply graph_ext
  intro u
  rw [fstC.app, tensorR_app, pairSeq_app_zero, fstC.app]

end DomainTheory.DataTypesAsLattices
```

eq_4_21.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.comp
public import DomainTheory.DataTypesAsLattices.Retracts.sndC
public import DomainTheory.DataTypesAsLattices.Retracts.tensorR
public import DomainTheory.DataTypesAsLattices.Retracts.graph_ext
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.pairSeq_app_one
public import DomainTheory.DataTypesAsLattices.Functionality.tensorR_app

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (4.21).** `snd ∘ (f ? f') = f' ∘ snd`. -/
theorem eq_4_21 (f f' : Pomega) :
    comp sndC (tensorR f f') = comp f' sndC := by
  simp only [comp]
  apply graph_ext
  intro u
  rw [sndC.app, tensorR_app, pairSeq_app_one, sndC.app]

end DomainTheory.DataTypesAsLattices
```

evalC_isScottContinuous.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Functionality.continuous_tuple
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem evalC_isScottContinuous :
    IsScottContinuous (fun u => funOf (funOf u (ofNat 0)) (funOf u (ofNat 1))) :=
  continuous_tuple
    (fun y => funOf.isScottContinuous_left y)
    (fun t => funOf.isScottContinuous t)
    (funOf.isScottContinuous_left (ofNat 0))
    (funOf.isScottContinuous_left (ofNat 1))

end DomainTheory.DataTypesAsLattices
```

evalC_app.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.evalC
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.evalC_isScottContinuous

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem evalC_app (u : Pomega) :
    funOf evalC u = funOf (funOf u (ofNat 0)) (funOf u (ofNat 1)) :=
  beta evalC_isScottContinuous u

end DomainTheory.DataTypesAsLattices
```

eq_4_24.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.comp
```

```
public import DomainTheory.DataTypesAsLattices.Retracts.Retract
public import DomainTheory.DataTypesAsLattices.Retracts.typed
public import DomainTheory.DataTypesAsLattices.Retracts.arrowR
public import DomainTheory.DataTypesAsLattices.Retracts.tensorR
public import DomainTheory.DataTypesAsLattices.Retracts.evalC
public import DomainTheory.DataTypesAsLattices.Retracts.retract_app
public import DomainTheory.DataTypesAsLattices.Retracts.graph_ext
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.pairSeq_app_zero
public import DomainTheory.DataTypesAsLattices.Functionality.pairSeq_app_one
public import DomainTheory.DataTypesAsLattices.Functionality.tensorR_app
public import DomainTheory.DataTypesAsLattices.Functionality.tensorR_isRetract
public import DomainTheory.DataTypesAsLattices.Functionality.evalC_app
public import DomainTheory.DataTypesAsLattices.Retracts.arrowR_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (4.24).** `eval ∘ ((b ∘→ c) ? b) : ((b ∘→ c) ? b) ∘→ c`. -/
theorem eq_4_24 {b c : Pomega} (hb : Retract b) (hc : Retract c) :
    typed (comp evalC (tensorR (arrowR b c) b))
      (arrowR (tensorR (arrowR b c) b) c) := by
  change comp evalC (tensorR (arrowR b c) b) =
    funOf (arrowR (tensorR (arrowR b c) b) c)
      (comp evalC (tensorR (arrowR b c) b))
  rw [arrowR.app]
  apply graph_ext
  intro u
  have hL :
      funOf evalC (funOf (tensorR (arrowR b c) b) u) =
        funOf c (funOf (funOf u (ofNat 0)) (funOf b (funOf u (ofNat 1)))) := by
    rw [evalC_app, tensorR_app, pairSeq_app_zero, pairSeq_app_one, arrowR.app]
    simp only [comp.app]
    rw [retract_app hb]
  have hR :
      funOf c (funOf evalC (funOf (tensorR (arrowR b c) b)
        (funOf (tensorR (arrowR b c) b) u))) =
        funOf c (funOf (funOf u (ofNat 0)) (funOf b (funOf u (ofNat 1)))) := by
    have hten := retract_app (tensorR_isRetract (arrowR.isRetract hb hc) hb) u
    rw [hten, hL, retract_app hc]
  simp only [comp.app]
  exact hL.trans hR.symm

end DomainTheory.DataTypesAsLattices
```

eq_4_25.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.comp
public import DomainTheory.DataTypesAsLattices.Retracts.Retract
public import DomainTheory.DataTypesAsLattices.Retracts.typed
```

```
public import DomainTheory.DataTypesAsLattices.Retracts.arrowR
public import DomainTheory.DataTypesAsLattices.Retracts.pairSeq
public import DomainTheory.DataTypesAsLattices.Retracts.tensorR
public import DomainTheory.DataTypesAsLattices.Retracts.curryC
public import DomainTheory.DataTypesAsLattices.Retracts.retract_app
public import DomainTheory.DataTypesAsLattices.Retracts.graph_ext
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.pairSeq_app_zero
public import DomainTheory.DataTypesAsLattices.Functionality.pairSeq_app_one
public import DomainTheory.DataTypesAsLattices.Functionality.
    pairSeq_isScottContinuous_left
public import DomainTheory.DataTypesAsLattices.Functionality.
    pairSeq_isScottContinuous_right
public import DomainTheory.DataTypesAsLattices.Functionality.tensorR_app
public import DomainTheory.DataTypesAsLattices.Functionality.tensorR_isRetract
public import DomainTheory.DataTypesAsLattices.Functionality.curryC_app
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
public import DomainTheory.DataTypesAsLattices.Retracts.arrowR_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (4.25).**
`curry ∘ ((a ? b) ∘→ c) : ((a ? b) ∘→ c) ∘→ (a ∘→ (b ∘→ c))`. -/
theorem eq_4_25 {a b c : Pomega}
    (ha : Retract a) (hb : Retract b) (hc : Retract c) :
    typed (comp curryC (arrowR (tensorR a b) c))
      (arrowR (arrowR (tensorR a b) c) (arrowR a (arrowR b c))) := by
  change comp curryC (arrowR (tensorR a b) c) =
    funOf (arrowR (arrowR (tensorR a b) c) (arrowR a (arrowR b c)))
      (comp curryC (arrowR (tensorR a b) c))
  rw [arrowR.app]
  have harr := arrowR.isRetract (tensorR_isRetract ha hb) hc
  apply graph_ext
  intro u
  simp only [comp.app]
  rw [retract_app harr]
  have hL :
      funOf curryC (funOf (arrowR (tensorR a b) c) u) =
        graph (fun x => graph (fun y =>
          funOf c (funOf u (pairSeq (funOf a x) (funOf b y))))) := by
    rw [curryC_app]
    apply graph_ext
    intro x
    apply graph_ext
    intro y
    rw [arrowR.app]
    simp only [comp.app]
    rw [tensorR_app, pairSeq_app_zero, pairSeq_app_one]
  have hR :
```

```
      funOf (arrowR a (arrowR b c))
        (funOf curryC (funOf (arrowR (tensorR a b) c) u)) =
        graph (fun x => graph (fun y =>
          funOf c (funOf u (pairSeq (funOf a x) (funOf b y))))) := by
    rw [arrowR.app, hL]
    apply graph_ext
    intro x
    simp only [comp.app]
    have hmid :
        funOf (graph (fun x => graph (fun y =>
          funOf c (funOf u (pairSeq (funOf a x) (funOf b y)))))) (funOf a x) =
          graph (fun y =>
            funOf c (funOf u (pairSeq (funOf a (funOf a x)) (funOf b y)))) :=
      beta (graph_const_isScottContinuous
        (fun x y => funOf c (funOf u (pairSeq (funOf a x) (funOf b y))))
        (fun y => theorem_1_3 (funOf.isScottContinuous c)
          (theorem_1_3 (funOf.isScottContinuous u)
            (theorem_1_3 (f := fun x => pairSeq x (funOf b y))
              (g := fun x => funOf a x)
              (pairSeq_isScottContinuous_left (funOf b y))
              (funOf.isScottContinuous a))))) (funOf a x)
    rw [hmid, retract_app ha, arrowR.app]
    apply graph_ext
    intro y
    simp only [comp.app]
    have hbeta :
        funOf (graph (fun y =>
          funOf c (funOf u (pairSeq (funOf a x) (funOf b y))))) (funOf b y) =
          funOf c (funOf u (pairSeq (funOf a x) (funOf b (funOf b y)))) :=
      beta (theorem_1_3 (funOf.isScottContinuous c)
        (theorem_1_3 (funOf.isScottContinuous u)
          (theorem_1_3 (f := fun y => pairSeq (funOf a x) y)
            (g := fun y => funOf b y)
            (pairSeq_isScottContinuous_right (funOf a x))
            (funOf.isScottContinuous b)))) (funOf b y)
    rw [hbeta, retract_app hb, retract_app hc]
  exact hL.trans hR.symm

end DomainTheory.DataTypesAsLattices
```

eq_4_32.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.comp
public import DomainTheory.DataTypesAsLattices.Retracts.Retract
public import DomainTheory.DataTypesAsLattices.Retracts.typed
public import DomainTheory.DataTypesAsLattices.Retracts.arrowR
public import DomainTheory.DataTypesAsLattices.Retracts.plusR
public import DomainTheory.DataTypesAsLattices.Retracts.retract_app
public import DomainTheory.DataTypesAsLattices.Retracts.graph_ext
public import DomainTheory.DataTypesAsLattices.Lambda.dcondSet
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.inleftC
```

```
public import DomainTheory.DataTypesAsLattices.Functionality.pairSeq_app_zero
public import DomainTheory.DataTypesAsLattices.Functionality.pairSeq_app_one
public import DomainTheory.DataTypesAsLattices.Functionality.plusR_app
public import DomainTheory.DataTypesAsLattices.Lambda.dcondSet_more
public import DomainTheory.DataTypesAsLattices.Retracts.arrowR_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (4.32).** ` (a ? b) ∘ inleft ∘ a : a ∘→ (a ? b) `. -/
theorem eq_4_32 {a b : Pomega} (ha : Retract a) :
    typed (comp (plusR a b) (comp inleftC a)) (arrowR a (plusR a b)) := by
  change comp (plusR a b) (comp inleftC a) =
    funOf (arrowR a (plusR a b)) (comp (plusR a b) (comp inleftC a))
  rw [arrowR.app]
  apply graph_ext
  intro x
  simp only [comp.app, inleftC.app, retract_app ha]
  rw [plusR_app.app, plusR_app.app, pairSeq_app_zero, pairSeq_app_one,
    dcondSet.ofNat_zero, retract_app ha]
  rw [pairSeq_app_zero, pairSeq_app_one, dcondSet.ofNat_zero, retract_app ha]

end DomainTheory.DataTypesAsLattices
```

inrightC_app.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.pairSeq
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.inrightC
public import DomainTheory.DataTypesAsLattices.Functionality.
    pairSeq_isScottContinuous_right

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem inrightC_app (x : Pomega) :
    funOf inrightC x = pairSeq (ofNat 1) x :=
  beta (pairSeq_isScottContinuous_right (ofNat 1)) x

end DomainTheory.DataTypesAsLattices
```

eq_4_33.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.comp
public import DomainTheory.DataTypesAsLattices.Retracts.Retract
```

```
public import DomainTheory.DataTypesAsLattices.Retracts.typed
public import DomainTheory.DataTypesAsLattices.Retracts.arrowR
public import DomainTheory.DataTypesAsLattices.Retracts.plusR
public import DomainTheory.DataTypesAsLattices.Retracts.retract_app
public import DomainTheory.DataTypesAsLattices.Retracts.graph_ext
public import DomainTheory.DataTypesAsLattices.Lambda.dcondSet
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.inrightC
public import DomainTheory.DataTypesAsLattices.Functionality.pairSeq_app_zero
public import DomainTheory.DataTypesAsLattices.Functionality.pairSeq_app_one
public import DomainTheory.DataTypesAsLattices.Functionality.plusR_app
public import DomainTheory.DataTypesAsLattices.Functionality.inrightC_app
public import DomainTheory.DataTypesAsLattices.Lambda.dcondSet_more
public import DomainTheory.DataTypesAsLattices.Retracts.arrowR_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (4.33).** `(a ? b) ∘ inright ∘ b : b o→ (a ? b) `. -/
theorem eq_4_33 {a b : Pomega} (hb : Retract b) :
    typed (comp (plusR a b) (comp inrightC b)) (arrowR b (plusR a b)) := by
  change comp (plusR a b) (comp inrightC b) =
    funOf (arrowR b (plusR a b)) (comp (plusR a b) (comp inrightC b))
  rw [arrowR.app]
  apply graph_ext
  intro x
  simp only [comp.app, inrightC_app, retract_app hb]
  rw [plusR_app.app, plusR_app.app, pairSeq_app_zero, pairSeq_app_one,
    dcondSet.ofNat_one, retract_app hb]
  rw [pairSeq_app_zero, pairSeq_app_one, dcondSet.ofNat_one, retract_app hb]

end DomainTheory.DataTypesAsLattices
```

outleftC.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Functionality.continuous_tuple
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section
```

```
namespace DomainTheory.DataTypesAsLattices

def outleftC : Pomega :=
  graph (fun u => condSet (funOf u (ofNat 0)) (funOf u (ofNat 1)) botElem)

namespace outleftC

theorem isScottContinuous :
    IsScottContinuous (fun u =>
      condSet (funOf u (ofNat 0)) (funOf u (ofNat 1)) botElem) :=
  continuous_tuple
    (fun y => condSet.isScottContinuous_left y botElem)
    (fun tes => condSet.isScottContinuous_mid tes botElem)
    (funOf.isScottContinuous_left (ofNat 0))
    (funOf.isScottContinuous_left (ofNat 1))

theorem app (u : Pomega) :
    funOf outleftC u = condSet (funOf u (ofNat 0)) (funOf u (ofNat 1)) botElem :=
  beta outleftC.isScottContinuous u

end outleftC

end DomainTheory.DataTypesAsLattices
```

eq_4_34.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.comp
public import DomainTheory.DataTypesAsLattices.Retracts.Retract
public import DomainTheory.DataTypesAsLattices.Retracts.typed
public import DomainTheory.DataTypesAsLattices.Retracts.arrowR
public import DomainTheory.DataTypesAsLattices.Retracts.plusR
public import DomainTheory.DataTypesAsLattices.Retracts.retract_app
public import DomainTheory.DataTypesAsLattices.Retracts.graph_ext
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.outleftC
public import DomainTheory.DataTypesAsLattices.Functionality.plusR_isRetract
public import DomainTheory.DataTypesAsLattices.Retracts.arrowR_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (4.34).** `a ∘ outleft ∘ (a ? b) : (a ? b) ∘→ a`. -/
theorem eq_4_34 {a b : Pomega} (ha : Retract a) (hb : Retract b) :
    typed (comp a (comp outleftC (plusR a b))) (arrowR (plusR a b) a) := by
  change comp a (comp outleftC (plusR a b)) =
    funOf (arrowR (plusR a b) a) (comp a (comp outleftC (plusR a b)))
  rw [arrowR.app]
  apply graph_ext
  intro u
  simp only [comp.app]
```

```
  rw [retract_app (plusR_isRetract ha hb), retract_app ha]

end DomainTheory.DataTypesAsLattices
```

outrightC.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Functionality.continuous_tuple
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def outrightC : Pomega :=
  graph (fun u => condSet (funOf u (ofNat 0)) botElem (funOf u (ofNat 1)))

namespace outrightC

theorem isScottContinuous :
    IsScottContinuous (fun u =>
      condSet (funOf u (ofNat 0)) botElem (funOf u (ofNat 1))) :=
  continuous_tuple
    (fun y => condSet.isScottContinuous_left botElem y)
    (fun tes => condSet.isScottContinuous_right tes botElem)
    (funOf.isScottContinuous_left (ofNat 0))
    (funOf.isScottContinuous_left (ofNat 1))

theorem app (u : Pomega) :
    funOf outrightC u = condSet (funOf u (ofNat 0)) botElem (funOf u (ofNat 1)) :=
  beta outrightC.isScottContinuous u

end outrightC

end DomainTheory.DataTypesAsLattices
```

eq_4_35.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.comp
public import DomainTheory.DataTypesAsLattices.Retracts.Retract
```

```
public import DomainTheory.DataTypesAsLattices.Retracts.typed
public import DomainTheory.DataTypesAsLattices.Retracts.arrowR
public import DomainTheory.DataTypesAsLattices.Retracts.plusR
public import DomainTheory.DataTypesAsLattices.Retracts.retract_app
public import DomainTheory.DataTypesAsLattices.Retracts.graph_ext
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.outrightC
public import DomainTheory.DataTypesAsLattices.Functionality.plusR_isRetract
public import DomainTheory.DataTypesAsLattices.Retracts.arrowR_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (4.35).** `b ∘ outright ∘ (a ? b) : (a ? b) ∘→ b`. -/
theorem eq_4_35 {a b : Pomega} (ha : Retract a) (hb : Retract b) :
    typed (comp b (comp outrightC (plusR a b))) (arrowR (plusR a b) b) := by
  change comp b (comp outrightC (plusR a b)) =
    funOf (arrowR (plusR a b) b) (comp b (comp outrightC (plusR a b)))
  rw [arrowR.app]
  apply graph_ext
  intro u
  simp only [comp.app]
  rw [retract_app (plusR_isRetract ha hb), retract_app hb]

end DomainTheory.DataTypesAsLattices
```

funOf_condSet.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem funOf_condSet (z x y w : Pomega) :
    funOf (condSet z x y) w = condSet z (funOf x w) (funOf y w) := by
  ext m
  constructor
  · intro ⟨n, hn, hp⟩
    rcases hp with ⟨hx, h0⟩ | ⟨hy, ht⟩
    · exact Or.inl ⟨⟨n, hn, hx⟩, h0⟩
    · exact Or.inr ⟨⟨n, hn, hy⟩, ht⟩
  · intro hm
    rcases hm with ⟨⟨n, hn, hx⟩, h0⟩ | ⟨⟨n, hn, hy⟩, ht⟩
    · exact ⟨n, hn, Or.inl ⟨hx, h0⟩⟩
    · exact ⟨n, hn, Or.inr ⟨hy, ht⟩⟩

end DomainTheory.DataTypesAsLattices
```

funOf_dcondSet.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.dcondSet
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.funOf_top
public import DomainTheory.DataTypesAsLattices.Functionality.funOf_condSet

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem funOf_dcondSet (z x y w : Pomega) :
    funOf (dcondSet z x y) w = dcondSet z (funOf x w) (funOf y w) := by
  simp only [dcondSet]
  rw [funOf_condSet, funOf_condSet, funOf_condSet, funOf_top]

end DomainTheory.DataTypesAsLattices
```

plusR_fst.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.plusR
public import DomainTheory.DataTypesAsLattices.Lambda.dcondSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.pairSeq_app_zero
public import DomainTheory.DataTypesAsLattices.Functionality.plusR_app
public import DomainTheory.DataTypesAsLattices.Functionality.funOf_dcondSet

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem plusR_fst (a b u : Pomega) :
    funOf (funOf (plusR a b) u) (ofNat 0) =
      dcondSet (funOf u (ofNat 0)) (ofNat 0) (ofNat 1) := by
  rw [plusR_app.app, funOf_dcondSet, pairSeq_app_zero, pairSeq_app_zero]

end DomainTheory.DataTypesAsLattices
```

eq_4_36.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.comp
public import DomainTheory.DataTypesAsLattices.Retracts.typed
public import DomainTheory.DataTypesAsLattices.Retracts.arrowR
public import DomainTheory.DataTypesAsLattices.Retracts.plusR
public import DomainTheory.DataTypesAsLattices.Retracts.boolR
```

```
public import DomainTheory.DataTypesAsLattices.Retracts.fstC
public import DomainTheory.DataTypesAsLattices.Retracts.graph_ext
public import DomainTheory.DataTypesAsLattices.Retracts.dcondSet_bool_idem
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.whichC
public import DomainTheory.DataTypesAsLattices.Functionality.plusR_fst
public import DomainTheory.DataTypesAsLattices.Retracts.arrowR_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (4.36).** `which ∘ (a ? b) : (a ? b) ∘→ bool`. -/
theorem eq_4_36 (a b : Pomega) :
    typed (comp whichC (plusR a b)) (arrowR (plusR a b) boolR) := by
  change comp whichC (plusR a b) =
    funOf (arrowR (plusR a b) boolR) (comp whichC (plusR a b))
  rw [arrowR.app]
  apply graph_ext
  intro u
  simp only [comp.app, whichC, fstC.app]
  rw [plusR_fst, boolR.app, plusR_fst, plusR_fst, dcondSet_bool_idem,
    dcondSet_bool_idem]

end DomainTheory.DataTypesAsLattices
```

eq_4_37.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.comp
public import DomainTheory.DataTypesAsLattices.Retracts.Retract
public import DomainTheory.DataTypesAsLattices.Retracts.typed
public import DomainTheory.DataTypesAsLattices.Retracts.arrowR
public import DomainTheory.DataTypesAsLattices.Retracts.plusR
public import DomainTheory.DataTypesAsLattices.Retracts.retract_app
public import DomainTheory.DataTypesAsLattices.Retracts.graph_ext
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.outC
public import DomainTheory.DataTypesAsLattices.Functionality.plusR_isRetract
public import DomainTheory.DataTypesAsLattices.Retracts.arrowR_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (4.37).** `a ∘ out ∘ (a ? a) : (a ? a) ∘→ a`. -/
theorem eq_4_37 {a : Pomega} (ha : Retract a) :
    typed (comp a (comp outC (plusR a a))) (arrowR (plusR a a) a) := by
  change comp a (comp outC (plusR a a)) =
    funOf (arrowR (plusR a a) a) (comp a (comp outC (plusR a a)))
  rw [arrowR.app]
  apply graph_ext
```

```
  intro u
  simp only [comp.app]
  rw [retract_app (plusR_isRetract ha ha), retract_app ha]

end DomainTheory.DataTypesAsLattices
```

eq_4_39.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.arrowR
public import DomainTheory.DataTypesAsLattices.Retracts.plusR
public import DomainTheory.DataTypesAsLattices.Retracts.intR
public import DomainTheory.DataTypesAsLattices.Retracts.lambF
public import DomainTheory.DataTypesAsLattices.Retracts.lambR
public import DomainTheory.DataTypesAsLattices.Lambda.theorem_2_5
public import DomainTheory.DataTypesAsLattices.FixedPoint.fix
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_4
public import DomainTheory.DataTypesAsLattices.Functionality.lambF_isScottContinuous

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (4.39).** `lamb = int ? (lamb o→ lamb)`. -/
theorem eq_4_39 : lambR = plusR intR (arrowR lambR lambR) := by
  have hY : lambR = fix lambF := theorem_2_5 lambF_isScottContinuous
  have hfix : lambF lambR = lambR := by
    rw [hY]
    exact (theorem_1_4 lambF_isScottContinuous).1
  exact hfix.symm

end DomainTheory.DataTypesAsLattices
```

eq_4_45.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.typed
public import DomainTheory.DataTypesAsLattices.Retracts.arrowR
public import DomainTheory.DataTypesAsLattices.Retracts.lambR
public import DomainTheory.DataTypesAsLattices.Retracts.envR
public import DomainTheory.DataTypesAsLattices.Retracts.expR
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Functionality.LambTerm
public import DomainTheory.DataTypesAsLattices.Functionality.encodeExp
public import DomainTheory.DataTypesAsLattices.Functionality.Htyped
public import DomainTheory.DataTypesAsLattices.Functionality.Hcomb
public import DomainTheory.DataTypesAsLattices.Functionality.encodeExp_typed
public import DomainTheory.DataTypesAsLattices.Functionality.Hfun

@[expose] public section

namespace DomainTheory.DataTypesAsLattices
```

```
/-- **Scott 1976, (4.45).** Encoded terms live in `exp`, `??τ?` is typed
`env → lamb`, and the continuous `?` decodes `encodeExp τ` to `??τ?`. -/
theorem eq_4_45 (τ : LambTerm) :
    typed (encodeExp τ) expR ∧
      typed (Htyped τ) (arrowR envR lambR) ∧
        funOf Hcomb (encodeExp τ) = Htyped τ := by
  refine ⟨encodeExp_typed τ, Htyped.typed τ, ?_⟩
  rw [Hcomb.app, ← encodeExp_typed τ, Hfun.decode]
  rfl

end DomainTheory.DataTypesAsLattices
```

eq_7_19.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.eq_2_9
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.Z
public import DomainTheory.DataTypesAsLattices.Functionality.sigmaJ
public import DomainTheory.DataTypesAsLattices.Functionality.eq_7_18_eq

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (7.19).**
Independent successor powers advance both tagged atoms in lockstep. -/
theorem eq_7_19 (j j' m : ℕ) :
    Z m (graph (sigmaJ j) ∪ graph (sigmaJ j'))
        (({pair j 0} : Pomega) ∪ {pair j' 0}) =
      ({pair j m} : Pomega) ∪ {pair j' m} := by
  by_cases h : j = j'
  · subst j'
    simp only [Set.union_self]
    exact eq_7_18_eq j m
  · induction m with
    | zero => rfl
    | succ m ih =>
      rw [Z, eq_2_9, ih, sigmaJ.app, sigmaJ.app,
        sigmaJ.pair_union j j' m h]
      rw [show sigmaJ j' (({pair j m} : Pomega) ∪ {pair j' m}) =
          ({pair j' (m + 1)} : Pomega) by
        simpa [Set.union_comm] using sigmaJ.pair_union j' j m (Ne.symm h)]

end DomainTheory.DataTypesAsLattices
```

eraseTag.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Pomega.e
```

```
public import DomainTheory.DataTypesAsLattices.Functionality.Z

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def eraseTag (j : ℕ) (u : Pomega) : Pomega :=
  {p | p ∈ u ∧ ∀ k, p = pair j k}

namespace eraseTag

theorem Z_finite (p q m : ℕ) :
    eraseTag (max p q + 1) (Z m (e p) (e q)) = Z m (e p) (e q) := by
  ext k
  constructor
  · exact fun hk => hk.1
  · intro hk
    exact ⟨hk, fun r hkr => Z.finite_no_tag p q m r (hkr ▸ hk)⟩

theorem Z_finite_union (p q m : ℕ) :
    eraseTag (max p q + 1)
        (Z m (e p) (e q) ∪ {pair (max p q + 1) m}) =
      Z m (e p) (e q) := by
  ext k
  constructor
  · intro ⟨hk, hnot⟩
    rcases hk with hk | hk
    · exact hk
    · exact (hnot m (by simpa using hk)).elim
  · intro hk
    exact ⟨Or.inl hk, fun r hkr => Z.finite_no_tag p q m r (hkr ▸ hk)⟩

end eraseTag

end DomainTheory.DataTypesAsLattices
```

eta2_map_isScottContinuous.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more

@[expose] public section
```

```
namespace DomainTheory.DataTypesAsLattices

theorem eta2_map_isScottContinuous (f : Pomega) :
    IsScottContinuous (fun x => graph (fun y => funOf (funOf f x) y)) :=
  graph_const_isScottContinuous
    (fun x y => funOf (funOf f x) y)
    (fun y =>
      theorem_1_3
        (f := fun t => funOf t y)
        (g := funOf f)
        (funOf.isScottContinuous_left y)
        (funOf.isScottContinuous f))

end DomainTheory.DataTypesAsLattices
```

eta2_app.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.
    eta2_map_isScottContinuous

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem eta2_app (f x : Pomega) :
    funOf (graph (fun z => graph (fun y => funOf (funOf f z) y))) x =
      graph (fun y => funOf (funOf f x) y) :=
  beta (eta2_map_isScottContinuous f) x

end DomainTheory.DataTypesAsLattices
```

graph_funOf_ext.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.graph_ext
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem graph_funOf_ext {f g : Pomega}
    (hf : f = graph (fun x => funOf f x))
    (hg : g = graph (fun x => funOf g x))
    (h : ∀ x, funOf f x = funOf g x) : f = g := by
  rw [hf, hg]
```

```
  exact graph_ext h

end DomainTheory.DataTypesAsLattices
```

iterateE.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.RestrictedEquiv
public import DomainTheory.DataTypesAsLattices.Functionality.Z

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- The identity on the orbit `{f?(x) | n ∈ ω}`. -/
def iterateE (f x : Pomega) : RestrictedEquiv where
  rel := fun u v => (∃ n, u = Z n f x) ∧ u = v
  symm := by
    intro u v ⟨hn, h⟩
    exact ⟨by rw [← h]; exact hn, h.symm⟩
  trans := by
    intro u v w ⟨hn, hxy⟩ ⟨_, hyz⟩
    exact ⟨hn, hxy.trans hyz⟩

end DomainTheory.DataTypesAsLattices
```

kerE.lean

```
module

public import DomainTheory.DataTypesAsLattices.Functionality.RestrictedEquiv
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.Retract
public import DomainTheory.DataTypesAsLattices.Retracts.retract_app

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def kerE (a : Pomega) : RestrictedEquiv where
  rel := fun x y => funOf a x = funOf a y
  symm := by
    intro x y h
    exact h.symm
  trans := by
    intro x y z hxy hyz
    exact hxy.trans hyz

namespace kerE

theorem repr {a : Pomega} (ha : Retract a) (x : Pomega) :
```

```
    (kerE a).rel x (funOf a x) :=
  (retract_app ha x).symm

end kerE

end DomainTheory.DataTypesAsLattices
```

pairMed.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.comp
public import DomainTheory.DataTypesAsLattices.Retracts.tensorR
public import DomainTheory.DataTypesAsLattices.Retracts.diagC
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, Theorem 4.4.** The product mediator
`h = (f ? g) ∘ diag ∘ c`. -/
def pairMed (f g c : Pomega) : Pomega :=
  comp (tensorR f g) (comp diagC c)

end DomainTheory.DataTypesAsLattices
```

plotkinGood.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.Z

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def plotkinGood (p q j : ℕ) (u : Pomega) : Prop :=
  ∃ m, u = Z m (e p) (e q) ∨
    u = Z m (e p) (e q) ∪ {pair j m}

end DomainTheory.DataTypesAsLattices
```

plotkinRel.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Pomega.e
```

```
public import DomainTheory.DataTypesAsLattices.Functionality.Z

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def plotkinRel (p q j : ℕ) (u v : Pomega) : Prop :=
  ∃ m, (u = Z m (e p) (e q) ∧
      v = Z m (e p) (e q) ∪ {pair j m}) ∨
    (v = Z m (e p) (e q) ∧
      u = Z m (e p) (e q) ∪ {pair j m})

end DomainTheory.DataTypesAsLattices
```

plotkinE.lean

```
module

public import DomainTheory.DataTypesAsLattices.Functionality.RestrictedEquiv
public import DomainTheory.DataTypesAsLattices.Functionality.eraseTag
public import DomainTheory.DataTypesAsLattices.Functionality.plotkinGood
public import DomainTheory.DataTypesAsLattices.Functionality.plotkinRel
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def plotkinE (p q j : ℕ) : RestrictedEquiv where
  rel u v :=
    plotkinGood p q j u ∧ plotkinGood p q j v ∧
      Relation.EqvGen (plotkinRel p q j) u v
  symm := by
    intro u v ⟨hu, hv, huv⟩
    exact ⟨hv, hu, Relation.EqvGen.symm _ _ huv⟩
  trans := by
    intro u v w ⟨hu, _hv, huv⟩ ⟨_hv', hw, hvw⟩
    exact ⟨hu, hw, Relation.EqvGen.trans _ _ _ huv hvw⟩

namespace plotkinE

theorem erase {p q : ℕ} {u v : Pomega}
    (h : (plotkinE p q (max p q + 1)).rel u v) :
    eraseTag (max p q + 1) u = eraseTag (max p q + 1) v := by
  rcases h with ⟨_hu, _hv, h⟩
  clear _hu _hv
  induction h with
  | rel x y hxy =>
      rcases hxy with ⟨m, ⟨rfl, rfl⟩ | ⟨rfl, rfl⟩⟩
      · rw [eraseTag.Z_finite, eraseTag.Z_finite_union]
      · rw [eraseTag.Z_finite, eraseTag.Z_finite_union]
  | refl => rfl
  | symm _ _ _ ih => exact ih.symm
  | trans _ _ _ _ _ ih₁ ih₂ => exact ih₁.trans ih₂
```

```
end plotkinE

end DomainTheory.DataTypesAsLattices
```

plotkin_finite_separation.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.eq_2_9
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.Z
public import DomainTheory.DataTypesAsLattices.Functionality.sigmaJ
public import DomainTheory.DataTypesAsLattices.Functionality.funOf_e_union_fresh
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem plotkin_finite_separation (p q m : ℕ) :
    Z m
        (graph (fun y => funOf (e p) y) ∪
          graph (sigmaJ (max p q + 1)))
        ((e q) ∪ {pair (max p q + 1) 0}) =
      Z m (e p) (e q) ∪ {pair (max p q + 1) m} := by
  induction m with
  | zero => rfl
  | succ m ih =>
      rw [Z, eq_2_9, ih]
      have hbeta :
          funOf (graph (fun y => funOf (e p) y))
              (Z m (e p) (e q) ∪ {pair (max p q + 1) m}) =
            funOf (e p) (Z m (e p) (e q) ∪ {pair (max p q + 1) m}) :=
        beta (funOf.isScottContinuous (e p)) _
      rw [hbeta, funOf_e_union_fresh p (max p q + 1) m _
        (Nat.lt_succ_of_le (Nat.le_max_left p q)), sigmaJ.app,
        sigmaJ.union_fresh (fun k => Z.finite_no_tag p q m k)]
      rfl

end DomainTheory.DataTypesAsLattices
```

plotkin_function_related.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.graph_ext
public import DomainTheory.DataTypesAsLattices.Lambda.eq_2_9
public import DomainTheory.DataTypesAsLattices.Lambda.eq_2_10
public import DomainTheory.DataTypesAsLattices.Graph.beta
```

```
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.arrowE
public import DomainTheory.DataTypesAsLattices.Functionality.Z
public import DomainTheory.DataTypesAsLattices.Functionality.sigmaJ
public import DomainTheory.DataTypesAsLattices.Functionality.plotkinE
public import DomainTheory.DataTypesAsLattices.Functionality.eraseTag
public import DomainTheory.DataTypesAsLattices.Functionality.normalized_finite_app
public import DomainTheory.DataTypesAsLattices.Functionality.
    normalized_finite_fresh_app
public import DomainTheory.DataTypesAsLattices.Functionality.finite_sigma_union_app
public import DomainTheory.DataTypesAsLattices.Functionality.finite_sigma_plain_app
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem plotkin_function_related (p q : ℕ) :
    (arrowE
      (plotkinE p q (max p q + 1))
      (plotkinE p q (max p q + 1))).rel
        (graph (fun y => funOf (e p) y))
        (graph (fun y => funOf (e p) y) ∪
          graph (sigmaJ (max p q + 1))) := by
  have hgf : graph (fun y => funOf (e p) y) =
      graph (fun y => funOf (graph (fun y => funOf (e p) y)) y) := by
    apply graph_ext
    intro y
    exact (beta (funOf.isScottContinuous (e p)) y).symm
  have hgu :
      graph (fun y => funOf (e p) y) ∪ graph (sigmaJ (max p q + 1)) =
        graph (fun y => funOf
          (graph (fun y => funOf (e p) y) ∪
            graph (sigmaJ (max p q + 1))) y) := by
    let F := fun y => funOf (e p) y ∪ sigmaJ (max p q + 1) y
    have hrep :
        graph (fun y => funOf (e p) y) ∪ graph (sigmaJ (max p q + 1)) =
          graph F := eq_2_10 _ _
    rw [hrep]
    congr 1
    funext y
    dsimp only [F]
    rw [← hrep]
    rw [eq_2_9, normalized_finite_app, sigmaJ.app]
  refine ⟨hgf, hgu, ?_⟩
  intro u v huv
  have herase := plotkinE.erase huv
  rcases huv.1 with ⟨r, hur | hur⟩
  <;> rcases huv.2.1 with ⟨s, hvs | hvs⟩
  all_goals
    subst u
    subst v
  · have hrs : Z r (e p) (e q) = Z s (e p) (e q) := by
```

```
    simpa [eraseTag.Z_finite] using herase
  have hrs' : Z (r + 1) (e p) (e q) = Z (s + 1) (e p) (e q) := by
    simpa [Z] using congrArg (fun w => funOf (e p) w) hrs
  have hleft :
      funOf (graph (fun y => funOf (e p) y)) (Z r (e p) (e q)) =
        Z (r + 1) (e p) (e q) := by
    rw [normalized_finite_app]
    rfl
  have hright :
      funOf
          (graph (fun y => funOf (e p) y) ∪
            graph (sigmaJ (max p q + 1)))
          (Z s (e p) (e q)) = Z (s + 1) (e p) (e q) :=
    finite_sigma_plain_app p q s
  rw [hleft, hright, ← hrs']
  exact ⟨⟨r + 1, Or.inl rfl⟩, ⟨r + 1, Or.inl rfl⟩,
    Relation.EqvGen.refl _⟩
· have hrs : Z r (e p) (e q) = Z s (e p) (e q) := by
    exact (eraseTag.Z_finite p q r).symm.trans
      (herase.trans (eraseTag.Z_finite_union p q s))
  have hrs' : Z (r + 1) (e p) (e q) = Z (s + 1) (e p) (e q) := by
    simpa [Z] using congrArg (fun w => funOf (e p) w) hrs
  have hleft :
      funOf (graph (fun y => funOf (e p) y)) (Z r (e p) (e q)) =
        Z (r + 1) (e p) (e q) := by
    rw [normalized_finite_app]
    rfl
  have hright :
      funOf
          (graph (fun y => funOf (e p) y) ∪
            graph (sigmaJ (max p q + 1)))
          (Z s (e p) (e q) ∪ {pair (max p q + 1) s}) =
        Z (s + 1) (e p) (e q) ∪
          {pair (max p q + 1) (s + 1)} :=
    finite_sigma_union_app p q s
  rw [hleft, hright, hrs']
  exact ⟨⟨s + 1, Or.inl rfl⟩, ⟨s + 1, Or.inr rfl⟩,
    Relation.EqvGen.rel _ _
      ⟨s + 1, Or.inl ⟨rfl, rfl⟩⟩⟩
· have hrs : Z r (e p) (e q) = Z s (e p) (e q) := by
    exact (eraseTag.Z_finite_union p q r).symm.trans
      (herase.trans (eraseTag.Z_finite p q s))
  have hrs' : Z (r + 1) (e p) (e q) = Z (s + 1) (e p) (e q) := by
    simpa [Z] using congrArg (fun w => funOf (e p) w) hrs
  have hleft :
      funOf (graph (fun y => funOf (e p) y))
          (Z r (e p) (e q) ∪ {pair (max p q + 1) r}) =
        Z (r + 1) (e p) (e q) :=
    normalized_finite_fresh_app p q r
  have hright :
      funOf
          (graph (fun y => funOf (e p) y) ∪
            graph (sigmaJ (max p q + 1)))
          (Z s (e p) (e q)) = Z (s + 1) (e p) (e q) :=
    finite_sigma_plain_app p q s
  rw [hleft, hright, ← hrs']
```

```
    exact ⟨⟨r + 1, Or.inl rfl⟩, ⟨r + 1, Or.inl rfl⟩,
      Relation.EqvGen.refl _⟩
  · have hrs : Z r (e p) (e q) = Z s (e p) (e q) := by
      simpa using (eraseTag.Z_finite_union p q r).symm.trans
        (herase.trans (eraseTag.Z_finite_union p q s))
    have hrs' : Z (r + 1) (e p) (e q) = Z (s + 1) (e p) (e q) := by
      simpa [Z] using congrArg (fun w => funOf (e p) w) hrs
    have hleft :
        funOf (graph (fun y => funOf (e p) y))
            (Z r (e p) (e q) ∪ {pair (max p q + 1) r}) =
          Z (r + 1) (e p) (e q) :=
      normalized_finite_fresh_app p q r
    have hright :
        funOf
            (graph (fun y => funOf (e p) y) ∪
              graph (sigmaJ (max p q + 1)))
            (Z s (e p) (e q) ∪ {pair (max p q + 1) s}) =
          Z (s + 1) (e p) (e q) ∪
            {pair (max p q + 1) (s + 1)} :=
      finite_sigma_union_app p q s
    rw [hleft, hright, hrs']
    exact ⟨⟨s + 1, Or.inl rfl⟩, ⟨s + 1, Or.inr rfl⟩,
      Relation.EqvGen.rel _ _
        ⟨s + 1, Or.inl ⟨rfl, rfl⟩⟩⟩

end DomainTheory.DataTypesAsLattices
```

plusR_isStrict.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.plusR
public import DomainTheory.DataTypesAsLattices.Retracts.IsStrict
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.plusR_app

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem plusR_isStrict (a b : Pomega) : IsStrict (plusR a b) :=
  plusR_app.bot a b

end DomainTheory.DataTypesAsLattices
```

plusR_typed_bot.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.typed
public import DomainTheory.DataTypesAsLattices.Retracts.plusR
public import DomainTheory.DataTypesAsLattices.Lambda.dcondSet
public import DomainTheory.DataTypesAsLattices.Graph.funOf
```

```
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.plusR_app
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
public import DomainTheory.DataTypesAsLattices.Lambda.dcondSet_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem plusR_typed_bot (a b : Pomega) : typed botElem (plusR a b) := by
  rw [typed, plusR_app.app, funOf.bot, dcondSet.bot]

end DomainTheory.DataTypesAsLattices
```

plusR_typed_inl.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.typed
public import DomainTheory.DataTypesAsLattices.Retracts.pairSeq
public import DomainTheory.DataTypesAsLattices.Retracts.plusR
public import DomainTheory.DataTypesAsLattices.Lambda.dcondSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.pairSeq_app_zero
public import DomainTheory.DataTypesAsLattices.Functionality.pairSeq_app_one
public import DomainTheory.DataTypesAsLattices.Functionality.plusR_app
public import DomainTheory.DataTypesAsLattices.Lambda.dcondSet_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem plusR_typed_inl {a b x : Pomega} (hx : typed x a) :
    typed (pairSeq (ofNat 0) x) (plusR a b) := by
  rw [typed, plusR_app.app, pairSeq_app_zero, pairSeq_app_one, dcondSet.ofNat_zero,
    ← hx]

end DomainTheory.DataTypesAsLattices
```

plusR_typed_inr.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.typed
public import DomainTheory.DataTypesAsLattices.Retracts.pairSeq
public import DomainTheory.DataTypesAsLattices.Retracts.plusR
public import DomainTheory.DataTypesAsLattices.Lambda.dcondSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.pairSeq_app_zero
public import DomainTheory.DataTypesAsLattices.Functionality.pairSeq_app_one
public import DomainTheory.DataTypesAsLattices.Functionality.plusR_app
```

```
public import DomainTheory.DataTypesAsLattices.Lambda.dcondSet_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem plusR_typed_inr {a b y : Pomega} (hy : typed y b) :
    typed (pairSeq (ofNat 1) y) (plusR a b) := by
  rw [typed, plusR_app.app, pairSeq_app_zero, pairSeq_app_one, dcondSet.ofNat_one,
    ← hy]

end DomainTheory.DataTypesAsLattices
```

plusR_typed_top.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.typed
public import DomainTheory.DataTypesAsLattices.Retracts.plusR
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.plusR_app

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem plusR_typed_top (a b : Pomega) : typed topElem (plusR a b) :=
  (plusR_app.top a b).symm

end DomainTheory.DataTypesAsLattices
```

plusR_typed_iff.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.Retract
public import DomainTheory.DataTypesAsLattices.Retracts.typed
public import DomainTheory.DataTypesAsLattices.Retracts.pairSeq
public import DomainTheory.DataTypesAsLattices.Retracts.plusR
public import DomainTheory.DataTypesAsLattices.Retracts.retract_app
public import DomainTheory.DataTypesAsLattices.Lambda.dcondSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.plusR_app
public import DomainTheory.DataTypesAsLattices.Functionality.plusR_typed_bot
public import DomainTheory.DataTypesAsLattices.Functionality.plusR_typed_inl
public import DomainTheory.DataTypesAsLattices.Functionality.plusR_typed_inr
public import DomainTheory.DataTypesAsLattices.Functionality.plusR_typed_top
public import DomainTheory.DataTypesAsLattices.Lambda.dcondSet_more

@[expose] public section
```

```
namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, Theorem 4.5 (ii).** Range of `a ? b`. -/
theorem plusR_typed_iff {a b u : Pomega} (ha : Retract a) (hb : Retract b) :
    typed u (plusR a b) ↔
      u = botElem ∨ u = topElem ∨
        (∃ x, u = pairSeq (ofNat 0) x ∧ typed x a) ∨
          (∃ y, u = pairSeq (ofNat 1) y ∧ typed y b) := by
  constructor
  · intro hu
    have hu' : u =
        dcondSet (funOf u (ofNat 0))
          (pairSeq (ofNat 0) (funOf a (funOf u (ofNat 1))))
          (pairSeq (ofNat 1) (funOf b (funOf u (ofNat 1)))) := by
      simpa [typed, plusR_app.app] using hu
    rcases dcondSet.cases (funOf u (ofNat 0)) with h | h | h | h
    · refine Or.inr (Or.inr (Or.inl ⟨funOf a (funOf u (ofNat 1)), ?_, ?_⟩))
      · exact hu'.trans (dcondSet.of_zero_not_pos h.1 h.2)
      · exact (retract_app ha (funOf u (ofNat 1))).symm
    · refine Or.inr (Or.inr (Or.inr ⟨funOf b (funOf u (ofNat 1)), ?_, ?_⟩))
      · exact hu'.trans (dcondSet.of_pos_not_zero h.1 h.2)
      · exact (retract_app hb (funOf u (ofNat 1))).symm
    · refine Or.inr (Or.inl ?_)
      exact hu'.trans (dcondSet.of_mixed h.1 h.2)
    · refine Or.inl ?_
      exact hu'.trans (dcondSet.of_empty h.1 h.2)
  · intro h
    rcases h with hu | hu | ⟨x, hx, hxt⟩ | ⟨y, hy, hyt⟩
    · rw [hu]; exact plusR_typed_bot a b
    · rw [hu]; exact plusR_typed_top a b
    · rw [hx]; exact plusR_typed_inl hxt
    · rw [hy]; exact plusR_typed_inr hyt

end DomainTheory.DataTypesAsLattices
```

prodE.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Functionality.RestrictedEquiv
public import DomainTheory.DataTypesAsLattices.Functionality.pairElem

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (7.7).** Product of restricted equivalences. -/
def prodE (A B : RestrictedEquiv) : RestrictedEquiv where
  rel := fun u v =>
    u = pairElem (funOf u (ofNat 0)) (funOf u (ofNat 1)) ∧
      v = pairElem (funOf v (ofNat 0)) (funOf v (ofNat 1)) ∧
        A.rel (funOf u (ofNat 0)) (funOf v (ofNat 0)) ∧
```

```
          B.rel (funOf u (ofNat 1)) (funOf v (ofNat 1))
  symm := by
    intro u v ⟨hu, hv, h0, h1⟩
    exact ⟨hv, hu, A.symm h0, B.symm h1⟩
  trans := by
    intro u v w ⟨hu, hv, h0, h1⟩ ⟨hv', hw, k0, k1⟩
    exact ⟨hu, hw, A.trans h0 k0, B.trans h1 k1⟩

end DomainTheory.DataTypesAsLattices
```

singletonE.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.RestrictedEquiv

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- Singleton identity `{⟨a, a⟩}`. -/
def singletonE (a : Pomega) : RestrictedEquiv where
  rel := fun x y => x = a ∧ y = a
  symm := by
    intro x y ⟨hx, hy⟩
    exact ⟨hy, hx⟩
  trans := by
    intro x y z ⟨hx, _hy⟩ ⟨_hy', hz⟩
    exact ⟨hx, hz⟩

end DomainTheory.DataTypesAsLattices
```

singletonE_mem.lean

```
module

public import DomainTheory.DataTypesAsLattices.Embedding.extend
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.mem
public import DomainTheory.DataTypesAsLattices.Functionality.singletonE

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem singletonE_mem (a x : Pomega) : (singletonE a).mem x ↔ x = a :=
  ⟨fun h => h.1, fun hx => ⟨hx, hx⟩⟩

end DomainTheory.DataTypesAsLattices
```

typed_isGraph.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.comp
public import DomainTheory.DataTypesAsLattices.Retracts.typed
public import DomainTheory.DataTypesAsLattices.Retracts.arrowR
public import DomainTheory.DataTypesAsLattices.Retracts.graph_ext
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.arrowR_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem typed_isGraph {a b f : Pomega} (hf : typed f (arrowR a b)) :
    f = graph (fun x => funOf f x) := by
  have hf' := arrowR.typed_of hf
  apply Eq.trans hf'
  apply graph_ext
  intro x
  have : funOf f x = funOf b (funOf f (funOf a x)) := by
    have h1 : funOf f x = funOf (comp b (comp f a)) x :=
      congrArg (fun w => funOf w x) hf'
    rw [h1, comp.app, comp.app]
  rw [comp.app, this]

end DomainTheory.DataTypesAsLattices
```

theorem_4_4_med.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.comp
public import DomainTheory.DataTypesAsLattices.Retracts.Retract
public import DomainTheory.DataTypesAsLattices.Retracts.typed
public import DomainTheory.DataTypesAsLattices.Retracts.arrowR
public import DomainTheory.DataTypesAsLattices.Retracts.fstC
public import DomainTheory.DataTypesAsLattices.Retracts.sndC
public import DomainTheory.DataTypesAsLattices.Retracts.pairSeq
public import DomainTheory.DataTypesAsLattices.Retracts.tensorR
public import DomainTheory.DataTypesAsLattices.Retracts.diagC
public import DomainTheory.DataTypesAsLattices.Retracts.retract_app
public import DomainTheory.DataTypesAsLattices.Retracts.graph_ext
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.pairMed
public import DomainTheory.DataTypesAsLattices.Functionality.pairSeq_app_zero
public import DomainTheory.DataTypesAsLattices.Functionality.pairSeq_app_one
public import DomainTheory.DataTypesAsLattices.Functionality.tensorR_app
public import DomainTheory.DataTypesAsLattices.Functionality.diagC_app
public import DomainTheory.DataTypesAsLattices.Functionality.graph_funOf_ext
public import DomainTheory.DataTypesAsLattices.Functionality.typed_isGraph
```

```
public import DomainTheory.DataTypesAsLattices.Functionality.comp_isGraph
public import DomainTheory.DataTypesAsLattices.Retracts.arrowR_more
public import DomainTheory.DataTypesAsLattices.Retracts.typed_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, Theorem 4.4.** Existence of the product mediator. -/
theorem theorem_4_4_med {a b c f g : Pomega}
    (_ha : Retract a) (_hb : Retract b) (hc : Retract c)
    (hf : typed f (arrowR c a)) (hg : typed g (arrowR c b)) :
    typed (pairMed f g c) (arrowR c (tensorR a b)) ∧
      comp fstC (pairMed f g c) = f ∧
      comp sndC (pairMed f g c) = g := by
  have hf' := arrowR.typed_of hf
  have hg' := arrowR.typed_of hg
  refine ⟨?_, ?_, ?_⟩
  · change pairMed f g c =
        funOf (arrowR c (tensorR a b)) (pairMed f g c)
    rw [arrowR.app, pairMed]
    apply graph_ext
    intro x
    have hfxc : funOf f (funOf c x) =
        funOf a (funOf f (funOf c x)) := by
      have := congrArg (fun w => funOf w (funOf c x)) hf'
      simpa [comp.app, retract_app hc] using this
    have hgxc : funOf g (funOf c x) =
        funOf b (funOf g (funOf c x)) := by
      have := congrArg (fun w => funOf w (funOf c x)) hg'
      simpa [comp.app, retract_app hc] using this
    have hL : funOf (tensorR f g) (funOf diagC (funOf c x)) =
        pairSeq (funOf f (funOf c x)) (funOf g (funOf c x)) := by
      rw [diagC_app, tensorR_app, pairSeq_app_zero, pairSeq_app_one]
    have hR : funOf (tensorR a b)
        (funOf (tensorR f g) (funOf diagC (funOf c (funOf c x)))) =
        pairSeq (funOf a (funOf f (funOf c x)))
          (funOf b (funOf g (funOf c x))) := by
      rw [retract_app hc, diagC_app]
      have hinter :
          funOf (tensorR f g) (pairSeq (funOf c x) (funOf c x)) =
            pairSeq (funOf f (funOf c x)) (funOf g (funOf c x)) := by
        rw [tensorR_app, pairSeq_app_zero, pairSeq_app_one]
      rw [hinter, tensorR_app, pairSeq_app_zero, pairSeq_app_one]
    simp only [comp.app]
    rw [hL, hR]
    exact congr (congrArg pairSeq hfxc) hgxc
  · apply graph_funOf_ext (comp_isGraph _ _) (typed_isGraph hf)
    intro x
    simp only [comp.app, pairMed, fstC.app, diagC_app]
    rw [tensorR_app, pairSeq_app_zero, pairSeq_app_zero]
    have := congrArg (fun w => funOf w x) (typed.arrowR_restrict hc hf)
    simpa [comp.app] using this.symm
  · apply graph_funOf_ext (comp_isGraph _ _) (typed_isGraph hg)
    intro x
    simp only [comp.app, pairMed, sndC.app, diagC_app]
```

```
    rw [tensorR_app, pairSeq_app_one, pairSeq_app_one]
    have := congrArg (fun w => funOf w x) (typed.arrowR_restrict hc hg)
    simpa [comp.app] using this.symm

end DomainTheory.DataTypesAsLattices
```

theorem_4_4_med_unique.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.comp
public import DomainTheory.DataTypesAsLattices.Retracts.Retract
public import DomainTheory.DataTypesAsLattices.Retracts.typed
public import DomainTheory.DataTypesAsLattices.Retracts.arrowR
public import DomainTheory.DataTypesAsLattices.Retracts.fstC
public import DomainTheory.DataTypesAsLattices.Retracts.sndC
public import DomainTheory.DataTypesAsLattices.Retracts.tensorR
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.pairMed
public import DomainTheory.DataTypesAsLattices.Functionality.pairSeq_app_zero
public import DomainTheory.DataTypesAsLattices.Functionality.pairSeq_app_one
public import DomainTheory.DataTypesAsLattices.Functionality.tensorR_app
public import DomainTheory.DataTypesAsLattices.Functionality.diagC_app
public import DomainTheory.DataTypesAsLattices.Functionality.graph_funOf_ext
public import DomainTheory.DataTypesAsLattices.Functionality.typed_isGraph
public import DomainTheory.DataTypesAsLattices.Functionality.theorem_4_4_med
public import DomainTheory.DataTypesAsLattices.Retracts.arrowR_more
public import DomainTheory.DataTypesAsLattices.Retracts.typed_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, Theorem 4.4.** The product mediator is unique. -/
theorem theorem_4_4_med_unique {a b c f g h : Pomega}
    (ha : Retract a) (hb : Retract b) (hc : Retract c)
    (hf : typed f (arrowR c a)) (hg : typed g (arrowR c b))
    (hh : typed h (arrowR c (tensorR a b)))
    (hfst : comp fstC h = f) (hsnd : comp sndC h = g) :
    h = pairMed f g c := by
  apply graph_funOf_ext (typed_isGraph hh)
    (typed_isGraph (theorem_4_4_med ha hb hc hf hg).1)
  intro x
  have hh' := arrowR.typed_of hh
  have hx : funOf h x =
      funOf (tensorR a b) (funOf h (funOf c x)) := by
    have := congrArg (fun w => funOf w x) hh'
    simpa [comp.app] using this
  have h0 : funOf (funOf h x) (ofNat 0) = funOf f x := by
    have := congrArg (fun w => funOf w x) hfst
    simpa [comp.app, fstC.app] using this
  have h1 : funOf (funOf h x) (ofNat 1) = funOf g x := by
```

```
    have := congrArg (fun w => funOf w x) hsnd
    simpa [comp.app, sndC.app] using this
  rw [hx, tensorR_app]
  have hfx : funOf a (funOf (funOf h (funOf c x)) (ofNat 0)) = funOf f x := by
    have h0c : funOf (funOf h (funOf c x)) (ofNat 0) = funOf f (funOf c x) := by
      have := congrArg (fun w => funOf w (funOf c x)) hfst
      simpa [comp.app, fstC.app] using this
    rw [h0c]
    have := congrArg (fun w => funOf w x) (arrowR.typed_of hf)
    simpa [comp.app] using this.symm
  have hgx : funOf b (funOf (funOf h (funOf c x)) (ofNat 1)) = funOf g x := by
    have h1c : funOf (funOf h (funOf c x)) (ofNat 1) = funOf g (funOf c x) := by
      have := congrArg (fun w => funOf w (funOf c x)) hsnd
      simpa [comp.app, sndC.app] using this
    rw [h1c]
    have := congrArg (fun w => funOf w x) (arrowR.typed_of hg)
    simpa [comp.app] using this.symm
  rw [hfx, hgx]
  simp only [pairMed, comp.app, diagC_app, tensorR_app, pairSeq_app_zero,
    pairSeq_app_one]
  have hfr : funOf f (funOf c x) = funOf f x := by
    have := congrArg (fun w => funOf w x) (typed.arrowR_restrict hc hf)
    simpa [comp.app] using this.symm
  have hgr : funOf g (funOf c x) = funOf g x := by
    have := congrArg (fun w => funOf w x) (typed.arrowR_restrict hc hg)
    simpa [comp.app] using this.symm
  rw [hfr, hgr]

end DomainTheory.DataTypesAsLattices
```

theorem_4_4_typed.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.typed
public import DomainTheory.DataTypesAsLattices.Retracts.pairSeq
public import DomainTheory.DataTypesAsLattices.Retracts.tensorR
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.pairSeq_app_zero
public import DomainTheory.DataTypesAsLattices.Functionality.pairSeq_app_one
public import DomainTheory.DataTypesAsLattices.Functionality.tensorR_app

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, Theorem 4.4 (ii).** `u : a ? b` iff `u = ⟨u₀, u₁⟩`
with `u₀ : a` and `u₁ : b`. -/
theorem theorem_4_4_typed {a b u : Pomega} :
    typed u (tensorR a b) ↔
      u = pairSeq (funOf u (ofNat 0)) (funOf u (ofNat 1)) ∧
        typed (funOf u (ofNat 0)) a ∧ typed (funOf u (ofNat 1)) b := by
```

```
  constructor
  · intro hu
    have hpair : u =
        pairSeq (funOf a (funOf u (ofNat 0)))
          (funOf b (funOf u (ofNat 1))) := by
      simpa [typed, tensorR_app] using hu
    have h0 : funOf u (ofNat 0) = funOf a (funOf u (ofNat 0)) := by
      have := congrArg (fun w => funOf w (ofNat 0)) hpair
      simpa [pairSeq_app_zero] using this
    have h1 : funOf u (ofNat 1) = funOf b (funOf u (ofNat 1)) := by
      have := congrArg (fun w => funOf w (ofNat 1)) hpair
      simpa [pairSeq_app_one] using this
    refine ⟨hpair.trans (congr (congrArg pairSeq h0.symm) h1.symm), h0, h1⟩
  · intro ⟨hpair, ha, hb⟩
    change u = funOf (tensorR a b) u
    rw [tensorR_app, ← ha, ← hb, ← hpair]

end DomainTheory.DataTypesAsLattices
```

theorem_4_5_sum.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.Retract
public import DomainTheory.DataTypesAsLattices.Retracts.typed
public import DomainTheory.DataTypesAsLattices.Retracts.pairSeq
public import DomainTheory.DataTypesAsLattices.Retracts.plusR
public import DomainTheory.DataTypesAsLattices.Retracts.IsStrict
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.plusR_isStrict
public import DomainTheory.DataTypesAsLattices.Functionality.plusR_isRetract
public import DomainTheory.DataTypesAsLattices.Functionality.plusR_typed_iff

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, Theorem 4.5 (The sum theorem).** -/
theorem theorem_4_5_sum {a b : Pomega} (ha : Retract a) (hb : Retract b) :
    Retract (plusR a b) ∧ IsStrict (plusR a b) ∧
      (∀ u, typed u (plusR a b) ↔
        u = botElem ∨ u = topElem ∨
          (∃ x, u = pairSeq (ofNat 0) x ∧ typed x a) ∨
            (∃ y, u = pairSeq (ofNat 1) y ∧ typed y b)) :=
  ⟨plusR_isRetract ha hb, plusR_isStrict a b, fun _u => plusR_typed_iff ha hb⟩

end DomainTheory.DataTypesAsLattices
```

theorem_7_1.lean

```
module
```

```
public import DomainTheory.DataTypesAsLattices.Embedding.extend
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Functionality.RestrictedEquiv
public import DomainTheory.DataTypesAsLattices.Functionality.pairElem
public import DomainTheory.DataTypesAsLattices.Functionality.mem
public import DomainTheory.DataTypesAsLattices.Functionality.arrowE
public import DomainTheory.DataTypesAsLattices.Functionality.prodE
public import DomainTheory.DataTypesAsLattices.Functionality.sumE

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, Theorem 7.1 (The closure theorem).** -/
theorem theorem_7_1 (A B : RestrictedEquiv) :
    (∀ f, (arrowE A B).mem f ↔
        f = graph (fun x => funOf f x) ∧
          ∀ x y, A.rel x y → B.rel (funOf f x) (funOf f y)) ∧
      (∀ {f x}, (arrowE A B).mem f → A.mem x → B.mem (funOf f x)) ∧
      (∀ u, (prodE A B).mem u ↔
        u = pairElem (funOf u (ofNat 0)) (funOf u (ofNat 1)) ∧
          A.mem (funOf u (ofNat 0)) ∧ B.mem (funOf u (ofNat 1))) ∧
      (∀ u, (sumE A B).mem u ↔
        (u = pairElem (ofNat 0) (funOf u (ofNat 1)) ∧
          A.mem (funOf u (ofNat 1))) ∨
        (u = pairElem (ofNat 1) (funOf u (ofNat 1)) ∧
          B.mem (funOf u (ofNat 1)))) := by
  refine ⟨?_, ?_, ?_, ?_⟩
  · intro f
    constructor
    · intro hf
      exact ⟨hf.1, fun x y hxy => hf.2.2 x y hxy⟩
    · intro ⟨hf, hpres⟩
      exact ⟨hf, hf, hpres⟩
  · intro f x hf hx
    exact hf.2.2 x x hx
  · intro u
    constructor
    · intro hu
      exact ⟨hu.1, hu.2.2.1, hu.2.2.2⟩
    · intro ⟨hu, hA, hB⟩
      exact ⟨hu, hu, hA, hB⟩
  · intro u
    constructor
    · intro hu
      rcases hu with ⟨hu, _, hA⟩ | ⟨hu, _, hB⟩
      · exact Or.inl ⟨hu, hA⟩
      · exact Or.inr ⟨hu, hB⟩
    · intro h
      rcases h with ⟨hu, hA⟩ | ⟨hu, hB⟩
      · exact Or.inl ⟨hu, hu, hA⟩
      · exact Or.inr ⟨hu, hu, hB⟩

end DomainTheory.DataTypesAsLattices
```

theorem_7_2.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.comp
public import DomainTheory.DataTypesAsLattices.Retracts.Retract
public import DomainTheory.DataTypesAsLattices.Retracts.typed
public import DomainTheory.DataTypesAsLattices.Retracts.arrowR
public import DomainTheory.DataTypesAsLattices.Retracts.tensorR
public import DomainTheory.DataTypesAsLattices.Retracts.plusR
public import DomainTheory.DataTypesAsLattices.Retracts.retract_app
public import DomainTheory.DataTypesAsLattices.Retracts.graph_ext
public import DomainTheory.DataTypesAsLattices.Retracts.theorem_4_3
public import DomainTheory.DataTypesAsLattices.Embedding.extend
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.mem
public import DomainTheory.DataTypesAsLattices.Functionality.arrowE
public import DomainTheory.DataTypesAsLattices.Functionality.prodE
public import DomainTheory.DataTypesAsLattices.Functionality.sumE
public import DomainTheory.DataTypesAsLattices.Functionality.Ea
public import DomainTheory.DataTypesAsLattices.Retracts.pairSeq
public import DomainTheory.DataTypesAsLattices.Functionality.pairSeq_app_zero
public import DomainTheory.DataTypesAsLattices.Functionality.pairSeq_app_one
public import DomainTheory.DataTypesAsLattices.Functionality.pairSeq_inj
public import DomainTheory.DataTypesAsLattices.Functionality.typed_pairSeq_tensorR
public import DomainTheory.DataTypesAsLattices.Functionality.pairElem
public import DomainTheory.DataTypesAsLattices.Functionality.plusR_typed_bot
public import DomainTheory.DataTypesAsLattices.Functionality.plusR_typed_inl
public import DomainTheory.DataTypesAsLattices.Functionality.plusR_typed_inr
public import DomainTheory.DataTypesAsLattices.Functionality.plusR_typed_top
public import DomainTheory.DataTypesAsLattices.Functionality.plusR_typed_iff
public import DomainTheory.DataTypesAsLattices.Functionality.theorem_4_4_typed
public import DomainTheory.DataTypesAsLattices.Retracts.arrowR_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices
/-- **Scott 1976, Theorem 7.2 (i).** `E_a` is the identity on the range of `a`. -/
theorem theorem_7_2.i (a x y : Pomega) :
    (Ea a).rel x y ↔ typed x a ∧ typed y a ∧ x = y :=
  Iff.rfl

/-- The spaces of `E_a` and of `{⟨x, y⟩ | a(x) = a(y)}` are isomorphic:
classes of the kernel are represented uniquely by range elements. -/
theorem theorem_7_2.i_iso {a : Pomega} (ha : Retract a) (x y : Pomega) :
    funOf a x = funOf a y ↔ (Ea a).rel (funOf a x) (funOf a y) := by
  constructor
  · intro h
    refine ⟨?_, ?_, h⟩
    · exact (retract_app ha x).symm
    · exact (retract_app ha y).symm
  · intro h
    exact h.2.2
```

```
theorem theorem_7_2.ii_of_typed {a b f : Pomega} (hf : typed f (arrowR a b)) :
    (arrowE (Ea a) (Ea b)).mem f := by
  have hf' : f = comp b (comp f a) := by
    simpa [typed, arrowR.app] using hf
  have hgraph : f = graph (fun x => funOf f x) := by
    apply Eq.trans hf'
    apply graph_ext
    intro x
    have : funOf f x = funOf b (funOf f (funOf a x)) := by
      have h1 : funOf f x = funOf (comp b (comp f a)) x :=
        congrArg (fun w => funOf w x) hf'
      rw [h1, comp.app, comp.app]
    rw [comp.app, this]
  refine ⟨hgraph, hgraph, ?_⟩
  intro x y hxy
  rcases hxy with ⟨hx, hy, rfl⟩
  have hfx : typed (funOf f x) b := theorem_4_3.ii hf hx
  exact ⟨hfx, hfx, rfl⟩

/-- **Scott 1976, Theorem 7.2 (ii).** Maps in `E_{a o→ b}` are graphs of
maps from `a` to `b`, corresponding to `E_a → E_b`. -/
theorem theorem_7_2.ii {a b : Pomega} (ha : Retract a) (hb : Retract b)
    (f g : Pomega) :
    (typed f (arrowR a b) → (arrowE (Ea a) (Ea b)).mem f) ∧
      ((arrowE (Ea a) (Ea b)).rel f g →
        (Ea (arrowR a b)).rel (funOf (arrowR a b) f) (funOf (arrowR a b) g)) ∧
      ((Ea (arrowR a b)).rel f g → (arrowE (Ea a) (Ea b)).rel f g) := by
  refine ⟨fun hf => theorem_7_2.ii_of_typed hf, ?_, ?_⟩
  · intro hfg
    rcases hfg with ⟨hfgraph, hggraph, hpres⟩
    have hftyped : typed (funOf (arrowR a b) f) (arrowR a b) :=
      (retract_app (arrowR.isRetract ha hb) f).symm
    have hgtyped : typed (funOf (arrowR a b) g) (arrowR a b) :=
      (retract_app (arrowR.isRetract ha hb) g).symm
    refine ⟨hftyped, hgtyped, ?_⟩
    rw [arrowR.app, arrowR.app]
    apply graph_ext
    intro x
    simp only [comp.app]
    have hax : typed (funOf a x) a := (retract_app ha x).symm
    have hsame := hpres (funOf a x) (funOf a x) ⟨hax, hax, rfl⟩
    have hfeq : funOf f (funOf a x) = funOf g (funOf a x) := hsame.2.2
    have hfb : typed (funOf f (funOf a x)) b := hsame.1
    have hgb : typed (funOf g (funOf a x)) b := hsame.2.1
    rw [← hfb, ← hgb, hfeq]
  · intro hfg
    rcases hfg with ⟨hf, hg, rfl⟩
    have hmem := theorem_7_2.ii_of_typed hf
    exact ⟨hmem.1, hmem.1, hmem.2.2⟩

/-- **Scott 1976, Theorem 7.2 (iii).** `E_{a ? b}` agrees with
`E_a × E_b` on `pairSeq` elements. -/
theorem theorem_7_2.iii (a b x y x' y' : Pomega) :
    (Ea (tensorR a b)).rel (pairSeq x y) (pairSeq x' y') ↔
      (prodE (Ea a) (Ea b)).rel (pairSeq x y) (pairSeq x' y') := by
  constructor
```

```
  · intro ⟨hu, hv, heq⟩
    obtain ⟨rfl, rfl⟩ := pairSeq_inj heq
    have hxy : typed x a ∧ typed y b := (typed_pairSeq_tensorR a b x y).mp hu
    refine ⟨?_, ?_, ?_, ?_⟩
    · rw [pairSeq_app_zero, pairSeq_app_one]; rfl
    · rw [pairSeq_app_zero, pairSeq_app_one]; rfl
    · rw [pairSeq_app_zero]
      exact ⟨hxy.1, hxy.1, rfl⟩
    · rw [pairSeq_app_one]
      exact ⟨hxy.2, hxy.2, rfl⟩
  · intro ⟨_hu, _hv, h0, h1⟩
    rw [pairSeq_app_zero, pairSeq_app_zero] at h0
    rw [pairSeq_app_one, pairSeq_app_one] at h1
    rcases h0 with ⟨hx, hx', heqx⟩
    rcases h1 with ⟨hy, hy', heqy⟩
    refine ⟨(typed_pairSeq_tensorR a b x y).mpr ⟨hx, hy⟩,
      (typed_pairSeq_tensorR a b x' y').mpr ⟨hx', hy'⟩, ?_⟩
    rw [heqx, heqy]

/-- **Scott 1976, Theorem 7.2 (iii).**
The printed identity `E_{a ? b} = E_a × E_b`, for arbitrary representatives. -/
theorem theorem_7_2.iii_exact (a b u v : Pomega) :
    (Ea (tensorR a b)).rel u v ↔ (prodE (Ea a) (Ea b)).rel u v := by
  constructor
  · intro huv
    rcases huv with ⟨hu, hv, rfl⟩
    rcases (theorem_4_4_typed.mp hu) with ⟨hpair, ha, hb⟩
    refine ⟨hpair, hpair, ?_, ?_⟩
    · exact ⟨ha, ha, rfl⟩
    · exact ⟨hb, hb, rfl⟩
  · intro huv
    rcases huv with ⟨hu, hv, h0, h1⟩
    rcases h0 with ⟨hx, hx', heqx⟩
    rcases h1 with ⟨hy, hy', heqy⟩
    have huvEq : u = v := by
      rw [hu, hv, heqx, heqy]
    refine ⟨?_, ?_, huvEq⟩
    · exact theorem_4_4_typed.mpr ⟨hu, hx, hy⟩
    · exact theorem_4_4_typed.mpr ⟨hv, hx', hy'⟩

/-- **Scott 1976, Theorem 7.2 (iv).**
The exact identity
`E_{a ? b} = E_a + E_b ∪ {⟨⊥,⊥⟩, ⟨⊤,⊤⟩}`. -/
theorem theorem_7_2.iv {a b : Pomega} (ha : Retract a) (hb : Retract b)
    (u v : Pomega) :
    (Ea (plusR a b)).rel u v ↔
      (sumE (Ea a) (Ea b)).rel u v ∨
        (u = botElem ∧ v = botElem) ∨ (u = topElem ∧ v = topElem) := by
  constructor
  · intro huv
    rcases huv with ⟨hu, hv, huv⟩
    subst v
    rcases (plusR_typed_iff ha hb).mp hu with
      rfl | rfl | ⟨x, rfl, hx⟩ | ⟨y, rfl, hy⟩
    · exact Or.inr (Or.inl ⟨rfl, rfl⟩)
    · exact Or.inr (Or.inr ⟨rfl, rfl⟩)
```

```
    · exact Or.inl (Or.inl ⟨by rw [pairElem.eq_pairSeq, pairSeq_app_one],
          by rw [pairElem.eq_pairSeq, pairSeq_app_one],
          by simpa [pairSeq_app_one] using (show (Ea a).rel x x from ⟨hx, hx, rfl⟩)⟩)
    · exact Or.inl (Or.inr ⟨by rw [pairElem.eq_pairSeq, pairSeq_app_one],
          by rw [pairElem.eq_pairSeq, pairSeq_app_one],
          by simpa [pairSeq_app_one] using (show (Ea b).rel y y from ⟨hy, hy, rfl⟩)⟩)
  · intro h
    rcases h with h | ⟨rfl, rfl⟩ | ⟨rfl, rfl⟩
    rcases h with ⟨hu, hv, hA⟩ | ⟨hu, hv, hB⟩
    · rcases hA with ⟨hxt, hyt, heq⟩
      have heq' : u = v := by
        rw [hu, hv, heq]
      refine ⟨?_, ?_, heq'⟩
      · rw [hu]; exact plusR_typed_inl hxt
      · rw [hv]; exact plusR_typed_inl hyt
    · rcases hB with ⟨hxt, hyt, heq⟩
      have heq' : u = v := by
        rw [hu, hv, heq]
      refine ⟨?_, ?_, heq'⟩
      · rw [hu]; exact plusR_typed_inr hxt
      · rw [hv]; exact plusR_typed_inr hyt
    · exact ⟨plusR_typed_bot a b, plusR_typed_bot a b, rfl⟩
    · exact ⟨plusR_typed_top a b, plusR_typed_top a b, rfl⟩

theorem theorem_7_2 {a b : Pomega} (ha : Retract a) (hb : Retract b) :
    (∀ x y, funOf a x = funOf a y ↔ (Ea a).rel (funOf a x) (funOf a y)) ∧
      (∀ f g, (typed f (arrowR a b) → (arrowE (Ea a) (Ea b)).mem f) ∧
        ((arrowE (Ea a) (Ea b)).rel f g →
          (Ea (arrowR a b)).rel (funOf (arrowR a b) f)
            (funOf (arrowR a b) g)) ∧
        ((Ea (arrowR a b)).rel f g → (arrowE (Ea a) (Ea b)).rel f g)) ∧
      (∀ u v, (Ea (tensorR a b)).rel u v ↔ (prodE (Ea a) (Ea b)).rel u v) ∧
      (∀ u v, (Ea (plusR a b)).rel u v ↔
        (sumE (Ea a) (Ea b)).rel u v ∨
          (u = botElem ∧ v = botElem) ∨ (u = topElem ∧ v = topElem)) :=
  ⟨fun x y => theorem_7_2.i_iso ha x y,
    fun f g => theorem_7_2.ii ha hb f g,
    theorem_7_2.iii_exact a b,
    theorem_7_2.iv ha hb⟩

end DomainTheory.DataTypesAsLattices
```

theorem_7_3.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.Icomb
public import DomainTheory.DataTypesAsLattices.Embedding.extend
public import DomainTheory.DataTypesAsLattices.Lambda.Kcomb
public import DomainTheory.DataTypesAsLattices.Lambda.Scomb
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Functionality.mem
public import DomainTheory.DataTypesAsLattices.Functionality.arrowE
public import DomainTheory.DataTypesAsLattices.Retracts.graph_ext
```

```
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.RestrictedEquiv
public import DomainTheory.DataTypesAsLattices.Functionality.emptyE
public import DomainTheory.DataTypesAsLattices.Functionality.singletonE
public import DomainTheory.DataTypesAsLattices.Functionality.singletonE_mem
public import DomainTheory.DataTypesAsLattices.Functionality.theorem_7_1
public import DomainTheory.DataTypesAsLattices.Functionality.graph_funOf_ext
public import DomainTheory.DataTypesAsLattices.Functionality.Icomb_isGraph
public import DomainTheory.DataTypesAsLattices.Functionality.Kcomb_app2
public import DomainTheory.DataTypesAsLattices.Functionality.Kcomb_isGraph
public import DomainTheory.DataTypesAsLattices.Functionality.Kcomb_app_isGraph
public import DomainTheory.DataTypesAsLattices.Functionality.Scomb_app3
public import DomainTheory.DataTypesAsLattices.Functionality.Scomb_isGraph
public import DomainTheory.DataTypesAsLattices.Functionality.Scomb_app_isGraph
public import DomainTheory.DataTypesAsLattices.Functionality.Scomb_app2_isGraph
public import DomainTheory.DataTypesAsLattices.Functionality.eta2_app
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices
/-- **Scott 1976, Theorem 7.3 (The functionality theorem).** -/

theorem theorem_7_3.I_unique (i : Pomega)
    (hi : ∀ A, (arrowE A A).mem i) : i = Icomb := by
  have hgraph : i = graph (fun x => funOf i x) := (hi emptyE).1
  apply graph_funOf_ext hgraph Icomb_isGraph
  intro a
  have ha : (singletonE a).mem (funOf i a) :=
    (theorem_7_1 (singletonE a) (singletonE a)).2.1 (hi (singletonE a))
      ((singletonE_mem a a).mpr rfl)
  simpa [Icomb.app, singletonE_mem] using ha

theorem theorem_7_3.K_unique (k : Pomega)
    (hk : ∀ A B, (arrowE A (arrowE B A)).mem k) : k = Kcomb := by
  have hgraph : k = graph (fun x => funOf k x) := (hk emptyE emptyE).1
  apply graph_funOf_ext hgraph Kcomb_isGraph
  intro a
  have hkA : (arrowE (singletonE a) (arrowE emptyE (singletonE a))).mem k :=
    hk (singletonE a) emptyE
  have hka : (arrowE emptyE (singletonE a)).mem (funOf k a) :=
    (theorem_7_1 (singletonE a) (arrowE emptyE (singletonE a))).2.1 hkA
      ((singletonE_mem a a).mpr rfl)
  apply graph_funOf_ext hka.1 (Kcomb_app_isGraph a)
  intro b
  have hkAB : (arrowE (singletonE a) (arrowE (singletonE b) (singletonE a))).mem k :=
    hk (singletonE a) (singletonE b)
  have hkab : (arrowE (singletonE b) (singletonE a)).mem (funOf k a) :=
    (theorem_7_1 (singletonE a) (arrowE (singletonE b) (singletonE a))).2.1 hkAB
      ((singletonE_mem a a).mpr rfl)
  have hab : (singletonE a).mem (funOf (funOf k a) b) :=
    (theorem_7_1 (singletonE b) (singletonE a)).2.1 hkab
      ((singletonE_mem b b).mpr rfl)
  simpa [Kcomb_app2, singletonE_mem] using hab
```

```
theorem theorem_7_3.S_unique (s : Pomega)
    (h?1 : ∀ f, funOf s f =
      funOf s (graph (fun x => graph (fun y => funOf (funOf f x) y))))
    (h?2 : ∀ f g, funOf (funOf s f) g =
      funOf (funOf s f) (graph (fun x => funOf g x)))
    (hs : ∀ A B C,
      (arrowE (arrowE A (arrowE B C))
        (arrowE (arrowE A B) (arrowE A C))).mem s) :
    s = Scomb := by
  have hsgraph : s = graph (fun f => funOf s f) := (hs emptyE emptyE emptyE).1
  apply graph_funOf_ext hsgraph Scomb_isGraph
  intro f
  let f' : Pomega := graph (fun x => graph (fun y => funOf (funOf f x) y))
  have hf'app (z : Pomega) : funOf f' z = graph (fun y => funOf (funOf f z) y) :=
    eta2_app f z
  have hf'graph : f' = graph (fun z => funOf f' z) := by
    apply graph_ext
    intro z
    exact (hf'app z).symm
  have hsf : funOf s f = funOf s f' := h?1 f
  have hsf'graph : funOf s f' = graph (fun g => funOf (funOf s f') g) := by
    have hf'mem : (arrowE emptyE (arrowE emptyE emptyE)).mem f' :=
      ⟨hf'graph, hf'graph, fun _ _ h => False.elim h⟩
    have := (theorem_7_1 (arrowE emptyE (arrowE emptyE emptyE))
      (arrowE (arrowE emptyE emptyE) (arrowE emptyE emptyE))).2.1
      (hs emptyE emptyE emptyE) hf'mem
    exact this.1
  apply graph_funOf_ext (by rw [hsf]; exact hsf'graph) (Scomb_app_isGraph f)
  intro g
  let g' : Pomega := graph (funOf g)
  have hg'app (z : Pomega) : funOf g' z = funOf g z :=
    beta (funOf.isScottContinuous g) z
  have hg'graph : g' = graph (fun z => funOf g' z) := by
    apply graph_ext
    intro z
    exact (hg'app z).symm
  have hsg : funOf (funOf s f) g = funOf (funOf s f) g' := h?2 f g
  have hsg' : funOf (funOf s f) g' = funOf (funOf s f') g' := by
    rw [hsf]
  have hsg'graph :
      funOf (funOf s f') g' =
        graph (fun z => funOf (funOf (funOf s f') g') z) := by
    have hg'mem : (arrowE emptyE emptyE).mem g' :=
      ⟨hg'graph, hg'graph, fun _ _ h => False.elim h⟩
    have hf'mem : (arrowE emptyE (arrowE emptyE emptyE)).mem f' :=
      ⟨hf'graph, hf'graph, fun _ _ h => False.elim h⟩
    have hsf'mem :
        (arrowE (arrowE emptyE emptyE) (arrowE emptyE emptyE)).mem (funOf s f') :=
      (theorem_7_1 (arrowE emptyE (arrowE emptyE emptyE))
        (arrowE (arrowE emptyE emptyE) (arrowE emptyE emptyE))).2.1
        (hs emptyE emptyE emptyE) hf'mem
    exact ((theorem_7_1 (arrowE emptyE emptyE)
      (arrowE emptyE emptyE)).2.1 hsf'mem hg'mem).1
  apply graph_funOf_ext
    (by rw [hsg, hsg']; exact hsg'graph) (Scomb_app2_isGraph f g)
  intro arg
```

```
  let A := singletonE arg
  let B := singletonE (funOf g arg)
  let C := singletonE (funOf (funOf f arg) (funOf g arg))
  have hf'memABC : (arrowE A (arrowE B C)).mem f' := by
    refine ⟨hf'graph, hf'graph, ?_⟩
    intro x₁ x₂ hx₁₂
    have hx₁ : x₁ = arg := hx₁₂.1
    have hx₂ : x₂ = arg := hx₁₂.2
    have hfxgraph (z : Pomega) :
        funOf f' z = graph (fun y => funOf (funOf f' z) y) := by
      rw [hf'app]
      apply graph_ext
      intro y
      exact (beta (funOf.isScottContinuous (funOf f z)) y).symm
    rw [hx₁, hx₂]
    refine ⟨hfxgraph arg, hfxgraph arg, ?_⟩
    intro y₁ y₂ hy
    have hy₁ : y₁ = funOf g arg := hy.1
    have hy₂ : y₂ = funOf g arg := hy.2
    rw [hy₁, hy₂, hf'app]
    have happ :
        funOf (graph (fun y => funOf (funOf f arg) y)) (funOf g arg) =
          funOf (funOf f arg) (funOf g arg) :=
      beta (funOf.isScottContinuous (funOf f arg)) (funOf g arg)
    exact happ.symm ▶ ⟨rfl, rfl⟩
  have hg'memAB : (arrowE A B).mem g' := by
    refine ⟨hg'graph, hg'graph, ?_⟩
    intro x₁ x₂ hx₁₂
    have hx₁ : x₁ = arg := hx₁₂.1
    have hx₂ : x₂ = arg := hx₁₂.2
    rw [hx₁, hx₂, hg'app]
    exact ⟨rfl, rfl⟩
  have hsf'mem :
      (arrowE (arrowE A B) (arrowE A C)).mem (funOf s f') :=
    (theorem_7_1 (arrowE A (arrowE B C))
      (arrowE (arrowE A B) (arrowE A C))).2.1 (hs A B C) hf'memABC
  have hsg'mem : (arrowE A C).mem (funOf (funOf s f') g') :=
    (theorem_7_1 (arrowE A B) (arrowE A C)).2.1 hsf'mem hg'memAB
  have hxA : A.mem arg := (singletonE_mem arg arg).mpr rfl
  have hval : C.mem (funOf (funOf (funOf s f') g') arg) :=
    (theorem_7_1 A C).2.1 hsg'mem hxA
  have hval' : funOf (funOf (funOf s f') g') arg =
      funOf (funOf f arg) (funOf g arg) :=
    (singletonE_mem _ _).mp hval
  have hgoal : funOf (funOf (funOf s f) g) arg =
      funOf (funOf (funOf s f') g') arg := by
    rw [hsg, hsg']
  rw [hgoal, hval', Scomb_app3]

/-- **Scott 1976, Theorem 7.3 (i).** `I : A → A`. -/
theorem theorem_7_3.i (A : RestrictedEquiv) :
    (arrowE A A).mem Icomb :=
  ⟨Icomb_isGraph, Icomb_isGraph, fun x y hxy => by simpa [Icomb.app] using hxy⟩

/-- **Scott 1976, Theorem 7.3 (ii).** `K : A → (B → A)`. -/
theorem theorem_7_3.ii (A B : RestrictedEquiv) :
```

```
    (arrowE A (arrowE B A)).mem Kcomb := by
  refine ⟨Kcomb_isGraph, Kcomb_isGraph, ?_⟩
  intro x x' hx
  refine ⟨Kcomb_app_isGraph x, Kcomb_app_isGraph x', ?_⟩
  intro y y' _hy
  simpa [Kcomb_app2] using hx

/-- **Scott 1976, Theorem 7.3 (iii).**
`S : (A → (B → C)) → ((A → B) → (A → C))`. -/
theorem theorem_7_3.iii (A B C : RestrictedEquiv) :
    (arrowE (arrowE A (arrowE B C))
      (arrowE (arrowE A B) (arrowE A C))).mem Scomb := by
  refine ⟨Scomb_isGraph, Scomb_isGraph, ?_⟩
  intro f f' hf
  refine ⟨Scomb_app_isGraph f, Scomb_app_isGraph f', ?_⟩
  intro g g' hg
  refine ⟨Scomb_app2_isGraph f g, Scomb_app2_isGraph f' g', ?_⟩
  intro x x' hx
  have hgx : (arrowE A B).rel g g' := hg
  have hfx : (arrowE A (arrowE B C)).rel f f' := hf
  have hB : B.rel (funOf g x) (funOf g' x') := hgx.2.2 x x' hx
  have hBC : (arrowE B C).rel (funOf f x) (funOf f' x') := hfx.2.2 x x' hx
  simpa [Scomb_app3] using hBC.2.2 (funOf g x) (funOf g' x') hB

theorem theorem_7_3 :
    (∀ A, (arrowE A A).mem Icomb) ∧
      (∀ A B, (arrowE A (arrowE B A)).mem Kcomb) ∧
      (∀ A B C,
        (arrowE (arrowE A (arrowE B C))
          (arrowE (arrowE A B) (arrowE A C))).mem Scomb) ∧
      (∀ i, (∀ A, (arrowE A A).mem i) → i = Icomb) ∧
      (∀ k, (∀ A B, (arrowE A (arrowE B A)).mem k) → k = Kcomb) ∧
      (∀ s,
        (∀ f, funOf s f =
          funOf s (graph (fun x => graph (fun y => funOf (funOf f x) y)))) →
        (∀ f g, funOf (funOf s f) g =
          funOf (funOf s f) (graph (fun x => funOf g x))) →
        (∀ A B C,
          (arrowE (arrowE A (arrowE B C))
            (arrowE (arrowE A B) (arrowE A C))).mem s) →
        s = Scomb) :=
  ⟨theorem_7_3.i, theorem_7_3.ii, theorem_7_3.iii,
    theorem_7_3.I_unique, theorem_7_3.K_unique, theorem_7_3.S_unique⟩

end DomainTheory.DataTypesAsLattices
```

theorem_7_4.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
```

```
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Embedding.extend
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Functionality.Ea
public import DomainTheory.DataTypesAsLattices.Functionality.HF
public import DomainTheory.DataTypesAsLattices.Functionality.Hfun
public import DomainTheory.DataTypesAsLattices.Functionality.Hinterp
public import DomainTheory.DataTypesAsLattices.Functionality.Htyped
public import DomainTheory.DataTypesAsLattices.Functionality.LambTerm
public import DomainTheory.DataTypesAsLattices.Functionality.Nk
public import DomainTheory.DataTypesAsLattices.Functionality.Nrel
public import DomainTheory.DataTypesAsLattices.Functionality.RestrictedEquiv
public import DomainTheory.DataTypesAsLattices.Functionality.Z
public import DomainTheory.DataTypesAsLattices.Functionality.Zcomb
public import DomainTheory.DataTypesAsLattices.Functionality.arrowE
public import DomainTheory.DataTypesAsLattices.Functionality.emptyE
public import DomainTheory.DataTypesAsLattices.Functionality.encodeExp
public import DomainTheory.DataTypesAsLattices.Functionality.eq_7_18_eq
public import DomainTheory.DataTypesAsLattices.Functionality.eq_7_19
public import DomainTheory.DataTypesAsLattices.Functionality.eraseTag
public import DomainTheory.DataTypesAsLattices.Functionality.expPayload
public import DomainTheory.DataTypesAsLattices.Functionality.expTag
public import DomainTheory.DataTypesAsLattices.Functionality.graph_funOf_ext
public import DomainTheory.DataTypesAsLattices.Functionality.hApp
public import DomainTheory.DataTypesAsLattices.Functionality.hApply
public import DomainTheory.DataTypesAsLattices.Functionality.hCond
public import DomainTheory.DataTypesAsLattices.Functionality.hPred
public import DomainTheory.DataTypesAsLattices.Functionality.hVar
public import DomainTheory.DataTypesAsLattices.Functionality.hZero
public import DomainTheory.DataTypesAsLattices.Functionality.inleftC
public import DomainTheory.DataTypesAsLattices.Functionality.inrightC
public import DomainTheory.DataTypesAsLattices.Functionality.iterateE
public import DomainTheory.DataTypesAsLattices.Functionality.kerE
public import DomainTheory.DataTypesAsLattices.Functionality.mem
public import DomainTheory.DataTypesAsLattices.Functionality.normalized_finite_app
public import DomainTheory.DataTypesAsLattices.Functionality.outC
public import DomainTheory.DataTypesAsLattices.Functionality.outleftC
public import DomainTheory.DataTypesAsLattices.Functionality.outrightC
public import DomainTheory.DataTypesAsLattices.Functionality.pairMed
public import DomainTheory.DataTypesAsLattices.Functionality.plotkinE
public import DomainTheory.DataTypesAsLattices.Functionality.plotkinGood
public import DomainTheory.DataTypesAsLattices.Functionality.plotkinRel
public import DomainTheory.DataTypesAsLattices.Functionality.plotkin_finite_separation
public import DomainTheory.DataTypesAsLattices.Functionality.plotkin_function_related
public import DomainTheory.DataTypesAsLattices.Functionality.prodE
public import DomainTheory.DataTypesAsLattices.Functionality.sigmaJ
public import DomainTheory.DataTypesAsLattices.Functionality.singletonE
public import DomainTheory.DataTypesAsLattices.Functionality.sumE
public import DomainTheory.DataTypesAsLattices.Functionality.theorem_7_1
public import DomainTheory.DataTypesAsLattices.Functionality.updateEnvUnion
public import DomainTheory.DataTypesAsLattices.Functionality.whichC
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.mu_law
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
```

```
public import DomainTheory.DataTypesAsLattices.Lambda.termDollarStep
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Retracts.graph_ext
public import DomainTheory.DataTypesAsLattices.Retracts.intR
public import DomainTheory.DataTypesAsLattices.Retracts.lfpAbove
public import DomainTheory.DataTypesAsLattices.Retracts.retractLe

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace theorem_7_4

theorem i (A : RestrictedEquiv) (n : ℕ) :
    (arrowE (arrowE A A) (arrowE A A)).mem (Zcomb n) := by
  refine ⟨Zcomb.isGraph n, Zcomb.isGraph n, ?_⟩
  intro f f' hf
  refine ⟨Zcomb.app_isGraph n f, Zcomb.app_isGraph n f', ?_⟩
  intro x y hxy
  simpa [Zcomb.app2] using Z.rel A n hf.2.2 hxy

/-- Plotkin: if `z` is typed as an iterator and is extensional in its first
argument, then `z(f)(x)` is some iterate `f?(x)`. -/

theorem plotkin (z : Pomega)
    (h? : ∀ f, funOf z f = funOf z (graph (fun y => funOf f y)))
    (hz : ∀ A, (arrowE (arrowE A A) (arrowE A A)).mem z)
    (f x : Pomega) :
    ∃ n, funOf (funOf z f) x = Z n f x := by
  let f' : Pomega := graph (fun y => funOf f y)
  have hf'graph : f' = graph (fun y => funOf f' y) := by
    apply graph_ext
    intro y
    exact (beta (funOf.isScottContinuous f) y).symm
  let A := iterateE f' x
  have hf'mem : (arrowE A A).mem f' := by
    refine ⟨hf'graph, hf'graph, ?_⟩
    intro u v huv
    rcases huv with ⟨⟨n, rfl⟩, rfl⟩
    refine ⟨⟨n + 1, rfl⟩, rfl⟩
  have hzA : (arrowE (arrowE A A) (arrowE A A)).mem z := hz A
  have hzf : (arrowE A A).mem (funOf z f') :=
    (theorem_7_1 (arrowE A A) (arrowE A A)).2.1 hzA hf'mem
  have hx : A.mem x := ⟨⟨0, by simp [Z]⟩, rfl⟩
  have hval : A.mem (funOf (funOf z f') x) :=
    (theorem_7_1 A A).2.1 hzf hx
  obtain ⟨⟨n, hn⟩, _⟩ := hval
  refine ⟨n, ?_⟩
  have h?f : funOf z f = funOf z f' := h? f
  rw [h?f, hn, Z.graph_funOf]

theorem finite (z : Pomega)
    (h? : ∀ f, funOf z f = funOf z (graph (fun y => funOf f y)))
    (hz : ∀ A, (arrowE (arrowE A A) (arrowE A A)).mem z)
    {n : ℕ}
```

```
  (hn : ∀ j, funOf (funOf z (graph (sigmaJ j))) {pair j 0} =
    ({pair j n} : Pomega))
  (p q : ℕ) :
  funOf (funOf z (e p)) (e q) = Z n (e p) (e q) := by
let j := max p q + 1
let gf := graph (fun y => funOf (e p) y)
let gu := gf ∪ graph (sigmaJ j)
have hfg : (arrowE
    (plotkinE p q j) (plotkinE p q j)).rel gf gu := by
  simpa [j, gf, gu] using plotkin_function_related p q
have hzfg :
    (arrowE (plotkinE p q j) (plotkinE p q j)).rel
      (funOf z gf) (funOf z gu) :=
  (hz (plotkinE p q j)).2.2 gf gu hfg
have hstart :
    (plotkinE p q j).rel (e q) (e q ∪ {pair j 0}) := by
  refine ⟨⟨0, Or.inl rfl⟩, ⟨0, Or.inr rfl⟩, ?_⟩
  exact Relation.EqvGen.rel _ _
    ⟨0, Or.inl ⟨rfl, rfl⟩⟩
have hout :
    (plotkinE p q j).rel
      (funOf (funOf z gf) (e q))
      (funOf (funOf z gu) (e q ∪ {pair j 0})) :=
  hzfg.2.2 _ _ hstart
obtain ⟨r, hr⟩ := theorem_7_4.plotkin z h? hz gf (e q)
obtain ⟨m, hm⟩ :=
  theorem_7_4.plotkin z h? hz gu (e q ∪ {pair j 0})
have hleft :
    funOf (funOf z gf) (e q) = Z r (e p) (e q) := by
  rw [hr]
  exact Z.graph_funOf r (e p) (e q)
have hright :
    funOf (funOf z gu) (e q ∪ {pair j 0}) =
      Z m (e p) (e q) ∪ {pair j m} := by
  rw [hm]
  simpa [j, gf, gu] using plotkin_finite_separation p q m
have herase := plotkinE.erase hout
have horbit : Z r (e p) (e q) = Z m (e p) (e q) := by
  rw [hleft, hright] at herase
  exact (eraseTag.Z_finite p q r).symm.trans
    (herase.trans (eraseTag.Z_finite_union p q m))
have hsigmaSub :
    funOf (funOf z (graph (sigmaJ j))) {pair j 0} ⊆
      funOf (funOf z gu) (e q ∪ {pair j 0}) := by
  exact mu_law
    (funOf.monotone_right z Set.subset_union_right)
    Set.subset_union_right
have htag : pair j n ∈ Z m (e p) (e q) ∪ {pair j m} := by
  rw [← hright]
  apply hsigmaSub
  rw [hn j]
  simp
have hnm : n = m := by
  rcases htag with hbad | htag
  · exact (Z.finite_no_tag p q m n (by simpa [j] using hbad)).elim
  · exact (pair_inj htag).2
```

```
  rw [h? (e p), hleft, horbit, ← hnm]

/-- Plotkin's independent successors force one common iterate count on all
tagged successor tests. -/

theorem sigma_index (z : Pomega)
    (h? : ∀ f, funOf z f = funOf z (graph (fun y => funOf f y)))
    (hz : ∀ A, (arrowE (arrowE A A) (arrowE A A)).mem z) :
    ∃ n, ∀ j,
      funOf (funOf z (graph (sigmaJ j))) {pair j 0} =
        ({pair j n} : Pomega) := by
  obtain ⟨n, hn⟩ :=
    theorem_7_4.plotkin z h? hz (graph (sigmaJ 0)) {pair 0 0}
  refine ⟨n, ?_⟩
  intro j
  have hn' :
      funOf (funOf z (graph (sigmaJ 0))) {pair 0 0} =
        ({pair 0 n} : Pomega) := by
    rw [hn, eq_7_18_eq]
  by_cases hj : j = 0
  · subst j
    exact hn'
  obtain ⟨nj, hnj⟩ :=
    theorem_7_4.plotkin z h? hz (graph (sigmaJ j)) {pair j 0}
  have hnj' :
      funOf (funOf z (graph (sigmaJ j))) {pair j 0} =
        ({pair j nj} : Pomega) := by
    rw [hnj, eq_7_18_eq]
  obtain ⟨k, hk⟩ := theorem_7_4.plotkin z h? hz
    (graph (sigmaJ 0) ∪ graph (sigmaJ j))
    (({pair 0 0} : Pomega) ∪ {pair j 0})
  have hk' :
      funOf
          (funOf z (graph (sigmaJ 0) ∪ graph (sigmaJ j)))
          (({pair 0 0} : Pomega) ∪ {pair j 0}) =
        ({pair 0 k} : Pomega) ∪ {pair j k} := by
    rw [hk, eq_7_19]
  have hmono0 :
      funOf (funOf z (graph (sigmaJ 0))) {pair 0 0} ⊆
        funOf
          (funOf z (graph (sigmaJ 0) ∪ graph (sigmaJ j)))
          (({pair 0 0} : Pomega) ∪ {pair j 0}) :=
    mu_law
      (funOf.monotone_right z Set.subset_union_left)
      Set.subset_union_left
  have hmonoj :
      funOf (funOf z (graph (sigmaJ j))) {pair j 0} ⊆
        funOf
          (funOf z (graph (sigmaJ 0) ∪ graph (sigmaJ j)))
          (({pair 0 0} : Pomega) ∪ {pair j 0}) :=
    mu_law
      (funOf.monotone_right z Set.subset_union_right)
      Set.subset_union_right
  have h0mem : pair 0 n ∈ ({pair 0 k} : Pomega) ∪ {pair j k} := by
    rw [← hk']
    apply hmono0
```

```
    rw [hn']
    simp
  have hjmem : pair j nj ∈ ({pair 0 k} : Pomega) ∪ {pair j k} := by
    rw [← hk']
    apply hmonoj
    rw [hnj']
    simp
  have hnk : n = k := by
    rcases h0mem with h0 | h0
    · exact (pair_inj h0).2
    · exact (hj ((pair_inj h0).1).symm).elim
  have hnjk : nj = k := by
    rcases hjmem with hj0 | hjj
    · exact (hj (pair_inj hj0).1).elim
    · exact (pair_inj hjj).2
  rw [hnj', hnjk, ← hnk]

theorem unique (z : Pomega)
    (h? : ∀ f, funOf z f = funOf z (graph (fun y => funOf f y)))
    (hz : ∀ A, (arrowE (arrowE A A) (arrowE A A)).mem z) :
    ∃ n, z = Zcomb n := by
  obtain ⟨n, hn⟩ := theorem_7_4.sigma_index z h? hz
  refine ⟨n, ?_⟩
  have hfinite (p : ℕ) :
      funOf z (e p) = funOf (Zcomb n) (e p) := by
    let gp := graph (fun y => funOf (e p) y)
    have hgpGraph : gp = graph (fun y => funOf gp y) := by
      apply graph_ext
      intro y
      exact (normalized_finite_app p y).symm
    have hgpMem : (arrowE emptyE emptyE).mem gp :=
      ⟨hgpGraph, hgpGraph, fun _ _ h => False.elim h⟩
    have hzgpGraph : funOf z gp = graph (fun y => funOf (funOf z gp) y) :=
      ((theorem_7_1 (arrowE emptyE emptyE) (arrowE emptyE emptyE)).2.1
        (hz emptyE) hgpMem).1
    have hzFiniteGraph :
        funOf z (e p) = graph (fun y => funOf (funOf z (e p)) y) := by
      rw [h? (e p)]
      exact hzgpGraph
    apply graph_funOf_ext hzFiniteGraph (Zcomb.app_isGraph n (e p))
    intro x
    have hright :
        IsScottContinuous (fun y => Z n (e p) y) :=
      Z.isScottContinuous_right n (e p)
    have hleft :
        IsScottContinuous (fun y => funOf (funOf z (e p)) y) :=
      funOf.isScottContinuous (funOf z (e p))
    have heq := congrFun
      (IsScottContinuous.determined hleft hright
        (fun q => theorem_7_4.finite z h? hz hn p q)) x
    simpa [Zcomb.app2] using heq
  have hall (f : Pomega) : funOf z f = funOf (Zcomb n) f := by
    let gf := graph (fun y => funOf f y)
    have hgfGraph : gf = graph (fun y => funOf gf y) := by
      apply graph_ext
      intro y
```

```
      exact (beta (funOf.isScottContinuous f) y).symm
    have hgfMem : (arrowE emptyE emptyE).mem gf :=
      ⟨hgfGraph, hgfGraph, fun _ _ h => False.elim h⟩
    have hzgfGraph : funOf z gf = graph (fun y => funOf (funOf z gf) y) :=
      ((theorem_7_1 (arrowE emptyE emptyE) (arrowE emptyE emptyE)).2.1
        (hz emptyE) hgfMem).1
    have hzfGraph :
        funOf z f = graph (fun y => funOf (funOf z f) y) := by
      rw [h? f]
      exact hzgfGraph
    apply graph_funOf_ext hzfGraph (Zcomb.app_isGraph n f)
    intro x
    have hleft :
        IsScottContinuous (fun g => funOf (funOf z g) x) :=
      theorem_1_3 (f := fun u => funOf u x) (g := funOf z)
        (funOf.isScottContinuous_left x)
        (funOf.isScottContinuous z)
    have hright : IsScottContinuous (fun g => Z n g x) :=
      Z.isScottContinuous_left n x
    have heq := congrFun
      (IsScottContinuous.determined hleft hright (fun p => by
        have := congrArg (fun w => funOf w x) (hfinite p)
        simpa [Zcomb.app2] using this)) f
    simpa [Zcomb.app2] using heq
  exact graph_funOf_ext (hz emptyE).1 (Zcomb.isGraph n) hall

end theorem_7_4

/-- **Scott 1976, Theorem 7.4 (The iterator theorem).**
The functionality is both satisfied by and globally characterizes the
iterators. -/
theorem theorem_7_4 :
    (∀ A n, (arrowE (arrowE A A) (arrowE A A)).mem (Zcomb n)) ∧
      ∀ z,
        (∀ f, funOf z f = funOf z (graph (fun y => funOf f y))) →
        (∀ A, (arrowE (arrowE A A) (arrowE A A)).mem z) →
        ∃ n, z = Zcomb n :=
  ⟨theorem_7_4.i, theorem_7_4.unique⟩

end DomainTheory.DataTypesAsLattices
```

theorem_4_4.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.seq2
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.Retract
public import DomainTheory.DataTypesAsLattices.Retracts.pairSeq
public import DomainTheory.DataTypesAsLattices.Retracts.bot_isRetract
public import DomainTheory.DataTypesAsLattices.Pomega.pair

@[expose] public section
```

```
namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, Theorem 4.4 (ii), pairing form.** A pair-as-function
projects to its components. -/
theorem theorem_4_4.ii (x y : Pomega) :
    funOf (pairSeq x y) (ofNat 0) = x ∧
      funOf (pairSeq x y) (ofNat 1) = y :=
  ⟨seq2.zero x y, seq2.one x y⟩

/-- **Scott 1976, Theorem 4.4 (The product theorem), core.** -/
theorem theorem_4_4 :
    (∀ x y, funOf (pairSeq x y) (ofNat 0) = x ∧
      funOf (pairSeq x y) (ofNat 1) = y) ∧
      Retract botElem :=
  ⟨fun x y => theorem_4_4.ii x y, bot_isRetract⟩

end DomainTheory.DataTypesAsLattices
```

theorem_4_5.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.dcondSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.Retract
public import DomainTheory.DataTypesAsLattices.Retracts.pairSeq
public import DomainTheory.DataTypesAsLattices.Retracts.bot_isRetract
public import DomainTheory.DataTypesAsLattices.Lambda.dcondSet_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, Theorem 4.5 (i), strictness of sums.** The zero summand
is always selected on `⊥`. -/
theorem theorem_4_5.strict (a b : Pomega) :
    dcondSet botElem
      (pairSeq (ofNat 0) (funOf a botElem))
      (pairSeq (ofNat 1) (funOf b botElem)) = botElem :=
  dcondSet.bot _ _

/-- **Scott 1976, Theorem 4.5 (The sum theorem), core.** -/
theorem theorem_4_5 :
    dcondSet botElem
        (pairSeq (ofNat 0) (funOf a botElem))
        (pairSeq (ofNat 1) (funOf b botElem)) = botElem ∧
      Retract botElem :=
  ⟨theorem_4_5.strict a b, bot_isRetract⟩

end DomainTheory.DataTypesAsLattices
```

2.8 Enumeration and degrees

Historical note. Reduction to a few combinators is old in combinatory logic. The device special to this model is (3.1), which folds the arithmetical combinators into the one combinator $\mathbf{G}$. The Gödel numbering and the second recursion theorem are the standard constructions, written so that arithmetic and the type-free combinators share a notation, and the incompleteness theorem is the standard one in that notation. Theorem 3.5 is the argument of Myhill and Shepherdson (1955); Martin Hyland pointed out the connection. Theorem 3.6 restates enumeration degrees, as in Rogers (1967), Sasso (1975), and Gutteridge (1971). Scott notes that this area was underdeveloped beside the theory of Turing degrees. Semigroups of combinators had been discussed by Church and by Böhm, and Theorem 3.7 reads that discussion inside $\mathbf{RE} \cap \mathbf{FUN}$.

Dependencies. Inside Enumeration, each box is one Lean file. A box named as another folder in this chapter is that whole folder. A box named as another chapter is that whole chapter. The arrowhead points at the file or folder that imports, and the number is how many direct imports.



Figure 26: Enumeration and degrees.

A great advantage of the combinators from the formal point of view is that (bound) variables are eliminated in favor of "algebraic" combinations. The disadvantage is that the algebra is not all that pretty, as the combinations tend to get rather long and general laws are rather few. Nevertheless as a technical device it is mildly remarkable that we can have a notation for all r.e. sets requiring so few primitives. In the model defined here the reduction to one combinator rests on a lemma about conditionals:

$$(3.1) \quad \mathbf{cond}(x)(y)(\mathbf{cond}(x)(y)) = y.$$

Recall that $\mathbf{cond}$ (or $\supset$) is a test for zero, so that:

$$(3.2) \quad \mathbf{cond}(x)(y)(0) = x.$$

This suggests that we lump all combinators of Theorem 2.4 into this one:

$$(3.3) \quad \mathbf{G} = \mathbf{cond}(\langle \mathbf{suc}, \mathbf{pred}, \mathbf{cond}, \mathbf{K}, \mathbf{S} \rangle)(0).$$

We can then readily prove:

- Theorem 3.1
- Theorem 3.2
- Theorem 3.3
- Theorem 3.4
- Theorem 3.5
- Theorem 3.6

- Theorem 3.7

Icomb_SK.lean

```
module

public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Lambda.Kcomb
public import DomainTheory.DataTypesAsLattices.Lambda.Scomb
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.Kcomb_more
public import DomainTheory.DataTypesAsLattices.Lambda.Scomb_more

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable

open Nat.Partrec (Code)

open Nat.Partrec.Code

def Icomb_SK : Pomega := funOf (funOf Scomb Kcomb) Kcomb

namespace Icomb_SK

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem app (x : Pomega) : funOf Icomb_SK x = x := by
  rw [Icomb_SK, Scomb.beta3, Kcomb.beta2]

end Icomb_SK

end DomainTheory.DataTypesAsLattices
```

graph_mem_FUN.lean

```
module

public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Lambda.FUN
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.theorem_1_2
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
```

```
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import Mathlib.Order.CompleteLattice.Defs
@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem graph_mem_FUN {f : Pomega → Pomega} (hf : IsScottContinuous f) :
    graph f ∈ FUN := by
  change graph f = graph (funOf (graph f))
  rw [theorem_1_2.i hf]

end DomainTheory.DataTypesAsLattices
```

seq2_eq_SK.lean

```
module

public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.seq2
public import DomainTheory.DataTypesAsLattices.Lambda.predSet
public import DomainTheory.DataTypesAsLattices.Lambda.id_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.eq_2_6
public import DomainTheory.DataTypesAsLattices.Lambda.predC
public import DomainTheory.DataTypesAsLattices.Lambda.Kcomb
public import DomainTheory.DataTypesAsLattices.Lambda.Scomb
public import DomainTheory.DataTypesAsLattices.Lambda.condC
public import DomainTheory.DataTypesAsLattices.Lambda.SKK_eq_I
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.zeroC
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
public import DomainTheory.DataTypesAsLattices.Lambda.Kcomb_more
public import DomainTheory.DataTypesAsLattices.Lambda.Scomb_more
public import DomainTheory.DataTypesAsLattices.Lambda.condC_more
public import DomainTheory.DataTypesAsLattices.Lambda.predC_more
public import DomainTheory.DataTypesAsLattices.Lambda.predSet_more

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices
```

```
/-- `seq2 x y` is the combinator `S (S (K (cond x)) (S (K (cond y ⊥)) pred)) I`. -/
theorem seq2_eq_SK (x y : Pomega) :
    seq2 x y =
      funOf (funOf Scomb
          (funOf (funOf Scomb (funOf Kcomb (funOf condC x)))
            (funOf (funOf Scomb
                (funOf Kcomb (funOf (funOf condC y) (funOf predC zeroC))))
              predC)))
        (funOf (funOf Scomb Kcomb) Kcomb) := by
  have hinter :
      funOf (funOf Scomb
          (funOf Kcomb (funOf (funOf condC y) (funOf predC zeroC))))
        predC =
      graph (fun z => condSet (predSet z) y botElem) := by
    rw [Scomb.beta2]
    apply congrArg graph
    funext z
    rw [Kcomb.beta2, predC.app, predC.app]
    have : zeroC = ofNat 0 := rfl
    rw [this, eq_2_6, condC.beta3]
  have hF :
      funOf (funOf Scomb (funOf Kcomb (funOf condC x)))
        (funOf (funOf Scomb
            (funOf Kcomb (funOf (funOf condC y) (funOf predC zeroC))))
          predC) =
      graph (fun z =>
        funOf (funOf condC x) (condSet (predSet z) y botElem)) := by
    rw [Scomb.beta2, hinter]
    apply congrArg graph
    funext z
    rw [Kcomb.beta2]
    apply congrArg (funOf (funOf condC x))
    exact beta (theorem_1_3 (condSet.isScottContinuous_left y botElem)
      predSet.isScottContinuous) z
  rw [Scomb.beta2, hF, SKK_eq_I]
  apply congrArg graph
  funext z
  have hz : funOf (graph (fun x => x)) z = z := beta id_isScottContinuous z
  rw [hz]
  have hβ : funOf (graph (fun z =>
      funOf (funOf condC x) (condSet (predSet z) y botElem))) z =
      funOf (funOf condC x) (condSet (predSet z) y botElem) :=
    beta (theorem_1_3 (funOf.isScottContinuous (funOf condC x))
      (theorem_1_3 (condSet.isScottContinuous_left y botElem)
        predSet.isScottContinuous)) z
  rw [hβ, condC.beta3]

end DomainTheory.DataTypesAsLattices
```

seq2_combinatory.lean

```
module

public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Lambda.IsCombinatory
```

```
public import DomainTheory.DataTypesAsLattices.Lambda.seq2
public import DomainTheory.DataTypesAsLattices.Lambda.zeroC
public import DomainTheory.DataTypesAsLattices.Lambda.predC
public import DomainTheory.DataTypesAsLattices.Lambda.condC
public import DomainTheory.DataTypesAsLattices.Lambda.Kcomb
public import DomainTheory.DataTypesAsLattices.Lambda.Scomb
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Enumeration.seq2_eq_SK

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

namespace seq2

theorem combinatory {x y : Pomega}
    (hx : IsCombinatory x) (hy : IsCombinatory y) :
    IsCombinatory (seq2 x y) := by
  have hyb : IsCombinatory (funOf (funOf condC y) (funOf predC zeroC)) :=
    .app (.app .cond hy) (.app .pred .zero)
  have hinter : IsCombinatory
      (funOf (funOf Scomb (funOf Kcomb
          (funOf (funOf condC y) (funOf predC zeroC)))) predC) :=
    .app (.app .S (.app .K hyb)) .pred
  have hF : IsCombinatory
      (funOf (funOf Scomb (funOf Kcomb (funOf condC x)))
        (funOf (funOf Scomb
            (funOf Kcomb (funOf (funOf condC y) (funOf predC zeroC))))
          predC)) :=
    .app (.app .S (.app .K (.app .cond hx))) hinter
  have hI : IsCombinatory (funOf (funOf Scomb Kcomb) Kcomb) :=
    .app (.app .S .K) .K
  have h := IsCombinatory.app (IsCombinatory.app IsCombinatory.S hF) hI
  simpa [seq2_eq_SK x y] using h

end seq2

end DomainTheory.DataTypesAsLattices
```

Gpack.lean

```
module

public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Lambda.seq2
public import DomainTheory.DataTypesAsLattices.Lambda.sucC
public import DomainTheory.DataTypesAsLattices.Lambda.predC
public import DomainTheory.DataTypesAsLattices.Lambda.condC
public import DomainTheory.DataTypesAsLattices.Lambda.Kcomb
public import DomainTheory.DataTypesAsLattices.Lambda.Scomb
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Graph.funOf
```

```
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.IsCombinatory
public import DomainTheory.DataTypesAsLattices.Enumeration.seq2_combinatory
public import DomainTheory.DataTypesAsLattices.Lambda.sucC_more

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

/-- Packed 5-tuple of the remaining combinators of Theorem 2.4. -/
def Gpack : Pomega :=
  seq2 sucC (seq2 predC (seq2 condC (seq2 Kcomb Scomb)))

namespace Gpack

theorem ne_bot : Gpack = botElem := by
  intro h
  have hs : sucC = funOf botElem (ofNat 0) := by
    have := congrArg (fun w => funOf w (ofNat 0)) h
    simpa [Gpack, seq2.zero] using this
  have h1 : (1 : ℕ) ∈ funOf sucC (ofNat 0) := by
    rw [sucC.app]; exact ⟨0, rfl, rfl⟩
  rw [hs] at h1
  simp [funOf, botElem] at h1

theorem combinatory : IsCombinatory Gpack :=
  seq2.combinatory .suc
    (seq2.combinatory .pred
      (seq2.combinatory .cond (seq2.combinatory .K .S)))

theorem one : funOf Gpack (ofNat 1) =
    seq2 predC (seq2 condC (seq2 Kcomb Scomb)) := by
  simp [Gpack, seq2.one]

theorem zero : funOf Gpack (ofNat 0) = sucC := by
  simp [Gpack, seq2.zero]

end Gpack

end DomainTheory.DataTypesAsLattices
```

Gcomb.lean

```
module
public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.termDollarStep
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.zeroC
public import DomainTheory.DataTypesAsLattices.Lambda.condC
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
```

```
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.sumMed
public import DomainTheory.DataTypesAsLattices.Functionality.Zcomb
public import DomainTheory.DataTypesAsLattices.Functionality.sigmaJ
public import DomainTheory.DataTypesAsLattices.Retracts.arrowR
public import DomainTheory.DataTypesAsLattices.Retracts.iterateFrom
public import DomainTheory.DataTypesAsLattices.Retracts.lfpAbove
public import DomainTheory.DataTypesAsLattices.Retracts.Icomb
public import DomainTheory.DataTypesAsLattices.Retracts.intF
public import DomainTheory.DataTypesAsLattices.Retracts.intR
public import DomainTheory.DataTypesAsLattices.Retracts.boolR
public import DomainTheory.DataTypesAsLattices.Enumeration.Gpack
public import DomainTheory.DataTypesAsLattices.Lambda.eq_2_7_bot
public import DomainTheory.DataTypesAsLattices.Lambda.eq_2_7_succ
public import DomainTheory.DataTypesAsLattices.Lambda.eq_2_7_zero

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices
/-- **Scott 1976, (3.3).** `G = cond(⟨suc, pred, cond, K, S⟩)(0)`. -/
def Gcomb : Pomega :=
  graph (fun z => condSet z Gpack zeroC)

end DomainTheory.DataTypesAsLattices
```

Subalgebra.lean

```
module

public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Enumeration.Gcomb

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable

open Nat.Partrec (Code)

open Nat.Partrec.Code

/-- Combinatory subalgebra: contains `G` and is closed under application. -/
class Subalgebra (A : Set Pomega) : Prop where
  out : Gcomb ∈ A ∧ ∀ {{u v}}, u ∈ A → v ∈ A → funOf u v ∈ A

end DomainTheory.DataTypesAsLattices
```

not_mem_zero_Gcomb.lean

```
module

public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.pair_zero_zero
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Enumeration.Gcomb

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

/-- `0 ∈ G`, so `G` is a positive non-zero test. -/
theorem not_mem_zero_Gcomb : 0 ∈ Gcomb := by
  intro h
  rcases h with ⟨n, m, hm, hmem⟩
  have hnm : n = 0 ∧ m = 0 := by
    have : pair n m = 0 := hm.symm
    have : pair n m = pair 0 0 := by simpa [pair_zero_zero] using this
    exact pair_inj this
  rcases hnm with ⟨rfl, rfl⟩
  simp [e_zero, condSet] at hmem

end DomainTheory.DataTypesAsLattices
```

Gcomb_more.lean

```
module

public import DomainTheory.DataTypesAsLattices.Enumeration.Gcomb
public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.zeroC
public import DomainTheory.DataTypesAsLattices.Lambda.condC
public import DomainTheory.DataTypesAsLattices.Lambda.condC_more
public import DomainTheory.DataTypesAsLattices.Lambda.IsCombinatory
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Enumeration.Gpack
public import DomainTheory.DataTypesAsLattices.Lambda.eq_2_7_bot
public import DomainTheory.DataTypesAsLattices.Lambda.eq_2_7_succ
public import DomainTheory.DataTypesAsLattices.Lambda.eq_2_7_zero
public import DomainTheory.DataTypesAsLattices.Enumeration.not_mem_zero_Gcomb
public import DomainTheory.DataTypesAsLattices.Lambda.eq_2_7_pos
public import DomainTheory.DataTypesAsLattices.Computability.IsRE
public import DomainTheory.DataTypesAsLattices.Computability.combinatory_isRE

@[expose] public section
```

```
namespace DomainTheory.DataTypesAsLattices

namespace Gcomb

theorem eq_condC : Gcomb = funOf (funOf condC Gpack) zeroC :=
  (condC.beta2 Gpack zeroC).symm

theorem app (z : Pomega) :
    funOf Gcomb z = condSet z Gpack zeroC :=
  beta (condSet.isScottContinuous_left Gpack zeroC) z

theorem zero : funOf Gcomb (ofNat 0) = Gpack := by
  rw [app, eq_2_7_zero]

theorem bot : funOf Gcomb botElem = botElem := by
  rw [app, eq_2_7_bot]

theorem succ (k : ℕ) : funOf Gcomb (ofNat (k + 1)) = zeroC := by
  rw [app, eq_2_7_succ]

theorem combinatory : IsCombinatory Gcomb := by
  simpa [eq_condC] using
    IsCombinatory.app (IsCombinatory.app IsCombinatory.cond Gpack.combinatory)
      IsCombinatory.zero

theorem app_self : funOf Gcomb Gcomb = zeroC := by
  have hne : Gcomb = botElem := by
    intro h
    have : Gpack = funOf botElem (ofNat 0) := by
      have := congrArg (fun w => funOf w (ofNat 0)) h
      simpa [zero] using this
    have hbot : funOf botElem (ofNat 0) = botElem := by
      ext; simp [funOf, botElem]
    exact Gpack.ne_bot (this.trans hbot)
  rw [app]
  exact eq_2_7_pos Gpack zeroC Gcomb not_mem_zero_Gcomb hne

theorem isRE : IsRE Gcomb :=
  combinatory_isRE combinatory

end Gcomb

end DomainTheory.DataTypesAsLattices
```

Deg.lean

```
module

public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Lambda.IsCombinatory
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.Kcomb
public import DomainTheory.DataTypesAsLattices.Lambda.Scomb
public import DomainTheory.DataTypesAsLattices.Enumeration.Icomb_SK
```

```
public import DomainTheory.DataTypesAsLattices.Enumeration.Gcomb
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Enumeration.Subalgebra
public import DomainTheory.DataTypesAsLattices.Enumeration.Gcomb_more
public import DomainTheory.DataTypesAsLattices.Lambda.Kcomb_more
public import DomainTheory.DataTypesAsLattices.Lambda.Scomb_more

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable

open Nat.Partrec (Code)

open Nat.Partrec.Code

/-- **Scott 1976, Definition.** Enumeration degree of `a`. -/
def Deg (a : Pomega) : Set Pomega :=
  {u | ∃ r, IsCombinatory r ∧ u = funOf r a}

namespace Deg

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem isSubalgebra (a : Pomega) : Subalgebra (Deg a) := by
  constructor
  constructor
  · refine ⟨funOf Kcomb Gcomb, IsCombinatory.app .K Gcomb.combinatory, ?_⟩
    rw [Kcomb.beta2]
  · intro x y hx hy
    obtain ⟨r, hr, rfl⟩ := hx
    obtain ⟨s, hs, rfl⟩ := hy
    refine ⟨funOf (funOf Scomb r) s, .app (.app .S hr) hs, ?_⟩
    rw [Scomb.beta3]

theorem self (a : Pomega) : a ∈ Deg a :=
  ⟨Icomb_SK, .app (.app .S .K) .K, (Icomb_SK.app a).symm⟩

end Deg

end DomainTheory.DataTypesAsLattices
```

seq2_left_isScottContinuous.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.seq2
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
```

```
public import DomainTheory.DataTypesAsLattices.Lambda.predSet

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

theorem seq2_left_isScottContinuous (b : Pomega) :
    IsScottContinuous (fun a => seq2 a b) :=
  graph_const_isScottContinuous
    (fun a z => condSet z a (condSet (predSet z) b botElem))
    (fun z => condSet.isScottContinuous_mid z _)

end DomainTheory.DataTypesAsLattices
```

seq2_right_isScottContinuous.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.seq2
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.predSet

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

theorem seq2_right_isScottContinuous (a : Pomega) :
    IsScottContinuous (fun b => seq2 a b) :=
  graph_const_isScottContinuous
    (fun b z => condSet z a (condSet (predSet z) b botElem))
    (fun z => theorem_1_3 (condSet.isScottContinuous_right z a)
      (condSet.isScottContinuous_mid (predSet z) botElem))

end DomainTheory.DataTypesAsLattices
```

Rcomb.lean

```
module

public import DomainTheory.DataTypesAsLattices.Enumeration.
    seq2_right_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.seq2
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Lambda.FUN
public import DomainTheory.DataTypesAsLattices.Graph.beta
```

```
public import DomainTheory.DataTypesAsLattices.Enumeration.graph_mem_FUN

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (3.13)-(3.14).** Semigroup generators. -/
def Rcomb : Pomega := graph (fun x => seq2 (ofNat 0) x)

namespace Rcomb

theorem app (x : Pomega) : funOf Rcomb x = seq2 (ofNat 0) x :=
  beta (seq2_right_isScottContinuous (ofNat 0)) x

theorem app_zero (x : Pomega) :
    funOf (funOf Rcomb x) (ofNat 0) = ofNat 0 := by
  rw [app, seq2.zero]

theorem mem_FUN : Rcomb ∈ FUN :=
  graph_mem_FUN (seq2_right_isScottContinuous (ofNat 0))

end Rcomb

end DomainTheory.DataTypesAsLattices
```

seqCons_right_isScottContinuous.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.seqCons
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.predSet
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

theorem seqCons_right_isScottContinuous (x : Pomega) :
    IsScottContinuous (fun xs => seqCons x xs) :=
  graph_const_isScottContinuous
    (fun xs z => condSet z x (funOf xs (predSet z)))
    (fun z => theorem_1_3 (condSet.isScottContinuous_right z x)
      (funOf.isScottContinuous_left (predSet z)))

end DomainTheory.DataTypesAsLattices
```

barPos.lean

```
module

public import DomainTheory.DataTypesAsLattices.Enumeration.
    seqCons_right_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Enumeration.seq2_left_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Enumeration.
    seq2_right_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Lambda.eq_2_22
public import DomainTheory.DataTypesAsLattices.Lambda.eq_2_7_succ
public import DomainTheory.DataTypesAsLattices.Lambda.eq_2_7_zero
public import DomainTheory.DataTypesAsLattices.Lambda.seq2
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.seqCons
public import DomainTheory.DataTypesAsLattices.Lambda.dollarStepBody
public import DomainTheory.DataTypesAsLattices.Lambda.predSet
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (3.15).** Positive branch of `u`: `⟨1, u, x₁⟩`. -/
def barPos (u x : Pomega) : Pomega :=
  seqCons (ofNat 1) (seq2 u (funOf x (ofNat 1)))

namespace barPos

theorem isScottContinuous_u (x : Pomega) :
    IsScottContinuous (fun u => barPos u x) :=
  theorem_1_3 (seqCons_right_isScottContinuous (ofNat 1))
    (seq2_left_isScottContinuous (funOf x (ofNat 1)))

theorem isScottContinuous_x (u : Pomega) :
    IsScottContinuous (fun x => barPos u x) :=
  theorem_1_3 (f := fun xs => seqCons (ofNat 1) xs)
    (g := fun x => seq2 u (funOf x (ofNat 1)))
    (seqCons_right_isScottContinuous (ofNat 1))
    (theorem_1_3 (seq2_right_isScottContinuous u)
      (funOf.isScottContinuous_left (ofNat 1)))

theorem zero (u x : Pomega) :
    funOf (barPos u x) (ofNat 0) = ofNat 1 := by
  simp [barPos, _root_.DomainTheory.DataTypesAsLattices.eq_2_22, eq_2_7_zero]

theorem one (u x : Pomega) :
    funOf (barPos u x) (ofNat 1) = u := by
  rw [barPos, _root_.DomainTheory.DataTypesAsLattices.eq_2_22]
  have hpred : predSet (ofNat 1) = ofNat 0 := by
    ext k; simp [predSet, ofNat]
  rw [hpred, eq_2_7_succ, seq2.zero]
```

```
theorem two (u x : Pomega) :
    funOf (barPos u x) (ofNat 2) = funOf x (ofNat 1) := by
  rw [barPos, _root_.DomainTheory.DataTypesAsLattices.eq_2_22]
  have hpred : predSet (ofNat 2) = ofNat 1 := by
    ext k; simp [predSet, ofNat]
  rw [hpred, eq_2_7_succ, seq2.one]

end barPos

end DomainTheory.DataTypesAsLattices
```

barStepBody.lean

```
module

public import DomainTheory.DataTypesAsLattices.Enumeration.barPos
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.dollarStepBody
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (3.15).** Step of `u = λx. x₀ ? ⟨1, u, x₁⟩, overline{u(x₁)(x₂)}`. -/
def barStepBody (B u x : Pomega) : Pomega :=
  condSet (funOf x (ofNat 0)) (barPos u x)
    (funOf (funOf B (funOf u (funOf x (ofNat 1)))) (funOf x (ofNat 2)))

namespace barStepBody

theorem isScottContinuous_B (u x : Pomega) :
    IsScottContinuous (fun B => barStepBody B u x) :=
  theorem_1_3 (condSet.isScottContinuous_right (funOf x (ofNat 0)) (barPos u x))
    (theorem_1_3
      (f := fun w => funOf w (funOf x (ofNat 2)))
      (g := fun B => funOf B (funOf u (funOf x (ofNat 1))))
      (funOf.isScottContinuous_left (funOf x (ofNat 2)))
      (funOf.isScottContinuous_left (funOf u (funOf x (ofNat 1)))))

theorem isScottContinuous_u (B x : Pomega) :
    IsScottContinuous (fun u => barStepBody B u x) :=
  theorem_1_3.tuple
    (fun y => condSet.isScottContinuous_mid (funOf x (ofNat 0)) y)
    (fun w => condSet.isScottContinuous_right (funOf x (ofNat 0)) w)
    (barPos.isScottContinuous_u x)
    (theorem_1_3
      (f := fun w => funOf w (funOf x (ofNat 2)))
```

```
      (g := fun u => funOf B (funOf u (funOf x (ofNat 1))))
      (funOf.isScottContinuous_left (funOf x (ofNat 2)))
      (theorem_1_3 (funOf.isScottContinuous B)
        (funOf.isScottContinuous_left (funOf x (ofNat 1)))))

theorem isScottContinuous_x (B u : Pomega) :
    IsScottContinuous (fun x => barStepBody B u x) :=
  theorem_1_3.nary
    (fun y z => condSet.isScottContinuous_left y z)
    (fun tes z => condSet.isScottContinuous_mid tes z)
    (fun tes th => condSet.isScottContinuous_right tes th)
    (funOf.isScottContinuous_left (ofNat 0))
    (barPos.isScottContinuous_x u)
    (theorem_1_3.tuple
      (fun y => funOf.isScottContinuous_left y)
      (fun w => funOf.isScottContinuous w)
      (theorem_1_3 (funOf.isScottContinuous B)
        (theorem_1_3 (funOf.isScottContinuous u)
          (funOf.isScottContinuous_left (ofNat 1))))
      (funOf.isScottContinuous_left (ofNat 2)))

end barStepBody

end DomainTheory.DataTypesAsLattices
```

barStep.lean

```
module

public import DomainTheory.DataTypesAsLattices.Enumeration.barStepBody
public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import Mathlib.Order.CompleteLattice.Defs
public import DomainTheory.DataTypesAsLattices.Lambda.dollarStepBody

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

def barStep (B : Pomega) : Pomega :=
  graph (fun u => graph (fun x => barStepBody B u x))

namespace barStep

open Encodable
```

```
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem app (B u : Pomega) :
    funOf (barStep B) u = graph (fun x => barStepBody B u x) :=
  beta (graph_const_isScottContinuous (fun u x => barStepBody B u x)
    (fun x => barStepBody.isScottContinuous_u B x)) u

theorem isScottContinuous : IsScottContinuous barStep :=
  graph_const_isScottContinuous
    (fun B u => graph (fun x => barStepBody B u x))
    (fun u => graph_const_isScottContinuous
      (fun B x => barStepBody B u x)
      (fun x => barStepBody.isScottContinuous_B u x))

end barStep

end DomainTheory.DataTypesAsLattices
```

enumBottomTerm.lean

```
module

public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Lambda.Term
public import DomainTheory.DataTypesAsLattices.Graph.lam
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.envSet
public import DomainTheory.DataTypesAsLattices.Lambda.interp
public import DomainTheory.DataTypesAsLattices.Enumeration.Avoids
public import DomainTheory.DataTypesAsLattices.Lambda.eq_2_14
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.graph

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable

open Nat.Partrec (Code)

open Nat.Partrec.Code

/-- A closed divergent term denoting bottom. -/
def enumBottomTerm : Term :=
  .app (.lam 9 (.app (.var 9) (.var 9)))
    (.lam 9 (.app (.var 9) (.var 9)))

namespace enumBottomTerm

open Encodable
open Nat.Partrec (Code)
```

```
open Nat.Partrec.Code

theorem avoids (i : ℕ) : enumBottomTerm.Avoids i := by
  unfold enumBottomTerm
  constructor <;>
    (by_cases h : 9 = i
     · exact Or.inl h
     · exact Or.inr ⟨h, h⟩)

theorem interp (? : ℕ → Pomega) :
    _root_.DomainTheory.DataTypesAsLattices.interp enumBottomTerm ? = botElem := by
  simp only [enumBottomTerm, _root_.DomainTheory.DataTypesAsLattices.interp, envSet,
    Function.update]
  exact _root_.DomainTheory.DataTypesAsLattices.eq_2_14

end enumBottomTerm

end DomainTheory.DataTypesAsLattices
```

valNat.lean

```
module

public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Enumeration.applyNat
public import DomainTheory.DataTypesAsLattices.Enumeration.Gcomb
public import DomainTheory.DataTypesAsLattices.Graph.Basic.notation3
public import DomainTheory.DataTypesAsLattices.Lambda.IsCombinatory
public import DomainTheory.DataTypesAsLattices.Enumeration.Gcomb_more

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices
/-- **Scott 1976, (3.7) / Theorem 3.2.** `val(0) = G` and
`val(apply(n)(m)) = val(n)(val(m))`. -/
noncomputable def valNat : ℕ → Pomega :=
  Nat.strongRec fun n rec =>
    match n with
    | 0 => Gcomb
    | n + 1 =>
        rec (unpair n).1
          (Nat.lt_of_lt_of_eq
            (pair_lt_left (unpair n).1 (unpair n).2)
            (congrArg (fun x => x + 1) (pair_unpair n))) ?
        rec (unpair n).2
          (Nat.lt_of_lt_of_eq
            (pair_lt_right (unpair n).1 (unpair n).2)
            (congrArg (fun x => x + 1) (pair_unpair n)))

namespace valNat
```

```
/-- **Scott 1976, Theorem 3.2 (i).** `val(0) = G`. -/
theorem zero : valNat 0 = Gcomb := by
  unfold valNat
  rw [Nat.strongRec]

theorem succ (n : ℕ) :
    valNat (n + 1) = valNat (unpair n).1 ? valNat (unpair n).2 := by
  unfold valNat
  rw [Nat.strongRec]

/-- **Scott 1976, Theorem 3.2 (ii).** `val(apply(n)(m)) = val n (val m)`. -/
theorem app (n m : ℕ) :
    valNat (applyNat n m) = valNat n ? valNat m := by
  rw [applyNat, succ, unpair_pair]

theorem combinatory : ∀ n, IsCombinatory (valNat n)
  | 0 => zero ▸ Gcomb.combinatory
  | n + 1 =>
    succ n ▸
      .app (combinatory (unpair n).1) (combinatory (unpair n).2)
decreasing_by
  · exact (pair_lt_left (unpair n).1 (unpair n).2).trans_le (by
      have := pair_unpair n
      omega)
  · exact (pair_lt_right (unpair n).1 (unpair n).2).trans_le (by
      have := pair_unpair n
      omega)

end valNat

end DomainTheory.DataTypesAsLattices
```

interp_avoids.lean

```
module

public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Lambda.Term
public import DomainTheory.DataTypesAsLattices.Lambda.envSet
public import DomainTheory.DataTypesAsLattices.Lambda.interp
public import DomainTheory.DataTypesAsLattices.Lambda.succSet
public import DomainTheory.DataTypesAsLattices.Lambda.predSet
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.lam
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Enumeration.Avoids
@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code
```

```
theorem interp_avoids {i : ℕ} (t : Term) (? : ℕ → Pomega) (v : Pomega)
    (h : t.Avoids i) : interp t (envSet ? i v) = interp t ? := by
  induction t generalizing ? i with
  | var j =>
    have hji : j = i := h
    simp [_root_.DomainTheory.DataTypesAsLattices.interp, envSet, Function.update, hji
    ]
  | zero => rfl
  | succ t ih => exact congrArg succSet (ih (? := ?) (i := i) h)
  | pred t ih => exact congrArg predSet (ih (? := ?) (i := i) h)
  | cond z x y ihz ihx ihy =>
    simp only [_root_.DomainTheory.DataTypesAsLattices.interp]
    rw [ihz (? := ?) (i := i) h.1, ihx (? := ?) (i := i) h.2.1,
      ihy (? := ?) (i := i) h.2.2]
  | app u x ihu ihx =>
    simp only [_root_.DomainTheory.DataTypesAsLattices.interp]
    rw [ihu (? := ?) (i := i) h.1, ihx (? := ?) (i := i) h.2]
  | lam j body ih =>
    simp only [_root_.DomainTheory.DataTypesAsLattices.interp]
    apply congrArg graph
    funext w
    cases h with
    | inl hji =>
      subst hji
      simp [envSet]
    | inr hb =>
      by_cases hji : j = i
      · subst hji
        simp [envSet]
      · rw [envSet.comm (? := ?) (i := i) (j := j) (Ne.symm hji) v w,
          ih (? := envSet ? j w) (i := i) hb]

end DomainTheory.DataTypesAsLattices
```

seq2Term.lean

```
module

public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Lambda.Term
public import DomainTheory.DataTypesAsLattices.Graph.lam
public import DomainTheory.DataTypesAsLattices.Enumeration.enumBottomTerm
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.envSet
public import DomainTheory.DataTypesAsLattices.Enumeration.interp_avoids
public import DomainTheory.DataTypesAsLattices.Lambda.interp
public import DomainTheory.DataTypesAsLattices.Lambda.seq2

@[expose] public section

set_option maxHeartbeats 2000000
```

```
namespace DomainTheory.DataTypesAsLattices

open Encodable

open Nat.Partrec (Code)

open Nat.Partrec.Code

/-- Term-level binary sequence constructor. -/
def seq2Term (a b : Term) : Term :=
  .lam 8 (.cond (.var 8) a
    (.cond (.pred (.var 8)) b enumBottomTerm))

namespace seq2Term

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem avoids8 (a b : Term) : (seq2Term a b).Avoids 8 :=
  Or.inl rfl

theorem interp (a b : Term) (? : ℕ → Pomega)
    (ha : a.Avoids 8) (hb : b.Avoids 8) :
    interp (seq2Term a b) ? = seq2 (interp a ?) (interp b ?) := by
  unfold seq2Term seq2
  apply congrArg graph
  funext z
  simp only [_root_.DomainTheory.DataTypesAsLattices.interp]
  rw [interp_avoids a ? z ha, interp_avoids b ? z hb, enumBottomTerm.interp]
  simp [envSet, Function.update]

end seq2Term

end DomainTheory.DataTypesAsLattices
```

seqConsTerm.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.interp
public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Lambda.Term
public import DomainTheory.DataTypesAsLattices.Graph.lam
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.seqCons
public import DomainTheory.DataTypesAsLattices.Enumeration.interp_avoids
public import DomainTheory.DataTypesAsLattices.Enumeration.Avoids
public import DomainTheory.DataTypesAsLattices.Lambda.envSet

@[expose] public section
```

```
set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable

open Nat.Partrec (Code)

open Nat.Partrec.Code

/-- Term-level sequence cons constructor. -/
def seqConsTerm (a xs : Term) : Term :=
  .lam 7 (.cond (.var 7) a (.app xs (.pred (.var 7))))

namespace seqConsTerm

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem interp (a xs : Term) (? : ℕ → Pomega)
    (ha : a.Avoids 7) (hxs : xs.Avoids 7) :
    interp (seqConsTerm a xs) ? =
      seqCons (interp a ?) (interp xs ?) := by
  unfold seqConsTerm seqCons
  apply congrArg graph
  funext z
  simp only[_root_.DomainTheory.DataTypesAsLattices.interp]
  rw [interp_avoids a ? z ha, interp_avoids xs ? z hxs]
  simp[envSet, Function.update]

end seqConsTerm

end DomainTheory.DataTypesAsLattices
```

barPosTerm.lean

```
module

public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Lambda.Term
public import DomainTheory.DataTypesAsLattices.Enumeration.seq2Term
public import DomainTheory.DataTypesAsLattices.Enumeration.seqConsTerm
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.succSet
public import DomainTheory.DataTypesAsLattices.Lambda.interp
public import DomainTheory.DataTypesAsLattices.Enumeration.enumBottomTerm
public import DomainTheory.DataTypesAsLattices.Enumeration.Avoids
public import DomainTheory.DataTypesAsLattices.Enumeration.barPos
public import DomainTheory.DataTypesAsLattices.Lambda.succSet_more
public import DomainTheory.DataTypesAsLattices.Lambda.seqCons
public import DomainTheory.DataTypesAsLattices.Lambda.seq2
public import DomainTheory.DataTypesAsLattices.Graph.funOf
```

```
@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable

open Nat.Partrec (Code)

open Nat.Partrec.Code

/-- Term for the positive branch `⟨1,u,x₁⟩` of Scott's bar operator. -/
def barPosTerm : Term :=
  seqConsTerm (.succ .zero)
    (seq2Term (.var 1) (.app (.var 2) (.succ .zero)))

namespace barPosTerm

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem interp (? : ℕ → Pomega) :
    _root_.DomainTheory.DataTypesAsLattices.interp barPosTerm ? = barPos (? 1) (? 2)
    := by
  have hzero : Term.Avoids 7 (.succ .zero) := by simp [Term.Avoids]
  have hseq : Term.Avoids 7
      (seq2Term (.var 1) (.app (.var 2) (.succ .zero))) := by
    change _ ∨ _
    refine Or.inr ?_
    refine ⟨?_, ?_, ?_⟩
    · exact (by omega : (8 : ℕ) = 7)
    · exact (by omega : (1 : ℕ) = 7)
    · refine ⟨?_, ?_, enumBottomTerm.avoids 7⟩
      · exact (by omega : (8 : ℕ) = 7)
      · exact ⟨(by omega : (2 : ℕ) = 7), trivial⟩
  have h1 : Term.Avoids 8 (.var 1) := (by omega : (1 : ℕ) = 8)
  have h2 : Term.Avoids 8 (.app (.var 2) (.succ .zero)) :=
    ⟨(by omega : (2 : ℕ) = 8), trivial⟩
  rw [barPosTerm, seqConsTerm.interp _ _ _ hzero hseq,
    seq2Term.interp _ _ _ h1 h2]
  simp only [_root_.DomainTheory.DataTypesAsLattices.interp, barPos]
  simp [succSet.ofNat]

end barPosTerm

end DomainTheory.DataTypesAsLattices
```

barStepTerm.lean

```
module

public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Lambda.Term
```

```
public import DomainTheory.DataTypesAsLattices.Graph.lam
public import DomainTheory.DataTypesAsLattices.Enumeration.barPosTerm
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.envSet
public import DomainTheory.DataTypesAsLattices.Lambda.succSet
public import DomainTheory.DataTypesAsLattices.Lambda.interp
public import DomainTheory.DataTypesAsLattices.Enumeration.barStepBody
public import DomainTheory.DataTypesAsLattices.Enumeration.Avoids
public import DomainTheory.DataTypesAsLattices.Lambda.succSet_more
public import DomainTheory.DataTypesAsLattices.Enumeration.barStep

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable

open Nat.Partrec (Code)

open Nat.Partrec.Code

/-- Closed term for the functional whose least fixed point is `barComb`. -/
def barStepTerm : Term :=
  .lam 0 (.lam 1 (.lam 2
    (.cond (.app (.var 2) .zero) barPosTerm
      (.app
        (.app (.var 0)
          (.app (.var 1) (.app (.var 2) (.succ .zero))))
        (.app (.var 2) (.succ (.succ .zero)))))))

namespace barStepTerm

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem interp :
    interp barStepTerm (fun _ => botElem) = graph barStep := by
  unfold barStepTerm barStep
  simp only [_root_.DomainTheory.DataTypesAsLattices.interp, envSet, Function.
    update_self]
  apply congrArg graph
  funext B
  apply congrArg graph
  funext u
  apply congrArg graph
  funext x
  simp [barStepBody, barPosTerm.interp, Function.update,
    succSet.ofNat]

end barStepTerm
```

```
end DomainTheory.DataTypesAsLattices
```

barComb.lean

```
module

public import DomainTheory.DataTypesAsLattices.Enumeration.barStep
public import DomainTheory.DataTypesAsLattices.FixedPoint.fix
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Lambda.Ycomb
public import DomainTheory.DataTypesAsLattices.Lambda.interp
public import DomainTheory.DataTypesAsLattices.Enumeration.barStepTerm
public import DomainTheory.DataTypesAsLattices.Lambda.Ycomb_more
public import DomainTheory.DataTypesAsLattices.Lambda.theorem_2_5

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (3.15).** The bar combinator as a least fixed point. -/
def barComb : Pomega := fix barStep

namespace barComb

theorem combinatory : IsCombinatory barComb := by
  have hY : funOf Ycomb (interp barStepTerm (fun _ => botElem)) = barComb := by
    rw [barStepTerm.interp, theorem_2_5 barStep.isScottContinuous]
    rfl
  exact hY ▸ IsCombinatory.app Ycomb.combinatory (theorem_2_4.closed barStepTerm)

end barComb

end DomainTheory.DataTypesAsLattices
```

eq_3_15.lean

```
module

public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_4
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Enumeration.barStep
public import DomainTheory.DataTypesAsLattices.Enumeration.barStepBody
public import DomainTheory.DataTypesAsLattices.Enumeration.barComb
@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices
```

```
open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

/-- **Scott 1976, (3.15).** Unfolding of the least bar combinator. -/
theorem eq_3_15 (u : Pomega) :
    funOf barComb u = graph (fun x => barStepBody barComb u x) := by
  have hfix : barStep barComb = barComb :=
    (theorem_1_4 barStep.isScottContinuous).1
  have ha := barStep.app barComb u
  rwa [hfix] at ha

end DomainTheory.DataTypesAsLattices
```

Gbar.lean

```
module

public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Enumeration.barComb
public import DomainTheory.DataTypesAsLattices.Enumeration.eq_3_15
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.theorem_1_2
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import Mathlib.Order.CompleteLattice.Defs
public import DomainTheory.DataTypesAsLattices.Lambda.FUN
public import DomainTheory.DataTypesAsLattices.Enumeration.barStepBody
public import DomainTheory.DataTypesAsLattices.Enumeration.Gcomb
public import DomainTheory.DataTypesAsLattices.Enumeration.Gcomb_more

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

/-- Scott's `G`. -/
def Gbar : Pomega := funOf barComb Gcomb

namespace Gbar

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code
```

```
theorem combinatory : IsCombinatory Gbar :=
  .app barComb.combinatory Gcomb.combinatory

theorem mem_FUN : Gbar ∈ FUN := by
  change Gbar = graph (funOf Gbar)
  have h : Gbar = graph (fun x => barStepBody barComb Gcomb x) :=
    _root_.DomainTheory.DataTypesAsLattices.eq_3_15 Gcomb
  have hf : IsScottContinuous (fun x => barStepBody barComb Gcomb x) :=
    barStepBody.isScottContinuous_x barComb Gcomb
  rw [h, theorem_1_2.i hf]

end Gbar

end DomainTheory.DataTypesAsLattices
```

GeneratedFromG.lean

```
module

public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Enumeration.Gcomb

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

/-- Generation from Scott's single generator `G` (since `G(G) = 0`). -/
inductive GeneratedFromG : Pomega → Prop
  | G : GeneratedFromG Gcomb
  | app {u v} : GeneratedFromG u → GeneratedFromG v →
      GeneratedFromG (funOf u v)

end DomainTheory.DataTypesAsLattices
```

Kcomb_from_G.lean

```
module

public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.zeroC
public import DomainTheory.DataTypesAsLattices.Lambda.Kcomb
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Enumeration.Gpack
public import DomainTheory.DataTypesAsLattices.Enumeration.Gcomb
public import DomainTheory.DataTypesAsLattices.Enumeration.Gcomb_more

@[expose] public section

set_option maxHeartbeats 2000000
```

```
namespace DomainTheory.DataTypesAsLattices

theorem Kcomb_from_G :
    Kcomb =
      funOf (funOf (funOf (funOf (funOf Gcomb zeroC) (ofNat 1)) (ofNat 1))
        (ofNat 1)) zeroC := by
  have : zeroC = ofNat 0 := rfl
  simp [this, Gcomb.zero, Gpack.one, seq2.one, seq2.one, seq2.zero]

end DomainTheory.DataTypesAsLattices
```

Scomb_from_G.lean

```
module

public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.zeroC
public import DomainTheory.DataTypesAsLattices.Lambda.Scomb
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Enumeration.Gpack
public import DomainTheory.DataTypesAsLattices.Enumeration.Gcomb
public import DomainTheory.DataTypesAsLattices.Enumeration.Gcomb_more

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

theorem Scomb_from_G :
    Scomb =
      funOf (funOf (funOf (funOf (funOf Gcomb zeroC) (ofNat 1)) (ofNat 1))
        (ofNat 1)) (ofNat 1) := by
  have : zeroC = ofNat 0 := rfl
  simp [this, Gcomb.zero, Gpack.one, seq2.one, seq2.one, seq2.one]

end DomainTheory.DataTypesAsLattices
```

condC_from_G.lean

```
module

public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.zeroC
public import DomainTheory.DataTypesAsLattices.Lambda.condC
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Enumeration.Gpack
public import DomainTheory.DataTypesAsLattices.Enumeration.Gcomb
public import DomainTheory.DataTypesAsLattices.Enumeration.Gcomb_more

@[expose] public section

set_option maxHeartbeats 2000000
```

```
namespace DomainTheory.DataTypesAsLattices

theorem condC_from_G :
    condC =
      funOf (funOf (funOf (funOf Gcomb zeroC) (ofNat 1)) (ofNat 1))
        zeroC := by
  have : zeroC = ofNat 0 := rfl
  simp [this, Gcomb.zero, Gpack.one, seq2.one, seq2.zero]

end DomainTheory.DataTypesAsLattices
```

predC_from_G.lean

```
module

public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.zeroC
public import DomainTheory.DataTypesAsLattices.Lambda.predC
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Enumeration.Gpack
public import DomainTheory.DataTypesAsLattices.Enumeration.Gcomb
public import DomainTheory.DataTypesAsLattices.Enumeration.Gcomb_more

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

theorem predC_from_G :
    predC = funOf (funOf (funOf Gcomb zeroC) (ofNat 1)) zeroC := by
  have : zeroC = ofNat 0 := rfl
  simp [this, Gcomb.zero, Gpack.one, seq2.zero]

end DomainTheory.DataTypesAsLattices
```

sucC_from_G.lean

```
module

public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.zeroC
public import DomainTheory.DataTypesAsLattices.Lambda.sucC
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Enumeration.Gpack
public import DomainTheory.DataTypesAsLattices.Enumeration.Gcomb
public import DomainTheory.DataTypesAsLattices.Enumeration.Gcomb_more

@[expose] public section

set_option maxHeartbeats 2000000
```

```
namespace DomainTheory.DataTypesAsLattices

/-- The six constants are recovered from `G` and `0` by projection. -/
theorem sucC_from_G : sucC = funOf (funOf Gcomb zeroC) zeroC := by
  have : zeroC = ofNat 0 := rfl
  simp [this, Gcomb.zero, Gpack.zero]

end DomainTheory.DataTypesAsLattices
```

theorem_3_1.lean

```
module

public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Lambda.IsCombinatory
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.zeroC
public import DomainTheory.DataTypesAsLattices.Lambda.sucC
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Enumeration.GeneratedFromG
public import DomainTheory.DataTypesAsLattices.Enumeration.sucC_from_G
public import DomainTheory.DataTypesAsLattices.Enumeration.predC_from_G
public import DomainTheory.DataTypesAsLattices.Enumeration.condC_from_G
public import DomainTheory.DataTypesAsLattices.Enumeration.Kcomb_from_G
public import DomainTheory.DataTypesAsLattices.Enumeration.Scomb_from_G
public import DomainTheory.DataTypesAsLattices.Enumeration.sucC_zero
public import DomainTheory.DataTypesAsLattices.Enumeration.Gcomb
public import DomainTheory.DataTypesAsLattices.Enumeration.Gcomb_more
@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

theorem theorem_3_1 {u : Pomega} :
    IsCombinatory u ↔ GeneratedFromG u := by
  have hzero : GeneratedFromG zeroC := by
    simpa [Gcomb.app_self] using
      GeneratedFromG.app GeneratedFromG.G GeneratedFromG.G
  constructor
  · intro hu
    have hsuc : GeneratedFromG sucC := by
      simpa [sucC_from_G] using
        GeneratedFromG.app (GeneratedFromG.app GeneratedFromG.G hzero) hzero
    have hone : GeneratedFromG (ofNat 1) := by
      simpa [sucC_zero] using GeneratedFromG.app hsuc hzero
    induction hu with
    | zero => exact hzero
    | suc => exact hsuc
    | pred =>
      simpa [predC_from_G] using
        GeneratedFromG.app
          (GeneratedFromG.app (GeneratedFromG.app GeneratedFromG.G hzero) hone) hzero
```

```
    | cond =>
      simpa [condC_from_G] using
        GeneratedFromG.app
          (GeneratedFromG.app
            (GeneratedFromG.app (GeneratedFromG.app GeneratedFromG.G hzero) hone) hone
    )
          hzero
    | K =>
      simpa [Kcomb_from_G] using
        GeneratedFromG.app
          (GeneratedFromG.app
            (GeneratedFromG.app
              (GeneratedFromG.app (GeneratedFromG.app GeneratedFromG.G hzero) hone)
                hone)
            hone)
          hzero
    | S =>
      simpa [Scomb_from_G] using
        GeneratedFromG.app
          (GeneratedFromG.app
            (GeneratedFromG.app
              (GeneratedFromG.app (GeneratedFromG.app GeneratedFromG.G hzero) hone)
                hone)
            hone)
          hone
    | app _ _ iu iv => exact .app iu iv
  · intro hu
    induction hu with
    | G => exact Gcomb.combinatory
    | app _ _ iu iv => exact .app iu iv

end DomainTheory.DataTypesAsLattices
```

theorem_3_2.lean

```
module

public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Enumeration.applyNat
public import DomainTheory.DataTypesAsLattices.Enumeration.RE
public import DomainTheory.DataTypesAsLattices.Enumeration.valNat
public import DomainTheory.DataTypesAsLattices.Enumeration.theorem_3_1
public import DomainTheory.DataTypesAsLattices.Enumeration.Gcomb
public import DomainTheory.DataTypesAsLattices.Graph.Basic.notation3
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, Theorem 3.2 (The enumeration theorem).**
`RE = {val n | n ∈ ω}`, with `val(0) = G` and
```

```
`val(apply(n)(m)) = val n (val m)`. -/
theorem theorem_3_2 :
    (∀ n, valNat n ∈ RE) ∧
      valNat 0 = Gcomb ∧
        (∀ n m, valNat (applyNat n m) = valNat n ? valNat m) :=
  ⟨fun n => valNat.combinatory n, valNat.zero, valNat.app⟩

namespace theorem_3_2

/-- Every combinatory element appears in Scott's enumeration. -/
theorem surjective {u : Pomega} (hu : IsCombinatory u) :
    ∃ n, valNat n = u := by
  revert hu
  rw [theorem_3_1]
  intro h
  induction h with
  | G => exact ⟨0, valNat.zero⟩
  | app _ _ iu iv =>
    obtain ⟨n, hn⟩ := iu
    obtain ⟨m, hm⟩ := iv
    exact ⟨applyNat n m, by rw [valNat.app, hn, hm]⟩

/-- **Scott 1976, Theorem 3.2 (i).** The zero test selects `G`. -/
theorem i : condSet (ofNat 0) Gcomb botElem = Gcomb := by
  ext n
  constructor
  · intro hn
    rcases hn with ⟨hG, _⟩ | ⟨_, ⟨k, hk⟩⟩
    · exact hG
    · simp [ofNat] at hk
  · intro hG
    exact Or.inl ⟨hG, rfl⟩

/-- **Scott 1976, Theorem 3.2.** `RE` is exactly the range of `val`. -/
theorem range : RE = Set.range valNat := by
  ext u
  constructor
  · intro hu
    exact theorem_3_2.surjective hu
  · intro ⟨n, hn⟩
    simpa [← hn] using (theorem_3_2).1 n

end theorem_3_2

end DomainTheory.DataTypesAsLattices
```

unionCode.lean

```
module

public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Enumeration.theorem_3_2
public import DomainTheory.DataTypesAsLattices.Enumeration.unionOperator_combinatory
```

```
@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

/-- One fixed Godel code for the binary union operator. -/
noncomputable def unionCode : ℕ :=
  (theorem_3_2.surjective unionOperator_combinatory).choose

end DomainTheory.DataTypesAsLattices
```

botCode.lean

```
module

public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Enumeration.bot_combinatory
public import DomainTheory.DataTypesAsLattices.Enumeration.theorem_3_2
@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

noncomputable def botCode : ℕ :=
  (theorem_3_2.surjective bot_combinatory).choose

end DomainTheory.DataTypesAsLattices
```

finAux.lean

```
module

public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Enumeration.applyNat
public import DomainTheory.DataTypesAsLattices.Enumeration.num
public import DomainTheory.DataTypesAsLattices.Enumeration.unionCode
public import DomainTheory.DataTypesAsLattices.Enumeration.botCode

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

/-- Primitive recursion scanning the first `k` bits of `j`. -/
noncomputable def finAux (j : ℕ) : ℕ → ℕ
  | 0 => botCode
  | k + 1 =>
      if j.testBit k then
        applyNat (applyNat unionCode (num k)) (finAux j k)
      else finAux j k

end DomainTheory.DataTypesAsLattices
```

fin.lean

```
module

public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Enumeration.finAux
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, before Theorem 3.5.** A uniform primitive-recursive code
for `e j`; unlike the former definition, this makes only one fixed choice,
namely a code for binary union. -/
noncomputable def fin (j : ℕ) : ℕ := finAux j (j + 1)

end DomainTheory.DataTypesAsLattices
```

theorem_2_6.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Lambda.IsCombinatory
public import DomainTheory.DataTypesAsLattices.Lambda.theorem_2_4
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Computability.IsRE
public import DomainTheory.DataTypesAsLattices.Computability.IsComputable
public import DomainTheory.DataTypesAsLattices.Computability.LambdaDefinable
public import DomainTheory.DataTypesAsLattices.Computability.ComputableFin
public import DomainTheory.DataTypesAsLattices.Computability.
    combinatory_isLambdaDefinable

public import DomainTheory.DataTypesAsLattices.Lambda.Term
public import DomainTheory.DataTypesAsLattices.Lambda.zeroC
public import DomainTheory.DataTypesAsLattices.Lambda.ScottContinuousFin
public import DomainTheory.DataTypesAsLattices.Lambda.curryFin
public import DomainTheory.DataTypesAsLattices.Lambda.envSet
public import DomainTheory.DataTypesAsLattices.Lambda.interp
public import DomainTheory.DataTypesAsLattices.Lambda.theorem_2_1
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Enumeration.Avoids
public import DomainTheory.DataTypesAsLattices.Enumeration.fin
public import DomainTheory.DataTypesAsLattices.Computability.IsREu2eiff_combinatory
@[expose] public section

set_option maxHeartbeats 2000000
```

```
namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, Theorem 2.6 (The definability theorem), unary case.**
For a continuous `f`, computability, recursive enumerability of the graph,
combinatory definability, and closed LAMBDA-definability agree. -/
theorem theorem_2_6 {f : Pomega → Pomega} (hf : IsScottContinuous f) :
    (IsComputable f ↔ IsRE (graph f)) ∧
      (IsRE (graph f) ↔ IsCombinatory (graph f)) ∧
      (LambdaDefinable (graph f) ↔ IsCombinatory (graph f)) := by
  refine ⟨?_, IsRE.iff_combinatory, ?_⟩
  · exact ⟨fun h => h.2, fun h => ⟨hf, h⟩⟩
  · exact ⟨fun h => by
      obtain ⟨t, ht⟩ := h.out
      simpa [ht] using theorem_2_4.closed t,
      combinatory_isLambdaDefinable⟩

namespace theorem_2_6

/-- Combinatory elements are generated from the six constants, matching
the LAMBDA-definable closed terms of Theorem 2.4. -/
theorem combinatory_zero : LambdaDefinable zeroC :=
  ⟨⟨Term.zero, rfl⟩⟩

/-- **Scott 1976, Theorem 2.6 (The definability theorem).**
For a `k`-ary continuous `f`, computability of the finite-argument relation,
recursive enumerability of the nested graph, combinatory definability, and
closed LAMBDA-definability agree. -/
theorem fin {n : ℕ} {f : (Fin n → Pomega) → Pomega}
    (hf : ScottContinuousFin f) :
    (ComputableFin f ↔ IsRE (curryFin f)) ∧
      (IsRE (curryFin f) ↔ IsCombinatory (curryFin f)) ∧
      (LambdaDefinable (curryFin f) ↔ IsCombinatory (curryFin f)) := by
  refine ⟨?_, IsRE.iff_combinatory, ?_⟩
  · exact ⟨fun h => h.out.2, fun h => ⟨⟨hf, h⟩⟩⟩
  · exact ⟨fun h => by
      obtain ⟨t, ht⟩ := h.out
      simpa [ht] using theorem_2_4.closed t,
      combinatory_isLambdaDefinable⟩

/-- LAMBDA-definable maps (in one free variable) are continuous, so
clause (iii) of Theorem 2.6 lands in the setting of (i)-(ii). -/
theorem lambda_continuous (t : Term) :
    IsScottContinuous (fun v => interp t (envSet (fun _ => botElem) 0 v)) :=
  theorem_2_1 t (fun _ => botElem) 0

/-- **Scott 1976, Theorem 2.6, r.e. direction for numerals.** -/
theorem ofNat_isRE (n : ℕ) : IsCombinatory (ofNat n) → IsRE (ofNat n) :=
  fun _ => IsRE.numeral n

end theorem_2_6

end DomainTheory.DataTypesAsLattices
```

Lcomb.lean

```
module
public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Enumeration.GeneratedFromG

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices
def Lcomb : Pomega :=
  graph (fun x => funOf (funOf x (ofNat 1)) (funOf x (ofNat 2)))

end DomainTheory.DataTypesAsLattices
```

funRetractTerm.lean

```
module

public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Lambda.Term
public import DomainTheory.DataTypesAsLattices.Graph.lam
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.envSet
public import DomainTheory.DataTypesAsLattices.Lambda.interp
public import DomainTheory.DataTypesAsLattices.Retracts.funRetract
public import DomainTheory.DataTypesAsLattices.Enumeration.Avoids

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable

open Nat.Partrec (Code)

open Nat.Partrec.Code

/-- Closed term for Scott's function-space retract `λu λx. u(x)`. -/
def funRetractTerm : Term :=
  .lam 0 (.lam 1 (.app (.var 0) (.var 1)))

namespace funRetractTerm

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem interp :
    interp funRetractTerm (fun _ => botElem) = funRetract := by
```

```
  unfold funRetractTerm funRetract
  simp[_root_.DomainTheory.DataTypesAsLattices.interp, envSet, Function.update]

end funRetractTerm

end DomainTheory.DataTypesAsLattices
```

funRetract_combinatory.lean

```
module

public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Lambda.IsCombinatory
public import DomainTheory.DataTypesAsLattices.Lambda.theorem_2_4
public import DomainTheory.DataTypesAsLattices.Retracts.funRetract
public import DomainTheory.DataTypesAsLattices.Enumeration.funRetractTerm
public import DomainTheory.DataTypesAsLattices.Lambda.interp
@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem funRetract_combinatory : IsCombinatory funRetract := by
  rw [← funRetractTerm.interp]
  exact theorem_2_4.closed funRetractTerm

end DomainTheory.DataTypesAsLattices
```

enumComp.lean

```
module

public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Computability.composeC
public import DomainTheory.DataTypesAsLattices.Lambda.FUN
public import DomainTheory.DataTypesAsLattices.Retracts.funRetract
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
public import DomainTheory.DataTypesAsLattices.Computability.SK_comp
public import DomainTheory.DataTypesAsLattices.Enumeration.funRetract_combinatory
public import DomainTheory.DataTypesAsLattices.Enumeration.graph_mem_FUN

@[expose] public section

set_option maxHeartbeats 2000000
```

```
namespace DomainTheory.DataTypesAsLattices

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

/-- Composition in the semigroup `FUN`. -/
def enumComp (u v : Pomega) : Pomega :=
  graph (fun x => funOf u (funOf v x))

namespace enumComp
open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem app (u v x : Pomega) :
    funOf (enumComp u v) x = funOf u (funOf v x) :=
  beta (theorem_1_3 (funOf.isScottContinuous u) (funOf.isScottContinuous v)) x

theorem eq_compose (u v : Pomega) :
    enumComp u v = funOf funRetract (composeC u v) := by
  rw [funRetract.app, composeC]
  apply congrArg graph
  funext x
  exact (SK_comp u v x).symm

theorem combinatory {u v : Pomega}
    (hu : IsCombinatory u) (hv : IsCombinatory v) :
    IsCombinatory (enumComp u v) := by
  rw [enumComp.eq_compose]
  exact .app funRetract_combinatory (composeC.combinatory hu hv)

theorem mem_FUN (u v : Pomega) : enumComp u v ∈ FUN :=
  graph_mem_FUN (theorem_1_3 (funOf.isScottContinuous u) (funOf.isScottContinuous v))

end enumComp

end DomainTheory.DataTypesAsLattices
```

GeneratedSemigroup.lean

```
module

public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.comp
public import DomainTheory.DataTypesAsLattices.Enumeration.Rcomb
public import DomainTheory.DataTypesAsLattices.Enumeration.Lcomb
public import DomainTheory.DataTypesAsLattices.Enumeration.enumComp
public import DomainTheory.DataTypesAsLattices.Enumeration.Gbar

@[expose] public section

set_option maxHeartbeats 2000000
```

```
namespace DomainTheory.DataTypesAsLattices

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

/-- The subsemigroup generated by `R`, `L`, and `G`. -/
inductive GeneratedSemigroup : Pomega → Prop
  | R : GeneratedSemigroup Rcomb
  | L : GeneratedSemigroup Lcomb
  | Gbar : GeneratedSemigroup Gbar
  | comp {u v} : GeneratedSemigroup u → GeneratedSemigroup v →
      GeneratedSemigroup (enumComp u v)

end DomainTheory.DataTypesAsLattices
```

haltEnum.lean

```
module
public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Enumeration.GeneratedFromG
public import DomainTheory.DataTypesAsLattices.Enumeration.GeneratedSemigroup

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices
open Encodable

open Nat.Partrec (Code)

open Nat.Partrec.Code

/-- Shifted enumerator of the halting set: `0` is the miss marker. -/
def haltEnum (t : ℕ) : ℕ :=
  bif (evaln t.unpair.2 (Denumerable.ofNat Code t.unpair.1) 0).isSome
    then t.unpair.1 + 1 else 0

end DomainTheory.DataTypesAsLattices
```

beforeHalt.lean

```
module

public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Enumeration.haltEnum

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices
```

```
open Encodable

open Nat.Partrec (Code)

open Nat.Partrec.Code

def beforeHalt (m j : ℕ) : Prop := ∀ i < j + 1, haltEnum i = m

instance : DecidableRel beforeHalt :=
  fun m j => decidable_of_iff (∀ i : Fin (j + 1), haltEnum i.val = m)
    ⟨fun h i hi => h ⟨i, hi⟩, fun h i => h i.val i.isLt⟩

end DomainTheory.DataTypesAsLattices
```

BasicInst85.lean

```
module

public import DomainTheory.DataTypesAsLattices.Enumeration.beforeHalt

@[expose] public section
```

Extensional.lean

```
module

public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, before Theorem 3.5.** Extensionality of a Godel-number map. -/
class Extensional (val : ℕ → Pomega) (p : ℕ → ℕ) : Prop where
  out : ∀ n m, val n = val m → val (p n) = val (p m)

end DomainTheory.DataTypesAsLattices
```

GeneratedSubalgebra.lean

```
module

public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Enumeration.Subalgebra

@[expose] public section

set_option maxHeartbeats 2000000
```

```
namespace DomainTheory.DataTypesAsLattices

open Encodable

open Nat.Partrec (Code)

open Nat.Partrec.Code

/-- The least combinatory subalgebra containing every member of `xs`. -/
def GeneratedSubalgebra (xs : List Pomega) : Set Pomega :=
  {u | ∀ A : Set Pomega, Subalgebra A →
    (∀ x, x ∈ xs → x ∈ A) → u ∈ A}

namespace GeneratedSubalgebra

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem generator {xs : List Pomega} {x : Pomega}
    (hx : x ∈ xs) : x ∈ GeneratedSubalgebra xs :=
  fun _ _ hxs => hxs x hx

theorem isSubalgebra (xs : List Pomega) :
    Subalgebra (GeneratedSubalgebra xs) := by
  constructor
  constructor
  · intro A hA _
    exact hA.out.1
  · intro u v hu hv A hA hxs
    exact hA.out.2 (hu A hA hxs) (hv A hA hxs)

end GeneratedSubalgebra

end DomainTheory.DataTypesAsLattices
```

Lterm.lean

```
module
public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Lambda.Term
public import DomainTheory.DataTypesAsLattices.Lambda.termDollarStep
public import DomainTheory.DataTypesAsLattices.Graph.lam
public import DomainTheory.DataTypesAsLattices.Functionality.sumMed
public import DomainTheory.DataTypesAsLattices.Functionality.Zcomb
public import DomainTheory.DataTypesAsLattices.Functionality.sigmaJ
public import DomainTheory.DataTypesAsLattices.Retracts.arrowR
public import DomainTheory.DataTypesAsLattices.Retracts.Icomb
public import DomainTheory.DataTypesAsLattices.Retracts.intF
public import DomainTheory.DataTypesAsLattices.Retracts.intR
public import DomainTheory.DataTypesAsLattices.Retracts.boolR
public import DomainTheory.DataTypesAsLattices.Enumeration.GeneratedFromG
public import DomainTheory.DataTypesAsLattices.Enumeration.GeneratedSemigroup

@[expose] public section
```

```
set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices
def Lterm : Term :=
  .lam 0
    (.app (.app (.var 0) (.succ .zero))
      (.app (.var 0) (.succ (.succ .zero))))

end DomainTheory.DataTypesAsLattices
```

Lterm_more.lean

```
module

public import DomainTheory.DataTypesAsLattices.Enumeration.Lterm
public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Lambda.Term
public import DomainTheory.DataTypesAsLattices.Enumeration.Lcomb
public import DomainTheory.DataTypesAsLattices.Lambda.interp
public import DomainTheory.DataTypesAsLattices.Lambda.envSet
public import DomainTheory.DataTypesAsLattices.Lambda.succSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Graph.lam
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace Lterm

theorem interp : DomainTheory.DataTypesAsLattices.interp Lterm (fun _ => botElem) =
    Lcomb := by
  unfold Lterm Lcomb
  simp [DomainTheory.DataTypesAsLattices.interp, envSet, Function.update, succSet.
    ofNat]

end Lterm

end DomainTheory.DataTypesAsLattices
```

Lcomb_more.lean

```
module

public import DomainTheory.DataTypesAsLattices.Enumeration.Lcomb
public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Graph.beta
```

```
public import DomainTheory.DataTypesAsLattices.Enumeration.Lterm
public import DomainTheory.DataTypesAsLattices.Enumeration.Lterm_more
public import DomainTheory.DataTypesAsLattices.Lambda.interp
public import DomainTheory.DataTypesAsLattices.Lambda.IsCombinatory
public import DomainTheory.DataTypesAsLattices.Lambda.FUN
public import DomainTheory.DataTypesAsLattices.Lambda.theorem_2_4
public import DomainTheory.DataTypesAsLattices.Enumeration.graph_mem_FUN

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace Lcomb

theorem app (x : Pomega) :
    funOf Lcomb x = funOf (funOf x (ofNat 1)) (funOf x (ofNat 2)) :=
  beta (theorem_1_3.tuple
    (fun y => funOf.isScottContinuous_left y)
    (fun t => funOf.isScottContinuous t)
    (funOf.isScottContinuous_left (ofNat 1))
    (funOf.isScottContinuous_left (ofNat 2))) x

theorem combinatory : IsCombinatory Lcomb := by
  rw [← Lterm.interp]
  exact theorem_2_4.closed Lterm

theorem mem_FUN : Lcomb ∈ FUN :=
  graph_mem_FUN (theorem_1_3.tuple
    (fun y => funOf.isScottContinuous_left y)
    (fun t => funOf.isScottContinuous t)
    (funOf.isScottContinuous_left (ofNat 1))
    (funOf.isScottContinuous_left (ofNat 2)))

end Lcomb

end DomainTheory.DataTypesAsLattices
```

REPredu2eexists_param.lean

```
module

public import Mathlib.Computability.Halting
public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Retracts.comp
public import DomainTheory.DataTypesAsLattices.Computability.REPredu2eexists
@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code
```

```
theorem REPred.exists_param {α} [Primcodable α] [Inhabited α] {p : α → ℕ → Prop}
    (hp : REPred fun q : α × ℕ => p q.1 q.2) :
    REPred fun a => ∃ n, p a n := by
  have hN : REPred fun q : ℕ × ℕ => p ((decode q.1).getD default) q.2 :=
    hp.comp (Computable.pair
      (Computable.option_getD (Computable.decode.comp Computable.fst)
        (Computable.const default))
      Computable.snd)
  have hex : REPred fun n : ℕ => ∃ m, p ((decode n).getD default) m :=
    REPred.exists (p := fun n m => p ((decode n).getD default) m) hN
  exact _root_.REPred.of_eq (hex.comp Computable.encode) fun a => by
    simp [encodek]

end DomainTheory.DataTypesAsLattices
```

Rterm.lean

```
module

public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Lambda.Term
public import DomainTheory.DataTypesAsLattices.Enumeration.seq2Term

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

/-- Closed terms for the two elementary semigroup generators. -/
def Rterm : Term := .lam 0 (seq2Term .zero (.var 0))

end DomainTheory.DataTypesAsLattices
```

Rcomb_more.lean

```
module

public import DomainTheory.DataTypesAsLattices.Enumeration.Rcomb
public import DomainTheory.DataTypesAsLattices.Enumeration.Rterm
public import DomainTheory.DataTypesAsLattices.Enumeration.seq2Term
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Lambda.envSet
public import DomainTheory.DataTypesAsLattices.Lambda.seq2
public import DomainTheory.DataTypesAsLattices.Lambda.interp
public import DomainTheory.DataTypesAsLattices.Enumeration.Avoids
public import DomainTheory.DataTypesAsLattices.Lambda.IsCombinatory
public import DomainTheory.DataTypesAsLattices.Lambda.theorem_2_4

@[expose] public section

set_option maxHeartbeats 2000000
```

```
namespace DomainTheory.DataTypesAsLattices

namespace Rterm

theorem interp :
    _root_.DomainTheory.DataTypesAsLattices.interp Rterm (fun _ => botElem) = Rcomb :=
     by
  unfold Rterm Rcomb
  apply congrArg graph
  funext x
  change _root_.DomainTheory.DataTypesAsLattices.interp (seq2Term .zero (.var 0))
      (envSet (fun _ => botElem) 0 x) =
    seq2 (ofNat 0) x
  rw [seq2Term.interp]
  · simp [_root_.DomainTheory.DataTypesAsLattices.interp, envSet, Function.update]
  · trivial
  · exact Nat.zero_ne_add_one 7

end Rterm

namespace Rcomb

theorem combinatory : IsCombinatory Rcomb := by
  rw [← Rterm.interp]
  exact theorem_2_4.closed Rterm

end Rcomb

end DomainTheory.DataTypesAsLattices
```

TOT.lean

```
module

public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Lambda.dollarC
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable

open Nat.Partrec (Code)

open Nat.Partrec.Code

/-- **Scott 1976, TOT.** Graphs of total number-theoretic functions:
`$`-invariant and integer-valued on integers. -/
```

```
def TOT : Set Pomega :=
  {u | u = funOf dollarC u ∧ ∀ n, ∃ k, funOf u (ofNat n) = ofNat k}

end DomainTheory.DataTypesAsLattices
```

ValTrace.lean

```
module

public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Enumeration.valNat

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable

open Nat.Partrec (Code)

open Nat.Partrec.Code

/-- Finite derivation that `member ∈ valNat code`, with the application
witness `t` stored on positive codes. -/
abbrev ValTrace := List (ℕ × ℕ × ℕ)

end DomainTheory.DataTypesAsLattices
```

Ycode.lean

```
module

public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Lambda.Ycomb
public import DomainTheory.DataTypesAsLattices.Enumeration.theorem_3_2
public import DomainTheory.DataTypesAsLattices.Lambda.Ycomb_more
@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

/-- A Godel number of `Y`, used for a Kleene-style fixed point on codes. -/
noncomputable def Ycode : ℕ :=
  (theorem_3_2.surjective Ycomb.combinatory).choose

end DomainTheory.DataTypesAsLattices
```

argNat.lean

```
module
```

```
public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Pomega.pair

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

noncomputable def argNat (k : ℕ) : ℕ := (unpair k).2

end DomainTheory.DataTypesAsLattices
```

barComb_more.lean

```
module

public import DomainTheory.DataTypesAsLattices.Enumeration.barComb
public import DomainTheory.DataTypesAsLattices.Enumeration.eq_3_15
public import DomainTheory.DataTypesAsLattices.Enumeration.barStepBody
public import DomainTheory.DataTypesAsLattices.Enumeration.Rcomb
public import DomainTheory.DataTypesAsLattices.Enumeration.barPos
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.beta

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

namespace barComb

theorem app2 (u x : Pomega) :
    funOf (funOf barComb u) x = barStepBody barComb u x := by
  have ha : funOf barComb u = graph (fun z => barStepBody barComb u z) :=
    _root_.DomainTheory.DataTypesAsLattices.eq_3_15 u
  rw [ha]
  exact beta (barStepBody.isScottContinuous_x barComb u) x

theorem on_R (u x : Pomega) :
    funOf (funOf barComb u) (funOf Rcomb x) =
      barPos u (funOf Rcomb x) := by
  rw [app2, barStepBody, Rcomb.app_zero, eq_2_7_zero]

theorem on_barPos_R (u v x : Pomega) :
    funOf (funOf barComb u) (barPos v (funOf Rcomb x)) =
      funOf (funOf barComb (funOf u v)) x := by
  rw [app2, barStepBody, barPos.zero, eq_2_7_succ,
    barPos.one, barPos.two, Rcomb.app, seq2.one]

end barComb

end DomainTheory.DataTypesAsLattices
```

eq_3_16.lean

```
module

public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Enumeration.Lcomb
public import DomainTheory.DataTypesAsLattices.Enumeration.Rcomb
public import DomainTheory.DataTypesAsLattices.Enumeration.barPos
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.lam
public import DomainTheory.DataTypesAsLattices.Enumeration.enumComp
public import DomainTheory.DataTypesAsLattices.Enumeration.barStepBody
public import DomainTheory.DataTypesAsLattices.Enumeration.barComb
public import DomainTheory.DataTypesAsLattices.Enumeration.barComb_more
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Enumeration.Lcomb_more
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

/-- **Scott 1976, (3.16).** `L(u⁺(R(x))) = u(x)` on the zero-test branch. -/
theorem eq_3_16 (u x : Pomega) :
    funOf Lcomb (barPos u (funOf Rcomb x)) = funOf u x := by
  rw [Lcomb.app, barPos.one, barPos.two, Rcomb.app, seq2.one]

namespace eq_3_16

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

/-- **Scott 1976, (3.16).** `L ∘ u ∘ R = λx. u(x)` from the Y-definition. -/
theorem bar (u x : Pomega) :
    funOf Lcomb (funOf (funOf barComb u) (funOf Rcomb x)) = funOf u x := by
  rw [barComb.app2, barStepBody, Rcomb.app_zero, eq_2_7_zero]
  exact eq_3_16 u x

theorem operator (u : Pomega) :
    enumComp Lcomb (enumComp (funOf barComb u) Rcomb) =
      graph (fun x => funOf u x) := by
  unfold enumComp
  apply congrArg graph
  funext x
  change funOf Lcomb
    (funOf (lam fun x => funOf (funOf barComb u) (funOf Rcomb x)) x) =
      funOf u x
```

```
  rw [beta (theorem_1_3 (funOf.isScottContinuous (funOf barComb u))
      (funOf.isScottContinuous Rcomb)) x]
  exact eq_3_16.bar u x

end eq_3_16

end DomainTheory.DataTypesAsLattices
```

eq_3_17.lean

```
module

public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.lam
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Enumeration.eq_3_15
public import DomainTheory.DataTypesAsLattices.Enumeration.eq_3_16
public import DomainTheory.DataTypesAsLattices.Enumeration.barComb
public import DomainTheory.DataTypesAsLattices.Enumeration.barComb_more
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Enumeration.enumComp
public import DomainTheory.DataTypesAsLattices.Enumeration.barStepBody
public import DomainTheory.DataTypesAsLattices.Enumeration.Rcomb
public import DomainTheory.DataTypesAsLattices.Enumeration.Lcomb
public import DomainTheory.DataTypesAsLattices.Enumeration.barPos
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

/-- **Scott 1976, (3.17).** Operator identity
`u ∘ v ∘ R = overline{u(v)}`. -/
theorem eq_3_17 (u v : Pomega) :
    enumComp (funOf barComb u) (enumComp (funOf barComb v) Rcomb) =
      funOf barComb (funOf u v) := by
  conv_rhs => rw [eq_3_15]
  unfold enumComp
  apply congrArg graph
  funext x
  change funOf (funOf barComb u)
    (funOf (lam fun x => funOf (funOf barComb v) (funOf Rcomb x)) x) =
      barStepBody barComb (funOf u v) x
  rw [beta (theorem_1_3 (funOf.isScottContinuous (funOf barComb v))
      (funOf.isScottContinuous Rcomb)) x]
  rw [barComb.on_R, barComb.on_barPos_R, barComb.app2]
```

```
namespace eq_3_17

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

/-- Projection calculation used in the proof of Scott 1976, (3.17). -/
theorem projections (u v x : Pomega) :
    funOf (barPos u (barPos v (funOf Rcomb x))) (ofNat 1) = u ∧
      funOf (barPos v (funOf Rcomb x)) (ofNat 1) = v ∧
        funOf (barPos v (funOf Rcomb x)) (ofNat 2) = x := by
  refine ⟨barPos.one _ _, barPos.one _ _, ?_⟩
  rw [barPos.two, Rcomb.app, seq2.one]

/-- Nested positive-branch packaging of (3.17): `u⁺(v⁺(R(x)))`
has first projection `u` and recovers `u(v)(x)` via `L` after
replacing the head by `u(v)`. -/
theorem semigroup (u v x : Pomega) :
    funOf (barPos u (barPos v (funOf Rcomb x))) (ofNat 1) = u ∧
      funOf Lcomb (barPos (funOf u v) (funOf Rcomb x)) =
        funOf (funOf u v) x :=
  ⟨(eq_3_17.projections u v x).1, eq_3_16 (funOf u v) x⟩

end eq_3_17

end DomainTheory.DataTypesAsLattices
```

bar_generated.lean

```
module

public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.comp
public import DomainTheory.DataTypesAsLattices.Enumeration.GeneratedFromG
public import DomainTheory.DataTypesAsLattices.Enumeration.GeneratedSemigroup
public import DomainTheory.DataTypesAsLattices.Enumeration.eq_3_17
public import DomainTheory.DataTypesAsLattices.Enumeration.barComb
public import DomainTheory.DataTypesAsLattices.Enumeration.Gbar

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem bar_generated {u : Pomega} (hu : GeneratedFromG u) :
    GeneratedSemigroup (funOf barComb u) := by
  induction hu with
```

```
  | G => exact GeneratedSemigroup.Gbar
  | app _ _ ihu ihv =>
      rw [← eq_3_17]
      exact .comp ihu (.comp ihv .R)

end DomainTheory.DataTypesAsLattices
```

bitList.lean

```
module

public import Mathlib.Computability.Halting

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

def bitList (j : ℕ) : List ℕ :=
  (List.range (j + 1)).filter (fun k => j.testBit k)

end DomainTheory.DataTypesAsLattices
```

combinatory_mem_Deg.lean

```
module

public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Lambda.IsCombinatory
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Enumeration.GeneratedFromG
public import DomainTheory.DataTypesAsLattices.Enumeration.Deg
public import DomainTheory.DataTypesAsLattices.Enumeration.theorem_3_1
@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

/-- Combinatory elements lie in every enumeration degree, since `Deg a`
contains `G` and is closed under application. -/
theorem combinatory_mem_Deg (a : Pomega) {u : Pomega}
    (hu : IsCombinatory u) : u ∈ Deg a := by
  have huG : GeneratedFromG u := (theorem_3_1 (u := u)).mp hu
  clear hu
  induction huG with
  | G => exact (Deg.isSubalgebra a).out.1
  | app _ _ ih ih' => exact (Deg.isSubalgebra a).out.2 ih ih'

end DomainTheory.DataTypesAsLattices
```

combinatory_mem_subalgebra.lean

```
module

public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Lambda.IsCombinatory
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Enumeration.GeneratedFromG
public import DomainTheory.DataTypesAsLattices.Enumeration.Subalgebra
public import DomainTheory.DataTypesAsLattices.Enumeration.theorem_3_1
@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem combinatory_mem_subalgebra {A : Set Pomega} (hA : Subalgebra A)
    {u : Pomega} (hu : IsCombinatory u) : u ∈ A := by
  have hgen : GeneratedFromG u := (theorem_3_1 (u := u)).mp hu
  clear hu
  induction hgen with
  | G => exact hA.out.1
  | app _ _ ih ih' => exact hA.out.2 ih ih'

end DomainTheory.DataTypesAsLattices
```

condC_app2.lean

```
module

public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.condC
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.condC_more
@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

theorem condC_app2 (x y : Pomega) :
    funOf (funOf condC x) y = graph (fun z => condSet z x y) :=
  condC.beta2 x y

end DomainTheory.DataTypesAsLattices
```

condClosed.lean

```
module

public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Lambda.Term
public import DomainTheory.DataTypesAsLattices.Graph.lam
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.envSet
public import DomainTheory.DataTypesAsLattices.Lambda.interp
public import DomainTheory.DataTypesAsLattices.Lambda.condC
public import DomainTheory.DataTypesAsLattices.Enumeration.Avoids

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable

open Nat.Partrec (Code)

open Nat.Partrec.Code

def condClosed : Term :=
  .lam 4 (.lam 5 (.lam 6 (.cond (.var 6) (.var 4) (.var 5))))

namespace condClosed

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem avoids8 : condClosed.Avoids 8 := by
  simp[condClosed, Term.Avoids]

theorem interp (? : ℕ → Pomega) : interp condClosed ? = condC := by
  simp[condClosed, condC, envSet, Function.update, _root_.DomainTheory.
    DataTypesAsLattices.interp]

end condClosed

end DomainTheory.DataTypesAsLattices
```

condCode.lean

```
module

public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Lambda.IsCombinatory
public import DomainTheory.DataTypesAsLattices.Enumeration.theorem_3_2
@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices
```

```
open Encodable

open Nat.Partrec (Code)

open Nat.Partrec.Code

noncomputable def condCode : ℕ :=
  (theorem_3_2.surjective IsCombinatory.cond).choose

end DomainTheory.DataTypesAsLattices
```

cond_graph_eq_bot_iff.lean

```
module

public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.eq_2_7_zero
public import DomainTheory.DataTypesAsLattices.Lambda.eq_2_7_succ
public import DomainTheory.DataTypesAsLattices.Lambda.condC
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.funOf_botElem
public import DomainTheory.DataTypesAsLattices.Enumeration.condC_app2
public import DomainTheory.DataTypesAsLattices.Lambda.condC_more

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

theorem cond_graph_eq_bot_iff (x y : Pomega) :
    funOf (funOf condC x) y = botElem ↔ x = botElem ∧ y = botElem := by
  constructor
  · intro h
    constructor
    · have := congrArg (fun w => funOf w (ofNat 0)) h
      rw [condC.beta3, eq_2_7_zero, funOf_botElem] at this
      exact this
    · have := congrArg (fun w => funOf w (ofNat 1)) h
      rw [condC.beta3, eq_2_7_succ, funOf_botElem] at this
      exact this
  · intro ⟨hx, hy⟩
    rw [condC_app2, hx, hy]
    ext p
    constructor
    · intro hp
      rcases hp with ⟨n, m, _, hm⟩
      simp [condSet, botElem] at hm
    · intro hp
      exact hp.elim
```

```
end DomainTheory.DataTypesAsLattices
```

interStepTerm.lean

```
module

public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Lambda.Term
public import DomainTheory.DataTypesAsLattices.Graph.lam
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Lambda.envSet
public import DomainTheory.DataTypesAsLattices.Lambda.interStep
public import DomainTheory.DataTypesAsLattices.Lambda.interp
public import DomainTheory.DataTypesAsLattices.Enumeration.Avoids
public import DomainTheory.DataTypesAsLattices.Lambda.eq_2_14

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

/-- A closed LAMBDA term for the body defining binary intersection. -/
def interStepTerm : Term :=
  let omega : Term := .lam 4 (.app (.var 4) (.var 4))
  .lam 0 (.lam 1 (.lam 2
    (.cond (.var 1)
      (.cond (.var 2) .zero (.app omega omega))
      (.succ (.app (.app (.var 0) (.pred (.var 1))) (.pred (.var 2)))))))

namespace interStepTerm

theorem interp :
    interp interStepTerm (fun _ => botElem) = graph interStep := by
  unfold interStepTerm interStep
  simp only[_root_.DomainTheory.DataTypesAsLattices.interp, envSet, Function.
    update_self]
  apply congrArg graph
  funext f
  apply congrArg graph
  funext x
  apply congrArg graph
  funext y
  congr 2
  exact _root_.DomainTheory.DataTypesAsLattices.eq_2_14

end interStepTerm

end DomainTheory.DataTypesAsLattices
```

interC_combinatory.lean

```
module
```

```
public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Lambda.IsCombinatory
public import DomainTheory.DataTypesAsLattices.Lambda.interStep
public import DomainTheory.DataTypesAsLattices.Lambda.interC
public import DomainTheory.DataTypesAsLattices.Lambda.theorem_2_4
public import DomainTheory.DataTypesAsLattices.Lambda.theorem_2_5
public import DomainTheory.DataTypesAsLattices.Lambda.Ycomb
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Enumeration.interStepTerm
public import DomainTheory.DataTypesAsLattices.Lambda.interp
public import DomainTheory.DataTypesAsLattices.Lambda.Ycomb_more
@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

theorem interC_combinatory : IsCombinatory interC := by
  have hbody : IsCombinatory (graph interStep) := by
    rw [← interStepTerm.interp]
    exact theorem_2_4.closed interStepTerm
  have hfix := IsCombinatory.app Ycomb.combinatory hbody
  rw [theorem_2_5 interStep.isScottContinuous] at hfix
  exact hfix

end DomainTheory.DataTypesAsLattices
```

interCode.lean

```
module

public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Enumeration.theorem_3_2
public import DomainTheory.DataTypesAsLattices.Enumeration.interC_combinatory

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

noncomputable def interCode : ℕ :=
  (theorem_3_2.surjective interC_combinatory).choose

end DomainTheory.DataTypesAsLattices
```

diagonalInterCode.lean

```
module

public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Enumeration.applyNat
public import DomainTheory.DataTypesAsLattices.Enumeration.num
public import DomainTheory.DataTypesAsLattices.Enumeration.interCode
```

```
@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

/-- The primitive-recursive diagonal intersection code used in Theorem 3.4. -/
noncomputable def diagonalInterCode (n : ℕ) : ℕ :=
  applyNat (applyNat interCode (num n)) n

end DomainTheory.DataTypesAsLattices
```

e_subset_ofNat_iff.lean

```
module

public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Pomega.e
@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem e_subset_ofNat_iff (code n : ℕ) :
    e code ⊆ ofNat n ↔ code = 0 ∨ code = 2 ^ n := by
  constructor
  · intro h
    by_cases hz : code = 0
    · exact Or.inl hz
    · have hbit : ∀ k, code.testBit k = true → k = n := by
        intro k hk
        simpa [ofNat] using h (mem_e.mpr hk)
      have hn : code.testBit n = true := by
        have hk : code.testBit code.log2 = true := Nat.testBit_log2 hz
        simpa [hbit _ hk] using hk
      refine Or.inr (Nat.eq_of_testBit_eq fun k => ?_)
      by_cases hk : k = n
      · subst hk
        simp [hn]
      · have hfalse : code.testBit k = false :=
          Bool.eq_false_iff.mpr fun hb => hk (hbit k hb)
        simp [hfalse, Ne.symm hk]
  · intro h
    rcases h with rfl | rfl
    · intro k hk
      simp [e_zero] at hk
    · intro k hk
      simpa [e_pow2, ofNat] using hk
```

```
end DomainTheory.DataTypesAsLattices
```

enumerationUnion_left_isScottContinuous.lean

```
module

public import Mathlib.Computability.Halting
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.mem_scottUnion
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import Mathlib.Order.CompleteLattice.Defs
@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

theorem enumerationUnion_left_isScottContinuous (y : Pomega) :
    IsScottContinuous (fun x => x ∪ y) := by
  constructor
  intro x
  ext n
  constructor
  · rintro (hn | hn)
    · exact mem_scottUnion.mpr ⟨2 ^ n,
        by simp [e_pow2, Set.singleton_subset_iff, hn],
        Or.inl (by simp [e_pow2])⟩
    · exact mem_scottUnion.mpr ⟨0, by simp [e_zero], Or.inr hn⟩
  · intro hn
    obtain ⟨i, hi, hni⟩ := mem_scottUnion.mp hn
    exact hni.elim (fun h => Or.inl (hi h)) Or.inr

end DomainTheory.DataTypesAsLattices
```

enumerationUnion_right_isScottContinuous.lean

```
module

public import Mathlib.Computability.Halting
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.mem_scottUnion
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import Mathlib.Order.CompleteLattice.Defs
@[expose] public section

set_option maxHeartbeats 2000000
```

```
namespace DomainTheory.DataTypesAsLattices

theorem enumerationUnion_right_isScottContinuous (x : Pomega) :
    IsScottContinuous (fun y => x ∪ y) := by
  constructor
  intro y
  ext n
  constructor
  · rintro (hn | hn)
    · exact mem_scottUnion.mpr ⟨0, by simp [e_zero], Or.inl hn⟩
    · exact mem_scottUnion.mpr ⟨2 ^ n,
        by simp [e_pow2, Set.singleton_subset_iff, hn],
        Or.inr (by simp [e_pow2])⟩
  · intro hn
    obtain ⟨i, hi, hni⟩ := mem_scottUnion.mp hn
    exact hni.elim Or.inl (fun h => Or.inr (hi h))

end DomainTheory.DataTypesAsLattices
```

not_mem_zero_cond_graph.lean

```
module

public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.pair_zero_zero
public import DomainTheory.DataTypesAsLattices.Lambda.condC
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Enumeration.condC_app2

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

/-- `0 ∈ cond(x)(y)`, since `0 = (0,0)` and `e_0 ? x, y = ⊥`. -/
theorem not_mem_zero_cond_graph (x y : Pomega) :
    0 ∈ funOf (funOf condC x) y := by
  intro h
  rw [condC_app2] at h
  rcases h with ⟨n, m, hm, hmem⟩
  have : pair n m = pair 0 0 := by
    simpa [pair_zero_zero] using hm.symm
  obtain ⟨rfl, rfl⟩ := pair_inj this
  simp [e_zero, condSet] at hmem

end DomainTheory.DataTypesAsLattices
```

eq_3_1.lean

```
module
```

```
public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.condC
public import DomainTheory.DataTypesAsLattices.Lambda.eq_2_7_pos
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Enumeration.not_mem_zero_cond_graph
public import DomainTheory.DataTypesAsLattices.Enumeration.cond_graph_eq_bot_iff
public import DomainTheory.DataTypesAsLattices.Lambda.condC_more

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (3.1).** `cond(x)(y)(cond(x)(y)) = y`. -/
theorem eq_3_1 (x y : Pomega) :
    funOf (funOf (funOf condC x) y) (funOf (funOf condC x) y) = y := by
  rw [condC.beta3]
  by_cases hxy : x = botElem ∧ y = botElem
  · rcases hxy with ⟨rfl, rfl⟩
    have : funOf (funOf condC botElem) botElem = botElem :=
      (cond_graph_eq_bot_iff botElem botElem).mpr ⟨rfl, rfl⟩
    simp [condSet, botElem]
  · have hne : funOf (funOf condC x) y = botElem := by
      intro h
      exact hxy ((cond_graph_eq_bot_iff x y).mp h)
    exact eq_2_7_pos x y (funOf (funOf condC x) y)
      (not_mem_zero_cond_graph x y) hne

end DomainTheory.DataTypesAsLattices
```

eq_3_10_apply.lean

```
module

public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Enumeration.applyNat
@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (3.10).** `apply(3)(1) = 12` and `val(12) = suc`. -/
theorem eq_3_10_apply : applyNat 3 1 = 12 := by simp [applyNat, pair]

end DomainTheory.DataTypesAsLattices
```

eq_3_8_apply.lean

```
module
```

```
public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Enumeration.applyNat
@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (3.8).** `apply(0)(0) = 1` and `val(1) = 0`. -/
theorem eq_3_8_apply : applyNat 0 0 = 1 := by simp [applyNat, pair]

end DomainTheory.DataTypesAsLattices
```

eq_3_8_val.lean

```
module

public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Lambda.zeroC
public import DomainTheory.DataTypesAsLattices.Enumeration.applyNat
public import DomainTheory.DataTypesAsLattices.Enumeration.valNat
public import DomainTheory.DataTypesAsLattices.Enumeration.eq_3_8_apply
public import DomainTheory.DataTypesAsLattices.Enumeration.Gcomb
public import DomainTheory.DataTypesAsLattices.Enumeration.Gcomb_more
@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

theorem eq_3_8_val : valNat 1 = zeroC := by
  have : valNat 1 = valNat (applyNat 0 0) := by simp [eq_3_8_apply]
  rw [this, valNat.app, valNat.zero, Gcomb.app_self]

end DomainTheory.DataTypesAsLattices
```

eq_3_9_apply.lean

```
module

public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Enumeration.applyNat
@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (3.9).** `apply(0)(1) = 3` and `val(3) = ⟨suc,...⟩`. -/
theorem eq_3_9_apply : applyNat 0 1 = 3 := by simp [applyNat, pair]

end DomainTheory.DataTypesAsLattices
```

eq_3_9_val.lean

```
module

public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.zeroC
public import DomainTheory.DataTypesAsLattices.Enumeration.applyNat
public import DomainTheory.DataTypesAsLattices.Enumeration.valNat
public import DomainTheory.DataTypesAsLattices.Enumeration.Gpack
public import DomainTheory.DataTypesAsLattices.Enumeration.eq_3_8_val
public import DomainTheory.DataTypesAsLattices.Enumeration.eq_3_9_apply
public import DomainTheory.DataTypesAsLattices.Enumeration.Gcomb
public import DomainTheory.DataTypesAsLattices.Enumeration.Gcomb_more

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

theorem eq_3_9_val : valNat 3 = Gpack := by
  have : valNat 3 = valNat (applyNat 0 1) := by simp [eq_3_9_apply]
  rw [this, valNat.app, valNat.zero, eq_3_8_val]
  have : zeroC = ofNat 0 := rfl
  simpa [this] using Gcomb.zero

end DomainTheory.DataTypesAsLattices
```

eq_3_10_val.lean

```
module

public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.zeroC
public import DomainTheory.DataTypesAsLattices.Lambda.sucC
public import DomainTheory.DataTypesAsLattices.Enumeration.applyNat
public import DomainTheory.DataTypesAsLattices.Enumeration.valNat
public import DomainTheory.DataTypesAsLattices.Enumeration.Gpack
public import DomainTheory.DataTypesAsLattices.Enumeration.eq_3_8_val
public import DomainTheory.DataTypesAsLattices.Enumeration.eq_3_9_val
public import DomainTheory.DataTypesAsLattices.Enumeration.eq_3_10_apply

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

theorem eq_3_10_val : valNat 12 = sucC := by
  have : valNat 12 = valNat (applyNat 3 1) := by simp [eq_3_10_apply]
  rw [this, valNat.app, eq_3_9_val, eq_3_8_val]
  have : zeroC = ofNat 0 := rfl
  simpa [this] using Gpack.zero
```

```
end DomainTheory.DataTypesAsLattices
```

eq_3_12.lean

```
module

public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Lambda.sucC
public import DomainTheory.DataTypesAsLattices.Lambda.succSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Enumeration.valNat
public import DomainTheory.DataTypesAsLattices.Enumeration.num
public import DomainTheory.DataTypesAsLattices.Enumeration.eq_3_8_val
public import DomainTheory.DataTypesAsLattices.Enumeration.eq_3_10_val
public import DomainTheory.DataTypesAsLattices.Lambda.sucC_more
public import DomainTheory.DataTypesAsLattices.Lambda.succSet_more

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (3.12).** `val(num(n)) = n`. -/
theorem eq_3_12 : ∀ n, valNat (num n) = ofNat n
  | 0 => eq_3_8_val
  | n + 1 => by
    rw [num, valNat.app, eq_3_10_val, eq_3_12, sucC.app, succSet.ofNat]

end DomainTheory.DataTypesAsLattices
```

eq_3_2.lean

```
module

public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Lambda.eq_2_7_zero
public import DomainTheory.DataTypesAsLattices.Lambda.condC
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.zeroC
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.condC_more
@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (3.2).** `cond(x)(y)(0) = x`. -/
theorem eq_3_2 (x y : Pomega) :
    funOf (funOf (funOf condC x) y) zeroC = x := by
  rw [condC.beta3]
  have : zeroC = ofNat 0 := rfl
  simpa [this] using eq_2_7_zero x y
```

```
end DomainTheory.DataTypesAsLattices
```

image_retr_wayBelow_nonempty.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeRetraction
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace IsContinuousLatticeRetraction

theorem image_retr_wayBelow_nonempty (R : IsContinuousLatticeRetraction D D') (d : D)
    :
    (Set.image (R.retr : D' → D) {x' | x' ≪ R.incl d}).Nonempty :=
  ⟨R.retr ⊥, ⊥, bot_wayBelow _, rfl⟩

end IsContinuousLatticeRetraction
end DomainTheory.ContinuousLattice
```

0

image_retr_wayBelow_nonempty.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeRetraction
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']
```

```
namespace IsContinuousLatticeRetraction

theorem image_retr_wayBelow_nonempty (R : IsContinuousLatticeRetraction D D') (d : D)
    :
    (Set.image (R.retr : D' → D) {x' | x' ≪ R.incl d}).Nonempty :=
  ⟨R.retr ⊥, ⊥, bot_wayBelow _, rfl⟩

end IsContinuousLatticeRetraction
end DomainTheory.ContinuousLattice
```

1

image_retr_wayBelow_nonempty.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeRetraction
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace IsContinuousLatticeRetraction

theorem image_retr_wayBelow_nonempty (R : IsContinuousLatticeRetraction D D') (d : D)
    :
    (Set.image (R.retr : D' → D) {x' | x' ≪ R.incl d}).Nonempty :=
  ⟨R.retr ⊥, ⊥, bot_wayBelow _, rfl⟩

end IsContinuousLatticeRetraction
end DomainTheory.ContinuousLattice
```

2

image_retr_wayBelow_nonempty.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeRetraction
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section
```

```
namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace IsContinuousLatticeRetraction

theorem image_retr_wayBelow_nonempty (R : IsContinuousLatticeRetraction D D') (d : D)
    :
    (Set.image (R.retr : D' → D) {x' | x' ≪ R.incl d}).Nonempty :=
  ⟨R.retr ⊥, ⊥, bot_wayBelow _, rfl⟩

end IsContinuousLatticeRetraction
end DomainTheory.ContinuousLattice
```

3

image_retr_wayBelow_nonempty.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeRetraction
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace IsContinuousLatticeRetraction

theorem image_retr_wayBelow_nonempty (R : IsContinuousLatticeRetraction D D') (d : D)
    :
    (Set.image (R.retr : D' → D) {x' | x' ≪ R.incl d}).Nonempty :=
  ⟨R.retr ⊥, ⊥, bot_wayBelow _, rfl⟩

end IsContinuousLatticeRetraction
end DomainTheory.ContinuousLattice
```

4

image_retr_wayBelow_nonempty.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeRetraction
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace IsContinuousLatticeRetraction

theorem image_retr_wayBelow_nonempty (R : IsContinuousLatticeRetraction D D') (d : D)
    :
    (Set.image (R.retr : D' → D) {x' | x' ≪ R.incl d}).Nonempty :=
  ⟨R.retr ⊥, ⊥, bot_wayBelow _, rfl⟩

end IsContinuousLatticeRetraction
end DomainTheory.ContinuousLattice
```


image_retr_wayBelow_nonempty.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeRetraction
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace IsContinuousLatticeRetraction
```

```
theorem image_retr_wayBelow_nonempty (R : IsContinuousLatticeRetraction D D') (d : D)
    :
    (Set.image (R.retr : D' → D) {x' | x' ≪ R.incl d}).Nonempty :=
  ⟨R.retr ⊥, ⊥, bot_wayBelow _, rfl⟩

end IsContinuousLatticeRetraction
end DomainTheory.ContinuousLattice
```

6

image_retr_wayBelow_nonempty.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeRetraction
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace IsContinuousLatticeRetraction

theorem image_retr_wayBelow_nonempty (R : IsContinuousLatticeRetraction D D') (d : D)
    :
    (Set.image (R.retr : D' → D) {x' | x' ≪ R.incl d}).Nonempty :=
  ⟨R.retr ⊥, ⊥, bot_wayBelow _, rfl⟩

end IsContinuousLatticeRetraction
end DomainTheory.ContinuousLattice
```

7

image_retr_wayBelow_nonempty.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeRetraction
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice
```

```
open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace IsContinuousLatticeRetraction

theorem image_retr_wayBelow_nonempty (R : IsContinuousLatticeRetraction D D') (d : D)
    :
    (Set.image (R.retr : D' → D) {x' | x' ≪ R.incl d}).Nonempty :=
  ⟨R.retr ⊥, ⊥, bot_wayBelow _, rfl⟩

end IsContinuousLatticeRetraction
end DomainTheory.ContinuousLattice
```

8

image_retr_wayBelow_nonempty.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeRetraction
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace IsContinuousLatticeRetraction

theorem image_retr_wayBelow_nonempty (R : IsContinuousLatticeRetraction D D') (d : D)
    :
    (Set.image (R.retr : D' → D) {x' | x' ≪ R.incl d}).Nonempty :=
  ⟨R.retr ⊥, ⊥, bot_wayBelow _, rfl⟩

end IsContinuousLatticeRetraction
end DomainTheory.ContinuousLattice
```

9

instCompleteLattice.lean

```
module
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaces.Basic.isLUB_sSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

namespace Projections

/-- **Scott 1972, Proposition 3.12.** `J_D` is a complete lattice (a `⊔`-closed
    subspace of
`[D → D]`). Suprema are inherited from `[D → D]`; infima are derived by `
    completeLatticeOfSup`. -/
noncomputable instance instCompleteLattice : CompleteLattice (Projections D) :=
  completeLatticeOfSup (Projections D) isLUB_sSup

end Projections

end Projections

end DomainTheory.ContinuousLattice
```

0

instCompleteLattice.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.Basic.isLUB_sSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]
```

```
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

namespace Projections

/-- **Scott 1972, Proposition 3.12.** `J_D` is a complete lattice (a `⊔`-closed
    subspace of
`[D → D]`). Suprema are inherited from `[D → D]`; infima are derived by `
    completeLatticeOfSup`. -/
noncomputable instance instCompleteLattice : CompleteLattice (Projections D) :=
  completeLatticeOfSup (Projections D) isLUB_sSup

end Projections

end Projections

end DomainTheory.ContinuousLattice
```

1

instCompleteLattice.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.Basic.isLUB_sSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

namespace Projections

/-- **Scott 1972, Proposition 3.12.** `J_D` is a complete lattice (a `⊔`-closed
    subspace of
`[D → D]`). Suprema are inherited from `[D → D]`; infima are derived by `
    completeLatticeOfSup`. -/
```

```
noncomputable instance instCompleteLattice : CompleteLattice (Projections D) :=
  completeLatticeOfSup (Projections D) isLUB_sSup

end Projections

end Projections

end DomainTheory.ContinuousLattice
```

2

instCompleteLattice.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.Basic.isLUB_sSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

namespace Projections

/-- **Scott 1972, Proposition 3.12.** `J_D` is a complete lattice (a `⊔`-closed
    subspace of
`[D → D]`). Suprema are inherited from `[D → D]`; infima are derived by `
    completeLatticeOfSup`. -/
noncomputable instance instCompleteLattice : CompleteLattice (Projections D) :=
  completeLatticeOfSup (Projections D) isLUB_sSup

end Projections

end Projections

end DomainTheory.ContinuousLattice
```

3

instCompleteLattice.lean

```
module
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaces.Basic.isLUB_sSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

namespace Projections

/-- **Scott 1972, Proposition 3.12.** `J_D` is a complete lattice (a `⊔`-closed
    subspace of
`[D → D]`). Suprema are inherited from `[D → D]`; infima are derived by `
    completeLatticeOfSup`. -/
noncomputable instance instCompleteLattice : CompleteLattice (Projections D) :=
  completeLatticeOfSup (Projections D) isLUB_sSup

end Projections

end Projections

end DomainTheory.ContinuousLattice
```

4

instCompleteLattice.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.Basic.isLUB_sSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]
```

```
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

namespace Projections

/-- **Scott 1972, Proposition 3.12.** `J_D` is a complete lattice (a `⊔`-closed
    subspace of
`[D → D]`). Suprema are inherited from `[D → D]`; infima are derived by `
    completeLatticeOfSup`. -/
noncomputable instance instCompleteLattice : CompleteLattice (Projections D) :=
  completeLatticeOfSup (Projections D) isLUB_sSup

end Projections

end Projections

end DomainTheory.ContinuousLattice
```

5

instCompleteLattice.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.Basic.isLUB_sSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

namespace Projections

/-- **Scott 1972, Proposition 3.12.** `J_D` is a complete lattice (a `⊔`-closed
    subspace of
`[D → D]`). Suprema are inherited from `[D → D]`; infima are derived by `
    completeLatticeOfSup`. -/
```

```
noncomputable instance instCompleteLattice : CompleteLattice (Projections D) :=
  completeLatticeOfSup (Projections D) isLUB_sSup

end Projections

end Projections

end DomainTheory.ContinuousLattice
```

6

instCompleteLattice.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.Basic.isLUB_sSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

namespace Projections

/-- **Scott 1972, Proposition 3.12.** `J_D` is a complete lattice (a `⊔`-closed
    subspace of
`[D → D]`). Suprema are inherited from `[D → D]`; infima are derived by `
    completeLatticeOfSup`. -/
noncomputable instance instCompleteLattice : CompleteLattice (Projections D) :=
  completeLatticeOfSup (Projections D) isLUB_sSup

end Projections

end Projections

end DomainTheory.ContinuousLattice
```

7

instCompleteLattice.lean

```
module
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaces.Basic.isLUB_sSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

namespace Projections

/-- **Scott 1972, Proposition 3.12.** `J_D` is a complete lattice (a `⊔`-closed
    subspace of
`[D → D]`). Suprema are inherited from `[D → D]`; infima are derived by `
    completeLatticeOfSup`. -/
noncomputable instance instCompleteLattice : CompleteLattice (Projections D) :=
  completeLatticeOfSup (Projections D) isLUB_sSup

end Projections

end Projections

end DomainTheory.ContinuousLattice
```

8

instCompleteLattice.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.Basic.isLUB_sSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]
```

```
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

namespace Projections

/-- **Scott 1972, Proposition 3.12.** `J_D` is a complete lattice (a `⊔`-closed
    subspace of
`[D → D]`). Suprema are inherited from `[D → D]`; infima are derived by `
    completeLatticeOfSup`. -/
noncomputable instance instCompleteLattice : CompleteLattice (Projections D) :=
  completeLatticeOfSup (Projections D) isLUB_sSup

end Projections

end Projections

end DomainTheory.ContinuousLattice
```

9

instSupSet.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst30
public import DomainTheory.ContinuousLattice.FunctionSpaces.Projections
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_sSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

namespace Projections

noncomputable instance instSupSet : SupSet (Projections D) :=
  ⟨fun T => ⟨sSup (Set.image Subtype.val T), isProjection_sSup (by rintro j ⟨p, _, rfl⟩;
```

```
    exact p.2)⟩⟩

end Projections

end Projections

end DomainTheory.ContinuousLattice
```

0

instSupSet.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst30
public import DomainTheory.ContinuousLattice.FunctionSpaces.Projections
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_sSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

namespace Projections

noncomputable instance instSupSet : SupSet (Projections D) :=
  ⟨fun T => ⟨sSup (Set.image Subtype.val T), isProjection_sSup (by rintro j ⟨p, _, rfl⟩;
    exact p.2)⟩⟩

end Projections

end Projections

end DomainTheory.ContinuousLattice
```

1

instSupSet.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst30
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaces.Projections
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_sSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

namespace Projections

noncomputable instance instSupSet : SupSet (Projections D) :=
  ⟨fun T => ⟨sSup (Set.image Subtype.val T), isProjection_sSup (by rintro j ⟨p, _, rfl⟩;
     exact p.2)⟩⟩

end Projections

end Projections

end DomainTheory.ContinuousLattice
```

2

instSupSet.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst30
public import DomainTheory.ContinuousLattice.FunctionSpaces.Projections
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_sSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]
```

```
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

namespace Projections

noncomputable instance instSupSet : SupSet (Projections D) :=
  ⟨fun T => ⟨sSup (Set.image Subtype.val T), isProjection_sSup (by rintro j ⟨p, _, rfl⟩;
     exact p.2)⟩⟩

end Projections

end Projections

end DomainTheory.ContinuousLattice
```

3

instSupSet.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst30
public import DomainTheory.ContinuousLattice.FunctionSpaces.Projections
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_sSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

namespace Projections

noncomputable instance instSupSet : SupSet (Projections D) :=
  ⟨fun T => ⟨sSup (Set.image Subtype.val T), isProjection_sSup (by rintro j ⟨p, _, rfl⟩;
     exact p.2)⟩⟩

end Projections
```

```
end Projections

end DomainTheory.ContinuousLattice
```

4

instSupSet.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst30
public import DomainTheory.ContinuousLattice.FunctionSpaces.Projections
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_sSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

namespace Projections

noncomputable instance instSupSet : SupSet (Projections D) :=
  ⟨fun T => ⟨sSup (Set.image Subtype.val T), isProjection_sSup (by rintro j ⟨p, _, rfl⟩;
     exact p.2)⟩⟩

end Projections

end Projections

end DomainTheory.ContinuousLattice
```

5

instSupSet.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst30
public import DomainTheory.ContinuousLattice.FunctionSpaces.Projections
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_sSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic
```

```
@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

namespace Projections

noncomputable instance instSupSet : SupSet (Projections D) :=
  ⟨fun T => ⟨sSup (Set.image Subtype.val T), isProjection_sSup (by rintro j ⟨p, _, rfl⟩;
     exact p.2)⟩⟩

end Projections

end Projections

end DomainTheory.ContinuousLattice
```

6

instSupSet.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst30
public import DomainTheory.ContinuousLattice.FunctionSpaces.Projections
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_sSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']
```

```
section Projections

namespace Projections

noncomputable instance instSupSet : SupSet (Projections D) :=
  ⟨fun T => ⟨sSup (Set.image Subtype.val T), isProjection_sSup (by rintro j ⟨p, _, rfl⟩;
     exact p.2)⟩⟩

end Projections

end Projections

end DomainTheory.ContinuousLattice
```

7

instSupSet.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst30
public import DomainTheory.ContinuousLattice.FunctionSpaces.Projections
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_sSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

namespace Projections

noncomputable instance instSupSet : SupSet (Projections D) :=
  ⟨fun T => ⟨sSup (Set.image Subtype.val T), isProjection_sSup (by rintro j ⟨p, _, rfl⟩;
     exact p.2)⟩⟩

end Projections

end Projections

end DomainTheory.ContinuousLattice
```

8

instSupSet.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst30
public import DomainTheory.ContinuousLattice.FunctionSpaces.Projections
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_sSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

namespace Projections

noncomputable instance instSupSet : SupSet (Projections D) :=
  ⟨fun T => ⟨sSup (Set.image Subtype.val T), isProjection_sSup (by rintro j ⟨p, _, rfl⟩;
     exact p.2)⟩⟩

end Projections

end Projections

end DomainTheory.ContinuousLattice
```

9

isLUB_sSup.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst28
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst30
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section
```

```
namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

theorem isLUB_sSup {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (F : Set (ScottMap D D')) :
    IsLUB F (sSup F) := by
  constructor
  · intro f hf
    rw [le_def]
    intro x
    rw [sSup_apply]
    exact le_sSup (Set.mem_image_of_mem _ hf)
  · intro g hg
    rw [le_def]
    intro x
    rw [sSup_apply]
    refine sSup_le ?_
    rintro _ ⟨f, hfF, rfl⟩
    exact (hg hfF) x

end ScottMap

end DomainTheory.ContinuousLattice
```

0

isLUB_sSup.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst28
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst30
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace
```

```
universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

theorem isLUB_sSup {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (F : Set (ScottMap D D')) :
    IsLUB F (sSup F) := by
  constructor
  · intro f hf
    rw [le_def]
    intro x
    rw [sSup_apply]
    exact le_sSup (Set.mem_image_of_mem _ hf)
  · intro g hg
    rw [le_def]
    intro x
    rw [sSup_apply]
    refine sSup_le ?_
    rintro _ ⟨f, hfF, rfl⟩
    exact (hg hfF) x

end ScottMap

end DomainTheory.ContinuousLattice
```

1

isLUB_sSup.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst28
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst30
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v
```

```
variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

theorem isLUB_sSup {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (F : Set (ScottMap D D')) :
    IsLUB F (sSup F) := by
  constructor
  · intro f hf
    rw [le_def]
    intro x
    rw [sSup_apply]
    exact le_sSup (Set.mem_image_of_mem _ hf)
  · intro g hg
    rw [le_def]
    intro x
    rw [sSup_apply]
    refine sSup_le ?_
    rintro _ ⟨f, hfF, rfl⟩
    exact (hg hfF) x

end ScottMap

end DomainTheory.ContinuousLattice
```

2

isLUB_sSup.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst28
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst30
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]
```

```
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

theorem isLUB_sSup {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (F : Set (ScottMap D D')) :
    IsLUB F (sSup F) := by
  constructor
  · intro f hf
    rw [le_def]
    intro x
    rw [sSup_apply]
    exact le_sSup (Set.mem_image_of_mem _ hf)
  · intro g hg
    rw [le_def]
    intro x
    rw [sSup_apply]
    refine sSup_le ?_
    rintro _ ⟨f, hfF, rfl⟩
    exact (hg hfF) x

end ScottMap

end DomainTheory.ContinuousLattice
```

3

isLUB_sSup.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst28
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst30
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
```

```
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

theorem isLUB_sSup {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (F : Set (ScottMap D D')) :
    IsLUB F (sSup F) := by
  constructor
  · intro f hf
    rw [le_def]
    intro x
    rw [sSup_apply]
    exact le_sSup (Set.mem_image_of_mem _ hf)
  · intro g hg
    rw [le_def]
    intro x
    rw [sSup_apply]
    refine sSup_le ?_
    rintro _ ⟨f, hfF, rfl⟩
    exact (hg hfF) x

end ScottMap

end DomainTheory.ContinuousLattice
```

4

isLUB_sSup.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst28
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst30
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']
```

```
namespace ScottMap

theorem isLUB_sSup {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (F : Set (ScottMap D D')) :
    IsLUB F (sSup F) := by
  constructor
  · intro f hf
    rw [le_def]
    intro x
    rw [sSup_apply]
    exact le_sSup (Set.mem_image_of_mem _ hf)
  · intro g hg
    rw [le_def]
    intro x
    rw [sSup_apply]
    refine sSup_le ?_
    rintro _ ⟨f, hfF, rfl⟩
    exact (hg hfF) x

end ScottMap

end DomainTheory.ContinuousLattice
```

5

isLUB_sSup.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst28
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst30
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

theorem isLUB_sSup {D : Type u} {D' : Type v}
```

```
    [CompleteLattice D] [CompleteLattice D'] (F : Set (ScottMap D D')) :
    IsLUB F (sSup F) := by
  constructor
  · intro f hf
    rw [le_def]
    intro x
    rw [sSup_apply]
    exact le_sSup (Set.mem_image_of_mem _ hf)
  · intro g hg
    rw [le_def]
    intro x
    rw [sSup_apply]
    refine sSup_le ?_
    rintro _ ⟨f, hfF, rfl⟩
    exact (hg hfF) x

end ScottMap

end DomainTheory.ContinuousLattice
```

6

isLUB_sSup.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst28
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst30
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

theorem isLUB_sSup {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (F : Set (ScottMap D D')) :
    IsLUB F (sSup F) := by
  constructor
```

```
  · intro f hf
    rw [le_def]
    intro x
    rw [sSup_apply]
    exact le_sSup (Set.mem_image_of_mem _ hf)
  · intro g hg
    rw [le_def]
    intro x
    rw [sSup_apply]
    refine sSup_le ?_
    rintro _ ⟨f, hfF, rfl⟩
    exact (hg hfF) x

end ScottMap

end DomainTheory.ContinuousLattice
```

7

isLUB_sSup.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst28
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst30
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

theorem isLUB_sSup {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (F : Set (ScottMap D D')) :
    IsLUB F (sSup F) := by
  constructor
  · intro f hf
    rw [le_def]
    intro x
```

```
    rw [sSup_apply]
    exact le_sSup (Set.mem_image_of_mem _ hf)
  · intro g hg
    rw [le_def]
    intro x
    rw [sSup_apply]
    refine sSup_le ?_
    rintro _ ⟨f, hfF, rfl⟩
    exact (hg hfF) x

end ScottMap

end DomainTheory.ContinuousLattice
```

8

isLUB_sSup.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst28
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst30
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

theorem isLUB_sSup {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (F : Set (ScottMap D D')) :
    IsLUB F (sSup F) := by
  constructor
  · intro f hf
    rw [le_def]
    intro x
    rw [sSup_apply]
    exact le_sSup (Set.mem_image_of_mem _ hf)
  · intro g hg
```

```
    rw [le_def]
    intro x
    rw [sSup_apply]
    refine sSup_le ?_
    rintro _ ⟨f, hfF, rfl⟩
    exact (hg hfF) x

end ScottMap

end DomainTheory.ContinuousLattice
```

9

isProjection_bot.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.IsProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_iff
public import DomainTheory.ContinuousLattice.FunctionSpaces.bot_apply
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

/-- `⊥` (the constant `⊥` map) is a projection -- Scott's `⊔∅ ∈ J_D`. -/
theorem isProjection_bot : IsProjection (⊥ : ScottMap D D) := by
  rw [isProjection_iff]
  refine ⟨fun x => ?_, fun x => ?_⟩
  · simp only [ScottMap.bot_apply]
  · rw [ScottMap.bot_apply]; exact bot_le

end Projections

end DomainTheory.ContinuousLattice
```

0

isProjection_bot.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.IsProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_iff
public import DomainTheory.ContinuousLattice.FunctionSpaces.bot_apply
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

/-- `⊥` (the constant `⊥` map) is a projection -- Scott's `⊔∅ ∈ J_D`. -/
theorem isProjection_bot : IsProjection (⊥ : ScottMap D D) := by
  rw [isProjection_iff]
  refine ⟨fun x => ?_, fun x => ?_⟩
  · simp only [ScottMap.bot_apply]
  · rw [ScottMap.bot_apply]; exact bot_le

end Projections

end DomainTheory.ContinuousLattice
```

1

isProjection_bot.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.IsProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_iff
public import DomainTheory.ContinuousLattice.FunctionSpaces.bot_apply
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace
```

```
variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

/-- `⊥` (the constant `⊥` map) is a projection -- Scott's `⊔∅ ∈ J_D`. -/
theorem isProjection_bot : IsProjection (⊥ : ScottMap D D) := by
  rw [isProjection_iff]
  refine ⟨fun x => ?_, fun x => ?_⟩
  · simp only [ScottMap.bot_apply]
  · rw [ScottMap.bot_apply]; exact bot_le

end Projections

end DomainTheory.ContinuousLattice
```

2

isProjection_bot.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.IsProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_iff
public import DomainTheory.ContinuousLattice.FunctionSpaces.bot_apply
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

/-- `⊥` (the constant `⊥` map) is a projection -- Scott's `⊔∅ ∈ J_D`. -/
theorem isProjection_bot : IsProjection (⊥ : ScottMap D D) := by
  rw [isProjection_iff]
  refine ⟨fun x => ?_, fun x => ?_⟩
```

```
  · simp only [ScottMap.bot_apply]
  · rw [ScottMap.bot_apply]; exact bot_le

end Projections

end DomainTheory.ContinuousLattice
```

3

isProjection_bot.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.IsProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_iff
public import DomainTheory.ContinuousLattice.FunctionSpaces.bot_apply
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

/-- `⊥` (the constant `⊥` map) is a projection -- Scott's `⊔∅ ∈ J_D`. -/
theorem isProjection_bot : IsProjection (⊥ : ScottMap D D) := by
  rw [isProjection_iff]
  refine ⟨fun x => ?_, fun x => ?_⟩
  · simp only [ScottMap.bot_apply]
  · rw [ScottMap.bot_apply]; exact bot_le

end Projections

end DomainTheory.ContinuousLattice
```

4

isProjection_bot.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.IsProjection
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_iff
public import DomainTheory.ContinuousLattice.FunctionSpaces.bot_apply
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

/-- `⊥` (the constant `⊥` map) is a projection -- Scott's `⊔∅ ∈ J_D`. -/
theorem isProjection_bot : IsProjection (⊥ : ScottMap D D) := by
  rw [isProjection_iff]
  refine ⟨fun x => ?_, fun x => ?_⟩
  · simp only [ScottMap.bot_apply]
  · rw [ScottMap.bot_apply]; exact bot_le

end Projections

end DomainTheory.ContinuousLattice
```

5

isProjection_bot.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.IsProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_iff
public import DomainTheory.ContinuousLattice.FunctionSpaces.bot_apply
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]
```

```
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

/-- `⊥` (the constant `⊥` map) is a projection -- Scott's `⊔∅ ∈ J_D`. -/
theorem isProjection_bot : IsProjection (⊥ : ScottMap D D) := by
  rw [isProjection_iff]
  refine ⟨fun x => ?_, fun x => ?_⟩
  · simp only [ScottMap.bot_apply]
  · rw [ScottMap.bot_apply]; exact bot_le

end Projections

end DomainTheory.ContinuousLattice
```

6

isProjection_bot.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.IsProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_iff
public import DomainTheory.ContinuousLattice.FunctionSpaces.bot_apply
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

/-- `⊥` (the constant `⊥` map) is a projection -- Scott's `⊔∅ ∈ J_D`. -/
theorem isProjection_bot : IsProjection (⊥ : ScottMap D D) := by
  rw [isProjection_iff]
  refine ⟨fun x => ?_, fun x => ?_⟩
  · simp only [ScottMap.bot_apply]
  · rw [ScottMap.bot_apply]; exact bot_le

end Projections
```

```
end DomainTheory.ContinuousLattice
```

7

isProjection_bot.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.IsProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_iff
public import DomainTheory.ContinuousLattice.FunctionSpaces.bot_apply
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

/-- `⊥` (the constant `⊥` map) is a projection -- Scott's `⊔∅ ∈ J_D`. -/
theorem isProjection_bot : IsProjection (⊥ : ScottMap D D) := by
  rw [isProjection_iff]
  refine ⟨fun x => ?_, fun x => ?_⟩
  · simp only [ScottMap.bot_apply]
  · rw [ScottMap.bot_apply]; exact bot_le

end Projections

end DomainTheory.ContinuousLattice
```

8

isProjection_bot.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.IsProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_iff
public import DomainTheory.ContinuousLattice.FunctionSpaces.bot_apply
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic
```

```
@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

/-- `⊥` (the constant `⊥` map) is a projection -- Scott's `⊔∅ ∈ J_D`. -/
theorem isProjection_bot : IsProjection (⊥ : ScottMap D D) := by
  rw [isProjection_iff]
  refine ⟨fun x => ?_, fun x => ?_⟩
  · simp only [ScottMap.bot_apply]
  · rw [ScottMap.bot_apply]; exact bot_le

end Projections

end DomainTheory.ContinuousLattice
```

9

isProjection_directedSup.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst30
public import DomainTheory.ContinuousLattice.FunctionSpaces.IsProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_iff
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
```

```
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

/-- `J_D` is closed under *directed* joins (Scott 1972, 3.12). Continuity of each
    member lets the
inner `(⊔S)`-application distribute over the directed family `{ j x : j ∈ S }`, and
    directedness
plus idempotency collapse the double family `{ k (j x) }` to `(⊔S) x`. -/
theorem isProjection_directedSup {S : Set (ScottMap D D)} (hSne : S.Nonempty)
    (hSdir : DirectedOn (· ≤ ·) S) (hS : ∀ j ∈ S, IsProjection j) :
    IsProjection (sSup S) := by
  have hidem : ∀ j ∈ S, ∀ x, (j : D → D) (j x) = j x :=
    fun j hj => (isProjection_iff.mp (hS j hj)).1
  have hle : ∀ j ∈ S, ∀ x, (j : D → D) x ≤ x := fun j hj => (isProjection_iff.mp (hS
    j hj)).2
  rw [isProjection_iff]
  refine ⟨fun x => ?_, fun x => ?_⟩
  · set A := Set.image (fun j : ScottMap D D => (j : D → D) x) S with hA
    have hAne : A.Nonempty := hSne.image _
    have hAdir : DirectedOn (· ≤ ·) A := by
      rintro _ ⟨j, hj, rfl⟩ _ ⟨k, hk, rfl⟩
      obtain ⟨m, hm, hjm, hkm⟩ := hSdir j hj k hk
      exact ⟨(m : D → D) x, Set.mem_image_of_mem _ hm, hjm x, hkm x⟩
    have hsSx : ((sSup S : ScottMap D D) : D → D) x = sSup A := ScottMap.sSup_apply S
    x
    apply le_antisymm
    · rw [hsSx, ScottMap.sSup_apply]
      refine sSup_le ?_
      rintro _ ⟨k, hk, rfl⟩
      show (k : D → D) (sSup A) ≤ sSup A
      rw [k.preservesDirectedSup_coe A hAne hAdir]
      refine sSup_le ?_
      rintro _ ⟨_, ⟨j, hj, rfl⟩, rfl⟩
      obtain ⟨m, hm, hjm, hkm⟩ := hSdir j hj k hk
      calc (k : D → D) (j x) ≤ (m : D → D) (j x) := hkm (j x)
        _ ≤ (m : D → D) (m x) := m.monotone (hjm x)
        _ = (m : D → D) x := hidem m hm x
        _ ≤ sSup A := le_sSup (Set.mem_image_of_mem _ hm)
    · rw [hsSx, ScottMap.sSup_apply]
      refine sSup_le ?_
      rintro _ ⟨j, hj, rfl⟩
      have hjxA : (j : D → D) x ≤ sSup A := le_sSup (Set.mem_image_of_mem _ hj)
      calc (j : D → D) x = (j : D → D) (j x) := (hidem j hj x).symm
        _ ≤ (j : D → D) (sSup A) := j.monotone hjxA
        _ ≤ sSup (Set.image (fun f : ScottMap D D => (f : D → D) (sSup A)) S) :=
              le_sSup (Set.mem_image_of_mem _ hj)
  · rw [ScottMap.sSup_apply]
    refine sSup_le ?_
    rintro _ ⟨j, hj, rfl⟩
    exact hle j hj x

end Projections

end DomainTheory.ContinuousLattice
```

isProjection_directedSup.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst30
public import DomainTheory.ContinuousLattice.FunctionSpaces.IsProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_iff
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

/-- `J_D` is closed under *directed* joins (Scott 1972, 3.12). Continuity of each
    member lets the
inner `(⊔S)`-application distribute over the directed family `{ j x : j ∈ S }`, and
    directedness
plus idempotency collapse the double family `{ k (j x) }` to `(⊔S) x`. -/
theorem isProjection_directedSup {S : Set (ScottMap D D)} (hSne : S.Nonempty)
    (hSdir : DirectedOn (· ≤ ·) S) (hS : ∀ j ∈ S, IsProjection j) :
    IsProjection (sSup S) := by
  have hidem : ∀ j ∈ S, ∀ x, (j : D → D) (j x) = j x :=
    fun j hj => (isProjection_iff.mp (hS j hj)).1
  have hle : ∀ j ∈ S, ∀ x, (j : D → D) x ≤ x := fun j hj => (isProjection_iff.mp (hS
    j hj)).2
  rw [isProjection_iff]
  refine ⟨fun x => ?_, fun x => ?_⟩
  · set A := Set.image (fun j : ScottMap D D => (j : D → D) x) S with hA
    have hAne : A.Nonempty := hSne.image _
    have hAdir : DirectedOn (· ≤ ·) A := by
      rintro _ ⟨j, hj, rfl⟩ _ ⟨k, hk, rfl⟩
      obtain ⟨m, hm, hjm, hkm⟩ := hSdir j hj k hk
      exact ⟨(m : D → D) x, Set.mem_image_of_mem _ hm, hjm x, hkm x⟩
    have hsSx : ((sSup S : ScottMap D D) : D → D) x = sSup A := ScottMap.sSup_apply S
    x
    apply le_antisymm
    · rw [hsSx, ScottMap.sSup_apply]
```

```
      refine sSup_le ?_
      rintro _ ⟨k, hk, rfl⟩
      show (k : D → D) (sSup A) ≤ sSup A
      rw [k.preservesDirectedSup_coe A hAne hAdir]
      refine sSup_le ?_
      rintro _ ⟨_, ⟨j, hj, rfl⟩, rfl⟩
      obtain ⟨m, hm, hjm, hkm⟩ := hSdir j hj k hk
      calc (k : D → D) (j x) ≤ (m : D → D) (j x) := hkm (j x)
        _ ≤ (m : D → D) (m x) := m.monotone (hjm x)
        _ = (m : D → D) x := hidem m hm x
        _ ≤ sSup A := le_sSup (Set.mem_image_of_mem _ hm)
    · rw [hsSx, ScottMap.sSup_apply]
      refine sSup_le ?_
      rintro _ ⟨j, hj, rfl⟩
      have hjxA : (j : D → D) x ≤ sSup A := le_sSup (Set.mem_image_of_mem _ hj)
      calc (j : D → D) x = (j : D → D) (j x) := (hidem j hj x).symm
        _ ≤ (j : D → D) (sSup A) := j.monotone hjxA
        _ ≤ sSup (Set.image (fun f : ScottMap D D => (f : D → D) (sSup A)) S) :=
              le_sSup (Set.mem_image_of_mem _ hj)
  · rw [ScottMap.sSup_apply]
    refine sSup_le ?_
    rintro _ ⟨j, hj, rfl⟩
    exact hle j hj x

end Projections

end DomainTheory.ContinuousLattice
```

1

isProjection_directedSup.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst30
public import DomainTheory.ContinuousLattice.FunctionSpaces.IsProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_iff
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
```

```
    CompleteLattice D₁']

section Projections

/-- `J_D` is closed under *directed* joins (Scott 1972, 3.12). Continuity of each
    member lets the
inner `(⊔S)`-application distribute over the directed family `{ j x : j ∈ S }`, and
    directedness
plus idempotency collapse the double family `{ k (j x) }` to `(⊔S) x`. -/
theorem isProjection_directedSup {S : Set (ScottMap D D)} (hSne : S.Nonempty)
    (hSdir : DirectedOn (· ≤ ·) S) (hS : ∀ j ∈ S, IsProjection j) :
    IsProjection (sSup S) := by
  have hidem : ∀ j ∈ S, ∀ x, (j : D → D) (j x) = j x :=
    fun j hj => (isProjection_iff.mp (hS j hj)).1
  have hle : ∀ j ∈ S, ∀ x, (j : D → D) x ≤ x := fun j hj => (isProjection_iff.mp (hS
    j hj)).2
  rw [isProjection_iff]
  refine ⟨fun x => ?_, fun x => ?_⟩
  · set A := Set.image (fun j : ScottMap D D => (j : D → D) x) S with hA
    have hAne : A.Nonempty := hSne.image _
    have hAdir : DirectedOn (· ≤ ·) A := by
      rintro _ ⟨j, hj, rfl⟩ _ ⟨k, hk, rfl⟩
      obtain ⟨m, hm, hjm, hkm⟩ := hSdir j hj k hk
      exact ⟨(m : D → D) x, Set.mem_image_of_mem _ hm, hjm x, hkm x⟩
    have hsSx : ((sSup S : ScottMap D D) : D → D) x = sSup A := ScottMap.sSup_apply S
    x
    apply le_antisymm
    · rw [hsSx, ScottMap.sSup_apply]
      refine sSup_le ?_
      rintro _ ⟨k, hk, rfl⟩
      show (k : D → D) (sSup A) ≤ sSup A
      rw [k.preservesDirectedSup_coe A hAne hAdir]
      refine sSup_le ?_
      rintro _ ⟨_, ⟨j, hj, rfl⟩, rfl⟩
      obtain ⟨m, hm, hjm, hkm⟩ := hSdir j hj k hk
      calc (k : D → D) (j x) ≤ (m : D → D) (j x) := hkm (j x)
        _ ≤ (m : D → D) (m x) := m.monotone (hjm x)
        _ = (m : D → D) x := hidem m hm x
        _ ≤ sSup A := le_sSup (Set.mem_image_of_mem _ hm)
    · rw [hsSx, ScottMap.sSup_apply]
      refine sSup_le ?_
      rintro _ ⟨j, hj, rfl⟩
      have hjxA : (j : D → D) x ≤ sSup A := le_sSup (Set.mem_image_of_mem _ hj)
      calc (j : D → D) x = (j : D → D) (j x) := (hidem j hj x).symm
        _ ≤ (j : D → D) (sSup A) := j.monotone hjxA
        _ ≤ sSup (Set.image (fun f : ScottMap D D => (f : D → D) (sSup A)) S) :=
              le_sSup (Set.mem_image_of_mem _ hj)
  · rw [ScottMap.sSup_apply]
    refine sSup_le ?_
    rintro _ ⟨j, hj, rfl⟩
    exact hle j hj x

end Projections

end DomainTheory.ContinuousLattice
```

2

isProjection_directedSup.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst30
public import DomainTheory.ContinuousLattice.FunctionSpaces.IsProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_iff
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

/-- `J_D` is closed under *directed* joins (Scott 1972, 3.12). Continuity of each
    member lets the
inner `(⊔S)`-application distribute over the directed family `{ j x : j ∈ S }`, and
    directedness
plus idempotency collapse the double family `{ k (j x) }` to `(⊔S) x`. -/
theorem isProjection_directedSup {S : Set (ScottMap D D)} (hSne : S.Nonempty)
    (hSdir : DirectedOn (· ≤ ·) S) (hS : ∀ j ∈ S, IsProjection j) :
    IsProjection (sSup S) := by
  have hidem : ∀ j ∈ S, ∀ x, (j : D → D) (j x) = j x :=
    fun j hj => (isProjection_iff.mp (hS j hj)).1
  have hle : ∀ j ∈ S, ∀ x, (j : D → D) x ≤ x := fun j hj => (isProjection_iff.mp (hS
    j hj)).2
  rw [isProjection_iff]
  refine ⟨fun x => ?_, fun x => ?_⟩
  · set A := Set.image (fun j : ScottMap D D => (j : D → D) x) S with hA
    have hAne : A.Nonempty := hSne.image _
    have hAdir : DirectedOn (· ≤ ·) A := by
      rintro _ ⟨j, hj, rfl⟩ _ ⟨k, hk, rfl⟩
      obtain ⟨m, hm, hjm, hkm⟩ := hSdir j hj k hk
      exact ⟨(m : D → D) x, Set.mem_image_of_mem _ hm, hjm x, hkm x⟩
    have hsSx : ((sSup S : ScottMap D D) : D → D) x = sSup A := ScottMap.sSup_apply S
    x
    apply le_antisymm
    · rw [hsSx, ScottMap.sSup_apply]
```

```
      refine sSup_le ?_
      rintro _ ⟨k, hk, rfl⟩
      show (k : D → D) (sSup A) ≤ sSup A
      rw [k.preservesDirectedSup_coe A hAne hAdir]
      refine sSup_le ?_
      rintro _ ⟨_, ⟨j, hj, rfl⟩, rfl⟩
      obtain ⟨m, hm, hjm, hkm⟩ := hSdir j hj k hk
      calc (k : D → D) (j x) ≤ (m : D → D) (j x) := hkm (j x)
        _ ≤ (m : D → D) (m x) := m.monotone (hjm x)
        _ = (m : D → D) x := hidem m hm x
        _ ≤ sSup A := le_sSup (Set.mem_image_of_mem _ hm)
    · rw [hsSx, ScottMap.sSup_apply]
      refine sSup_le ?_
      rintro _ ⟨j, hj, rfl⟩
      have hjxA : (j : D → D) x ≤ sSup A := le_sSup (Set.mem_image_of_mem _ hj)
      calc (j : D → D) x = (j : D → D) (j x) := (hidem j hj x).symm
        _ ≤ (j : D → D) (sSup A) := j.monotone hjxA
        _ ≤ sSup (Set.image (fun f : ScottMap D D => (f : D → D) (sSup A)) S) :=
              le_sSup (Set.mem_image_of_mem _ hj)
  · rw [ScottMap.sSup_apply]
    refine sSup_le ?_
    rintro _ ⟨j, hj, rfl⟩
    exact hle j hj x

end Projections

end DomainTheory.ContinuousLattice
```

3

isProjection_directedSup.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst30
public import DomainTheory.ContinuousLattice.FunctionSpaces.IsProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_iff
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
```

```
    CompleteLattice D₁']

section Projections

/-- `J_D` is closed under *directed* joins (Scott 1972, 3.12). Continuity of each
    member lets the
inner `(⊔S)`-application distribute over the directed family `{ j x : j ∈ S }`, and
    directedness
plus idempotency collapse the double family `{ k (j x) }` to `(⊔S) x`. -/
theorem isProjection_directedSup {S : Set (ScottMap D D)} (hSne : S.Nonempty)
    (hSdir : DirectedOn (· ≤ ·) S) (hS : ∀ j ∈ S, IsProjection j) :
    IsProjection (sSup S) := by
  have hidem : ∀ j ∈ S, ∀ x, (j : D → D) (j x) = j x :=
    fun j hj => (isProjection_iff.mp (hS j hj)).1
  have hle : ∀ j ∈ S, ∀ x, (j : D → D) x ≤ x := fun j hj => (isProjection_iff.mp (hS
    j hj)).2
  rw [isProjection_iff]
  refine ⟨fun x => ?_, fun x => ?_⟩
  · set A := Set.image (fun j : ScottMap D D => (j : D → D) x) S with hA
    have hAne : A.Nonempty := hSne.image _
    have hAdir : DirectedOn (· ≤ ·) A := by
      rintro _ ⟨j, hj, rfl⟩ _ ⟨k, hk, rfl⟩
      obtain ⟨m, hm, hjm, hkm⟩ := hSdir j hj k hk
      exact ⟨(m : D → D) x, Set.mem_image_of_mem _ hm, hjm x, hkm x⟩
    have hsSx : ((sSup S : ScottMap D D) : D → D) x = sSup A := ScottMap.sSup_apply S
    x
    apply le_antisymm
    · rw [hsSx, ScottMap.sSup_apply]
      refine sSup_le ?_
      rintro _ ⟨k, hk, rfl⟩
      show (k : D → D) (sSup A) ≤ sSup A
      rw [k.preservesDirectedSup_coe A hAne hAdir]
      refine sSup_le ?_
      rintro _ ⟨_, ⟨j, hj, rfl⟩, rfl⟩
      obtain ⟨m, hm, hjm, hkm⟩ := hSdir j hj k hk
      calc (k : D → D) (j x) ≤ (m : D → D) (j x) := hkm (j x)
        _ ≤ (m : D → D) (m x) := m.monotone (hjm x)
        _ = (m : D → D) x := hidem m hm x
        _ ≤ sSup A := le_sSup (Set.mem_image_of_mem _ hm)
    · rw [hsSx, ScottMap.sSup_apply]
      refine sSup_le ?_
      rintro _ ⟨j, hj, rfl⟩
      have hjxA : (j : D → D) x ≤ sSup A := le_sSup (Set.mem_image_of_mem _ hj)
      calc (j : D → D) x = (j : D → D) (j x) := (hidem j hj x).symm
        _ ≤ (j : D → D) (sSup A) := j.monotone hjxA
        _ ≤ sSup (Set.image (fun f : ScottMap D D => (f : D → D) (sSup A)) S) :=
              le_sSup (Set.mem_image_of_mem _ hj)
  · rw [ScottMap.sSup_apply]
    refine sSup_le ?_
    rintro _ ⟨j, hj, rfl⟩
    exact hle j hj x

end Projections

end DomainTheory.ContinuousLattice
```

4

isProjection_directedSup.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst30
public import DomainTheory.ContinuousLattice.FunctionSpaces.IsProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_iff
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

/-- `J_D` is closed under *directed* joins (Scott 1972, 3.12). Continuity of each
    member lets the
inner `(⊔S)`-application distribute over the directed family `{ j x : j ∈ S }`, and
    directedness
plus idempotency collapse the double family `{ k (j x) }` to `(⊔S) x`. -/
theorem isProjection_directedSup {S : Set (ScottMap D D)} (hSne : S.Nonempty)
    (hSdir : DirectedOn (· ≤ ·) S) (hS : ∀ j ∈ S, IsProjection j) :
    IsProjection (sSup S) := by
  have hidem : ∀ j ∈ S, ∀ x, (j : D → D) (j x) = j x :=
    fun j hj => (isProjection_iff.mp (hS j hj)).1
  have hle : ∀ j ∈ S, ∀ x, (j : D → D) x ≤ x := fun j hj => (isProjection_iff.mp (hS
    j hj)).2
  rw [isProjection_iff]
  refine ⟨fun x => ?_, fun x => ?_⟩
  · set A := Set.image (fun j : ScottMap D D => (j : D → D) x) S with hA
    have hAne : A.Nonempty := hSne.image _
    have hAdir : DirectedOn (· ≤ ·) A := by
      rintro _ ⟨j, hj, rfl⟩ _ ⟨k, hk, rfl⟩
      obtain ⟨m, hm, hjm, hkm⟩ := hSdir j hj k hk
      exact ⟨(m : D → D) x, Set.mem_image_of_mem _ hm, hjm x, hkm x⟩
    have hsSx : ((sSup S : ScottMap D D) : D → D) x = sSup A := ScottMap.sSup_apply S
    x
    apply le_antisymm
    · rw [hsSx, ScottMap.sSup_apply]
```

```
      refine sSup_le ?_
      rintro _ ⟨k, hk, rfl⟩
      show (k : D → D) (sSup A) ≤ sSup A
      rw [k.preservesDirectedSup_coe A hAne hAdir]
      refine sSup_le ?_
      rintro _ ⟨_, ⟨j, hj, rfl⟩, rfl⟩
      obtain ⟨m, hm, hjm, hkm⟩ := hSdir j hj k hk
      calc (k : D → D) (j x) ≤ (m : D → D) (j x) := hkm (j x)
        _ ≤ (m : D → D) (m x) := m.monotone (hjm x)
        _ = (m : D → D) x := hidem m hm x
        _ ≤ sSup A := le_sSup (Set.mem_image_of_mem _ hm)
    · rw [hsSx, ScottMap.sSup_apply]
      refine sSup_le ?_
      rintro _ ⟨j, hj, rfl⟩
      have hjxA : (j : D → D) x ≤ sSup A := le_sSup (Set.mem_image_of_mem _ hj)
      calc (j : D → D) x = (j : D → D) (j x) := (hidem j hj x).symm
        _ ≤ (j : D → D) (sSup A) := j.monotone hjxA
        _ ≤ sSup (Set.image (fun f : ScottMap D D => (f : D → D) (sSup A)) S) :=
              le_sSup (Set.mem_image_of_mem _ hj)
  · rw [ScottMap.sSup_apply]
    refine sSup_le ?_
    rintro _ ⟨j, hj, rfl⟩
    exact hle j hj x

end Projections

end DomainTheory.ContinuousLattice
```

5

isProjection_directedSup.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst30
public import DomainTheory.ContinuousLattice.FunctionSpaces.IsProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_iff
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
```

```
    CompleteLattice D₁']

section Projections

/-- `J_D` is closed under *directed* joins (Scott 1972, 3.12). Continuity of each
    member lets the
inner `(⊔S)`-application distribute over the directed family `{ j x : j ∈ S }`, and
    directedness
plus idempotency collapse the double family `{ k (j x) }` to `(⊔S) x`. -/
theorem isProjection_directedSup {S : Set (ScottMap D D)} (hSne : S.Nonempty)
    (hSdir : DirectedOn (· ≤ ·) S) (hS : ∀ j ∈ S, IsProjection j) :
    IsProjection (sSup S) := by
  have hidem : ∀ j ∈ S, ∀ x, (j : D → D) (j x) = j x :=
    fun j hj => (isProjection_iff.mp (hS j hj)).1
  have hle : ∀ j ∈ S, ∀ x, (j : D → D) x ≤ x := fun j hj => (isProjection_iff.mp (hS
    j hj)).2
  rw [isProjection_iff]
  refine ⟨fun x => ?_, fun x => ?_⟩
  · set A := Set.image (fun j : ScottMap D D => (j : D → D) x) S with hA
    have hAne : A.Nonempty := hSne.image _
    have hAdir : DirectedOn (· ≤ ·) A := by
      rintro _ ⟨j, hj, rfl⟩ _ ⟨k, hk, rfl⟩
      obtain ⟨m, hm, hjm, hkm⟩ := hSdir j hj k hk
      exact ⟨(m : D → D) x, Set.mem_image_of_mem _ hm, hjm x, hkm x⟩
    have hsSx : ((sSup S : ScottMap D D) : D → D) x = sSup A := ScottMap.sSup_apply S
    x
    apply le_antisymm
    · rw [hsSx, ScottMap.sSup_apply]
      refine sSup_le ?_
      rintro _ ⟨k, hk, rfl⟩
      show (k : D → D) (sSup A) ≤ sSup A
      rw [k.preservesDirectedSup_coe A hAne hAdir]
      refine sSup_le ?_
      rintro _ ⟨_, ⟨j, hj, rfl⟩, rfl⟩
      obtain ⟨m, hm, hjm, hkm⟩ := hSdir j hj k hk
      calc (k : D → D) (j x) ≤ (m : D → D) (j x) := hkm (j x)
        _ ≤ (m : D → D) (m x) := m.monotone (hjm x)
        _ = (m : D → D) x := hidem m hm x
        _ ≤ sSup A := le_sSup (Set.mem_image_of_mem _ hm)
    · rw [hsSx, ScottMap.sSup_apply]
      refine sSup_le ?_
      rintro _ ⟨j, hj, rfl⟩
      have hjxA : (j : D → D) x ≤ sSup A := le_sSup (Set.mem_image_of_mem _ hj)
      calc (j : D → D) x = (j : D → D) (j x) := (hidem j hj x).symm
        _ ≤ (j : D → D) (sSup A) := j.monotone hjxA
        _ ≤ sSup (Set.image (fun f : ScottMap D D => (f : D → D) (sSup A)) S) :=
              le_sSup (Set.mem_image_of_mem _ hj)
  · rw [ScottMap.sSup_apply]
    refine sSup_le ?_
    rintro _ ⟨j, hj, rfl⟩
    exact hle j hj x

end Projections

end DomainTheory.ContinuousLattice
```

6

isProjection_directedSup.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst30
public import DomainTheory.ContinuousLattice.FunctionSpaces.IsProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_iff
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

/-- `J_D` is closed under *directed* joins (Scott 1972, 3.12). Continuity of each
    member lets the
inner `(⊔S)`-application distribute over the directed family `{ j x : j ∈ S }`, and
    directedness
plus idempotency collapse the double family `{ k (j x) }` to `(⊔S) x`. -/
theorem isProjection_directedSup {S : Set (ScottMap D D)} (hSne : S.Nonempty)
    (hSdir : DirectedOn (· ≤ ·) S) (hS : ∀ j ∈ S, IsProjection j) :
    IsProjection (sSup S) := by
  have hidem : ∀ j ∈ S, ∀ x, (j : D → D) (j x) = j x :=
    fun j hj => (isProjection_iff.mp (hS j hj)).1
  have hle : ∀ j ∈ S, ∀ x, (j : D → D) x ≤ x := fun j hj => (isProjection_iff.mp (hS
    j hj)).2
  rw [isProjection_iff]
  refine ⟨fun x => ?_, fun x => ?_⟩
  · set A := Set.image (fun j : ScottMap D D => (j : D → D) x) S with hA
    have hAne : A.Nonempty := hSne.image _
    have hAdir : DirectedOn (· ≤ ·) A := by
      rintro _ ⟨j, hj, rfl⟩ _ ⟨k, hk, rfl⟩
      obtain ⟨m, hm, hjm, hkm⟩ := hSdir j hj k hk
      exact ⟨(m : D → D) x, Set.mem_image_of_mem _ hm, hjm x, hkm x⟩
    have hsSx : ((sSup S : ScottMap D D) : D → D) x = sSup A := ScottMap.sSup_apply S
    x
    apply le_antisymm
    · rw [hsSx, ScottMap.sSup_apply]
```

```
      refine sSup_le ?_
      rintro _ ⟨k, hk, rfl⟩
      show (k : D → D) (sSup A) ≤ sSup A
      rw [k.preservesDirectedSup_coe A hAne hAdir]
      refine sSup_le ?_
      rintro _ ⟨_, ⟨j, hj, rfl⟩, rfl⟩
      obtain ⟨m, hm, hjm, hkm⟩ := hSdir j hj k hk
      calc (k : D → D) (j x) ≤ (m : D → D) (j x) := hkm (j x)
        _ ≤ (m : D → D) (m x) := m.monotone (hjm x)
        _ = (m : D → D) x := hidem m hm x
        _ ≤ sSup A := le_sSup (Set.mem_image_of_mem _ hm)
    · rw [hsSx, ScottMap.sSup_apply]
      refine sSup_le ?_
      rintro _ ⟨j, hj, rfl⟩
      have hjxA : (j : D → D) x ≤ sSup A := le_sSup (Set.mem_image_of_mem _ hj)
      calc (j : D → D) x = (j : D → D) (j x) := (hidem j hj x).symm
        _ ≤ (j : D → D) (sSup A) := j.monotone hjxA
        _ ≤ sSup (Set.image (fun f : ScottMap D D => (f : D → D) (sSup A)) S) :=
              le_sSup (Set.mem_image_of_mem _ hj)
  · rw [ScottMap.sSup_apply]
    refine sSup_le ?_
    rintro _ ⟨j, hj, rfl⟩
    exact hle j hj x

end Projections

end DomainTheory.ContinuousLattice
```

7

isProjection_directedSup.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst30
public import DomainTheory.ContinuousLattice.FunctionSpaces.IsProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_iff
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
```

```
    CompleteLattice D₁']

section Projections

/-- `J_D` is closed under *directed* joins (Scott 1972, 3.12). Continuity of each
    member lets the
inner `(⊔S)`-application distribute over the directed family `{ j x : j ∈ S }`, and
    directedness
plus idempotency collapse the double family `{ k (j x) }` to `(⊔S) x`. -/
theorem isProjection_directedSup {S : Set (ScottMap D D)} (hSne : S.Nonempty)
    (hSdir : DirectedOn (· ≤ ·) S) (hS : ∀ j ∈ S, IsProjection j) :
    IsProjection (sSup S) := by
  have hidem : ∀ j ∈ S, ∀ x, (j : D → D) (j x) = j x :=
    fun j hj => (isProjection_iff.mp (hS j hj)).1
  have hle : ∀ j ∈ S, ∀ x, (j : D → D) x ≤ x := fun j hj => (isProjection_iff.mp (hS
    j hj)).2
  rw [isProjection_iff]
  refine ⟨fun x => ?_, fun x => ?_⟩
  · set A := Set.image (fun j : ScottMap D D => (j : D → D) x) S with hA
    have hAne : A.Nonempty := hSne.image _
    have hAdir : DirectedOn (· ≤ ·) A := by
      rintro _ ⟨j, hj, rfl⟩ _ ⟨k, hk, rfl⟩
      obtain ⟨m, hm, hjm, hkm⟩ := hSdir j hj k hk
      exact ⟨(m : D → D) x, Set.mem_image_of_mem _ hm, hjm x, hkm x⟩
    have hsSx : ((sSup S : ScottMap D D) : D → D) x = sSup A := ScottMap.sSup_apply S
    x
    apply le_antisymm
    · rw [hsSx, ScottMap.sSup_apply]
      refine sSup_le ?_
      rintro _ ⟨k, hk, rfl⟩
      show (k : D → D) (sSup A) ≤ sSup A
      rw [k.preservesDirectedSup_coe A hAne hAdir]
      refine sSup_le ?_
      rintro _ ⟨_, ⟨j, hj, rfl⟩, rfl⟩
      obtain ⟨m, hm, hjm, hkm⟩ := hSdir j hj k hk
      calc (k : D → D) (j x) ≤ (m : D → D) (j x) := hkm (j x)
        _ ≤ (m : D → D) (m x) := m.monotone (hjm x)
        _ = (m : D → D) x := hidem m hm x
        _ ≤ sSup A := le_sSup (Set.mem_image_of_mem _ hm)
    · rw [hsSx, ScottMap.sSup_apply]
      refine sSup_le ?_
      rintro _ ⟨j, hj, rfl⟩
      have hjxA : (j : D → D) x ≤ sSup A := le_sSup (Set.mem_image_of_mem _ hj)
      calc (j : D → D) x = (j : D → D) (j x) := (hidem j hj x).symm
        _ ≤ (j : D → D) (sSup A) := j.monotone hjxA
        _ ≤ sSup (Set.image (fun f : ScottMap D D => (f : D → D) (sSup A)) S) :=
              le_sSup (Set.mem_image_of_mem _ hj)
  · rw [ScottMap.sSup_apply]
    refine sSup_le ?_
    rintro _ ⟨j, hj, rfl⟩
    exact hle j hj x

end Projections

end DomainTheory.ContinuousLattice
```

8

isProjection_directedSup.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst30
public import DomainTheory.ContinuousLattice.FunctionSpaces.IsProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_iff
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

/-- `J_D` is closed under *directed* joins (Scott 1972, 3.12). Continuity of each
    member lets the
inner `(⊔S)`-application distribute over the directed family `{ j x : j ∈ S }`, and
    directedness
plus idempotency collapse the double family `{ k (j x) }` to `(⊔S) x`. -/
theorem isProjection_directedSup {S : Set (ScottMap D D)} (hSne : S.Nonempty)
    (hSdir : DirectedOn (· ≤ ·) S) (hS : ∀ j ∈ S, IsProjection j) :
    IsProjection (sSup S) := by
  have hidem : ∀ j ∈ S, ∀ x, (j : D → D) (j x) = j x :=
    fun j hj => (isProjection_iff.mp (hS j hj)).1
  have hle : ∀ j ∈ S, ∀ x, (j : D → D) x ≤ x := fun j hj => (isProjection_iff.mp (hS
    j hj)).2
  rw [isProjection_iff]
  refine ⟨fun x => ?_, fun x => ?_⟩
  · set A := Set.image (fun j : ScottMap D D => (j : D → D) x) S with hA
    have hAne : A.Nonempty := hSne.image _
    have hAdir : DirectedOn (· ≤ ·) A := by
      rintro _ ⟨j, hj, rfl⟩ _ ⟨k, hk, rfl⟩
      obtain ⟨m, hm, hjm, hkm⟩ := hSdir j hj k hk
      exact ⟨(m : D → D) x, Set.mem_image_of_mem _ hm, hjm x, hkm x⟩
    have hsSx : ((sSup S : ScottMap D D) : D → D) x = sSup A := ScottMap.sSup_apply S
    x
    apply le_antisymm
    · rw [hsSx, ScottMap.sSup_apply]
```

```
      refine sSup_le ?_
      rintro _ ⟨k, hk, rfl⟩
      show (k : D → D) (sSup A) ≤ sSup A
      rw [k.preservesDirectedSup_coe A hAne hAdir]
      refine sSup_le ?_
      rintro _ ⟨_, ⟨j, hj, rfl⟩, rfl⟩
      obtain ⟨m, hm, hjm, hkm⟩ := hSdir j hj k hk
      calc (k : D → D) (j x) ≤ (m : D → D) (j x) := hkm (j x)
        _ ≤ (m : D → D) (m x) := m.monotone (hjm x)
        _ = (m : D → D) x := hidem m hm x
        _ ≤ sSup A := le_sSup (Set.mem_image_of_mem _ hm)
    · rw [hsSx, ScottMap.sSup_apply]
      refine sSup_le ?_
      rintro _ ⟨j, hj, rfl⟩
      have hjxA : (j : D → D) x ≤ sSup A := le_sSup (Set.mem_image_of_mem _ hj)
      calc (j : D → D) x = (j : D → D) (j x) := (hidem j hj x).symm
        _ ≤ (j : D → D) (sSup A) := j.monotone hjxA
        _ ≤ sSup (Set.image (fun f : ScottMap D D => (f : D → D) (sSup A)) S) :=
              le_sSup (Set.mem_image_of_mem _ hj)
  · rw [ScottMap.sSup_apply]
    refine sSup_le ?_
    rintro _ ⟨j, hj, rfl⟩
    exact hle j hj x

end Projections

end DomainTheory.ContinuousLattice
```

9

isProjection_finsetSup.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.IsProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_sup
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_bot
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']
```

```
section Projections

/-- `J_D` is closed under finite joins. -/
theorem isProjection_finsetSup {T : Finset (ScottMap D D)} (hT : ∀ j ∈ T, IsProjection
     j) :
    IsProjection (T.sup id) :=
  Finset.sup_induction isProjection_bot (fun _ ha _ hb => isProjection_sup ha hb) hT

end Projections

end DomainTheory.ContinuousLattice
```

0

isProjection_finsetSup.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.IsProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_sup
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_bot
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

/-- `J_D` is closed under finite joins. -/
theorem isProjection_finsetSup {T : Finset (ScottMap D D)} (hT : ∀ j ∈ T, IsProjection
     j) :
    IsProjection (T.sup id) :=
  Finset.sup_induction isProjection_bot (fun _ ha _ hb => isProjection_sup ha hb) hT

end Projections

end DomainTheory.ContinuousLattice
```

1

isProjection_finsetSup.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.IsProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_sup
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_bot
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

/-- `J_D` is closed under finite joins. -/
theorem isProjection_finsetSup {T : Finset (ScottMap D D)} (hT : ∀ j ∈ T, IsProjection
     j) :
    IsProjection (T.sup id) :=
  Finset.sup_induction isProjection_bot (fun _ ha _ hb => isProjection_sup ha hb) hT

end Projections

end DomainTheory.ContinuousLattice
```

2

isProjection_finsetSup.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.IsProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_sup
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_bot
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
```

```
variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

/-- `J_D` is closed under finite joins. -/
theorem isProjection_finsetSup {T : Finset (ScottMap D D)} (hT : ∀ j ∈ T, IsProjection
     j) :
    IsProjection (T.sup id) :=
  Finset.sup_induction isProjection_bot (fun _ ha _ hb => isProjection_sup ha hb) hT

end Projections

end DomainTheory.ContinuousLattice
```

3

isProjection_finsetSup.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.IsProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_sup
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_bot
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

/-- `J_D` is closed under finite joins. -/
theorem isProjection_finsetSup {T : Finset (ScottMap D D)} (hT : ∀ j ∈ T, IsProjection
     j) :
    IsProjection (T.sup id) :=
  Finset.sup_induction isProjection_bot (fun _ ha _ hb => isProjection_sup ha hb) hT
```

```
end Projections

end DomainTheory.ContinuousLattice
```

4

isProjection_finsetSup.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.IsProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_sup
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_bot
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

/-- `J_D` is closed under finite joins. -/
theorem isProjection_finsetSup {T : Finset (ScottMap D D)} (hT : ∀ j ∈ T, IsProjection
     j) :
    IsProjection (T.sup id) :=
  Finset.sup_induction isProjection_bot (fun _ ha _ hb => isProjection_sup ha hb) hT

end Projections

end DomainTheory.ContinuousLattice
```

5

isProjection_finsetSup.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.IsProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_sup
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_bot
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic
```

```
@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

/-- `J_D` is closed under finite joins. -/
theorem isProjection_finsetSup {T : Finset (ScottMap D D)} (hT : ∀ j ∈ T, IsProjection
     j) :
    IsProjection (T.sup id) :=
  Finset.sup_induction isProjection_bot (fun _ ha _ hb => isProjection_sup ha hb) hT

end Projections

end DomainTheory.ContinuousLattice
```

6

isProjection_finsetSup.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.IsProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_sup
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_bot
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']
```

```
section Projections

/-- `J_D` is closed under finite joins. -/
theorem isProjection_finsetSup {T : Finset (ScottMap D D)} (hT : ∀ j ∈ T, IsProjection
     j) :
    IsProjection (T.sup id) :=
  Finset.sup_induction isProjection_bot (fun _ ha _ hb => isProjection_sup ha hb) hT

end Projections

end DomainTheory.ContinuousLattice
```

7

isProjection_finsetSup.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.IsProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_sup
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_bot
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

/-- `J_D` is closed under finite joins. -/
theorem isProjection_finsetSup {T : Finset (ScottMap D D)} (hT : ∀ j ∈ T, IsProjection
     j) :
    IsProjection (T.sup id) :=
  Finset.sup_induction isProjection_bot (fun _ ha _ hb => isProjection_sup ha hb) hT

end Projections

end DomainTheory.ContinuousLattice
```

8

isProjection_finsetSup.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.IsProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_sup
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_bot
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

/-- `J_D` is closed under finite joins. -/
theorem isProjection_finsetSup {T : Finset (ScottMap D D)} (hT : ∀ j ∈ T, IsProjection
     j) :
    IsProjection (T.sup id) :=
  Finset.sup_induction isProjection_bot (fun _ ha _ hb => isProjection_sup ha hb) hT

end Projections

end DomainTheory.ContinuousLattice
```

9

isProjection_iff.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.IsProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.comp_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.idMap_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
```

```
variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

/-- Pointwise characterization of projections: idempotent and deflationary. -/
theorem isProjection_iff {j : ScottMap D D} :
    IsProjection j ↔ (∀ x, (j : D → D) (j x) = j x) ∧ (∀ x, (j : D → D) x ≤ x) := by
  constructor
  · rintro ⟨hidem, hle⟩
    refine ⟨fun x => ?_, fun x => ?_⟩
    · have h : ((j.comp j : ScottMap D D) : D → D) x = (j : D → D) x := by rw [hidem]
      rwa [ScottMap.comp_apply] at h
    · have h := (ScottMap.le_def.mp hle) x
      rwa [ScottMap.idMap_apply] at h
  · rintro ⟨hidem, hle⟩
    exact ⟨ScottMap.ext fun x => by rw [ScottMap.comp_apply]; exact hidem x,
      ScottMap.le_def.mpr fun x => by rw [ScottMap.idMap_apply]; exact hle x⟩

end Projections

end DomainTheory.ContinuousLattice
```

0

isProjection_iff.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.IsProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.comp_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.idMap_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']
```

```
section Projections

/-- Pointwise characterization of projections: idempotent and deflationary. -/
theorem isProjection_iff {j : ScottMap D D} :
    IsProjection j ↔ (∀ x, (j : D → D) (j x) = j x) ∧ (∀ x, (j : D → D) x ≤ x) := by
  constructor
  · rintro ⟨hidem, hle⟩
    refine ⟨fun x => ?_, fun x => ?_⟩
    · have h : ((j.comp j : ScottMap D D) : D → D) x = (j : D → D) x := by rw [hidem]
      rwa [ScottMap.comp_apply] at h
    · have h := (ScottMap.le_def.mp hle) x
      rwa [ScottMap.idMap_apply] at h
  · rintro ⟨hidem, hle⟩
    exact ⟨ScottMap.ext fun x => by rw [ScottMap.comp_apply]; exact hidem x,
      ScottMap.le_def.mpr fun x => by rw [ScottMap.idMap_apply]; exact hle x⟩

end Projections

end DomainTheory.ContinuousLattice
```

1

isProjection_iff.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.IsProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.comp_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.idMap_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

/-- Pointwise characterization of projections: idempotent and deflationary. -/
theorem isProjection_iff {j : ScottMap D D} :
    IsProjection j ↔ (∀ x, (j : D → D) (j x) = j x) ∧ (∀ x, (j : D → D) x ≤ x) := by
  constructor
  · rintro ⟨hidem, hle⟩
    refine ⟨fun x => ?_, fun x => ?_⟩
```

```
    · have h : ((j.comp j : ScottMap D D) : D → D) x = (j : D → D) x := by rw [hidem]
      rwa [ScottMap.comp_apply] at h
    · have h := (ScottMap.le_def.mp hle) x
      rwa [ScottMap.idMap_apply] at h
  · rintro ⟨hidem, hle⟩
    exact ⟨ScottMap.ext fun x => by rw [ScottMap.comp_apply]; exact hidem x,
      ScottMap.le_def.mpr fun x => by rw [ScottMap.idMap_apply]; exact hle x⟩

end Projections

end DomainTheory.ContinuousLattice
```

2

isProjection_iff.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.IsProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.comp_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.idMap_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

/-- Pointwise characterization of projections: idempotent and deflationary. -/
theorem isProjection_iff {j : ScottMap D D} :
    IsProjection j ↔ (∀ x, (j : D → D) (j x) = j x) ∧ (∀ x, (j : D → D) x ≤ x) := by
  constructor
  · rintro ⟨hidem, hle⟩
    refine ⟨fun x => ?_, fun x => ?_⟩
    · have h : ((j.comp j : ScottMap D D) : D → D) x = (j : D → D) x := by rw [hidem]
      rwa [ScottMap.comp_apply] at h
    · have h := (ScottMap.le_def.mp hle) x
      rwa [ScottMap.idMap_apply] at h
  · rintro ⟨hidem, hle⟩
    exact ⟨ScottMap.ext fun x => by rw [ScottMap.comp_apply]; exact hidem x,
      ScottMap.le_def.mpr fun x => by rw [ScottMap.idMap_apply]; exact hle x⟩
```

```
end Projections

end DomainTheory.ContinuousLattice
```

3

isProjection_iff.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.IsProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.comp_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.idMap_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

/-- Pointwise characterization of projections: idempotent and deflationary. -/
theorem isProjection_iff {j : ScottMap D D} :
    IsProjection j ↔ (∀ x, (j : D → D) (j x) = j x) ∧ (∀ x, (j : D → D) x ≤ x) := by
  constructor
  · rintro ⟨hidem, hle⟩
    refine ⟨fun x => ?_, fun x => ?_⟩
    · have h : ((j.comp j : ScottMap D D) : D → D) x = (j : D → D) x := by rw [hidem]
      rwa [ScottMap.comp_apply] at h
    · have h := (ScottMap.le_def.mp hle) x
      rwa [ScottMap.idMap_apply] at h
  · rintro ⟨hidem, hle⟩
    exact ⟨ScottMap.ext fun x => by rw [ScottMap.comp_apply]; exact hidem x,
      ScottMap.le_def.mpr fun x => by rw [ScottMap.idMap_apply]; exact hle x⟩

end Projections

end DomainTheory.ContinuousLattice
```

4

isProjection_iff.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.IsProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.comp_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.idMap_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

/-- Pointwise characterization of projections: idempotent and deflationary. -/
theorem isProjection_iff {j : ScottMap D D} :
    IsProjection j ↔ (∀ x, (j : D → D) (j x) = j x) ∧ (∀ x, (j : D → D) x ≤ x) := by
  constructor
  · rintro ⟨hidem, hle⟩
    refine ⟨fun x => ?_, fun x => ?_⟩
    · have h : ((j.comp j : ScottMap D D) : D → D) x = (j : D → D) x := by rw [hidem]
      rwa [ScottMap.comp_apply] at h
    · have h := (ScottMap.le_def.mp hle) x
      rwa [ScottMap.idMap_apply] at h
  · rintro ⟨hidem, hle⟩
    exact ⟨ScottMap.ext fun x => by rw [ScottMap.comp_apply]; exact hidem x,
      ScottMap.le_def.mpr fun x => by rw [ScottMap.idMap_apply]; exact hle x⟩

end Projections

end DomainTheory.ContinuousLattice
```

5

isProjection_iff.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.IsProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.comp_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.idMap_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic
```

```
@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

/-- Pointwise characterization of projections: idempotent and deflationary. -/
theorem isProjection_iff {j : ScottMap D D} :
    IsProjection j ↔ (∀ x, (j : D → D) (j x) = j x) ∧ (∀ x, (j : D → D) x ≤ x) := by
  constructor
  · rintro ⟨hidem, hle⟩
    refine ⟨fun x => ?_, fun x => ?_⟩
    · have h : ((j.comp j : ScottMap D D) : D → D) x = (j : D → D) x := by rw [hidem]
      rwa [ScottMap.comp_apply] at h
    · have h := (ScottMap.le_def.mp hle) x
      rwa [ScottMap.idMap_apply] at h
  · rintro ⟨hidem, hle⟩
    exact ⟨ScottMap.ext fun x => by rw [ScottMap.comp_apply]; exact hidem x,
      ScottMap.le_def.mpr fun x => by rw [ScottMap.idMap_apply]; exact hle x⟩

end Projections

end DomainTheory.ContinuousLattice
```

6

isProjection_iff.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.IsProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.comp_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.idMap_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
```

```
variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

/-- Pointwise characterization of projections: idempotent and deflationary. -/
theorem isProjection_iff {j : ScottMap D D} :
    IsProjection j ↔ (∀ x, (j : D → D) (j x) = j x) ∧ (∀ x, (j : D → D) x ≤ x) := by
  constructor
  · rintro ⟨hidem, hle⟩
    refine ⟨fun x => ?_, fun x => ?_⟩
    · have h : ((j.comp j : ScottMap D D) : D → D) x = (j : D → D) x := by rw [hidem]
      rwa [ScottMap.comp_apply] at h
    · have h := (ScottMap.le_def.mp hle) x
      rwa [ScottMap.idMap_apply] at h
  · rintro ⟨hidem, hle⟩
    exact ⟨ScottMap.ext fun x => by rw [ScottMap.comp_apply]; exact hidem x,
      ScottMap.le_def.mpr fun x => by rw [ScottMap.idMap_apply]; exact hle x⟩

end Projections

end DomainTheory.ContinuousLattice
```

7

isProjection_iff.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.IsProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.comp_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.idMap_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']
```

```
section Projections

/-- Pointwise characterization of projections: idempotent and deflationary. -/
theorem isProjection_iff {j : ScottMap D D} :
    IsProjection j ↔ (∀ x, (j : D → D) (j x) = j x) ∧ (∀ x, (j : D → D) x ≤ x) := by
  constructor
  · rintro ⟨hidem, hle⟩
    refine ⟨fun x => ?_, fun x => ?_⟩
    · have h : ((j.comp j : ScottMap D D) : D → D) x = (j : D → D) x := by rw [hidem]
      rwa [ScottMap.comp_apply] at h
    · have h := (ScottMap.le_def.mp hle) x
      rwa [ScottMap.idMap_apply] at h
  · rintro ⟨hidem, hle⟩
    exact ⟨ScottMap.ext fun x => by rw [ScottMap.comp_apply]; exact hidem x,
      ScottMap.le_def.mpr fun x => by rw [ScottMap.idMap_apply]; exact hle x⟩

end Projections

end DomainTheory.ContinuousLattice
```

8

isProjection_iff.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.IsProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.comp_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.idMap_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

/-- Pointwise characterization of projections: idempotent and deflationary. -/
theorem isProjection_iff {j : ScottMap D D} :
    IsProjection j ↔ (∀ x, (j : D → D) (j x) = j x) ∧ (∀ x, (j : D → D) x ≤ x) := by
  constructor
  · rintro ⟨hidem, hle⟩
    refine ⟨fun x => ?_, fun x => ?_⟩
```

```
    · have h : ((j.comp j : ScottMap D D) : D → D) x = (j : D → D) x := by rw [hidem]
      rwa [ScottMap.comp_apply] at h
    · have h := (ScottMap.le_def.mp hle) x
      rwa [ScottMap.idMap_apply] at h
  · rintro ⟨hidem, hle⟩
    exact ⟨ScottMap.ext fun x => by rw [ScottMap.comp_apply]; exact hidem x,
      ScottMap.le_def.mpr fun x => by rw [ScottMap.idMap_apply]; exact hle x⟩

end Projections

end DomainTheory.ContinuousLattice
```

9

isProjection_sSup.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst30
public import DomainTheory.ContinuousLattice.FunctionSpaces.IsProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_directedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_finsetSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.directedOn_finsetSupOf
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_finsetSupOf
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

/-- **Scott 1972, Proposition 3.12 (`⊔`-closure).** `J_D` is closed under arbitrary
    suprema in
`[D → D]`: every supremum is the directed supremum of finite sub-joins (`finsetSupOf`)
    , each a
projection by `isProjection_finsetSup`. -/
theorem isProjection_sSup {T : Set (ScottMap D D)} (hT : ∀ j ∈ T, IsProjection j) :
    IsProjection (sSup T) := by
  rw [IsContinuousLatticeProjection.sSup_finsetSupOf T]
  refine isProjection_directedSup ⟨⊥, ∅, by simp, by simp⟩
    (IsContinuousLatticeProjection.directedOn_finsetSupOf T) ?_
  rintro z ⟨F, hF, rfl⟩
  exact isProjection_finsetSup fun j hj => hT j (hF j hj)
```

```
end Projections

end DomainTheory.ContinuousLattice
```

0

isProjection_sSup.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst30
public import DomainTheory.ContinuousLattice.FunctionSpaces.IsProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_directedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_finsetSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.directedOn_finsetSupOf
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_finsetSupOf
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

/-- **Scott 1972, Proposition 3.12 (`⊔`-closure).** `J_D` is closed under arbitrary
    suprema in
`[D → D]`: every supremum is the directed supremum of finite sub-joins (`finsetSupOf`)
    , each a
projection by `isProjection_finsetSup`. -/
theorem isProjection_sSup {T : Set (ScottMap D D)} (hT : ∀ j ∈ T, IsProjection j) :
    IsProjection (sSup T) := by
  rw [IsContinuousLatticeProjection.sSup_finsetSupOf T]
  refine isProjection_directedSup ⟨⊥, ∅, by simp, by simp⟩
    (IsContinuousLatticeProjection.directedOn_finsetSupOf T) ?_
  rintro z ⟨F, hF, rfl⟩
  exact isProjection_finsetSup fun j hj => hT j (hF j hj)

end Projections

end DomainTheory.ContinuousLattice
```

1

isProjection_sSup.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst30
public import DomainTheory.ContinuousLattice.FunctionSpaces.IsProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_directedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_finsetSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.directedOn_finsetSupOf
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_finsetSupOf
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

/-- **Scott 1972, Proposition 3.12 (`⊔`-closure).** `J_D` is closed under arbitrary
    suprema in
`[D → D]`: every supremum is the directed supremum of finite sub-joins (`finsetSupOf`)
    , each a
projection by `isProjection_finsetSup`. -/
theorem isProjection_sSup {T : Set (ScottMap D D)} (hT : ∀ j ∈ T, IsProjection j) :
    IsProjection (sSup T) := by
  rw [IsContinuousLatticeProjection.sSup_finsetSupOf T]
  refine isProjection_directedSup ⟨⊥, ∅, by simp, by simp⟩
    (IsContinuousLatticeProjection.directedOn_finsetSupOf T) ?_
  rintro z ⟨F, hF, rfl⟩
  exact isProjection_finsetSup fun j hj => hT j (hF j hj)

end Projections

end DomainTheory.ContinuousLattice
```

2

isProjection_sSup.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst30
public import DomainTheory.ContinuousLattice.FunctionSpaces.IsProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_directedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_finsetSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.directedOn_finsetSupOf
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_finsetSupOf
```

```
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

/-- **Scott 1972, Proposition 3.12 (`⊔`-closure).** `J_D` is closed under arbitrary
    suprema in
`[D → D]`: every supremum is the directed supremum of finite sub-joins (`finsetSupOf`)
    , each a
projection by `isProjection_finsetSup`. -/
theorem isProjection_sSup {T : Set (ScottMap D D)} (hT : ∀ j ∈ T, IsProjection j) :
    IsProjection (sSup T) := by
  rw [IsContinuousLatticeProjection.sSup_finsetSupOf T]
  refine isProjection_directedSup ⟨⊥, ∅, by simp, by simp⟩
    (IsContinuousLatticeProjection.directedOn_finsetSupOf T) ?_
  rintro z ⟨F, hF, rfl⟩
  exact isProjection_finsetSup fun j hj => hT j (hF j hj)

end Projections

end DomainTheory.ContinuousLattice
```

3

isProjection_sSup.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst30
public import DomainTheory.ContinuousLattice.FunctionSpaces.IsProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_directedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_finsetSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.directedOn_finsetSupOf
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_finsetSupOf
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace
```

```
variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

/-- **Scott 1972, Proposition 3.12 (`⊔`-closure).** `J_D` is closed under arbitrary
    suprema in
`[D → D]`: every supremum is the directed supremum of finite sub-joins (`finsetSupOf`)
    , each a
projection by `isProjection_finsetSup`. -/
theorem isProjection_sSup {T : Set (ScottMap D D)} (hT : ∀ j ∈ T, IsProjection j) :
    IsProjection (sSup T) := by
  rw [IsContinuousLatticeProjection.sSup_finsetSupOf T]
  refine isProjection_directedSup ⟨⊥, ∅, by simp, by simp⟩
    (IsContinuousLatticeProjection.directedOn_finsetSupOf T) ?_
  rintro z ⟨F, hF, rfl⟩
  exact isProjection_finsetSup fun j hj => hT j (hF j hj)

end Projections

end DomainTheory.ContinuousLattice
```

4

isProjection_sSup.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst30
public import DomainTheory.ContinuousLattice.FunctionSpaces.IsProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_directedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_finsetSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.directedOn_finsetSupOf
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_finsetSupOf
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']
```

```
section Projections

/-- **Scott 1972, Proposition 3.12 (`⊔`-closure).** `J_D` is closed under arbitrary
    suprema in
`[D → D]`: every supremum is the directed supremum of finite sub-joins (`finsetSupOf`)
    , each a
projection by `isProjection_finsetSup`. -/
theorem isProjection_sSup {T : Set (ScottMap D D)} (hT : ∀ j ∈ T, IsProjection j) :
    IsProjection (sSup T) := by
  rw [IsContinuousLatticeProjection.sSup_finsetSupOf T]
  refine isProjection_directedSup ⟨⊥, ∅, by simp, by simp⟩
    (IsContinuousLatticeProjection.directedOn_finsetSupOf T) ?_
  rintro z ⟨F, hF, rfl⟩
  exact isProjection_finsetSup fun j hj => hT j (hF j hj)

end Projections

end DomainTheory.ContinuousLattice
```

5

isProjection_sSup.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst30
public import DomainTheory.ContinuousLattice.FunctionSpaces.IsProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_directedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_finsetSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.directedOn_finsetSupOf
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_finsetSupOf
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

/-- **Scott 1972, Proposition 3.12 (`⊔`-closure).** `J_D` is closed under arbitrary
    suprema in
`[D → D]`: every supremum is the directed supremum of finite sub-joins (`finsetSupOf`)
    , each a
projection by `isProjection_finsetSup`. -/
```

```
theorem isProjection_sSup {T : Set (ScottMap D D)} (hT : ∀ j ∈ T, IsProjection j) :
    IsProjection (sSup T) := by
  rw [IsContinuousLatticeProjection.sSup_finsetSupOf T]
  refine isProjection_directedSup ⟨⊥, ∅, by simp, by simp⟩
    (IsContinuousLatticeProjection.directedOn_finsetSupOf T) ?_
  rintro z ⟨F, hF, rfl⟩
  exact isProjection_finsetSup fun j hj => hT j (hF j hj)

end Projections

end DomainTheory.ContinuousLattice
```

6

isProjection_sSup.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst30
public import DomainTheory.ContinuousLattice.FunctionSpaces.IsProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_directedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_finsetSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.directedOn_finsetSupOf
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_finsetSupOf
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

/-- **Scott 1972, Proposition 3.12 (`⊔`-closure).** `J_D` is closed under arbitrary
    suprema in
`[D → D]`: every supremum is the directed supremum of finite sub-joins (`finsetSupOf`)
    , each a
projection by `isProjection_finsetSup`. -/
theorem isProjection_sSup {T : Set (ScottMap D D)} (hT : ∀ j ∈ T, IsProjection j) :
    IsProjection (sSup T) := by
  rw [IsContinuousLatticeProjection.sSup_finsetSupOf T]
  refine isProjection_directedSup ⟨⊥, ∅, by simp, by simp⟩
    (IsContinuousLatticeProjection.directedOn_finsetSupOf T) ?_
  rintro z ⟨F, hF, rfl⟩
  exact isProjection_finsetSup fun j hj => hT j (hF j hj)
```

```
end Projections

end DomainTheory.ContinuousLattice
```

7

isProjection_sSup.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst30
public import DomainTheory.ContinuousLattice.FunctionSpaces.IsProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_directedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_finsetSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.directedOn_finsetSupOf
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_finsetSupOf
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

/-- **Scott 1972, Proposition 3.12 (`⊔`-closure).** `J_D` is closed under arbitrary
    suprema in
`[D → D]`: every supremum is the directed supremum of finite sub-joins (`finsetSupOf`)
    , each a
projection by `isProjection_finsetSup`. -/
theorem isProjection_sSup {T : Set (ScottMap D D)} (hT : ∀ j ∈ T, IsProjection j) :
    IsProjection (sSup T) := by
  rw [IsContinuousLatticeProjection.sSup_finsetSupOf T]
  refine isProjection_directedSup ⟨⊥, ∅, by simp, by simp⟩
    (IsContinuousLatticeProjection.directedOn_finsetSupOf T) ?_
  rintro z ⟨F, hF, rfl⟩
  exact isProjection_finsetSup fun j hj => hT j (hF j hj)

end Projections

end DomainTheory.ContinuousLattice
```

8

isProjection_sSup.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst30
public import DomainTheory.ContinuousLattice.FunctionSpaces.IsProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_directedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_finsetSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.directedOn_finsetSupOf
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_finsetSupOf
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

/-- **Scott 1972, Proposition 3.12 (`⊔`-closure).** `J_D` is closed under arbitrary
    suprema in
`[D → D]`: every supremum is the directed supremum of finite sub-joins (`finsetSupOf`)
    , each a
projection by `isProjection_finsetSup`. -/
theorem isProjection_sSup {T : Set (ScottMap D D)} (hT : ∀ j ∈ T, IsProjection j) :
    IsProjection (sSup T) := by
  rw [IsContinuousLatticeProjection.sSup_finsetSupOf T]
  refine isProjection_directedSup ⟨⊥, ∅, by simp, by simp⟩
    (IsContinuousLatticeProjection.directedOn_finsetSupOf T) ?_
  rintro z ⟨F, hF, rfl⟩
  exact isProjection_finsetSup fun j hj => hT j (hF j hj)

end Projections

end DomainTheory.ContinuousLattice
```

9

isProjection_sup.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.IsProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.sup_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_iff
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
```

```
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

/-- `J_D` is closed under binary joins (Scott 1972, 3.12). The key step: since `j x ⊔ k
     x ⊑ x`,
monotonicity and idempotency force `j (j x ⊔ k x) = j x` and `k (j x ⊔ k x) = k x`. -/
theorem isProjection_sup {j k : ScottMap D D} (hj : IsProjection j) (hk : IsProjection
     k) :
    IsProjection (j ⊔ k) := by
  rw [isProjection_iff] at hj hk ⊢
  obtain ⟨hjidem, hjle⟩ := hj
  obtain ⟨hkidem, hkle⟩ := hk
  refine ⟨fun x => ?_, fun x => ?_⟩
  · have hjkle : (j : D → D) x ⊔ k x ≤ x := sup_le (hjle x) (hkle x)
    have hj_eq : (j : D → D) (j x ⊔ k x) = j x :=
      le_antisymm (j.monotone hjkle) (le_of_eq_of_le (hjidem x).symm (j.monotone
    le_sup_left))
    have hk_eq : (k : D → D) (j x ⊔ k x) = k x :=
      le_antisymm (k.monotone hjkle) (le_of_eq_of_le (hkidem x).symm (k.monotone
    le_sup_right))
    rw [ScottMap.sup_apply, ScottMap.sup_apply, hj_eq, hk_eq]
  · rw [ScottMap.sup_apply]; exact sup_le (hjle x) (hkle x)

end Projections

end DomainTheory.ContinuousLattice
```

0

isProjection_sup.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.IsProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.sup_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_iff
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic
```

```
@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

/-- `J_D` is closed under binary joins (Scott 1972, 3.12). The key step: since `j x ⊔ k
     x ⊑ x`,
monotonicity and idempotency force `j (j x ⊔ k x) = j x` and `k (j x ⊔ k x) = k x`. -/
theorem isProjection_sup {j k : ScottMap D D} (hj : IsProjection j) (hk : IsProjection
     k) :
    IsProjection (j ⊔ k) := by
  rw [isProjection_iff] at hj hk ⊢
  obtain ⟨hjidem, hjle⟩ := hj
  obtain ⟨hkidem, hkle⟩ := hk
  refine ⟨fun x => ?_, fun x => ?_⟩
  · have hjkle : (j : D → D) x ⊔ k x ≤ x := sup_le (hjle x) (hkle x)
    have hj_eq : (j : D → D) (j x ⊔ k x) = j x :=
      le_antisymm (j.monotone hjkle) (le_of_eq_of_le (hjidem x).symm (j.monotone
    le_sup_left))
    have hk_eq : (k : D → D) (j x ⊔ k x) = k x :=
      le_antisymm (k.monotone hjkle) (le_of_eq_of_le (hkidem x).symm (k.monotone
    le_sup_right))
    rw [ScottMap.sup_apply, ScottMap.sup_apply, hj_eq, hk_eq]
  · rw [ScottMap.sup_apply]; exact sup_le (hjle x) (hkle x)

end Projections

end DomainTheory.ContinuousLattice
```

1

isProjection_sup.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.IsProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.sup_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_iff
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic
```

```
@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

/-- `J_D` is closed under binary joins (Scott 1972, 3.12). The key step: since `j x ⊔ k
     x ⊑ x`,
monotonicity and idempotency force `j (j x ⊔ k x) = j x` and `k (j x ⊔ k x) = k x`. -/
theorem isProjection_sup {j k : ScottMap D D} (hj : IsProjection j) (hk : IsProjection
     k) :
    IsProjection (j ⊔ k) := by
  rw [isProjection_iff] at hj hk ⊢
  obtain ⟨hjidem, hjle⟩ := hj
  obtain ⟨hkidem, hkle⟩ := hk
  refine ⟨fun x => ?_, fun x => ?_⟩
  · have hjkle : (j : D → D) x ⊔ k x ≤ x := sup_le (hjle x) (hkle x)
    have hj_eq : (j : D → D) (j x ⊔ k x) = j x :=
      le_antisymm (j.monotone hjkle) (le_of_eq_of_le (hjidem x).symm (j.monotone
    le_sup_left))
    have hk_eq : (k : D → D) (j x ⊔ k x) = k x :=
      le_antisymm (k.monotone hjkle) (le_of_eq_of_le (hkidem x).symm (k.monotone
    le_sup_right))
    rw [ScottMap.sup_apply, ScottMap.sup_apply, hj_eq, hk_eq]
  · rw [ScottMap.sup_apply]; exact sup_le (hjle x) (hkle x)

end Projections

end DomainTheory.ContinuousLattice
```

2

3 Neighborhood Systems (1980)

Dependency diagram. Each box is one section. The arrowhead is on the section that imports, and the number on it is how many import lines run that way. An arrow already implied by a longer path is left out. The introduction has no Lean file, so it has no arrow.

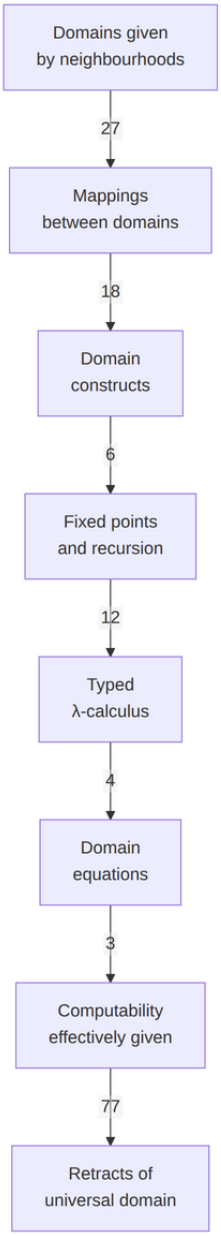


Figure 27: Neighborhood Systems (1980).

3.1 Introduction

These notes were written in conjunction with the lectures delivered by me for the Semantics of Programming Languages sequence during Michaelmas Term 1980 at Oxford. I started writing around the first week of October and finished at the end of November. The purpose of the course was to provide the foundations needed for the method of denotational semantics; in particular, I wanted to make the connections with recursive function theory more definite and to show explicit, effectively given solutions to domain equations. Roughly, these chapters cover the first half of the book of J.E. Stoy. I plan soon to expand the notes into a book by adding additional chapters on other theoretical topics that time did not permit me to cover in one eight-week term.

When I started writing Lecture I in October, I did not know what the later lectures would contain: I could see no further ahead than part of Lecture III in the beginning. The lectures had to be typed in advance of the class meetings, however, so there was at the time of composition no opportunity for second thoughts of any major proportions: I had to write the text straight through. As a consequence there are many remarks I would like to transpose and many additional points of explanation I see are needed; further worked examples and easier exercises are also required. During the spring, after receiving many helpful comments, I was able to introduce a few changes in the text and make some necessary corrections. However, a complete retyping was impossible. Nevertheless, this preliminary version of the book seems to provide a quite detailed introduction and is sufficient to exhibit the scope of the approach and several applications.

The idea of using neighbourhood systems to give set-theoretical representations of domains had been in the back of

my mind for some time in connection with specific examples. But the thought that a systematic development along these lines might be easier to follow than the more abstract lattice-theoretic and topological approach used by myself and others in many publications only came to me during the IFIP Working Group 2.2 meeting in Copenhagen in mid-June 1980. I gave a brief public presentation at ICALP '80 in Holland in mid-July.

One large mistake I have made is to de-emphasize partial orderings too much, since at the right point the concepts and the language are in fact helpful. The basic plan is that, instead of axiomatizing the theory using partial orderings, the necessary facts come out as *theorems*. For a neighbourhood system $\mathcal{D}$, the set of elements $|\mathcal{D}|$, which consists of filters, is naturally partially ordered. And approximable mappings naturally *preserve* the ordering. And so on. The advantage I see from the point of view of exposition is that properties can be brought out one at a time instead of having to put them down all in advance of any experience with the ideas. My own feeling after writing these chapters is that the plan has worked out far better than I could have dared to hope. I was especially glad that I could generate so many *exercises*, and I hope eventually to provide many more. One interesting place at which partial orderings prove their usefulness is in visualizing domains. As it stands now the text does not contain enough in the way of pictures. This will have to be remedied in a future version. Undoubtedly to include enough explanation, several of the lectures will have to be sub-divided into separate chapters.

One major improvement is needed: to bring Exercise 2.22 into the main text. I did not know in advance how often I would need this result for giving (easy) set-theoretical characterizations of domains and structure on them. This will be an easy repair, but it will cause quite a bit of rewriting. Clearly

much more has to be said about the interplay between elements and neighbourhoods. In particular, the character of the elements of a domain, like the power set of a set, has not been sufficiently illustrated, and quite a bit of expansion on this topic is also needed.

Finally I have to explain that I had no time whatsoever to put in references and a bibliography. The ideas I have used have occurred to many, many people who have worked on domains, and I do not wish to claim

originality. I *am* claiming some advantage to my style of representation, but I fully realize that a published version will have to have detailed historical references and notes at the ends of the lectures. Needless to say I should very much appreciate any advice or criticism from readers of this preliminary version.

I would like to give a warm word of thanks to the many people who have already commented on the preliminary text both at Oxford and in Boston, where I gave lectures. Very special thanks are due to Steve Comer and Steve Brookes, who spent many hours proof reading the typescripts. The biggest word of thanks, however, is reserved for Elsie Hinkes who, under very considerable pressure, did a wonderful job of typing.

Dana S. Scott
Merton College
Oxford
May 1981

3.2 Domains given by neighbourhoods

Dependency diagram. Each box is one item. The arrowhead is on the item that imports. An arrow already implied by a longer path is left out. The introduction has no Lean file, so it is not shown. Blue is a definition, green a factoid, amber an example, red a theorem, and indigo an exercise.



Figure 28: Domains given by neighbourhoods.

3.2.1 Introduction

LECTURE I DOMAINS GIVEN BY NEIGHBOURHOODS

Often an object (or element) can be determined by a selection of its properties. Often it is also the case that it is easier (more convenient, more elementary) to think of these properties than it is to think of the elements themselves. Let us term the properties under consideration *neighbourhoods*, the family of those allowed a *neighbourhood system*.

Generally, the collection of these neighbourhoods is, for one reason or another, somewhat restricted; that is, a completely arbitrary property may not be allowable as a neighbourhood. Therefore, the elements determined by selections of neighbourhoods may not be as separable into the discrete objects common to the classical view of set theory. This is particularly true in working with infinite objects: it is hard to specify an infinite element completely. The theory of elements to be studied here, then, is going to permit *partial elements* as well as *total elements*, and each neighbourhood system will define a *domain* of such elements.

Since we may wish to use a neighbourhood system to introduce elements not previously investigated, the

neighbourhoods do not have to be regarded as sets of the as-yet-to-be-defined objects. We can take a non-empty set Δ of *tokens* (or "traces") that function as "parts" of elements — or even as parts of "descriptions" of elements. Then a neighbourhood is a subset $X \subseteq \Delta$ containing all those tokens that provide sufficient information when taken together to "approximate" a possible element up to a certain "degree". All these words in inverted commas are vague, and in any case we shall have at the start only a *qualitative* theory of "degree of approximation". A token should be considered as a very "rough" representative of an element, and a neighbourhood should be regarded as "smoothing out" irrelevant details by grouping together *all* those representatives sharing some common feature. One neighbourhood, then,

may be only a very incomplete specification of an (ideal) element; fuller specifications can be secured by taking "convergent" sequences of neighbourhoods. Even then convergence need not be to a total element.

Let us call the family of allowed neighbourhoods $\mathcal{D}$; it is a family of subsets of the set Δ. An obvious first question is: when are two neighbourhoods $X, Y \in \mathcal{D}$ neighbourhoods of the "same" element? This question of course generalizes to a (finite) sequence of neighbourhoods. This property we will call the *consistency* of the sequence of neighbourhoods. By definition this will mean that the given neighbourhoods all contain a common neighbourhood in $\mathcal{D}$. That is, for X, Y to be consistent, there must be a $Z \in \mathcal{D}$ with $Z \subseteq X$ and $Z \subseteq Y$. This is not a very informative definition, but it has something of the flavour of a notion of consistency insofar as it can be expressed within $\mathcal{D}$. When consistency holds it seems reasonable enough at first glance to say that the intersection $X \cap Y$ is also an approximation to this common element. If this is reasonable, then $X \cap Y$ should also be regarded as a neighbourhood. This assumption has many consequences, but as a preliminary theory of approximation we will find it quite workable with many natural instances. Taking intersections just means taking more and more properties of the element and putting them together "conjunctively". It is something we do all the time. We therefore accept the idea for the present for giving our first principal definition.

3.2.2 Definition 1.1 — neighbourhood system

DEFINITION 1.1. A family $\mathcal{D}$ of subsets of a given set Δ is called a *neighbourhood system* (over Δ) iff it is a non-empty family closed under the intersection of finite consistent sequences of neighbourhoods. That is to say, $\mathcal{D}$ must fulfill these two conditions:

(i) $\Delta \in \mathcal{D}$;

(ii) whenever $X, Y, Z \in \mathcal{D}$ and $Z \subseteq X \cap Y$, then $X \cap Y \in \mathcal{D}$. □

We remark that by convention Δ corresponds to the intersection of an empty sequence of neighbourhoods; in particular, see Factoid 1.1.

isProjection_sup.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.IsProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.sup_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_iff
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic
```

```
@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

/-- `J_D` is closed under binary joins (Scott 1972, 3.12). The key step: since `j x ⊔ k
     x ⊑ x`,
monotonicity and idempotency force `j (j x ⊔ k x) = j x` and `k (j x ⊔ k x) = k x`. -/
theorem isProjection_sup {j k : ScottMap D D} (hj : IsProjection j) (hk : IsProjection
     k) :
    IsProjection (j ⊔ k) := by
  rw [isProjection_iff] at hj hk ⊢
  obtain ⟨hjidem, hjle⟩ := hj
  obtain ⟨hkidem, hkle⟩ := hk
  refine ⟨fun x => ?_, fun x => ?_⟩
  · have hjkle : (j : D → D) x ⊔ k x ≤ x := sup_le (hjle x) (hkle x)
    have hj_eq : (j : D → D) (j x ⊔ k x) = j x :=
      le_antisymm (j.monotone hjkle) (le_of_eq_of_le (hjidem x).symm (j.monotone
    le_sup_left))
    have hk_eq : (k : D → D) (j x ⊔ k x) = k x :=
      le_antisymm (k.monotone hjkle) (le_of_eq_of_le (hkidem x).symm (k.monotone
    le_sup_right))
    rw [ScottMap.sup_apply, ScottMap.sup_apply, hj_eq, hk_eq]
  · rw [ScottMap.sup_apply]; exact sup_le (hjle x) (hkle x)

end Projections

end DomainTheory.ContinuousLattice
```

3

3.2.3 Factoid 1.1 — empty intersection is delta; recursive finite intersection

Factoid 1.1a — empty intersection is delta

$$\bigcap_{i<n} X_i = \Delta, \text{ if } n = 0;$$

Factoid 1.1b — recursive finite intersection

$$= \left(\bigcap_{i<n-1} X_i \right) \cap X_{n-1}, \text{ if } n > 0.$$

Of course, from (ii), we can extend the intersection property to any finite sequence. Consequently, we can say $X_0, \ldots, X_{n-1}$ is consistent in $\mathcal{D}$ iff

$$\bigcap_{i<n} X_i \in \mathcal{D}.$$

Some examples will help us understand the notions better.

isProjection_sup.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.IsProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.sup_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_iff
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

/-- `J_D` is closed under binary joins (Scott 1972, 3.12). The key step: since `j x ⊔ k
     x ⊑ x`,
monotonicity and idempotency force `j (j x ⊔ k x) = j x` and `k (j x ⊔ k x) = k x`. -/
theorem isProjection_sup {j k : ScottMap D D} (hj : IsProjection j) (hk : IsProjection
     k) :
    IsProjection (j ⊔ k) := by
  rw [isProjection_iff] at hj hk ⊢
  obtain ⟨hjidem, hjle⟩ := hj
  obtain ⟨hkidem, hkle⟩ := hk
  refine ⟨fun x => ?_, fun x => ?_⟩
  · have hjkle : (j : D → D) x ⊔ k x ≤ x := sup_le (hjle x) (hkle x)
    have hj_eq : (j : D → D) (j x ⊔ k x) = j x :=
      le_antisymm (j.monotone hjkle) (le_of_eq_of_le (hjidem x).symm (j.monotone
    le_sup_left))
    have hk_eq : (k : D → D) (j x ⊔ k x) = k x :=
      le_antisymm (k.monotone hjkle) (le_of_eq_of_le (hkidem x).symm (k.monotone
    le_sup_right))
    rw [ScottMap.sup_apply, ScottMap.sup_apply, hj_eq, hk_eq]
  · rw [ScottMap.sup_apply]; exact sup_le (hjle x) (hkle x)
```

```
end Projections

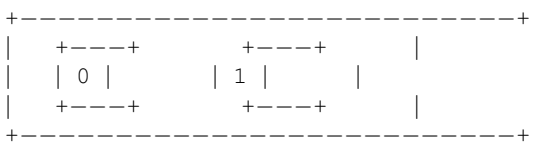
end DomainTheory.ContinuousLattice
```

4

3.2.4 Example 1.2 — two-token domain

EXAMPLE 1.2. Let $\Delta = \{0, 1\}$ and let $\mathcal{D} = \{\{0, 1\}, \{0\}, \{1\}\}$.

In pictures we have:



The intention is that 0 and 1 can be completely specified and that they can be identified with the total elements. As we shall see, there is only one partial element: either we give no information (the neighbourhood $\{0, 1\}$), or we decide between 0 and 1 (by giving $\{0\}$ or $\{1\}$). □

isProjection_sup.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.IsProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.sup_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_iff
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

/-- `J_D` is closed under binary joins (Scott 1972, 3.12). The key step: since `j x ⊔ k
     x ⊑ x`,
monotonicity and idempotency force `j (j x ⊔ k x) = j x` and `k (j x ⊔ k x) = k x`. -/
theorem isProjection_sup {j k : ScottMap D D} (hj : IsProjection j) (hk : IsProjection
     k) :
```

```
    IsProjection (j ⊔ k) := by
  rw [isProjection_iff] at hj hk ⊢
  obtain ⟨hjidem, hjle⟩ := hj
  obtain ⟨hkidem, hkle⟩ := hk
  refine ⟨fun x => ?_, fun x => ?_⟩
  · have hjkle : (j : D → D) x ⊔ k x ≤ x := sup_le (hjle x) (hkle x)
    have hj_eq : (j : D → D) (j x ⊔ k x) = j x :=
      le_antisymm (j.monotone hjkle) (le_of_eq_of_le (hjidem x).symm (j.monotone
    le_sup_left))
    have hk_eq : (k : D → D) (j x ⊔ k x) = k x :=
      le_antisymm (k.monotone hjkle) (le_of_eq_of_le (hkidem x).symm (k.monotone
    le_sup_right))
    rw [ScottMap.sup_apply, ScottMap.sup_apply, hj_eq, hk_eq]
  · rw [ScottMap.sup_apply]; exact sup_le (hjle x) (hkle x)

end Projections

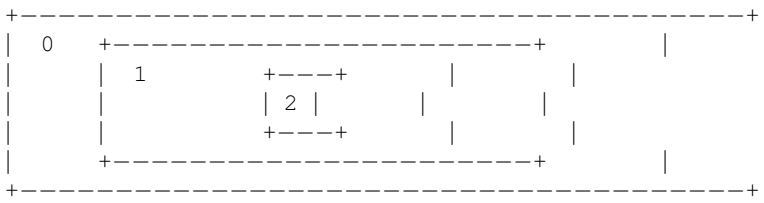
end DomainTheory.ContinuousLattice
```

5

3.2.5 Example 1.3 — three-step chain

EXAMPLE 1.3. Let $\Delta = \{0, 1, 2\}$ and let $\mathcal{D} = \{\{0, 1, 2\}, \{1, 2\}, \{2\}\}$.

In pictures we have:



Instead of stepping to the total element (here represented by 2) in one big step, the passage is divided into two steps. (Note 0 and 1 cannot be taken as representing total elements.) This example is not very interesting because the direction of approximation is unique. We need an example with some choice. □

isProjection_sup.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.IsProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.sup_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_iff
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace
```

```
variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

/-- `J_D` is closed under binary joins (Scott 1972, 3.12). The key step: since `j x ⊔ k
     x ⊑ x`,
monotonicity and idempotency force `j (j x ⊔ k x) = j x` and `k (j x ⊔ k x) = k x`. -/
theorem isProjection_sup {j k : ScottMap D D} (hj : IsProjection j) (hk : IsProjection
     k) :
    IsProjection (j ⊔ k) := by
  rw [isProjection_iff] at hj hk ⊢
  obtain ⟨hjidem, hjle⟩ := hj
  obtain ⟨hkidem, hkle⟩ := hk
  refine ⟨fun x => ?_, fun x => ?_⟩
  · have hjkle : (j : D → D) x ⊔ k x ≤ x := sup_le (hjle x) (hkle x)
    have hj_eq : (j : D → D) (j x ⊔ k x) = j x :=
      le_antisymm (j.monotone hjkle) (le_of_eq_of_le (hjidem x).symm (j.monotone
    le_sup_left))
    have hk_eq : (k : D → D) (j x ⊔ k x) = k x :=
      le_antisymm (k.monotone hjkle) (le_of_eq_of_le (hkidem x).symm (k.monotone
    le_sup_right))
    rw [ScottMap.sup_apply, ScottMap.sup_apply, hj_eq, hk_eq]
  · rw [ScottMap.sup_apply]; exact sup_le (hjle x) (hkle x)

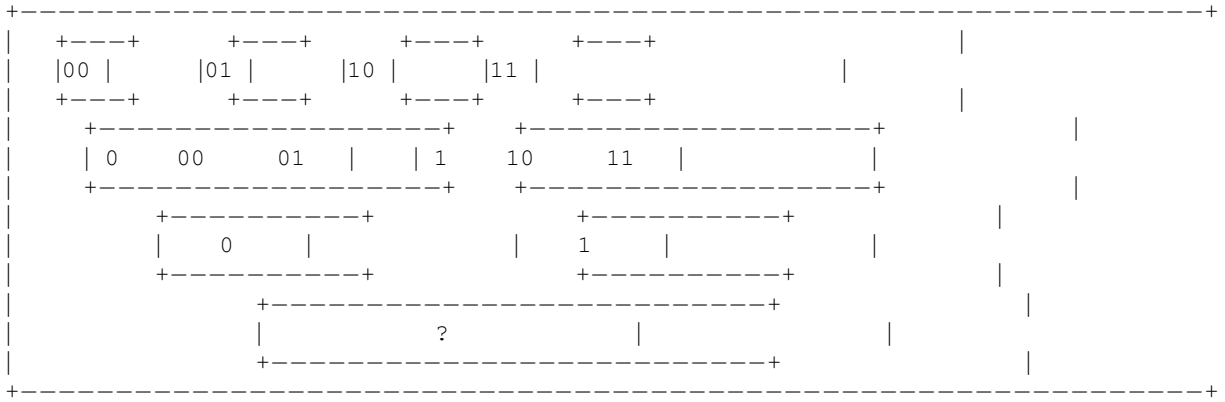
end Projections

end DomainTheory.ContinuousLattice
```


3.2.6 Example 1.4 — finite binary tree

EXAMPLE 1.4. Let $\Delta = \{\Lambda, 0, 1, 00, 01, 10, 11\}$ and let $\mathcal{D} = \{\Delta, \{0, 00, 01\}, \{1, 10, 11\}, \{00\}, \{01\}, \{10\}, \{11\}\}$.

Or more understandably in pictures:



The tokens are finite sequences of 0's and 1's (up to length 2) with Λ the empty sequence; they form — in the picture — the binary tree with the sequences as the nodes. The neighbourhoods are the subtrees of all nodes above a given node. Obviously this can be generalized to sequences of any length (and to trees less regular than the binary tree). The total elements of the example correspond to the top nodes 00, 01, 10, 11 and the lower nodes to the partial elements. When we are not at a top node we have only partially determined a sequence, and the branching indicates that we have some choice as to how the sequence can be extended. □

It should be noted that, in these three examples, the reason that we have a neighbourhood system is a simple consequence of a

Resolutions from the image: "is unique" (not "in unique"); $\mathcal{D}$ begins with Δ, not Λ; Pass 1's nested-box diagram matches the neighborhood boundaries in the scan; em dashes around "in the picture" as in the source.

isProjection_sup.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.IsProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.sup_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_iff
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

/-- `J_D` is closed under binary joins (Scott 1972, 3.12). The key step: since `j x ⊔ k
     x ⊑ x`,
monotonicity and idempotency force `j (j x ⊔ k x) = j x` and `k (j x ⊔ k x) = k x`. -/
theorem isProjection_sup {j k : ScottMap D D} (hj : IsProjection j) (hk : IsProjection
     k) :
    IsProjection (j ⊔ k) := by
  rw [isProjection_iff] at hj hk ⊢
  obtain ⟨hjidem, hjle⟩ := hj
  obtain ⟨hkidem, hkle⟩ := hk
  refine ⟨fun x => ?_, fun x => ?_⟩
  · have hjkle : (j : D → D) x ⊔ k x ≤ x := sup_le (hjle x) (hkle x)
    have hj_eq : (j : D → D) (j x ⊔ k x) = j x :=
```

```
      le_antisymm (j.monotone hjkle) (le_of_eq_of_le (hjidem x).symm (j.monotone
    le_sup_left))
    have hk_eq : (k : D → D) (j x ⊔ k x) = k x :=
      le_antisymm (k.monotone hjkle) (le_of_eq_of_le (hkidem x).symm (k.monotone
    le_sup_right))
    rw [ScottMap.sup_apply, ScottMap.sup_apply, hj_eq, hk_eq]
  · rw [ScottMap.sup_apply]; exact sup_le (hjle x) (hkle x)

end Projections

end DomainTheory.ContinuousLattice
```

7

3.2.7 Factoid 1.4a — nested or disjoint

very special circumstance: in these systems two neighbourhoods are either disjoint or one is included in the other. This arrangement of neighbourhoods is by no means necessary.

isProjection_sup.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.IsProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.sup_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_iff
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

/-- `J_D` is closed under binary joins (Scott 1972, 3.12). The key step: since `j x ⊔ k
     x ⊑ x`,
monotonicity and idempotency force `j (j x ⊔ k x) = j x` and `k (j x ⊔ k x) = k x`. -/
theorem isProjection_sup {j k : ScottMap D D} (hj : IsProjection j) (hk : IsProjection
     k) :
    IsProjection (j ⊔ k) := by
  rw [isProjection_iff] at hj hk ⊢
```

```
  obtain ⟨hjidem, hjle⟩ := hj
  obtain ⟨hkidem, hkle⟩ := hk
  refine ⟨fun x => ?_, fun x => ?_⟩
  · have hjkle : (j : D → D) x ⊔ k x ≤ x := sup_le (hjle x) (hkle x)
    have hj_eq : (j : D → D) (j x ⊔ k x) = j x :=
      le_antisymm (j.monotone hjkle) (le_of_eq_of_le (hjidem x).symm (j.monotone
    le_sup_left))
    have hk_eq : (k : D → D) (j x ⊔ k x) = k x :=
      le_antisymm (k.monotone hjkle) (le_of_eq_of_le (hkidem x).symm (k.monotone
    le_sup_right))
    rw [ScottMap.sup_apply, ScottMap.sup_apply, hj_eq, hk_eq]
  · rw [ScottMap.sup_apply]; exact sup_le (hjle x) (hkle x)

end Projections

end DomainTheory.ContinuousLattice
```

8

3.2.8 Example 1.5 — all nonempty subsets

EXAMPLE 1.5. Let $\Delta = \{0, 1, 2, 3\}$ and let $\mathcal{D}$ be the family of all non-empty subsets of Δ.

isProjection_sup.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.IsProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.sup_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_iff
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

/-- `J_D` is closed under binary joins (Scott 1972, 3.12). The key step: since `j x ⊔ k
     x ⊑ x`,
monotonicity and idempotency force `j (j x ⊔ k x) = j x` and `k (j x ⊔ k x) = k x`. -/
```

```
theorem isProjection_sup {j k : ScottMap D D} (hj : IsProjection j) (hk : IsProjection
     k) :
    IsProjection (j ⊔ k) := by
  rw [isProjection_iff] at hj hk ⊢
  obtain ⟨hjidem, hjle⟩ := hj
  obtain ⟨hkidem, hkle⟩ := hk
  refine ⟨fun x => ?_, fun x => ?_⟩
  · have hjkle : (j : D → D) x ⊔ k x ≤ x := sup_le (hjle x) (hkle x)
    have hj_eq : (j : D → D) (j x ⊔ k x) = j x :=
      le_antisymm (j.monotone hjkle) (le_of_eq_of_le (hjidem x).symm (j.monotone
    le_sup_left))
    have hk_eq : (k : D → D) (j x ⊔ k x) = k x :=
      le_antisymm (k.monotone hjkle) (le_of_eq_of_le (hkidem x).symm (k.monotone
    le_sup_right))
    rw [ScottMap.sup_apply, ScottMap.sup_apply, hj_eq, hk_eq]
  · rw [ScottMap.sup_apply]; exact sup_le (hjle x) (hkle x)

end Projections

end DomainTheory.ContinuousLattice
```

9

3.2.9 Factoid 1.5 — consistent iff nonempty; equivalent sequences same limit

Factoid 1.5a — consistent iff nonempty This system is a direct generalization of Example 1.2., which was special owing to the small number of tokens. (The other examples were special by virtue of the choice of neighbourhoods.) The verification that the present $\mathcal{D}$ is a neighbourhood system rests on nothing more than the remark that sets are consistent in $\mathcal{D}$ iff they have a non-empty intersection. Clearly the arrangement of neighbourhoods in $\mathcal{D}$ can be as varied as a four-element set will allow; if Δ were made larger, the possible combinations of neighbourhoods could be made as complex as you wish. □

Having some idea now of the variety of neighbourhood systems, we have to discuss what it is they do. As stressed before, the tokens do not have to correspond directly to the elements; but where, we ask, do the elements come from? One obvious suggestion for determining an element is to produce a sequence of "better and better" neighbourhoods:

$$X_0 \supseteq X_1 \supseteq \cdots \supseteq X_n \supseteq \cdots$$

Trivially, any finite initial segment of this sequence is consistent, and so each X_n is a partial approximation to the "limit". If $\mathcal{D}$ were always to be taken as *finite*, of course, there would be no point in discussing limits since any such sequence would eventually be constant. The elements in the finite case would therefore be completely represented by neighbourhoods with the *minimal* neighbourhoods corresponding to the total elements. But there are many reasons to go beyond the finite (though perhaps not too far beyond).

Suppose $\langle Y_n \rangle_{n=0}^{\infty}$ is another "convergent" sequence with

$Y_{n+1} \subseteq Y_n$ for all indices: when do the two sequences of neighbourhoods determine the *same limit*? The two sequences can surely be different; for example, $\langle Y_n \rangle_{n=0}^{\infty}$ could be a subsequence of $\langle X_n \rangle_{n=0}^{\infty}$, say, $Y_n = X_{2n}$. Still we would want to say that the same limit is obtained. Without being given any further structure on the neighbourhoods, a simple answer is just to say that each sequence goes *equally deep* as the other:

for each m there is an n with $X_n \subseteq Y_m$, and

for each n there is an m with $Y_m \subseteq X_n$.

This definition obviously puts sequences into equivalence classes, and so elements could be identified with these. But such a definition is clumsy for two reasons: it is always tiresome to work with equivalence classes, and there is no reason to think that simple infinite sequences are adequate for determining elements without some rather drastic assumptions on $\mathcal{D}$. Nevertheless, the idea is *suggestive*; we just have to find some construct to represent elements in a unique way and to phrase it in a general enough manner.

Start with $\langle X_n \rangle_{n=0}^{\infty}$ again, which "converges" as before. Think of all the other sequences equivalent to this one in the sense just defined. We can define the class of all terms of all such sequences very easily as being the family:

$$x = \{Z \in \mathcal{D} \mid X_n \subseteq Z \text{ for some } n\}.$$

Factoid 1.5b — equivalent sequences same limit It is easy to prove that if we form the analogous class for $\langle Y_n \rangle_{n=0}^{\infty}$, then the two families are *equal* if and only if the sequences are *equivalent*. Thus, we seem justified in letting x represent the limit of $\langle X_n \rangle_{n=0}^{\infty}$. All we have to do now is to remark on what sort of class x is as a subfamily of $\mathcal{D}$; what we abstract from the construction, however, will be just a bit more general than taking those x that result from sequences.

le.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

def le {D : Type u} {D' : Type v} [CompleteLattice D] [CompleteLattice D']
    (f g : ScottMap D D') : Prop :=
  letI : LE D' :=
    (ChainCompletePartialOrder.instOfCompleteLattice
      (α := D')).toPartialOrder.toPreorder.toLE
  ∀ x, f x ≤ g x
```

```
end ScottMap

end DomainTheory.ContinuousLattice
```

0

3.2.10 Definition 1.6 — ideal elements

DEFINITION 1.6. The (ideal) elements of a neighbourhood system $\mathcal{D}$ are those subfamilies $x \subseteq \mathcal{D}$ where:

(i) $\Delta \in x$;

(ii) $X, Y \in x$ always implies $X \cap Y \in x$; and

(iii) whenever $X \in x$ and $X \subseteq Y \in \mathcal{D}$, then $Y \in x$.

The *domain* of all such elements is written as $|\mathcal{D}|$. □

The idea of 1.6 is a well-known mathematical device: the families x satisfying (i)–(iii) are usually called *filters*. Most frequently the emphasis is put on the *maximal* filters, and these would be our *total* elements; however, in general, the proof that maximal filters exist is non-constructive, so for our purposes it is better not to neglect the partial filters. When maximal filters can be found, well and good, but we do not have to insist on them. Note that the generality of 1.6 is achieved by not requiring that there is a sequence of neighbourhoods that "generates" the filter x. (See Exercise 1.22.)

We have often said that neighbourhoods determine partial elements by themselves; we now make this remark precise.

le.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']
```

```
namespace ScottMap

def le {D : Type u} {D' : Type v} [CompleteLattice D] [CompleteLattice D']
    (f g : ScottMap D D') : Prop :=
  letI : LE D' :=
    (ChainCompletePartialOrder.instOfCompleteLattice
      (α := D')).toPartialOrder.toPreorder.toLE
  ∀ x, f x ≤ g x

end ScottMap

end DomainTheory.ContinuousLattice
```

1

3.2.11 Definition 1.7 — principal filter

DEFINITION 1.7. For $X \in \mathcal{D}$, the *principal filter* determined by X is defined by:

$$\uparrow X = \{Y \in \mathcal{D} \mid X \subseteq Y\}.$$

The principal filters form what we shall call the *finite elements* of the domain $|\mathcal{D}|$. □

le.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

def le {D : Type u} {D' : Type v} [CompleteLattice D] [CompleteLattice D']
    (f g : ScottMap D D') : Prop :=
  letI : LE D' :=
    (ChainCompletePartialOrder.instOfCompleteLattice
```

```
      (α := D')).toPartialOrder.toPreorder.toLE
  ∀ x, f x ≤ g x

end ScottMap

end DomainTheory.ContinuousLattice
```

2

3.2.12 Factoid 1.7 — principals are one-one; inclusion-reversing principals; union of principal filters

Factoid 1.7a1 — principals are one-one It is obvious that the correspondence between X and $\uparrow X$ is one-one and inclusion *reversing*, in the sense that

Factoid 1.7a2 — inclusion-reversing principals

$$X \subseteq Y \quad \text{iff} \quad \uparrow Y \subseteq \uparrow X$$

for all $X, Y \in \mathcal{D}$. But, except in very special cases, there is much more to $|\mathcal{D}|$ than just the finite elements. Much of our investigation will be concerned with finding out how much more. The finite elements are, in a certain sense, "dense" in $|\mathcal{D}|$, however, because it is also obvious from the definitions that for each $x \in |\mathcal{D}|$

Factoid 1.7b — union of principal filters

$$x = \bigcup \{\uparrow X \mid X \in x\}.$$

That is, every element is a certain type of "limit" of finite elements. (This statement is made more precise in Exercise 1.21)

We note that we have now had several occasions to use inclusion relationships between elements; this is an important relationship, and we give it a special name.

le.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]
```

```
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

def le {D : Type u} {D' : Type v} [CompleteLattice D] [CompleteLattice D']
    (f g : ScottMap D D') : Prop :=
  letI : LE D' :=
    (ChainCompletePartialOrder.instOfCompleteLattice
      (α := D')).toPartialOrder.toPreorder.toLE
  ∀ x, f x ≤ g x

end ScottMap

end DomainTheory.ContinuousLattice
```

3

3.2.13 Definition 1.8 — approximation and totality

DEFINITION 1.8. For $x, y \in |\mathcal{D}|$, we say that x *approximates* y iff $x \subseteq y$. The element that approximates all others, $\{\Delta\}$, is called $\perp$ (read: *bottom*); it is the "least defined" element, or the "most partial" element. Elements maximal with respect to the approximation relation are called *total elements*. □

le.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap
```

```
def le {D : Type u} {D' : Type v} [CompleteLattice D] [CompleteLattice D']
    (f g : ScottMap D D') : Prop :=
  letI : LE D' :=
    (ChainCompletePartialOrder.instOfCompleteLattice
      (α := D')).toPartialOrder.toPreorder.toLE
  ∀ x, f x ≤ g x

end ScottMap

end DomainTheory.ContinuousLattice
```

4

3.2.14 Factoid 1.8 — bottom element; bottom is least; finite filters are principal; prefix order; union of finite approximations

Factoid 1.8a1 — bottom element DEFINITION 1.8. For $x, y \in |\mathcal{D}|$, we say that x *approximates* y iff $x \subseteq y$. The element that approximates all others, $\{\Delta\}$, is called $\perp$ (read: *bottom*); it is the "least defined" element, or the "most partial" element. Elements maximal with respect to the approximation relation are called *total elements*. □

Factoid 1.8a2 — bottom is least DEFINITION 1.8. For $x, y \in |\mathcal{D}|$, we say that x *approximates* y iff $x \subseteq y$. The element that approximates all others, $\{\Delta\}$, is called $\perp$ (read: *bottom*); it is the "least defined" element, or the "most partial" element. Elements maximal with respect to the approximation relation are called *total elements*. □

Factoid 1.8b — finite filters are principal EXAMPLES 1.2 – 1.5 (Revisited). The examples as given were all finite, so any explicitly given filter x is principal, the element is finite, the minimal $X \in x$ tells us all we need to know. In such simple situations there is essentially no difference between elements and neighbourhoods — except for the reversal of the order as noted. This (necessary) reversal should not, however, become a matter of confusion: the smaller the neighbourhood has become, the more it has "converged", and so the better defined the element has become. In the approximation relation the "poorer" elements are placed below the "better" with the total up at the top. This will become clearer in discussing "infinite" elements.

Example 1.3 will be generalized in Exercise 1.12. Let us here generalize first 1.4. We let

$$\Delta = \Sigma^*,$$

where $\Sigma = \{0, 1\}$ and Σ^* means the set of all finite sequences of 0's and 1's, with Λ being the empty sequence. We write $\sigma\tau$ for the *concatenation* or *juxtaposition* of two sequences $\sigma, \tau \in \Sigma^*$. Define

$$B = \{\sigma\Sigma^* \mid \sigma \in \Sigma^*\}, \text{ where}$$

$$\sigma X = \{\sigma\tau \mid \tau \in X\},$$

for an arbitrary set $X \subseteq \Sigma^*$. In other words, a neighbourhood in B consists of all *extensions* of a given sequence σ. (Refer back to the finite version of 1.4.) We use the letter "B" to remind us of "binary", and this is an example we shall refer to many times. The proof (if it is not obvious) that B is a neighbourhood system should be done as an exercise.

What do we find in $|B|$? Of course $\perp = \{\Delta\} \in |B|$. For any $x \in |B|$ and $\sigma \in \Sigma^*$ define

$$\sigma x = \{Y \mid \sigma X \subseteq Y \text{ some } X \in x\}.$$

Factoid 1.8c — prefix order Again there is an exercise here to show $\sigma x \in |B|$. In particular $\sigma \perp \in |B|$ for all $\sigma \in \Sigma^*$, and these are just the finite elements. The minimal element of $\sigma \perp$ is $\sigma\Delta$. Note that $\sigma_0 \perp \subseteq \sigma_1 \perp$ if and only if σ_0 is an *initial segment* of the sequence σ_1.

If now $x \in |B|$ is any explicitly given element (that is, if we know for any $X \in B$ whether or not $X \in x$), we have but to work out from these definitions that

Factoid 1.8d — union of finite approximations

$$x = \bigcup_{n=0}^{\infty} \sigma_n \perp,$$

where the $\sigma_n \in \Sigma^*$ and each σ_n is an initial segment of the next σ_{n+1}. *In general, in any domain, an element is uniquely determined by its finite approximations, and we are just making this explicit in* $|B|$. When we have complete knowledge of x, then there are two cases: either the approximations $\sigma_n \perp$ become constant from some point on (where $n > n_0$), or not. In the first case x is finite and equal to $\sigma_{n_0} \perp$; in the second case x is infinite and the σ_n fill out an infinite (one-way) sequence.

The generalization of 1.5 to the infinite case where $\Delta = \mathbf{N} = \{0, 1, 2, 3, \ldots, n, \ldots\}$ can be made in more than one way: for instance either we use as neighbourhoods *all* non-empty subsets of Δ or just those omitting but a finite number of integers. And, as will become apparent, there are other choices giving domains of quite different characters. □

Many constructions (choices of $\mathcal{D}$) lead to the "same" domain; "sameness" is an important notion and it is to be defined in terms of "isomorphism", which in turn is to be defined in terms of approximation preserving correspondences.

le.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
```

```
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

def le {D : Type u} {D' : Type v} [CompleteLattice D] [CompleteLattice D']
    (f g : ScottMap D D') : Prop :=
  letI : LE D' :=
    (ChainCompletePartialOrder.instOfCompleteLattice
      (α := D')).toPartialOrder.toPreorder.toLE
  ∀ x, f x ≤ g x

end ScottMap

end DomainTheory.ContinuousLattice
```

5

3.2.15 Examples 1.2–1.5 (Revisited) — finite examples revisited

EXAMPLES 1.2 – 1.5 (Revisited). The examples as given were all finite, so any explicitly given filter x is principal, the element is finite, the minimal $X \in x$ tells us all we need to know. In such simple situations there is essentially no difference between elements and neighbourhoods — except for the reversal of the order as noted. This (necessary) reversal should not, however, become a matter of confusion: the smaller the neighbourhood has become, the more it has "converged", and so the better defined the element has become. In the approximation relation the "poorer" elements are placed below the "better" with the total up at the top. This will become clearer in discussing "infinite" elements.

Example 1.3 will be generalized in Exercise 1.12. Let us here generalize first 1.4. We let

$$\Delta = \Sigma^*,$$

where $\Sigma = \{0, 1\}$ and Σ^* means the set of all finite sequences of 0's and 1's, with Λ being the empty sequence. We write $\sigma\tau$ for the *concatenation* or *juxtaposition* of two sequences $\sigma, \tau \in \Sigma^*$. Define

$$B = \{\sigma\Sigma^* \mid \sigma \in \Sigma^*\}, \text{ where}$$

$$\sigma X = \{\sigma\tau \mid \tau \in X\},$$

for an arbitrary set $X \subseteq \Sigma^*$. In other words, a neighbourhood in B consists of all *extensions* of a given sequence σ. (Refer back to the finite version of 1.4.) We use the letter "B" to remind us of "binary", and this is an example we shall refer to many times. The proof (if it is not obvious) that B is a neighbourhood system should be done as an exercise.

What do we find in $|B|$? Of course $\perp = \{\Delta\} \in |B|$. For any $x \in |B|$ and $\sigma \in \Sigma^*$ define

$$\sigma x = \{Y \mid \sigma X \subseteq Y \text{ some } X \in x\}.$$

le.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

def le {D : Type u} {D' : Type v} [CompleteLattice D] [CompleteLattice D']
    (f g : ScottMap D D') : Prop :=
  letI : LE D' :=
    (ChainCompletePartialOrder.instOfCompleteLattice
      (α := D')).toPartialOrder.toPreorder.toLE
  ∀ x, f x ≤ g x

end ScottMap

end DomainTheory.ContinuousLattice
```

6

3.2.16 Definition 1.9 — isomorphic domains

DEFINITION 1.9. Two neighbourhood systems $\mathcal{D}_0$ and $\mathcal{D}_1$ determine *isomorphic domains* iff there is a one-one correspondence between $|\mathcal{D}_0|$ and $|\mathcal{D}_1|$ which preserves inclusion between the elements of the domains. In symbols we write $\mathcal{D}_0 \cong \mathcal{D}_1$. □

It is certain that the property of 1.9 is necessary, but it may not be so clear that it is sufficient. We shall in fact prove in the next lecture that an isomorphism between domains always maps finite elements to finite elements, so it always results from a one-one inclusion-preserving correspondence between neighbourhoods. This is surely as strong as could be hoped. This general result is not needed to see that particular domains are isomorphic.

In some of the examples tokens corresponded to total elements and in some to partial elements; it is not difficult to see (ex post facto) that every domain can be presented with tokens exactly corresponding to partial elements.

le.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

def le {D : Type u} {D' : Type v} [CompleteLattice D] [CompleteLattice D']
    (f g : ScottMap D D') : Prop :=
  letI : LE D' :=
    (ChainCompletePartialOrder.instOfCompleteLattice
      (α := D')).toPartialOrder.toPreorder.toLE
  ∀ x, f x ≤ g x

end ScottMap

end DomainTheory.ContinuousLattice
```

7

3.2.17 Theorem 1.10 — basic open neighbourhoods

THEOREM 1.10. Given any neighbourhood system $\mathcal{D}$, define for $X \in \mathcal{D}$

$$[X] = \{x \in |\mathcal{D}| \mid X \in x\}.$$

The subsets $[X] \subseteq |\mathcal{D}|$ for $X \in \mathcal{D}$ form a neighbourhood system over $|\mathcal{D}|$ which determines a domain isomorphic to $|\mathcal{D}|$.

Proof: We note first that

(1) $[\Delta] = |\mathcal{D}|$.

Next note that

(2) X, Y are consistent in $\mathcal{D}$ iff $[X] \cap [Y] \neq \emptyset$;

and that for $X, Y \in \mathcal{D}$

(3) $[X] \cap [Y] = [X \cap Y]$ if $X \cap Y \in \mathcal{D}$.

Inasmuch as

(4) $\uparrow X \in [X]$ for all $X \in \mathcal{D}$,

it easily follows that $\mathcal{D}$ and the family

$$\{[X] \mid X \in \mathcal{D}\}$$

are in a one-one, inclusion-preserving correspondence. Thus, we can induce the desired one-one correspondence between the elements of the two systems. □

The import of 1.10 is that the original tokens in Δ can be replaced by the elements of $|\mathcal{D}|$. This process replaces the neighbourhood $X \subseteq \Delta$ by the subset $[X] \subseteq |\mathcal{D}|$. As the passage is inclusion preserving, the domain has not really changed, only its presentation. Though of some theoretical charm, the theorem is not of much use since we still have to get $\mathcal{D}$ from somewhere. It does emphasize, though, that the rôle of the tokens is simply to keep the inclusions (and intersections) of neighbourhoods sorted out. It is *not* always true that the tokens can be identified with the total elements.

The last theorem in this lecture is a result on *closure properties* of a domain with respect to set-theoretical operations which have interesting meanings with respect to approximation.

le.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap
```

```
def le {D : Type u} {D' : Type v} [CompleteLattice D] [CompleteLattice D']
    (f g : ScottMap D D') : Prop :=
  letI : LE D' :=
    (ChainCompletePartialOrder.instOfCompleteLattice
      (α := D')).toPartialOrder.toPreorder.toLE
  ∀ x, f x ≤ g x

end ScottMap

end DomainTheory.ContinuousLattice
```

8

3.2.18 Theorem 1.11 — intersections and directed unions

THEOREM 1.11. If $\mathcal{D}$ is a neighbourhood system and $x_n \in |\mathcal{D}|$ for $n = 0, 1, 2, \ldots$, then

(i) $\bigcap_{n=0}^{\infty} x_n \in |\mathcal{D}|$; and

(ii) $\bigcup_{n=0}^{\infty} x_n \in |\mathcal{D}|$, provided

$$x_0 \subseteq x_1 \subseteq x_2 \subseteq \cdots \subseteq x_n \subseteq x_{n+1} \subseteq \cdots .$$

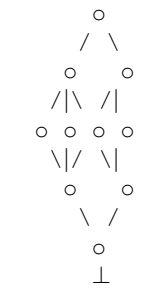
Proof: The conditions of 1.6 have to be checked. For the case of intersection, all of 1.6(i)–(iii) are quite obvious. For the case of union, only 1.6(ii) gives pause and it requires the proviso. If X and Y belong to the union, then $X \in x_n$, say, and $Y \in x_m$. But, either $n < m$ or $m < n$, and if $k = \max(n, m)$, then $X, Y \in x_k$. Since $x_k \in |\mathcal{D}|$, we have $X \cap Y \in x_k$; thus, $X \cap Y$ belongs to the union. This proves (ii). □

In words, the intersection is the best element that is at the same time an approximation to all of the elements x_n; the intersection is exactly what is common to all the given elements. The union on the other hand is just what the (increas-

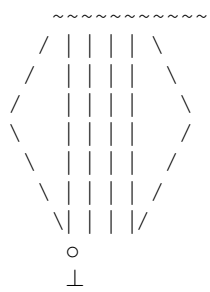
The union on the other hand is just what the (increasing sequence of the) x_n approximates; the union combines contributions from all the x_n into a "better" element – but no more than that.

In thinking about domains a rough diagram of the partial-ordering relation $\subseteq$ between elements is often helpful. The picture of 1.4 is an example where the nodes represent the elements. Any finite tree growing up from a root node would also be an example. Indeed, any finite partially ordered set with least element would be an example. (Here no distinction between tokens and elements is necessary.) A lattice diagram is also illustrated.

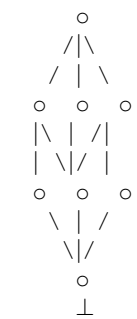
A TREE



A ROUGH PICTURE



A LATTICE



The root node is the element $\bot$ of $|\mathcal{D}|$; there need be no top node $\top$. *Approximation* is represented by a passage from a lower node to a higher node along the rising lines. The system $\mathcal{D}$ of neighbourhoods is the collection of sets each of which is all

the nodes above a given node. For infinite examples, however, care must be given to introduce limit nodes. The first few exercises should be provided with pictures to illustrate the structure.

le.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']
```

```
namespace ScottMap

def le {D : Type u} {D' : Type v} [CompleteLattice D] [CompleteLattice D']
    (f g : ScottMap D D') : Prop :=
  letI : LE D' :=
    (ChainCompletePartialOrder.instOfCompleteLattice
      (α := D')).toPartialOrder.toPreorder.toLE
  ∀ x, f x ≤ g x

end ScottMap

end DomainTheory.ContinuousLattice
```

9

3.2.19 Exercise 1.12 — final segments of integers

Let $\Delta = \mathbb{N} = \{0, 1, 2, \ldots, n, \ldots\}$ be the set of non-negative integers. Use as neighbourhoods final segments:

$$\{m \in \mathbb{N} \mid m > n\}$$

for $n \in \mathbb{N}$. Verify that this is a neighbourhood system. What are the total elements? What are the finite elements? Draw a picture of the approximation relation in this domain. (Hint: there is only one limit element.)

le_antisymm'.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.le
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

/-- Antisymmetry of the pointwise order. -/
theorem le_antisymm' {D : Type u} {D' : Type v}
```

```
    [CompleteLattice D] [CompleteLattice D'] (f g : ScottMap D D')
    (hfg : ScottMap.le f g)
    (hgf : ScottMap.le g f) : f = g :=
  ScottMap.ext fun x => le_antisymm (hfg x) (hgf x)

end ScottMap

end DomainTheory.ContinuousLattice
```

0

3.2.20 Exercise 1.13 — infinite binary tree

Verify all the assertions made about the system B defined as the infinite generalization of Example 1.4. Draw a picture similar to that given in the text which includes nodes for all $\sigma \in \Sigma^*$. Show the neighbourhoods, how the approximation relation behaves, and where the total elements lie. (The picture is closely related to the "binary tree", but has to have limit nodes all along the top.)

le_antisymm'.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.le
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

/-- Antisymmetry of the pointwise order. -/
theorem le_antisymm' {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (f g : ScottMap D D')
    (hfg : ScottMap.le f g)
    (hgf : ScottMap.le g f) : f = g :=
  ScottMap.ext fun x => le_antisymm (hfg x) (hgf x)

end ScottMap

end DomainTheory.ContinuousLattice
```

1

3.2.21 Exercise 1.14 — finite nonempty subsets

Let $\Delta = \mathbb{N}$ and let $\mathcal{V}$ be the family of finite non-empty subsets of Δ plus the set Δ. Show that this is a neighbourhood system. What are the total elements? What are the finite elements? Draw a picture.

le_antisymm'.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.le
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

/-- Antisymmetry of the pointwise order. -/
theorem le_antisymm' {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (f g : ScottMap D D')
    (hfg : ScottMap.le f g)
    (hgf : ScottMap.le g f) : f = g :=
  ScottMap.ext fun x => le_antisymm (hfg x) (hgf x)

end ScottMap

end DomainTheory.ContinuousLattice
```

2

3.2.22 Exercise 1.15 — finite elements without chains

Construct non-isomorphic infinite domains where all elements are finite but where there are no infinite chains $\langle x_n \rangle_{n=0}^{\infty}$ of elements with $x_n \sqsubseteq x_{n+1}$ but $x_n \neq x_{n+1}$ for all n.

le_antisymm'.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.le
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

/-- Antisymmetry of the pointwise order. -/
theorem le_antisymm' {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (f g : ScottMap D D')
    (hfg : ScottMap.le f g)
    (hgf : ScottMap.le g f) : f = g :=
  ScottMap.ext fun x => le_antisymm (hfg x) (hgf x)

end ScottMap

end DomainTheory.ContinuousLattice
```

3

3.2.23 Exercise 1.16 — cofinite integer subsets

Let $\Delta = \mathbf{N}$ and let $\mathcal{D}$ be the family of *cofinite* subsets of $\mathbf{N}$. Show that $|\mathcal{D}|$ is isomorphic to the partially ordered set of *all* subsets of $\mathbf{N}$ under inclusion. Construct some other neighbourhood systems where $\mathcal{D}$ is closed under finite intersection. What happens to the total elements in such systems?

`le_antisymm'.lean`

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.le
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice
```

```
open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

/-- Antisymmetry of the pointwise order. -/
theorem le_antisymm' {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (f g : ScottMap D D')
    (hfg : ScottMap.le f g)
    (hgf : ScottMap.le g f) : f = g :=
  ScottMap.ext fun x => le_antisymm (hfg x) (hgf x)

end ScottMap

end DomainTheory.ContinuousLattice
```

4

3.2.24 Exercise 1.17 — rational open intervals

Let $\Delta = \mathbf{R}$ be the real line. Let $\mathcal{D}$ be the set of non-empty open intervals with rational end points plus the set Δ. Show that this is a neighbourhood system. For any real $t \in \mathbf{R}$, show that $\{X \in \mathcal{D} \mid t \in X\}$ is a filter. Is it always total? What are the total elements of $|\mathcal{D}|$? (Hint: When t is rational consider all intervals with t as a right-hand end point.)

le_antisymm'.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.le
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]
```

```
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

/-- Antisymmetry of the pointwise order. -/
theorem le_antisymm' {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (f g : ScottMap D D')
    (hfg : ScottMap.le f g)
    (hgf : ScottMap.le g f) : f = g :=
  ScottMap.ext fun x => le_antisymm (hfg x) (hgf x)

end ScottMap

end DomainTheory.ContinuousLattice
```

5

3.2.25 Exercise 1.18 — consistency of neighbourhood families

Let $\mathcal{D}$ be a neighbourhood system. Call a subset $C \subseteq \mathcal{D}$ *consistent* iff every finite subset of C is consistent in $\mathcal{D}$. Give an example where C is a subset with more than two elements, every pair of neighbourhoods in C is consistent, but C is *not* consistent. Show that if C is consistent, then there is a *least* filter $x \in |\mathcal{D}|$ with $C \subseteq x$. Show generally that the *intersection* of any non-empty collection of filters is again a filter.

`le_antisymm'.lean`

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.le
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap
```

```
/-- Antisymmetry of the pointwise order. -/
theorem le_antisymm' {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (f g : ScottMap D D')
    (hfg : ScottMap.le f g)
    (hgf : ScottMap.le g f) : f = g :=
  ScottMap.ext fun x => le_antisymm (hfg x) (hgf x)

end ScottMap

end DomainTheory.ContinuousLattice
```

6

3.2.26 Exercise 1.19 — positive neighbourhood systems

Define a *positive neighbourhood system* to be a family $\mathcal{D}$ where (ii) of 1.1 is replaced by **(ii')** whenever $X, Y \in \mathcal{D}$, then $X \cap Y \neq \emptyset$ iff $X \cap Y \in \mathcal{D}$. Prove that a positive neighbourhood system is indeed a neighbourhood system in the sense of the earlier definition. Give an example of a neighbourhood system that is *not* positive. (Hint: (suggested by C.A.R. Hoare). Let $\Delta = \mathbf{N} \times \mathbf{N}$, in the plane. Let $\mathcal{D}$ be the family of subsets $X \subseteq \mathbf{N} \times \mathbf{N}$ where all but a finite number of places the *vertical* sections of X are the whole of $\mathbf{N}$ but at the other places the sections are finite and nonempty. Smaller examples are of course possible.)

`le_antisymm'.lean`

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.le
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

/-- Antisymmetry of the pointwise order. -/
theorem le_antisymm' {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (f g : ScottMap D D')
    (hfg : ScottMap.le f g)
    (hgf : ScottMap.le g f) : f = g :=
```

```
  ScottMap.ext fun x => le_antisymm (hfg x) (hgf x)

end ScottMap

end DomainTheory.ContinuousLattice
```

7

3.2.27 Exercise 1.20 — upward closed neighbourhoods

Let $\mathcal{D}$ be any neighbourhood system over a set Δ. Let $\Delta' = \mathcal{D}$ and define

$$\mathcal{D}' = \{\uparrow X \mid X \in \mathcal{D}\},$$

where

$$\uparrow X = \{Y \in \mathcal{D} \mid Y \subseteq X\}.$$

Show that $\mathcal{D}'$ is a positive neighbourhood system and that $|\mathcal{D}|$ and $|\mathcal{D}'|$ are isomorphic. Note that for $\mathcal{D}'$ finite elements and tokens are in a one-one correspondence.

le_antisymm'.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.le
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

/-- Antisymmetry of the pointwise order. -/
theorem le_antisymm' {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (f g : ScottMap D D')
    (hfg : ScottMap.le f g)
    (hgf : ScottMap.le g f) : f = g :=
  ScottMap.ext fun x => le_antisymm (hfg x) (hgf x)
```

```
end ScottMap

end DomainTheory.ContinuousLattice
```

8

3.2.28 Exercise 1.21 — complete neighbourhood systems

Work out in greater detail the proof of 1.10. Remark that the neighbourhood system over $|\mathcal{D}|$ so constructed is positive, thereby obtaining in a different way the same kind of conclusion as in 1.20. Show also that the system over $|\mathcal{D}|$ is complete in the sense that every filter is fixed by a unique member of the underlying set. (A filter is *fixed by a point* iff it is the filter of *all* neighbourhoods containing that point.) Remark that a complete system is one where tokens and (partial) elements can always be identified (under a suitable one-one correspondence). Show also that consistency of a set $\{X_i \mid i < n\}$ of neighbourhoods in $\mathcal{D}$ is equivalent to saying

$$\bigcap_{i<n} [X_i] \neq \emptyset.$$

le_antisymm'.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.le
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

/-- Antisymmetry of the pointwise order. -/
theorem le_antisymm' {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (f g : ScottMap D D')
    (hfg : ScottMap.le f g)
    (hgf : ScottMap.le g f) : f = g :=
  ScottMap.ext fun x => le_antisymm (hfg x) (hgf x)
```

```
end ScottMap

end DomainTheory.ContinuousLattice
```

9

3.2.29 Exercise 1.22 — topology of a domain

(For topologists). Show that the neighbourhoods $[X]$ for $X \in \mathcal{D}$ make $|\mathcal{D}|$ into a topological space where the open subsets $\mathcal{U} \subseteq |\mathcal{D}|$ can be characterized by the following two conditions:

(i) whenever $x \in \mathcal{U}$ and $x \sqsubseteq y \in |\mathcal{D}|$, then $y \in \mathcal{U}$; and

(ii) whenever $x \in \mathcal{U}$, then $[X] \in \mathcal{U}$ for some $X \in x$.

Prove also that the inclusion relation on $|\mathcal{D}|$ can be defined topologically as:

(iii) $x \sqsubseteq y$ iff for all open $\mathcal{U} \subseteq |\mathcal{D}|$, if $x \in \mathcal{U}$ then $y \in \mathcal{U}$.

Is $|D|$ ever a Hausdorff space?
Show that if $(x_n)_{n=0}^{\infty}$ is a sequence of elements of $|D|$ with $x_n \subseteq x_{n+1}$ for all n, then

$$\bigcup_{n=0}^{\infty} x_n$$

is not only in $|D|$ but is a topological limit point of the sequence. Show that any element x is a limit point of the set $\{\uparrow X \mid X \in x\}$. Are there other limit points?

le_def.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst28
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap
```

```
theorem le_def {f g : ScottMap D D'} : f ≤ g ↔ ∀ x, (f : D → D') x ≤ g x := Iff.rfl

end ScottMap

end DomainTheory.ContinuousLattice
```

0

3.2.30 Exercise 1.23 — decidable total elements

Suppose that the neighbourhood system D is countable, say,

$$D = \{X_0, X_1, X_2, \ldots, X_n, \ldots\}.$$

Suppose further that the property of consistency of finite sequences of neighbourhoods is decidable (or "effectively known"). Then the following sequence is well defined:

$$\begin{aligned} Y_0 &= X_0 \\ Y_{n+1} &= X_{n+1}, \text{ if this set is consistent with } Y_0, Y_1, \ldots, Y_n; \\ &= Y_n, \text{ if not.} \end{aligned}$$

Show that $\{Y_0, Y_1, \ldots, Y_n, \ldots\}$ is a total element of $|D|$. (Hint: Show first that $Y_0, Y_1, \ldots, Y_{n-1}$ is consistent for all n.) In such a system show that all filters can be determined by sequences.

le_def.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst28
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

theorem le_def {f g : ScottMap D D'} : f ≤ g ↔ ∀ x, (f : D → D') x ≤ g x := Iff.rfl

end ScottMap

end DomainTheory.ContinuousLattice
```

1

3.2.31 Exercise 1.24 — extending partial elements

(For set theorists). Prove, using the Axiom of Choice, or some equivalent principle, that in every domain a partial element can always be extended to a total element. Is this assertion equivalent to the Axiom of Choice? (Hint: Remember to prove that the union of every (transfinite) chain of filters is again a filter.)

le_def.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst28
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

theorem le_def {f g : ScottMap D D'} : f ≤ g ↔ ∀ x, (f : D → D') x ≤ g x := Iff.rfl

end ScottMap

end DomainTheory.ContinuousLattice
```

2

3.2.32 Exercise 1.25 — final segments of ordinals

(For set theorists). Let Δ be any well-ordered set (ordinal). (Even small ordinals like $\omega \cdot 3$ or ω^5 are interesting.) Let $\mathcal{D}$ be the family of non-empty *final* segments of Δ. What is $|\mathcal{D}|$? Are all elements finite? Is every approximation to a finite element finite?

le_def.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst28
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic
```

```
@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

theorem le_def {f g : ScottMap D D'} : f ≤ g ↔ ∀ x, (f : D → D') x ≤ g x := Iff.rfl

end ScottMap

end DomainTheory.ContinuousLattice
```

3

3.2.33 Exercise 1.26 — ideals of a ring

(For algebraists). Let A be a commutative ring with unit. Let Δ be the set of finite subsets $F \subseteq A$. Define

$$I(F) = \{G \in \Delta \mid F \subseteq \text{the ideal generated by } G\}.$$

Prove that the sets of the form $I(F)$ form a neighbourhood system, and that the corresponding domain is isomorphic to the set of ring-theoretic ideals of A partially ordered by inclusion. What would happen if we excluded from Δ all F with $I(F) = I(\{1\})$, where 1 is the unit of A?

le_def.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst28
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
```

```
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

theorem le_def {f g : ScottMap D D'} : f ≤ g ↔ ∀ x, (f : D → D') x ≤ g x := Iff.rfl

end ScottMap

end DomainTheory.ContinuousLattice
```

4

3.2.34 Exercise 1.27 — suprema of bounded sets

Further closure properties of domains can be proved for bounded sets. We say $X \subseteq |\mathcal{D}|$ is *bounded* iff for some $y \in |\mathcal{D}|$ we have $x \subseteq y$ for all $x \in X$. This y is called an *upper bound*. We let

$$\bigsqcup X = \bigcap \{y \in |\mathcal{D}| \mid x \subseteq y \text{ all } x \in X\}.$$

Prove that if X is bounded, then $\bigsqcup X$ is the *least upper bound* for X in $|\mathcal{D}|$. Prove also: if $U, V \in \mathcal{D}$ are neighbourhoods, then $\{U, V\}$ is consistent in $\mathcal{D}$ iff $\{\uparrow U, \uparrow V\}$ is bounded in $|\mathcal{D}|$. (That is, boundedness is for elements what consistency is for neighbourhoods.) Prove finally with the aid of 1.18 that $X \subseteq |\mathcal{D}|$ is bounded iff every finite subset of X is bounded.

le_def.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst28
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

theorem le_def {f g : ScottMap D D'} : f ≤ g ↔ ∀ x, (f : D → D') x ≤ g x := Iff.rfl

end ScottMap
```

```
end DomainTheory.ContinuousLattice
```

5

3.3 Mappings between domains

Dependency diagram. Each box is one item. The arrowhead is on the item that imports. An arrow already implied by a longer path is left out. The introduction has no Lean file, so it is not shown. Blue is a definition, green a proposition, amber an example, red a theorem, and indigo an exercise.

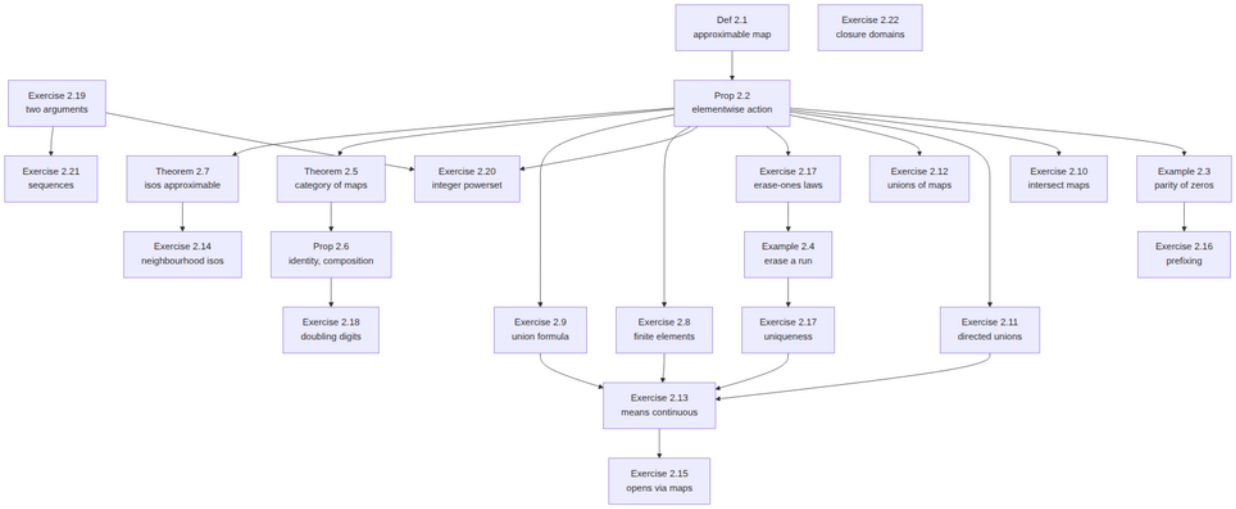


Figure 29: Mappings between domains.

3.3.1 Introduction

LECTURE II MAPPINGS BETWEEN DOMAINS

The elements of a domain are regarded as being specified by approximations: the neighbourhoods. With the idea of approximation as the dominant notion, therefore, it is natural to look for a concept of mapping (transformation of domains) that in some suitable sense preserves the spirit of the approximations. In a 'theory of computability,' where the (finite) approximations to the elements are all we can ever know at one time, the only mappings that can be computed are those that proceed by approximation, somehow passing from the neighbourhoods of one domain over to the neighbourhoods of the other.

Suppose $X \in \mathcal{D}_0$ is given - it is an approximation to certain elements of $|\mathcal{D}_0|$. (More precisely $\uparrow X$ is the approximation in the domain, but it is easier to speak of the neighbourhood X.) What can be said about the approximations of the images of these elements under the mapping we will call f? If X is not a very sharp approximation, then not very much can be said about the image in the other domain $|\mathcal{D}_1|$. Trivially, of course, we can say that Δ_1 is an approximation - because it approximates everything in its domain. Suppose, however, that we could say more. Suppose we could say that both Y and Y' approximate the image of X. If the mapping f is coherent, then it is reasonable to suppose that such a statement would imply that Y and Y' are *consistent* in $\mathcal{D}_1$. But if this is so, then since the two neighbourhoods are meant to cluster around the same images, we can feel some confidence in saying that $Y \cap Y'$ approximates these images. In other words

to specify f we do not supply a unique image of X, but we say which of the $Y \in \mathcal{D}_1$ approximate the (ideal) image. To make this idea work a *monotonicity condition* is also needed since we are trying to express the idea that "if we give at least X as an approximate input to f, then we can expect at least Y as output." Thus,

a mapping is taken as a kind of relation between neighbourhoods.

3.3.2 Definition 2.1 — approximable mapping

DEFINITION 2.1. An *approximable mapping* $f : D_0 \to D_1$ between domains is a binary relation $f \subseteq D_0 \times D_1$ between neighbourhoods such that

(i) $\Delta_0 f \Delta_1$;

(ii) $X f Y$ and $X f Y'$ always imply $X f (Y \cap Y')$;

(iii) $X f Y$, $X' \subseteq X$, and $Y \subseteq Y'$ always imply $X' f Y'$. □

Condition (i) we have already discussed; in a sense it means "ask me no questions and I shall tell you no lies." In other words "zero input can expect at least zero output." The other conditions are compatible with having

$$f = \{\langle X, \Delta_1 \rangle \mid X \in D_0\};$$

that is, f might be the least informative relation and nothing more. But if it is more, then (ii) is, as we explained, a consistency condition. To explain monotonicity in (iii), suppose a mapping relationship is already known, $X f Y$, say. If we *improve* the accuracy of X to $X' \subseteq X$ and if we *degrade* the accuracy of Y to $Y' \supseteq Y$, then we can still assert $X' f Y'$ since this relationship is *less informative* than the former relationship, which was already known. Thus, we see that conditions (i)–(iii) are all reasonably argued as necessary.

One indication that the conditions of 2.1 are sufficient for a definition is that they are exactly what we need to show that f as a neighbourhood relation determines an equivalent elementwise mapping from $|D_0|$ into $|D_1|$. (Owing to the equivalence, we use the same symbol f for both.)

le_def.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst28
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
```

```
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

theorem le_def {f g : ScottMap D D'} : f ≤ g ↔ ∀ x, (f : D → D') x ≤ g x := Iff.rfl

end ScottMap

end DomainTheory.ContinuousLattice
```

6

3.3.3 Proposition 2.2 — elementwise action

PROPOSITION 2.2. Given neighbourhood systems D_0 and D_1, an approximable mapping $f : D_0 \to D_1$ always determines a function $f : |D_0| \to |D_1|$ between domains by virtue of the formula:

(i) $f(x) = \{Y \in D_1 \mid \exists X \in x.\ X f Y\}$

for all $x \in |D_0|$. Conversely, this function uniquely determines
the original relation by the equivalence:

(ii) $X f Y$ iff $Y \in f(\uparrow X)$

for all $X \in D_0$ and $Y \in D_1$. Approximable functions are always monotone in the following sense:

(iii) $x \subseteq y$ always implies $f(x) \subseteq f(y)$,

for $x, y \in |D_0|$; moreover two approximable functions $f : D_0 \to D_1$ and $g : D_0 \to D_1$ are identical as relations iff

(iv) $f(x) = g(x)$, for all $x \in |D_0|$.

Proof: The argument that formula (i) always gives us $f(x) \in |D_1|$ when $x \in |D_0|$ can be safely left to the reader. Note, however, that all the conditions of 2.1 are required to show this. As for (ii), the implication from left to right follows directly from (i) because $X \in \uparrow X$. In the other direction $Y \in f(\uparrow X)$ means that $Z f Y$ holds for some $Z \in \uparrow X$. But from $X \subseteq Z$ it follows that $X f Y$, as we wished.

To prove monotonicity, assume $x \subseteq y$. Now $X \in x$ and $X f Y$ always imply $X \in y$ and $X f Y$. This means $Y \in f(x)$ always implies $Y \in f(y)$; that is, $f(x) \subseteq f(y)$.

Finally, to check that (iv) means $f = g$ as relations, all that has to be remarked that this follows from formulae (i) and (ii). □

Note that the right-hand side of (ii) can be written:

$$\uparrow Y \subseteq f(\uparrow X),$$

which can be read as saying that the partial element determined by the neighbourhood Y approximates the function value at the element determined by X. This precise relationship of course fits the informal discussion of mapping given earlier. Indeed whenever $x \in [X]$ and $X f Y$ hold, then $f(x) \in [Y]$ always

follows, which is another way to construe the mapping character of f. Some examples of mappings are now called for.

le_def.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst28
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

theorem le_def {f g : ScottMap D D'} : f ≤ g ↔ ∀ x, (f : D → D') x ≤ g x := Iff.rfl

end ScottMap

end DomainTheory.ContinuousLattice
```

7

3.3.4 Example 2.3 — parity of leading zeros

EXAMPLE 2.3. Let T be the neighbourhood system of the two-token domain of Example 1.2. To avoid confusion with some other domains, we will call the two total elements of $|T|$ respectively true and false. There is only one other finite element here, namely $\perp =$ undefined. We often use these elements as indicators of results: true indicates a positive outcome; false, a negative outcome; and $\perp$ indicates that there is not enough information to decide the outcome totally.

Let B be the system for the binary tree as in the last chapter. What we wish to define is an approximable mapping $f : B \to T$. The intuitive idea of the mapping we have in mind is that the binary sequence is being read from left to right, and we are counting the number of 0's seen before the first 1 is encountered. We then test the parity of this count; if it is even, the output is true; if not, false. Using a suggestive informal notation with three dots, some results of the function that does the counting and testing can be written as:

$$f(0000101\ldots) = \text{true}$$
$$f(1101110\ldots) = \text{true}$$
$$f(0111011\ldots) = \text{false}$$
$$f(0000000\ldots) = \perp .$$

The last equation is necessary, because 0000000 as a partial element cannot be counted as either even or odd since it can have inconsistent extensions:

$$0000000\ \perp \sqsubseteq 00000001\ \perp$$
$$0000000\ \perp \sqsubseteq 000000000001\ \perp .$$

So, as far as f is concerned, a plain string of 0's is indefinite. The same answer holds if the 0's go on infinitely.

To be more precise we want

$$f(0^n 1\ \perp) = \text{true} \quad \text{if } n \text{ is even;}$$
$$= \text{false} \quad \text{if } n \text{ is odd.}$$

As a binary relation $f \subseteq B \times T$ we will have

$X f Y$ iff $Y \in \perp$ or $X \sqsubseteq 0^n 1\Delta$ for some $n \in \mathbf{N}$ and either n is even and $Y \in$ true or n is odd and $Y \in$ false.

It should be checked that 2.1(i)–(ii) are satisfied. □

le_def.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst28
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

theorem le_def {f g : ScottMap D D'} : f ≤ g ↔ ∀ x, (f : D → D') x ≤ g x := Iff.rfl
```

```
end ScottMap

end DomainTheory.ContinuousLattice
```

8

3.3.5 Example 2.4 — erase a run of ones

EXAMPLE 2.4. Let us briefly describe an approximable mapping $g : B \to B$. Informally, g can be said to "read a sequence from left to right and eliminate the first consecutive run of 1's while copying all the other digits as read." We will have

$$g(0^n 1^k 0x) = 0^{n+1}x$$

provided $k > 0$. (Here 1^k means a string of 1's of length k.) However, if 1^∞ is the infinite sequences of 1's, then

$$g(1^\infty) = \bot, \text{ and}$$
$$g(0^n 1^\infty) = 0^n.$$

This example is instructive, since it shows that a non-trivial mapping can transform a total element into a partial element. □

Aside from our being able to define particular functions outright, we can combine functions in many different ways; the idea of composition is probably the most basic scheme of combination, and there is a technical name for a family of structures with mappings that can be so combined.

le_def.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst28
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

theorem le_def {f g : ScottMap D D'} : f ≤ g ↔ ∀ x, (f : D → D') x ≤ g x := Iff.rfl
```

```
end ScottMap

end DomainTheory.ContinuousLattice
```

9

3.3.6 Theorem 2.5 — category of approximable maps

THEOREM 2.5. The class of neighbourhood systems and approximable mappings form a *category*, where the identity mapping $I_D : D \to D$ relates $X, Y \in D$ as follows:

(i) $X\ I_D\ Y$ iff $X \subseteq Y$.

If $f : D_0 \to D_1$ and $g : D_1 \to D_2$ are given, then the *composition* $g \circ f : D_0 \to D_2$ relates $X \in D_0$ and $Z \in D_2$ as follows:

(ii) $X\ g \circ f\ Z$ iff $\exists Y \in D_1.\ X\ f\ Y$ and $Y\ g\ Z$.

Proof: (We may use MacLane [1971] as the standard reference on category theory, but we require hardly more than the basic definitions at this stage.) To check that we have a category, we need to know that the identity and composition maps really are maps in the category and that certain identity and associative laws hold. Now it is obvious that I_D satisfies 2.1 (i)-(iii). Moreover if $f : D_0 \to D_1$, all we have to prove is:

$$f \circ I_{D_0} = I_{D_1} \circ f = f$$

Checking one of these equations is enough. Thus, for $X \in D_0$ and $Z \in D_1$ we find

$$\begin{aligned} X\ f \circ I_{D_0}\ Z &\text{ iff } \exists Y \in D_0.\ X \subseteq Y \text{ and } Y\ f\ Z \\ &\text{iff } X\ f\ Z. \end{aligned}$$

So, f and $f \circ I_{D_0}$ are the same mapping.

Suppose now that $f : D_0 \to D_1$ and $g : D_1 \to D_2$. We have to verify that $g \circ f$ is an approximable mapping. First off, there is no trouble in seeing that $\Delta_0\ g \circ f\ \Delta_2$ holds. Next, suppose that $X\ g \circ f\ Z$ and $X\ g \circ f\ Z'$ hold. Then we have $X\ f\ Y$ and $Y\ g\ Z$ for some choice of $Y \in D_1$. Also $X\ f\ Y'$ and $Y'\ g\ Z'$ hold for some choice of $Y' \in D_1$. By 2.1 (ii) it follows that $X\ f\ (Y \cap Y')$. Since $Y \cap Y' \subseteq Y$, we conclude $(Y \cap Y')\ g\ Z$ by 2.1 (iii); similarly $(Y \cap Y')\ g\ Z'$. Invoking 2.1 (ii) again, we obtain $(Y \cap Y')\ g\ (Z \cap Z')$, and $X\ g \circ f\ (Z \cap Z')$ is proved.

Suppose finally that $X' \subseteq X$, $X\ g \circ f\ Z$, and $Z \subseteq Z'$. Now $X\ f\ Y$ and $Y\ g\ Z$ for some $Y \in D_1$. But then $X'\ f\ Y$ holds; for a similar reason $Y\ g\ Z'$ holds also. Therefore, $X'\ g \circ f\ Z'$ is established, which means that we have checked 2.1 (iii) for $g \circ f$ and have completed the proof that $g \circ f : D_0 \to D_2$.

The verification of associativity is a purely logical deduction. Thus suppose that in addition to f and g we have $h : D_2 \to D_3$. If $X \in D_0$ and $W \in D_3$ we find

$$\begin{aligned} X\ h \circ (g \circ f)\ W &\text{ iff } \exists Z \in D_2.\ X\ g \circ f\ Z \text{ and } Z\ h\ W \\ &\text{iff } \exists Z \in D_2\ \exists Y \in D_1.\ X\ f\ Y \text{ and } Y\ g\ Z \text{ and } Z\ h\ W \\ &\text{iff } \exists Y \in D_1\ \exists Z \in D_2.\ X\ f\ Y \text{ and } Y\ g\ Z \text{ and } Z\ h\ W \\ &\text{iff } \exists Y \in D_1.\ X\ f\ Y \text{ and } Y\ (h \circ g)\ W \\ &\text{iff } X\ (h \circ g) \circ f\ W. \end{aligned}$$

So, as relations, $h \circ (g \circ f) = (h \circ g) \circ f$. $\square$

It may seem as though we have, in the definition of composition, written things backwards. But the reason is that when mappings are taken as elementwise functions, then the order is preserved in expressions involving the usual function value notation. We have, for example:

le_of_incl_le.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_sup_of_preservesSSup
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- From (i)+(ii): `i` is order-reflecting (`x ⊑ y ↔ i x ⊑ i y`), since `x ⊑ y ↔ x ⊔
    y = y`. -/
theorem le_of_incl_le (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S))
    (hinj : Function.Injective i) {x y : D} (h : i x ≤ i y) : x ≤ y := by
  have h1 : i (x ⊔ y) = i y := by rw [incl_sup_of_preservesSSup hi, sup_eq_right.mpr h
    ]
  exact sup_eq_right.mp (hinj h1)

end Proposition310

end DomainTheory.ContinuousLattice
```

0

3.3.7 Proposition 2.6 — identity and composition

PROPOSITION 2.6. Given $f : D_0 \to D_1$ and $g : D_1 \to D_2$, the following equations hold:

(i) $I_{D_0}(x) = x$, and

(ii) $(g \circ f)(x) = g(f(x))$,

for all $x \in |D_0|$. $\square$

The proof is not troublesome and is left as an exercise. In technical language the result shows that the category defined in Theorem 2.5 is equivalent to a "concrete category" of sets and functions, namely the domains and elementwise transformations of 2.2.

Toward the end of the last lecture (see 1.9) we promised to show that isomorphisms of domains always come from approximable mappings, and this we now do. It means that the category contains all the isomorphisms it should have.

le_of_incl_le.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_sup_of_preservesSSup
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- From (i)+(ii): `i` is order-reflecting (`x ⊑ y ↔ i x ⊑ i y`), since `x ⊑ y ↔ x ⊔
    y = y`. -/
theorem le_of_incl_le (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S))
    (hinj : Function.Injective i) {x y : D} (h : i x ≤ i y) : x ≤ y := by
  have h1 : i (x ⊔ y) = i y := by rw [incl_sup_of_preservesSSup hi, sup_eq_right.mpr h
    ]
  exact sup_eq_right.mp (hinj h1)

end Proposition310

end DomainTheory.ContinuousLattice
```

1

3.3.8 Theorem 2.7 — isomorphisms are approximable

THEOREM 2.7. Every isomorphism between domains results from an approximable mapping between the neighbourhood systems. Moreover, finite elements are always transformed into finite elements.

Proof: Suppose that $f : |D_0| \to |D_1|$ is a one-one, inclusion-preserving function defined on elements, where the range of the function is the whole of $|D_1|$, of course. Taking the hint from 2.2, there is only one way we could define a neighbourhood mapping; namely, we consider the relation $Y \in f(\uparrow X)$ for $X \in D_0$ and $Y \in D_1$. What has to be shown is that this is an approximable mapping which determines the original function via the formula 2.2 (i).

The first part is easy; indeed, there is a general result that monotone functions on finite elements of one domain to arbitrary elements of another domain always determine approximable mappings (cf. Exercise 2.8). What remains, then, is to show that the relation re-defines the function. This comes down to showing that for $x \in |D_0|$

$$f(x) = \{Y \in D_1 \mid \exists X \in x.\, Y \in f(\uparrow X)\}.$$

Consider the right-hand side of this equation: it is a filter. (This either can be proved directly or Exercise 2.11 can be used.) Because f is an onto-function, we can call the right-hand side $f(x')$ for some $x' \in |D_0|$. But since $X \in x$ implies $\uparrow X \subseteq x$ and $f(\uparrow X) \subseteq f(x)$, the right-hand side is included in the left-hand side. In other words $f(x') \subseteq f(x)$. But, since f is an isomorphism, $x' \subseteq x$ follows.

In the other direction, if $X \in x$, then $f(\uparrow X) \subseteq f(x')$ holds by definition, so $\uparrow X \subseteq x'$. This implies $X \in x'$; and, as X is arbitrary, $x \subseteq x'$. So $x = x'$, and $f(x) = f(x')$ as desired.

Finally, consider any finite element $\uparrow X \in |D_0|$, where $X \in D_0$. What we have to show is that $f(\uparrow X)$ is finite in $|D_1|$. Because f is an isomorphism, we can associate uniquely to every $Y \in f(\uparrow X)$ an element $y_Y \subseteq\uparrow X$ in $|D_0|$ where $f(y_Y) =\uparrow Y$. (Just apply the inverse of the function f.) Define

$$z = \bigcup\{y_Y \mid Y \in f(\uparrow X)\}.$$

Because $Y' \subseteq Y$ always implies $y_{Y'} \subseteq y_Y$ and each $y_Y \in |D_0|$, it is easy to show z is a filter and hence is in $|D_0|$ also (cf. Exercise 2.11). Because each $y_Y \subseteq\uparrow X$, then $z \subseteq\uparrow X$, too. But each $y_Y \subseteq z$, so $\uparrow Y = f(y_Y) \subseteq f(z)$ and hence $Y \in f(z)$. As this holds for all $Y \in f(\uparrow X)$, the inclusion $f(\uparrow X) \subseteq f(z)$ follows, as well as $\uparrow X \subseteq z$. Therefore, $z =\uparrow X$ and so $X \in z$. But then $X \in y_Y$ for some $Y \in f(\uparrow X)$, by definition of z. Since $\uparrow X \subseteq y_Y$, we obtain $f(\uparrow X) \subseteq\uparrow Y$, but of course the opposite inclusion is also true from the choice of Y. This means that $f(\uparrow X) =\uparrow Y$ is finite in $|D_1|$ as claimed. We can apply the same argument to the inverse function; and, thus, the finite elements of $|D_0|$ and $|D_1|$ are in a one-one inclusion-preserving correspondence under the isomorphism. □

le_of_incl_le.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_sup_of_preservesSSup
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice
```

```
open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- From (i)+(ii): `i` is order-reflecting (`x ⊑ y ↔ i x ⊑ i y`), since `x ⊑ y ↔ x ⊔
    y = y`. -/
theorem le_of_incl_le (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S))
    (hinj : Function.Injective i) {x y : D} (h : i x ≤ i y) : x ≤ y := by
  have h1 : i (x ⊔ y) = i y := by rw [incl_sup_of_preservesSSup hi, sup_eq_right.mpr h
    ]
  exact sup_eq_right.mp (hinj h1)

end Proposition310

end DomainTheory.ContinuousLattice
```

2

3.3.9 Lecture II exercises

EXERCISES

3.3.10 Exercise 2.8 — determined by finite elements

With reference to the proof of 2.2 show that an approximable mapping is uniquely determined by its elementwise effect on finite elements. Moreover any arbitrary monotone function on finite elements of $|D_0|$ with values in $|D_1|$ comes from an approximable $f : D_0 \to D_1$.

le_of_incl_le.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_sup_of_preservesSSup
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace
```

```
variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- From (i)+(ii): `i` is order-reflecting (`x ⊑ y ↔ i x ⊑ i y`), since `x ⊑ y ↔ x ⊔
    y = y`. -/
theorem le_of_incl_le (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S))
    (hinj : Function.Injective i) {x y : D} (h : i x ≤ i y) : x ≤ y := by
  have h1 : i (x ⊔ y) = i y := by rw [incl_sup_of_preservesSSup hi, sup_eq_right.mpr h
    ]
  exact sup_eq_right.mp (hinj h1)

end Proposition310

end DomainTheory.ContinuousLattice
```

3

3.3.11 Exercise 2.9 — directed union formula

Prove that if $f : D_0 \to D_1$ is an approximable mapping, then the elementwise mapping $f : |D_0| \to |D_1|$ satisfies the equation

$$f(x) = \bigcup \{ f(\uparrow X) \mid X \in x \}$$

for all $x \in |D_0|$. Conversely, show that every elementwise function satisfying this equation comes from an approximable mapping as defined in 2.2.

le_of_incl_le.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_sup_of_preservesSSup
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace
```

```
variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- From (i)+(ii): `i` is order-reflecting (`x ⊑ y ↔ i x ⊑ i y`), since `x ⊑ y ↔ x ⊔
    y = y`. -/
theorem le_of_incl_le (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S))
    (hinj : Function.Injective i) {x y : D} (h : i x ≤ i y) : x ≤ y := by
  have h1 : i (x ⊔ y) = i y := by rw [incl_sup_of_preservesSSup hi, sup_eq_right.mpr h
    ]
  exact sup_eq_right.mp (hinj h1)

end Proposition310

end DomainTheory.ContinuousLattice
```

4

3.3.12 Exercise 2.10 — intersection of two maps

Carry out the proof of Proposition 2.6; and in addition show that, if $f, g : D_0 \to D_1$ are two approximable mappings, there exists $h : D_0 \to D_1$ such that

$$h(x) = f(x) \cap g(x)$$

for all $x \in |D_0|$.

le_of_incl_le.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_sup_of_preservesSSup
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
```

```
variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- From (i)+(ii): `i` is order-reflecting (`x ⊑ y ↔ i x ⊑ i y`), since `x ⊑ y ↔ x ⊔
    y = y`. -/
theorem le_of_incl_le (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S))
    (hinj : Function.Injective i) {x y : D} (h : i x ≤ i y) : x ≤ y := by
  have h1 : i (x ⊔ y) = i y := by rw [incl_sup_of_preservesSSup hi, sup_eq_right.mpr h
    ]
  exact sup_eq_right.mp (hinj h1)

end Proposition310

end DomainTheory.ContinuousLattice
```

5

3.3.13 Exercise 2.11 — directed unions of elements

Let $(I, \leq)$ be a non-empty abstract partially ordered set; suppose it is directed in the sense that whenever $i, j \in I$, then $i \leq k$ and $j \leq k$ for some $k \in I$. Suppose that $a : I \to |D|$ is such that

$$i \leq j \text{ implies } a_i \subseteq a_j$$

for all $i, j \in I$. Prove that

$$\bigcup\{a_i \mid i \in I\}$$

is always a filter in $|D|$. (Note the ways this lemma could be used in the proof of 2.7; but be careful in defining the partially ordered set and do not confuse $\subseteq$ and $\supseteq$.) In words we could say that the domain of filters is closed under directed unions. Prove also that if $f : D \to D'$ is an approximable mapping, then for any directed union

$$f(\bigcup\{a_i \mid i \in I\}) = \bigcup\{f(a_i) \mid i \in I\};$$

that is, approximable mappings always preserve directed unions. If an elementwise function preserves directed unions, must it come from an approximable mapping? (Hint: Invoke 2.9.)

`le_of_incl_le.lean`

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_sup_of_preservesSSup
public import Mathlib.Order.BourbakiWitt
```

```
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- From (i)+(ii): `i` is order-reflecting (`x ⊑ y ↔ i x ⊑ i y`), since `x ⊑ y ↔ x ⊔
    y = y`. -/
theorem le_of_incl_le (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S))
    (hinj : Function.Injective i) {x y : D} (h : i x ≤ i y) : x ≤ y := by
  have h1 : i (x ⊔ y) = i y := by rw [incl_sup_of_preservesSSup hi, sup_eq_right.mpr h
    ]
  exact sup_eq_right.mp (hinj h1)

end Proposition310

end DomainTheory.ContinuousLattice
```

6

3.3.14 Exercise 2.12 — directed unions of maps

Suppose $(I, \leq)$ is a directed, partially ordered set and $f_i : D_0 \to D_1$ is a family of approximable mappings indexed by $i \in I$, where we assume

$$i \leq j \text{ implies } f_i(x) \subseteq f_j(x)$$

for all $i, j \in I$ and all $x \in |D_0|$. Prove that there is an approximable mapping $g : D_0 \to D_1$ where

$$g(x) = \bigcup \{f_i(x) \mid i \in I\}$$

for all $x \in |D_0|$.

le_of_incl_le.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_sup_of_preservesSSup
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- From (i)+(ii): `i` is order-reflecting (`x ⊑ y ↔ i x ⊑ i y`), since `x ⊑ y ↔ x ⊔
    y = y`. -/
theorem le_of_incl_le (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S))
    (hinj : Function.Injective i) {x y : D} (h : i x ≤ i y) : x ≤ y := by
  have h1 : i (x ⊔ y) = i y := by rw [incl_sup_of_preservesSSup hi, sup_eq_right.mpr h
    ]
  exact sup_eq_right.mp (hinj h1)

end Proposition310

end DomainTheory.ContinuousLattice
```

7

3.3.15 Exercise 2.13 — approximable means continuous

(For topologists.) Recall Exercise 1.22 where it was shown that any domain $|D|$ is a topological space. Prove from Exercise 2.9 that the functions $f : |D_0| \to |D_1|$ determined by approximable mappings are exactly *the continuous functions between these spaces.* (Hint: To prove continuity, remark that by 2.9

$$f^{-1}[Y] = \bigcup\{[X] \mid Y \in f(\uparrow X)\};$$

hence, the inverse image of any open set is open. In the other direction, suppose that $f : |D_0| \to |D_1|$ is topologically continuous. Argue that for all $x \in |D_0|$ and all open subsets $U \subseteq |D_1|$ we have

$$f(x) \in U \text{ iff } \exists X \in x.\ f(\uparrow X) \in U.$$

This holds because an open subset of $|D_0|$ is always a union of basic open subsets of the form $[X']$ for $X \in D_0$ and because

$$x = \bigcup \{\uparrow X \mid X \in x\}$$

for all $x \in |D_0|$.)

le_of_incl_le.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_sup_of_preservesSSup
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- From (i)+(ii): `i` is order-reflecting (`x ⊑ y ↔ i x ⊑ i y`), since `x ⊑ y ↔ x ⊔
    y = y`. -/
theorem le_of_incl_le (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S))
    (hinj : Function.Injective i) {x y : D} (h : i x ≤ i y) : x ≤ y := by
  have h1 : i (x ⊔ y) = i y := by rw [incl_sup_of_preservesSSup hi, sup_eq_right.mpr h
    ]
  exact sup_eq_right.mp (hinj h1)

end Proposition310

end DomainTheory.ContinuousLattice
```

8

3.3.16 Exercise 2.14 — isomorphisms of neighbourhoods

Let $f : |D_0| \to |D_1|$ be an isomorphism between domains. Let $\varphi : D_0 \to D_1$ be the one-one correspondence between neighbourhoods provided by Theorem 2.7 where

$$f(\uparrow X) = \uparrow \varphi(X)$$

for all $X \in D_0$. Show that the approximable mapping determined by f is just the relationship $\varphi(X) \subseteq Y$. In addition prove that if $X, X' \in D_0$ are consistent, then

$$\varphi(X \cap X') = \varphi(X) \cap \varphi(X').$$

Remark that the isomorphisms between domains correspond exactly to the isomorphisms between neighbourhood systems (in the sense of one-one inclusion preserving correspondences).

le_of_incl_le.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_sup_of_preservesSSup
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- From (i)+(ii): `i` is order-reflecting (`x ⊑ y ↔ i x ⊑ i y`), since `x ⊑ y ↔ x ⊔
    y = y`. -/
theorem le_of_incl_le (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S))
    (hinj : Function.Injective i) {x y : D} (h : i x ≤ i y) : x ≤ y := by
  have h1 : i (x ⊔ y) = i y := by rw [incl_sup_of_preservesSSup hi, sup_eq_right.mpr h
    ]
  exact sup_eq_right.mp (hinj h1)

end Proposition310

end DomainTheory.ContinuousLattice
```

3.3.17 Exercise 2.15 — opens classified by maps

(For topologists.) Consider the one-token system with

$$\mathcal{O} = \{\{0\}, \emptyset\}.$$

We can regard $|\mathcal{O}|$ as having just two finite elements $\bot$ (bottom) and $\top$ (top), where $\bot \sqsubseteq \top$. For any system D, show that the open subsets U of $|D|$ are in a one-one correspondence with the approximable mappings $f : D \to \mathcal{O}$, where the correspondence is given by the equation

$$U = \{x \in |D| \mid f(x) = \top\}.$$

What are the open subsets of $|\mathcal{O}|$? of $|\mathbb{T}|$? of $|\mathbb{B}|$?

le_refl'.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.le
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

/-- Reflexivity of the pointwise order. -/
theorem le_refl' {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (f : ScottMap D D') :
    ScottMap.le f f :=
  fun _ => le_refl _

end ScottMap

end DomainTheory.ContinuousLattice
```

0

3.3.18 Exercise 2.16 — prefixing is approximable

In the discussion of $\mathbb{B}$ in Chapter 1 we defined a mapping $x \mapsto \sigma x$ for any given $\sigma \in \Sigma^*$. Is this (elementwise) mapping approximable? Show in addition that the mapping $f : \mathbb{B} \to \mathbb{T}$ of 2.3 is uniquely determined among approximable mappings by the equations:

$$f(1x) = \text{true},$$
$$f(01x) = \text{false, and}$$
$$f(00x) = f(x).$$

le_refl'.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.le
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

/-- Reflexivity of the pointwise order. -/
theorem le_refl' {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (f : ScottMap D D') :
    ScottMap.le f f :=
  fun _ => le_refl _

end ScottMap

end DomainTheory.ContinuousLattice
```

1

3.3.19 Exercise 2.17 — equations for erase-ones

Establish in detail that the mapping $g : \mathbb{B} \to \mathbb{B}$ of Exercise 2.4 is approximable. Is it uniquely determined by these equations:

$$g(0x) = 0g(x),$$
$$g(11x) = g(1x),$$
$$g(10x) = 0x,$$
$$g(1) = \bot,$$

or are some missing?

le_refl'.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.le
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

/-- Reflexivity of the pointwise order. -/
theorem le_refl' {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (f : ScottMap D D') :
    ScottMap.le f f :=
  fun _ => le_refl _

end ScottMap

end DomainTheory.ContinuousLattice
```

2

3.3.20 Exercise 2.17 — uniqueness of the erase-ones map

Establish in detail that the mapping $g : \mathbb{B} \to \mathbb{B}$ of Exercise 2.4 is approximable. Is it uniquely determined by these equations:

$$g(0x) = 0g(x),$$

$$g(11x) = g(1x),$$
$$g(10x) = 0x,$$
$$g(1) = \bot,$$

or are some missing?

le_refl'.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.le
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

/-- Reflexivity of the pointwise order. -/
theorem le_refl' {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (f : ScottMap D D') :
    ScottMap.le f f :=
  fun _ => le_refl _

end ScottMap

end DomainTheory.ContinuousLattice
```

3

3.3.21 Exercise 2.18 — doubling binary digits

What is the meaning in words of the approximable mapping $h : \mathbb{B} \to \mathbb{B}$, where

$$h(0x) = 00h(x), \text{ and}$$
$$h(1x) = 10h(x),$$

for all elements $x \in |\mathbb{B}|$? Is h an isomorphism? Does there exist a map $k : \mathbb{B} \to \mathbb{B}$ where

$$k \circ h = I_{\mathbb{B}},$$

and is k one-one?

le_refl'.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.le
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

/-- Reflexivity of the pointwise order. -/
theorem le_refl' {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (f : ScottMap D D') :
    ScottMap.le f f :=
  fun _ => le_refl _

end ScottMap

end DomainTheory.ContinuousLattice
```

4

3.3.22 Exercise 2.19 — maps of two arguments

Generalize Definition 2.1 in an appropriate way in order to define the concept of *an approximable mapping*

$$f : \mathcal{D}_0 \times \mathcal{D}_1 \to \mathcal{D}_2$$

of two variables. (Hint: f can be taken to be a certain kind of ternary relation

$$f \subseteq \mathcal{D}_0 \times \mathcal{D}_1 \times \mathcal{D}_2,$$

where we can write

$$X, Y \; f \; Z$$

for the relationship among neighbourhoods.) What is the corresponding version of Proposition 2.2 for functions of two variables?

le_refl'.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.le
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

/-- Reflexivity of the pointwise order. -/
theorem le_refl' {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (f : ScottMap D D') :
    ScottMap.le f f :=
  fun _ => le_refl _

end ScottMap

end DomainTheory.ContinuousLattice
```

5

3.3.23 Exercise 2.20 — powerset of the integers

Discuss again the example of Exercise 1.15 where the domain turns out to be the powerset (set of all subsets) of $\mathbf{N}$. Show how the finite elements can be taken to be the finite subsets of $\mathbf{N}$ and can be identified with the tokens of a suitable neighbourhood system $\mathcal{P}$. (Hint: Define $\uparrow F$ for finite sets $F \subseteq \mathbf{N}$.) Show that both union and intersection ($x \cup y$ and $x \cap y$) are functions on $|\mathcal{P}|$ that are approximable in the sense of Exercise 2.19. (The elements of $|\mathcal{P}|$ are being identified with arbitrary sets $x \subseteq \mathbf{N}$.) Show also that the following transformations are approximable:

$$x + 1 = \{n + 1 \mid n \in x\}, \text{ and}$$
$$x - 1 = \{n \mid n + 1 \in x\}.$$

le_refl'.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.le
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

/-- Reflexivity of the pointwise order. -/
theorem le_refl' {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (f : ScottMap D D') :
    ScottMap.le f f :=
  fun _ => le_refl _

end ScottMap

end DomainTheory.ContinuousLattice
```

6

3.3.24 Exercise 2.21 — finite and infinite sequences

The system $\mathcal{B}$ of 2.3 has as its total elements only the infinite sequences. Modify the construction of $\mathcal{B}$ to another neighbourhood system $\mathcal{C}$ which has *both* the finite and infinite sequences as total elements. (Hint: $\mathcal{B} \subseteq \mathcal{C}$.) Show that there is an approximable map xy on elements naturally extending ordinary juxtaposition of sequences. (Hint: Write 01001 for a *total* finite sequence and $01001 \perp$ for the corresponding finite partial element. Remember to distinguish between Λ (the total empty sequence) and $\perp$ (the undefined sequence). The definition should work out so that if x is an infinite sequence (hence, total), then $xy = x$ for all y. What will xy equal if x is not total? In other words, the construction possesses a rather strong left-to-right bias.)

le_refl'.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.le
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

/-- Reflexivity of the pointwise order. -/
theorem le_refl' {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (f : ScottMap D D') :
    ScottMap.le f f :=
  fun _ => le_refl _

end ScottMap

end DomainTheory.ContinuousLattice
```

7

3.3.25 Exercise 2.22 — domains from closure properties

(For set theorists). We have remarked in Exercise 1.18 and in Exercise 2.11 that any domain $|D|$, as a family of sets (in fact, a family of subsets of the set D itself), is closed under the intersection of an arbitrary non-empty sub family and under the union of any directed sub family. For those familiar with the subject matter, the example of the (proper) ideals of a commutative ring (with unit) is also seen to be such a family. What is the abstract situation? Let $\mathbf{C}$ be *any* family of sets with these closure properties. It is to be shown that $\mathbf{C}$ is inclusion-isomorphic to a domain. (Hint: Let Δ be the set of finite sets included in sets in $\mathbf{C}$. For $F \in \Delta$, define its "closure" by the equation:

$$\overline{F} = \bigcap \{X \in \mathbf{C} \mid F \subseteq X\}.$$

Every $\overline{F} \in \mathbf{C}$, and these will prove to be the "finite" elements of $\mathbf{C}$. The neighbourhood system $\mathcal{D}$ over Δ can be taken to be the sets of the form

$$C(F) = \{G \in \Delta \mid F \subseteq \overline{G}\}$$

for $F \in \Delta$. Notice that for all $X \in \mathbf{C}$

$$X = \bigcup\{\overline{F} \mid F \subseteq X \text{ and } F \in \Delta\}.$$

)

Check that approximable functions on these families are just those preserving directed unions.

LECTURE III

DOMAIN CONSTRUCTS

Having now seen a number of domains presented through their neighbourhood systems, we need next to introduce general constructs for forming new domains from old. There are an unlimited number of such constructs (technically called *functors*), but we have time only to single out a few of the more important ones. Outstanding among all of them is the notion of product of systems, which in our chosen category has all the expected properties. For the time being in order to simplify notation we assume of the underlying sets Δ_0 and Δ_1 of systems $\mathcal{D}_0$ and $\mathcal{D}_1$ that they are disjoint. There is no loss of generality as $\mathcal{D}_1$ can always be replaced by an isomorphic system disjoint from $\mathcal{D}_0$ in the required sense.

le_refl'.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.le
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

/-- Reflexivity of the pointwise order. -/
theorem le_refl' {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (f : ScottMap D D') :
    ScottMap.le f f :=
  fun _ => le_refl _

end ScottMap
```

```
end DomainTheory.ContinuousLattice
```

8

3.4 Domain constructs

Dependency diagram. Each box is one item. The arrowhead is on the item that imports. An arrow already implied by a longer path is left out. The introduction has no Lean file, so it is not shown. Blue is a definition, green a proposition or lemma, amber an example, red a theorem, and indigo an exercise.

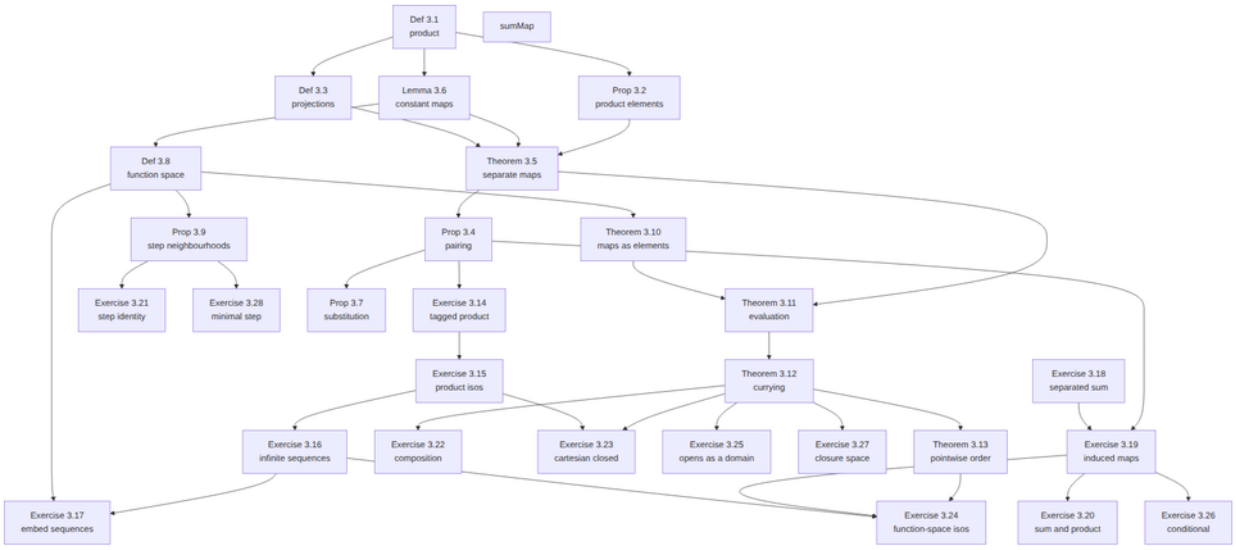


Figure 30: Domain constructs.

3.4.1 Introduction

LECTURE III

DOMAIN CONSTRUCTS

Having now seen a number of domains presented through their neighbourhood systems, we need next to introduce general constructs for forming new domains from old. There are an unlimited number of such constructs (technically called *functors*), but we have time only to single out a few of the more important ones. Outstanding among all of them is the notion of product of systems, which in our chosen category has all the expected properties. For the time being in order to simplify notation we assume of the underlying sets Δ_0 and Δ_1 of systems $\mathcal{D}_0$ and $\mathcal{D}_1$ that they are disjoint. There is no loss of generality as $\mathcal{D}_1$ can always be replaced by an isomorphic system disjoint from $\mathcal{D}_0$ in the required sense.

3.4.2 Definition 3.1 — product of systems

DEFINITION 3.1. Let neighbourhood systems $\mathcal{D}_0$ and $\mathcal{D}_1$ be given over disjoint sets Δ_0 and Δ_1. The *product system* over $\Delta_0 \cup \Delta_1$ is defined by:

$$\mathcal{D}_0 \times \mathcal{D}_1 = \{X \cup Y \mid X \in \mathcal{D}_0 \text{ and } Y \in \mathcal{D}_1\}.$$

For elements $x \in |\mathcal{D}_0|$ and $y \in |\mathcal{D}_1|$ we also define:

$$\langle x, y \rangle = \{X \cup Y \mid X \in x \text{ and } Y \in y\}.$$

□

le_refl'.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.le
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

/-- Reflexivity of the pointwise order. -/
theorem le_refl' {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (f : ScottMap D D') :
    ScottMap.le f f :=
  fun _ => le_refl _

end ScottMap

end DomainTheory.ContinuousLattice
```

9

3.4.3 Proposition 3.2 — elements of a product

PROPOSITION 3.2. The construct $\mathcal{D}_0 \times \mathcal{D}_1$ always gives a neighbourhood system where for elements $x, x' \in |\mathcal{D}_0|$ and $y, y' \in |\mathcal{D}_1|$ we have

(i) $\langle x, y \rangle \subseteq \langle x', y' \rangle$ iff $x \subseteq x'$ and $y \subseteq y'$.

Moreover, there is a one-one correspondence between the elements of $|\mathcal{D}_0 \times \mathcal{D}_1|$ and pairs of elements of $|\mathcal{D}_0|$ and $|\mathcal{D}_1|$ since all elements of $|\mathcal{D}_0 \times \mathcal{D}_1|$ are of the form $\langle x, y \rangle$.

Proof: Owing to the disjointness of Δ_0 and Δ_1, we note that for $X, X' \in \mathcal{D}_0$ and $Y, Y' \in \mathcal{D}_1$ we have

$$(1)\ X \cup Y \subseteq X' \cup Y' \text{ iff } X \subseteq X' \text{ and } Y \subseteq Y'.$$

Thus, $\{X \cup Y, X' \cup Y'\}$ is consistent in $\mathcal{D}_0 \times \mathcal{D}_1$ iff $\{X, X'\}$ is consistent in $\mathcal{D}_0$ and $\{Y, Y'\}$ is consistent in $\mathcal{D}_1$. In the consistent case we find

$$(2) \qquad (X \cup Y) \cap (X' \cup Y') = (X \cap X') \cup (Y \cap Y'),$$

and so $\mathcal{D}_0 \times \mathcal{D}_1$ is closed under consistent intersection. As $\Delta_0 \cup \Delta_1 \in \mathcal{D}_0 \times \mathcal{D}_1$, it is certainly a neighbourhood system.

It is easy to check by the previous calculations that $\langle x, y\rangle \in |\mathcal{D}_0 \times \mathcal{D}_1|$ if $x \in |\mathcal{D}_0|$ and $y \in |\mathcal{D}_1|$. The proof of 3.2(i) follows directly from the definition and (1).

Suppose $z \in |\mathcal{D}_0 \times \mathcal{D}_1|$. Define as a temporary notation:

$$z_0 = \{X \in \mathcal{D}_0 \mid X \cup \Delta_1 \in z\}, \quad \text{and} \quad z_1 = \{Y \in \mathcal{D}_1 \mid \Delta_0 \cup Y \in z\}.$$

Clearly, both $z_0 \in |\mathcal{D}_0|$ and $z_1 \in |\mathcal{D}_1|$. In view of the formula

$$(3) \qquad (X \cup \Delta_1) \cap (\Delta_0 \cup Y) = X \cup Y,$$

we can calculate that

$$z = \langle z_0, z_1\rangle.$$

Moreover, if $z = \langle x, y\rangle$, then $\langle x, y\rangle_0 = x$ and $\langle x, y\rangle_1 = y$.

The one-one correspondence required is thus established. □

There is more going on in the proof of 3.2 than just a one-one correspondence between elements and pairs. The extra information is best formalized by introducing a notation for mappings.

le_sSup_finsetSupOf.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.finsetSupOf
public import DomainTheory.ContinuousLattice.FunctionSpaces.mem_finsetSupOf_of_mem
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']
```

```
section Proposition310

open Finset

namespace IsContinuousLatticeProjection

theorem le_sSup_finsetSupOf (S : Set D) : sSup S ≤ sSup (finsetSupOf S) :=
  sSup_le fun _ hx => le_sSup (mem_finsetSupOf_of_mem hx)

end IsContinuousLatticeProjection

end Proposition310

end DomainTheory.ContinuousLattice
```

0

3.4.4 Definition 3.3 — projection mappings

DEFINITION 3.3. *Projection mappings*

$$p_0 : \mathcal{D}_0 \times \mathcal{D}_1 \to \mathcal{D}_0 \quad \text{and} \quad p_1 : \mathcal{D}_0 \times \mathcal{D}_1 \to \mathcal{D}_1$$

are defined as relations where

$$(X \cup Y)\ p_0\ X' \text{ iff } X \subseteq X', \quad \text{and} \quad (X \cup Y)\ p_1\ Y' \text{ iff } Y \subseteq Y'$$

hold for all $X, X' \in \mathcal{D}_0$ and $Y, Y' \in \mathcal{D}_1$. Given $f : \mathcal{D}_2 \to \mathcal{D}_0$ and $g : \mathcal{D}_2 \to \mathcal{D}_1$, the *paired mapping*

$$\langle f, g \rangle : \mathcal{D}_2 \to \mathcal{D}_0 \times \mathcal{D}_1$$

is defined as a relation where

$$Z\ \langle f, g \rangle\ (X \cup Y) \text{ iff } Z\ f\ X \text{ and } Z\ g\ Y$$

holds for all $X \in \mathcal{D}_0$, $Y \in \mathcal{D}_1$, and $Z \in \mathcal{D}_2$. □

le_sSup_finsetSupOf.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.finsetSupOf
public import DomainTheory.ContinuousLattice.FunctionSpaces.mem_finsetSupOf_of_mem
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
```

```
variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

theorem le_sSup_finsetSupOf (S : Set D) : sSup S ≤ sSup (finsetSupOf S) :=
  sSup_le fun _ hx => le_sSup (mem_finsetSupOf_of_mem hx)

end IsContinuousLatticeProjection

end Proposition310

end DomainTheory.ContinuousLattice
```

1

3.4.5 Proposition 3.4 — pairing and projections

PROPOSITION 3.4. The mappings p_0, p_1 and $\langle f, g\rangle$ are approximable mappings, provided f and g are, and we have:

(i) $p_0 \circ \langle f, g\rangle = f$ and $p_1 \circ \langle f, g\rangle = g$.

Moreover, for $z \in |\mathcal{D}_0 \times \mathcal{D}_1|$, we have:

(ii) $p_0(z) = z_0$ and $p_1(z) = z_1$,

in the notation of the proof of 3.2. Further if $h : \mathcal{D}_2 \to \mathcal{D}_0 \times \mathcal{D}_1$ is any approximable mapping, then

(iii) $h = \langle p_0 \circ h, p_1 \circ h\rangle$.

Moreover, for all $w \in |\mathcal{D}_2|$, we have:

(iv) $\langle f, g\rangle(w) = \langle f(w), g(w)\rangle$,

where again on the right-hand side the notation of the proof of 3.2 is used. □

The proof of this result is left as an exercise. Note the consequence that there is a one-one correspondence between pairs of approximable mappings $f : \mathcal{D}_2 \to \mathcal{D}_0$ and $g : \mathcal{D}_2 \to \mathcal{D}_1$ and mappings $h : \mathcal{D}_2 \to \mathcal{D}_0 \times \mathcal{D}_1$. It is clear that we generalize all this to products

$$\mathcal{D}_0 \times \mathcal{D}_1 \times \cdots \times \mathcal{D}_{n-1}$$

of several systems.

The product construct also neatly explains functions of several variables. In Exercise 2.19 we used the informal notation

$$f : \mathcal{D}_0 \times \mathcal{D}_1 \to \mathcal{D}_2$$

and suggested regarding f as a ternary relation

$$X, Y \; f \; Z.$$

But now with $\mathcal{D}_0 \times \mathcal{D}_1$ given an independent meaning, all we have to do is to regard f as a binary relation with

$$(X, Y) \; f \; Z$$

equivalent to the old relationship. We can also employ an element-wise notation as in $f(\langle x, y \rangle)$, which can more easily be written $f(x, y)$. Similar remarks apply to functions of more than two arguments.

We have discussed several times what it means for a function $f(x)$ to come from an approximable mapping. It is interesting to ask the analogous question for functions of several arguments.

le_sSup_finsetSupOf.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.finsetSupOf
public import DomainTheory.ContinuousLattice.FunctionSpaces.mem_finsetSupOf_of_mem
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

theorem le_sSup_finsetSupOf (S : Set D) : sSup S ≤ sSup (finsetSupOf S) :=
  sSup_le fun _ hx => le_sSup (mem_finsetSupOf_of_mem hx)

end IsContinuousLatticeProjection
```

```
end Proposition310

end DomainTheory.ContinuousLattice
```

2

3.4.6 Theorem 3.5 — separate approximability

THEOREM 3.5. An elementwise function

$$f : |D_0 \times D_1| \to |D_2|$$

of two arguments comes from an approximable mapping iff for each fixed $a \in |D_0|$ and each fixed $b \in |D_1|$ the transformations

$$x \mapsto f(x, b) \quad \text{and} \quad y \mapsto f(a, y)$$

come from approximable mappings of one argument.

Proof: As this is the first time we have had to deal with constants in functions, a lemma is useful.

le_sSup_finsetSupOf.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.finsetSupOf
public import DomainTheory.ContinuousLattice.FunctionSpaces.mem_finsetSupOf_of_mem
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

theorem le_sSup_finsetSupOf (S : Set D) : sSup S ≤ sSup (finsetSupOf S) :=
  sSup_le fun _ hx => le_sSup (mem_finsetSupOf_of_mem hx)

end IsContinuousLatticeProjection

end Proposition310
```

```
end DomainTheory.ContinuousLattice
```

3

3.4.7 Lemma 3.6 — constant functions

LEMMA 3.6. Given $b \in |D_1|$, the constant function

$$b : |D_0| \to |D_1|$$

where $b(x) = b$ for all $x \in |D_0|$, comes from the approximable mapping such that

$$X \; b \; Y \quad \text{iff} \quad Y \in b,$$

for all $X \in D_0$ and $Y \in D_1$. □

There is no real confusion here in using "b" both for function and value. Returning, then, to the proof of 3.5, we see that the reason that $x \mapsto f(x, b)$ comes from an approximable mapping is that the mapping in question is the composition of two approximable mappings, namely $f \circ \langle I_{D_0}, b \rangle$. Clearly we can interchange the rôles of D_0 and D_1 to get at $y \mapsto f(a, y)$.

Conversely, assume that both these functions come from approximable mappings no matter the choice of a and b. Clearly the mapping to determine f is the relation from $X \cup Y$ to Z where

$$Z \in f(\uparrow X, \uparrow Y) = f(\uparrow (X \cup Y)).$$

To prove that this determines f we calculate by the formula of Exercise 2.9:

$$\begin{aligned} f(x, y) &= \bigcup \{ f(\uparrow X, y) \mid X \in x \} \\ &= \bigcup \{ \bigcup \{ f(\uparrow X, \uparrow Y) \mid Y \in y \} \mid X \in x \} \\ &= \bigcup \{ f(\uparrow X, \uparrow Y) \mid X \in x \text{ and } Y \in y \} \\ &= \bigcup \{ f(\uparrow (X \cup Y)) \mid (X \cup Y) \in \langle x, y \rangle \}. \end{aligned}$$

And, again by 2.9, this is what was needed. □

Said more informarily, a function of several arguments is approximable in all the variables *jointly* if it is approximable in each of the variables *separately*.

The type of argument used in 3.5 in the first half of the proof also provides a generalization of 2.6 to functions of several arguments. When we form a function like

$$f(g(x, z, \ldots), h(y, x, \ldots), k(z, w, \ldots), \ldots)$$

from given functions $f, g, h, k, \ldots$; we call the process *substitution*.

`le_sSup_finsetSupOf.lean`

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.finsetSupOf
public import DomainTheory.ContinuousLattice.FunctionSpaces.mem_finsetSupOf_of_mem
public import Mathlib.Order.BourbakiWitt
```

```
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

theorem le_sSup_finsetSupOf (S : Set D) : sSup S ≤ sSup (finsetSupOf S) :=
  sSup_le fun _ hx => le_sSup (mem_finsetSupOf_of_mem hx)

end IsContinuousLatticeProjection

end Proposition310

end DomainTheory.ContinuousLattice
```

4

3.4.8 Proposition 3.7 — closure under substitution

PROPOSITION 3.7. The functions of several arguments between domains coming from approximable mappings are closed under substitution.

Proof: An example will establish the method. Suppose there are four variables involved taking values in domains provided by systems D_0, D_1, D_2, D_3. We might have a substitution like:

$$f(g(x_0, x_1), h(x_1, x_2), k(x_3, x_0, x_2)).$$

Here it might be that the values of the functions inside come from quite other systems; for instance,

$$k : D_3 \times D_0 \times D_2 \to D_4$$

might be possible. By using projections

$$p_i : D_0 \times D_1 \times D_2 \times D_3 \to D_i,$$

where $i < 4$, we can assure that we have several functions all on the same product; thus,

$$k \cdot \langle p_3, p_0, p_2 \rangle : D_0 \times D_1 \times D_2 \times D_3 \to D_4.$$

Now no matter on what domains f is defined, the following composition makes sense:

$$f \circ \langle g \circ \langle p_0, p_1 \rangle, h \circ \langle p_1, p_2 \rangle, k \circ \langle p_3, p_0, p_2 \rangle \rangle;$$

and in fact this is the desired function. Writing it this way makes it clear that the function comes from an approximable mapping: we apply 3.3 (generalized, of course, to products with several terms) to construe the parts between brackets $\langle$ and $\rangle$ as approximable mappings, and then by this trick the composition $\circ$ is the ordinary composition of 2.6. □

It has to be admitted that there is a slight point overlooked in forming products like $D \times D$ with two identical domains. This is discussed in Exercise 3.14, invoking explicit isomorphisms.

The construct that makes the whole theory of domains work so smoothly is the function-space construct: it is possible to regard functions as *objects* which form a domain. Look back at Definition 2.1 and compare it with the original definition of element in 1.6. There are obvious formal similarities, except that filters are sets of neighbourhoods and mappings are sets of pairs of neighbourhoods (relations). But as we saw in 1.10 it is possible to turn the filters into tokens via a simple definition of neighbourhood. We apply the same kind of definition to the mappings.

le_sSup_finsetSupOf.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.finsetSupOf
public import DomainTheory.ContinuousLattice.FunctionSpaces.mem_finsetSupOf_of_mem
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

theorem le_sSup_finsetSupOf (S : Set D) : sSup S ≤ sSup (finsetSupOf S) :=
  sSup_le fun _ hx => le_sSup (mem_finsetSupOf_of_mem hx)

end IsContinuousLatticeProjection

end Proposition310
```

```
end DomainTheory.ContinuousLattice
```

5

3.4.9 Definition 3.8 — function space

DEFINITION 3.8. Given neighbourhood systems D_0 and D_1, the *function space* $(D_0 \to D_1)$ is the system whose set of tokens is the set of approximable mappings of Definition 2.1 and whose neighbourhoods are finite non-empty intersections of sets of the form

$$[X, Y] = \{f : D_0 \to D_1 \mid X f Y\},$$

where $X \in D_0$ and $Y \in D_1$. □

We have been calling our mappings "approximable" for a long time now without saying exactly how they can be approximated! Definition 3.8 supplies the missing key, because once a domain has been defined, then the general theory gives an explicit meaning to the word approximation. We still have to verify, however, that the mappings do correspond to the elements of the domain.

le_sSup_finsetSupOf.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.finsetSupOf
public import DomainTheory.ContinuousLattice.FunctionSpaces.mem_finsetSupOf_of_mem
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

theorem le_sSup_finsetSupOf (S : Set D) : sSup S ≤ sSup (finsetSupOf S) :=
  sSup_le fun _ hx => le_sSup (mem_finsetSupOf_of_mem hx)

end IsContinuousLatticeProjection

end Proposition310
```

```
end DomainTheory.ContinuousLattice
```

6

3.4.10 Proposition 3.9 — consistent step neighbourhoods

PROPOSITION 3.9. Let neighbourhoods $X_i \in D_0$ and $Y_i \in D_1$ be given for $i < n$. Then the set of $[X_i, Y_i]$ for $i < n$ is consistent in $(D_0 \to D_1)$ iff the following condition holds:

(i) whenever $I \subseteq \{0, 1, \ldots, n-1\}$ and $\{X_i \mid i \in I\}$ is consistent in D_0, then $\{Y_i \mid i \in I\}$ must be consistent in D_1.

Moreover, when consistency holds, the least approximable mapping f_0 belonging to the intersection of the $[X_i, Y_i]$ is defined by:

(ii) $X f_0 Y$ iff $\bigcap\{Y_i \mid X \subseteq X_i\} \subseteq Y$ for $X \in D_0$ and $Y \in D_1$.

Proof: Suppose the $[X_i, Y_i]$ are consistent in $(D_0 \to D_1)$. Since the function space is being defined outright as a positive system, consistency means

$$f \in \bigcap\{[X_i, Y_i] \mid i < n\}$$

for some $f : D_0 \to D_1$. Now, with f in hand, let us check condition (i). Suppose $\{X_i \mid i \in I\}$ is consistent. This means

$$x \in \bigcap\{[X_i] \mid i \in I\}$$

for some $x \in |D_0|$. Suppose $i \in I$, so $x \in [X_i]$. Since $X_i f Y_i$ holds, $f(x) \in [Y_i]$. This means, therefore, that

$$f(x) \in \bigcap\{[Y_i] \mid i \in I\},$$

and so $\{Y_i \mid i \in I\}$ is consistent.

For the converse, suppose (i) is the case. We take (ii) as the definition of a mapping and remark that for an arbitrary $X \in D_0$, the set $\{X_i \mid X \subseteq X_i\}$ is automatically consistent in D_0. By our assumption, the set $\{Y_i \mid X \subseteq X_i\}$ is therefore consistent. This means that

$$\bigcap\{Y_i \mid X \subseteq X_i\} \in D_1.$$

(Keep in mind that i is restricted to those $i < n$, and there are only finitely many neighbourhoods being considered here.) It is thus almost immediate that the relation f_0 defined by (ii) satisfies conditions of 2.1 and so is an approximable mapping $f_0 : D_0 \to D_1$. By construction

$$X_i f_0 Y_i$$

holds trivially for all $i < n$; therefore,

$$f_0 \in \bigcap\{[X_i, Y_i] \mid i < n\}$$

and the desired consistency is established.

Finally suppose that f is any mapping in the neighbourhood under discussion; this means $X_i f Y_i$ holds for all $i < n$. Suppose $X f_0 Y$ holds. We have for $X \subseteq X_i$, $X f Y_i$; so

$$X f \bigcap \{Y_i \mid X \subseteq X_i\} \subseteq Y.$$

Thus, $X f Y$ follows; hence, as relations, $f_0 \subseteq f$. In other words f_0 is the minimal element of the neighbourhood. □

We note that, as a consequence of what we have just proved, when the neighbourhood is consistent, then

$$\bigcap \{[X_i, Y_i] \mid i < n\} \subseteq [X, Y]$$

is exactly equivalent to

$$\bigcap \{Y_i \mid X \subseteq X_i\} \subseteq Y.$$

Note also that a single neighbourhood $[X_0, Y_0]$ is always consistent since it contains the *constant mapping* k where

$$X k Y \text{ iff } Y_0 \subseteq Y,$$

for all $X \in \mathcal{D}_0$ and $Y \in \mathcal{D}_1$. Some other simple observations about these neighbourhoods are just translations of the conditions of Definition 2.1:

$$[\Delta_0, \Delta_1] = |\mathcal{D}_0 \to \mathcal{D}_1|;$$

$$[X, Y] \cap [X, Y'] = [X, Y \cap Y']; \text{ and}$$

$$X' \subseteq X \text{ and } Y \subseteq Y' \text{ imply } [X, Y] \subseteq [X', Y'],$$

for all $X, X' \in \mathcal{D}_0$ and $Y, Y' \in \mathcal{D}_1$. We are now ready to prove the main result about the construct.

le_sSup_finsetSupOf.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.finsetSupOf
public import DomainTheory.ContinuousLattice.FunctionSpaces.mem_finsetSupOf_of_mem
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]
```

```
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

theorem le_sSup_finsetSupOf (S : Set D) : sSup S ≤ sSup (finsetSupOf S) :=
  sSup_le fun _ hx => le_sSup (mem_finsetSupOf_of_mem hx)

end IsContinuousLatticeProjection

end Proposition310

end DomainTheory.ContinuousLattice
```

7

3.4.11 Theorem 3.10 — elements are mappings

THEOREM 3.10. Given neighbourhood systems $\mathcal{D}_0$ and $\mathcal{D}_1$, the function space system $(\mathcal{D}_0 \to \mathcal{D}_1)$ is complete in the sense that every filter in $|\mathcal{D}_0 \to \mathcal{D}_1|$ is fixed by a unique approximable mapping.

Proof: Let $f : \mathcal{D}_0 \to \mathcal{D}_1$ be an approximable mapping. By the very definition of $(\mathcal{D}_0 \to \mathcal{D}_1)$ it determines a filter by the definition:

$$\hat{f} = \{F \in (D_0 \to D_1) \mid f \in F\}.$$

Trivially $[X, Y] \in \hat{f}$ iff $f \in [X, Y]$ iff XfY; so this filter uniquely determines the relation f. What we have to show is that every filter in $|D_0 \to D_1|$ is of this form.

Suppose $\varphi \in |D_0 \to D_1|$ is any filter. A relation can be defined at once by

$$X\hat{\varphi}Y \quad \text{iff} \quad [X, Y] \in \varphi.$$

In view of the remarks we made just before stating this theorem, there is no problem in showing that $\hat{\varphi}$ is an approximable mapping. Since the neighbourhoods of the function space are in any case finite intersections of sets like $[X, Y]$, those $[X, Y] \in \varphi$ generate φ. This means that $\hat{\hat{\varphi}} = \varphi$. By definition $\hat{\hat{f}} = f$, so there is a one-one correspondence between mappings and filters. (This correspondence is obviously inclusion preserving, too.) □

We now know just about everything about $|D_0 \to D_1|$ as a domain: the elements correspond isomorphically to the approximable mappings; the finite elements are explained completely by 3.9; and we have seen how to calculate with neighbourhoods. The final step is to relate the function space to other domains by appropriate mappings. In doing this we shall freely construe elements of $|D_0 \to D_1|$ as approximable mappings in view of 3.10.

le_sSup_finsetSupOf.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.finsetSupOf
public import DomainTheory.ContinuousLattice.FunctionSpaces.mem_finsetSupOf_of_mem
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

theorem le_sSup_finsetSupOf (S : Set D) : sSup S ≤ sSup (finsetSupOf S) :=
  sSup_le fun _ hx => le_sSup (mem_finsetSupOf_of_mem hx)

end IsContinuousLatticeProjection

end Proposition310

end DomainTheory.ContinuousLattice
```

8

3.4.12 Theorem 3.11 — evaluation combinator

THEOREM 3.11. Given neighbourhood systems D_1 and D_2, there is a uniquely determined approximable mapping

$$\text{eval} : (D_1 \to D_2) \times D_1 \to D_2,$$

where for all $f : D_1 \to D_2$ and all $x \in |D_1|$ we have

(i) $\text{eval}(f, x) = f(x)$.

Proof: For $F \in (D_1 \to D_2)$ and $X \in D_1$ and $Y \in D_2$ define eval as a relation by:

$$F, X \text{ eval } Y \quad \text{iff} \quad X \mathrel{f} Y \text{ for all } f \in F.$$

Remember that neighbourhoods in the function space are sets of approximable mappings. It is easily checked that this definition makes eval approximable. We now calculate the function values by the formula of 2.2 (i):

$$\text{eval}(f,x) = \{Y \in D_2 \mid \exists F \in (D_1 \to D_2)\, \exists X \in x.\ f \in F \text{ and } F \cup X \text{ eval } Y\}$$

Because, again by 2.2 (i), we have

$$f(x) = \{Y \in D_2 \mid \exists X \in x.\ XfY\},$$

we can see from the definition of eval that $\text{eval}(f,x) \subseteq f(x)$. Suppose that $Y \in f(x)$. Then XfY holds for some $X \in x$. We can write $f \in [X,Y] \in (D_1 \to D_2)$ and it is clear that

$$[X,Y] \cup X \text{ eval } Y$$

holds by definition. Therefore, $Y \in \text{eval}(f,x)$, and so $f(x) \subseteq \text{eval}(f,x)$. □

This theorem is essential for our programme: it shows that in taking functions as objects the very basic operation of forming the function value is an approximable mapping. In other words we can treat the expression $f(x)$ not just as a function of x, as we have done from the start, but also as a function of f as well. The result also indicates that there are useful maps defined on domains that themselves are function spaces; we shall meet many more of these. The next theorem provides further examples.

le_sSup_finsetSupOf.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.finsetSupOf
public import DomainTheory.ContinuousLattice.FunctionSpaces.mem_finsetSupOf_of_mem
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

theorem le_sSup_finsetSupOf (S : Set D) : sSup S ≤ sSup (finsetSupOf S) :=
  sSup_le fun _ hx => le_sSup (mem_finsetSupOf_of_mem hx)
```

```
end IsContinuousLatticeProjection

end Proposition310

end DomainTheory.ContinuousLattice
```

9

3.4.13 Theorem 3.12 — currying

THEOREM 3.12. Given neighbourhood systems D_0, D_1, D_2 there is associated with every approximable mapping $g : D_0 \times D_1 \to D_2$ a uniquely determined approximable mapping

$$\text{curry}(g) : D_0 \to (D_1 \to D_2)$$

such that for $x \in |D_0|$ and $y \in |D_1|$

(i) $\text{curry}(g)(x)(y) = g(x, y)$.

Moreover we have these functional equations:

(ii) $\text{eval} \circ \langle \text{curry}(g) \circ p_0,\ p_1 \rangle = g$, and

(iii) $\text{curry}(\text{eval} \circ \langle h \circ p_0,\ p_1 \rangle) = h$,

where the $p_i : \mathcal{D}_0 \times \mathcal{D}_1 \to \mathcal{D}_i$ are the projection mappings and $h : \mathcal{D}_0 \to (\mathcal{D}_1 \to \mathcal{D}_2)$ is any approximable mapping. This provides an isomorphism between the domains $|\mathcal{D}_0 \times \mathcal{D}_1 \to \mathcal{D}_2|$ and $|\mathcal{D}_0 \to (\mathcal{D}_1 \to \mathcal{D}_2)|$ and so we can regard

$$\text{curry} : (\mathcal{D}_0 \times \mathcal{D}_1 \to \mathcal{D}_2) \to (\mathcal{D}_0 \to (\mathcal{D}_1 \to \mathcal{D}_2))$$

as itself being an approximable mapping.

Proof: Given g as indicated, we can define $\text{curry}(g)$ as a relation and as an approximable mapping by:

$$X\ \text{curry}(g)\ [Y, Z] \text{ iff } X \cup Y\ g\ Z \quad \text{(but see Ex. 3.21)}$$

for all $X \in \mathcal{D}_0$, $Y \in \mathcal{D}_1$, $Z \in \mathcal{D}_2$. This is sufficient because an approximable mapping is intersective in the right-hand neighbourhood, so we know from the above exactly what $X\ \text{curry}(g)\ \bigcap\{[Y_i, Z_i] \mid i < n\}$ means for all finite intersections. The remark after 3.9 is then helpful in checking that by this definition $\text{curry}(g)$ satisfies the monotonicity condition and so is indeed approximable. We now calculate:

$$\begin{aligned}
\text{curry}(g)(x)(y) &= \{Z \in \mathcal{D}_2 \mid \exists Y \in y.\ Y\ \text{curry}(g)\ (x)\ Z\} \\
&= \{Z \in \mathcal{D}_2 \mid \exists Y \in y\ \exists X \in x.\ X\ \text{curry}(g)\ [Y, Z]\} \\
&= \{Z \in \mathcal{D}_2 \mid \exists Y \in y\ \exists X \in x.\ X \cup Y\ g\ Z\} \\
&= \{Z \in \mathcal{D}_2 \mid \exists W \in \langle x, y \rangle.\ W\ g\ Z\} \\
&= g(\langle x, y \rangle) = g(x, y).
\end{aligned}$$

This proves (i). We also see, that if we take the left-hand side of (ii) and apply it to a pair $\langle x, y \rangle$, it reduces to $g(x, y)$ by virtue of (i). Thus, the two functions in (ii) are the same.

Turning to (iii), call the left-hand side k. Using (i) again, we find

$$\begin{aligned} k(x)(y) &= \mathrm{eval} \circ \langle h \circ p_0, p_1 \rangle (\langle x, y \rangle) \\ &= \mathrm{eval}(\langle h \circ p_0(\langle x, y \rangle), p_1(\langle x, y \rangle)\rangle) \\ &= \mathrm{eval}(\langle h(x), y \rangle) \\ &= h(x)(y). \end{aligned}$$

As this is true for all $y \in |\mathcal{D}_1|$, then $k(x) = h(x)$ follows. As this is true for all $x \in |\mathcal{D}_0|$, then $k = h$ follows, and (iii) is proved.

Taking (ii) and (iii) together, it is clear that the domains $|D_0 \times D_1 \to D_2|$ and $|D_0 \to (D_1 \to D_2)|$ are in a one-one correspondence.

$$\mathrm{curry}(g) \sqsubseteq \mathrm{curry}(g') \text{ iff } g \sqsubseteq g'.$$

Hence, curry is an isomorphism, and we can invoke 2.7 to conclude that it comes from an approximable mapping. □

We close this lecture with some order-theoretic properties of function spaces that characterize inclusion and upper bounds of functions in a "pointwise" manner.

le_trans'.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.le
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

/-- Transitivity of the pointwise order. -/
theorem le_trans' {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (f g h : ScottMap D D')
    (hfg : ScottMap.le f g)
    (hgh : ScottMap.le g h) : ScottMap.le f h :=
  fun x => le_trans (hfg x) (hgh x)
```

```
end ScottMap

end DomainTheory.ContinuousLattice
```

0

3.4.14 Theorem 3.13 — pointwise order of maps

THEOREM 3.13. For approximable functions $f, g : D_0 \to D_1$ we have

(i) $f \sqsubseteq g$ iff $f(x) \sqsubseteq g(x)$ for all $x \in |D_0|$.

For subsets $F \subseteq |D_0 \to D_1|$ we have

(ii) F is bounded in $|D_0 \to D_1|$ iff $\{f(x) \mid f \in F\}$ is bounded in $|D_1|$ for each $x \in |D_0|$;

and in that case for all $x \in |D_0|$:

(iii) $(\bigsqcup F)(x) = \bigsqcup\{f(x) \mid f \in F\}$.

Proof. The implication in (i) from left to right follows because evaluation is monotone in the function as well as the argument. The converse implication is a consequence of 2.2(ii).

For the proof of (ii) and (iii) we see that by (i) if F is bounded, so is every set $\{f(x) \mid f \in F\}$. For the converse direction, it is clear that (iii) defines *some* pointwise mapping; we have only to prove that it is *approximable*. The calculation that $\sqcup F$ preserves directed unions (see 2.9 and 2.11) is probably the simplest way to reach the conclusion. □

le_trans'.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.le
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap
```

```
/-- Transitivity of the pointwise order. -/
theorem le_trans' {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (f g h : ScottMap D D')
    (hfg : ScottMap.le f g)
    (hgh : ScottMap.le g h) : ScottMap.le f h :=
  fun x => le_trans (hfg x) (hgh x)

end ScottMap

end DomainTheory.ContinuousLattice
```

1

3.4.15 Exercise 3.14 — tagged product of tokens

For the most part we can assume that there is at most a countable number of tokens; thus, without loss of generality the underlying sets Δ_i of given systems $\mathcal{D}_i$ could be assumed to be subsets of Σ^* where $\Sigma = \{0, 1\}$. (Any denumerable set would do.) Show that the product $\mathcal{D}_0 \times \mathcal{D}_1$ could be defined as the system over the set $0\Delta_0 \cup 1\Delta_1$ where

$$\mathcal{D}_0 \times \mathcal{D}_1 = \{0X \cup 1Y \mid X \in \mathcal{D}_0 \text{ and } Y \in \mathcal{D}_1\}.$$

In other words, the assumption of the disjointness of Δ_0 and Δ_1 is unnecessary. Give, therefore, the revised definition of $\langle x, y\rangle$ for elements, and prove that for a single system $\mathcal{D}$, there exists an approximable mapping

$$\mathrm{diag} : \mathcal{D} \to \mathcal{D} \times \mathcal{D}$$

where $\mathrm{diag}(x) = \langle x, x\rangle$ for all $x \in |\mathcal{D}|$. Also extend the definition to a product of n-factors

$$\mathcal{D}_0 \times \mathcal{D}_1 \times \cdots \times \mathcal{D}_{n-1}$$

which will be a system over the set

$$\bigcup_{i<n} 1^i 0\Delta_i.$$

Note that for a 2-termed product we simplify $10\Delta_1$ to $1\Delta_1$.

le_trans'.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.le
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace
```

```
universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

/-- Transitivity of the pointwise order. -/
theorem le_trans' {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (f g h : ScottMap D D')
    (hfg : ScottMap.le f g)
    (hgh : ScottMap.le g h) : ScottMap.le f h :=
  fun x => le_trans (hfg x) (hgh x)

end ScottMap

end DomainTheory.ContinuousLattice
```

□

3.4.16 Exercise 3.15 — product isomorphisms

Establish the usual isomorphisms:

(i) $\mathcal{D}_0 \times \mathcal{D}_1 \cong \mathcal{D}_1 \times \mathcal{D}_0$;

(ii) $\mathcal{D}_0 \times (\mathcal{D}_1 \times \mathcal{D}_2) \cong (\mathcal{D}_0 \times \mathcal{D}_1) \times \mathcal{D}_2 \cong \mathcal{D}_0 \times \mathcal{D}_1 \times \mathcal{D}_2$.

How does the product of no factors fit in? Prove also:

(iii) $\mathcal{D}_0 \cong \mathcal{D}_0'$ and $\mathcal{D}_1 \cong \mathcal{D}_1'$ imply $\mathcal{D}_0 \times \mathcal{D}_1 \cong \mathcal{D}_0' \times \mathcal{D}_1'$.

le_trans'.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.le
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v
```

```
variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

/-- Transitivity of the pointwise order. -/
theorem le_trans' {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (f g h : ScottMap D D')
    (hfg : ScottMap.le f g)
    (hgh : ScottMap.le g h) : ScottMap.le f h :=
  fun x => le_trans (hfg x) (hgh x)

end ScottMap

end DomainTheory.ContinuousLattice
```

3

3.4.17 Exercise 3.16 — infinite sequences of elements

Let $\mathcal{D}$ be a given neighbourhood system over $\Delta \subseteq \Sigma^*$. Define

$$\Delta^\infty = \bigcup_{n=0}^{\infty} 1^n 0 \Delta,$$

so that Δ^∞ is split into infinitely many disjoint copies of Δ. Let $\mathcal{D}^\infty$ be the least family of subsets of Σ^* where

1. $\Delta^\infty \in \mathcal{D}^\infty$, and
2. whenever $X \in \mathcal{D}$ and $Y \in \mathcal{D}^\infty$, then $0X \cup 1Y \in \mathcal{D}^\infty$.

Show that $\mathcal{D}^\infty$ is a neighbourhood system over Δ^∞. Prove the isomorphism

$$\mathcal{D}^\infty \cong \mathcal{D} \times \mathcal{D}^\infty.$$

Show, moreover, that the elements of $|\mathcal{D}^\infty|$ are in a one-one correspondence with arbitrary infinite sequences $\langle x_n \rangle_{n=0}^\infty$ of elements $x_n \in |\mathcal{D}|$ by using combinations of neighbourhoods

$$0X_0 \cup 10X_1 \cup \cdots \cup 1^n 0X_n \cup \cdots,$$

where from some point on all the X_m are equal to Δ.

le_trans'.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.le
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

/-- Transitivity of the pointwise order. -/
theorem le_trans' {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (f g h : ScottMap D D')
    (hfg : ScottMap.le f g)
    (hgh : ScottMap.le g h) : ScottMap.le f h :=
  fun x => le_trans (hfg x) (hgh x)

end ScottMap

end DomainTheory.ContinuousLattice
```

4

3.4.18 Exercise 3.17 — embedding binary sequences

Using the $\mathcal{B}$ and $\mathcal{T}$ of Example 2.3 show there is a one-one approximable mapping

$$\{f : \mathcal{B} \to \mathcal{T}^\infty\}$$

and another approximable mapping

$$\{g : \mathcal{T}^\infty \to \mathcal{B}\}$$

such that

$$g \circ f = I_{\mathcal{B}} \quad \text{and} \quad f \circ g \subseteq I_{\mathcal{T}^\infty}.$$

Are $\mathcal{B}$ and $\mathcal{T}^\infty$ isomorphic? Are $\mathcal{B}$ and $\mathcal{T} \times \mathcal{B}$ isomorphic?

le_trans'.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.le
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

/-- Transitivity of the pointwise order. -/
theorem le_trans' {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (f g h : ScottMap D D')
    (hfg : ScottMap.le f g)
    (hgh : ScottMap.le g h) : ScottMap.le f h :=
  fun x => le_trans (hfg x) (hgh x)

end ScottMap

end DomainTheory.ContinuousLattice
```

5

3.4.19 Exercise 3.18 — separated sum

Let $\mathcal{D}_0$ and $\mathcal{D}_1$ be neighbourhood systems over Δ_0 and Δ_1, where we again assume that these are subsets of Σ^*. We assume that in addition *no neighbourhood is empty*. Why is this possible without loss of generality?

Define the *sum system* by:

$$\mathcal{D}_0 + \mathcal{D}_1 = \{\{\Lambda\} \cup 0\Delta_0 \cup 1\Delta_1\} \cup \{0X \mid X \in \mathcal{D}_0\} \cup \{1Y \mid Y \in \mathcal{D}_1\}.$$

Prove that this is a neighbourhood system over $\{\Lambda\} \cup 0\Delta_0 \cup 1\Delta_1$. (Throwing in $\{\Lambda\}$ was not all that necessary, but note that $\mathcal{B} = \mathcal{B} + \mathcal{B}$, and this is an equality of sets not just an isomorphism of systems.)

Prove that in general there are mappings

$$in_i : \mathcal{D}_i \to \mathcal{D}_0 + \mathcal{D}_1 \quad \text{and} \quad out_i : \mathcal{D}_0 + \mathcal{D}_1 \to \mathcal{D}_i,$$

where $out_i \circ in_i = I_{\mathcal{D}_i}$. Where does the assumption $\emptyset \notin \mathcal{D}_i$ come in? How can these sums be generalized to n-terms? (Hint: As for products use sets $1^i 0\Delta_i$.) Draw some pictures.

le_trans'.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.le
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

/-- Transitivity of the pointwise order. -/
theorem le_trans' {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (f g h : ScottMap D D')
    (hfg : ScottMap.le f g)
    (hgh : ScottMap.le g h) : ScottMap.le f h :=
  fun x => le_trans (hfg x) (hgh x)

end ScottMap

end DomainTheory.ContinuousLattice
```

6

3.4.20 Exercise 3.19 — induced maps on constructs

Suppose we are given systems and approximable mappings

$$f : \mathcal{D}_0 \to \mathcal{D}_0' \quad \text{and} \quad g : \mathcal{D}_1 \to \mathcal{D}_1'.$$

Prove there are approximable mappings

$$f \times g : \mathcal{D}_0 \times \mathcal{D}_1 \to \mathcal{D}_0' \times \mathcal{D}_1' \quad \text{and} \quad f + g : \mathcal{D}_0 + \mathcal{D}_1 \to \mathcal{D}_0' + \mathcal{D}_1'$$

such that

(i) $(f \times g)(x, y) = \langle f(x), g(y) \rangle$ for all $x \in |\mathcal{D}_0|$ and $y \in |\mathcal{D}_1|$, and rewrite this as:

(ii) $f \times g = \langle f \circ p_0, g \circ p_1 \rangle$.

In addition prove that

(iii) $out_0 \circ (f + g) \circ in_0 = f$, and

(iv) $out_1 \circ (f + g) \circ in_1 = g$.

Do equations (iii) and (iv) uniquely determine $f + g$?

le_trans'.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.le
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

/-- Transitivity of the pointwise order. -/
theorem le_trans' {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (f g h : ScottMap D D')
    (hfg : ScottMap.le f g)
    (hgh : ScottMap.le g h) : ScottMap.le f h :=
  fun x => le_trans (hfg x) (hgh x)

end ScottMap

end DomainTheory.ContinuousLattice
```

7

— Pref-folded `sumMap` companions

le_trans'.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.le
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic
```

```
@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

/-- Transitivity of the pointwise order. -/
theorem le_trans' {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (f g h : ScottMap D D')
    (hfg : ScottMap.le f g)
    (hgh : ScottMap.le g h) : ScottMap.le f h :=
  fun x => le_trans (hfg x) (hgh x)

end ScottMap

end DomainTheory.ContinuousLattice
```

8

3.4.21 Exercise 3.20 — sum and product functors

(For category theorists). Show that the result of 3.19 can be used to prove that $+$ and $\times$ on the category of domains and approximable maps are indeed functors. Show further that $\times$ is the categorical product for this category.

le_trans'.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.le
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v
```

```
variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

/-- Transitivity of the pointwise order. -/
theorem le_trans' {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (f g h : ScottMap D D')
    (hfg : ScottMap.le f g)
    (hgh : ScottMap.le g h) : ScottMap.le f h :=
  fun x => le_trans (hfg x) (hgh x)

end ScottMap

end DomainTheory.ContinuousLattice
```

9

3.4.22 Exercise 3.21 — identity of step neighbourhoods

In the proofs of 3.12 in the definition of curry (g) it is rather cavalierly assumed that the neighbourhood $[Y, Z]$ uniquely determines Y and Z. Show that this is true if $Z \neq \Delta_2$. (Hint: Find explicitly the least of $f \in [Y, Z]$.) Show that if $Z = \Delta_2$ the biconditional stated at the start of the proof is still valid even though Y is not uniquely determined. (Hint: Remember that $\Delta_1\ g\ \Delta_2$ must hold.) For arbitrary pairs of neighbourhoods of $(D_1 \to D_2)$ is there a simple criterion for identity?

lt.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.le
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
```

```
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

/-- Strict pointwise order induced by `ScottMap.le`. -/
def lt {D : Type u} {D' : Type v} [CompleteLattice D] [CompleteLattice D']
    (f g : ScottMap D D') : Prop :=
  ScottMap.le f g ∧ ¬ ScottMap.le g f

end ScottMap

end DomainTheory.ContinuousLattice
```

0

3.4.23 Exercise 3.22 — composition combinator

Prove that there is an approximable mapping

$$\text{comp} : (D_1 \to D_2) \times (D_0 \to D_1) \to (D_0 \to D_2)$$

where for all $g : D_1 \to D_2$ and $f : D_0 \to D_1$ we have

$$\text{comp}(g, f) = g \circ f.$$

Show this directly by writing down the neighbourhood relation and by building the mapping up from eval and curry (on suitable domains) using $\circ$ and $\langle , \rangle$. (Hint: Fill in maps in the following sequence of domains:

1. $(D_0 \to D_1) \times D_0 \to D_1$
2. $(D_1 \to D_2) \times ((D_0 \to D_1) \times D_0) \to (D_1 \to D_2) \times D_1$
3. $((D_1 \to D_2) \times (D_0 \to D_1)) \times D_0 \to (D_1 \to D_2) \times D_1$
4. $((D_1 \to D_2) \times (D_0 \to D_1)) \times D_0 \to D_2$
5. $(D_1 \to D_2) \times (D_0 \to D_1) \to (D_0 \to D_2)$.

The maps are of course not uniquely determined, but the shifting of brackets ought to suggest the right choice.)

lt.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.le
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice
```

```
open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

/-- Strict pointwise order induced by `ScottMap.le`. -/
def lt {D : Type u} {D' : Type v} [CompleteLattice D] [CompleteLattice D']
    (f g : ScottMap D D') : Prop :=
  ScottMap.le f g ∧ ¬ ScottMap.le g f

end ScottMap

end DomainTheory.ContinuousLattice
```

1

3.4.24 Exercise 3.23 — cartesian closed category

(For category theorists.) Show that the results of 3.11 and 3.12 prove that the category of domains and approximable mappings is a *cartesian closed category*. (Mac Lane [1971] pp. 95–96 may be consulted for a very brief introduction.) What is the *terminal domain* in this category? What sort of functor is $(D_0 \to D_1)$?

`lt.lean`

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.le
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
```

```
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

/-- Strict pointwise order induced by `ScottMap.le`. -/
def lt {D : Type u} {D' : Type v} [CompleteLattice D] [CompleteLattice D']
    (f g : ScottMap D D') : Prop :=
  ScottMap.le f g ∧ ¬ ScottMap.le g f

end ScottMap

end DomainTheory.ContinuousLattice
```

2

3.4.25 Exercise 3.24 — function-space isomorphisms

Establish some more isomorphisms :

(i) $(D_0 \to (D_1 \times D_2)) \cong (D_0 \to D_1) \times (D_0 \to D_2)$

(ii) $(D_0 \to D_1^{\infty}) \cong (D_0 \to D_1)^{\infty}$

(iii) $D_0 \times (D_1 + D_2) \cong (D_0 \times D_1) + (D_0 \times D_2)$

(iv) $(D_0 + D_1) \to D_2 \cong (D_0 \to D_2) \times (D_1 \to D_2) \ldots$

If some of the above are not true, perhaps at least some mapping relationships can be established.

lt.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.le
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']
```

```
namespace ScottMap

/-- Strict pointwise order induced by `ScottMap.le`. -/
def lt {D : Type u} {D' : Type v} [CompleteLattice D] [CompleteLattice D']
    (f g : ScottMap D D') : Prop :=
  ScottMap.le f g ∧ ¬ ScottMap.le g f

end ScottMap

end DomainTheory.ContinuousLattice
```

3

3.4.26 Exercise 3.25 — opens form a domain

(For topologists.) Recall from Exercises 1.21 and 2.13 on how to regard a domain $|D|$ as a topological space. Using 3.10 show that the family of open subsets of $|D|$ is isomorphic to a domain.

lt.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.le
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

/-- Strict pointwise order induced by `ScottMap.le`. -/
def lt {D : Type u} {D' : Type v} [CompleteLattice D] [CompleteLattice D']
    (f g : ScottMap D D') : Prop :=
  ScottMap.le f g ∧ ¬ ScottMap.le g f

end ScottMap

end DomainTheory.ContinuousLattice
```

4

3.4.27 Exercise 3.26 — conditional combinator

Show that for every domain D there is an approximable mapping

$$\mathrm{cond} : T \times D \times D \to D,$$

called the *conditional operator*, satisfying

(i) $\mathrm{cond}(\mathrm{true}, x, y) = x$

(ii) $\mathrm{cond}(\mathrm{false}, x, y) = y$

(iii) $\mathrm{cond}(\bot, x, y) = \bot$.

(Hint: Recalling that $T = \{\{0\}, \{1\}, \{0, 1\}\}$, define cond as a relation by

$$\begin{aligned} 0C \cup 10X \cup 110Y \ \mathrm{cond}\ Z \quad \text{iff} \quad & 0 \in C \text{ and } X \subseteq Z \text{ or} \\ & 1 \in C \text{ and } Y \subseteq Z \text{ or} \\ & 0, 1 \in C \text{ and } \Delta \subseteq Z, \end{aligned}$$

where $C \in T$ and $X \in D$ and $Y \in D$ and where we are using the construction of Exercise 3.14.) Find a similar operator in the domain

$$T \times D_0 \times D_1 \to D_0 + D_1.$$

Show also there is an approximable mapping

$$\mathrm{which} : D_0 + D_1 \to T$$

such that for all $x \in |D_0 + D_1|$

$$\mathrm{cond}(\mathrm{which}(x), \mathrm{in}_0(\mathrm{out}_0(x)), \mathrm{in}_1(\mathrm{out}_1(x))) = x.$$

lt.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.le
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]
```

```
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

/-- Strict pointwise order induced by `ScottMap.le`. -/
def lt {D : Type u} {D' : Type v} [CompleteLattice D] [CompleteLattice D']
    (f g : ScottMap D D') : Prop :=
  ScottMap.le f g ∧ ¬ ScottMap.le g f

end ScottMap

end DomainTheory.ContinuousLattice
```

5

3.4.28 Exercise 3.27 — function space via closure

(For set theorists.) Give another proof that the family of approximable mappings $f : D_0 \to D_1$ is isomorphic to a domain by employing the general argument of Exercise 2.22. How does this compare with the proof method of 3.9 and 3.10? Can the general remarks also be employed to show that

$$\text{eval} : (D_1 \to D_2) \times D_1 \to D_2$$

is approximable without bringing in the neighbourhoods in such an explicit way? (Hint: Use 3.5 and the idea of Exercise 2.12.)

lt.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.le
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap
```

```
/-- Strict pointwise order induced by `ScottMap.le`. -/
def lt {D : Type u} {D' : Type v} [CompleteLattice D] [CompleteLattice D']
    (f g : ScottMap D D') : Prop :=
  ScottMap.le f g ∧ ¬ ScottMap.le g f

end ScottMap

end DomainTheory.ContinuousLattice
```

6

3.4.29 Exercise 3.28 — minimal step function

In the function space $(D_0 \to D_1)$ let

$$\bigcap\{[X_i, Y_i] \mid i < n\}$$

be a (non-empty) neighbourhood. In 3.9 the minimal element of this neighbourhood is characterized as a relation f_0. Show that as an elementwise mapping it can be defined by the formula

$$f_0(x) = \bigsqcup\{\uparrow Y_i \mid x \in [X_i]\},$$

for $x \in |D_0|$. Try to draw a picture of $|D_0|$ with neighbourhoods $[X_i]$ and the corresponding values of the function f_0.

LECTURE IV FIXED POINTS AND RECURSION

Having at this point a large supply of examples of domains (and further constructs of new domains), we now have to consider some other ways of defining functions — other than by explicit compositions of the very basic functions already mentioned. One of the most fruitful techniques is an infinitely iterated composition that is at the back of the idea of recursion. We will use the process over and over again in these lectures, not only to define new functions but also to define new domains. The heart of the matter lies in the so-called "Fixed-point Theorem":

lt.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.le
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
```

```
variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

/-- Strict pointwise order induced by `ScottMap.le`. -/
def lt {D : Type u} {D' : Type v} [CompleteLattice D] [CompleteLattice D']
    (f g : ScottMap D D') : Prop :=
  ScottMap.le f g ∧ ¬ ScottMap.le g f

end ScottMap

end DomainTheory.ContinuousLattice
```

7

3.5 Fixed points and recursion

Dependency diagram. Each box is one item. The arrowhead is on the item that imports. An arrow already implied by a longer path is left out. The introduction has no Lean file, so it is not shown. Blue is a definition, amber an example, red a theorem, and indigo an exercise.

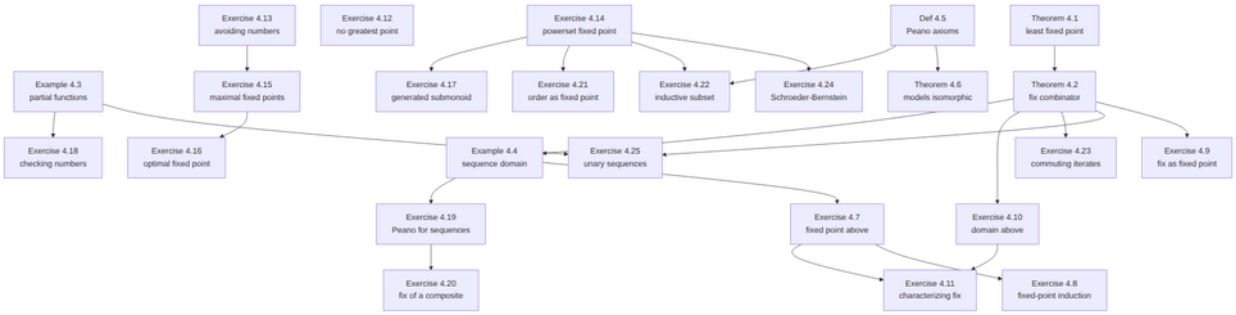


Figure 31: Fixed points and recursion.

3.5.1 Introduction

LECTURE IV FIXED POINTS AND RECURSION

Having at this point a large supply of examples of domains (and further constructs of new domains), we now have to consider some other ways of defining functions — other than by explicit compositions of the very basic functions already mentioned. One of the most fruitful techniques is an infinitely iterated composition that is at the back of the idea of recursion. We will use the process over and over again in these lectures, not only to define new functions but also to define new domains. The heart of the matter lies in the so-called "Fixed-point Theorem":

3.5.2 Theorem 4.1 — least fixed point

THEOREM 4.1. For any approximable mapping $f : D \to D$ on any domain, there exists a least element $x \in |D|$ where

$$f(x) = x.$$

Proof: Let f^n for $n \in \mathbf{N}$ stand for the n-fold composition of f with itself. That is,

$$f^0 = I_D, \text{ and}$$

$$f^{n+1} = f \circ f^n.$$

Define

$$x = \{X \in D \mid \Delta \; f^n \; X, \text{ for some } n \in \mathbf{N}\}.$$

We see $X \in x$ iff there is a finite sequence $\Delta = X_0, X_1, \ldots, X_n = X$ where $X_i \; f \; X_{i+1}$ holds for all $i < n$. Now since $\Delta \; f \; \Delta$ automatically holds, a sequence for an $X \in x$ can always be extended to a longer sequence just by adding more Δ's on the front.

We want to prove $x \in |D|$. Clearly $\Delta \in x$; and if $X \subseteq Y$ and $X \in x$, then $Y \in x$. All that remains to be shown is the closure of x under intersection. Note that if

$$U \; f \; V \quad \text{and} \quad U' \; f \; V'$$

hold and U, U' are consistent in D, then V and V' are consistent and
$(U \cap U') \; f \; (V \cap V')$
must hold. Generalizing this to sequences, if

$$\Delta = X_0 \; f \; X_1 \; f \; \cdots \; f \; X_n = X, \text{ and}$$

$$\Delta = Y_0 \; f \; Y_1 \; f \; \cdots \; f \; Y_n = Y$$

both hold (and note we have arranged the lengths of the two sequences to be equal), then each pair X_i, Y_i is consistent and we have

$$\Delta = (X_0 \cap Y_0) \; f \; (X_1 \cap Y_1) \; f \; \cdots \; f \; (X_n \cap Y_n) = X \cap Y.$$

This establishes the desired closure.

We also note that if $X \in x$ and $X \; f \; Y$ then $Y \in x$. Therefore, $f(x) \subseteq x$ and indeed by its very construction x is the *least* element of $|D|$ with this property. (Why?) But f is monotone, so $f(f(x)) \subseteq f(x)$; hence, $x = f(x)$. By what we have already said it must be the least such element. □

Because the element we have shown to exist in 4.1 is a least element, it is unique. That is, we have associated with each $f : D \to D$ a special element $x_f \in |D|$ determined by the choice of f. A function has therefore been defined mapping the set $|D \to D|$ into $|D|$. The next result shows that this function, or operator on functions, is in fact approximable.

`lt.lean`

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.le
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

/-- Strict pointwise order induced by `ScottMap.le`. -/
def lt {D : Type u} {D' : Type v} [CompleteLattice D] [CompleteLattice D']
    (f g : ScottMap D D') : Prop :=
  ScottMap.le f g ∧ ¬ ScottMap.le g f

end ScottMap

end DomainTheory.ContinuousLattice
```

8

3.5.3 Theorem 4.2 — fix combinator

THEOREM 4.2. For any domain D, there is an approximable mapping

$$\mathrm{fix} : (D \to D) \to D$$

such that if $f : D \to D$ is any approximable mapping, then

(i) $\mathrm{fix}(f) = f(\mathrm{fix}(f))$.

Furthermore, if $x \in |D|$, then

(ii) $f(x) \subseteq x$ implies $\mathrm{fix}(f) \subseteq x$.

And this last property implies that fix is unique. Explicitly we can characterize fix by the equation:

(iii)

$$\mathrm{fix}(f) = \bigcup_{n=0}^{\infty} f^n(\bot),$$

for all $f : D \to D$.

Proof: Formula (iii) can be put in a more elementary form:

$$\mathrm{fix}(f) = \{X \mid \Delta f^n X, \text{ for some } n \in \mathbf{N}\}.$$

To show an elementwise mapping approximable we can use the formula of Exercise 2.9, applied to the above as the definition of fix:

$$(*) \quad \mathrm{fix}(f) = \bigcup\{\mathrm{fix}(\uparrow F) \mid f \in [F]\},$$

where F ranges over the neighbourhoods of $(D \to D)$, and where $\uparrow F$ can be considered to be the least element of F as calculated in 3.9.

Now from the definition of fix, it is clear that whenever $f \subseteq g$, then $\mathrm{fix}(f) \subseteq \mathrm{fix}(g)$, because $f^n \subseteq g^n$. (That is, fix is obviously monotone.) Next, if $f \in F$, then $\uparrow F$ is a (finite) approximation to f; so $\uparrow F \subseteq f$ and $\mathrm{fix}(\uparrow F) \subseteq \mathrm{fix}(f)$. This means that half of equation (*) already holds by monotonicity. All that is left is to prove the other half.

So suppose $X \in \mathrm{fix}(f)$. Then, as we have already remarked, there is a finite sequence of neighbourhoods where

$$\Delta = X_0 \; f \; X_1 \; \ldots \; X_{n-1} \; f \; X_n = X.$$

Let the function-space neighbourhood be defined as

$$F = \bigcap\{[X_i, X_{i+1}] \mid i < n\},$$

and note that since $f \in [F]$ we have at once consistency. But, by 3.9, $\uparrow F \in [F]$, so the *same* sequence of X_i is sufficient to show that

$$X \in \mathrm{fix}(\uparrow F).$$

In other words, if X belongs to the left-hand side of *(), it also belongs to the right-hand side. This completes the proof of ()*.

Formula (i) is just a restatement of what we proved in 4.1. And (ii) follows easily, because $f(x) \subseteq x$ implies that $\Delta \in x$ and whenever $X \in x$ and $X \; f \; Y$, then $Y \in x$. Thus, by induction, if $\Delta f^n X$, then $X \in x$. So $\mathrm{fix}(f) \subseteq x$.

Finally, if $\mathrm{fax} : (D \to D) \to D$ were any other operator satisfying (i) and (ii), we would prove at once that

$$\mathrm{fix}(f) \subseteq \mathrm{fax}(f) \quad \text{and}$$

$$\mathrm{fax}(f) \subseteq \mathrm{fix}(f).$$

That is to say, the two operators are identical. □

The reader may have noticed that we used recursion in the proof of 4.1 (we had to define f^n for all $n \in \mathbf{N}$). But 4.1 and 4.2 can be used to justify definitions by recursion on a large number of domains —

definitions where the process of iteration is far from being as straightforward. In discussing this point, let us start with some basic examples.

lt.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.le
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

/-- Strict pointwise order induced by `ScottMap.le`. -/
def lt {D : Type u} {D' : Type v} [CompleteLattice D] [CompleteLattice D']
    (f g : ScottMap D D') : Prop :=
  ScottMap.le f g ∧ ¬ ScottMap.le g f

end ScottMap

end DomainTheory.ContinuousLattice
```

9

3.5.4 Example 4.3 — partial functions on integers

EXAMPLE 4.3. The infinite generalization of our original example 1.2 is the system

$$N = \{\{n\} \mid n \in \mathbf{N}\} \cup \{\mathbf{N}\}.$$

The total elements are clearly in a one-one correspondence with the integers in $\mathbf{N}$. We can apply the construction of Exercise 3.16 to obtain a domain

$$F = N^{\infty}.$$

So we already know quite a bit about this domain — but it has a much more familiar presentation.

Let Φ be the set of all *finite partial functions* $\varphi \subseteq \mathbf{N} \times \mathbf{N}$ (that is, finite sets of ordered pairs of integers where, if $(n, m) \in \varphi$ and $(n, m') \in \varphi$, then $m = m'$). Define

$$\uparrow \varphi = \{\psi \in \Phi \mid \varphi \subseteq \psi\}.$$

Consider the neighbourhood system

$$F' = \{\uparrow \varphi \mid \varphi \in \Phi\}.$$

It is an easy exercise to show that F and F' are isomorphic and that the elements of these domains correspond exactly to the (possibly infinite) *partial functions* $\pi \subseteq \mathbf{N} \times \mathbf{N}$. Moreover, the *total* elements just correspond to the *total* functions $\tau : \mathbf{N} \to \mathbf{N}$ ("function" in the ordinary, set-theoretical sense of the word).

Another easy exercise is to show that the domains

$$F \text{ and } (\mathbf{N} \to \mathbf{N})$$

by our definitions are *NOT* isomorphic; though the two domains are closely related. We can define a mapping

$$\text{val} : F \times N \to N$$

by the relationship

$$\uparrow \phi \cup \{n\} \text{ val } \{m\} \quad \text{iff} \quad (n, m) \in \phi.$$

(Of course val has to relate other neighbourhoods such as:

$$\uparrow \phi \cup N \text{ val } N,$$

but these are all.) It is then simple to prove that if $\pi \in |F|$ is regarded as a partial function $\pi : N \to N$ and if for $n \in N$ we define $\hat{n} \in |N|$ by

$$\hat{n} = \{\{n\}, N\},$$

then we have

$$\text{val}(\pi, \hat{n}) = \widehat{\pi(n)}, \text{ if } \pi \text{ is defined at } n;$$

$$= \{N\}, \text{ otherwise.}$$

(Remember that $\{N\} \in |N|$ is the "undefined" element.) This means that

$$\text{curry}(\text{val}) : F \to (N \to N)$$

is a one-one function on elements. (The rather slight trouble with $(N \to N)$ is that it has *more* elements than F.)

So much for the construction of F, we now wish to consider mappings

$$f : F \to F$$

and their uses. Consider the possibility

$$f(\pi)(n) = 0, \qquad \text{if } n = 0;$$

$$= \pi(n-1) + n - 1, \text{ if } n > 0.$$

If π were a total function, then $f(\pi)$ would be total. But if π is partial, and if it is, say, undefined at k, then $f(\pi)$ becomes undefined at $k+1$. Note that $f(\pi)$ is always defined at 0. Note, too, that f is an approximable mapping because it is completely determined by what it does to finite (partial) functions. Indeed,

$$f(\pi) = \bigcup\{f(\phi) \mid \phi \subseteq \pi\},$$

where ϕ ranges over Φ.

Well, we have proved that every approximable map of a domain into itself has a (*least*) fixed point. What is the least fixed point of this f? Suppose $\sigma = f(\sigma)$. Then $\sigma(0) = 0$, and

$$\sigma(n+1) = f(\sigma)(n+1)$$

$$= \sigma(n) + n.$$

By induction, then

$$\sigma(n) = \sum_{i<n} i$$

and σ is a total function. (Therefore, f has a *unique* fixed point.)

Actually, we can make the procedure more systematic by defining as fixed points elements of $(N \to N)$ rather than F. In the first place we have $\hat{0} \in |N|$, and from now on we will not distinguish between n and $\hat{n}$. Next we have two mappings:

$$\text{succ}, \text{pred} : N \to N$$

where, as approximable mappings we have

$$X \text{ succ } Y \text{ iff } \exists n \in N.\ n \in X \text{ and } n+1 \in Y,$$

$$X \text{ pred } Y \text{ iff } \exists n \in N.\ n+1 \in X \text{ and } n \in Y,$$

for all $X, Y \in N$. This is *correct*, but what we mean in more understandable terms is:

$$\text{succ}(n) = n + 1;$$

$$\text{pred}(n) = n - 1, \text{ if } n > 0;$$

$$= \perp, \text{ if } n = 0.$$

Here, n has been identified with $\hat{n} \in |N|$ and $\perp = \{N\} \in |N|$. Moreover, we have a mapping

$$\text{zero} : N \to T$$

which is such that

$$\text{zero}(n) = \text{true, if } n = 0;$$

$$= \text{false, if } n > 0.$$

The *structured domain*

$$\langle N, 0, \text{succ}, \text{pred}, \text{zero} \rangle$$

can be called "THE domain of integers" for our present theory. We shall meet many other structured domains in the sequel.

Now the iterated summation function σ can be completely characterized — as a map $\sigma : N \to N$ rather than as an element $\sigma \in |F|$ — by the following equation:

$$\sigma(n) = \text{cond}(\text{zero}(n),\ 0,\ \sigma(\text{pred}(n)) + \text{pred}(n)).$$

The only problem is that we have not defined $+ : N \times N \to N$. (A direct definition is left to the reader; general remarks are given later.) But $+$ could be any function of two variables in order to make the point about the form of the definition of σ. Remember

$$\text{cond} : T \times N \times N \to N,$$

as defined in Exercise 3.26. We do not put cond in as part of the structure of N because (as should be clear from 3.26) it is part of the structure of T.

The above equation for σ is properly called a *functional equation*; it will be written as a fixed-point equation in Lecture V when we have the notation for the λ-calculus. □

`lt_iff_le_not_ge'.lean`

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.lt
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]
```

```
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

/-- The chosen strict pointwise order is `≤ ∧ ¬ ≥`. -/
theorem lt_iff_le_not_ge' {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (f g : ScottMap D D') :
    ScottMap.lt f g ↔
      ScottMap.le f g ∧ ¬ ScottMap.le g f :=
  Iff.rfl

end ScottMap

end DomainTheory.ContinuousLattice
```

0

3.5.5 Example 4.4 — sequence domain

EXAMPLE 4.4. The domain C of finite or infinite binary sequences mentioned in Exercise 2.21 may be regarded as a generalization of N. This can be made plain by saying how we wish to regard C as a structured domain. To do this we should recall what C is as a neighbourhood system. In the first place

$$B = \{\sigma\Sigma^* \mid \sigma \in \Sigma^*\}$$

where $\Sigma = \{0, 1\}$. To form the system C we have

$$C = B \cup \{\{\sigma\} \mid \sigma \in \Sigma^*\}.$$

The total elements of B correspond to *infinite binary sequences*; while the total elements of C to *finite or infinite sequences*. To simplify notation let us write for $\sigma \in \Sigma^*$

$$\sigma = \uparrow \{\sigma\} \quad \text{(a total element);}$$

$$\sigma \perp = \uparrow \sigma\Sigma^* \quad \text{(a partial element).}$$

In other words we identify σ with the corresponding total element in $|C|$.

We wish now to think of C as a structured domain seen as a kind of generalization of N. The empty sequence Λ will play the rôle of $0 \in |N|$; the map succ has two different analogues for C, however. Just as for B we define for $x \in |C|$ and $\sigma \in \Sigma^*$:

$$\sigma x = \{Y \mid \sigma X \subseteq Y \text{ some } X \in x\},$$

where of course now X and Y range over C. It should be checked that $\sigma\tau$ has the right meaning whether we think of $\tau \in \Sigma^*$ or $\tau \in |C|$. The two "successor" mappings we are looking for are

$$x \mapsto 0x \quad \text{and} \quad x \mapsto 1x.$$

All the maps $x \mapsto \sigma x$ can be obtained as compositions of these iterated as many times as needed.

Here are two questions which we now should ask:

What plays the rôle of pred?

The mapping will be called **tail**, and it is characterized by:

$$\mathrm{tail}(0x) = x,$$

$$\mathrm{tail}(1x) = x, \text{ and}$$

$$\mathrm{tail}(\Lambda) = \perp .$$

It is left to the reader to show that **tail** exists as an approximable mapping.

What plays the rôle of zero?

The answer is not unique, because in C there are several distinctions that have to be made; in fact we will define three maps:

$$\mathrm{empty}, \mathrm{zero}, \mathrm{one} : C \to T$$

where the three maps take on truth-values to distinguish various kinds of elements in $|C|$ as follows:

$$\begin{aligned}
\mathrm{empty}\,(\Lambda) &= \mathrm{true},\\
\mathrm{empty}\,(0x) &= \mathrm{false},\\
\mathrm{empty}\,(1x) &= \mathrm{false},\\
\mathrm{zero}\,(\Lambda) &= \mathrm{false}\\
\mathrm{zero}\,(0x) &= \mathrm{true}\\
\mathrm{zero}\,(1x) &= \mathrm{false}\\
\mathrm{one}\,(\Lambda) &= \mathrm{false}\\
\mathrm{one}\,(0x) &= \mathrm{false}\\
\mathrm{one}\,(1x) &= \mathrm{true}.
\end{aligned}$$

Again, it is an exercise to show these are approximable. The structured domain is therefore

$$\langle C, \Lambda, 0, 1, \mathrm{tail}, \mathrm{empty}, \mathrm{zero}, \mathrm{one}\rangle.$$

Note that we have changed the meaning of some of the symbols in passing from N to C. Note too that there is a confusion between 0 as an element and 0 as the map $x \mapsto 0x$. There are just too few symbols! In any case this is only an example and not a philosophy of life, so the reader can be expected not to suffer too much.

An example of a definition of an *element* of $|C|$ by a fixed-point equation is:

$$a = 0\,1\,a.$$

This equation has one and only one solution in $|C|$, the infinite sequence that alternates 0's and 1's. Note that a is also characterized by:

$$a = 0101a.$$

Another element is

$$b = 010\,b,$$

which is quite different from a.
An example of a *map* in $|C \to C|$ has the characterization

$$\begin{aligned} d(\Lambda) &= \Lambda \\ d(0x) &= 00d(x), \quad \text{and} \\ d(1x) &= 11d(x). \end{aligned}$$

We can write:

$$\begin{aligned} d(x) = \mathrm{cond}(&\mathrm{empty}(x),\ \Lambda, \\ &\mathrm{cond}(\mathrm{zero}(x),\ 00d(\mathrm{tail}(x)),\ 11d(\mathrm{tail}(x)))). \end{aligned}$$

As we shall see in due course, this can be regarded as a fixed-point definition of d.
An example of a map in $|C \times C \to C|$ was suggested in 2.21. We can write:

$$\begin{aligned} xy = \mathrm{cond}(&\mathrm{empty}(x),\ y, \\ &\mathrm{cond}(\mathrm{zero}(x),\ 0(\mathrm{tail}(x)\,y),\ 1(\mathrm{tail}(x)\,y))). \end{aligned}$$

It should be checked that this equation exactly characterizes the intended mapping. □

The examples we have given with N and C are examples of definitions of functions by *recursion*. The literal meaning of "recursion" is "running backwards", and a look at the equations for our examples will show that the functions are characterized by giving their values either *outright* (e.g. at 0 or at Λ) or at *earlier* arguments (e.g. at $\mathrm{pred}(x)$ or at $\mathrm{tail}(x)$). The reader should keep in mind that a recursive "definition" is not really a definition in the sense of *explicit definition* but rather is a characterization; a theorem has to be proved to show that such functions exist. Now we have a general definition of domain and a general theorem on fixed points and a general construction of function-space domain; THEREFORE, we know that there are solutions to our equations PROVIDED THAT the variables range over elements of a domain and that the other, given functions that appear in the equations are already known to be approximable (continuous). This proviso is very important, and we shall remark on it time after time.

But, as is well known, recursion also can be done over sets like $\mathbf{N}$, and we should examine now the connection between the familiar kind of recursion and what we are doing over domains. Of course, one simple connection is already provided by the way we regard $\mathbf{N}$ as a subset of N. But there are other useful connections that can be employed in a way that may seem more direct.

`lt_iff_le_not_ge'.lean`

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.lt
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace
```

```
universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

/-- The chosen strict pointwise order is `≤ ∧ ¬ ≥`. -/
theorem lt_iff_le_not_ge' {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (f g : ScottMap D D') :
    ScottMap.lt f g ↔
      ScottMap.le f g ∧ ¬ ScottMap.le g f :=
  Iff.rfl

end ScottMap

end DomainTheory.ContinuousLattice
```

1

3.5.6 Definition 4.5 — Peano axioms

DEFINITION 4.5. A structured set $\langle \mathbf{N}, 0, {}^{+} \rangle$, where $0 \in \mathbf{N}$ and ${}^{+} : \mathbf{N} \to \mathbf{N}$ is a unary function, is said to be a *model for Peano's Axioms* if the following conditions are satisfied:

(i) $0 \neq n^{+}$, for all $n \in \mathbf{N}$;

(ii) $n^{+} = m^{+}$ implies $n = m$, for all $n, m \in \mathbf{N}$;

(iii) whenever $x \subseteq \mathbf{N}$ and $0 \in x$ and $x^{+} \subseteq x$, then $x = \mathbf{N}$.

Here $x^{+} = \{n^{+} \mid n \in x\}$. □

Clause (iii) is recognized as the principle of mathematical induction stated in terms of sets. We usually think of $\mathbf{N}$ as being "God given", and (i)–(iii) as known without question. Suppose God, however, decides to withdraw His set of integers and substitute another. We can ask: "Oh! Why did You take from us our beloved numbers? Why must we now live with these strange new beasts?" God will probably reply "Trust Me!" Perhaps we should in view of the theorem:

`lt_iff_le_not_ge'.lean`

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.lt
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section
```

```
namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

/-- The chosen strict pointwise order is `≤ ∧ ¬ ≥`. -/
theorem lt_iff_le_not_ge' {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (f g : ScottMap D D') :
    ScottMap.lt f g ↔
      ScottMap.le f g ∧ ¬ ScottMap.le g f :=
  Iff.rfl

end ScottMap

end DomainTheory.ContinuousLattice
```

2

3.5.7 Theorem 4.6 — Peano models isomorphic

THEOREM 4.6. All models of Peano's Axioms are isomorphic.

Proof: There are several ways to give the proof, but, for the sake of illustration, an application of the fixed-point theorem is appropriate here. Let $\langle \mathbf{N}, 0, {}^{+} \rangle$ be one model, and let $\langle \mathbf{M}, \Box, {}^{\#} \rangle$ be another. Let $\mathbf{N} \times \mathbf{M}$ be the ordinary cartesian product of the two sets and let

$$P(\mathbf{N} \times \mathbf{M})$$

be the powerset (set of all subsets) of $\mathbf{N} \times \mathbf{N}$. As in Exercises 1.15 and 2.20, we regard this set of elements as a domain, whose finite elements are just the finite subsets of the given set $\mathbf{N} \times \mathbf{M}$. The following mapping on $u \subseteq \mathbf{N} \times \mathbf{M}$ is easily proved approximable:

$$u \mapsto \{(0, \Box)\} \cup \{(n^{+}, m^{\#}) \mid (n, m) \in u\}.$$

(This assertion should be checked as an exercise.) We thus let r be the (least) fixed point:

$$r = \{(0, \Box)\} \cup \{(n^{+}, m^{\#}) \mid (n, m) \in r\}.$$

This $r \subseteq \mathbf{N} \times \mathbf{M}$ as a binary relation will turn out to be a one-one correspondence giving the required isomorphism.

First of all we see by construction that

(i) $0\ r\ \square$;

(ii) $n\ r\ m$ implies $n^+\ r\ m^\#$.

So, if r proves to be a one-one correspondence, it will then be the desired isomorphism. Now, the two sets shown in the equation

$$\{(0, \square)\} \cap \{(n^+, m^\#) \mid (n, m) \in r\} = \emptyset$$

are disjoint by virtue of axiom 4.5(i). Therefore, 0 in $\mathbf{N}$ corresponds by r to one and only one element of $\mathbf{M}$, namely the element $\square$. Let $x \subseteq \mathbf{N}$ be the set of all elements of $\mathbf{N}$ corresponding by r to a unique element of $\mathbf{M}$. We have just shown $0 \in x$. Suppose $n \in x$, and let $m \in \mathbf{M}$ be the unique element with $n\ r\ m$. Now $n^+\ r\ m^\#$ holds, so n^+ corresponds to at least one element of $\mathbf{M}$. If $n^+\ r\ k$ also holds, then since $(n^+, k) \neq (0, \square)$, the fixed-point equation implies

$$n^+ = n_0^+ \quad \text{and} \quad k = m_0^\#$$

for some $(n_0, m_0) \in r$. By axiom 4.5(ii), $n = n_0$, and, by uniqueness (remember $n \in x$), $m = m_0$; thus, $m^\#$ is the unique correspondent for n^+. We have proved $n^+ \in x$. Therefore, $x^+ \subseteq x$; so by axiom 4.5(iii), $x = \mathbf{N}$ holds. Otherwise said, every element in $\mathbf{N}$ corresponds to a unique element of $\mathbf{M}$.

Note that the rôles of $\mathbf{N}$ and $\mathbf{M}$ are completely symmetric, *and* they satisfy the same axioms as structured sets. It follows, then, that every element of $\mathbf{M}$ corresponds to a unique element of $\mathbf{N}$. The proof that r is a one-one correspondence is now complete. $\square$

`lt_iff_le_not_ge'.lean`

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.lt
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

/-- The chosen strict pointwise order is `≤ ∧ ¬ ≥`. -/
theorem lt_iff_le_not_ge' {D : Type u} {D' : Type v}
```

```
    [CompleteLattice D] [CompleteLattice D'] (f g : ScottMap D D') :
    ScottMap.lt f g ↔
      ScottMap.le f g ∧ ¬ ScottMap.le g f :=
  Iff.rfl

end ScottMap

end DomainTheory.ContinuousLattice
```

3

3.5.8 Lecture IV exercises

EXERCISES

3.5.9 Exercise 4.7 — fixed point above a seed

Formula 4.2(iii) shows how to find the *least* fixed point of $f : D \to D$. Suppose on the other hand that $a \in |D|$ is such that $a \subseteq f(a)$. Will there be a fixed point $x = f(x)$ with $a \subseteq x$?

(Hint: How do we know $\bigsqcup_{n=0}^{\infty} f^n(\bot) \in |D|$?)

lt_iff_le_not_ge'.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.lt
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

/-- The chosen strict pointwise order is `≤ ∧ ¬ ≥`. -/
theorem lt_iff_le_not_ge' {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (f g : ScottMap D D') :
    ScottMap.lt f g ↔
      ScottMap.le f g ∧ ¬ ScottMap.le g f :=
  Iff.rfl
```

```
end ScottMap

end DomainTheory.ContinuousLattice
```

4

3.5.10 Exercise 4.8 — fixed-point induction

Suppose $f : D \to D$ and $S \subseteq |D|$ are such that

(i) $\bot \in S$;

(ii) $x \in S$ always implies $f(x) \in S$;

(iii) whenever $\{x_n\}_{n=0}^{\infty} \subseteq S$ and $x_n \sqsubseteq x_{n+1}$ for all n, then $\bigsqcup_{n=0}^{\infty} x_n \in S$.

Conclude that $\mathrm{fix}(f) \in S$. (This could be called the principle of *fixed-point induction*.) Apply the method to a set of the form

$$S = \{x \in |D| \mid a(x) = b(x)\},$$

where $a, b : D \to D$ are approximable, and where we know $a(\bot) = b(\bot)$, and $f \circ a = a \circ f$ and $f \circ b = b \circ f$.

lt_iff_le_not_ge'.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.lt
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

/-- The chosen strict pointwise order is `≤ ∧ ¬ ≥`. -/
theorem lt_iff_le_not_ge' {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (f g : ScottMap D D') :
```

```
    ScottMap.lt f g ↔
      ScottMap.le f g ∧ ¬ ScottMap.le g f :=
  Iff.rfl

end ScottMap

end DomainTheory.ContinuousLattice
```

5

3.5.11 Exercise 4.9 — fix as a fixed point

Show that there is an approximable operator

$$\Psi : ((D \to D) \to D) \to ((D \to D) \to D)$$

such that for $\theta : (D \to D) \to D$ and $f : D \to D$ we have

$$\Psi(\theta)(f) = f(\theta(f)).$$

Prove further that $\mathrm{fix} : (D \to D) \to D$ is the *least* fixed point of Ψ.

lt_iff_le_not_ge'.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.lt
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

/-- The chosen strict pointwise order is `≤ ∧ ¬ ≥`. -/
theorem lt_iff_le_not_ge' {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (f g : ScottMap D D') :
    ScottMap.lt f g ↔
      ScottMap.le f g ∧ ¬ ScottMap.le g f :=
  Iff.rfl
```

```
end ScottMap

end DomainTheory.ContinuousLattice
```

6

3.5.12 Exercise 4.10 — domain above an element

Given a domain D and an element $a \in |D|$, construct a domain D_a where

$$|D_a| = \{x \in |D| \mid x \sqsubseteq a\}.$$

Show that if $f : D \to D$ is approximable, then f can be restricted to an approximable map $f' : D_{\mathrm{fix}(f)} \to D_{\mathrm{fix}(f)}$ where $f'(x) = f(x)$ for all $x \in |D_{\mathrm{fix}(f)}|$. How many fixed points does f' have in $|D_{\mathrm{fix}(f)}|$?

lt_iff_le_not_ge'.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.lt
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

/-- The chosen strict pointwise order is `≤ ∧ ¬ ≥`. -/
theorem lt_iff_le_not_ge' {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (f g : ScottMap D D') :
    ScottMap.lt f g ↔
      ScottMap.le f g ∧ ¬ ScottMap.le g f :=
  Iff.rfl

end ScottMap

end DomainTheory.ContinuousLattice
```

7

3.5.13 Exercise 4.11 — characterizing the fix combinator

(Suggested by G. Plotkin). We can regard fix as assigning a fixed-point operator to each domain D. Show that fix is uniquely determined by the following general conditions on an assignment $D \rightsquigarrow F_D$:

(i) $F_D : (D \to D) \to D$;

(ii) $F_D(f) = f(F_D(f))$ for all $f : D \to D$;

(iii) whenever $f_0 : D_0 \to D_0$ and $f_1 : D_1 \to D_1$ are given and $h : D_0 \to D_1$ is such that $h(\bot) = \bot$ and $h \circ f_0 = f_1 \circ h$, then $h(F_{D_0}(f_0)) = F_{D_1}(f_1)$.

(Hint: Apply 4.7 to prove fix satisfies (iii). In the other direction use 4.10.)

lt_iff_le_not_ge'.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.lt
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

/-- The chosen strict pointwise order is `≤ ∧ ¬ ≥`. -/
theorem lt_iff_le_not_ge' {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (f g : ScottMap D D') :
    ScottMap.lt f g ↔
      ScottMap.le f g ∧ ¬ ScottMap.le g f :=
  Iff.rfl

end ScottMap

end DomainTheory.ContinuousLattice
```

8

3.5.14 Exercise 4.12 — no greatest fixed point

Need an approximable $f : D \to D$ have a maximum fixed point? Give an example where there are many fixed points.

lt_iff_le_not_ge'.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.lt
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

/-- The chosen strict pointwise order is `≤ ∧ ¬ ≥`. -/
theorem lt_iff_le_not_ge' {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (f g : ScottMap D D') :
    ScottMap.lt f g ↔
      ScottMap.le f g ∧ ¬ ScottMap.le g f :=
  Iff.rfl

end ScottMap

end DomainTheory.ContinuousLattice
```

9

3.5.15 Exercise 4.13 — avoiding the natural numbers

The proof of 4.1 uses the integers, whereas the proof of 4.6 uses 4.1. There is a hint of circularity here! It can be eliminated by the following steps:

(1) If a domain D has an element a where, for $f : D \to D$, the relation $f(a) \sqsubseteq a$ holds, then the least fixed point can be defined by

$$\mathrm{fix}(f) = \bigcap \{x \in |D| \mid f(x) \sqsubseteq x\}.$$

Note that $\mathrm{fix}(f) \sqsubseteq a$. (Hint: Remark that by 1.17 the formula gives a well-defined element. Call the element b. Prove that $f(b) \sqsubseteq b$ by showing that $f(b) \sqsubseteq x$ whenever $f(x) \sqsubseteq x$. Then note that $f(f(b)) \sqsubseteq f(b)$ so that $b \sqsubseteq f(b)$ also. Conclude $b = \mathrm{fix}(f)$ as least fixed point.)

(2) Remark that this proof uses only the monotonicity property of $f : |D| \to |D|$. Remark, too, that (1) can always be applied to power-set domains PA for any set A.

(3) Review the proof of 4.6 and establish by a fixed-point method that for any structured set $\langle Z, z, \cdot \rangle$ there is a unique function $s : \mathbf{N} \to Z$ such that

(i) $s(0) = z$;

(ii) $s(n^+) = s(n)^{\cdot}$, for $n \in \mathbf{N}$.

(4) Employ (3) for the proof of 4.1 by identifying $\langle \mathbf{N}, 0, {}^+ \rangle$.

mem_finsetSupOf_of_mem.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.finsetSupOf
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

theorem mem_finsetSupOf_of_mem {S : Set D} {x : D} (hx : x ∈ S) :
    x ∈ finsetSupOf S :=
  ⟨{x}, by simp [hx], by simp⟩

end IsContinuousLatticeProjection

end Proposition310

end DomainTheory.ContinuousLattice
```

0

3.5.16 Exercise 4.14 — greatest powerset fixed point

Need a monotone function $f : PA \to PA$ always have a *maximum* fixed point?

mem_finsetSupOf_of_mem.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.finsetSupOf
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

theorem mem_finsetSupOf_of_mem {S : Set D} {x : D} (hx : x ∈ S) :
    x ∈ finsetSupOf S :=
  ⟨{x}, by simp [hx], by simp⟩

end IsContinuousLatticeProjection

end Proposition310

end DomainTheory.ContinuousLattice
```

1

3.5.17 Exercise 4.15 — maximal fixed points

(For set theorists.) Let $f : |D| \to |D|$ be a monotone function on (the elements of) a domain. Show that f has a *maximal* fixed point (i.e. a fixed point that cannot be extended to a larger fixed point). (Hint: By Zorn's Lemma consider a maximal chain

$$C \subseteq \{x \in |D| \mid x \sqsubseteq f(x)\},$$

and use 2.11 to remark that $\bigsqcup C \in |D|$.) Now argue that f has a least fixed point.

mem_finsetSupOf_of_mem.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.finsetSupOf
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

theorem mem_finsetSupOf_of_mem {S : Set D} {x : D} (hx : x ∈ S) :
    x ∈ finsetSupOf S :=
  ⟨{x}, by simp [hx], by simp⟩

end IsContinuousLatticeProjection

end Proposition310

end DomainTheory.ContinuousLattice
```

2

3.5.18 Exercise 4.16 — optimal fixed point

(For fixed-point nuts). Show that a monotone function as in 4.15 has an "optimal" fixed point in the sense that it is the greatest fixed point below all the maximal fixed points and at the same time it is the largest fixed point consistent with all other fixed points. *Consistency* for sets of elements means having a common upper bound. (Hint: Follow these steps:

(1) Show that any non-empty set S of fixed points has a largest fixed point below by using the formula

$$f(\bigsqcap S) \sqsubseteq \bigsqcap S$$

and finding the least fixed point over $\bigsqcap S$.

(2) Letting a be the fixed point of (1) constructed from the set of maximal fixed points, remark that a is consistent with any other fixed point $x = f(x)$, since x can be extended to a maximal one. Suppose b is consistent with all fixed points, then $b \sqsubseteq y$ if y is maximal. (Why?).)

mem_finsetSupOf_of_mem.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.finsetSupOf
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

theorem mem_finsetSupOf_of_mem {S : Set D} {x : D} (hx : x ∈ S) :
    x ∈ finsetSupOf S :=
  ⟨{x}, by simp [hx], by simp⟩

end IsContinuousLatticeProjection

end Proposition310

end DomainTheory.ContinuousLattice
```

3

3.5.19 Exercise 4.17 — generated submonoid

(For algebraists). Suppose $\langle S, 1, \cdot \rangle$ is a semi-group with unit (sometimes called a monoid). Remark that PS is a domain. For $a, b \in S$, what is the least $x \in PS$ such that

$$x = \{1\} \cup \{a, b\} \cup x \cdot x,$$

where in general for $x, y \subseteq S$

$$x \cdot y = \{t \cdot u \mid t \in x \text{ and } u \in y\}?$$

Need the fixed point be unique?

mem_finsetSupOf_of_mem.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.finsetSupOf
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

theorem mem_finsetSupOf_of_mem {S : Set D} {x : D} (hx : x ∈ S) :
    x ∈ finsetSupOf S :=
  ⟨{x}, by simp [hx], by simp⟩

end IsContinuousLatticeProjection

end Proposition310

end DomainTheory.ContinuousLattice
```

4

3.5.20 Exercise 4.18 — checking the number domain

In Example 4.3 there are many unproved assertions about N and F. These should be checked. In particular, the isomorphism theorem of 4.6 could be proved by constructing a simple domain M from $\mathbf{M}$ in the way N is constructed from $\mathbf{N}$.

mem_finsetSupOf_of_mem.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.finsetSupOf
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice
```

```
open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

theorem mem_finsetSupOf_of_mem {S : Set D} {x : D} (hx : x ∈ S) :
    x ∈ finsetSupOf S :=
  ⟨{x}, by simp [hx], by simp⟩

end IsContinuousLatticeProjection

end Proposition310

end DomainTheory.ContinuousLattice
```

5

3.5.21 Exercise 4.19 — Peano axioms for sequences

There are many unproved assertions in Example 4.4! In particular discuss "Peano's Axioms" for $\{0,1\}^*$. Show, moreover, that $\mathrm{one} : C \to T$ can be defined from the rest of the structure by a fixed-point equation.

mem_finsetSupOf_of_mem.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.finsetSupOf
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']
```

```
section Proposition310

open Finset

namespace IsContinuousLatticeProjection

theorem mem_finsetSupOf_of_mem {S : Set D} {x : D} (hx : x ∈ S) :
    x ∈ finsetSupOf S :=
  ⟨{x}, by simp [hx], by simp⟩

end IsContinuousLatticeProjection

end Proposition310

end DomainTheory.ContinuousLattice
```

6

3.5.22 Exercise 4.20 — fix of a composite

For approximable $f, g : D \to D$ prove that

$$\mathrm{fix}(f \circ g) = f(\mathrm{fix}(g \circ f)).$$

mem_finsetSupOf_of_mem.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.finsetSupOf
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

theorem mem_finsetSupOf_of_mem {S : Set D} {x : D} (hx : x ∈ S) :
```

```
    x ∈ finsetSupOf S :=
  ⟨{x}, by simp [hx], by simp⟩

end IsContinuousLatticeProjection

end Proposition310

end DomainTheory.ContinuousLattice
```

7

3.5.23 Exercise 4.21 — order as a fixed point

Show that the less-than-or-equal-to relation $\ell \subseteq \mathbf{N} \times \mathbf{N}$ is uniquely determined by the fixed point equation

$$\ell = \{(n, n) \mid n \in \mathbf{N}\} \cup \{(n, m^+) \mid (n, m) \in \ell\}.$$

Consider the structured set $\langle P\mathbf{N}, \mathbf{N}, {}^+\rangle$ where, as before, $x^+ = \{n^+ \mid n \in x\}$.

What is the unique function $[\cdot] : \mathbf{N} \to P\mathbf{N}$ given by 4.13(3)? Prove that the structures $\langle \mathbf{N}, 0, {}^+\rangle$ and $\langle [m], m, {}^+\rangle$ are uniquely isomorphic for each $m \in \mathbf{N}$, and connect the isomorphism with ordinary addition of integers. Can the same be done for multiplication? (Hint: Consider the fixed-point equation

$$n \cdot \mathbf{N} = \{0\} \cup \{n + m \mid m \in n \cdot \mathbf{N}\},$$

where $n \in \mathbf{N}$ is fixed.)

mem_finsetSupOf_of_mem.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.finsetSupOf
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection
```

```
theorem mem_finsetSupOf_of_mem {S : Set D} {x : D} (hx : x ∈ S) :
    x ∈ finsetSupOf S :=
  ⟨{x}, by simp [hx], by simp⟩

end IsContinuousLatticeProjection

end Proposition310

end DomainTheory.ContinuousLattice
```

8

3.5.24 Exercise 4.22 — inductive subset of a model

Suppose $\mathbf{N}^*$ is a structured set satisfying only axioms (i) and (ii) of 4.5. Must there be a subset $\mathbf{N} \subseteq \mathbf{N}^*$ that satisfies (i), (ii), and (iii)? (Hint: Use a least fixed point in $P\mathbf{N}^*$.) (For set theorists): How do we know from the axioms of set theory that there exists such a set $\mathbf{N}^*$?

mem_finsetSupOf_of_mem.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.finsetSupOf
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

theorem mem_finsetSupOf_of_mem {S : Set D} {x : D} (hx : x ∈ S) :
    x ∈ finsetSupOf S :=
  ⟨{x}, by simp [hx], by simp⟩

end IsContinuousLatticeProjection

end Proposition310

end DomainTheory.ContinuousLattice
```

9

3.5.25 Exercise 4.23 — commuting with the iterates

(Suggested by S. Eilenberg). Suppose $f : \mathcal{D} \to \mathcal{D}$ is approximable on a given domain $\mathcal{D}$. Suppose $a_n : \mathcal{D} \to \mathcal{D}$ is a sequence of approximable maps where

(i) $a_0(x) = \bot$, for all $x \in |\mathcal{D}|$;

(ii) $a_n \sqsubseteq a_{n+1}$ in $\mathcal{D} \to \mathcal{D}$, for all $n \in \mathbf{N}$;

(iii) $\bigsqcup_{n=0}^{\infty} a_n = I_{\mathcal{D}}$ in $\mathcal{D} \to \mathcal{D}$;

(iv) $a_{n+1} \circ f = a_{n+1} \circ f \circ a_n$, for all $n \in \mathbf{N}$.

Prove that f has a unique fixed point. (Hint: Show that if $x = f(x)$, then $a_n(x) \sqsubseteq a_n(\mathrm{fix}(f))$ for all $n \in \mathbf{N}$ by induction on n.)

monotone.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Order.BourbakiWitt
public import DomainTheory.ContinuousLattice.ScottMaps.proposition_2_5
public import DomainTheory.ContinuousLattice.ScottMaps.preservesDirectedSup_monotone

@[expose] public section

namespace DomainTheory.ContinuousLattice

variable {D D' : Type*} [CompleteLattice D] [CompleteLattice D']

namespace ScottMap

theorem monotone (f : ScottMap D D') : Monotone (f : D → D') :=
  preservesDirectedSup_monotone ((proposition_2_5 (Subtype.val f)).mp f.property)

end ScottMap

end DomainTheory.ContinuousLattice
```

0

3.5.26 Exercise 4.24 — Schroeder-Bernstein theorem

(For set theorists). Let $f : A \to B$ and $g : B \to A$ be one-one functions (into, not necessarily onto!) Prove the Schroeder - Bernstein theorem to the effect that there exists a one-one correspondence $h : A \leftrightarrow B$. (Hint: (Suggested by A. Tarski). By the fixed-point theorem find $X \subseteq A$ where

$$X = (A - g(B)) \cup g(f(X)),$$

where $f(X) =$ the image of the set X under the function f. Define $h \subseteq A \times B$ as a union of two restrictions:

$$h = f \restriction X \cup g^{-1} \restriction (A - X).$$

A picture helps.)

monotone.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Order.BourbakiWitt
public import DomainTheory.ContinuousLattice.ScottMaps.proposition_2_5
public import DomainTheory.ContinuousLattice.ScottMaps.preservesDirectedSup_monotone

@[expose] public section

namespace DomainTheory.ContinuousLattice

variable {D D' : Type*} [CompleteLattice D] [CompleteLattice D']

namespace ScottMap

theorem monotone (f : ScottMap D D') : Monotone (f : D → D') :=
  preservesDirectedSup_monotone ((proposition_2_5 (Subtype.val f)).mp f.property)

end ScottMap

end DomainTheory.ContinuousLattice
```

1

3.5.27 Exercise 4.25 — unary sequence domain

Perhaps the domains $\mathbf{N}$ and C are not exactly analogous? C was based on $\{0, 1\}$ as the underlying set of tokens. Construct a system C_1 based on $\{1\}^*$ (= finite strings of 1's) with neighbourhoods:

$$C_1 = \{\, \{1^m \mid m > n\} \mid n \in \mathbf{N} \,\} \cup \{\, \{1^n\} \mid n \in \mathbf{N} \,\}.$$

What structure should be put on C_1 strictly analogous to that on $C(= C_2)$? What kinds of approximable maps relate $\mathbf{N}$, C_1, and C_2? Draw some pictures.

monotone.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Order.BourbakiWitt
public import DomainTheory.ContinuousLattice.ScottMaps.proposition_2_5
public import DomainTheory.ContinuousLattice.ScottMaps.preservesDirectedSup_monotone

@[expose] public section

namespace DomainTheory.ContinuousLattice

variable {D D' : Type*} [CompleteLattice D] [CompleteLattice D']
```

```
namespace ScottMap

theorem monotone (f : ScottMap D D') : Monotone (f : D → D') :=
  preservesDirectedSup_monotone ((proposition_2_5 (Subtype.val f)).mp f.property)

end ScottMap

end DomainTheory.ContinuousLattice
```

2

3.6 Typed λ-calculus

Dependency diagram. Each box is one item. The arrowhead is on the item that imports. An arrow already implied by a longer path is left out. The introduction has no Lean file, so it is not shown. Blue is a definition, green a proposition, amber a table, red a theorem, and indigo an exercise.



Figure 32: Typed λ-calculus.

3.6.1 Introduction

LECTURE V TYPED λ - CALCULUS

In Examples 4.3 and 4.4, after suitable domains have been constructed, functions are characterized by recursion equations whose form of expression is — basically — a composition or substitution of known functions together with the function to be defined. This method can be made more precise and more easily usable by expanding our notation for functions — particularly by inventing a "temporary" notation for a function as a thing in itself without having to have special letters for functions. The device is called λ-*abstraction*. It is related to ordinary set abstraction (the $\{x \mid \ldots\}$ — notation already much used in these lectures), but we gear the approach to domains and their elements, and especially to function spaces.

At this stage it would not be so helpful to produce a rigorously formal definition of the syntax of the typed λ-calculus; we shall try to suggest what is needed by example. There are so many examples at hand, the less formal discussion ought to be sufficient.

In the first place we should set aside, in the notational store room as it were, a stock of variables

$$x, y, z, w, \ldots$$

These variables will be required in different "sizes" or "types". Roughly speaking there should be an infinite number of variables to range over the elements of each domain $\mathcal{D}$. We could perhaps write

$$x_0^{\mathcal{D}}, x_1^{\mathcal{D}}, x_2^{\mathcal{D}}, \ldots,$$

but the subscripts to insure an infinity of variables and the superscripts to record the typing of the variables lead to a notation as

tiresome to write as it is to read. We simply agree that we can have as many variables as we need and that they come in all the types.

Strictly speaking we should also introduce *type symbols* and not confuse types with domains. But if the reader will simply keep in mind that *form* in language has always to be kept distinct from *content*, the confusion at the type level will not matter so very much. A point at which the confusion might cause a real confusion concerns *compound types*. Given $\mathcal{D}_0$ and $\mathcal{D}_1$ we can form such compounds as

$$\mathcal{D}_0 + \mathcal{D}_1, \quad \mathcal{D}_0 \times \mathcal{D}_1, \quad \mathcal{D}_0 \to \mathcal{D}_1$$

What has to be remembered is that a compound domain (neighbourhood system), $\mathcal{D}_0 \times \mathcal{D}_1$ say, does not uniquely determine the "parts" $\mathcal{D}_0$ and $\mathcal{D}_1$. (We could make it do so, but it would cost some effort.) Of course, the symbol "$\mathcal{D}_0 \times \mathcal{D}_1$" has well defined parts. The point is that *different* ways of forming a compound domain could lead to the *same* result, meaning that a domain does not let us retrace its exact history of construction. Compound symbols, however, always carry their histories around with them, since otherwise they would not be readable. What we want, of course, are *both* domain symbols *and* domains, the latter being the *meanings* of the former. Most of the time we can happily pretend that it is only the domains themselves we have to think about.

Besides variables, we will also need certain *constants*. For instance, the symbol 0 (perhaps, better $0^{\mathbf{N}}$) denotes a certain element of $|\mathbf{N}|$. Similarly, in view of Theorem 4.2, for each domain $\mathcal{D}$ there is a well-determined element $\mathrm{fix}^{\mathcal{D}}$ of the compound type $((\mathcal{D} \to \mathcal{D}) \to \mathcal{D})$ denoting the least fixed-point operator. We have considered any number of similar constants of a great variety of types already (cf. 4.3 and 4.4; cond is an especially good one). We can say that the variables and constants are *atomic terms*, where "atomic" here means non-compound.

To form compound terms, there are several means: for example, if $\tau, \ldots, \sigma$ is a list of already obtained terms (including variables or constants), then we can form an ordered *tuple*

We have already done so in 3.1. If the types of $\tau, \ldots, \sigma$ are $D, \ldots, D'$, respectively, then the type of the tuple is the product domain

$$D \times \cdots \times D',$$

because we intend that the tuple denote an element of this domain. (The tuple notation for *functions* as in 3.3 is being forgotten for the time being.)

Next suppose that τ has type $(D_0 \to D_1)$ and σ has type D_0, then the usual *function-value* notation

$$\tau(\sigma)$$

is a compound term of type D_1. We also use

$$\tau(\sigma_0, \ldots, \sigma_{n-1})$$

as an abbreviation of

$$\tau(\langle \sigma_0, \ldots, \sigma_{n-1} \rangle),$$

where, if the types of $\sigma_0, \ldots, \sigma_{n-1}$ are $D_0, \ldots, D_{n-1}$, then the type of τ has to be of the form

$$((D_0 \times \cdots \times D_{n-1}) \to D_n)$$

where D_n is the type of the compound. In this manner, with functions applied to tuples, we have the full facility of substitution into functions of many variables just by iterating the notation.

Having taken into account *function value*, it remains to provide for *function definition*. Suppose that $x_0, \ldots, x_{n-1}$ is a list of distinct variables of types $D_0, \ldots, D_{n-1}$. Suppose further that τ is a term — no matter how complicated — of type D_n. Then we can regard τ as defining a function of n-variables of type

$$((D_0 \times \cdots \times D_{n-1}) \to D_n).$$

What we have not done is to reward our regard by, as yet, providing a quick-to-write "name" for that function. This we now do; it is called

$$\lambda x_0, \ldots, x_{n-1} \cdot \tau,$$

where we stress that the x_i must be *distinct* variables and that this
expression denotes the *whole function*. That is why we provide it with a special symbol.
Here is an example of the λ - notation

$$\lambda x, y.x,$$

which is read "lambda ex wye ... (pause) ... ex". If the types of x and y are D_0 and D_1, then the type of the above is

$$((D_0 \times D_1) \to D_0).$$

Indeed, we know this function very well: it is the *first projection function* p_0 of 3.3 and the equation

$$p_0 = \lambda x, y.x$$

is true, as is the equation

$$p_1 = \lambda x, y.y.$$

In the notation of 3.3, we also find the true equation

$$\langle f, g \rangle = \lambda w.\langle f(w), g(w) \rangle,$$

where on the right-hand side we are using "official" λ - notation for a function of type

$$(D_2 \to (D_0 \times D_1)).$$

The notation on the left is just an *abbreviation* and it should not be confused with the pair (2-tuple) of type

$$((D_2 \to D_0) \times (D_2 \to D_1)).$$

(Since the two domains just mentioned are *isomorphic*, the possible confusion is not all that serious. On the other hand, one confusion we will completely overlook is that between 1-tuples $\langle x \rangle$ and elements x. Strictly speaking they are different, but we shall not bother to make the distinction.)

Here are some other examples of true equations:

$$\text{eval} = \lambda f, x.f(x) \qquad \text{(cf. 3.11)}$$

$$\text{curry} = \lambda g \lambda x \lambda y . g(x, y) \qquad \text{(cf. 3.12)}$$

The first should be immediately clear; while the second is particularly *instructive*. What is being illustrated is that the λ - notation can

be *iterated*. The distinction being drawn is between

$$\lambda x_0, x_1, \ldots, x_{n-1} . \tau \quad \text{and} \quad \lambda x_0 \lambda x_1 \ldots \lambda x_{n-1} . \tau .$$

The first has type

$$((D_0 \times D_1 \times \cdots \times D_{n-1}) \to D_n);$$

while the second has type

$$(D_0 \to (D_1 \to (\ldots (D_{n-1} \to D_n) \ldots))).$$

This is related also to the true equation

$$\text{curry}(\lambda x, y . \tau) = \lambda x \lambda y . \tau,$$

which shows that there are operators relating to the two notations. The first is the *multivariate* form; the second is the *curried* form.

Here is another true equation

$$\text{fix} = \text{fix}(\lambda F \lambda f . f(F(f))),$$

where the fix on the left has type $((D \to D) \to D)$ and that on the right type

$$((((D \to D) \to D) \to ((D \to D) \to D)) \to ((D \to D) \to D)).$$

This is the content of Exercise 4.9. (This also shows why type superscripts are tiresome.)

The combination

$$\text{fix}(\lambda x . \tau)$$

occurs so often, that from time to time we abbreviate it as

$$!x . \tau,$$

but remember it only makes sense if x and τ have the *same* type. For example in 4.3 we could have written

$$\sigma = !f \lambda n . \text{cond}(\text{zero}(n), 0, f(\text{pred}(n)) + \text{pred}(n))$$

and read this as

"σ is the least (recursively defined) function f whose value at n is cond $(\ldots)$."

We note that in the so-called "body" of the expression inside the

cond-part the variable f occurs again. That is just the point! This is a *recursive definition*; it is made into an *explicit* definition by invoking the least fixed-point operator.

In a λ-expression, $\lambda x, y, z.\tau$, say, the variables x, y, z are being *bound* in τ; but τ may have other variables that are nowhere bound in τ and these remain *free variables* of the whole expression. Bound variables are *dummy variables* and may be rewritten by other variables; thus

$$\lambda x.\tau = \lambda y.\tau[y/x]$$

is a true equation *PROVIDED* the variable y does not occur in τ. In the equation the notation $\tau[y/x]$ means the result of *substituting* (*rewriting*) the variable y for the variable x throughout the term τ. We can also write $\tau[\sigma/x]$ for substituting a whole term σ for a variable in the other term.

We have already spoken of "true equations", but how do we know that these curious equations are meaningful at all? They are, but this is something that has to be proved.

3.6.2 Theorem 5.1 — terms denote approximable maps

THEOREM 5.1. Every typed λ-term τ defines an approximable function of its free variables.

Proof: We argue by an induction on the complexity of τ; there will only be a few cases to consider since the "syntax" of λ-terms is limited — even though terms can be of any length.

If τ is a variable or a constant there is nothing to prove. We already know that

$$x \mapsto x \qquad \text{and} \qquad x \mapsto k$$

are approximable functions.

Suppose τ has the form

$$\langle \sigma_0, \ldots, \sigma_{n-1} \rangle.$$

Then the σ_i are less complex terms, and so we can assume — as our induction hypothesis — that they define approximable functions of the free variables. Having said this, we just apply the already

proved 3.4 to conclude (after a suitable generalization to the multivariate case) that τ, which takes on tuples as values, also defines an approximable function.

Next, suppose τ has the form

$$\sigma_0(\sigma_1),$$

where we are sure that the types of all the terms match properly. Again we can assume the σ_i to be well behaved. But the values we seek can also be written as

$$\mathrm{eval}(\sigma_0, \sigma_1).$$

Since eval is approximable by 3.11, we just have to invoke an instance of 3.7 to gain the desired conclusion.

Finally, suppose that τ has the form

$$\lambda x.\sigma.$$

By a judicious choice of the order of the variables in σ (including x), we can assume that σ defines an approximable function

$$g : D_0 \times \cdots \times D_{n-1} \times D_n \to D'$$

where D' is the type of σ, D_n is the type of x, and $D_0, \ldots, D_{n-1}$ are the types of the remaining free variables of σ. We apply 3.12 and obtain an approximable function

$$\mathrm{curry}(g) : D_0 \times \cdots \times D_{n-1} \to (D_n \to D').$$

But, this is just exactly the function defined by τ.

We leave as an exercise the more general case of a term τ of the form

$$\lambda x_0, \ldots, x_{k-1}.\sigma,$$

which has a string of bound variables. □

We can now say more precisely what it means to call $\sigma = \tau$ a "true equation". This means that, if we employ the method of the proof of 5.1, the two terms define the *same function* of the free variables. For example,

$$\lambda x.\tau = \lambda y.\tau[y/x]$$

is true, provided y does not occur free in the term τ, since the systematic generation of the function defined by $\lambda x.\tau$ does not depend on what the variable x *looks like* but only on its *position* in the term τ. Some other obviously desirable rules for generating true equations are stated in the exercises. But one rule is so basic that we state it here in full generality.

monotone.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Order.BourbakiWitt
public import DomainTheory.ContinuousLattice.ScottMaps.proposition_2_5
public import DomainTheory.ContinuousLattice.ScottMaps.preservesDirectedSup_monotone

@[expose] public section

namespace DomainTheory.ContinuousLattice

variable {D D' : Type*} [CompleteLattice D] [CompleteLattice D']

namespace ScottMap

theorem monotone (f : ScottMap D D') : Monotone (f : D → D') :=
  preservesDirectedSup_monotone ((proposition_2_5 (Subtype.val f)).mp f.property)

end ScottMap

end DomainTheory.ContinuousLattice
```

3.6.3 Theorem 5.2 — beta conversion

THEOREM 5.2. For suitably typed λ-terms the following equation is true:

$$(\lambda x_0, \ldots, x_{n-1} \cdot \tau)(\sigma_0, \ldots, \sigma_{n-1}) = \tau[\sigma_0/x_0, \ldots, \sigma_{n-1}/x_{n-1}].$$

Proof: It will be sufficient to carry out the proof for $n = 1$. The proof proceeds by induction on the complexity of the term τ.

In case τ is a *constant* k, the result reads

$$(\lambda x.k)(\sigma) = k,$$

and this is a true equation.

In case τ is a *variable* (in particular, the variable x), the result reads

$$(\lambda x.x)(\sigma) = \sigma,$$

and again this is a true equation.

In case τ is a *tuple* (say, $\langle \tau_0, \tau_1 \rangle$) the result reads

$$(\lambda x.\langle \tau_0, \tau_1 \rangle)(\sigma) = \langle \tau_0[\sigma/x], \tau_1[\sigma/x] \rangle.$$

This is true, because the left-hand side can be transformed by the true equation

$$(\lambda x.\langle \tau_0, \tau_1 \rangle)(\sigma) = \langle (\lambda x.\tau_0)(\sigma), (\lambda x.\tau_1)(\sigma) \rangle;$$

and then we apply the inductive assumption for τ_0 and for τ_1.

In case τ is an *application*, we want (supposing the term is $\tau_0(\tau_1)$),

$$(\lambda x.\tau_0(\tau_1))(\sigma) = \tau_0[\sigma/x](\tau_1[\sigma/x]).$$

We can proceed as in the last case, noting that the left-hand side equals

$$\text{eval } ((\lambda x.\langle \tau_0, \tau_1 \rangle)\ (\sigma))\ .$$

In case τ is an *abstract* (say, $\lambda y.\tau_0$), we want

$$(\lambda x.\lambda y.\tau_0)\ (\sigma) = \lambda y.\tau_0[\sigma/x]$$

PROVIDED the variable y is not free in σ. For this we require the true equation

$$(\lambda x.\lambda y.\tau)\ (\sigma) = \lambda y.(\lambda x.\tau)\ (\sigma)\ .$$

We argue for this by letting g be the function of $n + 2$ free variables defined by τ. Then, by 5.1, the λ-term $\lambda x.\lambda y.\tau$ defines the function $\text{curry } (\text{curry } (g))$ of n arguments. We can call this function h for the moment. We can write

$$h(v)\ (\sigma)\ (y) = g(v, \sigma, y)\ ,$$

where v is a list of arguments. But, with an appropriate combinator inv, which applied to g inverts the order of the last two arguments, we can write

$$h(v)\ (\sigma)\ (y) = \text{curry}\ (\text{inv}\ (g))\ (v, y)\ (\sigma)\ .$$

But, curry (inv (g)) is just the function defined by $(\lambda x.\tau)$. So what we have proved as true is

$$(\lambda x.\lambda y.\tau)\ (\sigma)\ (y) = (\lambda x.\tau)\ (\sigma)\ .$$

But if y is not free in α and

$$\alpha(y) = \beta$$

is true, then so is

$$\alpha = \lambda y.\beta\ .$$

This completes the proof. □

We note that if τ' is the term $\lambda x, y.\tau$, then $\tau'(x, y)$ means the same as τ. This gives a convenient way of indicating free variables: we just write $\sigma(x, y)$ — where x, y are not free in σ — and this will have the same values as any term τ which does involve the extra free variables x and y. We use this notational device in the next theorem.

monotone.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Order.BourbakiWitt
public import DomainTheory.ContinuousLattice.ScottMaps.proposition_2_5
public import DomainTheory.ContinuousLattice.ScottMaps.preservesDirectedSup_monotone

@[expose] public section

namespace DomainTheory.ContinuousLattice

variable {D D' : Type*} [CompleteLattice D] [CompleteLattice D']

namespace ScottMap

theorem monotone (f : ScottMap D D') : Monotone (f : D → D') :=
  preservesDirectedSup_monotone ((proposition_2_5 (Subtype.val f)).mp f.property)

end ScottMap

end DomainTheory.ContinuousLattice
```

4

3.6.4 Proposition 5.3 — simultaneous fixed points

PROPOSITION 5.3. The least fixed point of $\lambda x, y.\ \langle \tau(x, y), \sigma(x, y)\rangle$ is the pair with coordinates
$!x.\ \tau(x,\ !y.\ \sigma(x, y))$ and
$!y.\ \sigma(!x.\ \tau(x, y), y)$.

Proof: (We are assuming that x and y are not free in τ and σ.) The purpose of the fixed-point search is to find the least solution of the pair of equations $x = \tau(x, y)$ and $y = \sigma(x, y)$. In other words, we are generalizing the fixed-point equation from one to two variables—and, of course, we could go much further to any number of variables. To this end, let

$y_* = !y.\ \sigma(!x.\ \tau(x, y), y)$, and

$x_* = !x.\ \tau(x, y_*)$.

Then

$x_* = \tau(x_*, y_*)$,

and

$y_* = \sigma(!x.\ \tau(x, y_*), y_*)$

$= \sigma(x_*, y_*)$.

This proves that $\langle x_*, y_* \rangle$ is one fixed-point pair.

Suppose, then, that $\langle x_0, y_0 \rangle$ is the least solution. (Why does a least solution have to exist? Hint: Consider a suitable mapping of type $D_0 \times D_1 \to D_0 \times D_1$, where D_0 is the type of x and D_1 the type of y.) Then we know $x_0 = \tau(x_0, y_0)$ and $y_0 = \sigma(x_0, y_0)$, and also $x_0 \sqsubseteq x_*$ and $y_0 \sqsubseteq y_*$. But from

$\tau(x_0, y_0) \sqsubseteq x_0$,

it follows that

$!x.\ \tau(x, y_0) \sqsubseteq x_0$.

Consequently,

$$\sigma(!x.\tau(x, y_0), y_0) \sqsubseteq \sigma(x_0, y_0) \sqsubseteq y_0.$$

By the fixed-point definition of y_*, we have $y_* \sqsubseteq y_0$, so $y_* = y_0$; whence,

$$x_* = !x.\tau(x, y_*) = !x.\tau(x, y_0) \sqsubseteq x_0.$$

So also $x_* = x_0$. □ We have the right formula for y_0, and a similar argument gives x_0.

The purpose of giving the above proof was to illustrate the use of the least fixed-point operator in proofs. We have such true principles as:

$$!x.\tau(x) = \tau(!x.\tau(x));$$

and

$$\tau(y) \sqsubseteq y \text{ implies } !x.\tau(x) \sqsubseteq y,$$

provided, of course, that x is not free in τ. These, together with the monotonicity of all the functions, were just the methods used in the above proof. Here is another example.

monotone.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Order.BourbakiWitt
public import DomainTheory.ContinuousLattice.ScottMaps.proposition_2_5
public import DomainTheory.ContinuousLattice.ScottMaps.preservesDirectedSup_monotone

@[expose] public section
```

```
namespace DomainTheory.ContinuousLattice

variable {D D' : Type*} [CompleteLattice D] [CompleteLattice D']

namespace ScottMap

theorem monotone (f : ScottMap D D') : Monotone (f : D → D') :=
  preservesDirectedSup_monotone ((proposition_2_5 (Subtype.val f)).mp f.property)

end ScottMap

end DomainTheory.ContinuousLattice
```

5

3.6.5 Proposition 5.4 — fixed point under a parameter

PROPOSITION 5.4. Let x, y, and $\tau(x, y)$ be of the same type D and let g be of type $(D \to D)$, then the equation

$$\lambda x.!y.\tau(x, y) =!g.\lambda x.\tau(x, g(x))$$

is true.

Proof : Let f be the function on the left-hand side. We can write

$$f(x) =!y.\tau(x, y) = \tau(x, f(x)).$$

Therefore

$$f = \lambda x.\tau(x, f(x)),$$

and it follows that

$$g_0 =!g.\lambda x.\tau(x, g(x)) \sqsubseteq f.$$

Then we have at once, by definition of g_0,

$$g_0(x) = \tau(x, g_0(x)),$$

for any given x. But by definition of f we find

$$f(x) =!y.\tau(x, y) \sqsubseteq g_0(x).$$

As this holds for all x, then $f \sqsubseteq g_0$ follows. So the equation is true. □

The last proof is instructive as it uses equations and inclusions between functions. In particular we have just made use of the principle:

> if $\tau \sqsubseteq \sigma$ holds for all values of x,
> then $\lambda x.\tau \sqsubseteq \lambda x.\sigma$ holds.

This is another form of Theorem 3.13(i).

monotone.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Order.BourbakiWitt
public import DomainTheory.ContinuousLattice.ScottMaps.proposition_2_5
public import DomainTheory.ContinuousLattice.ScottMaps.preservesDirectedSup_monotone

@[expose] public section

namespace DomainTheory.ContinuousLattice

variable {D D' : Type*} [CompleteLattice D] [CompleteLattice D']

namespace ScottMap

theorem monotone (f : ScottMap D D') : Monotone (f : D → D') :=
  preservesDirectedSup_monotone ((proposition_2_5 (Subtype.val f)).mp f.property)

end ScottMap

end DomainTheory.ContinuousLattice
```

6

3.6.6 Table 5.5 — table of combinators

TABLE 5.5. In the displayed table we give a summary of uses of the λ-notation to define various combinators. We have mentioned some of these equations before, and there are some combinators here we have not mentioned before — their meanings, however, should be clear.

$$
\begin{aligned}
P_0 &= \lambda x, y.x \\
P_1 &= \lambda x, y.y \\
\text{pair} &= \lambda x \lambda y.\langle x, y\rangle \\
\text{n-tuple} &= \lambda x_0 \lambda x_1 \ldots \lambda x_{n-1}.\langle x_0, x_1, \ldots, x_{n-1}\rangle \\
\text{diag} &= \lambda x.\langle x, x\rangle \\
\text{funpair} &= \lambda f \lambda g \lambda x.\langle f(x), g(x)\rangle \\
\text{proj}_i^n &= \lambda x_0, x_1, \ldots, x_{n-1}.x_i \\
\text{inv}_{i,j}^n &= \lambda x_0, \ldots, x_i, \ldots, x_j, \ldots, x_{n-1}.\langle x_0, \ldots, x_j, \ldots, x_i, \ldots, x_{n-1}\rangle
\end{aligned}
$$

$$
\begin{aligned}
\text{eval} &= \lambda f, x.f(x) \\
\text{curry} &= \lambda g \lambda x \lambda y.g(x, y) \\
\text{comp} &= \lambda g, f \lambda x.g(f(x)) \\
\text{const} &= \lambda k \lambda x.k \\
\text{fix} &= \lambda f! x.f(x)
\end{aligned}
$$

A TABLE OF COMBINATORS

It is important to note that since we have not typed the variables, these equations are ambiguous: they only become precise when the types are specified. It follows, therefore, that what we find in the table are *schemes* for combinators; there are actually infinitely many distinct combinators corresponding to any one

equation depending on how the variables have types chosen for them. Clearly it is better to imagine this variety of combinators than it is to try to notate them with type superscripts.

One interest of combinators is that it is often possible to write expressions without variables — if enough combinators are used. This is sometimes useful, but it can become clumsy. On the other hand, if the same combination occurs over and over, it is sometimes useful to give it a name. This is what we do with, say, *composition* where

$$\mathrm{comp}\ (g, f) = g \circ f.$$

On the one side we have the prefix notation, and on the other, the more common infix notation. With either notation the variable seen in $\lambda x.\ g(f(x))$ has been got rid of. The choice between equivalent notations ought to be based on a desire for readability. □

The reader will have noted that there are some combinators not appearing in Table 5.5. The reason is that combinators like cond, succ, pred, zero, 0 cannot be defined in the pure λ-notation but are specific to domains like T and $\mathbf{N}$; we, thus, have to regard them as primitive. But once they are in hand, a very large number of other functions can be defined from these combined with λ-expressions. The next theorem gives an indication of the possibilities.

monotone.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Order.BourbakiWitt
public import DomainTheory.ContinuousLattice.ScottMaps.proposition_2_5
public import DomainTheory.ContinuousLattice.ScottMaps.preservesDirectedSup_monotone

@[expose] public section

namespace DomainTheory.ContinuousLattice

variable {D D' : Type*} [CompleteLattice D] [CompleteLattice D']

namespace ScottMap

theorem monotone (f : ScottMap D D') : Monotone (f : D → D') :=
  preservesDirectedSup_monotone ((proposition_2_5 (Subtype.val f)).mp f.property)

end ScottMap

end DomainTheory.ContinuousLattice
```

7

3.6.7 Theorem 5.6 — partial recursive functions

THEOREM 5.6. For every partial recursive function $h : \mathbf{N} \to \mathbf{N}$, there is a λ-term τ of type $(\mathbf{N} \to \mathbf{N})$ such that the only constants occurring in τ are

$$\mathrm{cond},\ \mathrm{succ},\ \mathrm{pred},\ \mathrm{zero},\ 0$$

and where if $h(n) = m$, then

$$\tau(n) = m$$

is true; and if $h(n)$ is undefined, then

$$\tau(n) = \perp$$

is true. The equation $\tau(\perp) = \perp$ is also true.

Proof : We have only formulated the theorem for functions of one variable — but to give the proof, it is convenient to pass through functions of any number of (integer) variables. We shall also have to recall the precise definition of the notion of partial recursive function.

It is also convenient to work with (*very*)*strict* functions

$$f : \mathbf{N}^k \to \mathbf{N}.$$

These are functions such that if $n_0, \ldots, n_{k-1} \in \mathbf{N}$ and $n_i = \perp$ for at least one $i < k$, then

$$f(n_0, \ldots, n_{k-1}) = \perp.$$

It is easy to check that compositions of strict functions are strict. It is also easy to see that any *partial* function

$$g : \mathbf{N}^k \to \mathbf{N}$$

extends to a strict (*approximable*) function

$$\bar{g} : \mathbf{N}^k \to \mathbf{N},$$

which takes the same values as g as long as g is defined; otherwise $\bar{g}$ takes the value $\perp$. What we want to show for *partial recursive* g is that the corresponding $\bar{g}$ is defined by a λ - expression.

In the first place we have to check that *primitive recursive* functions have λ - definitions in this sense. We recall that *primitive recursive* functions are generated from certain elementary *starting* functions by multi-variate composition and the scheme of primitive recrusion. The *starting* functions are the constant function with value zero and the "identity" or "projection" functions. For example, $g(n_0, n_1, n_2) = n_1$ for all $n_0, n_1, n_2 \in \mathbf{N}$ is one of the *starting* functions. Now we cannot just use the λ-term

$$\lambda x_0, x_1, x_2.x_1$$

to represent $\bar{g}$, because the function so defined is not strict. But any function in $\mathbf{N}^k \to \mathbf{N}$ can be cut down to a strict function by a simple device. Consider

$$\lambda x.\ \mathrm{cond}(\mathrm{zero}(x), x, x)$$

with x of type $\mathbf{N}$. This is the strict version of the identity function of one argument. The strict projection function of two arguments can be defined by

$$\lambda x_0, x_1.\ \mathrm{cond}(\mathrm{zero}(x_1), x_0, x_0).$$

The one of three arguments by:

$$\lambda x_0, x_1, x_2.\ \mathrm{cond}(\mathrm{zero}(x_0), \mathrm{cond}(\mathrm{zero}(x_2), x_1, x_1), \mathrm{cond}(\mathrm{zero}(x_2), x_1, x_1)).$$

This is not done very elegantly, and the reader can find for himself a general solution based on perhaps a better notation for the required compositions of functions.

As we remarked, strict functions are closed under substitution, and any substitution of a batch of functions into another function can be given by a λ-term, if the various functions can themselves be so defined. It only remains to λ-define functions obtained by primitive recursion. Thus, suppose, for the sake of argument, that

$$\bar{f} : \mathbf{N} \to \mathbf{N} \qquad \text{and} \qquad \bar{g} : \mathbf{N}^3 \to \mathbf{N}$$

are given as total functions with $\bar{f}$ and $\bar{g}$ being λ-definable. From them, we obtain by primitive recursion $\bar{h} : \mathbf{N}^2 \to \mathbf{N}$ where

$$\bar{h}(0, m) = \bar{f}(m),$$

$$\bar{h}(n+1, m) = \bar{g}(n, m, \bar{h}(n, m))$$

for all $n, m \in \mathbf{N}$. The λ-term defining $\bar{h}$ is

$$!k\ \lambda x, y.\ \mathrm{cond}(\mathrm{zero}(x), \bar{f}(y), \bar{g}(\mathrm{pred}(x), y, k(\mathrm{pred}(x), y))).$$

Here we have had to use the fixed-point operator on a variable k of type $(\mathbf{N}^2 \to \mathbf{N})$. The variables x, y are of type $\mathbf{N}$ and the cond-construction puts the two traditional equations into two clauses of one expression. It is easy to see that the fixed-point function *is* strict and is nothing more than $\bar{h}$.

That completes the representation of *primitive recursive* functions. To obtain the *partial recursive* functions, the idea is to use the so-called μ-scheme (least number operator) and, further, to close up under substitution. We need only treat the μ-scheme. Suppose, by way of example, $f(n, m)$ is given as a

primitive recursive function. We then define h (generally, a partial function) by

$$h(m) = \text{the least } n \text{ where } f(n, m) = 0.$$

This is often written

$$h(m) = \mu n.\ f(n, m) = 0.$$

Supposing, as we may, $\bar{f}$ is λ-definable, we introduce first

$$\bar{g} = !g\ \lambda x, y.\ \mathrm{cond}(\mathrm{zero}(\bar{f}(x, y)), x, g(\mathrm{succ}(x), y)).$$

Then $\bar{h} = \lambda y.\ \bar{g}(0, y)$. This is easily seen to be strict. Also easy to see is that if $h(m)$ is defined, then $\bar{g}(0, m) = h(m)$. But, if $h(m)$ is not defined, it takes some argument to make sure that the least fixed-point construction forces $\bar{g}(0, m) = \perp$. However, the argument is not very difficult. $\square$

What is *not* said in 5.6 is that every λ-term defines a partial recursive function. This is true (with suitable control over the constants and types in the expression), but the proof requires a full analysis of computability properties of domain constructions. This is the topic of Lecture VII.

It should be remarked that the types of variables needed for the proof of 5.6 never get very high. In fact, types like N, N^k, and $(N^k \to N)$ were the only ones needed (with perhaps T thrown in also).

Recursion on N was the topic of 5.6; further examples of recursion on other domains are included in the exercises.

monotone.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Order.BourbakiWitt
public import DomainTheory.ContinuousLattice.ScottMaps.proposition_2_5
public import DomainTheory.ContinuousLattice.ScottMaps.preservesDirectedSup_monotone

@[expose] public section

namespace DomainTheory.ContinuousLattice

variable {D D' : Type*} [CompleteLattice D] [CompleteLattice D']

namespace ScottMap

theorem monotone (f : ScottMap D D') : Monotone (f : D → D') :=
  preservesDirectedSup_monotone ((proposition_2_5 (Subtype.val f)).mp f.property)

end ScottMap

end DomainTheory.ContinuousLattice
```

8

3.6.8 Exercise 5.7 — currying the lambda notation

Find definitions of

$$\lambda x, y.\, \tau \quad \text{and} \quad \sigma(x, y)$$

which use only λv with one variable and applications only to one argument at a time. Note that use must be made of the combinators p_0, p_1, pair. Generalize the result to functions of many variables.

monotone.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Order.BourbakiWitt
public import DomainTheory.ContinuousLattice.ScottMaps.proposition_2_5
public import DomainTheory.ContinuousLattice.ScottMaps.preservesDirectedSup_monotone

@[expose] public section

namespace DomainTheory.ContinuousLattice

variable {D D' : Type*} [CompleteLattice D] [CompleteLattice D']

namespace ScottMap

theorem monotone (f : ScottMap D D') : Monotone (f : D → D') :=
  preservesDirectedSup_monotone ((proposition_2_5 (Subtype.val f)).mp f.property)
```

```
end ScottMap

end DomainTheory.ContinuousLattice
```

9

3.6.9 Exercise 5.8 — combinatory completeness

(For combinator nuts.) Table 5.5 was meant to show how combinators could be defined in terms of λ-expressions. Can the tables be turned to show that with enough combinators available, every λ-expression can be defined by combining combinators, using $\sigma(\tau)$ as the *only* mode of combination?

pointwiseSubbasic_isOpen.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottMapPointwiseTopology
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem pointwiseSubbasic_isOpen (x : D) {U : Set D'} (hU : @IsOpen D'
    scottTopologicalSpace U) :
    @IsOpen (ScottMap D D') (scottMapPointwiseTopology D D')
      {f : ScottMap D D' | (f : D → D') x ∈ U} :=
  isOpen_generateFrom_of_mem ⟨x, U, hU, rfl⟩

end DomainTheory.ContinuousLattice
```

0

3.6.10 Exercise 5.9 — common fixed points

Suppose that $f, g : \mathcal{D} \to \mathcal{D}$ are approximable and $f \circ g = g \circ f$. Show that f and g have a *least common fixed point* $x = f(x) = g(x)$. (Hint: Refer back to Exercise 4.20.) If in addition $f(\bot) = g(\bot)$, show that $\mathrm{fix}(f) = \mathrm{fix}(g)$. In particular, will $\mathrm{fix}(f) = \mathrm{fix}(f^2)$? What if we only assume $f \circ g = g^2 \circ f$?

pointwiseSubbasic_isOpen.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottMapPointwiseTopology
public import Mathlib.Topology.Defs.Basic

@[expose] public section
```

```
namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem pointwiseSubbasic_isOpen (x : D) {U : Set D'} (hU : @IsOpen D'
    scottTopologicalSpace U) :
    @IsOpen (ScottMap D D') (scottMapPointwiseTopology D D')
      {f : ScottMap D D' | (f : D → D') x ∈ U} :=
  isOpen_generateFrom_of_mem ⟨x, U, hU, rfl⟩

end DomainTheory.ContinuousLattice
```

1

3.6.11 Exercise 5.10 — smash product

Suppose $\mathcal{D}_0$ and $\mathcal{D}_1$ are neighbourhood systems over disjoint sets Δ_0 and Δ_1. Define the *smash product* $\mathcal{D}_0 \otimes \mathcal{D}_1$ with neighbourhoods

$$\{\Delta_0 \cup \Delta_1\} \cup \{X \cup Y \mid X \in \mathcal{D}_0 \setminus \{\Delta_0\} \text{ and } Y \in \mathcal{D}_1 \setminus \{\Delta_1\}\}.$$

Show that this *is* a neighbourhood system. Define $(\mathcal{D}_0 \to_\perp \mathcal{D}_1)$ so that $|\mathcal{D}_0 \to_\perp \mathcal{D}_1|$ consists exactly of the *strict functions*. By introducing appropriate combinators, show that

$$(\mathcal{D}_0 \to_\perp (\mathcal{D}_1 \to_\perp \mathcal{D}_2)) \quad \text{and} \quad ((\mathcal{D}_0 \otimes \mathcal{D}_1) \to_\perp \mathcal{D}_2)$$

are isomorphic.

pointwiseSubbasic_isOpen.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottMapPointwiseTopology
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem pointwiseSubbasic_isOpen (x : D) {U : Set D'} (hU : @IsOpen D'
    scottTopologicalSpace U) :
    @IsOpen (ScottMap D D') (scottMapPointwiseTopology D D')
```

```
      {f : ScottMap D D' | (f : D → D') x ∈ U} :=
  isOpen_generateFrom_of_mem ⟨x, U, hU, rfl⟩

end DomainTheory.ContinuousLattice
```

2

3.6.12 Exercise 5.11 — stack combinators

For any domain $\mathcal{D}$ we may regard $\mathcal{D}^\infty$ as consisting of (bottomless) *stacks* of elements of $\mathcal{D}$. With this image in mind, define appropriate combinators with the obvious meanings:

$$\{\mathrm{head} : \mathcal{D}^\infty \to \mathcal{D}\}; \quad \{\mathrm{tail} : \mathcal{D}^\infty \to \mathcal{D}^\infty\}; \quad \{\mathrm{push} : \mathcal{D} \times \mathcal{D}^\infty \to \mathcal{D}^\infty\}.$$

Using the fixed-point theorem argue that there is a combinator

$$\{\mathrm{diag} : \mathcal{D} \to \mathcal{D}^\infty\}$$

where for all $x \in |\mathcal{D}|$ we have

$$\mathrm{diag}(x) = \langle x \rangle_{n=0}^{\infty}.$$

(Hint: Try a recursive definition, say

$$\mathrm{diag}(x) = \mathrm{push}(x, \mathrm{diag}(x)),$$

but be sure to prove *all* terms of $\mathrm{diag}(x)$ equal x.) Also introduce by an appropriate recursion a combinator

$$\{\mathrm{map} : (D \to D)^\infty \times D \to D^\infty\}$$

where for elements of the suitable types:

$$\mathrm{map}(\langle f_n \rangle_{n=0}^{\infty}, x) = \langle f_n(x) \rangle_{n=0}^{\infty}.$$

pointwiseSubbasic_isOpen.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottMapPointwiseTopology
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']
```

```
theorem pointwiseSubbasic_isOpen (x : D) {U : Set D'} (hU : @IsOpen D'
    scottTopologicalSpace U) :
    @IsOpen (ScottMap D D') (scottMapPointwiseTopology D D')
      {f : ScottMap D D' | (f : D → D') x ∈ U} :=
  isOpen_generateFrom_of_mem ⟨x, U, hU, rfl⟩

end DomainTheory.ContinuousLattice
```

3

3.6.13 Exercise 5.12 — while combinator

On any domain D introduce (as a least fixed point) a combinator

$$\text{while} : (D \to T) \times (D \to D) \to (D \to D)$$

by the recursion

$$\text{while}(p, f)(x) = \text{cond}(p(x), \text{while}(p, f)(f(x)), x).$$

Prove that

$$\text{while}(p, \text{while}(p, f)) = \text{while}(p, f).$$

Show how while could have been used to obtain the least number operator mentioned in the proof of 5.6. Generalize the idea to define a combinator

$$\{\text{find} : D^\infty \times (D \to T) \to D\}$$

with the meaning "find the first term of the sequence (if any) which satisfies the given precicate."

pointwiseSubbasic_isOpen.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottMapPointwiseTopology
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem pointwiseSubbasic_isOpen (x : D) {U : Set D'} (hU : @IsOpen D'
    scottTopologicalSpace U) :
    @IsOpen (ScottMap D D') (scottMapPointwiseTopology D D')
      {f : ScottMap D D' | (f : D → D') x ∈ U} :=
  isOpen_generateFrom_of_mem ⟨x, U, hU, rfl⟩
```

```
end DomainTheory.ContinuousLattice
```

4

3.6.14 Exercise 5.13 — pairing function on numbers

Prove the existence of a one-one function $\mathrm{num} : \mathbf{N} \times \mathbf{N} \to \mathbf{N}$ such that

$$\mathrm{num}(0,0) = 0;$$

$$\mathrm{num}(n, m+1) = \mathrm{num}(n+1, m) + 1;$$

$$\mathrm{num}(n+1, 0) = \mathrm{num}(0, n) + 1.$$

Draw a picture (i.e. an infinite matrix) for the function and find a closed form for its values, if possible. Use the function to prove the isomorphism of the domains

$$P\mathbf{N},\ P(\mathbf{N} \times \mathbf{N}),\ P\mathbf{N} \times P\mathbf{N}.$$

pointwiseSubbasic_isOpen.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottMapPointwiseTopology
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem pointwiseSubbasic_isOpen (x : D) {U : Set D'} (hU : @IsOpen D'
    scottTopologicalSpace U) :
    @IsOpen (ScottMap D D') (scottMapPointwiseTopology D D')
      {f : ScottMap D D' | (f : D → D') x ∈ U} :=
  isOpen_generateFrom_of_mem ⟨x, U, hU, rfl⟩

end DomainTheory.ContinuousLattice
```

5

3.6.15 Exercise 5.13 — powerset isomorphisms

Prove the existence of a one-one function $\mathrm{num} : \mathbf{N} \times \mathbf{N} \to \mathbf{N}$ such that

$$\mathrm{num}(0,0) = 0;$$

$$\mathrm{num}(n, m+1) = \mathrm{num}(n+1, m) + 1;$$

$$\mathrm{num}(n+1, 0) = \mathrm{num}(0, n) + 1.$$

Draw a picture (i.e. an infinite matrix) for the function and find a closed form for its values, if possible. Use the function to prove the isomorphism of the domains

$$P\mathbf{N},\ P(\mathbf{N} \times \mathbf{N}),\ P\mathbf{N} \times P\mathbf{N}.$$

pointwiseSubbasic_isOpen.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottMapPointwiseTopology
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem pointwiseSubbasic_isOpen (x : D) {U : Set D'} (hU : @IsOpen D'
    scottTopologicalSpace U) :
    @IsOpen (ScottMap D D') (scottMapPointwiseTopology D D')
      {f : ScottMap D D' | (f : D → D') x ∈ U} :=
  isOpen_generateFrom_of_mem ⟨x, U, hU, rfl⟩

end DomainTheory.ContinuousLattice
```

6

3.6.16 Exercise 5.14 — functions as graphs

Show that there are approximable mappings

$$\mathrm{graph} : (P\mathbf{N} \to P\mathbf{N}) \to P\mathbf{N} \quad \text{and} \quad \mathrm{fun} : P\mathbf{N} \to (P\mathbf{N} \to P\mathbf{N}),$$

where we have

$$\mathrm{fun} \circ \mathrm{graph} = \lambda f.\, f, \quad \text{and} \quad \mathrm{graph} \circ \mathrm{fun} \supseteq \lambda x.\, x.$$

(Hint: Using the notation

$$[n_0, n_1, \ldots, n_k] = \mathrm{num}(n_0, [n_1, \ldots, n_k])$$

two such combinators can be given by formulae

$$\mathrm{fun}(u)(x) = \{m \mid \exists n_0, \ldots, n_{k-1} \in x.\ [n_0 + 1, \ldots, n_{k-1} + 1, 0, m] \in u\}$$

$$\mathrm{graph}(f) = \{[n_0 + 1, \ldots, n_{k-1} + 1, 0, m] \mid m \in f(\{n_0, \ldots, n_{k-1}\})\}$$

where k is variable — meaning all finite sequences are to be considered.)

pointwiseSubbasic_isOpen.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottMapPointwiseTopology
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem pointwiseSubbasic_isOpen (x : D) {U : Set D'} (hU : @IsOpen D'
    scottTopologicalSpace U) :
    @IsOpen (ScottMap D D') (scottMapPointwiseTopology D D')
      {f : ScottMap D D' | (f : D → D') x ∈ U} :=
  isOpen_generateFrom_of_mem ⟨x, U, hU, rfl⟩

end DomainTheory.ContinuousLattice
```

7

3.6.17 Exercise 5.15 — star of a word

(For algebraists.) We can regard $\langle \{0,1\}^*, \Lambda, \cdot \rangle$ as the free semigroup on two generators 0 and 1. The powerset $P\{0,1\}^*$ is taken as a domain as in Exercise 4.17. For "words" $e \in \{0,1\}^*$ define

$$e^* = \{\Lambda, e, e^2, e^3, \ldots, e^n, \ldots\}.$$

Show that the least fixed point of

$$z = \{e\} \cdot z \cup \{e'\}$$

in $P\{0,1\}^*$ is $z = e^* \cdot \{e'\}$.

Show further (as suggested by David Park) that the least solution of

$$x = a \cdot x \cup b \cdot y \cup c,$$
$$y = b \cdot x \cup a \cdot y \cup d$$

has

$$x = (a \cup b \cdot a^* \cdot b)^* \cdot (c \cup b \cdot a^* \cdot d),$$

where the $\{\cdot\}$ has been dropped off $\{a\}$, $\{b\}$ etc., and where the *-notation has been extended to the whole domain, so that $z^* = \Lambda \cup z^* \cdot z$.

(Hint: Apply 5.3.)

pointwiseSubbasic_isOpen.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottMapPointwiseTopology
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem pointwiseSubbasic_isOpen (x : D) {U : Set D'} (hU : @IsOpen D'
    scottTopologicalSpace U) :
    @IsOpen (ScottMap D D') (scottMapPointwiseTopology D D')
      {f : ScottMap D D' | (f : D → D') x ∈ U} :=
  isOpen_generateFrom_of_mem ⟨x, U, hU, rfl⟩

end DomainTheory.ContinuousLattice
```

8

3.6.18 Exercise 5.16 — negation of sequences

Return to the discussion of Example 4.4 and the construction of the domain of finite and infinite binary sequences. Give a fixed-point definition of $\mathrm{neg} : C \to C$, where

$$\mathrm{neg}(0x) = 1\,\mathrm{neg}(x); \qquad \mathrm{neg}(1x) = 0\,\mathrm{neg}(x).$$

Prove that $\mathrm{neg}(\mathrm{neg}(x)) = x$ for all $x \in |C|$. Also define $\mathrm{merge} : C \times C \to C$, where for $\epsilon, \delta \in \{0, 1\}$ we have:

$$\mathrm{merge}(\epsilon x, \delta y) = \epsilon\,\delta\,\mathrm{merge}(x, y).$$

(Note: There may be a little trouble with $\mathrm{merge}(x, y)$ when x is finite and total and y is infinite—you have to decide what you want in e.g. $\mathrm{merge}(\Lambda, y)$.) Prove that

$$\mathrm{merge}(x, x) = d(x),$$

in the notation of 4.4. Consider also the infinite non-periodic sequence

$$t = 0\,\mathrm{merge}(\mathrm{neg}(t), \mathrm{tail}(t)).$$

Prove that the nth digit of t is the sum mod 2 of the digits of the number n written in the binary scale (a suggestion of J. Lambek). Show also that $t \neq u\,a\,a\,a\,v$ where a is any finite sequence $\neq \Lambda$, and where u is finite.

pointwiseSubbasic_isOpen.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottMapPointwiseTopology
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem pointwiseSubbasic_isOpen (x : D) {U : Set D'} (hU : @IsOpen D'
    scottTopologicalSpace U) :
    @IsOpen (ScottMap D D') (scottMapPointwiseTopology D D')
      {f : ScottMap D D' | (f : D → D') x ∈ U} :=
  isOpen_generateFrom_of_mem ⟨x, U, hU, rfl⟩

end DomainTheory.ContinuousLattice
```

9

3.7 Domain equations

Dependency diagram. Each box is one item. The arrowhead is on the item that imports. An arrow already implied by a longer path is left out. The introduction has no Lean file, so it is not shown. Blue is a definition, green a proposition or lemma, amber an example, red a theorem, and indigo an exercise.

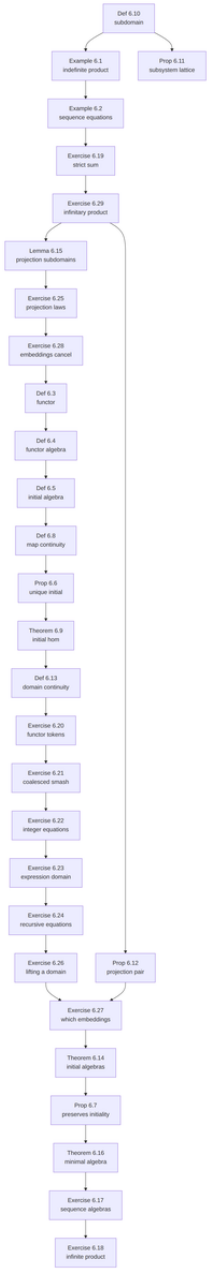


Figure 33: Domain equations.

3.7.1 Introduction

LECTURE VI INTRODUCTION TO DOMAIN EQUATIONS

The major reason for introducing the theory of domains is to have a notion of *computability* incorporating both finite and infinite elements. In our many examples already explored we have seen how functions (functionals, operators, combinators) can be defined on domains; owing to the property of *approximability (continuity)* of these functions, we have also seen how they can be "calculated" by finite approximation. In

this lecture further examples of domains will be constructed – especially domains having infinite elements, which can be introduced in a variety of ways giving rise to interesting structural possibilities. The next lecture then treats a precise notion of *computability* appropriate to these domains; while the last lecture opens up new methods of domain construction.

3.7.2 Example 6.1 — indefinite product

EXAMPLE 6.1. Let D be fixed as a given domain. We are now familiar with a useful construct like $D \times D$ whose elements are ordered pairs $\langle x, y\rangle$ of elements x, y of D. The question is: can this construct be iterated? The answer is obviously yes, since $D \times (D \times D)$ and $(D \times D) \times (D \times D)$ and so on can be formed with elements $\langle x, \langle y, z\rangle\rangle$ and $\langle\langle u, v\rangle, \langle x, y\rangle\rangle$ and the like. But the real question is: can the construct be iterated *indefinitely*? AND can the results be collected together into a *single* domain? The answer is yes, but it requires a bit of work to get it right. The method to be introduced will be open to many variations, so more than one answer is possible, giving non-isomorphic domains.

In order to collect all the iterates into one large domain we give ourselves first a very big domain inside of which the desired family of neighbourhoods will be found. There are many ways to make this choice, and we are fixing on one that will keep the notation simple. We have often used binary sequences for examples and constructions, but for this example let us use

ternary sequences. Let $\Sigma = \{0, 1, 2\}$ and let Σ^* be all finite sequences from this three-letter alphabet. We will select subsets of Σ^* for our neighbourhoods. As Σ^* is countably infinite, it is without much loss of generality to assume that $\mathcal{D}$ is a neighbourhood system over Δ, where we take $\Delta \subseteq \Sigma^*$. Also without loss of generality we can assume $\emptyset \notin \mathcal{D}$. (Why?)

We wish to find another set $\Gamma \subseteq \Sigma^*$ to be the set of tokens for the new domain. After we find it, we will still have to say just which $X \subseteq \Gamma$ are appropriate for the structure we want.

The totality $\{X \mid X \subseteq \Sigma^*\}$ is, as a powerset, isomorphic to the set of elements of a domain: a point we have remarked several times. So, by the Fixed-Point Theorem we know there is a set $\Gamma \subseteq \Sigma^*$ where

$$\Gamma = 0\Delta \cup 1\Gamma \cup 2\Gamma.$$

In fact $\Gamma = \{1, 2\}^*0\Delta$, because we can say:

$$\{1, 2\}^* = \{\Lambda\} \cup 1\{1, 2\}^* \cup 2\{1, 2\}^*.$$

The domain we are looking for will be found as a domain $\mathcal{D}^\S$ over Γ. The reason for splitting Γ up, as shown in the equation above, is to ensure that if $X, Y \in \mathcal{D}^\S$ are two neighbourhoods in the system $\mathcal{D}^\S$, then $1X \cup 2Y$ has a chance of being also in $\mathcal{D}^\S$ because

$$1X \cup 2Y \subseteq \Gamma.$$

This will make $\mathcal{D}^\S \times \mathcal{D}^\S$ isomorphic to a part of $\mathcal{D}^\S$. If we make $\mathcal{D}$ also isomorphic to a part of $\mathcal{D}^\S$, then all the iterated products will be contained in $\mathcal{D}^\S$.

What is a neighbourhood system? Just a set of sets. But $\mathcal{PP}\Sigma^*$ is a domain (as a power set) and because $\Gamma \subseteq \Sigma^*$, we find

$$\mathcal{D}^\S \in \mathcal{PP}\Sigma^*$$

as an element. But elements of domains can often be defined by fixed-point equations. Indeed we will introduce $\mathcal{D}^\S$ this way:

$$\mathcal{D}^{\S} = \{\Gamma\} \cup \{0X \mid X \in \mathcal{D}\} \cup \{1X \cup 2Y \mid X, Y \in \mathcal{D}^{\S}\}.$$

The reader should stop to think why $\mathcal{D}^{\S}$ can be immediately seen to exist by writing such an equation. Of course another way to describe $\mathcal{D}^{\S}$ is to say it is the least family of sets containing (i) the set Γ, (ii) the sets $0X$ for X in the given system $\mathcal{D}$, and (iii) sets $1X \cup 2Y$ whenever it already contains X and Y (closure under a set-forming operation). By saying "least" we mean (iv) nothing else belongs to $\mathcal{D}^{\S}$ except as allowed by (i)–(iii); this makes the truth of the equation for $\mathcal{D}^{\S}$ clear. So $\mathcal{D}^{\S}$ exists as a family of sets, but what good is it?

By our construction of Γ, all the sets we put into $\mathcal{D}^{\S}$ are subsets of Γ (why?); so $\mathcal{D}^{\S}$ has a chance of being a system over Γ if we can check the closure under intersection. So suppose $Z \subseteq X \cap Y$ where $Z, X, Y \in \mathcal{D}^{\S}$; we want to show $X \cap Y \in \mathcal{D}^{\S}$. We argue by induction on the number of steps required to put X and Y into $\mathcal{D}^{\S}$ by (i)–(iii). There are several cases.

If $X = \Gamma$ or $Y = \Gamma$, there is nothing to prove, because both sets are subsets of Γ. We note that $\emptyset \notin \mathcal{D}^{\S}$, because (i)–(iii) cannot introduce $\emptyset$ as a member of $\mathcal{D}^{\S}$. So, if $X = 0A$ for $A \in \mathcal{D}$, then Y must have this form also (if it is not Γ), because

$$0A \cap (1B \cup 2C) = \emptyset.$$

(That is, if Y had the form (iii), then $Z = \emptyset$ would be a consequence, which is impossible.) Thus, if $X = 0A$ for $A \in \mathcal{D}$, then $Y = 0B$ for some $B \in \mathcal{D}$. But by the same reasoning $Z = 0C$ for some $C \in \mathcal{D}$ also. But the relationship $0C \subseteq 0A \cap 0B$ is equivalent to $C \subseteq A \cap B$. We see, therefore, that $A \cap B \in \mathcal{D}$, and so

$$X \cap Y = 0A \cap 0B = 0(A \cap B)$$

must belong to $\mathcal{D}^{\S}$.

The final case has X, Y, Z all of the form (iii):

$$X = 1A_1 \cup 2A_2,$$

$$Y = 1B_1 \cup 2B_2, \text{ and}$$

$$Z = 1C_1 \cup 2C_2.$$

We can think of the A_i and B_i put into $\mathcal{D}^{\S}$ earlier and the intersection result as being already established for them. But the relationship $Z \subseteq X \cap Y$ is equivalent to $C_i \subseteq A_i \cap B_i$ for $i = 1, 2$. Therefore $A_i \cap B_i \in \mathcal{D}^{\S}$, and so does

$$X \cap Y = (1A_1 \cup 2A_2) \cap (1B_1 \cup 2B_2) = 1(A_1 \cap B_1) \cup 2(A_2 \cap B_2).$$

We have now seen that $D^{\S}$ is a neighbourhood system, but why was it constructed that way? The reason is simply this isomorphism (or domain equation):

$$D^{\S} \cong D + (D^{\S} \times D^{\S}),$$

as can be seen by reference to the equation for $D^{\S}$ and the definitions of $+$ and $\times$. What are the elements of $D^{\S}$? There is always

$$\bot = \{\Gamma\}.$$

Next if $x \in |D|$ we define

$$x^{\S} = \{\Gamma\} \cup \{0X \mid X \in x\}.$$

That gives an isomorphic injection

$$\lambda x.\, x^{\S} : D \to D^{\S}.$$

Then for $x, y \in |D^{\S}|$ we can define

$$\langle x, y\rangle = \{\Gamma\} \cup \{1X \cup 2Y \mid X \in x \text{ and } Y \in y\}.$$

We have another isomorphic injection

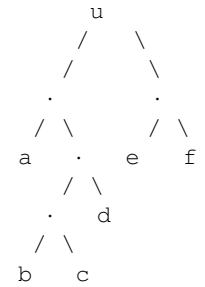
$$\lambda x, y.\, \langle x, y\rangle : D^{\S} \times D^{\S} \to D^{\S}.$$

Indeed by looking at the neighbourhood definition of $D^{\S}$ we conclude that the *finite* elements of $D^{\S}$ are exactly those that are either of the form (i) $\bot$, or (ii) $a^{\S}$, where a is finite in $|D|$, or (iii) $\langle a, b\rangle$, where a and b are previously obtained finite elements of $|D^{\S}|$.

Suppose $a, \ldots, f$ are finite in $|D|$. We can picture the element

$$u = \langle\langle a^{\S}, \langle\langle b^{\S}, c^{\S}\rangle, d^{\S}\rangle\rangle, \langle e^{\S}, f^{\S}\rangle\rangle$$

in $|D^{\S}|$ as a tree:

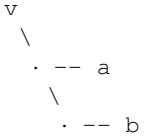


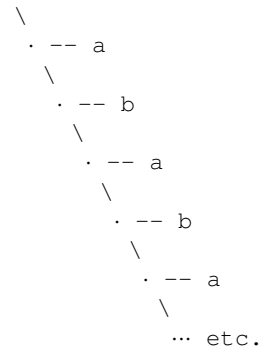
Note that the tree has binary branching with the elements of $|D|$ at the ends of the branches. Any such tree could be given a notation as an element of $|D^{\S}|$. The finite elements of $|D^{\S}|$ correspond exactly to such finite trees.

What of the infinite elements of $|D^{\S}|$? Are there infinite trees? Let $a, b \in |D^{\S}|$ be any elements of $|D^{\S}|$. Since pairing is an approximable mapping, we can solve the fixed-point equation

$$v = \langle a, \langle b, v\rangle\rangle.$$

In pictures we can diagram v roughly as:





The word is "roughly" here, since if a or b were not in the $|D|$ part of $|D^{\S}|$, then in the diagram the letters "a" and "b" should be replaced by the corresponding tree diagrams for a and b.

Suppose that a and b are finite. Then we can easily see that the infinite tree v is the limit of the following sequence of finite trees:

$$v_0 = \bot,$$

$$v_{n+1} = \langle a, \langle b, v_n \rangle \rangle,$$

and

$$v = \bigcup_{n=0}^{\infty} v_n.$$

The reader should think how to explain from tree diagrams the approximation relation $v_n \sqsubseteq v$ and more general such relationships.

We could call $D^{\S}$ a tree algebra over D. There may be others. A general one is a structure of the form

$$\langle E, \text{in}, \text{pair} \rangle,$$

where

$$\text{in} : D \to E, \text{ and}$$

$$\text{pair} : E \times E \to E.$$

The algebra

$$\langle D^{\S}, \lambda x.\, x^{\S}, \lambda x, y.\, \langle x, y \rangle \rangle,$$

however, is a very special one: it is "minimal" among all tree algebras over D in a sense we shall have to make precise.

To do this think of how E and $D^{\S}$ can differ. In view of the isomorphism that $D^{\S}$ satisfies, the injection of D and the pairing are one-one, so no "information" is lost by these mappings. The same may not at all be true of E, but it is reasonable to think that at least we can define an approximable mapping $g : D^{\S} \to E$ where

$$(1) \quad g(\bot) = \bot_E,$$

$$(2) \quad g(x^{\S}) = \text{in}(x), \text{ for } x \in |D|, \text{ and}$$

$$(3) \quad g(\langle x, y\rangle) = \text{pair}(g(x), g(y)), \text{ for } x, y \in |D^{\S}|.$$

By what we said earlier, g will be uniquely determined by (1)–(3), because these equations tell us exactly how to calculate g on all finite elements of $|D^{\S}|$. An approximable mapping is always determined by its action on the finite elements. But why does g exist?

It would not be too hard to give an inductive construction of g as a neighbourhood relation, but a fixed-point equation is easier to write down for the same purpose. We need, though, to have the inverse ("predecessor") functions:

$$\text{out} : D^{\S} \to D$$

$$\text{proj}_i : D^{\S} \to D^{\S}, \text{ for } i = 0, 1,$$

where

$$\text{out}(x^{\S}) = x,$$

$$\text{proj}_0(\langle x, y\rangle) = x, \text{ and}$$

$$\text{proj}_1(\langle x, y\rangle) = y.$$

We also need

$$\text{atom} : D^{\S} \to T,$$

where

$$\text{atom}(x^{\S}) = \text{true}, \text{ and}$$

$$\text{atom}(\langle x, y\rangle) = \text{false}.$$

We can then write

$$g(x) = \text{cond}\big(\text{atom}(x),\ \text{in}(\text{out}(x)),\ \text{pair}(g(\text{proj}_0(x)),\ g(\text{proj}_1(x)))\big).$$

This g exists by fixed-point theory, and it satisfies (1)–(3) by what we know about the structure of $|D^{\S}|$. As we said, g is unique because the values on finite elements are fixed.

In algebraic language g is a *homomorphism* of tree algebras; and $D^{\S}$ is called an *initial algebra*, because for any tree algebra E there is a unique homomorphism $g : D^{\S} \to E$. We note at once that any two initial algebras are isomorphic. For if D^* were another, there would exist homomorphisms in both directions between $D^{\S}$ and D^*. But the compositions of homomorphisms are again homomorphisms, and in the case

of $D^{\S}$ if we go from $D^{\S}$ to D^* and back to $D^{\S}$, the result must be the identity. The reason is that the identity can be the only homomorphism of an initial algebra into itself. We thus have a precise meaning of the minimal character of $D^{\S}$. But note it still took a construction to show that the domain $D^{\S}$ exists. □

pointwiseSubbasic_scottOpen.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ScottOpen
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- Each pointwise-subbasic set `{f | f x ∈ U}` (`U` Scott-open) is Scott-open in the
    lattice
`[D → D']`, because suprema there are pointwise. This is the easy inclusion pointwise
    ⊆ Scott. -/
theorem pointwiseSubbasic_scottOpen (x : D) {U : Set D'} (hU : ScottOpen U) :
    ScottOpen {f : ScottMap D D' | (f : D → D') x ∈ U} := by
  refine ⟨fun f f' hff' hf => hU.1 (hff' x) hf, fun F hFne hFdir hmem => ?_⟩
  simp only [Set.mem_ofPred_eq, ScottMap.sSup_apply] at hmem
  have hne : (Set.image (fun g : ScottMap D D' => (g : D → D') x) F).Nonempty := hFne.
    image _
  have hdir : DirectedOn (· ≤ ·) (Set.image (fun g : ScottMap D D' => (g : D → D') x)
    F) := by
    rintro _ ⟨a, ha, rfl⟩ _ ⟨b, hb, rfl⟩
    obtain ⟨c, hc, hac, hbc⟩ := hFdir a ha b hb
    exact ⟨(c : D → D') x, Set.mem_image_of_mem _ hc, hac x, hbc x⟩
  obtain ⟨_, ⟨g, hgF, rfl⟩, hg⟩ := hU.2 hne hdir hmem
  exact ⟨g, hgF, hg⟩

end DomainTheory.ContinuousLattice
```

0

3.7.3 Example 6.2 — sequence domain equations

EXAMPLE 6.2. Our staple examples B and C satisfy "domain equations" in the form of isomorphisms as we have previously seen. Indeed

$$B \cong B + B, \text{ and}$$

$$C \cong \{\{\Lambda\}\} + C + C,$$

where if we liked we could construct both systems over $\{0,1\}^*$ and have:

$$B = \{\{0,1\}^*\} \cup \{0X \mid X \in B\} \cup \{1X \mid X \in B\}, \text{ and}$$

$$C = \{\{0,1\}^*\} \cup \{\{\Lambda\}\} \cup \{0X \mid X \in C\} \cup \{1X \mid X \in C\}.$$

We leave to the exercises the explanations of what kinds of algebras B and C are and why they are initial. Here we want to propose a simple, yet interesting generalization of B.

Consider this domain equation:

$$A \cong A^n + A^n,$$

where A^n stands for the n-fold cartesian power of A. We can, with the aid of some encoding solve this equation as a neighbourhood system over $\{0,1\}^*$ as follows:

$$A = \{\{0,1\}^*\} \cup \bigcup_{i=0,1} \left\{ i \bigcup_{j<n} 1^j 0 X_j \mid X_j \in A \text{ for all } j < n \right\}.$$

For instance, if $n = 3$, then a typical neighbourhood in A is something like

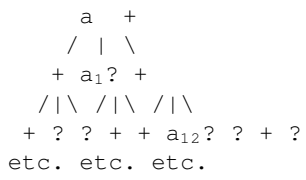
$$00X_0 \cup 010X_1 \cup 0110X_2,$$

where $X_0, X_1, X_2 \in A$. The first '0' could also be a '1' in front of each of the terms.

In words, an element of A (other than $\perp$) is an n-tuple of elements of A: but there are two separate copies of these, the left one and the right one. We can write for $a \in |A|$

$$a = \pm\langle a_0, a_1, \ldots, a_{n-1}\rangle,$$

where $+$ is chosen if a is on the right, and $-$ if on the left. As a tree diagram a might look like this for $n = 3$:



That is, a is an infinite ternary tree with $+$ or $-$ labels at each node. If each node (subtree) is truly infinite, the element is *total*; if $\perp$ is ever encountered, it is only *partial*; if every branch ends with $\perp$, the tree is a *finite element* of $|A|$.

What can be done with such trees? Let $\sigma \in \{0,1,\ldots,n-1\}^*$ be a finite sequence of "digits" each less than n. We let $\Sigma = \{0,1,\ldots,n-1\}$. We can define for $a \in |A|$ the operation $\sigma \mapsto a\sigma$ by recursion on σ:

$$a\Lambda = a, \text{ and}$$

$$ai\sigma = (a_i)\sigma.$$

The $a\sigma$ are just the *subtrees* of a with σ as a *selector*. We also have a map

$$\mathrm{pos} : A \to T$$

where

$$\mathrm{pos}(+\langle a_0, a_1, \ldots, a_{n-1}\rangle) = \mathrm{true}, \text{ and}$$

$$\mathrm{pos}(-\langle a_0, a_1, \ldots, a_{n-1}\rangle) = \mathrm{false}.$$

We say that a (total) tree a is *eventually periodic* iff the set $\{a\sigma \mid \sigma \in \Sigma^*\}$ is finite. The result is that the "language"

$$L_a = \{\sigma \in \Sigma^* \mid \mathrm{pos}(a\sigma) = \mathrm{true}\}$$

corresponding to an eventually periodic tree is always a *regular event* of automata theory, and every such language has this form. In fact, a just represents the initial state of an automaton, and $a\sigma$ represents the state after "reading" a tape σ. □

In order to formulate more generally the idea of a domain equation and initial algebra, we must introduce a small amount of the terminology of category theory. To be as specific as possible, think of systems D over sets $\Delta \subseteq \Sigma^*$ with $\Sigma = \{0, 1\}$, say. They form quite an interesting category with respect to the approximable maps $f : D \to D'$. Recall that to be a category of "domains" and "maps" all that is required is an associative composition $g \circ f$ of maps with identity maps $I : D \to D$ for each domain of the category. And this we certainly have for the systems indicated. And there are many other categories waiting around: for instance, restrict systems to those where $\emptyset \notin D$. This is not much of a restriction, as every system is isomorphic to one like this. Or restrict the maps to being the strict maps $f : D \to D'$ where $f(\bot_D) = \bot_{D'}$. This is an essentially different, though related category. We shall find many others.

What examples 6.1 and 6.2 suggest is the notion of a construct which makes new domains out of old. For example, with D fixed, 6.1 suggests for any domain X over $\Gamma \subseteq \Sigma^*$ a domain

$$T(X) = D + (X \times X).$$

More specifically (converting from $\Sigma = \{0, 1, 2\}$ to $\Sigma = \{0, 1\}$) we could write

$$T(X) = \{\Gamma'\} \cup \{0X \mid X \in D\} \cup \{10X \cup 11Y \mid X, Y \in X\},$$

where we have $\Gamma' = 0\Delta \cup 10\Gamma \cup 11\Gamma$. (By the way, here we definitely want to assume $\emptyset \notin D$ and $\emptyset \notin X$ and to get $\emptyset \notin T(X)$.) This construct is an example of a *functor*, a notion that can be defined abstractly on any category.

`pointwiseSubbasic_scottOpen.lean`

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ScottOpen
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic
```

```
@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- Each pointwise-subbasic set `{f | f x ∈ U}` (`U` Scott-open) is Scott-open in the
    lattice
`[D → D']`, because suprema there are pointwise. This is the easy inclusion pointwise
    ⊆ Scott. -/
theorem pointwiseSubbasic_scottOpen (x : D) {U : Set D'} (hU : ScottOpen U) :
    ScottOpen {f : ScottMap D D' | (f : D → D') x ∈ U} := by
  refine ⟨fun f f' hff' hf => hU.1 (hff' x) hf, fun F hFne hFdir hmem => ?_⟩
  simp only [Set.mem_ofPred_eq, ScottMap.sSup_apply] at hmem
  have hne : (Set.image (fun g : ScottMap D D' => (g : D → D') x) F).Nonempty := hFne.
    image _
  have hdir : DirectedOn (· ≤ ·) (Set.image (fun g : ScottMap D D' => (g : D → D') x)
    F) := by
    rintro _ ⟨a, ha, rfl⟩ _ ⟨b, hb, rfl⟩
    obtain ⟨c, hc, hac, hbc⟩ := hFdir a ha b hb
    exact ⟨(c : D → D') x, Set.mem_image_of_mem _ hc, hac x, hbc x⟩
  obtain ⟨_, ⟨g, hgF, rfl⟩, hg⟩ := hU.2 hne hdir hmem
  exact ⟨g, hgF, hg⟩

end DomainTheory.ContinuousLattice
```

1

3.7.4 Definition 6.3 — functor

DEFINITION 6.3. A *functor* on a category (into itself) associates with every domain X in the category another domain $T(X)$ and to every map

$$f : X \to Y$$

another map

$$T(f) : T(X) \to T(Y)$$

in such a way that identity maps and compositions are preserved:

$$T(I_X) = I_{T(X)}, \text{ and}$$

$$T(g \circ f) = T(g) \circ T(f),$$

whenever $f : X \to Y$ and $g : Y \to Z$. □

In the example from 6.1 we have not checked how the special **T** is a functor. The hint is that whenever $f : X \to Y$, then there is a map

$$f \times f : X \times X \to Y \times Y.$$

But there is also a map

$$I_D + f \times f : D + (X \times X) \to D + (Y \times Y)$$

and this suggests the definition of $T(f)$. The details are left to the exercises. Note that the map $T(f)$ just suggested is always strict, so **T** is a functor also for the category of strict maps.

One good reason for a little of the category-theoretic language is that the next definition becomes very neat indeed.

pointwiseSubbasic_scottOpen.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ScottOpen
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- Each pointwise-subbasic set `{f | f x ∈ U}` (`U` Scott-open) is Scott-open in the
    lattice
`[D → D']`, because suprema there are pointwise. This is the easy inclusion pointwise
    ⊆ Scott. -/
theorem pointwiseSubbasic_scottOpen (x : D) {U : Set D'} (hU : ScottOpen U) :
    ScottOpen {f : ScottMap D D' | (f : D → D') x ∈ U} := by
  refine ⟨fun f f' hff' hf => hU.1 (hff' x) hf, fun F hFne hFdir hmem => ?_⟩
  simp only [Set.mem_ofPred_eq, ScottMap.sSup_apply] at hmem
  have hne : (Set.image (fun g : ScottMap D D' => (g : D → D') x) F).Nonempty := hFne.
    image _
  have hdir : DirectedOn (· ≤ ·) (Set.image (fun g : ScottMap D D' => (g : D → D') x)
    F) := by
    rintro _ ⟨a, ha, rfl⟩ _ ⟨b, hb, rfl⟩
    obtain ⟨c, hc, hac, hbc⟩ := hFdir a ha b hb
    exact ⟨(c : D → D') x, Set.mem_image_of_mem _ hc, hac x, hbc x⟩
  obtain ⟨_, ⟨g, hgF, rfl⟩, hg⟩ := hU.2 hne hdir hmem
  exact ⟨g, hgF, hg⟩

end DomainTheory.ContinuousLattice
```

2

3.7.5 Definition 6.4 — algebra for a functor

DEFINITION 6.4. A ***T*-algebra** is a domain E in the category together with a map

$$k : T(E) \to E.$$

If $m : T(F) \to F$ is another ***T*-algebra**, then a *homomorphism* is a map $h : E \to F$ in the category such that the diagram

$$\begin{array}{ccc} T(E) & \stackrel{k}{\to} & E \\ \downarrow^{T(h)} & & \downarrow^{h} \\ T(F) & \stackrel{m}{\to} & F \end{array}$$

commutes; that is, the equation

$$h \circ k = m \circ T(h)$$

holds. □

In our example from 6.1 a T-algebra is a *strict* map

$$k : D + (E \times E) \to E.$$

But such strict maps are in a one-one correspondence with pairs of (not necessarily strict) maps

$$n : D \to E \quad \text{and} \quad p : E \times E \to E.$$

And the structure $\langle E, n, p \rangle$ is what we called a tree algebra. Definition 6.4 just makes this abstract. The reader should also work out the details showing that 6.4's definition of homomorphism is just what we ought to expect.

Note that the T-algebras and homomorphisms form a category. (Why?) The following definition is so abstract that it could be given for any category.

pointwiseSubbasic_scottOpen.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ScottOpen
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
```

```
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- Each pointwise-subbasic set `{f | f x ∈ U}` (`U` Scott-open) is Scott-open in the
    lattice
`[D → D']`, because suprema there are pointwise. This is the easy inclusion pointwise
    ⊆ Scott. -/
theorem pointwiseSubbasic_scottOpen (x : D) {U : Set D'} (hU : ScottOpen U) :
    ScottOpen {f : ScottMap D D' | (f : D → D') x ∈ U} := by
  refine ⟨fun f f' hff' hf => hU.1 (hff' x) hf, fun F hFne hFdir hmem => ?_⟩
  simp only [Set.mem_ofPred_eq, ScottMap.sSup_apply] at hmem
  have hne : (Set.image (fun g : ScottMap D D' => (g : D → D') x) F).Nonempty := hFne.
    image _
  have hdir : DirectedOn (· ≤ ·) (Set.image (fun g : ScottMap D D' => (g : D → D') x)
    F) := by
    rintro _ ⟨a, ha, rfl⟩ _ ⟨b, hb, rfl⟩
    obtain ⟨c, hc, hac, hbc⟩ := hFdir a ha b hb
    exact ⟨(c : D → D') x, Set.mem_image_of_mem _ hc, hac x, hbc x⟩
  obtain ⟨_, ⟨g, hgF, rfl⟩, hg⟩ := hU.2 hne hdir hmem
  exact ⟨g, hgF, hg⟩

end DomainTheory.ContinuousLattice
```

3

3.7.6 Definition 6.5 — initial algebra

DEFINITION 6.5. A T-algebra is *initial* if and only if there is a unique homomorphism from it into any other T-algebra. □

The word "other" here is not meant to imply "distinct". For an initial algebra there is one and only one homomorphism into itself: the identity map. As we already indicated in 6.1 it is a general fact that the next proposition holds.

pointwiseSubbasic_scottOpen.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ScottOpen
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']
```

```
/-- Each pointwise-subbasic set `{f | f x ∈ U}` (`U` Scott-open) is Scott-open in the
    lattice
`[D → D']`, because suprema there are pointwise. This is the easy inclusion pointwise
    ⊆ Scott. -/
theorem pointwiseSubbasic_scottOpen (x : D) {U : Set D'} (hU : ScottOpen U) :
    ScottOpen {f : ScottMap D D' | (f : D → D') x ∈ U} := by
  refine ⟨fun f f' hff' hf => hU.1 (hff' x) hf, fun F hFne hFdir hmem => ?_⟩
  simp only [Set.mem_ofPred_eq, ScottMap.sSup_apply] at hmem
  have hne : (Set.image (fun g : ScottMap D D' => (g : D → D') x) F).Nonempty := hFne.
    image _
  have hdir : DirectedOn (· ≤ ·) (Set.image (fun g : ScottMap D D' => (g : D → D') x)
    F) := by
    rintro _ ⟨a, ha, rfl⟩ _ ⟨b, hb, rfl⟩
    obtain ⟨c, hc, hac, hbc⟩ := hFdir a ha b hb
    exact ⟨(c : D → D') x, Set.mem_image_of_mem _ hc, hac x, hbc x⟩
  obtain ⟨_, ⟨g, hgF, rfl⟩, hg⟩ := hU.2 hne hdir hmem
  exact ⟨g, hgF, hg⟩

end DomainTheory.ContinuousLattice
```

4

3.7.7 Proposition 6.6 — uniqueness of initial algebras

PROPOSITION 6.6. Any two initial T-algebras are uniquely isomorphic. □

Slightly more interesting is the behaviour of T on initial algebras.

pointwiseSubbasic_scottOpen.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ScottOpen
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- Each pointwise-subbasic set `{f | f x ∈ U}` (`U` Scott-open) is Scott-open in the
    lattice
`[D → D']`, because suprema there are pointwise. This is the easy inclusion pointwise
    ⊆ Scott. -/
theorem pointwiseSubbasic_scottOpen (x : D) {U : Set D'} (hU : ScottOpen U) :
```

```
    ScottOpen {f : ScottMap D D' | (f : D → D') x ∈ U} := by
  refine ⟨fun f f' hff' hf => hU.1 (hff' x) hf, fun F hFne hFdir hmem => ?_⟩
  simp only [Set.mem_ofPred_eq, ScottMap.sSup_apply] at hmem
  have hne : (Set.image (fun g : ScottMap D D' => (g : D → D') x) F).Nonempty := hFne.
    image _
  have hdir : DirectedOn (· ≤ ·) (Set.image (fun g : ScottMap D D' => (g : D → D') x)
    F) := by
    rintro _ ⟨a, ha, rfl⟩ _ ⟨b, hb, rfl⟩
    obtain ⟨c, hc, hac, hbc⟩ := hFdir a ha b hb
    exact ⟨(c : D → D') x, Set.mem_image_of_mem _ hc, hac x, hbc x⟩
  obtain ⟨_, ⟨g, hgF, rfl⟩, hg⟩ := hU.2 hne hdir hmem
  exact ⟨g, hgF, hg⟩

end DomainTheory.ContinuousLattice
```

5

3.7.8 Proposition 6.7 — functor preserves initiality

PROPOSITION 6.7. If $i : T(D) \to D$ is an initial T-algebra, then so is $T(i) : T^2(D) \to T(D)$ and i is the isomorphism from $T(D)$ to D.

Proof: Clearly since T is a functor, the map $T(i)$ has the right mapping character to make $T(D)$ a T-algebra. Since D is initial, we have a commuting diagram:

$$\begin{array}{ccc} T(D) & \xrightarrow{i} & D \\ \downarrow{\scriptstyle T(j)} & & \downarrow{\scriptstyle j} \\ T^2(D) & \xrightarrow{T(i)} & T(D) \end{array}$$

But we also have the trivial diagram:

$$\begin{array}{ccc} T^2(D) & \xrightarrow{T(i)} & T(D) \\ \downarrow{\scriptstyle T(i)} & & \downarrow{\scriptstyle i} \\ T(D) & \xrightarrow{i} & D \end{array}$$

It follows that $i \circ j$ is a homomorphism, so

$$i \circ j = I_D.$$

But then because T is a functor we find:

$$T(i) \circ T(j) = I_{T(D)},$$

and, since j is a homomorphism, we have

$$j \circ i = I_{T(D)}.$$

This shows that i is an isomorphism. □

From 6.7 we see that if we are going to have initial algebras at all we have to satisfy the domain equation

$$D \cong T(D).$$

But generally that is not enough to assure that D is initial. There is a condition that our functors satisfy, however, which guarantees the existence of homomorphisms.

pointwiseSubbasic_scottOpen.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ScottOpen
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- Each pointwise-subbasic set `{f | f x ∈ U}` (`U` Scott-open) is Scott-open in the
    lattice
`[D → D']`, because suprema there are pointwise. This is the easy inclusion pointwise
    ⊆ Scott. -/
theorem pointwiseSubbasic_scottOpen (x : D) {U : Set D'} (hU : ScottOpen U) :
    ScottOpen {f : ScottMap D D' | (f : D → D') x ∈ U} := by
  refine ⟨fun f f' hff' hf => hU.1 (hff' x) hf, fun F hFne hFdir hmem => ?_⟩
  simp only [Set.mem_ofPred_eq, ScottMap.sSup_apply] at hmem
  have hne : (Set.image (fun g : ScottMap D D' => (g : D → D') x) F).Nonempty := hFne.
    image _
  have hdir : DirectedOn (· ≤ ·) (Set.image (fun g : ScottMap D D' => (g : D → D') x)
    F) := by
    rintro _ ⟨a, ha, rfl⟩ _ ⟨b, hb, rfl⟩
    obtain ⟨c, hc, hac, hbc⟩ := hFdir a ha b hb
    exact ⟨(c : D → D') x, Set.mem_image_of_mem _ hc, hac x, hbc x⟩
  obtain ⟨_, ⟨g, hgF, rfl⟩, hg⟩ := hU.2 hne hdir hmem
  exact ⟨g, hgF, hg⟩

end DomainTheory.ContinuousLattice
```

6

3.7.9 Definition 6.8 — continuity on maps

DEFINITION 6.8. On the category of domains and strict approximable maps a functor T is *continuous on maps* if for any systems D and E the induced mapping

$$\lambda f.\, T(f) : (D \to_\perp E) \to (T(D) \to_\perp T(E))$$

is approximable.

pointwiseSubbasic_scottOpen.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ScottOpen
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- Each pointwise-subbasic set `{f | f x ∈ U}` (`U` Scott-open) is Scott-open in the
    lattice
`[D → D']`, because suprema there are pointwise. This is the easy inclusion pointwise
    ⊆ Scott. -/
theorem pointwiseSubbasic_scottOpen (x : D) {U : Set D'} (hU : ScottOpen U) :
    ScottOpen {f : ScottMap D D' | (f : D → D') x ∈ U} := by
  refine ⟨fun f f' hff' hf => hU.1 (hff' x) hf, fun F hFne hFdir hmem => ?_⟩
  simp only [Set.mem_ofPred_eq, ScottMap.sSup_apply] at hmem
  have hne : (Set.image (fun g : ScottMap D D' => (g : D → D') x) F).Nonempty := hFne.
    image _
  have hdir : DirectedOn (· ≤ ·) (Set.image (fun g : ScottMap D D' => (g : D → D') x)
    F) := by
    rintro _ ⟨a, ha, rfl⟩ _ ⟨b, hb, rfl⟩
    obtain ⟨c, hc, hac, hbc⟩ := hFdir a ha b hb
    exact ⟨(c : D → D') x, Set.mem_image_of_mem _ hc, hac x, hbc x⟩
  obtain ⟨_, ⟨g, hgF, rfl⟩, hg⟩ := hU.2 hne hdir hmem
  exact ⟨g, hgF, hg⟩

end DomainTheory.ContinuousLattice
```

7

3.7.10 Theorem 6.9 — homomorphism from initial algebra

THEOREM 6.9. If the functor T is continuous on maps and if $D \cong T(D)$, so in particular D is a T-algebra, then for any T-algebra $k : T(E) \to E$ there is a homomorphism $h : D \to E$.

Proof: Let $i : T(D) \to D$ make D a T-algebra, where $j : D \to T(D)$ is the inverse so that i is an isomorphism of domains. Suppose that $k : T(E) \to E$ is any T-algebra. A homomorphism $h : D \to E$ would satisfy

$$h \circ i = k \circ T(h).$$

Rewrite this equation as

$$h = k \circ T(h) \circ j.$$

In the domain of strict maps $(D \to E)$ this is a fixed-point equation for an approximable map

$$\lambda h.\, k \circ T(h) \circ j$$

by our assumption on T. Thus, the desired homomorphism exists. □

The final question we have to answer is why in our category the minimal D exist. The reason is that the functors T that we have in mind possess further continuity properties on domains. This is conveniently expressed in terms of a notion of "subdomain".

pointwiseSubbasic_scottOpen.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ScottOpen
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- Each pointwise-subbasic set `{f | f x ∈ U}` (`U` Scott-open) is Scott-open in the
    lattice
`[D → D']`, because suprema there are pointwise. This is the easy inclusion pointwise
    ⊆ Scott. -/
theorem pointwiseSubbasic_scottOpen (x : D) {U : Set D'} (hU : ScottOpen U) :
    ScottOpen {f : ScottMap D D' | (f : D → D') x ∈ U} := by
  refine ⟨fun f f' hff' hf => hU.1 (hff' x) hf, fun F hFne hFdir hmem => ?_⟩
  simp only [Set.mem_ofPred_eq, ScottMap.sSup_apply] at hmem
  have hne : (Set.image (fun g : ScottMap D D' => (g : D → D') x) F).Nonempty := hFne.
    image _
  have hdir : DirectedOn (· ≤ ·) (Set.image (fun g : ScottMap D D' => (g : D → D') x)
    F) := by
    rintro _ ⟨a, ha, rfl⟩ _ ⟨b, hb, rfl⟩
    obtain ⟨c, hc, hac, hbc⟩ := hFdir a ha b hb
    exact ⟨(c : D → D') x, Set.mem_image_of_mem _ hc, hac x, hbc x⟩
  obtain ⟨_, ⟨g, hgF, rfl⟩, hg⟩ := hU.2 hne hdir hmem
  exact ⟨g, hgF, hg⟩

end DomainTheory.ContinuousLattice
```

3.7.11 Definition 6.10 — subdomain

DEFINITION 6.10. For two neighbourhood systems D and E we write

$$D \triangleleft E$$

to mean that these are neighbourhood systems over the same set of tokens Δ and not only is $D \subseteq E$ but whenever $X, Y \in D$ and $X \cap Y \in E$, then $X \cap Y \in D$. $\square$

For the subdomain relation $D \triangleleft E$ to hold, D has to be a smaller family of neighbourhoods, but the notion of consistency in D also has to be the same as in E. Note that if $D_0 \triangleleft E$ and $D_1 \triangleleft E$ then

$$D_0 \triangleleft D_1 \text{ iff } D_0 \subseteq D_1.$$

It is also easy to prove that the union of a directed family of subdomains of E is again a subdomain. As a consequence of this remark we have:

pointwiseSubbasic_scottOpen.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ScottOpen
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- Each pointwise-subbasic set `{f | f x ∈ U}` (`U` Scott-open) is Scott-open in the
    lattice
`[D → D']`, because suprema there are pointwise. This is the easy inclusion pointwise
    ⊆ Scott. -/
theorem pointwiseSubbasic_scottOpen (x : D) {U : Set D'} (hU : ScottOpen U) :
    ScottOpen {f : ScottMap D D' | (f : D → D') x ∈ U} := by
  refine ⟨fun f f' hff' hf => hU.1 (hff' x) hf, fun F hFne hFdir hmem => ?_⟩
  simp only [Set.mem_ofPred_eq, ScottMap.sSup_apply] at hmem
  have hne : (Set.image (fun g : ScottMap D D' => (g : D → D') x) F).Nonempty := hFne.
    image _
  have hdir : DirectedOn (· ≤ ·) (Set.image (fun g : ScottMap D D' => (g : D → D') x)
    F) := by
    rintro _ ⟨a, ha, rfl⟩ _ ⟨b, hb, rfl⟩
    obtain ⟨c, hc, hac, hbc⟩ := hFdir a ha b hb
```

```
    exact ⟨(c : D → D') x, Set.mem_image_of_mem _ hc, hac x, hbc x⟩
  obtain ⟨_, ⟨g, hgF, rfl⟩, hg⟩ := hU.2 hne hdir hmem
  exact ⟨g, hgF, hg⟩

end DomainTheory.ContinuousLattice
```

9

3.7.12 Proposition 6.11 — lattice of subsystems

PROPOSITION 6.11. For a given neighbourhood system E, the set of subsystems

$$\{D \mid D \triangleleft E\}$$

forms a domain in its own right. □

The subdomain relationship implies a mapping relationship between the domains.

pointwise_sSup_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

theorem pointwise_sSup_preservesDirectedSup (F : Set (ScottMap D D')) :
    PreservesDirectedSup (fun x => sSup (Set.image (fun f : ScottMap D D' => (f : D →
    D') x) F)) := by
  intro S hS hSdir
  set H := fun x => sSup (Set.image (fun f : ScottMap D D' => (f : D → D') x) F)
  show H (sSup S) = sSup (Set.image H S)
  apply le_antisymm
  · apply sSup_le
    intro b hb
    obtain ⟨f, hfF, rfl⟩ := hb
```

```
    change f (sSup S) ≤ sSup (Set.image H S)
    rw [f.preservesDirectedSup_coe S hS hSdir]
    apply sSup_le
    intro c hc
    obtain ⟨s, hsS, rfl⟩ := hc
    exact le_trans (le_sSup (Set.mem_image_of_mem (fun g : ScottMap D D' => (g : D → D
    ') s) hfF))
      (le_sSup (Set.mem_image_of_mem H hsS))
  · apply sSup_le
    intro b hb
    obtain ⟨s, hsS, rfl⟩ := hb
    apply sSup_le
    intro c hc
    obtain ⟨f, hfF, rfl⟩ := hc
    exact le_trans (ScottMap.monotone f (le_sSup hsS))
      (le_sSup (Set.mem_image_of_mem (fun g : ScottMap D D' => (g : D → D') (sSup S))
    hfF))

end ScottMap

end DomainTheory.ContinuousLattice
```

0

3.7.13 Proposition 6.12 — projection pair

PROPOSITION 6.12. If $D \triangleleft E$, then there exists a projection pair of approximable mappings:

$$i : D \to E \quad \text{and} \quad j : E \to D$$

where $j \circ i = I_D$ and $i \circ j \subseteq I_E$, which are determined as element-wise functions by these equations:

$$i(x) = \{Y \in E \mid \exists X \in x.\, X \subseteq Y\},$$

and

$$j(y) = y \cap D,$$

for all $x \in |D|$ and $y \in |E|$. □

The proof is left for the exercises.

pointwise_sSup_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section
```

```
namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

theorem pointwise_sSup_preservesDirectedSup (F : Set (ScottMap D D')) :
    PreservesDirectedSup (fun x => sSup (Set.image (fun f : ScottMap D D' => (f : D →
    D') x) F)) := by
  intro S hS hSdir
  set H := fun x => sSup (Set.image (fun f : ScottMap D D' => (f : D → D') x) F)
  show H (sSup S) = sSup (Set.image H S)
  apply le_antisymm
  · apply sSup_le
    intro b hb
    obtain ⟨f, hfF, rfl⟩ := hb
    change f (sSup S) ≤ sSup (Set.image H S)
    rw [f.preservesDirectedSup_coe S hS hSdir]
    apply sSup_le
    intro c hc
    obtain ⟨s, hsS, rfl⟩ := hc
    exact le_trans (le_sSup (Set.mem_image_of_mem (fun g : ScottMap D D' => (g : D → D
    ') s) hfF))
      (le_sSup (Set.mem_image_of_mem H hsS))
  · apply sSup_le
    intro b hb
    obtain ⟨s, hsS, rfl⟩ := hb
    apply sSup_le
    intro c hc
    obtain ⟨f, hfF, rfl⟩ := hc
    exact le_trans (ScottMap.monotone f (le_sSup hsS))
      (le_sSup (Set.mem_image_of_mem (fun g : ScottMap D D' => (g : D → D') (sSup S))
    hfF))

end ScottMap

end DomainTheory.ContinuousLattice
```

1

3.7.14 Definition 6.13 — continuity on domains

DEFINITION 6.13. A functor T is *monotone on domains* iff whenever $D \triangleleft E$, then not only do we have $T(D) \triangleleft T(E)$ but the projection pair i, j of 6.12 is mapped to the same kind of projection pair $T(i), T(j)$. A monotone functor is *continuous on domains* iff whenever E is a domain, then the mapping

$$\lambda D.\, T(D) : \{D \mid D \triangleleft E\} \to \{D' \mid D' \triangleleft T(E)\}$$

is approximable. □

We can now state an existence theorem that covers in fairly wide generality the examples of this lecture.

pointwise_sSup_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

theorem pointwise_sSup_preservesDirectedSup (F : Set (ScottMap D D')) :
    PreservesDirectedSup (fun x => sSup (Set.image (fun f : ScottMap D D' => (f : D →
    D') x) F)) := by
  intro S hS hSdir
  set H := fun x => sSup (Set.image (fun f : ScottMap D D' => (f : D → D') x) F)
  show H (sSup S) = sSup (Set.image H S)
  apply le_antisymm
  · apply sSup_le
    intro b hb
    obtain ⟨f, hfF, rfl⟩ := hb
    change f (sSup S) ≤ sSup (Set.image H S)
    rw [f.preservesDirectedSup_coe S hS hSdir]
    apply sSup_le
    intro c hc
    obtain ⟨s, hsS, rfl⟩ := hc
    exact le_trans (le_sSup (Set.mem_image_of_mem (fun g : ScottMap D D' => (g : D → D
    ') s) hfF))
      (le_sSup (Set.mem_image_of_mem H hsS))
  · apply sSup_le
    intro b hb
    obtain ⟨s, hsS, rfl⟩ := hb
    apply sSup_le
```

```
    intro c hc
    obtain ⟨f, hfF, rfl⟩ := hc
    exact le_trans (ScottMap.monotone f (le_sSup hsS))
      (le_sSup (Set.mem_image_of_mem (fun g : ScottMap D D' => (g : D → D') (sSup S))
    hfF))

end ScottMap

end DomainTheory.ContinuousLattice
```

2

3.7.15 Theorem 6.14 — existence of initial algebras

THEOREM 6.14. If the functor T is continuous on maps and monotone and continuous on domains, and if there is a set Γ such that

$$\{\Gamma\} \lhd T(\{\Gamma\}),$$

then there exists an initial T-algebra.

Proof: We proceed as in the proof of the fixed-point theorem by iterating the functor. The assumption about Γ means that, as a neighbourhood system, $T(\{\Gamma\})$ is a system over the same set Γ. Thus, if we iterate T to form $T^n(\{\Gamma\})$, all these systems are over Γ and indeed

$$T^n(\{\Gamma\}) \lhd T^{n+1}(\{\Gamma\})$$

for all n. We can thus introduce

$$\mathcal{D} = \bigcup_{n=0}^{\infty} T^n(\{\Gamma\}),$$

and it is easy to check that $\mathcal{D}$ is a system over Γ and

$$T^n(\{\Gamma\}) \lhd \mathcal{D}$$

holds for all n. But then we have for all n:

$$T^n(\{\Gamma\}) \lhd T^{n+1}(\{\Gamma\}) \lhd T(\mathcal{D}),$$

which implies $\mathcal{D} \lhd T(\mathcal{D})$. But T is continuous on domains, so

$$\begin{aligned} T(\mathcal{D}) &= T\left(\bigcup_{n=0}^{\infty} T^n(\{\Gamma\})\right) \\ &= \bigcup_{n=0}^{\infty} T^{n+1}(\{\Gamma\}) \\ &= \mathcal{D}. \end{aligned}$$

Thus, not only is $\mathcal{D}$ a T-algebra, but the isomorphism we get for $\mathcal{D}$ and $T(\mathcal{D})$ is just the identity mapping. We know by 6.9 that homomorphisms exist; what remains to show is that homomorphism from $\mathcal{D}$ are unique. As in the examples, we will show in effect they are determined uniquely on the finite elements of $\mathcal{D}$.

Since each $T^n(\{\Gamma\}) \triangleleft \mathcal{D}$, there are projection mappings $i_n : T^n(\{\Gamma\}) \to \mathcal{D}$ and $j_n : \mathcal{D} \to T^n(\{\Gamma\})$.

Define $\rho_n : \mathcal{D} \to \mathcal{D}$ by $\rho_n = i_n \circ j_n$. Projection pairs are always pairs of strict mappings (Why?), and so are in the category. By assumption and 6.13, the functor T preserves these maps, so we have

$$T(\rho_n) = T(i_n) \circ T(j_n) = i_{n+1} \circ j_{n+1} = \rho_{n+1}.$$

As a neighbourhood relation ρ_n can be characterized by :

$$X \rho_n Y \quad \text{iff} \quad \exists z \in T^n(\{\Gamma\}).\ X \subseteq z \subseteq Y.$$

We thus see that $\rho_n \subseteq \rho_{n+1}$ and

$$\bigcup_{n=0}^{\infty} \rho_n = I_{\mathcal{D}}.$$

Now suppose $k : T(E) \to E$ is any T-algebra and $h : \mathcal{D} \to E$ is a homomorphism. The mapping will satisfy the fixed-point equation

$$h = k \circ T(h),$$

where no other mappings need be written in because $\mathcal{D} = T(\mathcal{D})$ and so

$$T(h) : \mathcal{D} \to T(E).$$

We wish to show that h really is the least fixed point of this equation.

Define $h_n = h \circ \rho_n : \mathcal{D} \to E$. For $n = 0$, the map ρ_0 is the trivial map where $\rho_0(x) = \bot_{\mathcal{D}}$ for all $x \in |\mathcal{D}|$. But h must be strict, so $h_0(x) = \bot_E$ for all $x \in |\mathcal{D}|$; that is, h_0 is the least element of $|\mathcal{D} \to_\bot E|$. Now calculate :

$$\begin{aligned} k \circ T(h_n) &= k \circ T(h) \circ T(\rho_n) \\ &= h \circ \rho_{n+1} \\ &= h_{n+1}\ . \end{aligned}$$

This shows that the union of the h_n is the least fixed point of $\lambda h.\ k \circ T(h)$. But

$$\begin{aligned} \bigcup_{n=0}^{\infty} h_n &= \bigcup_{n=0}^{\infty} h \circ \rho_n \\ &= h \circ \bigcup_{n=0}^{\infty} \rho_n \\ &= h \circ I_D = h, \end{aligned}$$

so the given h is in fact the least fixed point. The homomorphism is uniquely determined, and D is the initial T-algebra. □

Having the existence of initial T-algebras, we can prove one more result that shows just how minimal they are. We need a lemma about projection pairs, first, that shows where sub-domains fit in. We write $D \trianglelefteq E$ as short for $D \cong D'$ for some $D' \triangleleft E$ in the following. The lemma gives a converse to 6.12.

pointwise_sSup_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

theorem pointwise_sSup_preservesDirectedSup (F : Set (ScottMap D D')) :
    PreservesDirectedSup (fun x => sSup (Set.image (fun f : ScottMap D D' => (f : D →
    D') x) F)) := by
  intro S hS hSdir
  set H := fun x => sSup (Set.image (fun f : ScottMap D D' => (f : D → D') x) F)
  show H (sSup S) = sSup (Set.image H S)
  apply le_antisymm
  · apply sSup_le
    intro b hb
    obtain ⟨f, hfF, rfl⟩ := hb
    change f (sSup S) ≤ sSup (Set.image H S)
    rw [f.preservesDirectedSup_coe S hS hSdir]
    apply sSup_le
    intro c hc
    obtain ⟨s, hsS, rfl⟩ := hc
    exact le_trans (le_sSup (Set.mem_image_of_mem (fun g : ScottMap D D' => (g : D → D
    ') s) hfF))
      (le_sSup (Set.mem_image_of_mem H hsS))
  · apply sSup_le
    intro b hb
    obtain ⟨s, hsS, rfl⟩ := hb
    apply sSup_le
    intro c hc
    obtain ⟨f, hfF, rfl⟩ := hc
    exact le_trans (ScottMap.monotone f (le_sSup hsS))
      (le_sSup (Set.mem_image_of_mem (fun g : ScottMap D D' => (g : D → D') (sSup S))
    hfF))

end ScottMap
```

```
end DomainTheory.ContinuousLattice
```

3

3.7.16 Lemma 6.15 — subdomains from projection pairs

LEMMA 6.15. For two neighbourhood systems D and E, if there exist a projection pair

$$i : D \to E \quad \text{and} \quad j : E \to D$$

with $j \circ i = I_D$ and $i \circ j \subseteq I_E$, then $D \trianglelefteq E$.

Proof. What we want to show is that i maps finite elements to finite elements, and that the desired D' is the image of D in E.

Suppose $X \in D$. We can write:

$$i(\uparrow X) = \bigcup \{\uparrow Y \mid Y \in i(\uparrow X)\} .$$

Applying j to both sides we have:

$$\uparrow X = j \circ i(\uparrow X) = \bigcup \{j(\uparrow Y) \mid Y \in i(\uparrow X)\}.$$

But then, since $X \in \uparrow X$, we find $X \in j(\uparrow Y)$ for some $Y \in i(\uparrow X)$. This implies

$$\uparrow X \subseteq j(\uparrow Y); \text{ and so } i(\uparrow X) \subseteq i \circ j(\uparrow Y) \subseteq \uparrow Y.$$

Since $\uparrow Y \subseteq i(\uparrow X)$ in any case, we conclude $i(\uparrow X) = \uparrow Y$. This proves finite elements are mapped to finite elements.

What of Δ; that is, what is $i(\uparrow \Delta)$? We find, supposing E to be a neighbourhood system over a set Δ', that since $\uparrow \Delta \subseteq j(\uparrow \Delta')$, then $i(\uparrow \Delta) \subseteq \uparrow \Delta'$ and so $i(\uparrow \Delta) = \uparrow \Delta'$. This means that Δ corresponds to Δ'. So we have established that $\mathcal{D}$ is in an inclusion preserving one-one correspondence with a subset $\mathcal{D}'$ of E where $\Delta' \in \mathcal{D}'$. But it remains to show that $\mathcal{D}'$ is a neighbourhood system and that $\mathcal{D}' \triangleleft E$ holds. All we really have to show is that $\mathcal{D}'$ is closed under intersection whenever the intersection belongs to E.

Suppose $Y', Z' \in \mathcal{D}'$ and $Y' \cap Z' \in E$. Let $X' = Y' \cap Z'$. We have, for suitable $Y, Z \in \mathcal{D}$,

$$i(\uparrow Y) = \uparrow Y', \text{ and so } \uparrow Y = j(\uparrow Y'); \text{ and}$$

$$i(\uparrow Z) = \uparrow Z', \text{ and so } \uparrow Z = j(\uparrow Z').$$

But $\uparrow Y' \subseteq \uparrow X'$ and $j(\uparrow Y') \subseteq j(\uparrow X')$; thus $Y \in j(\uparrow X')$. For similar reasons $Z \in j(\uparrow X')$. But then $X = Y \cap Z \in j(\uparrow X')$, and therefore $Y \cap Z \in \mathcal{D}$. (The element $j(\uparrow X')$ must be a filter.) Notice, however, that

$$\uparrow Y \subseteq \uparrow X, \text{ and so } \uparrow Y' \subseteq i(\uparrow X); \text{ and}$$

$$\uparrow Z \subseteq \uparrow X, \text{ and so } \uparrow Z' \subseteq i(\uparrow X).$$

It follows that $Y' \cap Z' = X' \in i(\uparrow X)$. On the other hand we already knew $X \in j(\uparrow X')$, which implies $i(\uparrow X) \subseteq \uparrow X'$. We may thus conclude that $i(\uparrow X) = \uparrow X'$. In other words $X' \in \mathcal{D}'$. □

pointwise_sSup_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

theorem pointwise_sSup_preservesDirectedSup (F : Set (ScottMap D D')) :
    PreservesDirectedSup (fun x => sSup (Set.image (fun f : ScottMap D D' => (f : D →
    D') x) F)) := by
  intro S hS hSdir
  set H := fun x => sSup (Set.image (fun f : ScottMap D D' => (f : D → D') x) F)
  show H (sSup S) = sSup (Set.image H S)
  apply le_antisymm
  · apply sSup_le
    intro b hb
    obtain ⟨f, hfF, rfl⟩ := hb
    change f (sSup S) ≤ sSup (Set.image H S)
    rw [f.preservesDirectedSup_coe S hS hSdir]
    apply sSup_le
    intro c hc
    obtain ⟨s, hsS, rfl⟩ := hc
    exact le_trans (le_sSup (Set.mem_image_of_mem (fun g : ScottMap D D' => (g : D → D
    ') s) hfF))
      (le_sSup (Set.mem_image_of_mem H hsS))
  · apply sSup_le
    intro b hb
    obtain ⟨s, hsS, rfl⟩ := hb
    apply sSup_le
    intro c hc
    obtain ⟨f, hfF, rfl⟩ := hc
    exact le_trans (ScottMap.monotone f (le_sSup hsS))
      (le_sSup (Set.mem_image_of_mem (fun g : ScottMap D D' => (g : D → D') (sSup S))
    hfF))
```

```
end ScottMap

end DomainTheory.ContinuousLattice
```

4

3.7.17 Theorem 6.16 — initial algebra is minimal

THEOREM 6.16. If on the category of domains and strict approximable maps the functor T is continuous on maps, and if D is an initial T-algebra, then for any system $E \cong T(E)$ we have $D \trianglelefteq E$.

Proof: There is a homomorphism $h : D \to E$. By 6.9 there is a homomorphism $g : E \to D$. Now $g \circ h : D \to D$ is also a homomorphism, so $g \circ h = I_D$ because D is initial. In view of 6.15, all we have to prove now is that $h \circ g \sqsubseteq I_E$.

Let the maps $i : T(D) \to D$ and $j : D \to T(D)$ give the isomorphism for D, and let $u : T(E) \to E$ and $v : E \to T(E)$ do the same for E. By the proof of 6.9 we know

$$g = i \circ T(g) \circ v \quad \text{and} \quad h = u \circ T(h) \circ j$$

and each of these maps is the least fixed point of its respective equation. Let

$$g_0 = \bot_{E \to D} \quad \text{and} \quad h_0 = \bot_{D \to E}$$

and define by recursion

$$g_{n+1} = i \circ T(g_n) \circ v \quad \text{and} \quad h_{n+1} = u \circ T(h_n) \circ j.$$

By the fixed-point calculation

$$g = \bigsqcup_{n=0}^{\infty} g_n \quad \text{and} \quad h = \bigsqcup_{n=0}^{\infty} h_n.$$

Now we see that

$$h_0 \circ g_0 = \bot_{E \to E},$$

and for each n that

$$\begin{aligned} h_{n+1} \circ g_{n+1} &= u \circ T(h_n) \circ j \circ i \circ T(g_n) \circ v \\ &= u \circ T(h_n) \circ T(g_n) \circ v \\ &= u \circ T(h_n \circ g_n) \circ v. \end{aligned}$$

But this means that

$$h \circ g = \bigsqcup_{n=0}^{\infty} (h_n \circ g_n)$$

is the least fixed point for the equation

$$k = u \circ T(k) \circ v.$$

But I_E is one of the fixed points; whence $h \circ g \sqsubseteq I_E$ must follow. □

pointwise_sSup_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

theorem pointwise_sSup_preservesDirectedSup (F : Set (ScottMap D D')) :
    PreservesDirectedSup (fun x => sSup (Set.image (fun f : ScottMap D D' => (f : D →
    D') x) F)) := by
  intro S hS hSdir
  set H := fun x => sSup (Set.image (fun f : ScottMap D D' => (f : D → D') x) F)
  show H (sSup S) = sSup (Set.image H S)
  apply le_antisymm
  · apply sSup_le
    intro b hb
    obtain ⟨f, hfF, rfl⟩ := hb
    change f (sSup S) ≤ sSup (Set.image H S)
    rw [f.preservesDirectedSup_coe S hS hSdir]
    apply sSup_le
    intro c hc
    obtain ⟨s, hsS, rfl⟩ := hc
    exact le_trans (le_sSup (Set.mem_image_of_mem (fun g : ScottMap D D' => (g : D → D
    ') s) hfF))
      (le_sSup (Set.mem_image_of_mem H hsS))
  · apply sSup_le
    intro b hb
    obtain ⟨s, hsS, rfl⟩ := hb
    apply sSup_le
    intro c hc
    obtain ⟨f, hfF, rfl⟩ := hc
    exact le_trans (ScottMap.monotone f (le_sSup hsS))
      (le_sSup (Set.mem_image_of_mem (fun g : ScottMap D D' => (g : D → D') (sSup S))
    hfF))
```

```
end ScottMap

end DomainTheory.ContinuousLattice
```

5

3.7.18 Lecture VI exercises

EXERCISES

3.7.19 Exercise 6.17 — algebras for sequence domains

What are the algebras for which C is initial? If A of 6.2 is a generalization of B, what is the corresponding generalization of C? Prove that it exists and explain what are the algebras involved.

pointwise_sSup_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

theorem pointwise_sSup_preservesDirectedSup (F : Set (ScottMap D D')) :
    PreservesDirectedSup (fun x => sSup (Set.image (fun f : ScottMap D D' => (f : D →
    D') x) F)) := by
  intro S hS hSdir
  set H := fun x => sSup (Set.image (fun f : ScottMap D D' => (f : D → D') x) F)
  show H (sSup S) = sSup (Set.image H S)
  apply le_antisymm
  · apply sSup_le
    intro b hb
    obtain ⟨f, hfF, rfl⟩ := hb
    change f (sSup S) ≤ sSup (Set.image H S)
    rw [f.preservesDirectedSup_coe S hS hSdir]
```

```
    apply sSup_le
    intro c hc
    obtain ⟨s, hsS, rfl⟩ := hc
    exact le_trans (le_sSup (Set.mem_image_of_mem (fun g : ScottMap D D' => (g : D → D
    ') s) hfF))
      (le_sSup (Set.mem_image_of_mem H hsS))
  · apply sSup_le
    intro b hb
    obtain ⟨s, hsS, rfl⟩ := hb
    apply sSup_le
    intro c hc
    obtain ⟨f, hfF, rfl⟩ := hc
    exact le_trans (ScottMap.monotone f (le_sSup hsS))
      (le_sSup (Set.mem_image_of_mem (fun g : ScottMap D D' => (g : D → D') (sSup S))
    hfF))

end ScottMap

end DomainTheory.ContinuousLattice
```

6

3.7.20 Exercise 6.18 — infinite product as initial

With reference back to Exercise 3.16 discuss the construction of $\mathcal{D}^\infty$ as an initial algebra and as a solution to the domain equation

$$\mathcal{D}^\infty \cong \mathcal{D} \times \mathcal{D}^\infty.$$

(I do not know whether all solutions must be of the form $\mathcal{D}^\infty \times E$.)

pointwise_sSup_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
```

```
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

theorem pointwise_sSup_preservesDirectedSup (F : Set (ScottMap D D')) :
    PreservesDirectedSup (fun x => sSup (Set.image (fun f : ScottMap D D' => (f : D →
    D') x) F)) := by
  intro S hS hSdir
  set H := fun x => sSup (Set.image (fun f : ScottMap D D' => (f : D → D') x) F)
  show H (sSup S) = sSup (Set.image H S)
  apply le_antisymm
  · apply sSup_le
    intro b hb
    obtain ⟨f, hfF, rfl⟩ := hb
    change f (sSup S) ≤ sSup (Set.image H S)
    rw [f.preservesDirectedSup_coe S hS hSdir]
    apply sSup_le
    intro c hc
    obtain ⟨s, hsS, rfl⟩ := hc
    exact le_trans (le_sSup (Set.mem_image_of_mem (fun g : ScottMap D D' => (g : D → D
    ') s) hfF))
      (le_sSup (Set.mem_image_of_mem H hsS))
  · apply sSup_le
    intro b hb
    obtain ⟨s, hsS, rfl⟩ := hb
    apply sSup_le
    intro c hc
    obtain ⟨f, hfF, rfl⟩ := hc
    exact le_trans (ScottMap.monotone f (le_sSup hsS))
      (le_sSup (Set.mem_image_of_mem (fun g : ScottMap D D' => (g : D → D') (sSup S))
    hfF))

end ScottMap

end DomainTheory.ContinuousLattice
```


3.7.21 Exercise 6.19 — strict sum and product

For the sake of uniformity restrict attention to systems $\mathcal{D}$ on sets $\Delta \subseteq \{0,1\}^*$, where $\Lambda \in \Delta$ and $\emptyset \notin \mathcal{D}$, and to the category of strict maps. Define sum and product by:

$$\mathcal{D}_0 + \mathcal{D}_1 = \{\{\Lambda\} \cup 0\Delta_0 \cup 1\Delta_1\} \cup \{0X \mid X \in \mathcal{D}_0\} \cup \{1Y \mid Y \in \mathcal{D}_1\},$$

$$\mathcal{D}_0 \times \mathcal{D}_1 = \{\{\Lambda\} \cup 0X \cup 1Y \mid X \in \mathcal{D}_0 \text{ and } Y \in \mathcal{D}_1\}.$$

Are these correct up to isomorphism? Now generate all constructs $T(X)$ formed by the constants (that is, $T(X) = \mathcal{D}$ for a fixed $\mathcal{D}$), by the identity ($T(X) = X$), and by sums and products ($T_0(X) + T_1(X)$, etc.). Show that these are all functors, continuous on maps, and monotone and continuous on domains.

pointwise_sSup_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

theorem pointwise_sSup_preservesDirectedSup (F : Set (ScottMap D D')) :
    PreservesDirectedSup (fun x => sSup (Set.image (fun f : ScottMap D D' => (f : D →
    D') x) F)) := by
  intro S hS hSdir
  set H := fun x => sSup (Set.image (fun f : ScottMap D D' => (f : D → D') x) F)
  show H (sSup S) = sSup (Set.image H S)
  apply le_antisymm
  · apply sSup_le
    intro b hb
    obtain ⟨f, hfF, rfl⟩ := hb
    change f (sSup S) ≤ sSup (Set.image H S)
    rw [f.preservesDirectedSup_coe S hS hSdir]
    apply sSup_le
    intro c hc
    obtain ⟨s, hsS, rfl⟩ := hc
    exact le_trans (le_sSup (Set.mem_image_of_mem (fun g : ScottMap D D' => (g : D → D
    ') s) hfF))
      (le_sSup (Set.mem_image_of_mem H hsS))
  · apply sSup_le
    intro b hb
    obtain ⟨s, hsS, rfl⟩ := hb
    apply sSup_le
    intro c hc
    obtain ⟨f, hfF, rfl⟩ := hc
    exact le_trans (ScottMap.monotone f (le_sSup hsS))
      (le_sSup (Set.mem_image_of_mem (fun g : ScottMap D D' => (g : D → D') (sSup S))
    hfF))

end ScottMap
```

```
end DomainTheory.ContinuousLattice
```

8

3.7.22 Exercise 6.20 — tokens of a functor

For any system $\mathcal{D}$ let $\mathrm{tok}(\mathcal{D})$ be the underlying set of tokens, so that $\mathcal{D}$ is a system over $\mathrm{tok}(\mathcal{D})$. For the category of Exercise 6.19 show that the function

$$\lambda\Gamma.\,\mathrm{tok}(T(\{\Gamma\}))$$

is continuous on the domain $\{\Gamma \subseteq \{0,1\}^* \mid \Lambda \in \Gamma\}$, where T is any of the functors generated in 6.19. Conclude that there must exist a set

$$\Gamma = \mathrm{tok}(T(\{\Gamma\})),$$

so that $\{\Gamma\} \triangleleft T(\{\Gamma\})$, and so 6.14 applies.

pointwise_sSup_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

theorem pointwise_sSup_preservesDirectedSup (F : Set (ScottMap D D')) :
    PreservesDirectedSup (fun x => sSup (Set.image (fun f : ScottMap D D' => (f : D →
    D') x) F)) := by
  intro S hS hSdir
  set H := fun x => sSup (Set.image (fun f : ScottMap D D' => (f : D → D') x) F)
  show H (sSup S) = sSup (Set.image H S)
  apply le_antisymm
  · apply sSup_le
    intro b hb
```

```
    obtain ⟨f, hfF, rfl⟩ := hb
    change f (sSup S) ≤ sSup (Set.image H S)
    rw [f.preservesDirectedSup_coe S hS hSdir]
    apply sSup_le
    intro c hc
    obtain ⟨s, hsS, rfl⟩ := hc
    exact le_trans (le_sSup (Set.mem_image_of_mem (fun g : ScottMap D D' => (g : D → D
    ') s) hfF))
      (le_sSup (Set.mem_image_of_mem H hsS))
  · apply sSup_le
    intro b hb
    obtain ⟨s, hsS, rfl⟩ := hb
    apply sSup_le
    intro c hc
    obtain ⟨f, hfF, rfl⟩ := hc
    exact le_trans (ScottMap.monotone f (le_sSup hsS))
      (le_sSup (Set.mem_image_of_mem (fun g : ScottMap D D' => (g : D → D') (sSup S))
    hfF))

end ScottMap

end DomainTheory.ContinuousLattice
```

9

3.7.23 Exercise 6.21 — coalesced sum and smash

Do the same as 6.19 and 6.20 when the functors are also allowed to be generated by the operations:

$$D_0 \oplus D_1 = \{\{\Lambda\} \cup 0\Delta_0 \cup 1\Delta_1\} \cup \{0X \mid X \in D_0 \setminus \{\Delta_0\}\} \cup \{1Y \mid Y \in D_1 \setminus \{\Delta_1\}\},$$

$$D_0 \otimes D_1 = \{\{\Lambda\} \cup 0\Delta_0 \cup 1\Delta_1\} \cup \{\{\Lambda\} \cup 0X \cup 1Y \mid X \in D_0 \setminus \{\Delta_0\} \text{ and } Y \in D_1 \setminus \{\Delta_1\}\}.$$

Generalize all of $+$, $\times$, $\oplus$, $\otimes$ to combinations of several terms, not just the binary sums and products.

preservesDirectedSup_coe.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Order.BourbakiWitt
public import DomainTheory.ContinuousLattice.ScottMaps.proposition_2_5
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
```

```
variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

theorem preservesDirectedSup_coe (f : ScottMap D D') (S : Set D) (hS : S.Nonempty)
    (hSdir : DirectedOn (· ≤ ·) S) :
    (f : D → D') (sSup S) = sSup (Set.image (f : D → D') S) := by
  have h := (proposition_2_5 (Subtype.val f)).mp f.property
  exact h hS hSdir

end ScottMap

end DomainTheory.ContinuousLattice
```

0

3.7.24 Exercise 6.22 — equations for the integers

Comment on these domain equations:

$$N \cong \{\{0\}, \{0, \Lambda\}\} \oplus N,$$

$$M \cong \{\{\Lambda\}\} + M,$$

$$N^* \cong N \oplus (N \otimes N^*).$$

preservesDirectedSup_coe.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Order.BourbakiWitt
public import DomainTheory.ContinuousLattice.ScottMaps.proposition_2_5
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
```

```
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

theorem preservesDirectedSup_coe (f : ScottMap D D') (S : Set D) (hS : S.Nonempty)
    (hSdir : DirectedOn (· ≤ ·) S) :
    (f : D → D') (sSup S) = sSup (Set.image (f : D → D') S) := by
  have h := (proposition_2_5 (Subtype.val f)).mp f.property
  exact h hS hSdir

end ScottMap

end DomainTheory.ContinuousLattice
```

1

3.7.25 Exercise 6.23 — syntactic expression domain

Construe the initial solution to

$$Exp \cong N \oplus ((Exp \times Exp) + (Exp \times Exp))$$

as a "syntactical domain" of expressions generated from infinitely many "variables" by means of two binary "operation symbols". Given an algebra D with two operations

$$u : D \times D \to D \quad \text{and} \quad v : D \times D \to D,$$

show how any strict map $s : N \to D$ determines a unique map

$$val(s) : Exp \to D$$

that can be regarded as the "evaluation of an expression".

preservesDirectedSup_coe.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Order.BourbakiWitt
public import DomainTheory.ContinuousLattice.ScottMaps.proposition_2_5
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]
```

```
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

theorem preservesDirectedSup_coe (f : ScottMap D D') (S : Set D) (hS : S.Nonempty)
    (hSdir : DirectedOn (· ≤ ·) S) :
    (f : D → D') (sSup S) = sSup (Set.image (f : D → D') S) := by
  have h := (proposition_2_5 (Subtype.val f)).mp f.property
  exact h hS hSdir

end ScottMap

end DomainTheory.ContinuousLattice
```

2

3.7.26 Exercise 6.24 — recursive domain equations

Show that there must exist domains satisfying:

$$D \cong D + (D \times E), \quad \text{and}$$

$$E \cong D + E,$$

by using a double fixed-point method. First decide what the underlying set of tokens should be, and then define D and E by simultaneous fixed points. (Syntactical domains as in 6.23 may very well require several simultaneous equations.)

preservesDirectedSup_coe.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Order.BourbakiWitt
public import DomainTheory.ContinuousLattice.ScottMaps.proposition_2_5
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
```

```
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

theorem preservesDirectedSup_coe (f : ScottMap D D') (S : Set D) (hS : S.Nonempty)
    (hSdir : DirectedOn (· ≤ ·) S) :
    (f : D → D') (sSup S) = sSup (Set.image (f : D → D') S) := by
  have h := (proposition_2_5 (Subtype.val f)).mp f.property
  exact h hS hSdir

end ScottMap

end DomainTheory.ContinuousLattice
```

3

3.7.27 Exercise 6.25 — projection pair laws

For a projection pair $g : \mathcal{D} \to \mathcal{E}$ and $h : \mathcal{E} \to \mathcal{D}$ show that for $x \in |\mathcal{D}|$ and $y \in |\mathcal{E}|$ we have:

$$g(x) \sqsubseteq y \text{ iff } x \sqsubseteq h(y).$$

Thus, conclude that:

$$h(y) = \bigsqcup\{x \in |\mathcal{D}| \mid g(x) \sqsubseteq y\}, \quad \text{and}$$

$$g(x) = \bigsqcap\{y \in |\mathcal{E}| \mid x \sqsubseteq h(y)\},$$

for all $x \in |\mathcal{D}|$ and $y \in |\mathcal{E}|$. So each of the functions determines the other. In the first equation check that the set on the right is directed, and in the second equation that the set on the right is non empty. Prove also that g maps consistent sets to consistent sets and preserves $\bigsqcup$ (not just directed unions).

preservesDirectedSup_coe.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Order.BourbakiWitt
public import DomainTheory.ContinuousLattice.ScottMaps.proposition_2_5
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]
```

```
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

theorem preservesDirectedSup_coe (f : ScottMap D D') (S : Set D) (hS : S.Nonempty)
    (hSdir : DirectedOn (· ≤ ·) S) :
    (f : D → D') (sSup S) = sSup (Set.image (f : D → D') S) := by
  have h := (proposition_2_5 (Subtype.val f)).mp f.property
  exact h hS hSdir

end ScottMap

end DomainTheory.ContinuousLattice
```

4

3.7.28 Exercise 6.26 — lifting a domain

For systems $\mathcal{D}$ as in 6.19 define

$$\mathcal{D}_\perp = \{\{\Lambda\} \cup 0\Delta\} \cup \{0X \mid X \in \mathcal{D}\}.$$

Describe the construct in terms of elements. Is this a suitable functor? Prove that

$$\mathcal{D}_\perp \oplus \mathcal{E}_\perp \cong \mathcal{D} + \mathcal{E}.$$

What is

$$\mathcal{D}_\perp \otimes \mathcal{E}_\perp \cong ??$$

preservesDirectedSup_coe.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Order.BourbakiWitt
public import DomainTheory.ContinuousLattice.ScottMaps.proposition_2_5
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
```

```
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

theorem preservesDirectedSup_coe (f : ScottMap D D') (S : Set D) (hS : S.Nonempty)
    (hSdir : DirectedOn (· ≤ ·) S) :
    (f : D → D') (sSup S) = sSup (Set.image (f : D → D') S) := by
  have h := (proposition_2_5 (Subtype.val f)).mp f.property
  exact h hS hSdir

end ScottMap

end DomainTheory.ContinuousLattice
```

5

3.7.29 Exercise 6.27 — which embeddings hold

Which of the following relationships are true:

$$(\mathcal{D} \otimes \mathcal{E}) \trianglelefteq (\mathcal{D} \times \mathcal{E})\,; \quad \mathcal{D} \trianglelefteq \mathcal{D} \times \mathcal{E}\,;$$

$$(\mathcal{D} \oplus \mathcal{E}) \trianglelefteq (\mathcal{D} + \mathcal{E})\,; \quad \mathcal{D} \trianglelefteq \mathcal{D} \oplus \mathcal{E}\,;$$

$$(\mathcal{D} \to_{\perp} \mathcal{E}) \trianglelefteq (\mathcal{D} \to \mathcal{E})\,; \quad \mathcal{D} \trianglelefteq \mathcal{D} \otimes \mathcal{E}\,?$$

preservesDirectedSup_coe.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Order.BourbakiWitt
public import DomainTheory.ContinuousLattice.ScottMaps.proposition_2_5
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']
```

```
namespace ScottMap

theorem preservesDirectedSup_coe (f : ScottMap D D') (S : Set D) (hS : S.Nonempty)
    (hSdir : DirectedOn (· ≤ ·) S) :
    (f : D → D') (sSup S) = sSup (Set.image (f : D → D') S) := by
  have h := (proposition_2_5 (Subtype.val f)).mp f.property
  exact h hS hSdir

end ScottMap

end DomainTheory.ContinuousLattice
```

6

3.7.30 Exercise 6.28 — finite embeddings cancel

(Suggested by G. Plotkin). Show that if $\mathcal{D}$ and $\mathcal{E}$ are *finite* systems and

$$\mathcal{D} \trianglelefteq \mathcal{E} \trianglelefteq \mathcal{D}\,,$$

then $\mathcal{D} \cong \mathcal{E}$. Need the same be true of infinite systems?

preservesDirectedSup_coe.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Order.BourbakiWitt
public import DomainTheory.ContinuousLattice.ScottMaps.proposition_2_5
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

theorem preservesDirectedSup_coe (f : ScottMap D D') (S : Set D) (hS : S.Nonempty)
    (hSdir : DirectedOn (· ≤ ·) S) :
    (f : D → D') (sSup S) = sSup (Set.image (f : D → D') S) := by
  have h := (proposition_2_5 (Subtype.val f)).mp f.property
  exact h hS hSdir
```

```
end ScottMap

end DomainTheory.ContinuousLattice
```

7

3.7.31 Exercise 6.29 — infinitary sum and product

Generalize $+$ and $\times$ to infinitary operations on domains:

$$\sum_{n=0}^{\infty} D_n \quad \text{and} \quad \prod_{n=0}^{\infty} D_n.$$

Would a similar generalization be possible for $\oplus$ and $\otimes$?

preservesDirectedSup_coe.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Order.BourbakiWitt
public import DomainTheory.ContinuousLattice.ScottMaps.proposition_2_5
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

theorem preservesDirectedSup_coe (f : ScottMap D D') (S : Set D) (hS : S.Nonempty)
    (hSdir : DirectedOn (· ≤ ·) S) :
    (f : D → D') (sSup S) = sSup (Set.image (f : D → D') S) := by
  have h := (proposition_2_5 (Subtype.val f)).mp f.property
  exact h hS hSdir

end ScottMap

end DomainTheory.ContinuousLattice
```

8

3.8 Computability in effectively given domains

Dependency diagram. Each box is one item. The arrowhead is on the item that imports. An arrow already implied by a longer path is left out. The introduction has no Lean file, so it is not shown. Blue is a definition, green a proposition, amber an example, red a theorem, and indigo an exercise.

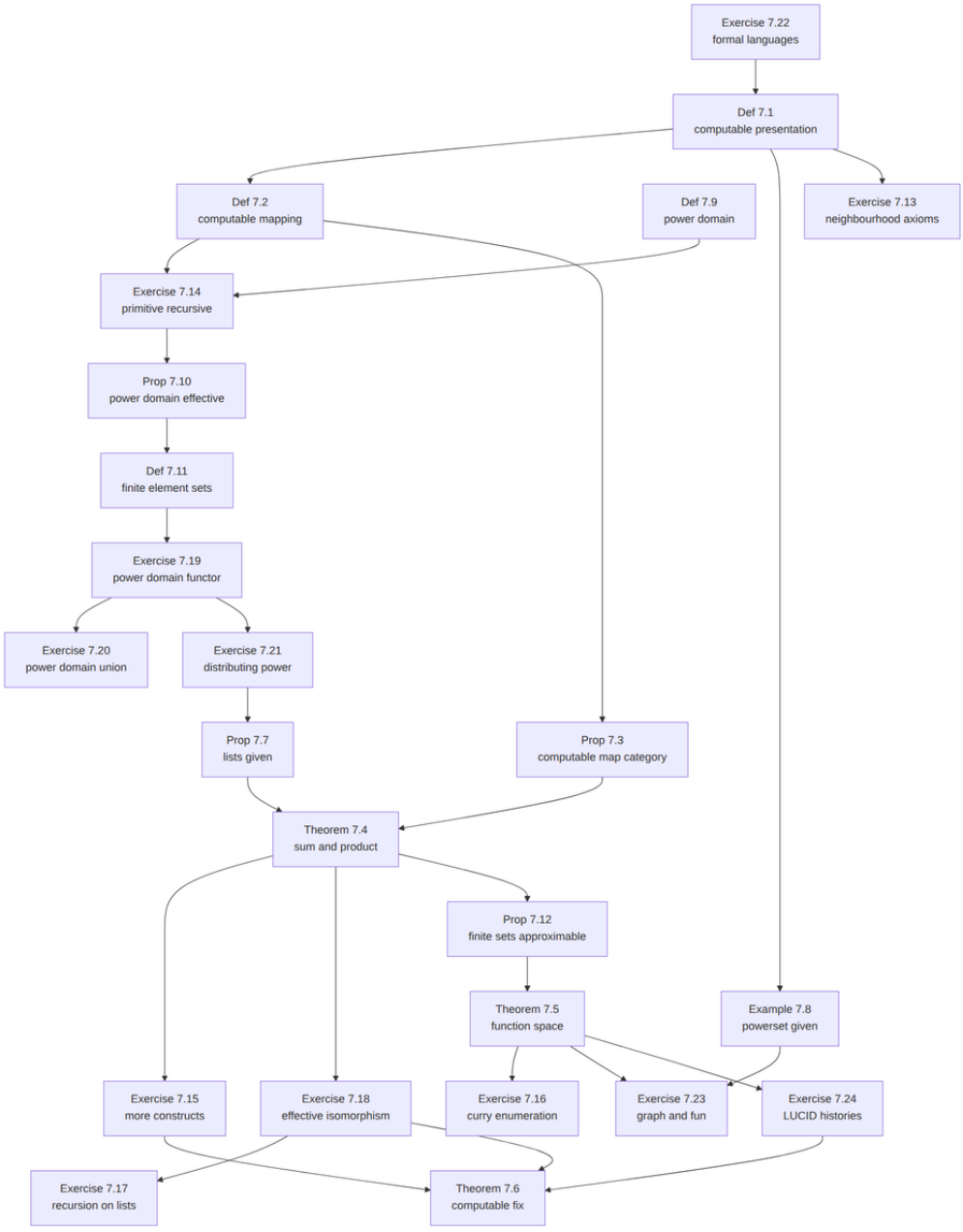


Figure 34: Computability in effectively given domains.

3.8.1 Introduction

LECTURE VII COMPUTABILITY IN EFFECTIVELY GIVEN DOMAINS

For the domain N the strict functions from N into N, the strict maps $f : N \to N$, correspond exactly to the partial functions $g : N \to N$ (as we wrote in 5.6 we had $f = \bar{g}$). For such functions there is a standard theory of computability: g is called computable if it can be defined as a partial recursive function with its "program" written down in a certain standard form. The non-strict maps $h : N \to N$ are all constant, and so are intuitively computable; so we know all about computable maps in $|N \to N|$ in general. The question is: what are the computable maps on (elements of) other domains?

The answer will of course depend on how the domain is presented to us. Even with N, there are continuum many isomorphisms $\pi : N \to N$ of N onto itself, not all of which can be computable. That is, if we permute N and, so to speak, present the integers in a different order, then a well-behaved computable function $f : N \to N$ may well be transformed into a non-computable function,

$$\pi \circ f \circ \pi^{-1} : N \to N.$$

(Hint: Consider the characteristic function e of the even numbers. Take $f = \bar{e}$ and let π be very horrid.) The reason we imagined we knew which were the computable $f : N \to N$ is that N is always thought of in a standard presentation. We must thus define "in general" a concept of an *effectively given domain*, that is to say, one with a sufficiently computable presentation to represent the additional knowledge about the domain.

The main idea will be that the *finite elements* of $|D|$ should be regarded as the ones initially known. Abstractly, to know a finite element is to know how it is *related* to other finite elements.

Of course, this will mean that we will allow at most a countable infinity of finite elements — but this restriction well accords with intuition. To make precise the terminology "related to" it proves most convenient to go back to the neighbourhoods (in any case they are in a one-one correspondence with the finite elements).

3.8.2 Definition 7.1 — computable presentation

DEFINITION 7.1. A neighbourhood system D has a *computable presentation* provided we can write

$$D = \{X_n \mid n \in N\},$$

where the following two relations

(i) $X_n \cap X_m = X_k$; and

(ii) $\exists k \in N.\, X_k \subseteq X_n$ and $X_k \subseteq X_m$

are recursively decidable (in integer indices n, m, k and in n, m, respectively). □

More strictly the sequence,

$$\langle X_n \rangle_{n=0}^{\infty},$$

is the presentation. Even more strictly, when it is required to cope with infinitely many domains at a time, it would be necessary to give the actual Gödel numbers of the recursive relations (i) and (ii) (rather than just saying there exists some way of showing them to be recursively decidable).

The intuitive idea of 7.1 is that the system is effectively given if you know how to do elementary "calculations" with neighbourhoods. The basic calculations are the forming of intersections. The neighbourhoods have to be laid out in a systematic way; and, if we are asked for an intersection of two given neighbourhoods, we have to be able to locate it in the standard sequence. Relation (ii) is the *consistency condition* , which is the necessary and sufficient condition for the intersection to exist in D. When (ii) is true, therefore, we have only to try $k = 0, 1, 2, \ldots$ until we discover that we have found the intersection. We are

assuming that these basic decisions can be carried out in "finite time". Note that the obvious biconditional,

$$X_n \subseteq X_m \text{ iff } X_n \cap X_m = X_n,$$

assures us that the inclusion relation between neighbourhoods is itself decidable in terms of the indices. So in (ii) if k exists, *then* it (or the first one) can indeed be found in finite time. The rub is that if it *does not exist*, no finite number of inclusion checks will determine that fact. That is why we have to *assume* that (ii) is always decidable. The information contained in (ii) is a fundamental part of the neighbourhood structure. (An axiomatic characterization of neighbourhood structures is given in Exercise 7.13, which may make clearer what we are assuming and what a presentation is.)

preservesDirectedSup_coe.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Order.BourbakiWitt
public import DomainTheory.ContinuousLattice.ScottMaps.proposition_2_5
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

theorem preservesDirectedSup_coe (f : ScottMap D D') (S : Set D) (hS : S.Nonempty)
    (hSdir : DirectedOn (· ≤ ·) S) :
    (f : D → D') (sSup S) = sSup (Set.image (f : D → D') S) := by
  have h := (proposition_2_5 (Subtype.val f)).mp f.property
  exact h hS hSdir

end ScottMap
```

```
end DomainTheory.ContinuousLattice
```

9

3.8.3 Definition 7.2 — computable mapping

DEFINITION 7.2. Given two recursively presented domains,

$$\mathcal{D} = \{X_n \mid n \in \mathbf{N}\} \text{ and } E = \{Y_m \mid m \in \mathbf{N}\},$$

an approximable mapping $f : \mathcal{D} \to E$ is said to be *computable* iff the relation

$$X_n f Y_m$$

is recursively enumerable in n and m. □

The question to ask first is why "recursively enumerable" rather than "recursive" (= "recursively decidable")? The answer will become clear when we let $\mathcal{D}$ degenerate to the one-element domain, $\mathcal{D} = \{\Delta\}$. Then what we are considering is merely a single element

$$y = f(\{\Delta\}) \in |E|.$$

Therefore, 7.2 incorporates the notion of a *computable element* of a domain. And the condition reduces to the statement that the filter $y \in |E|$ is such that the set

$$\{m \in \mathbf{N} \mid Y_m \in y\}$$

is a recursively enumerable set of integers. The point is that the elements of $|E|$ are finite or infinite. If y were finite, the set of indices above would indeed be recursive in view of

our assumptions on E. But an infinite element can in general only be approximated "a little at a time". We cannot expect to know the whole story of its approximations in a flash. What it means to be recursively enumerable is that there is a primitive recursive function (hence, a total function), $r : \mathbf{N} \to \mathbf{N}$, such that

$$y = \{Y_{r(i)} \mid i \in \mathbf{N}\}.$$

That is to say, *all* the approximations to y can eventually be listed. In the case of the mapping f we could write

$$f = \{(X_{s(i)}, Y_{r(i)}) \mid i \in \mathbf{N}\},$$

for a suitable pair of primitive recursive functions s and r.

Definitions 7.1 and 7.2 may very well irritate the person hearing them for the first time: instead of explaining computability in direct terms, the whole question is thrown into the lap of recursion theory! There are several answers. "You have to start somewhere" is one thing I always say. Recursion on the integers is a well-understood theory, and we shall not need the refined parts of the development, fortunately. In any case, our definitions apply to *many* domains of quite different structure, not just to the domain $\mathbf{N}$. And the next step we shall take is to show how to build up computable functions (and also effectively given domains) from simpler ones. Thus, often it will not be necessary to go back to the seemingly over-precise definitions involving the indices but to appeal to some broad general principles.

`projection.lean`

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.con_
public import DomainTheory.ContinuousLattice.FunctionSpaces.min
public import DomainTheory.ContinuousLattice.FunctionSpaces.const_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition313

/-- **Scott 1972, Proposition 3.13.** `(con, min)` makes `D` a projection of `[D → D]`:
`min ∘ con = id` (a constant's least value is its value) and `con ∘ min ⊑ id` (the
    constant `f(⊥)`
is `≤ f` pointwise, since `f(⊥) ⊑ f(y)` by monotonicity). -/
noncomputable def projection : IsContinuousLatticeProjection D (ScottMap D D) where
  incl := con
  retr := min
  retr_incl := fun _ => rfl
  incl_retr_le := fun f => by
    rw [ScottMap.le_def]
    intro y
    show ((ScottMap.const ((f : D → D) ⊥) : ScottMap D D) : D → D) y ≤ (f : D → D) y
    rw [ScottMap.const_apply]
    exact f.monotone bot_le

end Proposition313

end DomainTheory.ContinuousLattice
```

0

3.8.4 Proposition 7.3 — computable maps form a category

PROPOSITION 7.3. The identity map on an effectively given domain is computable; the composition of computable mappings on effectively given domains is again computable. □

The proofs for 7.3 are so trivial they are hardly worth an exercise. Note the immediate and useful

consequence: if $f : D \to E$ is computable and $x \in |D|$ is computable, then $f(x) \in |E|$ is also computable. The next result is, however, worth working out even though it is quite easy.

projection.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.con_
public import DomainTheory.ContinuousLattice.FunctionSpaces.min
public import DomainTheory.ContinuousLattice.FunctionSpaces.const_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition313

/-- **Scott 1972, Proposition 3.13.** `(con, min)` makes `D` a projection of `[D → D]`:
`min ∘ con = id` (a constant's least value is its value) and `con ∘ min ⊑ id` (the
    constant `f(⊥)`
is `≤ f` pointwise, since `f(⊥) ⊑ f(y)` by monotonicity). -/
noncomputable def projection : IsContinuousLatticeProjection D (ScottMap D D) where
  incl := con
  retr := min
  retr_incl := fun _ => rfl
  incl_retr_le := fun f => by
    rw [ScottMap.le_def]
    intro y
    show ((ScottMap.const ((f : D → D) ⊥) : ScottMap D D) : D → D) y ≤ (f : D → D) y
    rw [ScottMap.const_apply]
    exact f.monotone bot_le

end Proposition313

end DomainTheory.ContinuousLattice
```

1

3.8.5 Theorem 7.4 — sum and product effective

THEOREM 7.4. If $\mathcal{D}_0$ and $\mathcal{D}_1$ are effectively given, then so are $(\mathcal{D}_0 + \mathcal{D}_1)$ and $(\mathcal{D}_0 \times \mathcal{D}_1)$.

Moreover the combinators in_i and out_i and proj_i^2 are all computable; further, if f and g are computable maps, then so are $f + g$ and $f \times g$.

Proof: Let the computable presentations be given as:

$$\mathcal{D}_i = \{X_n^i \mid n \in \mathbf{N}\}.$$

We can assume that the sets of tokens Δ_0 and Δ_1 are disjoint and $\emptyset \notin \mathcal{D}_i$. Then the construction of the sum is just

$$\mathcal{D}_0 + \mathcal{D}_1 = \{\Delta_0 \cup \Delta_1\} \cup \mathcal{D}_0 \cup \mathcal{D}_1.$$

As an enumeration we define for $n \in \mathbf{N}$:

$$Z_0 = \Delta_0 \cup \Delta_1 \; ; \; Z_{2n+1} = X_n^0 \; ; \; Z_{2n+2} = X_n^1 \; .$$

We leave as an exercise the check of 7.1(i)–(ii).

For the product we want:

$$\mathcal{D}_0 \times \mathcal{D}_1 = \{X_n^0 \cup X_m^1 \mid n, m \in \mathbf{N}\}.$$

What we then need are recursive functions $p : \mathbf{N} \to \mathbf{N}$, $q : \mathbf{N} \to \mathbf{N}$, and $r : \mathbf{N} \times \mathbf{N} \to \mathbf{N}$ where for $m, n, k \in \mathbf{N}$ we have:

$$p(r(n, m)) = n \text{ and } q(r(n, m)) = m, \text{ and } r(p(k), q(k)) = k.$$

Thus r is a "one-one pairing function"; there are many ways to find such functions (see Exercise 5.13). We can then define for $k \in \mathbf{N}$:

$$W_k = X^0_{p(k)} \cup X^1_{q(k)} \; .$$

Again we leave as an exercise the check that this provides a computable presentation of $\mathcal{D}_0 \times \mathcal{D}_1$.

As for the combinators, the neighbourhood relations have to be worked out in terms of the indices. For example

$$X_n^0 \; \mathrm{in}_0 \; Z_m \text{ iff either } m = 0 \text{ or for some } k$$

$$m = 2k + 1 \text{ and } X_n^0 \subseteq X_k^0 \; .$$

and

$$W_k \; \mathrm{proj}_1^2 \; X_m^1 \text{ iff } X^1_{q(k)} \subseteq X_m^1 \; .$$

The reader needs to check that these are recursively enumerable

Relations in the indices. For this purpose it may be convenient to recall some closure properties of these relations: taking conjunctions, disjunctions, substituting recursive functions, applying an existential quantifier to the front. □

Products give us a way of providing an immediate meaning to the notion of a computable function of several variables. Note that the proof of 3.7 is "effective" and shows that substitution of computable functions of several variables into each other always gives computable functions. We turn next to the function spaces.

projection.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.con_
public import DomainTheory.ContinuousLattice.FunctionSpaces.min
public import DomainTheory.ContinuousLattice.FunctionSpaces.const_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition313

/-- **Scott 1972, Proposition 3.13.** `(con, min)` makes `D` a projection of `[D → D]`:
`min ∘ con = id` (a constant's least value is its value) and `con ∘ min ⊑ id` (the
    constant `f(⊥)`
is `≤ f` pointwise, since `f(⊥) ⊑ f(y)` by monotonicity). -/
noncomputable def projection : IsContinuousLatticeProjection D (ScottMap D D) where
  incl := con
  retr := min
  retr_incl := fun _ => rfl
  incl_retr_le := fun f => by
    rw [ScottMap.le_def]
    intro y
    show ((ScottMap.const ((f : D → D) ⊥) : ScottMap D D) : D → D) y ≤ (f : D → D) y
    rw [ScottMap.const_apply]
    exact f.monotone bot_le

end Proposition313

end DomainTheory.ContinuousLattice
```

3.8.6 Theorem 7.5 — function space effective

THEOREM 7.5. If $\mathcal{D}_0$ and $\mathcal{D}_1$ are effectively given, then so is $(\mathcal{D}_0 \to \mathcal{D}_1)$. The combinators eval and curry are computable, provided all the domains involved are effectively given. The computable elements $f \in |\mathcal{D}_0 \to \mathcal{D}_1|$ are exactly the computable maps $f : \mathcal{D}_0 \to \mathcal{D}_1$.

Proof: The proofs of 3.9, 3.11, and 3.12 were set up with this theorem in mind. If

$$\mathcal{D}_0 = \{X_n \mid n \in \mathbf{N}\} \quad \text{and} \quad \mathcal{D}_1 = \{Y_m \mid m \in \mathbf{N}\}$$

are two effectively given neighbourhood systems, then the neighbourhoods of $(\mathcal{D}_0 \to \mathcal{D}_1)$, by Definition 3.8, are non-empty intersections like

$$\bigcap_{i<q} [X_{n_i}, Y_{m_i}],$$

where $\langle n_0, n_1, \ldots, n_{q-1}\rangle$ and $\langle m_0, m_1, \ldots, m_{q-1}\rangle$ are two finite sequences of integers determining the choice of the function-space neighbourhood. In 3.9(i) the test for nonemptiness is given. Assuming the decidability of relations in $\mathcal{D}_0$ and $\mathcal{D}_1$, one remarks that the consistency of *finite sequences* of neighbourhoods is also decidable. (Hint: Test the first *two*, then form their intersection. Next test the third given neighbourhood against this one set; if consistent, form the intersection, and carry on.) By 3.9(i) at most $2 \cdot 2^q$ such sequential checks must be carried out to determine whether the function-space neighbourhood is non empty.

It may not be fun, but the checks can be carried out in finite time. Owing to this decidability, we can therefore enumerate in a systematic way *all* the pairs of finite sequences $\langle n_0, \ldots\rangle$ and $\langle m_0, \ldots\rangle$ that determine neighbourhoods: that is the way that $(\mathcal{D}_0 \to \mathcal{D}_1)$ obtains its enumeration.

Concerning the decidability of the required relations on $(\mathcal{D}_0 \to \mathcal{D}_1)$, we remark first off that consistency is more of the same: to test two finite intersections against each other, just form one big intersection and test it for non-emptiness as before. Secondly, the testing for intersection comes down in the end to testing one typical intersection of $[X, Y]$-neighbourhoods for equality with another. But equality amounts to two inclusions; inclusion in an intersection amounts to inclusion in each term. Therefore, what we need to do is to check a finite number of statements of the form:

$$\bigcap_{i<q} [X_{n_i}, Y_{m_i}] \subseteq [X_k, Y_\ell].$$

As we pointed out after the proof of 3.9, this inclusion is equivalent to

$$\bigcap \{Y_{m_i} \mid X_k \subseteq X_{n_i}\} \subseteq Y_\ell.$$

By decidability in $\mathcal{D}_0$, we can effectively find the n_i that are needed. Then in $\mathcal{D}_1$, we form the intersection of the corresponding Y_{m_i}. Finally, we check the inclusion. Again, one check in $(\mathcal{D}_0 \to \mathcal{D}_1)$ requires a whole sequence of checks in $\mathcal{D}_0$ and in $\mathcal{D}_1$, but the process is finite. So we have argued that $(\mathcal{D}_0 \to \mathcal{D}_1)$ is effectively given.

In showing that the combinators are computable, we refer first to the proof of 3.11. The typical pair of neighbourhoods possibly belonging to eval is

$$\bigcap_{i<q} [X_{n_i}, Y_{m_i}],\ X_k \text{ eval } Y_\ell.$$

As we needed not to be so specific, we expressed the holding of this relationship in terms of *all* the functions in the function-

space neighbourhood. But we know that the neighbourhood, by 3.9(ii), has a minimal element; it is then sufficient to test for the holding of $X_k f_0 Y_\ell$ at this minimal function f_0. But this test, we have already seen, is decidable. So the pairs in eval actually form a recursive set, not just a recursively enumerable set; thus, eval is a computable function.

The case of curry involves three domains and is a bit more messy. But again, if the required neighbourhoods are written out in full, it will be seen that curry, too, is computable. We leave this minor struggle to the exercises.

The final statement is an easy consequence of the fundamental connection between approximable $f : \mathcal{D}_0 \to \mathcal{D}_1$ as relations and as elements. Recall, as in the proof of 3.10, that we have

$$f \in [X, Y] \text{ iff } X f Y,$$

for all $X \in \mathcal{D}_0$ and $Y \in \mathcal{D}_1$. Therefore,

$$f \in \bigcap_{i<q} [X_{n_i}, Y_{m_i}] \text{ iff } \forall i < q.\, X_{n_i} f Y_{m_i}.$$

It follows that if f is recursively enumerable as a set of pairs, then, by forming all the non-empty intersections (as shown), we get an enumeration of all the neighbourhoods to which f belongs; and this is the same as the filter corresponding to f as an element of the function space. The converse direction is clear. □

We have nearly all our favourite combinators computable, but perhaps the most important one - since it is the key to recursive definitions - is the fixed-point combinator. It is not left out.

projection.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.con_
public import DomainTheory.ContinuousLattice.FunctionSpaces.min
public import DomainTheory.ContinuousLattice.FunctionSpaces.const_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
```

```
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition313

/-- **Scott 1972, Proposition 3.13.** `(con, min)` makes `D` a projection of `[D → D]`:
`min ∘ con = id` (a constant's least value is its value) and `con ∘ min ⊑ id` (the
    constant `f(⊥)`
is `≤ f` pointwise, since `f(⊥) ⊑ f(y)` by monotonicity). -/
noncomputable def projection : IsContinuousLatticeProjection D (ScottMap D D) where
  incl := con
  retr := min
  retr_incl := fun _ => rfl
  incl_retr_le := fun f => by
    rw [ScottMap.le_def]
    intro y
    show ((ScottMap.const ((f : D → D) ⊥) : ScottMap D D) : D → D) y ≤ (f : D → D) y
    rw [ScottMap.const_apply]
    exact f.monotone bot_le

end Proposition313

end DomainTheory.ContinuousLattice
```

3

3.8.7 Theorem 7.6 — fix is computable

THEOREM 7.6. For any effectively given domain $\mathcal{D}$, the combinator fix : $(\mathcal{D} \to \mathcal{D}) \to \mathcal{D}$ is computable.

Proof : Referring back to the proof of Theorem 4.2 and thinking of

$$\mathcal{D} = \{X_n \mid n \in \mathbb{N}\}$$

as effectively given, fix as a relation comes down to

$$\bigcap_{i<q} [X_{n_i}, X_{m_i}] \text{ fix } X_\ell \text{ iff for some finite sequence}$$

$$\Delta = X_{k_0}, \ldots, X_{k_p} = X_\ell,$$

we have, for each $j < p$,

$$\bigcap \{X_{m_i} \mid X_{k_j} \subseteq X_{n_i}\} \subseteq X_{k_{j+1}}.$$

Inside the "for some finite sequence" all the checks are decidable by assumption on D. But the existential quantification of a decidable predicate always gives a recursively enumerable predicate. (And, as there is no implied bound on the size of the finite sequence we are looking for, this really *is* an enumerable set and not generally a recursive set.) □

The major consequence of what we have done up to this point concerns typed λ-calculus. Any expression involving only *effectively given types* and, perhaps, some *basic computable constants* using only the $\lambda, !$-notation defines a computable function of its free variables. And such functions applied to computable arguments give computable values. And such functions have computable least fixed points. Etc., etc. In a

definite sense then we have in the "metalanguage", as people say, a quite precise and fully *mathematical programming language* for defining computable operators. It is not a machine implemented language, but it is a mathematically well-defined and easy-to-use language. And when we combine the usual type-definition facility together with *domain equations*, we have an especially powerful language.

projection.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.con_
public import DomainTheory.ContinuousLattice.FunctionSpaces.min
public import DomainTheory.ContinuousLattice.FunctionSpaces.const_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition313

/-- **Scott 1972, Proposition 3.13.** `(con, min)` makes `D` a projection of `[D → D]`:
`min ∘ con = id` (a constant's least value is its value) and `con ∘ min ⊑ id` (the
    constant `f(⊥)`
is `≤ f` pointwise, since `f(⊥) ⊑ f(y)` by monotonicity). -/
noncomputable def projection : IsContinuousLatticeProjection D (ScottMap D D) where
  incl := con
  retr := min
  retr_incl := fun _ => rfl
  incl_retr_le := fun f => by
    rw [ScottMap.le_def]
    intro y
    show ((ScottMap.const ((f : D → D) ⊥) : ScottMap D D) : D → D) y ≤ (f : D → D) y
    rw [ScottMap.const_apply]
    exact f.monotone bot_le

end Proposition313

end DomainTheory.ContinuousLattice
```

3.8.8 Proposition 7.7 — lists effectively given

PROPOSITION 7.7. For any effectively given domain D, the domain $D^{\S}$ is also effectively given, and all the combinators of Example 6.1 prove to be computable.

Proof: This proof is essentially an exercise, but it is useful to have an easy-to-grasp example. Indeed, to make things easy to reason about, we can assume that D is a system over $\Delta = \mathbf{N}$, and that in the presentation where

$$D = \{X_n \mid n \in \mathbf{N}\},$$

the relation $k \in X_n$ is *recursive* in k and n. (It is worth thinking why this is so.) Of course, a lot of other things are recursive also.

Now what kind of a system is $D^{\S}$? The construction of 6.1 made it a system over a certain set of strings Γ. For the sake of checking various assertions about computability, we are transposing everything back to $\mathbf{N}$. (These are all denumerable sets in any case.) The set Γ is divided into three equally big parts, and we can do the same for $\mathbf{N}$. Let us write for any $m, k \in \mathbf{N}$ and subset $X \subseteq \mathbf{N}$:

$$mX + k = \{m \cdot n + k \mid n \in X\}.$$

Then by splitting the integers modulo 3 we have:

$$\mathbf{N} = 3\mathbf{N} \cup (3\mathbf{N} + 1) \cup (3\mathbf{N} + 2),$$

and this equation is quite analogous to that for Γ. We then propose this definition for $D^{\S}$:

$$D^{\S} = \{\mathbf{N}\} \cup \{3X \mid X \in D\} \cup \{(3X + 1) \cup (3Y + 2) \mid X, Y \in D^{\S}\},$$

but this does not make the enumeration of $D^{\S}$ all that obvious. This is one way to do it:

$$V_0 = \mathbf{N}\ ;\ V_{2n+1} = 3X_n\ ;\ V_{2n+2} = (3V_{p(n)} + 1) \cup (3V_{q(n)} + 2).$$

Here p and q are the inverse of the pairing functions mentioned in 7.4. They must be chosen so that $p(n) < n$ and $q(n) < n$ for all $n \in \mathbf{N}$. Thus, in calculating V_k where $k = 2n + 2$ we will be using $V_{p(n)}$ and $V_{q(n)}$ where both subscripts are strictly less than k. This observation is required so that $m \in V_k$ is going to be a recursive relation. What we claim is that

$$D^{\S} = \{V_k \mid k \in \mathbf{N}\}.$$

It should be clear that everything on the right belongs to $D^{\S}$. What needs an inductive argument is that everything in $D^{\S}$ is eventually of the form V_k. But this should be fairly obvious owing to the properties of $r : \mathbf{N} \times \mathbf{N} \leftrightarrow \mathbf{N}$.

The reader also has to check that 7.1(i)–(ii) hold for the V_k. The idea is that any such check is either (1) trivial, or (2) something already assumed about D and the X_n, or (3) can be thrown back to some sets V_m with strictly smaller subscripts. Therefore, the checks will give an answer in finite time according to an effective reduction.

Next for the combinators, we have to translate neighbourhood relations into relations among integer indices. A selection of examples must suffice,

$$X_n\ (\lambda x.\, x^{\S})\ V_k \text{ iff } V_{2n+1} \subseteq V_k$$

$$V_m\ \mathrm{proj}_0\ V_k \text{ iff } k = 0 \text{ or } \exists n \in \mathbf{N}.\, m = 2n+2 \text{ and } V_{p(n)} \subseteq V_k.$$

The reader should write out other cases. □

projection.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.con_
public import DomainTheory.ContinuousLattice.FunctionSpaces.min
public import DomainTheory.ContinuousLattice.FunctionSpaces.const_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition313

/-- **Scott 1972, Proposition 3.13.** `(con, min)` makes `D` a projection of `[D → D]`:
`min ∘ con = id` (a constant's least value is its value) and `con ∘ min ⊑ id` (the
    constant `f(⊥)`
is `≤ f` pointwise, since `f(⊥) ⊑ f(y)` by monotonicity). -/
noncomputable def projection : IsContinuousLatticeProjection D (ScottMap D D) where
  incl := con
  retr := min
  retr_incl := fun _ => rfl
  incl_retr_le := fun f => by
    rw [ScottMap.le_def]
    intro y
    show ((ScottMap.const ((f : D → D) ⊥) : ScottMap D D) : D → D) y ≤ (f : D → D) y
    rw [ScottMap.const_apply]
    exact f.monotone bot_le

end Proposition313

end DomainTheory.ContinuousLattice
```

5

3.8.9 Example 7.8 — powerset effectively given

EXAMPLE 7.8. We have often made reference to the powerset $P\mathbf{N}$ as a domain and we should check here that it is effectively given. One easy way to see this is to note

$$P\mathbf{N} \cong |T^{\infty}|.$$

The (slight) trouble with $P\mathbf{N}$ is that we usually think of it in terms of *elements* rather than *neighbourhoods*. Going back to Exercise 1.16, we can argue that the neighbourhoods of $P\mathbf{N}$ are ordered not like the finite sets of integers but in the partial ordering *converse* to that. But this is of no trouble, since all will be decidable. What we need first is an enumeration of all finite sets of integers. We can do this by:

$$E_n = \{k \mid \exists i, j.\, i < 2^k \text{ and } n = i + 2^k + j \cdot 2^{k+1}\}.$$

The idea is that $k \in E_n$ means that the exponent k does occur in the binary expansion of n as a sum of powers of 2. All finite subsets of $\mathbf{N}$ are of the form E_n. We then find that as a neighbourhood system

$$(P\mathbf{N}) = \{\mathbf{N} \setminus E_n \mid n \in \mathbf{N}\}.$$

As the relationship $E_n \cup E_m = E_k$ is recursive, there is no trouble in proving that this is a computable presentation. In this system, of course, any two neighbourhoods are consistent. Various combinators on $P\mathbf{N}$ are suggested in Exercise 7.23. □

We end this chapter with an example of another kind of domain construct. This construct is known as the *Smyth Power Domain*. It is defined for any neighbourhood system $\mathcal{D}$ and results in a new system we shall call here $\mathbb{P}\mathcal{D}$. The elements of $\mathbb{P}\mathcal{D}$ behave rather like *sets of elements* of $\mathcal{D}$, but since our elements can be either partial or total, there are certain dangers to pushing the analogy too far. For some purposes a rival construct called the *Plotkin Power Domain* is better, but it leads outside the category of neighbourhood systems as defined in these lectures. Do not confuse $P\mathbf{N}$ with $\mathbb{P}\mathcal{D}$.

projection.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.con_
public import DomainTheory.ContinuousLattice.FunctionSpaces.min
public import DomainTheory.ContinuousLattice.FunctionSpaces.const_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
```

```
variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition313

/-- **Scott 1972, Proposition 3.13.** `(con, min)` makes `D` a projection of `[D → D]`:
`min ∘ con = id` (a constant's least value is its value) and `con ∘ min ⊑ id` (the
    constant `f(⊥)`
is `≤ f` pointwise, since `f(⊥) ⊑ f(y)` by monotonicity). -/
noncomputable def projection : IsContinuousLatticeProjection D (ScottMap D D) where
  incl := con
  retr := min
  retr_incl := fun _ => rfl
  incl_retr_le := fun f => by
    rw [ScottMap.le_def]
    intro y
    show ((ScottMap.const ((f : D → D) ⊥) : ScottMap D D) : D → D) y ≤ (f : D → D) y
    rw [ScottMap.const_apply]
    exact f.monotone bot_le

end Proposition313

end DomainTheory.ContinuousLattice
```

6

3.8.10 Definition 7.9 — power domain

DEFINITION 7.9. Let $\mathcal{D}$ be any neighbourhood system and define

$$\mathbb{P}\mathcal{D} = \left\{ \bigcup_{i<n} (\downarrow X_i) \mid \forall i < n.\, X_i \in \mathcal{D} \right\}.$$

We recall that for any $X \in \mathcal{D}$

$$\downarrow X = \{Y \in \mathcal{D} \mid Y \subseteq X\}.$$

The finite unions in $\mathbb{P}\mathcal{D}$ can be empty (i.e. if $n = 0$). □

Formally, the system $\mathbb{P}\mathcal{D}$ is just more or less the closure of $\mathcal{D}$ under finite unions; however, this would not be an isomorphism-invariant construct unless $\mathcal{D}$ is "prepared". The preparation consists of replacing $\mathcal{D}$ by the isomorphic domain

$$\mathcal{D}^\dagger = \{\downarrow X \mid X \in \mathcal{D}\}.$$

(In this connection refer back to Exercise 1.20.) We remark that

$$\downarrow X \cap \downarrow Y \neq \emptyset \text{ iff } \{X, Y\} \text{ is consistent in } \mathcal{D},$$

and in that case

$$\downarrow X \cap \downarrow Y = \downarrow (X \cap Y).$$

projection.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.con_
public import DomainTheory.ContinuousLattice.FunctionSpaces.min
public import DomainTheory.ContinuousLattice.FunctionSpaces.const_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition313

/-- **Scott 1972, Proposition 3.13.** `(con, min)` makes `D` a projection of `[D → D]`:
`min ∘ con = id` (a constant's least value is its value) and `con ∘ min ⊑ id` (the
    constant `f(⊥)`
is `≤ f` pointwise, since `f(⊥) ⊑ f(y)` by monotonicity). -/
noncomputable def projection : IsContinuousLatticeProjection D (ScottMap D D) where
  incl := con
  retr := min
  retr_incl := fun _ => rfl
  incl_retr_le := fun f => by
    rw [ScottMap.le_def]
    intro y
    show ((ScottMap.const ((f : D → D) ⊥) : ScottMap D D) : D → D) y ≤ (f : D → D) y
    rw [ScottMap.const_apply]
    exact f.monotone bot_le

end Proposition313

end DomainTheory.ContinuousLattice
```

3.8.11 Proposition 7.10 — power domain effective

PROPOSITION 7.10. The power domain $\mathbb{P}\mathcal{D}$ is a neighbourhood system if $\mathcal{D}$ is, and it is effectively given if $\mathcal{D}$ is.

Proof : The system $\mathcal{D}^\dagger$ is a neighbourhood system as we just remarked; indeed it is a positive neighbourhood system. It is easy to prove that the closure of any positive system under finite unions is a neighbourhood system, because the resulting family of sets is closed under *all* finite intersections. (If we left out the empty union, the result would be a positive system.) The proof is obvious since intersection of sets distributes over finite union. So $\mathbb{P}\mathcal{D}$ is a neighbourhood system.

For the second half of the proposition, we just have to constructivize the previous argument. Thus, if

$$\mathcal{D} = \{X_n \mid n \in \mathbb{N}\},$$

then the elements of $\mathbb{P}\mathcal{D}$ can be written as:

$$\bigcup_{i<q} (\downarrow X_{n_i}),$$

and hence are indexed by the finite sequences $\langle n_0, \ldots, n_{q-1} \rangle$ of integers. Now one of the standard devices of recursion theory is to put the finite sequences of integers into a recursive one-one correspondence with the integers themselves. This is the start of the recursive presentation of $\mathbb{P}\mathcal{D}$, since it means we can list effectively all the required neighbourhoods.

Next consider an intersection

$$\bigcup_{i<q} (\downarrow X_{n_i}) \cap \bigcup_{j<r} (\downarrow X_{m_j}) = \bigcup_{\substack{i<q \\ j<r}} \downarrow (X_{n_i} \cap X_{m_j}) \,.$$

Some of the terms which are $\emptyset$ have to be thrown out — but this requires only a finite number of decisions all computable by assumption. Now we have to rewrite

$$X_{n_i} \cap X_{m_j} = X_{k_{ij}} \,,$$

but the finding of k_{ij} is also computable. *Finally*, we have to re-order the doubly indexed sequence into a singly indexed sequence of length $q \cdot r$, but this is easily seen to be computable also. Therefore, intersections can be "calculated".

It remains to be shown that equality between neighbourhoods in $\mathbb{P}\mathcal{D}$ is decidable. The question really comes down to deciding something like:

$$\downarrow X_k \subseteq \bigcup_{i<q} \downarrow X_{n_i} \,.$$

Now since $X_k \in \downarrow X_k$, we find that the above is just equivalent to:

$$\exists i < q.\, X_k \subseteq X_{n_i} \,.$$

By our assumptions on $\mathcal{D}$, this is decidable. (It is this part of the argument that required the passage to $\mathcal{D}^\dagger$. It does not seem to be generally true that the closure under finite unions of an effectively given system is again effectively given.) $\square$

One of the main reasons that $\mathbb{P}\mathcal{D}$ is like a power domain is the possibility of forming "finite sets".

projection.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.con_
public import DomainTheory.ContinuousLattice.FunctionSpaces.min
public import DomainTheory.ContinuousLattice.FunctionSpaces.const_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition313

/-- **Scott 1972, Proposition 3.13.** `(con, min)` makes `D` a projection of `[D → D]`:
`min ∘ con = id` (a constant's least value is its value) and `con ∘ min ⊑ id` (the
    constant `f(⊥)`
is `≤ f` pointwise, since `f(⊥) ⊑ f(y)` by monotonicity). -/
noncomputable def projection : IsContinuousLatticeProjection D (ScottMap D D) where
  incl := con
  retr := min
  retr_incl := fun _ => rfl
  incl_retr_le := fun f => by
    rw [ScottMap.le_def]
    intro y
    show ((ScottMap.const ((f : D → D) ⊥) : ScottMap D D) : D → D) y ≤ (f : D → D) y
    rw [ScottMap.const_apply]
    exact f.monotone bot_le

end Proposition313

end DomainTheory.ContinuousLattice
```

8

3.8.12 Definition 7.11 — finite sets of elements

DEFINITION 7.11. For elements $x_0, \ldots, x_{n-1} \in |D|$ we define

$$\{x_0, \ldots, x_{n-1}\} = \{z \in \mathbb{P}D \mid \exists X_0 \in x_0 \ldots \exists X_{n-1} \in x_{n-1}. \bigcup_{i<n} (\uparrow X_i) \subseteq z\}.$$

(Note, we could also write $\forall i < n.\, X_i \in z$.) $\square$

projection.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.con_
public import DomainTheory.ContinuousLattice.FunctionSpaces.min
public import DomainTheory.ContinuousLattice.FunctionSpaces.const_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition313

/-- **Scott 1972, Proposition 3.13.** `(con, min)` makes `D` a projection of `[D → D]`:
`min ∘ con = id` (a constant's least value is its value) and `con ∘ min ⊑ id` (the
    constant `f(⊥)`
is `≤ f` pointwise, since `f(⊥) ⊑ f(y)` by monotonicity). -/
noncomputable def projection : IsContinuousLatticeProjection D (ScottMap D D) where
  incl := con
  retr := min
  retr_incl := fun _ => rfl
  incl_retr_le := fun f => by
    rw [ScottMap.le_def]
    intro y
    show ((ScottMap.const ((f : D → D) ⊥) : ScottMap D D) : D → D) y ≤ (f : D → D) y
    rw [ScottMap.const_apply]
    exact f.monotone bot_le

end Proposition313

end DomainTheory.ContinuousLattice
```

9

3.8.13 Proposition 7.12 — finite sets are approximable

PROPOSITION 7.12. The mapping

$$\lambda x_0, \ldots, x_{n-1}. \{x_0, \ldots, x_{n-1}\} : D^n \to \mathbb{P}D$$

is approximable and is computable if D is effectively given. Moreover, the map $\lambda x. \{x\}$ shows that $D \trianglelefteq \mathbb{P}D$, and we also have the law:

$$\{x_0, \ldots, x_{n-1}\} = \{x_0\} \cap \cdots \cap \{x_{n-1}\}$$

as an intersection of filters.

Proof : The second part shows that everything reduces to $\lambda x. \{x\}$. We see that

$$X_k \ (\lambda x. \{x\}) \bigcup_{i<q} (\uparrow X_{n_i}) \text{ iff } \exists i < q. X_k \sqsubseteq X_{n_i}.$$

Thus, $\lambda x. \{x\}$ is an approximable mapping and is computable in the effectively given case.

The proof of the law can be reduced to the special case

$$\{x\} \cap \{y\} = \{x, y\}$$

for the sake of illustration. In terms of finite elements of the two domains D and $\mathbb{P}D$ we find

$$\{\uparrow X\} = \uparrow\uparrow X,$$

and so,

$$\begin{aligned} \{\uparrow X\} \cap \{\uparrow Y\} &= \uparrow\uparrow X \cap \uparrow\uparrow Y \\ &= \uparrow (\uparrow X \cup \uparrow Y) \\ &= \{\uparrow X, \uparrow Y\}. \end{aligned}$$

An equation between approximable functions that checks for finite elements also holds for all elements.

Finally, we note that

$$D \cong D^\dagger \trianglelefteq \mathbb{P}D$$

and that the isomorphism involved is just $\lambda x. \{x\}$ by what we saw on the finite elements. □

Further combinators on the power domain are given in the exercises.

sInf_image_le_sInf_image_of_subset.lean

```
module
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
```

```
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

theorem sInf_image_le_sInf_image_of_subset {α : Type*} {f : α → D} {S T : Set α}
    (hST : S ⊆ T) : sInf (Set.image f T) ≤ sInf (Set.image f S) :=
  sInf_le_sInf fun _ hd => by
    obtain ⟨s, hs, rfl⟩ := hd
    exact ⟨s, hST hs, rfl⟩

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

0

3.8.14 Lecture VII exercises

EXERCISES

3.8.15 Exercise 7.13 — axiomatic neighbourhood structure

Show that an effectively given domain can always be identified with a relation $\mathrm{INCL}(n, m)$ on integers, where the two derived relations

$$\mathrm{CONS}(n, m) \text{ iff } \exists k.\, \mathrm{INCL}(k, n) \text{ and } \mathrm{INCL}(k, m);$$

$$\mathrm{MEET}(n, m, k) \text{ iff } \forall j\, [\mathrm{INCL}(j, k) \text{ iff } \mathrm{INCL}(j, n) \text{ and } \mathrm{INCL}(j, m)]$$

are both recursively decidable, and where the following axioms hold:

(i) $\forall n.\, \mathrm{INCL}(n, n)$;

(ii) $\forall n, m, k.\, \mathrm{INCL}(n, m)$ and $\mathrm{INCL}(m, k)$ imply $\mathrm{INCL}(n, k)$;

(iii) $\exists m\, \forall n.\, \mathrm{INCL}(n, m)$

(iv) $\forall n, m.\, \mathrm{CONS}(n, m)$ implies $\exists k.\, \mathrm{MEET}(n, m, k)$.

(Hint: Consider the neighbourhood system

$$\mathcal{D} = \big\{\{m \in \mathbf{N} \mid \mathrm{INCL}(m, n)\} \mid n \in \mathbf{N}\big\}.$$

Is this essentially any effectively given system?)

sInf_image_le_sInf_image_of_subset.lean

```
module
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

theorem sInf_image_le_sInf_image_of_subset {α : Type*} {f : α → D} {S T : Set α}
    (hST : S ⊆ T) : sInf (Set.image f T) ≤ sInf (Set.image f S) :=
  sInf_le_sInf fun _ hd => by
    obtain ⟨s, hs, rfl⟩ := hd
    exact ⟨s, hST hs, rfl⟩

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

1

3.8.16 Exercise 7.14 — primitive recursive witnesses

(For recursive-function theorists.) Prove the statements after definition 7.2 about the existence of primitive recursive functions for showing things recursively enumerable. (Recall that a non-empty set is r.e. iff it is the range of a primitive recursive function.) Show also that every computable element $y \in |E|$ can be written

$$y = \bigcup \{\uparrow Y_{t(i)} \mid i \in \mathbf{N}\},$$

where $t : \mathbf{N} \to \mathbf{N}$ is primitive recursive and where we may assume
$Y_{t(i+1)} \subseteq Y_{t(i)}$
for all $i \in \mathbf{N}$.

sInf_image_le_sInf_image_of_subset.lean

```
module
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

theorem sInf_image_le_sInf_image_of_subset {α : Type*} {f : α → D} {S T : Set α}
    (hST : S ⊆ T) : sInf (Set.image f T) ≤ sInf (Set.image f S) :=
  sInf_le_sInf fun _ hd => by
    obtain ⟨s, hs, rfl⟩ := hd
    exact ⟨s, hST hs, rfl⟩

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

2

3.8.17 Exercise 7.15 — further constructs effective

Finish the proof of 7.4 and establish similar results for the constructs $(D_0 \otimes D_1)$, $(D_0 \oplus D_1)$ and D^∞. Take into account the various appropriate combinators.

sInf_image_le_sInf_image_of_subset.lean

```
module
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

theorem sInf_image_le_sInf_image_of_subset {α : Type*} {f : α → D} {S T : Set α}
    (hST : S ⊆ T) : sInf (Set.image f T) ≤ sInf (Set.image f S) :=
  sInf_le_sInf fun _ hd => by
    obtain ⟨s, hs, rfl⟩ := hd
    exact ⟨s, hST hs, rfl⟩

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

3

3.8.18 Exercise 7.16 — curry as an enumeration

Let $D_0 = \{X_n \mid n \in \mathbf{N}\}$, $D_1 = \{Y_m \mid m \in \mathbf{N}\}$ and $D_2 = \{Z_k \mid k \in \mathbf{N}\}$ be three effectively given domains. Complete the proof of 7.5 by writing out curry as a relation between neighbourhoods. Is it a recursive set or only a recursively enumerable set?

sInf_image_le_sInf_image_of_subset.lean

```
module
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

theorem sInf_image_le_sInf_image_of_subset {α : Type*} {f : α → D} {S T : Set α}
    (hST : S ⊆ T) : sInf (Set.image f T) ≤ sInf (Set.image f S) :=
  sInf_le_sInf fun _ hd => by
    obtain ⟨s, hs, rfl⟩ := hd
    exact ⟨s, hST hs, rfl⟩

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

4

3.8.19 Exercise 7.17 — recursion over lists

Complete the proof of 7.7 for showing that $D^{\S}$ is effectively given if D is. Include all the combinators of 6.2. Prove also that if E is effectively given and $u : D \to E$ and $v : E \times E \to E$ are computable, then the

unique strict mapping $g : D^{\S} \to E$, where, for $x \in |D|$ and $y, z \in |E|$,

$$g(\mathrm{in}(x)) = u(x), \quad \text{and}$$

$$g(\mathrm{pair}(y, z)) = v(g(y), g(z)),$$

is a computable mapping.

sInf_image_le_sInf_image_of_subset.lean

```
module
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

theorem sInf_image_le_sInf_image_of_subset {α : Type*} {f : α → D} {S T : Set α}
    (hST : S ⊆ T) : sInf (Set.image f T) ≤ sInf (Set.image f S) :=
  sInf_le_sInf fun _ hd => by
    obtain ⟨s, hs, rfl⟩ := hd
    exact ⟨s, hST hs, rfl⟩

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

5

3.8.20 Exercise 7.18 — effective isomorphism

Two effectively given systems D and E are *effectively isomorphic* iff … (complete the sentence!). Show that if D is effectively given then the isomorphism

$$D^\infty \cong (D^\infty)^\infty$$

is effective.

sInf_image_le_sInf_image_of_subset.lean

```
module
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

theorem sInf_image_le_sInf_image_of_subset {α : Type*} {f : α → D} {S T : Set α}
    (hST : S ⊆ T) : sInf (Set.image f T) ≤ sInf (Set.image f S) :=
  sInf_le_sInf fun _ hd => by
    obtain ⟨s, hs, rfl⟩ := hd
    exact ⟨s, hST hs, rfl⟩

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

6

3.8.21 Exercise 7.19 — power domain functor

Prove that $D \mapsto \mathbb{P}D$ is a functor by defining for each $f : D \to E$ a mapping

$$\mathbb{P}f : \mathbb{P}D \to \mathbb{P}E$$

by the formula

$$\bigcup_{i<n} \downarrow X_i \quad \mathbb{P}f \quad \bigcup_{j<m} \downarrow Y_j \quad \text{iff} \quad \forall i < n \exists j < m.\, X_i f Y_j\,.$$

Be sure to check that $\mathbb{P}f$ is approximable and that $\mathbb{P}$ preserves identity maps and composition. If f is computable, is $\mathbb{P}f$? Is there a combinator $\lambda f.\, \mathbb{P}f$? What is

$$\mathbb{P}f(\{x, y\}) = ??$$

sInf_image_le_sInf_image_of_subset.lean

```
module
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

theorem sInf_image_le_sInf_image_of_subset {α : Type*} {f : α → D} {S T : Set α}
    (hST : S ⊆ T) : sInf (Set.image f T) ≤ sInf (Set.image f S) :=
  sInf_le_sInf fun _ hd => by
    obtain ⟨s, hs, rfl⟩ := hd
    exact ⟨s, hST hs, rfl⟩
```

```
end SubspaceExtension
end DomainTheory.ContinuousLattice
```

7

3.8.22 Exercise 7.20 — union on power domains

Show that there is a combinator

$$\text{union} : \mathbb{P}(\mathbb{P}D) \to \mathbb{P}D$$

where for suitable neighbourhoods

$$\bigcup_{i<n} \downarrow \Big(\bigcup_{j<m_i} \downarrow X_{ij} \Big) \quad \text{union} \quad \bigcup_{k<q} \downarrow Y_k \quad \text{iff} \quad \forall i < n \forall j < m_i \exists k < q.\, X_{ij} \subseteq Y_k \,.$$

Is union computable if D is effectively given? What is

$$\text{union}(\{\{x\}, \{y, z\}\}) = ??$$

Are $\mathbb{P}(\mathbb{P}D)$ and $\mathbb{P}D$ generally isomorphic??

sInf_image_le_sInf_image_of_subset.lean

```
module
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension
```

```
variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

theorem sInf_image_le_sInf_image_of_subset {α : Type*} {f : α → D} {S T : Set α}
    (hST : S ⊆ T) : sInf (Set.image f T) ≤ sInf (Set.image f S) :=
  sInf_le_sInf fun _ hd => by
    obtain ⟨s, hs, rfl⟩ := hd
    exact ⟨s, hST hs, rfl⟩

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

8

3.8.23 Exercise 7.21 — distributing the power domain

Is there a non-trivial combinator of type

$$\mathbb{P}(D \to E) \to (\mathbb{P}D \to \mathbb{P}E)\ ?$$

Are there in general any isomorphisms between the systems

$$(D \to \mathbb{P}E),\ \mathbb{P}(D \times E),\ \mathbb{P}D \times \mathbb{P}E\ ??$$

Is there a non-trivial combinator of type

$$\mathbb{P}(D \times E) \times \mathbb{P}(E \times F) \to \mathbb{P}(D \times F)\ ???$$

Is there any connection between

$$\mathbb{P}\mathbf{N} \text{ and } P\mathbf{N}\ ????$$

sInf_image_le_sInf_image_of_subset.lean

```
module
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace
```

```
variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

theorem sInf_image_le_sInf_image_of_subset {α : Type*} {f : α → D} {S T : Set α}
    (hST : S ⊆ T) : sInf (Set.image f T) ≤ sInf (Set.image f S) :=
  sInf_le_sInf fun _ hd => by
    obtain ⟨s, hs, rfl⟩ := hd
    exact ⟨s, hST hs, rfl⟩

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

9

3.8.24 Exercise 7.22 — domain of formal languages

(For algebraists.) Let $\Sigma = \{0,1\}^*$ be the free semigroup. A new domain is constructed by defining a family of sets by the least fixed point theorem as follows:

$$S = \{\Sigma\} \cup \{\{\sigma\} \mid \sigma \in \Sigma\} \cup \{XY \mid X, Y \in S\} \cup \{X \cap Y \mid X, Y \in S \text{ and } X \cap Y \neq \emptyset\}.$$

Here we write: $XY = \{\sigma\tau \mid \sigma \in X \text{ and } \tau \in Y\}$.

Prove that S is an effectively given, positive neighbourhood system. (Hint: The sets in S are each "regular events" in the terminology of automata theory, and we have a decision method for the set algebra of regular events.)

Define multiplication on $|S|$ by

$$xy = \{Z \in S \mid \exists X \in x\ \exists Y \in y.\ XY \subseteq Z\},$$

and show $|S|$ becomes a semigroup with Σ embedded into $|S|$ by the homomorphism $\sigma \mapsto \{X \in S \mid \sigma \in X\}$.

Investigate some *infinite words* in S, say those defined by least fixed points such as: $\vec{\sigma} = \sigma\vec{\sigma}$ and $\vec{\sigma} = \vec{\sigma}\sigma$. Are these equations true:

$$\vec{\sigma}\vec{\sigma} = \vec{\sigma}, \quad \vec{\sigma}\vec{\sigma}\vec{\sigma} = \vec{\sigma}, \quad \vec{\sigma}\vec{1}\vec{\sigma}\vec{1} = \vec{\sigma}\vec{1},$$

and $0\vec{1}\,0\vec{1}\,0\vec{1}\,0\vec{1} = 0\vec{1}\,0\vec{1}$?

sSupMaps.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.sSupMaps_continuous
```

```
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

noncomputable def sSupMaps {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (F : Set (ScottMap D D')) : ScottMap D D'
     :=
  ⟨fun x => sSup (Set.image (fun f : ScottMap D D' => (f : D → D') x) F),
    sSupMaps_continuous F⟩

end ScottMap

end DomainTheory.ContinuousLattice
```

0

3.8.25 Exercise 7.23 — graph and fun computable

Complete the discussion of $P\mathbf{N}$ of Example 7.8. Show that the combinators fun and graph of Exercise 5.14 are computable. Also do the same for

$$\lambda x, y.\, x \cap y, \quad \lambda x, y.\, x \cup y, \quad \text{and } \lambda x, y.\, x + y,$$

where for $x, y \in P\mathbf{N}$ we define

$$x + y = \{n + m \mid n \in x \text{ and } m \in y\}.$$

What are the computable elements of $P\mathbf{N}$?

sSupMaps.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.sSupMaps_continuous
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic
```

```
@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

noncomputable def sSupMaps {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (F : Set (ScottMap D D')) : ScottMap D D'
     :=
  ⟨fun x => sSup (Set.image (fun f : ScottMap D D' => (f : D → D') x) F),
    sSupMaps_continuous F⟩

end ScottMap

end DomainTheory.ContinuousLattice
```

1

3.8.26 Exercise 7.24 — LUCID history domain

(Suggested by the LUCID language of Ashcroft and Wadge: SIAM Jour. Comp. vol. 5 (1976).)

Define a set Γ by

$$\Gamma = \bigcup_{i=0}^{\infty} (\{i\} \times \Gamma) \cup \{\star\}.$$

Define a system

$$L = \{\Gamma\} \cup \{\{i\} \times X \mid i \in \mathbf{N} \text{ and } X \in L\}.$$

Show that L is effectively given. Show that the elements of $|L|$ can be identified with the finite and infinite sequences of natural numbers. What is the connection between B and L?

Show that the combinators of LUCID can be construed as computable mappings of type

$$(L \to T) \to (L \to T)$$

or of type

$$(L \to T) \times (L \to T) \to (L \to T).$$

Conclude that programs in LUCID define computable maps.

sSupMaps.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.sSupMaps_continuous
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

noncomputable def sSupMaps {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (F : Set (ScottMap D D')) : ScottMap D D'
     :=
  ⟨fun x => sSup (Set.image (fun f : ScottMap D D' => (f : D → D') x) F),
    sSupMaps_continuous F⟩

end ScottMap

end DomainTheory.ContinuousLattice
```

2

3.9 Retracts of the universal domain

Dependency diagram. Each box is one item. The arrowhead is on the item that imports. An arrow already implied by a longer path is left out. The introduction has no Lean file, so it is not shown. Blue is a definition, green a proposition, amber an example, red a theorem, and indigo an exercise.

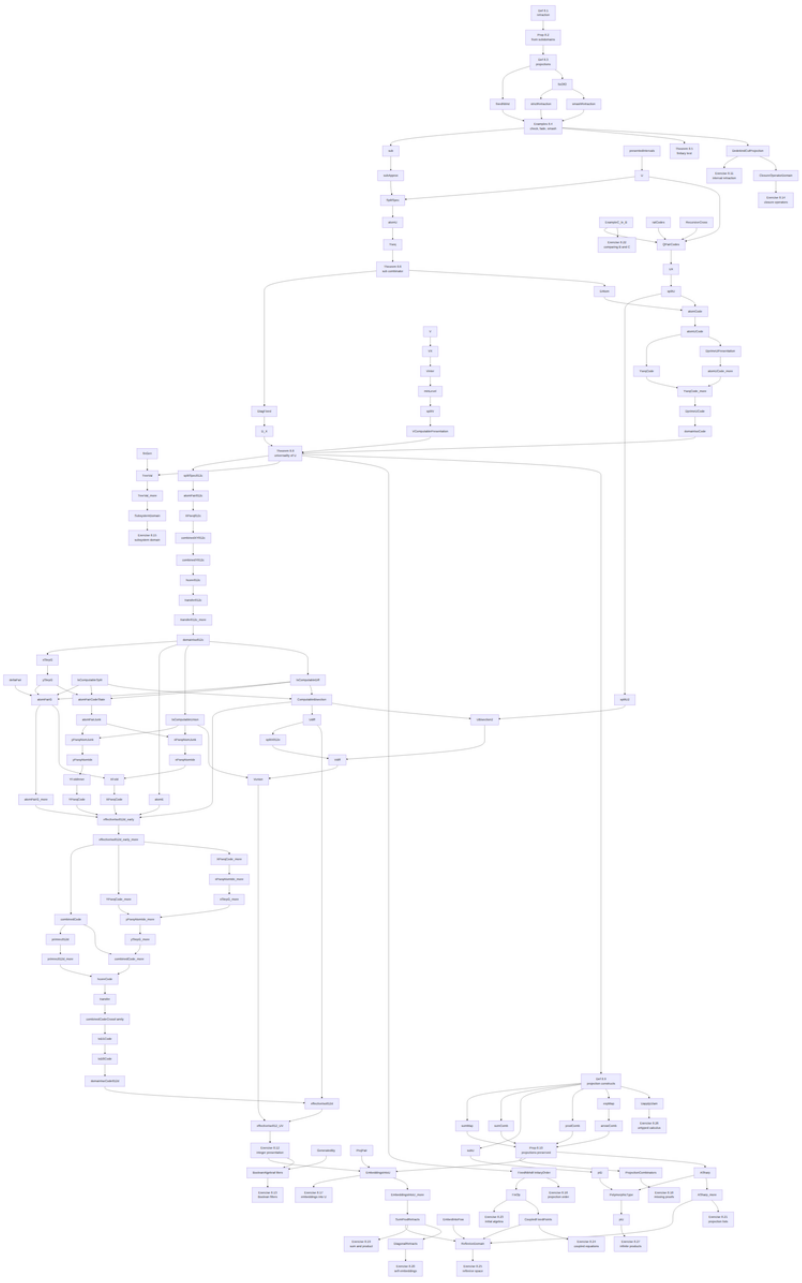


Figure 35: Retracts of the universal domain.

3.9.1 Introduction

LECTURE VIII RETRACTS OF THE UNIVERSAL DOMAIN

In order to be able to have a fully flexible method of solving domain equations and to be able to see why the domains obtained are effectively given, we shall embed all the desired domains in one "largest" domain. This universal domain will be easily shown to be effectively given, and the mappings needed to extract the other domains will be found to be computable. In order to be able to carry out this programme, we

investigate first how certain subdomains correspond to mappings — the so-called *retracts*. An advantage of this analysis is that all the necessary definitions can be written out in λ-calculus notation, thus demonstrating the power of our mathematical programming language.

3.9.2 Definition 8.1 — retraction

DEFINITION 8.1. A retraction of a given domain E is an approximable mapping $a : E \to E$ such that $a \circ a = a$. $\square$

sSupMaps.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.sSupMaps_continuous
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

noncomputable def sSupMaps {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (F : Set (ScottMap D D')) : ScottMap D D'
     :=
  ⟨fun x => sSup (Set.image (fun f : ScottMap D D' => (f : D → D') x) F),
    sSupMaps_continuous F⟩

end ScottMap

end DomainTheory.ContinuousLattice
```

3

3.9.3 Proposition 8.2 — retractions from subdomains

PROPOSITION 8.2. If $D \triangleleft E$ and if $a : E \to E$ is defined by

$$X \; a \; Z \quad \text{iff} \quad \exists Y \in D.\, X \subseteq Y \subseteq Z$$

for all $X, Z \in E$, then a is a retraction and $|D|$ is isomorphic to the fixed-point set of a, the set $\{y \in |E| \mid a(y) = y\}$, under inclusion.

Proof: That a is an approximable mapping is a direct consequence of Definition 6.10. Indeed, in the notation of Proposition 6.12, we have

$$a = i \circ j,$$

and this is another proof that a is approximable. This remark is also convenient, since we know from 6.10

$$j \circ i = I_D.$$

Therefore, we find:

$$a \circ a = i \circ j \circ i \circ j = i \circ j = a;$$

and so a is a retraction.

We can also employ i and j to give the isomorphism on $|D|$. If $x \in |D|$, then $i(x) \in |E|$ and we calculate:

$$a(i(x)) = i \circ j \circ i(x) = i(x).$$

Thus, $i(x)$ belongs to the fixed-point set of a. In the other direction, if $a(y) = y$, then $i(j(y)) = y$. But $j(y) \in |D|$, so i maps $|D|$ one-one and onto the fixed-point set of a. As i and j are monotone, the map is an isomorphism with respect to $\sqsubseteq$. $\square$

Not every retraction comes from a relationship like $D \triangleleft E$; in fact, we can see from the definition of a above that $a \sqsubseteq 1_E$. But, as is indicated in Exercise 8.11, even this condition is not sufficient to characterize the kind of retractions provided by 8.2. The characterization is as follows.

sSupMaps.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.sSupMaps_continuous
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']
```

```
namespace ScottMap

noncomputable def sSupMaps {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (F : Set (ScottMap D D')) : ScottMap D D'
     :=
  ⟨fun x => sSup (Set.image (fun f : ScottMap D D' => (f : D → D') x) F),
    sSupMaps_continuous F⟩

end ScottMap

end DomainTheory.ContinuousLattice
```

4

3.9.4 Definition 8.3 — projections

DEFINITION 8.3. A retraction $a : E \to E$ is called a *projection* provided

$$a \sqsubseteq 1_E;$$

it is *finitary* iff its fixed-point set is isomorphic to a domain. □

sSupMaps.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.sSupMaps_continuous
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

noncomputable def sSupMaps {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (F : Set (ScottMap D D')) : ScottMap D D'
     :=
  ⟨fun x => sSup (Set.image (fun f : ScottMap D D' => (f : D → D') x) F),
    sSupMaps_continuous F⟩
```

```
end ScottMap

end DomainTheory.ContinuousLattice
```

5

3.9.5 Examples 8.4 — check, fade, and smash

EXAMPLES 8.4. If a system D over Δ is not trivial, then the two element system $O = \{\{0\}, \{0,1\}\}$ comes from a retraction on D. Specifically, define a combinator

$$\text{check} : D \to O$$

by the relation

$$X \text{ check } Y \quad \text{iff either } Y = \{0,1\} \text{ or } X \neq \Delta.$$

We see $\text{check}(x) = \bot_O$ iff $x = \bot_D$. We leave to the reader the definition of a combinator:

$$\text{fade} : O \times D \to D,$$

where we have for $t \in |O|$ and $x \in |D|$:

$$\begin{aligned} \text{fade}(t, x) &= \bot_D, \text{ if } t = \bot_O; \\ &= x, \text{ if not.} \end{aligned}$$

Now, take any $u \in |D|$ with $u \neq \bot$, and define

$$a(x) = \text{fade}(\text{check}(x), u).$$

Then a is a retraction (not a projection in general) and the range of a is isomorphic to O.

Another way of using these combinators is to find $(D \to_\bot E)$ as a retraction of $(D \to E)$. Specifically, define a combinator

$$\text{strict} : (D \to E) \to (D \to E)$$

by the equation

$$\text{strict}(f) = \lambda x.\ \text{fade}(\text{check}(x), f(x)),$$

where this time

$$\text{fade} : O \times E \to E\ .$$

The range of strict consists exactly of the strict functions and this time strict is a projection whose range is indeed a domain.

Similarly, we can find a projection on $D \times E$ with a range isomorphic to $D \otimes E$ by the combinator such that:

$$\text{smash}(x, y) = \text{fade}(\text{check}(x), \text{fade}(\text{check}(y), \langle x, y \rangle)),$$

for $x \in |D|$ and $y \in |E|$. □

sSupMaps.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.sSupMaps_continuous
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

noncomputable def sSupMaps {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (F : Set (ScottMap D D')) : ScottMap D D'
     :=
  ⟨fun x => sSup (Set.image (fun f : ScottMap D D' => (f : D → D') x) F),
    sSupMaps_continuous F⟩

end ScottMap

end DomainTheory.ContinuousLattice
```

6

sSupMaps.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.sSupMaps_continuous
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v
```

```
variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

noncomputable def sSupMaps {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (F : Set (ScottMap D D')) : ScottMap D D'
     :=
  ⟨fun x => sSup (Set.image (fun f : ScottMap D D' => (f : D → D') x) F),
    sSupMaps_continuous F⟩

end ScottMap

end DomainTheory.ContinuousLattice
```

7

sSupMaps.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.sSupMaps_continuous
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

noncomputable def sSupMaps {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (F : Set (ScottMap D D')) : ScottMap D D'
     :=
  ⟨fun x => sSup (Set.image (fun f : ScottMap D D' => (f : D → D') x) F),
    sSupMaps_continuous F⟩
```

```
end ScottMap

end DomainTheory.ContinuousLattice
```

8

sSupMaps.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.sSupMaps_continuous
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

noncomputable def sSupMaps {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (F : Set (ScottMap D D')) : ScottMap D D'
     :=
  ⟨fun x => sSup (Set.image (fun f : ScottMap D D' => (f : D → D') x) F),
    sSupMaps_continuous F⟩

end ScottMap

end DomainTheory.ContinuousLattice
```

9

sSupMaps_apply.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSupMaps
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice
```

```
open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

theorem sSupMaps_apply (F : Set (ScottMap D D')) (x : D) :
    (sSupMaps F : D → D') x =
      sSup (Set.image (fun f : ScottMap D D' => (f : D → D') x) F) :=
  rfl

end ScottMap
end DomainTheory.ContinuousLattice
```

0

3.9.6 Theorem 8.5 — finitary projection test

THEOREM 8.5. For an approximable mapping $a : E \to E$ the following are equivalent:

(i) a is a finitary projection;

(ii) $a(x) = \{Y \in E \mid \exists X \in x.\, X \subseteq Y \wedge X\ a\ X\}$, for all $x \in |E|$.

Proof: Suppose a satisfies (ii) first. Inasmuch as $X \in x$ and $X \subseteq Y$ always imply $Y \in x$, for all $x \in |E|$, we see $a(x) \subseteq x$ must always hold. Moreover, it is obvious that $X \in x$ and $X\ a\ X$ always imply $X \in a(x)$; therefore, $a(x) \subseteq a(a(x))$ for all $x \in |E|$. This shows that a
is indeed a projection.

Let $D = \{X \in E \mid X\ a\ X\}$, then it is easy to check that $D \triangleleft E$ and that a is determined from D exactly as in 8.2; thus, the fixed-point set of a is isomorphic to a domain, by what we have already proved. So we have shown (ii) implies (i).

In the converse direction, assume that a is a finitary projection. And let the system D be isomorphic to the fixed-point set of a. We have the situation of Theorem 6.15. There is a projection pair,

$$i : D \to E \quad \text{and} \quad j : E \to D,$$

where the connection with a gives:

$$j \circ i = I_D \quad \text{and} \quad i \circ j = a \subseteq I_E.$$

By 6.15, $D \cong D' \triangleleft E$, and we want to identify D' in terms of a as follows:

$$D' = \{X \in E \mid X\ a\ X\}.$$

Now from a reading of the proof of 6.15 the neighbourhoods of D' are just those corresponding to the finite elements of D. But any such element is a fixed point of a. We have

$$X \in D' \quad \text{implies} \quad a(\uparrow X) = \uparrow X \quad \text{implies} \quad X\ a\ X.$$

Conversely, if $X\ a\ X$ holds, then $\uparrow X \subseteq a(\uparrow X)$. But a is a projection, so $\uparrow X$ is a fixed point. But $i(j(\uparrow X)) = \uparrow X$ means $j(\uparrow X)$ is a finite element of $|D|$. So $X \in D'$, and we have D' identified as desired.

Finally, if we calculate $a = i \circ j$ by the formulae of 6.12 (with D' for D, of course), we obtain our formula (ii). □

The criterion for being a finitary projection just obtained provides us with a very interesting new combinator.

sSupMaps_apply.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSupMaps
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

theorem sSupMaps_apply (F : Set (ScottMap D D')) (x : D) :
    (sSupMaps F : D → D') x =
      sSup (Set.image (fun f : ScottMap D D' => (f : D → D') x) F) :=
  rfl

end ScottMap
end DomainTheory.ContinuousLattice
```

1

3.9.7 Theorem 8.6 — sub combinator

THEOREM 8.6. For any domain E define

$$\mathrm{sub} : (E \to E) \to (E \to E)$$

by the formula

$$X\ \mathrm{sub}\ (f)\ Z \quad \text{iff} \quad \exists Y \in E.\quad X \subseteq Y \ \ f\ Y \subseteq Z,$$

for all $X, Z \in E$ and all $f : E \to E$. Then the range of sub consists exactly of the finitary projections on E, and moreover sub itself is a finitary projection on $(E \to E)$. If E is effectively given, then sub is computable.

Proof: It is trivial to check that $\mathrm{sub}(f)$ is always approximable. Also, it is obvious from the definition that the correspondence

$$f \mapsto \mathrm{sub}(f)$$

preserves directed unions of f's. Thus, sub is itself approximable. We note that

$$X \subseteq Y \;\; f\ Y \subseteq Z \text{ always implies } X\ f\ Z;$$

hence, $\mathrm{sub}(f) \subseteq f$ holds. Also

$$Y\ f\ Y \text{ always implies } Y\ \mathrm{sub}(f)\ Y,$$

hence, $\mathrm{sub}(f) \subseteq \mathrm{sub}(\mathrm{sub}(f))$ holds. This shows sub to be a projection on $(E \to E)$. The effectiveness of the definition makes it also clear that sub is computable when E has a computable presentation.

Since, sub is a projection, its range is the same as its fixed-point set. If

$$\mathrm{sub}(a) = a,$$

then there is no problem in checking that a satisfies 8.5(ii) *and conversely*. So the range of sub picks out exactly the finitary projections in view of 8.5.

Finally, to prove that sub is a finitary projection of $(E \to E)$, we invoke 6.11 and remark that, in view of 8.2, the fixed point set (range) of sub is in a one-one inclusion-preserving correspondence with the domain $\{D \mid D \lhd E\}$. $\square$

These results have almost completely translated the theory of $\lhd$-subdomains into λ-calculus via the sub-combinator. One last step will complete the passage, and then we shall be able to return to solving domain equations.

sSupMaps_apply.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSupMaps
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

theorem sSupMaps_apply (F : Set (ScottMap D D')) (x : D) :
    (sSupMaps F : D → D') x =
      sSup (Set.image (fun f : ScottMap D D' => (f : D → D') x) F) :=
  rfl
```

```
end ScottMap
end DomainTheory.ContinuousLattice
```

2

sSupMaps_apply.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSupMaps
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

theorem sSupMaps_apply (F : Set (ScottMap D D')) (x : D) :
    (sSupMaps F : D → D') x =
      sSup (Set.image (fun f : ScottMap D D' => (f : D → D') x) F) :=
  rfl

end ScottMap
end DomainTheory.ContinuousLattice
```

3

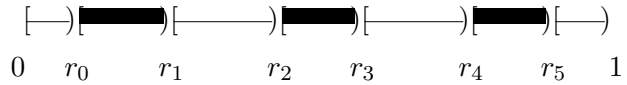
3.9.8 Definition 8.7 — interval presentation of U

DEFINITION 8.7. Let $\mathbb{Q}$ be the set of rational numbers, and let

$$[0,1) = \{q \in \mathbb{Q} \mid 0 \le q < 1\},$$

and similarly for $[r,s)$ for any $r < s$ in $\mathbb{Q}$. The neighbourhood system $\mathcal{U}$ over $[0,1)$ is the set of all non-empty finite unions of rational intervals $[r,s)$ with $0 \le r < s \le 1$. □

A picture of a typical element of $\mathcal{U}$ could be drawn like this:



Note that any union can be taken as a *disjoint* union of the form

$$\bigcup_{i \le n} [r_{2i}, r_{2i+1})$$

where $0 \leq r_0 < r_1 < r_2 < \cdots < r_{2n} < r_{2n+1} \leq 1$. (Hint: Any overlapping intervals or abutting intervals can always be combined into one long interval.) It is a most elementary exercise to show that, by virtue of this representation, the system $\mathcal{U}$ has a computable presentation. (Some isomorphic versions of $\mathcal{U}$ — equally effective — are recorded in the exercises.) Note that $\mathcal{U}$ has no minimal neighbourhoods: every set in $\mathcal{U}$ can be written as the union of two disjoint sets in $\mathcal{U}$. (Hint: Use the density of the ordering of $\mathbb{Q}$.) The significance of $\mathcal{U}$ can now be explained.

sSupMaps_apply.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSupMaps
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

theorem sSupMaps_apply (F : Set (ScottMap D D')) (x : D) :
    (sSupMaps F : D → D') x =
      sSup (Set.image (fun f : ScottMap D D' => (f : D → D') x) F) :=
  rfl

end ScottMap
end DomainTheory.ContinuousLattice
```


sSupMaps_apply.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSupMaps
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
```

```
    CompleteLattice D₁']

namespace ScottMap

theorem sSupMaps_apply (F : Set (ScottMap D D')) (x : D) :
    (sSupMaps F : D → D') x =
      sSup (Set.image (fun f : ScottMap D D' => (f : D → D') x) F) :=
  rfl

end ScottMap
end DomainTheory.ContinuousLattice
```

5

sSupMaps_apply.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSupMaps
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

theorem sSupMaps_apply (F : Set (ScottMap D D')) (x : D) :
    (sSupMaps F : D → D') x =
      sSup (Set.image (fun f : ScottMap D D' => (f : D → D') x) F) :=
  rfl

end ScottMap
end DomainTheory.ContinuousLattice
```

6

sSupMaps_apply.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSupMaps
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice
```

```
open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

theorem sSupMaps_apply (F : Set (ScottMap D D')) (x : D) :
    (sSupMaps F : D → D') x =
      sSup (Set.image (fun f : ScottMap D D' => (f : D → D') x) F) :=
  rfl

end ScottMap
end DomainTheory.ContinuousLattice
```

7

sSupMaps_apply.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSupMaps
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

theorem sSupMaps_apply (F : Set (ScottMap D D')) (x : D) :
    (sSupMaps F : D → D') x =
      sSup (Set.image (fun f : ScottMap D D' => (f : D → D') x) F) :=
  rfl

end ScottMap
end DomainTheory.ContinuousLattice
```

8

sSupMaps_apply.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSupMaps
public import Mathlib.Topology.Defs.Basic
```

```
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

theorem sSupMaps_apply (F : Set (ScottMap D D')) (x : D) :
    (sSupMaps F : D → D') x =
      sSup (Set.image (fun f : ScottMap D D' => (f : D → D') x) F) :=
  rfl

end ScottMap
end DomainTheory.ContinuousLattice
```

9

3.9.9 Theorem 8.8 — universality of U

THEOREM 8.8. The system $\mathcal{U}$ is universal in the sense that, for every countable neighbourhood system $\mathcal{D}$, we have

$$\mathcal{D} \trianglelefteq \mathcal{U}.$$

Moreover, if $\mathcal{D}$ is effectively given, then the projection pair making the embedding can be taken as computable. Indeed there is a correspondence between effectively presented domains and the computable, finitary projections of $\mathcal{U}$.

Proof: As $\mathcal{D}$ is countable, we can assume that

$$\mathcal{D} = \{X_n \mid n \in \mathbb{N}\},$$

where $\mathcal{D}$ is a system over a set Δ (say, $X_0 = \Delta$). We shall do the effective and general cases together, where for the latter all remarks on recursiveness are just left out. So, if we want $\mathcal{D}$ effectively given, the above enumeration should be taken as the computable presentation.

Without loss of generality we can assume $\mathcal{D} \cong \mathcal{D}^\dagger$, since otherwise we would just replace $\mathcal{D}$ by $\mathcal{D}^\dagger$. The advantage of this preparation is that unions in $\mathcal{D}^\dagger$ keep things rather *separate* (as we noticed in constructing $\mathbb{P}\mathcal{D}$). In particular, we can be sure of this equivalence:

$$(\blacklozenge) \qquad X_m \subseteq \bigcup_{i<k} X_{n_i} \quad \text{iff} \quad \exists i < k.\, X_m \subseteq X_{n_i}.$$

This property, for example, fails for the system $\mathcal{U}$ as presented in Definition 8.7. However, that observation is of no moment, because we are employing the assumption with respect to $\mathcal{D}$ not $\mathcal{U}$.

The reason for the assumption is this: for $\delta \in \{+, -\}$ define for $X \in \mathcal{D}$:

$$\begin{aligned} \delta X &= X && \text{if } \delta = +; \\ &= \Delta \setminus X && \text{if } \delta = -. \end{aligned}$$

(A similar notation will be used for $Y \in \mathcal{U}$.) Then for $\delta \in \{+,-\}^n$ the sets of the form

$$\bigcap_{i<n} \delta_i X_i \quad (= X_\delta, \text{ for short})$$

form a partition of Δ into (at most) 2^n parts. The reason for assumption (♦) is that we can effectively decide for each $\delta \in \{+,-\}^n$ whether one of these intersections is empty or not. (Why? — assuming that $\mathcal{D}$ is effectively given, of course). If for some reason we had not wanted to pass to $\mathcal{D}^\dagger$, we could have made this stronger assumption of decidability on the (positive) system $\mathcal{D}$. ($\mathcal{U}$, for example, satisfies it.)

Suppose, corresponding to $X_0, X_1, \ldots, X_{n-1}$, we have selected $Y_0, Y_1, \ldots, Y_{n-1} \in \mathcal{U}$ so that, for all $\delta \in \{+,-\}^n$,

$$(\blacksquare) \qquad \bigcap_{i<n} \delta_i X_i = \emptyset \quad \text{iff} \quad \bigcap_{i<n} \delta_i Y_i = \emptyset.$$

We wish to show - effectively - how to choose Y_n corresponding to X_n, so that (■) holds with $n+1$ replacing n. Proceeding inductively, we obtain a recursive enumeration of sets $Y_n \in \mathcal{U}$ so that

$$\mathcal{D} \equiv \{Y_n \mid n \in \mathbb{N}\} \triangleleft \mathcal{U}.$$

Clearly the isomorphism (matching X_i to Y_i) will be computable and the projection is computable. (It will then remain only to consider the arbitrary finitary computable projection to complete the proof of the theorem.)

So, consider X_n; for each $\delta \in \{+,-\}^n$ there are four cases:

$$\begin{aligned} X_\delta \cap X_n = \emptyset, \quad X_\delta \cap -X_n = \emptyset, \\ X_\delta \cap X_n \neq \emptyset, \quad X_\delta \cap -X_n \neq \emptyset. \end{aligned}$$

Corresponding to X_δ is a similar intersection Y_δ. If X_δ were $\emptyset$, then Y_δ would be also. If not, $Y_\delta \subseteq [0,1)$ is a union of rational intervals that can be written down explicitly. (Why?) In our four cases on X_n, the first implies the fourth. (Why?) Thus, we need only make some choices in these circumstances:

$$\begin{aligned} X_\delta \cap X_n = \emptyset &: \text{choose } I_{\delta,n} = \emptyset; \\ X_\delta \cap -X_n = \emptyset &: \text{choose } I_{\delta,n} = Y_\delta; \\ \text{otherwise} &: \text{choose } I_{\delta,n} \subseteq Y_\delta, \text{ with } \emptyset \neq I_{\delta,n} \neq Y_\delta. \end{aligned}$$

All these cases are decidable by assumption on $\mathcal{D}$, and the effective choice of (unions of) intervals is effective by construction of $\mathcal{U}$. Now set

$$Y_n = \bigcup_{\delta \in \{+,-\}^n} I_{\delta,n} \neq \emptyset.$$

The set $Y_n \in \mathcal{U}$, it can be found effectively, and (■) is obviously satisfied for $n+1$.

Finally, suppose that a is a computable, finitary projection of $\mathcal{U}$. As we have seen in the proof of 8.5, the domain corresponding to the range of a is isomorphic to the neighbourhood system

$$\{Y \in \mathcal{U} \mid Y \; a \; Y\} \triangleleft \mathcal{U}.$$

Clearly, if a as a set of ordered pairs of neighbourhoods is recursively enumerable, then the above set is also recursively enumerable (because equality between neighbourhoods is decidable). It follows easily that the subsystem is effectively given as a neighbourhood system in its own right. □

We have now proved that $\mathcal{U}$ is a nice and big domain that is nicely behaved with respect to computable mappings. It has some very interesting subdomains; to name a few:

$$\mathcal{U}+\mathcal{U}, \quad \mathcal{U}\oplus\mathcal{U}, \quad \mathcal{U}\times\mathcal{U}, \quad \mathcal{U}\otimes\mathcal{U}$$

$$\mathcal{U}_\perp, \quad \mathcal{U}^\infty, \quad \mathcal{U}^\S, \quad \mathbb{P}\mathcal{U}, \quad \mathcal{U}\to\mathcal{U}$$

That all of these are $\trianglelefteq \mathcal{U}$ follows from knowing that they are all effectively presented. What we wish to check next is that they all combine well with respect to projections. To this end the explicit definitions are given for the constructs $+$, $\times$, and $\to$, and the details of the others are left for the exercises.

sSupMaps_continuous.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    pointwise_sSup_preservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

/-- The pointwise supremum function is Scott-continuous. Kept as a named
proof boundary so the Challenge fixes the function while leaving this proof open. -/
theorem sSupMaps_continuous {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (F : Set (ScottMap D D')) :
    @Continuous D D' scottTopologicalSpace scottTopologicalSpace
      (fun x => sSup (Set.image (fun f : ScottMap D D' => (f : D → D') x) F)) :=
```

```
  continuous_of_preservesDirectedSup (pointwise_sSup_preservesDirectedSup F)

end ScottMap

end DomainTheory.ContinuousLattice
```

0

sSupMaps_continuous.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    pointwise_sSup_preservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

/-- The pointwise supremum function is Scott-continuous. Kept as a named
proof boundary so the Challenge fixes the function while leaving this proof open. -/
theorem sSupMaps_continuous {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (F : Set (ScottMap D D')) :
    @Continuous D D' scottTopologicalSpace scottTopologicalSpace
      (fun x => sSup (Set.image (fun f : ScottMap D D' => (f : D → D') x) F)) :=
  continuous_of_preservesDirectedSup (pointwise_sSup_preservesDirectedSup F)

end ScottMap

end DomainTheory.ContinuousLattice
```

1

sSupMaps_continuous.lean

```
module
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    pointwise_sSup_preservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

/-- The pointwise supremum function is Scott-continuous. Kept as a named
proof boundary so the Challenge fixes the function while leaving this proof open. -/
theorem sSupMaps_continuous {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (F : Set (ScottMap D D')) :
    @Continuous D D' scottTopologicalSpace scottTopologicalSpace
      (fun x => sSup (Set.image (fun f : ScottMap D D' => (f : D → D') x) F)) :=
  continuous_of_preservesDirectedSup (pointwise_sSup_preservesDirectedSup F)

end ScottMap

end DomainTheory.ContinuousLattice
```

2

3.9.10 Definition 8.9 — constructs on projections

DEFINITION 8.9. Let the computable projection pairs $i_+ : \mathcal{U} + \mathcal{U} \to \mathcal{U}$ and $j_+ : \mathcal{U} \to \mathcal{U} + \mathcal{U}$ be fixed. Similarly choose $i_\times, j_\times$ and $i_\to, j_\to$ for $\mathcal{U} \times \mathcal{U}$ and $\mathcal{U} \to \mathcal{U}$. Define:

$$a + b = \mathrm{cond} \circ \langle \mathrm{which}, i_+ \circ \mathrm{in}_0 \circ a \circ \mathrm{out}_0, i_+ \circ \mathrm{in}_1 \circ b \circ \mathrm{out}_1 \rangle \circ j_+ \, ;$$

$$a \times b = i_\times \circ \langle a \circ \mathrm{proj}_0, b \circ \mathrm{proj}_1 \rangle \circ j_\times \, ;$$

$$a \to b = i_\to \circ (\lambda f.\, b \circ f \circ a) \circ j_\to \, ,$$

for all $a, b : \mathcal{U} \to \mathcal{U}$. □

These interesting(computable!) combinators on elements of $\mathcal{U} \to \mathcal{U}$ have many, many properties. We shall, however, only see what they do to projections.

— embeddings and `lamComb`

sSupMaps_continuous.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    pointwise_sSup_preservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

/-- The pointwise supremum function is Scott-continuous. Kept as a named
proof boundary so the Challenge fixes the function while leaving this proof open. -/
theorem sSupMaps_continuous {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (F : Set (ScottMap D D')) :
    @Continuous D D' scottTopologicalSpace scottTopologicalSpace
      (fun x => sSup (Set.image (fun f : ScottMap D D' => (f : D → D') x) F)) :=
  continuous_of_preservesDirectedSup (pointwise_sSup_preservesDirectedSup F)

end ScottMap

end DomainTheory.ContinuousLattice
```

3

— `sumComb` and Pref-folded companions

sSupMaps_continuous.lean

```
module
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    pointwise_sSup_preservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

/-- The pointwise supremum function is Scott-continuous. Kept as a named
proof boundary so the Challenge fixes the function while leaving this proof open. -/
theorem sSupMaps_continuous {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (F : Set (ScottMap D D')) :
    @Continuous D D' scottTopologicalSpace scottTopologicalSpace
      (fun x => sSup (Set.image (fun f : ScottMap D D' => (f : D → D') x) F)) :=
  continuous_of_preservesDirectedSup (pointwise_sSup_preservesDirectedSup F)

end ScottMap

end DomainTheory.ContinuousLattice
```

4

— `prodComb` and Pref-folded companions

`sSupMaps_continuous.lean`

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    pointwise_sSup_preservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section
```

```
namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

/-- The pointwise supremum function is Scott-continuous. Kept as a named
proof boundary so the Challenge fixes the function while leaving this proof open. -/
theorem sSupMaps_continuous {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (F : Set (ScottMap D D')) :
    @Continuous D D' scottTopologicalSpace scottTopologicalSpace
      (fun x => sSup (Set.image (fun f : ScottMap D D' => (f : D → D') x) F)) :=
  continuous_of_preservesDirectedSup (pointwise_sSup_preservesDirectedSup F)

end ScottMap

end DomainTheory.ContinuousLattice
```

5

— `arrowComb` and Pref-folded companions

sSupMaps_continuous.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    pointwise_sSup_preservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]
```

```
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

/-- The pointwise supremum function is Scott-continuous. Kept as a named
proof boundary so the Challenge fixes the function while leaving this proof open. -/
theorem sSupMaps_continuous {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (F : Set (ScottMap D D')) :
    @Continuous D D' scottTopologicalSpace scottTopologicalSpace
      (fun x => sSup (Set.image (fun f : ScottMap D D' => (f : D → D') x) F)) :=
  continuous_of_preservesDirectedSup (pointwise_sSup_preservesDirectedSup F)

end ScottMap

end DomainTheory.ContinuousLattice
```

6 — expMap and Pref-folded companions

sSupMaps_continuous.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    pointwise_sSup_preservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

/-- The pointwise supremum function is Scott-continuous. Kept as a named
proof boundary so the Challenge fixes the function while leaving this proof open. -/
theorem sSupMaps_continuous {D : Type u} {D' : Type v}
```

```
    [CompleteLattice D] [CompleteLattice D'] (F : Set (ScottMap D D')) :
    @Continuous D D' scottTopologicalSpace scottTopologicalSpace
      (fun x => sSup (Set.image (fun f : ScottMap D D' => (f : D → D') x) F)) :=
  continuous_of_preservesDirectedSup (pointwise_sSup_preservesDirectedSup F)

end ScottMap

end DomainTheory.ContinuousLattice
```

7

3.9.11 Proposition 8.10 — constructs preserve projections

PROPOSITION 8.10. If $a, b : \mathcal{U} \to \mathcal{U}$ are projections, then so are $a + b$, $a \times b$, and $a \to b$. If a and b are finitary, then so are the others; for the fixed-point set of each of them is isomorphic to the corresponding construct applied to the domains determined by a and b.

Proof: Suppose that $a, b \sqsubseteq I_{\mathcal{U}}$ (= I for short). Then

$$a + b \sqsubseteq I + I = i_+ \circ j_+ \sqsubseteq I.$$

The other cases are similar.

Suppose $a = a \circ a$ and $b = b \circ b$, then, for example,

$$\begin{aligned}(a \times b) \circ (a \times b) &= i_\times \circ \langle a \circ \mathrm{proj}_0,\ b \circ \mathrm{proj}_1 \rangle \circ \langle a \circ \mathrm{proj}_0,\ b \circ \mathrm{proj}_1 \rangle \circ j_\times \\ &= i_\times \circ \langle a \circ a \circ \mathrm{proj}_0,\ b \circ b \circ \mathrm{proj}_1 \rangle \circ j_\times \\ &= a \times b.\end{aligned}$$

The other cases are similar.

Now in case the fixed-point sets of a and b are domains, they are respectively isomorphic to

$$D_a = \{X \in \mathcal{U} \mid X\ a\ X\} \quad \text{and}$$
$$D_b = \{Y \in \mathcal{U} \mid Y\ b\ Y\}.$$

We have to show, for example, that

$$D_a \to D_b \cong D_{a \to b}.$$

Now to simplify matters, remark that the fixed-point set of $a \to b$ on $\mathcal{U}$ is isomorphic to the fixed-point set of $\lambda f.\ b \circ f \circ a$ on $(\mathcal{U} \to \mathcal{U})$. (Hint: use $i_\to$ and $j_\to$ to set up the isomorphism.) So we have to think what it is for an $f : \mathcal{U} \to \mathcal{U}$ to satisfy

$$f = b \circ f \circ a.$$

Notice that we might as well say that $a : \mathcal{U} \to D_a$ and that this map is the other half of an obvious projection pair where

$$i_a : D_a \to \mathcal{U},$$

and $i_a \circ a = a$ and $a \circ i_a = i_a$. So if $g : D_a \to D_b$, let

$$f = i_b \circ g \circ a;$$

then $b \circ f \circ a = f$. Conversely, if f is like this, then let

$$g = b \circ f \circ i_a.$$

Thus, $i_b \circ g \circ a = b \circ f \circ a = f$; so there is an order-preserving isomorphism between the $g : D_a \to D_b$ and the $f = b \circ f \circ a$.

The isomorphism proofs for $+$ and $\times$ are similar. $\Box$

Well, this was a lot of work, but the pay-off is rather handsome. What we have done is transpose all the

$$D_a \triangleleft U$$

over to finitary projections $a : U \to U$. This transposition is an isomorphism, because

$$D_a \triangleleft D_b \text{ iff } a \sqsubseteq b.$$

Moreover, by the method of 8.9 and 8.10, all our favourite constructs have been made into *combinators*, that is, approximable — even computable — maps on the domain of finitary projections. *ALL APPROXIMABLE (COMPUTABLE) MAPS HAVE (COMPUTABLE) FIXED POINTS.* And there you are! The standard fixed-point method is available to obtain computable (i.e. effectively given) solutions to *all* domain equations (even sets of equations) where the constructs can be reworked in *this* way to be defined on projections. Examples are suggested in the exercises.

Another pay-off concerns the λ-calculus itself. Inasmuch as

$$U + U, \quad U \times U, \quad U \to U \triangleleft U,$$

we might just as well forget the outside world and regard all these useful domains as being part of U. For example, on the left we have the new notation and on the right the old notation:

$$\begin{aligned}
\mathrm{which}(z) &= \mathrm{which}(j_+(z)); \\
\mathrm{in}_i(x) &= i_+(\mathrm{in}_i(x)), \quad i = 0, 1; \\
\mathrm{out}_i(x) &= \mathrm{out}_i(j_+(x)), \quad i = 0, 1; \\
\langle x, y \rangle &= i_\times(\langle x, y \rangle); \\
\mathrm{proj}_i(z) &= \mathrm{proj}_i(j_\times(z)), \quad i = 0, 1; \\
u(x) &= j_\to(u)(x); \\
\lambda x.\, \tau &\cong i_\to(\lambda x.\, \tau).
\end{aligned}$$

And, there is no reason to stop here. The system

$$T \cong \{\{0, 1/2\}, \{1/2, 1\}, \{0, 1\}\} \triangleleft U,$$

so we might as well think of

$$\text{true, false} \in |U|$$

and think of $\mathrm{cond} : U \times U \times U \to U$. No! that is wrong: under the new regime *EVERYTHING IS AN ELEMENT OF U*. With the new meaning of λ, all functions, all pairs, all combinators, all constructs become elements of U.

It takes a little time to get used to "universal conscription" with all elements doing (at least) double duty in the same domain, but there are many advantages, both notational and conceptual.

sSupMaps_continuous.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    pointwise_sSup_preservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

/-- The pointwise supremum function is Scott-continuous. Kept as a named
proof boundary so the Challenge fixes the function while leaving this proof open. -/
theorem sSupMaps_continuous {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (F : Set (ScottMap D D')) :
    @Continuous D D' scottTopologicalSpace scottTopologicalSpace
      (fun x => sSup (Set.image (fun f : ScottMap D D' => (f : D → D') x) F)) :=
  continuous_of_preservesDirectedSup (pointwise_sSup_preservesDirectedSup F)

end ScottMap

end DomainTheory.ContinuousLattice
```

8

3.9.12 Lecture VIII exercises

EXERCISES

3.9.13 Exercise 8.11 — retraction on intervals

Let $\mathbb{Q}$ be the set of rational numbers and define a neighbourhood system by the equation

$$R = \{[0, r) \mid r \in \mathbb{Q} \text{ and } 0 < r \leq 1\}.$$

Show that the following defines an approximable map $a : R \to R$:

$$[0, r)\ a\ [0, s) \quad \text{iff} \quad r < s \text{ or } r = s = 1.$$

Show in addition that a is a projection where the fixed-point set of a is in a one-one correspondence with the *real* numbers between 0 and 1 inclusive. (Hint: Recall Dedekind cuts and show $\subseteq$ matches $\leq$.) Conclude that a is *NOT* finitary. (Hint: Aside from $\perp$ there are no finite elements for $\{x \mid x = a(x)\}$.)

Tip summary The tip records that Scott's map `a` on `R` is a projection and is not finitary.

Development Builds the Dedekind-cut system `R` and proves the projection `a` is not finitary.

sSupMaps_continuous.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    pointwise_sSup_preservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

/-- The pointwise supremum function is Scott-continuous. Kept as a named
proof boundary so the Challenge fixes the function while leaving this proof open. -/
theorem sSupMaps_continuous {D : Type u} {D' : Type v}
    [CompleteLattice D] [CompleteLattice D'] (F : Set (ScottMap D D')) :
    @Continuous D D' scottTopologicalSpace scottTopologicalSpace
```

```
      (fun x => sSup (Set.image (fun f : ScottMap D D' => (f : D → D') x) F)) :=
  continuous_of_preservesDirectedSup (pointwise_sSup_preservesDirectedSup F)

end ScottMap

end DomainTheory.ContinuousLattice
```

9

sSup_apply.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst30
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

theorem sSup_apply (F : Set (ScottMap D D')) (x : D) :
    ((sSup F : ScottMap D D') : D → D') x =
      sSup (Set.image (fun f : ScottMap D D' => (f : D → D') x) F) :=
  rfl

end ScottMap

end DomainTheory.ContinuousLattice
```

0

The tip then assembles those tools into Scott's statement.

— exercise_8_11

3.9.14 Exercise 8.12 — integer presentation of U

sSup_apply.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst30
```

```
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

theorem sSup_apply (F : Set (ScottMap D D')) (x : D) :
    ((sSup F : ScottMap D D') : D → D') x =
      sSup (Set.image (fun f : ScottMap D D' => (f : D → D') x) F) :=
  rfl

end ScottMap

end DomainTheory.ContinuousLattice
```

1

sSup_apply.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst30
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap
```

```
theorem sSup_apply (F : Set (ScottMap D D')) (x : D) :
    ((sSup F : ScottMap D D') : D → D') x =
      sSup (Set.image (fun f : ScottMap D D' => (f : D → D') x) F) :=
  rfl

end ScottMap

end DomainTheory.ContinuousLattice
```

2

sSup_apply.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst30
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

theorem sSup_apply (F : Set (ScottMap D D')) (x : D) :
    ((sSup F : ScottMap D D') : D → D') x =
      sSup (Set.image (fun f : ScottMap D D' => (f : D → D') x) F) :=
  rfl

end ScottMap

end DomainTheory.ContinuousLattice
```

3

sSup_apply.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst30
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section
```

```
namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

theorem sSup_apply (F : Set (ScottMap D D')) (x : D) :
    ((sSup F : ScottMap D D') : D → D') x =
      sSup (Set.image (fun f : ScottMap D D' => (f : D → D') x) F) :=
  rfl

end ScottMap

end DomainTheory.ContinuousLattice
```

4

sSup_apply.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst30
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

theorem sSup_apply (F : Set (ScottMap D D')) (x : D) :
    ((sSup F : ScottMap D D') : D → D') x =
      sSup (Set.image (fun f : ScottMap D D' => (f : D → D') x) F) :=
  rfl
```

```
end ScottMap

end DomainTheory.ContinuousLattice
```

5

sSup_apply.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst30
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

theorem sSup_apply (F : Set (ScottMap D D')) (x : D) :
    ((sSup F : ScottMap D D') : D → D') x =
      sSup (Set.image (fun f : ScottMap D D' => (f : D → D') x) F) :=
  rfl

end ScottMap

end DomainTheory.ContinuousLattice
```

6

sSup_apply.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst30
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace
```

```
variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

theorem sSup_apply (F : Set (ScottMap D D')) (x : D) :
    ((sSup F : ScottMap D D') : D → D') x =
      sSup (Set.image (fun f : ScottMap D D' => (f : D → D') x) F) :=
  rfl

end ScottMap

end DomainTheory.ContinuousLattice
```

7

sSup_apply.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst30
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

theorem sSup_apply (F : Set (ScottMap D D')) (x : D) :
    ((sSup F : ScottMap D D') : D → D') x =
      sSup (Set.image (fun f : ScottMap D D' => (f : D → D') x) F) :=
  rfl

end ScottMap

end DomainTheory.ContinuousLattice
```

8

EXERCISE 8.12. Generalize the notation $2X + 1$ for subsets $X \subseteq \mathbb{N}$ to sets of the form

$$2^k X + \ell, \quad \text{where } \ell < 2^k.$$

Let V be the non-empty finite unions of sets $2^k\mathbb{N} + \ell$. Show that $U \cong V$ and that the isomorphism is effective, thus obtaining another presentation of U.

Tip summary Scott's claim is a single conjunction: V is effectively given, and U is effectively isomorphic to V. The Lean tip `exercise_8_12` packages exactly that, assembling `V_isEffectivelyGiven` with the concrete effective isomorphism `effectiveIso812_UV`.

Development Scott gives no written proof—only the construction of V and the demand that the isomorphism be effective. The formalization therefore supplies a structured development under the exercise: present V, build an abstract domain isomorphism, make it effective, equip both sides with computable bisections/splits, and assemble the effective iso.

Presenting V The integer neighbourhood system V is built from affine residue classes $2^k\mathbb{N} + \ell$ and their non-empty finite unions (`affine` / `levelSet` / `upsample`). Pure neighbourhood facts live under `namespace V` (`V.union_mem`, `V.diff_mem`, `V.isPositive`, ...).

— `V` and Pref-folded neighbourhood lemmas

Its computable presentation (indices, decoding, intersection/union/difference codes, and the canonical bisection `splitV*`) lives in a dedicated library module.

— computable presentation of `V`, including `VComputablePresentation` and `V.isEffectivelyGiven`

Abstract isomorphism (8.12(c)) Given two positive, difference-closed, atomless neighbourhood systems with enumerations that cover every neighbourhood and start at the master set, one builds paired atom sequences (`XPseq`/`YPseq`) from mutual `SplitSpec'` probes and obtains a domain isomorphism. The headline is `isomorphic_812c`.

— difference-closure / no-minimal apparatus, atom pairing, and `isomorphic_812c`

`sSup_apply.lean`

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst30
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
```

```
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

theorem sSup_apply (F : Set (ScottMap D D')) (x : D) :
    ((sSup F : ScottMap D D') : D → D') x =
      sSup (Set.image (fun f : ScottMap D D' => (f : D → D') x) F) :=
  rfl

end ScottMap

end DomainTheory.ContinuousLattice
```

9

Abstract E-side atoms The one-sided atom apparatus against an abstract neighbourhood system `E` (the `atomU` analogue) lives under `atomE`, with Pref-folded companions `atomE.zero`, `atomE.succ`, `atomE.congr`, `atomE.invariant`, …

— `atomE`, `YseqE`, and Pref-folded `atomE.xxx` lemmas

`sSup_finsetSupOf.lean`

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.finsetSupOf
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_le_finsetSupOf
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_sSup_finsetSupOf
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

theorem sSup_finsetSupOf (S : Set D) : sSup S = sSup (finsetSupOf S) :=
  le_antisymm (le_sSup_finsetSupOf S) (sSup_le_finsetSupOf S)
```

```
end IsContinuousLatticeProjection

end Proposition310

end DomainTheory.ContinuousLattice
```

0

Computable presentation predicates The data-carrying structures for computable difference, split, union, and bisection each live in their own library module, with Pref-folded companions under the structure namespace.

— `IsComputableDiff` and Diff-side deciders

`sSup_finsetSupOf.lean`

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.finsetSupOf
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_le_finsetSupOf
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_sSup_finsetSupOf
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

theorem sSup_finsetSupOf (S : Set D) : sSup S = sSup (finsetSupOf S) :=
  le_antisymm (le_sSup_finsetSupOf S) (sSup_le_finsetSupOf S)

end IsContinuousLatticeProjection

end Proposition310

end DomainTheory.ContinuousLattice
```

1

— `IsComputableSplit`

— IsComputableUnion and combineFound2

sSup_finsetSupOf.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.finsetSupOf
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_le_finsetSupOf
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_sSup_finsetSupOf
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

theorem sSup_finsetSupOf (S : Set D) : sSup S = sSup (finsetSupOf S) :=
  le_antisymm (le_sSup_finsetSupOf S) (sSup_le_finsetSupOf S)

end IsContinuousLatticeProjection

end Proposition310

end DomainTheory.ContinuousLattice
```

2

— ComputableBisection and bisection-derived splits

sSup_finsetSupOf.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.finsetSupOf
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_le_finsetSupOf
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_sSup_finsetSupOf
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice
```

```
open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

theorem sSup_finsetSupOf (S : Set D) : sSup S = sSup (finsetSupOf S) :=
  le_antisymm (le_sSup_finsetSupOf S) (sSup_le_finsetSupOf S)

end IsContinuousLatticeProjection

end Proposition310

end DomainTheory.ContinuousLattice
```

3

Effective isomorphism (8.12(d)) The same back-and-forth is carried out on computable presentations: coded atom sequences, primitivity of the step maps, and the order isomorphism `domainIsoCode812d` lift to an `EffectiveIso`. The generic assembly is `effectiveIso812d`.

— coded pairing, primrec witnesses, and `effectiveIso812d`

sSup_finsetSupOf.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.finsetSupOf
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_le_finsetSupOf
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_sSup_finsetSupOf
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
```

```
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

theorem sSup_finsetSupOf (S : Set D) : sSup S = sSup (finsetSupOf S) :=
  le_antisymm (le_sSup_finsetSupOf S) (sSup_le_finsetSupOf S)

end IsContinuousLatticeProjection

end Proposition310

end DomainTheory.ContinuousLattice
```

4

Computable split of U against V (8.12(e)) With `U`'s computable difference and V's bisection in hand, `splitX812e` is the concrete `IsComputableSplit` of U probing V (`isComputableSplit.«812e»`).
— `U_isComputableDiff`, `splitX812e`, and `isComputableSplit.«812e»`

sSup_finsetSupOf.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.finsetSupOf
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_le_finsetSupOf
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_sSup_finsetSupOf
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

theorem sSup_finsetSupOf (S : Set D) : sSup S = sSup (finsetSupOf S) :=
```

```
  le_antisymm (le_sSup_finsetSupOf S) (sSup_le_finsetSupOf S)

end IsContinuousLatticeProjection

end Proposition310

end DomainTheory.ContinuousLattice
```

5

Symmetric split and UV assembly (8.12(f)–(g)) The mirror split `splitX812f` (V probing U via `UBisection2`), both `SplitSpec'` halves, computable unions, and the final `EffectiveIso U V` are assembled as `effectiveIso812_UV`.

— «`812f`», `hxSplit812e`/`hySplit812f`, unions, and `effectiveIso812_UV`

sSup_finsetSupOf.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.finsetSupOf
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_le_finsetSupOf
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_sSup_finsetSupOf
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

theorem sSup_finsetSupOf (S : Set D) : sSup S = sSup (finsetSupOf S) :=
  le_antisymm (le_sSup_finsetSupOf S) (sSup_le_finsetSupOf S)

end IsContinuousLatticeProjection

end Proposition310

end DomainTheory.ContinuousLattice
```

6

The tip then simply pairs effective givenness of V with that isomorphism, which is Exercise 8.12 in Scott's sense.

— Scott tip `exercise_8_12`

`sSup_finsetSupOf.lean`

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.finsetSupOf
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_le_finsetSupOf
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_sSup_finsetSupOf
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

theorem sSup_finsetSupOf (S : Set D) : sSup S = sSup (finsetSupOf S) :=
  le_antisymm (le_sSup_finsetSupOf S) (sSup_le_finsetSupOf S)

end IsContinuousLatticeProjection

end Proposition310

end DomainTheory.ContinuousLattice
```

7

3.9.15 Exercise 8.13 — Boolean-algebra filters

(For logicians.) Prove that the universal domain U is isomorphic to the domain of all proper filters of the free Boolean algebra on $\aleph_0$-generators (= the Lindenbaum algebra of propositional calculus). (For topologists.) Connect this

representation of $\mathcal{U}$ with the collection of non-empty open subsets of the product space $2^{\mathbf{N}}$ (= Cantor space).

Tip summary The tip is the isomorphism $U \cong V$, obtained from the effective iso of Exercise 8.12 together with the Boolean-algebra presentation of `V`.

Development Defines the Boolean algebra generated by bit-test generators.

`sSup_finsetSupOf.lean`

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.finsetSupOf
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_le_finsetSupOf
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_sSup_finsetSupOf
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

theorem sSup_finsetSupOf (S : Set D) : sSup S = sSup (finsetSupOf S) :=
  le_antisymm (le_sSup_finsetSupOf S) (sSup_le_finsetSupOf S)

end IsContinuousLatticeProjection

end Proposition310

end DomainTheory.ContinuousLattice
```

8
Independence and filter facts that identify `V` with filters of that algebra.
The tip then assembles those tools into Scott's statement.
— `exercise_8_13`

`sSup_finsetSupOf.lean`

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.finsetSupOf
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_le_finsetSupOf
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_sSup_finsetSupOf
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

theorem sSup_finsetSupOf (S : Set D) : sSup S = sSup (finsetSupOf S) :=
  le_antisymm (le_sSup_finsetSupOf S) (sSup_le_finsetSupOf S)

end IsContinuousLatticeProjection

end Proposition310

end DomainTheory.ContinuousLattice
```

9

3.9.16 Exercise 8.14 — closure operators

A retraction $a : D \to D$ is called a *closure operator* iff $I_D \sqsubseteq a$. On a domain like $P\mathbf{N}$, give some examples of closure operators. (Hint: Close up a set of integers under addition. Is this continuous on $P\mathbf{N}$?) Prove in general for any closure $a : D \to D$ that the fixed-point set of a is always a finitary domain. (Hint: Show that the fixed-point set is closed under intersections and directed unions.) What are the finite elements of the fixed-point set?

Tip summary The tip states that the fixed-point set of any closure operator is a finitary domain.

Development Defines closure operators and shows `Fix(a)` is a finitary domain.

sSup_image_prod_mk_left.lean

```
module
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Order.CompleteLattice.Basic
```

```
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem sSup_image_prod_mk_left (x : D) (S : Set D') (hS : S.Nonempty) :
    sSup (Set.image (fun y => (x, y)) S) = (x, sSup S) := by
  have himage : Set.image (fun y => (x, y)) S = ({x} : Set D) ×ˢ S := by
    ext ⟨a, b⟩
    simp only [Set.mem_image, Set.mem_prod, Set.mem_singleton_iff]
    constructor
    · rintro ⟨y, hy, h⟩
      obtain ⟨ha, hb⟩ := Prod.ext_iff.mp h
      subst hb
      exact ⟨ha.symm, hy⟩
    · rintro ⟨ha, hb⟩
      refine ⟨b, hb, Prod.ext_iff.mpr ⟨ha.symm, rfl⟩⟩
  have hx : sSup ({x} : Set D) = x := by
    apply le_antisymm
    · exact sSup_le fun z hz => by rw [Set.mem_singleton_iff] at hz; rw [hz]
    · exact le_sSup (Set.mem_singleton x)
  rw [himage, sSup_prod (hs := ⟨x, Set.mem_singleton x⟩) (ht := hS), hx]

end DomainTheory.ContinuousLattice
```

0

sSup_image_prod_mk_left.lean

```
module
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
```

```
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem sSup_image_prod_mk_left (x : D) (S : Set D') (hS : S.Nonempty) :
    sSup (Set.image (fun y => (x, y)) S) = (x, sSup S) := by
  have himage : Set.image (fun y => (x, y)) S = ({x} : Set D) ×ˢ S := by
    ext ⟨a, b⟩
    simp only [Set.mem_image, Set.mem_prod, Set.mem_singleton_iff]
    constructor
    · rintro ⟨y, hy, h⟩
      obtain ⟨ha, hb⟩ := Prod.ext_iff.mp h
      subst hb
      exact ⟨ha.symm, hy⟩
    · rintro ⟨ha, hb⟩
      refine ⟨b, hb, Prod.ext_iff.mpr ⟨ha.symm, rfl⟩⟩
  have hx : sSup ({x} : Set D) = x := by
    apply le_antisymm
    · exact sSup_le fun z hz => by rw [Set.mem_singleton_iff] at hz; rw [hz]
    · exact le_sSup (Set.mem_singleton x)
  rw [himage, sSup_prod (hs := ⟨x, Set.mem_singleton x⟩) (ht := hS), hx]

end DomainTheory.ContinuousLattice
```

1

The tip then assembles those tools into Scott's statement.

— `exercise_8_14`

3.9.17 Exercise 8.15 — domain of subsystems

Give a direct proof that the domain $\{X \mid X \triangleleft D\}$ is effectively presented if D is. (Hint: The finite elements of the domain correspond exactly to the finite systems $X \triangleleft D$.) In the case of $D = \mathcal{U}$, show that the

computable elements of the domain correspond exactly to the effectively presented domains (up to effective isomorphism).

Tip summary The tip states that if `D` is effectively given then so is (a presentation of) the domain of its subsystems.

Development Library tools: finitely generated subsystems and the reconstructed system `SubD`.
— `finGen`, `finGenSys`, `SubD`

sSup_image_prod_mk_left.lean

```
module
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem sSup_image_prod_mk_left (x : D) (S : Set D') (hS : S.Nonempty) :
    sSup (Set.image (fun y => (x, y)) S) = (x, sSup S) := by
  have himage : Set.image (fun y => (x, y)) S = ({x} : Set D) ×ˢ S := by
    ext ⟨a, b⟩
    simp only [Set.mem_image, Set.mem_prod, Set.mem_singleton_iff]
    constructor
    · rintro ⟨y, hy, h⟩
      obtain ⟨ha, hb⟩ := Prod.ext_iff.mp h
      subst hb
      exact ⟨ha.symm, hy⟩
    · rintro ⟨ha, hb⟩
      refine ⟨b, hb, Prod.ext_iff.mpr ⟨ha.symm, rfl⟩⟩
```

```
  have hx : sSup ({x} : Set D) = x := by
    apply le_antisymm
    · exact sSup_le fun z hz => by rw [Set.mem_singleton_iff] at hz; rw [hz]
    · exact le_sSup (Set.mem_singleton x)
  rw [himage, sSup_prod (hs := ⟨x, Set.mem_singleton x⟩) (ht := hS), hx]

end DomainTheory.ContinuousLattice
```

2

— QBody, QFuel, TreeVal, Q′X

sSup_image_prod_mk_left.lean

```
module
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem sSup_image_prod_mk_left (x : D) (S : Set D') (hS : S.Nonempty) :
    sSup (Set.image (fun y => (x, y)) S) = (x, sSup S) := by
  have himage : Set.image (fun y => (x, y)) S = ({x} : Set D) ×ˢ S := by
    ext ⟨a, b⟩
    simp only [Set.mem_image, Set.mem_prod, Set.mem_singleton_iff]
    constructor
    · rintro ⟨y, hy, h⟩
      obtain ⟨ha, hb⟩ := Prod.ext_iff.mp h
      subst hb
      exact ⟨ha.symm, hy⟩
    · rintro ⟨ha, hb⟩
      refine ⟨b, hb, Prod.ext_iff.mpr ⟨ha.symm, rfl⟩⟩
```

```
  have hx : sSup ({x} : Set D) = x := by
    apply le_antisymm
    · exact sSup_le fun z hz => by rw [Set.mem_singleton_iff] at hz; rw [hz]
    · exact le_sSup (Set.mem_singleton x)
  rw [himage, sSup_prod (hs := ⟨x, Set.mem_singleton x⟩) (ht := hS), hx]

end DomainTheory.ContinuousLattice
```

3

— Q2X, Q′xPresentation

sSup_image_prod_mk_left.lean

```
module
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem sSup_image_prod_mk_left (x : D) (S : Set D') (hS : S.Nonempty) :
    sSup (Set.image (fun y => (x, y)) S) = (x, sSup S) := by
  have himage : Set.image (fun y => (x, y)) S = ({x} : Set D) ×ˢ S := by
    ext ⟨a, b⟩
    simp only [Set.mem_image, Set.mem_prod, Set.mem_singleton_iff]
    constructor
    · rintro ⟨y, hy, h⟩
      obtain ⟨ha, hb⟩ := Prod.ext_iff.mp h
      subst hb
      exact ⟨ha.symm, hy⟩
    · rintro ⟨ha, hb⟩
      refine ⟨b, hb, Prod.ext_iff.mpr ⟨ha.symm, rfl⟩⟩
```

```
  have hx : sSup ({x} : Set D) = x := by
    apply le_antisymm
    · exact sSup_le fun z hz => by rw [Set.mem_singleton_iff] at hz; rw [hz]
    · exact le_sSup (Set.mem_singleton x)
  rw [himage, sSup_prod (hs := ⟨x, Set.mem_singleton x⟩) (ht := hS), hx]

end DomainTheory.ContinuousLattice
```

4

Assembles those tools into the Exercise 8.15 development (order iso, presentations, U-case).

NeighbourhoodSystems/RetractsOfTheUniversalDomain/finGen.lean

The tip then assembles those tools into Scott's statement.

— `exercise_8_15`

sSup_image_prod_mk_left.lean

```
module
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem sSup_image_prod_mk_left (x : D) (S : Set D') (hS : S.Nonempty) :
    sSup (Set.image (fun y => (x, y)) S) = (x, sSup S) := by
  have himage : Set.image (fun y => (x, y)) S = ({x} : Set D) ×ˢ S := by
    ext ⟨a, b⟩
    simp only [Set.mem_image, Set.mem_prod, Set.mem_singleton_iff]
    constructor
    · rintro ⟨y, hy, h⟩
      obtain ⟨ha, hb⟩ := Prod.ext_iff.mp h
```

```
      subst hb
      exact ⟨ha.symm, hy⟩
    · rintro ⟨ha, hb⟩
      refine ⟨b, hb, Prod.ext_iff.mpr ⟨ha.symm, rfl⟩⟩
  have hx : sSup ({x} : Set D) = x := by
    apply le_antisymm
    · exact sSup_le fun z hz => by rw [Set.mem_singleton_iff] at hz; rw [hz]
    · exact le_sSup (Set.mem_singleton x)
  rw [himage, sSup_prod (hs := ⟨x, Set.mem_singleton x⟩) (ht := hS), hx]

end DomainTheory.ContinuousLattice
```

5

3.9.18 Exercise 8.16 — order of projections

For finitary projections $a : E \to E$, write

$$D_a = \{X \in E \mid X \sqsubseteq aX\}$$

(cf. 8.5.). Show that for any two such projections $a, b : E \to E$ we have

$$a \sqsubseteq b \quad \text{iff} \quad D_a \lhd D_b.$$

(This fills in the gap at the end of the proof of 8.6.) Also finish off the proof of 8.8 by showing that if E is effectively given and $a : E \to E$ is computable, then D_a is effectively given.

Tip summary The tip packages the order correspondence for finitary projections with the effectivity of their fixed neighbourhoods.

Development Matches the order of finitary projections with subdomain order and records effectivity of `D_a`.

`sSup_image_prod_mk_left.lean`

```
module
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section
```

```
namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem sSup_image_prod_mk_left (x : D) (S : Set D') (hS : S.Nonempty) :
    sSup (Set.image (fun y => (x, y)) S) = (x, sSup S) := by
  have himage : Set.image (fun y => (x, y)) S = ({x} : Set D) ×ˢ S := by
    ext ⟨a, b⟩
    simp only [Set.mem_image, Set.mem_prod, Set.mem_singleton_iff]
    constructor
    · rintro ⟨y, hy, h⟩
      obtain ⟨ha, hb⟩ := Prod.ext_iff.mp h
      subst hb
      exact ⟨ha.symm, hy⟩
    · rintro ⟨ha, hb⟩
      refine ⟨b, hb, Prod.ext_iff.mpr ⟨ha.symm, rfl⟩⟩
  have hx : sSup ({x} : Set D) = x := by
    apply le_antisymm
    · exact sSup_le fun z hz => by rw [Set.mem_singleton_iff] at hz; rw [hz]
    · exact le_sSup (Set.mem_singleton x)
  rw [himage, sSup_prod (hs := ⟨x, Set.mem_singleton x⟩) (ht := hS), hx]

end DomainTheory.ContinuousLattice
```

6

The tip then assembles those tools into Scott's statement.

— exercise_8_16

sSup_image_prod_mk_left.lean

```
module
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section
```

```
namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem sSup_image_prod_mk_left (x : D) (S : Set D') (hS : S.Nonempty) :
    sSup (Set.image (fun y => (x, y)) S) = (x, sSup S) := by
  have himage : Set.image (fun y => (x, y)) S = ({x} : Set D) ×ˢ S := by
    ext ⟨a, b⟩
    simp only [Set.mem_image, Set.mem_prod, Set.mem_singleton_iff]
    constructor
    · rintro ⟨y, hy, h⟩
      obtain ⟨ha, hb⟩ := Prod.ext_iff.mp h
      subst hb
      exact ⟨ha.symm, hy⟩
    · rintro ⟨ha, hb⟩
      refine ⟨b, hb, Prod.ext_iff.mpr ⟨ha.symm, rfl⟩⟩
  have hx : sSup ({x} : Set D) = x := by
    apply le_antisymm
    · exact sSup_le fun z hz => by rw [Set.mem_singleton_iff] at hz; rw [hz]
    · exact le_sSup (Set.mem_singleton x)
  rw [himage, sSup_prod (hs := ⟨x, Set.mem_singleton x⟩) (ht := hS), hx]

end DomainTheory.ContinuousLattice
```

7

3.9.19 Exercise 8.17 — explicit embeddings into U

sSup_image_prod_mk_left.lean

```
module
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints
```

```
@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem sSup_image_prod_mk_left (x : D) (S : Set D') (hS : S.Nonempty) :
    sSup (Set.image (fun y => (x, y)) S) = (x, sSup S) := by
  have himage : Set.image (fun y => (x, y)) S = ({x} : Set D) ×ˢ S := by
    ext ⟨a, b⟩
    simp only [Set.mem_image, Set.mem_prod, Set.mem_singleton_iff]
    constructor
    · rintro ⟨y, hy, h⟩
      obtain ⟨ha, hb⟩ := Prod.ext_iff.mp h
      subst hb
      exact ⟨ha.symm, hy⟩
    · rintro ⟨ha, hb⟩
      refine ⟨b, hb, Prod.ext_iff.mpr ⟨ha.symm, rfl⟩⟩
  have hx : sSup ({x} : Set D) = x := by
    apply le_antisymm
    · exact sSup_le fun z hz => by rw [Set.mem_singleton_iff] at hz; rw [hz]
    · exact le_sSup (Set.mem_singleton x)
  rw [himage, sSup_prod (hs := ⟨x, Set.mem_singleton x⟩) (ht := hS), hx]

end DomainTheory.ContinuousLattice
```

8

EXERCISE 8.17. Find explicitly (if possible) the projection pairs for $\mathcal{U}+\mathcal{U}$, $\mathcal{U}\times\mathcal{U}$, and $\mathcal{U}\to\mathcal{U}$ needed for 8.9. Are any of these domains isomorphic with $\mathcal{U}$? (The author does not know a really good construction for $\mathcal{U}\to\mathcal{U}$.) Find a universal domain $V \neq \mathcal{U}$.

Tip summary The tip asserts universality of `V` together with the product/sum/function-space embeddings into `U`.

Development Proves `V` is universal and records the standard embeddings into `U`.

— `ProjPair` and composition/iso/subsystem constructors

— universality and the concrete `U`/`V` embeddings

`sSup_image_prod_mk_left.lean`

```
module
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
```

```
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem sSup_image_prod_mk_left (x : D) (S : Set D') (hS : S.Nonempty) :
    sSup (Set.image (fun y => (x, y)) S) = (x, sSup S) := by
  have himage : Set.image (fun y => (x, y)) S = ({x} : Set D) ×ˢ S := by
    ext ⟨a, b⟩
    simp only [Set.mem_image, Set.mem_prod, Set.mem_singleton_iff]
    constructor
    · rintro ⟨y, hy, h⟩
      obtain ⟨ha, hb⟩ := Prod.ext_iff.mp h
      subst hb
      exact ⟨ha.symm, hy⟩
    · rintro ⟨ha, hb⟩
      refine ⟨b, hb, Prod.ext_iff.mpr ⟨ha.symm, rfl⟩⟩
  have hx : sSup ({x} : Set D) = x := by
    apply le_antisymm
    · exact sSup_le fun z hz => by rw [Set.mem_singleton_iff] at hz; rw [hz]
    · exact le_sSup (Set.mem_singleton x)
  rw [himage, sSup_prod (hs := ⟨x, Set.mem_singleton x⟩) (ht := hS), hx]

end DomainTheory.ContinuousLattice
```

9

The tip then assembles those tools into Scott's statement.

— exercise_8_17

sSup_image_prod_mk_right.lean

```
module
public import Mathlib.Order.BourbakiWitt
```

```
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem sSup_image_prod_mk_right (y : D') (S : Set D) (hS : S.Nonempty) :
    sSup (Set.image (fun x => (x, y)) S) = (sSup S, y) := by
  have himage : Set.image (fun x => (x, y)) S = S ×ˢ ({y} : Set D') := by
    ext ⟨a, b⟩
    simp only [Set.mem_image, Set.mem_prod, Set.mem_singleton_iff]
    constructor
    · rintro ⟨x, hx, h⟩
      obtain ⟨ha, hb⟩ := Prod.ext_iff.mp h
      subst ha
      exact ⟨hx, hb.symm⟩
    · rintro ⟨ha, hb⟩
      exact ⟨a, ha, Prod.ext_iff.mpr ⟨rfl, hb.symm⟩⟩
  have hy : sSup ({y} : Set D') = y := by
    apply le_antisymm
    · exact sSup_le fun z hz => by rw [Set.mem_singleton_iff] at hz; rw [hz]
    · exact le_sSup (Set.mem_singleton y)
  rw [himage, sSup_prod (hs := hS) (ht := ⟨y, Set.mem_singleton y⟩), hy]

end DomainTheory.ContinuousLattice
```

0

3.9.20 Exercise 8.18 — missing proofs of 8.10

Many of the cases of 8.10 were left unproved. Please establish these assertions explicitly.

Tip summary The tip asserts that the sum/product/arrow combinators preserve projections and finitary projections.

Development Shows sum/product/arrow combinators preserve (finitary) projections.

sSup_image_prod_mk_right.lean

```
module
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem sSup_image_prod_mk_right (y : D') (S : Set D) (hS : S.Nonempty) :
    sSup (Set.image (fun x => (x, y)) S) = (sSup S, y) := by
  have himage : Set.image (fun x => (x, y)) S = S ×ˢ ({y} : Set D') := by
    ext ⟨a, b⟩
    simp only [Set.mem_image, Set.mem_prod, Set.mem_singleton_iff]
    constructor
    · rintro ⟨x, hx, h⟩
      obtain ⟨ha, hb⟩ := Prod.ext_iff.mp h
      subst ha
      exact ⟨hx, hb.symm⟩
    · rintro ⟨ha, hb⟩
      exact ⟨a, ha, Prod.ext_iff.mpr ⟨rfl, hb.symm⟩⟩
  have hy : sSup ({y} : Set D') = y := by
    apply le_antisymm
    · exact sSup_le fun z hz => by rw [Set.mem_singleton_iff] at hz; rw [hz]
    · exact le_sSup (Set.mem_singleton y)
  rw [himage, sSup_prod (hs := hS) (ht := ⟨y, Set.mem_singleton y⟩), hy]
```

```
end DomainTheory.ContinuousLattice
```

1

The tip then assembles those tools into Scott's statement.

— exercise_8_18

sSup_image_prod_mk_right.lean

```
module
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem sSup_image_prod_mk_right (y : D') (S : Set D) (hS : S.Nonempty) :
    sSup (Set.image (fun x => (x, y)) S) = (sSup S, y) := by
  have himage : Set.image (fun x => (x, y)) S = S ×ˢ ({y} : Set D') := by
    ext ⟨a, b⟩
    simp only [Set.mem_image, Set.mem_prod, Set.mem_singleton_iff]
    constructor
    · rintro ⟨x, hx, h⟩
      obtain ⟨ha, hb⟩ := Prod.ext_iff.mp h
      subst ha
      exact ⟨hx, hb.symm⟩
    · rintro ⟨ha, hb⟩
      exact ⟨a, ha, Prod.ext_iff.mpr ⟨rfl, hb.symm⟩⟩
  have hy : sSup ({y} : Set D') = y := by
    apply le_antisymm
    · exact sSup_le fun z hz => by rw [Set.mem_singleton_iff] at hz; rw [hz]
    · exact le_sSup (Set.mem_singleton y)
  rw [himage, sSup_prod (hs := hS) (ht := ⟨y, Set.mem_singleton y⟩), hy]
```

```
end DomainTheory.ContinuousLattice
```

2

3.9.21 Exercise 8.19 — closure under sum and product

Suppose we know both

$$T \quad \text{and} \quad E \to E \trianglelefteq E\,.$$

Does it follow that $E + E$ and $E \times E \trianglelefteq E$?

Tip summary The tip asserts closure of $\trianglelefteq$ under sum and product under Scott's hypotheses.

Development Closes $\trianglelefteq$ under sum and product given a terminal object and a function-space retract.

sSup_image_prod_mk_right.lean

```
module
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem sSup_image_prod_mk_right (y : D') (S : Set D) (hS : S.Nonempty) :
    sSup (Set.image (fun x => (x, y)) S) = (sSup S, y) := by
  have himage : Set.image (fun x => (x, y)) S = S ×ˢ ({y} : Set D') := by
    ext ⟨a, b⟩
```

```
    simp only [Set.mem_image, Set.mem_prod, Set.mem_singleton_iff]
    constructor
    · rintro ⟨x, hx, h⟩
      obtain ⟨ha, hb⟩ := Prod.ext_iff.mp h
      subst ha
      exact ⟨hx, hb.symm⟩
    · rintro ⟨ha, hb⟩
      exact ⟨a, ha, Prod.ext_iff.mpr ⟨rfl, hb.symm⟩⟩
  have hy : sSup ({y} : Set D') = y := by
    apply le_antisymm
    · exact sSup_le fun z hz => by rw [Set.mem_singleton_iff] at hz; rw [hz]
    · exact le_sSup (Set.mem_singleton y)
  rw [himage, sSup_prod (hs := hS) (ht := ⟨y, Set.mem_singleton y⟩), hy]

end DomainTheory.ContinuousLattice
```

3

The tip then assembles those tools into Scott's statement.

— exercise_8_19

sSup_image_prod_mk_right.lean

```
module
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem sSup_image_prod_mk_right (y : D') (S : Set D) (hS : S.Nonempty) :
    sSup (Set.image (fun x => (x, y)) S) = (sSup S, y) := by
  have himage : Set.image (fun x => (x, y)) S = S ×ˢ ({y} : Set D') := by
```

```
    ext ⟨a, b⟩
    simp only [Set.mem_image, Set.mem_prod, Set.mem_singleton_iff]
    constructor
    · rintro ⟨x, hx, h⟩
      obtain ⟨ha, hb⟩ := Prod.ext_iff.mp h
      subst ha
      exact ⟨hx, hb.symm⟩
    · rintro ⟨ha, hb⟩
      exact ⟨a, ha, Prod.ext_iff.mpr ⟨rfl, hb.symm⟩⟩
  have hy : sSup ({y} : Set D') = y := by
    apply le_antisymm
    · exact sSup_le fun z hz => by rw [Set.mem_singleton_iff] at hz; rw [hz]
    · exact le_sSup (Set.mem_singleton y)
  rw [himage, sSup_prod (hs := hS) (ht := ⟨y, Set.mem_singleton y⟩), hy]

end DomainTheory.ContinuousLattice
```

4

3.9.22 Exercise 8.20 — self-embeddings of a domain

For any system we know $D \trianglelefteq D + D$, but what about

$$D \trianglelefteq D \times D \quad \text{and} \quad D \trianglelefteq D \to D\,?$$

Would these projections be computable if D is effectively given? Are there more than one projection pair in each case?

Tip summary The tip asserts $D \trianglelefteq D \times D$ and $D \trianglelefteq D \to D$ for every domain D.

Development Proves the unconditional self-embeddings $D \trianglelefteq D \times D$ and $D \trianglelefteq D \to D$.

sSup_image_prod_mk_right.lean

```
module
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice
```

```
open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem sSup_image_prod_mk_right (y : D') (S : Set D) (hS : S.Nonempty) :
    sSup (Set.image (fun x => (x, y)) S) = (sSup S, y) := by
  have himage : Set.image (fun x => (x, y)) S = S ×ˢ ({y} : Set D') := by
    ext ⟨a, b⟩
    simp only [Set.mem_image, Set.mem_prod, Set.mem_singleton_iff]
    constructor
    · rintro ⟨x, hx, h⟩
      obtain ⟨ha, hb⟩ := Prod.ext_iff.mp h
      subst ha
      exact ⟨hx, hb.symm⟩
    · rintro ⟨ha, hb⟩
      exact ⟨a, ha, Prod.ext_iff.mpr ⟨rfl, hb.symm⟩⟩
  have hy : sSup ({y} : Set D') = y := by
    apply le_antisymm
    · exact sSup_le fun z hz => by rw [Set.mem_singleton_iff] at hz; rw [hz]
    · exact le_sSup (Set.mem_singleton y)
  rw [himage, sSup_prod (hs := hS) (ht := ⟨y, Set.mem_singleton y⟩), hy]

end DomainTheory.ContinuousLattice
```

5

The tip then assembles those tools into Scott's statement.

— exercise_8_20

sSup_image_prod_mk_right.lean

```
module
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice
```

```
open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem sSup_image_prod_mk_right (y : D') (S : Set D) (hS : S.Nonempty) :
    sSup (Set.image (fun x => (x, y)) S) = (sSup S, y) := by
  have himage : Set.image (fun x => (x, y)) S = S ×ˢ ({y} : Set D') := by
    ext ⟨a, b⟩
    simp only [Set.mem_image, Set.mem_prod, Set.mem_singleton_iff]
    constructor
    · rintro ⟨x, hx, h⟩
      obtain ⟨ha, hb⟩ := Prod.ext_iff.mp h
      subst ha
      exact ⟨hx, hb.symm⟩
    · rintro ⟨ha, hb⟩
      exact ⟨a, ha, Prod.ext_iff.mpr ⟨rfl, hb.symm⟩⟩
  have hy : sSup ({y} : Set D') = y := by
    apply le_antisymm
    · exact sSup_le fun z hz => by rw [Set.mem_singleton_iff] at hz; rw [hz]
    · exact le_sSup (Set.mem_singleton y)
  rw [himage, sSup_prod (hs := hS) (ht := ⟨y, Set.mem_singleton y⟩), hy]

end DomainTheory.ContinuousLattice
```

6

3.9.23 Exercise 8.21 — lists of a projection

Using the fixed-point construction, show that there is a continuous and computable operator $\lambda a.\, a^{\S}$, such that if a is a finitary projection of U, then

$$D_{a^{\S}} \cong (D_a)^{\S} .$$

Tip summary The tip asserts that the sharp of a finitary projection is again a finitary projection.

Development Develops the sharp construction and shows it yields a finitary projection.

sSup_image_prod_mk_right.lean

```
module
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
```

```
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem sSup_image_prod_mk_right (y : D') (S : Set D) (hS : S.Nonempty) :
    sSup (Set.image (fun x => (x, y)) S) = (sSup S, y) := by
  have himage : Set.image (fun x => (x, y)) S = S ×ˢ ({y} : Set D') := by
    ext ⟨a, b⟩
    simp only [Set.mem_image, Set.mem_prod, Set.mem_singleton_iff]
    constructor
    · rintro ⟨x, hx, h⟩
      obtain ⟨ha, hb⟩ := Prod.ext_iff.mp h
      subst ha
      exact ⟨hx, hb.symm⟩
    · rintro ⟨ha, hb⟩
      exact ⟨a, ha, Prod.ext_iff.mpr ⟨rfl, hb.symm⟩⟩
  have hy : sSup ({y} : Set D') = y := by
    apply le_antisymm
    · exact sSup_le fun z hz => by rw [Set.mem_singleton_iff] at hz; rw [hz]
    · exact le_sSup (Set.mem_singleton y)
  rw [himage, sSup_prod (hs := hS) (ht := ⟨y, Set.mem_singleton y⟩), hy]

end DomainTheory.ContinuousLattice
```

7

— aSharp projection/computability companions and Ex 8.25 closing

sSup_image_prod_mk_right.lean

```
module
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
```

```
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem sSup_image_prod_mk_right (y : D') (S : Set D) (hS : S.Nonempty) :
    sSup (Set.image (fun x => (x, y)) S) = (sSup S, y) := by
  have himage : Set.image (fun x => (x, y)) S = S ×ˢ ({y} : Set D') := by
    ext ⟨a, b⟩
    simp only [Set.mem_image, Set.mem_prod, Set.mem_singleton_iff]
    constructor
    · rintro ⟨x, hx, h⟩
      obtain ⟨ha, hb⟩ := Prod.ext_iff.mp h
      subst ha
      exact ⟨hx, hb.symm⟩
    · rintro ⟨ha, hb⟩
      exact ⟨a, ha, Prod.ext_iff.mpr ⟨rfl, hb.symm⟩⟩
  have hy : sSup ({y} : Set D') = y := by
    apply le_antisymm
    · exact sSup_le fun z hz => by rw [Set.mem_singleton_iff] at hz; rw [hz]
    · exact le_sSup (Set.mem_singleton y)
  rw [himage, sSup_prod (hs := hS) (ht := ⟨y, Set.mem_singleton y⟩), hy]

end DomainTheory.ContinuousLattice
```

8

The tip then assembles those tools into Scott's statement.

— exercise_8_21

sSup_image_prod_mk_right.lean

```
module
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
```

```
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem sSup_image_prod_mk_right (y : D') (S : Set D) (hS : S.Nonempty) :
    sSup (Set.image (fun x => (x, y)) S) = (sSup S, y) := by
  have himage : Set.image (fun x => (x, y)) S = S ×ˢ ({y} : Set D') := by
    ext ⟨a, b⟩
    simp only [Set.mem_image, Set.mem_prod, Set.mem_singleton_iff]
    constructor
    · rintro ⟨x, hx, h⟩
      obtain ⟨ha, hb⟩ := Prod.ext_iff.mp h
      subst ha
      exact ⟨hx, hb.symm⟩
    · rintro ⟨ha, hb⟩
      exact ⟨a, ha, Prod.ext_iff.mpr ⟨rfl, hb.symm⟩⟩
  have hy : sSup ({y} : Set D') = y := by
    apply le_antisymm
    · exact sSup_le fun z hz => by rw [Set.mem_singleton_iff] at hz; rw [hz]
    · exact le_sSup (Set.mem_singleton y)
  rw [himage, sSup_prod (hs := hS) (ht := ⟨y, Set.mem_singleton y⟩), hy]

end DomainTheory.ContinuousLattice
```

9

3.9.24 Exercise 8.22 — comparing B and C

Which of the two relations hold:

$$B \trianglelefteq C \quad \text{or} \quad C \trianglelefteq B\,?$$

Or do they both hold? In general if we use domain equations

$$D = T(D) + S(D)\,, \quad \text{and}$$

$$E = T(E)\,,$$

will $E \trianglelefteq D$ hold? What projections do you see in the examples in 6.2?

Tip summary The tip asserts the concrete embedding relation between Scott's examples `C` and `B`.

Development Compares Scott's concrete examples `B` and `C`.

sSup_image_retr_wayBelow.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeRetraction
public import DomainTheory.ContinuousLattice.WayBelow.directedOn_wayBelow
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace IsContinuousLatticeRetraction

/-- For `d` in the retract, the ambient way-below set of `i(d)` pushed back by `j` is a
     directed
family whose supremum (computed in `D`) is `d`. This is the identity `j(⊔S′) = ⊔S`
    applied to
`S = {j(x') : x' ≪ i(d)}`, combined with continuity of `D'`. -/
theorem sSup_image_retr_wayBelow (R : IsContinuousLatticeRetraction D D')
    (hD' : ContinuousLattice D') (d : D) :
    sSup (Set.image (R.retr : D' → D) {x' | x' ≪ R.incl d}) = d := by
  have hne : ({x' : D' | x' ≪ R.incl d}).Nonempty := ⟨⊥, bot_wayBelow _⟩
  have hdir : DirectedOn (· ≤ ·) {x' : D' | x' ≪ R.incl d} := directedOn_wayBelow _
  rw [← R.retr.preservesDirectedSup_coe _ hne hdir, hD'.sSup_wayBelow (R.incl d), R.
    retr_incl d]

end IsContinuousLatticeRetraction
end DomainTheory.ContinuousLattice
```

sSup_image_retr_wayBelow.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeRetraction
public import DomainTheory.ContinuousLattice.WayBelow.directedOn_wayBelow
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace IsContinuousLatticeRetraction

/-- For `d` in the retract, the ambient way-below set of `i(d)` pushed back by `j` is a
     directed
family whose supremum (computed in `D`) is `d`. This is the identity `j(⊔S') = ⊔S`
    applied to
`S = {j(x') : x' ≪ i(d)}`, combined with continuity of `D'`. -/
theorem sSup_image_retr_wayBelow (R : IsContinuousLatticeRetraction D D')
    (hD' : ContinuousLattice D') (d : D) :
    sSup (Set.image (R.retr : D' → D) {x' | x' ≪ R.incl d}) = d := by
  have hne : ({x' : D' | x' ≪ R.incl d}).Nonempty := ⟨⊥, bot_wayBelow _⟩
  have hdir : DirectedOn (· ≤ ·) {x' : D' | x' ≪ R.incl d} := directedOn_wayBelow _
  rw [← R.retr.preservesDirectedSup_coe _ hne hdir, hD'.sSup_wayBelow (R.incl d), R.
    retr_incl d]

end IsContinuousLatticeRetraction
end DomainTheory.ContinuousLattice
```

1

The tip then assembles those tools into Scott's statement.

— exercise_8_22

3.9.25 Exercise 8.23 — initial algebra inside U

Suppose a construct T on domains can be made into a computable operator $t : (U \to U) \to (U \to U)$ so that whenever $a : U \to U$ is a finitary projection, then so is $t(a)$ and

$$D_{t(a)} \cong T(D_a) .$$

Does it follow that $\|t\| = \mathrm{fix}(t)$ is such that

$$D_{\|t\|} \cong T(D_{\|t\|})$$

really is the initial solution of the domain equation with respect to projections? Since t is computable, will this solution be effectively given?

Tip summary The tip asserts that `fixOp t` is a finitary projection whenever `t` preserves them.

Development Defines `fixOp` and proves it is finitary when the transformer preserves finitary projections.

sSup_image_retr_wayBelow.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeRetraction
public import DomainTheory.ContinuousLattice.WayBelow.directedOn_wayBelow
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace IsContinuousLatticeRetraction

/-- For `d` in the retract, the ambient way-below set of `i(d)` pushed back by `j` is a
     directed
family whose supremum (computed in `D`) is `d`. This is the identity `j(⊔S') = ⊔S`
    applied to
`S = {j(x') : x' ≪ i(d)}`, combined with continuity of `D'`. -/
theorem sSup_image_retr_wayBelow (R : IsContinuousLatticeRetraction D D')
    (hD' : ContinuousLattice D') (d : D) :
    sSup (Set.image (R.retr : D' → D) {x' | x' ≪ R.incl d}) = d := by
  have hne : ({x' : D' | x' ≪ R.incl d}).Nonempty := ⟨⊥, bot_wayBelow _⟩
  have hdir : DirectedOn (· ≤ ·) {x' : D' | x' ≪ R.incl d} := directedOn_wayBelow _
  rw [← R.retr.preservesDirectedSup_coe _ hne hdir, hD'.sSup_wayBelow (R.incl d), R.
    retr_incl d]

end IsContinuousLatticeRetraction
end DomainTheory.ContinuousLattice
```

2

The tip then assembles those tools into Scott's statement.

— exercise_8_23

sSup_image_retr_wayBelow.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeRetraction
public import DomainTheory.ContinuousLattice.WayBelow.directedOn_wayBelow
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace IsContinuousLatticeRetraction

/-- For `d` in the retract, the ambient way-below set of `i(d)` pushed back by `j` is a
     directed
family whose supremum (computed in `D`) is `d`. This is the identity `j(⨆S′) = ⨆S`
    applied to
`S = {j(x') : x' ≪ i(d)}`, combined with continuity of `D'`. -/
theorem sSup_image_retr_wayBelow (R : IsContinuousLatticeRetraction D D')
    (hD' : ContinuousLattice D') (d : D) :
    sSup (Set.image (R.retr : D' → D) {x' | x' ≪ R.incl d}) = d := by
  have hne : ({x' : D' | x' ≪ R.incl d}).Nonempty := ⟨⊥, bot_wayBelow _⟩
  have hdir : DirectedOn (· ≤ ·) {x' : D' | x' ≪ R.incl d} := directedOn_wayBelow _
  rw [← R.retr.preservesDirectedSup_coe _ hne hdir, hD'.sSup_wayBelow (R.incl d), R.
    retr_incl d]

end IsContinuousLatticeRetraction
end DomainTheory.ContinuousLattice
```

3

3.9.26 Exercise 8.24 — coupled domain equations

Suppose S and T are two (binary-argument) constructs on domains that can be made into computable operators on projections of the universal domain. Show that we can therefore find a pair of effectively presented domains such that

$$D \cong S(D, E) \quad \text{and} \quad E \cong T(D, E).$$

Tip summary The tip asserts that the coupled solutions `aStar`, `bStar` are finitary projections.

Development Solves the coupled domain equations with finitary projections `aStar`, `bStar`.

sSup_image_retr_wayBelow.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeRetraction
public import DomainTheory.ContinuousLattice.WayBelow.directedOn_wayBelow
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace IsContinuousLatticeRetraction

/-- For `d` in the retract, the ambient way-below set of `i(d)` pushed back by `j` is a
     directed
family whose supremum (computed in `D`) is `d`. This is the identity `j(⊔S') = ⊔S`
    applied to
`S = {j(x') : x' ≪ i(d)}`, combined with continuity of `D'`. -/
theorem sSup_image_retr_wayBelow (R : IsContinuousLatticeRetraction D D')
    (hD' : ContinuousLattice D') (d : D) :
    sSup (Set.image (R.retr : D' → D) {x' | x' ≪ R.incl d}) = d := by
  have hne : ({x' : D' | x' ≪ R.incl d}).Nonempty := ⟨⊥, bot_wayBelow _⟩
  have hdir : DirectedOn (· ≤ ·) {x' : D' | x' ≪ R.incl d} := directedOn_wayBelow _
  rw [← R.retr.preservesDirectedSup_coe _ hne hdir, hD'.sSup_wayBelow (R.incl d), R.
    retr_incl d]

end IsContinuousLatticeRetraction
end DomainTheory.ContinuousLattice
```

4

The tip then assembles those tools into Scott's statement.

— `exercise_8_24`

sSup_image_retr_wayBelow.lean

```
module
```

```
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeRetraction
public import DomainTheory.ContinuousLattice.WayBelow.directedOn_wayBelow
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace IsContinuousLatticeRetraction

/-- For `d` in the retract, the ambient way-below set of `i(d)` pushed back by `j` is a
     directed
family whose supremum (computed in `D`) is `d`. This is the identity `j(⊔S') = ⊔S`
    applied to
`S = {j(x') : x' ≪ i(d)}`, combined with continuity of `D'`. -/
theorem sSup_image_retr_wayBelow (R : IsContinuousLatticeRetraction D D')
    (hD' : ContinuousLattice D') (d : D) :
    sSup (Set.image (R.retr : D' → D) {x' | x' ≪ R.incl d}) = d := by
  have hne : ({x' : D' | x' ≪ R.incl d}).Nonempty := ⟨⊥, bot_wayBelow _⟩
  have hdir : DirectedOn (· ≤ ·) {x' : D' | x' ≪ R.incl d} := directedOn_wayBelow _
  rw [← R.retr.preservesDirectedSup_coe _ hne hdir, hD'.sSup_wayBelow (R.incl d), R.
    retr_incl d]

end IsContinuousLatticeRetraction
end DomainTheory.ContinuousLattice
```

5

3.9.27 Exercise 8.25 — reflexive function space

The problem is to find non-trivial solutions to the domain equation

$$(\spadesuit) \qquad D \cong D \to D.$$

Show that the "obvious" solution by retracts is of no use because

$$1 \to 1 = 1$$

for projections. Change the method as follows. Show first

$$U^\infty \times U^\infty \cong U^\infty.$$

Next solve

$$D \cong D \to U^{\infty}$$

and remark that $U \triangleleft D$; so D is universal and non-trivial. Finally prove (♠) for this D. (Hint: First show

$$D \times D \cong D,$$

and then show D satisfies (♠).) Is this D effectively given?

Tip summary The tip asserts existence of a non-trivial domain isomorphic to its function space and containing `U`.

Development Rational-interval coding used by effective presentations.

sSup_image_retr_wayBelow.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeRetraction
public import DomainTheory.ContinuousLattice.WayBelow.directedOn_wayBelow
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace IsContinuousLatticeRetraction

/-- For `d` in the retract, the ambient way-below set of `i(d)` pushed back by `j` is a
     directed
family whose supremum (computed in `D`) is `d`. This is the identity `j(⨆S') = ⨆S`
    applied to
`S = {j(x') : x' ≪ i(d)}`, combined with continuity of `D'`. -/
theorem sSup_image_retr_wayBelow (R : IsContinuousLatticeRetraction D D')
    (hD' : ContinuousLattice D') (d : D) :
    sSup (Set.image (R.retr : D' → D) {x' | x' ≪ R.incl d}) = d := by
  have hne : ({x' : D' | x' ≪ R.incl d}).Nonempty := ⟨⊥, bot_wayBelow _⟩
  have hdir : DirectedOn (· ≤ ·) {x' : D' | x' ≪ R.incl d} := directedOn_wayBelow _
  rw [← R.retr.preservesDirectedSup_coe _ hne hdir, hD'.sSup_wayBelow (R.incl d), R.
    retr_incl d]

end IsContinuousLatticeRetraction
end DomainTheory.ContinuousLattice
```

6

sSup_image_retr_wayBelow.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeRetraction
public import DomainTheory.ContinuousLattice.WayBelow.directedOn_wayBelow
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace IsContinuousLatticeRetraction

/-- For `d` in the retract, the ambient way-below set of `i(d)` pushed back by `j` is a
     directed
family whose supremum (computed in `D`) is `d`. This is the identity `j(⨆S') = ⨆S`
    applied to
`S = {j(x') : x' ≪ i(d)}`, combined with continuity of `D'`. -/
theorem sSup_image_retr_wayBelow (R : IsContinuousLatticeRetraction D D')
    (hD' : ContinuousLattice D') (d : D) :
    sSup (Set.image (R.retr : D' → D) {x' | x' ≪ R.incl d}) = d := by
  have hne : ({x' : D' | x' ≪ R.incl d}).Nonempty := ⟨⊥, bot_wayBelow _⟩
  have hdir : DirectedOn (· ≤ ·) {x' : D' | x' ≪ R.incl d} := directedOn_wayBelow _
  rw [← R.retr.preservesDirectedSup_coe _ hne hdir, hD'.sSup_wayBelow (R.incl d), R.
    retr_incl d]

end IsContinuousLatticeRetraction
end DomainTheory.ContinuousLattice
```

7

sSup_image_retr_wayBelow.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeRetraction
```

```
public import DomainTheory.ContinuousLattice.WayBelow.directedOn_wayBelow
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace IsContinuousLatticeRetraction

/-- For `d` in the retract, the ambient way-below set of `i(d)` pushed back by `j` is a
     directed
family whose supremum (computed in `D`) is `d`. This is the identity `j(⨆S') = ⨆S`
    applied to
`S = {j(x') : x' ≪ i(d)}`, combined with continuity of `D'`. -/
theorem sSup_image_retr_wayBelow (R : IsContinuousLatticeRetraction D D')
    (hD' : ContinuousLattice D') (d : D) :
    sSup (Set.image (R.retr : D' → D) {x' | x' ≪ R.incl d}) = d := by
  have hne : ({x' : D' | x' ≪ R.incl d}).Nonempty := ⟨⊥, bot_wayBelow _⟩
  have hdir : DirectedOn (· ≤ ·) {x' : D' | x' ≪ R.incl d} := directedOn_wayBelow _
  rw [← R.retr.preservesDirectedSup_coe _ hne hdir, hD'.sSup_wayBelow (R.incl d), R.
    retr_incl d]

end IsContinuousLatticeRetraction
end DomainTheory.ContinuousLattice
```

8

sSup_image_retr_wayBelow.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeRetraction
public import DomainTheory.ContinuousLattice.WayBelow.directedOn_wayBelow
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
```

```
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace IsContinuousLatticeRetraction

/-- For `d` in the retract, the ambient way-below set of `i(d)` pushed back by `j` is a
     directed
family whose supremum (computed in `D`) is `d`. This is the identity `j(⨆S') = ⨆S`
    applied to
`S = {j(x') : x' ≪ i(d)}`, combined with continuity of `D'`. -/
theorem sSup_image_retr_wayBelow (R : IsContinuousLatticeRetraction D D')
    (hD' : ContinuousLattice D') (d : D) :
    sSup (Set.image (R.retr : D' → D) {x' | x' ≪ R.incl d}) = d := by
  have hne : ({x' : D' | x' ≪ R.incl d}).Nonempty := ⟨⊥, bot_wayBelow _⟩
  have hdir : DirectedOn (· ≤ ·) {x' : D' | x' ≪ R.incl d} := directedOn_wayBelow _
  rw [← R.retr.preservesDirectedSup_coe _ hne hdir, hD'.sSup_wayBelow (R.incl d), R.
    retr_incl d]

end IsContinuousLatticeRetraction
end DomainTheory.ContinuousLattice
```

9

sSup_le_finsetSupOf.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.finsetSupOf
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

theorem sSup_le_finsetSupOf (S : Set D) : sSup (finsetSupOf S) ≤ sSup S :=
  sSup_le fun z hz => by
```

```
    obtain ⟨T, hTS, rfl⟩ := hz
    exact Finset.sup_le fun t ht => le_sSup (hTS t ht)

end IsContinuousLatticeProjection

end Proposition310

end DomainTheory.ContinuousLattice
```

0

sSup_le_finsetSupOf.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.finsetSupOf
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

theorem sSup_le_finsetSupOf (S : Set D) : sSup (finsetSupOf S) ≤ sSup S :=
  sSup_le fun z hz => by
    obtain ⟨T, hTS, rfl⟩ := hz
    exact Finset.sup_le fun t ht => le_sSup (hTS t ht)

end IsContinuousLatticeProjection

end Proposition310

end DomainTheory.ContinuousLattice
```

1

sSup_le_finsetSupOf.lean

```
module
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaces.finsetSupOf
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

theorem sSup_le_finsetSupOf (S : Set D) : sSup (finsetSupOf S) ≤ sSup S :=
  sSup_le fun z hz => by
    obtain ⟨T, hTS, rfl⟩ := hz
    exact Finset.sup_le fun t ht => le_sSup (hTS t ht)

end IsContinuousLatticeProjection

end Proposition310

end DomainTheory.ContinuousLattice
```

2

sSup_le_finsetSupOf.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.finsetSupOf
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]
```

```
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

theorem sSup_le_finsetSupOf (S : Set D) : sSup (finsetSupOf S) ≤ sSup S :=
  sSup_le fun z hz => by
    obtain ⟨T, hTS, rfl⟩ := hz
    exact Finset.sup_le fun t ht => le_sSup (hTS t ht)

end IsContinuousLatticeProjection

end Proposition310

end DomainTheory.ContinuousLattice
```

3

sSup_le_finsetSupOf.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.finsetSupOf
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

theorem sSup_le_finsetSupOf (S : Set D) : sSup (finsetSupOf S) ≤ sSup S :=
  sSup_le fun z hz => by
```

```
    obtain ⟨T, hTS, rfl⟩ := hz
    exact Finset.sup_le fun t ht => le_sSup (hTS t ht)

end IsContinuousLatticeProjection

end Proposition310

end DomainTheory.ContinuousLattice
```

4

sSup_le_finsetSupOf.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.finsetSupOf
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

theorem sSup_le_finsetSupOf (S : Set D) : sSup (finsetSupOf S) ≤ sSup S :=
  sSup_le fun z hz => by
    obtain ⟨T, hTS, rfl⟩ := hz
    exact Finset.sup_le fun t ht => le_sSup (hTS t ht)

end IsContinuousLatticeProjection

end Proposition310

end DomainTheory.ContinuousLattice
```

5

sSup_le_finsetSupOf.lean

```
module
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaces.finsetSupOf
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

theorem sSup_le_finsetSupOf (S : Set D) : sSup (finsetSupOf S) ≤ sSup S :=
  sSup_le fun z hz => by
    obtain ⟨T, hTS, rfl⟩ := hz
    exact Finset.sup_le fun t ht => le_sSup (hTS t ht)

end IsContinuousLatticeProjection

end Proposition310

end DomainTheory.ContinuousLattice
```

6

sSup_le_finsetSupOf.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.finsetSupOf
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]
```

```
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

theorem sSup_le_finsetSupOf (S : Set D) : sSup (finsetSupOf S) ≤ sSup S :=
  sSup_le fun z hz => by
    obtain ⟨T, hTS, rfl⟩ := hz
    exact Finset.sup_le fun t ht => le_sSup (hTS t ht)

end IsContinuousLatticeProjection

end Proposition310

end DomainTheory.ContinuousLattice
```

7

sSup_le_finsetSupOf.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.finsetSupOf
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

theorem sSup_le_finsetSupOf (S : Set D) : sSup (finsetSupOf S) ≤ sSup S :=
  sSup_le fun z hz => by
```

```
    obtain ⟨T, hTS, rfl⟩ := hz
    exact Finset.sup_le fun t ht => le_sSup (hTS t ht)

end IsContinuousLatticeProjection

end Proposition310

end DomainTheory.ContinuousLattice
```

8

sSup_le_finsetSupOf.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.finsetSupOf
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

theorem sSup_le_finsetSupOf (S : Set D) : sSup (finsetSupOf S) ≤ sSup S :=
  sSup_le fun z hz => by
    obtain ⟨T, hTS, rfl⟩ := hz
    exact Finset.sup_le fun t ht => le_sSup (hTS t ht)

end IsContinuousLatticeProjection

end Proposition310

end DomainTheory.ContinuousLattice
```

9

specializationLe_generateOpen.lean

```
module
```

```
public import DomainTheory.ContinuousLattice.Specialization.SpecializationLe
public import Mathlib.Topology.Order
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottFunctionEvalSubbasis
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem specializationLe_generateOpen (f g : C(X, Y)) (hfg : ∀ x, SpecializationLe (f x
    ) (g x))
    {V : Set (C(X, Y))} (hV : GenerateOpen (scottFunctionEvalSubbasis X Y) V) (hf : f
    ∈ V) :
    g ∈ V :=
  match hV with
  | .basic W hW => by
    obtain ⟨x, U, hU, rfl⟩ := hW
    exact hfg x U hU hf
  | .univ => hf
  | .inter s t hs ht => by
    rcases hf with ⟨hfs, hft⟩
    exact Set.mem_inter (specializationLe_generateOpen f g hfg hs hfs)
      (specializationLe_generateOpen f g hfg ht hft)
  | .sUnion S hS => by
    obtain ⟨W, hWS, hfW⟩ := Set.mem_sUnion.mp hf
    exact Set.mem_sUnion.mpr ⟨W, hWS, specializationLe_generateOpen f g hfg (hS W hWS)
    hfW⟩

end DomainTheory.ContinuousLattice
```

0

specializationLe_generateOpen.lean

```
module

public import DomainTheory.ContinuousLattice.Specialization.SpecializationLe
public import Mathlib.Topology.Order
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottFunctionEvalSubbasis
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace
```

```
variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem specializationLe_generateOpen (f g : C(X, Y)) (hfg : ∀ x, SpecializationLe (f x
    ) (g x))
    {V : Set (C(X, Y))} (hV : GenerateOpen (scottFunctionEvalSubbasis X Y) V) (hf : f
    ∈ V) :
    g ∈ V :=
  match hV with
  | .basic W hW => by
    obtain ⟨x, U, hU, rfl⟩ := hW
    exact hfg x U hU hf
  | .univ => hf
  | .inter s t hs ht => by
    rcases hf with ⟨hfs, hft⟩
    exact Set.mem_inter (specializationLe_generateOpen f g hfg hs hfs)
      (specializationLe_generateOpen f g hfg ht hft)
  | .sUnion S hS => by
    obtain ⟨W, hWS, hfW⟩ := Set.mem_sUnion.mp hf
    exact Set.mem_sUnion.mpr ⟨W, hWS, specializationLe_generateOpen f g hfg (hS W hWS)
    hfW⟩

end DomainTheory.ContinuousLattice
```

1

specializationLe_generateOpen.lean

```
module

public import DomainTheory.ContinuousLattice.Specialization.SpecializationLe
public import Mathlib.Topology.Order
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottFunctionEvalSubbasis
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem specializationLe_generateOpen (f g : C(X, Y)) (hfg : ∀ x, SpecializationLe (f x
    ) (g x))
    {V : Set (C(X, Y))} (hV : GenerateOpen (scottFunctionEvalSubbasis X Y) V) (hf : f
    ∈ V) :
    g ∈ V :=
  match hV with
```

```
  | .basic W hW => by
    obtain ⟨x, U, hU, rfl⟩ := hW
    exact hfg x U hU hf
  | .univ => hf
  | .inter s t hs ht => by
    rcases hf with ⟨hfs, hft⟩
    exact Set.mem_inter (specializationLe_generateOpen f g hfg hs hfs)
      (specializationLe_generateOpen f g hfg ht hft)
  | .sUnion S hS => by
    obtain ⟨W, hWS, hfW⟩ := Set.mem_sUnion.mp hf
    exact Set.mem_sUnion.mpr ⟨W, hWS, specializationLe_generateOpen f g hfg (hS W hWS)
    hfW⟩

end DomainTheory.ContinuousLattice
```

2

specializationLe_generateOpen.lean

```
module

public import DomainTheory.ContinuousLattice.Specialization.SpecializationLe
public import Mathlib.Topology.Order
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottFunctionEvalSubbasis
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem specializationLe_generateOpen (f g : C(X, Y)) (hfg : ∀ x, SpecializationLe (f x
    ) (g x))
    {V : Set (C(X, Y))} (hV : GenerateOpen (scottFunctionEvalSubbasis X Y) V) (hf : f
    ∈ V) :
    g ∈ V :=
  match hV with
  | .basic W hW => by
    obtain ⟨x, U, hU, rfl⟩ := hW
    exact hfg x U hU hf
  | .univ => hf
  | .inter s t hs ht => by
    rcases hf with ⟨hfs, hft⟩
    exact Set.mem_inter (specializationLe_generateOpen f g hfg hs hfs)
      (specializationLe_generateOpen f g hfg ht hft)
  | .sUnion S hS => by
    obtain ⟨W, hWS, hfW⟩ := Set.mem_sUnion.mp hf
    exact Set.mem_sUnion.mpr ⟨W, hWS, specializationLe_generateOpen f g hfg (hS W hWS)
    hfW⟩
```

```
end DomainTheory.ContinuousLattice
```

3

specializationLe_generateOpen.lean

```
module

public import DomainTheory.ContinuousLattice.Specialization.SpecializationLe
public import Mathlib.Topology.Order
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottFunctionEvalSubbasis
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem specializationLe_generateOpen (f g : C(X, Y)) (hfg : ∀ x, SpecializationLe (f x
    ) (g x))
    {V : Set (C(X, Y))} (hV : GenerateOpen (scottFunctionEvalSubbasis X Y) V) (hf : f
    ∈ V) :
    g ∈ V :=
  match hV with
  | .basic W hW => by
    obtain ⟨x, U, hU, rfl⟩ := hW
    exact hfg x U hU hf
  | .univ => hf
  | .inter s t hs ht => by
    rcases hf with ⟨hfs, hft⟩
    exact Set.mem_inter (specializationLe_generateOpen f g hfg hs hfs)
      (specializationLe_generateOpen f g hfg ht hft)
  | .sUnion S hS => by
    obtain ⟨W, hWS, hfW⟩ := Set.mem_sUnion.mp hf
    exact Set.mem_sUnion.mpr ⟨W, hWS, specializationLe_generateOpen f g hfg (hS W hWS)
    hfW⟩

end DomainTheory.ContinuousLattice
```

4

specializationLe_generateOpen.lean

```
module

public import DomainTheory.ContinuousLattice.Specialization.SpecializationLe
public import Mathlib.Topology.Order
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottFunctionEvalSubbasis
```

```
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem specializationLe_generateOpen (f g : C(X, Y)) (hfg : ∀ x, SpecializationLe (f x
    ) (g x))
    {V : Set (C(X, Y))} (hV : GenerateOpen (scottFunctionEvalSubbasis X Y) V) (hf : f
    ∈ V) :
    g ∈ V :=
  match hV with
  | .basic W hW => by
    obtain ⟨x, U, hU, rfl⟩ := hW
    exact hfg x U hU hf
  | .univ => hf
  | .inter s t hs ht => by
    rcases hf with ⟨hfs, hft⟩
    exact Set.mem_inter (specializationLe_generateOpen f g hfg hs hfs)
      (specializationLe_generateOpen f g hfg ht hft)
  | .sUnion S hS => by
    obtain ⟨W, hWS, hfW⟩ := Set.mem_sUnion.mp hf
    exact Set.mem_sUnion.mpr ⟨W, hWS, specializationLe_generateOpen f g hfg (hS W hWS)
    hfW⟩

end DomainTheory.ContinuousLattice
```

5

specializationLe_generateOpen.lean

```
module

public import DomainTheory.ContinuousLattice.Specialization.SpecializationLe
public import Mathlib.Topology.Order
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottFunctionEvalSubbasis
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
```

```
    CompleteLattice D₁']

theorem specializationLe_generateOpen (f g : C(X, Y)) (hfg : ∀ x, SpecializationLe (f x
    ) (g x))
    {V : Set (C(X, Y))} (hV : GenerateOpen (scottFunctionEvalSubbasis X Y) V) (hf : f
    ∈ V) :
    g ∈ V :=
  match hV with
  | .basic W hW => by
    obtain ⟨x, U, hU, rfl⟩ := hW
    exact hfg x U hU hf
  | .univ => hf
  | .inter s t hs ht => by
    rcases hf with ⟨hfs, hft⟩
    exact Set.mem_inter (specializationLe_generateOpen f g hfg hs hfs)
      (specializationLe_generateOpen f g hfg ht hft)
  | .sUnion S hS => by
    obtain ⟨W, hWS, hfW⟩ := Set.mem_sUnion.mp hf
    exact Set.mem_sUnion.mpr ⟨W, hWS, specializationLe_generateOpen f g hfg (hS W hWS)
    hfW⟩

end DomainTheory.ContinuousLattice
```

6

specializationLe_generateOpen.lean

```
module

public import DomainTheory.ContinuousLattice.Specialization.SpecializationLe
public import Mathlib.Topology.Order
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottFunctionEvalSubbasis
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem specializationLe_generateOpen (f g : C(X, Y)) (hfg : ∀ x, SpecializationLe (f x
    ) (g x))
    {V : Set (C(X, Y))} (hV : GenerateOpen (scottFunctionEvalSubbasis X Y) V) (hf : f
    ∈ V) :
    g ∈ V :=
  match hV with
  | .basic W hW => by
    obtain ⟨x, U, hU, rfl⟩ := hW
    exact hfg x U hU hf
  | .univ => hf
```

```
  | .inter s t hs ht => by
    rcases hf with ⟨hfs, hft⟩
    exact Set.mem_inter (specializationLe_generateOpen f g hfg hs hfs)
      (specializationLe_generateOpen f g hfg ht hft)
  | .sUnion S hS => by
    obtain ⟨W, hWS, hfW⟩ := Set.mem_sUnion.mp hf
    exact Set.mem_sUnion.mpr ⟨W, hWS, specializationLe_generateOpen f g hfg (hS W hWS)
    hfW⟩

end DomainTheory.ContinuousLattice
```

7

specializationLe_generateOpen.lean

```
module

public import DomainTheory.ContinuousLattice.Specialization.SpecializationLe
public import Mathlib.Topology.Order
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottFunctionEvalSubbasis
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem specializationLe_generateOpen (f g : C(X, Y)) (hfg : ∀ x, SpecializationLe (f x
    ) (g x))
    {V : Set (C(X, Y))} (hV : GenerateOpen (scottFunctionEvalSubbasis X Y) V) (hf : f
    ∈ V) :
    g ∈ V :=
  match hV with
  | .basic W hW => by
    obtain ⟨x, U, hU, rfl⟩ := hW
    exact hfg x U hU hf
  | .univ => hf
  | .inter s t hs ht => by
    rcases hf with ⟨hfs, hft⟩
    exact Set.mem_inter (specializationLe_generateOpen f g hfg hs hfs)
      (specializationLe_generateOpen f g hfg ht hft)
  | .sUnion S hS => by
    obtain ⟨W, hWS, hfW⟩ := Set.mem_sUnion.mp hf
    exact Set.mem_sUnion.mpr ⟨W, hWS, specializationLe_generateOpen f g hfg (hS W hWS)
    hfW⟩

end DomainTheory.ContinuousLattice
```

8

specializationLe_generateOpen.lean

```
module

public import DomainTheory.ContinuousLattice.Specialization.SpecializationLe
public import Mathlib.Topology.Order
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottFunctionEvalSubbasis
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem specializationLe_generateOpen (f g : C(X, Y)) (hfg : ∀ x, SpecializationLe (f x
    ) (g x))
    {V : Set (C(X, Y))} (hV : GenerateOpen (scottFunctionEvalSubbasis X Y) V) (hf : f
    ∈ V) :
    g ∈ V :=
  match hV with
  | .basic W hW => by
    obtain ⟨x, U, hU, rfl⟩ := hW
    exact hfg x U hU hf
  | .univ => hf
  | .inter s t hs ht => by
    rcases hf with ⟨hfs, hft⟩
    exact Set.mem_inter (specializationLe_generateOpen f g hfg hs hfs)
      (specializationLe_generateOpen f g hfg ht hft)
  | .sUnion S hS => by
    obtain ⟨W, hWS, hfW⟩ := Set.mem_sUnion.mp hf
    exact Set.mem_sUnion.mpr ⟨W, hWS, specializationLe_generateOpen f g hfg (hS W hWS)
    hfW⟩

end DomainTheory.ContinuousLattice
```

9

stepFun.lean

```
module
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
```

```
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- The (value of the) step function `e[e, e']` at `x`: `e'` if `e ≪ x`, else `⊥`. -/
noncomputable def stepFun (e : D) (e' : D') (x : D) : D' := ⋃ _ : e ≪ x, e'

end DomainTheory.ContinuousLattice
```

0

stepFun.lean

```
module
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]
```

```
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- The (value of the) step function `e[e, e']` at `x`: `e'` if `e ≪ x`, else `⊥`. -/
noncomputable def stepFun (e : D) (e' : D') (x : D) : D' := ⋃ _ : e ≪ x, e'

end DomainTheory.ContinuousLattice
```

1

stepFun.lean

```
module
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- The (value of the) step function `e[e, e']` at `x`: `e'` if `e ≪ x`, else `⊥`. -/
noncomputable def stepFun (e : D) (e' : D') (x : D) : D' := ⋃ _ : e ≪ x, e'

end DomainTheory.ContinuousLattice
```

2

stepFun.lean

```
module
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
```

```
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- The (value of the) step function `e[e, e']` at `x`: `e'` if `e ≪ x`, else `⊥`. -/
noncomputable def stepFun (e : D) (e' : D') (x : D) : D' := ⋃ _ : e ≪ x, e'

end DomainTheory.ContinuousLattice
```

3

stepFun.lean

```
module
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section
```

```
namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- The (value of the) step function `e[e, e']` at `x`: `e'` if `e ≪ x`, else `⊥`. -/
noncomputable def stepFun (e : D) (e' : D') (x : D) : D' := ⋃ _ : e ≪ x, e'

end DomainTheory.ContinuousLattice
```

4

stepFun.lean

```
module
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- The (value of the) step function `e[e, e']` at `x`: `e'` if `e ≪ x`, else `⊥`. -/
noncomputable def stepFun (e : D) (e' : D') (x : D) : D' := ⋃ _ : e ≪ x, e'
```

```
end DomainTheory.ContinuousLattice
```

5

stepFun.lean

```
module
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- The (value of the) step function `e[e, e']` at `x`: `e'` if `e ≪ x`, else `⊥`. -/
noncomputable def stepFun (e : D) (e' : D') (x : D) : D' := ⋃ _ : e ≪ x, e'

end DomainTheory.ContinuousLattice
```

6

stepFun.lean

```
module
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
```

```
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- The (value of the) step function `e[e, e']` at `x`: `e'` if `e ≪ x`, else `⊥`. -/
noncomputable def stepFun (e : D) (e' : D') (x : D) : D' := ⋃ _ : e ≪ x, e'

end DomainTheory.ContinuousLattice
```

7

stepFun.lean

```
module
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]
```

```
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- The (value of the) step function `e[e, e']` at `x`: `e'` if `e ≪ x`, else `⊥`. -/
noncomputable def stepFun (e : D) (e' : D') (x : D) : D' := ⋃ _ : e ≪ x, e'

end DomainTheory.ContinuousLattice
```

8

stepFun.lean

```
module
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- The (value of the) step function `e[e, e']` at `x`: `e'` if `e ≪ x`, else `⊥`. -/
noncomputable def stepFun (e : D) (e' : D') (x : D) : D' := ⋃ _ : e ≪ x, e'

end DomainTheory.ContinuousLattice
```

9

stepFun_of_not_wayBelow.lean

```
module
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem stepFun_of_not_wayBelow {e : D} {e' : D'} {x : D} (h : ¬ e ≪ x) :
    stepFun e e' x = ⊥ :=
  iSup_neg h

end DomainTheory.ContinuousLattice
```

0

stepFun_of_not_wayBelow.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem stepFun_of_not_wayBelow {e : D} {e' : D'} {x : D} (h : ¬ e ≪ x) :
    stepFun e e' x = ⊥ :=
  iSup_neg h

end DomainTheory.ContinuousLattice
```

1

stepFun_of_not_wayBelow.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem stepFun_of_not_wayBelow {e : D} {e' : D'} {x : D} (h : ¬ e ≪ x) :
    stepFun e e' x = ⊥ :=
  iSup_neg h

end DomainTheory.ContinuousLattice
```

2

stepFun_of_not_wayBelow.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']
```

```
theorem stepFun_of_not_wayBelow {e : D} {e' : D'} {x : D} (h : ¬ e ≪ x) :
    stepFun e e' x = ⊥ :=
  iSup_neg h

end DomainTheory.ContinuousLattice
```

3

stepFun_of_not_wayBelow.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem stepFun_of_not_wayBelow {e : D} {e' : D'} {x : D} (h : ¬ e ≪ x) :
    stepFun e e' x = ⊥ :=
  iSup_neg h

end DomainTheory.ContinuousLattice
```

4

stepFun_of_not_wayBelow.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]
```

```
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem stepFun_of_not_wayBelow {e : D} {e' : D'} {x : D} (h : ¬ e ≪ x) :
    stepFun e e' x = ⊥ :=
  iSup_neg h

end DomainTheory.ContinuousLattice
```

5

stepFun_of_not_wayBelow.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem stepFun_of_not_wayBelow {e : D} {e' : D'} {x : D} (h : ¬ e ≪ x) :
    stepFun e e' x = ⊥ :=
  iSup_neg h

end DomainTheory.ContinuousLattice
```

6

stepFun_of_not_wayBelow.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice
```

```
open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem stepFun_of_not_wayBelow {e : D} {e' : D'} {x : D} (h : ¬ e ≪ x) :
    stepFun e e' x = ⊥ :=
  iSup_neg h

end DomainTheory.ContinuousLattice
```

7

stepFun_of_not_wayBelow.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem stepFun_of_not_wayBelow {e : D} {e' : D'} {x : D} (h : ¬ e ≪ x) :
    stepFun e e' x = ⊥ :=
  iSup_neg h

end DomainTheory.ContinuousLattice
```

8

stepFun_of_not_wayBelow.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun
```

```
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem stepFun_of_not_wayBelow {e : D} {e' : D'} {x : D} (h : ¬ e ≪ x) :
    stepFun e e' x = ⊥ :=
  iSup_neg h

end DomainTheory.ContinuousLattice
```

9

stepFun_of_wayBelow.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem stepFun_of_wayBelow {e : D} {e' : D'} {x : D} (h : e ≪ x) : stepFun e e' x = e
    ' :=
  iSup_pos h

end DomainTheory.ContinuousLattice
```

0

stepFun_of_wayBelow.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem stepFun_of_wayBelow {e : D} {e' : D'} {x : D} (h : e ≪ x) : stepFun e e' x = e
    ' :=
  iSup_pos h

end DomainTheory.ContinuousLattice
```

1

stepFun_of_wayBelow.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem stepFun_of_wayBelow {e : D} {e' : D'} {x : D} (h : e ≪ x) : stepFun e e' x = e
    ' :=
```

```
  iSup_pos h

end DomainTheory.ContinuousLattice
```

2

stepFun_of_wayBelow.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem stepFun_of_wayBelow {e : D} {e' : D'} {x : D} (h : e ≪ x) : stepFun e e' x = e
    ' :=
  iSup_pos h

end DomainTheory.ContinuousLattice
```

3

stepFun_of_wayBelow.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
```

```
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem stepFun_of_wayBelow {e : D} {e' : D'} {x : D} (h : e ≪ x) : stepFun e e' x = e
    ' :=
  iSup_pos h

end DomainTheory.ContinuousLattice
```

4

stepFun_of_wayBelow.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem stepFun_of_wayBelow {e : D} {e' : D'} {x : D} (h : e ≪ x) : stepFun e e' x = e
    ' :=
  iSup_pos h

end DomainTheory.ContinuousLattice
```

5

stepFun_of_wayBelow.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace
```

```
variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem stepFun_of_wayBelow {e : D} {e' : D'} {x : D} (h : e ≪ x) : stepFun e e' x = e
    ' :=
  iSup_pos h

end DomainTheory.ContinuousLattice
```

6

stepFun_of_wayBelow.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem stepFun_of_wayBelow {e : D} {e' : D'} {x : D} (h : e ≪ x) : stepFun e e' x = e
    ' :=
  iSup_pos h

end DomainTheory.ContinuousLattice
```

7

stepFun_of_wayBelow.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic
```

```
@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem stepFun_of_wayBelow {e : D} {e' : D'} {x : D} (h : e ≪ x) : stepFun e e' x = e
    ' :=
  iSup_pos h

end DomainTheory.ContinuousLattice
```

8

stepFun_of_wayBelow.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem stepFun_of_wayBelow {e : D} {e' : D'} {x : D} (h : e ≪ x) : stepFun e e' x = e
    ' :=
  iSup_pos h

end DomainTheory.ContinuousLattice
```

9

stepFun_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun_of_not_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun_of_wayBelow
public import Mathlib.Order.BourbakiWitt
public import DomainTheory.ContinuousLattice.WayBelow.scottOpen_wayBelow
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- Step functions are Scott-continuous: `{x | e ≪ x}` is Scott-open, so the
    conditional commutes
with directed suprema. -/
theorem stepFun_preservesDirectedSup (e : D) (e' : D') :
    PreservesDirectedSup (stepFun e e') := by
  intro S hS hSdir
  show stepFun e e' (sSup S) = sSup (Set.image (stepFun e e') S)
  by_cases h : e ≪ sSup S
  · rw [stepFun_of_wayBelow h]
    obtain ⟨s, hsS, hes⟩ := (scottOpen_wayBelow e).2 hS hSdir h
    apply le_antisymm
    · refine le_sSup_of_le (Set.mem_image_of_mem _ hsS) ?_
      rw [stepFun_of_wayBelow hes]
    · refine sSup_le ?_
      rintro _ ⟨x, _, rfl⟩
      exact iSup_le fun _ => le_rfl
  · rw [stepFun_of_not_wayBelow h]
    refine (sSup_eq_bot.2 ?_).symm
    rintro _ ⟨x, hxS, rfl⟩
    refine stepFun_of_not_wayBelow (fun hex => ?_)
    exact h (hex.trans_le (le_sSup hxS))

end DomainTheory.ContinuousLattice
```

0

stepFun_preservesDirectedSup.lean

```
module
```

```
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun_of_not_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun_of_wayBelow
public import Mathlib.Order.BourbakiWitt
public import DomainTheory.ContinuousLattice.WayBelow.scottOpen_wayBelow
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- Step functions are Scott-continuous: `{x | e ≪ x}` is Scott-open, so the
    conditional commutes
with directed suprema. -/
theorem stepFun_preservesDirectedSup (e : D) (e' : D') :
    PreservesDirectedSup (stepFun e e') := by
  intro S hS hSdir
  show stepFun e e' (sSup S) = sSup (Set.image (stepFun e e') S)
  by_cases h : e ≪ sSup S
  · rw [stepFun_of_wayBelow h]
    obtain ⟨s, hsS, hes⟩ := (scottOpen_wayBelow e).2 hS hSdir h
    apply le_antisymm
    · refine le_sSup_of_le (Set.mem_image_of_mem _ hsS) ?_
      rw [stepFun_of_wayBelow hes]
    · refine sSup_le ?_
      rintro _ ⟨x, _, rfl⟩
      exact iSup_le fun _ => le_rfl
  · rw [stepFun_of_not_wayBelow h]
    refine (sSup_eq_bot.2 ?_).symm
    rintro _ ⟨x, hxS, rfl⟩
    refine stepFun_of_not_wayBelow (fun hex => ?_)
    exact h (hex.trans_le (le_sSup hxS))

end DomainTheory.ContinuousLattice
```

1

stepFun_preservesDirectedSup.lean

```
module
```

```
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun_of_not_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun_of_wayBelow
public import Mathlib.Order.BourbakiWitt
public import DomainTheory.ContinuousLattice.WayBelow.scottOpen_wayBelow
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- Step functions are Scott-continuous: `{x | e ≪ x}` is Scott-open, so the
    conditional commutes
with directed suprema. -/
theorem stepFun_preservesDirectedSup (e : D) (e' : D') :
    PreservesDirectedSup (stepFun e e') := by
  intro S hS hSdir
  show stepFun e e' (sSup S) = sSup (Set.image (stepFun e e') S)
  by_cases h : e ≪ sSup S
  · rw [stepFun_of_wayBelow h]
    obtain ⟨s, hsS, hes⟩ := (scottOpen_wayBelow e).2 hS hSdir h
    apply le_antisymm
    · refine le_sSup_of_le (Set.mem_image_of_mem _ hsS) ?_
      rw [stepFun_of_wayBelow hes]
    · refine sSup_le ?_
      rintro _ ⟨x, _, rfl⟩
      exact iSup_le fun _ => le_rfl
  · rw [stepFun_of_not_wayBelow h]
    refine (sSup_eq_bot.2 ?_).symm
    rintro _ ⟨x, hxS, rfl⟩
    refine stepFun_of_not_wayBelow (fun hex => ?_)
    exact h (hex.trans_le (le_sSup hxS))

end DomainTheory.ContinuousLattice
```

2

stepFun_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
```

```
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun_of_not_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun_of_wayBelow
public import Mathlib.Order.BourbakiWitt
public import DomainTheory.ContinuousLattice.WayBelow.scottOpen_wayBelow
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- Step functions are Scott-continuous: `{x | e ≪ x}` is Scott-open, so the
    conditional commutes
with directed suprema. -/
theorem stepFun_preservesDirectedSup (e : D) (e' : D') :
    PreservesDirectedSup (stepFun e e') := by
  intro S hS hSdir
  show stepFun e e' (sSup S) = sSup (Set.image (stepFun e e') S)
  by_cases h : e ≪ sSup S
  · rw [stepFun_of_wayBelow h]
    obtain ⟨s, hsS, hes⟩ := (scottOpen_wayBelow e).2 hS hSdir h
    apply le_antisymm
    · refine le_sSup_of_le (Set.mem_image_of_mem _ hsS) ?_
      rw [stepFun_of_wayBelow hes]
    · refine sSup_le ?_
      rintro _ ⟨x, _, rfl⟩
      exact iSup_le fun _ => le_rfl
  · rw [stepFun_of_not_wayBelow h]
    refine (sSup_eq_bot.2 ?_).symm
    rintro _ ⟨x, hxS, rfl⟩
    refine stepFun_of_not_wayBelow (fun hex => ?_)
    exact h (hex.trans_le (le_sSup hxS))

end DomainTheory.ContinuousLattice
```

3

stepFun_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun_of_not_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun_of_wayBelow
public import Mathlib.Order.BourbakiWitt
public import DomainTheory.ContinuousLattice.WayBelow.scottOpen_wayBelow
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- Step functions are Scott-continuous: `{x | e ≪ x}` is Scott-open, so the
    conditional commutes
with directed suprema. -/
theorem stepFun_preservesDirectedSup (e : D) (e' : D') :
    PreservesDirectedSup (stepFun e e') := by
  intro S hS hSdir
  show stepFun e e' (sSup S) = sSup (Set.image (stepFun e e') S)
  by_cases h : e ≪ sSup S
  · rw [stepFun_of_wayBelow h]
    obtain ⟨s, hsS, hes⟩ := (scottOpen_wayBelow e).2 hS hSdir h
    apply le_antisymm
    · refine le_sSup_of_le (Set.mem_image_of_mem _ hsS) ?_
      rw [stepFun_of_wayBelow hes]
    · refine sSup_le ?_
      rintro _ ⟨x, _, rfl⟩
      exact iSup_le fun _ => le_rfl
  · rw [stepFun_of_not_wayBelow h]
    refine (sSup_eq_bot.2 ?_).symm
    rintro _ ⟨x, hxS, rfl⟩
    refine stepFun_of_not_wayBelow (fun hex => ?_)
    exact h (hex.trans_le (le_sSup hxS))

end DomainTheory.ContinuousLattice
```

4

stepFun_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun_of_not_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun_of_wayBelow
public import Mathlib.Order.BourbakiWitt
public import DomainTheory.ContinuousLattice.WayBelow.scottOpen_wayBelow
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- Step functions are Scott-continuous: `{x | e ≪ x}` is Scott-open, so the
    conditional commutes
with directed suprema. -/
theorem stepFun_preservesDirectedSup (e : D) (e' : D') :
    PreservesDirectedSup (stepFun e e') := by
  intro S hS hSdir
  show stepFun e e' (sSup S) = sSup (Set.image (stepFun e e') S)
  by_cases h : e ≪ sSup S
  · rw [stepFun_of_wayBelow h]
    obtain ⟨s, hsS, hes⟩ := (scottOpen_wayBelow e).2 hS hSdir h
    apply le_antisymm
    · refine le_sSup_of_le (Set.mem_image_of_mem _ hsS) ?_
      rw [stepFun_of_wayBelow hes]
    · refine sSup_le ?_
      rintro _ ⟨x, _, rfl⟩
      exact iSup_le fun _ => le_rfl
  · rw [stepFun_of_not_wayBelow h]
    refine (sSup_eq_bot.2 ?_).symm
    rintro _ ⟨x, hxS, rfl⟩
    refine stepFun_of_not_wayBelow (fun hex => ?_)
    exact h (hex.trans_le (le_sSup hxS))

end DomainTheory.ContinuousLattice
```

5

stepFun_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun_of_not_wayBelow
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun_of_wayBelow
public import Mathlib.Order.BourbakiWitt
public import DomainTheory.ContinuousLattice.WayBelow.scottOpen_wayBelow
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- Step functions are Scott-continuous: `{x | e ≪ x}` is Scott-open, so the
    conditional commutes
with directed suprema. -/
theorem stepFun_preservesDirectedSup (e : D) (e' : D') :
    PreservesDirectedSup (stepFun e e') := by
  intro S hS hSdir
  show stepFun e e' (sSup S) = sSup (Set.image (stepFun e e') S)
  by_cases h : e ≪ sSup S
  · rw [stepFun_of_wayBelow h]
    obtain ⟨s, hsS, hes⟩ := (scottOpen_wayBelow e).2 hS hSdir h
    apply le_antisymm
    · refine le_sSup_of_le (Set.mem_image_of_mem _ hsS) ?_
      rw [stepFun_of_wayBelow hes]
    · refine sSup_le ?_
      rintro _ ⟨x, _, rfl⟩
      exact iSup_le fun _ => le_rfl
  · rw [stepFun_of_not_wayBelow h]
    refine (sSup_eq_bot.2 ?_).symm
    rintro _ ⟨x, hxS, rfl⟩
    refine stepFun_of_not_wayBelow (fun hex => ?_)
    exact h (hex.trans_le (le_sSup hxS))

end DomainTheory.ContinuousLattice
```

6

stepFun_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun_of_not_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun_of_wayBelow
```

```
public import Mathlib.Order.BourbakiWitt
public import DomainTheory.ContinuousLattice.WayBelow.scottOpen_wayBelow
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- Step functions are Scott-continuous: `{x | e ≪ x}` is Scott-open, so the
    conditional commutes
with directed suprema. -/
theorem stepFun_preservesDirectedSup (e : D) (e' : D') :
    PreservesDirectedSup (stepFun e e') := by
  intro S hS hSdir
  show stepFun e e' (sSup S) = sSup (Set.image (stepFun e e') S)
  by_cases h : e ≪ sSup S
  · rw [stepFun_of_wayBelow h]
    obtain ⟨s, hsS, hes⟩ := (scottOpen_wayBelow e).2 hS hSdir h
    apply le_antisymm
    · refine le_sSup_of_le (Set.mem_image_of_mem _ hsS) ?_
      rw [stepFun_of_wayBelow hes]
    · refine sSup_le ?_
      rintro _ ⟨x, _, rfl⟩
      exact iSup_le fun _ => le_rfl
  · rw [stepFun_of_not_wayBelow h]
    refine (sSup_eq_bot.2 ?_).symm
    rintro _ ⟨x, hxS, rfl⟩
    refine stepFun_of_not_wayBelow (fun hex => ?_)
    exact h (hex.trans_le (le_sSup hxS))

end DomainTheory.ContinuousLattice
```

7

stepFun_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun_of_not_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun_of_wayBelow
public import Mathlib.Order.BourbakiWitt
```

```
public import DomainTheory.ContinuousLattice.WayBelow.scottOpen_wayBelow
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- Step functions are Scott-continuous: `{x | e ≪ x}` is Scott-open, so the
    conditional commutes
with directed suprema. -/
theorem stepFun_preservesDirectedSup (e : D) (e' : D') :
    PreservesDirectedSup (stepFun e e') := by
  intro S hS hSdir
  show stepFun e e' (sSup S) = sSup (Set.image (stepFun e e') S)
  by_cases h : e ≪ sSup S
  · rw [stepFun_of_wayBelow h]
    obtain ⟨s, hsS, hes⟩ := (scottOpen_wayBelow e).2 hS hSdir h
    apply le_antisymm
    · refine le_sSup_of_le (Set.mem_image_of_mem _ hsS) ?_
      rw [stepFun_of_wayBelow hes]
    · refine sSup_le ?_
      rintro _ ⟨x, _, rfl⟩
      exact iSup_le fun _ => le_rfl
  · rw [stepFun_of_not_wayBelow h]
    refine (sSup_eq_bot.2 ?_).symm
    rintro _ ⟨x, hxS, rfl⟩
    refine stepFun_of_not_wayBelow (fun hex => ?_)
    exact h (hex.trans_le (le_sSup hxS))

end DomainTheory.ContinuousLattice
```

8

stepFun_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun_of_not_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun_of_wayBelow
public import Mathlib.Order.BourbakiWitt
public import DomainTheory.ContinuousLattice.WayBelow.scottOpen_wayBelow
```

```
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- Step functions are Scott-continuous: `{x | e ≪ x}` is Scott-open, so the
    conditional commutes
with directed suprema. -/
theorem stepFun_preservesDirectedSup (e : D) (e' : D') :
    PreservesDirectedSup (stepFun e e') := by
  intro S hS hSdir
  show stepFun e e' (sSup S) = sSup (Set.image (stepFun e e') S)
  by_cases h : e ≪ sSup S
  · rw [stepFun_of_wayBelow h]
    obtain ⟨s, hsS, hes⟩ := (scottOpen_wayBelow e).2 hS hSdir h
    apply le_antisymm
    · refine le_sSup_of_le (Set.mem_image_of_mem _ hsS) ?_
      rw [stepFun_of_wayBelow hes]
    · refine sSup_le ?_
      rintro _ ⟨x, _, rfl⟩
      exact iSup_le fun _ => le_rfl
  · rw [stepFun_of_not_wayBelow h]
    refine (sSup_eq_bot.2 ?_).symm
    rintro _ ⟨x, hxS, rfl⟩
    refine stepFun_of_not_wayBelow (fun hex => ?_)
    exact h (hex.trans_le (le_sSup hxS))

end DomainTheory.ContinuousLattice
```

9

stepMap.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    stepFun_preservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic
```

```
@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- The step function `e[e, e']` as a Scott map. -/
noncomputable def stepMap (e : D) (e' : D') : ScottMap D D' :=
  ⟨stepFun e e', continuous_of_preservesDirectedSup (stepFun_preservesDirectedSup e e')
    ⟩

end DomainTheory.ContinuousLattice
```

0

Builds a non-trivial reflexive domain $D \cong D \to D$ containing `U`.

The tip then assembles those tools into Scott's statement.

— `exercise_8_25`

3.9.28 Exercise 8.26 — untyped calculus in U

Discuss in more detail the "pay-off" for U, namely the translation of "untyped" λ-calculus into U as shown by the equations at the end of the lecture after the proof of 8.9. In particular show how the whole of the **typed** λ-calculus can be retranslated back into U with the aid of projections. (Hint: Whenever you want to write

$$f : D_a \to D_b,$$

write instead

$$f = b \circ f \circ a,$$

where a, b are finitary projections. Whenever you want to form a λ-abstraction

$$\lambda x^{D_a}.\, \sigma,$$

where σ is of type D_b, instead form

$$\lambda x.\, b(\sigma'[a(x)/x]),$$

where σ' is the further translation of σ into untyped λ-calculus. Be sure to show that this result "has the right type" in the sense defined above.)

Tip summary The tip asserts β-soundness of the self-hosted encoding: `Uapply (Ulam f) x = f(x)`.

Development Encodes self-application and self-abstraction inside `U` and proves β-soundness.

stepMap.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    stepFun_preservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- The step function `e[e, e']` as a Scott map. -/
noncomputable def stepMap (e : D) (e' : D') : ScottMap D D' :=
  ⟨stepFun e e', continuous_of_preservesDirectedSup (stepFun_preservesDirectedSup e e')
    ⟩

end DomainTheory.ContinuousLattice
```

1

stepMap.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    stepFun_preservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section
```

```
namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- The step function `e[e, e']` as a Scott map. -/
noncomputable def stepMap (e : D) (e' : D') : ScottMap D D' :=
  ⟨stepFun e e', continuous_of_preservesDirectedSup (stepFun_preservesDirectedSup e e')
    ⟩

end DomainTheory.ContinuousLattice
```

2

The tip then assembles those tools into Scott's statement.

— `exercise_8_26`

3.9.29 Exercise 8.27 — infinite cartesian products

(Suggested by James Donahue.)

Finite cartesian products of domains are formed by the $D_0 \times D_1$-construct we have used so often. The problem is to define — computably — some *infinite* cartesian products. In particular, as applied to the universal domain U, the combinator `sub` is to be regarded as a finitary projection of U whose fixed points are exactly *all* the finitary projections. A map

$$d = \mathrm{sub} \circ d \circ \mathrm{sub}$$

can be regarded as a *polymorphic type* (because, whenever t is a finitary projection (= type), then so is $d(t)$). The *continuous product* of *all* these types would be the domain of all approximable functions x such that

$$x(t) = d(t)(x(t))$$

for all types t. (Why does this equation mean that x is in the product?) Define Π as a combinator by

$$\Pi = \lambda d\, \lambda x\, \lambda t.\, \mathrm{sub}(d(\mathrm{sub}(t)))(x(\mathrm{sub}(t))).$$

Show that for d a polymorphic type, $\Pi(d)$ is a type. (Hint: It is easy to check that $\Pi(d)$ is a projection; the problem is to show it is *finitary*.)

Tip summary The tip asserts that for any polymorphic type `d`, $\Pi(d)$ is a finitary projection.

Development Defines polymorphic types and proves the `piD` side of finitariness.

stepMap.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    stepFun_preservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- The step function `e[e, e']` as a Scott map. -/
noncomputable def stepMap (e : D) (e' : D') : ScottMap D D' :=
  ⟨stepFun e e', continuous_of_preservesDirectedSup (stepFun_preservesDirectedSup e e')
    ⟩

end DomainTheory.ContinuousLattice
```


stepMap.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    stepFun_preservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice
```

```
open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- The step function `e[e, e']` as a Scott map. -/
noncomputable def stepMap (e : D) (e' : D') : ScottMap D D' :=
  ⟨stepFun e e', continuous_of_preservesDirectedSup (stepFun_preservesDirectedSup e e')
    ⟩

end DomainTheory.ContinuousLattice
```

4

stepMap.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    stepFun_preservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- The step function `e[e, e']` as a Scott map. -/
noncomputable def stepMap (e : D) (e' : D') : ScottMap D D' :=
  ⟨stepFun e e', continuous_of_preservesDirectedSup (stepFun_preservesDirectedSup e e')
    ⟩

end DomainTheory.ContinuousLattice
```

5

stepMap.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    stepFun_preservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- The step function `e[e, e']` as a Scott map. -/
noncomputable def stepMap (e : D) (e' : D') : ScottMap D D' :=
  ⟨stepFun e e', continuous_of_preservesDirectedSup (stepFun_preservesDirectedSup e e')
    ⟩

end DomainTheory.ContinuousLattice
```


stepMap.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    stepFun_preservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace
```

```
variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- The step function `e[e, e']` as a Scott map. -/
noncomputable def stepMap (e : D) (e' : D') : ScottMap D D' :=
  ⟨stepFun e e', continuous_of_preservesDirectedSup (stepFun_preservesDirectedSup e e')
    ⟩

end DomainTheory.ContinuousLattice
```

7

Carves `piU` and folds `XXX_piU` companions as `XXX.piU`.

— `piU`, `toElementMap.piU`, `isProjection.piU`, `isFinitaryProjection.piU`, …

The tip then assembles those tools into Scott's statement.

— `exercise_8_27`

4 Information Systems (1982)

The purpose of the theory of domains is to give models for spaces on which to define computable functions. The kinds of spaces needed for denotational semantics involve not only spaces of higher type (e.g. function spaces) but also spaces defined recursively (e.g. reflexive domains). Also required are many special domain constructs (or functors) in order to create the desired structures. There are several choices of a suitable category of domains, but the basic one which has the simplest properties is the one sometimes called *consistently complete algebraic* cpo*'s*. This category of domains is studied in this chapter from a new, and it is to be hoped, simpler point of view incorporating the approaches of many authors into a unified presentation. Briefly, the domains of elements are represented set theoretically with the aid of structures called *information systems*. These systems are very familiar from mathematical logic, and their use seems to accord well with intuition. Many things that were done previously axiomatically can now be proved in a straightforward way as theorems. The present chapter discusses many examples in an informal way that should serve as an introduction to the subject.

Dependencies. Each box is one folder. The arrowhead points at the folder that imports, and the number is how many direct imports.

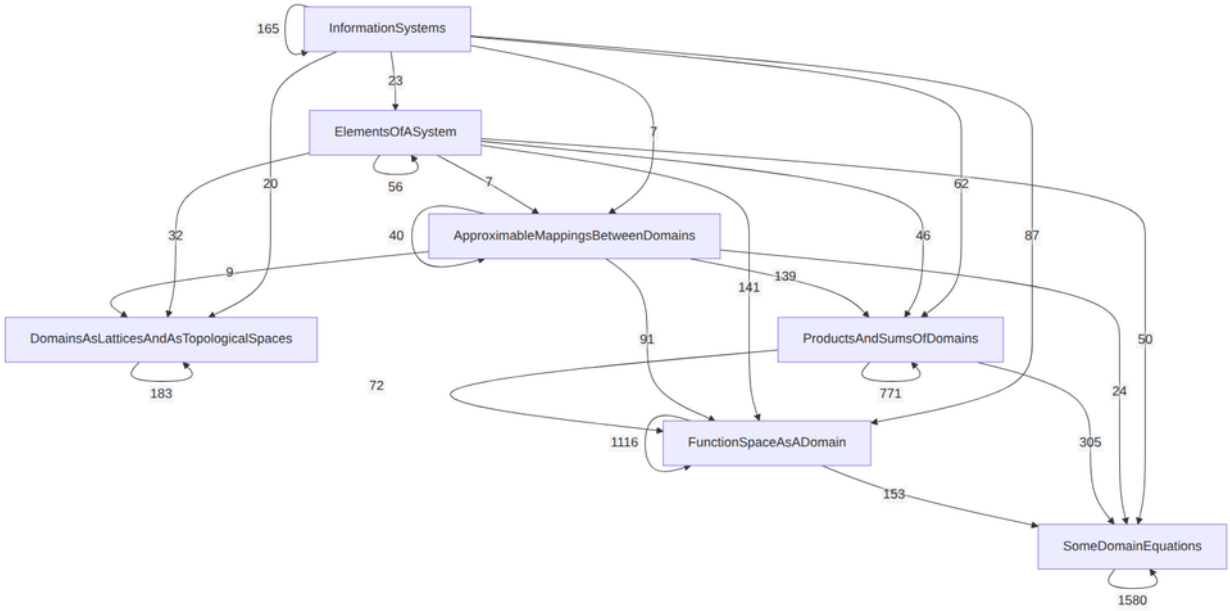


Figure 36: Information Systems (1982).

4.1 Introduction

The difficulty in the presentation of the subject is in justifying the level of abstraction used in comparison with the payoff: too often the effort needed for understanding the abstractions does not seem worth the trouble—especially if the notions are unfamiliar or excessively general.

For example, Category Theory, which is much used in discussions of the Theory of Domains, seems

far too abstract to many people. On the other hand, Automata Theory—which has often benefited from some use of Category Theory—is quite abstract and, in many aspects perhaps, quite useless, but the level of abstraction is fairly low and the reasons for the definitions are usually evident. Thus, the subject has become by now a standard topic even for undergraduate courses. Michael Rabin and Scott had much to do with influencing the subsequent development of the theory by formulating some years ago some basic concepts (many previously known) in a way that made sense to anyone who had been exposed to a little Abstract Algebra, Logic and Set Theory. This expository part of their paper did not require originality, but rather a certain frame of mind and a certain style of writing to put the ideas and problems in sharp relief. (The original paper is Rabin-Scott [1959]; see the excellent historical review of the subject in Greibach [1981].)

Lattices are very familiar to logicians and those interested in Universal Algebra. But it takes some time to learn the special terminology and to become comfortable with the necessary examples. Indeed, without a stock of examples, it is impossible to have sufficient intuition for making the required constructions. True, some people took to the approach, and the lattice-theoretic definitions were simple enough to motivate. But a much greater percentage of people Scott came in contact with found the large number of things to learn a definite roadblock to understanding. And the Lattice Theory was only partly standard, since this approach to the theory has to employ complete lattices and topological notions in order to explain *limits* and *continuity* adequately. In fact, Topology of a rather mathematically simple set-theoretical kind could have been used as the whole foundation much as in Scott [1972], but that kind of mathematics seems even less attractive to computer scientists. Nevertheless, the lattice-theoretic approach has great consistency and coherence; it has been carried out in full detail in Gierz, *et al.* [1980], which also gives the topological connections. That book contains a large number of references to the mathematical literature to show the connections of the theory to many other topics. What we had to miss out there is a clear hook-up with computability theory, owing simply to a lack of space and time to be able to cover everything.

In Scott [1981], Scott tried out another, different presentation (which is also explained in Scott [1982].) However, Scott found that the use of *neighborhood systems* causes confusion in people not used to thinking in terms of sets of sets of sets of sets . . . , even though the set theory required is quite simple. This section tries out another plan for making the story easy and natural. The notion to be used we call an *information system*. This is a "static" notion appropriate to bodies of information about coherent groups of elements. The "dynamic" ideas enter in the ways different systems can be related and in the semantical definitions about meanings of programming language constructs. (For discussions of semantical definitions see Stoy [1977], Gordon [1979], and Tennent [1981].) Neighborhood systems are, in a precise sense, equivalent to information systems, but there seems to be an interesting trade-off between what properties of structures are *axiomatized* and what are *implicit* in the form of the structure. Perhaps, despite their simplicity, neighborhoods leave too much implicit, too much to be extracted by definition; information systems, on the other hand, do seem to be more flexible in doing several of the important constructions. It all comes down to just where one imposes the closure conditions on the structures. Information systems give the right mix of axioms *vs.* definitions.

Another word about Category Theory: it is particularly significant and important for the theory and for the whole area of semantics. But it must be approached with great caution, for the sheer number of definitions and axioms can try the most patient reader. It is especially necessary in discussing applications of abstract mathematical ideas to keep the motivation strongly in mind. This is often hard to do if the categories get too thick, but of course it all depends on the writer. Category Theory is especially useful in stating *general* properties of structures and in *characterizing* constructions uniquely; however, there often is a problem actually justifying the *existence* of certain constructions, and a direct approach can be quicker than quoting lots of theorems. But, man cannot live by construction alone: theorems have to be proved in order to get

the proper value out of the work. Domain Theory must also be convenient for demonstrating the soundness of various proof rules for properties of recursively defined objects and recursively defined domains. Category Theory can be helpful here. A step in the right direction has been made in the LCF system (see Gordon-Milner-Wadsworth [1979]), which, however, does not take advantage of general Category Theory.

The main purpose of the Theory of Domains in Denotational Semantics is to give a mathematical model of a system of *types*—including function-space types—and to explain a notion of computability with respect to these types. There are many possible theories of types, but the construction of domains is meant to justify recursive definitions at all types, and—most essentially—to justify recursive definitions of types. Pursuing these goals has certain consequences, as we shall see. The model presented here is only one approach, and some comparisons with other methods will be made. Again some help from Category Theory would be welcome in comparing the different kinds of modelling.

One benefit of Domain Theory is that it is possible to make sense of types containing *infinite* elements. The cost required seems to be that the domains therefore have to contain *partial* elements as well as *total* elements. And this is where the lattice-theoretic definitions entered in the earlier presentations: the partial ordering with relations like $x \sqsubseteq y$ was used to express the fact that the element x was "less defined" (i.e. more partial) than y but "contained in" y. The relation $\sqsubseteq$, however, must be subjected to many axioms to have a theory suitable to the desired applications. Instead of such a big group of axioms, we put forward here a construction where the "skeleton" (or better: "backbone") of a domain is introduced by just a few axioms. Then the domain itself is defined as a certain construct from the backbone in order to define the appropriate notion of element. There are several advantages to this approach:

- Simple definitions of the basic concepts can be given.
- Detailed properties are proved as theorems rather than assumed as axioms.
- Emphasis can be given to the constructive nature of the definitions.
- Domains can be made more visible.
- The theory of domains is made more available for applications, because it is easier to produce the needed complex domains.

4.2 Information systems

Dependencies. Inside InformationSystems, each box is one subfolder, or one Lean file sitting directly in the folder. A box named as another folder in this chapter is that whole folder. A box named as another chapter is that whole chapter. The arrowhead points at the file or folder that imports, and the number is how many direct imports.

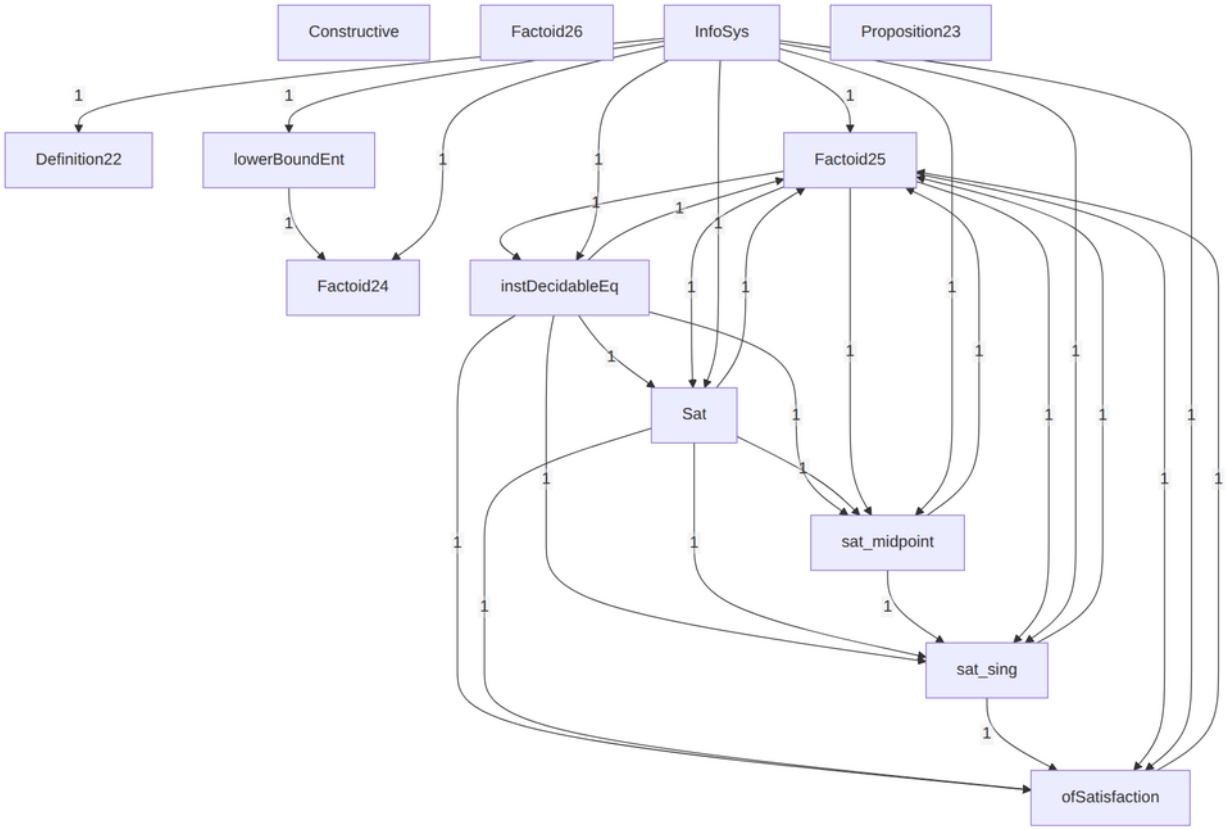


Figure 37: Information systems.

Intuitively, an information system is a set of "propositions" that can be made about "possible elements" of the desired domain. We will assume that sufficiently many propositions have been supplied to distinguish between distinct elements; as a consequence, an element can be constructed abstractly as the set of all the propositions that are true of it. Partial elements have "small" sets; while total elements have "large" sets (even: maximal). To make this somewhat rough idea precise, we have to explain—by a suitable, but small choice of axioms—how the collection of all propositions relevant to the domain hangs together, or better, is structured as a set of abstract propositions. Fortunately, the axioms for this structure are very simple and familiar, which is a great help in making up examples.

- Definition 2.1
- A first example
- A second example
- A third example
- Definition 2.2
- Proposition 2.3

So much for the definition of what an information system is as a mathematical structure. The next question is: What is it good for? More precisely, if information systems are the backbones of domains, then where are the elements? That is the topic of the next section.

stepMap.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    stepFun_preservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- The step function `e[e, e']` as a Scott map. -/
noncomputable def stepMap (e : D) (e' : D') : ScottMap D D' :=
  ⟨stepFun e e', continuous_of_preservesDirectedSup (stepFun_preservesDirectedSup e e')
    ⟩

end DomainTheory.ContinuousLattice
```


stepMap.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    stepFun_preservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section
```

```
namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- The step function `e[e, e']` as a Scott map. -/
noncomputable def stepMap (e : D) (e' : D') : ScottMap D D' :=
  ⟨stepFun e e', continuous_of_preservesDirectedSup (stepFun_preservesDirectedSup e e')
    ⟩

end DomainTheory.ContinuousLattice
```

9

stepMap_apply_of_wayBelow.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun_of_wayBelow
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem stepMap_apply_of_wayBelow {e : D} {e' : D'} {x : D} (h : e ≪ x) :
    (stepMap e e' : D → D') x = e' :=
  stepFun_of_wayBelow h

end DomainTheory.ContinuousLattice
```

0

stepMap_apply_of_wayBelow.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun_of_wayBelow
public import Mathlib.Topology.Defs.Basic
```

```
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem stepMap_apply_of_wayBelow {e : D} {e' : D'} {x : D} (h : e ≪ x) :
    (stepMap e e' : D → D') x = e' :=
  stepFun_of_wayBelow h

end DomainTheory.ContinuousLattice
```

1

stepMap_apply_of_wayBelow.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun_of_wayBelow
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem stepMap_apply_of_wayBelow {e : D} {e' : D'} {x : D} (h : e ≪ x) :
    (stepMap e e' : D → D') x = e' :=
  stepFun_of_wayBelow h

end DomainTheory.ContinuousLattice
```

2

stepMap_apply_of_wayBelow.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun_of_wayBelow
public import Mathlib.Topology.Defs.Basic
```

```
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem stepMap_apply_of_wayBelow {e : D} {e' : D'} {x : D} (h : e ≪ x) :
    (stepMap e e' : D → D') x = e' :=
  stepFun_of_wayBelow h

end DomainTheory.ContinuousLattice
```

3

stepMap_apply_of_wayBelow.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun_of_wayBelow
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem stepMap_apply_of_wayBelow {e : D} {e' : D'} {x : D} (h : e ≪ x) :
    (stepMap e e' : D → D') x = e' :=
  stepFun_of_wayBelow h

end DomainTheory.ContinuousLattice
```

4

stepMap_apply_of_wayBelow.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun_of_wayBelow
public import Mathlib.Topology.Defs.Basic
```

```
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem stepMap_apply_of_wayBelow {e : D} {e' : D'} {x : D} (h : e ≪ x) :
    (stepMap e e' : D → D') x = e' :=
  stepFun_of_wayBelow h

end DomainTheory.ContinuousLattice
```

5

stepMap_apply_of_wayBelow.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun_of_wayBelow
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem stepMap_apply_of_wayBelow {e : D} {e' : D'} {x : D} (h : e ≪ x) :
    (stepMap e e' : D → D') x = e' :=
  stepFun_of_wayBelow h

end DomainTheory.ContinuousLattice
```

6

stepMap_apply_of_wayBelow.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun_of_wayBelow
public import Mathlib.Topology.Defs.Basic
```

```
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem stepMap_apply_of_wayBelow {e : D} {e' : D'} {x : D} (h : e ≪ x) :
    (stepMap e e' : D → D') x = e' :=
  stepFun_of_wayBelow h

end DomainTheory.ContinuousLattice
```

7

stepMap_apply_of_wayBelow.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun_of_wayBelow
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem stepMap_apply_of_wayBelow {e : D} {e' : D'} {x : D} (h : e ≪ x) :
    (stepMap e e' : D → D') x = e' :=
  stepFun_of_wayBelow h

end DomainTheory.ContinuousLattice
```

8

stepMap_apply_of_wayBelow.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun_of_wayBelow
public import Mathlib.Topology.Defs.Basic
```

```
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem stepMap_apply_of_wayBelow {e : D} {e' : D'} {x : D} (h : e ≪ x) :
    (stepMap e e' : D → D') x = e' :=
  stepFun_of_wayBelow h

end DomainTheory.ContinuousLattice
```

9

stepMap_le_of_wayBelow.lean

```
module
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_apply_of_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun_of_not_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst28
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- If `e' ≪ g e` then the step function `e[e, e']` lies below `g` (a pointwise
    argument). -/
theorem stepMap_le_of_wayBelow {e : D} {e' : D'} {g : ScottMap D D'}
    (h : e' ≪ (g : D → D') e) : stepMap e e' ≤ g := by
  rw [ScottMap.le_def]
  intro x
  by_cases hex : e ≪ x
  · rw [stepMap_apply_of_wayBelow hex]
    exact h.le.trans (g.monotone hex.le)
```

```
  · show stepFun e e' x ≤ (g : D → D') x
    rw [stepFun_of_not_wayBelow hex]
    exact bot_le

end DomainTheory.ContinuousLattice
```

0

stepMap_le_of_wayBelow.lean

```
module
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_apply_of_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun_of_not_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst28
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- If `e' ≪ g e` then the step function `e[e, e']` lies below `g` (a pointwise
    argument). -/
theorem stepMap_le_of_wayBelow {e : D} {e' : D'} {g : ScottMap D D'}
    (h : e' ≪ (g : D → D') e) : stepMap e e' ≤ g := by
  rw [ScottMap.le_def]
  intro x
  by_cases hex : e ≪ x
  · rw [stepMap_apply_of_wayBelow hex]
    exact h.le.trans (g.monotone hex.le)
  · show stepFun e e' x ≤ (g : D → D') x
    rw [stepFun_of_not_wayBelow hex]
    exact bot_le

end DomainTheory.ContinuousLattice
```

1

stepMap_le_of_wayBelow.lean

```
module
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_apply_of_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun_of_not_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst28
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- If `e' ≪ g e` then the step function `e[e, e']` lies below `g` (a pointwise
    argument). -/
theorem stepMap_le_of_wayBelow {e : D} {e' : D'} {g : ScottMap D D'}
    (h : e' ≪ (g : D → D') e) : stepMap e e' ≤ g := by
  rw [ScottMap.le_def]
  intro x
  by_cases hex : e ≪ x
  · rw [stepMap_apply_of_wayBelow hex]
    exact h.le.trans (g.monotone hex.le)
  · show stepFun e e' x ≤ (g : D → D') x
    rw [stepFun_of_not_wayBelow hex]
    exact bot_le

end DomainTheory.ContinuousLattice
```

2

4.3 The elements of a system

Dependencies. Inside ElementsOfASystem, each box is one subfolder, or one Lean file sitting directly in the folder. A box named as another folder in this chapter is that whole folder. A box named as another chapter is that whole chapter. The arrowhead points at the file or folder that imports, and the number is how many direct imports.

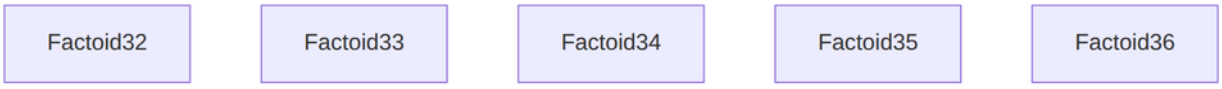


Figure 38: The elements of a system.

Let $\mathbf{A}$ be an information system, and suppose we already had a concept of being an element of $\mathbf{A}$. The data objects are meant to be propositions about elements, so if X is in $D_{\mathbf{A}}$ and if x is an element, then we can be expected to know what it means to say that X is true of x. Since all we are given is the set of data

objects $D_{\mathbf{A}}$, we have to assume that it contains enough objects to distinguish between distinct elements. (If there were not enough of them, we would have to change the set $D_{\mathbf{A}}$.) Formally, we can write of two elements x and y:

$$x = y \quad \text{iff all } X \in D_{\mathbf{A}} \text{ which are true of } x \text{ are also true of } y, \text{ and conversely.}$$

If this principle is accepted, then the elements can be *identified* with the sets of propositions true of them; formally we can assert:

$$x = \{X \in D_{\mathbf{A}} \mid X \text{ is true of } x\}.$$

There can be no confusion injected here in identifying elements that ought to be distinct, since we are assuming that there are enough data objects in the system. If for some reason we felt we ought to have more of them to distinguish between more elements, then we would have to pass to a larger and *different* information system. By agreement, then, the above equation is a tautology. So, for the sake of simplicity, elements can be taken to be *sets* of data objects. But, we hasten to ask, which sets? The question is really: What are the properties of truth? The answers are well known, and, in fact, we have already used them in our examples in the last section. In words, the set of true propositions about a possible element must be (i) consistent in itself, and (ii) closed under entailment (or *deductively closed*, for short). Condition (i) is clear because we are talking about a *possible* element, not an impossible one. Condition (ii) is acceptable, because, by the very meaning of the word, entailment should be truth preserving. In addition, we are also saying the converse: *any* set having properties (i) and (ii) corresponds to a (partial) element. Here is the formal statement:

- Definition 3.1

stepMap_le_of_wayBelow.lean

```
module
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_apply_of_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun_of_not_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst28
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']
```

```
/-- If `e' ≪ g e` then the step function `e[e, e']` lies below `g` (a pointwise
    argument). -/
theorem stepMap_le_of_wayBelow {e : D} {e' : D'} {g : ScottMap D D'}
    (h : e' ≪ (g : D → D') e) : stepMap e e' ≤ g := by
  rw [ScottMap.le_def]
  intro x
  by_cases hex : e ≪ x
  · rw [stepMap_apply_of_wayBelow hex]
    exact h.le.trans (g.monotone hex.le)
  · show stepFun e e' x ≤ (g : D → D') x
    rw [stepFun_of_not_wayBelow hex]
    exact bot_le

end DomainTheory.ContinuousLattice
```

3

stepMap_le_of_wayBelow.lean

```
module
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_apply_of_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun_of_not_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst28
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- If `e' ≪ g e` then the step function `e[e, e']` lies below `g` (a pointwise
    argument). -/
theorem stepMap_le_of_wayBelow {e : D} {e' : D'} {g : ScottMap D D'}
    (h : e' ≪ (g : D → D') e) : stepMap e e' ≤ g := by
  rw [ScottMap.le_def]
  intro x
  by_cases hex : e ≪ x
  · rw [stepMap_apply_of_wayBelow hex]
    exact h.le.trans (g.monotone hex.le)
  · show stepFun e e' x ≤ (g : D → D') x
    rw [stepFun_of_not_wayBelow hex]
    exact bot_le

end DomainTheory.ContinuousLattice
```

4

stepMap_le_of_wayBelow.lean

```
module
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_apply_of_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun_of_not_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst28
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- If `e' ≪ g e` then the step function `e[e, e']` lies below `g` (a pointwise
    argument). -/
theorem stepMap_le_of_wayBelow {e : D} {e' : D'} {g : ScottMap D D'}
    (h : e' ≪ (g : D → D') e) : stepMap e e' ≤ g := by
  rw [ScottMap.le_def]
  intro x
  by_cases hex : e ≪ x
  · rw [stepMap_apply_of_wayBelow hex]
    exact h.le.trans (g.monotone hex.le)
  · show stepFun e e' x ≤ (g : D → D') x
    rw [stepFun_of_not_wayBelow hex]
    exact bot_le

end DomainTheory.ContinuousLattice
```

5

stepMap_le_of_wayBelow.lean

```
module
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_apply_of_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun_of_not_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst28
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic
```

```
@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- If `e' ≪ g e` then the step function `e[e, e']` lies below `g` (a pointwise
    argument). -/
theorem stepMap_le_of_wayBelow {e : D} {e' : D'} {g : ScottMap D D'}
    (h : e' ≪ (g : D → D') e) : stepMap e e' ≤ g := by
  rw [ScottMap.le_def]
  intro x
  by_cases hex : e ≪ x
  · rw [stepMap_apply_of_wayBelow hex]
    exact h.le.trans (g.monotone hex.le)
  · show stepFun e e' x ≤ (g : D → D') x
    rw [stepFun_of_not_wayBelow hex]
    exact bot_le

end DomainTheory.ContinuousLattice
```

6

stepMap_le_of_wayBelow.lean

```
module
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_apply_of_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun_of_not_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst28
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- If `e' ≪ g e` then the step function `e[e, e']` lies below `g` (a pointwise
```

```
    argument). -/
theorem stepMap_le_of_wayBelow {e : D} {e' : D'} {g : ScottMap D D'}
    (h : e' ≪ (g : D → D') e) : stepMap e e' ≤ g := by
  rw [ScottMap.le_def]
  intro x
  by_cases hex : e ≪ x
  · rw [stepMap_apply_of_wayBelow hex]
    exact h.le.trans (g.monotone hex.le)
  · show stepFun e e' x ≤ (g : D → D') x
    rw [stepFun_of_not_wayBelow hex]
    exact bot_le

end DomainTheory.ContinuousLattice
```

7

stepMap_le_of_wayBelow.lean

```
module
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_apply_of_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun_of_not_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst28
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- If `e' ≪ g e` then the step function `e[e, e']` lies below `g` (a pointwise
    argument). -/
theorem stepMap_le_of_wayBelow {e : D} {e' : D'} {g : ScottMap D D'}
    (h : e' ≪ (g : D → D') e) : stepMap e e' ≤ g := by
  rw [ScottMap.le_def]
  intro x
  by_cases hex : e ≪ x
  · rw [stepMap_apply_of_wayBelow hex]
    exact h.le.trans (g.monotone hex.le)
  · show stepFun e e' x ≤ (g : D → D') x
    rw [stepFun_of_not_wayBelow hex]
    exact bot_le

end DomainTheory.ContinuousLattice
```

8

stepMap_le_of_wayBelow.lean

```
module
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_apply_of_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepFun_of_not_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst28
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- If `e' ≪ g e` then the step function `e[e, e']` lies below `g` (a pointwise
    argument). -/
theorem stepMap_le_of_wayBelow {e : D} {e' : D'} {g : ScottMap D D'}
    (h : e' ≪ (g : D → D') e) : stepMap e e' ≤ g := by
  rw [ScottMap.le_def]
  intro x
  by_cases hex : e ≪ x
  · rw [stepMap_apply_of_wayBelow hex]
    exact h.le.trans (g.monotone hex.le)
  · show stepFun e e' x ≤ (g : D → D') x
    rw [stepFun_of_not_wayBelow hex]
    exact bot_le

end DomainTheory.ContinuousLattice
```

9

stepMap_pointwise_sSup.lean

```
module
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice

public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.WayBelow.directedOn_wayBelow
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic
```

```
@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Pointwise reconstruction.** For a Scott map `f`, the supremum of the values of
    the step
functions below `f` recovers `f x`: `f x = ⊔ {e' | ∃ e ≪ x, e' ≪ f e}`. Uses
    continuity of `D`
(`x = ⊔{e | e ≪ x}`), Scott-continuity of `f`, and continuity of `D''`. -/
theorem stepMap_pointwise_sSup (hD : ContinuousLattice D) (hD' : ContinuousLattice D')
    (f : ScottMap D D') (x : D) :
    sSup {e' : D' | ∃ e, e ≪ x ∧ e' ≪ (f : D → D') e} = (f : D → D') x := by
  apply le_antisymm
  · refine sSup_le ?_
    rintro e' ⟨e, hex, he'⟩
    exact he'.le.trans (f.monotone hex.le)
  · rw [← hD'.sSup_wayBelow ((f : D → D') x)]
    refine sSup_le ?_
    intro w hw
    have hfx : (f : D → D') x = sSup (Set.image (f : D → D') {e | e ≪ x}) := by
      rw [← f.preservesDirectedSup_coe {e | e ≪ x} ⟨⊥, bot_wayBelow x⟩ (
    directedOn_wayBelow x),
        hD.sSup_wayBelow x]
    rw [hfx] at hw
    have hne : (Set.image (f : D → D') {e | e ≪ x}).Nonempty := ⟨f ⊥, ⊥, bot_wayBelow
     x, rfl⟩
    have hdir : DirectedOn (· ≤ ·) (Set.image (f : D → D') {e | e ≪ x}) := by
      rintro _ ⟨a, ha, rfl⟩ _ ⟨b, hb, rfl⟩
      obtain ⟨c, hc, hac, hbc⟩ := directedOn_wayBelow x a ha b hb
      exact ⟨f c, Set.mem_image_of_mem _ hc, f.monotone hac, f.monotone hbc⟩
    obtain ⟨_, ⟨e, hex, rfl⟩, hwe⟩ := (WayBelow.sSup_iff hne hdir).1 hw
    exact le_sSup ⟨e, hex, hwe⟩

end DomainTheory.ContinuousLattice
```

0

stepMap_pointwise_sSup.lean

```
module
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice

public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.WayBelow.directedOn_wayBelow
```

```
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Pointwise reconstruction.** For a Scott map `f`, the supremum of the values of
    the step
functions below `f` recovers `f x`: `f x = ⊔ {e' | ∃ e ≪ x, e' ≪ f e}`. Uses
    continuity of `D`
(`x = ⊔{e | e ≪ x}`), Scott-continuity of `f`, and continuity of `D''`. -/
theorem stepMap_pointwise_sSup (hD : ContinuousLattice D) (hD' : ContinuousLattice D')
    (f : ScottMap D D') (x : D) :
    sSup {e' : D' | ∃ e, e ≪ x ∧ e' ≪ (f : D → D') e} = (f : D → D') x := by
  apply le_antisymm
  · refine sSup_le ?_
    rintro e' ⟨e, hex, he'⟩
    exact he'.le.trans (f.monotone hex.le)
  · rw [← hD'.sSup_wayBelow ((f : D → D') x)]
    refine sSup_le ?_
    intro w hw
    have hfx : (f : D → D') x = sSup (Set.image (f : D → D') {e | e ≪ x}) := by
      rw [← f.preservesDirectedSup_coe {e | e ≪ x} ⟨⊥, bot_wayBelow x⟩ (
    directedOn_wayBelow x),
        hD.sSup_wayBelow x]
    rw [hfx] at hw
    have hne : (Set.image (f : D → D') {e | e ≪ x}).Nonempty := ⟨f ⊥, ⊥, bot_wayBelow
     x, rfl⟩
    have hdir : DirectedOn (· ≤ ·) (Set.image (f : D → D') {e | e ≪ x}) := by
      rintro _ ⟨a, ha, rfl⟩ _ ⟨b, hb, rfl⟩
      obtain ⟨c, hc, hac, hbc⟩ := directedOn_wayBelow x a ha b hb
      exact ⟨f c, Set.mem_image_of_mem _ hc, f.monotone hac, f.monotone hbc⟩
    obtain ⟨_, ⟨e, hex, rfl⟩, hwe⟩ := (WayBelow.sSup_iff hne hdir).1 hw
    exact le_sSup ⟨e, hex, hwe⟩

end DomainTheory.ContinuousLattice
```

1

stepMap_pointwise_sSup.lean

```
module
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice

public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.WayBelow.directedOn_wayBelow
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Pointwise reconstruction.** For a Scott map `f`, the supremum of the values of
    the step
functions below `f` recovers `f x`: `f x = ⊔ {e' | ∃ e ≪ x, e' ≪ f e}`. Uses
    continuity of `D`
(`x = ⊔{e | e ≪ x}`), Scott-continuity of `f`, and continuity of `D''`. -/
theorem stepMap_pointwise_sSup (hD : ContinuousLattice D) (hD' : ContinuousLattice D')
    (f : ScottMap D D') (x : D) :
    sSup {e' : D' | ∃ e, e ≪ x ∧ e' ≪ (f : D → D') e} = (f : D → D') x := by
  apply le_antisymm
  · refine sSup_le ?_
    rintro e' ⟨e, hex, he'⟩
    exact he'.le.trans (f.monotone hex.le)
  · rw [← hD'.sSup_wayBelow ((f : D → D') x)]
    refine sSup_le ?_
    intro w hw
    have hfx : (f : D → D') x = sSup (Set.image (f : D → D') {e | e ≪ x}) := by
      rw [← f.preservesDirectedSup_coe {e | e ≪ x} ⟨⊥, bot_wayBelow x⟩ (
    directedOn_wayBelow x),
        hD.sSup_wayBelow x]
    rw [hfx] at hw
    have hne : (Set.image (f : D → D') {e | e ≪ x}).Nonempty := ⟨f ⊥, ⊥, bot_wayBelow
     x, rfl⟩
    have hdir : DirectedOn (· ≤ ·) (Set.image (f : D → D') {e | e ≪ x}) := by
      rintro _ ⟨a, ha, rfl⟩ _ ⟨b, hb, rfl⟩
      obtain ⟨c, hc, hac, hbc⟩ := directedOn_wayBelow x a ha b hb
      exact ⟨f c, Set.mem_image_of_mem _ hc, f.monotone hac, f.monotone hbc⟩
    obtain ⟨_, ⟨e, hex, rfl⟩, hwe⟩ := (WayBelow.sSup_iff hne hdir).1 hw
    exact le_sSup ⟨e, hex, hwe⟩

end DomainTheory.ContinuousLattice
```

2

stepMap_pointwise_sSup.lean

```
module
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.WayBelow.directedOn_wayBelow
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Pointwise reconstruction.** For a Scott map `f`, the supremum of the values of
    the step
functions below `f` recovers `f x`: `f x = ⊔ {e' | ∃ e ≪ x, e' ≪ f e}`. Uses
    continuity of `D`
(`x = ⊔{e | e ≪ x}`), Scott-continuity of `f`, and continuity of `D''`. -/
theorem stepMap_pointwise_sSup (hD : ContinuousLattice D) (hD' : ContinuousLattice D')
    (f : ScottMap D D') (x : D) :
    sSup {e' : D' | ∃ e, e ≪ x ∧ e' ≪ (f : D → D') e} = (f : D → D') x := by
  apply le_antisymm
  · refine sSup_le ?_
    rintro e' ⟨e, hex, he'⟩
    exact he'.le.trans (f.monotone hex.le)
  · rw [← hD'.sSup_wayBelow ((f : D → D') x)]
    refine sSup_le ?_
    intro w hw
    have hfx : (f : D → D') x = sSup (Set.image (f : D → D') {e | e ≪ x}) := by
      rw [← f.preservesDirectedSup_coe {e | e ≪ x} ⟨⊥, bot_wayBelow x⟩ (
    directedOn_wayBelow x),
        hD.sSup_wayBelow x]
    rw [hfx] at hw
    have hne : (Set.image (f : D → D') {e | e ≪ x}).Nonempty := ⟨f ⊥, ⊥, bot_wayBelow
     x, rfl⟩
    have hdir : DirectedOn (· ≤ ·) (Set.image (f : D → D') {e | e ≪ x}) := by
      rintro _ ⟨a, ha, rfl⟩ _ ⟨b, hb, rfl⟩
      obtain ⟨c, hc, hac, hbc⟩ := directedOn_wayBelow x a ha b hb
      exact ⟨f c, Set.mem_image_of_mem _ hc, f.monotone hac, f.monotone hbc⟩
    obtain ⟨_, ⟨e, hex, rfl⟩, hwe⟩ := (WayBelow.sSup_iff hne hdir).1 hw
    exact le_sSup ⟨e, hex, hwe⟩

end DomainTheory.ContinuousLattice
```

3

stepMap_pointwise_sSup.lean

```
module
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
```

```
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice

public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.WayBelow.directedOn_wayBelow
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Pointwise reconstruction.** For a Scott map `f`, the supremum of the values of
    the step
functions below `f` recovers `f x`: `f x = ⊔ {e' | ∃ e ≪ x, e' ≪ f e}`. Uses
    continuity of `D`
(`x = ⊔{e | e ≪ x}`), Scott-continuity of `f`, and continuity of `D''`. -/
theorem stepMap_pointwise_sSup (hD : ContinuousLattice D) (hD' : ContinuousLattice D')
    (f : ScottMap D D') (x : D) :
    sSup {e' : D' | ∃ e, e ≪ x ∧ e' ≪ (f : D → D') e} = (f : D → D') x := by
  apply le_antisymm
  · refine sSup_le ?_
    rintro e' ⟨e, hex, he'⟩
    exact he'.le.trans (f.monotone hex.le)
  · rw [← hD'.sSup_wayBelow ((f : D → D') x)]
    refine sSup_le ?_
    intro w hw
    have hfx : (f : D → D') x = sSup (Set.image (f : D → D') {e | e ≪ x}) := by
      rw [← f.preservesDirectedSup_coe {e | e ≪ x} ⟨⊥, bot_wayBelow x⟩ (
    directedOn_wayBelow x),
        hD.sSup_wayBelow x]
    rw [hfx] at hw
    have hne : (Set.image (f : D → D') {e | e ≪ x}).Nonempty := ⟨f ⊥, ⊥, bot_wayBelow
     x, rfl⟩
    have hdir : DirectedOn (· ≤ ·) (Set.image (f : D → D') {e | e ≪ x}) := by
      rintro _ ⟨a, ha, rfl⟩ _ ⟨b, hb, rfl⟩
      obtain ⟨c, hc, hac, hbc⟩ := directedOn_wayBelow x a ha b hb
      exact ⟨f c, Set.mem_image_of_mem _ hc, f.monotone hac, f.monotone hbc⟩
    obtain ⟨_, ⟨e, hex, rfl⟩, hwe⟩ := (WayBelow.sSup_iff hne hdir).1 hw
    exact le_sSup ⟨e, hex, hwe⟩

end DomainTheory.ContinuousLattice
```

4

stepMap_pointwise_sSup.lean

```
module
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice

public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.WayBelow.directedOn_wayBelow
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Pointwise reconstruction.** For a Scott map `f`, the supremum of the values of
    the step
functions below `f` recovers `f x`: `f x = ⊔ {e' | ∃ e ≪ x, e' ≪ f e}`. Uses
    continuity of `D`
(`x = ⊔{e | e ≪ x}`), Scott-continuity of `f`, and continuity of `D''`. -/
theorem stepMap_pointwise_sSup (hD : ContinuousLattice D) (hD' : ContinuousLattice D')
    (f : ScottMap D D') (x : D) :
    sSup {e' : D' | ∃ e, e ≪ x ∧ e' ≪ (f : D → D') e} = (f : D → D') x := by
  apply le_antisymm
  · refine sSup_le ?_
    rintro e' ⟨e, hex, he'⟩
    exact he'.le.trans (f.monotone hex.le)
  · rw [← hD'.sSup_wayBelow ((f : D → D') x)]
    refine sSup_le ?_
    intro w hw
    have hfx : (f : D → D') x = sSup (Set.image (f : D → D') {e | e ≪ x}) := by
      rw [← f.preservesDirectedSup_coe {e | e ≪ x} ⟨⊥, bot_wayBelow x⟩ (
    directedOn_wayBelow x),
        hD.sSup_wayBelow x]
    rw [hfx] at hw
    have hne : (Set.image (f : D → D') {e | e ≪ x}).Nonempty := ⟨f ⊥, ⊥, bot_wayBelow
     x, rfl⟩
    have hdir : DirectedOn (· ≤ ·) (Set.image (f : D → D') {e | e ≪ x}) := by
      rintro _ ⟨a, ha, rfl⟩ _ ⟨b, hb, rfl⟩
      obtain ⟨c, hc, hac, hbc⟩ := directedOn_wayBelow x a ha b hb
      exact ⟨f c, Set.mem_image_of_mem _ hc, f.monotone hac, f.monotone hbc⟩
    obtain ⟨_, ⟨e, hex, rfl⟩, hwe⟩ := (WayBelow.sSup_iff hne hdir).1 hw
    exact le_sSup ⟨e, hex, hwe⟩

end DomainTheory.ContinuousLattice
```

stepMap_pointwise_sSup.lean

```
module
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice

public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.WayBelow.directedOn_wayBelow
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Pointwise reconstruction.** For a Scott map `f`, the supremum of the values of
    the step
functions below `f` recovers `f x`: `f x = ⊔ {e' | ∃ e ≪ x, e' ≪ f e}`. Uses
    continuity of `D`
(`x = ⊔{e | e ≪ x}`), Scott-continuity of `f`, and continuity of `D''`. -/
theorem stepMap_pointwise_sSup (hD : ContinuousLattice D) (hD' : ContinuousLattice D')
    (f : ScottMap D D') (x : D) :
    sSup {e' : D' | ∃ e, e ≪ x ∧ e' ≪ (f : D → D') e} = (f : D → D') x := by
  apply le_antisymm
  · refine sSup_le ?_
    rintro e' ⟨e, hex, he'⟩
    exact he'.le.trans (f.monotone hex.le)
  · rw [← hD'.sSup_wayBelow ((f : D → D') x)]
    refine sSup_le ?_
    intro w hw
    have hfx : (f : D → D') x = sSup (Set.image (f : D → D') {e | e ≪ x}) := by
      rw [← f.preservesDirectedSup_coe {e | e ≪ x} ⟨⊥, bot_wayBelow x⟩ (
    directedOn_wayBelow x),
        hD.sSup_wayBelow x]
    rw [hfx] at hw
    have hne : (Set.image (f : D → D') {e | e ≪ x}).Nonempty := ⟨f ⊥, ⊥, bot_wayBelow
     x, rfl⟩
    have hdir : DirectedOn (· ≤ ·) (Set.image (f : D → D') {e | e ≪ x}) := by
      rintro _ ⟨a, ha, rfl⟩ _ ⟨b, hb, rfl⟩
      obtain ⟨c, hc, hac, hbc⟩ := directedOn_wayBelow x a ha b hb
      exact ⟨f c, Set.mem_image_of_mem _ hc, f.monotone hac, f.monotone hbc⟩
    obtain ⟨_, ⟨e, hex, rfl⟩, hwe⟩ := (WayBelow.sSup_iff hne hdir).1 hw
    exact le_sSup ⟨e, hex, hwe⟩

end DomainTheory.ContinuousLattice
```

6

stepMap_pointwise_sSup.lean

```
module
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice

public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.WayBelow.directedOn_wayBelow
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Pointwise reconstruction.** For a Scott map `f`, the supremum of the values of
    the step
functions below `f` recovers `f x`: `f x = ⊔ {e' | ∃ e ≪ x, e' ≪ f e}`. Uses
    continuity of `D`
(`x = ⊔{e | e ≪ x}`), Scott-continuity of `f`, and continuity of `D''`. -/
theorem stepMap_pointwise_sSup (hD : ContinuousLattice D) (hD' : ContinuousLattice D')
    (f : ScottMap D D') (x : D) :
    sSup {e' : D' | ∃ e, e ≪ x ∧ e' ≪ (f : D → D') e} = (f : D → D') x := by
  apply le_antisymm
  · refine sSup_le ?_
    rintro e' ⟨e, hex, he'⟩
    exact he'.le.trans (f.monotone hex.le)
  · rw [← hD'.sSup_wayBelow ((f : D → D') x)]
    refine sSup_le ?_
    intro w hw
    have hfx : (f : D → D') x = sSup (Set.image (f : D → D') {e | e ≪ x}) := by
      rw [← f.preservesDirectedSup_coe {e | e ≪ x} ⟨⊥, bot_wayBelow x⟩ (
    directedOn_wayBelow x),
        hD.sSup_wayBelow x]
    rw [hfx] at hw
    have hne : (Set.image (f : D → D') {e | e ≪ x}).Nonempty := ⟨f ⊥, ⊥, bot_wayBelow
     x, rfl⟩
    have hdir : DirectedOn (· ≤ ·) (Set.image (f : D → D') {e | e ≪ x}) := by
      rintro _ ⟨a, ha, rfl⟩ _ ⟨b, hb, rfl⟩
      obtain ⟨c, hc, hac, hbc⟩ := directedOn_wayBelow x a ha b hb
      exact ⟨f c, Set.mem_image_of_mem _ hc, f.monotone hac, f.monotone hbc⟩
    obtain ⟨_, ⟨e, hex, rfl⟩, hwe⟩ := (WayBelow.sSup_iff hne hdir).1 hw
    exact le_sSup ⟨e, hex, hwe⟩
```

```
end DomainTheory.ContinuousLattice
```

7

stepMap_pointwise_sSup.lean

```
module
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice

public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.WayBelow.directedOn_wayBelow
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Pointwise reconstruction.** For a Scott map `f`, the supremum of the values of
    the step
functions below `f` recovers `f x`: `f x = ⊔ {e' | ∃ e ≪ x, e' ≪ f e}`. Uses
    continuity of `D`
(`x = ⊔{e | e ≪ x}`), Scott-continuity of `f`, and continuity of `D''`. -/
theorem stepMap_pointwise_sSup (hD : ContinuousLattice D) (hD' : ContinuousLattice D')
    (f : ScottMap D D') (x : D) :
    sSup {e' : D' | ∃ e, e ≪ x ∧ e' ≪ (f : D → D') e} = (f : D → D') x := by
  apply le_antisymm
  · refine sSup_le ?_
    rintro e' ⟨e, hex, he'⟩
    exact he'.le.trans (f.monotone hex.le)
  · rw [← hD'.sSup_wayBelow ((f : D → D') x)]
    refine sSup_le ?_
    intro w hw
    have hfx : (f : D → D') x = sSup (Set.image (f : D → D') {e | e ≪ x}) := by
      rw [← f.preservesDirectedSup_coe {e | e ≪ x} ⟨⊥, bot_wayBelow x⟩ (
    directedOn_wayBelow x),
        hD.sSup_wayBelow x]
    rw [hfx] at hw
    have hne : (Set.image (f : D → D') {e | e ≪ x}).Nonempty := ⟨f ⊥, ⊥, bot_wayBelow
     x, rfl⟩
    have hdir : DirectedOn (· ≤ ·) (Set.image (f : D → D') {e | e ≪ x}) := by
      rintro _ ⟨a, ha, rfl⟩ _ ⟨b, hb, rfl⟩
      obtain ⟨c, hc, hac, hbc⟩ := directedOn_wayBelow x a ha b hb
      exact ⟨f c, Set.mem_image_of_mem _ hc, f.monotone hac, f.monotone hbc⟩
    obtain ⟨_, ⟨e, hex, rfl⟩, hwe⟩ := (WayBelow.sSup_iff hne hdir).1 hw
```

```
    exact le_sSup ⟨e, hex, hwe⟩

end DomainTheory.ContinuousLattice
```

8

stepMap_pointwise_sSup.lean

```
module
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice

public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.WayBelow.directedOn_wayBelow
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Pointwise reconstruction.** For a Scott map `f`, the supremum of the values of
    the step
functions below `f` recovers `f x`: `f x = ⊔ {e' | ∃ e ≪ x, e' ≪ f e}`. Uses
    continuity of `D`
(`x = ⊔{e | e ≪ x}`), Scott-continuity of `f`, and continuity of `D''`. -/
theorem stepMap_pointwise_sSup (hD : ContinuousLattice D) (hD' : ContinuousLattice D')
    (f : ScottMap D D') (x : D) :
    sSup {e' : D' | ∃ e, e ≪ x ∧ e' ≪ (f : D → D') e} = (f : D → D') x := by
  apply le_antisymm
  · refine sSup_le ?_
    rintro e' ⟨e, hex, he'⟩
    exact he'.le.trans (f.monotone hex.le)
  · rw [← hD'.sSup_wayBelow ((f : D → D') x)]
    refine sSup_le ?_
    intro w hw
    have hfx : (f : D → D') x = sSup (Set.image (f : D → D') {e | e ≪ x}) := by
      rw [← f.preservesDirectedSup_coe {e | e ≪ x} ⟨⊥, bot_wayBelow x⟩ (
    directedOn_wayBelow x),
        hD.sSup_wayBelow x]
    rw [hfx] at hw
    have hne : (Set.image (f : D → D') {e | e ≪ x}).Nonempty := ⟨f ⊥, ⊥, bot_wayBelow
     x, rfl⟩
    have hdir : DirectedOn (· ≤ ·) (Set.image (f : D → D') {e | e ≪ x}) := by
      rintro _ ⟨a, ha, rfl⟩ _ ⟨b, hb, rfl⟩
      obtain ⟨c, hc, hac, hbc⟩ := directedOn_wayBelow x a ha b hb
```

```
      exact ⟨f c, Set.mem_image_of_mem _ hc, f.monotone hac, f.monotone hbc⟩
    obtain ⟨_, ⟨e, hex, rfl⟩, hwe⟩ := (WayBelow.sSup_iff hne hdir).1 hw
    exact le_sSup ⟨e, hex, hwe⟩

end DomainTheory.ContinuousLattice
```

9

stepMap_wayBelow.lean

```
module
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_le_of_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **The step function is way below `f`.** If `e' ≪ f e`, then `e[e, e'] ≪ f` in `[D
    → D']`,
witnessed by the Scott-open set `{g | e' ≪ g e}` (open because suprema in `[D → D']`
    are pointwise
and `{· ≪ ·}` is inaccessible by directed suprema). -/
theorem stepMap_wayBelow {e : D} {e' : D'} {f : ScottMap D D'}
    (he' : e' ≪ (f : D → D') e) : stepMap e e' ≪ f := by
  refine ⟨{g : ScottMap D D' | e' ≪ (g : D → D') e}, ⟨?_, ?_⟩, he', ?_⟩
  · intro g g' hgg' hg
    exact hg.trans_le (hgg' e)
  · intro F hFne hFdir hmem
    simp only [Set.mem_ofPred_eq, ScottMap.sSup_apply] at hmem
    have hne : (Set.image (fun g : ScottMap D D' => (g : D → D') e) F).Nonempty :=
    hFne.image _
    have hdir : DirectedOn (· ≤ ·) (Set.image (fun g : ScottMap D D' => (g : D → D') e
    ) F) := by
      rintro _ ⟨a, ha, rfl⟩ _ ⟨b, hb, rfl⟩
      obtain ⟨c, hc, hac, hbc⟩ := hFdir a ha b hb
      exact ⟨(c : D → D') e, Set.mem_image_of_mem _ hc, hac e, hbc e⟩
    obtain ⟨_, ⟨g, hgF, rfl⟩, hg⟩ := (WayBelow.sSup_iff hne hdir).1 hmem
    exact ⟨g, hgF, hg⟩
  · intro g hg
    exact Set.mem_Ici.2 (stepMap_le_of_wayBelow hg)

end DomainTheory.ContinuousLattice
```

0

stepMap_wayBelow.lean

```
module
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_le_of_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **The step function is way below `f`.** If `e' ≪ f e`, then `e[e, e'] ≪ f` in `[D
    → D']`,
witnessed by the Scott-open set `{g | e' ≪ g e}` (open because suprema in `[D → D']`
    are pointwise
and `{· ≪ ·}` is inaccessible by directed suprema). -/
theorem stepMap_wayBelow {e : D} {e' : D'} {f : ScottMap D D'}
    (he' : e' ≪ (f : D → D') e) : stepMap e e' ≪ f := by
  refine ⟨{g : ScottMap D D' | e' ≪ (g : D → D') e}, ⟨?_, ?_⟩, he', ?_⟩
  · intro g g' hgg' hg
    exact hg.trans_le (hgg' e)
  · intro F hFne hFdir hmem
    simp only [Set.mem_ofPred_eq, ScottMap.sSup_apply] at hmem
    have hne : (Set.image (fun g : ScottMap D D' => (g : D → D') e) F).Nonempty :=
    hFne.image _
    have hdir : DirectedOn (· ≤ ·) (Set.image (fun g : ScottMap D D' => (g : D → D') e
    ) F) := by
      rintro _ ⟨a, ha, rfl⟩ _ ⟨b, hb, rfl⟩
      obtain ⟨c, hc, hac, hbc⟩ := hFdir a ha b hb
      exact ⟨(c : D → D') e, Set.mem_image_of_mem _ hc, hac e, hbc e⟩
    obtain ⟨_, ⟨g, hgF, rfl⟩, hg⟩ := (WayBelow.sSup_iff hne hdir).1 hmem
    exact ⟨g, hgF, hg⟩
  · intro g hg
    exact Set.mem_Ici.2 (stepMap_le_of_wayBelow hg)

end DomainTheory.ContinuousLattice
```

1

stepMap_wayBelow.lean

```
module
```

```
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_le_of_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **The step function is way below `f`.** If `e' ≪ f e`, then `e[e, e'] ≪ f` in `[D
    → D']`,
witnessed by the Scott-open set `{g | e' ≪ g e}` (open because suprema in `[D → D']`
    are pointwise
and `{· ≪ ·}` is inaccessible by directed suprema). -/
theorem stepMap_wayBelow {e : D} {e' : D'} {f : ScottMap D D'}
    (he' : e' ≪ (f : D → D') e) : stepMap e e' ≪ f := by
  refine ⟨{g : ScottMap D D' | e' ≪ (g : D → D') e}, ⟨?_, ?_⟩, he', ?_⟩
  · intro g g' hgg' hg
    exact hg.trans_le (hgg' e)
  · intro F hFne hFdir hmem
    simp only [Set.mem_ofPred_eq, ScottMap.sSup_apply] at hmem
    have hne : (Set.image (fun g : ScottMap D D' => (g : D → D') e) F).Nonempty :=
    hFne.image _
    have hdir : DirectedOn (· ≤ ·) (Set.image (fun g : ScottMap D D' => (g : D → D') e
    ) F) := by
      rintro _ ⟨a, ha, rfl⟩ _ ⟨b, hb, rfl⟩
      obtain ⟨c, hc, hac, hbc⟩ := hFdir a ha b hb
      exact ⟨(c : D → D') e, Set.mem_image_of_mem _ hc, hac e, hbc e⟩
    obtain ⟨_, ⟨g, hgF, rfl⟩, hg⟩ := (WayBelow.sSup_iff hne hdir).1 hmem
    exact ⟨g, hgF, hg⟩
  · intro g hg
    exact Set.mem_Ici.2 (stepMap_le_of_wayBelow hg)

end DomainTheory.ContinuousLattice
```

2

stepMap_wayBelow.lean

```
module
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_le_of_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
```

```
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **The step function is way below `f`.** If `e' ≪ f e`, then `e[e, e'] ≪ f` in `[D
    → D']`,
witnessed by the Scott-open set `{g | e' ≪ g e}` (open because suprema in `[D → D']`
    are pointwise
and `{· ≪ ·}` is inaccessible by directed suprema). -/
theorem stepMap_wayBelow {e : D} {e' : D'} {f : ScottMap D D'}
    (he' : e' ≪ (f : D → D') e) : stepMap e e' ≪ f := by
  refine ⟨{g : ScottMap D D' | e' ≪ (g : D → D') e}, ⟨?_, ?_⟩, he', ?_⟩
  · intro g g' hgg' hg
    exact hg.trans_le (hgg' e)
  · intro F hFne hFdir hmem
    simp only [Set.mem_ofPred_eq, ScottMap.sSup_apply] at hmem
    have hne : (Set.image (fun g : ScottMap D D' => (g : D → D') e) F).Nonempty :=
    hFne.image _
    have hdir : DirectedOn (· ≤ ·) (Set.image (fun g : ScottMap D D' => (g : D → D') e
    ) F) := by
      rintro _ ⟨a, ha, rfl⟩ _ ⟨b, hb, rfl⟩
      obtain ⟨c, hc, hac, hbc⟩ := hFdir a ha b hb
      exact ⟨(c : D → D') e, Set.mem_image_of_mem _ hc, hac e, hbc e⟩
    obtain ⟨_, ⟨g, hgF, rfl⟩, hg⟩ := (WayBelow.sSup_iff hne hdir).1 hmem
    exact ⟨g, hgF, hg⟩
  · intro g hg
    exact Set.mem_Ici.2 (stepMap_le_of_wayBelow hg)

end DomainTheory.ContinuousLattice
```

3

stepMap_wayBelow.lean

```
module
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_le_of_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section
```

```
namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **The step function is way below `f`.** If `e' ≪ f e`, then `e[e, e'] ≪ f` in `[D
    → D']`,
witnessed by the Scott-open set `{g | e' ≪ g e}` (open because suprema in `[D → D']`
    are pointwise
and `{· ≪ ·}` is inaccessible by directed suprema). -/
theorem stepMap_wayBelow {e : D} {e' : D'} {f : ScottMap D D'}
    (he' : e' ≪ (f : D → D') e) : stepMap e e' ≪ f := by
  refine ⟨{g : ScottMap D D' | e' ≪ (g : D → D') e}, ⟨?_, ?_⟩, he', ?_⟩
  · intro g g' hgg' hg
    exact hg.trans_le (hgg' e)
  · intro F hFne hFdir hmem
    simp only [Set.mem_ofPred_eq, ScottMap.sSup_apply] at hmem
    have hne : (Set.image (fun g : ScottMap D D' => (g : D → D') e) F).Nonempty :=
    hFne.image _
    have hdir : DirectedOn (· ≤ ·) (Set.image (fun g : ScottMap D D' => (g : D → D') e
    ) F) := by
      rintro _ ⟨a, ha, rfl⟩ _ ⟨b, hb, rfl⟩
      obtain ⟨c, hc, hac, hbc⟩ := hFdir a ha b hb
      exact ⟨(c : D → D') e, Set.mem_image_of_mem _ hc, hac e, hbc e⟩
    obtain ⟨_, ⟨g, hgF, rfl⟩, hg⟩ := (WayBelow.sSup_iff hne hdir).1 hmem
    exact ⟨g, hgF, hg⟩
  · intro g hg
    exact Set.mem_Ici.2 (stepMap_le_of_wayBelow hg)

end DomainTheory.ContinuousLattice
```

4

stepMap_wayBelow.lean

```
module
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_le_of_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
```

```
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **The step function is way below `f`.** If `e' ≪ f e`, then `e[e, e'] ≪ f` in `[D
    → D']`,
witnessed by the Scott-open set `{g | e' ≪ g e}` (open because suprema in `[D → D']`
    are pointwise
and `{· ≪ ·}` is inaccessible by directed suprema). -/
theorem stepMap_wayBelow {e : D} {e' : D'} {f : ScottMap D D'}
    (he' : e' ≪ (f : D → D') e) : stepMap e e' ≪ f := by
  refine ⟨{g : ScottMap D D' | e' ≪ (g : D → D') e}, ⟨?_, ?_⟩, he', ?_⟩
  · intro g g' hgg' hg
    exact hg.trans_le (hgg' e)
  · intro F hFne hFdir hmem
    simp only [Set.mem_ofPred_eq, ScottMap.sSup_apply] at hmem
    have hne : (Set.image (fun g : ScottMap D D' => (g : D → D') e) F).Nonempty :=
    hFne.image _
    have hdir : DirectedOn (· ≤ ·) (Set.image (fun g : ScottMap D D' => (g : D → D') e
    ) F) := by
      rintro _ ⟨a, ha, rfl⟩ _ ⟨b, hb, rfl⟩
      obtain ⟨c, hc, hac, hbc⟩ := hFdir a ha b hb
      exact ⟨(c : D → D') e, Set.mem_image_of_mem _ hc, hac e, hbc e⟩
    obtain ⟨_, ⟨g, hgF, rfl⟩, hg⟩ := (WayBelow.sSup_iff hne hdir).1 hmem
    exact ⟨g, hgF, hg⟩
  · intro g hg
    exact Set.mem_Ici.2 (stepMap_le_of_wayBelow hg)

end DomainTheory.ContinuousLattice
```

5

stepMap_wayBelow.lean

```
module
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_le_of_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']
```

```
/-- **The step function is way below `f`.** If `e' ≪ f e`, then `e[e, e'] ≪ f` in `[D
    → D']`,
witnessed by the Scott-open set `{g | e' ≪ g e}` (open because suprema in `[D → D']`
    are pointwise
and `{· ≪ ·}` is inaccessible by directed suprema). -/
theorem stepMap_wayBelow {e : D} {e' : D'} {f : ScottMap D D'}
    (he' : e' ≪ (f : D → D') e) : stepMap e e' ≪ f := by
  refine ⟨{g : ScottMap D D' | e' ≪ (g : D → D') e}, ⟨?_, ?_⟩, he', ?_⟩
  · intro g g' hgg' hg
    exact hg.trans_le (hgg' e)
  · intro F hFne hFdir hmem
    simp only [Set.mem_ofPred_eq, ScottMap.sSup_apply] at hmem
    have hne : (Set.image (fun g : ScottMap D D' => (g : D → D') e) F).Nonempty :=
    hFne.image _
    have hdir : DirectedOn (· ≤ ·) (Set.image (fun g : ScottMap D D' => (g : D → D') e
    ) F) := by
      rintro _ ⟨a, ha, rfl⟩ _ ⟨b, hb, rfl⟩
      obtain ⟨c, hc, hac, hbc⟩ := hFdir a ha b hb
      exact ⟨(c : D → D') e, Set.mem_image_of_mem _ hc, hac e, hbc e⟩
    obtain ⟨_, ⟨g, hgF, rfl⟩, hg⟩ := (WayBelow.sSup_iff hne hdir).1 hmem
    exact ⟨g, hgF, hg⟩
  · intro g hg
    exact Set.mem_Ici.2 (stepMap_le_of_wayBelow hg)

end DomainTheory.ContinuousLattice
```

6

stepMap_wayBelow.lean

```
module
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_le_of_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **The step function is way below `f`.** If `e' ≪ f e`, then `e[e, e'] ≪ f` in `[D
    → D']`,
witnessed by the Scott-open set `{g | e' ≪ g e}` (open because suprema in `[D → D']`
    are pointwise
and `{· ≪ ·}` is inaccessible by directed suprema). -/
```

```
theorem stepMap_wayBelow {e : D} {e' : D'} {f : ScottMap D D'}
    (he' : e' ≪ (f : D → D') e) : stepMap e e' ≪ f := by
  refine ⟨{g : ScottMap D D' | e' ≪ (g : D → D') e}, ⟨?_, ?_⟩, he', ?_⟩
  · intro g g' hgg' hg
    exact hg.trans_le (hgg' e)
  · intro F hFne hFdir hmem
    simp only [Set.mem_ofPred_eq, ScottMap.sSup_apply] at hmem
    have hne : (Set.image (fun g : ScottMap D D' => (g : D → D') e) F).Nonempty :=
    hFne.image _
    have hdir : DirectedOn (· ≤ ·) (Set.image (fun g : ScottMap D D' => (g : D → D') e
    ) F) := by
      rintro _ ⟨a, ha, rfl⟩ _ ⟨b, hb, rfl⟩
      obtain ⟨c, hc, hac, hbc⟩ := hFdir a ha b hb
      exact ⟨(c : D → D') e, Set.mem_image_of_mem _ hc, hac e, hbc e⟩
    obtain ⟨_, ⟨g, hgF, rfl⟩, hg⟩ := (WayBelow.sSup_iff hne hdir).1 hmem
    exact ⟨g, hgF, hg⟩
  · intro g hg
    exact Set.mem_Ici.2 (stepMap_le_of_wayBelow hg)

end DomainTheory.ContinuousLattice
```

7

stepMap_wayBelow.lean

```
module
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_le_of_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **The step function is way below `f`.** If `e' ≪ f e`, then `e[e, e'] ≪ f` in `[D
    → D']`,
witnessed by the Scott-open set `{g | e' ≪ g e}` (open because suprema in `[D → D']`
    are pointwise
and `{· ≪ ·}` is inaccessible by directed suprema). -/
theorem stepMap_wayBelow {e : D} {e' : D'} {f : ScottMap D D'}
    (he' : e' ≪ (f : D → D') e) : stepMap e e' ≪ f := by
  refine ⟨{g : ScottMap D D' | e' ≪ (g : D → D') e}, ⟨?_, ?_⟩, he', ?_⟩
  · intro g g' hgg' hg
    exact hg.trans_le (hgg' e)
```

```
  · intro F hFne hFdir hmem
    simp only [Set.mem_ofPred_eq, ScottMap.sSup_apply] at hmem
    have hne : (Set.image (fun g : ScottMap D D' => (g : D → D') e) F).Nonempty :=
    hFne.image _
    have hdir : DirectedOn (· ≤ ·) (Set.image (fun g : ScottMap D D' => (g : D → D') e
    ) F) := by
      rintro _ ⟨a, ha, rfl⟩ _ ⟨b, hb, rfl⟩
      obtain ⟨c, hc, hac, hbc⟩ := hFdir a ha b hb
      exact ⟨((c : D → D') e, Set.mem_image_of_mem _ hc, hac e, hbc e⟩
    obtain ⟨_, ⟨g, hgF, rfl⟩, hg⟩ := (WayBelow.sSup_iff hne hdir).1 hmem
    exact ⟨g, hgF, hg⟩
  · intro g hg
    exact Set.mem_Ici.2 (stepMap_le_of_wayBelow hg)

end DomainTheory.ContinuousLattice
```

8

stepMap_wayBelow.lean

```
module
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_le_of_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **The step function is way below `f`.** If `e' ≪ f e`, then `e[e, e'] ≪ f` in `[D
    → D']`,
witnessed by the Scott-open set `{g | e' ≪ g e}` (open because suprema in `[D → D']`
    are pointwise
and `{· ≪ ·}` is inaccessible by directed suprema). -/
theorem stepMap_wayBelow {e : D} {e' : D'} {f : ScottMap D D'}
    (he' : e' ≪ (f : D → D') e) : stepMap e e' ≪ f := by
  refine ⟨{g : ScottMap D D' | e' ≪ (g : D → D') e}, ⟨?_, ?_⟩, he', ?_⟩
  · intro g g' hgg' hg
    exact hg.trans_le (hgg' e)
  · intro F hFne hFdir hmem
    simp only [Set.mem_ofPred_eq, ScottMap.sSup_apply] at hmem
    have hne : (Set.image (fun g : ScottMap D D' => (g : D → D') e) F).Nonempty :=
    hFne.image _
    have hdir : DirectedOn (· ≤ ·) (Set.image (fun g : ScottMap D D' => (g : D → D') e
```

```
    ) F) := by
      rintro _ ⟨a, ha, rfl⟩ _ ⟨b, hb, rfl⟩
      obtain ⟨c, hc, hac, hbc⟩ := hFdir a ha b hb
      exact ⟨(c : D → D') e, Set.mem_image_of_mem _ hc, hac e, hbc e⟩
    obtain ⟨_, ⟨g, hgF, rfl⟩, hg⟩ := (WayBelow.sSup_iff hne hdir).1 hmem
    exact ⟨g, hgF, hg⟩
  · intro g hg
    exact Set.mem_Ici.2 (stepMap_le_of_wayBelow hg)

end DomainTheory.ContinuousLattice
```

9

sup_apply.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

/-- The (completeLatticeOfSup-derived) binary join of Scott maps is computed pointwise
    . -/
theorem sup_apply (f g : ScottMap D D') (x : D) :
    ((f ⊔ g : ScottMap D D') : D → D') x = f x ⊔ g x := by
  have h : (f ⊔ g : ScottMap D D') = sSup ({f, g} : Set (ScottMap D D')) := rfl
  rw [h, sSup_apply, Set.image_pair]
  exact sSup_pair

end ScottMap

end DomainTheory.ContinuousLattice
```

0

sup_apply.lean

```
module
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

/-- The (completeLatticeOfSup-derived) binary join of Scott maps is computed pointwise
    . -/
theorem sup_apply (f g : ScottMap D D') (x : D) :
    ((f ⊔ g : ScottMap D D') : D → D') x = f x ⊔ g x := by
  have h : (f ⊔ g : ScottMap D D') = sSup ({f, g} : Set (ScottMap D D')) := rfl
  rw [h, sSup_apply, Set.image_pair]
  exact sSup_pair

end ScottMap

end DomainTheory.ContinuousLattice
```

1

sup_apply.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
```

```
    CompleteLattice D₁']

namespace ScottMap

/-- The (completeLatticeOfSup-derived) binary join of Scott maps is computed pointwise
    . -/
theorem sup_apply (f g : ScottMap D D') (x : D) :
    ((f ⊔ g : ScottMap D D') : D → D') x = f x ⊔ g x := by
  have h : (f ⊔ g : ScottMap D D') = sSup ({f, g} : Set (ScottMap D D')) := rfl
  rw [h, sSup_apply, Set.image_pair]
  exact sSup_pair

end ScottMap

end DomainTheory.ContinuousLattice
```

2

sup_apply.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

/-- The (completeLatticeOfSup-derived) binary join of Scott maps is computed pointwise
    . -/
theorem sup_apply (f g : ScottMap D D') (x : D) :
    ((f ⊔ g : ScottMap D D') : D → D') x = f x ⊔ g x := by
  have h : (f ⊔ g : ScottMap D D') = sSup ({f, g} : Set (ScottMap D D')) := rfl
  rw [h, sSup_apply, Set.image_pair]
  exact sSup_pair

end ScottMap

end DomainTheory.ContinuousLattice
```

3

sup_apply.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

/-- The (completeLatticeOfSup-derived) binary join of Scott maps is computed pointwise
    . -/
theorem sup_apply (f g : ScottMap D D') (x : D) :
    ((f ⊔ g : ScottMap D D') : D → D') x = f x ⊔ g x := by
  have h : (f ⊔ g : ScottMap D D') = sSup ({f, g} : Set (ScottMap D D')) := rfl
  rw [h, sSup_apply, Set.image_pair]
  exact sSup_pair

end ScottMap

end DomainTheory.ContinuousLattice
```

4

sup_apply.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]
```

```
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

/-- The (completeLatticeOfSup-derived) binary join of Scott maps is computed pointwise
    . -/
theorem sup_apply (f g : ScottMap D D') (x : D) :
    ((f ⊔ g : ScottMap D D') : D → D') x = f x ⊔ g x := by
  have h : (f ⊔ g : ScottMap D D') = sSup ({f, g} : Set (ScottMap D D')) := rfl
  rw [h, sSup_apply, Set.image_pair]
  exact sSup_pair

end ScottMap

end DomainTheory.ContinuousLattice
```

5

4.4 Domains as lattices and as topological spaces

Dependencies. Inside DomainsAsLatticesAndAsTopologicalSpaces, each box is one subfolder, or one Lean file sitting directly in the folder. A box named as another folder in this chapter is that whole folder. A box named as another chapter is that whole chapter. The arrowhead points at the file or folder that imports, and the number is how many direct imports.



Figure 39: Domains as lattices and as topological spaces.

I will attempt to keep this section as informal as possible—especially as Sections 2 and 3 were quite formal (i.e. mathematical) enough. The main purpose of the discussion of this section is to relate the new presentation of the Theory of Domains to previously published ones. (Some alternatives are discussed at the end of the paper.)

Lattice-theoretic considerations. Let $\mathbf{A}$ be an information system. Because the elements of $|\mathbf{A}|$ are introduced as sets, these elements can be given structure from what we know already about ordinary sets. For instance, the set-theoretic inclusion relation between sets can certainly be applied to elements. The question is: What does $x \subseteq y$ mean intuitively? In fact, we have already mentioned this relationship before. It means that every proposition (among the ones given by the information system) true of x is also true of y (though not necessarily conversely). In other words, x is perhaps only partially determined, while y is more fully determined in a way that includes everything true of x. Clearly, then, if we grant the interest of partial elements, then this relation is a natural and basic one. We often read "$x \subseteq y$" as "x approximates y".

Now here is one of the main points of the new exposition: Because $x \subseteq y$ is *defined* in terms of a familiar mathematical notion, then as a relation between elements it inherits all the well-known properties of the set-theoretical relation. It should not be necessary to write them out, for everyone who can read this

far knows that $\subseteq$ is in particular reflexive and transitive. We say that the domain $|\mathbf{A}|$ is a *partially ordered set under inclusion*. We note that this (trivial) result is *proved* from the definition rather than assumed as an axiom.

Let us try out another notion. Elements are consistent, deductively closed sets of data objects. Right? Every subset of a consistent set is consistent, but not every subset is closed under entailment, in general. Right? But suppose that x and y are two elements. It can easily be seen that their intersection $x \cap y$ is again consistent and deductively closed (by a 2 nanosecond proof). Therefore, the set of elements of a domain is closed under intersection. This means that, as a partially ordered set, $|\mathbf{A}|$ is an *inf semilattice*.

Well, the exact terminology is quite unimportant, but the properties of $\cap$ when combined with the properties of $\subseteq$ are pleasantly "algebraic" (and run to several lines when written out). We all know them: $\cap$ is idempotent, commutative, and associative. The element $\bot$, being the The element $\bot$, being the smallest element of the domain, is a zero element for $\cap$. The operation $\cap$ is monotonic with respect to $\subseteq$, and we have the connection:

$$x \subseteq y \text{ iff } x \cap y = x.$$

What has just been alluded to in words is the standard mathematical axiomatization of the properties of an infimum (or meet) operation in a partially ordered set. (Remember, however, to have an operation well defined within a set it is necessary also to have closure under that operation.)

What about infs of subsets? No problem. For any *non-empty* subfamily of $|\mathbf{A}|$, it is just as easy to prove that the set-theoretical intersection of all the elements in the subfamily is again an element of the domain. This makes $|\mathbf{A}|$ a (conditionally) complete inf semilattice. Even if you do not have any idea what I am talking about, it does not matter because the properties of the domain are built in by the very definition; you do not have to worry *at the start* about their formulation, for you can prove them when you need them (if ever).

What about suprema (or sups or joins)? Here there is a slight problem, because in the way we set things up they *do not* always have to exist. Consider what happens with just two elements x and y. The set-theoretical union of the two elements as subsets of $D_{\mathbf{A}}$ need be neither consistent nor deductively closed; and even if $x \cup y$ is consistent, it need not be closed under entailment. So if we want sups *within* $|\mathbf{A}|$, we cannot get by as cheaply as we can with a domain of arbitrary sets for which the simple union is the answer.

Let us write the sup (supposing it exists in $|\mathbf{A}|$) as $x \sqcup y$. What do we have when we actually have it? By the lore of partially ordered sets, $x \sqcup y$ must be the *least element* in $|\mathbf{A}|$ which includes (in the sense of $\subseteq$) both x and y. Now we have just spoken about infs of families of elements: they exist if the family is non-empty. This indicates that $x \sqcup y$ exists exactly when there is at least one element z in $|\mathbf{A}|$ such that $x \subseteq z$ and $y \subseteq z$. This turns out to be a way to say—entirely inside $|\mathbf{A}|$—that $x \cup y$ is consistent. In other words (and more generally) a sup of a subfamily exists if, and only if, the union of the family is consistent. (And in that case the sup is just the deductive closure of the union—that is, it is generally larger than the simple set-theoretical union.) This has an axiomatic version, but it is slightly complicated by the fact that sups do not always exist. In case $\top_{\mathbf{A}}$ exists, then we always have all the sups, and $|\mathbf{A}|$ is a *complete lattice*. (This is discussed in full axiomatic splendor in Gierz, et al. [1980].) But, let it hasten to be added, not every complete lattice is isomorphic to a domain. The lattices that correspond to domains are the so-called *algebraic lattices*. (See the discussion loc. cit. for the additional axiomatics required.) When the top is missing, the partially ordered structures corresponding to domains are called *conditionally complete, algebraic cpo's*.

There is, however, an important case in $|\mathbf{A}|$ where the union is consistent and is, in fact, the sup in the domain. Suppose we have a sequence of elements such that

$$x_0 \subseteq x_1 \subseteq x_2 \subseteq \cdots \subseteq x_n \subseteq x_{n+1} \subseteq \cdots$$

As n increases, we can say that the elements x_n are getting "better and better"; thus, they must be approaching something even more desirable *in the limit*. Let

$$y = \bigcup_{n=0}^{\infty} x_n,$$

then there is no question that y is a subset of $D_{\mathbf{A}}$. But is it an element? Consistency, recall, means that every *finite* subset is in $\mathrm{Con}_{\mathbf{A}}$. The trick is that a finite subset of y must be a subset of one of the x_n, because the sequence of elements x_n is increasing. But all the terms of our given sequence are elements, and so, consistent; therefore, every finite subset of y is consistent.

The same argument also shows that the set y of data objects is closed under entailment, because $\vdash_{\mathbf{A}}$ is defined as a relation between finite (consistent) sets and data objects. It follows that since each x_n is a set closed under $\vdash_{\mathbf{A}}$, then so is y. In other words, y is indeed an element. Otherwise said, the domain $|\mathbf{A}|$ is closed under the formation of unions of increasing chains of elements, and the union is, of course, the sup in the sense of the partial ordering. Again we have proved closure rather than assuming it, and the necessary properties of the sup follow from its definition as a union.

This last argument can be generalized—as is well known to mathematicians—to the case of *directed* families of elements. The word "directed," or better "upward-directed," means that every finite number of elements in the family is included in some further element of the family; chains always have this property. A good example of the use of directed sets of elements figures in the discussion of the *finite elements* of the domain.

As we have remarked several times, any consistent set of data objects *generates* an element by closing it up under the entailment relation; thus, in particular, any set $u \in \mathrm{Con}$ generates an element. Let us write

$$\overline{u} = \{X \in D_{\mathbf{A}} \mid u \vdash_{\mathbf{A}} X\}$$

for the *closure* of u under entailment. (This notation could be used for any consistent subset of $D_{\mathbf{A}}$, but the case of finitely generated elements is especially important.) Such a $\overline{u}$ is always an element of $|\mathbf{A}|$; the totality of such elements form, by definition, the *finite elements* of the domain. The notation $\overline{u}$ should of course be decorated with a subscript $\mathbf{A}$, but, as there is no good place to write it in, we leave it out.

We have for all elements x of the domain the basic formula:

$$x = \bigcup \{\overline{u} \mid u \in \mathrm{Con}_{\mathbf{A}} \text{ and } u \subseteq x\}.$$

This can be read intuitively as saying that every element of the domain is the *limit* of its *finite approximations*. And, having used the word "limit" so often, we ought now to say something about how to make the meaning of this word correct mathematically. Note that in the limit representation for x, the union is a directed one. (Why?)

Topological considerations. Geometrically speaking, a topological space is a collection of "points" which group themselves in various "neighborhoods" providing thereby a sense of "nearness." More specifically, a neighborhood of a point in a *metric space* consists of all those points within a certain positive "distance" from the given point; that is, nearness is limited, but it is not pushed to *zero*. The major reason for leaving some "breathing space" around the point comes out in defining continuity of functions.

A function f from one topological space to another is said to be *continuous* provided that, for every point x of the first space, and for every neighborhood in the second space around $f(x)$, it is possible to find—in

the first space—a neighborhood around x with the following property. If the function is restricted to this neighborhood, then its values lie entirely in the given neighborhood around $f(x)$. In the metric case we can say that if we do not want the function to wander any great distance from $f(x)$, then there is a restricted distance around x giving a neighborhood over which the function values stay as close to $f(x)$ as specified. This keeps the function from jumping around wildly, since small variations in x lead to only small variations in $f(x)$.

The intuition about continuity is no doubt very clear in the more geometric examples, and geometric intuition can carry us quite far. But the spaces obtained from domain theory are not really geometric spaces, so we need to make some shifts in our metaphors. The geometric notion of a point usually implies that two points are perfectly distinguishable: if x and y are different, then they have disjoint neighborhoods. This is certainly true in the metric case, where, if two points are different, then they are at a positive distance apart. Early on in the study of topological spaces, however, it was found that not all spaces were metric, and weaker separation properties of pairs of distinct points were uncovered. One of the very weak versions of such a condition is called the "T_0-axiom." The best way to put it is that if two points have the same neighborhoods, then they are the same. The contrapositive statement sounds odd: if two points are distinct, then there is a neighborhood that contains one but not the other. The oddity of this statement lies in the feeling that you cannot make up your mind over which is the better point.

In our domains there is already a notion of "betterness" which proves to be very closely related to the implicit neighborhood structure. The reason why our domains, as spaces of points, are not "geometric" in the familiar sense is that they contain partial elements. A geometric point, on the other hand, is "perfect" or totally determined; it cannot be made better than it already is. The metric in common spaces is measuring something other than betterness, for there are competing notions of approximation afoot here. Thus, we must keep our special goals in defining domains clearly in mind to avoid confusion.

Consider an information system $\mathbf{A}$. For each $u \in \mathrm{Con}_{\mathbf{A}}$, we define a corresponding neighborhood of $|\mathbf{A}|$ by the equation:

$$[u]_{\mathbf{A}} = \{x \in |\mathbf{A}| \mid u \subseteq x\}.$$

The neighborhoods of an element x are all those sets $[u]_{\mathbf{A}}$, where $u \subseteq x$. Note that this is the same as saying $\overline{u} \subseteq x$ or $\overline{u}$ approximates x. Indeed, the neighborhoods are in a one-one correspondence with the finite elements of $|\mathbf{A}|$. A neighborhood collects together all those elements sharing the same finite amount of information.

If both $u \subseteq x$ and $v \subseteq x$, then it is easy to see that the intersection of the two neighborhoods is again a neighborhood of x by virtue of the formula:

$$[u]_{\mathbf{A}} \cap [v]_{\mathbf{A}} = [u \cup v]_{\mathbf{A}}.$$

In case u and v are inconsistent with each other, then the intersection will be the empty set $\emptyset$. Because every element of our domain $|\mathbf{A}|$ is the union of the finite elements it contains, it is trivial to prove that two elements with exactly the same neighborhoods are the same. For these reasons, then, it follows that $|\mathbf{A}|$ is a T_0-topological space. The immediate reaction to this intelligence is: So what?

The answer to this scepticism will appear in the best light when we discuss in the next section the notion of function or mapping appropriate to our domains. The argument will be based on the extremely elementary form of the definition that characterizes the general continuous function together with the fact that the geometric intuition is suggestive of theorems to prove. In Section 6 we will find that the space of all continuous functions between two given domains is also a domain, a most useful result that is interesting

topologically. But further interest for Domain Theory comes from the circumstance that there is a connection between geometry and domains. We cannot go into the full details here, but I can make some brief remarks.

Before continuing that discussion, though, it should be remarked that once the topology of the domain has been uncovered, then the unions of chains of elements (or directed sets of elements) are topological limits according to accepted general definitions. Thus, what we felt intuitively was a limiting process (the getting better and better up to the union) is in fact a limit formation in a precise mathematical sense. This helps convince us that we are on the right track.

Total elements as a space. In any domain it can be proved that any element x can be extended to a *total element* t, which is characterized by the property that it is a *maximal* element of the partial ordering. (Perhaps there are many such t, but there is always at least one with $x \subseteq t$.) Recall that the set of all total elements of $|\mathbf{A}|$ is denoted by $\mathrm{Tot}_{\mathbf{A}}$. Because the set $\mathrm{Tot}_{\mathbf{A}} \subseteq |\mathbf{A}|$, it is a topological space itself. Suppose s and t are two distinct total elements, then they must be inconsistent with each other—but we shall prove more. If every finite consistent $u \subseteq s$ also satisfied $u \subseteq t$, then $s \subseteq t$ would follow. But this is impossible, because s is maximal. Let, then, $u \in \mathrm{Con}_{\mathbf{A}}$ with $u \subseteq s$ be chosen so that $u \subseteq t$ fails. Now u must be inconsistent with t, since otherwise there would be an element containing them both. It follows that u must be inconsistent with some $v \in \mathrm{Con}_{\mathbf{A}}$ with $v \subseteq t$. In this way we show that the neighborhoods $[u]_{\mathbf{A}}$ and $[v]_{\mathbf{A}}$ are disjoint.

This argument proves that $\mathrm{Tot}_{\mathbf{A}}$ is a *Hausdorff space* (or T_2-space). We cannot go further into the details here, but it is also a totally disconnected, compact Hausdorff space. (These spaces—well known from studies of Logic and the Theory of Boolean Algebras—are also zero-dimensional.) Assuming the countability of $\mathrm{Con}_{\mathbf{A}}$, which is not such a bold assumption, all such spaces can be conveniently embedded into the real line so as to preserve the topological structure. Perhaps this does not seem so surprising, but it shows that the topological nature of $\mathrm{Tot}_{\mathbf{A}}$, the uppermost level of the T_0 space $|\mathbf{A}|$, is indeed quite geometric. And it can also be shown that any arbitrary continuous function on $\mathrm{Tot}_{\mathbf{A}}$ into *any* topological space can be extended to a continuous function on $|\mathbf{A}|$, so the whole domain proves to be quite a "roomy" space with many good connections to more usual topological spaces.

A note on neighborhood structures. Topological spaces can be defined in several ways, and in general they do not carry a preferred neighborhood structure. (For example, there is no topological significance to the usual rational intervals we use to define neighborhoods for the real line: any similarly dense set of points would do in place of the rational numbers.) Domains, the way we have defined them, however, do have preferred neighborhoods. The set of finite elements of $|\mathbf{A}|$ can be defined topologically, and as we have seen they determine a convenient set of neighborhoods. (The reason for all this is the so-called zero-dimensional character of the domains. There is a higher-dimensional theory, but this leads to continuous lattices and can only be explained using the kind of presentation in Gierz, *et al.* [1980]. It is too long-winded a story for the present paper. The higher-dimensional domains do not have a preferred set of neighborhoods, however.)

Having realized this, it is a short step to giving an algebraic axiomatization of the neighborhood structure. Smyth [1975] contains a presentation in terms of the equivalent notion of

sup_apply.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section
```

```
namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

/-- The (completeLatticeOfSup-derived) binary join of Scott maps is computed pointwise
    . -/
theorem sup_apply (f g : ScottMap D D') (x : D) :
    ((f ⊔ g : ScottMap D D') : D → D') x = f x ⊔ g x := by
  have h : (f ⊔ g : ScottMap D D') = sSup ({f, g} : Set (ScottMap D D')) := rfl
  rw [h, sSup_apply, Set.image_pair]
  exact sSup_pair

end ScottMap

end DomainTheory.ContinuousLattice
```

6

sup_apply.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

/-- The (completeLatticeOfSup-derived) binary join of Scott maps is computed pointwise
    . -/
```

```
theorem sup_apply (f g : ScottMap D D') (x : D) :
    ((f ⊔ g : ScottMap D D') : D → D') x = f x ⊔ g x := by
  have h : (f ⊔ g : ScottMap D D') = sSup ({f, g} : Set (ScottMap D D')) := rfl
  rw [h, sSup_apply, Set.image_pair]
  exact sSup_pair

end ScottMap

end DomainTheory.ContinuousLattice
```

7

sup_apply.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

/-- The (completeLatticeOfSup-derived) binary join of Scott maps is computed pointwise
    . -/
theorem sup_apply (f g : ScottMap D D') (x : D) :
    ((f ⊔ g : ScottMap D D') : D → D') x = f x ⊔ g x := by
  have h : (f ⊔ g : ScottMap D D') = sSup ({f, g} : Set (ScottMap D D')) := rfl
  rw [h, sSup_apply, Set.image_pair]
  exact sSup_pair

end ScottMap

end DomainTheory.ContinuousLattice
```

8

sup_apply.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
```

```
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace ScottMap

/-- The (completeLatticeOfSup-derived) binary join of Scott maps is computed pointwise
    . -/
theorem sup_apply (f g : ScottMap D D') (x : D) :
    ((f ⊔ g : ScottMap D D') : D → D') x = f x ⊔ g x := by
  have h : (f ⊔ g : ScottMap D D') = sSup ({f, g} : Set (ScottMap D D')) := rfl
  rw [h, sSup_apply, Set.image_pair]
  exact sSup_pair

end ScottMap

end DomainTheory.ContinuousLattice
```

9

theorem_3_3.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottMapPointwiseTopology
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import Mathlib.Topology.Defs.Basic

public import DomainTheory.ContinuousLattice.FunctionSpaces.theorem_3_3_more

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']
```

```
/-- **Scott 1972, Theorem 3.3 (full statement).** For continuous lattices `D`, `D'`,
    the function
space `[D → D']` is a continuous lattice (`theorem_3_3.isContinuousLattice`) whose
    Scott topology
agrees with the topology of pointwise convergence (`theorem_3_3.topology`). -/
theorem theorem_3_3 (hD : ContinuousLattice D) (hD' : ContinuousLattice D') :
    ContinuousLattice (ScottMap D D') ∧
      (scottTopologicalSpace : TopologicalSpace (ScottMap D D')) =
    scottMapPointwiseTopology D D' :=
  ⟨theorem_3_3.isContinuousLattice hD hD', theorem_3_3.topology hD hD'⟩

end DomainTheory.ContinuousLattice
```

0

theorem_3_3.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottMapPointwiseTopology
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import Mathlib.Topology.Defs.Basic

public import DomainTheory.ContinuousLattice.FunctionSpaces.theorem_3_3_more

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Theorem 3.3 (full statement).** For continuous lattices `D`, `D'`,
    the function
space `[D → D']` is a continuous lattice (`theorem_3_3.isContinuousLattice`) whose
    Scott topology
agrees with the topology of pointwise convergence (`theorem_3_3.topology`). -/
theorem theorem_3_3 (hD : ContinuousLattice D) (hD' : ContinuousLattice D') :
    ContinuousLattice (ScottMap D D') ∧
      (scottTopologicalSpace : TopologicalSpace (ScottMap D D')) =
    scottMapPointwiseTopology D D' :=
  ⟨theorem_3_3.isContinuousLattice hD hD', theorem_3_3.topology hD hD'⟩

end DomainTheory.ContinuousLattice
```

1

theorem_3_3.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottMapPointwiseTopology
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import Mathlib.Topology.Defs.Basic

public import DomainTheory.ContinuousLattice.FunctionSpaces.theorem_3_3_more

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Theorem 3.3 (full statement).** For continuous lattices `D`, `D'`,
    the function
space `[D → D']` is a continuous lattice (`theorem_3_3.isContinuousLattice`) whose
    Scott topology
agrees with the topology of pointwise convergence (`theorem_3_3.topology`). -/
theorem theorem_3_3 (hD : ContinuousLattice D) (hD' : ContinuousLattice D') :
    ContinuousLattice (ScottMap D D') ∧
      (scottTopologicalSpace : TopologicalSpace (ScottMap D D')) =
    scottMapPointwiseTopology D D' :=
  ⟨theorem_3_3.isContinuousLattice hD hD', theorem_3_3.topology hD hD'⟩

end DomainTheory.ContinuousLattice
```

2

theorem_3_3.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottMapPointwiseTopology
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import Mathlib.Topology.Defs.Basic

public import DomainTheory.ContinuousLattice.FunctionSpaces.theorem_3_3_more

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
```

```
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Theorem 3.3 (full statement).** For continuous lattices `D`, `D'`,
    the function
space `[D → D']` is a continuous lattice (`theorem_3_3.isContinuousLattice`) whose
    Scott topology
agrees with the topology of pointwise convergence (`theorem_3_3.topology`). -/
theorem theorem_3_3 (hD : ContinuousLattice D) (hD' : ContinuousLattice D') :
    ContinuousLattice (ScottMap D D') ∧
      (scottTopologicalSpace : TopologicalSpace (ScottMap D D')) =
    scottMapPointwiseTopology D D' :=
  ⟨theorem_3_3.isContinuousLattice hD hD', theorem_3_3.topology hD hD'⟩

end DomainTheory.ContinuousLattice
```

3

theorem_3_3.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottMapPointwiseTopology
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import Mathlib.Topology.Defs.Basic

public import DomainTheory.ContinuousLattice.FunctionSpaces.theorem_3_3_more

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Theorem 3.3 (full statement).** For continuous lattices `D`, `D'`,
    the function
space `[D → D']` is a continuous lattice (`theorem_3_3.isContinuousLattice`) whose
    Scott topology
agrees with the topology of pointwise convergence (`theorem_3_3.topology`). -/
theorem theorem_3_3 (hD : ContinuousLattice D) (hD' : ContinuousLattice D') :
    ContinuousLattice (ScottMap D D') ∧
      (scottTopologicalSpace : TopologicalSpace (ScottMap D D')) =
    scottMapPointwiseTopology D D' :=
  ⟨theorem_3_3.isContinuousLattice hD hD', theorem_3_3.topology hD hD'⟩

end DomainTheory.ContinuousLattice
```

4

theorem_3_3.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottMapPointwiseTopology
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import Mathlib.Topology.Defs.Basic

public import DomainTheory.ContinuousLattice.FunctionSpaces.theorem_3_3_more

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Theorem 3.3 (full statement).** For continuous lattices `D`, `D'`,
    the function
space `[D → D']` is a continuous lattice (`theorem_3_3.isContinuousLattice`) whose
    Scott topology
agrees with the topology of pointwise convergence (`theorem_3_3.topology`). -/
theorem theorem_3_3 (hD : ContinuousLattice D) (hD' : ContinuousLattice D') :
    ContinuousLattice (ScottMap D D') ∧
      (scottTopologicalSpace : TopologicalSpace (ScottMap D D')) =
    scottMapPointwiseTopology D D' :=
  ⟨theorem_3_3.isContinuousLattice hD hD', theorem_3_3.topology hD hD'⟩

end DomainTheory.ContinuousLattice
```

5

theorem_3_3.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottMapPointwiseTopology
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import Mathlib.Topology.Defs.Basic

public import DomainTheory.ContinuousLattice.FunctionSpaces.theorem_3_3_more

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
```

```
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Theorem 3.3 (full statement).** For continuous lattices `D`, `D'`,
    the function
space `[D → D']` is a continuous lattice (`theorem_3_3.isContinuousLattice`) whose
    Scott topology
agrees with the topology of pointwise convergence (`theorem_3_3.topology`). -/
theorem theorem_3_3 (hD : ContinuousLattice D) (hD' : ContinuousLattice D') :
    ContinuousLattice (ScottMap D D') ∧
      (scottTopologicalSpace : TopologicalSpace (ScottMap D D')) =
    scottMapPointwiseTopology D D' :=
  ⟨theorem_3_3.isContinuousLattice hD hD', theorem_3_3.topology hD hD'⟩

end DomainTheory.ContinuousLattice
```

6

theorem_3_3.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottMapPointwiseTopology
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import Mathlib.Topology.Defs.Basic

public import DomainTheory.ContinuousLattice.FunctionSpaces.theorem_3_3_more

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Theorem 3.3 (full statement).** For continuous lattices `D`, `D'`,
    the function
space `[D → D']` is a continuous lattice (`theorem_3_3.isContinuousLattice`) whose
    Scott topology
agrees with the topology of pointwise convergence (`theorem_3_3.topology`). -/
theorem theorem_3_3 (hD : ContinuousLattice D) (hD' : ContinuousLattice D') :
    ContinuousLattice (ScottMap D D') ∧
      (scottTopologicalSpace : TopologicalSpace (ScottMap D D')) =
    scottMapPointwiseTopology D D' :=
  ⟨theorem_3_3.isContinuousLattice hD hD', theorem_3_3.topology hD hD'⟩

end DomainTheory.ContinuousLattice
```

7

theorem_3_3.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottMapPointwiseTopology
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import Mathlib.Topology.Defs.Basic

public import DomainTheory.ContinuousLattice.FunctionSpaces.theorem_3_3_more

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Theorem 3.3 (full statement).** For continuous lattices `D`, `D'`,
    the function
space `[D → D']` is a continuous lattice (`theorem_3_3.isContinuousLattice`) whose
    Scott topology
agrees with the topology of pointwise convergence (`theorem_3_3.topology`). -/
theorem theorem_3_3 (hD : ContinuousLattice D) (hD' : ContinuousLattice D') :
    ContinuousLattice (ScottMap D D') ∧
      (scottTopologicalSpace : TopologicalSpace (ScottMap D D')) =
    scottMapPointwiseTopology D D' :=
  ⟨theorem_3_3.isContinuousLattice hD hD', theorem_3_3.topology hD hD'⟩

end DomainTheory.ContinuousLattice
```

8

theorem_3_3.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottMapPointwiseTopology
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import Mathlib.Topology.Defs.Basic

public import DomainTheory.ContinuousLattice.FunctionSpaces.theorem_3_3_more

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace
```

```
variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Theorem 3.3 (full statement).** For continuous lattices `D`, `D'`,
    the function
space `[D → D']` is a continuous lattice (`theorem_3_3.isContinuousLattice`) whose
    Scott topology
agrees with the topology of pointwise convergence (`theorem_3_3.topology`). -/
theorem theorem_3_3 (hD : ContinuousLattice D) (hD' : ContinuousLattice D') :
    ContinuousLattice (ScottMap D D') ∧
      (scottTopologicalSpace : TopologicalSpace (ScottMap D D')) =
    scottMapPointwiseTopology D D' :=
  ⟨theorem_3_3.isContinuousLattice hD hD', theorem_3_3.topology hD hD'⟩

end DomainTheory.ContinuousLattice
```

9

theorem_3_3_more.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_pointwise_sSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_apply_of_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    wayBelow_le_finset_sup_step
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    pointwiseSubbasic_scottOpen
public import DomainTheory.ContinuousLattice.FunctionSpaces.pointwiseSubbasic_isOpen
public import DomainTheory.ContinuousLattice.FunctionSpaces.wayBelow_finset_sup
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottMapPointwiseTopology
public import DomainTheory.ContinuousLattice.WayBelow.ScottOpen
public import DomainTheory.ContinuousLattice.WayBelow.exists_wayBelow_subset
public import DomainTheory.ContinuousLattice.WayBelow.scottOpen_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottMapPointwiseTopology_eq_induced_pi
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottMapInducedPiTopology

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
```

```
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Theorem 3.3 (order content).** If `D` and `D'` are continuous
    lattices, then so
is `[D → D']` under the pointwise order. Every `f` is the supremum of the step
    functions way below
it: `f = ⊔ {e[e,e'] | e' ≪ f e}`, and each such step function is way below `f`. -/
theorem theorem_3_3.isContinuousLattice (hD : ContinuousLattice D)
    (hD' : ContinuousLattice D') : ContinuousLattice (ScottMap D D') := by
  refine { out := ?_ }
  intro f
  refine ⟨fun h hh => hh.le, fun g hg => ?_⟩
  rw [ScottMap.le_def]
  intro x
  rw [← stepMap_pointwise_sSup hD hD' f x]
  refine sSup_le ?_
  rintro e' ⟨e, hex, he'⟩
  have hle : stepMap e e' ≤ g := hg (stepMap_wayBelow he')
  have hx := (ScottMap.le_def.1 hle) x
  rwa [stepMap_apply_of_wayBelow hex] at hx

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Theorem 3.3(b).** The Scott (lattice) topology on `[D → D']` agrees
    with the
topology of pointwise convergence. -/
theorem theorem_3_3.topology (hD : ContinuousLattice D) (hD' : ContinuousLattice D') :
    (scottTopologicalSpace : TopologicalSpace (ScottMap D D')) =
    scottMapPointwiseTopology D D' := by
  have hL : ContinuousLattice (ScottMap D D') := theorem_3_3.isContinuousLattice hD hD
    '
  apply le_antisymm
  · -- pointwise ⊆ Scott: each subbasic set is Scott-open in the lattice
    apply le_generateFrom_iff_subset_isOpen.2
    rintro W ⟨x, U, hUopen, rfl⟩
    exact ScottOpen.isOpen_iff.mpr (pointwiseSubbasic_scottOpen x (ScottOpen.
    isOpen_iff.mp hUopen))
  · -- Scott ⊆ pointwise: each Scott-open lattice set is pointwise-open, via the `⇑φ`
    basis
    intro U hU
    rw [ScottOpen.isOpen_iff] at hU
    rw [@isOpen_iff_forall_mem_open (ScottMap D D') (scottMapPointwiseTopology D D')]
    intro g hgU
    obtain ⟨φ, hφg, hφsub⟩ := exists_wayBelow_subset hL hU hgU
    obtain ⟨F, hF, hφF⟩ := wayBelow_le_finset_sup_step hD hD' hφg
    refine ⟨⋂ p ∈ F, {h : ScottMap D D' | (p.2 : D') ≪ (h : D → D') p.1}, ?_, ?_, ?_⟩
    · intro h hh
      refine hφsub (WayBelow.le_trans hφF (wayBelow_finset_sup fun p hp => ?_))
      exact stepMap_wayBelow (Set.mem_iInter₂.1 hh p hp)
```

```
    · exact @isOpen_biInter_finset (ScottMap D D') (D × D') (scottMapPointwiseTopology
    D D') F _
        (fun p _ => pointwiseSubbasic_isOpen p.1 (ScottOpen.isOpen_iff.mpr (
    scottOpen_wayBelow p.2)))
    · exact Set.mem_iInter₂.2 fun p hp => hF p hp

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- The literal product/subspace form of Theorem 3.3: the Scott lattice topology on `[D
     → D']`
is induced from Mathlib's Pi topology on the underlying functions. -/
theorem theorem_3_3.topology_induced_pi (hD : ContinuousLattice D)
    (hD' : ContinuousLattice D') :
    (scottTopologicalSpace : TopologicalSpace (ScottMap D D')) =
      scottMapInducedPiTopology D D' := by
  rw [theorem_3_3.topology hD hD', scottMapPointwiseTopology_eq_induced_pi]

end DomainTheory.ContinuousLattice
```

0

theorem_3_3_more.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_pointwise_sSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_apply_of_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    wayBelow_le_finset_sup_step
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    pointwiseSubbasic_scottOpen
public import DomainTheory.ContinuousLattice.FunctionSpaces.pointwiseSubbasic_isOpen
public import DomainTheory.ContinuousLattice.FunctionSpaces.wayBelow_finset_sup
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottMapPointwiseTopology
public import DomainTheory.ContinuousLattice.WayBelow.ScottOpen
public import DomainTheory.ContinuousLattice.WayBelow.exists_wayBelow_subset
public import DomainTheory.ContinuousLattice.WayBelow.scottOpen_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottMapPointwiseTopology_eq_induced_pi
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottMapInducedPiTopology

@[expose] public section

namespace DomainTheory.ContinuousLattice
```

```
open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Theorem 3.3 (order content).** If `D` and `D'` are continuous
    lattices, then so
is `[D → D']` under the pointwise order. Every `f` is the supremum of the step
    functions way below
it: `f = ⊔ {e[e,e'] | e' ≪ f e}`, and each such step function is way below `f`. -/
theorem theorem_3_3.isContinuousLattice (hD : ContinuousLattice D)
    (hD' : ContinuousLattice D') : ContinuousLattice (ScottMap D D') := by
  refine { out := ?_ }
  intro f
  refine ⟨fun h hh => hh.le, fun g hg => ?_⟩
  rw [ScottMap.le_def]
  intro x
  rw [← stepMap_pointwise_sSup hD hD' f x]
  refine sSup_le ?_
  rintro e' ⟨e, hex, he'⟩
  have hle : stepMap e e' ≤ g := hg (stepMap_wayBelow he')
  have hx := (ScottMap.le_def.1 hle) x
  rwa [stepMap_apply_of_wayBelow hex] at hx

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Theorem 3.3(b).** The Scott (lattice) topology on `[D → D']` agrees
    with the
topology of pointwise convergence. -/
theorem theorem_3_3.topology (hD : ContinuousLattice D) (hD' : ContinuousLattice D') :
    (scottTopologicalSpace : TopologicalSpace (ScottMap D D')) =
    scottMapPointwiseTopology D D' := by
  have hL : ContinuousLattice (ScottMap D D') := theorem_3_3.isContinuousLattice hD hD
    '
  apply le_antisymm
  · -- pointwise ⊆ Scott: each subbasic set is Scott-open in the lattice
    apply le_generateFrom_iff_subset_isOpen.2
    rintro W ⟨x, U, hUopen, rfl⟩
    exact ScottOpen.isOpen_iff.mpr (pointwiseSubbasic_scottOpen x (ScottOpen.
    isOpen_iff.mp hUopen))
  · -- Scott ⊆ pointwise: each Scott-open lattice set is pointwise-open, via the `⇑φ`
    basis
    intro U hU
    rw [ScottOpen.isOpen_iff] at hU
    rw [@isOpen_iff_forall_mem_open (ScottMap D D') (scottMapPointwiseTopology D D')]
    intro g hgU
    obtain ⟨φ, hφg, hφsub⟩ := exists_wayBelow_subset hL hU hgU
    obtain ⟨F, hF, hφF⟩ := wayBelow_le_finset_sup_step hD hD' hφg
    refine ⟨⋂ p ∈ F, {h : ScottMap D D' | (p.2 : D') ≪ (h : D → D') p.1}, ?_, ?_, ?_⟩
```

```
    · intro h hh
      refine hφsub (WayBelow.le_trans hφF (wayBelow_finset_sup fun p hp => ?_))
      exact stepMap_wayBelow (Set.mem_iInter₂.1 hh p hp)
    · exact @isOpen_biInter_finset (ScottMap D D') (D × D') (scottMapPointwiseTopology
    D D') F _
        (fun p _ => pointwiseSubbasic_isOpen p.1 (ScottOpen.isOpen_iff.mpr (
    scottOpen_wayBelow p.2)))
    · exact Set.mem_iInter₂.2 fun p hp => hF p hp

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- The literal product/subspace form of Theorem 3.3: the Scott lattice topology on `[D
     → D']`
is induced from Mathlib's Pi topology on the underlying functions. -/
theorem theorem_3_3.topology_induced_pi (hD : ContinuousLattice D)
    (hD' : ContinuousLattice D') :
    (scottTopologicalSpace : TopologicalSpace (ScottMap D D')) =
      scottMapInducedPiTopology D D' := by
  rw [theorem_3_3.topology hD hD', scottMapPointwiseTopology_eq_induced_pi]

end DomainTheory.ContinuousLattice
```

1

theorem_3_3_more.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_pointwise_sSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_apply_of_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    wayBelow_le_finset_sup_step
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    pointwiseSubbasic_scottOpen
public import DomainTheory.ContinuousLattice.FunctionSpaces.pointwiseSubbasic_isOpen
public import DomainTheory.ContinuousLattice.FunctionSpaces.wayBelow_finset_sup
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottMapPointwiseTopology
public import DomainTheory.ContinuousLattice.WayBelow.ScottOpen
public import DomainTheory.ContinuousLattice.WayBelow.exists_wayBelow_subset
public import DomainTheory.ContinuousLattice.WayBelow.scottOpen_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottMapPointwiseTopology_eq_induced_pi
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottMapInducedPiTopology

@[expose] public section
```

```
namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Theorem 3.3 (order content).** If `D` and `D'` are continuous
    lattices, then so
is `[D → D']` under the pointwise order. Every `f` is the supremum of the step
    functions way below
it: `f = ⊔ {e[e,e'] | e' ≪ f e}`, and each such step function is way below `f`. -/
theorem theorem_3_3.isContinuousLattice (hD : ContinuousLattice D)
    (hD' : ContinuousLattice D') : ContinuousLattice (ScottMap D D') := by
  refine { out := ?_ }
  intro f
  refine ⟨fun h hh => hh.le, fun g hg => ?_⟩
  rw [ScottMap.le_def]
  intro x
  rw [← stepMap_pointwise_sSup hD hD' f x]
  refine sSup_le ?_
  rintro e' ⟨e, hex, he'⟩
  have hle : stepMap e e' ≤ g := hg (stepMap_wayBelow he')
  have hx := (ScottMap.le_def.1 hle) x
  rwa [stepMap_apply_of_wayBelow hex] at hx

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Theorem 3.3(b).** The Scott (lattice) topology on `[D → D']` agrees
    with the
topology of pointwise convergence. -/
theorem theorem_3_3.topology (hD : ContinuousLattice D) (hD' : ContinuousLattice D') :
    (scottTopologicalSpace : TopologicalSpace (ScottMap D D')) =
    scottMapPointwiseTopology D D' := by
  have hL : ContinuousLattice (ScottMap D D') := theorem_3_3.isContinuousLattice hD hD
    '
  apply le_antisymm
  · -- pointwise ⊆ Scott: each subbasic set is Scott-open in the lattice
    apply le_generateFrom_iff_subset_isOpen.2
    rintro W ⟨x, U, hUopen, rfl⟩
    exact ScottOpen.isOpen_iff.mpr (pointwiseSubbasic_scottOpen x (ScottOpen.
    isOpen_iff.mp hUopen))
  · -- Scott ⊆ pointwise: each Scott-open lattice set is pointwise-open, via the `⇑φ`
    basis
    intro U hU
    rw [ScottOpen.isOpen_iff] at hU
    rw [@isOpen_iff_forall_mem_open (ScottMap D D') (scottMapPointwiseTopology D D')]
    intro g hgU
```

```
    obtain ⟨φ, hφg, hφsub⟩ := exists_wayBelow_subset hL hU hgU
    obtain ⟨F, hF, hφF⟩ := wayBelow_le_finset_sup_step hD hD' hφg
    refine ⟨⋂ p ∈ F, {h : ScottMap D D' | (p.2 : D') ≪ (h : D → D') p.1}, ?_, ?_, ?_⟩
    · intro h hh
      refine hφsub (WayBelow.le_trans hφF (wayBelow_finset_sup fun p hp => ?_))
      exact stepMap_wayBelow (Set.mem_iInter₂.1 hh p hp)
    · exact @isOpen_biInter_finset (ScottMap D D') (D × D') (scottMapPointwiseTopology
    D D') F _
        (fun p _ => pointwiseSubbasic_isOpen p.1 (ScottOpen.isOpen_iff.mpr (
    scottOpen_wayBelow p.2)))
    · exact Set.mem_iInter₂.2 fun p hp => hF p hp

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- The literal product/subspace form of Theorem 3.3: the Scott lattice topology on `[D
     → D']`
is induced from Mathlib's Pi topology on the underlying functions. -/
theorem theorem_3_3.topology_induced_pi (hD : ContinuousLattice D)
    (hD' : ContinuousLattice D') :
    (scottTopologicalSpace : TopologicalSpace (ScottMap D D')) =
      scottMapInducedPiTopology D D' := by
  rw [theorem_3_3.topology hD hD', scottMapPointwiseTopology_eq_induced_pi]

end DomainTheory.ContinuousLattice
```

2

theorem_3_3_more.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_pointwise_sSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_apply_of_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    wayBelow_le_finset_sup_step
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    pointwiseSubbasic_scottOpen
public import DomainTheory.ContinuousLattice.FunctionSpaces.pointwiseSubbasic_isOpen
public import DomainTheory.ContinuousLattice.FunctionSpaces.wayBelow_finset_sup
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottMapPointwiseTopology
public import DomainTheory.ContinuousLattice.WayBelow.ScottOpen
public import DomainTheory.ContinuousLattice.WayBelow.exists_wayBelow_subset
public import DomainTheory.ContinuousLattice.WayBelow.scottOpen_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottMapPointwiseTopology_eq_induced_pi
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottMapInducedPiTopology

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Theorem 3.3 (order content).** If `D` and `D'` are continuous
    lattices, then so
is `[D → D']` under the pointwise order. Every `f` is the supremum of the step
    functions way below
it: `f = ⊔ {e[e,e'] | e' ≪ f e}`, and each such step function is way below `f`. -/
theorem theorem_3_3.isContinuousLattice (hD : ContinuousLattice D)
    (hD' : ContinuousLattice D') : ContinuousLattice (ScottMap D D') := by
  refine { out := ?_ }
  intro f
  refine ⟨fun h hh => hh.le, fun g hg => ?_⟩
  rw [ScottMap.le_def]
  intro x
  rw [← stepMap_pointwise_sSup hD hD' f x]
  refine sSup_le ?_
  rintro e' ⟨e, hex, he'⟩
  have hle : stepMap e e' ≤ g := hg (stepMap_wayBelow he')
  have hx := (ScottMap.le_def.1 hle) x
  rwa [stepMap_apply_of_wayBelow hex] at hx

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Theorem 3.3(b).** The Scott (lattice) topology on `[D → D']` agrees
    with the
topology of pointwise convergence. -/
theorem theorem_3_3.topology (hD : ContinuousLattice D) (hD' : ContinuousLattice D') :
    (scottTopologicalSpace : TopologicalSpace (ScottMap D D')) =
    scottMapPointwiseTopology D D' := by
  have hL : ContinuousLattice (ScottMap D D') := theorem_3_3.isContinuousLattice hD hD
    '
  apply le_antisymm
  · -- pointwise ⊆ Scott: each subbasic set is Scott-open in the lattice
    apply le_generateFrom_iff_subset_isOpen.2
    rintro W ⟨x, U, hUopen, rfl⟩
    exact ScottOpen.isOpen_iff.mpr (pointwiseSubbasic_scottOpen x (ScottOpen.
    isOpen_iff.mp hUopen))
  · -- Scott ⊆ pointwise: each Scott-open lattice set is pointwise-open, via the `⇑φ`
    basis
    intro U hU
```

```
    rw [ScottOpen.isOpen_iff] at hU
    rw [@isOpen_iff_forall_mem_open (ScottMap D D') (scottMapPointwiseTopology D D')]
    intro g hgU
    obtain ⟨φ, hφg, hφsub⟩ := exists_wayBelow_subset hL hU hgU
    obtain ⟨F, hF, hφF⟩ := wayBelow_le_finset_sup_step hD hD' hφg
    refine ⟨⋂ p ∈ F, {h : ScottMap D D' | (p.2 : D') ≪ (h : D → D') p.1}, ?_, ?_, ?_⟩
    · intro h hh
      refine hφsub (WayBelow.le_trans hφF (wayBelow_finset_sup fun p hp => ?_))
      exact stepMap_wayBelow (Set.mem_iInter₂.1 hh p hp)
    · exact @isOpen_biInter_finset (ScottMap D D') (D × D') (scottMapPointwiseTopology
    D D') F _
        (fun p _ => pointwiseSubbasic_isOpen p.1 (ScottOpen.isOpen_iff.mpr (
    scottOpen_wayBelow p.2)))
    · exact Set.mem_iInter₂.2 fun p hp => hF p hp

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- The literal product/subspace form of Theorem 3.3: the Scott lattice topology on `[D
     → D']`
is induced from Mathlib's Pi topology on the underlying functions. -/
theorem theorem_3_3.topology_induced_pi (hD : ContinuousLattice D)
    (hD' : ContinuousLattice D') :
    (scottTopologicalSpace : TopologicalSpace (ScottMap D D')) =
      scottMapInducedPiTopology D D' := by
  rw [theorem_3_3.topology hD hD', scottMapPointwiseTopology_eq_induced_pi]

end DomainTheory.ContinuousLattice
```

3

theorem_3_3_more.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_pointwise_sSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_apply_of_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    wayBelow_le_finset_sup_step
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    pointwiseSubbasic_scottOpen
public import DomainTheory.ContinuousLattice.FunctionSpaces.pointwiseSubbasic_isOpen
public import DomainTheory.ContinuousLattice.FunctionSpaces.wayBelow_finset_sup
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottMapPointwiseTopology
public import DomainTheory.ContinuousLattice.WayBelow.ScottOpen
public import DomainTheory.ContinuousLattice.WayBelow.exists_wayBelow_subset
```

```
public import DomainTheory.ContinuousLattice.WayBelow.scottOpen_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottMapPointwiseTopology_eq_induced_pi
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottMapInducedPiTopology

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Theorem 3.3 (order content).** If `D` and `D'` are continuous
    lattices, then so
is `[D → D']` under the pointwise order. Every `f` is the supremum of the step
    functions way below
it: `f = ⊔ {e[e,e'] | e' ≪ f e}`, and each such step function is way below `f`. -/
theorem theorem_3_3.isContinuousLattice (hD : ContinuousLattice D)
    (hD' : ContinuousLattice D') : ContinuousLattice (ScottMap D D') := by
  refine { out := ?_ }
  intro f
  refine ⟨fun h hh => hh.le, fun g hg => ?_⟩
  rw [ScottMap.le_def]
  intro x
  rw [← stepMap_pointwise_sSup hD hD' f x]
  refine sSup_le ?_
  rintro e' ⟨e, hex, he'⟩
  have hle : stepMap e e' ≤ g := hg (stepMap_wayBelow he')
  have hx := (ScottMap.le_def.1 hle) x
  rwa [stepMap_apply_of_wayBelow hex] at hx

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Theorem 3.3(b).** The Scott (lattice) topology on `[D → D']` agrees
    with the
topology of pointwise convergence. -/
theorem theorem_3_3.topology (hD : ContinuousLattice D) (hD' : ContinuousLattice D') :
    (scottTopologicalSpace : TopologicalSpace (ScottMap D D')) =
    scottMapPointwiseTopology D D' := by
  have hL : ContinuousLattice (ScottMap D D') := theorem_3_3.isContinuousLattice hD hD
    '
  apply le_antisymm
  · -- pointwise ⊆ Scott: each subbasic set is Scott-open in the lattice
    apply le_generateFrom_iff_subset_isOpen.2
    rintro W ⟨x, U, hUopen, rfl⟩
    exact ScottOpen.isOpen_iff.mpr (pointwiseSubbasic_scottOpen x (ScottOpen.
    isOpen_iff.mp hUopen))
```

```
  · -- Scott ⊆ pointwise: each Scott-open lattice set is pointwise-open, via the `⇑φ`
    basis
    intro U hU
    rw [ScottOpen.isOpen_iff] at hU
    rw [@isOpen_iff_forall_mem_open (ScottMap D D') (scottMapPointwiseTopology D D')]
    intro g hgU
    obtain ⟨φ, hφg, hφsub⟩ := exists_wayBelow_subset hL hU hgU
    obtain ⟨F, hF, hφF⟩ := wayBelow_le_finset_sup_step hD hD' hφg
    refine ⟨⋂ p ∈ F, {h : ScottMap D D' | (p.2 : D') ≪ (h : D → D') p.1}, ?_, ?_, ?_⟩
    · intro h hh
      refine hφsub (WayBelow.le_trans hφF (wayBelow_finset_sup fun p hp => ?_))
      exact stepMap_wayBelow (Set.mem_iInter₂.1 hh p hp)
    · exact @isOpen_biInter_finset (ScottMap D D') (D × D') (scottMapPointwiseTopology
    D D') F _
        (fun p _ => pointwiseSubbasic_isOpen p.1 (ScottOpen.isOpen_iff.mpr (
    scottOpen_wayBelow p.2)))
    · exact Set.mem_iInter₂.2 fun p hp => hF p hp

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- The literal product/subspace form of Theorem 3.3: the Scott lattice topology on `[D
     → D']`
is induced from Mathlib's Pi topology on the underlying functions. -/
theorem theorem_3_3.topology_induced_pi (hD : ContinuousLattice D)
    (hD' : ContinuousLattice D') :
    (scottTopologicalSpace : TopologicalSpace (ScottMap D D')) =
      scottMapInducedPiTopology D D' := by
  rw [theorem_3_3.topology hD hD', scottMapPointwiseTopology_eq_induced_pi]

end DomainTheory.ContinuousLattice
```

4

theorem_3_3_more.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_pointwise_sSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_apply_of_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    wayBelow_le_finset_sup_step
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    pointwiseSubbasic_scottOpen
public import DomainTheory.ContinuousLattice.FunctionSpaces.pointwiseSubbasic_isOpen
public import DomainTheory.ContinuousLattice.FunctionSpaces.wayBelow_finset_sup
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottMapPointwiseTopology
public import DomainTheory.ContinuousLattice.WayBelow.ScottOpen
public import DomainTheory.ContinuousLattice.WayBelow.exists_wayBelow_subset
public import DomainTheory.ContinuousLattice.WayBelow.scottOpen_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottMapPointwiseTopology_eq_induced_pi
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottMapInducedPiTopology

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Theorem 3.3 (order content).** If `D` and `D'` are continuous
    lattices, then so
is `[D → D']` under the pointwise order. Every `f` is the supremum of the step
    functions way below
it: `f = ⊔ {e[e,e'] | e' ≪ f e}`, and each such step function is way below `f`. -/
theorem theorem_3_3.isContinuousLattice (hD : ContinuousLattice D)
    (hD' : ContinuousLattice D') : ContinuousLattice (ScottMap D D') := by
  refine { out := ?_ }
  intro f
  refine ⟨fun h hh => hh.le, fun g hg => ?_⟩
  rw [ScottMap.le_def]
  intro x
  rw [← stepMap_pointwise_sSup hD hD' f x]
  refine sSup_le ?_
  rintro e' ⟨e, hex, he'⟩
  have hle : stepMap e e' ≤ g := hg (stepMap_wayBelow he')
  have hx := (ScottMap.le_def.1 hle) x
  rwa [stepMap_apply_of_wayBelow hex] at hx

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Theorem 3.3(b).** The Scott (lattice) topology on `[D → D']` agrees
    with the
topology of pointwise convergence. -/
theorem theorem_3_3.topology (hD : ContinuousLattice D) (hD' : ContinuousLattice D') :
    (scottTopologicalSpace : TopologicalSpace (ScottMap D D')) =
    scottMapPointwiseTopology D D' := by
  have hL : ContinuousLattice (ScottMap D D') := theorem_3_3.isContinuousLattice hD hD
    '
  apply le_antisymm
  · -- pointwise ⊆ Scott: each subbasic set is Scott-open in the lattice
    apply le_generateFrom_iff_subset_isOpen.2
```

```
    rintro W ⟨x, U, hUopen, rfl⟩
    exact ScottOpen.isOpen_iff.mpr (pointwiseSubbasic_scottOpen x (ScottOpen.
    isOpen_iff.mp hUopen))
  · -- Scott ⊆ pointwise: each Scott-open lattice set is pointwise-open, via the `⇑φ`
    basis
    intro U hU
    rw [ScottOpen.isOpen_iff] at hU
    rw [@isOpen_iff_forall_mem_open (ScottMap D D') (scottMapPointwiseTopology D D')]
    intro g hgU
    obtain ⟨φ, hφg, hφsub⟩ := exists_wayBelow_subset hL hU hgU
    obtain ⟨F, hF, hφF⟩ := wayBelow_le_finset_sup_step hD hD' hφg
    refine ⟨⋂ p ∈ F, {h : ScottMap D D' | (p.2 : D') ≪ (h : D → D') p.1}, ?_, ?_, ?_⟩
    · intro h hh
      refine hφsub (WayBelow.le_trans hφF (wayBelow_finset_sup fun p hp => ?_))
      exact stepMap_wayBelow (Set.mem_iInter₂.1 hh p hp)
    · exact @isOpen_biInter_finset (ScottMap D D') (D × D') (scottMapPointwiseTopology
    D D') F _
        (fun p _ => pointwiseSubbasic_isOpen p.1 (ScottOpen.isOpen_iff.mpr (
    scottOpen_wayBelow p.2)))
    · exact Set.mem_iInter₂.2 fun p hp => hF p hp

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- The literal product/subspace form of Theorem 3.3: the Scott lattice topology on `[D
    → D']`
is induced from Mathlib's Pi topology on the underlying functions. -/
theorem theorem_3_3.topology_induced_pi (hD : ContinuousLattice D)
    (hD' : ContinuousLattice D') :
    (scottTopologicalSpace : TopologicalSpace (ScottMap D D')) =
      scottMapInducedPiTopology D D' := by
  rw [theorem_3_3.topology hD hD', scottMapPointwiseTopology_eq_induced_pi]

end DomainTheory.ContinuousLattice
```

5

theorem_3_3_more.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_pointwise_sSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_apply_of_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    wayBelow_le_finset_sup_step
public import DomainTheory.ContinuousLattice.FunctionSpaces.
```

```
    pointwiseSubbasic_scottOpen
public import DomainTheory.ContinuousLattice.FunctionSpaces.pointwiseSubbasic_isOpen
public import DomainTheory.ContinuousLattice.FunctionSpaces.wayBelow_finset_sup
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottMapPointwiseTopology
public import DomainTheory.ContinuousLattice.WayBelow.ScottOpen
public import DomainTheory.ContinuousLattice.WayBelow.exists_wayBelow_subset
public import DomainTheory.ContinuousLattice.WayBelow.scottOpen_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottMapPointwiseTopology_eq_induced_pi
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottMapInducedPiTopology

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Theorem 3.3 (order content).** If `D` and `D'` are continuous
    lattices, then so
is `[D → D']` under the pointwise order. Every `f` is the supremum of the step
    functions way below
it: `f = ⊔ {e[e,e'] | e' ≪ f e}`, and each such step function is way below `f`. -/
theorem theorem_3_3.isContinuousLattice (hD : ContinuousLattice D)
    (hD' : ContinuousLattice D') : ContinuousLattice (ScottMap D D') := by
  refine { out := ?_ }
  intro f
  refine ⟨fun h hh => hh.le, fun g hg => ?_⟩
  rw [ScottMap.le_def]
  intro x
  rw [← stepMap_pointwise_sSup hD hD' f x]
  refine sSup_le ?_
  rintro e' ⟨e, hex, he'⟩
  have hle : stepMap e e' ≤ g := hg (stepMap_wayBelow he')
  have hx := (ScottMap.le_def.1 hle) x
  rwa [stepMap_apply_of_wayBelow hex] at hx

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Theorem 3.3(b).** The Scott (lattice) topology on `[D → D']` agrees
    with the
topology of pointwise convergence. -/
theorem theorem_3_3.topology (hD : ContinuousLattice D) (hD' : ContinuousLattice D') :
    (scottTopologicalSpace : TopologicalSpace (ScottMap D D')) =
    scottMapPointwiseTopology D D' := by
  have hL : ContinuousLattice (ScottMap D D') := theorem_3_3.isContinuousLattice hD hD
    '
```

```
  apply le_antisymm
  · -- pointwise ⊆ Scott: each subbasic set is Scott-open in the lattice
    apply le_generateFrom_iff_subset_isOpen.2
    rintro W ⟨x, U, hUopen, rfl⟩
    exact ScottOpen.isOpen_iff.mpr (pointwiseSubbasic_scottOpen x (ScottOpen.
    isOpen_iff.mp hUopen))
  · -- Scott ⊆ pointwise: each Scott-open lattice set is pointwise-open, via the `⇑φ`
    basis
    intro U hU
    rw [ScottOpen.isOpen_iff] at hU
    rw [@isOpen_iff_forall_mem_open (ScottMap D D') (scottMapPointwiseTopology D D')]
    intro g hgU
    obtain ⟨φ, hφg, hφsub⟩ := exists_wayBelow_subset hL hU hgU
    obtain ⟨F, hF, hφF⟩ := wayBelow_le_finset_sup_step hD hD' hφg
    refine ⟨⋂ p ∈ F, {h : ScottMap D D' | (p.2 : D') ≪ (h : D → D') p.1}, ?_, ?_, ?_⟩
    · intro h hh
      refine hφsub (WayBelow.le_trans hφF (wayBelow_finset_sup fun p hp => ?_))
      exact stepMap_wayBelow (Set.mem_iInter₂.1 hh p hp)
    · exact @isOpen_biInter_finset (ScottMap D D') (D × D') (scottMapPointwiseTopology
    D D') F _
        (fun p _ => pointwiseSubbasic_isOpen p.1 (ScottOpen.isOpen_iff.mpr (
    scottOpen_wayBelow p.2)))
    · exact Set.mem_iInter₂.2 fun p hp => hF p hp

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- The literal product/subspace form of Theorem 3.3: the Scott lattice topology on `[D
    → D']`
is induced from Mathlib's Pi topology on the underlying functions. -/
theorem theorem_3_3.topology_induced_pi (hD : ContinuousLattice D)
    (hD' : ContinuousLattice D') :
    (scottTopologicalSpace : TopologicalSpace (ScottMap D D')) =
      scottMapInducedPiTopology D D' := by
  rw [theorem_3_3.topology hD hD', scottMapPointwiseTopology_eq_induced_pi]

end DomainTheory.ContinuousLattice
```

6

theorem_3_3_more.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_pointwise_sSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_apply_of_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    wayBelow_le_finset_sup_step
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    pointwiseSubbasic_scottOpen
public import DomainTheory.ContinuousLattice.FunctionSpaces.pointwiseSubbasic_isOpen
public import DomainTheory.ContinuousLattice.FunctionSpaces.wayBelow_finset_sup
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottMapPointwiseTopology
public import DomainTheory.ContinuousLattice.WayBelow.ScottOpen
public import DomainTheory.ContinuousLattice.WayBelow.exists_wayBelow_subset
public import DomainTheory.ContinuousLattice.WayBelow.scottOpen_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottMapPointwiseTopology_eq_induced_pi
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottMapInducedPiTopology

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Theorem 3.3 (order content).** If `D` and `D'` are continuous
    lattices, then so
is `[D → D']` under the pointwise order. Every `f` is the supremum of the step
    functions way below
it: `f = ⊔ {e[e,e'] | e' ≪ f e}`, and each such step function is way below `f`. -/
theorem theorem_3_3.isContinuousLattice (hD : ContinuousLattice D)
    (hD' : ContinuousLattice D') : ContinuousLattice (ScottMap D D') := by
  refine { out := ?_ }
  intro f
  refine ⟨fun h hh => hh.le, fun g hg => ?_⟩
  rw [ScottMap.le_def]
  intro x
  rw [← stepMap_pointwise_sSup hD hD' f x]
  refine sSup_le ?_
  rintro e' ⟨e, hex, he'⟩
  have hle : stepMap e e' ≤ g := hg (stepMap_wayBelow he')
  have hx := (ScottMap.le_def.1 hle) x
  rwa [stepMap_apply_of_wayBelow hex] at hx

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Theorem 3.3(b).** The Scott (lattice) topology on `[D → D']` agrees
    with the
topology of pointwise convergence. -/
theorem theorem_3_3.topology (hD : ContinuousLattice D) (hD' : ContinuousLattice D') :
    (scottTopologicalSpace : TopologicalSpace (ScottMap D D')) =
```

```
    scottMapPointwiseTopology D D' := by
  have hL : ContinuousLattice (ScottMap D D') := theorem_3_3.isContinuousLattice hD hD
    '
  apply le_antisymm
  · -- pointwise ⊆ Scott: each subbasic set is Scott-open in the lattice
    apply le_generateFrom_iff_subset_isOpen.2
    rintro W ⟨x, U, hUopen, rfl⟩
    exact ScottOpen.isOpen_iff.mpr (pointwiseSubbasic_scottOpen x (ScottOpen.
    isOpen_iff.mp hUopen))
  · -- Scott ⊆ pointwise: each Scott-open lattice set is pointwise-open, via the `⇑φ`
    basis
    intro U hU
    rw [ScottOpen.isOpen_iff] at hU
    rw [@isOpen_iff_forall_mem_open (ScottMap D D') (scottMapPointwiseTopology D D')]
    intro g hgU
    obtain ⟨φ, hφg, hφsub⟩ := exists_wayBelow_subset hL hU hgU
    obtain ⟨F, hF, hφF⟩ := wayBelow_le_finset_sup_step hD hD' hφg
    refine ⟨⋂ p ∈ F, {h : ScottMap D D' | (p.2 : D') ≪ (h : D → D') p.1}, ?_, ?_, ?_⟩
    · intro h hh
      refine hφsub (WayBelow.le_trans hφF (wayBelow_finset_sup fun p hp => ?_))
      exact stepMap_wayBelow (Set.mem_iInter₂.1 hh p hp)
    · exact @isOpen_biInter_finset (ScottMap D D') (D × D') (scottMapPointwiseTopology
    D D') F _
        (fun p _ => pointwiseSubbasic_isOpen p.1 (ScottOpen.isOpen_iff.mpr (
    scottOpen_wayBelow p.2)))
    · exact Set.mem_iInter₂.2 fun p hp => hF p hp

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- The literal product/subspace form of Theorem 3.3: the Scott lattice topology on `[D
     → D']`
is induced from Mathlib's Pi topology on the underlying functions. -/
theorem theorem_3_3.topology_induced_pi (hD : ContinuousLattice D)
    (hD' : ContinuousLattice D') :
    (scottTopologicalSpace : TopologicalSpace (ScottMap D D')) =
      scottMapInducedPiTopology D D' := by
  rw [theorem_3_3.topology hD hD', scottMapPointwiseTopology_eq_induced_pi]

end DomainTheory.ContinuousLattice
```

7

theorem_3_3_more.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_pointwise_sSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_apply_of_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    wayBelow_le_finset_sup_step
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    pointwiseSubbasic_scottOpen
public import DomainTheory.ContinuousLattice.FunctionSpaces.pointwiseSubbasic_isOpen
public import DomainTheory.ContinuousLattice.FunctionSpaces.wayBelow_finset_sup
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottMapPointwiseTopology
public import DomainTheory.ContinuousLattice.WayBelow.ScottOpen
public import DomainTheory.ContinuousLattice.WayBelow.exists_wayBelow_subset
public import DomainTheory.ContinuousLattice.WayBelow.scottOpen_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottMapPointwiseTopology_eq_induced_pi
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottMapInducedPiTopology

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Theorem 3.3 (order content).** If `D` and `D'` are continuous
    lattices, then so
is `[D → D']` under the pointwise order. Every `f` is the supremum of the step
    functions way below
it: `f = ⊔ {e[e,e'] | e' ≪ f e}`, and each such step function is way below `f`. -/
theorem theorem_3_3.isContinuousLattice (hD : ContinuousLattice D)
    (hD' : ContinuousLattice D') : ContinuousLattice (ScottMap D D') := by
  refine { out := ?_ }
  intro f
  refine ⟨fun h hh => hh.le, fun g hg => ?_⟩
  rw [ScottMap.le_def]
  intro x
  rw [← stepMap_pointwise_sSup hD hD' f x]
  refine sSup_le ?_
  rintro e' ⟨e, hex, he'⟩
  have hle : stepMap e e' ≤ g := hg (stepMap_wayBelow he')
  have hx := (ScottMap.le_def.1 hle) x
  rwa [stepMap_apply_of_wayBelow hex] at hx

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Theorem 3.3(b).** The Scott (lattice) topology on `[D → D']` agrees
    with the
```

```
topology of pointwise convergence. -/
theorem theorem_3_3.topology (hD : ContinuousLattice D) (hD' : ContinuousLattice D') :
    (scottTopologicalSpace : TopologicalSpace (ScottMap D D')) =
    scottMapPointwiseTopology D D' := by
  have hL : ContinuousLattice (ScottMap D D') := theorem_3_3.isContinuousLattice hD hD
    '
  apply le_antisymm
  · -- pointwise ⊆ Scott: each subbasic set is Scott-open in the lattice
    apply le_generateFrom_iff_subset_isOpen.2
    rintro W ⟨x, U, hUopen, rfl⟩
    exact ScottOpen.isOpen_iff.mpr (pointwiseSubbasic_scottOpen x (ScottOpen.
    isOpen_iff.mp hUopen))
  · -- Scott ⊆ pointwise: each Scott-open lattice set is pointwise-open, via the `⇑φ`
    basis
    intro U hU
    rw [ScottOpen.isOpen_iff] at hU
    rw [@isOpen_iff_forall_mem_open (ScottMap D D') (scottMapPointwiseTopology D D')]
    intro g hgU
    obtain ⟨φ, hφg, hφsub⟩ := exists_wayBelow_subset hL hU hgU
    obtain ⟨F, hF, hφF⟩ := wayBelow_le_finset_sup_step hD hD' hφg
    refine ⟨⋂ p ∈ F, {h : ScottMap D D' | (p.2 : D') ≪ (h : D → D') p.1}, ?_, ?_, ?_⟩
    · intro h hh
      refine hφsub (WayBelow.le_trans hφF (wayBelow_finset_sup fun p hp => ?_))
      exact stepMap_wayBelow (Set.mem_iInter₂.1 hh p hp)
    · exact @isOpen_biInter_finset (ScottMap D D') (D × D') (scottMapPointwiseTopology
    D D') F _
        (fun p _ => pointwiseSubbasic_isOpen p.1 (ScottOpen.isOpen_iff.mpr (
    scottOpen_wayBelow p.2)))
    · exact Set.mem_iInter₂.2 fun p hp => hF p hp

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- The literal product/subspace form of Theorem 3.3: the Scott lattice topology on `[D
    → D']`
is induced from Mathlib's Pi topology on the underlying functions. -/
theorem theorem_3_3.topology_induced_pi (hD : ContinuousLattice D)
    (hD' : ContinuousLattice D') :
    (scottTopologicalSpace : TopologicalSpace (ScottMap D D')) =
      scottMapInducedPiTopology D D' := by
  rw [theorem_3_3.topology hD hD', scottMapPointwiseTopology_eq_induced_pi]

end DomainTheory.ContinuousLattice
```

8

theorem_3_3_more.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_pointwise_sSup
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_apply_of_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    wayBelow_le_finset_sup_step
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    pointwiseSubbasic_scottOpen
public import DomainTheory.ContinuousLattice.FunctionSpaces.pointwiseSubbasic_isOpen
public import DomainTheory.ContinuousLattice.FunctionSpaces.wayBelow_finset_sup
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottMapPointwiseTopology
public import DomainTheory.ContinuousLattice.WayBelow.ScottOpen
public import DomainTheory.ContinuousLattice.WayBelow.exists_wayBelow_subset
public import DomainTheory.ContinuousLattice.WayBelow.scottOpen_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottMapPointwiseTopology_eq_induced_pi
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottMapInducedPiTopology

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Theorem 3.3 (order content).** If `D` and `D'` are continuous
    lattices, then so
is `[D → D']` under the pointwise order. Every `f` is the supremum of the step
    functions way below
it: `f = ⊔ {e[e,e'] | e' ≪ f e}`, and each such step function is way below `f`. -/
theorem theorem_3_3.isContinuousLattice (hD : ContinuousLattice D)
    (hD' : ContinuousLattice D') : ContinuousLattice (ScottMap D D') := by
  refine { out := ?_ }
  intro f
  refine ⟨fun h hh => hh.le, fun g hg => ?_⟩
  rw [ScottMap.le_def]
  intro x
  rw [← stepMap_pointwise_sSup hD hD' f x]
  refine sSup_le ?_
  rintro e' ⟨e, hex, he'⟩
  have hle : stepMap e e' ≤ g := hg (stepMap_wayBelow he')
  have hx := (ScottMap.le_def.1 hle) x
  rwa [stepMap_apply_of_wayBelow hex] at hx

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']
```

```
/-- **Scott 1972, Theorem 3.3(b).** The Scott (lattice) topology on `[D → D']` agrees
    with the
topology of pointwise convergence. -/
theorem theorem_3_3.topology (hD : ContinuousLattice D) (hD' : ContinuousLattice D') :
    (scottTopologicalSpace : TopologicalSpace (ScottMap D D')) =
    scottMapPointwiseTopology D D' := by
  have hL : ContinuousLattice (ScottMap D D') := theorem_3_3.isContinuousLattice hD hD
    '
  apply le_antisymm
  · -- pointwise ⊆ Scott: each subbasic set is Scott-open in the lattice
    apply le_generateFrom_iff_subset_isOpen.2
    rintro W ⟨x, U, hUopen, rfl⟩
    exact ScottOpen.isOpen_iff.mpr (pointwiseSubbasic_scottOpen x (ScottOpen.
    isOpen_iff.mp hUopen))
  · -- Scott ⊆ pointwise: each Scott-open lattice set is pointwise-open, via the `⇑φ`
    basis
    intro U hU
    rw [ScottOpen.isOpen_iff] at hU
    rw [@isOpen_iff_forall_mem_open (ScottMap D D') (scottMapPointwiseTopology D D')]
    intro g hgU
    obtain ⟨φ, hφg, hφsub⟩ := exists_wayBelow_subset hL hU hgU
    obtain ⟨F, hF, hφF⟩ := wayBelow_le_finset_sup_step hD hD' hφg
    refine ⟨⋂ p ∈ F, {h : ScottMap D D' | (p.2 : D') ≪ (h : D → D') p.1}, ?_, ?_, ?_⟩
    · intro h hh
      refine hφsub (WayBelow.le_trans hφF (wayBelow_finset_sup fun p hp => ?_))
      exact stepMap_wayBelow (Set.mem_iInter₂.1 hh p hp)
    · exact @isOpen_biInter_finset (ScottMap D D') (D × D') (scottMapPointwiseTopology
    D D') F _
        (fun p _ => pointwiseSubbasic_isOpen p.1 (ScottOpen.isOpen_iff.mpr (
    scottOpen_wayBelow p.2)))
    · exact Set.mem_iInter₂.2 fun p hp => hF p hp

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- The literal product/subspace form of Theorem 3.3: the Scott lattice topology on `[D
    → D']`
is induced from Mathlib's Pi topology on the underlying functions. -/
theorem theorem_3_3.topology_induced_pi (hD : ContinuousLattice D)
    (hD' : ContinuousLattice D') :
    (scottTopologicalSpace : TopologicalSpace (ScottMap D D')) =
      scottMapInducedPiTopology D D' := by
  rw [theorem_3_3.topology hD hD', scottMapPointwiseTopology_eq_induced_pi]

end DomainTheory.ContinuousLattice
```

9

toOrderHom.lean

```
module
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- The monotone map underlying a Scott map, suitable for `OrderHom.lfp`. -/
def toOrderHom (f : ScottMap D D) : D →o D := ⟨(f : D → D), f.monotone⟩

end Proposition314

end DomainTheory.ContinuousLattice
```

0

toOrderHom.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- The monotone map underlying a Scott map, suitable for `OrderHom.lfp`. -/
```

```
def toOrderHom (f : ScottMap D D) : D →o D := ⟨(f : D → D), f.monotone⟩

end Proposition314

end DomainTheory.ContinuousLattice
```

1

toOrderHom.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- The monotone map underlying a Scott map, suitable for `OrderHom.lfp`. -/
def toOrderHom (f : ScottMap D D) : D →o D := ⟨(f : D → D), f.monotone⟩

end Proposition314

end DomainTheory.ContinuousLattice
```

2

toOrderHom.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace
```

```
variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- The monotone map underlying a Scott map, suitable for `OrderHom.lfp`. -/
def toOrderHom (f : ScottMap D D) : D →o D := ⟨(f : D → D), f.monotone⟩

end Proposition314

end DomainTheory.ContinuousLattice
```

3

toOrderHom.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- The monotone map underlying a Scott map, suitable for `OrderHom.lfp`. -/
def toOrderHom (f : ScottMap D D) : D →o D := ⟨(f : D → D), f.monotone⟩

end Proposition314

end DomainTheory.ContinuousLattice
```

4

toOrderHom.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- The monotone map underlying a Scott map, suitable for `OrderHom.lfp`. -/
def toOrderHom (f : ScottMap D D) : D →o D := ⟨(f : D → D), f.monotone⟩

end Proposition314

end DomainTheory.ContinuousLattice
```

5

toOrderHom.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314
```

```
/-- The monotone map underlying a Scott map, suitable for `OrderHom.lfp`. -/
def toOrderHom (f : ScottMap D D) : D →o D := ⟨(f : D → D), f.monotone⟩

end Proposition314

end DomainTheory.ContinuousLattice
```

6

toOrderHom.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- The monotone map underlying a Scott map, suitable for `OrderHom.lfp`. -/
def toOrderHom (f : ScottMap D D) : D →o D := ⟨(f : D → D), f.monotone⟩

end Proposition314

end DomainTheory.ContinuousLattice
```

7

toOrderHom.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice
```

```
open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- The monotone map underlying a Scott map, suitable for `OrderHom.lfp`. -/
def toOrderHom (f : ScottMap D D) : D →o D := ⟨(f : D → D), f.monotone⟩

end Proposition314

end DomainTheory.ContinuousLattice
```

8

toOrderHom.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- The monotone map underlying a Scott map, suitable for `OrderHom.lfp`. -/
def toOrderHom (f : ScottMap D D) : D →o D := ⟨(f : D → D), f.monotone⟩

end Proposition314

end DomainTheory.ContinuousLattice
```

9

wayBelow_finset_sup.lean

```
module

public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- A finite sup of elements way below `g` is way below `g`. -/
theorem wayBelow_finset_sup {ι L : Type*} [CompleteLattice L] {s : Finset ι} {f : ι → L
    } {g : L}
    (h : ∀ i ∈ s, f i ≪ g) : s.sup f ≪ g :=
  Finset.sup_induction (p := fun a => a ≪ g) (bot_wayBelow g) (fun _ ha _ hb => ha.sup
    hb) h

end DomainTheory.ContinuousLattice
```

0

wayBelow_finset_sup.lean

```
module

public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
```

```
public import Mathlib.Order.ScottContinuity
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- A finite sup of elements way below `g` is way below `g`. -/
theorem wayBelow_finset_sup {ι L : Type*} [CompleteLattice L] {s : Finset ι} {f : ι → L
    } {g : L}
    (h : ∀ i ∈ s, f i ≪ g) : s.sup f ≪ g :=
  Finset.sup_induction (p := fun a => a ≪ g) (bot_wayBelow g) (fun _ ha _ hb => ha.sup
    hb) h

end DomainTheory.ContinuousLattice
```

1

wayBelow_finset_sup.lean

```
module

public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
```

```
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- A finite sup of elements way below `g` is way below `g`. -/
theorem wayBelow_finset_sup {ι L : Type*} [CompleteLattice L] {s : Finset ι} {f : ι → L
    } {g : L}
    (h : ∀ i ∈ s, f i ≪ g) : s.sup f ≪ g :=
  Finset.sup_induction (p := fun a => a ≪ g) (bot_wayBelow g) (fun _ ha _ hb => ha.sup
    hb) h

end DomainTheory.ContinuousLattice
```

2

wayBelow_finset_sup.lean

```
module

public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- A finite sup of elements way below `g` is way below `g`. -/
theorem wayBelow_finset_sup {ι L : Type*} [CompleteLattice L] {s : Finset ι} {f : ι → L
    } {g : L}
    (h : ∀ i ∈ s, f i ≪ g) : s.sup f ≪ g :=
  Finset.sup_induction (p := fun a => a ≪ g) (bot_wayBelow g) (fun _ ha _ hb => ha.sup
    hb) h

end DomainTheory.ContinuousLattice
```

3

wayBelow_finset_sup.lean

```
module

public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- A finite sup of elements way below `g` is way below `g`. -/
theorem wayBelow_finset_sup {ι L : Type*} [CompleteLattice L] {s : Finset ι} {f : ι → L
    } {g : L}
    (h : ∀ i ∈ s, f i ≪ g) : s.sup f ≪ g :=
  Finset.sup_induction (p := fun a => a ≪ g) (bot_wayBelow g) (fun _ ha _ hb => ha.sup
    hb) h

end DomainTheory.ContinuousLattice
```

4

wayBelow_finset_sup.lean

```
module

public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
```

```
public import Mathlib.Order.ScottContinuity
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- A finite sup of elements way below `g` is way below `g`. -/
theorem wayBelow_finset_sup {ι L : Type*} [CompleteLattice L] {s : Finset ι} {f : ι → L
    } {g : L}
    (h : ∀ i ∈ s, f i ≪ g) : s.sup f ≪ g :=
  Finset.sup_induction (p := fun a => a ≪ g) (bot_wayBelow g) (fun _ ha _ hb => ha.sup
    hb) h

end DomainTheory.ContinuousLattice
```

5

wayBelow_finset_sup.lean

```
module

public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
```

```
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- A finite sup of elements way below `g` is way below `g`. -/
theorem wayBelow_finset_sup {ι L : Type*} [CompleteLattice L] {s : Finset ι} {f : ι → L
    } {g : L}
    (h : ∀ i ∈ s, f i ≪ g) : s.sup f ≪ g :=
  Finset.sup_induction (p := fun a => a ≪ g) (bot_wayBelow g) (fun _ ha _ hb => ha.sup
    hb) h

end DomainTheory.ContinuousLattice
```

6

wayBelow_finset_sup.lean

```
module

public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- A finite sup of elements way below `g` is way below `g`. -/
theorem wayBelow_finset_sup {ι L : Type*} [CompleteLattice L] {s : Finset ι} {f : ι → L
    } {g : L}
    (h : ∀ i ∈ s, f i ≪ g) : s.sup f ≪ g :=
  Finset.sup_induction (p := fun a => a ≪ g) (bot_wayBelow g) (fun _ ha _ hb => ha.sup
    hb) h

end DomainTheory.ContinuousLattice
```

7

wayBelow_finset_sup.lean

```
module

public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- A finite sup of elements way below `g` is way below `g`. -/
theorem wayBelow_finset_sup {ι L : Type*} [CompleteLattice L] {s : Finset ι} {f : ι → L
    } {g : L}
    (h : ∀ i ∈ s, f i ≪ g) : s.sup f ≪ g :=
  Finset.sup_induction (p := fun a => a ≪ g) (bot_wayBelow g) (fun _ ha _ hb => ha.sup
    hb) h

end DomainTheory.ContinuousLattice
```


wayBelow_finset_sup.lean

```
module

public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
```

```
public import Mathlib.Order.ScottContinuity
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- A finite sup of elements way below `g` is way below `g`. -/
theorem wayBelow_finset_sup {ι L : Type*} [CompleteLattice L] {s : Finset ι} {f : ι → L
    } {g : L}
    (h : ∀ i ∈ s, f i ≪ g) : s.sup f ≪ g :=
  Finset.sup_induction (p := fun a => a ≪ g) (bot_wayBelow g) (fun _ ha _ hb => ha.sup
    hb) h

end DomainTheory.ContinuousLattice
```

9

wayBelow_le_finset_sup_step.lean

```
module
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice

public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_pointwise_sSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_le_of_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_apply_of_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']
```

```
/-- **Step-function decomposition of `≪`.** If `φ ≪ g` in `[D → D']`, then `φ` lies
    below a finite
join of step functions `e[eᵢ,eᵢ']` with `eᵢ' ≪ g eᵢ`. The finite joins of such step
    functions form
a directed family with supremum `g`, so `WayBelow.sSup_iff` produces one above `φ`. -/
theorem wayBelow_le_finset_sup_step (hD : ContinuousLattice D) (hD' :
    ContinuousLattice D')
    {φ g : ScottMap D D'} (h : φ ≪ g) :
    ∃ F : Finset (D × D'), (∀ p ∈ F, (p.2 : D') ≪ (g : D → D') p.1) ∧
      φ ≤ F.sup (fun p => stepMap p.1 p.2) := by
  classical
  set Sg : Set (ScottMap D D') :=
    (fun F : Finset (D × D') => F.sup (fun p => stepMap p.1 p.2)) ''
      {F | ∀ p ∈ F, (p.2 : D') ≪ (g : D → D') p.1} with hSg
  have hSgne : Sg.Nonempty := by
    refine ⟨_, ∅, ?_, rfl⟩
    intro p hp
    exact absurd hp (Finset.notMem_empty p)
  have hSgdir : DirectedOn (· ≤ ·) Sg := by
    rintro _ ⟨F₁, hF₁, rfl⟩ _ ⟨F₂, hF₂, rfl⟩
    refine ⟨(F₁ ∪ F₂).sup (fun p => stepMap p.1 p.2), ⟨F₁ ∪ F₂, fun p hp => ?_, rfl⟩,
      Finset.sup_mono Finset.subset_union_left, Finset.sup_mono Finset.
    subset_union_right⟩
    rcases Finset.mem_union.1 hp with hp | hp
    · exact hF₁ p hp
    · exact hF₂ p hp
  have hSgsup : sSup Sg = g := by
    apply le_antisymm
    · refine sSup_le ?_
      rintro _ ⟨F, hF, rfl⟩
      exact Finset.sup_le fun p hp => stepMap_le_of_wayBelow (hF p hp)
    · rw [ScottMap.le_def]
      intro x
      rw [← stepMap_pointwise_sSup hD hD' g x]
      refine sSup_le ?_
      rintro e' ⟨e, hex, he'⟩
      have hmemSg : stepMap e e' ∈ Sg := by
        refine ⟨{(e, e')}, fun p hp => ?_, ?_⟩
        · rw [Finset.mem_singleton] at hp; subst hp; exact he'
        · show ({(e, e')} : Finset (D × D')).sup (fun p => stepMap p.1 p.2) = stepMap e
    e'
          rw [Finset.sup_singleton]
      have hx : (stepMap e e' : D → D') x ≤ ((sSup Sg : ScottMap D D') : D → D') x :=
    by
        rw [ScottMap.sSup_apply]
        exact le_sSup ⟨stepMap e e', hmemSg, rfl⟩
      rwa [stepMap_apply_of_wayBelow hex] at hx
  rw [← hSgsup] at h
  obtain ⟨_, ⟨F, hF, rfl⟩, hφs⟩ := (WayBelow.sSup_iff hSgne hSgdir).1 h
  exact ⟨F, hF, hφs.le⟩

end DomainTheory.ContinuousLattice
```

0

wayBelow_le_finset_sup_step.lean

```
module
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice

public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_pointwise_sSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_le_of_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_apply_of_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Step-function decomposition of `≪`.** If `φ ≪ g` in `[D → D']`, then `φ` lies
    below a finite
join of step functions `e[eᵢ,eᵢ']` with `eᵢ' ≪ g eᵢ`. The finite joins of such step
    functions form
a directed family with supremum `g`, so `WayBelow.sSup_iff` produces one above `φ`. -/
theorem wayBelow_le_finset_sup_step (hD : ContinuousLattice D) (hD' :
    ContinuousLattice D')
    {φ g : ScottMap D D'} (h : φ ≪ g) :
    ∃ F : Finset (D × D'), (∀ p ∈ F, (p.2 : D') ≪ (g : D → D') p.1) ∧
      φ ≤ F.sup (fun p => stepMap p.1 p.2) := by
  classical
  set Sg : Set (ScottMap D D') :=
    (fun F : Finset (D × D') => F.sup (fun p => stepMap p.1 p.2)) ''
      {F | ∀ p ∈ F, (p.2 : D') ≪ (g : D → D') p.1} with hSg
  have hSgne : Sg.Nonempty := by
    refine ⟨_, ∅, ?_, rfl⟩
    intro p hp
    exact absurd hp (Finset.notMem_empty p)
  have hSgdir : DirectedOn (· ≤ ·) Sg := by
    rintro _ ⟨F₁, hF₁, rfl⟩ _ ⟨F₂, hF₂, rfl⟩
    refine ⟨(F₁ ∪ F₂).sup (fun p => stepMap p.1 p.2), ⟨F₁ ∪ F₂, fun p hp => ?_, rfl⟩,
      Finset.sup_mono Finset.subset_union_left, Finset.sup_mono Finset.
    subset_union_right⟩
    rcases Finset.mem_union.1 hp with hp | hp
    · exact hF₁ p hp
    · exact hF₂ p hp
  have hSgsup : sSup Sg = g := by
    apply le_antisymm
    · refine sSup_le ?_
      rintro _ ⟨F, hF, rfl⟩
```

```
      exact Finset.sup_le fun p hp => stepMap_le_of_wayBelow (hF p hp)
    · rw [ScottMap.le_def]
      intro x
      rw [← stepMap_pointwise_sSup hD hD' g x]
      refine sSup_le ?_
      rintro e' ⟨e, hex, he'⟩
      have hmemSg : stepMap e e' ∈ Sg := by
        refine ⟨{(e, e')}, fun p hp => ?_, ?_⟩
        · rw [Finset.mem_singleton] at hp; subst hp; exact he'
        · show ({(e, e')} : Finset (D × D')).sup (fun p => stepMap p.1 p.2) = stepMap e
    e'
          rw [Finset.sup_singleton]
      have hx : (stepMap e e' : D → D') x ≤ ((sSup Sg : ScottMap D D') : D → D') x :=
    by
        rw [ScottMap.sSup_apply]
        exact le_sSup ⟨stepMap e e', hmemSg, rfl⟩
      rwa [stepMap_apply_of_wayBelow hex] at hx
  rw [← hSgsup] at h
  obtain ⟨_, ⟨F, hF, rfl⟩, hφs⟩ := (WayBelow.sSup_iff hSgne hSgdir).1 h
  exact ⟨F, hF, hφs.le⟩

end DomainTheory.ContinuousLattice
```

1

wayBelow_le_finset_sup_step.lean

```
module
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice

public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_pointwise_sSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_le_of_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_apply_of_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Step-function decomposition of `≪`.** If `φ ≪ g` in `[D → D']`, then `φ` lies
    below a finite
join of step functions `e[eᵢ,eᵢ']` with `eᵢ' ≪ g eᵢ`. The finite joins of such step
```

```
    functions form
a directed family with supremum `g`, so `WayBelow.sSup_iff` produces one above `φ`. -/
theorem wayBelow_le_finset_sup_step (hD : ContinuousLattice D) (hD' :
    ContinuousLattice D')
    {φ g : ScottMap D D'} (h : φ ≪ g) :
    ∃ F : Finset (D × D'), (∀ p ∈ F, (p.2 : D') ≪ (g : D → D') p.1) ∧
      φ ≤ F.sup (fun p => stepMap p.1 p.2) := by
  classical
  set Sg : Set (ScottMap D D') :=
    (fun F : Finset (D × D') => F.sup (fun p => stepMap p.1 p.2)) ''
      {F | ∀ p ∈ F, (p.2 : D') ≪ (g : D → D') p.1} with hSg
  have hSgne : Sg.Nonempty := by
    refine ⟨_, ∅, ?_, rfl⟩
    intro p hp
    exact absurd hp (Finset.notMem_empty p)
  have hSgdir : DirectedOn (· ≤ ·) Sg := by
    rintro _ ⟨F₁, hF₁, rfl⟩ _ ⟨F₂, hF₂, rfl⟩
    refine ⟨(F₁ ∪ F₂).sup (fun p => stepMap p.1 p.2), ⟨F₁ ∪ F₂, fun p hp => ?_, rfl⟩,
      Finset.sup_mono Finset.subset_union_left, Finset.sup_mono Finset.
    subset_union_right⟩
    rcases Finset.mem_union.1 hp with hp | hp
    · exact hF₁ p hp
    · exact hF₂ p hp
  have hSgsup : sSup Sg = g := by
    apply le_antisymm
    · refine sSup_le ?_
      rintro _ ⟨F, hF, rfl⟩
      exact Finset.sup_le fun p hp => stepMap_le_of_wayBelow (hF p hp)
    · rw [ScottMap.le_def]
      intro x
      rw [← stepMap_pointwise_sSup hD hD' g x]
      refine sSup_le ?_
      rintro e' ⟨e, hex, he'⟩
      have hmemSg : stepMap e e' ∈ Sg := by
        refine ⟨{(e, e')}, fun p hp => ?_, ?_⟩
        · rw [Finset.mem_singleton] at hp; subst hp; exact he'
        · show ({(e, e')} : Finset (D × D')).sup (fun p => stepMap p.1 p.2) = stepMap e
     e'
          rw [Finset.sup_singleton]
      have hx : (stepMap e e' : D → D') x ≤ ((sSup Sg : ScottMap D D') : D → D') x :=
     by
        rw [ScottMap.sSup_apply]
        exact le_sSup ⟨stepMap e e', hmemSg, rfl⟩
      rwa [stepMap_apply_of_wayBelow hex] at hx
  rw [← hSgsup] at h
  obtain ⟨_, ⟨F, hF, rfl⟩, hφs⟩ := (WayBelow.sSup_iff hSgne hSgdir).1 h
  exact ⟨F, hF, hφs.le⟩

end DomainTheory.ContinuousLattice
```

2

wayBelow_le_finset_sup_step.lean

```
module
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
```

```
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice

public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_pointwise_sSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_le_of_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_apply_of_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Step-function decomposition of `≪`.** If `φ ≪ g` in `[D → D']`, then `φ` lies
    below a finite
join of step functions `e[eᵢ,eᵢ']` with `eᵢ' ≪ g eᵢ`. The finite joins of such step
    functions form
a directed family with supremum `g`, so `WayBelow.sSup_iff` produces one above `φ`. -/
theorem wayBelow_le_finset_sup_step (hD : ContinuousLattice D) (hD' :
    ContinuousLattice D')
    {φ g : ScottMap D D'} (h : φ ≪ g) :
    ∃ F : Finset (D × D'), (∀ p ∈ F, (p.2 : D') ≪ (g : D → D') p.1) ∧
      φ ≤ F.sup (fun p => stepMap p.1 p.2) := by
  classical
  set Sg : Set (ScottMap D D') :=
    (fun F : Finset (D × D') => F.sup (fun p => stepMap p.1 p.2)) ''
      {F | ∀ p ∈ F, (p.2 : D') ≪ (g : D → D') p.1} with hSg
  have hSgne : Sg.Nonempty := by
    refine ⟨_, ∅, ?_, rfl⟩
    intro p hp
    exact absurd hp (Finset.notMem_empty p)
  have hSgdir : DirectedOn (· ≤ ·) Sg := by
    rintro _ ⟨F₁, hF₁, rfl⟩ _ ⟨F₂, hF₂, rfl⟩
    refine ⟨(F₁ ∪ F₂).sup (fun p => stepMap p.1 p.2), ⟨F₁ ∪ F₂, fun p hp => ?_, rfl⟩,
      Finset.sup_mono Finset.subset_union_left, Finset.sup_mono Finset.
    subset_union_right⟩
    rcases Finset.mem_union.1 hp with hp | hp
    · exact hF₁ p hp
    · exact hF₂ p hp
  have hSgsup : sSup Sg = g := by
    apply le_antisymm
    · refine sSup_le ?_
      rintro _ ⟨F, hF, rfl⟩
      exact Finset.sup_le fun p hp => stepMap_le_of_wayBelow (hF p hp)
    · rw [ScottMap.le_def]
```

```
      intro x
      rw [← stepMap_pointwise_sSup hD hD' g x]
      refine sSup_le ?_
      rintro e' ⟨e, hex, he'⟩
      have hmemSg : stepMap e e' ∈ Sg := by
        refine ⟨{(e, e')}, fun p hp => ?_, ?_⟩
        · rw [Finset.mem_singleton] at hp; subst hp; exact he'
        · show ({(e, e')} : Finset (D × D')).sup (fun p => stepMap p.1 p.2) = stepMap e
    e'
          rw [Finset.sup_singleton]
      have hx : (stepMap e e' : D → D') x ≤ ((sSup Sg : ScottMap D D') : D → D') x :=
    by
        rw [ScottMap.sSup_apply]
        exact le_sSup ⟨stepMap e e', hmemSg, rfl⟩
      rwa [stepMap_apply_of_wayBelow hex] at hx
  rw [← hSgsup] at h
  obtain ⟨_, ⟨F, hF, rfl⟩, hφs⟩ := (WayBelow.sSup_iff hSgne hSgdir).1 h
  exact ⟨F, hF, hφs.le⟩

end DomainTheory.ContinuousLattice
```

3

wayBelow_le_finset_sup_step.lean

```
module
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice

public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_pointwise_sSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_le_of_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_apply_of_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Step-function decomposition of `≪`.** If `φ ≪ g` in `[D → D']`, then `φ` lies
    below a finite
join of step functions `e[eᵢ,eᵢ']` with `eᵢ' ≪ g eᵢ`. The finite joins of such step
    functions form
a directed family with supremum `g`, so `WayBelow.sSup_iff` produces one above `φ`. -/
```

```
theorem wayBelow_le_finset_sup_step (hD : ContinuousLattice D) (hD' :
    ContinuousLattice D')
    {φ g : ScottMap D D'} (h : φ ≪ g) :
    ∃ F : Finset (D × D'), (∀ p ∈ F, (p.2 : D') ≪ (g : D → D') p.1) ∧
      φ ≤ F.sup (fun p => stepMap p.1 p.2) := by
  classical
  set Sg : Set (ScottMap D D') :=
    (fun F : Finset (D × D') => F.sup (fun p => stepMap p.1 p.2)) ''
      {F | ∀ p ∈ F, (p.2 : D') ≪ (g : D → D') p.1} with hSg
  have hSgne : Sg.Nonempty := by
    refine ⟨_, ∅, ?_, rfl⟩
    intro p hp
    exact absurd hp (Finset.notMem_empty p)
  have hSgdir : DirectedOn (· ≤ ·) Sg := by
    rintro _ ⟨F₁, hF₁, rfl⟩ _ ⟨F₂, hF₂, rfl⟩
    refine ⟨(F₁ ∪ F₂).sup (fun p => stepMap p.1 p.2), ⟨F₁ ∪ F₂, fun p hp => ?_, rfl⟩,
      Finset.sup_mono Finset.subset_union_left, Finset.sup_mono Finset.
    subset_union_right⟩
    rcases Finset.mem_union.1 hp with hp | hp
    · exact hF₁ p hp
    · exact hF₂ p hp
  have hSgsup : sSup Sg = g := by
    apply le_antisymm
    · refine sSup_le ?_
      rintro _ ⟨F, hF, rfl⟩
      exact Finset.sup_le fun p hp => stepMap_le_of_wayBelow (hF p hp)
    · rw [ScottMap.le_def]
      intro x
      rw [← stepMap_pointwise_sSup hD hD' g x]
      refine sSup_le ?_
      rintro e' ⟨e, hex, he'⟩
      have hmemSg : stepMap e e' ∈ Sg := by
        refine ⟨{(e, e')}, fun p hp => ?_, ?_⟩
        · rw [Finset.mem_singleton] at hp; subst hp; exact he'
        · show ({(e, e')} : Finset (D × D')).sup (fun p => stepMap p.1 p.2) = stepMap e
    e'
          rw [Finset.sup_singleton]
      have hx : (stepMap e e' : D → D') x ≤ ((sSup Sg : ScottMap D D') : D → D') x :=
    by
        rw [ScottMap.sSup_apply]
        exact le_sSup ⟨stepMap e e', hmemSg, rfl⟩
      rwa [stepMap_apply_of_wayBelow hex] at hx
  rw [← hSgsup] at h
  obtain ⟨_, ⟨F, hF, rfl⟩, hφs⟩ := (WayBelow.sSup_iff hSgne hSgdir).1 h
  exact ⟨F, hF, hφs.le⟩

end DomainTheory.ContinuousLattice
```

4

wayBelow_le_finset_sup_step.lean

```
module
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_pointwise_sSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_le_of_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_apply_of_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Step-function decomposition of `≪`.** If `φ ≪ g` in `[D → D']`, then `φ` lies
    below a finite
join of step functions `e[eᵢ,eᵢ']` with `eᵢ' ≪ g eᵢ`. The finite joins of such step
    functions form
a directed family with supremum `g`, so `WayBelow.sSup_iff` produces one above `φ`. -/
theorem wayBelow_le_finset_sup_step (hD : ContinuousLattice D) (hD' :
    ContinuousLattice D')
    {φ g : ScottMap D D'} (h : φ ≪ g) :
    ∃ F : Finset (D × D'), (∀ p ∈ F, (p.2 : D') ≪ (g : D → D') p.1) ∧
      φ ≤ F.sup (fun p => stepMap p.1 p.2) := by
  classical
  set Sg : Set (ScottMap D D') :=
    (fun F : Finset (D × D') => F.sup (fun p => stepMap p.1 p.2)) ''
      {F | ∀ p ∈ F, (p.2 : D') ≪ (g : D → D') p.1} with hSg
  have hSgne : Sg.Nonempty := by
    refine ⟨_, ∅, ?_, rfl⟩
    intro p hp
    exact absurd hp (Finset.notMem_empty p)
  have hSgdir : DirectedOn (· ≤ ·) Sg := by
    rintro _ ⟨F₁, hF₁, rfl⟩ _ ⟨F₂, hF₂, rfl⟩
    refine ⟨(F₁ ∪ F₂).sup (fun p => stepMap p.1 p.2), ⟨F₁ ∪ F₂, fun p hp => ?_, rfl⟩,
      Finset.sup_mono Finset.subset_union_left, Finset.sup_mono Finset.
    subset_union_right⟩
    rcases Finset.mem_union.1 hp with hp | hp
    · exact hF₁ p hp
    · exact hF₂ p hp
  have hSgsup : sSup Sg = g := by
    apply le_antisymm
    · refine sSup_le ?_
      rintro _ ⟨F, hF, rfl⟩
      exact Finset.sup_le fun p hp => stepMap_le_of_wayBelow (hF p hp)
    · rw [ScottMap.le_def]
      intro x
      rw [← stepMap_pointwise_sSup hD hD' g x]
```

```
      refine sSup_le ?_
      rintro e' ⟨e, hex, he'⟩
      have hmemSg : stepMap e e' ∈ Sg := by
        refine ⟨{(e, e')}, fun p hp => ?_, ?_⟩
        · rw [Finset.mem_singleton] at hp; subst hp; exact he'
        · show ({(e, e')} : Finset (D × D')).sup (fun p => stepMap p.1 p.2) = stepMap e
    e'
          rw [Finset.sup_singleton]
      have hx : (stepMap e e' : D → D') x ≤ ((sSup Sg : ScottMap D D') : D → D') x :=
    by
        rw [ScottMap.sSup_apply]
        exact le_sSup ⟨stepMap e e', hmemSg, rfl⟩
      rwa [stepMap_apply_of_wayBelow hex] at hx
  rw [← hSgsup] at h
  obtain ⟨_, ⟨F, hF, rfl⟩, hφs⟩ := (WayBelow.sSup_iff hSgne hSgdir).1 h
  exact ⟨F, hF, hφs.le⟩

end DomainTheory.ContinuousLattice
```

5

wayBelow_le_finset_sup_step.lean

```
module
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice

public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_pointwise_sSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_le_of_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_apply_of_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Step-function decomposition of `≪`.** If `φ ≪ g` in `[D → D']`, then `φ` lies
    below a finite
join of step functions `e[eᵢ,eᵢ']` with `eᵢ' ≪ g eᵢ`. The finite joins of such step
    functions form
a directed family with supremum `g`, so `WayBelow.sSup_iff` produces one above `φ`. -/
theorem wayBelow_le_finset_sup_step (hD : ContinuousLattice D) (hD' :
    ContinuousLattice D')
```

```
    {φ g : ScottMap D D'} (h : φ ≪ g) :
    ∃ F : Finset (D × D'), (∀ p ∈ F, (p.2 : D') ≪ (g : D → D') p.1) ∧
      φ ≤ F.sup (fun p => stepMap p.1 p.2) := by
  classical
  set Sg : Set (ScottMap D D') :=
    (fun F : Finset (D × D') => F.sup (fun p => stepMap p.1 p.2)) ''
      {F | ∀ p ∈ F, (p.2 : D') ≪ (g : D → D') p.1} with hSg
  have hSgne : Sg.Nonempty := by
    refine ⟨_, ∅, ?_, rfl⟩
    intro p hp
    exact absurd hp (Finset.notMem_empty p)
  have hSgdir : DirectedOn (· ≤ ·) Sg := by
    rintro _ ⟨F₁, hF₁, rfl⟩ _ ⟨F₂, hF₂, rfl⟩
    refine ⟨(F₁ ∪ F₂).sup (fun p => stepMap p.1 p.2), ⟨F₁ ∪ F₂, fun p hp => ?_, rfl⟩,
      Finset.sup_mono Finset.subset_union_left, Finset.sup_mono Finset.
    subset_union_right⟩
    rcases Finset.mem_union.1 hp with hp | hp
    · exact hF₁ p hp
    · exact hF₂ p hp
  have hSgsup : sSup Sg = g := by
    apply le_antisymm
    · refine sSup_le ?_
      rintro _ ⟨F, hF, rfl⟩
      exact Finset.sup_le fun p hp => stepMap_le_of_wayBelow (hF p hp)
    · rw [ScottMap.le_def]
      intro x
      rw [← stepMap_pointwise_sSup hD hD' g x]
      refine sSup_le ?_
      rintro e' ⟨e, hex, he'⟩
      have hmemSg : stepMap e e' ∈ Sg := by
        refine ⟨{(e, e')}, fun p hp => ?_, ?_⟩
        · rw [Finset.mem_singleton] at hp; subst hp; exact he'
        · show ({(e, e')} : Finset (D × D')).sup (fun p => stepMap p.1 p.2) = stepMap e
    e'
          rw [Finset.sup_singleton]
      have hx : (stepMap e e' : D → D') x ≤ ((sSup Sg : ScottMap D D') : D → D') x :=
    by
        rw [ScottMap.sSup_apply]
        exact le_sSup ⟨stepMap e e', hmemSg, rfl⟩
      rwa [stepMap_apply_of_wayBelow hex] at hx
  rw [← hSgsup] at h
  obtain ⟨_, ⟨F, hF, rfl⟩, hφs⟩ := (WayBelow.sSup_iff hSgne hSgdir).1 h
  exact ⟨F, hF, hφs.le⟩

end DomainTheory.ContinuousLattice
```

6

wayBelow_le_finset_sup_step.lean

```
module
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice

public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_pointwise_sSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_le_of_wayBelow
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_apply_of_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Step-function decomposition of `≪`.** If `φ ≪ g` in `[D → D']`, then `φ` lies
    below a finite
join of step functions `e[eᵢ,eᵢ']` with `eᵢ' ≪ g eᵢ`. The finite joins of such step
    functions form
a directed family with supremum `g`, so `WayBelow.sSup_iff` produces one above `φ`. -/
theorem wayBelow_le_finset_sup_step (hD : ContinuousLattice D) (hD' :
    ContinuousLattice D')
    {φ g : ScottMap D D'} (h : φ ≪ g) :
    ∃ F : Finset (D × D'), (∀ p ∈ F, (p.2 : D') ≪ (g : D → D') p.1) ∧
      φ ≤ F.sup (fun p => stepMap p.1 p.2) := by
  classical
  set Sg : Set (ScottMap D D') :=
    (fun F : Finset (D × D') => F.sup (fun p => stepMap p.1 p.2)) ''
      {F | ∀ p ∈ F, (p.2 : D') ≪ (g : D → D') p.1} with hSg
  have hSgne : Sg.Nonempty := by
    refine ⟨_, ∅, ?_, rfl⟩
    intro p hp
    exact absurd hp (Finset.notMem_empty p)
  have hSgdir : DirectedOn (· ≤ ·) Sg := by
    rintro _ ⟨F₁, hF₁, rfl⟩ _ ⟨F₂, hF₂, rfl⟩
    refine ⟨(F₁ ∪ F₂).sup (fun p => stepMap p.1 p.2), ⟨F₁ ∪ F₂, fun p hp => ?_, rfl⟩,
      Finset.sup_mono Finset.subset_union_left, Finset.sup_mono Finset.
    subset_union_right⟩
    rcases Finset.mem_union.1 hp with hp | hp
    · exact hF₁ p hp
    · exact hF₂ p hp
  have hSgsup : sSup Sg = g := by
    apply le_antisymm
    · refine sSup_le ?_
      rintro _ ⟨F, hF, rfl⟩
      exact Finset.sup_le fun p hp => stepMap_le_of_wayBelow (hF p hp)
    · rw [ScottMap.le_def]
      intro x
      rw [← stepMap_pointwise_sSup hD hD' g x]
      refine sSup_le ?_
      rintro e' ⟨e, hex, he'⟩
```

```
      have hmemSg : stepMap e e' ∈ Sg := by
        refine ⟨{(e, e')}, fun p hp => ?_, ?_⟩
        · rw [Finset.mem_singleton] at hp; subst hp; exact he'
        · show ({(e, e')} : Finset (D × D')).sup (fun p => stepMap p.1 p.2) = stepMap e
    e'
          rw [Finset.sup_singleton]
      have hx : (stepMap e e' : D → D') x ≤ ((sSup Sg : ScottMap D D') : D → D') x :=
    by
        rw [ScottMap.sSup_apply]
        exact le_sSup ⟨stepMap e e', hmemSg, rfl⟩
      rwa [stepMap_apply_of_wayBelow hex] at hx
  rw [← hSgsup] at h
  obtain ⟨_, ⟨F, hF, rfl⟩, hφs⟩ := (WayBelow.sSup_iff hSgne hSgdir).1 h
  exact ⟨F, hF, hφs.le⟩

end DomainTheory.ContinuousLattice
```

7

wayBelow_le_finset_sup_step.lean

```
module
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice

public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_pointwise_sSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_le_of_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_apply_of_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Step-function decomposition of `≪`.** If `φ ≪ g` in `[D → D']`, then `φ` lies
    below a finite
join of step functions `e[eᵢ,eᵢ']` with `eᵢ' ≪ g eᵢ`. The finite joins of such step
    functions form
a directed family with supremum `g`, so `WayBelow.sSup_iff` produces one above `φ`. -/
theorem wayBelow_le_finset_sup_step (hD : ContinuousLattice D) (hD' :
    ContinuousLattice D')
    {φ g : ScottMap D D'} (h : φ ≪ g) :
    ∃ F : Finset (D × D'), (∀ p ∈ F, (p.2 : D') ≪ (g : D → D') p.1) ∧
```

```
      φ ≤ F.sup (fun p => stepMap p.1 p.2) := by
  classical
  set Sg : Set (ScottMap D D') :=
    (fun F : Finset (D × D') => F.sup (fun p => stepMap p.1 p.2)) ''
      {F | ∀ p ∈ F, (p.2 : D') ≪ (g : D → D') p.1} with hSg
  have hSgne : Sg.Nonempty := by
    refine ⟨_, ∅, ?_, rfl⟩
    intro p hp
    exact absurd hp (Finset.notMem_empty p)
  have hSgdir : DirectedOn (· ≤ ·) Sg := by
    rintro _ ⟨F₁, hF₁, rfl⟩ _ ⟨F₂, hF₂, rfl⟩
    refine ⟨(F₁ ∪ F₂).sup (fun p => stepMap p.1 p.2), ⟨F₁ ∪ F₂, fun p hp => ?_, rfl⟩,
      Finset.sup_mono Finset.subset_union_left, Finset.sup_mono Finset.
    subset_union_right⟩
    rcases Finset.mem_union.1 hp with hp | hp
    · exact hF₁ p hp
    · exact hF₂ p hp
  have hSgsup : sSup Sg = g := by
    apply le_antisymm
    · refine sSup_le ?_
      rintro _ ⟨F, hF, rfl⟩
      exact Finset.sup_le fun p hp => stepMap_le_of_wayBelow (hF p hp)
    · rw [ScottMap.le_def]
      intro x
      rw [← stepMap_pointwise_sSup hD hD' g x]
      refine sSup_le ?_
      rintro e' ⟨e, hex, he'⟩
      have hmemSg : stepMap e e' ∈ Sg := by
        refine ⟨{(e, e')}, fun p hp => ?_, ?_⟩
        · rw [Finset.mem_singleton] at hp; subst hp; exact he'
        · show ({(e, e')} : Finset (D × D')).sup (fun p => stepMap p.1 p.2) = stepMap e
    e'
          rw [Finset.sup_singleton]
      have hx : (stepMap e e' : D → D') x ≤ ((sSup Sg : ScottMap D D') : D → D') x :=
    by
        rw [ScottMap.sSup_apply]
        exact le_sSup ⟨stepMap e e', hmemSg, rfl⟩
      rwa [stepMap_apply_of_wayBelow hex] at hx
  rw [← hSgsup] at h
  obtain ⟨_, ⟨F, hF, rfl⟩, hφs⟩ := (WayBelow.sSup_iff hSgne hSgdir).1 h
  exact ⟨F, hF, hφs.le⟩

end DomainTheory.ContinuousLattice
```

8

wayBelow_le_finset_sup_step.lean

```
module
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice

public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_pointwise_sSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_le_of_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_apply_of_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Step-function decomposition of `≪`.** If `φ ≪ g` in `[D → D']`, then `φ` lies
    below a finite
join of step functions `e[eᵢ,eᵢ']` with `eᵢ' ≪ g eᵢ`. The finite joins of such step
    functions form
a directed family with supremum `g`, so `WayBelow.sSup_iff` produces one above `φ`. -/
theorem wayBelow_le_finset_sup_step (hD : ContinuousLattice D) (hD' :
    ContinuousLattice D')
    {φ g : ScottMap D D'} (h : φ ≪ g) :
    ∃ F : Finset (D × D'), (∀ p ∈ F, (p.2 : D') ≪ (g : D → D') p.1) ∧
      φ ≤ F.sup (fun p => stepMap p.1 p.2) := by
  classical
  set Sg : Set (ScottMap D D') :=
    (fun F : Finset (D × D') => F.sup (fun p => stepMap p.1 p.2)) ''
      {F | ∀ p ∈ F, (p.2 : D') ≪ (g : D → D') p.1} with hSg
  have hSgne : Sg.Nonempty := by
    refine ⟨_, ∅, ?_, rfl⟩
    intro p hp
    exact absurd hp (Finset.notMem_empty p)
  have hSgdir : DirectedOn (· ≤ ·) Sg := by
    rintro _ ⟨F₁, hF₁, rfl⟩ _ ⟨F₂, hF₂, rfl⟩
    refine ⟨(F₁ ∪ F₂).sup (fun p => stepMap p.1 p.2), ⟨F₁ ∪ F₂, fun p hp => ?_, rfl⟩,
      Finset.sup_mono Finset.subset_union_left, Finset.sup_mono Finset.
    subset_union_right⟩
    rcases Finset.mem_union.1 hp with hp | hp
    · exact hF₁ p hp
    · exact hF₂ p hp
  have hSgsup : sSup Sg = g := by
    apply le_antisymm
    · refine sSup_le ?_
      rintro _ ⟨F, hF, rfl⟩
      exact Finset.sup_le fun p hp => stepMap_le_of_wayBelow (hF p hp)
    · rw [ScottMap.le_def]
      intro x
      rw [← stepMap_pointwise_sSup hD hD' g x]
      refine sSup_le ?_
      rintro e' ⟨e, hex, he'⟩
      have hmemSg : stepMap e e' ∈ Sg := by
        refine ⟨{(e, e')}, fun p hp => ?_, ?_⟩
```

```
        · rw [Finset.mem_singleton] at hp; subst hp; exact he'
        · show ({(e, e')} : Finset (D × D')).sup (fun p => stepMap p.1 p.2) = stepMap e
    e'
          rw [Finset.sup_singleton]
      have hx : (stepMap e e' : D → D') x ≤ ((sSup Sg : ScottMap D D') : D → D') x :=
    by
       rw [ScottMap.sSup_apply]
       exact le_sSup ⟨stepMap e e', hmemSg, rfl⟩
     rwa [stepMap_apply_of_wayBelow hex] at hx
  rw [← hSgsup] at h
  obtain ⟨_, ⟨F, hF, rfl⟩, hφs⟩ := (WayBelow.sSup_iff hSgne hSgdir).1 h
  exact ⟨F, hF, hφs.le⟩

end DomainTheory.ContinuousLattice
```

9

corollary_3_4.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.Basic
public import DomainTheory.ContinuousLattice.FunctionSpaces.corollary_3_4_eval_on_C
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottFunctionTopology
public import Mathlib.Data.Set.Basic
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Corollary 3.4 (fixed `x`).** Evaluation at fixed `x` is continuous on
     `[D → D']`
(with Scott topologies on `D` and `D'`); this is one half of the separate-continuity
    input to
joint continuity. -/
theorem corollary_3_4 (x : D) :
    @Continuous (ScottC D D') D'
      (@scottFunctionTopology D D' scottTopologicalSpace scottTopologicalSpace)
      scottTopologicalSpace (fun f : ScottC D D' => f x) :=
  corollary_3_4_eval_on_C (D := D) (D' := D') x

end DomainTheory.ContinuousLattice
```

0

corollary_3_4.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.Basic
public import DomainTheory.ContinuousLattice.FunctionSpaces.corollary_3_4_eval_on_C
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottFunctionTopology
public import Mathlib.Data.Set.Basic
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Corollary 3.4 (fixed `x`).** Evaluation at fixed `x` is continuous on
     `[D → D']`
(with Scott topologies on `D` and `D'`); this is one half of the separate-continuity
    input to
joint continuity. -/
theorem corollary_3_4 (x : D) :
    @Continuous (ScottC D D') D'
      (@scottFunctionTopology D D' scottTopologicalSpace scottTopologicalSpace)
      scottTopologicalSpace (fun f : ScottC D D' => f x) :=
  corollary_3_4_eval_on_C (D := D) (D' := D') x

end DomainTheory.ContinuousLattice
```

1

4.5 Approximable mappings between domains

Dependencies. Inside ApproximableMappingsBetweenDomains, each box is one subfolder, or one Lean file sitting directly in the folder. A box named as another folder in this chapter is that whole folder. A box named as another chapter is that whole chapter. The arrowhead points at the file or folder that imports, and the number is how many direct imports.



Figure 40: Approximable mappings between domains.

Once a general notion of set or domain has been defined mathematically, the next major issue is: How are the different domains to be related one to another? The answer to this important question can many times be given by defining an appropriate concept of mapping between domains. The answer need not be unique; the same collection of domains may support more than one idea of function depending on the special aspects

of the domains that need to be brought out. In the case of the kinds of domains being studied here, there are two principal answers, one of which is to be explained in this section.

In order to understand an appropriate notion of a mapping or a function between domains constructed from information systems, we have to refer back to the way in which elements are introduced. As we have seen in Section 3, an element is a consistent, closed set of data objects. To generate an element, one has to generate more and more of the finite consistent subsets of the element. Note that the separate data objects have to be grouped into these finite consistent sets, because the entailment relation may require a finite set of arbitrary size on the left in making the necessary entailments for closure. It is this kind of passage from a finite set to its closure that is to be generalized in defining mappings.

Consider two domains. To map from one to another, some information about a possible element of the first is presented as input to the function f. Then as output the function f starts generating an element. If the input were u, a consistent set of the first domain, then part of the output in the second domain might be a consistent set v. We could say that there is an input/output relationship set up by f, and indicate by $u\ f\ v$ that this relation holds going from u in the first domain to v in the second. Of course to get the full effect of f, it is necessary to take all the v's related to a given u, because even a small finite amount of input may cause an infinite amount of output. But every element of a domain is just the sum total of its finite subsets (finite approximations), so it is sufficient to make the mapping relationship go between finite sets. Here is the formal definition with the exact conditions that the relation f must satisfy.

- Definition 5.1
- Definition 5.2
- Proposition 5.3
- Proposition 5.4
- Proposition 5.5
- Proposition 5.6

The proof is immediate, since we need only interpret the input/output relation $u\ (\mathrm{const}\ b)\ v$ as meaning $v \subseteq b$. We note that our notation is somewhat ambiguous since, for example, in 5.6(iii) we are using $\mathrm{const}\ b$ in two senses: once as a mapping from $\mathbf{A}$, and once as a mapping from $\mathbf{D}$. In fact, we should hang two subscripts on the thing to fix both the source and the target of the mapping. This is too painful, however, and we generally rely on context to make the meaning clear. (Formal rules of type checking can be given, so on a computer-based system all the necessary subscripts could be put back in. Thus, the kind of ambiguity we are dealing with here is not very serious.)

corollary_3_4.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.Basic
public import DomainTheory.ContinuousLattice.FunctionSpaces.corollary_3_4_eval_on_C
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottFunctionTopology
public import Mathlib.Data.Set.Basic
public import Mathlib.Topology.Defs.Basic

@[expose] public section
```

```
namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Corollary 3.4 (fixed `x`).** Evaluation at fixed `x` is continuous on
     `[D → D']`
(with Scott topologies on `D` and `D'`); this is one half of the separate-continuity
    input to
joint continuity. -/
theorem corollary_3_4 (x : D) :
    @Continuous (ScottC D D') D'
      (@scottFunctionTopology D D' scottTopologicalSpace scottTopologicalSpace)
      scottTopologicalSpace (fun f : ScottC D D' => f x) :=
  corollary_3_4_eval_on_C (D := D) (D' := D') x

end DomainTheory.ContinuousLattice
```

2

corollary_3_4.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.Basic
public import DomainTheory.ContinuousLattice.FunctionSpaces.corollary_3_4_eval_on_C
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottFunctionTopology
public import Mathlib.Data.Set.Basic
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Corollary 3.4 (fixed `x`).** Evaluation at fixed `x` is continuous on
     `[D → D']`
(with Scott topologies on `D` and `D'`); this is one half of the separate-continuity
    input to
joint continuity. -/
theorem corollary_3_4 (x : D) :
    @Continuous (ScottC D D') D'
      (@scottFunctionTopology D D' scottTopologicalSpace scottTopologicalSpace)
```

```
      scottTopologicalSpace (fun f : ScottC D D' => f x) :=
  corollary_3_4_eval_on_C (D := D) (D' := D') x

end DomainTheory.ContinuousLattice
```

3

corollary_3_4.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.Basic
public import DomainTheory.ContinuousLattice.FunctionSpaces.corollary_3_4_eval_on_C
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottFunctionTopology
public import Mathlib.Data.Set.Basic
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Corollary 3.4 (fixed `x`).** Evaluation at fixed `x` is continuous on
     `[D → D']`
(with Scott topologies on `D` and `D'`); this is one half of the separate-continuity
    input to
joint continuity. -/
theorem corollary_3_4 (x : D) :
    @Continuous (ScottC D D') D'
      (@scottFunctionTopology D D' scottTopologicalSpace scottTopologicalSpace)
      scottTopologicalSpace (fun f : ScottC D D' => f x) :=
  corollary_3_4_eval_on_C (D := D) (D' := D') x

end DomainTheory.ContinuousLattice
```

4

corollary_3_4.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.Basic
public import DomainTheory.ContinuousLattice.FunctionSpaces.corollary_3_4_eval_on_C
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottFunctionTopology
public import Mathlib.Data.Set.Basic
public import Mathlib.Topology.Defs.Basic

@[expose] public section
```

```
namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Corollary 3.4 (fixed `x`).** Evaluation at fixed `x` is continuous on
     `[D → D']`
(with Scott topologies on `D` and `D'`); this is one half of the separate-continuity
    input to
joint continuity. -/
theorem corollary_3_4 (x : D) :
    @Continuous (ScottC D D') D'
      (@scottFunctionTopology D D' scottTopologicalSpace scottTopologicalSpace)
      scottTopologicalSpace (fun f : ScottC D D' => f x) :=
  corollary_3_4_eval_on_C (D := D) (D' := D') x

end DomainTheory.ContinuousLattice
```

5

corollary_3_4.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.Basic
public import DomainTheory.ContinuousLattice.FunctionSpaces.corollary_3_4_eval_on_C
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottFunctionTopology
public import Mathlib.Data.Set.Basic
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Corollary 3.4 (fixed `x`).** Evaluation at fixed `x` is continuous on
     `[D → D']`
(with Scott topologies on `D` and `D'`); this is one half of the separate-continuity
    input to
joint continuity. -/
theorem corollary_3_4 (x : D) :
    @Continuous (ScottC D D') D'
      (@scottFunctionTopology D D' scottTopologicalSpace scottTopologicalSpace)
      scottTopologicalSpace (fun f : ScottC D D' => f x) :=
```

```
  corollary_3_4_eval_on_C (D := D) (D' := D') x

end DomainTheory.ContinuousLattice
```

6

corollary_3_4.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.Basic
public import DomainTheory.ContinuousLattice.FunctionSpaces.corollary_3_4_eval_on_C
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottFunctionTopology
public import Mathlib.Data.Set.Basic
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Corollary 3.4 (fixed `x`).** Evaluation at fixed `x` is continuous on
     `[D → D']`
(with Scott topologies on `D` and `D'`); this is one half of the separate-continuity
    input to
joint continuity. -/
theorem corollary_3_4 (x : D) :
    @Continuous (ScottC D D') D'
      (@scottFunctionTopology D D' scottTopologicalSpace scottTopologicalSpace)
      scottTopologicalSpace (fun f : ScottC D D' => f x) :=
  corollary_3_4_eval_on_C (D := D) (D' := D') x

end DomainTheory.ContinuousLattice
```

7

corollary_3_4.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.Basic
public import DomainTheory.ContinuousLattice.FunctionSpaces.corollary_3_4_eval_on_C
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottFunctionTopology
public import Mathlib.Data.Set.Basic
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice
```

```
open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Corollary 3.4 (fixed `x`).** Evaluation at fixed `x` is continuous on
     `[D → D']`
(with Scott topologies on `D` and `D'`); this is one half of the separate-continuity
    input to
joint continuity. -/
theorem corollary_3_4 (x : D) :
    @Continuous (ScottC D D') D'
      (@scottFunctionTopology D D' scottTopologicalSpace scottTopologicalSpace)
      scottTopologicalSpace (fun f : ScottC D D' => f x) :=
  corollary_3_4_eval_on_C (D := D) (D' := D') x

end DomainTheory.ContinuousLattice
```

8

corollary_3_4.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.Basic
public import DomainTheory.ContinuousLattice.FunctionSpaces.corollary_3_4_eval_on_C
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottFunctionTopology
public import Mathlib.Data.Set.Basic
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Corollary 3.4 (fixed `x`).** Evaluation at fixed `x` is continuous on
     `[D → D']`
(with Scott topologies on `D` and `D'`); this is one half of the separate-continuity
    input to
joint continuity. -/
theorem corollary_3_4 (x : D) :
    @Continuous (ScottC D D') D'
      (@scottFunctionTopology D D' scottTopologicalSpace scottTopologicalSpace)
      scottTopologicalSpace (fun f : ScottC D D' => f x) :=
  corollary_3_4_eval_on_C (D := D) (D' := D') x
```

```
end DomainTheory.ContinuousLattice
```

9

4.6 Products and sums of domains

Dependencies. Inside ProductsAndSumsOfDomains, each box is one subfolder, or one Lean file sitting directly in the folder. A box named as another folder in this chapter is that whole folder. A box named as another chapter is that whole chapter. The arrowhead points at the file or folder that imports, and the number is how many direct imports.

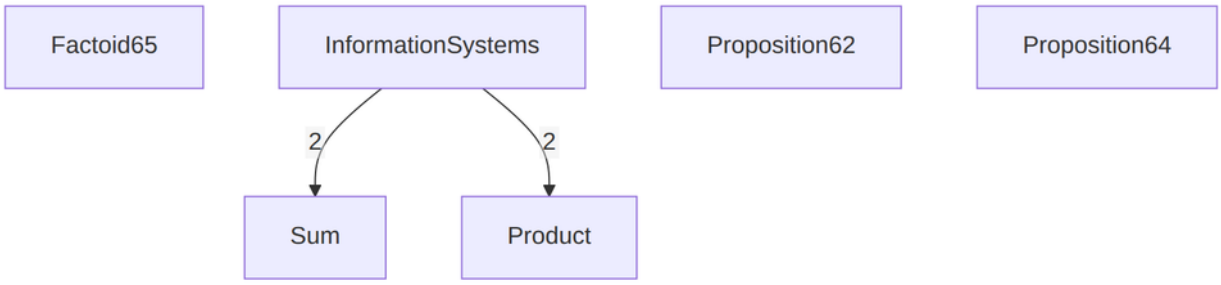


Figure 41: Products and sums of domains.

In the category of sets, the product (the cartesian product) of two sets is by definition just the set of ordered pairs of elements. We could use the same definition in the category of domains—provided we worked in terms of the elements of the domains. The disadvantage is that the determination of the product domain from data objects is lost, or at least pushed into the background. We shall therefore give a construction of products directly in terms of the given data objects, and then show how to prove that the product is just the one expected when we look at the elements.

The idea for the definition is a simple one. Think of two domains $\mathbf{A}$ and $\mathbf{B}$. A data object $X \in D_{\mathbf{A}}$ can be giving information about a possible element $x \in |\mathbf{A}|$, and a data object $Y \in D_{\mathbf{B}}$ can be giving information about a possible element $y \in |\mathbf{B}|$. How do we wish to give information about the pair (x, y)? The first piece of advice is not to say all that is known all at once. For example, if pressed, we can say we know X about the first coordinate of the pair; only later, if really pressed, we can say we also know Y about the second coordinate. The point is that data objects provide only partial information, and it does not matter if it takes several of them to give fairly complete information about the whole element. This attitude motivates, then, the particular form of our official definition.

- Definition 6.1
- Proposition 6.2
- Definition 6.3
- Proposition 6.4

It can also be shown that Propositions 6.2 and 6.4 uniquely characterize the domains $\mathbf{A} \times \mathbf{B}$ and $\mathbf{A} + \mathbf{B}$ up to isomorphism, and they give us the existence of additional mappings that are needed to show that

product and sum are functors on the category of domains. We can also show from these results that the domain

$$\mathbf{BOOL} = \mathbf{1} + \mathbf{1}$$

has two elements true and false, such that any mapping on $\mathbf{BOOL}$ is uniquely determined by its action on true, false and $\perp_{\mathbf{BOOL}}$, and the values on the first two elements may be arbitrarily chosen.

corollary_3_4_eval_on_C.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.Basic
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottFunctionSubbasis_isOpen_scott
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottFunctionTopology
public import Mathlib.Data.Set.Basic
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem corollary_3_4_eval_on_C (x : D) :
    @Continuous (ScottC D D') D'
      (@scottFunctionTopology D D' scottTopologicalSpace scottTopologicalSpace)
      scottTopologicalSpace (fun f : ScottC D D' => f x) :=
  continuous_def.2 fun _U hU => scottFunctionSubbasis_isOpen_scott (D := D) (D' := D')
    (x := x) hU

end DomainTheory.ContinuousLattice
```

0

corollary_3_4_eval_on_C.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.Basic
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottFunctionSubbasis_isOpen_scott
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottFunctionTopology
public import Mathlib.Data.Set.Basic
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice
```

```
open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem corollary_3_4_eval_on_C (x : D) :
    @Continuous (ScottC D D') D'
      (@scottFunctionTopology D D' scottTopologicalSpace scottTopologicalSpace)
      scottTopologicalSpace (fun f : ScottC D D' => f x) :=
  continuous_def.2 fun _U hU => scottFunctionSubbasis_isOpen_scott (D := D) (D' := D')
     (x := x) hU

end DomainTheory.ContinuousLattice
```

1

corollary_3_4_eval_on_C.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.Basic
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottFunctionSubbasis_isOpen_scott
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottFunctionTopology
public import Mathlib.Data.Set.Basic
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem corollary_3_4_eval_on_C (x : D) :
    @Continuous (ScottC D D') D'
      (@scottFunctionTopology D D' scottTopologicalSpace scottTopologicalSpace)
      scottTopologicalSpace (fun f : ScottC D D' => f x) :=
  continuous_def.2 fun _U hU => scottFunctionSubbasis_isOpen_scott (D := D) (D' := D')
     (x := x) hU

end DomainTheory.ContinuousLattice
```

2

corollary_3_4_eval_on_C.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.Basic
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottFunctionSubbasis_isOpen_scott
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottFunctionTopology
public import Mathlib.Data.Set.Basic
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem corollary_3_4_eval_on_C (x : D) :
    @Continuous (ScottC D D') D'
      (@scottFunctionTopology D D' scottTopologicalSpace scottTopologicalSpace)
      scottTopologicalSpace (fun f : ScottC D D' => f x) :=
  continuous_def.2 fun _U hU => scottFunctionSubbasis_isOpen_scott (D := D) (D' := D')
    (x := x) hU

end DomainTheory.ContinuousLattice
```

3

corollary_3_4_eval_on_C.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.Basic
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottFunctionSubbasis_isOpen_scott
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottFunctionTopology
public import Mathlib.Data.Set.Basic
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem corollary_3_4_eval_on_C (x : D) :
```

```
    @Continuous (ScottC D D') D'
      (@scottFunctionTopology D D' scottTopologicalSpace scottTopologicalSpace)
      scottTopologicalSpace (fun f : ScottC D D' => f x) :=
  continuous_def.2 fun _U hU => scottFunctionSubbasis_isOpen_scott (D := D) (D' := D')
    (x := x) hU

end DomainTheory.ContinuousLattice
```

4

corollary_3_4_eval_on_C.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.Basic
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottFunctionSubbasis_isOpen_scott
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottFunctionTopology
public import Mathlib.Data.Set.Basic
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem corollary_3_4_eval_on_C (x : D) :
    @Continuous (ScottC D D') D'
      (@scottFunctionTopology D D' scottTopologicalSpace scottTopologicalSpace)
      scottTopologicalSpace (fun f : ScottC D D' => f x) :=
  continuous_def.2 fun _U hU => scottFunctionSubbasis_isOpen_scott (D := D) (D' := D')
    (x := x) hU

end DomainTheory.ContinuousLattice
```

5

corollary_3_4_eval_on_C.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.Basic
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottFunctionSubbasis_isOpen_scott
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottFunctionTopology
public import Mathlib.Data.Set.Basic
public import Mathlib.Topology.Defs.Basic

@[expose] public section
```

```
namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem corollary_3_4_eval_on_C (x : D) :
    @Continuous (ScottC D D') D'
      (@scottFunctionTopology D D' scottTopologicalSpace scottTopologicalSpace)
      scottTopologicalSpace (fun f : ScottC D D' => f x) :=
  continuous_def.2 fun _U hU => scottFunctionSubbasis_isOpen_scott (D := D) (D' := D')
    (x := x) hU

end DomainTheory.ContinuousLattice
```

6

corollary_3_4_eval_on_C.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.Basic
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottFunctionSubbasis_isOpen_scott
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottFunctionTopology
public import Mathlib.Data.Set.Basic
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem corollary_3_4_eval_on_C (x : D) :
    @Continuous (ScottC D D') D'
      (@scottFunctionTopology D D' scottTopologicalSpace scottTopologicalSpace)
      scottTopologicalSpace (fun f : ScottC D D' => f x) :=
  continuous_def.2 fun _U hU => scottFunctionSubbasis_isOpen_scott (D := D) (D' := D')
    (x := x) hU

end DomainTheory.ContinuousLattice
```

7

corollary_3_4_eval_on_C.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.Basic
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottFunctionSubbasis_isOpen_scott
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottFunctionTopology
public import Mathlib.Data.Set.Basic
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem corollary_3_4_eval_on_C (x : D) :
    @Continuous (ScottC D D') D'
      (@scottFunctionTopology D D' scottTopologicalSpace scottTopologicalSpace)
      scottTopologicalSpace (fun f : ScottC D D' => f x) :=
  continuous_def.2 fun _U hU => scottFunctionSubbasis_isOpen_scott (D := D) (D' := D')
    (x := x) hU

end DomainTheory.ContinuousLattice
```


corollary_3_4_eval_on_C.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.Basic
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottFunctionSubbasis_isOpen_scott
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottFunctionTopology
public import Mathlib.Data.Set.Basic
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']
```

```
theorem corollary_3_4_eval_on_C (x : D) :
    @Continuous (ScottC D D') D'
      (@scottFunctionTopology D D' scottTopologicalSpace scottTopologicalSpace)
      scottTopologicalSpace (fun f : ScottC D D' => f x) :=
  continuous_def.2 fun _U hU => scottFunctionSubbasis_isOpen_scott (D := D) (D' := D')
    (x := x) hU

end DomainTheory.ContinuousLattice
```

9

corollary_3_4_jointly_continuous.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    corollary_3_4_preservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Corollary 3.4.** The evaluation map `eval : [D → D'] × D → D'`, `(f
    , x) ↦ f x`,
is (jointly) Scott-continuous. Via Theorem 3.3(b) and Proposition 2.9(b) the Scott
    topology of the
product lattice is the product of the pointwise topology on `[D → D']` and the Scott
    topology on
`D`, so this is exactly joint continuity for Scott's product topology. -/
theorem corollary_3_4_jointly_continuous :
    @Continuous (ScottMap D D' × D) D' scottTopologicalSpace scottTopologicalSpace
      (fun p : ScottMap D D' × D => (p.1 : D → D') p.2) :=
  continuous_of_preservesDirectedSup corollary_3_4_preservesDirectedSup

end DomainTheory.ContinuousLattice
```

0

corollary_3_4_jointly_continuous.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    corollary_3_4_preservesDirectedSup
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Corollary 3.4.** The evaluation map `eval : [D → D'] × D → D'`, `(f
    , x) ↦ f x`,
is (jointly) Scott-continuous. Via Theorem 3.3(b) and Proposition 2.9(b) the Scott
    topology of the
product lattice is the product of the pointwise topology on `[D → D']` and the Scott
    topology on
`D`, so this is exactly joint continuity for Scott's product topology. -/
theorem corollary_3_4_jointly_continuous :
    @Continuous (ScottMap D D' × D) D' scottTopologicalSpace scottTopologicalSpace
      (fun p : ScottMap D D' × D => (p.1 : D → D') p.2) :=
  continuous_of_preservesDirectedSup corollary_3_4_preservesDirectedSup

end DomainTheory.ContinuousLattice
```

1

corollary_3_4_jointly_continuous.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    corollary_3_4_preservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']
```

```
/-- **Scott 1972, Corollary 3.4.** The evaluation map `eval : [D → D'] × D → D'`, `(f
    , x) ↦ f x`,
is (jointly) Scott-continuous. Via Theorem 3.3(b) and Proposition 2.9(b) the Scott
    topology of the
product lattice is the product of the pointwise topology on `[D → D']` and the Scott
    topology on
`D`, so this is exactly joint continuity for Scott's product topology. -/
theorem corollary_3_4_jointly_continuous :
    @Continuous (ScottMap D D' × D) D' scottTopologicalSpace scottTopologicalSpace
      (fun p : ScottMap D D' × D => (p.1 : D → D') p.2) :=
  continuous_of_preservesDirectedSup corollary_3_4_preservesDirectedSup

end DomainTheory.ContinuousLattice
```

2

corollary_3_4_jointly_continuous.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    corollary_3_4_preservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Corollary 3.4.** The evaluation map `eval : [D → D'] × D → D'`, `(f
    , x) ↦ f x`,
is (jointly) Scott-continuous. Via Theorem 3.3(b) and Proposition 2.9(b) the Scott
    topology of the
product lattice is the product of the pointwise topology on `[D → D']` and the Scott
    topology on
`D`, so this is exactly joint continuity for Scott's product topology. -/
theorem corollary_3_4_jointly_continuous :
    @Continuous (ScottMap D D' × D) D' scottTopologicalSpace scottTopologicalSpace
      (fun p : ScottMap D D' × D => (p.1 : D → D') p.2) :=
  continuous_of_preservesDirectedSup corollary_3_4_preservesDirectedSup

end DomainTheory.ContinuousLattice
```

3

corollary_3_4_jointly_continuous.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    corollary_3_4_preservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Corollary 3.4.** The evaluation map `eval : [D → D'] × D → D'`, `(f
    , x) ↦ f x`,
is (jointly) Scott-continuous. Via Theorem 3.3(b) and Proposition 2.9(b) the Scott
    topology of the
product lattice is the product of the pointwise topology on `[D → D']` and the Scott
    topology on
`D`, so this is exactly joint continuity for Scott's product topology. -/
theorem corollary_3_4_jointly_continuous :
    @Continuous (ScottMap D D' × D) D' scottTopologicalSpace scottTopologicalSpace
      (fun p : ScottMap D D' × D => (p.1 : D → D') p.2) :=
  continuous_of_preservesDirectedSup corollary_3_4_preservesDirectedSup

end DomainTheory.ContinuousLattice
```

4

corollary_3_4_jointly_continuous.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    corollary_3_4_preservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace
```

```
variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Corollary 3.4.** The evaluation map `eval : [D → D'] × D → D'`, `(f
    , x) ↦ f x`,
is (jointly) Scott-continuous. Via Theorem 3.3(b) and Proposition 2.9(b) the Scott
    topology of the
product lattice is the product of the pointwise topology on `[D → D']` and the Scott
    topology on
`D`, so this is exactly joint continuity for Scott's product topology. -/
theorem corollary_3_4_jointly_continuous :
    @Continuous (ScottMap D D' × D) D' scottTopologicalSpace scottTopologicalSpace
      (fun p : ScottMap D D' × D => (p.1 : D → D') p.2) :=
  continuous_of_preservesDirectedSup corollary_3_4_preservesDirectedSup

end DomainTheory.ContinuousLattice
```

5

corollary_3_4_jointly_continuous.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    corollary_3_4_preservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Corollary 3.4.** The evaluation map `eval : [D → D'] × D → D'`, `(f
    , x) ↦ f x`,
is (jointly) Scott-continuous. Via Theorem 3.3(b) and Proposition 2.9(b) the Scott
    topology of the
product lattice is the product of the pointwise topology on `[D → D']` and the Scott
    topology on
`D`, so this is exactly joint continuity for Scott's product topology. -/
theorem corollary_3_4_jointly_continuous :
    @Continuous (ScottMap D D' × D) D' scottTopologicalSpace scottTopologicalSpace
      (fun p : ScottMap D D' × D => (p.1 : D → D') p.2) :=
  continuous_of_preservesDirectedSup corollary_3_4_preservesDirectedSup
```

```
end DomainTheory.ContinuousLattice
```

6

corollary_3_4_jointly_continuous.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    corollary_3_4_preservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Corollary 3.4.** The evaluation map `eval : [D → D'] × D → D'`, `(f
    , x) ↦ f x`,
is (jointly) Scott-continuous. Via Theorem 3.3(b) and Proposition 2.9(b) the Scott
    topology of the
product lattice is the product of the pointwise topology on `[D → D']` and the Scott
    topology on
`D`, so this is exactly joint continuity for Scott's product topology. -/
theorem corollary_3_4_jointly_continuous :
    @Continuous (ScottMap D D' × D) D' scottTopologicalSpace scottTopologicalSpace
      (fun p : ScottMap D D' × D => (p.1 : D → D') p.2) :=
  continuous_of_preservesDirectedSup corollary_3_4_preservesDirectedSup

end DomainTheory.ContinuousLattice
```

7

corollary_3_4_jointly_continuous.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    corollary_3_4_preservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section
```

```
namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Corollary 3.4.** The evaluation map `eval : [D → D'] × D → D'`, `(f
    , x) ↦ f x`,
is (jointly) Scott-continuous. Via Theorem 3.3(b) and Proposition 2.9(b) the Scott
    topology of the
product lattice is the product of the pointwise topology on `[D → D']` and the Scott
    topology on
`D`, so this is exactly joint continuity for Scott's product topology. -/
theorem corollary_3_4_jointly_continuous :
    @Continuous (ScottMap D D' × D) D' scottTopologicalSpace scottTopologicalSpace
      (fun p : ScottMap D D' × D => (p.1 : D → D') p.2) :=
  continuous_of_preservesDirectedSup corollary_3_4_preservesDirectedSup

end DomainTheory.ContinuousLattice
```

8

corollary_3_4_jointly_continuous.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    corollary_3_4_preservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Corollary 3.4.** The evaluation map `eval : [D → D'] × D → D'`, `(f
    , x) ↦ f x`,
is (jointly) Scott-continuous. Via Theorem 3.3(b) and Proposition 2.9(b) the Scott
    topology of the
product lattice is the product of the pointwise topology on `[D → D']` and the Scott
    topology on
```

```
`D`, so this is exactly joint continuity for Scott's product topology. -/
theorem corollary_3_4_jointly_continuous :
    @Continuous (ScottMap D D' × D) D' scottTopologicalSpace scottTopologicalSpace
      (fun p : ScottMap D D' × D => (p.1 : D → D') p.2) :=
  continuous_of_preservesDirectedSup corollary_3_4_preservesDirectedSup

end DomainTheory.ContinuousLattice
```

9

corollary_3_4_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.Basic
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.ScottMaps.proposition_2_6
public import DomainTheory.ContinuousLattice.ScottMaps.proposition_2_5
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Corollary 3.4 (joint continuity, core).** Evaluation `[D → D'] × D
    → D'`,
`(f, x) ↦ f x`, preserves directed suprema. By Proposition 2.6 it suffices to check
    separate
Scott-continuity: in `x` (with `f` fixed) it is `f`'s own continuity, and in `f` (with `
    x` fixed)
it is the pointwise formula for suprema in `[D → D']` (`ScottMap.sSup_apply`). -/
theorem corollary_3_4_preservesDirectedSup :
    PreservesDirectedSup (fun p : ScottMap D D' × D => (p.1 : D → D') p.2) := by
  rw [proposition_2_6]
  constructor
  · intro y S hS hSdir
    show ((sSup S : ScottMap D D') : D → D') y
        = sSup ((fun x : ScottMap D D' => (x : D → D') y) '' S)
    rw [ScottMap.sSup_apply]
  · intro x
    exact (proposition_2_5 _).mp x.continuous

end DomainTheory.ContinuousLattice
```

0

corollary_3_4_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.Basic
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.ScottMaps.proposition_2_6
public import DomainTheory.ContinuousLattice.ScottMaps.proposition_2_5
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Corollary 3.4 (joint continuity, core).** Evaluation `[D → D'] × D
    → D'`,
`(f, x) ↦ f x`, preserves directed suprema. By Proposition 2.6 it suffices to check
    separate
Scott-continuity: in `x` (with `f` fixed) it is `f`'s own continuity, and in `f` (with `
    x` fixed)
it is the pointwise formula for suprema in `[D → D']` (`ScottMap.sSup_apply`). -/
theorem corollary_3_4_preservesDirectedSup :
    PreservesDirectedSup (fun p : ScottMap D D' × D => (p.1 : D → D') p.2) := by
  rw [proposition_2_6]
  constructor
  · intro y S hS hSdir
    show ((sSup S : ScottMap D D') : D → D') y
        = sSup ((fun x : ScottMap D D' => (x : D → D') y) '' S)
    rw [ScottMap.sSup_apply]
  · intro x
    exact (proposition_2_5 _).mp x.continuous

end DomainTheory.ContinuousLattice
```

1

corollary_3_4_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.Basic
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.ScottMaps.proposition_2_6
```

```
public import DomainTheory.ContinuousLattice.ScottMaps.proposition_2_5
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Corollary 3.4 (joint continuity, core).** Evaluation `[D → D'] × D
    → D'`,
`(f, x) ↦ f x`, preserves directed suprema. By Proposition 2.6 it suffices to check
    separate
Scott-continuity: in `x` (with `f` fixed) it is `f`'s own continuity, and in `f` (with `
    x` fixed)
it is the pointwise formula for suprema in `[D → D']` (`ScottMap.sSup_apply`). -/
theorem corollary_3_4_preservesDirectedSup :
    PreservesDirectedSup (fun p : ScottMap D D' × D => (p.1 : D → D') p.2) := by
  rw [proposition_2_6]
  constructor
  · intro y S hS hSdir
    show ((sSup S : ScottMap D D') : D → D') y
        = sSup ((fun x : ScottMap D D' => (x : D → D') y) '' S)
    rw [ScottMap.sSup_apply]
  · intro x
    exact (proposition_2_5 _).mp x.continuous

end DomainTheory.ContinuousLattice
```

2

corollary_3_4_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.Basic
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.ScottMaps.proposition_2_6
public import DomainTheory.ContinuousLattice.ScottMaps.proposition_2_5
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace
```

```
variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Corollary 3.4 (joint continuity, core).** Evaluation `[D → D'] × D
    → D'`,
`(f, x) ↦ f x`, preserves directed suprema. By Proposition 2.6 it suffices to check
    separate
Scott-continuity: in `x` (with `f` fixed) it is `f`'s own continuity, and in `f` (with `
    x` fixed)
it is the pointwise formula for suprema in `[D → D']` (`ScottMap.sSup_apply`). -/
theorem corollary_3_4_preservesDirectedSup :
    PreservesDirectedSup (fun p : ScottMap D D' × D => (p.1 : D → D') p.2) := by
  rw [proposition_2_6]
  constructor
  · intro y S hS hSdir
    show ((sSup S : ScottMap D D') : D → D') y
        = sSup ((fun x : ScottMap D D' => (x : D → D') y) '' S)
    rw [ScottMap.sSup_apply]
  · intro x
    exact (proposition_2_5 _).mp x.continuous

end DomainTheory.ContinuousLattice
```

3

corollary_3_4_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.Basic
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.ScottMaps.proposition_2_6
public import DomainTheory.ContinuousLattice.ScottMaps.proposition_2_5
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Corollary 3.4 (joint continuity, core).** Evaluation `[D → D'] × D
    → D'`,
```

```
`(f, x) ↦ f x`, preserves directed suprema. By Proposition 2.6 it suffices to check
    separate
Scott-continuity: in `x` (with `f` fixed) it is `f`'s own continuity, and in `f` (with `
    x` fixed)
it is the pointwise formula for suprema in `[D → D']` (`ScottMap.sSup_apply`). -/
theorem corollary_3_4_preservesDirectedSup :
    PreservesDirectedSup (fun p : ScottMap D D' × D => (p.1 : D → D') p.2) := by
  rw [proposition_2_6]
  constructor
  · intro y S hS hSdir
    show ((sSup S : ScottMap D D') : D → D') y
        = sSup ((fun x : ScottMap D D' => (x : D → D') y) '' S)
    rw [ScottMap.sSup_apply]
  · intro x
    exact (proposition_2_5 _).mp x.continuous

end DomainTheory.ContinuousLattice
```

4

corollary_3_4_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.Basic
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.ScottMaps.proposition_2_6
public import DomainTheory.ContinuousLattice.ScottMaps.proposition_2_5
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Corollary 3.4 (joint continuity, core).** Evaluation `[D → D'] × D
    → D'`,
`(f, x) ↦ f x`, preserves directed suprema. By Proposition 2.6 it suffices to check
    separate
Scott-continuity: in `x` (with `f` fixed) it is `f`'s own continuity, and in `f` (with `
    x` fixed)
it is the pointwise formula for suprema in `[D → D']` (`ScottMap.sSup_apply`). -/
theorem corollary_3_4_preservesDirectedSup :
    PreservesDirectedSup (fun p : ScottMap D D' × D => (p.1 : D → D') p.2) := by
  rw [proposition_2_6]
  constructor
```

```
  · intro y S hS hSdir
    show ((sSup S : ScottMap D D') : D → D') y
        = sSup ((fun x : ScottMap D D' => (x : D → D') y) '' S)
    rw [ScottMap.sSup_apply]
  · intro x
    exact (proposition_2_5 _).mp x.continuous

end DomainTheory.ContinuousLattice
```

5

corollary_3_4_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.Basic
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.ScottMaps.proposition_2_6
public import DomainTheory.ContinuousLattice.ScottMaps.proposition_2_5
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Corollary 3.4 (joint continuity, core).** Evaluation `[D → D'] × D
    → D'`,
`(f, x) ↦ f x`, preserves directed suprema. By Proposition 2.6 it suffices to check
    separate
Scott-continuity: in `x` (with `f` fixed) it is `f`'s own continuity, and in `f` (with `
    x` fixed)
it is the pointwise formula for suprema in `[D → D']` (`ScottMap.sSup_apply`). -/
theorem corollary_3_4_preservesDirectedSup :
    PreservesDirectedSup (fun p : ScottMap D D' × D => (p.1 : D → D') p.2) := by
  rw [proposition_2_6]
  constructor
  · intro y S hS hSdir
    show ((sSup S : ScottMap D D') : D → D') y
        = sSup ((fun x : ScottMap D D' => (x : D → D') y) '' S)
    rw [ScottMap.sSup_apply]
  · intro x
    exact (proposition_2_5 _).mp x.continuous

end DomainTheory.ContinuousLattice
```

6

corollary_3_4_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.Basic
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.ScottMaps.proposition_2_6
public import DomainTheory.ContinuousLattice.ScottMaps.proposition_2_5
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Corollary 3.4 (joint continuity, core).** Evaluation `[D → D'] × D
    → D'`,
`(f, x) ↦ f x`, preserves directed suprema. By Proposition 2.6 it suffices to check
    separate
Scott-continuity: in `x` (with `f` fixed) it is `f`'s own continuity, and in `f` (with `
    x` fixed)
it is the pointwise formula for suprema in `[D → D']` (`ScottMap.sSup_apply`). -/
theorem corollary_3_4_preservesDirectedSup :
    PreservesDirectedSup (fun p : ScottMap D D' × D => (p.1 : D → D') p.2) := by
  rw [proposition_2_6]
  constructor
  · intro y S hS hSdir
    show ((sSup S : ScottMap D D') : D → D') y
        = sSup ((fun x : ScottMap D D' => (x : D → D') y) '' S)
    rw [ScottMap.sSup_apply]
  · intro x
    exact (proposition_2_5 _).mp x.continuous

end DomainTheory.ContinuousLattice
```

7

corollary_3_4_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.Basic
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.ScottMaps.proposition_2_6
public import DomainTheory.ContinuousLattice.ScottMaps.proposition_2_5
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Corollary 3.4 (joint continuity, core).** Evaluation `[D → D'] × D
    → D'`,
`(f, x) ↦ f x`, preserves directed suprema. By Proposition 2.6 it suffices to check
    separate
Scott-continuity: in `x` (with `f` fixed) it is `f`'s own continuity, and in `f` (with `
    x` fixed)
it is the pointwise formula for suprema in `[D → D']` (`ScottMap.sSup_apply`). -/
theorem corollary_3_4_preservesDirectedSup :
    PreservesDirectedSup (fun p : ScottMap D D' × D => (p.1 : D → D') p.2) := by
  rw [proposition_2_6]
  constructor
  · intro y S hS hSdir
    show ((sSup S : ScottMap D D') : D → D') y
        = sSup ((fun x : ScottMap D D' => (x : D → D') y) '' S)
    rw [ScottMap.sSup_apply]
  · intro x
    exact (proposition_2_5 _).mp x.continuous

end DomainTheory.ContinuousLattice
```

8

corollary_3_4_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.Basic
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.ScottMaps.proposition_2_6
public import DomainTheory.ContinuousLattice.ScottMaps.proposition_2_5
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice
```

```
open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Corollary 3.4 (joint continuity, core).** Evaluation `[D → D'] × D
    → D'`,
`(f, x) ↦ f x`, preserves directed suprema. By Proposition 2.6 it suffices to check
    separate
Scott-continuity: in `x` (with `f` fixed) it is `f`'s own continuity, and in `f` (with `
    x` fixed)
it is the pointwise formula for suprema in `[D → D']` (`ScottMap.sSup_apply`). -/
theorem corollary_3_4_preservesDirectedSup :
    PreservesDirectedSup (fun p : ScottMap D D' × D => (p.1 : D → D') p.2) := by
  rw [proposition_2_6]
  constructor
  · intro y S hS hSdir
    show ((sSup S : ScottMap D D') : D → D') y
        = sSup ((fun x : ScottMap D D' => (x : D → D') y) '' S)
    rw [ScottMap.sSup_apply]
  · intro x
    exact (proposition_2_5 _).mp x.continuous

end DomainTheory.ContinuousLattice
```

9

scottFunctionEvalSubbasis.lean

```
module
public import Mathlib.Topology.ContinuousMap.Defs
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
```

```
variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- Subbasic sets for Scott's function-space topology (pointwise convergence). -/
def scottFunctionEvalSubbasis (X Y : Type*) [TopologicalSpace X] [TopologicalSpace Y]
    :
    Set (Set (C(X, Y))) :=
  { W | ∃ (x : X) (U : Set Y), IsOpen U ∧ W = {f : C(X, Y) | f x ∈ U} }

end DomainTheory.ContinuousLattice
```

0

scottFunctionEvalSubbasis.lean

```
module
public import Mathlib.Topology.ContinuousMap.Defs
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- Subbasic sets for Scott's function-space topology (pointwise convergence). -/
def scottFunctionEvalSubbasis (X Y : Type*) [TopologicalSpace X] [TopologicalSpace Y]
    :
    Set (Set (C(X, Y))) :=
  { W | ∃ (x : X) (U : Set Y), IsOpen U ∧ W = {f : C(X, Y) | f x ∈ U} }

end DomainTheory.ContinuousLattice
```

1

scottFunctionEvalSubbasis.lean

```
module
public import Mathlib.Topology.ContinuousMap.Defs
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- Subbasic sets for Scott's function-space topology (pointwise convergence). -/
def scottFunctionEvalSubbasis (X Y : Type*) [TopologicalSpace X] [TopologicalSpace Y]
    :
    Set (Set (C(X, Y))) :=
  { W | ∃ (x : X) (U : Set Y), IsOpen U ∧ W = {f : C(X, Y) | f x ∈ U} }

end DomainTheory.ContinuousLattice
```

2

scottFunctionEvalSubbasis.lean

```
module
public import Mathlib.Topology.ContinuousMap.Defs
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
```

```
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- Subbasic sets for Scott's function-space topology (pointwise convergence). -/
def scottFunctionEvalSubbasis (X Y : Type*) [TopologicalSpace X] [TopologicalSpace Y]
    :
    Set (Set (C(X, Y))) :=
  { W | ∃ (x : X) (U : Set Y), IsOpen U ∧ W = {f : C(X, Y) | f x ∈ U} }

end DomainTheory.ContinuousLattice
```

3

scottFunctionEvalSubbasis.lean

```
module
public import Mathlib.Topology.ContinuousMap.Defs
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]
```

```
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- Subbasic sets for Scott's function-space topology (pointwise convergence). -/
def scottFunctionEvalSubbasis (X Y : Type*) [TopologicalSpace X] [TopologicalSpace Y]
    :
    Set (Set (C(X, Y))) :=
  { W | ∃ (x : X) (U : Set Y), IsOpen U ∧ W = {f : C(X, Y) | f x ∈ U} }

end DomainTheory.ContinuousLattice
```

4

scottFunctionEvalSubbasis.lean

```
module
public import Mathlib.Topology.ContinuousMap.Defs
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- Subbasic sets for Scott's function-space topology (pointwise convergence). -/
def scottFunctionEvalSubbasis (X Y : Type*) [TopologicalSpace X] [TopologicalSpace Y]
    :
    Set (Set (C(X, Y))) :=
  { W | ∃ (x : X) (U : Set Y), IsOpen U ∧ W = {f : C(X, Y) | f x ∈ U} }

end DomainTheory.ContinuousLattice
```

5

scottFunctionEvalSubbasis.lean

```
module
public import Mathlib.Topology.ContinuousMap.Defs
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- Subbasic sets for Scott's function-space topology (pointwise convergence). -/
def scottFunctionEvalSubbasis (X Y : Type*) [TopologicalSpace X] [TopologicalSpace Y]
    :
    Set (Set (C(X, Y))) :=
  { W | ∃ (x : X) (U : Set Y), IsOpen U ∧ W = {f : C(X, Y) | f x ∈ U} }

end DomainTheory.ContinuousLattice
```

6

4.7 The function space as a domain

Dependencies. Inside FunctionSpaceAsADomain, each box is one subfolder, or one Lean file sitting directly in the folder. A box named as another folder in this chapter is that whole folder. A box named as another chapter is that whole chapter. The arrowhead points at the file or folder that imports, and the number is how many direct imports.

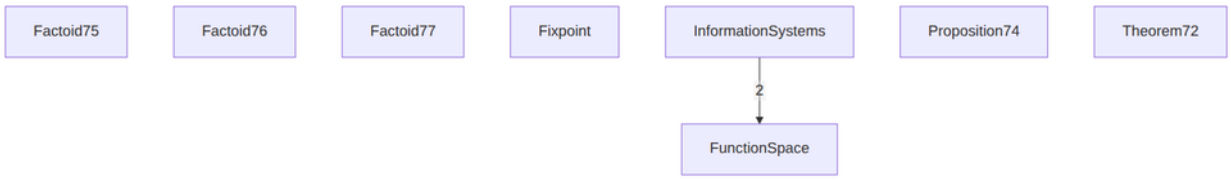


Figure 42: The function space as a domain.

Functions or mappings between domains are of basic importance for our theory, since it is through them that we most easily transform data and relate the structures into which the elements defined by the data objects enter. There are many possible functions, and large groups of them can be treated in a uniform manner. For instance, if the source and target domains match properly, any pair of functions can be composed—composition is an operation on functions of general significance. Now, if in the theory we could combine functions into domains themselves, then an operation like composition might become a mapping of the theory. Indeed, this is exactly what happens: suitably construed, composition is an approximable mapping of two arguments. Of course, for each configuration of linked source and target domains, there is a separate composition operation.

In order to make approximable mappings elements of a suitable domain, we have to discover first what their appropriate data objects are. In Section 5 this was hinted at already. To determine an approximable mapping $f : \mathbf{A} \to \mathbf{B}$, we have to say which pairs (u, v) with $u \in D_{\mathbf{A}}$ and $v \in D_{\mathbf{B}}$ stand in the mapping relation $u\ f\ v$. One such pair gives a certain (finite) amount of information about the possible functions that contain it, and an approximable mapping is completely determined by such pairs. Therefore, if there are appropriate notions of consistency and entailment for these pairs, we will be able to form a domain having functions as elements. Let us try out a formal definition first, and then look to an explanation of how it works.

- Definition 7.1
- Theorem 7.2
- Theorem 7.3
- Proposition 7.4

scottFunctionEvalSubbasis.lean

```
module
public import Mathlib.Topology.ContinuousMap.Defs
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
```

```
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- Subbasic sets for Scott's function-space topology (pointwise convergence). -/
def scottFunctionEvalSubbasis (X Y : Type*) [TopologicalSpace X] [TopologicalSpace Y]
    :
    Set (Set (C(X, Y))) :=
  { W | ∃ (x : X) (U : Set Y), IsOpen U ∧ W = {f : C(X, Y) | f x ∈ U} }

end DomainTheory.ContinuousLattice
```

7

scottFunctionEvalSubbasis.lean

```
module
public import Mathlib.Topology.ContinuousMap.Defs
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
```

```
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- Subbasic sets for Scott's function-space topology (pointwise convergence). -/
def scottFunctionEvalSubbasis (X Y : Type*) [TopologicalSpace X] [TopologicalSpace Y]
    :
    Set (Set (C(X, Y))) :=
  { W | ∃ (x : X) (U : Set Y), IsOpen U ∧ W = {f : C(X, Y) | f x ∈ U} }

end DomainTheory.ContinuousLattice
```

8

scottFunctionEvalSubbasis.lean

```
module
public import Mathlib.Topology.ContinuousMap.Defs
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- Subbasic sets for Scott's function-space topology (pointwise convergence). -/
def scottFunctionEvalSubbasis (X Y : Type*) [TopologicalSpace X] [TopologicalSpace Y]
    :
    Set (Set (C(X, Y))) :=
  { W | ∃ (x : X) (U : Set Y), IsOpen U ∧ W = {f : C(X, Y) | f x ∈ U} }

end DomainTheory.ContinuousLattice
```

9

curry_left_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_image_prod_mk_right
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- Currying in the `x`-variable: `x ↦ f (x, y)` is Scott-continuous (used for
    continuity of
`lambda f` as a map `D → [D' → D'']`). -/
theorem curry_left_preservesDirectedSup (f : ScottMap (D × D') D'') (y : D') :
    PreservesDirectedSup (fun x => f (x, y)) := by
  intro S hS hSdir
  have hS' : DirectedOn (· ≤ ·) (Set.image (fun x => (x, y)) S) := by
    intro p hp q hq
    obtain ⟨a, ha, rfl⟩ := hp
    obtain ⟨b, hb, rfl⟩ := hq
    obtain ⟨c, hc, hac, hbc⟩ := hSdir a ha b hb
    exact ⟨(c, y), Set.mem_image_of_mem _ hc, And.intro (And.intro hac le_rfl) (And.
    intro hbc le_rfl)⟩
  have hS'ne : (Set.image (fun x => (x, y)) S).Nonempty := by
    obtain ⟨s, hs⟩ := hS
    exact ⟨(s, y), s, hs, rfl⟩
  rw [show (fun x => f (x, y)) (sSup S) = f (sSup S, y) from rfl,
    ← sSup_image_prod_mk_right y S hS,
    f.preservesDirectedSup_coe (Set.image (fun x => (x, y)) S) hS'ne hS']
  congr 1
  simp [Set.image_image]

end DomainTheory.ContinuousLattice
```

0

curry_left_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_image_prod_mk_right
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- Currying in the `x`-variable: `x ↦ f (x, y)` is Scott-continuous (used for
    continuity of
`lambda f` as a map `D → [D' → D'']`). -/
theorem curry_left_preservesDirectedSup (f : ScottMap (D × D') D'') (y : D') :
    PreservesDirectedSup (fun x => f (x, y)) := by
  intro S hS hSdir
  have hS' : DirectedOn (· ≤ ·) (Set.image (fun x => (x, y)) S) := by
    intro p hp q hq
    obtain ⟨a, ha, rfl⟩ := hp
    obtain ⟨b, hb, rfl⟩ := hq
    obtain ⟨c, hc, hac, hbc⟩ := hSdir a ha b hb
    exact ⟨(c, y), Set.mem_image_of_mem _ hc, And.intro (And.intro hac le_rfl) (And.
    intro hbc le_rfl)⟩
  have hS'ne : (Set.image (fun x => (x, y)) S).Nonempty := by
    obtain ⟨s, hs⟩ := hS
    exact ⟨(s, y), s, hs, rfl⟩
  rw [show (fun x => f (x, y)) (sSup S) = f (sSup S, y) from rfl,
    ← sSup_image_prod_mk_right y S hS,
    f.preservesDirectedSup_coe (Set.image (fun x => (x, y)) S) hS'ne hS']
  congr 1
  simp [Set.image_image]

end DomainTheory.ContinuousLattice
```

1

curry_left_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_image_prod_mk_right
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic
```

```
@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- Currying in the `x`-variable: `x ↦ f (x, y)` is Scott-continuous (used for
    continuity of
`lambda f` as a map `D → [D' → D'']`). -/
theorem curry_left_preservesDirectedSup (f : ScottMap (D × D') D'') (y : D') :
    PreservesDirectedSup (fun x => f (x, y)) := by
  intro S hS hSdir
  have hS' : DirectedOn (· ≤ ·) (Set.image (fun x => (x, y)) S) := by
    intro p hp q hq
    obtain ⟨a, ha, rfl⟩ := hp
    obtain ⟨b, hb, rfl⟩ := hq
    obtain ⟨c, hc, hac, hbc⟩ := hSdir a ha b hb
    exact ⟨(c, y), Set.mem_image_of_mem _ hc, And.intro (And.intro hac le_rfl) (And.
    intro hbc le_rfl)⟩
  have hS'ne : (Set.image (fun x => (x, y)) S).Nonempty := by
    obtain ⟨s, hs⟩ := hS
    exact ⟨(s, y), s, hs, rfl⟩
  rw [show (fun x => f (x, y)) (sSup S) = f (sSup S, y) from rfl,
    ← sSup_image_prod_mk_right y S hS,
    f.preservesDirectedSup_coe (Set.image (fun x => (x, y)) S) hS'ne hS']
  congr 1
  simp [Set.image_image]

end DomainTheory.ContinuousLattice
```

2

curry_left_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_image_prod_mk_right
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice
```

```
open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- Currying in the `x`-variable: `x ↦ f (x, y)` is Scott-continuous (used for
    continuity of
`lambda f` as a map `D → [D' → D'']`). -/
theorem curry_left_preservesDirectedSup (f : ScottMap (D × D') D'') (y : D') :
    PreservesDirectedSup (fun x => f (x, y)) := by
  intro S hS hSdir
  have hS' : DirectedOn (· ≤ ·) (Set.image (fun x => (x, y)) S) := by
    intro p hp q hq
    obtain ⟨a, ha, rfl⟩ := hp
    obtain ⟨b, hb, rfl⟩ := hq
    obtain ⟨c, hc, hac, hbc⟩ := hSdir a ha b hb
    exact ⟨(c, y), Set.mem_image_of_mem _ hc, And.intro (And.intro hac le_rfl) (And.
    intro hbc le_rfl)⟩
  have hS'ne : (Set.image (fun x => (x, y)) S).Nonempty := by
    obtain ⟨s, hs⟩ := hS
    exact ⟨(s, y), s, hs, rfl⟩
  rw [show (fun x => f (x, y)) (sSup S) = f (sSup S, y) from rfl,
    ← sSup_image_prod_mk_right y S hS,
    f.preservesDirectedSup_coe (Set.image (fun x => (x, y)) S) hS'ne hS']
  congr 1
  simp [Set.image_image]

end DomainTheory.ContinuousLattice
```

3

curry_left_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_image_prod_mk_right
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
```

```
variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- Currying in the `x`-variable: `x ↦ f (x, y)` is Scott-continuous (used for
    continuity of
`lambda f` as a map `D → [D' → D'']`). -/
theorem curry_left_preservesDirectedSup (f : ScottMap (D × D') D'') (y : D') :
    PreservesDirectedSup (fun x => f (x, y)) := by
  intro S hS hSdir
  have hS' : DirectedOn (· ≤ ·) (Set.image (fun x => (x, y)) S) := by
    intro p hp q hq
    obtain ⟨a, ha, rfl⟩ := hp
    obtain ⟨b, hb, rfl⟩ := hq
    obtain ⟨c, hc, hac, hbc⟩ := hSdir a ha b hb
    exact ⟨(c, y), Set.mem_image_of_mem _ hc, And.intro (And.intro hac le_rfl) (And.
    intro hbc le_rfl)⟩
  have hS'ne : (Set.image (fun x => (x, y)) S).Nonempty := by
    obtain ⟨s, hs⟩ := hS
    exact ⟨(s, y), s, hs, rfl⟩
  rw [show (fun x => f (x, y)) (sSup S) = f (sSup S, y) from rfl,
    ← sSup_image_prod_mk_right y S hS,
    f.preservesDirectedSup_coe (Set.image (fun x => (x, y)) S) hS'ne hS']
  congr 1
  simp [Set.image_image]

end DomainTheory.ContinuousLattice
```

4

curry_left_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_image_prod_mk_right
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
```

```
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- Currying in the `x`-variable: `x ↦ f (x, y)` is Scott-continuous (used for
    continuity of
`lambda f` as a map `D → [D' → D'']`). -/
theorem curry_left_preservesDirectedSup (f : ScottMap (D × D') D'') (y : D') :
    PreservesDirectedSup (fun x => f (x, y)) := by
  intro S hS hSdir
  have hS' : DirectedOn (· ≤ ·) (Set.image (fun x => (x, y)) S) := by
    intro p hp q hq
    obtain ⟨a, ha, rfl⟩ := hp
    obtain ⟨b, hb, rfl⟩ := hq
    obtain ⟨c, hc, hac, hbc⟩ := hSdir a ha b hb
    exact ⟨(c, y), Set.mem_image_of_mem _ hc, And.intro (And.intro hac le_rfl) (And.
    intro hbc le_rfl)⟩
  have hS'ne : (Set.image (fun x => (x, y)) S).Nonempty := by
    obtain ⟨s, hs⟩ := hS
    exact ⟨(s, y), s, hs, rfl⟩
  rw [show (fun x => f (x, y)) (sSup S) = f (sSup S, y) from rfl,
    ← sSup_image_prod_mk_right y S hS,
    f.preservesDirectedSup_coe (Set.image (fun x => (x, y)) S) hS'ne hS']
  congr 1
  simp [Set.image_image]

end DomainTheory.ContinuousLattice
```

5

curry_left_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_image_prod_mk_right
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']
```

```
/-- Currying in the `x`-variable: `x ↦ f (x, y)` is Scott-continuous (used for
    continuity of
`lambda f` as a map `D → [D' → D'']`). -/
theorem curry_left_preservesDirectedSup (f : ScottMap (D × D') D'') (y : D') :
    PreservesDirectedSup (fun x => f (x, y)) := by
  intro S hS hSdir
  have hS' : DirectedOn (· ≤ ·) (Set.image (fun x => (x, y)) S) := by
    intro p hp q hq
    obtain ⟨a, ha, rfl⟩ := hp
    obtain ⟨b, hb, rfl⟩ := hq
    obtain ⟨c, hc, hac, hbc⟩ := hSdir a ha b hb
    exact ⟨(c, y), Set.mem_image_of_mem _ hc, And.intro (And.intro hac le_rfl) (And.
    intro hbc le_rfl)⟩
  have hS'ne : (Set.image (fun x => (x, y)) S).Nonempty := by
    obtain ⟨s, hs⟩ := hS
    exact ⟨(s, y), s, hs, rfl⟩
  rw [show (fun x => f (x, y)) (sSup S) = f (sSup S, y) from rfl,
    ← sSup_image_prod_mk_right y S hS,
    f.preservesDirectedSup_coe (Set.image (fun x => (x, y)) S) hS'ne hS']
  congr 1
  simp [Set.image_image]

end DomainTheory.ContinuousLattice
```

6

curry_left_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_image_prod_mk_right
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- Currying in the `x`-variable: `x ↦ f (x, y)` is Scott-continuous (used for
    continuity of
`lambda f` as a map `D → [D' → D'']`). -/
theorem curry_left_preservesDirectedSup (f : ScottMap (D × D') D'') (y : D') :
```

```
    PreservesDirectedSup (fun x => f (x, y)) := by
  intro S hS hSdir
  have hS' : DirectedOn (· ≤ ·) (Set.image (fun x => (x, y)) S) := by
    intro p hp q hq
    obtain ⟨a, ha, rfl⟩ := hp
    obtain ⟨b, hb, rfl⟩ := hq
    obtain ⟨c, hc, hac, hbc⟩ := hSdir a ha b hb
    exact ⟨(c, y), Set.mem_image_of_mem _ hc, And.intro (And.intro hac le_rfl) (And.
    intro hbc le_rfl)⟩
  have hS'ne : (Set.image (fun x => (x, y)) S).Nonempty := by
    obtain ⟨s, hs⟩ := hS
    exact ⟨(s, y), s, hs, rfl⟩
  rw [show (fun x => f (x, y)) (sSup S) = f (sSup S, y) from rfl,
    ← sSup_image_prod_mk_right y S hS,
    f.preservesDirectedSup_coe (Set.image (fun x => (x, y)) S) hS'ne hS']
  congr 1
  simp [Set.image_image]

end DomainTheory.ContinuousLattice
```

7

curry_left_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_image_prod_mk_right
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- Currying in the `x`-variable: `x ↦ f (x, y)` is Scott-continuous (used for
    continuity of
`lambda f` as a map `D → [D' → D'']`). -/
theorem curry_left_preservesDirectedSup (f : ScottMap (D × D') D'') (y : D') :
    PreservesDirectedSup (fun x => f (x, y)) := by
  intro S hS hSdir
  have hS' : DirectedOn (· ≤ ·) (Set.image (fun x => (x, y)) S) := by
    intro p hp q hq
```

```
    obtain ⟨a, ha, rfl⟩ := hp
    obtain ⟨b, hb, rfl⟩ := hq
    obtain ⟨c, hc, hac, hbc⟩ := hSdir a ha b hb
    exact ⟨(c, y), Set.mem_image_of_mem _ hc, And.intro (And.intro hac le_rfl) (And.
    intro hbc le_rfl)⟩
  have hS'ne : (Set.image (fun x => (x, y)) S).Nonempty := by
    obtain ⟨s, hs⟩ := hS
    exact ⟨(s, y), s, hs, rfl⟩
  rw [show (fun x => f (x, y)) (sSup S) = f (sSup S, y) from rfl,
    ← sSup_image_prod_mk_right y S hS,
    f.preservesDirectedSup_coe (Set.image (fun x => (x, y)) S) hS'ne hS']
  congr 1
  simp [Set.image_image]

end DomainTheory.ContinuousLattice
```

8

curry_left_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_image_prod_mk_right
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- Currying in the `x`-variable: `x ↦ f (x, y)` is Scott-continuous (used for
    continuity of
`lambda f` as a map `D → [D' → D'']`). -/
theorem curry_left_preservesDirectedSup (f : ScottMap (D × D') D'') (y : D') :
    PreservesDirectedSup (fun x => f (x, y)) := by
  intro S hS hSdir
  have hS' : DirectedOn (· ≤ ·) (Set.image (fun x => (x, y)) S) := by
    intro p hp q hq
    obtain ⟨a, ha, rfl⟩ := hp
    obtain ⟨b, hb, rfl⟩ := hq
    obtain ⟨c, hc, hac, hbc⟩ := hSdir a ha b hb
    exact ⟨(c, y), Set.mem_image_of_mem _ hc, And.intro (And.intro hac le_rfl) (And.
```

```
    intro hbc le_rfl)⟩
  have hS'ne : (Set.image (fun x => (x, y)) S).Nonempty := by
    obtain ⟨s, hs⟩ := hS
    exact ⟨(s, y), s, hs, rfl⟩
  rw [show (fun x => f (x, y)) (sSup S) = f (sSup S, y) from rfl,
    ← sSup_image_prod_mk_right y S hS,
    f.preservesDirectedSup_coe (Set.image (fun x => (x, y)) S) hS'ne hS']
  congr 1
  simp [Set.image_image]

end DomainTheory.ContinuousLattice
```

9

curry_right_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_image_prod_mk_left
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem curry_right_preservesDirectedSup (f : ScottMap (D × D') D'') (x : D) :
    PreservesDirectedSup (fun y => f (x, y)) := by
  intro S hS hSdir
  have hS' : DirectedOn (· ≤ ·) (Set.image (fun y => (x, y)) S) := by
    intro p hp q hq
    obtain ⟨a, ha, rfl⟩ := hp
    obtain ⟨b, hb, rfl⟩ := hq
    obtain ⟨c, hc, hac, hbc⟩ := hSdir a ha b hb
    exact ⟨(x, c), Set.mem_image_of_mem _ hc, And.intro (And.intro le_rfl hac) (And.
    intro le_rfl hbc)⟩
  have hS'ne : (Set.image (fun y => (x, y)) S).Nonempty := by
    obtain ⟨s, hs⟩ := hS
    exact ⟨(x, s), s, hs, rfl⟩
  rw [show (fun y => f (x, y)) (sSup S) = f (x, sSup S) from rfl,
    ← sSup_image_prod_mk_left x S hS,
    f.preservesDirectedSup_coe (Set.image (fun y => (x, y)) S) hS'ne hS']
```

```
  congr 1
  simp [Set.image_image]

end DomainTheory.ContinuousLattice
```

0

curry_right_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_image_prod_mk_left
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem curry_right_preservesDirectedSup (f : ScottMap (D × D') D'') (x : D) :
    PreservesDirectedSup (fun y => f (x, y)) := by
  intro S hS hSdir
  have hS' : DirectedOn (· ≤ ·) (Set.image (fun y => (x, y)) S) := by
    intro p hp q hq
    obtain ⟨a, ha, rfl⟩ := hp
    obtain ⟨b, hb, rfl⟩ := hq
    obtain ⟨c, hc, hac, hbc⟩ := hSdir a ha b hb
    exact ⟨(x, c), Set.mem_image_of_mem _ hc, And.intro (And.intro le_rfl hac) (And.
    intro le_rfl hbc)⟩
  have hS'ne : (Set.image (fun y => (x, y)) S).Nonempty := by
    obtain ⟨s, hs⟩ := hS
    exact ⟨(x, s), s, hs, rfl⟩
  rw [show (fun y => f (x, y)) (sSup S) = f (x, sSup S) from rfl,
    ← sSup_image_prod_mk_left x S hS,
    f.preservesDirectedSup_coe (Set.image (fun y => (x, y)) S) hS'ne hS']
  congr 1
  simp [Set.image_image]

end DomainTheory.ContinuousLattice
```

1

curry_right_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_image_prod_mk_left
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem curry_right_preservesDirectedSup (f : ScottMap (D × D') D'') (x : D) :
    PreservesDirectedSup (fun y => f (x, y)) := by
  intro S hS hSdir
  have hS' : DirectedOn (· ≤ ·) (Set.image (fun y => (x, y)) S) := by
    intro p hp q hq
    obtain ⟨a, ha, rfl⟩ := hp
    obtain ⟨b, hb, rfl⟩ := hq
    obtain ⟨c, hc, hac, hbc⟩ := hSdir a ha b hb
    exact ⟨(x, c), Set.mem_image_of_mem _ hc, And.intro (And.intro le_rfl hac) (And.
    intro le_rfl hbc)⟩
  have hS'ne : (Set.image (fun y => (x, y)) S).Nonempty := by
    obtain ⟨s, hs⟩ := hS
    exact ⟨(x, s), s, hs, rfl⟩
  rw [show (fun y => f (x, y)) (sSup S) = f (x, sSup S) from rfl,
    ← sSup_image_prod_mk_left x S hS,
    f.preservesDirectedSup_coe (Set.image (fun y => (x, y)) S) hS'ne hS']
  congr 1
  simp [Set.image_image]

end DomainTheory.ContinuousLattice
```

2

curry_right_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_image_prod_mk_left
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem curry_right_preservesDirectedSup (f : ScottMap (D × D') D'') (x : D) :
    PreservesDirectedSup (fun y => f (x, y)) := by
  intro S hS hSdir
  have hS' : DirectedOn (· ≤ ·) (Set.image (fun y => (x, y)) S) := by
    intro p hp q hq
    obtain ⟨a, ha, rfl⟩ := hp
    obtain ⟨b, hb, rfl⟩ := hq
    obtain ⟨c, hc, hac, hbc⟩ := hSdir a ha b hb
    exact ⟨(x, c), Set.mem_image_of_mem _ hc, And.intro (And.intro le_rfl hac) (And.
    intro le_rfl hbc)⟩
  have hS'ne : (Set.image (fun y => (x, y)) S).Nonempty := by
    obtain ⟨s, hs⟩ := hS
    exact ⟨(x, s), s, hs, rfl⟩
  rw [show (fun y => f (x, y)) (sSup S) = f (x, sSup S) from rfl,
    ← sSup_image_prod_mk_left x S hS,
    f.preservesDirectedSup_coe (Set.image (fun y => (x, y)) S) hS'ne hS']
  congr 1
  simp [Set.image_image]

end DomainTheory.ContinuousLattice
```

3

curry_right_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_image_prod_mk_left
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice
```

```
open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem curry_right_preservesDirectedSup (f : ScottMap (D × D') D'') (x : D) :
    PreservesDirectedSup (fun y => f (x, y)) := by
  intro S hS hSdir
  have hS' : DirectedOn (· ≤ ·) (Set.image (fun y => (x, y)) S) := by
    intro p hp q hq
    obtain ⟨a, ha, rfl⟩ := hp
    obtain ⟨b, hb, rfl⟩ := hq
    obtain ⟨c, hc, hac, hbc⟩ := hSdir a ha b hb
    exact ⟨(x, c), Set.mem_image_of_mem _ hc, And.intro (And.intro le_rfl hac) (And.
    intro le_rfl hbc)⟩
  have hS'ne : (Set.image (fun y => (x, y)) S).Nonempty := by
    obtain ⟨s, hs⟩ := hS
    exact ⟨(x, s), s, hs, rfl⟩
  rw [show (fun y => f (x, y)) (sSup S) = f (x, sSup S) from rfl,
    ← sSup_image_prod_mk_left x S hS,
    f.preservesDirectedSup_coe (Set.image (fun y => (x, y)) S) hS'ne hS']
  congr 1
  simp [Set.image_image]

end DomainTheory.ContinuousLattice
```

4

curry_right_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_image_prod_mk_left
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]
```

```
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem curry_right_preservesDirectedSup (f : ScottMap (D × D') D'') (x : D) :
    PreservesDirectedSup (fun y => f (x, y)) := by
  intro S hS hSdir
  have hS' : DirectedOn (· ≤ ·) (Set.image (fun y => (x, y)) S) := by
    intro p hp q hq
    obtain ⟨a, ha, rfl⟩ := hp
    obtain ⟨b, hb, rfl⟩ := hq
    obtain ⟨c, hc, hac, hbc⟩ := hSdir a ha b hb
    exact ⟨(x, c), Set.mem_image_of_mem _ hc, And.intro (And.intro le_rfl hac) (And.
    intro le_rfl hbc)⟩
  have hS'ne : (Set.image (fun y => (x, y)) S).Nonempty := by
    obtain ⟨s, hs⟩ := hS
    exact ⟨(x, s), s, hs, rfl⟩
  rw [show (fun y => f (x, y)) (sSup S) = f (x, sSup S) from rfl,
    ← sSup_image_prod_mk_left x S hS,
    f.preservesDirectedSup_coe (Set.image (fun y => (x, y)) S) hS'ne hS']
  congr 1
  simp [Set.image_image]

end DomainTheory.ContinuousLattice
```

5

curry_right_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_image_prod_mk_left
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem curry_right_preservesDirectedSup (f : ScottMap (D × D') D'') (x : D) :
    PreservesDirectedSup (fun y => f (x, y)) := by
```

```
  intro S hS hSdir
  have hS' : DirectedOn (· ≤ ·) (Set.image (fun y => (x, y)) S) := by
    intro p hp q hq
    obtain ⟨a, ha, rfl⟩ := hp
    obtain ⟨b, hb, rfl⟩ := hq
    obtain ⟨c, hc, hac, hbc⟩ := hSdir a ha b hb
    exact ⟨(x, c), Set.mem_image_of_mem _ hc, And.intro (And.intro le_rfl hac) (And.
    intro le_rfl hbc)⟩
  have hS'ne : (Set.image (fun y => (x, y)) S).Nonempty := by
    obtain ⟨s, hs⟩ := hS
    exact ⟨(x, s), s, hs, rfl⟩
  rw [show (fun y => f (x, y)) (sSup S) = f (x, sSup S) from rfl,
    ← sSup_image_prod_mk_left x S hS,
    f.preservesDirectedSup_coe (Set.image (fun y => (x, y)) S) hS'ne hS']
  congr 1
  simp [Set.image_image]

end DomainTheory.ContinuousLattice
```

6

curry_right_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_image_prod_mk_left
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem curry_right_preservesDirectedSup (f : ScottMap (D × D') D'') (x : D) :
    PreservesDirectedSup (fun y => f (x, y)) := by
  intro S hS hSdir
  have hS' : DirectedOn (· ≤ ·) (Set.image (fun y => (x, y)) S) := by
    intro p hp q hq
    obtain ⟨a, ha, rfl⟩ := hp
    obtain ⟨b, hb, rfl⟩ := hq
    obtain ⟨c, hc, hac, hbc⟩ := hSdir a ha b hb
    exact ⟨(x, c), Set.mem_image_of_mem _ hc, And.intro (And.intro le_rfl hac) (And.
```

```
    intro le_rfl hbc)⟩
  have hS'ne : (Set.image (fun y => (x, y)) S).Nonempty := by
    obtain ⟨s, hs⟩ := hS
    exact ⟨(x, s), s, hs, rfl⟩
  rw [show (fun y => f (x, y)) (sSup S) = f (x, sSup S) from rfl,
    ← sSup_image_prod_mk_left x S hS,
    f.preservesDirectedSup_coe (Set.image (fun y => (x, y)) S) hS'ne hS']
  congr 1
  simp [Set.image_image]

end DomainTheory.ContinuousLattice
```

7

curry_right_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_image_prod_mk_left
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem curry_right_preservesDirectedSup (f : ScottMap (D × D') D'') (x : D) :
    PreservesDirectedSup (fun y => f (x, y)) := by
  intro S hS hSdir
  have hS' : DirectedOn (· ≤ ·) (Set.image (fun y => (x, y)) S) := by
    intro p hp q hq
    obtain ⟨a, ha, rfl⟩ := hp
    obtain ⟨b, hb, rfl⟩ := hq
    obtain ⟨c, hc, hac, hbc⟩ := hSdir a ha b hb
    exact ⟨(x, c), Set.mem_image_of_mem _ hc, And.intro (And.intro le_rfl hac) (And.
    intro le_rfl hbc)⟩
  have hS'ne : (Set.image (fun y => (x, y)) S).Nonempty := by
    obtain ⟨s, hs⟩ := hS
    exact ⟨(x, s), s, hs, rfl⟩
  rw [show (fun y => f (x, y)) (sSup S) = f (x, sSup S) from rfl,
    ← sSup_image_prod_mk_left x S hS,
    f.preservesDirectedSup_coe (Set.image (fun y => (x, y)) S) hS'ne hS']
```

```
  congr 1
  simp [Set.image_image]

end DomainTheory.ContinuousLattice
```

8

curry_right_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_image_prod_mk_left
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

theorem curry_right_preservesDirectedSup (f : ScottMap (D × D') D'') (x : D) :
    PreservesDirectedSup (fun y => f (x, y)) := by
  intro S hS hSdir
  have hS' : DirectedOn (· ≤ ·) (Set.image (fun y => (x, y)) S) := by
    intro p hp q hq
    obtain ⟨a, ha, rfl⟩ := hp
    obtain ⟨b, hb, rfl⟩ := hq
    obtain ⟨c, hc, hac, hbc⟩ := hSdir a ha b hb
    exact ⟨(x, c), Set.mem_image_of_mem _ hc, And.intro (And.intro le_rfl hac) (And.
    intro le_rfl hbc)⟩
  have hS'ne : (Set.image (fun y => (x, y)) S).Nonempty := by
    obtain ⟨s, hs⟩ := hS
    exact ⟨(x, s), s, hs, rfl⟩
  rw [show (fun y => f (x, y)) (sSup S) = f (x, sSup S) from rfl,
    ← sSup_image_prod_mk_left x S hS,
    f.preservesDirectedSup_coe (Set.image (fun y => (x, y)) S) hS'ne hS']
  congr 1
  simp [Set.image_image]

end DomainTheory.ContinuousLattice
```

9

lambda_outer_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottLambdaAt
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    curry_left_preservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- The outer half of currying: `x ↦ (y ↦ f (x, y))` preserves directed suprema (so `
    lambda f` is a
Scott map `D → [D' → D'']`). Equality in `[D' → D'']` is pointwise, reducing to `
    curry_left`. -/
theorem lambda_outer_preservesDirectedSup (f : ScottMap (D × D') D'') :
    PreservesDirectedSup (fun x => scottLambdaAt f x) := by
  intro S hS hSdir
  apply ScottMap.ext
  intro y
  show f (sSup S, y)
      = ((sSup (Set.image (fun x => scottLambdaAt f x) S) : ScottMap D' D'') : D' → D
    '') y
  rw [ScottMap.sSup_apply, Set.image_image]
  exact curry_left_preservesDirectedSup f y hS hSdir

end DomainTheory.ContinuousLattice
```

0

lambda_outer_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottLambdaAt
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    curry_left_preservesDirectedSup
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- The outer half of currying: `x ↦ (y ↦ f (x, y))` preserves directed suprema (so `
    lambda f` is a
Scott map `D → [D' → D'']`). Equality in `[D' → D'']` is pointwise, reducing to `
    curry_left`. -/
theorem lambda_outer_preservesDirectedSup (f : ScottMap (D × D') D'') :
    PreservesDirectedSup (fun x => scottLambdaAt f x) := by
  intro S hS hSdir
  apply ScottMap.ext
  intro y
  show f (sSup S, y)
      = ((sSup (Set.image (fun x => scottLambdaAt f x) S) : ScottMap D' D'') : D' → D
    '') y
  rw [ScottMap.sSup_apply, Set.image_image]
  exact curry_left_preservesDirectedSup f y hS hSdir

end DomainTheory.ContinuousLattice
```

1

lambda_outer_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottLambdaAt
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    curry_left_preservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace
```

```
variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- The outer half of currying: `x ↦ (y ↦ f (x, y))` preserves directed suprema (so `
    lambda f` is a
Scott map `D → [D' → D'']`). Equality in `[D' → D'']` is pointwise, reducing to `
    curry_left`. -/
theorem lambda_outer_preservesDirectedSup (f : ScottMap (D × D') D'') :
    PreservesDirectedSup (fun x => scottLambdaAt f x) := by
  intro S hS hSdir
  apply ScottMap.ext
  intro y
  show f (sSup S, y)
      = ((sSup (Set.image (fun x => scottLambdaAt f x) S) : ScottMap D' D'') : D' → D
    '') y
  rw [ScottMap.sSup_apply, Set.image_image]
  exact curry_left_preservesDirectedSup f y hS hSdir

end DomainTheory.ContinuousLattice
```

2

lambda_outer_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottLambdaAt
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    curry_left_preservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']
```

```
/-- The outer half of currying: `x ↦ (y ↦ f (x, y))` preserves directed suprema (so `
    lambda f` is a
Scott map `D → [D' → D'']`). Equality in `[D' → D'']` is pointwise, reducing to `
    curry_left`. -/
theorem lambda_outer_preservesDirectedSup (f : ScottMap (D × D') D'') :
    PreservesDirectedSup (fun x => scottLambdaAt f x) := by
  intro S hS hSdir
  apply ScottMap.ext
  intro y
  show f (sSup S, y)
      = ((sSup (Set.image (fun x => scottLambdaAt f x) S) : ScottMap D' D'') : D' → D
    '') y
  rw [ScottMap.sSup_apply, Set.image_image]
  exact curry_left_preservesDirectedSup f y hS hSdir

end DomainTheory.ContinuousLattice
```

3

lambda_outer_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottLambdaAt
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    curry_left_preservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- The outer half of currying: `x ↦ (y ↦ f (x, y))` preserves directed suprema (so `
    lambda f` is a
Scott map `D → [D' → D'']`). Equality in `[D' → D'']` is pointwise, reducing to `
    curry_left`. -/
theorem lambda_outer_preservesDirectedSup (f : ScottMap (D × D') D'') :
    PreservesDirectedSup (fun x => scottLambdaAt f x) := by
  intro S hS hSdir
  apply ScottMap.ext
```

```
  intro y
  show f (sSup S, y)
      = ((sSup (Set.image (fun x => scottLambdaAt f x) S) : ScottMap D' D'') : D' → D
    '') y
  rw [ScottMap.sSup_apply, Set.image_image]
  exact curry_left_preservesDirectedSup f y hS hSdir

end DomainTheory.ContinuousLattice
```

4

lambda_outer_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottLambdaAt
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    curry_left_preservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- The outer half of currying: `x ↦ (y ↦ f (x, y))` preserves directed suprema (so `
    lambda f` is a
Scott map `D → [D' → D'']`). Equality in `[D' → D'']` is pointwise, reducing to `
    curry_left`. -/
theorem lambda_outer_preservesDirectedSup (f : ScottMap (D × D') D'') :
    PreservesDirectedSup (fun x => scottLambdaAt f x) := by
  intro S hS hSdir
  apply ScottMap.ext
  intro y
  show f (sSup S, y)
      = ((sSup (Set.image (fun x => scottLambdaAt f x) S) : ScottMap D' D'') : D' → D
    '') y
  rw [ScottMap.sSup_apply, Set.image_image]
  exact curry_left_preservesDirectedSup f y hS hSdir

end DomainTheory.ContinuousLattice
```

5

lambda_outer_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottLambdaAt
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    curry_left_preservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- The outer half of currying: `x ↦ (y ↦ f (x, y))` preserves directed suprema (so `
    lambda f` is a
Scott map `D → [D' → D'']`). Equality in `[D' → D'']` is pointwise, reducing to `
    curry_left`. -/
theorem lambda_outer_preservesDirectedSup (f : ScottMap (D × D') D'') :
    PreservesDirectedSup (fun x => scottLambdaAt f x) := by
  intro S hS hSdir
  apply ScottMap.ext
  intro y
  show f (sSup S, y)
      = ((sSup (Set.image (fun x => scottLambdaAt f x) S) : ScottMap D' D'') : D' → D
    '') y
  rw [ScottMap.sSup_apply, Set.image_image]
  exact curry_left_preservesDirectedSup f y hS hSdir

end DomainTheory.ContinuousLattice
```

6

lambda_outer_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottLambdaAt
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    curry_left_preservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- The outer half of currying: `x ↦ (y ↦ f (x, y))` preserves directed suprema (so `
    lambda f` is a
Scott map `D → [D' → D'']`). Equality in `[D' → D'']` is pointwise, reducing to `
    curry_left`. -/
theorem lambda_outer_preservesDirectedSup (f : ScottMap (D × D') D'') :
    PreservesDirectedSup (fun x => scottLambdaAt f x) := by
  intro S hS hSdir
  apply ScottMap.ext
  intro y
  show f (sSup S, y)
      = ((sSup (Set.image (fun x => scottLambdaAt f x) S) : ScottMap D' D'') : D' → D
    '') y
  rw [ScottMap.sSup_apply, Set.image_image]
  exact curry_left_preservesDirectedSup f y hS hSdir

end DomainTheory.ContinuousLattice
```

7

lambda_outer_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottLambdaAt
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    curry_left_preservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice
```

```
open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- The outer half of currying: `x ↦ (y ↦ f (x, y))` preserves directed suprema (so `
    lambda f` is a
Scott map `D → [D' → D'']`). Equality in `[D' → D'']` is pointwise, reducing to `
    curry_left`. -/
theorem lambda_outer_preservesDirectedSup (f : ScottMap (D × D') D'') :
    PreservesDirectedSup (fun x => scottLambdaAt f x) := by
  intro S hS hSdir
  apply ScottMap.ext
  intro y
  show f (sSup S, y)
      = ((sSup (Set.image (fun x => scottLambdaAt f x) S) : ScottMap D' D'') : D' → D
    '') y
  rw [ScottMap.sSup_apply, Set.image_image]
  exact curry_left_preservesDirectedSup f y hS hSdir

end DomainTheory.ContinuousLattice
```

8

lambda_outer_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottLambdaAt
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    curry_left_preservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
```

```
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- The outer half of currying: `x ↦ (y ↦ f (x, y))` preserves directed suprema (so `
    lambda f` is a
Scott map `D → [D' → D'']`). Equality in `[D' → D'']` is pointwise, reducing to `
    curry_left`. -/
theorem lambda_outer_preservesDirectedSup (f : ScottMap (D × D') D'') :
    PreservesDirectedSup (fun x => scottLambdaAt f x) := by
  intro S hS hSdir
  apply ScottMap.ext
  intro y
  show f (sSup S, y)
      = ((sSup (Set.image (fun x => scottLambdaAt f x) S) : ScottMap D' D'') : D' → D
    '') y
  rw [ScottMap.sSup_apply, Set.image_image]
  exact curry_left_preservesDirectedSup f y hS hSdir

end DomainTheory.ContinuousLattice
```

9

proposition_3_5.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    proposition_3_5_preservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottLambda
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Proposition 3.5.** Functional abstraction `lambda` is Scott-
    continuous. -/
theorem proposition_3_5 :
    @Continuous (ScottMap (D × D') D'') (ScottMap D (ScottMap D' D''))
      scottTopologicalSpace scottTopologicalSpace scottLambda :=
  continuous_of_preservesDirectedSup proposition_3_5_preservesDirectedSup

end DomainTheory.ContinuousLattice
```

0

proposition_3_5.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    proposition_3_5_preservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottLambda
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Proposition 3.5.** Functional abstraction `lambda` is Scott-
    continuous. -/
theorem proposition_3_5 :
    @Continuous (ScottMap (D × D') D'') (ScottMap D (ScottMap D' D''))
      scottTopologicalSpace scottTopologicalSpace scottLambda :=
  continuous_of_preservesDirectedSup proposition_3_5_preservesDirectedSup

end DomainTheory.ContinuousLattice
```

1

proposition_3_5.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    proposition_3_5_preservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottLambda
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']
```

```
/-- **Scott 1972, Proposition 3.5.** Functional abstraction `lambda` is Scott-
    continuous. -/
theorem proposition_3_5 :
    @Continuous (ScottMap (D × D') D'') (ScottMap D (ScottMap D' D''))
      scottTopologicalSpace scottTopologicalSpace scottLambda :=
  continuous_of_preservesDirectedSup proposition_3_5_preservesDirectedSup

end DomainTheory.ContinuousLattice
```

2

proposition_3_5.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    proposition_3_5_preservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottLambda
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Proposition 3.5.** Functional abstraction `lambda` is Scott-
    continuous. -/
theorem proposition_3_5 :
    @Continuous (ScottMap (D × D') D'') (ScottMap D (ScottMap D' D''))
      scottTopologicalSpace scottTopologicalSpace scottLambda :=
  continuous_of_preservesDirectedSup proposition_3_5_preservesDirectedSup

end DomainTheory.ContinuousLattice
```

3

proposition_3_5.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    proposition_3_5_preservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottLambda
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic
```

```
@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Proposition 3.5.** Functional abstraction `lambda` is Scott-
    continuous. -/
theorem proposition_3_5 :
    @Continuous (ScottMap (D × D') D'') (ScottMap D (ScottMap D' D''))
      scottTopologicalSpace scottTopologicalSpace scottLambda :=
  continuous_of_preservesDirectedSup proposition_3_5_preservesDirectedSup

end DomainTheory.ContinuousLattice
```

4

proposition_3_5.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    proposition_3_5_preservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottLambda
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Proposition 3.5.** Functional abstraction `lambda` is Scott-
    continuous. -/
theorem proposition_3_5 :
    @Continuous (ScottMap (D × D') D'') (ScottMap D (ScottMap D' D''))
      scottTopologicalSpace scottTopologicalSpace scottLambda :=
  continuous_of_preservesDirectedSup proposition_3_5_preservesDirectedSup

end DomainTheory.ContinuousLattice
```

5

proposition_3_5.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    proposition_3_5_preservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottLambda
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Proposition 3.5.** Functional abstraction `lambda` is Scott-
    continuous. -/
theorem proposition_3_5 :
    @Continuous (ScottMap (D × D') D'') (ScottMap D (ScottMap D' D''))
      scottTopologicalSpace scottTopologicalSpace scottLambda :=
  continuous_of_preservesDirectedSup proposition_3_5_preservesDirectedSup

end DomainTheory.ContinuousLattice
```

6

proposition_3_5.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    proposition_3_5_preservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottLambda
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
```

```
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Proposition 3.5.** Functional abstraction `lambda` is Scott-
    continuous. -/
theorem proposition_3_5 :
    @Continuous (ScottMap (D × D') D'') (ScottMap D (ScottMap D' D''))
      scottTopologicalSpace scottTopologicalSpace scottLambda :=
  continuous_of_preservesDirectedSup proposition_3_5_preservesDirectedSup

end DomainTheory.ContinuousLattice
```

7

proposition_3_5.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    proposition_3_5_preservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottLambda
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Proposition 3.5.** Functional abstraction `lambda` is Scott-
    continuous. -/
theorem proposition_3_5 :
    @Continuous (ScottMap (D × D') D'') (ScottMap D (ScottMap D' D''))
      scottTopologicalSpace scottTopologicalSpace scottLambda :=
  continuous_of_preservesDirectedSup proposition_3_5_preservesDirectedSup

end DomainTheory.ContinuousLattice
```

8

proposition_3_5.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    proposition_3_5_preservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottLambda
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
```

```
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Proposition 3.5.** Functional abstraction `lambda` is Scott-
    continuous. -/
theorem proposition_3_5 :
    @Continuous (ScottMap (D × D') D'') (ScottMap D (ScottMap D' D''))
      scottTopologicalSpace scottTopologicalSpace scottLambda :=
  continuous_of_preservesDirectedSup proposition_3_5_preservesDirectedSup

end DomainTheory.ContinuousLattice
```

9

proposition_3_5_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottLambda_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottLambda
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- `lambda` preserves directed suprema: both sides evaluate, pointwise at `(x, y)`, to
`⨆ {f (x, y) | f ∈ 𝓕}`, since suprema in every function lattice involved are pointwise
    . -/
theorem proposition_3_5_preservesDirectedSup :
    PreservesDirectedSup
      (scottLambda : ScottMap (D × D') D'' → ScottMap D (ScottMap D' D'')) := by
  intro 𝓕 h𝓕 h𝓕dir
```

```
  apply ScottMap.ext
  intro x
  apply ScottMap.ext
  intro y
  rw [scottLambda_apply, ScottMap.sSup_apply, ScottMap.sSup_apply, ScottMap.sSup_apply
    ,
    Set.image_image, Set.image_image]
  congr 1

end DomainTheory.ContinuousLattice
```

0

proposition_3_5_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottLambda_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottLambda
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- `lambda` preserves directed suprema: both sides evaluate, pointwise at `(x, y)`, to
`⨆ {f (x, y) | f ∈ 𝓕}`, since suprema in every function lattice involved are pointwise
    . -/
theorem proposition_3_5_preservesDirectedSup :
    PreservesDirectedSup
      (scottLambda : ScottMap (D × D') D'' → ScottMap D (ScottMap D' D'')) := by
  intro 𝓕 h𝓕 h𝓕dir
  apply ScottMap.ext
  intro x
  apply ScottMap.ext
  intro y
  rw [scottLambda_apply, ScottMap.sSup_apply, ScottMap.sSup_apply, ScottMap.sSup_apply
    ,
    Set.image_image, Set.image_image]
  congr 1

end DomainTheory.ContinuousLattice
```

1

proposition_3_5_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottLambda_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottLambda
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- `lambda` preserves directed suprema: both sides evaluate, pointwise at `(x, y)`, to
`⨆ {f (x, y) | f ∈ 𝓕}`, since suprema in every function lattice involved are pointwise
    . -/
theorem proposition_3_5_preservesDirectedSup :
    PreservesDirectedSup
      (scottLambda : ScottMap (D × D') D'' → ScottMap D (ScottMap D' D'')) := by
  intro 𝓕 h𝓕 h𝓕dir
  apply ScottMap.ext
  intro x
  apply ScottMap.ext
  intro y
  rw [scottLambda_apply, ScottMap.sSup_apply, ScottMap.sSup_apply, ScottMap.sSup_apply
    ,
    Set.image_image, Set.image_image]
  congr 1

end DomainTheory.ContinuousLattice
```

2

proposition_3_5_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottLambda_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottLambda
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section
```

```
namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- `lambda` preserves directed suprema: both sides evaluate, pointwise at `(x, y)`, to
`⨆ {f (x, y) | f ∈ ℱ}`, since suprema in every function lattice involved are pointwise
    . -/
theorem proposition_3_5_preservesDirectedSup :
    PreservesDirectedSup
      (scottLambda : ScottMap (D × D') D'' → ScottMap D (ScottMap D' D'')) := by
  intro ℱ hℱ hℱdir
  apply ScottMap.ext
  intro x
  apply ScottMap.ext
  intro y
  rw [scottLambda_apply, ScottMap.sSup_apply, ScottMap.sSup_apply, ScottMap.sSup_apply
    ,
    Set.image_image, Set.image_image]
  congr 1

end DomainTheory.ContinuousLattice
```

3

proposition_3_5_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottLambda_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottLambda
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- `lambda` preserves directed suprema: both sides evaluate, pointwise at `(x, y)`, to
`⨆ {f (x, y) | f ∈ ℱ}`, since suprema in every function lattice involved are pointwise
    . -/
```

```
theorem proposition_3_5_preservesDirectedSup :
    PreservesDirectedSup
      (scottLambda : ScottMap (D × D') D'' → ScottMap D (ScottMap D' D'')) := by
  intro 𝓕 h𝓕 h𝓕dir
  apply ScottMap.ext
  intro x
  apply ScottMap.ext
  intro y
  rw [scottLambda_apply, ScottMap.sSup_apply, ScottMap.sSup_apply, ScottMap.sSup_apply
    ,
    Set.image_image, Set.image_image]
  congr 1

end DomainTheory.ContinuousLattice
```

4

proposition_3_5_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottLambda_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottLambda
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- `lambda` preserves directed suprema: both sides evaluate, pointwise at `(x, y)`, to
`⨆ {f (x, y) | f ∈ 𝓕}`, since suprema in every function lattice involved are pointwise
    . -/
theorem proposition_3_5_preservesDirectedSup :
    PreservesDirectedSup
      (scottLambda : ScottMap (D × D') D'' → ScottMap D (ScottMap D' D'')) := by
  intro 𝓕 h𝓕 h𝓕dir
  apply ScottMap.ext
  intro x
  apply ScottMap.ext
  intro y
  rw [scottLambda_apply, ScottMap.sSup_apply, ScottMap.sSup_apply, ScottMap.sSup_apply
    ,
    Set.image_image, Set.image_image]
  congr 1
```

```
end DomainTheory.ContinuousLattice
```

5

proposition_3_5_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottLambda_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottLambda
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- `lambda` preserves directed suprema: both sides evaluate, pointwise at `(x, y)`, to
`⨆ {f (x, y) | f ∈ 𝓕}`, since suprema in every function lattice involved are pointwise
    . -/
theorem proposition_3_5_preservesDirectedSup :
    PreservesDirectedSup
      (scottLambda : ScottMap (D × D') D'' → ScottMap D (ScottMap D' D'')) := by
  intro 𝓕 h𝓕 h𝓕dir
  apply ScottMap.ext
  intro x
  apply ScottMap.ext
  intro y
  rw [scottLambda_apply, ScottMap.sSup_apply, ScottMap.sSup_apply, ScottMap.sSup_apply
    ,
    Set.image_image, Set.image_image]
  congr 1

end DomainTheory.ContinuousLattice
```

6

4.8 Some domain equations

Dependencies. Inside SomeDomainEquations, each box is one subfolder, or one Lean file sitting directly in the folder. A box named as another folder in this chapter is that whole folder. A box named as another chapter is that whole chapter. The arrowhead points at the file or folder that imports, and the number is how many direct imports.



Figure 43: Some domain equations.

Having outlined the theory of several domain constructs, the final topic for this paper will be the discussion of the *iteration* of these constructs in giving *recursive*, rather than direct definitions of domains. These recursively defined systems have often been called "reflexive," because the domains generally contain copies of themselves as a part of their very structure. The way that this self-containment takes place is best expressed by the so-called *domain equations*, which are really isomorphisms that relate the domain as a whole to a combination of domains—usually with the main domain as a component. This description is rough, since recursion equations for domains can be as complex as recursion equations for functions. We will not enter into a full theory of domain equations now but will just review some preliminary examples to illustrate how the new presentation makes the constructions more explicit.

4.8.1 Factoid 8.1

A domain of trees or S-expressions. This is everyone's favorite example. And a very nice example it is, but we should not think that it contains all the meat of the theory of domain equations. Even if we generalize the kinds of equations to contain all iterations of the domain constructs $+$ and $\times$, the full power of the method has not been exploited. We will try to make this clear in the further examples.

Let a domain (information system) $\mathbf{A}$ be given. What we want to construct is a domain $\mathbf{T}$ of "trees" built up from elements of $\mathbf{A}$ as "atoms". For simplicity we consider unlabelled binary trees here, but more complex trees are easy to accommodate. The domain equation we want to "solve" is this one:

$$\mathbf{T} \cong \mathbf{A} + (\mathbf{T} \times \mathbf{T}).$$

If such a domain exists, then we can say that (up to isomorphism) the elements of the domain $\mathbf{T}$ are either bottom, or elements of the given domain $\mathbf{A}$, or pairs of elements from the domain $\mathbf{T}$ itself. And these are the only kinds of elements that $\mathbf{T}$ has.

To prove that such a domain exists it is only necessary to ask what information has to be given about a prospective element. The answer may involve us in a regress, but the running backwards need not be infinite—at least for the finite elements. As we shall see, the infinite elements of $\mathbf{T}$ can be self-replicating; but, to define a domain fully, all we have to do is to build up the finite elements out of the data objects in a systematic way. Fortunately, in order to satisfy the above equation, the required closure conditions on data objects are simple to achieve.

In the first place, we need copies of all the data objects of $\mathbf{A}$ to put into the sum. The easy way to do this is to take an object Δ not in $D_{\mathbf{A}}$, and to let, by definition,

$$\Delta_{\mathbf{T}} = (\Delta, \Delta).$$

That gives us one member of $D_{\mathbf{T}}$, the one we always have to have in any case. The copy of an $X \in D_{\mathbf{A}}$ is just going to be (X, Δ). The other members of $D_{\mathbf{T}}$ will be of the form (Δ, U), where U gives us information about the other kind of elements of $\mathbf{T}$. The point is that $\mathbf{T}$ has to be a sum, and we are using just the scheme of Definition 6.3 to set this up.

Next we have to think what kind of information the U above should contain. Because we want a product, we refer back to Definition 6.1 and imagine we have already defined $D_{\mathbf{T}}$. What 6.1(i) suggests is that we throw in a bunch of other data objects into $D_{\mathbf{T}}$. The only point that needs care is that the data objects for the product must be copied into the overall sum. With this in mind, the following clauses give us the inductive definition of $D_{\mathbf{T}}$:

(1) $\Delta_{\mathbf{T}} \in D_{\mathbf{T}}$;

(2) $(X, \Delta) \in D_{\mathbf{T}}$ whenever $X \in D_{\mathbf{A}}$; and

(3) $(\Delta, (Y, \Delta_{\mathbf{T}})) \in D_{\mathbf{T}}$, and $(\Delta, (\Delta_{\mathbf{T}}, Z)) \in D_{\mathbf{T}}$ whenever $Y, Z \in D_{\mathbf{T}}$.

Of course, when we say "inductive definition," we mean that $D_{\mathbf{T}}$ is the least class satisfying (1)–(3). By standard arguments it can be shown that $D_{\mathbf{T}}$ satisfies this set-theoretical equation:

$$D_{\mathbf{T}} = \{\Delta_{\mathbf{T}}\} \cup \{(X, \Delta) \mid X \in D_{\mathbf{A}}\} \cup \{(\Delta, (Y, \Delta_{\mathbf{T}})) \mid Y \in D_{\mathbf{T}}\} \cup \{(\Delta, (\Delta_{\mathbf{T}}, Z)) \mid Z \in D_{\mathbf{T}}\}.$$

In fact, with some very mild assumptions about ordered pairs in set theory, $D_{\mathbf{T}}$ is the only solution to the above equation.

Defining the data objects is but part of the story: the same data objects can enter into quite different information systems. Data objects are just "tokens" and are only given "meaning" when $\mathrm{Con}_{\mathbf{T}}$ and $\vdash_{\mathbf{T}}$ are defined. Let us consider the problem of consistency first. We already understand the notion as it applies to sum and product systems, so we must merely copy over the parts of the previous definitions in the right position for the definition of $\mathrm{Con}_{\mathbf{T}}$. There are two forms we could give this definition; perhaps the best is the inductive one. We have:

(4) $\emptyset \in \mathrm{Con}_{\mathbf{T}}$;

(5) $u \cup \{\Delta_{\mathbf{T}}\} \in \mathrm{Con}_{\mathbf{T}}$ whenever $u \in \mathrm{Con}_{\mathbf{T}}$;

(6) $\{(X, \Delta) \mid X \in w\} \in \mathrm{Con}_{\mathbf{T}}$ whenever $w \in \mathrm{Con}_{\mathbf{A}}$;

(7) $\{(\Delta, (Y, \Delta_{\mathbf{T}})) \mid Y \in u\} \cup \{(\Delta, (\Delta_{\mathbf{T}}, Z)) \mid Z \in v\} \in \mathrm{Con}_{\mathbf{T}}$ whenever $u, v \in \mathrm{Con}_{\mathbf{T}}$.

Conditions (4)–(7) certainly make the inductive character of $\mathrm{Con}_{\mathbf{T}}$ clear—again, let us emphasize, the set being specified is the least such. Also clear from the definition is the fact that a consistent set of $\mathbf{T}$—aside from containing $\Delta_{\mathbf{T}}$—is either a copy of a consistent set of $\mathbf{A}$ or a copy of a consistent set of $\mathbf{T} \times \mathbf{T}$. We could thus state a set-theoretical equation for $\mathrm{Con}_{\mathbf{T}}$ similar to the one for $D_{\mathbf{T}}$.

It remains to define entailment for $\mathbf{T}$. Here are the inductive clauses which are pretty much forced on us by our objective of solving the domain equation:

(8) $u \vdash_{\mathbf{T}} \Delta_{\mathbf{T}}$ always;

(9) $u \cup \{\Delta_{\mathbf{T}}\} \vdash_{\mathbf{T}} Y$ whenever $u \vdash_{\mathbf{T}} Y$;

(10) $\{(X, \Delta) \mid X \in w\} \vdash_{\mathbf{T}} (W, \Delta)$ whenever $w \vdash_{\mathbf{A}} W$;

(11) $\{(\Delta, (Y, \Delta_{\mathbf{T}})) \mid Y \in u\} \cup \{(\Delta, (\Delta_{\mathbf{T}}, Z)) \mid Z \in v\} \vdash_{\mathbf{T}} (\Delta, (X, \Delta_{\mathbf{T}}))$ whenever $u \vdash_{\mathbf{T}} X$ and $v \in \mathrm{Con}_{\mathbf{T}}$; and

(12) $\{(\Delta,(Y,\Delta_{\mathbf{T}})) \mid Y \in u\} \cup \{(\Delta,(\Delta_{\mathbf{T}},Z)) \mid Z \in v\} \vdash_{\mathbf{T}} (\Delta,(\Delta_{\mathbf{T}},X))$ whenever $v \vdash_{\mathbf{T}} X$ and $u \in \mathrm{Con}_{\mathbf{T}}$.

Inductive definitions engender inductive proofs. It now has to be checked that consistency and entailment for $\mathbf{T}$ satisfy the axioms of 2.1. The steps needed for this check are mechanical. (The proof may be aided by noting that the cases in (4)–(7) and in (8)–(12) are disjoint—except for a trivial overlap between (8) and (9). The cases get invoked typically by asking, when confronted with an entailment to prove, for the nature of the data object on the right of the turnstile.)

Having defined and verified that $\mathbf{T}$ is an information system, the validity of the domain equation for $\mathbf{T}$ is secured by forming the right-hand side and noting that $\mathbf{T}$ is *identical* to $\mathbf{A} + (\mathbf{T} \times \mathbf{T})$. The reason is that we carefully chose the notation to match the official definitions of sums and products. (In general, in solving domain equations some transformation might have to take place to "re-format" data objects if things are not set up to be literally the same.)

It should be remarked that the sense can be made precise in which $\mathbf{T}$ is the *least* solution of the given domain equation. (It is an initial algebra in a suitable category of algebras and algebra homomorphisms.) It is pretty obvious that it is minimal in some sense, because we put into it only what was strictly required by the problem and nothing more.

It is also fairly obvious that there are *many* solutions to this domain equation. A non-constructive way to obtain non-minimal solutions is to interpret the whole construction of $\mathbf{T}$ in a non-standard model of set theory. Though, in the definition of $D_{\mathbf{T}}$, it looks like we are only working with very finite objects, everything we did could be made abstract and could be carried out in some funny universe. The result would be a system of "finite" data objects having all the right formal properties but containing things not in the standard minimal system. We would then take the notions of consistency and entailment that also exist in the funny universe and restrict them to sets of data objects that are actually finite in the standard sense. It can be seen from the formal properties of the construction that the resulting notions satisfy our axioms for an information system and that the domain equation holds—BUT the system would have many different elements beyond what we put into the original $\mathbf{T}$. To make this construction work, by the way, we would have to force $\mathbf{A}$ to be absolute in the model: if it is actually finite (say, $\mathbf{A} = \mathbf{BOOL}$), then there is no problem. (Constructive methods for introducing "nonstandard" data objects can also be given.)

Finally, we must remark on why we called $\mathbf{T}$ a domain of S-expressions. The answer becomes clear when we structure $\mathbf{T}$ as an algebra. First, there is an approximable mapping

$$\mathrm{atom} : \mathbf{A} \to \mathbf{T},$$

which injects $\mathbf{A}$ into $\mathbf{T}$ making the elements of $\mathbf{A}$ "atoms" of $\mathbf{T}$. Then there is a truth-valued predicate on $\mathbf{T}$ which decides whether an element is an atom:

$$\mathrm{isatom} : \mathbf{T} \to \mathbf{BOOL}.$$

Finally, since $\mathbf{T} \times \mathbf{T}$ is a part of $\mathbf{T}$, we can redefine the paring functions so that:

$$\mathrm{pair} : \mathbf{T} \times \mathbf{T} \to \mathbf{T}, \quad \mathrm{fst} : \mathbf{T} \to \mathbf{T}, \quad \text{and } \mathrm{snd} : \mathbf{T} \to \mathbf{T}.$$

In LISP terminology, these operations are the same as the familiar cons, car, and cdr. This makes $\mathbf{T}$ into an algebra where, starting from atoms, elements—expressions—can be built up by iterated pairing.

But why is our system different from the usual way of regarding S-expressions? The answer is that by including partial expressions (those involving $\Delta_{\mathbf{T}}$) and by completing the domain with limits, *infinite* expressions are introduced. For instance, if $a \in |\mathbf{T}|$, then we can solve the fixed-point equation:

$$x = \mathrm{pair}(\mathrm{atom}(a), x),$$

which is an infinite list of a's. This is but one example; the possibilities have been discussed in many papers too numerous to mention here.

As is common to remark, S-expressions can also be thought of as trees: the parse tree that gives the grammatical form of the expression. What we have added to the idea of a tree is the possibility of having infinite trees, and having all these trees as elements of a domain. ■

proposition_3_5_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottLambda_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottLambda
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- `lambda` preserves directed suprema: both sides evaluate, pointwise at `(x, y)`, to
`⨆ {f (x, y) | f ∈ 𝓕}`, since suprema in every function lattice involved are pointwise
    . -/
theorem proposition_3_5_preservesDirectedSup :
    PreservesDirectedSup
      (scottLambda : ScottMap (D × D') D'' → ScottMap D (ScottMap D' D'')) := by
  intro 𝓕 h𝓕 h𝓕dir
  apply ScottMap.ext
  intro x
  apply ScottMap.ext
  intro y
  rw [scottLambda_apply, ScottMap.sSup_apply, ScottMap.sSup_apply, ScottMap.sSup_apply
    ,
    Set.image_image, Set.image_image]
  congr 1

end DomainTheory.ContinuousLattice
```

7

proposition_3_5_preservesDirectedSup.lean

```
module
```

```
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottLambda_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottLambda
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- `lambda` preserves directed suprema: both sides evaluate, pointwise at `(x, y)`, to
`⊔ {f (x, y) | f ∈ 𝓕}`, since suprema in every function lattice involved are pointwise
    . -/
theorem proposition_3_5_preservesDirectedSup :
    PreservesDirectedSup
      (scottLambda : ScottMap (D × D') D'' → ScottMap D (ScottMap D' D'')) := by
  intro 𝓕 h𝓕 h𝓕dir
  apply ScottMap.ext
  intro x
  apply ScottMap.ext
  intro y
  rw [scottLambda_apply, ScottMap.sSup_apply, ScottMap.sSup_apply, ScottMap.sSup_apply
    ,
    Set.image_image, Set.image_image]
  congr 1

end DomainTheory.ContinuousLattice
```

8

proposition_3_5_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottLambda_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottLambda
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace
```

```
variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- `lambda` preserves directed suprema: both sides evaluate, pointwise at `(x, y)`, to
`⨆ {f (x, y) | f ∈ 𝓕}`, since suprema in every function lattice involved are pointwise
    . -/
theorem proposition_3_5_preservesDirectedSup :
    PreservesDirectedSup
      (scottLambda : ScottMap (D × D') D'' → ScottMap D (ScottMap D' D'')) := by
  intro 𝓕 h𝓕 h𝓕dir
  apply ScottMap.ext
  intro x
  apply ScottMap.ext
  intro y
  rw [scottLambda_apply, ScottMap.sSup_apply, ScottMap.sSup_apply, ScottMap.sSup_apply
    ,
    Set.image_image, Set.image_image]
  congr 1

end DomainTheory.ContinuousLattice
```

9

scottLambda.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottLambdaAt
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    lambda_outer_preservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Proposition 3.5.** Functional abstraction
```

```
`lambda : [[D × D'] → D''] → [D → [D' → D'']]`, `lambda f x y = f (x, y)`. By
    Theorem 3.3,
`[D → [D' → D'']]` is itself a continuous lattice, and `lambda f` is a Scott map. -/
noncomputable def scottLambda (f : ScottMap (D × D') D'') : ScottMap D (ScottMap D' D
    '') :=
  ⟨fun x => scottLambdaAt f x,
    continuous_of_preservesDirectedSup (lambda_outer_preservesDirectedSup f)⟩

end DomainTheory.ContinuousLattice
```

0

scottLambda.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottLambdaAt
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    lambda_outer_preservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Proposition 3.5.** Functional abstraction
`lambda : [[D × D'] → D''] → [D → [D' → D'']]`, `lambda f x y = f (x, y)`. By
    Theorem 3.3,
`[D → [D' → D'']]` is itself a continuous lattice, and `lambda f` is a Scott map. -/
noncomputable def scottLambda (f : ScottMap (D × D') D'') : ScottMap D (ScottMap D' D
    '') :=
  ⟨fun x => scottLambdaAt f x,
    continuous_of_preservesDirectedSup (lambda_outer_preservesDirectedSup f)⟩

end DomainTheory.ContinuousLattice
```

1

scottLambda.lean

```
module
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottLambdaAt
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    lambda_outer_preservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Proposition 3.5.** Functional abstraction
`lambda : [[D × D'] → D''] → [D → [D' → D'']]`, `lambda f x y = f (x, y)`. By
    Theorem 3.3,
`[D → [D' → D'']]` is itself a continuous lattice, and `lambda f` is a Scott map. -/
noncomputable def scottLambda (f : ScottMap (D × D') D'') : ScottMap D (ScottMap D' D
    '') :=
  ⟨fun x => scottLambdaAt f x,
    continuous_of_preservesDirectedSup (lambda_outer_preservesDirectedSup f)⟩

end DomainTheory.ContinuousLattice
```

2

scottLambda.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottLambdaAt
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    lambda_outer_preservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace
```

```
variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Proposition 3.5.** Functional abstraction
`lambda : [[D × D'] → D''] → [D → [D' → D'']]`, `lambda f x y = f (x, y)`. By
    Theorem 3.3,
`[D → [D' → D'']]` is itself a continuous lattice, and `lambda f` is a Scott map. -/
noncomputable def scottLambda (f : ScottMap (D × D') D'') : ScottMap D (ScottMap D' D
    '') :=
  ⟨fun x => scottLambdaAt f x,
    continuous_of_preservesDirectedSup (lambda_outer_preservesDirectedSup f)⟩

end DomainTheory.ContinuousLattice
```

3

scottLambda.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottLambdaAt
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    lambda_outer_preservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Proposition 3.5.** Functional abstraction
`lambda : [[D × D'] → D''] → [D → [D' → D'']]`, `lambda f x y = f (x, y)`. By
    Theorem 3.3,
`[D → [D' → D'']]` is itself a continuous lattice, and `lambda f` is a Scott map. -/
noncomputable def scottLambda (f : ScottMap (D × D') D'') : ScottMap D (ScottMap D' D
```

```
    '') :=
  ⟨fun x => scottLambdaAt f x,
    continuous_of_preservesDirectedSup (lambda_outer_preservesDirectedSup f)⟩

end DomainTheory.ContinuousLattice
```

4

scottLambda.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottLambdaAt
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    lambda_outer_preservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Proposition 3.5.** Functional abstraction
`lambda : [[D × D'] → D''] → [D → [D' → D'']]`, `lambda f x y = f (x, y)`. By
    Theorem 3.3,
`[D → [D' → D'']]` is itself a continuous lattice, and `lambda f` is a Scott map. -/
noncomputable def scottLambda (f : ScottMap (D × D') D'') : ScottMap D (ScottMap D' D
    '') :=
  ⟨fun x => scottLambdaAt f x,
    continuous_of_preservesDirectedSup (lambda_outer_preservesDirectedSup f)⟩

end DomainTheory.ContinuousLattice
```

5

scottLambda.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottLambdaAt
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    lambda_outer_preservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Proposition 3.5.** Functional abstraction
`lambda : [[D × D'] → D''] → [D → [D' → D'']]`, `lambda f x y = f (x, y)`. By
    Theorem 3.3,
`[D → [D' → D'']]` is itself a continuous lattice, and `lambda f` is a Scott map. -/
noncomputable def scottLambda (f : ScottMap (D × D') D'') : ScottMap D (ScottMap D' D
    '') :=
  ⟨fun x => scottLambdaAt f x,
    continuous_of_preservesDirectedSup (lambda_outer_preservesDirectedSup f)⟩

end DomainTheory.ContinuousLattice
```

6

scottLambda.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottLambdaAt
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    lambda_outer_preservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
```

```
variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Proposition 3.5.** Functional abstraction
`lambda : [[D × D'] → D''] → [D → [D' → D'']]`, `lambda f x y = f (x, y)`. By
    Theorem 3.3,
`[D → [D' → D'']]` is itself a continuous lattice, and `lambda f` is a Scott map. -/
noncomputable def scottLambda (f : ScottMap (D × D') D'') : ScottMap D (ScottMap D' D
    '') :=
  ⟨fun x => scottLambdaAt f x,
    continuous_of_preservesDirectedSup (lambda_outer_preservesDirectedSup f)⟩

end DomainTheory.ContinuousLattice
```

7

scottLambda.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottLambdaAt
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    lambda_outer_preservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Proposition 3.5.** Functional abstraction
`lambda : [[D × D'] → D''] → [D → [D' → D'']]`, `lambda f x y = f (x, y)`. By
    Theorem 3.3,
`[D → [D' → D'']]` is itself a continuous lattice, and `lambda f` is a Scott map. -/
noncomputable def scottLambda (f : ScottMap (D × D') D'') : ScottMap D (ScottMap D' D
    '') :=
  ⟨fun x => scottLambdaAt f x,
    continuous_of_preservesDirectedSup (lambda_outer_preservesDirectedSup f)⟩
```

```
end DomainTheory.ContinuousLattice
```

8

scottLambda.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottLambdaAt
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    lambda_outer_preservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Proposition 3.5.** Functional abstraction
`lambda : [[D × D'] → D''] → [D → [D' → D'']]`, `lambda f x y = f (x, y)`. By
    Theorem 3.3,
`[D → [D' → D'']]` is itself a continuous lattice, and `lambda f` is a Scott map. -/
noncomputable def scottLambda (f : ScottMap (D × D') D'') : ScottMap D (ScottMap D' D
    '') :=
  ⟨fun x => scottLambdaAt f x,
    continuous_of_preservesDirectedSup (lambda_outer_preservesDirectedSup f)⟩

end DomainTheory.ContinuousLattice
```

9

scottLambdaAt.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    curry_right_preservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
```

```
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Proposition 3.5 (right).** Currying in the `y`-variable is Scott-
    continuous. -/
noncomputable def scottLambdaAt (f : ScottMap (D × D') D'') (x : D) : ScottMap D' D''
    :=
  ⟨fun y => f (x, y), continuous_of_preservesDirectedSup (
    curry_right_preservesDirectedSup f x)⟩

end DomainTheory.ContinuousLattice
```

0

scottLambdaAt.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    curry_right_preservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Proposition 3.5 (right).** Currying in the `y`-variable is Scott-
```

```
    continuous. -/
noncomputable def scottLambdaAt (f : ScottMap (D × D') D'') (x : D) : ScottMap D' D''
    :=
  ⟨fun y => f (x, y), continuous_of_preservesDirectedSup (
    curry_right_preservesDirectedSup f x)⟩

end DomainTheory.ContinuousLattice
```

1

scottLambdaAt.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    curry_right_preservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Proposition 3.5 (right).** Currying in the `y`-variable is Scott-
    continuous. -/
noncomputable def scottLambdaAt (f : ScottMap (D × D') D'') (x : D) : ScottMap D' D''
    :=
  ⟨fun y => f (x, y), continuous_of_preservesDirectedSup (
    curry_right_preservesDirectedSup f x)⟩

end DomainTheory.ContinuousLattice
```

2

scottLambdaAt.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    curry_right_preservesDirectedSup
```

```
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Proposition 3.5 (right).** Currying in the `y`-variable is Scott-
    continuous. -/
noncomputable def scottLambdaAt (f : ScottMap (D × D') D'') (x : D) : ScottMap D' D''
    :=
  ⟨fun y => f (x, y), continuous_of_preservesDirectedSup (
    curry_right_preservesDirectedSup f x)⟩

end DomainTheory.ContinuousLattice
```

3

scottLambdaAt.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    curry_right_preservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
```

```
    CompleteLattice D₁']

/-- **Scott 1972, Proposition 3.5 (right).** Currying in the `y`-variable is Scott-
    continuous. -/
noncomputable def scottLambdaAt (f : ScottMap (D × D') D'') (x : D) : ScottMap D' D''
    :=
  ⟨fun y => f (x, y), continuous_of_preservesDirectedSup (
    curry_right_preservesDirectedSup f x)⟩

end DomainTheory.ContinuousLattice
```

4

scottLambdaAt.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    curry_right_preservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Proposition 3.5 (right).** Currying in the `y`-variable is Scott-
    continuous. -/
noncomputable def scottLambdaAt (f : ScottMap (D × D') D'') (x : D) : ScottMap D' D''
    :=
  ⟨fun y => f (x, y), continuous_of_preservesDirectedSup (
    curry_right_preservesDirectedSup f x)⟩

end DomainTheory.ContinuousLattice
```

5

scottLambdaAt.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    curry_right_preservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Proposition 3.5 (right).** Currying in the `y`-variable is Scott-
    continuous. -/
noncomputable def scottLambdaAt (f : ScottMap (D × D') D'') (x : D) : ScottMap D' D''
    :=
  ⟨fun y => f (x, y), continuous_of_preservesDirectedSup (
    curry_right_preservesDirectedSup f x)⟩

end DomainTheory.ContinuousLattice
```

6

scottLambdaAt.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    curry_right_preservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
```

```
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Proposition 3.5 (right).** Currying in the `y`-variable is Scott-
    continuous. -/
noncomputable def scottLambdaAt (f : ScottMap (D × D') D'') (x : D) : ScottMap D' D''
    :=
  ⟨fun y => f (x, y), continuous_of_preservesDirectedSup (
    curry_right_preservesDirectedSup f x)⟩

end DomainTheory.ContinuousLattice
```

7

scottLambdaAt.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    curry_right_preservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Proposition 3.5 (right).** Currying in the `y`-variable is Scott-
    continuous. -/
noncomputable def scottLambdaAt (f : ScottMap (D × D') D'') (x : D) : ScottMap D' D''
    :=
  ⟨fun y => f (x, y), continuous_of_preservesDirectedSup (
    curry_right_preservesDirectedSup f x)⟩

end DomainTheory.ContinuousLattice
```

8

scottLambdaAt.lean

```
module
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    curry_right_preservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Proposition 3.5 (right).** Currying in the `y`-variable is Scott-
    continuous. -/
noncomputable def scottLambdaAt (f : ScottMap (D × D') D'') (x : D) : ScottMap D' D''
    :=
  ⟨fun y => f (x, y), continuous_of_preservesDirectedSup (
    curry_right_preservesDirectedSup f x)⟩

end DomainTheory.ContinuousLattice
```

9

scottLambda_apply.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.scottLambda
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
```

```
    CompleteLattice D₁']

@[simp] theorem scottLambda_apply (f : ScottMap (D × D') D'') (x : D) (y : D') :
    ((scottLambda f x : ScottMap D' D'') : D' → D'') y = (f : (D × D') → D'') (x, y)
    :=
  rfl

end DomainTheory.ContinuousLattice
```

0

scottLambda_apply.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.scottLambda
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

@[simp] theorem scottLambda_apply (f : ScottMap (D × D') D'') (x : D) (y : D') :
    ((scottLambda f x : ScottMap D' D'') : D' → D'') y = (f : (D × D') → D'') (x, y)
    :=
  rfl

end DomainTheory.ContinuousLattice
```

1

scottLambda_apply.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.scottLambda
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace
```

```
variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

@[simp] theorem scottLambda_apply (f : ScottMap (D × D') D'') (x : D) (y : D') :
    ((scottLambda f x : ScottMap D' D'') : D' → D'') y = (f : (D × D') → D'') (x, y)
    :=
  rfl

end DomainTheory.ContinuousLattice
```

2

scottLambda_apply.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.scottLambda
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

@[simp] theorem scottLambda_apply (f : ScottMap (D × D') D'') (x : D) (y : D') :
    ((scottLambda f x : ScottMap D' D'') : D' → D'') y = (f : (D × D') → D'') (x, y)
    :=
  rfl

end DomainTheory.ContinuousLattice
```

3

scottLambda_apply.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.scottLambda
```

```
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

@[simp] theorem scottLambda_apply (f : ScottMap (D × D') D'') (x : D) (y : D') :
    ((scottLambda f x : ScottMap D' D'') : D' → D'') y = (f : (D × D') → D'') (x, y)
    :=
  rfl

end DomainTheory.ContinuousLattice
```

4

scottLambda_apply.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.scottLambda
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

@[simp] theorem scottLambda_apply (f : ScottMap (D × D') D'') (x : D) (y : D') :
    ((scottLambda f x : ScottMap D' D'') : D' → D'') y = (f : (D × D') → D'') (x, y)
    :=
  rfl

end DomainTheory.ContinuousLattice
```

5

scottLambda_apply.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.scottLambda
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

@[simp] theorem scottLambda_apply (f : ScottMap (D × D') D'') (x : D) (y : D') :
    ((scottLambda f x : ScottMap D' D'') : D' → D'') y = (f : (D × D') → D'') (x, y)
    :=
  rfl

end DomainTheory.ContinuousLattice
```

6

scottLambda_apply.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.scottLambda
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']
```

```
@[simp] theorem scottLambda_apply (f : ScottMap (D × D') D'') (x : D) (y : D') :
    ((scottLambda f x : ScottMap D' D'') : D' → D'') y = (f : (D × D') → D'') (x, y)
    :=
  rfl

end DomainTheory.ContinuousLattice
```

7

scottLambda_apply.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.scottLambda
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

@[simp] theorem scottLambda_apply (f : ScottMap (D × D') D'') (x : D) (y : D') :
    ((scottLambda f x : ScottMap D' D'') : D' → D'') y = (f : (D × D') → D'') (x, y)
    :=
  rfl

end DomainTheory.ContinuousLattice
```

8

scottLambda_apply.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.scottLambda
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace
```

```
variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

@[simp] theorem scottLambda_apply (f : ScottMap (D × D') D'') (x : D) (y : D') :
    ((scottLambda f x : ScottMap D' D'') : D' → D'') y = (f : (D × D') → D'') (x, y)
    :=
  rfl

end DomainTheory.ContinuousLattice
```

9

IsContinuousLatticeProjection.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeRetraction
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

universe u v

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Definition 3.6.** A *projection* of continuous lattices: a retract
    with
`i ∘ j ⊑ id`. -/
structure IsContinuousLatticeProjection (D : Type u) (D' : Type v)
    [CompleteLattice D] [CompleteLattice D']
    extends IsContinuousLatticeRetraction D D' where
  incl_retr_le : ∀ d,
    @LE.le D'
      (ChainCompletePartialOrder.instOfCompleteLattice (α := D')).toPartialOrder.
    toPreorder.toLE
      (incl (retr d)) d
```

```
end DomainTheory.ContinuousLattice
```

0

IsContinuousLatticeProjection.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeRetraction
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

universe u v

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Definition 3.6.** A *projection* of continuous lattices: a retract
    with
`i ∘ j ⊑ id`. -/
structure IsContinuousLatticeProjection (D : Type u) (D' : Type v)
    [CompleteLattice D] [CompleteLattice D']
    extends IsContinuousLatticeRetraction D D' where
  incl_retr_le : ∀ d,
    @LE.le D'
      (ChainCompletePartialOrder.instOfCompleteLattice (α := D')).toPartialOrder.
    toPreorder.toLE
      (incl (retr d)) d

end DomainTheory.ContinuousLattice
```

1

IsContinuousLatticeProjection.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeRetraction
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic
```

```
public import Mathlib.Data.Set.Basic

@[expose] public section

universe u v

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Definition 3.6.** A *projection* of continuous lattices: a retract
    with
`i ∘ j ⊑ id`. -/
structure IsContinuousLatticeProjection (D : Type u) (D' : Type v)
    [CompleteLattice D] [CompleteLattice D']
    extends IsContinuousLatticeRetraction D D' where
  incl_retr_le : ∀ d,
    @LE.le D'
      (ChainCompletePartialOrder.instOfCompleteLattice (α := D')).toPartialOrder.
    toPreorder.toLE
      (incl (retr d)) d

end DomainTheory.ContinuousLattice
```

2

IsContinuousLatticeProjection.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeRetraction
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

universe u v

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]
```

```
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Definition 3.6.** A *projection* of continuous lattices: a retract
    with
`i ∘ j ⊑ id`. -/
structure IsContinuousLatticeProjection (D : Type u) (D' : Type v)
    [CompleteLattice D] [CompleteLattice D']
    extends IsContinuousLatticeRetraction D D' where
  incl_retr_le : ∀ d,
    @LE.le D'
      (ChainCompletePartialOrder.instOfCompleteLattice (α := D')).toPartialOrder.
    toPreorder.toLE
      (incl (retr d)) d

end DomainTheory.ContinuousLattice
```

3

IsContinuousLatticeProjection.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeRetraction
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

universe u v

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Definition 3.6.** A *projection* of continuous lattices: a retract
    with
`i ∘ j ⊑ id`. -/
structure IsContinuousLatticeProjection (D : Type u) (D' : Type v)
    [CompleteLattice D] [CompleteLattice D']
    extends IsContinuousLatticeRetraction D D' where
  incl_retr_le : ∀ d,
```

```
    @LE.le D'
      (ChainCompletePartialOrder.instOfCompleteLattice (α := D')).toPartialOrder.
    toPreorder.toLE
      (incl (retr d)) d

end DomainTheory.ContinuousLattice
```

4

IsContinuousLatticeProjection.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeRetraction
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

universe u v

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Definition 3.6.** A *projection* of continuous lattices: a retract
    with
`i ∘ j ⊑ id`. -/
structure IsContinuousLatticeProjection (D : Type u) (D' : Type v)
    [CompleteLattice D] [CompleteLattice D']
    extends IsContinuousLatticeRetraction D D' where
  incl_retr_le : ∀ d,
    @LE.le D'
      (ChainCompletePartialOrder.instOfCompleteLattice (α := D')).toPartialOrder.
    toPreorder.toLE
      (incl (retr d)) d

end DomainTheory.ContinuousLattice
```

5

IsContinuousLatticeProjection.lean

```
module
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeRetraction
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

universe u v

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Definition 3.6.** A *projection* of continuous lattices: a retract
    with
`i ∘ j ⊑ id`. -/
structure IsContinuousLatticeProjection (D : Type u) (D' : Type v)
    [CompleteLattice D] [CompleteLattice D']
    extends IsContinuousLatticeRetraction D D' where
  incl_retr_le : ∀ d,
    @LE.le D'
      (ChainCompletePartialOrder.instOfCompleteLattice (α := D')).toPartialOrder.
    toPreorder.toLE
      (incl (retr d)) d

end DomainTheory.ContinuousLattice
```

6

IsContinuousLatticeProjection.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeRetraction
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

universe u v

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace
```

```
variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Definition 3.6.** A *projection* of continuous lattices: a retract
    with
`i ∘ j ⊑ id`. -/
structure IsContinuousLatticeProjection (D : Type u) (D' : Type v)
    [CompleteLattice D] [CompleteLattice D']
    extends IsContinuousLatticeRetraction D D' where
  incl_retr_le : ∀ d,
    @LE.le D'
      (ChainCompletePartialOrder.instOfCompleteLattice (α := D')).toPartialOrder.
    toPreorder.toLE
      (incl (retr d)) d

end DomainTheory.ContinuousLattice
```

7

IsContinuousLatticeProjection.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeRetraction
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

universe u v

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Definition 3.6.** A *projection* of continuous lattices: a retract
    with
`i ∘ j ⊑ id`. -/
```

```
structure IsContinuousLatticeProjection (D : Type u) (D' : Type v)
    [CompleteLattice D] [CompleteLattice D']
    extends IsContinuousLatticeRetraction D D' where
  incl_retr_le : ∀ d,
    @LE.le D'
      (ChainCompletePartialOrder.instOfCompleteLattice (α := D')).toPartialOrder.
    toPreorder.toLE
      (incl (retr d)) d

end DomainTheory.ContinuousLattice
```

8

IsContinuousLatticeProjection.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeRetraction
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

universe u v

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Definition 3.6.** A *projection* of continuous lattices: a retract
    with
`i ∘ j ⊑ id`. -/
structure IsContinuousLatticeProjection (D : Type u) (D' : Type v)
    [CompleteLattice D] [CompleteLattice D']
    extends IsContinuousLatticeRetraction D D' where
  incl_retr_le : ∀ d,
    @LE.le D'
      (ChainCompletePartialOrder.instOfCompleteLattice (α := D')).toPartialOrder.
    toPreorder.toLE
      (incl (retr d)) d

end DomainTheory.ContinuousLattice
```

9

IsContinuousLatticeRetraction.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

universe u v

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Definition 3.6.** A *retraction* of continuous lattices. -/
structure IsContinuousLatticeRetraction (D : Type u) (D' : Type v)
    [CompleteLattice D] [CompleteLattice D'] where
  incl : ScottMap D D'
  retr : ScottMap D' D
  retr_incl : ∀ d, retr (incl d) = d

end DomainTheory.ContinuousLattice
```

0

IsContinuousLatticeRetraction.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

universe u v

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]
```

```
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Definition 3.6.** A *retraction* of continuous lattices. -/
structure IsContinuousLatticeRetraction (D : Type u) (D' : Type v)
    [CompleteLattice D] [CompleteLattice D'] where
  incl : ScottMap D D'
  retr : ScottMap D' D
  retr_incl : ∀ d, retr (incl d) = d

end DomainTheory.ContinuousLattice
```

1

IsContinuousLatticeRetraction.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

universe u v

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Definition 3.6.** A *retraction* of continuous lattices. -/
structure IsContinuousLatticeRetraction (D : Type u) (D' : Type v)
    [CompleteLattice D] [CompleteLattice D'] where
  incl : ScottMap D D'
  retr : ScottMap D' D
  retr_incl : ∀ d, retr (incl d) = d

end DomainTheory.ContinuousLattice
```

2

IsContinuousLatticeRetraction.lean

```
module
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

universe u v

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Definition 3.6.** A *retraction* of continuous lattices. -/
structure IsContinuousLatticeRetraction (D : Type u) (D' : Type v)
    [CompleteLattice D] [CompleteLattice D'] where
  incl : ScottMap D D'
  retr : ScottMap D' D
  retr_incl : ∀ d, retr (incl d) = d

end DomainTheory.ContinuousLattice
```

3

IsContinuousLatticeRetraction.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

universe u v

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']
```

```
/-- **Scott 1972, Definition 3.6.** A *retraction* of continuous lattices. -/
structure IsContinuousLatticeRetraction (D : Type u) (D' : Type v)
    [CompleteLattice D] [CompleteLattice D'] where
  incl : ScottMap D D'
  retr : ScottMap D' D
  retr_incl : ∀ d, retr (incl d) = d

end DomainTheory.ContinuousLattice
```

4

IsContinuousLatticeRetraction.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

universe u v

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Definition 3.6.** A *retraction* of continuous lattices. -/
structure IsContinuousLatticeRetraction (D : Type u) (D' : Type v)
    [CompleteLattice D] [CompleteLattice D'] where
  incl : ScottMap D D'
  retr : ScottMap D' D
  retr_incl : ∀ d, retr (incl d) = d

end DomainTheory.ContinuousLattice
```

5

IsContinuousLatticeRetraction.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic
```

```
@[expose] public section

universe u v

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Definition 3.6.** A *retraction* of continuous lattices. -/
structure IsContinuousLatticeRetraction (D : Type u) (D' : Type v)
    [CompleteLattice D] [CompleteLattice D'] where
  incl : ScottMap D D'
  retr : ScottMap D' D
  retr_incl : ∀ d, retr (incl d) = d

end DomainTheory.ContinuousLattice
```

6

IsContinuousLatticeRetraction.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

universe u v

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Definition 3.6.** A *retraction* of continuous lattices. -/
structure IsContinuousLatticeRetraction (D : Type u) (D' : Type v)
    [CompleteLattice D] [CompleteLattice D'] where
```

```
  incl : ScottMap D D'
  retr : ScottMap D' D
  retr_incl : ∀ d, retr (incl d) = d

end DomainTheory.ContinuousLattice
```

7

IsContinuousLatticeRetraction.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

universe u v

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Definition 3.6.** A *retraction* of continuous lattices. -/
structure IsContinuousLatticeRetraction (D : Type u) (D' : Type v)
    [CompleteLattice D] [CompleteLattice D'] where
  incl : ScottMap D D'
  retr : ScottMap D' D
  retr_incl : ∀ d, retr (incl d) = d

end DomainTheory.ContinuousLattice
```

8

IsContinuousLatticeRetraction.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

universe u v
```

```
namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Definition 3.6.** A *retraction* of continuous lattices. -/
structure IsContinuousLatticeRetraction (D : Type u) (D' : Type v)
    [CompleteLattice D] [CompleteLattice D'] where
  incl : ScottMap D D'
  retr : ScottMap D' D
  retr_incl : ∀ d, retr (incl d) = d

end DomainTheory.ContinuousLattice
```

9

IsProjection.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.comp
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst28
public import DomainTheory.ContinuousLattice.FunctionSpaces.idMap
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

/-- **Scott 1972, Definition 3.11.** `J_D = { j ∈ [D → D] : j = j ∘ j ⊑ id }`: the
    Scott-continuous
projections of `D` (idempotent self-maps below the identity), as a predicate on `[D →
    D]`. -/
def IsProjection (j : ScottMap D D) : Prop :=
```

```
  j.comp j = j ∧ j ≤ ScottMap.idMap

end Projections

end DomainTheory.ContinuousLattice
```

0

IsProjection.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.comp
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst28
public import DomainTheory.ContinuousLattice.FunctionSpaces.idMap
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

/-- **Scott 1972, Definition 3.11.** `J_D = { j ∈ [D → D] : j = j ∘ j ⊑ id }`: the
    Scott-continuous
projections of `D` (idempotent self-maps below the identity), as a predicate on `[D →
    D]`. -/
def IsProjection (j : ScottMap D D) : Prop :=
  j.comp j = j ∧ j ≤ ScottMap.idMap

end Projections

end DomainTheory.ContinuousLattice
```

1

IsProjection.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.comp
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst28
public import DomainTheory.ContinuousLattice.FunctionSpaces.idMap
public import Mathlib.Topology.Defs.Basic
```

```
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

/-- **Scott 1972, Definition 3.11.** `J_D = { j ∈ [D → D] : j = j ∘ j ⊑ id }`: the
    Scott-continuous
projections of `D` (idempotent self-maps below the identity), as a predicate on `[D →
    D]`. -/
def IsProjection (j : ScottMap D D) : Prop :=
  j.comp j = j ∧ j ≤ ScottMap.idMap

end Projections

end DomainTheory.ContinuousLattice
```

2

IsProjection.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.comp
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst28
public import DomainTheory.ContinuousLattice.FunctionSpaces.idMap
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']
```

```
section Projections

/-- **Scott 1972, Definition 3.11.** `J_D = { j ∈ [D → D] : j = j ∘ j ⊑ id }`: the
    Scott-continuous
projections of `D` (idempotent self-maps below the identity), as a predicate on `[D →
    D]`. -/
def IsProjection (j : ScottMap D D) : Prop :=
  j.comp j = j ∧ j ≤ ScottMap.idMap

end Projections

end DomainTheory.ContinuousLattice
```

3

IsProjection.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.comp
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst28
public import DomainTheory.ContinuousLattice.FunctionSpaces.idMap
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

/-- **Scott 1972, Definition 3.11.** `J_D = { j ∈ [D → D] : j = j ∘ j ⊑ id }`: the
    Scott-continuous
projections of `D` (idempotent self-maps below the identity), as a predicate on `[D →
    D]`. -/
def IsProjection (j : ScottMap D D) : Prop :=
  j.comp j = j ∧ j ≤ ScottMap.idMap

end Projections

end DomainTheory.ContinuousLattice
```

4

IsProjection.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.comp
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst28
public import DomainTheory.ContinuousLattice.FunctionSpaces.idMap
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

/-- **Scott 1972, Definition 3.11.** `J_D = { j ∈ [D → D] : j = j ∘ j ⊑ id }`: the
    Scott-continuous
projections of `D` (idempotent self-maps below the identity), as a predicate on `[D →
    D]`. -/
def IsProjection (j : ScottMap D D) : Prop :=
  j.comp j = j ∧ j ≤ ScottMap.idMap

end Projections

end DomainTheory.ContinuousLattice
```

5

4.8.2 Factoid 8.2

A domain for λ-claculus. A lengthy discussion with many references on λ-calculus models can be found in Longo [1982]. All we wish to remark on here is how the method of construction by solving a domain equation can be fit into the new presentation. What I have added to the previous ideas (that in any case came out of an analysis of finite elements of models) is the general view of information systems. In particular the models obtained this way are *not* lattices—hence, the need for the calculations with Con. I hope that the presentation here makes it clearer how "pure" λ-calculus models can be related to other domains having other types of structures—for instance, those needed in denotational semantics.

The domain equation we wish to solve is:

$$\mathbf{D} \cong \mathbf{A} + (\mathbf{D} \to \mathbf{D}).$$

We proceed in much the same way we did for $\mathbf{T}$, except we must now put in data objects appropriate to the function space. Here is construction, where again Δ is chosen outside $D_{\mathbf{A}}$ and $\Delta_{\mathbf{D}} = (\Delta, \Delta)$:

(1) $\Delta_{\mathbf{D}} \in D_{\mathbf{D}}$;

(2) $(X, \Delta) \in D_{\mathbf{D}}$ whenever $X \in D_{\mathbf{A}}$;

(3) $(\Delta, (u, v)) \in D_{\mathbf{D}}$ whenever $u, v \in \mathrm{Con}_{\mathbf{D}}$;

(4) $\emptyset \in \mathrm{Con}_{\mathbf{D}}$;

(5) $u \cup \{\Delta_{\mathbf{D}}\} \in \mathrm{Con}_{\mathbf{D}}$ whenever $u \in \mathrm{Con}_{\mathbf{D}}$;

(6) $\{(X, \Delta) \mid X \in w\} \in \mathrm{Con}_{\mathbf{D}}$ whenever $w \in \mathrm{Con}_{\mathbf{A}}$; and

(7) $\{(\Delta, (u_0, v_0)), \ldots, (\Delta, (u_{n-1}, v_{n-1}))\} \in \mathrm{Con}_{\mathbf{D}}$ provided $u_i, v_i \in \mathrm{Con}_{\mathbf{D}}$ for all $i < n$ and whenever $I \subseteq \{0, \ldots, n-1\}$ and $\bigcup\{u_i \mid i \in I\} \in \mathrm{Con}_{\mathbf{D}}$, then $\bigcup\{v_i \mid i \in I\} \in \mathrm{Con}_{\mathbf{D}}$.

What is different here from the definition of $\mathbf{T}$ is the fact that the concepts $D_{\mathbf{D}}$ and $\mathrm{Con}_{\mathbf{D}}$ are mutually recursive because the data objects are themselves built from consistent sets. The scheme is based on a combination of the sum construct and the function-space construct, but the mutual recursion allows "feedback" to occur.

To complete the definition we have to give the clauses for the inductive definition of entailment. They are:

(8) $u \vdash_{\mathbf{D}} \Delta_{\mathbf{D}}$ always;

(9) $u \cup \{\Delta_{\mathbf{D}}\} \vdash_{\mathbf{D}} Y$ whenever $u \vdash_{\mathbf{D}} Y$;

(10) $\{(X, \Delta) \mid X \in w\} \vdash_{\mathbf{D}} (W, \Delta)$ whenever $w \vdash_{\mathbf{A}} W$;

(11) $\{(\Delta, (u_0, v_0)), \ldots, (\Delta, (u_{n-1}, v_{n-1}))\} \vdash_{\mathbf{D}} (\Delta, (u', v'))$ whenever $\bigcup\{v_i \mid u' \vdash_{\mathbf{D}} u_i\} \vdash_{\mathbf{D}} v'$ and the set on the left is in $\mathrm{Con}_{\mathbf{D}}$.

Obviously these definitions are much shorter if we have a domain in which all sets are consistent, but there are many reasons for retaining the consistency concept throughout. The check that $\mathbf{D}$ is an information system and satisfies the domain equation is mechanical. We cannot detail here how this construction provides a λ-calculus model.

It is clear that these definitions are constructive, and that, with a suitable Gödel numbering of the data objects, the predicates for consistency and entailment are recursively enumerable. However, the recursion used builds up the predicates by going from less complicated data objects to more complicated ones; therefore, the predicates must be recursive, because, for a certain size data object, the derivation that puts it into the predicate is of a bounded length. This observation helps in the discussion of the computability of the operators defined on these domains—another topic we cannot discuss here. ■

IsProjection.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.comp
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst28
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaces.idMap
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

/-- **Scott 1972, Definition 3.11.** `J_D = { j ∈ [D → D] : j = j ∘ j ⊑ id }`: the
    Scott-continuous
projections of `D` (idempotent self-maps below the identity), as a predicate on `[D →
    D]`. -/
def IsProjection (j : ScottMap D D) : Prop :=
  j.comp j = j ∧ j ≤ ScottMap.idMap

end Projections

end DomainTheory.ContinuousLattice
```

6

IsProjection.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.comp
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst28
public import DomainTheory.ContinuousLattice.FunctionSpaces.idMap
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
```

```
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

/-- **Scott 1972, Definition 3.11.** `J_D = { j ∈ [D → D] : j = j ∘ j ⊑ id }`: the
    Scott-continuous
projections of `D` (idempotent self-maps below the identity), as a predicate on `[D →
    D]`. -/
def IsProjection (j : ScottMap D D) : Prop :=
  j.comp j = j ∧ j ≤ ScottMap.idMap

end Projections

end DomainTheory.ContinuousLattice
```

7

IsProjection.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.comp
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst28
public import DomainTheory.ContinuousLattice.FunctionSpaces.idMap
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

/-- **Scott 1972, Definition 3.11.** `J_D = { j ∈ [D → D] : j = j ∘ j ⊑ id }`: the
    Scott-continuous
projections of `D` (idempotent self-maps below the identity), as a predicate on `[D →
    D]`. -/
def IsProjection (j : ScottMap D D) : Prop :=
  j.comp j = j ∧ j ≤ ScottMap.idMap

end Projections

end DomainTheory.ContinuousLattice
```

8

IsProjection.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.comp
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst28
public import DomainTheory.ContinuousLattice.FunctionSpaces.idMap
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

/-- **Scott 1972, Definition 3.11.** `J_D = { j ∈ [D → D] : j = j ∘ j ⊑ id }`: the
    Scott-continuous
projections of `D` (idempotent self-maps below the identity), as a predicate on `[D →
    D]`. -/
def IsProjection (j : ScottMap D D) : Prop :=
  j.comp j = j ∧ j ≤ ScottMap.idMap

end Projections

end DomainTheory.ContinuousLattice
```

9

converseProjection.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_converseRetr_le
public import DomainTheory.ContinuousLattice.FunctionSpaces.converseRetr_incl
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    converseRetr_preservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
```

```
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- The projection assembled from a map `i` satisfying 3.10(i)-(iii). -/
noncomputable def converseProjection
    (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S)) (hinj : Function.Injective i)
    (hwb : ∀ {x y : D}, x ≪ y → i x ≪ i y) (hD : ContinuousLattice D) :
    IsContinuousLatticeProjection D D' where
  incl := ⟨i, continuous_of_preservesDirectedSup (fun _ _ _ => hi _)⟩
  retr := ⟨converseRetr i, continuous_of_preservesDirectedSup
    (converseRetr_preservesDirectedSup hi hwb hD)⟩
  retr_incl := fun d => converseRetr_incl hi hinj d
  incl_retr_le := fun x' => incl_converseRetr_le hi x'

end Proposition310

end DomainTheory.ContinuousLattice
```

0

converseProjection.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_converseRetr_le
public import DomainTheory.ContinuousLattice.FunctionSpaces.converseRetr_incl
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    converseRetr_preservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
```

```
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- The projection assembled from a map `i` satisfying 3.10(i)-(iii). -/
noncomputable def converseProjection
    (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S)) (hinj : Function.Injective i)
    (hwb : ∀ {x y : D}, x ≪ y → i x ≪ i y) (hD : ContinuousLattice D) :
    IsContinuousLatticeProjection D D' where
  incl := ⟨i, continuous_of_preservesDirectedSup (fun _ _ _ => hi _)⟩
  retr := ⟨converseRetr i, continuous_of_preservesDirectedSup
    (converseRetr_preservesDirectedSup hi hwb hD)⟩
  retr_incl := fun d => converseRetr_incl hi hinj d
  incl_retr_le := fun x' => incl_converseRetr_le hi x'

end Proposition310

end DomainTheory.ContinuousLattice
```

1

converseProjection.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_converseRetr_le
public import DomainTheory.ContinuousLattice.FunctionSpaces.converseRetr_incl
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    converseRetr_preservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic
```

```
@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- The projection assembled from a map `i` satisfying 3.10(i)-(iii). -/
noncomputable def converseProjection
    (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S)) (hinj : Function.Injective i)
    (hwb : ∀ {x y : D}, x ≪ y → i x ≪ i y) (hD : ContinuousLattice D) :
    IsContinuousLatticeProjection D D' where
  incl := ⟨i, continuous_of_preservesDirectedSup (fun _ _ _ => hi _)⟩
  retr := ⟨converseRetr i, continuous_of_preservesDirectedSup
    (converseRetr_preservesDirectedSup hi hwb hD)⟩
  retr_incl := fun d => converseRetr_incl hi hinj d
  incl_retr_le := fun x' => incl_converseRetr_le hi x'

end Proposition310

end DomainTheory.ContinuousLattice
```

2

converseProjection.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_converseRetr_le
public import DomainTheory.ContinuousLattice.FunctionSpaces.converseRetr_incl
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    converseRetr_preservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic
```

```
@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- The projection assembled from a map `i` satisfying 3.10(i)-(iii). -/
noncomputable def converseProjection
    (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S)) (hinj : Function.Injective i)
    (hwb : ∀ {x y : D}, x ≪ y → i x ≪ i y) (hD : ContinuousLattice D) :
    IsContinuousLatticeProjection D D' where
  incl := ⟨i, continuous_of_preservesDirectedSup (fun _ _ _ => hi _)⟩
  retr := ⟨converseRetr i, continuous_of_preservesDirectedSup
    (converseRetr_preservesDirectedSup hi hwb hD)⟩
  retr_incl := fun d => converseRetr_incl hi hinj d
  incl_retr_le := fun x' => incl_converseRetr_le hi x'

end Proposition310

end DomainTheory.ContinuousLattice
```

3

converseProjection.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_converseRetr_le
public import DomainTheory.ContinuousLattice.FunctionSpaces.converseRetr_incl
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    converseRetr_preservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section
```

```
namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- The projection assembled from a map `i` satisfying 3.10(i)-(iii). -/
noncomputable def converseProjection
    (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S)) (hinj : Function.Injective i)
    (hwb : ∀ {x y : D}, x ≪ y → i x ≪ i y) (hD : ContinuousLattice D) :
    IsContinuousLatticeProjection D D' where
  incl := ⟨i, continuous_of_preservesDirectedSup (fun _ _ _ => hi _)⟩
  retr := ⟨converseRetr i, continuous_of_preservesDirectedSup
    (converseRetr_preservesDirectedSup hi hwb hD)⟩
  retr_incl := fun d => converseRetr_incl hi hinj d
  incl_retr_le := fun x' => incl_converseRetr_le hi x'

end Proposition310

end DomainTheory.ContinuousLattice
```

4

converseProjection.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_converseRetr_le
public import DomainTheory.ContinuousLattice.FunctionSpaces.converseRetr_incl
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    converseRetr_preservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section
```

```
namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- The projection assembled from a map `i` satisfying 3.10(i)-(iii). -/
noncomputable def conversesProjection
    (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S)) (hinj : Function.Injective i)
    (hwb : ∀ {x y : D}, x ≪ y → i x ≪ i y) (hD : ContinuousLattice D) :
    IsContinuousLatticeProjection D D' where
  incl := ⟨i, continuous_of_preservesDirectedSup (fun _ _ _ => hi _)⟩
  retr := ⟨converseRetr i, continuous_of_preservesDirectedSup
    (converseRetr_preservesDirectedSup hi hwb hD)⟩
  retr_incl := fun d => converseRetr_incl hi hinj d
  incl_retr_le := fun x' => incl_converseRetr_le hi x'

end Proposition310

end DomainTheory.ContinuousLattice
```

5

conversesProjection.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_converseRetr_le
public import DomainTheory.ContinuousLattice.FunctionSpaces.converseRetr_incl
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    converseRetr_preservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice
```

```
open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- The projection assembled from a map `i` satisfying 3.10(i)-(iii). -/
noncomputable def converseProjection
    (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S)) (hinj : Function.Injective i)
    (hwb : ∀ {x y : D}, x ≪ y → i x ≪ i y) (hD : ContinuousLattice D) :
    IsContinuousLatticeProjection D D' where
  incl := ⟨i, continuous_of_preservesDirectedSup (fun _ _ _ => hi _)⟩
  retr := ⟨converseRetr i, continuous_of_preservesDirectedSup
    (converseRetr_preservesDirectedSup hi hwb hD)⟩
  retr_incl := fun d => converseRetr_incl hi hinj d
  incl_retr_le := fun x' => incl_converseRetr_le hi x'

end Proposition310

end DomainTheory.ContinuousLattice
```

6

converseProjection.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_converseRetr_le
public import DomainTheory.ContinuousLattice.FunctionSpaces.converseRetr_incl
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    converseRetr_preservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice
```

```
open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- The projection assembled from a map `i` satisfying 3.10(i)-(iii). -/
noncomputable def converseProjection
    (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S)) (hinj : Function.Injective i)
    (hwb : ∀ {x y : D}, x ≪ y → i x ≪ i y) (hD : ContinuousLattice D) :
    IsContinuousLatticeProjection D D' where
  incl := ⟨i, continuous_of_preservesDirectedSup (fun _ _ _ => hi _)⟩
  retr := ⟨converseRetr i, continuous_of_preservesDirectedSup
    (converseRetr_preservesDirectedSup hi hwb hD)⟩
  retr_incl := fun d => converseRetr_incl hi hinj d
  incl_retr_le := fun x' => incl_converseRetr_le hi x'

end Proposition310

end DomainTheory.ContinuousLattice
```

7

converseProjection.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_converseRetr_le
public import DomainTheory.ContinuousLattice.FunctionSpaces.converseRetr_incl
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    converseRetr_preservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace
```

```
variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- The projection assembled from a map `i` satisfying 3.10(i)-(iii). -/
noncomputable def converseProjection
    (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S)) (hinj : Function.Injective i)
    (hwb : ∀ {x y : D}, x ≪ y → i x ≪ i y) (hD : ContinuousLattice D) :
    IsContinuousLatticeProjection D D' where
  incl := ⟨i, continuous_of_preservesDirectedSup (fun _ _ _ => hi _)⟩
  retr := ⟨converseRetr i, continuous_of_preservesDirectedSup
    (converseRetr_preservesDirectedSup hi hwb hD)⟩
  retr_incl := fun d => converseRetr_incl hi hinj d
  incl_retr_le := fun x' => incl_converseRetr_le hi x'

end Proposition310

end DomainTheory.ContinuousLattice
```

8

converseProjection.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_converseRetr_le
public import DomainTheory.ContinuousLattice.FunctionSpaces.converseRetr_incl
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    converseRetr_preservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace
```

```
variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- The projection assembled from a map `i` satisfying 3.10(i)-(iii). -/
noncomputable def converseProjection
    (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S)) (hinj : Function.Injective i)
    (hwb : ∀ {x y : D}, x ≪ y → i x ≪ i y) (hD : ContinuousLattice D) :
    IsContinuousLatticeProjection D D' where
  incl := ⟨i, continuous_of_preservesDirectedSup (fun _ _ _ => hi _)⟩
  retr := ⟨converseRetr i, continuous_of_preservesDirectedSup
    (converseRetr_preservesDirectedSup hi hwb hD)⟩
  retr_incl := fun d => converseRetr_incl hi hinj d
  incl_retr_le := fun x' => incl_converseRetr_le hi x'

end Proposition310

end DomainTheory.ContinuousLattice
```

9

converseRetr.lean

```
module
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace
```

```
variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- **Scott 1972, Proposition 3.10(iv).** Scott's candidate retraction `j(x') = ⊔ { x
    | i(x) ⊑ x' }`. -/
noncomputable def converseRetr (i : D → D') (x' : D') : D :=
  sSup {x | i x ≤ x'}

end Proposition310

end DomainTheory.ContinuousLattice
```

0

converseRetr.lean

```
module
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
```

```
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- **Scott 1972, Proposition 3.10(iv).** Scott's candidate retraction `j(x') = ⊔ { x
    | i(x) ⊑ x' }`. -/
noncomputable def converseRetr (i : D → D') (x' : D') : D :=
  sSup {x | i x ≤ x'}

end Proposition310

end DomainTheory.ContinuousLattice
```

1

converseRetr.lean

```
module
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset
```

```
variable {i : D → D'}

/-- **Scott 1972, Proposition 3.10(iv).** Scott's candidate retraction `j(x') = ⊔ { x
    | i(x) ⊑ x' }`. -/
noncomputable def converseRetr (i : D → D') (x' : D') : D :=
  sSup {x | i x ≤ x'}

end Proposition310

end DomainTheory.ContinuousLattice
```

2

converseRetr.lean

```
module
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- **Scott 1972, Proposition 3.10(iv).** Scott's candidate retraction `j(x') = ⊔ { x
    | i(x) ⊑ x' }`. -/
noncomputable def converseRetr (i : D → D') (x' : D') : D :=
```

```
  sSup {x | i x ≤ x'}

end Proposition310

end DomainTheory.ContinuousLattice
```

3

converseRetr.lean

```
module
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- **Scott 1972, Proposition 3.10(iv).** Scott's candidate retraction `j(x') = ⊔ { x
    | i(x) ⊑ x' }`. -/
noncomputable def converseRetr (i : D → D') (x' : D') : D :=
  sSup {x | i x ≤ x'}

end Proposition310

end DomainTheory.ContinuousLattice
```

4

converseRetr.lean

```
module
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- **Scott 1972, Proposition 3.10(iv).** Scott's candidate retraction `j(x') = ⊔ { x
    | i(x) ⊑ x' }`. -/
noncomputable def converseRetr (i : D → D') (x' : D') : D :=
  sSup {x | i x ≤ x'}

end Proposition310

end DomainTheory.ContinuousLattice
```

5

converseRetr.lean

```
module
public import Mathlib.Order.BourbakiWitt
```

```
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- **Scott 1972, Proposition 3.10(iv).** Scott's candidate retraction `j(x') = ⊔ { x
    | i(x) ⊑ x' }`. -/
noncomputable def converseRetr (i : D → D') (x' : D') : D :=
  sSup {x | i x ≤ x'}

end Proposition310

end DomainTheory.ContinuousLattice
```

6

converseRetr.lean

```
module
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
```

```
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- **Scott 1972, Proposition 3.10(iv).** Scott's candidate retraction `j(x') = ⊔ { x
    | i(x) ⊑ x' }`. -/
noncomputable def converseRetr (i : D → D') (x' : D') : D :=
  sSup {x | i x ≤ x'}

end Proposition310

end DomainTheory.ContinuousLattice
```

7

converseRetr.lean

```
module
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
```

```
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- **Scott 1972, Proposition 3.10(iv).** Scott's candidate retraction `j(x') = ⊔ { x
    | i(x) ⊑ x' }`. -/
noncomputable def converseRetr (i : D → D') (x' : D') : D :=
  sSup {x | i x ≤ x'}

end Proposition310

end DomainTheory.ContinuousLattice
```

8

converseRetr.lean

```
module
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice
```

```
open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- **Scott 1972, Proposition 3.10(iv).** Scott's candidate retraction `j(x') = ⊔ { x
    | i(x) ⊑ x' }`. -/
noncomputable def converseRetr (i : D → D') (x' : D') : D :=
  sSup {x | i x ≤ x'}

end Proposition310

end DomainTheory.ContinuousLattice
```

9

converseRetr_incl.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.converseRetr
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_of_incl_le
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_mono_of_preservesSSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset
```

```
variable {i : D → D'}

/-- `j ∘ i = id` (uses (i)+(ii)): `{x | i x ⊑ i y} = Iic y`, whose join is `y`. -/
theorem converseRetr_incl (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S))
    (hinj : Function.Injective i) (y : D) : converseRetr i (i y) = y := by
  have hset : {x | i x ≤ i y} = Set.Iic y := by
    ext x
    simp only [Set.mem_ofPred_eq, Set.mem_Iic]
    exact ⟨le_of_incl_le hi hinj, fun hx => incl_mono_of_preservesSSup hi hx⟩
  show sSup {x | i x ≤ i y} = y
  rw [hset]
  exact le_antisymm (sSup_le fun _ hx => hx) (le_sSup le_rfl)

end Proposition310

end DomainTheory.ContinuousLattice
```

0

converseRetr_incl.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.converseRetr
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_of_incl_le
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_mono_of_preservesSSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- `j ∘ i = id` (uses (i)+(ii)): `{x | i x ⊑ i y} = Iic y`, whose join is `y`. -/
theorem converseRetr_incl (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S))
    (hinj : Function.Injective i) (y : D) : converseRetr i (i y) = y := by
  have hset : {x | i x ≤ i y} = Set.Iic y := by
    ext x
```

```
    simp only [Set.mem_ofPred_eq, Set.mem_Iic]
    exact ⟨le_of_incl_le hi hinj, fun hx => incl_mono_of_preservesSSup hi hx⟩
  show sSup {x | i x ≤ i y} = y
  rw [hset]
  exact le_antisymm (sSup_le fun _ hx => hx) (le_sSup le_rfl)

end Proposition310

end DomainTheory.ContinuousLattice
```

1

converseRetr_incl.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.converseRetr
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_of_incl_le
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_mono_of_preservesSSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- `j ∘ i = id` (uses (i)+(ii)): `{x | i x ⊑ i y} = Iic y`, whose join is `y`. -/
theorem converseRetr_incl (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S))
    (hinj : Function.Injective i) (y : D) : converseRetr i (i y) = y := by
  have hset : {x | i x ≤ i y} = Set.Iic y := by
    ext x
    simp only [Set.mem_ofPred_eq, Set.mem_Iic]
    exact ⟨le_of_incl_le hi hinj, fun hx => incl_mono_of_preservesSSup hi hx⟩
  show sSup {x | i x ≤ i y} = y
  rw [hset]
  exact le_antisymm (sSup_le fun _ hx => hx) (le_sSup le_rfl)

end Proposition310
```

```
end DomainTheory.ContinuousLattice
```

2

converseRetr_incl.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.converseRetr
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_of_incl_le
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_mono_of_preservesSSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- `j ∘ i = id` (uses (i)+(ii)): `{x | i x ⊑ i y} = Iic y`, whose join is `y`. -/
theorem converseRetr_incl (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S))
    (hinj : Function.Injective i) (y : D) : converseRetr i (i y) = y := by
  have hset : {x | i x ≤ i y} = Set.Iic y := by
    ext x
    simp only [Set.mem_ofPred_eq, Set.mem_Iic]
    exact ⟨le_of_incl_le hi hinj, fun hx => incl_mono_of_preservesSSup hi hx⟩
  show sSup {x | i x ≤ i y} = y
  rw [hset]
  exact le_antisymm (sSup_le fun _ hx => hx) (le_sSup le_rfl)

end Proposition310

end DomainTheory.ContinuousLattice
```

3

converseRetr_incl.lean

```
module
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaces.converseRetr
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_of_incl_le
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_mono_of_preservesSSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- `j ∘ i = id` (uses (i)+(ii)): `{x | i x ⊑ i y} = Iic y`, whose join is `y`. -/
theorem converseRetr_incl (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S))
    (hinj : Function.Injective i) (y : D) : converseRetr i (i y) = y := by
  have hset : {x | i x ≤ i y} = Set.Iic y := by
    ext x
    simp only [Set.mem_ofPred_eq, Set.mem_Iic]
    exact ⟨le_of_incl_le hi hinj, fun hx => incl_mono_of_preservesSSup hi hx⟩
  show sSup {x | i x ≤ i y} = y
  rw [hset]
  exact le_antisymm (sSup_le fun _ hx => hx) (le_sSup le_rfl)

end Proposition310

end DomainTheory.ContinuousLattice
```

4

converseRetr_incl.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.converseRetr
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_of_incl_le
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_mono_of_preservesSSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section
```

```
namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- `j ∘ i = id` (uses (i)+(ii)): `{x | i x ⊑ i y} = Iic y`, whose join is `y`. -/
theorem converseRetr_incl (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S))
    (hinj : Function.Injective i) (y : D) : converseRetr i (i y) = y := by
  have hset : {x | i x ≤ i y} = Set.Iic y := by
    ext x
    simp only [Set.mem_ofPred_eq, Set.mem_Iic]
    exact ⟨le_of_incl_le hi hinj, fun hx => incl_mono_of_preservesSSup hi hx⟩
  show sSup {x | i x ≤ i y} = y
  rw [hset]
  exact le_antisymm (sSup_le fun _ hx => hx) (le_sSup le_rfl)

end Proposition310

end DomainTheory.ContinuousLattice
```

5

converseRetr_incl.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.converseRetr
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_of_incl_le
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_mono_of_preservesSSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]
```

```
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- `j ∘ i = id` (uses (i)+(ii)): `{x | i x ⊑ i y} = Iic y`, whose join is `y`. -/
theorem converseRetr_incl (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S))
    (hinj : Function.Injective i) (y : D) : converseRetr i (i y) = y := by
  have hset : {x | i x ≤ i y} = Set.Iic y := by
    ext x
    simp only [Set.mem_ofPred_eq, Set.mem_Iic]
    exact ⟨le_of_incl_le hi hinj, fun hx => incl_mono_of_preservesSSup hi hx⟩
  show sSup {x | i x ≤ i y} = y
  rw [hset]
  exact le_antisymm (sSup_le fun _ hx => hx) (le_sSup le_rfl)

end Proposition310

end DomainTheory.ContinuousLattice
```

6

converseRetr_incl.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.converseRetr
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_of_incl_le
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_mono_of_preservesSSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset
```

```
variable {i : D → D'}

/-- `j ∘ i = id` (uses (i)+(ii)): `{x | i x ⊑ i y} = Iic y`, whose join is `y`. -/
theorem converseRetr_incl (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S))
    (hinj : Function.Injective i) (y : D) : converseRetr i (i y) = y := by
  have hset : {x | i x ≤ i y} = Set.Iic y := by
    ext x
    simp only [Set.mem_ofPred_eq, Set.mem_Iic]
    exact ⟨le_of_incl_le hi hinj, fun hx => incl_mono_of_preservesSSup hi hx⟩
  show sSup {x | i x ≤ i y} = y
  rw [hset]
  exact le_antisymm (sSup_le fun _ hx => hx) (le_sSup le_rfl)

end Proposition310

end DomainTheory.ContinuousLattice
```

7

converseRetr_incl.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.converseRetr
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_of_incl_le
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_mono_of_preservesSSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- `j ∘ i = id` (uses (i)+(ii)): `{x | i x ⊑ i y} = Iic y`, whose join is `y`. -/
theorem converseRetr_incl (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S))
    (hinj : Function.Injective i) (y : D) : converseRetr i (i y) = y := by
  have hset : {x | i x ≤ i y} = Set.Iic y := by
    ext x
```

```
    simp only [Set.mem_ofPred_eq, Set.mem_Iic]
    exact ⟨le_of_incl_le hi hinj, fun hx => incl_mono_of_preservesSSup hi hx⟩
  show sSup {x | i x ≤ i y} = y
  rw [hset]
  exact le_antisymm (sSup_le fun _ hx => hx) (le_sSup le_rfl)

end Proposition310

end DomainTheory.ContinuousLattice
```

8

converseRetr_incl.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.converseRetr
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_of_incl_le
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_mono_of_preservesSSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- `j ∘ i = id` (uses (i)+(ii)): `{x | i x ⊑ i y} = Iic y`, whose join is `y`. -/
theorem converseRetr_incl (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S))
    (hinj : Function.Injective i) (y : D) : converseRetr i (i y) = y := by
  have hset : {x | i x ≤ i y} = Set.Iic y := by
    ext x
    simp only [Set.mem_ofPred_eq, Set.mem_Iic]
    exact ⟨le_of_incl_le hi hinj, fun hx => incl_mono_of_preservesSSup hi hx⟩
  show sSup {x | i x ≤ i y} = y
  rw [hset]
  exact le_antisymm (sSup_le fun _ hx => hx) (le_sSup le_rfl)

end Proposition310
```

```
end DomainTheory.ContinuousLattice
```

9

converseRetr_mono.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.converseRetr
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

theorem converseRetr_mono (i : D → D') : Monotone (converseRetr i) := by
  intro a b hab
  exact sSup_le_sSup fun _ hx => le_trans hx hab

end Proposition310

end DomainTheory.ContinuousLattice
```

0

converseRetr_mono.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.converseRetr
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace
```

```
variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

theorem converseRetr_mono (i : D → D') : Monotone (converseRetr i) := by
  intro a b hab
  exact sSup_le_sSup fun _ hx => le_trans hx hab

end Proposition310

end DomainTheory.ContinuousLattice
```

1

converseRetr_mono.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.converseRetr
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

theorem converseRetr_mono (i : D → D') : Monotone (converseRetr i) := by
```

```
  intro a b hab
  exact sSup_le_sSup fun _ hx => le_trans hx hab

end Proposition310

end DomainTheory.ContinuousLattice
```

2

converseRetr_mono.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.converseRetr
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

theorem converseRetr_mono (i : D → D') : Monotone (converseRetr i) := by
  intro a b hab
  exact sSup_le_sSup fun _ hx => le_trans hx hab

end Proposition310

end DomainTheory.ContinuousLattice
```

3

converseRetr_mono.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.converseRetr
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic
```

```
@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

theorem converseRetr_mono (i : D → D') : Monotone (converseRetr i) := by
  intro a b hab
  exact sSup_le_sSup fun _ hx => le_trans hx hab

end Proposition310

end DomainTheory.ContinuousLattice
```

4

converseRetr_mono.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.converseRetr
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310
```

```
open Finset

variable {i : D → D'}

theorem converseRetr_mono (i : D → D') : Monotone (converseRetr i) := by
  intro a b hab
  exact sSup_le_sSup fun _ hx => le_trans hx hab

end Proposition310

end DomainTheory.ContinuousLattice
```

5

converseRetr_mono.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.converseRetr
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

theorem converseRetr_mono (i : D → D') : Monotone (converseRetr i) := by
  intro a b hab
  exact sSup_le_sSup fun _ hx => le_trans hx hab

end Proposition310

end DomainTheory.ContinuousLattice
```

6

converseRetr_mono.lean

```
module
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaces.converseRetr
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

theorem converseRetr_mono (i : D → D') : Monotone (converseRetr i) := by
  intro a b hab
  exact sSup_le_sSup fun _ hx => le_trans hx hab

end Proposition310

end DomainTheory.ContinuousLattice
```

7

converseRetr_mono.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.converseRetr
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
```

```
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

theorem converseRetr_mono (i : D → D') : Monotone (converseRetr i) := by
  intro a b hab
  exact sSup_le_sSup fun _ hx => le_trans hx hab

end Proposition310

end DomainTheory.ContinuousLattice
```

8

converseRetr_mono.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.converseRetr
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

theorem converseRetr_mono (i : D → D') : Monotone (converseRetr i) := by
  intro a b hab
  exact sSup_le_sSup fun _ hx => le_trans hx hab

end Proposition310

end DomainTheory.ContinuousLattice
```

9

converseRetr_preservesDirectedSup.lean

```
module
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.converseRetr
public import DomainTheory.ContinuousLattice.FunctionSpaces.converseRetr_mono
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- `j` is Scott-continuous (uses (i)+(iii) and continuity of `D`). For directed `S'`:
    monotonicity
gives `⊔ j''S' ⊑ j(⊔S')`; conversely if `i x ⊑ ⊔S'` then for every `z ≪ x` we have `i
    z ≪ i x ⊑ ⊔S'`,
so `i z ⊑ x'` for some `x' ∈ S'` (directedness), whence `z ⊑ j(x') ⊑ ⊔ j''S'`;
    continuity of `D`
then gives `x ⊑ ⊔ j''S'`. -/
theorem converseRetr_preservesDirectedSup
    (_hi : ∀ S : Set D, i (sSup S) = sSup (i '' S))
    (hwb : ∀ {x y : D}, x ≪ y → i x ≪ i y) (hD : ContinuousLattice D) :
    PreservesDirectedSup (converseRetr i) := by
  intro S' hS'ne hS'dir
  apply le_antisymm
  · show sSup {x | i x ≤ sSup S'} ≤ sSup (converseRetr i '' S')
    apply sSup_le
    intro x hx
    have hx' : i x ≤ sSup S' := hx
    rw [← hD.sSup_wayBelow x]
    apply sSup_le
    intro z hz
    have hiz : i z ≪ sSup S' := (hwb hz).trans_le hx'
```

```
    obtain ⟨x', hx'S', hzx'⟩ := (WayBelow.sSup_iff hS'ne hS'dir).1 hiz
    have hz_mem : z ∈ {x | i x ≤ x'} := hzx'.le
    exact le_trans (le_sSup hz_mem) (le_sSup ⟨x', hx'S', rfl⟩)
  · apply sSup_le
    rintro _ ⟨x', hx'S', rfl⟩
    exact converseRetr_mono i (le_sSup hx'S')

end Proposition310

end DomainTheory.ContinuousLattice
```

0

converseRetr_preservesDirectedSup.lean

```
module
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.converseRetr
public import DomainTheory.ContinuousLattice.FunctionSpaces.converseRetr_mono
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- `j` is Scott-continuous (uses (i)+(iii) and continuity of `D`). For directed `S'`:
    monotonicity
gives `⊔ j''S' ⊑ j(⊔S')`; conversely if `i x ⊑ ⊔S'` then for every `z ≪ x` we have `i
    z ≪ i x ⊑ ⊔S'`,
so `i z ⊑ x'` for some `x' ∈ S'` (directedness), whence `z ⊑ j(x') ⊑ ⊔ j''S'`;
    continuity of `D`
then gives `x ⊑ ⊔ j''S'`. -/
theorem converseRetr_preservesDirectedSup
    (_hi : ∀ S : Set D, i (sSup S) = sSup (i '' S))
    (hwb : ∀ {x y : D}, x ≪ y → i x ≪ i y) (hD : ContinuousLattice D) :
```

```
    PreservesDirectedSup (converseRetr i) := by
  intro S' hS'ne hS'dir
  apply le_antisymm
  · show sSup {x | i x ≤ sSup S'} ≤ sSup (converseRetr i '' S')
    apply sSup_le
    intro x hx
    have hx' : i x ≤ sSup S' := hx
    rw [← hD.sSup_wayBelow x]
    apply sSup_le
    intro z hz
    have hiz : i z ≪ sSup S' := (hwb hz).trans_le hx'
    obtain ⟨x', hx'S', hzx'⟩ := (WayBelow.sSup_iff hS'ne hS'dir).1 hiz
    have hz_mem : z ∈ {x | i x ≤ x'} := hzx'.le
    exact le_trans (le_sSup hz_mem) (le_sSup ⟨x', hx'S', rfl⟩)
  · apply sSup_le
    rintro _ ⟨x', hx'S', rfl⟩
    exact converseRetr_mono i (le_sSup hx'S')

end Proposition310

end DomainTheory.ContinuousLattice
```

1

converseRetr_preservesDirectedSup.lean

```
module
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.converseRetr
public import DomainTheory.ContinuousLattice.FunctionSpaces.converseRetr_mono
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}
```

```
/-- `j` is Scott-continuous (uses (i)+(iii) and continuity of `D`). For directed `S'`:
    monotonicity
gives `⊔ j''S' ⊑ j(⊔S')`; conversely if `i x ⊑ ⊔S'` then for every `z ≪ x` we have `i
    z ≪ i x ⊑ ⊔S'`,
so `i z ⊑ x'` for some `x' ∈ S'` (directedness), whence `z ⊑ j(x') ⊑ ⊔ j''S'`;
    continuity of `D`
then gives `x ⊑ ⊔ j''S'`. -/
theorem converseRetr_preservesDirectedSup
    (_hi : ∀ S : Set D, i (sSup S) = sSup (i '' S))
    (hwb : ∀ {x y : D}, x ≪ y → i x ≪ i y) (hD : ContinuousLattice D) :
    PreservesDirectedSup (converseRetr i) := by
  intro S' hS'ne hS'dir
  apply le_antisymm
  · show sSup {x | i x ≤ sSup S'} ≤ sSup (converseRetr i '' S')
    apply sSup_le
    intro x hx
    have hx' : i x ≤ sSup S' := hx
    rw [← hD.sSup_wayBelow x]
    apply sSup_le
    intro z hz
    have hiz : i z ≪ sSup S' := (hwb hz).trans_le hx'
    obtain ⟨x', hx'S', hzx'⟩ := (WayBelow.sSup_iff hS'ne hS'dir).1 hiz
    have hz_mem : z ∈ {x | i x ≤ x'} := hzx'.le
    exact le_trans (le_sSup hz_mem) (le_sSup ⟨x', hx'S', rfl⟩)
  · apply sSup_le
    rintro _ ⟨x', hx'S', rfl⟩
    exact converseRetr_mono i (le_sSup hx'S')

end Proposition310

end DomainTheory.ContinuousLattice
```

2

converseRetr_preservesDirectedSup.lean

```
module
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.converseRetr
public import DomainTheory.ContinuousLattice.FunctionSpaces.converseRetr_mono
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]
```

```
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- `j` is Scott-continuous (uses (i)+(iii) and continuity of `D`). For directed `S'`:
    monotonicity
gives `⊔ j''S' ⊑ j(⊔S')`; conversely if `i x ⊑ ⊔S'` then for every `z ≪ x` we have `i
    z ≪ i x ⊑ ⊔S'`,
so `i z ⊑ x'` for some `x' ∈ S'` (directedness), whence `z ⊑ j(x') ⊑ ⊔ j''S'`;
    continuity of `D`
then gives `x ⊑ ⊔ j''S'`. -/
theorem converseRetr_preservesDirectedSup
    (_hi : ∀ S : Set D, i (sSup S) = sSup (i '' S))
    (hwb : ∀ {x y : D}, x ≪ y → i x ≪ i y) (hD : ContinuousLattice D) :
    PreservesDirectedSup (converseRetr i) := by
  intro S' hS'ne hS'dir
  apply le_antisymm
  · show sSup {x | i x ≤ sSup S'} ≤ sSup (converseRetr i '' S')
    apply sSup_le
    intro x hx
    have hx' : i x ≤ sSup S' := hx
    rw [← hD.sSup_wayBelow x]
    apply sSup_le
    intro z hz
    have hiz : i z ≪ sSup S' := (hwb hz).trans_le hx'
    obtain ⟨x', hx'S', hzx'⟩ := (WayBelow.sSup_iff hS'ne hS'dir).1 hiz
    have hz_mem : z ∈ {x | i x ≤ x'} := hzx'.le
    exact le_trans (le_sSup hz_mem) (le_sSup ⟨x', hx'S', rfl⟩)
  · apply sSup_le
    rintro _ ⟨x', hx'S', rfl⟩
    exact converseRetr_mono i (le_sSup hx'S')

end Proposition310

end DomainTheory.ContinuousLattice
```

3

converseRetr_preservesDirectedSup.lean

```
module
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.converseRetr
public import DomainTheory.ContinuousLattice.FunctionSpaces.converseRetr_mono
public import Mathlib.Topology.Defs.Basic
```

```
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- `j` is Scott-continuous (uses (i)+(iii) and continuity of `D`). For directed `S'`:
    monotonicity
gives `⊔ j''S' ⊑ j(⊔S')`; conversely if `i x ⊑ ⊔S'` then for every `z ≪ x` we have `i
    z ≪ i x ⊑ ⊔S'`,
so `i z ⊑ x'` for some `x' ∈ S'` (directedness), whence `z ⊑ j(x') ⊑ ⊔ j''S'`;
    continuity of `D`
then gives `x ⊑ ⊔ j''S'`. -/
theorem converseRetr_preservesDirectedSup
    (_hi : ∀ S : Set D, i (sSup S) = sSup (i '' S))
    (hwb : ∀ {x y : D}, x ≪ y → i x ≪ i y) (hD : ContinuousLattice D) :
    PreservesDirectedSup (converseRetr i) := by
  intro S' hS'ne hS'dir
  apply le_antisymm
  · show sSup {x | i x ≤ sSup S'} ≤ sSup (converseRetr i '' S')
    apply sSup_le
    intro x hx
    have hx' : i x ≤ sSup S' := hx
    rw [← hD.sSup_wayBelow x]
    apply sSup_le
    intro z hz
    have hiz : i z ≪ sSup S' := (hwb hz).trans_le hx'
    obtain ⟨x', hx'S', hzx'⟩ := (WayBelow.sSup_iff hS'ne hS'dir).1 hiz
    have hz_mem : z ∈ {x | i x ≤ x'} := hzx'.le
    exact le_trans (le_sSup hz_mem) (le_sSup ⟨x', hx'S', rfl⟩)
  · apply sSup_le
    rintro _ ⟨x', hx'S', rfl⟩
    exact converseRetr_mono i (le_sSup hx'S')

end Proposition310

end DomainTheory.ContinuousLattice
```

converseRetr_preservesDirectedSup.lean

```
module
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.converseRetr
public import DomainTheory.ContinuousLattice.FunctionSpaces.converseRetr_mono
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- `j` is Scott-continuous (uses (i)+(iii) and continuity of `D`). For directed `S'`:
    monotonicity
gives `⊔ j''S' ⊑ j(⊔S')`; conversely if `i x ⊑ ⊔S'` then for every `z ≪ x` we have `i
    z ≪ i x ⊑ ⊔S'`,
so `i z ⊑ x'` for some `x' ∈ S'` (directedness), whence `z ⊑ j(x') ⊑ ⊔ j''S'`;
    continuity of `D`
then gives `x ⊑ ⊔ j''S'`. -/
theorem converseRetr_preservesDirectedSup
    (_hi : ∀ S : Set D, i (sSup S) = sSup (i '' S))
    (hwb : ∀ {x y : D}, x ≪ y → i x ≪ i y) (hD : ContinuousLattice D) :
    PreservesDirectedSup (converseRetr i) := by
  intro S' hS'ne hS'dir
  apply le_antisymm
  · show sSup {x | i x ≤ sSup S'} ≤ sSup (converseRetr i '' S')
    apply sSup_le
    intro x hx
    have hx' : i x ≤ sSup S' := hx
    rw [← hD.sSup_wayBelow x]
    apply sSup_le
    intro z hz
    have hiz : i z ≪ sSup S' := (hwb hz).trans_le hx'
    obtain ⟨x', hx'S', hzx'⟩ := (WayBelow.sSup_iff hS'ne hS'dir).1 hiz
    have hz_mem : z ∈ {x | i x ≤ x'} := hzx'.le
```

```
    exact le_trans (le_sSup hz_mem) (le_sSup ⟨x', hx'S', rfl⟩)
  · apply sSup_le
    rintro _ ⟨x', hx'S', rfl⟩
    exact converseRetr_mono i (le_sSup hx'S')

end Proposition310

end DomainTheory.ContinuousLattice
```

5

converseRetr_preservesDirectedSup.lean

```
module
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.converseRetr
public import DomainTheory.ContinuousLattice.FunctionSpaces.converseRetr_mono
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- `j` is Scott-continuous (uses (i)+(iii) and continuity of `D`). For directed `S'`:
    monotonicity
gives `⊔ j''S' ⊑ j(⊔S')`; conversely if `i x ⊑ ⊔S'` then for every `z ≪ x` we have `i
    z ≪ i x ⊑ ⊔S'`,
so `i z ⊑ x'` for some `x' ∈ S'` (directedness), whence `z ⊑ j(x') ⊑ ⊔ j''S'`;
    continuity of `D`
then gives `x ⊑ ⊔ j''S'`. -/
theorem converseRetr_preservesDirectedSup
    (_hi : ∀ S : Set D, i (sSup S) = sSup (i '' S))
    (hwb : ∀ {x y : D}, x ≪ y → i x ≪ i y) (hD : ContinuousLattice D) :
    PreservesDirectedSup (converseRetr i) := by
  intro S' hS'ne hS'dir
```

```
  apply le_antisymm
  · show sSup {x | i x ≤ sSup S'} ≤ sSup (converseRetr i '' S')
    apply sSup_le
    intro x hx
    have hx' : i x ≤ sSup S' := hx
    rw [← hD.sSup_wayBelow x]
    apply sSup_le
    intro z hz
    have hiz : i z ≪ sSup S' := (hwb hz).trans_le hx'
    obtain ⟨x', hx'S', hzx'⟩ := (WayBelow.sSup_iff hS'ne hS'dir).1 hiz
    have hz_mem : z ∈ {x | i x ≤ x'} := hzx'.le
    exact le_trans (le_sSup hz_mem) (le_sSup ⟨x', hx'S', rfl⟩)
  · apply sSup_le
    rintro _ ⟨x', hx'S', rfl⟩
    exact converseRetr_mono i (le_sSup hx'S')

end Proposition310

end DomainTheory.ContinuousLattice
```

6

converseRetr_preservesDirectedSup.lean

```
module
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.converseRetr
public import DomainTheory.ContinuousLattice.FunctionSpaces.converseRetr_mono
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- `j` is Scott-continuous (uses (i)+(iii) and continuity of `D`). For directed `S'`:
```

```
    monotonicity
gives `⊔ j''S' ⊑ j(⊔S')`; conversely if `i x ⊑ ⊔S'` then for every `z ≪ x` we have `i
    z ≪ i x ⊑ ⊔S'`,
so `i z ⊑ x'` for some `x' ∈ S'` (directedness), whence `z ⊑ j(x') ⊑ ⊔ j''S'`;
    continuity of `D`
then gives `x ⊑ ⊔ j''S'`. -/
theorem converseRetr_preservesDirectedSup
    (_hi : ∀ S : Set D, i (sSup S) = sSup (i '' S))
    (hwb : ∀ {x y : D}, x ≪ y → i x ≪ i y) (hD : ContinuousLattice D) :
    PreservesDirectedSup (converseRetr i) := by
  intro S' hS'ne hS'dir
  apply le_antisymm
  · show sSup {x | i x ≤ sSup S'} ≤ sSup (converseRetr i '' S')
    apply sSup_le
    intro x hx
    have hx' : i x ≤ sSup S' := hx
    rw [← hD.sSup_wayBelow x]
    apply sSup_le
    intro z hz
    have hiz : i z ≪ sSup S' := (hwb hz).trans_le hx'
    obtain ⟨x', hx'S', hzx'⟩ := (WayBelow.sSup_iff hS'ne hS'dir).1 hiz
    have hz_mem : z ∈ {x | i x ≤ x'} := hzx'.le
    exact le_trans (le_sSup hz_mem) (le_sSup ⟨x', hx'S', rfl⟩)
  · apply sSup_le
    rintro _ ⟨x', hx'S', rfl⟩
    exact converseRetr_mono i (le_sSup hx'S')

end Proposition310

end DomainTheory.ContinuousLattice
```

7

converseRetr_preservesDirectedSup.lean

```
module
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.converseRetr
public import DomainTheory.ContinuousLattice.FunctionSpaces.converseRetr_mono
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
```

```
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- `j` is Scott-continuous (uses (i)+(iii) and continuity of `D`). For directed `S'`:
    monotonicity
gives `⊔ j''S' ⊑ j(⊔S')`; conversely if `i x ⊑ ⊔S'` then for every `z ≪ x` we have `i
    z ≪ i x ⊑ ⊔S'`,
so `i z ⊑ x'` for some `x' ∈ S'` (directedness), whence `z ⊑ j(x') ⊑ ⊔ j''S'`;
    continuity of `D`
then gives `x ⊑ ⊔ j''S'`. -/
theorem converseRetr_preservesDirectedSup
    (_hi : ∀ S : Set D, i (sSup S) = sSup (i '' S))
    (hwb : ∀ {x y : D}, x ≪ y → i x ≪ i y) (hD : ContinuousLattice D) :
    PreservesDirectedSup (converseRetr i) := by
  intro S' hS'ne hS'dir
  apply le_antisymm
  · show sSup {x | i x ≤ sSup S'} ≤ sSup (converseRetr i '' S')
    apply sSup_le
    intro x hx
    have hx' : i x ≤ sSup S' := hx
    rw [← hD.sSup_wayBelow x]
    apply sSup_le
    intro z hz
    have hiz : i z ≪ sSup S' := (hwb hz).trans_le hx'
    obtain ⟨x', hx'S', hzx'⟩ := (WayBelow.sSup_iff hS'ne hS'dir).1 hiz
    have hz_mem : z ∈ {x | i x ≤ x'} := hzx'.le
    exact le_trans (le_sSup hz_mem) (le_sSup ⟨x', hx'S', rfl⟩)
  · apply sSup_le
    rintro _ ⟨x', hx'S', rfl⟩
    exact converseRetr_mono i (le_sSup hx'S')

end Proposition310

end DomainTheory.ContinuousLattice
```

8

converseRetr_preservesDirectedSup.lean

```
module
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.converseRetr
public import DomainTheory.ContinuousLattice.FunctionSpaces.converseRetr_mono
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic
```

```
@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- `j` is Scott-continuous (uses (i)+(iii) and continuity of `D`). For directed `S'`:
    monotonicity
gives `⊔ j''S' ⊑ j(⊔S')`; conversely if `i x ⊑ ⊔S'` then for every `z ≪ x` we have `i
    z ≪ i x ⊑ ⊔S'`,
so `i z ⊑ x'` for some `x' ∈ S'` (directedness), whence `z ⊑ j(x') ⊑ ⊔ j''S'`;
    continuity of `D`
then gives `x ⊑ ⊔ j''S'`. -/
theorem converseRetr_preservesDirectedSup
    (_hi : ∀ S : Set D, i (sSup S) = sSup (i '' S))
    (hwb : ∀ {x y : D}, x ≪ y → i x ≪ i y) (hD : ContinuousLattice D) :
    PreservesDirectedSup (converseRetr i) := by
  intro S' hS'ne hS'dir
  apply le_antisymm
  · show sSup {x | i x ≤ sSup S'} ≤ sSup (converseRetr i '' S')
    apply sSup_le
    intro x hx
    have hx' : i x ≤ sSup S' := hx
    rw [← hD.sSup_wayBelow x]
    apply sSup_le
    intro z hz
    have hiz : i z ≪ sSup S' := (hwb hz).trans_le hx'
    obtain ⟨x', hx'S', hzx'⟩ := (WayBelow.sSup_iff hS'ne hS'dir).1 hiz
    have hz_mem : z ∈ {x | i x ≤ x'} := hzx'.le
    exact le_trans (le_sSup hz_mem) (le_sSup ⟨x', hx'S', rfl⟩)
  · apply sSup_le
    rintro _ ⟨x', hx'S', rfl⟩
    exact converseRetr_mono i (le_sSup hx'S')

end Proposition310

end DomainTheory.ContinuousLattice
```

9

incl_bot.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.retr_bot
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

/-- **Scott 1972, Proposition 3.10(i), empty case.** `i(⊥) = ⊥`. -/
theorem incl_bot (P : IsContinuousLatticeProjection D D') : P.incl (⊥ : D) = ⊥ :=
  le_antisymm (by simpa [retr_bot P] using P.incl_retr_le (⊥ : D')) bot_le

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

0

incl_bot.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.retr_bot
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']
```

```
section Proposition310

open Finset

namespace IsContinuousLatticeProjection

/-- **Scott 1972, Proposition 3.10(i), empty case.** `i(⊥) = ⊥`. -/
theorem incl_bot (P : IsContinuousLatticeProjection D D') : P.incl (⊥ : D) = ⊥ :=
  le_antisymm (by simpa [retr_bot P] using P.incl_retr_le (⊥ : D')) bot_le

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

1

incl_bot.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.retr_bot
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

/-- **Scott 1972, Proposition 3.10(i), empty case.** `i(⊥) = ⊥`. -/
theorem incl_bot (P : IsContinuousLatticeProjection D D') : P.incl (⊥ : D) = ⊥ :=
  le_antisymm (by simpa [retr_bot P] using P.incl_retr_le (⊥ : D')) bot_le

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

2

incl_bot.lean

```
module
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaces.retr_bot
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

/-- **Scott 1972, Proposition 3.10(i), empty case.** `i(⊥) = ⊥`. -/
theorem incl_bot (P : IsContinuousLatticeProjection D D') : P.incl (⊥ : D) = ⊥ :=
  le_antisymm (by simpa [retr_bot P] using P.incl_retr_le (⊥ : D')) bot_le

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

3

incl_bot.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.retr_bot
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310
```

```
open Finset

namespace IsContinuousLatticeProjection

/-- **Scott 1972, Proposition 3.10(i), empty case.** `i(⊥) = ⊥`. -/
theorem incl_bot (P : IsContinuousLatticeProjection D D') : P.incl (⊥ : D) = ⊥ :=
  le_antisymm (by simpa [retr_bot P] using P.incl_retr_le (⊥ : D')) bot_le

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

4

incl_bot.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.retr_bot
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

/-- **Scott 1972, Proposition 3.10(i), empty case.** `i(⊥) = ⊥`. -/
theorem incl_bot (P : IsContinuousLatticeProjection D D') : P.incl (⊥ : D) = ⊥ :=
  le_antisymm (by simpa [retr_bot P] using P.incl_retr_le (⊥ : D')) bot_le

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

5

incl_bot.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.retr_bot
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

/-- **Scott 1972, Proposition 3.10(i), empty case.** `i(⊥) = ⊥`. -/
theorem incl_bot (P : IsContinuousLatticeProjection D D') : P.incl (⊥ : D) = ⊥ :=
  le_antisymm (by simpa [retr_bot P] using P.incl_retr_le (⊥ : D')) bot_le

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

6

incl_bot.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.retr_bot
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310
```

```
open Finset

namespace IsContinuousLatticeProjection

/-- **Scott 1972, Proposition 3.10(i), empty case.** `i(⊥) = ⊥`. -/
theorem incl_bot (P : IsContinuousLatticeProjection D D') : P.incl (⊥ : D) = ⊥ :=
  le_antisymm (by simpa [retr_bot P] using P.incl_retr_le (⊥ : D')) bot_le

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

7

incl_bot.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.retr_bot
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

/-- **Scott 1972, Proposition 3.10(i), empty case.** `i(⊥) = ⊥`. -/
theorem incl_bot (P : IsContinuousLatticeProjection D D') : P.incl (⊥ : D) = ⊥ :=
  le_antisymm (by simpa [retr_bot P] using P.incl_retr_le (⊥ : D')) bot_le

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

8

incl_bot.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.retr_bot
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

/-- **Scott 1972, Proposition 3.10(i), empty case.** `i(⊥) = ⊥`. -/
theorem incl_bot (P : IsContinuousLatticeProjection D D') : P.incl (⊥ : D) = ⊥ :=
  le_antisymm (by simpa [retr_bot P] using P.incl_retr_le (⊥ : D')) bot_le

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

9

incl_converseRetr_le.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.converseRetr
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']
```

```
section Proposition310

open Finset

variable {i : D → D'}

/-- `i ∘ j ⊑ id` (uses only (i)). -/
theorem incl_converseRetr_le (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S)) (x' : D') :
    i (converseRetr i x') ≤ x' := by
  show i (sSup {x | i x ≤ x'}) ≤ x'
  rw [hi]
  exact sSup_le (by rintro _ ⟨x, hx, rfl⟩; exact hx)

end Proposition310

end DomainTheory.ContinuousLattice
```

0

incl_converseRetr_le.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.converseRetr
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- `i ∘ j ⊑ id` (uses only (i)). -/
theorem incl_converseRetr_le (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S)) (x' : D') :
    i (converseRetr i x') ≤ x' := by
  show i (sSup {x | i x ≤ x'}) ≤ x'
  rw [hi]
  exact sSup_le (by rintro _ ⟨x, hx, rfl⟩; exact hx)

end Proposition310
```

```
end DomainTheory.ContinuousLattice
```

1

incl_converseRetr_le.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.converseRetr
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- `i ∘ j ⊑ id` (uses only (i)). -/
theorem incl_converseRetr_le (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S)) (x' : D') :
    i (converseRetr i x') ≤ x' := by
  show i (sSup {x | i x ≤ x'}) ≤ x'
  rw [hi]
  exact sSup_le (by rintro _ ⟨x, hx, rfl⟩; exact hx)

end Proposition310

end DomainTheory.ContinuousLattice
```

2

incl_converseRetr_le.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.converseRetr
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section
```

```
namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- `i ∘ j ⊑ id` (uses only (i)). -/
theorem incl_converseRetr_le (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S)) (x' : D') :
    i (converseRetr i x') ≤ x' := by
  show i (sSup {x | i x ≤ x'}) ≤ x'
  rw [hi]
  exact sSup_le (by rintro _ ⟨x, hx, rfl⟩; exact hx)

end Proposition310

end DomainTheory.ContinuousLattice
```

3

incl_converseRetr_le.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.converseRetr
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']
```

```
section Proposition310

open Finset

variable {i : D → D'}

/-- `i ∘ j ⊑ id` (uses only (i)). -/
theorem incl_converseRetr_le (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S)) (x' : D') :
    i (converseRetr i x') ≤ x' := by
  show i (sSup {x | i x ≤ x'}) ≤ x'
  rw [hi]
  exact sSup_le (by rintro _ ⟨x, hx, rfl⟩; exact hx)

end Proposition310

end DomainTheory.ContinuousLattice
```

4

incl_converseRetr_le.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.converseRetr
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- `i ∘ j ⊑ id` (uses only (i)). -/
theorem incl_converseRetr_le (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S)) (x' : D') :
    i (converseRetr i x') ≤ x' := by
  show i (sSup {x | i x ≤ x'}) ≤ x'
  rw [hi]
  exact sSup_le (by rintro _ ⟨x, hx, rfl⟩; exact hx)

end Proposition310
```

```
end DomainTheory.ContinuousLattice
```

5

incl_converseRetr_le.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.converseRetr
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- `i ∘ j ⊑ id` (uses only (i)). -/
theorem incl_converseRetr_le (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S)) (x' : D') :
    i (converseRetr i x') ≤ x' := by
  show i (sSup {x | i x ≤ x'}) ≤ x'
  rw [hi]
  exact sSup_le (by rintro _ ⟨x, hx, rfl⟩; exact hx)

end Proposition310

end DomainTheory.ContinuousLattice
```

6

incl_converseRetr_le.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.converseRetr
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section
```

```
namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- `i ∘ j ⊑ id` (uses only (i)). -/
theorem incl_converseRetr_le (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S)) (x' : D') :
    i (converseRetr i x') ≤ x' := by
  show i (sSup {x | i x ≤ x'}) ≤ x'
  rw [hi]
  exact sSup_le (by rintro _ ⟨x, hx, rfl⟩; exact hx)

end Proposition310

end DomainTheory.ContinuousLattice
```

7

incl_converseRetr_le.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.converseRetr
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']
```

```
section Proposition310

open Finset

variable {i : D → D'}

/-- `i ∘ j ⊑ id` (uses only (i)). -/
theorem incl_converseRetr_le (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S)) (x' : D') :
    i (converseRetr i x') ≤ x' := by
  show i (sSup {x | i x ≤ x'}) ≤ x'
  rw [hi]
  exact sSup_le (by rintro _ ⟨x, hx, rfl⟩; exact hx)

end Proposition310

end DomainTheory.ContinuousLattice
```

8

incl_converseRetr_le.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.converseRetr
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- `i ∘ j ⊑ id` (uses only (i)). -/
theorem incl_converseRetr_le (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S)) (x' : D') :
    i (converseRetr i x') ≤ x' := by
  show i (sSup {x | i x ≤ x'}) ≤ x'
  rw [hi]
  exact sSup_le (by rintro _ ⟨x, hx, rfl⟩; exact hx)

end Proposition310
```

```
end DomainTheory.ContinuousLattice
```

9

incl_finset_sup.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_sup
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_bot
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

theorem incl_finset_sup (P : IsContinuousLatticeProjection D D') (T : Finset D) :
    P.incl (T.sup (fun x => x)) = T.sup (fun x => P.incl x) := by
  classical
  refine Finset.induction_on T ?base ?step
  · simp only [sup_empty]
    exact incl_bot P
  · intro a s _ ih
    simp only [sup_insert]
    rw [incl_sup P, ih]

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

0

incl_finset_sup.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_sup
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_bot
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
```

```
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

theorem incl_finset_sup (P : IsContinuousLatticeProjection D D') (T : Finset D) :
    P.incl (T.sup (fun x => x)) = T.sup (fun x => P.incl x) := by
  classical
  refine Finset.induction_on T ?base ?step
  · simp only [sup_empty]
    exact incl_bot P
  · intro a s _ ih
    simp only [sup_insert]
    rw [incl_sup P, ih]

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

1

incl_finset_sup.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_sup
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_bot
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
```

```
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

theorem incl_finset_sup (P : IsContinuousLatticeProjection D D') (T : Finset D) :
    P.incl (T.sup (fun x => x)) = T.sup (fun x => P.incl x) := by
  classical
  refine Finset.induction_on T ?base ?step
  · simp only [sup_empty]
    exact incl_bot P
  · intro a s _ ih
    simp only [sup_insert]
    rw [incl_sup P, ih]

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

2

incl_finset_sup.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_sup
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_bot
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

theorem incl_finset_sup (P : IsContinuousLatticeProjection D D') (T : Finset D) :
    P.incl (T.sup (fun x => x)) = T.sup (fun x => P.incl x) := by
  classical
```

```
  refine Finset.induction_on T ?base ?step
  · simp only [sup_empty]
    exact incl_bot P
  · intro a s _ ih
    simp only [sup_insert]
    rw [incl_sup P, ih]

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

3

incl_finset_sup.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_sup
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_bot
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

theorem incl_finset_sup (P : IsContinuousLatticeProjection D D') (T : Finset D) :
    P.incl (T.sup (fun x => x)) = T.sup (fun x => P.incl x) := by
  classical
  refine Finset.induction_on T ?base ?step
  · simp only [sup_empty]
    exact incl_bot P
  · intro a s _ ih
    simp only [sup_insert]
    rw [incl_sup P, ih]

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

4

incl_finset_sup.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_sup
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_bot
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

theorem incl_finset_sup (P : IsContinuousLatticeProjection D D') (T : Finset D) :
    P.incl (T.sup (fun x => x)) = T.sup (fun x => P.incl x) := by
  classical
  refine Finset.induction_on T ?base ?step
  · simp only [sup_empty]
    exact incl_bot P
  · intro a s _ ih
    simp only [sup_insert]
    rw [incl_sup P, ih]

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

5

incl_finset_sup.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_sup
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_bot
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section
```

```
namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

theorem incl_finset_sup (P : IsContinuousLatticeProjection D D') (T : Finset D) :
    P.incl (T.sup (fun x => x)) = T.sup (fun x => P.incl x) := by
  classical
  refine Finset.induction_on T ?base ?step
  · simp only [sup_empty]
    exact incl_bot P
  · intro a s _ ih
    simp only [sup_insert]
    rw [incl_sup P, ih]

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

6

incl_finset_sup.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_sup
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_bot
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310
```

```
open Finset

namespace IsContinuousLatticeProjection

theorem incl_finset_sup (P : IsContinuousLatticeProjection D D') (T : Finset D) :
    P.incl (T.sup (fun x => x)) = T.sup (fun x => P.incl x) := by
  classical
  refine Finset.induction_on T ?base ?step
  · simp only [sup_empty]
    exact incl_bot P
  · intro a s _ ih
    simp only [sup_insert]
    rw [incl_sup P, ih]

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

7

incl_finset_sup.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_sup
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_bot
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

theorem incl_finset_sup (P : IsContinuousLatticeProjection D D') (T : Finset D) :
    P.incl (T.sup (fun x => x)) = T.sup (fun x => P.incl x) := by
  classical
  refine Finset.induction_on T ?base ?step
  · simp only [sup_empty]
    exact incl_bot P
  · intro a s _ ih
    simp only [sup_insert]
```

```
    rw [incl_sup P, ih]

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

8

incl_finset_sup.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_sup
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_bot
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

theorem incl_finset_sup (P : IsContinuousLatticeProjection D D') (T : Finset D) :
    P.incl (T.sup (fun x => x)) = T.sup (fun x => P.incl x) := by
  classical
  refine Finset.induction_on T ?base ?step
  · simp only [sup_empty]
    exact incl_bot P
  · intro a s _ ih
    simp only [sup_insert]
    rw [incl_sup P, ih]

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

9

incl_injective.lean

```
module
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

/-- **Scott 1972, Proposition 3.10(ii).** The inclusion map of a projection is
    injective. -/
theorem incl_injective (P : IsContinuousLatticeProjection D D') {x y : D} (h : P.incl
    x = P.incl y) : x = y := by
  rw [← P.retr_incl x, ← P.retr_incl y, h]

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

0

incl_injective.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310
```

```
open Finset

namespace IsContinuousLatticeProjection

/-- **Scott 1972, Proposition 3.10(ii).** The inclusion map of a projection is
    injective. -/
theorem incl_injective (P : IsContinuousLatticeProjection D D') {x y : D} (h : P.incl
    x = P.incl y) : x = y := by
  rw [← P.retr_incl x, ← P.retr_incl y, h]

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

1

incl_injective.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

/-- **Scott 1972, Proposition 3.10(ii).** The inclusion map of a projection is
    injective. -/
theorem incl_injective (P : IsContinuousLatticeProjection D D') {x y : D} (h : P.incl
    x = P.incl y) : x = y := by
  rw [← P.retr_incl x, ← P.retr_incl y, h]

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

2

incl_injective.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

/-- **Scott 1972, Proposition 3.10(ii).** The inclusion map of a projection is
    injective. -/
theorem incl_injective (P : IsContinuousLatticeProjection D D') {x y : D} (h : P.incl
    x = P.incl y) : x = y := by
  rw [← P.retr_incl x, ← P.retr_incl y, h]

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

3

incl_injective.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']
```

```
section Proposition310

open Finset

namespace IsContinuousLatticeProjection

/-- **Scott 1972, Proposition 3.10(ii).** The inclusion map of a projection is
    injective. -/
theorem incl_injective (P : IsContinuousLatticeProjection D D') {x y : D} (h : P.incl
    x = P.incl y) : x = y := by
  rw [← P.retr_incl x, ← P.retr_incl y, h]

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

4

incl_injective.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

/-- **Scott 1972, Proposition 3.10(ii).** The inclusion map of a projection is
    injective. -/
theorem incl_injective (P : IsContinuousLatticeProjection D D') {x y : D} (h : P.incl
    x = P.incl y) : x = y := by
  rw [← P.retr_incl x, ← P.retr_incl y, h]

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

5

incl_injective.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

/-- **Scott 1972, Proposition 3.10(ii).** The inclusion map of a projection is
    injective. -/
theorem incl_injective (P : IsContinuousLatticeProjection D D') {x y : D} (h : P.incl
    x = P.incl y) : x = y := by
  rw [← P.retr_incl x, ← P.retr_incl y, h]

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

6

incl_injective.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
```

```
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

/-- **Scott 1972, Proposition 3.10(ii).** The inclusion map of a projection is
    injective. -/
theorem incl_injective (P : IsContinuousLatticeProjection D D') {x y : D} (h : P.incl
    x = P.incl y) : x = y := by
  rw [← P.retr_incl x, ← P.retr_incl y, h]

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

7

incl_injective.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

/-- **Scott 1972, Proposition 3.10(ii).** The inclusion map of a projection is
    injective. -/
theorem incl_injective (P : IsContinuousLatticeProjection D D') {x y : D} (h : P.incl
    x = P.incl y) : x = y := by
  rw [← P.retr_incl x, ← P.retr_incl y, h]

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

8

incl_injective.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

/-- **Scott 1972, Proposition 3.10(ii).** The inclusion map of a projection is
    injective. -/
theorem incl_injective (P : IsContinuousLatticeProjection D D') {x y : D} (h : P.incl
    x = P.incl y) : x = y := by
  rw [← P.retr_incl x, ← P.retr_incl y, h]

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

9

incl_mono_of_preservesSSup.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_sup_of_preservesSSup
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
```

```
variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- From (i): `i` is monotone. -/
theorem incl_mono_of_preservesSSup (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S)) :
    Monotone i := by
  intro x y hxy
  have h := incl_sup_of_preservesSSup hi x y
  rw [sup_eq_right.mpr hxy] at h
  rw [h]; exact le_sup_left

end Proposition310

end DomainTheory.ContinuousLattice
```

0

incl_mono_of_preservesSSup.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_sup_of_preservesSSup
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset
```

```
variable {i : D → D'}

/-- From (i): `i` is monotone. -/
theorem incl_mono_of_preservesSSup (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S)) :
    Monotone i := by
  intro x y hxy
  have h := incl_sup_of_preservesSSup hi x y
  rw [sup_eq_right.mpr hxy] at h
  rw [h]; exact le_sup_left

end Proposition310

end DomainTheory.ContinuousLattice
```

1

incl_mono_of_preservesSSup.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_sup_of_preservesSSup
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- From (i): `i` is monotone. -/
theorem incl_mono_of_preservesSSup (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S)) :
    Monotone i := by
  intro x y hxy
  have h := incl_sup_of_preservesSSup hi x y
  rw [sup_eq_right.mpr hxy] at h
  rw [h]; exact le_sup_left

end Proposition310
```

```
end DomainTheory.ContinuousLattice
```

2

incl_mono_of_preservesSSup.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_sup_of_preservesSSup
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- From (i): `i` is monotone. -/
theorem incl_mono_of_preservesSSup (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S)) :
    Monotone i := by
  intro x y hxy
  have h := incl_sup_of_preservesSSup hi x y
  rw [sup_eq_right.mpr hxy] at h
  rw [h]; exact le_sup_left

end Proposition310

end DomainTheory.ContinuousLattice
```

3

incl_mono_of_preservesSSup.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_sup_of_preservesSSup
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic
```

```
@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- From (i): `i` is monotone. -/
theorem incl_mono_of_preservesSSup (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S)) :
    Monotone i := by
  intro x y hxy
  have h := incl_sup_of_preservesSSup hi x y
  rw [sup_eq_right.mpr hxy] at h
  rw [h]; exact le_sup_left

end Proposition310

end DomainTheory.ContinuousLattice
```

4

incl_mono_of_preservesSSup.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_sup_of_preservesSSup
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
```

```
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- From (i): `i` is monotone. -/
theorem incl_mono_of_preservesSSup (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S)) :
    Monotone i := by
  intro x y hxy
  have h := incl_sup_of_preservesSSup hi x y
  rw [sup_eq_right.mpr hxy] at h
  rw [h]; exact le_sup_left

end Proposition310

end DomainTheory.ContinuousLattice
```

5

incl_mono_of_preservesSSup.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_sup_of_preservesSSup
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- From (i): `i` is monotone. -/
theorem incl_mono_of_preservesSSup (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S)) :
```

```
    Monotone i := by
  intro x y hxy
  have h := incl_sup_of_preservesSSup hi x y
  rw [sup_eq_right.mpr hxy] at h
  rw [h]; exact le_sup_left

end Proposition310

end DomainTheory.ContinuousLattice
```

6

incl_mono_of_preservesSSup.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_sup_of_preservesSSup
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- From (i): `i` is monotone. -/
theorem incl_mono_of_preservesSSup (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S)) :
    Monotone i := by
  intro x y hxy
  have h := incl_sup_of_preservesSSup hi x y
  rw [sup_eq_right.mpr hxy] at h
  rw [h]; exact le_sup_left

end Proposition310

end DomainTheory.ContinuousLattice
```

7

incl_mono_of_preservesSSup.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_sup_of_preservesSSup
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- From (i): `i` is monotone. -/
theorem incl_mono_of_preservesSSup (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S)) :
    Monotone i := by
  intro x y hxy
  have h := incl_sup_of_preservesSSup hi x y
  rw [sup_eq_right.mpr hxy] at h
  rw [h]; exact le_sup_left

end Proposition310

end DomainTheory.ContinuousLattice
```


incl_mono_of_preservesSSup.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_sup_of_preservesSSup
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice
```

```
open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- From (i): `i` is monotone. -/
theorem incl_mono_of_preservesSSup (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S)) :
    Monotone i := by
  intro x y hxy
  have h := incl_sup_of_preservesSSup hi x y
  rw [sup_eq_right.mpr hxy] at h
  rw [h]; exact le_sup_left

end Proposition310

end DomainTheory.ContinuousLattice
```

9

incl_sSup.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_finset_sup
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_bot
public import DomainTheory.ContinuousLattice.FunctionSpaces.directedOn_finsetSupOf
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_finsetSupOf
public import DomainTheory.ContinuousLattice.FunctionSpaces.mem_finsetSupOf_of_mem
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']
```

```
section Proposition310

open Finset

namespace IsContinuousLatticeProjection

/-- **Scott 1972, Proposition 3.10(i).** Projections preserve arbitrary suprema. -/
theorem incl_sSup (P : IsContinuousLatticeProjection D D') (S : Set D) :
    P.incl (sSup S) = sSup (Set.image (P.incl : D → D') S) := by
  rcases S.eq_empty_or_nonempty with hS | hS
  · subst hS
    simp only [sSup_empty, Set.image_empty, incl_bot P]
  · rw [sSup_finsetSupOf S]
    have hdir := directedOn_finsetSupOf S
    have hne : (finsetSupOf S).Nonempty := by
      obtain ⟨x, hx⟩ := hS
      exact ⟨x, mem_finsetSupOf_of_mem hx⟩
    rw [P.incl.preservesDirectedSup_coe (finsetSupOf S) hne hdir]
    apply le_antisymm
    · apply sSup_le
      intro b hb
      obtain ⟨z, hz, rfl⟩ := hb
      obtain ⟨T, hT, rfl⟩ := hz
      have h := incl_finset_sup P T
      conv_lhs => rw [show T.sup id = T.sup (fun x => x) from rfl]
      rw [h]
      apply Finset.sup_le
      intro t ht
      exact le_sSup (Set.mem_image_of_mem (P.incl : D → D') (hT t ht))
    · apply sSup_le
      intro b hb
      obtain ⟨x, hxS, rfl⟩ := (Set.mem_image (P.incl : D → D') S b).1 hb
      exact le_sSup (Set.mem_image_of_mem (P.incl : D → D') (mem_finsetSupOf_of_mem
    hxS))

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

0

incl_sSup.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_finset_sup
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_bot
public import DomainTheory.ContinuousLattice.FunctionSpaces.directedOn_finsetSupOf
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_finsetSupOf
public import DomainTheory.ContinuousLattice.FunctionSpaces.mem_finsetSupOf_of_mem
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic

@[expose] public section
```

```
namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

/-- **Scott 1972, Proposition 3.10(i).** Projections preserve arbitrary suprema. -/
theorem incl_sSup (P : IsContinuousLatticeProjection D D') (S : Set D) :
    P.incl (sSup S) = sSup (Set.image (P.incl : D → D') S) := by
  rcases S.eq_empty_or_nonempty with hS | hS
  · subst hS
    simp only [sSup_empty, Set.image_empty, incl_bot P]
  · rw [sSup_finsetSupOf S]
    have hdir := directedOn_finsetSupOf S
    have hne : (finsetSupOf S).Nonempty := by
      obtain ⟨x, hx⟩ := hS
      exact ⟨x, mem_finsetSupOf_of_mem hx⟩
    rw [P.incl.preservesDirectedSup_coe (finsetSupOf S) hne hdir]
    apply le_antisymm
    · apply sSup_le
      intro b hb
      obtain ⟨z, hz, rfl⟩ := hb
      obtain ⟨T, hT, rfl⟩ := hz
      have h := incl_finset_sup P T
      conv_lhs => rw [show T.sup id = T.sup (fun x => x) from rfl]
      rw [h]
      apply Finset.sup_le
      intro t ht
      exact le_sSup (Set.mem_image_of_mem (P.incl : D → D') (hT t ht))
    · apply sSup_le
      intro b hb
      obtain ⟨x, hxS, rfl⟩ := (Set.mem_image (P.incl : D → D') S b).1 hb
      exact le_sSup (Set.mem_image_of_mem (P.incl : D → D') (mem_finsetSupOf_of_mem
    hxS))

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

1

incl_sSup.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_finset_sup
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_bot
public import DomainTheory.ContinuousLattice.FunctionSpaces.directedOn_finsetSupOf
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_finsetSupOf
public import DomainTheory.ContinuousLattice.FunctionSpaces.mem_finsetSupOf_of_mem
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

/-- **Scott 1972, Proposition 3.10(i).** Projections preserve arbitrary suprema. -/
theorem incl_sSup (P : IsContinuousLatticeProjection D D') (S : Set D) :
    P.incl (sSup S) = sSup (Set.image (P.incl : D → D') S) := by
  rcases S.eq_empty_or_nonempty with hS | hS
  · subst hS
    simp only [sSup_empty, Set.image_empty, incl_bot P]
  · rw [sSup_finsetSupOf S]
    have hdir := directedOn_finsetSupOf S
    have hne : (finsetSupOf S).Nonempty := by
      obtain ⟨x, hx⟩ := hS
      exact ⟨x, mem_finsetSupOf_of_mem hx⟩
    rw [P.incl.preservesDirectedSup_coe (finsetSupOf S) hne hdir]
    apply le_antisymm
    · apply sSup_le
      intro b hb
      obtain ⟨z, hz, rfl⟩ := hb
      obtain ⟨T, hT, rfl⟩ := hz
      have h := incl_finset_sup P T
      conv_lhs => rw [show T.sup id = T.sup (fun x => x) from rfl]
      rw [h]
      apply Finset.sup_le
      intro t ht
      exact le_sSup (Set.mem_image_of_mem (P.incl : D → D') (hT t ht))
    · apply sSup_le
      intro b hb
      obtain ⟨x, hxS, rfl⟩ := (Set.mem_image (P.incl : D → D') S b).1 hb
      exact le_sSup (Set.mem_image_of_mem (P.incl : D → D') (mem_finsetSupOf_of_mem
    hxS))
```

```
end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

2

incl_sSup.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_finset_sup
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_bot
public import DomainTheory.ContinuousLattice.FunctionSpaces.directedOn_finsetSupOf
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_finsetSupOf
public import DomainTheory.ContinuousLattice.FunctionSpaces.mem_finsetSupOf_of_mem
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

/-- **Scott 1972, Proposition 3.10(i).** Projections preserve arbitrary suprema. -/
theorem incl_sSup (P : IsContinuousLatticeProjection D D') (S : Set D) :
    P.incl (sSup S) = sSup (Set.image (P.incl : D → D') S) := by
  rcases S.eq_empty_or_nonempty with hS | hS
  · subst hS
    simp only [sSup_empty, Set.image_empty, incl_bot P]
  · rw [sSup_finsetSupOf S]
    have hdir := directedOn_finsetSupOf S
    have hne : (finsetSupOf S).Nonempty := by
      obtain ⟨x, hx⟩ := hS
      exact ⟨x, mem_finsetSupOf_of_mem hx⟩
    rw [P.incl.preservesDirectedSup_coe (finsetSupOf S) hne hdir]
    apply le_antisymm
    · apply sSup_le
      intro b hb
      obtain ⟨z, hz, rfl⟩ := hb
      obtain ⟨T, hT, rfl⟩ := hz
      have h := incl_finset_sup P T
      conv_lhs => rw [show T.sup id = T.sup (fun x => x) from rfl]
```

```
      rw [h]
      apply Finset.sup_le
      intro t ht
      exact le_sSup (Set.mem_image_of_mem (P.incl : D → D') (hT t ht))
    · apply sSup_le
      intro b hb
      obtain ⟨x, hxS, rfl⟩ := (Set.mem_image (P.incl : D → D') S b).1 hb
      exact le_sSup (Set.mem_image_of_mem (P.incl : D → D') (mem_finsetSupOf_of_mem
    hxS))

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

3

incl_sSup.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_finset_sup
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_bot
public import DomainTheory.ContinuousLattice.FunctionSpaces.directedOn_finsetSupOf
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_finsetSupOf
public import DomainTheory.ContinuousLattice.FunctionSpaces.mem_finsetSupOf_of_mem
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

/-- **Scott 1972, Proposition 3.10(i).** Projections preserve arbitrary suprema. -/
theorem incl_sSup (P : IsContinuousLatticeProjection D D') (S : Set D) :
    P.incl (sSup S) = sSup (Set.image (P.incl : D → D') S) := by
  rcases S.eq_empty_or_nonempty with hS | hS
  · subst hS
    simp only [sSup_empty, Set.image_empty, incl_bot P]
  · rw [sSup_finsetSupOf S]
    have hdir := directedOn_finsetSupOf S
    have hne : (finsetSupOf S).Nonempty := by
```

```
      obtain ⟨x, hx⟩ := hS
      exact ⟨x, mem_finsetSupOf_of_mem hx⟩
    rw [P.incl.preservesDirectedSup_coe (finsetSupOf S) hne hdir]
    apply le_antisymm
    · apply sSup_le
      intro b hb
      obtain ⟨z, hz, rfl⟩ := hb
      obtain ⟨T, hT, rfl⟩ := hz
      have h := incl_finset_sup P T
      conv_lhs => rw [show T.sup id = T.sup (fun x => x) from rfl]
      rw [h]
      apply Finset.sup_le
      intro t ht
      exact le_sSup (Set.mem_image_of_mem (P.incl : D → D') (hT t ht))
    · apply sSup_le
      intro b hb
      obtain ⟨x, hxS, rfl⟩ := (Set.mem_image (P.incl : D → D') S b).1 hb
      exact le_sSup (Set.mem_image_of_mem (P.incl : D → D') (mem_finsetSupOf_of_mem
    hxS))

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

4

incl_sSup.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_finset_sup
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_bot
public import DomainTheory.ContinuousLattice.FunctionSpaces.directedOn_finsetSupOf
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_finsetSupOf
public import DomainTheory.ContinuousLattice.FunctionSpaces.mem_finsetSupOf_of_mem
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection
```

```
/-- **Scott 1972, Proposition 3.10(i).** Projections preserve arbitrary suprema. -/
theorem incl_sSup (P : IsContinuousLatticeProjection D D') (S : Set D) :
    P.incl (sSup S) = sSup (Set.image (P.incl : D → D') S) := by
  rcases S.eq_empty_or_nonempty with hS | hS
  · subst hS
    simp only [sSup_empty, Set.image_empty, incl_bot P]
  · rw [sSup_finsetSupOf S]
    have hdir := directedOn_finsetSupOf S
    have hne : (finsetSupOf S).Nonempty := by
      obtain ⟨x, hx⟩ := hS
      exact ⟨x, mem_finsetSupOf_of_mem hx⟩
    rw [P.incl.preservesDirectedSup_coe (finsetSupOf S) hne hdir]
    apply le_antisymm
    · apply sSup_le
      intro b hb
      obtain ⟨z, hz, rfl⟩ := hb
      obtain ⟨T, hT, rfl⟩ := hz
      have h := incl_finset_sup P T
      conv_lhs => rw [show T.sup id = T.sup (fun x => x) from rfl]
      rw [h]
      apply Finset.sup_le
      intro t ht
      exact le_sSup (Set.mem_image_of_mem (P.incl : D → D') (hT t ht))
    · apply sSup_le
      intro b hb
      obtain ⟨x, hxS, rfl⟩ := (Set.mem_image (P.incl : D → D') S b).1 hb
      exact le_sSup (Set.mem_image_of_mem (P.incl : D → D') (mem_finsetSupOf_of_mem
    hxS))

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

5

incl_sSup.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_finset_sup
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_bot
public import DomainTheory.ContinuousLattice.FunctionSpaces.directedOn_finsetSupOf
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_finsetSupOf
public import DomainTheory.ContinuousLattice.FunctionSpaces.mem_finsetSupOf_of_mem
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
```

```
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

/-- **Scott 1972, Proposition 3.10(i).** Projections preserve arbitrary suprema. -/
theorem incl_sSup (P : IsContinuousLatticeProjection D D') (S : Set D) :
    P.incl (sSup S) = sSup (Set.image (P.incl : D → D') S) := by
  rcases S.eq_empty_or_nonempty with hS | hS
  · subst hS
    simp only [sSup_empty, Set.image_empty, incl_bot P]
  · rw [sSup_finsetSupOf S]
    have hdir := directedOn_finsetSupOf S
    have hne : (finsetSupOf S).Nonempty := by
      obtain ⟨x, hx⟩ := hS
      exact ⟨x, mem_finsetSupOf_of_mem hx⟩
    rw [P.incl.preservesDirectedSup_coe (finsetSupOf S) hne hdir]
    apply le_antisymm
    · apply sSup_le
      intro b hb
      obtain ⟨z, hz, rfl⟩ := hb
      obtain ⟨T, hT, rfl⟩ := hz
      have h := incl_finset_sup P T
      conv_lhs => rw [show T.sup id = T.sup (fun x => x) from rfl]
      rw [h]
      apply Finset.sup_le
      intro t ht
      exact le_sSup (Set.mem_image_of_mem (P.incl : D → D') (hT t ht))
    · apply sSup_le
      intro b hb
      obtain ⟨x, hxS, rfl⟩ := (Set.mem_image (P.incl : D → D') S b).1 hb
      exact le_sSup (Set.mem_image_of_mem (P.incl : D → D') (mem_finsetSupOf_of_mem
    hxS))

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

6

incl_sSup.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_finset_sup
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_bot
public import DomainTheory.ContinuousLattice.FunctionSpaces.directedOn_finsetSupOf
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_finsetSupOf
public import DomainTheory.ContinuousLattice.FunctionSpaces.mem_finsetSupOf_of_mem
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.
```

```
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

/-- **Scott 1972, Proposition 3.10(i).** Projections preserve arbitrary suprema. -/
theorem incl_sSup (P : IsContinuousLatticeProjection D D') (S : Set D) :
    P.incl (sSup S) = sSup (Set.image (P.incl : D → D') S) := by
  rcases S.eq_empty_or_nonempty with hS | hS
  · subst hS
    simp only [sSup_empty, Set.image_empty, incl_bot P]
  · rw [sSup_finsetSupOf S]
    have hdir := directedOn_finsetSupOf S
    have hne : (finsetSupOf S).Nonempty := by
      obtain ⟨x, hx⟩ := hS
      exact ⟨x, mem_finsetSupOf_of_mem hx⟩
    rw [P.incl.preservesDirectedSup_coe (finsetSupOf S) hne hdir]
    apply le_antisymm
    · apply sSup_le
      intro b hb
      obtain ⟨z, hz, rfl⟩ := hb
      obtain ⟨T, hT, rfl⟩ := hz
      have h := incl_finset_sup P T
      conv_lhs => rw [show T.sup id = T.sup (fun x => x) from rfl]
      rw [h]
      apply Finset.sup_le
      intro t ht
      exact le_sSup (Set.mem_image_of_mem (P.incl : D → D') (hT t ht))
    · apply sSup_le
      intro b hb
      obtain ⟨x, hxS, rfl⟩ := (Set.mem_image (P.incl : D → D') S b).1 hb
      exact le_sSup (Set.mem_image_of_mem (P.incl : D → D') (mem_finsetSupOf_of_mem
    hxS))

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

incl_sSup.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_finset_sup
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_bot
public import DomainTheory.ContinuousLattice.FunctionSpaces.directedOn_finsetSupOf
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_finsetSupOf
public import DomainTheory.ContinuousLattice.FunctionSpaces.mem_finsetSupOf_of_mem
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

/-- **Scott 1972, Proposition 3.10(i).** Projections preserve arbitrary suprema. -/
theorem incl_sSup (P : IsContinuousLatticeProjection D D') (S : Set D) :
    P.incl (sSup S) = sSup (Set.image (P.incl : D → D') S) := by
  rcases S.eq_empty_or_nonempty with hS | hS
  · subst hS
    simp only [sSup_empty, Set.image_empty, incl_bot P]
  · rw [sSup_finsetSupOf S]
    have hdir := directedOn_finsetSupOf S
    have hne : (finsetSupOf S).Nonempty := by
      obtain ⟨x, hx⟩ := hS
      exact ⟨x, mem_finsetSupOf_of_mem hx⟩
    rw [P.incl.preservesDirectedSup_coe (finsetSupOf S) hne hdir]
    apply le_antisymm
    · apply sSup_le
      intro b hb
      obtain ⟨z, hz, rfl⟩ := hb
      obtain ⟨T, hT, rfl⟩ := hz
      have h := incl_finset_sup P T
      conv_lhs => rw [show T.sup id = T.sup (fun x => x) from rfl]
      rw [h]
      apply Finset.sup_le
      intro t ht
      exact le_sSup (Set.mem_image_of_mem (P.incl : D → D') (hT t ht))
    · apply sSup_le
      intro b hb
```

```
      obtain ⟨x, hxS, rfl⟩ := (Set.mem_image (P.incl : D → D') S b).1 hb
      exact le_sSup (Set.mem_image_of_mem (P.incl : D → D') (mem_finsetSupOf_of_mem
    hxS))

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

8

incl_sSup.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_finset_sup
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_bot
public import DomainTheory.ContinuousLattice.FunctionSpaces.directedOn_finsetSupOf
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_finsetSupOf
public import DomainTheory.ContinuousLattice.FunctionSpaces.mem_finsetSupOf_of_mem
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

/-- **Scott 1972, Proposition 3.10(i).** Projections preserve arbitrary suprema. -/
theorem incl_sSup (P : IsContinuousLatticeProjection D D') (S : Set D) :
    P.incl (sSup S) = sSup (Set.image (P.incl : D → D') S) := by
  rcases S.eq_empty_or_nonempty with hS | hS
  · subst hS
    simp only [sSup_empty, Set.image_empty, incl_bot P]
  · rw [sSup_finsetSupOf S]
    have hdir := directedOn_finsetSupOf S
    have hne : (finsetSupOf S).Nonempty := by
      obtain ⟨x, hx⟩ := hS
      exact ⟨x, mem_finsetSupOf_of_mem hx⟩
    rw [P.incl.preservesDirectedSup_coe (finsetSupOf S) hne hdir]
    apply le_antisymm
    · apply sSup_le
      intro b hb
```

```
      obtain ⟨z, hz, rfl⟩ := hb
      obtain ⟨T, hT, rfl⟩ := hz
      have h := incl_finset_sup P T
      conv_lhs => rw [show T.sup id = T.sup (fun x => x) from rfl]
      rw [h]
      apply Finset.sup_le
      intro t ht
      exact le_sSup (Set.mem_image_of_mem (P.incl : D → D') (hT t ht))
    · apply sSup_le
      intro b hb
      obtain ⟨x, hxS, rfl⟩ := (Set.mem_image (P.incl : D → D') S b).1 hb
      exact le_sSup (Set.mem_image_of_mem (P.incl : D → D') (mem_finsetSupOf_of_mem
    hxS))

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

9

incl_sup.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

/-- **Scott 1972, Proposition 3.10(i), binary case.** `i(x ⊔ y) = i(x) ⊔ i(y)`. -/
theorem incl_sup (P : IsContinuousLatticeProjection D D') (x y : D) :
    P.incl (x ⊔ y) = P.incl x ⊔ P.incl y := by
  apply le_antisymm
  · have hx : x ≤ P.retr (P.incl x ⊔ P.incl y) :=
      le_trans (le_of_eq (P.retr_incl x).symm) (P.retr.monotone le_sup_left)
    have hy : y ≤ P.retr (P.incl x ⊔ P.incl y) :=
      le_trans (le_of_eq (P.retr_incl y).symm) (P.retr.monotone le_sup_right)
    exact le_trans (P.incl.monotone (sup_le hx hy)) (P.incl_retr_le (P.incl x ⊔ P.incl
```

```
    y))
  · exact sup_le (P.incl.monotone le_sup_left) (P.incl.monotone le_sup_right)

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

0

incl_sup.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

/-- **Scott 1972, Proposition 3.10(i), binary case.** `i(x ⊔ y) = i(x) ⊔ i(y)`. -/
theorem incl_sup (P : IsContinuousLatticeProjection D D') (x y : D) :
    P.incl (x ⊔ y) = P.incl x ⊔ P.incl y := by
  apply le_antisymm
  · have hx : x ≤ P.retr (P.incl x ⊔ P.incl y) :=
      le_trans (le_of_eq (P.retr_incl x).symm) (P.retr.monotone le_sup_left)
    have hy : y ≤ P.retr (P.incl x ⊔ P.incl y) :=
      le_trans (le_of_eq (P.retr_incl y).symm) (P.retr.monotone le_sup_right)
    exact le_trans (P.incl.monotone (sup_le hx hy)) (P.incl_retr_le (P.incl x ⊔ P.incl
     y))
  · exact sup_le (P.incl.monotone le_sup_left) (P.incl.monotone le_sup_right)

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

1

incl_sup.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

/-- **Scott 1972, Proposition 3.10(i), binary case.** `i(x ⊔ y) = i(x) ⊔ i(y)`. -/
theorem incl_sup (P : IsContinuousLatticeProjection D D') (x y : D) :
    P.incl (x ⊔ y) = P.incl x ⊔ P.incl y := by
  apply le_antisymm
  · have hx : x ≤ P.retr (P.incl x ⊔ P.incl y) :=
      le_trans (le_of_eq (P.retr_incl x).symm) (P.retr.monotone le_sup_left)
    have hy : y ≤ P.retr (P.incl x ⊔ P.incl y) :=
      le_trans (le_of_eq (P.retr_incl y).symm) (P.retr.monotone le_sup_right)
    exact le_trans (P.incl.monotone (sup_le hx hy)) (P.incl_retr_le (P.incl x ⊔ P.incl
     y))
  · exact sup_le (P.incl.monotone le_sup_left) (P.incl.monotone le_sup_right)

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

2

incl_sup.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice
```

```
open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

/-- **Scott 1972, Proposition 3.10(i), binary case.** `i(x ⊔ y) = i(x) ⊔ i(y)`. -/
theorem incl_sup (P : IsContinuousLatticeProjection D D') (x y : D) :
    P.incl (x ⊔ y) = P.incl x ⊔ P.incl y := by
  apply le_antisymm
  · have hx : x ≤ P.retr (P.incl x ⊔ P.incl y) :=
      le_trans (le_of_eq (P.retr_incl x).symm) (P.retr.monotone le_sup_left)
    have hy : y ≤ P.retr (P.incl x ⊔ P.incl y) :=
      le_trans (le_of_eq (P.retr_incl y).symm) (P.retr.monotone le_sup_right)
    exact le_trans (P.incl.monotone (sup_le hx hy)) (P.incl_retr_le (P.incl x ⊔ P.incl
     y))
  · exact sup_le (P.incl.monotone le_sup_left) (P.incl.monotone le_sup_right)

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

3

incl_sup.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset
```

```
namespace IsContinuousLatticeProjection

/-- **Scott 1972, Proposition 3.10(i), binary case.** `i(x ⊔ y) = i(x) ⊔ i(y)`. -/
theorem incl_sup (P : IsContinuousLatticeProjection D D') (x y : D) :
    P.incl (x ⊔ y) = P.incl x ⊔ P.incl y := by
  apply le_antisymm
  · have hx : x ≤ P.retr (P.incl x ⊔ P.incl y) :=
      le_trans (le_of_eq (P.retr_incl x).symm) (P.retr.monotone le_sup_left)
    have hy : y ≤ P.retr (P.incl x ⊔ P.incl y) :=
      le_trans (le_of_eq (P.retr_incl y).symm) (P.retr.monotone le_sup_right)
    exact le_trans (P.incl.monotone (sup_le hx hy)) (P.incl_retr_le (P.incl x ⊔ P.incl
     y))
  · exact sup_le (P.incl.monotone le_sup_left) (P.incl.monotone le_sup_right)

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

4

incl_sup.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

/-- **Scott 1972, Proposition 3.10(i), binary case.** `i(x ⊔ y) = i(x) ⊔ i(y)`. -/
theorem incl_sup (P : IsContinuousLatticeProjection D D') (x y : D) :
    P.incl (x ⊔ y) = P.incl x ⊔ P.incl y := by
  apply le_antisymm
  · have hx : x ≤ P.retr (P.incl x ⊔ P.incl y) :=
      le_trans (le_of_eq (P.retr_incl x).symm) (P.retr.monotone le_sup_left)
    have hy : y ≤ P.retr (P.incl x ⊔ P.incl y) :=
      le_trans (le_of_eq (P.retr_incl y).symm) (P.retr.monotone le_sup_right)
```

```
    exact le_trans (P.incl.monotone (sup_le hx hy)) (P.incl_retr_le (P.incl x ⊔ P.incl
     y))
  · exact sup_le (P.incl.monotone le_sup_left) (P.incl.monotone le_sup_right)

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

5

incl_sup.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

/-- **Scott 1972, Proposition 3.10(i), binary case.** `i(x ⊔ y) = i(x) ⊔ i(y)`. -/
theorem incl_sup (P : IsContinuousLatticeProjection D D') (x y : D) :
    P.incl (x ⊔ y) = P.incl x ⊔ P.incl y := by
  apply le_antisymm
  · have hx : x ≤ P.retr (P.incl x ⊔ P.incl y) :=
      le_trans (le_of_eq (P.retr_incl x).symm) (P.retr.monotone le_sup_left)
    have hy : y ≤ P.retr (P.incl x ⊔ P.incl y) :=
      le_trans (le_of_eq (P.retr_incl y).symm) (P.retr.monotone le_sup_right)
    exact le_trans (P.incl.monotone (sup_le hx hy)) (P.incl_retr_le (P.incl x ⊔ P.incl
     y))
  · exact sup_le (P.incl.monotone le_sup_left) (P.incl.monotone le_sup_right)

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

6

incl_sup.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

/-- **Scott 1972, Proposition 3.10(i), binary case.** `i(x ⊔ y) = i(x) ⊔ i(y)`. -/
theorem incl_sup (P : IsContinuousLatticeProjection D D') (x y : D) :
    P.incl (x ⊔ y) = P.incl x ⊔ P.incl y := by
  apply le_antisymm
  · have hx : x ≤ P.retr (P.incl x ⊔ P.incl y) :=
      le_trans (le_of_eq (P.retr_incl x).symm) (P.retr.monotone le_sup_left)
    have hy : y ≤ P.retr (P.incl x ⊔ P.incl y) :=
      le_trans (le_of_eq (P.retr_incl y).symm) (P.retr.monotone le_sup_right)
    exact le_trans (P.incl.monotone (sup_le hx hy)) (P.incl_retr_le (P.incl x ⊔ P.incl
     y))
  · exact sup_le (P.incl.monotone le_sup_left) (P.incl.monotone le_sup_right)

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

7

incl_sup.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice
```

```
open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

/-- **Scott 1972, Proposition 3.10(i), binary case.** `i(x ⊔ y) = i(x) ⊔ i(y)`. -/
theorem incl_sup (P : IsContinuousLatticeProjection D D') (x y : D) :
    P.incl (x ⊔ y) = P.incl x ⊔ P.incl y := by
  apply le_antisymm
  · have hx : x ≤ P.retr (P.incl x ⊔ P.incl y) :=
      le_trans (le_of_eq (P.retr_incl x).symm) (P.retr.monotone le_sup_left)
    have hy : y ≤ P.retr (P.incl x ⊔ P.incl y) :=
      le_trans (le_of_eq (P.retr_incl y).symm) (P.retr.monotone le_sup_right)
    exact le_trans (P.incl.monotone (sup_le hx hy)) (P.incl_retr_le (P.incl x ⊔ P.incl
     y))
  · exact sup_le (P.incl.monotone le_sup_left) (P.incl.monotone le_sup_right)

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

8

incl_sup.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset
```

```
namespace IsContinuousLatticeProjection

/-- **Scott 1972, Proposition 3.10(i), binary case.** `i(x ⊔ y) = i(x) ⊔ i(y)`. -/
theorem incl_sup (P : IsContinuousLatticeProjection D D') (x y : D) :
    P.incl (x ⊔ y) = P.incl x ⊔ P.incl y := by
  apply le_antisymm
  · have hx : x ≤ P.retr (P.incl x ⊔ P.incl y) :=
      le_trans (le_of_eq (P.retr_incl x).symm) (P.retr.monotone le_sup_left)
    have hy : y ≤ P.retr (P.incl x ⊔ P.incl y) :=
      le_trans (le_of_eq (P.retr_incl y).symm) (P.retr.monotone le_sup_right)
    exact le_trans (P.incl.monotone (sup_le hx hy)) (P.incl_retr_le (P.incl x ⊔ P.incl
     y))
  · exact sup_le (P.incl.monotone le_sup_left) (P.incl.monotone le_sup_right)

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

9

incl_sup_of_preservesSSup.lean

```
module
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset
```

```
variable {i : D → D'}

/-- From (i): `i` preserves binary joins (Scott checks (i) on two-element sets). -/
theorem incl_sup_of_preservesSSup (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S)) (x y :
    D) :
    i (x ⊔ y) = i x ⊔ i y := by
  calc i (x ⊔ y) = i (sSup {x, y}) := by rw [sSup_pair]
    _ = sSup (i '' {x, y}) := hi _
    _ = sSup {i x, i y} := by rw [Set.image_pair]
    _ = i x ⊔ i y := sSup_pair

end Proposition310

end DomainTheory.ContinuousLattice
```

0

incl_sup_of_preservesSSup.lean

```
module
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}
```

```
/-- From (i): `i` preserves binary joins (Scott checks (i) on two-element sets). -/
theorem incl_sup_of_preservesSSup (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S)) (x y :
    D) :
    i (x ⊔ y) = i x ⊔ i y := by
  calc i (x ⊔ y) = i (sSup {x, y}) := by rw [sSup_pair]
    _ = sSup (i '' {x, y}) := hi _
    _ = sSup {i x, i y} := by rw [Set.image_pair]
    _ = i x ⊔ i y := sSup_pair

end Proposition310

end DomainTheory.ContinuousLattice
```

1

incl_sup_of_preservesSSup.lean

```
module
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- From (i): `i` preserves binary joins (Scott checks (i) on two-element sets). -/
theorem incl_sup_of_preservesSSup (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S)) (x y :
    D) :
```

```
    i (x ⊔ y) = i x ⊔ i y := by
  calc i (x ⊔ y) = i (sSup {x, y}) := by rw [sSup_pair]
    _ = sSup (i '' {x, y}) := hi _
    _ = sSup {i x, i y} := by rw [Set.image_pair]
    _ = i x ⊔ i y := sSup_pair

end Proposition310

end DomainTheory.ContinuousLattice
```

2

incl_sup_of_preservesSSup.lean

```
module
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- From (i): `i` preserves binary joins (Scott checks (i) on two-element sets). -/
theorem incl_sup_of_preservesSSup (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S)) (x y :
     D) :
    i (x ⊔ y) = i x ⊔ i y := by
  calc i (x ⊔ y) = i (sSup {x, y}) := by rw [sSup_pair]
    _ = sSup (i '' {x, y}) := hi _
```

```
    _ = sSup {i x, i y} := by rw [Set.image_pair]
    _ = i x ⊔ i y := sSup_pair

end Proposition310

end DomainTheory.ContinuousLattice
```

3

incl_sup_of_preservesSSup.lean

```
module
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- From (i): `i` preserves binary joins (Scott checks (i) on two-element sets). -/
theorem incl_sup_of_preservesSSup (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S)) (x y :
     D) :
    i (x ⊔ y) = i x ⊔ i y := by
  calc i (x ⊔ y) = i (sSup {x, y}) := by rw [sSup_pair]
    _ = sSup (i '' {x, y}) := hi _
    _ = sSup {i x, i y} := by rw [Set.image_pair]
    _ = i x ⊔ i y := sSup_pair
```

```
end Proposition310

end DomainTheory.ContinuousLattice
```

4

incl_sup_of_preservesSSup.lean

```
module
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- From (i): `i` preserves binary joins (Scott checks (i) on two-element sets). -/
theorem incl_sup_of_preservesSSup (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S)) (x y :
     D) :
    i (x ⊔ y) = i x ⊔ i y := by
  calc i (x ⊔ y) = i (sSup {x, y}) := by rw [sSup_pair]
    _ = sSup (i '' {x, y}) := hi _
    _ = sSup {i x, i y} := by rw [Set.image_pair]
    _ = i x ⊔ i y := sSup_pair

end Proposition310

end DomainTheory.ContinuousLattice
```

5

incl_sup_of_preservesSSup.lean

```
module
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- From (i): `i` preserves binary joins (Scott checks (i) on two-element sets). -/
theorem incl_sup_of_preservesSSup (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S)) (x y :
     D) :
    i (x ⊔ y) = i x ⊔ i y := by
  calc i (x ⊔ y) = i (sSup {x, y}) := by rw [sSup_pair]
    _ = sSup (i '' {x, y}) := hi _
    _ = sSup {i x, i y} := by rw [Set.image_pair]
    _ = i x ⊔ i y := sSup_pair

end Proposition310

end DomainTheory.ContinuousLattice
```

6

incl_sup_of_preservesSSup.lean

```
module
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- From (i): `i` preserves binary joins (Scott checks (i) on two-element sets). -/
theorem incl_sup_of_preservesSSup (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S)) (x y :
     D) :
    i (x ⊔ y) = i x ⊔ i y := by
  calc i (x ⊔ y) = i (sSup {x, y}) := by rw [sSup_pair]
    _ = sSup (i '' {x, y}) := hi _
    _ = sSup {i x, i y} := by rw [Set.image_pair]
    _ = i x ⊔ i y := sSup_pair

end Proposition310

end DomainTheory.ContinuousLattice
```

7

incl_sup_of_preservesSSup.lean

```
module
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Order.CompleteLattice.Basic
```

```
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- From (i): `i` preserves binary joins (Scott checks (i) on two-element sets). -/
theorem incl_sup_of_preservesSSup (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S)) (x y :
     D) :
    i (x ⊔ y) = i x ⊔ i y := by
  calc i (x ⊔ y) = i (sSup {x, y}) := by rw [sSup_pair]
    _ = sSup (i '' {x, y}) := hi _
    _ = sSup {i x, i y} := by rw [Set.image_pair]
    _ = i x ⊔ i y := sSup_pair

end Proposition310

end DomainTheory.ContinuousLattice
```

8

incl_sup_of_preservesSSup.lean

```
module
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
```

```
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- From (i): `i` preserves binary joins (Scott checks (i) on two-element sets). -/
theorem incl_sup_of_preservesSSup (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S)) (x y :
     D) :
    i (x ⊔ y) = i x ⊔ i y := by
  calc i (x ⊔ y) = i (sSup {x, y}) := by rw [sSup_pair]
    _ = sSup (i '' {x, y}) := hi _
    _ = sSup {i x, i y} := by rw [Set.image_pair]
    _ = i x ⊔ i y := sSup_pair

end Proposition310

end DomainTheory.ContinuousLattice
```

9

incl_wayBelow.lean

```
module
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice

public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
```

```
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

/-- **Scott 1972, Proposition 3.10(iii).** Projections preserve the way-below relation
    . -/
theorem incl_wayBelow (P : IsContinuousLatticeProjection D D') (hD' :
    ContinuousLattice D')
    {x y : D} (h : x ≪ y) : @WayBelow D' _ (P.incl x) (P.incl y) := by
  set W : Set D' := { z' | @WayBelow D' _ z' (P.incl y) }
  have hWne : W.Nonempty := ⟨(⊥ : D'), @bot_wayBelow D' _ (P.incl y)⟩
  have hWdir : DirectedOn (· ≤ ·) W := by
    intro p hp q hq
    exact ⟨p ⊔ q, @WayBelow.sup D' _ p q (P.incl y) hp hq, le_sup_left, le_sup_right⟩
  have hy : y = sSup (Set.image (P.retr : D' → D) W) := by
    rw [← P.retr.preservesDirectedSup_coe W hWne hWdir, hD'.sSup_wayBelow (P.incl y),
      P.retr_incl y]
  have hImgne : (Set.image (P.retr : D' → D) W).Nonempty := by
    obtain ⟨z', hz'⟩ := hWne
    exact ⟨P.retr z', z', hz', rfl⟩
  have hImgdir : DirectedOn (· ≤ ·) (Set.image (P.retr : D' → D) W) := by
    intro p hp q hq
    obtain ⟨a, ha, rfl⟩ := hp
    obtain ⟨b, hb, rfl⟩ := hq
    refine ⟨P.retr (a ⊔ b), ⟨a ⊔ b, @WayBelow.sup D' _ a b (P.incl y) ha hb, rfl⟩, ?_,
    ?_⟩
    · exact P.retr.monotone le_sup_left
    · exact P.retr.monotone le_sup_right
  have hx : x ≪ sSup (Set.image (P.retr : D' → D) W) := by rwa [← hy]
  obtain ⟨prz, hprz, hxpr⟩ := (WayBelow.sSup_iff hImgne hImgdir).1 hx
  obtain ⟨z', hz', rfl⟩ := hprz
  exact @WayBelow.le_trans D' _ (P.incl x) z' (P.incl y)
    (le_trans (P.incl.monotone hxpr.le) (P.incl_retr_le z')) hz'

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

0

incl_wayBelow.lean

```
module
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice

public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

/-- **Scott 1972, Proposition 3.10(iii).** Projections preserve the way-below relation
    . -/
theorem incl_wayBelow (P : IsContinuousLatticeProjection D D') (hD' :
    ContinuousLattice D')
    {x y : D} (h : x ≪ y) : @WayBelow D' _ (P.incl x) (P.incl y) := by
  set W : Set D' := { z' | @WayBelow D' _ z' (P.incl y) }
  have hWne : W.Nonempty := ⟨(⊥ : D'), @bot_wayBelow D' _ (P.incl y)⟩
  have hWdir : DirectedOn (· ≤ ·) W := by
    intro p hp q hq
    exact ⟨p ⊔ q, @WayBelow.sup D' _ p q (P.incl y) hp hq, le_sup_left, le_sup_right⟩
  have hy : y = sSup (Set.image (P.retr : D' → D) W) := by
    rw [← P.retr.preservesDirectedSup_coe W hWne hWdir, hD'.sSup_wayBelow (P.incl y),
      P.retr_incl y]
  have hImgne : (Set.image (P.retr : D' → D) W).Nonempty := by
    obtain ⟨z', hz'⟩ := hWne
    exact ⟨P.retr z', z', hz', rfl⟩
  have hImgdir : DirectedOn (· ≤ ·) (Set.image (P.retr : D' → D) W) := by
    intro p hp q hq
    obtain ⟨a, ha, rfl⟩ := hp
    obtain ⟨b, hb, rfl⟩ := hq
    refine ⟨P.retr (a ⊔ b), ⟨a ⊔ b, @WayBelow.sup D' _ a b (P.incl y) ha hb, rfl⟩, ?_,
    ?_⟩
    · exact P.retr.monotone le_sup_left
    · exact P.retr.monotone le_sup_right
  have hx : x ≪ sSup (Set.image (P.retr : D' → D) W) := by rwa [← hy]
```

```
  obtain ⟨prz, hprz, hxpr⟩ := (WayBelow.sSup_iff hImgne hImgdir).1 hx
  obtain ⟨z', hz', rfl⟩ := hprz
  exact @WayBelow.le_trans D' _ (P.incl x) z' (P.incl y)
    (le_trans (P.incl.monotone hxpr.le) (P.incl_retr_le z')) hz'

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

1

incl_wayBelow.lean

```
module
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice

public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

/-- **Scott 1972, Proposition 3.10(iii).** Projections preserve the way-below relation
    . -/
theorem incl_wayBelow (P : IsContinuousLatticeProjection D D') (hD' :
    ContinuousLattice D')
    {x y : D} (h : x ≪ y) : @WayBelow D' _ (P.incl x) (P.incl y) := by
  set W : Set D' := { z' | @WayBelow D' _ z' (P.incl y) }
  have hWne : W.Nonempty := ⟨(⊥ : D'), @bot_wayBelow D' _ (P.incl y)⟩
  have hWdir : DirectedOn (· ≤ ·) W := by
    intro p hp q hq
    exact ⟨p ⊔ q, @WayBelow.sup D' _ p q (P.incl y) hp hq, le_sup_left, le_sup_right⟩
  have hy : y = sSup (Set.image (P.retr : D' → D) W) := by
    rw [← P.retr.preservesDirectedSup_coe W hWne hWdir, hD'.sSup_wayBelow (P.incl y),
      P.retr_incl y]
  have hImgne : (Set.image (P.retr : D' → D) W).Nonempty := by
```

```
    obtain ⟨z', hz'⟩ := hWne
    exact ⟨P.retr z', z', hz', rfl⟩
  have hImgdir : DirectedOn (· ≤ ·) (Set.image (P.retr : D' → D) W) := by
    intro p hp q hq
    obtain ⟨a, ha, rfl⟩ := hp
    obtain ⟨b, hb, rfl⟩ := hq
    refine ⟨P.retr (a ⊔ b), ⟨a ⊔ b, @WayBelow.sup D' _ a b (P.incl y) ha hb, rfl⟩, ?_,
    ?_⟩
    · exact P.retr.monotone le_sup_left
    · exact P.retr.monotone le_sup_right
  have hx : x ≪ sSup (Set.image (P.retr : D' → D) W) := by rwa [← hy]
  obtain ⟨prz, hprz, hxpr⟩ := (WayBelow.sSup_iff hImgne hImgdir).1 hx
  obtain ⟨z', hz', rfl⟩ := hprz
  exact @WayBelow.le_trans D' _ (P.incl x) z' (P.incl y)
    (le_trans (P.incl.monotone hxpr.le) (P.incl_retr_le z')) hz'

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

2

incl_wayBelow.lean

```
module
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice

public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

/-- **Scott 1972, Proposition 3.10(iii).** Projections preserve the way-below relation
    . -/
theorem incl_wayBelow (P : IsContinuousLatticeProjection D D') (hD' :
```

```
    ContinuousLattice D')
    {x y : D} (h : x ≪ y) : @WayBelow D' _ (P.incl x) (P.incl y) := by
  set W : Set D' := { z' | @WayBelow D' _ z' (P.incl y) }
  have hWne : W.Nonempty := ⟨(⊥ : D'), @bot_wayBelow D' _ (P.incl y)⟩
  have hWdir : DirectedOn (· ≤ ·) W := by
    intro p hp q hq
    exact ⟨p ⊔ q, @WayBelow.sup D' _ p q (P.incl y) hp hq, le_sup_left, le_sup_right⟩
  have hy : y = sSup (Set.image (P.retr : D' → D) W) := by
    rw [← P.retr.preservesDirectedSup_coe W hWne hWdir, hD'.sSup_wayBelow (P.incl y),
      P.retr_incl y]
  have hImgne : (Set.image (P.retr : D' → D) W).Nonempty := by
    obtain ⟨z', hz'⟩ := hWne
    exact ⟨P.retr z', z', hz', rfl⟩
  have hImgdir : DirectedOn (· ≤ ·) (Set.image (P.retr : D' → D) W) := by
    intro p hp q hq
    obtain ⟨a, ha, rfl⟩ := hp
    obtain ⟨b, hb, rfl⟩ := hq
    refine ⟨P.retr (a ⊔ b), ⟨a ⊔ b, @WayBelow.sup D' _ a b (P.incl y) ha hb, rfl⟩, ?_,
    ?_⟩
    · exact P.retr.monotone le_sup_left
    · exact P.retr.monotone le_sup_right
  have hx : x ≪ sSup (Set.image (P.retr : D' → D) W) := by rwa [← hy]
  obtain ⟨prz, hprz, hxpr⟩ := (WayBelow.sSup_iff hImgne hImgdir).1 hx
  obtain ⟨z', hz', rfl⟩ := hprz
  exact @WayBelow.le_trans D' _ (P.incl x) z' (P.incl y)
    (le_trans (P.incl.monotone hxpr.le) (P.incl_retr_le z')) hz'

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

3

incl_wayBelow.lean

```
module
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice

public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
```

```
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

/-- **Scott 1972, Proposition 3.10(iii).** Projections preserve the way-below relation
    . -/
theorem incl_wayBelow (P : IsContinuousLatticeProjection D D') (hD' :
    ContinuousLattice D')
    {x y : D} (h : x ≪ y) : @WayBelow D' _ (P.incl x) (P.incl y) := by
  set W : Set D' := { z' | @WayBelow D' _ z' (P.incl y) }
  have hWne : W.Nonempty := ⟨(⊥ : D'), @bot_wayBelow D' _ (P.incl y)⟩
  have hWdir : DirectedOn (· ≤ ·) W := by
    intro p hp q hq
    exact ⟨p ⊔ q, @WayBelow.sup D' _ p q (P.incl y) hp hq, le_sup_left, le_sup_right⟩
  have hy : y = sSup (Set.image (P.retr : D' → D) W) := by
    rw [← P.retr.preservesDirectedSup_coe W hWne hWdir, hD'.sSup_wayBelow (P.incl y),
      P.retr_incl y]
  have hImgne : (Set.image (P.retr : D' → D) W).Nonempty := by
    obtain ⟨z', hz'⟩ := hWne
    exact ⟨P.retr z', z', hz', rfl⟩
  have hImgdir : DirectedOn (· ≤ ·) (Set.image (P.retr : D' → D) W) := by
    intro p hp q hq
    obtain ⟨a, ha, rfl⟩ := hp
    obtain ⟨b, hb, rfl⟩ := hq
    refine ⟨P.retr (a ⊔ b), ⟨a ⊔ b, @WayBelow.sup D' _ a b (P.incl y) ha hb, rfl⟩, ?_,
    ?_⟩
    · exact P.retr.monotone le_sup_left
    · exact P.retr.monotone le_sup_right
  have hx : x ≪ sSup (Set.image (P.retr : D' → D) W) := by rwa [← hy]
  obtain ⟨prz, hprz, hxpr⟩ := (WayBelow.sSup_iff hImgne hImgdir).1 hx
  obtain ⟨z', hz', rfl⟩ := hprz
  exact @WayBelow.le_trans D' _ (P.incl x) z' (P.incl y)
    (le_trans (P.incl.monotone hxpr.le) (P.incl_retr_le z')) hz'

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

4

incl_wayBelow.lean

```
module
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice

public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic
```

```
@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

/-- **Scott 1972, Proposition 3.10(iii).** Projections preserve the way-below relation
    . -/
theorem incl_wayBelow (P : IsContinuousLatticeProjection D D') (hD' :
    ContinuousLattice D')
    {x y : D} (h : x ≪ y) : @WayBelow D' _ (P.incl x) (P.incl y) := by
  set W : Set D' := { z' | @WayBelow D' _ z' (P.incl y) }
  have hWne : W.Nonempty := ⟨(⊥ : D'), @bot_wayBelow D' _ (P.incl y)⟩
  have hWdir : DirectedOn (· ≤ ·) W := by
    intro p hp q hq
    exact ⟨p ⊔ q, @WayBelow.sup D' _ p q (P.incl y) hp hq, le_sup_left, le_sup_right⟩
  have hy : y = sSup (Set.image (P.retr : D' → D) W) := by
    rw [← P.retr.preservesDirectedSup_coe W hWne hWdir, hD'.sSup_wayBelow (P.incl y),
      P.retr_incl y]
  have hImgne : (Set.image (P.retr : D' → D) W).Nonempty := by
    obtain ⟨z', hz'⟩ := hWne
    exact ⟨P.retr z', z', hz', rfl⟩
  have hImgdir : DirectedOn (· ≤ ·) (Set.image (P.retr : D' → D) W) := by
    intro p hp q hq
    obtain ⟨a, ha, rfl⟩ := hp
    obtain ⟨b, hb, rfl⟩ := hq
    refine ⟨P.retr (a ⊔ b), ⟨a ⊔ b, @WayBelow.sup D' _ a b (P.incl y) ha hb, rfl⟩, ?_,
    ?_⟩
    · exact P.retr.monotone le_sup_left
    · exact P.retr.monotone le_sup_right
  have hx : x ≪ sSup (Set.image (P.retr : D' → D) W) := by rwa [← hy]
  obtain ⟨prz, hprz, hxpr⟩ := (WayBelow.sSup_iff hImgne hImgdir).1 hx
  obtain ⟨z', hz', rfl⟩ := hprz
  exact @WayBelow.le_trans D' _ (P.incl x) z' (P.incl y)
    (le_trans (P.incl.monotone hxpr.le) (P.incl_retr_le z')) hz'

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

5

incl_wayBelow.lean

```
module
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice

public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

/-- **Scott 1972, Proposition 3.10(iii).** Projections preserve the way-below relation
    . -/
theorem incl_wayBelow (P : IsContinuousLatticeProjection D D') (hD' :
    ContinuousLattice D')
    {x y : D} (h : x ≪ y) : @WayBelow D' _ (P.incl x) (P.incl y) := by
  set W : Set D' := { z' | @WayBelow D' _ z' (P.incl y) }
  have hWne : W.Nonempty := ⟨(⊥ : D'), @bot_wayBelow D' _ (P.incl y)⟩
  have hWdir : DirectedOn (· ≤ ·) W := by
    intro p hp q hq
    exact ⟨p ⊔ q, @WayBelow.sup D' _ p q (P.incl y) hp hq, le_sup_left, le_sup_right⟩
  have hy : y = sSup (Set.image (P.retr : D' → D) W) := by
    rw [← P.retr.preservesDirectedSup_coe W hWne hWdir, hD'.sSup_wayBelow (P.incl y),
      P.retr_incl y]
  have hImgne : (Set.image (P.retr : D' → D) W).Nonempty := by
    obtain ⟨z', hz'⟩ := hWne
    exact ⟨P.retr z', z', hz', rfl⟩
  have hImgdir : DirectedOn (· ≤ ·) (Set.image (P.retr : D' → D) W) := by
    intro p hp q hq
    obtain ⟨a, ha, rfl⟩ := hp
    obtain ⟨b, hb, rfl⟩ := hq
    refine ⟨P.retr (a ⊔ b), ⟨a ⊔ b, @WayBelow.sup D' _ a b (P.incl y) ha hb, rfl⟩, ?_,
    ?_⟩
    · exact P.retr.monotone le_sup_left
    · exact P.retr.monotone le_sup_right
  have hx : x ≪ sSup (Set.image (P.retr : D' → D) W) := by rwa [← hy]
  obtain ⟨prz, hprz, hxpr⟩ := (WayBelow.sSup_iff hImgne hImgdir).1 hx
  obtain ⟨z', hz', rfl⟩ := hprz
```

```
  exact @WayBelow.le_trans D' _ (P.incl x) z' (P.incl y)
    (le_trans (P.incl.monotone hxpr.le) (P.incl_retr_le z')) hz'

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

6

incl_wayBelow.lean

```
module
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice

public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

/-- **Scott 1972, Proposition 3.10(iii).** Projections preserve the way-below relation
    . -/
theorem incl_wayBelow (P : IsContinuousLatticeProjection D D') (hD' :
    ContinuousLattice D')
    {x y : D} (h : x ≪ y) : @WayBelow D' _ (P.incl x) (P.incl y) := by
  set W : Set D' := { z' | @WayBelow D' _ z' (P.incl y) }
  have hWne : W.Nonempty := ⟨(⊥ : D'), @bot_wayBelow D' _ (P.incl y)⟩
  have hWdir : DirectedOn (· ≤ ·) W := by
    intro p hp q hq
    exact ⟨p ⊔ q, @WayBelow.sup D' _ p q (P.incl y) hp hq, le_sup_left, le_sup_right⟩
  have hy : y = sSup (Set.image (P.retr : D' → D) W) := by
    rw [← P.retr.preservesDirectedSup_coe W hWne hWdir, hD'.sSup_wayBelow (P.incl y),
      P.retr_incl y]
  have hImgne : (Set.image (P.retr : D' → D) W).Nonempty := by
    obtain ⟨z', hz'⟩ := hWne
    exact ⟨P.retr z', z', hz', rfl⟩
```

```
  have hImgdir : DirectedOn (· ≤ ·) (Set.image (P.retr : D' → D) W) := by
    intro p hp q hq
    obtain ⟨a, ha, rfl⟩ := hp
    obtain ⟨b, hb, rfl⟩ := hq
    refine ⟨P.retr (a ⊔ b), ⟨a ⊔ b, @WayBelow.sup D' _ a b (P.incl y) ha hb, rfl⟩, ?_,
    ?_⟩
    · exact P.retr.monotone le_sup_left
    · exact P.retr.monotone le_sup_right
  have hx : x ≪ sSup (Set.image (P.retr : D' → D) W) := by rwa [← hy]
  obtain ⟨prz, hprz, hxpr⟩ := (WayBelow.sSup_iff hImgne hImgdir).1 hx
  obtain ⟨z', hz', rfl⟩ := hprz
  exact @WayBelow.le_trans D' _ (P.incl x) z' (P.incl y)
    (le_trans (P.incl.monotone hxpr.le) (P.incl_retr_le z')) hz'

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

7

incl_wayBelow.lean

```
module
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice

public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

/-- **Scott 1972, Proposition 3.10(iii).** Projections preserve the way-below relation
    . -/
theorem incl_wayBelow (P : IsContinuousLatticeProjection D D') (hD' :
    ContinuousLattice D')
    {x y : D} (h : x ≪ y) : @WayBelow D' _ (P.incl x) (P.incl y) := by
```

```
  set W : Set D' := { z' | @WayBelow D' _ z' (P.incl y) }
  have hWne : W.Nonempty := ⟨(⊥ : D'), @bot_wayBelow D' _ (P.incl y)⟩
  have hWdir : DirectedOn (· ≤ ·) W := by
    intro p hp q hq
    exact ⟨p ⊔ q, @WayBelow.sup D' _ p q (P.incl y) hp hq, le_sup_left, le_sup_right⟩
  have hy : y = sSup (Set.image (P.retr : D' → D) W) := by
    rw [← P.retr.preservesDirectedSup_coe W hWne hWdir, hD'.sSup_wayBelow (P.incl y),
      P.retr_incl y]
  have hImgne : (Set.image (P.retr : D' → D) W).Nonempty := by
    obtain ⟨z', hz'⟩ := hWne
    exact ⟨P.retr z', z', hz', rfl⟩
  have hImgdir : DirectedOn (· ≤ ·) (Set.image (P.retr : D' → D) W) := by
    intro p hp q hq
    obtain ⟨a, ha, rfl⟩ := hp
    obtain ⟨b, hb, rfl⟩ := hq
    refine ⟨P.retr (a ⊔ b), ⟨a ⊔ b, @WayBelow.sup D' _ a b (P.incl y) ha hb, rfl⟩, ?_,
    ?_⟩
    · exact P.retr.monotone le_sup_left
    · exact P.retr.monotone le_sup_right
  have hx : x ≪ sSup (Set.image (P.retr : D' → D) W) := by rwa [← hy]
  obtain ⟨prz, hprz, hxpr⟩ := (WayBelow.sSup_iff hImgne hImgdir).1 hx
  obtain ⟨z', hz', rfl⟩ := hprz
  exact @WayBelow.le_trans D' _ (P.incl x) z' (P.incl y)
    (le_trans (P.incl.monotone hxpr.le) (P.incl_retr_le z')) hz'

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

8

incl_wayBelow.lean

```
module
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice

public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']
```

```
section Proposition310

open Finset

namespace IsContinuousLatticeProjection

/-- **Scott 1972, Proposition 3.10(iii).** Projections preserve the way-below relation
    . -/
theorem incl_wayBelow (P : IsContinuousLatticeProjection D D') (hD' :
    ContinuousLattice D')
    {x y : D} (h : x ≪ y) : @WayBelow D' _ (P.incl x) (P.incl y) := by
  set W : Set D' := { z' | @WayBelow D' _ z' (P.incl y) }
  have hWne : W.Nonempty := ⟨(⊥ : D'), @bot_wayBelow D' _ (P.incl y)⟩
  have hWdir : DirectedOn (· ≤ ·) W := by
    intro p hp q hq
    exact ⟨p ⊔ q, @WayBelow.sup D' _ p q (P.incl y) hp hq, le_sup_left, le_sup_right⟩
  have hy : y = sSup (Set.image (P.retr : D' → D) W) := by
    rw [← P.retr.preservesDirectedSup_coe W hWne hWdir, hD'.sSup_wayBelow (P.incl y),
      P.retr_incl y]
  have hImgne : (Set.image (P.retr : D' → D) W).Nonempty := by
    obtain ⟨z', hz'⟩ := hWne
    exact ⟨P.retr z', z', hz', rfl⟩
  have hImgdir : DirectedOn (· ≤ ·) (Set.image (P.retr : D' → D) W) := by
    intro p hp q hq
    obtain ⟨a, ha, rfl⟩ := hp
    obtain ⟨b, hb, rfl⟩ := hq
    refine ⟨P.retr (a ⊔ b), ⟨a ⊔ b, @WayBelow.sup D' _ a b (P.incl y) ha hb, rfl⟩, ?_,
    ?_⟩
    · exact P.retr.monotone le_sup_left
    · exact P.retr.monotone le_sup_right
  have hx : x ≪ sSup (Set.image (P.retr : D' → D) W) := by rwa [← hy]
  obtain ⟨prz, hprz, hxpr⟩ := (WayBelow.sSup_iff hImgne hImgdir).1 hx
  obtain ⟨z', hz', rfl⟩ := hprz
  exact @WayBelow.le_trans D' _ (P.incl x) z' (P.incl y)
    (le_trans (P.incl.monotone hxpr.le) (P.incl_retr_le z')) hz'

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

9

proposition_3_7_projection.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMapProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace
```

```
variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Proposition 3.7 (projection).** Function spaces inherit projections.
    -/
def proposition_3_7_projection (P₀ : IsContinuousLatticeProjection D₀ D₀')
    (P₁ : IsContinuousLatticeProjection D₁ D₁') :
    ScottMapProjection D₀ D₀' D₁ D₁' :=
  ScottMapProjection.functionSpace P₀ P₁

end DomainTheory.ContinuousLattice
```

0

proposition_3_7_projection.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMapProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Proposition 3.7 (projection).** Function spaces inherit projections.
    -/
def proposition_3_7_projection (P₀ : IsContinuousLatticeProjection D₀ D₀')
    (P₁ : IsContinuousLatticeProjection D₁ D₁') :
    ScottMapProjection D₀ D₀' D₁ D₁' :=
  ScottMapProjection.functionSpace P₀ P₁

end DomainTheory.ContinuousLattice
```

1

proposition_3_7_projection.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMapProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Proposition 3.7 (projection).** Function spaces inherit projections.
    -/
def proposition_3_7_projection (P₀ : IsContinuousLatticeProjection D₀ D₀')
    (P₁ : IsContinuousLatticeProjection D₁ D₁') :
    ScottMapProjection D₀ D₀' D₁ D₁' :=
  ScottMapProjection.functionSpace P₀ P₁

end DomainTheory.ContinuousLattice
```

2

proposition_3_7_projection.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMapProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Proposition 3.7 (projection).** Function spaces inherit projections.
```

```
    -/
def proposition_3_7_projection (P₀ : IsContinuousLatticeProjection D₀ D₀')
    (P₁ : IsContinuousLatticeProjection D₁ D₁') :
    ScottMapProjection D₀ D₀' D₁ D₁' :=
  ScottMapProjection.functionSpace P₀ P₁

end DomainTheory.ContinuousLattice
```

3

proposition_3_7_projection.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMapProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Proposition 3.7 (projection).** Function spaces inherit projections.
    -/
def proposition_3_7_projection (P₀ : IsContinuousLatticeProjection D₀ D₀')
    (P₁ : IsContinuousLatticeProjection D₁ D₁') :
    ScottMapProjection D₀ D₀' D₁ D₁' :=
  ScottMapProjection.functionSpace P₀ P₁

end DomainTheory.ContinuousLattice
```

4

proposition_3_7_projection.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMapProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice
```

```
open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Proposition 3.7 (projection).** Function spaces inherit projections.
     -/
def proposition_3_7_projection (P₀ : IsContinuousLatticeProjection D₀ D₀')
    (P₁ : IsContinuousLatticeProjection D₁ D₁') :
    ScottMapProjection D₀ D₀' D₁ D₁' :=
  ScottMapProjection.functionSpace P₀ P₁

end DomainTheory.ContinuousLattice
```

5

proposition_3_7_projection.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMapProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Proposition 3.7 (projection).** Function spaces inherit projections.
     -/
def proposition_3_7_projection (P₀ : IsContinuousLatticeProjection D₀ D₀')
    (P₁ : IsContinuousLatticeProjection D₁ D₁') :
    ScottMapProjection D₀ D₀' D₁ D₁' :=
  ScottMapProjection.functionSpace P₀ P₁

end DomainTheory.ContinuousLattice
```

6

proposition_3_7_projection.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMapProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Proposition 3.7 (projection).** Function spaces inherit projections.
    -/
def proposition_3_7_projection (P₀ : IsContinuousLatticeProjection D₀ D₀')
    (P₁ : IsContinuousLatticeProjection D₁ D₁') :
    ScottMapProjection D₀ D₀' D₁ D₁' :=
  ScottMapProjection.functionSpace P₀ P₁

end DomainTheory.ContinuousLattice
```


proposition_3_7_projection.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMapProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']
```

```
/-- **Scott 1972, Proposition 3.7 (projection).** Function spaces inherit projections.
    -/
def proposition_3_7_projection (P₀ : IsContinuousLatticeProjection D₀ D₀')
    (P₁ : IsContinuousLatticeProjection D₁ D₁') :
    ScottMapProjection D₀ D₀' D₁ D₁' :=
  ScottMapProjection.functionSpace P₀ P₁

end DomainTheory.ContinuousLattice
```

8

proposition_3_7_projection.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMapProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Proposition 3.7 (projection).** Function spaces inherit projections.
    -/
def proposition_3_7_projection (P₀ : IsContinuousLatticeProjection D₀ D₀')
    (P₁ : IsContinuousLatticeProjection D₁ D₁') :
    ScottMapProjection D₀ D₀' D₁ D₁' :=
  ScottMapProjection.functionSpace P₀ P₁

end DomainTheory.ContinuousLattice
```

9

4.8.3 Factoid 8.3

A universal domain. As a final example of building up domains recursively, we give a construction of a "universal" domain U. (The reason for the name will be explained presently.) The best way to define U seems to be to define a domain V with a top element first, and then to remove the top.

The recursion for V is remarkably simple. We begin with two distinct objects Δ and ∇ that give information about the bottom and top of V, respectively. Thus, $\Delta_V = \Delta$ by definition. We assume that

these two special data objects are "atomic" in the sense that they are not equal to any ordered pair of objects. For the definition of D_V we have these clauses:

(1) $\Delta, \nabla \in D_V$;

(2) $(X, \Delta) \in D_V$ and $(\Delta, Y) \in D_V$ whenever $X, Y \in D_V$.

In other words, we begin with two objects and close up under two flavors of copies of these objects. (A product result is involved here, so that is the reason for structuring the flavors the way we have.)

For V all subsets of D_V are consistent, so all we have left is to define entailment for this domain. The clauses are:

(3) $u \vdash_V \Delta$ always;

(4) $u \vdash_V \nabla$ whenever either $\nabla \in u$ or $\{X \mid (X, \Delta) \in u\} \vdash_V \nabla$ and $\{Y \mid (\Delta, Y) \in u\} \vdash_V \nabla$;

(5) $u \vdash_V (X', \Delta)$ whenever either $\nabla \in u$ or $\{X \mid (X, \Delta) \in u\} \vdash_V X'$; and

(6) $u \vdash_V (\Delta, Y')$ whenever either $\nabla \in u$ or $\{Y \mid (\Delta, Y) \in u\} \vdash_V Y'$.

The proof that V is an information system proceeds as before. Note that, under the above definition of entailment, the data objects Δ, (Δ, Δ), $((\Delta, \Delta), \Delta)$, etc. are all equivalent. There is, however, no other data object equivalent to ∇. The domain equation satisfied by V is:

$$V \cong V \times V.$$

Of course, there are an unlimited number of solutions to this equation, so the fact that V satisfies it tells us very little.

Because ∇ entails everything, we can regard it as a "rogue" object that ought to be banned from polite company: the only element of V it gives any information about is the top element, which is as unhelpful as any element could be. We should simply throw it out as being "inconsistent." What remains is the domain U. Formally we have:

(7) $D_U = D_V - \{\nabla\}$;

(8) $\Delta_U = \Delta_V$;

(9) $\mathrm{Con}_U = \{u \subseteq D_U \mid u \text{ finite and } u \nvdash_V \nabla\}$; and

(10) $u \vdash_U Y$ iff $u \in \mathrm{Con}_U$, $Y \in D_U$ and $u \vdash_V Y$. The same style of definition would work in any situation when an information system has a rogue data object that entails everything: there always is a system that results from eliminating all those objects that entail everything. Indeed, we could have always included such an object in any domain and altered the definition to take as elements those deductively closed sets of data objects that *do* not have the rogue object as a member. We did not do this for the reason that superfluous elements cause lots of exceptions in constructs such as product, where there is a temptation to let them enter into various combinations.

Now in U we *do* allow ∇ to enter into combinations—and this is part of the secret of the construction. The consequence is, however, that the domain equation which U satisfies is not too easy to state since it involves an unfamiliar functor. So it is not through such equations that we will understand its nature in a direct way. But it is possible to explain how it works by reference to the steps in the construction.

Imagine the full (infinite) binary tree. The data objects of U are giving information about possible paths in the tree. We think to the tree starting at the root node at the top of the page and growing down. The object Δ gives no information—so no paths are *excluded*. (If we would have allowed ∇, then the information it would have been giving is that all paths are excluded.) The data object (X, Δ) tells us about a path that *either* it is unrestricted on the right half of the tree, *or* on the left, when we start at the node directly below the root, the paths that are excluded from the subtree are those excluded according to X. This makes sense because the subtrees of the binary tree look exactly like the whole tree, so information can be relativized or translated to other positions. With (Δ, Y) the rôles of right and left are interchanged. We could have introduced data objects of the form (X, Y) which tell us information about both halves of the tree at the same time, but the consistent set $\{(X, \Delta), (\Delta, Y)\}$ does the same job. In general consistent sets should be thought of as *conjunctions*; while, in this example, the comma in the ordered pair should be thought of as a *disjunction* when "reading" information objects.

We can now see that a single data object (if it contains ∇) looks down the tree along a finite path to some depth and then *excludes* the rest of the tree below that node. A consistent set of data objects leaves at least one hole, so at least one path is not excluded. The maximal consistent sets of information objects are those giving true information about one single (infinite) path—the total elements of the domain U correspond exactly to the infinite paths in the binary tree. The partial elements are harder to describe geometrically, however. In accumulating information into a consistent set, holes can be left all over the tree. A partial object is therefore of an indeterminate character, since the "path" we are describing might sneak through any one of the holes. (There is, by the way, a precise topological explanation of what is happening. The total elements of U form a well-known topological space, the so-called Cantor space, and the partially ordered set of elements of U is isomorphic to the lattice of *open* sets of the space—save that the whole space is not allowed.)

This is all very well, but what, we ask, is the good of this domain, and why is it called "universal". The proof cannot be given here, but the result is as follows. As a consequence of standard facts about countable Boolean algebras, it can be proved that every "countably based" domain is a subdomain of U. More specifically, if $\mathbf{A}$ is an information system, and if $D_{\mathbf{A}}$ is countable, then there exists a pair of approximable mappings

$$a : \mathbf{A} \to \mathbf{U} \text{ and } b : \mathbf{U} \to \mathbf{A},$$

such that

$$b \circ a = I_{\mathbf{A}} \text{ and } a \circ b \subseteq I_{\mathbf{U}}.$$

This makes $\mathbf{A}$ a special kind of retract of $\mathbf{U}$. The mappings a and b are far from unique, but at least there is one way to give a one-one embedding of the elements of $\mathbf{A}$ into the elements of $\mathbf{U}$.

The universal property of $\mathbf{U}$ can be applied quite widely. For example, since $(\mathbf{U} \to \mathbf{U})$ is a system with only countably many data objects (by explicit construction!), this system is a retract of $\mathbf{U}$. Fixing on one such retraction pair as above, makes $\mathbf{U}$ also into a model of the λ-calculus. Whether different retractions give essentially different models I do not know. But the point of the remark is to show that domains can contain their own function spaces for a variety of interesting reasons. ?

proposition_3_7_retraction.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMapRetraction
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Proposition 3.7 (retraction).** Function spaces inherit retractions.
    -/
def proposition_3_7_retraction (R₀ : IsContinuousLatticeRetraction D₀ D₀')
    (R₁ : IsContinuousLatticeRetraction D₁ D₁') :
    ScottMapRetraction D₀ D₀' D₁ D₁' :=
  ScottMapRetraction.functionSpace R₀ R₁

end DomainTheory.ContinuousLattice
```

0

proposition_3_7_retraction.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMapRetraction
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']
```

```
/-- **Scott 1972, Proposition 3.7 (retraction).** Function spaces inherit retractions.
    -/
def proposition_3_7_retraction (R₀ : IsContinuousLatticeRetraction D₀ D₀')
    (R₁ : IsContinuousLatticeRetraction D₁ D₁') :
    ScottMapRetraction D₀ D₀' D₁ D₁' :=
  ScottMapRetraction.functionSpace R₀ R₁

end DomainTheory.ContinuousLattice
```

1

proposition_3_7_retraction.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMapRetraction
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Proposition 3.7 (retraction).** Function spaces inherit retractions.
    -/
def proposition_3_7_retraction (R₀ : IsContinuousLatticeRetraction D₀ D₀')
    (R₁ : IsContinuousLatticeRetraction D₁ D₁') :
    ScottMapRetraction D₀ D₀' D₁ D₁' :=
  ScottMapRetraction.functionSpace R₀ R₁

end DomainTheory.ContinuousLattice
```

2

proposition_3_7_retraction.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMapRetraction
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section
```

```
namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Proposition 3.7 (retraction).** Function spaces inherit retractions.
    -/
def proposition_3_7_retraction (R₀ : IsContinuousLatticeRetraction D₀ D₀')
    (R₁ : IsContinuousLatticeRetraction D₁ D₁') :
    ScottMapRetraction D₀ D₀' D₁ D₁' :=
  ScottMapRetraction.functionSpace R₀ R₁

end DomainTheory.ContinuousLattice
```

3

proposition_3_7_retraction.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMapRetraction
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Proposition 3.7 (retraction).** Function spaces inherit retractions.
    -/
def proposition_3_7_retraction (R₀ : IsContinuousLatticeRetraction D₀ D₀')
    (R₁ : IsContinuousLatticeRetraction D₁ D₁') :
    ScottMapRetraction D₀ D₀' D₁ D₁' :=
  ScottMapRetraction.functionSpace R₀ R₁

end DomainTheory.ContinuousLattice
```

4

proposition_3_7_retraction.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMapRetraction
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Proposition 3.7 (retraction).** Function spaces inherit retractions.
    -/
def proposition_3_7_retraction (R₀ : IsContinuousLatticeRetraction D₀ D₀')
    (R₁ : IsContinuousLatticeRetraction D₁ D₁') :
    ScottMapRetraction D₀ D₀' D₁ D₁' :=
  ScottMapRetraction.functionSpace R₀ R₁

end DomainTheory.ContinuousLattice
```

5

proposition_3_7_retraction.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMapRetraction
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
```

```
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Proposition 3.7 (retraction).** Function spaces inherit retractions.
    -/
def proposition_3_7_retraction (R₀ : IsContinuousLatticeRetraction D₀ D₀')
    (R₁ : IsContinuousLatticeRetraction D₁ D₁') :
    ScottMapRetraction D₀ D₀' D₁ D₁' :=
  ScottMapRetraction.functionSpace R₀ R₁

end DomainTheory.ContinuousLattice
```

6

proposition_3_7_retraction.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMapRetraction
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Proposition 3.7 (retraction).** Function spaces inherit retractions.
    -/
def proposition_3_7_retraction (R₀ : IsContinuousLatticeRetraction D₀ D₀')
    (R₁ : IsContinuousLatticeRetraction D₁ D₁') :
    ScottMapRetraction D₀ D₀' D₁ D₁' :=
  ScottMapRetraction.functionSpace R₀ R₁

end DomainTheory.ContinuousLattice
```

7

proposition_3_7_retraction.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMapRetraction
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic
```

```
@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Proposition 3.7 (retraction).** Function spaces inherit retractions.
    -/
def proposition_3_7_retraction (R₀ : IsContinuousLatticeRetraction D₀ D₀')
    (R₁ : IsContinuousLatticeRetraction D₁ D₁') :
    ScottMapRetraction D₀ D₀' D₁ D₁' :=
  ScottMapRetraction.functionSpace R₀ R₁

end DomainTheory.ContinuousLattice
```

8

proposition_3_7_retraction.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMapRetraction
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Proposition 3.7 (retraction).** Function spaces inherit retractions.
    -/
def proposition_3_7_retraction (R₀ : IsContinuousLatticeRetraction D₀ D₀')
    (R₁ : IsContinuousLatticeRetraction D₁ D₁') :
    ScottMapRetraction D₀ D₀' D₁ D₁' :=
  ScottMapRetraction.functionSpace R₀ R₁
```

```
end DomainTheory.ContinuousLattice
```

9

retr_ambientSSup_eq_sSup.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.ambientSSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace IsContinuousLatticeRetraction

/-- **Scott 1972, Prop 2.10 / March 1972 correction (p. 135).** For directed `S ⊆ D`,
`j(⊔S') = ⊔S`: the retraction sends the ambient join back to the subspace join. -/
theorem retr_ambientSSup_eq_sSup (R : IsContinuousLatticeRetraction D D') (S : Set D)
    (hS : S.Nonempty) (hdir : DirectedOn (· ≤ ·) S) :
    R.retr (ambientSSup R S) = sSup S := by
  unfold ambientSSup
  have hdir' : DirectedOn (· ≤ ·) (Set.image (R.incl : D → D') S) := by
    intro p hp q hq
    obtain ⟨a, ha, rfl⟩ := hp
    obtain ⟨b, hb, rfl⟩ := hq
    obtain ⟨c, hc, hac, hbc⟩ := hdir a ha b hb
    exact ⟨R.incl c, Set.mem_image_of_mem _ hc, R.incl.monotone hac, R.incl.monotone
    hbc⟩
  have hne' : (Set.image (R.incl : D → D') S).Nonempty := Set.image_nonempty.2 hS
  rw [R.retr.preservesDirectedSup_coe (Set.image (R.incl : D → D') S) hne' hdir']
  rw [Set.image_image, Set.image_congr fun a _ => R.retr_incl a]
  simp

end IsContinuousLatticeRetraction
end DomainTheory.ContinuousLattice
```

0

retr_ambientSSup_eq_sSup.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.ambientSSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace IsContinuousLatticeRetraction

/-- **Scott 1972, Prop 2.10 / March 1972 correction (p. 135).** For directed `S ⊆ D`,
`j(⊔S') = ⊔S`: the retraction sends the ambient join back to the subspace join. -/
theorem retr_ambientSSup_eq_sSup (R : IsContinuousLatticeRetraction D D') (S : Set D)
    (hS : S.Nonempty) (hdir : DirectedOn (· ≤ ·) S) :
    R.retr (ambientSSup R S) = sSup S := by
  unfold ambientSSup
  have hdir' : DirectedOn (· ≤ ·) (Set.image (R.incl : D → D') S) := by
    intro p hp q hq
    obtain ⟨a, ha, rfl⟩ := hp
    obtain ⟨b, hb, rfl⟩ := hq
    obtain ⟨c, hc, hac, hbc⟩ := hdir a ha b hb
    exact ⟨R.incl c, Set.mem_image_of_mem _ hc, R.incl.monotone hac, R.incl.monotone
    hbc⟩
  have hne' : (Set.image (R.incl : D → D') S).Nonempty := Set.image_nonempty.2 hS
  rw [R.retr.preservesDirectedSup_coe (Set.image (R.incl : D → D') S) hne' hdir']
  rw [Set.image_image, Set.image_congr fun a _ => R.retr_incl a]
  simp

end IsContinuousLatticeRetraction
end DomainTheory.ContinuousLattice
```

1

retr_ambientSSup_eq_sSup.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.ambientSSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice
```

```
open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace IsContinuousLatticeRetraction

/-- **Scott 1972, Prop 2.10 / March 1972 correction (p. 135).** For directed `S ⊆ D`,
`j(⊔S') = ⊔S`: the retraction sends the ambient join back to the subspace join. -/
theorem retr_ambientSSup_eq_sSup (R : IsContinuousLatticeRetraction D D') (S : Set D)
    (hS : S.Nonempty) (hdir : DirectedOn (· ≤ ·) S) :
    R.retr (ambientSSup R S) = sSup S := by
  unfold ambientSSup
  have hdir' : DirectedOn (· ≤ ·) (Set.image (R.incl : D → D') S) := by
    intro p hp q hq
    obtain ⟨a, ha, rfl⟩ := hp
    obtain ⟨b, hb, rfl⟩ := hq
    obtain ⟨c, hc, hac, hbc⟩ := hdir a ha b hb
    exact ⟨R.incl c, Set.mem_image_of_mem _ hc, R.incl.monotone hac, R.incl.monotone
    hbc⟩
  have hne' : (Set.image (R.incl : D → D') S).Nonempty := Set.image_nonempty.2 hS
  rw [R.retr.preservesDirectedSup_coe (Set.image (R.incl : D → D') S) hne' hdir']
  rw [Set.image_image, Set.image_congr fun a _ => R.retr_incl a]
  simp

end IsContinuousLatticeRetraction
end DomainTheory.ContinuousLattice
```

2

retr_ambientSSup_eq_sSup.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.ambientSSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace IsContinuousLatticeRetraction
```

```
/-- **Scott 1972, Prop 2.10 / March 1972 correction (p. 135).** For directed `S ⊆ D`,
`j(⨆S') = ⨆S`: the retraction sends the ambient join back to the subspace join. -/
theorem retr_ambientSSup_eq_sSup (R : IsContinuousLatticeRetraction D D') (S : Set D)
    (hS : S.Nonempty) (hdir : DirectedOn (· ≤ ·) S) :
    R.retr (ambientSSup R S) = sSup S := by
  unfold ambientSSup
  have hdir' : DirectedOn (· ≤ ·) (Set.image (R.incl : D → D') S) := by
    intro p hp q hq
    obtain ⟨a, ha, rfl⟩ := hp
    obtain ⟨b, hb, rfl⟩ := hq
    obtain ⟨c, hc, hac, hbc⟩ := hdir a ha b hb
    exact ⟨R.incl c, Set.mem_image_of_mem _ hc, R.incl.monotone hac, R.incl.monotone
    hbc⟩
  have hne' : (Set.image (R.incl : D → D') S).Nonempty := Set.image_nonempty.2 hS
  rw [R.retr.preservesDirectedSup_coe (Set.image (R.incl : D → D') S) hne' hdir']
  rw [Set.image_image, Set.image_congr fun a _ => R.retr_incl a]
  simp

end IsContinuousLatticeRetraction
end DomainTheory.ContinuousLattice
```

3

retr_ambientSSup_eq_sSup.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.ambientSSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace IsContinuousLatticeRetraction

/-- **Scott 1972, Prop 2.10 / March 1972 correction (p. 135).** For directed `S ⊆ D`,
`j(⨆S') = ⨆S`: the retraction sends the ambient join back to the subspace join. -/
theorem retr_ambientSSup_eq_sSup (R : IsContinuousLatticeRetraction D D') (S : Set D)
    (hS : S.Nonempty) (hdir : DirectedOn (· ≤ ·) S) :
    R.retr (ambientSSup R S) = sSup S := by
  unfold ambientSSup
  have hdir' : DirectedOn (· ≤ ·) (Set.image (R.incl : D → D') S) := by
    intro p hp q hq
```

```
    obtain ⟨a, ha, rfl⟩ := hp
    obtain ⟨b, hb, rfl⟩ := hq
    obtain ⟨c, hc, hac, hbc⟩ := hdir a ha b hb
    exact ⟨R.incl c, Set.mem_image_of_mem _ hc, R.incl.monotone hac, R.incl.monotone
    hbc⟩
  have hne' : (Set.image (R.incl : D → D') S).Nonempty := Set.image_nonempty.2 hS
  rw [R.retr.preservesDirectedSup_coe (Set.image (R.incl : D → D') S) hne' hdir']
  rw [Set.image_image, Set.image_congr fun a _ => R.retr_incl a]
  simp

end IsContinuousLatticeRetraction
end DomainTheory.ContinuousLattice
```

4

retr_ambientSSup_eq_sSup.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.ambientSSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace IsContinuousLatticeRetraction

/-- **Scott 1972, Prop 2.10 / March 1972 correction (p. 135).** For directed `S ⊆ D`,
`j(⊔S') = ⊔S`: the retraction sends the ambient join back to the subspace join. -/
theorem retr_ambientSSup_eq_sSup (R : IsContinuousLatticeRetraction D D') (S : Set D)
    (hS : S.Nonempty) (hdir : DirectedOn (· ≤ ·) S) :
    R.retr (ambientSSup R S) = sSup S := by
  unfold ambientSSup
  have hdir' : DirectedOn (· ≤ ·) (Set.image (R.incl : D → D') S) := by
    intro p hp q hq
    obtain ⟨a, ha, rfl⟩ := hp
    obtain ⟨b, hb, rfl⟩ := hq
    obtain ⟨c, hc, hac, hbc⟩ := hdir a ha b hb
    exact ⟨R.incl c, Set.mem_image_of_mem _ hc, R.incl.monotone hac, R.incl.monotone
    hbc⟩
  have hne' : (Set.image (R.incl : D → D') S).Nonempty := Set.image_nonempty.2 hS
  rw [R.retr.preservesDirectedSup_coe (Set.image (R.incl : D → D') S) hne' hdir']
  rw [Set.image_image, Set.image_congr fun a _ => R.retr_incl a]
  simp
```

```
end IsContinuousLatticeRetraction
end DomainTheory.ContinuousLattice
```

5

retr_ambientSSup_eq_sSup.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.ambientSSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace IsContinuousLatticeRetraction

/-- **Scott 1972, Prop 2.10 / March 1972 correction (p. 135).** For directed `S ⊆ D`,
`j(⊔S') = ⊔S`: the retraction sends the ambient join back to the subspace join. -/
theorem retr_ambientSSup_eq_sSup (R : IsContinuousLatticeRetraction D D') (S : Set D)
    (hS : S.Nonempty) (hdir : DirectedOn (· ≤ ·) S) :
    R.retr (ambientSSup R S) = sSup S := by
  unfold ambientSSup
  have hdir' : DirectedOn (· ≤ ·) (Set.image (R.incl : D → D') S) := by
    intro p hp q hq
    obtain ⟨a, ha, rfl⟩ := hp
    obtain ⟨b, hb, rfl⟩ := hq
    obtain ⟨c, hc, hac, hbc⟩ := hdir a ha b hb
    exact ⟨R.incl c, Set.mem_image_of_mem _ hc, R.incl.monotone hac, R.incl.monotone
    hbc⟩
  have hne' : (Set.image (R.incl : D → D') S).Nonempty := Set.image_nonempty.2 hS
  rw [R.retr.preservesDirectedSup_coe (Set.image (R.incl : D → D') S) hne' hdir']
  rw [Set.image_image, Set.image_congr fun a _ => R.retr_incl a]
  simp

end IsContinuousLatticeRetraction
end DomainTheory.ContinuousLattice
```

6

retr_ambientSSup_eq_sSup.lean

```
module
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaces.ambientSSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace IsContinuousLatticeRetraction

/-- **Scott 1972, Prop 2.10 / March 1972 correction (p. 135).** For directed `S ⊆ D`,
`j(⊔S') = ⊔S`: the retraction sends the ambient join back to the subspace join. -/
theorem retr_ambientSSup_eq_sSup (R : IsContinuousLatticeRetraction D D') (S : Set D)
    (hS : S.Nonempty) (hdir : DirectedOn (· ≤ ·) S) :
    R.retr (ambientSSup R S) = sSup S := by
  unfold ambientSSup
  have hdir' : DirectedOn (· ≤ ·) (Set.image (R.incl : D → D') S) := by
    intro p hp q hq
    obtain ⟨a, ha, rfl⟩ := hp
    obtain ⟨b, hb, rfl⟩ := hq
    obtain ⟨c, hc, hac, hbc⟩ := hdir a ha b hb
    exact ⟨R.incl c, Set.mem_image_of_mem _ hc, R.incl.monotone hac, R.incl.monotone
    hbc⟩
  have hne' : (Set.image (R.incl : D → D') S).Nonempty := Set.image_nonempty.2 hS
  rw [R.retr.preservesDirectedSup_coe (Set.image (R.incl : D → D') S) hne' hdir']
  rw [Set.image_image, Set.image_congr fun a _ => R.retr_incl a]
  simp

end IsContinuousLatticeRetraction
end DomainTheory.ContinuousLattice
```

7

retr_ambientSSup_eq_sSup.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.ambientSSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section
```

```
namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace IsContinuousLatticeRetraction

/-- **Scott 1972, Prop 2.10 / March 1972 correction (p. 135).** For directed `S ⊆ D`,
`j(⊔S') = ⊔S`: the retraction sends the ambient join back to the subspace join. -/
theorem retr_ambientSSup_eq_sSup (R : IsContinuousLatticeRetraction D D') (S : Set D)
    (hS : S.Nonempty) (hdir : DirectedOn (· ≤ ·) S) :
    R.retr (ambientSSup R S) = sSup S := by
  unfold ambientSSup
  have hdir' : DirectedOn (· ≤ ·) (Set.image (R.incl : D → D') S) := by
    intro p hp q hq
    obtain ⟨a, ha, rfl⟩ := hp
    obtain ⟨b, hb, rfl⟩ := hq
    obtain ⟨c, hc, hac, hbc⟩ := hdir a ha b hb
    exact ⟨R.incl c, Set.mem_image_of_mem _ hc, R.incl.monotone hac, R.incl.monotone
    hbc⟩
  have hne' : (Set.image (R.incl : D → D') S).Nonempty := Set.image_nonempty.2 hS
  rw [R.retr.preservesDirectedSup_coe (Set.image (R.incl : D → D') S) hne' hdir']
  rw [Set.image_image, Set.image_congr fun a _ => R.retr_incl a]
  simp

end IsContinuousLatticeRetraction
end DomainTheory.ContinuousLattice
```

8

retr_ambientSSup_eq_sSup.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.ambientSSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']
```

```
namespace IsContinuousLatticeRetraction

/-- **Scott 1972, Prop 2.10 / March 1972 correction (p. 135).** For directed `S ⊆ D`,
`j(⊔S') = ⊔S`: the retraction sends the ambient join back to the subspace join. -/
theorem retr_ambientSSup_eq_sSup (R : IsContinuousLatticeRetraction D D') (S : Set D)
    (hS : S.Nonempty) (hdir : DirectedOn (· ≤ ·) S) :
    R.retr (ambientSSup R S) = sSup S := by
  unfold ambientSSup
  have hdir' : DirectedOn (· ≤ ·) (Set.image (R.incl : D → D') S) := by
    intro p hp q hq
    obtain ⟨a, ha, rfl⟩ := hp
    obtain ⟨b, hb, rfl⟩ := hq
    obtain ⟨c, hc, hac, hbc⟩ := hdir a ha b hb
    exact ⟨R.incl c, Set.mem_image_of_mem _ hc, R.incl.monotone hac, R.incl.monotone
    hbc⟩
  have hne' : (Set.image (R.incl : D → D') S).Nonempty := Set.image_nonempty.2 hS
  rw [R.retr.preservesDirectedSup_coe (Set.image (R.incl : D → D') S) hne' hdir']
  rw [Set.image_image, Set.image_congr fun a _ => R.retr_incl a]
  simp

end IsContinuousLatticeRetraction
end DomainTheory.ContinuousLattice
```

9

retr_bot.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

theorem retr_bot (P : IsContinuousLatticeProjection D D') : (P.retr : D' → D) ⊥ = ⊥
    := by
  have hle : (⊥ : D') ≤ P.incl (⊥ : D) := bot_le
```

```
  have heq : (P.retr : D' → D) (P.incl (⊥ : D)) = (⊥ : D) := P.retr_incl (d := (⊥ : D
    ))
  exact le_antisymm (le_trans (P.retr.monotone hle) (le_of_eq heq)) bot_le

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

0

retr_bot.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

theorem retr_bot (P : IsContinuousLatticeProjection D D') : (P.retr : D' → D) ⊥ = ⊥
    := by
  have hle : (⊥ : D') ≤ P.incl (⊥ : D) := bot_le
  have heq : (P.retr : D' → D) (P.incl (⊥ : D)) = (⊥ : D) := P.retr_incl (d := (⊥ : D
    ))
  exact le_antisymm (le_trans (P.retr.monotone hle) (le_of_eq heq)) bot_le

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

1

retr_bot.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
```

```
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

theorem retr_bot (P : IsContinuousLatticeProjection D D') : (P.retr : D' → D) ⊥ = ⊥
    := by
  have hle : (⊥ : D') ≤ P.incl (⊥ : D) := bot_le
  have heq : (P.retr : D' → D) (P.incl (⊥ : D)) = (⊥ : D) := P.retr_incl (d := (⊥ : D
    ))
  exact le_antisymm (le_trans (P.retr.monotone hle) (le_of_eq heq)) bot_le

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

2

retr_bot.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310
```

```
open Finset

namespace IsContinuousLatticeProjection

theorem retr_bot (P : IsContinuousLatticeProjection D D') : (P.retr : D' → D) ⊥ = ⊥
    := by
  have hle : (⊥ : D') ≤ P.incl (⊥ : D) := bot_le
  have heq : (P.retr : D' → D) (P.incl (⊥ : D)) = (⊥ : D) := P.retr_incl (d := (⊥ : D
    ))
  exact le_antisymm (le_trans (P.retr.monotone hle) (le_of_eq heq)) bot_le

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

3

retr_bot.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

theorem retr_bot (P : IsContinuousLatticeProjection D D') : (P.retr : D' → D) ⊥ = ⊥
    := by
  have hle : (⊥ : D') ≤ P.incl (⊥ : D) := bot_le
  have heq : (P.retr : D' → D) (P.incl (⊥ : D)) = (⊥ : D) := P.retr_incl (d := (⊥ : D
    ))
  exact le_antisymm (le_trans (P.retr.monotone hle) (le_of_eq heq)) bot_le

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

4

retr_bot.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

theorem retr_bot (P : IsContinuousLatticeProjection D D') : (P.retr : D' → D) ⊥ = ⊥
    := by
  have hle : (⊥ : D') ≤ P.incl (⊥ : D) := bot_le
  have heq : (P.retr : D' → D) (P.incl (⊥ : D)) = (⊥ : D) := P.retr_incl (d := (⊥ : D
    ))
  exact le_antisymm (le_trans (P.retr.monotone hle) (le_of_eq heq)) bot_le

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

5

retr_bot.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace
```

```
variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

theorem retr_bot (P : IsContinuousLatticeProjection D D') : (P.retr : D' → D) ⊥ = ⊥
    := by
  have hle : (⊥ : D') ≤ P.incl (⊥ : D) := bot_le
  have heq : (P.retr : D' → D) (P.incl (⊥ : D)) = (⊥ : D) := P.retr_incl (d := (⊥ : D
    ))
  exact le_antisymm (le_trans (P.retr.monotone hle) (le_of_eq heq)) bot_le

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

6

retr_bot.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

theorem retr_bot (P : IsContinuousLatticeProjection D D') : (P.retr : D' → D) ⊥ = ⊥
    := by
  have hle : (⊥ : D') ≤ P.incl (⊥ : D) := bot_le
```

```
  have heq : (P.retr : D' → D) (P.incl (⊥ : D)) = (⊥ : D) := P.retr_incl (d := (⊥ : D
    ))
  exact le_antisymm (le_trans (P.retr.monotone hle) (le_of_eq heq)) bot_le

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

7

retr_bot.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

theorem retr_bot (P : IsContinuousLatticeProjection D D') : (P.retr : D' → D) ⊥ = ⊥
    := by
  have hle : (⊥ : D') ≤ P.incl (⊥ : D) := bot_le
  have heq : (P.retr : D' → D) (P.incl (⊥ : D)) = (⊥ : D) := P.retr_incl (d := (⊥ : D
    ))
  exact le_antisymm (le_trans (P.retr.monotone hle) (le_of_eq heq)) bot_le

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

8

retr_bot.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
```

```
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

theorem retr_bot (P : IsContinuousLatticeProjection D D') : (P.retr : D' → D) ⊥ = ⊥
    := by
  have hle : (⊥ : D') ≤ P.incl (⊥ : D) := bot_le
  have heq : (P.retr : D' → D) (P.incl (⊥ : D)) = (⊥ : D) := P.retr_incl (d := (⊥ : D
    ))
  exact le_antisymm (le_trans (P.retr.monotone hle) (le_of_eq heq)) bot_le

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

9

retr_eq_sSup.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310
```

```
open Finset

namespace IsContinuousLatticeProjection

/-- **Scott 1972, Proposition 3.10(iv), uniqueness.** The retraction of any projection
    is forced to
be Scott's formula `j(x') = ⊔ { x | i(x) ⊑ x' }`. `≤`: `j(x')` itself satisfies `i(j x
    ') ⊑ x'`
(by `i ∘ j ⊑ id`), so it is a member of the set; `≥`: each member `x` (with `i x ⊑ x'`)
    satisfies
`x = j(i x) ⊑ j(x')` by `j ∘ i = id` and monotonicity. -/
theorem retr_eq_sSup (P : IsContinuousLatticeProjection D D') (x' : D') :
    (P.retr : D' → D) x' = sSup {x | (P.incl : D → D') x ≤ x'} := by
  apply le_antisymm
  · exact le_sSup (show (P.incl : D → D') (P.retr x') ≤ x' from P.incl_retr_le x')
  · refine sSup_le fun x hx => ?_
    calc x = (P.retr : D' → D) (P.incl x) := (P.retr_incl x).symm
      _ ≤ (P.retr : D' → D) x' := P.retr.monotone hx

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

0

retr_eq_sSup.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

/-- **Scott 1972, Proposition 3.10(iv), uniqueness.** The retraction of any projection
    is forced to
be Scott's formula `j(x') = ⊔ { x | i(x) ⊑ x' }`. `≤`: `j(x')` itself satisfies `i(j x
```

```
    ') ⊑ x'`
(by `i ∘ j ⊑ id`), so it is a member of the set; `≥`: each member `x` (with `i x ⊑ x'`)
     satisfies
`x = j(i x) ⊑ j(x')` by `j ∘ i = id` and monotonicity. -/
theorem retr_eq_sSup (P : IsContinuousLatticeProjection D D') (x' : D') :
    (P.retr : D' → D) x' = sSup {x | (P.incl : D → D') x ≤ x'} := by
  apply le_antisymm
  · exact le_sSup (show (P.incl : D → D') (P.retr x') ≤ x' from P.incl_retr_le x')
  · refine sSup_le fun x hx => ?_
    calc x = (P.retr : D' → D) (P.incl x) := (P.retr_incl x).symm
      _ ≤ (P.retr : D' → D) x' := P.retr.monotone hx

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

1

retr_eq_sSup.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

/-- **Scott 1972, Proposition 3.10(iv), uniqueness.** The retraction of any projection
     is forced to
be Scott's formula `j(x') = ⊔ { x | i(x) ⊑ x' }`. `≤`: `j(x')` itself satisfies `i(j x
    ') ⊑ x'`
(by `i ∘ j ⊑ id`), so it is a member of the set; `≥`: each member `x` (with `i x ⊑ x'`)
     satisfies
`x = j(i x) ⊑ j(x')` by `j ∘ i = id` and monotonicity. -/
theorem retr_eq_sSup (P : IsContinuousLatticeProjection D D') (x' : D') :
    (P.retr : D' → D) x' = sSup {x | (P.incl : D → D') x ≤ x'} := by
  apply le_antisymm
  · exact le_sSup (show (P.incl : D → D') (P.retr x') ≤ x' from P.incl_retr_le x')
```

```
  · refine sSup_le fun x hx => ?_
    calc x = (P.retr : D' → D) (P.incl x) := (P.retr_incl x).symm
      _ ≤ (P.retr : D' → D) x' := P.retr.monotone hx

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

2

retr_eq_sSup.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

/-- **Scott 1972, Proposition 3.10(iv), uniqueness.** The retraction of any projection
     is forced to
be Scott's formula `j(x') = ⊔ { x | i(x) ⊑ x' }`. `≤`: `j(x')` itself satisfies `i(j x
    ') ⊑ x'`
(by `i ∘ j ⊑ id`), so it is a member of the set; `≥`: each member `x` (with `i x ⊑ x'`)
     satisfies
`x = j(i x) ⊑ j(x')` by `j ∘ i = id` and monotonicity. -/
theorem retr_eq_sSup (P : IsContinuousLatticeProjection D D') (x' : D') :
    (P.retr : D' → D) x' = sSup {x | (P.incl : D → D') x ≤ x'} := by
  apply le_antisymm
  · exact le_sSup (show (P.incl : D → D') (P.retr x') ≤ x' from P.incl_retr_le x')
  · refine sSup_le fun x hx => ?_
    calc x = (P.retr : D' → D) (P.incl x) := (P.retr_incl x).symm
      _ ≤ (P.retr : D' → D) x' := P.retr.monotone hx

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

3

retr_eq_sSup.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

/-- **Scott 1972, Proposition 3.10(iv), uniqueness.** The retraction of any projection
     is forced to
be Scott's formula `j(x') = ⊔ { x | i(x) ⊑ x' }`. `≤`: `j(x')` itself satisfies `i(j x
    ') ⊑ x'`
(by `i ∘ j ⊑ id`), so it is a member of the set; `≥`: each member `x` (with `i x ⊑ x'`)
     satisfies
`x = j(i x) ⊑ j(x')` by `j ∘ i = id` and monotonicity. -/
theorem retr_eq_sSup (P : IsContinuousLatticeProjection D D') (x' : D') :
    (P.retr : D' → D) x' = sSup {x | (P.incl : D → D') x ≤ x'} := by
  apply le_antisymm
  · exact le_sSup (show (P.incl : D → D') (P.retr x') ≤ x' from P.incl_retr_le x')
  · refine sSup_le fun x hx => ?_
    calc x = (P.retr : D' → D) (P.incl x) := (P.retr_incl x).symm
      _ ≤ (P.retr : D' → D) x' := P.retr.monotone hx

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

4

retr_eq_sSup.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
```

```
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

/-- **Scott 1972, Proposition 3.10(iv), uniqueness.** The retraction of any projection
     is forced to
be Scott's formula `j(x') = ⊔ { x | i(x) ⊑ x' }`. `≤`: `j(x')` itself satisfies `i(j x
    ') ⊑ x'`
(by `i ∘ j ⊑ id`), so it is a member of the set; `≥`: each member `x` (with `i x ⊑ x'`)
     satisfies
`x = j(i x) ⊑ j(x')` by `j ∘ i = id` and monotonicity. -/
theorem retr_eq_sSup (P : IsContinuousLatticeProjection D D') (x' : D') :
    (P.retr : D' → D) x' = sSup {x | (P.incl : D → D') x ≤ x'} := by
  apply le_antisymm
  · exact le_sSup (show (P.incl : D → D') (P.retr x') ≤ x' from P.incl_retr_le x')
  · refine sSup_le fun x hx => ?_
    calc x = (P.retr : D' → D) (P.incl x) := (P.retr_incl x).symm
      _ ≤ (P.retr : D' → D) x' := P.retr.monotone hx

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

5

retr_eq_sSup.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace
```

```
variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

/-- **Scott 1972, Proposition 3.10(iv), uniqueness.** The retraction of any projection
     is forced to
be Scott's formula `j(x') = ⊔ { x | i(x) ⊑ x' }`. `≤`: `j(x')` itself satisfies `i(j x
    ') ⊑ x'`
(by `i ∘ j ⊑ id`), so it is a member of the set; `≥`: each member `x` (with `i x ⊑ x'`)
     satisfies
`x = j(i x) ⊑ j(x')` by `j ∘ i = id` and monotonicity. -/
theorem retr_eq_sSup (P : IsContinuousLatticeProjection D D') (x' : D') :
    (P.retr : D' → D) x' = sSup {x | (P.incl : D → D') x ≤ x'} := by
  apply le_antisymm
  · exact le_sSup (show (P.incl : D → D') (P.retr x') ≤ x' from P.incl_retr_le x')
  · refine sSup_le fun x hx => ?_
    calc x = (P.retr : D' → D) (P.incl x) := (P.retr_incl x).symm
      _ ≤ (P.retr : D' → D) x' := P.retr.monotone hx

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

6

retr_eq_sSup.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310
```

```
open Finset

namespace IsContinuousLatticeProjection

/-- **Scott 1972, Proposition 3.10(iv), uniqueness.** The retraction of any projection
    is forced to
be Scott's formula `j(x') = ⊔ { x | i(x) ⊑ x' }`. `≤`: `j(x')` itself satisfies `i(j x
    ') ⊑ x'`
(by `i ∘ j ⊑ id`), so it is a member of the set; `≥`: each member `x` (with `i x ⊑ x'`)
    satisfies
`x = j(i x) ⊑ j(x')` by `j ∘ i = id` and monotonicity. -/
theorem retr_eq_sSup (P : IsContinuousLatticeProjection D D') (x' : D') :
    (P.retr : D' → D) x' = sSup {x | (P.incl : D → D') x ≤ x'} := by
  apply le_antisymm
  · exact le_sSup (show (P.incl : D → D') (P.retr x') ≤ x' from P.incl_retr_le x')
  · refine sSup_le fun x hx => ?_
    calc x = (P.retr : D' → D) (P.incl x) := (P.retr_incl x).symm
      _ ≤ (P.retr : D' → D) x' := P.retr.monotone hx

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

7

retr_eq_sSup.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

/-- **Scott 1972, Proposition 3.10(iv), uniqueness.** The retraction of any projection
    is forced to
be Scott's formula `j(x') = ⊔ { x | i(x) ⊑ x' }`. `≤`: `j(x')` itself satisfies `i(j x
```

```
    ') ⊑ x'`
(by `i ∘ j ⊑ id`), so it is a member of the set; `≥`: each member `x` (with `i x ⊑ x'`)
     satisfies
`x = j(i x) ⊑ j(x')` by `j ∘ i = id` and monotonicity. -/
theorem retr_eq_sSup (P : IsContinuousLatticeProjection D D') (x' : D') :
    (P.retr : D' → D) x' = sSup {x | (P.incl : D → D') x ≤ x'} := by
  apply le_antisymm
  · exact le_sSup (show (P.incl : D → D') (P.retr x') ≤ x' from P.incl_retr_le x')
  · refine sSup_le fun x hx => ?_
    calc x = (P.retr : D' → D) (P.incl x) := (P.retr_incl x).symm
      _ ≤ (P.retr : D' → D) x' := P.retr.monotone hx

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

8

retr_eq_sSup.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

/-- **Scott 1972, Proposition 3.10(iv), uniqueness.** The retraction of any projection
     is forced to
be Scott's formula `j(x') = ⊔ { x | i(x) ⊑ x' }`. `≤`: `j(x')` itself satisfies `i(j x
    ') ⊑ x'`
(by `i ∘ j ⊑ id`), so it is a member of the set; `≥`: each member `x` (with `i x ⊑ x'`)
     satisfies
`x = j(i x) ⊑ j(x')` by `j ∘ i = id` and monotonicity. -/
theorem retr_eq_sSup (P : IsContinuousLatticeProjection D D') (x' : D') :
    (P.retr : D' → D) x' = sSup {x | (P.incl : D → D') x ≤ x'} := by
  apply le_antisymm
  · exact le_sSup (show (P.incl : D → D') (P.retr x') ≤ x' from P.incl_retr_le x')
```

```
  · refine sSup_le fun x hx => ?_
    calc x = (P.retr : D' → D) (P.incl x) := (P.retr_incl x).symm
      _ ≤ (P.retr : D' → D) x' := P.retr.monotone hx

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

9

retr_wayBelow_of_wayBelow_incl.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_preservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeRetraction
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import DomainTheory.ContinuousLattice.ScottMaps.scottOpen_preimage
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace IsContinuousLatticeRetraction

/-- **Heart of Scott's proof of 2.10.** If `x' ≪ i(d)` in the ambient continuous
    lattice `D'`,
then its image `j(x')` is way below `d` in the retract `D`. The Scott-open witness in `
    D` is the
preimage `i^-1V'` of an ambient Scott-open witness `V'` (Scott-open because `i`
    preserves directed
suprema); for `z ∈ i^-1V'` we have `x' ⊑ i(z)`, hence `j(x') ⊑ j(i(z)) = z`. No
    projection
hypothesis is needed -- `j ∘ i = id` and monotonicity suffice. -/
theorem retr_wayBelow_of_wayBelow_incl (R : IsContinuousLatticeRetraction D D')
    {d : D} {x' : D'} (hx' : x' ≪ R.incl d) : R.retr x' ≪ d := by
  obtain ⟨U', hU'open, hd_in, hU'sub⟩ := hx'
  refine ⟨(R.incl : D → D') ^-1' U', scottOpen_preimage R.incl_preservesDirectedSup hU
    'open,
    hd_in, fun z hz => ?_⟩
  have hle : x' ≤ R.incl z := Set.mem_Ici.1 (hU'sub hz)
  have hjle := R.retr.monotone hle
  rw [R.retr_incl z] at hjle
  exact Set.mem_Ici.2 hjle
```

```
end IsContinuousLatticeRetraction
end DomainTheory.ContinuousLattice
```

0

retr_wayBelow_of_wayBelow_incl.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_preservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeRetraction
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import DomainTheory.ContinuousLattice.ScottMaps.scottOpen_preimage
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace IsContinuousLatticeRetraction

/-- **Heart of Scott's proof of 2.10.** If `x' ≪ i(d)` in the ambient continuous
    lattice `D'`,
then its image `j(x')` is way below `d` in the retract `D`. The Scott-open witness in `
    D` is the
preimage `i^-1V'` of an ambient Scott-open witness `V'` (Scott-open because `i`
    preserves directed
suprema); for `z ∈ i^-1V'` we have `x' ⊑ i(z)`, hence `j(x') ⊑ j(i(z)) = z`. No
    projection
hypothesis is needed -- `j ∘ i = id` and monotonicity suffice. -/
theorem retr_wayBelow_of_wayBelow_incl (R : IsContinuousLatticeRetraction D D')
    {d : D} {x' : D'} (hx' : x' ≪ R.incl d) : R.retr x' ≪ d := by
  obtain ⟨U', hU'open, hd_in, hU'sub⟩ := hx'
  refine ⟨(R.incl : D → D') ^-1' U', scottOpen_preimage R.incl_preservesDirectedSup hU
    'open,
    hd_in, fun z hz => ?_⟩
  have hle : x' ≤ R.incl z := Set.mem_Ici.1 (hU'sub hz)
  have hjle := R.retr.monotone hle
  rw [R.retr_incl z] at hjle
  exact Set.mem_Ici.2 hjle

end IsContinuousLatticeRetraction
end DomainTheory.ContinuousLattice
```

1

retr_wayBelow_of_wayBelow_incl.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_preservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeRetraction
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import DomainTheory.ContinuousLattice.ScottMaps.scottOpen_preimage
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace IsContinuousLatticeRetraction

/-- **Heart of Scott's proof of 2.10.** If `x' ≪ i(d)` in the ambient continuous
    lattice `D'`,
then its image `j(x')` is way below `d` in the retract `D`. The Scott-open witness in `
    D` is the
preimage `i^-1V'` of an ambient Scott-open witness `V'` (Scott-open because `i`
    preserves directed
suprema); for `z ∈ i^-1V'` we have `x' ⊑ i(z)`, hence `j(x') ⊑ j(i(z)) = z`. No
    projection
hypothesis is needed -- `j ∘ i = id` and monotonicity suffice. -/
theorem retr_wayBelow_of_wayBelow_incl (R : IsContinuousLatticeRetraction D D')
    {d : D} {x' : D'} (hx' : x' ≪ R.incl d) : R.retr x' ≪ d := by
  obtain ⟨U', hU'open, hd_in, hU'sub⟩ := hx'
  refine ⟨(R.incl : D → D') ^-1' U', scottOpen_preimage R.incl_preservesDirectedSup hU
    'open,
    hd_in, fun z hz => ?_⟩
  have hle : x' ≤ R.incl z := Set.mem_Ici.1 (hU'sub hz)
  have hjle := R.retr.monotone hle
  rw [R.retr_incl z] at hjle
  exact Set.mem_Ici.2 hjle

end IsContinuousLatticeRetraction
end DomainTheory.ContinuousLattice
```

2

4.8.4 Factoid 8.4

A domain of domains. Not many details can be presented here, but we would also like to remark that even domains can be made into a domain. One way of getting an idea of how this is possible is to note that since

subdomains of $\mathbf{U}$ correspond to certain kinds of functions on $\mathbf{U}$, and since the function space of $\mathbf{U}$ is also a subdomain of $\mathbf{U}$, it might be suspected that the subdomains of $\mathbf{U}$ form a single subdomain of $\mathbf{U}$.

That is a fairly sophisticated way of reaching the conclusion (and many details have to be worked out). A more elementary approach would be just to ask what it means to give a finite amount of information about a domain. For the sake of uniformity, suppose that the data objects of the possible domain are drawn from the non-negative integers, and that we conventionally use 0 for Δ. Then to give a finite amount of information about a domain is—roughly—to specify a finite part of Con and a finite part of $\vdash$. To make the formulation easier, we will reserve for 1 a rôle like the one recently played by ∇. What the specifications will boil down to is pairs (u, v) of finite sets of integers used as data objects to convey one piece of information about an entailment relation.

But hold, entailment relations are very closely connected to approximable mappings. Indeed, we remarked before that the identity function as an approximable mapping on a domain is just represented as the underlying entailment relation itself. Suppose we take as our domain the domain of all sets of integers. It is a powerset, so call it $\mathbf{P}$. That is to say, the integers are the data objects, all finite sets are consistent, and the entailment relation is the minimal possible one. (As far as elements go, an arbitrary set of integers is equal to the union of all its finite subsets, which means that the elements of the domain are in a one-one correspondence with the arbitrary sets of integers.) The question is: which approximable mappings on $\mathbf{P}$ into itself correspond to entailment relations on the integers as data objects?

The answer can be expressed most succinctly using our standard notation. If we think of $r : \mathbf{P} \to \mathbf{P}$ as a relation between finite sets in the usual way, then to say that r is reflexive is to say:

(1) $I_{\mathbf{P}} \subseteq r$.

To say that r is transitive is to say:

(2) $r \circ r = r$. To say that for r the object 0 plays at being Δ is to say:

$$(3) \quad \bar{0} \subseteq r(\bot),$$

where in general $\bar{n}$ is short for $\{n\}$ in the domain $\mathbf{P}$. Then, to say that 1 plays at being a rogue object is to say:

$$(4) \quad \top = r(\bar{1}).$$

Finally, to say that 1 is an inconsistent object that has to be excluded is to say:

$$(5) \quad \bar{1} \not\subseteq r(\bar{0}).$$

That's it. The collection of approximable mappings satisfying (1)–(5) gives us all the entailment relations we need. Condition (5) is a consistency condition, and for r the consistent finite sets u are those such that $\bar{1} \not\subseteq r(\bar{u})$. What we are asserting is that the totality of r satisfying (1)–(5) forms the elements of a domain—one that has been derived from $(\mathbf{P} \to \mathbf{P})$ in a way similar to the way we derived $\mathbf{U}$ from $\mathbf{V}$ above.

Having made domains into a domain, the next step is to see how constructs on domain (i.e. functors) can be made into approximable mappings. But the retelling and development of that story will have to wait for another publication along with the very interesting chapter on powerdomains. I only hope the ground covered here makes the theory of domains seem more elementary and more natural. ?

retr_wayBelow_of_wayBelow_incl.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_preservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeRetraction
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import DomainTheory.ContinuousLattice.ScottMaps.scottOpen_preimage
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace IsContinuousLatticeRetraction

/-- **Heart of Scott's proof of 2.10.** If `x' ≪ i(d)` in the ambient continuous
    lattice `D'`,
then its image `j(x')` is way below `d` in the retract `D`. The Scott-open witness in `
    D` is the
preimage `i^-1V'` of an ambient Scott-open witness `V'` (Scott-open because `i`
    preserves directed
suprema); for `z ∈ i^-1V'` we have `x' ⊑ i(z)`, hence `j(x') ⊑ j(i(z)) = z`. No
    projection
hypothesis is needed -- `j ∘ i = id` and monotonicity suffice. -/
theorem retr_wayBelow_of_wayBelow_incl (R : IsContinuousLatticeRetraction D D')
    {d : D} {x' : D'} (hx' : x' ≪ R.incl d) : R.retr x' ≪ d := by
  obtain ⟨U', hU'open, hd_in, hU'sub⟩ := hx'
  refine ⟨(R.incl : D → D') ^-1' U', scottOpen_preimage R.incl_preservesDirectedSup hU
    'open,
    hd_in, fun z hz => ?_⟩
  have hle : x' ≤ R.incl z := Set.mem_Ici.1 (hU'sub hz)
  have hjle := R.retr.monotone hle
  rw [R.retr_incl z] at hjle
  exact Set.mem_Ici.2 hjle

end IsContinuousLatticeRetraction
end DomainTheory.ContinuousLattice
```

3

retr_wayBelow_of_wayBelow_incl.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_preservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.
```

```
    IsContinuousLatticeRetraction
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import DomainTheory.ContinuousLattice.ScottMaps.scottOpen_preimage
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace IsContinuousLatticeRetraction

/-- **Heart of Scott's proof of 2.10.** If `x' ≪ i(d)` in the ambient continuous
    lattice `D'`,
then its image `j(x')` is way below `d` in the retract `D`. The Scott-open witness in `
    D` is the
preimage `i^-1V'` of an ambient Scott-open witness `V'` (Scott-open because `i`
    preserves directed
suprema); for `z ∈ i^-1V'` we have `x' ⊑ i(z)`, hence `j(x') ⊑ j(i(z)) = z`. No
    projection
hypothesis is needed -- `j ∘ i = id` and monotonicity suffice. -/
theorem retr_wayBelow_of_wayBelow_incl (R : IsContinuousLatticeRetraction D D')
    {d : D} {x' : D'} (hx' : x' ≪ R.incl d) : R.retr x' ≪ d := by
  obtain ⟨U', hU'open, hd_in, hU'sub⟩ := hx'
  refine ⟨(R.incl : D → D') ^-1' U', scottOpen_preimage R.incl_preservesDirectedSup hU
    'open,
    hd_in, fun z hz => ?_⟩
  have hle : x' ≤ R.incl z := Set.mem_Ici.1 (hU'sub hz)
  have hjle := R.retr.monotone hle
  rw [R.retr_incl z] at hjle
  exact Set.mem_Ici.2 hjle

end IsContinuousLatticeRetraction
end DomainTheory.ContinuousLattice
```

4

retr_wayBelow_of_wayBelow_incl.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_preservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeRetraction
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import DomainTheory.ContinuousLattice.ScottMaps.scottOpen_preimage
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import Mathlib.Topology.Defs.Basic
```

```
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace IsContinuousLatticeRetraction

/-- **Heart of Scott's proof of 2.10.** If `x' ≪ i(d)` in the ambient continuous
    lattice `D'`,
then its image `j(x')` is way below `d` in the retract `D`. The Scott-open witness in `
    D` is the
preimage `i^-1V'` of an ambient Scott-open witness `V'` (Scott-open because `i`
    preserves directed
suprema); for `z ∈ i^-1V'` we have `x' ⊑ i(z)`, hence `j(x') ⊑ j(i(z)) = z`. No
    projection
hypothesis is needed -- `j ∘ i = id` and monotonicity suffice. -/
theorem retr_wayBelow_of_wayBelow_incl (R : IsContinuousLatticeRetraction D D')
    {d : D} {x' : D'} (hx' : x' ≪ R.incl d) : R.retr x' ≪ d := by
  obtain ⟨U', hU'open, hd_in, hU'sub⟩ := hx'
  refine ⟨(R.incl : D → D') ^-1' U', scottOpen_preimage R.incl_preservesDirectedSup hU
    'open,
    hd_in, fun z hz => ?_⟩
  have hle : x' ≤ R.incl z := Set.mem_Ici.1 (hU'sub hz)
  have hjle := R.retr.monotone hle
  rw [R.retr_incl z] at hjle
  exact Set.mem_Ici.2 hjle

end IsContinuousLatticeRetraction
end DomainTheory.ContinuousLattice
```

5

retr_wayBelow_of_wayBelow_incl.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_preservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeRetraction
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import DomainTheory.ContinuousLattice.ScottMaps.scottOpen_preimage
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice
```

```
open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace IsContinuousLatticeRetraction

/-- **Heart of Scott's proof of 2.10.** If `x' ≪ i(d)` in the ambient continuous
    lattice `D'`,
then its image `j(x')` is way below `d` in the retract `D`. The Scott-open witness in `
    D` is the
preimage `i^-1V'` of an ambient Scott-open witness `V'` (Scott-open because `i`
    preserves directed
suprema); for `z ∈ i^-1V'` we have `x' ⊑ i(z)`, hence `j(x') ⊑ j(i(z)) = z`. No
    projection
hypothesis is needed -- `j ∘ i = id` and monotonicity suffice. -/
theorem retr_wayBelow_of_wayBelow_incl (R : IsContinuousLatticeRetraction D D')
    {d : D} {x' : D'} (hx' : x' ≪ R.incl d) : R.retr x' ≪ d := by
  obtain ⟨U', hU'open, hd_in, hU'sub⟩ := hx'
  refine ⟨(R.incl : D → D') ^-1' U', scottOpen_preimage R.incl_preservesDirectedSup hU
    'open,
    hd_in, fun z hz => ?_⟩
  have hle : x' ≤ R.incl z := Set.mem_Ici.1 (hU'sub hz)
  have hjle := R.retr.monotone hle
  rw [R.retr_incl z] at hjle
  exact Set.mem_Ici.2 hjle

end IsContinuousLatticeRetraction
end DomainTheory.ContinuousLattice
```

6

retr_wayBelow_of_wayBelow_incl.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_preservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeRetraction
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import DomainTheory.ContinuousLattice.ScottMaps.scottOpen_preimage
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
```

```
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace IsContinuousLatticeRetraction

/-- **Heart of Scott's proof of 2.10.** If `x' ≪ i(d)` in the ambient continuous
    lattice `D'`,
then its image `j(x')` is way below `d` in the retract `D`. The Scott-open witness in `
    D` is the
preimage `i^-1V'` of an ambient Scott-open witness `V'` (Scott-open because `i`
    preserves directed
suprema); for `z ∈ i^-1V'` we have `x' ⊑ i(z)`, hence `j(x') ⊑ j(i(z)) = z`. No
    projection
hypothesis is needed -- `j ∘ i = id` and monotonicity suffice. -/
theorem retr_wayBelow_of_wayBelow_incl (R : IsContinuousLatticeRetraction D D')
    {d : D} {x' : D'} (hx' : x' ≪ R.incl d) : R.retr x' ≪ d := by
  obtain ⟨U', hU'open, hd_in, hU'sub⟩ := hx'
  refine ⟨(R.incl : D → D') ^-1' U', scottOpen_preimage R.incl_preservesDirectedSup hU
    'open,
    hd_in, fun z hz => ?_⟩
  have hle : x' ≤ R.incl z := Set.mem_Ici.1 (hU'sub hz)
  have hjle := R.retr.monotone hle
  rw [R.retr_incl z] at hjle
  exact Set.mem_Ici.2 hjle

end IsContinuousLatticeRetraction
end DomainTheory.ContinuousLattice
```

7

retr_wayBelow_of_wayBelow_incl.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_preservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeRetraction
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import DomainTheory.ContinuousLattice.ScottMaps.scottOpen_preimage
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace IsContinuousLatticeRetraction
```

```
/-- **Heart of Scott's proof of 2.10.** If `x' ≪ i(d)` in the ambient continuous
    lattice `D'`,
then its image `j(x')` is way below `d` in the retract `D`. The Scott-open witness in `
    D` is the
preimage `i^-1V'` of an ambient Scott-open witness `V'` (Scott-open because `i`
    preserves directed
suprema); for `z ∈ i^-1V'` we have `x' ⊑ i(z)`, hence `j(x') ⊑ j(i(z)) = z`. No
    projection
hypothesis is needed -- `j ∘ i = id` and monotonicity suffice. -/
theorem retr_wayBelow_of_wayBelow_incl (R : IsContinuousLatticeRetraction D D')
    {d : D} {x' : D'} (hx' : x' ≪ R.incl d) : R.retr x' ≪ d := by
  obtain ⟨U', hU'open, hd_in, hU'sub⟩ := hx'
  refine ⟨(R.incl : D → D') ^-1' U', scottOpen_preimage R.incl_preservesDirectedSup hU
    'open,
    hd_in, fun z hz => ?_⟩
  have hle : x' ≤ R.incl z := Set.mem_Ici.1 (hU'sub hz)
  have hjle := R.retr.monotone hle
  rw [R.retr_incl z] at hjle
  exact Set.mem_Ici.2 hjle

end IsContinuousLatticeRetraction
end DomainTheory.ContinuousLattice
```

8

retr_wayBelow_of_wayBelow_incl.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_preservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeRetraction
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import DomainTheory.ContinuousLattice.ScottMaps.scottOpen_preimage
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace IsContinuousLatticeRetraction

/-- **Heart of Scott's proof of 2.10.** If `x' ≪ i(d)` in the ambient continuous
    lattice `D'`,
then its image `j(x')` is way below `d` in the retract `D`. The Scott-open witness in `
    D` is the
```

```
preimage `i^-1V'` of an ambient Scott-open witness `V'` (Scott-open because `i`
    preserves directed
suprema); for `z ∈ i^-1V'` we have `x' ⊑ i(z)`, hence `j(x') ⊑ j(i(z)) = z`. No
    projection
hypothesis is needed -- `j ∘ i = id` and monotonicity suffice. -/
theorem retr_wayBelow_of_wayBelow_incl (R : IsContinuousLatticeRetraction D D')
    {d : D} {x' : D'} (hx' : x' ≪ R.incl d) : R.retr x' ≪ d := by
  obtain ⟨U', hU'open, hd_in, hU'sub⟩ := hx'
  refine ⟨(R.incl : D → D') ^-1' U', scottOpen_preimage R.incl_preservesDirectedSup hU
    'open,
    hd_in, fun z hz => ?_⟩
  have hle : x' ≤ R.incl z := Set.mem_Ici.1 (hU'sub hz)
  have hjle := R.retr.monotone hle
  rw [R.retr_incl z] at hjle
  exact Set.mem_Ici.2 hjle

end IsContinuousLatticeRetraction
end DomainTheory.ContinuousLattice
```

9

proposition_3_8.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.Specialization.scottTopologicalSpace
public import DomainTheory.ContinuousLattice.Constructions.scottExtend_maximal
public import DomainTheory.ContinuousLattice.Constructions.scottExtend_continuous
public import DomainTheory.ContinuousLattice.Constructions.
    scottExtend_eq_of_continuous
public import DomainTheory.ContinuousLattice.Constructions.scottExtend
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

variable {D : Type*} [CompleteLattice D]

section InjectiveExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] {E : Type*} [
    CompleteLattice E]

/-- **Scott 1972, Proposition 3.8.** If `E` is a continuous lattice and `e : X → Y` a
    subspace
embedding, then for each continuous `f : X → E` the explicit formula `scottExtend e f`
     is the
*maximal extension* of `f` to `[Y → E]`: it is Scott-continuous (`
    scottExtend_continuous`), it
restricts to `f` along `e` (`scottExtend_eq_of_continuous`), and it dominates every
    continuous
```

```
solution of `f' ∘ e = f` (`scottExtend_maximal`). -/
theorem proposition_3_8 (hE : ContinuousLattice E) (e : X → Y) (he : IsEmbedding e)
    (f : X → E) (hf : @Continuous X E _ scottTopologicalSpace f) :
    @Continuous Y E _ scottTopologicalSpace (scottExtend e f)
      ∧ (∀ x, scottExtend e f (e x) = f x)
      ∧ (∀ f' : Y → E, @Continuous Y E _ scottTopologicalSpace f' → (∀ x, f' (e x) =
    f x) →
          ∀ y, f' y ≤ scottExtend e f y) :=
  ⟨scottExtend_continuous hE e f,
   fun x => scottExtend_eq_of_continuous hE e he f hf x,
   fun _f' hf' h_ext y => scottExtend_maximal hE e hf' h_ext y⟩

end InjectiveExtension
end DomainTheory.ContinuousLattice
```

0

proposition_3_8.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.Specialization.scottTopologicalSpace
public import DomainTheory.ContinuousLattice.Constructions.scottExtend_maximal
public import DomainTheory.ContinuousLattice.Constructions.scottExtend_continuous
public import DomainTheory.ContinuousLattice.Constructions.
    scottExtend_eq_of_continuous
public import DomainTheory.ContinuousLattice.Constructions.scottExtend
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

variable {D : Type*} [CompleteLattice D]

section InjectiveExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] {E : Type*} [
    CompleteLattice E]

/-- **Scott 1972, Proposition 3.8.** If `E` is a continuous lattice and `e : X → Y` a
    subspace
embedding, then for each continuous `f : X → E` the explicit formula `scottExtend e f`
     is the
*maximal extension* of `f` to `[Y → E]`: it is Scott-continuous (`
    scottExtend_continuous`), it
restricts to `f` along `e` (`scottExtend_eq_of_continuous`), and it dominates every
    continuous
solution of `f' ∘ e = f` (`scottExtend_maximal`). -/
theorem proposition_3_8 (hE : ContinuousLattice E) (e : X → Y) (he : IsEmbedding e)
    (f : X → E) (hf : @Continuous X E _ scottTopologicalSpace f) :
    @Continuous Y E _ scottTopologicalSpace (scottExtend e f)
```

```
      ∧ (∀ x, scottExtend e f (e x) = f x)
      ∧ (∀ f' : Y → E, @Continuous Y E _ scottTopologicalSpace f' → (∀ x, f' (e x) =
    f x) →
          ∀ y, f' y ≤ scottExtend e f y) :=
  ⟨scottExtend_continuous hE e f,
   fun x => scottExtend_eq_of_continuous hE e he f hf x,
   fun _f' hf' h_ext y => scottExtend_maximal hE e hf' h_ext y⟩

end InjectiveExtension
end DomainTheory.ContinuousLattice
```

1

proposition_3_8.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.Specialization.scottTopologicalSpace
public import DomainTheory.ContinuousLattice.Constructions.scottExtend_maximal
public import DomainTheory.ContinuousLattice.Constructions.scottExtend_continuous
public import DomainTheory.ContinuousLattice.Constructions.
    scottExtend_eq_of_continuous
public import DomainTheory.ContinuousLattice.Constructions.scottExtend
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

variable {D : Type*} [CompleteLattice D]

section InjectiveExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] {E : Type*} [
    CompleteLattice E]

/-- **Scott 1972, Proposition 3.8.** If `E` is a continuous lattice and `e : X → Y` a
    subspace
embedding, then for each continuous `f : X → E` the explicit formula `scottExtend e f`
     is the
*maximal extension* of `f` to `[Y → E]`: it is Scott-continuous (`
    scottExtend_continuous`), it
restricts to `f` along `e` (`scottExtend_eq_of_continuous`), and it dominates every
    continuous
solution of `f' ∘ e = f` (`scottExtend_maximal`). -/
theorem proposition_3_8 (hE : ContinuousLattice E) (e : X → Y) (he : IsEmbedding e)
    (f : X → E) (hf : @Continuous X E _ scottTopologicalSpace f) :
    @Continuous Y E _ scottTopologicalSpace (scottExtend e f)
      ∧ (∀ x, scottExtend e f (e x) = f x)
      ∧ (∀ f' : Y → E, @Continuous Y E _ scottTopologicalSpace f' → (∀ x, f' (e x) =
    f x) →
          ∀ y, f' y ≤ scottExtend e f y) :=
```

```
  ⟨scottExtend_continuous hE e f,
   fun x => scottExtend_eq_of_continuous hE e he f hf x,
   fun _f' hf' h_ext y => scottExtend_maximal hE e hf' h_ext y⟩

end InjectiveExtension
end DomainTheory.ContinuousLattice
```

2

proposition_3_8.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.Specialization.scottTopologicalSpace
public import DomainTheory.ContinuousLattice.Constructions.scottExtend_maximal
public import DomainTheory.ContinuousLattice.Constructions.scottExtend_continuous
public import DomainTheory.ContinuousLattice.Constructions.
    scottExtend_eq_of_continuous
public import DomainTheory.ContinuousLattice.Constructions.scottExtend
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

variable {D : Type*} [CompleteLattice D]

section InjectiveExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] {E : Type*} [
    CompleteLattice E]

/-- **Scott 1972, Proposition 3.8.** If `E` is a continuous lattice and `e : X → Y` a
    subspace
embedding, then for each continuous `f : X → E` the explicit formula `scottExtend e f`
     is the
*maximal extension* of `f` to `[Y → E]`: it is Scott-continuous (`
    scottExtend_continuous`), it
restricts to `f` along `e` (`scottExtend_eq_of_continuous`), and it dominates every
    continuous
solution of `f' ∘ e = f` (`scottExtend_maximal`). -/
theorem proposition_3_8 (hE : ContinuousLattice E) (e : X → Y) (he : IsEmbedding e)
    (f : X → E) (hf : @Continuous X E _ scottTopologicalSpace f) :
    @Continuous Y E _ scottTopologicalSpace (scottExtend e f)
      ∧ (∀ x, scottExtend e f (e x) = f x)
      ∧ (∀ f' : Y → E, @Continuous Y E _ scottTopologicalSpace f' → (∀ x, f' (e x) =
    f x) →
          ∀ y, f' y ≤ scottExtend e f y) :=
  ⟨scottExtend_continuous hE e f,
   fun x => scottExtend_eq_of_continuous hE e he f hf x,
   fun _f' hf' h_ext y => scottExtend_maximal hE e hf' h_ext y⟩
```

```
end InjectiveExtension
end DomainTheory.ContinuousLattice
```

3

proposition_3_8.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.Specialization.scottTopologicalSpace
public import DomainTheory.ContinuousLattice.Constructions.scottExtend_maximal
public import DomainTheory.ContinuousLattice.Constructions.scottExtend_continuous
public import DomainTheory.ContinuousLattice.Constructions.
    scottExtend_eq_of_continuous
public import DomainTheory.ContinuousLattice.Constructions.scottExtend
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

variable {D : Type*} [CompleteLattice D]

section InjectiveExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] {E : Type*} [
    CompleteLattice E]

/-- **Scott 1972, Proposition 3.8.** If `E` is a continuous lattice and `e : X → Y` a
    subspace
embedding, then for each continuous `f : X → E` the explicit formula `scottExtend e f`
     is the
*maximal extension* of `f` to `[Y → E]`: it is Scott-continuous (`
    scottExtend_continuous`), it
restricts to `f` along `e` (`scottExtend_eq_of_continuous`), and it dominates every
    continuous
solution of `f' ∘ e = f` (`scottExtend_maximal`). -/
theorem proposition_3_8 (hE : ContinuousLattice E) (e : X → Y) (he : IsEmbedding e)
    (f : X → E) (hf : @Continuous X E _ scottTopologicalSpace f) :
    @Continuous Y E _ scottTopologicalSpace (scottExtend e f)
      ∧ (∀ x, scottExtend e f (e x) = f x)
      ∧ (∀ f' : Y → E, @Continuous Y E _ scottTopologicalSpace f' → (∀ x, f' (e x) =
    f x) →
          ∀ y, f' y ≤ scottExtend e f y) :=
  ⟨scottExtend_continuous hE e f,
   fun x => scottExtend_eq_of_continuous hE e he f hf x,
   fun _f' hf' h_ext y => scottExtend_maximal hE e hf' h_ext y⟩

end InjectiveExtension
end DomainTheory.ContinuousLattice
```

4

proposition_3_8.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.Specialization.scottTopologicalSpace
public import DomainTheory.ContinuousLattice.Constructions.scottExtend_maximal
public import DomainTheory.ContinuousLattice.Constructions.scottExtend_continuous
public import DomainTheory.ContinuousLattice.Constructions.
    scottExtend_eq_of_continuous
public import DomainTheory.ContinuousLattice.Constructions.scottExtend
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

variable {D : Type*} [CompleteLattice D]

section InjectiveExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] {E : Type*} [
    CompleteLattice E]

/-- **Scott 1972, Proposition 3.8.** If `E` is a continuous lattice and `e : X → Y` a
    subspace
embedding, then for each continuous `f : X → E` the explicit formula `scottExtend e f`
     is the
*maximal extension* of `f` to `[Y → E]`: it is Scott-continuous (`
    scottExtend_continuous`), it
restricts to `f` along `e` (`scottExtend_eq_of_continuous`), and it dominates every
    continuous
solution of `f' ∘ e = f` (`scottExtend_maximal`). -/
theorem proposition_3_8 (hE : ContinuousLattice E) (e : X → Y) (he : IsEmbedding e)
    (f : X → E) (hf : @Continuous X E _ scottTopologicalSpace f) :
    @Continuous Y E _ scottTopologicalSpace (scottExtend e f)
      ∧ (∀ x, scottExtend e f (e x) = f x)
      ∧ (∀ f' : Y → E, @Continuous Y E _ scottTopologicalSpace f' → (∀ x, f' (e x) =
    f x) →
          ∀ y, f' y ≤ scottExtend e f y) :=
  ⟨scottExtend_continuous hE e f,
   fun x => scottExtend_eq_of_continuous hE e he f hf x,
   fun _f' hf' h_ext y => scottExtend_maximal hE e hf' h_ext y⟩

end InjectiveExtension
end DomainTheory.ContinuousLattice
```

5

proposition_3_8.lean

```
module
```

```
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.Specialization.scottTopologicalSpace
public import DomainTheory.ContinuousLattice.Constructions.scottExtend_maximal
public import DomainTheory.ContinuousLattice.Constructions.scottExtend_continuous
public import DomainTheory.ContinuousLattice.Constructions.
    scottExtend_eq_of_continuous
public import DomainTheory.ContinuousLattice.Constructions.scottExtend
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

variable {D : Type*} [CompleteLattice D]

section InjectiveExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] {E : Type*} [
    CompleteLattice E]

/-- **Scott 1972, Proposition 3.8.** If `E` is a continuous lattice and `e : X → Y` a
    subspace
embedding, then for each continuous `f : X → E` the explicit formula `scottExtend e f`
     is the
*maximal extension* of `f` to `[Y → E]`: it is Scott-continuous (`
    scottExtend_continuous`), it
restricts to `f` along `e` (`scottExtend_eq_of_continuous`), and it dominates every
    continuous
solution of `f' ∘ e = f` (`scottExtend_maximal`). -/
theorem proposition_3_8 (hE : ContinuousLattice E) (e : X → Y) (he : IsEmbedding e)
    (f : X → E) (hf : @Continuous X E _ scottTopologicalSpace f) :
    @Continuous Y E _ scottTopologicalSpace (scottExtend e f)
      ∧ (∀ x, scottExtend e f (e x) = f x)
      ∧ (∀ f' : Y → E, @Continuous Y E _ scottTopologicalSpace f' → (∀ x, f' (e x) =
    f x) →
          ∀ y, f' y ≤ scottExtend e f y) :=
  ⟨scottExtend_continuous hE e f,
   fun x => scottExtend_eq_of_continuous hE e he f hf x,
   fun _f' hf' h_ext y => scottExtend_maximal hE e hf' h_ext y⟩

end InjectiveExtension
end DomainTheory.ContinuousLattice
```

6

proposition_3_8.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.Specialization.scottTopologicalSpace
public import DomainTheory.ContinuousLattice.Constructions.scottExtend_maximal
public import DomainTheory.ContinuousLattice.Constructions.scottExtend_continuous
```

```
public import DomainTheory.ContinuousLattice.Constructions.
    scottExtend_eq_of_continuous
public import DomainTheory.ContinuousLattice.Constructions.scottExtend
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

variable {D : Type*} [CompleteLattice D]

section InjectiveExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] {E : Type*} [
    CompleteLattice E]

/-- **Scott 1972, Proposition 3.8.** If `E` is a continuous lattice and `e : X → Y` a
    subspace
embedding, then for each continuous `f : X → E` the explicit formula `scottExtend e f`
     is the
*maximal extension* of `f` to `[Y → E]`: it is Scott-continuous (`
    scottExtend_continuous`), it
restricts to `f` along `e` (`scottExtend_eq_of_continuous`), and it dominates every
    continuous
solution of `f' ∘ e = f` (`scottExtend_maximal`). -/
theorem proposition_3_8 (hE : ContinuousLattice E) (e : X → Y) (he : IsEmbedding e)
    (f : X → E) (hf : @Continuous X E _ scottTopologicalSpace f) :
    @Continuous Y E _ scottTopologicalSpace (scottExtend e f)
      ∧ (∀ x, scottExtend e f (e x) = f x)
      ∧ (∀ f' : Y → E, @Continuous Y E _ scottTopologicalSpace f' → (∀ x, f' (e x) =
    f x) →
          ∀ y, f' y ≤ scottExtend e f y) :=
  ⟨scottExtend_continuous hE e f,
   fun x => scottExtend_eq_of_continuous hE e he f hf x,
   fun _f' hf' h_ext y => scottExtend_maximal hE e hf' h_ext y⟩

end InjectiveExtension
end DomainTheory.ContinuousLattice
```

7

proposition_3_8.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.Specialization.scottTopologicalSpace
public import DomainTheory.ContinuousLattice.Constructions.scottExtend_maximal
public import DomainTheory.ContinuousLattice.Constructions.scottExtend_continuous
public import DomainTheory.ContinuousLattice.Constructions.
    scottExtend_eq_of_continuous
public import DomainTheory.ContinuousLattice.Constructions.scottExtend
public import Mathlib.Topology.Defs.Basic
```

```
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

variable {D : Type*} [CompleteLattice D]

section InjectiveExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] {E : Type*} [
    CompleteLattice E]

/-- **Scott 1972, Proposition 3.8.** If `E` is a continuous lattice and `e : X → Y` a
    subspace
embedding, then for each continuous `f : X → E` the explicit formula `scottExtend e f`
     is the
*maximal extension* of `f` to `[Y → E]`: it is Scott-continuous (`
    scottExtend_continuous`), it
restricts to `f` along `e` (`scottExtend_eq_of_continuous`), and it dominates every
    continuous
solution of `f' ∘ e = f` (`scottExtend_maximal`). -/
theorem proposition_3_8 (hE : ContinuousLattice E) (e : X → Y) (he : IsEmbedding e)
    (f : X → E) (hf : @Continuous X E _ scottTopologicalSpace f) :
    @Continuous Y E _ scottTopologicalSpace (scottExtend e f)
      ∧ (∀ x, scottExtend e f (e x) = f x)
      ∧ (∀ f' : Y → E, @Continuous Y E _ scottTopologicalSpace f' → (∀ x, f' (e x) =
    f x) →
          ∀ y, f' y ≤ scottExtend e f y) :=
  ⟨scottExtend_continuous hE e f,
   fun x => scottExtend_eq_of_continuous hE e he f hf x,
   fun _f' hf' h_ext y => scottExtend_maximal hE e hf' h_ext y⟩

end InjectiveExtension
end DomainTheory.ContinuousLattice
```

8

proposition_3_8.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.Specialization.scottTopologicalSpace
public import DomainTheory.ContinuousLattice.Constructions.scottExtend_maximal
public import DomainTheory.ContinuousLattice.Constructions.scottExtend_continuous
public import DomainTheory.ContinuousLattice.Constructions.
    scottExtend_eq_of_continuous
public import DomainTheory.ContinuousLattice.Constructions.scottExtend
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section
```

```
namespace DomainTheory.ContinuousLattice

open Topology Set

variable {D : Type*} [CompleteLattice D]

section InjectiveExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] {E : Type*} [
    CompleteLattice E]

/-- **Scott 1972, Proposition 3.8.** If `E` is a continuous lattice and `e : X → Y` a
    subspace
embedding, then for each continuous `f : X → E` the explicit formula `scottExtend e f`
     is the
*maximal extension* of `f` to `[Y → E]`: it is Scott-continuous (`
    scottExtend_continuous`), it
restricts to `f` along `e` (`scottExtend_eq_of_continuous`), and it dominates every
    continuous
solution of `f' ∘ e = f` (`scottExtend_maximal`). -/
theorem proposition_3_8 (hE : ContinuousLattice E) (e : X → Y) (he : IsEmbedding e)
    (f : X → E) (hf : @Continuous X E _ scottTopologicalSpace f) :
    @Continuous Y E _ scottTopologicalSpace (scottExtend e f)
      ∧ (∀ x, scottExtend e f (e x) = f x)
      ∧ (∀ f' : Y → E, @Continuous Y E _ scottTopologicalSpace f' → (∀ x, f' (e x) =
    f x) →
          ∀ y, f' y ≤ scottExtend e f y) :=
  ⟨scottExtend_continuous hE e f,
   fun x => scottExtend_eq_of_continuous hE e he f hf x,
   fun _f' hf' h_ext y => scottExtend_maximal hE e hf' h_ext y⟩

end InjectiveExtension
end DomainTheory.ContinuousLattice
```

9

scottSubspaceExtend.lean

```
module

public import DomainTheory.ContinuousLattice.Specialization.scottTopologicalSpace
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
```

```
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

/-- **Scott 1972, Proposition 3.8.** Scott's maximal subspace extension `f : Y → D`.
    -/
noncomputable def scottSubspaceExtend (e : X → Y) (f : X → D) (y : Y) : D :=
  sSup { d | ∃ U, @IsOpen Y scottTopologicalSpace U ∧ y ∈ U ∧ d =
    scottSubspaceExtendInf e f U }

end SubspaceExtension

end DomainTheory.ContinuousLattice
```

0

scottSubspaceExtend.lean

```
module

public import DomainTheory.ContinuousLattice.Specialization.scottTopologicalSpace
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

/-- **Scott 1972, Proposition 3.8.** Scott's maximal subspace extension `f : Y → D`.
    -/
noncomputable def scottSubspaceExtend (e : X → Y) (f : X → D) (y : Y) : D :=
  sSup { d | ∃ U, @IsOpen Y scottTopologicalSpace U ∧ y ∈ U ∧ d =
    scottSubspaceExtendInf e f U }

end SubspaceExtension
```

```
end DomainTheory.ContinuousLattice
```

1

scottSubspaceExtend.lean

```
module

public import DomainTheory.ContinuousLattice.Specialization.scottTopologicalSpace
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

/-- **Scott 1972, Proposition 3.8.** Scott's maximal subspace extension `f : Y → D`.
    -/
noncomputable def scottSubspaceExtend (e : X → Y) (f : X → D) (y : Y) : D :=
  sSup { d | ∃ U, @IsOpen Y scottTopologicalSpace U ∧ y ∈ U ∧ d =
    scottSubspaceExtendInf e f U }

end SubspaceExtension

end DomainTheory.ContinuousLattice
```

2

scottSubspaceExtend.lean

```
module

public import DomainTheory.ContinuousLattice.Specialization.scottTopologicalSpace
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice
```

```
open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

/-- **Scott 1972, Proposition 3.8.** Scott's maximal subspace extension `f : Y → D`.
    -/
noncomputable def scottSubspaceExtend (e : X → Y) (f : X → D) (y : Y) : D :=
  sSup { d | ∃ U, @IsOpen Y scottTopologicalSpace U ∧ y ∈ U ∧ d =
    scottSubspaceExtendInf e f U }

end SubspaceExtension

end DomainTheory.ContinuousLattice
```

3

scottSubspaceExtend.lean

```
module

public import DomainTheory.ContinuousLattice.Specialization.scottTopologicalSpace
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]
```

```
/-- **Scott 1972, Proposition 3.8.** Scott's maximal subspace extension `f : Y → D`.
    -/
noncomputable def scottSubspaceExtend (e : X → Y) (f : X → D) (y : Y) : D :=
  sSup { d | ∃ U, @IsOpen Y scottTopologicalSpace U ∧ y ∈ U ∧ d =
    scottSubspaceExtendInf e f U }

end SubspaceExtension

end DomainTheory.ContinuousLattice
```

4

scottSubspaceExtend.lean

```
module

public import DomainTheory.ContinuousLattice.Specialization.scottTopologicalSpace
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

/-- **Scott 1972, Proposition 3.8.** Scott's maximal subspace extension `f : Y → D`.
    -/
noncomputable def scottSubspaceExtend (e : X → Y) (f : X → D) (y : Y) : D :=
  sSup { d | ∃ U, @IsOpen Y scottTopologicalSpace U ∧ y ∈ U ∧ d =
    scottSubspaceExtendInf e f U }

end SubspaceExtension

end DomainTheory.ContinuousLattice
```

5

scottSubspaceExtend.lean

```
module
```

```
public import DomainTheory.ContinuousLattice.Specialization.scottTopologicalSpace
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

/-- **Scott 1972, Proposition 3.8.** Scott's maximal subspace extension `f : Y → D`.
    -/
noncomputable def scottSubspaceExtend (e : X → Y) (f : X → D) (y : Y) : D :=
  sSup { d | ∃ U, @IsOpen Y scottTopologicalSpace U ∧ y ∈ U ∧ d =
    scottSubspaceExtendInf e f U }

end SubspaceExtension

end DomainTheory.ContinuousLattice
```

6

scottSubspaceExtend.lean

```
module

public import DomainTheory.ContinuousLattice.Specialization.scottTopologicalSpace
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
```

```
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

/-- **Scott 1972, Proposition 3.8.** Scott's maximal subspace extension `f : Y → D`.
    -/
noncomputable def scottSubspaceExtend (e : X → Y) (f : X → D) (y : Y) : D :=
  sSup { d | ∃ U, @IsOpen Y scottTopologicalSpace U ∧ y ∈ U ∧ d =
    scottSubspaceExtendInf e f U }

end SubspaceExtension

end DomainTheory.ContinuousLattice
```

5 Bridge Theorems

Dana Scott developed three presentations of the same class of domains: **continuous lattices** (1972), **neighbourhood systems** (PRG-19 / 1980–81), and **information systems** (1982). In this chapter we supply **bridge theorems**: order isomorphisms and constructivity audits showing that the three presentations determine the same domains (up to isomorphism), including products, separated sums, and function spaces at the information-system level, with a transport of 1972 Scott maps along the round presentation.

The Lean development is sorry-free. The continuous-lattice identification `D ≃o RoundFilter` audits to `{propext, Quot.sound}`. Every InfoSys-facing declaration inherits `Classical.choice` from Mathlib 4.33's `Finset` instances, even when the proof term extracts no witness. Genuine classical frontiers remain Scott's 1972 topological definition of $\ll$ (algebraic $\Rightarrow$ continuous; ScottMap conjugation) and trichotomy for separated sums.

Dependencies. Each box is one folder. BridgeTheorems is the Lean files in that folder, not in a subfolder. The arrowhead points at the folder that imports, and the number is how many direct imports. A pair of arrows between the same boxes is a cycle.

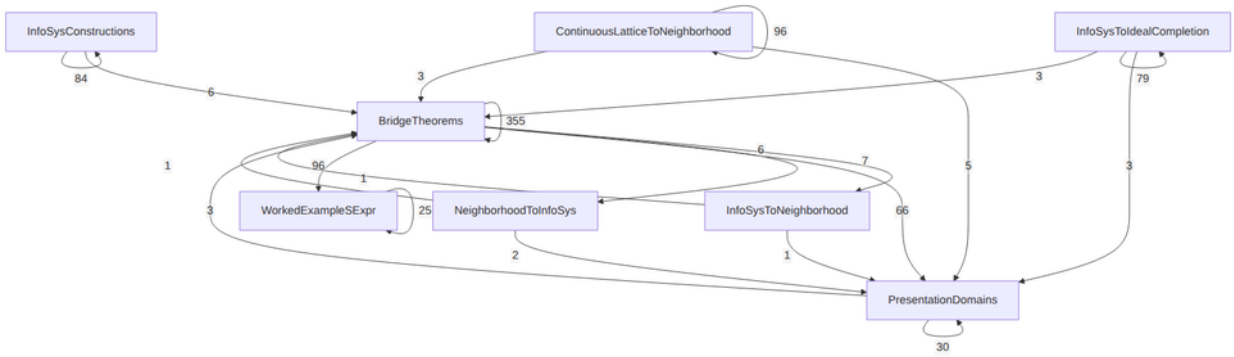


Figure 44: Bridge Theorems.

5.1 Introduction

Neighbourhood systems and information systems are equivalent in a precise sense. The mathematical folklore is stronger still: all three presentations carve out the same class of domains, related by ideal completion and the Scott topology. That equivalence is already known; this article is the write-up of a machine-checked proof of the named bridges, not a first presentation of the result. The three presentations were formalized in separate libraries; the maps between them are the content of this package.

We do **not** re-prove Scott's internal theorems; we import the finished sibling packages and build cross-presentation maps:

Presentation	Lean package	Characteristic object
Continuous lattices [1]	`scott1972`	`ContinuousLattice D`, way-below $\ll$, `ScottMap`

Presentation	Lean package	Characteristic object
Neighbourhood systems [2]	`scott1980`	`NeighborhoodSystem`, filters as domain elements
Information systems [3]	`scott1982`	`InfoSys`, elements as consistent closed token sets

The main subtlety is that a naïve reading of "domains = filters of neighbourhoods" overstates the continuous-lattice corner: for a continuous lattice `D`, the filter `{⇑a | a ≤ x}` has retract `x` but properly contains the principal round filter `{⇑a | a ≪ x}`. The correct identification is **`D ≃o RoundFilter`**, not raw `|𝒟|`.

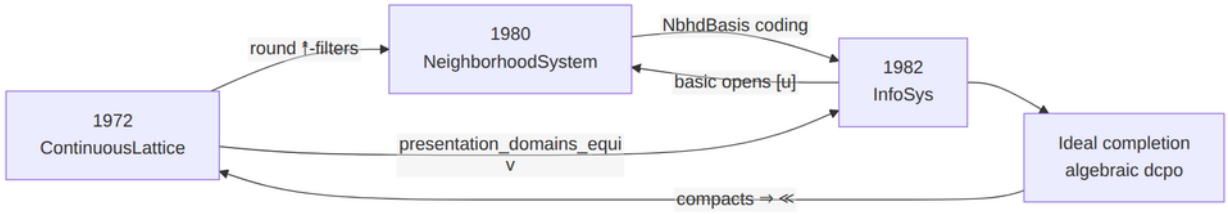


Figure 45: Introduction.

We prove **order isomorphisms of domains** (and of the named construction objects), not a 2-categorical equivalence of the full categories of continuous lattices / neighbourhood systems / information systems with all morphisms. Morphisms are linked where the sibling packages already supply them:

- approximable maps ↔ Scott-continuous maps on `|A|` (Factoid 4.6);
- Scott maps of continuous lattices ↔ their conjugates on the round presentation.

A full functorial equivalence (preserving products, exponentials, and inverse limits simultaneously across all three presentations) would require additional coherence theorems.

5.2 Catalog of bridge theorems

Theorem	Direction	Lean module	Scope
`continuousLattice_to_neighborhoodSystem`	1972 → 1980	`ContinuousLatticeToNeighborhood.lean`	library
`neighborhoodSystem_to_infoSys`	1980 → 1982	`NeighborhoodToInfoSys.lean`	compared
`infoSys_to_neighborhoodSystem`	1982 → 1980	`InfoSysToNeighborhood.lean`	compared
`infoSys_to_idealCompletion`	1982 → algebraic	`InfoSysToIdealCompletion.lean`	library

Theorem	Direction	Lean module	Scope
`idealCompletion_to_continuousLattice`	algebraic → 1972	`IdealCompletionToContinuousLattice.lean`	library
`presentation_domains_equiv`	three-way	`PresentationDomainsEquiv.lean`	compared
`infoSys_constructions_equiv`	constructions	`InfoSysConstructionsEquiv.lean`, `ScottMapBridge.lean`	library
`sexNeighborhoodIso` / `sexIdealIso` / `sexDomainEquationIso`	S-expression instance	`WorkedExampleSExpr.lean`, `SexDomainEquation.lean`	compared

The sibling packages are **finished dependencies**, not work items of this paper: `scott1972`, `scott1980`, and `scott1982` (information systems through Factoid 8.4 / domain equations). They are vendored in `vendor/` at frozen SHAs (see `vendor/FROZEN.txt` / `lakefile.toml`), with the local modifications recorded there and in `formalization.yaml`; the remotes remain the per-paper homes. The compared family (Palomar `Challenge.lean` / `comparator.json`) is the 1980 ↔ 1982 coding, the round three-way identification, and the Factoid 8.1 instance; the other catalog rows are proved in `ScottModels/` but are not Comparator targets. This package was verified and registered in the Palomar registry as PALOMAR-2026-08-25-000003 v1 [4]. What Palomar verified is the claim:

> Order isomorphisms relating Scott's three presentations of domains — continuous lattices (1972), neighbourhood systems (PRG-19), and information systems (1982) — including the round identification `D ≃o RoundFilter ≃o RoundInfoSysElement` and the S-expression equation `T ≅ A + (T × T)` as `|T| ≃o |A + (T × T)|` (`SexSys.Element ≃o SexRhs.Element`).

This document (`view.pdf`) is the narrative account, not a formalization of the three papers. The module map below is left-to-right: sibling packages on the left, local `ScottModels/` modules on the right, with arrows for **direct** imports (transitive imports inside the siblings are omitted). `Equivalence` re-exports the bridges; only the `ScottMapBridge → Equivalence` edge is drawn to avoid clutter. Root `ScottModels.lean` also opens 1972 `ContinuousLattice.Specialization`, 1980 `Neighborhood.Basic`, and 1982 `InfoSys`.

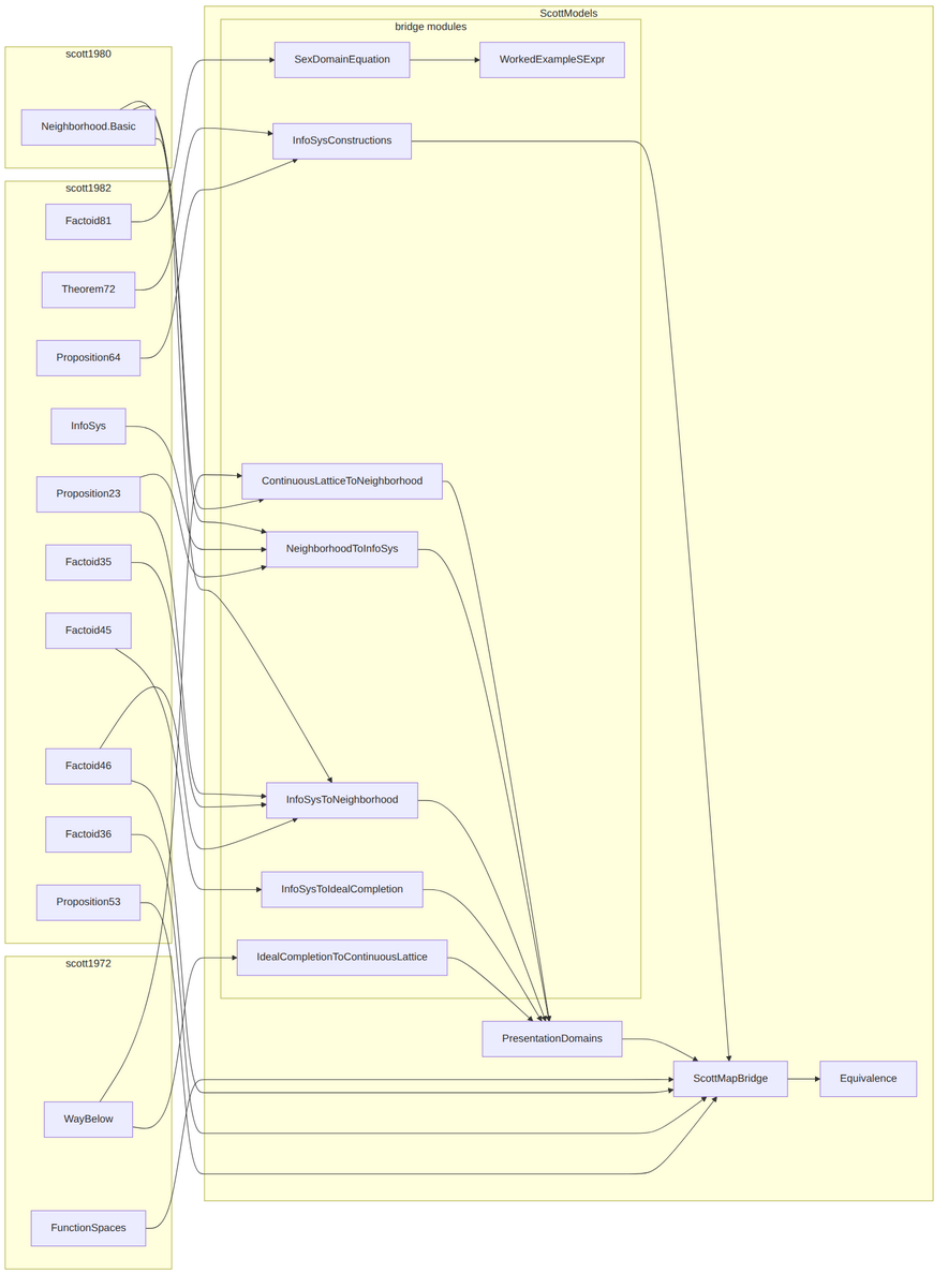


Figure 46: Lean module map.

5.3 Proof notes

5.3.1 Continuous lattices → neighbourhood systems

Claim. For a continuous lattice `D`, the sets `⇑a = {z | a ≪ z}` form a `NeighborhoodSystem` on token type `D`, and under `ContinuousLattice` one has an order isomorphism `D ≃o RoundFilter`.

Construction. `wayBelowUp a` is upward closed; `⇑⊥ = univ`; `⇑a ∩ ⇑b = ⇑(a ⊔ b)` (inter-

polation / directedness of way-below). Principal filters `toFilter x = {⇑a | a ≪ x}` are round: membership of ⇑a yields, by interpolation in a continuous lattice, some b with `a ≪ b ≪ x`, hence ⇑b `∈ toFilter x`.

Why raw $|\mathcal{D}|$ fails. The larger filter `{⇑a | a ≤ x}` still has `ofFilter = x` (supremum of codes), but is not equal to `toFilter x` whenever there are elements way-below strictly below x. Roundness (`⇑a ∈ f ⇒ ∃ b, a ≪ b ∧ ⇑b ∈ f`) cuts exactly to the image of `toFilter`.

Axioms. `domainOrderIso : D ≃o RoundFilter` audits to `{propext, Quot.sound}`.

5.3.2 Neighbourhood systems → information systems

Claim. A neighbourhood system equipped with a decidable exhaustive coding `NbhdBasis ι α` of its neighbourhood family $\mathcal{D}$ induces an `InfoSys` on tokens ι with $|\mathcal{D}| \simeq_o$ the InfoSys domain.

Construction. Tokens are neighbourhood indices. Consistency of a finite `u ⊆ ι` is membership of $\bigcap_{\{i \in u\}}$ `nbhd i` in $\mathcal{D}$ (empty intersection = master set Δ). Entailment `u ⊢ j` means the intersection is a neighbourhood contained in `nbhd j`. Filters of the neighbourhood system correspond to InfoSys elements via the coding.

Constructivity. `DecidableEq ι` is required so `InfoSys` can use the choice-free `Finset` prelude from `scott1982`. The proof of `ent_con` avoids classical `by_cases` / `em`. The maps extract no witnesses; the kernel audit still reports inherited `Classical.choice` from the Mathlib 4.33 `Finset` instance frontier (§4).

5.3.3 Information systems → neighbourhood systems

Claim. For an information system A, the basic opens `[u] = {x ∈ |A| | ↑u ⊆ x}` (`u ∈ Con`) form a `NeighborhoodSystem` on `|A|`, and filters recover elements: `|A| ≃o` the filter domain.

Proof note. Scott's Factoid 4.6 supplies the basic-open vocabulary. The Lean proof initially pulled `Classical.choice` through `simp` on `basicOpen_empty` / finset unions; those simps were removed so the construction extracts no witnesses. The kernel audit still reports inherited `Classical.choice` through `InfoSys/Finset` (§4). Together with §3.2 this is the **1980 ↔ 1982** equivalence under coding.

5.3.4 Information systems → ideal completion

Claim. `|A| ≃o Ideal (FiniteElement A)`, where finite elements are closures $\bar{u}$ of consistent finsets (`Factoid 3.5`).

Construction. `toIdeal x` is the ideal of finite approximants of x; `ofIdeal` takes directed suprema of finite elements (1982 Factoids 4.4–4.5: `directedSup`, `eq_directedSup_finiteApproximants`, `compact_closure`). The maps extract no witnesses; the kernel audit inherits `Classical.choice` through `InfoSys/Finset` (§4).

5.3.5 Algebraic complete lattices → continuous lattices

Claim. If every element of a complete lattice is the directed supremum of compact elements below it (`AlgebraicLattice`), then `ContinuousLattice` holds.

Proof note. Order-theoretic compactness implies `Set.Ici k` is Scott-open, hence `k ≪ y` whenever `k ≤ y`. Algebraicity then yields `y = ⊔{k compact | k ≤ y} ⊆ ⊔{x | x ≪ y}`. This is the **classical frontier**: Scott's ≪ is defined topologically in `scott1972`, so the footprint inherits that classicality even though the order argument is elementary.

5.3.6 Three-presentation equivalence (`presentation_domains_equiv`)

Claim. Under `ContinuousLattice D` and `DecidableEq D`,

```
D ≃o RoundFilter ≃o RoundInfoSysElement
```

where `RoundInfoSysElement` is the subtype of elements of the InfoSys coded by `wayBelowNbhdBasis` (tokens = elements of `D`, neighbourhoods = $\Uparrow$`a`) that correspond to round filters. An extended form routes through the ideal-completion subtype.

Glue. `wayBelowNbhdBasis` packages the $\Uparrow$-system as an `NbhdBasis`; `roundFilter_infoSys_iso` transports roundness along `NbhdBasis.domainOrderIso`; `presentation_domains_equiv` is the composite with `continuousLattice_roundFilter_iso`.

What is not claimed. Raw $|\mathcal{D}|$ and the full InfoSys domain `|A|` remain properly larger than `D`. The equivalence is on the **round** corner that matches continuous lattice points.

Axioms. Under Mathlib 4.33 the kernel audit includes `{propext, Classical.choice, Quot.sound}` through the inherited InfoSys/`Finset` instance frontier described in §4; the round-filter construction itself performs no choice-based witness extraction.

5.3.7 Constructions (`infoSys_constructions_equiv`)

Products `|A| × |B| ≃o |A × B|` via 1982 `pairElements` / `fstMap` / `sndMap` (Prop 6.2). The maps use explicit projections and assembly, with no witness selection.

Separated sums `WithBot (|A| ⊕ |B|) ≃o |A + B|` via `inl` / `inr` classify-and-assemble (Prop 6.4). Trichotomy on token polarity uses classical case-split (`Classical.choice` in the footprint).

Function spaces `ApproximableMap A B ≃o |A → B|` packages Theorem 7.2 (`approxMap_toElement` / `element_toApproxMap`) with pointwise `Le` as `PartialOrder`. No additional choice is used beyond the inherited InfoSys/`Finset` instance frontier.

Factoid 4.6 bridge `ApproximableMap A B ≃o ScottContinuous A B` via `toScottContinuous` / `ofScottContinuous`, using Prop 5.3(v) (`rel_iff_closure_le`) and `closure_le_element` for the round-trip on relations. The relation-level proof performs no witness extraction; its kernel audit inherits the InfoSys/`Finset` frontier in §4.

ScottMap conjugation A 1972 `ScottMap D E` conjugates along any pair of order isomorphisms ι`D : D ≃o D'`, ι`E : E ≃o E'` to a pointwise-ordered map `D' → E'`. Specializing to `continuousLattice_roundFilter_iso` / `presentation_domains_equiv` yields `scottMap_roundFilter_iso` and `scottMap_roundInfoSys_iso`. The conjugation itself is order-theoretic; the footprint includes `Classical.choice` because `ScottMap` is defined via Scott continuity in `scott1972`.

Out of scope (documented). Identifying `ApproximableMap` on the coded $\Uparrow$-InfoSys with `ScottMap D E` (needs roundness preservation of approximable maps), and relating `wayBelow (D × E)` to the InfoSys product of factors (cylinder basis).

`scottSubspaceExtend.lean`

```
module

public import DomainTheory.ContinuousLattice.Specialization.scottTopologicalSpace
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

/-- **Scott 1972, Proposition 3.8.** Scott's maximal subspace extension `f : Y → D`.
    -/
noncomputable def scottSubspaceExtend (e : X → Y) (f : X → D) (y : Y) : D :=
  sSup { d | ∃ U, @IsOpen Y scottTopologicalSpace U ∧ y ∈ U ∧ d =
    scottSubspaceExtendInf e f U }

end SubspaceExtension

end DomainTheory.ContinuousLattice
```

8

scottSubspaceExtend.lean

```
module

public import DomainTheory.ContinuousLattice.Specialization.scottTopologicalSpace
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
```

```
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

/-- **Scott 1972, Proposition 3.8.** Scott's maximal subspace extension `f : Y → D`.
    -/
noncomputable def scottSubspaceExtend (e : X → Y) (f : X → D) (y : Y) : D :=
  sSup { d | ∃ U, @IsOpen Y scottTopologicalSpace U ∧ y ∈ U ∧ d =
    scottSubspaceExtendInf e f U }

end SubspaceExtension

end DomainTheory.ContinuousLattice
```

9

scottSubspaceExtendInf.lean

```
module
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

/-- **Scott 1972, Proposition 3.8.** The infimum term `⊓ { f(x) : e(x) ∈ U }` for open
```

```
    `U ⊆ Y`. -/
noncomputable def scottSubspaceExtendInf (e : X → Y) (f : X → D) (U : Set Y) : D :=
  sInf (f '' (e ^-1' U))

end SubspaceExtension

end DomainTheory.ContinuousLattice
```

0

scottSubspaceExtendInf.lean

```
module
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

/-- **Scott 1972, Proposition 3.8.** The infimum term `⊓ { f(x) : e(x) ∈ U }` for open
    `U ⊆ Y`. -/
noncomputable def scottSubspaceExtendInf (e : X → Y) (f : X → D) (U : Set Y) : D :=
  sInf (f '' (e ^-1' U))

end SubspaceExtension

end DomainTheory.ContinuousLattice
```

1

scottSubspaceExtendInf.lean

```
module
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

/-- **Scott 1972, Proposition 3.8.** The infimum term `⊓ { f(x) : e(x) ∈ U }` for open
     `U ⊆ Y`. -/
noncomputable def scottSubspaceExtendInf (e : X → Y) (f : X → D) (U : Set Y) : D :=
  sInf (f '' (e ^-1' U))

end SubspaceExtension

end DomainTheory.ContinuousLattice
```

2

scottSubspaceExtendInf.lean

```
module
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
```

```
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

/-- **Scott 1972, Proposition 3.8.** The infimum term `⊓ { f(x) : e(x) ∈ U }` for open
     `U ⊆ Y`. -/
noncomputable def scottSubspaceExtendInf (e : X → Y) (f : X → D) (U : Set Y) : D :=
  sInf (f '' (e ^-1' U))

end SubspaceExtension

end DomainTheory.ContinuousLattice
```

3

scottSubspaceExtendInf.lean

```
module
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section
```

```
namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

/-- **Scott 1972, Proposition 3.8.** The infimum term `⊓ { f(x) : e(x) ∈ U }` for open
     `U ⊆ Y`. -/
noncomputable def scottSubspaceExtendInf (e : X → Y) (f : X → D) (U : Set Y) : D :=
  sInf (f '' (e ^-1' U))

end SubspaceExtension

end DomainTheory.ContinuousLattice
```

4

scottSubspaceExtendInf.lean

```
module
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]
```

```
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

/-- **Scott 1972, Proposition 3.8.** The infimum term `⊓ { f(x) : e(x) ∈ U }` for open
     `U ⊆ Y`. -/
noncomputable def scottSubspaceExtendInf (e : X → Y) (f : X → D) (U : Set Y) : D :=
  sInf (f '' (e ^-1' U))

end SubspaceExtension

end DomainTheory.ContinuousLattice
```

5

scottSubspaceExtendInf.lean

```
module
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]
```

```
/-- **Scott 1972, Proposition 3.8.** The infimum term `⊓ { f(x) : e(x) ∈ U }` for open
    `U ⊆ Y`. -/
noncomputable def scottSubspaceExtendInf (e : X → Y) (f : X → D) (U : Set Y) : D :=
  sInf (f '' (e ^-1' U))

end SubspaceExtension

end DomainTheory.ContinuousLattice
```

6

scottSubspaceExtendInf.lean

```
module
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

/-- **Scott 1972, Proposition 3.8.** The infimum term `⊓ { f(x) : e(x) ∈ U }` for open
    `U ⊆ Y`. -/
noncomputable def scottSubspaceExtendInf (e : X → Y) (f : X → D) (U : Set Y) : D :=
  sInf (f '' (e ^-1' U))

end SubspaceExtension

end DomainTheory.ContinuousLattice
```

7

scottSubspaceExtendInf.lean

```
module
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

/-- **Scott 1972, Proposition 3.8.** The infimum term `⊓ { f(x) : e(x) ∈ U }` for open
     `U ⊆ Y`. -/
noncomputable def scottSubspaceExtendInf (e : X → Y) (f : X → D) (U : Set Y) : D :=
  sInf (f '' (e ^-1' U))

end SubspaceExtension

end DomainTheory.ContinuousLattice
```

8

scottSubspaceExtendInf.lean

```
module
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
```

```
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
public import Mathlib.Order.ScottContinuity
public import Mathlib.Data.Finset.Lattice.Fold
public import Mathlib.Topology.ContinuousMap.Basic
public import Mathlib.Topology.Order
public import Mathlib.Order.FixedPoints

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

/-- **Scott 1972, Proposition 3.8.** The infimum term `⊓ { f(x) : e(x) ∈ U }` for open
     `U ⊆ Y`. -/
noncomputable def scottSubspaceExtendInf (e : X → Y) (f : X → D) (U : Set Y) : D :=
  sInf (f '' (e ^-1' U))

end SubspaceExtension

end DomainTheory.ContinuousLattice
```

9

scottSubspaceExtendInf_eq_of_comp.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
```

```
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] in
theorem scottSubspaceExtendInf_eq_of_comp {e : X → Y} {f : X → D} {U : Set Y}
    (f' : ScottMap Y D) (h : ∀ x, f' (e x) = f x) :
    scottSubspaceExtendInf e f U = sInf (Set.image (f' : Y → D) (Set.image e Set.univ
    ∩ U)) := by
  have hset : f '' (e ^-1' U) = Set.image (f' : Y → D) (Set.image e Set.univ ∩ U) :=
    by
    ext d
    simp only [Set.mem_image, Set.mem_preimage, Set.mem_inter_iff, Set.mem_univ,
    true_and]
    constructor
    · rintro ⟨x, hxU, heq⟩
      refine ⟨e x, ⟨?_, hxU⟩, ?_⟩
      · exact ⟨x, rfl⟩
      · rw [h x, heq]
    · rintro ⟨y, ⟨hex, hyU⟩, heq⟩
      obtain ⟨x, _, rfl⟩ := hex
      exact ⟨x, hyU, (h x).symm.trans heq⟩
  rw [scottSubspaceExtendInf, hset]

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

0

scottSubspaceExtendInf_eq_of_comp.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]
```

```
omit [TopologicalSpace X] [TopologicalSpace Y] in
theorem scottSubspaceExtendInf_eq_of_comp {e : X → Y} {f : X → D} {U : Set Y}
    (f' : ScottMap Y D) (h : ∀ x, f' (e x) = f x) :
    scottSubspaceExtendInf e f U = sInf (Set.image (f' : Y → D) (Set.image e Set.univ
    ∩ U)) := by
  have hset : f '' (e ^-1' U) = Set.image (f' : Y → D) (Set.image e Set.univ ∩ U) :=
    by
    ext d
    simp only [Set.mem_image, Set.mem_preimage, Set.mem_inter_iff, Set.mem_univ,
    true_and]
    constructor
    · rintro ⟨x, hxU, heq⟩
      refine ⟨e x, ⟨?_, hxU⟩, ?_⟩
      · exact ⟨x, rfl⟩
      · rw [h x, heq]
    · rintro ⟨y, ⟨hex, hyU⟩, heq⟩
      obtain ⟨x, _, rfl⟩ := hex
      exact ⟨x, hyU, (h x).symm.trans heq⟩
  rw [scottSubspaceExtendInf, hset]

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

1

scottSubspaceExtendInf_eq_of_comp.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] in
theorem scottSubspaceExtendInf_eq_of_comp {e : X → Y} {f : X → D} {U : Set Y}
    (f' : ScottMap Y D) (h : ∀ x, f' (e x) = f x) :
    scottSubspaceExtendInf e f U = sInf (Set.image (f' : Y → D) (Set.image e Set.univ
    ∩ U)) := by
  have hset : f '' (e ^-1' U) = Set.image (f' : Y → D) (Set.image e Set.univ ∩ U) :=
```

```
    by
    ext d
    simp only [Set.mem_image, Set.mem_preimage, Set.mem_inter_iff, Set.mem_univ,
    true_and]
    constructor
    · rintro ⟨x, hxU, heq⟩
      refine ⟨e x, ⟨?_, hxU⟩, ?_⟩
      · exact ⟨x, rfl⟩
      · rw [h x, heq]
    · rintro ⟨y, ⟨hex, hyU⟩, heq⟩
      obtain ⟨x, _, rfl⟩ := hex
      exact ⟨x, hyU, (h x).symm.trans heq⟩
  rw [scottSubspaceExtendInf, hset]

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

2

scottSubspaceExtendInf_eq_of_comp.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] in
theorem scottSubspaceExtendInf_eq_of_comp {e : X → Y} {f : X → D} {U : Set Y}
    (f' : ScottMap Y D) (h : ∀ x, f' (e x) = f x) :
    scottSubspaceExtendInf e f U = sInf (Set.image (f' : Y → D) (Set.image e Set.univ
    ∩ U)) := by
  have hset : f '' (e ^-1' U) = Set.image (f' : Y → D) (Set.image e Set.univ ∩ U) :=
    by
    ext d
    simp only [Set.mem_image, Set.mem_preimage, Set.mem_inter_iff, Set.mem_univ,
    true_and]
    constructor
    · rintro ⟨x, hxU, heq⟩
      refine ⟨e x, ⟨?_, hxU⟩, ?_⟩
```

```
      · exact ⟨x, rfl⟩
      · rw [h x, heq]
    · rintro ⟨y, ⟨hex, hyU⟩, heq⟩
      obtain ⟨x, _, rfl⟩ := hex
      exact ⟨x, hyU, (h x).symm.trans heq⟩
  rw [scottSubspaceExtendInf, hset]

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

3

scottSubspaceExtendInf_eq_of_comp.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] in
theorem scottSubspaceExtendInf_eq_of_comp {e : X → Y} {f : X → D} {U : Set Y}
    (f' : ScottMap Y D) (h : ∀ x, f' (e x) = f x) :
    scottSubspaceExtendInf e f U = sInf (Set.image (f' : Y → D) (Set.image e Set.univ
    ∩ U)) := by
  have hset : f '' (e ^-1' U) = Set.image (f' : Y → D) (Set.image e Set.univ ∩ U) :=
    by
    ext d
    simp only [Set.mem_image, Set.mem_preimage, Set.mem_inter_iff, Set.mem_univ,
    true_and]
    constructor
    · rintro ⟨x, hxU, heq⟩
      refine ⟨e x, ⟨?_, hxU⟩, ?_⟩
      · exact ⟨x, rfl⟩
      · rw [h x, heq]
    · rintro ⟨y, ⟨hex, hyU⟩, heq⟩
      obtain ⟨x, _, rfl⟩ := hex
      exact ⟨x, hyU, (h x).symm.trans heq⟩
  rw [scottSubspaceExtendInf, hset]
```

```
end SubspaceExtension
end DomainTheory.ContinuousLattice
```

4

scottSubspaceExtendInf_eq_of_comp.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] in
theorem scottSubspaceExtendInf_eq_of_comp {e : X → Y} {f : X → D} {U : Set Y}
    (f' : ScottMap Y D) (h : ∀ x, f' (e x) = f x) :
    scottSubspaceExtendInf e f U = sInf (Set.image (f' : Y → D) (Set.image e Set.univ
    ∩ U)) := by
  have hset : f '' (e ^-1' U) = Set.image (f' : Y → D) (Set.image e Set.univ ∩ U) :=
    by
    ext d
    simp only [Set.mem_image, Set.mem_preimage, Set.mem_inter_iff, Set.mem_univ,
    true_and]
    constructor
    · rintro ⟨x, hxU, heq⟩
      refine ⟨e x, ⟨?_, hxU⟩, ?_⟩
      · exact ⟨x, rfl⟩
      · rw [h x, heq]
    · rintro ⟨y, ⟨hex, hyU⟩, heq⟩
      obtain ⟨x, _, rfl⟩ := hex
      exact ⟨x, hyU, (h x).symm.trans heq⟩
  rw [scottSubspaceExtendInf, hset]

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

5

scottSubspaceExtendInf_eq_of_comp.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] in
theorem scottSubspaceExtendInf_eq_of_comp {e : X → Y} {f : X → D} {U : Set Y}
    (f' : ScottMap Y D) (h : ∀ x, f' (e x) = f x) :
    scottSubspaceExtendInf e f U = sInf (Set.image (f' : Y → D) (Set.image e Set.univ
    ∩ U)) := by
  have hset : f '' (e ^-1' U) = Set.image (f' : Y → D) (Set.image e Set.univ ∩ U) :=
    by
    ext d
    simp only [Set.mem_image, Set.mem_preimage, Set.mem_inter_iff, Set.mem_univ,
    true_and]
    constructor
    · rintro ⟨x, hxU, heq⟩
      refine ⟨e x, ⟨?_, hxU⟩, ?_⟩
      · exact ⟨x, rfl⟩
      · rw [h x, heq]
    · rintro ⟨y, ⟨hex, hyU⟩, heq⟩
      obtain ⟨x, _, rfl⟩ := hex
      exact ⟨x, hyU, (h x).symm.trans heq⟩
  rw [scottSubspaceExtendInf, hset]

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

6

scottSubspaceExtendInf_eq_of_comp.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section
```

```
namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] in
theorem scottSubspaceExtendInf_eq_of_comp {e : X → Y} {f : X → D} {U : Set Y}
    (f' : ScottMap Y D) (h : ∀ x, f' (e x) = f x) :
    scottSubspaceExtendInf e f U = sInf (Set.image (f' : Y → D) (Set.image e Set.univ
    ∩ U)) := by
  have hset : f '' (e ^-1' U) = Set.image (f' : Y → D) (Set.image e Set.univ ∩ U) :=
    by
    ext d
    simp only [Set.mem_image, Set.mem_preimage, Set.mem_inter_iff, Set.mem_univ,
    true_and]
    constructor
    · rintro ⟨x, hxU, heq⟩
      refine ⟨e x, ⟨?_, hxU⟩, ?_⟩
      · exact ⟨x, rfl⟩
      · rw [h x, heq]
    · rintro ⟨y, ⟨hex, hyU⟩, heq⟩
      obtain ⟨x, _, rfl⟩ := hex
      exact ⟨x, hyU, (h x).symm.trans heq⟩
  rw [scottSubspaceExtendInf, hset]

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

7

scottSubspaceExtendInf_eq_of_comp.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
```

```
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] in
theorem scottSubspaceExtendInf_eq_of_comp {e : X → Y} {f : X → D} {U : Set Y}
    (f' : ScottMap Y D) (h : ∀ x, f' (e x) = f x) :
    scottSubspaceExtendInf e f U = sInf (Set.image (f' : Y → D) (Set.image e Set.univ
    ∩ U)) := by
  have hset : f '' (e ^-1' U) = Set.image (f' : Y → D) (Set.image e Set.univ ∩ U) :=
    by
    ext d
    simp only [Set.mem_image, Set.mem_preimage, Set.mem_inter_iff, Set.mem_univ,
    true_and]
    constructor
    · rintro ⟨x, hxU, heq⟩
      refine ⟨e x, ⟨?_, hxU⟩, ?_⟩
      · exact ⟨x, rfl⟩
      · rw [h x, heq]
    · rintro ⟨y, ⟨hex, hyU⟩, heq⟩
      obtain ⟨x, _, rfl⟩ := hex
      exact ⟨x, hyU, (h x).symm.trans heq⟩
  rw [scottSubspaceExtendInf, hset]

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

8

scottSubspaceExtendInf_eq_of_comp.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]
```

```
omit [TopologicalSpace X] [TopologicalSpace Y] in
theorem scottSubspaceExtendInf_eq_of_comp {e : X → Y} {f : X → D} {U : Set Y}
    (f' : ScottMap Y D) (h : ∀ x, f' (e x) = f x) :
    scottSubspaceExtendInf e f U = sInf (Set.image (f' : Y → D) (Set.image e Set.univ
    ∩ U)) := by
  have hset : f '' (e ^-1' U) = Set.image (f' : Y → D) (Set.image e Set.univ ∩ U) :=
    by
    ext d
    simp only [Set.mem_image, Set.mem_preimage, Set.mem_inter_iff, Set.mem_univ,
    true_and]
    constructor
    · rintro ⟨x, hxU, heq⟩
      refine ⟨e x, ⟨?_, hxU⟩, ?_⟩
      · exact ⟨x, rfl⟩
      · rw [h x, heq]
    · rintro ⟨y, ⟨hex, hyU⟩, heq⟩
      obtain ⟨x, _, rfl⟩ := hex
      exact ⟨x, hyU, (h x).symm.trans heq⟩
  rw [scottSubspaceExtendInf, hset]

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

9

scottSubspaceExtendInf_eq_of_ext.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y] in
theorem scottSubspaceExtendInf_eq_of_ext {e : X → Y} {f g' : X → D} (h : ∀ x, f x = g
    ' x) (U : Set Y) :
    scottSubspaceExtendInf e f U = scottSubspaceExtendInf e g' U := by
  unfold scottSubspaceExtendInf
  congr 1
  ext d
```

```
  simp [Set.mem_image, Set.mem_preimage, h]

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

0

scottSubspaceExtendInf_eq_of_ext.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y] in
theorem scottSubspaceExtendInf_eq_of_ext {e : X → Y} {f g' : X → D} (h : ∀ x, f x = g
    ' x) (U : Set Y) :
    scottSubspaceExtendInf e f U = scottSubspaceExtendInf e g' U := by
  unfold scottSubspaceExtendInf
  congr 1
  ext d
  simp [Set.mem_image, Set.mem_preimage, h]

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

1

scottSubspaceExtendInf_eq_of_ext.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace
```

```
variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y] in
theorem scottSubspaceExtendInf_eq_of_ext {e : X → Y} {f g' : X → D} (h : ∀ x, f x = g
    ' x) (U : Set Y) :
    scottSubspaceExtendInf e f U = scottSubspaceExtendInf e g' U := by
  unfold scottSubspaceExtendInf
  congr 1
  ext d
  simp [Set.mem_image, Set.mem_preimage, h]

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

2

scottSubspaceExtendInf_eq_of_ext.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y] in
theorem scottSubspaceExtendInf_eq_of_ext {e : X → Y} {f g' : X → D} (h : ∀ x, f x = g
    ' x) (U : Set Y) :
    scottSubspaceExtendInf e f U = scottSubspaceExtendInf e g' U := by
  unfold scottSubspaceExtendInf
  congr 1
  ext d
  simp [Set.mem_image, Set.mem_preimage, h]
```

```
end SubspaceExtension
end DomainTheory.ContinuousLattice
```

3

scottSubspaceExtendInf_eq_of_ext.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y] in
theorem scottSubspaceExtendInf_eq_of_ext {e : X → Y} {f g' : X → D} (h : ∀ x, f x = g
    ' x) (U : Set Y) :
    scottSubspaceExtendInf e f U = scottSubspaceExtendInf e g' U := by
  unfold scottSubspaceExtendInf
  congr 1
  ext d
  simp [Set.mem_image, Set.mem_preimage, h]

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

4

scottSubspaceExtendInf_eq_of_ext.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace
```

```
variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y] in
theorem scottSubspaceExtendInf_eq_of_ext {e : X → Y} {f g' : X → D} (h : ∀ x, f x = g
    ' x) (U : Set Y) :
    scottSubspaceExtendInf e f U = scottSubspaceExtendInf e g' U := by
  unfold scottSubspaceExtendInf
  congr 1
  ext d
  simp [Set.mem_image, Set.mem_preimage, h]

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

5

scottSubspaceExtendInf_eq_of_ext.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y] in
theorem scottSubspaceExtendInf_eq_of_ext {e : X → Y} {f g' : X → D} (h : ∀ x, f x = g
    ' x) (U : Set Y) :
    scottSubspaceExtendInf e f U = scottSubspaceExtendInf e g' U := by
  unfold scottSubspaceExtendInf
  congr 1
  ext d
  simp [Set.mem_image, Set.mem_preimage, h]
```

```
end SubspaceExtension
end DomainTheory.ContinuousLattice
```

6

scottSubspaceExtendInf_eq_of_ext.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y] in
theorem scottSubspaceExtendInf_eq_of_ext {e : X → Y} {f g' : X → D} (h : ∀ x, f x = g
    ' x) (U : Set Y) :
    scottSubspaceExtendInf e f U = scottSubspaceExtendInf e g' U := by
  unfold scottSubspaceExtendInf
  congr 1
  ext d
  simp [Set.mem_image, Set.mem_preimage, h]

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

7

scottSubspaceExtendInf_eq_of_ext.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
```

```
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y] in
theorem scottSubspaceExtendInf_eq_of_ext {e : X → Y} {f g' : X → D} (h : ∀ x, f x = g
    ' x) (U : Set Y) :
    scottSubspaceExtendInf e f U = scottSubspaceExtendInf e g' U := by
  unfold scottSubspaceExtendInf
  congr 1
  ext d
  simp [Set.mem_image, Set.mem_preimage, h]

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

8

scottSubspaceExtendInf_eq_of_ext.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y] in
theorem scottSubspaceExtendInf_eq_of_ext {e : X → Y} {f g' : X → D} (h : ∀ x, f x = g
    ' x) (U : Set Y) :
    scottSubspaceExtendInf e f U = scottSubspaceExtendInf e g' U := by
  unfold scottSubspaceExtendInf
  congr 1
  ext d
  simp [Set.mem_image, Set.mem_preimage, h]

end SubspaceExtension
```

```
end DomainTheory.ContinuousLattice
```

9

scottSubspaceExtendInf_le_of_mem.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace Y] in
theorem scottSubspaceExtendInf_le_of_mem {f' : ScottMap Y D} {U : Set Y} {y : Y}
    (hyU : y ∈ U) : sInf (Set.image (f' : Y → D) U) ≤ f' y :=
  sInf_le (Set.mem_image_of_mem _ hyU)

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

0

scottSubspaceExtendInf_le_of_mem.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
```

```
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace Y] in
theorem scottSubspaceExtendInf_le_of_mem {f' : ScottMap Y D} {U : Set Y} {y : Y}
    (hyU : y ∈ U) : sInf (Set.image (f' : Y → D) U) ≤ f' y :=
  sInf_le (Set.mem_image_of_mem _ hyU)

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

1

scottSubspaceExtendInf_le_of_mem.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace Y] in
theorem scottSubspaceExtendInf_le_of_mem {f' : ScottMap Y D} {U : Set Y} {y : Y}
    (hyU : y ∈ U) : sInf (Set.image (f' : Y → D) U) ≤ f' y :=
  sInf_le (Set.mem_image_of_mem _ hyU)

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

2

scottSubspaceExtendInf_le_of_mem.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Order.BourbakiWitt
```

```
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace Y] in
theorem scottSubspaceExtendInf_le_of_mem {f' : ScottMap Y D} {U : Set Y} {y : Y}
    (hyU : y ∈ U) : sInf (Set.image (f' : Y → D) U) ≤ f' y :=
  sInf_le (Set.mem_image_of_mem _ hyU)

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

3

scottSubspaceExtendInf_le_of_mem.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace Y] in
theorem scottSubspaceExtendInf_le_of_mem {f' : ScottMap Y D} {U : Set Y} {y : Y}
    (hyU : y ∈ U) : sInf (Set.image (f' : Y → D) U) ≤ f' y :=
```

```
  sInf_le (Set.mem_image_of_mem _ hyU)

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

4

scottSubspaceExtendInf_le_of_mem.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace Y] in
theorem scottSubspaceExtendInf_le_of_mem {f' : ScottMap Y D} {U : Set Y} {y : Y}
    (hyU : y ∈ U) : sInf (Set.image (f' : Y → D) U) ≤ f' y :=
  sInf_le (Set.mem_image_of_mem _ hyU)

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

5

scottSubspaceExtendInf_le_of_mem.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
```

```
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace Y] in
theorem scottSubspaceExtendInf_le_of_mem {f' : ScottMap Y D} {U : Set Y} {y : Y}
    (hyU : y ∈ U) : sInf (Set.image (f' : Y → D) U) ≤ f' y :=
  sInf_le (Set.mem_image_of_mem _ hyU)

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

6

scottSubspaceExtendInf_le_of_mem.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace Y] in
theorem scottSubspaceExtendInf_le_of_mem {f' : ScottMap Y D} {U : Set Y} {y : Y}
    (hyU : y ∈ U) : sInf (Set.image (f' : Y → D) U) ≤ f' y :=
  sInf_le (Set.mem_image_of_mem _ hyU)

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

7

scottSubspaceExtendInf_le_of_mem.lean

```
module
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace Y] in
theorem scottSubspaceExtendInf_le_of_mem {f' : ScottMap Y D} {U : Set Y} {y : Y}
    (hyU : y ∈ U) : sInf (Set.image (f' : Y → D) U) ≤ f' y :=
  sInf_le (Set.mem_image_of_mem _ hyU)

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

8

scottSubspaceExtendInf_le_of_mem.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace Y] in
```

```
theorem scottSubspaceExtendInf_le_of_mem {f' : ScottMap Y D} {U : Set Y} {y : Y}
    (hyU : y ∈ U) : sInf (Set.image (f' : Y → D) U) ≤ f' y :=
  sInf_le (Set.mem_image_of_mem _ hyU)

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

9

scottSubspaceExtendInf_mono.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y] in
theorem scottSubspaceExtendInf_mono {e : X → Y} {f g₀ : X → D} (hfg : ∀ x, f x ≤ g₀ x
    ) (U : Set Y) :
    scottSubspaceExtendInf e f U ≤ scottSubspaceExtendInf e g₀ U := by
  apply le_sInf
  intro d hd
  obtain ⟨x, hxU, rfl⟩ := hd
  exact le_trans (sInf_le ⟨x, hxU, rfl⟩) (hfg x)

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

0

scottSubspaceExtendInf_mono.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section
```

```
namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y] in
theorem scottSubspaceExtendInf_mono {e : X → Y} {f g₀ : X → D} (hfg : ∀ x, f x ≤ g₀ x
    ) (U : Set Y) :
    scottSubspaceExtendInf e f U ≤ scottSubspaceExtendInf e g₀ U := by
  apply le_sInf
  intro d hd
  obtain ⟨x, hxU, rfl⟩ := hd
  exact le_trans (sInf_le ⟨x, hxU, rfl⟩) (hfg x)

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

1

scottSubspaceExtendInf_mono.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y] in
theorem scottSubspaceExtendInf_mono {e : X → Y} {f g₀ : X → D} (hfg : ∀ x, f x ≤ g₀ x
    ) (U : Set Y) :
```

```
    scottSubspaceExtendInf e f U ≤ scottSubspaceExtendInf e g₀ U := by
  apply le_sInf
  intro d hd
  obtain ⟨x, hxU, rfl⟩ := hd
  exact le_trans (sInf_le ⟨x, hxU, rfl⟩) (hfg x)

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

2

scottSubspaceExtendInf_mono.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y] in
theorem scottSubspaceExtendInf_mono {e : X → Y} {f g₀ : X → D} (hfg : ∀ x, f x ≤ g₀ x
    ) (U : Set Y) :
    scottSubspaceExtendInf e f U ≤ scottSubspaceExtendInf e g₀ U := by
  apply le_sInf
  intro d hd
  obtain ⟨x, hxU, rfl⟩ := hd
  exact le_trans (sInf_le ⟨x, hxU, rfl⟩) (hfg x)

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

3

scottSubspaceExtendInf_mono.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic
```

```
@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y] in
theorem scottSubspaceExtendInf_mono {e : X → Y} {f g₀ : X → D} (hfg : ∀ x, f x ≤ g₀ x
    ) (U : Set Y) :
    scottSubspaceExtendInf e f U ≤ scottSubspaceExtendInf e g₀ U := by
  apply le_sInf
  intro d hd
  obtain ⟨x, hxU, rfl⟩ := hd
  exact le_trans (sInf_le ⟨x, hxU, rfl⟩) (hfg x)

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

4

scottSubspaceExtendInf_mono.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y] in
```

```
theorem scottSubspaceExtendInf_mono {e : X → Y} {f g₀ : X → D} (hfg : ∀ x, f x ≤ g₀ x
    ) (U : Set Y) :
    scottSubspaceExtendInf e f U ≤ scottSubspaceExtendInf e g₀ U := by
  apply le_sInf
  intro d hd
  obtain ⟨x, hxU, rfl⟩ := hd
  exact le_trans (sInf_le ⟨x, hxU, rfl⟩) (hfg x)

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

5

scottSubspaceExtendInf_mono.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y] in
theorem scottSubspaceExtendInf_mono {e : X → Y} {f g₀ : X → D} (hfg : ∀ x, f x ≤ g₀ x
    ) (U : Set Y) :
    scottSubspaceExtendInf e f U ≤ scottSubspaceExtendInf e g₀ U := by
  apply le_sInf
  intro d hd
  obtain ⟨x, hxU, rfl⟩ := hd
  exact le_trans (sInf_le ⟨x, hxU, rfl⟩) (hfg x)

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

6

scottSubspaceExtendInf_mono.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import Mathlib.Order.BourbakiWitt
```

```
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y] in
theorem scottSubspaceExtendInf_mono {e : X → Y} {f g₀ : X → D} (hfg : ∀ x, f x ≤ g₀ x
    ) (U : Set Y) :
    scottSubspaceExtendInf e f U ≤ scottSubspaceExtendInf e g₀ U := by
  apply le_sInf
  intro d hd
  obtain ⟨x, hxU, rfl⟩ := hd
  exact le_trans (sInf_le ⟨x, hxU, rfl⟩) (hfg x)

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

7

scottSubspaceExtendInf_mono.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]
```

```
omit [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y] in
theorem scottSubspaceExtendInf_mono {e : X → Y} {f g₀ : X → D} (hfg : ∀ x, f x ≤ g₀ x
    ) (U : Set Y) :
    scottSubspaceExtendInf e f U ≤ scottSubspaceExtendInf e g₀ U := by
  apply le_sInf
  intro d hd
  obtain ⟨x, hxU, rfl⟩ := hd
  exact le_trans (sInf_le ⟨x, hxU, rfl⟩) (hfg x)

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

8

scottSubspaceExtendInf_mono.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y] in
theorem scottSubspaceExtendInf_mono {e : X → Y} {f g₀ : X → D} (hfg : ∀ x, f x ≤ g₀ x
    ) (U : Set Y) :
    scottSubspaceExtendInf e f U ≤ scottSubspaceExtendInf e g₀ U := by
  apply le_sInf
  intro d hd
  obtain ⟨x, hxU, rfl⟩ := hd
  exact le_trans (sInf_le ⟨x, hxU, rfl⟩) (hfg x)

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

9

scottSubspaceExtendInf_mono_incl.lean

```
module
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y] in
theorem scottSubspaceExtendInf_mono_incl {e : X → Y} {f : X → D} {g : X → D'}
    (P : IsContinuousLatticeProjection D D') (hfg : ∀ x, f x = P.retr (g x)) (U : Set Y
    ) :
    scottSubspaceExtendInf e (fun x => P.incl (f x)) U ≤ scottSubspaceExtendInf e g U
    := by
  apply le_sInf
  intro d hd
  obtain ⟨x, hxU, rfl⟩ := hd
  have hx : x ∈ e ^-1' U := Set.mem_preimage.mpr hxU
  have hmem : P.incl (f x) ∈ (fun x => P.incl (f x)) '' (e ^-1' U) :=
    Set.mem_image_of_mem _ hx
  have hincl : P.incl (f x) ≤ g x := by rw [hfg x]; exact P.incl_retr_le (g x)
  exact le_trans (sInf_le hmem) hincl

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

0

scottSubspaceExtendInf_mono_incl.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace
```

```
variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y] in
theorem scottSubspaceExtendInf_mono_incl {e : X → Y} {f : X → D} {g : X → D'}
    (P : IsContinuousLatticeProjection D D') (hfg : ∀ x, f x = P.retr (g x)) (U : Set Y
    ) :
    scottSubspaceExtendInf e (fun x => P.incl (f x)) U ≤ scottSubspaceExtendInf e g U
    := by
  apply le_sInf
  intro d hd
  obtain ⟨x, hxU, rfl⟩ := hd
  have hx : x ∈ e ^-1' U := Set.mem_preimage.mpr hxU
  have hmem : P.incl (f x) ∈ (fun x => P.incl (f x)) '' (e ^-1' U) :=
    Set.mem_image_of_mem _ hx
  have hincl : P.incl (f x) ≤ g x := by rw [hfg x]; exact P.incl_retr_le (g x)
  exact le_trans (sInf_le hmem) hincl

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

1

scottSubspaceExtendInf_mono_incl.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]
```

```
omit [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y] in
theorem scottSubspaceExtendInf_mono_incl {e : X → Y} {f : X → D} {g : X → D'}
    (P : IsContinuousLatticeProjection D D') (hfg : ∀ x, f x = P.retr (g x)) (U : Set Y
    ) :
    scottSubspaceExtendInf e (fun x => P.incl (f x)) U ≤ scottSubspaceExtendInf e g U
    := by
  apply le_sInf
  intro d hd
  obtain ⟨x, hxU, rfl⟩ := hd
  have hx : x ∈ e ^-1' U := Set.mem_preimage.mpr hxU
  have hmem : P.incl (f x) ∈ (fun x => P.incl (f x)) '' (e ^-1' U) :=
    Set.mem_image_of_mem _ hx
  have hincl : P.incl (f x) ≤ g x := by rw [hfg x]; exact P.incl_retr_le (g x)
  exact le_trans (sInf_le hmem) hincl

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

2

scottSubspaceExtendInf_mono_incl.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y] in
theorem scottSubspaceExtendInf_mono_incl {e : X → Y} {f : X → D} {g : X → D'}
    (P : IsContinuousLatticeProjection D D') (hfg : ∀ x, f x = P.retr (g x)) (U : Set Y
    ) :
    scottSubspaceExtendInf e (fun x => P.incl (f x)) U ≤ scottSubspaceExtendInf e g U
    := by
  apply le_sInf
  intro d hd
  obtain ⟨x, hxU, rfl⟩ := hd
  have hx : x ∈ e ^-1' U := Set.mem_preimage.mpr hxU
  have hmem : P.incl (f x) ∈ (fun x => P.incl (f x)) '' (e ^-1' U) :=
```

```
    Set.mem_image_of_mem _ hx
  have hincl : P.incl (f x) ≤ g x := by rw [hfg x]; exact P.incl_retr_le (g x)
  exact le_trans (sInf_le hmem) hincl

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

3

scottSubspaceExtendInf_mono_incl.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y] in
theorem scottSubspaceExtendInf_mono_incl {e : X → Y} {f : X → D} {g : X → D'}
    (P : IsContinuousLatticeProjection D D') (hfg : ∀ x, f x = P.retr (g x)) (U : Set Y
    ) :
    scottSubspaceExtendInf e (fun x => P.incl (f x)) U ≤ scottSubspaceExtendInf e g U
    := by
  apply le_sInf
  intro d hd
  obtain ⟨x, hxU, rfl⟩ := hd
  have hx : x ∈ e ^-1' U := Set.mem_preimage.mpr hxU
  have hmem : P.incl (f x) ∈ (fun x => P.incl (f x)) '' (e ^-1' U) :=
    Set.mem_image_of_mem _ hx
  have hincl : P.incl (f x) ≤ g x := by rw [hfg x]; exact P.incl_retr_le (g x)
  exact le_trans (sInf_le hmem) hincl

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

4

scottSubspaceExtendInf_mono_incl.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y] in
theorem scottSubspaceExtendInf_mono_incl {e : X → Y} {f : X → D} {g : X → D'}
    (P : IsContinuousLatticeProjection D D') (hfg : ∀ x, f x = P.retr (g x)) (U : Set Y
    ) :
    scottSubspaceExtendInf e (fun x => P.incl (f x)) U ≤ scottSubspaceExtendInf e g U
    := by
  apply le_sInf
  intro d hd
  obtain ⟨x, hxU, rfl⟩ := hd
  have hx : x ∈ e ^-1' U := Set.mem_preimage.mpr hxU
  have hmem : P.incl (f x) ∈ (fun x => P.incl (f x)) '' (e ^-1' U) :=
    Set.mem_image_of_mem _ hx
  have hincl : P.incl (f x) ≤ g x := by rw [hfg x]; exact P.incl_retr_le (g x)
  exact le_trans (sInf_le hmem) hincl

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

5

scottSubspaceExtendInf_mono_incl.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice
```

```
open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y] in
theorem scottSubspaceExtendInf_mono_incl {e : X → Y} {f : X → D} {g : X → D'}
    (P : IsContinuousLatticeProjection D D') (hfg : ∀ x, f x = P.retr (g x)) (U : Set Y
    ) :
    scottSubspaceExtendInf e (fun x => P.incl (f x)) U ≤ scottSubspaceExtendInf e g U
    := by
  apply le_sInf
  intro d hd
  obtain ⟨x, hxU, rfl⟩ := hd
  have hx : x ∈ e ^-1' U := Set.mem_preimage.mpr hxU
  have hmem : P.incl (f x) ∈ (fun x => P.incl (f x)) '' (e ^-1' U) :=
    Set.mem_image_of_mem _ hx
  have hincl : P.incl (f x) ≤ g x := by rw [hfg x]; exact P.incl_retr_le (g x)
  exact le_trans (sInf_le hmem) hincl

end SubspaceExtension
end DomainTheory.ContinuousLattice
```


scottSubspaceExtendInf_mono_incl.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]
```

```
omit [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y] in
theorem scottSubspaceExtendInf_mono_incl {e : X → Y} {f : X → D} {g : X → D'}
    (P : IsContinuousLatticeProjection D D') (hfg : ∀ x, f x = P.retr (g x)) (U : Set Y
    ) :
    scottSubspaceExtendInf e (fun x => P.incl (f x)) U ≤ scottSubspaceExtendInf e g U
    := by
  apply le_sInf
  intro d hd
  obtain ⟨x, hxU, rfl⟩ := hd
  have hx : x ∈ e ^-1' U := Set.mem_preimage.mpr hxU
  have hmem : P.incl (f x) ∈ (fun x => P.incl (f x)) '' (e ^-1' U) :=
    Set.mem_image_of_mem _ hx
  have hincl : P.incl (f x) ≤ g x := by rw [hfg x]; exact P.incl_retr_le (g x)
  exact le_trans (sInf_le hmem) hincl

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

7

scottSubspaceExtendInf_mono_incl.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y] in
theorem scottSubspaceExtendInf_mono_incl {e : X → Y} {f : X → D} {g : X → D'}
    (P : IsContinuousLatticeProjection D D') (hfg : ∀ x, f x = P.retr (g x)) (U : Set Y
    ) :
    scottSubspaceExtendInf e (fun x => P.incl (f x)) U ≤ scottSubspaceExtendInf e g U
    := by
  apply le_sInf
  intro d hd
  obtain ⟨x, hxU, rfl⟩ := hd
  have hx : x ∈ e ^-1' U := Set.mem_preimage.mpr hxU
```

```
  have hmem : P.incl (f x) ∈ (fun x => P.incl (f x)) '' (e ^-1' U) :=
    Set.mem_image_of_mem _ hx
  have hincl : P.incl (f x) ≤ g x := by rw [hfg x]; exact P.incl_retr_le (g x)
  exact le_trans (sInf_le hmem) hincl

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

8

scottSubspaceExtendInf_mono_incl.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y] in
theorem scottSubspaceExtendInf_mono_incl {e : X → Y} {f : X → D} {g : X → D'}
    (P : IsContinuousLatticeProjection D D') (hfg : ∀ x, f x = P.retr (g x)) (U : Set Y
    ) :
    scottSubspaceExtendInf e (fun x => P.incl (f x)) U ≤ scottSubspaceExtendInf e g U
    := by
  apply le_sInf
  intro d hd
  obtain ⟨x, hxU, rfl⟩ := hd
  have hx : x ∈ e ^-1' U := Set.mem_preimage.mpr hxU
  have hmem : P.incl (f x) ∈ (fun x => P.incl (f x)) '' (e ^-1' U) :=
    Set.mem_image_of_mem _ hx
  have hincl : P.incl (f x) ≤ g x := by rw [hfg x]; exact P.incl_retr_le (g x)
  exact le_trans (sInf_le hmem) hincl

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

9

scottSubspaceExtendInf_mono_incl_apply.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y] in
theorem scottSubspaceExtendInf_mono_incl_apply {e : X → Y} {f : X → D} (U : Set Y)
    (P : IsContinuousLatticeProjection D D') :
    P.incl (scottSubspaceExtendInf e f U) ≤ scottSubspaceExtendInf e (fun x => P.incl
    (f x)) U := by
  apply le_sInf
  intro d hd
  obtain ⟨x, hxU, rfl⟩ := hd
  exact P.incl.monotone (sInf_le (Set.mem_image_of_mem f (Set.mem_preimage.mpr hxU)))

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

0

scottSubspaceExtendInf_mono_incl_apply.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
```

```
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y] in
theorem scottSubspaceExtendInf_mono_incl_apply {e : X → Y} {f : X → D} (U : Set Y)
    (P : IsContinuousLatticeProjection D D') :
    P.incl (scottSubspaceExtendInf e f U) ≤ scottSubspaceExtendInf e (fun x => P.incl
    (f x)) U := by
  apply le_sInf
  intro d hd
  obtain ⟨x, hxU, rfl⟩ := hd
  exact P.incl.monotone (sInf_le (Set.mem_image_of_mem f (Set.mem_preimage.mpr hxU)))

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

1

scottSubspaceExtendInf_mono_incl_apply.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y] in
theorem scottSubspaceExtendInf_mono_incl_apply {e : X → Y} {f : X → D} (U : Set Y)
    (P : IsContinuousLatticeProjection D D') :
    P.incl (scottSubspaceExtendInf e f U) ≤ scottSubspaceExtendInf e (fun x => P.incl
    (f x)) U := by
  apply le_sInf
```

```
  intro d hd
  obtain ⟨x, hxU, rfl⟩ := hd
  exact P.incl.monotone (sInf_le (Set.mem_image_of_mem f (Set.mem_preimage.mpr hxU)))

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

2

scottSubspaceExtendInf_mono_incl_apply.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y] in
theorem scottSubspaceExtendInf_mono_incl_apply {e : X → Y} {f : X → D} (U : Set Y)
    (P : IsContinuousLatticeProjection D D') :
    P.incl (scottSubspaceExtendInf e f U) ≤ scottSubspaceExtendInf e (fun x => P.incl
    (f x)) U := by
  apply le_sInf
  intro d hd
  obtain ⟨x, hxU, rfl⟩ := hd
  exact P.incl.monotone (sInf_le (Set.mem_image_of_mem f (Set.mem_preimage.mpr hxU)))

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

3

scottSubspaceExtendInf_mono_incl_apply.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y] in
theorem scottSubspaceExtendInf_mono_incl_apply {e : X → Y} {f : X → D} (U : Set Y)
    (P : IsContinuousLatticeProjection D D') :
    P.incl (scottSubspaceExtendInf e f U) ≤ scottSubspaceExtendInf e (fun x => P.incl
    (f x)) U := by
  apply le_sInf
  intro d hd
  obtain ⟨x, hxU, rfl⟩ := hd
  exact P.incl.monotone (sInf_le (Set.mem_image_of_mem f (Set.mem_preimage.mpr hxU)))

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

4

scottSubspaceExtendInf_mono_incl_apply.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']
```

```
section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y] in
theorem scottSubspaceExtendInf_mono_incl_apply {e : X → Y} {f : X → D} (U : Set Y)
    (P : IsContinuousLatticeProjection D D') :
    P.incl (scottSubspaceExtendInf e f U) ≤ scottSubspaceExtendInf e (fun x => P.incl
    (f x)) U := by
  apply le_sInf
  intro d hd
  obtain ⟨x, hxU, rfl⟩ := hd
  exact P.incl.monotone (sInf_le (Set.mem_image_of_mem f (Set.mem_preimage.mpr hxU)))

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

5

scottSubspaceExtendInf_mono_incl_apply.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y] in
theorem scottSubspaceExtendInf_mono_incl_apply {e : X → Y} {f : X → D} (U : Set Y)
    (P : IsContinuousLatticeProjection D D') :
    P.incl (scottSubspaceExtendInf e f U) ≤ scottSubspaceExtendInf e (fun x => P.incl
    (f x)) U := by
  apply le_sInf
  intro d hd
  obtain ⟨x, hxU, rfl⟩ := hd
  exact P.incl.monotone (sInf_le (Set.mem_image_of_mem f (Set.mem_preimage.mpr hxU)))
```

```
end SubspaceExtension
end DomainTheory.ContinuousLattice
```

6

scottSubspaceExtendInf_mono_incl_apply.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y] in
theorem scottSubspaceExtendInf_mono_incl_apply {e : X → Y} {f : X → D} (U : Set Y)
    (P : IsContinuousLatticeProjection D D') :
    P.incl (scottSubspaceExtendInf e f U) ≤ scottSubspaceExtendInf e (fun x => P.incl
    (f x)) U := by
  apply le_sInf
  intro d hd
  obtain ⟨x, hxU, rfl⟩ := hd
  exact P.incl.monotone (sInf_le (Set.mem_image_of_mem f (Set.mem_preimage.mpr hxU)))

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

7

scottSubspaceExtendInf_mono_incl_apply.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic
```

```
@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y] in
theorem scottSubspaceExtendInf_mono_incl_apply {e : X → Y} {f : X → D} (U : Set Y)
    (P : IsContinuousLatticeProjection D D') :
    P.incl (scottSubspaceExtendInf e f U) ≤ scottSubspaceExtendInf e (fun x => P.incl
    (f x)) U := by
  apply le_sInf
  intro d hd
  obtain ⟨x, hxU, rfl⟩ := hd
  exact P.incl.monotone (sInf_le (Set.mem_image_of_mem f (Set.mem_preimage.mpr hxU)))

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

8

scottSubspaceExtendInf_mono_incl_apply.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]
```

```
omit [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y] in
theorem scottSubspaceExtendInf_mono_incl_apply {e : X → Y} {f : X → D} (U : Set Y)
    (P : IsContinuousLatticeProjection D D') :
    P.incl (scottSubspaceExtendInf e f U) ≤ scottSubspaceExtendInf e (fun x => P.incl
    (f x)) U := by
  apply le_sInf
  intro d hd
  obtain ⟨x, hxU, rfl⟩ := hd
  exact P.incl.monotone (sInf_le (Set.mem_image_of_mem f (Set.mem_preimage.mpr hxU)))

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

9

scottSubspaceExtendInf_mono_retr.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y] in
theorem scottSubspaceExtendInf_mono_retr {e : X → Y} {g : X → D'}
    (P : IsContinuousLatticeProjection D D') (U : Set Y) :
    P.retr (scottSubspaceExtendInf e g U) ≤ scottSubspaceExtendInf e (fun x => P.retr
    (g x)) U := by
  apply le_sInf
  intro d hd
  obtain ⟨x, hxU, rfl⟩ := hd
  have hx : x ∈ e ^-1' U := Set.mem_preimage.mpr hxU
  exact (P.retr.monotone (sInf_le (Set.mem_image_of_mem g hx))).trans le_rfl

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

0

scottSubspaceExtendInf_mono_retr.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y] in
theorem scottSubspaceExtendInf_mono_retr {e : X → Y} {g : X → D'}
    (P : IsContinuousLatticeProjection D D') (U : Set Y) :
    P.retr (scottSubspaceExtendInf e g U) ≤ scottSubspaceExtendInf e (fun x => P.retr
    (g x)) U := by
  apply le_sInf
  intro d hd
  obtain ⟨x, hxU, rfl⟩ := hd
  have hx : x ∈ e ^-1' U := Set.mem_preimage.mpr hxU
  exact (P.retr.monotone (sInf_le (Set.mem_image_of_mem g hx))).trans le_rfl

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

1

scottSubspaceExtendInf_mono_retr.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section
```

```
namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y] in
theorem scottSubspaceExtendInf_mono_retr {e : X → Y} {g : X → D'}
    (P : IsContinuousLatticeProjection D D') (U : Set Y) :
    P.retr (scottSubspaceExtendInf e g U) ≤ scottSubspaceExtendInf e (fun x => P.retr
    (g x)) U := by
  apply le_sInf
  intro d hd
  obtain ⟨x, hxU, rfl⟩ := hd
  have hx : x ∈ e ^-1' U := Set.mem_preimage.mpr hxU
  exact (P.retr.monotone (sInf_le (Set.mem_image_of_mem g hx))).trans le_rfl

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

2

scottSubspaceExtendInf_mono_retr.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]
```

```
omit [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y] in
theorem scottSubspaceExtendInf_mono_retr {e : X → Y} {g : X → D'}
    (P : IsContinuousLatticeProjection D D') (U : Set Y) :
    P.retr (scottSubspaceExtendInf e g U) ≤ scottSubspaceExtendInf e (fun x => P.retr
    (g x)) U := by
  apply le_sInf
  intro d hd
  obtain ⟨x, hxU, rfl⟩ := hd
  have hx : x ∈ e ^-1' U := Set.mem_preimage.mpr hxU
  exact (P.retr.monotone (sInf_le (Set.mem_image_of_mem g hx))).trans le_rfl

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

3

scottSubspaceExtendInf_mono_retr.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y] in
theorem scottSubspaceExtendInf_mono_retr {e : X → Y} {g : X → D'}
    (P : IsContinuousLatticeProjection D D') (U : Set Y) :
    P.retr (scottSubspaceExtendInf e g U) ≤ scottSubspaceExtendInf e (fun x => P.retr
    (g x)) U := by
  apply le_sInf
  intro d hd
  obtain ⟨x, hxU, rfl⟩ := hd
  have hx : x ∈ e ^-1' U := Set.mem_preimage.mpr hxU
  exact (P.retr.monotone (sInf_le (Set.mem_image_of_mem g hx))).trans le_rfl

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

4

scottSubspaceExtendInf_mono_retr.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y] in
theorem scottSubspaceExtendInf_mono_retr {e : X → Y} {g : X → D'}
    (P : IsContinuousLatticeProjection D D') (U : Set Y) :
    P.retr (scottSubspaceExtendInf e g U) ≤ scottSubspaceExtendInf e (fun x => P.retr
    (g x)) U := by
  apply le_sInf
  intro d hd
  obtain ⟨x, hxU, rfl⟩ := hd
  have hx : x ∈ e ^-1' U := Set.mem_preimage.mpr hxU
  exact (P.retr.monotone (sInf_le (Set.mem_image_of_mem g hx))).trans le_rfl

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

5

scottSubspaceExtendInf_mono_retr.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section
```

```
namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y] in
theorem scottSubspaceExtendInf_mono_retr {e : X → Y} {g : X → D'}
    (P : IsContinuousLatticeProjection D D') (U : Set Y) :
    P.retr (scottSubspaceExtendInf e g U) ≤ scottSubspaceExtendInf e (fun x => P.retr
    (g x)) U := by
  apply le_sInf
  intro d hd
  obtain ⟨x, hxU, rfl⟩ := hd
  have hx : x ∈ e ^-1' U := Set.mem_preimage.mpr hxU
  exact (P.retr.monotone (sInf_le (Set.mem_image_of_mem g hx))).trans le_rfl

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

6

scottSubspaceExtendInf_mono_retr.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]
```

```
omit [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y] in
theorem scottSubspaceExtendInf_mono_retr {e : X → Y} {g : X → D'}
    (P : IsContinuousLatticeProjection D D') (U : Set Y) :
    P.retr (scottSubspaceExtendInf e g U) ≤ scottSubspaceExtendInf e (fun x => P.retr
    (g x)) U := by
  apply le_sInf
  intro d hd
  obtain ⟨x, hxU, rfl⟩ := hd
  have hx : x ∈ e ^-1' U := Set.mem_preimage.mpr hxU
  exact (P.retr.monotone (sInf_le (Set.mem_image_of_mem g hx))).trans le_rfl

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

7

scottSubspaceExtendInf_mono_retr.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y] in
theorem scottSubspaceExtendInf_mono_retr {e : X → Y} {g : X → D'}
    (P : IsContinuousLatticeProjection D D') (U : Set Y) :
    P.retr (scottSubspaceExtendInf e g U) ≤ scottSubspaceExtendInf e (fun x => P.retr
    (g x)) U := by
  apply le_sInf
  intro d hd
  obtain ⟨x, hxU, rfl⟩ := hd
  have hx : x ∈ e ^-1' U := Set.mem_preimage.mpr hxU
  exact (P.retr.monotone (sInf_le (Set.mem_image_of_mem g hx))).trans le_rfl

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

8

scottSubspaceExtendInf_mono_retr.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y] in
theorem scottSubspaceExtendInf_mono_retr {e : X → Y} {g : X → D'}
    (P : IsContinuousLatticeProjection D D') (U : Set Y) :
    P.retr (scottSubspaceExtendInf e g U) ≤ scottSubspaceExtendInf e (fun x => P.retr
    (g x)) U := by
  apply le_sInf
  intro d hd
  obtain ⟨x, hxU, rfl⟩ := hd
  have hx : x ∈ e ^-1' U := Set.mem_preimage.mpr hxU
  exact (P.retr.monotone (sInf_le (Set.mem_image_of_mem g hx))).trans le_rfl

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

9

scottSubspaceExtend_eq_of_ext.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottSubspaceExtendInf_eq_of_ext
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtend
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice
```

```
open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] in
theorem scottSubspaceExtend_eq_of_ext {e : X → Y} {f g' : X → D} (h : ∀ x, f x = g' x
    ) (y : Y) :
    scottSubspaceExtend e f y = scottSubspaceExtend e g' y := by
  unfold scottSubspaceExtend
  congr 1
  ext d
  simp [Set.mem_ofPred_eq, scottSubspaceExtendInf_eq_of_ext h]

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

0

scottSubspaceExtend_eq_of_ext.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottSubspaceExtendInf_eq_of_ext
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtend
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] in
theorem scottSubspaceExtend_eq_of_ext {e : X → Y} {f g' : X → D} (h : ∀ x, f x = g' x
    ) (y : Y) :
    scottSubspaceExtend e f y = scottSubspaceExtend e g' y := by
```

```
  unfold scottSubspaceExtend
  congr 1
  ext d
  simp [Set.mem_ofPred_eq, scottSubspaceExtendInf_eq_of_ext h]

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

1

scottSubspaceExtend_eq_of_ext.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottSubspaceExtendInf_eq_of_ext
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtend
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] in
theorem scottSubspaceExtend_eq_of_ext {e : X → Y} {f g' : X → D} (h : ∀ x, f x = g' x
    ) (y : Y) :
    scottSubspaceExtend e f y = scottSubspaceExtend e g' y := by
  unfold scottSubspaceExtend
  congr 1
  ext d
  simp [Set.mem_ofPred_eq, scottSubspaceExtendInf_eq_of_ext h]

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

2

scottSubspaceExtend_eq_of_ext.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottSubspaceExtendInf_eq_of_ext
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtend
public import Mathlib.Topology.Defs.Basic
```

```
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] in
theorem scottSubspaceExtend_eq_of_ext {e : X → Y} {f g' : X → D} (h : ∀ x, f x = g' x
    ) (y : Y) :
    scottSubspaceExtend e f y = scottSubspaceExtend e g' y := by
  unfold scottSubspaceExtend
  congr 1
  ext d
  simp [Set.mem_ofPred_eq, scottSubspaceExtendInf_eq_of_ext h]

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

3

scottSubspaceExtend_eq_of_ext.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottSubspaceExtendInf_eq_of_ext
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtend
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]
```

```
omit [TopologicalSpace X] [TopologicalSpace Y] in
theorem scottSubspaceExtend_eq_of_ext {e : X → Y} {f g' : X → D} (h : ∀ x, f x = g' x
    ) (y : Y) :
    scottSubspaceExtend e f y = scottSubspaceExtend e g' y := by
  unfold scottSubspaceExtend
  congr 1
  ext d
  simp [Set.mem_ofPred_eq, scottSubspaceExtendInf_eq_of_ext h]

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

4

scottSubspaceExtend_eq_of_ext.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottSubspaceExtendInf_eq_of_ext
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtend
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] in
theorem scottSubspaceExtend_eq_of_ext {e : X → Y} {f g' : X → D} (h : ∀ x, f x = g' x
    ) (y : Y) :
    scottSubspaceExtend e f y = scottSubspaceExtend e g' y := by
  unfold scottSubspaceExtend
  congr 1
  ext d
  simp [Set.mem_ofPred_eq, scottSubspaceExtendInf_eq_of_ext h]

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

5

scottSubspaceExtend_eq_of_ext.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottSubspaceExtendInf_eq_of_ext
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtend
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] in
theorem scottSubspaceExtend_eq_of_ext {e : X → Y} {f g' : X → D} (h : ∀ x, f x = g' x
    ) (y : Y) :
    scottSubspaceExtend e f y = scottSubspaceExtend e g' y := by
  unfold scottSubspaceExtend
  congr 1
  ext d
  simp [Set.mem_ofPred_eq, scottSubspaceExtendInf_eq_of_ext h]

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

6

scottSubspaceExtend_eq_of_ext.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottSubspaceExtendInf_eq_of_ext
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtend
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
```

```
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] in
theorem scottSubspaceExtend_eq_of_ext {e : X → Y} {f g' : X → D} (h : ∀ x, f x = g' x
    ) (y : Y) :
    scottSubspaceExtend e f y = scottSubspaceExtend e g' y := by
  unfold scottSubspaceExtend
  congr 1
  ext d
  simp [Set.mem_ofPred_eq, scottSubspaceExtendInf_eq_of_ext h]

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

7

scottSubspaceExtend_eq_of_ext.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottSubspaceExtendInf_eq_of_ext
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtend
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] in
theorem scottSubspaceExtend_eq_of_ext {e : X → Y} {f g' : X → D} (h : ∀ x, f x = g' x
    ) (y : Y) :
    scottSubspaceExtend e f y = scottSubspaceExtend e g' y := by
  unfold scottSubspaceExtend
  congr 1
  ext d
  simp [Set.mem_ofPred_eq, scottSubspaceExtendInf_eq_of_ext h]

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

8

scottSubspaceExtend_eq_of_ext.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottSubspaceExtendInf_eq_of_ext
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtend
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] in
theorem scottSubspaceExtend_eq_of_ext {e : X → Y} {f g' : X → D} (h : ∀ x, f x = g' x
    ) (y : Y) :
    scottSubspaceExtend e f y = scottSubspaceExtend e g' y := by
  unfold scottSubspaceExtend
  congr 1
  ext d
  simp [Set.mem_ofPred_eq, scottSubspaceExtendInf_eq_of_ext h]

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

9

scottSubspaceExtend_maximal.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottSubspaceExtendInf_eq_of_comp
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottMap_eq_sSup_openInfs
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    sInf_image_le_sInf_image_of_subset
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtend
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic
```

```
@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

/-- **Scott 1972, Proposition 3.8 (subspace variant).** `f` (with the Scott topology on
     `Y`) is
the maximal extension along a subspace embedding. The faithful statement (arbitrary
    topology on
`Y`) is `proposition_3_8` in `Constructions.lean`. -/
theorem scottSubspaceExtend_maximal (hD : ContinuousLattice D) (e : X → Y) (_he :
    IsEmbedding e)
    (f : X → D) (f' : ScottMap Y D) (h_ext : ∀ x, f' (e x) = f x) (y : Y) :
    (f' : Y → D) y ≤ scottSubspaceExtend e f y := by
  have hEq := scottMap_eq_sSup_openInfs hD f' y
  have hmain :
      sSup { d | ∃ U, @IsOpen Y scottTopologicalSpace U ∧ y ∈ U ∧
        d = sInf (Set.image (f' : Y → D) U) } ≤ scottSubspaceExtend e f y := by
    unfold scottSubspaceExtend
    apply sSup_le
    intro d hd
    obtain ⟨U, hUopen, hyU, rfl⟩ := hd
    have hmem : scottSubspaceExtendInf e f U ∈
        { d | ∃ U, @IsOpen Y scottTopologicalSpace U ∧ y ∈ U ∧ d =
    scottSubspaceExtendInf e f U } :=
      ⟨U, hUopen, hyU, rfl⟩
    refine le_trans ?_ (le_sSup hmem)
    have hrestrict :
        sInf (Set.image (f' : Y → D) U) ≤
          sInf (Set.image (f' : Y → D) (Set.image e Set.univ ∩ U)) :=
      sInf_image_le_sInf_image_of_subset
        (show Set.image e Set.univ ∩ U ⊆ U from inter_subset_right)
    refine le_trans hrestrict (le_of_eq (scottSubspaceExtendInf_eq_of_comp (f' := f')
    h_ext).symm)
  exact le_trans (le_of_eq hEq) hmain

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

0

scottSubspaceExtend_maximal.lean

```
module
```

```
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottSubspaceExtendInf_eq_of_comp
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottMap_eq_sSup_openInfs
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    sInf_image_le_sInf_image_of_subset
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtend
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

/-- **Scott 1972, Proposition 3.8 (subspace variant).** `f` (with the Scott topology on
     `Y`) is
the maximal extension along a subspace embedding. The faithful statement (arbitrary
    topology on
`Y`) is `proposition_3_8` in `Constructions.lean`. -/
theorem scottSubspaceExtend_maximal (hD : ContinuousLattice D) (e : X → Y) (_he :
    IsEmbedding e)
    (f : X → D) (f' : ScottMap Y D) (h_ext : ∀ x, f' (e x) = f x) (y : Y) :
    (f' : Y → D) y ≤ scottSubspaceExtend e f y := by
  have hEq := scottMap_eq_sSup_openInfs hD f' y
  have hmain :
      sSup { d | ∃ U, @IsOpen Y scottTopologicalSpace U ∧ y ∈ U ∧
        d = sInf (Set.image (f' : Y → D) U) } ≤ scottSubspaceExtend e f y := by
    unfold scottSubspaceExtend
    apply sSup_le
    intro d hd
    obtain ⟨U, hUopen, hyU, rfl⟩ := hd
    have hmem : scottSubspaceExtendInf e f U ∈
        { d | ∃ U, @IsOpen Y scottTopologicalSpace U ∧ y ∈ U ∧ d =
    scottSubspaceExtendInf e f U } :=
      ⟨U, hUopen, hyU, rfl⟩
    refine le_trans ?_ (le_sSup hmem)
    have hrestrict :
        sInf (Set.image (f' : Y → D) U) ≤
          sInf (Set.image (f' : Y → D) (Set.image e Set.univ ∩ U)) :=
      sInf_image_le_sInf_image_of_subset
        (show Set.image e Set.univ ∩ U ⊆ U from inter_subset_right)
    refine le_trans hrestrict (le_of_eq (scottSubspaceExtendInf_eq_of_comp (f' := f')
    h_ext).symm)
```

```
  exact le_trans (le_of_eq hEq) hmain

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

1

scottSubspaceExtend_maximal.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottSubspaceExtendInf_eq_of_comp
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottMap_eq_sSup_openInfs
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    sInf_image_le_sInf_image_of_subset
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtend
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

/-- **Scott 1972, Proposition 3.8 (subspace variant).** `f` (with the Scott topology on
     `Y`) is
the maximal extension along a subspace embedding. The faithful statement (arbitrary
    topology on
`Y`) is `proposition_3_8` in `Constructions.lean`. -/
theorem scottSubspaceExtend_maximal (hD : ContinuousLattice D) (e : X → Y) (_he :
    IsEmbedding e)
    (f : X → D) (f' : ScottMap Y D) (h_ext : ∀ x, f' (e x) = f x) (y : Y) :
    (f' : Y → D) y ≤ scottSubspaceExtend e f y := by
  have hEq := scottMap_eq_sSup_openInfs hD f' y
  have hmain :
      sSup { d | ∃ U, @IsOpen Y scottTopologicalSpace U ∧ y ∈ U ∧
        d = sInf (Set.image (f' : Y → D) U) } ≤ scottSubspaceExtend e f y := by
    unfold scottSubspaceExtend
    apply sSup_le
    intro d hd
    obtain ⟨U, hUopen, hyU, rfl⟩ := hd
    have hmem : scottSubspaceExtendInf e f U ∈
```

```
        { d | ∃ U, @IsOpen Y scottTopologicalSpace U ∧ y ∈ U ∧ d =
    scottSubspaceExtendInf e f U } :=
      ⟨U, hUopen, hyU, rfl⟩
    refine le_trans ?_ (le_sSup hmem)
    have hrestrict :
        sInf (Set.image (f' : Y → D) U) ≤
          sInf (Set.image (f' : Y → D) (Set.image e Set.univ ∩ U)) :=
      sInf_image_le_sInf_image_of_subset
        (show Set.image e Set.univ ∩ U ⊆ U from inter_subset_right)
    refine le_trans hrestrict (le_of_eq (scottSubspaceExtendInf_eq_of_comp (f' := f')
    h_ext).symm)
  exact le_trans (le_of_eq hEq) hmain

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

2

scottSubspaceExtend_maximal.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottSubspaceExtendInf_eq_of_comp
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottMap_eq_sSup_openInfs
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    sInf_image_le_sInf_image_of_subset
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtend
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

/-- **Scott 1972, Proposition 3.8 (subspace variant).** `f` (with the Scott topology on
     `Y`) is
the maximal extension along a subspace embedding. The faithful statement (arbitrary
    topology on
`Y`) is `proposition_3_8` in `Constructions.lean`. -/
theorem scottSubspaceExtend_maximal (hD : ContinuousLattice D) (e : X → Y) (_he :
    IsEmbedding e)
```

```
    (f : X → D) (f' : ScottMap Y D) (h_ext : ∀ x, f' (e x) = f x) (y : Y) :
    (f' : Y → D) y ≤ scottSubspaceExtend e f y := by
  have hEq := scottMap_eq_sSup_openInfs hD f' y
  have hmain :
      sSup { d | ∃ U, @IsOpen Y scottTopologicalSpace U ∧ y ∈ U ∧
        d = sInf (Set.image (f' : Y → D) U) } ≤ scottSubspaceExtend e f y := by
    unfold scottSubspaceExtend
    apply sSup_le
    intro d hd
    obtain ⟨U, hUopen, hyU, rfl⟩ := hd
    have hmem : scottSubspaceExtendInf e f U ∈
        { d | ∃ U, @IsOpen Y scottTopologicalSpace U ∧ y ∈ U ∧ d =
    scottSubspaceExtendInf e f U } :=
      ⟨U, hUopen, hyU, rfl⟩
    refine le_trans ?_ (le_sSup hmem)
    have hrestrict :
        sInf (Set.image (f' : Y → D) U) ≤
          sInf (Set.image (f' : Y → D) (Set.image e Set.univ ∩ U)) :=
      sInf_image_le_sInf_image_of_subset
        (show Set.image e Set.univ ∩ U ⊆ U from inter_subset_right)
    refine le_trans hrestrict (le_of_eq (scottSubspaceExtendInf_eq_of_comp (f' := f')
    h_ext).symm)
  exact le_trans (le_of_eq hEq) hmain

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

3

scottSubspaceExtend_maximal.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottSubspaceExtendInf_eq_of_comp
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottMap_eq_sSup_openInfs
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    sInf_image_le_sInf_image_of_subset
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtend
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']
```

```
section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

/-- **Scott 1972, Proposition 3.8 (subspace variant).** `f` (with the Scott topology on
    `Y`) is
the maximal extension along a subspace embedding. The faithful statement (arbitrary
   topology on
`Y`) is `proposition_3_8` in `Constructions.lean`. -/
theorem scottSubspaceExtend_maximal (hD : ContinuousLattice D) (e : X → Y) (_he :
   IsEmbedding e)
    (f : X → D) (f' : ScottMap Y D) (h_ext : ∀ x, f' (e x) = f x) (y : Y) :
    (f' : Y → D) y ≤ scottSubspaceExtend e f y := by
  have hEq := scottMap_eq_sSup_openInfs hD f' y
  have hmain :
      sSup { d | ∃ U, @IsOpen Y scottTopologicalSpace U ∧ y ∈ U ∧
        d = sInf (Set.image (f' : Y → D) U) } ≤ scottSubspaceExtend e f y := by
    unfold scottSubspaceExtend
    apply sSup_le
    intro d hd
    obtain ⟨U, hUopen, hyU, rfl⟩ := hd
    have hmem : scottSubspaceExtendInf e f U ∈
        { d | ∃ U, @IsOpen Y scottTopologicalSpace U ∧ y ∈ U ∧ d =
    scottSubspaceExtendInf e f U } :=
      ⟨U, hUopen, hyU, rfl⟩
    refine le_trans ?_ (le_sSup hmem)
    have hrestrict :
        sInf (Set.image (f' : Y → D) U) ≤
          sInf (Set.image (f' : Y → D) (Set.image e Set.univ ∩ U)) :=
      sInf_image_le_sInf_image_of_subset
        (show Set.image e Set.univ ∩ U ⊆ U from inter_subset_right)
    refine le_trans hrestrict (le_of_eq (scottSubspaceExtendInf_eq_of_comp (f' := f')
    h_ext).symm)
  exact le_trans (le_of_eq hEq) hmain

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

4

scottSubspaceExtend_maximal.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottSubspaceExtendInf_eq_of_comp
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottMap_eq_sSup_openInfs
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    sInf_image_le_sInf_image_of_subset
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtend
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic

@[expose] public section
```

```
namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

/-- **Scott 1972, Proposition 3.8 (subspace variant).** `f` (with the Scott topology on
     `Y`) is
the maximal extension along a subspace embedding. The faithful statement (arbitrary
    topology on
`Y`) is `proposition_3_8` in `Constructions.lean`. -/
theorem scottSubspaceExtend_maximal (hD : ContinuousLattice D) (e : X → Y) (_he :
    IsEmbedding e)
    (f : X → D) (f' : ScottMap Y D) (h_ext : ∀ x, f' (e x) = f x) (y : Y) :
    (f' : Y → D) y ≤ scottSubspaceExtend e f y := by
  have hEq := scottMap_eq_sSup_openInfs hD f' y
  have hmain :
      sSup { d | ∃ U, @IsOpen Y scottTopologicalSpace U ∧ y ∈ U ∧
        d = sInf (Set.image (f' : Y → D) U) } ≤ scottSubspaceExtend e f y := by
    unfold scottSubspaceExtend
    apply sSup_le
    intro d hd
    obtain ⟨U, hUopen, hyU, rfl⟩ := hd
    have hmem : scottSubspaceExtendInf e f U ∈
        { d | ∃ U, @IsOpen Y scottTopologicalSpace U ∧ y ∈ U ∧ d =
    scottSubspaceExtendInf e f U } :=
      ⟨U, hUopen, hyU, rfl⟩
    refine le_trans ?_ (le_sSup hmem)
    have hrestrict :
        sInf (Set.image (f' : Y → D) U) ≤
          sInf (Set.image (f' : Y → D) (Set.image e Set.univ ∩ U)) :=
      sInf_image_le_sInf_image_of_subset
        (show Set.image e Set.univ ∩ U ⊆ U from inter_subset_right)
    refine le_trans hrestrict (le_of_eq (scottSubspaceExtendInf_eq_of_comp (f' := f')
    h_ext).symm)
  exact le_trans (le_of_eq hEq) hmain

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

5

scottSubspaceExtend_maximal.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottSubspaceExtendInf_eq_of_comp
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottMap_eq_sSup_openInfs
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    sInf_image_le_sInf_image_of_subset
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtend
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

/-- **Scott 1972, Proposition 3.8 (subspace variant).** `f` (with the Scott topology on
     `Y`) is
the maximal extension along a subspace embedding. The faithful statement (arbitrary
    topology on
`Y`) is `proposition_3_8` in `Constructions.lean`. -/
theorem scottSubspaceExtend_maximal (hD : ContinuousLattice D) (e : X → Y) (_he :
    IsEmbedding e)
    (f : X → D) (f' : ScottMap Y D) (h_ext : ∀ x, f' (e x) = f x) (y : Y) :
    (f' : Y → D) y ≤ scottSubspaceExtend e f y := by
  have hEq := scottMap_eq_sSup_openInfs hD f' y
  have hmain :
      sSup { d | ∃ U, @IsOpen Y scottTopologicalSpace U ∧ y ∈ U ∧
        d = sInf (Set.image (f' : Y → D) U) } ≤ scottSubspaceExtend e f y := by
    unfold scottSubspaceExtend
    apply sSup_le
    intro d hd
    obtain ⟨U, hUopen, hyU, rfl⟩ := hd
    have hmem : scottSubspaceExtendInf e f U ∈
        { d | ∃ U, @IsOpen Y scottTopologicalSpace U ∧ y ∈ U ∧ d =
    scottSubspaceExtendInf e f U } :=
      ⟨U, hUopen, hyU, rfl⟩
    refine le_trans ?_ (le_sSup hmem)
    have hrestrict :
        sInf (Set.image (f' : Y → D) U) ≤
          sInf (Set.image (f' : Y → D) (Set.image e Set.univ ∩ U)) :=
      sInf_image_le_sInf_image_of_subset
        (show Set.image e Set.univ ∩ U ⊆ U from inter_subset_right)
    refine le_trans hrestrict (le_of_eq (scottSubspaceExtendInf_eq_of_comp (f' := f')
    h_ext).symm)
  exact le_trans (le_of_eq hEq) hmain
```

```
end SubspaceExtension
end DomainTheory.ContinuousLattice
```

6

scottSubspaceExtend_maximal.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottSubspaceExtendInf_eq_of_comp
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottMap_eq_sSup_openInfs
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    sInf_image_le_sInf_image_of_subset
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtend
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

/-- **Scott 1972, Proposition 3.8 (subspace variant).** `f` (with the Scott topology on
     `Y`) is
the maximal extension along a subspace embedding. The faithful statement (arbitrary
    topology on
`Y`) is `proposition_3_8` in `Constructions.lean`. -/
theorem scottSubspaceExtend_maximal (hD : ContinuousLattice D) (e : X → Y) (_he :
    IsEmbedding e)
    (f : X → D) (f' : ScottMap Y D) (h_ext : ∀ x, f' (e x) = f x) (y : Y) :
    (f' : Y → D) y ≤ scottSubspaceExtend e f y := by
  have hEq := scottMap_eq_sSup_openInfs hD f' y
  have hmain :
      sSup { d | ∃ U, @IsOpen Y scottTopologicalSpace U ∧ y ∈ U ∧
        d = sInf (Set.image (f' : Y → D) U) } ≤ scottSubspaceExtend e f y := by
    unfold scottSubspaceExtend
    apply sSup_le
    intro d hd
    obtain ⟨U, hUopen, hyU, rfl⟩ := hd
    have hmem : scottSubspaceExtendInf e f U ∈
        { d | ∃ U, @IsOpen Y scottTopologicalSpace U ∧ y ∈ U ∧ d =
```

```
    scottSubspaceExtendInf e f U } :=
      ⟨U, hUopen, hyU, rfl⟩
    refine le_trans ?_ (le_sSup hmem)
    have hrestrict :
        sInf (Set.image (f' : Y → D) U) ≤
          sInf (Set.image (f' : Y → D) (Set.image e Set.univ ∩ U)) :=
      sInf_image_le_sInf_image_of_subset
        (show Set.image e Set.univ ∩ U ⊆ U from inter_subset_right)
    refine le_trans hrestrict (le_of_eq (scottSubspaceExtendInf_eq_of_comp (f' := f')
    h_ext).symm)
  exact le_trans (le_of_eq hEq) hmain

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

7

scottSubspaceExtend_maximal.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottSubspaceExtendInf_eq_of_comp
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottMap_eq_sSup_openInfs
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    sInf_image_le_sInf_image_of_subset
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtend
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

/-- **Scott 1972, Proposition 3.8 (subspace variant).** `f` (with the Scott topology on
     `Y`) is
the maximal extension along a subspace embedding. The faithful statement (arbitrary
    topology on
`Y`) is `proposition_3_8` in `Constructions.lean`. -/
theorem scottSubspaceExtend_maximal (hD : ContinuousLattice D) (e : X → Y) (_he :
    IsEmbedding e)
    (f : X → D) (f' : ScottMap Y D) (h_ext : ∀ x, f' (e x) = f x) (y : Y) :
```

```
    (f' : Y → D) y ≤ scottSubspaceExtend e f y := by
  have hEq := scottMap_eq_sSup_openInfs hD f' y
  have hmain :
      sSup { d | ∃ U, @IsOpen Y scottTopologicalSpace U ∧ y ∈ U ∧
        d = sInf (Set.image (f' : Y → D) U) } ≤ scottSubspaceExtend e f y := by
    unfold scottSubspaceExtend
    apply sSup_le
    intro d hd
    obtain ⟨U, hUopen, hyU, rfl⟩ := hd
    have hmem : scottSubspaceExtendInf e f U ∈
        { d | ∃ U, @IsOpen Y scottTopologicalSpace U ∧ y ∈ U ∧ d =
    scottSubspaceExtendInf e f U } :=
      ⟨U, hUopen, hyU, rfl⟩
    refine le_trans ?_ (le_sSup hmem)
    have hrestrict :
        sInf (Set.image (f' : Y → D) U) ≤
          sInf (Set.image (f' : Y → D) (Set.image e Set.univ ∩ U)) :=
      sInf_image_le_sInf_image_of_subset
        (show Set.image e Set.univ ∩ U ⊆ U from inter_subset_right)
    refine le_trans hrestrict (le_of_eq (scottSubspaceExtendInf_eq_of_comp (f' := f')
    h_ext).symm)
  exact le_trans (le_of_eq hEq) hmain

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

8

scottSubspaceExtend_maximal.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottSubspaceExtendInf_eq_of_comp
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottMap_eq_sSup_openInfs
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    sInf_image_le_sInf_image_of_subset
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtend
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension
```

```
variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

/-- **Scott 1972, Proposition 3.8 (subspace variant).** `f` (with the Scott topology on
    `Y`) is
the maximal extension along a subspace embedding. The faithful statement (arbitrary
    topology on
`Y`) is `proposition_3_8` in `Constructions.lean`. -/
theorem scottSubspaceExtend_maximal (hD : ContinuousLattice D) (e : X → Y) (_he :
    IsEmbedding e)
    (f : X → D) (f' : ScottMap Y D) (h_ext : ∀ x, f' (e x) = f x) (y : Y) :
    (f' : Y → D) y ≤ scottSubspaceExtend e f y := by
  have hEq := scottMap_eq_sSup_openInfs hD f' y
  have hmain :
      sSup { d | ∃ U, @IsOpen Y scottTopologicalSpace U ∧ y ∈ U ∧
        d = sInf (Set.image (f' : Y → D) U) } ≤ scottSubspaceExtend e f y := by
    unfold scottSubspaceExtend
    apply sSup_le
    intro d hd
    obtain ⟨U, hUopen, hyU, rfl⟩ := hd
    have hmem : scottSubspaceExtendInf e f U ∈
        { d | ∃ U, @IsOpen Y scottTopologicalSpace U ∧ y ∈ U ∧ d =
    scottSubspaceExtendInf e f U } :=
      ⟨U, hUopen, hyU, rfl⟩
    refine le_trans ?_ (le_sSup hmem)
    have hrestrict :
        sInf (Set.image (f' : Y → D) U) ≤
          sInf (Set.image (f' : Y → D) (Set.image e Set.univ ∩ U)) :=
      sInf_image_le_sInf_image_of_subset
        (show Set.image e Set.univ ∩ U ⊆ U from inter_subset_right)
    refine le_trans hrestrict (le_of_eq (scottSubspaceExtendInf_eq_of_comp (f' := f')
    h_ext).symm)
  exact le_trans (le_of_eq hEq) hmain

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

9

scottSubspaceExtend_mono.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottSubspaceExtendInf_mono
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtend
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
```

```
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] in
theorem scottSubspaceExtend_mono {e : X → Y} {f g₀ : X → D} (hfg : ∀ x, f x ≤ g₀ x) (
    y : Y) :
    scottSubspaceExtend e f y ≤ scottSubspaceExtend e g₀ y := by
  apply sSup_le
  intro d hd
  obtain ⟨U, hUopen, hyU, rfl⟩ := hd
  have hmem : scottSubspaceExtendInf e g₀ U ∈
      { d | ∃ U, @IsOpen Y scottTopologicalSpace U ∧ y ∈ U ∧ d =
    scottSubspaceExtendInf e g₀ U } :=
    ⟨U, hUopen, hyU, rfl⟩
  exact le_trans (scottSubspaceExtendInf_mono hfg U) (le_sSup hmem)

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

0

scottSubspaceExtend_mono.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottSubspaceExtendInf_mono
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtend
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] in
theorem scottSubspaceExtend_mono {e : X → Y} {f g₀ : X → D} (hfg : ∀ x, f x ≤ g₀ x) (
    y : Y) :
```

```
    scottSubspaceExtend e f y ≤ scottSubspaceExtend e g₀ y := by
  apply sSup_le
  intro d hd
  obtain ⟨U, hUopen, hyU, rfl⟩ := hd
  have hmem : scottSubspaceExtendInf e g₀ U ∈
      { d | ∃ U, @IsOpen Y scottTopologicalSpace U ∧ y ∈ U ∧ d =
    scottSubspaceExtendInf e g₀ U } :=
    ⟨U, hUopen, hyU, rfl⟩
  exact le_trans (scottSubspaceExtendInf_mono hfg U) (le_sSup hmem)

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

1

scottSubspaceExtend_mono.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottSubspaceExtendInf_mono
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtend
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] in
theorem scottSubspaceExtend_mono {e : X → Y} {f g₀ : X → D} (hfg : ∀ x, f x ≤ g₀ x) (
    y : Y) :
    scottSubspaceExtend e f y ≤ scottSubspaceExtend e g₀ y := by
  apply sSup_le
  intro d hd
  obtain ⟨U, hUopen, hyU, rfl⟩ := hd
  have hmem : scottSubspaceExtendInf e g₀ U ∈
      { d | ∃ U, @IsOpen Y scottTopologicalSpace U ∧ y ∈ U ∧ d =
    scottSubspaceExtendInf e g₀ U } :=
    ⟨U, hUopen, hyU, rfl⟩
  exact le_trans (scottSubspaceExtendInf_mono hfg U) (le_sSup hmem)

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

2

scottSubspaceExtend_mono.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottSubspaceExtendInf_mono
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtend
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] in
theorem scottSubspaceExtend_mono {e : X → Y} {f g₀ : X → D} (hfg : ∀ x, f x ≤ g₀ x) (
    y : Y) :
    scottSubspaceExtend e f y ≤ scottSubspaceExtend e g₀ y := by
  apply sSup_le
  intro d hd
  obtain ⟨U, hUopen, hyU, rfl⟩ := hd
  have hmem : scottSubspaceExtendInf e g₀ U ∈
      { d | ∃ U, @IsOpen Y scottTopologicalSpace U ∧ y ∈ U ∧ d =
    scottSubspaceExtendInf e g₀ U } :=
    ⟨U, hUopen, hyU, rfl⟩
  exact le_trans (scottSubspaceExtendInf_mono hfg U) (le_sSup hmem)

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

3

scottSubspaceExtend_mono.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottSubspaceExtendInf_mono
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtend
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic
```

```
@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] in
theorem scottSubspaceExtend_mono {e : X → Y} {f g₀ : X → D} (hfg : ∀ x, f x ≤ g₀ x) (
    y : Y) :
    scottSubspaceExtend e f y ≤ scottSubspaceExtend e g₀ y := by
  apply sSup_le
  intro d hd
  obtain ⟨U, hUopen, hyU, rfl⟩ := hd
  have hmem : scottSubspaceExtendInf e g₀ U ∈
      { d | ∃ U, @IsOpen Y scottTopologicalSpace U ∧ y ∈ U ∧ d =
    scottSubspaceExtendInf e g₀ U } :=
    ⟨U, hUopen, hyU, rfl⟩
  exact le_trans (scottSubspaceExtendInf_mono hfg U) (le_sSup hmem)

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

4

scottSubspaceExtend_mono.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottSubspaceExtendInf_mono
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtend
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']
```

```
section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] in
theorem scottSubspaceExtend_mono {e : X → Y} {f g₀ : X → D} (hfg : ∀ x, f x ≤ g₀ x) (
    y : Y) :
    scottSubspaceExtend e f y ≤ scottSubspaceExtend e g₀ y := by
  apply sSup_le
  intro d hd
  obtain ⟨U, hUopen, hyU, rfl⟩ := hd
  have hmem : scottSubspaceExtendInf e g₀ U ∈
      { d | ∃ U, @IsOpen Y scottTopologicalSpace U ∧ y ∈ U ∧ d =
    scottSubspaceExtendInf e g₀ U } :=
    ⟨U, hUopen, hyU, rfl⟩
  exact le_trans (scottSubspaceExtendInf_mono hfg U) (le_sSup hmem)

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

5

scottSubspaceExtend_mono.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottSubspaceExtendInf_mono
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtend
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] in
theorem scottSubspaceExtend_mono {e : X → Y} {f g₀ : X → D} (hfg : ∀ x, f x ≤ g₀ x) (
    y : Y) :
    scottSubspaceExtend e f y ≤ scottSubspaceExtend e g₀ y := by
  apply sSup_le
  intro d hd
  obtain ⟨U, hUopen, hyU, rfl⟩ := hd
  have hmem : scottSubspaceExtendInf e g₀ U ∈
```

```
      { d | ∃ U, @IsOpen Y scottTopologicalSpace U ∧ y ∈ U ∧ d =
    scottSubspaceExtendInf e g₀ U } :=
    ⟨U, hUopen, hyU, rfl⟩
  exact le_trans (scottSubspaceExtendInf_mono hfg U) (le_sSup hmem)

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

6

scottSubspaceExtend_mono.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottSubspaceExtendInf_mono
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtend
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] in
theorem scottSubspaceExtend_mono {e : X → Y} {f g₀ : X → D} (hfg : ∀ x, f x ≤ g₀ x) (
    y : Y) :
    scottSubspaceExtend e f y ≤ scottSubspaceExtend e g₀ y := by
  apply sSup_le
  intro d hd
  obtain ⟨U, hUopen, hyU, rfl⟩ := hd
  have hmem : scottSubspaceExtendInf e g₀ U ∈
      { d | ∃ U, @IsOpen Y scottTopologicalSpace U ∧ y ∈ U ∧ d =
    scottSubspaceExtendInf e g₀ U } :=
    ⟨U, hUopen, hyU, rfl⟩
  exact le_trans (scottSubspaceExtendInf_mono hfg U) (le_sSup hmem)

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

7

scottSubspaceExtend_mono.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottSubspaceExtendInf_mono
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtend
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] in
theorem scottSubspaceExtend_mono {e : X → Y} {f g₀ : X → D} (hfg : ∀ x, f x ≤ g₀ x) (
    y : Y) :
    scottSubspaceExtend e f y ≤ scottSubspaceExtend e g₀ y := by
  apply sSup_le
  intro d hd
  obtain ⟨U, hUopen, hyU, rfl⟩ := hd
  have hmem : scottSubspaceExtendInf e g₀ U ∈
      { d | ∃ U, @IsOpen Y scottTopologicalSpace U ∧ y ∈ U ∧ d =
    scottSubspaceExtendInf e g₀ U } :=
    ⟨U, hUopen, hyU, rfl⟩
  exact le_trans (scottSubspaceExtendInf_mono hfg U) (le_sSup hmem)

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

8

scottSubspaceExtend_mono.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottSubspaceExtendInf_mono
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtend
public import Mathlib.Order.BourbakiWitt
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice
```

```
open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] in
theorem scottSubspaceExtend_mono {e : X → Y} {f g₀ : X → D} (hfg : ∀ x, f x ≤ g₀ x) (
    y : Y) :
    scottSubspaceExtend e f y ≤ scottSubspaceExtend e g₀ y := by
  apply sSup_le
  intro d hd
  obtain ⟨U, hUopen, hyU, rfl⟩ := hd
  have hmem : scottSubspaceExtendInf e g₀ U ∈
      { d | ∃ U, @IsOpen Y scottTopologicalSpace U ∧ y ∈ U ∧ d =
    scottSubspaceExtendInf e g₀ U } :=
    ⟨U, hUopen, hyU, rfl⟩
  exact le_trans (scottSubspaceExtendInf_mono hfg U) (le_sSup hmem)

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

9

lemma_3_9.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.Basic
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.Constructions.scottExtend_maximal_le
public import DomainTheory.ContinuousLattice.Constructions.scottExtend_maximal
public import DomainTheory.ContinuousLattice.Constructions.scottExtend_continuous
public import DomainTheory.ContinuousLattice.Constructions.
    scottExtend_eq_of_continuous
public import DomainTheory.ContinuousLattice.Constructions.scottExtend
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

section Lemma39

variable {X Y D D' : Type*} [TopologicalSpace X] [TopologicalSpace Y]
```

```
  [CompleteLattice D] [CompleteLattice D']

/-- **Scott 1972, Lemma 3.9.** With `e : X → Y` a subspace embedding and `i, j : D ⇄
    D'` a
projection on the range, if continuous `f : X → D` and `g : X → D'` satisfy `f = j ∘ g
    `, then the
maximal extensions (Proposition 3.8) satisfy `f = j ∘ g`.

The proof is Scott's, recast through the faithful `scottExtend`:
* `j ∘ g ⊑ f`: `j ∘ g` is a continuous solution of `(j ∘ g) ∘ e = j ∘ g = f`, so
    maximality of
  `f` (`scottExtend_maximal`) gives the bound;
* `i ∘ f ⊑ g`: `(i ∘ f) ∘ e = i ∘ f = i ∘ j ∘ g ⊑ g` (since `i ∘ j ⊑ id`), so the *sub
    *-solution
  maximality of `g` (`scottExtend_maximal_le`, the remark after 3.8) gives the bound;
* combining: `f = j ∘ i ∘ f ⊑ j ∘ g ⊑ f` (using `j ∘ i = id` and monotonicity of `j`).
    -/
theorem lemma_3_9 (hD : ContinuousLattice D) (hD' : ContinuousLattice D')
    (P : IsContinuousLatticeProjection D D') (e : X → Y) (he : IsEmbedding e)
    {f : X → D} {g : X → D'} (hf : @Continuous X D _ scottTopologicalSpace f)
    (hg : @Continuous X D' _ scottTopologicalSpace g)
    (hfg : ∀ x, f x = (P.retr : D' → D) (g x)) (y : Y) :
    scottExtend e f y = (P.retr : D' → D) (scottExtend e g y) := by
  have hfbar : @Continuous Y D _ scottTopologicalSpace (scottExtend e f) :=
    scottExtend_continuous hD e f
  have hgbar : @Continuous Y D' _ scottTopologicalSpace (scottExtend e g) :=
    scottExtend_continuous hD' e g
  -- `j ∘ g` and `i ∘ f` are continuous (Scott topology is not an instance; register it
     locally so
  -- `Continuous.comp` can fire, scoped to avoid the lattice `≤` clashing with
    specialization order).
  have hjg : @Continuous Y D _ scottTopologicalSpace (fun z => (P.retr : D' → D) (
    scottExtend e g z)) := by
    let : TopologicalSpace D := scottTopologicalSpace
    let : TopologicalSpace D' := scottTopologicalSpace
    exact P.retr.continuous.comp hgbar
  have hif : @Continuous Y D' _ scottTopologicalSpace (fun z => (P.incl : D → D') (
    scottExtend e f z)) := by
    let : TopologicalSpace D := scottTopologicalSpace
    let : TopologicalSpace D' := scottTopologicalSpace
    exact P.incl.continuous.comp hfbar
  -- Step A: `j ∘ g ⊑ f`.
  have hA : ∀ z, (P.retr : D' → D) (scottExtend e g z) ≤ scottExtend e f z := by
    intro z
    refine scottExtend_maximal hD e hjg ?_ z
    intro x
    show (P.retr : D' → D) (scottExtend e g (e x)) = f x
    rw [scottExtend_eq_of_continuous hD' e he g hg x, ← hfg x]
  -- Step B: `i ∘ f ⊑ g`.
  have hB : ∀ z, (P.incl : D → D') (scottExtend e f z) ≤ scottExtend e g z := by
    intro z
    refine scottExtend_maximal_le hD' e hif ?_ z
    intro x
    show (P.incl : D → D') (scottExtend e f (e x)) ≤ g x
    rw [scottExtend_eq_of_continuous hD e he f hf x, hfg x]
    exact P.incl_retr_le (g x)
```

```
  refine le_antisymm ?_ (hA y)
  calc scottExtend e f y
      = (P.retr : D' → D) ((P.incl : D → D') (scottExtend e f y)) :=
        (P.retr_incl (scottExtend e f y)).symm
    _ ≤ (P.retr : D' → D) (scottExtend e g y) := P.retr.monotone (hB y)

end Lemma39
end DomainTheory.ContinuousLattice
```

0

lemma_3_9.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.Basic
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.Constructions.scottExtend_maximal_le
public import DomainTheory.ContinuousLattice.Constructions.scottExtend_maximal
public import DomainTheory.ContinuousLattice.Constructions.scottExtend_continuous
public import DomainTheory.ContinuousLattice.Constructions.
    scottExtend_eq_of_continuous
public import DomainTheory.ContinuousLattice.Constructions.scottExtend
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

section Lemma39

variable {X Y D D' : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  [CompleteLattice D] [CompleteLattice D']

/-- **Scott 1972, Lemma 3.9.** With `e : X → Y` a subspace embedding and `i, j : D ⇄
    D'` a
projection on the range, if continuous `f : X → D` and `g : X → D'` satisfy `f = j ∘ g
    `, then the
maximal extensions (Proposition 3.8) satisfy `f = j ∘ g`.

The proof is Scott's, recast through the faithful `scottExtend`:
* `j ∘ g ⊑ f`: `j ∘ g` is a continuous solution of `(j ∘ g) ∘ e = j ∘ g = f`, so
    maximality of
  `f` (`scottExtend_maximal`) gives the bound;
* `i ∘ f ⊑ g`: `(i ∘ f) ∘ e = i ∘ f = i ∘ j ∘ g ⊑ g` (since `i ∘ j ⊑ id`), so the *sub
    *-solution
  maximality of `g` (`scottExtend_maximal_le`, the remark after 3.8) gives the bound;
* combining: `f = j ∘ i ∘ f ⊑ j ∘ g ⊑ f` (using `j ∘ i = id` and monotonicity of `j`).
    -/
theorem lemma_3_9 (hD : ContinuousLattice D) (hD' : ContinuousLattice D')
    (P : IsContinuousLatticeProjection D D') (e : X → Y) (he : IsEmbedding e)
```

```
    {f : X → D} {g : X → D'} (hf : @Continuous X D _ scottTopologicalSpace f)
    (hg : @Continuous X D' _ scottTopologicalSpace g)
    (hfg : ∀ x, f x = (P.retr : D' → D) (g x)) (y : Y) :
    scottExtend e f y = (P.retr : D' → D) (scottExtend e g y) := by
  have hfbar : @Continuous Y D _ scottTopologicalSpace (scottExtend e f) :=
    scottExtend_continuous hD e f
  have hgbar : @Continuous Y D' _ scottTopologicalSpace (scottExtend e g) :=
    scottExtend_continuous hD' e g
  -- `j ∘ g` and `i ∘ f` are continuous (Scott topology is not an instance; register it
     locally so
  -- `Continuous.comp` can fire, scoped to avoid the lattice `≤` clashing with
    specialization order).
  have hjg : @Continuous Y D _ scottTopologicalSpace (fun z => (P.retr : D' → D) (
    scottExtend e g z)) := by
    let : TopologicalSpace D := scottTopologicalSpace
    let : TopologicalSpace D' := scottTopologicalSpace
    exact P.retr.continuous.comp hgbar
  have hif : @Continuous Y D' _ scottTopologicalSpace (fun z => (P.incl : D → D') (
    scottExtend e f z)) := by
    let : TopologicalSpace D := scottTopologicalSpace
    let : TopologicalSpace D' := scottTopologicalSpace
    exact P.incl.continuous.comp hfbar
  -- Step A: `j ∘ g ⊑ f`.
  have hA : ∀ z, (P.retr : D' → D) (scottExtend e g z) ≤ scottExtend e f z := by
    intro z
    refine scottExtend_maximal hD e hjg ?_ z
    intro x
    show (P.retr : D' → D) (scottExtend e g (e x)) = f x
    rw [scottExtend_eq_of_continuous hD' e he g hg x, ← hfg x]
  -- Step B: `i ∘ f ⊑ g`.
  have hB : ∀ z, (P.incl : D → D') (scottExtend e f z) ≤ scottExtend e g z := by
    intro z
    refine scottExtend_maximal_le hD' e hif ?_ z
    intro x
    show (P.incl : D → D') (scottExtend e f (e x)) ≤ g x
    rw [scottExtend_eq_of_continuous hD e he f hf x, hfg x]
    exact P.incl_retr_le (g x)
  refine le_antisymm ?_ (hA y)
  calc scottExtend e f y
      = (P.retr : D' → D) ((P.incl : D → D') (scottExtend e f y)) :=
        (P.retr_incl (scottExtend e f y)).symm
    _ ≤ (P.retr : D' → D) (scottExtend e g y) := P.retr.monotone (hB y)

end Lemma39
end DomainTheory.ContinuousLattice
```

1

lemma_3_9.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.Basic
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
```

```
public import DomainTheory.ContinuousLattice.Constructions.scottExtend_maximal_le
public import DomainTheory.ContinuousLattice.Constructions.scottExtend_maximal
public import DomainTheory.ContinuousLattice.Constructions.scottExtend_continuous
public import DomainTheory.ContinuousLattice.Constructions.
    scottExtend_eq_of_continuous
public import DomainTheory.ContinuousLattice.Constructions.scottExtend
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

section Lemma39

variable {X Y D D' : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  [CompleteLattice D] [CompleteLattice D']

/-- **Scott 1972, Lemma 3.9.** With `e : X → Y` a subspace embedding and `i, j : D ⇄
    D'` a
projection on the range, if continuous `f : X → D` and `g : X → D'` satisfy `f = j ∘ g
    `, then the
maximal extensions (Proposition 3.8) satisfy `f = j ∘ g`.

The proof is Scott's, recast through the faithful `scottExtend`:
* `j ∘ g ⊑ f`: `j ∘ g` is a continuous solution of `(j ∘ g) ∘ e = j ∘ g = f`, so
    maximality of
  `f` (`scottExtend_maximal`) gives the bound;
* `i ∘ f ⊑ g`: `(i ∘ f) ∘ e = i ∘ f = i ∘ j ∘ g ⊑ g` (since `i ∘ j ⊑ id`), so the *sub
    *-solution
  maximality of `g` (`scottExtend_maximal_le`, the remark after 3.8) gives the bound;
* combining: `f = j ∘ i ∘ f ⊑ j ∘ g ⊑ f` (using `j ∘ i = id` and monotonicity of `j`).
    -/
theorem lemma_3_9 (hD : ContinuousLattice D) (hD' : ContinuousLattice D')
    (P : IsContinuousLatticeProjection D D') (e : X → Y) (he : IsEmbedding e)
    {f : X → D} {g : X → D'} (hf : @Continuous X D _ scottTopologicalSpace f)
    (hg : @Continuous X D' _ scottTopologicalSpace g)
    (hfg : ∀ x, f x = (P.retr : D' → D) (g x)) (y : Y) :
    scottExtend e f y = (P.retr : D' → D) (scottExtend e g y) := by
  have hfbar : @Continuous Y D _ scottTopologicalSpace (scottExtend e f) :=
    scottExtend_continuous hD e f
  have hgbar : @Continuous Y D' _ scottTopologicalSpace (scottExtend e g) :=
    scottExtend_continuous hD' e g
  -- `j ∘ g` and `i ∘ f` are continuous (Scott topology is not an instance; register it
     locally so
  -- `Continuous.comp` can fire, scoped to avoid the lattice `≤` clashing with
    specialization order).
  have hjg : @Continuous Y D _ scottTopologicalSpace (fun z => (P.retr : D' → D) (
    scottExtend e g z)) := by
    let : TopologicalSpace D := scottTopologicalSpace
    let : TopologicalSpace D' := scottTopologicalSpace
    exact P.retr.continuous.comp hgbar
  have hif : @Continuous Y D' _ scottTopologicalSpace (fun z => (P.incl : D → D') (
    scottExtend e f z)) := by
```

```
    let : TopologicalSpace D := scottTopologicalSpace
    let : TopologicalSpace D' := scottTopologicalSpace
    exact P.incl.continuous.comp hfbar
  -- Step A: `j ∘ g ⊑ f`.
  have hA : ∀ z, (P.retr : D' → D) (scottExtend e g z) ≤ scottExtend e f z := by
    intro z
    refine scottExtend_maximal hD e hjg ?_ z
    intro x
    show (P.retr : D' → D) (scottExtend e g (e x)) = f x
    rw [scottExtend_eq_of_continuous hD' e he g hg x, ← hfg x]
  -- Step B: `i ∘ f ⊑ g`.
  have hB : ∀ z, (P.incl : D → D') (scottExtend e f z) ≤ scottExtend e g z := by
    intro z
    refine scottExtend_maximal_le hD' e hif ?_ z
    intro x
    show (P.incl : D → D') (scottExtend e f (e x)) ≤ g x
    rw [scottExtend_eq_of_continuous hD e he f hf x, hfg x]
    exact P.incl_retr_le (g x)
  refine le_antisymm ?_ (hA y)
  calc scottExtend e f y
      = (P.retr : D' → D) ((P.incl : D → D') (scottExtend e f y)) :=
        (P.retr_incl (scottExtend e f y)).symm
    _ ≤ (P.retr : D' → D) (scottExtend e g y) := P.retr.monotone (hB y)

end Lemma39
end DomainTheory.ContinuousLattice
```

2

lemma_3_9.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.Basic
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.Constructions.scottExtend_maximal_le
public import DomainTheory.ContinuousLattice.Constructions.scottExtend_maximal
public import DomainTheory.ContinuousLattice.Constructions.scottExtend_continuous
public import DomainTheory.ContinuousLattice.Constructions.
    scottExtend_eq_of_continuous
public import DomainTheory.ContinuousLattice.Constructions.scottExtend
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

section Lemma39

variable {X Y D D' : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  [CompleteLattice D] [CompleteLattice D']
```

```
/-- **Scott 1972, Lemma 3.9.** With `e : X → Y` a subspace embedding and `i, j : D ⇄
    D'` a
projection on the range, if continuous `f : X → D` and `g : X → D'` satisfy `f = j ∘ g
    `, then the
maximal extensions (Proposition 3.8) satisfy `f = j ∘ g`.

The proof is Scott's, recast through the faithful `scottExtend`:
* `j ∘ g ⊑ f`: `j ∘ g` is a continuous solution of `(j ∘ g) ∘ e = j ∘ g = f`, so
    maximality of
  `f` (`scottExtend_maximal`) gives the bound;
* `i ∘ f ⊑ g`: `(i ∘ f) ∘ e = i ∘ f = i ∘ j ∘ g ⊑ g` (since `i ∘ j ⊑ id`), so the *sub
    *-solution
  maximality of `g` (`scottExtend_maximal_le`, the remark after 3.8) gives the bound;
* combining: `f = j ∘ i ∘ f ⊑ j ∘ g ⊑ f` (using `j ∘ i = id` and monotonicity of `j`).
    -/
theorem lemma_3_9 (hD : ContinuousLattice D) (hD' : ContinuousLattice D')
    (P : IsContinuousLatticeProjection D D') (e : X → Y) (he : IsEmbedding e)
    {f : X → D} {g : X → D'} (hf : @Continuous X D _ scottTopologicalSpace f)
    (hg : @Continuous X D' _ scottTopologicalSpace g)
    (hfg : ∀ x, f x = (P.retr : D' → D) (g x)) (y : Y) :
    scottExtend e f y = (P.retr : D' → D) (scottExtend e g y) := by
  have hfbar : @Continuous Y D _ scottTopologicalSpace (scottExtend e f) :=
    scottExtend_continuous hD e f
  have hgbar : @Continuous Y D' _ scottTopologicalSpace (scottExtend e g) :=
    scottExtend_continuous hD' e g
  -- `j ∘ g` and `i ∘ f` are continuous (Scott topology is not an instance; register it
     locally so
  -- `Continuous.comp` can fire, scoped to avoid the lattice `≤` clashing with
    specialization order).
  have hjg : @Continuous Y D _ scottTopologicalSpace (fun z => (P.retr : D' → D) (
    scottExtend e g z)) := by
    let : TopologicalSpace D := scottTopologicalSpace
    let : TopologicalSpace D' := scottTopologicalSpace
    exact P.retr.continuous.comp hgbar
  have hif : @Continuous Y D' _ scottTopologicalSpace (fun z => (P.incl : D → D') (
    scottExtend e f z)) := by
    let : TopologicalSpace D := scottTopologicalSpace
    let : TopologicalSpace D' := scottTopologicalSpace
    exact P.incl.continuous.comp hfbar
  -- Step A: `j ∘ g ⊑ f`.
  have hA : ∀ z, (P.retr : D' → D) (scottExtend e g z) ≤ scottExtend e f z := by
    intro z
    refine scottExtend_maximal hD e hjg ?_ z
    intro x
    show (P.retr : D' → D) (scottExtend e g (e x)) = f x
    rw [scottExtend_eq_of_continuous hD' e he g hg x, ← hfg x]
  -- Step B: `i ∘ f ⊑ g`.
  have hB : ∀ z, (P.incl : D → D') (scottExtend e f z) ≤ scottExtend e g z := by
    intro z
    refine scottExtend_maximal_le hD' e hif ?_ z
    intro x
    show (P.incl : D → D') (scottExtend e f (e x)) ≤ g x
    rw [scottExtend_eq_of_continuous hD e he f hf x, hfg x]
    exact P.incl_retr_le (g x)
  refine le_antisymm ?_ (hA y)
```

```
  calc scottExtend e f y
      = (P.retr : D' → D) ((P.incl : D → D') (scottExtend e f y)) :=
        (P.retr_incl (scottExtend e f y)).symm
    _ ≤ (P.retr : D' → D) (scottExtend e g y) := P.retr.monotone (hB y)

end Lemma39
end DomainTheory.ContinuousLattice
```

3

lemma_3_9.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.Basic
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.Constructions.scottExtend_maximal_le
public import DomainTheory.ContinuousLattice.Constructions.scottExtend_maximal
public import DomainTheory.ContinuousLattice.Constructions.scottExtend_continuous
public import DomainTheory.ContinuousLattice.Constructions.
    scottExtend_eq_of_continuous
public import DomainTheory.ContinuousLattice.Constructions.scottExtend
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

section Lemma39

variable {X Y D D' : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  [CompleteLattice D] [CompleteLattice D']

/-- **Scott 1972, Lemma 3.9.** With `e : X → Y` a subspace embedding and `i, j : D ⇄
    D'` a
projection on the range, if continuous `f : X → D` and `g : X → D'` satisfy `f = j ∘ g
    `, then the
maximal extensions (Proposition 3.8) satisfy `f = j ∘ g`.

The proof is Scott's, recast through the faithful `scottExtend`:
* `j ∘ g ⊑ f`: `j ∘ g` is a continuous solution of `(j ∘ g) ∘ e = j ∘ g = f`, so
    maximality of
  `f` (`scottExtend_maximal`) gives the bound;
* `i ∘ f ⊑ g`: `(i ∘ f) ∘ e = i ∘ f = i ∘ j ∘ g ⊑ g` (since `i ∘ j ⊑ id`), so the *sub
    *-solution
  maximality of `g` (`scottExtend_maximal_le`, the remark after 3.8) gives the bound;
* combining: `f = j ∘ i ∘ f ⊑ j ∘ g ⊑ f` (using `j ∘ i = id` and monotonicity of `j`).
    -/
theorem lemma_3_9 (hD : ContinuousLattice D) (hD' : ContinuousLattice D')
    (P : IsContinuousLatticeProjection D D') (e : X → Y) (he : IsEmbedding e)
    {f : X → D} {g : X → D'} (hf : @Continuous X D _ scottTopologicalSpace f)
```

```
    (hg : @Continuous X D' _ scottTopologicalSpace g)
    (hfg : ∀ x, f x = (P.retr : D' → D) (g x)) (y : Y) :
    scottExtend e f y = (P.retr : D' → D) (scottExtend e g y) := by
  have hfbar : @Continuous Y D _ scottTopologicalSpace (scottExtend e f) :=
    scottExtend_continuous hD e f
  have hgbar : @Continuous Y D' _ scottTopologicalSpace (scottExtend e g) :=
    scottExtend_continuous hD' e g
  -- `j ∘ g` and `i ∘ f` are continuous (Scott topology is not an instance; register it
     locally so
  -- `Continuous.comp` can fire, scoped to avoid the lattice `≤` clashing with
    specialization order).
  have hjg : @Continuous Y D _ scottTopologicalSpace (fun z => (P.retr : D' → D) (
    scottExtend e g z)) := by
    let : TopologicalSpace D := scottTopologicalSpace
    let : TopologicalSpace D' := scottTopologicalSpace
    exact P.retr.continuous.comp hgbar
  have hif : @Continuous Y D' _ scottTopologicalSpace (fun z => (P.incl : D → D') (
    scottExtend e f z)) := by
    let : TopologicalSpace D := scottTopologicalSpace
    let : TopologicalSpace D' := scottTopologicalSpace
    exact P.incl.continuous.comp hfbar
  -- Step A: `j ∘ g ⊑ f`.
  have hA : ∀ z, (P.retr : D' → D) (scottExtend e g z) ≤ scottExtend e f z := by
    intro z
    refine scottExtend_maximal hD e hjg ?_ z
    intro x
    show (P.retr : D' → D) (scottExtend e g (e x)) = f x
    rw [scottExtend_eq_of_continuous hD' e he g hg x, ← hfg x]
  -- Step B: `i ∘ f ⊑ g`.
  have hB : ∀ z, (P.incl : D → D') (scottExtend e f z) ≤ scottExtend e g z := by
    intro z
    refine scottExtend_maximal_le hD' e hif ?_ z
    intro x
    show (P.incl : D → D') (scottExtend e f (e x)) ≤ g x
    rw [scottExtend_eq_of_continuous hD e he f hf x, hfg x]
    exact P.incl_retr_le (g x)
  refine le_antisymm ?_ (hA y)
  calc scottExtend e f y
      = (P.retr : D' → D) ((P.incl : D → D') (scottExtend e f y)) :=
        (P.retr_incl (scottExtend e f y)).symm
    _ ≤ (P.retr : D' → D) (scottExtend e g y) := P.retr.monotone (hB y)

end Lemma39
end DomainTheory.ContinuousLattice
```

4

lemma_3_9.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.Basic
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.Constructions.scottExtend_maximal_le
```

```
public import DomainTheory.ContinuousLattice.Constructions.scottExtend_maximal
public import DomainTheory.ContinuousLattice.Constructions.scottExtend_continuous
public import DomainTheory.ContinuousLattice.Constructions.
    scottExtend_eq_of_continuous
public import DomainTheory.ContinuousLattice.Constructions.scottExtend
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

section Lemma39

variable {X Y D D' : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  [CompleteLattice D] [CompleteLattice D']

/-- **Scott 1972, Lemma 3.9.** With `e : X → Y` a subspace embedding and `i, j : D ⇄
    D'` a
projection on the range, if continuous `f : X → D` and `g : X → D'` satisfy `f = j ∘ g
    `, then the
maximal extensions (Proposition 3.8) satisfy `f = j ∘ g`.

The proof is Scott's, recast through the faithful `scottExtend`:
* `j ∘ g ⊑ f`: `j ∘ g` is a continuous solution of `(j ∘ g) ∘ e = j ∘ g = f`, so
    maximality of
  `f` (`scottExtend_maximal`) gives the bound;
* `i ∘ f ⊑ g`: `(i ∘ f) ∘ e = i ∘ f = i ∘ j ∘ g ⊑ g` (since `i ∘ j ⊑ id`), so the *sub
    *-solution
  maximality of `g` (`scottExtend_maximal_le`, the remark after 3.8) gives the bound;
* combining: `f = j ∘ i ∘ f ⊑ j ∘ g ⊑ f` (using `j ∘ i = id` and monotonicity of `j`).
    -/
theorem lemma_3_9 (hD : ContinuousLattice D) (hD' : ContinuousLattice D')
    (P : IsContinuousLatticeProjection D D') (e : X → Y) (he : IsEmbedding e)
    {f : X → D} {g : X → D'} (hf : @Continuous X D _ scottTopologicalSpace f)
    (hg : @Continuous X D' _ scottTopologicalSpace g)
    (hfg : ∀ x, f x = (P.retr : D' → D) (g x)) (y : Y) :
    scottExtend e f y = (P.retr : D' → D) (scottExtend e g y) := by
  have hfbar : @Continuous Y D _ scottTopologicalSpace (scottExtend e f) :=
    scottExtend_continuous hD e f
  have hgbar : @Continuous Y D' _ scottTopologicalSpace (scottExtend e g) :=
    scottExtend_continuous hD' e g
  -- `j ∘ g` and `i ∘ f` are continuous (Scott topology is not an instance; register it
     locally so
  -- `Continuous.comp` can fire, scoped to avoid the lattice `≤` clashing with
    specialization order).
  have hjg : @Continuous Y D _ scottTopologicalSpace (fun z => (P.retr : D' → D) (
    scottExtend e g z)) := by
    let : TopologicalSpace D := scottTopologicalSpace
    let : TopologicalSpace D' := scottTopologicalSpace
    exact P.retr.continuous.comp hgbar
  have hif : @Continuous Y D' _ scottTopologicalSpace (fun z => (P.incl : D → D') (
    scottExtend e f z)) := by
    let : TopologicalSpace D := scottTopologicalSpace
```

```
    let : TopologicalSpace D' := scottTopologicalSpace
    exact P.incl.continuous.comp hfbar
  -- Step A: `j ∘ g ⊑ f`.
  have hA : ∀ z, (P.retr : D' → D) (scottExtend e g z) ≤ scottExtend e f z := by
    intro z
    refine scottExtend_maximal hD e hjg ?_ z
    intro x
    show (P.retr : D' → D) (scottExtend e g (e x)) = f x
    rw [scottExtend_eq_of_continuous hD' e he g hg x, ← hfg x]
  -- Step B: `i ∘ f ⊑ g`.
  have hB : ∀ z, (P.incl : D → D') (scottExtend e f z) ≤ scottExtend e g z := by
    intro z
    refine scottExtend_maximal_le hD' e hif ?_ z
    intro x
    show (P.incl : D → D') (scottExtend e f (e x)) ≤ g x
    rw [scottExtend_eq_of_continuous hD e he f hf x, hfg x]
    exact P.incl_retr_le (g x)
  refine le_antisymm ?_ (hA y)
  calc scottExtend e f y
      = (P.retr : D' → D) ((P.incl : D → D') (scottExtend e f y)) :=
        (P.retr_incl (scottExtend e f y)).symm
    _ ≤ (P.retr : D' → D) (scottExtend e g y) := P.retr.monotone (hB y)

end Lemma39
end DomainTheory.ContinuousLattice
```

5

lemma_3_9.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.Basic
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.Constructions.scottExtend_maximal_le
public import DomainTheory.ContinuousLattice.Constructions.scottExtend_maximal
public import DomainTheory.ContinuousLattice.Constructions.scottExtend_continuous
public import DomainTheory.ContinuousLattice.Constructions.
    scottExtend_eq_of_continuous
public import DomainTheory.ContinuousLattice.Constructions.scottExtend
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

section Lemma39

variable {X Y D D' : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  [CompleteLattice D] [CompleteLattice D']
```

```
/-- **Scott 1972, Lemma 3.9.** With `e : X → Y` a subspace embedding and `i, j : D ⇄
    D'` a
projection on the range, if continuous `f : X → D` and `g : X → D'` satisfy `f = j ∘ g
    `, then the
maximal extensions (Proposition 3.8) satisfy `f = j ∘ g`.

The proof is Scott's, recast through the faithful `scottExtend`:
* `j ∘ g ⊑ f`: `j ∘ g` is a continuous solution of `(j ∘ g) ∘ e = j ∘ g = f`, so
    maximality of
  `f` (`scottExtend_maximal`) gives the bound;
* `i ∘ f ⊑ g`: `(i ∘ f) ∘ e = i ∘ f = i ∘ j ∘ g ⊑ g` (since `i ∘ j ⊑ id`), so the *sub
    *-solution
  maximality of `g` (`scottExtend_maximal_le`, the remark after 3.8) gives the bound;
* combining: `f = j ∘ i ∘ f ⊑ j ∘ g ⊑ f` (using `j ∘ i = id` and monotonicity of `j`).
    -/
theorem lemma_3_9 (hD : ContinuousLattice D) (hD' : ContinuousLattice D')
    (P : IsContinuousLatticeProjection D D') (e : X → Y) (he : IsEmbedding e)
    {f : X → D} {g : X → D'} (hf : @Continuous X D _ scottTopologicalSpace f)
    (hg : @Continuous X D' _ scottTopologicalSpace g)
    (hfg : ∀ x, f x = (P.retr : D' → D) (g x)) (y : Y) :
    scottExtend e f y = (P.retr : D' → D) (scottExtend e g y) := by
  have hfbar : @Continuous Y D _ scottTopologicalSpace (scottExtend e f) :=
    scottExtend_continuous hD e f
  have hgbar : @Continuous Y D' _ scottTopologicalSpace (scottExtend e g) :=
    scottExtend_continuous hD' e g
  -- `j ∘ g` and `i ∘ f` are continuous (Scott topology is not an instance; register it
     locally so
  -- `Continuous.comp` can fire, scoped to avoid the lattice `≤` clashing with
    specialization order).
  have hjg : @Continuous Y D _ scottTopologicalSpace (fun z => (P.retr : D' → D) (
    scottExtend e g z)) := by
    let : TopologicalSpace D := scottTopologicalSpace
    let : TopologicalSpace D' := scottTopologicalSpace
    exact P.retr.continuous.comp hgbar
  have hif : @Continuous Y D' _ scottTopologicalSpace (fun z => (P.incl : D → D') (
    scottExtend e f z)) := by
    let : TopologicalSpace D := scottTopologicalSpace
    let : TopologicalSpace D' := scottTopologicalSpace
    exact P.incl.continuous.comp hfbar
  -- Step A: `j ∘ g ⊑ f`.
  have hA : ∀ z, (P.retr : D' → D) (scottExtend e g z) ≤ scottExtend e f z := by
    intro z
    refine scottExtend_maximal hD e hjg ?_ z
    intro x
    show (P.retr : D' → D) (scottExtend e g (e x)) = f x
    rw [scottExtend_eq_of_continuous hD' e he g hg x, ← hfg x]
  -- Step B: `i ∘ f ⊑ g`.
  have hB : ∀ z, (P.incl : D → D') (scottExtend e f z) ≤ scottExtend e g z := by
    intro z
    refine scottExtend_maximal_le hD' e hif ?_ z
    intro x
    show (P.incl : D → D') (scottExtend e f (e x)) ≤ g x
    rw [scottExtend_eq_of_continuous hD e he f hf x, hfg x]
    exact P.incl_retr_le (g x)
  refine le_antisymm ?_ (hA y)
  calc scottExtend e f y
```

```
      = (P.retr : D' → D) ((P.incl : D → D') (scottExtend e f y)) :=
        (P.retr_incl (scottExtend e f y)).symm
    _ ≤ (P.retr : D' → D) (scottExtend e g y) := P.retr.monotone (hB y)

end Lemma39
end DomainTheory.ContinuousLattice
```

6

5.4 Constructivity summary

Bridge	Footprint	Notes
Nbhd ↔ InfoSys (both directions)	+ inherited `Classical.choice`	Decidable coding; Mathlib 4.33 `Finset` instance frontier
CL ↔ RoundFilter	`{propext, Quot.sound}`	Roundness is order-theoretic
InfoSys ↔ Ideal	+ inherited `Classical.choice`	Factoids 4.4–4.5; type-level `Finset` frontier
Algebraic ⇒ CL	classical	1972 topological $\ll$
Product / function space (1982)	+ inherited `Classical.choice`	No witness extraction in the maps
Separated sum (1982)	+ `Classical.choice`	Token polarity trichotomy
Factoid 4.6 ApproxMap ↔ ScottContinuous	+ inherited `Classical.choice`	Type-level `Finset` frontier
ScottMap conjugation	+ `Classical.choice`	Via 1972 `ScottMap`

Target discipline (from the 1982 package): prefer constructive proofs wherever Scott's 1982 text emphasizes constructivity; call out classical frontiers explicitly rather than hiding choice in automation.

Mathlib 4.33 makes `Finset.instSetLike` and `Finset.instPartialOrder` depend on `Classical.choice`. Consequently `#print axioms` reports choice for declarations whose types traverse `InfoSys`, even when their proof terms only manipulate explicit finite data. Here "no witness extraction" distinguishes that inherited library-instance footprint from genuine uses such as separated-sum polarity classification.

5.5 Worked example — S-expressions / trees

Dependencies. Inside WorkedExampleSExpr, each box is one Lean file. A box named as another folder in this chapter is that whole folder. A box named as another chapter is that whole chapter. BridgeTheorems is the Lean files in that chapter folder, not in a subfolder. The arrowhead points at the file or folder that imports, and the number is how many direct imports.

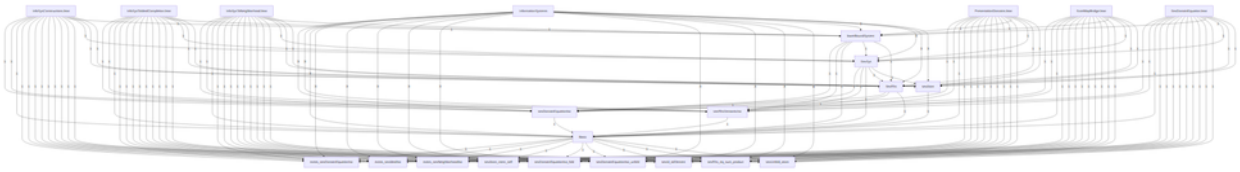


Figure 47: Worked example — S-expressions / trees.

Lean packaging: `WorkedExampleSExpr.lean`, `SexDomainEquation.lean`. The Palomar compared family includes `sexNeighborhoodIso`, `sexIdealIso`, and `sexDomainEquationIso` (Challenge / `comparator.json`). The compared domain-equation declaration is the order isomorphism `SexSys.Element ≃o SexRhs.Element` (`|T| ≃o |A + (T × T)|`), not the generic semantic factor `WithBot (|A| ⊕ (|T| × |T|)) ≃o |A + (T × T)|`.

5.5.1 Overview

The bridges of §§2–3 are abstract. This section fixes **one** concrete domain — Scott's S-expression / tree equation `T ≅ A + (T × T)` (**[Sco82]**, Factoid 8.1) over the $\mathbb{N}$ lower-bound atom system (Factoid 2.4) — and asks what that domain looks like in each paradigm:

Paradigm	What an element of `T` is
1982	a consistent, deductively closed set of tree tokens
1980	a filter of basic opens `[u]` on that token domain
1972	a point of an algebraic continuous lattice (ideal of finite elements)

We build the domain natively as an information system (§5.2), then **read the same carrier** as a neighbourhood system (§5.3) and as a continuous lattice (§5.4). Section 5.5 exhibits the order isomorphisms for this instance — so that, on S-expressions,

`T_1982 ≃o T_1980 ≃o Ideal(K(T))` and `Ideal(K(T))` is a continuous lattice,

hence transitively `T_1982 ≃o T_1980` and (via the algebraic ⇒ continuous frontier) the 1972 presentation of the same domain. Order isomorphism is an equivalence relation, so the three presentations determine one domain up to `≃o`. Section 5.6 records two extras that live on top of the carriers: the domain equation at the level of domains, and identity as a morphism in both the approximable-map and Scott-continuous languages.

5.5.2 Information system (1982)

Atoms are propositions `n ≤ x` on $\mathbb{N}$ (`lowerBoundSystem`). The tree system `SexSys := treeSystem lowerBoundSystem` has inductive tokens `TreeToken ℕ` (`bot` / `atom` / `pairL` / `pairR`). Consistency and entailment are Scott's sum×product clauses; Factoid 8.1 records that this is literally the information system of the right-hand side `A + (T × T)`:

- `SexRhs = sumSystem A (productSystem T T)` (`sexRhs_eq_sum_product`);
- token unfolding `treeUnfold` sends `atom n` to the left summand (`sexUnfold_atom`) and is a retraction onto the official sum-of-product carrier, with section `treeFold`;

- TreeEnt matches official product `ent_bot` on the two encodings of `(Δ, Δ)` (`pairL Δ` and `pairR Δ`), so closed elements are saturated for `ker(treeUnfold)`.

Finite elements include singleton atom closures `sexAtom n = ū` for `u = {atom n}`. Elements of `|T|` are the consistent closed sets of tokens — the 1982 presentation of the S-expression domain.

5.5.3 Neighbourhood system (1980)

From `|T|` one obtains a neighbourhood system whose basic opens are `[u] = {x ∈ |T| | ↑u ⊆ x}`. In this paradigm an element of the domain is a **filter** of such opens (not a raw set of tokens). Nothing new is invented: it is the image of `SexSys` under `InfoSysToNeighborhood`. The concrete iso of §5.5 below is exactly "read each token-element as its filter of basic opens."

5.5.4 Continuous lattice (1972)

Finite elements `K(T)` are the closures of consistent finsets. In the algebraic style that feeds Scott's 1972 theory, a domain element is an **ideal** of `K(T)` — the ideal completion of the compact basis. That poset of ideals is an algebraic complete lattice, hence a continuous lattice (`AlgebraicLattice ⇒ ContinuousLattice`, §3.5). This is the 1972 presentation of the same S-expression domain: points ordered by inclusion of ideals, with way-below recovered from the compact basis.

We do not claim a choice-free `D ≃o RoundFilter` story built from trees alone; the round ⇑-filter identification of §3.1 applies once one is already in the continuous-lattice setting.

5.5.5 Equivalence for this instance

Specializing the bridges of §3 to `SexSys` yields order isomorphisms of carriers:

- `sexNeighborhoodIso : |T| ≃o` filters of `[u]` (`InfoSysToNeighborhood.domainOrderIso`) — **1982 ≃o 1980**;
- `sexIdealIso : |T| ≃o Ideal (FiniteElement T)` (`InfoSysToIdealCompletion.domainOr`) — **1982 ≃o algebraic / 1972 carrier**;
- `sexNeighborhoodIdealIso` (`neighborhood_ideal_iso`) — the constructive triangle closing **1980 ≃o Ideal(K(T))** without going back through tokens.

Composing any two edges recovers the third: isomorphism is transitive, so on this example the three presentations are pairwise equivalent. The 1972 corner uses the algebraic ⇒ continuous implication already flagged in §3.5 / §4. The neighbourhood ↔ information ↔ ideal triangle inherits `Classical.choice` only through the `Finset` instance frontier of §4.

5.5.6 Domain equation and morphisms

Beyond identifying the carriers, the same example exercises constructions and maps.

Domain equation. Factoid 8.1's token unfolding `treeUnfold` (section `treeFold`) lifts to an order isomorphism of the tree domain with the official RHS domain:

`sexDomainEquationIso : SexSys.Element ≃o SexRhs.Element`

(treeDomainIso; |T| ≃o |A + (T × T)|). The maps are verified unfolding: the carrier of sexDomainEquationIso x is treeUnfold ″ x.carrier, and the carrier of the inverse is the preimage treeUnfold ⁻¹′ y.carrier (sexDomainEquationIso_unfold / sexDomainEquationIso_fold). The only non-injectivity of treeUnfold is Scott's two encodings of the product bottom (pairL Δ and pairR Δ); after the ent_bot match, closed elements are saturated for that kernel, so image and preimage invert. The generic semantic factor WithBot (|A| ⊕ (|T| × |T|)) ≃o SexRhs.Element is the uncompared sexRhsSemanticIso (classical only through the 1982 sum trichotomy); it is **not** the compared fixed-point equation.

Morphisms. The identity approximable map idMap T (Prop 5.4) transports along Factoid 4.6 to a Scott-continuous endomap sexIdScottContinuous with toFun = id (sexId_toElement) — the same endomap, named once in each morphism language.

Axioms. Object-level neighbourhood / ideal isos, sexDomainEquationIso, and Factoid 4.6 for idMap audit to {propext, Classical.choice, Quot.sound} through the InfoSys/Finset frontier of §4. The uncompared semantic factor sexRhsSemanticIso additionally uses the 1982 sum trichotomy.

lemma_3_9.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.Basic
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.Constructions.scottExtend_maximal_le
public import DomainTheory.ContinuousLattice.Constructions.scottExtend_maximal
public import DomainTheory.ContinuousLattice.Constructions.scottExtend_continuous
public import DomainTheory.ContinuousLattice.Constructions.
    scottExtend_eq_of_continuous
public import DomainTheory.ContinuousLattice.Constructions.scottExtend
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

section Lemma39

variable {X Y D D' : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  [CompleteLattice D] [CompleteLattice D']

/-- **Scott 1972, Lemma 3.9.** With `e : X → Y` a subspace embedding and `i, j : D ⇄
    D'` a
projection on the range, if continuous `f : X → D` and `g : X → D'` satisfy `f = j ∘ g
    `, then the
maximal extensions (Proposition 3.8) satisfy `f = j ∘ g`.

The proof is Scott's, recast through the faithful `scottExtend`:
* `j ∘ g ⊑ f`: `j ∘ g` is a continuous solution of `(j ∘ g) ∘ e = j ∘ g = f`, so
    maximality of
```

```
  `f` (`scottExtend_maximal`) gives the bound;
* `i ∘ f ⊑ g`: `(i ∘ f) ∘ e = i ∘ f = i ∘ j ∘ g ⊑ g` (since `i ∘ j ⊑ id`), so the *sub
    *-solution
  maximality of `g` (`scottExtend_maximal_le`, the remark after 3.8) gives the bound;
* combining: `f = j ∘ i ∘ f ⊑ j ∘ g ⊑ f` (using `j ∘ i = id` and monotonicity of `j`).
    -/
theorem lemma_3_9 (hD : ContinuousLattice D) (hD' : ContinuousLattice D')
    (P : IsContinuousLatticeProjection D D') (e : X → Y) (he : IsEmbedding e)
    {f : X → D} {g : X → D'} (hf : @Continuous X D _ scottTopologicalSpace f)
    (hg : @Continuous X D' _ scottTopologicalSpace g)
    (hfg : ∀ x, f x = (P.retr : D' → D) (g x)) (y : Y) :
    scottExtend e f y = (P.retr : D' → D) (scottExtend e g y) := by
  have hfbar : @Continuous Y D _ scottTopologicalSpace (scottExtend e f) :=
    scottExtend_continuous hD e f
  have hgbar : @Continuous Y D' _ scottTopologicalSpace (scottExtend e g) :=
    scottExtend_continuous hD' e g
  -- `j ∘ g` and `i ∘ f` are continuous (Scott topology is not an instance; register it
     locally so
  -- `Continuous.comp` can fire, scoped to avoid the lattice `≤` clashing with
    specialization order).
  have hjg : @Continuous Y D _ scottTopologicalSpace (fun z => (P.retr : D' → D) (
    scottExtend e g z)) := by
    let : TopologicalSpace D := scottTopologicalSpace
    let : TopologicalSpace D' := scottTopologicalSpace
    exact P.retr.continuous.comp hgbar
  have hif : @Continuous Y D' _ scottTopologicalSpace (fun z => (P.incl : D → D') (
    scottExtend e f z)) := by
    let : TopologicalSpace D := scottTopologicalSpace
    let : TopologicalSpace D' := scottTopologicalSpace
    exact P.incl.continuous.comp hfbar
  -- Step A: `j ∘ g ⊑ f`.
  have hA : ∀ z, (P.retr : D' → D) (scottExtend e g z) ≤ scottExtend e f z := by
    intro z
    refine scottExtend_maximal hD e hjg ?_ z
    intro x
    show (P.retr : D' → D) (scottExtend e g (e x)) = f x
    rw [scottExtend_eq_of_continuous hD' e he g hg x, ← hfg x]
  -- Step B: `i ∘ f ⊑ g`.
  have hB : ∀ z, (P.incl : D → D') (scottExtend e f z) ≤ scottExtend e g z := by
    intro z
    refine scottExtend_maximal_le hD' e hif ?_ z
    intro x
    show (P.incl : D → D') (scottExtend e f (e x)) ≤ g x
    rw [scottExtend_eq_of_continuous hD e he f hf x, hfg x]
    exact P.incl_retr_le (g x)
  refine le_antisymm ?_ (hA y)
  calc scottExtend e f y
      = (P.retr : D' → D) ((P.incl : D → D') (scottExtend e f y)) :=
        (P.retr_incl (scottExtend e f y)).symm
    _ ≤ (P.retr : D' → D) (scottExtend e g y) := P.retr.monotone (hB y)

end Lemma39
end DomainTheory.ContinuousLattice
```

lemma_3_9.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.Basic
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.Constructions.scottExtend_maximal_le
public import DomainTheory.ContinuousLattice.Constructions.scottExtend_maximal
public import DomainTheory.ContinuousLattice.Constructions.scottExtend_continuous
public import DomainTheory.ContinuousLattice.Constructions.
    scottExtend_eq_of_continuous
public import DomainTheory.ContinuousLattice.Constructions.scottExtend
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

section Lemma39

variable {X Y D D' : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  [CompleteLattice D] [CompleteLattice D']

/-- **Scott 1972, Lemma 3.9.** With `e : X → Y` a subspace embedding and `i, j : D ⇄
    D'` a
projection on the range, if continuous `f : X → D` and `g : X → D'` satisfy `f = j ∘ g
    `, then the
maximal extensions (Proposition 3.8) satisfy `f = j ∘ g`.

The proof is Scott's, recast through the faithful `scottExtend`:
* `j ∘ g ⊑ f`: `j ∘ g` is a continuous solution of `(j ∘ g) ∘ e = j ∘ g = f`, so
    maximality of
  `f` (`scottExtend_maximal`) gives the bound;
* `i ∘ f ⊑ g`: `(i ∘ f) ∘ e = i ∘ f = i ∘ j ∘ g ⊑ g` (since `i ∘ j ⊑ id`), so the *sub
    *-solution
  maximality of `g` (`scottExtend_maximal_le`, the remark after 3.8) gives the bound;
* combining: `f = j ∘ i ∘ f ⊑ j ∘ g ⊑ f` (using `j ∘ i = id` and monotonicity of `j`).
    -/
theorem lemma_3_9 (hD : ContinuousLattice D) (hD' : ContinuousLattice D')
    (P : IsContinuousLatticeProjection D D') (e : X → Y) (he : IsEmbedding e)
    {f : X → D} {g : X → D'} (hf : @Continuous X D _ scottTopologicalSpace f)
    (hg : @Continuous X D' _ scottTopologicalSpace g)
    (hfg : ∀ x, f x = (P.retr : D' → D) (g x)) (y : Y) :
    scottExtend e f y = (P.retr : D' → D) (scottExtend e g y) := by
  have hfbar : @Continuous Y D _ scottTopologicalSpace (scottExtend e f) :=
    scottExtend_continuous hD e f
  have hgbar : @Continuous Y D' _ scottTopologicalSpace (scottExtend e g) :=
    scottExtend_continuous hD' e g
  -- `j ∘ g` and `i ∘ f` are continuous (Scott topology is not an instance; register it
    locally so
  -- `Continuous.comp` can fire, scoped to avoid the lattice `≤` clashing with
```

```
    specialization order).
  have hjg : @Continuous Y D _ scottTopologicalSpace (fun z => (P.retr : D' → D) (
    scottExtend e g z)) := by
    let : TopologicalSpace D := scottTopologicalSpace
    let : TopologicalSpace D' := scottTopologicalSpace
    exact P.retr.continuous.comp hgbar
  have hif : @Continuous Y D' _ scottTopologicalSpace (fun z => (P.incl : D → D') (
    scottExtend e f z)) := by
    let : TopologicalSpace D := scottTopologicalSpace
    let : TopologicalSpace D' := scottTopologicalSpace
    exact P.incl.continuous.comp hfbar
  -- Step A: `j ∘ g ⊑ f`.
  have hA : ∀ z, (P.retr : D' → D) (scottExtend e g z) ≤ scottExtend e f z := by
    intro z
    refine scottExtend_maximal hD e hjg ?_ z
    intro x
    show (P.retr : D' → D) (scottExtend e g (e x)) = f x
    rw [scottExtend_eq_of_continuous hD' e he g hg x, ← hfg x]
  -- Step B: `i ∘ f ⊑ g`.
  have hB : ∀ z, (P.incl : D → D') (scottExtend e f z) ≤ scottExtend e g z := by
    intro z
    refine scottExtend_maximal_le hD' e hif ?_ z
    intro x
    show (P.incl : D → D') (scottExtend e f (e x)) ≤ g x
    rw [scottExtend_eq_of_continuous hD e he f hf x, hfg x]
    exact P.incl_retr_le (g x)
  refine le_antisymm ?_ (hA y)
  calc scottExtend e f y
      = (P.retr : D' → D) ((P.incl : D → D') (scottExtend e f y)) :=
        (P.retr_incl (scottExtend e f y)).symm
    _ ≤ (P.retr : D' → D) (scottExtend e g y) := P.retr.monotone (hB y)

end Lemma39
end DomainTheory.ContinuousLattice
```

8

lemma_3_9.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.Basic
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.Constructions.scottExtend_maximal_le
public import DomainTheory.ContinuousLattice.Constructions.scottExtend_maximal
public import DomainTheory.ContinuousLattice.Constructions.scottExtend_continuous
public import DomainTheory.ContinuousLattice.Constructions.
    scottExtend_eq_of_continuous
public import DomainTheory.ContinuousLattice.Constructions.scottExtend
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section
```

```
namespace DomainTheory.ContinuousLattice

open Topology Set

section Lemma39

variable {X Y D D' : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  [CompleteLattice D] [CompleteLattice D']

/-- **Scott 1972, Lemma 3.9.** With `e : X → Y` a subspace embedding and `i, j : D ⇄
    D'` a
projection on the range, if continuous `f : X → D` and `g : X → D'` satisfy `f = j ∘ g
    `, then the
maximal extensions (Proposition 3.8) satisfy `f = j ∘ g`.

The proof is Scott's, recast through the faithful `scottExtend`:
* `j ∘ g ⊑ f`: `j ∘ g` is a continuous solution of `(j ∘ g) ∘ e = j ∘ g = f`, so
    maximality of
  `f` (`scottExtend_maximal`) gives the bound;
* `i ∘ f ⊑ g`: `(i ∘ f) ∘ e = i ∘ f = i ∘ j ∘ g ⊑ g` (since `i ∘ j ⊑ id`), so the ⋆sub
    ⋆-solution
  maximality of `g` (`scottExtend_maximal_le`, the remark after 3.8) gives the bound;
* combining: `f = j ∘ i ∘ f ⊑ j ∘ g ⊑ f` (using `j ∘ i = id` and monotonicity of `j`).
    -/
theorem lemma_3_9 (hD : ContinuousLattice D) (hD' : ContinuousLattice D')
    (P : IsContinuousLatticeProjection D D') (e : X → Y) (he : IsEmbedding e)
    {f : X → D} {g : X → D'} (hf : @Continuous X D _ scottTopologicalSpace f)
    (hg : @Continuous X D' _ scottTopologicalSpace g)
    (hfg : ∀ x, f x = (P.retr : D' → D) (g x)) (y : Y) :
    scottExtend e f y = (P.retr : D' → D) (scottExtend e g y) := by
  have hfbar : @Continuous Y D _ scottTopologicalSpace (scottExtend e f) :=
    scottExtend_continuous hD e f
  have hgbar : @Continuous Y D' _ scottTopologicalSpace (scottExtend e g) :=
    scottExtend_continuous hD' e g
  -- `j ∘ g` and `i ∘ f` are continuous (Scott topology is not an instance; register it
     locally so
  -- `Continuous.comp` can fire, scoped to avoid the lattice `≤` clashing with
    specialization order).
  have hjg : @Continuous Y D _ scottTopologicalSpace (fun z => (P.retr : D' → D) (
    scottExtend e g z)) := by
    let : TopologicalSpace D := scottTopologicalSpace
    let : TopologicalSpace D' := scottTopologicalSpace
    exact P.retr.continuous.comp hgbar
  have hif : @Continuous Y D' _ scottTopologicalSpace (fun z => (P.incl : D → D') (
    scottExtend e f z)) := by
    let : TopologicalSpace D := scottTopologicalSpace
    let : TopologicalSpace D' := scottTopologicalSpace
    exact P.incl.continuous.comp hfbar
  -- Step A: `j ∘ g ⊑ f`.
  have hA : ∀ z, (P.retr : D' → D) (scottExtend e g z) ≤ scottExtend e f z := by
    intro z
    refine scottExtend_maximal hD e hjg ?_ z
    intro x
    show (P.retr : D' → D) (scottExtend e g (e x)) = f x
    rw [scottExtend_eq_of_continuous hD' e he g hg x, ← hfg x]
  -- Step B: `i ∘ f ⊑ g`.
```

```
  have hB : ∀ z, (P.incl : D → D') (scottExtend e f z) ≤ scottExtend e g z := by
    intro z
    refine scottExtend_maximal_le hD' e hif ?_ z
    intro x
    show (P.incl : D → D') (scottExtend e f (e x)) ≤ g x
    rw [scottExtend_eq_of_continuous hD e he f hf x, hfg x]
    exact P.incl_retr_le (g x)
  refine le_antisymm ?_ (hA y)
  calc scottExtend e f y
      = (P.retr : D' → D) ((P.incl : D → D') (scottExtend e f y)) :=
        (P.retr_incl (scottExtend e f y)).symm
    _ ≤ (P.retr : D' → D) (scottExtend e g y) := P.retr.monotone (hB y)

end Lemma39
end DomainTheory.ContinuousLattice
```

9

lemma_3_9_incl_inf.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottSubspaceExtendInf_mono_incl_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottSubspaceExtendInf_mono_incl
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y] in
/-- **Scott 1972, Lemma 3.9 (inclusion at infimum level).** Used in the inverse-limit
argument (Theorem 4.4). -/
theorem lemma_3_9_incl_inf {e : X → Y} (P : IsContinuousLatticeProjection D D')
    {f : X → D} {g : X → D'} (hfg : ∀ x, f x = P.retr (g x)) (U : Set Y) :
    P.incl (scottSubspaceExtendInf e f U) ≤ scottSubspaceExtendInf e g U :=
  le_trans (scottSubspaceExtendInf_mono_incl_apply (e := e) (f := f) U P)
    (scottSubspaceExtendInf_mono_incl (e := e) (f := f) (g := g) P hfg U)
```

```
end SubspaceExtension
end DomainTheory.ContinuousLattice
```

0

lemma_3_9_incl_inf.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottSubspaceExtendInf_mono_incl_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottSubspaceExtendInf_mono_incl
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y] in
/-- **Scott 1972, Lemma 3.9 (inclusion at infimum level).** Used in the inverse-limit
argument (Theorem 4.4). -/
theorem lemma_3_9_incl_inf {e : X → Y} (P : IsContinuousLatticeProjection D D')
    {f : X → D} {g : X → D'} (hfg : ∀ x, f x = P.retr (g x)) (U : Set Y) :
    P.incl (scottSubspaceExtendInf e f U) ≤ scottSubspaceExtendInf e g U :=
  le_trans (scottSubspaceExtendInf_mono_incl_apply (e := e) (f := f) U P)
    (scottSubspaceExtendInf_mono_incl (e := e) (f := f) (g := g) P hfg U)

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

1

lemma_3_9_incl_inf.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottSubspaceExtendInf_mono_incl_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottSubspaceExtendInf_mono_incl
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y] in
/-- **Scott 1972, Lemma 3.9 (inclusion at infimum level).** Used in the inverse-limit
argument (Theorem 4.4). -/
theorem lemma_3_9_incl_inf {e : X → Y} (P : IsContinuousLatticeProjection D D')
    {f : X → D} {g : X → D'} (hfg : ∀ x, f x = P.retr (g x)) (U : Set Y) :
    P.incl (scottSubspaceExtendInf e f U) ≤ scottSubspaceExtendInf e g U :=
  le_trans (scottSubspaceExtendInf_mono_incl_apply (e := e) (f := f) U P)
    (scottSubspaceExtendInf_mono_incl (e := e) (f := f) (g := g) P hfg U)

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

2

lemma_3_9_incl_inf.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottSubspaceExtendInf_mono_incl_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottSubspaceExtendInf_mono_incl
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
```

```
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y] in
/-- **Scott 1972, Lemma 3.9 (inclusion at infimum level).** Used in the inverse-limit
argument (Theorem 4.4). -/
theorem lemma_3_9_incl_inf {e : X → Y} (P : IsContinuousLatticeProjection D D')
    {f : X → D} {g : X → D'} (hfg : ∀ x, f x = P.retr (g x)) (U : Set Y) :
    P.incl (scottSubspaceExtendInf e f U) ≤ scottSubspaceExtendInf e g U :=
  le_trans (scottSubspaceExtendInf_mono_incl_apply (e := e) (f := f) U P)
    (scottSubspaceExtendInf_mono_incl (e := e) (f := f) (g := g) P hfg U)

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

3

lemma_3_9_incl_inf.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottSubspaceExtendInf_mono_incl_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottSubspaceExtendInf_mono_incl
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y] in
/-- **Scott 1972, Lemma 3.9 (inclusion at infimum level).** Used in the inverse-limit
argument (Theorem 4.4). -/
theorem lemma_3_9_incl_inf {e : X → Y} (P : IsContinuousLatticeProjection D D')
    {f : X → D} {g : X → D'} (hfg : ∀ x, f x = P.retr (g x)) (U : Set Y) :
```

```
    P.incl (scottSubspaceExtendInf e f U) ≤ scottSubspaceExtendInf e g U :=
  le_trans (scottSubspaceExtendInf_mono_incl_apply (e := e) (f := f) U P)
    (scottSubspaceExtendInf_mono_incl (e := e) (f := f) (g := g) P hfg U)

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

4

lemma_3_9_incl_inf.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottSubspaceExtendInf_mono_incl_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottSubspaceExtendInf_mono_incl
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y] in
/-- **Scott 1972, Lemma 3.9 (inclusion at infimum level).** Used in the inverse-limit
argument (Theorem 4.4). -/
theorem lemma_3_9_incl_inf {e : X → Y} (P : IsContinuousLatticeProjection D D')
    {f : X → D} {g : X → D'} (hfg : ∀ x, f x = P.retr (g x)) (U : Set Y) :
    P.incl (scottSubspaceExtendInf e f U) ≤ scottSubspaceExtendInf e g U :=
  le_trans (scottSubspaceExtendInf_mono_incl_apply (e := e) (f := f) U P)
    (scottSubspaceExtendInf_mono_incl (e := e) (f := f) (g := g) P hfg U)

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

5

lemma_3_9_incl_inf.lean

```
module
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottSubspaceExtendInf_mono_incl_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottSubspaceExtendInf_mono_incl
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y] in
/-- **Scott 1972, Lemma 3.9 (inclusion at infimum level).** Used in the inverse-limit
argument (Theorem 4.4). -/
theorem lemma_3_9_incl_inf {e : X → Y} (P : IsContinuousLatticeProjection D D')
    {f : X → D} {g : X → D'} (hfg : ∀ x, f x = P.retr (g x)) (U : Set Y) :
    P.incl (scottSubspaceExtendInf e f U) ≤ scottSubspaceExtendInf e g U :=
  le_trans (scottSubspaceExtendInf_mono_incl_apply (e := e) (f := f) U P)
    (scottSubspaceExtendInf_mono_incl (e := e) (f := f) (g := g) P hfg U)

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

6

lemma_3_9_incl_inf.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottSubspaceExtendInf_mono_incl_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottSubspaceExtendInf_mono_incl
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice
```

```
open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y] in
/-- **Scott 1972, Lemma 3.9 (inclusion at infimum level).** Used in the inverse-limit
argument (Theorem 4.4). -/
theorem lemma_3_9_incl_inf {e : X → Y} (P : IsContinuousLatticeProjection D D')
    {f : X → D} {g : X → D'} (hfg : ∀ x, f x = P.retr (g x)) (U : Set Y) :
    P.incl (scottSubspaceExtendInf e f U) ≤ scottSubspaceExtendInf e g U :=
  le_trans (scottSubspaceExtendInf_mono_incl_apply (e := e) (f := f) U P)
    (scottSubspaceExtendInf_mono_incl (e := e) (f := f) (g := g) P hfg U)

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

7

lemma_3_9_incl_inf.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottSubspaceExtendInf_mono_incl_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottSubspaceExtendInf_mono_incl
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]
```

```
omit [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y] in
/-- **Scott 1972, Lemma 3.9 (inclusion at infimum level).** Used in the inverse-limit
argument (Theorem 4.4). -/
theorem lemma_3_9_incl_inf {e : X → Y} (P : IsContinuousLatticeProjection D D')
    {f : X → D} {g : X → D'} (hfg : ∀ x, f x = P.retr (g x)) (U : Set Y) :
    P.incl (scottSubspaceExtendInf e f U) ≤ scottSubspaceExtendInf e g U :=
  le_trans (scottSubspaceExtendInf_mono_incl_apply (e := e) (f := f) U P)
    (scottSubspaceExtendInf_mono_incl (e := e) (f := f) (g := g) P hfg U)

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

8

lemma_3_9_incl_inf.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottSubspaceExtendInf_mono_incl_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottSubspaceExtendInf_mono_incl
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y] in
/-- **Scott 1972, Lemma 3.9 (inclusion at infimum level).** Used in the inverse-limit
argument (Theorem 4.4). -/
theorem lemma_3_9_incl_inf {e : X → Y} (P : IsContinuousLatticeProjection D D')
    {f : X → D} {g : X → D'} (hfg : ∀ x, f x = P.retr (g x)) (U : Set Y) :
    P.incl (scottSubspaceExtendInf e f U) ≤ scottSubspaceExtendInf e g U :=
  le_trans (scottSubspaceExtendInf_mono_incl_apply (e := e) (f := f) U P)
    (scottSubspaceExtendInf_mono_incl (e := e) (f := f) (g := g) P hfg U)

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

9

lemma_3_9_retr_inf.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottSubspaceExtendInf_mono_retr
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y] in
/-- **Scott 1972, Lemma 3.9 (retraction at infimum level).** The global equality
`f = j ∘ g` assembles these infimum bounds via Proposition 3.10(i). -/
theorem lemma_3_9_retr_inf {e : X → Y} (P : IsContinuousLatticeProjection D D') {g : X
     → D'}
    (U : Set Y) :
    P.retr (scottSubspaceExtendInf e g U) ≤ scottSubspaceExtendInf e (fun x => P.retr
    (g x)) U :=
  scottSubspaceExtendInf_mono_retr (e := e) (g := g) P U

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

0

lemma_3_9_retr_inf.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottSubspaceExtendInf_mono_retr
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice
```

```
open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y] in
/-- **Scott 1972, Lemma 3.9 (retraction at infimum level).** The global equality
`f = j ∘ g` assembles these infimum bounds via Proposition 3.10(i). -/
theorem lemma_3_9_retr_inf {e : X → Y} (P : IsContinuousLatticeProjection D D') {g : X
     → D'}
    (U : Set Y) :
    P.retr (scottSubspaceExtendInf e g U) ≤ scottSubspaceExtendInf e (fun x => P.retr
    (g x)) U :=
  scottSubspaceExtendInf_mono_retr (e := e) (g := g) P U

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

1

lemma_3_9_retr_inf.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottSubspaceExtendInf_mono_retr
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y] in
```

```
/-- **Scott 1972, Lemma 3.9 (retraction at infimum level).** The global equality
`f = j ∘ g` assembles these infimum bounds via Proposition 3.10(i). -/
theorem lemma_3_9_retr_inf {e : X → Y} (P : IsContinuousLatticeProjection D D') {g : X
    → D'}
    (U : Set Y) :
    P.retr (scottSubspaceExtendInf e g U) ≤ scottSubspaceExtendInf e (fun x => P.retr
    (g x)) U :=
  scottSubspaceExtendInf_mono_retr (e := e) (g := g) P U

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

2

lemma_3_9_retr_inf.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottSubspaceExtendInf_mono_retr
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y] in
/-- **Scott 1972, Lemma 3.9 (retraction at infimum level).** The global equality
`f = j ∘ g` assembles these infimum bounds via Proposition 3.10(i). -/
theorem lemma_3_9_retr_inf {e : X → Y} (P : IsContinuousLatticeProjection D D') {g : X
    → D'}
    (U : Set Y) :
    P.retr (scottSubspaceExtendInf e g U) ≤ scottSubspaceExtendInf e (fun x => P.retr
    (g x)) U :=
  scottSubspaceExtendInf_mono_retr (e := e) (g := g) P U

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

3

lemma_3_9_retr_inf.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottSubspaceExtendInf_mono_retr
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y] in
/-- **Scott 1972, Lemma 3.9 (retraction at infimum level).** The global equality
`f = j ∘ g` assembles these infimum bounds via Proposition 3.10(i). -/
theorem lemma_3_9_retr_inf {e : X → Y} (P : IsContinuousLatticeProjection D D') {g : X
     → D'}
    (U : Set Y) :
    P.retr (scottSubspaceExtendInf e g U) ≤ scottSubspaceExtendInf e (fun x => P.retr
    (g x)) U :=
  scottSubspaceExtendInf_mono_retr (e := e) (g := g) P U

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

4

lemma_3_9_retr_inf.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottSubspaceExtendInf_mono_retr
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice
```

```
open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y] in
/-- **Scott 1972, Lemma 3.9 (retraction at infimum level).** The global equality
`f = j ∘ g` assembles these infimum bounds via Proposition 3.10(i). -/
theorem lemma_3_9_retr_inf {e : X → Y} (P : IsContinuousLatticeProjection D D') {g : X
     → D'}
    (U : Set Y) :
    P.retr (scottSubspaceExtendInf e g U) ≤ scottSubspaceExtendInf e (fun x => P.retr
    (g x)) U :=
  scottSubspaceExtendInf_mono_retr (e := e) (g := g) P U

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

5

lemma_3_9_retr_inf.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottSubspaceExtendInf_mono_retr
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y] in
```

```
/-- **Scott 1972, Lemma 3.9 (retraction at infimum level).** The global equality
`f = j ∘ g` assembles these infimum bounds via Proposition 3.10(i). -/
theorem lemma_3_9_retr_inf {e : X → Y} (P : IsContinuousLatticeProjection D D') {g : X
    → D'}
    (U : Set Y) :
    P.retr (scottSubspaceExtendInf e g U) ≤ scottSubspaceExtendInf e (fun x => P.retr
    (g x)) U :=
  scottSubspaceExtendInf_mono_retr (e := e) (g := g) P U

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

6

lemma_3_9_retr_inf.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottSubspaceExtendInf_mono_retr
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y] in
/-- **Scott 1972, Lemma 3.9 (retraction at infimum level).** The global equality
`f = j ∘ g` assembles these infimum bounds via Proposition 3.10(i). -/
theorem lemma_3_9_retr_inf {e : X → Y} (P : IsContinuousLatticeProjection D D') {g : X
    → D'}
    (U : Set Y) :
    P.retr (scottSubspaceExtendInf e g U) ≤ scottSubspaceExtendInf e (fun x => P.retr
    (g x)) U :=
  scottSubspaceExtendInf_mono_retr (e := e) (g := g) P U

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

7

lemma_3_9_retr_inf.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottSubspaceExtendInf_mono_retr
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y] in
/-- **Scott 1972, Lemma 3.9 (retraction at infimum level).** The global equality
`f = j ∘ g` assembles these infimum bounds via Proposition 3.10(i). -/
theorem lemma_3_9_retr_inf {e : X → Y} (P : IsContinuousLatticeProjection D D') {g : X
     → D'}
    (U : Set Y) :
    P.retr (scottSubspaceExtendInf e g U) ≤ scottSubspaceExtendInf e (fun x => P.retr
    (g x)) U :=
  scottSubspaceExtendInf_mono_retr (e := e) (g := g) P U

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

8

lemma_3_9_retr_inf.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottSubspaceExtendInf_mono_retr
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice
```

```
open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y] in
/-- **Scott 1972, Lemma 3.9 (retraction at infimum level).** The global equality
`f = j ∘ g` assembles these infimum bounds via Proposition 3.10(i). -/
theorem lemma_3_9_retr_inf {e : X → Y} (P : IsContinuousLatticeProjection D D') {g : X
     → D'}
    (U : Set Y) :
    P.retr (scottSubspaceExtendInf e g U) ≤ scottSubspaceExtendInf e (fun x => P.retr
    (g x)) U :=
  scottSubspaceExtendInf_mono_retr (e := e) (g := g) P U

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

9

proposition_3_10_converse.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.converseProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset
```

```
variable {i : D → D'}

/-- **Scott 1972, Proposition 3.10 (converse).** If `i : D → D'` (between continuous
    lattices)
satisfies (i) preservation of arbitrary suprema, (ii) injectivity, and (iii)
    preservation of `≪`,
then there is a continuous `j` making `D` a projection of `D'` via `i`, with `j` given
    by Scott's
formula (iv) `j(x') = ⊔ { x | i(x) ⊑ x' }`. -/
theorem proposition_3_10_converse
    (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S)) (hinj : Function.Injective i)
    (hwb : ∀ {x y : D}, x ≪ y → i x ≪ i y) (hD : ContinuousLattice D) :
    ∃ P : IsContinuousLatticeProjection D D',
      (∀ d, (P.incl : D → D') d = i d)
        ∧ (∀ x', (P.retr : D' → D) x' = sSup {x | i x ≤ x'}) :=
  ⟨converseProjection hi hinj hwb hD, fun _ => rfl, fun _ => rfl⟩

end Proposition310
end DomainTheory.ContinuousLattice
```

0

proposition_3_10_converse.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.converseProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- **Scott 1972, Proposition 3.10 (converse).** If `i : D → D'` (between continuous
    lattices)
satisfies (i) preservation of arbitrary suprema, (ii) injectivity, and (iii)
    preservation of `≪`,
```

```
then there is a continuous `j` making `D` a projection of `D'` via `i`, with `j` given
    by Scott's
formula (iv) `j(x') = ⊔ { x | i(x) ⊑ x' }`. -/
theorem proposition_3_10_converse
    (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S)) (hinj : Function.Injective i)
    (hwb : ∀ {x y : D}, x ≪ y → i x ≪ i y) (hD : ContinuousLattice D) :
    ∃ P : IsContinuousLatticeProjection D D',
      (∀ d, (P.incl : D → D') d = i d)
        ∧ (∀ x', (P.retr : D' → D) x' = sSup {x | i x ≤ x'}) :=
  ⟨converseProjection hi hinj hwb hD, fun _ => rfl, fun _ => rfl⟩

end Proposition310
end DomainTheory.ContinuousLattice
```

1

proposition_3_10_converse.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.converseProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- **Scott 1972, Proposition 3.10 (converse).** If `i : D → D'` (between continuous
    lattices)
satisfies (i) preservation of arbitrary suprema, (ii) injectivity, and (iii)
    preservation of `≪`,
then there is a continuous `j` making `D` a projection of `D'` via `i`, with `j` given
    by Scott's
formula (iv) `j(x') = ⊔ { x | i(x) ⊑ x' }`. -/
theorem proposition_3_10_converse
    (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S)) (hinj : Function.Injective i)
    (hwb : ∀ {x y : D}, x ≪ y → i x ≪ i y) (hD : ContinuousLattice D) :
```

```
    ∃ P : IsContinuousLatticeProjection D D',
      (∀ d, (P.incl : D → D') d = i d)
        ∧ (∀ x', (P.retr : D' → D) x' = sSup {x | i x ≤ x'}) :=
  ⟨converseProjection hi hinj hwb hD, fun _ => rfl, fun _ => rfl⟩

end Proposition310
end DomainTheory.ContinuousLattice
```

2

proposition_3_10_converse.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.converseProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- **Scott 1972, Proposition 3.10 (converse).** If `i : D → D''` (between continuous
    lattices)
satisfies (i) preservation of arbitrary suprema, (ii) injectivity, and (iii)
    preservation of `≪`,
then there is a continuous `j` making `D` a projection of `D''` via `i`, with `j` given
    by Scott's
formula (iv) `j(x') = ⊔ { x | i(x) ⊑ x' }`. -/
theorem proposition_3_10_converse
    (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S)) (hinj : Function.Injective i)
    (hwb : ∀ {x y : D}, x ≪ y → i x ≪ i y) (hD : ContinuousLattice D) :
    ∃ P : IsContinuousLatticeProjection D D',
      (∀ d, (P.incl : D → D') d = i d)
        ∧ (∀ x', (P.retr : D' → D) x' = sSup {x | i x ≤ x'}) :=
  ⟨converseProjection hi hinj hwb hD, fun _ => rfl, fun _ => rfl⟩

end Proposition310
```

```
end DomainTheory.ContinuousLattice
```

3

proposition_3_10_converse.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.converseProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- **Scott 1972, Proposition 3.10 (converse).** If `i : D → D'` (between continuous
    lattices)
satisfies (i) preservation of arbitrary suprema, (ii) injectivity, and (iii)
    preservation of `≪`,
then there is a continuous `j` making `D` a projection of `D'` via `i`, with `j` given
    by Scott's
formula (iv) `j(x') = ⊔ { x | i(x) ⊑ x' }`. -/
theorem proposition_3_10_converse
    (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S)) (hinj : Function.Injective i)
    (hwb : ∀ {x y : D}, x ≪ y → i x ≪ i y) (hD : ContinuousLattice D) :
    ∃ P : IsContinuousLatticeProjection D D',
      (∀ d, (P.incl : D → D') d = i d)
        ∧ (∀ x', (P.retr : D' → D) x' = sSup {x | i x ≤ x'}) :=
  ⟨converseProjection hi hinj hwb hD, fun _ => rfl, fun _ => rfl⟩

end Proposition310
end DomainTheory.ContinuousLattice
```

4

proposition_3_10_converse.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.converseProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- **Scott 1972, Proposition 3.10 (converse).** If `i : D → D'` (between continuous
    lattices)
satisfies (i) preservation of arbitrary suprema, (ii) injectivity, and (iii)
    preservation of `≪`,
then there is a continuous `j` making `D` a projection of `D'` via `i`, with `j` given
    by Scott's
formula (iv) `j(x') = ⊔ { x | i(x) ⊑ x' }`. -/
theorem proposition_3_10_converse
    (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S)) (hinj : Function.Injective i)
    (hwb : ∀ {x y : D}, x ≪ y → i x ≪ i y) (hD : ContinuousLattice D) :
    ∃ P : IsContinuousLatticeProjection D D',
      (∀ d, (P.incl : D → D') d = i d)
        ∧ (∀ x', (P.retr : D' → D) x' = sSup {x | i x ≤ x'}) :=
  ⟨converseProjection hi hinj hwb hD, fun _ => rfl, fun _ => rfl⟩

end Proposition310
end DomainTheory.ContinuousLattice
```

5

proposition_3_10_converse.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.converseProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.
```

```
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- **Scott 1972, Proposition 3.10 (converse).** If `i : D → D'` (between continuous
    lattices)
satisfies (i) preservation of arbitrary suprema, (ii) injectivity, and (iii)
    preservation of `≪`,
then there is a continuous `j` making `D` a projection of `D'` via `i`, with `j` given
    by Scott's
formula (iv) `j(x') = ⊔ { x | i(x) ⊑ x' }`. -/
theorem proposition_3_10_converse
    (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S)) (hinj : Function.Injective i)
    (hwb : ∀ {x y : D}, x ≪ y → i x ≪ i y) (hD : ContinuousLattice D) :
    ∃ P : IsContinuousLatticeProjection D D',
      (∀ d, (P.incl : D → D') d = i d)
        ∧ (∀ x', (P.retr : D' → D) x' = sSup {x | i x ≤ x'}) :=
  ⟨conserveProjection hi hinj hwb hD, fun _ => rfl, fun _ => rfl⟩

end Proposition310
end DomainTheory.ContinuousLattice
```

6

proposition_3_10_converse.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.conserveProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section
```

```
namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- **Scott 1972, Proposition 3.10 (converse).** If `i : D → D'` (between continuous
    lattices)
satisfies (i) preservation of arbitrary suprema, (ii) injectivity, and (iii)
    preservation of `≪`,
then there is a continuous `j` making `D` a projection of `D'` via `i`, with `j` given
    by Scott's
formula (iv) `j(x') = ⊔ { x | i(x) ⊑ x' }`. -/
theorem proposition_3_10_converse
    (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S)) (hinj : Function.Injective i)
    (hwb : ∀ {x y : D}, x ≪ y → i x ≪ i y) (hD : ContinuousLattice D) :
    ∃ P : IsContinuousLatticeProjection D D',
      (∀ d, (P.incl : D → D') d = i d)
        ∧ (∀ x', (P.retr : D' → D) x' = sSup {x | i x ≤ x'}) :=
  ⟨converseProjection hi hinj hwb hD, fun _ => rfl, fun _ => rfl⟩

end Proposition310
end DomainTheory.ContinuousLattice
```

7

proposition_3_10_converse.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.converseProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
```

```
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

variable {i : D → D'}

/-- **Scott 1972, Proposition 3.10 (converse).** If `i : D → D'` (between continuous
    lattices)
satisfies (i) preservation of arbitrary suprema, (ii) injectivity, and (iii)
    preservation of `≪`,
then there is a continuous `j` making `D` a projection of `D'` via `i`, with `j` given
    by Scott's
formula (iv) `j(x') = ⊔ { x | i(x) ⊑ x' }`. -/
theorem proposition_3_10_converse
    (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S)) (hinj : Function.Injective i)
    (hwb : ∀ {x y : D}, x ≪ y → i x ≪ i y) (hD : ContinuousLattice D) :
    ∃ P : IsContinuousLatticeProjection D D',
      (∀ d, (P.incl : D → D') d = i d)
        ∧ (∀ x', (P.retr : D' → D) x' = sSup {x | i x ≤ x'}) :=
  ⟨conserveProjection hi hinj hwb hD, fun _ => rfl, fun _ => rfl⟩

end Proposition310
end DomainTheory.ContinuousLattice
```

8

proposition_3_10_converse.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.conserveProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310
```

```
open Finset

variable {i : D → D'}

/-- **Scott 1972, Proposition 3.10 (converse).** If `i : D → D'` (between continuous
    lattices)
satisfies (i) preservation of arbitrary suprema, (ii) injectivity, and (iii)
    preservation of `≪`,
then there is a continuous `j` making `D` a projection of `D'` via `i`, with `j` given
    by Scott's
formula (iv) `j(x') = ⊔ { x | i(x) ⊑ x' }`. -/
theorem proposition_3_10_converse
    (hi : ∀ S : Set D, i (sSup S) = sSup (i '' S)) (hinj : Function.Injective i)
    (hwb : ∀ {x y : D}, x ≪ y → i x ≪ i y) (hD : ContinuousLattice D) :
    ∃ P : IsContinuousLatticeProjection D D',
      (∀ d, (P.incl : D → D') d = i d)
        ∧ (∀ x', (P.retr : D' → D) x' = sSup {x | i x ≤ x'}) :=
  ⟨conserveProjection hi hinj hwb hD, fun _ => rfl, fun _ => rfl⟩

end Proposition310
end DomainTheory.ContinuousLattice
```

9

proposition_3_10_forward.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_sSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_injective
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection
```

```
/-- **Scott 1972, Proposition 3.10(i)-(iii), bundled.** The inclusion `i` of a
    projection preserves
arbitrary suprema, is injective, and preserves the way-below relation. -/
theorem proposition_3_10_forward (P : IsContinuousLatticeProjection D D')
    (hD' : ContinuousLattice D') :
    (∀ S : Set D, (P.incl : D → D') (sSup S) = sSup (Set.image (P.incl : D → D') S))
      ∧ Function.Injective (P.incl : D → D')
      ∧ (∀ x y : D, x ≪ y → (P.incl : D → D') x ≪ (P.incl : D → D') y) :=
  ⟨fun S => P.incl_sSup S, fun _ _ h => P.incl_injective h,
    fun _ _ h => P.incl_wayBelow hD' h⟩

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

0

proposition_3_10_forward.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_sSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_injective
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

/-- **Scott 1972, Proposition 3.10(i)-(iii), bundled.** The inclusion `i` of a
    projection preserves
arbitrary suprema, is injective, and preserves the way-below relation. -/
theorem proposition_3_10_forward (P : IsContinuousLatticeProjection D D')
    (hD' : ContinuousLattice D') :
    (∀ S : Set D, (P.incl : D → D') (sSup S) = sSup (Set.image (P.incl : D → D') S))
      ∧ Function.Injective (P.incl : D → D')
```

```
      ∧ (∀ x y : D, x ≪ y → (P.incl : D → D') x ≪ (P.incl : D → D') y) :=
  ⟨fun S => P.incl_sSup S, fun _ _ h => P.incl_injective h,
    fun _ _ h => P.incl_wayBelow hD' h⟩

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

1

proposition_3_10_forward.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_sSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_injective
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

/-- **Scott 1972, Proposition 3.10(i)-(iii), bundled.** The inclusion `i` of a
    projection preserves
arbitrary suprema, is injective, and preserves the way-below relation. -/
theorem proposition_3_10_forward (P : IsContinuousLatticeProjection D D')
    (hD' : ContinuousLattice D') :
    (∀ S : Set D, (P.incl : D → D') (sSup S) = sSup (Set.image (P.incl : D → D') S))
      ∧ Function.Injective (P.incl : D → D')
      ∧ (∀ x y : D, x ≪ y → (P.incl : D → D') x ≪ (P.incl : D → D') y) :=
  ⟨fun S => P.incl_sSup S, fun _ _ h => P.incl_injective h,
    fun _ _ h => P.incl_wayBelow hD' h⟩

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

2

proposition_3_10_forward.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_sSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_injective
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

/-- **Scott 1972, Proposition 3.10(i)-(iii), bundled.** The inclusion `i` of a
    projection preserves
arbitrary suprema, is injective, and preserves the way-below relation. -/
theorem proposition_3_10_forward (P : IsContinuousLatticeProjection D D')
    (hD' : ContinuousLattice D') :
    (∀ S : Set D, (P.incl : D → D') (sSup S) = sSup (Set.image (P.incl : D → D') S))
      ∧ Function.Injective (P.incl : D → D')
      ∧ (∀ x y : D, x ≪ y → (P.incl : D → D') x ≪ (P.incl : D → D') y) :=
  ⟨fun S => P.incl_sSup S, fun _ _ h => P.incl_injective h,
    fun _ _ h => P.incl_wayBelow hD' h⟩

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

3

proposition_3_10_forward.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
```

```
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_sSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_injective
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

/-- **Scott 1972, Proposition 3.10(i)-(iii), bundled.** The inclusion `i` of a
    projection preserves
arbitrary suprema, is injective, and preserves the way-below relation. -/
theorem proposition_3_10_forward (P : IsContinuousLatticeProjection D D')
    (hD' : ContinuousLattice D') :
    (∀ S : Set D, (P.incl : D → D') (sSup S) = sSup (Set.image (P.incl : D → D') S))
      ∧ Function.Injective (P.incl : D → D')
      ∧ (∀ x y : D, x ≪ y → (P.incl : D → D') x ≪ (P.incl : D → D') y) :=
  ⟨fun S => P.incl_sSup S, fun _ _ h => P.incl_injective h,
    fun _ _ h => P.incl_wayBelow hD' h⟩

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

4

proposition_3_10_forward.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_sSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_injective
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
```

```
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

/-- **Scott 1972, Proposition 3.10(i)-(iii), bundled.** The inclusion `i` of a
    projection preserves
arbitrary suprema, is injective, and preserves the way-below relation. -/
theorem proposition_3_10_forward (P : IsContinuousLatticeProjection D D')
    (hD' : ContinuousLattice D') :
    (∀ S : Set D, (P.incl : D → D') (sSup S) = sSup (Set.image (P.incl : D → D') S))
      ∧ Function.Injective (P.incl : D → D')
      ∧ (∀ x y : D, x ≪ y → (P.incl : D → D') x ≪ (P.incl : D → D') y) :=
  ⟨fun S => P.incl_sSup S, fun _ _ h => P.incl_injective h,
    fun _ _ h => P.incl_wayBelow hD' h⟩

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

5

proposition_3_10_forward.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_sSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_injective
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace
```

```
variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

/-- **Scott 1972, Proposition 3.10(i)-(iii), bundled.** The inclusion `i` of a
    projection preserves
arbitrary suprema, is injective, and preserves the way-below relation. -/
theorem proposition_3_10_forward (P : IsContinuousLatticeProjection D D')
    (hD' : ContinuousLattice D') :
    (∀ S : Set D, (P.incl : D → D') (sSup S) = sSup (Set.image (P.incl : D → D') S))
      ∧ Function.Injective (P.incl : D → D')
      ∧ (∀ x y : D, x ≪ y → (P.incl : D → D') x ≪ (P.incl : D → D') y) :=
  ⟨fun S => P.incl_sSup S, fun _ _ h => P.incl_injective h,
    fun _ _ h => P.incl_wayBelow hD' h⟩

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

6

proposition_3_10_forward.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_sSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_injective
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']
```

```
section Proposition310

open Finset

namespace IsContinuousLatticeProjection

/-- **Scott 1972, Proposition 3.10(i)-(iii), bundled.** The inclusion `i` of a
    projection preserves
arbitrary suprema, is injective, and preserves the way-below relation. -/
theorem proposition_3_10_forward (P : IsContinuousLatticeProjection D D')
    (hD' : ContinuousLattice D') :
    (∀ S : Set D, (P.incl : D → D') (sSup S) = sSup (Set.image (P.incl : D → D') S))
      ∧ Function.Injective (P.incl : D → D')
      ∧ (∀ x y : D, x ≪ y → (P.incl : D → D') x ≪ (P.incl : D → D') y) :=
  ⟨fun S => P.incl_sSup S, fun _ _ h => P.incl_injective h,
    fun _ _ h => P.incl_wayBelow hD' h⟩

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

7

proposition_3_10_forward.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_sSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_injective
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

/-- **Scott 1972, Proposition 3.10(i)-(iii), bundled.** The inclusion `i` of a
```

```
    projection preserves
arbitrary suprema, is injective, and preserves the way-below relation. -/
theorem proposition_3_10_forward (P : IsContinuousLatticeProjection D D')
    (hD' : ContinuousLattice D') :
    (∀ S : Set D, (P.incl : D → D') (sSup S) = sSup (Set.image (P.incl : D → D') S))
      ∧ Function.Injective (P.incl : D → D')
      ∧ (∀ x y : D, x ≪ y → (P.incl : D → D') x ≪ (P.incl : D → D') y) :=
  ⟨fun S => P.incl_sSup S, fun _ _ h => P.incl_injective h,
    fun _ _ h => P.incl_wayBelow hD' h⟩

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

8

proposition_3_10_forward.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_sSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_injective
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

/-- **Scott 1972, Proposition 3.10(i)-(iii), bundled.** The inclusion `i` of a
    projection preserves
arbitrary suprema, is injective, and preserves the way-below relation. -/
theorem proposition_3_10_forward (P : IsContinuousLatticeProjection D D')
    (hD' : ContinuousLattice D') :
    (∀ S : Set D, (P.incl : D → D') (sSup S) = sSup (Set.image (P.incl : D → D') S))
      ∧ Function.Injective (P.incl : D → D')
      ∧ (∀ x y : D, x ≪ y → (P.incl : D → D') x ≪ (P.incl : D → D') y) :=
```

```
  ⟨fun S => P.incl_sSup S, fun _ _ h => P.incl_injective h,
    fun _ _ h => P.incl_wayBelow hD' h⟩

end IsContinuousLatticeProjection
end Proposition310
end DomainTheory.ContinuousLattice
```

9

proposition_3_12.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.instCompleteLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.Projections
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_sSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst30
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

/-- **Scott 1972, Proposition 3.12.** For a (complete, in particular continuous)
    lattice `D`, the
projections `J_D` form a complete lattice as a `⊔`-closed subspace of `[D → D]`: the `
    ⊔`-closure is
`isProjection_sSup`, and the complete-lattice structure is `Projections.
    instCompleteLattice`. -/
theorem proposition_3_12 :
    (∀ T : Set (ScottMap D D), (∀ j ∈ T, IsProjection j) → IsProjection (sSup T))
      ∧ Nonempty (CompleteLattice (Projections D)) :=
  ⟨fun _ h => isProjection_sSup h, ⟨Projections.instCompleteLattice⟩⟩

end Projections
end DomainTheory.ContinuousLattice
```

0

proposition_3_12.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.instCompleteLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.Projections
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_sSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst30
```

```
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

/-- **Scott 1972, Proposition 3.12.** For a (complete, in particular continuous)
    lattice `D`, the
projections `J_D` form a complete lattice as a `⊔`-closed subspace of `[D → D]`: the `
    ⊔`-closure is
`isProjection_sSup`, and the complete-lattice structure is `Projections.
    instCompleteLattice`. -/
theorem proposition_3_12 :
    (∀ T : Set (ScottMap D D), (∀ j ∈ T, IsProjection j) → IsProjection (sSup T))
      ∧ Nonempty (CompleteLattice (Projections D)) :=
  ⟨fun _ h => isProjection_sSup h, ⟨Projections.instCompleteLattice⟩⟩

end Projections
end DomainTheory.ContinuousLattice
```

1

proposition_3_12.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.instCompleteLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.Projections
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_sSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst30
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections
```

```
/-- **Scott 1972, Proposition 3.12.** For a (complete, in particular continuous)
    lattice `D`, the
projections `J_D` form a complete lattice as a `⊔`-closed subspace of `[D → D]`: the `
    ⊔`-closure is
`isProjection_sSup`, and the complete-lattice structure is `Projections.
    instCompleteLattice`. -/
theorem proposition_3_12 :
    (∀ T : Set (ScottMap D D), (∀ j ∈ T, IsProjection j) → IsProjection (sSup T))
      ∧ Nonempty (CompleteLattice (Projections D)) :=
  ⟨fun _ h => isProjection_sSup h, ⟨Projections.instCompleteLattice⟩⟩

end Projections
end DomainTheory.ContinuousLattice
```

2

proposition_3_12.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.instCompleteLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.Projections
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_sSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst30
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

/-- **Scott 1972, Proposition 3.12.** For a (complete, in particular continuous)
    lattice `D`, the
projections `J_D` form a complete lattice as a `⊔`-closed subspace of `[D → D]`: the `
    ⊔`-closure is
`isProjection_sSup`, and the complete-lattice structure is `Projections.
    instCompleteLattice`. -/
theorem proposition_3_12 :
    (∀ T : Set (ScottMap D D), (∀ j ∈ T, IsProjection j) → IsProjection (sSup T))
      ∧ Nonempty (CompleteLattice (Projections D)) :=
  ⟨fun _ h => isProjection_sSup h, ⟨Projections.instCompleteLattice⟩⟩

end Projections
end DomainTheory.ContinuousLattice
```

3

proposition_3_12.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.instCompleteLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.Projections
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_sSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst30
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

/-- **Scott 1972, Proposition 3.12.** For a (complete, in particular continuous)
    lattice `D`, the
projections `J_D` form a complete lattice as a `⊔`-closed subspace of `[D → D]`: the `
    ⊔`-closure is
`isProjection_sSup`, and the complete-lattice structure is `Projections.
    instCompleteLattice`. -/
theorem proposition_3_12 :
    (∀ T : Set (ScottMap D D), (∀ j ∈ T, IsProjection j) → IsProjection (sSup T))
      ∧ Nonempty (CompleteLattice (Projections D)) :=
  ⟨fun _ h => isProjection_sSup h, ⟨Projections.instCompleteLattice⟩⟩

end Projections
end DomainTheory.ContinuousLattice
```

4

proposition_3_12.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.instCompleteLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.Projections
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_sSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst30
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace
```

```
variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

/-- **Scott 1972, Proposition 3.12.** For a (complete, in particular continuous)
    lattice `D`, the
projections `J_D` form a complete lattice as a `⊔`-closed subspace of `[D → D]`: the `
    ⊔`-closure is
`isProjection_sSup`, and the complete-lattice structure is `Projections.
    instCompleteLattice`. -/
theorem proposition_3_12 :
    (∀ T : Set (ScottMap D D), (∀ j ∈ T, IsProjection j) → IsProjection (sSup T))
      ∧ Nonempty (CompleteLattice (Projections D)) :=
  ⟨fun _ h => isProjection_sSup h, ⟨Projections.instCompleteLattice⟩⟩

end Projections
end DomainTheory.ContinuousLattice
```

5

proposition_3_12.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.instCompleteLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.Projections
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_sSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst30
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

/-- **Scott 1972, Proposition 3.12.** For a (complete, in particular continuous)
    lattice `D`, the
projections `J_D` form a complete lattice as a `⊔`-closed subspace of `[D → D]`: the `
    ⊔`-closure is
`isProjection_sSup`, and the complete-lattice structure is `Projections.
    instCompleteLattice`. -/
theorem proposition_3_12 :
    (∀ T : Set (ScottMap D D), (∀ j ∈ T, IsProjection j) → IsProjection (sSup T))
```

```
      ∧ Nonempty (CompleteLattice (Projections D)) :=
  ⟨fun _ h => isProjection_sSup h, ⟨Projections.instCompleteLattice⟩⟩

end Projections
end DomainTheory.ContinuousLattice
```

6

proposition_3_12.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.instCompleteLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.Projections
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_sSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst30
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

/-- **Scott 1972, Proposition 3.12.** For a (complete, in particular continuous)
    lattice `D`, the
projections `J_D` form a complete lattice as a `⊔`-closed subspace of `[D → D]`: the `
    ⊔`-closure is
`isProjection_sSup`, and the complete-lattice structure is `Projections.
    instCompleteLattice`. -/
theorem proposition_3_12 :
    (∀ T : Set (ScottMap D D), (∀ j ∈ T, IsProjection j) → IsProjection (sSup T))
      ∧ Nonempty (CompleteLattice (Projections D)) :=
  ⟨fun _ h => isProjection_sSup h, ⟨Projections.instCompleteLattice⟩⟩

end Projections
end DomainTheory.ContinuousLattice
```

7

proposition_3_12.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.instCompleteLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.Projections
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_sSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst30
public import Mathlib.Topology.Defs.Basic
```

```
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections

/-- **Scott 1972, Proposition 3.12.** For a (complete, in particular continuous)
    lattice `D`, the
projections `J_D` form a complete lattice as a `⊔`-closed subspace of `[D → D]`: the `
    ⊔`-closure is
`isProjection_sSup`, and the complete-lattice structure is `Projections.
    instCompleteLattice`. -/
theorem proposition_3_12 :
    (∀ T : Set (ScottMap D D), (∀ j ∈ T, IsProjection j) → IsProjection (sSup T))
      ∧ Nonempty (CompleteLattice (Projections D)) :=
  ⟨fun _ h => isProjection_sSup h, ⟨Projections.instCompleteLattice⟩⟩

end Projections
end DomainTheory.ContinuousLattice
```

8

proposition_3_12.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.instCompleteLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.Projections
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_sSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst30
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

section Projections
```

```
/-- **Scott 1972, Proposition 3.12.** For a (complete, in particular continuous)
    lattice `D`, the
projections `J_D` form a complete lattice as a `⊔`-closed subspace of `[D → D]`: the `
    ⊔`-closure is
`isProjection_sSup`, and the complete-lattice structure is `Projections.
    instCompleteLattice`. -/
theorem proposition_3_12 :
    (∀ T : Set (ScottMap D D), (∀ j ∈ T, IsProjection j) → IsProjection (sSup T))
      ∧ Nonempty (CompleteLattice (Projections D)) :=
  ⟨fun _ h => isProjection_sSup h, ⟨Projections.instCompleteLattice⟩⟩

end Projections
end DomainTheory.ContinuousLattice
```

9

con_.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.const
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.const_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition313

/-- **Scott 1972, Proposition 3.13.** `con : D → [D → D]` sends `x` to the constant
    function `x`. -/
noncomputable def con : ScottMap D (ScottMap D D) :=
  ⟨fun x => ScottMap.const x, continuous_of_preservesDirectedSup (by
    intro S _ _
    apply ScottMap.ext
    intro y
    rw [ScottMap.sSup_apply, Set.image_image]
    simp only [ScottMap.const_apply, Set.image_id'])⟩
```

```
end Proposition313

end DomainTheory.ContinuousLattice
```

0

con_.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.const
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.const_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition313

/-- **Scott 1972, Proposition 3.13.** `con : D → [D → D]` sends `x` to the constant
    function `x`. -/
noncomputable def con : ScottMap D (ScottMap D D) :=
  ⟨fun x => ScottMap.const x, continuous_of_preservesDirectedSup (by
    intro S _ _
    apply ScottMap.ext
    intro y
    rw [ScottMap.sSup_apply, Set.image_image]
    simp only [ScottMap.const_apply, Set.image_id'])⟩

end Proposition313

end DomainTheory.ContinuousLattice
```

1

con_.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.const
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.const_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition313

/-- **Scott 1972, Proposition 3.13.** `con : D → [D → D]` sends `x` to the constant
    function `x`. -/
noncomputable def con : ScottMap D (ScottMap D D) :=
  ⟨fun x => ScottMap.const x, continuous_of_preservesDirectedSup (by
    intro S _ _
    apply ScottMap.ext
    intro y
    rw [ScottMap.sSup_apply, Set.image_image]
    simp only [ScottMap.const_apply, Set.image_id'])⟩

end Proposition313

end DomainTheory.ContinuousLattice
```

2

con_.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.const
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.const_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
```

```
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition313

/-- **Scott 1972, Proposition 3.13.** `con : D → [D → D]` sends `x` to the constant
    function `x`. -/
noncomputable def con : ScottMap D (ScottMap D D) :=
  ⟨fun x => ScottMap.const x, continuous_of_preservesDirectedSup (by
    intro S _ _
    apply ScottMap.ext
    intro y
    rw [ScottMap.sSup_apply, Set.image_image]
    simp only [ScottMap.const_apply, Set.image_id'])⟩

end Proposition313

end DomainTheory.ContinuousLattice
```

3

con_.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.const
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.const_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace
```

```
variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition313

/-- **Scott 1972, Proposition 3.13.** `con : D → [D → D]` sends `x` to the constant
    function `x`. -/
noncomputable def con : ScottMap D (ScottMap D D) :=
  ⟨fun x => ScottMap.const x, continuous_of_preservesDirectedSup (by
    intro S _ _
    apply ScottMap.ext
    intro y
    rw [ScottMap.sSup_apply, Set.image_image]
    simp only [ScottMap.const_apply, Set.image_id'])⟩

end Proposition313

end DomainTheory.ContinuousLattice
```

4

con_.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.const
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.const_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']
```

```
namespace Proposition313

/-- **Scott 1972, Proposition 3.13.** `con : D → [D → D]` sends `x` to the constant
    function `x`. -/
noncomputable def con : ScottMap D (ScottMap D D) :=
  ⟨fun x => ScottMap.const x, continuous_of_preservesDirectedSup (by
    intro S _ _
    apply ScottMap.ext
    intro y
    rw [ScottMap.sSup_apply, Set.image_image]
    simp only [ScottMap.const_apply, Set.image_id'])⟩

end Proposition313

end DomainTheory.ContinuousLattice
```

5

con_.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.const
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.const_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition313

/-- **Scott 1972, Proposition 3.13.** `con : D → [D → D]` sends `x` to the constant
    function `x`. -/
noncomputable def con : ScottMap D (ScottMap D D) :=
  ⟨fun x => ScottMap.const x, continuous_of_preservesDirectedSup (by
    intro S _ _
    apply ScottMap.ext
    intro y
```

```
    rw [ScottMap.sSup_apply, Set.image_image]
    simp only [ScottMap.const_apply, Set.image_id'])⟩

end Proposition313

end DomainTheory.ContinuousLattice
```

6

con_.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.const
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.const_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition313

/-- **Scott 1972, Proposition 3.13.** `con : D → [D → D]` sends `x` to the constant
    function `x`. -/
noncomputable def con : ScottMap D (ScottMap D D) :=
  ⟨fun x => ScottMap.const x, continuous_of_preservesDirectedSup (by
    intro S _ _
    apply ScottMap.ext
    intro y
    rw [ScottMap.sSup_apply, Set.image_image]
    simp only [ScottMap.const_apply, Set.image_id'])⟩

end Proposition313

end DomainTheory.ContinuousLattice
```

7

con_.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.const
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.const_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition313

/-- **Scott 1972, Proposition 3.13.** `con : D → [D → D]` sends `x` to the constant
    function `x`. -/
noncomputable def con : ScottMap D (ScottMap D D) :=
  ⟨fun x => ScottMap.const x, continuous_of_preservesDirectedSup (by
    intro S _ _
    apply ScottMap.ext
    intro y
    rw [ScottMap.sSup_apply, Set.image_image]
    simp only [ScottMap.const_apply, Set.image_id'])⟩

end Proposition313

end DomainTheory.ContinuousLattice
```

8

con_.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.const
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.const_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
```

```
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition313

/-- **Scott 1972, Proposition 3.13.** `con : D → [D → D]` sends `x` to the constant
    function `x`. -/
noncomputable def con : ScottMap D (ScottMap D D) :=
  ⟨fun x => ScottMap.const x, continuous_of_preservesDirectedSup (by
    intro S _ _
    apply ScottMap.ext
    intro y
    rw [ScottMap.sSup_apply, Set.image_image]
    simp only [ScottMap.const_apply, Set.image_id'])⟩

end Proposition313

end DomainTheory.ContinuousLattice
```

9

min.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace
```

```
variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition313

/-- **Scott 1972, Proposition 3.13.** `min : [D → D] → D` sends `f` to its least value
    `f(⊥)`. -/
noncomputable def min : ScottMap (ScottMap D D) D :=
  ⟨fun f => (f : D → D) ⊥, continuous_of_preservesDirectedSup (by
    intro F _ _
    show ((sSup F : ScottMap D D) : D → D) ⊥
      = sSup (Set.image (fun f : ScottMap D D => (f : D → D) ⊥) F)
    rw [ScottMap.sSup_apply])⟩

end Proposition313

end DomainTheory.ContinuousLattice
```

0

min.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition313

/-- **Scott 1972, Proposition 3.13.** `min : [D → D] → D` sends `f` to its least value
```

```
    `f(⊥)`. -/
noncomputable def min : ScottMap (ScottMap D D) D :=
  ⟨fun f => (f : D → D) ⊥, continuous_of_preservesDirectedSup (by
    intro F _ _
    show ((sSup F : ScottMap D D) : D → D) ⊥
      = sSup (Set.image (fun f : ScottMap D D => (f : D → D) ⊥) F)
    rw [ScottMap.sSup_apply])⟩

end Proposition313

end DomainTheory.ContinuousLattice
```

1

min.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition313

/-- **Scott 1972, Proposition 3.13.** `min : [D → D] → D` sends `f` to its least value
    `f(⊥)`. -/
noncomputable def min : ScottMap (ScottMap D D) D :=
  ⟨fun f => (f : D → D) ⊥, continuous_of_preservesDirectedSup (by
    intro F _ _
    show ((sSup F : ScottMap D D) : D → D) ⊥
      = sSup (Set.image (fun f : ScottMap D D => (f : D → D) ⊥) F)
    rw [ScottMap.sSup_apply])⟩

end Proposition313

end DomainTheory.ContinuousLattice
```

2

min.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition313

/-- **Scott 1972, Proposition 3.13.** `min : [D → D] → D` sends `f` to its least value
     `f(⊥)`. -/
noncomputable def min : ScottMap (ScottMap D D) D :=
  ⟨fun f => (f : D → D) ⊥, continuous_of_preservesDirectedSup (by
    intro F _ _
    show ((sSup F : ScottMap D D) : D → D) ⊥
      = sSup (Set.image (fun f : ScottMap D D => (f : D → D) ⊥) F)
    rw [ScottMap.sSup_apply])⟩

end Proposition313

end DomainTheory.ContinuousLattice
```

3

min.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
```

```
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition313

/-- **Scott 1972, Proposition 3.13.** `min : [D → D] → D` sends `f` to its least value
    `f(⊥)`. -/
noncomputable def min : ScottMap (ScottMap D D) D :=
  ⟨fun f => (f : D → D) ⊥, continuous_of_preservesDirectedSup (by
    intro F _ _
    show ((sSup F : ScottMap D D) : D → D) ⊥
      = sSup (Set.image (fun f : ScottMap D D => (f : D → D) ⊥) F)
    rw [ScottMap.sSup_apply])⟩

end Proposition313

end DomainTheory.ContinuousLattice
```

4

min.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]
```

```
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition313

/-- **Scott 1972, Proposition 3.13.** `min : [D → D] → D` sends `f` to its least value
    `f(⊥)`. -/
noncomputable def min : ScottMap (ScottMap D D) D :=
  ⟨fun f => (f : D → D) ⊥, continuous_of_preservesDirectedSup (by
    intro F _ _
    show ((sSup F : ScottMap D D) : D → D) ⊥
      = sSup (Set.image (fun f : ScottMap D D => (f : D → D) ⊥) F)
    rw [ScottMap.sSup_apply])⟩

end Proposition313

end DomainTheory.ContinuousLattice
```

5

min.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition313

/-- **Scott 1972, Proposition 3.13.** `min : [D → D] → D` sends `f` to its least value
    `f(⊥)`. -/
noncomputable def min : ScottMap (ScottMap D D) D :=
  ⟨fun f => (f : D → D) ⊥, continuous_of_preservesDirectedSup (by
    intro F _ _
```

```
    show ((sSup F : ScottMap D D) : D → D) ⊥
      = sSup (Set.image (fun f : ScottMap D D => (f : D → D) ⊥) F)
    rw [ScottMap.sSup_apply])⟩

end Proposition313

end DomainTheory.ContinuousLattice
```

6

min.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition313

/-- **Scott 1972, Proposition 3.13.** `min : [D → D] → D` sends `f` to its least value
    `f(⊥)`. -/
noncomputable def min : ScottMap (ScottMap D D) D :=
  ⟨fun f => (f : D → D) ⊥, continuous_of_preservesDirectedSup (by
    intro F _ _
    show ((sSup F : ScottMap D D) : D → D) ⊥
      = sSup (Set.image (fun f : ScottMap D D => (f : D → D) ⊥) F)
    rw [ScottMap.sSup_apply])⟩

end Proposition313

end DomainTheory.ContinuousLattice
```

7

min.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition313

/-- **Scott 1972, Proposition 3.13.** `min : [D → D] → D` sends `f` to its least value
     `f(⊥)`. -/
noncomputable def min : ScottMap (ScottMap D D) D :=
  ⟨fun f => (f : D → D) ⊥, continuous_of_preservesDirectedSup (by
    intro F _ _
    show ((sSup F : ScottMap D D) : D → D) ⊥
      = sSup (Set.image (fun f : ScottMap D D => (f : D → D) ⊥) F)
    rw [ScottMap.sSup_apply])⟩

end Proposition313

end DomainTheory.ContinuousLattice
```

8

min.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section
```

```
namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition313

/-- **Scott 1972, Proposition 3.13.** `min : [D → D] → D` sends `f` to its least value
     `f(⊥)`. -/
noncomputable def min : ScottMap (ScottMap D D) D :=
  ⟨fun f => (f : D → D) ⊥, continuous_of_preservesDirectedSup (by
    intro F _ _
    show ((sSup F : ScottMap D D) : D → D) ⊥
      = sSup (Set.image (fun f : ScottMap D D => (f : D → D) ⊥) F)
    rw [ScottMap.sSup_apply])⟩

end Proposition313

end DomainTheory.ContinuousLattice
```

9

proposition_3_13.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.projection
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Proposition 3.13.** Every continuous lattice `D` is a projection of
    its function
```

```
space `[D → D]`, via `con`/`min` (`Proposition313.projection`). -/
theorem proposition_3_13 (_hD : ContinuousLattice D) :
    Nonempty (IsContinuousLatticeProjection D (ScottMap D D)) :=
  ⟨Proposition313.projection⟩

end DomainTheory.ContinuousLattice
```

0

proposition_3_13.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.projection
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Proposition 3.13.** Every continuous lattice `D` is a projection of
    its function
space `[D → D]`, via `con`/`min` (`Proposition313.projection`). -/
theorem proposition_3_13 (_hD : ContinuousLattice D) :
    Nonempty (IsContinuousLatticeProjection D (ScottMap D D)) :=
  ⟨Proposition313.projection⟩

end DomainTheory.ContinuousLattice
```

1

proposition_3_13.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.projection
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic
```

```
@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Proposition 3.13.** Every continuous lattice `D` is a projection of
    its function
space `[D → D]`, via `con`/`min` (`Proposition313.projection`). -/
theorem proposition_3_13 (_hD : ContinuousLattice D) :
    Nonempty (IsContinuousLatticeProjection D (ScottMap D D)) :=
  ⟨Proposition313.projection⟩

end DomainTheory.ContinuousLattice
```

2

proposition_3_13.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.projection
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Proposition 3.13.** Every continuous lattice `D` is a projection of
    its function
space `[D → D]`, via `con`/`min` (`Proposition313.projection`). -/
theorem proposition_3_13 (_hD : ContinuousLattice D) :
    Nonempty (IsContinuousLatticeProjection D (ScottMap D D)) :=
  ⟨Proposition313.projection⟩

end DomainTheory.ContinuousLattice
```

3

proposition_3_13.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.projection
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Proposition 3.13.** Every continuous lattice `D` is a projection of
    its function
space `[D → D]`, via `con`/`min` (`Proposition313.projection`). -/
theorem proposition_3_13 (_hD : ContinuousLattice D) :
    Nonempty (IsContinuousLatticeProjection D (ScottMap D D)) :=
  ⟨Proposition313.projection⟩

end DomainTheory.ContinuousLattice
```

4

proposition_3_13.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.projection
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
```

```
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Proposition 3.13.** Every continuous lattice `D` is a projection of
    its function
space `[D → D]`, via `con`/`min` (`Proposition313.projection`). -/
theorem proposition_3_13 (_hD : ContinuousLattice D) :
    Nonempty (IsContinuousLatticeProjection D (ScottMap D D)) :=
  ⟨Proposition313.projection⟩

end DomainTheory.ContinuousLattice
```

5

proposition_3_13.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.projection
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Proposition 3.13.** Every continuous lattice `D` is a projection of
    its function
space `[D → D]`, via `con`/`min` (`Proposition313.projection`). -/
theorem proposition_3_13 (_hD : ContinuousLattice D) :
    Nonempty (IsContinuousLatticeProjection D (ScottMap D D)) :=
  ⟨Proposition313.projection⟩

end DomainTheory.ContinuousLattice
```

6

proposition_3_13.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaces.projection
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Proposition 3.13.** Every continuous lattice `D` is a projection of
    its function
space `[D → D]`, via `con`/`min` (`Proposition313.projection`). -/
theorem proposition_3_13 (_hD : ContinuousLattice D) :
    Nonempty (IsContinuousLatticeProjection D (ScottMap D D)) :=
  ⟨Proposition313.projection⟩

end DomainTheory.ContinuousLattice
```

7

proposition_3_13.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.projection
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Proposition 3.13.** Every continuous lattice `D` is a projection of
    its function
```

```
space `[D → D]`, via `con`/`min` (`Proposition313.projection`). -/
theorem proposition_3_13 (_hD : ContinuousLattice D) :
    Nonempty (IsContinuousLatticeProjection D (ScottMap D D)) :=
  ⟨Proposition313.projection⟩

end DomainTheory.ContinuousLattice
```

8

proposition_3_13.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.projection
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Proposition 3.13.** Every continuous lattice `D` is a projection of
    its function
space `[D → D]`, via `con`/`min` (`Proposition313.projection`). -/
theorem proposition_3_13 (_hD : ContinuousLattice D) :
    Nonempty (IsContinuousLatticeProjection D (ScottMap D D)) :=
  ⟨Proposition313.projection⟩

end DomainTheory.ContinuousLattice
```

9

fix.lean

```
module

public import Mathlib.Order.FixedPoints
public import DomainTheory.ContinuousLattice.FunctionSpaces.toOrderHom
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice
```

```
open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- **Scott 1972, Proposition 3.14.** `fix f` is the least (pre-)fixed point of `f`,
    supplied by
mathlib's `OrderHom.lfp`. -/
noncomputable def fix (f : ScottMap D D) : D := (toOrderHom f).lfp

end Proposition314

end DomainTheory.ContinuousLattice
```

0

fix.lean

```
module

public import Mathlib.Order.FixedPoints
public import DomainTheory.ContinuousLattice.FunctionSpaces.toOrderHom
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- **Scott 1972, Proposition 3.14.** `fix f` is the least (pre-)fixed point of `f`,
    supplied by
mathlib's `OrderHom.lfp`. -/
noncomputable def fix (f : ScottMap D D) : D := (toOrderHom f).lfp
```

```
end Proposition314

end DomainTheory.ContinuousLattice
```

1

fix.lean

```
module

public import Mathlib.Order.FixedPoints
public import DomainTheory.ContinuousLattice.FunctionSpaces.toOrderHom
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- **Scott 1972, Proposition 3.14.** `fix f` is the least (pre-)fixed point of `f`,
    supplied by
mathlib's `OrderHom.lfp`. -/
noncomputable def fix (f : ScottMap D D) : D := (toOrderHom f).lfp

end Proposition314

end DomainTheory.ContinuousLattice
```

2

fix.lean

```
module

public import Mathlib.Order.FixedPoints
public import DomainTheory.ContinuousLattice.FunctionSpaces.toOrderHom
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice
```

```
open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- **Scott 1972, Proposition 3.14.** `fix f` is the least (pre-)fixed point of `f`,
    supplied by
mathlib's `OrderHom.lfp`. -/
noncomputable def fix (f : ScottMap D D) : D := (toOrderHom f).lfp

end Proposition314

end DomainTheory.ContinuousLattice
```

3

fix.lean

```
module

public import Mathlib.Order.FixedPoints
public import DomainTheory.ContinuousLattice.FunctionSpaces.toOrderHom
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- **Scott 1972, Proposition 3.14.** `fix f` is the least (pre-)fixed point of `f`,
    supplied by
mathlib's `OrderHom.lfp`. -/
noncomputable def fix (f : ScottMap D D) : D := (toOrderHom f).lfp

end Proposition314
```

```
end DomainTheory.ContinuousLattice
```

4

fix.lean

```
module

public import Mathlib.Order.FixedPoints
public import DomainTheory.ContinuousLattice.FunctionSpaces.toOrderHom
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- **Scott 1972, Proposition 3.14.** `fix f` is the least (pre-)fixed point of `f`,
    supplied by
mathlib's `OrderHom.lfp`. -/
noncomputable def fix (f : ScottMap D D) : D := (toOrderHom f).lfp

end Proposition314

end DomainTheory.ContinuousLattice
```

5

fix.lean

```
module

public import Mathlib.Order.FixedPoints
public import DomainTheory.ContinuousLattice.FunctionSpaces.toOrderHom
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace
```

```
variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- **Scott 1972, Proposition 3.14.** `fix f` is the least (pre-)fixed point of `f`,
    supplied by
mathlib's `OrderHom.lfp`. -/
noncomputable def fix (f : ScottMap D D) : D := (toOrderHom f).lfp

end Proposition314

end DomainTheory.ContinuousLattice
```

6

fix.lean

```
module

public import Mathlib.Order.FixedPoints
public import DomainTheory.ContinuousLattice.FunctionSpaces.toOrderHom
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- **Scott 1972, Proposition 3.14.** `fix f` is the least (pre-)fixed point of `f`,
    supplied by
mathlib's `OrderHom.lfp`. -/
noncomputable def fix (f : ScottMap D D) : D := (toOrderHom f).lfp

end Proposition314
```

```
end DomainTheory.ContinuousLattice
```

7

fix.lean

```
module

public import Mathlib.Order.FixedPoints
public import DomainTheory.ContinuousLattice.FunctionSpaces.toOrderHom
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- **Scott 1972, Proposition 3.14.** `fix f` is the least (pre-)fixed point of `f`,
    supplied by
mathlib's `OrderHom.lfp`. -/
noncomputable def fix (f : ScottMap D D) : D := (toOrderHom f).lfp

end Proposition314

end DomainTheory.ContinuousLattice
```

8

fix.lean

```
module

public import Mathlib.Order.FixedPoints
public import DomainTheory.ContinuousLattice.FunctionSpaces.toOrderHom
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace
```

```
variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- **Scott 1972, Proposition 3.14.** `fix f` is the least (pre-)fixed point of `f`,
    supplied by
mathlib's `OrderHom.lfp`. -/
noncomputable def fix (f : ScottMap D D) : D := (toOrderHom f).lfp

end Proposition314

end DomainTheory.ContinuousLattice
```

9

fixMap.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_preservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- **Scott 1972, Proposition 3.14.** The fixed-point operator as a Scott-continuous
    map. -/
noncomputable def fixMap : ScottMap (ScottMap D D) D :=
  ⟨fix, continuous_of_preservesDirectedSup fix_preservesDirectedSup⟩
```

```
end Proposition314

end DomainTheory.ContinuousLattice
```

0

fixMap.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_preservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- **Scott 1972, Proposition 3.14.** The fixed-point operator as a Scott-continuous
    map. -/
noncomputable def fixMap : ScottMap (ScottMap D D) D :=
  ⟨fix, continuous_of_preservesDirectedSup fix_preservesDirectedSup⟩

end Proposition314

end DomainTheory.ContinuousLattice
```

1

fixMap.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_preservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
```

```
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- **Scott 1972, Proposition 3.14.** The fixed-point operator as a Scott-continuous
    map. -/
noncomputable def fixMap : ScottMap (ScottMap D D) D :=
  ⟨fix, continuous_of_preservesDirectedSup fix_preservesDirectedSup⟩

end Proposition314

end DomainTheory.ContinuousLattice
```

2

fixMap.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_preservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']
```

```
namespace Proposition314

/-- **Scott 1972, Proposition 3.14.** The fixed-point operator as a Scott-continuous
    map. -/
noncomputable def fixMap : ScottMap (ScottMap D D) D :=
  ⟨fix, continuous_of_preservesDirectedSup fix_preservesDirectedSup⟩

end Proposition314

end DomainTheory.ContinuousLattice
```

3

fixMap.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_preservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- **Scott 1972, Proposition 3.14.** The fixed-point operator as a Scott-continuous
    map. -/
noncomputable def fixMap : ScottMap (ScottMap D D) D :=
  ⟨fix, continuous_of_preservesDirectedSup fix_preservesDirectedSup⟩

end Proposition314

end DomainTheory.ContinuousLattice
```

4

fixMap.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_preservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- **Scott 1972, Proposition 3.14.** The fixed-point operator as a Scott-continuous
    map. -/
noncomputable def fixMap : ScottMap (ScottMap D D) D :=
  ⟨fix, continuous_of_preservesDirectedSup fix_preservesDirectedSup⟩

end Proposition314

end DomainTheory.ContinuousLattice
```

5

fixMap.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_preservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace
```

```
variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- **Scott 1972, Proposition 3.14.** The fixed-point operator as a Scott-continuous
    map. -/
noncomputable def fixMap : ScottMap (ScottMap D D) D :=
  ⟨fix, continuous_of_preservesDirectedSup fix_preservesDirectedSup⟩

end Proposition314

end DomainTheory.ContinuousLattice
```

6

fixMap.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_preservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- **Scott 1972, Proposition 3.14.** The fixed-point operator as a Scott-continuous
    map. -/
noncomputable def fixMap : ScottMap (ScottMap D D) D :=
  ⟨fix, continuous_of_preservesDirectedSup fix_preservesDirectedSup⟩
```

```
end Proposition314

end DomainTheory.ContinuousLattice
```

7

fixMap.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_preservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- **Scott 1972, Proposition 3.14.** The fixed-point operator as a Scott-continuous
    map. -/
noncomputable def fixMap : ScottMap (ScottMap D D) D :=
  ⟨fix, continuous_of_preservesDirectedSup fix_preservesDirectedSup⟩

end Proposition314

end DomainTheory.ContinuousLattice
```

8

fixMap.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_preservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Topology.Defs.Basic
```

```
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- **Scott 1972, Proposition 3.14.** The fixed-point operator as a Scott-continuous
    map. -/
noncomputable def fixMap : ScottMap (ScottMap D D) D :=
  ⟨fix, continuous_of_preservesDirectedSup fix_preservesDirectedSup⟩

end Proposition314

end DomainTheory.ContinuousLattice
```

9

fix_eq.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.fix
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- `fix f` is a fixed point of `f`. -/
```

```
theorem fix_eq (f : ScottMap D D) : (f : D → D) (fix f) = fix f :=
  (toOrderHom f).map_lfp

end Proposition314

end DomainTheory.ContinuousLattice
```

0

fix_eq.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.fix
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- `fix f` is a fixed point of `f`. -/
theorem fix_eq (f : ScottMap D D) : (f : D → D) (fix f) = fix f :=
  (toOrderHom f).map_lfp

end Proposition314

end DomainTheory.ContinuousLattice
```

1

fix_eq.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.fix
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice
```

```
open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- `fix f` is a fixed point of `f`. -/
theorem fix_eq (f : ScottMap D D) : (f : D → D) (fix f) = fix f :=
  (toOrderHom f).map_lfp

end Proposition314

end DomainTheory.ContinuousLattice
```

2

fix_eq.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.fix
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- `fix f` is a fixed point of `f`. -/
theorem fix_eq (f : ScottMap D D) : (f : D → D) (fix f) = fix f :=
  (toOrderHom f).map_lfp

end Proposition314

end DomainTheory.ContinuousLattice
```

3

fix_eq.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.fix
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- `fix f` is a fixed point of `f`. -/
theorem fix_eq (f : ScottMap D D) : (f : D → D) (fix f) = fix f :=
  (toOrderHom f).map_lfp

end Proposition314

end DomainTheory.ContinuousLattice
```

4

fix_eq.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.fix
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
```

```
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- `fix f` is a fixed point of `f`. -/
theorem fix_eq (f : ScottMap D D) : (f : D → D) (fix f) = fix f :=
  (toOrderHom f).map_lfp

end Proposition314

end DomainTheory.ContinuousLattice
```

5

fix_eq.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.fix
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- `fix f` is a fixed point of `f`. -/
theorem fix_eq (f : ScottMap D D) : (f : D → D) (fix f) = fix f :=
  (toOrderHom f).map_lfp

end Proposition314

end DomainTheory.ContinuousLattice
```

6

fix_eq.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.fix
```

```
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- `fix f` is a fixed point of `f`. -/
theorem fix_eq (f : ScottMap D D) : (f : D → D) (fix f) = fix f :=
  (toOrderHom f).map_lfp

end Proposition314

end DomainTheory.ContinuousLattice
```

7

fix_eq.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.fix
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- `fix f` is a fixed point of `f`. -/
```

```
theorem fix_eq (f : ScottMap D D) : (f : D → D) (fix f) = fix f :=
  (toOrderHom f).map_lfp

end Proposition314

end DomainTheory.ContinuousLattice
```

8

fix_eq.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.fix
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- `fix f` is a fixed point of `f`. -/
theorem fix_eq (f : ScottMap D D) : (f : D → D) (fix f) = fix f :=
  (toOrderHom f).map_lfp

end Proposition314

end DomainTheory.ContinuousLattice
```

9

fix_le.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.fix
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice
```

```
open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- `fix f` is below every pre-fixed point: `f x ⊑ x ⟹ fix f ⊑ x`. -/
theorem fix_le {f : ScottMap D D} {x : D} (h : (f : D → D) x ≤ x) : fix f ≤ x :=
  (toOrderHom f).lfp_le h

end Proposition314

end DomainTheory.ContinuousLattice
```

0

fix_le.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.fix
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- `fix f` is below every pre-fixed point: `f x ⊑ x ⟹ fix f ⊑ x`. -/
theorem fix_le {f : ScottMap D D} {x : D} (h : (f : D → D) x ≤ x) : fix f ≤ x :=
  (toOrderHom f).lfp_le h

end Proposition314

end DomainTheory.ContinuousLattice
```

1

fix_le.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.fix
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- `fix f` is below every pre-fixed point: `f x ⊑ x ⟹ fix f ⊑ x`. -/
theorem fix_le {f : ScottMap D D} {x : D} (h : (f : D → D) x ≤ x) : fix f ≤ x :=
  (toOrderHom f).lfp_le h

end Proposition314

end DomainTheory.ContinuousLattice
```

2

fix_le.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.fix
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
```

```
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- `fix f` is below every pre-fixed point: `f x ⊑ x ⟹ fix f ⊑ x`. -/
theorem fix_le {f : ScottMap D D} {x : D} (h : (f : D → D) x ≤ x) : fix f ≤ x :=
  (toOrderHom f).lfp_le h

end Proposition314

end DomainTheory.ContinuousLattice
```

3

fix_le.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.fix
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- `fix f` is below every pre-fixed point: `f x ⊑ x ⟹ fix f ⊑ x`. -/
theorem fix_le {f : ScottMap D D} {x : D} (h : (f : D → D) x ≤ x) : fix f ≤ x :=
  (toOrderHom f).lfp_le h

end Proposition314

end DomainTheory.ContinuousLattice
```

4

fix_le.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.fix
```

```
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- `fix f` is below every pre-fixed point: `f x ⊑ x ⟹ fix f ⊑ x`. -/
theorem fix_le {f : ScottMap D D} {x : D} (h : (f : D → D) x ≤ x) : fix f ≤ x :=
  (toOrderHom f).lfp_le h

end Proposition314

end DomainTheory.ContinuousLattice
```

5

fix_le.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.fix
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- `fix f` is below every pre-fixed point: `f x ⊑ x ⟹ fix f ⊑ x`. -/
```

```
theorem fix_le {f : ScottMap D D} {x : D} (h : (f : D → D) x ≤ x) : fix f ≤ x :=
  (toOrderHom f).lfp_le h

end Proposition314

end DomainTheory.ContinuousLattice
```

6

fix_le.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.fix
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- `fix f` is below every pre-fixed point: `f x ⊑ x ⟹ fix f ⊑ x`. -/
theorem fix_le {f : ScottMap D D} {x : D} (h : (f : D → D) x ≤ x) : fix f ≤ x :=
  (toOrderHom f).lfp_le h

end Proposition314

end DomainTheory.ContinuousLattice
```

7

fix_le.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.fix
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice
```

```
open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- `fix f` is below every pre-fixed point: `f x ⊑ x ⟹ fix f ⊑ x`. -/
theorem fix_le {f : ScottMap D D} {x : D} (h : (f : D → D) x ≤ x) : fix f ≤ x :=
  (toOrderHom f).lfp_le h

end Proposition314

end DomainTheory.ContinuousLattice
```

8

fix_le.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.fix
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- `fix f` is below every pre-fixed point: `f x ⊑ x ⟹ fix f ⊑ x`. -/
theorem fix_le {f : ScottMap D D} {x : D} (h : (f : D → D) x ≤ x) : fix f ≤ x :=
  (toOrderHom f).lfp_le h

end Proposition314

end DomainTheory.ContinuousLattice
```

9

fix_mono.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst28
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_le
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_eq
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- `fix` is monotone in `f`: if `f ⊑ f'` then `fix f ⊑ fix f'`, since `f (fix f') ⊑ f'
     (fix f') =
fix f'` makes `fix f'` a pre-fixed point of `f`. -/
theorem fix_mono {f f' : ScottMap D D} (hff' : f ≤ f') : fix f ≤ fix f' :=
  fix_le (le_trans (ScottMap.le_def.mp hff' (fix f')) (le_of_eq (fix_eq f')))

end Proposition314

end DomainTheory.ContinuousLattice
```

0

fix_mono.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst28
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_le
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_eq
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section
```

```
namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- `fix` is monotone in `f`: if `f ⊑ f'` then `fix f ⊑ fix f'`, since `f (fix f') ⊑ f'
     (fix f') =
fix f'` makes `fix f'` a pre-fixed point of `f`. -/
theorem fix_mono {f f' : ScottMap D D} (hff' : f ≤ f') : fix f ≤ fix f' :=
  fix_le (le_trans (ScottMap.le_def.mp hff' (fix f')) (le_of_eq (fix_eq f')))

end Proposition314

end DomainTheory.ContinuousLattice
```

1

fix_mono.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst28
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_le
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_eq
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314
```

```
/-- `fix` is monotone in `f`: if `f ⊑ f'` then `fix f ⊑ fix f'`, since `f (fix f') ⊑ f'
     (fix f') =
fix f'` makes `fix f'` a pre-fixed point of `f`. -/
theorem fix_mono {f f' : ScottMap D D} (hff' : f ≤ f') : fix f ≤ fix f' :=
  fix_le (le_trans (ScottMap.le_def.mp hff' (fix f')) (le_of_eq (fix_eq f')))

end Proposition314

end DomainTheory.ContinuousLattice
```

2

fix_mono.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst28
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_le
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_eq
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- `fix` is monotone in `f`: if `f ⊑ f'` then `fix f ⊑ fix f'`, since `f (fix f') ⊑ f'
     (fix f') =
fix f'` makes `fix f'` a pre-fixed point of `f`. -/
theorem fix_mono {f f' : ScottMap D D} (hff' : f ≤ f') : fix f ≤ fix f' :=
  fix_le (le_trans (ScottMap.le_def.mp hff' (fix f')) (le_of_eq (fix_eq f')))

end Proposition314

end DomainTheory.ContinuousLattice
```

3

fix_mono.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst28
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_le
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_eq
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- `fix` is monotone in `f`: if `f ⊑ f'` then `fix f ⊑ fix f'`, since `f (fix f') ⊑ f'
     (fix f') =
fix f'` makes `fix f'` a pre-fixed point of `f`. -/
theorem fix_mono {f f' : ScottMap D D} (hff' : f ≤ f') : fix f ≤ fix f' :=
  fix_le (le_trans (ScottMap.le_def.mp hff' (fix f')) (le_of_eq (fix_eq f')))

end Proposition314

end DomainTheory.ContinuousLattice
```

4

fix_mono.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst28
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_le
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_eq
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace
```

```
variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- `fix` is monotone in `f`: if `f ⊑ f'` then `fix f ⊑ fix f'`, since `f (fix f') ⊑ f'
     (fix f') =
fix f'` makes `fix f'` a pre-fixed point of `f`. -/
theorem fix_mono {f f' : ScottMap D D} (hff' : f ≤ f') : fix f ≤ fix f' :=
  fix_le (le_trans (ScottMap.le_def.mp hff' (fix f')) (le_of_eq (fix_eq f')))

end Proposition314

end DomainTheory.ContinuousLattice
```

5

fix_mono.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst28
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_le
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_eq
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- `fix` is monotone in `f`: if `f ⊑ f'` then `fix f ⊑ fix f'`, since `f (fix f') ⊑ f'
     (fix f') =
fix f'` makes `fix f'` a pre-fixed point of `f`. -/
```

```
theorem fix_mono {f f' : ScottMap D D} (hff' : f ≤ f') : fix f ≤ fix f' :=
  fix_le (le_trans (ScottMap.le_def.mp hff' (fix f')) (le_of_eq (fix_eq f')))

end Proposition314

end DomainTheory.ContinuousLattice
```

6

fix_mono.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst28
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_le
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_eq
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- `fix` is monotone in `f`: if `f ⊑ f'` then `fix f ⊑ fix f'`, since `f (fix f') ⊑ f'
     (fix f') =
fix f'` makes `fix f'` a pre-fixed point of `f`. -/
theorem fix_mono {f f' : ScottMap D D} (hff' : f ≤ f') : fix f ≤ fix f' :=
  fix_le (le_trans (ScottMap.le_def.mp hff' (fix f')) (le_of_eq (fix_eq f')))

end Proposition314

end DomainTheory.ContinuousLattice
```

7

fix_mono.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst28
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_le
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_eq
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- `fix` is monotone in `f`: if `f ⊑ f'` then `fix f ⊑ fix f'`, since `f (fix f') ⊑ f'
     (fix f') =
fix f'` makes `fix f'` a pre-fixed point of `f`. -/
theorem fix_mono {f f' : ScottMap D D} (hff' : f ≤ f') : fix f ≤ fix f' :=
  fix_le (le_trans (ScottMap.le_def.mp hff' (fix f')) (le_of_eq (fix_eq f')))

end Proposition314

end DomainTheory.ContinuousLattice
```

8

fix_mono.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst28
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_le
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_eq
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]
```

```
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- `fix` is monotone in `f`: if `f ⊑ f'` then `fix f ⊑ fix f'`, since `f (fix f') ⊑ f'
     (fix f') =
fix f'` makes `fix f'` a pre-fixed point of `f`. -/
theorem fix_mono {f f' : ScottMap D D} (hff' : f ≤ f') : fix f ≤ fix f' :=
  fix_le (le_trans (ScottMap.le_def.mp hff' (fix f')) (le_of_eq (fix_eq f')))

end Proposition314

end DomainTheory.ContinuousLattice
```

9

fix_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_mono
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_le
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_eq
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- **Scott 1972, Proposition 3.14 (continuity).** `fix` preserves directed suprema.
    Direct
lattice argument (no Kleene iteration): write `g = ⊔S` and `a = ⊔{fix f : f ∈ S}`. The
```

```
    reverse
bound `a ⊑ fix g` is `fix`-monotonicity. For `fix g ⊑ a` it suffices (by `fix_le`) that
     `a` is a
pre-fixed point of `g`. Now `g a = ⊔_{f∈S} f a` (pointwise sup), and each `f a = ⊔_{f'∈
    S} f (fix f')`
(continuity of `f` on the directed family `{fix f'}`); for any `f, f' ∈ S` pick `h ∈ S`
     above both,
so `f (fix f') ⊑ h (fix f') ⊑ h (fix h) = fix h ⊑ a`. Hence `g a ⊑ a`. -/
theorem fix_preservesDirectedSup : PreservesDirectedSup (fix : ScottMap D D → D) := by
  intro S hSne hSdir
  set T : Set D := Set.image fix S with hTdef
  have hTne : T.Nonempty := hSne.image fix
  have hTdir : DirectedOn (· ≤ ·) T := by
    rintro _ ⟨f, hf, rfl⟩ _ ⟨f', hf', rfl⟩
    obtain ⟨h, hh, hfh, hf'h⟩ := hSdir f hf f' hf'
    exact ⟨fix h, Set.mem_image_of_mem fix hh, fix_mono hfh, fix_mono hf'h⟩
  show fix (sSup S) = sSup T
  apply le_antisymm
  · apply fix_le
    show ((sSup S : ScottMap D D) : D → D) (sSup T) ≤ sSup T
    rw [ScottMap.sSup_apply]
    apply sSup_le
    rintro _ ⟨f, hf, rfl⟩
    show (f : D → D) (sSup T) ≤ sSup T
    rw [f.preservesDirectedSup_coe T hTne hTdir]
    apply sSup_le
    rintro _ ⟨_, ⟨f', hf', rfl⟩, rfl⟩
    obtain ⟨h, hh, hfh, hf'h⟩ := hSdir f hf f' hf'
    calc (f : D → D) (fix f')
        ≤ (h : D → D) (fix f') := ScottMap.le_def.mp hfh (fix f')
      _ ≤ (h : D → D) (fix h) := h.monotone (fix_mono hf'h)
      _ = fix h := fix_eq h
      _ ≤ sSup T := le_sSup (Set.mem_image_of_mem fix hh)
  · apply sSup_le
    rintro _ ⟨f, hf, rfl⟩
    exact fix_mono (le_sSup hf)

end Proposition314

end DomainTheory.ContinuousLattice
```

0

fix_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_mono
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_le
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_eq
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
```

```
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- **Scott 1972, Proposition 3.14 (continuity).** `fix` preserves directed suprema.
    Direct
lattice argument (no Kleene iteration): write `g = ⊔S` and `a = ⊔{fix f : f ∈ S}`. The
    reverse
bound `a ⊑ fix g` is `fix`-monotonicity. For `fix g ⊑ a` it suffices (by `fix_le`) that
     `a` is a
pre-fixed point of `g`. Now `g a = ⊔_{f∈S} f a` (pointwise sup), and each `f a = ⊔_{f'∈
    S} f (fix f')`
(continuity of `f` on the directed family `{fix f'}`); for any `f, f' ∈ S` pick `h ∈ S`
     above both,
so `f (fix f') ⊑ h (fix f') ⊑ h (fix h) = fix h ⊑ a`. Hence `g a ⊑ a`. -/
theorem fix_preservesDirectedSup : PreservesDirectedSup (fix : ScottMap D D → D) := by
  intro S hSne hSdir
  set T : Set D := Set.image fix S with hTdef
  have hTne : T.Nonempty := hSne.image fix
  have hTdir : DirectedOn (· ≤ ·) T := by
    rintro _ ⟨f, hf, rfl⟩ _ ⟨f', hf', rfl⟩
    obtain ⟨h, hh, hfh, hf'h⟩ := hSdir f hf f' hf'
    exact ⟨fix h, Set.mem_image_of_mem fix hh, fix_mono hfh, fix_mono hf'h⟩
  show fix (sSup S) = sSup T
  apply le_antisymm
  · apply fix_le
    show ((sSup S : ScottMap D D) : D → D) (sSup T) ≤ sSup T
    rw [ScottMap.sSup_apply]
    apply sSup_le
    rintro _ ⟨f, hf, rfl⟩
    show (f : D → D) (sSup T) ≤ sSup T
    rw [f.preservesDirectedSup_coe T hTne hTdir]
    apply sSup_le
    rintro _ ⟨_, ⟨f', hf', rfl⟩, rfl⟩
    obtain ⟨h, hh, hfh, hf'h⟩ := hSdir f hf f' hf'
    calc (f : D → D) (fix f')
        ≤ (h : D → D) (fix f') := ScottMap.le_def.mp hfh (fix f')
      _ ≤ (h : D → D) (fix h) := h.monotone (fix_mono hf'h)
      _ = fix h := fix_eq h
      _ ≤ sSup T := le_sSup (Set.mem_image_of_mem fix hh)
```

```
  · apply sSup_le
    rintro _ ⟨f, hf, rfl⟩
    exact fix_mono (le_sSup hf)

end Proposition314

end DomainTheory.ContinuousLattice
```

1

fix_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_mono
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_le
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_eq
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- **Scott 1972, Proposition 3.14 (continuity).** `fix` preserves directed suprema.
    Direct
lattice argument (no Kleene iteration): write `g = ⊔S` and `a = ⊔{fix f : f ∈ S}`. The
    reverse
bound `a ⊑ fix g` is `fix`-monotonicity. For `fix g ⊑ a` it suffices (by `fix_le`) that
     `a` is a
pre-fixed point of `g`. Now `g a = ⊔_{f∈S} f a` (pointwise sup), and each `f a = ⊔_{f'∈
    S} f (fix f')`
(continuity of `f` on the directed family `{fix f'}`); for any `f, f' ∈ S` pick `h ∈ S`
     above both,
so `f (fix f') ⊑ h (fix f') ⊑ h (fix h) = fix h ⊑ a`. Hence `g a ⊑ a`. -/
theorem fix_preservesDirectedSup : PreservesDirectedSup (fix : ScottMap D D → D) := by
  intro S hSne hSdir
```

```
  set T : Set D := Set.image fix S with hTdef
  have hTne : T.Nonempty := hSne.image fix
  have hTdir : DirectedOn (· ≤ ·) T := by
    rintro _ ⟨f, hf, rfl⟩ _ ⟨f', hf', rfl⟩
    obtain ⟨h, hh, hfh, hf'h⟩ := hSdir f hf f' hf'
    exact ⟨fix h, Set.mem_image_of_mem fix hh, fix_mono hfh, fix_mono hf'h⟩
  show fix (sSup S) = sSup T
  apply le_antisymm
  · apply fix_le
    show ((sSup S : ScottMap D D) : D → D) (sSup T) ≤ sSup T
    rw [ScottMap.sSup_apply]
    apply sSup_le
    rintro _ ⟨f, hf, rfl⟩
    show (f : D → D) (sSup T) ≤ sSup T
    rw [f.preservesDirectedSup_coe T hTne hTdir]
    apply sSup_le
    rintro _ ⟨_, ⟨f', hf', rfl⟩, rfl⟩
    obtain ⟨h, hh, hfh, hf'h⟩ := hSdir f hf f' hf'
    calc (f : D → D) (fix f')
        ≤ (h : D → D) (fix f') := ScottMap.le_def.mp hfh (fix f')
      _ ≤ (h : D → D) (fix h) := h.monotone (fix_mono hf'h)
      _ = fix h := fix_eq h
      _ ≤ sSup T := le_sSup (Set.mem_image_of_mem fix hh)
  · apply sSup_le
    rintro _ ⟨f, hf, rfl⟩
    exact fix_mono (le_sSup hf)

end Proposition314

end DomainTheory.ContinuousLattice
```

2

fix_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_mono
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_le
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_eq
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
```

```
variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- **Scott 1972, Proposition 3.14 (continuity).** `fix` preserves directed suprema.
    Direct
lattice argument (no Kleene iteration): write `g = ⊔S` and `a = ⊔{fix f : f ∈ S}`. The
    reverse
bound `a ⊑ fix g` is `fix`-monotonicity. For `fix g ⊑ a` it suffices (by `fix_le`) that
     `a` is a
pre-fixed point of `g`. Now `g a = ⊔_{f∈S} f a` (pointwise sup), and each `f a = ⊔_{f'∈
    S} f (fix f')`
(continuity of `f` on the directed family `{fix f'}`); for any `f, f' ∈ S` pick `h ∈ S`
     above both,
so `f (fix f') ⊑ h (fix f') ⊑ h (fix h) = fix h ⊑ a`. Hence `g a ⊑ a`. -/
theorem fix_preservesDirectedSup : PreservesDirectedSup (fix : ScottMap D D → D) := by
  intro S hSne hSdir
  set T : Set D := Set.image fix S with hTdef
  have hTne : T.Nonempty := hSne.image fix
  have hTdir : DirectedOn (· ≤ ·) T := by
    rintro _ ⟨f, hf, rfl⟩ _ ⟨f', hf', rfl⟩
    obtain ⟨h, hh, hfh, hf'h⟩ := hSdir f hf f' hf'
    exact ⟨fix h, Set.mem_image_of_mem fix hh, fix_mono hfh, fix_mono hf'h⟩
  show fix (sSup S) = sSup T
  apply le_antisymm
  · apply fix_le
    show ((sSup S : ScottMap D D) : D → D) (sSup T) ≤ sSup T
    rw [ScottMap.sSup_apply]
    apply sSup_le
    rintro _ ⟨f, hf, rfl⟩
    show (f : D → D) (sSup T) ≤ sSup T
    rw [f.preservesDirectedSup_coe T hTne hTdir]
    apply sSup_le
    rintro _ ⟨_, ⟨f', hf', rfl⟩, rfl⟩
    obtain ⟨h, hh, hfh, hf'h⟩ := hSdir f hf f' hf'
    calc (f : D → D) (fix f')
        ≤ (h : D → D) (fix f') := ScottMap.le_def.mp hfh (fix f')
      _ ≤ (h : D → D) (fix h) := h.monotone (fix_mono hf'h)
      _ = fix h := fix_eq h
      _ ≤ sSup T := le_sSup (Set.mem_image_of_mem fix hh)
  · apply sSup_le
    rintro _ ⟨f, hf, rfl⟩
    exact fix_mono (le_sSup hf)

end Proposition314

end DomainTheory.ContinuousLattice
```

fix_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_mono
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_le
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_eq
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- **Scott 1972, Proposition 3.14 (continuity).** `fix` preserves directed suprema.
    Direct
lattice argument (no Kleene iteration): write `g = ⊔S` and `a = ⊔{fix f : f ∈ S}`. The
    reverse
bound `a ⊑ fix g` is `fix`-monotonicity. For `fix g ⊑ a` it suffices (by `fix_le`) that
     `a` is a
pre-fixed point of `g`. Now `g a = ⊔_{f∈S} f a` (pointwise sup), and each `f a = ⊔_{f'∈
    S} f (fix f')`
(continuity of `f` on the directed family `{fix f'}`); for any `f, f' ∈ S` pick `h ∈ S`
     above both,
so `f (fix f') ⊑ h (fix f') ⊑ h (fix h) = fix h ⊑ a`. Hence `g a ⊑ a`. -/
theorem fix_preservesDirectedSup : PreservesDirectedSup (fix : ScottMap D D → D) := by
  intro S hSne hSdir
  set T : Set D := Set.image fix S with hTdef
  have hTne : T.Nonempty := hSne.image fix
  have hTdir : DirectedOn (· ≤ ·) T := by
    rintro _ ⟨f, hf, rfl⟩ _ ⟨f', hf', rfl⟩
    obtain ⟨h, hh, hfh, hf'h⟩ := hSdir f hf f' hf'
    exact ⟨fix h, Set.mem_image_of_mem fix hh, fix_mono hfh, fix_mono hf'h⟩
  show fix (sSup S) = sSup T
  apply le_antisymm
  · apply fix_le
    show ((sSup S : ScottMap D D) : D → D) (sSup T) ≤ sSup T
```

```
    rw [ScottMap.sSup_apply]
    apply sSup_le
    rintro _ ⟨f, hf, rfl⟩
    show (f : D → D) (sSup T) ≤ sSup T
    rw [f.preservesDirectedSup_coe T hTne hTdir]
    apply sSup_le
    rintro _ ⟨_, ⟨f', hf', rfl⟩, rfl⟩
    obtain ⟨h, hh, hfh, hf'h⟩ := hSdir f hf f' hf'
    calc (f : D → D) (fix f')
        ≤ (h : D → D) (fix f') := ScottMap.le_def.mp hfh (fix f')
      _ ≤ (h : D → D) (fix h) := h.monotone (fix_mono hf'h)
      _ = fix h := fix_eq h
      _ ≤ sSup T := le_sSup (Set.mem_image_of_mem fix hh)
  · apply sSup_le
    rintro _ ⟨f, hf, rfl⟩
    exact fix_mono (le_sSup hf)

end Proposition314

end DomainTheory.ContinuousLattice
```

4

fix_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_mono
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_le
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_eq
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314
```

```
/-- **Scott 1972, Proposition 3.14 (continuity).** `fix` preserves directed suprema.
    Direct
lattice argument (no Kleene iteration): write `g = ⊔S` and `a = ⊔{fix f : f ∈ S}`. The
    reverse
bound `a ⊑ fix g` is `fix`-monotonicity. For `fix g ⊑ a` it suffices (by `fix_le`) that
     `a` is a
pre-fixed point of `g`. Now `g a = ⊔_{f∈S} f a` (pointwise sup), and each `f a = ⊔_{f'∈
    S} f (fix f')`
(continuity of `f` on the directed family `{fix f'}`); for any `f, f' ∈ S` pick `h ∈ S`
     above both,
so `f (fix f') ⊑ h (fix f') ⊑ h (fix h) = fix h ⊑ a`. Hence `g a ⊑ a`. -/
theorem fix_preservesDirectedSup : PreservesDirectedSup (fix : ScottMap D D → D) := by
  intro S hSne hSdir
  set T : Set D := Set.image fix S with hTdef
  have hTne : T.Nonempty := hSne.image fix
  have hTdir : DirectedOn (· ≤ ·) T := by
    rintro _ ⟨f, hf, rfl⟩ _ ⟨f', hf', rfl⟩
    obtain ⟨h, hh, hfh, hf'h⟩ := hSdir f hf f' hf'
    exact ⟨fix h, Set.mem_image_of_mem fix hh, fix_mono hfh, fix_mono hf'h⟩
  show fix (sSup S) = sSup T
  apply le_antisymm
  · apply fix_le
    show ((sSup S : ScottMap D D) : D → D) (sSup T) ≤ sSup T
    rw [ScottMap.sSup_apply]
    apply sSup_le
    rintro _ ⟨f, hf, rfl⟩
    show (f : D → D) (sSup T) ≤ sSup T
    rw [f.preservesDirectedSup_coe T hTne hTdir]
    apply sSup_le
    rintro _ ⟨_, ⟨f', hf', rfl⟩, rfl⟩
    obtain ⟨h, hh, hfh, hf'h⟩ := hSdir f hf f' hf'
    calc (f : D → D) (fix f')
        ≤ (h : D → D) (fix f') := ScottMap.le_def.mp hfh (fix f')
      _ ≤ (h : D → D) (fix h) := h.monotone (fix_mono hf'h)
      _ = fix h := fix_eq h
      _ ≤ sSup T := le_sSup (Set.mem_image_of_mem fix hh)
  · apply sSup_le
    rintro _ ⟨f, hf, rfl⟩
    exact fix_mono (le_sSup hf)

end Proposition314

end DomainTheory.ContinuousLattice
```

5

fix_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_mono
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_le
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_eq
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- **Scott 1972, Proposition 3.14 (continuity).** `fix` preserves directed suprema.
    Direct
lattice argument (no Kleene iteration): write `g = ⊔S` and `a = ⊔{fix f : f ∈ S}`. The
    reverse
bound `a ⊑ fix g` is `fix`-monotonicity. For `fix g ⊑ a` it suffices (by `fix_le`) that
     `a` is a
pre-fixed point of `g`. Now `g a = ⊔_{f∈S} f a` (pointwise sup), and each `f a = ⊔_{f'∈
    S} f (fix f')`
(continuity of `f` on the directed family `{fix f'}`); for any `f, f' ∈ S` pick `h ∈ S`
     above both,
so `f (fix f') ⊑ h (fix f') ⊑ h (fix h) = fix h ⊑ a`. Hence `g a ⊑ a`. -/
theorem fix_preservesDirectedSup : PreservesDirectedSup (fix : ScottMap D D → D) := by
  intro S hSne hSdir
  set T : Set D := Set.image fix S with hTdef
  have hTne : T.Nonempty := hSne.image fix
  have hTdir : DirectedOn (· ≤ ·) T := by
    rintro _ ⟨f, hf, rfl⟩ _ ⟨f', hf', rfl⟩
    obtain ⟨h, hh, hfh, hf'h⟩ := hSdir f hf f' hf'
    exact ⟨fix h, Set.mem_image_of_mem fix hh, fix_mono hfh, fix_mono hf'h⟩
  show fix (sSup S) = sSup T
  apply le_antisymm
  · apply fix_le
    show ((sSup S : ScottMap D D) : D → D) (sSup T) ≤ sSup T
    rw [ScottMap.sSup_apply]
    apply sSup_le
    rintro _ ⟨f, hf, rfl⟩
    show (f : D → D) (sSup T) ≤ sSup T
    rw [f.preservesDirectedSup_coe T hTne hTdir]
    apply sSup_le
    rintro _ ⟨_, ⟨f', hf', rfl⟩, rfl⟩
    obtain ⟨h, hh, hfh, hf'h⟩ := hSdir f hf f' hf'
    calc (f : D → D) (fix f')
        ≤ (h : D → D) (fix f') := ScottMap.le_def.mp hfh (fix f')
```

```
      _ ≤ (h : D → D) (fix h) := h.monotone (fix_mono hf'h)
      _ = fix h := fix_eq h
      _ ≤ sSup T := le_sSup (Set.mem_image_of_mem fix hh)
  · apply sSup_le
    rintro _ ⟨f, hf, rfl⟩
    exact fix_mono (le_sSup hf)

end Proposition314

end DomainTheory.ContinuousLattice
```

6

fix_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_mono
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_le
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_eq
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- **Scott 1972, Proposition 3.14 (continuity).** `fix` preserves directed suprema.
    Direct
lattice argument (no Kleene iteration): write `g = ⊔S` and `a = ⊔{fix f : f ∈ S}`. The
    reverse
bound `a ⊑ fix g` is `fix`-monotonicity. For `fix g ⊑ a` it suffices (by `fix_le`) that
     `a` is a
pre-fixed point of `g`. Now `g a = ⊔_{f∈S} f a` (pointwise sup), and each `f a = ⊔_{f'∈
    S} f (fix f')`
(continuity of `f` on the directed family `{fix f'}`); for any `f, f' ∈ S` pick `h ∈ S`
     above both,
```

```
so `f (fix f') ⊑ h (fix f') ⊑ h (fix h) = fix h ⊑ a`. Hence `g a ⊑ a`. -/
theorem fix_preservesDirectedSup : PreservesDirectedSup (fix : ScottMap D D → D) := by
  intro S hSne hSdir
  set T : Set D := Set.image fix S with hTdef
  have hTne : T.Nonempty := hSne.image fix
  have hTdir : DirectedOn (· ≤ ·) T := by
    rintro _ ⟨f, hf, rfl⟩ _ ⟨f', hf', rfl⟩
    obtain ⟨h, hh, hfh, hf'h⟩ := hSdir f hf f' hf'
    exact ⟨fix h, Set.mem_image_of_mem fix hh, fix_mono hfh, fix_mono hf'h⟩
  show fix (sSup S) = sSup T
  apply le_antisymm
  · apply fix_le
    show ((sSup S : ScottMap D D) : D → D) (sSup T) ≤ sSup T
    rw [ScottMap.sSup_apply]
    apply sSup_le
    rintro _ ⟨f, hf, rfl⟩
    show (f : D → D) (sSup T) ≤ sSup T
    rw [f.preservesDirectedSup_coe T hTne hTdir]
    apply sSup_le
    rintro _ ⟨_, ⟨f', hf', rfl⟩, rfl⟩
    obtain ⟨h, hh, hfh, hf'h⟩ := hSdir f hf f' hf'
    calc (f : D → D) (fix f')
        ≤ (h : D → D) (fix f') := ScottMap.le_def.mp hfh (fix f')
      _ ≤ (h : D → D) (fix h) := h.monotone (fix_mono hf'h)
      _ = fix h := fix_eq h
      _ ≤ sSup T := le_sSup (Set.mem_image_of_mem fix hh)
  · apply sSup_le
    rintro _ ⟨f, hf, rfl⟩
    exact fix_mono (le_sSup hf)

end Proposition314

end DomainTheory.ContinuousLattice
```

7

fix_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_mono
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_le
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_eq
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice
```

```
open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- **Scott 1972, Proposition 3.14 (continuity).** `fix` preserves directed suprema.
    Direct
lattice argument (no Kleene iteration): write `g = ⊔S` and `a = ⊔{fix f : f ∈ S}`. The
    reverse
bound `a ⊑ fix g` is `fix`-monotonicity. For `fix g ⊑ a` it suffices (by `fix_le`) that
     `a` is a
pre-fixed point of `g`. Now `g a = ⊔_{f∈S} f a` (pointwise sup), and each `f a = ⊔_{f'∈
    S} f (fix f')`
(continuity of `f` on the directed family `{fix f'}`); for any `f, f' ∈ S` pick `h ∈ S`
     above both,
so `f (fix f') ⊑ h (fix f') ⊑ h (fix h) = fix h ⊑ a`. Hence `g a ⊑ a`. -/
theorem fix_preservesDirectedSup : PreservesDirectedSup (fix : ScottMap D D → D) := by
  intro S hSne hSdir
  set T : Set D := Set.image fix S with hTdef
  have hTne : T.Nonempty := hSne.image fix
  have hTdir : DirectedOn (· ≤ ·) T := by
    rintro _ ⟨f, hf, rfl⟩ _ ⟨f', hf', rfl⟩
    obtain ⟨h, hh, hfh, hf'h⟩ := hSdir f hf f' hf'
    exact ⟨fix h, Set.mem_image_of_mem fix hh, fix_mono hfh, fix_mono hf'h⟩
  show fix (sSup S) = sSup T
  apply le_antisymm
  · apply fix_le
    show ((sSup S : ScottMap D D) : D → D) (sSup T) ≤ sSup T
    rw [ScottMap.sSup_apply]
    apply sSup_le
    rintro _ ⟨f, hf, rfl⟩
    show (f : D → D) (sSup T) ≤ sSup T
    rw [f.preservesDirectedSup_coe T hTne hTdir]
    apply sSup_le
    rintro _ ⟨_, ⟨f', hf', rfl⟩, rfl⟩
    obtain ⟨h, hh, hfh, hf'h⟩ := hSdir f hf f' hf'
    calc (f : D → D) (fix f')
        ≤ (h : D → D) (fix f') := ScottMap.le_def.mp hfh (fix f')
      _ ≤ (h : D → D) (fix h) := h.monotone (fix_mono hf'h)
      _ = fix h := fix_eq h
      _ ≤ sSup T := le_sSup (Set.mem_image_of_mem fix hh)
  · apply sSup_le
    rintro _ ⟨f, hf, rfl⟩
    exact fix_mono (le_sSup hf)

end Proposition314

end DomainTheory.ContinuousLattice
```

fix_preservesDirectedSup.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_mono
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_le
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_eq
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- **Scott 1972, Proposition 3.14 (continuity).** `fix` preserves directed suprema.
    Direct
lattice argument (no Kleene iteration): write `g = ⊔S` and `a = ⊔{fix f : f ∈ S}`. The
    reverse
bound `a ⊑ fix g` is `fix`-monotonicity. For `fix g ⊑ a` it suffices (by `fix_le`) that
     `a` is a
pre-fixed point of `g`. Now `g a = ⊔_{f∈S} f a` (pointwise sup), and each `f a = ⊔_{f'∈
    S} f (fix f')`
(continuity of `f` on the directed family `{fix f'}`); for any `f, f' ∈ S` pick `h ∈ S`
     above both,
so `f (fix f') ⊑ h (fix f') ⊑ h (fix h) = fix h ⊑ a`. Hence `g a ⊑ a`. -/
theorem fix_preservesDirectedSup : PreservesDirectedSup (fix : ScottMap D D → D) := by
  intro S hSne hSdir
  set T : Set D := Set.image fix S with hTdef
  have hTne : T.Nonempty := hSne.image fix
  have hTdir : DirectedOn (· ≤ ·) T := by
    rintro _ ⟨f, hf, rfl⟩ _ ⟨f', hf', rfl⟩
    obtain ⟨h, hh, hfh, hf'h⟩ := hSdir f hf f' hf'
    exact ⟨fix h, Set.mem_image_of_mem fix hh, fix_mono hfh, fix_mono hf'h⟩
  show fix (sSup S) = sSup T
  apply le_antisymm
```

```
  · apply fix_le
    show ((sSup S : ScottMap D D) : D → D) (sSup T) ≤ sSup T
    rw [ScottMap.sSup_apply]
    apply sSup_le
    rintro _ ⟨f, hf, rfl⟩
    show (f : D → D) (sSup T) ≤ sSup T
    rw [f.preservesDirectedSup_coe T hTne hTdir]
    apply sSup_le
    rintro _ ⟨_, ⟨f', hf', rfl⟩, rfl⟩
    obtain ⟨h, hh, hfh, hf'h⟩ := hSdir f hf f' hf'
    calc (f : D → D) (fix f')
        ≤ (h : D → D) (fix f') := ScottMap.le_def.mp hfh (fix f')
      _ ≤ (h : D → D) (fix h) := h.monotone (fix_mono hf'h)
      _ = fix h := fix_eq h
      _ ≤ sSup T := le_sSup (Set.mem_image_of_mem fix hh)
  · apply sSup_le
    rintro _ ⟨f, hf, rfl⟩
    exact fix_mono (le_sSup hf)

end Proposition314

end DomainTheory.ContinuousLattice
```

9

fix_unique.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_le
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_eq
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- Uniqueness: any value that is a fixed point of `f` and below every pre-fixed point
    equals
`fix f` (the least fixed point is unique). -/
theorem fix_unique {f : ScottMap D D} {a : D} (hfix : (f : D → D) a = a)
    (hleast : ∀ x, (f : D → D) x ≤ x → a ≤ x) : a = fix f :=
  le_antisymm (hleast (fix f) (le_of_eq (fix_eq f))) (fix_le hfix.le)

end Proposition314
```

```
end DomainTheory.ContinuousLattice
```

0

fix_unique.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_le
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_eq
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- Uniqueness: any value that is a fixed point of `f` and below every pre-fixed point
    equals
`fix f` (the least fixed point is unique). -/
theorem fix_unique {f : ScottMap D D} {a : D} (hfix : (f : D → D) a = a)
    (hleast : ∀ x, (f : D → D) x ≤ x → a ≤ x) : a = fix f :=
  le_antisymm (hleast (fix f) (le_of_eq (fix_eq f))) (fix_le hfix.le)

end Proposition314
end DomainTheory.ContinuousLattice
```

1

fix_unique.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_le
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_eq
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
```

```
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- Uniqueness: any value that is a fixed point of `f` and below every pre-fixed point
    equals
`fix f` (the least fixed point is unique). -/
theorem fix_unique {f : ScottMap D D} {a : D} (hfix : (f : D → D) a = a)
    (hleast : ∀ x, (f : D → D) x ≤ x → a ≤ x) : a = fix f :=
  le_antisymm (hleast (fix f) (le_of_eq (fix_eq f))) (fix_le hfix.le)

end Proposition314
end DomainTheory.ContinuousLattice
```

2

fix_unique.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_le
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_eq
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- Uniqueness: any value that is a fixed point of `f` and below every pre-fixed point
    equals
`fix f` (the least fixed point is unique). -/
theorem fix_unique {f : ScottMap D D} {a : D} (hfix : (f : D → D) a = a)
    (hleast : ∀ x, (f : D → D) x ≤ x → a ≤ x) : a = fix f :=
  le_antisymm (hleast (fix f) (le_of_eq (fix_eq f))) (fix_le hfix.le)

end Proposition314
end DomainTheory.ContinuousLattice
```

3

fix_unique.lean

```
module
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_le
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_eq
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- Uniqueness: any value that is a fixed point of `f` and below every pre-fixed point
    equals
`fix f` (the least fixed point is unique). -/
theorem fix_unique {f : ScottMap D D} {a : D} (hfix : (f : D → D) a = a)
    (hleast : ∀ x, (f : D → D) x ≤ x → a ≤ x) : a = fix f :=
  le_antisymm (hleast (fix f) (le_of_eq (fix_eq f))) (fix_le hfix.le)

end Proposition314
end DomainTheory.ContinuousLattice
```

4

fix_unique.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_le
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_eq
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- Uniqueness: any value that is a fixed point of `f` and below every pre-fixed point
```

```
    equals
`fix f` (the least fixed point is unique). -/
theorem fix_unique {f : ScottMap D D} {a : D} (hfix : (f : D → D) a = a)
    (hleast : ∀ x, (f : D → D) x ≤ x → a ≤ x) : a = fix f :=
  le_antisymm (hleast (fix f) (le_of_eq (fix_eq f))) (fix_le hfix.le)

end Proposition314
end DomainTheory.ContinuousLattice
```

5

fix_unique.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_le
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_eq
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- Uniqueness: any value that is a fixed point of `f` and below every pre-fixed point
    equals
`fix f` (the least fixed point is unique). -/
theorem fix_unique {f : ScottMap D D} {a : D} (hfix : (f : D → D) a = a)
    (hleast : ∀ x, (f : D → D) x ≤ x → a ≤ x) : a = fix f :=
  le_antisymm (hleast (fix f) (le_of_eq (fix_eq f))) (fix_le hfix.le)

end Proposition314
end DomainTheory.ContinuousLattice
```

6

fix_unique.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_le
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_eq
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section
```

```
namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- Uniqueness: any value that is a fixed point of `f` and below every pre-fixed point
    equals
`fix f` (the least fixed point is unique). -/
theorem fix_unique {f : ScottMap D D} {a : D} (hfix : (f : D → D) a = a)
    (hleast : ∀ x, (f : D → D) x ≤ x → a ≤ x) : a = fix f :=
  le_antisymm (hleast (fix f) (le_of_eq (fix_eq f))) (fix_le hfix.le)

end Proposition314
end DomainTheory.ContinuousLattice
```

7

fix_unique.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_le
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_eq
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- Uniqueness: any value that is a fixed point of `f` and below every pre-fixed point
    equals
`fix f` (the least fixed point is unique). -/
theorem fix_unique {f : ScottMap D D} {a : D} (hfix : (f : D → D) a = a)
    (hleast : ∀ x, (f : D → D) x ≤ x → a ≤ x) : a = fix f :=
  le_antisymm (hleast (fix f) (le_of_eq (fix_eq f))) (fix_le hfix.le)

end Proposition314
```

```
end DomainTheory.ContinuousLattice
```

8

fix_unique.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_le
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_eq
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

namespace Proposition314

/-- Uniqueness: any value that is a fixed point of `f` and below every pre-fixed point
    equals
`fix f` (the least fixed point is unique). -/
theorem fix_unique {f : ScottMap D D} {a : D} (hfix : (f : D → D) a = a)
    (hleast : ∀ x, (f : D → D) x ≤ x → a ≤ x) : a = fix f :=
  le_antisymm (hleast (fix f) (le_of_eq (fix_eq f))) (fix_le hfix.le)

end Proposition314
end DomainTheory.ContinuousLattice
```

9

proposition_3_14.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_unique
public import DomainTheory.ContinuousLattice.FunctionSpaces.fixMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_le
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_eq
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice
```

```
open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Proposition 3.14.** For a continuous lattice `D` there is a uniquely
    determined
continuous mapping `fix : [D → D] → D` such that `f (fix f) = fix f` for all `f`, and
    `fix f ⊑ x`
whenever `f x ⊑ x`. Existence and continuity are `Proposition314.fixMap`; the defining
    equations are
`fix_eq`/`fix_le`; uniqueness (any operator with these two properties agrees with `fix
    `) is
`fix_unique`. -/
theorem proposition_3_14 (_hD : ContinuousLattice D) :
    ∃ Fix : ScottMap (ScottMap D D) D,
      (∀ f : ScottMap D D, (f : D → D) (Fix f) = Fix f)
        ∧ (∀ (f : ScottMap D D) (x : D), (f : D → D) x ≤ x → (Fix f : D) ≤ x)
        ∧ (∀ g : ScottMap D D → D,
            (∀ f : ScottMap D D, (f : D → D) (g f) = g f) →
            (∀ (f : ScottMap D D) (x : D), (f : D → D) x ≤ x → g f ≤ x) →
            ∀ f : ScottMap D D, g f = Fix f) :=
  ⟨Proposition314.fixMap, fun f => Proposition314.fix_eq f,
    fun _ _ h => Proposition314.fix_le h,
    fun _ hfix hleast f => Proposition314.fix_unique (hfix f) (fun x hx => hleast f x
    hx)⟩

end DomainTheory.ContinuousLattice
```

0

proposition_3_14.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_unique
public import DomainTheory.ContinuousLattice.FunctionSpaces.fixMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_le
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_eq
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
```

```
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Proposition 3.14.** For a continuous lattice `D` there is a uniquely
    determined
continuous mapping `fix : [D → D] → D` such that `f (fix f) = fix f` for all `f`, and
    `fix f ⊑ x`
whenever `f x ⊑ x`. Existence and continuity are `Proposition314.fixMap`; the defining
    equations are
`fix_eq`/`fix_le`; uniqueness (any operator with these two properties agrees with `fix
    `) is
`fix_unique`. -/
theorem proposition_3_14 (_hD : ContinuousLattice D) :
    ∃ Fix : ScottMap (ScottMap D D) D,
      (∀ f : ScottMap D D, (f : D → D) (Fix f) = Fix f)
        ∧ (∀ (f : ScottMap D D) (x : D), (f : D → D) x ≤ x → (Fix f : D) ≤ x)
        ∧ (∀ g : ScottMap D D → D,
            (∀ f : ScottMap D D, (f : D → D) (g f) = g f) →
            (∀ (f : ScottMap D D) (x : D), (f : D → D) x ≤ x → g f ≤ x) →
            ∀ f : ScottMap D D, g f = Fix f) :=
  ⟨Proposition314.fixMap, fun f => Proposition314.fix_eq f,
    fun _ _ h => Proposition314.fix_le h,
    fun _ hfix hleast f => Proposition314.fix_unique (hfix f) (fun x hx => hleast f x
    hx)⟩

end DomainTheory.ContinuousLattice
```

1

proposition_3_14.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_unique
public import DomainTheory.ContinuousLattice.FunctionSpaces.fixMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_le
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_eq
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Proposition 3.14.** For a continuous lattice `D` there is a uniquely
    determined
```

```
continuous mapping `fix : [D → D] → D` such that `f (fix f) = fix f` for all `f`, and
    `fix f ⊑ x`
whenever `f x ⊑ x`. Existence and continuity are `Proposition314.fixMap`; the defining
    equations are
`fix_eq`/`fix_le`; uniqueness (any operator with these two properties agrees with `fix
    `) is
`fix_unique`. -/
theorem proposition_3_14 (_hD : ContinuousLattice D) :
    ∃ Fix : ScottMap (ScottMap D D) D,
      (∀ f : ScottMap D D, (f : D → D) (Fix f) = Fix f)
        ∧ (∀ (f : ScottMap D D) (x : D), (f : D → D) x ≤ x → (Fix f : D) ≤ x)
        ∧ (∀ g : ScottMap D D → D,
            (∀ f : ScottMap D D, (f : D → D) (g f) = g f) →
            (∀ (f : ScottMap D D) (x : D), (f : D → D) x ≤ x → g f ≤ x) →
            ∀ f : ScottMap D D, g f = Fix f) :=
  ⟨Proposition314.fixMap, fun f => Proposition314.fix_eq f,
    fun _ _ h => Proposition314.fix_le h,
    fun _ hfix hleast f => Proposition314.fix_unique (hfix f) (fun x hx => hleast f x
    hx)⟩

end DomainTheory.ContinuousLattice
```

2

proposition_3_14.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_unique
public import DomainTheory.ContinuousLattice.FunctionSpaces.fixMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_le
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_eq
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Proposition 3.14.** For a continuous lattice `D` there is a uniquely
    determined
continuous mapping `fix : [D → D] → D` such that `f (fix f) = fix f` for all `f`, and
    `fix f ⊑ x`
whenever `f x ⊑ x`. Existence and continuity are `Proposition314.fixMap`; the defining
    equations are
`fix_eq`/`fix_le`; uniqueness (any operator with these two properties agrees with `fix
```

```
    `) is
`fix_unique`. -/
theorem proposition_3_14 (_hD : ContinuousLattice D) :
    ∃ Fix : ScottMap (ScottMap D D) D,
      (∀ f : ScottMap D D, (f : D → D) (Fix f) = Fix f)
        ∧ (∀ (f : ScottMap D D) (x : D), (f : D → D) x ≤ x → (Fix f : D) ≤ x)
        ∧ (∀ g : ScottMap D D → D,
            (∀ f : ScottMap D D, (f : D → D) (g f) = g f) →
            (∀ (f : ScottMap D D) (x : D), (f : D → D) x ≤ x → g f ≤ x) →
            ∀ f : ScottMap D D, g f = Fix f) :=
  ⟨Proposition314.fixMap, fun f => Proposition314.fix_eq f,
    fun _ _ h => Proposition314.fix_le h,
    fun _ hfix hleast f => Proposition314.fix_unique (hfix f) (fun x hx => hleast f x
    hx)⟩

end DomainTheory.ContinuousLattice
```

3

proposition_3_14.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_unique
public import DomainTheory.ContinuousLattice.FunctionSpaces.fixMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_le
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_eq
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Proposition 3.14.** For a continuous lattice `D` there is a uniquely
    determined
continuous mapping `fix : [D → D] → D` such that `f (fix f) = fix f` for all `f`, and
    `fix f ⊑ x`
whenever `f x ⊑ x`. Existence and continuity are `Proposition314.fixMap`; the defining
    equations are
`fix_eq`/`fix_le`; uniqueness (any operator with these two properties agrees with `fix
    `) is
`fix_unique`. -/
theorem proposition_3_14 (_hD : ContinuousLattice D) :
    ∃ Fix : ScottMap (ScottMap D D) D,
      (∀ f : ScottMap D D, (f : D → D) (Fix f) = Fix f)
```

```
        ∧ (∀ (f : ScottMap D D) (x : D), (f : D → D) x ≤ x → (Fix f : D) ≤ x)
        ∧ (∀ g : ScottMap D D → D,
            (∀ f : ScottMap D D, (f : D → D) (g f) = g f) →
            (∀ (f : ScottMap D D) (x : D), (f : D → D) x ≤ x → g f ≤ x) →
            ∀ f : ScottMap D D, g f = Fix f) :=
  ⟨Proposition314.fixMap, fun f => Proposition314.fix_eq f,
    fun _ _ h => Proposition314.fix_le h,
    fun _ hfix hleast f => Proposition314.fix_unique (hfix f) (fun x hx => hleast f x
    hx)⟩

end DomainTheory.ContinuousLattice
```

4

proposition_3_14.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_unique
public import DomainTheory.ContinuousLattice.FunctionSpaces.fixMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_le
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_eq
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Proposition 3.14.** For a continuous lattice `D` there is a uniquely
    determined
continuous mapping `fix : [D → D] → D` such that `f (fix f) = fix f` for all `f`, and
    `fix f ⊑ x`
whenever `f x ⊑ x`. Existence and continuity are `Proposition314.fixMap`; the defining
    equations are
`fix_eq`/`fix_le`; uniqueness (any operator with these two properties agrees with `fix
    `) is
`fix_unique`. -/
theorem proposition_3_14 (_hD : ContinuousLattice D) :
    ∃ Fix : ScottMap (ScottMap D D) D,
      (∀ f : ScottMap D D, (f : D → D) (Fix f) = Fix f)
        ∧ (∀ (f : ScottMap D D) (x : D), (f : D → D) x ≤ x → (Fix f : D) ≤ x)
        ∧ (∀ g : ScottMap D D → D,
            (∀ f : ScottMap D D, (f : D → D) (g f) = g f) →
            (∀ (f : ScottMap D D) (x : D), (f : D → D) x ≤ x → g f ≤ x) →
            ∀ f : ScottMap D D, g f = Fix f) :=
```

```
  ⟨Proposition314.fixMap, fun f => Proposition314.fix_eq f,
    fun _ _ h => Proposition314.fix_le h,
    fun _ hfix hleast f => Proposition314.fix_unique (hfix f) (fun x hx => hleast f x
    hx)⟩

end DomainTheory.ContinuousLattice
```

5

proposition_3_14.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_unique
public import DomainTheory.ContinuousLattice.FunctionSpaces.fixMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_le
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_eq
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Proposition 3.14.** For a continuous lattice `D` there is a uniquely
    determined
continuous mapping `fix : [D → D] → D` such that `f (fix f) = fix f` for all `f`, and
    `fix f ⊑ x`
whenever `f x ⊑ x`. Existence and continuity are `Proposition314.fixMap`; the defining
    equations are
`fix_eq`/`fix_le`; uniqueness (any operator with these two properties agrees with `fix
    `) is
`fix_unique`. -/
theorem proposition_3_14 (_hD : ContinuousLattice D) :
    ∃ Fix : ScottMap (ScottMap D D) D,
      (∀ f : ScottMap D D, (f : D → D) (Fix f) = Fix f)
        ∧ (∀ (f : ScottMap D D) (x : D), (f : D → D) x ≤ x → (Fix f : D) ≤ x)
        ∧ (∀ g : ScottMap D D → D,
            (∀ f : ScottMap D D, (f : D → D) (g f) = g f) →
            (∀ (f : ScottMap D D) (x : D), (f : D → D) x ≤ x → g f ≤ x) →
            ∀ f : ScottMap D D, g f = Fix f) :=
  ⟨Proposition314.fixMap, fun f => Proposition314.fix_eq f,
    fun _ _ h => Proposition314.fix_le h,
    fun _ hfix hleast f => Proposition314.fix_unique (hfix f) (fun x hx => hleast f x
    hx)⟩
```

```
end DomainTheory.ContinuousLattice
```

6

proposition_3_14.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_unique
public import DomainTheory.ContinuousLattice.FunctionSpaces.fixMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_le
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_eq
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Proposition 3.14.** For a continuous lattice `D` there is a uniquely
    determined
continuous mapping `fix : [D → D] → D` such that `f (fix f) = fix f` for all `f`, and
    `fix f ⊑ x`
whenever `f x ⊑ x`. Existence and continuity are `Proposition314.fixMap`; the defining
    equations are
`fix_eq`/`fix_le`; uniqueness (any operator with these two properties agrees with `fix
    `) is
`fix_unique`. -/
theorem proposition_3_14 (_hD : ContinuousLattice D) :
    ∃ Fix : ScottMap (ScottMap D D) D,
      (∀ f : ScottMap D D, (f : D → D) (Fix f) = Fix f)
        ∧ (∀ (f : ScottMap D D) (x : D), (f : D → D) x ≤ x → (Fix f : D) ≤ x)
        ∧ (∀ g : ScottMap D D → D,
            (∀ f : ScottMap D D, (f : D → D) (g f) = g f) →
            (∀ (f : ScottMap D D) (x : D), (f : D → D) x ≤ x → g f ≤ x) →
            ∀ f : ScottMap D D, g f = Fix f) :=
  ⟨Proposition314.fixMap, fun f => Proposition314.fix_eq f,
    fun _ _ h => Proposition314.fix_le h,
    fun _ hfix hleast f => Proposition314.fix_unique (hfix f) (fun x hx => hleast f x
    hx)⟩

end DomainTheory.ContinuousLattice
```

7

proposition_3_14.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_unique
public import DomainTheory.ContinuousLattice.FunctionSpaces.fixMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_le
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_eq
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Proposition 3.14.** For a continuous lattice `D` there is a uniquely
    determined
continuous mapping `fix : [D → D] → D` such that `f (fix f) = fix f` for all `f`, and
    `fix f ⊑ x`
whenever `f x ⊑ x`. Existence and continuity are `Proposition314.fixMap`; the defining
    equations are
`fix_eq`/`fix_le`; uniqueness (any operator with these two properties agrees with `fix
    `) is
`fix_unique`. -/
theorem proposition_3_14 (_hD : ContinuousLattice D) :
    ∃ Fix : ScottMap (ScottMap D D) D,
      (∀ f : ScottMap D D, (f : D → D) (Fix f) = Fix f)
        ∧ (∀ (f : ScottMap D D) (x : D), (f : D → D) x ≤ x → (Fix f : D) ≤ x)
        ∧ (∀ g : ScottMap D D → D,
            (∀ f : ScottMap D D, (f : D → D) (g f) = g f) →
            (∀ (f : ScottMap D D) (x : D), (f : D → D) x ≤ x → g f ≤ x) →
            ∀ f : ScottMap D D, g f = Fix f) :=
  ⟨Proposition314.fixMap, fun f => Proposition314.fix_eq f,
    fun _ _ h => Proposition314.fix_le h,
    fun _ hfix hleast f => Proposition314.fix_unique (hfix f) (fun x hx => hleast f x
    hx)⟩

end DomainTheory.ContinuousLattice
```

8

proposition_3_14.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_unique
public import DomainTheory.ContinuousLattice.FunctionSpaces.fixMap
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_le
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_eq
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D₀ D₀' D₁ D₁' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D₀] [CompleteLattice D₀'] [CompleteLattice D₁] [
    CompleteLattice D₁']

/-- **Scott 1972, Proposition 3.14.** For a continuous lattice `D` there is a uniquely
    determined
continuous mapping `fix : [D → D] → D` such that `f (fix f) = fix f` for all `f`, and
    `fix f ⊑ x`
whenever `f x ⊑ x`. Existence and continuity are `Proposition314.fixMap`; the defining
    equations are
`fix_eq`/`fix_le`; uniqueness (any operator with these two properties agrees with `fix
    `) is
`fix_unique`. -/
theorem proposition_3_14 (_hD : ContinuousLattice D) :
    ∃ Fix : ScottMap (ScottMap D D) D,
      (∀ f : ScottMap D D, (f : D → D) (Fix f) = Fix f)
        ∧ (∀ (f : ScottMap D D) (x : D), (f : D → D) x ≤ x → (Fix f : D) ≤ x)
        ∧ (∀ g : ScottMap D D → D,
            (∀ f : ScottMap D D, (f : D → D) (g f) = g f) →
            (∀ (f : ScottMap D D) (x : D), (f : D → D) x ≤ x → g f ≤ x) →
            ∀ f : ScottMap D D, g f = Fix f) :=
  ⟨Proposition314.fixMap, fun f => Proposition314.fix_eq f,
    fun _ _ h => Proposition314.fix_le h,
    fun _ hfix hleast f => Proposition314.fix_unique (hfix f) (fun x hx => hleast f x
    hx)⟩

end DomainTheory.ContinuousLattice
```

9

Compatible.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set
```

```
universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- Compatibility of a sequence: `j_n(x_{n+1}) = x_n` for all `n`. -/
def Compatible (x : ∀ n, D n) : Prop := ∀ n, (P n).retr (x (n + 1)) = x n

end InverseLimit

end DomainTheory.ContinuousLattice
```

0

Compatible.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- Compatibility of a sequence: `j_n(x_{n+1}) = x_n` for all `n`. -/
def Compatible (x : ∀ n, D n) : Prop := ∀ n, (P n).retr (x (n + 1)) = x n

end InverseLimit

end DomainTheory.ContinuousLattice
```

1

Compatible.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Data.Set.Basic

@[expose] public section
```

```
namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- Compatibility of a sequence: `j_n(x_{n+1}) = x_n` for all `n`. -/
def Compatible (x : ∀ n, D n) : Prop := ∀ n, (P n).retr (x (n + 1)) = x n

end InverseLimit

end DomainTheory.ContinuousLattice
```

2

Compatible.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- Compatibility of a sequence: `j_n(x_{n+1}) = x_n` for all `n`. -/
def Compatible (x : ∀ n, D n) : Prop := ∀ n, (P n).retr (x (n + 1)) = x n

end InverseLimit

end DomainTheory.ContinuousLattice
```

3

Compatible.lean

```
module
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- Compatibility of a sequence: `j_n(x_{n+1}) = x_n` for all `n`. -/
def Compatible (x : ∀ n, D n) : Prop := ∀ n, (P n).retr (x (n + 1)) = x n

end InverseLimit

end DomainTheory.ContinuousLattice
```

4

Compatible.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- Compatibility of a sequence: `j_n(x_{n+1}) = x_n` for all `n`. -/
def Compatible (x : ∀ n, D n) : Prop := ∀ n, (P n).retr (x (n + 1)) = x n

end InverseLimit

end DomainTheory.ContinuousLattice
```

5

Compatible.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- Compatibility of a sequence: `jₙ(x_{n+1}) = xₙ` for all `n`. -/
def Compatible (x : ∀ n, D n) : Prop := ∀ n, (P n).retr (x (n + 1)) = x n

end InverseLimit

end DomainTheory.ContinuousLattice
```


Compatible.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- Compatibility of a sequence: `jₙ(x_{n+1}) = xₙ` for all `n`. -/
def Compatible (x : ∀ n, D n) : Prop := ∀ n, (P n).retr (x (n + 1)) = x n

end InverseLimit
```

```
end DomainTheory.ContinuousLattice
```

7

Compatible.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- Compatibility of a sequence: `j_n(x_{n+1}) = x_n` for all `n`. -/
def Compatible (x : ∀ n, D n) : Prop := ∀ n, (P n).retr (x (n + 1)) = x n

end InverseLimit

end DomainTheory.ContinuousLattice
```

8

Compatible.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- Compatibility of a sequence: `j_n(x_{n+1}) = x_n` for all `n`. -/
```

```
def Compatible (x : ∀ n, D n) : Prop := ∀ n, (P n).retr (x (n + 1)) = x n

end InverseLimit

end DomainTheory.ContinuousLattice
```

9

InverseLimit.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.Compatible
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- **Scott 1972, §4.** The inverse limit `D_∞` as the subspace of compatible
    sequences. -/
abbrev InverseLimit : Type u := {x : ∀ n, D n // Compatible D P x}

end InverseLimit

end DomainTheory.ContinuousLattice
```

0

InverseLimit.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.Compatible
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit
```

```
variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- **Scott 1972, §4.** The inverse limit `D_∞` as the subspace of compatible
    sequences. -/
abbrev InverseLimit : Type u := {x : ∀ n, D n // Compatible D P x}

end InverseLimit

end DomainTheory.ContinuousLattice
```

1

InverseLimit.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.Compatible
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- **Scott 1972, §4.** The inverse limit `D_∞` as the subspace of compatible
    sequences. -/
abbrev InverseLimit : Type u := {x : ∀ n, D n // Compatible D P x}

end InverseLimit

end DomainTheory.ContinuousLattice
```

2

InverseLimit.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.Compatible
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice
```

```
open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- **Scott 1972, §4.** The inverse limit `D_∞` as the subspace of compatible
    sequences. -/
abbrev InverseLimit : Type u := {x : ∀ n, D n // Compatible D P x}

end InverseLimit

end DomainTheory.ContinuousLattice
```

3

InverseLimit.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.Compatible
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- **Scott 1972, §4.** The inverse limit `D_∞` as the subspace of compatible
    sequences. -/
abbrev InverseLimit : Type u := {x : ∀ n, D n // Compatible D P x}

end InverseLimit

end DomainTheory.ContinuousLattice
```

4

InverseLimit.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.Compatible
```

```
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- **Scott 1972, §4.** The inverse limit `D_∞` as the subspace of compatible
    sequences. -/
abbrev InverseLimit : Type u := {x : ∀ n, D n // Compatible D P x}

end InverseLimit

end DomainTheory.ContinuousLattice
```

5

InverseLimit.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.Compatible
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- **Scott 1972, §4.** The inverse limit `D_∞` as the subspace of compatible
    sequences. -/
abbrev InverseLimit : Type u := {x : ∀ n, D n // Compatible D P x}

end InverseLimit

end DomainTheory.ContinuousLattice
```

6

InverseLimit.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.Compatible
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- **Scott 1972, §4.** The inverse limit `D_∞` as the subspace of compatible
    sequences. -/
abbrev InverseLimit : Type u := {x : ∀ n, D n // Compatible D P x}

end InverseLimit

end DomainTheory.ContinuousLattice
```


InverseLimit.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.Compatible
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- **Scott 1972, §4.** The inverse limit `D_∞` as the subspace of compatible
    sequences. -/
abbrev InverseLimit : Type u := {x : ∀ n, D n // Compatible D P x}
```

```
end InverseLimit

end DomainTheory.ContinuousLattice
```

8

InverseLimit.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.Compatible
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- **Scott 1972, §4.** The inverse limit `D_∞` as the subspace of compatible
    sequences. -/
abbrev InverseLimit : Type u := {x : ∀ n, D n // Compatible D P x}

end InverseLimit

end DomainTheory.ContinuousLattice
```

9

projLE.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.comp
public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u
```

```
section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- Descend the tower of projections: for `m ≤ n`, `projLE h = j_m ∘ ... ∘ j_{n-1} :
    D_n → D_m`. -/
def projLE {m n : ℕ} (h : m ≤ n) (x : D n) : D m :=
  Nat.leRecOn (C := fun k => D k → D m) h
    (fun {k} (g : D k → D m) (w : D (k + 1)) => g ((P k).retr w)) id x

namespace projLE

theorem self {m : ℕ} (h : m ≤ m) (x : D m) : projLE D P h x = x := by
  simp only [projLE, Nat.leRecOn_self, id_eq]

theorem succ {m n : ℕ} (h1 : m ≤ n) (h2 : m ≤ n + 1) (z : D (n + 1)) :
    projLE D P h2 z = projLE D P h1 ((P n).retr z) := by
  simp only [projLE]
  rw [Nat.leRecOn_succ (C := fun k => D k → D m) h1]

theorem mono {m n : ℕ} (h : m ≤ n) {x y : D n} (hxy : x ≤ y) :
    projLE D P h x ≤ projLE D P h y := by
  induction n, h using Nat.le_induction with
  | base =>
      rw [self, self]
      exact hxy
  | succ k hk ih =>
      rw [succ D P hk (Nat.le_succ_of_le hk), succ D P hk (Nat.le_succ_of_le hk)]
      exact ih ((P k).retr.monotone hxy)

/-- For a compatible sequence `y`, descending from level `n` to `m ≤ n` recovers `y_m`.
    -/
theorem compatible {m : ℕ} (y : InverseLimit D P) :
    ∀ {n : ℕ} (h : m ≤ n), projLE D P h (y.1 n) = y.1 m := by
  intro n h
  induction n, h using Nat.le_induction with
  | base => rw [self]
  | succ k hk ih => rw [succ D P hk (Nat.le_succ_of_le hk), y.2 k, ih]

/-- Peeling the last projection: `(P m).retr ∘ projLE (m+1 ≤ n) = projLE (m ≤ n)`. -/
theorem retr {m : ℕ} : ∀ {n : ℕ} (h : m + 1 ≤ n) (x : D n),
    (P m).retr (projLE D P h x) = projLE D P (Nat.le_of_succ_le h) x := by
  intro n h
  induction n, h using Nat.le_induction with
  | base =>
      intro x
      rw [succ D P (le_refl m) (Nat.le_of_succ_le (le_refl (m + 1))) x]
      simp only [self]
  | succ k hk ih =>
      intro x
      rw [succ D P hk (Nat.le_succ_of_le hk) x, ih ((P k).retr x),
        succ D P (Nat.le_of_succ_le hk) (Nat.le_of_succ_le (Nat.le_succ_of_le hk)) x]
```

```
/-- The tower of projections is Scott-continuous (a composite of the `j_k`). -/
theorem preservesDirectedSup {m n : ℕ} (h : m ≤ n) :
    PreservesDirectedSup (projLE D P h) := by
  induction n, h using Nat.le_induction with
  | base =>
      have hid : projLE D P (le_refl m) = id := funext (fun x => self D P (le_refl m)
    x)
      rw [hid]; intro S _ _; simp
  | succ k hk ih =>
      have hfun : projLE D P (Nat.le_succ_of_le hk)
          = projLE D P hk ∘ (fun z : D (k + 1) => (P k).retr z) :=
        funext (fun x => succ D P hk (Nat.le_succ_of_le hk) x)
      rw [hfun]
      exact ScottMap.preservesDirectedSup_comp ih
        (fun S hS hd => (P k).retr.preservesDirectedSup_coe S hS hd)

end projLE

end InverseLimit

end DomainTheory.ContinuousLattice
```

0

projLE.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.comp
public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- Descend the tower of projections: for `m ≤ n`, `projLE h = j_m ∘ ... ∘ j_{n-1} :
    D_n → D_m`. -/
def projLE {m n : ℕ} (h : m ≤ n) (x : D n) : D m :=
  Nat.leRecOn (C := fun k => D k → D m) h
    (fun {k} (g : D k → D m) (w : D (k + 1)) => g ((P k).retr w)) id x
```

```
namespace projLE

theorem self {m : ℕ} (h : m ≤ m) (x : D m) : projLE D P h x = x := by
  simp only [projLE, Nat.leRecOn_self, id_eq]

theorem succ {m n : ℕ} (h1 : m ≤ n) (h2 : m ≤ n + 1) (z : D (n + 1)) :
    projLE D P h2 z = projLE D P h1 ((P n).retr z) := by
  simp only [projLE]
  rw [Nat.leRecOn_succ (C := fun k => D k → D m) h1]

theorem mono {m n : ℕ} (h : m ≤ n) {x y : D n} (hxy : x ≤ y) :
    projLE D P h x ≤ projLE D P h y := by
  induction n, h using Nat.le_induction with
  | base =>
      rw [self, self]
      exact hxy
  | succ k hk ih =>
      rw [succ D P hk (Nat.le_succ_of_le hk), succ D P hk (Nat.le_succ_of_le hk)]
      exact ih ((P k).retr.monotone hxy)

/-- For a compatible sequence `y`, descending from level `n` to `m ≤ n` recovers `yₘ`.
    -/
theorem compatible {m : ℕ} (y : InverseLimit D P) :
    ∀ {n : ℕ} (h : m ≤ n), projLE D P h (y.1 n) = y.1 m := by
  intro n h
  induction n, h using Nat.le_induction with
  | base => rw [self]
  | succ k hk ih => rw [succ D P hk (Nat.le_succ_of_le hk), y.2 k, ih]

/-- Peeling the last projection: `(P m).retr ∘ projLE (m+1 ≤ n) = projLE (m ≤ n)`. -/
theorem retr {m : ℕ} : ∀ {n : ℕ} (h : m + 1 ≤ n) (x : D n),
    (P m).retr (projLE D P h x) = projLE D P (Nat.le_of_succ_le h) x := by
  intro n h
  induction n, h using Nat.le_induction with
  | base =>
      intro x
      rw [succ D P (le_refl m) (Nat.le_of_succ_le (le_refl (m + 1))) x]
      simp only [self]
  | succ k hk ih =>
      intro x
      rw [succ D P hk (Nat.le_succ_of_le hk) x, ih ((P k).retr x),
        succ D P (Nat.le_of_succ_le hk) (Nat.le_of_succ_le (Nat.le_succ_of_le hk)) x]

/-- The tower of projections is Scott-continuous (a composite of the `jₖ`). -/
theorem preservesDirectedSup {m n : ℕ} (h : m ≤ n) :
    PreservesDirectedSup (projLE D P h) := by
  induction n, h using Nat.le_induction with
  | base =>
      have hid : projLE D P (le_refl m) = id := funext (fun x => self D P (le_refl m)
    x)
      rw [hid]; intro S _ _; simp
  | succ k hk ih =>
      have hfun : projLE D P (Nat.le_succ_of_le hk)
          = projLE D P hk ∘ (fun z : D (k + 1) => (P k).retr z) :=
        funext (fun x => succ D P hk (Nat.le_succ_of_le hk) x)
      rw [hfun]
```

```
      exact ScottMap.preservesDirectedSup_comp ih
        (fun S hS hd => (P k).retr.preservesDirectedSup_coe S hS hd)

end projLE

end InverseLimit

end DomainTheory.ContinuousLattice
```

1

projLE.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.comp
public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- Descend the tower of projections: for `m ≤ n`, `projLE h = j_m ∘ ... ∘ j_{n-1} :
    D_n → D_m`. -/
def projLE {m n : ℕ} (h : m ≤ n) (x : D n) : D m :=
  Nat.leRecOn (C := fun k => D k → D m) h
    (fun {k} (g : D k → D m) (w : D (k + 1)) => g ((P k).retr w)) id x

namespace projLE

theorem self {m : ℕ} (h : m ≤ m) (x : D m) : projLE D P h x = x := by
  simp only [projLE, Nat.leRecOn_self, id_eq]

theorem succ {m n : ℕ} (h1 : m ≤ n) (h2 : m ≤ n + 1) (z : D (n + 1)) :
    projLE D P h2 z = projLE D P h1 ((P n).retr z) := by
  simp only [projLE]
  rw [Nat.leRecOn_succ (C := fun k => D k → D m) h1]

theorem mono {m n : ℕ} (h : m ≤ n) {x y : D n} (hxy : x ≤ y) :
    projLE D P h x ≤ projLE D P h y := by
  induction n, h using Nat.le_induction with
```

```
  | base =>
      rw [self, self]
      exact hxy
  | succ k hk ih =>
      rw [succ D P hk (Nat.le_succ_of_le hk), succ D P hk (Nat.le_succ_of_le hk)]
      exact ih ((P k).retr.monotone hxy)

/-- For a compatible sequence `y`, descending from level `n` to `m ≤ n` recovers `y_m`.
    -/
theorem compatible {m : ℕ} (y : InverseLimit D P) :
    ∀ {n : ℕ} (h : m ≤ n), projLE D P h (y.1 n) = y.1 m := by
  intro n h
  induction n, h using Nat.le_induction with
  | base => rw [self]
  | succ k hk ih => rw [succ D P hk (Nat.le_succ_of_le hk), y.2 k, ih]

/-- Peeling the last projection: `(P m).retr ∘ projLE (m+1 ≤ n) = projLE (m ≤ n)`. -/
theorem retr {m : ℕ} : ∀ {n : ℕ} (h : m + 1 ≤ n) (x : D n),
    (P m).retr (projLE D P h x) = projLE D P (Nat.le_of_succ_le h) x := by
  intro n h
  induction n, h using Nat.le_induction with
  | base =>
      intro x
      rw [succ D P (le_refl m) (Nat.le_of_succ_le (le_refl (m + 1))) x]
      simp only [self]
  | succ k hk ih =>
      intro x
      rw [succ D P hk (Nat.le_succ_of_le hk) x, ih ((P k).retr x),
        succ D P (Nat.le_of_succ_le hk) (Nat.le_of_succ_le (Nat.le_succ_of_le hk)) x]

/-- The tower of projections is Scott-continuous (a composite of the `j_k`). -/
theorem preservesDirectedSup {m n : ℕ} (h : m ≤ n) :
    PreservesDirectedSup (projLE D P h) := by
  induction n, h using Nat.le_induction with
  | base =>
      have hid : projLE D P (le_refl m) = id := funext (fun x => self D P (le_refl m)
    x)
      rw [hid]; intro S _ _; simp
  | succ k hk ih =>
      have hfun : projLE D P (Nat.le_succ_of_le hk)
          = projLE D P hk ∘ (fun z : D (k + 1) => (P k).retr z) :=
        funext (fun x => succ D P hk (Nat.le_succ_of_le hk) x)
      rw [hfun]
      exact ScottMap.preservesDirectedSup_comp ih
        (fun S hS hd => (P k).retr.preservesDirectedSup_coe S hS hd)

end projLE

end InverseLimit

end DomainTheory.ContinuousLattice
```

2

projLE.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.comp
public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- Descend the tower of projections: for `m ≤ n`, `projLE h = j_m ∘ ... ∘ j_{n-1} :
    D_n → D_m`. -/
def projLE {m n : ℕ} (h : m ≤ n) (x : D n) : D m :=
  Nat.leRecOn (C := fun k => D k → D m) h
    (fun {k} (g : D k → D m) (w : D (k + 1)) => g ((P k).retr w)) id x

namespace projLE

theorem self {m : ℕ} (h : m ≤ m) (x : D m) : projLE D P h x = x := by
  simp only [projLE, Nat.leRecOn_self, id_eq]

theorem succ {m n : ℕ} (h1 : m ≤ n) (h2 : m ≤ n + 1) (z : D (n + 1)) :
    projLE D P h2 z = projLE D P h1 ((P n).retr z) := by
  simp only [projLE]
  rw [Nat.leRecOn_succ (C := fun k => D k → D m) h1]

theorem mono {m n : ℕ} (h : m ≤ n) {x y : D n} (hxy : x ≤ y) :
    projLE D P h x ≤ projLE D P h y := by
  induction n, h using Nat.le_induction with
  | base =>
      rw [self, self]
      exact hxy
  | succ k hk ih =>
      rw [succ D P hk (Nat.le_succ_of_le hk), succ D P hk (Nat.le_succ_of_le hk)]
      exact ih ((P k).retr.monotone hxy)

/-- For a compatible sequence `y`, descending from level `n` to `m ≤ n` recovers `y_m`.
    -/
theorem compatible {m : ℕ} (y : InverseLimit D P) :
    ∀ {n : ℕ} (h : m ≤ n), projLE D P h (y.1 n) = y.1 m := by
  intro n h
  induction n, h using Nat.le_induction with
```

```
  | base => rw [self]
  | succ k hk ih => rw [succ D P hk (Nat.le_succ_of_le hk), y.2 k, ih]

/-- Peeling the last projection: `(P m).retr ∘ projLE (m+1 ≤ n) = projLE (m ≤ n)`. -/
theorem retr {m : ℕ} : ∀ {n : ℕ} (h : m + 1 ≤ n) (x : D n),
    (P m).retr (projLE D P h x) = projLE D P (Nat.le_of_succ_le h) x := by
  intro n h
  induction n, h using Nat.le_induction with
  | base =>
      intro x
      rw [succ D P (le_refl m) (Nat.le_of_succ_le (le_refl (m + 1))) x]
      simp only [self]
  | succ k hk ih =>
      intro x
      rw [succ D P hk (Nat.le_succ_of_le hk) x, ih ((P k).retr x),
        succ D P (Nat.le_of_succ_le hk) (Nat.le_of_succ_le (Nat.le_succ_of_le hk)) x]

/-- The tower of projections is Scott-continuous (a composite of the `jₖ`). -/
theorem preservesDirectedSup {m n : ℕ} (h : m ≤ n) :
    PreservesDirectedSup (projLE D P h) := by
  induction n, h using Nat.le_induction with
  | base =>
      have hid : projLE D P (le_refl m) = id := funext (fun x => self D P (le_refl m)
    x)
      rw [hid]; intro S _ _; simp
  | succ k hk ih =>
      have hfun : projLE D P (Nat.le_succ_of_le hk)
          = projLE D P hk ∘ (fun z : D (k + 1) => (P k).retr z) :=
        funext (fun x => succ D P hk (Nat.le_succ_of_le hk) x)
      rw [hfun]
      exact ScottMap.preservesDirectedSup_comp ih
        (fun S hS hd => (P k).retr.preservesDirectedSup_coe S hS hd)

end projLE

end InverseLimit

end DomainTheory.ContinuousLattice
```

3

projLE.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.comp
public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice
```

```
open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- Descend the tower of projections: for `m ≤ n`, `projLE h = j_m ∘ ... ∘ j_{n-1} :
    D_n → D_m`. -/
def projLE {m n : ℕ} (h : m ≤ n) (x : D n) : D m :=
  Nat.leRecOn (C := fun k => D k → D m) h
    (fun {k} (g : D k → D m) (w : D (k + 1)) => g ((P k).retr w)) id x

namespace projLE

theorem self {m : ℕ} (h : m ≤ m) (x : D m) : projLE D P h x = x := by
  simp only [projLE, Nat.leRecOn_self, id_eq]

theorem succ {m n : ℕ} (h1 : m ≤ n) (h2 : m ≤ n + 1) (z : D (n + 1)) :
    projLE D P h2 z = projLE D P h1 ((P n).retr z) := by
  simp only [projLE]
  rw [Nat.leRecOn_succ (C := fun k => D k → D m) h1]

theorem mono {m n : ℕ} (h : m ≤ n) {x y : D n} (hxy : x ≤ y) :
    projLE D P h x ≤ projLE D P h y := by
  induction n, h using Nat.le_induction with
  | base =>
      rw [self, self]
      exact hxy
  | succ k hk ih =>
      rw [succ D P hk (Nat.le_succ_of_le hk), succ D P hk (Nat.le_succ_of_le hk)]
      exact ih ((P k).retr.monotone hxy)

/-- For a compatible sequence `y`, descending from level `n` to `m ≤ n` recovers `y_m`.
    -/
theorem compatible {m : ℕ} (y : InverseLimit D P) :
    ∀ {n : ℕ} (h : m ≤ n), projLE D P h (y.1 n) = y.1 m := by
  intro n h
  induction n, h using Nat.le_induction with
  | base => rw [self]
  | succ k hk ih => rw [succ D P hk (Nat.le_succ_of_le hk), y.2 k, ih]

/-- Peeling the last projection: `(P m).retr ∘ projLE (m+1 ≤ n) = projLE (m ≤ n)`. -/
theorem retr {m : ℕ} : ∀ {n : ℕ} (h : m + 1 ≤ n) (x : D n),
    (P m).retr (projLE D P h x) = projLE D P (Nat.le_of_succ_le h) x := by
  intro n h
  induction n, h using Nat.le_induction with
  | base =>
      intro x
      rw [succ D P (le_refl m) (Nat.le_of_succ_le (le_refl (m + 1))) x]
      simp only [self]
  | succ k hk ih =>
```

```
      intro x
      rw [succ D P hk (Nat.le_succ_of_le hk) x, ih ((P k).retr x),
        succ D P (Nat.le_of_succ_le hk) (Nat.le_of_succ_le (Nat.le_succ_of_le hk)) x]

/-- The tower of projections is Scott-continuous (a composite of the `j_k`). -/
theorem preservesDirectedSup {m n : ℕ} (h : m ≤ n) :
    PreservesDirectedSup (projLE D P h) := by
  induction n, h using Nat.le_induction with
  | base =>
      have hid : projLE D P (le_refl m) = id := funext (fun x => self D P (le_refl m)
    x)
      rw [hid]; intro S _ _; simp
  | succ k hk ih =>
      have hfun : projLE D P (Nat.le_succ_of_le hk)
          = projLE D P hk ∘ (fun z : D (k + 1) => (P k).retr z) :=
        funext (fun x => succ D P hk (Nat.le_succ_of_le hk) x)
      rw [hfun]
      exact ScottMap.preservesDirectedSup_comp ih
        (fun S hS hd => (P k).retr.preservesDirectedSup_coe S hS hd)

end projLE

end InverseLimit

end DomainTheory.ContinuousLattice
```

4

projLE.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.comp
public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- Descend the tower of projections: for `m ≤ n`, `projLE h = j_m ∘ ... ∘ j_{n-1} :
    D_n → D_m`. -/
```

```
def projLE {m n : ℕ} (h : m ≤ n) (x : D n) : D m :=
  Nat.leRecOn (C := fun k => D k → D m) h
    (fun {k} (g : D k → D m) (w : D (k + 1)) => g ((P k).retr w)) id x

namespace projLE

theorem self {m : ℕ} (h : m ≤ m) (x : D m) : projLE D P h x = x := by
  simp only [projLE, Nat.leRecOn_self, id_eq]

theorem succ {m n : ℕ} (h1 : m ≤ n) (h2 : m ≤ n + 1) (z : D (n + 1)) :
    projLE D P h2 z = projLE D P h1 ((P n).retr z) := by
  simp only [projLE]
  rw [Nat.leRecOn_succ (C := fun k => D k → D m) h1]

theorem mono {m n : ℕ} (h : m ≤ n) {x y : D n} (hxy : x ≤ y) :
    projLE D P h x ≤ projLE D P h y := by
  induction n, h using Nat.le_induction with
  | base =>
      rw [self, self]
      exact hxy
  | succ k hk ih =>
      rw [succ D P hk (Nat.le_succ_of_le hk), succ D P hk (Nat.le_succ_of_le hk)]
      exact ih ((P k).retr.monotone hxy)

/-- For a compatible sequence `y`, descending from level `n` to `m ≤ n` recovers `y_m`.
    -/
theorem compatible {m : ℕ} (y : InverseLimit D P) :
    ∀ {n : ℕ} (h : m ≤ n), projLE D P h (y.1 n) = y.1 m := by
  intro n h
  induction n, h using Nat.le_induction with
  | base => rw [self]
  | succ k hk ih => rw [succ D P hk (Nat.le_succ_of_le hk), y.2 k, ih]

/-- Peeling the last projection: `(P m).retr ∘ projLE (m+1 ≤ n) = projLE (m ≤ n)`. -/
theorem retr {m : ℕ} : ∀ {n : ℕ} (h : m + 1 ≤ n) (x : D n),
    (P m).retr (projLE D P h x) = projLE D P (Nat.le_of_succ_le h) x := by
  intro n h
  induction n, h using Nat.le_induction with
  | base =>
      intro x
      rw [succ D P (le_refl m) (Nat.le_of_succ_le (le_refl (m + 1))) x]
      simp only [self]
  | succ k hk ih =>
      intro x
      rw [succ D P hk (Nat.le_succ_of_le hk) x, ih ((P k).retr x),
        succ D P (Nat.le_of_succ_le hk) (Nat.le_of_succ_le (Nat.le_succ_of_le hk)) x]

/-- The tower of projections is Scott-continuous (a composite of the `j_k`). -/
theorem preservesDirectedSup {m n : ℕ} (h : m ≤ n) :
    PreservesDirectedSup (projLE D P h) := by
  induction n, h using Nat.le_induction with
  | base =>
      have hid : projLE D P (le_refl m) = id := funext (fun x => self D P (le_refl m)
    x)
      rw [hid]; intro S _ _; simp
  | succ k hk ih =>
```

```
      have hfun : projLE D P (Nat.le_succ_of_le hk)
          = projLE D P hk ∘ (fun z : D (k + 1) => (P k).retr z) :=
        funext (fun x => succ D P hk (Nat.le_succ_of_le hk) x)
      rw [hfun]
      exact ScottMap.preservesDirectedSup_comp ih
        (fun S hS hd => (P k).retr.preservesDirectedSup_coe S hS hd)

end projLE

end InverseLimit

end DomainTheory.ContinuousLattice
```

5

projLE.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.comp
public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- Descend the tower of projections: for `m ≤ n`, `projLE h = j_m ∘ ... ∘ j_{n-1} :
    D_n → D_m`. -/
def projLE {m n : ℕ} (h : m ≤ n) (x : D n) : D m :=
  Nat.leRecOn (C := fun k => D k → D m) h
    (fun {k} (g : D k → D m) (w : D (k + 1)) => g ((P k).retr w)) id x

namespace projLE

theorem self {m : ℕ} (h : m ≤ m) (x : D m) : projLE D P h x = x := by
  simp only [projLE, Nat.leRecOn_self, id_eq]

theorem succ {m n : ℕ} (h1 : m ≤ n) (h2 : m ≤ n + 1) (z : D (n + 1)) :
    projLE D P h2 z = projLE D P h1 ((P n).retr z) := by
  simp only [projLE]
  rw [Nat.leRecOn_succ (C := fun k => D k → D m) h1]
```

```
theorem mono {m n : ℕ} (h : m ≤ n) {x y : D n} (hxy : x ≤ y) :
    projLE D P h x ≤ projLE D P h y := by
  induction n, h using Nat.le_induction with
  | base =>
      rw [self, self]
      exact hxy
  | succ k hk ih =>
      rw [succ D P hk (Nat.le_succ_of_le hk), succ D P hk (Nat.le_succ_of_le hk)]
      exact ih ((P k).retr.monotone hxy)

/-- For a compatible sequence `y`, descending from level `n` to `m ≤ n` recovers `yₘ`.
    -/
theorem compatible {m : ℕ} (y : InverseLimit D P) :
    ∀ {n : ℕ} (h : m ≤ n), projLE D P h (y.1 n) = y.1 m := by
  intro n h
  induction n, h using Nat.le_induction with
  | base => rw [self]
  | succ k hk ih => rw [succ D P hk (Nat.le_succ_of_le hk), y.2 k, ih]

/-- Peeling the last projection: `(P m).retr ∘ projLE (m+1 ≤ n) = projLE (m ≤ n)`. -/
theorem retr {m : ℕ} : ∀ {n : ℕ} (h : m + 1 ≤ n) (x : D n),
    (P m).retr (projLE D P h x) = projLE D P (Nat.le_of_succ_le h) x := by
  intro n h
  induction n, h using Nat.le_induction with
  | base =>
      intro x
      rw [succ D P (le_refl m) (Nat.le_of_succ_le (le_refl (m + 1))) x]
      simp only [self]
  | succ k hk ih =>
      intro x
      rw [succ D P hk (Nat.le_succ_of_le hk) x, ih ((P k).retr x),
        succ D P (Nat.le_of_succ_le hk) (Nat.le_of_succ_le (Nat.le_succ_of_le hk)) x]

/-- The tower of projections is Scott-continuous (a composite of the `jₖ`). -/
theorem preservesDirectedSup {m n : ℕ} (h : m ≤ n) :
    PreservesDirectedSup (projLE D P h) := by
  induction n, h using Nat.le_induction with
  | base =>
      have hid : projLE D P (le_refl m) = id := funext (fun x => self D P (le_refl m)
    x)
      rw [hid]; intro S _ _; simp
  | succ k hk ih =>
      have hfun : projLE D P (Nat.le_succ_of_le hk)
          = projLE D P hk ∘ (fun z : D (k + 1) => (P k).retr z) :=
        funext (fun x => succ D P hk (Nat.le_succ_of_le hk) x)
      rw [hfun]
      exact ScottMap.preservesDirectedSup_comp ih
        (fun S hS hd => (P k).retr.preservesDirectedSup_coe S hS hd)

end projLE

end InverseLimit

end DomainTheory.ContinuousLattice
```

6

projLE.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.comp
public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- Descend the tower of projections: for `m ≤ n`, `projLE h = j_m ∘ ... ∘ j_{n-1} :
    D_n → D_m`. -/
def projLE {m n : ℕ} (h : m ≤ n) (x : D n) : D m :=
  Nat.leRecOn (C := fun k => D k → D m) h
    (fun {k} (g : D k → D m) (w : D (k + 1)) => g ((P k).retr w)) id x

namespace projLE

theorem self {m : ℕ} (h : m ≤ m) (x : D m) : projLE D P h x = x := by
  simp only [projLE, Nat.leRecOn_self, id_eq]

theorem succ {m n : ℕ} (h1 : m ≤ n) (h2 : m ≤ n + 1) (z : D (n + 1)) :
    projLE D P h2 z = projLE D P h1 ((P n).retr z) := by
  simp only [projLE]
  rw [Nat.leRecOn_succ (C := fun k => D k → D m) h1]

theorem mono {m n : ℕ} (h : m ≤ n) {x y : D n} (hxy : x ≤ y) :
    projLE D P h x ≤ projLE D P h y := by
  induction n, h using Nat.le_induction with
  | base =>
      rw [self, self]
      exact hxy
  | succ k hk ih =>
      rw [succ D P hk (Nat.le_succ_of_le hk), succ D P hk (Nat.le_succ_of_le hk)]
      exact ih ((P k).retr.monotone hxy)

/-- For a compatible sequence `y`, descending from level `n` to `m ≤ n` recovers `y_m`.
    -/
```

```
theorem compatible {m : ℕ} (y : InverseLimit D P) :
    ∀ {n : ℕ} (h : m ≤ n), projLE D P h (y.1 n) = y.1 m := by
  intro n h
  induction n, h using Nat.le_induction with
  | base => rw [self]
  | succ k hk ih => rw [succ D P hk (Nat.le_succ_of_le hk), y.2 k, ih]

/-- Peeling the last projection: `(P m).retr ∘ projLE (m+1 ≤ n) = projLE (m ≤ n)`. -/
theorem retr {m : ℕ} : ∀ {n : ℕ} (h : m + 1 ≤ n) (x : D n),
    (P m).retr (projLE D P h x) = projLE D P (Nat.le_of_succ_le h) x := by
  intro n h
  induction n, h using Nat.le_induction with
  | base =>
      intro x
      rw [succ D P (le_refl m) (Nat.le_of_succ_le (le_refl (m + 1))) x]
      simp only [self]
  | succ k hk ih =>
      intro x
      rw [succ D P hk (Nat.le_succ_of_le hk) x, ih ((P k).retr x),
        succ D P (Nat.le_of_succ_le hk) (Nat.le_of_succ_le (Nat.le_succ_of_le hk)) x]

/-- The tower of projections is Scott-continuous (a composite of the `jₖ`). -/
theorem preservesDirectedSup {m n : ℕ} (h : m ≤ n) :
    PreservesDirectedSup (projLE D P h) := by
  induction n, h using Nat.le_induction with
  | base =>
      have hid : projLE D P (le_refl m) = id := funext (fun x => self D P (le_refl m)
    x)
      rw [hid]; intro S _ _; simp
  | succ k hk ih =>
      have hfun : projLE D P (Nat.le_succ_of_le hk)
          = projLE D P hk ∘ (fun z : D (k + 1) => (P k).retr z) :=
        funext (fun x => succ D P hk (Nat.le_succ_of_le hk) x)
      rw [hfun]
      exact ScottMap.preservesDirectedSup_comp ih
        (fun S hS hd => (P k).retr.preservesDirectedSup_coe S hS hd)

end projLE

end InverseLimit

end DomainTheory.ContinuousLattice
```

7

projLE.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.comp
public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import Mathlib.Data.Set.Basic
```

```
@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- Descend the tower of projections: for `m ≤ n`, `projLE h = j_m ∘ ... ∘ j_{n-1} :
    D_n → D_m`. -/
def projLE {m n : ℕ} (h : m ≤ n) (x : D n) : D m :=
  Nat.leRecOn (C := fun k => D k → D m) h
    (fun {k} (g : D k → D m) (w : D (k + 1)) => g ((P k).retr w)) id x

namespace projLE

theorem self {m : ℕ} (h : m ≤ m) (x : D m) : projLE D P h x = x := by
  simp only [projLE, Nat.leRecOn_self, id_eq]

theorem succ {m n : ℕ} (h1 : m ≤ n) (h2 : m ≤ n + 1) (z : D (n + 1)) :
    projLE D P h2 z = projLE D P h1 ((P n).retr z) := by
  simp only [projLE]
  rw [Nat.leRecOn_succ (C := fun k => D k → D m) h1]

theorem mono {m n : ℕ} (h : m ≤ n) {x y : D n} (hxy : x ≤ y) :
    projLE D P h x ≤ projLE D P h y := by
  induction n, h using Nat.le_induction with
  | base =>
      rw [self, self]
      exact hxy
  | succ k hk ih =>
      rw [succ D P hk (Nat.le_succ_of_le hk), succ D P hk (Nat.le_succ_of_le hk)]
      exact ih ((P k).retr.monotone hxy)

/-- For a compatible sequence `y`, descending from level `n` to `m ≤ n` recovers `y_m`.
    -/
theorem compatible {m : ℕ} (y : InverseLimit D P) :
    ∀ {n : ℕ} (h : m ≤ n), projLE D P h (y.1 n) = y.1 m := by
  intro n h
  induction n, h using Nat.le_induction with
  | base => rw [self]
  | succ k hk ih => rw [succ D P hk (Nat.le_succ_of_le hk), y.2 k, ih]

/-- Peeling the last projection: `(P m).retr ∘ projLE (m+1 ≤ n) = projLE (m ≤ n)`. -/
theorem retr {m : ℕ} : ∀ {n : ℕ} (h : m + 1 ≤ n) (x : D n),
    (P m).retr (projLE D P h x) = projLE D P (Nat.le_of_succ_le h) x := by
  intro n h
  induction n, h using Nat.le_induction with
  | base =>
```

```
      intro x
      rw [succ D P (le_refl m) (Nat.le_of_succ_le (le_refl (m + 1))) x]
      simp only [self]
  | succ k hk ih =>
      intro x
      rw [succ D P hk (Nat.le_succ_of_le hk) x, ih ((P k).retr x),
        succ D P (Nat.le_of_succ_le hk) (Nat.le_of_succ_le (Nat.le_succ_of_le hk)) x]

/-- The tower of projections is Scott-continuous (a composite of the `jₖ`). -/
theorem preservesDirectedSup {m n : ℕ} (h : m ≤ n) :
    PreservesDirectedSup (projLE D P h) := by
  induction n, h using Nat.le_induction with
  | base =>
      have hid : projLE D P (le_refl m) = id := funext (fun x => self D P (le_refl m)
    x)
      rw [hid]; intro S _ _; simp
  | succ k hk ih =>
      have hfun : projLE D P (Nat.le_succ_of_le hk)
          = projLE D P hk ∘ (fun z : D (k + 1) => (P k).retr z) :=
        funext (fun x => succ D P hk (Nat.le_succ_of_le hk) x)
      rw [hfun]
      exact ScottMap.preservesDirectedSup_comp ih
        (fun S hS hd => (P k).retr.preservesDirectedSup_coe S hS hd)

end projLE

end InverseLimit

end DomainTheory.ContinuousLattice
```

8

projLE.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.comp
public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]
```

```
variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- Descend the tower of projections: for `m ≤ n`, `projLE h = j_m ∘ ... ∘ j_{n-1} :
    D_n → D_m`. -/
def projLE {m n : ℕ} (h : m ≤ n) (x : D n) : D m :=
  Nat.leRecOn (C := fun k => D k → D m) h
    (fun {k} (g : D k → D m) (w : D (k + 1)) => g ((P k).retr w)) id x

namespace projLE

theorem self {m : ℕ} (h : m ≤ m) (x : D m) : projLE D P h x = x := by
  simp only [projLE, Nat.leRecOn_self, id_eq]

theorem succ {m n : ℕ} (h1 : m ≤ n) (h2 : m ≤ n + 1) (z : D (n + 1)) :
    projLE D P h2 z = projLE D P h1 ((P n).retr z) := by
  simp only [projLE]
  rw [Nat.leRecOn_succ (C := fun k => D k → D m) h1]

theorem mono {m n : ℕ} (h : m ≤ n) {x y : D n} (hxy : x ≤ y) :
    projLE D P h x ≤ projLE D P h y := by
  induction n, h using Nat.le_induction with
  | base =>
      rw [self, self]
      exact hxy
  | succ k hk ih =>
      rw [succ D P hk (Nat.le_succ_of_le hk), succ D P hk (Nat.le_succ_of_le hk)]
      exact ih ((P k).retr.monotone hxy)

/-- For a compatible sequence `y`, descending from level `n` to `m ≤ n` recovers `y_m`.
    -/
theorem compatible {m : ℕ} (y : InverseLimit D P) :
    ∀ {n : ℕ} (h : m ≤ n), projLE D P h (y.1 n) = y.1 m := by
  intro n h
  induction n, h using Nat.le_induction with
  | base => rw [self]
  | succ k hk ih => rw [succ D P hk (Nat.le_succ_of_le hk), y.2 k, ih]

/-- Peeling the last projection: `(P m).retr ∘ projLE (m+1 ≤ n) = projLE (m ≤ n)`. -/
theorem retr {m : ℕ} : ∀ {n : ℕ} (h : m + 1 ≤ n) (x : D n),
    (P m).retr (projLE D P h x) = projLE D P (Nat.le_of_succ_le h) x := by
  intro n h
  induction n, h using Nat.le_induction with
  | base =>
      intro x
      rw [succ D P (le_refl m) (Nat.le_of_succ_le (le_refl (m + 1))) x]
      simp only [self]
  | succ k hk ih =>
      intro x
      rw [succ D P hk (Nat.le_succ_of_le hk) x, ih ((P k).retr x),
        succ D P (Nat.le_of_succ_le hk) (Nat.le_of_succ_le (Nat.le_succ_of_le hk)) x]

/-- The tower of projections is Scott-continuous (a composite of the `j_k`). -/
theorem preservesDirectedSup {m n : ℕ} (h : m ≤ n) :
    PreservesDirectedSup (projLE D P h) := by
  induction n, h using Nat.le_induction with
  | base =>
```

```
      have hid : projLE D P (le_refl m) = id := funext (fun x => self D P (le_refl m)
    x)
      rw [hid]; intro S _ _; simp
  | succ k hk ih =>
      have hfun : projLE D P (Nat.le_succ_of_le hk)
          = projLE D P hk ∘ (fun z : D (k + 1) => (P k).retr z) :=
        funext (fun x => succ D P hk (Nat.le_succ_of_le hk) x)
      rw [hfun]
      exact ScottMap.preservesDirectedSup_comp ih
        (fun S hS hd => (P k).retr.preservesDirectedSup_coe S hS hd)

end projLE

end InverseLimit

end DomainTheory.ContinuousLattice
```

9

projection_galoisConnection.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- The embedding-projection pair of a projection is a Galois connection `iₙ ⊣ jₙ`:
    from
`jₙ ∘ iₙ = id` and `iₙ ∘ jₙ ⊑ id` we get `iₙ x ⊑ y ↔ x ⊑ jₙ y`. In particular `jₙ` (the
    upper
adjoint) preserves arbitrary infima. -/
theorem projection_galoisConnection (n : ℕ) :
    GaloisConnection ((P n).incl : D n → D (n + 1)) ((P n).retr : D (n + 1) → D n) :=
     by
  intro x y
  constructor
  · intro h
    have h' := (P n).retr.monotone h
    rwa [(P n).retr_incl] at h'
  · intro h
    exact le_trans ((P n).incl.monotone h) ((P n).incl_retr_le y)
```

```
end InverseLimit

end DomainTheory.ContinuousLattice
```

0

projection_galoisConnection.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- The embedding-projection pair of a projection is a Galois connection `iₙ ⊣ jₙ`:
    from
`jₙ ∘ iₙ = id` and `iₙ ∘ jₙ ⊑ id` we get `iₙ x ⊑ y ↔ x ⊑ jₙ y`. In particular `jₙ` (the
    upper
adjoint) preserves arbitrary infima. -/
theorem projection_galoisConnection (n : ℕ) :
    GaloisConnection ((P n).incl : D n → D (n + 1)) ((P n).retr : D (n + 1) → D n) :=
     by
  intro x y
  constructor
  · intro h
    have h' := (P n).retr.monotone h
    rwa [(P n).retr_incl] at h'
  · intro h
    exact le_trans ((P n).incl.monotone h) ((P n).incl_retr_le y)

end InverseLimit

end DomainTheory.ContinuousLattice
```

1

projection_galoisConnection.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- The embedding-projection pair of a projection is a Galois connection `iₙ ⊣ jₙ`:
    from
`jₙ ∘ iₙ = id` and `iₙ ∘ jₙ ⊑ id` we get `iₙ x ⊑ y ↔ x ⊑ jₙ y`. In particular `jₙ` (the
    upper
adjoint) preserves arbitrary infima. -/
theorem projection_galoisConnection (n : ℕ) :
    GaloisConnection ((P n).incl : D n → D (n + 1)) ((P n).retr : D (n + 1) → D n) :=
     by
  intro x y
  constructor
  · intro h
    have h' := (P n).retr.monotone h
    rwa [(P n).retr_incl] at h'
  · intro h
    exact le_trans ((P n).incl.monotone h) ((P n).incl_retr_le y)

end InverseLimit

end DomainTheory.ContinuousLattice
```

2

projection_galoisConnection.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit
```

```
variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- The embedding-projection pair of a projection is a Galois connection `iₙ ⊣ jₙ`:
    from
`jₙ ∘ iₙ = id` and `iₙ ∘ jₙ ⊑ id` we get `iₙ x ⊑ y ↔ x ⊑ jₙ y`. In particular `jₙ` (the
    upper
adjoint) preserves arbitrary infima. -/
theorem projection_galoisConnection (n : ℕ) :
    GaloisConnection ((P n).incl : D n → D (n + 1)) ((P n).retr : D (n + 1) → D n) :=
     by
  intro x y
  constructor
  · intro h
    have h' := (P n).retr.monotone h
    rwa [(P n).retr_incl] at h'
  · intro h
    exact le_trans ((P n).incl.monotone h) ((P n).incl_retr_le y)

end InverseLimit

end DomainTheory.ContinuousLattice
```

3

projection_galoisConnection.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- The embedding-projection pair of a projection is a Galois connection `iₙ ⊣ jₙ`:
    from
`jₙ ∘ iₙ = id` and `iₙ ∘ jₙ ⊑ id` we get `iₙ x ⊑ y ↔ x ⊑ jₙ y`. In particular `jₙ` (the
    upper
adjoint) preserves arbitrary infima. -/
theorem projection_galoisConnection (n : ℕ) :
    GaloisConnection ((P n).incl : D n → D (n + 1)) ((P n).retr : D (n + 1) → D n) :=
```

```
    by
  intro x y
  constructor
  · intro h
    have h' := (P n).retr.monotone h
    rwa [(P n).retr_incl] at h'
  · intro h
    exact le_trans ((P n).incl.monotone h) ((P n).incl_retr_le y)

end InverseLimit

end DomainTheory.ContinuousLattice
```

4

projection_galoisConnection.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- The embedding-projection pair of a projection is a Galois connection `iₙ ⊣ jₙ`:
    from
`jₙ ∘ iₙ = id` and `iₙ ∘ jₙ ⊑ id` we get `iₙ x ⊑ y ↔ x ⊑ jₙ y`. In particular `jₙ` (the
    upper
adjoint) preserves arbitrary infima. -/
theorem projection_galoisConnection (n : ℕ) :
    GaloisConnection ((P n).incl : D n → D (n + 1)) ((P n).retr : D (n + 1) → D n) :=
     by
  intro x y
  constructor
  · intro h
    have h' := (P n).retr.monotone h
    rwa [(P n).retr_incl] at h'
  · intro h
    exact le_trans ((P n).incl.monotone h) ((P n).incl_retr_le y)

end InverseLimit

end DomainTheory.ContinuousLattice
```

5

projection_galoisConnection.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- The embedding-projection pair of a projection is a Galois connection `iₙ ⊣ jₙ`:
    from
`jₙ ∘ iₙ = id` and `iₙ ∘ jₙ ⊑ id` we get `iₙ x ⊑ y ↔ x ⊑ jₙ y`. In particular `jₙ` (the
    upper
adjoint) preserves arbitrary infima. -/
theorem projection_galoisConnection (n : ℕ) :
    GaloisConnection ((P n).incl : D n → D (n + 1)) ((P n).retr : D (n + 1) → D n) :=
     by
  intro x y
  constructor
  · intro h
    have h' := (P n).retr.monotone h
    rwa [(P n).retr_incl] at h'
  · intro h
    exact le_trans ((P n).incl.monotone h) ((P n).incl_retr_le y)

end InverseLimit

end DomainTheory.ContinuousLattice
```

6

projection_galoisConnection.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Data.Set.Basic

@[expose] public section
```

```
namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- The embedding-projection pair of a projection is a Galois connection `iₙ ⊣ jₙ`:
    from
`jₙ ∘ iₙ = id` and `iₙ ∘ jₙ ⊑ id` we get `iₙ x ⊑ y ↔ x ⊑ jₙ y`. In particular `jₙ` (the
    upper
adjoint) preserves arbitrary infima. -/
theorem projection_galoisConnection (n : ℕ) :
    GaloisConnection ((P n).incl : D n → D (n + 1)) ((P n).retr : D (n + 1) → D n) :=
     by
  intro x y
  constructor
  · intro h
    have h' := (P n).retr.monotone h
    rwa [(P n).retr_incl] at h'
  · intro h
    exact le_trans ((P n).incl.monotone h) ((P n).incl_retr_le y)

end InverseLimit

end DomainTheory.ContinuousLattice
```

7

projection_galoisConnection.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))
```

```
/-- The embedding-projection pair of a projection is a Galois connection `iₙ ⊣ jₙ`:
    from
`jₙ ∘ iₙ = id` and `iₙ ∘ jₙ ⊑ id` we get `iₙ x ⊑ y ↔ x ⊑ jₙ y`. In particular `jₙ` (the
    upper
adjoint) preserves arbitrary infima. -/
theorem projection_galoisConnection (n : ℕ) :
    GaloisConnection ((P n).incl : D n → D (n + 1)) ((P n).retr : D (n + 1) → D n) :=
     by
  intro x y
  constructor
  · intro h
    have h' := (P n).retr.monotone h
    rwa [(P n).retr_incl] at h'
  · intro h
    exact le_trans ((P n).incl.monotone h) ((P n).incl_retr_le y)

end InverseLimit

end DomainTheory.ContinuousLattice
```


projection_galoisConnection.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- The embedding-projection pair of a projection is a Galois connection `iₙ ⊣ jₙ`:
    from
`jₙ ∘ iₙ = id` and `iₙ ∘ jₙ ⊑ id` we get `iₙ x ⊑ y ↔ x ⊑ jₙ y`. In particular `jₙ` (the
    upper
adjoint) preserves arbitrary infima. -/
theorem projection_galoisConnection (n : ℕ) :
    GaloisConnection ((P n).incl : D n → D (n + 1)) ((P n).retr : D (n + 1) → D n) :=
     by
  intro x y
  constructor
  · intro h
    have h' := (P n).retr.monotone h
```

```
    rwa [(P n).retr_incl] at h'
  · intro h
    exact le_trans ((P n).incl.monotone h) ((P n).incl_retr_le y)

end InverseLimit

end DomainTheory.ContinuousLattice
```

9

projInf.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.instCompleteLattice
public import DomainTheory.ContinuousLattice.InverseLimits.coe_sSup_of_directed
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- The projection `j_{∞n} : D_∞ → Dₙ` as a bare function. -/
def projInfFun (n : ℕ) (y : InverseLimit D P) : D n := y.1 n

namespace projInfFun

theorem preservesDirectedSup (n : ℕ) : PreservesDirectedSup (projInfFun D P n) := by
  intro S hS hSdir
  have hL : (sSup S : InverseLimit D P).1 n = sSup ((fun y : InverseLimit D P => y.1 n
    ) '' S) := by
    rw [coe_sSup_of_directed D P S hS hSdir, sSup_apply_eq_sSup_image]
    congr 1
    rw [Set.image_image]
  exact hL

end projInfFun

/-- The projection `j_{∞n} : D_∞ → Dₙ`, Scott-continuous. -/
noncomputable def projInf (n : ℕ) : ScottMap (InverseLimit D P) (D n) :=
  ⟨projInfFun D P n, continuous_of_preservesDirectedSup (projInfFun.
    preservesDirectedSup D P n)⟩
```

```
end InverseLimit

end DomainTheory.ContinuousLattice
```

0

projInf.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.instCompleteLattice
public import DomainTheory.ContinuousLattice.InverseLimits.coe_sSup_of_directed
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- The projection `j_{∞n} : D_∞ → Dₙ` as a bare function. -/
def projInfFun (n : ℕ) (y : InverseLimit D P) : D n := y.1 n

namespace projInfFun

theorem preservesDirectedSup (n : ℕ) : PreservesDirectedSup (projInfFun D P n) := by
  intro S hS hSdir
  have hL : (sSup S : InverseLimit D P).1 n = sSup ((fun y : InverseLimit D P => y.1 n
    ) '' S) := by
    rw [coe_sSup_of_directed D P S hS hSdir, sSup_apply_eq_sSup_image]
    congr 1
    rw [Set.image_image]
  exact hL

end projInfFun

/-- The projection `j_{∞n} : D_∞ → Dₙ`, Scott-continuous. -/
noncomputable def projInf (n : ℕ) : ScottMap (InverseLimit D P) (D n) :=
  ⟨projInfFun D P n, continuous_of_preservesDirectedSup (projInfFun.
    preservesDirectedSup D P n)⟩

end InverseLimit

end DomainTheory.ContinuousLattice
```

1

projInf.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.instCompleteLattice
public import DomainTheory.ContinuousLattice.InverseLimits.coe_sSup_of_directed
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- The projection `j_{∞n} : D_∞ → Dₙ` as a bare function. -/
def projInfFun (n : ℕ) (y : InverseLimit D P) : D n := y.1 n

namespace projInfFun

theorem preservesDirectedSup (n : ℕ) : PreservesDirectedSup (projInfFun D P n) := by
  intro S hS hSdir
  have hL : (sSup S : InverseLimit D P).1 n = sSup ((fun y : InverseLimit D P => y.1 n
    ) '' S) := by
    rw [coe_sSup_of_directed D P S hS hSdir, sSup_apply_eq_sSup_image]
    congr 1
    rw [Set.image_image]
  exact hL

end projInfFun

/-- The projection `j_{∞n} : D_∞ → Dₙ`, Scott-continuous. -/
noncomputable def projInf (n : ℕ) : ScottMap (InverseLimit D P) (D n) :=
  ⟨projInfFun D P n, continuous_of_preservesDirectedSup (projInfFun.
    preservesDirectedSup D P n)⟩

end InverseLimit

end DomainTheory.ContinuousLattice
```

2

projInf.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.instCompleteLattice
public import DomainTheory.ContinuousLattice.InverseLimits.coe_sSup_of_directed
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- The projection `j_{∞n} : D_∞ → Dₙ` as a bare function. -/
def projInfFun (n : ℕ) (y : InverseLimit D P) : D n := y.1 n

namespace projInfFun

theorem preservesDirectedSup (n : ℕ) : PreservesDirectedSup (projInfFun D P n) := by
  intro S hS hSdir
  have hL : (sSup S : InverseLimit D P).1 n = sSup ((fun y : InverseLimit D P => y.1 n
    ) '' S) := by
    rw [coe_sSup_of_directed D P S hS hSdir, sSup_apply_eq_sSup_image]
    congr 1
    rw [Set.image_image]
  exact hL

end projInfFun

/-- The projection `j_{∞n} : D_∞ → Dₙ`, Scott-continuous. -/
noncomputable def projInf (n : ℕ) : ScottMap (InverseLimit D P) (D n) :=
  ⟨projInfFun D P n, continuous_of_preservesDirectedSup (projInfFun.
    preservesDirectedSup D P n)⟩

end InverseLimit

end DomainTheory.ContinuousLattice
```

3

projInf.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.instCompleteLattice
public import DomainTheory.ContinuousLattice.InverseLimits.coe_sSup_of_directed
```

```
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- The projection `j_{∞n} : D_∞ → D_n` as a bare function. -/
def projInfFun (n : ℕ) (y : InverseLimit D P) : D n := y.1 n

namespace projInfFun

theorem preservesDirectedSup (n : ℕ) : PreservesDirectedSup (projInfFun D P n) := by
  intro S hS hSdir
  have hL : (sSup S : InverseLimit D P).1 n = sSup ((fun y : InverseLimit D P => y.1 n
    ) '' S) := by
    rw [coe_sSup_of_directed D P S hS hSdir, sSup_apply_eq_sSup_image]
    congr 1
    rw [Set.image_image]
  exact hL

end projInfFun

/-- The projection `j_{∞n} : D_∞ → D_n`, Scott-continuous. -/
noncomputable def projInf (n : ℕ) : ScottMap (InverseLimit D P) (D n) :=
  ⟨projInfFun D P n, continuous_of_preservesDirectedSup (projInfFun.
    preservesDirectedSup D P n)⟩

end InverseLimit

end DomainTheory.ContinuousLattice
```

4

projInf.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.instCompleteLattice
public import DomainTheory.ContinuousLattice.InverseLimits.coe_sSup_of_directed
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Data.Set.Basic
```

```
@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- The projection `j_{∞n} : D_∞ → Dₙ` as a bare function. -/
def projInfFun (n : ℕ) (y : InverseLimit D P) : D n := y.1 n

namespace projInfFun

theorem preservesDirectedSup (n : ℕ) : PreservesDirectedSup (projInfFun D P n) := by
  intro S hS hSdir
  have hL : (sSup S : InverseLimit D P).1 n = sSup ((fun y : InverseLimit D P => y.1 n
    ) '' S) := by
    rw [coe_sSup_of_directed D P S hS hSdir, sSup_apply_eq_sSup_image]
    congr 1
    rw [Set.image_image]
  exact hL

end projInfFun

/-- The projection `j_{∞n} : D_∞ → Dₙ`, Scott-continuous. -/
noncomputable def projInf (n : ℕ) : ScottMap (InverseLimit D P) (D n) :=
  ⟨projInfFun D P n, continuous_of_preservesDirectedSup (projInfFun.
    preservesDirectedSup D P n)⟩

end InverseLimit

end DomainTheory.ContinuousLattice
```

5

projInf.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.instCompleteLattice
public import DomainTheory.ContinuousLattice.InverseLimits.coe_sSup_of_directed
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice
```

```
open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- The projection `j_{∞n} : D_∞ → Dₙ` as a bare function. -/
def projInfFun (n : ℕ) (y : InverseLimit D P) : D n := y.1 n

namespace projInfFun

theorem preservesDirectedSup (n : ℕ) : PreservesDirectedSup (projInfFun D P n) := by
  intro S hS hSdir
  have hL : (sSup S : InverseLimit D P).1 n = sSup ((fun y : InverseLimit D P => y.1 n
    ) '' S) := by
    rw [coe_sSup_of_directed D P S hS hSdir, sSup_apply_eq_sSup_image]
    congr 1
    rw [Set.image_image]
  exact hL

end projInfFun

/-- The projection `j_{∞n} : D_∞ → Dₙ`, Scott-continuous. -/
noncomputable def projInf (n : ℕ) : ScottMap (InverseLimit D P) (D n) :=
  ⟨projInfFun D P n, continuous_of_preservesDirectedSup (projInfFun.
    preservesDirectedSup D P n)⟩

end InverseLimit

end DomainTheory.ContinuousLattice
```

6

projInf.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.instCompleteLattice
public import DomainTheory.ContinuousLattice.InverseLimits.coe_sSup_of_directed
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u
```

```
section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- The projection `j_{∞n} : D_∞ → Dₙ` as a bare function. -/
def projInfFun (n : ℕ) (y : InverseLimit D P) : D n := y.1 n

namespace projInfFun

theorem preservesDirectedSup (n : ℕ) : PreservesDirectedSup (projInfFun D P n) := by
  intro S hS hSdir
  have hL : (sSup S : InverseLimit D P).1 n = sSup ((fun y : InverseLimit D P => y.1 n
    ) '' S) := by
    rw [coe_sSup_of_directed D P S hS hSdir, sSup_apply_eq_sSup_image]
    congr 1
    rw [Set.image_image]
  exact hL

end projInfFun

/-- The projection `j_{∞n} : D_∞ → Dₙ`, Scott-continuous. -/
noncomputable def projInf (n : ℕ) : ScottMap (InverseLimit D P) (D n) :=
  ⟨projInfFun D P n, continuous_of_preservesDirectedSup (projInfFun.
    preservesDirectedSup D P n)⟩

end InverseLimit

end DomainTheory.ContinuousLattice
```

7

projInf.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.instCompleteLattice
public import DomainTheory.ContinuousLattice.InverseLimits.coe_sSup_of_directed
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]
```

```
variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- The projection `j_{∞n} : D_∞ → Dₙ` as a bare function. -/
def projInfFun (n : ℕ) (y : InverseLimit D P) : D n := y.1 n

namespace projInfFun

theorem preservesDirectedSup (n : ℕ) : PreservesDirectedSup (projInfFun D P n) := by
  intro S hS hSdir
  have hL : (sSup S : InverseLimit D P).1 n = sSup ((fun y : InverseLimit D P => y.1 n
    ) '' S) := by
    rw [coe_sSup_of_directed D P S hS hSdir, sSup_apply_eq_sSup_image]
    congr 1
    rw [Set.image_image]
  exact hL

end projInfFun

/-- The projection `j_{∞n} : D_∞ → Dₙ`, Scott-continuous. -/
noncomputable def projInf (n : ℕ) : ScottMap (InverseLimit D P) (D n) :=
  ⟨projInfFun D P n, continuous_of_preservesDirectedSup (projInfFun.
    preservesDirectedSup D P n)⟩

end InverseLimit

end DomainTheory.ContinuousLattice
```

8

projInf.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.instCompleteLattice
public import DomainTheory.ContinuousLattice.InverseLimits.coe_sSup_of_directed
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- The projection `j_{∞n} : D_∞ → Dₙ` as a bare function. -/
```

```
def projInfFun (n : ℕ) (y : InverseLimit D P) : D n := y.1 n

namespace projInfFun

theorem preservesDirectedSup (n : ℕ) : PreservesDirectedSup (projInfFun D P n) := by
  intro S hS hSdir
  have hL : (sSup S : InverseLimit D P).1 n = sSup ((fun y : InverseLimit D P => y.1 n
    ) '' S) := by
    rw [coe_sSup_of_directed D P S hS hSdir, sSup_apply_eq_sSup_image]
    congr 1
    rw [Set.image_image]
  exact hL

end projInfFun

/-- The projection `j_{∞n} : D_∞ → Dₙ`, Scott-continuous. -/
noncomputable def projInf (n : ℕ) : ScottMap (InverseLimit D P) (D n) :=
  ⟨projInfFun D P n, continuous_of_preservesDirectedSup (projInfFun.
    preservesDirectedSup D P n)⟩

end InverseLimit

end DomainTheory.ContinuousLattice
```

9

embLE.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.comp
public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- Climb the tower of embeddings: for `n ≤ m`, `embLE h = i_{m-1} ∘ ... ∘ iₙ : Dₙ →
    D_m`. -/
def embLE {n m : ℕ} (h : n ≤ m) (x : D n) : D m :=
```

```
  Nat.leRecOn h (fun {k} (y : D k) => (P k).incl y) x

theorem embLE_self {n : ℕ} (h : n ≤ n) (x : D n) : embLE D P h x = x := by
  simp only [embLE, Nat.leRecOn_self]

theorem embLE_succ {n m : ℕ} (h1 : n ≤ m) (h2 : n ≤ m + 1) (x : D n) :
    embLE D P h2 x = (P m).incl (embLE D P h1 x) := by
  simp only [embLE]
  rw [Nat.leRecOn_succ h1]

theorem embLE_succ_left {n m : ℕ} (h1 : n ≤ m) (h2 : n + 1 ≤ m) (x : D n) :
    embLE D P h2 ((P n).incl x) = embLE D P h1 x := by
  simp only [embLE]
  exact Nat.leRecOn_succ_left x h1 h2

/-- For a compatible sequence `y`, climbing `yₙ` up to level `m ≥ n` stays below `yₘ`.
    -/
theorem embLE_le {n : ℕ} (y : InverseLimit D P) {m : ℕ} (h : n ≤ m) :
    embLE D P h (y.1 n) ≤ y.1 m := by
  induction m, h using Nat.le_induction with
  | base => exact le_of_eq (embLE_self D P _ _)
  | succ k hk ih =>
      rw [embLE_succ D P hk (Nat.le_succ_of_le hk)]
      calc (P k).incl (embLE D P hk (y.1 n))
          ≤ (P k).incl (y.1 k) := (P k).incl.monotone ih
        _ = (P k).incl ((P k).retr (y.1 (k + 1))) := by rw [y.2 k]
        _ ≤ y.1 (k + 1) := (P k).incl_retr_le _

theorem embLE_mono {n m : ℕ} (h : n ≤ m) {x y : D n} (hxy : x ≤ y) :
    embLE D P h x ≤ embLE D P h y := by
  induction m, h using Nat.le_induction with
  | base =>
      rw [embLE_self, embLE_self]
      exact hxy
  | succ k hk ih =>
      rw [embLE_succ D P hk (Nat.le_succ_of_le hk), embLE_succ D P hk (Nat.
    le_succ_of_le hk)]
      exact (P k).incl.monotone ih

/-- The tower of embeddings is Scott-continuous (a composite of the `iₖ`). -/
theorem embLE_preservesDirectedSup {n m : ℕ} (h : n ≤ m) :
    PreservesDirectedSup (embLE D P h) := by
  induction m, h using Nat.le_induction with
  | base =>
      have hid : embLE D P (le_refl n) = id := funext (fun x => embLE_self D P (
    le_refl n) x)
      rw [hid]; intro S _ _; simp
  | succ k hk ih =>
      have hfun : embLE D P (Nat.le_succ_of_le hk)
          = (fun y : D k => (P k).incl y) ∘ embLE D P hk :=
        funext (fun x => embLE_succ D P hk (Nat.le_succ_of_le hk) x)
      rw [hfun]
      exact ScottMap.preservesDirectedSup_comp
        (fun S hS hd => (P k).incl.preservesDirectedSup_coe S hS hd) ih

end InverseLimit
```

```
end DomainTheory.ContinuousLattice
```

0

embLE.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.comp
public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- Climb the tower of embeddings: for `n ≤ m`, `embLE h = i_{m-1} ∘ ... ∘ iₙ : Dₙ →
    D_m`. -/
def embLE {n m : ℕ} (h : n ≤ m) (x : D n) : D m :=
  Nat.leRecOn h (fun {k} (y : D k) => (P k).incl y) x

theorem embLE_self {n : ℕ} (h : n ≤ n) (x : D n) : embLE D P h x = x := by
  simp only [embLE, Nat.leRecOn_self]

theorem embLE_succ {n m : ℕ} (h1 : n ≤ m) (h2 : n ≤ m + 1) (x : D n) :
    embLE D P h2 x = (P m).incl (embLE D P h1 x) := by
  simp only [embLE]
  rw [Nat.leRecOn_succ h1]

theorem embLE_succ_left {n m : ℕ} (h1 : n ≤ m) (h2 : n + 1 ≤ m) (x : D n) :
    embLE D P h2 ((P n).incl x) = embLE D P h1 x := by
  simp only [embLE]
  exact Nat.leRecOn_succ_left x h1 h2

/-- For a compatible sequence `y`, climbing `yₙ` up to level `m ≥ n` stays below `yₘ`.
    -/
theorem embLE_le {n : ℕ} (y : InverseLimit D P) {m : ℕ} (h : n ≤ m) :
    embLE D P h (y.1 n) ≤ y.1 m := by
  induction m, h using Nat.le_induction with
  | base => exact le_of_eq (embLE_self D P _ _)
```

```
  | succ k hk ih =>
      rw [embLE_succ D P hk (Nat.le_succ_of_le hk)]
      calc (P k).incl (embLE D P hk (y.1 n))
          ≤ (P k).incl (y.1 k) := (P k).incl.monotone ih
        _ = (P k).incl ((P k).retr (y.1 (k + 1))) := by rw [y.2 k]
        _ ≤ y.1 (k + 1) := (P k).incl_retr_le _

theorem embLE_mono {n m : ℕ} (h : n ≤ m) {x y : D n} (hxy : x ≤ y) :
    embLE D P h x ≤ embLE D P h y := by
  induction m, h using Nat.le_induction with
  | base =>
      rw [embLE_self, embLE_self]
      exact hxy
  | succ k hk ih =>
      rw [embLE_succ D P hk (Nat.le_succ_of_le hk), embLE_succ D P hk (Nat.
    le_succ_of_le hk)]
      exact (P k).incl.monotone ih

/-- The tower of embeddings is Scott-continuous (a composite of the `iₖ`). -/
theorem embLE_preservesDirectedSup {n m : ℕ} (h : n ≤ m) :
    PreservesDirectedSup (embLE D P h) := by
  induction m, h using Nat.le_induction with
  | base =>
      have hid : embLE D P (le_refl n) = id := funext (fun x => embLE_self D P (
    le_refl n) x)
      rw [hid]; intro S _ _; simp
  | succ k hk ih =>
      have hfun : embLE D P (Nat.le_succ_of_le hk)
          = (fun y : D k => (P k).incl y) ∘ embLE D P hk :=
        funext (fun x => embLE_succ D P hk (Nat.le_succ_of_le hk) x)
      rw [hfun]
      exact ScottMap.preservesDirectedSup_comp
        (fun S hS hd => (P k).incl.preservesDirectedSup_coe S hS hd) ih

end InverseLimit

end DomainTheory.ContinuousLattice
```

1

embLE.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.comp
public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice
```

```
open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- Climb the tower of embeddings: for `n ≤ m`, `embLE h = i_{m-1} ∘ ... ∘ iₙ : Dₙ →
    D_m`. -/
def embLE {n m : ℕ} (h : n ≤ m) (x : D n) : D m :=
  Nat.leRecOn h (fun {k} (y : D k) => (P k).incl y) x

theorem embLE_self {n : ℕ} (h : n ≤ n) (x : D n) : embLE D P h x = x := by
  simp only [embLE, Nat.leRecOn_self]

theorem embLE_succ {n m : ℕ} (h1 : n ≤ m) (h2 : n ≤ m + 1) (x : D n) :
    embLE D P h2 x = (P m).incl (embLE D P h1 x) := by
  simp only [embLE]
  rw [Nat.leRecOn_succ h1]

theorem embLE_succ_left {n m : ℕ} (h1 : n ≤ m) (h2 : n + 1 ≤ m) (x : D n) :
    embLE D P h2 ((P n).incl x) = embLE D P h1 x := by
  simp only [embLE]
  exact Nat.leRecOn_succ_left x h1 h2

/-- For a compatible sequence `y`, climbing `yₙ` up to level `m ≥ n` stays below `y_m`.
    -/
theorem embLE_le {n : ℕ} (y : InverseLimit D P) {m : ℕ} (h : n ≤ m) :
    embLE D P h (y.1 n) ≤ y.1 m := by
  induction m, h using Nat.le_induction with
  | base => exact le_of_eq (embLE_self D P _ _)
  | succ k hk ih =>
      rw [embLE_succ D P hk (Nat.le_succ_of_le hk)]
      calc (P k).incl (embLE D P hk (y.1 n))
          ≤ (P k).incl (y.1 k) := (P k).incl.monotone ih
        _ = (P k).incl ((P k).retr (y.1 (k + 1))) := by rw [y.2 k]
        _ ≤ y.1 (k + 1) := (P k).incl_retr_le _

theorem embLE_mono {n m : ℕ} (h : n ≤ m) {x y : D n} (hxy : x ≤ y) :
    embLE D P h x ≤ embLE D P h y := by
  induction m, h using Nat.le_induction with
  | base =>
      rw [embLE_self, embLE_self]
      exact hxy
  | succ k hk ih =>
      rw [embLE_succ D P hk (Nat.le_succ_of_le hk), embLE_succ D P hk (Nat.
    le_succ_of_le hk)]
      exact (P k).incl.monotone ih

/-- The tower of embeddings is Scott-continuous (a composite of the `i_k`). -/
theorem embLE_preservesDirectedSup {n m : ℕ} (h : n ≤ m) :
    PreservesDirectedSup (embLE D P h) := by
```

```
  induction m, h using Nat.le_induction with
  | base =>
      have hid : embLE D P (le_refl n) = id := funext (fun x => embLE_self D P (
    le_refl n) x)
      rw [hid]; intro S _ _; simp
  | succ k hk ih =>
      have hfun : embLE D P (Nat.le_succ_of_le hk)
          = (fun y : D k => (P k).incl y) ∘ embLE D P hk :=
        funext (fun x => embLE_succ D P hk (Nat.le_succ_of_le hk) x)
      rw [hfun]
      exact ScottMap.preservesDirectedSup_comp
        (fun S hS hd => (P k).incl.preservesDirectedSup_coe S hS hd) ih

end InverseLimit

end DomainTheory.ContinuousLattice
```

2

embLE.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.comp
public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- Climb the tower of embeddings: for `n ≤ m`, `embLE h = i_{m-1} ∘ ... ∘ iₙ : Dₙ →
    D_m`. -/
def embLE {n m : ℕ} (h : n ≤ m) (x : D n) : D m :=
  Nat.leRecOn h (fun {k} (y : D k) => (P k).incl y) x

theorem embLE_self {n : ℕ} (h : n ≤ n) (x : D n) : embLE D P h x = x := by
  simp only [embLE, Nat.leRecOn_self]

theorem embLE_succ {n m : ℕ} (h1 : n ≤ m) (h2 : n ≤ m + 1) (x : D n) :
    embLE D P h2 x = (P m).incl (embLE D P h1 x) := by
```

```
  simp only [embLE]
  rw [Nat.leRecOn_succ h1]

theorem embLE_succ_left {n m : ℕ} (h1 : n ≤ m) (h2 : n + 1 ≤ m) (x : D n) :
    embLE D P h2 ((P n).incl x) = embLE D P h1 x := by
  simp only [embLE]
  exact Nat.leRecOn_succ_left x h1 h2

/-- For a compatible sequence `y`, climbing `yₙ` up to level `m ≥ n` stays below `yₘ`.
    -/
theorem embLE_le {n : ℕ} (y : InverseLimit D P) {m : ℕ} (h : n ≤ m) :
    embLE D P h (y.1 n) ≤ y.1 m := by
  induction m, h using Nat.le_induction with
  | base => exact le_of_eq (embLE_self D P _ _)
  | succ k hk ih =>
      rw [embLE_succ D P hk (Nat.le_succ_of_le hk)]
      calc (P k).incl (embLE D P hk (y.1 n))
          ≤ (P k).incl (y.1 k) := (P k).incl.monotone ih
        _ = (P k).incl ((P k).retr (y.1 (k + 1))) := by rw [y.2 k]
        _ ≤ y.1 (k + 1) := (P k).incl_retr_le _

theorem embLE_mono {n m : ℕ} (h : n ≤ m) {x y : D n} (hxy : x ≤ y) :
    embLE D P h x ≤ embLE D P h y := by
  induction m, h using Nat.le_induction with
  | base =>
      rw [embLE_self, embLE_self]
      exact hxy
  | succ k hk ih =>
      rw [embLE_succ D P hk (Nat.le_succ_of_le hk), embLE_succ D P hk (Nat.
    le_succ_of_le hk)]
      exact (P k).incl.monotone ih

/-- The tower of embeddings is Scott-continuous (a composite of the `iₖ`). -/
theorem embLE_preservesDirectedSup {n m : ℕ} (h : n ≤ m) :
    PreservesDirectedSup (embLE D P h) := by
  induction m, h using Nat.le_induction with
  | base =>
      have hid : embLE D P (le_refl n) = id := funext (fun x => embLE_self D P (
    le_refl n) x)
      rw [hid]; intro S _ _; simp
  | succ k hk ih =>
      have hfun : embLE D P (Nat.le_succ_of_le hk)
          = (fun y : D k => (P k).incl y) ∘ embLE D P hk :=
        funext (fun x => embLE_succ D P hk (Nat.le_succ_of_le hk) x)
      rw [hfun]
      exact ScottMap.preservesDirectedSup_comp
        (fun S hS hd => (P k).incl.preservesDirectedSup_coe S hS hd) ih

end InverseLimit

end DomainTheory.ContinuousLattice
```

3

embLE.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.comp
public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- Climb the tower of embeddings: for `n ≤ m`, `embLE h = i_{m-1} ∘ ... ∘ iₙ : Dₙ →
    D_m`. -/
def embLE {n m : ℕ} (h : n ≤ m) (x : D n) : D m :=
  Nat.leRecOn h (fun {k} (y : D k) => (P k).incl y) x

theorem embLE_self {n : ℕ} (h : n ≤ n) (x : D n) : embLE D P h x = x := by
  simp only [embLE, Nat.leRecOn_self]

theorem embLE_succ {n m : ℕ} (h1 : n ≤ m) (h2 : n ≤ m + 1) (x : D n) :
    embLE D P h2 x = (P m).incl (embLE D P h1 x) := by
  simp only [embLE]
  rw [Nat.leRecOn_succ h1]

theorem embLE_succ_left {n m : ℕ} (h1 : n ≤ m) (h2 : n + 1 ≤ m) (x : D n) :
    embLE D P h2 ((P n).incl x) = embLE D P h1 x := by
  simp only [embLE]
  exact Nat.leRecOn_succ_left x h1 h2

/-- For a compatible sequence `y`, climbing `yₙ` up to level `m ≥ n` stays below `yₘ`.
    -/
theorem embLE_le {n : ℕ} (y : InverseLimit D P) {m : ℕ} (h : n ≤ m) :
    embLE D P h (y.1 n) ≤ y.1 m := by
  induction m, h using Nat.le_induction with
  | base => exact le_of_eq (embLE_self D P _ _)
  | succ k hk ih =>
      rw [embLE_succ D P hk (Nat.le_succ_of_le hk)]
      calc (P k).incl (embLE D P hk (y.1 n))
          ≤ (P k).incl (y.1 k) := (P k).incl.monotone ih
        _ = (P k).incl ((P k).retr (y.1 (k + 1))) := by rw [y.2 k]
        _ ≤ y.1 (k + 1) := (P k).incl_retr_le _
```

```
theorem embLE_mono {n m : ℕ} (h : n ≤ m) {x y : D n} (hxy : x ≤ y) :
    embLE D P h x ≤ embLE D P h y := by
  induction m, h using Nat.le_induction with
  | base =>
      rw [embLE_self, embLE_self]
      exact hxy
  | succ k hk ih =>
      rw [embLE_succ D P hk (Nat.le_succ_of_le hk), embLE_succ D P hk (Nat.
    le_succ_of_le hk)]
      exact (P k).incl.monotone ih

/-- The tower of embeddings is Scott-continuous (a composite of the `iₖ`). -/
theorem embLE_preservesDirectedSup {n m : ℕ} (h : n ≤ m) :
    PreservesDirectedSup (embLE D P h) := by
  induction m, h using Nat.le_induction with
  | base =>
      have hid : embLE D P (le_refl n) = id := funext (fun x => embLE_self D P (
    le_refl n) x)
      rw [hid]; intro S _ _; simp
  | succ k hk ih =>
      have hfun : embLE D P (Nat.le_succ_of_le hk)
          = (fun y : D k => (P k).incl y) ∘ embLE D P hk :=
        funext (fun x => embLE_succ D P hk (Nat.le_succ_of_le hk) x)
      rw [hfun]
      exact ScottMap.preservesDirectedSup_comp
        (fun S hS hd => (P k).incl.preservesDirectedSup_coe S hS hd) ih

end InverseLimit

end DomainTheory.ContinuousLattice
```

4

embLE.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.comp
public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit
```

```
variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- Climb the tower of embeddings: for `n ≤ m`, `embLE h = i_{m-1} ∘ ... ∘ iₙ : Dₙ →
    D_m`. -/
def embLE {n m : ℕ} (h : n ≤ m) (x : D n) : D m :=
  Nat.leRecOn h (fun {k} (y : D k) => (P k).incl y) x

theorem embLE_self {n : ℕ} (h : n ≤ n) (x : D n) : embLE D P h x = x := by
  simp only [embLE, Nat.leRecOn_self]

theorem embLE_succ {n m : ℕ} (h1 : n ≤ m) (h2 : n ≤ m + 1) (x : D n) :
    embLE D P h2 x = (P m).incl (embLE D P h1 x) := by
  simp only [embLE]
  rw [Nat.leRecOn_succ h1]

theorem embLE_succ_left {n m : ℕ} (h1 : n ≤ m) (h2 : n + 1 ≤ m) (x : D n) :
    embLE D P h2 ((P n).incl x) = embLE D P h1 x := by
  simp only [embLE]
  exact Nat.leRecOn_succ_left x h1 h2

/-- For a compatible sequence `y`, climbing `yₙ` up to level `m ≥ n` stays below `yₘ`.
    -/
theorem embLE_le {n : ℕ} (y : InverseLimit D P) {m : ℕ} (h : n ≤ m) :
    embLE D P h (y.1 n) ≤ y.1 m := by
  induction m, h using Nat.le_induction with
  | base => exact le_of_eq (embLE_self D P _ _)
  | succ k hk ih =>
      rw [embLE_succ D P hk (Nat.le_succ_of_le hk)]
      calc (P k).incl (embLE D P hk (y.1 n))
          ≤ (P k).incl (y.1 k) := (P k).incl.monotone ih
        _ = (P k).incl ((P k).retr (y.1 (k + 1))) := by rw [y.2 k]
        _ ≤ y.1 (k + 1) := (P k).incl_retr_le _

theorem embLE_mono {n m : ℕ} (h : n ≤ m) {x y : D n} (hxy : x ≤ y) :
    embLE D P h x ≤ embLE D P h y := by
  induction m, h using Nat.le_induction with
  | base =>
      rw [embLE_self, embLE_self]
      exact hxy
  | succ k hk ih =>
      rw [embLE_succ D P hk (Nat.le_succ_of_le hk), embLE_succ D P hk (Nat.
    le_succ_of_le hk)]
      exact (P k).incl.monotone ih

/-- The tower of embeddings is Scott-continuous (a composite of the `iₖ`). -/
theorem embLE_preservesDirectedSup {n m : ℕ} (h : n ≤ m) :
    PreservesDirectedSup (embLE D P h) := by
  induction m, h using Nat.le_induction with
  | base =>
      have hid : embLE D P (le_refl n) = id := funext (fun x => embLE_self D P (
    le_refl n) x)
      rw [hid]; intro S _ _; simp
  | succ k hk ih =>
      have hfun : embLE D P (Nat.le_succ_of_le hk)
```

```
          = (fun y : D k => (P k).incl y) ∘ embLE D P hk :=
        funext (fun x => embLE_succ D P hk (Nat.le_succ_of_le hk) x)
      rw [hfun]
      exact ScottMap.preservesDirectedSup_comp
        (fun S hS hd => (P k).incl.preservesDirectedSup_coe S hS hd) ih

end InverseLimit

end DomainTheory.ContinuousLattice
```

5

embLE.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.comp
public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- Climb the tower of embeddings: for `n ≤ m`, `embLE h = i_{m-1} ∘ ... ∘ iₙ : Dₙ →
    D_m`. -/
def embLE {n m : ℕ} (h : n ≤ m) (x : D n) : D m :=
  Nat.leRecOn h (fun {k} (y : D k) => (P k).incl y) x

theorem embLE_self {n : ℕ} (h : n ≤ n) (x : D n) : embLE D P h x = x := by
  simp only [embLE, Nat.leRecOn_self]

theorem embLE_succ {n m : ℕ} (h1 : n ≤ m) (h2 : n ≤ m + 1) (x : D n) :
    embLE D P h2 x = (P m).incl (embLE D P h1 x) := by
  simp only [embLE]
  rw [Nat.leRecOn_succ h1]

theorem embLE_succ_left {n m : ℕ} (h1 : n ≤ m) (h2 : n + 1 ≤ m) (x : D n) :
    embLE D P h2 ((P n).incl x) = embLE D P h1 x := by
  simp only [embLE]
  exact Nat.leRecOn_succ_left x h1 h2
```

```
/-- For a compatible sequence `y`, climbing `yₙ` up to level `m ≥ n` stays below `yₘ`.
    -/
theorem embLE_le {n : ℕ} (y : InverseLimit D P) {m : ℕ} (h : n ≤ m) :
    embLE D P h (y.1 n) ≤ y.1 m := by
  induction m, h using Nat.le_induction with
  | base => exact le_of_eq (embLE_self D P _ _)
  | succ k hk ih =>
      rw [embLE_succ D P hk (Nat.le_succ_of_le hk)]
      calc (P k).incl (embLE D P hk (y.1 n))
          ≤ (P k).incl (y.1 k) := (P k).incl.monotone ih
        _ = (P k).incl ((P k).retr (y.1 (k + 1))) := by rw [y.2 k]
        _ ≤ y.1 (k + 1) := (P k).incl_retr_le _

theorem embLE_mono {n m : ℕ} (h : n ≤ m) {x y : D n} (hxy : x ≤ y) :
    embLE D P h x ≤ embLE D P h y := by
  induction m, h using Nat.le_induction with
  | base =>
      rw [embLE_self, embLE_self]
      exact hxy
  | succ k hk ih =>
      rw [embLE_succ D P hk (Nat.le_succ_of_le hk), embLE_succ D P hk (Nat.
    le_succ_of_le hk)]
      exact (P k).incl.monotone ih

/-- The tower of embeddings is Scott-continuous (a composite of the `iₖ`). -/
theorem embLE_preservesDirectedSup {n m : ℕ} (h : n ≤ m) :
    PreservesDirectedSup (embLE D P h) := by
  induction m, h using Nat.le_induction with
  | base =>
      have hid : embLE D P (le_refl n) = id := funext (fun x => embLE_self D P (
    le_refl n) x)
      rw [hid]; intro S _ _; simp
  | succ k hk ih =>
      have hfun : embLE D P (Nat.le_succ_of_le hk)
          = (fun y : D k => (P k).incl y) ∘ embLE D P hk :=
        funext (fun x => embLE_succ D P hk (Nat.le_succ_of_le hk) x)
      rw [hfun]
      exact ScottMap.preservesDirectedSup_comp
        (fun S hS hd => (P k).incl.preservesDirectedSup_coe S hS hd) ih

end InverseLimit

end DomainTheory.ContinuousLattice
```

6

embLE.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.comp
```

```
public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- Climb the tower of embeddings: for `n ≤ m`, `embLE h = i_{m-1} ∘ ... ∘ iₙ : Dₙ →
    D_m`. -/
def embLE {n m : ℕ} (h : n ≤ m) (x : D n) : D m :=
  Nat.leRecOn h (fun {k} (y : D k) => (P k).incl y) x

theorem embLE_self {n : ℕ} (h : n ≤ n) (x : D n) : embLE D P h x = x := by
  simp only [embLE, Nat.leRecOn_self]

theorem embLE_succ {n m : ℕ} (h1 : n ≤ m) (h2 : n ≤ m + 1) (x : D n) :
    embLE D P h2 x = (P m).incl (embLE D P h1 x) := by
  simp only [embLE]
  rw [Nat.leRecOn_succ h1]

theorem embLE_succ_left {n m : ℕ} (h1 : n ≤ m) (h2 : n + 1 ≤ m) (x : D n) :
    embLE D P h2 ((P n).incl x) = embLE D P h1 x := by
  simp only [embLE]
  exact Nat.leRecOn_succ_left x h1 h2

/-- For a compatible sequence `y`, climbing `yₙ` up to level `m ≥ n` stays below `yₘ`.
    -/
theorem embLE_le {n : ℕ} (y : InverseLimit D P) {m : ℕ} (h : n ≤ m) :
    embLE D P h (y.1 n) ≤ y.1 m := by
  induction m, h using Nat.le_induction with
  | base => exact le_of_eq (embLE_self D P _ _)
  | succ k hk ih =>
      rw [embLE_succ D P hk (Nat.le_succ_of_le hk)]
      calc (P k).incl (embLE D P hk (y.1 n))
          ≤ (P k).incl (y.1 k) := (P k).incl.monotone ih
        _ = (P k).incl ((P k).retr (y.1 (k + 1))) := by rw [y.2 k]
        _ ≤ y.1 (k + 1) := (P k).incl_retr_le _

theorem embLE_mono {n m : ℕ} (h : n ≤ m) {x y : D n} (hxy : x ≤ y) :
    embLE D P h x ≤ embLE D P h y := by
  induction m, h using Nat.le_induction with
  | base =>
      rw [embLE_self, embLE_self]
      exact hxy
  | succ k hk ih =>
```

```
      rw [embLE_succ D P hk (Nat.le_succ_of_le hk), embLE_succ D P hk (Nat.
    le_succ_of_le hk)]
      exact (P k).incl.monotone ih

/-- The tower of embeddings is Scott-continuous (a composite of the `iₖ`). -/
theorem embLE_preservesDirectedSup {n m : ℕ} (h : n ≤ m) :
    PreservesDirectedSup (embLE D P h) := by
  induction m, h using Nat.le_induction with
  | base =>
      have hid : embLE D P (le_refl n) = id := funext (fun x => embLE_self D P (
    le_refl n) x)
      rw [hid]; intro S _ _; simp
  | succ k hk ih =>
      have hfun : embLE D P (Nat.le_succ_of_le hk)
          = (fun y : D k => (P k).incl y) ∘ embLE D P hk :=
        funext (fun x => embLE_succ D P hk (Nat.le_succ_of_le hk) x)
      rw [hfun]
      exact ScottMap.preservesDirectedSup_comp
        (fun S hS hd => (P k).incl.preservesDirectedSup_coe S hS hd) ih

end InverseLimit

end DomainTheory.ContinuousLattice
```

7

embLE.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.comp
public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- Climb the tower of embeddings: for `n ≤ m`, `embLE h = i_{m-1} ∘ ... ∘ iₙ : Dₙ →
    D_m`. -/
def embLE {n m : ℕ} (h : n ≤ m) (x : D n) : D m :=
```

```
  Nat.leRecOn h (fun {k} (y : D k) => (P k).incl y) x

theorem embLE_self {n : ℕ} (h : n ≤ n) (x : D n) : embLE D P h x = x := by
  simp only [embLE, Nat.leRecOn_self]

theorem embLE_succ {n m : ℕ} (h1 : n ≤ m) (h2 : n ≤ m + 1) (x : D n) :
    embLE D P h2 x = (P m).incl (embLE D P h1 x) := by
  simp only [embLE]
  rw [Nat.leRecOn_succ h1]

theorem embLE_succ_left {n m : ℕ} (h1 : n ≤ m) (h2 : n + 1 ≤ m) (x : D n) :
    embLE D P h2 ((P n).incl x) = embLE D P h1 x := by
  simp only [embLE]
  exact Nat.leRecOn_succ_left x h1 h2

/-- For a compatible sequence `y`, climbing `yₙ` up to level `m ≥ n` stays below `yₘ`.
    -/
theorem embLE_le {n : ℕ} (y : InverseLimit D P) {m : ℕ} (h : n ≤ m) :
    embLE D P h (y.1 n) ≤ y.1 m := by
  induction m, h using Nat.le_induction with
  | base => exact le_of_eq (embLE_self D P _ _)
  | succ k hk ih =>
      rw [embLE_succ D P hk (Nat.le_succ_of_le hk)]
      calc (P k).incl (embLE D P hk (y.1 n))
          ≤ (P k).incl (y.1 k) := (P k).incl.monotone ih
        _ = (P k).incl ((P k).retr (y.1 (k + 1))) := by rw [y.2 k]
        _ ≤ y.1 (k + 1) := (P k).incl_retr_le _

theorem embLE_mono {n m : ℕ} (h : n ≤ m) {x y : D n} (hxy : x ≤ y) :
    embLE D P h x ≤ embLE D P h y := by
  induction m, h using Nat.le_induction with
  | base =>
      rw [embLE_self, embLE_self]
      exact hxy
  | succ k hk ih =>
      rw [embLE_succ D P hk (Nat.le_succ_of_le hk), embLE_succ D P hk (Nat.
    le_succ_of_le hk)]
      exact (P k).incl.monotone ih

/-- The tower of embeddings is Scott-continuous (a composite of the `iₖ`). -/
theorem embLE_preservesDirectedSup {n m : ℕ} (h : n ≤ m) :
    PreservesDirectedSup (embLE D P h) := by
  induction m, h using Nat.le_induction with
  | base =>
      have hid : embLE D P (le_refl n) = id := funext (fun x => embLE_self D P (
    le_refl n) x)
      rw [hid]; intro S _ _; simp
  | succ k hk ih =>
      have hfun : embLE D P (Nat.le_succ_of_le hk)
          = (fun y : D k => (P k).incl y) ∘ embLE D P hk :=
        funext (fun x => embLE_succ D P hk (Nat.le_succ_of_le hk) x)
      rw [hfun]
      exact ScottMap.preservesDirectedSup_comp
        (fun S hS hd => (P k).incl.preservesDirectedSup_coe S hS hd) ih

end InverseLimit
```

```
end DomainTheory.ContinuousLattice
```

8

embLE.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.comp
public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- Climb the tower of embeddings: for `n ≤ m`, `embLE h = i_{m-1} ∘ ... ∘ iₙ : Dₙ →
    D_m`. -/
def embLE {n m : ℕ} (h : n ≤ m) (x : D n) : D m :=
  Nat.leRecOn h (fun {k} (y : D k) => (P k).incl y) x

theorem embLE_self {n : ℕ} (h : n ≤ n) (x : D n) : embLE D P h x = x := by
  simp only [embLE, Nat.leRecOn_self]

theorem embLE_succ {n m : ℕ} (h1 : n ≤ m) (h2 : n ≤ m + 1) (x : D n) :
    embLE D P h2 x = (P m).incl (embLE D P h1 x) := by
  simp only [embLE]
  rw [Nat.leRecOn_succ h1]

theorem embLE_succ_left {n m : ℕ} (h1 : n ≤ m) (h2 : n + 1 ≤ m) (x : D n) :
    embLE D P h2 ((P n).incl x) = embLE D P h1 x := by
  simp only [embLE]
  exact Nat.leRecOn_succ_left x h1 h2

/-- For a compatible sequence `y`, climbing `yₙ` up to level `m ≥ n` stays below `yₘ`.
    -/
theorem embLE_le {n : ℕ} (y : InverseLimit D P) {m : ℕ} (h : n ≤ m) :
    embLE D P h (y.1 n) ≤ y.1 m := by
  induction m, h using Nat.le_induction with
  | base => exact le_of_eq (embLE_self D P _ _)
```

```
  | succ k hk ih =>
      rw [embLE_succ D P hk (Nat.le_succ_of_le hk)]
      calc (P k).incl (embLE D P hk (y.1 n))
          ≤ (P k).incl (y.1 k) := (P k).incl.monotone ih
        _ = (P k).incl ((P k).retr (y.1 (k + 1))) := by rw [y.2 k]
        _ ≤ y.1 (k + 1) := (P k).incl_retr_le _

theorem embLE_mono {n m : ℕ} (h : n ≤ m) {x y : D n} (hxy : x ≤ y) :
    embLE D P h x ≤ embLE D P h y := by
  induction m, h using Nat.le_induction with
  | base =>
      rw [embLE_self, embLE_self]
      exact hxy
  | succ k hk ih =>
      rw [embLE_succ D P hk (Nat.le_succ_of_le hk), embLE_succ D P hk (Nat.
    le_succ_of_le hk)]
      exact (P k).incl.monotone ih

/-- The tower of embeddings is Scott-continuous (a composite of the `iₖ`). -/
theorem embLE_preservesDirectedSup {n m : ℕ} (h : n ≤ m) :
    PreservesDirectedSup (embLE D P h) := by
  induction m, h using Nat.le_induction with
  | base =>
      have hid : embLE D P (le_refl n) = id := funext (fun x => embLE_self D P (
    le_refl n) x)
      rw [hid]; intro S _ _; simp
  | succ k hk ih =>
      have hfun : embLE D P (Nat.le_succ_of_le hk)
          = (fun y : D k => (P k).incl y) ∘ embLE D P hk :=
        funext (fun x => embLE_succ D P hk (Nat.le_succ_of_le hk) x)
      rw [hfun]
      exact ScottMap.preservesDirectedSup_comp
        (fun S hS hd => (P k).incl.preservesDirectedSup_coe S hS hd) ih

end InverseLimit

end DomainTheory.ContinuousLattice
```

9

iComp.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.embLE
public import DomainTheory.ContinuousLattice.InverseLimits.projLE
public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import DomainTheory.ContinuousLattice.InverseLimits.Compatible
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.preservesDirectedSup_monotone
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice
```

```
open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- Scott's embedding component `i_{n∞}(x)_m`: climb for `m ≥ n`, descend for `m < n`.
    -/
def iComp (n : ℕ) (x : D n) (m : ℕ) : D m :=
  if h : n ≤ m then embLE D P h x else projLE D P (le_of_lt (not_le.mp h)) x

theorem iComp_of_le {n m : ℕ} (h : n ≤ m) (x : D n) : iComp D P n x m = embLE D P h x
    :=
  dite_eq_left h

theorem iComp_of_ge {n m : ℕ} (h : ¬ n ≤ m) (x : D n) :
    iComp D P n x m = projLE D P (le_of_lt (not_le.mp h)) x :=
  dite_eq_right h

theorem iComp_self (n : ℕ) (x : D n) : iComp D P n x n = x := by
  rw [iComp_of_le D P (le_refl n), embLE_self]

/-- `i_{n∞}(y_n) ⊑ y` coordinatewise (the heart of `incl_retr_le` for `j_{∞n}`). -/
theorem iComp_incl_le (n : ℕ) (y : InverseLimit D P) (m : ℕ) :
    iComp D P n (y.1 n) m ≤ y.1 m := by
  by_cases h : n ≤ m
  · rw [iComp_of_le D P h]; exact embLE_le D P y h
  · rw [iComp_of_ge D P h]
    exact le_of_eq (projLE.compatible D P y (le_of_lt (not_le.mp h)))

theorem iComp_preservesDirectedSup (n m : ℕ) :
    PreservesDirectedSup (fun x : D n => iComp D P n x m) := by
  by_cases h : n ≤ m
  · have hfun : (fun x : D n => iComp D P n x m) = embLE D P h :=
      funext (fun x => iComp_of_le D P h x)
    rw [hfun]; exact embLE_preservesDirectedSup D P h
  · have hfun : (fun x : D n => iComp D P n x m) = projLE D P (le_of_lt (not_le.mp h))
    :=
      funext (fun x => iComp_of_ge D P h x)
    rw [hfun]; exact projLE.preservesDirectedSup D P _

theorem iComp_monotone (n m : ℕ) : Monotone (fun x : D n => iComp D P n x m) :=
  preservesDirectedSup_monotone (iComp_preservesDirectedSup D P n m)

/-- The sequence `i_{n∞}(x)` is compatible, hence a genuine point of `D_∞`. -/
theorem iComp_compatible (n : ℕ) (x : D n) : Compatible D P (iComp D P n x) := by
  intro m
  by_cases h1 : n ≤ m
  · rw [iComp_of_le D P h1, iComp_of_le D P (Nat.le_succ_of_le h1),
      embLE_succ D P h1 (Nat.le_succ_of_le h1), (P m).retr_incl]
  · by_cases h2 : n ≤ m + 1
    · have hn : n = m + 1 := le_antisymm h2 (not_le.mp h1)
```

```
      subst hn
      rw [iComp_of_le D P (le_refl (m + 1)), embLE_self, iComp_of_ge D P h1,
        projLE.succ D P (le_refl m) (le_of_lt (not_le.mp h1)) x, projLE.self]
    · rw [iComp_of_ge D P h2, iComp_of_ge D P h1,
        projLE.retr D P (le_of_lt (not_le.mp h2)) x]

end InverseLimit

end DomainTheory.ContinuousLattice
```

0

iComp.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.embLE
public import DomainTheory.ContinuousLattice.InverseLimits.projLE
public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import DomainTheory.ContinuousLattice.InverseLimits.Compatible
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.preservesDirectedSup_monotone
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- Scott's embedding component `i_{n∞}(x)_m`: climb for `m ≥ n`, descend for `m < n`.
    -/
def iComp (n : ℕ) (x : D n) (m : ℕ) : D m :=
  if h : n ≤ m then embLE D P h x else projLE D P (le_of_lt (not_le.mp h)) x

theorem iComp_of_le {n m : ℕ} (h : n ≤ m) (x : D n) : iComp D P n x m = embLE D P h x
    :=
  dite_eq_left h

theorem iComp_of_ge {n m : ℕ} (h : ¬ n ≤ m) (x : D n) :
    iComp D P n x m = projLE D P (le_of_lt (not_le.mp h)) x :=
  dite_eq_right h

theorem iComp_self (n : ℕ) (x : D n) : iComp D P n x n = x := by
  rw [iComp_of_le D P (le_refl n), embLE_self]

/-- `i_{n∞}(y_n) ⊑ y` coordinatewise (the heart of `incl_retr_le` for `j_{∞n}`). -/
theorem iComp_incl_le (n : ℕ) (y : InverseLimit D P) (m : ℕ) :
```

```
    iComp D P n (y.1 n) m ≤ y.1 m := by
  by_cases h : n ≤ m
  · rw [iComp_of_le D P h]; exact embLE_le D P y h
  · rw [iComp_of_ge D P h]
    exact le_of_eq (projLE.compatible D P y (le_of_lt (not_le.mp h)))

theorem iComp_preservesDirectedSup (n m : ℕ) :
    PreservesDirectedSup (fun x : D n => iComp D P n x m) := by
  by_cases h : n ≤ m
  · have hfun : (fun x : D n => iComp D P n x m) = embLE D P h :=
      funext (fun x => iComp_of_le D P h x)
    rw [hfun]; exact embLE_preservesDirectedSup D P h
  · have hfun : (fun x : D n => iComp D P n x m) = projLE D P (le_of_lt (not_le.mp h))
    :=
      funext (fun x => iComp_of_ge D P h x)
    rw [hfun]; exact projLE.preservesDirectedSup D P _

theorem iComp_monotone (n m : ℕ) : Monotone (fun x : D n => iComp D P n x m) :=
  preservesDirectedSup_monotone (iComp_preservesDirectedSup D P n m)

/-- The sequence `i_{n∞}(x)` is compatible, hence a genuine point of `D_∞`. -/
theorem iComp_compatible (n : ℕ) (x : D n) : Compatible D P (iComp D P n x) := by
  intro m
  by_cases h1 : n ≤ m
  · rw [iComp_of_le D P h1, iComp_of_le D P (Nat.le_succ_of_le h1),
      embLE_succ D P h1 (Nat.le_succ_of_le h1), (P m).retr_incl]
  · by_cases h2 : n ≤ m + 1
    · have hn : n = m + 1 := le_antisymm h2 (not_le.mp h1)
      subst hn
      rw [iComp_of_le D P (le_refl (m + 1)), embLE_self, iComp_of_ge D P h1,
        projLE.succ D P (le_refl m) (le_of_lt (not_le.mp h1)) x, projLE.self]
    · rw [iComp_of_ge D P h2, iComp_of_ge D P h1,
        projLE.retr D P (le_of_lt (not_le.mp h2)) x]

end InverseLimit

end DomainTheory.ContinuousLattice
```

1

iComp.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.embLE
public import DomainTheory.ContinuousLattice.InverseLimits.projLE
public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import DomainTheory.ContinuousLattice.InverseLimits.Compatible
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.preservesDirectedSup_monotone
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice
```

```
open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- Scott's embedding component `i_{n∞}(x)_m`: climb for `m ≥ n`, descend for `m < n`.
    -/
def iComp (n : ℕ) (x : D n) (m : ℕ) : D m :=
  if h : n ≤ m then embLE D P h x else projLE D P (le_of_lt (not_le.mp h)) x

theorem iComp_of_le {n m : ℕ} (h : n ≤ m) (x : D n) : iComp D P n x m = embLE D P h x
    :=
  dite_eq_left h

theorem iComp_of_ge {n m : ℕ} (h : ¬ n ≤ m) (x : D n) :
    iComp D P n x m = projLE D P (le_of_lt (not_le.mp h)) x :=
  dite_eq_right h

theorem iComp_self (n : ℕ) (x : D n) : iComp D P n x n = x := by
  rw [iComp_of_le D P (le_refl n), embLE_self]

/-- `i_{n∞}(y_n) ⊑ y` coordinatewise (the heart of `incl_retr_le` for `j_{∞n}`). -/
theorem iComp_incl_le (n : ℕ) (y : InverseLimit D P) (m : ℕ) :
    iComp D P n (y.1 n) m ≤ y.1 m := by
  by_cases h : n ≤ m
  · rw [iComp_of_le D P h]; exact embLE_le D P y h
  · rw [iComp_of_ge D P h]
    exact le_of_eq (projLE.compatible D P y (le_of_lt (not_le.mp h)))

theorem iComp_preservesDirectedSup (n m : ℕ) :
    PreservesDirectedSup (fun x : D n => iComp D P n x m) := by
  by_cases h : n ≤ m
  · have hfun : (fun x : D n => iComp D P n x m) = embLE D P h :=
      funext (fun x => iComp_of_le D P h x)
    rw [hfun]; exact embLE_preservesDirectedSup D P h
  · have hfun : (fun x : D n => iComp D P n x m) = projLE D P (le_of_lt (not_le.mp h))
    :=
      funext (fun x => iComp_of_ge D P h x)
    rw [hfun]; exact projLE.preservesDirectedSup D P _

theorem iComp_monotone (n m : ℕ) : Monotone (fun x : D n => iComp D P n x m) :=
  preservesDirectedSup_monotone (iComp_preservesDirectedSup D P n m)

/-- The sequence `i_{n∞}(x)` is compatible, hence a genuine point of `D_∞`. -/
theorem iComp_compatible (n : ℕ) (x : D n) : Compatible D P (iComp D P n x) := by
  intro m
  by_cases h1 : n ≤ m
  · rw [iComp_of_le D P h1, iComp_of_le D P (Nat.le_succ_of_le h1),
      embLE_succ D P h1 (Nat.le_succ_of_le h1), (P m).retr_incl]
  · by_cases h2 : n ≤ m + 1
    · have hn : n = m + 1 := le_antisymm h2 (not_le.mp h1)
```

```
      subst hn
      rw [iComp_of_le D P (le_refl (m + 1)), embLE_self, iComp_of_ge D P h1,
        projLE.succ D P (le_refl m) (le_of_lt (not_le.mp h1)) x, projLE.self]
    · rw [iComp_of_ge D P h2, iComp_of_ge D P h1,
        projLE.retr D P (le_of_lt (not_le.mp h2)) x]

end InverseLimit

end DomainTheory.ContinuousLattice
```

2

iComp.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.embLE
public import DomainTheory.ContinuousLattice.InverseLimits.projLE
public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import DomainTheory.ContinuousLattice.InverseLimits.Compatible
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.preservesDirectedSup_monotone
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- Scott's embedding component `i_{n∞}(x)_m`: climb for `m ≥ n`, descend for `m < n`.
    -/
def iComp (n : ℕ) (x : D n) (m : ℕ) : D m :=
  if h : n ≤ m then embLE D P h x else projLE D P (le_of_lt (not_le.mp h)) x

theorem iComp_of_le {n m : ℕ} (h : n ≤ m) (x : D n) : iComp D P n x m = embLE D P h x
    :=
  dite_eq_left h

theorem iComp_of_ge {n m : ℕ} (h : ¬ n ≤ m) (x : D n) :
    iComp D P n x m = projLE D P (le_of_lt (not_le.mp h)) x :=
  dite_eq_right h

theorem iComp_self (n : ℕ) (x : D n) : iComp D P n x n = x := by
  rw [iComp_of_le D P (le_refl n), embLE_self]

/-- `i_{n∞}(y_n) ⊑ y` coordinatewise (the heart of `incl_retr_le` for `j_{∞n}`). -/
theorem iComp_incl_le (n : ℕ) (y : InverseLimit D P) (m : ℕ) :
```

```
    iComp D P n (y.1 n) m ≤ y.1 m := by
  by_cases h : n ≤ m
  · rw [iComp_of_le D P h]; exact embLE_le D P y h
  · rw [iComp_of_ge D P h]
    exact le_of_eq (projLE.compatible D P y (le_of_lt (not_le.mp h)))

theorem iComp_preservesDirectedSup (n m : ℕ) :
    PreservesDirectedSup (fun x : D n => iComp D P n x m) := by
  by_cases h : n ≤ m
  · have hfun : (fun x : D n => iComp D P n x m) = embLE D P h :=
      funext (fun x => iComp_of_le D P h x)
    rw [hfun]; exact embLE_preservesDirectedSup D P h
  · have hfun : (fun x : D n => iComp D P n x m) = projLE D P (le_of_lt (not_le.mp h))
    :=
      funext (fun x => iComp_of_ge D P h x)
    rw [hfun]; exact projLE.preservesDirectedSup D P _

theorem iComp_monotone (n m : ℕ) : Monotone (fun x : D n => iComp D P n x m) :=
  preservesDirectedSup_monotone (iComp_preservesDirectedSup D P n m)

/-- The sequence `i_{n∞}(x)` is compatible, hence a genuine point of `D_∞`. -/
theorem iComp_compatible (n : ℕ) (x : D n) : Compatible D P (iComp D P n x) := by
  intro m
  by_cases h1 : n ≤ m
  · rw [iComp_of_le D P h1, iComp_of_le D P (Nat.le_succ_of_le h1),
      embLE_succ D P h1 (Nat.le_succ_of_le h1), (P m).retr_incl]
  · by_cases h2 : n ≤ m + 1
    · have hn : n = m + 1 := le_antisymm h2 (not_le.mp h1)
      subst hn
      rw [iComp_of_le D P (le_refl (m + 1)), embLE_self, iComp_of_ge D P h1,
        projLE.succ D P (le_refl m) (le_of_lt (not_le.mp h1)) x, projLE.self]
    · rw [iComp_of_ge D P h2, iComp_of_ge D P h1,
        projLE.retr D P (le_of_lt (not_le.mp h2)) x]

end InverseLimit

end DomainTheory.ContinuousLattice
```

3

iComp.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.embLE
public import DomainTheory.ContinuousLattice.InverseLimits.projLE
public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import DomainTheory.ContinuousLattice.InverseLimits.Compatible
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.preservesDirectedSup_monotone
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice
```

```
open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- Scott's embedding component `i_{n∞}(x)_m`: climb for `m ≥ n`, descend for `m < n`.
    -/
def iComp (n : ℕ) (x : D n) (m : ℕ) : D m :=
  if h : n ≤ m then embLE D P h x else projLE D P (le_of_lt (not_le.mp h)) x

theorem iComp_of_le {n m : ℕ} (h : n ≤ m) (x : D n) : iComp D P n x m = embLE D P h x
    :=
  dite_eq_left h

theorem iComp_of_ge {n m : ℕ} (h : ¬ n ≤ m) (x : D n) :
    iComp D P n x m = projLE D P (le_of_lt (not_le.mp h)) x :=
  dite_eq_right h

theorem iComp_self (n : ℕ) (x : D n) : iComp D P n x n = x := by
  rw [iComp_of_le D P (le_refl n), embLE_self]

/-- `i_{n∞}(y_n) ⊑ y` coordinatewise (the heart of `incl_retr_le` for `j_{∞n}`). -/
theorem iComp_incl_le (n : ℕ) (y : InverseLimit D P) (m : ℕ) :
    iComp D P n (y.1 n) m ≤ y.1 m := by
  by_cases h : n ≤ m
  · rw [iComp_of_le D P h]; exact embLE_le D P y h
  · rw [iComp_of_ge D P h]
    exact le_of_eq (projLE.compatible D P y (le_of_lt (not_le.mp h)))

theorem iComp_preservesDirectedSup (n m : ℕ) :
    PreservesDirectedSup (fun x : D n => iComp D P n x m) := by
  by_cases h : n ≤ m
  · have hfun : (fun x : D n => iComp D P n x m) = embLE D P h :=
      funext (fun x => iComp_of_le D P h x)
    rw [hfun]; exact embLE_preservesDirectedSup D P h
  · have hfun : (fun x : D n => iComp D P n x m) = projLE D P (le_of_lt (not_le.mp h))
    :=
      funext (fun x => iComp_of_ge D P h x)
    rw [hfun]; exact projLE.preservesDirectedSup D P _

theorem iComp_monotone (n m : ℕ) : Monotone (fun x : D n => iComp D P n x m) :=
  preservesDirectedSup_monotone (iComp_preservesDirectedSup D P n m)

/-- The sequence `i_{n∞}(x)` is compatible, hence a genuine point of `D_∞`. -/
theorem iComp_compatible (n : ℕ) (x : D n) : Compatible D P (iComp D P n x) := by
  intro m
  by_cases h1 : n ≤ m
  · rw [iComp_of_le D P h1, iComp_of_le D P (Nat.le_succ_of_le h1),
      embLE_succ D P h1 (Nat.le_succ_of_le h1), (P m).retr_incl]
  · by_cases h2 : n ≤ m + 1
    · have hn : n = m + 1 := le_antisymm h2 (not_le.mp h1)
```

```
      subst hn
      rw [iComp_of_le D P (le_refl (m + 1)), embLE_self, iComp_of_ge D P h1,
        projLE.succ D P (le_refl m) (le_of_lt (not_le.mp h1)) x, projLE.self]
    · rw [iComp_of_ge D P h2, iComp_of_ge D P h1,
        projLE.retr D P (le_of_lt (not_le.mp h2)) x]

end InverseLimit

end DomainTheory.ContinuousLattice
```

4

iComp.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.embLE
public import DomainTheory.ContinuousLattice.InverseLimits.projLE
public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import DomainTheory.ContinuousLattice.InverseLimits.Compatible
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.preservesDirectedSup_monotone
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- Scott's embedding component `i_{n∞}(x)_m`: climb for `m ≥ n`, descend for `m < n`.
    -/
def iComp (n : ℕ) (x : D n) (m : ℕ) : D m :=
  if h : n ≤ m then embLE D P h x else projLE D P (le_of_lt (not_le.mp h)) x

theorem iComp_of_le {n m : ℕ} (h : n ≤ m) (x : D n) : iComp D P n x m = embLE D P h x
    :=
  dite_eq_left h

theorem iComp_of_ge {n m : ℕ} (h : ¬ n ≤ m) (x : D n) :
    iComp D P n x m = projLE D P (le_of_lt (not_le.mp h)) x :=
  dite_eq_right h

theorem iComp_self (n : ℕ) (x : D n) : iComp D P n x n = x := by
  rw [iComp_of_le D P (le_refl n), embLE_self]

/-- `i_{n∞}(y_n) ⊑ y` coordinatewise (the heart of `incl_retr_le` for `j_{∞n}`). -/
theorem iComp_incl_le (n : ℕ) (y : InverseLimit D P) (m : ℕ) :
```

```
    iComp D P n (y.1 n) m ≤ y.1 m := by
  by_cases h : n ≤ m
  · rw [iComp_of_le D P h]; exact embLE_le D P y h
  · rw [iComp_of_ge D P h]
    exact le_of_eq (projLE.compatible D P y (le_of_lt (not_le.mp h)))

theorem iComp_preservesDirectedSup (n m : ℕ) :
    PreservesDirectedSup (fun x : D n => iComp D P n x m) := by
  by_cases h : n ≤ m
  · have hfun : (fun x : D n => iComp D P n x m) = embLE D P h :=
      funext (fun x => iComp_of_le D P h x)
    rw [hfun]; exact embLE_preservesDirectedSup D P h
  · have hfun : (fun x : D n => iComp D P n x m) = projLE D P (le_of_lt (not_le.mp h))
    :=
      funext (fun x => iComp_of_ge D P h x)
    rw [hfun]; exact projLE.preservesDirectedSup D P _

theorem iComp_monotone (n m : ℕ) : Monotone (fun x : D n => iComp D P n x m) :=
  preservesDirectedSup_monotone (iComp_preservesDirectedSup D P n m)

/-- The sequence `i_{n∞}(x)` is compatible, hence a genuine point of `D_∞`. -/
theorem iComp_compatible (n : ℕ) (x : D n) : Compatible D P (iComp D P n x) := by
  intro m
  by_cases h1 : n ≤ m
  · rw [iComp_of_le D P h1, iComp_of_le D P (Nat.le_succ_of_le h1),
      embLE_succ D P h1 (Nat.le_succ_of_le h1), (P m).retr_incl]
  · by_cases h2 : n ≤ m + 1
    · have hn : n = m + 1 := le_antisymm h2 (not_le.mp h1)
      subst hn
      rw [iComp_of_le D P (le_refl (m + 1)), embLE_self, iComp_of_ge D P h1,
        projLE.succ D P (le_refl m) (le_of_lt (not_le.mp h1)) x, projLE.self]
    · rw [iComp_of_ge D P h2, iComp_of_ge D P h1,
        projLE.retr D P (le_of_lt (not_le.mp h2)) x]

end InverseLimit

end DomainTheory.ContinuousLattice
```

5

iComp.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.embLE
public import DomainTheory.ContinuousLattice.InverseLimits.projLE
public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import DomainTheory.ContinuousLattice.InverseLimits.Compatible
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.preservesDirectedSup_monotone
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice
```

```
open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- Scott's embedding component `i_{n∞}(x)_m`: climb for `m ≥ n`, descend for `m < n`.
    -/
def iComp (n : ℕ) (x : D n) (m : ℕ) : D m :=
  if h : n ≤ m then embLE D P h x else projLE D P (le_of_lt (not_le.mp h)) x

theorem iComp_of_le {n m : ℕ} (h : n ≤ m) (x : D n) : iComp D P n x m = embLE D P h x
    :=
  dite_eq_left h

theorem iComp_of_ge {n m : ℕ} (h : ¬ n ≤ m) (x : D n) :
    iComp D P n x m = projLE D P (le_of_lt (not_le.mp h)) x :=
  dite_eq_right h

theorem iComp_self (n : ℕ) (x : D n) : iComp D P n x n = x := by
  rw [iComp_of_le D P (le_refl n), embLE_self]

/-- `i_{n∞}(y_n) ⊑ y` coordinatewise (the heart of `incl_retr_le` for `j_{∞n}`). -/
theorem iComp_incl_le (n : ℕ) (y : InverseLimit D P) (m : ℕ) :
    iComp D P n (y.1 n) m ≤ y.1 m := by
  by_cases h : n ≤ m
  · rw [iComp_of_le D P h]; exact embLE_le D P y h
  · rw [iComp_of_ge D P h]
    exact le_of_eq (projLE.compatible D P y (le_of_lt (not_le.mp h)))

theorem iComp_preservesDirectedSup (n m : ℕ) :
    PreservesDirectedSup (fun x : D n => iComp D P n x m) := by
  by_cases h : n ≤ m
  · have hfun : (fun x : D n => iComp D P n x m) = embLE D P h :=
      funext (fun x => iComp_of_le D P h x)
    rw [hfun]; exact embLE_preservesDirectedSup D P h
  · have hfun : (fun x : D n => iComp D P n x m) = projLE D P (le_of_lt (not_le.mp h))
    :=
      funext (fun x => iComp_of_ge D P h x)
    rw [hfun]; exact projLE.preservesDirectedSup D P _

theorem iComp_monotone (n m : ℕ) : Monotone (fun x : D n => iComp D P n x m) :=
  preservesDirectedSup_monotone (iComp_preservesDirectedSup D P n m)

/-- The sequence `i_{n∞}(x)` is compatible, hence a genuine point of `D_∞`. -/
theorem iComp_compatible (n : ℕ) (x : D n) : Compatible D P (iComp D P n x) := by
  intro m
  by_cases h1 : n ≤ m
  · rw [iComp_of_le D P h1, iComp_of_le D P (Nat.le_succ_of_le h1),
      embLE_succ D P h1 (Nat.le_succ_of_le h1), (P m).retr_incl]
  · by_cases h2 : n ≤ m + 1
    · have hn : n = m + 1 := le_antisymm h2 (not_le.mp h1)
```

```
      subst hn
      rw [iComp_of_le D P (le_refl (m + 1)), embLE_self, iComp_of_ge D P h1,
        projLE.succ D P (le_refl m) (le_of_lt (not_le.mp h1)) x, projLE.self]
    · rw [iComp_of_ge D P h2, iComp_of_ge D P h1,
        projLE.retr D P (le_of_lt (not_le.mp h2)) x]

end InverseLimit

end DomainTheory.ContinuousLattice
```

6

iComp.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.embLE
public import DomainTheory.ContinuousLattice.InverseLimits.projLE
public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import DomainTheory.ContinuousLattice.InverseLimits.Compatible
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.preservesDirectedSup_monotone
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- Scott's embedding component `i_{n∞}(x)_m`: climb for `m ≥ n`, descend for `m < n`.
    -/
def iComp (n : ℕ) (x : D n) (m : ℕ) : D m :=
  if h : n ≤ m then embLE D P h x else projLE D P (le_of_lt (not_le.mp h)) x

theorem iComp_of_le {n m : ℕ} (h : n ≤ m) (x : D n) : iComp D P n x m = embLE D P h x
    :=
  dite_eq_left h

theorem iComp_of_ge {n m : ℕ} (h : ¬ n ≤ m) (x : D n) :
    iComp D P n x m = projLE D P (le_of_lt (not_le.mp h)) x :=
  dite_eq_right h

theorem iComp_self (n : ℕ) (x : D n) : iComp D P n x n = x := by
  rw [iComp_of_le D P (le_refl n), embLE_self]

/-- `i_{n∞}(y_n) ⊑ y` coordinatewise (the heart of `incl_retr_le` for `j_{∞n}`). -/
theorem iComp_incl_le (n : ℕ) (y : InverseLimit D P) (m : ℕ) :
```

```
    iComp D P n (y.1 n) m ≤ y.1 m := by
  by_cases h : n ≤ m
  · rw [iComp_of_le D P h]; exact embLE_le D P y h
  · rw [iComp_of_ge D P h]
    exact le_of_eq (projLE.compatible D P y (le_of_lt (not_le.mp h)))

theorem iComp_preservesDirectedSup (n m : ℕ) :
    PreservesDirectedSup (fun x : D n => iComp D P n x m) := by
  by_cases h : n ≤ m
  · have hfun : (fun x : D n => iComp D P n x m) = embLE D P h :=
      funext (fun x => iComp_of_le D P h x)
    rw [hfun]; exact embLE_preservesDirectedSup D P h
  · have hfun : (fun x : D n => iComp D P n x m) = projLE D P (le_of_lt (not_le.mp h))
    :=
      funext (fun x => iComp_of_ge D P h x)
    rw [hfun]; exact projLE.preservesDirectedSup D P _

theorem iComp_monotone (n m : ℕ) : Monotone (fun x : D n => iComp D P n x m) :=
  preservesDirectedSup_monotone (iComp_preservesDirectedSup D P n m)

/-- The sequence `i_{n∞}(x)` is compatible, hence a genuine point of `D_∞`. -/
theorem iComp_compatible (n : ℕ) (x : D n) : Compatible D P (iComp D P n x) := by
  intro m
  by_cases h1 : n ≤ m
  · rw [iComp_of_le D P h1, iComp_of_le D P (Nat.le_succ_of_le h1),
      embLE_succ D P h1 (Nat.le_succ_of_le h1), (P m).retr_incl]
  · by_cases h2 : n ≤ m + 1
    · have hn : n = m + 1 := le_antisymm h2 (not_le.mp h1)
      subst hn
      rw [iComp_of_le D P (le_refl (m + 1)), embLE_self, iComp_of_ge D P h1,
        projLE.succ D P (le_refl m) (le_of_lt (not_le.mp h1)) x, projLE.self]
    · rw [iComp_of_ge D P h2, iComp_of_ge D P h1,
        projLE.retr D P (le_of_lt (not_le.mp h2)) x]

end InverseLimit

end DomainTheory.ContinuousLattice
```

7

iComp.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.embLE
public import DomainTheory.ContinuousLattice.InverseLimits.projLE
public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import DomainTheory.ContinuousLattice.InverseLimits.Compatible
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.preservesDirectedSup_monotone
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice
```

```
open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- Scott's embedding component `i_{n∞}(x)_m`: climb for `m ≥ n`, descend for `m < n`.
    -/
def iComp (n : ℕ) (x : D n) (m : ℕ) : D m :=
  if h : n ≤ m then embLE D P h x else projLE D P (le_of_lt (not_le.mp h)) x

theorem iComp_of_le {n m : ℕ} (h : n ≤ m) (x : D n) : iComp D P n x m = embLE D P h x
    :=
  dite_eq_left h

theorem iComp_of_ge {n m : ℕ} (h : ¬ n ≤ m) (x : D n) :
    iComp D P n x m = projLE D P (le_of_lt (not_le.mp h)) x :=
  dite_eq_right h

theorem iComp_self (n : ℕ) (x : D n) : iComp D P n x n = x := by
  rw [iComp_of_le D P (le_refl n), embLE_self]

/-- `i_{n∞}(y_n) ⊑ y` coordinatewise (the heart of `incl_retr_le` for `j_{∞n}`). -/
theorem iComp_incl_le (n : ℕ) (y : InverseLimit D P) (m : ℕ) :
    iComp D P n (y.1 n) m ≤ y.1 m := by
  by_cases h : n ≤ m
  · rw [iComp_of_le D P h]; exact embLE_le D P y h
  · rw [iComp_of_ge D P h]
    exact le_of_eq (projLE.compatible D P y (le_of_lt (not_le.mp h)))

theorem iComp_preservesDirectedSup (n m : ℕ) :
    PreservesDirectedSup (fun x : D n => iComp D P n x m) := by
  by_cases h : n ≤ m
  · have hfun : (fun x : D n => iComp D P n x m) = embLE D P h :=
      funext (fun x => iComp_of_le D P h x)
    rw [hfun]; exact embLE_preservesDirectedSup D P h
  · have hfun : (fun x : D n => iComp D P n x m) = projLE D P (le_of_lt (not_le.mp h))
    :=
      funext (fun x => iComp_of_ge D P h x)
    rw [hfun]; exact projLE.preservesDirectedSup D P _

theorem iComp_monotone (n m : ℕ) : Monotone (fun x : D n => iComp D P n x m) :=
  preservesDirectedSup_monotone (iComp_preservesDirectedSup D P n m)

/-- The sequence `i_{n∞}(x)` is compatible, hence a genuine point of `D_∞`. -/
theorem iComp_compatible (n : ℕ) (x : D n) : Compatible D P (iComp D P n x) := by
  intro m
  by_cases h1 : n ≤ m
  · rw [iComp_of_le D P h1, iComp_of_le D P (Nat.le_succ_of_le h1),
      embLE_succ D P h1 (Nat.le_succ_of_le h1), (P m).retr_incl]
  · by_cases h2 : n ≤ m + 1
    · have hn : n = m + 1 := le_antisymm h2 (not_le.mp h1)
```

```
      subst hn
      rw [iComp_of_le D P (le_refl (m + 1)), embLE_self, iComp_of_ge D P h1,
        projLE.succ D P (le_refl m) (le_of_lt (not_le.mp h1)) x, projLE.self]
    · rw [iComp_of_ge D P h2, iComp_of_ge D P h1,
        projLE.retr D P (le_of_lt (not_le.mp h2)) x]

end InverseLimit

end DomainTheory.ContinuousLattice
```

8

iComp.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.embLE
public import DomainTheory.ContinuousLattice.InverseLimits.projLE
public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import DomainTheory.ContinuousLattice.InverseLimits.Compatible
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.preservesDirectedSup_monotone
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- Scott's embedding component `i_{n∞}(x)_m`: climb for `m ≥ n`, descend for `m < n`.
    -/
def iComp (n : ℕ) (x : D n) (m : ℕ) : D m :=
  if h : n ≤ m then embLE D P h x else projLE D P (le_of_lt (not_le.mp h)) x

theorem iComp_of_le {n m : ℕ} (h : n ≤ m) (x : D n) : iComp D P n x m = embLE D P h x
    :=
  dite_eq_left h

theorem iComp_of_ge {n m : ℕ} (h : ¬ n ≤ m) (x : D n) :
    iComp D P n x m = projLE D P (le_of_lt (not_le.mp h)) x :=
  dite_eq_right h

theorem iComp_self (n : ℕ) (x : D n) : iComp D P n x n = x := by
  rw [iComp_of_le D P (le_refl n), embLE_self]

/-- `i_{n∞}(y_n) ⊑ y` coordinatewise (the heart of `incl_retr_le` for `j_{∞n}`). -/
theorem iComp_incl_le (n : ℕ) (y : InverseLimit D P) (m : ℕ) :
```

```
    iComp D P n (y.1 n) m ≤ y.1 m := by
  by_cases h : n ≤ m
  · rw [iComp_of_le D P h]; exact embLE_le D P y h
  · rw [iComp_of_ge D P h]
    exact le_of_eq (projLE.compatible D P y (le_of_lt (not_le.mp h)))

theorem iComp_preservesDirectedSup (n m : ℕ) :
    PreservesDirectedSup (fun x : D n => iComp D P n x m) := by
  by_cases h : n ≤ m
  · have hfun : (fun x : D n => iComp D P n x m) = embLE D P h :=
      funext (fun x => iComp_of_le D P h x)
    rw [hfun]; exact embLE_preservesDirectedSup D P h
  · have hfun : (fun x : D n => iComp D P n x m) = projLE D P (le_of_lt (not_le.mp h))
    :=
      funext (fun x => iComp_of_ge D P h x)
    rw [hfun]; exact projLE.preservesDirectedSup D P _

theorem iComp_monotone (n m : ℕ) : Monotone (fun x : D n => iComp D P n x m) :=
  preservesDirectedSup_monotone (iComp_preservesDirectedSup D P n m)

/-- The sequence `i_{n∞}(x)` is compatible, hence a genuine point of `D_∞`. -/
theorem iComp_compatible (n : ℕ) (x : D n) : Compatible D P (iComp D P n x) := by
  intro m
  by_cases h1 : n ≤ m
  · rw [iComp_of_le D P h1, iComp_of_le D P (Nat.le_succ_of_le h1),
      embLE_succ D P h1 (Nat.le_succ_of_le h1), (P m).retr_incl]
  · by_cases h2 : n ≤ m + 1
    · have hn : n = m + 1 := le_antisymm h2 (not_le.mp h1)
      subst hn
      rw [iComp_of_le D P (le_refl (m + 1)), embLE_self, iComp_of_ge D P h1,
        projLE.succ D P (le_refl m) (le_of_lt (not_le.mp h1)) x, projLE.self]
    · rw [iComp_of_ge D P h2, iComp_of_ge D P h1,
        projLE.retr D P (le_of_lt (not_le.mp h2)) x]

end InverseLimit

end DomainTheory.ContinuousLattice
```

9

embInfFun.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import DomainTheory.ContinuousLattice.InverseLimits.iComp
public import DomainTheory.ContinuousLattice.InverseLimits.instCompleteLattice
public import DomainTheory.ContinuousLattice.InverseLimits.coe_sSup_of_directed
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set
```

```
universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- The embedding `i_{n∞} : D_n → D_∞` as a bare function into the inverse limit. -/
def embInfFun (n : ℕ) (x : D n) : InverseLimit D P := ⟨iComp D P n x, iComp_compatible
    D P n x⟩

@[simp] theorem embInfFun_coe (n : ℕ) (x : D n) : (embInfFun D P n x).1 = iComp D P n
    x := rfl

theorem embInf_monotone (n : ℕ) : Monotone (embInfFun D P n) := by
  intro x x' hxx
  rw [← Subtype.coe_le_coe]
  intro m
  exact iComp_monotone D P n m hxx

theorem embInf_preservesDirectedSup (n : ℕ) : PreservesDirectedSup (embInfFun D P n)
    := by
  intro S hS hSdir
  have hTne : (embInfFun D P n '' S).Nonempty := hS.image _
  have hTdir : DirectedOn (· ≤ ·) (embInfFun D P n '' S) := by
    rintro _ ⟨a, ha, rfl⟩ _ ⟨b, hb, rfl⟩
    obtain ⟨c, hc, hac, hbc⟩ := hSdir a ha b hb
    exact ⟨embInfFun D P n c, Set.mem_image_of_mem _ hc,
      embInf_monotone D P n hac, embInf_monotone D P n hbc⟩
  apply Subtype.ext
  rw [coe_sSup_of_directed D P (embInfFun D P n '' S) hTne hTdir]
  funext m
  have heq : Function.eval m '' (Subtype.val '' (embInfFun D P n '' S))
      = (fun x => iComp D P n x m) '' S := by
    simp only [Set.image_image, embInfFun_coe, Function.eval]
  rw [sSup_apply_eq_sSup_image, heq]
  exact iComp_preservesDirectedSup D P n m hS hSdir

end InverseLimit

end DomainTheory.ContinuousLattice
```

0

embInfFun.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import DomainTheory.ContinuousLattice.InverseLimits.iComp
public import DomainTheory.ContinuousLattice.InverseLimits.instCompleteLattice
public import DomainTheory.ContinuousLattice.InverseLimits.coe_sSup_of_directed
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import Mathlib.Data.Set.Basic
```

```
@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- The embedding `i_{n∞} : Dₙ → D_∞` as a bare function into the inverse limit. -/
def embInfFun (n : ℕ) (x : D n) : InverseLimit D P := ⟨iComp D P n x, iComp_compatible
    D P n x⟩

@[simp] theorem embInfFun_coe (n : ℕ) (x : D n) : (embInfFun D P n x).1 = iComp D P n
    x := rfl

theorem embInf_monotone (n : ℕ) : Monotone (embInfFun D P n) := by
  intro x x' hxx
  rw [← Subtype.coe_le_coe]
  intro m
  exact iComp_monotone D P n m hxx

theorem embInf_preservesDirectedSup (n : ℕ) : PreservesDirectedSup (embInfFun D P n)
    := by
  intro S hS hSdir
  have hTne : (embInfFun D P n '' S).Nonempty := hS.image _
  have hTdir : DirectedOn (· ≤ ·) (embInfFun D P n '' S) := by
    rintro _ ⟨a, ha, rfl⟩ _ ⟨b, hb, rfl⟩
    obtain ⟨c, hc, hac, hbc⟩ := hSdir a ha b hb
    exact ⟨embInfFun D P n c, Set.mem_image_of_mem _ hc,
      embInf_monotone D P n hac, embInf_monotone D P n hbc⟩
  apply Subtype.ext
  rw [coe_sSup_of_directed D P (embInfFun D P n '' S) hTne hTdir]
  funext m
  have heq : Function.eval m '' (Subtype.val '' (embInfFun D P n '' S))
      = (fun x => iComp D P n x m) '' S := by
    simp only [Set.image_image, embInfFun_coe, Function.eval]
  rw [sSup_apply_eq_sSup_image, heq]
  exact iComp_preservesDirectedSup D P n m hS hSdir

end InverseLimit

end DomainTheory.ContinuousLattice
```

1

embInfFun.lean

```
module
```

```
public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import DomainTheory.ContinuousLattice.InverseLimits.iComp
public import DomainTheory.ContinuousLattice.InverseLimits.instCompleteLattice
public import DomainTheory.ContinuousLattice.InverseLimits.coe_sSup_of_directed
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- The embedding `i_{n∞} : D_n → D_∞` as a bare function into the inverse limit. -/
def embInfFun (n : ℕ) (x : D n) : InverseLimit D P := ⟨iComp D P n x, iComp_compatible
    D P n x⟩

@[simp] theorem embInfFun_coe (n : ℕ) (x : D n) : (embInfFun D P n x).1 = iComp D P n
    x := rfl

theorem embInf_monotone (n : ℕ) : Monotone (embInfFun D P n) := by
  intro x x' hxx
  rw [← Subtype.coe_le_coe]
  intro m
  exact iComp_monotone D P n m hxx

theorem embInf_preservesDirectedSup (n : ℕ) : PreservesDirectedSup (embInfFun D P n)
    := by
  intro S hS hSdir
  have hTne : (embInfFun D P n '' S).Nonempty := hS.image _
  have hTdir : DirectedOn (· ≤ ·) (embInfFun D P n '' S) := by
    rintro _ ⟨a, ha, rfl⟩ _ ⟨b, hb, rfl⟩
    obtain ⟨c, hc, hac, hbc⟩ := hSdir a ha b hb
    exact ⟨embInfFun D P n c, Set.mem_image_of_mem _ hc,
      embInf_monotone D P n hac, embInf_monotone D P n hbc⟩
  apply Subtype.ext
  rw [coe_sSup_of_directed D P (embInfFun D P n '' S) hTne hTdir]
  funext m
  have heq : Function.eval m '' (Subtype.val '' (embInfFun D P n '' S))
      = (fun x => iComp D P n x m) '' S := by
    simp only [Set.image_image, embInfFun_coe, Function.eval]
  rw [sSup_apply_eq_sSup_image, heq]
  exact iComp_preservesDirectedSup D P n m hS hSdir

end InverseLimit

end DomainTheory.ContinuousLattice
```

2

embInfFun.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import DomainTheory.ContinuousLattice.InverseLimits.iComp
public import DomainTheory.ContinuousLattice.InverseLimits.instCompleteLattice
public import DomainTheory.ContinuousLattice.InverseLimits.coe_sSup_of_directed
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- The embedding `i_{n∞} : Dₙ → D_∞` as a bare function into the inverse limit. -/
def embInfFun (n : ℕ) (x : D n) : InverseLimit D P := ⟨iComp D P n x, iComp_compatible
    D P n x⟩

@[simp] theorem embInfFun_coe (n : ℕ) (x : D n) : (embInfFun D P n x).1 = iComp D P n
    x := rfl

theorem embInf_monotone (n : ℕ) : Monotone (embInfFun D P n) := by
  intro x x' hxx
  rw [← Subtype.coe_le_coe]
  intro m
  exact iComp_monotone D P n m hxx

theorem embInf_preservesDirectedSup (n : ℕ) : PreservesDirectedSup (embInfFun D P n)
    := by
  intro S hS hSdir
  have hTne : (embInfFun D P n '' S).Nonempty := hS.image _
  have hTdir : DirectedOn (· ≤ ·) (embInfFun D P n '' S) := by
    rintro _ ⟨a, ha, rfl⟩ _ ⟨b, hb, rfl⟩
    obtain ⟨c, hc, hac, hbc⟩ := hSdir a ha b hb
    exact ⟨embInfFun D P n c, Set.mem_image_of_mem _ hc,
      embInf_monotone D P n hac, embInf_monotone D P n hbc⟩
  apply Subtype.ext
  rw [coe_sSup_of_directed D P (embInfFun D P n '' S) hTne hTdir]
  funext m
  have heq : Function.eval m '' (Subtype.val '' (embInfFun D P n '' S))
      = (fun x => iComp D P n x m) '' S := by
    simp only [Set.image_image, embInfFun_coe, Function.eval]
  rw [sSup_apply_eq_sSup_image, heq]
```

```
  exact iComp_preservesDirectedSup D P n m hS hSdir

end InverseLimit

end DomainTheory.ContinuousLattice
```

3

embInfFun.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import DomainTheory.ContinuousLattice.InverseLimits.iComp
public import DomainTheory.ContinuousLattice.InverseLimits.instCompleteLattice
public import DomainTheory.ContinuousLattice.InverseLimits.coe_sSup_of_directed
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- The embedding `i_{n∞} : Dₙ → D_∞` as a bare function into the inverse limit. -/
def embInfFun (n : ℕ) (x : D n) : InverseLimit D P := ⟨iComp D P n x, iComp_compatible
    D P n x⟩

@[simp] theorem embInfFun_coe (n : ℕ) (x : D n) : (embInfFun D P n x).1 = iComp D P n
    x := rfl

theorem embInf_monotone (n : ℕ) : Monotone (embInfFun D P n) := by
  intro x x' hxx
  rw [← Subtype.coe_le_coe]
  intro m
  exact iComp_monotone D P n m hxx

theorem embInf_preservesDirectedSup (n : ℕ) : PreservesDirectedSup (embInfFun D P n)
    := by
  intro S hS hSdir
  have hTne : (embInfFun D P n '' S).Nonempty := hS.image _
  have hTdir : DirectedOn (· ≤ ·) (embInfFun D P n '' S) := by
    rintro _ ⟨a, ha, rfl⟩ _ ⟨b, hb, rfl⟩
    obtain ⟨c, hc, hac, hbc⟩ := hSdir a ha b hb
    exact ⟨embInfFun D P n c, Set.mem_image_of_mem _ hc,
      embInf_monotone D P n hac, embInf_monotone D P n hbc⟩
  apply Subtype.ext
```

```
  rw [coe_sSup_of_directed D P (embInfFun D P n '' S) hTne hTdir]
  funext m
  have heq : Function.eval m '' (Subtype.val '' (embInfFun D P n '' S))
      = (fun x => iComp D P n x m) '' S := by
    simp only [Set.image_image, embInfFun_coe, Function.eval]
  rw [sSup_apply_eq_sSup_image, heq]
  exact iComp_preservesDirectedSup D P n m hS hSdir

end InverseLimit

end DomainTheory.ContinuousLattice
```

4

embInfFun.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import DomainTheory.ContinuousLattice.InverseLimits.iComp
public import DomainTheory.ContinuousLattice.InverseLimits.instCompleteLattice
public import DomainTheory.ContinuousLattice.InverseLimits.coe_sSup_of_directed
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- The embedding `i_{n∞} : D_n → D_∞` as a bare function into the inverse limit. -/
def embInfFun (n : ℕ) (x : D n) : InverseLimit D P := ⟨iComp D P n x, iComp_compatible
    D P n x⟩

@[simp] theorem embInfFun_coe (n : ℕ) (x : D n) : (embInfFun D P n x).1 = iComp D P n
    x := rfl

theorem embInf_monotone (n : ℕ) : Monotone (embInfFun D P n) := by
  intro x x' hxx
  rw [← Subtype.coe_le_coe]
  intro m
  exact iComp_monotone D P n m hxx

theorem embInf_preservesDirectedSup (n : ℕ) : PreservesDirectedSup (embInfFun D P n)
    := by
  intro S hS hSdir
  have hTne : (embInfFun D P n '' S).Nonempty := hS.image _
```

```
  have hTdir : DirectedOn (· ≤ ·) (embInfFun D P n '' S) := by
    rintro _ ⟨a, ha, rfl⟩ _ ⟨b, hb, rfl⟩
    obtain ⟨c, hc, hac, hbc⟩ := hSdir a ha b hb
    exact ⟨embInfFun D P n c, Set.mem_image_of_mem _ hc,
      embInf_monotone D P n hac, embInf_monotone D P n hbc⟩
  apply Subtype.ext
  rw [coe_sSup_of_directed D P (embInfFun D P n '' S) hTne hTdir]
  funext m
  have heq : Function.eval m '' (Subtype.val '' (embInfFun D P n '' S))
      = (fun x => iComp D P n x m) '' S := by
    simp only [Set.image_image, embInfFun_coe, Function.eval]
  rw [sSup_apply_eq_sSup_image, heq]
  exact iComp_preservesDirectedSup D P n m hS hSdir

end InverseLimit

end DomainTheory.ContinuousLattice
```

5

embInfFun.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import DomainTheory.ContinuousLattice.InverseLimits.iComp
public import DomainTheory.ContinuousLattice.InverseLimits.instCompleteLattice
public import DomainTheory.ContinuousLattice.InverseLimits.coe_sSup_of_directed
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- The embedding `i_{n∞} : D_n → D_∞` as a bare function into the inverse limit. -/
def embInfFun (n : ℕ) (x : D n) : InverseLimit D P := ⟨iComp D P n x, iComp_compatible
    D P n x⟩

@[simp] theorem embInfFun_coe (n : ℕ) (x : D n) : (embInfFun D P n x).1 = iComp D P n
    x := rfl

theorem embInf_monotone (n : ℕ) : Monotone (embInfFun D P n) := by
  intro x x' hxx
  rw [← Subtype.coe_le_coe]
  intro m
```

```
  exact iComp_monotone D P n m hxx

theorem embInf_preservesDirectedSup (n : ℕ) : PreservesDirectedSup (embInfFun D P n)
    := by
  intro S hS hSdir
  have hTne : (embInfFun D P n '' S).Nonempty := hS.image _
  have hTdir : DirectedOn (· ≤ ·) (embInfFun D P n '' S) := by
    rintro _ ⟨a, ha, rfl⟩ _ ⟨b, hb, rfl⟩
    obtain ⟨c, hc, hac, hbc⟩ := hSdir a ha b hb
    exact ⟨embInfFun D P n c, Set.mem_image_of_mem _ hc,
      embInf_monotone D P n hac, embInf_monotone D P n hbc⟩
  apply Subtype.ext
  rw [coe_sSup_of_directed D P (embInfFun D P n '' S) hTne hTdir]
  funext m
  have heq : Function.eval m '' (Subtype.val '' (embInfFun D P n '' S))
      = (fun x => iComp D P n x m) '' S := by
    simp only [Set.image_image, embInfFun_coe, Function.eval]
  rw [sSup_apply_eq_sSup_image, heq]
  exact iComp_preservesDirectedSup D P n m hS hSdir

end InverseLimit

end DomainTheory.ContinuousLattice
```

6

embInfFun.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import DomainTheory.ContinuousLattice.InverseLimits.iComp
public import DomainTheory.ContinuousLattice.InverseLimits.instCompleteLattice
public import DomainTheory.ContinuousLattice.InverseLimits.coe_sSup_of_directed
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- The embedding `i_{n∞} : Dₙ → D_∞` as a bare function into the inverse limit. -/
def embInfFun (n : ℕ) (x : D n) : InverseLimit D P := ⟨iComp D P n x, iComp_compatible
    D P n x⟩

@[simp] theorem embInfFun_coe (n : ℕ) (x : D n) : (embInfFun D P n x).1 = iComp D P n
```

```
    x := rfl

theorem embInf_monotone (n : ℕ) : Monotone (embInfFun D P n) := by
  intro x x' hxx
  rw [← Subtype.coe_le_coe]
  intro m
  exact iComp_monotone D P n m hxx

theorem embInf_preservesDirectedSup (n : ℕ) : PreservesDirectedSup (embInfFun D P n)
    := by
  intro S hS hSdir
  have hTne : (embInfFun D P n '' S).Nonempty := hS.image _
  have hTdir : DirectedOn (· ≤ ·) (embInfFun D P n '' S) := by
    rintro _ ⟨a, ha, rfl⟩ _ ⟨b, hb, rfl⟩
    obtain ⟨c, hc, hac, hbc⟩ := hSdir a ha b hb
    exact ⟨embInfFun D P n c, Set.mem_image_of_mem _ hc,
      embInf_monotone D P n hac, embInf_monotone D P n hbc⟩
  apply Subtype.ext
  rw [coe_sSup_of_directed D P (embInfFun D P n '' S) hTne hTdir]
  funext m
  have heq : Function.eval m '' (Subtype.val '' (embInfFun D P n '' S))
      = (fun x => iComp D P n x m) '' S := by
    simp only [Set.image_image, embInfFun_coe, Function.eval]
  rw [sSup_apply_eq_sSup_image, heq]
  exact iComp_preservesDirectedSup D P n m hS hSdir

end InverseLimit

end DomainTheory.ContinuousLattice
```

7

embInfFun.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import DomainTheory.ContinuousLattice.InverseLimits.iComp
public import DomainTheory.ContinuousLattice.InverseLimits.instCompleteLattice
public import DomainTheory.ContinuousLattice.InverseLimits.coe_sSup_of_directed
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))
```

```
/-- The embedding `i_{n∞} : D_n → D_∞` as a bare function into the inverse limit. -/
def embInfFun (n : ℕ) (x : D n) : InverseLimit D P := ⟨iComp D P n x, iComp_compatible
    D P n x⟩

@[simp] theorem embInfFun_coe (n : ℕ) (x : D n) : (embInfFun D P n x).1 = iComp D P n
    x := rfl

theorem embInf_monotone (n : ℕ) : Monotone (embInfFun D P n) := by
  intro x x' hxx
  rw [← Subtype.coe_le_coe]
  intro m
  exact iComp_monotone D P n m hxx

theorem embInf_preservesDirectedSup (n : ℕ) : PreservesDirectedSup (embInfFun D P n)
    := by
  intro S hS hSdir
  have hTne : (embInfFun D P n '' S).Nonempty := hS.image _
  have hTdir : DirectedOn (· ≤ ·) (embInfFun D P n '' S) := by
    rintro _ ⟨a, ha, rfl⟩ _ ⟨b, hb, rfl⟩
    obtain ⟨c, hc, hac, hbc⟩ := hSdir a ha b hb
    exact ⟨embInfFun D P n c, Set.mem_image_of_mem _ hc,
      embInf_monotone D P n hac, embInf_monotone D P n hbc⟩
  apply Subtype.ext
  rw [coe_sSup_of_directed D P (embInfFun D P n '' S) hTne hTdir]
  funext m
  have heq : Function.eval m '' (Subtype.val '' (embInfFun D P n '' S))
      = (fun x => iComp D P n x m) '' S := by
    simp only [Set.image_image, embInfFun_coe, Function.eval]
  rw [sSup_apply_eq_sSup_image, heq]
  exact iComp_preservesDirectedSup D P n m hS hSdir

end InverseLimit

end DomainTheory.ContinuousLattice
```

8

embInfFun.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import DomainTheory.ContinuousLattice.InverseLimits.iComp
public import DomainTheory.ContinuousLattice.InverseLimits.instCompleteLattice
public import DomainTheory.ContinuousLattice.InverseLimits.coe_sSup_of_directed
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u
```

```
section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- The embedding `i_{n∞} : Dₙ → D_∞` as a bare function into the inverse limit. -/
def embInfFun (n : ℕ) (x : D n) : InverseLimit D P := ⟨iComp D P n x, iComp_compatible
    D P n x⟩

@[simp] theorem embInfFun_coe (n : ℕ) (x : D n) : (embInfFun D P n x).1 = iComp D P n
    x := rfl

theorem embInf_monotone (n : ℕ) : Monotone (embInfFun D P n) := by
  intro x x' hxx
  rw [← Subtype.coe_le_coe]
  intro m
  exact iComp_monotone D P n m hxx

theorem embInf_preservesDirectedSup (n : ℕ) : PreservesDirectedSup (embInfFun D P n)
    := by
  intro S hS hSdir
  have hTne : (embInfFun D P n '' S).Nonempty := hS.image _
  have hTdir : DirectedOn (· ≤ ·) (embInfFun D P n '' S) := by
    rintro _ ⟨a, ha, rfl⟩ _ ⟨b, hb, rfl⟩
    obtain ⟨c, hc, hac, hbc⟩ := hSdir a ha b hb
    exact ⟨embInfFun D P n c, Set.mem_image_of_mem _ hc,
      embInf_monotone D P n hac, embInf_monotone D P n hbc⟩
  apply Subtype.ext
  rw [coe_sSup_of_directed D P (embInfFun D P n '' S) hTne hTdir]
  funext m
  have heq : Function.eval m '' (Subtype.val '' (embInfFun D P n '' S))
      = (fun x => iComp D P n x m) '' S := by
    simp only [Set.image_image, embInfFun_coe, Function.eval]
  rw [sSup_apply_eq_sSup_image, heq]
  exact iComp_preservesDirectedSup D P n m hS hSdir

end InverseLimit

end DomainTheory.ContinuousLattice
```

9

embInf.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.instCompleteLattice
public import DomainTheory.ContinuousLattice.InverseLimits.embInfFun
public import DomainTheory.ContinuousLattice.InverseLimits.embLE
public import DomainTheory.ContinuousLattice.InverseLimits.iComp
public import DomainTheory.ContinuousLattice.InverseLimits.projLE
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Data.Set.Basic
```

```
@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- The embedding `i_{n∞} : D_n → D_∞`, Scott-continuous. -/
noncomputable def embInf (n : ℕ) : ScottMap (D n) (InverseLimit D P) :=
  ⟨embInfFun D P n, continuous_of_preservesDirectedSup (embInf_preservesDirectedSup D P
     n)⟩

/-- **Scott 1972, §4 (recursion equation).** `i_{n∞} = i_{(n+1)∞} ∘ i_n`. -/
theorem embInf_succ (n : ℕ) (x : D n) :
    embInf D P (n + 1) ((P n).incl x) = embInf D P n x := by
  apply Subtype.ext
  funext m
  show iComp D P (n + 1) ((P n).incl x) m = iComp D P n x m
  by_cases h1 : n ≤ m
  · by_cases h2 : n + 1 ≤ m
    · rw [iComp_of_le D P h2, iComp_of_le D P h1, embLE_succ_left D P h1 h2]
    · have hmn : m = n := le_antisymm (Nat.lt_succ_iff.mp (not_le.mp h2)) h1
      subst hmn
      rw [iComp_self, iComp_of_ge D P h2,
        projLE.succ D P (le_refl m) (le_of_lt (not_le.mp h2)) ((P m).incl x),
        projLE.self, (P m).retr_incl]
  · have h2 : ¬ n + 1 ≤ m := fun h => h1 (le_trans (Nat.le_succ n) h)
    rw [iComp_of_ge D P h2, iComp_of_ge D P h1,
      projLE.succ D P (le_of_lt (not_le.mp h1)) (le_of_lt (not_le.mp h2)) ((P n).incl
    x),
      (P n).retr_incl]

/-- The family `n ↦ i_{n∞}(x_n)` is monotone (the lub defining `x` is monotone). -/
theorem embInf_le_succ (x : InverseLimit D P) (n : ℕ) :
    embInf D P n (x.1 n) ≤ embInf D P (n + 1) (x.1 (n + 1)) := by
  rw [← embInf_succ D P n (x.1 n)]
  exact embInf_monotone D P (n + 1)
    (by calc (P n).incl (x.1 n) = (P n).incl ((P n).retr (x.1 (n + 1))) := by rw [x.2
    n]
          _ ≤ x.1 (n + 1) := (P n).incl_retr_le _)

theorem embInf_family_directed (x : InverseLimit D P) :
    DirectedOn (· ≤ ·) (Set.range (fun n => embInf D P n (x.1 n))) :=
  directedOn_range.2 (monotone_nat_of_le_succ (embInf_le_succ D P x)).directed_le

end InverseLimit

end DomainTheory.ContinuousLattice
```

embInf.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.instCompleteLattice
public import DomainTheory.ContinuousLattice.InverseLimits.embInfFun
public import DomainTheory.ContinuousLattice.InverseLimits.embLE
public import DomainTheory.ContinuousLattice.InverseLimits.iComp
public import DomainTheory.ContinuousLattice.InverseLimits.projLE
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- The embedding `i_{n∞} : D_n → D_∞`, Scott-continuous. -/
noncomputable def embInf (n : ℕ) : ScottMap (D n) (InverseLimit D P) :=
  ⟨embInfFun D P n, continuous_of_preservesDirectedSup (embInf_preservesDirectedSup D P
     n)⟩

/-- **Scott 1972, §4 (recursion equation).** `i_{n∞} = i_{(n+1)∞} ∘ i_n`. -/
theorem embInf_succ (n : ℕ) (x : D n) :
    embInf D P (n + 1) ((P n).incl x) = embInf D P n x := by
  apply Subtype.ext
  funext m
  show iComp D P (n + 1) ((P n).incl x) m = iComp D P n x m
  by_cases h1 : n ≤ m
  · by_cases h2 : n + 1 ≤ m
    · rw [iComp_of_le D P h2, iComp_of_le D P h1, embLE_succ_left D P h1 h2]
    · have hmn : m = n := le_antisymm (Nat.lt_succ_iff.mp (not_le.mp h2)) h1
      subst hmn
      rw [iComp_self, iComp_of_ge D P h2,
        projLE.succ D P (le_refl m) (le_of_lt (not_le.mp h2)) ((P m).incl x),
        projLE.self, (P m).retr_incl]
  · have h2 : ¬ n + 1 ≤ m := fun h => h1 (le_trans (Nat.le_succ n) h)
    rw [iComp_of_ge D P h2, iComp_of_ge D P h1,
      projLE.succ D P (le_of_lt (not_le.mp h1)) (le_of_lt (not_le.mp h2)) ((P n).incl
    x),
      (P n).retr_incl]

/-- The family `n ↦ i_{n∞}(x_n)` is monotone (the lub defining `x` is monotone). -/
theorem embInf_le_succ (x : InverseLimit D P) (n : ℕ) :
```

```
    embInf D P n (x.1 n) ≤ embInf D P (n + 1) (x.1 (n + 1)) := by
  rw [← embInf_succ D P n (x.1 n)]
  exact embInf_monotone D P (n + 1)
    (by calc (P n).incl (x.1 n) = (P n).incl ((P n).retr (x.1 (n + 1))) := by rw [x.2
    n]
          _ ≤ x.1 (n + 1) := (P n).incl_retr_le _)

theorem embInf_family_directed (x : InverseLimit D P) :
    DirectedOn (· ≤ ·) (Set.range (fun n => embInf D P n (x.1 n))) :=
  directedOn_range.2 (monotone_nat_of_le_succ (embInf_le_succ D P x)).directed_le

end InverseLimit

end DomainTheory.ContinuousLattice
```

1

embInf.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.instCompleteLattice
public import DomainTheory.ContinuousLattice.InverseLimits.embInfFun
public import DomainTheory.ContinuousLattice.InverseLimits.embLE
public import DomainTheory.ContinuousLattice.InverseLimits.iComp
public import DomainTheory.ContinuousLattice.InverseLimits.projLE
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- The embedding `i_{n∞} : D_n → D_∞`, Scott-continuous. -/
noncomputable def embInf (n : ℕ) : ScottMap (D n) (InverseLimit D P) :=
  ⟨embInfFun D P n, continuous_of_preservesDirectedSup (embInf_preservesDirectedSup D P
    n)⟩

/-- **Scott 1972, §4 (recursion equation).** `i_{n∞} = i_{(n+1)∞} ∘ i_n`. -/
theorem embInf_succ (n : ℕ) (x : D n) :
    embInf D P (n + 1) ((P n).incl x) = embInf D P n x := by
  apply Subtype.ext
  funext m
  show iComp D P (n + 1) ((P n).incl x) m = iComp D P n x m
  by_cases h1 : n ≤ m
```

```
  · by_cases h2 : n + 1 ≤ m
    · rw [iComp_of_le D P h2, iComp_of_le D P h1, embLE_succ_left D P h1 h2]
    · have hmn : m = n := le_antisymm (Nat.lt_succ_iff.mp (not_le.mp h2)) h1
      subst hmn
      rw [iComp_self, iComp_of_ge D P h2,
        projLE.succ D P (le_refl m) (le_of_lt (not_le.mp h2)) ((P m).incl x),
        projLE.self, (P m).retr_incl]
  · have h2 : ¬ n + 1 ≤ m := fun h => h1 (le_trans (Nat.le_succ n) h)
    rw [iComp_of_ge D P h2, iComp_of_ge D P h1,
      projLE.succ D P (le_of_lt (not_le.mp h1)) (le_of_lt (not_le.mp h2)) ((P n).incl
    x),
      (P n).retr_incl]

/-- The family `n ↦ i_{n∞}(x_n)` is monotone (the lub defining `x` is monotone). -/
theorem embInf_le_succ (x : InverseLimit D P) (n : ℕ) :
    embInf D P n (x.1 n) ≤ embInf D P (n + 1) (x.1 (n + 1)) := by
  rw [← embInf_succ D P n (x.1 n)]
  exact embInf_monotone D P (n + 1)
    (by calc (P n).incl (x.1 n) = (P n).incl ((P n).retr (x.1 (n + 1))) := by rw [x.2
    n]
          _ ≤ x.1 (n + 1) := (P n).incl_retr_le _)

theorem embInf_family_directed (x : InverseLimit D P) :
    DirectedOn (· ≤ ·) (Set.range (fun n => embInf D P n (x.1 n))) :=
  directedOn_range.2 (monotone_nat_of_le_succ (embInf_le_succ D P x)).directed_le

end InverseLimit

end DomainTheory.ContinuousLattice
```

2

embInf.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.instCompleteLattice
public import DomainTheory.ContinuousLattice.InverseLimits.embInfFun
public import DomainTheory.ContinuousLattice.InverseLimits.embLE
public import DomainTheory.ContinuousLattice.InverseLimits.iComp
public import DomainTheory.ContinuousLattice.InverseLimits.projLE
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]
```

```
variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- The embedding `i_{n∞} : D_n → D_∞`, Scott-continuous. -/
noncomputable def embInf (n : ℕ) : ScottMap (D n) (InverseLimit D P) :=
  ⟨embInfFun D P n, continuous_of_preservesDirectedSup (embInf_preservesDirectedSup D P
    n)⟩

/-- **Scott 1972, §4 (recursion equation).** `i_{n∞} = i_{(n+1)∞} ∘ i_n`. -/
theorem embInf_succ (n : ℕ) (x : D n) :
    embInf D P (n + 1) ((P n).incl x) = embInf D P n x := by
  apply Subtype.ext
  funext m
  show iComp D P (n + 1) ((P n).incl x) m = iComp D P n x m
  by_cases h1 : n ≤ m
  · by_cases h2 : n + 1 ≤ m
    · rw [iComp_of_le D P h2, iComp_of_le D P h1, embLE_succ_left D P h1 h2]
    · have hmn : m = n := le_antisymm (Nat.lt_succ_iff.mp (not_le.mp h2)) h1
      subst hmn
      rw [iComp_self, iComp_of_ge D P h2,
        projLE.succ D P (le_refl m) (le_of_lt (not_le.mp h2)) ((P m).incl x),
        projLE.self, (P m).retr_incl]
  · have h2 : ¬ n + 1 ≤ m := fun h => h1 (le_trans (Nat.le_succ n) h)
    rw [iComp_of_ge D P h2, iComp_of_ge D P h1,
      projLE.succ D P (le_of_lt (not_le.mp h1)) (le_of_lt (not_le.mp h2)) ((P n).incl
    x),
      (P n).retr_incl]

/-- The family `n ↦ i_{n∞}(x_n)` is monotone (the lub defining `x` is monotone). -/
theorem embInf_le_succ (x : InverseLimit D P) (n : ℕ) :
    embInf D P n (x.1 n) ≤ embInf D P (n + 1) (x.1 (n + 1)) := by
  rw [← embInf_succ D P n (x.1 n)]
  exact embInf_monotone D P (n + 1)
    (by calc (P n).incl (x.1 n) = (P n).incl ((P n).retr (x.1 (n + 1))) := by rw [x.2
    n]
          _ ≤ x.1 (n + 1) := (P n).incl_retr_le _)

theorem embInf_family_directed (x : InverseLimit D P) :
    DirectedOn (· ≤ ·) (Set.range (fun n => embInf D P n (x.1 n))) :=
  directedOn_range.2 (monotone_nat_of_le_succ (embInf_le_succ D P x)).directed_le

end InverseLimit

end DomainTheory.ContinuousLattice
```

3

embInf.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.instCompleteLattice
public import DomainTheory.ContinuousLattice.InverseLimits.embInfFun
public import DomainTheory.ContinuousLattice.InverseLimits.embLE
public import DomainTheory.ContinuousLattice.InverseLimits.iComp
public import DomainTheory.ContinuousLattice.InverseLimits.projLE
```

```
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- The embedding `i_{n∞} : D_n → D_∞`, Scott-continuous. -/
noncomputable def embInf (n : ℕ) : ScottMap (D n) (InverseLimit D P) :=
  ⟨embInfFun D P n, continuous_of_preservesDirectedSup (embInf_preservesDirectedSup D P
     n)⟩

/-- **Scott 1972, §4 (recursion equation).** `i_{n∞} = i_{(n+1)∞} ∘ i_n`. -/
theorem embInf_succ (n : ℕ) (x : D n) :
    embInf D P (n + 1) ((P n).incl x) = embInf D P n x := by
  apply Subtype.ext
  funext m
  show iComp D P (n + 1) ((P n).incl x) m = iComp D P n x m
  by_cases h1 : n ≤ m
  · by_cases h2 : n + 1 ≤ m
    · rw [iComp_of_le D P h2, iComp_of_le D P h1, embLE_succ_left D P h1 h2]
    · have hmn : m = n := le_antisymm (Nat.lt_succ_iff.mp (not_le.mp h2)) h1
      subst hmn
      rw [iComp_self, iComp_of_ge D P h2,
        projLE.succ D P (le_refl m) (le_of_lt (not_le.mp h2)) ((P m).incl x),
        projLE.self, (P m).retr_incl]
  · have h2 : ¬ n + 1 ≤ m := fun h => h1 (le_trans (Nat.le_succ n) h)
    rw [iComp_of_ge D P h2, iComp_of_ge D P h1,
      projLE.succ D P (le_of_lt (not_le.mp h1)) (le_of_lt (not_le.mp h2)) ((P n).incl
    x),
      (P n).retr_incl]

/-- The family `n ↦ i_{n∞}(x_n)` is monotone (the lub defining `x` is monotone). -/
theorem embInf_le_succ (x : InverseLimit D P) (n : ℕ) :
    embInf D P n (x.1 n) ≤ embInf D P (n + 1) (x.1 (n + 1)) := by
  rw [← embInf_succ D P n (x.1 n)]
  exact embInf_monotone D P (n + 1)
    (by calc (P n).incl (x.1 n) = (P n).incl ((P n).retr (x.1 (n + 1))) := by rw [x.2
    n]
          _ ≤ x.1 (n + 1) := (P n).incl_retr_le _)

theorem embInf_family_directed (x : InverseLimit D P) :
    DirectedOn (· ≤ ·) (Set.range (fun n => embInf D P n (x.1 n))) :=
  directedOn_range.2 (monotone_nat_of_le_succ (embInf_le_succ D P x)).directed_le
```

```
end InverseLimit

end DomainTheory.ContinuousLattice
```

4

embInf.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.instCompleteLattice
public import DomainTheory.ContinuousLattice.InverseLimits.embInfFun
public import DomainTheory.ContinuousLattice.InverseLimits.embLE
public import DomainTheory.ContinuousLattice.InverseLimits.iComp
public import DomainTheory.ContinuousLattice.InverseLimits.projLE
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- The embedding `i_{n∞} : Dₙ → D_∞`, Scott-continuous. -/
noncomputable def embInf (n : ℕ) : ScottMap (D n) (InverseLimit D P) :=
  ⟨embInfFun D P n, continuous_of_preservesDirectedSup (embInf_preservesDirectedSup D P
     n)⟩

/-- **Scott 1972, §4 (recursion equation).** `i_{n∞} = i_{(n+1)∞} ∘ iₙ`. -/
theorem embInf_succ (n : ℕ) (x : D n) :
    embInf D P (n + 1) ((P n).incl x) = embInf D P n x := by
  apply Subtype.ext
  funext m
  show iComp D P (n + 1) ((P n).incl x) m = iComp D P n x m
  by_cases h1 : n ≤ m
  · by_cases h2 : n + 1 ≤ m
    · rw [iComp_of_le D P h2, iComp_of_le D P h1, embLE_succ_left D P h1 h2]
    · have hmn : m = n := le_antisymm (Nat.lt_succ_iff.mp (not_le.mp h2)) h1
      subst hmn
      rw [iComp_self, iComp_of_ge D P h2,
        projLE.succ D P (le_refl m) (le_of_lt (not_le.mp h2)) ((P m).incl x),
        projLE.self, (P m).retr_incl]
  · have h2 : ¬ n + 1 ≤ m := fun h => h1 (le_trans (Nat.le_succ n) h)
    rw [iComp_of_ge D P h2, iComp_of_ge D P h1,
      projLE.succ D P (le_of_lt (not_le.mp h1)) (le_of_lt (not_le.mp h2)) ((P n).incl
    x),
```

```
      (P n).retr_incl]

/-- The family `n ↦ i_{n∞}(x_n)` is monotone (the lub defining `x` is monotone). -/
theorem embInf_le_succ (x : InverseLimit D P) (n : ℕ) :
    embInf D P n (x.1 n) ≤ embInf D P (n + 1) (x.1 (n + 1)) := by
  rw [← embInf_succ D P n (x.1 n)]
  exact embInf_monotone D P (n + 1)
    (by calc (P n).incl (x.1 n) = (P n).incl ((P n).retr (x.1 (n + 1))) := by rw [x.2
    n]
          _ ≤ x.1 (n + 1) := (P n).incl_retr_le _)

theorem embInf_family_directed (x : InverseLimit D P) :
    DirectedOn (· ≤ ·) (Set.range (fun n => embInf D P n (x.1 n))) :=
  directedOn_range.2 (monotone_nat_of_le_succ (embInf_le_succ D P x)).directed_le

end InverseLimit

end DomainTheory.ContinuousLattice
```

5

embInf.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.instCompleteLattice
public import DomainTheory.ContinuousLattice.InverseLimits.embInfFun
public import DomainTheory.ContinuousLattice.InverseLimits.embLE
public import DomainTheory.ContinuousLattice.InverseLimits.iComp
public import DomainTheory.ContinuousLattice.InverseLimits.projLE
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- The embedding `i_{n∞} : D_n → D_∞`, Scott-continuous. -/
noncomputable def embInf (n : ℕ) : ScottMap (D n) (InverseLimit D P) :=
  ⟨embInfFun D P n, continuous_of_preservesDirectedSup (embInf_preservesDirectedSup D P
    n)⟩

/-- **Scott 1972, §4 (recursion equation).** `i_{n∞} = i_{(n+1)∞} ∘ i_n`. -/
theorem embInf_succ (n : ℕ) (x : D n) :
    embInf D P (n + 1) ((P n).incl x) = embInf D P n x := by
```

```
  apply Subtype.ext
  funext m
  show iComp D P (n + 1) ((P n).incl x) m = iComp D P n x m
  by_cases h1 : n ≤ m
  · by_cases h2 : n + 1 ≤ m
    · rw [iComp_of_le D P h2, iComp_of_le D P h1, embLE_succ_left D P h1 h2]
    · have hmn : m = n := le_antisymm (Nat.lt_succ_iff.mp (not_le.mp h2)) h1
      subst hmn
      rw [iComp_self, iComp_of_ge D P h2,
        projLE.succ D P (le_refl m) (le_of_lt (not_le.mp h2)) ((P m).incl x),
        projLE.self, (P m).retr_incl]
  · have h2 : ¬ n + 1 ≤ m := fun h => h1 (le_trans (Nat.le_succ n) h)
    rw [iComp_of_ge D P h2, iComp_of_ge D P h1,
      projLE.succ D P (le_of_lt (not_le.mp h1)) (le_of_lt (not_le.mp h2)) ((P n).incl
    x),
      (P n).retr_incl]

/-- The family `n ↦ i_{n∞}(x_n)` is monotone (the lub defining `x` is monotone). -/
theorem embInf_le_succ (x : InverseLimit D P) (n : ℕ) :
    embInf D P n (x.1 n) ≤ embInf D P (n + 1) (x.1 (n + 1)) := by
  rw [← embInf_succ D P n (x.1 n)]
  exact embInf_monotone D P (n + 1)
    (by calc (P n).incl (x.1 n) = (P n).incl ((P n).retr (x.1 (n + 1))) := by rw [x.2
    n]
          _ ≤ x.1 (n + 1) := (P n).incl_retr_le _)

theorem embInf_family_directed (x : InverseLimit D P) :
    DirectedOn (· ≤ ·) (Set.range (fun n => embInf D P n (x.1 n))) :=
  directedOn_range.2 (monotone_nat_of_le_succ (embInf_le_succ D P x)).directed_le

end InverseLimit

end DomainTheory.ContinuousLattice
```

6

embInf.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.instCompleteLattice
public import DomainTheory.ContinuousLattice.InverseLimits.embInfFun
public import DomainTheory.ContinuousLattice.InverseLimits.embLE
public import DomainTheory.ContinuousLattice.InverseLimits.iComp
public import DomainTheory.ContinuousLattice.InverseLimits.projLE
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u
```

```
section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- The embedding `i_{n∞} : Dₙ → D_∞`, Scott-continuous. -/
noncomputable def embInf (n : ℕ) : ScottMap (D n) (InverseLimit D P) :=
  ⟨embInfFun D P n, continuous_of_preservesDirectedSup (embInf_preservesDirectedSup D P
     n)⟩

/-- **Scott 1972, §4 (recursion equation).** `i_{n∞} = i_{(n+1)∞} ∘ iₙ`. -/
theorem embInf_succ (n : ℕ) (x : D n) :
    embInf D P (n + 1) ((P n).incl x) = embInf D P n x := by
  apply Subtype.ext
  funext m
  show iComp D P (n + 1) ((P n).incl x) m = iComp D P n x m
  by_cases h1 : n ≤ m
  · by_cases h2 : n + 1 ≤ m
    · rw [iComp_of_le D P h2, iComp_of_le D P h1, embLE_succ_left D P h1 h2]
    · have hmn : m = n := le_antisymm (Nat.lt_succ_iff.mp (not_le.mp h2)) h1
      subst hmn
      rw [iComp_self, iComp_of_ge D P h2,
        projLE.succ D P (le_refl m) (le_of_lt (not_le.mp h2)) ((P m).incl x),
        projLE.self, (P m).retr_incl]
  · have h2 : ¬ n + 1 ≤ m := fun h => h1 (le_trans (Nat.le_succ n) h)
    rw [iComp_of_ge D P h2, iComp_of_ge D P h1,
      projLE.succ D P (le_of_lt (not_le.mp h1)) (le_of_lt (not_le.mp h2)) ((P n).incl
    x),
      (P n).retr_incl]

/-- The family `n ↦ i_{n∞}(xₙ)` is monotone (the lub defining `x` is monotone). -/
theorem embInf_le_succ (x : InverseLimit D P) (n : ℕ) :
    embInf D P n (x.1 n) ≤ embInf D P (n + 1) (x.1 (n + 1)) := by
  rw [← embInf_succ D P n (x.1 n)]
  exact embInf_monotone D P (n + 1)
    (by calc (P n).incl (x.1 n) = (P n).incl ((P n).retr (x.1 (n + 1))) := by rw [x.2
    n]
          _ ≤ x.1 (n + 1) := (P n).incl_retr_le _)

theorem embInf_family_directed (x : InverseLimit D P) :
    DirectedOn (· ≤ ·) (Set.range (fun n => embInf D P n (x.1 n))) :=
  directedOn_range.2 (monotone_nat_of_le_succ (embInf_le_succ D P x)).directed_le

end InverseLimit

end DomainTheory.ContinuousLattice
```

7

embInf.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.instCompleteLattice
```

```
public import DomainTheory.ContinuousLattice.InverseLimits.embInfFun
public import DomainTheory.ContinuousLattice.InverseLimits.embLE
public import DomainTheory.ContinuousLattice.InverseLimits.iComp
public import DomainTheory.ContinuousLattice.InverseLimits.projLE
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- The embedding `i_{n∞} : D_n → D_∞`, Scott-continuous. -/
noncomputable def embInf (n : ℕ) : ScottMap (D n) (InverseLimit D P) :=
  ⟨embInfFun D P n, continuous_of_preservesDirectedSup (embInf_preservesDirectedSup D P
     n)⟩

/-- **Scott 1972, §4 (recursion equation).** `i_{n∞} = i_{(n+1)∞} ∘ i_n`. -/
theorem embInf_succ (n : ℕ) (x : D n) :
    embInf D P (n + 1) ((P n).incl x) = embInf D P n x := by
  apply Subtype.ext
  funext m
  show iComp D P (n + 1) ((P n).incl x) m = iComp D P n x m
  by_cases h1 : n ≤ m
  · by_cases h2 : n + 1 ≤ m
    · rw [iComp_of_le D P h2, iComp_of_le D P h1, embLE_succ_left D P h1 h2]
    · have hmn : m = n := le_antisymm (Nat.lt_succ_iff.mp (not_le.mp h2)) h1
      subst hmn
      rw [iComp_self, iComp_of_ge D P h2,
        projLE.succ D P (le_refl m) (le_of_lt (not_le.mp h2)) ((P m).incl x),
        projLE.self, (P m).retr_incl]
  · have h2 : ¬ n + 1 ≤ m := fun h => h1 (le_trans (Nat.le_succ n) h)
    rw [iComp_of_ge D P h2, iComp_of_ge D P h1,
      projLE.succ D P (le_of_lt (not_le.mp h1)) (le_of_lt (not_le.mp h2)) ((P n).incl
    x),
      (P n).retr_incl]

/-- The family `n ↦ i_{n∞}(x_n)` is monotone (the lub defining `x` is monotone). -/
theorem embInf_le_succ (x : InverseLimit D P) (n : ℕ) :
    embInf D P n (x.1 n) ≤ embInf D P (n + 1) (x.1 (n + 1)) := by
  rw [← embInf_succ D P n (x.1 n)]
  exact embInf_monotone D P (n + 1)
    (by calc (P n).incl (x.1 n) = (P n).incl ((P n).retr (x.1 (n + 1))) := by rw [x.2
    n]
          _ ≤ x.1 (n + 1) := (P n).incl_retr_le _)
```

```
theorem embInf_family_directed (x : InverseLimit D P) :
    DirectedOn (· ≤ ·) (Set.range (fun n => embInf D P n (x.1 n))) :=
  directedOn_range.2 (monotone_nat_of_le_succ (embInf_le_succ D P x)).directed_le

end InverseLimit

end DomainTheory.ContinuousLattice
```

8

embInf.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.instCompleteLattice
public import DomainTheory.ContinuousLattice.InverseLimits.embInfFun
public import DomainTheory.ContinuousLattice.InverseLimits.embLE
public import DomainTheory.ContinuousLattice.InverseLimits.iComp
public import DomainTheory.ContinuousLattice.InverseLimits.projLE
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- The embedding `i_{n∞} : Dₙ → D_∞`, Scott-continuous. -/
noncomputable def embInf (n : ℕ) : ScottMap (D n) (InverseLimit D P) :=
  ⟨embInfFun D P n, continuous_of_preservesDirectedSup (embInf_preservesDirectedSup D P
    n)⟩

/-- **Scott 1972, §4 (recursion equation).** `i_{n∞} = i_{(n+1)∞} ∘ iₙ`. -/
theorem embInf_succ (n : ℕ) (x : D n) :
    embInf D P (n + 1) ((P n).incl x) = embInf D P n x := by
  apply Subtype.ext
  funext m
  show iComp D P (n + 1) ((P n).incl x) m = iComp D P n x m
  by_cases h1 : n ≤ m
  · by_cases h2 : n + 1 ≤ m
    · rw [iComp_of_le D P h2, iComp_of_le D P h1, embLE_succ_left D P h1 h2]
    · have hmn : m = n := le_antisymm (Nat.lt_succ_iff.mp (not_le.mp h2)) h1
      subst hmn
      rw [iComp_self, iComp_of_ge D P h2,
        projLE.succ D P (le_refl m) (le_of_lt (not_le.mp h2)) ((P m).incl x),
        projLE.self, (P m).retr_incl]
```

```
  · have h2 : ¬ n + 1 ≤ m := fun h => h1 (le_trans (Nat.le_succ n) h)
    rw [iComp_of_ge D P h2, iComp_of_ge D P h1,
      projLE.succ D P (le_of_lt (not_le.mp h1)) (le_of_lt (not_le.mp h2)) ((P n).incl
    x),
      (P n).retr_incl]

/-- The family `n ↦ i_{n∞}(x_n)` is monotone (the lub defining `x` is monotone). -/
theorem embInf_le_succ (x : InverseLimit D P) (n : ℕ) :
    embInf D P n (x.1 n) ≤ embInf D P (n + 1) (x.1 (n + 1)) := by
  rw [← embInf_succ D P n (x.1 n)]
  exact embInf_monotone D P (n + 1)
    (by calc (P n).incl (x.1 n) = (P n).incl ((P n).retr (x.1 (n + 1))) := by rw [x.2
    n]
          _ ≤ x.1 (n + 1) := (P n).incl_retr_le _)

theorem embInf_family_directed (x : InverseLimit D P) :
    DirectedOn (· ≤ ·) (Set.range (fun n => embInf D P n (x.1 n))) :=
  directedOn_range.2 (monotone_nat_of_le_succ (embInf_le_succ D P x)).directed_le

end InverseLimit

end DomainTheory.ContinuousLattice
```

9

coconeInf.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import DomainTheory.ContinuousLattice.InverseLimits.embLE
public import DomainTheory.ContinuousLattice.InverseLimits.projLE
public import DomainTheory.ContinuousLattice.InverseLimits.embInf
public import DomainTheory.ContinuousLattice.InverseLimits.iComp
public import DomainTheory.ContinuousLattice.InverseLimits.instCompleteLattice
public import DomainTheory.ContinuousLattice.InverseLimits.projInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

variable {D' : Type*} [CompleteLattice D']
```

```
/-- The mediating map of Corollary 4.3: `f∞(x) = ⋃ₙ fₙ(xₙ)`. -/
def coconeInf (f : ∀ n, ScottMap (D n) D') (x : InverseLimit D P) : D' :=
  ⋃ n, f n (x.1 n)

theorem coconeInf_apply (f : ∀ n, ScottMap (D n) D') (x : InverseLimit D P) :
    coconeInf D P f x = ⋃ n, f n (x.1 n) := rfl

/-- Climbing then applying `f` is constant: `f_m(i_{m-1}...iₙ x) = fₙ(x)`. -/
theorem coconeInf_climb (f : ∀ n, ScottMap (D n) D')
    (hf : ∀ n x, f n x = f (n + 1) ((P n).incl x)) {n : ℕ} (x : D n) :
    ∀ {m : ℕ} (h : n ≤ m), f m (embLE D P h x) = f n x := by
  intro m h
  induction m, h using Nat.le_induction with
  | base => rw [embLE_self]
  | succ k hk ih => rw [embLE_succ D P hk (Nat.le_succ_of_le hk), ← hf k, ih]

/-- Descending then applying `f` only decreases: `f_m(j_m...j_{n-1} x) ⊑ fₙ(x)`. -/
theorem coconeInf_descend (f : ∀ n, ScottMap (D n) D')
    (hf : ∀ n x, f n x = f (n + 1) ((P n).incl x)) {m : ℕ} :
    ∀ {n : ℕ} (h : m ≤ n) (x : D n), f m (projLE D P h x) ≤ f n x := by
  intro n h
  induction n, h using Nat.le_induction with
  | base => intro x; exact le_of_eq (congrArg (f m) (projLE.self D P (le_refl m) x))
  | succ k hk ih =>
      intro x
      rw [projLE.succ D P hk (Nat.le_succ_of_le hk)]
      calc f m (projLE D P hk ((P k).retr x)) ≤ f k ((P k).retr x) := ih ((P k).retr x
    )
        _ = f (k + 1) ((P k).incl ((P k).retr x)) := hf k _
        _ ≤ f (k + 1) x := (f (k + 1)).monotone ((P k).incl_retr_le x)

/-- **Scott 1972, Corollary 4.3 (factorization).** `fₙ = f∞ ∘ i_{n∞}`. -/
theorem coconeInf_comp_embInf (f : ∀ n, ScottMap (D n) D')
    (hf : ∀ n x, f n x = f (n + 1) ((P n).incl x)) (n : ℕ) (x : D n) :
    coconeInf D P f (embInf D P n x) = f n x := by
  apply le_antisymm
  · show (⋃ m, f m ((embInf D P n x).1 m)) ≤ f n x
    refine iSup_le fun m => ?_
    show f m (iComp D P n x m) ≤ f n x
    by_cases h : n ≤ m
    · rw [iComp_of_le D P h]; exact le_of_eq (coconeInf_climb D P f hf x h)
    · rw [iComp_of_ge D P h]; exact coconeInf_descend D P f hf (le_of_lt (not_le.mp h))
     x
  · refine le_trans (le_of_eq ?_) (le_iSup (fun m => f m ((embInf D P n x).1 m)) n)
    rw [show (embInf D P n x).1 n = x from iComp_self D P n x]

/-- `f∞` is Scott-continuous: `f∞(⊔S) = ⊔ f∞(S)` for directed `S` (each `fₙ` is
    continuous and a
double `⋃` over `ℕ × S` commutes). -/
theorem coconeInf_preservesDirectedSup (f : ∀ n, ScottMap (D n) D') :
    PreservesDirectedSup (coconeInf D P f) := by
  intro S hS hSdir
  have hev : ∀ n, DirectedOn (· ≤ ·) ((fun s : InverseLimit D P => s.1 n) '' S) := by
    intro n
    rintro _ ⟨a, ha, rfl⟩ _ ⟨b, hb, rfl⟩
```

```
    obtain ⟨c, hc, hac, hbc⟩ := hSdir a ha b hb
    exact ⟨c.1 n, ⟨c, hc, rfl⟩, hac n, hbc n⟩
  have key : ∀ n, f n ((sSup S).1 n) = sSup ((fun s : InverseLimit D P => f n (s.1 n))
    '' S) := by
    intro n
    have h1 : (sSup S).1 n = sSup ((fun s : InverseLimit D P => s.1 n) '' S) :=
      projInfFun.preservesDirectedSup D P n hS hSdir
    rw [h1, (f n).preservesDirectedSup_coe _ (hS.image _) (hev n), Set.image_image]
  show (⋃ n, f n ((sSup S).1 n)) = sSup (coconeInf D P f '' S)
  simp_rw [key, sSup_image', coconeInf_apply]
  rw [iSup_comm]

end InverseLimit

end DomainTheory.ContinuousLattice
```

0

coconeInf.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import DomainTheory.ContinuousLattice.InverseLimits.embLE
public import DomainTheory.ContinuousLattice.InverseLimits.projLE
public import DomainTheory.ContinuousLattice.InverseLimits.embInf
public import DomainTheory.ContinuousLattice.InverseLimits.iComp
public import DomainTheory.ContinuousLattice.InverseLimits.instCompleteLattice
public import DomainTheory.ContinuousLattice.InverseLimits.projInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

variable {D' : Type*} [CompleteLattice D']

/-- The mediating map of Corollary 4.3: `f∞(x) = ⋃ₙ fₙ(xₙ)`. -/
def coconeInf (f : ∀ n, ScottMap (D n) D') (x : InverseLimit D P) : D' :=
  ⋃ n, f n (x.1 n)

theorem coconeInf_apply (f : ∀ n, ScottMap (D n) D') (x : InverseLimit D P) :
    coconeInf D P f x = ⋃ n, f n (x.1 n) := rfl
```

```
/-- Climbing then applying `f` is constant: `f_m(i_{m-1}...i_n x) = f_n(x)`. -/
theorem coconeInf_climb (f : ∀ n, ScottMap (D n) D')
    (hf : ∀ n x, f n x = f (n + 1) ((P n).incl x)) {n : ℕ} (x : D n) :
    ∀ {m : ℕ} (h : n ≤ m), f m (embLE D P h x) = f n x := by
  intro m h
  induction m, h using Nat.le_induction with
  | base => rw [embLE_self]
  | succ k hk ih => rw [embLE_succ D P hk (Nat.le_succ_of_le hk), ← hf k, ih]

/-- Descending then applying `f` only decreases: `f_m(j_m...j_{n-1} x) ⊑ f_n(x)`. -/
theorem coconeInf_descend (f : ∀ n, ScottMap (D n) D')
    (hf : ∀ n x, f n x = f (n + 1) ((P n).incl x)) {m : ℕ} :
    ∀ {n : ℕ} (h : m ≤ n) (x : D n), f m (projLE D P h x) ≤ f n x := by
  intro n h
  induction n, h using Nat.le_induction with
  | base => intro x; exact le_of_eq (congrArg (f m) (projLE.self D P (le_refl m) x))
  | succ k hk ih =>
      intro x
      rw [projLE.succ D P hk (Nat.le_succ_of_le hk)]
      calc f m (projLE D P hk ((P k).retr x)) ≤ f k ((P k).retr x) := ih ((P k).retr x
    )
        _ = f (k + 1) ((P k).incl ((P k).retr x)) := hf k _
        _ ≤ f (k + 1) x := (f (k + 1)).monotone ((P k).incl_retr_le x)

/-- **Scott 1972, Corollary 4.3 (factorization).** `f_n = f∞ ∘ i_{n∞}`. -/
theorem coconeInf_comp_embInf (f : ∀ n, ScottMap (D n) D')
    (hf : ∀ n x, f n x = f (n + 1) ((P n).incl x)) (n : ℕ) (x : D n) :
    coconeInf D P f (embInf D P n x) = f n x := by
  apply le_antisymm
  · show (⋃ m, f m ((embInf D P n x).1 m)) ≤ f n x
    refine iSup_le fun m => ?_
    show f m (iComp D P n x m) ≤ f n x
    by_cases h : n ≤ m
    · rw [iComp_of_le D P h]; exact le_of_eq (coconeInf_climb D P f hf x h)
    · rw [iComp_of_ge D P h]; exact coconeInf_descend D P f hf (le_of_lt (not_le.mp h))
     x
  · refine le_trans (le_of_eq ?_) (le_iSup (fun m => f m ((embInf D P n x).1 m)) n)
    rw [show (embInf D P n x).1 n = x from iComp_self D P n x]

/-- `f∞` is Scott-continuous: `f∞(⊔S) = ⊔ f∞(S)` for directed `S` (each `f_n` is
    continuous and a
double `⋃` over `ℕ × S` commutes). -/
theorem coconeInf_preservesDirectedSup (f : ∀ n, ScottMap (D n) D') :
    PreservesDirectedSup (coconeInf D P f) := by
  intro S hS hSdir
  have hev : ∀ n, DirectedOn (· ≤ ·) ((fun s : InverseLimit D P => s.1 n) '' S) := by
    intro n
    rintro _ ⟨a, ha, rfl⟩ _ ⟨b, hb, rfl⟩
    obtain ⟨c, hc, hac, hbc⟩ := hSdir a ha b hb
    exact ⟨c.1 n, ⟨c, hc, rfl⟩, hac n, hbc n⟩
  have key : ∀ n, f n ((sSup S).1 n) = sSup ((fun s : InverseLimit D P => f n (s.1 n))
    '' S) := by
    intro n
    have h1 : (sSup S).1 n = sSup ((fun s : InverseLimit D P => s.1 n) '' S) :=
      projInfFun.preservesDirectedSup D P n hS hSdir
```

```
    rw [h1, (f n).preservesDirectedSup_coe _ (hS.image _) (hev n), Set.image_image]
  show (⋃ n, f n ((sSup S).1 n)) = sSup (coconeInf D P f '' S)
  simp_rw [key, sSup_image', coconeInf_apply]
  rw [iSup_comm]

end InverseLimit

end DomainTheory.ContinuousLattice
```

1

coconeInf.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import DomainTheory.ContinuousLattice.InverseLimits.embLE
public import DomainTheory.ContinuousLattice.InverseLimits.projLE
public import DomainTheory.ContinuousLattice.InverseLimits.embInf
public import DomainTheory.ContinuousLattice.InverseLimits.iComp
public import DomainTheory.ContinuousLattice.InverseLimits.instCompleteLattice
public import DomainTheory.ContinuousLattice.InverseLimits.projInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

variable {D' : Type*} [CompleteLattice D']

/-- The mediating map of Corollary 4.3: `f∞(x) = ⋃ₙ fₙ(xₙ)`. -/
def coconeInf (f : ∀ n, ScottMap (D n) D') (x : InverseLimit D P) : D' :=
  ⋃ n, f n (x.1 n)

theorem coconeInf_apply (f : ∀ n, ScottMap (D n) D') (x : InverseLimit D P) :
    coconeInf D P f x = ⋃ n, f n (x.1 n) := rfl

/-- Climbing then applying `f` is constant: `f_m(i_{m-1}...iₙ x) = fₙ(x)`. -/
theorem coconeInf_climb (f : ∀ n, ScottMap (D n) D')
    (hf : ∀ n x, f n x = f (n + 1) ((P n).incl x)) {n : ℕ} (x : D n) :
    ∀ {m : ℕ} (h : n ≤ m), f m (embLE D P h x) = f n x := by
  intro m h
  induction m, h using Nat.le_induction with
```

```
  | base => rw [embLE_self]
  | succ k hk ih => rw [embLE_succ D P hk (Nat.le_succ_of_le hk), ← hf k, ih]

/-- Descending then applying `f` only decreases: `f_m(j_m...j_{n-1} x) ⊑ f_n(x)`. -/
theorem coconeInf_descend (f : ∀ n, ScottMap (D n) D')
    (hf : ∀ n x, f n x = f (n + 1) ((P n).incl x)) {m : ℕ} :
    ∀ {n : ℕ} (h : m ≤ n) (x : D n), f m (projLE D P h x) ≤ f n x := by
  intro n h
  induction n, h using Nat.le_induction with
  | base => intro x; exact le_of_eq (congrArg (f m) (projLE.self D P (le_refl m) x))
  | succ k hk ih =>
      intro x
      rw [projLE.succ D P hk (Nat.le_succ_of_le hk)]
      calc f m (projLE D P hk ((P k).retr x)) ≤ f k ((P k).retr x) := ih ((P k).retr x
    )
        _ = f (k + 1) ((P k).incl ((P k).retr x)) := hf k _
        _ ≤ f (k + 1) x := (f (k + 1)).monotone ((P k).incl_retr_le x)

/-- **Scott 1972, Corollary 4.3 (factorization).** `f_n = f∞ ∘ i_{n∞}`. -/
theorem coconeInf_comp_embInf (f : ∀ n, ScottMap (D n) D')
    (hf : ∀ n x, f n x = f (n + 1) ((P n).incl x)) (n : ℕ) (x : D n) :
    coconeInf D P f (embInf D P n x) = f n x := by
  apply le_antisymm
  · show (⋃ m, f m ((embInf D P n x).1 m)) ≤ f n x
    refine iSup_le fun m => ?_
    show f m (iComp D P n x m) ≤ f n x
    by_cases h : n ≤ m
    · rw [iComp_of_le D P h]; exact le_of_eq (coconeInf_climb D P f hf x h)
    · rw [iComp_of_ge D P h]; exact coconeInf_descend D P f hf (le_of_lt (not_le.mp h))
     x
  · refine le_trans (le_of_eq ?_) (le_iSup (fun m => f m ((embInf D P n x).1 m)) n)
    rw [show (embInf D P n x).1 n = x from iComp_self D P n x]

/-- `f∞` is Scott-continuous: `f∞(⊔S) = ⊔ f∞(S)` for directed `S` (each `f_n` is
    continuous and a
double `⋃` over `ℕ × S` commutes). -/
theorem coconeInf_preservesDirectedSup (f : ∀ n, ScottMap (D n) D') :
    PreservesDirectedSup (coconeInf D P f) := by
  intro S hS hSdir
  have hev : ∀ n, DirectedOn (· ≤ ·) ((fun s : InverseLimit D P => s.1 n) '' S) := by
    intro n
    rintro _ ⟨a, ha, rfl⟩ _ ⟨b, hb, rfl⟩
    obtain ⟨c, hc, hac, hbc⟩ := hSdir a ha b hb
    exact ⟨c.1 n, ⟨c, hc, rfl⟩, hac n, hbc n⟩
  have key : ∀ n, f n ((sSup S).1 n) = sSup ((fun s : InverseLimit D P => f n (s.1 n))
    '' S) := by
    intro n
    have h1 : (sSup S).1 n = sSup ((fun s : InverseLimit D P => s.1 n) '' S) :=
      projInfFun.preservesDirectedSup D P n hS hSdir
    rw [h1, (f n).preservesDirectedSup_coe _ (hS.image _) (hev n), Set.image_image]
  show (⋃ n, f n ((sSup S).1 n)) = sSup (coconeInf D P f '' S)
  simp_rw [key, sSup_image', coconeInf_apply]
  rw [iSup_comm]

end InverseLimit
```

```
end DomainTheory.ContinuousLattice
```

2

coconeInf.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import DomainTheory.ContinuousLattice.InverseLimits.embLE
public import DomainTheory.ContinuousLattice.InverseLimits.projLE
public import DomainTheory.ContinuousLattice.InverseLimits.embInf
public import DomainTheory.ContinuousLattice.InverseLimits.iComp
public import DomainTheory.ContinuousLattice.InverseLimits.instCompleteLattice
public import DomainTheory.ContinuousLattice.InverseLimits.projInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

variable {D' : Type*} [CompleteLattice D']

/-- The mediating map of Corollary 4.3: `f∞(x) = ⋃ₙ fₙ(xₙ)`. -/
def coconeInf (f : ∀ n, ScottMap (D n) D') (x : InverseLimit D P) : D' :=
  ⋃ n, f n (x.1 n)

theorem coconeInf_apply (f : ∀ n, ScottMap (D n) D') (x : InverseLimit D P) :
    coconeInf D P f x = ⋃ n, f n (x.1 n) := rfl

/-- Climbing then applying `f` is constant: `f_m(i_{m-1}...iₙ x) = fₙ(x)`. -/
theorem coconeInf_climb (f : ∀ n, ScottMap (D n) D')
    (hf : ∀ n x, f n x = f (n + 1) ((P n).incl x)) {n : ℕ} (x : D n) :
    ∀ {m : ℕ} (h : n ≤ m), f m (embLE D P h x) = f n x := by
  intro m h
  induction m, h using Nat.le_induction with
  | base => rw [embLE_self]
  | succ k hk ih => rw [embLE_succ D P hk (Nat.le_succ_of_le hk), ← hf k, ih]

/-- Descending then applying `f` only decreases: `f_m(j_m...j_{n-1} x) ⊑ fₙ(x)`. -/
theorem coconeInf_descend (f : ∀ n, ScottMap (D n) D')
    (hf : ∀ n x, f n x = f (n + 1) ((P n).incl x)) {m : ℕ} :
    ∀ {n : ℕ} (h : m ≤ n) (x : D n), f m (projLE D P h x) ≤ f n x := by
```

```
  intro n h
  induction n, h using Nat.le_induction with
  | base => intro x; exact le_of_eq (congrArg (f m) (projLE.self D P (le_refl m) x))
  | succ k hk ih =>
      intro x
      rw [projLE.succ D P hk (Nat.le_succ_of_le hk)]
      calc f m (projLE D P hk ((P k).retr x)) ≤ f k ((P k).retr x) := ih ((P k).retr x
    )
        _ = f (k + 1) ((P k).incl ((P k).retr x)) := hf k _
        _ ≤ f (k + 1) x := (f (k + 1)).monotone ((P k).incl_retr_le x)

/-- **Scott 1972, Corollary 4.3 (factorization).** `f_n = f∞ ∘ i_{n∞}`. -/
theorem coconeInf_comp_embInf (f : ∀ n, ScottMap (D n) D')
    (hf : ∀ n x, f n x = f (n + 1) ((P n).incl x)) (n : ℕ) (x : D n) :
    coconeInf D P f (embInf D P n x) = f n x := by
  apply le_antisymm
  · show (⋃ m, f m ((embInf D P n x).1 m)) ≤ f n x
    refine iSup_le fun m => ?_
    show f m (iComp D P n x m) ≤ f n x
    by_cases h : n ≤ m
    · rw [iComp_of_le D P h]; exact le_of_eq (coconeInf_climb D P f hf x h)
    · rw [iComp_of_ge D P h]; exact coconeInf_descend D P f hf (le_of_lt (not_le.mp h))
     x
  · refine le_trans (le_of_eq ?_) (le_iSup (fun m => f m ((embInf D P n x).1 m)) n)
    rw [show (embInf D P n x).1 n = x from iComp_self D P n x]

/-- `f∞` is Scott-continuous: `f∞(⊔S) = ⊔ f∞(S)` for directed `S` (each `f_n` is
    continuous and a
double `⋃` over `ℕ × S` commutes). -/
theorem coconeInf_preservesDirectedSup (f : ∀ n, ScottMap (D n) D') :
    PreservesDirectedSup (coconeInf D P f) := by
  intro S hS hSdir
  have hev : ∀ n, DirectedOn (· ≤ ·) ((fun s : InverseLimit D P => s.1 n) '' S) := by
    intro n
    rintro _ ⟨a, ha, rfl⟩ _ ⟨b, hb, rfl⟩
    obtain ⟨c, hc, hac, hbc⟩ := hSdir a ha b hb
    exact ⟨c.1 n, ⟨c, hc, rfl⟩, hac n, hbc n⟩
  have key : ∀ n, f n ((sSup S).1 n) = sSup ((fun s : InverseLimit D P => f n (s.1 n))
    '' S) := by
    intro n
    have h1 : (sSup S).1 n = sSup ((fun s : InverseLimit D P => s.1 n) '' S) :=
      projInfFun.preservesDirectedSup D P n hS hSdir
    rw [h1, (f n).preservesDirectedSup_coe _ (hS.image _) (hev n), Set.image_image]
  show (⋃ n, f n ((sSup S).1 n)) = sSup (coconeInf D P f '' S)
  simp_rw [key, sSup_image', coconeInf_apply]
  rw [iSup_comm]

end InverseLimit

end DomainTheory.ContinuousLattice
```

3

coconeInf.lean

```
module
```

```
public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import DomainTheory.ContinuousLattice.InverseLimits.embLE
public import DomainTheory.ContinuousLattice.InverseLimits.projLE
public import DomainTheory.ContinuousLattice.InverseLimits.embInf
public import DomainTheory.ContinuousLattice.InverseLimits.iComp
public import DomainTheory.ContinuousLattice.InverseLimits.instCompleteLattice
public import DomainTheory.ContinuousLattice.InverseLimits.projInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

variable {D' : Type*} [CompleteLattice D']

/-- The mediating map of Corollary 4.3: `f∞(x) = ⋃ₙ fₙ(xₙ)`. -/
def coconeInf (f : ∀ n, ScottMap (D n) D') (x : InverseLimit D P) : D' :=
  ⋃ n, f n (x.1 n)

theorem coconeInf_apply (f : ∀ n, ScottMap (D n) D') (x : InverseLimit D P) :
    coconeInf D P f x = ⋃ n, f n (x.1 n) := rfl

/-- Climbing then applying `f` is constant: `f_m(i_{m-1}...iₙ x) = fₙ(x)`. -/
theorem coconeInf_climb (f : ∀ n, ScottMap (D n) D')
    (hf : ∀ n x, f n x = f (n + 1) ((P n).incl x)) {n : ℕ} (x : D n) :
    ∀ {m : ℕ} (h : n ≤ m), f m (embLE D P h x) = f n x := by
  intro m h
  induction m, h using Nat.le_induction with
  | base => rw [embLE_self]
  | succ k hk ih => rw [embLE_succ D P hk (Nat.le_succ_of_le hk), ← hf k, ih]

/-- Descending then applying `f` only decreases: `f_m(j_m...j_{n-1} x) ⊑ fₙ(x)`. -/
theorem coconeInf_descend (f : ∀ n, ScottMap (D n) D')
    (hf : ∀ n x, f n x = f (n + 1) ((P n).incl x)) {m : ℕ} :
    ∀ {n : ℕ} (h : m ≤ n) (x : D n), f m (projLE D P h x) ≤ f n x := by
  intro n h
  induction n, h using Nat.le_induction with
  | base => intro x; exact le_of_eq (congrArg (f m) (projLE.self D P (le_refl m) x))
  | succ k hk ih =>
      intro x
      rw [projLE.succ D P hk (Nat.le_succ_of_le hk)]
      calc f m (projLE D P hk ((P k).retr x)) ≤ f k ((P k).retr x) := ih ((P k).retr x
```

```
    )
        _ = f (k + 1) ((P k).incl ((P k).retr x)) := hf k _
        _ ≤ f (k + 1) x := (f (k + 1)).monotone ((P k).incl_retr_le x)

/-- **Scott 1972, Corollary 4.3 (factorization).** `f_n = f∞ ∘ i_{n∞}`. -/
theorem coconeInf_comp_embInf (f : ∀ n, ScottMap (D n) D')
    (hf : ∀ n x, f n x = f (n + 1) ((P n).incl x)) (n : ℕ) (x : D n) :
    coconeInf D P f (embInf D P n x) = f n x := by
  apply le_antisymm
  · show (⋃ m, f m ((embInf D P n x).1 m)) ≤ f n x
    refine iSup_le fun m => ?_
    show f m (iComp D P n x m) ≤ f n x
    by_cases h : n ≤ m
    · rw [iComp_of_le D P h]; exact le_of_eq (coconeInf_climb D P f hf x h)
    · rw [iComp_of_ge D P h]; exact coconeInf_descend D P f hf (le_of_lt (not_le.mp h))
     x
  · refine le_trans (le_of_eq ?_) (le_iSup (fun m => f m ((embInf D P n x).1 m)) n)
    rw [show (embInf D P n x).1 n = x from iComp_self D P n x]

/-- `f∞` is Scott-continuous: `f∞(⊔S) = ⊔ f∞(S)` for directed `S` (each `f_n` is
    continuous and a
double `⋃` over `ℕ × S` commutes). -/
theorem coconeInf_preservesDirectedSup (f : ∀ n, ScottMap (D n) D') :
    PreservesDirectedSup (coconeInf D P f) := by
  intro S hS hSdir
  have hev : ∀ n, DirectedOn (· ≤ ·) ((fun s : InverseLimit D P => s.1 n) '' S) := by
    intro n
    rintro _ ⟨a, ha, rfl⟩ _ ⟨b, hb, rfl⟩
    obtain ⟨c, hc, hac, hbc⟩ := hSdir a ha b hb
    exact ⟨c.1 n, ⟨c, hc, rfl⟩, hac n, hbc n⟩
  have key : ∀ n, f n ((sSup S).1 n) = sSup ((fun s : InverseLimit D P => f n (s.1 n))
    '' S) := by
    intro n
    have h1 : (sSup S).1 n = sSup ((fun s : InverseLimit D P => s.1 n) '' S) :=
      projInfFun.preservesDirectedSup D P n hS hSdir
    rw [h1, (f n).preservesDirectedSup_coe _ (hS.image _) (hev n), Set.image_image]
  show (⋃ n, f n ((sSup S).1 n)) = sSup (coconeInf D P f '' S)
  simp_rw [key, sSup_image', coconeInf_apply]
  rw [iSup_comm]

end InverseLimit

end DomainTheory.ContinuousLattice
```

4

coconeInf.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import DomainTheory.ContinuousLattice.InverseLimits.embLE
public import DomainTheory.ContinuousLattice.InverseLimits.projLE
public import DomainTheory.ContinuousLattice.InverseLimits.embInf
public import DomainTheory.ContinuousLattice.InverseLimits.iComp
public import DomainTheory.ContinuousLattice.InverseLimits.instCompleteLattice
```

```
public import DomainTheory.ContinuousLattice.InverseLimits.projInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

variable {D' : Type*} [CompleteLattice D']

/-- The mediating map of Corollary 4.3: `f∞(x) = ⋃ₙ fₙ(xₙ)`. -/
def coconeInf (f : ∀ n, ScottMap (D n) D') (x : InverseLimit D P) : D' :=
  ⋃ n, f n (x.1 n)

theorem coconeInf_apply (f : ∀ n, ScottMap (D n) D') (x : InverseLimit D P) :
    coconeInf D P f x = ⋃ n, f n (x.1 n) := rfl

/-- Climbing then applying `f` is constant: `f_m(i_{m-1}...iₙ x) = fₙ(x)`. -/
theorem coconeInf_climb (f : ∀ n, ScottMap (D n) D')
    (hf : ∀ n x, f n x = f (n + 1) ((P n).incl x)) {n : ℕ} (x : D n) :
    ∀ {m : ℕ} (h : n ≤ m), f m (embLE D P h x) = f n x := by
  intro m h
  induction m, h using Nat.le_induction with
  | base => rw [embLE_self]
  | succ k hk ih => rw [embLE_succ D P hk (Nat.le_succ_of_le hk), ← hf k, ih]

/-- Descending then applying `f` only decreases: `f_m(j_m...j_{n-1} x) ⊑ fₙ(x)`. -/
theorem coconeInf_descend (f : ∀ n, ScottMap (D n) D')
    (hf : ∀ n x, f n x = f (n + 1) ((P n).incl x)) {m : ℕ} :
    ∀ {n : ℕ} (h : m ≤ n) (x : D n), f m (projLE D P h x) ≤ f n x := by
  intro n h
  induction n, h using Nat.le_induction with
  | base => intro x; exact le_of_eq (congrArg (f m) (projLE.self D P (le_refl m) x))
  | succ k hk ih =>
      intro x
      rw [projLE.succ D P hk (Nat.le_succ_of_le hk)]
      calc f m (projLE D P hk ((P k).retr x)) ≤ f k ((P k).retr x) := ih ((P k).retr x
    )
        _ = f (k + 1) ((P k).incl ((P k).retr x)) := hf k _
        _ ≤ f (k + 1) x := (f (k + 1)).monotone ((P k).incl_retr_le x)

/-- **Scott 1972, Corollary 4.3 (factorization).** `fₙ = f∞ ∘ i_{n∞}`. -/
theorem coconeInf_comp_embInf (f : ∀ n, ScottMap (D n) D')
    (hf : ∀ n x, f n x = f (n + 1) ((P n).incl x)) (n : ℕ) (x : D n) :
```

```
    coconeInf D P f (embInf D P n x) = f n x := by
  apply le_antisymm
  · show (⋃ m, f m ((embInf D P n x).1 m)) ≤ f n x
    refine iSup_le fun m => ?_
    show f m (iComp D P n x m) ≤ f n x
    by_cases h : n ≤ m
    · rw [iComp_of_le D P h]; exact le_of_eq (coconeInf_climb D P f hf x h)
    · rw [iComp_of_ge D P h]; exact coconeInf_descend D P f hf (le_of_lt (not_le.mp h))
     x
  · refine le_trans (le_of_eq ?_) (le_iSup (fun m => f m ((embInf D P n x).1 m)) n)
    rw [show (embInf D P n x).1 n = x from iComp_self D P n x]

/-- `f∞` is Scott-continuous: `f∞(⊔S) = ⊔ f∞(S)` for directed `S` (each `fₙ` is
    continuous and a
double `⋃` over `ℕ × S` commutes). -/
theorem coconeInf_preservesDirectedSup (f : ∀ n, ScottMap (D n) D') :
    PreservesDirectedSup (coconeInf D P f) := by
  intro S hS hSdir
  have hev : ∀ n, DirectedOn (· ≤ ·) ((fun s : InverseLimit D P => s.1 n) '' S) := by
    intro n
    rintro _ ⟨a, ha, rfl⟩ _ ⟨b, hb, rfl⟩
    obtain ⟨c, hc, hac, hbc⟩ := hSdir a ha b hb
    exact ⟨c.1 n, ⟨c, hc, rfl⟩, hac n, hbc n⟩
  have key : ∀ n, f n ((sSup S).1 n) = sSup ((fun s : InverseLimit D P => f n (s.1 n))
    '' S) := by
    intro n
    have h1 : (sSup S).1 n = sSup ((fun s : InverseLimit D P => s.1 n) '' S) :=
      projInfFun.preservesDirectedSup D P n hS hSdir
    rw [h1, (f n).preservesDirectedSup_coe _ (hS.image _) (hev n), Set.image_image]
  show (⋃ n, f n ((sSup S).1 n)) = sSup (coconeInf D P f '' S)
  simp_rw [key, sSup_image', coconeInf_apply]
  rw [iSup_comm]

end InverseLimit

end DomainTheory.ContinuousLattice
```

5

coconeInf.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import DomainTheory.ContinuousLattice.InverseLimits.embLE
public import DomainTheory.ContinuousLattice.InverseLimits.projLE
public import DomainTheory.ContinuousLattice.InverseLimits.embInf
public import DomainTheory.ContinuousLattice.InverseLimits.iComp
public import DomainTheory.ContinuousLattice.InverseLimits.instCompleteLattice
public import DomainTheory.ContinuousLattice.InverseLimits.projInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import Mathlib.Data.Set.Basic

@[expose] public section
```

```
namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

variable {D' : Type*} [CompleteLattice D']

/-- The mediating map of Corollary 4.3: `f∞(x) = ⋃ₙ fₙ(xₙ)`. -/
def coconeInf (f : ∀ n, ScottMap (D n) D') (x : InverseLimit D P) : D' :=
  ⋃ n, f n (x.1 n)

theorem coconeInf_apply (f : ∀ n, ScottMap (D n) D') (x : InverseLimit D P) :
    coconeInf D P f x = ⋃ n, f n (x.1 n) := rfl

/-- Climbing then applying `f` is constant: `f_m(i_{m-1}...iₙ x) = fₙ(x)`. -/
theorem coconeInf_climb (f : ∀ n, ScottMap (D n) D')
    (hf : ∀ n x, f n x = f (n + 1) ((P n).incl x)) {n : ℕ} (x : D n) :
    ∀ {m : ℕ} (h : n ≤ m), f m (embLE D P h x) = f n x := by
  intro m h
  induction m, h using Nat.le_induction with
  | base => rw [embLE_self]
  | succ k hk ih => rw [embLE_succ D P hk (Nat.le_succ_of_le hk), ← hf k, ih]

/-- Descending then applying `f` only decreases: `f_m(j_m...j_{n-1} x) ⊑ fₙ(x)`. -/
theorem coconeInf_descend (f : ∀ n, ScottMap (D n) D')
    (hf : ∀ n x, f n x = f (n + 1) ((P n).incl x)) {m : ℕ} :
    ∀ {n : ℕ} (h : m ≤ n) (x : D n), f m (projLE D P h x) ≤ f n x := by
  intro n h
  induction n, h using Nat.le_induction with
  | base => intro x; exact le_of_eq (congrArg (f m) (projLE.self D P (le_refl m) x))
  | succ k hk ih =>
      intro x
      rw [projLE.succ D P hk (Nat.le_succ_of_le hk)]
      calc f m (projLE D P hk ((P k).retr x)) ≤ f k ((P k).retr x) := ih ((P k).retr x
    )
        _ = f (k + 1) ((P k).incl ((P k).retr x)) := hf k _
        _ ≤ f (k + 1) x := (f (k + 1)).monotone ((P k).incl_retr_le x)

/-- **Scott 1972, Corollary 4.3 (factorization).** `fₙ = f∞ ∘ i_{n∞}`. -/
theorem coconeInf_comp_embInf (f : ∀ n, ScottMap (D n) D')
    (hf : ∀ n x, f n x = f (n + 1) ((P n).incl x)) (n : ℕ) (x : D n) :
    coconeInf D P f (embInf D P n x) = f n x := by
  apply le_antisymm
  · show (⋃ m, f m ((embInf D P n x).1 m)) ≤ f n x
    refine iSup_le fun m => ?_
    show f m (iComp D P n x m) ≤ f n x
    by_cases h : n ≤ m
    · rw [iComp_of_le D P h]; exact le_of_eq (coconeInf_climb D P f hf x h)
```

```
    · rw [iComp_of_ge D P h]; exact coconeInf_descend D P f hf (le_of_lt (not_le.mp h))
      x
  · refine le_trans (le_of_eq ?_) (le_iSup (fun m => f m ((embInf D P n x).1 m)) n)
    rw [show (embInf D P n x).1 n = x from iComp_self D P n x]

/-- `f∞` is Scott-continuous: `f∞(⊔S) = ⊔ f∞(S)` for directed `S` (each `fₙ` is
    continuous and a
double `⋃` over `ℕ × S` commutes). -/
theorem coconeInf_preservesDirectedSup (f : ∀ n, ScottMap (D n) D') :
    PreservesDirectedSup (coconeInf D P f) := by
  intro S hS hSdir
  have hev : ∀ n, DirectedOn (· ≤ ·) ((fun s : InverseLimit D P => s.1 n) '' S) := by
    intro n
    rintro _ ⟨a, ha, rfl⟩ _ ⟨b, hb, rfl⟩
    obtain ⟨c, hc, hac, hbc⟩ := hSdir a ha b hb
    exact ⟨c.1 n, ⟨c, hc, rfl⟩, hac n, hbc n⟩
  have key : ∀ n, f n ((sSup S).1 n) = sSup ((fun s : InverseLimit D P => f n (s.1 n))
    '' S) := by
    intro n
    have h1 : (sSup S).1 n = sSup ((fun s : InverseLimit D P => s.1 n) '' S) :=
      projInfFun.preservesDirectedSup D P n hS hSdir
    rw [h1, (f n).preservesDirectedSup_coe _ (hS.image _) (hev n), Set.image_image]
  show (⋃ n, f n ((sSup S).1 n)) = sSup (coconeInf D P f '' S)
  simp_rw [key, sSup_image', coconeInf_apply]
  rw [iSup_comm]

end InverseLimit

end DomainTheory.ContinuousLattice
```

6

coconeInf.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import DomainTheory.ContinuousLattice.InverseLimits.embLE
public import DomainTheory.ContinuousLattice.InverseLimits.projLE
public import DomainTheory.ContinuousLattice.InverseLimits.embInf
public import DomainTheory.ContinuousLattice.InverseLimits.iComp
public import DomainTheory.ContinuousLattice.InverseLimits.instCompleteLattice
public import DomainTheory.ContinuousLattice.InverseLimits.projInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u
```

```
section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

variable {D' : Type*} [CompleteLattice D']

/-- The mediating map of Corollary 4.3: `f∞(x) = ⋃ₙ fₙ(xₙ)`. -/
def coconeInf (f : ∀ n, ScottMap (D n) D') (x : InverseLimit D P) : D' :=
  ⋃ n, f n (x.1 n)

theorem coconeInf_apply (f : ∀ n, ScottMap (D n) D') (x : InverseLimit D P) :
    coconeInf D P f x = ⋃ n, f n (x.1 n) := rfl

/-- Climbing then applying `f` is constant: `f_m(i_{m-1}...iₙ x) = fₙ(x)`. -/
theorem coconeInf_climb (f : ∀ n, ScottMap (D n) D')
    (hf : ∀ n x, f n x = f (n + 1) ((P n).incl x)) {n : ℕ} (x : D n) :
    ∀ {m : ℕ} (h : n ≤ m), f m (embLE D P h x) = f n x := by
  intro m h
  induction m, h using Nat.le_induction with
  | base => rw [embLE_self]
  | succ k hk ih => rw [embLE_succ D P hk (Nat.le_succ_of_le hk), ← hf k, ih]

/-- Descending then applying `f` only decreases: `f_m(j_m...j_{n-1} x) ⊑ fₙ(x)`. -/
theorem coconeInf_descend (f : ∀ n, ScottMap (D n) D')
    (hf : ∀ n x, f n x = f (n + 1) ((P n).incl x)) {m : ℕ} :
    ∀ {n : ℕ} (h : m ≤ n) (x : D n), f m (projLE D P h x) ≤ f n x := by
  intro n h
  induction n, h using Nat.le_induction with
  | base => intro x; exact le_of_eq (congrArg (f m) (projLE.self D P (le_refl m) x))
  | succ k hk ih =>
      intro x
      rw [projLE.succ D P hk (Nat.le_succ_of_le hk)]
      calc f m (projLE D P hk ((P k).retr x)) ≤ f k ((P k).retr x) := ih ((P k).retr x
    )
        _ = f (k + 1) ((P k).incl ((P k).retr x)) := hf k _
        _ ≤ f (k + 1) x := (f (k + 1)).monotone ((P k).incl_retr_le x)

/-- **Scott 1972, Corollary 4.3 (factorization).** `fₙ = f∞ ∘ i_{n∞}`. -/
theorem coconeInf_comp_embInf (f : ∀ n, ScottMap (D n) D')
    (hf : ∀ n x, f n x = f (n + 1) ((P n).incl x)) (n : ℕ) (x : D n) :
    coconeInf D P f (embInf D P n x) = f n x := by
  apply le_antisymm
  · show (⋃ m, f m ((embInf D P n x).1 m)) ≤ f n x
    refine iSup_le fun m => ?_
    show f m (iComp D P n x m) ≤ f n x
    by_cases h : n ≤ m
    · rw [iComp_of_le D P h]; exact le_of_eq (coconeInf_climb D P f hf x h)
    · rw [iComp_of_ge D P h]; exact coconeInf_descend D P f hf (le_of_lt (not_le.mp h))
     x
  · refine le_trans (le_of_eq ?_) (le_iSup (fun m => f m ((embInf D P n x).1 m)) n)
    rw [show (embInf D P n x).1 n = x from iComp_self D P n x]

/-- `f∞` is Scott-continuous: `f∞(⊔S) = ⊔ f∞(S)` for directed `S` (each `fₙ` is
    continuous and a
```

```
double `⋃` over `ℕ × S` commutes). -/
theorem coconeInf_preservesDirectedSup (f : ∀ n, ScottMap (D n) D') :
    PreservesDirectedSup (coconeInf D P f) := by
  intro S hS hSdir
  have hev : ∀ n, DirectedOn (· ≤ ·) ((fun s : InverseLimit D P => s.1 n) '' S) := by
    intro n
    rintro _ ⟨a, ha, rfl⟩ _ ⟨b, hb, rfl⟩
    obtain ⟨c, hc, hac, hbc⟩ := hSdir a ha b hb
    exact ⟨c.1 n, ⟨c, hc, rfl⟩, hac n, hbc n⟩
  have key : ∀ n, f n ((sSup S).1 n) = sSup ((fun s : InverseLimit D P => f n (s.1 n))
    '' S) := by
    intro n
    have h1 : (sSup S).1 n = sSup ((fun s : InverseLimit D P => s.1 n) '' S) :=
      projInfFun.preservesDirectedSup D P n hS hSdir
    rw [h1, (f n).preservesDirectedSup_coe _ (hS.image _) (hev n), Set.image_image]
  show (⋃ n, f n ((sSup S).1 n)) = sSup (coconeInf D P f '' S)
  simp_rw [key, sSup_image', coconeInf_apply]
  rw [iSup_comm]

end InverseLimit

end DomainTheory.ContinuousLattice
```

7

coconeInf.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import DomainTheory.ContinuousLattice.InverseLimits.embLE
public import DomainTheory.ContinuousLattice.InverseLimits.projLE
public import DomainTheory.ContinuousLattice.InverseLimits.embInf
public import DomainTheory.ContinuousLattice.InverseLimits.iComp
public import DomainTheory.ContinuousLattice.InverseLimits.instCompleteLattice
public import DomainTheory.ContinuousLattice.InverseLimits.projInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

variable {D' : Type*} [CompleteLattice D']
```

```
/-- The mediating map of Corollary 4.3: `f∞(x) = ⋃ₙ fₙ(xₙ)`. -/
def coconeInf (f : ∀ n, ScottMap (D n) D') (x : InverseLimit D P) : D' :=
  ⋃ n, f n (x.1 n)

theorem coconeInf_apply (f : ∀ n, ScottMap (D n) D') (x : InverseLimit D P) :
    coconeInf D P f x = ⋃ n, f n (x.1 n) := rfl

/-- Climbing then applying `f` is constant: `f_m(i_{m-1}...iₙ x) = fₙ(x)`. -/
theorem coconeInf_climb (f : ∀ n, ScottMap (D n) D')
    (hf : ∀ n x, f n x = f (n + 1) ((P n).incl x)) {n : ℕ} (x : D n) :
    ∀ {m : ℕ} (h : n ≤ m), f m (embLE D P h x) = f n x := by
  intro m h
  induction m, h using Nat.le_induction with
  | base => rw [embLE_self]
  | succ k hk ih => rw [embLE_succ D P hk (Nat.le_succ_of_le hk), ← hf k, ih]

/-- Descending then applying `f` only decreases: `f_m(j_m...j_{n-1} x) ⊑ fₙ(x)`. -/
theorem coconeInf_descend (f : ∀ n, ScottMap (D n) D')
    (hf : ∀ n x, f n x = f (n + 1) ((P n).incl x)) {m : ℕ} :
    ∀ {n : ℕ} (h : m ≤ n) (x : D n), f m (projLE D P h x) ≤ f n x := by
  intro n h
  induction n, h using Nat.le_induction with
  | base => intro x; exact le_of_eq (congrArg (f m) (projLE.self D P (le_refl m) x))
  | succ k hk ih =>
      intro x
      rw [projLE.succ D P hk (Nat.le_succ_of_le hk)]
      calc f m (projLE D P hk ((P k).retr x)) ≤ f k ((P k).retr x) := ih ((P k).retr x
    )
        _ = f (k + 1) ((P k).incl ((P k).retr x)) := hf k _
        _ ≤ f (k + 1) x := (f (k + 1)).monotone ((P k).incl_retr_le x)

/-- **Scott 1972, Corollary 4.3 (factorization).** `fₙ = f∞ ∘ i_{n∞}`. -/
theorem coconeInf_comp_embInf (f : ∀ n, ScottMap (D n) D')
    (hf : ∀ n x, f n x = f (n + 1) ((P n).incl x)) (n : ℕ) (x : D n) :
    coconeInf D P f (embInf D P n x) = f n x := by
  apply le_antisymm
  · show (⋃ m, f m ((embInf D P n x).1 m)) ≤ f n x
    refine iSup_le fun m => ?_
    show f m (iComp D P n x m) ≤ f n x
    by_cases h : n ≤ m
    · rw [iComp_of_le D P h]; exact le_of_eq (coconeInf_climb D P f hf x h)
    · rw [iComp_of_ge D P h]; exact coconeInf_descend D P f hf (le_of_lt (not_le.mp h))
     x
  · refine le_trans (le_of_eq ?_) (le_iSup (fun m => f m ((embInf D P n x).1 m)) n)
    rw [show (embInf D P n x).1 n = x from iComp_self D P n x]

/-- `f∞` is Scott-continuous: `f∞(⊔S) = ⊔ f∞(S)` for directed `S` (each `fₙ` is
    continuous and a
double `⋃` over `ℕ × S` commutes). -/
theorem coconeInf_preservesDirectedSup (f : ∀ n, ScottMap (D n) D') :
    PreservesDirectedSup (coconeInf D P f) := by
  intro S hS hSdir
  have hev : ∀ n, DirectedOn (· ≤ ·) ((fun s : InverseLimit D P => s.1 n) '' S) := by
    intro n
    rintro _ ⟨a, ha, rfl⟩ _ ⟨b, hb, rfl⟩
```

```
    obtain ⟨c, hc, hac, hbc⟩ := hSdir a ha b hb
    exact ⟨c.1 n, ⟨c, hc, rfl⟩, hac n, hbc n⟩
  have key : ∀ n, f n ((sSup S).1 n) = sSup ((fun s : InverseLimit D P => f n (s.1 n))
    '' S) := by
    intro n
    have h1 : (sSup S).1 n = sSup ((fun s : InverseLimit D P => s.1 n) '' S) :=
      projInfFun.preservesDirectedSup D P n hS hSdir
    rw [h1, (f n).preservesDirectedSup_coe _ (hS.image _) (hev n), Set.image_image]
  show (⋃ n, f n ((sSup S).1 n)) = sSup (coconeInf D P f '' S)
  simp_rw [key, sSup_image', coconeInf_apply]
  rw [iSup_comm]

end InverseLimit

end DomainTheory.ContinuousLattice
```

8

coconeInf.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import DomainTheory.ContinuousLattice.InverseLimits.embLE
public import DomainTheory.ContinuousLattice.InverseLimits.projLE
public import DomainTheory.ContinuousLattice.InverseLimits.embInf
public import DomainTheory.ContinuousLattice.InverseLimits.iComp
public import DomainTheory.ContinuousLattice.InverseLimits.instCompleteLattice
public import DomainTheory.ContinuousLattice.InverseLimits.projInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

variable {D' : Type*} [CompleteLattice D']

/-- The mediating map of Corollary 4.3: `f∞(x) = ⋃ₙ fₙ(xₙ)`. -/
def coconeInf (f : ∀ n, ScottMap (D n) D') (x : InverseLimit D P) : D' :=
  ⋃ n, f n (x.1 n)

theorem coconeInf_apply (f : ∀ n, ScottMap (D n) D') (x : InverseLimit D P) :
    coconeInf D P f x = ⋃ n, f n (x.1 n) := rfl
```

```
/-- Climbing then applying `f` is constant: `f_m(i_{m-1}...i_n x) = f_n(x)`. -/
theorem coconeInf_climb (f : ∀ n, ScottMap (D n) D')
    (hf : ∀ n x, f n x = f (n + 1) ((P n).incl x)) {n : ℕ} (x : D n) :
    ∀ {m : ℕ} (h : n ≤ m), f m (embLE D P h x) = f n x := by
  intro m h
  induction m, h using Nat.le_induction with
  | base => rw [embLE_self]
  | succ k hk ih => rw [embLE_succ D P hk (Nat.le_succ_of_le hk), ← hf k, ih]

/-- Descending then applying `f` only decreases: `f_m(j_m...j_{n-1} x) ⊑ f_n(x)`. -/
theorem coconeInf_descend (f : ∀ n, ScottMap (D n) D')
    (hf : ∀ n x, f n x = f (n + 1) ((P n).incl x)) {m : ℕ} :
    ∀ {n : ℕ} (h : m ≤ n) (x : D n), f m (projLE D P h x) ≤ f n x := by
  intro n h
  induction n, h using Nat.le_induction with
  | base => intro x; exact le_of_eq (congrArg (f m) (projLE.self D P (le_refl m) x))
  | succ k hk ih =>
      intro x
      rw [projLE.succ D P hk (Nat.le_succ_of_le hk)]
      calc f m (projLE D P hk ((P k).retr x)) ≤ f k ((P k).retr x) := ih ((P k).retr x
    )
        _ = f (k + 1) ((P k).incl ((P k).retr x)) := hf k _
        _ ≤ f (k + 1) x := (f (k + 1)).monotone ((P k).incl_retr_le x)

/-- **Scott 1972, Corollary 4.3 (factorization).** `f_n = f∞ ∘ i_{n∞}`. -/
theorem coconeInf_comp_embInf (f : ∀ n, ScottMap (D n) D')
    (hf : ∀ n x, f n x = f (n + 1) ((P n).incl x)) (n : ℕ) (x : D n) :
    coconeInf D P f (embInf D P n x) = f n x := by
  apply le_antisymm
  · show (⋃ m, f m ((embInf D P n x).1 m)) ≤ f n x
    refine iSup_le fun m => ?_
    show f m (iComp D P n x m) ≤ f n x
    by_cases h : n ≤ m
    · rw [iComp_of_le D P h]; exact le_of_eq (coconeInf_climb D P f hf x h)
    · rw [iComp_of_ge D P h]; exact coconeInf_descend D P f hf (le_of_lt (not_le.mp h))
     x
  · refine le_trans (le_of_eq ?_) (le_iSup (fun m => f m ((embInf D P n x).1 m)) n)
    rw [show (embInf D P n x).1 n = x from iComp_self D P n x]

/-- `f∞` is Scott-continuous: `f∞(⊔S) = ⊔ f∞(S)` for directed `S` (each `f_n` is
    continuous and a
double `⋃` over `ℕ × S` commutes). -/
theorem coconeInf_preservesDirectedSup (f : ∀ n, ScottMap (D n) D') :
    PreservesDirectedSup (coconeInf D P f) := by
  intro S hS hSdir
  have hev : ∀ n, DirectedOn (· ≤ ·) ((fun s : InverseLimit D P => s.1 n) '' S) := by
    intro n
    rintro _ ⟨a, ha, rfl⟩ _ ⟨b, hb, rfl⟩
    obtain ⟨c, hc, hac, hbc⟩ := hSdir a ha b hb
    exact ⟨c.1 n, ⟨c, hc, rfl⟩, hac n, hbc n⟩
  have key : ∀ n, f n ((sSup S).1 n) = sSup ((fun s : InverseLimit D P => f n (s.1 n))
    '' S) := by
    intro n
    have h1 : (sSup S).1 n = sSup ((fun s : InverseLimit D P => s.1 n) '' S) :=
      projInfFun.preservesDirectedSup D P n hS hSdir
```

```
    rw [h1, (f n).preservesDirectedSup_coe _ (hS.image _) (hev n), Set.image_image]
  show (⋃ n, f n ((sSup S).1 n)) = sSup (coconeInf D P f '' S)
  simp_rw [key, sSup_image', coconeInf_apply]
  rw [iSup_comm]

end InverseLimit

end DomainTheory.ContinuousLattice
```

9

inverseLimitLE.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- Coordinatewise order on compatible sequences. Kept reducible so existing
subtype-order lemmas see the underlying pointwise relation definitionally. -/
abbrev inverseLimitLE (x y : InverseLimit D P) : Prop :=
  ∀ n,
    @LE.le (D n)
      (ChainCompletePartialOrder.instOfCompleteLattice
        (α := D n)).toPartialOrder.toPreorder.toLE
      (x.1 n) (y.1 n)

namespace inverseLimitLE

/-- Reflexivity of the coordinatewise inverse-limit order. -/
theorem refl (x : InverseLimit D P) :
    inverseLimitLE D P x x :=
  fun n => le_refl (x.1 n)

/-- Transitivity of the coordinatewise inverse-limit order. -/
theorem trans (x y z : InverseLimit D P)
    (hxy : inverseLimitLE D P x y) (hyz : inverseLimitLE D P y z) :
    inverseLimitLE D P x z :=
  fun n => le_trans (hxy n) (hyz n)

/-- Antisymmetry of the coordinatewise inverse-limit order. -/
theorem antisymm (x y : InverseLimit D P)
```

```
    (hxy : inverseLimitLE D P x y) (hyx : inverseLimitLE D P y x) :
    x = y :=
  Subtype.ext (funext fun n => le_antisymm (hxy n) (hyx n))

end inverseLimitLE

end InverseLimit

end DomainTheory.ContinuousLattice
```

0

inverseLimitLE.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- Coordinatewise order on compatible sequences. Kept reducible so existing
subtype-order lemmas see the underlying pointwise relation definitionally. -/
abbrev inverseLimitLE (x y : InverseLimit D P) : Prop :=
  ∀ n,
    @LE.le (D n)
      (ChainCompletePartialOrder.instOfCompleteLattice
        (α := D n)).toPartialOrder.toPreorder.toLE
      (x.1 n) (y.1 n)

namespace inverseLimitLE

/-- Reflexivity of the coordinatewise inverse-limit order. -/
theorem refl (x : InverseLimit D P) :
    inverseLimitLE D P x x :=
  fun n => le_refl (x.1 n)

/-- Transitivity of the coordinatewise inverse-limit order. -/
theorem trans (x y z : InverseLimit D P)
    (hxy : inverseLimitLE D P x y) (hyz : inverseLimitLE D P y z) :
    inverseLimitLE D P x z :=
  fun n => le_trans (hxy n) (hyz n)

/-- Antisymmetry of the coordinatewise inverse-limit order. -/
```

```
theorem antisymm (x y : InverseLimit D P)
    (hxy : inverseLimitLE D P x y) (hyx : inverseLimitLE D P y x) :
    x = y :=
  Subtype.ext (funext fun n => le_antisymm (hxy n) (hyx n))

end inverseLimitLE

end InverseLimit

end DomainTheory.ContinuousLattice
```

1

inverseLimitLE.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- Coordinatewise order on compatible sequences. Kept reducible so existing
subtype-order lemmas see the underlying pointwise relation definitionally. -/
abbrev inverseLimitLE (x y : InverseLimit D P) : Prop :=
  ∀ n,
    @LE.le (D n)
      (ChainCompletePartialOrder.instOfCompleteLattice
        (α := D n)).toPartialOrder.toPreorder.toLE
      (x.1 n) (y.1 n)

namespace inverseLimitLE

/-- Reflexivity of the coordinatewise inverse-limit order. -/
theorem refl (x : InverseLimit D P) :
    inverseLimitLE D P x x :=
  fun n => le_refl (x.1 n)

/-- Transitivity of the coordinatewise inverse-limit order. -/
theorem trans (x y z : InverseLimit D P)
    (hxy : inverseLimitLE D P x y) (hyz : inverseLimitLE D P y z) :
    inverseLimitLE D P x z :=
  fun n => le_trans (hxy n) (hyz n)
```

```
/-- Antisymmetry of the coordinatewise inverse-limit order. -/
theorem antisymm (x y : InverseLimit D P)
    (hxy : inverseLimitLE D P x y) (hyx : inverseLimitLE D P y x) :
    x = y :=
  Subtype.ext (funext fun n => le_antisymm (hxy n) (hyx n))

end inverseLimitLE

end InverseLimit

end DomainTheory.ContinuousLattice
```

2

inverseLimitLE.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- Coordinatewise order on compatible sequences. Kept reducible so existing
subtype-order lemmas see the underlying pointwise relation definitionally. -/
abbrev inverseLimitLE (x y : InverseLimit D P) : Prop :=
  ∀ n,
    @LE.le (D n)
      (ChainCompletePartialOrder.instOfCompleteLattice
        (α := D n)).toPartialOrder.toPreorder.toLE
      (x.1 n) (y.1 n)

namespace inverseLimitLE

/-- Reflexivity of the coordinatewise inverse-limit order. -/
theorem refl (x : InverseLimit D P) :
    inverseLimitLE D P x x :=
  fun n => le_refl (x.1 n)

/-- Transitivity of the coordinatewise inverse-limit order. -/
theorem trans (x y z : InverseLimit D P)
    (hxy : inverseLimitLE D P x y) (hyz : inverseLimitLE D P y z) :
    inverseLimitLE D P x z :=
  fun n => le_trans (hxy n) (hyz n)
```

```
/-- Antisymmetry of the coordinatewise inverse-limit order. -/
theorem antisymm (x y : InverseLimit D P)
    (hxy : inverseLimitLE D P x y) (hyx : inverseLimitLE D P y x) :
    x = y :=
  Subtype.ext (funext fun n => le_antisymm (hxy n) (hyx n))

end inverseLimitLE

end InverseLimit

end DomainTheory.ContinuousLattice
```

3

inverseLimitLE.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- Coordinatewise order on compatible sequences. Kept reducible so existing
subtype-order lemmas see the underlying pointwise relation definitionally. -/
abbrev inverseLimitLE (x y : InverseLimit D P) : Prop :=
  ∀ n,
    @LE.le (D n)
      (ChainCompletePartialOrder.instOfCompleteLattice
        (α := D n)).toPartialOrder.toPreorder.toLE
      (x.1 n) (y.1 n)

namespace inverseLimitLE

/-- Reflexivity of the coordinatewise inverse-limit order. -/
theorem refl (x : InverseLimit D P) :
    inverseLimitLE D P x x :=
  fun n => le_refl (x.1 n)

/-- Transitivity of the coordinatewise inverse-limit order. -/
theorem trans (x y z : InverseLimit D P)
    (hxy : inverseLimitLE D P x y) (hyz : inverseLimitLE D P y z) :
    inverseLimitLE D P x z :=
```

```
  fun n => le_trans (hxy n) (hyz n)

/-- Antisymmetry of the coordinatewise inverse-limit order. -/
theorem antisymm (x y : InverseLimit D P)
    (hxy : inverseLimitLE D P x y) (hyx : inverseLimitLE D P y x) :
    x = y :=
  Subtype.ext (funext fun n => le_antisymm (hxy n) (hyx n))

end inverseLimitLE

end InverseLimit

end DomainTheory.ContinuousLattice
```

4

inverseLimitLE.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- Coordinatewise order on compatible sequences. Kept reducible so existing
subtype-order lemmas see the underlying pointwise relation definitionally. -/
abbrev inverseLimitLE (x y : InverseLimit D P) : Prop :=
  ∀ n,
    @LE.le (D n)
      (ChainCompletePartialOrder.instOfCompleteLattice
        (α := D n)).toPartialOrder.toPreorder.toLE
      (x.1 n) (y.1 n)

namespace inverseLimitLE

/-- Reflexivity of the coordinatewise inverse-limit order. -/
theorem refl (x : InverseLimit D P) :
    inverseLimitLE D P x x :=
  fun n => le_refl (x.1 n)

/-- Transitivity of the coordinatewise inverse-limit order. -/
theorem trans (x y z : InverseLimit D P)
    (hxy : inverseLimitLE D P x y) (hyz : inverseLimitLE D P y z) :
```

```
    inverseLimitLE D P x z :=
  fun n => le_trans (hxy n) (hyz n)

/-- Antisymmetry of the coordinatewise inverse-limit order. -/
theorem antisymm (x y : InverseLimit D P)
    (hxy : inverseLimitLE D P x y) (hyx : inverseLimitLE D P y x) :
    x = y :=
  Subtype.ext (funext fun n => le_antisymm (hxy n) (hyx n))

end inverseLimitLE

end InverseLimit

end DomainTheory.ContinuousLattice
```

5

inverseLimitLE.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- Coordinatewise order on compatible sequences. Kept reducible so existing
subtype-order lemmas see the underlying pointwise relation definitionally. -/
abbrev inverseLimitLE (x y : InverseLimit D P) : Prop :=
  ∀ n,
    @LE.le (D n)
      (ChainCompletePartialOrder.instOfCompleteLattice
        (α := D n)).toPartialOrder.toPreorder.toLE
      (x.1 n) (y.1 n)

namespace inverseLimitLE

/-- Reflexivity of the coordinatewise inverse-limit order. -/
theorem refl (x : InverseLimit D P) :
    inverseLimitLE D P x x :=
  fun n => le_refl (x.1 n)

/-- Transitivity of the coordinatewise inverse-limit order. -/
theorem trans (x y z : InverseLimit D P)
```

```
    (hxy : inverseLimitLE D P x y) (hyz : inverseLimitLE D P y z) :
    inverseLimitLE D P x z :=
  fun n => le_trans (hxy n) (hyz n)

/-- Antisymmetry of the coordinatewise inverse-limit order. -/
theorem antisymm (x y : InverseLimit D P)
    (hxy : inverseLimitLE D P x y) (hyx : inverseLimitLE D P y x) :
    x = y :=
  Subtype.ext (funext fun n => le_antisymm (hxy n) (hyx n))

end inverseLimitLE

end InverseLimit

end DomainTheory.ContinuousLattice
```

6

inverseLimitLE.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- Coordinatewise order on compatible sequences. Kept reducible so existing
subtype-order lemmas see the underlying pointwise relation definitionally. -/
abbrev inverseLimitLE (x y : InverseLimit D P) : Prop :=
  ∀ n,
    @LE.le (D n)
      (ChainCompletePartialOrder.instOfCompleteLattice
        (α := D n)).toPartialOrder.toPreorder.toLE
      (x.1 n) (y.1 n)

namespace inverseLimitLE

/-- Reflexivity of the coordinatewise inverse-limit order. -/
theorem refl (x : InverseLimit D P) :
    inverseLimitLE D P x x :=
  fun n => le_refl (x.1 n)

/-- Transitivity of the coordinatewise inverse-limit order. -/
```

```
theorem trans (x y z : InverseLimit D P)
    (hxy : inverseLimitLE D P x y) (hyz : inverseLimitLE D P y z) :
    inverseLimitLE D P x z :=
  fun n => le_trans (hxy n) (hyz n)

/-- Antisymmetry of the coordinatewise inverse-limit order. -/
theorem antisymm (x y : InverseLimit D P)
    (hxy : inverseLimitLE D P x y) (hyx : inverseLimitLE D P y x) :
    x = y :=
  Subtype.ext (funext fun n => le_antisymm (hxy n) (hyx n))

end inverseLimitLE

end InverseLimit

end DomainTheory.ContinuousLattice
```

7

inverseLimitLE.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- Coordinatewise order on compatible sequences. Kept reducible so existing
subtype-order lemmas see the underlying pointwise relation definitionally. -/
abbrev inverseLimitLE (x y : InverseLimit D P) : Prop :=
  ∀ n,
    @LE.le (D n)
      (ChainCompletePartialOrder.instOfCompleteLattice
        (α := D n)).toPartialOrder.toPreorder.toLE
      (x.1 n) (y.1 n)

namespace inverseLimitLE

/-- Reflexivity of the coordinatewise inverse-limit order. -/
theorem refl (x : InverseLimit D P) :
    inverseLimitLE D P x x :=
  fun n => le_refl (x.1 n)
```

```
/-- Transitivity of the coordinatewise inverse-limit order. -/
theorem trans (x y z : InverseLimit D P)
    (hxy : inverseLimitLE D P x y) (hyz : inverseLimitLE D P y z) :
    inverseLimitLE D P x z :=
  fun n => le_trans (hxy n) (hyz n)

/-- Antisymmetry of the coordinatewise inverse-limit order. -/
theorem antisymm (x y : InverseLimit D P)
    (hxy : inverseLimitLE D P x y) (hyx : inverseLimitLE D P y x) :
    x = y :=
  Subtype.ext (funext fun n => le_antisymm (hxy n) (hyx n))

end inverseLimitLE

end InverseLimit

end DomainTheory.ContinuousLattice
```

8

inverseLimitLE.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- Coordinatewise order on compatible sequences. Kept reducible so existing
subtype-order lemmas see the underlying pointwise relation definitionally. -/
abbrev inverseLimitLE (x y : InverseLimit D P) : Prop :=
  ∀ n,
    @LE.le (D n)
      (ChainCompletePartialOrder.instOfCompleteLattice
        (α := D n)).toPartialOrder.toPreorder.toLE
      (x.1 n) (y.1 n)

namespace inverseLimitLE

/-- Reflexivity of the coordinatewise inverse-limit order. -/
theorem refl (x : InverseLimit D P) :
    inverseLimitLE D P x x :=
  fun n => le_refl (x.1 n)
```

```
/-- Transitivity of the coordinatewise inverse-limit order. -/
theorem trans (x y z : InverseLimit D P)
    (hxy : inverseLimitLE D P x y) (hyz : inverseLimitLE D P y z) :
    inverseLimitLE D P x z :=
  fun n => le_trans (hxy n) (hyz n)

/-- Antisymmetry of the coordinatewise inverse-limit order. -/
theorem antisymm (x y : InverseLimit D P)
    (hxy : inverseLimitLE D P x y) (hyx : inverseLimitLE D P y x) :
    x = y :=
  Subtype.ext (funext fun n => le_antisymm (hxy n) (hyx n))

end inverseLimitLE

end InverseLimit

end DomainTheory.ContinuousLattice
```

9

inverseLimitLT.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.inverseLimitLE
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- Strict coordinatewise order induced by `inverseLimitLE`. -/
abbrev inverseLimitLT (x y : InverseLimit D P) : Prop :=
  inverseLimitLE D P x y ∧ ¬ inverseLimitLE D P y x

namespace inverseLimitLT

/-- The chosen strict inverse-limit order is `≤ ∧ ¬ ≥`. -/
theorem iff_LE_not_GE (x y : InverseLimit D P) :
    inverseLimitLT D P x y ↔
      inverseLimitLE D P x y ∧ ¬ inverseLimitLE D P y x :=
  Iff.rfl

end inverseLimitLT
```

```
end InverseLimit

end DomainTheory.ContinuousLattice
```

0

inverseLimitLT.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.inverseLimitLE
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- Strict coordinatewise order induced by `inverseLimitLE`. -/
abbrev inverseLimitLT (x y : InverseLimit D P) : Prop :=
  inverseLimitLE D P x y ∧ ¬ inverseLimitLE D P y x

namespace inverseLimitLT

/-- The chosen strict inverse-limit order is `≤ ∧ ¬ ≥`. -/
theorem iff_LE_not_GE (x y : InverseLimit D P) :
    inverseLimitLT D P x y ↔
      inverseLimitLE D P x y ∧ ¬ inverseLimitLE D P y x :=
  Iff.rfl

end inverseLimitLT

end InverseLimit

end DomainTheory.ContinuousLattice
```

1

inverseLimitLT.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.inverseLimitLE
public import Mathlib.Data.Set.Basic

@[expose] public section
```

```
namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- Strict coordinatewise order induced by `inverseLimitLE`. -/
abbrev inverseLimitLT (x y : InverseLimit D P) : Prop :=
  inverseLimitLE D P x y ∧ ¬ inverseLimitLE D P y x

namespace inverseLimitLT

/-- The chosen strict inverse-limit order is `≤ ∧ ¬ ≥`. -/
theorem iff_LE_not_GE (x y : InverseLimit D P) :
    inverseLimitLT D P x y ↔
      inverseLimitLE D P x y ∧ ¬ inverseLimitLE D P y x :=
  Iff.rfl

end inverseLimitLT

end InverseLimit

end DomainTheory.ContinuousLattice
```

2

inverseLimitLT.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.inverseLimitLE
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- Strict coordinatewise order induced by `inverseLimitLE`. -/
abbrev inverseLimitLT (x y : InverseLimit D P) : Prop :=
  inverseLimitLE D P x y ∧ ¬ inverseLimitLE D P y x
```

```
namespace inverseLimitLT

/-- The chosen strict inverse-limit order is `≤ ∧ ¬ ≥`. -/
theorem iff_LE_not_GE (x y : InverseLimit D P) :
    inverseLimitLT D P x y ↔
      inverseLimitLE D P x y ∧ ¬ inverseLimitLE D P y x :=
  Iff.rfl

end inverseLimitLT

end InverseLimit

end DomainTheory.ContinuousLattice
```

3

inverseLimitLT.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.inverseLimitLE
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- Strict coordinatewise order induced by `inverseLimitLE`. -/
abbrev inverseLimitLT (x y : InverseLimit D P) : Prop :=
  inverseLimitLE D P x y ∧ ¬ inverseLimitLE D P y x

namespace inverseLimitLT

/-- The chosen strict inverse-limit order is `≤ ∧ ¬ ≥`. -/
theorem iff_LE_not_GE (x y : InverseLimit D P) :
    inverseLimitLT D P x y ↔
      inverseLimitLE D P x y ∧ ¬ inverseLimitLE D P y x :=
  Iff.rfl

end inverseLimitLT

end InverseLimit

end DomainTheory.ContinuousLattice
```

4

inverseLimitLT.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.inverseLimitLE
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- Strict coordinatewise order induced by `inverseLimitLE`. -/
abbrev inverseLimitLT (x y : InverseLimit D P) : Prop :=
  inverseLimitLE D P x y ∧ ¬ inverseLimitLE D P y x

namespace inverseLimitLT

/-- The chosen strict inverse-limit order is `≤ ∧ ¬ ≥`. -/
theorem iff_LE_not_GE (x y : InverseLimit D P) :
    inverseLimitLT D P x y ↔
      inverseLimitLE D P x y ∧ ¬ inverseLimitLE D P y x :=
  Iff.rfl

end inverseLimitLT

end InverseLimit

end DomainTheory.ContinuousLattice
```

5

inverseLimitLT.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.inverseLimitLE
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set
```

```
universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- Strict coordinatewise order induced by `inverseLimitLE`. -/
abbrev inverseLimitLT (x y : InverseLimit D P) : Prop :=
  inverseLimitLE D P x y ∧ ¬ inverseLimitLE D P y x

namespace inverseLimitLT

/-- The chosen strict inverse-limit order is `≤ ∧ ¬ ≥`. -/
theorem iff_LE_not_GE (x y : InverseLimit D P) :
    inverseLimitLT D P x y ↔
      inverseLimitLE D P x y ∧ ¬ inverseLimitLE D P y x :=
  Iff.rfl

end inverseLimitLT

end InverseLimit

end DomainTheory.ContinuousLattice
```

6

inverseLimitLT.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.inverseLimitLE
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- Strict coordinatewise order induced by `inverseLimitLE`. -/
abbrev inverseLimitLT (x y : InverseLimit D P) : Prop :=
  inverseLimitLE D P x y ∧ ¬ inverseLimitLE D P y x

namespace inverseLimitLT

/-- The chosen strict inverse-limit order is `≤ ∧ ¬ ≥`. -/
```

```
theorem iff_LE_not_GE (x y : InverseLimit D P) :
    inverseLimitLT D P x y ↔
      inverseLimitLE D P x y ∧ ¬ inverseLimitLE D P y x :=
  Iff.rfl

end inverseLimitLT

end InverseLimit

end DomainTheory.ContinuousLattice
```

7

inverseLimitLT.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.inverseLimitLE
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- Strict coordinatewise order induced by `inverseLimitLE`. -/
abbrev inverseLimitLT (x y : InverseLimit D P) : Prop :=
  inverseLimitLE D P x y ∧ ¬ inverseLimitLE D P y x

namespace inverseLimitLT

/-- The chosen strict inverse-limit order is `≤ ∧ ¬ ≥`. -/
theorem iff_LE_not_GE (x y : InverseLimit D P) :
    inverseLimitLT D P x y ↔
      inverseLimitLE D P x y ∧ ¬ inverseLimitLE D P y x :=
  Iff.rfl

end inverseLimitLT

end InverseLimit

end DomainTheory.ContinuousLattice
```

8

inverseLimitLT.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.inverseLimitLE
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- Strict coordinatewise order induced by `inverseLimitLE`. -/
abbrev inverseLimitLT (x y : InverseLimit D P) : Prop :=
  inverseLimitLE D P x y ∧ ¬ inverseLimitLE D P y x

namespace inverseLimitLT

/-- The chosen strict inverse-limit order is `≤ ∧ ¬ ≥`. -/
theorem iff_LE_not_GE (x y : InverseLimit D P) :
    inverseLimitLT D P x y ↔
      inverseLimitLE D P x y ∧ ¬ inverseLimitLE D P y x :=
  Iff.rfl

end inverseLimitLT

end InverseLimit

end DomainTheory.ContinuousLattice
```

9

inverseLimitPartialOrder.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.inverseLimitLE
public import DomainTheory.ContinuousLattice.InverseLimits.inverseLimitLT
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit
```

```
variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- The inverse limit carries the coordinatewise order inherited from its
ambient product. This declaration makes that order explicit instead of
leaving typeclass search to choose an equivalent route. -/
instance inverseLimitPartialOrder : PartialOrder (InverseLimit D P) where
  le := inverseLimitLE D P
  lt := inverseLimitLT D P
  le_refl := inverseLimitLE.refl D P
  le_trans := inverseLimitLE.trans D P
  lt_iff_le_not_ge := inverseLimitLT.iff_LE_not_GE D P
  le_antisymm := inverseLimitLE.antisymm D P

end InverseLimit

end DomainTheory.ContinuousLattice
```

0

inverseLimitPartialOrder.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.inverseLimitLE
public import DomainTheory.ContinuousLattice.InverseLimits.inverseLimitLT
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- The inverse limit carries the coordinatewise order inherited from its
ambient product. This declaration makes that order explicit instead of
leaving typeclass search to choose an equivalent route. -/
instance inverseLimitPartialOrder : PartialOrder (InverseLimit D P) where
  le := inverseLimitLE D P
  lt := inverseLimitLT D P
  le_refl := inverseLimitLE.refl D P
  le_trans := inverseLimitLE.trans D P
  lt_iff_le_not_ge := inverseLimitLT.iff_LE_not_GE D P
  le_antisymm := inverseLimitLE.antisymm D P

end InverseLimit
```

```
end DomainTheory.ContinuousLattice
```

1

inverseLimitPartialOrder.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.inverseLimitLE
public import DomainTheory.ContinuousLattice.InverseLimits.inverseLimitLT
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- The inverse limit carries the coordinatewise order inherited from its
ambient product. This declaration makes that order explicit instead of
leaving typeclass search to choose an equivalent route. -/
instance inverseLimitPartialOrder : PartialOrder (InverseLimit D P) where
  le := inverseLimitLE D P
  lt := inverseLimitLT D P
  le_refl := inverseLimitLE.refl D P
  le_trans := inverseLimitLE.trans D P
  lt_iff_le_not_ge := inverseLimitLT.iff_LE_not_GE D P
  le_antisymm := inverseLimitLE.antisymm D P

end InverseLimit

end DomainTheory.ContinuousLattice
```

2

inverseLimitPartialOrder.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.inverseLimitLE
public import DomainTheory.ContinuousLattice.InverseLimits.inverseLimitLT
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice
```

```
open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- The inverse limit carries the coordinatewise order inherited from its
ambient product. This declaration makes that order explicit instead of
leaving typeclass search to choose an equivalent route. -/
instance inverseLimitPartialOrder : PartialOrder (InverseLimit D P) where
  le := inverseLimitLE D P
  lt := inverseLimitLT D P
  le_refl := inverseLimitLE.refl D P
  le_trans := inverseLimitLE.trans D P
  lt_iff_le_not_ge := inverseLimitLT.iff_LE_not_GE D P
  le_antisymm := inverseLimitLE.antisymm D P

end InverseLimit

end DomainTheory.ContinuousLattice
```

3

inverseLimitPartialOrder.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.inverseLimitLE
public import DomainTheory.ContinuousLattice.InverseLimits.inverseLimitLT
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- The inverse limit carries the coordinatewise order inherited from its
ambient product. This declaration makes that order explicit instead of
leaving typeclass search to choose an equivalent route. -/
instance inverseLimitPartialOrder : PartialOrder (InverseLimit D P) where
  le := inverseLimitLE D P
  lt := inverseLimitLT D P
  le_refl := inverseLimitLE.refl D P
```

```
  le_trans := inverseLimitLE.trans D P
  lt_iff_le_not_ge := inverseLimitLT.iff_LE_not_GE D P
  le_antisymm := inverseLimitLE.antisymm D P

end InverseLimit

end DomainTheory.ContinuousLattice
```

4

inverseLimitPartialOrder.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.inverseLimitLE
public import DomainTheory.ContinuousLattice.InverseLimits.inverseLimitLT
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- The inverse limit carries the coordinatewise order inherited from its
ambient product. This declaration makes that order explicit instead of
leaving typeclass search to choose an equivalent route. -/
instance inverseLimitPartialOrder : PartialOrder (InverseLimit D P) where
  le := inverseLimitLE D P
  lt := inverseLimitLT D P
  le_refl := inverseLimitLE.refl D P
  le_trans := inverseLimitLE.trans D P
  lt_iff_le_not_ge := inverseLimitLT.iff_LE_not_GE D P
  le_antisymm := inverseLimitLE.antisymm D P

end InverseLimit

end DomainTheory.ContinuousLattice
```

5

inverseLimitPartialOrder.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.inverseLimitLE
public import DomainTheory.ContinuousLattice.InverseLimits.inverseLimitLT
public import Mathlib.Data.Set.Basic
```

```
@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- The inverse limit carries the coordinatewise order inherited from its
ambient product. This declaration makes that order explicit instead of
leaving typeclass search to choose an equivalent route. -/
instance inverseLimitPartialOrder : PartialOrder (InverseLimit D P) where
  le := inverseLimitLE D P
  lt := inverseLimitLT D P
  le_refl := inverseLimitLE.refl D P
  le_trans := inverseLimitLE.trans D P
  lt_iff_le_not_ge := inverseLimitLT.iff_LE_not_GE D P
  le_antisymm := inverseLimitLE.antisymm D P

end InverseLimit

end DomainTheory.ContinuousLattice
```

6

inverseLimitPartialOrder.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.inverseLimitLE
public import DomainTheory.ContinuousLattice.InverseLimits.inverseLimitLT
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- The inverse limit carries the coordinatewise order inherited from its
ambient product. This declaration makes that order explicit instead of
```

```
leaving typeclass search to choose an equivalent route. -/
instance inverseLimitPartialOrder : PartialOrder (InverseLimit D P) where
  le := inverseLimitLE D P
  lt := inverseLimitLT D P
  le_refl := inverseLimitLE.refl D P
  le_trans := inverseLimitLE.trans D P
  lt_iff_le_not_ge := inverseLimitLT.iff_LE_not_GE D P
  le_antisymm := inverseLimitLE.antisymm D P

end InverseLimit

end DomainTheory.ContinuousLattice
```

7

inverseLimitPartialOrder.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.inverseLimitLE
public import DomainTheory.ContinuousLattice.InverseLimits.inverseLimitLT
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- The inverse limit carries the coordinatewise order inherited from its
ambient product. This declaration makes that order explicit instead of
leaving typeclass search to choose an equivalent route. -/
instance inverseLimitPartialOrder : PartialOrder (InverseLimit D P) where
  le := inverseLimitLE D P
  lt := inverseLimitLT D P
  le_refl := inverseLimitLE.refl D P
  le_trans := inverseLimitLE.trans D P
  lt_iff_le_not_ge := inverseLimitLT.iff_LE_not_GE D P
  le_antisymm := inverseLimitLE.antisymm D P

end InverseLimit

end DomainTheory.ContinuousLattice
```

8

inverseLimitPartialOrder.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.inverseLimitLE
public import DomainTheory.ContinuousLattice.InverseLimits.inverseLimitLT
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- The inverse limit carries the coordinatewise order inherited from its
ambient product. This declaration makes that order explicit instead of
leaving typeclass search to choose an equivalent route. -/
instance inverseLimitPartialOrder : PartialOrder (InverseLimit D P) where
  le := inverseLimitLE D P
  lt := inverseLimitLT D P
  le_refl := inverseLimitLE.refl D P
  le_trans := inverseLimitLE.trans D P
  lt_iff_le_not_ge := inverseLimitLT.iff_LE_not_GE D P
  le_antisymm := inverseLimitLE.antisymm D P

end InverseLimit

end DomainTheory.ContinuousLattice
```

9

inverseLimitSInfCoe.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]
```

```
variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- The coordinatewise infimum of a family of compatible sequences, before
packaging the compatibility proof. -/
noncomputable def inverseLimitSInfCoe (S : Set (InverseLimit D P)) : ∀ n, D n :=
  fun n => sInf ((fun x : InverseLimit D P => x.1 n) '' S)

end InverseLimit

end DomainTheory.ContinuousLattice
```

0

inverseLimitSInfCoe.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- The coordinatewise infimum of a family of compatible sequences, before
packaging the compatibility proof. -/
noncomputable def inverseLimitSInfCoe (S : Set (InverseLimit D P)) : ∀ n, D n :=
  fun n => sInf ((fun x : InverseLimit D P => x.1 n) '' S)

end InverseLimit

end DomainTheory.ContinuousLattice
```

1

inverseLimitSInfCoe.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice
```

```
open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- The coordinatewise infimum of a family of compatible sequences, before
packaging the compatibility proof. -/
noncomputable def inverseLimitSInfCoe (S : Set (InverseLimit D P)) : ∀ n, D n :=
  fun n => sInf ((fun x : InverseLimit D P => x.1 n) '' S)

end InverseLimit

end DomainTheory.ContinuousLattice
```

2

inverseLimitSInfCoe.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- The coordinatewise infimum of a family of compatible sequences, before
packaging the compatibility proof. -/
noncomputable def inverseLimitSInfCoe (S : Set (InverseLimit D P)) : ∀ n, D n :=
  fun n => sInf ((fun x : InverseLimit D P => x.1 n) '' S)

end InverseLimit

end DomainTheory.ContinuousLattice
```

3

inverseLimitSInfCoe.lean

```
module
```

```
public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- The coordinatewise infimum of a family of compatible sequences, before
packaging the compatibility proof. -/
noncomputable def inverseLimitSInfCoe (S : Set (InverseLimit D P)) : ∀ n, D n :=
  fun n => sInf ((fun x : InverseLimit D P => x.1 n) '' S)

end InverseLimit

end DomainTheory.ContinuousLattice
```

4

inverseLimitSInfCoe.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- The coordinatewise infimum of a family of compatible sequences, before
packaging the compatibility proof. -/
noncomputable def inverseLimitSInfCoe (S : Set (InverseLimit D P)) : ∀ n, D n :=
  fun n => sInf ((fun x : InverseLimit D P => x.1 n) '' S)

end InverseLimit
```

```
end DomainTheory.ContinuousLattice
```

5

inverseLimitSInfCoe.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- The coordinatewise infimum of a family of compatible sequences, before
packaging the compatibility proof. -/
noncomputable def inverseLimitSInfCoe (S : Set (InverseLimit D P)) : ∀ n, D n :=
  fun n => sInf ((fun x : InverseLimit D P => x.1 n) '' S)

end InverseLimit

end DomainTheory.ContinuousLattice
```

6

inverseLimitSInfCoe.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]
```

```
variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- The coordinatewise infimum of a family of compatible sequences, before
packaging the compatibility proof. -/
noncomputable def inverseLimitSInfCoe (S : Set (InverseLimit D P)) : ∀ n, D n :=
  fun n => sInf ((fun x : InverseLimit D P => x.1 n) '' S)

end InverseLimit

end DomainTheory.ContinuousLattice
```

7

inverseLimitSInfCoe.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- The coordinatewise infimum of a family of compatible sequences, before
packaging the compatibility proof. -/
noncomputable def inverseLimitSInfCoe (S : Set (InverseLimit D P)) : ∀ n, D n :=
  fun n => sInf ((fun x : InverseLimit D P => x.1 n) '' S)

end InverseLimit

end DomainTheory.ContinuousLattice
```

8

inverseLimitSInfCoe.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice
```

```
open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- The coordinatewise infimum of a family of compatible sequences, before
packaging the compatibility proof. -/
noncomputable def inverseLimitSInfCoe (S : Set (InverseLimit D P)) : ∀ n, D n :=
  fun n => sInf ((fun x : InverseLimit D P => x.1 n) '' S)

end InverseLimit

end DomainTheory.ContinuousLattice
```

9

instInfSet.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.inverseLimitPartialOrder
public import DomainTheory.ContinuousLattice.InverseLimits.inverseLimitSInfCoe
public import DomainTheory.ContinuousLattice.InverseLimits.projection_galoisConnection
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- `jₙ` preserves arbitrary infima (it is the upper adjoint of `iₙ`). -/
theorem retr_sInf (n : ℕ) (A : Set (D (n + 1))) :
    (P n).retr (sInf A) = sInf ((P n).retr '' A) := by
  rw [(projection_galoisConnection D P n).u_sInf, sInf_image]

/-- The pointwise infimum of compatible sequences is compatible. -/
theorem compatible_sInf (S : Set (InverseLimit D P)) :
    Compatible D P (inverseLimitSInfCoe D P S) := by
  intro n
  rw [inverseLimitSInfCoe, inverseLimitSInfCoe, retr_sInf]
```

```
  congr 1
  rw [Set.image_image]
  exact Set.image_congr (by rintro x hx; exact x.2 n)

noncomputable instance instInfSet : InfSet (InverseLimit D P) :=
  ⟨fun S => ⟨inverseLimitSInfCoe D P S, compatible_sInf D P S⟩⟩

theorem coe_sInf (S : Set (InverseLimit D P)) :
    ((sInf S : InverseLimit D P) : ∀ n, D n) = inverseLimitSInfCoe D P S := rfl

theorem isGLB_sInf' (S : Set (InverseLimit D P)) :
    @IsGLB (InverseLimit D P)
      (inverseLimitPartialOrder D P).toPreorder.toLE S (sInf S) := by
  constructor
  · intro x hx n
    exact sInf_le (Set.mem_image_of_mem _ hx)
  · intro b hb n
    refine le_sInf ?_
    rintro _ ⟨x, hx, rfl⟩
    exact hb hx n

end InverseLimit

end DomainTheory.ContinuousLattice
```

0

instInfSet.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.inverseLimitPartialOrder
public import DomainTheory.ContinuousLattice.InverseLimits.inverseLimitSInfCoe
public import DomainTheory.ContinuousLattice.InverseLimits.projection_galoisConnection
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- `jₙ` preserves arbitrary infima (it is the upper adjoint of `iₙ`). -/
theorem retr_sInf (n : ℕ) (A : Set (D (n + 1))) :
    (P n).retr (sInf A) = sInf ((P n).retr '' A) := by
  rw [(projection_galoisConnection D P n).u_sInf, sInf_image]
```

```
/-- The pointwise infimum of compatible sequences is compatible. -/
theorem compatible_sInf (S : Set (InverseLimit D P)) :
    Compatible D P (inverseLimitSInfCoe D P S) := by
  intro n
  rw [inverseLimitSInfCoe, inverseLimitSInfCoe, retr_sInf]
  congr 1
  rw [Set.image_image]
  exact Set.image_congr (by rintro x hx; exact x.2 n)

noncomputable instance instInfSet : InfSet (InverseLimit D P) :=
  ⟨fun S => ⟨inverseLimitSInfCoe D P S, compatible_sInf D P S⟩⟩

theorem coe_sInf (S : Set (InverseLimit D P)) :
    ((sInf S : InverseLimit D P) : ∀ n, D n) = inverseLimitSInfCoe D P S := rfl

theorem isGLB_sInf' (S : Set (InverseLimit D P)) :
    @IsGLB (InverseLimit D P)
      (inverseLimitPartialOrder D P).toPreorder.toLE S (sInf S) := by
  constructor
  · intro x hx n
    exact sInf_le (Set.mem_image_of_mem _ hx)
  · intro b hb n
    refine le_sInf ?_
    rintro _ ⟨x, hx, rfl⟩
    exact hb hx n

end InverseLimit

end DomainTheory.ContinuousLattice
```

1

instInfSet.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.inverseLimitPartialOrder
public import DomainTheory.ContinuousLattice.InverseLimits.inverseLimitSInfCoe
public import DomainTheory.ContinuousLattice.InverseLimits.projection_galoisConnection
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]
```

```
variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- `jₙ` preserves arbitrary infima (it is the upper adjoint of `iₙ`). -/
theorem retr_sInf (n : ℕ) (A : Set (D (n + 1))) :
    (P n).retr (sInf A) = sInf ((P n).retr '' A) := by
  rw [(projection_galoisConnection D P n).u_sInf, sInf_image]

/-- The pointwise infimum of compatible sequences is compatible. -/
theorem compatible_sInf (S : Set (InverseLimit D P)) :
    Compatible D P (inverseLimitSInfCoe D P S) := by
  intro n
  rw [inverseLimitSInfCoe, inverseLimitSInfCoe, retr_sInf]
  congr 1
  rw [Set.image_image]
  exact Set.image_congr (by rintro x hx; exact x.2 n)

noncomputable instance instInfSet : InfSet (InverseLimit D P) :=
  ⟨fun S => ⟨inverseLimitSInfCoe D P S, compatible_sInf D P S⟩⟩

theorem coe_sInf (S : Set (InverseLimit D P)) :
    ((sInf S : InverseLimit D P) : ∀ n, D n) = inverseLimitSInfCoe D P S := rfl

theorem isGLB_sInf' (S : Set (InverseLimit D P)) :
    @IsGLB (InverseLimit D P)
      (inverseLimitPartialOrder D P).toPreorder.toLE S (sInf S) := by
  constructor
  · intro x hx n
    exact sInf_le (Set.mem_image_of_mem _ hx)
  · intro b hb n
    refine le_sInf ?_
    rintro _ ⟨x, hx, rfl⟩
    exact hb hx n

end InverseLimit

end DomainTheory.ContinuousLattice
```

2

instInfSet.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.inverseLimitPartialOrder
public import DomainTheory.ContinuousLattice.InverseLimits.inverseLimitSInfCoe
public import DomainTheory.ContinuousLattice.InverseLimits.projection_galoisConnection
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set
```

```
universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- `jₙ` preserves arbitrary infima (it is the upper adjoint of `iₙ`). -/
theorem retr_sInf (n : ℕ) (A : Set (D (n + 1))) :
    (P n).retr (sInf A) = sInf ((P n).retr '' A) := by
  rw [(projection_galoisConnection D P n).u_sInf, sInf_image]

/-- The pointwise infimum of compatible sequences is compatible. -/
theorem compatible_sInf (S : Set (InverseLimit D P)) :
    Compatible D P (inverseLimitSInfCoe D P S) := by
  intro n
  rw [inverseLimitSInfCoe, inverseLimitSInfCoe, retr_sInf]
  congr 1
  rw [Set.image_image]
  exact Set.image_congr (by rintro x hx; exact x.2 n)

noncomputable instance instInfSet : InfSet (InverseLimit D P) :=
  ⟨fun S => ⟨inverseLimitSInfCoe D P S, compatible_sInf D P S⟩⟩

theorem coe_sInf (S : Set (InverseLimit D P)) :
    ((sInf S : InverseLimit D P) : ∀ n, D n) = inverseLimitSInfCoe D P S := rfl

theorem isGLB_sInf' (S : Set (InverseLimit D P)) :
    @IsGLB (InverseLimit D P)
      (inverseLimitPartialOrder D P).toPreorder.toLE S (sInf S) := by
  constructor
  · intro x hx n
    exact sInf_le (Set.mem_image_of_mem _ hx)
  · intro b hb n
    refine le_sInf ?_
    rintro _ ⟨x, hx, rfl⟩
    exact hb hx n

end InverseLimit

end DomainTheory.ContinuousLattice
```

3

instInfSet.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.inverseLimitPartialOrder
public import DomainTheory.ContinuousLattice.InverseLimits.inverseLimitSInfCoe
public import DomainTheory.ContinuousLattice.InverseLimits.projection_galoisConnection
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Data.Set.Basic
```

```
@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- `jₙ` preserves arbitrary infima (it is the upper adjoint of `iₙ`). -/
theorem retr_sInf (n : ℕ) (A : Set (D (n + 1))) :
    (P n).retr (sInf A) = sInf ((P n).retr '' A) := by
  rw [(projection_galoisConnection D P n).u_sInf, sInf_image]

/-- The pointwise infimum of compatible sequences is compatible. -/
theorem compatible_sInf (S : Set (InverseLimit D P)) :
    Compatible D P (inverseLimitSInfCoe D P S) := by
  intro n
  rw [inverseLimitSInfCoe, inverseLimitSInfCoe, retr_sInf]
  congr 1
  rw [Set.image_image]
  exact Set.image_congr (by rintro x hx; exact x.2 n)

noncomputable instance instInfSet : InfSet (InverseLimit D P) :=
  ⟨fun S => ⟨inverseLimitSInfCoe D P S, compatible_sInf D P S⟩⟩

theorem coe_sInf (S : Set (InverseLimit D P)) :
    ((sInf S : InverseLimit D P) : ∀ n, D n) = inverseLimitSInfCoe D P S := rfl

theorem isGLB_sInf' (S : Set (InverseLimit D P)) :
    @IsGLB (InverseLimit D P)
      (inverseLimitPartialOrder D P).toPreorder.toLE S (sInf S) := by
  constructor
  · intro x hx n
    exact sInf_le (Set.mem_image_of_mem _ hx)
  · intro b hb n
    refine le_sInf ?_
    rintro _ ⟨x, hx, rfl⟩
    exact hb hx n

end InverseLimit

end DomainTheory.ContinuousLattice
```

4

instInfSet.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.inverseLimitPartialOrder
```

```
public import DomainTheory.ContinuousLattice.InverseLimits.inverseLimitSInfCoe
public import DomainTheory.ContinuousLattice.InverseLimits.projection_galoisConnection
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : ℕ → Type u) [∀ n, CompleteLattice (D n)]

variable (P : ∀ n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- `jₙ` preserves arbitrary infima (it is the upper adjoint of `iₙ`). -/
theorem retr_sInf (n : ℕ) (A : Set (D (n + 1))) :
    (P n).retr (sInf A) = sInf ((P n).retr '' A) := by
  rw [(projection_galoisConnection D P n).u_sInf, sInf_image]

/-- The pointwise infimum of compatible sequences is compatible. -/
theorem compatible_sInf (S : Set (InverseLimit D P)) :
    Compatible D P (inverseLimitSInfCoe D P S) := by
  intro n
  rw [inverseLimitSInfCoe, inverseLimitSInfCoe, retr_sInf]
  congr 1
  rw [Set.image_image]
  exact Set.image_congr (by rintro x hx; exact x.2 n)

noncomputable instance instInfSet : InfSet (InverseLimit D P) :=
  ⟨fun S => ⟨inverseLimitSInfCoe D P S, compatible_sInf D P S⟩⟩

theorem coe_sInf (S : Set (InverseLimit D P)) :
    ((sInf S : InverseLimit D P) : ∀ n, D n) = inverseLimitSInfCoe D P S := rfl

theorem isGLB_sInf' (S : Set (InverseLimit D P)) :
    @IsGLB (InverseLimit D P)
      (inverseLimitPartialOrder D P).toPreorder.toLE S (sInf S) := by
  constructor
  · intro x hx n
    exact sInf_le (Set.mem_image_of_mem _ hx)
  · intro b hb n
    refine le_sInf ?_
    rintro _ ⟨x, hx, rfl⟩
    exact hb hx n

end InverseLimit

end DomainTheory.ContinuousLattice
```

5.6 ContinuousLatticeToNeighborhood

Dependencies. Inside ContinuousLatticeToNeighborhood, each box is one Lean file. A box named as another folder in this chapter is that whole folder. A box named as another chapter is that whole chapter. The arrowhead points at the file or folder that imports, and the number is how many direct imports.

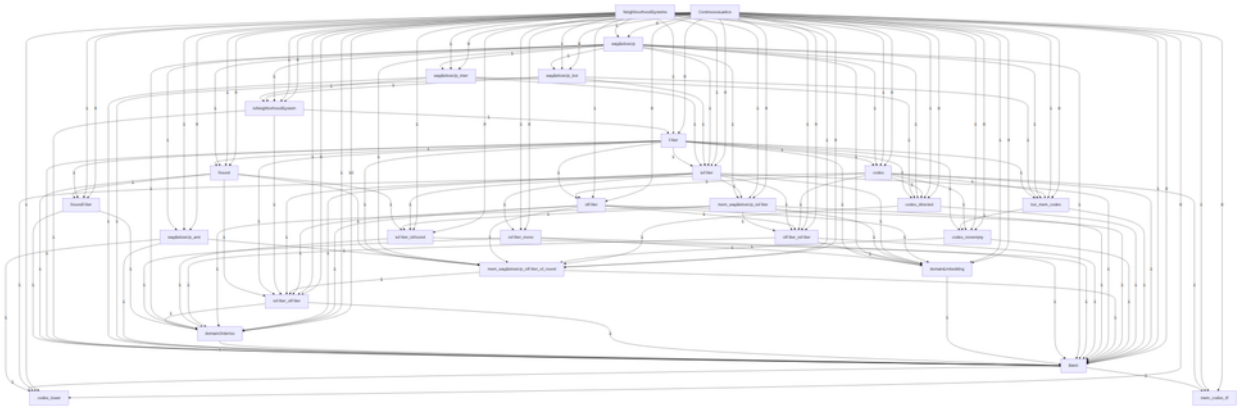


Figure 48: ContinuousLatticeToNeighborhood.

5.7 InfoSysConstructions

Dependencies. Inside InfoSysConstructions, each box is one Lean file. A box named as another folder in this chapter is that whole folder. A box named as another chapter is that whole chapter. The arrowhead points at the file or folder that imports, and the number is how many direct imports.



Figure 49: InfoSysConstructions.

5.8 InfoSysToIdealCompletion

Dependencies. Inside InfoSysToIdealCompletion, each box is one Lean file. A box named as another folder in this chapter is that whole folder. A box named as another chapter is that whole chapter. The arrowhead points at the file or folder that imports, and the number is how many direct imports.

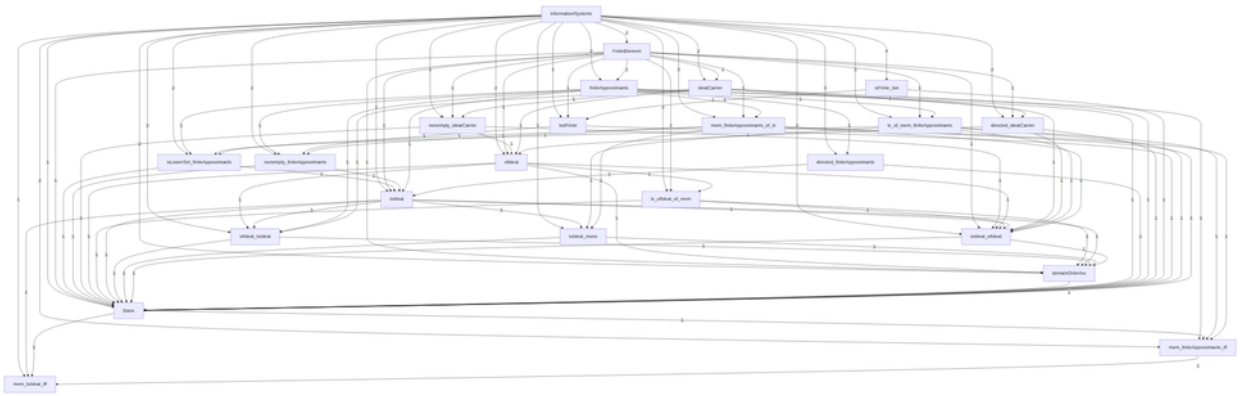


Figure 50: InfoSysToIdealCompletion.

5.9 InfoSysToNeighborhood

Dependencies. Inside InfoSysToNeighborhood, each box is one Lean file. A box named as another folder in this chapter is that whole folder. A box named as another chapter is that whole chapter. BridgeTheorems is the Lean files in that chapter folder, not in a subfolder. The arrowhead points at the file or folder that imports, and the number is how many direct imports.

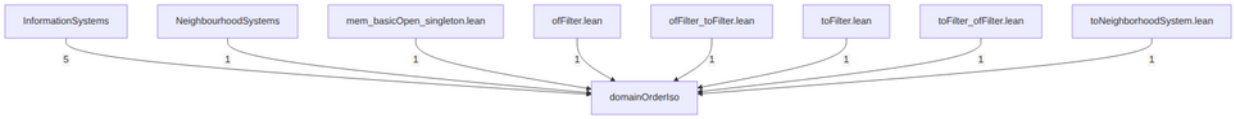


Figure 51: InfoSysToNeighborhood.

5.10 NeighborhoodToInfoSys

Dependencies. Inside NeighborhoodToInfoSys, each box is one Lean file. A box named as another folder in this chapter is that whole folder. A box named as another chapter is that whole chapter. BridgeTheorems is the Lean files in that chapter folder, not in a subfolder. The arrowhead points at the file or folder that imports, and the number is how many direct imports.

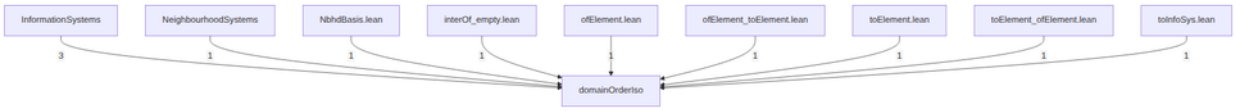


Figure 52: NeighborhoodToInfoSys.

5.11 PresentationDomains

Dependencies. Inside PresentationDomains, each box is one Lean file. A box named as another folder in this chapter is that whole folder. A box named as another chapter is that whole chapter. BridgeTheorems is the Lean files in that chapter folder, not in a subfolder. The arrowhead points at the file or folder that imports, and the number is how many direct imports.

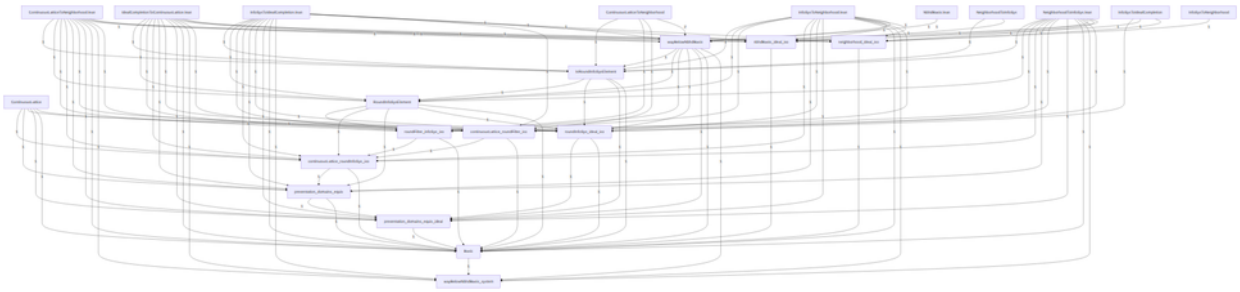


Figure 53: PresentationDomains.

6 Acknowledgments

6.1 AI-assisted development

The human author retains sole responsibility for the mathematical content, the choice of formalization route, and every formal claim in this work. Following standard publisher practice (e.g., COPE guidance on authorship and AI tools [5]), **no large language model is listed as a co-author** — authorship implies an accountability that automated systems cannot bear.

We gratefully acknowledge assistance from the following tools in this revision (auto-generated from `scripts/ai_model_cards.py` when building `arxiv.tex`). Source versions of the four Scott papers and the bridge theorems record their own model cards in their respective writeups under `vendor/`:

- **Cursor Composer 2.5** [6] — routine multi-step work: module scaffolding, dependency-ordered wiring of `DomainTheory/`, documentation and Mermaid blueprints, and medium proof obligations where the strategy was already fixed (`composer-2.5`).
- **Cursor Grok 4.7** [7] — primary agent model for the DomainTheory sessions: domain-theory constructions, inventory design in `arxiv.md`, constructivity audits, and adaptation of portable patterns from prior Scott formalizations. Used here via the Cursor agent environment.
- **Google Gemini 3.8** [8] — exploratory and support passes (typographic conventions, scope decisions, and cross-checks against the narrative inventory).

All definitions, constructivity audits, and final prose were reviewed by the human author, who takes full responsibility for them.

6.2 Artifact availability and reproducibility

The development is at `github.com/catskillsresearch/DomainTheory`. GitHub Actions CI runs `lake build` on `main`.

Locally:

```
lake exe cache get
lake build DomainTheory
```

Sibling packages `scott1972`, `scott1976`, `scott1980`, `scott1982`, and `scott_models` are vendored under `vendor/` (frozen revisions in `vendor/FROZEN.txt`); they are not Lake path dependencies. `arxiv.md` is the durable inventory and proof narrative. Axiom audits: `#print axioms` on the compared names in `comparator.json`.

Build the arXiv PDF / submission zip (Lean Code appendix = GitHub hyperlinks):

```
bash scripts/build_arxiv_tex.sh      # arxiv.md → arxiv.tex + figures/
bash scripts/build_arxiv_pdf.sh      # compile PDF + package dist/arxiv_submit.zip
# or: bash scripts/package_arxiv_submit.sh
```

7 References

References

[1] D. Scott. Continuous lattices, 1972. in F. W. Lawvere (ed.), Toposes, Algebraic Geometry and Logic, Lecture Notes in Mathematics, vol. 274, Springer-Verlag, pp. 97–136. [Also: Tech. Monograph PRG-7, Oxford Univ. Computing Lab., Programming Research Group (1971).].

[2] D. Scott. Lectures on a mathematical theory of computation, 1981. Technical Monograph PRG-19, May 1981, Oxford University Computing Laboratory, Programming Research Group (neighbourhood systems).

[3] D. Scott. Domains for denotational semantics, 1982. ICALP 1982, LNCS 140, Springer, pp. 577–613.

[4] Palomar Registry. entry `PALOMAR-2026-08-25-000003`, version 1, 2026. Verified claim: order isomorphisms relating Scott's three presentations of domains — continuous lattices (1972), neighbourhood systems (PRG-19), and information systems (1982) — including the round identification `D ≃o RoundFilter ≃o RoundInfoSysElement` and the S-expression equation `T ≅ A + (T × T)` as `|T| ≃o |A + (T × T)|` (`SexSys.Element ≃o SexRhs.Element`). https://palomar-registry.org/entry?id=PALOMAR-2026-08-25-000003&version=1.

[5] Committee on Publication Ethics (COPE). Authorship and ai tools: Cope position statement, 2024. 2024. https://publicationethics.org/guidance/cope-position/authorship-and-ai-tools.

[6] Anysphere, Inc. Composer 2.5. Model announcement and documentation, https://cursor.com/blog/composer-2-5; model card as integrated in Cursor, https://cursor.com/docs/models, 2026. accessed 2026.

[7] xAI and Anysphere, Inc. Grok 4.7. Model documentation, https://docs.x.ai/docs/models; Cursor model catalog, https://cursor.com/docs/models, 2026. accessed 2026.

[8] Google DeepMind. Gemini 3.8. Technical documentation and model cards, https://ai.google.dev/gemini-api/docs/models, 2026. accessed 2026.

[9] ed J. C. Abbott. Trends in lattice theory, 1970. Van Nostrand Reinhold Mathematical Studies, vol. 31 (1970).

[10] S. Abramsky and A. Jung. Domain theory, 1994. In Handbook of Logic in Computer Science, Vol. 3, Oxford University Press, 1994.

[11] M. Aiello L. Aiello and R. W. Weyhrauch. The semantics of pascal in lcf, 1974. Artificial Intelligence Lab. Memo. AIM-221, Stanford Univ., Stanford, Calif.

[12] S. Alagić. Categorical theory of tree processing, 1974. Category Theory Applied to Computation and Control, E. Manes, ed., Univ. of Mass., pp. 80–99.

[13] P. Alexandroff and H. Hopf. Topologie i, 1935. Springer-Verlag, (1935).

[14] M. A. Arbib and E. G. Manes. Fuzzy morphisms in automata theory, 1974. 1974a; , Category Theory Applied to Computation and Control, E. Manes, ed., Univ. of Mass., pp. 98–105.

[15] M. A. Arbib and E. G. Manes. Categorists' view of automata and systems, 1974. 1974b; , Category Theory Applied to Computation and Control, E. Manes, ed., Univ. of Mass., pp. 62–79.

[16] M. A. Arbib and E. G. Manes. Basic concepts of category theory applicable to computation and control, 1974. 1974c; , Category Theory Applied to Computation and Control, E. Manes, ed., Univ. of Mass., pp. 2–41.

[17] J. W. de Bakker. Semantics of programming languages, 1969. Advances in Information Systems Science, vol. 2, Plenum Press, New York.

[18] J. W. de Bakker and D. Scott. A theory of programs, 1969. Unpublished notes, IBM Seminar, Vienna, 1969.

[19] J. W. de Bakker. Recursive procedures, 1971. 1971a; , Mathematical Centre Tracts, vol. 24, Amsterdam.

[20] J. W. de Bakker. Recursion, induction and symbol manipulation, 1971. 1971b; , Proc. MC-25 Informatica Symposium, Mathematical Centre Tracts, vol. 38, Amsterdam.

[21] J. W. de Bakker and W. P. de Roever. A calculus for recursive program schemes, 1972. Proc. IRIA Colloq. on Automata, North-Holland, Amsterdam.

[22] J. W. de Bakker and L. G. L. T. Meertens. On the completeness of the inductive assertion method, 1973. Mathematical Centre Rep. IW 12/72, Amsterdam.

[23] J. W. de Bakker and L. G. L. T. Meertens. Fixed points in programming theory, 1974. Foundations of Computer Science, J. W. de Bakker, ed., Mathematical Centre Tract, vol. 63, Amsterdam.

[24] H. P. Barendregt. Some extensional term models for combinatory logics and λ-calculi, 1971. Ph.D. thesis, Univ. of Utrecht.

[25] H. Bekić. Definable operations in general algebra, and the theory of automata and flowcharts, 1969. Rep., IBM Lab., Vienna.

[26] H. Bekić. Towards a mathematical theory of processes, 1971. Tech. Rep. TR 25.125, IBM Lab., Vienna.

[27] H. Bekić et al. A formal definition of a pl/i subset, parts i and ii, 1974. Tech. Rep. TR 25.139, IBM Lab., Vienna.

[28] E. Binz and H. H. Keller. Funktionenräume in der kategorie der limesräume, 1966. Annales Academiae Scientiarum Fennicae, Series A, I. Mathematica, no. 383 (1966), 21 pp.

[29] R. S. Bird. Unsolvability of the inclusion problem for dbs schemas, 1974. Rep. RCS22, Dept. of Computer Sci., Univ. of Reading.

[30] G. Birkhoff. Lattice theory, 1967. 3rd ed., American Mathematical Society Colloquium Publications, vol. 25, Providence, R.I.

[31] A. Blikle. Algorithmically definable functions: A contribution towards the semantics of programming languages, 1971. Dissertationes Math. Rozprawy Mat., 85.

[32] A. Blikle. Equational languages, 1972. Information Control, pp. 134–147.

[33] A. Blikle and A. Mazurkiewicz. An algebraic approach to the theory of programs, algorithms, languages and recursiveness, 1972. Proc. Internat. Sympos. and Summer School on the Mathematical Foundations of Computer Sci., Warsaw-Jabłonna.

[34] A. Blikle. An algebraic approach to programs and their computations, 1973. Proc. Symp. and Summer School on the Mathematical Foundations of Computer Sci., High Tatras, Czechoslovakia.

[35] A. Blikle. Proving programs by sets of computations, 1974. Proc. 3rd Symp. on the Mathematical Foundation of Computer Sci., Jadwisin, Poland; Lecture Notes in Computer Science, Springer-Verlag, Berlin, 1974.

[36] C. Böhm. The cuch as a formal and descriptive language, 1966. Formal Language Description Languages for Computer Programming, T. B. Steel, ed., North-Holland, Amsterdam, pp. 179–197.

[37] C. Böhm. Alcune proprietà delle forme β-η-normali nel λ-κ-calcolo, 1968. Consiglio Nazionale delle Ricerche: Pubblicazione 696 dell' Istituto per le Applicazioni del Calcolo, Roma.

[38] C. Böhm. λ-calculus and computer science theory, 1975. Proc. Rome Symp., March 1975, Lecture Notes in Computer Science, vol. 37, Springer-Verlag, Berlin.

[39] R. M. Burstall. Proving properties of programs by structural induction, 1969. Comput. J., 12, pp. 41–48.

[40] R. M. Burstall. Some techniques for proving the correctness of programs which alter data structures, 1972. 1972a; , Machine Intelligence vol. 7, B. Meltzer and D. Michie, eds., Edinburgh, pp. 23–50.

[41] R. M. Burstall. An algebraic description of programs with assertions, verification and simulation, 1972. 1972b; , Proc. ACM Conf. on Proving Assertions about Programs, Las Cruces, New Mexico, pp. 7–14.

[42] R. Burstall and J. W. Thatcher. The algebraic theory of recursive program schemes, 1974. Category Theory Applied to Computation and Control, E. Manes, ed., Univ. of Mass., pp. 154–160.

[43] J. M. Cadiou. Recursive definitions of partial functions and their computations, 1973. Ph.D. thesis, Computer Sci. Dept., Stanford Univ., Stanford, Calif.

[44] J. M. Cadiou and J. J. Levy. Mechanizable proofs about parallel processes, 1973. Presented at the 14th Ann. Symp. on Switching and Automata Theory.

[45] J. W. Case. Enumeration reducibility and the partial degrees, 1971. Ann. Math. Logic, 2, pp. 419–440.

[46] E. Čech. Topological spaces, 1966. (revised by Z. Frolík and M. Katětov), Prague.

[47] A. K. Chandra and Z. Manna. On the power of programming features, 1973. Artificial Intelligence Memo. AIM-185, Stanford Univ., Stanford, Calif.

[48] A. K. Chandra. Degrees of translatability and canonical forms in program schemas part i, 1974. IBM Res. Rep. RC4733, Yorktown Heights, N.Y.

[49] A. K. Chandra. The power of parallelism and nondeterminism in programming, 1974. 1974a; , IBM Res. Rep. RC4776, Yorktown Heights, N.Y.

[50] A. K. Chandra. Generalized program schemas, 1974. 1974b; , IBM Res. Rep. RC4827, Yorktown Heights, N.Y.

[51] A. Church. The calculi of λ-conversion, 1951. Annals of Mathematical Studies, vol. 6, Princeton University Press, Princeton, N.J.

[52] M. J. Clint. Program proving: Co-routines, 1972. Acta Informatica, 2, pp. 50–63.

[53] R. C. Constable and D. Gries. On classes of program schemata, 1972. this Journal, 1, pp. 66–118.

[54] D. C. Cooper. The equivalence of certain computations, 1966. Comput. J., 9, pp. 45–52.

[55] B. Courcelle and J. Vuillemin. Complétude d'un système formel pour des équations récursives simples, 1974. Compte-Rendu du Colloque de Paris.

[56] B. Courcelle and J. Vuillemin. Semantics and axiomatics of a simple recursive language, 1974. Rapport Laboria, vol. 60, IRIA; Also: 6th Ann. ACM Symp. on the Theory of Computing, Seattle, Washington.

[57] G. Kahn B. Courcelle and J. Vuillemin. Algorithmes d'équivalence pour des équations récursives simples, 1974. Proc. 2nd Colloq. on Automata Languages and Programming, Saarbrücken; Lecture Notes in Computer Science, Springer-Verlag, Berlin, pp. 213–220.

[58] H. B. Curry and R. Feys. Combinatory logic, 1958. vol. 1, North-Holland, Amsterdam.

[59] J. R. Hindley H. B. Curry and J. Seldin. Combinatory logic, 1971. vol. 2, North-Holland, Amsterdam.

[60] M. Davis. Computability and unsolvability, 1958. McGraw-Hill, New York.

[61] E. W. Dijkstra. Notes on structured programming, 1972. Structural Programming, C. A. R. Hoare, E. W. Dijkstra and O. J. Dahl, Academic Press, New York.

[62] E. W. Dijkstra. A simple axiomatic basis for programming language constructs, 1974. Indag. Math., 36, pp. 1–15.

[63] E. W. Dijkstra. A discipline of programming, 1976. Prentice-Hall, Englewood Cliffs, N.J., to appear.

[64] J. Donahue. Scottery child's guide no. 1, 1974. Dept. of Comput. Sci., Univ. of Toronto.

[65] B. N. Duong. A high level, variable free calculus for recursive programming, 1974. Comput. Sci. Dept. Rep. CS 74 03, Univ. of Waterloo.

[66] S. Eilenberg and C. C. Elgot. Recursiveness, 1970. Academic Press, New York.

[67] S. Eilenberg and C. C. Elgot. Automata, languages and machines, 1974. vol. A, Academic Press, New York (vol. B in press).

[68] A. Robinson C. C. Elgot and J. D. Rutledge. Multiple control computer models, 1966. IBM Res. Rep. RC 1622, Yorktown Heights, N.Y.

[69] C. C. Elgot. Abstract algorithms and diagram closure, 1967. IBM Res. Rep. RC 1750, Yorktown Heights, N.Y.

[70] C. C. Elgot. Algebraic theories and program schemes, 1971. Symposium on Semantics of Algorithmic Languages, E. Engeler, ed., Lecture Notes in Mathematics, vol. 188, Springer-Verlag, New York.

[71] M. H. van Emden and R. A. Kowalski. The semantics of predicate logic as a programming language, 1974. Memo. no. 83 (MIP-R-103), Dept of Machine Intelligence, Univ. of Edinburgh.

[72] E. Engeler. Algorithmic properties of structures, 1967. Mathematical Systems Theory, 1, pp. 183–195.

[73] E. Engeler. Formal languages: Automata and structures, 1968. Lectures in Advanced Mathematics, Markham.

[74] E. Engeler. Algorithmic logic, 1974. Foundations of Computer Science, J. W. de Bakker, ed., Mathematical Centre Tract MCT 63, Amsterdam.

[75] J. Engelfriet. Simple program schemes and formal languages, 1974. Springer Lecture Notes in Computer Science, vol. 20.

[76] J. Engelfriet and E. M. Schmidt. Io and oi, 1975. DAIMI PB-47, Matematisk Institut, Aarhus Universitet, Datalogisk Afdeling, Denmark.

[77] Ju. L. Ershov. Computable numerations of morphisms, 1971. Algebra and Logic, 10, pp. 247–308.

[78] Ju. L. Ershov. Computable functions of finite types, 1972. 1972a; Algebra and Logic, 11, pp. 367–437.

[79] Ju. L. Ershov. Everywhere defined (total) continuous functionals, 1972. 1972b; Algebra and Logic, 11, pp. 656–665.

[80] Ju. L. Ershov. Continuous lattices and a-spaces, 1972. 1972d; , Dokl. Akad. Nauk USSR, 207, pp. 523–526.

[81] Ju. L. Ershov. Theory of a-spaces, 1972. 1972e; , Algebra and Logic, 12, pp. 369–416.

[82] Ju. L. Ershov. Theorie der numerierungen, 1973. Veb Deutscher Verlag der Wissenschaften, Berlin.

[83] M. J. Fischer. λ-calculus schemata, 1972. Proc. ACM Conf. Proving Assertions about Programs, Las Cruces, N.M., pp. 104–109.

[84] A. E. Fischer and M. J. Fischer. Mode modules as representations of domains, 1973. Proc. ACM Symp. on Principles of Programming Languages, Boston, pp. 139–143.

[85] R. W. Floyd. Assigning meanings to programs, 1967. Proc. Symp. in Appl. Math., vol. 19, Mathematical Aspects of Computer Science, J. T. Schwartz, ed., pp. 33–41.

[86] M. M. Fokkinga. Inductive assertion patterns for recursive procedures, 1974. Compte-Rendu de Colloque de Paris.

[87] R. M. Friedberg and H. R. Rogers. Reducibility and completeness for sets of integers, 1959. Z. Math. Logik Grundlagen Math., 5, pp. 117–125.

[88] H. Friedman. Algorithmic procedures, generalized turing algorithms, and elementary recursion theory, 1971. Logic Colloquium 1969, North-Holland, Amsterdam, pp. 361–389.

[89] S. J. Garland and D. C. Luckham. Translating recursion schemes into program schemes, 1972. Proc. ACM Conf. on Proving Assertions about Programs, Las Cruces, N.M., pp. 83–96.

[90] H. Gericke. Lattice theory, 1966. Harrap.

[91] G. Gierz et al. Continuous lattices and domains, 2003. Cambridge University Press, 2003.

[92] J. A. Goguen. l-fuzzy sets, 1967. J. Math. Anal. Appl., 18, pp. 145–174.

[93] J. A. Goguen. Categories of l-sets, 1969. Bull. Amer. Math. Soc., 75, pp. 524–622.

[94] J. A. Goguen. On homomorphisms, simulations, correctness, subroutines and termination for programs and program schemes, 1972. Proc. 13th IEEE Conf. on Switching and Automata Theory, pp. 52–60.

[95] J. A. Goguen. Semantics of computation, 1974. Category Theory Applied to Computation and Control, E. Manes, ed., Univ. of Mass., pp. 234–249.

[96] J. A. Goguen et al. Initial algebra semantics, 1975. IBM Res. Rep. RC 5243, Yorktown Heights, N.Y.

[97] M. J. C. Gordon. Models of pure lisp, 1973. 1973a; , Experimental Programming Rep. 31, School of Artificial Intelligence, Univ. of Edinburgh. (Also Ph.D. thesis, Evaluation and denotation of pure LISP programs: A worked example in semantics.).

[98] M. J. C. Gordon. An extended abstract of "models of pure lisp", 1973. 1973b; , Memo. SAI-RM-7, School of Artificial Intelligence, Univ. of Edinburgh.

[99] M. J. C. Gordon. Operational reasoning and denotational semantics, 1975. 1975a; , Artificial Intelligence Memo. 264, Stanford Univ., Stanford, Calif.

[100] M. J. C. Gordon. Towards a semantic theory of dynamic binding, 1975. 1975b; , Artificial Intelligence Memo. 265, Stanford Univ., Stanford, Calif.

[101] G. Grätzer. Universal algebra, 1968. Van Nostrand, New York.

[102] I. Guessarian. Sur une réduction des schémas de programmes polyadiques à des schémas monadiques et ses applications, 1974. Compte-Rendu de Colloque de Paris.

[103] L. Gutteridge. Some results on enumeration reducibility, 1971. Ph.D. dissertation, Simon Fraser Univ., Burnaby, B.C., Canada.

[104] B. Lercher J. R. Hindley and J. P. Seldin. Introduction to combinatory logic, 1972. London Mathematical Society Lecture Note Series, vol. 7, Cambridge University Press, Cambridge, England.

[105] J. R. Hindley and G. Mitschke. Some remarks about the connections between combinatory logic and axiomatic recursion theory, 1975. Preprint 203, Fachbereich Mathematik, Technische Hochschule Darmstadt.

[106] P. Hitchcock and D. M. R. Park. Induction rules and proofs of termination, 1972. Proc. Colloques IRIA, Theorie des Automates des Languages et de la Programmation.

[107] P. Hitchcock. An approach to formal reasoning about programs, 1974. Ph.D. thesis, Univ. of Warwick, England.

[108] C. A. R. Hoare. An axiomatic basis of computer programming, 1969. Comm. ACM, 12, pp. 576–580, 583.

[109] C. A. R. Hoare. Proof of a program: Find, 1971. 1971a; , Comm. ACM, 14, pp. 39–45.

[110] C. A. R. Hoare. Procedures and parameters: An axiomatic approach, 1971. 1971b; , Symposium on Semantics of Programming Languages, E. Engeler, ed., Lecture Notes in Mathematics, vol. 188, Springer-Verlag, New York, pp. 102–116.

[111] C. A. R. Hoare. Notes on data structuring, 1972. Structured Programming, C. A. R. Hoare, E. W. Dijkstra and O. J. Dahl, Academic Press, New York.

[112] C. A. R. Hoare and N. Wirth. An axiomatic definition of the programming language pascal, 1972. Bericht der Fachgruppe Computer-Wissenschaften 6, Eidgenössische Technische Hochschule, Zürich.

[113] C. A. R. Hoare and P. Lauer. Consistent and complementary formal theories of the semantics of programming languages, 1974. Acta Informatica, 3, pp. 135–153.

[114] C. Hosono and M. Sato. A solution to scott's problem: "do the retracts in $p\omega$ form a continuous lattice?", 1975. Research Inst. for Mathematical Sci., Kyoto (preprint).

[115] G. Hotz. Eindeutigkeit und mehrdeutigkeit formaler sprachen, 1966. Electron. Informationsverarbeit. Kybernetik (Berlin), 2, pp. 235–246.

[116] J. M. E. Hyland. Recursion theory on the countable functionals, 1975. 1975a; , D. Phil. thesis, Oxford Univ., Oxford, England.

[117] J. M. E. Hyland. A survey of some useful partial order relations in terms of the lambda calculus, 1975. 1975b; , Proc. Conf. on λ-Calculus and Computer Science Theory, Rome, pp. 83–95.

[118] J. M. E. Hyland. A syntactic characterization of the equality in some models for the lambda calculus. to appear; , J. London Math. Soc.

[119] Y. I. Ianov. The logical scheme of algorithms, 1970. Problems in Cybernetics, vol. 1, Pergamon Press, New York.

[120] S. Igarashi. Admissibility of fixed point induction in first order logic of typed theories, 1972. Artificial Intelligence Memo. AIM-168, Computer Sci. Dept., Stanford Univ., Stanford, Calif.

[121] K. Indermark. Gleichungsdefinierbarkeit in relationalstrukturen, 1974. Habilitationsschrift, Mathematisch-Naturwissenschaftlichen Fakultät der Rheinischen Freidrich-Wilhelms-Universatat, Bonn, W. Germany.

[122] C. B. Jones. Mathematical semantics of goto: Exit formulation and its relation to continuations, 1974. preprint.

[123] G. Kahn. A preliminary theory of parallel programs, 1973. Rapport Laboria IRIA.

[124] D. M. Kaplan. Regular expressions and the equivalence of programs, 1969. J. Comput. System Sci., 3, pp. 361–385.

[125] R. M. Karp and R. E. Miller. Parallel program schemata, 1969. J. Comput. System Sci., 3, pp. 147–195.

[126] J. L. Kelley. General topology, 1955. Van Nostrand, (1955).

[127] D. J. Kfoury. Comparing algebraic structures up to algorithmic equivalence, 1972. Automata, Languages and Programming, M. Nivat, ed., North-Holland, Amsterdam, pp. 253–263.

[128] D. J. Kfoury. Translatability of schemas over restricted interpretations, 1974. J. Comput. System Sci., 8, pp. 387–408.

[129] S. C. Kleene. Introduction to metamathematics, 1950. Van Nostrand, New York.

[130] B. Knaster. Un théorème sur les fonctions d'ensembles, 1928. Ann. Soc. Polon. de Math., 8, pp. 133–134.

[131] P. J. Landin. The mechanical evaluation of expressions, 1964. Comput. J., 6, pp. 308–320.

[132] P. J. Landin. A correspondence between algol 60 and church's lambda notation, 1965. Comm. ACM, 8, pp. 89–101.

[133] P. J. Landin. A λ-calculus approach, 1966. 1966a; , Advances in Programming and Non-numerical Computation, L. Fox, ed., Pergamon Press, New York, pp. 97–141.

[134] P. J. Landin. A formal description of algol 60, 1966. 1966a; , Formal Language Description Languages for Computer Programming, T. B. Steel, ed., North-Holland, Amsterdam, pp. 266–294.

[135] P. J. Landin. The next 700 programming languages, 1966. 1966b; , Comm. ACM, 9, pp. 157–164.

[136] P. J. Landin. A program/machine symmetric automata theory, 1969. Machine Intelligence, vol. 5, B. Meltzer and D. Michie, eds., Edinburgh University Press, pp. 99–120.

[137] P. J. Landin and R. M. Burstall. Programs and their proofs: An algebraic approach, 1969. Machine Intelligence, vol. 4, American Elsevier, New York, pp. 17–44.

[138] J. Leszczykowski. A theorem on resolving equations in the space of languages, 1971. Bull. Acad. Polon. Sci. Sér. Sci. Math. Astronom. Phys., 19, pp. 967–970.

[139] C. H. Lewis and B. K. Rosen. Recursively defined data types, parts i and ii, 1973. 1973–74; IBM Res. Reps. RC 4429 (1973), RC 4713 (1974), Yorktown Heights, N.Y.

[140] B. Liskov and S. Zilles. An approach to abstraction, 1973. Group Memo. 88, Project MAC, Mass. Inst. of Tech., Cambridge, Mass.

[141] D. M. R. Park D. Luckham and M. Paterson. On formalized computer programs, 1970. J. Comput. System Sci., 4, pp. 220–249.

[142] S. Mac Lane. Categories for the working mathematician, 1972. Springer-Verlag, New York.

[143] E. Manes. Category theory applied to computation and control, 1974. Proc. 1st Internat. Symp. Math. Dept. and the Dept. of Computer and Information Sci., Univ. Mass., Amherst. Also: Lecture Notes in Computer Science, vol. 26, Springer-Verlag, New York.

[144] Z. Manna. The correctness of programs, 1969. J. Comput. System Sci., 3, pp. 119–127.

[145] Z. Manna and A. Pnueli. Formalization of properties of functional programs, 1970. J. Assoc. Comput. Mach., 17, pp. 555–569.

[146] Z. Manna and J. McCarthy. Properties of programs and partial function logic, 1970. Machine Intelligence 5, B. Meltzer and D. Michie, eds., Edinburgh University Press, pp. 27–38.

[147] Z. Manna and A. Pnueli. Axiomatic approach to total correctness of programs, 1972. Artificial Intelligence Lab. Memo. AIM-210, Stanford Univ., Stanford, Calif.

[148] Z. Manna and J. M. Cadiou. Recursive definitions of partial functions and their computations, 1972. Proc. ACM Conf. on Proving Assertions about Programs, Las Cruces, N.M., pp. 58–65.

[149] Z. Manna and J. Vuillemin. Fixpoint approach to the theory of computation, 1972. Comm. ACM, 15, pp. 528–536.

[150] Z. Ness Z. Manna and J. Vuillemin. Inductive methods for proving properties of programs, 1972. Proc. ACM Conf. on Proving Assertions about Programs, Las Cruces, N.M., pp. 27–50.

[151] Z. Manna. Mathematical theory of computation, 1974. McGraw-Hill, New York.

[152] G. Markowsky. Categories of chain-complete posets, 1974. RC 5100, Comput. Sci. Dept. IBM T. J. Watson Research Center, Yorktown Heights, N.Y.

[153] G. Markowsky. Chain-complete posets and directed sets with applications, 1974. RC 5024, IBM T. J. Watson Research Center, Yorktown Heights, N. Y.

[154] A. Mazurkiewicz. Proving algorithms by tail functions, 1971. Working Paper for IFIP WG2.2, Feb. 1970. Since published in: Information and Control, 18 (1971), pp. 220–226.

[155] A. Mazurkiewicz. Proving properties of processes, 1973. PRACE CO PAN-CC PAS Reports, vol. 134, Warsaw.

[156] J. McCarthy. The lisp 1.5 programmers' manual, 1962. MIT Press, Cambridge, Mass.

[157] J. McCarthy. A basis for a mathematical theory of computation, 1963. 1963a; , Computer Programming and Formal Systems, D. Braffort and D. Hirshberg, eds., North-Holland, Amsterdam, pp. 33–70.

[158] J. McCarthy. Towards a mathematical science of computation, 1963. 1963b; , Information Processing 1962, Proc. IFIP Congress 1962, C. M. Poppleworth, ed., North-Holland, Amsterdam, pp. 21–28.

[159] J. McCarthy. A formal description of a subset of algol, 1966. Formal Language Description Languages for Computer Programming, R. B. Steel, ed., North-Holland, Amsterdam, pp. 1–12.

[160] J. McCarthy and J. Painter. Correctness of a computer for arithmetic expressions, 1967. Mathematical Aspects of Computer Science, J. T. Schwartz, ed., Proc. of a Symposium in Applied Mathematics, vol. 19, pp. 33–41.

[161] E. Michael. Topologies on spaces of subsets, 1951. Transactions of the American Mathematical Society, vol. 77 (1951), pp. 152-182.

[162] R. E. Milne. The formal semantics of computer languages and their implementations, 1974. Ph.D. thesis, Cambridge Univ., Cambridge, England. [Also: Technical Monograph PRG-13, Oxford Univ. Computing Lab., Programming Research Group].

[163] R. Milner. Program schemes and recursive function theory, 1969. Machine Intelligence 5, B. Meltzer and D. Michie, eds., Edinburgh University Press, pp. 39–58.

[164] R. Milner. Algebraic theory of computable polyadic functions, 1970. 1970a; , Computer Sci. Memo., vol. 12, University College, Swansea.

[165] R. Milner. Equivalences on program schemes, 1970. 1970b; , J. Comput. System Sci., 4, pp. 205–219.

[166] R. Milner. Implementation and application of scott's logic for computable functions, 1972. Proc. ACM Conf., Las Cruces, N.M., pp. 1–6.

[167] R. Milner and R. Weyhrauch. Proving compiler correctness in a mechanized logic, 1972. Machine Intelligence, 7, pp. 51–72.

[168] R. Milner. Models of lcf, 1973. Artificial Intelligence Memo. AIM-186, Computer Sci. Dept., Stanford Univ., Stanford, Calif.

[169] R. Milner. An approach to the semantics of parallel programs, 1973. 1973a; , Proc. Convegno di Informatica Teorica, Instituto di Elaborazione delle Informazione, Pisa, Italy.

[170] R. Milner. Processes: A mathematical model of computing agents, 1973. 1973b; , Proc. Colloq in Mathematical Logic, Bristol, England.

[171] J. H. Morris. λ-calculus models of programming languages, 1968. Ph.D. thesis, Sloan School of Management, MIT MAC Reprint TR-57, Mass. Inst. of Tech., Cambridge, Mass.

[172] F. Lockwood Morris. The next 700 programming language descriptions, 1970. Computer Center, Univ. of Essex (typescript).

[173] J. H. Morris. Another recursion induction principle, 1971. Comm. ACM, 14, pp. 351–354.

[174] F. Lockwood Morris. Correctness of translations of programming languages, 1972. Computer Sci. Memo. CS 72-303, Stanford Univ., Stanford, Calif.

[175] J. H. Morris. A correctness proof using recursively defined functions, 1972. Formal Semantics of Programming Languages, R. Rustin, ed., Prentice-Hall, Englewood Cliffs, N.J., pp. 107–224.

[176] F. Lockwood Morris. Advice on structuring compilers and proving them correct, 1973. Proc. SIGACT/SIGPLAN Symp. on Principles of Programming Languages, Boston, pp. 144–152.

[177] J. H. Morris. Types are not sets, 1973. Proc. ACM Symp. on Principles of Programming Languages, Boston, pp. 120–124.

[178] J. Morris and R. Nakajima. Mechanical characterisation of the partial order in lattice models of λ-calculus, 1973. Tech. Rep. 18, Univ. of Calif., Berkeley.

[179] P. D. Mosses. The mathematical semantics of algol 60, 1974. Tech. Monograph PRG-12, Oxford Univ. Computing Lab., Programming Research Group.

[180] P. D. Mosses. The semantics of semantic equations, 1975. 1975a; , Mathematical Foundations of Computer Science, Goos and Hartmanis, eds., Lecture Notes in Computer Science, vol. 28, Springer-Verlag, New York, pp. 409–422.

[181] P. D. Mosses. Mathematical semantics and compiler generation, 1975. 1975b; , Thesis, Oxford Univ., Oxford, England.

[182] J. Myhill and J. C. Shepherdson. Effective operations on partial recursive functions, 1955. Z. Math. Logik Grundlagen Math., 1, pp. 310–317.

[183] R. Nakajima. Infinite normal forms for λ-calculus, 1975. λ-Calculus and Computer Science Theory, C. Böhm, ed., Springer-Verlag, Berlin, pp. 62–82.

[184] A. Nerode. Some stone spaces and recursion theory, 1959. Duke Mathematical Journal, vol. 26, pp. 397–406.

[185] M. Nivat. Langages algébriques sur le magma libre et sémantique des schémas de programmes, 1972. Proc. IRIA Symposium on Automata Formal Languages and Programming, North-Holland, Amsterdam.

[186] D. C. Oppen. On logic and program verification, 1975. Tech. Rep. 82, Dept. Computer Sci., Univ. of Toronto.

[187] D. M. R. Park. Fixpoint induction and proofs of program properties, 1970. 1970a; , Machine Intelligence, 5, B. Meltzer and D. Michie, eds., American Elsevier, New York, pp. 59–78.

[188] D. M. R. Park. Notes on a formalism for reasoning about schemes, 1970. 1970b; , Univ. of Warwick, unpublished notes.

[189] D. M. R. Park. The y combinator scott's lambda-calculus models, 1970. 1970c; , Univ. of Warwick, unpublished notes.

[190] D. M. R. Park. Finiteness is mu-ineffable, 1970. 1970d; , Theory of Computation Report, No. 3, Dept. of Computer Sci., Univ. of Warwick (1974).

[191] M. S. Paterson. Program schemata, 1963. Machine Intelligence, 3, D. Michie, ed., American Elsevier, New York, pp. 19–32.

[192] M. S. Paterson and C. S. Hewitt. Comparative schematology, 1970. Record of Project MAC Conf. on Concurrent Systems and Parallel Computation, Association for Computing Machinery, New York, pp. 119ff.

[193] R. Platek. New foundations for recursion theory, 1964. Thesis, Stanford Univ., Stanford, Calif., unpublished.

[194] G. D. Plotkin. A set-theoretical definition of application, 1972. Memo. MIP-R-95, School of Artificial Intelligence, University of Edinburgh.

[195] G. D. Plotkin. The λ-calculus is ω-incomplete, 1973. 1973a; , Res. Memo. SAI-RM-2, School of Artificial Intelligence, Univ. of Edinburgh.

[196] G. D. Plotkin. Lambda-definability and logical relations, 1973. 1973b; , Memo. SAI-RM-4, School of Artificial Intelligence, Univ. of Edinburgh.

[197] G. D. Plotkin. Call-by-name, call-by-value and the λ-calculus, 1973. 1973c; , J. Res. Memo. SAI-RM-6, School of Artificial Intelligence, Univ. of Edinburgh.

[198] G. D. Plotkin. A powerdomain construction, 1975. Dept. of Artificial Intelligence Res. Rep. 3, Univ. of Edinburgh.

[199] H. Rasiowa. On ω-1973.-valued algorithmic logic and related problems*, Supplement to Proc. Symp. and Summer School on Mathematical Foundations of Computer Sci., High Tatras, Czechoslovakia.

[200] P. Raulefs. The overtyped lambda calculus, 1975. 1975a; , Interner Bericht Nr. 2, Institut für Informatik, Universität Karlsruhe.

[201] P. Raulefs. Standard models of the overtyped lambda-calculus, 1975. 1975b; , Interner Bericht Nr. 3, Institut für Informatik, Universität Karlsruhe.

[202] R. R. Redziejowski. The theory of general events and its application to parallel programming, 1972. T.P. 18.220 IBM Nordic Laboratory, Sweden.

[203] J. C. Reynolds. Gedanken: A simple typeless language, 1969. Reprint 7621, Argonne National Laboratory, Argonne, Ill.

[204] J. C. Reynolds. Definitional interpreters for higher order programming languages, 1972. 1972a; , Proc. ACM 25th Nat. Conf., Boston, vol. 2, pp. 717–740.

[205] J. C. Reynolds. Notes on a lattice-theoretic approach to the theory of computation, 1972. 1972b; , Systems and Information Sci. Dept., Syracuse Univ., Syracuse, N.Y.

[206] J. C. Reynolds. On the relation between direct and continuation semantics, 1974. 1974a; , Proc. 2nd Colloq. on Automata, Languages and Programming, Saarbrücken, Lecture Notes in Computer Science, vol. 14, Springer-Verlag, Berlin, pp. 141–156.

[207] J. C. Reynolds. Semantics of the lattice of flow diagrams, 1974. 1974b; , Systems and Information Science Dept., Syracuse Univ., Syracuse, N.Y. (preprint).

[208] J. C. Reynolds. On the interpretations of scott's domains. to appear; Symposia Mathematica.

[209] J. C. Reynolds. Towards a theory of type structure. to appear; , Colloq. on Programming, Paris, April 1974, Lecture Notes in Computer Science, Springer-Verlag, Berlin.

[210] W. P. de Roever. Recursion and parameter mechanisms: An axiomatic approach, 1974. 1974a; Proc. 2nd Colloq. on Automata Languages and Programming, Saarbrucken, Lecture Notes in Computer Science, vol. 14, Springer-Verlag, Berlin, pp. 34–65.

[211] W. P. de Roever. Operational, mathematical and axiomatized semantics for recursive procedures and data structures, 1974. 1974b; , Mathematical Centre Report ID/1, Amsterdam.

[212] H. R. Rogers. Theory of recursive functions and effective computability, 1967. McGraw-Hill, New York.

[213] B. K. Rosen. Tree-manipulating systems and church-rosser theorems, 1973. J. Assoc. Comput. Mach., 20.

[214] J. D. Rutledge. Program schemata as automata, part i, 1970. 1970a; IBM Res. RC 3098. J. Comput and System Sci., 7 (1973), pp. 543–578.

[215] J. D. Rutledge. Parallel processes . . . schemata and transformation, 1970. 1970b; , IBM Res. Rep. RC 2912, Yorktown Heights, N.Y.

[216] A. Salwicki. Formalized algorithmic languages, 1970. 1970a; , Bull. Acad. Polon. Sci. Sér. Math. Astronom. Phys., 18, pp. 227–232.

[217] A. Salwicki. On the equivalence of fs-expressions and programs, 1970. 1970b; , Bull. Acad. Polon. Sci. Sér. Math. Astronom. Phys., 18, pp. 275–278.

[218] A. Salwicki. On the predicate calculi with iteration quantifiers, 1970. 1970c; , Bull. Acad. Polon. Sci. Sér. Math. Astronom. Phys., 18, pp. 279–286.

[219] J. G. Sanderson. The λ calculus, lattice theory and reflexive domains, 1973. Mathematical Institute Lecture Notes, Oxford, England.

[220] L. P. Sasso. Degrees of unsolvability of partial functions, 1971. Ph.D. dissertation, Univ. of Calif., Berkeley.

[221] L. P. Sasso. A minimal partial degree, 1973. Proc. Amer. Math. Soc., 38, pp. 388–392.

[222] L. P. Sasso. A survey of partial degrees, 1975. J. Symbolic Logic, 40, pp. 130–140.

[223] D. Scott. A type-theoretical alternative to iswim, cuch, owhy, 1969. Unpublished notes, Oxford, England.

[224] D. Scott. Outline of a mathematical theory of computation, 1970. Proceedings of the Fourth Annual Princeton Conference on Information Sciences and Systems, pp. 169–176. [Also: Tech. Monograph PRG-2, Oxford Univ. Computing Lab., Programming Research Group (1970).].

[225] D. Scott. Lattice theory, data types, and semantics, 1971. New York University Symposia in Areas of Current Interest in Computer Science (Randall Rustin ed.) (1971) to appear.

[226] D. Scott. The lattice of flow diagrams, 1971. Symposium on Semantics of Algorithmic Languages, E. Engeler, ed., Lecture Notes in Mathematics, vol. 188, Springer-Verlag, New York, pp. 311–366. [Also: Tech. Monograph PRG-3, Oxford Univ. Computing Lab., Programming Research Group (1970).].

[227] D. Scott and C. Strachey. Toward a mathematical semantics for computer languages, 1971. Proc. Symposium on Computers and Automata, Polytechnic Inst. of Brooklyn, vol. 21, pp. 19–46. [Also: Technical Monograph PRG-6, Oxford Univ. Computing Lab., Programming Research Group (1971).].

[228] D. Scott. Mathematical concepts in programming language semantics, 1972. 1972b; , AFIPS Conf. Proc., vol. 40, pp. 225–234.

[229] D. Scott. Data types as lattices, 1973. 1973a; , Unpublished lecture notes, Mathematical Centre, Amsterdam, 1973.

[230] D. Scott. Models for various type-free calculi, 1973. 1973b; , Proc. IVth Internat. Cong. for Logic, Methodology and the Philosophy of Science, Bucharest, P. Suppes et al., eds., North-Holland, Amsterdam, pp. 157–187.

[231] D. Scott. Lambda calculus and recursion theory, 1975. 1975a; , Proc. 3rd Scandinavian Logic Symposium, Stig Kanger, ed., North-Holland, Amsterdam, pp. 154–193.

[232] D. Scott. Combinators and classes, 1975. 1975b; , λ-Calculus and computer science theory, Proc. Rome Symp., March 1975; Lecture Notes in Computer Science, vol. 37, Springer-Verlag, Berlin, pp. 1–26.

[233] D. Scott. Some philosophical issues concerning theories of combinators, 1975. 1975c; , λ-Calculus and computer science theory, Proc. Rome Symp., March 1975; Lecture Notes in Computer Science, vol. 37, Springer-Verlag, Berlin, pp. 346–366.

[234] J. C. Shepherdson. Computation over abstract structures: Serial and parallel procedures and friedman's effective definitional schemes, 1975. Logic Colloquium 1973, H. E. Rose and J. C. Shepherdson, eds., North-Holland, Amsterdam, pp. 445–513.

[235] E. Spanier. Quasi-topologies, 1963. Duke Mathematical Journal, vol. 30 (1963) pp. 1-14.

[236] C. Strachey. Towards a formal semantics, 1966. Formal Language Description Languages for Computer Programming, T. B. Steel, ed., North-Holland, Amsterdam, pp. 198–220.

[237] C. Strachey. Fundamental concepts in programming languages, 1967. Unpublished lecture notes for the NATO Summer School, Copenhagen.

[238] C. Strachey. Varieties of programming languages, 1972. Proc. Internat. Computing Symp., Cini Foundation, Venice, pp. 222–233. [Also: Tech. Monograph PRG-10, Oxford Univ. Computing Lab., Programming Research Group (1972).].

[239] C. Strachey and C. Wadsworth. Continuations: A mathematical semantics for handling full jumps, 1974. Tech. Monograph PRG-11, Oxford Univ. Computing Lab., Programming Research Group.

[240] W. Strother. Fixed points, fixed sets, and m-retracts, 1955. Duke Mathematical Journal, vol. 22 (1955), pp. 551-556.

[241] A. Tang. Recursion theory and descriptive set theory in effectively given t_0 spaces, 1974. Ph.D. thesis, Princeton Univ., Princeton, N.J.

[242] A. Tang. Borel sets in $p\omega$, 1975. 1975a; , IRIA-Laboria, preprint.

[243] A. Tang. Notes on subsets on $p\omega$ with extra-finitary property, 1975. 1975b; , IRIA, preprint.

[244] A. Tang. A hierarchy of b_δ sets in $p\omega$, 1975. 1975c; , IRIA, preprint.

[245] A. Tang. Sets of the form $\{x \mid R(x) = T\}$ in $p\omega$, 1975. 1975c; , IRIA, preprint.

[246] A. Tang. $\leqq_e$ degrees in p, iria, preprint, 1975. 1975d.

[247] A. Tarski. On some fundamental concepts of mathematics, 1930. ; translated by J. H. Woodger in Logic, Semantics, Metamathematics, Cambridge, 1956, pp. 30–37.

[248] A. Tarski. A lattice-theoretical fixpoint theorem and its applications, 1955. Pacific J. Math., 5, pp. 285–309.

[249] R. D. Tennent. Mathematical semantics and design of programming languages, 1973. Ph.D. thesis, Univ. of Toronto. [Also: Tech. Rep. 59, Univ. of Toronto (1973).].

[250] R. D. Tennent. The mathematical semantics of programming languages, 1974. 1974a; , Dept. of Computing and Information Sci., Queen's Univ., Kingston, Ontario, Canada (preprint).

[251] R. D. Tennent. A contribution to the development of pascal-like languages, 1974. 1974b; , Tech. Rep. 74-25, Dept. of Computing and Information Sci., Queen's Univ., Kingston, Ontario, Canada.

[252] R. D. Tennent. Pasqual: A proposed generalization of pascal, 1975. Tech. Rep. 75-32, Dept. of Computing and Information Sci., Queen's Univ., Kingston, Ontario, Canada.

[253] J. Vuillemin. Proof techniques for recursive programs, 1973. 1973a; , IRIA—Laboria Rep.

[254] J. Vuillemin. Correct and optimal implementations of recursion in a simple programming language, 1973. 1973b; , IRIA—Laboria Rep. 24.

[255] C. P. Wadsworth. Semantics and pragmatics of the lambda-calculus, 1971. D. Phil. thesis, Oxford Univ., Oxford, England.

[256] C. P. Wadsworth. The relation between lambda-expressions and their denotation, 1975. 1975a; , Dept. of Systems and Information Sci., Syracuse Univ., Syracuse, N.Y. (preprint).

[257] C. P. Wadsworth. Approximate reduction and lambda-calculus models, 1975. 1975b; , Dept. of Systems and Information Sci., Syracuse Univ., Syracuse, N.Y.

[258] C. P. Wadsworth. Some unusual λ-calculus numeral systems, 1975. 1975c; , in preparation.

[259] C. P. Wadsworth. On the topological ordering in the d-model of the lambda-calculus. to appear; , in preparation.

[260] E. G. Wagner. Languages for defining sets in arbitrary algebras, 1971. 1971a; , Proc. of the 11th IEEE Conf. on Switching and Automata Theory, pp. 192–201.

[261] E. G. Wagner. An algebraic theory of recursive definitions and recursive languages, 1971. 1971b; , Proc. of the 3rd ACM Symp. on Theory of Computing.

[262] E. G. Wagner. Notes on categories, algebras and programming languages, 1974. Unpublished lecture notes, London.

[263] M. Wand. Mathematical foundations of formal language theory, 1973. MAC-TR-108, Project MAC, Mass. Inst. of Tech., Cambridge, Mass.

[264] M. Wand. An algebraic formulation of the chomsky hierarchy, 1974. 1974a; , Category Theory Applied to Computation and Control, E. Manes, ed., Univ. of Mass., pp. 216–221.

[265] M. Wand. On the recursive specification of data types, 1974. 1974b; , Category Theory Applied to Computation and Control, E. Manes, ed., Univ. of Mass., pp. 222–225.

[266] M. Wand. Realizing data structures as lattices, 1974. 1974c; , Tech. Rep. 11, Computer Sci. Dept., Indiana Univ.

[267] M. Wand. Fixed-point constructions in order-enriched categories, 1975. Tech. Rep. 23, Computer Sci. Dept., Indiana Univ.

[268] P. Wegner and D. Lehmann. Algebraic and topological prerequisites to scott's theory of computation, 1972. Tech. Rep. 72-2, Dept. of Computer Sci., Hebrew Univ. of Jerusalem.

[269] R. W. Weyrauch and R. Milner. Program correctness in a mechanized logic, 1972. Proc. of the 1st USA-JAPAN Computer Conf., pp. 384–390.

[270] J. B. Wright. Characterization of recursively enumerable sets, 1972. J. Symbolic Logic, 37, pp. 507–511.